Neurális hálózatok és CNN-ek

Matematika, tanítás, alkalmazás és reprodukálható kísérlet programozóknak

Tartalomjegyzék

1 AI-modellek, neurális hálók és tudományos kérdésfeltevés 7
1.1 Modell, tanulás és felhasználási szerződés . . . 7
1.2 Mitől egyetemi szintű egy modellezési állítás? . . . 16
1.3 Adatfelosztás és a megfigyelési egység . . . 22
1.4 Tanulási diagnózis és kísérleti napló . . . 30

2 Programozási és numerikus munkaasztal 48
2.1 Python, NumPy és tenzoralak . . . 48
2.2 Lebegőpont és stabil elemi számítás . . . 55
2.3 Izolált környezet, notebook és verzió . . . 64
2.4 Kis tudományos szoftverprojekt . . . 74

3 Lineáris algebra neurális hálózatokhoz 86
3.1 Vektor, norma és skalárszorzat . . . 86
3.2 Mátrixszorzás és affín réteg . . . 97
3.3 Vetítés, legkisebb négyzetek és kondicionáltság . . . 106
3.4 Sajátérték, SVD és spektrális határ . . . 116

4 Kalkulus, láncszabály és vektor–Jacobi-szorzat 129
4.1 Függvény, határérték és derivált . . . 129
4.2 Parciális derivált és gradiens . . . 138
4.3 Jacobi-mátrix, láncszabály és VJP . . . 149
4.4 I. ellenőrző pont: matematika és kód megfelelése . . . 161

5 Valószínűség és statisztika a modellkísérlet mögött 171
5.1 Esemény, feltétel és véletlen változó . . . 171
5.2 Várható érték, variancia és likelihood . . . 183
5.3 Mintavételi bizonytalanság és intervallum . . . 194
5.4 Függő képek, seedek és resampling . . . 211

6 Empirikus kockázat és optimalizálás 220
6.1 Objektív, reguláris tag és gradienslépés . . . 220
6.2 Konvexitás és sima optimalizálási határ . . . 229
6.3 Sztochasztikus és minibatch gradiens . . . 242
6.4 Momentum és adaptív optimizálók . . . 252

7 Információelmélet, veszteségek és stabil valószínűségi predikció 262
7.1 Entrópia, cross-entropy és KL . . . 262
7.2 Softmax és többosztályos cross-entropy . . . 270
7.3 Bináris, multilabel és regressziós cél . . . 278
7.4 Numerikus stabilitás és jelentésmegőrző implementáció . . . 286

8 Általánosítás és statisztikai tanulási alapok 298
8.1 Kockázat, mintavétel és baseline . . . 298
8.2 Véges hipotézisosztály egyenletes korlátja . . . 308

8.3 Kapacitás, regularizálás és modellválasztás . . . 319
8.4 II. ellenőrző pont: korrekt tanulási állítás . . . 329

9 Perceptron, MLP és reprezentáció 341
9.1 Neuron és perceptron . . . 341
9.2 MLP és nemlinearitás . . . 350
9.3 Expresszivitás és approximációs állítások . . . 360
9.4 Aktivációk és gradiensáramlás . . . 370

10 Backpropagation a könyvtár előtt 381
10.1 Számítási gráf és lokális derivált . . . 381
10.2 Affín réteg és aktiváció backward . . . 389
10.3 Gradiensellenőrzés és numerikus hibamérce . . . 398
10.4 Tanítási lépés és tanuló MLP . . . 407

11 PyTorch: autograd, modul és adatbetöltés 417
11.1 Tensor és autograd-életciklus . . . 417
11.2 nn.Module és paraméterregisztráció . . . 425
11.3 Dataset és DataLoader a split után . . . 434
11.4 Egy teljes PyTorch tanítási iteráció . . . 442

12 Megismételhető tanítási kísérlet 451
12.1 Tanítás, validáció és kiértékelési mód . . . 451
12.2 Checkpoint és folytathatóság . . . 458
12.3 Sanity check és kis mintára túlillesztés . . . 468
12.4 III. ellenőrző pont: önállóan tanuló kis háló . . . 477

13 Diszkrét jelek, képek és konvolúciós műveletek 488
13.1 Kép mint diszkrét többcsatornás jel . . . 488
13.2 Konvolúció és cross-correlation . . . 496
13.3 Lokális szűrők és tanult reprezentáció . . . 507
13.4 Határkezelés és frekvenciaintuíció . . . 519

14 CNN-réteg: alakok, csatornák és számítási költség 528
14.1 Stride, padding, dilation és outputméret . . . 528
14.2 Több bemeneti és kimeneti csatorna . . . 537
14.3 Paraméterszám, MAC és aktivációmemória . . . 546
14.4 Csoportos és depthwise konvolúció . . . 554

15 Konvolúciós backward és transzponált operátor 564
15.1 Súly- és biasgradiens . . . 564
15.2 Inputgradiens és átfedő recepciós mezők . . . 573
15.3 Transposed convolution mint adjungált . . . 582
15.4 Független gradiensellenőrzési protokoll . . . 593

16 Szimmetria, pooling és recepciós mező 604
16.1 Eltolási equivariance és invariance . . . 604
16.2 Pooling, downsampling és aliasing . . . 616
16.3 Elméleti és effektív recepciós mező . . . 626
16.4 IV. ellenőrző pont: a CNN-operátor teljes megértése . . . 634

17 Klasszikus CNN-architektúrák és tervezési döntések 644
17.1 LeNet és a lokális megosztott reprezentáció . . . 644

17.2 AlexNet és a kísérleti kontextus . . . 653
17.3 VGG és kis kernelek kompozíciója . . . 662
17.4 Architektúra összehasonlítása kontrollált keretben . . . 672

18 Reziduális tanulás és normalizálás 681
18.1 Reziduális blokk és identitásút . . . 681
18.2 Batch normalization train és eval módban . . . 691
18.3 Pre-activation, kis batch és alternatív normalizálás . . . 701
18.4 Reziduális blokk ablatív vizsgálata . . . 711

19 Hatékony és korszerű CNN-blokkok 722
19.1 Depthwise separable és pointwise réteg . . . 722
19.2 MobileNet és inverted residual . . . 731
19.3 EfficientNet és architektúraskálázás . . . 740
19.4 ConvNeXt és a recept/architektúra elválasztása . . . 748

20 Transzfertanulás és előtanított modellek helyes használata 756
20.1 Feature extractor és új fej . . . 756
20.2 Finomhangolás és tanulási ráta . . . 766
20.3 Preprocessing, domain shift és súlylicenc . . . 775
20.4 V. ellenőrző pont: indokolt CNN-választás . . . 783

21 Tanítási recept: inicializálás, regularizálás és ütemezés 792
21.1 Inicializálás és varianciaterjedés . . . 792
21.2 Learning rate és ütemezés . . . 803
21.3 Dropout, súlyregularizálás és augmentáció . . . 813
21.4 Osztályarány, súlyozás és céleloszlás . . . 823

22 Képadat, annotáció és augmentációs szerződés 837
22.1 Adateredet, címke és csoportos split . . . 837
22.2 Címkemegőrző transzformációk . . . 847
22.3 Zajos és hiányos címke . . . 858
22.4 Augmentációs kísérlet és domainváltozás . . . 868

23 Metrikák, kalibráció és független modellértékelés 880
23.1 Confusion matrix és aggregálási konvenció . . . 880
23.2 Kalibráció és döntési küszöb . . . 893
23.3 Páros összehasonlítás és csoportszintű intervallum . . . 903
23.4 Hibaszeletek és alkalmazási határ . . . 913

24 Hibakeresés, profiling és számítási erőforrás 922
24.1 Tanulási görbe és hibadiagnózis . . . 922
24.2 CPU/GPU profiling és helyes időmérés . . . 932
24.3 Mixed precision és nagyobb batch . . . 942
24.4 VI. ellenőrző pont: megbízható CNN-training . . . 951

25 Képosztályozás: bináris, többosztályos és multilabel 964
25.1 Teljes képosztályozó adatfolyam . . . 964
25.2 Multilabel és hierarchikus cél . . . 972
25.3 Kis adat és feature-alapú baseline . . . 984
25.4 Nyílt halmaz és ismeretlen input . . . 995

26 Objektumdetektálás és lokalizáció **1006**
26.1 Dobozkoordináta, IoU és lokalizációs loss . . . 1006
26.2 Egy- és kétlépcsős detektor . . . 1015
26.3 NMS, confidence és AP . . . 1025
26.4 Detektálási pipeline és hibaelemzés . . . 1037

27 Szemantikus és példányszintű szegmentálás **1047**
27.1 Pixelszintű klasszifikáció és FCN . . . 1047
27.2 U-Net, encoder–decoder és skip . . . 1056
27.3 Dice, IoU és aggregálás . . . 1064
27.4 Instance mask és összetett feladat . . . 1075

28 Metrikus tanulás, hasonlósági keresés és reprezentációhasználat **1085**
28.1 Embedding, távolság és normalizálás . . . 1085
28.2 Sziámi és triplet cél . . . 1095
28.3 Retrieval-értékelés és adatfelosztás . . . 1103
28.4 VII. ellenőrző pont: CNN alkalmazási szerződés . . . 1113

29 Autoencoder, denoising és reprezentációs korlát **1122**
29.1 Lineáris és nemlineáris autoencoder . . . 1122
29.2 Denoising és korrupciós modell . . . 1129
29.3 Regularizált latent reprezentáció . . . 1139
29.4 Anomáliapontszám rekonstrukcióból . . . 1148

30 Önfelügyelt és kontrasztív reprezentációtanulás **1160**
30.1 Pretext és downstream feladat . . . 1160
30.2 Kontrasztív cél és temperature . . . 1167
30.3 Negatív nélküli módszerek és collapse . . . 1175
30.4 Önfelügyelt kísérlet korrekt összevetése . . . 1183

31 Generatív modellek: VAE, GAN és diffúziós alapgondolat **1192**
31.1 VAE és variációs alsó korlát . . . 1192
31.2 GAN mint minimax probléma . . . 1200
31.3 Diffúziós zajmodell és CNN-denoiser . . . 1211
31.4 Generatív felhasználás és értékelés . . . 1220

32 Szekvenciák, attention és CNN–transformer összevetés **1231**
32.1 Szekvenciamodell és változó hossz . . . 1231
32.2 Scaled dot-product attention . . . 1240
32.3 Vision Transformer és hibrid felhasználás . . . 1250
32.4 VIII. ellenőrző pont: célhoz illő neurális modell . . . 1259

33 Inference, export és modellátadás **1271**
33.1 Inference-szerződés és modellcsomag . . . 1271
33.2 Batch és kis szolgáltatás . . . 1280
33.3 torch.export és ONNX . . . 1288
33.4 Üzemeltetés, drift és késő címke . . . 1295

34 Hatékonyság, kvantálás és tudásdesztilláció **1305**
34.1 Mért késleltetés, throughput és memória . . . 1305
34.2 Kvantálás és hibatovábbterjedés . . . 1314
34.3 Pruning és struktúra . . . 1323
34.4 Teacher–student desztilláció . . . 1332

35 Magyarázhatóság, robusztusság és felelős felhasználás **1343**
35.1 Saliency és Grad-CAM . . . 1343
35.2 Magyarázat sanity check . . . 1350
35.3 Perturbáció és adversarial példa . . . 1361
35.4 Korlátok, eredet és döntési felelősség . . . 1371

36 Kutatási módszertan és reprodukálható eredményközlés **1380**
36.1 Állításból kísérleti terv . . . 1380
36.2 Ismétlések és eredménybizonytalanság . . . 1389
36.3 Szakirodalmi reprodukció és attribúció . . . 1398
36.4 IX. ellenőrző pont: lektorálható modellállítás . . . 1406

37 Projekt I: konvolúciós réteg levezetéstől referenciakódig **1415**
37.1 Operátorspecifikáció és kézi referencia . . . 1415
37.2 Backward és ellenőrzési lefedettség . . . 1424
37.3 Linearitás, adjungált és határfeltétel . . . 1435
37.4 Rétegből kis tanulómodell . . . 1444

38 Projekt II: teljes CNN-osztályozási kísérlet **1457**
38.1 Adat, split és előzetes protokoll . . . 1457
38.2 Modell, training és best checkpoint . . . 1468
38.3 Végső teszt és páros korrupció . . . 1483
38.4 Mentett modellből független inference . . . 1493

39 Projekt III: szegmentálás vagy lokalizáció **1507**
39.1 Feladat- és annotációs döntés . . . 1507
39.2 Encoder–decoder vagy lokalizációs fej . . . 1518
39.3 Geometriai kontroll és metrika . . . 1528
39.4 Alkalmazási riport és átadás . . . 1538

40 Tudományos szintézis, vizsgafeladatok és lektori átadás **1552**
40.1 Összetett levezetési vizsga . . . 1552
40.2 Kísérleti és hibakeresési vizsga . . . 1563
40.3 Tudományos kézirat és forrásaudit . . . 1577
40.4 X. ellenőrző pont: önálló neurális modell és bizonyíték . . . 1585

1 AI-modellek, neurális hálók és tudományos kérdésfeltevés

1.1 Modell, tanulás és felhasználási szerződés

1.1.1 Cél, előismeret és induló kérdés

Ebben az egységben azt tanuljuk meg, hogyan lesz egy alkalmazási ötletből ellenőrizhető modellezési feladat: bemeneti tér, döntési tér, megfigyelt cél és mérhető sikerkritérium. Előismeret nem szükséges a könyvből; a programozói előismeret annyi, hogy a hallgató ciklust, függvényt és tömböt tud írni. Induló becslésként válaszold meg magadban: ha egy levélszemét-szűrő azt a kézzel írt szabályt használja, hogy az „ingyen” szó előfordulása spamet jelent, tanult modellről beszélünk-e? Mi hiányzik ahhoz, hogy a szűrő adatból tanuljon?

A könyv az angol szakzsargon egyes kifejezéseit (például batch, split, seed, baseline, checkpoint, inference) magyar zárójeles magyarázattal használja; a részletes magyar–angol szójegyzék a C. Magyar–angol szójegyzék függelékben található.

1.1.2 AI, ML, DL, CNN: nem szinonimák

Az alábbi meghatározások e könyv munkadefiníciói, tehát *definíciók*, nem bizonyítandó tételek. Az **AI** (artificial intelligence, mesterséges intelligencia) tág munkakategória: olyan rendszer, amely emberi módon leírható, döntés jellegű feladatot lát el. Ennek nincs egységes formális definíciója, ezért a további fogalmakat ettől függetlenül, az adat szerepén keresztül pontosítjuk.

- **ML** (machine learning, gépi tanulás): a rendszer a döntési függvényét nem kézzel rögzített szabályokkal, hanem megfigyelt adatok alapján, algoritmikusan választja meg.
- **DL** (deep learning, mélytanulás): az ML azon családja, ahol a modell több, egymásra épülő tanulható transzformáció kompozíciója, jellemzően nemlineáris aktivációkkal elválasztott rétegekből álló neurális háló. A **mélység** az egymásra épülő tanulható, nemlineáris lépések száma; a mélység számolása választott konvenció (például hogy a pooling vagy az aktivációs réteg számít-e), nem általános definíció. Ettől külön fogalom a **szélesség** (egy réteg egységeinek vagy csatornáinak száma) és a **paraméterszám**: egy sekély, de széles háló sok paraméterű lehet, egy mély, keskeny háló kevés paraméterű.
- **CNN** (convolutional neural network, konvolúciós neurális háló): olyan neurális hálócsalád, amely legalább egy tanulható, lineáris konvolúciós/keresztkorrelációs réteget tartalmaz, azaz egy lokális, térbeli ablakokra alkalmazott, a pozíciók között megosztott paraméterű lineáris operátort. (A pusztán „lokális, súlymegosztott művelet” ennél tágabb: egy lokális attention is ilyen lehet, de nem konvolúció.) A pontos tenzorműveletet, a keresztkorreláció és a matematikai konvolúció különbségét később definiáljuk (13.2. Konvolúció és cross-correlation felé vezető úton); itt a modellcsalád megkülönböztetése a cél. A konvolúciós művelet maga lineáris; a háló nemlinearitását az aktivációs függvények adják. A CNN fogalma nem követel nagy mélységet: egyetlen konvolúciós réteg lineáris kimeneti réteggel is CNN. Mivel ebben a határesetben a két lineáris lépés egyetlen lineáris leképezéssé vonható

össze, ez a háló CNN, de nem DL. A könyv főként több konvolúciós réteget egymásra építő, mély CNN-eket tárgyal.

A viszony tehát: a könyvben tárgyalt mély CNN-ek a DL részhalmazát alkotják (az egyetlen konvolúciós rétegű háló a határeset), a DL az ML részhalmaza, az ML pedig az AI egyik, adatközpontú részhalmaza. A Stanford CS231n kurzus [S01] is ebben az értelemben tárgyalja a konvolúciós hálókat mint a mély modellek egy osztályát; a matematikai háttér (lineáris algebra, valószínűség, optimalizálás) rendszeres tárgyalása [B01]. A CNN klasszikus, dokumentumfelismerő alkalmazását [P01] tárgyalja; a részletes architektúrák és mérési eredmények pontos forráshelyeinek megjelölése a 17.1. LeNet és a lokális megosztott reprezentáció–19.4. ConvNeXt és a recept/architektúra elválasztása egységek feladata.

Ellenpélda a szinonima-kezelésre: egy 1990-es évekbeli szabályalapú spam-szűrő lehet AI-alkalmazás, de nem ML, mert az adat nem befolyásolta a döntési függvényt. Egy legkisebb négyzetes lineáris regresszió ML, de nem DL és nem CNN. Egy CNN alapú képfelismerő mindhárom kategóriába tartozik.

1.1.3 Modell, tanítás és inference mint külön életciklus

A feladat szerződéséhez négy objektumot választunk szét. (A könyvben a *szerződés* szót végig specifikáció értelemben használjuk: a bemenetek, kimenetek, alakok és viselkedési feltételek rögzített leírása.)

Definíció (feladatszerződés). Adott egy $\mathcal{X}$ bemeneti tér és egy $\mathcal{Y}$ döntési tér. A megfigyelés $x_i \in \mathcal{X}$, a *cél* (target) felügyelt esetben $y_i \in \mathcal{Y}$, a *döntés* pedig $\hat{y}_i \in \mathcal{Y}$. A *sikerkritérium* egy olyan $S(\hat{y}, y)$ függvény, amely a döntés és a cél egyezését méri; többosztályos osztályozásnál $y, \hat{y} \in \{0, \dots, K-1\}$, és az egyszerű választás az egyezési arány

$$S(\hat{y}, y) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i],$$

(itt $\hat{y} = (\hat{y}_1, \dots, \hat{y}_N)$ és $y = (y_1, \dots, y_N)$ N elemű vektor: a *skalár* cél egyetlen mintára szól, a *batch* cél és a döntés alakja $(N,)$; a laborban is a döntés N elemű, nem egyetlen osztály.) ahol $\mathbf{1}[\cdot]$ az indikátorfüggvény: igaz esetén 1, hamis esetén 0.

Definíció (modell, hipotézisosztály, tanítás, inference). A *döntési modell* egy paraméterezett leképezés $f_\theta : \mathcal{X} \to \mathcal{Y}$, ahol $\theta \in \Theta$ a paraméter. Többosztályos neurális osztályozásban érdemes különválasztani a hálózat $g_\theta : \mathcal{X} \to \mathbb{R}^K$ logitfüggvényét és a $\delta : \mathbb{R}^K \to \mathcal{Y}$ döntési szabályt (például az argmaxot): $f_\theta = \delta \circ g_\theta$. A *hipotézisosztály* az összes ilyen döntési leképezés halmaza:

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}.$$

A *tanító adat* $D = \{(x_i, y_i)\}_{i=1}^{N}$. A *tanító algoritmus* $\mathcal{A}$ olyan eljárás, amely D alapján egy $\hat{\theta} \in \Theta$ paramétert választ; a *tanítás* tehát az $\hat{\theta} = \mathcal{A}(D)$ folyamat. Az *inference* a már rögzített $f_{\hat{\theta}}$ kiértékelése új x bemeneten; ekkor θ nem változik.

A két életciklus szétválasztása azért fontos, mert a tanítás alatt az algoritmus látja a célokat, az inference alatt pedig a döntést a bemenetből kell előállítani. Egy modell önmagában nem „tud” semmit: a tanító algoritmus választotta ki a hipotézisosztály egy elemét.

Tenzoralak szempontjából a későbbi képosztályozási tanulási útvonal ezt a konvenciót használja: a képbatch $X \in \mathbb{R}^{N \times C \times H \times W}$ (batch, csatorna, magasság, szélesség), az osztályozási cél $y \in \{0, \dots, K-1\}^N$, a döntést pedig egy $Z \in \mathbb{R}^{N \times K}$ pontszámtömbből, a logitból képezzük. A 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték alfejezet 13. szakaszának A. függelékében

leírt GlyphLab alapadata erre konkrét példa: $C = 1$, $H = W = 24$, az adathalmaz pedig 360 képből áll.

1.1.4 Feladattípus: supervised, unsupervised, generatív

A tanulási mód leírja, mi a megfigyelt cél:

- **Supervised** (felügyelt): minden tanító megfigyeléshez adott egy y_i cél, például osztálycímke. A feladat explicit $x_i \mapsto y_i$ kapcsolatot tanulni.
- **Unsupervised** (felügyelet nélküli): előre adott y_i cél nincs; a kísérlet definiál egy adatból származtatott célt, például az x_i rekonstrukcióját, klaszterezési feladatot vagy pretextfeladatot.
- **Generative** (generatív): a modell felhasználási célja új, a tanító eloszlással konzisztens x megfigyelés előállítása. Ez nem szinonimája az unsupervised tanulásnak: egy feltételes generatív modell tanulhat címkékkel, tehát supervised módon.

Ugyanaz a háló lehet unsupervised módon tanított reprezentáció, amelyet később felügyelt osztályozó fejjel használunk; a tanulási mód és a felhasználási cél külön dimenzió. Ezt a megkülönböztetést később a 30.1. Pretext és downstream feladat és a 31.1. VAE és variációs alsó korlát egységekben is használjuk.

1.1.5 Levezetési kötelezettség: adat, hipotézisosztály és algoritmus külön objektum

A következő kis példa nem tétel, hanem három objektum definíciója és számítási kontroll, amely megalapozza a későbbi állítások feltételrendszerét.

Adat. Legyen a kétdimenziós XOR-osztályozási példa

$$D = \{((0,0),0),((1,0),1),((0,1),1),((1,1),0)\},$$

tehát $N = 4$, $x_i \in \mathbb{R}^2$, $y_i \in \{0,1\}$. A $d = 2$ dimenziós vektorokat elvileg oszlopvektorként írjuk; a Python-kódban a batch egy sora az oszlopvektor transzponáltja.

Hipotézisosztály. A lineáris küszöbös osztályozók halmaza

$$\mathcal{H}_{\text{lin}} = \{h_{w,b}(x) = \mathbf{1}[w^\top x + b > 0] : w \in \mathbb{R}^2, b \in \mathbb{R}\}.$$

Mindegyik elem tehát egy félsík, amelyet a w normálvektor és a b eltolás ad meg.

Algoritmus. A $\mathcal{A}_{\text{rács}}$ tanító algoritmus egy véges rácson keresi a legkisebb tanítóhibájú elemet:

$$R_{\text{tr}}(h) = \frac{1}{4}\sum_{i=1}^{4} \mathbf{1}[h(x_i) \neq y_i].$$

A rács: $w_1, w_2 \in \{-1, 0, 1\}$ és $b \in \{-2, -1, 0, 1, 2\}$; holtversenyben a bejárási sorrend szerinti első modellt választjuk. Ez az algoritmus tehát nem azonos a hipotézisosztállyal: $\mathcal{H}_{\text{lin}}$ végtelen, az algoritmus viszont csak 45 modellt vizsgál meg.

A $w = (1,1)^\top$, $b = 0$ modellre $z = x_1 + x_2$, így

$$(0,0) \mapsto 0 \text{ helyes}, \quad (1,0) \mapsto 1 \text{ helyes}, \quad (0,1) \mapsto 1 \text{ helyes}, \quad (1,1) \mapsto 1 \text{ hibás}.$$

Ezért $R_{\text{tr}} = 1/4$. A XOR-pontok nem lineárisan szeparálhatók: a $(0,0)$ és $(1,1)$ pontokat összekötő szakasz a 0 osztályú pontok konvex burkában van, a $(1,0)$ és $(0,1)$ pontokat összekötő szakasz az 1 osztályú pontok konvex burkában van, és a két szakasz metszi egymást a (0,5; 0,5) pontban. Ezért nincs olyan félsík, amely mind a négy pontot helyesen osztályozná, tehát 1/4 a teljes $\mathcal{H}_{\text{lin}}$ osztályon is a minimális tanítóhiba. A példa így azt is mutatja, hogy a hipotézisosztály megválasztása korlátozza az elérhető tanítóhibát, függetlenül az algoritmus ügyességétől.

1.1.6 Kézi példa és minimális kód

A rácskeresés kézzel áttekinthető, de a kód a hallgató számára ellenőrizhetővé teszi. Input: `X` float32, `y` int64; a kimenet a legjobb rácspont és a hozzá tartozó tanítóhiba.

```
# munkakönyvtár: lab/
import numpy as np

X = np.array([[0.0, 0.0],
              [1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]], dtype=np.float32)  # (4, 2)
y = np.array([0, 1, 1, 0], dtype=np.int64)   # (4,)

def train_error(w1, w2, b):
    z = X @ np.array([w1, w2], dtype=np.float32) + b
    pred = (z > 0).astype(np.int64)
    return float(np.mean(pred != y))

grid_w = [-1, 0, 1]
grid_b = [-2, -1, 0, 1, 2]
best = (None, None, None, np.inf)
for w1 in grid_w:
    for w2 in grid_w:
        for b in grid_b:
            err = train_error(w1, w2, b)
            if err < best[3]:
                best = (w1, w2, b, err)
print(best)  # (-1, -1, 2, 0.25)
```

A kiírt érték a megadott bejárási sorrendből számított eredmény: a minimális 0,25 hibát több rácspont is eléri, de az első a $(-1, -1, 2)$ paraméterhármas. A hallgató futtassa le, és a kapott négyest illessze a F3. feladat kontrolljához.

1.1.7 Labor: GlyphLab problémalap osztályozóhoz és pixelmaszkhoz

Ez a labor dokumentációs labor, nem betanított modell. Célja, hogy a hallgató a mellékelt GlyphLab adatra két külön felhasználási feltétellel rendelkező problémalapot írjon.

Előfeltétel. Az 1.1.3 feladatszerződés és az 1.1.5 XOR-példa ismerete; a labor nem igényel betanított modellt és GPU-t. A futtatáshoz a `cnn-labor` csomag és a `lab/` munkakönyvtár szükséges.

Laborcsomag. A könyv minden laborja a `cnn-labor` laborcsomagra épül, amely a könyv weboldaláról közvetlenül letölthető: `cnn-labor-1.0.0.zip`, az ellenőrzőösszeg pedig a mellette lévő `cnn-labor-1.0.0.zip.sha256` fájlban van (letöltés után: `sha256sum -c cnn-labor-1.0.0.zip.sha256`; a csomag a kiadott forrásszövegből készül, ezért az összeg minden kiadásnál az adott kiadáshoz tartozó fájlból olvasandó). A nyomtatott/PDF változatban ugyanez a csomag a könyv melléklete. A csomag a jelen kiadás forrásszövegéből készül. Tartalma: a `lab/` munkakönyvtár a GlyphLab eredeti adatgenerátorával (`generate_data.py`, seed `20261020`, generátorverzió 1.0), az előre legenerált `data/` fájlokkal, a `quickcheck.py` ellenőrzővel és a könyv `lab/...` fejlécű laborfájljaival; a lockolt környezet (`lab/pyproject.toml`, `lab/uv.lock`: Python 3.12, NumPy 2.3.5, CPU-s PyTorch 2.14.1, Matplotlib 3.10.8, pandas 2.3.3, SciPy 1.18.1); a könyv minden kódblokkja alfejezetenként (`examples/`, `catalog.json`) és alfejezetenkénti notebookokban (`notebooks/`); valamint a fájlonkénti SHA-256 lenyomatokat tartalmazó `RELEASE_MANIFEST.json`. **Licenc:** a kód MIT, a generált GlyphLab-adat és a dokumentáció CC BY 4.0 (a csomag `LICENSE` fájlja). Az adat szintetikus: külső képet, személyes adatot vagy

letöltött adatkészletet nem tartalmaz. **Telepítés** kicsomagolás után egyetlen paranccsal: `cd cnn-labor-1.0.0/lab && uv sync` (a környezetről részletesen: 2.3. Izolált környezet, notebook és verzió). Az adat a csomagban is benne van, de üres `data/` könyvtárból is előállítható: `uv run python generate_data.py --output data`, majd `uv run python quickcheck.py`. A generátor azonos NumPy-verzió és platform mellett a fenti lenyomatokkal egyező fájlokat állít elő; más környezetben a hash-egyezés nem garantált. A referencia-lenyomatok (a `data/manifest.json` `sha256` mezője, amelyet a `quickcheck.py` ellenőriz):

```
images.npy
    cdb84fcbcdf6f2b6a09bfeba3b1f81b83a2c1f120b5be7cdac5fffb3b65a48e2
masks.npy
    bd884de178f2fc43c4bea3203cc21c47c466a99e0438fd783a34dc8cb4d966b2
labels.npy
    073013f0be8aa25c5c1cf81bf9aeb85a07b501335493af2481799921e3286539
corrupted_test.npy
    0936e20e7830744acd1519d81d3f0364fc2a1707985370fe4378d5cfc2967515
metadata.csv
    5acc5c5d4c6e1866b30519c84c35c895b60b499feff12c680bfbaf02fb5c1b9b
```

Adatellenőrzés (mért). A `quickcheck.py` a csomagolt `data/` könyvtáron a szerkesztéskor lefutott: mind az öt hash egyezett, a split szétválasztása és a doboz–maszk egyezés `true`, a konvolúciós referencia legnagyobb abszolút gradienshibája $4{,}3 \cdot 10^{-10}$ alatt maradt. A futás környezete: Python 3.12.3, NumPy 2.5.4, egy szál (`OMP_NUM_THREADS=1`), seed `20261020`; a laborcsomag lockolt profilja NumPy 2.3.5-öt rögzít, a hash-ellenőrzés pedig a csomagolt, előre generált fájlokat hasonlítja össze. A tényleges kimenet:

```
{
  "synthetic": true,
  "images": 360,
  "scene_counts": {
    "train": 120,
    "val": 30,
    "test": 30
  },
  "split_disjoint": true,
  "bboxes_match_masks": true,
  "numeric_gradient_max_error": {
    "x": 4.234644301881474e-10,
    "w": 4.2803538491398285e-10,
    "bias": 1.6975815197994848e-10
  }
}
```

Környezeti adatok. Munkakönyvtár: `lab/`. Az adatfájlok: `data/images.npy` alakja `(360,1,24,24)` és típusa `float32`, értékkészlete $[0, 1]$; `data/labels.npy` alakja `(360,)` és típusa `int64`, értékei `{0,1,2}`; `data/masks.npy` alakja `(360,1,24,24)`, típusa `uint8`, értékei `{0,1}`; `data/metadata.csv` oszlopai `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax`. A `data/corrupted_test.npy` csak a test sorrendjét követi, alakja `(60,1,24,24)`. A generálási seed `20261020`; a későbbi tanítási seed ettől külön rögzítendő. A jövőbeli modell bemenete `float32`, a címke `int64`, a maszk `uint8`; eszköz CPU, ettől eltérő eszközt külön dokumentálni kell. Az adat-ellenőrző parancs a hallgatói futtatáshoz: `uv run python quickcheck.py` a `lab/` könyvtárban; a kimenetét a hallgató a saját jegyzőkönyvébe menti. A lockolt profil (Python 3.12, NumPy 2.3.5) rögzíti a generátor környezetét; a `quickcheck.py` a fenti lenyomatokkal ellenőrzi az adatot.

A leadandó `lab/problem_specs.md` két blokkból áll. Mindkettő kötelező rovatai: *Bemenet*, *Döntés*, *Cél*, *Siker*, *Split*, *Megfigyelési egység*, *Baseline*, *Felhasználási feltételek*.

A. Orientáció-osztályozó. Bemenet: a képtensor $X \in \mathbb{R}^{N\times 1\times 24\times 24}$. Döntés: $\hat{y} \in \{0,1,2\}$ (0 függőleges, 1 vízszintes, 2 átlós). Cél: $y \in \{0,1,2\}^N$, int64. Siker: test-beli per-kép egyezési arány; a predikció és metrika elemi egysége a kép, a split és a bizonytalanságbecslés független mintavételi egysége a `scene_id`, mert egy jelenet két nézete nem független. Split: osztályonként train 40, val 10, test 10 jelenet; összesen 120/30/30 jelenet, azaz jelenetenként két nézettel 240/60/60 kép. Baseline: a train-beli leggyakoribb osztály konstans döntése; a három osztály egyenlő jelenetszáma miatt fellépő holtversenyt a legkisebb osztályindex (0) feloldja. Felhasználási feltételek: modellt és hiperparamétert csak train/val alapján választunk; a testet a rögzített választás után legfeljebb egyszer értékeljük; a korrupt tesztet csak befagyasztott modellen, utólagos ráhangolás nélkül értékeljük; a maszk, doboz és `scene_id` nem lehet az osztályozó bemenete; a 90°-os forgatás nem címkemegőrző augmentáció, mert a függőleges és vízszintes címkét felcseréli.

B. Pixelmaszk. Bemenet: a képtensor. Döntés: $\hat{M} \in \{0,1\}^{N\times 1\times 24\times 24}$ előtér-háttér előrejelzés. Cél: a `masks.npy` M tömb. Siker: képenkénti

$$\mathrm{IoU} = \frac{|M \cap \hat{M}|}{|M \cup \hat{M}|},$$

ahol a halmaz a nemnulla pixelek halmaza; a GlyphLab képei tartalmaznak előteret, ezért az üres unió esete nem fordul elő, de ha mégis, a megfigyelést kizárjuk és jelentjük. Megfigyelési egység: kép és jelenet; a per-pixel átlag nem keverendő a per-image átlaggal. Felhasználási feltételek: a maszk csak cél, nem bemenet; a kép és a maszk geometriai transzformációját együtt kell végezni; a `metadata.csv` doboza a maszk legkisebb befoglaló, nullától indexelt, félig nyílt xyxy téglalapja, $x_{\min} \le x < x_{\max}$, $y_{\min} \le y < y_{\max}$.

A két problémalap külön feltételeket rögzít, mert az osztályozó egy képszintű döntést, a maszkmodell pedig pixelenkénti döntést ad; a kettő sikerfogalma és adatszivárgási felülete sem azonos.

1.1.8 Feladatok

F1 (fogalmi, nehézség 1). Sorold be az AI/ML/DL/CNN halmazokba: (a) kézzel írt szabályokkal működő spam-szűrő; (b) legkisebb négyzetes lineáris regresszió; (c) konvolúciós háló képfelismerésre; (d) az a klasszifikátor, amely a train halmaz leggyakoribb osztályát adja vissza. Cáfold meg a „minden ML-rendszer mélytanulás” állítást. *Beadandó:* 4–8 mondat indoklással.

F2 (levezetési, nehézség 2). Adott a $D = \{(-1,0),(0,1),(1,0)\}$ egydimenziós adat, a $\mathcal{H} = \{h_a(x) = \mathbf{1}[x > a] : a \in \mathbb{R}\}$ hipotézisosztály és a félreosztályozási arány mint tanítóhiba. Igazold esetszétválasztással, hogy a minimális tanítóhiba 1/3. Adj meg egy bővebb hipotézisosztályt, amelyen a tanítóhiba 0, konkrét paraméterrel. *Beadandó:* az esetek, a számolás és a bővebb modell paramétere.

F3 (implementációs, nehézség 2). Írj futtatható Python függvényt `grid_linear(X, y)` néven az XOR-rácskeresésre. Input: `X` float32 `(N,2)`, `y` int64 `(N,)`; output `(w1, w2, b, err, pred)` a fenti rács legkisebb hibájú pontjával, holtversenyben a bejárási sorrend szerinti elsővel. Futtasd a XOR-adaton, és ellenőrizd a `(-1,-1,2)`, `err=0.25`, `pred.shape=(4,)` értékeket. *Beadandó:* a teljes függvény importokkal, inputtal és `assert` kontrollal.

F4 (hibakeresési, nehézség 2). Az alábbi vázlat a küszöböt a teljes adaton választja, majd ugyanazon az adaton számol hibát. A vázlatban a `...` és a `[...]` a konkrét adat, illetve a küszöbrács helye; a hiányzó definíciókat a hallgatónak kell pótolnia:

```
X = ...  # a teljes megfigyelés
y = ...
grid = [...]
best = min(grid, key=lambda a: np.mean((X > a) != y))
report = np.mean((X > best) != y)
```

Lokalizáld a hibát: miért nem tekinthető a `report` független teszt eredménynek? *Beadandó:* az ok, a következmény, a javított protokoll és a visszaellenőrzés módja.

F5 (kísérleti/transzfer, nehézség 3). A GlyphLab osztályozót olyan új jeleneteken szeretnénk vizsgálni, amelyek 48×48 felbontásúak és erősebb zajt kaptak. Tervezz előzetes kísérletet: hipotézis, változó, kontroll, baseline, megfigyelési egység, adatválasztási szabály, metrika, seedek, döntési szabály, erőforrás, protokoll és érvényességi korlátok. A modellt a célválasztáshoz ne hangold újra. *Beadandó:* egy protokollterv a fenti rovatokkal, a korlátok indoklásával.

1.1.9 Megoldások és indoklás

F1 megoldás. (a) A szabályalapú spam-szűrő AI-alkalmazás lehet, de nem ML, mert a döntési függvényt nem adatból választottuk. (b) A lineáris regresszió adatból becsli a paramétereit, tehát ML és AI, de nem DL, mert nem sok rétegű paraméterezett háló. (c) A képfelismerő CNN ML és a tág munkadefiníció szerint AI. A DL besorolás a háló mélységétől függ: az 1.1.2 szerint egyetlen konvolúciós réteg lineáris fejjel is CNN, de ez a háló nem DL; ha a feladatban szereplő háló több, tanult átalakítást egymásra építő, aktivációkkal tagolt (mély) háló, akkor DL is. (d) A leggyakoribb osztályt visszaadó klasszifikátor tanul, ha a többségi osztályt a trainből becsüljük; ez ML, de egyben a legegyszerűbb baseline, és sem nem DL, sem nem CNN. A „minden ML-rendszer mélytanulás” hamis, mert (b) és (d) ML, de nem DL.

F2 megoldás. Az a paraméter esetei: ha $a < -1$, mindhárom predikció 1, hiba $2/3$; ha $-1 \le a < 0$, a (-1)-hez 0, a másik kettőhöz 1 tartozik, hiba $1/3$ (az $a = -1$ határeset ugyanígy); ha $a = 0$, a 0 ponthoz 0 tartozik, pedig célja 1, az 1 ponthoz 1 tartozik, pedig célja 0, hiba $2/3$; ha $0 < a < 1$, hasonlóan $2/3$; ha $a \ge 1$, az (1) pont jó, a (0) pont rossz, hiba $1/3$. Tehát a minimum $1/3$. Bővebb hipotézisosztály az intervallum-jellegű

$$\mathcal{H}_{\text{int}} = \{h_{l,r}(x) = \mathbf{1}[l < x < r] : \ l \in \mathbb{R},\ r \in \mathbb{R} \cup \{+\infty\},\ l < r\};$$

ez valóban bővebb, mert $r = +\infty$ mellett $h_{a,+\infty} = h_a$, tehát $\mathcal{H} \subseteq \mathcal{H}_{\text{int}}$. A $l = -0{,}5$, $r = 0{,}5$ paraméter mindhárom pontot helyesen osztályozza, hibája 0. (A $+\infty$ végpont nélkül az intervallumosztály nem tartalmazná $\mathcal{H}$-t; ekkor csak a megadott véges mintán vett megszorítások kifejezőerejét hasonlíthatnánk össze.)

F3 megoldás. A teljes függvény:

```python
# lab/f3_grid_linear.py
import numpy as np

def grid_linear(X, y):
    assert X.ndim == 2 and X.shape[1] == 2
    assert X.dtype == np.float32
    assert y.ndim == 1 and y.dtype == np.int64
    assert X.shape[0] == y.shape[0] > 0, "azonos, pozitív mintaszám kell"
    best = None
    grid_w = [-1, 0, 1]
    grid_b = [-2, -1, 0, 1, 2]
    for w1 in grid_w:
        for w2 in grid_w:
            w = np.array([w1, w2], dtype=np.float32)
            for b in grid_b:
                z = X @ w + b
                pred = (z > 0).astype(np.int64)
                err = np.mean(pred != y)
                if best is None or err < best[3]:
                    best = (w1, w2, b, err, pred.copy())
    return best
```

```
X = np.array([[0.0, 0.0], [1.0, 0.0],
              [0.0, 1.0], [1.0, 1.0]], dtype=np.float32)
y = np.array([0, 1, 1, 0], dtype=np.int64)
w1, w2, b, err, pred = grid_linear(X, y)
assert (w1, w2, b) == (-1, -1, 2)
np.testing.assert_allclose(err, 0.25, rtol=1e-7, atol=1e-10)
assert pred.shape == y.shape == (4,)
np.testing.assert_array_equal(pred, np.array([1, 1, 1, 0], dtype=np.int64))
```

A bemeneti `assert`-ek csak azt állítják, amit ténylegesen ellenőriznek: `X` kétdimenziós, két oszlopú és float32 típusú; `y` egydimenziós és int64; a mintaszám pozitív, és `X` meg `y` sorainak száma azonos. Ebből következik, hogy a `pred` alakja megegyezik `y` alakjával, és hogy az átlagolt hiba nem üres tömbön számolódik. Például `X.shape == (4, 2)` és `y.shape == (1,)` esetén a függvény `AssertionError: azonos, pozitív mintaszám kell` hibával áll meg, ahelyett hogy a `pred != y` összehasonlítás csendben broadcastolna. A numerikus tolerancia az átlag pontos értékére vonatkozik, mert négy mintán a hiba $k/4$ alakú.

F4 megoldás. A hiba oka adatszivárgás: a küszöb kiválasztása a teljes `X, y` páron történik, tehát a „teszt” megfigyelések is befolyásolták a modellt. Következménye optimista torzítás: a jelentett hiba a már látott mintákon mér, ezért a 0 hiba nem általánosítási bizonyíték. Javítás: a splitet a keresés előtt rögzítjük, a küszöböt csak a train indexeken választjuk, majd a testen egyszer számolunk. A `fixed_split` itt helyőrző: a hallgatónak kell megírnia a splitet determinisztikusan előállító függvényt; a `...` a split paramétereit jelöli.

```
train_idx, test_idx = fixed_split(...)
best = min(grid, key=lambda a: np.mean((X[train_idx] > a) != y[train_idx]))
report = np.mean((X[test_idx] > best) != y[test_idx])
```

Visszaellenőrzés: rögzítsük a split azonosítót és a nyers hibát; ismételt futtatáskor ugyanazzal a splittel ugyanazt az eredményt kell kapnunk, és a test nem vehet részt a `min` keresésben.

F5 megoldás (minta). A kísérlet előfeltétele egy korábbi fejezetben, rögzített protokollal betanított GlyphLab-osztályozó (a Projekt II modellje, 38.2. Modell, training és best checkpoint); az 1.1. labor önmagában nem tartalmaz betanított modellt. Kérdés: mennyivel romlik ez a modell, ha a felbontás 24 → 48 és a zaj nő? Hipotézis: a teszt accuracy legalább 0,10-del csökken a megnövelt felbontású, zajosabb adaton. Változó: a felbontás és a zajszórás; kontroll: az eredeti 24×24-es, tiszta teszt. Baseline: a régi train leggyakoribb osztályát konstansként adó szabály az új teszten, hogy a modell ennél jobb-e. Megfigyelési egység: `scene_id`; az új generátor 30 tesztjelenetből 60 képet készít, osztályonként 10 jelenetet, hogy a korábbi split szerkezet megmaradjon. A modellválasztás kizárólag a régi train/val halmazon történt; az új adat csak zárt kiértékelésre szolgál. Metrika: per-kép accuracy, majd scene-resamplinggel számított bootstrap intervallum. Protokoll: a modell súlyait befagyasztjuk, és rögzítjük a bemeneti szerződést: a befagyasztott modell fix `flatten` fejjel 24×24-es bemenetet vár, ezért a 48×48-as képeket vagy előre 24×24-re méretezzük vissza (bilineáris resize, a lépést a naplóban dokumentálva), vagy a fejet `AdaptiveAvgPool2d` alapú, felbontásfüggetlen változatra cseréljük. A fejcserével keletkező modell új modellváltozat: a train/val halmazon kell ellenőrizni, és külön dokumentálni; a befagyasztott eredeti modell referenciaként a resize-os ágon marad. Az új generátor paraméterezése: a jelenlegi generátor (1.0 verzió) csak `--output` és `--seed` kapcsolót ismer, ezért a kísérlethez a generátort 1.1 verzióra kell bővíteni `--size` és `--noise` kapcsolóval; a zajszórás a tiszta adat $\sigma = 0{,}055$ értékének kétszerese, azaz 0,11 legyen, a seedet külön rögzítjük. A predikciókat futásonként egyszer mentjük, a nyers hibamátrixot és a seedeket naplózzuk. Erőforrás: CPU, az új generálás és a kiértékelés néhány perc; a pontos időt a futási napló rögzíti. Érvényességi korlát: a felbontás és a zaj egyszerre változik, ezért a kettő hatása nem választható szét faktoriális kísérlet nélkül; a GlyphLab továbbra sem természetes képi benchmark, így a romlás nem általánosítható valós alkalmazásra.

1.1.10 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés, hogy a CNN az összes AI-modell szinonimája. Ellenpélda: a $z = ax + b$ lineáris regresszió ML-modell, mert (a, b) paramétereit adatból becsüljük, de nincs konvolúciós rétege, ezért nem CNN. Megfordítva, egy konvolúciós háló sem fedi le az összes AI-t: egy kézzel írt sakkszabály-motor AI lehet, de nincs tanuló paraméterezése. A rövidítések tehát halmazviszonyt jelölnek, nem stíluscímkét.

1.1.11 Önellenőrzés és visszatekintés

Ellenőrizd magad: minden állításnál meg tudod-e nevezni a bemeneti teret, a döntési teret, a célt és a sikerkritériumot? El tudod-e különíteni, hogy egy lépés a tanításhoz vagy az inference-hez tartozik? Korábbi könyvfogalom híján a visszatekintés a feltételezett programozói előismeretre korlátozódik: a tanító algoritmus is egy függvény, csak a visszatérési értéke nem közvetlen predikció, hanem az a $\hat{\theta}$ paraméter, amellyel később predikciót számítunk. **Kulcs:** (1) a feladatszerződés négy eleme: bemenet, döntés, cél és sikerkritérium; (2) a tanítás a paramétert választja, az inference a rögzített modellt értékeli új bemeneten.

1.1.12 Rövid összefoglaló

Ebben az egységben az AI, ML, DL és CNN fogalmát különválasztottuk: az AI tág munkakategória, az ML adatból választ modellt, a DL paraméterezett neurális hálócsalád, a CNN lokális súlymegosztott rétegeket használ. A modell, a tanítás és az inference külön életciklus; a feladatszerződés bemenetből, döntésből, célból és mérhető sikerből áll. Az XOR-példán definiáltuk az adatot, a hipotézisosztályt és a kereső algoritmust: a legjobb lineáris modell tanítóhibája 1/4, mert a pontok nem lineárisan szeparálhatók. A GlyphLab-labor két szerződést rögzít: orientáció-osztályozás és pixelmaszk, eltérő megfigyelési és felhasználási feltételekkel. Az induló kérdésre a válasz: a kézzel írt „ingyen” szabály nem tanult modell, mert a döntési függvényt nem adatból választottuk; tanuláshoz címkézett adat és a paramétert adatból kiválasztó tanító algoritmus kell.

1.1.13 Továbblépés

A következő egység azt tárgyalja, mitől egyetemi szintű egy modellezési állítás: a feltételeket, a bizonyítékok típusait és az ellenőrzés módszereit. Az itteni feladatszerződést ott hipotézisekkel és kísérleti naplóval egészítjük ki.

1.1.14 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [P01] LeCun, Y., Bottou, L., Bengio, Y. és Haffner, P. (1998). *Gradient-based Learning Applied to Document Recognition*. Proceedings of the IEEE. DOI: 10.1109/5.726791.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. A könyvben szereplő példák saját kidolgozásúak.

1.2 Mitől egyetemi szintű egy modellezési állítás?

Az 1.1. Modell, tanulás és felhasználási szerződés egységben a modell, a tanítás és az inference életciklusát, valamint a feladatszerződés négy elemét rögzítettük. Ott az XOR-példa egy konkrét, ellenőrizhető állítást adott: a lineáris hipotézisosztály legjobb elemének tanítóhibája 1/4. Ebben az egységben azt vizsgáljuk, milyen bizonyíték illik egy modellezési állítás mellé, és miért nem mindegy, milyen típusú kijelentést teszünk. Induló kérdés: „A tanító kódom hiba nélkül lefutott, és 98%-os pontosságot írt ki; miért ne írhatnám a jelentésbe, hogy a rendszer 98%-ban megbízható?”

1.2.1 Állítástípusok és ellenőrzési szerződésük

A szakmailag megalapozott modellezési szövegben minden lényeges kijelentésnek felismerhető típusúnak kell lennie, mert a típus határozza meg, milyen ellenőrzéssel fogadható el. A következő hat típust különböztetjük meg.

Definíció. Rögzíti egy objektum jelölését, tartományát, alakját vagy konvencióját. Például: „legyen $x \in \mathbb{R}^{24\times 24}$ egy szürkeárnyalatos GlyphLab-kép, az intenzitások a $[0, 1]$ intervallumba esnek”. A GlyphLab adatszerződését a függelék A. szakasza rögzíti. A definíciót nem adatok cáfolják, hanem az ellentmondás-mentesség és a használhatóság; ha két helyen eltérően használjuk, az szerkesztési hiba.

Tétel vagy lemma. Feltételekből következtetést vezet le, és rendelkezik bizonyítással vagy pontosan megjelölt, átvett eredménnyel. Példa: ha f lineáris leképezés, akkor $f(0) = 0$. Egyetlen ellenpélda cáfolhatja, de egy futtatás nem igazolhatja; a bizonyítás státuszát mindig meg kell adni.

Levezetés. Egyenlőségek vagy becslések lépésről lépésre követhető sorozata, ahol minden átalakítás indokolt. Ellenőrzése indexhatárok, dimenziók, konstansok és szélsőesetek vizsgálatával történik. A „nyilvánvaló” szó nem helyettesíti a hiányzó lépést.

Empirikus állítás. Adott adaton, adott protokollal, adott metrikával mért eredményt közöl. Például: „az adott spliten, az adott seeddel a modell validációs accuracyje 0,93 volt”. Csak a kimondott feltételek között általánosít; nyers eredményt, reprodukciós utat és korlátot igényel.

Heurisztika. Gyakorlati szabály, amely motivációt, ismert korlátot és ellenőrzési tervet tartalmaz, de nem univerzális garancia. Például: „a kis tanulási ráta gyakran stabilabb tanítást ad”; ez nem tétel, mert túl kis ráta mellett a tanítás nagyon lassan halad, és az adott lépésszám alatt nem konvergál.

Szakirodalmi eredmény. Más szerzőtől származó állítás, amelyhez pontos változat, konkrét szakasz és az ellenőrzés státusza tartozik. Az absztraktból ismert eredmény nem azonos a teljes cikkből olvasott módszerrel.

A hat típus nem zárja ki egymást: egy jó kísérleti jelentés tartalmaz definíciót, levezetést és empirikus állítást is, de ezeket nem keveri össze.

1.2.2 Feltétel, bizonyíték, ellenpélda, reprodukció, replikáció, attribúció

Hat keresztfogalom kapcsolja össze a fenti típusokat.

Feltétel. Minden tételhez és empirikus állításhoz tartozik egy feltételhalmaz. A tételnél a feltételek a matematikai előfeltevések; az empirikus állításnál az adat, a split, a seed, a modellváltozat, a metrika és a környezet. Ha egy feltétel hiányzik, az állítás nem ellenőrizhető.

Bizonyíték. A típusnak megfelelő ellenőrzés: definíciónál konzisztencia, tételnél bizonyítás, levezetésnél lépésenkénti indoklás, empirikus állításnál nyers mérés és reprodukciós út, szakirodalmi eredménynél olvasott forrásszakasz.

Ellenpélda. Olyan konkrét eset, amelyben a feltételek teljesülnek, de a következtetés hamis. Az ellenpélda a cáfolat eszköze; a következő szakaszban a fő kötelezettséget ezzel teljesítjük.

Reprodukció. Ugyanazon kód, adat, seed és környezet futtatása; a cél az eredmény megismételhetősége. A reprodukció nem helyettesíti a bizonyítást, és nem bizonyítja az általánosítást.

Replikáció. Ugyanazon kérdés megválaszolása független újraimplementálással vagy új adaton, azonos protokoll mellett; a cél annak ellenőrzése, hogy a következtetés iránya és nagyságrendje stabil-e. A replikáció nem azonos a reprodukcióval: a reprodukció ugyanazt a számot, a replikáció ugyanazt a következtetést próbálja megkapni.

Attribúció. A szakirodalmi eredmény származásának pontos megjelölése. A forrás határozza meg, milyen részletességű állítás támasztható alá: egy absztrakt alkalmas lehet a kutatási cél és a fő módszer azonosítására, de nem helyettesíti az architektúra vagy egy tétel feltételeinek ellenőrzését a teljes szövegben. A [P01] dokumentumfelismerési alkalmazása, a [B01] matematikai háttere és az [S01] konvolúciós ismertetése különböző célú forrás; az átvett állításokat és a saját levezetéseket egyértelműen el kell választani.

1.2.3 Ellenpélda: a sikeres kódlefutás nem bizonyít általánosítást

A gyakori, implicit állítást így formalizálhatjuk.

(H) Hamis állítás. Legyen m a D_{train} tanítóhalmazon mért accuracy, $0 \leq m \leq 1$. Ha egy program hiba nélkül lefut, és a D_{train} tanítóhalmazon $m = 1$ accuracyt ír ki, akkor a program által reprezentált döntési szabály a D_{use} felhasználási eloszláson is $m = 1$ accuracyt ér el.

Ellenpélda. Tekintsünk bináris osztályozást. Legyen D_{train} száz olyan mintából álló halmaz, amelyben minden $y_i = 0$; a program a konstans $f(x) = 0$ prediktort valósítja meg. A kód szintaktikailag helyesen lefut, és az accuracyt $\frac{100}{100} = 1$-ként írja ki, tehát a (H) feltételei teljesülnek. Legyen most D_{use} olyan felhasználási eloszlás, amelyben minden minta címkéje 1. Ugyanez a program ezen $\frac{0}{100} = 0$ accuracyt ad. A feltételek fennállnak, a következtetés hamis, ezért (H) nem érvényes.

A példa tanulsága nem az, hogy a tanító accuracy haszontalan, hanem az, hogy a kód futása és a tanító metrika nem hordozza a felhasználási eloszlásra vonatkozó feltételt. Az általánosítási állításhoz külön bizonyíték kell: független, csoportosított split, nyers tesztmetrika, bizonytalansági határ és a domaineltérés vizsgálata.

1.2.4 Kézi számpélda és minimális kód

Legyen D_{train} száz címkéből álló tömb: 90 darab 0 és 10 darab 1. A konstans 0 prediktor tanító accuracyje

$$\frac{90}{100} = 0{,}9.$$

Legyen D_{test} eltérő címkeeloszlású: 10 darab 0 és 90 darab 1. Ugyanazon prediktor teszt accuracyje

$$\frac{10}{100} = 0{,}1.$$

A két szám ugyanabból a hibátlanul futó programból származik; a különbséget nem a kód, hanem a két halmaz eltérő címkeeloszlása okozza. Az alábbi kód közvetlenül ellenőrizhető; a kimenet a definícióból is kiszámolható, és a kód `assert`-jei ellenőrzik.

```
import numpy as np

y_train = np.concatenate([
    np.zeros(90, dtype=np.int64),
    np.ones(10, dtype=np.int64)
])
y_test = np.concatenate([
    np.zeros(10, dtype=np.int64),
    np.ones(90, dtype=np.int64)
])

def accuracy(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    assert y_true.shape == y_pred.shape
    return float(np.mean(y_true == y_pred))

pred = np.zeros_like(y_train)
print("train accuracy:", accuracy(y_train, pred))  # 0.9
print("test accuracy:", accuracy(y_test, pred))    # 0.1
np.testing.assert_allclose(accuracy(y_train, pred), 0.9, atol=1e-12)
np.testing.assert_allclose(accuracy(y_test, pred), 0.1, atol=1e-12)
```

A `mean` alapértelmezés szerint az összes elemet átlagolja, tehát a visszatérési érték skalár; a `float` konverzió a kiírás egyértelműségét szolgálja. Ez a példa nem használ véletlent, ezért seed nem szerepel benne; a tömbök determinisztikusan adottak. A kód a szerkesztéskor lefutott; a tényleges kimenet `train accuracy: 0.9` és `test accuracy: 0.1` (Python 3.12.3, NumPy 2.5.4, egy szál), és a két `assert_allclose` sor ezt ellenőrzi.

1.2.5 Labor: tíz állítás besorolása és bizonyítékjegyzék

Előfeltétel. Az 1.1. egység feladatszerződése és az állítástípusok ismerete; numerikus futás nem szükséges.

Ebben a laborban nincs numerikus futás, ezért a seed és a device nem értelmezett; a dtype csak az F3 kódpéldájában jelenik meg, a futtatható ellenőrző részletet pedig az F3 feladat tartalmazza. A bemenet az alábbi tíz rövid eredményállítás:

1. **A1:** „A GlyphLab tanítókódja az én gépemen hiba nélkül lefutott.”
2. **A2:** „Az én CNN-em a GlyphLab teszt splitjén 0,93 accuracyt ért el.”
3. **A3:** „A softmax kimenet elemei nemnegatívak és 1-re összegződnek.”
4. **A4:** „Ha $b \neq 0$, az $f(x) = Wx + b$ leképezés nem lineáris.”
5. **A5:** „Minél több konvolúciós réteget teszünk a hálóba, annál kisebb lesz a teszt hiba.”
6. **A6:** „A [P01] forrás gradiensalapú tanulást alkalmaz dokumentumfelismerésre.”
7. **A7:** „A tanító veszteség az első öt epochban csökkent, tehát az optimalizáló konvergált a globális minimumhoz.”
8. **A8:** „A BatchNorm mindig javítja a pontosságot.”
9. **A9:** „A modell 0,97-es validációs accuracye garancia arra, hogy éles használatban is 0,97-es accuracy lesz.”
10. **A10:** „A ReLU definíció szerint $\max(0, x)$.”

Beadandó eredmény. Minden állításhoz készítsen legfeljebb négy oszlopos táblázatot: állítás azonosítója, besorolás a hat típus valamelyikébe, hiányzó feltétel vagy bizonyíték, ellenőrzési terv. Ahol egy mondat több típust kever, a záró következtetést külön is jelölje. A besorolást a Megoldások és indoklás szakasz ellenőrzi.

Bizonyítékjegyzék saját kísérlethez. Egy empirikus állítás mellé az alábbi elemeket kell rögzíteni:

1. a kutatási kérdés,
2. az adat forrása és verziója,
3. a split és a megfigyelési egység,
4. a modell és a hipotézisosztály,
5. a baseline,
6. a metrika és az aggregálás módja,
7. a seed és a környezet,
8. a nyers eredmény és a reprodukciós parancs,
9. az értelmezési korlát.

Ha egy későbbi GlyphLab-kísérlet azt állítaná, hogy egy adott CNN a teszt halmazon 0,93 accuracyt ért el, a jegyzék például így nézne ki: kérdés: az orientáció osztályozása; adat: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`, a manifest hash-sel; split: `scene_id` szerint train/val/test, osztályonként 10 független jelenet a tesztben; modell: a kísérletben rögzített architektúra; baseline: konstans osztályprediktor; metrika: per-kép accuracy, nyers confusion matrix; seed: a tanítás seedje; nyers eredmény: a tényleges mérés, nem pedig egy előre feltételezett szám; korlát: a GlyphLab nem természetes képi benchmark. A `0{,}93` itt szemléltető helyőrző: sem mért, sem levezetett érték, hanem a jegyzék kitöltésének példája; a helyére a hallgató saját nyers eredménye kerül. Az osztályonként 10 tesztjelenetből becsült 0,93 accuracy bizonytalansága nagy; a pontos intervallumot és a döntési szabályt a 5.3. Mintavételi bizonytalanság és intervallum tárgyalja.

1.2.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a reprodukció és a replikáció közötti különbséget, és adjon egy-egy konkrét GlyphLab-példát mindkettőre. Beadandó: 4–6 mondatos magyarázat.

F2 (levezetési, nehézség 1). Bizonyítsa be, hogy az $f : \mathbb{R} \to \mathbb{R}$, $f(x) = 3x + 2$ függvény nem lineáris leképezés. Beadandó: a linearitás definíciójából induló, ellentmondásra vezető levezetés.

F3 (implementációs, nehézség 2). Írjon Python-függvényt többosztályos accuracy kiszámítására, majd a konstans prediktoros ellenpéldával mutassa meg, hogy a tanító accuracy 1,0 lehet, miközben a teszt accuracy 0,0. Adja meg a dtype-ot és az ellenőrző kimenetet. Beadandó: teljes, futtatható kód.

F4 (hibakeresési, nehézség 2). Adott az érvelés: „A kód lefutott, a validációs accuracy 0,95, ezért a rendszer a jövőben is 95%-ban helyes.” Lokalizálja a hibát, írja át korrekt empirikus állítássá, és sorolja fel a hiányzó feltételeket. Beadandó: ok, következmény, javítás, visszaellenőrzés.

F5 (kísérlettervezési, nehézség 3). Tervezzen kísérletet a GlyphLab adaton a tanító–teszt accuracy rés mérésére. Rögzítse a hipotézist, a változót, a kontrollt, a protokollt, a splitet, a seedeket, a baseline-t, a metrikát, a megfigyelési egységet, a döntési szabályt, az erőforrást és az érvényességi korlátot; futtatás nem szükséges. Beadandó: protokollvázlat és a várható következtetések határa.

1.2.7 Megoldások és indoklás

A labor besorolása.

A1 empirikus állítás a kód futásáról, nem teljesítményállítás; hiányzó bizonyíték: környezet, verzió, parancs, kimenet. Ellenőrzési terv: a futtatás naplózása és megismétlése.

A2 empirikus teljesítményállítás; hiányzik a split, a seed, az architektúra, a nyers confusion matrix és a reprodukciós parancs. Ellenőrzési terv: a bizonyítékjegyzék mind a kilenc elemének rögzítése.

A3 tétel/lemma, amely a softmax definíciójából következik; bizonyítása: az exponenciális függvény pozitivitása és a normalizáló összeggel való osztás. Itt később, a 7.2. Softmax és többosztályos cross-entropy egységben részletezzük.

A4 lemma; bizonyítás: lineáris leképezésre $f(0) = 0$, de $f(0) = b \neq 0$, ami ellentmondás.

A5 univerzális empirikus általánosítás („minél több …, annál kisebb …”) formájában megfogalmazott heurisztika; nem tétel, mert adat cáfolhatja, és a feltételek (paraméterszám, regularizáció, adat) nincsenek rögzítve. Besorolási szabály: ha a mondat gyakorlati szabályt ad, heurisztika; ha minden esetre szóló „minél … annál …” következtetést állít, empirikus általánosítás, amelyet ellenpélda cáfolhat. Ellenőrzési terv: kontrollált kísérlet, ahol csak a rétegszám változik, a paraméterszám és a regularizáció rögzített.

A6 szakirodalmi állítás; a dokumentumfelismerési alkalmazás a [P01] forrás összefoglalójából alátámasztható, a részletes hálófelépítésre viszont csak a teljes tanulmány alapján szabad hivatkozni.

A7 empirikus megfigyelés hibás következtetéssel: a veszteségcsökkenés nem bizonyít globális konvergenciát. Ellenőrzés: gradiensnorma, több seed, hosszabb tanulási görbe.

A8 hamis univerzális állítás (a „mindig” miatt), amelyet egyetlen ellenpélda cáfol; a mögötte álló gyakorlati szabály heurisztika. A hatást kísérletenként kell mérni. Ellenőrzés: BatchNorm nélküli és BatchNormos modell azonos feltételekkel.

A9 téves általánosítás; az empirikus állításból nem következik felhasználási garancia. Hiányzik a domaineltérés, a kalibráció és a bizonytalansági intervallum.

A10 definíció; konvenció, nem cáfolható adattal.

F1 megoldás. A reprodukció ugyanazon kód, adat, seed és környezet futtatása; célja ugyanazon számérték visszakapása. GlyphLab-példa: a `data/manifest.json` hash-egyezése mellett a tanító script újrafuttatása ugyanazzal a seeddel. A replikáció független újraimplementálás vagy új adat ugyanazon protokollal; például egy másik kutató saját CNN-keretrendszerbeli implementációt ír a GlyphLab orientációs feladatához, és azt vizsgálja, hogy a következtetés iránya azonos marad-e.

F2 megoldás. Tegyük fel indirekt, hogy f lineáris. Ekkor minden $x, y \in \mathbb{R}$ esetén $f(x+y) = f(x)+f(y)$. Válasszuk $x = 1$, $y = 1$ értékeket. Egyrészt $f(x+y) = f(2) = 3 \cdot 2 + 2 = 8$. Másrészt $f(x)+f(y) = (3 \cdot 1 + 2) + (3 \cdot 1 + 2) = 5 + 5 = 10$. Mivel $8 \neq 10$, ellentmondásra jutottunk, tehát f nem lineáris. Ugyanerre vezet a zéruselem-vizsgálat: $f(0) = 2 \neq 0$.

F3 megoldás. A teljes ellenőrizhető kód:

```
import numpy as np

def accuracy(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    assert y_true.shape == y_pred.shape
    return float(np.mean(y_true == y_pred))
```

```python
# Háromosztályos címkék; a konstans prediktor mindig a 0 osztályt adja.
y_train = np.zeros(100, dtype=np.int64)                    # minden címke 0
y_test = np.concatenate([np.ones(50, dtype=np.int64),      # 50 db 1
                         np.full(50, 2, dtype=np.int64)])  # 50 db 2
pred_train = np.zeros(100, dtype=np.int64)
pred_test = np.zeros(100, dtype=np.int64)

print("tanító accuracy:", accuracy(y_train, pred_train))  # 1.0
print("teszt accuracy:", accuracy(y_test, pred_test))     # 0.0
np.testing.assert_allclose(accuracy(y_train, pred_train), 1.0, atol=1e-12)
np.testing.assert_allclose(accuracy(y_test, pred_test), 0.0, atol=1e-12)
```

A `dtype=np.int64` a címkék szerződését tükrözi; az összehasonlítás bool tömböt ad, amelynek `mean` értéke float64. A példa tiszta NumPy, ezért eszközválasztás (device) nem merül fel. A szerkesztéskori futás kimenete `tanító accuracy: 1.0` és `teszt accuracy: 0.0` (Python 3.12.3, NumPy 2.5.4, egy szál); a két `assert_allclose` sor ezt ellenőrzi.

F4 megoldás. Hiba: a kód futása és a validációs érték nem hordozza a jövőbeli felhasználási eloszlást. Következmény: az állítás erősebb, mint a rendelkezésre álló bizonyíték. Javítás: „Az adott spliten, a rögzített seeddel és protokollal a validációs accuracy 0,95 volt.” Hiányzó feltételek: nyers confusion matrix, a validáció függetlensége a modellválasztástól, a felhasználási eloszlás leírása, domaineltérés-vizsgálat, kalibráció és bizonytalansági határ. Visszaellenőrzés: kérjük a nyers mátrixot és a split azonosítót; ha ezek nincsenek meg, az állítás nincs kellően alátámasztva.

F5 megoldás. Lehetséges protokollvázlat: (1) kérdés: mekkora a tanító és teszt accuracy különbsége? (2) adat: GlyphLab `data/*.npy`, hash-ellenőrzés; (3) split: `scene_id` szerint train/val/test, osztályonként 10 független jelenet a tesztben; (4) modell: a 38.2. Modell, training és best checkpoint egységben rögzített GlyphLab-modell, változatlan architektúrával; (5) baseline: konstans osztályprediktor; (6) metrika: per-kép accuracy, nyers confusion matrix; (7) seedek: legalább három tanítási seed, a seed-szórást külön közölve; (8) nyers eredmény: tanító és teszt accuracy minden seedre; (9) döntési szabály: a rés pozitívnak tekinthető, ha mindhárom seeden nagyobb 0,05-nél, és a scene-szintű bootstrap intervallum nem tartalmaz nullát; (10) erőforrás: CPU, három tanítási seed, tervezett futásidő néhány perc; (11) korlát: a GlyphLab nem természetes képi benchmark, ezért a számot nem általánosítjuk más adatra. A várható következtetés: a rés iránya és nagysága az adott protokollra érvényes, nem globális modelltulajdonság.

1.2.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. A magabiztos szöveg és a sok hivatkozás nem helyettesíti az ellenőrzött érvelést. Egy dokumentum írhatja: „A modellünk garantáltan felismeri a ferde vonalakat, lásd 42 hivatkozás.” Ha egyetlen saját nyers mérést, protokollt vagy tételfeltételt sem közöl, az állítás nem vált ellenőrzötté; a hivatkozások más körülményekre vonatkoznak, és nem igazolják a jelenlegi modellt.

Önellenőrzés. (1) Melyik állítástípus cáfolható egyetlen ellenpéldával? Válasz: az univerzális állítás (minden esetre szóló következtetés), illetve a tételjelölt vagy az univerzális empirikus általánosítás; egy már bizonyított tételt nem cáfol ellenpélda. (2) Mit kell rögzíteni egy empirikus mérés reprodukciójához? Válasz: kód, adatverzió, split, seed, környezet, metrika és nyers eredmény. (3) Miért nem bizonyíték egy futtatás egy tételre? Válasz: mert a tétel minden feltételnek eleget tevő esetről szól, a futtatás pedig véges, konkrét eset.

Visszatekintés 1.1. Modell, tanulás és felhasználási szerződés alfejezetből. Az ottani feladatszerződés négy elemet rögzített: bemenet, döntés, cél és mérhető sikerkritérium. A mostani

egység ezt egészíti ki azzal, hogy a sikerkritériumhoz illő bizonyíték típusát is meg kell nevezni; a kettő együtt ad korrekt modellezési állítást.

1.2.9 Rövid összefoglaló

Ebben az egységben az állításokat típusuk szerint választottuk szét: a definíció konvenciót rögzít, a tétel feltételekből következményt bizonyít, a levezetés lépésenként ellenőrizhető, az empirikus állítás adott adaton és protokollal mért eredményt közöl, a heurisztika gyakorlati szabály, a szakirodalmi eredmény attribúciót és ellenőrzési státuszt igényel. A kulcspélda megmutatta, hogy a hibátlan kódlefutás és a magas tanító accuracy nem bizonyít általánosítást: konstans prediktorral 1,0 tanító és 0,0 felhasználási accuracy érhető el. A labor tíz állítás besorolását és egy bizonyítékjegyzék-sablont rögzített; a feladatok a reprodukció, a linearitás, az accuracy-kód, a hibás érvelés javítása és egy kísérleti protokoll köré épültek. Az induló kérdésre a válasz: a 98%-os pontosság önmagában nem jogosít a „98%-ban megbízható” állításra, mert hiányzik a független teszt, a bizonytalanság és a felhasználási eloszlás; a helyes állítás csak a feltételekkel együtt érvényes.

Továbblépés. A következő egység az adatfelosztást és a megfigyelési egységet tárgyalja; az itteni bizonyítékjegyzékben szereplő split és `scene_id` ott válnak precíz, ellenőrizhető fogalommá.

Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [S01] Stanford CS231n (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/.
- [P01] LeCun, Y., Bottou, L., Bengio, Y. és Haffner, P. (1998). *Gradient-based Learning Applied to Document Recognition*. Proceedings of the IEEE; DOI: 10.1109/5.726791.

1.3 Adatfelosztás és a megfigyelési egység

1.3.1 Cél, előismeret és induló kérdés

Az egység célja, hogy a programozói adatkezelést pontos megfigyelési egységgel, csoportos felosztással és ellenőrizhető szivárgásdefinícióval kapcsolja össze. Előismeretként csak az 1.2. Mitől egyetemi szintű egy modellezési állítás? állítástípusait és bizonyítékjegyzékét használjuk; haladó valószínűségszámításra nincs szükség.

Induló becslés. A függelék A. szakaszában leírt GlyphLab adatbázis 360 darab 24×24-es szürke képet tartalmaz, de ezek 180 jelenetből (*scene*) készültek, jelenetenként két zajos nézettel (*view*). Hány független megfigyelésünk van: 360 vagy 180? Ha valaki képszintű train/validation/test felosztást készít, pontosan milyen hibát követ el? Tippeljen, és a labor végén vesse össze a levezetett értékkel.

1.3.2 Definíciók és intuíció

Megfigyelési egység (observation unit). Az az elemi rekord, amelyről a modell egy előrejelzést ad. A GlyphLabban ez egy kép, tehát a modell bemenete egy $x^{(i)} \in \mathbb{R}^{C\times H\times W}$ tenzor; a kezdőadatban $C = 1$, $H = W = 24$, float32, $[0, 1]$ intenzitás. A hozzá tartozó cél $y^{(i)} \in \{0, 1, 2\}$, int64 osztályindex.

Scene és view. A *scene* az a rejtett, közös képződmény, amelyből a képek keletkeznek: ugyanaz a geometriai elrendezés, alakzatvastagság és pozíció. A *view* ennek egy zajos, önálló képi megfigyelése. A `metadata.csv` minden sorához `scene_id` és `view_id` tartozik; egy scene mindkét view-ja ugyanabba a splitbe kerül.

Csoport (group). Egy megfigyeléscsoport azon rekordok halmaza, amelyek közös rejtett véletlen forrásból származnak. Formálisan legyen $g(i)$ az i rekord csoportazonosítója; a GlyphLabban $g(i) = \text{scene_id}(i)$. Az érvényes felosztás feltétele:

$$\text{ha } g(i) = g(j), \text{ akkor } \text{split}(i) = \text{split}(j).$$

A kép a predikció és a képenkénti metrika megfigyelési egysége, míg a `scene_id` a split és a csoportos bizonytalanságbecslés egysége. A `scene_id` tehát a független mintavételi egység: ha a tesztkészletből új jelenetekre akarunk következtetni, akkor a felosztást jelenetekre, nem képekre kell elvégezni.

Train/validation/test. A *train* a paraméterek illesztésére, a *validation* a modell- és hiperparaméterválasztásra, a *test* a rögzített protokoll végén, egyszeri, független értékelésre szolgál. A GlyphLab split 120, 30, 30 független jelenetet, azaz 240, 60, 60 képet tartalmaz. A két view nem növeli a független jelenetek számát.

Transzformáció. Az a leképezés, amely a nyers rekordból a modell bemenetét állítja elő: $x^{(i)} = T(r^{(i)})$. Ha a transzformáció adaptív paramétereket tanul, például globális standardizálást vagy PCA-alapú redukciót, akkor ezeket csak a train felosztáson illesztjük; a teszt statisztikája nem szivároghat a trainbe. A GlyphLab `corrupted_test.npy` állománya a rögzített split **után** alkalmazott teszt-transzformáció: ugyanazokat a jelenet–nézet párokat tartalmazza, ugyanabban a sorrendben, mint a tiszta teszt, ezért a clean–corrupt összehasonlítás páros; így nem befolyásolja a tanítást.

Adatszivárgás (data leakage). Két fő formája van. Az egyik a *csoportszintű átszivárgás*: ugyanazon scene képei különböző splitbe kerülnek, így a tesztben olyan rejtett jelenet szerepel, amelynek egy másik megfigyelését a train már látta. A másik a *preprocessing-átszivárgás*: a felosztás elvégzése előtt a teljes adaton számolunk statisztikát, alkalmazunk imputációt vagy választunk hiperparamétert. Mindkettő torzítja az általánosítási becslést.

1.3.3 Levezetés: miért nem független két zajos nézet?

A függetlenség formális valószínűségi definícióját az 5.1. Esemény, feltétel és véletlen változó egységben építjük fel; itt a diszkrét esetre szorítkozunk: X és Y akkor független, ha minden x, y értékre $P(X = x, Y = y) = P(X = x)P(Y = y)$. A relatív gyakoriság ezt csak empirikusan becsli, ezért a lemmában eloszlásokkal számolunk. A feltételes függetlenséget csak a megadott szorzatszabály formájában használjuk: adott S mellett $P(X_1 = x_1, X_2 = x_2 \mid S = s) = P(X_1 = x_1 \mid S = s)P(X_2 = x_2 \mid S = s)$. A következő lemmát saját, teljesen diszkrét generatív modellel bizonyítjuk.

Lemma (közös rejtett jelenetből származó nézetek függősége). *Legyen $S \in \{a, b\}$ rejtett jelenet, $P(S = a) = P(S = b) = 1/2$. Minden nézet egy zajos csatornán halad át: ha $S = a$, a nézet a értéket ad 0,8 valószínűséggel és b értéket 0,2 valószínűséggel; ha $S = b$, a szerepek felcserélődnek. A két csatorna zajai adott S mellett függetlenek. Ekkor a két nézet X_1 és X_2 nem független.*

Bizonyítás. Először a peremeloszlást számítjuk. Az S feltételeit összegezve:

$$P(X_1 = a) = 0{,}8 \cdot \tfrac{1}{2} + 0{,}2 \cdot \tfrac{1}{2} = 0{,}5,$$

és ugyanígy $P(X_1 = b) = 0{,}5$. A szimmetria miatt ugyanez igaz X_2-re.

Most az együttes eloszlást írjuk fel. Ha a két csatorna adott S mellett független, akkor a helyes–helyes együttes valószínűség $0{,}8 \cdot 0{,}8$, a hibás–hibás $0{,}2 \cdot 0{,}2$. A két S-érték együttesen:

$$P(X_1 = a, X_2 = a) = \tfrac{1}{2}(0{,}8^2 + 0{,}2^2) = \tfrac{1}{2} \cdot 0{,}68 = 0{,}34.$$

Ha X_1 és X_2 függetlenek lennének, akkor a peremek szorzatának kellene teljesülnie:

$$P(X_1 = a)P(X_2 = a) = 0{,}5 \cdot 0{,}5 = 0{,}25.$$

Mivel $0{,}34 \neq 0{,}25$, a két nézet nem független.

A bizonyításban a feltételes függetlenség és a perem-összegzés szerepét érdemes külön látni: adott S mellett a két zajdobás független, de a közös rejtett S miatt a megfigyelések összekapcsolódnak. Ha a csatorna hibája 0 lenne, az együttes tömeg $0{,}5 \cdot (1^2 + 0^2) = 0{,}5$ volna, tehát maximális függést kapnánk. Ha a csatorna teljesen zajos lenne, vagyis a kimenet független lenne S-től, akkor a nézetek is függetlenné válnának; a függés forrása tehát a közös jel.

1.3.4 Kézi kis példa

Ugyanezt a generatív modellt egy háromjelenetes, kétnézetes miniatűr adatbázison szemléltetjük, hogy a csoportos split feltétele látható legyen.

scene_id	view_id	megfigyelt nézetérték (a lemma csatornája)
1	1	a
1	2	a
2	1	b
2	2	b
3	1	a
3	2	b

A táblázat a lemma bináris (a/b) csatornáját szemlélteti; a GlyphLab tényleges címkéje ezzel szemben jelenetszintű, egész értékű osztályindex $y \in \{0, 1, 2\}$, amelyet a két nézet közösen hordoz, nem nézetenként változó megfigyelés. A fenti táblában a scene 1 és 2 nézetei determinisztikusan azonosak, a scene 3 nézetei zajosak. Ha képszintű felosztást készítenénk, például a páratlan sorokat trainbe, a párosakat testbe tennénk, akkor a scene 1 és 2 egyik nézete a trainben, másik a testben lenne; a tesztben szereplő kép a trainből ismert jelenetet mutatná. A csoportos split ezzel szemben teljes scene-eket oszt be: az egyik lehetséges érvényes kimenet train = {scene 1, scene 2}, test = {scene 3}. Ekkor a teszt minden képe olyan jelenetből származik, amelyet a modell a trainben nem látott.

1.3.5 Labor: scene_id alapú splitellenőrzés

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Input és szerződés. A munkakönyvtár a `lab/`; a bemenet `data/metadata.csv`, amelynek oszlopai `index`, `scene_id`, `view_id`, `label`, `split`, `xmin`, `ymin`, `xmax`, `ymax`. A `scene_id` szöveges azonosító (`G0-0000` alakú), a `view_id`, a `label` és a dobozkoordináták `int64`, a `split` string értékű (`train`, `val`, `test`). A labor nem tölti be a képtensort, nem használ GPU-t, és képi tenzorokon nem végez lebegőpontos műveletet; a valószínűségszámítás egyetlen valós osztást használ, a dtype-egyezés pedig a CSV-ből olvasott oszlopokon értendő.

Futtatási parancs. A `lab/` könyvtárból:

```
python n003_split_check.py
```

Kód.

```
from __future__ import annotations

import math
import numpy as np
import pandas as pd
from pathlib import Path

DATA_DIR = Path("data")
meta = pd.read_csv(DATA_DIR / "metadata.csv")
# A metadata `index` oszlopa a képtömb sorindexe; a CSV fájlsorrend nem köt.
assert list(meta.columns) == ["index", "scene_id", "view_id", "label", "split",
                              "xmin", "ymin", "xmax", "ymax"], meta.columns
assert len(meta) == 360, len(meta)
assert meta["index"].tolist() == list(range(len(meta)))

def violating_scenes(df: pd.DataFrame, split_col: str) -> list[str]:
    """Azon scene_id-k, amelyek több különböző splitben szerepelnek."""
    nunique = df.groupby("scene_id")[split_col].nunique()
    return nunique[nunique > 1].index.tolist()

def scene_counts(df: pd.DataFrame, split_col: str) -> dict[str, int]:
    return df.groupby(split_col)["scene_id"].nunique().to_dict()

# 1. Jó split ellenőrzése a gyártó szerint
bad_good = violating_scenes(meta, "split")
counts_good = scene_counts(meta, "split")
print("Jó split:")
print("  split-számláló scene-ekre:", counts_good)
print("  szabálytalan scene-ek száma:", len(bad_good))
assert bad_good == [], bad_good
assert counts_good == {"train": 120, "val": 30, "test": 30}, counts_good

# 2. Szándékosan hibás, képszintű split
rng = np.random.default_rng(12345)  # a hibás split seedje
idx = rng.permutation(len(meta))
n_train, n_val = 240, 60
bad_split = np.full(len(meta), "unused", dtype="<U6")
bad_split[idx[:n_train]] = "train"
bad_split[idx[n_train:n_train + n_val]] = "val"
bad_split[idx[n_train + n_val:]] = "test"

meta_bad = meta.copy()
meta_bad["split_bad"] = bad_split
bad_bad = violating_scenes(meta_bad, "split_bad")
```

```
counts_bad = scene_counts(meta_bad, "split_bad")
print("Hibás split:")
print("  split-számláló scene-ekre:", counts_bad)
print("  szabálytalan scene-ek száma:", len(bad_bad))
assert len(bad_bad) > 0, "a képszintű splitnek szabálytalan scene-t kell adnia"

# 3. Várható szabálytalan scene-szám a hibás splithez
C = math.comb  # Python 3.8+: pontos egész binomiális együttható
p_same = (C(240, 2) + C(60, 2) + C(60, 2)) / C(360, 2)
print(f"Egy scene együtt maradásának valószínűsége: {p_same:.4f}")
print(f"Várható szabálytalan scene-szám: {180 * (1 - p_same):.2f}")
assert f"{180 * (1 - p_same):.2f}" == "90.25"
```

A fenti kód tényleges kimenete (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, egy szál; a laborcsomag lockolt profilja NumPy 2.3.5-öt és pandas 2.3.3-at rögzít; a hallgató a saját futásának kimenetét naplózza; a `data/metadata.csv` SHA-256 lenyomata `5acc5c5d4c6e1866b30519c84c35c895b60b499feff12c680bfbaf02fb5c1b9b`):

```
Jó split:
  split-számláló scene-ekre: {'test': 30, 'train': 120, 'val': 30}
  szabálytalan scene-ek száma: 0
Hibás split:
  split-számláló scene-ekre: {'test': 53, 'train': 160, 'val': 55}
  szabálytalan scene-ek száma: 88
Egy scene együtt maradásának valószínűsége: 0.4986
Várható szabálytalan scene-szám: 90.25
```

Mért eredmény és értelmezés. A jó splitre a `violating_scenes` üres lista, és a scene-számláló mindhárom kulcsra az előírt értéket adja (`train`: 120, `val`: 30, `test`: 30), ahogyan a GlyphLab szerződése rögzíti; a fenti kimenetben a pandas a szótárat kulcs szerint rendezve írja ki (`test`, `train`, `val` sorrendben), ami nem változtat a számok jelentésén. A szándékosan hibás felosztásban ezzel szemben a képeket a `scene_id` figyelmen kívül hagyásával permutáltuk, ezért sok scene két view-ja különböző splitbe kerül.

A hibás splithez tartozó várható szabálytalan scene-szám levezethető elemi kombinatorikával. Egy rögzített scene két view-ja akkor marad együtt, ha mindkettő ugyanabba a szegmensbe esik:

$$p_{\text{együtt}} = \frac{\binom{240}{2} + \binom{60}{2} + \binom{60}{2}}{\binom{360}{2}} = \frac{32220}{64620} \approx 0{,}4986.$$

Ezért a várhatóan szabálytalan scene-ek száma

$$180 \cdot (1 - p_{\text{együtt}}) \approx 90{,}25.$$

A mért futásban a `12345` seeddel a szabálytalan scene-ek száma 88, a levezetett várható érték 90,25; a szabálytalan scene-ek száma közelítőleg binomiális eloszlású, szórása $\sqrt{180 \cdot 0{,}5014 \cdot 0{,}4986} \approx 6{,}7$, ezért a seedtől függő eltérés néhány egységnyi. Az ellenőrzés kritériuma az, hogy a hibás splitre pozitív, a jóra nulla legyen, amit a kód `assert`-jei rögzítenek. Ez a labor éppen a csoportos split feltételét teszteli: a jó felosztásban soha nem lép át scene, a hibásban a mért futásban 180-ból 88 jelenet igen.

1.3.6 Feladatok

F1 fogalmi (1-es nehézség). Határozza meg a `scene` és a `view` különbségét, és indokolja három mondatban, miért nem helyettesítheti a 360 képet 360 független jelenetnek. Elvárt eredmény: pontos fogalmi megkülönböztetés és a közös rejtett okra hivatkozó indoklás.

F2 levezetési (2-es nehézség). Tekintse az $S \in \{0, 1, 2\}$ egyenletes eloszlású rejtett jelenetet. Az egyes nézetek az S értékét 0,7 valószínűséggel helyesen adják, a másik két értékre egyenként 0,15–0,15 valószínűséggel tévednek; a két nézet adott S mellett független. Számítsa ki $P(X_1 = X_2)$ értékét, és mutassa meg, miért nem független a két nézet. Elvárt eredmény: levezetett 0,535 és a függetlenségi összehasonlítás 1/3-dal.

F3 implementációs (2-es nehézség). Írjon `split_integrity(df, split_col)` függvényt, amely visszaadja azon `scene_id`-k listáját, amelyek több különböző splitben szerepelnek. Tesztelje a `data/metadata.csv`-n a `split` oszlopra, majd egy mesterségesen megcserélt `split_bad` oszlopra. Elvárt eredmény: futtatható kód, üres lista a jó, nem üres a rossz oszlopra.

F4 hibakeresési (2-es nehézség). Az alábbi kód képszintű splitet készít. Azonosítsa a hibát, írja le a teszt-accuracyra gyakorolt következményét, és adja meg a scene-szintű javítást, valamint annak ellenőrzését.

```
import numpy as np

def split_by_view(meta, rng):
    meta = meta.copy()
    idx = rng.permutation(len(meta))
    split = np.full(len(meta), "train", dtype="<U5")
    split[idx[240:300]] = "val"
    split[idx[300:]] = "test"
    meta["split"] = split
    return meta
```

Elvárt eredmény: a hiba lokalizálása, a torzítás irányának indoklása, valamint a javított függvény és a `violating_scenes` ellenőrzés futási kritériuma.

F5 kísérlettervezési/transzfer (3-as nehézség). A GlyphLab bővített változatában jelenetenként 10 time-lapse nézetet készítünk, és a felhasználási cél ismeretlen jelenetek osztályozása. Tervezzen protokollt: mi a megfigyelési egység, hogyan választja szét a train/validation/test készleteket, mi legyen a baseline, és milyen metrikával, illetve ellenőrzéssel mutatná ki az esetleges szivárgást? Elvárt eredmény: egy lehetséges terv, annak korlátjaival; alternatív terv megadása esetén az eltérő döntés indoklása.

1.3.7 Megoldások és indoklás

F1 megoldás. A scene a közös rejtett geometriai elrendezés, a view ennek egy zajos képi vetülete. Két view ugyanarról a scene-ről nem cserélhető fel két független jelenettel, mert az egyik nézet alapján a másikra jobb előrejelzés adható, mint a peremeloszlásból. Ezért a független mintavételi egység a scene; a képek száma csak a modellbemenetek számát adja, nem a statisztikai bizonyíték erejét.

F2 megoldás. Adott $S = s$ mellett a két csatorna azonos válaszának valószínűsége az esélyek négyzetösszege:

$$q = 0{,}7^2 + 0{,}15^2 + 0{,}15^2 = 0{,}49 + 0{,}0225 + 0{,}0225 = 0{,}535.$$

Mivel q nem függ s-től,

$$P(X_1 = X_2) = 0{,}535.$$

A peremeloszlás egyenletes marad: tetszőleges s-re

$$P(X_1 = s) = \tfrac{1}{3} \cdot 0{,}7 + 2 \cdot \tfrac{1}{3} \cdot 0{,}15 = \tfrac{1}{3}.$$

Ha a két nézet független lenne, akkor $P(X_1 = X_2) = \sum_{s=0}^{2} (1/3)^2 = 1/3$ volna. Mivel $0{,}535 \neq 1/3$, a nézetek nem függetlenek; a közös rejtett S az együttes eloszlást a peremek szorzatától eltéríti.

F3 megoldás. A feladat `split_integrity` függvénye ugyanazt a csoportosítást végzi, mint a laborbeli `violating_scenes`; a megoldásban önállóan definiáljuk, majd a labor adatán teszteljük. Nem elég teljes jelenetek splitjét megváltoztatni: az nem sérti a csoportszintű feltételt. A teszthez két kiválasztott jelenetnek csak **egy-egy nézetét** kell az eredetitől eltérő splitbe tenni:

```
def split_integrity(df, split_col):
    """Azon scene_id-k, amelyek több különböző splitben szerepelnek."""
    grouped = df.groupby("scene_id")[split_col].nunique()
    return grouped[grouped > 1].index.tolist()

meta_test = meta.copy()
meta_test["split_bad"] = meta_test["split"]
selected = meta_test["scene_id"].unique()[:2]
for sid in selected:
    row_id = meta_test.index[meta_test["scene_id"] == sid][0]
    old_split = meta_test.loc[row_id, "split_bad"]
    meta_test.loc[row_id, "split_bad"] = (
        "test" if old_split != "test" else "train"
    )

assert split_integrity(meta, "split") == []
bad = split_integrity(meta_test, "split_bad")
assert set(bad) == set(selected) and len(bad) == 2
```

Az ellenőrzés típusa halmaz-egyezés, nem lebegőpontos tolerancia: a jó oszlopra a lista üres, a hibásra pontosan a két módosított `scene_id` jelenik meg.

F4 megoldás. A hiba az, hogy a `np.random.permutation` a 360 képet függetlenül sorsolja, ezért a `scene_id`-t nem veszi figyelembe: ugyanazon scene két view-ja különböző splitbe kerülhet. Következmény: a testben olyan jelenetek jelennek meg, amelyek egy másik nézete a trainben szerepel; a teszt-accuracy felfelé torzul, mert a modell ismert geometriákra válaszol, nem új jelenetekre.

Javítás: először a scene-ekből kell permutációt készíteni, majd a scene-hez tartozó összes view-t együtt kell beosztani. Egy lehetséges vázlat:

```
def split_by_scene(meta, rng):
    meta = meta.copy()
    scenes = meta["scene_id"].unique()
    perm = rng.permutation(len(scenes))
    split_map = {}
    for s in scenes[perm[:120]]:
        split_map[s] = "train"
    for s in scenes[perm[120:150]]:
        split_map[s] = "val"
    for s in scenes[perm[150:]]:
        split_map[s] = "test"
```

```
    meta["split_fixed"] = meta["scene_id"].map(split_map)
    return meta
```

Ellenőrzés: a `violating_scenes(meta, "split_fixed")` eredményének üresnek, a scene-számlálónak pedig `train: 120, val: 30, test: 30` értékűnek kell lennie.

F5 megoldásminta. A megfigyelési egység továbbra is a scene, mert a time-lapse sorozat tagjai közös jelenetből erednek. A felosztást scene-szintű osztály-rétegzett permutációval végeznénk, például 120/30/30 jelenettel; egy scene összes nézete egy splitbe kerülne. A rétegzés osztályonként 40 train, 10 validation és 10 test jelenetet ad (három osztályra $3 \cdot 40 + 3 \cdot 10 + 3 \cdot 10 = 120 + 30 + 30$); a jelenetenkénti nézetszám a splitet nem befolyásolja:

```
def stratified_scene_split(meta, rng):
    meta = meta.copy()
    scenes = meta.groupby("scene_id")["label"].first()
    split_map = {}
    for _, group in scenes.groupby(scenes):
        ids = group.index.to_numpy()[rng.permutation(len(group))]
        split_map.update({sid: "train" for sid in ids[:40]})
        split_map.update({sid: "val" for sid in ids[40:50]})
        split_map.update({sid: "test" for sid in ids[50:]})
    meta["split"] = meta["scene_id"].map(split_map)
    return meta
```

A `violating_scenes` ellenőrzés itt is üres listát ad, a scene-számláló pedig `train: 120, val: 30, test: 30`; emellett az osztályonkénti scene-szám a split mindhárom részében 40/10/10. Baseline-ként egy olyan modellt választanánk, amely a train-osztályok relatív gyakoriságát jósolja, vagy egy korábbi kis CNN-t, mert a kérdés az, hogy a jeleneten belüli plusz nézetek javítják-e az új jelenetekre való általánosítást.

Metrika: scene-szintű átlagos accuracy és per-kép accuracy, valamint páros ellenőrzés, hogy a testben nem szerepel olyan scene, amelynek bármely view-ja a trainben vagy a valben van. A szivárgás detektálására a `split_integrity` függvényt futtatnánk a végső `metadata.csv`-n. Fontos korlát: jelenetenként pontosan 10 érvényes nézet mellett a per-kép accuracy számtanilag egyenlő a scene-szintű átlaggal, mert minden jelenet 10 képpel szerepel; a nézetek azonban nem függetlenek, ezért a per-kép accuracy körüli bizonytalanságot nem a képszám (1800), hanem az effektív mintanagyság, a 180 jelenet alapján kell becsülni. Ezért a fő metrika a scene-szintű átlag, a bizonytalanságot pedig jelenetszintű bootstrap adja; ha egyes jeleneteknél hiányzik néhány nézet, a per-kép átlag már nem egyezik a scene-szintűvel.

1.3.8 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: *a több kép több független megfigyelést jelent.* A GlyphLab ellenpéldája: a 360 kép 180 jelenetből áll; ha képszintű splitet készítünk, várhatóan körülbelül 90 jelenetnek az egyik nézete a trainbe, a másik a testbe kerül. A teszt-accuracy ekkor olyan jeleneteken is mér, amelyeket a modell a trainből részben ismer; a magas érték nem tekinthető az ismeretlen jelenetre való általánosítás bizonyítékának, szivárgásra utalhat. A képek darabszáma tehát a számítási költséget és a modellbemenetek számát jellemzi, nem a független bizonyítékok számát.

1.3.9 Önellenőrzés és visszatekintés

1. Tudja-e definiálni a csoportot anélkül, hogy a `view_id`-t használná?
2. Miért üres a jó splitre a `violating_scenes` függvény visszatérési listája?
3. Mi a különbség a csoportszintű és a preprocessing-átszivárgás között?

4. Miért nem elegendő a teszt-accuracy magas értéke a helyes split igazolására?

Az 1.2. Mitől egyetemi szintű egy modellezési állítás? áttekintése: az empirikus állítás típusa megköveteli az adatot, a splitet és a számítási protokollt. A most bevezetett `scene_id` azt a hiányzó protokoll-összetevőt adja meg, amely nélkül a pontossági számítás nem minősülhet megalapozott empirikus állításnak.

1.3.10 Rövid összefoglaló

A megfigyelési egység nem mindig azonos a modell bemenetével: a GlyphLabban a bemenet a kép, de a független egység a `scene_id`. A GlyphLabban ezért 180 független megfigyelési egység van, nem 360; a 360 kép két-két nézete ugyanahhoz a jelenethez tartozik. A train/validation/test felosztás érvényességi feltétele, hogy egy csoport minden rekordja ugyanabba a splitbe kerüljön; ennek megsértése csoportszintű adatszivárgást okoz. A zajos nézetek függőségét diszkrét generatív modellel bizonyítottuk: a közös rejtett jelenet miatt az együttes eloszlás nem a peremek szorzata. A labor a jó splitre nulla, a képszintű hibás splitre a mért futásban 88 szabálytalan jelenetet mutat. A feladatok a fogalmi megkülönböztetést, a levezetést, az ellenőrző kódot, a hibás felosztás javítását és egy time-lapse transzferprotokollt fedik le.

Továbblépés. Az 1.4. Tanulási diagnózis és kísérleti napló egységben a tanulási diagnózist és a kísérleti naplót építjük fel; a mostani split-ellenőrzés lesz a napló első kötelező rovata.

Források

- A GlyphLab adatszerződést a függelék A. A GlyphLab adatkészlet szakasza rögzíti; a jelen egység kimenetei a rögzített környezetben futtatott kód tényleges kimenetéből és a levezetett várható értékből származnak.

1.4 Tanulási diagnózis és kísérleti napló

1.4.1 Cél és induló kérdés

Ez az egység diagnózis, nem új elméleti fejezet: a hallgató a saját programozói előismeretét méri fel kézi számpéldán és egy kis laboron. Az előfeltétel az 1.3. Adatfelosztás és a megfigyelési egység fogalomkészlete: train/validation/test, `scene_id` mint megfigyelési egység, csoportos split és adatszivárgás; az adatkészlet szerződését a függelék A. A GlyphLab adatkészlet szakasza rögzíti. A cél az, hogy a hallgató az első változatát rögzítse, és a hibák alapján célzott alapozó egységet válasszon a későbbi fejezetekből.

Induló kérdés. Adott a $v = (3, 4)^\top$ vektor. Mekkora az euklideszi hossza, és mi a mértékegysége, ha a komponensek pixelben adottak? Előzetes becslés nélkül írja le a válaszát; a diagnózisnak éppen ez a nyers első változat a tárgya.

1.4.2 Fogalmak, jelek és mértékegységek

Vektor. A d-dimenziós valós vektor $x = (x_1, \dots, x_d)^\top \in \mathbb{R}^d$; oszlopvektorként írjuk az absztrakt képletekben. Az euklideszi norma

$$\|x\| = \sqrt{\sum_{i=1}^{d} x_i^2}.$$

A norma csak akkor fizikailag értelmes, ha a komponenseknek közös mértékegysége van: ha minden $[x_i] = u$, akkor $[\|x\|] = u$. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab intenzitásvektorainál a komponensek dimenzió nélküli $[0, 1]$ értékek, tehát a norma is dimenzió nélküli.

Függvény. Az $f : D \to \mathbb{R}$ hozzárendelés egy $D \subseteq \mathbb{R}$ definíciós tartomány minden u eleméhez egy $f(u) \in \mathbb{R}$ értéket rendel. A bemenet egységét $[u]$, a kimenetét $[f]$ jelöli; a kettő általában különbözik. Az $f(u) = 2u - 1$ affin kifejezésnél, ha $[u] = \text{mm}$ és az eredmény pixelben értendő, akkor az eredmény egysége px, a konstans -1 egysége is px, a 2 együttható egysége pedig px/mm.

Átlag. Az $x = (x_1, \dots, x_N)$ valós minta átlaga

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i, \qquad N \geq 1.$$

A reduction explicit: az összegzés mind az N elemet fedi, az osztás N-nel történik. Az üres minta ($N = 0$) átlaga nem értelmes. A mértékegység azonos a minta egységével: $[\bar{x}] = [x_i]$.

Deriváltintuíció. A differenciahányados

$$D_h f(t) = \frac{f(t + h) - f(t)}{h}, \qquad h \neq 0,$$

a t és $t + h$ közötti szelő meredeksége. A derivált ennek határértéke $h \to 0$ mellett; a formális definíciót a 4.1. Függvény, határérték és derivált, a láncszabályt pedig a 4.3. Jacobi-mátrix, láncszabály és VJP építi fel. Itt csak az intuíciót mérjük. A szimmetrikus változat

$$S_h f(t) = \frac{f(t + h) - f(t - h)}{2h}, \qquad h \neq 0,$$

gyakran jobb numerikus közelítés; ezt használjuk majd a gradiensellenőrzésben is. A mértékegység $[S_h f] = [f]/[t]$.

Tenzor. A tenzor többdimenziós számrendezés; a könyv ajánlott tanulási útvonalán a képbatch $X \in \mathbb{R}^{N \times C \times H \times W}$, NCHW sorrendben: batch, csatorna, magasság, szélesség. A dimenziók darabszámok, ezért dimenzió nélküliek. Az elemszám a dimenziók szorzata. Például $X \in \mathbb{R}^{2 \times 1 \times 24 \times 24}$ elemszáma $2 \cdot 1 \cdot 24 \cdot 24 = 1152$; a dtype (például `float32`) a tárolás pontosságát rögzíti.

Seed. A pszeudovéletlen generátor egy s seedből (magból) egy $(u_1, u_2, \dots)$ sorozatot állít elő. A seed mértékegysége nincs, általában nemnegatív egész. Ugyanaz a seed csak ugyanazon algoritmus, verzió és kiértékelési sorrend mellett ígér azonos sorozatot; a seed önmagában nem hordozhatósági garancia [D02].

Futásazonosító. A futásazonosító (ρ, run id) egy konkrét kísérleti végrehajtást címkéző karakterlánc. Nem azonos a seeddel: a seed egy generátor bemenete, a futásazonosító a napló kulcsa, amely a seed, az adatverzió, a kódverzió és az időbélyeg mellé kerül. A kettő felcserélése megnehezíti annak megállapítását, hogy két eltérő eredmény közül melyik tartozik ugyanahhoz a beállításhoz.

1.4.3 Három feladat teljes specifikációja

A bizonyítási feladat: minden jelre, tartományra és mértékegységre kiterjedő specifikációt adunk három feladatra.

F-A: vektornorma. Jelek: $d = 2$, $x = (3, 4)^\top \in \mathbb{R}^2$, $\|x\| \in \mathbb{R}_{\geq 0}$. Tartomány: a komponensek közös egysége u; itt $u = \text{px}$, tehát $[\|x\|] = \text{px}$. Számítás: $\|x\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$ px.

F-B: átlag. Jelek: $N = 3$, $x = (1.0, 3.0, 5.0)$, $\bar{x} \in \mathbb{R}$. Tartomány: az értékek pixelintenzitások, dimenzió nélküliek. Számítás: $\bar{x} = \frac{1}{3}(1.0 + 3.0 + 5.0) = 3.0$, dimenzió nélkül.

F-C: differenciahányados. Jelek: $f(t) = t^2$, $t_0 = 2$, $h = 0.01$, $S_h f(t_0) \in \mathbb{R}$. Tartomány: t idő ($[t] = \text{s}$), f hossz ($[f] = \text{m}$). Dimenziósan helyesen $f(t) = a\,t^2$ alakban írjuk, ahol $a = 1$ m/s^2; így $[a\,t^2] = \text{m}$, és $[S_h f] = \text{m/s}$. Az $f(t) = t^2$ írásmód ennek rövidítése: t a másodpercben, f a méterben vett numerikus érték. Számítás: a lenti segédszámítás adja $S_h f(t) = 2t$ minden $h \neq 0$-ra; $t_0 = 2$-nél az eredmény 4 m/s.

Segédszámítás (másodfokú eset). Ha $f(t) = t^2$ és $h \neq 0$, akkor $S_h f(t) = 2t$ minden $t \in \mathbb{R}$ esetén.

Levezetés. A definícióból:

$$S_h f(t) = \frac{(t+h)^2 - (t-h)^2}{2h} = \frac{(t^2 + 2th + h^2) - (t^2 - 2th + h^2)}{2h} = \frac{4th}{2h} = 2t.$$

Az első egyenlőség a definíció, a második a binomiális kifejtés, a harmadik az összevonás, a negyedik a $h \neq 0$ feltétel melletti egyszerűsítés.

A formula érvényességi köre. A másodfokú formula nem áll harmadfokú függvényre: ha $g(t) = t^3$, akkor $S_h g(t) = 3t^2 + h^2$, ami $h \neq 0$ esetén nem egyenlő a $3t^2$ deriválttal. A $h \to 0$ határátmenet az, amit a későbbi kalkulusfejezet formalizál; a segédszámítás tehát a másodfokú alak következménye, nem általános simasági állítás.

1.4.4 Kézi példa

Az F-A, F-B, F-C feladatokat kézzel ellenőrizzük. F-A: 5 px. F-B: 3.0. F-C: $S_h f(2) = \frac{2.01^2 - 1.99^2}{0.02} = \frac{4.0401 - 3.9601}{0.02} = \frac{0.08}{0.02} = 4.0$ m/s. Az alakellenőrzés: a norma és az átlag skalár; a differenciahányados egysége az elvárt m/s.

A tenzorfeladat: $X \in \mathbb{R}^{2\times 1\times 24\times 24}$ elemszáma 1152, egész szám, dimenzió nélkül. A seed-feladat nem numerikus, hanem a labor naplójába mentett indoklás.

1.4.5 Labor: első változat és napló

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor a hallgatótól öt rövid feladat válaszát kéri be, ezeket a scriptbe épített `EXPECTED` értékekkel veti össze `float64` toleranciával, elmenti az első változatot egy append-only JSON-naplóba, és rögzíti a választott alapozó egységet. A program nem oldja meg a feladatokat: a hallgató számolja ki az öt értéket a kért sorrendben: (1) az F-A vektornorma, (2) az $f(u) = 2u - 1$ függvény értéke $u = 3$-nál, azaz $f(3)$, (3) az F-B átlag, (4) az F-C szimmetrikus differenciahányados $t_0 = 2$-nél, (5) a tenzor elemszáma. Munkakönyvtár: a kurzusprojekt gyökere; a napló a `lab_n004/` könyvtárba

kerül. A futás CPU-n történik; a labor nem generál véletlent, a GlyphLab generáló seedje (20261020) csak a napló szerződésrovatában szerepel.

```
#!/usr/bin/env python3
"""n004 diagnosztikai labor: első változat mentése és napló."""
import argparse
import hashlib
import json
import os
import sys
from datetime import datetime, timezone

EXPECTED = {
    "norm": {"value": 5.0, "tol": 1e-6},
    "function": {"value": 5.0, "tol": 1e-6},
    "mean": {"value": 3.0, "tol": 1e-6},
    "derivative": {"value": 4.0, "tol": 1e-5},
    "tensor_elements": {"value": 1152, "exact": True},
}

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--answers", nargs=5, type=float, required=True,
                    help="sorrend: vektornorma, f(3), átlag, "
                         "szimmetrikus differenciahányados, tensor-elemszám")
    ap.add_argument("--basis", required=True,
                    help="választott célzott alapozó egység, pl. n009")
    ap.add_argument("--seed-note", required=True,
                    help="magyarázat: miért nem elég a seed önmagában")
    ap.add_argument("--output-dir", default="lab_n004")
    args = ap.parse_args()

    os.makedirs(args.output_dir, exist_ok=True)
    log_path = os.path.join(args.output_dir, "log.json")
    first_version_kept = not os.path.exists(log_path)
    if not first_version_kept:
        print(f"Hiba: {log_path} már létezik. "
              "A napló append-only; ne írd felül az első változatot.")
        return 2

    tensor_float = args.answers[4]
    if abs(tensor_float - round(tensor_float)) > 1e-9:
        print("Hiba: a tensor-elemszám egész szám legyen.")
        return 2
    answers = {
        "norm": args.answers[0],
        "function": args.answers[1],
        "mean": args.answers[2],
        "derivative": args.answers[3],
        "tensor_elements": int(round(tensor_float)),
    }
    assert set(answers) == set(EXPECTED)

    results = []
    for key, spec in EXPECTED.items():
        got = answers[key]
        if spec.get("exact"):
            err = abs(got - spec["value"])
            ok = got == spec["value"]
        else:
            err = abs(got - spec["value"])
            ok = err <= spec["tol"]
        results.append({
```

```
            "task": key,
            "expected": spec["value"],
            "got": got,
            "error": err,
            "ok": ok,
            "tolerance": spec.get("tol", 0),
        })

    now_dt = datetime.now(timezone.utc)
    now = now_dt.isoformat()
    stamp = now_dt.strftime("%Y%m%dT%H%M%S")
    with open(__file__, "rb") as fh:
        code_sha8 = hashlib.sha256(fh.read()).hexdigest()[:8]
    config = {
        "answers": answers,
        "chosen_basis_unit": args.basis,
        "seed_note": args.seed_note,
        "code_sha8": code_sha8,
    }
    material = now + json.dumps(config, sort_keys=True)
    run_id = "run-" + stamp + "-" + hashlib.sha256(material.encode()).hexdigest()[:8]

    record = {
        "unit": "n004",
        "run_id": run_id,
        "code_sha8": code_sha8,
        "timestamp": now,
        "seed": None,
        "data_version": "nincs külső adat (a labor nem olvas fájlt)",
        "device": "cpu",
        "dtype": "float64 numerikus ellenőrzés; tensor-elemszám int",
        "glyphlab_seed_note":
            "GlyphLab-generáló seed 20261020; ez a labor nem generál véletlent",
        "diagnosis_not_grading": True,
        "answers": answers,
        "results": results,
        "all_ok": all(r["ok"] for r in results),
        "chosen_basis_unit": args.basis,
        "seed_note": args.seed_note,
        "first_version_kept": first_version_kept,
    }
    with open(log_path, "w", encoding="utf-8") as fh:
        json.dump(record, fh, ensure_ascii=False, indent=2)

    print("run_id:", run_id)
    print("log:", log_path)
    print("results:")
    for r in results:
        print(f"  {r['task']:>15s}: ok={r['ok']}, error={r['error']:.3g}")
    print("all_ok:", record["all_ok"])
    print("chosen_basis_unit:", args.basis)
    return 0 if record["all_ok"] else 1

if __name__ == "__main__":
    sys.exit(main())
```

Futtatási parancs, ha a hallgató mind az öt feladatot helyesen oldotta meg, és például a vektoros alapozást választja:

```
python n004_diagnosis.py --answers 5 5 3 4 1152 --basis n009 \
  --seed-note "A seed csak a generátor kezdőállapota; ugyanaz a seed \
más NumPy-verzióban vagy más algoritmussal eltérő sorozatot adhat."
```

A következő kimenet egy tényleges futásból származik (Python 3.12.3, CPU, egy szál; a script csak a standard könyvtárat használja, véletlent nem generál, ezért seed nem szerepel). A `run_id` a futás UTC időbélyegét (másodperc pontossággal) és a beállítás hash-ének első nyolc jegyét tartalmazza; a hash a válaszok mellett a választott alapozó egységet, a seed-megjegyzést és a script SHA-256-jának első nyolc jegyét fedi, ezért újrafuttatáskor más lesz:

```
run_id: run-20261030T094959-a4251020
log: lab_n004/log.json
results:
             norm: ok=True, error=0
         function: ok=True, error=0
             mean: ok=True, error=0
       derivative: ok=True, error=0
  tensor_elements: ok=True, error=0
all_ok: True
chosen_basis_unit: n009
```

Ha a fájl már létezik, a script `exit code 2`-vel megáll, és nem írja felül az első változatot. Ez a napló append-only/első-változat elve: a fejlődést csak úgy lehet később mérni, ha a diagnózis nyers állapota rögzítve marad. Az eredményértelmezés a hibás feladatokhoz kötött:

Hibás feladat	Javasolt első alapozó egység
norm	3.1. Vektor, norma és skalárszorzat
function	3.2. Mátrixszorzás és affín réteg vagy 4.1. Függvény, határérték és derivált
mean / tensor_elements	2.1. Python, NumPy és tenzoralak
derivative	4.1. Függvény, határérték és derivált

A választás nem büntetés, hanem útvonaltervezés: aki a differenciahányadost eltéveszti, annak érdemes a kalkulusfejezetnél lassabban haladnia, de a diagnózis ettől nem minősíti a kurzusbeli kilátásait.

1.4.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a seed és a futásazonosító különbségét, és adjon egy konkrét helyzetet, amelyben ugyanaz a seed két környezetben eltérő sorozatot adhat. Beadandó: a két definíció saját szavakkal, egy helyzet és egy mondat arról, hogy a naplóban miért kell mindkettő.

F2 (levezetési, nehézség 1). Számítsa ki az $f(t) = 3t^2 - 2t$ függvény $t = 1$ pontjához tartozó szimmetrikus differenciahányadost $h = 0.1$ mellett. Adja meg minden lépését, az eredmény mértékegységét és a 3, illetve 2 együttható mértékegységét, ha $[t] = \mathrm{s}$ és $[f] = \mathrm{m}$. Beadandó: a teljes levezetés és a mértékegység-ellenőrzés.

F3 (implementációs, nehézség 2). Írjon Python-függvényt, amely egy valós lista euklideszi normáját és átlagát számítja; üres listára a függvény `ValueError`-t dobjon. Beadandó: teljes kód, három teszteset ($v = (3, 4)$, $x = (1, 3, 5)$, üres lista) és a tolerancia.

F4 (hibakeresési, nehézség 2). Az alábbi függvény hibás:

```python
def mean(x):
    return sum(x) / (len(x) - 1)
```

Lokalizálja a hibát, adja meg a következményét egy egyelemű és egy kételemű bemeneten, javítsa, és ellenőrizze a javítást teszttel. Beadandó: ok, következmény, javított kód, teszteredmény.

F5 (kísérleti/transzfer, nehézség 3). Tervezze meg egy GlyphLab-osztályozó kísérlet naplójának rovatszerkezetét JSON-ban. A terv tartalmazza a run id-t, seedet, adatverziót, a `scene_id` szerinti csoportos split ellenőrzését, a választási protokollt (best checkpoint val loss alapján) és a záróteszt helyét. Indokolja, miért nem a test metrika alapján választ modellt. Beadandó: séma, kitöltési szabály és az érvényességi korlát egy bekezdésben.

1.4.7 Megoldások és indoklás

F1. A seed a pszeudovéletlen generátor bemenő kezdőállapota; a futásazonosító a kísérleti napló egy rekordját címkéző egyedi karakterlánc. Helyzet: `np.random.seed(42)` és `np.random.default_rng(42)` más algoritmus miatt eltérő sorozatot ad; ugyanígy PyTorch CPU és CUDA kernelváltozata is eltérhet. A naplóban a seed önmagában nem azonosítja a futást, mert a kódverzió vagy az adatverzió változása mellett ugyanaz a seed más eredményhez tartozhat; a run id ezért a teljes beállítás címkéje. A labor szkriptjében a run id az időbélyeg mellett a válaszokat, a választott alapozó egységet, a seed-megjegyzést és a script SHA-256-jának első nyolc jegyét fedi; külön seed és adatverzió nincs, mert a labor nem generál véletlent és nem olvas külső adatot, ezt a napló `seed: null` és `data_version` mezője rögzíti.

F2. A szimmetrikus hányados:

$$S_h f(1) = \frac{f(1.1) - f(0.9)}{0.2}.$$

Kiszámítva: $f(1.1) = 3(1.21) - 2.2 = 3.63 - 2.2 = 1.43$, és $f(0.9) = 3(0.81) - 1.8 = 2.43 - 1.8 = 0.63$. Így

$$S_h f(1) = \frac{1.43 - 0.63}{0.2} = \frac{0.8}{0.2} = 4.0.$$

Alternatív algebra: az $f(t) = 3t^2 - 2t$ alakra a segédszámítás mintájára $S_h f(t) = 6t - 2$ adódik, mert a másodfokú tag szimmetrikus hányadosa $3 \cdot 2t = 6t$, a lineáris $-2t$ tagé pedig -2. $t = 1$-nél az eredmény 4.0. A mértékegység $[S_h f] = [f]/[t] = \mathrm{m/s}$. Dimenziósan $f(t) = a\,t^2 - b\,t$ a pontos alak, ahol $a = 3\ \mathrm{m/s^2}$ és $b = 2\ \mathrm{m/s}$: a két együttható egysége különböző, mert mindkét tagnak méterben kell lennie. Ennek megfelelően $S_h f(t) = 2a\,t - b$, ahol mindkét tag egysége m/s. Az alakellenőrzés: a hányados skalár, az egység a számlálóé osztva a nevezőével.

F3.

```
import math

def vector_norm(v):
    if len(v) == 0:
        raise ValueError("üres tömbnek nincs normája")
    return math.sqrt(sum(x * x for x in v))

def mean(v):
    if len(v) == 0:
        raise ValueError("üres tömbnek nincs átlaga")
    return sum(v) / len(v)

# Tesztek
assert abs(vector_norm([3.0, 4.0]) - 5.0) < 1e-12
assert abs(mean([1.0, 3.0, 5.0]) - 3.0) < 1e-12
```

```
for f in (vector_norm, mean):
    try:
        f([])
    except ValueError:
        pass
    else:
        raise AssertionError(f"{f.__name__} üres listára nem dobott ValueError-t")
```

A tolerancia 10^{-12}, mert a megadott bemeneteken (a $(3, 4)$ vektor és az $(1, 3, 5)$ lista) az összeg és a négyzetgyök eredménye float64-ben pontosan ábrázolható; ez a szigorú korlát tehát a konkrét tesztesetekre indokolt, általános valós bemenetre a lebegőpontos kerekítés miatt nagyobb, relatív tolerancia kellene. Az üres lista a szélsőeset: az átlag definíciója $N \geq 1$-re, a norma definíciója $d \geq 1$-re érvényes, ezért a `ValueError` dobása a specifikáció része.

F4. Hiba: a nevező $N - 1$, nem N. Következmény: $N = 1$ esetén nullával osztás (`ZeroDivisionError`); $N = 2$ esetén az eredmény a helyes átlag kétszerese, mert $\frac{x_1+x_2}{1}$ a $\frac{x_1+x_2}{2}$ kétszerese. Javítás: a nevező legyen `len(x)`. Ellenőrző teszt:

```
def mean_fixed(x):
    if len(x) == 0:
        raise ValueError("üres tömbnek nincs átlaga")
    return sum(x) / len(x)

assert mean_fixed([2.0]) == 2.0
assert abs(mean_fixed([1.0, 3.0]) - 2.0) < 1e-12
try:
    mean_fixed([])
except ValueError:
    pass
else:
    raise AssertionError("üres lista nem dobott ValueError-t")
```

A javított függvény a $N = 1$ szélsőesetet is helyesen kezeli, és az üres listára ugyanazt a `ValueError`-t adja, mint a F3 specifikációja; a hiba a korrigálatlan változatban a definíció szerinti normálási konstans tévesztése volt.

F5. Egy lehetséges séma:

```
{
  "run_id": "run-20261030T085412-3f9a1c2b",
  "kerdes": "GlyphLab tájolás alapvonal",
  "data_version": "GlyphLab 1.0 (generáló seed: 20261020)",
  "split_check": "scene_id szerint csoportosítva; train/val/test = 120/30/30 jelenet = 240/60/60
  ↪ kép",
  "seed": 42,
  "model": "GlyphCNN (lab/glyph_models.py, 38.2. fejezet)",
  "protocol": "best checkpoint val loss alapján; a záróteszt egyszer fut",
  "metric": "accuracy",
  "raw_results": null
}
```

Kitöltési szabály: a `seed` a tanítási seed (itt 42), a `data_version` a generáló seedet (20261020) tartalmazza; a kettő nem ugyanaz, és nem cserélhető fel. A `run_id` a futás után időbélyegből és a beállítás hash-éből képzett egyedi kulcs, a fenti érték csak formai példa. A `raw_results` a futás előtt `null`, a futás után a nyers eredményfájl relatív útja kerül ide; a metrikaértéket csak ebből a fájlból szabad átvezetni. A `model` mező a tényleges modellforrásra mutat, nem a később kitalálandó architektúrára.

Indoklás: ha a modellválasztás a test metrika alapján történne, a test halmaz a választási folyamat részévé válna, és a mért pontosság adaptív túlillesztést tartalmazna. A val halmaz a kiválasztásé,

a test a rögzített protokoll végén egyszeri értékelésé. A GlyphLab érvényességi korlátja: a generált alakzatokon mért eredmény nem természetes képi benchmark és nem termelési validáció.

1.4.8 Gyakori tévedés ellenpéldával

A diagnózist gyakran hivatalos felvételiként vagy végleges készségminősítésként értelmezik. Ez tévedés. Ha egy hallgató a differenciahányadost például $S_h f(2) = 2t + h$ értéknek számolja, az nem azt jelenti, hogy a kurzusból kizárt, hanem azt, hogy a határérték-intuíciót a 4.1. Függvény, határérték és derivált egységnél kell elmélyítenie. Ellenpélda a minősítő értelmezésre: két hallgató azonos hibát követ el, de az egyik korábban még soha nem látott deriváltat, a másik pedig elfelejtette; a hibapontszám nem különbözteti meg őket, a kurzus viszont mindkettőjüknek felépíti a hiányzó lépést. A diagnózis érvényes következménye elsősorban a célzott alapozó egység kiválasztása, nem a hallgató osztályozása; a továbblépés feltételét a PASS-szabály (1.4.11) rögzíti, amely a bemutatott bizonyítékot, nem a személyt minősíti.

1.4.9 Matematikai szintfelmérő megoldókulccsal

A 3–5. fejezet előtt oldja meg (kb. 20 perc, segédeszköz nélkül), majd vesse össze a kulccsal; a hibás témához a megjelölt útvonal tartozik.

1. Számolja ki: $\sum_{i=1}^{4} (2i - 1)$. — *Kulcs: 16.* Hiba esetén: 2.1. Python, NumPy és tenzoralak (indexek, összegzés).
2. Legyen $A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$, $x = (3, 1)^\top$. Számolja ki Ax-et. — *Kulcs:* $(5, 1)^\top$. Hiba esetén: 3.2. Mátrixszorzás és affín réteg.
3. Bizonyítsa, hogy ha $Q^\top Q = I$, akkor $\|Qx\| = \|x\|$. — *Kulcs:* $\|Qx\|^2 = x^\top Q^\top Q x = x^\top x$. Hiba esetén: 3.4. Sajátérték, SVD és spektrális határ.
4. Deriválja: $f(x) = x^2 e^x$. — *Kulcs:* $(2x + x^2)e^x$. Hiba esetén: 4.1. Függvény, határérték és derivált.
5. Két érmefeldobás (függetlenül, $p = \frac{1}{2}$): mennyi annak a valószínűsége, hogy pontosan egy fej van? — *Kulcs:* $\frac{1}{2}$. Hiba esetén: 5.1. Esemény, feltétel és véletlen változó.

Az 5-ből legalább 4 helyes válasz javasolt; kevesebb esetén a megjelölt alfejezetet a továbblépés előtt érdemes átdolgozni. Ez diagnosztika, nem felvételi.

1.4.10 Önellenőrzés és visszatekintés

Ellenőrizze: (1) az átlag mértékegysége miért egyezik az adatéval; (2) a t^2-es segédszámítás melyik lépése használja a $h \neq 0$ feltételt; (3) a run id és a seed közül melyiket írjuk a generátornak szánt bemenetként. Válaszok: (1) mert az összeg mértékegysége az adaté, az $1/N$ osztó dimenzió nélküli; (2) a $\frac{4th}{2h}$ egyszerűsítése; (3) a seedet. Visszatekintés az 1.3. Adatfelosztás és a megfigyelési egység alfejezetből: a GlyphLabban a független egység a `scene_id`, ezért a napló `split_check` rovata azt igazolja, hogy egy jelenet minden nézete ugyanabba a splitbe került; a rovat a 120/30/30 jelenetszintű (240/60/60 képszintű) darabszámot rögzíti. A képszintű felosztás a 180 jelenet mintegy felében, körülbelül 90 jelenetben szakítaná szét a két nézetet (a pontos, 90,25 várható értékű levezetést az 1.3.5 labor adja).

1.4.11 Ajánlott tanulási útvonal és ellenőrző pontok

Ez a szakasz a könyv egészére érvényes, központi szabályokat adja meg. A tíz ellenőrző pont (4.4. I. ellenőrző pont: matematika és kód megfelelése, 8.4. II. ellenőrző pont: korrekt tanulási állítás, …, 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) ezeket a szabályokat nem ismétli meg, hanem a saját blokkja követelményein keresztül alkalmazza.

Az ajánlott tanulási útvonal. A könyv tíz, egyenként négy fejezetből álló blokkra tagolódik; minden blokk a negyedik fejezete utolsó alfejezetében lévő ellenőrző ponttal zárul. A munkaterhelés tervezési becslés, nem mért érték: alfejezetenként 2–4 óra olvasás, feladat és labor, ellenőrző pontonként 6–10 óra.

1. **1–4. fejezet:** szerződés, Python/NumPy, lineáris algebra, derivált és VJP. Előfeltétel: nincs. Ellenőrző pont: 4.4. I. ellenőrző pont: matematika és kód megfelelése. Becsült munka: 40–60 óra.
2. **5–8. fejezet:** valószínűség, optimalizálás, veszteségek, általánosítás. Előfeltétel: az 1. blokk. Ellenőrző pont: 8.4. II. ellenőrző pont: korrekt tanulási állítás. Becsült munka: 40–60 óra.
3. **9–12. fejezet:** perceptron, MLP, backprop, PyTorch-tanítás. Előfeltétel: az 1–2. blokk. Ellenőrző pont: 12.4. III. ellenőrző pont: önállóan tanuló kis háló. Becsült munka: 40–60 óra.
4. **13–16. fejezet:** kép, konvolúció, gradiensei, equivariance, pooling. Előfeltétel: a 3. blokk. Ellenőrző pont: 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése. Becsült munka: 40–60 óra.
5. **17–20. fejezet:** klasszikus, reziduális, hatékony CNN, transfer. Előfeltétel: a 4. blokk. Ellenőrző pont: 20.4. V. ellenőrző pont: indokolt CNN-választás. Becsült munka: 40–60 óra.
6. **21–24. fejezet:** inicializálás, regularizálás, adat, értékelés, diagnózis. Előfeltétel: az 5. blokk. Ellenőrző pont: 24.4. VI. ellenőrző pont: megbízható CNN-training. Becsült munka: 40–60 óra.
7. **25–28. fejezet:** osztályozás, detektálás, szegmentálás, embedding. Előfeltétel: a 6. blokk. Ellenőrző pont: 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés. Becsült munka: 40–60 óra.
8. **29–32. fejezet:** autoencoder, önfelügyelt, generatív, szekvencia, attention. Előfeltétel: a 7. blokk. Ellenőrző pont: 32.4. VIII. ellenőrző pont: célhoz illő neurális modell. Becsült munka: 40–60 óra.
9. **33–36. fejezet:** inference, hatékonyság, magyarázat, kísérlettervezés. Előfeltétel: a 6–7. blokk. Ellenőrző pont: 36.4. IX. ellenőrző pont: lektorálható modellállítás. Becsült munka: 40–60 óra.
10. **37–40. fejezet:** saját operátor, Projekt II–III, záróvizsga. Előfeltétel: az 1–9. blokk. Ellenőrző pont: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték. Becsült munka: 60–80 óra.

Az előfeltételek a blokkok közti kötelező sorrendet adja: a 9. blokk a 8.-tól függetlenül is elkezdhető, ha a 6. és 7. blokk PASS; minden más blokk az előzőre épül.

Közös rubrika (0–4). Minden ellenőrző pontot öt dimenzióban értékelünk: (D1) matematikai helyesség, (D2) kód és futtathatóság, (D3) adat- és mérési protokoll (split, megfigyelési egység, tesztfegyelem), (D4) bizonyíték és nyomon követhetőség (napló, seed, verzió, hash, forrás), (D5) az állítás és korlátai (az állítás nem lép túl a bizonyítékon). Szintek: 0 hiányzik; 1 hibás; 2 lényegében helyes, de hiányosan ellenőrzött; 3 helyes és ellenőrzött; 4 helyes, ellenőrzött, és új helyzetre is indokoltan alkalmazott. A rubrika belső értékelési eszköz, nem hivatalos osztályzat; a

PASS-szabály (lásd alább) a továbblépéshez mégis mind az öt dimenzióban legalább 3-as szintet ír elő, tehát a rubrika a tanulási útvonal kapuja, nem a hallgató minősítése.

Szintpéldák (D1–D5). A 3. szint megfigyelhető jele: a tanuló a hibát *lokalizálja* és a javítást *visszaellenőrzi* (például az `assert` elbukik, a javítás után átmegy). A 2. szint: a hibát észreveszi, de a javítás nem ellenőrzött. Az 1. szint: a hiba tünetét leírja, de nem lokalizálja. A 0. szint: hiányzik. Például D2-nél 3 pont: a kód friss környezetben lefut, és a labor `assert`-jei átmennek; 2 pont: lefut, de az `assert` hiányzik; 1 pont: nem fut le friss környezetben.

Mintabeadások kalibrációhoz. (a) *Teljesen kitöltött jó beadás* (D1–D5 mind 4): minden állításhoz bizonyítás vagy forrásnyom, futási jegyzék SHA-256-tal, korlátok kimondva. (b) *Részben hibás beadás* (például D1=3, D2=2, D3=3, D4=2, D5=2): a levezetés jó, de a futási jegyzék seed nélküli, és az állítás túllép a bizonyítékon; az adott dimenziók célzott remedial útja a futási jegyzék és az állítás-korlát gyakorlása. (c) *Elégtelen beadás* (D2=1 vagy D3=1): a kód nem fut vagy a tesztet a modellválasztásra használja; ez nem kompenzálható más dimenziókkal. A pontozást minden dimenziónál az indok kíséri (mi látszik a beadásban, és mi hiányzik). *Önellenőrizhető*: D1 (kézi számolás), D2 (assert), D3 (split-szabály). *Szakmai visszajelzést érdemel*: D4–D5 (forrásnyom és az állítás korlátai).

Célzott remedial út. Az első két hiba után nem a teljes blokk elejére lépünk vissza, hanem az elbukott dimenzió alfejezetére és a hozzá tartozó laborra (lásd a Visszutak listát), majd csak az ellenőrző pont érintett részét ismételjük; a teljes blokk elejére csak ismétlődő, azonos dimenzióbeli elakadás esetén lépünk vissza (a Visszutak listát lezáró szabály szerint).

PASS-szabály. Továbbléphet, ha (a) az ellenőrző pont laborjának minden `assert`-je és toleranciaellenőrzése a saját futtatásában átmegy, (b) mind az öt dimenzió legalább 3, és (c) a riport csak ténylegesen lefuttatott kód kimenetét közli. **Közös riportszabály:** a riport a 36.4. IX. ellenőrző pont: lektorálható modellállítás hat komponensét tartalmazza (bizonyítás, forrásnyom, kód, futási jegyzék, páros eredmény, korlátok); a futási jegyzék a seedet, a csomag- és eszközverziót és a bemeneti fájlok SHA-256 lenyomatát rögzíti.

Tanulási térkép: a blokkok teljessége. A 26–32. fejezet blokkjai nem egyformán teljes, végigfuttatott tanfolyamok. *Teljes futtatható tanulási útvonal*: az 1–25. fejezet (a GlyphLab teljes útvonala) és a 33–40. fejezet fő részei. *Részleges implementáció*: a detektálás (26.), a szegmentálás (27.) és a retrieval (28.) fő képletei és egy-egy fixture-je. *Elméleti kitekintés*: a generatív modellek (31.) és a transformer-blokkok (32.). *Továbbvezető feladat*: ahol a szöveg tervet vagy illusztrációt ad. A puszta témacím ne sugalljon igazolt gyakorlati jártasságot.

Mért állítások követhetősége. Minden futást állító táblázathoz tartozzon konkrét parancs, környezeti profil, adat/hash és nyers eredményfájl; a nem futtatott tervek maradjanak láthatóan megjelölve (“nem futtatott”, “számított”, “terv”).

Visszautak. Ha egy ellenőrzés elbukik, a hibatípushoz tartozó alfejezetet kell újra feldolgozni: a feladatait megoldani, a laborját újrafuttatni, majd az ellenőrző pontot megismételni.

- **alakhiba: `shape mismatch`, váratlan broadcast, rossz `Linear` bemenet:** 2.1. Python, NumPy és tenzoralak tenzoralak, 3.2. Mátrixszorzás és affín réteg mátrixszorzás, 14.1. Stride, padding, dilation és outputméret outputméret, 14.2. Több bemeneti és kimeneti csatorna csatornák.
- **gradienshiba: a kézi gradiens eltér az autogradtól vagy a véges differenciától:** 4.3. Jacobi-mátrix, láncszabály és VJP VJP, 10.2. Affín réteg és aktiváció backward backward, 10.3. Gradiensellenőrzés és numerikus hibamérce gradiensellenőrzés, 15.1.–15.2. konvolúció gradiense, 15.4. Független gradiensellenőrzési protokoll protokoll.
- **numerikus hiba: `nan`, `inf`, $\log 0$, túlcsordulás:** 2.2. Lebegőpont és stabil elemi számítás lebegőpont, 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil veszteség.

- **statisztikai teszt vagy intervallum hibás, függő mintákon számolt bizonytalanság:** 5.2. Várható érték, variancia és likelihood variancia, 5.3. Mintavételi bizonytalanság és intervallum intervallum, 5.4. Függő képek, seedek és resampling függő képek és resampling, 23.3. Páros összehasonlítás és csoportszintű intervallum páros összehasonlítás, 36.2. Ismétlések és eredménybizonytalanság ismétlések.
- **split- vagy szivárgáshiba, jelenet több splitben:** 1.3. Adatfelosztás és a megfigyelési egység megfigyelési egység, 11.3. Dataset és DataLoader a split után Dataset a split után, 22.1. Adateredet, címke és csoportos split csoportos split.
- **train/eval mód, checkpoint, visszatöltés:** 12.1. Tanítás, validáció és kiértékelési mód kiértékelési mód, 12.2. Checkpoint és folytathatóság checkpoint.
- **a tanítás nem indul vagy nem konvergál:** 12.3. Sanity check és kis mintára túllillesztés sanity check, 21.2. Learning rate és ütemezés tanulási ráta, 24.1. Tanulási görbe és hibadiagnózis tanulási görbe.
- **metrika- vagy aggregálási hiba:** 23.1. Confusion matrix és aggregálási konvenció aggregálási konvenció, 27.3. Dice, IoU és aggregálás Dice és IoU.

Ha ugyanaz a dimenzió két egymást követő kísérletben is 3 alatt marad, a blokk első fejezetétől kell visszalépni.

1.4.12 Egységjegyzék

A folyószöveg a többi alfejezetre `fejezet.alfejezet` számmal hivatkozik (például 4.3. Jacobi-mátrix, láncszabály és VJP). Az `nNNN` azonosító technikai kulcs: ez a forrásfájl neve (`chapters/nNNN.md`), a hivatkozási horgony és a laborfájlok előtagja. Az alábbi jegyzék mindkettőt megadja. A *labor* rovat a `lab/` alatti, a könyvben teljes egészében közölt laborfájlokat sorolja fel; az *adat* rovat a kódblokkokban olvasott GlyphLab-fájlokat (I = `images.npy`, L = `labels.npy`, M = `masks.npy`, C = `corrupted_test.npy`, D = `metadata.csv`); a *kimenet* rovat a kódban szó szerint megnevezett kimeneti fájlokat. Változóval megadott kimeneti útvonalat a jegyzék nem sorol fel; ezeket az adott labor szakasza írja le.

- **1.1** `n001` Modell, tanulás és felhasználási szerződés
  labor: `f3_grid_linear.py`
- **1.2** `n002` Mitől egyetemi szintű egy modellezési állítás?
- **1.3** `n003` Adatfelosztás és a megfigyelési egység
- **1.4** `n004` Tanulási diagnózis és kísérleti napló
  labor: `n004_diagnosis.py`
- **2.1** `n005` Python, NumPy és tenzoralak
  labor: `n005/lab_batch_mean.py`
- **2.2** `n006` Lebegőpont és stabil elemi számítás
- **2.3** `n007` Izolált környezet, notebook és verzió
- **2.4** `n008` Kis tudományos szoftverprojekt
  kimenet: `input.npy`
- **3.1** `n009` Vektor, norma és skalárszorzat
- **3.2** `n010` Mátrixszorzás és affín réteg
- **3.3** `n011` Vetítés, legkisebb négyzetek és kondicionáltság
  labor: `n011_lstsq/n011_lstsq.py`, `n011_lstsq/least_squares.py`
- **3.4** `n012` Sajátérték, SVD és spektrális határ
- **4.1** `n013` Függvény, határérték és derivált
- **4.2** `n014` Parciális derivált és gradiens
  labor: `n014_grad_demo.py`

- **4.3** `n015` Jacobi-mátrix, láncszabály és VJP
  labor: `n015/vjp_lab.py`, `n015/f3_solution.py`
- **4.4** `n016` I. ellenőrző pont: matematika és kód megfelelése
- **5.1** `n017` Esemény, feltétel és véletlen változó
- **5.2** `n018` Várható érték, variancia és likelihood
  labor: `n018_likelihood.py`
- **5.3** `n019` Mintavételi bizonytalanság és intervallum
- **5.4** `n020` Függő képek, seedek és resampling
- **6.1** `n021` Objektív, reguláris tag és gradienslépés
- **6.2** `n022` Konvexitás és sima optimalizálási határ
- **6.3** `n023` Sztochasztikus és minibatch gradiens
- **6.4** `n024` Momentum és adaptív optimizálók
  labor: `n024/optim_steps.py`
- **7.1** `n025` Entrópia, cross-entropy és KL
  labor: `n025_entropy_lab.py`
- **7.2** `n026` Softmax és többosztályos cross-entropy
  labor: `n026/check_ce_grad.py`
- **7.3** `n027` Bináris, multilabel és regressziós cél
- **7.4** `n028` Numerikus stabilitás és jelentésmegőrző implementáció
- **8.1** `n029` Kockázat, mintavétel és baseline
  labor: `n029_simulation.py`
- **8.2** `n030` Véges hipotézisosztály egyenletes korlátja
- **8.3** `n031` Kapacitás, regularizálás és modellválasztás
  adat: ILD
- **8.4** `n032` II. ellenőrző pont: korrekt tanulási állítás
  kimenet: `evidence.json`
- **9.1** `n033` Neuron és perceptron
  labor: `n033_perceptron/perceptron_lab.py`
- **9.2** `n034` MLP és nemlinearitás
- **9.3** `n035` Expresszivitás és approximációs állítások
- **9.4** `n036` Aktivációk és gradiensáramlás
  labor: `n036/activation_gradients.py`, `n036/feladat_f3.py`
- **10.1** `n037` Számítási gráf és lokális derivált
- **10.2** `n038` Affín réteg és aktiváció backward
- **10.3** `n039` Gradiensellenőrzés és numerikus hibamérce
- **10.4** `n040` Tanítási lépés és tanuló MLP
- **11.1** `n041` Tensor és autograd-életciklus
  labor: `n041/detach_demo.py`, `n041/accumulation_demo.py`, `n041/check_autograd.py`, `n041/split_path.py`, `n041/lifetime_counterexample.py`
- **11.2** `n042` nn.Module és paraméterregisztráció
- **11.3** `n043` Dataset és DataLoader a split után
  labor: `n043_dataset.py`, `n043_check.py`; adat: D
- **11.4** `n044` Egy teljes PyTorch tanítási iteráció
  labor: `n044_train_step.py`
- **12.1** `n045` Tanítás, validáció és kiértékelési mód
  adat: ILD
- **12.2** `n046` Checkpoint és folytathatóság
- **12.3** `n047` Sanity check és kis mintára túlillesztés
  adat: ILD
- **12.4** `n048` III. ellenőrző pont: önállóan tanuló kis háló
- **13.1** `n049` Kép mint diszkrét többcsatornás jel

labor: `n049_layout/lab_n049.py`
kimenet: `mini_images.npz`

- **13.2** `n050` Konvolúció és cross-correlation
  labor: `n050/check_convs.py`
- **13.3** `n051` Lokális szűrők és tanult reprezentáció
  adat: ILD
- **13.4** `n052` Határkezelés és frekvenciaintuíció
  labor: `n052_boundary_check.py`, `n052_lab.py`
- **14.1** `n053` Stride, padding, dilation és outputméret
- **14.2** `n054` Több bemeneti és kimeneti csatorna
- **14.3** `n055` Paraméterszám, MAC és aktivációmemória
  labor: `n055/param_mac_check.py`, `n055/feladat_f3.py`
- **14.4** `n056` Csoportos és depthwise konvolúció
- **15.1** `n057` Súly- és biasgradiens
  labor: `n057/weight_grad_check.py`
- **15.2** `n058` Inputgradiens és átfedő recepciós mezők
  labor: `n058_dx_check.py`, `n058_f3_1d.py`
- **15.3** `n059` Transposed convolution mint adjungált
- **15.4** `n060` Független gradiensellenőrzési protokoll
  kimenet: `gradcheck_protocol.json`
- **16.1** `n061` Eltolási equivariance és invariance
- **16.2** `n062` Pooling, downsampling és aliasing
- **16.3** `n063` Elméleti és effektív recepciós mező
- **16.4** `n064` IV. ellenőrző pont: a CNN-operátor teljes megértése
  labor: `n064_checkpoint.py`
- **17.1** `n065` LeNet és a lokális megosztott reprezentáció
  labor: `tiny_lenet_check.py`; adat: I
- **17.2** `n066` AlexNet és a kísérleti kontextus
- **17.3** `n067` VGG és kis kernelek kompozíciója
- **17.4** `n068` Architektúra összehasonlítása kontrollált keretben
  labor: `n068/train_compare.py`
- **18.1** `n069` Reziduális blokk és identitásút
- **18.2** `n070` Batch normalization train és eval módban
- **18.3** `n071` Pre-activation, kis batch és alternatív normalizálás
- **18.4** `n072` Reziduális blokk ablatív vizsgálata
  labor: `n072_resablation.py`
- **19.1** `n073` Depthwise separable és pointwise réteg
  labor: `n073_lab.py`, `n073_f3.py`
- **19.2** `n074` MobileNet és inverted residual
  labor: `n074/inverted_residual_demo.py`
- **19.3** `n075` EfficientNet és architektúraskálázás
  labor: `n075/n075_scale_bench.py`
- **19.4** `n076` ConvNeXt és a recept/architektúra elválasztása
- **20.1** `n077` Feature extractor és új fej
  labor: `n077/manual_check.py`, `n077/head_swap_lab.py`; adat: ILD
- **20.2** `n078` Finomhangolás és tanulási ráta
  adat: ILD
- **20.3** `n079` Preprocessing, domain shift és súlylicenc
  labor: `preprocess_check.py`, `check_preprocessing.py`, `preprocess_normalize.py`; adat: I
  kimenet: `images_norm.npy`

- **20.4** `n080` V. ellenőrző pont: indokolt CNN-választás
  labor: `n080_residual_check.py`, `n080_decision_check.py`
- **21.1** `n081` Inicializálás és varianciaterjedés
  labor: `n081_mini_scale.py`, `n081_init_scale.py`
- **21.2** `n082` Learning rate és ütemezés
  labor: `n082_schedule_compare.py`; adat: ILD
- **21.3** `n083` Dropout, súlyregularizálás és augmentáció
  adat: ILD
- **21.4** `n084` Osztályarány, súlyozás és céleloszlás
  labor: `f3_weighted_ce.py`; adat: ILD
- **22.1** `n085` Adateredet, címke és csoportos split
- **22.2** `n086` Címkemegőrző transzformációk
- **22.3** `n087` Zajos és hiányos címke
  adat: ILD
- **22.4** `n088` Augmentációs kísérlet és domainváltozás
  labor: `augment_experiment.py`
- **23.1** `n089` Confusion matrix és aggregálási konvenció
  adat: L
  kimenet: `eval_preds.npz`
- **23.2** `n090` Kalibráció és döntési küszöb
  adat: ILD
- **23.3** `n091` Páros összehasonlítás és csoportszintű intervallum
  labor: `scene_interval.py`; adat: D
  kimenet: `test_preds.npy`
- **23.4** `n092` Hibaszeletek és alkalmazási határ
- **24.1** `n093` Tanulási görbe és hibadiagnózis
  labor: `n093_diagnose.py`; adat: ILD
- **24.2** `n094` CPU/GPU profiling és helyes időmérés
  adat: ILD
  kimenet: `measure_report.json`
- **24.3** `n095` Mixed precision és nagyobb batch
- **24.4** `n096` VI. ellenőrző pont: megbízható CNN-training
  labor: `n096_checkpoint.py`, `check_best.py`; adat: ILCD
  kimenet: `runlog.json`
- **25.1** `n097` Teljes képosztályozó adatfolyam
  labor: `model_def.py`, `train_classifier.py`, `infer.py`; adat: ILD
- **25.2** `n098` Multilabel és hierarchikus cél
- **25.3** `n099` Kis adat és feature-alapú baseline
- **25.4** `n100` Nyílt halmaz és ismeretlen input
  labor: `rejection_rule.py`
- **26.1** `n101` Dobozkoordináta, IoU és lokalizációs loss
  labor: `n101_lab.py`; adat: MD
- **26.2** `n102` Egy- és kétlépcsős detektor
- **26.3** `n103` NMS, confidence és AP
  labor: `n103_nms_lab.py`, `n103_map.py`
- **26.4** `n104` Detektálási pipeline és hibaelemzés
- **27.1** `n105` Pixelszintű klasszifikáció és FCN
  adat: ILMD
- **27.2** `n106` U-Net, encoder–decoder és skip
- **27.3** `n107` Dice, IoU és aggregálás
- **27.4** `n108` Instance mask és összetett feladat

- **28.1** `n109` Embedding, távolság és normalizálás
  labor: `embedding_exact.py`, `extract_embeddings.py`
- **28.2** `n110` Sziámi és triplet cél
  labor: `n110_triplet_lab.py`
- **28.3** `n111` Retrieval-értékelés és adatfelosztás
  labor: `retrieval_report.py`
- **28.4** `n112` VII. ellenőrző pont: CNN alkalmazási szerződés
  labor: `n112_dotpair/retrieval_eval.py`
- **29.1** `n113` Lineáris és nemlineáris autoencoder
- **29.2** `n114` Denoising és korrupciós modell
  adat: ID
- **29.3** `n115` Regularizált latent reprezentáció
  labor: `n115_latent_probe.py`
- **29.4** `n116` Anomáliapontszám rekonstrukcióból
  labor: `score_hand_check.py`, `anomaly_lab.py`
  kimenet: `anomaly_report.json`
- **30.1** `n117` Pretext és downstream feladat
  adat: ID
- **30.2** `n118` Kontrasztív cél és temperature
  labor: `n118/nt_xent_pairs.py`
- **30.3** `n119` Negatív nélküli módszerek és collapse
- **30.4** `n120` Önfelügyelt kísérlet korrekt összevetése
- **31.1** `n121` VAE és variációs alsó korlát
- **31.2** `n122` GAN mint minimax probléma
- **31.3** `n123` Diffúziós zajmodell és CNN-denoiser
- **31.4** `n124` Generatív felhasználás és értékelés
- **32.1** `n125` Szekvenciamodell és változó hossz
- **32.2** `n126` Scaled dot-product attention
  labor: `n126/n126_attention.py`
- **32.3** `n127` Vision Transformer és hibrid felhasználás
- **32.4** `n128` VIII. ellenőrző pont: célhoz illő neurális modell
  adat: ILD
- **33.1** `n129` Inference-szerződés és modellcsomag
  adat: I
  kimenet: `control_input.npy`, `manifest.json`, `reference_logits.npy`
- **33.2** `n130` Batch és kis szolgáltatás
- **33.3** `n131` torch.export és ONNX
- **33.4** `n132` Üzemeltetés, drift és késő címke
  labor: `n132/monitor_demo.py`, `n132/rollback_policy.py`, `n132/test_schema.py`, `n132/test_join.py`
- **34.1** `n133` Mért késleltetés, throughput és memória
  kimenet: `latency_results.json`
- **34.2** `n134` Kvantálás és hibatovábbterjedés
- **34.3** `n135` Pruning és struktúra
  labor: `n135_prune_lab.py`
- **34.4** `n136` Teacher–student desztilláció
  labor: `n136_distill/soft_target_example.py`, `n136_distill/train_distill.py`, `n136_distill/manual_kd.py`; adat: IL
- **35.1** `n137` Saliency és Grad-CAM
  labor: `n137_gradcam.py`
- **35.2** `n138` Magyarázat sanity check

labor: `n138_sanity_check.py`

- **35.3** `n139` Perturbáció és adversarial példa
  labor: `n139/manual_fgsm.py`, `n139/fgsm_demo.py`
- **35.4** `n140` Korlátok, eredet és döntési felelősség
- **36.1** `n141` Állításból kísérleti terv
- **36.2** `n142` Ismétlések és eredménybizonytalanság
  adat: ILD
- **36.3** `n143` Szakirodalmi reprodukció és attribúció
- **36.4** `n144` IX. ellenőrző pont: lektorálható modellállítás
- **37.1** `n145` Operátorspecifikáció és kézi referencia
- **37.2** `n146` Backward és ellenőrzési lefedettség
- **37.3** `n147` Linearitás, adjungált és határfeltétel
  labor: `n147_operator_report.py`
- **37.4** `n148` Rétegből kis tanulómodell
- **38.1** `n149` Adat, split és előzetes protokoll
  labor: `protocol_check.py`; adat: ILMCD
  kimenet: `protocol_n149.json`
- **38.2** `n150` Modell, training és best checkpoint
  labor: `glyph_models.py`, `train_small_cnn.py`, `run_project2.py`
- **38.3** `n151` Végső teszt és páros korrupció
  labor: `eval_test.py`; adat: ILCD
  kimenet: `n151_logits_clean.npy`, `n151_pred_clean.npy`, `n151_pred_corrupt.npy`, `n151_report.json`, `n151_test_index.npy`
- **38.4** `n152` Mentett modellből független inference
  labor: `package_project2.py`, `PROJECTII/models.py`, `PROJECTII/inference.py`
  kimenet: `model_card.json`, `test_indices.npy`, `test_reference.npz`
- **39.1** `n153` Feladat- és annotációs döntés
  labor: `project3/annotation_spec.py`
- **39.2** `n154` Encoder–decoder vagy lokalizációs fej
  labor: `project3/segmentation_head.py`, `project3/train_seg.py`; adat: IMD
  kimenet: `seg_train_log.json`
- **39.3** `n155` Geometriai kontroll és metrika
  labor: `project3/n155_geom_control.py`; adat: MD
  kimenet: `control_pred.npy`, `control_target.npy`, `n155_control_report.json`
- **39.4** `n156` Alkalmazási riport és átadás
  labor: `project3/export_delivery.py`, `project3/report_audit.py`; adat: ICD
  kimenet: `manifest.json`, `model_card.json`
- **40.1** `n157` Összetett levezetési vizsga
- **40.2** `n158` Kísérleti és hibakeresési vizsga
  adat: ILCD
- **40.3** `n159` Tudományos kézirat és forrásaudit
  adat: D
- **40.4** `n160` X. ellenőrző pont: önálló neurális modell és bizonyíték
  labor: `n160_bbox_height.py`; adat: ID
  kimenet: `inference_contract.json`, `test_predictions.npz`

1.4.13 Rövid összefoglaló

Ez a diagnózis nem osztályozó vizsga, hanem az alapozó útvonal kiválasztását segítő első mérés. A vektor normája csak közös komponens-egység mellett értelmes; az átlag egysége az adaté; a differenciahányados egysége $[f]/[t]$. A másodfokú t^2 szimmetrikus hányadosa pontosan $2t$,

amit algebrailag igazoltunk; harmadfokú esetre a segédszámítás nem áll, és a határértéket a 4.1. Függvény, határérték és derivált formalizálja. A tenzor alakja dimenzió nélküli, dtype-ja a tárolást rögzíti. A seed a generátor kezdőállapota, a futásazonosító a napló kulcsa; a kettő nem felcserélhető. A labor az első változatot append-only JSON-ba menti, és a hibák alapján célzott alapozó egységet választ.

Továbblépés. A 2.1. Python, NumPy és tenzoralak egység a tenzoralak és a NumPy-műveletek pontos kezelését vezeti be; ha a diagnózis a `mean` vagy `tensor_elements` feladatban hibát mutatott, ez az első célzott alapozó.

Források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.

2 Programozási és numerikus munkaasztal

2.1 Python, NumPy és tenzoralak

2.1.1 Cél és indító becslés

Ez az egység a NumPy `ndarray` objektumán keresztül rögzíti a skalár, vektor, mátrix és négydimenziós képtenzor közötti mozgás szabályait. A cél nem a NumPy API katalógusa, hanem az, hogy minden művelet előtt és után egyértelmű legyen a `shape`, a `dtype` és a redukált tengely. Az objektumok alakja a modell bemeneti és kimeneti specifikációjának (az input és az output előírt alakjának, dtype-jának és jelentésének) része 1.1. Modell, tanulás és felhasználási szerződés; a tévesen értelmezett alak csendben hibás számolást adhat.

Indító becslés: legyen x alakja `(4,)`, y alakja `(4,1)`. Az `x + y` kifejezés a NumPy-értelmezés szerint milyen alakot ad? Becsülje meg, mielőtt tovább olvas, és jegyezze fel a tippjét. Ha `(4,4)`-et tippelt, a broadcasting szabályát már érzi; ha `(4,)`-et, akkor éppen ez az egység fogja tisztázni, miért nem az.

2.1.2 `ndarray`, `dtype`, `shape` és `axis`

Definíció (`ndarray`). A NumPy `ndarray` homogén, többdimenziós tömbobjektum: minden elemének azonos a típusa. Az `ndarray` objektum és az adatpuffer külön dolog: a `reshape` lehetőség szerint nézetet ad ugyanarra a pufferre, a `shape` attribútum kompatibilis esetben közvetlenül átírható, a `resize` pedig helyben megváltoztathatja a méretet. Ebben a könyvben ezért az alakot az adatszerződés részének tekintjük, és módosítás helyett új, explicit alakú nézetet vagy tömböt készítünk. A memóriában az elemek lineáris sorrendben tárolódnak; a többdimenziós indexelést a `shape` és a hozzá tartozó `strides` írja le. Az `ndarray.ndim` a dimenziók száma, `ndarray.size` az elemek darabszáma, `ndarray.shape` pedig egy tuple, amelynek i-edik eleme az i-edik tengely hossza. A `shape` tehát az alak: a tengelyhosszak rendezett tuple-je; a `(4,)` és a `(4,1)` ezért két különböző alak.

Definíció (`dtype`). A `dtype` az elemi tárolási típus: `np.float32`, `np.float64`, `np.int64`, `np.uint8`, `np.bool_` stb. A `dtype` nem tartalmaz mértékegységet, és a `shape` sem; mindkettő mértékegység nélküli tárolási és szerkezeti információ, függetlenül attól a dimenziófogalomtól, amelyet a fizikában használunk 1.4. Tanulási diagnózis és kísérleti napló. A jelentést (például „intenzitás a $[0, 1]$ intervallumban”) a programozó dokumentálja, nem a tömb hordozza. A tananyag a tanításhoz elsősorban float32-et használ, mert a neurális hálók tanításában CPU-n és GPU-n egyaránt ez a legelterjedtebb numerikus típus; az ellenőrzésekhez helyenként float64-et használunk.

Definíció (`axis`). Az `axis` a redukciós műveletek tengelyparamétere. Az `axis=0` az első, `axis=1` a második tengely mentén összegez; negatív index a végétől számol. `axis=None` az összes elem feletti redukciót jelenti. Például $X \in \mathbb{R}^{N \times C \times H \times W}$ esetén `axis=0` a batch (köteg), `axis=1` a csatorna tengely.

A Python-szintaxis egyetlen pontja, amely itt félreérthető: a `(n,)` egyelemű tuple, tehát n hosszú egydimenziós vektor; a `(n,1)` kétdimenziós oszlopvektor. A kettő nem ugyanaz az objektum, és a broadcasting eltérően kezeli őket.

2.1.3 Broadcasting alakszabálya és a külső összeg lemmája

Definíció (broadcasting alakszabálya, [D04] szerinti megfogalmazás). Legyen a két operandus alakja $s_A = (a_1, \dots, a_r)$ és $s_B = (b_1, \dots, b_q)$. A rövidebb alak elé írjunk 1-eseket, amíg a két tuple hossza egyenlő nem lesz, majd jobbról balra igazítsuk a méreteket. Az elemenkénti bináris művelet akkor és csak akkor értelmezett, ha minden igazított k pozícióban $a_k = b_k$, vagy $a_k = 1$, vagy $b_k = 1$. Az eredmény mérete pozíciónként az a méret, amelyik nem 1 (ha mindkettő 1, akkor 1); ez általában a két méret maximuma, de a nulla méret kivétel: a $(0,)$ és az $(1,)$ eredménye $(0,)$, a $(0,1)$ és az $(1,0)$ eredménye $(0,0)$. A végrehajtáskor az 1 méretű dimenzió virtuálisan ismétlődik az eredmény méretére.

Ez a szabály a NumPy API definíciója, nem tétel. A gyakorlatban az 1 méretű dimenzió ismétlését nem másolat, hanem 0 lépésközű virtuális tengely valósítja meg: az ilyen tengelyen a memóriabeli lépésköz (stride) 0, ezért ugyanaz az elem többször olvasódik, másolás nélkül [D04]. Az alábbi lemma a szabály első, gyakran félreértett következménye.

Lemma (külső összeg). Legyen $x \in \mathbb{R}^n$ alakja `(n,)` és $y \in \mathbb{R}^{n\times 1}$ alakja `(n,1)`. Ekkor az `x + y` művelet eredménye `(n,n)` alakú, és az eredmény $C_{ij} = x_j + y_i$.

Bizonyítás. A jobbról igazított alakok: x esetén $(1, n)$ — a hiányzó első dimenziót 1-nek tekintjük —, y esetén $(n, 1)$. Az első pozícióban 1 és n szerepel, tehát broadcastelhető; a másodikban n és 1, szintén broadcastelhető. Az eredmény mérete az első pozícióban az 1-től különböző n, a másodikban szintén n, tehát (n, n); ez $n = 0$ esetén is igaz (az eredmény $(0, 0)$ alakú, üres), ezért itt nem a maximumot, hanem a „nem 1” szabályt használjuk. A virtuális ismétlés következménye: x a második tengelyen n hosszú, ezért a kimenet minden sorában megismétlődik, vagyis az (i, j) elemhez x_j járul; y az első tengelyen n hosszú, a második tengelyen 1, tehát minden oszlopban ismétlődik, vagyis az (i, j) elemhez y_i járul. Az összeadás elemenkénti, így $C_{ij} = x_j + y_i$.

A lemma mutatja, hogy a `(n,)` vektor és a `(n,1)` oszlopvektor összeadása nem a kívánt komponensenkénti összeadás, hanem az összes lehetséges pár összegét tartalmazó mátrix. Ez a labor és a gyakori tévedés szakasz központi esete.

Az indító becslés két alakjára a jobbról igazítás:

pozíció jobbról	2.	1.
`x` alakja `(4,)`	1	4
`y` alakja `(4,1)`	4	1
eredmény alakja	4	4

A rövidebb `(4,)` alak elé írt 1-gyel a két alak `(1,4)` és `(4,1)`; mindkét pozícióban 1 és 4 áll, ezért az eredmény `(4,4)`, és eleme $x_j + y_i$.

Az NCHW-elrendezés tengelyei és a jobbról igazítás együtt (a 2×1×2×3 példa `X` tömbjével és egy `(1,3)` alakú `b` vektorral):

pozíció jobbról	4.	3.	2.	1.
tengely	N (batch)	C (csatorna)	H (magasság)	W (szélesség)
`X` alakja	2	1	2	3

pozíció jobbról	4.	3.	2.	1.
`b` alakja	1	1	1	3
eredmény alakja	2	1	2	3

A `b (1,3)` alakját a rövidebb tuple elé írt 1-esekkel `(1,1,1,3)`-ra egészítjük ki. A 4., 3. és 2. pozícióban az 1 tetszőleges hosszal broadcastelhető, ezért `b` a batch- és a magasságtengelyen ismétlődik, a csatornatengelyen 1 marad, a szélességtengelyen pedig elemenként párosul.

2.1.4 Kézi példa NCHW alakellenőrzéssel

Legyen a vizsgált képtenzor

$$X \in \mathbb{R}^{2\times 1\times 2\times 3}, \qquad X = \left[\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}\right], \begin{bmatrix} 6 & 7 & 8 \\ 9 & 10 & 11 \end{bmatrix}\Bigg],$$

ahol a mátrixos írásmód az áttekinthetőség kedvéért elhagyja az 1 hosszú csatornatengelyt (a teljes alak `(2,1,2,3)`), a `dtype` pedig float32. A matematikai jelölésben az indexelés 1-től indul, a Python 0-tól számol; az alábbi képletekben és a kódban egységesen a Python 0-tól induló indexelését használjuk.

A teljes átlag:

$$\operatorname{mean}(X) = \frac{1}{2\cdot 1\cdot 2\cdot 3}\sum_{n=0}^{1}\sum_{c=0}^{0}\sum_{h=0}^{1}\sum_{w=0}^{2} X_{n,c,h,w} = \frac{66}{12} = 5{,}5,$$

amelyet a NumPy alapértelmezésben *skalárként* ad vissza (`type(X.mean())` `numpy.float32`, nem `ndarray`); a `shape == ()` ezt nem különíti el a 0-dimenziós tömbtől, amit `np.asarray(X.mean())` ad (`ndim == 0`, alakja `()`). A típust ezért `isinstance(m, np.generic)` és `isinstance(np.asarray(m), np.ndarray)` kontrollal ellenőrizzük. A batch-átlag `axis=0` mentén redukál, tehát a két kép pixelenkénti átlaga:

$$\bar{X}_{c,h,w} = \frac{1}{2}(X_{0,c,h,w} + X_{1,c,h,w}), \qquad \bar{X} = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix},$$

alakja `(1,2,3)`. A csatornaátlag a batch, magasság és szélesség felett átlagol, `axis=(0,2,3)`, eredménye `(1,)` és értéke [5,5]. A képenkénti átlag `axis=(1,2,3)`, eredménye `(2,)`, értékei [2,5; 8,5], mert a két kép elemeinek összege 15, illetve 51, egyenként 6 elemmel osztva.

Vegyük észre a tengelyek szerepét: ugyanaz a X más-más kérdésre más alakú eredményt ad. A modellkódban a `mean` tengelyparamétere ezért része a specifikációnak.

2.1.5 Labor: batch-átlag és hibajavítás

Előfeltétel: Python 3.12 és `uv`; a laborcsomag `lab/` projektje, NumPy a lock fájlban rögzített verzióval [D01]. A futtatás CPU-n történik, PyTorch nem kell hozzá. A laborcsomag gyökeréből a parancs:

```
cd lab
uv sync
uv run python n005/lab_batch_mean.py
```

A szkript:

```
# lab/n005/lab_batch_mean.py
"""Kézi és NumPy batch-átlag, valamint a (n,)+(n,1) hiba javítása."""
import numpy as np

X = np.array([[1.0, 0.0, 2.0],
              [0.0, 2.0, 1.0],
              [2.0, 1.0, 0.0],
              [1.0, 1.0, 2.0]], dtype=np.float32)

# Kézi átlag oszloponként: shape (3,)
manual_mean = np.array([X[:, j].mean() for j in range(X.shape[1])],
                       dtype=np.float32)
np_mean = X.mean(axis=0)
print("X shape:", X.shape)
print("manual_mean:", manual_mean)
print("np_mean:", np_mean)

# Ellenőrzés: az alakok és a két átlag egyezése
assert X.shape == (4, 3), X.shape
assert np_mean.shape == (3,), np_mean.shape
np.testing.assert_allclose(manual_mean, np_mean, rtol=1e-6, atol=1e-6)
expected_mean = np.array([1.0, 1.0, 1.25], dtype=np.float32)
np.testing.assert_allclose(np_mean, expected_mean, rtol=1e-6, atol=1e-6)

# Hiba: (3,) és (3,1) összeadása (3,3) mátrixot ad
x = np.array([1.0, 2.0, 3.0], dtype=np.float32)      # (3,)
y = np.array([[10.0], [20.0], [30.0]], dtype=np.float32)  # (3,1)
bad = x + y          # (3,3) -- csendes, de rossz művelet
good = x + y[:, 0]   # (3,)  -- explicit 1D vektor
print("bad shape:", bad.shape)
print("good shape:", good.shape, good)

# Ellenőrzés: a rossz alak (3,3), a jó (3,), és az értékek stimmelnek
assert bad.shape == (3, 3), bad.shape
assert good.shape == (3,), good.shape
np.testing.assert_allclose(bad, np.array([[11, 12, 13],
                                          [21, 22, 23],
                                          [31, 32, 33]], dtype=np.float32))
np.testing.assert_allclose(good, np.array([11.0, 22.0, 33.0],
                                          dtype=np.float32))
print("OK")
```

MÉRT KIMENET (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; a bemenet determinisztikus, ezért seed nem kell):

```
X shape: (4, 3)
manual_mean: [1.   1.   1.25]
np_mean: [1.   1.   1.25]
bad shape: (3, 3)
good shape: (3,) [11. 22. 33.]
OK
```

Az utolsó `OK` sor az asszertek sikeres lefutását jelzi.

A beadandó eredmény: a kézi és a NumPy-átlag egyezése, a `bad` és `good` alakok megkülönböztetése, valamint a saját futtatás kimenete. A seed (a véletlenszám-generátor kezdőállapota) itt nem szükséges, mert az adatok determinisztikusan vannak megadva; véletlen adatok generálásakor az azonos seed azonban önmagában nem garantál minden platformon és NumPy-verzión bitazonos ismételhetőséget [D04], ezért a protokoll a seed mellett a NumPy-verziót is rögzíti.

2.1.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem felcserélhető a `(n,)`, `(n,1)` és `(1,n)` alak. Konkrétan: legyen X alakja `(5,8)`, b_1 alakja `(8,)`, b_2 alakja `(1,8)`, b_3 alakja `(8,1)`. Mely `X+b_i` kifejezések értelmesek, és mi az eredmény alakja?

F2 (levezetési, nehézség 2). Legyen A alakja `(3,4)`, b alakja `(4,)`, c alakja `(3,1)`. A broadcasting szabályból vezesse le, hogy `A+b` alakja `(3,4)` és eleme $A_{ij} + b_j$; mutassa meg azt is, hogy `b[:,None] + c.T` alakja `(4,3)` és eleme $b_i + c_j$. Mindkét állítást igazolja az alakszabály pozíciónkénti alkalmazásával.

F3 (implementációs, nehézség 2). Írjon `batch_and_channel_mean(x)` függvényt, amely `x` alak `(N,C,H,W)` bemenetre visszaadja a batch-átlagot (`(C,H,W)`) és a csatornaátlagot (`(C,)`) float32 típusban. Ellenőrizze a fenti 2×1×2×3 példán az `np.testing.assert_allclose` eszközzel; a tűrés `rtol=1e-6, atol=1e-6` (float32 mellett a gépi epszilon nagyságrendje $1{,}2 \cdot 10^{-7}$, ezért ennél lazább tűrés indokolt).

F4 (hibakeresési, nehézség 2). Az alábbi függvény hibát okoz, amikor a hívó `(4,3)` kimenetet vár:

```
def add_bias(X, b):
    return X + b   # X.shape == (4,3), b.shape == (3,1)
```

Azonosítsa a hiba okát a broadcasting szabály alapján, adja meg a következményt, javítsa a függvényt úgy, hogy a kimenet `(4,3)` legyen, és ellenőrizze egy konkrét tömbön.

F5 (kísérlettervezési, nehézség 3). Tervezzen kis kísérletet annak eldöntésére, hogy egy adott `preprocess(images)` függvény a csatornaátlag kivonását batch-szintű statisztikával végzi-e, vagy rögzített, előre kiszámolt átlaggal. Adja meg a kérdést, a két összehasonlítandó útvonalat, a használt GlyphLab-állományt (a könyv saját, procedurálisan generált szürke képadatkészletét: 180 jelenet, jelenetenként két nézet, összesen 360 kép, `metadata.csv`-vel; a teljes szerződést a függelék A. A GlyphLab adatkészlet szakasza rögzíti), a metrikát és a következtetés érvényességi korlátját.

2.1.7 Megoldások és indoklás

F1 megoldás. A `(n,)` egydimenziós, a `(n,1)` kétdimenziós oszlop-, a `(1,n)` kétdimenziós sorvektor. $X + b_1$: jobbról igazítva b_1 alakja `(1,8)`, az első pozíció 5 és 1 broadcastelhető, a második 8 és 8 egyenlő, eredmény `(5,8)`. $X + b_2$: alakok `(5,8)` és `(1,8)`, ugyancsak `(5,8)`. $X + b_3$: alakok `(5,8)` és `(8,1)`; jobbról a második pozíció 8 és 1 broadcastelhető, de az első pozíció 5 és 8 sem nem egyenlő, sem nem 1, tehát `ValueError`. Az első két kifejezés tehát értelmes, a harmadik nem. Az oszlopvektor-bias nem adható hozzá így; csak akkor lenne kompatibilis, ha X első dimenziója is 8 lenne (ekkor azonban soronként egy-egy skalár adódna hozzá, nem jellemzőnkénti bias).

F2 megoldás. `A+b`: a `b` alakja `(4,)`, helyettesítve `(1,4)`. Az első pozíció 3 és 1 → broadcast, a második 4 és 4 → egyenlő. Az eredmény `(3,4)`. Az (i, j) kimeneti elemhez A_{ij} járul; b stride-0 első dimenziója miatt minden sorban ugyanaz a b ismétlődik, tehát a hozzájárulás b_j. `b[:,None] + c.T`: a bal oldal alakja `(4,1)`, a jobb oldalé `(1,3)`. Az első pozíció 4 és 1, a második 1 és 3, mindkettő broadcastelhető; eredmény `(4,3)`. A bal oldal a második tengelyen ismétlődik, ezért eleme az (i, j) pontban b_i; a jobb oldal az első tengelyen ismétlődik, ezért c_j; az összeg $b_i + c_j$.

F3 megoldás.

```python
import numpy as np

def batch_and_channel_mean(x: np.ndarray):
    assert x.ndim == 4, f"Várt shape (N,C,H,W), kapott: {x.shape}"
    batch_mean = x.mean(axis=0)            # (C,H,W)
    channel_mean = x.mean(axis=(0, 2, 3))  # (C,)
    return batch_mean.astype(np.float32), channel_mean.astype(np.float32)

X = np.arange(12, dtype=np.float32).reshape(2, 1, 2, 3)
batch, channel = batch_and_channel_mean(X)
expected_batch = np.array([[[3, 4, 5], [6, 7, 8]]], dtype=np.float32)
expected_channel = np.array([5.5], dtype=np.float32)
np.testing.assert_allclose(batch, expected_batch, rtol=1e-6, atol=1e-6)
np.testing.assert_allclose(channel, expected_channel, rtol=1e-6, atol=1e-6)
print("OK", batch.shape, channel.shape)  # OK (1, 2, 3) (1,)
```

A szélsőeset: $N = 1$ esetén a batch-átlag maga a kép, a csatornaátlag a képenkénti átlaggal egyezik.

F4 megoldás. A hiba oka: `(4,3)` és `(3,1)` jobbról igazítva az első pozícióban 4 és 3 áll, amelyek sem egyenlők, sem 1, tehát a broadcasting szabály szerint a művelet nem értelmezett; a NumPy `ValueError`-t dob. A következmény az, hogy a hívó egyáltalán nem kap kimenetet. A javítás: alakítsuk a bias-t egydimenzióssá, például `b.reshape(-1)` vagy `b[:, 0]`; ekkor az alak `(3,)`, a broadcasting `(1,3)`-ként értelmezi, és az eredmény `(4,3)`. Ellenőrzés:

```python
import numpy as np

def add_bias(X, b):
    return X + b.reshape(-1)

X = np.ones((4, 3), dtype=np.float32)
b = np.array([[1.0], [2.0], [3.0]], dtype=np.float32)
out = add_bias(X, b)
assert out.shape == (4, 3)
np.testing.assert_allclose(out[0], [2.0, 3.0, 4.0], rtol=1e-6, atol=1e-6)
```

F5 megoldásminta. Hipotézis: a `preprocess` a csatornaátlag kivonásakor a batch statisztikáját használja-e (batch-függő viselkedés), vagy egy rögzített, előre kiszámolt átlagot. Protokoll: a GlyphLab `data/images.npy` `(360,1,24,24)` alakú float32 tömbjéből számítsuk ki a csatornaátlagot a teljes train halmazon, és rögzítsük; ezután válasszunk egy *másik* batch-et (például egy validation batch-et), amelynek saját átlaga a rögzített értéktől mérhetően eltér. Az első útvonal a teljes train halmazon számolt, rögzített átlagot vonja ki, a második útvonal ugyanarra a batch-re a batch saját átlagát. Metrika: a két kimenet maximális abszolút eltérése, valamint külön-külön az eltérés a két hipotézis referenciakimenetétől. Döntési szabály: ha a `preprocess` kimenete a rögzített átlagú referenciával egyezik a tűrésen belül, a függvény nem batch-függő; ha a batch-átlagú referenciával egyezik, batch-szintű normalizálás van jelen. Kontroll: a két referencia különbsége legyen nagyobb a tűrésnél, különben a kísérlet nem dönt. Érvényességi korlát: a mérés egy implementációs tulajdonságot mutat, nem a modellminőséget; a tanítás és kiértékelés közötti eltérés hatását külön, rögzített protokollú kísérletben kell mérni.

2.1.8 Gyakori tévedés: a lefutó broadcasting lehet rossz művelet

A legveszélyesebb eset nem az, amikor a broadcasting hibát dob, hanem amikor lefut, de más eredményt ad. A fenti lemmában `x (3,)` és `y (3,1)` összeadása `(3,3)` mátrixot hoz létre; ha a kód egy `(3,)` vektort vár, a hiba csak akkor derül ki, amikor egy későbbi függvény rossz alakot kap, vagy ami rosszabb, a modell csendben rossz értékekkel tanul.

Ellenpélda a „mindig működik, tehát jó” érvelésre: az $X \in \mathbb{R}^{4\times 3}$ batch és a $b \in \mathbb{R}^3$ bias összeadása jellemzőnkénti torzítást ad, ahogyan az F4 javítása mutatja; ha viszont valaki tévedésből a bias-t `(4,1)` alakban adja meg, a broadcasting lefut, mert a `(4,3)` és a `(4,1)` alak kompatibilis (első pozíció: 4 és 4; második: 3 és 1). Az eredmény `(4,3)` lesz, de minden sorhoz egyetlen skalár adódik hozzá oszlopirányban, nem jellemzőnkénti bias; ez a csendes hibaművelet éppen a 4-es (vagy 1-es) batch-méretnél fut le, más batch-méretnél viszont `ValueError` jön. A shape-assert és a kimenet explicit ellenőrzése ezért nem adminisztratív teher, hanem a modell bemeneti specifikációjának része 1.1. Modell, tanulás és felhasználási szerződés.

A nézet/másolat különbsége is hasonló csendes hibát okozhat: az alapszeletelés nézetet (`view`) ad, amely osztozik a memórián, ezért

```
x = np.arange(6, dtype=np.float32).reshape(2, 3)
row = x[0]      # view
row[0] = -1.0   # x[0,0] is megváltozik
```

Ha független másolatra van szükség, explicit `x.copy()` kell. A Python-listák szeletelése ezzel szemben másolatot ad; a két viselkedés összekeverése nehezen lokalizálható hibákhoz vezet.

2.1.9 Önellenőrzés

1. Mi az alakja a `np.ones((3,1)) + np.arange(4)` kifejezésnek? Válasz: `(3,4)`. Jobbról igazítva a `(3,1)` és az `(1,4)` alak áll; az első pozícióban 3 és 1, a másodikban 1 és 4, mindkettő broadcastelhető, ezért az eredmény `(3,4)`.
2. Adott X alak `(2,1,2,3)`. Mi az alakja a `X.mean(axis=0)` és a `X.mean(axis=1)` kifejezésnek? Válasz: `(1,2,3)`, illetve `(2,2,3)`.
3. A `x[0]` NumPy esetén nézet vagy másolat? Válasz: nézet, hacsak a kód nem készít explicit másolatot.

Visszatekintés: az 1.1. Modell, tanulás és felhasználási szerződés alfejezetben a modell bemenete X batch, a hipotézisosztály pedig a paraméterezett leképezések halmaza; a shape mindkettő szerződésének része. Az 1.4. Tanulási diagnózis és kísérleti napló diagnózisban a tenzor alakját és dtype-ját különválasztottuk: az alak a tengelyek hossza, a dtype a tárolás típusa. A seed a generátor kezdőállapota, a futásazonosító a kísérleti napló kulcsa; ebben a laborban determinisztikus adat miatt seed nem kellett, de a jövőbeli véletlen adatgeneráláshoz a protokoll rögzíti [D04].

2.1.10 Rövid összefoglaló

Az `ndarray` homogén, rögzített alakú többdimenziós tömb; a `dtype` a tárolási típust, a `shape` a tengelyhosszakat, az `axis` a redukciós irányt adja meg. A broadcasting alakszabálya jobbról balra igazítva engedi meg az 1 méretű dimenziók kiterjesztését; a `(n,)` és `(n,1)` alakok összeadása ezért `(n,n)` mátrixot ad, nem vektort. A labor a kézi és a NumPy batch-átlagot állítja elő, majd a téves broadcastingból származó hibát explicit átalakítással javítja; a laborban assertekkel ellenőrzött kulcsértékek: az átlag `[1. 1. 1.25]`, a `bad` alak `(3, 3)`, a `good` alak `(3,)`. A nézet osztozik a memórián, a másolat független; alapszeletelésnél ez eltér a Python-lista viselkedésétől.

A shape-ellenőrzés a modell bemeneti specifikációjának része 1.1. Modell, tanulás és felhasználási szerződés. Az induló becslésre a válasz: az `x + y` alakja `(4,4)`, mert a `(4,)` vektor a második, a `(4,1)` oszlopvektor az első tengelyen ismétlődik (a külső összeg lemmája).

2.1.11 Továbblépés

A 2.2. Lebegőpont és stabil elemi számítás egység a lebegőpontos ábrázolás és a stabil elemi számítások felé lép: a float32 kerekítés, a katasztrofális törlés és a log-sum-exp alapjai ott épülnek erre a tenzoralapra.

2.1.12 Források

- [D01] Astral (2026). *uv project syncing.* https://docs.astral.sh/uv/concepts/projects/sync/. Hozzáférés: 2026-10-30.
- [D04] NumPy developers (2026). *NumPy documentation: Broadcasting* és *Random sampling.* https://numpy.org/doc/stable/user/basics.broadcasting.html; https://numpy.org/doc/stable/reference/random/index.html. Hozzáférés: 2026-10-30.

2.2 Lebegőpont és stabil elemi számítás

2.2.1 Cél, előismeret és induló kérdés

Ennek az egységnek a célja, hogy a programozó a `ndarray`, `dtype`, `shape` és `axis` fogalmai után 2.1. Python, NumPy és tenzoralak megértse: a kódban tárolt szám nem ugyanaz az objektum, mint a matematikai valós szám, és a kettő közötti eltérést explicit hibamértékkel kell ellenőrizni. A tanulás végére meg tudja különböztetni a matematikai egyenlőséget a numerikus közelítő egyenlőségtől, indokolni tudja az `allclose` tűréseit, és stabilan tudja kiszámítani a $\ln \sum_i e^{x_i}$ mennyiséget nagy, illetve negatív bemenetekre. Előismeret: a Python skalár, az `np.ndarray`, a `dtype`, a broadcasting és a redukció alapjai 2.1. Python, NumPy és tenzoralak; az állítástípusok megkülönböztetése 1.2. Mitől egyetemi szintű egy modellezési állítás?.

Induló kérdés: miért lehet, hogy a `0.1 + 0.2 == 0.3` kifejezés `False` értéket ad Pythonban, miközben a három szám tizedestört-alakban pontosan ennyi? És mi lehet a helyes ellenőrzés, ha két, elvileg azonos eredményű számítást akarunk összevetni?

2.2.2 Lebegőpontos ábrázolás és a legfontosabb jelenségek

Definíció 1: float32 és float64 normális számok. A kódban használt `float32` és `float64` az IEEE 754-2019 szabvány bináris lebegőpontos formátumaira épülő, véges pontosságú ábrázolás [S04]. A *szignifikand* (más néven mantissza) a szám értékes jegyeit hordozó rész: az implicit vezető egyes és a 23 bites tört rész együtt. Egy normális `float32` szám

$$x = (-1)^s (1 + m_1 2^{-1} + m_2 2^{-2} + \cdots + m_{23} 2^{-23}) \cdot 2^{e-127},$$

ahol $s \in \{0, 1\}$ az előjelbit, $m_1, \ldots, m_{23}$ a 23 bites tört rész, e pedig 8 bites, $1 \le e \le 254$ kitevőmező. A kettes számrendszerbeli vezető 1 implicit, ezért összesen 24 bites pontosságról szokás beszélni. A `float64` analóg: 1 előjelbit, 11 bites kitevő, 52 bites explicit tört, 53 bites

teljes pontosság, kitevőeltolás 1023. A nullához közeli, nem normális számok a szubnormális tartományt alkotják, amelyet itt nem használunk részletesen. A `float32` bitmezői:

mező	bitek (`float32`)	jelentés
előjel	1	s: 0 nemnegatív, 1 negatív
kitevő	8	e: eltolt kitevő, normális számokra $1 \le e \le 254$
tört	23	$m_1, \dots, m_{23}$: a vezető egyes utáni jegyek

A normális tartományban a számegyenesen a $[2^k, 2^{k+1})$ intervallumban a reprezentálható értékek egyenletes, 2^{k-23} lépésközű rácsot alkotnak; az ulp tehát a kettes alapú nagyságrenddel kettőződik, és például $[2^{24}, 2^{25})$ között 2, $[2^{25}, 2^{26})$ között 4.

Jelölések és alapértékek. Legyen x a pontos valós szám, $\tilde{x}$ a reprezentált érték. A gépi epszilon (`np.finfo(...).eps`) az 1 és az utána következő ábrázolható szám távolsága, azaz $\varepsilon = \mathrm{nextafter}(1, +\infty) - 1$; legközelebbi értékre kerekítésnél az egységnyi kerekítési hibakorlát ennek fele, $u = \varepsilon/2$:

$$\varepsilon_{32} = 2^{-23} \approx 1{,}1920929 \cdot 10^{-7}, \qquad \varepsilon_{64} = 2^{-52} \approx 2{,}2204460 \cdot 10^{-16}, \qquad u = \tfrac{1}{2}\varepsilon.$$

Az egységnyi kerekítési lépés, az ulp (unit in the last place) normális tartományban

$$\mathrm{ulp}(x) = 2^{\lfloor \log_2 |x| \rfloor + 1 - p},$$

ahol $p = 24$ `float32` és $p = 53$ `float64` esetén. A legnagyobb normális `float32` érték $(2 - 2^{-23}) \cdot 2^{127} \approx 3{,}4028235 \cdot 10^{38}$, míg a legnagyobb `float64` érték $(2 - 2^{-52}) \cdot 2^{1023} \approx 1{,}7976931 \cdot 10^{308}$. A legkisebb pozitív normális `float32` $2^{-126} \approx 1{,}1754944 \cdot 10^{-38}$.

Definíció 2: túlcsordulás, alulcsordulás, kiejtés, NaN.

- *Túlcsordulás (overflow)*: ha egy közbülső vagy végső eredmény abszolút értéke meghaladja az adott dtype legnagyobb véges értékét, az eredmény `inf` vagy `-inf` lesz. Például `np.exp(np.float32(1000))` túlcsordul, mert $\ln(3{,}4028235 \cdot 10^{38}) \approx 88{,}7228$, tehát e^{1000} már nem fér el `float32`-ben.
- *Alulcsordulás (underflow)*: ha a nemnulla eredmény abszolút értéke kisebb a legkisebb normálisnál, az ábrázolás szubnormálissá, majd nullává válik; ezzel relatív pontosság veszhet el.
- *Kiejtés (cancellation, katasztrofális kivonás)*: ha két közeli, nagy abszolút értékű számot vonunk ki, a bemenetek tárolási hibája változatlan abszolút nagyságú marad, miközben az eredmény kicsi lesz, ezért az eredmény relatív hibája megnőhet.
- *NaN* (*not a number*): érvénytelen művelet, például `np.float64(0.0) / np.float64(0.0)` (figyelmeztetéssel) vagy `np.inf - np.inf` eredménye. A tiszta Python `0.0 / 0.0` nem NaN-t ad, hanem `ZeroDivisionError`-t dob; a NaN-t a NumPy skalár- vagy tömbaritmetika állítja elő. A NaN nem egyenlő önmagával: `np.nan == np.nan` `False`; az ellenőrzésre az `np.isnan` való.

Példa a kerekítésre. A `0.1` tizedestört nem véges kettes tizedestört, ezért sem `float32`, sem `float64` nem tárolja pontosan. Emiatt a `0.1 + 0.2 == 0.3` kifejezés `False` lehet: a három tizedestört külön-külön kerekített közelítése, és a kerekítések után a kettes számrendszerbeli összeg nem pontosan a `0.3` kerekített értéke. A jelenség nem hiba a hardverben, hanem a véges ábrázolás következménye.

2.2.3 Abszolút és relatív hiba levezetése egy kis példán

Definíció 3: hibamértékek. Legyen x a pontos matematikai érték, $\tilde{x}$ a kódban kapott érték. Az *abszolút hiba*

$$\Delta_a = |x - \tilde{x}|,$$

a *relatív hiba* pedig $x \neq 0$ esetén

$$\Delta_r = \frac{|x - \tilde{x}|}{|x|}.$$

Ha $x = 0$, a relatív hiba nem értelmes; ilyenkor abszolút hibát vagy nullához közeli nevezővel stabilizált mennyiséget kell használni.

Vizsgáljuk meg a `float32` összeadás $a = 2^{24}$ és $b = 1$ példáján. A pontos matematikai összeg $x = 2^{24} + 1 = 16\,777\,217$. A `float32` ábrázolásban 2^{24} pontosan megvan, mert a 24 bites szignifikandban az egyetlen vezető egyes és utána 23 nulla éppen elfér. A következő ábrázolható érték ezen a tartományon $2^{24} + 2$, tehát

$$\mathrm{ulp}(2^{24}) = 2^{24-23} = 2.$$

A $2^{24} + 1$ pontosan félúton van a két reprezentálható szám között. A leggyakrabban használt kerekítési szabály, a legközelebbi értékre kerekítés döntetlennél a páros alsó bittel rendelkező értéket választja; itt ez a 2^{24} érték. Ezért a tárolt összeg

$$\tilde{x} = 2^{24} = 16\,777\,216.$$

Az abszolút hiba tehát

$$\Delta_a = |16\,777\,217 - 16\,777\,216| = 1,$$

a relatív hiba pedig

$$\Delta_r = \frac{1}{16\,777\,217} \approx 5{,}9604641 \cdot 10^{-8}.$$

Ezt kétféleképpen is értelmezhetjük. Egyrészt a hiba 0,5 ulp, mert az ulp most 2, és $1 = 0{,}5 \cdot 2$. Másrészt a relatív hiba közelítőleg 2^{-24}, azaz a `float32` gépi epszilon felének ($u = \varepsilon/2 \approx 5{,}96 \cdot 10^{-8}$) megfelelő nagyságú; pontosan $\Delta_r = 1/(2^{24} + 1)$, ami 2^{-24}-nél kissé kisebb. A levezetésből látszik a két hibamérték eltérő szerepe: az abszolút hiba értékfüggő, nagyobb számnál nagyobb lehet, a normális tartományban a szabályos kerekítés relatív hibája viszont korlátos; a példában a korlát 2^{-24}.

Korlát mint alkalmazási szabály, nem általános tétel. A fenti megfigyelés a normális tartományra és a legközelebbi értékre kerekítésre vonatkozik. Szubnormális számokra és túlcsorduló eredményekre nem ad ilyen relatív garanciát. A gyakorlatban ezért nem a gépi epszilon önmagában, hanem az adott számítás skálájához igazított `atol` és `rtol` a helyes ellenőrzési eszköz.

2.2.4 Stabil exponenciális összegzés: logsumexp előkép

A többosztályos modellek későbbi tárgyalásához szükségünk lesz a log-sum-exp műveletre. A *logit* a softmax előtti, normalizálatlan pontszám (a modell nyers kimenete); a logitvektor K osztály pontszámait tartalmazza. Legyen $z = (z_1, \dots, z_K) \in \mathbb{R}^K$ egy logitvektor. A *softmax* a logitvektort valószínűségvektorrá képezi: $\hat{p}_i = e^{z_i} / \sum_{j=1}^{K} e^{z_j}$. Ekkor

$$L(z) = \ln \left(\sum_{i=1}^{K} e^{z_i} \right).$$

Ha a z_i értékek nagyok, például 1000 körüliek, a naiv `np.exp` hívás `float32`-ben túlcsordul. A cél egy olyan átalakítás, amely ugyanazt a matematikai értéket adja, de véges közbülső értékekkel.

Állítás 1: a max-levonásos stabil alak. Legyen $K \geq 1$, minden z_i véges, és legyen $m = \max_i z_i$. Ekkor

$$L(z) = m + \ln \left(\sum_{i=1}^{K} e^{z_i - m} \right).$$

Levezetés. Az e^{z_i} tagok pozitívak, ezért az összeg pozitív. Az exponenciális szorzási azonossága alapján

$$\sum_{i=1}^{K} e^{z_i} = \sum_{i=1}^{K} e^{m + (z_i - m)} = e^m \sum_{i=1}^{K} e^{z_i - m}.$$

A természetes logaritmus a szorzatot a tényezők logaritmusának összegére bontja: pozitív a, b-re $\ln(ab) = \ln a + \ln b$, ezért

$$\ln \left(e^m \sum_{i=1}^{K} e^{z_i - m} \right) = \ln(e^m) + \ln \left(\sum_{i=1}^{K} e^{z_i - m} \right).$$

Mivel $\ln(e^m) = m$, a keresett azonosságot kapjuk. A levonás után minden $z_i - m \leq 0$, tehát minden $e^{z_i - m}$ érték a $(0, 1]$ tartományba esik, az összeg $[1, K]$-ba, a logaritmus pedig $[0, \ln K]$-ba. Ezért az exponenciális túlcsordulás elkerülhető, feltéve, hogy m maga véges.

Megjegyzés a tartományokról. Ha a legnagyobb logit is `inf`, a stabil formula nem segít; az `inf` bemenet vagy korábbi numerikus hiba, vagy olyan logit, amelyet előbb kezelni kell. Ha a logitok között hatalmas a különbség, a kis tagoknál az $e^{z_i - m}$ alulcsordulhat nullává. Ez a végösszeget legfeljebb $K \cdot 2^{-149}$ abszolút értékkel változtatja meg (a `float32` legkisebb szubnormálisa 2^{-149}), miközben a levonás után a legnagyobb tag $e^0 = 1$, tehát az összeg legalább 1; a hiba így legfeljebb $K \cdot 2^{-149}$ relatív, a logaritmuson pedig ennél is kisebb. Az alulcsordulás tehát tudatos, de elhanyagolható korlát, amíg K nem óriási.

2.2.5 Kézi kis számpélda

Legyen $K = 3$ és $z = (1000, 1001, 1002)$. A pontos matematikai eredmény `float64`-ben:

$$e^{-2} = 0{,}1353352832366127, \quad e^{-1} = 0{,}36787944117144233, \quad e^{0} = 1,$$

összegük 1,5032147244080552, ennek természetes logaritmusa 0,4076059644443803, így

$$L = m + 0{,}4076059644443803 = 1002{,}4076059644444.$$

A naiv út `float32`-ben: `np.exp(np.float32(1000))` `inf`, az összeg `inf`, a logaritmus `inf`. A stabil út `float32`-ben $m = 1002$, a levonás `[-2, -1, 0]` tartományú, az eredmény körülbelül 1002,4076 lesz. A `float32` kerekítés okozta abszolút hiba körülbelül az ulp feléig terjedhet; 1002 körül ($\lfloor \log_2 1002 \rfloor = 9$) az ulp $2^{9-23} = 2^{-14} = 0{,}00006103515625$, a fél ulp 0,000030517578125, tehát a kerekítési eltérés jellemzően 10^{-5} nagyságrendű. A `float64` referencia és a `float32` stabil eredmény összevetéséhez ezért az abszolút tűrést az ulp nagyságrendjéhez kell igazítani.

2.2.6 Labor: naiv és stabil exponenciális összegzés

Előfeltétel: Python 3.12 és `uv`; a laborcsomag `lab/` projektje, NumPy a lock fájlban rögzített verzióval [D01]. A labor kódja kis CPU-s NumPy-környezetben fut. Nincs véletlen seed, a műveletek determinisztikusak; a `dtype` explicit `float32`, az eszköz a CPU. A laborcsomag gyökeréből a parancs:

```
cd lab
uv sync
uv run python n006/lab_logsumexp.py
```

```
# lab/n006/lab_logsumexp.py - naiv és stabil logsumexp nagy logitokon
import numpy as np

# Bemenet: 1-D float32 logitvektor
z = np.array([1000.0, 1001.0, 1002.0], dtype=np.float32)

def naive_logsumexp(v):
    # v alakja (K,), float32; itt overflow várható
    return np.log(np.sum(np.exp(v)))

def stable_logsumexp(v):
    # v alakja (K,), float32
    m = np.max(v)                 # max-logit, skalár float32
    shifted = v - m               # (K,) float32, minden elem <= 0
    return m + np.log(np.sum(np.exp(shifted)))

# float32 eredmények
naive = naive_logsumexp(z)
stable = stable_logsumexp(z)

# float64 referencia, a NumPy beépített stabilitását használva
ref64 = np.logaddexp.reduce(z.astype(np.float64))

print("naive:", naive)
print("  finite:", bool(np.isfinite(naive)))
print("stable:", stable)
print("  finite:", bool(np.isfinite(stable)))
print("ref64: ", ref64)
```

```
# Ellenőrzés: a stabil float32 érték véges legyen, és
# a float64 referenciához képest a skálához illő tűréssel egyezzen.
assert np.isfinite(stable), "A stabil eredmény nem véges."
assert np.allclose(stable, ref64, rtol=1e-6, atol=2e-4), (
    "A stabil float32 eredmény nem esik a megadott tűrésen belülre."
)

# Negatív nagy logitokon a naiv út alulcsordul: minden exp() nulla lesz,
# ezért log(0) = -inf; a NumPy RuntimeWarningot ad
z2 = np.array([-1000.0, -1001.0, -1002.0], dtype=np.float32)
naive_neg = naive_logsumexp(z2)   # RuntimeWarning: divide by zero encountered in log
print("naive negatív:", naive_neg)
print("stable negatív:", stable_logsumexp(z2))
print("finite negatív:", bool(np.isfinite(stable_logsumexp(z2))))
```

MÉRT KIMENET (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; a műveletek determinisztikusak, seed nem kell); a konkrét utolsó jegyek a NumPy-verziótól és a kerekítéstől függhetnek:

```
naive: inf
  finite: False
stable: 1002.4076
  finite: True
ref64:  1002.4076059644444
naive negatív: -inf
stable negatív: -999.5924
finite negatív: True
```

A negatív ág a naiv útvonal másik hibamódját mutatja meg: ott nem túlcsordulás, hanem alulcsordulás történik, mert e^{-1000} `float32`-ben már 0, és $\ln 0 = -\infty$; a NumPy ehhez `RuntimeWarning: divide by zero encountered in log` figyelmeztetést ad. A mért futás stderr kimenetén a két figyelmeztetés `overflow encountered in exp`, illetve `divide by zero encountered in log` szöveggel jelenik meg. A stabil útvonal mindkét irányban véges marad.

Az `np.allclose` ellenőrzés az alábbi egyenlőtlenséget vizsgálja elemenként [D04]:

$$|a - b| \leq \text{atol} + \text{rtol} \cdot |b|.$$

Ebben az ellenőrzésben b a `float64` referencia. Mivel b körülbelül 1002,4, a relatív tag $10^{-6} \cdot 1002{,}4 \approx 10^{-3}$, ezért a teljes tűrés $2 \cdot 10^{-4} + 1{,}0024 \cdot 10^{-3} \approx 1{,}2 \cdot 10^{-3}$, és a relatív tag dominál; ez 1002 körül mintegy 20 ulp ($2^{-14} \approx 6{,}1 \cdot 10^{-5}$), tehát a tűrés a `float32` ulp-skálájához igazodik. Az `atol=2e-4` csak kis $|b|$ esetén válik meghatározóvá. A tanulság: nem az `atol` és az `rtol` közül kell az egyiket „szigorúbbnak” nevezni, hanem a kettő együttes, $|b|$-vel skálázódó hatását kell az ulp nagyságrendjéhez mérni. A negatív bemenetnél a naiv út alulcsordul ($\ln 0 = -\infty$), a stabil út viszont a max-levonás miatt véges marad.

A labor értelmezése: a naiv alak `inf`-et ad, tehát a matematikai érték elveszett; a stabil alak véges, és a magasabb pontosságú referenciával való egyezése a skálához illő abszolút és relatív hibahatáron belül van. Ez a numerikus egyezés implementációs ellenőrzés, nem a matematikai azonosság bizonyítéka; az azonosságot az Állítás 1 levezetése bizonyítja.

2.2.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért lehet egy matematikailag igaz egyenlőség a kódban hamis. Térjen ki a tárolt érték, a kerekítés és az `allclose` szerepére. Adjon egy saját kis példát is.

F2 (levezetési, nehézség 2). A `float32` ábrázolásban $2^{25} = 33\,554\,432$, az ulp ezen a tartományon $2^{25-23} = 4$. Mi lesz a tárolt érték, ha a `float32` reprezentációban kiszámítjuk $2^{25} + 1$-et? Vezesse le az abszolút és relatív hibát.

F3 (implementációs, nehézség 2). Írjon `stable_softmax` függvényt, amely egy tetszőleges `(N, K)` alakú `float32` logitmátrix minden sorára kiszámítja a stabil softmaxot. A függvény először vonja le a sor maximumát, majd normalizáljon. Ellenőrizze, hogy minden sor összege 1-hez közeli, és a `[[1000, 1000], [0, 1]]` bemenetre a kimenet `[[0.5, 0.5], [1/(1+e), e/(1+e)]]` közelítő értéke.

F4 (hibakeresési, nehézség 2). Az alábbi kód `float32`-ben egy varianciaszámítást végez:

```
x = np.array([1_000_000.0, 1_000_000.1, 1_000_000.2], dtype=np.float32)
mean_x = np.mean(x)
var_naive = np.mean(x * x) - mean_x * mean_x
print(var_naive)
```

Mi a matematikai variancia az adott három értékre, és miért kaphat a kód hibás, akár negatív eredményt? Javítsa a számítást központosítással, és adja meg a helyes ellenőrzést `float64` referenciával.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy kis protokollt annak eldöntésére, hogy egy saját `float32` függvény és a `float64` referencia implementációja egy adott bemeneti tartományon elfogadhatóan egyezik-e. A protokollban adja meg: a bemeneti vektorok számát és tartományát, a megfigyelési egységet, a kiszámított hibamérőket, a hibahatár indoklását és a döntési szabályt. A feladat nem egy előre megjósolt győztest, hanem egy lefuttatható tervet kér.

2.2.8 Megoldások és indoklás

F1 megoldása. A matematikai egyenlőség a valós számokon értett azonosság. A kódban a számok véges `dtype`-beli közelítések, a műveletek pedig ezeken a közelítéseken hajtódnak végre, ezért a kerekítési hibák műveletenként halmozódnak. Az `0.1 + 0.2 == 0.3` példában a `0.1`, `0.2` és `0.3` egyike sem pontos kettes számrendszerbeli tört, és a közelítő összeg nem pontosan a `0.3` kerekített közelítése. A helyes ellenőrzés nem a pontos egyenlőség, hanem az alkalmazás skálájához illő abszolút és relatív tűréssel végzett összevetés, például `np.allclose(a, b, rtol=..., atol=...)`. Ugyanakkor az `allclose` használata sem bizonyít matematikai azonosságot; csak azt jelenti, hogy az adott bemeneteken a két implementáció közelítőleg egyezik.

F2 megoldása. A pontos matematikai összeg $x = 2^{25} + 1 = 33\,554\,433$. A $[2^{25}, 2^{26})$ tartományban az ulp $2^{25-23} = 4$, tehát a reprezentálható számok $2^{25}, 2^{25} + 4, 2^{25} + 8, \ldots$ A $2^{25} + 1$ távolsága 2^{25}-től 1, a következő reprezentálható értéktől, $2^{25} + 4$-től pedig 3. A legközelebbi értékre kerekítés ezért a 2^{25} értéket választja, mert a $2^{25} + 1$ ehhez van közelebb; döntetlen nincs, így a páros alsó bit szabálya nem lép közbe:

$$\tilde{x} = 2^{25} = 33\,554\,432.$$

Az abszolút hiba:

$$\Delta_a = |x - \tilde{x}| = |33\,554\,433 - 33\,554\,432| = 1,$$

a relatív hiba pedig

$$\Delta_r = \frac{1}{33\,554\,433} \approx 2{,}9802322 \cdot 10^{-8}.$$

Alakellenőrzés: az abszolút hiba 1, ami az ulp negyede; a relatív hiba kisebb, mint a `float32` kerekítési korlátja $u = \varepsilon/2 \approx 5{,}96 \cdot 10^{-8}$, összhangban a normális tartomány kerekítési korlátjával. Szélsőeset: ha a bemenet pontosan 2^{25}, a hiba nulla; ha a bemenet $2^{25} + 2$ lenne, az pontosan félúton állna két reprezentálható szám között, és a páros alsó bit szabálya döntene.

F3 megoldása. A *softmax* a logitvektort valószínűségekre képezi: $\hat{p}_i = e^{z_i} / \sum_j e^{z_j}$, ahol $\hat{p}_i > 0$ és $\sum_i \hat{p}_i = 1$. A stabil alak minden z_i logitra, $m = \max_j z_j$ mellett

$$\hat{p}_i = \frac{e^{z_i - m}}{\sum_{j=1}^{K} e^{z_j - m}}.$$

A függvény:

```
import numpy as np

def stable_softmax(Z):
    # Z alakja (N, K), float32
    m = np.max(Z, axis=1, keepdims=True)       # (N, 1)
    E = np.exp(Z - m)                          # (N, K)
    return E / np.sum(E, axis=1, keepdims=True) # (N, K)

Z = np.array([[1000.0, 1000.0],
              [0.0, 1.0]], dtype=np.float32)
P = stable_softmax(Z)
print(P)
print("Sorösszegek:", np.sum(P, axis=1))

# Ellenőrzés: a sorösszegek 1-hez közeliek, és az ismert értékek is stimmelnek
np.testing.assert_allclose(np.sum(P, axis=1), 1.0, atol=1e-6)
expected = np.array([[0.5, 0.5],
                     [1.0 / (1.0 + np.e), np.e / (1.0 + np.e)]],
                    dtype=np.float32)
np.testing.assert_allclose(P, expected, rtol=1e-6, atol=1e-6)
```

MÉRT KIMENET (Python 3.12.3, NumPy 2.5.4, CPU):

```
[[0.5       0.5      ]
 [0.2689414 0.7310586]]
Sorösszegek: [1. 1.]
```

`float32` kerekítéssel a sorösszeg legfeljebb néhány ulp-nyit térhet el 1-től, ezért a fenti `atol=1e-6` ellenőrzés indokolt. A stabilizálás nélkül az első sor `inf/inf` NaN-t adna, mert e^{1000} `float32`-ben túlcsordul.

F4 megoldása. A megadott tizedes értékekre a matematikai variancia az átlag 1 000 000,1 és az eltérések $-0{,}1$; 0; 0,1 mellett $\frac{0{,}01+0+0{,}01}{3} = \frac{0{,}02}{3} \approx 0{,}0066667$. A `float32` azonban a bemenetet előbb kvantálja: $\mathrm{ulp}(10^6) = 0{,}0625$, ezért a tárolt értékek 1 000 000, 1 000 000,125 és 1 000 000,1875 (a 0,1 a 0,125-re, a 0,2 a 0,1875-re kerekítődik). A naiv $\mathrm{E}[x^2] - \mathrm{E}[x]^2$ képlet az $x \cdot x$ szorzatokat 10^{12} nagyságrendű `float32` értékekként tárolja, ahol az ulp már 65 536; a kivonás után a naiv eredmény ezekre az értékekre 65 536,0, ami több mint hét nagyságrenddel nagyobb a keresett varianciánál (más bemenetre negatív is lehet). A központosított változat előbb kivonja az átlagot, így a nagy számok kiesnek; a `float32` eredmény 0,0078125. Ez sem egyezik pontosan a 0,0066667-tel, mert a bemenet már kvantált, és a `float32` átlag kerekítése miatt az eltérések $-0{,}0625$; 0,0625; 0,125; a `float64` referencia ugyanezeken a tárolt értékeken 0,0060764.

A központosítás tehát a kiejtést szünteti meg, a bemeneti kvantálást nem. Az ellenőrzés ezért a `float64` referenciához képest csak a kvantálásból származó hibabecsléssel végezhető: a fél ulpnyi átlagkerekítés miatt itt néhány 10^{-3} eltérés várható, ezért $2 \cdot 10^{-2}$ abszolút korlát indokolt, 10^{-6} nem:

```
x = np.array([1_000_000.0, 1_000_000.1, 1_000_000.2], dtype=np.float32)
d = x - np.mean(x)
var_center = np.mean(d * d)
x64 = x.astype(np.float64)
ref = np.mean((x64 - np.mean(x64)) ** 2)
print("javított:", var_center)        # 0.0078125
print("referencia:", ref)             # 0.0060763889
assert var_center > 0, "A központosított variancia nem negatív."
assert abs(var_center - ref) < 2e-2, (
    "A float32 variancia eltérése a bemeneti kvantálás miatt nagyobb a megengedettnél."
)
```

F5 megoldásmintája. Egy lehetséges protokoll:

- *Kérdés*: a saját `float32` függvény és a `float64` referencia adott tartományon egyezik-e.
- *Bemenetek*: 40 determinisztikusan generált, egyenként $K = 20$ elemű logitvektor, a $z_{r,k} = -200 + 400 \cdot \frac{(37r+61k) \bmod 1001}{1000}$ szabállyal ($r = 0, \dots, 39$, $k = 0, \dots, 19$), tehát az elemek $[-200, 200]$-ból valók; a determinisztikus rács miatt seed nem szükséges.
- *Megfigyelési egység*: egy vektor; a hibajelentés minden vektorra a maximális abszolút hiba és a maximális stabilizált relatív hiba, ahol a stabilizált relatív hiba $\Delta_r = \frac{|a-b|}{|b|+10^{-12}}$; a 10^{-12} csak a $b = 0$ esetben véd a nullával osztástól, a logsumexp körülbelül 1000 nagyságú kimenetén nincs gyakorlati hatása.
- *Hibahatár*: a `float32` relatív kerekítési hibája a normális tartományban 2^{-24} körüli, ezért 10^{-6} relatív határ kényelmes; az abszolút határ 10^{-4}, mert a legnagyobb logit 200, ahol $\mathrm{ulp}(200) = 2^{-16} \approx 1{,}5 \cdot 10^{-5}$, és a 10^{-4} ennek mintegy 6,5-szerese, ami fedezi a max-levonás és a logaritmus néhány ulpnyi hibáját.
- *Döntés*: ha mind a 40 vektorra mindkét hibamérő a határ alatt van, akkor az implementáció az adott tartományon elfogadott; ha bármelyik vektoron valamelyik hibamérő meghaladja a határt, akkor az implementációt vagy a tartomány érvényességét kell vizsgálni.
- *Korlát*: az eredmény csak a megadott tartományra és dtype-ra érvényes; más platformon, könyvtárverzióval vagy gyorsított kernelszámítással a kerekítési sorrend eltérhet, ezért a toleranciát ott újra kell igazolni [D04].

2.2.9 Tipikus tévedés ellenpéldával

Az algebrailag ekvivalens képletek nem feltétlenül numerikusan egyenértékűek. Tekintsük a

$$f_1(x) = \sqrt{x+1} - \sqrt{x}, \qquad f_2(x) = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

azonosságot. Matematikailag $f_1(x) = f_2(x)$ minden $x \geq 0$-ra, mert a második az első gyöktelenített alakja. Nagy x-re `float32`-ben a $\sqrt{x+1}$ és $\sqrt{x}$ értékek kerekítve egyenlővé válhatnak, így f_1 nullát adhat, miközben a pontos érték pozitív. Konkrétan $x = 10^8$ esetén $\mathrm{ulp}(10^8) = 8$, ezért a `float32`-ben tárolt $x+1$ még x-szel egyenlő; a két négyzetgyök egyaránt 10^4, így $f_1 = 0$, holott a pontos érték $1/(\sqrt{10^8+1} + \sqrt{10^8}) \approx 5 \cdot 10^{-5}$. A második alak nem végez közeli értékű tagok kivonását, ezért $f_2 \approx 5 \cdot 10^{-5}$ `float32`-ben is. Az ellenpélda tanulsága: az ekvivalens átalakítás a matematikai objektumon nem jelenti azt, hogy a lebegőpontos implementációk is egyformán stabilak.

2.2.10 Önellenőrzés és korábbi fogalmak áttekintése

Az abszolút hiba mérési skálától függ, a relatív hiba a mérési skálát kiküszöböli, de nullánál nem értelmes. A `np.allclose` két tűrést kombinál, és a `b` értékkel arányos relatív tűrést használ. Ha egy ellenőrzés `inf` vagy NaN értéket tartalmaz, az `np.allclose` viselkedése nem egységes: két azonos előjelű `inf` értéket alapértelmezésben egyenlőnek tekint (`np.allclose(np.inf, np.inf)` igaz), a NaN-t viszont `equal_nan=False` mellett még önmagával sem tartja egyenlőnek, ezért `False`-t ad. A végesség ellenőrzését ezért külön kell elvégezni: a `np.isfinite` (illetve a `np.isnan`) szűri ki a nem véges értékeket, mielőtt az `allclose`-ra kerül a sor. A korábbi fogalomhoz visszatérve: a `dtype` és a `shape` előzetes rögzítése nélkül a numerikus hiba sem lokalizálható 2.1. Python, NumPy és tenzoralak. A numerikus teszt adott bemenetre vonatkozó empirikus állítás; a matematikai azonosság bizonyítása a levezetés típusú állítás 1.2. Mitől egyetemi szintű egy modellezési állítás?.

2.2.11 Rövid összefoglaló

A `float32` és `float64` véges pontosságú ábrázolás, a hibát abszolút és relatív értékkel mérjük. A `0.1 + 0.2 == 0.3` hamis volta a tizedestört kerekítésének következménye. Az overflow, a katasztrofális kivonás, a NaN és az ulp fogalma megmagyarázza, miért nem alkalmas a pontos egyenlőség a lebegőpontos eredmények ellenőrzésére, és miért kell toleranciás összevetést (például `allclose`) használni. A stabil logsumexp a maximum levonásával elkerüli az exponenciális túlcsordulást, és a kis példában véges 1002,4076 értéket ad az `inf` helyett. A numerikus egyezés implementációs ellenőrzés; a matematikai eredményt levezetés igazolja, a toleranciát pedig az adott skála és dtype kerekítési tulajdonságaihoz kell indokolni.

2.2.12 Továbblépés

A 2.3. Izolált környezet, notebook és verzió egység izolált Python-környezetet, notebook-használatot és verziózást tárgyal. Az ottani verziórögzítés és újrafuttathatóság teszi ellenőrizhetővé az itt bemutatott tolerancia- és végesérték-teszteket; a környezetváltozás ugyanis ugyanolyan hibát okoz, mint egy megváltozott `dtype`.

2.2.13 Források

- [D01] Astral (2026). *uv project syncing*. https://docs.astral.sh/uv/concepts/projects/sync/. Hozzáférés: 2026-10-30.
- [D04] NumPy developers (2026). *NumPy documentation: numpy.allclose*. https://numpy.org/doc/stable/reference/generated/numpy.allclose.html. Hozzáférés: 2026-10-30.
- [S04] IEEE (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*. IEEE. A `round half to even` konvenciót a szabvány rögzíti.

2.3 Izolált környezet, notebook és verzió

2.3.1 Cél, előismeret és indítókérdés

Ebben az egységben nem a modellt, hanem a modellt futtató környezetet vizsgáljuk. A 2.2. Lebegőpont és stabil elemi számítás egységben láttuk, hogy egy lebegőpontos eredmény csak toleranciával értelmezhető; most azt a kérdést tesszük fel, hogy egyáltalán *melyik* Python hajtotta

végre a számítást, *milyen* csomagverziókkal és *milyen* kernelben. Induló kérdés: egy notebook cellája `0.91` pontosságot mutat, de a cella tegnap futott. Milyen további adat kell ahhoz, hogy ezt a jelenlegi kód eredményének tekinthessük? A válasz a fejezet végére áll össze: rögzíteni kell az interpretert és a csomagprofilt, ismerni kell a notebookhoz tartozó kernelt, és a kimenetet Restart & Run All után kell elmenteni; ha ezek bármelyike hiányzik, a `0.91` érték nem tulajdonítható a jelenleg látható kódnak.

Előismeret: 2.2. Lebegőpont és stabil elemi számítás fogalmai (`float32`/`float64`, `allclose`, tolerancia), valamint alapvető parancssori és fájlkezelési ismeret. Nem feltételezünk csomagkezelői vagy notebook-szerver üzemeltetői gyakorlatot.

2.3.2 Interpreter, kernel és csomagprofil

A három fogalom különböző rétegre vonatkozik, ezért pontosan definiáljuk őket.

Definíció (Python interpreter). A Python interpreter az a folyamat, amely a Python bytecode-ot végrehajtja. Egy futó Python program számára az interpreter elérési útja a `sys.executable` karakterlánc. Ugyanazon a gépen több interpreter is létezhet, például a rendszer `/usr/bin/python3` és egy projekt `.venv/bin/python` fájlja; a kettő különböző csomagkörnyezetet láthat.

Definíció (Jupyter kernel). A Jupyter kernel egy háttérfolyamat, amely egy adott interpreterrel hajtja végre a notebook celláinak kódját, és a notebook szerverrel kommunikál. A notebook dokumentum (`.ipynb`) csak cellákat, kimeneteket és metaadatokat tárol; a kernel nem a notebook része.

Definíció (csomagprofil). A csomagprofil a futtatáshoz specifikált csomagok és azok verzióinak zárt halmaza. Egy lock fájl (függőségzáró fájl, angolul *lock file*), például `uv.lock`, a közvetlen függőségek mellett a tranzitív függőségeket is rögzíti, ezért két `uv sync` futtatás elvileg ugyanazt a profilt állítja elő.

A három réteget a következő példa választja el. Tegyük fel, hogy a notebook kernelt `python3` néven indítottuk, de a projekt interpretere `.venv/bin/python`. A notebook cellája ekkor a *kernel* interpreterével fut, nem feltétlenül azzal, amelyet a parancssorban `uv run python` hív. A pontos állapotot a notebookban is a `sys.executable` mutatja; ha a cella más interpreterrel fut, a `numpy` verzió is eltérhet.

A három réteg viszonyát a következő táblázat foglalja össze.

Réteg	Mit ír le	Hol ellenőrizhető
interpreter	a bytecode-ot végrehajtó folyamat	`sys.executable`
kernel	a notebook celláit futtató háttérfolyamat	`jupyter kernelspec list`, a Jupyter Kernel menüje
csomagprofil	a csomagok és verzióik zárt halmaza	`uv.lock`, `pip freeze`

2.3.3 Reprodukciós feltételek levezetése

A fejezet bizonyítási feladata: milyen feltételek mellett várjuk, hogy azonos input azonos ellenőrző számítást adjon? Ehhez formalizáljuk a futás fogalmát.

Legyen P egy *determinisztikus program*, azaz olyan program, amelynek minden állapotátmenete csak a megelőző állapottól és a bemenettől függ; nincs benne rejtett véletlen forrás, időbélyeg vagy külső hálózati válasz. Legyen I a bemenet, $E = (e, V, B)$ a környezeti leíró, ahol

- e az interpreter azonosítója és verziója,
- V a csomagprofil, azaz a releváns könyvtárak verzióinak halmaza,
- B a hardver- és buildleíró, például CPU-utasításkészlet, a rendelkezésre álló szálak száma és natív könyvtárfordítás.

Jelölje $P_E(I)$ a program kimenetét az E környezetben. A reprodukálhatóság alapesete a következő.

Lemma (azonos környezet, azonos kimenet). Ha P determinisztikus, akkor rögzített E és I mellett $P_E(I)$ minden futásban ugyanaz az érték.

Bizonyításvázlat. A futás állapotsorozatát $s_0, s_1, \ldots, s_T$ jelöli, ahol s_0 a kezdőállapot az I bemenettel. A determinisztikusság miatt minden t lépéshez van olyan F_t átmenetfüggvény, hogy $s_{t+1} = F_t(s_t)$, és F_t csak a programkódtól és E-től függ. Mivel E és I rögzített, s_0 rögzített; teljes indukcióval minden s_t rögzített, így $P_E(I) = s_T$ is. A bizonyítás erőssége, hogy a környezeti leíró *minden* befolyásoló komponensét rögzítettük; ha bármelyik rejtve változik, a lemma feltétele nem teljesül. □

A lemma feltétele szigorú: a program állapotátmenete nem függhet futásidejű ütemezéstől. Egy többszálas redukció eredménye azonos E mellett is eltérhet, mert a szálak összeadási sorrendje nem része a környezeti leíró rögzített adatainak; az ilyen program a lemma értelmében nem determinisztikus. A determinisztikusság helyreállítható a redukció sorrendjének rögzítésével, például egy szálon futtatva vagy determinisztikus összegzési sorrendet használva; a B komponens a szálak számát és a buildet írja le, nem a futásidejű ütemezést.

A lemma azonban nem ígér bitazonosságot, ha B változik. Ugyanaz a Python- és könyvtárverzió két különböző CPU-n más lebegőpontos utasítást, más összegzési (reduction-) sorrendet vagy más gyorsított könyvtárat használhat. A reduction-sorrend az összegzés lépéseinek sorrendje; a lebegőpontos összeadás nem asszociatív, ezért a sorrend az eredmény utolsó jegyeit befolyásolhatja. A [D02] dokumentáció kifejezetten rögzíti, hogy azonos seed sem garantál platformok és verziók közötti bitazonos eredményt. Ezért a gyakorlati reprodukciós szerződés nem pontos egyenlőséget, hanem a 2.2. Lebegőpont és stabil elemi számítás egységből ismert toleranciás egyezést követel meg:

$$|\tilde{x} - x| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}}\,|x|\,,$$

ahol x a referenciaként elfogadott érték, $\tilde{x}$ az új környezetben kapott érték, ϵ_{abs} és ϵ_{rel} pedig a számítás skálájához és dtype-jához indokolt korlát. `float64` gradienseket ellenőrző kis konvolúciós példán $\epsilon_{\text{abs}} = 10^{-11}$, $\epsilon_{\text{rel}} = 10^{-9}$ észszerű középső differenciás ellenőrzéshez. Az indoklás a labor `conv_reference.py` középső differenciájára támaszkodik: ott $h = 10^{-5}$, így a csonkítási hiba $O(h^2|f'''|)$, a kerekítési hiba pedig $O(\varepsilon_{\text{mach}}|f|/h)$ nagyságrendű, ahol $\varepsilon_{\text{mach}} \approx 2{,}2 \cdot 10^{-16}$. Ezekkel a csonkítási tag 10^{-11}–10^{-10}, a kerekítési tag $2 \cdot 10^{-11}|f|$ nagyságrendű; a referenciafutásban mért maximális abszolút hiba $4{,}3 \cdot 10^{-10}$. Az $\epsilon_{\text{abs}} = 10^{-11}$ önmagában ezt nem fedné, a relatív tag viszont a jellemzően $O(1)$–$O(10)$ gradiensértékek mellett igen; ezért a két korlát együtt indokolt. A pontos értéket a konkrét számítás kondíciója és az alkalmazott differencia lépése határozza meg. Ha a program `float32` redukciót tartalmaz, a tolerancia nem húzható be tetszőlegesen.

2.3.4 Izolált környezet: uv és venv

A Pythonhoz több környezetkezelő létezik; az ajánlott útvonal a [D01] dokumentáció alapján az `uv`. A `uv` egy parancssori eszköz, amely a `pyproject.toml` és a `uv.lock` alapján szinkronizálja a projekt csomagprofilját. A labor az 1.1. Modell, tanulás és felhasználási szerződés egységben bemutatott kezdőlabor (`cnn-labor-1.0.0` laborcsomag) `lab/` könyvtárában dolgozik; ha a `uv` még nincs telepítve, a [D01] szerinti telepítő vagy a `python -m pip install uv` parancs adja. A labor könyvtárában a következő parancsok hozzák létre a tanulási profilt:

```
cd lab
uv venv --python 3.12
uv sync
uv run python --version
```

A `uv sync` a lock fájlban rögzített csomagokat telepíti a projekt `.venv` könyvtárába; önmagában is létrehozza a környezetet, a `uv venv` lépés csak az interpreter választását teszi explicitté. A `--python 3.12` a tanulási interpreter verzióját adja meg; a megengedett tartományt (`>=3.12,<3.13`) a labor `pyproject.toml`-ja rögzíti, a csomagverziókat (NumPy 2.3.5, PyTorch 2.14.1 CPU-build, Matplotlib 3.10.8, pandas 2.3.3, SciPy 1.18.1) a `uv.lock` lock fájl rögzíti. A fejezet ellenőrző számítása csak a NumPy-t és a PyTorch-ot használja; a többi csomag a labor teljes profiljához tartozik. A `uv run` előtaggal futtatott parancsok a projekt környezetét használják anélkül, hogy a felhasználónak kézzel aktiválnia kellene a virtuális környezetet. Ez a megkülönböztetés fontos: a `python` parancs a shell aktuális környezetét hívja, a `uv run python` a projekt profilját. A `uv venv` és a `uv sync` szerepe is különbözik: a `uv venv` a virtuális környezet helyét és interpreterét rögzíti, a `uv sync` a lock fájl csomagjait telepíti. Ha a `lab/.venv` már létezik, de nem a kért 3.12-es interpreterrel, a `uv venv --python 3.12` a kért interpreterrel érvényesíti a környezetet; ezt követően a `uv sync` a lock fájlhoz igazítja a csomagokat.

A CPU/GPU build kérdése szintén a csomagprofil része. A *wheel* (`.whl`) egy előre lefordított, telepíthető csomagformátum; a *build* itt a wheelben lévő natív kód változatát jelenti. A PyTorch-nak külön wheel-változatai vannak CPU-s és CUDA-s builddel, és ugyanaz a `torch` csomagnév más binárist takarhat. A labor `pyproject.toml`-ja ezt indexszinten rögzíti: a `[[tool.uv.index]]` blokk `pytorch-cpu` néven definiálja a `https://download.pytorch.org/whl/cpu` indexet `explicit = true` mellett, a `[tool.uv.sources]` pedig a `torch` csomagot ehhez az indexhez köti (Linuxon és Windowson; macOS-en a marker miatt az alapértelmezett index érvényes); ezért a `uv sync` a CPU-s wheelt telepíti. A labor ajánlott útvonala ezt a CPU-s buildet használja. Ha valaki CUDA-s buildet telepít, a `torch.cuda.is_available()` értéke változhat; a modellkódnak ezt explicit módon kell kezelnie, például `device = "cuda" if torch.cuda.is_available() else "cpu"` formában. A build azért is része a reprodukciós feltételnek, mert ugyanaz a Python-csomagnév más natív kódot takarhat.

2.3.5 Jupyter notebook és a kernel állapota

A Jupyter notebook egy JSON dokumentum, amely cellákból, azok kimeneteiből és metaadatokból áll. A kernel a cella végrehajtásakor interpretert indít vagy használ; a notebook kimenete a végrehajtás *múltbeli* rögzítése. A `sys.executable` a kernelben is elérhető, és a következő cellával ellenőrizhető:

```
import sys
print(sys.executable)
```

A kimenet például `.../lab/.venv/bin/python` lehet, de csak akkor, ha a kernel valóban ezt az interpretert használja. A `jupyter kernelspec list` parancs a rendelkezésre álló kernelek nevét és indítási útvonalát mutatja; ugyanaz a `python3` név több környezetre is mutathat. A

Jupyter felületén a notebookhoz tartozó kernel neve a jobb felső sarokban látható, és a **Kernel > Change kernel** menüponttal váltható; a kívánt kernel kiválasztása után a `sys.executable` cellát újra kell futtatni, hogy a tényleges interpreter látható legyen.

A notebook régi kimenete nem bizonyítja a jelenlegi kód futását. Ennek oka, hogy a notebook cellái megosztott kernelállapotot használnak: ha az 1. cellában `a = 1` szerepelt, majd a cellát töröltük, a 2. cella `print(a)` akkor is `1`-et írhat ki, ha az aktuális notebookban az 1. cella már nincs jelen. A megbízható ellenőrzés a **Restart & Run All**, amely újraindítja a kernelt, és minden cellát a dokumentumbeli sorrendben futtat. Ezt követően a mentett notebook kimenete már a jelenlegi kód és a friss kernelállapot együttes eredménye.

2.3.6 Labor: verziójelentés és konvolúciós kontroll új környezetben

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor két részből áll, és a kiinduló `lab` könyvtárban végezhető el. Az első rész verziójelentést készít notebookból és parancssorból, a második rész új környezetben reprodukálja a konvolúciós ellenőrző számítást.

Első lépés: verziójelentés parancssorból. Hozzuk létre a `lab/version_report.py` fájlt:

```
import json
import platform
import sys

import numpy as np

try:
    import torch
except ImportError:
    torch = None

cuda_available = bool(torch is not None and torch.cuda.is_available())
report = {
    "py_executable": sys.executable,
    "python_version": platform.python_version(),
    "python_implementation": platform.python_implementation(),
    "platform": platform.platform(),
    "numpy_version": np.__version__,
    "torch_version": torch.__version__ if torch is not None else None,
    "cuda_available": cuda_available,
    "device_contract": "cuda" if cuda_available else "cpu",
}
print(json.dumps(report, indent=2, sort_keys=True))
```

Futtatás és elmentés:

```
cd lab
mkdir -p reports
uv run python version_report.py | tee reports/version_report_cli.json
```

A `mkdir -p reports` a `tee` előtt hozza létre a könyvtárat; enélkül a `tee` nem tudná megnyitni a kimeneti fájlt. Windows PowerShell alatt a `tee` és a `>` másként működik: a fenti parancs megfelelője `uv run python version_report.py | Tee-Object -FilePath reports\version_report_cli.json`, a `mkdir -p reports` helyett pedig `New-Item -ItemType Directory -Force reports` használható; a későbbi `> reports/conv_check_local.json` átirányítás megfelelője `> reports\conv_check_local.json`. A könyv ellenőrző környezetében mért

kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, OMP_NUM_THREADS=1; a szkript nem használ véletlent); a `py_executable` és a `platform` mező a futtató gép útvonalát mutatja, itt rövidítve:

```
{
  "cuda_available": false,
  "device_contract": "cpu",
  "numpy_version": "2.5.4",
  "platform": "Linux-…-x86_64-with-glibc2.39",
  "py_executable": "…/cnn/venv/bin/python",
  "python_implementation": "CPython",
  "python_version": "3.12.3",
  "torch_version": "2.15.0+cpu"
}
```

A fenti értékek a könyv ellenőrző környezetét írják le; a labor lock fájljának profiljában (`lab/uv.lock`) NumPy 2.3.5 és PyTorch 2.14.1+cpu szerepel, ezért a hallgató saját, a lock fájlhoz kötött futásában ezek a verziók jelennek meg. A `device_contract` nem előre beírt konstans: a szkript a `torch.cuda.is_available()` értéke alapján választja `cpu` vagy `cuda` értékre, és ugyanezt a `cuda_available` mező is rögzíti. A jelentés mindig a tényleges futtató környezetet rögzíti, ezért a lock fájllal való egyezést a hallgatónak kell ellenőriznie; a Python javítóverziója (`3.12.x`) és a platformsor gépenként eltérhet.

Második lépés: ugyanaz a jelentés notebookból. A Jupyter és az `ipykernel` nincs a labor lock fájllal rögzített profiljában, ezért a szervert ideiglenes réteggel indítjuk; a `--with` kapcsoló a futás idejére teszi elérhetővé a csomagokat, a `pyproject.toml`-t és a lock fájlt nem módosítja:

```
cd lab
uv run --with jupyter --with ipykernel jupyter notebook
```

Előbb érdemes a projekt interpreteréhez kötött kernelt regisztrálni; a `cnn-labor` név szabadon választott címke, nem a csomag neve:

```
uv run --with ipykernel python -m ipykernel install --user --name cnn-labor --display-name "CNN
↪  labor (lab/.venv)"
```

Egy új notebook első cellájában futtassuk a fenti `version_report.py` tartalmát, a kernelként a `cnn-labor` kernelt válasszuk, majd válasszuk a **Restart & Run All** műveletet, és mentsük `reports/version_report.ipynb` néven. A notebook kernelét úgy válasszuk, hogy a cella `sys.executable` értéke megegyezzen a CLI-jelentés `py_executable` mezőjével. Ha a két érték eltér, a notebook nem a projekt interpreterét használja; ilyenkor a kernelkonfigurációt kell javítani, nem a kódot.

Harmadik lépés: konvolúciós kontroll új környezetben. Ez a lépés még nem a konvolúció megértését vizsgálja (azt a Diszkrét jelek, képek és konvolúciós műveletek és a Konvolúciós backward és transzponált operátor fejezet tanítja), hanem azt, hogy a környezet működik: a hibákat ezért típus szerint különítjük el. *Telepítési* hiba: `ImportError`, rossz interpreter vagy a parancssori jelentés (CLI) nem nulla kilépési kódja. *Alakhiba*: a tenzorok alakjának (`shape`) eltérése. *Matematikai/numerikus* hiba: a jelentés értékei a megadott toleranciát meghaladják; a tolerancia itt előírt érték, nem levezetett. A laborcsomag konvolúciós referenciája, a `lab/conv_reference.py`, a NumPy-referenciát véges differenciával, `--torch` kapcsolóval PyTorch autograd (automatikus differenciálás) segítségével is összeveti (float64, CPU, rögzített bemenetek), és az eredményt JSON-ként a standard kimenetre írja. A csomagban lévő `lab/conv_check_report.json` a lock fájl profiljának verziómezőivel (NumPy 2.3.5, PyTorch 2.14.1+cpu) szállított baseline (referenciaérték). Az új környezetben a cél annak ellenőrzése, hogy a számítás a jelentésben rögzített értékekkel toleranciásan egyezik:

```
cd lab
uv run python conv_reference.py --torch > reports/conv_check_local.json
```

A `lab/conv_reference.py` és a `lab/conv_check_report.json` a letölthető `cnn-labor-1.0.0` csomag része, nem a fejezet generált melléklete; a baseline a lock fájlhoz tartozó profil szállított referenciaértéke, amelynek numerikus mezőit a MÉRT KIMENETben dokumentált ellenőrző futás a tűrésen belül reprodukálta. A `py_executable` és a `platform` a futtató gépre jellemző szöveges mező, ezért a MÉRT KIMENET blokkban rövidítve szerepel; a numerikus mezők összehasonlítását ez nem érinti, a `numpy_version` és `torch_version` szövegmezőket pedig a kód csak kiírja.

Az összehasonlítást a `version_report.py` mintájára egy kis ellenőrző kóddal végezzük (a `lab` könyvtárból futtatva): beolvassuk a `conv_check_report.json` és a friss `reports/conv_check_local.json` mezőit (a beágyazott szótárakat is), és minden numerikus mezőre `np.allclose`-t alkalmazunk `rtol=1e-9, atol=1e-11` paraméterrel; a `numpy_version` és `torch_version` szövegmezőket csak kiírjuk. Az `np.allclose(a,b)` a $|a-b| \le \text{atol} + \text{rtol}|b|$ feltételt ellenőrzi, ezért a második argumentum a referencia; a hívásban ezért a baseline-t adjuk másodikként.

```
import json
import numpy as np

baseline = json.load(open("conv_check_report.json", encoding="utf-8"))
fresh = json.load(open("reports/conv_check_local.json", encoding="utf-8"))

def check(a, b, path="report"):
    if isinstance(a, dict):
        assert set(a) == set(b), f"eltérő kulcsok: {path}"
        for key in a:
            check(a[key], b[key], f"{path}.{key}")
    elif isinstance(a, list):
        assert len(a) == len(b), f"eltérő hossz: {path}"
        for i, (x, y) in enumerate(zip(a, b)):
            check(x, y, f"{path}[{i}]")
    elif isinstance(a, str):
        if a != b:
            print(f"szöveges mező: {path}: {a!r} -> {b!r}")
    else:
        assert np.allclose(b, a, rtol=1e-9, atol=1e-11), f"numerikus eltérés: {path}"

check(baseline, fresh)
print("Minden numerikus mező a tűrésen belül egyezik.")
```

MÉRT KIMENET (a `conv_reference.py` bemeneteit a `default_rng(20261020)` generálja; Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, OMP_NUM_THREADS=1):

```
szöveges mező: report.numpy_version: '2.3.5' -> '2.5.4'
szöveges mező: report.torch_version: '2.14.1+cpu' -> '2.15.0+cpu'
Minden numerikus mező a tűrésen belül egyezik.
```

A könyv ellenőrző környezetében lefuttatva minden numerikus mező a tűrésen belül egyezett; a szöveges verziómezők tértek el a baseline-hoz képest. A beadandó eredmény a három fájl: `version_report_cli.json`, `version_report.ipynb`, `conv_check_local.json`, valamint az összehasonlítás rövid összegzése, amely megnevezi az eltérő mezőket, ha vannak.

A labor célja nem annak bizonyítása, hogy a konvolúció matematikailag helyes; azt a későbbi Súly- és biasgradiens – Független gradiensellenőrzési protokoll egységekben önállóan levezetjük. Itt a környezet reprodukálhatóságát ellenőrizzük: ha a friss környezetben ugyanaz a seed, dtype

és device ugyanazt az ellenőrző számítást adja, akkor a csomagprofil és a futási szerződés konzisztens.

2.3.7 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés, hogy a notebook régi kimenetét az aktuális kód futási bizonyítékának tekintjük. Ellenpélda két cellával: az első cella `a = 41`, a második `print(a + 1)`. Restart & Run All után a kimenet `42`. Ezután írjuk át az első cellát `a = 100`-ra, de ne futtassuk újra, csak a második cellát: a kimenet továbbra is `42`, pedig a jelenleg látható forrás szerint `101`-nek kellene megjelennie. A kimenet tehát nem a dokumentumban látható kódból, hanem a kernel korábbi állapotából származik. A Restart & Run All ezt a rejtett állapotot megszünteti: ekkor a második cella `101`-et ír ki, és ha az első cellát időközben töröltük, a `print(a + 1)` cella `NameError`-t ad. A példa szándékosan nem lebegőpontos értéken mutatja be a jelenséget: a hiba forrása a cellák futtatási sorrendje és a megmaradó kernelállapot, nem a lebegőpontos kerekítés.

2.3.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a Python interpreter, a Jupyter kernel és a csomagprofil között. Adjon egy konkrét forgatókönyvet, amelyben ugyanaz a notebook két különböző `sys.executable` értékkel futhat. Beadandó: három definíció és a forgatókönyv.

F2 (levezetési, nehézség 2). A szövegben kimondott lemma szerint determinisztikus program azonos környezeti leíróval azonos kimenetet ad. Bontsa ki a bizonyítást arra az esetre, amikor a program három lépésből áll: $s_1 = F_1(s_0)$, $s_2 = F_2(s_1)$, $s_3 = F_3(s_2)$. Mutassa meg, hogy ha minden F_t determinisztikus, akkor s_3 egyértelmű. Ezután indokolja meg, miért nem következik ebből a bitazonosság két különböző CPU-n. Beadandó: a háromlépéses levezetés, a feltételek és egy konkrét ellenpélda a lebegőpontos eltérésre.

F3 (implementációs, nehézség 1). Készítsen `environment_check.py` CLI-szkriptet, amely JSON formában kiírja a `sys.executable`, a Python-verzió, a Python-implementation (`platform.python_implementation()`), a platform (`platform.platform()`), a NumPy verzió, a PyTorch verzió, a CUDA-elérhetőség és a `device_contract` mezőket. A szkript fogadjon egy `--device` argumentumot, amelynek értéke `cpu` vagy `cuda`. Ha `cuda` kérése esetén a CUDA nem elérhető, a szkript írjon `error` mezőt, és 1-es kóddal lépjen ki. Beadandó: a teljes kód, egy futtatási parancs, a kimenet és a hibaellenőrzés.

F4 (hibakeresési, nehézség 2). Egy hallgató a notebookban a `sys.executable` cellát futtatja, és a kimenet `/usr/bin/python3`. A parancssori `uv run python -c 'import sys; print(sys.executable)'` viszont `.../lab/.venv/bin/python`-t ad. A hallgató arra következtet, hogy a `uv` elromlott. Mi a valódi hiba? Mi a következménye annak, ha a notebookot a rendszerinterpretálóval futtatja? Hogyan ellenőrizhető a javítás? Beadandó: ok, következmény, javítás és a visszaellenőrzés módja.

F5 (kísérlettervezési, nehézség 3). Tervezzen kis kísérletet annak eldöntésére, hogy a konvolúciós ellenőrző számítás két izolált környezetben toleranciásan reprodukálható-e. Az egyik környezet a labor CPU-profilja, a másik egy frissen létrehozott `uv` környezet. Adja meg a kérdést, a rögzített seedet, dtype-ot, devicet, baseline-t, metrikát, szükséges erőforrást és a következtetési korlátot. Beadandó: a protokoll és annak magyarázata, hogy miért nem állítható ebből általános platformfüggetlen bitazonosság.

2.3.9 Megoldások és indoklás

F1 megoldás. Az interpreter a Python bytecode-ot végrehajtó folyamat; azonosítója a `sys.executable` útvonal. A Jupyter kernel egy szerverhez kapcsolódó folyamat, amely egy interpreterrel futtatja a notebook celláit; a kernel és a notebook dokumentum külön objektum. A csomagprofil a specifikált függőségek és verziók zárt halmaza, például `uv.lock`. Forgatókönyv: a notebookot a rendszer `python3` kerneljével indítjuk, miközben a projekt `.venv`-je már létezik. Ekkor a notebook `sys.executable` értéke `/usr/bin/python3`, míg ugyanaz a kód `uv run python` alatt a `.venv/bin/python` értéket adja; a NumPy verziók is eltérhetnek.

F2 megoldás. A három lépésre: s_0 rögzített, mert I és E adott. Ekkor $s_1 = F_1(s_0)$ egyértelmű, mert F_1 függvény. Ha s_1 egyértelmű, akkor $s_2 = F_2(s_1)$ is egyértelmű, végül $s_3 = F_3(s_2)$ is. A bizonyítás a függvény fogalmából és a teljes indukció specializálásából áll. A bitazonosság hiánya: két CPU eltérhet az utasításkészletben és a lebegőpontos reduction sorrendjében. Például egy nyolcelemű összeget az egyik gép $(a_1 + a_2) + (a_3 + a_4) + \cdots$ módon, a másik páronkénti vagy szekvenciális sorrendben számol; a kerekítési hibák miatt a két eredmény utolsó bitje eltérhet. Számpélda float64-ben: $(10^{16} + 1) - 10^{16} = 0$, mert a 10^{16} nagyságrendjében az 1 már nem fér el a mantisszában, míg $10^{16} - 10^{16} + 1 = 1$; ugyanazok a műveletek más sorrendben más eredményt adnak. Float32-ban $(2 \cdot 10^7 + 1) + 1 = 2 \cdot 10^7$, mert a $2 \cdot 10^7$ mellett az 1 nem ábrázolható külön, viszont $(1 + 1) + 2 \cdot 10^7 = 2{,}0000002 \cdot 10^7$. Ezért az azonos Python- és PyTorch-verzió sem garantál bitazonos kimenetet különböző platformon.

F3 megoldás. A kód:

```
import argparse
import json
import platform
import sys

import numpy as np

try:
    import torch
except ImportError:
    torch = None

def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--device", choices=["cpu", "cuda"], default="cpu")
    args = parser.parse_args()

    cuda_available = bool(torch is not None and torch.cuda.is_available())
    report = {
        "py_executable": sys.executable,
        "python_version": platform.python_version(),
        "python_implementation": platform.python_implementation(),
        "platform": platform.platform(),
        "numpy_version": np.__version__,
        "torch_version": torch.__version__ if torch is not None else None,
        "cuda_available": cuda_available,
        "device_contract": args.device,
    }
    if args.device == "cuda" and not cuda_available:
        report["error"] = "CUDA requested but not available"
        print(json.dumps(report, indent=2, sort_keys=True), file=sys.stderr)
        return 1

    print(json.dumps(report, indent=2, sort_keys=True))
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Futtatás: `uv run python environment_check.py --device cpu`. A CPU-s kimenet `cuda_available: false` mellett `device_contract: "cpu"`; a `--device cuda` változat CPU-s gépen `error` mezővel és 1-es visszatérési kóddal zárul, a hibajelentés a standard hibára (`stderr`) kerül, a standard kimenet üres marad. Az ellenőrzés a kilépési kódból és a JSON `error` mezőjének jelenlétéből áll.

F4 megoldás. Az ok nem a `uv` hibája, hanem az, hogy a notebook kernelét a rendszer-interpreterhez kötötték, nem a projekt környezetéhez. Következmény: a notebook a rendszer csomagprofilját látja, így a verziójelentés nem a labor profilját írja le, és a reprodukálhatósági állítás sérül. A javítás a Jupyter kernelspec átállítása vagy új kernel létrehozása a `.venv` interpreterével. Az `ipykernel` nincs a labor profiljában, ezért a regisztráláskor a `--with ipykernel` ideiglenes réteget használjuk: `uv run --with ipykernel python -m ipykernel install --user --name cnn-labor --display-name "CNN labor (lab/.venv)"`; a `cnn-labor` szabadon választott kernelnév, nem a csomag neve. Ezután a notebookban a `cnn-labor` kernelt kell kiválasztani. Visszaellenőrzés: Restart & Run All után a notebook `sys.executable` értékének meg kell egyeznie a `uv run python -c 'import sys; print(sys.executable)'` kimenettel.

F5 megoldás. Kérdés: ugyanazt az ellenőrző számítást adja-e a laborprofil és egy frissen létrehozott izolált környezet? Protokoll: a második környezetet `uv venv --python 3.12` és `uv sync` hozza létre ugyanabból a lock fájlból. Mivel mindkét környezet ugyanabból a lock fájlból épül, a kísérlet a lock fájl által rögzített profil reprodukálhatóságát méri, nem két különböző build összevetését; két eltérő build (például CPU-s és CUDA-s wheel, vagy más Python-javítóverzió) összehasonlítása külön kísérlet, amelyben bitazonosság nem várható. Mindkét környezetben a `conv_reference.py --torch` fut (float64, CPU, a szkriptben rögzített bemenetekkel). Baseline: a kezdőlabor `lab/conv_check_report.json` numerikus értékei. Metrika: minden számszerű mezőre `np.allclose(..., rtol=1e-9, atol=1e-11)`; ezen kívül a jelentés metaadatai közül a `python_version`, a `python_implementation`, a `numpy_version`, a `torch_version`, a `cuda_available` és a `device_contract` mezőnek egyeznie kell, míg a `platform` és a `py_executable` gépenként és útvonalanként eltérhet, ezért ezeket csak rögzítjük, egyezésüket nem követeljük. Erőforrás: kis CPU-s futás; a könyv ellenőrző környezetében a `conv_reference.py --torch` (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, `OMP_NUM_THREADS=1`) egy mért futásban 0,88 s fali idő alatt fejeződött be, ez azonban egy gép és egy futás adata, nem általánosítható. Következtetési korlát: a toleranciás egyezés a két adott környezetre vonatkozik; nem bizonyít platformfüggetlen bitazonosságot, és ha a másik gép buildje eltér, az egyezés meghiúsulhat anélkül, hogy a kód hibás lenne. A [D02] dokumentáció szerint azonos seed mellett sem garantált a bitazonos eredmény platformok és verziók között; a seed tehát önmagában nem elegendő.

2.3.10 Önellenőrzés és visszatekintés

Önellenőrző kérdések: meg tudom-e nevezni a `sys.executable` értéket abban a környezetben, ahol a modellemet futtattam? Rögzítettem-e a csomagprofilt lock fájlban? Kiválasztottam-e a notebookban a projekt interpreteréhez kötött kernelt, és egyezik-e a cella `sys.executable` értéke a parancssori jelentés `py_executable` mezőjével? A notebook kimenetét Restart & Run All után mentettem-e? Ha bármelyik válasz nem, a kísérleti naplóból hiányzik a reprodukciós feltétel.

Visszatekintés az 1.3. Adatfelosztás és a megfigyelési egység egységből: ahogyan a split érvényessége attól függ, hogy a megfigyelési egység szerint csoportosítottunk-e, úgy a futási állítás érvényessége attól függ, hogy a környezeti leíró minden befolyásoló komponensét rögzítettük-e.

Visszatekintés a 2.2. Lebegőpont és stabil elemi számítás egységből: a numerikus egyenlőséget `allclose`-zal és indokolt toleranciával ellenőrizzük, nem `==` művelettel.

2.3.11 Rövid összefoglaló

A Python interpreter, a Jupyter kernel és a csomagprofil három különböző réteg. A reprodukálhatóság akkor értelmes, ha a determinisztikus program bemenete mellett az interpretert, a lock fájlban rögzített függőségeket és a buildkörnyezetet is rögzítjük; ebből azonos környezetben azonos kimenet következik. Platformok között bitazonosság nem ígérhető, ezért a toleranciás `allclose` a helyes ellenőrzés. A notebook csak Restart & Run All után tekinthető az aktuális kód futási bizonyítékának. A labor verziójelentést készít CLI-ből és notebookból, majd új környezetben reprodukálja a konvolúciós kontrollt a kezdőlabor jelentésével összehasonlítva.

2.3.12 Továbblépés

A 2.4. Kis tudományos szoftverprojekt egységben kis tudományos szoftverprojektet szervezünk: a most rögzített környezeti jelentés lesz a projekt `README` fájljának (a projekt rövid leírása) és futtatási szerződésének első szakasza. A verziójelentés adatai akkor válnak hasznossá, amikor egy hiba vagy eltérő eredmény okát kell lokalizálni.

2.3.13 Források

- [D01] Astral (2026). *uv project syncing*. https://docs.astral.sh/uv/concepts/projects/sync/. Hozzáférés: 2026-10-30.
- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.

2.4 Kis tudományos szoftverprojekt

2.4.1 Cél, előismeret és induló kérdés

A 2.3. Izolált környezet, notebook és verzió egységben rögzítettük, melyik interpreter, kernel és lockolt csomagprofil futtatja a kódot. Ebben az alfejezetben a notebookban kipróbált cellát valódi, tesztelt szoftverprojektté alakítjuk. A cél nem az, hogy „szebb” kódot írjunk, hanem hogy a kísérlet bemenete, kimenete, konfigurációja és ellenőrzése együtt legyen újrafuttatható. Induló kérdés: mikor mondhatjuk, hogy a notebookbeli függvény és a modulba emelt változat *ugyanazt a matematikai leképezést* állítja elő, és mit tud ebből a teszt ténylegesen ellenőrizni? A fejezet mintaprojektje a **GlyphStats**: a GlyphLab adathalmaz (a függelék A. szakasza) köré épített kis szoftverprojekt. A GlyphLab az adatok és az adatszerződés neve, a GlyphStats a most létrehozott kódé; a szöveg a kettőt következetesen megkülönbözteti.

Előismeret: a 2.3. Izolált környezet, notebook és verzió környezeti szerződése. Visszatekintés az 1.4. Tanulási diagnózis és kísérleti napló egységből: a seed a generátor kezdőállapota, a futásazonosító a napló kulcsa; a kettő nem cserélhető fel.

2.4.2 Definíciók és intuíció

Tiszta függvény. Egy Python-függvény akkor **tiszta**, ha *referenciálisan átlátszó* és *nincs megfigyelhető mellékhatása*:

1. a visszatérési érték csak az argumentumoktól függ, ezért minden hívása helyettesíthető a visszaadott értékkel anélkül, hogy a program viselkedése megváltozna (ugyanazon argumentumokra mindig ugyanazt adja, és nem olvas rejtett állapotot: globális változót, fájlt, órát vagy implicit véletlengenerátort);
2. nincs kívülről megfigyelhető mellékhatása: nem módosítja az argumentumait, nem ír fájlt, nem változtat globális vagy modulállapotot.

A tisztaság a kód *szerkezeti* tulajdonsága, és független attól, hogy a függvény helyes-e. A **helyesség** külön állítás: ha a matematikai leképezés $f : \mathcal{D} \to \mathcal{C}$, akkor a Python-implementáció F helyes, ha $F(x) = f(x)$ minden $x \in \mathcal{D}$-re (lebegőpontos számításnál egy rögzített toleranciával). Egy teszt ennek a helyességnek csak véges sok esetét ellenőrzi; a tisztaság viszont azt biztosítja, hogy a tesztben látott eredmény nem függ a processz korábbi állapotától, ezért megismételhető. A fájlírás ezzel szemben eljárás, nem tiszta függvény.

Modul. Olyan Python-fájl, amely importálható névtérként szolgál. A notebookcella kódja modulba emelve függetleníthető a cellák végrehajtási sorrendjétől.

CLI. Parancssori interfész, amely a bemenetet, a kimenetet és az opciókat argumentumként kapja. Ezzel a konfiguráció nem a kódba égetett konstans, hanem a parancs része.

Manifest, futási jegyzék. A futás bemenetének, paramétereinek, környezetének és kimeneti lenyomatának géppel ellenőrizhető rögzítése. A függelék A. szakaszában leírt GlyphLab `manifest.json`-ja ugyanezt az elvet követi: a fájlhash és a splitenkénti elemszám együttesen azonosítja az adatállapotot.

Tesztfixture. Előre, determinisztikusan előállított bemenet és hozzá tartozó elvárt tulajdonság. A fixture nem „példa, ami véletlenül jó”, hanem a teszt szerződésének rögzített inputja. A pytest keretrendszerben a `@pytest.fixture` dekorátorral jelölt függvény állítja elő és adja át ezt az adatot a tesztnek; a dekorátor tehát nem maga a fixture (az adat), hanem annak előállítási módja.

Lokális artefaktum. A futtatás által létrehozott, nem forráskód jellegű fájl: modellkimenet, napló, gyorsítótár. A verziókezelőből kizárjuk, mert a futás eredménye, nem a projekt definíciója. A futási jegyzék (manifest) ezzel szemben megőrzendő: kis JSON, amely a futás azonosításához és reprodukálásához kell, ezért nem kerül a kizárt fájlok közé.

Git. Verziókezelő, amely a projekt *forrásállapotát* rögzíti. A commit a kód és a fixture időbeli kontrollja; a futás reprodukálásához szükség van még a környezeti lockra és a manifestre.

2.4.3 A refaktor által őrzött invariáns

Állítás (refaktor-egyezés, gyenge determinisztikus lemma). Legyen F_0 a notebookban definiált függvény, F_1 a modulba emelt függvény. Tegyük fel, hogy

- F_1 függvénytörzse ugyanazt az utasítássorozatot hajtja végre, mint F_0, és minden, a törzsön kívül definiált név vagy argumentummá vált, vagy a modulban változatlan értékkel lett definiálva;
- a futtatás ugyanazon Python-interpreterrel, ugyanazon NumPy-verzióval, ugyanazon dtypetal és ugyanazon processz-állapotban történik;

- a függvény nem használja a `__name__`-et, a `globals()`-t, introspekciót, relatív importot, objektumazonosságot (`is`) vagy megosztott módosítható állapotot: ezek notebookban és modulban eltérhetnek, és a lemma rájuk nem vonatkozik.

Ekkor bármely x bemenetre $F_0(x) = F_1(x)$.

Indoklás. A feltételek mellett a program determinisztikus: minden utasítás ugyanazt az átmenetet hajtja végre, függetlenül attól, hogy a kód fizikailag notebookcellában vagy modulban van-e. A szekvenciális végrehajtás definíciója szerint azonos kezdőállapotból azonos közbülső állapotok és azonos végeredmény adódik. Ez nyelvi szemantikai érvelés, nem mély matematikai tétel; az állítás ereje a feltételek explicitségében van. A teljes tartományra vonatkozó egyenlőséget ez az érvelés adja, a unit teszt önmagában csak véges mintán ellenőriz.

Mit nem ellenőrizhet a unit teszt? A teszt csak a fixture-ökre vonatkozik; nem bizonyítja a le nem fedett bemenetek viselkedését. Nem dönti el, hogy a függvény a *tudományos konvenciónak* megfelelő mennyiséget számítja-e (például populációs vagy mintabeli szórást). Nem ad platformok közötti bitazonossági garanciát: az azonos seed csak ugyanazon generátorverzió mellett ígér azonos számsorozatot, más builden vagy verzióváltásnál a lebegőpontos eredmény eltérhet, amint a [D04] dokumentáció is rögzíti. Végül a zöld teszt nem helyettesíti a modellértékelés helyességét.

Ellenpélda a feltétel szükségességére. Ha a notebookcella a törzsén kívül definiált, implicit `rng = np.random.default_rng()` objektumot használ, a modulba emelt változat más processzben más seed nélkül más kimenetet adhat. A cella tehát nem volt tiszta; a refaktornak a generátort argumentummá kell tennie, különben a lemma 1. feltétele sérül. A generátor argumentummá tétele önmagában még nem teszi tisztává a függvényt: a NumPy `Generator` mintavétele mutálja az átadott objektum állapotát, az ismételt hívás ezért más eredményt ad. Ez a viselkedés explicit függőséggel tesztelhető, és az így kapott függvény *állapotfüggő*. Tiszta változathoz a függvény a seedből helyi új generátort hoz létre (`np.random.default_rng(seed)`), vagy az RNG-állapotot explicit be- és kimenetként viszi; a *tisztaság* és a *reprodukálhatóság* külön fogalom.

2.4.4 Kézi példa a standardizáló függvényre

A labor egy oszloponkénti standardizáló függvényt emel ki. Legyen $X \in \mathbb{R}^{N\times d}$; a j oszlopra

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} X_{ij}, \qquad \sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(X_{ij}-\mu_j)^2,$$

és egy rögzített $\varepsilon \geq 0$ véges küszöbbel a kimenet

$$Y_{ij} = \begin{cases} \frac{X_{ij}-\mu_j}{\sigma_j}, & \text{ha } \sigma_j > \varepsilon, \\ 0, & \text{ha } \sigma_j \leq \varepsilon. \end{cases}$$

Ezt nevezzük ε-küszöbös populációs standardizálásnak. Az $\varepsilon = 0$ eset a tiszta matematikai operátor, amely csak a valóban konstans ($\sigma_j = 0$) oszlopot nullázza. Az $\varepsilon > 0$ küszöb **skálafüggő** degenerációt vezet be: minden olyan oszlop nullává válik, amelynek szórása ε alatti, akkor is, ha nem konstans. Például az $X = (0,\ 10^{-13},\ 2\cdot 10^{-13})^\top$ oszlopra $\sigma \approx 8{,}16\cdot 10^{-14}$; $\varepsilon = 0$ mellett a kimenet $(-1{,}2247;\ 0;\ 1{,}2247)$, a labor alapértelmezett $\varepsilon = 10^{-12}$ küszöbével viszont nullavektor. A küszöb tehát a bemenet mértékegységéhez kötött döntés, nem numerikus apróság.

Két szóráskonvenció és az $\varepsilon = 0$ numerikus határa. A fenti σ_j populációs szórás ($1/N$ normalizálás, NumPy-ban `ddof=0`), nem az $1/(N-1)$-es mintabeli becslés; a kettőt a függvény dokumentációjában és a manifestben is meg kell nevezni. Az $\varepsilon = 0$ eset *matematikailag* valóban

csak a $\sigma_j = 0$ oszlopot nullázza, *lebegőpontosan* azonban nem garantált, hogy egy konstans oszlop számított szórása pontosan nulla. Például a labor `normalize_columns` képletével az $x = (0{,}1;\ 0{,}1;\ 0{,}1)$ oszlop központozott értékei $-1{,}39{\cdot}10^{-17}$ körüliek, a számított szórás $1{,}39{\cdot}10^{-17} > 0$, ezért $\varepsilon = 0$ mellett a kimenet $(-1; -1; -1)$ lesz a definiált $(0; 0; 0)$ helyett (a venvben, NumPy 2.5.4-gyel ellenőrizve); a labor $\varepsilon = 10^{-12}$ küszöbével ugyanez az oszlop nullázódik. Ezért a gyakorlati laborban $\varepsilon > 0$ kell, és az $\varepsilon = 0$ csak a matematikai határesetet szemlélteti.

Kézi ellenőrzés három megfigyeléssel, két oszloppal:

$$X = \begin{pmatrix} 0 & 3 \\ 2 & 5 \\ 4 & 1 \end{pmatrix}, \quad N = 3,\ d = 2.$$

Az első oszlop: $\mu_1 = (0+2+4)/3 = 2$, a központozott vektor $(-2, 0, 2)$, variancia $(4+0+4)/3 = 8/3$, szórás $\sigma_1 = \sqrt{8/3}$. A standardizált oszlop $(-2, 0, 2)/\sqrt{8/3}$; átlaga 0, populációs varianciája pedig a négyzetes átlag:

$$\frac{1}{3}\left[\left(\frac{-2}{\sqrt{8/3}}\right)^2 + 0^2 + \left(\frac{2}{\sqrt{8/3}}\right)^2\right] = \frac{1}{3}\cdot\frac{8}{8/3} = 1.$$

A második oszlopra $\mu_2 = 3$, a központozott vektor $(0, 2, -2)$, ugyanaz a variancia; a standardizált oszlop $(0, 2, -2)/\sqrt{8/3}$, amelynek az átlaga (0) és a varianciája (1) az elsőével azonos, az értékek sorrendje viszont eltér. Egy konstans harmadik oszlop, például $(5, 5, 5)^\top$, $\sigma = 0$ miatt a $(0, 0, 0)^\top$ konvenciót kapja.

2.4.5 Konkrét labor: notebookból projekt

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor kiindulópontja a 2.3. Izolált környezet, notebook és verzió notebook egy cellája, amely a fenti standardizálást keveri egy ad hoc kiírással. A feladat a tiszta számítás kiemelése, majd a projekt felépítése. A kiinduló notebook-cella (a refaktor *előtti* állapot) egyetlen cellában tölt be, számol és ír ki:

```
# Kiinduló notebook-cella: betöltés, számítás és kiírás egy cellában
import numpy as np

X = np.load("data/input.npy")
mu = X.mean(axis=0, keepdims=True)
sd = X.std(axis=0, keepdims=True)
Y = (X - mu) / np.where(sd > 1e-12, sd, 1.0)
Y = np.where(sd > 1e-12, Y, 0.0)
print(Y.shape, Y.mean(axis=0), Y.std(axis=0))
```

Ez a cella nem tiszta: az `X` a rejtett kernelállapotból jön, a `print` mellékhatás, a `data/input.npy` útvonal pedig a munkakönyvtártól függ. A refaktor *utáni* állapotban ugyanaz a számítás a `normalize_columns` függvényben él (lásd alább), a betöltés és a kiírás a CLI feladata. Az átalakítás helyességének ellenőrzése: az új függvény a fenti `X`-re ugyanazt a `Y`-t adja, mint a cella.

A projektstruktúra:

- `glyphstats/__init__.py`: csomagjelölő (üres); commitolandó.

- `glyphstats/normalize.py`: tiszta függvény (standardizálás); commitolandó.
- `glyphstats/cli.py`: parancssori belépés, manifest (argparse + hash); commitolandó.
- `tests/test_normalize.py`: teszt (fixture-ellenőrzés); commitolandó.
- `data/input.npy`: fixture (determinisztikus bemenet); commitolandó.
- `manifests/manifest.json`: futási jegyzék; commitolandó (kis JSON).
- `uv.lock`: a telepített verziókat rögzítő lockfájl; commitolandó.
- `runs/`: lokális artefaktum (kimenet és napló); **nem** commitolandó.

A `normalize.py`:

```
"""Oszloponkénti standardizálás az n008 laborhoz."""
from __future__ import annotations

import numpy as np


def normalize_columns(x: np.ndarray, eps: float = 1e-12) -> np.ndarray:
    """Standardizálja az (N, d) mátrix oszlopait, eps-küszöbbel.

    A szórás konvenció: populációs, 1/N normalizálással.
    Ha egy oszlop szórása <= eps, a kimenet 0 (eps=0: csak a konstans oszlop).
    """
    if x.ndim != 2:
        raise ValueError("x.shape-nak (N, d) kétdimenziósnak kell lennie")
    if not (np.isfinite(eps) and eps >= 0):
        raise ValueError("eps-nek véges, nemnegatív számnak kell lennie")
    centered = x - x.mean(axis=0, keepdims=True)
    var = np.mean(centered * centered, axis=0, keepdims=True)
    sd = np.sqrt(var)
    denom = np.where(sd > eps, sd, 1.0)
    out = centered / denom
    out = np.where(sd > eps, out, 0.0)
    return np.ascontiguousarray(out)
```

A függvény három apró numerikus döntését érdemes megérteni. A `np.where(sd > eps, sd, 1.0)` a nulla nevezőt 1-re cseréli, hogy a osztás ne adjon `nan` vagy `inf` értéket; a második `np.where(sd > eps, out, 0.0)` ezután a küszöb alatti oszlopokat nullázza. A `np.ascontiguousarray` a kimenetet C-folytonos memórialayoutba rendezi, ami az `np.save` és a további vektorizált műveletek szempontjából kiszámítható; az eredmény értékét nem változtatja meg. Mini példa: a `normalize_columns(np.array([[0.0], [2.0]]))` hívás kimenete `[[-1.], [1.]]`, és ismételt hívásra is ugyanez, mert a függvény csak a bemenetétől függ (a venvben, NumPy 2.5.4-gyel ellenőrizve).

A `tools/make_fixture.py` determinisztikus fixture-t állít elő:

```
"""Az n008 tesztfixture generátora: seedelt, ismételhető."""
from __future__ import annotations

from pathlib import Path

import numpy as np


def main() -> None:
    out_dir = Path(__file__).resolve().parent.parent / "data"
    out_dir.mkdir(exist_ok=True)
    rng = np.random.default_rng(20261020)
    x = rng.normal(loc=2.0, scale=4.0, size=(10, 3))
    x[:, 2] = 5.0  # konstans oszlop a degenerált esethez
    np.save(out_dir / "input.npy", x.astype(np.float64))
```

```
if __name__ == "__main__":
    main()
```

A `cli.py` explicit bemenetet, kimenetet és manifestet kezel:

```
"""Parancssori belépőpont és futási jegyzék."""
from __future__ import annotations

import argparse
import hashlib
import json
import platform
import sys
import uuid
from datetime import datetime, timezone
from pathlib import Path

import numpy as np

from .normalize import normalize_columns


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(65536), b""):
            digest.update(chunk)
    return digest.hexdigest()


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="GlyphStats normalizáló CLI")
    parser.add_argument("--input", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--manifest", type=Path, default=None)
    parser.add_argument("--eps", type=float, default=1e-12)
    return parser


def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    input_path = args.input.expanduser().resolve()
    output_path = args.output.expanduser().resolve()
    # A np.save kiterjesztés nélküli névhez .npy-t fűzne; a manifestnek
    # a ténylegesen létrejött fájlra kell mutatnia.
    if output_path.suffix != ".npy":
        output_path = output_path.with_suffix(".npy")
    output_path.parent.mkdir(parents=True, exist_ok=True)

    # Csak saját fixture-t töltünk; allow_pickle=False biztonsági okból.
    loaded = np.load(input_path, allow_pickle=False)
    input_dtype = str(loaded.dtype)
    x = loaded.astype(np.float64, copy=False)
    y = normalize_columns(x, eps=args.eps)
    np.save(output_path, y)

    if args.manifest is not None:
        record = {
            "run_id": uuid.uuid4().hex,
            "created_utc": datetime.now(timezone.utc).isoformat(),
            "input_file": input_path.name,
            "input_sha256": sha256_file(input_path),
```

```
            "output_file": output_path.name,
            "output_sha256": sha256_file(output_path),
            "params": {"eps": args.eps, "input_dtype": input_dtype},
            "python": sys.version.split()[0],
            "numpy": np.__version__,
            "machine": platform.platform(),
            "normalization": "population_std",
        }
        manifest_path = args.manifest.expanduser().resolve()
        manifest_path.parent.mkdir(parents=True, exist_ok=True)
        manifest_path.write_text(json.dumps(record, indent=2), encoding="utf-8")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

A CLI négy biztonsági és memóriahasználati döntése: a `sha256_file` 65536 bájtos darabokban olvassa a fájlt, így a memóriahasználat a fájlmérettől függetlenül korlátozott, és nagy fájlok hash-elése sem tölti be a teljes tartalmat. A `np.load(input_path, allow_pickle=False)` tiltja a pickle-formátumú objektumok betöltését, mert a pickle tetszőleges Python-kódot futtathat; a labor csak saját, numerikus `.npy` fixture-t tölt be. Az `astype(np.float64, copy=False)` csak akkor másol, ha a bemenet nem már `float64`; `float32` bemenetnél tehát konvertál, `float64`-nél nem. A `np.save` a kiterjesztés nélküli névhez `.npy`-t fűzne, ezért a kód a mentés előtt a manifestben rögzítendő tényleges útvonalra igazítja a kimenetet.

A teszt `tests/test_normalize.py`:

```
"""Az n008 labor szerződését rögzítő unit tesztek."""
from __future__ import annotations

from pathlib import Path

import numpy as np
import pytest

from glyphstats.normalize import normalize_columns


@pytest.fixture()
def input_array() -> np.ndarray:
    path = Path(__file__).resolve().parent.parent / "data" / "input.npy"
    return np.load(path, allow_pickle=False)


def test_output_shape_matches_input(input_array: np.ndarray) -> None:
    y = normalize_columns(input_array)
    assert y.shape == input_array.shape
    assert y.dtype == np.float64


def test_normalized_columns_are_standardized(input_array: np.ndarray) -> None:
    y = normalize_columns(input_array)
    expected_std = np.array([1.0, 1.0, 0.0])
    np.testing.assert_allclose(y.mean(axis=0), np.zeros(3), atol=1e-12)
    np.testing.assert_allclose(y.std(axis=0), expected_std, atol=1e-12)


def test_zero_variance_columns_are_zero(input_array: np.ndarray) -> None:
    y = normalize_columns(input_array)
    np.testing.assert_array_equal(y[:, 2], np.zeros(input_array.shape[0]))
```

```python
def test_eps_threshold_is_scale_dependent() -> None:
    x = np.array([[0.0], [1e-13], [2e-13]])
    np.testing.assert_array_equal(normalize_columns(x), np.zeros((3, 1)))   # eps=1e-12
    np.testing.assert_allclose(normalize_columns(x, eps=0.0).ravel(),
                               [-np.sqrt(1.5), 0.0, np.sqrt(1.5)], rtol=1e-9, atol=1e-12)
    with pytest.raises(ValueError):
        normalize_columns(x, eps=-1.0)


def test_rejects_non_matrix(input_array: np.ndarray) -> None:
    with pytest.raises(ValueError):
        normalize_columns(input_array[:, 0])
```

A `pyproject.toml` a tanulási profilhoz:

```toml
[project]
name = "glyphstats"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["numpy>=2.0"]

[dependency-groups]
dev = ["pytest>=8"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

A `pythonpath = ["."]` beállítás azért kell, mert a `glyphstats` csomag nincs telepítve a környezetbe; enélkül a `pytest` a projekt gyökerét nem teszi a `sys.path`-ra, és a tesztgyűjtés `ModuleNotFoundError: No module named 'glyphstats'` hibával áll meg (a könyv ellenőrző környezetében mért hibaüzenet). A beállítás a pytest `pythonpath` ini-opciója. A `[build-system]` szakasz szándékosan hiányzik: a tanulási profilban a `glyphstats` nem telepített kerékcsomag, hanem a projekt gyökeréből importált forrásmappa, amelyet a `glyphstats/__init__.py` jelöl csomagnak. A telepítés helyett a projekt gyökeréből futtatunk; a `python -m glyphstats.cli` a munkakönyvtárat használja a csomag feloldásához.

A `.gitignore` itt a lokális, újragenerálható artefaktumokat zárja ki; ez helyi repószabály, nem az artifact definíciója. A reprodukcióhoz a manifestek, konfigurációk, a kiválasztott checkpoint és a nyers mérési kimenetek megőrzése szükséges, akár Gitben (kis fájloknál), akár verziózott külső tárban (nagy fájloknál); az azonosítás és visszakeresés útja a futásnaplóban rögzített lenyomat (SHA-256) és a tárolóhely. A következő `.gitignore` tehát csak a *nem* megőrzendő, újragenerálható fájlokat nevezi meg:

```
__pycache__/
.pytest_cache/
.venv/
runs/
```

A `.venv/` a helyi virtuális környezet, a `runs/` a futás kimenete; egyik sem újragenerálható forrás, a `manifests/` viszont commitolandó.

A parancsok a projekt gyökeréből, lépésenként; minden lépés után van ellenőrizhető köztes eredmény:

1. Hozzuk létre a `glyphstats/`, `tests/`, `tools/` könyvtárat és a fenti fájlokat, valamint a `.gitignore`-t. Ellenőrzés: a `glyphstats/__init__.py` létezik, tehát a csomag importálható.

2. `uv sync` – a lock fájlból telepít. Várt eredmény: a `.venv` létrejön, a parancs hibaüzenet nélkül, 0 kóddal tér vissza.
3. `uv run python tools/make_fixture.py` – létrehozza a `data/input.npy` fájlt `(10, 3)` alakkal, `float64` dtype-dal; a fájl SHA-256 kezdete `7970b0a5…`, ezt a venvben reprodukáltuk (a fixture seedje 20261020).
4. `uv run pytest -q` – várt kimenet: `5 passed`; a könyv ellenőrző környezetében mért pontos kimenet alább szerepel.
5. `uv run python -m glyphstats.cli --input data/input.npy --output runs/output.npy --manifest manifests/manifest.json` – létrehozza a `runs/output.npy` és a `manifests/manifest.json` fájlt; a rögzített bemenet mellett az `output_sha256` kezdete `ce5d53cf…` (a venvben reprodukálva). A `run_id` és a `created_utc` futásonként változik.

```
uv sync
uv run python tools/make_fixture.py
uv run pytest -q
uv run python -m glyphstats.cli \
  --input data/input.npy \
  --output runs/output.npy \
  --manifest manifests/manifest.json
```

MÉRT KIMENET (a `pytest -q` kimenete; Python 3.12.3, NumPy 2.5.4, pytest 9.2.0, CPU, egy szál; a fixture seedje 20261020):

```
.....                                                                    [100%]
5 passed in 0.04s
```

A futtatás után a jegyzék szerkezete a következő sémát követi; a `run_id` és a `created_utc` futásonként változik, a hash-ek viszont a rögzített bemenet és paraméterek mellett determinisztikusak. A blokk ezért placeholderként szerepel, nem mért outputként:

```
{
  "run_id": "<generált-uuid>",
  "created_utc": "<ISO-8601 időbélyeg>",
  "input_file": "input.npy",
  "input_sha256": "<64 hex karakter>",
  "output_file": "output.npy",
  "output_sha256": "<64 hex karakter>",
  "params": {"eps": 1e-12, "input_dtype": "<a betöltött tömb dtype-ja, pl. float64>"},
  "python": "3.12.x",
  "numpy": "2.x.y",
  "machine": "<platform.platform() kimenet>",
  "normalization": "population_std"
}
```

A `README.md` első szakasza a 2.3. Izolált környezet, notebook és verzió verziójelentését másolja be: interpreter, kernel, lock-állapot. Így a projekt nemcsak fut, hanem megmondja, milyen feltételek között futott.

2.4.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem tiszta függvény egy olyan `next_batch(xs, batch_size)` függvény, amely modulszintű `_cursor` globális változót léptet, és adjon olyan hívás-sorrendet, amelyben ugyanaz az argumentum más értéket ad vissza. Elvárt beadandó: definíció szerinti indoklás és konkrét ellenpélda.

F2 (levezetési, nehézség 2). Bizonyítsa be a fenti konvencióval: ha $\sigma_j > 0$, akkor a standardizált Y oszlopának átlaga 0 és populációs varianciája 1. Kezelje a $\sigma_j = 0$ esetet is. Elvárt: minden lépés, indexhatár és a degenerált eset indoklása.

F3 (implementációs, nehézség 1). Készítse el a labor projektet, futtassa az öt tesztet, majd a CLI-t. Adja meg a `manifest.json` `run_id`, `input_sha256` és `output_sha256` kulcsait a tényleges futásából. Elvárt beadandó: a parancsok, a tesztkimenet és a manifest három kulcsa.

F4 (hibakeresési, nehézség 2). Az alábbi függvény dokumentációja mintabeli szórást ígér, a 2.4.4 konvenciója szerint viszont a *standardizáló* nevező populációs szórás; a két mennyiséget nem szabad egyetlen `ddof` értékkel összevonni:

```python
def misleading_standardize(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Visszaadja a standardizált mátrixot és a mintabeli szórást."""
    mu = x.mean(axis=0)
    sd = x.std(axis=0)          # hibagyanús sor
    denom = np.where(sd > 1e-12, sd, 1.0)
    y = (x - mu) / denom
    return y, sd
```

Lokalizálja a hibát, mutassa meg a következményét az $x = (0, 4)$ vektoron, javítsa a kódot, és írjon tesztet, amely a javítás előtt megbukna. Elvárt: ok, következmény, javítás, visszaellenőrzés.

F5 (kísérlettervezési, nehézség 3). Tervezzen ellenőrzést arra, hogy a GlyphLab `data/images.npy` mind a 360 képén a notebookból kiemelt régi függvény és az új modul azonos kimenetet ad-e. Ne futtassa le a teljes kísérletet; írja le a protokollt: input, környezet, a régi és új függvény egy processzben való futtatása, metrika, tolerancia, és a következtetés korlátja. Indokolja, miért nem elegendő az azonos seed, ha a platform megváltozik.

2.4.7 Megoldások és indoklás

F1 megoldás. A tiszta függvény első feltétele sérül: a kimenet függ a modul rejtett állapotától. Legyen `xs = [10, 20, 30]`, `_cursor = 0`; ekkor `next_batch(xs, 1)` először `[10]`-et ad, másodszor `[20]`-at, holott az argumentumok azonosak. A két hívás között a globális állapot megváltozott, tehát a függvény nem tisztán az argumentumaira épül.

F2 megoldás. Rögzítsük a j oszlopot, és legyen $\sigma_j > 0$. Indexhatárok: a j oszlopindex $1 \le j \le d$, az összegzés mindenütt $i = 1, \dots, N$ felett fut, és az átlag valamint a variancia oszloponként értendő, ezért a végeredmény d darab átlag és d darab variancia, nem egyetlen skalár. A központozott érték $C_{ij} = X_{ij} - \mu_j$, a kimenet $Y_{ij} = C_{ij}/\sigma_j$. Az átlag:

$$\frac{1}{N}\sum_{i=1}^{N} Y_{ij} = \frac{1}{\sigma_j}\left(\frac{1}{N}\sum_{i=1}^{N} X_{ij} - \mu_j\right) = 0,$$

mert a zárójelben $\mu_j - \mu_j = 0$ áll. A populációs variancia:

$$\frac{1}{N}\sum_{i=1}^{N} Y_{ij}^2 = \frac{1}{N\sigma_j^2}\sum_{i=1}^{N} C_{ij}^2 = \frac{\sigma_j^2}{\sigma_j^2} = 1,$$

mert a központozott értékek populációs varianciája definíció szerint σ_j^2. Alakellenőrzés: $Y \in \mathbb{R}^{N \times d}$, az átlag oszlopvektor $\mathbb{R}^d$-beli, a variancia nemnegatív skalárok sorozata. Ha $\sigma_j = 0$, akkor a nemnegatív négyzetösszeg nulla, tehát minden i-re $C_{ij} = 0$, azaz $X_{ij} = \mu_j$; a definiált konvenció $Y_{ij} = 0$, így az átlag 0, a variancia 0; a variancia 1 voltára vonatkozó állítás tehát csak $\sigma_j > 0$ esetén érvényes.

F3 megoldás. A labor a fenti öt tesztet futtatja. A tényleges futásból a `manifest.json`-t a hallgató olvassa ki; a kulcsok szerkezete a sémában rögzített. A könyv ellenőrző környezetében (Python 3.12.3, NumPy 2.5.4, pytest 9.2.0, CPU) a `pytest -q` kimenete `5 passed` volt; a CLI futásából származó `run_id` 32 hex karakter, az `input_sha256` `7970b0a5…`, az `output_sha256` `ce5d53cf…`. A hallgató saját futása az elsődleges bizonyíték, és a saját jegyzékéből ezeket az értékeket kell kiolvasnia. Az ellenőrzéshez a `run_id` 32 hex karakter hosszúságú, mindkét hash 64 hex karakter.

F4 megoldás. A függvény két külön mennyiséget von össze egyetlen `sd` változóban: a standardizálás nevezője a 2.4.4 konvenciója szerint populációs szórás, a dokumentált visszatérési érték viszont mintabeli szórás. A hiba oka: a NumPy `std` alapértelmezett `ddof=0` értéke populációs normalizálást használ, ezért a visszaadott `sd` nem a dokumentált mintabeli szórás. Következmény az $x = (0, 4)$ vektoron: a mintabeli szórás $s = \sqrt{((0-2)^2 + (4-2)^2)/(2-1)} = \sqrt{8} = 2\sqrt{2} \approx 2{,}8284$, a hibás kód viszont $\sqrt{8/2} = \sqrt{4} = 2$-t ad, tehát a jelentett bizonytalanság kisebb a dokumentált konvenciónál. Javítás: a két szerepet szét kell választani, és `ddof=1`-et csak a jelentett értékre alkalmazni:

```python
def misleading_standardize(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Visszaadja a standardizált mátrixot és a mintabeli szórást."""
    mu = x.mean(axis=0)
    sd_pop = x.std(axis=0)              # standardizáló nevező: populációs
    sd_sample = x.std(axis=0, ddof=1)  # dokumentált jelentett minta-szórás
    denom = np.where(sd_pop > 1e-12, sd_pop, 1.0)
    y = (x - mu) / denom
    return y, sd_sample
```

Visszaellenőrző teszt:

```python
def test_misleading_standardize_separates_sample_sd() -> None:
    x = np.array([[0.0], [4.0]])
    y, sd = misleading_standardize(x)
    np.testing.assert_allclose(sd, [2.0 * np.sqrt(2.0)], atol=1e-12)
    np.testing.assert_allclose(y.ravel(), [-1.0, 1.0], atol=1e-12)
```

A javítás előtt ez a teszt megbukik, mert a hibás kód 2-t ad $2\sqrt{2}$ helyett; utána a kívánt konvenciót rögzíti. A második állítás azt is ellenőrzi, hogy a nevező továbbra is a populációs szórás: ha valaki a nevezőre is `ddof=1`-et tenne, a standardizált érték $(-1; 1)$ helyett $(-1/\sqrt{2}; 1/\sqrt{2}) \approx (-0{,}7071; 0{,}7071)$ lenne, ami sérti a 2.4.4 populációs konvencióját.

F5 megoldás (terv, nem mért eredmény). Input: `data/images.npy`, alak `(360,1,24,24)`, float32; a sorrendet a letöltött GlyphLab adatkészlet `metadata.csv` fájljának `index` oszlopa rögzíti (ez az adathalmaz része, nem a GlyphStats projekté). A régi függvényt a kísérlet előtt változatlanul kimentjük egy `legacy.py` fájlba; az új függvényt a projektmodulból importáljuk. Ugyanabban a Python-processzben, ugyanazon a NumPy-verzión és dtype-on futtatjuk mindkettőt minden képen. Mivel a kimenet kép, a metrika a képenkénti és elemenkénti abszolút különbség maximuma, $d = \max_i \max_p |F_0(x_i)_p - F_1(x_i)_p|$, ahol p a 24×24-es kimenet elemeinek indexe; ez a skalár a legnagyobb elemenkénti eltérést méri, nem a két kép normáját. Emellett rögzítjük az `np.allclose(F_0(X), F_1(X), atol=1e-6, rtol=1e-5)` teljes tömbös döntésének eredményét is float32 mellett. A nullhipotézis: a refaktor nem változtat. A 360 képen elért egyezés csak a tesztelt tartományra lenne empirikus bizonyíték; a teljes tartományra a determinisztikus lemma ad érvelést, de csak azonos futási feltételek (interpreter, NumPy-verzió, dtype) mellett. Platformváltáskor az azonos seed nem garantál bitazonosságot [D04], ezért az összehasonlítást ugyanazon gépen kell elvégezni, vagy a toleranciát a platform numerikus zajához kell kalibrálni.

2.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. „A tiszta kód és a zöld teszt garantálja a tudományos állítás helyességét.” Ez hamis. A kódminőség az implementáció kontrollja; a modellértékelés helyessége a konvención, a spliten és a metrikán múlik. Ellenpélda: az F4 hibás függvénye kifogástalanul strukturált, és egy kizárólag a standardizált outputra épített teszt zöld lehet, mert a hiba a dokumentált statisztikai konvenció és a kód közötti eltérésben van. A teszt csak azt rögzíti, amit explicit módon beleírunk.

Önellenőrzés. (1) Meg tudom-e mondani minden függvényem bemeneti és kimeneti alakját? (2) A teszt a matematikai tulajdonságot ellenőrzi, vagy csak a kód önmagára hivatkozik? (3) A manifest tartalmazza-e a bemeneti hash-t, a környezetet és a paramétereket? (4) A `.gitignore` kizárja-e a futás közben keletkező fájlokat?

Visszatekintés. Az 1.4. Tanulási diagnózis és kísérleti napló megkülönböztette a seedet és a futásazonosítót; itt mindkettő szerephez jut: a fixture a seedből születik, a manifest `run_id`-ja a napló kulcsa. A 2.3. Izolált környezet, notebook és verzió környezeti jelentése a `README` első szakasza lett; ezzel a projekt a futtatási feltételeket a kóddal együtt szállítja.

2.4.9 Rövid összefoglaló

Ez az egység a notebookcellát kis szoftverprojektté alakítja: a pathlib a fájlútvonalakat, a modul a tiszta függvényt, a CLI az input–output–manifest szerződést kezeli. A refaktor a bemenet–kimenet leképezést őrzi; a determinisztikus lemma kimondja a feltételeket, a unit teszt ezt véges fixture-ökön ellenőrzi. A zöld teszt nem bizonyítja a teljes tartományt, és nem pótolja a statisztikai konvenció vagy a modellértékelés helyességét. A futási jegyzék hash-t, környezetet és paramétereket rögzít, a lokális artefaktumokat a `.gitignore` zárja ki. A következő egység a lineáris algebrai alapokat építi fel.

2.4.10 Források

- [D04] NumPy developers (2026). *NumPy documentation: Random sampling.* https://numpy.org/doc/stable/reference/random/index.html. Hozzáférés: 2026-10-30.

3 Lineáris algebra neurális hálózatokhoz

3.1 Vektor, norma és skalárszorzat

3.1.1 Cél, előismeret és induló kérdés

Ez az egység a neurális hálózatok mögötti lineáris algebra első, geometriai rétegét építi fel. A cél, hogy a vektor ne csak tömb legyen, hanem koordinátákkal leírható geometriai és adatreprezentációs objektum; a norma, a skalárszorzat, a távolság és az ortogonalitás pedig ellenőrizhető, dimenzióhelyes művelet legyen. Előismeret a 2.1. Python, NumPy és tenzoralak `ndarray`, `shape`, `axis` és broadcasting fogalma, valamint a 2.4. Kis tudományos szoftverprojekt kis projektben megszokott input–output szerződés.

Induló kérdés: a függelék A. szakaszában leírt GlyphLab egy 24×24-es szürke képe lapítva egy 576 elemű vektor. Ha két kép két ilyen vektor, mit jelent, hogy „közeliek”, és miért nem elég csak a pontonkénti szorzatösszegüket nézni? Becsülje meg, mekkora lehet egy ilyen képpár euklideszi távolsága, ha minden intenzitás a $[0, 1]$ intervallumban van: $\sqrt{576} = 24$ a lehető legnagyobb érték, de a tipikus érték az adat tartalmától függ. A kérdésre itt nem adunk számszerű becslést: a $[0, 1]$ intervallum csak a maximumot rögzíti, a tipikus távolságot a GlyphLab adatán kell megmérni (a labor F5 feladata), és az eredmény a scene-ek pixelstruktúrájától függ.

3.1.2 Vektor, koordináta és adatreprezentáció

Definíció (valós vektor és koordináta). Legyen $d \geq 1$ egész. Az $\mathbb{R}^d$ tér egy eleme $x = (x_1, \dots, x_d)^\top$; a könyv absztrakt képleteiben ez oszlopvektor; a batchelt mátrixos alakban viszont minden sor egy-egy megfigyelés. A standard bázisban az x_j valós számok a vektor j-edik koordinátái, vagyis $x = \sum_{j=1}^{d} x_j e_j$, ahol e_j az a vektor, amelynek j-edik koordinátája 1, a többi 0.

Két vektor, x és y, akkor és csak akkor egyenlő, ha minden $j \in \{1, \dots, d\}$ koordinátára $x_j = y_j$. A koordináták nem abszolút tulajdonságok: más bázisban ugyanahhoz a geometriai vektorhoz más számsor tartozhat. Ebben a fejezetben mindig a standard bázisban dolgozunk; a bázisváltoztatás a későbbi sajátérték-témakörben tér vissza 3.4. Sajátérték, SVD és spektrális határ.

Adatreprezentációs olvasatban egy $x \in \mathbb{R}^d$ lehet egy megfigyelés számszerű jellemzősora. A szerződés szerint egy tiszta batch-mátrix $X \in \mathbb{R}^{N \times d}$ soronként egy megfigyelést tárol: $x^{(n)} = (X_{n1}, \dots, X_{nd})^\top$. Egy `(1,24,24)` NCHW szürke képből az `x = image.reshape(-1)` kifejezés `(576,)` alakú vektort állít elő; a pixel intenzitásértékei ekkor a vektor koordinátái, a tengelyek sorrendje pedig a választott lapítási konvenció.

3.1.3 Skalárszorzat

Definíció (skalárszorzat, dot product). Az $x, y \in \mathbb{R}^d$ vektorok skalárszorzata

$$\langle x, y \rangle \;=\; \sum_{j=1}^{d} x_j y_j.$$

A kifejezés másik szokásos jelölése $x^\top y$, ahol a felső $\top$ a transzponálást jelöli; ennek mátrixos használata a 3.2. Mátrixszorzás és affín réteg egységben pontosítjuk. Az eredmény skalár, tehát nincs tengelye. Ha batchben dolgozunk, az $X, Y \in \mathbb{R}^{N \times d}$ mátrixok sorainak páronkénti skalárszorzata

$$s_n = \sum_{j=1}^{d} X_{nj} Y_{nj}, \quad n = 1, \dots, N,$$

ami `np.sum(X*Y, axis=1)` alakú, `(N,)` eredménnyel. A szorzás itt elemenkénti, a redukció a j tengelyen történik.

Az alábbi tulajdonságok közvetlenül a véges összeg definíciójából következnek, ezért külön tétel helyett levezetésként rögzítjük őket. Legyen $x, y, z \in \mathbb{R}^d$ és $\alpha, \beta \in \mathbb{R}$.

1. **Szimmetria:** $\langle x, y \rangle = \langle y, x \rangle$, mert a szorzás kommutatív.
2. **Linearitás az első változóban:** $\langle \alpha x + \beta z, y \rangle = \alpha \langle x, y \rangle + \beta \langle z, y \rangle$. A szimmetria miatt ugyanez a második változóban is igaz, ezért a művelet **bilineáris**; a szó azt jelenti, hogy a művelet mindkét változójában lineáris.
3. **Pozitív definitség:** $\langle x, x \rangle = \sum_j x_j^2 \geq 0$, és $\langle x, x \rangle = 0$ pontosan akkor, ha minden $x_j = 0$, azaz $x = 0$.

A pozitív definitség azért döntő, mert ebből lesz a geometriai hossz. A skalárszorzat önmagában nem hasonlósági döntés: az értéke függ a vektorok hosszától is.

3.1.4 Normák és távolság

Definíció (euklideszi norma). Egy $x \in \mathbb{R}^d$ vektor euklideszi normája, más néven ℓ_2-normája:

$$\|x\|_2 = \sqrt{\langle x, x \rangle} = \left(\sum_{j=1}^{d} x_j^2 \right)^{1/2}.$$

A nemnegatív gyök egyértelmű; $\|x\|_2 \geq 0$, és $\|x\|_2 = 0$ pontosan akkor, ha $x = 0$. A norma a vektor origótól mért geometriai hossza. A skalárszorzatból a $\|x\|_2 = \sqrt{\langle x, x \rangle}$ képlettel kapott normát **indukált normának** nevezzük; nem minden norma indukált ilyen módon (az ℓ_1-norma például nem). Érvényes a homogenitás: $\|\alpha x\|_2 = |\alpha| \, \|x\|_2$, mert

$$\|\alpha x\|_2 = \left(\sum_{j=1}^{d} (\alpha x_j)^2 \right)^{1/2} = \left(\alpha^2 \sum_{j=1}^{d} x_j^2 \right)^{1/2} = \sqrt{\alpha^2} \, \|x\|_2 = |\alpha| \, \|x\|_2,$$

ahol a $\sqrt{\alpha^2} = |\alpha|$ lépés minden valós α-ra érvényes (negatív α esetén is).

Általánosabban, $p \geq 1$ esetén az ℓ_p-norma

$$\|x\|_p = \left(\sum_{j=1}^{d} |x_j|^p \right)^{1/p},$$

és $\|x\|_\infty = \max_j |x_j|$. A $p = 1$ eset a Manhattan-norma. A háromszög-egyenlőtlenség általános ℓ_p-normákra nem *következménye*, hanem maga a Minkowski-egyenlőtlenség (amely éppen az ℓ_p-norma háromszög-egyenlőtlensége); ezt itt nem bizonyítjuk, csak a tanulási útvonalhoz szükséges $p = 2$ esetet vezetjük le a következő szakaszban.

Definíció (euklideszi távolság). Az $x, y \in \mathbb{R}^d$ pontok távolsága

$$d(x, y) = \|x - y\|_2.$$

A definícióból és a norma tulajdonságaiból azonnal adódik, hogy $d(x, y) \geq 0$, $d(x, y) = 0$ pontosan akkor, ha $x = y$, és $d(x, y) = d(y, x)$. A háromszög-egyenlőtlenséget a következő tétel garantálja.

3.1.5 Ortogonalitás és koszinusz-hasonlóság

Definíció (ortogonalitás). Az $x, y \in \mathbb{R}^d$ vektorok ortogonálisak, jelölésben $x \perp y$, ha $\langle x, y \rangle = 0$. A nullvektor minden vektorra ortogonális, mert minden szorzatösszeg tagja nulla. Egy $u \in \mathbb{R}^d$ vektor egységvektor, ha $\|u\|_2 = 1$. Az $\{u_1, \ldots, u_k\}$ halmaz ortonormált, ha minden elem egységvektor és páronként ortogonális.

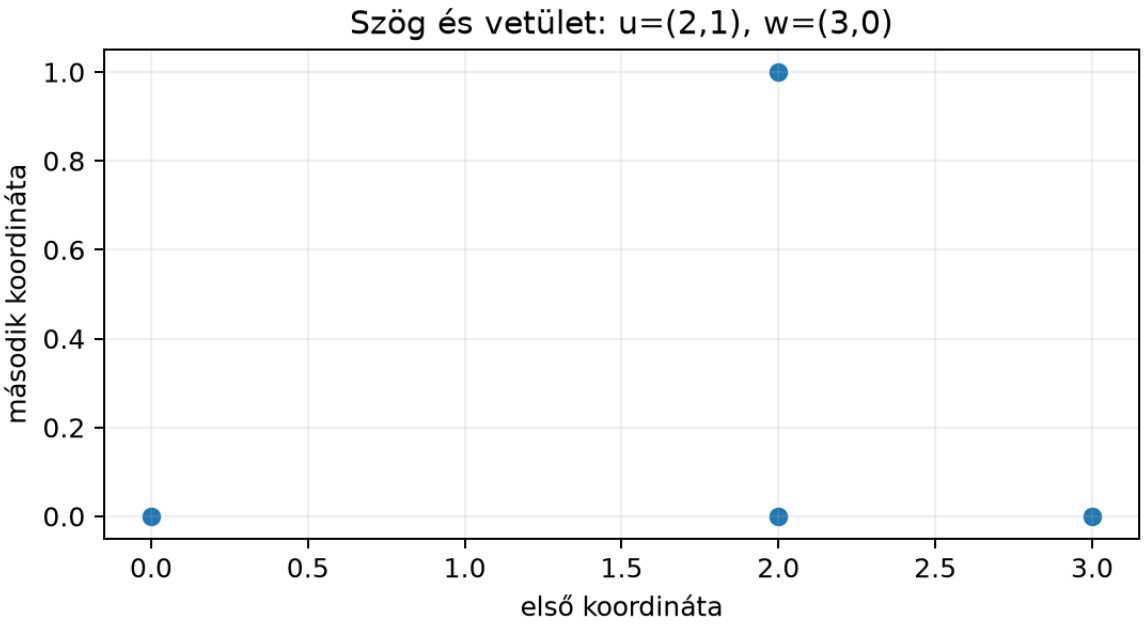
Ha $x \neq 0$, az $x/\|x\|_2$ vektor egységvektor, az x irányába mutat. A normalizálás a nullvektorra nem értelmezhető.

Definíció (koszinusz-hasonlóság). Ha $x \neq 0$ és $y \neq 0$, akkor

$$\cos(x, y) = \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}.$$

A következő szakasz Cauchy–Schwarz-lemmája szerint ez mindig a $[-1, 1]$ intervallumba esik. A név geometriai: ez a két irány közötti szög koszinusza. Az érték 1, ha a vektorok azonos irányú pozitív skalárszorosai, -1, ha ellentétes irányúak, és 0, ha ortogonálisak. A koszinusz-hasonlóság nem távolság, mert az azonos irányú, de különböző hosszúságú vektorokra 1, noha a vektorok nem egyenlők.

A szög és a vetület geometriai jelentését egy kétdimenziós példa mutatja. Legyen $u = (2, 1)^\top$ és $w = (3, 0)^\top$; ekkor $\cos(u, w) = \frac{6}{\sqrt{5} \cdot 3} = \frac{2}{\sqrt{5}} \approx 0{,}894$, és az u vetülete a w irányára a $(2, 0)^\top$ pont, mert $\frac{\langle u, w \rangle}{\langle w, w \rangle} w = \frac{6}{9}(3, 0)^\top = (2, 0)^\top$.



3.1. ábra: Szög és vetület: u=(2,1), w=(3,0)

Képletből számolt szemléltetés, nem mérés: az origó, az u és w végpontja, valamint az u vetülete a w irányára. A cos(u,w)=2/sqrt(5) érték és a (2,0) vetületpont a szövegben levezetett számításból származik.

3.1.6 Cauchy–Schwarz és a háromszög-egyenlőtlenség

Az alábbi lemma a valós skalárszorzatra vonatkozó Cauchy–Schwarz-egyenlőtlenség. A bizonyítás diszkriminánson alapuló érvelés; nem használ mértékelméletet és nem támaszkodik külső tételre. A terminológia a [B01] lineáris algebrai keretével konzisztens, de a részletes levezetés itt saját kidolgozás. Két vektor lineárisan összefüggő, ha valamelyik a másik skalárszorosa; ellenkező esetben lineárisan független.

Lemma (Cauchy–Schwarz). *Legyen $x, y \in \mathbb{R}^d$, és legyen $\|x\|_2 = \sqrt{\langle x, x \rangle}$. Ekkor*

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2.$$

Egyenlőség pontosan akkor áll fenn, ha x és y lineárisan összefüggő, vagyis $x = 0$, $y = 0$, vagy valamelyik a másik skalárszorosa.

Bizonyítás. Ha $y = 0$, akkor mindkét oldal nulla, és az állítás igaz. Legyen most $y \neq 0$. Minden $t \in \mathbb{R}$ esetén

$$0 \leq \|x + ty\|_2^2 = \langle x + ty, x + ty \rangle = t^2 \|y\|_2^2 + 2t\langle x, y \rangle + \|x\|_2^2,$$

ahol az átalakítás a bilinearitás és szimmetria definíciójából következik. Ez t-ben másodfokú kifejezés, amelynek főegyütthatója $A = \|y\|_2^2 > 0$. Mivel a kifejezés soha nem negatív, a másodfokú polinomnak legfeljebb egy valós gyöke lehet, tehát diszkriminánsa nem pozitív:

$$D = (2\langle x, y\rangle)^2 - 4\|y\|_2^2\|x\|_2^2 \le 0.$$

Osztva 4-gyel és átrendezve $\langle x, y\rangle^2 \le \|x\|_2^2\|y\|_2^2$. A nemnegatív oldalakon a gyökvonás monoton, így $|\langle x, y\rangle| \le \|x\|_2\|y\|_2$.

Az egyenlőség esete: ha a diszkrimináns nulla, akkor a másodfokú kifejezésnek van t_0 valós gyöke, amire $\|x + t_0 y\|_2^2 = 0$, tehát a pozitív definitség miatt $x + t_0 y = 0$, azaz $x = -t_0 y$, így a két vektor lineárisan összefüggő (az $y \neq 0$ ágban). Megfordítva, tegyük fel, hogy $y \neq 0$ és $x = \alpha y$ valamely $\alpha \in \mathbb{R}$-re. Ekkor

$$\langle x, y\rangle = \alpha\|y\|_2^2, \qquad \|x\|_2\|y\|_2 = |\alpha|\,\|y\|_2^2,$$

ezért $|\langle x, y\rangle| = |\alpha|\,\|y\|_2^2 = \|x\|_2\|y\|_2$. Az $\alpha = 0$ esetben $x = 0$, és mindkét oldal nulla. A $y = 0$ esetben szintén mindkét oldal nulla, és x, y lineárisan összefüggő. A két irány együtt adja a kimondott feltételt. □

Tétel (háromszög-egyenlőtlenség az euklideszi normára). *Legyen $x, y \in \mathbb{R}^d$, és $\|x\|_2 = \sqrt{\langle x, x\rangle}$. Ekkor*

$$\|x + y\|_2 \le \|x\|_2 + \|y\|_2.$$

Egyenlőség pontosan akkor áll fenn, ha $y = 0$, $x = 0$, vagy $y = \alpha x$ valamely $\alpha \ge 0$ valós számra.

Bizonyítás. A norma definícióját kétszer alkalmazva:

$$\|x + y\|_2^2 = \langle x + y, x + y\rangle = \|x\|_2^2 + 2\langle x, y\rangle + \|y\|_2^2.$$

Mivel $\langle x, y\rangle \le |\langle x, y\rangle|$, a Cauchy–Schwarz-lemma szerint

$$\|x + y\|_2^2 \le \|x\|_2^2 + 2\|x\|_2\|y\|_2 + \|y\|_2^2 = (\|x\|_2 + \|y\|_2)^2.$$

Mindkét oldal nemnegatív, ezért a négyzetgyök monotonitása miatt $\|x + y\|_2 \le \|x\|_2 + \|y\|_2$.

Az egyenlőséghez a fenti egyenlőtlenségek mindegyikében egyenlőség kell. A Cauchy–Schwarz-lemma egyenlőségi feltétele a lineáris összefüggőség; a $\langle x, y\rangle = |\langle x, y\rangle|$ feltétel pedig azt kívánja, hogy a skalárszorzat nemnegatív legyen. Ez pontosan az állításban szereplő azonos irányú eset. □

A tételből a távolságra is megkapjuk a háromszög-egyenlőtlenséget: tetszőleges $x, y, z \in \mathbb{R}^d$ esetén

$$d(x, z) = \|(x - y) + (y - z)\|_2 \le \|x - y\|_2 + \|y - z\|_2 = d(x, y) + d(y, z).$$

Ezzel az euklideszi távolság a metrikaaxiómákat is teljesíti: (M1) $d(x, y) \ge 0$, és $d(x, y) = 0$ pontosan akkor, ha $x = y$; (M2) $d(x, y) = d(y, x)$; (M3) $d(x, z) \le d(x, y) + d(y, z)$. A három axióma közül az első kettőt a norma tulajdonságai, a harmadikat az előző levezetés adja.

3.1.7 Kézi példa és ellenőrző kód

Legyen $x = (3, 4)^\top$ és $y = (-4, 3)^\top$. A skalárszorzat:

$$\langle x, y \rangle = 3 \cdot (-4) + 4 \cdot 3 = 0,$$

tehát a két vektor ortogonális. Mindkettő normája $\sqrt{9 + 16} = 5$, így ortonormált párt alkotnak a normalizált $\hat{x} = (0.6, 0.8)^\top$ és $\hat{y} = (-0.8, 0.6)^\top$ vektorok (a kalap a normalizálást jelöli; ezek nem tévesztendők össze a standard e_j bázisvektorokkal). A távolság

$$d(x, y) = \|x - y\|_2 = \|(7, 1)^\top\|_2 = \sqrt{50} \approx 7.071.$$

A Cauchy–Schwarz-egyenlőtlenség itt $0 \le 25$, a háromszög-egyenlőtlenség pedig $\|x + y\|_2 = \sqrt{50} \approx 7.071 \le 10$. Ortogonális vektoroknál $\|x + y\|_2 = \|x - y\|_2$, mert a kereszttag eltűnik.

A skálázás hatása ugyanezen a páron: az αx vektor és y skalárszorzata $\alpha \cdot 0 = 0$ minden α-ra, tehát az ortogonalitás minden α esetén megmarad (a zérusvektor is ortogonális minden vektorhoz), de $\alpha = 0$ esetén a koszinusz-hasonlóság nem értelmezhető. A távolság viszont

$$d(\alpha x, y) = \|(\alpha x) - y\|_2 = \sqrt{(3\alpha + 4)^2 + (4\alpha - 3)^2} = \sqrt{25\alpha^2 + 25},$$

ami $\alpha = 0$-nál 5, $\alpha = \pm 1$-nél $\sqrt{50}$, és nagy $|\alpha|$-ra körülbelül $5|\alpha|$; tehát a távolság általában nem $|\alpha| d(x, y)$.

A munkakönyvtár `labs/n009_vector_norm/`. Az alábbi munkamenet CPU-n fut, a számítás `float64` pontosságú, véletlen számot nem használ, ezért seedre nincs szükség, és PyTorchöt sem használ. Futtatási környezet: Python 3.12.3, NumPy 2.5.4, CPU, egy szál (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`). A kód a levezetett értékeket `assert`-tel ellenőrzi.

```
# labs/n009_vector_norm/vector_lab.py
import numpy as np

def dot(x, y):
    return float(np.dot(x, y))

def norm(x):
    return float(np.sqrt(np.dot(x, x)))

def cosine(x, y):
    # a laborpéldákban egyik vektor sem nullvektor
    return dot(x, y) / (norm(x) * norm(y))

x = np.array([3.0, 4.0], dtype=np.float64)
y = np.array([-4.0, 3.0], dtype=np.float64)

print("dot", dot(x, y))
print("norm_x", norm(x), "norm_y", norm(y))
print("cosine", cosine(x, y))
print("distance", norm(x - y))
print("triangle_lhs", norm(x + y), "triangle_rhs", norm(x) + norm(y))

assert abs(dot(x, y)) < 1e-12
assert abs(norm(x) - 5.0) < 1e-12
assert abs(norm(y) - 5.0) < 1e-12
assert abs(cosine(x, y)) < 1e-12
```

```
assert abs(norm(x - y) - np.sqrt(50.0)) < 1e-12
assert norm(x + y) <= norm(x) + norm(y) + 1e-12
```

A futtatott kód tényleges kimenete:

```
dot 0.0
norm_x 5.0 norm_y 5.0
cosine 0.0
distance 7.0710678118654755
triangle_lhs 7.0710678118654755 triangle_rhs 10.0
```

3.1.8 Labor: három vektorpár és a skálázás hatása

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Beadandó eredmény. Tekintse az alábbi három kétdimenziós vektorpárt:

$$u_1 = (2,1)^\top,\ v_1 = (-1,2)^\top; \quad u_2 = (3,0)^\top,\ v_2 = (0,4)^\top; \quad u_3 = (1,1)^\top,\ v_3 = (2,2)^\top.$$

Kézzel számítsa ki mindhárom párra a skalárszorzatot, a két normát, a koszinusz-hasonlóságot és a távolságot. Az ellenőrző indoklás: $u_1 \perp v_1$, mert $2 \cdot (-1) + 1 \cdot 2 = 0$; $u_2 \perp v_2$, mert a tengelyvektorokra a szorzat 0; $v_3 = 2u_3$, ezért a pár párhuzamos és azonos irányú.

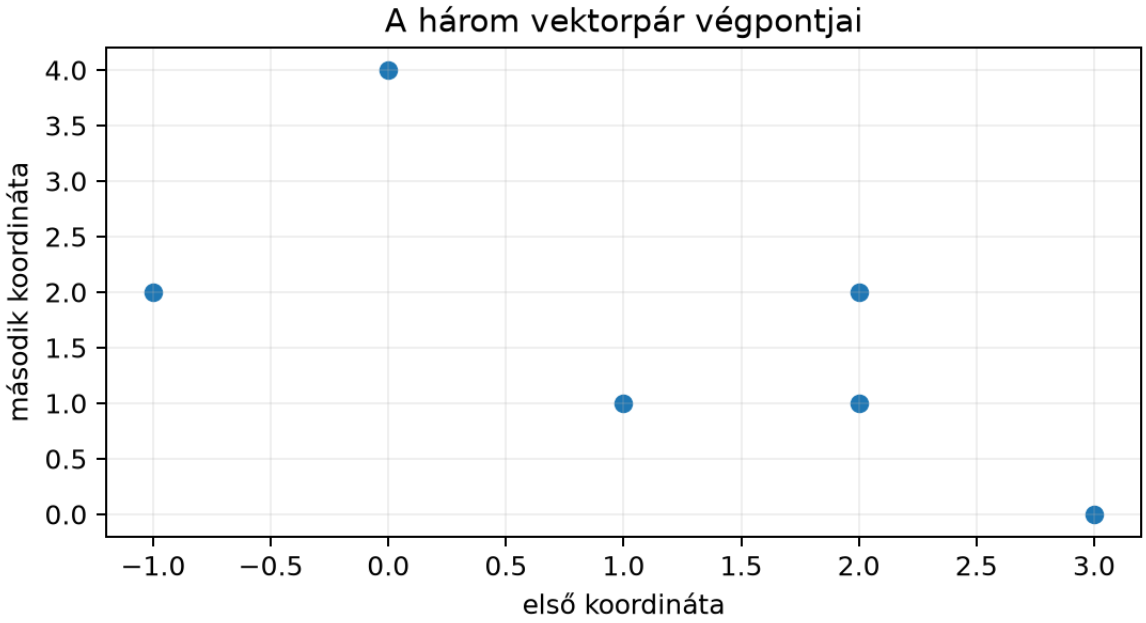
pár	skalárszorzat	koszinusz	euklideszi távolság
(u_1, v_1)	0	0	$\sqrt{10} \approx 3.162$
(u_2, v_2)	0	0	5
(u_3, v_3)	4	1	$\sqrt{2} \approx 1.414$

A kihagyott normák: u_1 és v_1 normája $\sqrt{5} \approx 2.236$; u_2 normája 3, v_2 normája 4; u_3 normája $\sqrt{2}$, v_3 normája $2\sqrt{2}$. A harmadik párra a Cauchy–Schwarz-egyenlőtlenség egyenlőséggel teljesül: $4 = \sqrt{2} \cdot 2\sqrt{2}$. A háromszög-egyenlőtlenség is egyenlőséggel teljesül: $\|u_3 + v_3\|_2 = \|(3,3)^\top\|_2 = 3\sqrt{2}$, míg $\|u_3\|_2 + \|v_3\|_2 = \sqrt{2} + 2\sqrt{2} = 3\sqrt{2}$.

A labor második része a skálázás. Rögzítse $u = u_1$, $v = v_1$, és vizsgálja az αu vektort. A skalárszorzat $\langle \alpha u, v \rangle = \alpha \cdot 0 = 0$ minden α-ra, tehát az ortogonalitás $\alpha \neq 0$ esetén megmarad. A koszinusz $\alpha > 0$-ra 0, $\alpha < 0$-ra 0, $\alpha = 0$-ra nem értelmezett. A távolság viszont

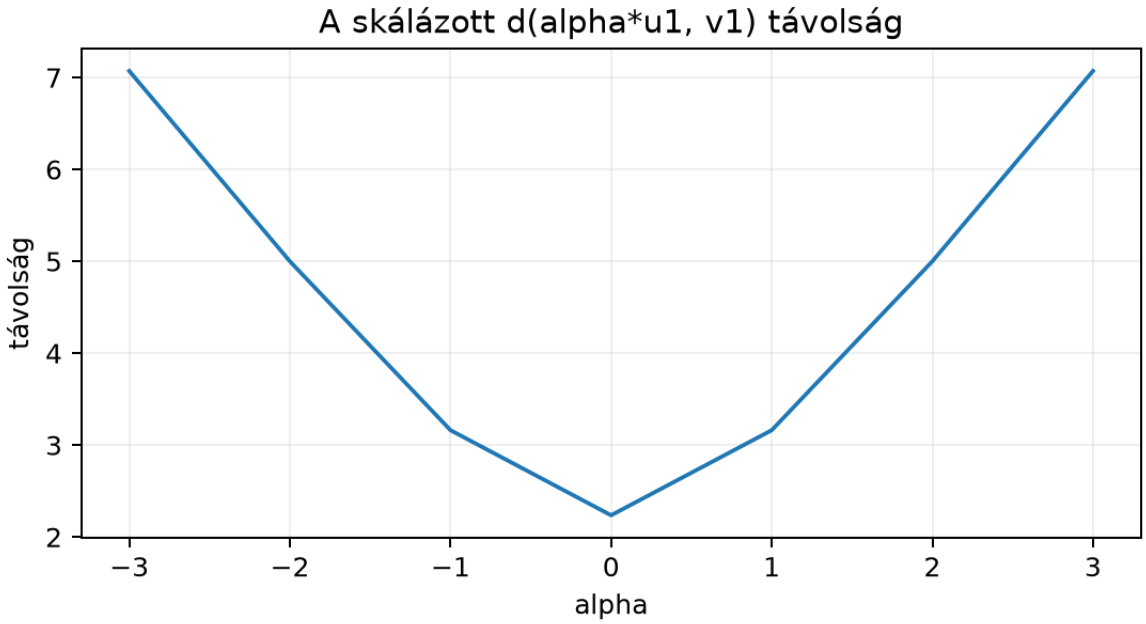
$$d(\alpha u, v) = \|(\alpha u) - v\|_2 = \sqrt{(2\alpha + 1)^2 + (\alpha - 2)^2} = \sqrt{5\alpha^2 + 5}.$$

A görbe minimuma $\alpha = 0$-nál $\sqrt{5} \approx 2.236$; az $\alpha = 1$ érték $\sqrt{10} \approx 3.162$. Ezért a távolság nem mondható meg pusztán a skálázott vektor hosszából, és a „nagyobb hossz közelebb visz” állítás hamis.



3.2. ábra: A három vektorpár végpontjai

A vektorok origóból induló szakaszok; a diagram csak a kézzel számított végpontokat jelöli. A külön futtatott vector_plot.py a nyilakat quiverrel rajzolja.



3.3. ábra: A skálázott d(alpha*u1, v1) távolság

*Képletből számolt szemléltetés, nem mért futás. d = sqrt(5*alpha^2 + 5).**

A vektorok origóból induló ábrázolásához az alábbi laborparancs használható; a `quiver` az origóból a végpontba mutató nyilakat rajzol. A szkriptet a korábban megadott környezetben

futtattuk (Python 3.12.3, NumPy 2.5.4, matplotlib 3.11.2, CPU, egy szál, `MPLBACKEND=Agg`), és a `vector_pairs.png` elkészült.

```
# labs/n009_vector_norm/vector_plot.py
import matplotlib.pyplot as plt
import numpy as np

pairs = [
    (np.array([2.0, 1.0]), np.array([-1.0, 2.0])),
    (np.array([3.0, 0.0]), np.array([0.0, 4.0])),
    (np.array([1.0, 1.0]), np.array([2.0, 2.0])),
]
assert abs(np.dot(pairs[0][0], pairs[0][1])) < 1e-12
assert abs(np.dot(pairs[1][0], pairs[1][1])) < 1e-12
assert np.allclose(pairs[2][1], 2.0 * pairs[2][0])
fig, ax = plt.subplots(figsize=(5, 5))
for u, v in pairs:
    ax.quiver(0, 0, u[0], u[1], angles="xy", scale_units="xy",
              scale=1, color="black")
    ax.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy",
              scale=1, color="gray")
ax.set_xlim(-2, 4)
ax.set_ylim(-2, 5)
ax.set_aspect("equal")
ax.grid(True)
fig.savefig("vector_pairs.png", dpi=150)
```

A futtatott ábrán az első pár két, egymásra merőleges szakasz; a második pár a két tengelyen fekvő szakasz; a harmadik pár két azonos irányú, 1:2 hosszarányú szakasz.

3.1.9 Feladatok

F1 (fogalmi, nehézség 1). Legyen $x = (2, 0)^\top$, $y = (2, \varepsilon)^\top$ és $z = (0, 10)^\top$. Számítsa ki $\langle x, y\rangle$, $\langle x, z\rangle$, $d(x, y)$ és $d(x, z)$ értékét $\varepsilon = 0.1$ esetén. Elvárt beadandó eredmény: a négy numerikus érték, valamint egy saját szavas indoklás arról, miért hamis az az állítás, hogy a nagyobb skalárszorzat nagyobb hasonlóságot jelent.

F2 (levezetési/bizonyítási, nehézség 2). Bizonyítsa be a paralelogramma-azonosságot az euklideszi normára: minden $x, y \in \mathbb{R}^d$ esetén $\|x + y\|_2^2 + \|x - y\|_2^2 = 2\|x\|_2^2 + 2\|y\|_2^2$. Mutassa meg az $x = (1, 0)^\top$, $y = (0, 1)^\top$ párral, hogy az ℓ_1-normára ugyanez az azonosság nem teljesül. Elvárt beadandó eredmény: a levezetés minden skalárszorzat-felbontással, és a konkrét ℓ_1-ellenőrzés számai.

F3 (implementációs, nehézség 1). Írjon két tiszta NumPy-függvényt: `batched_norm(X)` és `batched_cosine(X, Y)`. Az input $X, Y \in \mathbb{R}^{N \times d}$, dtype `float64`; a kimenet mindkét esetben `(N,)` alakú, dtype `float64`. A koszinuszfüggvény nulla normájú sorra `nan` értéket adjon, és ne adjon ki osztási figyelmeztetést. Elvárt beadandó eredmény: a függvénykód, egy `(3,2)` tesztbemenet és annak ellenőrzött kimenete.

F4 (hibakeresési, nehézség 2). Az alábbi függvény koszinusz-hasonlóságot ígér:

```
import numpy as np

def cosine_buggy(a, b):
    return np.dot(a, b) / np.linalg.norm(a * b)
```

Lokalizálja a hibát, mutassa meg egy konkrét számpárral, hogy a kimenet helytelen, és írja be a javított sort. Elvárt beadandó eredmény: a hibás részkifejezés azonosítása, az $a = (1, 1)^\top$, $b = (2, 0)^\top$ pár hibás és helyes értéke, valamint a javított kód.

F5 (kísérleti/transzfer, nehézség 2). Tervezzen egy leíró, nem modellválasztó kísérletet a GlyphLab adatán: az azonos scene két view-jának lapított, 576 elemű vektoraira számított euklideszi távolságokat hasonlítsa össze különböző scene-ek vektorpárjaival. A GlyphLab adatszerződését a függelék A. szakasza írja le: egy `scene_id`-hez két zajos `view` tartozik, és a két view ugyanabba a splitbe kerül; a scene a csoportosítás egysége a 1.1. Modell, tanulás és felhasználási szerződés szerint, a megfigyelési egységet az 1.3. Adatfelosztás és a megfigyelési egység rögzíti. Előfeltétel továbbá a 2.1. Python, NumPy és tenzoralak lapítás. Elvárt beadandó eredmény: mintavételi egység, párosítási protokoll, baseline, legalább két összefoglaló metrika és a következtetési korlát.

3.1.10 Megoldások és indoklás

F1 megoldása. A skalárszorzatok: $\langle x, y\rangle = 4$, $\langle x, z\rangle = 0$. A távolságok: $d(x, y) = \|(0, -0.1)^\top\|_2 = 0.1$, $d(x, z) = \|(2, -10)^\top\|_2 = \sqrt{104} \approx 10.198$. A skalárszorzat tehát y-ra nagyobb, mint z-re, de az euklideszi távolság szerint y sokkal közelebb van, mint z. Az ellentmondás oka, hogy a skalárszorzat a hosszakkal és az iránnyal együtt nő: z hossza 10, ezért a nulla skalárszorzat ellenére nagy távolságra van.

F2 megoldása. A norma négyzetét a skalárszorzatra visszavezetve:

$$\|x + y\|_2^2 + \|x - y\|_2^2 = \langle x + y, x + y\rangle + \langle x - y, x - y\rangle.$$

A bilinearitással felbontva az első tag $\|x\|_2^2 + 2\langle x, y\rangle + \|y\|_2^2$, a második $\|x\|_2^2 - 2\langle x, y\rangle + \|y\|_2^2$. Az összegben a kereszttag kiesik, marad $2\|x\|_2^2 + 2\|y\|_2^2$. Ez az azonosság a Cauchy–Schwarz lemmától független, csak a skalárszorzat bilinearitását használja.

Az ℓ_1-ellenőrzés: $x = (1, 0)^\top$, $y = (0, 1)^\top$ esetén $x + y = (1, 1)^\top$ és $x - y = (1, -1)^\top$. A bal oldal $2^2 + 2^2 = 8$, a jobb oldal $2 \cdot 1^2 + 2 \cdot 1^2 = 4$. Az ℓ_1-norma tehát valódi norma, de nem euklideszi indukált norma.

F3 megoldása.

```
import numpy as np

def batched_norm(X):
    return np.sqrt(np.einsum("ij,ij->i", X, X))

def batched_cosine(X, Y):
    denom = batched_norm(X) * batched_norm(Y)
    num = np.einsum("ij,ij->i", X, Y)
    with np.errstate(divide="ignore", invalid="ignore"):
        return num / denom

X = np.array([[2.0, 1.0],
              [3.0, 0.0],
              [1.0, 1.0]], dtype=np.float64)
Y = np.array([[-1.0, 2.0],
              [0.0, 4.0],
              [2.0, 2.0]], dtype=np.float64)

print("X shape", X.shape, "Y shape", Y.shape)
print("norm", batched_norm(X))
print("cosine", batched_cosine(X, Y))
```

```
assert X.shape == (3, 2) and Y.shape == (3, 2)
assert np.allclose(batched_norm(X), [np.sqrt(5.0), 3.0, np.sqrt(2.0)])
assert np.allclose(batched_cosine(X, Y), [0.0, 0.0, 1.0])
```

A futtatott kód tényleges kimenete:

```
X shape (3, 2) Y shape (3, 2)
norm [2.23606798 3.         1.41421356]
cosine [0. 0. 1.]
```

Alakellenőrzés: az `einsum` a j tengelyt húzza össze, így `(N,)` az eredmény. A nulla soros inputra a nevező 0, és a `np.errstate` alatt a hányados `nan` lesz.

F4 megoldása. A hiba a nevezőben van: `a * b` NumPy-ban elemenkénti szorzat, így `np.linalg.norm(a * b)` az $(a_1 b_1, \dots, a_d b_d)$ vektor normája, nem pedig $\|a\|_2 \|b\|_2$. Az $a = (1,1)^\top$, $b = (2,0)^\top$ párra a hibás nevező $\|(2,0)^\top\|_2 = 2$, a számláló 2, tehát a függvény 1-et ad. A helyes érték $2/(\sqrt{2} \cdot 2) = 1/\sqrt{2} \approx 0.707$. A javítás:

```
def cosine_fixed(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

F5 megoldásminta. Megfigyelési egység: `scene_id`, mert egy jelenet két nézete nem független 1.3. Adatfelosztás és a megfigyelési egység. Párosítás: mind a 180 azonos jelenetű nézetpárt használjuk; kontrollként minden jelenet `view_id` $= 0$ nézetét a következő jelenet `view_id` $= 0$ nézetével párítjuk, és a sorrendet ciklikusan zárjuk (az utolsó jelenet után az első következik), így 180 különböző jelenetű pár keletkezik; a ciklikus zárás a párosítási protokoll része, nem külön döntés. Előfeldolgozás: `img.reshape(-1)` a `(1,24,24)` tenzorból, `float64` számítás, nincs további skálázás. Metrika: a páronkénti $d(x,y) = \|x - y\|_2$, összefoglalásként a két csoport átlaga és a 95. percentilis. Baseline: a különböző párok távolságeloszlása a kontroll. Egy véletlen párosítás várható találati arányát (az azonos jelenetű párok arányát véletlen hozzárendelésnél) itt nem használjuk döntési küszöbként: az értéke a scene-ek számától és a párosítás módjától függ, a leíró kísérletben pedig nincs bináris döntés. Ha az azonos jelenetű párok átlagos távolsága kisebb, az összefüggést jelez a jelenetazonosság és a pixelközelség között; ok-okozati következtetés (például hogy a közös pixelstruktúra okozza a kisebb távolságot) nem vonható le a leíró statisztikából. Korlát: a kísérlet nem választ modellt, csak leíró; az eredmény függ a nyers pixelreprezentációtól és a megvilágítási eltolódástól, ezért nem általános képminőségi állítás.

3.1.11 Tipikus tévedés ellenpéldával

Gyakori hiba: „a skalárszorzat nagyobb, tehát a két vektor hasonlóbb". Az F1 ellenpélda ezt cáfolja: legyen $x = (2,0)^\top$, $z' = (1000,0)^\top$ és $w = (0;\, 0{,}1)^\top$. Ekkor $\langle x, z' \rangle = 2000$ és $\langle x, w \rangle = 0$, mégis $d(x,w) = \sqrt{4 + 0{,}01} \approx 2{,}0025$, míg $d(x,z') = 998$. A skalárszorzat nyers, hosszfüggő mennyiség. A koszinusz-hasonlóság ezt a hosszhatást kiküszöböli, de a hasonlóság még így sem abszolút: ha $u = (1,1)^\top$ és $v = (2,2)^\top$, a koszinusz 1, noha $d(u,v) = \sqrt{2}$. A komponensenkénti eltérő skálázás ugyancsak megváltoztatja a geometriát: $u = (1,0)^\top$, $v = (0,1)^\top$ távolsága $\sqrt{2}$, de ha az első koordinátát 100-zal, a másodikat 1-gyel szorozzuk, $u' = (100,0)^\top$, $v' = (0,1)^\top$ távolsága $\sqrt{10001} \approx 100$. A választott reprezentáció és mértékegység tehát része a hasonlósági állításnak; ortonormált bázisváltoztatás nélkül a megőrzés nem automatikus.

3.1.12 Önellenőrzés és visszacsatolás

A háromszög-egyenlőtlenség bizonyításában a négyzetre emelés megfordítása azért érvényes, mert mindkét oldal nemnegatív. A Cauchy–Schwarz-lemmában az $y = 0$ esetet külön kezeltük, mert ott a diszkriminánson alapuló érvelés nem alkalmazható. A batch-számításnál az `axis=1` redukció `(N,)` eredményt ad; az `axis` nélküli `np.sum(X*Y)` skalárrá összegez, ami nem a soronkénti dot. A koszinusz csak nemnulla vektorokra definiált; a nullvektoros normalizálás `nan`-t vagy hibát ad, és ezt a hívó szerződésben rögzíteni kell. Ez a pont visszahívja a 2.1. Python, NumPy és tenzoralak alak- és broadcasting-ellenőrzését: a redukció tengelye a matematikai indexelésnek felel meg, nem tetszőleges megjelenítési részlet.

3.1.13 Rövid összefoglaló

A d-dimenziós valós vektor standard bázisbeli koordinátákkal adott; a skalárszorzat koordinátánkénti szorzatösszeg. Az euklideszi norma a skalárszorzatból indukált hossz, a távolság a különbségvektor hossza. A Cauchy–Schwarz-egyenlőtlenségből levezett háromszög-egyenlőtlenség a távolság metrikatulajdonságát adja. Az ortogonalitás nulla skalárszorzat; a koszinusz-hasonlóság a hosszakra normalizált irányhasonlóság. A labor három vektorpárját kézzel számítottuk ki; a vektorábra szkriptje a skalárszorzatokat, valamint az ortogonalitást és a párhuzamosságot NumPy-ellenőrzéssel is rögzíti, a normák, a koszinusz és a távolság számait viszont a kézi számítás adja. A labor megmutatta, hogy a távolság egy vektor skálázásakor általában nem lineárisan változik. A hasonlóság és távolság a választott bázistól és skálától függ.

3.1.14 Továbblépés

A 3.2. Mátrixszorzás és affín réteg egység a skalárszorzatot a lineáris réteg építőköveként használja: a $W \in \mathbb{R}^{m \times d}$ súlymátrix minden sora és az x bemenet egy-egy skalárszorzatot képez, a bias hozzáadása pedig az affin eltolást adja.

3.1.15 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.

3.2 Mátrixszorzás és affín réteg

3.2.1 Cél, előismeret és induló becslés

Ez az egység a 3.1. Vektor, norma és skalárszorzat alfejezetben bevezetett skalárszorzatot használja fel: egy lineáris réteg minden kimeneti komponense a súlymátrix egy sora és a bemeneti vektor skalárszorzata. A cél a lineáris és az affin leképezés pontos megkülönböztetése (a bias és az affin leképezés definícióját a 3.2.3. szakasz adja), a kötegelt (batchelt) $XW^\top + b$ forma indoklása, valamint annak bizonyítása, hogy két lineáris réteg kompozíciója lineáris, míg nemnulla effektív bias ($W_2 b_1 + b_2 \neq 0$) esetén az affin kompozíció nem lineáris. A numerikus ellenőrzéshez a 2.2. Lebegőpont és stabil elemi számítás alfejezetben tárgyalt `allclose` és tolerancia fogalmát idézzük vissza.

Induló becslés. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab a könyv saját, procedurálisan generált 24×24-es szürkeárnyalatos képkészlete; egy képet sorvektorrá

lapítva $24 \cdot 24 = 576$ dimenziós. Egy háromkimenetű, bias nélküli lineáris réteg hány skaláris szorzat–összeadást végez egy mintán? A számítás: minden kimeneti komponenshez egy 576 hosszú skalárszorzat tartozik, tehát $3 \cdot 576 = 1728$ szorzás–összeadás; a bias hozzáadása ezen felül kimenetenként egy összeadás.

3.2.2 Lineáris operátor és mátrixreprezentáció

Definíció (lineáris leképezés). Az $L : \mathbb{R}^d \to \mathbb{R}^m$ leképezés *lineáris*, ha minden $x, y \in \mathbb{R}^d$ és $\alpha, \beta \in \mathbb{R}$ esetén

$$L(\alpha x + \beta y) = \alpha L(x) + \beta L(y).$$

Egy ilyen leképezést *lineáris operátornak* is nevezünk. Rögzített bázisban minden lineáris leképezés egy mátrixszal reprezentálható. A standard bázis legyen $e_1, \dots, e_d$, ahol e_j j-edik koordinátája 1, a többi 0. Mivel minden $x \in \mathbb{R}^d$ felírható $x = \sum_{j=1}^{d} x_j e_j$ alakban, a linearitásból

$$L(x) = \sum_{j=1}^{d} x_j L(e_j).$$

Ha $L(e_j) \in \mathbb{R}^m$ vektorokat egy $W \in \mathbb{R}^{m \times d}$ mátrix oszlopaiként rendezzük, azaz $W_{:,j} = L(e_j)$, akkor a fenti összeg éppen a Wx mátrix–vektor szorzat. Megfordítva, tetszőleges $W \in \mathbb{R}^{m \times d}$ mátrix esetén $L(x) = Wx$ lineáris, mert a mátrixszorzás elosztó az összeadásra és felcserélhető a skalárral. A Wx i-edik komponense a W i-edik sorának és x-nek a skalárszorzata:

$$(Wx)_i = \sum_{j=1}^{d} W_{ij} x_j.$$

Definíció (rang). A $W \in \mathbb{R}^{m \times d}$ mátrix *rangja* a lineáris leképezés képterének dimenziója:

$$\operatorname{rank}(W) = \dim\{Wx : x \in \mathbb{R}^d\}.$$

Mivel a képtér legfeljebb d-dimenziós és $\mathbb{R}^m$ altere, $\operatorname{rank}(W) \le \min(m, d)$. Az oszloprang és a sorrang egyenlősége ismert lineáris algebrai tétel; a bizonyítást itt nem ismételjük meg, a pontos feltételekhez és bizonyításhoz [B01] lineáris algebrai fejezete (2. fejezet) szolgál háttérként.

Definíció (transzponált). Az $A \in \mathbb{R}^{m \times d}$ mátrix transzponáltja az $A^\top \in \mathbb{R}^{d \times m}$ mátrix, amelyre $(A^\top)_{ij} = A_{ji}$. Két mátrix szorzatának transzponáltjára

$$(AB)^\top = B^\top A^\top,$$

amelyet komponensenként ellenőrizhetünk:

$$((AB)^\top)_{ij} = (AB)_{ji} = \sum_{k} A_{jk} B_{ki} = \sum_{k} (B^\top)_{ik} (A^\top)_{kj} = (B^\top A^\top)_{ij}.$$

3.2.3 Affin réteg, bias és batchelt alak

Definíció (affin leképezés). Az $f : \mathbb{R}^d \to \mathbb{R}^m$ leképezés *affin*, ha $f(x) = Wx + b$ alakú, ahol $W \in \mathbb{R}^{m \times d}$ és $b \in \mathbb{R}^m$ a *bias* (eltolás) vektor. A neurális hálózatok irodalmában az affin réteget gyakran pontatlanul „lineáris rétegnek” nevezik; ebben a könyvben a két fogalmat mindig megkülönböztetjük (a fogalom hálózati irodalmi hátteréhez lásd [B02] 6. fejezetét).

A $b \neq 0$ affin leképezés nem lineáris: $f(0) = W0 + b = b \neq 0$, míg minden lineáris leképezés a nullvektort nullvektorba viszi, hiszen $L(0) = L(0 \cdot 0) = 0 \cdot L(0) = 0$. A bias tehát a lineáris rész eltolása; az $f(x) = 2x + 3$ egyenes grafikonja nem megy át az origón, ami vizuálisan is jelzi, hogy a leképezés nem lineáris.

A gyakorlatban egyszerre N mintát dolgozunk fel. A batchelt bemenet $X \in \mathbb{R}^{N \times d}$, ahol egy megfigyelés egy sor. A kimenet

$$Z = XW^\top + b, \qquad Z \in \mathbb{R}^{N \times m}.$$

Az $XW^\top$ szorzat (i, j) eleme

$$(XW^\top)_{ij} = \sum_{k=1}^{d} X_{ik} W_{jk},$$

tehát az i-edik minta $X_{i,:}$ sorvektorának és a j-edik kimeneti komponenshez tartozó $W_{j,:}$ súlysornak a skalárszorzata; ehhez adjuk a b_j bias-t. A $b \in \mathbb{R}^m$ vektor soriránybanbroadcastolódik (a NumPy a rövidebb b tömböt a hosszabbhoz igazítja: minden sorhoz ugyanaz a kimeneti csatornánkénti eltolás adódik). Az absztrakt oszlopvektoros $Wx+b$ és a batchelt $XW^\top+b$ ugyanazt a leképezést jelöli, csak az utóbbiban x sorokba rendezve szerepel; a transzponálás éppen a skalárszorzatok irányát állítja helyre.

A számítás méretlánca három lépésből áll: $X\,(N \times d)$ és $W^\top\,(d \times m)$ szorzata $XW^\top\,(N \times m)$, majd a hozzáadott $b\,(m)$ után $Z\,(N \times m)$. Az alakok ellenőrző táblázata:

Mennyiség	Alak
X	$N \times d$
W	$m \times d$
$W^\top$	$d \times m$
$XW^\top$	$N \times m$
b	m
$Z = XW^\top + b$	$N \times m$

A leggyakoribb alakhiba: az `X @ W` szorzat a $(N \times d) \cdot (m \times d)$ esetet kísérli meg, ami $d \neq m$ esetén `ValueError`-t ad; a helyes alak `X @ W.T`. A tipikus alakhibák összefoglalva:

Hibás kód	Következmény	Helyes alak
`X @ W`, $d \neq m$	`ValueError`: a belső dimenziók nem egyeznek	`X @ W.T`
`X @ W`, $d = m$	lefut, de a súly oszlopait használja sorok helyett	`X @ W.T`
`b` alakja `(N,)`	ha $N = m$, oszloponként adódik; különben `ValueError`	`b` alakja `(m,)`

Hibás kód	Következmény	Helyes alak
`b` alakja `(N,1)`	lefut, de mintánként egyetlen eltolást ad minden kimenetre	`b` alakja `(m,)`

3.2.4 Két lineáris réteg kompozíciója: állítás és bizonyítás

Állítás. Legyen $W_1 \in \mathbb{R}^{h \times d}$ és $W_2 \in \mathbb{R}^{m \times h}$, valamint $L_1(x) = W_1 x$ és $L_2(y) = W_2 y$. Ekkor a $L_2 \circ L_1 : \mathbb{R}^d \to \mathbb{R}^m$ kompozíció lineáris, és a reprezentáló mátrixa $W_2 W_1 \in \mathbb{R}^{m \times d}$.

Bizonyítás. Először megmutatjuk, hogy a kompozíció mátrixa a mátrixszorzat. Tetszőleges $x \in \mathbb{R}^d$ esetén az i-edik kimeneti komponens:

$$(L_2 \circ L_1)(x)_i = (W_2(W_1 x))_i = \sum_{j=1}^{h} (W_2)_{ij} (W_1 x)_j = \sum_{j=1}^{h} (W_2)_{ij} \sum_{k=1}^{d} (W_1)_{jk} x_k.$$

A két összegzés felcserélhető, mert véges összegekről van szó: az $\sum_{j=1}^{h} \sum_{k=1}^{d} a_{jk}$ és a $\sum_{k=1}^{d} \sum_{j=1}^{h} a_{jk}$ összeg ugyanaz, hiszen a véges összeg tagjai tetszőleges sorrendben csoportosíthatók (a valós összeadás kommutativitása és asszociativitása miatt):

$$= \sum_{k=1}^{d} \left(\sum_{j=1}^{h} (W_2)_{ij} (W_1)_{jk} \right) x_k = ((W_2 W_1) x)_i.$$

Ez minden i-re teljesül, tehát $(L_2 \circ L_1)(x) = (W_2 W_1)x$.

Most ellenőrizzük a linearitás két tulajdonságát. Legyen $u, v \in \mathbb{R}^d$ és $\alpha, \beta \in \mathbb{R}$. Az előbbi reprezentációt használva:

$$(L_2 \circ L_1)(\alpha u + \beta v) = (W_2 W_1)(\alpha u + \beta v).$$

A mátrix–vektor szorzás linearitása miatt

$$(W_2 W_1)(\alpha u + \beta v) = \alpha (W_2 W_1) u + \beta (W_2 W_1) v = \alpha (L_2 \circ L_1)(u) + \beta (L_2 \circ L_1)(v).$$

Ezzel az additivitást és a homogenitást is beláttuk, tehát a kompozíció lineáris. ■

A bizonyítás dimenzióellenőrzése: W_1 alakja $h \times d$, W_2 alakja $m \times h$, így $W_2 W_1$ alakja $m \times d$, ami éppen egy $\mathbb{R}^d \to \mathbb{R}^m$ leképezés mátrixának felel meg. A rangra a kompozíció nem növeli a dimenziót: $\operatorname{rank}(W_2 W_1) \le \min(\operatorname{rank}(W_1), \operatorname{rank}(W_2))$; ez standard tétel, bizonyítását lásd [B01] 2. fejezet.

3.2.5 Ellenpélda: nemnulla effektív bias esetén a kompozíció nem lineáris

Tekintsük a két affin réteget:

$$f_1(x) = W_1 x + b_1, \qquad f_2(y) = W_2 y + b_2,$$

ahol $b_1 \in \mathbb{R}^h$, $b_2 \in \mathbb{R}^m$. A kompozíció:

$$(f_2 \circ f_1)(x) = W_2(W_1 x + b_1) + b_2 = W_2 W_1 x + W_2 b_1 + b_2.$$

A kompozíció tehát ismét affin, effektív mátrixszal $W_2 W_1$ és effektív biasszal $W_2 b_1 + b_2$. A kompozíció pontosan akkor lineáris, ha $W_2 b_1 + b_2 = 0$; például $W_1 = W_2 = 1$, $b_1 = 1$, $b_2 = -1$ esetén $(f_2 \circ f_1)(x) = x$, ami lineáris, noha b_1 és b_2 külön-külön nem nulla. Ha viszont $W_2 b_1 + b_2 \neq 0$, akkor a kompozíció nem lineáris. Konkrét numerikus ellenpélda: legyen $d = m = 1$, $W_1 = W_2 = 1$, $b_1 = 1$, $b_2 = 2$. Ekkor

$$(f_2 \circ f_1)(x) = x + 3.$$

Az additivitás már $u = v = 1$ esetén sérül:

$$(f_2 \circ f_1)(1 + 1) = 2 + 3 = 5, \qquad (f_2 \circ f_1)(1) + (f_2 \circ f_1)(1) = 4 + 4 = 8,$$

és $5 \neq 8$. Ezzel megadtuk a kért ellenpéldát: két affin, nemnulla biasú réteg kompozíciója affin, de általában nem lineáris.

3.2.6 Kézi példa és NumPy-labor

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Legyen $d = 2$, $m = 3$, $N = 2$, tehát két minta és három kimenet. A súly, a bias és a bemenet:

$$X = \begin{pmatrix} 1 & 3 \\ -2 & 0{,}5 \end{pmatrix}, \qquad W = \begin{pmatrix} 1 & -2 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}, \qquad b = (0{,}5;\ -1;\ 0{,}2).$$

Az első mintára az $XW^\top$ sorai:

- $j = 1$: $1 \cdot 1 + 3 \cdot (-2) = 1 - 6 = -5$;
- $j = 2$: $1 \cdot 0 + 3 \cdot 1 = 3$;
- $j = 3$: $1 \cdot 2 + 3 \cdot 0 = 2$.

A bias hozzáadása után az első kimeneti sor $(-5 + 0{,}5,\ 3 - 1,\ 2 + 0{,}2) = (-4{,}5,\ 2,\ 2{,}2)$. A második minta:

- $j = 1$: $-2 \cdot 1 + 0{,}5 \cdot (-2) = -2 - 1 = -3$;
- $j = 2$: $-2 \cdot 0 + 0{,}5 \cdot 1 = 0{,}5$;
- $j = 3$: $-2 \cdot 2 + 0{,}5 \cdot 0 = -4$.

Bias után: $(-3+0{,}5,\ 0{,}5-1,\ -4+0{,}2) = (-2{,}5,\ -0{,}5,\ -3{,}8)$. Tehát

$$Z = \begin{pmatrix} -4{,}5 & 2 & 2{,}2 \\ -2{,}5 & -0{,}5 & -3{,}8 \end{pmatrix}.$$

Labor. Munkakönyvtár: `lab/n010/`. A következő szkript a fenti kézi számítást ellenőrzi NumPy-ban. A kódban a `@` a mátrixszorzás operátora, a `.T` a transzponált; a `b` hozzáadása soriányú broadcast (a b vektor minden sorhoz ugyanúgy adódik); az `allclose` elemenként, abszolút és relatív toleranciával hasonlít; a `float32` a könyv képinput-konvenciója, egyben a lebegőpontos kerekítés szemléltetése.

```
# futtatás: python n010_affine_lab.py
import numpy as np

X = np.array([[1.0, 3.0],
              [-2.0, 0.5]], dtype=np.float32)      # (2, 2)
W = np.array([[ 1.0, -2.0],
              [ 0.0,  1.0],
              [ 2.0,  0.0]], dtype=np.float32)     # (3, 2)
b = np.array([0.5, -1.0, 0.2], dtype=np.float32)   # (3,)

Z = X @ W.T + b

Z_expected = np.array([[-4.5, 2.0, 2.2],
                       [-2.5, -0.5, -3.8]], dtype=np.float32)

print("Z =")
print(Z)
assert Z.shape == (2, 3)
assert np.allclose(Z, Z_expected, atol=1e-6, rtol=1e-5)
print("OK: alak és numerikus egyezés a kézi számítással")
```

A `float32` típusban a 0,5 pontosan ábrázolható (kettő negatív hatványa), a 0,2 viszont nem, ezért a Z elemei a valós értékektől kis mértékben eltérhetnek; az egyenlőséget ezért `allclose` kezeli a 2.2. Lebegőpont és stabil elemi számítás alfejezetben bevezetett abszolút és relatív toleranciával.

Futtatási környezet: Python 3.12.3, NumPy 2.5.4, CPU, egy szál (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`); a feladat determinisztikus, ezért seedre nincs szükség, és PyTorchöt sem használ. A futtatott szkript tényleges kimenete:

```
Z =
[[-4.5  2.   2.2]
 [-2.5 -0.5 -3.8]]
OK: alak és numerikus egyezés a kézi számítással
```

A kimenet ellenőrző kulcssorai: a Z alakja $(2, 3)$; a Z mátrix hat száma; az `OK` üzenet. A fenti kimenet a megadott környezetben futtatott szkript tényleges kimenete; a saját futásnak az `assert`-ek toleranciáján belül ezzel kell egyeznie.

3.2.7 Feladatok

F1 (fogalmi, nehézség 1). Adja meg az $L : \mathbb{R}^2 \to \mathbb{R}^2$, $L(x) = (2x_1 - x_2,\ x_1 + x_2)$ leképezés standard bázisbeli mátrixát. Számítsa ki $L(x)$-et $x = (1, 2)$ esetén, és ellenőrizze a homogenitást $\alpha = -3$ mellett; a teljes igazoláshoz lássa be általános α-ra is. Miért nem lineáris az $f(x) = L(x) + (1, 0)$ leképezés? Beadandó: a 2×2 mátrix, két kiszámított vektor, az általános α-ra vonatkozó homogenitásszámítás és egy szöveges indoklás.

F2 (levezetési, nehézség 2). Legyen

$$W_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{3\times 2}, \qquad W_2 = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & 2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix} \in \mathbb{R}^{4\times 3}.$$

Bizonyítsa a fenti állítás alkalmazásával, hogy $L_2 \circ L_1$ lineáris, adja meg a $W_2W_1 \in \mathbb{R}^{4\times 2}$ mátrixot, és számítsa ki a $(3,2)$ elemét. Ellenőrizze, hogy $(L_2 \circ L_1)(0,0) = (0,0,0,0)$. Beadandó: a bizonyítás lépései, a teljes 4×2 mátrix és a $(3,2)$ elem levezetése.

F3 (implementációs, nehézség 2). Írjon `affine_forward(X, W, b)` függvényt, amely ellenőrzi, hogy X alakja (N,d), W alakja (m,d), b alakja $(m,)$, majd $XW^\top + b$-t számol. Hibás alak esetén `ValueError` kivételt adjon érthető, ékezetes üzenettel (az `assert`-et kerülje, mert a `python -O` kapcsoló kikapcsolja). Tesztelje a labor X, W, b értékeivel, és ellenőrizze `np.allclose` segítségével `atol=1e-6, rtol=1e-5` mellett a fenti Z_{expected} mátrixszal. Beadandó: teljes függvény, teszt és a hibaüzenet.

F4 (hibakeresési, nehézség 2). Tekintse a következő függvényt:

```
def forward_wrong(X, W, b):
    return X @ W + b
```

A hívás `X.shape=(2,2)`, `W.shape=(3,2)`, `b.shape=(3,)`. Mi a hiba oka, mi a közvetlen következménye, és hogyan javítja? Ellenőrizze a javítást a labor számadataival. Beadandó: a hiba oka, a hibás kimenet vagy kivétel leírása, a javított sor és a visszaellenőrzés.

F5 (kísérlettervezési/transzfer, nehézség 3). A GlyphLab `data/images.npy` tömbje $(360, 1, 24, 24)$ alakú. Tervezzen baseline kísérletet, amely a képeket $24 \cdot 24 = 576$ dimenziós vektorrá lapítja, és egy háromkimenetű affin réteget alkalmaz. Adja meg a paraméterszámot, egy kézi kontrollt a helyes alak ellenőrzésére, a betanítás előtti baseline-t, a metrikát és az érvényességi korlátot. Ez nyitott feladat: a protokoll minden kötelező elemét (hipotézis, változó, kontroll, metrika, egység, döntési szabály, erőforrás) tartalmazó, legfeljebb féloldalas terv szükséges. Beadandó: a legfeljebb féloldalas terv.

3.2.8 Megoldások és indoklás

F1 megoldás. A standard bázisvektorok képei: $L(e_1) = L(1,0) = (2,1)$, $L(e_2) = L(0,1) = (-1,1)$. Ezért

$$W = \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix}.$$

$x = (1,2)$ esetén $L(x) = (2 \cdot 1 - 2,\ 1 + 2) = (0,3)$. A homogenitás ellenőrzése $\alpha = -3$ mellett: $L(-3x) = L(-3,-6) = (-6+6,\ -3-6) = (0,-9)$, és $-3L(x) = -3(0,3) = (0,-9)$, tehát egyenlők. Általános α-ra ugyanez képletben: $L(\alpha x) = (2\alpha x_1 - \alpha x_2,\ \alpha x_1 + \alpha x_2) = \alpha(2x_1 - x_2,\ x_1 + x_2) = \alpha L(x)$, ami minden $x \in \mathbb{R}^2$ és $\alpha \in \mathbb{R}$ esetén teljesül, tehát ez már bizonyítás, nem csupán egy számpélda. Az $f(x) = L(x) + (1,0)$ leképezés nem lineáris, mert $f(0,0) = (1,0) \neq (0,0)$, márpedig lineáris leképezés a nullvektort nullvektorba viszi.

F2 megoldás. Mivel L_1, L_2 lineárisak, a bizonyított állítás szerint $L_2 \circ L_1$ lineáris, mátrixa W_2W_1. Számoljuk ki W_2W_1 sorait úgy, hogy W_2 egy sorát skalárisan szorozzuk W_1 oszlopaival. W_1 oszlopai $(1,0,1)$ és $(0,1,1)$.

- Első sor $(1,-1,0)$: $(1\cdot 1+(-1)\cdot 0+0\cdot 1,\ 1\cdot 0+(-1)\cdot 1+0\cdot 1)=(1,-1)$.
- Második sor $(0,1,2)$: $(0+0+2,\ 0+1+2)=(2,3)$.
- Harmadik sor $(1,0,-1)$: $(1+0-1,\ 0+0-1)=(0,-1)$.
- Negyedik sor $(2,1,0)$: $(2+0+0,\ 0+1+0)=(2,1)$.

Tehát

$$W_2W_1=\begin{pmatrix}1&-1\\2&3\\0&-1\\2&1\end{pmatrix},$$

és a $(3,2)$ elem a harmadik sor második oszlopában -1. A nullvektor képe $W_2W_1(0,0)=(0,0,0,0)$, ahogy minden lineáris leképezésre elvárható. Alakellenőrzés: 4×3 szorozva 3×2 mátrixszal 4×2-t ad, ezért a négy sor két-két eleme helyes.

F3 megoldás.

```
import numpy as np

def affine_forward(X, W, b):
    if X.ndim != 2:
        raise ValueError("X-nek (N, d) alakúnak kell lennie")
    if W.ndim != 2:
        raise ValueError("W-nek (m, d) alakúnak kell lennie")
    if b.ndim != 1:
        raise ValueError("b-nek (m,) alakúnak kell lennie")
    N, d = X.shape
    m, d_w = W.shape
    if d != d_w:
        raise ValueError(f"bemeneti dimenzió eltérés: X {d}, W {d_w}")
    if b.shape[0] != m:
        raise ValueError(f"bias méret {b.shape[0]}, de {m} kell")
    return X @ W.T + b

X = np.array([[1.0, 3.0], [-2.0, 0.5]], dtype=np.float32)
W = np.array([[1.0, -2.0], [0.0, 1.0], [2.0, 0.0]], dtype=np.float32)
b = np.array([0.5, -1.0, 0.2], dtype=np.float32)
Z = affine_forward(X, W, b)
Z_expected = np.array([[-4.5, 2.0, 2.2], [-2.5, -0.5, -3.8]], dtype=np.float32)
assert Z.shape == (2, 3)
assert np.allclose(Z, Z_expected, atol=1e-6, rtol=1e-5)
```

A hibaüzenet például `X.shape=(2,3)`, `W.shape=(3,2)` esetén: `bemeneti dimenzió eltérés: X 3, W 2`.

F4 megoldás. A hiba oka: a batchelt konvencióban $Z=XW^\top+b$ szerepel, a kód azonban XW-t számol, tehát elmarad a transzponálás. Következmény: a hívott alakokkal X $(2,2)$ és W $(3,2)$ belső dimenziói 2 és 3, ezért a szorzás `ValueError` kivételt ad; ha véletlenül $d=m$ lenne, a kód lefutna, de a súly oszlopait használná sorok helyett, ami általában más értéket ad. Javítás: `return X @ W.T + b`. Visszaellenőrzés: a fenti Z kizárólag a javított változat kimenete, nem a hibásé; a javított hívás eredménye `allclose` szerint egyezik a kézi számítással, a hibás változat pedig ugyanezekkel az alakokkal `ValueError`-t ad, tehát számszerű kimenetet nem is produkál.

F5 megoldásminta. A lapítás után $X_{\text{flat}}\in\mathbb{R}^{360\times 576}$, a réteg $W\in\mathbb{R}^{3\times 576}$, $b\in\mathbb{R}^3$, a kimenet $Z=X_{\text{flat}}W^\top+b\in\mathbb{R}^{360\times 3}$. Paraméterszám: $3\cdot 576+3=1731$. Kézi kontroll: válasszunk egy rögzített képet, $W=0$, $b=0$ mellett $Z=0$; majd egyetlen W elem 1-re állításával a megfelelő pixel értéke jelenik meg a kimenetben. Ez a linearitás közvetlen ellenőrzése. Két baseline-t

rögzítünk: (a) többségi baseline, amely mindig a tanítóhalmaz legnagyobb gyakoriságú osztályát jósolja; (b) esély (véletlen) baseline, amely egyenletesen választ osztályt, így kiegyensúlyozott osztályoknál a várható pontossága 1/3. Metrika: pontosság (accuracy). A modellt csak a tanító (`train`) részen tanítjuk, a validációs (`val`) részen választunk, a teszt (`test`) részt egyszer, a végén értékeljük; a megfigyelési egység a `scene_id`, azaz a GlyphLab egy jelenetének azonosítója (egy jelenethez két nézet tartozik, és mindkettő ugyanabba a részbe kerül). Érvényességi korlát: az affin modell csak globális intenzitásmintázatot lát, eltolási invarianciája nincs, ezért ez kizárólag alsó baseline, nem várható tőle a konvolúciós modellek teljesítménye. A terv nem állít mért eredményt; a futtatás előtt a protokollt rögzíteni kell.

3.2.9 Gyakori tévedés, önellenőrzés és visszatekintés

Gyakori tévedés. A PyTorch `nn.Linear(in_features, out_features)` súlymátrixa `(out_features, in_features)` alakú, tehát $W \in \mathbb{R}^{m\times d}$. Ha egy lineáris algebrai könyv a sorvektoros $x^\top$ konvenciót követi, ott a mátrix $W \in \mathbb{R}^{d\times m}$ alakban jelenik meg; a mátrixreprezentáció és a bázisváltás részleteihez lásd [B01] 2. fejezetét. A két mátrix egymás transzponáltja; ha összekeverjük őket, az előre irányú (forward) számítás rossz lesz. Konkrét ellenpélda: legyen a PyTorch-konvenciójú súly

$$W_{\text{pt}} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix},$$

tehát $d = m = 2$. Az $x = (1, 1)$ bemenetre a helyes kimenet $W_{\text{pt}}x = (3, 7)$. Ha valaki tévesen a sorvektoros $x^\top W_{\text{pt}}$ kifejezést számolja, az $(1, 1)W_{\text{pt}} = (4, 6)$ eredményt ad; ez nem a kívánt leképezés. Batchelt alakban a hiba jellemzően az `X @ W` használata `X @ W.T` helyett, ami $d \neq m$ esetén alakhibát, $d = m$ esetén rossz értéket okoz.

Önellenőrzés. (1) Miért $Z = XW^\top + b$ és nem $XW + b$? (2) Mondja ki, miért viszi a lineáris leképezés a nullvektort nullvektorba. (3) Két affin réteg kompozíciójának effektív bias-vektora miért $W_2b_1 + b_2$? (4) Ellenőrizze a laborban, hogy a `b` a kimenet melyik tengelyéhez adódik. A válaszok a fenti levezetésekben megtalálhatók; ha bármelyik kérdésre nem tud indokolni, térjen vissza a megfelelő szakaszhoz.

Visszatekintés. A 3.1. Vektor, norma és skalárszorzat skalárszorzat-fogalma itt a W sorai és az x közötti szorzatösszegben jelenik meg. A 2.2. Lebegőpont és stabil elemi számítás `allclose`- és `float32`-tolerancia-gondolata indokolja, miért nem `==`-tel ellenőrizzük a 0,2-t tartalmazó kézi eredményt.

3.2.10 Rövid összefoglaló

A lineáris leképezés két feltételt teljesít: additivitás és homogenitás. Rögzített bázisban minden lineáris leképezés egy $W \in \mathbb{R}^{m\times d}$ mátrixszal reprezentálható, amelynek oszlopai a bázisvektorok képei; a rang a képtere dimenziója. Az affin leképezés $Wx + b$; ha $b \neq 0$, nem lineáris, mert a nullvektor képe nem nulla. Batchelt bemenetre a kimenet $Z = XW^\top + b$, ahol a bias soriránybanbroadcastolódik. Két lineáris réteg kompozíciója lineáris, mátrixa W_2W_1; két affin réteg kompozíciója affin, effektív biasszal $W_2b_1 + b_2$, és pontosan akkor lineáris, ha ez az effektív bias nulla. A NumPy-labor két mintára és három kimenetre ellenőrizte a kézi számítást.

3.2.11 Továbblépés

A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság a vetítést és a legkisebb négyzeteket tárgyalja: az affin réteg ott jelenik meg mint a legjobban illeszkedő affin modell keresésének alapművelete, a kondicionáltság pedig a W mátrix numerikus tulajdonságait kapcsolja a gradiensalapú tanításhoz.

3.2.12 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

3.3 Vetítés, legkisebb négyzetek és kondicionáltság

3.3.1 Induló kérdés és előismeret

A 3.2. Mátrixszorzás és affín réteg affin rétege $Z = XW^\top + b$ alakú batchelt leképezést definiált: $X \in \mathbb{R}^{N\times d}$, $W \in \mathbb{R}^{m\times d}$, és a bias sorirányban broadcastolódik. Ha ezt a réteget egy **regressziós baseline-ként** használjuk, akkor adott bemenetekhez egy valós célértéket szeretnénk közelíteni. Tekintsük az $A \in \mathbb{R}^{n\times d}$ tervező- vagy adatmátrixot, amelynek egy sora egy megfigyelés d jellemzőjét tartalmazza, és a $b \in \mathbb{R}^n$ célvektort (figyelem: ebben az egységben a b a célvektor, nem a 3.2. biasvektor; az adatmátrix jele A, mérete $n \times d$, míg a 3.2-ben X és N szerepelt). Az $Ax = b$ lineáris egyenletrendszer általában **túlhatározott**, ha $n > d$: nincs olyan x, amely minden egyenletet egyszerre kielégítene. A legkisebb négyzetes módszer azt az x-et keresi, amelyre a hiba négyzetösszege minimális.

A legkisebb négyzetes módszer lineáris algebrai hátterét [B01] 9.2. szakasza, gépitanulás-alkalmazását [B02] 5.1.4. szakasza tárgyalja.

Induló becslés: ha A két oszlopa majdnem párhuzamos, akkor a két megfelelő együtthatót nehéz külön-külön azonosítani, még akkor is, ha az előrejelzés összességében jól illeszkedik. Miért? Ezt a vetítés és a kondicionáltság fogalma teszi pontossá.

A 2.3. Izolált környezet, notebook és verzió környezeti konvencióját itt is használjuk: a labor kódja CPU-n fut, `float64` adattípusú, nem használ véletlent, ezért a laborhoz seedre nincs szükség; az F5 kísérlet viszont generált véletlen adatot használ, ott a seed kötelező. Az ellenőrzés toleranciás `allclose` összehasonlítás.

3.3.2 A reziduum és a legkisebb négyzetes vetítés

Definíció 1 (reziduum). Legyen $A \in \mathbb{R}^{n\times d}$, $b \in \mathbb{R}^n$, $x \in \mathbb{R}^d$. Az x együtthatóvektorhoz tartozó **reziduumvektor**

$$r(x) = b - Ax \in \mathbb{R}^n,$$

a **négyzetes hiba** pedig

$$F(x) = \|r(x)\|_2^2 = \sum_{i=1}^{n} \left(b_i - \sum_{j=1}^{d} A_{ij} x_j \right)^2 .$$

A b vektor $U = \mathrm{range}(A)$ altérre vett ortogonális vetülete az a $p \in U$, amelyre $\|b-p\|_2$ minimális. A vetület létezik és egyértelmű. A $\|b-p\|_2$ folytonos U-n, és a $B = \{p \in U : \|p\|_2 \le 2\|b\|_2\}$ zárt, korlátos gömbön kívül $\|b-p\|_2 \ge \|p\|_2 - \|b\|_2 > \|b\|_2 = \|b-0\|_2$, tehát a minimum nem eshet a gömbön kívülre; B korlátos és zárt, ezért a folytonos függvény felveszi rajta a minimumát. Az egyértelműséghez tegyük fel, hogy $p_1 \neq p_2$ is minimalizál; ekkor a $\|b-p\|_2^2$ szigorú konvexitása miatt a $\frac{p_1+p_2}{2}$ pontban szigorúan kisebb értéket kapnánk, ami ellentmondás. Mivel minden $p = Ax$ valamilyen x-re, a legkisebb négyzetes megoldás olyan $\hat{x}$, amelynek képe a vetület: $A\hat{x} = p$. Teljes oszloprang esetén ez az $\hat{x}$ egyértelmű (ezt a 3.3.3. tétel mondja ki); ranghiányos esetben viszont végtelen sok ilyen x van, miközben a predikció p továbbra is egyértelmű.

2. lemma (ortogonalitási kritérium). Az $x^* \in \mathbb{R}^d$ pontosan akkor minimalizálja F-et, ha $r^* = b - Ax^*$ merőleges U minden elemére, azaz $A^\top r^* = 0$.

Bizonyítás. Legyen $p = Ax$. Ha p minimális távolságú pont U-ban, akkor bármely $q = Az \in U$ és $t \in \mathbb{R}$ esetén a

$$\varphi(t) = \|b - (p + tq)\|_2^2 = \|r\|_2^2 - 2t\, r^\top q + t^2 \|q\|_2^2$$

függvénynek $t = 0$-ban minimuma van, ahol $r = b-p$. A $t = 0$-beli derivált $-2r^\top q$, tehát $r^\top q = 0$ minden $q \in U$-ra. Ez éppen azt jelenti, hogy r merőleges U minden elemére, speciálisan A minden oszlopára, ami vektoralakban $A^\top r = 0$. Megfordítva, ha $r \perp U$, akkor bármely $w \in U$ mellett $r \perp w$; minden $q \in U$ felírható $q = p + w$ alakban valamilyen $w \in U$-val, így $b - q = r - w$, és a Pitagorasz-tétel szerint

$$\|b - (p+w)\|_2^2 = \|r - w\|_2^2 = \|r\|_2^2 + \|w\|_2^2 \ge \|r\|_2^2,$$

tehát p valóban minimális távolságú pont.

A lemma két iránya együtt adja a normálegyenletet: mivel A oszlopai generálják U-t, a merőlegesség ekvivalens azzal, hogy

$$A^\top (b - Ax) = 0 \quad \Longleftrightarrow \quad A^\top A x = A^\top b.$$

3.3.3 A normálegyenlet és az egyértelműség feltétele

A fenti átalakítás már megadja a bizonyítási feladat első felét. A megoldhatóság és az egyértelműség kérdéséhez a $A^\top A$ Gram-mátrix invertálhatóságát kell ellenőrizni.

Definíció (pozitív (szemi)definit és Gram-mátrix). A szimmetrikus $M \in \mathbb{R}^{d\times d}$ mátrix *pozitív szemidefinit*, ha $z^\top M z \ge 0$ minden $z \in \mathbb{R}^d$-re, és *pozitív definit*, ha $z \neq 0$ esetén $z^\top M z > 0$. Egy $A \in \mathbb{R}^{n\times d}$ mátrix esetén az $A^\top A$ szorzatot az A oszlopai **Gram-mátrixának** nevezzük.

3. tétel (normálegyenlet, teljes oszloprang esetén). Legyen $A \in \mathbb{R}^{n\times d}$ **teljes oszloprangú**, azaz $\mathrm{rank}(A) = d$. Ekkor:

1. $A^\top A \in \mathbb{R}^{d\times d}$ szimmetrikus, pozitív definit és invertálható;
2. az F minimalizálója pontosan egy létezik, és

$$\hat{x} = (A^\top A)^{-1} A^\top b.$$

Bizonyítás. Elsőként igazoljuk (1)-et. A Gram-mátrix nyilván szimmetrikus: $(A^\top A)^\top = A^\top A$. Legyen $z \in \mathbb{R}^d$. Ekkor

$$z^\top A^\top A z = (Az)^\top (Az) = \|Az\|_2^2 \geq 0,$$

tehát $A^\top A$ pozitív szemidefinit. Ha $z^\top A^\top A z = 0$, akkor $\|Az\|_2^2 = 0$, így $Az = 0$. A teljes oszloprang feltétel szerint A magtere csak a nullvektort tartalmazza, ezért $z = 0$. Ez azt jelenti, hogy $z^\top A^\top A z > 0$ minden $z \neq 0$ esetén, vagyis a mátrix pozitív definit. Pozitív definit mátrix invertálható.

A (2) állításhoz a 2. lemma szerint x^* pontosan akkor minimalizáló, ha kielégíti az $A^\top A x = A^\top b$ normálegyenletet. Az (1) pont miatt az együtthatómátrix invertálható, tehát a normálegyenletnek pontosan egy megoldása van, mégpedig a fenti zárt alakú $\hat{x}$.

Fontos, hogy a tétel **létezési és egyértelműségi állítás**, nem numerikus algoritmusajánlás. Erre a fejezet Tipikus tévedés és ellenpélda szakaszában (3.3.9.) visszatérünk, ahol az explicit inverz numerikus hátrányát mutatjuk meg.

Ha $\operatorname{rank}(A) = r < d$, azaz **ranghiányos** az adatmátrix, akkor $A^\top A$ szinguláris. A normálegyenlet továbbra is konzisztens (van megoldása), de végtelen sok megoldása van; mind megvalósítja ugyanazt a $p = A\hat{x}$ vetületet, az együtthatók azonban nem azonosíthatók egyértelműen. Ilyenkor a könyvtárak tipikusan azt a megoldást adják vissza, amelynek saját normája, $\|x\|_2$, a legkisebb a minimalizálók között; ezt **minimum-norma megoldásnak** nevezzük.

Egy valós regressziós baseline-ban a bias-t úgy kezeljük, hogy a tervezőmátrixot egy csupa 1-es oszloppal egészítjük ki. Ekkor az Ax modell affin a további jellemzőkben, összhangban a 3.2. Mátrixszorzás és affín réteg alfejezetben bevezetett affin réteg fogalmával.

3.3.4 Kondicionáltság és stabilitás

Egy mátrix numerikus viselkedését a kondíciószám jellemzi. Teljes oszloprangú A esetén az $A^\top A$ pozitív definit mátrix sajátértékei pozitívak; jelölje $\lambda_{\max}$ a legnagyobb, $\lambda_{\min}$ a legkisebb sajátértéket. A $\lambda_i = \sigma_i^2$ összefüggés szerint, ahol $\sigma_1 \geq \cdots \geq \sigma_d > 0$ az A szinguláris értékei (az $A^\top A$ sajátértékeinek négyzetgyökei), az $A^\top A$ mátrix **2-es kondíciószáma**

$$\kappa_2(A^\top A) = \frac{\lambda_{\max}(A^\top A)}{\lambda_{\min}(A^\top A)} = \frac{\sigma_1^2}{\sigma_d^2} \geq 1,$$

az A **2-es kondíciószáma** pedig ezzel ekvivalens módon

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_d} = \sqrt{\frac{\lambda_{\max}(A^\top A)}{\lambda_{\min}(A^\top A)}}.$$

A szinguláris értékek és a $\lambda_i = \sigma_i^2$ azonosság pontos levezetését a 3.4. Sajátérték, SVD és spektrális határ egységben adjuk meg; itt elegendő az az intuíció, hogy $\kappa_2(A)$ az operátor által okozott maximális és minimális nyújtás aránya. Közel kollineáris oszlopoknál van egy irány, amelyben a leképezés csak nagyon gyengén nyújt (kicsi σ_d), ezért a kondíciószám nagy.

A normálegyenletes megoldás szempontjából döntő összefüggés:

$$\kappa_2(A^\top A) = \kappa_2(A)^2.$$

Ez a $\lambda_i = \sigma_i^2$ összefüggésből azonnal következik: $\kappa_2(A^\top A) = \sigma_1^2/\sigma_d^2 = (\sigma_1/\sigma_d)^2 = \kappa_2(A)^2$. Ha $\kappa_2(A)$ nagy, akkor $A^\top A$ kondíciószáma a négyzetével nagyobb, ezért az explicit inverzzel történő megoldás sokkal érzékenyebb a kerekítési hibákra, mint a közvetlen, QR- vagy SVD-alapú eljárás (a $A = QR$ QR-felbontásban Q ortogonális, R felső háromszög; az SVD, azaz szingulárisérték-felbontás teljes levezetését a 3.4. Sajátérték, SVD és spektrális határ egység adja).

3.3.5 Kézi példa: pontos illeszkedés és ranghiány

Legyen

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

Ekkor $n = 3$, $d = 2$, és A oszlopai függetlenek. A normálegyenlet mátrixa és jobb oldala

$$A^\top A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad A^\top b = \begin{pmatrix} 4 \\ 5 \end{pmatrix}.$$

A 2×2-es mátrix determinánsa 3, inverze $\frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$, így

$$\hat{x} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Ezzel $A\hat{x} = (1, 2, 3)^\top = b$, tehát a reziduum nulla: a célvektor a képtérben van, és a vetítés önmaga.

Ranghiányos ellenpéldaként vegyük az

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

mátrixot. Itt a második oszlop az első kétszerese, tehát $\operatorname{rank}(A) = 1$. A normálegyenlet

$$\begin{pmatrix} 5 & 10 \\ 10 & 20 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 10 \end{pmatrix},$$

ami az $x_1 + 2x_2 = 1$ egyetlen független feltételre redukálódik. Az összes megoldás $(1 - 2t, t)$ alakú. A legkisebb normájú megoldás a feltétel mellett a $\|(x_1, x_2)\|_2^2$ minimuma. Helyettesítsük be $x = (1 - 2t, t)$-t: $\|x\|_2^2 = (1 - 2t)^2 + t^2 = 1 - 4t + 5t^2$. Ez t konvex másodfokú függvénye, minimuma a $g'(t) = -4 + 10t = 0$ egyenletből $t = 2/5$, ahol $g(t) = 1 - 8/5 + 4/5 = 1/5$. Így $\hat{x} = (1 - 2 \cdot 2/5;\, 2/5)^\top = (1/5;\, 2/5)^\top$. Ugyanezt adja a Lagrange-multiplikátoros levezetés is: a kényszerfeltételes szélsőérték-keresésben az $x_1 + 2x_2 = 1$ feltételt egy λ szorzóval a célfüggvényhez adjuk, majd a kapott függvény parciális deriváltjait nullázzuk. Ellenőrzés: $A\hat{x} = (1/5 + 4/5;\, 2/5 + 8/5)^\top = (1; 2)^\top = b$, tehát a reziduum itt is nulla. A két példa különbsége: az elsőben az együtthatók egyértelműek, a másodikban a predikció egyértelmű, de az együtthatók nem.

3.3.6 Labor: közel kollineáris oszlopok és `lstsq`

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A cél annak megmutatása, hogy a normálegyenlet elméleti helyességéből nem következik az explicit inverz numerikus jósága. Válasszunk 3 megfigyelést és 2 jellemzőt:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1+\varepsilon \\ 1 & 1+2\varepsilon \end{pmatrix}, \qquad \varepsilon = 10^{-5}.$$

Írjuk fel a pontos legkisebb négyzetes referenciafeladatot: legyen $x_{\text{ref}} = (1,2)^\top$, és a célvektor

$$b = Ax_{\text{ref}} + \delta, \qquad \delta = (1,-2,1)^\top,$$

ahol δ merőleges A mindkét oszlopára. Ez az $A^\top\delta = 0$ skalárszorzatokkal ellenőrizhető: a két oszlop $a_1 = (1,1,1)^\top$ és $a_2 = (1,1+\varepsilon,1+2\varepsilon)^\top$, így $a_1^\top\delta = 1-2+1 = 0$ és $a_2^\top\delta = 1-2(1+\varepsilon)+(1+2\varepsilon) = 0$. Ezért a legkisebb négyzetes megoldás pontosan x_{ref} marad. A megfelelő $b = Ax_{\text{ref}} + \delta = (4, 1+2\varepsilon, 4+4\varepsilon)^\top$; a pontos reziduum δ, négyzetes hibája $\|\delta\|_2^2 = 6$.

A labor munkakönyvtára `lab/n011_lstsq/`, futtatási parancsa:

```
python lab/n011_lstsq/n011_lstsq.py
```

A futtatási környezet: Python 3.12.3, NumPy 2.5.4, CPU, egy szál (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`); a feladat determinisztikus, ezért seedre nincs szükség, és PyTorchöt sem használ.

```python
# lab/n011_lstsq/n011_lstsq.py
# Input: nincs külső adat; CPU, float64, seed nem szükséges.
import numpy as np

EPS = 1e-5
A = np.array(
    [[1.0, 1.0],
     [1.0, 1.0 + EPS],
     [1.0, 1.0 + 2.0 * EPS]],
    dtype=np.float64,
)

x_ref = np.array([1.0, 2.0], dtype=np.float64)
delta = np.array([1.0, -2.0, 1.0], dtype=np.float64)
assert np.allclose(A.T @ delta, np.zeros(2), atol=1e-14)
b = A @ x_ref + delta

# 1) Expliciten invertált normálegyenlet.
x_explicit = np.linalg.inv(A.T @ A) @ A.T @ b
err_explicit = np.linalg.norm(x_explicit - x_ref)

# 2) Stabil, SVD-alapú megoldó; rcond=None: automatikus SVD-küszöb.
sv = np.linalg.svd(A, compute_uv=False)
rcond_eff = np.finfo(A.dtype).eps * max(A.shape)
print(f"SVD-küszöb  = {rcond_eff * sv[0]} (rcond_eff = {rcond_eff})")
x_lstsq, residuals, rank_lstsq, _ = np.linalg.lstsq(A, b, rcond=None)
err_lstsq = np.linalg.norm(x_lstsq - x_ref)

print("x_ref      =", x_ref)
```

```
print("explicit   =", x_explicit, "hiba =", err_explicit)
print("lstsq      =", x_lstsq, "hiba =", err_lstsq)
print("rank       =", rank_lstsq, "reziduum-norma^2 =", residuals[0])
print("cond(A.T A) =", np.linalg.cond(A.T @ A))
print("cond(A)     =", np.linalg.cond(A))

# Elméleti kontrollok, amelyek a stabil megoldóra vonatkoznak.
assert rank_lstsq == 2
assert np.allclose(x_lstsq, x_ref, rtol=0.0, atol=2e-5)
assert np.allclose(residuals[0], 6.0, rtol=1e-10, atol=1e-10)
print("Ellenőrzés: rank, x_lstsq és reziduum rendben.")
```

A futtatott szkript tényleges kimenete a megadott környezetben:

```
x_ref      = [1. 2.]
explicit   = [1.00000646 1.99999609] hiba = 7.550081794037606e-06
lstsq      = [1.00000066 1.99999934] hiba = 9.265380619425146e-07
SVD-küszöb  = 1.6316961050863485e-15 (rcond_eff = 6.661338147750939e-16)
rank       = 2 reziduum-norma^2 = 6.000000000000001
cond(A.T A) = 60001198387.472664
cond(A)     = 244951.42378662003
Ellenőrzés: rank, x_lstsq és reziduum rendben.
```

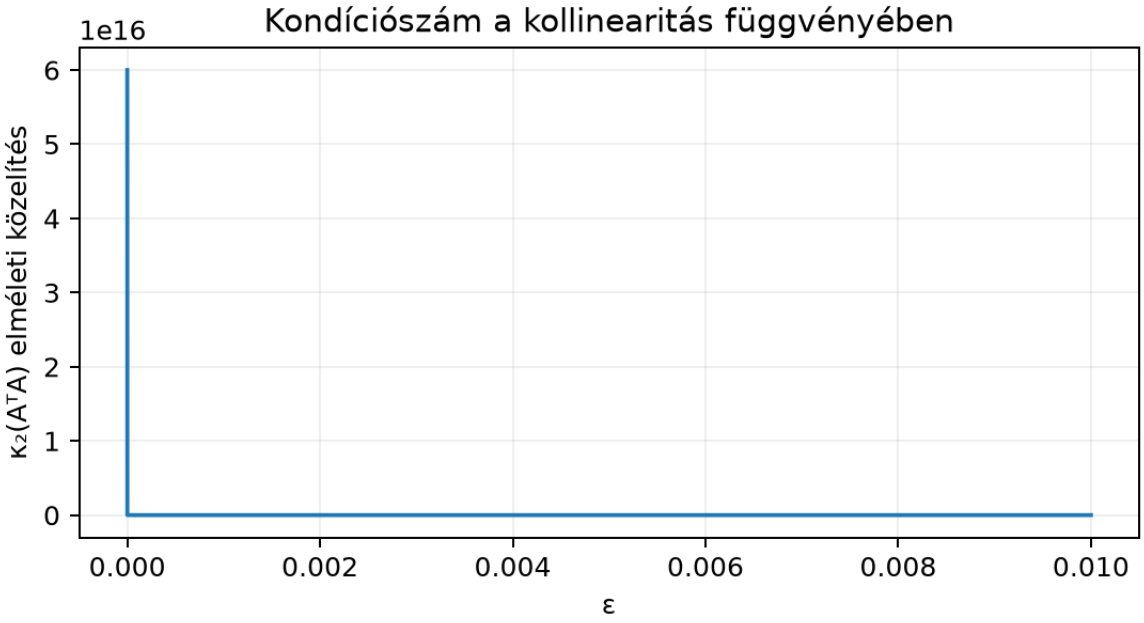
A levezetésből adódó kontrollérték $x_{\text{ref}} = (1, 2)^\top$ és a reziduum-norma négyzete 6; a futtatás ezeket a toleranción belül visszaadta. A kimenet további kulcssorai: a numerikus rang 2; a `lstsq` megoldás hibája $9.27 \cdot 10^{-7}$, ami a $2 \cdot 10^{-5}$ abszolút paramétertolerancia alatt van; az explicit normálegyenlet hibája ebben a futásban $7.55 \cdot 10^{-6}$, tehát nagyobb, de mértéke platformfüggő. Az illesztett predikció és a reziduum stabilabb ellenőrzési tárgy, mint a rosszul kondicionált együtthatóvektor.

A kondíciószám elméleti közelítése kis ε-ra $\kappa_2(A^\top A) \approx 6/\varepsilon^2$, azaz $\varepsilon = 10^{-5}$ esetén mintegy $6 \cdot 10^{10}$; a mért érték $6.0001 \cdot 10^{10}$, ami már jelentősen felerősítheti a kerekítési hibát. A közelítés levezetése:

$$A^\top A = \begin{pmatrix} 3 & 3+3\varepsilon \\ 3+3\varepsilon & 3+6\varepsilon+5\varepsilon^2 \end{pmatrix},$$

amelynek determinánsa $3(3+6\varepsilon+5\varepsilon^2) - (3+3\varepsilon)^2 = 6\varepsilon^2$, nyoma pedig $6+6\varepsilon+5\varepsilon^2$. Kis ε-ra a nagyobb sajátérték $\lambda_{\max} \approx 6$, a kisebb $\lambda_{\min} = \det(A^\top A)/\lambda_{\max} \approx \varepsilon^2$, ezért $\kappa_2(A^\top A) = \lambda_{\max}/\lambda_{\min} \approx 6/\varepsilon^2$.

A $2 \cdot 10^{-5}$ abszolút paramétertolerancia gyakorlati küszöb, nem elméleti hibakorlát: a mért `lstsq`-hiba $9{,}27 \cdot 10^{-7}$, tehát a tolerancia mintegy hússzoros tartalékot ad. Az elsőrendű becslés $\kappa_2(A)\varepsilon_{\text{gép}} \approx 2{,}45 \cdot 10^5 \cdot 2{,}22 \cdot 10^{-16} \approx 5{,}4 \cdot 10^{-11}$; a mért hiba ennél nagyobb, mert a δ és az A float64-ben ábrázolt oszlopai között a kerekítés miatt a skalárszorzat már nem pontosan nulla, és a közel szinguláris irány ezt $1/\sigma_d^2$ nagyságrenddel felerősíti. Ezért a feladat és a megoldás ellenőrzése egységesen $2 \cdot 10^{-5}$ abszolút toleranciát használ x-re; a 10^{-8} nagyságú érték ezen a kondicionáltságon nem lenne reális elvárás.



3.4. ábra: Kondíciószám a kollinearitás függvényében

A $6/\varepsilon^2$ közelítésből számolt szemléltetés, nem mért futás.

A chart pontjai a $\kappa_2(A^\top A) \approx 6/\varepsilon^2$ közelítésből számolt értékek: a vízszintes tengelyen ε, a függőlegesen a közelített kondíciószám szerepel. A pontok tehát nem mérési adatok, hanem a képletből előállított szemléltetés; a tényleges mátrixra számolt érték a labor kimenetében $6{,}0001 \cdot 10^{10}$ $\varepsilon = 10^{-5}$ mellett.

A kód az `rcond=None` mellett a NumPy által használt SVD-küszöböt is kiírja: `rcond_eff = eps * max(n, d)`, a küszöb pedig `rcond_eff * max  1,63·10^-15`; a rang ennél a küszöbnél még 2, tehát a mátrix matematikai értelemben teljes oszloprangú. A numerikus nehézséget nem szingularitás, hanem a nagy kondíciószám okozza.

3.3.7 Feladatok

F1 (fogalmi, nehézség 1). Legyen $A \in \mathbb{R}^{n \times d}$ és $b \in \mathbb{R}^n$. Definiálja a reziduumot és a vetületet. Magyarázza el, miért merőleges a legkisebb négyzetes megoldás reziduuma A minden oszlopára, és miért nem elegendő ez a merőlegesség a ranghiányos eset egyértelműségéhez.

F2 (levezetési, nehézség 2). Alkalmazza a 3. tételt a $d = 1$ speciális esetre, amikor $A = a \in \mathbb{R}^n$ egyetlen oszlopvektor. Vezesse le a normálegyenletet, adja meg az invertálhatóság feltételét, és fejezze ki $\hat{x}$-et az a és b vektorok skalárszorzataival.

F3 (implementációs, nehézség 2). Írjon `lab/n011_lstsq/least_squares.py` fájlt, amely a

```
def least_squares(A: np.ndarray, b: np.ndarray) -> tuple[np.ndarray, np.ndarray, int]:
    ...
```

függvényt tartalmazza. A függvény `np.linalg.lstsq`-t használjon, adja vissza a megoldást, a reziduumot és a numerikus rangot. Ellenőrizze a főszöveg 3×2-es példáján, hogy a visszaadott megoldás $2 \cdot 10^{-5}$ abszolút toleranción belül $(1, 2)^\top$, a reziduum $(1, -2, 1)^\top$, a rang pedig 2.

F4 (hibakeresési, nehézség 2). Valaki a következő függvényt írja:

```
def bad_solve(A, b):
    return np.linalg.inv(A.T @ A) @ A.T @ b
```

Hol van a kettős hiba? Elemezze a ranghiányos esetet és a nagy kondíciószámú, de matematikailag teljes rangú esetet is. Adjon olyan javítást, amely mindkét hibát kezeli, és írja meg az ellenőrző tesztet.

F5 (kísérlettervezés, nehézség 3). Tervezzen kísérletet annak vizsgálatára, hogy a harmadik, a két közel kollineáris oszloptól független oszlop hozzáadása hogyan változtatja meg a legkisebb négyzetes megoldás stabilitását. Adja meg a kérdést, a generált adat sémáját, a baseline-t, a változó tényezőt, a metrikát és a következtetési korlátot. Nem szükséges futtatni; a protokollnak lektorálhatónak kell lennie.

3.3.8 Megoldások és indoklás

F1. A reziduum $r(x) = b - Ax$. A vetület az $U = \operatorname{range}(A)$ altér azon eleme, amelyre a távolság minimális. A 2. lemma bizonyítja, hogy a minimalizáló reziduum merőleges U minden elemére, tehát A minden oszlopára is, mert az oszlopok U-t generálják. Ranghiányos A esetén a merőlegesség $A^\top r = 0$ még teljesülhet, de a normálegyenletnek végtelen sok megoldása van: az oszlopok lineáris függősége miatt több x adhatja ugyanazt a p-t. A reziduum tehát a predikciót jellemzi, nem az együtthatók azonosíthatóságát.

F2. Ha $A = a$, akkor a normálegyenlet

$$a^\top a\, \hat{x} = a^\top b.$$

Az invertálhatóság feltétele $a^\top a = \|a\|_2^2 > 0$, azaz $a \neq 0$. Ilyenkor

$$\hat{x} = \frac{a^\top b}{a^\top a} = \frac{\langle a, b\rangle}{\|a\|_2^2}.$$

Ez éppen a b-nek az a irányára vett ortogonális vetületi együtthatója. Ha $a = 0$, akkor $U = \{0\}$, a vetület 0, és nincs egyértelmű $\hat{x}$; a normálegyenlet $0\,\hat{x} = 0$ alakú.

F3. Lehetséges teljes fájl:

```
# lab/n011_lstsq/least_squares.py
import numpy as np

def least_squares(A: np.ndarray, b: np.ndarray) -> tuple[np.ndarray, np.ndarray, int]:
    if A.ndim != 2 or b.ndim != 1:
        raise ValueError("A legyen 2D, b legyen 1D")
    if A.shape[0] != b.shape[0]:
        raise ValueError("A sorainak száma egyezzen b hosszával")
    x, _, rank, _ = np.linalg.lstsq(A, b, rcond=None)
    residual = b - A @ x
    return x, residual, rank

if __name__ == "__main__":
    EPS = 1e-5
    A = np.array([[1.0, 1.0], [1.0, 1.0 + EPS], [1.0, 1.0 + 2.0 * EPS]])
    x_ref = np.array([1.0, 2.0])
    b = A @ x_ref + np.array([1.0, -2.0, 1.0])
    x, r, rank = least_squares(A, b)
```

```
    assert rank == 2
    assert np.allclose(x, x_ref, rtol=0.0, atol=2e-5)
    assert np.allclose(r, [1.0, -2.0, 1.0], atol=1e-8)
    print("F3 kontroll: mindhárom feltétel teljesült.")
```

Futtatás: `python lab/n011_lstsq/least_squares.py`. A ranghiányos szélsőeseten, például $A = [[1, 2], [2, 4]]$ és $b = [1, 2]$, a visszaadott rang 1, a reziduum nulla, a megoldás pedig a minimum-norma $(0.2, 0.4)^\top$ lesz a NumPy `lstsq` konvenciója szerint; ezt a felhasználónak is tesztelnie kell.

F4. A `bad_solve` két hibája:

1. **Explicit inverz a normálegyenletben.** A $A^\top A$ képzése a kondíciószámot négyzetre emeli, az inverz pedig további $O(d^3)$ műveletet és kerekítést igényel. Közel kollineáris oszlopoknál ez pontatlan vagy használhatatlan eredményt ad, miközben a matematikai feladat stabilan megoldható.
2. **Nincs rang- vagy kondícióvizsgálat.** Ranghiányos A esetén a függvény szinguláris mátrix inverzét kísérli meg, ami kivételt vethet vagy tetszőleges, kerekítésfüggő értéket adhat.

Javítás: használjon SVD- vagy QR-alapú megoldót, például

```
def good_solve(A, b):
    x, _, rank, _ = np.linalg.lstsq(A, b, rcond=None)
    if rank < A.shape[1]:
        raise np.linalg.LinAlgError("ranghiányos adatmátrix")
    return x
```

Ellenőrző teszt: a fenti `good_solve` függvényt tegyük `good_solve.py` fájlba, a tesztet pedig `test_good_solve.py` fájlba.

```
import numpy as np
import pytest
from good_solve import good_solve

def test_kozel_kollinearis_referencia():
    EPS = 1e-5
    A = np.array([[1.0, 1.0], [1.0, 1.0 + EPS], [1.0, 1.0 + 2 * EPS]])
    x_ref = np.array([1.0, 2.0])
    b = A @ x_ref + np.array([1.0, -2.0, 1.0])
    x = good_solve(A, b)
    assert np.allclose(x, x_ref, rtol=0.0, atol=2e-5)

def test_ranghianyos_kivetel():
    A = np.array([[1.0, 2.0], [2.0, 4.0]])
    b = np.array([1.0, 2.0])
    with pytest.raises(np.linalg.LinAlgError):
        good_solve(A, b)
```

A teszt a rögzített venvben futtatva 2 tesztet teljesít; a közel kollineáris esetben a megoldás hibája $9{,}27 \cdot 10^{-7}$, ami a $2 \cdot 10^{-5}$ abszolút tolerancia alatt van. A tolerancia indoklása számszerű: a laborban mért hiba $9{,}27 \cdot 10^{-7}$, tehát a küszöb mintegy hússzoros tartalékot ad; az elsőrendű elméleti becslés $\kappa_2(A)\varepsilon_{\text{gép}} \approx 5{,}4 \cdot 10^{-11}$ ennél optimistább, mert a közel szinguláris irány a kerekítési hibát felerősíti. A $2 \cdot 10^{-5}$ tehát gyakorlati döntési küszöb, nem elméleti hibakorlát; `float64`-ben a gép epsilonja $2{,}22 \cdot 10^{-16}$, és kisebb, 10^{-8} nagyságú tolerancia ezen a kondicionáltságon nem lenne reális elvárás.

F5. Egy lehetséges terv:

- **Kérdés:** egy harmadik, az első kettőtől lineárisan független oszlop hogyan változtatja meg a két közel kollineáris együttható becslésének stabilitását?
- **Adat:** $n = 120$ sor. Az első oszlop $a_1 = (1, \dots, 1)^\top$; a második $a_2 = a_1 + \varepsilon u$, ahol $\varepsilon = 10^{-3}$ és $u_i \sim N(0,1)$ független standard normálisok. A harmadik oszlop $a_3 = (\sin(2\pi i/n))_{i=1}^n$, amely matematikailag merőleges a_1-re, mert $\sum_{i=1}^n \sin(2\pi i/n) = 0$, ezért a konstans oszloppal korrelálatlan, és az első kettőtől lineárisan független. A célvektor $b = A(1,2,0)^\top + \delta$, ahol $\delta \sim N(0, 0.1^2 I)$; a harmadik együttható értéke 0, ezért a kétoszlopos baseline nem misszpecifikált, és a két modell összehasonlítása a stabilitást méri, nem a kihagyott változó torzítását. A seed rögzített (például 20261030), mert itt generált véletlen adat szerepel.
- **Baseline:** kétoszlopos modell a közel kollineáris párral, `np.linalg.lstsq`-val, ugyanazon a b-n.
- **Változó:** A_2 (két oszlop) vagy A_3 (három oszlop).
- **Metrika:** $\kappa_2(A)$, az első két becsült együttható eltérése a referencia $(1,2)$-től, a predikció hibája és a reziduum-norma.
- **Döntési szabály:** a harmadik oszlop akkor tekinthető semlegesnek, ha az első két együttható hibája nem nő a kétoszlopos baseline-hoz képest; ha nő, a többletparaméter stabilitási költsége látható.
- **Következtetési korlát:** a szintetikus adat nem a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab, azaz a könyv saját, procedurálisan generált képadatkészlete, ezért a numerikus viselkedés igazol, de nem állít elő valós regressziós teljesítményt. A seed és a perturbáció eloszlása rögzítendő; az eredmény csak az adott generálási sémára vonatkozik.

3.3.9 Tipikus tévedés és ellenpélda

A normálegyenlet levezetése nem teszi az explicit inverzt ajánlott numerikus algoritmussá. A tétel egzakt aritmetikában érvényes: ha az inverz pontosan számítható, a zárt képlet a pontos megoldást adja. A lebegőpontos számítás viszont kerekít, és eközben a $A^\top A$ mátrix kondíciószáma $\kappa_2(A)^2$. A labor 3×2-es mátrixánál $\varepsilon = 10^{-5}$ esetén ez mintegy $6 \cdot 10^{10}$, tehát az explicit normálegyenlet megoldása érezhetően érzékeny a kerekítésre. Ugyanaz a feladat `np.linalg.lstsq`-val, amely közvetlenül A-n dolgozik, stabilabban oldható meg, mert ott a kondíciószám csak $\kappa_2(A)$.

Ranghiányos ellenpélda a kézi részben szerepelt: $A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$ esetén a normálegyenlet mátrixa szinguláris, így az explicit inverz még egzakt aritmetikában sem létezik, holott a legkisebb négyzetes **predikció** tökéletesen definiált.

3.3.10 Önellenőrzés

Ellenőrizze az alakokat: $A^\top A$ alakja $d \times d$, $A^\top b$ alakja d, a megoldás d-dimenziós, a reziduum pedig n-dimenziós. Ha a feladat F helyett $\frac{1}{n}F$ átlagos négyzetes hibát minimalizál, a skálázás nem változtatja meg a minimalizálót. Ranghiányos esetben a predikció $p = A\hat{x}$ egyértelmű, de $\hat{x}$ nem; a kettőt ne keverje össze. A nagy kondíciószám nem azonos a szingularitással: a labor mátrixa rangja 2, mégis instabil a normálegyenlet. Visszatekintés 3.2. Mátrixszorzás és affín réteg alfejezetből: az affin réteg csak akkor lineáris, ha a bias nulla; a regressziós baseline konstans oszloppal éppen affin modellt ad.

3.3.11 Rövid összefoglaló

A legkisebb négyzetes feladat a célvektor vetülete az adatmátrix képterére. A minimalizáló reziduum merőleges a képtérre, ami az $A^\top Ax = A^\top b$ normálegyenletet adja. Teljes oszloprang esetén $A^\top A$ pozitív definit és invertálható, ezért a megoldás egyértelmű: $\hat{x} = (A^\top A)^{-1} A^\top b$. Ranghiányos esetben a predikció egyértelmű, az együttható nem; a könyvtárak minimum-norma megoldást adnak. A normálegyenlet mátrixának kondíciószáma $\kappa_2(A)^2$, ezért közel kollineáris oszlopoknál az explicit inverz numerikusan megbízhatatlan; a stabil `lstsq` közvetlenül az eredeti mátrixon dolgozik. A kondíciószám nagy értéke nem szingularitás, hanem érzékenységi figyelmeztetés.

3.3.12 Továbblépés

A 3.4. Sajátérték, SVD és spektrális határ a sajátértékeket, a szinguláris értékeket és a spektrális határokat tárgyalja; ott a kondíciószám pontos SVD-alapú definícióját és a perturbációs korlátokat is levezetjük.

3.3.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

3.4 Sajátérték, SVD és spektrális határ

3.4.1 Cél, előismeret és induló kérdés

A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság a legkisebb négyzetes feladatot vetítésként és normálegyenletként vizsgálta. Ott a kondíciószámot már megelőlegeztük, mélyebb definíció nélkül. Most az a kérdés, hogy egy $A \in \mathbb{R}^{m \times n}$ lineáris leképezés **mely irányban nyújtja a legerősebben** az egységvektorokat, és ez a maximális nyújtás hogyan ad felső korlátot minden $\|Ax\|_2$ értékre.

Induló becslésként használjuk az oszlopnormákat. Ha A oszlopai legfeljebb 1 hosszúak, akkor a háromszög-egyenlőtlenséggel

$$\|Ax\|_2 \le \sum_j |x_j| \, \|a_j\|_2 \le \sum_j |x_j| \le \sqrt{n} \, \|x\|_2,$$

tehát valamilyen $\sqrt{n}$-es korlát mindig van. Ez azonban általában nem éles: ortonormált oszlopú mátrixra (az oszlopok ortogonálisak *és* egységhosszúak) a maximális nyújtás 1, nem $\sqrt{n}$. Az ortogonalitás önmagában nem jelent egységhosszt: ha az oszlophosszak $\frac{1}{2}$-del egyenlők (a fenti legfeljebb-1 feltétel mellett ez megengedett), a maximális nyújtás $\frac{1}{2}$. A fejezet célja az éles állandó, a **spektrális norma** bevezetése.

A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetből ismert az ortogonális mátrix fogalma, a vetítés és a ranghiány alapgondolata. A 2.4. Kis tudományos szoftverprojekt alfejezetből a kódprojekt-fegyelem: explicit input, reprodukálható ellenőrzés, futási jegyzék. Az itteni labor ugyanezt a formát követi.

3.4.2 Sajátérték és amiért nem elég

Definíció. Legyen $A \in \mathbb{R}^{n\times n}$ négyzetes mátrix. A $\lambda \in \mathbb{C}$ szám **sajátértéke** A-nak, ha létezik $v \neq 0$, $v \in \mathbb{C}^n$, amelyre

$$Av = \lambda v.$$

A v vektor a λ-hoz tartozó sajátvektor. A definíció csak négyzetes mátrixra van értelmezve, mert Av és λv alakjának meg kell egyeznie. Az $x \mapsto Ax$ leképezés tehát egy sajátirányban csak skáláz: a kép párhuzamos az inputtal.

A sajátérték nem mindig jellemzi a torzítást. Tekintsük a

$$J = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

mátrixot. Karakterisztikus polinomja λ^2, tehát egyetlen sajátértéke $\lambda = 0$, algebrai multiplicitással 2 (a karakterisztikus polinomban a λ gyöke kétszeres). A leképezés mégsem nulla: $Je_2 = e_1$, így a e_2 egységvektor képe 1 hosszú. Általában egy nem szimmetrikus mátrix sajátvektorai nem alkotnak ortonormált bázist, ezért a maximális nyújtást nem olvashatjuk le pusztán a legnagyobb $|\lambda|$-ból. Erre való a szinguláris érték.

A híd a két fogalom között a következő. Szimmetrikus A-ra a spektráltétel ortonormált sajátbázist ad, ezért ott $\|A\|_2 = \max_i |\lambda_i|$. Általános, nem szimmetrikus A-ra viszont az $A^\top A$ szimmetrikus, pozitív szemidefinit (azaz $z^\top A^\top A z \geq 0$ minden z-re) mátrix ortonormált sajátbázisa adja a jobb oldali szinguláris irányokat, és az $A^\top A$ sajátértékeinek négyzetgyökei a szinguláris értékek. Az SVD-tétel éppen ezt a konstrukciót fogalmazza meg minden téglalapmátrixra; a következő szakaszban ezt használjuk fel a spektrális norma kiszámításához.

3.4.3 A teljes SVD-tétel és a szinguláris értékek

Tétel (teljes SVD, felhasznált állítás). Minden $A \in \mathbb{R}^{m\times n}$ mátrix előáll

$$A = U\Sigma V^\top,$$

alakban, ahol $U \in \mathbb{R}^{m\times m}$ és $V \in \mathbb{R}^{n\times n}$ ortogonális mátrixok, $\Sigma \in \mathbb{R}^{m\times n}$ olyan mátrix, amelynek csak a $\Sigma_{ii},\ i = 1, \dots, \min(m,n)$ elemei lehetnek nullától különbözők, és ezek

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0, \qquad \sigma_{r+1} = \dots = \sigma_{\min(m,n)} = 0$$

szerint rendezettek. Az r a mátrix rangja, a σ_i számok a **szinguláris értékek**. Ebből következnek a kapcsolatok

$$Av_i = \sigma_i u_i, \qquad A^\top u_i = \sigma_i v_i \qquad (i = 1, \dots, r).$$

Ezt a létezési és ortonormáltsági állítást a [B01] tárgyalja; a teljes bizonyítást itt tudatosan nem másoljuk, és csak az itt megnevezett állításokat használjuk. A felhasznált rész tehát: U, V ortogonális, Σ diagonális szerkezetű, és a nemnulla σ_i-k száma r.

A **kompakt (rank-r) SVD** elhagyja a nulla szinguláris értékekhez tartozó oszlopokat (nem azonos a NumPy `full_matrices=False` *economy* SVD-jével, amely $\min(m,n)$ oszlopot tart meg, és a nulla szinguláris értékeket nem dobja el):

$$A = U_r \Sigma_r V_r^\top,$$

ahol $U_r \in \mathbb{R}^{m\times r}$, $\Sigma_r \in \mathbb{R}^{r\times r}$ pozitív diagonális, $V_r \in \mathbb{R}^{n\times r}$. Mivel A képtere legfeljebb r-dimenziós, a redukált alak is egzakt egyenlőség.

Rang. Az A **rangja** a képtér dimenziója, azaz a lineárisan független oszlopok száma. A fenti tételben éppen a pozitív szinguláris értékek száma. Az A **magtere** (más néven nulltere) a $\{x \in \mathbb{R}^n : Ax = 0\}$ altér; ez éppen a nulla szinguláris értékhez tartozó jobb oldali szinguláris vektorok terének feszítője, képtérét pedig az $u_1, \dots, u_r$ vektorok feszítik. A gyakorlatban a rangot gyakran numerikus küszöbbel becsüljük: a nagyon kicsi σ_i-t nulla rangnak tekintjük, de ez a skálafüggő küszöb heurisztika, nem a matematikai rang definíciója.

Kézi példa.

$$A = \begin{pmatrix} 3 & 0 \\ 4 & 0 \end{pmatrix}.$$

A mátrix második oszlopa nulla, ezért a képtér az $\binom{3}{4}$ által feszített egyenes, tehát $\operatorname{rank}(A) = 1$. Az SVD:

$$U = \begin{pmatrix} 3/5 & -4/5 \\ 4/5 & 3/5 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 5 & 0 \\ 0 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Ellenőrzésül:

$$U\Sigma V^\top = \begin{pmatrix} 3/5 \\ 4/5 \end{pmatrix} 5 \begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 4 & 0 \end{pmatrix} = A.$$

Tehát $\sigma_1 = 5$, $\sigma_2 = 0$. Az $Ae_1 = \binom{3}{4}$ vektor hossza 5, az $Ae_2 = 0$; a maximális nyújtás sejthetően 5. A könyvtári ellenőrző munkamenet futtatott kimenete (Python 3.12.3, NumPy 2.5.4, CPU, egy szál; seed nincs, mert a példa determinisztikus):

```
import numpy as np

A = np.array([[3.0, 0.0],
              [4.0, 0.0]], dtype=np.float64)
print(np.linalg.svd(A, compute_uv=False))
print(np.linalg.matrix_rank(A))

assert np.allclose(np.linalg.svd(A, compute_uv=False), [5.0, 0.0])
assert np.linalg.matrix_rank(A) == 1
```

A futtatott kód tényleges kimenete:

```
[5. 0.]
1
```

Az $A^\top A$ mátrix $\left(\begin{smallmatrix} 25 & 0 \\ 0 & 0 \end{smallmatrix}\right)$, sajátértékei 25 és 0; a pozitív sajátérték négyzetgyöke éppen $\sigma_1 = 5$. Általában $A^\top A = V\Sigma^\top \Sigma V^\top$, tehát $A^\top A$ sajátértékei a σ_i^2 értékek. Numerikusan mégsem az $A^\top A$ sajátfelbontásából számolunk szinguláris értékeket, mert a kondíciószám négyzetre emelkedik; az `np.linalg.svd` közvetlenül az A-n dolgozik.

3.4.4 Spektrális norma és a normakorlát levezetése

Definíció. Az $A \in \mathbb{R}^{m \times n}$ mátrix **spektrális normája**

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2.$$

Ez a leképezés **maximális nyújtási aránya**. A maximum létezik, mert véges dimenzióban az egységgömb kompakt, és az $x \mapsto \|Ax\|_2$ függvény folytonos.

Először a kötelező levezetést diagonális operátorra végezzük el.

Lemma (diagonális operátor normakorlátja). Legyen $D \in \mathbb{R}^{m\times n}$ olyan mátrix, amelynek csak a $D_{ii} = d_i$, $i = 1, \dots, q = \min(m, n)$ elemei lehetnek nullától különbözők. Ekkor

$$\|D\|_2 = \max_{1\le i\le q} |d_i|.$$

Bizonyítás. Legyen $d_* = \max_i |d_i|$, és legyen j olyan index, amelyre $|d_j| = d_*$. Az $e_j \in \mathbb{R}^n$ standard egységvektorra $De_j = d_j e_j$ teljesül, tehát

$$\|De_j\|_2 = |d_j| = d_*,$$

így $\|D\|_2 \ge d_*$.

A felső korláthoz legyen $x \in \mathbb{R}^n$, $\|x\|_2 = 1$. A diagonális szerkezetből $(Dx)_i = d_i x_i$ az $i \le q$ koordinátákra, és nulla a többi helyen. Ezért

$$\|Dx\|_2^2 = \sum_{i=1}^{q} d_i^2 x_i^2 \le d_*^2 \sum_{i=1}^{q} x_i^2 \le d_*^2 \sum_{i=1}^{n} x_i^2 = d_*^2 \|x\|_2^2 = d_*^2.$$

Négyzetgyököt vonva $\|Dx\|_2 \le d_*$, és mivel ez minden egységvektorra igaz, $\|D\|_2 \le d_*$. A két egyenlőtlenségből az állítás. $\square$

A tetszőleges A esetét erre vezetjük vissza a megnevezett SVD-állítás segítségével.

Levezetés: a spektrális norma SVD-ből. Legyen $A = U\Sigma V^\top$ teljes SVD. Egy tetszőleges egységvektorra legyen $y = V^\top x$. Mivel V ortogonális,

$$\|y\|_2^2 = y^\top y = x^\top V V^\top x = x^\top x = \|x\|_2^2,$$

tehát y is egységvektor. Hasonlóan U ortogonalitása miatt $\|Uz\|_2 = \|z\|_2$. Így

$$\|Ax\|_2 = \|U\Sigma V^\top x\|_2 = \|\Sigma (V^\top x)\|_2 = \|\Sigma y\|_2.$$

A lemma szerint $\|\Sigma y\|_2 \le \sigma_1 \|y\|_2 = \sigma_1 \|x\|_2$, mert a Σ diagonális elemei nemnegatívak, csökkenően rendezettek, és a legnagyobb σ_1. Ezért

$$\|Ax\|_2 \le \sigma_1 \|x\|_2 \qquad \text{minden } x \in \mathbb{R}^n \text{ esetén.}$$

Az $x = v_1$ választással $y = e_1$, így $Av_1 = \sigma_1 u_1$, és $\|Av_1\|_2 = \sigma_1 \|u_1\|_2 = \sigma_1$. Tehát az egyenlőség teljesül, és

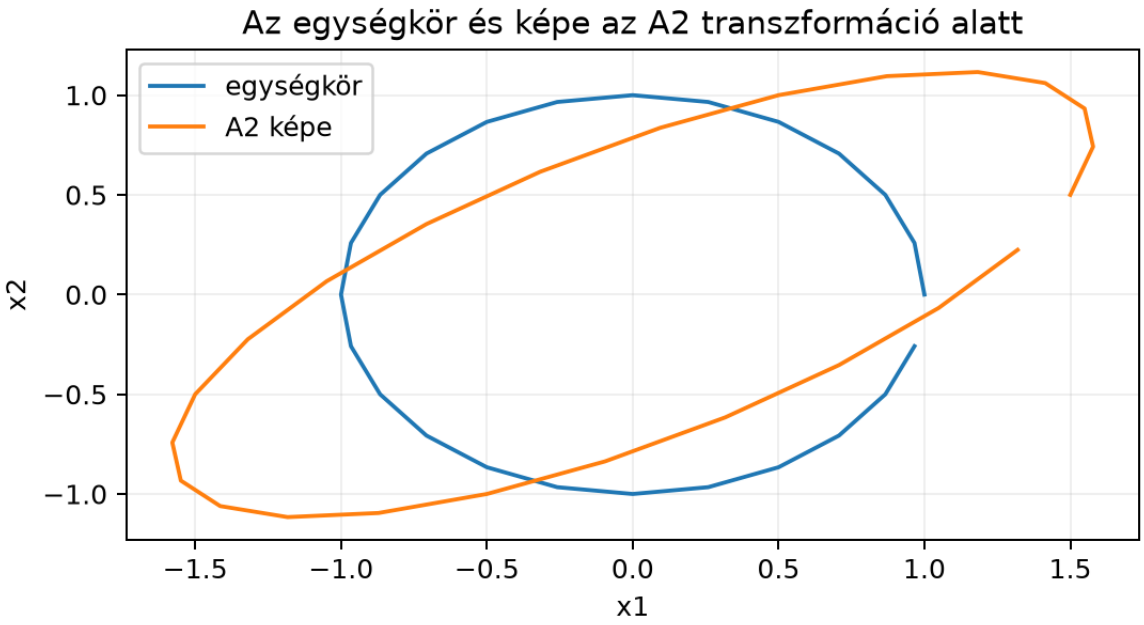
$$\|A\|_2 = \sigma_1 = \sigma_{\max}(A).$$

Ez a hasznos **normakorlát** tehát:

$$\|Ax\|_2 \le \|A\|_2 \|x\|_2.$$

A korlát éles, de nem minden irányban teljesül egyenlőséggel: egyenlőség pontosan azokra az x-ekre áll fenn, amelyek a legnagyobb szinguláris értékhez tartozó jobb szinguláris altérben vannak (és a nullvektorra). Szemléltetés az $A = \mathrm{diag}(3, 1)$, $x = e_2$: $\|Ae_2\|_2 = 1 < 3 = \|A\|_2 \|e_2\|_2$, noha e_2 jobb szinguláris vektor; ez nem cáfolja a korlátot, csak azt mutatja, hogy az egyenlőség nem minden irányban teljesül.

Az SVD geometriai tartalma röviden: az egységkör képe ellipszis, amelynek féltengelyei σ_1 és σ_2 hosszúak, irányai pedig u_1 és u_2. A következő szemléltetés az $A_2 = \begin{pmatrix} 1{,}5 & 0{,}5 \\ 0{,}5 & 1{,}0 \end{pmatrix}$ mátrixszal mutatja ezt; szinguláris értékei $\sigma_1 \approx 1{,}809$, $\sigma_2 \approx 0{,}691$.



3.5. ábra: Az egységkör és képe az A2 transzformáció alatt

A 24 pontból álló egységkör és az A2 mátrixszal transzformált képe; képletből számolt szemléltetés, nem mérés.

A chart két görbéje tehát ugyanannak a körnek az eredeti és transzformált változata; a transzformált görbe féltengelyeinek hossza a két szinguláris érték, iránya pedig a bal oldali szinguláris vektoroké.

3.4.5 Kapcsolat a kondíciószámmal és az alacsony rangú közelítéssel

A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetben igértük meg a kondíciószám pontos definícióját. Ha $A \in \mathbb{R}^{m\times n}$ teljes oszloprangú ($\sigma_n > 0$), akkor

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_n} = \frac{\|A\|_2}{\sigma_{\min}(A)}.$$

Ranghiányos esetben $\sigma_{\min} = 0$, ezért a kondíciószámot $+\infty$ szimbólummal jelöljük. Ebből azonnal látszik a 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetbeli figyelmeztetés indoka is: $A^\top A$ sajátértékei σ_i^2, tehát

$$\kappa_2(A^\top A) = \frac{\sigma_1^2}{\sigma_n^2} = \kappa_2(A)^2.$$

A normálegyenlet mátrixa tehát pontosan a négyzetére növelt érzékenységet örökli.

A szinguláris értékek csökkenő sorrendje természetes tömörítést is ad. Rögzítsünk $1 \le k \le r$-t, és legyen

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top.$$

Jelölje A_k az A **alacsony rangú közelítését**. A $\operatorname{rank}(A_k) = k$, mert a k darab diád (az $u_i v_i^\top$ alakú rang-1 mátrix) lineárisan független képtér-irányt feszít.

Tétel (Eckart–Young–Mirsky, megnevezett eredmény, itt nem bizonyított). Legyen $1 \le k < \min(m, n)$. Ekkor tetszőleges $B \in \mathbb{R}^{m \times n}$, $\operatorname{rank}(B) \le k$ mátrixra

$$\|A - B\|_2 \ge \sigma_{k+1},$$

és a fenti A_k választással egyenlőség teljesül. Ha $k \ge r$, akkor $A_k = A$, és a $\sigma_{k+1} = 0$ konvencióval az állítás triviális; az érdemi eset $k < r$.

A tétel feltételeihez és bizonyításához a [B01] 4. fejezetét, illetve az eredeti Eckart–Young (1936) cikket kell használni: a szinguláris érték-felbontást a [B01] 4.5., a mátrix-közelítést és benne az Eckart–Young-tételt a 4.6. szakasz tárgyalja. Ez az egység csak a spektrális norma alakú állítását nevezi meg, a bizonyítást nem ismétli meg. A kép- és mátrixtömörítésben ugyanez a konstrukció a kiindulópont: a legnagyobb σ_i-khez tartozó tagok hordozzák a legerősebb lineáris struktúrát.

A **PCA-rekonstrukció** (főkomponens-analízis, PCA) szempontjából ez a következőt jelenti. Legyen $X \in \mathbb{R}^{N \times d}$ adatmátrix, amelyben minden megfigyelés egy sor, és legyen $\mu = \frac{1}{N} X^\top \mathbf{1} \in \mathbb{R}^d$ a jellemzőnkénti (oszlopátlag) vektor. Az oszloponként centrált mátrix $X_c = X - \mathbf{1}\mu^\top$, vagyis minden jellemzőből a minták közötti átlagát vonjuk le. Az $X_c^\top X_c$ szimmetrikus mátrix sajátfelbontásának főtengelyeit a jobb oldali szinguláris vektorok adják, mert $X_c = U\Sigma V^\top$. Az első főtengely v_1, azaz a legnagyobb szinguláris értékhez tartozó irány a d-dimenziós bemeneti térben. Egy centrált minta oszlopvektorként $x \in \mathbb{R}^d$; kódja $z = V_k^\top x \in \mathbb{R}^k$, lineáris **rekonstrukciója** pedig

$$\hat{x} = V_k z = V_k V_k^\top x.$$

A k kódkoordinátából tehát az adatpontnak csak a V_k által feszített altérbe eső részét kapjuk vissza; a komplementer rész az eldobott szinguláris irányokban maradt hiba. A formális variancia- és valószínűségértelmezés az 5.1.–5.2. tárgya; itt a „szóródási irány” csak a centrált négyzetes eltérések lineáris algebrai jellemzését jelenti.

3.4.6 Labor: négyzet és kör két lineáris transzformáció alatt

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor két fix 2×2-es mátrixszal transzformál egy $[-1, 1]^2$ négyzetet és egy egységkörvonalat, majd a véges pontmintán mért maximális nyújtást a szinguláris értékkel hasonlítja össze. Az inputok: $A_1 = \operatorname{diag}(2;\ 0{,}5)$, $A_2 = \left(\begin{smallmatrix} 1{,}5 & 0{,}5 \\ 0{,}5 & 1{,}0 \end{smallmatrix}\right)$. Az alakzatok a 2D-sík pontjai; a kód float64-ben, CPU-n dolgozik. Futtatási környezet: Python 3.12.3, NumPy 2.5.4, matplotlib 3.11.2 (`MPLBACKEND=Agg`), CPU, egy szál (`OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`); a labor nem használ PyTorchöt. A példa determinisztikus, ezért seedre nincs szükség; ha a hallgató később zajt ad a ponthalmazhoz, akkor `np.random.default_rng(20261020)` használandó.

```
"""n012_shape_stretch.py

A négyzet és kör maximális véges-mintás nyújtása két lineáris
transzformáció alatt, összevetve a spektrális normával.

Munkakönyvtár: lab/n012
Futtatás:      python n012_shape_stretch.py

A kimeneti 'shape_under_transforms.png' a torzított alakzatokat menti.
A kimenet rögzített környezetben futtatott eredmény (Python 3.12.3,
NumPy 2.5.4, matplotlib 3.11.2, CPU, egy szál).
"""

from pathlib import Path
```

```
import matplotlib.pyplot as plt
import numpy as np

WORK = Path("lab/n012")
WORK.mkdir(parents=True, exist_ok=True)
DTYPE = np.float64

# Két vizsgált lineáris transzformáció.
A1 = np.array([[2.0, 0.0],
               [0.0, 0.5]], dtype=DTYPE)
A2 = np.array([[1.5, 0.5],
               [0.5, 1.0]], dtype=DTYPE)


def unit_circle(n: int = 512) -> np.ndarray:
    """Az egységkörvonal n darab pontja, shape (n, 2)."""
    theta = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False, dtype=DTYPE)
    return np.column_stack((np.cos(theta), np.sin(theta)))


def unit_square() -> np.ndarray:
    """A [-1,1]^2 zárt négyzet csúcsai és oldalfelező pontjai; az első pont ismétlődik."""
    return np.array([[-1.0, -1.0],
                     [ 0.0, -1.0],
                     [ 1.0, -1.0],
                     [ 1.0,  0.0],
                     [ 1.0,  1.0],
                     [ 0.0,  1.0],
                     [-1.0,  1.0],
                     [-1.0,  0.0],
                     [-1.0, -1.0]], dtype=DTYPE)


def max_finite_stretch(A: np.ndarray, X: np.ndarray) -> float:
    """A véges ponthalmaz pontjaiban mért maximális ||Ax||/||x|| arány."""
    AX = X @ A.T
    return float(np.max(np.linalg.norm(AX, axis=1)
                        / np.linalg.norm(X, axis=1)))


def spectral_norm(A: np.ndarray) -> float:
    """A spektrális norma, azaz a legnagyobb szinguláris érték."""
    return float(np.linalg.svd(A, compute_uv=False)[0])


def main() -> None:
    square = unit_square()
    circle = unit_circle()

    results = []
    for name, A in [("A1", A1), ("A2", A2)]:
        sigma = spectral_norm(A)
        square_stretch = max_finite_stretch(A, square)
        circle_stretch = max_finite_stretch(A, circle)
        print(f"{name}: sigma={sigma:.16f}, "
              f"square_max={square_stretch:.16f}, "
              f"circle_max={circle_stretch:.16f}")
        results.append((A, sigma, square_stretch, circle_stretch))

    # A1: mindkét szinguláris irány szerepel a négyzet mintapontjai között is,
    # ezért a négyzetminta is pontosan eléri a spektrális normát.
    (_, sigma1, square1, circle1) = results[0]
```

```
    np.testing.assert_allclose(square1, sigma1, rtol=0.0, atol=1e-12)
    np.testing.assert_allclose(circle1, sigma1, rtol=0.0, atol=1e-12)

    # A2: a négyzetminta általában nem tartalmazza a legnagyobb nyújtás
    # irányát, tehát csak alsó becslés; a sűrű körvonal az irányt közelíti.
    (_, sigma2, square2, circle2) = results[1]
    assert square2 <= sigma2 * (1.0 + 1e-12) + 1e-15
    np.testing.assert_allclose(circle2, sigma2, rtol=5e-4, atol=5e-4)

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    for ax, (A, _, _, _) in zip(axes, results):
        ax.plot(square[:, 0], square[:, 1], "--", label="eredeti négyzet")
        ax.plot((square @ A.T)[:, 0], (square @ A.T)[:, 1],
                label="transzformált négyzet")
        ax.plot(circle[:, 0], circle[:, 1], ":", label="eredeti kör")
        ax.plot((circle @ A.T)[:, 0], (circle @ A.T)[:, 1],
                label="transzformált kör", linewidth=2.0)
        ax.set_aspect("equal")
        ax.set_title(f"A = {A.tolist()}")
        ax.legend(loc="upper left")
        ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(WORK / "shape_under_transforms.png", dpi=150)
    print(f"Ábra: {WORK / 'shape_under_transforms.png'}")


if __name__ == "__main__":
    main()
```

A futtatott kód kimenete és értelmezése:

```
A1: sigma=2.0000000000000000, square_max=2.0000000000000000, circle_max=2.0000000000000000
A2: sigma=1.8090169943749477, square_max=1.7677669529663687, circle_max=1.8090156045563908
Ábra: lab/n012/shape_under_transforms.png
```

- $A_1 = \mathrm{diag}(2;\ 0{,}5)$, tehát $\sigma_1 = 2$, $\sigma_2 = 0{,}5$. A négyzet csúcsai és oldalfelező pontjai között szerepel e_1 és e_2 iránya is, ezért a mért maximális nyújtás pontosan 2,0.
- $A_2 = \left(\begin{smallmatrix} 1{,}5 & 0{,}5 \\ 0{,}5 & 1{,}0 \end{smallmatrix}\right)$ szimmetrikus, pozitív definit (azaz $z^\top A_2 z > 0$ minden $z \neq 0$-ra), sajátértékei $(2.5 \pm \sqrt{1.25})/2$, így $\sigma_1 \approx 1.80901699$; a mért érték 1.8090169943749477. A négyzetminta maximális aránya $2.5/\sqrt{2} \approx 1.76777$, mert a négyzet csúcsai nem esnek a pontos legmeredekebb irányba; a mért érték 1.7677669529663687. A 512 pontos körvonalon a legközelebbi pont relatív hibája $7.7 \cdot 10^{-7}$, a 5×10^{-4} tűrésen belül; ez a véges minta diszkretizációs hibája. A tűrés a mért hiba mintegy 650-szeres tartaléka, ezért a diszkretizáció mellett a platformfüggő kerekítés számára is marad hely; a tűrés nem lazítja a matematikai korlátot, csak a numerikus összehasonlítás küszöbe.

A labor legfontosabb tanulsága: **a véges alakzatpontokból mért maximum alsó becslés** a spektrális normára. A spektrális norma minden irány felett veszi a szuprémumot; az egyenletesen mintavételezett, sűrű körvonal már jól közelíti ezt, a négyzet viszont nem feltétlenül.

3.4.7 Gyakori tévedés ellenpéldával

A nagy varianciájú irány **nem** szükségképpen a predikció szempontjából legjobb irány. A PCA az X mátrix szóródását magyarázza, nem azt, hogy egy irány mennyire hasznos egy külső y cél előrejelzésére.

Ellenpélda futtatás nélkül is látható. Legyen a megfigyelések első koordinátája $x_{i1} = 10u_i$, a második $x_{i2} = 3t_i$, ahol u_i és t_i független, egységvarianciájú ($\mathrm{Var}\, u = \mathrm{Var}\, t = 1$, $\mathrm{Cov}(u, t) = 0$; itt

Var a variancia, Cov a kovariancia szokásos jelölése) változók; így $\operatorname{Var} x_{i1} = 100 > 9 = \operatorname{Var} x_{i2}$, és u_i a predikciótól független erős háttérjel. A cél legyen $y_i = x_{i2}$. A centrált X szóródási mátrixát az első koordináta dominálja, mert az első koordináta szórása 10, a másodiké 3, tehát az első főtengely lényegében e_1. Az y-t azonban csak e_2 irány mentén tudjuk jól lineárisan magyarázni. A legnagyobb variancia iránya tehát a lehető legrosszabb prediktor, a címke szempontjából hasznos irány pedig kisebb teljes varianciájú. Ez a heurisztika irányválogatásnál fontos: a PCA célfüggetlen dimenziócsökkentés, nem címkeoptimalizáló transzformáció.

3.4.8 Önellenőrzés

1. Miért nem értelmezünk sajátértéket 3×2-es mátrixra, szinguláris értéket viszont igen?
2. Ha $A^\top A$ legkisebb sajátértéke nulla, mit mondhatunk $\sigma_{\min}$-ről és a legkisebb négyzetes megoldás egyértelműségéről?

A válaszok a fejezet végén, az önellenőrzés válaszai bekezdésben találhatók; előbb érdemes önállóan végiggondolni őket.

3.4.9 Feladatok

3.4.9.1 F1 fogalmi — sajátérték és szinguláris érték viszonya

Nehézség: 1. Előfeltétel: 3.3. Vetítés, legkisebb négyzetek és kondicionáltság ortogonális mátrix, jelen egység SVD-definíciója. Adja meg, hogy egy $A \in \mathbb{R}^{m \times n}$ mátrix szinguláris értékei hogyan viszonyulnak az $A^\top A$ és az $AA^\top$ mátrixok sajátértékeihez. Válaszát egy 2×2-es, nem szimmetrikus példán ellenőrizze. Beadandó: a két sajátérték-kapcsolat pontos megfogalmazása, alakokkal, és a példa kiszámolt értékei.

3.4.9.2 F2 levezetési — szubmultiplikativitás és ortogonális invariancia

Nehézség: 2. Előfeltétel: a spektrális norma definíciója. Legyen $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$. Igazolja, hogy

$$\|AB\|_2 \le \|A\|_2 \|B\|_2.$$

Ezután mutassa meg, hogy bármely $m \times m$-es ortogonális Q mátrixra $\|QA\|_2 = \|A\|_2$, és bármely $n \times n$-es ortogonális Q mátrixra $\|AQ\|_2 = \|A\|_2$. Beadandó: teljes, indokolt levezetés; annak jelzése, hogy a szubmultiplikativitási egyenlőtlenség általában nem éles, és hogy az ortogonális tényezős alakoknál egyenlőség áll fenn.

3.4.9.3 F3 implementációs — nyújtási arány ellenőrző függvény

Nehézség: 2. Előfeltétel: jelen egység laborja, 2.4. Kis tudományos szoftverprojekt tesztfegyelem. Írjon olyan `ratio_and_check(A, X)` függvényt, amely minden `X` sorra kiszámítja a $\|Ax\|_2/\|x\|_2$ arányt, ellenőrzi, hogy az nem haladja meg a $\|A\|_2(1 + \tau_{\text{rel}}) + \tau_{\text{abs}}$ korlátot, és visszaadja a maximális arányt. Tesztelje a jelen egység $A = \left(\begin{smallmatrix} 3 & 0 \\ 4 & 0 \end{smallmatrix}\right)$ példáján az e_1, e_2 és $(e_1 + e_2)/\sqrt{2}$ vektorokra a $\tau_{\text{rel}} = 10^{-12}$, $\tau_{\text{abs}} = 10^{-15}$ toleranciákkal. Beadandó: teljes kód, a három arány és az ellenőrzés kimenete.

3.4.9.4 F4 hibakeresési — miért rossz a sajátérték-alapú korlát?

Nehézség: 2. Előfeltétel: a J ellenpélda ebből az egységből. Az alábbi függvény állítólag a maximális nyújtást adja meg:

```
def wrong_max_stretch(A):
    import numpy as np
    return float(np.max(np.abs(np.linalg.eigvals(A))))
```

Adjon konkrét mátrixot, amelyre ez hibás eredményt ad. Lokalizálja a hiba okát, mondja meg a következményt, javítsa a függvényt, és ellenőrizze a javítást a konkrét példán. Beadandó: hibás és helyes kimenet, a hiba magyarázata, javított kód és visszaellenőrzés.

3.4.9.5 F5 kísérlettervezési/transzfer — varianciairány kontra prediktív irány

Nehézség: 3. Előfeltétel: 3.3. Vetítés, legkisebb négyzetek és kondicionáltság legkisebb négyzetek, 1.3. Adatfelosztás és a megfigyelési egység adatfelosztás, jelen egység PCA-rekonstrukciója. Tervezzen olyan kis szimulációs protokollt, amely azt vizsgálja, hogy az első főkomponensre vetített egyetlen prediktor jobb-e, mint egy kézzel kiválasztott, kisebb varianciájú jelirány. Használjon folytonos y célváltozót. Rögzítse a megfigyelési egységet, a generáló sémát, a seedet, a felosztást, a baseline-t, a metrikát és a következtetési korlátot. Beadandó: egy megvalósítható protokoll leírása, amelyen a két prediktor összehasonlítható anélkül, hogy a kimenetét előre „előírnánk". Előre garantált győztes nem szükséges.

3.4.10 Megoldások és indoklás

3.4.10.1 F1 megoldása

Az SVD-alakból

$$A^\top A = V\Sigma^\top \Sigma V^\top, \qquad AA^\top = U\Sigma\Sigma^\top U^\top.$$

A V és U ortogonalitása miatt ezek valódi sajátfelbontások. Ezért $A^\top A$ pozitív szemidefinit, sajátértékei σ_i^2, $AA^\top$ sajátértékei σ_i^2; a két mátrix nemnulla sajátértékeinek halmaza megegyezik, a nulla sajátértékek multiplicitása az m, n méretkülönbség szerint változhat.

Példa: $A = \left(\begin{smallmatrix} 3 & 0 \\ 4 & 0 \end{smallmatrix}\right)$. Ekkor $A^\top A = \left(\begin{smallmatrix} 25 & 0 \\ 0 & 0 \end{smallmatrix}\right)$, sajátértékei $25, 0$, tehát a szinguláris értékek $5, 0$. Az $AA^\top = \left(\begin{smallmatrix} 9 & 12 \\ 12 & 16 \end{smallmatrix}\right)$, karakterisztikus polinomja $\lambda^2 - 25\lambda$, sajátértékei szintén $25, 0$.

3.4.10.2 F2 megoldása

Legyen x egységvektor. A normakorlátot kétszer alkalmazzuk:

$$\|(AB)x\|_2 = \|A(Bx)\|_2 \le \|A\|_2 \|Bx\|_2 \le \|A\|_2 \|B\|_2 \|x\|_2 = \|A\|_2 \|B\|_2.$$

Mivel ez minden egység x-re fennáll, a bal oldali maximumra is igaz:

$$\|AB\|_2 \le \|A\|_2 \|B\|_2.$$

Ortogonális Q-ra $\|Qz\|_2 = \|z\|_2$ minden z-re, ezért

$$\|QA\|_2 = \max_{\|x\|=1} \|QAx\|_2 = \max_{\|x\|=1} \|Ax\|_2 = \|A\|_2.$$

Az AQ esetben $z = Qx$ az egységgömböt önmagára képezi, ezért az $\{AQx : \|x\| = 1\}$ halmaz azonos az $\{Az : \|z\| = 1\}$ halmazzal, tehát a két maximum egyenlő. Az ortogonális mátrix mindig négyzetes: QA esetén $Q \in \mathbb{R}^{m\times m}$, AQ esetén $Q \in \mathbb{R}^{n\times n}$.

A szubmultiplikativitási egyenlőtlenség általában nem éles. Például $A = \mathrm{diag}(1, 0)$ és $B = \mathrm{diag}(0, 1)$ esetén $AB = 0$, tehát $\|AB\|_2 = 0$, miközben $\|A\|_2\|B\|_2 = 1$. Ortogonális tényező esetén viszont az invariancia miatt végig egyenlőség áll fenn.

3.4.10.3 F3 megoldása

```python
import numpy as np

def ratio_and_check(A, X, rtol=1e-12, atol=1e-15):
    A = np.asarray(A, dtype=np.float64)
    X = np.asarray(X, dtype=np.float64)
    if X.ndim != 2:
        raise ValueError("X legyen (n, d) alakú.")
    denom = np.linalg.norm(X, axis=1)
    if np.any(denom == 0.0):
        raise ValueError("A nullvektorhoz nincs nyújtási arány.")
    ratios = np.linalg.norm(X @ A.T, axis=1) / denom
    bound = np.linalg.norm(A, 2) * (1.0 + rtol) + atol
    if np.any(ratios > bound):
        raise AssertionError(
            f"Spektrálisnorma-sértés: max={ratios.max():.16f}, bound={bound:.16f}"
        )
    return float(np.max(ratios))

A = np.array([[3.0, 0.0],
              [4.0, 0.0]], dtype=np.float64)
X = np.array([[1.0, 0.0],
              [0.0, 1.0],
              [1.0/np.sqrt(2.0), 1.0/np.sqrt(2.0)]], dtype=np.float64)

ratios = np.linalg.norm(X @ A.T, axis=1) / np.linalg.norm(X, axis=1)
assert np.allclose(ratios, [5.0, 0.0, 5.0 / np.sqrt(2.0)])
print(ratios)
print(ratio_and_check(A, X))
# A futtatott kimenet:
# [5.         0.         3.53553391]
# 5.0
# Az első sor a három arány (5.0, 0.0 és 5/sqrt(2) ~= 3.5355339059327378),
# a második a függvény visszatérési értéke, a három arány maximuma.
```

A függvény a $\binom{3}{4}$ képet adó e_1 irányban éri el a 5-ös felső korlátot; az e_2 a magtérbe esik, aránya 0; a diagonális irány aránya $5/\sqrt{2}$. A toleranciák a lebegőpontos kerekítést fedik le, nem lazítják a matematikai korlátot.

3.4.10.4 F4 megoldása

A hibás függvény a nem szimmetrikus $J = \left(\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right)$ mátrixra a sajátértékeket vizsgálja. A két sajátérték 0, így `wrong_max_stretch(J)` a 0.0 értéket adná. A valódi spektrális norma $\sigma_1 = 1$, mert $Je_2 = e_1$ egységvektor. A hiba oka: a sajátértékek a sajátirányok menti skálázást mérik, de nem szimmetrikus mátrixnál a sajátirányok nem alkotnak ortonormált bázist, ezért a legnagyobb nyújtás nem olvasható le belőlük. Itt a J nilpotens, minden sajátértéke 0, mégis $\sigma_1 = 1$, mert $Je_2 = e_1$. Általában $\sigma_{\max}(A) \geq |\lambda|_{\max}$: ha $Av = \lambda v$ és $\|v\|_2 = 1$, akkor $|\lambda| = \|Av\|_2 \leq \sigma_{\max}(A)$.

Egyenlőség például normális (így szimmetrikus) mátrixokra áll fenn. Következmény: a sajátérték-alapú becslés a tényleges maximális nyújtást alulbecsülheti; a spektrális norma a normakorlát révén minden irányra érvényes felső korlátot ad.

Javítás:

```
def correct_max_stretch(A):
    import numpy as np
    return float(np.linalg.svd(A, compute_uv=False)[0])
```

Visszaellenőrzés: `correct_max_stretch(J)` eredménye 1.0, a `np.linalg.svd(J, compute_uv=False)` szinguláris értékei $[1, 0]$.

3.4.10.5 F5 megoldásminta

Egy lehetséges protokoll:

- **Kérdés:** a centrált adatmátrix első főkomponensére vetített egyváltozós prediktor jobb-e, mint a generátor által ismert jelirány?
- **Megfigyelési egység:** egy szimulált rekord; a rekord két bemeneti koordinátát és egy folytonos célt tartalmaz.
- **Generáló séma:** seed `20261020`; $N = 2000$; $t_i, u_i \sim N(0,1)$ független, $\eta_i \sim N(0, 0.1^2)$; $x_{i1} = 10u_i$, $x_{i2} = 3t_i$, $y_i = 3t_i + \eta_i$. Így $\mathrm{Var}(x_{i1}) = 100$ és $\mathrm{Var}(x_{i2}) = 9$, összhangban a fejezet Gyakori tévedés ellenpéldával szakaszának egységvarianciás feltevésével.
- **Split:** 60% train, 20% valid, 20% test, azonos seeddel; folytonos cél esetén egyszerű véletlen részekre bontás elég, mert nincs osztálycsoport, de a véletlen sorrendet rögzíteni kell.
- **PCA:** csak a train adat oszlopátlagával centrált mátrixból szabad a főtengelyt kiszámítani; ugyanezt a train-átlagot és főtengelyt kell a valid/test adatra alkalmazni.
- **Prediktorok:** $p_{\mathrm{PCA},i} = x_i^\top v_1$ az első főtengely menti kód; $p_{\mathrm{jel},i} = x_{i2}$ a generátor által ismert, adatból nem becsült **orákulum-irány**. Ez utóbbi felső viszonyítási pont, nem tanult modell.
- **Baseline:** intercept-only átlagprediktor, amely mindig a train célátlagát adja.
- **Metrika:** a least squares illesztést csak a train adaton végezzük, majd a valid és a teszt halmazon számolunk MSE-t és R^2-et. A baseline train-R^2-je pontosan 0, mert a train célátlagát használja; a teszt-R^2 negatív is lehet, mert a train-átlag eltérhet a tesztátlagtól.
- **Következtetési határ:** a generátor szerint $v_1 \approx e_1$ várható, a p_{PCA} lényegében zaj, tehát a gyenge PCA-eredmény a konstrukció következménye, nem általános tétel minden adatra. Egy seed és egy séma nem globális PCA-bizonyíték; a protokoll ezt a konkrét „nagy variancia, de címke-irreleváns” jelenetet ellenőrzi.

Az önellenőrzés válaszai. Az elsőnél az $Av = \lambda v$ alak-egyezése dönt: $Av \in \mathbb{R}^3$, $\lambda v \in \mathbb{R}^2$, ez csak $m = n$ esetben lehet általánosan értelmes. A másodiknál $\sigma_{\min} = 0$, a mátrix ranghiányos, így az együttható nem egyértelmű; a predikció ettől még lehet egyértelmű, ahogy 3.3. Vetítés, legkisebb négyzetek és kondicionáltság tárgyalta.

3.4.11 Rövid összefoglaló

A szinguláris érték-felbontás minden valós téglalapmátrixra ortonormált U, V mátrixokkal adja az $A = U\Sigma V^\top$ alakot. A pozitív σ_i értékek száma a rang, a legnagyobb σ_1 a spektrális norma. A diagonális operátorra közvetlenül beláttuk, hogy maximális nyújtása a legnagyobb $|d_i|$, majd a teljes SVD-tétel megnevezett létezési és ortonormáltsági állításából levezettük, hogy tetszőleges A-ra $\|A\|_2 = \sigma_{\max}$. Ebből következik a $\|Ax\|_2 \le \sigma_1 \|x\|_2$ normakorlát és a 3.3. Vetítés, legkisebb

négyzetek és kondicionáltság alfejezetben megelőlegezett $\kappa_2(A) = \sigma_1/\sigma_{\min}$ definíció. Az alacsony rangú A_k közelítés a legnagyobb szinguláris irányokat tartja meg, ez a PCA-rekonstrukció alapja.

3.4.12 Továbblépés: függvény, határérték és derivált

A 4.1. Függvény, határérték és derivált a függvény-határértéket és a deriváltat vezeti be; ott a lineáris közelítés érzékenységét, majd a 4.2.–4.3. a gradienst és a vektor–Jacobi-szorzatot tárgyalja.

3.4.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- Eckart, C., Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

4 Kalkulus, láncszabály és vektor–Jacobi-szorzat

4.1 Függvény, határérték és derivált

4.1.1 Cél, előismeret és induló kérdés

Ez az egység a valós függvények differenciálszámításának azt a minimális részét építi fel, amelyre a neurális hálózatok tanításának matematikája épül. A 3.1. Vektor, norma és skalárszorzat alfejezetben bevezetett távolságfogalmat használjuk: a differenciahányados nevezője két pont távolsága, a számláló pedig a hozzájuk tartozó függvényértékek különbsége. A 3.4. Sajátérték, SVD és spektrális határ alfejezetből most csak annyit használunk fel, hogy a lineáris leképezés a későbbi többdimenziós derivált központi gondolata; ebben az alfejezetben minden objektum skalár–skalár függvény.

Induló becslés: egy képfeldolgozó eljárás a 0,31 bemenetre 0,252 értéket ad, a 0,32 bemenetre pedig 0,261 értéket. A bemenet egy normalizált intenzitás a $[0, 1]$ intervallumból, a kimenet az eljárás dimenzió nélküli pontszáma; mindkét mennyiség dimenzió nélküli, azonos skálán él, ezért a különbségüket közvetlenül lehet viszonyítani. Körülbelül mennyit változik a kimenet, ha a bemenetet 0,31-ről 0,313-ra állítjuk? A válaszhoz a változás és a bemenetváltozás hányadosát kell értelmeznünk.

4.1.2 A differenciahányados és a derivált definíciója

Legyen $f : I \subseteq \mathbb{R} \to \mathbb{R}$ valós függvény, és legyen $a \in I$ az I tartomány **belső pontja**, azaz valamely $\delta > 0$ esetén az $(a - \delta, a + \delta)$ intervallum is I-ben van. Minden olyan $h \neq 0$ lépésközre, amelyre $a + h \in I$, definiáljuk a **differenciahányadost** (difference quotient):

$$D_h f(a) = \frac{f(a + h) - f(a)}{h}.$$

A kifejezés az $(a, f(a))$ és $(a+h, f(a+h))$ pontokon átmenő szelő meredeksége. A h lehet negatív is; ekkor a szelő a bal oldali pont felé mutat.

A **derivált** (derivative) az a pontban a differenciahányados határértéke, ha ez a véges határérték létezik:

$$f'(a) = \lim_{h \to 0} D_h f(a) = \lim_{h \to 0} \frac{f(a + h) - f(a)}{h}.$$

Pontosabban: $f'(a) = L \in \mathbb{R}$, ha minden $\varepsilon > 0$ számhoz van olyan $\delta > 0$, hogy minden $0 < |h| < \delta$, $a + h \in I$ esetén $|D_h f(a) - L| < \varepsilon$. A $0 < |h|$ feltétel fontos: a differenciahányados $h = 0$-ban nincs értelmezve, a határérték mégis közelíthető mindkét oldalról. Ha a határérték nem létezik vagy nem véges, akkor f az a pontban **nem differenciálható**.

Jelölésmegjegyzés: az értelmezési tartományt ebben az egységben I jelöli, hogy ne keveredjen a D_h differencia operátorral; a δ a határérték-definíció sugara, a középső differenciát pedig a

későbbi szakaszban Δ_h jelöli. A Taylor-maradék függvényét $\eta(h)$-val írjuk, így az ε csak a határérték-tűrést jelenti.

Az induló kérdésben a két adat közötti differenciahányados

$$\frac{0{,}261 - 0{,}252}{0{,}32 - 0{,}31} = \frac{0{,}009}{0{,}01} = 0{,}9.$$

Ha feltesszük, hogy a kimenet közelítőleg ezzel a lokális sebességgel változik, akkor a 0,313 bemenethez tartozó becsült kimenet

$$0{,}252 + 0{,}9 \cdot (0{,}313 - 0{,}31) = 0{,}252 + 0{,}0027 = 0{,}2547.$$

Ez a gondolatmenet pontosan a derivált elsőrendű közelítését használja; a hibáját a későbbi, „Taylor első rendű közelítése” pontban vizsgáljuk.

4.1.3 Folytonosság és differenciálhatóság viszonya

Az f **folytonos** az a pontban, ha $\lim_{x \to a} f(x) = f(a)$. Szavakkal: elég kis bemenetváltozásra a kimenet is tetszőlegesen kis változást mutat. A folytonosság gyengébb feltétel, mint a differenciálhatóság.

Tétel (differenciálhatóságból következik a folytonosság). Legyen a az I belső pontja. Ha f differenciálható az a pontban, akkor f folytonos is a-ban.

Bizonyítás. A különbséget szorzattá alakítjuk. Minden $x \neq a$ esetén, amelyre a hányados értelmes,

$$f(x) - f(a) = \frac{f(x) - f(a)}{x - a} \cdot (x - a).$$

Ha $x \to a$, akkor az első tényező a definíció szerint $f'(a)$-hoz tart, a második pedig 0-hoz. A szorzat határértéke a két véges határérték szorzata, tehát

$$\lim_{x \to a} \bigl(f(x) - f(a)\bigr) = f'(a) \cdot 0 = 0.$$

Ebből $\lim_{x \to a} f(x) = f(a)$, vagyis f folytonos a-ban. Az $x = a$ eset triviálisan teljesíti az egyenlőséget. Az állítást beláttuk.

A megfordítás nem igaz. Az $f(x) = |x|$ függvény folytonos 0-ban, de nem differenciálható ott. Valóban, $h > 0$ esetén

$$D_h f(0) = \frac{|h| - |0|}{h} = \frac{h}{h} = 1,$$

míg $h < 0$ esetén

$$D_h f(0) = \frac{|h|}{h} = \frac{-h}{h} = -1.$$

A jobb és bal oldali hányados különböző, így a kétoldali határérték nem létezik.

4.1.4 Levezetés: a négyzetfüggvény deriváltja

A levezetést a $q : \mathbb{R} \to \mathbb{R}$, $q(x) = x^2$ függvényre végezzük el, közvetlenül a definícióból. Rögzített $a \in \mathbb{R}$ és $h \neq 0$ esetén

$$\begin{aligned} D_h q(a) &= \frac{(a+h)^2 - a^2}{h} \\ &= \frac{a^2 + 2ah + h^2 - a^2}{h} \\ &= \frac{2ah + h^2}{h} \\ &= 2a + h. \end{aligned}$$

Az utolsó lépésben $h \neq 0$ miatt egyszerűsíthetünk. A $2a + h$ kifejezés $h \to 0$ esetén a $2a$ értékhez tart, mert a konstans $2a$-hoz egy nullához tartó tagot adunk. Ezért

$$q'(a) = \lim_{h \to 0} (2a + h) = 2a.$$

Mivel az a tetszőleges volt, a deriváltfüggvény $q'(x) = 2x$ minden $x \in \mathbb{R}$ pontban. A levezetés feltétele pusztán $h \neq 0$; a négyzetfüggvény mindenütt differenciálható.

Ellenőrzésként az $a = 3$ pontban a valódi derivált $q'(3) = 6$. A $h = 0{,}1$ differenciahányados:

$$\frac{3{,}1^2 - 3^2}{0{,}1} = \frac{9{,}61 - 9}{0{,}1} = 6{,}1.$$

A $h = 0{,}01$ hányados 6,01; a $2a + h$ képlet éppen ezt adja. A hiba a fenti levezetésből pontosan h.

4.1.5 Taylor első rendű közelítése

A derivált definíciójának átrendezésével kapjuk a lokális lineáris közelítést.

Lemma (elsőrendű Taylor-maradék). Legyen a az I belső pontja. Ha f differenciálható a-ban, akkor megadható olyan, a 0 egy környezetén értelmezett $\eta(h)$ függvény, amelyre $\eta(0) = 0$, $\eta(h) \to 0$ $(h \to 0)$, és minden elég kis $|h|$ esetén

$$f(a + h) = f(a) + f'(a)h + h\eta(h).$$

Bizonyítás. $h \neq 0$-ra, $a + h \in I$ esetén legyen

$$\eta(h) = D_h f(a) - f'(a) = \frac{f(a+h) - f(a)}{h} - f'(a),$$

és legyen $\eta(0) = 0$. A derivált definíciója szerint $\eta(h) \to 0$, ha $h \to 0$; ezzel η a nullában is folytonos. Ezt átrendezve:

$$\frac{f(a+h) - f(a)}{h} = f'(a) + \eta(h),$$

majd h-val szorozva és $f(a)$-t mindkét oldalhoz adva a kívánt alakot kapjuk minden $h \neq 0$-ra; a $h = 0$ esetben mindkét oldal $f(a)$, összhangban az $\eta(0) = 0$ választással.

A maradéktag rendjének leírásához vezessük be a következő jelölést: $r(h) = o(h)$, ha $r(h)/h \to 0$, amint $h \to 0$. Itt $h\eta(h)/h = \eta(h) \to 0$, tehát $h\eta(h) = o(h)$.

Az **elsőrendű Taylor-közelítés** ennek a maradéktag elhagyásával kapott lineáris függvénye:

$$T_1(x) = f(a) + f'(a)(x - a).$$

A négyzetfüggvénynél $a = 3$ esetén $T_1(x) = 9+6(x-3)$. Például $x = 3{,}1$-re $T_1(3{,}1) = 9+6\cdot 0{,}1 = 9{,}6$, míg a pontos érték $3{,}1^2 = 9{,}61$. A hiba $0{,}01 = h^2$, és ez a négyzetfüggvényre minden h-ra pontosan így alakul, mert

$$(3 + h)^2 = 9 + 6h + h^2.$$

Tétel (másodrendű Taylor-maradék). Ha f az a egy környezetében kétszer folytonosan differenciálható, és ott $|f''| \leq M$, akkor minden elég kis $|h|$-ra

$$\bigl|f(a + h) - f(a) - f'(a)h\bigr| \leq \frac{M}{2}h^2.$$

Ez a Lagrange-féle maradéktag standard következménye; a bizonyítást itt nem ismételjük, a Taylor-polinom tárgyalását lásd a [B01] 5.1.1. szakaszában. A két állítás címkéje a szerepüket tükrözi: az elsőrendű alak a differenciálhatóság definíciójának átfogalmazása (Lemma), a másodrendű becslés feltételes állítás (Tétel).

4.1.6 Numerikus lépésköz és a kerekítési határ

A számítógépes gyakorlatban a deriváltat gyakran véges differenciákkal közelítjük. Az **előre mutató** (forward) differencia a már ismert $D_h f(a)$. A **középső** (central) differencia:

$$\Delta_h f(a) = \frac{f(a + h) - f(a - h)}{2h}.$$

A $q(x) = x^2$ függvényre a középső differencia valós aritmetikában **minden** $h \neq 0$-ra pontosan $2a$, mert

$$\frac{(a + h)^2 - (a - h)^2}{2h} = \frac{4ah}{2h} = 2a.$$

A forward differencia hibája ezzel szemben h, amint fent láttuk. Ez a **csonkolási** (truncation) hiba.

A lebegőpontos számítás azonban egy második hibaforrást is bevezet. Jelölje $u = 2^{-53} \approx 1{,}11 \cdot 10^{-16}$ a float64 egységnyi kerekítési hibakorlátját (unit roundoff), azaz azt a legnagyobb relatív hibát, amelyet egyetlen kerekítés okozhat; ez a gépi epszilon ($\varepsilon_{\text{gép}} \approx 2{,}22 \cdot 10^{-16}$) fele, és nem azonos a határérték-definíció ε betűjével. Ha $f(a) \approx 9$ nagyságrendű, akkor $f(a + h)$ és $f(a - h)$ kiértékelése egyenként legfeljebb körülbelül $u|f(a)|$ abszolút hibát hordozhat. A kivonás után a két hiba legfeljebb összeadódik, így a számláló abszolút hibája legfeljebb $2u|f(a)|$. A középső differencia $2h$ nevezőjével osztva ($h > 0$) a kerekítési eredetű deriválthiba

$$E_{\text{kerek}}(h) \approx \frac{2u|f(a)|}{2h} = \frac{u|f(a)|}{h},$$

a forward differencia h nevezőjével pedig $2u|f(a)|/h$.

Ez h csökkenésével nő. A forward differencia teljes hibája a $q = x^2$ esetben közelítőleg

$$E_{\text{forward}}(h) \approx |h| + \frac{2u|f(a)|}{h}.$$

Két ellentétes tag összege: a csonkolási hiba h-val csökken, a kerekítési hiba $1/h$-val nő. A minimum nagyságrendileg ott van, ahol a két tag egyenlő:

$$h \approx \sqrt{2u|f(a)|}.$$

$a = 3$ esetén ez körülbelül $\sqrt{2 \cdot 1{,}11 \cdot 10^{-16} \cdot 9} \approx 4{,}5 \cdot 10^{-8}$. Általános sima függvényre a középső differencia csonkolási hibája h^2 rendű, a kerekítési tag pedig $u|f(a)|/h$ alakú. A két tag nagyságrendi kiegyenlítése $h^2 \approx u|f(a)|/h$, azaz $h^3 \approx u|f(a)|$, amiből $h \approx (u|f(a)|)^{1/3}$. Az $|f(a)| \approx 1$ egységszintű skálán ez $u^{1/3} \approx 4{,}8 \cdot 10^{-6}$. Ezek **heurisztikus nagyságrendbecslések**, nem univerzális képletek: a konstans függ az f nagyságától, a deriváltak méretétől és a hardveres kerekítési módtól.

A szélsőséges eset jól mutatja a korlátot. Ha $a = 3$ és $h = 10^{-16}$, akkor float64-ben a $3 + 10^{-16}$ összeg a 3 legközelebbi lebegőpontos számára kerekedik, mert a 3 körüli két ábrázolható szám távolsága körülbelül $4{,}44 \cdot 10^{-16}$. Ekkor $f(a+h) - f(a) = 9 - 9 = 0$, így mind a forward, mind a középső differencia 0-t ad a valódi 6 helyett. A tetszőlegesen kis lépésköz tehát **nem** garantál egyre pontosabb deriváltat.

4.1.7 Kézi példa

Tekintsük a $q(x) = x^2$ függvényt az $a = 3$ pontban. A valódi derivált 6. Kézi számítással:

- $h = 1$: forward: $(16 - 9)/1 = 7$, hiba 1; középső: $(16 - 4)/2 = 6$, hiba 0.
- $h = 0{,}1$: forward: $(9{,}61 - 9)/0{,}1 = 6{,}1$, hiba 0,1; középső: $(9{,}61 - 8{,}41)/0{,}2 = 6$, hiba 0.
- $h = 0{,}01$: forward: 6,01, hiba 0,01; középső: 6, hiba 0.

Valós aritmetikában a középső differencia a négyzetfüggvényre mindig 6-ot ad. A forward differencia hibája éppen h. A float64-ben a nagyon kis h-nál a fenti kerekítési határ érvényesül: $h = 10^{-16}$ esetén a függvényértékek numerikusan egybeesnek, és a deriváltbecslés 0 lesz.

4.1.8 Labor: hibagörbe és ellenőrző kód

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A következő önálló, CPU-s Python munkamenet a `lab/n013_numerical_derivative.py` fájlba menthető. A kód nem használ véletlen számot, ezért seedre (a véletlenszám-generátor kiinduló értékére) és GPU-ra (grafikus feldolgozóegységre) nincs szükség; a futás CPU-n történik. A Python `float` típusa alapértelmezés szerint float64, ezt a kommentben rögzítjük. Futtatás: `python n013_numerical_derivative.py`.

```python
"""n013 labor: forward és centrális differencia hibagörbéje q(x)=x^2-re.

Futtatás: python n013_numerical_derivative.py
Környezet: CPU, Python 3, float64; nincs véletlen és nincs GPU.
"""
def q(x):
    """A vizsgált négyzetfüggvény."""
    return x * x

def forward_diff(f, x, h):
    """Előre mutató differenciahányados, h > 0."""
    return (f(x + h) - f(x)) / h

def central_diff(f, x, h):
    """Középső differencia, h > 0."""
    return (f(x + h) - f(x - h)) / (2.0 * h)

def main():
    x0 = 3.0
    exact = 2.0 * x0
    steps = [1.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5,
             1e-6, 1e-8, 1e-10, 1e-12, 1e-16]
    print(f"{'h':>9} {'forward':>14} {'central':>14} "
          f"{'err_forward':>14} {'err_central':>14}")
    for h in steps:
        fwd = forward_diff(q, x0, h)
        cen = central_diff(q, x0, h)
        print(f"{h:9.1e} {fwd:14.10e} {cen:14.10e} "
              f"{abs(fwd-exact):14.6e} {abs(cen-exact):14.6e}")

    # Nevezetes értékek ellenőrzése a definícióból (nem a kiírt táblázatból):
    assert abs(forward_diff(q, x0, 1.0) - 7.0) < 1e-12
    assert abs(central_diff(q, x0, 1.0) - 6.0) < 1e-12
    assert abs(forward_diff(q, x0, 1e-2) - 6.01) < 1e-9
    assert abs(central_diff(q, x0, 1e-2) - 6.0) < 1e-9
    assert forward_diff(q, x0, 1e-16) == 0.0
    assert central_diff(q, x0, 1e-16) == 0.0

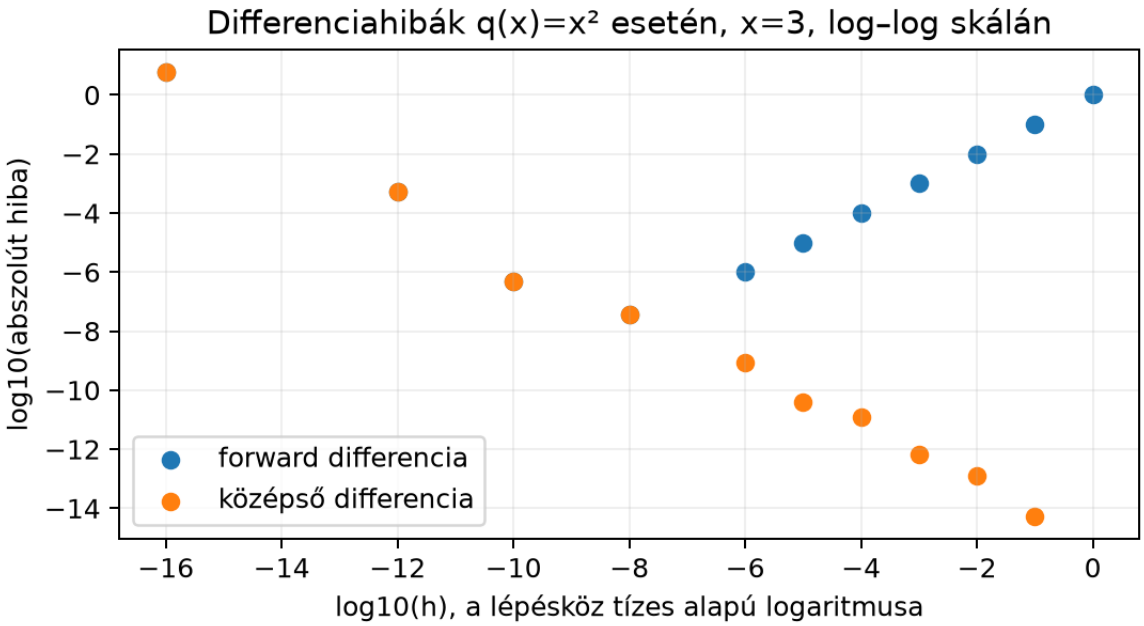
if __name__ == "__main__":
    main()
```

A fenti kód mért kimenete (Python 3.12.3, NumPy 2.5.4, float64, CPU, egyszálú futás: OMP/OpenBLAS/MKL=1; a kód nem használ véletlen számot):

```
        h        forward        central    err_forward    err_central
  1.0e+00 7.0000000000e+00 6.0000000000e+00   1.000000e+00   0.000000e+00
  1.0e-01 6.1000000000e+00 6.0000000000e+00   1.000000e-01   5.329071e-15
  1.0e-02 6.0100000000e+00 6.0000000000e+00   1.000000e-02   1.278977e-13
  1.0e-03 6.0010000000e+00 6.0000000000e+00   1.000000e-03   6.608047e-13
  1.0e-04 6.0001000000e+00 6.0000000000e+00   1.000000e-04   1.266187e-11
  1.0e-05 6.0000100000e+00 6.0000000000e+00   9.999951e-06   3.930634e-11
  1.0e-06 6.0000010009e+00 6.0000000008e+00   1.000928e-06   8.386678e-10
  1.0e-08 5.9999999635e+00 5.9999999635e+00   3.646483e-08   3.646483e-08
  1.0e-10 6.0000004964e+00 6.0000004964e+00   4.964422e-07   4.964422e-07
  1.0e-12 6.0005334035e+00 6.0005334035e+00   5.334035e-04   5.334035e-04
  1.0e-16 0.0000000000e+00 0.0000000000e+00   6.000000e+00   6.000000e+00
```

A 10^{-16}-os sor a float64 kerekítés következménye: $3 + 10^{-16}$ a legközelebbi ábrázolható számra kerekedik, ezért a számláló nulla, és a becslés a valódi 6 helyett 0-t ad.

A hibagörbe szerkezete log–log ábrán, mindkét differenciára. A pontok a fenti mért táblázat hibasoraiból számított $\log_{10}$ értékek, tehát a szkript kimenetéből reprodukálhatók:



4.1. ábra: Differenciahibák q(x)=x² esetén, x=3, log–log skálán

A fenti mért kimeneti táblázat hibasoraiból számított log10 értékek; a középső differencia h=1-nél pontosan 0 hibát ad, ez logaritmikus ábrán nem ábrázolható, ezért az a pont kimarad.

Az értelmezés: a forward hiba $h = 10^{-6}$-ig közel lineárisan csökken (a csonkolási tag h nagyságrendű), majd a kerekítési hiba miatt nő; a log–log ábrán ez a csonkolási szakaszban $+1$, a kerekítési szakaszban -1 meredekségű egyenesként látszik. A $q(x) = x^2$ esetén a középső differencia valós aritmetikában pontos, a csonkolási tag nulla, ezért a hibáját gyakorlatilag csak a kerekítés adja: a h csökkenésével nagyjából -1 meredekséggel nő, és a 10^{-16} lépésnél a kimenet 0, a hiba pedig a teljes derivált értéke (6).

4.1.9 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a differenciahányadost és a deriváltat. Magyarázza el, miért következik a differenciálhatóságból a folytonosság, és adjon olyan függvényt, amely folytonos, de nem differenciálható. A példán számítsa ki a jobb és bal oldali differenciahányadosokat.

F2 (levezetési, nehézség 2). Vezesse le a $c(x) = x^3$ függvény deriváltját a definícióból. Adja meg a deriváltat az $x = 2$ pontban, majd az elsőrendű Taylor-közelítést $h = 0{,}1$ esetén. Számítsa ki a pontos függvényértéket, a közelítést és a különbséget.

F3 (implementációs, nehézség 1). Írjon Python-függvényt, amely az f függvényhez, az x ponthoz és a h lépésközhöz visszaadja a forward és a középső differenciát. Tesztelje a $q(x) = x^2$, $x = 3$, $h = 10^{-2}$ bemenettel. Ellenőrizze, hogy a forward érték 6,01-hez, a középső érték 6-hoz van 10^{-12} abszolút tűrésen belül.

F4 (hibakeresési, nehézség 2). Egy hallgató a deriváltat a lehető legkisebb h-val akarja pontosabbá tenni, ezért $h = 10^{-30}$-at ír a kódba. A $q(x) = x^2$, $x = 3$ esetben a függvény 0,0 deriváltat ad vissza. Lokalizálja a hiba okát, mondja meg a következményt, javítsa a lépésközt, és ellenőrizze $h = 10^{-5}$ esetén a javított eredményt.

F5 (kísérlettervezési, nehézség 3). Tervezzen kísérletet egy általánosabb, de zárt alakban deriválható függvény, például $f(x) = \sin x$, $x = 1$ esetén annak meghatározására, hogy a középső differencia mely h értéknél adja a legkisebb abszolút hibát a $\cos 1$ referenciaértékhez képest. Adja meg a h-rácsot, a metrikát, a döntési szabályt, az ismétlés szükségességét és az érvényességi korlátokat. Nem egyetlen előre megadott „helyes” h-t várunk, hanem korrekt protokollt.

4.1.10 Megoldások és indoklás

F1. A definíciókat a fenti szakasz tartalmazza. A differenciálhatóságból a folytonosság a szorzat határértékén keresztül következik; a bizonyítás a tételnél olvasható. Ellenpélda: $f(x) = |x|$, folytonos 0-ban, mert $\lim_{x\to 0} |x| = 0 = |0|$. A jobb oldali differenciahányados $h > 0$-ra 1, a bal oldali $h < 0$-ra -1. A két oldali határérték különbözik, ezért a derivált nem létezik.

F2. Rögzített a-ra és $h \neq 0$-ra:

$$\begin{aligned} D_h c(a) &= \frac{(a+h)^3 - a^3}{h} \\ &= \frac{a^3 + 3a^2 h + 3ah^2 + h^3 - a^3}{h} \\ &= \frac{3a^2 h + 3ah^2 + h^3}{h} \\ &= 3a^2 + 3ah + h^2. \end{aligned}$$

$h \to 0$ esetén a $3ah$ és h^2 tagok nullához tartanak, tehát $c'(a) = 3a^2$. Az $a = 2$ pontban $c'(2) = 12$. Az elsőrendű Taylor-közelítés:

$$T_1(2 + h) = c(2) + c'(2)h = 8 + 12h.$$

$h = 0{,}1$-re $T_1(2{,}1) = 8 + 1{,}2 = 9{,}2$. A pontos érték $2{,}1^3 = 9{,}261$, a hiba $0{,}061$. A hiba összetevői az elhagyott $3ah^2 + h^3 = 6 \cdot 0{,}01 + 0{,}001 = 0{,}061$ tagok.

F3. Az előírt függvény:

```
def q(x):
    return x * x

def numdiff(f, x, h):
    forward = (f(x + h) - f(x)) / h
    central = (f(x + h) - f(x - h)) / (2.0 * h)
    return forward, central

fwd, cen = numdiff(q, 3.0, 1e-2)
print(fwd, cen)  # várhatóan 6,01 és 6,0 körül
assert abs(fwd - 6.01) < 1e-12
assert abs(cen - 6.0) < 1e-12
```

A `h` pozitív, az `x` float64 skalár. A középső differencia a q-ra valós aritmetikában pontosan $2x$; float64-ben a mért hiba $h = 10^{-2}$-nél $1{,}28 \cdot 10^{-13}$ a 6-hoz, illetve $1{,}51 \cdot 10^{-13}$ a forward érték 6,01-hez képest. A 10^{-12}-es tűrés ennek mintegy hétszerese: elegendő tartalék a platformfüggő kerekítésre, de valódi eltérést nem enged át.

F4. A hiba oka: $h = 10^{-30}$ esetén float64-ben $3 + h$ a legközelebbi ábrázolható számra, gyakorlatilag 3,0-ra kerekedik, mert a 3 körüli gépi felbontás nagyjából $4{,}44 \cdot 10^{-16}$. Ezért $q(3 + h) - q(3) = 0$, és a hányados 0. A következmény: a derivált becslése a valódi 6 helyett 0,

vagyis a hiba 100%-os. Javítás: válasszunk mérsékelt lépésközt, például $h = 10^{-5}$-öt. A 4.1.6. szakasz becslése szerint a forward differencia optimális lépésköze $h \approx \sqrt{2u|f(a)|} \approx 4{,}5 \cdot 10^{-8}$; a 10^{-5} tehát nem optimum, hanem tudatosan mérsékelt választás: a $3 + h$ még ábrázolható, a kerekítési tag $2u|f(a)|/h \approx 2{,}0 \cdot 10^{-10}$, a csonkolási tag pedig 10^{-5} nagyságrendű, így a példa a két hiba egyensúlyát láthatóvá teszi. Ekkor $q(3{,}00001) - q(3)$ kiszámítható, és a forward differencia körülbelül 6,00001; a középső differencia 6-hoz közeli. A visszaellenőrzés: az F3 függvényével $h = 10^{-5}$-re a forward hiba $9{,}999951 \cdot 10^{-6}$, a középső hiba $3{,}93 \cdot 10^{-11}$; ez utóbbi a kerekítési tag $u|f|/h \approx 10^{-10}$ nagyságrendje miatt nem teljesíti a 10^{-12}-es tűrést, ezért $h = 10^{-5}$ szemléltetésre jó, szigorú toleranciás tesztre nem.

F5. Egy lehetséges protokoll. Cél: h hatásának feltérképezése a középső differencia hibájára. Megfigyelési egység: egy (f, x, h) hármas egyetlen determinisztikus kiértékelése; a mért mennyiség a deriváltbecslés abszolút hibája, nem valószínűségi változó. Mivel nincs mintavételi zaj, ismétlés nem szükséges; a robusztusságot több x ponton lehet ellenőrizni. Várható nagyságrend: a 4.1.6. szakasz két hibatagjának kiegyenlítésével $f(x) = \sin x$, $x = 1$ esetén $h \approx 1{,}0 \cdot 10^{-5}$ körül várható a minimum, amelynek értéke nagyjából $9 \cdot 10^{-12}$; ez becslés, nem mért érték, és a rácstól, valamint a tényleges kerekítési módtól függ. Rács: geometriai sorozat, például $h \in \{10^{-1}, 10^{-2}, \ldots, 10^{-16}\}$, a várható optimum környékén, 10^{-4} és 10^{-7} között sűrítve. Metrika: $|\Delta_h f(1) - \cos 1|$, a referencia a zárt alakú derivált. Döntés: azt a h-t jelentjük, amely a rácson a legkisebb hibát adja, és mellette megadjuk a hibát a szomszédos h értékeken is, ne csak egyetlen minimumot. Érvényességi korlát: az optimum függ az f skálájától, a hardveres kerekítéstől és a float64-től; más függvényre vagy más x pontra nem vihető át automatikusan. A mérést determinisztikus, seed nélküli CPU-kóddal lehet elvégezni, és a nyers hibasorozatot kell rögzíteni.

4.1.11 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: „minél kisebb a numerikus lépésköz, annál pontosabb a derivált”. A fenti $q(x) = x^2$, $x = 3$ példa cáfolja: $h = 10^{-16}$ esetén float64-ben $x + h = x$, ezért a számláló nulla, és a becsült derivált 0 a valódi 6 helyett. A hiba oka a kivonási jegyvesztés (catastrophic cancellation): két közel egyenlő szám kivonásakor a vezető jegyek kiesnek, ezért a különbség relatív hibája nagyon megnő; ezt a $2u|f(a)|/h$ nagyságrendű kerekítési tag írja le. A helyes szemlélet: a lépésközt a csonkolási és kerekítési hiba együttes optimuma körül kell választani, és az optimum függ a feladattól.

4.1.12 Önellenőrzés és visszatekintés

Ellenőrizze, hogy ki tudja mondani a derivált ε–δ definícióját, le tudja vezetni x^2 deriváltját, és el tudja magyarázni, miért folytonos minden differenciálható függvény. Korábbi fogalom áttekintése: a 3.1. Vektor, norma és skalárszorzat alfejezetben a két pont távolságát a különbségvektor euklideszi normájaként definiáltuk; a differenciahányados nevezője éppen ilyen távolság a valós egyenesen, a számláló pedig a függvényértékek különbsége.

4.1.13 Rövid összefoglaló

A differenciahányados két függvényérték különbségét osztja a bemenetek különbségével, és a szelő meredekségét adja. A derivált ennek határértéke, ha a lépésköz nullához tart; a négyzetfüggvényre a definícióból $2x$ adódik. A differenciálhatóságból következik a folytonosság, fordítva nem, amit az abszolútérték-függvény mutat. Az elsőrendű Taylor-közelítés a lokális lineáris helyettesítést adja, $o(h)$ maradékkal. Numerikusan a túl kicsi lépésköz kivonási jegyvesztést okoz: a csonkolási hiba csökken, a kerekítési hiba nő, ezért van optimális h.

4.1.14 Továbblépés

A 4.2. Parciális derivált és gradiens a többváltozós függvények parciális deriváltját és gradiensét vezeti be. A 4.3. Jacobi-mátrix, láncszabály és VJP pedig a Jacobi-mátrixszal és a vektor–Jacobi-szorzattal (VJP, vector–Jacobian product: egy vektor és a Jacobi-mátrix szorzata) a backpropagation (hibavisszaterjesztés) pontos algebrai nyelvét építi fel; ekkor a skalár lépésköz helyett már az iránymenti deriváltakat vizsgáljuk.

4.1.15 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.

4.2 Parciális derivált és gradiens

4.2.1 Cél, előismeret és induló kérdés

A 4.1. Függvény, határérték és derivált egyetlen valós paraméter esetén értelmezte a deriváltat: a függvény lokális lineáris közelítésének meredekségét. A neurális hálózat tanítása során azonban a veszteség egyszerre sok súlytól függ, ezért azt kérdezzük: ha egy d dimenziós w paramétervektort egyszerre több irányban is elmozdíthatunk, melyik irányban változik a leggyorsabban a veszteség, és hogyan foglalható össze a változás egyetlen vektorban? A fejezet a parciális deriváltat, a gradienst, az iránymenti deriváltat és a többváltozós differenciálhatóságot építi fel. Előismeret: határérték és egyváltozós derivált a 4.1. Függvény, határérték és derivált alfejezetből, valamint a mátrixszorzás és az oszlopvektor-konvenció a 3.2. Mátrixszorzás és affín réteg alfejezetből. Induló becslésként gondoljuk végig: ha $f(w_1, w_2) = w_1^2 + 4w_2^2$, akkor a $(2, 1)$ pontból a $(-1, 1)$ irányba tett kis lépés vajon növeli vagy csökkenti az értéket? A válaszhoz mindkét koordináta menti változást össze kell kapcsolnunk; az iránymenti deriváltról szóló szakaszban visszatérünk a kérdésre, és kiszámítjuk a választ.

4.2.2 Parciális derivált

Legyen $U \subseteq \mathbb{R}^d$ nyílt halmaz, $f : U \to \mathbb{R}$, és $x \in U$. Az i-edik koordináta szerinti **parciális derivált** (partial derivative) az x pontban az egyváltozós derivált mintájára definiálható:

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x + he_i) - f(x)}{h},$$

ahol $e_i \in \mathbb{R}^d$ az i-edik standard bázisvektor, $h \in \mathbb{R} \setminus \{0\}$ pedig olyan, hogy $x + he_i \in U$. A parciális derivált tehát úgy vizsgálja a függvényt, hogy csak az i-edik koordinátát mozgatjuk, a többit rögzítjük. A $\partial f / \partial x_i(x)$, $D_i f(x)$ és $f_{x_i}(x)$ jelölések mind ugyanezt jelentik. Az eredmény skalár, mértékegysége a függvényérték és a koordináta mértékegységének hányadosa; a könyv későbbi fejezeteiben jellemzően „veszteségegység per súlyegység”.

1. példa. Legyen $f : \mathbb{R}^2 \to \mathbb{R}$, $f(w_1, w_2) = w_1^2 + 3w_1 w_2 + 2w_2^2$. Az első parciális derivált a w_2 rögzítése mellett a $w_1 \mapsto w_1^2 + 3w_1 w_2 + 2w_2^2$ egyváltozós függvény deriváltja:

$$\frac{\partial f}{\partial w_1}(w_1, w_2) = 2w_1 + 3w_2,$$

és hasonlóan

$$\frac{\partial f}{\partial w_2}(w_1, w_2) = 3w_1 + 4w_2.$$

A $(1, -1)$ pontban a két parciális derivált $\partial f/\partial w_1 = -1$ és $\partial f/\partial w_2 = -1$. Az első azt mondja meg, hogy csak a w_1 koordinátát mozgatva a függvény lokálisan -1 meredekséggel változik; a második ugyanezt mondja a w_2 irányra.

A parciális deriváltakból képzett vektort **gradiensnek** (gradient) nevezzük:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_d}(x) \end{pmatrix} \in \mathbb{R}^d.$$

A könyv oszlopvektor-konvenciót használ: a gradiens alakja megegyezik az input x alakjával. Ha a függvény argumentuma mátrix vagy magasabb dimenziós tömb (tenzor, azaz többdimenziós numerikus tömb), a gradiens a parciális deriváltaknak ugyanolyan alakú, komponensenkénti rendezett gyűjteménye. A PyTorch-ban a `.grad` mező alakja ezért megegyezik annak a tenzornak az alakjával, amelyre a skalár veszteséget deriváltuk [D05]. Az 1. példában $\nabla f(1, -1) = (-1, -1)^\top$.

4.2.3 Iránymenti derivált és kapcsolata a gradienssel

A parciális deriváltak csak a koordinátatengelyek menti változást írják le. Általános irányhoz rögzítsünk egy $v \in \mathbb{R}^d$, $v \neq 0$ vektort. A v irányú **iránymenti derivált** (directional derivative) az

$$D_v f(x) = \lim_{h \to 0} \frac{f(x + hv) - f(x)}{h}$$

határérték, ha létezik. Ez a $t \mapsto f(x + tv)$ egyváltozós függvény deriváltja a $t = 0$ pontban. Ha $\|v\| = 1$, az érték a v irányú meredekség; ha v nem egységvektor, a definíció arányosan nagyobb lépést vizsgál, ezért $D_{\alpha v} f(x) = \alpha D_v f(x)$ minden $\alpha \neq 0$-ra.

A gradiens és az iránymenti derivált kapcsolatát a következő lemma adja. A lemma feltételében szereplő differenciálhatóságot a következő szakasz definiálja részletesen; a bizonyítás közvetlenül erre a definícióra épül, ezért a lemma addig is követhető.

Lemma (iránymenti derivált a gradiensből). Ha f differenciálható az x pontban, akkor minden $v \in \mathbb{R}^d$ vektorra

$$D_v f(x) = \nabla f(x)^\top v.$$

Bizonyítás. A többváltozós differenciálhatóság definíciója szerint létezik olyan $\nabla f(x) \in \mathbb{R}^d$ vektor, amellyel

$$f(x + u) = f(x) + \nabla f(x)^\top u + r(u),$$

és a maradéktag teljesíti a $\lim_{u \to 0} r(u)/\|u\| = 0$ feltételt. Rögzítsünk egy $v \neq 0$ irányt, és helyettesítsünk $u = hv$-t:

$$\frac{f(x+hv)-f(x)}{h} = \nabla f(x)^\top v + \frac{r(hv)}{h}.$$

Mivel $\|hv\| = |h|\,\|v\|$,

$$\lim_{h\to 0} \frac{|r(hv)|}{|h|} = \lim_{h\to 0} \frac{|r(hv)|}{\|hv\|}\,\|v\| = 0,$$

tehát a differenciahányados határértéke $\nabla f(x)^\top v$. A $v = 0$ esetre mindkét oldal nulla. □

A lemma közvetlen következménye a legmeredekebb változás iránya. Ha $\|v\| = 1$, akkor a 3.1. Vektor, norma és skalárszorzat alfejezetben bevezetett Cauchy–Schwarz-egyenlőtlenséggel

$$D_v f(x) = \nabla f(x)^\top v \le \|\nabla f(x)\|\,\|v\| = \|\nabla f(x)\|,$$

és ha $\nabla f(x) \neq 0$, egyenlőség pontosan akkor áll, ha $v = \nabla f(x)/\|\nabla f(x)\|$. Ha $\nabla f(x) = 0$, akkor minden egységirányra $D_v f(x) = 0$: nincs kitüntetett legmeredekebb irány, a normalizált gradiens nem definiált (ez a stacionárius pont). A $\nabla f(x) \neq 0$ esetben a gradiens iránya tehát a legmeredekebb növekedés iránya, nagysága pedig az egységnyi hosszúságú lépésre jutó maximális növekedés. A negatív gradiens iránya a legmeredekebb csökkenésé; erre épül majd a gradienslépés a 6.1. Objektív, reguláris tag és gradienslépés alfejezetben.

Visszatérve az induló kérdésre: az $f(w_1, w_2) = w_1^2 + 4w_2^2$ függvény gradiense $\nabla f(w_1, w_2) = (2w_1, 8w_2)^\top$, tehát a $(2,1)$ pontban $\nabla f(2,1) = (4,8)^\top$. A $v = (-1,1)^\top$ irányú iránymenti derivált a lemma szerint $D_v f(2,1) = 4\cdot(-1) + 8\cdot 1 = 4 > 0$, ezért a függvényérték ebben az irányban nő. Mivel v nem egységvektor, az egységnyi hosszra átszámolt meredekség $4/\sqrt{2} \approx 2{,}83$, ami szintén pozitív; az irány tehát valóban növekedést ad, nem csökkenést.

4.2.4 Többváltozós differenciálhatóság

A parciális deriváltak puszta létezése nem elegendő ahhoz, hogy a fenti lineáris közelítés igaz legyen. A pontos definíció a következő. Az $f : U \to \mathbb{R}$ függvény **differenciálható** (differentiable) az $x \in U$ pontban, ha létezik olyan $a \in \mathbb{R}^d$ vektor, amellyel

$$f(x+u) = f(x) + a^\top u + r(u), \quad \text{és} \quad \lim_{u\to 0} \frac{r(u)}{\|u\|} = 0.$$

Ekkor az a vektor éppen a gradiens: $a = \nabla f(x)$. Szavakkal: a függvény jól közelíthető egy $a^\top u$ lineáris funkcionállal, és a hiba a $\|u\|$-nál gyorsabban tart nullához. A gradiens ennek a lineáris funkcionálnak a reprezentánsa a standard belső szorzatban.

Az a vektor egyértelmű. Ha a és a' is teljesíti a felbontást, akkor kivonással $(a-a')^\top u = o(\|u\|)$ adódik. Az $u = t(a-a')$, $t > 0$ választással $t\|a-a'\|^2 = o(t\|a-a'\|)$, amiből t-vel osztva $\|a-a'\|^2 = o(\|a-a'\|)$, tehát $a = a'$.

A gyakorlatban hasznos elégséges feltétel a következő.

Tétel (folytonos parciális deriváltak). Ha az f függvény minden parciális deriváltja létezik az x egy környezetében, és azok folytonosak az x pontban, akkor f differenciálható x-ben.

Bizonyításvázlat. Legyen u olyan kicsi, hogy az x-ből u irányába vezető, koordinátánként haladó töröttvonal az U-ban marad, és vezessük be az $x^{(0)} = x$, $x^{(k)} = x + \sum_{i=1}^{k} u_i e_i$ köztes pontokat. Ekkor

$$f(x+u)-f(x)=\sum_{k=1}^{d}\bigl(f(x^{(k)})-f(x^{(k-1)})\bigr).$$

A k-adik tagban csak a k-adik koordináta változik, ezért az egyváltozós középértéktétel szerint $f(x^{(k)})-f(x^{(k-1)})=\partial_k f(\xi^{(k)})\,u_k$ valamely, az $x^{(k-1)}$ és $x^{(k)}$ közötti $\xi^{(k)}$ pontra. Így

$$f(x+u)-f(x)-\nabla f(x)^{\top}u=\sum_{k=1}^{d}\bigl(\partial_k f(\xi^{(k)})-\partial_k f(x)\bigr)u_k.$$

A parciális deriváltak x-beli folytonossága miatt a $\xi^{(k)}$ pontok x-hez tartanak, amikor $u\to 0$, ezért a zárójeles tényezők együttesen nullához tartanak. Ha $\varepsilon(u)=\max_k|\partial_k f(\xi^{(k)})-\partial_k f(x)|$, akkor a maradék abszolút értéke legfeljebb $\varepsilon(u)\sum_k|u_k|\le\varepsilon(u)\sqrt{d}\,\|u\|$, és $\varepsilon(u)\to 0$; ez éppen a $\lim_{u\to 0} r(u)/\|u\|=0$ feltétel. A részletes, a többváltozós középértéktételre építő tárgyalás a [B01] többváltozós differenciálszámítási fejezeteiben olvasható. A fontos gyakorlati üzenet: ha a parciális deriváltakat zárt képletből, folytonos elemi műveletekkel kapjuk, a differenciálhatóság általában teljesül; a nem folytonos pontokban külön vizsgálat szükséges.

4.2.5 A kvadratikus veszteség gradiense

A neurális hálózatok tanításában gyakran előforduló kvadratikus veszteség a következő. Legyen $X\in\mathbb{R}^{N\times d}$ a batchelt (egy csomagban együtt feldolgozott) bemenet (N megfigyelés, d jellemző), $y\in\mathbb{R}^{N}$ a célvektor, $w\in\mathbb{R}^{d}$ a paramétervektor. A közönséges legkisebb négyzetek felezett négyzetösszege:

$$f(w)=\frac{1}{2}\|Xw-y\|_2^2=\frac{1}{2}\sum_{i=1}^{N}\Bigl((Xw)_i-y_i\Bigr)^2.$$

A $\frac{1}{2}$ konstans a későbbi gradiens egyszerűsítésére szolgál; a minimum helyét nem befolyásolja. Számítsuk ki a j-edik parciális deriváltat komponensenként. Az $(Xw)_i=\sum_{k=1}^{d}X_{ik}w_k$ lineáris kifejezés w_j szerinti deriváltja X_{ij}. Az egyváltozós láncszabály (amelyet többváltozós alakban a 4.3. Jacobi-mátrix, láncszabály és VJP formalizál) és az összeg deriválási szabálya szerint

$$\frac{\partial f}{\partial w_j}(w)=\sum_{i=1}^{N}\Bigl((Xw)_i-y_i\Bigr)X_{ij}=\Bigl(X^{\top}(Xw-y)\Bigr)_j.$$

Mivel ez minden $j=1,\dots,d$-re fennáll, a teljes gradiens:

$$\nabla f(w)=X^{\top}(Xw-y)\in\mathbb{R}^{d}.$$

Ez a levezetés a könyv saját kidolgozása; a mátrixderivált-szabályok ugyanezt adják. Az alakellenőrzés kulcsa: $X^{\top}\in\mathbb{R}^{d\times N}$, $(Xw-y)\in\mathbb{R}^{N}$, így a szorzat d dimenziós, pontosan a w alakja. A bias nélküli lineáris réteg veszteségének gradiense ezzel a képlettel számítható; a biasos affin esetben $f(w,b)=\frac{1}{2}\|Xw+b\mathbf{1}-y\|^2$ mellett $\nabla_w f=X^{\top}(Xw+b\mathbf{1}-y)$, a bias deriváltja viszont $\partial f/\partial b=\mathbf{1}^{\top}(Xw+b\mathbf{1}-y)$. Ugyanez adódik, ha X-et egy csupa egyesből álló oszloppal bővítjük, és b-t a súlyvektor utolsó komponensének tekintjük. A részleteket a 4.3. Jacobi-mátrix, láncszabály és VJP és 10.2. Affín réteg és aktiváció backward fejti ki.

4.2.6 Kontúrok és a gradiens merőlegessége

Egy $c \in \mathbb{R}$ szinthez tartozó **szintvonal** (level set) az

$$C_c = \{x \in U : f(x) = c\}$$

halmaz. Két paraméter esetén ezt a halmazt a grafikus ábrázolásban **kontúrvonalnak** (contour) is nevezik; magasabb dimenzióban a megfelelő fogalom a szintfelület. A továbbiakban egységesen szintvonalat írunk, és a kontúr szót csak az ábrázolás kapcsán említjük.

Lemma (gradiens merőleges a szintvonalra). Legyen f differenciálható, és legyen $\gamma : (a, b) \to C_c$ differenciálható görbe a szintvonalon, $\gamma(t_0) = x$. Ekkor

$$\nabla f(x)^\top \gamma'(t_0) = 0,$$

vagyis a gradiens merőleges a szintvonal x pontbeli érintőjére.

Bizonyítás. Mivel $\gamma(t) \in C_c$ minden t-re, $f(\gamma(t)) = c$ konstans. Deriváljuk mindkét oldalt t szerint a t_0 pontban. A láncszabály és az iránymenti derivált lemma szerint

$$0 = \frac{d}{dt} f(\gamma(t))\Big|_{t=t_0} = \nabla f(\gamma(t_0))^\top \gamma'(t_0).$$

Ez pontosan a merőlegességi állítás. $\square$

A lemma indokolja, miért mutat a gradiens a szintvonalakra merőlegesen: ha egy szintvonalon mozogva a függvényérték nem változik, akkor a leggyorsabb változás irányának merőlegesnek kell lennie erre a mozgásra. A gradiens iránya ezen felül — ha nem nulla — a növekvő függvényértékek felé mutat; ezt a legmeredekebb növekedésről szóló, előző szakaszbeli megfontolás adja. A gradiens nagysága szemléletesen a szintvonalak sűrűségével függ össze: ahol a szintvonalak sűrűbben futnak, ott egységnyi távolságon nagyobb a függvényérték változása. Ez a kép iránymutató, de önmagában nem tétel; a pontos állításokat a lemma és a $D_v f(x) = \nabla f(x)^\top v$ formula rögzíti.

4.2.7 Eredeti kézi példa és ellenőrzés

Tekintsük a

$$f(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 + 1)^2$$

kétparaméteres veszteséget. A parciális deriváltak:

$$\frac{\partial f}{\partial w_1} = 2(w_1 - 1), \qquad \frac{\partial f}{\partial w_2} = 4(w_2 + 1),$$

tehát a gradiens

$$\nabla f(w_1, w_2) = \begin{pmatrix} 2(w_1 - 1) \\ 4(w_2 + 1) \end{pmatrix}.$$

A $w^{(0)} = (2, -2)^\top$ pontban:

$$f(w^{(0)}) = (2-1)^2 + 2(-2+1)^2 = 1 + 2 = 3, \qquad \nabla f(w^{(0)}) = \begin{pmatrix} 2 \\ -4 \end{pmatrix}.$$

Ellenőrizzük először az iránymenti deriváltat a $v = (1, 0.25)^\top$ irányban. A lemmából $D_v f(w^{(0)}) = \nabla f(w^{(0)})^\top v = 2 \cdot 1 + (-4) \cdot 0.25 = 1$. Numerikusan, $h = 10^{-5}$ mellett a differenciahányados:

$$\frac{f(w^{(0)} + hv) - f(w^{(0)})}{h}$$

a mért közelítés 1,00001125 (NumPy 2.5.4, float64, $h = 10^{-5}$). A labor F3 feladata nem ezt az iránymenti deriváltat, hanem a gradiens centrális differenciás ellenőrzését végzi el, ezért a kettőt ne azonosítsuk.

Végezzünk egy gradienslépést $\eta = 0{,}1$ lépésközzel a negatív gradiens irányában:

$$w^{(1)} = w^{(0)} - \eta \nabla f(w^{(0)}) = \begin{pmatrix} 2 \\ -2 \end{pmatrix} - 0{,}1 \begin{pmatrix} 2 \\ -4 \end{pmatrix} = \begin{pmatrix} 1{,}8 \\ -1{,}6 \end{pmatrix}.$$

Az új függvényérték:

$$f(w^{(1)}) = (1{,}8 - 1)^2 + 2(-1{,}6 + 1)^2 = 0{,}64 + 0{,}72 = 1{,}36,$$

ami kisebb, mint a kezdeti 3. A lépés irányában a veszteség tehát csökkent. Hangsúlyozzuk: ez egy konkrét numerikus példa, nem általános konvergenciaállítás; a lépésköz megválasztásának feltételeit a 6.2. Konvexitás és sima optimalizálási határ tárgyalja.

4.2.8 Labor: gradienslépés kétparaméteres felületen

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor a fenti f függvényre rajzol szintvonalakat és egy gradienslépést, majd assertekkel ellenőrzi a lépést. A kód NumPy-t és Matplotlibet használ; a függvény számításához `float64` típust, a kiindulóponthoz $w^{(0)} = (2, -2)^\top$ értéket rögzítünk.

```
# lab/n014_grad_demo.py
# Futtatás: python lab/n014_grad_demo.py
import numpy as np
import matplotlib.pyplot as plt

def f(w):
    w = np.asarray(w, dtype=np.float64)
    return (w[0] - 1.0) ** 2 + 2.0 * (w[1] + 1.0) ** 2

def grad_f(w):
    w = np.asarray(w, dtype=np.float64)
    return np.array([2.0 * (w[0] - 1.0),
                     4.0 * (w[1] + 1.0)], dtype=np.float64)

w0 = np.array([2.0, -2.0], dtype=np.float64)
eta = 0.1
g0 = grad_f(w0)
w_next = w0 - eta * g0
print("w0 =", w0, " f(w0) =", f(w0))
```

```
print("grad f(w0) =", g0)
print("w_next =", w_next, " f(w_next) =", f(w_next))

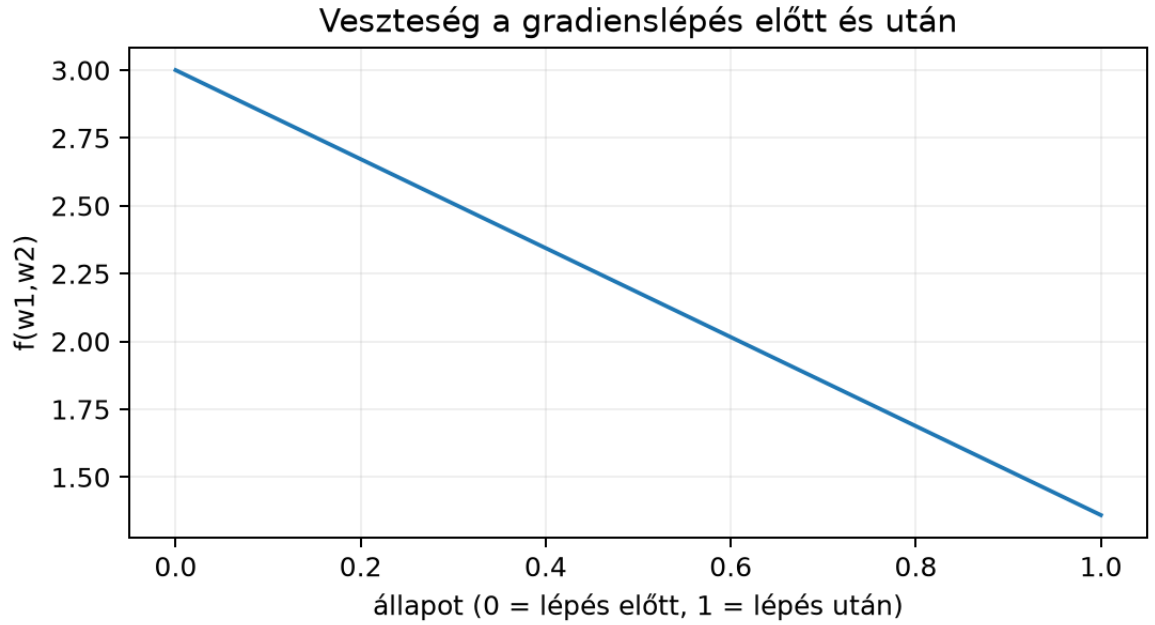
assert abs(f(w0) - 3.0) < 1e-12
assert np.allclose(g0, np.array([2.0, -4.0], dtype=np.float64))
assert abs(f(w_next) - 1.36) < 1e-12

# Szintvonalak és a lépés számított ábrázolása
w1_vals = np.linspace(-1.0, 3.0, 101)
w2_vals = np.linspace(-3.0, 1.0, 101)
W1, W2 = np.meshgrid(w1_vals, w2_vals)
F = (W1 - 1.0) ** 2 + 2.0 * (W2 + 1.0) ** 2
plt.contour(W1, W2, F, levels=[0.25, 0.5, 1.0, 2.0, 4.0])
plt.quiver(w0[0], w0[1], -eta * g0[0], -eta * g0[1],
           angles='xy', scale_units='xy', scale=1,
           color='tab:red', width=0.012)
plt.scatter([w0[0], w_next[0]], [w0[1], w_next[1]], color='black', s=25)
plt.gca().set_aspect('equal')
plt.xlabel('w1')
plt.ylabel('w2')
plt.title('Gradiens lépés: számított szemléltetés')
plt.savefig('lab/n014_grad_step.png', dpi=120)
```

A fenti kód mért kimenete (Python 3.12.3, NumPy 2.5.4, Matplotlib 3.11.2, float64, CPU, egyszálú futás: OMP/OpenBLAS/MKL=1; a kód nem használ véletlen számot). Az utolsó érték a lebegőpontos összeadás miatt 1.3600000000000003; az analitikus érték 1,36:

```
w0 = [ 2. -2.]  f(w0) = 3.0
grad f(w0) = [ 2. -4.]
w_next = [ 1.8 -1.6]  f(w_next) = 1.3600000000000003
```

A veszteség értéke a lépés előtt és után (a szintvonalas, nyilas ábrát a labor szkriptje állítja elő `lab/n014_grad_step.png` néven, a `plt.contour` és `plt.quiver` hívásokkal, tehát az is számított ábra):



4.2. ábra: Veszteség a gradienslépés előtt és után

A labor mért kimenetéből (f(w0)=3,0; f(w_next)=1,3600000000000003, az ábrán 1,36-ra kerekítve); a szintvonalas ábrát szintén a labor szkriptje számolja.

Az ábra és a kimenet ellenőrzése három ponton történik: a gradiens értéke $(2, -4)$; a frissített pont $(1{,}8;\ -1{,}6)$; a veszteség 3-ról 1,36-ra csökken. Ha a lépés irányát a pozitív gradiensre cserélnénk, a veszteség nőne; ez a labor lehetséges negatív kontrollja.

4.2.9 Feladatok

F1 (fogalmi, nehézség 1). Definiálja pontosan a parciális deriváltat és a gradienst egy $f : \mathbb{R}^3 \to \mathbb{R}$ függvényre. Magyarázza el, miért nem elegendő a három parciális derivált létezése a differenciálhatósághoz. Adjon rövid ellenpélda-vázlatot.

F2 (levezetési, nehézség 2). Legyen $f(w_1, w_2) = w_1^2 + 3w_1 w_2 + 2w_2^2$. Számítsa ki $\nabla f(1, -1)$ vektort, a $v = (1, 2)^\top$ irányú $D_v f(1, -1)$ iránymenti deriváltat, és a legmeredekebb növekedés irányába mutató egységvektort az $(1, -1)$ pontban. Ellenőrizze az eredményt a $D_v f$ definíciójából is egy konkrét kis h-ra.

F3 (implementációs, nehézség 2). Írjon Python függvényt, amely centrális differenciával közelíti egy $f : \mathbb{R}^2 \to \mathbb{R}$ függvény gradiensét: $\partial f/\partial w_i \approx [f(w + he_i) - f(w - he_i)]/(2h)$. Használja a labor $f(w_1, w_2) = (w_1 - 1)^2 + 2(w_2 + 1)^2$ függvényét, $h = 10^{-5}$, `float64` típust és a $w = (2, -2)^\top$ pontot. Az elvárt beadandó eredmény: a numerikus gradiens és az analitikus $(2, -4)$ közötti abszolút hiba mindkét komponensben 10^{-4} alatt van.

F4 (hibakeresési, nehézség 2). A következő függvény a labor gradiensét hivatott számítani:

```
def hibas_grad(w):
    w = np.asarray(w, dtype=np.float64)
    return np.array([4.0 * (w[0] - 1.0),
                     2.0 * (w[1] + 1.0)], dtype=np.float64)
```

A $w = (2, -2)$ pontban a helyes gradiens $(2, -4)$, a függvény azonban $(4, -2)$ értéket ad. Lokalizálja a hibát, magyarázza el a következményét egy gradienslépésre, javítsa a kódot, és ellenőrizze a javítást ugyanabban a pontban.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen protokollt annak ellenőrzésére, hogy a negatív gradiens irányú lépés mikor csökkenti a veszteséget. Megfigyelési egység: egy rögzített w pont; a változó tényező: a η lépésköz; a metrika: $f(w - \eta \nabla f(w)) - f(w)$ előjeles különbség. Számítsa ki a labor függvényére a $w^{(0)} = (2, -2)^\top$, $\eta = 0{,}1; 0{,}5; 1{,}0$ eseteket, és indokolja, miért nő a veszteség $\eta = 1{,}0$-nál. Adja meg, milyen további η értékeket vizsgálna, és mi a kísérlet érvényességi korlátja.

4.2.10 Megoldások és indoklás

F1 megoldás. A parciális derivált definíciója $f : \mathbb{R}^3 \to \mathbb{R}$ esetén $i = 1, 2, 3$-ra:

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x + he_i) - f(x)}{h},$$

ahol e_i a háromdimenziós standard bázisvektor. A gradiens

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}(x), \frac{\partial f}{\partial x_2}(x), \frac{\partial f}{\partial x_3}(x) \right)^\top \in \mathbb{R}^3.$$

A puszta létezés nem elég, mert a differenciálhatóság ennél erősebb, közös lineáris közelítést követel: a maradéktagnak minden irányból $\|u\|$-nál gyorsabban kell nullához tartania. Ellenpéldavázlat: az origón kívül $f(x_1, x_2) = x_1 x_2/(x_1^2 + x_2^2)$, az origóban 0; mindkét parciális derivált létezik és nulla az origóban, de a függvény nem folytonos ott, így nem differenciálható.

F2 megoldás. A parciális deriváltak $\partial f/\partial w_1 = 2w_1 + 3w_2$ és $\partial f/\partial w_2 = 3w_1 + 4w_2$. A $(1, -1)$ pontban:

$$\nabla f(1,-1) = \begin{pmatrix} 2\cdot 1 + 3(-1) \\ 3\cdot 1 + 4(-1) \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

Az iránymenti derivált a lemma alapján $D_v f(1,-1) = (-1)(1) + (-1)(2) = -3$. A $\|v\| = \sqrt{5}$ normájú vektor miatt az egységirányra normált érték $-3/\sqrt{5}$. A legmeredekebb növekedés egységvektora $\nabla f/\|\nabla f\| = (-1,-1)^\top/\sqrt{2}$. Numerikus ellenőrzés $h = 10^{-5}$-tel. Mivel $f(1,-1) = 0$, a kifejtésből $f(1+h, -1+2h) = -3h + 15h^2$, ezért a differenciahányados pontosan $-3 + 15h$, ami $h = 10^{-5}$ esetén $-2{,}99985$. A mért érték (Python 3.12.3, NumPy 2.5.4, float64): a differenciahányados $-2{,}999850000007242$, a zárt alakú $-2{,}99985$-től való eltérés $-7{,}2 \cdot 10^{-12}$; ez a float64 kerekítéséből adódó hiba, jóval a $15h = 1{,}5 \cdot 10^{-4}$ nagyságrendű $O(h)$-eltérés alatt.

F3 megoldás. A teljes függvény:

```
import numpy as np

def f(w):
    w = np.asarray(w, dtype=np.float64)
    return (w[0] - 1.0) ** 2 + 2.0 * (w[1] + 1.0) ** 2

def numerical_grad(f, w, h=1e-5):
    w = np.asarray(w, dtype=np.float64)
    g = np.zeros_like(w)
    for i in range(w.size):
        e = np.zeros_like(w)
        e[i] = 1.0
        g[i] = (f(w + h * e) - f(w - h * e)) / (2.0 * h)
    return g

w = np.array([2.0, -2.0], dtype=np.float64)
g_num = numerical_grad(f, w)
g_an = np.array([2.0, -4.0], dtype=np.float64)
print("numerikus:", g_num)
print("abszolút hiba:", np.abs(g_num - g_an))
```

A centrális differencia $O(h^2)$ csonkolási hibája és a 10^{-5} lépésköz melletti kerekítési hiba együttesen 10^{-11} nagyságrendű eltérést ad, ami bőven a 10^{-4} korlát alatt van. A fenti kód mért kimenete (Python 3.12.3, NumPy 2.5.4, float64, CPU, egyszálú futás):

```
numerikus: [ 2. -4.]
abszolút hiba: [1.31024080e-11 2.62048161e-11]
```

F4 megoldás. A hiba oka: a `hibas_grad` a w_1 szerinti parciális derivált együtthatójának 2 helyett 4-et, a w_2 szerintinek 4 helyett 2-t használ; a két komponens együtthatója fel van cserélve. Következmény: a frissítés $\eta = 0{,}1$ mellett $\Delta w = -0{,}1 \cdot (4, -2) = (-0{,}4; 0{,}2)$ lenne, ami a helyes $\Delta w = (-0{,}2; 0{,}4)$ helyett rossz irányú és hibás nagyságú lépést ad; a f ebben a konkrét pontban még csökkenhetne ugyan, de más pontokban növekedést is okozhat, és a tanítás torzul. Javítás: az együtthatókat rendre 2-re és 4-re kell cserélni:

```python
def javitott_grad(w):
    w = np.asarray(w, dtype=np.float64)
    return np.array([2.0 * (w[0] - 1.0),
                     4.0 * (w[1] + 1.0)], dtype=np.float64)
```

Ellenőrzés a $(2,-2)$ pontban: `javitott_grad(w)` értéke $(2,-4)$, megegyezik az analitikus gradienssel.

F5 megoldás. A protokoll rögzíti a $w^{(0)}=(2,-2)^\top$ pontot és a $\nabla f(w^{(0)})=(2,-4)^\top$ gradienst. A változó az $\eta\in\{0{,}1;0{,}5;1{,}0\}$ lépésköz, a kimenet $f(w^{(0)}-\eta\nabla f(w^{(0)}))-f(w^{(0)})$. Számolás:

- $\eta=0{,}1$: $w=(1{,}8;-1{,}6)$, $f=1{,}36$, különbség $-1{,}64$; csökken.
- $\eta=0{,}5$: $w=(1;0)$, $f=2$, különbség -1; csökken.
- $\eta=1{,}0$: $w=(0;2)$, $f=(0-1)^2+2(2+1)^2=1+18=19$, különbség $+16$; nő.

Vezessük le a különbséget zárt alakban. A frissített pont $w(\eta)=(2-2\eta,\,-2+4\eta)$, ezért

$$f(w(\eta))-f(w^{(0)})=(1-2\eta)^2-1+2(4\eta-1)^2-2=-20\eta+36\eta^2.$$

Ez $\eta=0$-nál nulla, $\eta=5/9\approx 0{,}556$-nál ismét nulla, és $0<\eta<5/9$ között negatív; tehát a csökkenés határa $\eta=5/9$. A koordináták szerint is látszik a mechanizmus: az optimumtól vett eltérés a w_1 koordinátában $(1-2\eta)$-szorosára, a w_2-ben $(1-4\eta)$-szorosára változik. Ezért $\eta>0{,}5$ esetén $|1-4\eta|>1$, a w_2 komponens túlleng a minimumon (oszcillál), és a lépés „átugorja” a minimumot; a teljes függvényérték mégis csökken $\eta=5/9$-ig, mert a w_1 tag csökkenése ezt kompenzálja. A csökkenési tartomány határát tehát a teljes f képletéből, nem egyetlen koordináta viselkedéséből kell megállapítani.

A kísérlet ezért csak azt állíthatja, hogy elég kis η-ra a lépés csökkenti a veszteséget, nem azt, hogy minden η-ra csökken. A határ két oldalát további η értékekkel érdemes vizsgálni: $0{,}5;\,0{,}55;\,0{,}56;\,0{,}6$; az érvényességi korlát, hogy az eredmény erre az egy függvényre és pontra vonatkozik, nem általános optimalizálási garancia.

4.2.11 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés: *ha minden parciális derivált létezik egy pontban, akkor a függvény ott differenciálható.* Ez hamis. A klasszikus ellenpélda a

$$f(x_1,x_2)=\begin{cases}\dfrac{x_1x_2}{x_1^2+x_2^2}, & (x_1,x_2)\neq(0,0),\\[2mm] 0, & (x_1,x_2)=(0,0)\end{cases}$$

függvény. Az origóban mindkét parciális derivált létezik és nulla:

$$\frac{\partial f}{\partial x_1}(0,0)=\lim_{h\to 0}\frac{f(h,0)-f(0,0)}{h}=\lim_{h\to 0}\frac{0}{h}=0,$$

és szimmetrikusan $\partial f/\partial x_2(0,0)=0$. A függvény mégsem differenciálható az origóban, mert nem is folytonos ott: az $x_1=x_2=t\neq 0$ egyenes mentén $f(t,t)=t^2/(2t^2)=1/2$, ami nem tart a 0-hoz, ha $t\to 0$. A 4.1. Függvény, határérték és derivált alfejezetbeli tétel többváltozós megfelelője ugyanezt mondja ki: a differenciálhatóságból folytonosság következik, ezért a folytonosság hiánya kizárja a differenciálhatóságot. Ráadásul az $(1,1)$ irányú iránymenti derivált sem létezik, mert

$$\frac{f(h,h)-f(0,0)}{h}=\frac{1/2}{h}$$

nem konvergál $h \to 0$ esetén. A gradiens-képletből számított $\nabla f(0,0)^\top(1,1)=0$ tehát nem adná a tényleges iránymenti viselkedést; ez is mutatja, hogy a lemma feltétele, a differenciálhatóság, lényeges.

4.2.12 Önellenőrzés és visszatekintés

Ellenőrizze a fejezet állításait három ponton. (1) A kvadratikus veszteség gradiensének alakja: $X \in \mathbb{R}^{3\times 2}$, $w \in \mathbb{R}^2$, $y \in \mathbb{R}^3$ esetén $X^\top(Xw-y) \in \mathbb{R}^2$, azonos alakú a w paraméterrel. (2) Az iránymenti derivált homogenitása: $D_{2v}f = 2D_v f$ a definícióból közvetlenül látszik, és a lemmával is konzisztens. (3) A szintvonalra merőlegesség: a szintvonalon f konstans, így az érintő irányú derivált nulla. Visszatekintés a 3.2. Mátrixszorzás és affín réteg alfejezetből: a mátrixszorzás alak-egyeztetése itt a gradiens alakjának ellenőrzésében ismétlődik; a 4.1. Függvény, határérték és derivált alfejezetből: az egyváltozós határértékes definíciót terjesztettük ki irányonként.

4.2.13 Rövid összefoglaló

A parciális derivált egy többváltozós függvény egy koordinátája szerinti, rögzített többi változójú deriváltja. A gradiens a parciális deriváltak oszlopvektora, alakja az input alakjával egyezik. Az iránymenti derivált a v irányú egyváltozós derivált; differenciálható függvény esetén $D_v f(x) = \nabla f(x)^\top v$. Ebből következik, hogy a gradiens a legmeredekebb növekedés iránya, és merőleges a szintvonalakra. A differenciálhatóság erősebb, mint a parciális deriváltak létezése: a lineáris közelítés maradékának minden irányból $\|u\|$-nál gyorsabban kell nullához tartania. A kvadratikus veszteség gradiense $X^\top(Xw-y)$; a labor ezt két paraméteren, kézi értékekkel és gradienslépéssel ellenőrizte.

4.2.14 Továbblépés

A 4.3. Jacobi-mátrix, láncszabály és VJP a többváltozós láncszabályt és a Jacobi-mátrixot vezeti be. A gradiens itt tanult alak-egyeztetése ott válik a vektor–Jacobi-szorzat alapjává: a rétegenkénti visszafelé számításban a skalár veszteség gradiense mindig a megfelelő tenzor alakjában jelenik meg.

4.2.15 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

4.3 Jacobi-mátrix, láncszabály és VJP

4.3.1 Cél, előismeret és induló kérdés

A 4.2. Parciális derivált és gradiens alfejezetben egy $f : \mathbb{R}^d \to \mathbb{R}$ skalár függvény gradiensét értelmeztük. A neurális réteg azonban tipikusan vektorból vektort képez: egy affin réteg $\mathbb{R}^d$-ből $\mathbb{R}^m$-be, egy aktiváció réteg szintén $\mathbb{R}^m$-ből $\mathbb{R}^m$-be. A tanítás során a veszteség skalár, tehát a visszafelé haladó számításnak „csak” egy-egy vektor–Jacobi-szorzatot (VJP-t) kell elvégeznie, nem kell felépítenie a teljes mátrix deriváltat.

Induló becslés: ha $f : \mathbb{R}^{10000} \to \mathbb{R}^{1000}$, akkor a teljes Jacobi-mátrix $1000 \cdot 10000 = 10^7$ számot tartalmazna. Ha viszont csak egy $g \in \mathbb{R}^{1000}$ kimeneti vektort vetítünk vissza, a kívánt bemeneti gradiens maga is csupán 10000 szám. A kérdés: milyen mátrix–vektor szorzat és milyen transzponálási konvenció adja helyesen ezt az olcsóbb számítást?

A cél egy pontos, alakhelyes nyelv: Jacobi-mátrix, kompozíció, JVP, VJP, batch-redukció, valamint az affin réteg súly-, bias- és inputgradiensének levezetése. Előfeltétel a 4.2. Parciális derivált és gradiens parciális derivált- és gradiensfogalma; a 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetből a legkisebb négyzetes normálegyenletre is röviden visszautalunk.

4.3.2 A Jacobi-mátrix: definíció és alak

Definíció (Jacobi-mátrix). Legyen $U \subseteq \mathbb{R}^d$ nyílt, $f : U \to \mathbb{R}^m$, és legyen f differenciálható az $x \in U$ pontban. Ekkor az f Jacobi-mátrixa

$$J_f(x) \in \mathbb{R}^{m \times d}, \qquad [J_f(x)]_{i,j} = \frac{\partial f_i}{\partial x_j}(x).$$

A mátrix **sora az output**, **oszlopa az input** komponenshez tartozik. A mátrix i-edik sora az f_i skalár függvény gradiensének transzponáltja:

$$J_f(x) = \begin{bmatrix} (\nabla f_1(x))^\top \\ \vdots \\ (\nabla f_m(x))^\top \end{bmatrix}.$$

A differenciálhatóság definíciója éppen azt jelenti, hogy a maradékos lineáris közelítés minden irányból jó:

$$f(x+u) = f(x) + J_f(x)u + r(u), \qquad \lim_{u \to 0} \frac{\|r(u)\|}{\|u\|} = 0.$$

Alakellenőrzés: $J_f(x)$ mérete $m \times d$, u mérete $d \times 1$, tehát $J_f(x)u$ mérete $m \times 1$, azonos $f(x+u) - f(x)$ alakjával. Ha $m = 1$, akkor $J_f(x)$ egy $1 \times d$ sor, és $J_f(x)^\top = \nabla f(x)$, ami visszaadja a 4.2. Parciális derivált és gradiens gradiensét.

4.3.3 Kompozíció és a láncszabály

A rétegzett háló matematikailag kompozíció: a következő réteg bemenete az előző réteg kimenete. A láncszabály mondja meg, hogyan szorzódnak a Jacobi-mátrixok. A láncszabály és a Jacobi-mátrixok szorzási rendjének részletes tárgyalása a [B01] 5. fejezetében, azon belül az 5.3. szakaszban található.

Tétel (többváltozós láncszabály). Legyen $U \subseteq \mathbb{R}^d$ és $V \subseteq \mathbb{R}^m$ nyílt, $f : U \to V$ differenciálható $x \in U$-ban, $\varphi : V \to \mathbb{R}^k$ differenciálható $f(x) \in V$-ben. Ekkor $h = \varphi \circ f$ differenciálható x-ben, és

$$J_h(x) = J_\varphi(f(x))\, J_f(x).$$

A szorzat alakjai: $J_\varphi(f(x)) \in \mathbb{R}^{k\times m}$, $J_f(x) \in \mathbb{R}^{m\times d}$, ezért a szorzat $\mathbb{R}^{k\times d}$-beli.

A bizonyításvázlatban $r(u) = o(\|u\|)$ azt jelenti, hogy $\|r(u)\|/\|u\| \to 0$, ha $u \to 0$, és $a(u) = O(\|u\|)$ azt, hogy $\|a(u)\|/\|u\|$ korlátos $u \to 0$ esetén.

Bizonyításvázlat. A bizonyítás a differenciálhatóság elsőrendű maradékos alakját használja. Az $u = 0$ esetben az állítás triviális, ezért a továbbiakban $u \neq 0$. Rögzítsünk egy $u \neq 0$ vektort, és írjuk

$$f(x+u) = f(x) + J_f(x)u + r_f(u), \qquad \frac{\|r_f(u)\|}{\|u\|} \to 0.$$

Legyen $w = f(x+u) - f(x) = J_f(x)u + r_f(u)$. Mivel φ differenciálható $f(x)$-ben,

$$\varphi(f(x)+w) = \varphi(f(x)) + J_\varphi(f(x))w + r_\varphi(w), \qquad \frac{\|r_\varphi(w)\|}{\|w\|} \to 0.$$

A φ differenciálhatóságából kapott r_φ maradékot $r_\varphi(0) = 0$-ra egészítjük ki; így a $w = 0$ esetben is érvényes a felbontás. Behelyettesítve w értékét:

$$h(x+u) = h(x) + J_\varphi(f(x))J_f(x)u + J_\varphi(f(x))r_f(u) + r_\varphi(w).$$

Az első maradéktag $o(\|u\|)$, mert $J_\varphi(f(x))$ korlátos lineáris leképezés és $r_f(u) = o(\|u\|)$. A másodikhoz azt használjuk, hogy $w = O(\|u\|)$: a $J_f(x)$ lineáris leképezés véges dimenzióban korlátos, ezért alkalmas $C > 0$ mátrixnormával $\|J_f(x)u\| \le C\|u\|$, és $\|r_f(u)\|/\|u\|$ korlátos elég kis u-ra. Így $w \to 0$, amikor $u \to 0$, tehát $\|r_\varphi(w)\|/\|w\| \to 0$, miközben $\|w\|/\|u\|$ korlátos. Ezért a teljes maradék $o(\|u\|)$, ami éppen a kívánt lineáris közelítést adja. A teljes, határértékekkel dolgozó kidolgozás a szokásos elemi kalkulus bizonyítás; a fenti vázlat csak a kimondott differenciálhatósági feltételeket használja.

A tétel feltételei elégségesek, de nem szükségesek. Példa: $f(x) = 0$ konstans, $\varphi(t) = |t|$. Ekkor φ nem differenciálható $f(x) = 0$-ban, ezért a láncszabály képlete nem alkalmazható; a $\varphi \circ f$ mégis azonosan nulla, tehát differenciálható. Ezért a tétel használatakor mindig ellenőrizni kell, hogy a belső függvény kimeneti pontjában a külső függvény tényleg differenciálható-e.

4.3.4 VJP, JVP, batch és reduction

Definíció (JVP). Adott $f : \mathbb{R}^d \to \mathbb{R}^m$ és adott $v \in \mathbb{R}^d$ irányvektor esetén a **Jacobi–vektor-szorzat** (JVP, angolul Jacobian–vector product)

$$\mathrm{JVP}_f(x; v) = J_f(x)v \in \mathbb{R}^m.$$

Ez az irány menti lineáris megváltozás: $f(x + \varepsilon v) \approx f(x) + \varepsilon\, J_f(x)v$.

Definíció (VJP). Adott $g \in \mathbb{R}^m$ kimeneti vektor esetén a **vektor–Jacobi-szorzat** (VJP, angolul vector–Jacobian product)

$$\mathrm{VJP}_f(x; g) = J_f(x)^\top g \in \mathbb{R}^d.$$

A megegyezésünk szerint az absztrakt vektorok oszlopvektorok, ezért a VJP-ben a Jacobi-mátrix transzponáltja áll. A mátrixok mérete: $J_f^\top \in \mathbb{R}^{d\times m}$, $g \in \mathbb{R}^{m\times 1}$, tehát az eredmény $d \times 1$.

A két művelet nem szinonima. A JVP egy inputirányt visz előre, a VJP egy outputgradienst visz hátra. A teljes Jacobi m VJP-vel (reverse mode) vagy d JVP-vel (forward mode) áll elő; ha $m \ll d$, a VJP olcsóbb, ha pedig $d \ll m$, a JVP. A klasszikus backpropagation egy skalár veszteség gradiense miatt viszont VJP-kből épül fel; a visszaterjesztés rétegenkénti láncszabályának tankönyvi tárgyalása a [B02] 6.5. szakaszában olvasható. A kapcsolatot a láncszabály adja: legyen $L : \mathbb{R}^m \to \mathbb{R}$ differenciálható, és $h(x) = L(f(x))$. Skalár kimenetű h-ra a Jacobi-mátrix $1 \times d$ sor, transzponáltja a gradiens:

$$\nabla h(x) = J_f(x)^\top \nabla L(f(x)).$$

Itt $g = \nabla L(f(x))$, tehát $\nabla h(x) = \mathrm{VJP}_f(x; g)$.

Elemenkénti aktiváció. A hálózatokban gyakori réteg az elemenkénti aktiváció: $\sigma : \mathbb{R}^m \to \mathbb{R}^m$, $\sigma(z) = (\sigma(z_1), \dots, \sigma(z_m))^\top$, ahol σ egyváltozós differenciálható függvény (például szigmoid vagy tanh). Ennek Jacobi-mátrixa átlós:

$$J_\sigma(z) = \mathrm{diag}(\sigma'(z_1), \dots, \sigma'(z_m)),$$

ezért a VJP nem mátrixszorzás, hanem elemenkénti szorzat: $J_\sigma(z)^\top g = g \odot \sigma'(z)$, ahol $\sigma'(z)$ a deriváltakból álló vektor, $\odot$ pedig az elemenkénti (Hadamard-) szorzat. Ez magyarázza, miért olcsó a backprop az aktivációkon: a Jacobi átlós, így nem kell $m \times m$-es mátrixot tárolni. A ReLU esetében $\sigma'(t) = 1$, ha $t > 0$, és $\sigma'(t) = 0$, ha $t < 0$; a $t = 0$ pontban a derivált nem létezik, ezért ott külön konvenció szükséges (a PyTorchban mért viselkedés szerint a hátramenet 0-t ad, ezt a 2.15.0+cpu verzióban ellenőriztük).

Batch és reduction. A tanításban a veszteség általában egy batch-átlag:

$$L = \frac{1}{N}\sum_{i=1}^{N} \ell(z_i, y_i),$$

ahol $z_i = f(x_i) \in \mathbb{R}^m$ az i-edik megfigyelés kimenete, y_i pedig ugyanennek a megfigyelésnek a célértéke (címkéje). Legyen $Z \in \mathbb{R}^{N\times m}$ a batchelt kimenet, és $G = \nabla_Z L \in \mathbb{R}^{N\times m}$; a G i-edik sora $(\nabla_{z_i} L)^\top$. Az $1/N$ szorzó nem a láncszabályból, hanem az átlag definíciójából származik: a

Z szerinti deriváláskor egyszerűen a kimeneti gradiensbe kerül, a transzponálási szabályt nem érinti:

$$\frac{\partial L}{\partial z_{i,p}} = \frac{1}{N}\frac{\partial \ell(z_i, y_i)}{\partial z_{i,p}}.$$

Ha valaki `sum` redukcióval számol, akkor a G_{sum} mátrix i-edik sorára $(\nabla_{z_i}\ell)^\top$ adódik, és $G_{\text{mean}} = (1/N)G_{\text{sum}}$. A fontos szabály: a redukció **nem** a Jacobi-mátrix transzponálásába épül, hanem a kimeneti gradiens skálájába. Az autograd (a PyTorch beépített automatikus differenciáló mechanizmusa) ezt a G értéket szolgáltatja a hátramenetben [D05], és a réteg feladata már csak az adott G-hez tartozó VJP-összeg kiszámítása.

4.3.5 Az affin réteg súly-, bias- és inputgradiense

Most teljesítjük a fejezet fő bizonyítási feladatát. Legyen $X \in \mathbb{R}^{N\times d}$ batchelt bemenet, $W \in \mathbb{R}^{m\times d}$ súly, $b \in \mathbb{R}^m$ bias, és

$$Z = XW^\top + b,$$

ahol a bias soriránybanban broadcast; komponensenként

$$Z_{i,p} = \sum_{q=1}^{d} X_{i,q}W_{p,q} + b_p.$$

Ez a `torch.nn.Linear` súlyalakja is: `weight` mérete `(out_features, in_features)`, a forward pedig `x @ weight.T + bias`.

Legyen L skalár, és tegyük fel, hogy L mint Z függvénye differenciálható. Jelölje

$$G = \nabla_Z L \in \mathbb{R}^{N\times m}, \qquad G_{i,r} = \frac{\partial L}{\partial Z_{i,r}}.$$

Az X, W és b gradienseit külön-külön vezetjük le.

Inputgradiens. Az $X_{i,q}$ csak az i-edik sor $Z_{i,:}$ kimenetén keresztül hat. Mivel $Z_{i,p} = \sum_{q=1}^{d} X_{i,q}W_{p,q} + b_p$, a szükséges lokális derivált $\partial Z_{i,p}/\partial X_{i,q} = W_{p,q}$. A láncszabály:

$$\frac{\partial L}{\partial X_{i,q}} = \sum_{p=1}^{m} \frac{\partial L}{\partial Z_{i,p}}\frac{\partial Z_{i,p}}{\partial X_{i,q}} = \sum_{p=1}^{m} G_{i,p}W_{p,q} = (GW)_{i,q}.$$

Oszlopvektoros alakban egy megfigyelésre: $\nabla_{x_i} L = W^\top g_i$, ahol $g_i = \nabla_{z_i} L$ oszlopvektor. A teljes batch mátrixalakja:

$$\boxed{\nabla_X L = GW \in \mathbb{R}^{N\times d}}.$$

Súlygradiens. A $W_{p,q}$ súly minden i sorhoz hozzájárul, ezért a gradiens az i szerinti külső összeg. Mivel (ahol $\delta_{r,p}$ a Kronecker-delta: értéke 1, ha $r = p$, különben 0)

$$\frac{\partial Z_{i,r}}{\partial W_{p,q}} = \delta_{r,p} X_{i,q},$$

a láncszabály:

$$\frac{\partial L}{\partial W_{p,q}} = \sum_{i=1}^{N} \sum_{r=1}^{m} \frac{\partial L}{\partial Z_{i,r}} \frac{\partial Z_{i,r}}{\partial W_{p,q}} = \sum_{i=1}^{N} G_{i,p} X_{i,q} = (G^\top X)_{p,q}.$$

Tehát

$$\boxed{\nabla_W L = G^\top X \in \mathbb{R}^{m \times d}}.$$

Biasgradiens. A b_p az összes megfigyelés $Z_{i,p}$ eleméhez ugyanazzal az 1 együtthatóval járul hozzá, mert a forwardben a bias soriránybanbroadcastolódik. A backwardben ez a broadcast megfordul: a b_p-hez tartozó összes járulék összeadódik, ezért redukálunk az i tengely mentén:

$$\frac{\partial L}{\partial b_p} = \sum_{i=1}^{N} G_{i,p}.$$

Mátrixos írásmóddal ez a G oszlopösszege:

$$\boxed{(\nabla_b L)_p = \sum_{i=1}^{N} G_{i,p}, \qquad \nabla_b L = G^\top \mathbf{1}_N \in \mathbb{R}^m.}$$

PyTorch/Numpy írásmódban: `db = G.sum(axis=0)`.

A három eredmény egységes konvenciója: az absztrakt oszlopvektoros VJP $W^\top g$, a batch-soros tárolás viszont GW. Aki a kettőt keveri, transzponálási hibát kap; erre a fejezet Tipikus tévedés, önellenőrzés és visszatekintés szakaszában (4.3.10.) visszatérünk.

Az $L = \frac{1}{2}\sum_{i,p} Z_{i,p}^2$ speciális esetben $G = Z$, tehát például $\nabla_W L = Z^\top X$. A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság legkisebb négyzetes célfüggvénye $\frac{1}{2}\|Xw - y\|_2^2$; a $Z = Xw - y$ választással ez éppen a fenti L, ezért $G = Z = Xw - y$, és a láncszabály $\nabla_w L = X^\top(Xw - y)$-t ad. A $\frac{1}{2}$ tehát a Z szerinti deriváltnál kiesik, és a gradiensben nem jelenik meg külön szorzóként.

A számítási lánc alakjai. Illusztrációként tekintsünk két affin rétegből és egy elemenkénti aktivációból álló láncot: $X \to Z_1 \to A \to Z_2$, ahol $W_1 \in \mathbb{R}^{h \times d}$, $W_2 \in \mathbb{R}^{m \times h}$, $b_1 \in \mathbb{R}^h$, $b_2 \in \mathbb{R}^m$. A következő táblázat az előremenet és a hátramenet mennyiségeit és alakjait foglalja össze; az első négy sor az előremenet, az utolsó három a hátramenet.

Mennyiség	Alak	Meghatározás
X	$N \times d$	bemenet
$Z_1 = XW_1^\top + b_1$	$N \times h$	első affin réteg
$A = \sigma(Z_1)$	$N \times h$	elemenkénti aktiváció
$Z_2 = AW_2^\top + b_2$	$N \times m$	második affin réteg
$G_2 = \nabla_{Z_2} L$	$N \times m$	kimeneti gradiens
$G_1 = G_2 W_2 \odot \sigma'(Z_1)$	$N \times h$	aktiváció hátramenete
$\nabla_X L = G_1 W_1$	$N \times d$	inputgradiens

A hátramenet minden lépése a következő sor gradiensét használja: G_2-ből G_1, majd G_1-ből $\nabla_X L$ áll elő. A táblázat egyben alakellenőrzés is: minden szorzat belső mérete illeszkedik, a kimenet pedig az input alakját adja vissza.

4.3.6 Kézi kis példa és minimális kód

Legyen

$$f(x) = \begin{bmatrix} x_1 \\ x_1 + x_2^2 \\ x_1 x_2 \end{bmatrix}, \qquad x = (1,2)^\top.$$

A parciális deriváltak:

- $\nabla f_1 = (1, 0)$,
- $\nabla f_2 = (1, 2x_2) = (1, 4)$,
- $\nabla f_3 = (x_2, x_1) = (2, 1)$,

ezért

$$J_f(1,2) = \begin{bmatrix} 1 & 0 \\ 1 & 4 \\ 2 & 1 \end{bmatrix}.$$

Válasszuk a kimeneti gradiens értékét $g = (1, -2, 3)^\top$-nak. A VJP:

$$J_f^\top g = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 4 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 1 \cdot (-2) + 2 \cdot 3 \\ 0 \cdot 1 + 4 \cdot (-2) + 1 \cdot 3 \end{bmatrix} = \begin{bmatrix} 5 \\ -5 \end{bmatrix}.$$

A következő kód ugyanezt explicit Jacobi-mátrixszal és PyTorch-autograddal is ellenőrzi. A `torch.dot(z,g)` skalár veszteséget képez, tehát az autograd a láncszabály szerinti $\nabla_x(g^\top f(x)) = J_f^\top g$ VJP-t adja vissza.

```
# lab/n015/vjp_lab.py
import torch

torch.manual_seed(20261020)  # a példa determinisztikus; véletlen nincs benne
DTYPE = torch.float64
DEVICE = torch.device("cpu")

x = torch.tensor([1.0, 2.0], dtype=DTYPE, device=DEVICE, requires_grad=True)
g = torch.tensor([1.0, -2.0, 3.0], dtype=DTYPE, device=DEVICE)

def f(u: torch.Tensor) -> torch.Tensor:
    return torch.stack([
        u[0],
        u[0] + u[1] ** 2,
        u[0] * u[1],
    ])

z = f(x)

# Az x = (1,2) pontban kézzel levezetett Jacobi-mátrix:
J = torch.tensor(
```

```
    [
        [1.0, 0.0],
        [1.0, 2.0 * x[1].item()],
        [x[1].item(), x[0].item()],
    ],
    dtype=DTYPE,
    device=DEVICE,
)

vjp_explicit = J.T @ g
# loss = g·f(x), ezért grad_x(loss) = J_f(x)^T g:
loss = torch.dot(z, g)
vjp_autograd, = torch.autograd.grad(loss, x)

print("J =")
print(J)
max_abs_diff = (vjp_explicit - vjp_autograd).abs().max().item()
print("vjp_explicit =", vjp_explicit)
print("vjp_autograd =", vjp_autograd)
print("max_abs_diff =", max_abs_diff)
expected = torch.tensor([5.0, -5.0], dtype=DTYPE, device=DEVICE)
assert max_abs_diff < 1e-12
assert torch.allclose(vjp_explicit, expected, atol=1e-12, rtol=1e-12)
```

Futtatás a `lab/n015` munkakönyvtárból:

```
python vjp_lab.py
```

Mért kimenet (Python 3.12.3, torch 2.15.0+cpu, float64, CPU, egyszálú futás: OMP/OpenBLAS/MKL=1, seed 20261020):

```
J =
tensor([[1., 0.],
        [1., 4.],
        [2., 1.]], dtype=torch.float64)
vjp_explicit = tensor([ 5., -5.], dtype=torch.float64)
vjp_autograd = tensor([ 5., -5.], dtype=torch.float64)
max_abs_diff = 0.0
```

4.3.7 Labor: explicit Jacobian és autograd-VJP összehasonlítása

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor kódja a fenti `lab/n015/vjp_lab.py`.

- **Input:** `x=(1,2)` és `g=(1,-2,3)`, mindkettő `float64`; nincs külső adatfájl, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-adatkészletre itt nincs szükség.
- **Import:** `torch`; a labor a CPU-autograd utat használja, `device="cpu"`.
- **Seed/dtype/device:** `torch.manual_seed(20261020)`, `torch.float64`, CPU. A seed itt csak a protokoll része, mert a kód nem használ véletlen számot.
- **Adatút:** forrásfájl `lab/n015/vjp_lab.py`, futtatási parancs fent.
- **Ellenőrzés:** a `vjp_explicit` a kézzel felírt $J_f^\top g$, a `vjp_autograd` a $g \cdot f(x)$ skalár veszteség gradiense. Alaktolerancia: mindkettő `(2,)`; f64 esetén az eltérésnek nulla közeli értéknek kell lennie, elfogadott numerikus korlát `atol=1e-12`, `rtol=1e-12`. A kiírt `max_abs_diff` a mért futásban 0.0.
- **Értelmezés:** az egyezés azt mutatja, hogy a skalár veszteség hátramenete nem a 3×2 Jacobi-mátrixot tárolja, hanem annak transzponáltjával szorozva a g vektort. A $J_f^\top g$ irány tehát az, amit a backprop VJP-ként használ.

A fenti kimenet mért futás eredménye a megadott környezetben; a kódot a tanulólaborban ugyanígy kell futtatni, és a `max_abs_diff` értékét a fenti toleranciával ellenőrizni. A kódba épített assertek a futást is elbuktatják, ha az egyezés nem teljesül.

4.3.8 Feladatok

F1, fogalmi, nehézség 1. Legyen $W = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$, $b = 0$, $f(x) = Wx + b$. Adja meg J_f alakját, a JVP-t $v = (1, 0, -1)^\top$, és a VJP-t $g = (1, -1)^\top$ esetén. Ne feledje megnevezni, melyik művelet melyik Jacobi-irányt használja.

F2, levezetési, nehézség 2. Legyen

$$f(x_1, x_2, x_3) = \begin{bmatrix} x_1 + x_2 x_3 \\ x_2^2 + x_3 \end{bmatrix}, \qquad x_0 = (1, 2, 3)^\top.$$

Számítsa ki $J_f(x_0)$ mátrixot és a $g = (-1, 2)^\top$ vektorhoz tartozó VJP-t. Minden lépést indokoljon, és ellenőrizze az alakokat.

F3, implementációs, nehézség 2. Írjon NumPy-függvényt

```
def affine_grads(X: np.ndarray, W: np.ndarray, b: np.ndarray, G: np.ndarray):
    ...
```

amely `float64` bemenetekkel visszaadja a `dX, dW, db` hármast az $X \in \mathbb{R}^{N \times d}$, $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, $G \in \mathbb{R}^{N \times m}$ konvencióban. Tesztelje az $N = 2, d = 3, m = 2$ esetet: $X = [[1, 0, 2], [0, 1, 1]]$, $W = [[1, 2, 0], [0, 1, 2]]$, $b = [-1, 1]$, $G = [[2, -1], [0, -3]]$. Adja meg a három elvárt mátrixot/vektort.

F4, hibakeresési, nehézség 1. Az alábbi kód a batchelt affin réteg inputgradiensét szeretné visszaadni $G = \nabla_Z L$ ismeretében:

```
def buggy_dX(X, W, G):
    # X: N x d, W: m x d, G: N x m
    return W.T @ G
```

Keresse meg a hibát, magyarázza el az okát, következményét, és javítsa a kódot. Ellenőrizze a javítást az F3 numerikus példáján.

F5, kísérleti/transzfer, nehézség 3. Tervezzen CPU-s PyTorch-mérést annak eldöntésére, hogy az $f : \mathbb{R}^{1000} \to \mathbb{R}^{1000}$, $f(x) = W \sin(x) + b$ leképezésnél (a sin elemenként értendő, $W \in \mathbb{R}^{1000 \times 1000}$ és $b \in \mathbb{R}^{1000}$ rögzített) a teljes Jacobi-mátrix anyagiasítása (materialization: tényleges kiszámítása és memóriában tartása) vagy az autograd-VJP használ-e kisebb csúcsmemóriát és időt. Nincs szükség tényleges futtatásra; adjon protokollt, baseline-t, változó tényezőt, metrikát, ellenőrző egyezést és érvényességi korlátot. Jelölje meg, mi lenne a nyers eredménytábla két sora.

4.3.9 Megoldások és indoklás

F1 megoldás. f affin, ezért a Jacobi-mátrix konstans: $J_f = W \in \mathbb{R}^{2\times 3}$. Sor–oszlop konvenció: az 1. sor az $f_1 = x_1 + 2x_2 + 3x_3$, a 2. sor az $f_2 = 4x_1 + 5x_2 + 6x_3$ parciális deriváltjait tartalmazza.

JVP: $J_f v = Wv = (1\cdot 1 + 2\cdot 0 + 3\cdot(-1),\ 4\cdot 1 + 5\cdot 0 + 6\cdot(-1)) = (-2,-2)^\top$, mérete 2×1.

VJP: $J_f^\top g = W^\top g = (1\cdot 1 + 4\cdot(-1),\ 2\cdot 1 + 5\cdot(-1),\ 3\cdot 1 + 6\cdot(-1)) = (-3,-3,-3)^\top$, mérete 3×1. Az első a Jacobi-mátrixszal, a második annak transzponáltjával szoroz.

F2 megoldás. Parciális deriváltak:

$$\frac{\partial f_1}{\partial x_1} = 1, \quad \frac{\partial f_1}{\partial x_2} = x_3, \quad \frac{\partial f_1}{\partial x_3} = x_2,$$

$$\frac{\partial f_2}{\partial x_1} = 0, \quad \frac{\partial f_2}{\partial x_2} = 2x_2, \quad \frac{\partial f_2}{\partial x_3} = 1.$$

Ezért a 2×3 Jacobi-mátrix:

$$J_f(x) = \begin{bmatrix} 1 & x_3 & x_2 \\ 0 & 2x_2 & 1 \end{bmatrix}, \qquad J_f(1,2,3) = \begin{bmatrix} 1 & 3 & 2 \\ 0 & 4 & 1 \end{bmatrix}.$$

A VJP:

$$J_f(1,2,3)^\top g = \begin{bmatrix} 1 & 0 \\ 3 & 4 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 5 \\ 0 \end{bmatrix}.$$

Alakellenőrzés: $J_f^\top$ mérete 3×2, g mérete 2×1, az eredmény 3×1, valóban az inputtér gradiense.

F3 megoldás. A szükséges kód:

```
# lab/n015/f3_solution.py
import numpy as np

def affine_grads(X, W, b, G):
    assert X.dtype == np.float64 and W.dtype == np.float64
    assert G.dtype == np.float64 and b.dtype == np.float64
    dX = G @ W
    dW = G.T @ X
    db = G.sum(axis=0)
    return dX, dW, db

X = np.array([[1.0, 0.0, 2.0],
              [0.0, 1.0, 1.0]], dtype=np.float64)
W = np.array([[1.0, 2.0, 0.0],
              [0.0, 1.0, 2.0]], dtype=np.float64)
b = np.array([-1.0, 1.0], dtype=np.float64)
G = np.array([[2.0, -1.0],
              [0.0, -3.0]], dtype=np.float64)

dX, dW, db = affine_grads(X, W, b, G)
print("dX ="); print(dX)
print("dW ="); print(dW)
print("db ="); print(db)

expected_dX = np.array([[2.0, 3.0, -2.0],
```

```
                        [0.0, -3.0, -6.0]])
expected_dW = np.array([[2.0, 0.0, 4.0],
                        [-1.0, -3.0, -5.0]])
expected_db = np.array([2.0, -4.0])
np.testing.assert_allclose(dX, expected_dX, atol=1e-12, rtol=1e-12)
np.testing.assert_allclose(dW, expected_dW, atol=1e-12, rtol=1e-12)
np.testing.assert_allclose(db, expected_db, atol=1e-12, rtol=1e-12)
assert dX.shape == (2, 3) and dW.shape == (2, 3) and db.shape == (2,)
```

A fenti kód mért kimenete (Python 3.12.3, NumPy 2.5.4, float64, CPU, egyszálú futás: OMP/OpenBLAS/MKL=1; a kód nem használ véletlen számot). Az `assert_allclose` hívások és az alakellenőrzés hiba nélkül lefutottak:

```
dX =
[[ 2.  3. -2.]
 [ 0. -3. -6.]]
dW =
[[ 2.  0.  4.]
 [-1. -3. -5.]]
db =
[ 2. -4.]
```

Kézi mátrixszorzással:

$dX = GW$, ahol G mérete 2×2, W mérete 2×3, tehát $dX \in \mathbb{R}^{2\times 3}$:

$$dX = \begin{bmatrix} 2 & 3 & -2 \\ 0 & -3 & -6 \end{bmatrix}.$$

$dW = G^\top X$, mérete 2×3:

$$dW = \begin{bmatrix} 2 & 0 & 4 \\ -1 & -3 & -5 \end{bmatrix}.$$

A biasgradiens a G oszlopösszege:

$$db = (2 + 0,\ -1 - 3) = (2, -4).$$

A kód a `b` értékét a gradiensben nem használja, ami helyes: a biasgradiens független a bias aktuális értékétől. Szélsőeset-ellenőrzés: $G = 0$ esetén mindhárom gradiens nulla, a függvény `(N×d, m×d, m)` alakkal tér vissza; $N = 1$ esetén `db=G[0,:]`.

F4 megoldás. A hiba oka: a kód a batch-soros `G@W` irány helyett `W.T@G`-t ír. Az absztrakt oszlopvektoros szabály valóban $W^\top g_i$ egy megfigyelésre, de a batchelt soros tárolásban a transzponálás után a teljes mátrixszorzat `G@W`; a G sorai a $g_i^\top$ vektorok. A `W.T@G` alakja `(d×m) @ (N×m)`, ami csak akkor szorozható össze, ha $N = m$; ha $N \neq m$, a NumPy futásidejű `ValueError`-t ad a matmul belső méretének eltéréséről (a mért üzenet: `matmul: Input operand 1 has a mismatch in its core dimension 0 ...`). Ha véletlenül $N = m$, a szorzás lefut, de `(d×m)` alakú, tartalmában rossz eredményt ad a helyes `(N×d)` helyett. Következmény: a tipikus esetben futásidejű matmul-alakhiba, az $N = m$ különleges esetben pedig numerikusan rossz gradiens, amely a súlyfrissítést elrontja.

Javítás:

```
def buggy_dX_corrected(X, W, G):
    return G @ W  # N x d
```

Ellenőrzés az F3 példáján. Itt $N = m = 2$, ezért a hibás változat lefut, és a mért eredmény `[[2,-1],[4,-5],[0,-6]]`, alakja `(3,2)`, miközben a helyes `dX` alakja `(2,3)`. A javított változat a fenti `dX` értéket adja; a következő ellenőrzés f64 alatt hiba nélkül lefut:

```
expected_dX = np.array([[2.0, 3.0, -2.0],
                        [0.0, -3.0, -6.0]])
np.testing.assert_allclose(buggy_dX_corrected(X, W, G), expected_dX,
                           rtol=1e-12, atol=1e-12)
```

F5 megoldásminta. A kérdés memória- és időkorlátra vonatkozó átviteli kérdés, nem matematikai azonosság: a teljes Jacobi-mátrix 1000×1000 f64 elem, azaz $8 \cdot 10^6$ bájt $= 8$ MB $\approx 7{,}63$ MiB csak a mátrixra; a VJP egyetlen 1000 hosszú vektort ad vissza. A terv:

- **Kérdés:** azonos matematikai eredményt feltételezve, a Jacobi-anyagiasítás vagy a VJP kisebb csúcsmemóriájú-e CPU-n.
- **Baseline:** `J = torch.autograd.functional.jacobian(f, x)`, majd `v1 = J.T @ g`; ez anyagiasítja a teljes mátrixot. A `jacobian` alapértelmezett `vectorize=False` beállítása mellett a Jacobi minden sorára külön `autograd.grad` hívást végez (ezt a torch 2.15.0 dokumentációja rögzíti), ezért a mátrix előállítása soronkénti számolással történik.
- **Változat:** `y = f(x)`, `loss = torch.dot(y,g)`, `v2, = torch.autograd.grad(loss,x)`.
- **Fix keret:** `dtype=float64`, CPU, `torch.manual_seed(20261020)`, W véletlen 1000×1000 mátrix, x és g rögzített vektor; warmup háromszor, mérés ötvenszer.
- **Metrika:** medián és interkvartilis terjedelem a falidőből, valamint a folyamat csúcsmemóriája (`ru_maxrss`, Linuxon KiB-ban); minden ismétlés előtt új folyamat a nyomkövető overhead elkerülésére. Mivel a `ru_maxrss` a teljes folyamat csúcsát méri (benne az interpreter és az importok memóriájával), a jelentett érték a forward-only baseline folyamat csúcsához viszonyított különbség; a nyers értéket is rögzíteni kell. A csúcsmemória lekérdezése:

  ```
  import resource
  peak_kib = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
  ```

- **Kontroll:** `torch.allclose(v1, v2, rtol=1e-9, atol=1e-9)` igaz legyen, és mindkét output alakja `(1000,)`. A 10^{-12}-es küszöb 1000 tagú skalárszorzatoknál túl szigorú lehet, mert a két számítás összegzési sorrendje eltér; a 10^{-9} érték a protokoll paramétere, nem mért eredmény, a tényleges eltérést a mérés rögzíti.
- **Értelmezési korlát:** az eredmény erre a toy leképezésre, e framework-verzióra és CPU-ra érvényes; a mérés overheadje, a kernelindítási küszöb és a $d = m$ szimmetria miatt nem általános GPU-, batch- vagy termelési állítás. A nyers táblában a két módszer `wall_time_median`, `wall_time_iqr`, `ru_maxrss_kib` értéke, valamint a kontroll `allclose` eredménye szerepel.

4.3.10 Tipikus tévedés, önellenőrzés és visszatekintés

A leggyakoribb hiba a transzponálási konvenció elhallgatása. Legyen $f(x) = Wx$, ahol

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad g = (1, -1)^\top.$$

Itt $m = 2$ és $d = 3$, tehát a Jacobi-mátrix 2×3-as. Ekkor $J_f = W$, a helyes VJP

$$J_f^\top g = W^\top g = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = (-3, -3, -3)^\top,$$

ami 3 hosszú, vagyis az inputtérbeli gradiens. Ha valaki tévesen $J_f g = Wg$-t próbál számítani, az már alakhibát ad, mert a 2×3-as mátrix nem szorozható a 2 hosszú vektorral; a JVP-hez 3 hosszú irányvektor kellene. Ha viszont $m = d$, a tévedés alakhiba nélkül is rejtve maradhat: a JVP és a VJP is d hosszú vektort ad, csak más tartalommal, ezért ilyenkor különösen fontos a szabály ismerete. Batch-soros tárolásnál a fenti affin szabály szerint `dX=G@W`, nem `W.T@G`; a kettő egyetlen megfigyelés oszlopgradiensére átszámítva $g_i^\top W = (\nabla_{x_i} L)^\top$.

Önellenőrzés ehhez a fejezethez: minden láncszabály-szorzatnál a szomszédos Jacobi-méreteknek illeszkedniük kell ($k \times m$ és $m \times d$); a VJP bemenete m-hosszú, kimenete d-hosszú; a mean-redukció $1/N$-je a G-ben van, nem a $W^\top$-ben. Ha a kód olyan gradiensvektort ad, amelynek hossza m a kívánt d helyett, akkor szinte biztosan JVP-t számoltunk VJP helyett.

Visszatekintés: a 4.2. Parciális derivált és gradiens alfejezetben a gradiens oszlopvektor volt; itt ezért lett a láncszabályból $J_f^\top g$. A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság legkisebb négyzetes gradiense, $X^\top(Xw - y)$, ugyanennek a VJP-formának a speciális esete: $f(w) = Xw - y$, $J_f = X$, és a kimeneti gradiens az $r = Xw - y$ reziduum.

4.3.11 Rövid összefoglaló

A Jacobi-mátrix az $f : \mathbb{R}^d \to \mathbb{R}^m$ deriváltjának $m \times d$ mátrixa, sora az output, oszlopa az input. Differenciálható kompozícióban a Jacobikat a $J_\varphi J_f$ sorrendben szorozzuk. A backprop nem a teljes Jacobit építi fel, hanem a VJP-t: $J_f^\top g$, ahol $g = \nabla_{f(x)} L$. Az affin rétegre $Z = XW^\top + b$ esetén a gradiensek: $\nabla_X L = GW$, $\nabla_W L = G^\top X$, és $\nabla_b L$ a G oszlopösszege. A mean-redukció $1/N$ szorzója a G-ben van, nem a transzponálási szabályban. A kidolgozott f64 példa mért VJP-értéke $(5, -5)^\top$, és az autograd ellenőrzés ugyanezt a $J^\top g$ irányt teszteli.

4.3.12 Továbblépés

A 4.4. I. ellenőrző pont: matematika és kód megfelelése ellenőrző pont a matematika és kód megfelelését vizsgálja. Ott a most levezetett affin gradiensképleteket, a JVP/VJP irányt és a numerikus toleranciát egy összetettebb, kis dimenziós ellenőrzési feladaton kell együtt használni.

4.3.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

4.4 I. ellenőrző pont: matematika és kód megfelelése

4.4.1 Cél, előismeret és induló becslés

Ez az ellenőrző pont a 4.1.–4.3. láncát zárja le: egyetlen kis számítási gráfon keresztülmegyünk a jelöléstől a kézi visszaterjesztésen át a véges differenciás kódellenőrzésig. A cél nem egy új rétegtípus bevezetése, hanem az, hogy a matematikai képlet és a kód közötti megfelelést típus szerint ellenőrizni tudjuk. Az előismeret a 4.3. Jacobi-mátrix, láncszabály és VJP láncszabály-, VJP- és affinréteg-anyaga; a 3.4. Sajátérték, SVD és spektrális határ normafogalmát csak a numerikus hibabecslés egy pontján idézzük vissza.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Ajánlott tanulási útvonal és ellenőrző pontok” szakasza adja meg; itt csak az eltérés: a PASS-szabály (a) pontja (a labor minden `assert`-je és toleranciaellenőrzése átmegy) itt a kézi backward és a központi differencia egyezését jelenti a fejezet toleranciáján belül; a súlypont a D1 (matematikai helyesség) és a D2 (kód és futtathatóság).

Induló becslés: egy $m = 500$, $d = 1000$ méretű affin réteg belső Jacobi-mátrixa $500 \cdot 1000 = 5 \cdot 10^5$ skalár. A backpropagation viszont nem ezt a mátrixot tárolja köztes mennyiségként, hanem a kimeneti gradienst és a súlymátrixszal vett szorzatokat. (A W paramétermátrix és a $\nabla_W L$ gradiens természetesen egyenként $5 \cdot 10^5$ skalár; ezek a modell paraméterei, nem a Jacobi-mátrix köztes másolatai.) Becsülje meg, mekkora a lánc köztes tárigénye a VJP-alapú visszaterjesztésnél ennél a rétegnél! (Várható válasz: a z-gradiens $m = 500$ eleme, az x-gradiens $d = 1000$ eleme, tehát nagyságrendileg 10^3 skalár köztes mennyiség, szemben a teljes $5 \cdot 10^5$-ös Jacobi-mátrixszal.)

4.4.2 A gradiens alakja skalár, vektor és tenzor bemenetre

A kiinduló konvenciót egyszer rögzítjük.

> **Definíció (gradiens alakja).** Ha $L : \mathbb{R}^{a_1 \times \cdots \times a_k} \to \mathbb{R}$ differenciálható valós értékű függvény, akkor $\nabla_X L$ az X-szel azonos alakú tenzor, és
>
> $$(\nabla_X L)_{i_1 \cdots i_k} = \frac{\partial L}{\partial X_{i_1 \cdots i_k}}.$$
>
> Speciálisan $x \in \mathbb{R}^d$ esetén $\nabla_x L \in \mathbb{R}^d$; $W \in \mathbb{R}^{m \times d}$ esetén $\nabla_W L \in \mathbb{R}^{m \times d}$.

Tehát a „gradiens alakja megegyezik a paraméter alakjával” konvenciót használjuk mindvégig. Ez összhangban van azzal, ahogyan az autograd-rendszerek a `.grad` tenzort visszaadják; az autograd mechanizmusának hivatalos hátteréhez lásd [D05], de a jelen levezetés és kód saját, NumPy-alapú ellenőrzés.

4.4.3 A kiválasztott kis gráf

A teljes gráf három lépésből áll: affin réteg, elemenkénti sima nemlinearitás, majd egy olyan skalár veszteség, amelynek gradiense kellemesen egyszerű.

Legyen $x \in \mathbb{R}^d$ a bemenet, $W \in \mathbb{R}^{m \times d}$ a súly, $b \in \mathbb{R}^m$ az eltolás. Definiáljuk:

$$z = Wx + b, \qquad y_j = \phi(z_j), \quad j = 1, \dots, m, \qquad L = \frac{1}{2}\|y\|_2^2 = \frac{1}{2}\sum_{j=1}^{m} y_j^2.$$

Itt $\phi : \mathbb{R} \to \mathbb{R}$ folytonosan differenciálható elemenkénti függvény, $\Phi(z) = (\phi(z_1), \dots, \phi(z_m))^\top$. A veszteséget szándékosan egyszerű, $\frac{1}{2}\sum y^2$ alakúnak választottuk, amelyre $\nabla_y L = y$; ez nem MSE és nem a regularizálás $\frac{\lambda}{2}\|W\|_F^2$ tagja, ahol $\lambda \geq 0$ a regularizáció erőssége.

4.4.4 Lemma: a teljes gráf visszaterjesztési képletei

Lemma. A fenti gráfban legyen $g = \nabla_z L$. Ekkor

$$\nabla_y L = y, \qquad g = y \odot \Phi'(z), \qquad \nabla_x L = W^\top g, \qquad \nabla_W L = g\, x^\top, \qquad \nabla_b L = g,$$

ahol $\odot$ elemenkénti szorzás és $\Phi'(z) = (\phi'(z_1), \dots, \phi'(z_m))^\top$.

Bizonyítás. A bizonyítást elemenként végezzük, és minden lépésnél ellenőrizzük az alakot.

1. $L = \frac{1}{2}\sum_{j=1}^{m} y_j^2$, ezért
$$\frac{\partial L}{\partial y_j} = y_j,$$
tehát $\nabla_y L = y \in \mathbb{R}^m$.

2. Az $y_j = \phi(z_j)$ elemenkénti műveletben z_j csak y_j-t befolyásolja, ezért a láncszabállyal
$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial y_j}\frac{dy_j}{dz_j} = y_j\, \phi'(z_j).$$
A vektoralak $g = y \odot \Phi'(z)$, ahol mindkét tényező $\mathbb{R}^m$-beli.

3. Az affin rész: $z_j = \sum_{k=1}^{d} W_{jk} x_k + b_j$. Az x_i bemeneti koordináta minden z_j-ben a W_{ji} együtthatóval jelenik meg, tehát
$$\frac{\partial L}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial L}{\partial z_j}\frac{\partial z_j}{\partial x_i} = \sum_{j=1}^{m} g_j W_{ji} = (W^\top g)_i.$$
Az eredmény $W^\top \in \mathbb{R}^{d\times m}$ és $g \in \mathbb{R}^m$ szorzata, azaz $\nabla_x L \in \mathbb{R}^d$. Itt a $W^\top$ szorzó a VJP-irány: a láncban a z-gradienst visszük át a bemenetre.

4. A W_{ji} súly szerinti parciális deriváltban csak z_j függ W_{ji}-től, és $\partial z_j / \partial W_{ji} = x_i$, ezért
$$\frac{\partial L}{\partial W_{ji}} = g_j\, x_i = (g\, x^\top)_{ji}.$$
Az outer szorzat $g \in \mathbb{R}^m$ és $x \in \mathbb{R}^d$ tényezőkből $\mathbb{R}^{m\times d}$ mátrixot ad, azonosat W alakjával.

5. A b_j eltolásra $\partial z_j / \partial b_j = 1$, így
$$\nabla_b L = g \in \mathbb{R}^m.$$

Ezzel a gráf minden élén megkaptuk a visszaterjesztett gradienst, és minden lépésben feltüntettük a tenzoralakot. A batchos $X \in \mathbb{R}^{N\times d}$ eset 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben a GW és $G^\top X$ alakú általánosítás; az itteni $N = 1$ eset az outer szorzatos formula.

4.4.5 Kézi számpélda: P0 pont

Legyen $d = 3$, $m = 2$, $\phi(t) = t^2$, és

$$x = \begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix}, \qquad W = \begin{pmatrix} 0{,}5 & -1 & 2 \\ 1 & 0{,}5 & -1 \end{pmatrix}, \qquad b = \begin{pmatrix} 0{,}2 \\ -0{,}3 \end{pmatrix}.$$

Előremenet (forward):

$$z = Wx + b = \begin{pmatrix} -3{,}3 \\ 2{,}7 \end{pmatrix}, \quad y = \Phi(z) = \begin{pmatrix} 10{,}89 \\ 7{,}29 \end{pmatrix}, \quad L = \tfrac{1}{2}(10{,}89^2 + 7{,}29^2) = 85{,}8681.$$

Mivel $\phi'(t) = 2t$, a visszaterjesztés:

$$g = y \odot \Phi'(z) = \begin{pmatrix} 10{,}89 \cdot (-6{,}6) \\ 7{,}29 \cdot 5{,}4 \end{pmatrix} = \begin{pmatrix} -71{,}874 \\ 39{,}366 \end{pmatrix}.$$

Ezután

$$\nabla_x L = W^\top g = \begin{pmatrix} 3{,}429 \\ 91{,}557 \\ -183{,}114 \end{pmatrix},$$

$$\nabla_W L = g\, x^\top = \begin{pmatrix} -71{,}874 & -143{,}748 & 71{,}874 \\ 39{,}366 & 78{,}732 & -39{,}366 \end{pmatrix},$$

$$\nabla_b L = g = \begin{pmatrix} -71{,}874 \\ 39{,}366 \end{pmatrix}.$$

4.4.6 Új pont: P1 kontrollszámítás

Az ellenőrző laborhoz egy második, kisebb gradiensű pontot is rögzítünk, hogy a hibamérce nullközeli értékeken is értelmes legyen.

$$x = \begin{pmatrix} 0{,}5 \\ -2 \\ 1{,}5 \end{pmatrix}, \qquad W = \begin{pmatrix} 2 & 0 & -1 \\ 0 & 1 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 0{,}1 \\ 0 \end{pmatrix}.$$

Az előremenet (forward):

$$z = \begin{pmatrix} -0{,}4 \\ -0{,}5 \end{pmatrix}, \quad y = \begin{pmatrix} 0{,}16 \\ 0{,}25 \end{pmatrix}, \quad L = 0{,}04405.$$

A programkimenetben ezek a z, y, L értékek a P1 sorában jelennek meg, ezért a kézi számítás a futás eredményével közvetlenül összevethető.

A hátramenet (backward):

$$g = \begin{pmatrix} 0{,}16 \cdot (-0{,}8) \\ 0{,}25 \cdot (-1) \end{pmatrix} = \begin{pmatrix} -0{,}128 \\ -0{,}25 \end{pmatrix},$$

$$\nabla_x L = W^\top g = \begin{pmatrix} -0{,}256 \\ -0{,}25 \\ -0{,}122 \end{pmatrix},$$

$$\nabla_W L = g\, x^\top = \begin{pmatrix} -0{,}064 & 0{,}256 & -0{,}192 \\ -0{,}125 & 0{,}5 & -0{,}375 \end{pmatrix},$$

$$\nabla_b L = \begin{pmatrix} -0{,}128 \\ -0{,}25 \end{pmatrix}.$$

4.4.7 A véges differenciás ellenőrzés hibamérlege

Az analitikus gradienst középső differenciával hasonlítjuk össze, mert ennek csonkolási hibája ε^2 rendű, szemben az előredifferencia ε rendű hibájával. Egy x_i irányú perturbációra

$$\Delta_i(\varepsilon) = \frac{L(x + \varepsilon e_i) - L(x - \varepsilon e_i)}{2\varepsilon}.$$

Itt $\Delta_i(\varepsilon)$ az i-edik koordináta szerinti középső differencia operátora, a 4.1. alfejezetben bevezetett Δ_h megfelelője; a Δ jelölés egyben elkerüli az ütközést a feladatok F1–F5 címkéivel. Ha az iránymenti harmadik deriváltak korlátosak, a Taylor-sorral

$$\Delta_i(\varepsilon) = \frac{\partial L}{\partial x_i} + O(\varepsilon^2) + O\!\left(\frac{u\,|L|}{\varepsilon}\right),$$

ahol u a gépi pontosság f64 esetén ($u \approx 2{,}2 \cdot 10^{-16}$), a második tag pedig a két közeli függvényérték kivonásának kerekítési hibája. Az $O(\varepsilon^2)$ csonkolási tag nagysága az iránymenti harmadik derivált korlátjától függ, nem a pont simaságától önmagában. A ϕ simasága emellett azért kell, hogy a klasszikus derivált egyáltalán létezzen: a ReLU (rectified linear unit, $\max(0, t)$) nullpontjában nincs ilyen, ezért a véges differenciás ellenőrzést töréspontól távol vagy sima ϕ-re végezzük.

A két hibaforrás ellentétesen függ ε-tól: a csonkolás ε^2-tel csökken, a kerekítés viszont $1/\varepsilon$-nal nő. Az optimális lépésköz ezért nagyságrendileg $(u|L|)^{1/3}$. A P0 példában $L \approx 86$, ezért

$$(2{,}2 \cdot 10^{-16} \cdot 86)^{1/3} \approx (1{,}9 \cdot 10^{-14})^{1/3} \approx 2{,}7 \cdot 10^{-5}.$$

A laborban $\varepsilon = 10^{-6}$ f64 lépésközt használunk. A $2{,}7 \cdot 10^{-5}$ a P0 pont optimális lépésköze; a P1 ponton viszont $|L| = 0{,}04405$, ezért ott az optimum $(2{,}2 \cdot 10^{-16} \cdot 0{,}04405)^{1/3} \approx 2{,}1 \cdot 10^{-6}$, amihez a 10^{-6} közel esik. Mivel a labor egyetlen közös lépésközt használ mindhárom ponton, a 10^{-6} választás a P1 optimumához közelít, a P0-n pedig a mért maximális eltérés $1{,}46 \cdot 10^{-8}$, ami megfelel a kerekítési tag $u|L|/\varepsilon \approx 1{,}9 \cdot 10^{-8}$ nagyságrendjének, és jóval a 10^{-7}-es tűrés alatt marad. Az optimum tehát pontonként változik; a labornak nem az egyes pontok optimumát kell eltalálnia, hanem olyan közös lépésközt, amely mellett minden pont a tűrésen belül marad. Az ellenőrzést a stabil hibaképlettel végezzük:

$$|a - b| \;\le\; \tau_a + \tau_r \max\{|a|, |b|\},$$

ahol τ_a az abszolút, τ_r a relatív tűrés. A nullközeli gradiensnél az abszolút tag dominál, ezért a P1 pont kis gradienseinél is van értelmes elfogadási küszöb.

4.4.8 Labor: NumPy forward, backward és véges differencia

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor egyetlen fájl a `lab/n016/` munkakönyvtárban. A futtatás CPU-n történik, float64 NumPy dtype-pal; GPU vagy PyTorch nem szükséges. A futtatási parancs:

```
python n016_gradcheck.py --seed 20261020
```

Az alábbi kód a P0 és P1 ponton, valamint egy seedelt véletlen ponton az analitikus gradienst a központi differenciával veti össze, és a forward köztes értékeit (z, y, L) pontonként kiírja, hogy a fenti kézi számításokkal összevethetők legyenek. A kód futás közben assertekkel is megáll, ha bármelyik ellenőrzés nem teljesül.

```
"""n016_gradcheck.py - teljes kis gráf gradiensellenőrzése.

A gráf: z = Wx + b, y = z**2, L = 0.5 * sum(y**2).
A kód f64-ben, középső differenciával ellenőrzi x, W és b gradiensét,
majd egy normalizált v irányban az iránymenti deriváltat is.
"""
import argparse
import numpy as np

def phi(z):
    return np.square(z)

def phi_prime(z):
    return 2.0 * z

def loss(x, W, b):
    y = phi(W @ x + b)
    return 0.5 * np.sum(np.square(y))

def backward(x, W, b):
    z = W @ x + b
    y = phi(z)
    g_z = y * phi_prime(z)
    g_x = W.T @ g_z
    g_W = np.outer(g_z, x)
    g_b = g_z
    return g_x, g_W, g_b

def close(a, b, atol=1e-7, rtol=1e-7):
    diff = np.abs(a - b)
    scale = np.maximum(np.abs(a), np.abs(b))
    return bool(np.all(diff <= atol + rtol * scale))

def finite_diff_all(x, W, b, eps=1e-6):
    num_x = np.zeros_like(x)
    for i in range(x.shape[0]):
        xp, xm = x.copy(), x.copy()
        xp[i] += eps
        xm[i] -= eps
        num_x[i] = (loss(xp, W, b) - loss(xm, W, b)) / (2 * eps)

    num_W = np.zeros_like(W)
    for i in range(W.shape[0]):
```

```
        for j in range(W.shape[1]):
            Wp, Wm = W.copy(), W.copy()
            Wp[i, j] += eps
            Wm[i, j] -= eps
            num_W[i, j] = (loss(x, Wp, b) - loss(x, Wm, b)) / (2 * eps)

    num_b = np.zeros_like(b)
    for i in range(b.shape[0]):
        bp, bm = b.copy(), b.copy()
        bp[i] += eps
        bm[i] -= eps
        num_b[i] = (loss(x, W, bp) - loss(x, W, bm)) / (2 * eps)

    return num_x, num_W, num_b

def check_point(name, x, W, b, eps=1e-6):
    g_x, g_W, g_b = backward(x, W, b)
    n_x, n_W, n_b = finite_diff_all(x, W, b, eps=eps)

    z = W @ x + b
    y = phi(z)
    print(f"{name}: z={np.array2string(z, precision=4)}, "
          f"y={np.array2string(y, precision=4)}, L={loss(x, W, b):.6f}")

    ok_x = close(g_x, n_x)
    ok_W = close(g_W, n_W)
    ok_b = close(g_b, n_b)

    v = np.arange(1.0, x.shape[0] + 1, dtype=np.float64)
    v = v / np.linalg.norm(v)
    dir_num = (loss(x + eps * v, W, b) - loss(x - eps * v, W, b)) / (2 * eps)
    dir_ana = float(v @ g_x)
    ok_dir = close(dir_ana, dir_num)

    print(f"{name}: x={ok_x}, W={ok_W}, b={ok_b}, iranymenti={ok_dir}")
    print(f"  max|x-hiba|={np.max(np.abs(g_x - n_x)):.2e}, "
          f"max|W-hiba|={np.max(np.abs(g_W - n_W)):.2e}, "
          f"max|b-hiba|={np.max(np.abs(g_b - n_b)):.2e}")
    assert ok_x and ok_W and ok_b and ok_dir, f"{name}: gradiensellenorzes nem teljesult"

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--seed", type=int, default=20261020)
    args = ap.parse_args()
    rng = np.random.default_rng(args.seed)

    # P0: a szovegben kezzel kiszamolt pont
    x0 = np.array([1.0, 2.0, -1.0], dtype=np.float64)
    W0 = np.array([[0.5, -1.0, 2.0],
                   [1.0, 0.5, -1.0]], dtype=np.float64)
    b0 = np.array([0.2, -0.3], dtype=np.float64)
    check_point("P0", x0, W0, b0)

    # P1: masodik, kis gradiensu kontrollpont
    x1 = np.array([0.5, -2.0, 1.5], dtype=np.float64)
    W1 = np.array([[2.0, 0.0, -1.0],
                   [0.0, 1.0, 1.0]], dtype=np.float64)
    b1 = np.array([0.1, 0.0], dtype=np.float64)
    check_point("P1", x1, W1, b1)

    # Seedelt uj pont: a seed rogziti a veletlen pontot
    xr = rng.standard_normal(3)
    Wr = rng.standard_normal((2, 3))
```

```
    br = rng.standard_normal(2)
    check_point("random_seed", xr, Wr, br)

if __name__ == "__main__":
    main()
```

A kód mért kimenete (Python 3.12.3, NumPy 2.5.4, float64, CPU, egyszálú futás: OMP/OpenBLAS/MKL=1, seed 20261020):

```
P0: z=[-3.3  2.7], y=[10.89  7.29], L=85.868100
P0: x=True, W=True, b=True, iranymenti=True
  max|x-hiba|=4.63e-09, max|W-hiba|=7.83e-09, max|b-hiba|=1.46e-08
P1: z=[-0.4 -0.5], y=[0.16 0.25], L=0.044050
P1: x=True, W=True, b=True, iranymenti=True
  max|x-hiba|=1.22e-11, max|W-hiba|=1.75e-11, max|b-hiba|=2.02e-12
random_seed: z=[-3.4743 -6.6583], y=[12.0708 44.3324], L=1055.534829
random_seed: x=True, W=True, b=True, iranymenti=True
  max|x-hiba|=1.84e-07, max|W-hiba|=2.05e-07, max|b-hiba|=1.64e-07
```

A fenti mért futásban a P0 pont maximális abszolút eltérése $1{,}46 \cdot 10^{-8}$, P1-nél $1{,}75 \cdot 10^{-11}$; a `close` függvény ezért $\tau_a = \tau_r = 10^{-7}$ mellett mindenhol zöld eredményt ad. A kimenet köztes értékei a kézi számítással is egyeznek: P0-nál $z = (-3{,}3; 2{,}7)$, $y = (10{,}89; 7{,}29)$, $L = 85{,}8681$, P1-nél $z = (-0{,}4; -0{,}5)$, $y = (0{,}16; 0{,}25)$, $L = 0{,}04405$. A seedelt véletlen ponton a maximális abszolút eltérés $2{,}05 \cdot 10^{-7}$, ami a nagyobb gradiensértékek miatt a relatív taggal együtt még a tűrésen belül van. A seedelt véletlen pont egy független harmadik ellenőrzés; önmagában azonban nem helyettesíti a levezetést, csak numerikus hibakeresési bizonyítékot ad.

4.4.9 Gyakori tévedés: egyetlen pont nem általános bizonyíték

Tipikus hiba lenne azt állítani, hogy „a gradienskód helyes, mert egy ponton megegyezett a véges differenciával". Ez logikailag nem következik. Ellenpélda a valós függvények szintjén:

$$f(t) = t^2, \qquad g(t) = t^2 + 2(t-1)^2.$$

A $t = 1$ pontban $f(1) = g(1) = 1$ és $f'(1) = g'(1) = 2$, tehát érték és gradiens is egyezik; ennek ellenére $f(2) = 4$, míg $g(2) = 6$, tehát a két függvény máshol különbözik. Ugyanígy egy hibás backward megvalósítás lehet, hogy P1-en a kis gradiensek és a laza tűrés miatt átmegy, P0-on viszont eltér. Az F4 feladat erre épít.

4.4.10 Önellenőrzés és korábbi fogalmak áttekintése

- Minden képletnél az alakok: $g_x \in \mathbb{R}^d$, $g_W \in \mathbb{R}^{m \times d}$, $g_b \in \mathbb{R}^m$.
- A VJP-irány a $W^\top g$ szorzat; az iránymenti véges differencia ezzel ellenőrizhető: $v^\top \nabla_x L \approx [L(x + \varepsilon v) - L(x - \varepsilon v)]/(2\varepsilon)$.
- A 4.3. Jacobi-mátrix, láncszabály és VJP mean-redukciójánál az $1/N$ skála a G mátrixban jelent meg; mivel itt egyetlen pontra és $\frac{1}{2}\sum y^2$ veszteségre dolgozunk, $1/N$ nem lép fel. A $\frac{1}{2}$ szorzó a y szerinti deriváláskor kiesik, mert $\frac{\partial}{\partial y_j}(\frac{1}{2}y_j^2) = y_j$; ezért $g = y \odot \Phi'(z)$-ben már nem jelenik meg külön tényezőként.
- A 3.4. Sajátérték, SVD és spektrális határ alfejezetből a spektrális norma a lineáris közelítés hibájára legrosszabb irányban is érvényes korlátot ad; a jelen fejezet hibamérlege azonban nem használja, mert a központi differencia csonkolási tagját a harmadik derivált korlátjával becsüljük.

4.4.11 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért tárol el kevesebb számot a VJP-alapú backpropagation, mint a teljes Jacobi-mátrix, ha $d = 1000$ és $m = 500$. Adja meg mindkét tárigényt skalárok számában, és mondja ki a gradiens alakjára vonatkozó konvenciót.

F2 (levezetési, nehézség 2). Tekintse ugyanazt a gráfot, de a veszteség legyen

$$L_A = \frac{1}{2}\, y^\top A\, y, \qquad A \in \mathbb{R}^{m\times m},\ A = A^\top.$$

Vezesse le, hogy $\nabla_y L_A = Ay$, és adja meg ezzel az új $g = \nabla_z L_A$ képletét. Ellenőrizze, hogy $A = I$ visszaadja a lemmabeli y-t.

F3 (implementációs, nehézség 2). Írjon egy `safe_close(a, b, atol, rtol)` NumPy-függvényt a stabil hibaképlethez, és alakítsa át a labor `check_point` függvényét úgy, hogy a tűréseket `atol`/`rtol` paraméterként kapja, a stabil képlettel ellenőrizze, és `assert`-tel álljon meg hiba esetén. Adjon egy kis tesztet, amelyben a nullközeli értékre az abszolút tűrés dominál.

F4 (hibakeresési, nehézség 2). Az alábbi hibás backward

```
def backward_bad(x, y, W):
    g_z = y  # hiba: phi'(z) kimaradt
    return W.T @ g_z, np.outer(g_z, x), g_z
```

a P1 ponton $g^x_{\text{bad}} = W^\top y = (0{,}32;\ 0{,}25;\ 0{,}09)^\top$, szemben a korrekt $(-0{,}256;\ -0{,}25;\ -0{,}122)^\top$ értékkel. Mutassa meg, hogy a maximális abszolút eltérés 0,576, tehát egy $\tau_a = 1$ tűrésű teszt zöldet adna. Mi a hiba következménye egy tanítási lépésben, és milyen két módosítással deríthető ki megbízhatóan?

F5 (kísérleti/transzfer, nehézség 3). Tervezzen ellenőrzési protokollt arra a kérdésre, hogy a fenti gradiensképletek float32-es tanítási útvonalon is elfogadhatóan működnek-e. Adjon meg legalább egy lehetséges tervet: hány seedelt pont, milyen ε, milyen baseline, milyen tűrés és milyen következtetési korlát. Ne feltételezzen előre megadott győztest.

4.4.12 Megoldások és indoklás

F1. A teljes J_f mátrix $m \times d = 500\,000$ skalár. A VJP-úton a z-gradiens $m = 500$, a bemeneti gradiens $d = 1000$, tehát a lánc köztes tárigénye nagyságrendileg $1{,}5 \cdot 10^3$ skalár, és a mátrixszorzások a $W^\top g$ irányban végzik el ugyanazt az információáramlást. Ettől külön kell kezelni a paramétereket: W és $\nabla_W L$ egyenként $m \cdot d = 500\,000$ skalár, ezeket a modell tárolja, akár VJP-vel, akár teljes Jacobi-mátrixszal számolunk; a megtakarítás tehát a Jacobi-mátrix köztes példányán van, nem a paramétereken. A konvenció: $\nabla_X L$ alakja megegyezik X alakjával; parciális deriváltja minden X-indexhez egy skalár.

F2. Legyen $L_A = \frac{1}{2}\sum_{i,j} y_i A_{ij} y_j$. Ekkor

$$\frac{\partial L_A}{\partial y_k} = \frac{1}{2}\sum_j A_{kj} y_j + \frac{1}{2}\sum_i y_i A_{ik} = \frac{1}{2}(Ay)_k + \frac{1}{2}(A^\top y)_k = (Ay)_k,$$

mert $A = A^\top$. (Ha A nem szimmetrikus, ugyanez a levezetés $\nabla_y L_A = \frac{1}{2}(A + A^\top)y$-t ad, ami szimmetrikus A esetén (Ay)-ra egyszerűsödik.) Ezért $\nabla_y L_A = Ay$; a lánc második lépése változatlan,

$$g = \nabla_z L_A = (Ay) \odot \Phi'(z).$$

Az $A = I$ esetben $Ay = y$, tehát visszakapjuk a lemmabeli $g = y \odot \Phi'(z)$ formulát. Alakellenőrzés: $A \in \mathbb{R}^{m \times m}$, $y \in \mathbb{R}^m$, tehát $Ay \in \mathbb{R}^m$; az elemenkénti szorzat is $\mathbb{R}^m$.

F3. Egy teljes minimális függvény:

```
def safe_close(a, b, atol=1e-7, rtol=1e-7):
    diff = np.abs(np.asarray(a, dtype=np.float64)
                  - np.asarray(b, dtype=np.float64))
    scale = np.maximum(np.abs(np.asarray(a, dtype=np.float64)),
                       np.abs(np.asarray(b, dtype=np.float64)))
    return bool(np.all(diff <= atol + rtol * scale))
```

A függvény ugyanazt a képletet használja, mint a labor `close` függvénye, csak a tűrések paraméterként jönnek, és a bemeneteket előbb float64-re alakítja. A `check_point` átirata a tűréseket paraméterként veszi át:

```
def check_point(name, x, W, b, eps=1e-6, atol=1e-7, rtol=1e-7):
    g_x, g_W, g_b = backward(x, W, b)
    n_x, n_W, n_b = finite_diff_all(x, W, b, eps=eps)
    ok_x = safe_close(g_x, n_x, atol, rtol)
    ok_W = safe_close(g_W, n_W, atol, rtol)
    ok_b = safe_close(g_b, n_b, atol, rtol)
    v = np.arange(1.0, x.shape[0] + 1, dtype=np.float64)
    v = v / np.linalg.norm(v)
    dir_num = (loss(x + eps * v, W, b) - loss(x - eps * v, W, b)) / (2 * eps)
    dir_ana = float(v @ g_x)
    ok_dir = safe_close(dir_ana, dir_num, atol, rtol)
    assert ok_x and ok_W and ok_b and ok_dir, f"{name}: gradiensellenorzes nem teljesult"
```

A nullközeli teszt kóddal:

```
a = np.array([1e-12])
b = np.array([0.0])
assert safe_close(a, b, atol=1e-7, rtol=1e-7)           # az abszolút ág dönt
assert not safe_close(np.array([1e-6]), np.array([0.0]),
                      atol=1e-7, rtol=1e-7)             # ekkora eltérés nem fér bele
```

Az első esetben `|a-b|=10^{-12}<\tau_a=10^{-7}`, tehát a vizsgálat az abszolút ágon megy át; ez a kívánt viselkedés, mert nullközeli gradiensnél a relatív önmagában túl szigorú lenne. A második esetben `|a-b|=10^{-6}`, ami a `10^{-7}+10^{-7}\cdot10^{-6}` küszöböt meghaladja, ezért a teszt elutasít.

F4. A hiba következménye: a $\phi'(z)$ faktor elhagyása miatt a visszaterjesztett gradiens a nemlinearitás meredeksége nélkül számolódik, ezért a súlyfrissítés iránya és nagysága is torzul; a tanulás lassulhat vagy rossz irányba léphet. A kód javítása: `g_z = y * phi_prime(z)`. Két hibafelismerési mód: (1) a tűrés szigorítása $\tau_a = \tau_r = 10^{-7}$ értékre, és (2) az ellenőrzés legalább három, eltérő nagyságrendű ponton (P0, P1, seedelt véletlen pont), nemcsak egyetlen eseten. A P0 pont azért is fontos, mert ott a $\nabla_x L$ komponenseiben az eltérés 197 körüli, ami azonnal kimutatja a hibát.

F5. Egy lehetséges terv. Kérdés: a f32 változat gradienshibája elfogadható-e. Baseline: a f64 analitikus gradiensek (ezek a viszonyítási alap); a f64 középső differencia $\varepsilon = 10^{-6}$ lépésközzel csak független ellenőrzés. Változó tényező: f32 tárolás és számítás. Lépések: nyolc, rögzített seedű véletlen pont; minden ponton számítunk f64 analitikus gradienst, f32 forward/backward gradienst és f64 FD-referenciát. Az f32 analitikus gradienst a f64 analitikushoz $\tau_a = \tau_r = 10^{-4}$ tűréssel hasonlítjuk; ez körülbelül három nagyságrenddel a f32 gépi pontossága ($u \approx 1{,}2 \cdot 10^{-7}$) fölött van,

de a hiányzó $\phi'(z)$ tényezőhöz hasonló, $O(1)$ hibákat elkapja. Az f32-es FD csak durva kontroll lehet: a hibamérleg szerint a kerekítés $u|L|/\varepsilon$, a csonkolás $O(\varepsilon^2)$, ezért $\varepsilon \approx (u|L|)^{1/3}$, ami a P0 ponton $|L| \approx 86$ mellett $\varepsilon \approx 2 \cdot 10^{-2}$; a 10^{-3} lépésköz ugyanott $u|L|/\varepsilon \approx 10^{-2}$ kerekítési hibát ad, ami a kis gradienskomponenseknél a $10^{-4} + 10^{-4} \max(|a|, |b|)$ tűrést is meghaladja. A negyedfokú veszteségnél a harmadik derivált állandója is nagy (a W koordinátáiban x_i^3 szorzóval), ezért a f32 FD elérhető pontossága a nagy gradiensű pontokon is csak néhány 10^{-3}–10^{-2} nagyságrendű; ehhez $\tau_a = \tau_r = 10^{-2}$ laza tűrés és pontonként, kis ε-rácsból kalibrált lépésköz illik. Szigorú 10^{-4}-es tűrést tehát csak a f32 analitikus gradiens és a f64 analitikus baseline összevetésétől várunk, a f32 FD-től nem. Következtetési korlát: az eredmény csak ennek a kis gráfnak és ennyi pontnak a numerikus viselkedését mutatja; nem bizonyít általános tanítási konvergenciát, és a seedek közti szórás külön vizsgálatot igényel.

4.4.13 Rövid összefoglaló

Egy kis gráfon — affin réteg, elemenkénti sima ϕ, majd $\frac{1}{2}\|y\|^2$ veszteség — teljes visszaterjesztést bizonyítottunk elemenként. A gradiensek: $g = y \odot \phi'(z)$, $\nabla_x L = W^\top g$, $\nabla_W L = g\,x^\top$, $\nabla_b L = g$. Két kézi ponton ellenőriztük az alakokat, majd NumPy-kódot adtunk, amely középső differenciával, f64 lépésközzel és stabil abszolút/relatív hibaképlettel vizsgál. A döntő módszertani állítás: egyetlen pontbeli egyezés nem bizonyítja az implementációt, ezért több pont, az iránymenti derivált ellenőrzése és szigorú tűrés szükséges.

4.4.14 Továbblépés

Az 5.1. Esemény, feltétel és véletlen változó alfejezetben a valószínűségi nyelvre térünk át. A gradiens és VJP-ellenőrzés eszközei a későbbi veszteségfüggvények likelihood-alapú levezetésében és azok numerikus kontrolljában térnek vissza.

4.4.15 Források

- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

5 Valószínűség és statisztika a modellkísérlet mögött

5.1 Esemény, feltétel és véletlen változó

5.1.1 Cél, előismeret és induló kérdés

Ebben az egységben a valószínűségszámítás azon alapfogalmait építjük fel, amelyek a későbbi neurálisháló-veszteségek, likelihood-ok és kiértékelési bizonytalanságok (például konfidenciasávok) nyelvét adják. Feltételezzük a 4.1. Függvény, határérték és derivált határérték- és deriváltfogalmát; mértékelméleti előismeret nem szükséges. A cél kettős: véges mintatéren pontosan tudjunk feltételes valószínűséggel és Bayes-tétellel számolni, folytonos esetben pedig a sűrűségfüggvényt területként értelmezni.

Induló kérdés: egy osztályozó a tesztképek 90%-ában helyesen ismeri fel a *vertical* osztályt, és a *horizontal* képeknek csak 5%-át sorolja tévesen *vertical* címkével. Ha a teszthalmazon 20% a *vertical* képek aránya, és az osztályozó egy képre *vertical* címkét ad, mekkora a valószínűsége, hogy a kép tényleg *vertical*? Első becslésként sokan 90%-ot válaszolnak; a helyes érték a Bayes-tételből körülbelül 0,818, ahogyan az alábbiakban levezetjük.

5.1.2 Esemény és valószínűség

Definíció (véges valószínűségi modell). Legyen Ω egy véges, nem üres halmaz, a *mintatér*. Egy $P : \Omega \to [0,1]$ függvényt *valószínűségnek* nevezünk, ha

$$\sum_{\omega \in \Omega} P(\omega) = 1.$$

Egy tetszőleges $A \subseteq \Omega$ részhalmazt *eseménynek* hívunk, és valószínűségét

$$P(A) = \sum_{\omega \in A} P(\omega)$$

definiálja. Az üres eseményre $P(\emptyset) = 0$, a biztos eseményre $P(\Omega) = 1$. A komplementer eseményre

$$P(\bar{A}) = 1 - P(A),$$

mert $\Omega = A \cup \bar{A}$ diszjunkt unió. Két tetszőleges A, B eseményre az *addíciós szabály*

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

A $A \cap B$ metszet az az esemény, hogy mindkettő bekövetkezik; a kivonás azért szükséges, mert a metszetet kétszer számoltuk.

Példa (kockadobás). $\Omega = \{1, \dots, 6\}$, minden kimenetelre $P(\omega) = 1/6$. Az $A = \{\text{páros}\} = \{2, 4, 6\}$ és $B = \{\text{hatos}\} = \{6\}$ eseményekre $P(A) = 3/6 = 1/2$, $P(B) = 1/6$.

A gépi tanulásban az Ω gyakran egy megfigyelés összes lehetséges értékének halmaza: például a függelék A. szakaszában leírt GlyphLab egy képénél $Y \in \{0, 1, 2\}$ az orientációs címke (0 = *vertical*, 1 = *horizontal*, 2 = *diagonal*), és a $P(Y = k)$ valószínűségek adják az osztályok eloszlását.

5.1.3 Feltételes valószínűség, függetlenség és Bayes-tétel

A feltételes valószínűség azt méri, hogyan változik egy esemény esélye, ha egy másik esemény bekövetkezését már ismerjük.

Definíció (feltételes valószínűség). Legyen $A, B \subseteq \Omega$ és $P(B) > 0$. Az A esemény B-re vonatkozó feltételes valószínűsége

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

A feltétel lényege, hogy az eredeti Ω helyett a B által meghatározott részhalmazra szűkítünk, majd azon belül nézzük A részarányát. A $P(B) > 0$ feltétel elengedhetetlen: nullvalószínűségű eseményre nem oszthatunk.

Példa. A kockánál $P(B \mid A) = P(\{6\} \mid \{2, 4, 6\}) = \frac{1/6}{1/2} = 1/3$: ha tudjuk, hogy párosat dobtunk, a hatos esélye $1/3$.

Definíció (függetlenség). Az A és B események függetlenek, ha

$$P(A \cap B) = P(A)P(B).$$

Ha $P(B) > 0$, a definíció ekvivalens azzal, hogy $P(A \mid B) = P(A)$. Valóban, a feltételes valószínűség definíciójából $P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A)P(B)}{P(B)} = P(A)$; megfordítva, ha $P(A \mid B) = P(A)$, akkor $P(A \cap B) = P(A \mid B)P(B) = P(A)P(B)$. A B ismerete tehát nem változtat A valószínűségén. A függetlenség nem keverendő össze a diszjunktsággal; két diszjunkt, pozitív valószínűségű esemény soha nem független, mert $P(A \cap B) = 0 \neq P(A)P(B)$.

Tétel (Bayes-tétel és teljes valószínűség tétele, véges mintatéren). Legyen A olyan esemény, amelyre $P(A) > 0$, és legyen $B_1, \dots, B_n$ az Ω egy partíciója, azaz a B_i-k páronként diszjunktak és $\bigcup_{i=1}^{n} B_i = \Omega$, valamint $P(B_i) > 0$ minden i-re. Ekkor

$$P(B_i \mid A) = \frac{P(A \mid B_i)P(B_i)}{\sum_{j=1}^{n} P(A \mid B_j)P(B_j)}.$$

A tétel a [B01] 6. fejezetének és a [B02] 3. fejezetének standard anyaga; a teljesség kedvéért a bizonyítást is megadjuk.

Bizonyítás. Először a feltételes valószínűség definícióját alkalmazzuk kétszer. Egyrészt

$$P(A \cap B_i) = P(A \mid B_i)P(B_i),$$

másrészt ugyanez a metszet felírható

$$P(A \cap B_i) = P(B_i \mid A)P(A)$$

alakban is. A két kifejezést egyenlővé téve és $P(A) > 0$-val osztva kapjuk a Bayes-formula magját:

$$P(B_i \mid A) = \frac{P(A \mid B_i)P(B_i)}{P(A)}. \tag{1}$$

Most a nevezőt fejezzük ki az ismert mennyiségekkel. Mivel a B_j-k partíciót alkotnak, az A esemény felbomlik diszjunkt metszetekre:

$$A = \bigcup_{j=1}^{n} (A \cap B_j).$$

A metszetek valóban diszjunktak: ha $j \neq k$, akkor $(A \cap B_j) \cap (A \cap B_k) \subseteq B_j \cap B_k = \emptyset$, mert a partíció elemei páronként diszjunktak. Az unió valóban A: minden $\omega \in A$ pontosan egy B_j-be esik, tehát pontosan egy $A \cap B_j$-nek eleme. Az additivitás miatt

$$P(A) = \sum_{j=1}^{n} P(A \cap B_j) = \sum_{j=1}^{n} P(A \mid B_j)P(B_j). \tag{2}$$

A (2) kifejezést az (1) nevezőjébe helyettesítve adódik a tétel állítása. □

A (2) összefüggés önmagában is fontos, és *teljes valószínűség tételének* nevezzük: a teljes $P(A)$ valószínűséget a partíció elemein keresztül súlyozott feltételes valószínűségek összegeként kapjuk meg. A bizonyítás lépései: (i) a metszet kétféle feltételes felírása, (ii) diszjunkt felbontás, (iii) additivitás. A tétel feltételei között szerepel $P(A) > 0$ és $P(B_j) > 0$; ha valamelyik nulla, a megfelelő feltételes valószínűség nem definiált, és a formula nem alkalmazható.

5.1.4 Diszkrét eloszlások: Bernoulli és kategorikus

Definíció (diszkrét véletlen változó). Egy $X : \Omega \to \mathbb{R}$ függvényt véletlen változónak hívunk; diszkrét, ha értékkészlete véges vagy megszámlálhatóan végtelen. Ekkor az eloszlását a $p_X(x) = P(X = x)$ valószínűségi tömegfüggvény adja, amelyre $p_X(x) \geq 0$ és $\sum_x p_X(x) = 1$.

A neurális hálózatokban két diszkrét eloszlás alapvető.

Definíció (Bernoulli-eloszlás). Az X véletlen változó Bernoulli-eloszlású $p \in [0, 1]$ paraméterrel, ha $X \in \{0, 1\}$ és

$$P(X = 1) = p, \qquad P(X = 0) = 1 - p.$$

Ez a bináris osztályozás célváltozója: például „vertical vagy nem vertical”. A p paraméter a pozitív osztály valószínűsége.

Definíció (kategorikus eloszlás). Az Y véletlen változó kategorikus eloszlású K osztállyal és $\pi = (\pi_0, \ldots, \pi_{K-1})$ paraméterrel, ha $Y \in \{0, \ldots, K-1\}$ és

$$P(Y=k)=\pi_k, \qquad \pi_k \geq 0, \quad \sum_{k=0}^{K-1} \pi_k = 1.$$

A π vektor tehát valószínűségi vektor: elemei nemnegatívak és összegük 1. A többosztályos kép-osztályozásban $K=3$, a GlyphLab címkéi a fenti $0,1,2$ megfeleltetést követik. Egy m elemű minibatch címkevektora $y \in \{0,\dots,K-1\}^m$, ahogyan a könyv egységes jelölése rögzíti. A modell minden képre egy $\hat{\pi}$ valószínűségi vektort becsül; a kalap a becsült mennyiséget jelöli, tehát $\hat{\pi}$ a valódi π eloszlás becslése. A nyers pontszámokból $(z_0,\dots,z_{K-1})$ a softmax réteg készít normalizált valószínűségeket, $\hat{\pi}_k = e^{z_k} / \sum_{j=0}^{K-1} e^{z_j}$; részletesen a 7.2. Softmax és többosztályos cross-entropy egységben.

5.1.5 Folytonos eloszlás: sűrűség és Riemann-integrál

Ha a véletlen változó abszolút folytonos eloszlású, azaz sűrűségfüggvény írja le, akkor az egyes pontok valószínűsége nulla, ezért intervallumok valószínűségével dolgozunk. Pusztán az, hogy az értékkészlet folytonos intervallum, önmagában nem garantálja ezt; a továbbiakban csak sűrűségfüggvénnyel leírható eloszlásokkal foglalkozunk. Ehhez a határozott integrált mint területet használjuk, amelyet a 4.1. Függvény, határérték és derivált alfejezetben megismert határérték-fogalommal Riemann-összegek határaként értelmezünk.

Definíció (Riemann-összeg). Legyen $f:[a,b]\to\mathbb{R}$ korlátos függvény. A teljes definícióban az $[a,b]$ intervallumot tetszőleges $a=x_0<x_1<\dots<x_n=b$ felosztással bontjuk fel, és minden $[x_{i-1},x_i]$ részintervallumból kiválasztunk egy tetszőleges $t_i\in[x_{i-1},x_i]$ közbülső pontot; ekkor

$$S_n=\sum_{i=1}^{n} f(t_i)(x_i-x_{i-1}).$$

Ebben az egységben az egyszerűség kedvéért egyenletes felosztással dolgozunk, $\Delta x=(b-a)/n$ lépésközzel, és csak két pontválasztást használunk: $x_i=a+i\Delta x$ (jobb végpontos választás) vagy $x_i=a+(i-1)\Delta x$ (bal végpontos választás). A teljes definícióban a felosztás finomságát, azaz a legnagyobb részintervallum hosszát csökkentjük minden határon túl. Ha így a S_n határértéke létezik és független a felosztás és a közbülső pontok választásától, akkor f Riemann-integrálható, és a határértéket

$$\int_a^b f(x)\,dx$$

jelöli. Ez a határozott integrál mint *előjeles terület*: a görbe alatti területet pozitív, a tengely alatti területet negatív előjellel számoljuk. Sűrűségfüggvényekre $f\geq 0$, ezért ott a valószínűség közönséges terület.

Definíció (sűrűségfüggvény). Egy $f:\mathbb{R}\to[0,\infty)$ függvény *valószínűségi sűrűségfüggvény*, ha

$$\int_{-\infty}^{\infty} f(x)\,dx = 1,$$

ahol az improper integrált a szokásos határátmenettel értjük. Ekkor egy X véletlen változó eloszlását ez a sűrűség adja, ha tetszőleges $a\leq b$ esetén

$$P(a \le X \le b) = \int_a^b f(x)\,dx.$$

A normalizálási feltétel garantálja, hogy a teljes valószínűség 1. A sűrűség értéke egyetlen pontban **nem** valószínűség; csak intervallumra integrálva kapunk valószínűséget.

Tétel (egyenletes eloszlás normalizálása és intervallum-valószínűsége). Legyen $a < b$ valós szám, és definiáljuk

$$f(x) = \begin{cases} \frac{1}{b-a}, & \text{ha } a \le x \le b, \\ 0, & \text{különben.} \end{cases}$$

Ekkor f sűrűségfüggvény, és bármely $[c, d] \subseteq [a, b]$ intervallumra

$$P(c \le X \le d) = \frac{d-c}{b-a}.$$

Bizonyítás. A normalizáláshoz elegendő az $[a, b]$-n integrálni, mert a függvény azon kívül nulla:

$$\int_{-\infty}^{\infty} f(x)\,dx = \int_a^b \frac{1}{b-a}\,dx = \frac{1}{b-a}\int_a^b 1\,dx = \frac{1}{b-a}\big[x\big]_a^b = \frac{1}{b-a}(b-a) = 1.$$

Az első egyenlőségnél kihasználtuk, hogy a sűrűség tartóján kívüli integrál nulla, a harmadiknál pedig az 1 konstans függvény primitív függvénye x. Az intervallum valószínűsége hasonlóan:

$$P(c \le X \le d) = \int_c^d \frac{1}{b-a}\,dx = \frac{1}{b-a}\big[x\big]_c^d = \frac{d-c}{b-a}.$$

A $\square$ jelöli a bizonyítás végét. A levezetés csak az alapintegrálokat és a határozott integrál linearitását használja, ezért a feltételek között szerepel az $a < b$ rendezés; a formula a $[c, d]$ intervallum hosszának és a teljes $[a, b]$ hosszának arányát adja.

Példa. $X \sim U[0, 1]$ esetén $f(x) = 1$ a $[0, 1]$-en. Ekkor $P(0{,}25 \le X \le 0{,}5) = 0{,}25$, miközben $f(0{,}5) = 1$. Látjuk a lényegi különbséget: a sűrűség magas értéke nem jelenti azt, hogy az adott pont valószínűsége magas; a pont valószínűsége nulla, mert $\int_{0{,}5}^{0{,}5} f(x)\,dx = 0$.

5.1.6 Kézi példa és numerikus labor

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

5.1.6.1 Osztályozási példa abszolút esetszámokkal

Vegyünk egy 100 elemű tesztmintát a fenti arányokkal: 20 *vertical* és 80 *horizontal* kép. A példában Y a valódi címke, $\hat{Y}$ a modell predikciója; a *vertical* osztály rövid jelölése v, a *horizontal* osztályé h. Az osztályozó a következőket teszi:

- a 20 *vertical* képből 18-at *vertical*, 2-t tévesen *horizontal* címkével lát el (szenzitivitás: $P(\hat{Y} = v \mid Y = v) = 18/20 = 0{,}9$),
- a 80 *horizontal* képből 76-ot *horizontal*, 4-et tévesen *vertical* címkével lát el (téves pozitív arány: $P(\hat{Y} = v \mid Y = h) = 4/80 = 0{,}05$).

A *szenzitivitás* (true positive rate) a valódi pozitív esetek helyesen felismert aránya, a *téves pozitív arány* (false positive rate) a valódi negatív esetek tévesen pozitívnak sorolt aránya; mindkettőt a valódi címkére feltételezve értelmezzük.

Az abszolút gyakoriságokat kontingenciatáblába rendezzük:

Valós \ Predikált	vertical	horizontal	Összesen
vertical	18	2	20
horizontal	4	76	80
Összesen	22	78	100

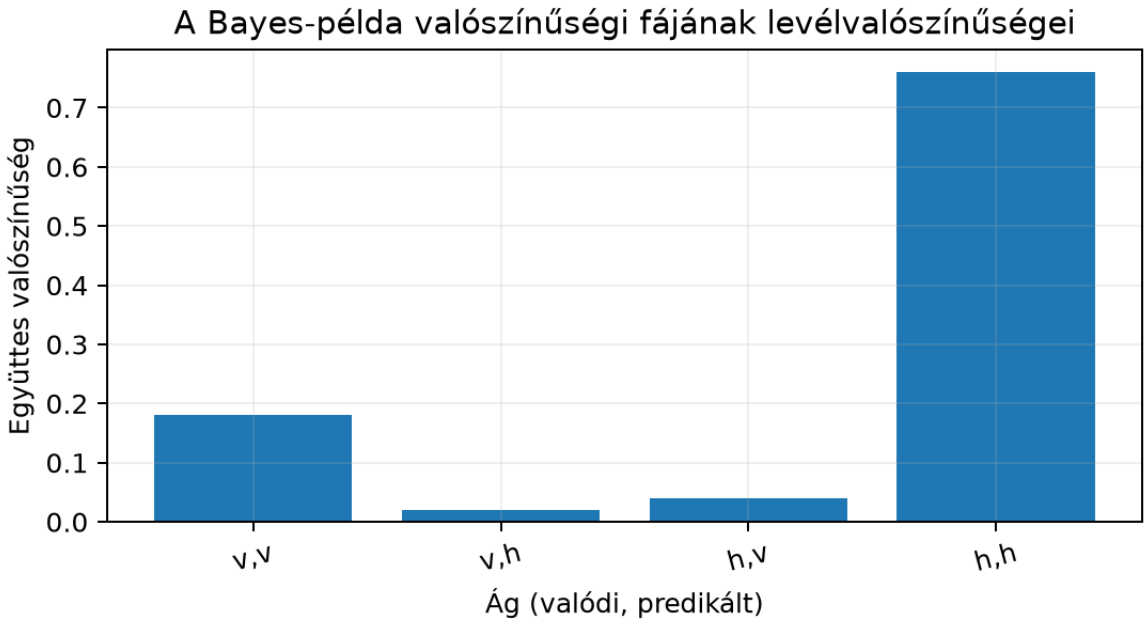
A kérdés: $P(\text{valós } v \mid \text{predikált } v)$. A táblázatból közvetlenül: 22 predikált *vertical* esetből 18 valóban az, tehát $18/22 \approx 0{,}818$. Ugyanez a Bayes-tétellel, $A = \{\hat{Y} = v\}$, $B_1 = \{Y = v\}$, $B_2 = \{Y = h\}$ partícióval:

$$P(Y = v \mid \hat{Y} = v) = \frac{P(\hat{Y} = v \mid Y = v)P(Y = v)}{P(\hat{Y} = v \mid Y = v)P(Y = v) + P(\hat{Y} = v \mid Y = h)P(Y = h)}$$

$$= \frac{0{,}9 \cdot 0{,}2}{0{,}9 \cdot 0{,}2 + 0{,}05 \cdot 0{,}8} = \frac{0{,}18}{0{,}22} \approx 0{,}81818.$$

A két út — esetszámokból és feltételes valószínűségekből — ugyanarra az eredményre vezet, mert a nevező 0,22 éppen a predikált *vertical* esetek aránya.

Az alábbi diagram a kétlépcsős kísérlet valószínűségi fájának négy levelét, azaz a $(Y, \hat{Y})$ párok együttes valószínűségeit mutatja; a v-predikcióhoz vezető két levél összege $0{,}18 + 0{,}04 = 0{,}22$, és ezen belül a helyes ág aránya $0{,}18/0{,}22$.



5.1. ábra: A Bayes-példa valószínűségi fájának levélvalószínűségei

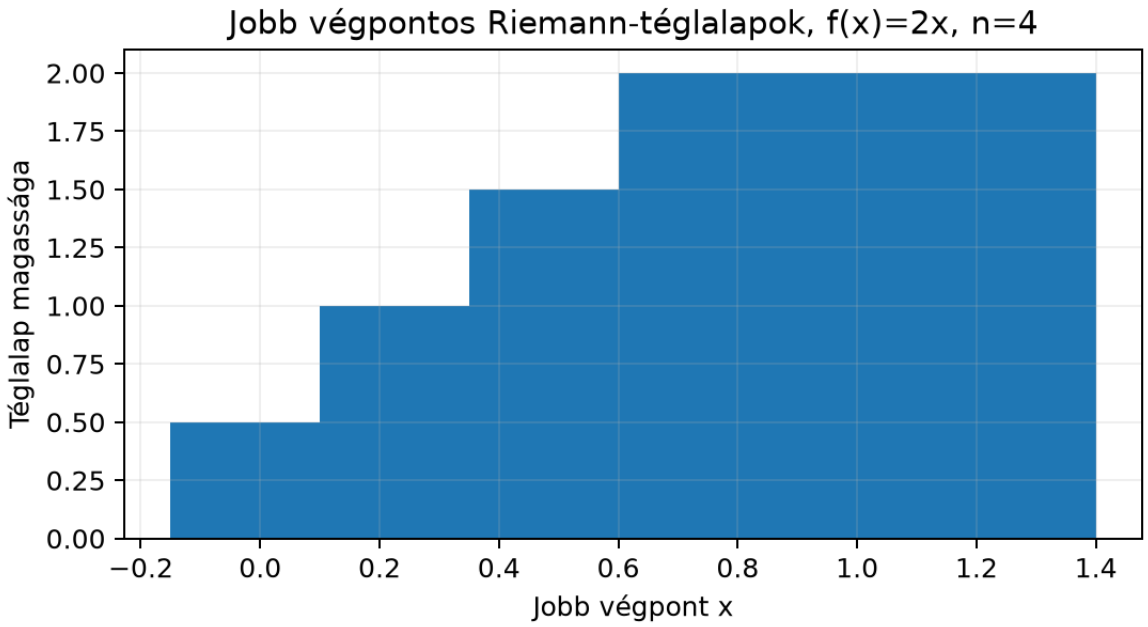
Képletből számított szemléltetés, nem mérés. A v-predikcióhoz vezető ágak összege 0,18+0,04=0,22, a posterior 0,18/0,22=0,818.

5.1.6.2 Numerikus területösszeg sűrűségre

Legyen $f(x) = 2x$ a $[0, 1]$ intervallumon. Ez normalizált, mert

$$\int_0^1 2x\,dx = \left[x^2\right]_0^1 = 1.$$

Az alábbi diagram a jobb végpontos téglalapokat szemlélteti $n = 4$ esetén: a téglalapok szélessége 0,25, magasságaik $f(0{,}25) = 0{,}5$, $f(0{,}5) = 1$, $f(0{,}75) = 1{,}5$ és $f(1) = 2$, összterületük ezért 1,25, ami felülbecsli a pontos 1-et.



5.2. ábra: Jobb végpontos Riemann-téglalapok, f(x)=2x, n=4

Képletből számított szemléltetés, nem mérés; a téglalapok szélessége 0,25, összterületük 1,25, a pontos integrál 1.

Számítsuk ki a [0,25; 0,5] intervallum valószínűségét analitikusan:

$$P(0{,}25 \le X \le 0{,}5) = \int_{0{,}25}^{0{,}5} 2x\,dx = \left[x^2\right]_{0{,}25}^{0{,}5} = 0{,}25 - 0{,}0625 = 0{,}1875.$$

Ezt ellenőrizzük Riemann-összeggel. A függvény neve `riemann_sum`, mert általános Riemann-összeget számol; sűrűségfüggvényre alkalmazva ez az intervallum valószínűségének közelítése. A kód véletlen számot nem használ, ezért seed nem szükséges, és a számítás float64-ben, CPU-n fut (a `dx` osztás és a szorzás float64-et ad). A lebegőpontos összegzés sorrendje miatt az utolsó bitek platformonként eltérhetnek, ezért csak a megadott tűréseket érdemes ellenőrizni.

```
import numpy as np

def riemann_sum(func, a, b, n, endpoint="right"):
    """Jobb vagy bal végpontos Riemann-összeg."""
    dx = (b - a) / n
    if endpoint == "right":
        x = a + np.arange(1, n + 1) * dx
    elif endpoint == "left":
        x = a + np.arange(0, n) * dx
    else:
        raise ValueError("endpoint legyen 'left' vagy 'right'")
    return np.sum(func(x)) * dx

def f(x):
    return 2.0 * x

# Normalizálás ellenőrzése
print("Normalizálás, pontos érték: 1.0")
for n in [10, 100, 1000, 10000]:
```

```
    approx = riemann_sum(f, 0.0, 1.0, n)
    print(f"  n={n:5d}  jobb={approx:.6f}  hiba={abs(approx-1.0):.2e}")

# Intervallum valószínűsége, pontos érték: 0.1875
print("\nP(0.25<=X<=0.5), pontos érték: 0.1875")
for n in [100, 1000, 10000]:
    approx = riemann_sum(f, 0.25, 0.5, n)
    print(f"  n={n:5d}  jobb={approx:.6f}  hiba={abs(approx-0.1875):.2e}")

# Ellenőrzés: a jobb végpontos hiba ~1/n, a felülről közelítés iránya helyes
assert abs(riemann_sum(f, 0.0, 1.0, 10000) - 1.0) < 2e-4
assert abs(riemann_sum(f, 0.25, 0.5, 10000) - 0.1875) < 1e-5
assert riemann_sum(f, 0.0, 1.0, 100) > 1.0
print("\nellenorzes: a Riemann-hibak a varhato rendben, a jobb osszeg felulrol kozelit")
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; a kód nem használ véletlen számot, ezért seed nem szükséges; az utolsó bitek környezetfüggően eltérhetnek):

```
Normalizálás, pontos érték: 1.0
  n=   10  jobb=1.100000  hiba=1.00e-01
  n=  100  jobb=1.010000  hiba=1.00e-02
  n= 1000  jobb=1.001000  hiba=1.00e-03
  n=10000  jobb=1.000100  hiba=1.00e-04

P(0.25<=X<=0.5), pontos érték: 0.1875
  n=  100  jobb=0.188125  hiba=6.25e-04
  n= 1000  jobb=0.187562  hiba=6.25e-05
  n=10000  jobb=0.187506  hiba=6.25e-06

ellenorzes: a Riemann-hibak a varhato rendben, a jobb osszeg felulrol kozelit
```

A $[0,1]$-re vett normalizálási integrálnál a jobb végpontos közelítés hibája $n = 10000$ esetén 10^{-4} nagyságrendű, mert a lépésköz 10^{-4}; a fenti $[0.25, 0.5]$ intervallumon a lépésköz $2{,}5 \cdot 10^{-5}$, ezért ott a hiba $6{,}25 \cdot 10^{-6}$. A hiba mindkét esetben lineárisan csökken. A függvény monoton növekvő, ezért a jobb összeg felülről, a bal összeg alulról közelít. A labor nem használ véletlen számot, ezért seed nem szükséges, és CPU-n fut; a kódba épített `assert`-ek a hiba nagyságrendjét és a felülről közelítés irányát ellenőrzik.

5.1.7 Feladatok

Az alábbi feladatok mindegyike a fenti fogalmakra épül. A nehézség 1 = közvetlen alkalmazás, 2 = több fogalom összekapcsolása, 3 = nyitott tervezési döntés.

F1 (fogalmi, nehézség 1). Adjon egy konkrét, négyelemű egyenletes Ω mintateret és két A, B eseményt úgy, hogy $P(A \mid B) = 1/2$, de $P(B \mid A) = 1$. Magyarázza el szavakkal, miért lehetséges, hogy a két feltételes valószínűség különbözik, pedig ugyanaz a metszet szerepel bennük.

F2 (levezetési, nehézség 2). Vezesse le lépésről lépésre a teljes valószínűség tételét két elemű $\{B_1, B_2\}$ partícióra, majd alkalmazza a fenti osztályozó példára. Számolja ki $P(Y = v \mid \hat{Y} = v)$ értékét a $P(Y = v) = 0{,}2$, $P(\hat{Y} = v \mid Y = v) = 0{,}9$, $P(\hat{Y} = v \mid Y = h) = 0{,}05$ adatokból, és ellenőrizze, hogy a nevező valóban 0,22.

F3 (implementációs, nehézség 2). Egészítse ki a fenti kódot a bal végpontos közelítéssel is. Futtassa $n = 10, 100, 1000$ értékekre az $f(x) = 2x$ sűrűség $[0,1]$-re vett normalizálási integráljára, és mindkét módszerre írja ki a hibát. Magyarázza meg, miért ad a bal összeg monoton növekvő függvénynél alsó, a jobb összeg felső becslést.

F4 (hibakeresési, nehézség 2). Egy hallgató a Bayes-tételt így kódolta:

```
P_hat_v_given_y_v = 0.9
P_y_v = 0.2
P_y_h = 0.8
P_hat_v = 0.9 * 0.2   # hiba itt
P_y_v_given_hat_v = P_hat_v_given_y_v * P_y_v / P_hat_v
print(P_y_v_given_hat_v)
```

Lokalizálja a hibát, magyarázza el, miért helytelen a $P(\hat{Y} = v)$ számítás, és adja meg a javított kódot, amely 0,81818-ot ad. Ellenőrizze a javítást abszolút esetszámokból kiindulva is (18 és 4, majd 22). A hibás kód kimenete 1,0, mert a számláló és a nevező is $0{,}9 \cdot 0{,}2$; ez az érték önmagában is jelzi, hogy a nevezőből hiányzik egy tag.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy reprodukálható kísérletet annak mérésére, hogy egy rögzített, betanított kép-osztályozó modell predikciós konfidenciája mennyire megbízható különböző osztályarányú teszthalmazokon. Használja a GlyphLab adatszerződés (A. függelék) szerinti adatokat, amelyekben minden képhez tartozik egy `scene_id`: az azonos jelenethez tartozó (két nézet) kép egy csoportot alkot, és a csoport a train/val/test felosztásban együtt marad. A teszteléshez a *test* splitből scene-csoport szintű újramintavételezéssel (ismételt mintavétellel, nem az egyedi képek súlyozásával) készítsen három változatot, például 80/10/10, 33/33/34 és 10/80/10 célaránnyal; ha egy osztály scene-csoportjai nem teszik ki pontosan a célarányt, használja a legközelebbi elérhető arányt, és rögzítse a tényleges arányokat. A modell betanított súlyait (checkpointját) a *train* részen rögzítse, a *val* részen válassza ki, és a tesztelés alatt ne változtassa. Metrikaként számítsa a konfidenciasávokra (bin) bontott expected calibration error-t (ECE):

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left| \mathrm{acc}_b - \mathrm{conf}_b \right|,$$

ahol B a sávok száma, n_b a b-edik sávba eső predikciók száma, N az összes predikció száma, acc_b a sávban mért pontosság, conf_b pedig a sávba eső predikciós konfidenciák átlaga. Adja meg a sávok számát és határait, a kiértékelési protokollt és legalább két érvényességi korlátot. Konkrét eredményt nem várunk; a terv legyen végrehajtható.

5.1.8 Megoldások és indoklás

F1 megoldás. Legyen $\Omega = \{1, 2, 3, 4\}$ egyenletes eloszlással, $P(\omega) = 1/4$. Válasszuk $A = \{1\}$ és $B = \{1, 2\}$. Ekkor $P(A) = 1/4$, $P(B) = 1/2$, $P(A \cap B) = P(\{1\}) = 1/4$. A feltételes valószínűségek:

$$P(A \mid B) = \frac{1/4}{1/2} = \frac{1}{2}, \qquad P(B \mid A) = \frac{1/4}{1/4} = 1.$$

A két érték azért különbözik, mert az A-ra szűkített mintatérben a B biztos esemény, míg a B-re szűkített mintatérben az A csak a B felét fedi le. Ugyanaz a metszet, de más nevezővel osztunk.

F2 megoldás. A levezetés a főszövegben szereplő bizonyítás két elemű speciális esete. Mivel $\{B_1, B_2\}$ partíció, $A = (A \cap B_1) \cup (A \cap B_2)$ diszjunkt unió. Az additivitásból:

$$P(A) = P(A \cap B_1) + P(A \cap B_2) = P(A \mid B_1)P(B_1) + P(A \mid B_2)P(B_2).$$

Itt $A = \{\hat{Y} = v\}$, $B_1 = \{Y = v\}$, $B_2 = \{Y = h\}$. Behelyettesítve:

$$P(\hat{Y} = v) = 0{,}9 \cdot 0{,}2 + 0{,}05 \cdot 0{,}8 = 0{,}18 + 0{,}04 = 0{,}22.$$

A Bayes-formulával:

$$P(Y = v \mid \hat{Y} = v) = \frac{0{,}9 \cdot 0{,}2}{0{,}22} = \frac{0{,}18}{0{,}22} = \frac{18}{22} = \frac{9}{11} \approx 0{,}81818.$$

A nevező ellenőrzése megtörtént, és megegyezik az esetszámokból kapott 22-vel (100 képből).

F3 megoldás. A kiegészített kód:

```
import numpy as np

def riemann_sum(func, a, b, n, endpoint="right"):
    """Jobb vagy bal végpontos Riemann-összeg."""
    dx = (b - a) / n
    if endpoint == "right":
        x = a + np.arange(1, n + 1) * dx
    elif endpoint == "left":
        x = a + np.arange(0, n) * dx
    else:
        raise ValueError("endpoint legyen 'left' vagy 'right'")
    return np.sum(func(x)) * dx

def f(x):
    return 2.0 * x

for n in [10, 100, 1000]:
    left = riemann_sum(f, 0.0, 1.0, n, "left")
    right = riemann_sum(f, 0.0, 1.0, n, "right")
    print(f"n={n:4d}  bal hiba={left-1.0:+.6f}  jobb hiba={right-1.0:+.6f}")
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU; a kód nem használ véletlen számot):

```
n=  10  bal hiba=-0.100000  jobb hiba=+0.100000
n= 100  bal hiba=-0.010000  jobb hiba=+0.010000
n=1000  bal hiba=-0.001000  jobb hiba=+0.001000
```

Az $f(x) = 2x$ függvényre a hibák zárt alakban is ellenőrizhetők: a jobb összeg $\int_0^1 2x\,dx + 1/n$, a bal összeg $\int_0^1 2x\,dx - 1/n$, ezért a hiba nagysága éppen $1/n$. A bal összeg hibája negatív, a jobbé pozitív, mert f monoton növekvő: minden $[x_{i-1}, x_i]$ részintervallumon és minden $x \in [x_{i-1}, x_i]$-re $f(x_{i-1}) \le f(x) \le f(x_i)$, ezért $f(x_{i-1})\Delta x \le \int_{x_{i-1}}^{x_i} f(x)\,dx \le f(x_i)\Delta x$; a részintervallumokra összegezve a bal összeg alulbecsli, a jobb összeg felülbecsli az integrált. Ahogy n nő, mindkettő az 1 pontos értékhez tart.

F4 megoldás. A hiba a $P(\hat{Y} = v)$ számításában van: a hallgató csak a $P(\hat{Y} = v \mid Y = v)P(Y = v)$ tagot vette figyelembe, és elhagyta a $P(\hat{Y} = v \mid Y = h)P(Y = h)$ tagot, tehát a nevező nem a teljes valószínűséget adja. A javított kód:

```
import math

P_hat_v_given_y_v = 0.9
P_hat_v_given_y_h = 0.05
P_y_v = 0.2
P_y_h = 0.8
P_hat_v = P_hat_v_given_y_v * P_y_v + P_hat_v_given_y_h * P_y_h
P_y_v_given_hat_v = P_hat_v_given_y_v * P_y_v / P_hat_v
print(P_y_v_given_hat_v)  # 0.8181818181818181
assert math.isclose(P_y_v_given_hat_v, 18 / 22, rel_tol=1e-12)
```

Esetszámokból: 20 *vertical* képből 18 kap *vertical* predikciót; 80 *horizontal* képből 4 kap tévesen *vertical* predikciót. Összes *vertical* predikció $18 + 4 = 22$, ebből helyes 18, arány $18/22 \approx 0{,}818$. A kimenet lebegőpontos, ezért a pontos $18/22$ értékkel `math.isclose(..., rel_tol=1e-12)` szerint érdemes összevetni; a relatív tűrés a bináris lebegőpontos ábrázolás miatt kell, nem a képlet bizonytalansága miatt. A közölt kimenet ezt a tűrést teljesíti.

F5 megoldás. Egy lehetséges terv: rögzítsünk egy betanított kép-osztályozó modellt a GlyphLab *train* részén, a *val* részen válasszuk ki a legjobb checkpointot (a mentett súlyokat), majd a *test* részt ne változtassuk meg. Helyette hozzunk létre három újramintavételezett tesztváltozatot: mindegyikben scene-csoportok szintjén mintavételezzünk újra a kívánt osztályarányok eléréséhez (a csoporton belüli képek és címkék párosítása változatlan maradjon), és rögzítsük a ténylegesen elért arányokat. Mindegyik változatra számoljuk ki a modell softmax-konfidenciáját, és a fenti ECE-képlettel mérjük a kalibrációt: a konfidenciasávokba sorolt predikciók átlagos konfidenciája és a tényleges pontosság közötti, a sávok létszámával súlyozott eltérést. Hipotézis: a kalibráció romlik, ha a teszthalmaz osztályaránya eltér a tanítási aránytól. Érvényességi korlát: a GlyphLab kis, procedurális adat, ezért az eredmény nem általánosítható természetes képekre; továbbá a mesterséges arányok megváltoztatják a prior eloszlást, ami önmagában is befolyásolja a konfidencia értelmezését, ezért a mérést rögzített modellsúlyokkal kell végezni, és a sávok számát előre rögzíteni kell, hogy a különböző változatok ECE-értékei összevethetők legyenek.

5.1.9 Tipikus tévedés és ellenpélda

Két gyakori hiba kapcsolódik ehhez az anyaghoz.

1. $P(A \mid B)$ és $P(B \mid A)$ felcserélése. A fenti osztályozó példában $P(\hat{Y} = v \mid Y = v) = 0{,}9$, míg $P(Y = v \mid \hat{Y} = v) \approx 0{,}818$. Ugyanannak a feltételes kapcsolatnak a két iránya különböző, mert a nevezőben más esemény valószínűsége áll. A felcserélés a klasszikus bázisráta-tévedés (base rate fallacy) forrása. Számolt példa: tegyük fel, hogy egy betegség előfordulási aránya (prevalenciája) 1%, a teszt szenzitivitása 99%, a téves pozitív aránya pedig 5%. 10 000 emberből átlagosan 100 beteg, közülük 99 kap pozitív eredményt; a 9 900 egészséges közül 5%, azaz 495 kap tévesen pozitív eredményt. A pozitív eredményűek között tehát $99/(99 + 495) = 99/594 = 1/6 \approx 0{,}167$ a betegség aránya, jóval alacsonyabb, mint a 99%-os szenzitivitás. A téves pozitívak nagy száma a ritka betegség populációjában a bázisráta (1%) miatt van: a 99%-os szenzitivitás önmagában nem mondja meg a pozitív eredmény utáni valószínűséget, mert a nevezőben a hamis pozitívak is szerepelnek.

2. A sűrűség pontbeli értékét valószínűségként értelmezni. Tekintsük az $U[0, 1]$ egyenletes eloszlást. Itt $f(0{,}5) = 1$, tehát a sűrűség értéke meghaladhatja az 1-et is (például $U[0; 0{,}1]$ esetén $f = 10$), miközben $P(X = 0{,}5) = 0$. A hiba következménye: ha valaki egy modell predikciójának sűrűségét egyetlen pontban nézi, tévesen magas vagy alacsony valószínűséget tulajdonít egy konkrét érték bekövetkezésének. Helyesen mindig intervallumra kell integrálni.

5.1.10 Önellenőrzés és korábbi fogalmak áttekintése

Ellenőrző kérdések a tanuláshoz: (1) Az általam használt partíció elemei lefedik-e a teljes Ω-t és páronként diszjunktak-e? (2) A feltételes valószínűség nevezője pozitív-e? (3) A sűrűségfüggvényem integrálja 1-et ad-e a teljes tartón? (4) A Bayes-tétel alkalmazásakor a nevező a teljes valószínűség tételéből származik-e?

A 4.1. Függvény, határérték és derivált egységben a deriváltat a differenciahányados határértékeként definiáltuk. Most ugyanezt a határérték-szemléletet alkalmaztuk a Riemann-összegre: a numerikus közelítés finomítása a pontos integrálhoz konvergál, és a hiba a lépésközzel arányosan

csökken. Az 1.1. Modell, tanulás és felhasználási szerződés modellfogalmából pedig a kategorikus eloszlás az, amely a többosztályos cél változóját írja le: a modell nem egyetlen determinisztikus címkét, hanem egy π valószínűségi vektort becsül.

5.1.11 Rövid összefoglaló

A véges mintatéren az esemény valószínűsége a kimenetelek valószínűségeinek összege. A feltételes valószínűség az $A \cap B$ metszetet a B esemény valószínűségéhez viszonyítja, és a Bayes-tétel ebből a definícióból, valamint egy partíció diszjunkt felbontásából vezethető le. A Bernoulli- és kategorikus eloszlás a bináris, illetve többosztályos címkék alapmodellje. Folytonos esetben a sűrűségfüggvény normalizált, integrálja 1, és egy intervallum valószínűségét a görbe alatti terület adja; a Riemann-összeg ezt a területet közelíti numerikusan. A sűrűség pontbeli értéke nem valószínűség, és a feltételes valószínűség két iránya általában nem cserélhető fel.

5.1.12 Továbblépés

A következő egység, 5.2. Várható érték, variancia és likelihood, a várható értéket, varianciát és likelihood-ot vezeti be; ott a sűrűség és eloszlás fogalmát használjuk a veszteségfüggvények valószínűségi értelmezéséhez.

5.1.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

5.2 Várható érték, variancia és likelihood

5.2.1 Cél, előismeret és induló becslés

Ebben az egységben az 5.1. Esemény, feltétel és véletlen változó alfejezetben bevezetett eloszlásokhoz rendelünk összefoglaló számokat, és megfordítjuk a kérdés irányát: nem azt kérdezzük, hogy adott paraméter mellett mi a megfigyelés valószínűsége, hanem hogy adott megfigyeléshez melyik paraméter a legvalószínűbb magyarázat. Ehhez definiáljuk a várható értéket, varianciát, kovarianciát, majd a likelihoodot és a maximum likelihood becslést (MLE). A 4.2. Parciális derivált és gradiens alfejezetből a deriválást használjuk a maximum megtalálásához, az 1.2. Mitől egyetemi szintű egy modellezési állítás? alfejezetből pedig az állítástípus-fegyelmet: a momentumok levezetése tétel, a numerikus Riemann-közelítés empirikus ellenőrzés.

Induló becslés. Egy szabályos dobókocka sok dobásának átlaga hosszú távon hová tart? Tippeljen egy számot, mielőtt a várható értéket bevezetjük. (A helyes válasz 3,5; a levezetést lentebb adjuk.)

5.2.2 Adat, modell, paraméter és becslési eljárás

A likelihood tárgyalásához négy fogalmat külön kell választani; a felcserélésük gyakori hibaforrás a modellezési jelentésekben.

Definíció (adat). Az adat a megfigyelt realizációk véges sorozata. Ebben az egységben skalár megfigyelésekkel dolgozunk: $x = (x_1, \dots, x_N)$, ahol Bernoulli-címke esetén $x_i \in \{0, 1\}$, kategorikus címke esetén $x_i \in \{0, \dots, K-1\}$. A tenzoralak a programban $x \in \mathbb{Z}^N$ vagy $\mathbb{R}^N$; az N a minta elemszáma.

Definíció (modell és paraméter). A modell egy eloszláscsalád

$$\{p(x;\theta) : \theta \in \Theta\},$$

ahol minden rögzített θ paraméterhez egy-egy eloszlás tartozik, és Θ a lehetséges paraméterek halmaza. Bernoulli-modellnél $\Theta = [0,1]$ és $p(x;\theta) = \theta^x(1-\theta)^{1-x}$. A paraméter a modell egy pontját választja ki; a valódi adatgeneráló mechanizmus ismeretlen.

Definíció (becslési eljárás és becslés). A becslési eljárás egy T függvény, amely minden lehetséges x mintához egy Θ-beli értéket rendel: $\widehat{\theta} = T(x)$. A konkrét realizációból számolt $\widehat{\theta}$ a becslés. Az eljárás a recept, a becslés az eredmény — ugyanaz a különbség, mint a függvény és a függvényérték között.

A függelék A. szakaszában leírt GlyphLab adatban $N = 360$ kép címkéje kategorikus modellt követhet; a kép maga nem ebben az egységben modellezett megfigyelés, csak a címke. Az ebben az egységben szereplő labor nem a GlyphLab fájljait használja: saját, szimulált kategorikus mintát generál $N = 200$ elemmel (lásd a labor leírását), a GlyphLab $N = 360$ képe itt csupán példa a kategorikus címkére. A modellválasztás külön döntés, a paraméter becslése pedig az adaton végzett számítás.

5.2.3 Várható érték

Definíció (diszkrét várható érték). Legyen X diszkrét valószínűségi változó $p_X(x) = P(X = x)$ valószínűségi tömegfüggvénnyel. Ha

$$\sum_{x} |x|\, p_X(x) < \infty,$$

akkor X várható értéke

$$E[X] = \sum_{x} x\, p_X(x).$$

A várható érték és a variancia tulajdonságainak rendszerezett tárgyalása a [B01] 6. fejezetében és a [B02] 3. fejezetében található.

Példa (kocka várható értéke). A szabályos dobókockánál $X \in \{1,2,3,4,5,6\}$ és $p_X(k) = 1/6$ minden k-ra, ezért

$$E[X] = \sum_{k=1}^{6} k \cdot \frac{1}{6} = \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3{,}5.$$

Ez a válasz az induló becslés kérdésére: sok dobás átlaga hosszú távon 3,5 körül ingadozik.

A feltétel nem díszítés. A $\sum_x |x| p_X(x) < \infty$ abszolút összegezhetőség biztosítja, hogy az összeg sorrendje ne változtassa az eredményt. Példa divergens esetre: legyen $P(X = 2^k) = 2^{-k}$ minden

$k = 1, 2, \ldots$ egészre; ez valóban eloszlás, mert $\sum_{k=1}^{\infty} 2^{-k} = 1$, de $\sum_k 2^k \cdot 2^{-k} = \sum_k 1 = \infty$, tehát $E[X]$ nem létezik véges értékként.

Definíció (folytonos várható érték). Legyen X folytonos valószínűségi változó f_X sűrűségfüggvénnyel. Ha $\int_{-\infty}^{\infty} |x| f_X(x)\,dx < \infty$, akkor

$$E[X] = \int_{-\infty}^{\infty} x\, f_X(x)\,dx.$$

A sűrűség és a határozott integrál kapcsolatát 5.1. Esemény, feltétel és véletlen változó tisztázta: $\int_a^b f_X(x)\,dx = P(a \le X \le b)$, a sűrűség pontbeli értéke nem valószínűség.

Tétel (linearitás). Ha $E[X]$ létezik, akkor tetszőleges $a, b \in \mathbb{R}$ esetén

$$E[aX + b] = aE[X] + b.$$

Bizonyítás (diszkrét eset). A definícióból:

$$E[aX+b] = \sum_x (ax+b)\,p_X(x) = a\sum_x x\,p_X(x) + b\sum_x p_X(x) = aE[X] + b,$$

mert $\sum_x p_X(x) = 1$. Folytonos esetben az összeget integrálra cserélve ugyanez a lépés. A $g(X)$ típusú kifejezések várható értékére használjuk a helyettesítési szabályt (transzformációs tétel, angolul law of the unconscious statistician): diszkrét esetben $E[g(X)] = \sum_x g(x) p_X(x)$; folytonos esetben $E[g(X)] = \int g(x) f_X(x)\,dx$. Diszkrét esetben az állítás a definícióból azonnal adódik: a $g(X) = y$ esemény azon x kimenetelek uniója, amelyekre $g(x) = y$, ezért $P(g(X) = y) = \sum_{x : g(x) = y} p_X(x)$, és az y szerinti összegzés visszaadja a fenti képletet. Folytonos esetben a bizonyítást itt elhagyjuk; a felhasznált konkrét g-k mind polinomok, amelyekre az eredményt közvetlen integrállal is ellenőrizzük.

5.2.4 Variancia és kovariancia

Definíció (variancia és szórás). Ha $E[X]$ létezik, akkor

$$\operatorname{Var}(X) = E\left[(X - E[X])^2\right],$$

feltéve, hogy ez a várható érték véges. A szórás $\sigma_X = \sqrt{\operatorname{Var}(X)}$, mértékegysége azonos X-ével.

Tétel (számítóképlet). $\operatorname{Var}(X) = E[X^2] - (E[X])^2$ és $\operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X)$.

Bizonyítás. Legyen $\mu = E[X]$. A linearitást alkalmazva:

$$E[(X-\mu)^2] = E[X^2 - 2\mu X + \mu^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - \mu^2.$$

A második állításhoz $E[aX + b] = a\mu + b$, így

$$\operatorname{Var}(aX+b) = E[(aX + b - a\mu - b)^2] = a^2 E[(X-\mu)^2] = a^2 \operatorname{Var}(X).$$

Definíció (kovariancia). Két véges varianciájú X, Y valószínűségi változó kovarianciája

$$\operatorname{Cov}(X, Y) = E\left[(X - E[X])(Y - E[Y])\right].$$

A korreláció $\rho = \operatorname{Cov}(X,Y)/(\sigma_X \sigma_Y)$, amennyiben mindkét szórás pozitív; ez $[-1, 1]$ értékű, és csak lineáris kapcsolatot mér.

Tétel (összeg varianciája). Legyenek X és Y véges varianciájúak. Ekkor

$$\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X,Y).$$

Kettőnél több tagra a képlet indukcióval adódik: véges varianciájú $X_1, \dots, X_N$ esetén

$$\operatorname{Var}\Big(\sum_{i=1}^{N} X_i\Big) = \sum_{i=1}^{N} \operatorname{Var}(X_i) + 2 \sum_{1 \le i < j \le N} \operatorname{Cov}(X_i, X_j).$$

Ha a változók páronként korrelálatlanok, azaz $\operatorname{Cov}(X_i, X_j) = 0$ minden $i \neq j$-re, akkor a második összeg eltűnik, és $\operatorname{Var}(\sum_i X_i) = \sum_i \operatorname{Var}(X_i)$.

Bizonyítás. Jelölje $\mu_X = E[X]$, $\mu_Y = E[Y]$. A linearitásból $E[X+Y] = \mu_X + \mu_Y$, így

$$\begin{aligned} \operatorname{Var}(X+Y) &= E\left[(X+Y-\mu_X-\mu_Y)^2\right] \\ &= E\left[(X-\mu_X)^2\right] + E\left[(Y-\mu_Y)^2\right] + 2E\left[(X-\mu_X)(Y-\mu_Y)\right]. \end{aligned}$$

A többtagú formula ebből a kétváltozós esetből indukcióval következik. Ha X és Y független, akkor $E[XY] = E[X]E[Y]$, amiből $\operatorname{Cov}(X,Y) = 0$, tehát $\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y)$. A megfordítás **nem** igaz. Ellenpélda: legyen X egyenletes eloszlású a $\{-1, 0, 1\}$ halmazon, $Y = X^2$. Ekkor $E[X] = 0$, $E[Y] = 2/3$, és $E[XY] = E[X^3] = 0$, ezért a kovariancia 0. Mégsem függetlenek, mert $P(X=1, Y=1) = P(X=1) = 1/3$, míg $P(X=1)P(Y=1) = (1/3)(2/3) = 2/9$.

5.2.5 Newton–Leibniz-tétel és az egyenletes eloszlás momentumai

A folytonos momentumok kiszámításához a Riemann-integrál egyetlen tételét használjuk.

Tétel (Newton–Leibniz-formula, polinomokra kimondva). Ha f folytonos a zárt $[a,b]$ intervallumon, és F'=f az (a,b)-n, továbbá F folytonos $[a,b]$-n, akkor

$$\int_a^b f(x)\,dx = F(b) - F(a).$$

$r \ge 0$ egészre a $f(x) = x^r$ polinom primitív függvénye $F(x) = x^{r+1}/(r+1)$.

A tételt itt a szükséges polinom-esetre alkalmazzuk; általános bizonyítását nem végezzük el, mert az integrálelmélet körültekintő felépítése a könyv keretein kívül esik. A folytonosság elegendő feltétel a Riemann-integrálhatósághoz, de a formula a megadott feltételpár mellett lépésről lépésre használható.

Tétel (egyenletes eloszlás momentumai). Legyen $X \sim U(a,b)$, azaz sűrűsége $f_X(x) = 1/(b-a)$ az $[a,b]$ intervallumon és 0 azon kívül; itt $a < b$ valós számok. Ekkor

$$E[X] = \frac{a+b}{2}, \qquad \operatorname{Var}(X) = \frac{(b-a)^2}{12}.$$

Bizonyítás. Először a várható érték:

$$\begin{aligned} E[X] &= \int_a^b x\,\frac{1}{b-a}\,dx = \frac{1}{b-a}\left[\frac{x^2}{2}\right]_a^b \\ &= \frac{1}{b-a}\left(\frac{b^2}{2} - \frac{a^2}{2}\right) = \frac{b^2-a^2}{2(b-a)} = \frac{a+b}{2}. \end{aligned}$$

Itt az utolsó lépés a $b^2-a^2=(b-a)(a+b)$ azonosság, ami a $b>a$ feltétel miatt egyszerűsíthető. A második momentum:

$$E[X^2]=\int_a^b x^2\,\frac{1}{b-a}\,dx=\frac{1}{b-a}\left[\frac{x^3}{3}\right]_a^b=\frac{b^3-a^3}{3(b-a)}=\frac{a^2+ab+b^2}{3}.$$

A számítóképletből:

$$\begin{aligned}\operatorname{Var}(X)&=\frac{a^2+ab+b^2}{3}-\frac{(a+b)^2}{4}=\frac{4a^2+4ab+4b^2-3a^2-6ab-3b^2}{12}\\&=\frac{a^2-2ab+b^2}{12}=\frac{(b-a)^2}{12}.\end{aligned}$$

A $[0,1]$-en egyenletes esetben $E[X]=1/2$, $\operatorname{Var}(X)=1/12$. Ez a momentum közvetlenül kapcsolódik 5.1. Esemény, feltétel és véletlen változó Riemann-összegéhez: az integrál ugyanaz a terület, amelyet ott téglalapokkal közelítettünk.

5.2.6 Likelihood, log-likelihood és a Bernoulli-minta maximuma

Definíció (likelihood). Rögzített $x=(x_1,\dots,x_N)$ megfigyeléshez a likelihood függvény

$$\mathcal{L}(\theta;x)=\prod_{i=1}^{N}p(x_i;\theta),$$

diszkrét esetben; folytonos modellnél p helyett f sűrűség áll. A log-likelihood

$$\ell(\theta;x)=\log\mathcal{L}(\theta;x)=\sum_{i=1}^{N}\log p(x_i;\theta),$$

ahol log természetes logaritmus. A szorzatból összeg lesz, ami numerikusan stabilabb; a log szigorú monotonitása miatt $\mathcal{L}$ és ℓ maximumhelye azonos.

A likelihood fontos tulajdonsága: rögzített adat mellett a **paraméter függvénye**. Nem a paraméter valószínűsége, nem normalizált eloszlás, és nem egyenlő a Bayes-féle posteriorral (a paraméter adat utáni feltételes eloszlásával, amelyhez prior is kell) — erre a későbbi, „Tipikus tévedés ellenpéldával” pontban adunk ellenpéldát.

Tétel (Bernoulli-minta MLE). Legyen $x_1,\dots,x_N\in\{0,1\}$ minta, és $s=\sum_{i=1}^{N}x_i$. A Bernoulli-modell likelihoodja

$$\mathcal{L}(\theta)=\theta^s(1-\theta)^{N-s},\qquad\theta\in[0,1],$$

és ha $0<s<N$, akkor az egyetlen belső maximumhely

$$\hat\theta_{\mathrm{ML}}=\frac{s}{N}=\bar x,$$

ahol $\bar x=\frac{1}{N}\sum_{i=1}^{N}x_i$ a mintaátlag. Ha $s=0$, akkor a maximum $\hat\theta=0$; ha $s=N$, akkor $\hat\theta=1$. A likelihood képletében a θ^0 és $(1-\theta)^0$ tényezőt 1-nek vesszük; a 0^0 itt használt konvencionális értéke 1, összhangban azzal, hogy a szélső eseteket határértékkel kezeljük.

Bizonyítás. Minden egyes megfigyelés likelihoodja $\theta^{x_i}(1-\theta)^{1-x_i}$; független minta esetén ezek szorzata adja $\mathcal{L}(\theta)$-t. A kitevők összeadódnak: az 1-esek száma s, a 0-ké $N-s$. A log-likelihood

$$\ell(\theta)=s\log\theta+(N-s)\log(1-\theta).$$

Feltéve $0 < s < N$, az első derivált

$$\ell'(\theta) = \frac{s}{\theta} - \frac{N-s}{1-\theta}.$$

A második derivált

$$\ell''(\theta) = -\frac{s}{\theta^2} - \frac{N-s}{(1-\theta)^2} < 0$$

minden $\theta \in (0,1)$-re, tehát ℓ szigorúan konkáv a $(0,1)$ intervallumon (azaz bármely két pontját összekötő húr alatt halad); egyetlen stacionárius pontja globális maximum. A $\ell'(\theta) = 0$ egyenlet:

$$\frac{s}{\theta} = \frac{N-s}{1-\theta} \quad\Longleftrightarrow\quad s(1-\theta) = (N-s)\theta \quad\Longleftrightarrow\quad s = N\theta \quad\Longleftrightarrow\quad \hat{\theta} = \frac{s}{N}.$$

A szélső esetek: $s = 0$ esetén $\ell(\theta) = N\log(1-\theta)$, ami $\theta = 0$-ban maximális; $s = N$ esetén $\ell(\theta) = N\log\theta$, ami $\theta = 1$-ben maximális. Ilyenkor a derivált zérushelye nincs a belső tartományban, ezért a maximumot a tartomány szélén keressük — a belsőponti feltétel a kulcs.

Tétel (kategorikus minta MLE). Legyen $y_1, \dots, y_N \in \{0, \dots, K-1\}$ minta, és $n_k = \#\{i : y_i = k\}$ az egyes osztályok gyakorisága, így $\sum_{k=0}^{K-1} n_k = N$. A kategorikus modell likelihoodja

$$\mathcal{L}(\pi_0, \dots, \pi_{K-1}) = \prod_{k=0}^{K-1} \pi_k^{n_k}, \qquad \pi_k \geq 0, \quad \sum_{k=0}^{K-1} \pi_k = 1,$$

és a maximumhely $\hat{\pi}_k = n_k/N$; a nulla gyakoriságú osztályra $\hat{\pi}_k = 0$.

Bizonyítás. A log-likelihood

$$\ell(\pi) = \sum_{k=0}^{K-1} n_k \log \pi_k,$$

ahol a nulla gyakoriságú tagot $0 \log 0 = 0$ értelemben vesszük. A $\sum_k \pi_k = 1$ feltételt Lagrange-szorzóval kezeljük:

$$\frac{\partial}{\partial \pi_k}\Big(\ell(\pi) - \lambda\Big(\sum_{j=0}^{K-1} \pi_j - 1\Big)\Big) = \frac{n_k}{\pi_k} - \lambda.$$

A stacionaritási feltétel belső pontban $\pi_k = n_k/\lambda$; a feltételbe helyettesítve $1 = \sum_k n_k/\lambda = N/\lambda$, tehát $\lambda = N$ és $\hat{\pi}_k = n_k/N$. A log függvény konkáv, ezért a ℓ konkáv; a stacionárius pont a feltételi halmazon globális maximum. Ha $n_k = 0$, a képlet $\hat{\pi}_k = 0$-t ad, és ez valóban optimális: $\pi_k > 0$ esetén a π_k csökkentése a derivált $-\lambda < 0$ előjele miatt növeli ℓ-t, tehát a maximum a határon van. $\square$

A likelihood értékét a maximumban a kézi példában számítjuk ki.

5.2.7 Kézi példa a likelihoodra

Legyen $N = 5$ pénzérme-feldobás eredménye $x = (1, 0, 1, 1, 0)$. Ekkor $s = 3$, az MLE $\hat{\theta} = 3/5 = 0{,}6$. A likelihood a maximumhelyen:

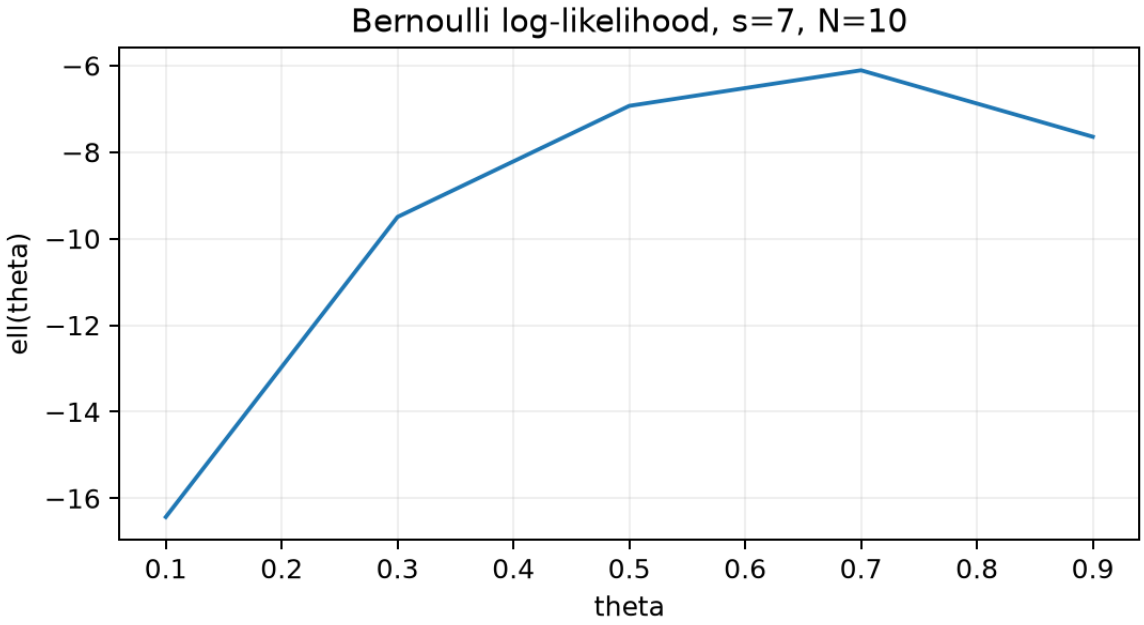
$$\mathcal{L}(0{,}6) = 0{,}6^3 \cdot 0{,}4^2 = 0{,}216 \cdot 0{,}16 = 0{,}03456,$$

a log-likelihood pedig

$$\ell(0{,}6) = 3\log 0{,}6 + 2\log 0{,}4 \approx -3{,}365.$$

Ellenőrzés közeli pontban: $\ell(0{,}5) = 5\log 0{,}5 \approx -3{,}466 < -3{,}365$, tehát a numerikus kép összhangban van a tétellel.

A következő ábra egy nagyobb minta, $s = 7$, $N = 10$ log-likelihoodját mutatja. A vízszintes tengely a $\theta \in [0, 1]$ paramétert, a függőleges tengely az $\ell(\theta)$ log-likelihoodot ábrázolja. Az ábra a $\ell(\theta) = 7 \log \theta + 3 \log(1 - \theta)$ képletből számított értékeket mutatja, nem mérést; a maximum $\theta = 0{,}7$-nél van.



5.3. ábra: Bernoulli log-likelihood, s=7, N=10

Képletből számított szemléltetés; a maximum helye 0,7, egyezésben s/N-nel.

A vonaldiagram kerekített; a pontos értékek $\ell(\theta) = 7 \log \theta + 3 \log(1 - \theta)$ közvetlen behelyettesítéséből adódnak, a maximum ténylegesen 0,7.

5.2.8 Labor: kategorikus minta és folytonos momentum

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor két célt ellenőriz egyszerre: (1) szimulált kategorikus mintából MLE-t illesztünk, beleértve a nullszámosságú osztály szélső esetét; (2) a folytonos egyenletes momentumot a Newton–Leibniz-formulából adódó zárt képlettel és felezőpontos (midpoint) Riemann-összeggel is kiszámítjuk. A kód a `lab/n018_likelihood.py` fájlban készül, az inputot maga generálja, tehát nincs külső adatletöltés. A véletlenszám-generátor vetőmagja (seed) `20261020`, a lebegőpontos adattípus (dtype) `float64`.

```
# lab/n018_likelihood.py
# Futtatás: python lab/n018_likelihood.py
import numpy as np

rng = np.random.default_rng(20261020)

# --- 1. Kategorikus minta és MLE ---
p_true = np.array([0.5, 0.3, 0.2, 0.0], dtype=np.float64)
K = p_true.size
N = 200
```

```
y = rng.choice(K, size=N, p=p_true)          # int64 osztályindexek
counts = np.bincount(y, minlength=K)
p_hat = counts / N                           # MLE kategorikus modellre

print("counts     :", counts)
print("p_hat      :", p_hat)
print("sum        :", p_hat.sum())
print("nemnegativ :", np.all(p_hat >= 0.0))

# A generáló p negyedik komponense 0, ezért a minta sem tartalmazhatja.
assert counts[3] == 0
assert p_hat[3] == 0.0
assert np.isclose(p_hat.sum(), 1.0, atol=1e-12)

# --- 2. Stabil Bernoulli log-likelihood a szélekkel ---
def bernoulli_loglik(theta, s, n):
    if theta == 0.0:
        return 0.0 if s == 0 else -np.inf
    if theta == 1.0:
        return 0.0 if s == n else -np.inf
    return s * np.log(theta) + (n - s) * np.log(1.0 - theta)

# Szélső konvenciók ellenőrzése
assert bernoulli_loglik(0.0, 0, 10) == 0.0
assert bernoulli_loglik(1.0, 10, 10) == 0.0
assert bernoulli_loglik(0.0, 1, 10) == -np.inf

# A Bernoulli-MLE helyének ellenőrzése rácsmaximummal: s/N = 0.6
grid = np.linspace(0.0, 1.0, 1001)
ll_grid = np.array([bernoulli_loglik(t, 3, 5) for t in grid])
assert np.isclose(grid[np.argmax(ll_grid)], 0.6, atol=1e-3)

# --- 3. Egyenletes [0,1] momentumok: zárt képlet és midpoint Riemann ---
a_val, b_val = 0.0, 1.0
n_bins = 1000
h = (b_val - a_val) / n_bins
mid = a_val + (np.arange(n_bins, dtype=np.float64) + 0.5) * h
density = 1.0 / (b_val - a_val)

Ex_num = np.sum(mid * density * h)
Ex2_num = np.sum(mid**2 * density * h)
Ex_analytic = (a_val + b_val) / 2.0
Ex2_analytic = (a_val**2 + a_val * b_val + b_val**2) / 3.0
var_analytic = (b_val - a_val)**2 / 12.0

print("Ex numerikus  :", Ex_num)
print("Ex analitikus :", Ex_analytic)
print("Ex2 numerikus :", Ex2_num)
print("Ex2 analitikus:", Ex2_analytic)
print("abs hiba Ex   :", abs(Ex_num - Ex_analytic))
print("Var numerikus :", Ex2_num - Ex_num**2)
print("Var analitikus:", var_analytic)

# Ellenőrzés: a midpoint szabály lineárisra pontos, a varianciahiba ~1/n_bins^2
assert np.isclose(Ex_num, Ex_analytic, atol=1e-12, rtol=0.0)
assert np.isclose(Ex2_num, Ex2_analytic, atol=1e-6, rtol=0.0)
assert abs((Ex2_num - Ex_num**2) - var_analytic) < 1e-6
print("ellenorzes: a momentumok a kezi integraltol a megadott tolerancian belul vannak")
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1, seed 20261020):

```
counts     : [110  58  32   0]
```

```
p_hat      : [0.55 0.29 0.16 0.  ]
sum        : 1.0
nemnegativ : True
Ex numerikus  : 0.5
Ex analitikus : 0.5
Ex2 numerikus : 0.33333325
Ex2 analitikus: 0.3333333333333333
abs hiba Ex   : 0.0
Var numerikus : 0.08333325000000003
Var analitikus: 0.08333333333333333
ellenorzes: a momentumok a kezi integraltol a megadott tolerancian belul vannak
```

Az utolsó rész egyezése: a $U(0,1)$ várható érték analitikus értéke 0,5; a midpoint szabály lineáris függvényekre a szimmetria miatt pontos, ezért az `Ex_num` 0,5-öt ad, és `abs hiba Ex` ezen a környezeten 0,0. Ez nem általános bitgarancia: a szimmetrikus összegzés miatt a kerekítési hiba itt nulla, ezért az `assert` 10^{-12} abszolút tűrést használ. A varianciánál a numerikus $E[X^2]-(E[X])^2$ körülbelül $1/12$; a midpoint szabály hibájának elméleti nagyságrendje $h^2/12$ a másodfokú x^2 függvényre, ahol $h=10^{-3}$, azaz $8{,}3\cdot10^{-8}$. A mért eltérés ezzel egyezik; a választott 10^{-6} tűrés egy nagyságrenddel a hibahatár felett van, hogy a platformonkénti kerekítési eltéréseket is elviselje. A `counts` értékei a fenti rögzített környezetben $[110, 58, 32, 0]$; a kategorikus rész három `assert`-je a nulla gyakoriságú osztályra determinisztikusan teljesül, mert a generáló $p_3=0$; a Bernoulli-log-likelihood szélső konvencióit további három `assert`, a Bernoulli-MLE helyét egy rácsmaximumot ellenőrző `assert`, a momentumokat pedig három `assert` ellenőrzi.

5.2.9 Feladatok

F1 — fogalmi, nehézség 1. Definiálja a diszkrét várható értéket, és mutassa meg, hogy a $P(X=2^k)=2^{-k}$, $k=1,2,\ldots$ eloszlás teljesíti a normalizálást, de nincs véges várható értéke. Adja meg, melyik feltétel sérül.

F2 — levezetési, nehézség 2. Vezesse le a Bernoulli-eloszlás varianciáját $E[X]=p$ és $E[X^2]=p$ felhasználásával, majd N független Bernoulli-változó $S=\sum_i X_i$ összegének varianciáját a kovarianciás összegképletből. Mondja ki, hol használja a függetlenséget.

F3 — implementációs, nehézség 2. Írjon `categorical_mle(y, K)` függvényt, amely az `y` int64 tömb és a K osztályszám alapján visszaadja a $\hat{p}_k=n_k/N$ becsléseket. Kezelje a nullszámosságot $\hat{p}_k=0$ értékkel, a kimenet nemnegatív és 1 összegű legyen. Ellenőrizze a bemenetet: üres minta, nem pozitív K, illetve a $[0,K-1]$ tartományon kívüli címke esetén jelezzen hibát. Tesztelje `y = np.array([0,0,2])`, `K=3` bemeneten, valamint a hibás bemenetek egy-egy példáján.

F4 — hibakeresési, nehézség 2. Az alábbi kód a Bernoulli MLE-t akarja kiszámítani.

```python
def bad_mle(s, N):
    if s == 0:
        return 0.0
    if s == N:
        return 1.0
    return s / (N - s)   # HIBA
```

Lokalizálja a hibát, adja meg az okát, a következményét $s=3, N=5$ esetén, és a javított képletet. Ellenőrizze a javítást úgy, hogy a log-likelihood deriváltja a kapott pontban nulla legyen.

F5 — kísérleti/transzfer, nehézség 3. Tervezzen kísérletet: a kategorikus MLE becslési hibája hogyan változik a minta elemszámával? A generáló eloszlás legyen $p=(0{,}5;\,0{,}3;\,0{,}2)$, $N\in\{50,200,1000\}$, legalább 3 vetőmag. Metrika: $L_1(\hat{p},p)=\sum_k|\hat{p}_k-p_k|$. Adjon protokollt,

viszonyítási alapot (baseline), megfigyelési egységet és következtetési korlátot. Előre megadott győztest nem kérünk.

5.2.10 Megoldások és indoklás

F1. A diszkrét várható érték definíciója: $E[X] = \sum_x x\, p_X(x)$, ha $\sum_x |x| p_X(x) < \infty$. A megadott eloszlás normalizált: $\sum_{k=1}^{\infty} 2^{-k} = 1$. A momentumhoz tartozó sor $\sum_{k=1}^{\infty} 2^k \cdot 2^{-k} = \sum_{k=1}^{\infty} 1$, ami divergál, tehát az abszolút összegezhetőség feltétele sérül; $E[X]$ nem létezik véges értékként. Ez mutatja, hogy a normalizálás önmagában nem garantál momentumot.

F2. Bernoulli esetén $E[X] = 1 \cdot p + 0 \cdot (1-p) = p$, és mivel $X^2 = X$, ezért $E[X^2] = p$. A számítóképletből $\mathrm{Var}(X) = E[X^2] - (E[X])^2 = p - p^2 = p(1-p)$. Az összeg varianciájához a kovarianciás képletet iterálva:

$$\mathrm{Var}(S) = \sum_{i} \mathrm{Var}(X_i) + \sum_{i \neq j} \mathrm{Cov}(X_i, X_j).$$

Független X_i, X_j esetén a kovariancia 0; itt használjuk a függetlenséget. Így $\mathrm{Var}(S) = \sum_i p(1-p) = Np(1-p)$. Alakellenőrzés: $S \in \{0, \dots, N\}$, a variancia nemnegatív, és $p = 0$ vagy $p = 1$ esetén 0, ahogy a konstans változótól várjuk.

F3. A függvény:

```
import numpy as np

def categorical_mle(y, K):
    y = np.asarray(y)
    if K <= 0:
        raise ValueError("K-nak pozitívnak kell lennie")
    if y.size == 0:
        raise ValueError("az y minta nem lehet ures")
    if not np.issubdtype(y.dtype, np.integer):
        raise TypeError("az y cimkei egesz szamok legyenek")
    if np.any(y < 0) or np.any(y >= K):
        raise ValueError("a cimkeknek 0 es K-1 koze kell esniuk")
    counts = np.bincount(y, minlength=K).astype(np.float64)
    return counts / counts.sum()

y = np.array([0, 0, 2], dtype=np.int64)
p = categorical_mle(y, 3)
assert p.shape == (3,)
assert np.all(p >= 0.0)
assert np.isclose(p.sum(), 1.0, atol=1e-12)
assert p[1] == 0.0 and np.isclose(p[0], 2.0 / 3.0)

# Hibas bemenetek: mindegyiknek kivetelt kell dobnia
for bad_y, bad_K in [(np.array([], dtype=np.int64), 3),
                     (np.array([0, 3], dtype=np.int64), 3),
                     (np.array([0, -1], dtype=np.int64), 3)]:
    try:
        categorical_mle(bad_y, bad_K)
    except ValueError:
        pass
    else:
        raise AssertionError("hibas bemenetre nem jelzett hibat")
```

A `minlength=K` biztosítja, hogy a nem látott osztály is szerepeljen 0 számmal. A bemenetellenőrzés nélkül az üres `y` a `counts.sum() == 0` miatt nullával osztana, a K-nál nagyobb címke

pedig a `bincount` kimenetét K-nál hosszabbra növelné, és a kimenet hossza nem egyezne a várt K-val. Az ellenőrzés ezeket a hibákat a számítás előtt, egyértelmű kivétellel jelzi.

F4. A hiba a derivált egyenletének hibás megoldása. A $\ell'(\theta) = 0$ feltételből $\theta/(1-\theta) = s/(N-s)$ adódik, nem $\theta = s/(N-s)$. A hibás képlet $s = 3, N = 5$ esetén $3/2 = 1{,}5$-öt ad, ami kívül esik a $[0, 1]$ paramétertartományon, tehát nem valószínűség. Következmény: a „becslés" értelmetlen, és a likelihood nem is értelmezett ott. Javítás: $\hat{\theta} = s/N = 0{,}6$. Ellenőrzés: $\ell'(0{,}6) = 3/0{,}6 - 2/0{,}4 = 5 - 5 = 0$, továbbá $\ell''(0{,}6) < 0$, tehát maximum.

F5. Egy lehetséges terv: a megfigyelési egység egy adott N méretű, adott vetőmaggal generált kategorikus minta; a generáló p rögzített, tehát a becsült $\hat{p}$ eltérése az ismert igazságtól mérhető. Minden $(N, \text{vetőmag})$ cellában kiszámítjuk $L_1(\hat{p}, p)$-t, és az azonos N-hez tartozó értékeket átlagoljuk. Viszonyítási alap (baseline): mindig az egyenletes $1/K$ becslés L_1 hibája, hogy a tanult becslés ennél jobb-e. A protokoll rögzíti a vetőmagokat $(20261020, 1, 2)$, a NumPy-verziót és a `default_rng` generátort. Várt trend: nagyobb N mellett az átlagos L_1 csökken, mert a számláló becslés konzisztens (a minta elemszámának növekedésével valószínűségben a valódi paraméterhez tart); ezt nem garantáljuk előre az adott 3 vetőmagra. Következtetési korlát: három seed kis minta a becslési eljárás szórására, nem bizonyítja a konvergenciát; az eredmény a generáló eloszlásra érvényes, más eloszlásra nem. Döntési szabály: a számláló becslő akkor tekinthető jobbnak az egyenletes baseline-nál, ha az átlagos L_1 mindhárom rögzített seednél kisebb; erőforrás: CPU, NumPy, a rögzített `cnn-labor` környezet.

5.2.11 Tipikus tévedés ellenpéldával

A leggyakoribb értelmezési hiba: *„a likelihood a paraméter valószínűsége, tehát normalizálással eloszlást kapunk prior nélkül."* Ez nem igaz, és a normalizálás önkényes eredményre vezet. Tekintsük az $s = 7$, $N = 10$ Bernoulli-mintát. A likelihood értékei $\theta = 0{,}5$ és $\theta = 0{,}7$ esetén $0{,}5^{10} = 0{,}000977$ és $0{,}7^7 \cdot 0{,}3^3 \approx 0{,}002224$; a log-likelihoodok $\ell(0{,}5) \approx -6{,}931$ és $\ell(0{,}7) = 7 \log 0{,}7 + 3 \log 0{,}3 \approx -6{,}109$. Ha a $\Theta = \{0{,}5;\ 0{,}7\}$ rácson normalizálunk, a 0,7-hez tartozó súly $0{,}002224/(0{,}002224 + 0{,}000977) \approx 0{,}69$. Ha viszont a rács $\{0{,}3;\ 0{,}7\}$, a 0,7-hez tartozó súly $0{,}002224/(0{,}002224 + 0{,}000075) \approx 0{,}97$. Ugyanannak a mintának és paraméterpontnak „valószínűsége" meredeken függ attól, milyen más pontokat vettünk fel a rácson — egy valódi eloszlás ezt nem teheti. A likelihood csak hányadosokban hordoz összehasonlító erőt: $\mathcal{L}(0{,}7)/\mathcal{L}(0{,}5) \approx 2{,}28$ rácsfüggetlen. Ezzel szemben a posterior az 5.1. Esemény, feltétel és véletlen változó Bayes-tételéből származik, és prior nélkül nem definiált.

5.2.12 Önellenőrzés és visszatekintés

- Ellenőrizze, hogy minden momentum-integrálnál megadta az abszolút integrálhatóságot, és a varianciaszámításnál a $b > a$ feltételt.
- A likelihood levezetésnél a szélső eseteket külön kezelte-e, nemcsak a belső derivált zérushelyét?
- A numerikus összevetésnél megadta-e a vetőmagot (seed), az adattípust (dtype), a toleranciát és azt, hogy a kimenet szemléltetés vagy tényleges mérés?

Visszatekintés: 5.1. Esemény, feltétel és véletlen változó alfejezetben a sűrűséget területként értelmeztük — ezt használtuk az egyenletes momentum integráljában. 4.2. Parciális derivált és gradiens alfejezetben a gradiens alak-egyeztetését tanultuk — a likelihood maximumának keresése egyváltozós deriválás, de ugyanaz a stacionáriuspont-logika.

5.2.13 Rövid összefoglaló

A várható érték az eloszlás középpontját, a variancia a szóródását méri; mindkettőhöz véges momentum szükséges. A kovariancia két változó együttes ingadozását összegzi, de a nulla kovariancia nem jelent függetlenséget. Egyenletes eloszlásra az első két momentum a Newton–Leibniz-formulával, polinom-primitív függvényekkel vezethető le. A likelihood rögzített adat mellett a paraméter függvénye, szorzatalakját a logaritmus összeggé alakítja. Bernoulli-minta esetén a log-likelihood konkáv, a belső maximum s/N, a 0 és N szélső eseteket külön kell kezelni. A likelihood nem normalizálható a paraméter eloszlásává prior nélkül.

5.2.14 Továbblépés

Az 5.3. Mintavételi bizonytalanság és intervallum a becslés bizonytalanságát vizsgálja: az MLE egyetlen szám, de más minta más becslést adna. Ott a konfidenciaintervallum és a bootstrap (újramintavételezéses becslés) segítségével különítjük el a paraméter valódi értékét a mintavételi ingadozástól.

5.2.15 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

5.3 Mintavételi bizonytalanság és intervallum

5.3.1 Cél, előismeret és induló becslés

Az 5.2. Várható érték, variancia és likelihood egységben a Bernoulli-minta likelihoodjából megkaptuk, hogy egy N elemű megfigyelés mellett az ismeretlen p valószínűség maximum-likelihood becslése $\hat{p} = s/N$, ahol s a sikerek száma. Ez a becslés azonban egyetlen szám. Ha a generátor másik véletlen mintát adott volna, a becsült $\hat{p}$ is más lenne: a becslés maga is véletlen mennyiség. Jelen egységben ezt az ingadozást formalizáljuk, és azt a kérdést tesszük fel, hogyan tudunk olyan tartományt adni, amely megbízhatóan lefedi az igazi paramétert.

Kezdő becslésként gondoljunk egy $N = 10$ méretű Bernoulli-mintára. Ha $s = 4$ sikert kapunk, az MLE $\hat{p} = 0{,}4$. Az olvasó becsülje meg: mekkora az az intervallum, amelyet „biztonságosnak” érez a valódi p számára? A fejezet végére ezt az intuitív képet precíz eljárással váltjuk ki.

A szükséges előismeret az 5.2. Várható érték, variancia és likelihood valószínűségi alapfogalma: véletlen változó, várható érték, variancia, likelihood és sűrűségfüggvény. A 4.3. Jacobi-mátrix, láncszabály és VJP dimenzió-ellenőrzési szokása annyiban tér vissza, hogy minden becslésnél megadjuk a tenzor alakját, az 1.3. Adatfelosztás és a megfigyelési egység megfigyelési-egység elve pedig a függetlenségi feltétel pontos megfogalmazásakor lesz döntő.

5.3.2 A minta és a becslő definíciója

Legyen

$$X_1, \dots, X_N \quad \text{azonos eloszlású és független (iid)}$$

Bernoulli-minta, azaz minden i-re $X_i \in \{0,1\}$ és $P(X_i = 1 \mid p) = p$. Az egyes megfigyelések értékeit jelölje x_i. A vektorba rendezett megfigyelés $x = (x_1, \dots, x_N)^\top \in \{0,1\}^N$; Python-tenzor alakja `(N,)`, `float32` vagy `int64`, bár a Bernoulli-célhoz az `int64` a természetes. Itt az oszlopvektor-konvenció a minta matematikai tárgyalását szolgálja; a kódban a NumPy-tömb hossza adott.

A becslési eljárás és a becsült érték megkülönböztetése kulcskérdés.

Definíció (statisztika). A mintából számolt tetszőleges $T : \{0,1\}^N \to \mathbb{R}$ függvényt statisztikának nevezzük. Fontos esete a Mintaösszeg-számláló:

$$S(x) = \sum_{i=1}^{N} x_i,$$

alakja Pythonban egyetlen `int64` skalár.

Definíció (becslő). A p paraméter becslője

$$\hat{p}_N(x) = \frac{S(x)}{N} = \frac{1}{N} \sum_{i=1}^{N} x_i.$$

A $x \mapsto \hat{p}_N(x)$ leképezés a minta teréből a $[0,1]$ paramétertartományba képez; ugyanaz a becslő más mintán más értéket ad. Ezért a $\hat{p}_N$ maga is véletlen változó, amelynek eloszlása van. A p paraméter ezzel szemben ismeretlen konstans. Ez a felfogás a klasszikus, gyakorisági statisztika kerete.

Várható érték. Mivel a X_i-k Bernoulli-eloszlásúak p-re, a várható érték linearitása miatt

$$\mathbb{E}[\hat{p}_N] = \mathbb{E}\left[\frac{1}{N}\sum_{i=1}^{N} X_i\right] = \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}[X_i] = \frac{1}{N}\sum_{i=1}^{N} p = p.$$

Ez a torzítatlanság: a becslő eloszlása a valódi paraméter köré központosul.

Variancia. A függetlenség miatt a tagok kovarianciája nulla, ezért

$$\mathrm{Var}(\hat{p}_N) = \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} X_i\right) = \frac{1}{N^2}\sum_{i=1}^{N} \mathrm{Var}(X_i) = \frac{1}{N^2}\sum_{i=1}^{N} p(1-p) = \frac{p(1-p)}{N}.$$

Itt felhasználtuk, hogy Bernoulli-eloszlásra $\mathrm{Var}(X_i) = p(1-p)$ [B01].

A szóródás mértékegységben kifejezett változata a standard hiba.

Definíció (standard hiba). A $\hat{p}_N$ becslő standard hibája

$$\mathrm{SE}(\hat{p}_N) = \sqrt{\mathrm{Var}(\hat{p}_N)} = \sqrt{\frac{p(1-p)}{N}}.$$

Ez az érték az ismételt mintavételi ingadozás tipikus nagyságrendjét méri, de a valódi p-től függ, így önmagában nem ismert. A megfigyelt mintából képezhető standardhiba-becslés

$$\widehat{\mathrm{SE}} = \sqrt{\frac{\hat{p}_N(1-\hat{p}_N)}{N}},$$

amely a valódi p helyére az MLE-t írja. Ez már kiszámítható a mintából.

5.3.3 A becslő ingadozása ismételt mintavétellel

A standard hiba elvont, analitikus képlet. A kérdés az, hogyan mutatható meg közvetlenül az ingadozás. Ehhez szimulációs protokollt rögzítünk, amely a valódi paraméter ismeretét többszöri független minta generálására használja.

Protokoll (szimulációs kísérlet).

- Kérdés: Az MLE ingadozása $N = 20$ mellett mekkora, és a standardhiba-képlet közelíti-e a tapasztalt szórást?
- Mit csinálunk: rögzítünk egy igazi p_0-t, generálunk $R = 1000$ független mintát, mindegyiket $N = 20$ eleművé, minden mintára kiszámoljuk az MLE-t, és a kapott becslésvektorból tapasztalati szórást és hisztogramot készítünk.
- Fontos: ez a szimuláció a mintavételi eloszlást vizsgálja; a gyakorlatban csak egy minta áll rendelkezésre.

Legyen a kézenfekvő példa: $p_0 = 0{,}7$, $N = 20$, $R = 1000$. A lehetséges minták száma $2^{20} = 1\,048\,576$; a szimuláció $R = 1000$ független mintát véletlenszerűen választ ebből a térből, tehát mintavételez, és nem járja be teljesen a mintateret. A standard hiba a képletből

$$\mathrm{SE} = \sqrt{\frac{0{,}7 \cdot 0{,}3}{20}} = \sqrt{\frac{0{,}21}{20}} = \sqrt{0{,}0105} \approx 0{,}10247.$$

Ennek ellenőrzése a szimulációval a szakasz laborjának célja. A tapasztalati szórás várhatóan közel lesz 0,10247-hez, de nem pontosan egyenlő vele, hiszen 1000 mintából kapott becslés ingadozik.

A becslő minőségének mérésére két további fogalom szolgál.

Definíció (torzítás). A $\hat{\theta}$ becslő torzítása

$$\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

A fenti MLE torzítatlan, így $\mathrm{Bias}(\hat{p}_N) = 0$. Torzítás nélkül is lehet nagy a szórás; a kettőt együtt méri a négyzetes hiba.

Definíció (négyzetes hiba). $\mathrm{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$.

A várható érték linearitásával, a $\mathbb{E}[\hat{\theta}] = \mu$ jelöléssel,

$$\begin{aligned}
\mathrm{MSE}(\hat{\theta}) &= \mathbb{E}\left[\left((\hat{\theta} - \mu) + (\mu - \theta)\right)^2\right] \\
&= \mathbb{E}[(\hat{\theta} - \mu)^2] + 2(\mu - \theta)\mathbb{E}[\hat{\theta} - \mu] + (\mu - \theta)^2 \\
&= \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2.
\end{aligned}$$

Az MLE esetében ezért

$$\mathrm{MSE}(\hat{p}_N) = \frac{p(1-p)}{N}.$$

Ez a dekompozíció az, amely miatt a torzítást és a szórást külön érdemes mérni: egy torzítatlan, de nagy szórású becslő rosszabb lehet, mint egy enyhén torzított, de kis szórású. A dekompozíció levezetett állítás, feltétele csupán az, hogy mindkét momentum véges legyen.

5.3.3.1 A mintavételi eloszlás mérése

A protokollt önálló szkripttel hajtjuk végre; a lenti kimenet a rögzített környezetben mért érték. A munkakönyvtár: `lab/n019`, a kód `sampling_distribution.py` néven fut.

```
"""Az MLE mintavételi eloszlása ismételt mintavétellel.

N=20, p0=0.7, R=1000 független minta; tapasztalati szórás és hisztogram.
"""
from __future__ import annotations

import numpy as np

rng = np.random.default_rng(20261020)
N = 20
R = 1000
p0 = 0.7

x = rng.binomial(1, p0, size=(R, N))
p_hat = x.mean(axis=1)
se_theory = np.sqrt(p0 * (1 - p0) / N)

print(f"p0={p0}, N={N}, R={R}")
print("tapasztalati szórás:", p_hat.std(ddof=1))
print("elméleti standard hiba:", se_theory)

# Egész számlálóból képzett, a lehetséges p_hat értékekkel egyező osztályhatárok.
bins = np.arange(5, 21) / 20.0
hist, edges = np.histogram(p_hat, bins=bins)
for lo, hi, cnt in zip(edges[:-1], edges[1:], hist):
    right = "]" if hi == 1.0 else ")"
    print(f"[{lo:.2f},{hi:.2f}{right}: {cnt}")
print("0.25 alatt:", (p_hat < 0.25).sum())

assert p_hat.shape == (R,)
assert abs(p_hat.std(ddof=1) - se_theory) < 0.01
assert hist.sum() + (p_hat < 0.25).sum() == R
print("ellenorzes: alak, Monte-Carlo-hiba es hisztogram-lefedettseg rendben")
```

A mért kimenet:

```
p0=0.7, N=20, R=1000
tapasztalati szórás: 0.1034559840736177
elméleti standard hiba: 0.10246950765959599
[0.25,0.30): 0
[0.30,0.35): 0
[0.35,0.40): 0
[0.40,0.45): 7
[0.45,0.50): 6
[0.50,0.55): 34
[0.55,0.60): 71
```

```
[0.60,0.65): 116
[0.65,0.70): 166
[0.70,0.75): 183
[0.75,0.80): 164
[0.80,0.85): 143
[0.85,0.90): 78
[0.90,0.95): 22
[0.95,1.00]: 10
0.25 alatt: 0
ellenorzes: alak, Monte-Carlo-hiba es hisztogram-lefedettseg rendben
```

Értelmezés: a tapasztalati szórás 0,10346, az elméleti standard hiba 0,10247; a különbség 0,00099, ami jóval a 1000 minta melletti Monte-Carlo-hiba ($\sigma/\sqrt{2R} \approx 0{,}0023$) alatt van. A hisztogram a 0,25 és 1,0 közötti értékeket 0,05 széles osztályokba sorolja; a legfelső osztály a 0,95 és 1,0 értékeket is tartalmazza, a 0,25 alatti minták száma 0, ezért a 15 osztály együttesen 1000 mintát ad. Az alak haranggörbére emlékeztet, ami a centrális határeloszlás-tétel előrevetítése; a Wald-intervallum tárgyalásánál ezt az aszimptotikus normalitást használjuk.

5.3.4 Konfidenciaintervallum

A konfidenciaintervallum az ismételt mintavétel nyelvén fogalmaz meg garanciát. Nem a paraméterről állít valószínűséget; a véletlen az intervallum készítésében van.

Definíció (konfidenciaintervallum). Az $1-\alpha$ szintű konfidenciaintervallum-eljárás olyan $[L(X), U(X)]$, ahol L, U a minta mérhető függvényei, és minden megengedett p paraméterre

$$P_p(L(X) \le p \le U(X)) \ge 1-\alpha.$$

A P_p jelölés hangsúlyozza: az eljárást ugyanazzal a p-vel ismételten mintavéve, hosszú távon legalább $1-\alpha$ arányban fedi le az intervallum a paramétert. Az egyenlőtlenség a konzervatív változat; sok gyakorlati intervallum csak aszimptotikusan éri el a névleges lefedettséget.

Ez a definíció az, ami a gyakori félreértést eloszlatja: a 95%-os szint nem jelenti azt, hogy az aktuálisan kiszámított $[l, u]$ 0,95 valószínűséggel tartalmazza a paramétert. A paraméter vagy benne van, vagy nincs; a valószínűségi állítás az eljárás hosszú távú viselkedéséről szól. Egy adott intervallum utólagos valószínűsége posterior-értelmezést igényelne, amelyhez prior kell.

Ebben az egységben a Bernoulli-paraméterre egy egyszerű, konzervatív intervallumot építünk, amely a Hoeffding-korláton alapul. A szükséges koncentrációs állítást pontos feltételekkel mondjuk ki.

Tétel (Hoeffding-korlát Bernoulli-mintára). Legyen $X_1, \dots, X_N$ iid Bernoulli(p) minta. Ekkor bármely $\varepsilon > 0$ mellett

$$P(|\hat{p}_N - p| \ge \varepsilon) \le 2\exp(-2N\varepsilon^2).$$

Bizonyítás. Legyen $Y_i = X_i - p$, ekkor $\mathbb{E}[Y_i] = 0$, és vezessük be az $S = \sum_{i=1}^{N} Y_i = N(\hat{p}_N - p)$ jelölést. Rögzítsünk egy $t > 0$ értéket. A Markov-egyenlőtlenséget az e^{tS} nemnegatív változóra alkalmazva

$$P(\hat{p}_N - p \ge \varepsilon) = P(S \ge N\varepsilon) = P\big(e^{tS} \ge e^{tN\varepsilon}\big) \le e^{-tN\varepsilon}\,\mathbb{E}[e^{tS}].$$

A Y_i változók függetlenek, ezért az exponenciálisok szorzattá esnek szét:

$$\mathbb{E}[e^{tS}] = \prod_{i=1}^{N} \mathbb{E}[e^{tY_i}] = \left(\mathbb{E}[e^{tY_1}]\right)^N.$$

Bernoulli-változóra

$$\mathbb{E}[e^{tY_1}] = (1-p)e^{-tp} + pe^{t(1-p)} = e^{-tp}\left(1 - p + pe^{t}\right) = e^{\varphi(t)},$$

ahol $\varphi(t) = -tp + \ln(1 - p + pe^t)$. Itt $\varphi(0) = 0$, és az első derivált

$$\varphi'(t) = -p + \frac{pe^t}{1 - p + pe^t}.$$

A $q(t) = \frac{pe^t}{1-p+pe^t} \in (0,1)$ jelöléssel $\varphi'(t) = -p + q(t)$, ezért $\varphi'(0) = 0$. A második derivált

$$\varphi''(t) = q(t)\,(1 - q(t)) \le \frac{1}{4},$$

mert a $q(1-q)$ parabola maximuma $q = 1/2$-nél $1/4$. A Taylor-formula Lagrange-maradéktaggal ezért valamely $\xi \in (0, t)$ mellett

$$\varphi(t) = \varphi(0) + t\varphi'(0) + \frac{t^2}{2}\varphi''(\xi) \le \frac{t^2}{8}.$$

Visszahelyettesítve

$$P(\hat{p}_N - p \ge \varepsilon) \le \exp\left(-tN\varepsilon + \frac{Nt^2}{8}\right).$$

A kitevő t-ben másodfokú, és a $t = 4\varepsilon$ választás adja a minimumát:

$$-tN\varepsilon + \frac{Nt^2}{8} = -4N\varepsilon^2 + \frac{16N\varepsilon^2}{8} = -2N\varepsilon^2.$$

Ezért $P(\hat{p}_N - p \ge \varepsilon) \le \exp(-2N\varepsilon^2)$. Az alsó irányra alkalmazzuk ugyanezt a $Z_i = 1 - X_i$ Bernoulli$(1-p)$ mintára; ekkor $\bar{Z} - (1-p) = -(\hat{p}_N - p)$, így

$$P(\hat{p}_N - p \le -\varepsilon) = P(\bar{Z} - (1-p) \ge \varepsilon) \le \exp(-2N\varepsilon^2).$$

A két egyoldalú korlátot az unió-korlát (union bound) köti össze:

$$P(|\hat{p}_N - p| \ge \varepsilon) \le 2\exp(-2N\varepsilon^2).$$

$\square$

A bizonyítás saját, rövid levezetés; az általános $[0,1]$-beli változókra vonatkozó Hoeffding-lemma a koncentrációs egyenlőtlenségek szakirodalmában, például Boucheron, Lugosi és Massart [B04] munkájában található. A szimuláció megmutatja, hogy a korlát mennyire konzervatív.

Következmény (Hoeffding-féle intervallum). Válasszunk $\alpha \in (0,1)$ szintet, és legyen

$$\varepsilon_N = \sqrt{\frac{\log(2/\alpha)}{2N}}.$$

Ekkor az intervallum

$$[\hat{p}_N - \varepsilon_N,\ \hat{p}_N + \varepsilon_N]$$

legalább $1-\alpha$ lefedettségű konfidenciaintervallum.

Levezetés. A tételben a lefedettség romlását $2\exp(-2N\varepsilon^2) \le \alpha$ akarjuk, ami ekvivalens $\varepsilon^2 \ge \frac{\log(2/\alpha)}{2N}$ feltétellel. Az egyenlőséggel választott ε_N tehát garantálja, hogy a lefedési hiba legfeljebb α. Mivel az intervallum szimmetrikus és a tétel a $|\hat{p}_N - p|$-re vonatkozik, az állítás következik.

Ez a levezetés a Bernoulli-esetre szól. A mögötte álló általános Hoeffding-tételt nem bizonyítjuk, csak hivatkozzuk: ha $Y_1, \dots, Y_N$ független, $[a,b]$-korlátos változók, akkor

$$P\big(|\bar{Y} - \mathbb{E}\bar{Y}| \ge \varepsilon\big) \le 2\exp\left(-\frac{2N\varepsilon^2}{(b-a)^2}\right).$$

Ebben az alakban nem kell sem azonos eloszlás, sem diszkrétség, ezért bármely $[a,b]$-be eső, akár folytonos célváltozóra is érvényes; a paraméter szerepét az átlagos várható érték veszi át. Bernoulli-mintára $Y_i = X_i$, $\bar{Y} = \hat{p}_N$, $\mathbb{E}\bar{Y} = p$ és $b-a = 1$, ami visszaadja a fenti alakot. A korlát akkor nem alkalmazható, ha a megfigyelések függnek egymástól (például ugyanazon jelenet két nézete, 5.4. Függő képek, seedek és resampling), vagy ha a változó nem korlátos.

Számpéldával: legyen $N = 12$ és $\alpha = 0{,}05$, azaz 95%-os szint. Ekkor

$$\varepsilon_{12} = \sqrt{\frac{\log(40)}{24}} = \sqrt{\frac{3{,}6889}{24}} = \sqrt{0{,}1537} \approx 0{,}3921.$$

Ha a mintában $s = 5$ sikert figyeltünk meg, akkor $\hat{p} = 5/12 \approx 0{,}4167$, így az intervallum:

$$[0{,}0246,\ 0{,}8087].$$

Ez nagyon széles. A Hoeffding-korlát erőssége az, hogy minden p-re és minden N-re érvényes; a hátránya az, hogy a névleges szintnél sokkal konzervatívabb, különösen közepes N-re. Ezért nagyobb mintánál pontosabb, normális approximációra épülő intervallumokat használunk.

5.3.5 A Wald-intervallum és lefedettségi vizsgálata

Állítás (a mintaátlag aszimptotikus normalitása). Ha $X_1, \dots, X_N$ iid Bernoulli(p) minta, akkor a centrális határeloszlás-tétel szerint

$$\frac{\hat{p}_N - p}{\sqrt{p(1-p)/N}} \xrightarrow{d} \mathcal{N}(0,1) \quad (N \to \infty).$$

Ez a Bernoulli-esetben a binomiális eloszlás normális approximációja. A gyakorlatban p helyére a konzisztens $\hat{p}_N$ becslést írva, a folytonos leképezés tételével és Slutsky-tételével ugyanez érvényes marad a becsült standard hibával is:

$$\frac{\hat{p}_N - p}{\widehat{\mathrm{SE}}} \xrightarrow{d} \mathcal{N}(0,1).$$

Ezen aszimptotikus normalitás alapján a kézenfekvő közelítő intervallum

$$\hat{p}_N \pm z_{1-\alpha/2}\,\widehat{\mathrm{SE}},$$

ahol $z_{1-\alpha/2}$ a standard normális eloszlás kvantilise. Ez a Wald-intervallum. Csak nagy mintára érvényes közelítő eljárás, ezért a definícióban a névleges lefedettség nem garantált. Ennek ellenére a programozói gyakorlatban a lefedettségvizsgálat a leggyorsabb ellenőrzés arra, hogy az eljárás egy konkrét p, N páros mellett mennyire használható.

A Wald-intervallum névleges eljárása:

$$L = \hat{p}_N - z\sqrt{\frac{\hat{p}_N(1-\hat{p}_N)}{N}}, \quad U = \hat{p}_N + z\sqrt{\frac{\hat{p}_N(1-\hat{p}_N)}{N}}.$$

Az intervallumot a $[0,1]$ tartományra szokás metsszeni, mivel a paraméter eleve oda esik; a mi laborunk ezt a lépést is elvégzi.

A lefedettségi szimuláció a definíciót operacionalizálja. Rögzítünk egy p rácsot, például 0,01; 0,02; ...; 0,99, minden p-re sok, egymástól független mintát generálunk, minden mintából intervallumot készítünk, és megnézzük, hány intervallum tartalmazza az adott p-t. A lefedettségi arány a rács függvényében rajzolva megmutatja a Wald-intervallum híres gyengeségeit.

A gyakori tévedés itt már látható: a szimuláció a p ismeretére épül, tehát tanulási célra alkalmas, de valódi kísérletben a paramétert nem ismerjük. A lefedettségi szimuláció nem bizonyíték a Wald-intervallum általános érvényességére; ez egy empirikus mérés, amelynek határa a szimulált p-tartomány és az N választása.

5.3.6 Kézi kis példa

Tekintsünk egy $N = 5$ elemű Bernoulli-mintát: $x = (0,1,0,1,1)^\top$. Számoljuk ki a becslést, a standardhiba-becslést és a Hoeffding-intervallumot.

1. A sikerek száma $S = 0+1+0+1+1 = 3$, ezért $\hat{p} = 3/5 = 0{,}6$.
2. A standardhiba-becslés:

$$\widehat{\mathrm{SE}} = \sqrt{\frac{0{,}6 \cdot 0{,}4}{5}} = \sqrt{\frac{0{,}24}{5}} = \sqrt{0{,}048} \approx 0{,}2191.$$

3. A Hoeffding-intervallumhoz $\alpha = 0{,}05$ mellett

$$\varepsilon = \sqrt{\frac{\log(40)}{10}} = \sqrt{\frac{3{,}6889}{10}} = \sqrt{0{,}36889} \approx 0{,}6074.$$

Így a konzervatív intervallum $[0{,}6 - 0{,}6074;\ 0{,}6 + 0{,}6074]$, a $[0,1]$ metszet után $[0,1]$, ami természetesen mindig lefedi a paramétert, de nem informatív.

Ez a példa azt mutatja, hogy a kis minta és a konzervatív korlát kombinációja gyakorlatilag használhatatlan intervallumhoz vezethet. A Wald-intervallum ugyanerre a mintára $z_{0,975} \approx 1{,}960$ mellett:

$$\left[0{,}6 - 1{,}960 \cdot \sqrt{0{,}048};\ 0{,}6 + 1{,}960 \cdot \sqrt{0{,}048}\right] \approx [0{,}1706;\ 1{,}0294],$$

amelyet a $[0, 1]$ intervallumra metszve $[0{,}1706;\ 1]$-t kapunk. A Wald-intervallum itt már szűkebb a Hoeffding-féle $[0, 1]$-nél, de kis mintán nem garantálja a névleges lefedettséget.

Térjünk vissza az induló példához: $N = 10$, $s = 4$, $\hat{p} = 0{,}4$. A becsült standard hiba

$$\hat{\mathrm{SE}} = \sqrt{\frac{0{,}4 \cdot 0{,}6}{10}} = \sqrt{0{,}024} \approx 0{,}1549.$$

A Hoeffding-intervallum $\alpha = 0{,}05$ mellett $\varepsilon_{10} = \sqrt{\log(40)/20} = \sqrt{0{,}184444} \approx 0{,}4295$, így a nyers intervallum $[-0{,}0295;\ 0{,}8295]$, a $[0, 1]$-re metszés után $[0;\ 0{,}8295]$. A Wald-intervallum ugyanerre a mintára

$$[0{,}4 - 1{,}959964 \cdot 0{,}154919;\ 0{,}4 + 1{,}959964 \cdot 0{,}154919] \approx [0{,}0964;\ 0{,}7036].$$

A két intervallum szélessége jól mutatja a különbséget: a Hoeffding-féle konzervatív, a Wald-féle szűkebb, de kis mintán a névleges lefedettséget nem garantálja.

5.3.7 Labor: lefedettségi diagram sok szintetikus mintából

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor egyetlen NumPy-munkamenetben elvégzi a szimulációt; a lenti kimenetek rögzített környezetben (Python 3.12.3, NumPy 2.5.4, SciPy 1.18.1, CPU, OMP_NUM_THREADS=1, seed 20261020) mért értékek. A konkrét számok a NumPy PCG64 bitgenerátorára és az adott verzióra érvényesek; más NumPy-verzió vagy más bitgenerátor eltérő véletlen sorozatot adhat, ezért a kimenet csak a rögzített környezetben reprodukálható.

A munkakönyvtár: `lab/n019`. A kód `coverage_binomial.py` néven fut.

```
"""Bernoulli-becslő lefedettségi szimulációja.

A kód a Wald-intervallum és a Hoeffding-intervallum lefedettségét
becsüli ismételt mintavétellel. A kimeneteket a konzolra írja.
"""
from __future__ import annotations

import numpy as np
from scipy.stats import norm

rng = np.random.default_rng(20261020)
N = 20
R = 2000
alpha = 0.05
z = norm.ppf(1 - alpha / 2)
# 99 pontos rács; egész számlálóból osztva elkerüljük a float-lépés kerekítését
p_grid = np.arange(1, 100) / 100.0

cover_wald = np.zeros_like(p_grid)
cover_hoeff = np.zeros_like(p_grid)
```

```
for idx, p in enumerate(p_grid):
    x = rng.binomial(1, p, size=(R, N))
    p_hat = x.mean(axis=1)                          # egyszer számoljuk ki
    se = np.sqrt(np.clip(p_hat * (1 - p_hat), 0.0, 1.0) / N)
    low = np.maximum(0.0, p_hat - z * se)           # [0,1]-re metszés
    high = np.minimum(1.0, p_hat + z * se)
    cover_wald[idx] = np.mean((low <= p) & (p <= high))
    eps = np.sqrt(np.log(2 / alpha) / (2 * N))
    cover_hoeff[idx] = np.mean(np.abs(p_hat - p) <= eps)

print(f"N={N}, R={R}, alpha={alpha}")
print("Legkisebb Wald-lefedettség:", cover_wald.min())
print("Legnagyobb Wald-lefedettség:", cover_wald.max())
print("Legkisebb Hoeffding-lefedettség:", cover_hoeff.min())
print("Legnagyobb Hoeffding-lefedettség:", cover_hoeff.max())
print("Wald-lefedettség p=0.1-nél:", cover_wald[9])
print("Wald-lefedettség p=0.5-nél:", cover_wald[49])

# Beépített ellenőrzések: alak, [0,1] tartomány, Hoeffding tűréshatár
assert cover_wald.shape == p_grid.shape == (99,)
assert np.all((cover_wald >= 0.0) & (cover_wald <= 1.0))
assert np.all((cover_hoeff >= 0.0) & (cover_hoeff <= 1.0))
assert cover_hoeff.min() >= 1 - alpha - 5 * np.sqrt(alpha * (1 - alpha) / R)
print("ellenorzes: alak, lefedettsegi tartomany es Hoeffding-tures rendben")
```

A kódot a `python coverage_binomial.py` paranccsal futtatjuk. A mért kimenet:

```
N=20, R=2000, alpha=0.05
Legkisebb Wald-lefedettség: 0.169
Legnagyobb Wald-lefedettség: 0.956
Legkisebb Hoeffding-lefedettség: 0.991
Legnagyobb Hoeffding-lefedettség: 1.0
Wald-lefedettség p=0.1-nél: 0.882
Wald-lefedettség p=0.5-nél: 0.951
ellenorzes: alak, lefedettsegi tartomany es Hoeffding-tures rendben
```

Értelmezés:

- A Wald-lefedettség a tartomány szélein drámaian alacsony ($p = 0{,}01$ közelében 0,2 alatti), a közepén pedig a névleges 0,95 alatt és körül ingadozik.
- A Hoeffding-lefedettség minden p-re legalább $1 - \alpha$, és 1 közelében marad, mert a korlát konzervatív, az intervallum széles.
- A szimulált értékek $R = 2000$ ismétlés Monte-Carlo-hibáját hordozzák (névleges 0,95 mellett a standard hiba $\approx 0{,}005$); a pontos görbe a binomiális eloszlásból egzaktul is kiszámolható.

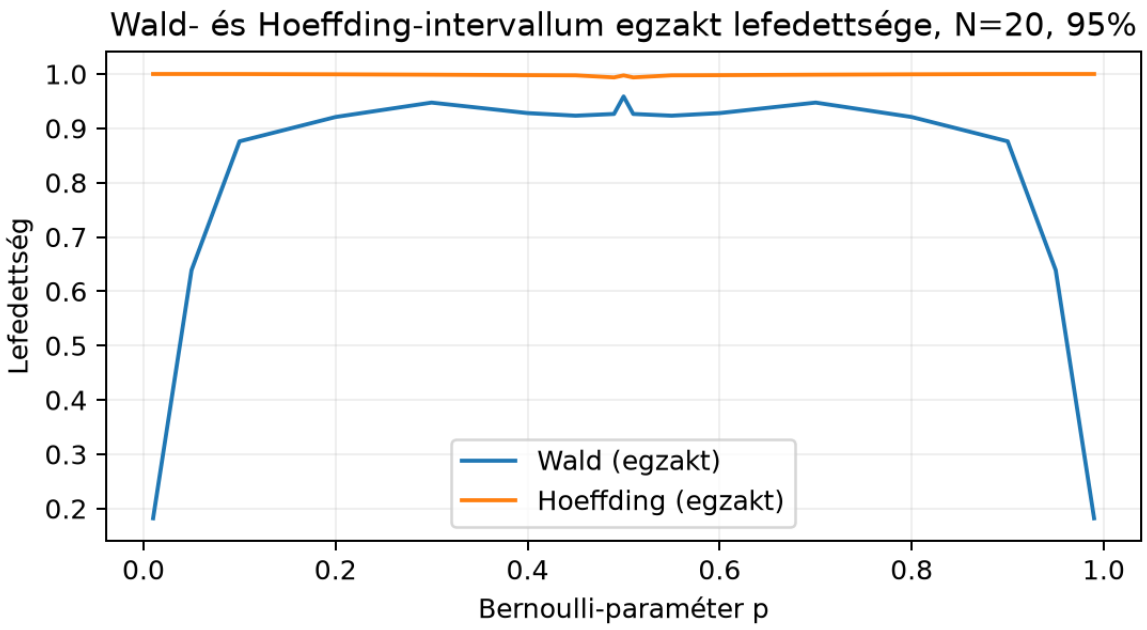
Az $R = 2000$ ismétlésből kapott 0,169 és az egzakt 0,18205 eltérése nem hiba: a szimulált arány standard hibája $p = 0{,}01$ mellett $\sqrt{0{,}182 \cdot 0{,}818/2000} \approx 0{,}0086$, így a 0,013-as különbség mintegy 1,5 standard hiba. Ehhez hozzájárul, hogy a minimumot 99 korrelált becslés fölött vesszük, ami a minimumot lefelé torzítja. A mérvadó görbe az egzakt számítás, a szimuláció csak Monte-Carlo-hibán belül követi.

A kód a következő ellenőrzéseket építi be:

- A `cover_wald.shape == p_grid.shape` és a `p_grid.size == 99`, mert `np.arange(1, 100) / 100.0` a 0,01-től 0,99-ig terjedő rácsot 99 elemként állítja elő; az egész számlálóból való osztás miatt a lebegőpontos lépés kerekítése nem befolyásolja a hosszt.
- A lefedettségi arányok a $[0, 1]$ intervallumba esnek.

- Az elméleti Hoeffding-lefedettség legalább $1-\alpha$, de a véges R futásból *becsült* arány mintavételi ingadozás miatt helyes kóddal is lehet $1-\alpha$ alatt (a szórása kb. $\sqrt{\hat{c}(1-\hat{c})/R}$). Az ellenőrzési szabály ezért nem a szigorú $\geq 1-\alpha$, hanem a becslés hibasávjával korrigált tűrés.
- A `se` képletben a `clip` védekező kód: matematikailag $\hat{p}(1-\hat{p}) \geq 0$ minden $\hat{p} \in [0,1]$-re, ezért negatív érték nem fordulhat elő; a `clip` a kerekítési hibákkal szembeni biztonsági háló, nem szükséges a helyességhez.

A diagram a Wald- és a Hoeffding-intervallum **egzakt** lefedettségét ábrázolja $N = 20$, $\alpha = 0{,}05$ mellett. A Wald-lefedettség $C_W(p) = \sum_{k=0}^{N} \binom{N}{k} p^k (1-p)^{N-k}\, \mathbf{1}[p \in W(k)]$, ahol $W(k)$ a $\hat{p} = k/N$-hez tartozó Wald-intervallum; a Hoeffding-lefedettség ugyanezzel az összeggel $C_H(p) = \sum_{k=0}^{N} \binom{N}{k} p^k (1-p)^{N-k}\, \mathbf{1}[|k/N - p| \leq \varepsilon_N]$, ahol $\varepsilon_N = \sqrt{\log(2/\alpha)/(2N)}$. A számítás véletlenmentes (nincs seed és ismétlésszám), a pontok a p rács kiválasztott értékei.



5.4. ábra: Wald- és Hoeffding-intervallum egzakt lefedettsége, N=20, 95%

Egzakt binomiális összeg, N=20, alpha=0,05; nem szimuláció.

A Wald-görbe nem sima „U” és nem is sima harang: a két szélen meredeken a névleges szint alá zuhan ($p = 0{,}01$ és 0,99 mellett 0,18205), a középső tartományban pedig a binomiális eloszlás diszkrétsége miatt fűrészfogszerűen ingadozik, többnyire 0,95 alatt; a teljes 99 pontos rácson a legnagyobb érték $p = 0{,}5$-nél 0,95861. A Hoeffding-görbe ezzel szemben minden ábrázolt pontban 0,99 felett marad: a teljes rácson a minimum 0,993707 ($p = 0{,}49$ és 0,51 mellett), a maximum $p = 0{,}99$-nél gyakorlatilag 1. A labor $R = 2000$ mintás becslése ezt Monte-Carlo-hibán belül visszaadja. A kirajzolt pontok csak mintavételezik a szakadásos függvényeket, ezért a két pont közötti egyenes nem a valódi lefutás.

5.3.8 A posterior-értelmezés hiánya

A leggyakoribb tévedés a konfidenciaintervallum és a posterior tartomány összekeverése. A konfidenciaintervallum definíciója az ismételt mintavételre hivatkozik. Egy prior és a Bayes-tétel nélkül az állítás

$$P(p \in [l, u] \mid x) \geq 0,95$$

nem értelmes, mert a p-nek nincs eloszlása. Ennek pontosítására:

- A gyakorisági eljárás azt állítja, hogy a $[l, u]$ eljárás hosszú távon 95%-ban fed.
- Tekintsünk egy olyan kísérletet, ahol a paraméter mindig egy rögzített, ismert p_0. Ekkor egy **realizált** intervallum fedési indikátora, $\mathbf{1}[p_0 \in [l, u]]$, valóban 0 vagy 1, de az **eljárás** lefedettsége, $P_{p_0}(p_0 \in C(X))$, az X véletlenségén át számolt valószínűség, és lehet például 0,95. A konfidenciaszint az utóbbira vonatkozik. Az aktuálisan kiszámított egyetlen intervallumra az „utólagos valószínűség” csak akkor 0,95, ha a prior éppen olyan, hogy az intervallum posterior tömege 0,95; ezt a konfidenciaintervallum nem garantálja.

Számított ellenpélda: az $N = 10$, $s = 4$ mintához tartozó Wald-intervallum [0,0964; 0,7036]. Ha a prior egyenletes a $[0, 1]$ intervallumon, azaz Beta$(1, 1)$, akkor a posterior Beta$(5, 7)$, és ennek tömege a Wald-intervallumon 0,9775. Ha a prior Beta$(10, 10)$, a posterior Beta$(14, 16)$, és ugyanennek az intervallumnak a posterior tömege 0,9964. Ugyanaz az intervallum tehát a priortól függően más posterior valószínűséggel rendelkezik; a 0,95 nem az intervallum belső tulajdonsága.

Gyakorlati haszna mégis van a konfidenciaintervallumnak: ha a becslési eljárást ismételten végrehajtanánk a felhasználás során, a lefedettségi arány garantálható. A modellkísérletekben a különböző seedekből származó eredmények szórása 5.4. Függő képek, seedek és resampling témája, és nem azonos a paraméterre vonatkozó konfidenciaintervallummal.

5.3.9 A bizonyítás és a szimuláció elhatárolása

Ebben az egységben három állítástípus szerepel:

1. **Levezetett analitikus állítások:** a torzítás, variancia és MSE dekompozíció. Feltételük az iid Bernoulli-minta és a véges momentumok. Ezeket a várható érték linearitásából igazoltuk.
2. **Levezetett tétel (Bernoulli-eset):** a Hoeffding-korlát Bernoulli-mintára. A fenti rövid bizonyítás csak a Bernoulli-esetre szól; az általános Hoeffding-tételt (független, korlátos, nem feltétlenül azonos eloszlású változókra) nem bizonyítjuk, csak hivatkozzuk.
3. **Empirikus szimuláció:** a lefedettségi arány. Ez mérés, nem bizonyíték. A szimuláció csak a választott p-rácsra és N-re érvényes, és a maga statisztikai hibájával terhelt.

Ez az elhatárolás teszi lehetővé, hogy a hallgató ne keverje a korlátot és a becsült lefedettséget. A Hoeffding-korlát garantált, a Wald-lefedettség szimulációja nem.

5.3.10 Gyakori tévedés ellenpéldával

A 95%-os konfidenciaintervallumot gyakran úgy értelmezik, hogy „95% valószínűséggel a paraméter az intervallumban van”. Ez nem állítható a gyakorisági definícióból. Az ellenpélda: ha $N = 1$ és $x_1 = 0$, a Wald-intervallum $[0, 0]$ metszet a $[0, 1]$-gyel. Ha a valódi paraméter $p = 0,4$, az intervallum nem tartalmazza p-t, és az eljárás lefedettsége $p = 0,4$-nél nem 0,95, hanem annál kisebb. Ez azonban *rossz intervallumeljárás* példája (a Wald-intervallum kis N-nél alulfed), nem a posterior-értelmezés általános cáfolata; a két kérdés külön áll. Gyakorlati használatra helyes lefedettségű eljárás kell: a Wilson-intervallum vagy az egzakt (Clopper–Pearson) intervallum, amelyek kis N-nél is a névleges szinthez közelebbi (Clopper–Pearson esetén legalább névleges) lefedettséget adnak [S03], [S04]. A Wilson-intervallum a z-statisztika invertálásából adódik:

$$\frac{\hat{p} + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}},$$

ahol $z = z_{1-\alpha/2}$. A Clopper–Pearson-intervallum [S04] a binomiális eloszlás pontos kvantiliseit használja: ha s a sikerek száma, akkor az alsó végpont a Beta($\alpha/2$; s, $N - s + 1$), a felső a Beta($1 - \alpha/2$; $s + 1$, $N - s$) eloszlás kvantilise, a $s = 0$, illetve $s = N$ szélső esetben a megfelelő határértékkel (0, illetve 1). A $N = 10$, $s = 4$ példában a Wilson-intervallum [0,1682; 0,6873], a Clopper–Pearson-intervallum [0,1216; 0,7376]; mindkettő szűkebb a Hoeffding-féle [0; 0,8295]-nél, és a névleges lefedettséget kis N-nél is jobban közelíti. A frekventista lefedettség és a posterior-értelmezés más objektumról szól.

5.3.11 Önellenőrzés

A hallgató a következő kérdésekkel tesztelheti a fogalmak elsajátítását:

1. Mi a különbség a $\hat{p}$ becslő és a $\hat{p}(x)$ becslés között?
2. Miért igaz az MSE dekompozíció torzítatlan becslőre?
3. Mit jelent a P_p alsó index a konfidenciaintervallum definíciójában?
4. Miért nem használható a Hoeffding-korlát folytonos, nem korlátos célra?
5. Miért nem szükséges matematikailag a `clip` a standardhiba-becslés előtt, és miért használjuk mégis védekező kódként?

Rövid válaszok.

1. A becslő a minta teréből a paramétertérbe képező függvény, tehát véletlen változó; a becslés ennek egy konkrét mintán felvett értéke, azaz szám.
2. Torzítatlan becslőre Bias $= 0$, ezért MSE $=$ Var $+$Bias$^2 =$ Var.
3. A P_p alsó index azt jelöli, hogy a valószínűséget a rögzített p paraméter mellett, az ismételt mintavétel véletlenségén át számoljuk; nem a paraméter eloszlására utal.
4. A Hoeffding-korlát a változók korlátosságát használja; nem korlátos célváltozóra a korlát nem érvényes, és a bizonyítás exponenciális momentumának kezelése sem működik. A Hoeffding-lemma $[a, b]$-korlátos változókra szól.
5. Matematikailag $\hat{p}(1 - \hat{p}) \geq 0$ minden $\hat{p} \in [0, 1]$-re, ezért a `clip` nem szükséges a helyes képlethez; védekező kódként a kerekítési hibák és a későbbi módosítások ellen marad.

Rövid visszatekintés 5.2. Várható érték, variancia és likelihood: a várható érték linearitása az, amely a torzítatlansági levezetést lehetővé teszi, a függetlenség pedig a kovarianciás tagok eltűnéséért felelős.

5.3.12 Feladatok

5.3.12.1 F1 – Fogalmi megkülönböztetés

Nehézség: 1. **Előfeltétel:** 5.1. Esemény, feltétel és véletlen változó és 5.2. Várható érték, variancia és likelihood fogalmainak ismerete. **Beadandó:** saját szavakkal írt magyarázat és egy példa.

Magyarázza el, mi a különbség a becslő és a becslés között, valamint a standard hiba és a becsült standard hiba között. Adjon egy $N = 3$ elemű Bernoulli-mintát, amelyre a becslő értéke $\hat{p} = 1/3$, és számolja ki a becsült standard hibát.

5.3.12.2 F2 – Levezetési feladat

Nehézség: 2. **Előfeltétel:** a variancia és a függetlenség alapjai. **Beadandó:** teljes levezetés és a végeredmény.

Legyen $Y_1, \dots, Y_M$ iid Bernoulli(p) minta, és $\tilde{p} = \frac{1}{M}\sum_{i=1}^{M} Y_i$. Vezesse le a $\tilde{p}$ várható értékét, varianciáját és standard hibáját. Ellenőrizze a képletet az $M = 1$ szélső esetben: mi a standard hiba?

5.3.12.3 F3 – Implementációs feladat

Nehézség: 2. **Előfeltétel:** a labor kódjának megértése. **Beadandó:** futtatható függvény a kért kimenettel.

Írjon Python-függvényt `wald_interval(x, alpha)` néven, amely egy `(N,)` alakú Bernoulli-mintából kiszámolja a Wald-intervallum $[L, U]$ végpontjait. A függvény a `scipy.stats.norm.ppf` használatával, `float64` pontossággal dolgozzon, és a végpontokat a $[0, 1]$ tartományra metssze. Az ellenőrzéshez használja az $x = (1, 0, 0, 1)^\top$, $\alpha = 0{,}05$ mintát. A várt intervallumot hat tizedesjegyre számolja ki.

5.3.12.4 F4 – Hibakeresési feladat

Nehézség: 2. **Előfeltétel:** a labor kódjának ismerete. **Beadandó:** a hiba oka, következménye, javítása és ellenőrzése.

A következő kódrészlet a Hoeffding-lefedettséget méri, de hibás:

```
eps = np.sqrt(np.log(2 / alpha) / (2 * N))
cover = np.mean(np.abs(x.mean(axis=1) - p) < eps)
```

Mi a hiba, és miért vezet a lefedettség alulbecsléséhez? Javítsa a kódot, és indokolja a választott összehasonlító operátort a definíció alapján.

5.3.12.5 F5 – Transzfer feladat

Nehézség: 3. **Előfeltétel:** a lefedettségi szimuláció elve. **Beadandó:** protokollterv, lehetséges baseline és érvényességi korlát.

Tervezzen szimulációt annak vizsgálatára, hogy a Hoeffding-intervallum lefedettsége hogyan viselkedik, ha a minták nem függetlenek, hanem az 1.3. Adatfelosztás és a megfigyelési egység alfejezetben tárgyalt, a függelék A. szakaszában leírt GlyphLab-struktúrához hasonlóan párokba rendeződnek: minden pár azonos rejtett paraméterű Bernoulli-változókból áll. Milyen protokollal különítené el a függetlenség megsértésének hatását? Milyen korlátot érvényesítene a konklúzióban?

5.3.13 Megoldások és indoklás

5.3.13.1 F1 megoldás

A becslő a minta teréből a paramétertérbe képező függvény, tehát $\hat{p}_N$ maga is véletlen változó. A becslés e függvény adott realizált mintára vett értéke: $\hat{p}_N(x) = s/N$. A standard hiba a becslő eloszlásának szórása, mely a valódi p-től függ; a becsült standard hiba a paraméter ismeretlen értékét a mintából becsült értékkel helyettesíti.

Példa: $x = (1, 0, 0)^\top$. Ekkor $s = 1$, $\hat{p} = 1/3$, és

$$\widehat{\mathrm{SE}} = \sqrt{\frac{(1/3)(2/3)}{3}} = \sqrt{\frac{2/9}{3}} = \sqrt{\frac{2}{27}} \approx 0{,}2722.$$

5.3.13.2 F2 megoldás

A várható érték linearitásával, mivel minden Y_i Bernoulli-eloszlású p-re,

$$\mathbb{E}[\tilde{p}] = \frac{1}{M}\sum_{i=1}^{M} \mathbb{E}[Y_i] = \frac{1}{M}\sum_{i=1}^{M} p = p.$$

Így $\tilde{p}$ torzítatlan. A varianciához a függetlenséget használjuk:

$$\mathrm{Var}(\tilde{p}) = \mathrm{Var}\left(\frac{1}{M}\sum_{i=1}^{M} Y_i\right) = \frac{1}{M^2}\sum_{i=1}^{M} \mathrm{Var}(Y_i) = \frac{1}{M^2}\sum_{i=1}^{M} p(1-p) = \frac{p(1-p)}{M}.$$

A standard hiba ennek négyzetgyöke:

$$\mathrm{SE}(\tilde{p}) = \sqrt{\frac{p(1-p)}{M}}.$$

Szélső eset: $M = 1$. Ekkor $\mathrm{SE}(\tilde{p}) = \sqrt{p(1-p)}$, ami éppen a Bernoulli-változó szórása. Ez konzisztens: egyetlen megfigyelésből a becslés maga a megfigyelés, így ingadozása megegyezik az eredeti változóéval.

5.3.13.3 F3 megoldás

```python
"""Wald-intervallum Bernoulli-mintára."""
from __future__ import annotations

import numpy as np
from scipy.stats import norm


def wald_interval(x: np.ndarray, alpha: float) -> tuple[float, float]:
    """A Wald-féle konfidenciaintervallum [L, U] végpontjai.

    x: (N,) alakú 0/1 értékű minta; a függvény float64-re konvertálja.
    alpha: a hiba valószínűsége, pl. 0.05.
    """
    x = np.asarray(x, dtype=np.float64)
    N = x.shape[0]
```

```
    p_hat = x.mean()
    se = np.sqrt(p_hat * (1 - p_hat) / N)
    z = norm.ppf(1 - alpha / 2)
    L = max(0.0, p_hat - z * se)
    U = min(1.0, p_hat + z * se)
    return L, U


x = np.array([1, 0, 0, 1], dtype=np.float64)
alpha = 0.05
L, U = wald_interval(x, alpha)
print(f"L={L:.6f}, U={U:.6f}")
```

Számolás: $\hat{p} = 2/4 = 0{,}5$, $\widehat{\mathrm{SE}} = \sqrt{0{,}25/4} = 0{,}25$, $z \approx 1{,}959964$. Az intervallum végpontjai:

$$L = 0{,}5 - 1{,}959964 \cdot 0{,}25 = 0{,}010009,$$

$$U = 0{,}5 + 1{,}959964 \cdot 0{,}25 = 0{,}989991.$$

Az elvárt kimenet `L=0.010009, U=0.989991`, amit a metszet nem módosít, hiszen mindkét végpont a $[0, 1]$ belsejében van.

5.3.13.4 F4 megoldás

A hiba a szigorú `<` operátor. A lefedettségi definíció megengedő egyenlőtlenséget ír elő: $L \leq p \leq U$. Ezért a

$$|\hat{p} - p| \leq \varepsilon$$

eseményt kell vizsgálni. A Bernoulli-mintaátlag diszkrét: $\hat{p}$ csak a $0, 1/N, \ldots, 1$ értékeket veheti fel. Ezért az egyenlőség eseményének **lehet pozitív valószínűsége**, amikor $|\hat{p} - p| = \varepsilon$ teljesül valamely lehetséges mintaátlagra. A szigorú `<` ilyenkor ténylegesen alulbecsülheti a zárt intervallum lefedettségét. A két változat pontosan akkor ad azonos lefedettséget, ha $P(|\hat{p} - p| = \varepsilon) = 0$; diszkrét $\hat{p}$ mellett ez nem minden p-re teljesül, ezért kell a megengedő $\leq$.

Javított kód:

```
cover = np.mean(np.abs(x.mean(axis=1) - p) <= eps)
```

Ellenőrzés: a lefedettségi arány így konzisztens a Hoeffding-korlát állításával, miszerint a hiba valószínűsége legfeljebb α. Az elméleti lefedettség a Hoeffding-feltételek mellett legalább $1 - \alpha$; a véges R szimulációban mért lefedettség véletlen ingadozása miatt lehet a névleges szint alatt is.

5.3.13.5 F5 megoldás

A transzferfeladat protokollja:

1. **Kérdés:** csökken-e a Hoeffding-intervallum lefedettsége, ha a függetlenség megsértése páros struktúrát követ?
2. **Adatgeneráló modell:** minden `scene_id`-hez húzunk egy közös p_s rejtett értéket; a két nézet X_{s1}, X_{s2} feltételesen független Bernoulli(p_s) változó. A függetlenség megsértését az jelenti, hogy a p_s scene-enként változik, de a nézeteken belül közös.

3. **Kontroll (baseline):** ugyanakkora N mellett független Bernoulli-minta, ahol minden X_i azonos p_0-ból jön.
4. **Változó tényező:** a p_s szóródása a scene-en belül, például két szint: kis variancia, nagy variancia.
5. **Mérés:** minden beállításban 1000 ismételt mintából lefedettségi arány.
6. **Kiinduló hipotézis:** a klaszterezett minta esetén a Hoeffding-lefedettség elmaradhat a névlegestől, mert a hatékony mintaméret kisebb, mint N.
7. **Érvényességi korlát:** a konklúzió csak a szimulált modellre és a választott szórásszintekre vonatkozik; nem általános bizonyíték a klaszterezett adatokra.

A hallgató a feladatmegoldásban a fenti szerkezetet töltse ki a konkrét szimulációs lépésekkel. A megoldásminta a terv szerkezetét és az alternatívák indoklását értékeli, nem egy előre adott numerikus eredményt.

5.3.14 Rövid összefoglaló

A becslő a minta függvénye, így maga is véletlen változó; a becslés a konkrét mintán kapott érték. Az iid Bernoulli-minta MLE-je torzítatlan, varianciája $p(1-p)/N$, a standard hiba ennek négyzetgyöke. A négyzetes hiba varianciára és torzításnégyzetre bomlik. A konfidenciaintervallum lefedettsége ismételt mintavételi gyakoriság: a Hoeffding-intervallum garantáltan legalább $1-\alpha$ szintű, míg a Wald-intervallum csak aszimptotikus közelítés. A labor sok szintetikus mintából méri a lefedettséget; a kapott arány szimulációs mérés, nem analitikus garancia. A 95%-os intervallum nem azonos a paraméter 95%-os posterior valószínűségével, mert utóbbihoz prior szükséges.

5.3.15 Továbblépés

A lefedettségi szimuláció a bizonytalanság egy fajtáját méri. A gyakorlatban a különböző seeddel betanított modellek szóródása is mintavételi bizonytalanság, ám az adatok függhetnek egymástól: ugyanazon jelenet két nézete nem független. Az 5.4. Függő képek, seedek és resampling egység ezt a függő struktúrát és a bootstrap alapjait vezeti be.

A Wald-intervallum kis mintás lefedettségét a fejezet nem külső forrásból veszi át, hanem saját számítással mutatja meg: a diagram a binomiális eloszlásból egzaktul számolt értékeket ábrázolja ($N = 20$, $\alpha = 0{,}05$), a labor szimulációja ($R = 2000$, seed 20261020) pedig Monte-Carlo-hibán belül ugyanezt adja. Más N esetén a görbe részletei megváltoznak, a szélek alá-lefedése és a diszkrétség okozta ingadozás nem.

5.3.16 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B04] Boucheron, S., Lugosi, G., Massart, P. (2013). *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press.
- [S03] E. B. Wilson (1927). *Probable inference, the law of succession, and statistical inference*. Journal of the American Statistical Association.
- [S04] C. J. Clopper és E. S. Pearson (1934). *The use of confidence or fiducial limits illustrated in the case of the binomial*. Biometrika.

5.4 Függő képek, seedek és resampling

5.4.1 Cél, előismeret és induló kérdés

Az előző egységben, az 5.3. Mintavételi bizonytalanság és intervallum alfejezetben a mintavételi bizonytalanságot független, azonos eloszlású (iid) megfigyelésekre vizsgáltuk. A gyakorlati képi adatok azonban gyakran csoportokba rendeződnek: a függelék A. szakaszában leírt GlyphLab (procedurálisan generált, háromosztályos grayscale adatkészlet 180 jelenettel, jelenetenként két zajos nézettel) egy jelenetének két nézete (view) ugyanazt a geometriát tartalmazza, csak a zaj különbözik. Ha ezeket független képként kezeljük, a becslés bizonytalanságát alulbecsülhetjük. Az egység célja a statisztikai elemzés egységének, a páros összehasonlításnak és a klaszterbootstrapnek a bevezetése, valamint az adatminta- és a tanítási seed hatásának szétválasztása.

Előismeret: az 5.3. Mintavételi bizonytalanság és intervallum fogalmai (becslő, standard hiba, lefedettség, konfidenciaintervallum), valamint a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetből a determinisztikus reprodukálhatóság és az 1.4. Tanulási diagnózis és kísérleti napló alfejezetből a seed, illetve futásazonosító megkülönböztetése. Nem tételezünk fel haladó statisztikai elméletet.

Induló kérdés: a GlyphLab tanítóhalmaza 240 képet tartalmaz, de ezek 120 jelenetből származnak, jelenetenként két nézettel. Hány független megfigyelésünk van: 240 vagy 120? A válasz a kérdés feltevésétől függ: a rekordok (megfigyelési egységek) száma 240, a független statisztikai egységeké 120, mert a két nézet ugyanahhoz a jelenethez tartozik, és a jelenetek közötti szóródás közös. A bootstrap és a konfidenciaintervallum szempontjából a 120 a mérvadó; ez az egész egység központi problémája.

5.4.2 Definíció és intuíció: statisztikai egység, klaszter, páros megfigyelés

Definíció (megfigyelési egység és klaszter). Legyen adott egy véges sokaság, amelyből mintát veszünk. A *megfigyelési egység* az a legkisebb rekord, amelyen a célváltozót mérjük — itt egy kép. A *klaszter* (csoport, angolul *cluster*) megfigyelések olyan halmaza, amelyen belül a függetlenség nem indokolt; a mintavételi terv a klaszter egészét választja ki vagy hagyja el. A GlyphLab-ban a klaszter a `scene_id`: egy jelenet két nézete közös véletlen geometriai konfigurációból származik, ezért a képszintű függetlenség sérül. A kettő közötti különbség nem pusztán formális: ha a jeleneten belüli képek korreláltak, akkor N kép kevesebb információt hordoz, mint N független megfigyelés. A *klaszterbootstrap* (cluster bootstrap) ezt a szerkezetet tartja meg: a klasztereket húzza visszatevéssel, és minden kiválasztott klaszter összes megfigyelését beveszi a replikátumba.

Definíció (páros megfigyelés). Két mérés akkor *páros*, ha ugyanazon megfigyelési egység (itt: ugyanaz a kép) két feltételhez tartozó változata, és a két feltétel közötti különbséget ezen az egységen belül értelmezzük. A GlyphLab zárótesztjében (a split utolsó, csak kiértékelésre szánt részében) a clean (tiszta) és a corrupted (korrupt) változat ugyanazt a 60 tesztképet módosítja; a két feltétel képenként párosítható, ezért a pontosságkülönbség páros összehasonlítással elemezhető. A párosítás nem tévesztendő össze a klaszterezéssel: a különbséget képenként számoljuk, de a bootstrap mintavételi egysége továbbra is a jelenet, mert a 60 tesztkép 30 jelenetből származik. A páros elemzés előnye, hogy a jelenetek közötti szórást kiküszöbölheti a különbségből — hasonlóan ahhoz, ahogy az 5.3. Mintavételi bizonytalanság és intervallum alfejezetben a becslő varianciája a megfigyelések varianciájából származott.

Jelölés. Legyen M a klaszterek száma, r_i az i-edik klaszter elemszáma, $\bar{r} = \frac{1}{M}\sum_i r_i$ az átlagos klaszterméret, és $N = \sum_i r_i$ a teljes képszám. A GlyphLab tanítóhalmazában $M = 120$, $r_i = 2$

minden i-re, így $\bar{r} = 2$ és $N = 240$. A lemma kiegyensúlyozott esetben dolgozik, ahol minden klaszter mérete ugyanaz az r, és ekkor $r = \bar{r}$. Egy becslőt $\widehat{\theta}$-val jelölünk; a bootstrap replikátumait $\widehat{\theta}_1^*, \dots, \widehat{\theta}_B^*$, ahol B a replikátumok száma.

Intuíció. Ha egy jelenet két nézete szinte azonos, a második nézet alig ad új információt. A képenkénti elemzés úgy tesz, mintha minden kép független lenne; a jelenetenkénti elemzés a jelenetet tekinti a véletlen mintavétel egységének. A kettő közötti választást nem a kényelem, hanem az adatgeneráló folyamat és a kutatási kérdés határozza meg.

5.4.3 Levezetési kötelezettség: kétcsoportos ellenpélda a képszám és a független egység eltérésére

A következő állítás mutatja meg, hogy a képszám növelése nem feltétlenül csökkenti a becslés varianciáját, ha a klaszterszerkezetet figyelmen kívül hagyjuk.

Lemma (klaszterkorrelált átlag varianciája). Legyen M klaszter, mindegyikben pontosan r megfigyelés. Tegyük fel, hogy az X_{ij} érték ($i = 1, \dots, M$, $j = 1, \dots, r$) felbontása

$$X_{ij} = \mu + U_i + E_{ij},$$

ahol μ konstans, a $\{U_i\}_{i=1}^{M}$ és a $\{E_{ij}\}_{i,j}$ változók **kölcsönösen függetlenek** (a két család egymástól is), $\mathbb{E}[U_i] = \mathbb{E}[E_{ij}] = 0$, $\operatorname{Var}(U_i) = \tau^2 \geq 0$, $\operatorname{Var}(E_{ij}) = \sigma^2 > 0$. Ekkor a teljes mintaátlag

$$\bar{X} = \frac{1}{Mr} \sum_{i=1}^{M} \sum_{j=1}^{r} X_{ij}$$

varianciája

$$\operatorname{Var}(\bar{X}) = \frac{\sigma^2}{Mr} + \frac{\tau^2}{M}.$$

Bizonyítás. A kovarianciák a felbontásból közvetlenül adódnak. Ha $i = i'$ és $j = j'$, akkor $\operatorname{Cov}(X_{ij}, X_{i'j'}) = \operatorname{Var}(U_i + E_{ij}) = \tau^2 + \sigma^2$. Ha $i = i'$, de $j \neq j'$, akkor a közös U_i miatt a kovariancia τ^2. Ha $i \neq i'$, akkor a függetlenség miatt a kovariancia 0. A variancia a kettős összeg kovarianciáinak összege, osztva $(Mr)^2$-tel:

$$\operatorname{Var}(\bar{X}) = \frac{1}{(Mr)^2} \sum_{i,i',j,j'} \operatorname{Cov}(X_{ij}, X_{i'j'}).$$

Az $i = i'$ esetben M klaszter van, klaszterenként r azonos pár és $r(r-1)$ különböző pár. Így

$$\operatorname{Var}(\bar{X}) = \frac{1}{(Mr)^2} \Big[Mr(\tau^2 + \sigma^2) + Mr(r-1)\tau^2 \Big] = \frac{Mr\sigma^2 + Mr^2\tau^2}{(Mr)^2} = \frac{\sigma^2}{Mr} + \frac{\tau^2}{M}.$$

Ezzel az állítást beláttuk.

Kétcsoportos ellenpélda. Legyen $M = 2$ jelenet, jelenetenként $r = 10$ kép, tehát $N = 20$ kép. Válasszunk $\tau^2 = 25$ és $\sigma^2 = 1$ értékeket, amelyek azt fejezik ki, hogy a jelenetek közötti szóródás dominál. A lemma szerint a mintaátlag tényleges varianciája

$$\frac{1}{20} + \frac{25}{2} = 0.05 + 12.5 = 12.55.$$

Ha valaki tévesen 20 független képet feltételez, akkor az iid képlet alapján — amely az 5.3. Mintavételi bizonytalanság és intervallum alfejezetből ismert $\mathrm{Var}(\bar{X}) = \mathrm{Var}(X)/N$ alak — azt becsülné, hogy

$$\frac{\tau^2 + \sigma^2}{20} = \frac{26}{20} = 1.3,$$

ami csaknem tízszer kisebb. Ezt a design effect (tervezési hatás) és az effektív mintaméret számszerűsíti: $\mathrm{deff} = \mathrm{Var}(\bar{X})/\mathrm{Var}_{\mathrm{iid}}(\bar{X}) = 12{,}55/1{,}3 \approx 9{,}65$, amiből $N_{\mathrm{eff}} = N/\mathrm{deff} \approx 20/9{,}65 \approx 2{,}07$. A képszám tehát valóban 20, de az effektív mintaméret a variancia szempontjából alig haladja meg a 2-t: a τ^2/M tag nem csökken r növelésével. A levezetés saját; a klaszterkorrelált varianciabontás az [B01] 6. fejezetében tárgyalt variancia- és kovarianciafogalmakra épül, de a konkrét lemma itteni kidolgozás.

5.4.4 Kézi kis példa

Vegyünk két jelenetet, mindegyikből két képet. A mért értékek legyenek az alábbiak.

Jelenet	Kép 1	Kép 2
A	10	12
B	20	22

A teljes mintaátlag $\bar{x} = (10 + 12 + 20 + 22)/4 = 16$. A jelenetenkénti bootstrap a két jelenet közül húz visszatevéssel kettőt, és minden kiválasztott jelenethez mindkét képét hozzáveszi. A lehetséges húzások:

- (A, A): a minta (10, 12, 10, 12), átlag 11;
- (A, B) vagy (B, A): a minta (10, 12, 20, 22), átlag 16;
- (B, B): a minta (20, 22, 20, 22), átlag 21.

A jelenetalapú replikátumok eloszlása: 11 (valószínűsége 1/4), 16 (valószínűsége 1/2, két húzásból) és 21 (1/4). Ebből a bootstrap-átlag várható értéke $(11 + 16 + 16 + 21)/4 = 16$, ami megegyezik a teljes mintaátlaggal, és a szórásnégyzete $\frac{1}{4}[(11-16)^2 + 0 + 0 + (21-16)^2] = 12.5$.

A képenkénti bootstrap ezzel szemben a négy kép közül húz négyet visszatevéssel; a $4^4 = 256$ egyformán valószínű rendezett minta teljes felsorolható. Ennek az eloszlásnak a **tartóhalmaza** (support), azaz a pozitív valószínűséggel előforduló minták halmaza bővebb: olyan minták is keletkeznek, mint (10, 10, 10, 10), átlag 10, vagy (22, 22, 22, 22), átlag 22. A tartóhalmaz szélessége azonban nem a szóródás mértéke: ezek az extrém minták egyenként csak 1/256 valószínűségűek. A szóródást a variancia méri. A visszatevéses empirikus bootstrap átlagának pontos varianciája a populációs (1/N-es) tapasztalati variancia osztva N-nel: a négy érték négyzetes eltéréseinek összege a 16-os átlag körül $36 + 16 + 16 + 36 = 104$, így

$$\mathrm{Var}^*(\bar{x}^*) = \frac{104/4}{4} = 6{,}5,$$

ami a 256 minta felsorolásával pontosan ellenőrizhető. Ezektől a bootstrap-varianciáktól külön mennyiség a klasszikus, korrigált mintavariancián alapuló standard hiba, amely az $s^2 = 104/3 \approx$

34,67 értéket használja: $\sqrt{s^2/N} = \sqrt{(104/3)/4} \approx 2{,}9439$. Ez a képenkénti bootstrap szórásánál ($\sqrt{6{,}5} \approx 2{,}55$) valamivel nagyobb, a jelenetalapúénál ($\sqrt{12{,}5} \approx 3{,}54$) jóval kisebb, mert a képszintű függetlenséget feltételezi. Az összevetés tehát a varianciákon: a képenkénti bootstrap varianciája 6,5, a jelenetenkéntié 12,5, vagyis a képenkénti bootstrap **szűkebb** bizonytalanságot mutat, mert a két nézetet független megfigyelésnek kezeli, és figyelmen kívül hagyja, hogy a jelenetek közötti eltérés (A: 11, B: 21) adja a szóródás nagyobb részét. A jelenetalapú megközelítés a klaszterek közti szórást tükrözi; ez a kis példa azt mutatja, hogy a két bootstrap más bizonytalanságot mér, és a jelenetalapú a helyes, ha a jelenet a független mintavételi egység.

5.4.5 Labor: képenkénti és jelenetenkénti bootstrap a GlyphLab két nézetén

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor a GlyphLab tanítóhalmazán hasonlítja össze a két bootstrapstratégiát. A becslő $\hat{\theta}$ az átlagos pixelintenzitás a tanítóképeken; ez egyszerű, és jól mutatja a klaszterhatást, mert ugyanazon jelenet két nézete erősen korrelált. A feladat nem a modellpontosság becslése, hanem a bootstrap egységének szerepe. A kódot CPU-n, NumPy-val futtatjuk; a képek `float32` típusúak, a `boot_img` és `boot_scene` tömb a $\hat{\theta}^*_1, \ldots, \hat{\theta}^*_B$ replikátumokat gyűjti `float64`-ben.

```
# n020_lab.py - futtatás: python n020_lab.py
# Munkakönyvtár: a projekt gyökere, ahol a lab/data/ elérhető.
import numpy as np
import pandas as pd
from pathlib import Path

data_dir = Path("lab/data")
rng = np.random.default_rng(20261020)  # seed a replikátumokhoz
images = np.load(data_dir / "images.npy")          # (360,1,24,24), float32
# A metadata.csv oszlopai: index (globális képazonosító az images.npy-ban),
# scene_id (klaszter), view_id (nézet), label (osztály), split (train/val/test),
# majd a befoglaló doboz koordinátái. A labor az index, scene_id, label és
# split oszlopokat használja.
meta = pd.read_csv(data_dir / "metadata.csv")

train = meta[meta["split"] == "train"].reset_index(drop=True)
# train: 240 kép, 120 jelenet, osztályonként 40 jelenet
train_idx = train["index"].to_numpy()              # (240,)
train_images = images[train_idx]                   # (240,1,24,24)

def estimator(imgs: np.ndarray) -> float:
    """Átlagos intenzitás; float64-re konvertálva a stabil összegzéshez."""
    return float(np.asarray(imgs, dtype=np.float64).mean())

B = 2000
# 1) Képenkénti bootstrap: 240 képből 240 húzás visszatevéssel.
boot_img = np.empty(B, dtype=np.float64)
for b in range(B):
    idx = rng.integers(0, len(train_images), size=len(train_images))
    boot_img[b] = estimator(train_images[idx])

# 2) Jelenetenkénti, osztályonként rétegzett bootstrap.
# A GlyphLab generálása osztályonként rögzített jelenetszámot használ;
# a bootstrap ezt a rétegzést megőrzi: osztályonként 40 jelenetet húz.
scenes_by_label = {lab: np.unique(train.loc[train["label"] == lab, "scene_id"])
                   for lab in np.unique(train["label"])}
scene_to_idx = train.groupby("scene_id")["index"].apply(np.array).to_dict()

boot_scene = np.empty(B, dtype=np.float64)
```

```
for b in range(B):
    chosen = []
    for lab, scene_ids in scenes_by_label.items():
        drawn = rng.choice(scene_ids, size=len(scene_ids), replace=True)
        for sid in drawn:
            chosen.extend(scene_to_idx[sid])
    sel = np.array(chosen, dtype=np.int64)
    assert len(sel) == len(train_images), "a rétegzett bootstrap mérete sérült"
    # sel globális indexeket tartalmaz, ezért az images tömbből kell venni
    boot_scene[b] = estimator(images[sel])

ci_img = np.quantile(boot_img, [0.025, 0.975])
ci_scene = np.quantile(boot_scene, [0.025, 0.975])
print(f"train képek száma: {len(train_images)}, jelenetek száma: {train['scene_id'].nunique()}")
print(f"képenkénti bootstrap: átlag={boot_img.mean():.6f}, sd={boot_img.std(ddof=1):.6f}, "
      f"95% percentilis CI=({ci_img[0]:.6f}, {ci_img[1]:.6f})")
print(f"jelenetenkénti bootstrap: átlag={boot_scene.mean():.6f},
↪  sd={boot_scene.std(ddof=1):.6f}, "
      f"95% percentilis CI=({ci_scene[0]:.6f}, {ci_scene[1]:.6f})")
ratio = boot_scene.std(ddof=1) / boot_img.std(ddof=1)
sample_mean = estimator(images[train_idx])
assert abs(boot_img.mean() - sample_mean) < 1e-3
assert abs(boot_scene.mean() - sample_mean) < 1e-3
assert ratio > 1.0, "a jelenetalapu szoras nem nagyobb a kepenkinel"
print(f"szórások aránya (jelenet/kép): {ratio:.3f}")
print("ellenorzes: a bootstrap-atlagok a mintaatlag kozeleben, az arany > 1")
```

Ellenőrzés és értelmezés. A kód négy beépített kontrollt végez: a rétegzett minta mérete minden replikátumban pontosan 240, továbbá mindkét bootstrapátlag a teljes tanítóhalmaz átlagához közel van, és a jelenetalapú szórás nagyobb a képenkéntinél — a bootstrap várható értéke közelíti a mintabeli becslést, de ez önmagában nem garancia. A mért kimenet (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, CPU, OMP_NUM_THREADS=1, seed 20261020):

```
train képek száma: 240, jelenetek száma: 120
képenkénti bootstrap: átlag=0.135421, sd=0.000369, 95% percentilis CI=(0.134714, 0.136133)
jelenetenkénti bootstrap: átlag=0.135427, sd=0.000509, 95% percentilis CI=(0.134439, 0.136410)
szórások aránya (jelenet/kép): 1.381
ellenorzes: a bootstrap-atlagok a mintaatlag kozeleben, az arany > 1
```

A mért mintázat: a jelenetenkénti szórás nagyobb, mint a képenkénti, mert a jeleneten belüli korreláció csökkenti az effektív mintaméretet. Ezt a kód csak irányában ellenőrzi (`ratio > 1.0`); az arány konkrét nagysága adatspecifikus és nincs ellenőrizve. A GlyphLab két nézete közös geometriájú, de külön zajú, ezért az arány 1 fölötti, a mértéke a jeleneteken belüli és közötti szórástól függ. A percentilis CI (a bootstrap-replikátumok 2,5%-os és 97,5%-os kvantiliséből képzett intervallum) a bootstrap-szakirodalom [B05] szerinti percentilis-intervallum; értelmezése az [S02]-ben az egyváltozós átlagra tárgyalt ismételt mintavételi intervalluméval rokon, azzal a különbséggel, hogy a bootstrap esetén a lefedettség csak közelítő, és a klaszterek számának növelésével javul.

5.4.6 Feladatok

F1 (fogalmi, nehézség 1). Fogalmazza meg saját szavaival, mi a különbség a megfigyelési egység és a statisztikai mintavételi egység között. Adjon egy olyan konkrét ellenpéldát, amelyben a képszám nagy, de a független egységek száma kicsi.

F2 (levezetési, nehézség 2). A lemmában $M = 3$ klaszter, klaszterenként $r = 4$ megfigyelés, $\tau^2 = 9$, $\sigma^2 = 1$ szerepel. Számítsa ki a mintaátlag tényleges varianciáját, a naiv iid képletből adódó értéket, és magyarázza el a különbséget. Minden lépést írjon le a kovarianciaösszegzésig.

F3 (implementációs, nehézség 2). Írjon Python-függvényt `cluster_bootstrap(data, clusters, B, rng)` néven, ahol `data` egy `(N,)` alakú tömb, `clusters` egy `(N,)` alakú, klaszter-azonosítókat tartalmazó tömb, `B` a replikátumok száma, és `rng` egy `numpy.random.Generator`. A függvény minden replikátumban klasztereket húz visszatevéssel, az összes hozzájuk tartozó elemet megtartja, és a becslő (átlag) replikátumait adja vissza `(B,)` alakú `float64` tömbben. Ellenőrizze egy háromklaszteres, ismert átlagú adaton, hogy a replikátumok átlaga a teljes mintaátlag közelében van (tolerancia 0,02, a Monte-Carlo-hibához igazítva).

F4 (hibakeresési, nehézség 2). Tekintse a következő kódrészletet, amely a GlyphLab train képeire bootstrapel:

```
idx_all = np.arange(len(train_images))
for b in range(B):
    sample = rng.permutation(idx_all)[:len(idx_all)]  # visszatevés nélkül!
    boot[b] = estimator(train_images[sample])
```

Azonosítsa a két hibát: miért nem jelenetalapú ez a mintavétel, és miért hibás a visszatevés nélküli permutáció? Mi a következményük a konfidenciaintervallumra, és hogyan javítaná a kódot?

F5 (kísérleti transzfer, nehézség 3). Tervezzen kísérletet a GlyphLab clean és corrupted tesztkészletének páros összehasonlítására. Adja meg: a kutatási kérdést, a nullhipotézist, a metrikát, a párosítás egységét, a bootstrap protokollt (jelenetalapú, rétegzett, a három osztály mindegyikéből 10 jelenet), a szükséges modellválasztási korlátot (a corrupted teszt nem használható modellválasztásra), és egy lehetséges következtetési korlátot. Nem kell futtatnia, de a tervnek végrehajthatónak kell lennie az 1.3. Adatfelosztás és a megfigyelési egység alfejezetben rögzített split és a GlyphLab séma szerint.

5.4.7 Megoldások és indoklás

F1 megoldás. A *megfigyelési egység* a mérés legkisebb rekordja: itt egy kép. A *statisztikai mintavételi egység* az a szint, amelyen a függetlenség feltételezhető, és amelyet a bootstrapben visszatevéssel húzunk: itt a jelenet. A kettő akkor tér el, ha több megfigyelés származik ugyanabból a klaszterből. Ellenpélda: 1000 képet készítünk egyetlen jelenetből kismértékű zajvariációval. A képszám 1000 nagy, de a független egységek száma 1; az átlagos intenzitás konfidenciaintervalluma nem szűkül a képszám növelésével arányosan, mert a jelenethatás közös minden képen. Megfigyelési egység és mintavételi egység tehát nem felcserélhető fogalom.

F2 megoldás. A lemma képlete szerint $M = 3$, $r = 4$, így

$$\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{Mr} + \frac{\tau^2}{M} = \frac{1}{12} + \frac{9}{3} = 0.0833 + 3 = 3.0833.$$

A naiv iid képlet az összes $N = 12$ megfigyelést függetlennek tekinti, és a per-megfigyelés varianciát $\tau^2 + \sigma^2 = 10$-nek veszi:

$$\mathrm{Var}_{\mathrm{naiv}}(\bar{X}) = \frac{10}{12} = 0.8333.$$

A valódi variancia tehát $3.0833/0.8333 \approx 3.7$-szerese a naiv értéknek. A levezetés a lemma bizonyításának mintájára: az $i = i'$ esetben $Mr = 12$ azonos pár és $Mr(r-1) = 3 \cdot 4 \cdot 3 = 36$ különböző, azonos klaszterbe tartozó pár van; a keresztklaszteres párok kovarianciája nulla. Az összeg $12(\tau^2 + \sigma^2) + 36\tau^2 = 12 \cdot 10 + 36 \cdot 9 = 444$; osztva $(Mr)^2 = 144$-gyel: $444/144 = 3.0833$. Ez az ellenőrzés megerősíti a képletet.

F3 megoldás. A függvény:

```
import numpy as np

def cluster_bootstrap(data, clusters, B, rng):
    data = np.asarray(data, dtype=np.float64)
    clusters = np.asarray(clusters)
    unique_clusters = np.unique(clusters)
    idx_by_cluster = {c: np.flatnonzero(clusters == c)
                      for c in unique_clusters}
    result = np.empty(B, dtype=np.float64)
    for b in range(B):
        drawn = rng.choice(unique_clusters, size=len(unique_clusters),
                           replace=True)
        idx = np.concatenate([idx_by_cluster[c] for c in drawn])
        result[b] = data[idx].mean()
    return result
```

Ellenőrzés: legyen három klaszter, mindegyik két elemmel, értékek rendre 1,1, 2,2, 3,3. A teljes átlag 2. `rng = np.random.default_rng(0)` mellett $B = 5000$ replikátumot generálunk; a klaszterek húzása visszatevéses, és a klaszterenkénti indexlista összefűzésével a replikátumok mindig hat eleműek, az átlaguk pedig a három klaszterátlag (1, 2, 3) újramintavételezett átlaga. Az elvárt átlag pontosan 2. A replikátumok szórása a klaszterátlagok szórásából számolva mintegy 0,47, ezért a B replikátum átlagának standard hibája $0{,}47/\sqrt{5000} \approx 0{,}0066$; a 0,02-os tolerancia mintegy három standard hiba, míg a 10^{-3}-os tolerancia a Monte-Carlo-ingadozás miatt a futtatások többségében tévesen bukna. Szélsőeset: ha minden klaszter mérete 1, a klaszterbootstrap a szokásos iid bootstrapre redukálódik, mert minden húzott klaszterhez egyetlen elem tartozik; az ellenőrző teszt ezt is lefedi.

F4 megoldás. Az első hiba: a kód a képek indexeit keveri, nem a jeleneteket, ezért az azonos `scene_id`-hez tartozó két nézet szétválasztható, és a replikátumban a klaszterstruktúra megszűnik. A második hiba: a `rng.permutation` visszatevés nélküli permutáció, így minden replikátum pontosan az eredeti képek egy átrendezése; a bootstrap becslő ingadozása eltűnik, mert az átlag permutációinvariáns — minden replikátum ugyanazt az átlagértéket adja (a replikátumok értékei azonosak), a CI nulla szélességű. A következmény: a képenkénti és visszatevés nélküli mintavétel együtt drasztikusan alulbecsli a bizonytalanságot. Javítás: a replikátumon belül először a jelenet-azonosítókból húzunk visszatevéssel, majd a kiválasztott jelenetek összes képét válogatjuk be; a jelenetek húzása `rng.choice(scene_ids, size=len(scene_ids), replace=True)` legyen. A javított változatot ellenőrizzük: egy replikátumban sem szerepelhet egy jelenetnek csak az egyik nézete, és a replikátumok szórása nullától különböző.

F5 megoldás. Kérdés: mennyivel romlik a modell pontossága a korrupt tesztváltozaton a tiszta teszthez képest? Nullhipotézis: a modell várható pontossága a két feltételen azonos; a különbség 0. Metrika: páronkénti pontosságkülönbség, azaz képenként $d_j = \mathbf{1}[\hat{y}_j^{\text{clean}} = y_j] - \mathbf{1}[\hat{y}_j^{\text{corrupt}} = y_j]$; a becslő a 60 páron vett átlag. A párosítás egysége a kép, de a bootstrap egysége a jelenet: a három osztályból húzunk visszatevéssel 30 tesztjelenetet, osztályonként rétegzetten (osztályonként 10 jelenet), és minden kiválasztott jelenet mindkét view-ját, clean és corrupt változatát együtt tartjuk. A modellt csak a train/val adaton szabad kiválasztani; a corrupted teszt kizárólag a végső, befagyasztott modell kiértékelésére szolgál. A 95%-os percentilis CI a jelenetalapú replikátumokból számítható. Következtetési korlát: a GlyphLab szintetikus és kicsi, ezért a robusztussági következtetés nem vihető át valós képi adatra; továbbá a bootstrap a modellválasztási bizonytalanságot nem tartalmazza, mert ugyanazt a betanított modellt értékeljük újra.

5.4.8 Tipikus tévedés és ellenpélda

Tévedés: „Ha több tanítási seeddel futtatom ugyanazt a modellt ugyanazon az adaton, akkor a kapott pontosságok szórása a valódi adatfelvétel bizonytalanságát méri.” Ez hibás. A seed az inicializálást, a minibatch (részhalmaz) sorrendjét és az esetleges dropout (véletlen elemkihagyás) maszkokat változtatja meg; az adateloszlás és a minta rögzített marad. A több seedből származó szórás a *tanítási algoritmus sztochasztikusságát* méri. Az adatminta bizonytalanságát csak új, független minta vagy a jelenetalapú bootstrap közelítheti, és ez utóbbi is feltételezi, hogy a jelenetek reprezentatív mintái a sokaságnak. A két komponens össze is adódhat. A teljes variancia törvénye (law of total variance) szerint

$$\mathrm{Var}(T) = \mathbb{E}_D\left[\mathrm{Var}_S(T \mid D)\right] + \mathrm{Var}_D\left(\mathbb{E}_S[T \mid D]\right),$$

ahol T a mért pontosság, D az adatminta és S a seed. A levezetés a feltételes várható érték tulajdonságain alapul: a $T - \mathbb{E}T = (T - \mathbb{E}_S[T \mid D]) + (\mathbb{E}_S[T \mid D] - \mathbb{E}T)$ felbontásban a vegyes tag feltételes várható értéke nulla, ezért a négyzet várható értéke a két tag négyzetének várható értéke; az első tag átlaga $\mathbb{E}_D[\mathrm{Var}_S(T \mid D)]$, a másodiké $\mathrm{Var}_D(\mathbb{E}_S[T \mid D])$. Az első tag a seedből, a második az adatmintából származó variancia; a két feltétel nélküli szórás egyszerű összege nem ez. Forrás: [B01] valószínűségszámítási fejezete a feltételes várható érték és variancia definíciójához. Szemléltetés (nem mért, kitalált számokkal): öt különböző seeddel tanított modell kiértékelése a 60 képes validációs halmazon például 0,91 és 0,93 közötti pontosságot adhat, miközben a jelenetalapú bootstrap CI 0,85–0,97 lehet; a seed-szórás önmagában a széles adatbizonytalanságot nem mutatja meg.

5.4.9 Önellenőrzés és korábbi fogalmak áttekintése

Ellenőrző kérdések: (1) Mi a bootstrap mintavételi egysége a GlyphLab esetén, és miért? (2) Mi a különbség a páros és a független csoportos összehasonlítás között a varianciakomponensek szempontjából? (3) Melyik 5.3. Mintavételi bizonytalanság és intervallum alfejezetbeli fogalom adja a konfidenciaintervallum ismételt mintavételi értelmezését? (4) Miért nem elegendő egy seed a tanítási sztochasztikusság jellemzésére, és miért nem azonos ez az adatminta újrahúzásával?

Válaszok röviden: (1) a jelenet, mert a két nézet közös geometriájú, korrelált; (2) a páros elemzés a klaszterek közötti szórást kiejtheti a különbségből, ezért gyakran kisebb standard hibát ad; (3) a lefedettség fogalma; (4) a seed csak az algoritmus véletlenjét változtatja, az adatot nem, ezért a kettő két külön varianciaforrás. A visszatekintés a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetből: a gradiensellenőrzés determinisztikus numerikus protokollt követ; itt a statisztikai szórás más természetű, és nem keverhető a lebegőpontos hibával. Az 1.4. Tanulási diagnózis és kísérleti napló alfejezetből: a seed a generátor állapota, a futásazonosító a napló kulcsa; a többseed-kísérletben mindkettőt rögzíteni kell.

5.4.10 Rövid összefoglaló

A megfigyelési egység és a statisztikai mintavételi egység akkor tér el, ha képek klaszterekbe, például jelenetekbe rendeződnek. A klaszteren belüli korreláció miatt a mintaátlag varianciája $\sigma^2/(Mr) + \tau^2/M$, ahol M a jelenetek, r a képek száma; a képszám növelése a τ^2/M tagot nem csökkenti. A jelenetalapú bootstrap a teljes klasztereket húzza visszatevéssel, és szélesebb konfidenciaintervallumot adhat, mint a képenkénti. A páros összehasonlítás a klaszteren belüli különbséget elemzi. A tanítási seed ismétlése az algoritmus, nem az adatminta bizonytalanságát méri; a kettőt külön kísérleti tervben kell kezelni.

5.4.11 Továbblépés

A 6.1. Objektív, reguláris tag és gradienslépés alfejezetben az empirikus kockázat és a gradienslépés következik. A jelenetalapú bootstrap és a páros gondolat a későbbi modellértékelésben, a 23.3. Páros összehasonlítás és csoportszintű intervallum alfejezetben tér vissza csoportszintű intervallumokkal.

5.4.12 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B05] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm.

6 Empirikus kockázat és optimalizálás

6.1 Objektív, reguláris tag és gradienslépés

6.1.1 Cél, előismeret és induló kérdés

Ez az egység a tanulás matematikai magját, az **empirikus kockázat minimalizálását** és annak első numerikus eszközét, a **gradienslépést** vezeti be. A 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben egy rögzített pontban számoltunk gradienst és ellenőriztük annak helyességét; ott a kérdés a *számítás*, nem a *paraméterfrissítés* volt. Az 5.4. Függő képek, seedek és resampling alfejezetben láttuk, hogy a minta statisztikai szerkezete — például a jelenetekbe rendezett képek — befolyásolja, mit jelent egy átlag bizonytalansága. Most ezt a két szálat kapcsoljuk össze: egy batch megfigyeléséből átlagolt veszteséget definiálunk, a modellparaméterekre vett gradiensét levezetjük, majd abból tényleges paraméterfrissítést készítünk.

Egy kérdés a mélytanulási kódok mindennapjából: miért szerepel sok tankönyvi levezetésben és kézzel definiált veszteségben egy $\frac{1}{2}$ szorzó, holott a beépített MSE-implementációk — például a PyTorch `MSELoss`-a [D23] — nem tartalmazzák; és miért beszélünk külön *adathibáról* és *regularizáló tagról*, holott a kódban a kettő egyszerűen összeadódik? A válaszhoz előbb pontosan definiálnunk kell az objektívfüggvényt, a hozzá tartozó tenzoralakokat, majd a derivált skáláját — ez utóbbi a batchméret és a konstansszorzók kezelésén áll vagy bukik.

6.1.2 Empirikus kockázat, batch-átlag és penalty

Definíció (empirikus kockázat). Legyen $D = \{(x_i, y_i)\}_{i=1}^{N}$ egy véges megfigyelés-sorozat, ahol $x_i \in \mathbb{R}^d$ a bemeneti vektor és y_i a hozzá tartozó cél, amely jelen egységben regressziós esetben $y_i \in \mathbb{R}^m$, osztályozásnál később osztályindex. Legyen $f_\theta : \mathbb{R}^d \to \mathbb{R}^m$ egy θ paraméterű modell és $\ell : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ egy elemenkénti veszteség. Az empirikus kockázat (a mintából becsült kockázat, jele $\hat{R}_N$)

$$\hat{R}_N(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i).$$

A kifejezés azért kockázat, mert a teljes adathalmazra vett átlagot jelenti; a szó *empirikus* arra utal, hogy a várható értéket a véges mintával helyettesítettük [B01]. Folytonos cél esetén a négyzetes veszteség $\ell(f, y) = \frac{1}{2}\|f - y\|_2^2$; a $\frac{1}{2}$ kizárólag a derivált kényelmét szolgálja, és nincs statisztikai jelentése. Osztályozási logitokhoz — a softmax előtti valós értékű pontszámokhoz — ugyanitt a 7.2. Softmax és többosztályos cross-entropy alfejezetben részletesen bevezetett cross-entropy kerül; a logitot nem kezeljük valószínűségként, a softmax a veszteségen belül történik.

A gyakorlati tanítás nem a teljes D-re, hanem egy $B \subseteq \{1, \dots, N\}$ minibatchre számol. Legyen $M = |B|$ a batch elemszáma. A batch bemenete $X_B \in \mathbb{R}^{M \times d}$, ahol minden sor egy megfigyelés; a cél $Y_B \in \mathbb{R}^{M \times m}$. A **batch-átlag** veszteség:

$$L_B(\theta) = \frac{1}{M} \sum_{i \in B} \ell(f_\theta(x_i), y_i).$$

Ha $B = \{1, \dots, N\}$, akkor $L_B = \hat{R}_N$; általában azonban a batch-átlag csak közelíti az empirikus kockázatot. Az 5.4. Függő képek, seedek és resampling alfejezetből tudjuk, hogy ha a megfigyelések klaszterekbe rendeződnek, a batch-átlag statisztikai értelmezése is finomabb: a képek szerinti átlagolás nem ugyanazt a független egységet becsli, mint a jelenetek szerinti.

Definíció (regularizáló penalty). Legyen $W \in \mathbb{R}^{m \times d}$ a modell lineáris paramétermátrixa, $b \in \mathbb{R}^m$ a biasvektor. Az L^2-regularizáló tag

$$P_\lambda(W) = \frac{\lambda}{2} \|W\|_F^2 = \frac{\lambda}{2} \sum_{a=1}^{m} \sum_{b=1}^{d} W_{ab}^2, \qquad \lambda \geq 0.$$

A λ skálaparaméter a regularizáció erőssége; a $\frac{1}{2}$ itt is a gradiens egyszerűbb alakját szolgálja. A Frobenius-norma a mátrix elemnégyzet-összegének négyzetgyöke, tehát $\|W\|_F^2$ a mátrixelemek négyzetösszege. A biasra rendszerint **nem** alkalmazunk L^2-tagot, mert a bias eltolást kódol, nem pedig a bemenetre ható erősítést.

Definíció (teljes batch-objektív). A regularizált batch-célfüggvény

$$F_B(W, b) = L_B(W, b) + P_\lambda(W).$$

A fenti L_B a W, b paraméterektől függ, mert a konkrét affin modell ezeket használja; az általános jelölésben a modell paraméterei θ, itt $\theta = (W, b)$. A gradienslépés a **gradient descent**:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W F_B(W^{(t)}, b^{(t)}), \qquad b^{(t+1)} = b^{(t)} - \eta \nabla_b F_B(W^{(t)}, b^{(t)}),$$

ahol $\eta > 0$ a **step size** (tanulási sebesség). A későbbi sztochasztikus változatnál 6.3. Sztochasztikus és minibatch gradiens a batchgradiens a minibatchből származik; itt egyelőre a frissítés alakjára koncentrálunk.

Jelölés. A fejezetben θ a modell összes paramétere; az affin modellnél $\theta = (W, b)$, és a képletekben ezt a bontást írjuk ki. Az objektívfüggvény jele F (batch esetén F_B), míg az F1–F5 a feladatok sorszáma: a feladatcímkék nem függvények. A batch-veszteség L_B, a kézi példa adatvesztesége $L(W)$, a regularizáló tag P_λ; a [P02] forrásazonosító, nem paraméter. A hibamátrix jele $R = Z - Y$, az empirikus kockázaté $\hat{R}_N$; a két R különböző objektum.

6.1.3 A gradiens levezetése: fél-MSE és $\lambda/2$ súlynorma

Tétel (a regularizált fél-MSE gradiense). Legyen a modell affin transzformáció: $Z = XW^\top + b$, ahol $X \in \mathbb{R}^{M \times d}$, $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$ sorirányú broadcast mellett, és $Z \in \mathbb{R}^{M \times m}$. Legyen a batch-veszteség

$$L_B = \frac{1}{2M} \|Z - Y\|_F^2 = \frac{1}{2M} \sum_{i=1}^{M} \sum_{k=1}^{m} (Z_{ik} - Y_{ik})^2,$$

és a teljes cél $F_B = L_B + \frac{\lambda}{2} \|W\|_F^2$. Ekkor

$$\nabla_W F_B = \frac{1}{M}(Z-Y)^\top X + \lambda W, \qquad \nabla_b F_B = \frac{1}{M}(Z-Y)^\top \mathbf{1}_M,$$

ahol $\mathbf{1}_M \in \mathbb{R}^M$ a csupa egyes vektor, és $\nabla_b F_B \in \mathbb{R}^m$.

A kifejezésben $(Z-Y)^\top X$ egy $m \times d$ mátrix, tehát $\nabla_W F_B$ alakja mindvégig megegyezik W alakjával. A biasgradiens a hibamátrix oszlopösszege osztva M-mel, ami m-dimenziós vektort ad. A tétel a láncszabály definíció szerinti alkalmazása; a levezetést teljesen elvégezzük, mert a skálatényezők — az $\frac{1}{M}$ és a $\frac{1}{2}$ — pontosan itt dőlnek el.

Levezetés. Jelölje $R = Z - Y$ a hibamátrixot. A veszteség elemenkénti deriváltja Z szerint:

$$\frac{\partial L_B}{\partial Z_{pq}} = \frac{1}{M}(Z_{pq} - Y_{pq}) = \frac{1}{M} R_{pq},$$

mert az $(Z_{pq}-Y_{pq})^2$-nél a láncszabály $2(Z_{pq}-Y_{pq})$ faktort ad, amelyet a $\frac{1}{2}$ kiejt, és az $\frac{1}{M}$ átlagolás megmarad. Most $Z_{ik} = \sum_{j=1}^{d} X_{ij} W_{kj} + b_k$, tehát

$$\frac{\partial Z_{ik}}{\partial W_{ab}} = \delta_{ka} X_{ib}, \qquad \frac{\partial Z_{ik}}{\partial b_a} = \delta_{ka},$$

ahol $\delta_{ka} = 1$, ha $k = a$, egyébként 0. A W_{ab} szerinti parciális derivált a többváltozós láncszabállyal 4.3. Jacobi-mátrix, láncszabály és VJP:

$$\frac{\partial L_B}{\partial W_{ab}} = \sum_{i=1}^{M}\sum_{k=1}^{m} \frac{\partial L_B}{\partial Z_{ik}} \frac{\partial Z_{ik}}{\partial W_{ab}} = \sum_{i,k} \frac{1}{M} R_{ik}\, \delta_{ka} X_{ib} = \frac{1}{M}\sum_{i=1}^{M} R_{ia} X_{ib}.$$

Az utolsó összeg a $R^\top X$ mátrix (a,b) eleme, mert $(R^\top X)_{ab} = \sum_i (R^\top)_{ai} X_{ib} = \sum_i R_{ia} X_{ib}$. Tehát $\nabla_W L_B = \frac{1}{M} R^\top X$. A biasra hasonlóan

$$\frac{\partial L_B}{\partial b_a} = \sum_{i,k} \frac{1}{M} R_{ik} \delta_{ka} = \frac{1}{M}\sum_{i=1}^{M} R_{ia}.$$

A biasgradiens tehát $\frac{1}{M} R^\top \mathbf{1}_M$, mert $R^\top$ alakja $m \times M$, és $\mathbf{1}_M$-mel szorozva az oszlopösszegeket kapjuk.

A regularizáló gradienséhez $\frac{\partial}{\partial W_{ab}} \frac{\lambda}{2} \sum_{c,d} W_{cd}^2 = \frac{\lambda}{2} \cdot 2 W_{ab} = \lambda W_{ab}$, tehát $\nabla_W P_\lambda = \lambda W$. Az $F_B = L_B + P_\lambda$ összeg deriváltja tagok összege, amiből a tétel képlete adódik. □

Fontos megkülönböztetés: a fenti $\frac{1}{2M}$ skálázású kvadratikus tag a **fél-MSE**. Két, gyakran „MSE”-nek nevezett skálázás van. Az $\frac{1}{M}\|Z-Y\|_F^2$ a **mintánkénti négyzetes norma** átlaga (a kimeneti komponensek összegét nem osztja el m-mel), gradiense $\frac{2}{M} R^\top X$. A PyTorch `MSELoss(reduction="mean")` ezzel szemben az összes elemre átlagol: $\frac{1}{Mm}\|Z-Y\|_F^2$ (m a kimeneti komponensek száma, lásd a 7.3. Bináris, multilabel és regressziós cél alfejezetet is). A kettő konstansszorzóban tér el, és ugyanazt a minimumot adja csak akkor, ha a regularizációs együtthatót is átskálázzuk: $\frac{2}{m}\left[\frac{1}{2M}\|Z-Y\|_F^2 + \frac{m\lambda}{4}\|W\|_F^2\right]$, tehát a fél-MSE λ-ja a PyTorch-skálázású λ_{PT}-nek $m/2$-szerese: $\lambda = \frac{m}{2}\lambda_{\text{PT}}$. Ezért a λ mellé mindig dokumentálni kell az MSE-skálát és az m szorzót. A gyakorlati kódok ezért expliciten dokumentálják, hogy a $\frac{1}{2}$-es skálázást hol alkalmazzák.

A **VJP-olvasat** 4.3. Jacobi-mátrix, láncszabály és VJP szerint: legyen $g = \nabla_Z F_B = \frac{1}{M} R$; az affin réteg backward lépéséből a veszteség *adatrészének* paramétergradiense $g^\top X$, illetve

a biasra a g oszlopösszege; a **teljes** (regularizált) gradiens ehhez még a közvetlen ágat adja: $\nabla_W F_B = g^\top X + \lambda W$. A két rész külön objektum. A fenti tétel pontosan ezt a programozói formát adja.

6.1.4 Kézi egyparaméteres példa és a lépések igazolása

A labor célja a gradienslépés mechanikájának kézi, ellenőrizhető követése. Legyen egyetlen bemeneti és kimeneti dimenzió, $d = m = 1$, és egyetlen mintapár: $x = 3$, $y = 9$. A modell $z = Wx$ (bias nélkül, a példa tisztasága érdekében). A fél-MSE egyetlen megfigyelésre:

$$L(W) = \frac{1}{2}(3W - 9)^2.$$

A regularizáló $\lambda = 0.2$ mellett $P(W) = 0.1\,W^2$. A teljes cél:

$$F(W) = \frac{1}{2}(3W - 9)^2 + 0.1W^2.$$

A gradiens: $F'(W) = 3(3W - 9) + 0.2W = 9.2W - 27$. Analitikus optimum: $F'(W^*) = 0$, azaz $W^* = 27/9.2 = 2.934782608\ldots$

Induljunk $W_0 = 0$-ból, $\eta = 0.05$ lépésközzel. A számított lépések:

- $t = 0$: $F(0) = \frac{1}{2}(81) = 40.5$, $F'(0) = -27$, frissítés $W_1 = 0 - 0.05 \cdot (-27) = 1.35$.
- $t = 1$: $F(1.35) \approx 12.25125 + 0.18225 = 12.4335$, $F'(1.35) \approx 9.2 \cdot 1.35 - 27 = -14.58$, frissítés $W_2 = 1.35 - 0.05 \cdot (-14.58) = 2.079$.
- $t = 2$: $3W_2 - 9 \approx -2.763$, négyzet ≈ 7.634, fél ≈ 3.817; regularizáció $0.1 \cdot 4.322 = 0.4322$; $F(2.079) \approx 4.249$; gradiens $F'(2.079) \approx 19.1268 - 27 = -7.8732$.
- $t = 3$: $W_3 = 2.079 + 0.39366 = 2.47266$.

Megfigyelés: F monoton csökken az első lépésekben, $F(0) = 40.5$, $F(1.35) \approx 12.43$, $F(2.079) \approx 4.25$. A gradiens abszolút értéke is csökken: 27, 14.58, 7.87; ez a konvergencia numerikus jele, de önmagában nem bizonyítja a végtelenben vett minimumot — a 6.2. Konvexitás és sima optimalizálási határ alfejezetben adunk majd simasági feltételek melletti kvantitatív korlátot.

Elsőrendű Taylor-indoklás: a $W_{t+1} = W_t - \eta F'(W_t)$ lépést behelyettesítve a lineáris közelítésbe $F(W_{t+1}) \approx F(W_t) + F'(W_t)(-\eta F'(W_t)) = F(W_t) - \eta |F'(W_t)|^2$, tehát minden nem nulla gradiensű pontban elegendően kis η mellett a lineáris közelítés csökkenést jelez. Ez lokális állítás; nagy η-ra a kvadratikus tag felülírhatja, és a cél nőhet. A konkrét kvadratikus célra a Taylor-közelítés másodrendig egzakt, ezért a lépés behelyettesítése zárt alakot ad:

$$F(W_{t+1}) = F(W_t) - \eta \left(1 - \frac{\eta F''}{2}\right) |F'(W_t)|^2,$$

ahol $F'' = 9{,}2$ állandó. Ebből a cél minden nem nulla gradiensű lépésben akkor és csak akkor csökken, ha $0 < \eta < 2/F'' = 2/9{,}2 \approx 0{,}2174$; a használt $\eta = 0{,}05$ esetén az egy lépésre jutó csökkenési együttható $\eta(1 - \eta F''/2) = 0{,}0385 > 0$.

6.1.5 Labor: NumPy gradienslépés egyparaméteres célra

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Munkakönyvtár: `ch06_empirical_risk/`. **Fájl:** `n021_gd_demo.py`.

A feladat az előző kézi példa teljes megismétlése NumPy-ban, ahol minden lépés kimenetét külön ellenőrizzük. Nincs véletlen komponens, ezért seed nem szükséges; ezt kimondjuk. A dtype mindenütt float64, mert a gradiensellenőrzési fegyelem 4.4. I. ellenőrző pont: matematika és kód megfelelése szerint a magasabb pontosság a numerikus hibák elkülönítését segíti. Az eszköz CPU; PyTorch nem kell hozzá.

```
# n021_gd_demo.py
# Az alábbi kimenet a kód futtatásával kapott mért érték.
import numpy as np

x = np.array(3.0, dtype=np.float64)       # bemeneti skalár
y = np.array(9.0, dtype=np.float64)       # cél
lam = 0.2                                 # regularizációs erősség
eta = 0.05                                # step size

def objective(w):
    z = w * x
    return 0.5 * (z - y) ** 2 + 0.5 * lam * w ** 2   # fél-MSE + lambda/2 w^2

def gradient(w):
    z = w * x
    return (z - y) * x + lam * w                     # (3w-9)*3 + 0.2w

def step(w):
    return w - eta * gradient(w)

w = np.array(0.0, dtype=np.float64)
for t in range(6):
    g = gradient(w)
    print(f"t={t}  w={w:.8f}  F={objective(w):.8f}  grad={g:.8f}")
    w = step(w)

# Ellenőrzés: az első három frissítés egyezzen a kézi lépésekkel
w_repro = np.array(0.0, dtype=np.float64)
for expected in (1.35, 2.079, 2.47266):
    w_repro = step(w_repro)
    np.testing.assert_allclose(w_repro, expected, rtol=0.0, atol=5e-6)
print("ellenorzes: az elso harom lepes a kezi szamitassal egyezik (atol=5e-6)")
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges):

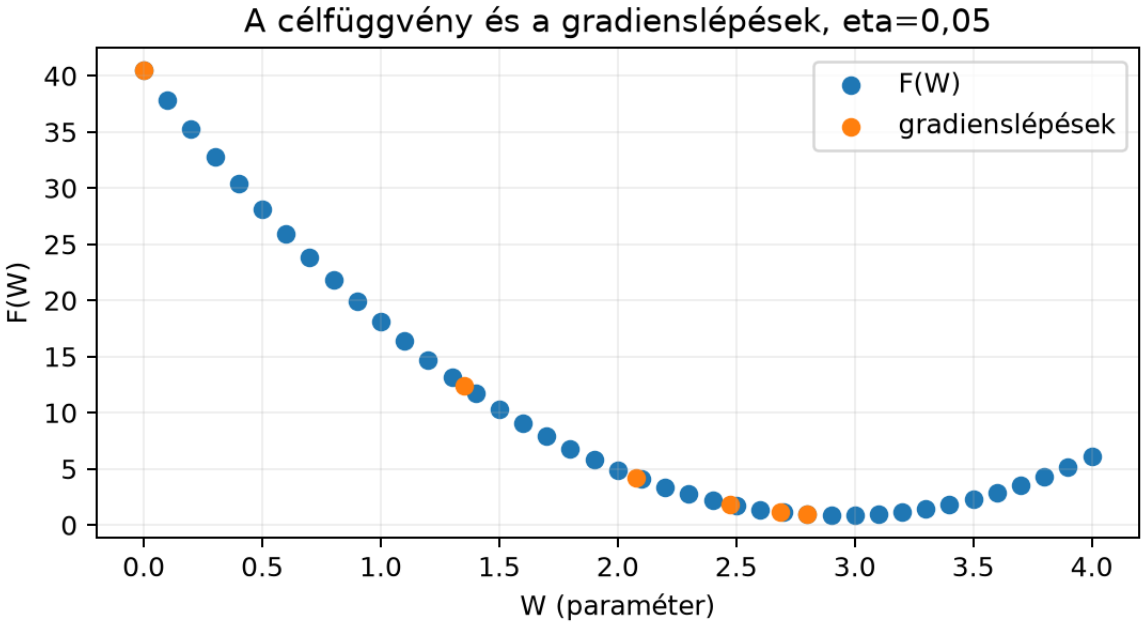
```
t=0  w=0.00000000  F=40.50000000  grad=-27.00000000
t=1  w=1.35000000  F=12.43350000  grad=-14.58000000
t=2  w=2.07900000  F=4.24930860  grad=-7.87320000
t=3  w=2.47266000  F=1.86279839  grad=-4.25152800
t=4  w=2.68523640  F=1.16689201  grad=-2.29582512
t=5  w=2.80002766  F=0.96396571  grad=-1.23974556
ellenorzes: az elso harom lepes a kezi szamitassal egyezik (atol=5e-6)
```

Futtatási parancs: `python n021_gd_demo.py`.

Eredményértelmezés. Az értékek a kód futtatásával mért értékek, amelyek az analitikus képletekkel is egyeznek; a beépített ellenőrzés az első három lépést hasonlítja össze a kézi számítással. Az analitikus optimum $W^* = 27/9.2 = 2.9347826\ldots$, és

$$F^* = \tfrac{1}{2}(3W^* - 9)^2 + 0.1\,W^{*2} \approx 0.880435.$$

A kód hat lépést futtat; a hatodik lépésnél $w = 2{,}862014934\ldots$ és $F \approx 0{,}9047924$. A tizedik lépésre a zárt rekurzió $W_{t+1} = 0{,}54\,W_t + 1{,}35$ alapján $W_{10} = W^* - W^*(0{,}54)^{10} = 2{,}9285951\ldots$, amiből $F(W_{10}) \approx 0{,}8806109$; ezt az értéket a kód nem futtatja, hanem a rekurzióból számoltuk. A konvergencia lineáris, mert $F''(W) = 9{,}2$ állandó: a $W_{t+1} - W^* = (1 - \eta F'')(W_t - W^*)$ hibarekurzióból $|W_t - W^*| = (0{,}54)^t |W_0 - W^*| = (0{,}54)^t \cdot 2{,}9348$, tehát a $0{,}54 < 1$ tényező pontosan a $W_t - W^*$ hiba geometriai csökkenése. A stabil lépésköz feltétele $0 < \eta < 2/F'' = 2/9{,}2 \approx 0{,}2174$, amit a használt $\eta = 0{,}05$ teljesít; tíz lépés után $|W_{10} - W^*| \approx 0{,}00619$.



6.1. ábra: A célfüggvény és a gradienslépések, eta=0,05

A parabola analitikus, a pontok a labor kódjának mért iterátumai.

6.1.6 Feladatok

F1 (fogalmi, nehézség 1). Fogalmazd meg saját szavakkal a különbséget az empirikus kockázat, a batch-átlag és a regularizáló penalty között. Adj egy példát, amelyben a batch-átlag nem egyenlő az empirikus kockázattal. *Beadandó:* rövid definíciók és az ellenpélda konkrét N-nel, batchmérettel.

F2 (levezetési, nehézség 2). Legyen $M = 2$, $d = 2$, $m = 1$, $X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $Y = \begin{pmatrix} 5 \\ 7 \end{pmatrix}$, modell $Z = XW^\top$ (bias nélkül), $W = (W_{11}, W_{12})$, és cél $F = \frac{1}{2M}\|Z - Y\|_F^2 + \frac{\lambda}{2}\|W\|_F^2$. Vezesd le $\nabla_W F$ értékét a $W = (0, 0)$ pontban, $\lambda = 0.1$ esetén. *Beadandó:* teljes levezetés, az alakellenőrzés és a konkrét numerikus gradiensvektor.

F3 (implementációs, nehézség 2). Implementáld a fenti laborlépést általános x, y skalár, λ, η bemenettel egy `f3_gd.py` függvényben úgy, hogy a futás a W_1, W_2, W_3 értékeket adja vissza. Tedd ellenőrizhetővé: az $x = 3, y = 9, \lambda = 0.2, \eta = 0.05$ bemenetre az első három frissítés 1.35, 2.079, 2.47266. *Beadandó:* a fájl teljes szövege, inputok, tolerancia és futtatási parancs.

F4 (hibakeresési, nehézség 2). A következő függvény a paramétergradienst számítja:

```
def buggy_grad(W, X, Y):
    Z = X @ W.T         # (M, m); W: (m, d)
    R = Z - Y           # (M, m)
    return R.T @ X      # (m, d), de hiányzik az 1/M átlagolás
```

Magyarázd meg, milyen hibát követ el, mi a következménye a gradienslépésre, és hogyan javítanád. *Beadandó:* a hiba oka, hatása és a javított függvény, valamint egy numerikus kontroll, amelyben $M = 2$ és a javítás utáni gradiens éppen a felét adja a hibásnak.

F5 (kísérleti transzfer, nehézség 3). Tervezz kísérletet annak mérésére, hogy a λ regularizáció hogyan hat egy kis, procedurálisan generált lineáris regressziós feladat tanítási és validációs hibájára. A tervnek tartalmaznia kell: adatgenerálási recept, megfigyelési egység, split, baseline, a változó λ-értékek, a kiértékelendő metrika, a számítási erőforrás és a következtetési határ. *Beadandó:* a protokoll és legalább két lehetséges eredményértelmezés a korlátokkal.

6.1.7 Megoldások és indoklás

F1 megoldás. Az empirikus kockázat a teljes D mintára vett átlag $\frac{1}{N}\sum_{i=1}^{N}\ell_i$; a batch-átlag egy $B \subseteq \{1,\dots,N\}$ részhalmazra vett $\frac{1}{M}\sum_{i\in B}\ell_i$; a penalty a modellparaméterektől függő, adattól független büntetőtag, például $\frac{\lambda}{2}\|W\|_F^2$. Ha $N = 6$, $B = \{1,2\}$, akkor $L_B = \frac{1}{2}(\ell_1+\ell_2)$, míg $\hat{R}_N = \frac{1}{6}(\ell_1+\cdots+\ell_6)$; a kettő általában nem egyenlő, kivéve ha a batch éppen a teljes halmaz. A penalty akkor sem válik empirikus kockázattá, ha a batch az egész adat: az adatfüggő és paraméterbüntető tag más forrásból származik.

F2 megoldás. A tétel képletéből $\nabla_W F = \frac{1}{M}R^\top X + \lambda W$. Kezdőpontban $W = (0,0)$, tehát $\lambda W = 0$. Ekkor $Z = XW^\top = (0,0)^\top$, $R = Z - Y = (-5,-7)^\top$. A szorzat:

$$R^\top X = (-5,-7)\begin{pmatrix}1 & 2\\ 3 & 4\end{pmatrix} = (-5-21,\ -10-28) = (-26,-38).$$

Így $\frac{1}{M}R^\top X = \frac{1}{2}(-26,-38) = (-13,-19)$. A regularizáló zérus, tehát $\nabla_W F = (-13,-19)$. Alakellenőrzés: $W \in \mathbb{R}^{1\times 2}$, a gradiens is 1×2, helyes. A W_{11} szerinti komponens -13: ha W_{11}-et kis ε-nal növeljük, mindkét minta predikciója nő, a hibák negatívak, tehát a veszteség csökken, ezért a gradiens negatív.

F3 megoldás. A teljes fájl:

```
# f3_gd.py
import numpy as np

def gd_first_three(x=3.0, y=9.0, lam=0.2, eta=0.05):
    # A cél 0.5*(w*x-y)**2 + 0.5*lam*w**2; a lépéshez csak a gradiens kell.

    def grad(w):
        return (w * x - y) * x + lam * w

    w = 0.0
    out = []
    for _ in range(3):
        w = w - eta * grad(w)
        out.append(w)
    return np.array(out, dtype=np.float64)

if __name__ == "__main__":
    res = gd_first_three()
```

```
    print(res)
    np.testing.assert_allclose(res, [1.35, 2.079, 2.47266],
                               rtol=0.0, atol=5e-6)
    print("ok")
```

Futtatás: `python f3_gd.py`. Tolerancia: abszolút $5 \cdot 10^{-6}$. A várt értékek nem kerekített közelítések, hanem a $W_{t+1} = 0.54W_t + 1.35$ rekurzióból adódnak ($W_3 = 2.47266$); a tolerancia a bináris lebegőpontos ábrázolás legfeljebb 10^{-15} nagyságrendű eltérését fedi.

F4 megoldás. A `buggy_grad` a batch-összeget számítja, nem a batch-átlagot: $R^\top X$ értéke M-szerese az $\frac{1}{M}R^\top X$ átlaggradiensnek. Következmény: ha a lépésközt a normalizált gradiensre hangoltuk, a hibás gradiens ötszörös, tízszeres, százszoros lépést adhat, ami a célfüggvény növekedéséhez és divergenciához vezet. Javítás: osszunk a batch méretével, vagy használjunk `np.mean`-alapú szorzást:

```
def fixed_grad(W, X, Y):
    M = X.shape[0]
    Z = X @ W.T
    R = Z - Y
    return (R.T @ X) / M      # (m, d), batch-átlag
```

Numerikus kontroll: $W = (0, 0)$ (1×2), $X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $Y = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$. Ekkor $Z = XW^\top = (0, 0)^\top$, $R = (-2, -4)^\top$, $R^\top X = (-2, -4)$; a hibás grad $(-2, -4)$, a javított $\frac{1}{2}(-2, -4) = (-1, -2)$.

F5 megoldásminta. *Adat:* $N = 300$, $d = 5$ független standard normális x_i, cél $y_i = 2x_{i1} - x_{i2} + 0{,}3x_{i3} + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0; 0{,}5^2)$ — a zaj szórása tehát 0,5. *Modell:* bias nélküli lineáris regresszió, $W \in \mathbb{R}^5$. *Megfigyelési egység:* egy mintapont; itt nincs klaszterezés, ezért az egyszerű split elfogadható. *Split:* 60/20/20 train/validation/test, rögzített seeddel, például `rng = np.random.default_rng(20261030)`; folytonos cél esetén a rétegzés a cél öt kvantilisébe sorolással történik, és mindhárom részben a kvantilisek arányosan jelennek meg (train 180, validation 60, test 60 pont). A jellemzőket a train halmaz átlagával és szórásával standardizáljuk, és ugyanezt a transzformációt alkalmazzuk a validation és a test halmazra. *Baseline:* $\lambda = 0$ lineáris regresszió. *Változó:* $\lambda \in \{0; 0{,}001; 0{,}01; 0{,}1; 1\}$. *Optimalizálás:* minden λ-ra azonos $\eta = 0{,}05$ lépésköz; előzetesen ellenőrizzük, hogy $\eta < 2/L$, ahol L a $X_{\text{train}}^\top X_{\text{train}}/N_{\text{train}}$ legnagyobb sajátértéke plusz λ (a legnagyobb λ mellett a legszigorúbb). A futás minden λ-ra ugyanazzal a leállási feltétellel ér véget: $\|\nabla F\|_2 < 10^{-6}$, de legfeljebb 5000 lépés; a lépésszámot naplózzuk, és az 5000 lépést elérő futást nem tekintjük konvergáltnak, így a λ-értékeket azonos konvergenciaállapotban hasonlítjuk össze. *Metrika:* validációs MSE; a döntés után a teszt MSE-t csak egyszer számítjuk. *Erőforrás:* CPU, NumPy, az adatgenerálás és a split seedje rögzítve. *Értelmezés:* ha a validációs görbe U alakú, a kicsi λ alul-, a nagy λ túlregularizál; mivel az adat szintetikus és alacsony dimenziós, az eredmény nem vihető át nagy képmodellre. Lehetséges negatív eredmény: ha a zaj kicsi, a $\lambda = 0$ is jó, és a regularizálás nem javít; ez is érvényes, dokumentálandó eredmény.

6.1.8 Tipikus tévedés és önellenőrzés

Tévedés: a sum-reduction és a mean-reduction gradientse csak egy konstansban tér el, ezért a tanulási ráta átállításával mindegy, melyiket használjuk. A konstans valóban egy batch M-szeres skálája, de a *gyakorlati* következmény éles: legyen $M = 2$, $x_1 = 3, y_1 = 9$, $x_2 = 1, y_2 = 3$, modell $z_i = wx_i$. A fél-MSE mean-veszteség $L_{\text{mean}} = \frac{1}{4}[(3w - 9)^2 + (w - 3)^2]$, gradiense $w = 0$-nál $\frac{1}{2}[3(-9) + (-3)] = -15$; a sum-változat jele L_{sum}, gradiense -30. Ha a lépésközt $\eta = 0{,}05$ a mean-hez hangoltuk, a sum használata effektív $\eta = 0{,}10$-es lépést ad. Ez a példában még a stabilitási tartományon belül van: $L''_{\text{mean}} = 5$, ezért a mean-változatra $\eta < 2/5 = 0{,}4$ a feltétel, a sum-változat pedig $\eta < 2/10 = 0{,}2$-t enged. A sum-lépésköznek 0,2 fölé kellene kerülnie ahhoz,

hogy az effektív lépésköz meghaladja a 0,4-et és a példa divergáljon; a 0,05 még biztonságos, de felemészti a ráhagyást. A tanulság: a reduction nem *puszta átskálázás*, hanem a célfüggvény definíciójának része; a λ hangolása csak azonos skálázás mellett összehasonlítható.

Önellenőrzési lista.

1. A gradiens alakja egyezik-e W alakjával ($m \times d$)?
2. Az $\frac{1}{M}$ tényező minden adatgradiensben megjelenik-e, és a regularizáló tagban nem jelenik-e meg véletlenül?
3. A $\frac{1}{2}$ szorzó a fél-MSE-ben és a $\frac{\lambda}{2}$ tagban azonos konvencióból származik-e?
4. Az AdamW decoupled weight decay (a súlynorma-büntetés a gradienslépéstől elkülönítve, közvetlenül a paraméterből levonva) [P02] vajon azonos-e a fenti lossba tett L^2-taggal? Itt nem bizonyítottuk az azonosságot; a 6.4. Momentum és adaptív optimizálók fogja a kettő gradiensútvonalát megkülönböztetni.
5. A Taylor-érvelés (elsőrendű, lineáris közelítés) lokális; fogalmaztunk-e úgy, mintha globális konvergenciát bizonyítottunk volna? A nem-sima optimum vagy a nagy η ellenpélda, valamint a konvexitás (a függvény a két pontja közötti húr alatt marad) és a Lipschitz-simaság (a gradiens változásának korlátja) a 6.2. Konvexitás és sima optimalizálási határ alfejezetben kerül elő.

A fenti tétel levezetése a 2.1. Python, NumPy és tenzoralak alfejezetben bevezetett tenzoralak és broadcasting, valamint a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben bemutatott elemenkénti láncszabály közvetlen következménye; a biasgradiens a broadcast backward szabálya szerint oszlopösszegből áll. A tenzorértékű gradiens és a láncszabály általános háttere a [B01] 5. fejezetében található.

6.1.9 Rövid összefoglaló

Az empirikus kockázat a teljes minta átlaga, a batch-átlag annak minibatch-közelítése, a regularizáló penalty pedig adatfüggetlen paraméterbüntetés. A fél-MSE $\frac{1}{2M}\|XW^\top + b - Y\|_F^2$ gradiense $\frac{1}{M}(Z-Y)^\top X$, a biasgradiens $\frac{1}{M}(Z-Y)^\top \mathbf{1}_M$; az L^2-tag $\frac{\lambda}{2}\|W\|_F^2$ gradiense λW. A gradienslépés $W^{(t+1)} = W^{(t)} - \eta \nabla_W F$. A Taylor-sor szerint elegendően kis lépésköz csökkentést jelez, de ez lokális állítás, nem globális konvergenciabizonyíték. A sum és mean reduction a gradiens skáláját batchméretezővel változtatja; ez nem közömbös konvenció.

6.1.10 Továbblépés

A 6.2. Konvexitás és sima optimalizálási határ alfejezetben a konvexitás (a függvény a két pontja közötti húr alatt marad) és a simaság (a gradiens Lipschitz-feltétele: a gradiens változása legfeljebb L-szeres távolsággal korlátozott) fogalmát vezetjük be, hogy a gradienslépés csökkenésére ne csak lokális Taylor-érvünk legyen, hanem megadott Lipschitz-feltétel melletti kvantitatív korlát. A 6.3. Sztochasztikus és minibatch gradiens alfejezetben a minibatch-mintavétel varianciáját elemezzük, ahol az itt levezetett skálák döntik el a lépésköz és a batchméret együttes hangolását.

6.1.11 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [P02] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. International Conference on Learning Representations (ICLR 2019). https://arxiv.org/abs/1711.05101.

- [D23] PyTorch contributors (2026). *MSELoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.MSELoss.html. Hozzáférés: 2026-10-30.

6.2 Konvexitás és sima optimalizálási határ

6.2.1 1. Cél, előismeret és induló kérdés

A 6.1. Objektív, reguláris tag és gradienslépés egységben definiáltuk az empirikus kockázatot és a gradienslépést, de a Taylor-érv csak lokálisan, kis η lépésközre jelzett csökkenést. Itt két fogalmat építünk fel, amelyekkel a csökkenés kvantitatívan garantálható: a **konvexitást** és az **L-simaságot**. A cél annak a határnak a pontos megértése, amelyen belül a gradienslépés bizonyíthatóan csökkenti a veszteséget, és annak a felismerése, hogy ez a bizonyítás nem vihető át automatikusan a mély hálók nemkonvex veszteségére.

Előismeret: a 6.1. Objektív, reguláris tag és gradienslépés gradienslépés-képlete, az 5.2. Várható érték, variancia és likelihood határozott integráljai és a 4.3. Jacobi-mátrix, láncszabály és VJP láncszabálya; numerikus oldalról a 2.2. Lebegőpont és stabil elemi számítás toleranciafogalma. A képletekben $x \in \mathbb{R}^d$ oszlopvektor, a gradiens $\nabla f(x) \in \mathbb{R}^d$ ugyanilyen alakú, a norma az euklideszi norma, a skalárszorzat jele $\langle u, v \rangle = u^\top v$.

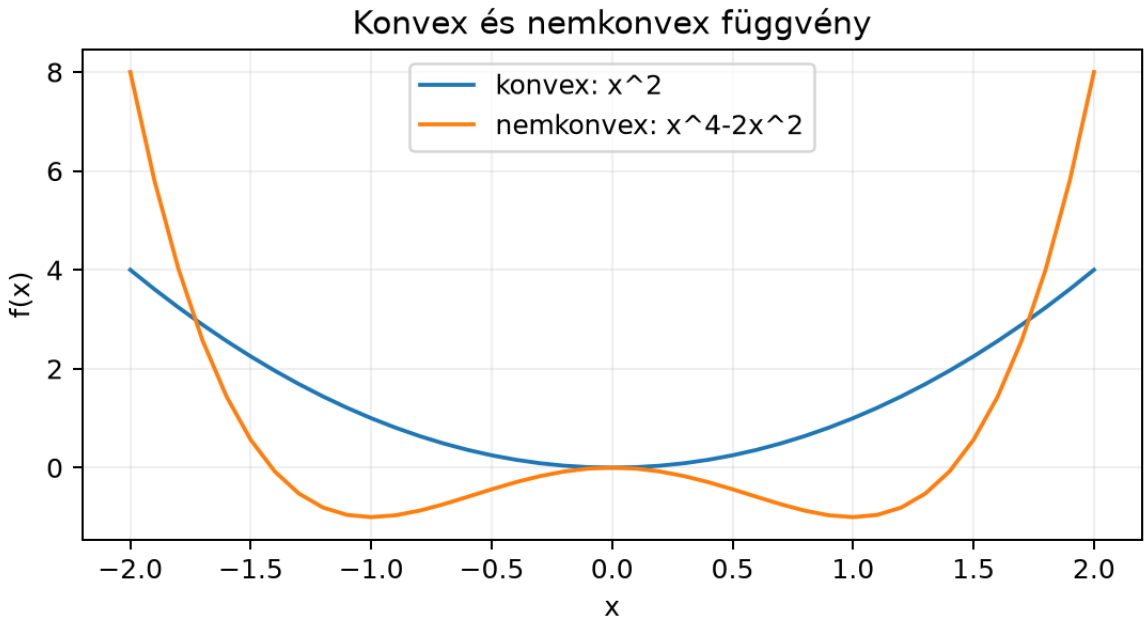
Induló kérdés: legyen $f(x) = x_1^2 + 3x_2^2$ és $x = (2, -1)$. Mekkora η mellett biztos, hogy egyetlen gradienslépés csökkenti az f értékét? A sejtés ellenőrzése után a leckében elégséges feltételt adunk arra, hogy egy lépés csökkentse az értéket; az adott pontra szabott pontos küszöb ennél élesebb lehet.

6.2.2 2. Konvexitás, lokális és globális optimum

Definíció (konvex függvény). Az $f : \mathbb{R}^d \to \mathbb{R}$ függvény konvex, ha minden $x, y \in \mathbb{R}^d$ és $\theta \in [0, 1]$ esetén

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta) f(y).$$

A definíció azt mondja, hogy a grafikon bármely húrja a függvényértékek fölött vagy azokon halad. Példa: $f(x) = \|x\|_2^2 = x_1^2 + \cdots + x_d^2$ konvex; $f(x) = \sin x$ nem konvex. A konvexitás önmagában nem követeli meg a differenciálhatóságot, de a továbbiakban differenciálható függvényekkel dolgozunk.



6.2. ábra: Konvex és nemkonvex függvény

Analitikus függvények; nem mérés.

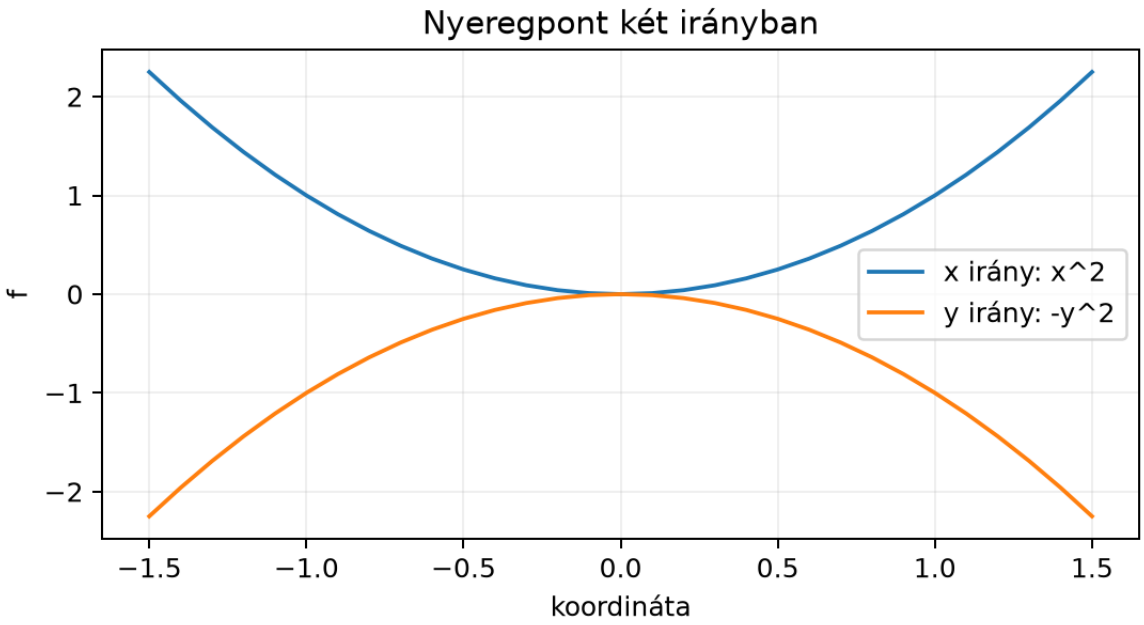
A konvex függvényeket használó bizonyításhoz a standard elsőrendű jellemzést használjuk: ha f differenciálható és konvex, akkor minden x, y pontban

$$f(y) \geq f(x) + \langle \nabla f(x),\, y - x \rangle.$$

Ennek bizonyítását itt nem ismételjük meg; a jellemzés a konvex elemzés standard tétele, és a [B01] 7. fejezetében (Continuous Optimization, 7.3. szakasz: Convex Optimization) megtalálható. A jelen lecke saját bizonyítási feladata a csökkenési lemma, ezért ezt a jellemzést kimondott feltételként használjuk.

Definíció (lokális és globális minimum). Az x^* pont **lokális minimuma** f-nek, ha létezik $r > 0$, hogy minden $\|x - x^*\| < r$ esetén $f(x) \geq f(x^*)$. Az x^* **globális minimum**, ha az egyenlőtlenség minden $x \in \mathbb{R}^d$ pontra teljesül. Szigorú minimumról akkor beszélünk, ha $x \neq x^*$ esetén az egyenlőtlenség szigorú.

Definíció (stacionárius pont). Az x^* pont **stacionárius pont**, ha $\nabla f(x^*) = 0$. Nemkonvex függvényen a stacionárius pont lehet lokális minimum, lokális maximum vagy nyeregpont; az $f(x, y) = x^2 - y^2$ függvény $(0, 0)$ pontja nyeregpont, mert x irányában minimum, y irányában maximum.



6.3. ábra: Nyeregpont két irányban

Az f(x,y)=x^{2-y}2 függvény két tengely menti metszete; nem mérés.

A konvexitás éppen abban segít, hogy a stacionárius pont és a globális optimum egybeesik.

Tétel (konvex függvény stacionárius pontja). *Legyen $f : \mathbb{R}^d \to \mathbb{R}$ differenciálható és konvex. Az $x^* \in \mathbb{R}^d$ pont akkor és csak akkor globális minimum, ha $\nabla f(x^*) = 0$.*

Bizonyítás. Először tegyük fel, hogy $\nabla f(x^*) = 0$. A fenti elsőrendű jellemzésből minden y pontra

$$f(y) \geq f(x^*) + \langle 0,\, y - x^* \rangle = f(x^*),$$

tehát x^* globális minimum. Megfordítva, tegyük fel, hogy x^* globális minimum. Ekkor minden y pontra és $t > 0$ értékre

$$f(x^* + t(y - x^*)) \geq f(x^*),$$

átrendezve

$$\frac{f(x^* + t(y - x^*)) - f(x^*)}{t} \geq 0.$$

A bal oldal a $t \mapsto f(x^* + t(y - x^*))$ összetett függvény különbségi hányadosa a 0-ban. Ez a függvény differenciálható $t = 0$-ban: a láncszabály szerint a deriváltja $\langle \nabla f(x^*),\, y - x^* \rangle$, mert a belső $t \mapsto x^* + t(y - x^*)$ függvény deriváltja $y - x^*$, a külső f gradiense pedig skalárszorzatként hat. Mivel a hányados minden $t > 0$-ra nemnegatív, és $t \to 0^+$ esetén a deriválthoz tart, a határérték is nemnegatív: $\langle \nabla f(x^*),\, y - x^* \rangle \geq 0$. Válasszuk az $y = x^* - \nabla f(x^*)$ pontot, ekkor

$$-\|\nabla f(x^*)\|_2^2 \geq 0,$$

ami csak $\nabla f(x^*) = 0$ mellett teljesülhet. $\square$

A tétel feltétele lényeges: az $f(x) = x^3$ függvényre $\nabla f(0) = 0$, de 0 nem minimum, mert f nem konvex. A mély hálók vesztesége a súlyparaméterekben általában nem konvex: már két súly szorzata, például $g(w_1, w_2) = (w_1 w_2 - 1)^2$, sem konvex függvény, egy több rejtett rétegű előrecsatolt háló (MLP) vagy CNN vesztesége pedig ilyen nemkonvex tagok kompozícióját tartalmazza.

6.2.3 3. L-simaság és a simasági egyenlőtlenség

A gradienslépés csökkenésének igazolásához elég, ha a gradiens nem változik túl gyorsan; az alábbi L-simaság ezt rögzíti. Konvexitás nem szükséges hozzá: a 6.2.4. szakaszban definiált csökkenési lemma (descent lemma) bizonyítása nem használja a konvexitást.

Definíció (L-sima függvény). Legyen $L > 0$. Az $f : \mathbb{R}^d \to \mathbb{R}$ folytonosan differenciálható függvény **L-sima**, ha a gradiense L-Lipschitz-folytonos, azaz a gradiens megváltozásának normája legfeljebb L-szerese a pontok távolságának:

$$\|\nabla f(x) - \nabla f(y)\|_2 \le L\|x - y\|_2$$

minden $x, y \in \mathbb{R}^d$ pontra. Az L konstans a gradiensváltozás felső korlátja: a gradiens két pont közötti eltérésének normája legfeljebb L-szerese a pontok távolságának. Ha f kétszer differenciálható, akkor L a Hesse-mátrix (a második parciális deriváltakból álló szimmetrikus mátrix) spektrálnormájának (a mátrix legnagyobb szinguláris értékének) felső korlátja, másodrendű közelítésben a legnagyobb görbület szerepét tölti be.

Példa: $f(x) = \frac{1}{2}\|x\|_2^2$ gradiensére $\nabla f(x) = x$, tehát

$$\|\nabla f(x) - \nabla f(y)\|_2 = \|x - y\|_2,$$

ezért f 1-sima, és $L = 1$ éles. Az $f(x) = \frac{1}{2}x^\top Q x + c^\top x$ szimmetrikus $Q \in \mathbb{R}^{d\times d}$ esetén $\nabla f(x) = Qx + c$, ezért $L = \|Q\|_2$, a Q spektrálnormája (a legnagyobb szinguláris értéke); szimmetrikus Q esetén ez $\max_i |\lambda_i(Q)|$, pozitív szemidefinit Q esetén pedig $\lambda_{\max}(Q)$, a legnagyobb sajátérték.

Lemma (simasági egyenlőtlenség). *Ha f L-sima, akkor minden $x, y \in \mathbb{R}^d$ pontra*

$$f(y) \le f(x) + \langle \nabla f(x),\, y - x\rangle + \frac{L}{2}\|y - x\|_2^2.$$

Bizonyítás. Rögzítsük x-et és y-t, legyen $d = y - x$, és definiáljuk az egyváltozós $g(t) = f(x + td)$ függvényt a $[0, 1]$ intervallumon. Írjuk fel g-t $g = f \circ \gamma$ összetett függvényként a $\gamma(t) = x + td$ affin pályával; ekkor $\gamma'(t) = d$. A többváltozós láncszabály 4.3. Jacobi-mátrix, láncszabály és VJP szerint $g'(t) = \nabla f(\gamma(t))^\top \gamma'(t)$, azaz $g'(t) = \langle \nabla f(x + td),\, d\rangle$. Az 5.2. Várható érték, variancia és likelihood alfejezetben polinomokra kimondott Newton–Leibniz-formulát alkalmazva (a differenciálható függvényekre vonatkozó általános alak a matematikai analízis tétele)

$$f(y) - f(x) = g(1) - g(0) = \int_0^1 g'(t)\, dt = \int_0^1 \langle \nabla f(x + td),\, d\rangle\, dt.$$

Válasszuk le az első tagot:

$$f(y) - f(x) = \langle \nabla f(x),\, d\rangle + \int_0^1 \langle \nabla f(x+td) - \nabla f(x),\, d\rangle \, dt.$$

A Cauchy–Schwarz-egyenlőtlenséggel és az L-Lipschitz-feltétellel a maradéktag minden t esetén

$$|\langle \nabla f(x+td) - \nabla f(x),\, d\rangle| \le \|\nabla f(x+td) - \nabla f(x)\|_2 \|d\|_2 \le Lt\|d\|_2^2.$$

Az integrál monotonitását használva

$$\int_0^1 \langle \nabla f(x+td) - \nabla f(x),\, d\rangle \, dt \le \int_0^1 Lt\|d\|_2^2 \, dt = \frac{L}{2}\|d\|_2^2.$$

Behelyettesítve $d = y - x$ adja az állítást. $\square$

A simasági egyenlőtlenség egy kvadratikus felső korlát: az f grafikonja az x pontbeli érintő és egy $L/2$-súlyú másodrendű tag alatt marad. Ez a standard egyenlőtlenség az alapja a csökkenési lemmának.

6.2.4 4. Csökkenési lemma (descent lemma)

Most elvégezzük a lecke fő bizonyítási feladatát: kimondott simasági feltételek mellett igazoljuk a gradienslépés csökkenését $0 < \eta < 2/L$ esetén.

Tétel (descent lemma). *Legyen $f : \mathbb{R}^d \to \mathbb{R}$ folytonosan differenciálható és L-sima, $L > 0$. Legyen $0 < \eta < 2/L$, és legyen*

$$x' = x - \eta \nabla f(x).$$

Ekkor

$$f(x') \le f(x) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla f(x)\|_2^2.$$

Ha $\nabla f(x) \neq 0$, akkor az egyenlőtlenség jobb oldala szigorúan kisebb $f(x)$-nél, tehát $f(x') < f(x)$. Ha $\nabla f(x) = 0$, akkor $x' = x$.

Bizonyítás. Alkalmazzuk a simasági egyenlőtlenséget az $y = x'$ választással. Ekkor $y - x = x' - x = -\eta \nabla f(x)$, ezért

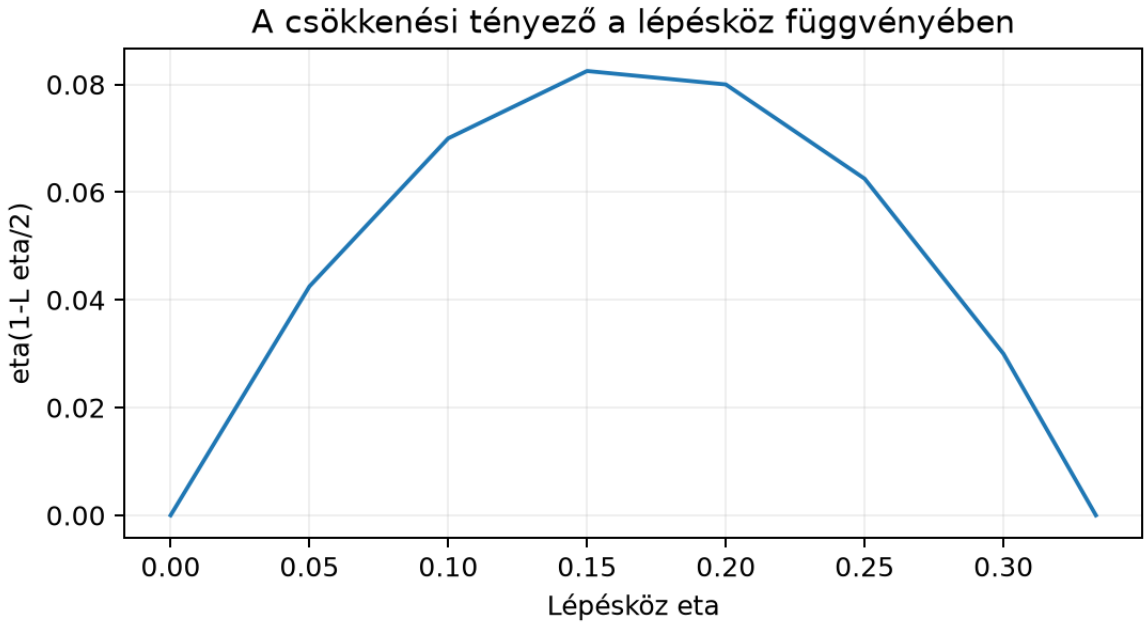
$$f(x') \le f(x) + \langle \nabla f(x),\, -\eta \nabla f(x)\rangle + \frac{L}{2}\| - \eta \nabla f(x)\|_2^2.$$

A skalárszorzat $\langle \nabla f(x), -\eta \nabla f(x)\rangle = -\eta \|\nabla f(x)\|_2^2$, a norma négyzete pedig $\eta^2 \|\nabla f(x)\|_2^2$. Így

$$f(x') \le f(x) - \eta\|\nabla f(x)\|_2^2 + \frac{L\eta^2}{2}\|\nabla f(x)\|_2^2 = f(x) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla f(x)\|_2^2.$$

Mivel $0 < \eta < 2/L$, a $1 - L\eta/2$ tényező pozitív. Ha $\nabla f(x) \neq 0$, akkor $\|\nabla f(x)\|_2^2 > 0$, tehát a kivont tag pozitív, és $f(x') < f(x)$. Ha $\nabla f(x) = 0$, akkor a kivont tag nulla, és $x' = x$. $\square$

A tétel két dolgot különít el. Egyrészt a csökkenés mértékét: a kivont tag $\eta(1 - L\eta/2)\|\nabla f(x)\|_2^2$, tehát erős gradiens esetén nagyobb a garantált csökkenés. Másrészt a lépésköz megengedett tartományát: ha η megközelíti a $2/L$ határt, a csökkenési tényező nullához tart; ha $\eta = 2/L$, a lemma már nem ad szigorú csökkenést. A felső korlát L-től függ, ezért erősebb görbület esetén kisebb lépésköz kell.



6.4. ábra: A csökkenési tényező a lépésköz függvényében

Az L=6 esetre a képletből számolt értékek; nem mért futás.

A bizonyítás nem használta a konvexitást, tehát a descent lemma nemkonvex függvényre is érvényes, amíg a simaság teljesül. Amit a lemma *nem* állít: nem bizonyítja, hogy az iteráció globális minimumhoz tart, még konvex esetben sem ad sebességbecslést, és $\eta < 2/L$ mellett is megállhat a sorozat stacionárius pontban. A lemma egy lépésre szóló monotonitási állítás.

6.2.5 5. Kézi kis példa

Tekintsük az induló kérdés függvényét, az

$$f(x_1, x_2) = x_1^2 + 3x_2^2$$

kvadratikus függvényt. Gradiense $\nabla f(x) = (2x_1,\, 6x_2)$. A gradiens különbségére

$$\|\nabla f(x) - \nabla f(y)\|_2^2 = 4(x_1 - y_1)^2 + 36(x_2 - y_2)^2 \le 36\|x - y\|_2^2,$$

tehát $L = 6$, és a megengedett tartomány $0 < \eta < 1/3$. Válasszuk az $x = (2, -1)$ pontot, ahol $f(x) = 4 + 3 = 7$ és $\nabla f(x) = (4, -6)$.

Először $\eta = 0{,}25$: a lépés

$$x_1' = 2 - 0{,}25 \cdot 4 = 1, \qquad x_2' = -1 - 0{,}25 \cdot (-6) = 0{,}5,$$

így $f(x') = 1^2 + 3 \cdot 0{,}5^2 = 1{,}75 < 7$. A descent lemma felső korlátja $L = 6$ mellett

$$7 - 0{,}25\left(1 - \frac{6 \cdot 0{,}25}{2}\right)(4^2 + 6^2) = 7 - 0{,}25 \cdot 0{,}25 \cdot 52 = 3{,}75,$$

és valóban $1{,}75 \leq 3{,}75$.

Most $\eta = 0{,}5$, ami kívül esik a $0 < \eta < 1/3$ tartományon. A lépés

$$x_1' = 2 - 0{,}5 \cdot 4 = 0, \qquad x_2' = -1 - 0{,}5 \cdot (-6) = 2,$$

tehát $f(x') = 0 + 3 \cdot 4 = 12 > 7$. A lépés nem csökkenti a veszteséget, mert a másodrendű tag, $\frac{L}{2}\eta^2\|\nabla f(x)\|_2^2$, felülírja a lineáris csökkenést. A példa azt mutatja, hogy a $2/L$ korlát nem pusztán technikai feltétel.

Az adott $x_0 = (2, -1)$ pontban egy lépés után $f(x') = 7 - 52\eta + 124\eta^2$, ami pontosan $\eta < 13/31 \approx 0{,}419$ esetén csökken. Ez az adott pontra szabott küszöb élesebb a lemma elégséges $0 < \eta < 1/3$ feltételénél: $\eta = 0{,}4$ kívül esik a lemma tartományán, mégis csökkenti a veszteséget ($f(x') = 6{,}04 < 7$). A lemma tehát elégséges, de nem szükséges feltételt ad.

6.2.6 6. Labor: túl nagy lépés miatti divergencia

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor egyetlen NumPy-fájl, amely ugyanazon a kétdimenziós kvadratikus példán hasonlítja össze a stabil és a túl nagy lépésközt. **Bemenet:** nincs külső adatfájl; a függvény, a kiindulási pont és az L konstans a kódban rögzített. **Seed:** a számítás determinisztikus, véletlen komponenst nem használ, ezért seed nem szükséges; az ismételhetőséget a konstans `x0`, `eta` és a rögzített rekurzió biztosítja. **Adattípus:** `float64`. **Eszköz:** CPU. **Munkakönyvtár:** `labs/n022`. **Futtatás:** `python n022_lab.py`.

```python
# labs/n022/n022_lab.py
import numpy as np

def f(x):
    return x[0]**2 + 3.0 * x[1]**2

def grad_f(x):
    return np.array([2.0 * x[0], 6.0 * x[1]], dtype=np.float64)

L = 6.0  # f az L=6 sima függvény

def run_steps(eta, x0, steps=4):
    x = x0.astype(np.float64).copy()
    x_prev = x.copy()
    f_prev = f(x_prev)
    g_prev = grad_f(x_prev)
    valid = 0.0 < eta < 2.0 / L
    print(f"\neta = {eta:g}, L = {L:g}, 2/L = {2.0/L:.6f}")
    print(f"{'t':>2} {'x1':>12} {'x2':>12} {'f(x)':>14} {'descent-felso':>14}")
    # 0. lépés: nincs még felső korlát, magát a pontot írjuk ki
    for t in range(steps + 1):
```

```
        if t == 0:
            upper_str = "-"
        else:
            upper = (
                f_prev
                - eta * (1.0 - 0.5 * L * eta)
                * float(np.dot(g_prev, g_prev))
            )
            upper_str = f"{upper:.8f}" if valid else "nem alkalmazhato"
        # a t. iterált kiírása
        print(f"{t:>2} {x[0]:>12.8f} {x[1]:>12.8f} {f(x):>14.8f} {upper_str:>14}")
        if t < steps:
            x_prev = x.copy()
            f_prev = f(x_prev)
            g_prev = grad_f(x_prev)
            x = x_prev - eta * g_prev
    return x

x0 = np.array([2.0, -1.0], dtype=np.float64)
x_stable = run_steps(0.25, x0, steps=4)   # stabil: 0.25 < 1/3
x_div = run_steps(0.50, x0, steps=4)      # túl nagy lépés: 0.50 > 1/3

# Ellenőrzés: a négy lépés utáni pontok a kézi számítással egyeznek
assert np.allclose(x_stable, [0.125, -0.0625], atol=1e-12, rtol=0.0)
assert np.allclose(x_div, [0.0, -16.0], atol=1e-12, rtol=0.0)
assert f(x_div) == 768.0
print("\nellenorzes: a stabil es a divergalo palya a kezi szamitassal egyezik")
```

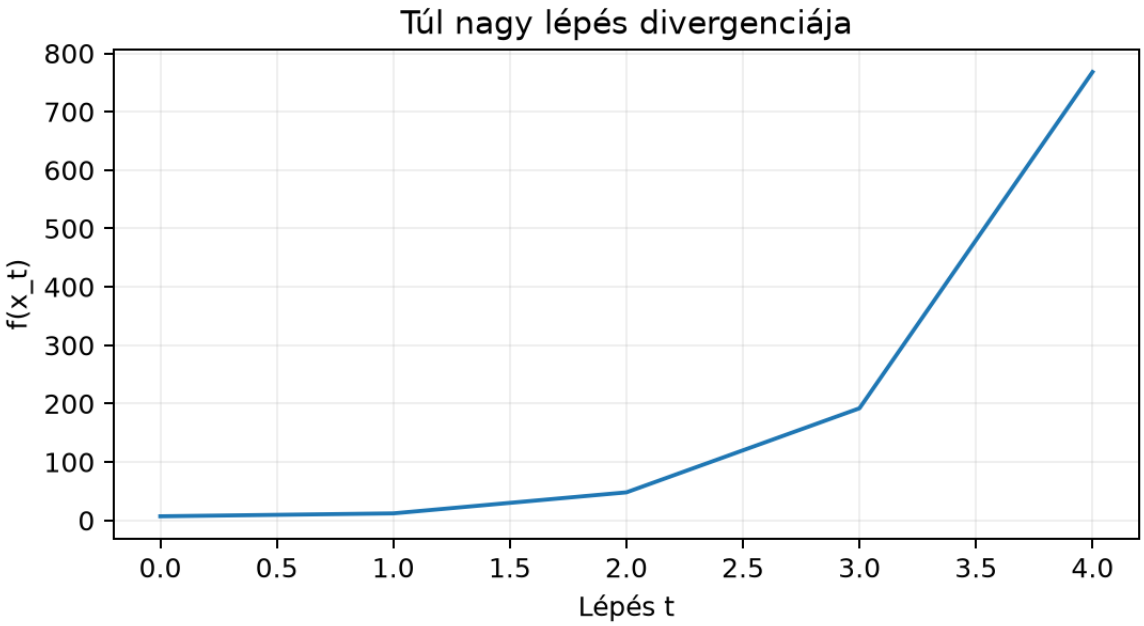
Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; a számítás determinisztikus, seed nem szükséges):

```
eta = 0.25, L = 6, 2/L = 0.333333
 t           x1           x2           f(x)  descent-felso
 0   2.00000000  -1.00000000     7.00000000              -
 1   1.00000000   0.50000000     1.75000000     3.75000000
 2   0.50000000  -0.25000000     0.43750000     0.93750000
 3   0.25000000   0.12500000     0.10937500     0.23437500
 4   0.12500000  -0.06250000     0.02734375     0.05859375

eta = 0.5, L = 6, 2/L = 0.333333
 t           x1           x2           f(x)  descent-felso
 0   2.00000000  -1.00000000     7.00000000              -
 1   0.00000000   2.00000000    12.00000000 nem alkalmazhato
 2   0.00000000  -4.00000000    48.00000000 nem alkalmazhato
 3   0.00000000   8.00000000   192.00000000 nem alkalmazhato
 4   0.00000000 -16.00000000   768.00000000 nem alkalmazhato

ellenorzes: a stabil es a divergalo palya a kezi szamitassal egyezik
```

Ellenőrzés és értelmezés. Az első futásban $\eta = 0{,}25 < 1/3$, ezért minden lépésre igaz a descent lemma felső korlátja; például $t = 1$-ben $1{,}75 \le 3{,}75$. A második futásban $\eta = 0{,}5 > 1/3$, ezért a lemma feltétele sérül, és a kód a felső korlát helyett a `nem alkalmazhato` jelzést írja. Az x_2 koordináta tényezője $1 - 6 \cdot 0{,}5 = -2$, tehát a koordináta előjelet vált és minden lépésben kétszereződik; az f értéke $7 \to 12 \to 48 \to 192 \to 768$ szerint divergál.



6.5. ábra: Túl nagy lépés divergenciája

Az eta=0.5, L=6 eset mért kimenete a laborfutásból; az értékek a rekurzió szerint divergálnak.

6.2.7 7. Gyakori tévedés ellenpéldával

Tévedés. *„A konvex kvadratikus példán a gradienslépés a globális minimumhoz tart, tehát ugyanez a bizonyítás garantálja a CNN-training globális konvergenciáját is."*

A hiba két lépésből áll. Egyrészt a descent lemma nem használ konvexitást, és nem is bizonyít globális konvergenciát; csak egy lépésre ad monoton csökkenést, ha $\eta < 2/L$. Másrészt a CNN-veszteség a paraméterekben nem konvex, ezért a konvex függvényekre kimondott stacionáriuspont-tétel feltétele nem teljesül.

Ellenpélda: az $f(x_1, x_2) = x_1^2 - x_2^2$ függvény gradiensére $\nabla f = (2x_1, -2x_2)$, és

$$\|\nabla f(x) - \nabla f(y)\|_2 = 2\|x - y\|_2,$$

tehát f 2-sima, így $\eta < 1$ mellett a descent lemma érvényes. Induljunk az $x^{(0)} = (\varepsilon, 0)$ pontból, $\varepsilon > 0$-val. A gradienslépés szabálya szerint

$$x_1^{(t+1)} = (1 - 2\eta)x_1^{(t)}, \qquad x_2^{(t+1)} = (1 + 2\eta)x_2^{(t)} = 0.$$

Mivel $|1 - 2\eta| < 1$, az $x_1^{(t)} \to 0$, tehát a sorozat a $(0, 0)$ ponthoz tart, és $f(x^{(t)}) \to 0$. A határpont stacionárius pont, de nem lokális minimum: az $(0, \delta)$ irányban $f(0, \delta) = -\delta^2 < 0$. A descent lemma tehát teljesülhet, miközben a módszer egy nyeregpontba konvergál. Konvex függvényre ez nem fordulhat elő, mert ott minden stacionárius pont globális minimum; a CNN-loss nem ilyen.

6.2.8 8. Feladatok

6.2.8.1 F1 (fogalmi, nehézség 1)

Definiálja a stacionárius pontot, és adjon példát olyan egyváltozós differenciálható függvényre, amelynek stacionárius pontja nem lokális minimum. Adja meg azt a feltételt, amely a konvex függvények tételében kizárja ezt a példát.

6.2.8.2 F2 (levezetési, nehézség 2)

Igazolja, hogy az $f(x_1, x_2) = x_1^2 + 3x_2^2$ függvény 6-sima, és hogy az $L = 6$ konstans éles. Az $x_0 = (2, -1)$ pontban $\eta = 0{,}25$ mellett számolja ki a descent lemma felső korlátját, majd hasonlítsa össze az $f(x^{(1)})$ tényleges értékével, ahol $x^{(1)}$ az első iterált. Ellenőrizze azt is, hogy $\eta = 0{,}4$ kívül esik a lemma által garantált $0 < \eta < 1/3$ tartományon, és magyarázza meg, miért csökkentheti mégis a veszteséget ebben a konkrét pontban.

6.2.8.3 F3 (implementációs, nehézség 2)

Írjon Python-függvényt `check_descent(f, grad_f, x0, eta, L, steps)` néven, amely a megadott lépésszámig elvégzi a gradienslépéseket, és minden lépésre visszaadja a $(t, x, f(x))$ pályát, valamint logikai értékkel jelzi, hogy a descent lemma felső korlátja teljesült-e. A függvény `float64` típusokat használjon, és a korlát ellenőrzéséhez relatív toleranciát alkalmazzon; érvénytelen lépésköznél jelezze, hogy a korlát nem értékelhető. Tesztelje az $f(x) = x_1^2 + 3x_2^2$, $L = 6$, $x_0 = (2, -1)$, $\eta = 0{,}25$ és $\eta = 0{,}5$ eseteken.

6.2.8.4 F4 (hibakeresési, nehézség 2)

Az alábbi függvény a lépésköz biztonságosságát hivatott eldönteni:

```
def is_safe(eta, L=6.0):
    return eta < 2.0
```

Az $L = 6$ példában a függvény az $\eta = 0{,}5$ értéket biztonságosnak minősíti. Azonosítsa a hibát, mondja meg a következményét a gradienslépésre, javítsa a függvényt úgy, hogy a teljes $0 < \eta < 2/L$ tartományt ellenőrizze (az alsó korlátot is), és egy konkrét $x_0 = (2, -1)$ ellenőrzéssel mutassa meg, hogy a javított változat már elutasítja a lépésközt.

6.2.8.5 F5 (kísérlettervezési/transzfer, nehézség 3)

Tervezzen transzfer jellegű kísérletet — a mérési elvet úgy kell megadni, hogy más beállításra is átvihető legyen — annak vizsgálatára, hogy a függelék A. szakaszában leírt GlyphLab adathalmaz (procedurálisan generált, háromosztályos grayscale adatkészlet, jelenetenként két nézettel) kis CNN-jén a teljes tanítási veszteség (a tanítóhalmazon mért átlagos veszteség) hogyan függ a lépésköztől. A GlyphLab adathalmaz 120/30/30 jelenetből és 240/60/60 képből áll; a jelenet-azonosító `scene_id`, a nézet azonosítója `view_id`, a metaadat `metadata.csv`. A CNN-veszteség nem konvex, ezért a descent lemma $\eta < 2/L$ feltétele nem ellenőrizhető előre; a kísérlet célja az empirikus stabilitás feltérképezése. Adja meg az adatforrást, a megfigyelési egységet, a baseline-t (viszonyítási alapot), a változó tényezőt, a metrikát és a következtetés érvényességi korlátját.

6.2.9 9. Megoldások és indoklás

6.2.9.1 F1 megoldása

A stacionárius pont definíciója: $\nabla f(x^*) = 0$. Az $f(x) = x^3$ függvényre $f'(x) = 3x^2$, tehát $x^* = 0$ stacionárius pont. Ez a pont mégsem lokális minimum: $x < 0$ esetén $f(x) < 0 = f(0)$, tehát minden $r > 0$ sugarú környezetben van a függvényértéknél kisebb érték. Ugyanígy lokális maximum sem: $x > 0$ esetén $f(x) > 0 = f(0)$, tehát minden környezetben van nála nagyobb érték is. Az x^3 függvénynek $x = 0$-ban inflexiós pontja van, nem szélsőértéke. A konvex függvényekről szóló tétel azért nem alkalmazható, mert x^3 nem konvex az egész számegyenesen. Cáfoló pontpár: $x = -2$, $y = 1$, $\theta = \frac{1}{2}$ esetén a konvex kombinált pont $-0{,}5$, ahol $f(-0{,}5) = -0{,}125$, míg a függvényértékek ugyanezen súlyú kombinációja $0{,}5 \cdot (-8) + 0{,}5 \cdot 1 = -3{,}5$. Mivel $-0{,}125 > -3{,}5$, sérti a konvexitás definícióját. Tehát a stacionárius pont nem minimum, és a konvexitás hiánya miatt a tétel nem is garantálhatja, hogy az legyen.

6.2.9.2 F2 megoldása

A gradiens $\nabla f(x) = (2x_1, 6x_2)$. Két x, y pontra

$$\|\nabla f(x) - \nabla f(y)\|_2^2 = (2(x_1 - y_1))^2 + (6(x_2 - y_2))^2 = 4(x_1 - y_1)^2 + 36(x_2 - y_2)^2.$$

Mivel

$$4(x_1 - y_1)^2 + 36(x_2 - y_2)^2 \le 36\left[(x_1 - y_1)^2 + (x_2 - y_2)^2\right] = 36\|x - y\|_2^2,$$

négyzetgyököt vonva $\|\nabla f(x) - \nabla f(y)\|_2 \le 6\|x - y\|_2$, tehát f 6-sima. Az élességhez válasszuk $x = (0, 1)$, $y = (0, 0)$: a különbség normája 6, és $\|x - y\|_2 = 1$, tehát a hányados 6.

$x_0 = (2, -1)$ esetén $\nabla f(x_0) = (4, -6)$, $\|\nabla f(x_0)\|_2^2 = 16 + 36 = 52$. A descent lemma felső korlátja $\eta = 0{,}25$ mellett

$$f(x_0) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla f(x_0)\|_2^2 = 7 - 0{,}25\left(1 - \frac{6 \cdot 0{,}25}{2}\right)52 = 7 - 3{,}25 = 3{,}75.$$

Az $x^{(1)} = (1;\, 0{,}5)$ pontban $f(x^{(1)}) = 1 + 3 \cdot 0{,}25 = 1{,}75 \le 3{,}75$. A feltétel $\eta < 2/L = 1/3 \approx 0{,}333$, ezért $\eta = 0{,}4$ kívül esik a descent lemma garantált tartományán. Ez a lemma feltétele elégséges, de nem szükséges: az adott x_0 pontban egy lépés után $f(x') = 7 - 52\eta + 124\eta^2$, ami $\eta < 13/31 \approx 0{,}419$ esetén csökken; $\eta = 0{,}4$-nál $f(x') = 6{,}04 < 7$.

6.2.9.3 F3 megoldása

A teljes függvény és a két teszt:

```
# labs/n022/feladat_f3.py
import numpy as np

def check_descent(f, grad_f, x0, eta, L, steps):
    x0 = np.asarray(x0, dtype=np.float64)
    x = x0.copy()
    trace = [(0, x.copy(), f(x))]
    valid_eta = 0.0 < eta < 2.0 / L
    bounds_ok = []
```

```
    for t in range(1, steps + 1):
        x_prev = x.copy()
        g = grad_f(x_prev)
        upper = f(x_prev) - eta * (1.0 - 0.5 * L * eta) * float(np.dot(g, g))
        x = x_prev - eta * g
        actual = f(x)
        trace.append((t, x.copy(), actual))
        if valid_eta:
            tol = 1e-12 * max(1.0, abs(upper))
            bounds_ok.append(bool(actual <= upper + tol))
        else:
            bounds_ok.append(None)  # a lemma nem alkalmazható, nincs korlát
    return trace, valid_eta, bounds_ok

def f(x):
    return x[0]**2 + 3.0 * x[1]**2

def grad_f(x):
    return np.array([2.0 * x[0], 6.0 * x[1]], dtype=np.float64)

x0 = np.array([2.0, -1.0], dtype=np.float64)
for eta in (0.25, 0.5):
    trace, valid, ok = check_descent(f, grad_f, x0, eta, L=6.0, steps=4)
    print(f"eta={eta:g}, eta_ervenyes={valid}, korlatok={ok}")
    print("  utolso iteralt:", trace[-1][1], "f:", trace[-1][2])
```

A várt kimenet a korábbi laborértékeket adja: $\eta = 0{,}25$ esetén `eta_ervenyes=True`, a korlátok mind teljesülnek, és $f(4) = 0{,}02734375$; $\eta = 0{,}5$ esetén `eta_ervenyes=False`, a `korlatok` lista `None` elemeket tartalmaz, mert a lemma nem alkalmazható, és $f(4) = 768$. Az érvényes esetben az `actual <= upper + tol` ellenőrzés közvetlenül a lemma egyenlőtlenségét teszteli; a `tol` relatív tolerancia a lebegőpontos kerekítésre. A `np.allclose` nem alkalmas erre, mert két érték egyezését vizsgálná, nem az egyenlőtlenséget, és az `actual <= upper` ág önmagában is elég; a kettő együtt felesleges.

6.2.9.4 F4 megoldása

A hiba oka: az `is_safe` a $2/L$ feltételt a konstans 2-vel helyettesíti, elfelejtve az L-lel való osztást. Következménye: nagy L esetén túl nagy η-t enged, ami a másodrendű tag miatt növelheti a veszteséget. Az $L = 6$ példában $\eta = 0{,}5 > 1/3$, ezért a $(2, -1)$ pontban a lépés az $f = 7$ értékről $f = 12$-re növel. Javítás:

```
def is_safe(eta, L=6.0):
    return 0.0 < eta < 2.0 / L
```

Ellenőrzés: `is_safe(0.5, 6.0)` hamis, `is_safe(0.25, 6.0)` igaz. A javított változat már elutasítja a divergáló lépésközt.

6.2.9.5 F5 megoldásminta

Protokolltervezet. *Adat:* a GlyphLab adathalmaz (120/30/30 jelenet, 240/60/60 kép), train/val/test jelenetszintű felosztás; a modell kiválasztása csak a validációs veszteség alapján. *Megfigyelési egység:* egy tanítási futás rögzített seeddel. *Baseline:* SGD (sztochasztikus gradiensleszállás) $\eta = 0{,}01$, minibatch-méret 32 (egy lépésben feldolgozott minták száma), rögzített inicializálás. *Változó tényező:* $\eta \in \{0{,}005; 0{,}01; 0{,}05; 0{,}1\}$. *Metrika:* az első 10 epoch (a tanítóhalmaz egy teljes bejárása) tanítási vesztesége (a tanítóhalmazon mért átlagos veszteség) epochonként, valamint az a jelzés, ha a tanítási veszteség meghaladja a kezdeti érték tízszeresét

vagy `NaN`-né válik. *Erőforrás:* CPU, `float32` a tanítás szokásos menetében; minden futást ugyanazzal a seeddel és ugyanabban a környezetben indítunk. *Elvárt beadandó eredmény:* lépésközönként egy táblázat az epochonkénti tanítási veszteségről és a leállás okáról (normál befejezés, küszöb túllépése vagy `NaN`), valamint egy rövid jegyzőkönyv. *Érvényességi korlát:* az eredmény azt mutatja, melyik lépésköz stabil ezen a kis CNN-en; nem bizonyítja a globális optimumot, nem ad becslést az ismeretlen L-re, és más architektúrára nem vihető át mérés nélkül.

6.2.10 10. Önellenőrzés, visszatekintés és tipikus hibák

- **Önellenőrző kérdés.** Az $f(x) = \frac{1}{2}x^\top Q x$ pozitív definit Q-val miért biztosítja a descent lemma, hogy $\eta = 1/\lambda_{\max}(Q)$ csökkent, és mi történik $\eta = 3/\lambda_{\max}(Q)$ választásnál? *Válasz:* $L = \lambda_{\max}(Q)$, ezért $\eta = 1/L$ a megengedett tartományban van; $\eta = 3/L$ kívül esik, mert $2/L$ a felső határ, és a lemma nem garantál csökkenést. A $\eta = 3/L$ esetben a hiba egy koordinátája a $(1 - \eta\lambda)$ tényezővel szorzódik; a legnagyobb sajátérték irányában ez $1 - 3 = -2$, abszolút értékben nagyobb 1-nél, ezért a hiba nem csökken, hanem előjelet váltva kétszereződik. A fenti laborban $L = 6$, $\eta = 0{,}5 = 3/L$, és a második koordináta divergál.
- **Visszatekintés 6.1. Objektív, reguláris tag és gradienslépés:** az $F(W) = f(W) + \frac{\lambda}{2}\|W\|_F^2$ regularizált cél L-simasága legfeljebb $L_f + \lambda$: itt $\nabla F(W) = \nabla f(W) + \lambda W$, ezért a háromszög-egyenlőtlenséggel $\|(\nabla f(W_1) + \lambda W_1) - (\nabla f(W_2) + \lambda W_2)\| \le (L_f + \lambda)\|W_1 - W_2\|$. A regularizáló tehát növeli a görbületet, és a megengedett lépésközt a $2/(L_f + \lambda) < 2/L_f$ korláton keresztül csökkenti. A [P02] tárgyalja, hogy AdamW decoupled weight decayje nem azonos a lossba írt L2 tag Adam alatti hatásával; ezt 6.4. Momentum és adaptív optimizálók részletezi.
- **Visszatekintés 2.2. Lebegőpont és stabil elemi számítás:** a labor `float64`-et használ. A példa értékei (a 0,25 és 0,5 lépésköz, valamint a kettőhatvány-koordináták) `float32`-ben is pontosan ábrázolhatók lennének, a `float64` választása mégis a numerikus egyezés bemutatását és a kerekítési hibák kizárását szolgálja; a `float32` körülbelül 7, a `float64` körülbelül 16 tizedesjegyet ad. A numerikus egyezés a descent lemma ellenőrzésében implementációs bizonyíték, nem helyettesíti a levezetést.

6.2.11 Rövid összefoglaló

A konvexitás azt jelenti, hogy a grafikon minden húr alatt vagy a húron halad; differenciálható konvex függvényre a nulla gradiens globális minimumot azonosít. Az L-simaság a gradiens Lipschitz-folytonossága, és belőle a simasági egyenlőtlenség, majd a csökkenési lemma következik: ha $0 < \eta < 2/L$, akkor a gradienslépés szigorúan csökkenti a veszteséget, amíg a gradiens nem nulla. A korlát túllépése kvadratikus példán divergenciát okoz, mert a másodrendű tag felülírja a lineáris csökkenést. A lemma nem igényel konvexitást, de nem bizonyít globális konvergenciát; nemkonvex függvényen a határpont lehet nyeregpont is, ezért a CNN-trainingre nem alkalmazható automatikusan.

6.2.12 Továbblépés

A 6.3. Sztochasztikus és minibatch gradiens a sztochasztikus gradiens módszerét tárgyalja: ott a teljes empirikus kockázat gradiense helyett minibatch-átlaggal dolgozunk, és a descent lemma feltételes várható értékben ad csökkenést, ha a minibatch-gradiens torzítatlan és a lépésköz a zaj kovarianciájával összeegyeztethető. A [D06] dokumentáció a `torch.optim` gradienslépéseit írja le. Ez a labor NumPy-alapú, ezért PyTorch-verziót nem használ; a reprodukálhatósághoz a NumPy verzióját és a konstans bemeneteket kell feljegyezni.

6.2.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. Hozzáférés: 2026-10-30.
- [P02] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. Előzetes változat: arXiv:1711.05101 (2017). https://arxiv.org/abs/1711.05101. Hozzáférés: 2026-10-30.
- [D06] PyTorch contributors (2026). *torch.optim*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/optim.html. Hozzáférés: 2026-10-30.

6.3 Sztochasztikus és minibatch gradiens

6.3.1 Cél, előismeret és induló kérdés

A 6.2. Konvexitás és sima optimalizálási határ egységben a teljes empirikus kockázat gradiensére épülő lépés csökkenési feltételét vizsgáltuk. A gyakorlatban N nagy lehet, és minden lépésben N darab mintánkénti gradiens kiszámítása drága. Ehelyett egy kisebb, véletlenül választott **minibatch** átlagával lépünk: ez a sztochasztikus gradiens módszer (SGD). Ehhez a gradiensbecslés fogalmára van szükség, amelyet az 5.3. Mintavételi bizonytalanság és intervallum egységből ismerünk: a becslő maga is véletlen változó, ezért torzításról és varianciáról beszélünk. A 2.3. Izolált környezet, notebook és verzió reprodukálhatósági elvei szerint a véletlen batchválasztás seedjét is rögzítenünk kell.

Induló kérdés. Egy $N = 100$ mintás hálótanításnál $m = 10$ méretű batch gradiensátlagát számoljuk. Vajon ez a részátlag várható értékben egyenlő a teljes gradienssel? És mekkora lehet egyetlen realizáció eltérése? Írjon le egy számpéldát, ahol a várható érték egyezik, de a konkrét batchérték távol van a céljától.

6.3.2 Definíciók és jelölések

Legyen $\theta \in \mathbb{R}^d$ a paramétervektor, és legyen az i-edik megfigyelés veszteségtagja $\ell_i(\theta) = \ell(\theta; x_i, y_i)$. A teljes empirikus kockázat az átlagos veszteség:

$$R(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\theta).$$

A gradiensoperátor lineáris, ezért a teljes gradiens a mintánkénti gradiensek átlaga:

$$\nabla R(\theta) = \frac{1}{N} \sum_{i=1}^{N} g_i, \qquad g_i = \nabla \ell_i(\theta) \in \mathbb{R}^d.$$

A g_i vektor a paraméter-tenzorral azonos alakú: skalár paraméternél $d = 1$, egy d-dimenziós súlyvektornál d-dimenziós.

Minibatch-gradiens. Legyen $B \subseteq \{1, \dots, N\}$ egy $m = |B|$ elemű indexhalmaz, amelyet az összes $\binom{N}{m}$ részhalmaz közül **egyenletesen, visszatevés nélkül** választunk. A minibatch-gradiens

$$G_B(\theta) = \frac{1}{m} \sum_{i \in B} g_i.$$

A G_B mennyiséget röviden **batchátlagnak** (minibatch-átlagnak) is nevezzük; a „batch” itt a kiválasztott minibatch-et jelöli, nem a teljes adathalmazt. Az $m = N$ határeset a teljes batch gradiens (determinisztikus, adott θ mellett), az $m = 1$ eset pedig a „tiszta” sztochasztikus lépés.

SGD-lépés. Az $\eta > 0$ tanulási ráta mellett

$$\theta_{t+1} = \theta_t - \eta\, G_B(\theta_t).$$

A tétel rögzített θ-ra és *új, független* mintavételre szól. Epochon belüli random reshuffling (véletlen újrakeverés: az adatok véletlen sorrendben, visszatevés nélkül, a DataLoader—a PyTorch batch-iterátora—szokásos működése) esetén egyetlen batch marginálisan pontosan egyenletes m elemű részhalmaz, ezért feltételesen torzítatlan. A batchek azonban nem függetlenek egymástól: a korábbi batch kizárja a későbbiekből ugyanazokat az indexeket, és a θ_t is függ a korábbi batchaktől. Ezért a független újramintavételre kimondott varianciaképlet csak közelítés; a torzítatlanság viszont batch-szinten továbbra is pontos, mert minden egyes batch marginálisan egyenletes. A „sztochasztikus” jelző a B választásának véletlenjére utal, nem a gradiensminták belső zajára. Az irányvektor a gradienszajjal is felírható:

$$G_B(\theta) = \nabla R(\theta) + \varepsilon_B(\theta),$$

ahol az ε_B hibatag eloszlása a mintavételi eljárásból származik.

Torzítatlanság. A $G_B(\theta)$ becslő **torzítatlan** $\nabla R(\theta)$-ra, ha a batchválasztás szerinti várható értéke megegyezik a céllal:

$$\mathbb{E}_B[G_B(\theta)] = \nabla R(\theta).$$

Torzítatlanság esetén a gradienszaj feltételes várható értéke nulla: $\mathbb{E}_B[\varepsilon_B(\theta) \mid \theta] = 0$. A feltételes megfogalmazás azért hasznos, mert a tanítás során θ_t maga is véletlen; a batchválasztás azonban θ_t-től független.

6.3.3 Tétel: a minibatch-gradiens torzítatlansága

Tétel (egyenletes, visszatevés nélküli minibatch torzítatlansága). Legyenek $N \geq m \geq 1$ egészek, $\theta \in \mathbb{R}^d$ olyan pont, ahol minden ℓ_i differenciálható, és $g_i = \nabla \ell_i(\theta)$. Ha B az $\{1, \dots, N\}$ halmaz egyenletes eloszlású m elemű részhalmaza visszatevés nélkül, akkor

$$\mathbb{E}_B[G_B(\theta)] = \nabla R(\theta).$$

Bizonyítás. Minden $i \in \{1, \dots, N\}$ indexhez vezessük be az indikátort: $I_i = 1$, ha $i \in B$, különben $I_i = 0$. Ekkor

$$G_B = \frac{1}{m} \sum_{i=1}^{N} I_i g_i.$$

Mivel B egyenletes eloszlású, az i elem határvalószínűsége a kedvező $\binom{N-1}{m-1}$ részhalmaz és az összes $\binom{N}{m}$ részhalmaz aránya:

$$P(i \in B) = \frac{\binom{N-1}{m-1}}{\binom{N}{m}} = \frac{(N-1)!}{(m-1)!(N-m)!} \cdot \frac{m!(N-m)!}{N!} = \frac{m}{N}.$$

A várható érték linearitása (véges összeg, ezért a sorrendcsere jogos) és az indikátor várható értéke alapján

$$\mathbb{E}_B[G_B] = \frac{1}{m} \sum_{i=1}^{N} \mathbb{E}_B[I_i]\, g_i = \frac{1}{m} \sum_{i=1}^{N} \frac{m}{N}\, g_i = \frac{1}{N} \sum_{i=1}^{N} g_i = \nabla R(\theta).$$

Ezzel a tételt beláttuk. A bizonyítás saját levezetés; a várható érték és a kovariancia alapfogalmai a [B01] hátterét követik. A tétel $m = 1$ és $m = N$ esetén is érvényes.

6.3.4 A batchméret és a gradienszaj varianciája

A torzítatlanság csak az átlagos viselkedést írja le. A gyakorlati lépésben a zaj nagysága döntő, ezért a kovarianciát is megvizsgáljuk.

Állítás (visszatevés nélküli batchzaj kovarianciája, $N \geq 2$). Legyen $1 \leq m \leq N$ és $\bar{g} = \nabla R(\theta)$. Az egyenletes, visszatevés nélküli választás mellett

$$\mathrm{Cov}_B(G_B) = \frac{N-m}{m(N-1)} \cdot \frac{1}{N} \sum_{i=1}^{N} (g_i - \bar{g})(g_i - \bar{g})^{\top}.$$

Levezetés. A g_i vektorok oszlopvektorok, ezért $g_i g_i^{\top}$ $d \times d$-s mátrix. Az előző indikátoros alakból

$$\mathrm{Cov}(G_B) = \frac{1}{m^2} \left[\sum_{i=1}^{N} g_i g_i^{\top} \mathrm{Var}(I_i) + \sum_{i \neq j} g_i g_j^{\top} \mathrm{Cov}(I_i, I_j) \right].$$

Az indikátorok marginálisai és páronkénti valószínűségei:

$$\mathrm{Var}(I_i) = \frac{m}{N}\left(1 - \frac{m}{N}\right) = \frac{m(N-m)}{N^2},$$

és $i \neq j$ esetén

$$\mathrm{Cov}(I_i, I_j) = P(i, j \in B) - P(i \in B)P(j \in B) = \frac{m(m-1)}{N(N-1)} - \frac{m^2}{N^2} = -\frac{m(N-m)}{N^2(N-1)}.$$

A $\sum_{i \neq j} g_i g_j^{\top} = (\sum_i g_i)(\sum_i g_i)^{\top} - \sum_i g_i g_i^{\top}$ azonosságot behelyettesítve, és a $\sum_i g_i = N\bar{g}$ jelölést használva

$$\mathrm{Cov}(G_B) = \frac{N-m}{mN(N-1)} \left[\sum_{i=1}^{N} g_i g_i^{\top} - \frac{1}{N} \left(\sum_{i=1}^{N} g_i \right) \left(\sum_{i=1}^{N} g_i \right)^{\top} \right] = \frac{N-m}{mN(N-1)} \sum_{i=1}^{N} (g_i - \bar{g})(g_i - \bar{g})^{\top}.$$

A zárójel utolsó lépése a centrált négyzetösszeg kifejtése. A $\sum_i g_i = N\bar{g}$ jelöléssel

$$\sum_{i=1}^{N} g_i g_i^\top - \frac{1}{N}\left(\sum_{i=1}^{N} g_i\right)\left(\sum_{i=1}^{N} g_i\right)^\top = \sum_{i=1}^{N} g_i g_i^\top - N\bar{g}\bar{g}^\top = \sum_{i=1}^{N}(g_i-\bar{g})(g_i-\bar{g})^\top,$$

mert a jobb oldal kifejtésében a kereszttagok $-2N\bar{g}\bar{g}^\top$, a $\bar{g}\bar{g}^\top$ tag pedig N-szer szerepel. Ezzel az állítás adódik.

A mátrix nyomára áttérve megkapjuk a gradienszaj skalár mértékét. A kovarianciamátrix írja le a vektor értékű becslő teljes szóródását; ennek nyoma a koordinátánkénti varianciák összege, amelyet **variancianyomnak** nevezünk. Legyen

$$\sigma_G^2 = \frac{1}{N}\sum_{i=1}^{N} \|g_i - \bar{g}\|^2.$$

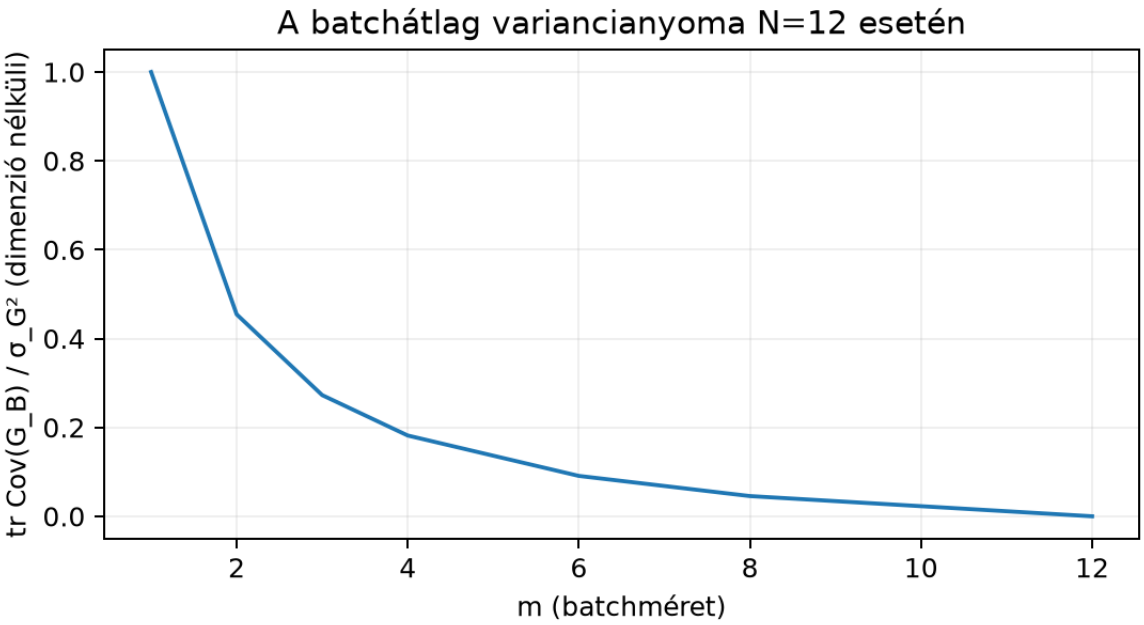
Ez a teljes sokaság adott mennyisége, nem becslő; a kalapos jelölést a mintából számított becslőknek tartjuk fenn.

Ekkor

$$\operatorname{tr}\operatorname{Cov}(G_B) = \mathbb{E}_B\|G_B - \bar{g}\|^2 = \frac{N-m}{m(N-1)}\,\sigma_G^2.$$

A $\frac{N-m}{N-1}$ tényező a **véges sokasági korrekció** (finite population correction). Visszatevéses mintavételnél ez a korrekció hiányzik, és a nyom σ_G^2/m lenne, mert a húzott pozíciók függetlenek. Visszatevés nélkül egy batchen belül egy index sosem ismétlődik; a variancia azért kisebb, mert a beválasztási indikátorok negatívan korreláltak (az egyik elem beválasztása csökkenti a többi esélyét), és ez a negatív kovariancia adja a véges sokasági korrekciót.

A variancia nullára csökken, ha $m = N$, hiszen ekkor G_B determinisztikusan a teljes gradiens. Az $m = 1$ szélső esetben a képlet a populáció szerinti varianciát adja: $\frac{1}{N}\sum_i \|g_i - \bar{g}\|^2$. Ha $N = 1$, akkor nincs választási bizonytalanság, ezért a variancia nulla; a fenti képlet az $N - 1$ nevező miatt ezt az esetet nem fedi le, külön kell kezelni.



6.6. ábra: A batchátlag variancianyoma N=12 esetén

A (N-m)/(m(N-1)) képletből számolt elméleti görbe, nem mérés.

A diagram a fenti képletből számolt elméleti görbe. A variancianyom pontosan a $\frac{N-m}{m(N-1)}$ szorzó szerint csökken; nagy N és kis m mellett ez közelítőleg $1/m$, a véges sokasági korrekció miatt pedig $m = N$-nél pontosan nullát ér el.

6.3.5 A zaj hatása a csökkenési egyenlőtlenségre

A 6.2. Konvexitás és sima optimalizálási határ simasági egyenlőtlensége szerint L-sima R függvényre (azaz R folytonosan differenciálható, és a gradiense L-Lipschitz-folytonos) és tetszőleges g irányra

$$R(\theta - \eta g) \le R(\theta) - \eta \langle \nabla R(\theta), g \rangle + \frac{\eta^2 L}{2} \|g\|^2.$$

Válasszuk a $g = G_B(\theta)$ minibatch-gradienst, és vegyük a batchválasztás feltételes várható értékét. A torzítatlanság és a második momentum dekompozíciója szerint

$$\mathbb{E}_B[G_B \mid \theta] = \nabla R(\theta), \qquad \mathbb{E}_B[\|G_B\|^2 \mid \theta] = \|\nabla R(\theta)\|^2 + \operatorname{tr} \operatorname{Cov}_B(G_B).$$

A második azonosság az 5.3. Mintavételi bizonytalanság és intervallum négyzetes hibafelbontás vektoros alakja: a második momentum a várható érték normanégyzete plusz a variancianyom. Ezzel

$$\mathbb{E}_B[R(\theta - \eta G_B(\theta)) \mid \theta] \le R(\theta) - \eta \left(1 - \frac{\eta L}{2}\right) \|\nabla R(\theta)\|^2 + \frac{\eta^2 L}{2} \operatorname{tr} \operatorname{Cov}_B(G_B).$$

Ez a levezetett egyenlőtlenség mutatja, hogy a zaj a jobb oldalon additív, η^2-szeres tagot ad. Ha $0 < \eta < 2/L$, a gradiensnormos tag csökkentő. A jobb oldal azonban csak felső korlát: ha a zaj

varianciája nagy, a felső korlát pozitív lehet, és a feltételes várható veszteség akár növekedhet is, tehát a csökkenés nem garantált. Ez nem jelenti azt, hogy a várható veszteség szükségképpen nő, hanem azt, hogy az egyenlőtlenség a növekedést sem zárja ki. Ez **nem** konvergenciabizonyítás: nem állítjuk, hogy a sorozat egy lokális minimumhoz tart; csupán egy lépés feltételes várható változását korlátozzuk. A minibatch-zaj varianciája a batchméret növelésével csökkenthető, de eközben a lépés számítási költsége nő.

6.3.6 Kézi kis példa

Legyen $d = 1$, $N = 3$, és egy rögzített θ pontban a mintánkénti gradiensek

$$g_1 = 2, \qquad g_2 = 5, \qquad g_3 = -1.$$

A teljes gradiens

$$\bar{g} = \frac{2 + 5 - 1}{3} = 2.$$

Visszatevés nélkül, $m = 1$ esetén a három lehetséges batchgradiens 2, 5 és -1, átlaguk $(2 + 5 - 1)/3 = 2$. $m = 2$ esetén a három lehetséges pár átlaga:

$$\{1, 2\} : \frac{2 + 5}{2} = 3{,}5, \qquad \{1, 3\} : \frac{2 - 1}{2} = 0{,}5, \qquad \{2, 3\} : \frac{5 - 1}{2} = 2.$$

Ezek átlaga $\frac{3{,}5+0{,}5+2}{3} = 2$. A becslő tehát mindkét batchméretnél torzítatlan. Ugyanakkor a konkrét realizációk szóródnak: például $m = 1$-nél a $g_2 = 5$ érték 3 egységgel tér el a teljes gradiensétől.

A variancia $m = 1$-nél

$$\mathbb{E}[G_B^2] - 2^2 = \frac{2^2 + 5^2 + (-1)^2}{3} - 4 = 10 - 4 = 6,$$

$m = 2$-nél pedig

$$\frac{3{,}5^2 + 0{,}5^2 + 2^2}{3} - 4 = \frac{12{,}25 + 0{,}25 + 4}{3} - 4 = 5{,}5 - 4 = 1{,}5.$$

A képlet ellenőrzése $m = 2$-re: $\sigma_G^2 = \frac{(2-2)^2+(5-2)^2+(-1-2)^2}{3} = 6$, így $\frac{3-2}{2(3-1)} \cdot 6 = \frac{6}{4} = 1{,}5$. A batchméret duplázása tehát a varianciát a negyedére csökkentette. Ez az erős csökkenés a kis N és a véges sokasági korrekció következménye: $N = 3$ mellett $m = 1$-nél a szorzó $\frac{3-1}{1\cdot(3-1)} = 1$, $m = 2$-nél $\frac{3-2}{2\cdot(3-1)} = \frac{1}{4}$, arányuk tehát $\frac{1}{4}$. Nagy N és kis m mellett a $\frac{N-m}{N-1}$ tényező közel 1, a szorzó pedig közelítőleg $1/m$, ezért a batchméret duplázása csak felezné a varianciát.

6.3.7 Labor: batchméret és gradiensvariancia mérése

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Kérdés. Hogyan csökken a gradienszaj varianciája a batchméret növelésével, és egyezik-e a mért variancianyom a levezetett képlettel?

Protokoll. A labor nem tanít neurális hálót, csak a batchválasztás statisztikáját méri. Ezért $N = 12$ darab szintetikus, $d = 4$ dimenziós minta-gradienst generálunk egy rögzített θ reprezentációjaként; ezekre a G_B becslő varianciáját számítjuk. Minden batchméretre $R = 20\,000$ húzást végzünk; egy húzás a teljes N indexből választ m-et visszatevés nélkül, a húzások egymástól függetlenek. A mért kovariancianyom torzítatlan becslője

$$\widehat{\operatorname{tr}\operatorname{Cov}} = \frac{1}{R-1}\sum_{r=1}^{R}\left\| G_{B_r} - \frac{1}{R}\sum_{s=1}^{R} G_{B_s}\right\|^2 .$$

A kód `float64` pontosságú, CPU-n fut, a seed a 2.3. Izolált környezet, notebook és verzió szerinti reprodukálhatósághoz rögzített. **A kódot a megadott seeddel lefuttattuk; az alábbi kivonat a mért kimenet kulcssorait mutatja.**

```
"""n023 labor: batchgradiens varianciája.

Futtatás: python n023_sgd_variance.py
"""
import numpy as np

rng = np.random.default_rng(20261020)
N, d, R = 12, 4, 20_000

# Szintetikus minta-gradiensek egy rögzített theta pontban.
# A labor csak a batchválasztás eloszlását méri.
G = rng.normal(size=(N, d)).astype(np.float64)
full = G.mean(axis=0)                       # teljes gradiens, (d,)
sigma2 = ((G - full) ** 2).sum() / N        # populációs variancianyom

def sample_trace(m):
    # m = N esetén a batchátlag determinisztikus, a variancia pontosan 0;
    # ezt a határesetet nem mérjük, hanem elméleti értékként jelöljük.
    assert 1 <= m < N
    draws = np.empty((R, d), dtype=np.float64)
    for r in range(R):
        idx = rng.choice(N, size=m, replace=False)
        draws[r] = G[idx].mean(axis=0)
    emp_mean = draws.mean(axis=0)
    emp = ((draws - emp_mean) ** 2).sum() / (R - 1)
    theo = (N - m) / (m * (N - 1)) * sigma2
    if theo > 0:
        rel = abs(emp - theo) / theo
    else:
        rel = 0.0 if emp == 0.0 else float("inf")
    return emp, theo, rel

for m in (1, 2, 4, 6):
    emp, theo, rel = sample_trace(m)
    print(f"m={m:2d}  emp={emp:.6f}  theo={theo:.6f}  rel={rel:.4f}")
    # R=20000-nél a relatív szórás Gauss-közelítéssel ~1%,
    # a 5%-os tűrés nagyjából öt szórás.
    assert rel < 0.05, "mintavételi vagy képlethiba jelölt értéknél"
```

```
print("m=12  elméleti határeset: VÁRT ÉRTÉK (levezetésből, nem mért): 0")
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1, seed 20261020). A kulcssorok:

```
m= 1  emp=4.173517  theo=4.164586  rel=0.0021
m= 2  emp=1.880483  theo=1.892994  rel=0.0066
m= 4  emp=0.752894  theo=0.757198  rel=0.0057
m= 6  emp=0.381548  theo=0.378599  rel=0.0078
m=12  elméleti határeset: VÁRT ÉRTÉK (levezetésből, nem mért): 0
```

A konkrét számok a NumPy verziójától függően kis mértékben eltérhetnek; a tendencia és a képlettel való egyezés nem.

A várt tulajdonság: $m = 1$-nél a legnagyobb a variancianyom, $m = 12$-nél matematikailag pontosan nulla a variancia; ezt a kód külön határesetként jelöli, mert mérés helyett a képletből adódik. A közbülső értékek a $\frac{N-m}{m(N-1)}$ tényezőt követik. Ha a relatív eltérés tartósan 5% fölé kerül, akkor a húzás vagy a variancia becslőjének normalizálása hibás; a visszatevéses húzás nem hiba, hanem más varianciaképletet ad. A számított csökkenés a gradienszaj mérnöki szabályozásának alapja: nagyobb batch kisebb zajt, de több számítást jelent lépésenként.

6.3.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el egy mondatban, mi a különbség egy torzítatlan gradiensbecslő eloszlása és egyetlen realizációja között. Adjon példát a fenti kézi példából arra, hogy egy realizáció messze van a teljes gradiensétől, miközben a becslő torzítatlan. Elvárt eredmény: a két fogalom definíciója és egy konkrét számpár.

F2 (levezetési, nehézség 2). Vezesse le a visszatevéses egyenletes mintavétel batchgradiensének kovarianciáját: mutassa meg, hogy $\mathrm{Cov}(G_B) = \frac{1}{m}\Sigma_G$, ahol $\Sigma_G = \frac{1}{N}\sum_i (g_i - \bar{g})(g_i - \bar{g})^\top$. Számítsa ki $N = 10$, $m = 2$ mellett a variancianyom szorzóját mindkét mintavételi módra, és indokolja a különbséget. Elvárt eredmény: teljes levezetés és a két numerikus tényező.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt, amely adott $G \in \mathbb{R}^{N\times d}$ gradiensmátrixhoz, m batchmérethez és R ismétléshez visszaadja a mért variancianyomot és az Állítás képletéből számolt elméleti értéket. Tesztelje a kézi példa $N = 3$, $g = (2, 5, -1)$ esetén $m = 2$, $R = 100\,000$ mellett; a mért érték 1,5-höz 2%-nál közelebb legyen. Elvárt eredmény: kód, futtatási parancs és a megadott tolerancia teljesülése.

F4 (hibakeresési, nehézség 2). Valaki a következő PyTorch-kóddal húz batchindexeket, és közben azt hiszi, hogy visszatevés nélkül mintavételez:

```
import torch

N, m = 3, 2
G = torch.tensor([[2.0], [5.0], [-1.0]])
idx = torch.randint(0, N, (m,))
g_batch = G[idx].mean(dim=0)
```

Mi a hiba, mi a következménye a varianciára, és hogyan javítható? Ellenőrizze a javítást az $N = 3$, $m = 3$ határesettel. Elvárt eredmény: a hiba oka, következménye, javított kód és a határeset indoklása.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen protokollt annak eldöntésére, hogy egy $d = 2$ dimenziós nemkonvex veszteség minimalizálásánál a batchméret növelése csökkenti-e a különböző seedekből származó végparaméterek szórását. Adja meg az adatot, a splitet, a

baseline-t, a vizsgált batchméreteket, a lépésköltségvetést, a metrikát és legalább egy érvényességi korlátot. Elvárt eredmény: egy szöveges protokoll és indokolt alternatíva; a tényleges futtatás nem kötelező.

6.3.9 Megoldások és indoklás

F1. A torzítatlanság a becslő eloszlásának várható értékére tett állítás: $\mathbb{E}_B[G_B] = \nabla R$. Egyetlen realizáció ettől tetszőlegesen messze lehet. A kézi példában $m = 1$, $B = \{2\}$ realizáció: $G_B = 5$, a teljes gradiens 2, az eltérés 3. Ugyanez a becslő a három lehetséges B felett átlagolva 2-t ad, tehát torzítatlan.

F2. Visszatevéses húzásnál a batch m darab független és azonos eloszlású pozícióból áll. Egyetlen I indexre g_I várható értéke $\bar{g}$, kovarianciája Σ_G, mert $P(I = i) = 1/N$. Az átlag kovarianciája ezért

$$\operatorname{Cov}\left(\frac{1}{m}\sum_{j=1}^{m} g_{I_j}\right) = \frac{1}{m^2}\sum_{j=1}^{m}\Sigma_G = \frac{1}{m}\Sigma_G.$$

A nyom szorzója visszatevéssel $1/m = 0.5$; visszatevés nélkül $\frac{N-m}{m(N-1)} = \frac{8}{2\cdot 9} = \frac{4}{9} \approx 0.444$. Visszatevés nélkül ugyanazt az indexet nem húzhatjuk kétszer: a beválasztási indikátorok kovarianciája negatív, ezért a batchátlag varianciája kisebb, mint a független (visszatevéses) esetben.

F3. Egy lehetséges megoldás:

```
import numpy as np

def variance_check(G, m, R, seed=20261020):
    N = G.shape[0]
    rng = np.random.default_rng(seed)
    full = G.mean(axis=0)
    sigma2 = ((G - full) ** 2).sum() / N
    draws = np.empty((R, full.shape[0]))
    for r in range(R):
        idx = rng.choice(N, size=m, replace=False)
        draws[r] = G[idx].mean(axis=0)
    emp = ((draws - draws.mean(axis=0)) ** 2).sum() / (R - 1)
    theo = (N - m) / (m * (N - 1)) * sigma2
    return emp, theo

G = np.array([[2.0], [5.0], [-1.0]])
emp, theo = variance_check(G, m=2, R=100_000)
print(emp, theo)
assert abs(emp - theo) < 0.02 * theo
```

Futtatás: `python feladat_f3.py`.

A becslő a G_B eloszlásának kovariancianyomát becsli, nem a σ_G^2 populációs mennyiséget; a kettőt a $\frac{N-m}{m(N-1)}$ szorzó köti össze. A normálás $R-1$-gyel történik, mert ez a G_B varianciájának torzítatlan becslője. A 100 000 húzás mellett a minta-variancia relatív szórása Gauss-közelítéssel körülbelül $\sqrt{2/99\,999} \approx 0.0045$, de a háromértékű példán a G_B diszkrét eloszlása miatt a minta-variancia relatív szórása ennél kisebb, körülbelül 0.22%; a 2%-os tűrés így is teljesül.

F4. A `torch.randint` [D25] minden pozícióban függetlenül húz, ezért ugyanaz az index többször is szerepelhet: visszatevéses mintavétel. Következmény: a becsült gradiens torzítatlan marad, de a variancia a nagyobb Σ_G/m képletet követi, és $m = N$ esetén sem lesz determinisztikusan a teljes gradiens. A javítás:

```
idx = torch.randperm(N)[:m]
g_batch = G[idx].mean(dim=0)
```

A `randperm` [D26] garantáltan különböző indexeket ad. A határeset $N = m = 3$-nál: a hibás kód akár az $\{1, 1, 1\}$ indexhármast is húzhatja, így $G_B = (g_1 + g_1 + g_1)/3 = g_1$, ami általában nem a teljes gradiens; a javított változat mindig a három különböző indexet adja, ezért $G_B = \bar{g}$ determinisztikusan teljesül.

F5. Lehetséges terv. Cél: annak eldöntése, hogy a batchméret növelése csökkenti-e a különböző seedekből származó végparaméterek szórását. Adat: rögzített $N = 400$ szintetikus pont, a veszteség $\ell_i(\theta) = \frac{1}{4}(\theta_1^2 - 1)^2 + \frac{1}{2}(\theta_2 - a_i)^2$, ahol $a_i \in [-1, 1]$ rögzített, egyenletesen elosztott érték; ez a kétgödrös veszteség kétdimenziós változata, amelynek két lokális minimuma van. Split: nincs külön validációs modellválasztás; a végső paramétert előre rögzített lépésszám után mentjük. Optimalizáló: SGD, tanulási ráta $\eta = 0{,}05$; kezdeti paraméter minden futásban ugyanaz a $\theta_0 = (1{,}2;\ 0)$. Baseline: $m = 1$. Vizsgált értékek: $m \in \{1, 10, 40, 400\}$. Mindegyikre $S = 20$ független seed. Két költségvetést kell megkülönböztetni: (a) rögzített 400 lépés, ekkor a gradiens-kiértékelések száma $400m$; (b) rögzített 400 gradiens-kiértékelés, ekkor a lépésszám $400/m$. Metrika: a 20 végparaméter empirikus kovariancianyoma. Döntési szabály: a batchméret növelését akkor tekintjük a szórást csökkentőnek, ha a $m = 40$ és $m = 1$ futások nyomának aránya 1 alatt van, és a seedekre párosított bootstrap 95%-os intervalluma nem tartalmazza az 1-et. Erőforrás: CPU, `float64`, a seedet és a könyvtárverziót rögzítjük. Korlát: a becsült szóródás a minimalizálási pálya tulajdonsága; a gradienszaj varianciaképletéből nem következik, hogy a végső paraméterek szórása pontosan $1/m$-szeres, mert a nemkonvex pálya nemlineárisan torzíthat, és a rögzített lépésszám mellett a nagyobb batch kevesebb frissítést jelent.

6.3.10 Tipikus tévedés és ellenpélda

Gyakori hiba azt hinni, hogy **minden** véletlen vagy adaptív batchválasztás automatikusan torzítatlan gradienst ad. A fenti tétel feltétele az egyenletes mintavétel. Súlyozott célfüggvénynél a korrekció nélküli átlag torzított.

Példa: legyen $N = 3$, a mintánkénti gradiensek $g_1 = 4$, $g_2 = 0$, $g_3 = 0$, a mintasúlyok pedig $w_1 = 4$, $w_2 = w_3 = 1$, $W = \sum_i w_i = 6$. A súlyozott célgradiens $R_w = \frac{1}{W}\sum_i w_i g_i = \frac{4\cdot 4 + 1\cdot 0 + 1\cdot 0}{6} = \frac{8}{3}$. Ha $m = 1$ egyenletesen húz, a gradiensbecslés várható értéke $\frac{4+0+0}{3} = \frac{4}{3} \neq \frac{8}{3}$, tehát a korrekció nélküli átlag torzított a súlyozott célra. A helyes eljárás vagy a súlyokkal arányos mintavétel (az i indexet w_i/W valószínűséggel húzzuk), vagy egyenletes húzás utáni $\frac{w_i N}{W}$ korrekciós súlyozás. A tanítási könyvtárak osztálysúlyozása a veszteségtagok súlyozását jelenti; ez önmagában nem változtatja meg a mintavétel egyenletességét, és az egyenletes batchválasztás mellett a gradiens torzítatlansága megmarad. Korrekcióra akkor van szükség, ha a mintavételt is a súlyokhoz igazítjuk.

6.3.11 Önellenőrzés és visszatekintés

Ellenőrző kérdések a szakasz után: (1) A tétel bizonyításában melyik lépés használja az egyenletes választást? (2) Mit ad a varianciaképlet $m = N$-re, és miért nulla? (3) Miért kell a csökkenési egyenlőtlenségben feltételes várható értéket venni? Válaszok vázlata: (1) a $P(I_i = 1) = m/N$ határvalószínűség; (2) a véges korrekció $N - m = 0$ tényezőt ad, mivel nincs választási véletlen; (3) mert θ_t maga is véletlen, és a zaj csak a batchválasztásból származik.

Visszatekintés az 5.3. Mintavételi bizonytalanság és intervallum egységből: a becslő a minta függvénye, a becslés pedig egy konkrét realizáció; a torzítás és variancia a becslő eloszlásának

tulajdonsága. A 6.2. Konvexitás és sima optimalizálási határ simasági egyenlőtlensége itt a zaj additív büntetőtagját adta, de a globális konvergencia továbbra sem következik belőle.

6.3.12 Rövid összefoglaló

A minibatch-gradiens m darab, egyenletesen választott minta gradiensének átlaga. A torzítatlansági tétel szerint várható értéke a teljes ∇R gradiens, feltéve, hogy a választás egyenletes és a paramétertől független. A bizonyítás indikátorokkal mutatja meg, hogy $P(i \in B) = m/N$, majd a várható érték linearitását használja. A batchzaj varianciája a véges sokasági korrekcióval $\frac{N-m}{m(N-1)}\sigma_G^2$; ez $m = N$-nél nulla. A labor a mért variancianyomot hasonlítja a képlethez, a batchméret növelése tehát mérhetően csökkenti a lépés zaját. A torzítatlanság csak átlagos tulajdonság, ezért egy kis batch realizációja önmagában nem helyettesíti a teljes gradienst. Az induló kérdésre a válasz: a részátlag várható értéke a teljes gradiens, tehát torzítatlan; $N = 100$, $m = 10$ esetén a variancianyom szorzója $\frac{100-10}{10\cdot 99} = \frac{1}{11} \approx 0{,}0909$, egyetlen realizáció ettől még eltérhet. A kért számpélda a kézi példából adódik: $m = 1$, $B = \{2\}$ esetén $G_B = 5$, a teljes gradiens 2, az eltérés 3, miközben a becslő várható értéke továbbra is 2.

6.3.13 Továbblépés

A 6.4. Momentum és adaptív optimizálók egységben a momentum és az adaptív optimizálók következnek. Ott az SGD-lépés zaját mozgóátlaggal simítjuk, illetve a második momentummal normáljuk a koordinátánkénti skálát. A most levezetett torzítatlansági és varianciaképlet kiindulópontja lehet annak a magyarázatnak, hogy a mozgóátlag miért csökkenti a pálya fluktuációját; ez azonban heurisztikus kép, nem bizonyítás, és a gyakorlati Adam/AdamW viselkedés nem vezethető le pusztán ebből a zajmodellből.

6.3.14 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. 6. fejezet (Probability and Distributions) és 7. fejezet (Continuous Optimization). https://mml-book.github.io/. Hozzáférés: 2026-10-30.
- [D25] PyTorch contributors (2026). *torch.randint*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.randint.html. Hozzáférés: 2026-10-30.
- [D26] PyTorch contributors (2026). *torch.randperm*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.randperm.html. Hozzáférés: 2026-10-30.

6.4 Momentum és adaptív optimizálók

6.4.1 Cél, előismeret és induló kérdés

A 6.3. Sztochasztikus és minibatch gradiens alfejezetben a minibatch-gradiens torzítatlanságáról és zajáról volt szó. Ott a lépés közvetlenül a becsült gradiens negatív irányába mutatott. A gyakorlatban az SGD azonban lassan haladhat olyan irányokban, ahol a gradiens kicsi, és erősen fluktuálhat olyan irányokban, ahol a gradiens nagy varianciájú. Ebben az egységben olyan frissítési szabályokat vizsgálunk, amelyek paraméteren kívül **állapotváltozókat** is tárolnak, és ezekből számítják a tényleges lépést. Az 5.4. Függő képek, seedek és resampling alfejezetből tudjuk, hogy a seed és a minta szerinti bizonytalanságot külön kell kezelni: az optimizer választása az algoritmikus bizonytalanságot befolyásolja, nem magát az adatmintát.

Induló becslés: miért lehet hasznos egy gradiens mozgóátlaga akkor is, ha az egyes gradiensbecslések torzítatlanok? Mit ront el, ha a súlycsökkentést egyszerűen hozzáadjuk a gradienshez egy adaptív normalizálás előtt?

6.4.2 Momentum mint állapotváltozó

Legyen a paramétervektor $\theta_t \in \mathbb{R}^d$, a gradiensbecslés $g_t \in \mathbb{R}^d$, a tanulási ráta $\eta > 0$, a momentum-együttható $\beta \in [0, 1)$. A PyTorch momentumos SGD-jének állapotváltozója $m_t \in \mathbb{R}^d$, alakja azonos θ alakjával, és inicializálása $m_0 = 0$. A frissítés:

$$m_t = \beta m_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta\, m_t.$$

Ez nem normalizált exponenciális mozgóátlag: konstans gradiens $g_t \equiv g$ esetén $m_t = (1 + \beta + \cdots + \beta^{t-1})g$, ami $t \to \infty$ esetén $g/(1-\beta)$-hoz tart. A PyTorch momentumos SGD-je `dampening=0` mellett ezt a konvenciót használja; `dampening>0` esetén a gradiens tag $(1 - \text{dampening})$-nel skálázódik, ezért a képlet nem egyezik. Ezért a momentum növeli az effektív lépésnagyságot egy konzisztens gradiensirányban; a tanulási rátát általában kisebbre kell választani, mint sima SGD-nél. A momentum célja heurisztikus: a konzisztens irányt simítja és gyorsítja, a nagy varianciájú, előjelváltó komponenseket részben kiátlagolja. Ez nem bizonyított konvergenciajavítás minden feladatra. A momentum és az adaptív lépések optimalizálási hátterének áttekintése a [B01] 7. fejezetében (Continuous Optimization) található.

6.4.3 Adam: első és második momentum

Az Adam három állapotot tart nyilván a paraméter mellett. Legyenek $\beta_1, \beta_2 \in [0, 1)$, $\epsilon > 0$; a PyTorch-alapértelmezések szerint gyakran $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$. A t-edik lépésben a g_t gradiensbecslésből:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

ahol g_t^2 elemenkénti négyzet, $m_0 = v_0 = 0$, alakjuk azonos θ alakjával. Mivel az inicializálás nulla, a kezdeti lépésekben m_t és v_t szisztematikusan kisebb, mint amit egy régóta futó mozgóátlagtól várnánk. A **bias correction** (elfogultság-korrekció) ezért:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Az Adam-frissítés:

$$\theta_{t+1} = \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

A nevező műveletei elemenkéntiek, ϵ hozzáadása a négyzetgyök után történik a PyTorch-konvenció szerint; például az Optax `scale_by_adam` függvényének `eps_root` argumentuma a gyökön belülre teszi a tagot, ezért pontos numerikus reprodukció előtt az adott API-t ellenőrizni kell. Jelölés: a szövegben ϵ, a kódban `eps`; a tanulási ráta jele η, a kódban `lr`. Az ϵ szerepe numerikus: ha $\hat{v}_t$ valamely koordinátája nulla közelében van, az osztás instabillá válna.

Az Adam tehát két különböző mennyiséget simít, és azok arányát használja. Az első momentum a gradiens irányának simítása, a második momentum a gradienskomponensek négyzetes nagyságának simítása; az arányuk az egyes koordinátákra eltérő effektív lépésméretet ad.

6.4.4 AdamW és a szétválasztott súlycsökkentés (decoupled weight decay)

A klasszikus L2-regularizálás a veszteséghez ad hozzá $(\lambda/2)\|\theta\|^2$ tagot, így a gradiens $g_t + \lambda\theta_t$ lesz. Az AdamW ehelyett a súlycsökkentést elkülönítve, a momentumoktól függetlenül alkalmazza:

$$\theta_{t+1} = \theta_t - \eta\Big(\lambda\theta_t + \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\Big),$$

ahol $\lambda \geq 0$ a weight decay (súlycsökkentés; a PyTorch `torch.optim.Adam` `weight_decay` argumentuma alapértelmezésben L2-tagot ad a gradienshez), $\hat{m}_t, \hat{v}_t$ továbbra is csak a g_t adatgradiensből számítódik. Ezt nevezzük **decoupled decay**-nek (szétválasztott súlycsökkentés): a $\lambda\theta_t$ tag nincs benne sem m_t-ben, sem v_t-ben, és nincs osztva az adaptív nevezővel. A gondolat forrása [P02]; a PyTorch `AdamW` osztálya ezt az alakot valósítja meg, míg a `torch.optim.Adam` `weight_decay` argumentuma a hagyományos L2-stílusú gradienshozzáadást jelenti. A PyTorch 2.15.0+cpu futtatott verziójában a `torch.optim.Adam` emellett `decoupled_weight_decay` kapcsolót is kínál; ennek alapértelmezése `False`, és `True` értékkel a szakasz skalárpéldáján ugyanazt az eredményt adja, mint az `AdamW` ($\theta_1 = 1{,}8$, $\theta_2 = 1{,}61$); ezt a könyv ellenőrzésekor futtatott kód igazolta. Az API verziófüggő, ezért a futtatott verziót rögzíteni kell [D06].

6.4.5 Két skalárfrissítés: miért más a decoupled decay, mint a naiv L2 Adam alatt?

Állítás (indoklással). Legyen $\theta \in \mathbb{R}$, adott $g_t \in \mathbb{R}$ adatgradiens és $\lambda > 0$. Az AdamW frissítése

$$\theta_{t+1} = \theta_t - \eta\lambda\theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

ahol m_t, v_t csak g_t-ből képződik. A naiv L2 Adam ezzel szemben a $\tilde{g}_t = g_t + \lambda\theta_t$ gradienst használja, ezért

$$\tilde{m}_t = \beta_1\tilde{m}_{t-1} + (1-\beta_1)\tilde{g}_t, \qquad \tilde{v}_t = \beta_2\tilde{v}_{t-1} + (1-\beta_2)\tilde{g}_t^2,$$

és a frissítés

$$\theta_{t+1} = \theta_t - \eta\frac{\tilde{m}_t/(1-\beta_1^t)}{\sqrt{\tilde{v}_t/(1-\beta_2^t)} + \epsilon}.$$

A két szabály csak akkor lenne azonos, ha a $\lambda\theta_t$ tag az adaptív normalizálásból kiemelhető lenne. Nem emelhető ki, mert egyrészt $\hat{m}_t$ és $\hat{v}_t$ a korábbi θ_s értékeket is tartalmazza, másrészt a L2-tag a nevezővel is osztódik. Ezért a L2 Adam alatti effektív decay nem $\eta\lambda\theta_t$, hanem egy momentumokkal simított, adaptívan skálázott mennyiség. Ez indoklás, nem zárt bizonyítás: azt mutatja meg, hogy a két frissítés algebrailag nem azonos alakú.

6.4.5.1 Kétlépéses numerikus példa

Legyen $\theta_0 = 2$, $g_1 = g_2 = 1$, $\eta = 0{,}1$, $\beta_1 = \beta_2 = 0{,}5$, $\lambda = 0{,}5$, $\epsilon = 0$. A $\epsilon = 0$ választás a kézi számolást egyszerűsíti; a PyTorch-alapértelmezés nem nulla, de ebben a példában a nevezők nullától távol vannak, így a különbség csak numerikus apróság.

AdamW, 1. lépés. $m_1 = 0{,}5$, $v_1 = 0{,}5$, bias korrekció: $1 - 0{,}5 = 0{,}5$, ezért $\hat{m}_1 = 1$, $\hat{v}_1 = 1$. Ekkor

$$\theta_1 = 2 - 0{,}1 \cdot 0{,}5 \cdot 2 - 0{,}1 \cdot \frac{1}{1+0} = 2 - 0{,}1 - 0{,}1 = 1{,}8.$$

AdamW, 2. lépés. $m_2 = 0{,}5 \cdot 0{,}5 + 0{,}5 \cdot 1 = 0{,}75$, $v_2 = 0{,}75$, bias korrekció: $1 - 0{,}5^2 = 0{,}75$, ezért $\hat{m}_2 = \hat{v}_2 = 1$. Így

$$\theta_2 = 1{,}8 - 0{,}1 \cdot 0{,}5 \cdot 1{,}8 - 0{,}1 = 1{,}8 - 0{,}09 - 0{,}1 = 1{,}61.$$

Naiv L2 Adam, 1. lépés. $\tilde{g}_1 = 1 + 0{,}5 \cdot 2 = 2$. $m_1 = 1$, $v_1 = 2$, bias korrekcióval $\hat{m}_1 = 2$, $\hat{v}_1 = 4$, $\sqrt{\hat{v}_1} = 2$. A frissítés:

$$\theta_1 = 2 - 0{,}1 \cdot \frac{2}{2+0} = 2 - 0{,}1 = 1{,}9.$$

Naiv L2 Adam, 2. lépés. $\tilde{g}_2 = 1 + 0{,}5 \cdot 1{,}9 = 1{,}95$. Ekkor $m_2 = 0{,}5 \cdot 1 + 0{,}5 \cdot 1{,}95 = 1{,}475$, $v_2 = 0{,}5 \cdot 2 + 0{,}5 \cdot 1{,}95^2 = 2{,}90125$. A bias korrekció 0,75: $\hat{m}_2 \approx 1{,}9667$, $\hat{v}_2 \approx 3{,}8683$, $\sqrt{\hat{v}_2} \approx 1{,}9668$. A hányados majdnem 1, így

$$\theta_2 \approx 1{,}9 - 0{,}0999996 \approx 1{,}800007.$$

A két lépés után AdamW $\theta_2 = 1{,}61$, a L2 Adam $\theta_2 \approx 1{,}800007$. A különbség oka látható: az AdamW-ben a decay tag $\eta\lambda\theta_t$ külön levonódik, a L2 Adam esetén viszont a decay bekerül a momentumokba, és az adaptív normalizálás részben elfedi a nagyságát. A példa nem állítja, hogy az egyik mindig jobb; csak azt mutatja, hogy a két frissítési szabály matematikailag nem azonos.

6.4.6 Labor: két optimizerlépés kézzel és NumPy-függvénnyel

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Munkakönyvtár: `lab/n024/`. A kód csak CPU-t és NumPy-t használ; `dtype=np.float64` a pontos összehasonlításhoz. A számítás determinisztikus, véletlen komponenst nem használ, ezért seed nem szükséges; a gradiensértékeket explicit módon adjuk meg.

```
#!/usr/bin/env python3
# lab/n024/optim_steps.py
import numpy as np

def adamw_step_scalar(theta, m, v, g, lr, beta1, beta2, lam, t, eps):
    m = beta1 * m + (1.0 - beta1) * g
    v = beta2 * v + (1.0 - beta2) * g * g
    m_hat = m / (1.0 - beta1 ** t)
    v_hat = v / (1.0 - beta2 ** t)
```

```
    theta_new = (
        theta
        - lr * lam * theta
        - lr * m_hat / (np.sqrt(v_hat) + eps)
    )
    return theta_new, m, v

def adam_l2_step_scalar(theta, m, v, g, lr, beta1, beta2, lam, t, eps):
    grad = g + lam * theta
    m = beta1 * m + (1.0 - beta1) * grad
    v = beta2 * v + (1.0 - beta2) * grad * grad
    m_hat = m / (1.0 - beta1 ** t)
    v_hat = v / (1.0 - beta2 ** t)
    theta_new = theta - lr * m_hat / (np.sqrt(v_hat) + eps)
    return theta_new, m, v

def main():
    theta0 = np.float64(2.0)
    g = np.float64(1.0)
    lr = np.float64(0.1)
    beta1 = np.float64(0.5)
    beta2 = np.float64(0.5)
    lam = np.float64(0.5)
    eps = np.float64(0.0)

    # AdamW két lépés
    theta = theta0
    m = np.float64(0.0)
    v = np.float64(0.0)
    for t in (1, 2):
        theta, m, v = adamw_step_scalar(
            theta, m, v, g, lr, beta1, beta2, lam, t, eps
        )
        print(f"AdamW step {t}: theta={theta:.12f}, m={m:.12f}, v={v:.12f}")
        expected = {1: 1.8, 2: 1.61}[t]
        assert np.isclose(theta, expected, rtol=0.0, atol=1e-12)

    # Naiv L2 Adam két lépés
    theta = theta0
    m = np.float64(0.0)
    v = np.float64(0.0)
    for t in (1, 2):
        theta, m, v = adam_l2_step_scalar(
            theta, m, v, g, lr, beta1, beta2, lam, t, eps
        )
        print(f"L2 Adam step {t}: theta={theta:.12f}, m={m:.12f}, v={v:.12f}")
        expected = {1: 1.9, 2: 1.800007181071}[t]
        assert np.isclose(theta, expected, rtol=0.0, atol=1e-12)

if __name__ == "__main__":
    main()
```

Futtatási parancs a `lab/n024/` könyvtárból:

```
python optim_steps.py
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges):

```
AdamW step 1: theta=1.800000000000, m=0.500000000000, v=0.500000000000
AdamW step 2: theta=1.610000000000, m=0.750000000000, v=0.750000000000
L2 Adam step 1: theta=1.900000000000, m=1.000000000000, v=2.000000000000
L2 Adam step 2: theta=1.800007181071, m=1.475000000000, v=2.901250000000
```

A kimenet a kód futtatásával kapott érték; a beépített `np.isclose`-ellenőrzések (`rtol=0`, `atol=1e-12`) a kézi levezetéssel való egyezést tesztelik.

6.4.7 Feladatok

6.4.7.1 F1 fogalmi, nehézség 1

Magyarázza el, mi a különbség az Adam első és második momentuma között, és miért szükséges a bias correction. Térjen ki arra, hogy $m_0 = v_0 = 0$ miért torzítja lefelé az első lépéseket. Elvárt eredmény: legfeljebb fél oldal saját magyarázat, benne a $1 - \beta^t$ tag szerepe.

6.4.7.2 F2 levezetési, nehézség 2

Igazolja, hogy ha egy d-dimenziós feladatban az első t gradiens mindegyike ugyanaz a $c \in \mathbb{R}^d$ vektor, akkor az Adam bias-korrigált első momentuma $\hat{m}_t = c$, és a második momentum $\hat{v}_t = c^2$, ahol c^2 elemenkénti négyzet. Mondja ki, hogy ez az állítás milyen feltétel mellett érvényes, és mutasson ellenpéldát arra, amikor a gradiensek nem egyenlők, de a bias correction mégsem adja vissza az utolsó gradienst.

6.4.7.3 F3 implementációs, nehézség 1

A `lab/n024/optim_steps.py` mellé írjon egy `sgd_momentum_step(theta, buf, g, lr, beta)` függvényt a PyTorch momentumos SGD képlete alapján. Bemenet: $\theta_0 = 0{,}5$, $g_1 = g_2 = 0{,}2$, $\eta = 0{,}1$, $\beta = 0{,}9$, `dtype=np.float64`. Adja meg θ_2-t és ellenőrizze, hogy $\theta_2 = 0{,}442$ legyen. Elvárt beadandó eredmény: teljes függvénykód, futtatási parancs és a kapott `theta` érték.

6.4.7.4 F4 hibakeresési, nehézség 2

Az alábbi Adam-implementáció hibás:

```
def rossz_adam(theta, m, v, g, t, lr, beta1, beta2, eps):
    m = beta1 * m + (1-beta1) * g
    v = beta2 * v + (1-beta2) * g * g
    m_hat = m / (1 - beta1 ** t)
    theta = theta - lr * m_hat / (np.sqrt(v) + eps)
    return theta, m, v
```

Azonosítsa a hibát, magyarázza el a következményét a második lépésben, és javítsa a kódot. Igazolja a javítást a szakasz kézi példáján, a következő értékekkel: $\theta_0 = 2$, $g_1 = g_2 = 1$, $m_0 = v_0 = 0$, $\beta_1 = \beta_2 = 0{,}5$, $\eta = 0{,}1$, $\epsilon = 0$; a λ súlycsökkentési együtthatót itt nem kell használni. Elvárt eredmény: hibaleírás, javított kód és a javított változat θ_1, θ_2 értéke.

6.4.7.5 F5 kísérleti/transzfer, nehézség 3

Tervezzen egy kis CPU-s protokollt, amely ugyanazon a rögzített modellen és adathalmazon hasonlítja össze az SGD-t, a momentumos SGD-t, az Adamet és az AdamW-t. Határozza meg: a modellt, a veszteséget, a splitet, a metrikát, a tanítási költségkeretet, a seedkezelést és a hiperparaméter-keresést. Az értékelés végén mondja meg, milyen következtetést nem szabad levonni egyetlen futtatásból. Elvárt eredmény: protokollterv és a lehetséges érvényességi korlátok indoklása, futás nélkül.

6.4.8 Megoldások és indoklás

6.4.8.1 F1 megoldás

Az m_t a gradiensvektor mozgóátlaga, ezért elsősorban az irányt simítja; a v_t a gradiens elemenkénti négyzetének mozgóátlaga, ezért a négyzetes nagyságot becsli. A nevezőbeli $\sqrt{\hat{v}_t}$ koordinátánként eltérő léptéket ad. Az inicializálás $m_0 = v_0 = 0$: az első lépésben $m_1 = (1-\beta_1)g_1$, tehát a g_1 érték helyett annak csak $(1-\beta_1)$-szeresét tároljuk. A t-edik lépésig a mozgóátlag súlyainak összege $1-\beta_1^t$, ezért osztunk ezzel. Hasonlóan v_t esetén $1-\beta_2^t$. A bias correction tehát nem a gradiens torzítását javítja, hanem a nulla kezdőállapotból adódó skálázást.

6.4.8.2 F2 megoldás

Ha $g_s = c$ minden $s = 1, \dots, t$ esetén, akkor a rekurziót kibontva:

$$m_t = (1-\beta_1)\sum_{s=1}^{t}\beta_1^{t-s}c = (1-\beta_1)\frac{1-\beta_1^t}{1-\beta_1}c = (1-\beta_1^t)c.$$

Ezért $\hat{m}_t = m_t/(1-\beta_1^t) = c$. Ugyanígy $v_t = (1-\beta_2^t)c^2$, tehát $\hat{v}_t = c^2$. A feltétel az, hogy a gradiensek determinisztikusan azonosak legyenek. Ha például $g_1 = 1$ és $g_2 = -1$ skalár esetén $t = 2$ és $\beta_1 = 0{,}5$, akkor $m_1 = 0{,}5$ és $m_2 = 0{,}5 \cdot 0{,}5 + 0{,}5 \cdot (-1) = -0{,}25$, a bias-korrigált érték pedig $\hat{m}_2 = -0{,}25/(1-0{,}25) = -1/3$. Az utolsó gradiens -1, tehát $\hat{m}_2 = -1/3 \neq g_2$; a bias correction nem reprodukálja az időben változó gradienst.

6.4.8.3 F3 megoldás

```
import numpy as np

def sgd_momentum_step(theta, buf, g, lr, beta):
    buf = beta * buf + g
    theta = theta - lr * buf
    return theta, buf

theta = np.float64(0.5)
buf = np.float64(0.0)
g = np.float64(0.2)
lr = np.float64(0.1)
beta = np.float64(0.9)

for _ in range(2):
    theta, buf = sgd_momentum_step(theta, buf, g, lr, beta)

print(f"theta={theta:.12f}")
assert np.isclose(theta, 0.442, rtol=0.0, atol=1e-12)
```

Ellenőrzés: 1. lépésben $buf = 0{,}2$, $\theta = 0{,}48$; 2. lépésben $buf = 0{,}38$, $\theta = 0{,}48 - 0{,}038 = 0{,}442$. Az F3 ugyanabban a `lab/n024/` munkakönyvtárban használható, mint az `optim_steps.py`; a momentumos SGD lépése ugyanazt az állapotváltozós szerkezetet követi, de a `buf` frissítése $\beta\, buf + g$, a θ frissítése pedig $\theta - \eta\, buf$.

6.4.8.4 F4 megoldás

A kód a második momentumot nem korrigálja a $1 - \beta_2^t$ tényezővel. Emiatt $\hat{v}_t$ alulbecsült marad, ami túl kicsi nevezőt és túl nagy adaptív lépést okozhat, különösen a kezdeti lépésekben. A javítás:

```
v_hat = v / (1 - beta2 ** t)
theta = theta - lr * m_hat / (np.sqrt(v_hat) + eps)
```

A szakasz példáján $\beta_1 = \beta_2 = 0{,}5$, $g_1 = g_2 = 1$ esetén $v_1 = 0{,}5$, a hibás kód $\sqrt{v_1} \approx 0{,}707$, a javított $\sqrt{\hat{v}_1} = 1$ nevezőt ad. A javított változatban $\theta_1 = 2 - 0{,}1 = 1{,}9$, majd $\theta_2 = 1{,}9 - 0{,}1 = 1{,}8$. A hibás kód első lépése $\theta_1 = 2 - 0{,}1/\sqrt{0{,}5} \approx 1{,}85858$, a második pedig $\theta_2 \approx 1{,}74311$, tehát a hiba már két lépés után láthatóan eltéríti a pályát.

6.4.8.5 F5 megoldásminta

Modell: logisztikus regresszió (lineáris osztályozó, amely a kimenetét sigmoiddal valószínűséggé alakítja) a függelék A. szakaszában leírt GlyphLab-példa 24×24 képének előre kivett hisztogram-jellemzőin, vagy egy kis MLP. A GlyphLab adathalmaz 120/30/30 jelenetből és 240/60/60 képből áll; a `scene_id` a jelenet azonosítója, a `view_id` a nézet azonosítója. Veszteség: cross-entropy (a logisztikus regresszió negatív log-likelihood vesztesége, amelyet a 7.1. részletesen tárgyal). Adat: a GlyphLab `data/` könyvtár train/val/test splitje, csoportosítás `scene_id` szerint. Metrika: ROC-AUC (rangalapú metrika, amely a küszöbtől függetlenül méri az elválasztást) vagy log loss (valószínűségi hiba) a validációs halmazon, a végső jelentés a test halmazon. Költségkeret: legfeljebb 50 epoch (a tanítóhalmaz 50 teljes bejárása) CPU-n, rögzített batchméret. Seedkezelés: minden optimizerhez legalább három különböző seed, és a kezdeti súlyok azonos seedű inicializálása. A döntési szabály előre rögzítendő: a validációs metrikákat azonos seedek szerint párosítjuk, és egy optimizert csak akkor tekintünk jobbnak, ha a párosított különbségek átlaga a seed-szórás mellett is egy irányba mutat; három seed mellett ez gyenge bizonyíték, ezért a következtetést a korlátok között kell megfogalmazni. Keresés: minden optimizer tanulási rátájára kis logaritmikus rács, a validációs metrika alapján választva. Nem szabad levonni, hogy egyetlen legjobb seed alapján az egyik optimizer általánosan jobb; a seedek közötti szórás az algoritmikus bizonytalanság része, és a kis, játékszerű (toy) feladat nem reprezentálja a valós CNN-feladatokat.

6.4.9 Tipikus tévedés ellenpéldával

Gyakori hiba azt hinni, hogy egy optimizer neve önmagában garantál jobb pontosságot vagy konvergenciát. Tekintsük az $f(\theta) = \frac{1}{2}\theta^2$ függvényt, gradiens $g_t = \theta_t$, $\theta_0 = 10$. Sima SGD $\eta = 0{,}1$ esetén $\theta_t = 10 \cdot 0{,}9^t$, tehát monoton csökken és 100 lépés után $\theta_{100} = 10 \cdot 0{,}9^{100} \approx 2{,}66 \cdot 10^{-4}$. Ha Adamet használunk $\beta_1 = \beta_2 = 0$, $\eta = 0{,}1$, $\epsilon = 0$ mellett, akkor az Adam sign-SGD-vé redukálódik. Valóban, $\beta_1 = 0$ miatt $m_t = g_t$, $\beta_2 = 0$ miatt $v_t = g_t^2$, és $t \geq 1$ esetén $1 - 0^t = 1$, ezért $\hat{m}_t = g_t$, $\sqrt{\hat{v}_t} = |g_t|$; így a frissítés $-\eta\, g_t/|g_t| = -\eta \cdot \mathrm{sign}(g_t)$, minden lépés 0,1 nagyságú. Az $|\theta_t| \leq 0{,}1$ hibaküszöbig a sima SGD-nek $10 \cdot 0{,}9^t \leq 0{,}1$ miatt 44 lépés kell, az ilyen Adamnek 99, tehát lényegesen lassabb. A nulla közelében a példa nem folytatható: $\theta = 0$-nál $g = 0$, $m = v = 0$, és

$\epsilon = 0$ mellett a frissítés 0/0, definiálatlan. Ezért itt megállási szabály ($|\theta| \leq$ küszöb) vagy $\epsilon > 0$ kell, és a lassúságot csak a megadott hibaküszöbig állítjuk. Ez a mesterséges ellenpélda nem bizonyítja, hogy az SGD mindig jobb; azt mutatja, hogy az Adam normalizált lépése bizonyos skálázású feladatoknál elveszítheti a gradiens nagyságából származó előnyt.

Lépés t	SGD $\theta_t = 10 \cdot 0{,}9^t$	Adam $\theta_t = 10 - 0{,}1t$
0	10	10
10	3,49	9,0
20	1,22	8,0
30	0,424	7,0
40	0,148	6,0
44	0,0970	5,6
50	0,0515	5,0
99	$2{,}66 \cdot 10^{-4}$	0,1

A táblázat VÁRT ÉRTÉK (levezetésből, nem mért): a két zárt képletből számolt érték, nem futtatott mérés.

6.4.10 Önellenőrzés és visszatekintés

Önellenőrzés: írja fel az AdamW frissítést skalár esetben, majd ellenőrizze, hogy a decay tag valóban kívül van-e a nevezőn. Ha a frissítésben $\lambda\theta_t$ az adaptív nevezőn belül szerepel, akkor az L2 Adam, nem AdamW. Visszatekintés a 6.3. Sztochasztikus és minibatch gradiens alfejezetből: a minibatch-gradiens torzítatlan becslése a momentum és az Adam állapotváltozóiba kerül, így az ott levezetett variancia csökkenése a lépés zaját, nem feltétlenül az optimizer teljes pályájának stabilizálását írja le. A 2.4. Kis tudományos szoftverprojekt alfejezetből pedig az következik, hogy az optimizer kódját ugyanúgy kis, rögzített tesztbemeneteken (fixture: a teszt bemenetét és elvárt kimenetét együtt rögzítő teszteset) kell tesztelni, mint bármely más modult.

6.4.11 Rövid összefoglaló

A momentumos SGD a gradiens rekurzív mozgóátlagát tárolja, és ezzel csökkenti a lépés irányának fluktuációját. Az Adam két állapotot tart: az első momentum a gradiens irányát, a második momentum a koordinátánkénti négyzetes nagyságot becsli; a bias correction a nulla kezdőállapot miatti kezdeti lefelé torzítást korrigálja. Az AdamW a súlycsökkentést elkülönítve alkalmazza: a $\lambda\theta_t$ tag nem kerül a momentumokba, ezért matematikailag nem azonos a naiv L2 Adam-mel. A kézi kétlépéses skalárpélda numerikusan is mutatja az eltérést. Az optimizer neve azonban nem garancia, a választást mindig a feladaton és a seedismétléseken kell ellenőrizni.

6.4.12 Továbblépés

A 7.1. Entrópia, cross-entropy és KL alfejezetben az entrópia, a cross-entropy és a KL-divergencia következik. Ott a veszteségfüggvény pontos valószínűségi értelmezését építjük fel; az itt tárgyalt frissítési szabályokat ezután konkrét célfüggvények gradiensére alkalmazzuk.

6.4.13 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. 7. fejezet (Continuous Optimization). https://mml-book.github.io/. Hozzáférés: 2026-10-30.
- [P02] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. Előzetes változat: arXiv:1711.05101 (2017). https://arxiv.org/abs/1711.05101. Hozzáférés: 2026-10-30.
- [D06] PyTorch contributors (2026). *torch.optim*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/optim.html. Hozzáférés: 2026-10-30.

7 Információelmélet, veszteségek és stabil valószínűségi predikció

7.1 Entrópia, cross-entropy és KL

7.1.1 Cél, előismeret és induló kérdés

Ez az alfejezet a diszkrét valószínűségi vektorok három alapvető információelméleti mennyiségét, az entrópiát, a cross-entropy-t és a Kullback–Leibler-divergenciát (KL) vezeti be. A cél, hogy a hallgató pontosan számolni tudjon, értse a support (tartóhalmaz) szerepét, és felkészüljön a következő alfejezetben tárgyalt, logitokból (normalizálatlan nyers pontszámokból) számolt osztályozási veszteségre.

Előismeret a 3.1. Vektor, norma és skalárszorzat vektor- és távolságfogalma, az 5.4. Függő képek, seedek és resampling statisztikai egységek kezelése és a 6.1. Objektív, reguláris tag és gradienslépés batch-átlag/reduction konvenciója. A 6.4. Momentum és adaptív optimizálók optimalizálóból tudjuk, hogy a gradienslépés célfüggvényt csökkent; itt azt alapozzuk meg, hogy a veszteség mit mér valószínűségi értelemben.

Induló kérdés. Egy háromosztályos feladatban az igaz címke one-hot eloszlása (egyetlen 1-est, a többi helyen 0-t tartalmazó eloszlás) $p = (1, 0, 0)$. Két modell $q_A = (0{,}9; 0{,}05; 0{,}05)$ és $q_B = (0{,}6; 0{,}3; 0{,}1)$ eloszlást ad. q_A nyilván jobb, de mennyivel? A természetes logaritmus egységében (nat) $-\ln 0{,}9 \approx 0{,}105$, míg $-\ln 0{,}6 \approx 0{,}511$, a két cross-entropy különbsége tehát $-\ln 0{,}6 + \ln 0{,}9 \approx 0{,}405$ nat. A q_B rosszabb, mint q_A, és a különbség nagyobb, mint a nyers $0{,}9 - 0{,}6 = 0{,}3$ sugallja.

7.1.2 Definíciók és intuíció

Diszkrét valószínűségi vektor. Legyen $K \geq 1$ egész. A $p = (p_1, \dots, p_K)$ vektor valószínűségi vektor, ha

$$p_i \geq 0 \ (1 \leq i \leq K), \qquad \sum_{i=1}^{K} p_i = 1.$$

A továbbiakban egyetlen valószínűségi vektorral dolgozunk; a több példás, mátrixos esetre a 7.2. Softmax és többosztályos cross-entropy alfejezet tér vissza.

Support. A p valószínűségi vektor supportja (tartóhalmaza) azon indexek halmaza, ahol pozitív a tömeg:

$$\operatorname{supp}(p) = \{\, i \in \{1, \dots, K\} : p_i > 0 \,\}.$$

Entrópia. A p valószínűségi vektor entrópiája

$$H(p) = -\sum_{i=1}^{K} p_i \ln p_i,$$

ahol ln a természetes logaritmus (natural logarithm), az egység pedig nat. A $0 \ln 0$ tagot a 0-val azonosítjuk, mert $\lim_{x \downarrow 0} x \ln x = 0$. Az entrópia a bizonytalanság mértéke: $H = 0$ pontosan akkor, ha p egyetlen kimenetelre koncentrálódik, a maximum pedig az egyenletes eloszlásnál áll elő, értéke $\ln K$. Ez utóbbit a felbontási tétel után, a KL-divergencia nemnegativitásával igazoljuk.

Cross-entropy. Két valószínűségi vektor, p és q esetén a p célhoz és q predikcióhoz tartozó cross-entropy

$$H(p, q) = -\sum_{i=1}^{K} p_i \ln q_i,$$

a $0 \ln 0$ konvencióval a $p_i = 0$ tagokra. Ha valamely i-re $p_i > 0$ és $q_i = 0$, akkor $H(p, q) = +\infty$; ezt a support-csapdát (a cél tartóhalmazán kívül eső, nulla predikciós valószínűséget) később külön elemezzük. Az első argumentum a megfigyelt vagy cél-eloszlás, a második a modell által adott eloszlás.

KL-divergencia. Legyenek p és q az előző bekezdés szerinti valószínűségi vektorok.

$$D_{KL}(p \| q) = \sum_{i=1}^{K} p_i \ln \frac{p_i}{q_i} = \sum_{i=1}^{K} p_i \ln p_i - \sum_{i=1}^{K} p_i \ln q_i,$$

ahol a $p_i = 0$ tagok nullák. A KL a p és q eloszlások közötti, irányított információnövekmény; nem szimmetrikus általában, és nem metrikus távolság. A mértékegysége szintén nat, ha természetes logaritmust használunk.

7.1.3 A fő levezetés: a cross-entropy felbontása

Tétel (a cross-entropy entrópia–KL felbontása). Legyenek $p, q \in [0, 1]^K$ valószínűségi vektorok. A fenti konvenciókkal

$$H(p, q) = H(p) + D_{KL}(p \| q).$$

$H(p)$ mindig véges; $H(p, q)$ és $D_{KL}(p \| q)$ pontosan akkor véges, ha $\operatorname{supp}(p) \subseteq \operatorname{supp}(q)$.

Bizonyítás. A definíciókat behelyettesítve:

Legyen $S = \{i : p_i > 0\}$ a p tartója. A $p_i = 0$ tagok a $0 \ln 0 = 0$ konvenció miatt (és mert $0 \cdot \ln(0/q_i) = 0$) kihagyhatók, ezért minden összeget S-re korlátozunk; az S-en kívüli indexeken sem $\ln p_i$-t, sem q_i/p_i-t nem írunk fel. Ha $\operatorname{supp}(p) \subseteq \operatorname{supp}(q)$, azaz $q_i > 0$ minden $i \in S$-re, akkor

$$H(p, q) = -\sum_{i \in S} p_i \ln q_i = -\sum_{i \in S} p_i (\ln p_i + \ln \tfrac{q_i}{p_i}) = -\sum_{i \in S} p_i \ln p_i + \sum_{i \in S} p_i \ln \frac{p_i}{q_i} = H(p) + D_{KL}(p \| q).$$

A második egyenlőségnél az S-en belül írtuk át $\ln q_i = \ln p_i + \ln(q_i/p_i)$-t, ami $p_i > 0$ miatt értelmes. A véges-értékű feltételhez: $H(p)$ mindig véges, mert véges sok $p_i \ln p_i$ tag összege. A cross-entropy és a KL pontosan akkor véges, ha nincs olyan i, amelyre $p_i > 0$ és $q_i = 0$, vagyis

$\mathrm{supp}(p) \subseteq \mathrm{supp}(q)$. Ha a tartalmazás sérül, akkor $H(p,q) = +\infty$ és $D_{KL}(p\|q) = +\infty$, miközben $H(p) < \infty$; a $H(p,q) = H(p) + D_{KL}(p\|q)$ azonosság ilyenkor csak a kiterjesztett valós számokon ($+\infty = H(p) + \infty$) áll fenn, a véges esetre vonatkozó algebrai átalakítás nem használható. A továbbiakban a véges esetet feltételezzük. $\square$

A nemnegativitás külön is indokolható a $\ln x \le x - 1$ elemi egyenlőtlenségből, amely minden $x > 0$-ra igaz, egyenlőséggel csak $x = 1$-ben. Ha a support feltétel teljesül, minden i-re $x_i = q_i/p_i > 0$, ezért

$$D_{KL}(p\|q) = -\sum_{i:p_i>0} p_i \ln \frac{q_i}{p_i} \ge -\sum_{i:p_i>0} p_i \left(\frac{q_i}{p_i} - 1\right) = \sum_{i:p_i>0} (p_i - q_i) = 1 - \sum_{i:p_i>0} q_i \;\ge\; 0.$$

Az utolsó lépésben $\sum_{i:p_i>0} p_i = 1$, a q tartón kívüli része pedig nemnegatív, ezért $\sum_{i:p_i>0} q_i \le 1$. A $\mathrm{supp}(p) \subseteq \mathrm{supp}(q)$ tartalmazás nem jelenti, hogy q a p tartóján kívül nulla, ezért ez az összeg általában kisebb 1-nél: például $p = (1,0)$, $q = (\tfrac12, \tfrac12)$ mellett $\sum_{i:p_i>0}(p_i - q_i) = \tfrac12$, míg $\sum_{i=1}^{K}(p_i - q_i) = 0$.

Ebből az is következik, hogy $H(p,q) \ge H(p)$ minden support-feltételt teljesítő q-ra. Egyenlőség, $D_{KL}(p\|q) = 0$, pontosan akkor áll, ha a lánc mindkét egyenlőtlensége egyenlőség: a $\ln x \le x - 1$ egyenlőtlenség minden $p_i > 0$ tagra, vagyis $q_i/p_i = 1$, tehát $q_i = p_i$ a p tartóján; és $\sum_{i:p_i>0} q_i = 1$, vagyis q a tartón kívül nulla. A kettő együtt $p = q$, és fordítva $p = q$ esetén nyilván $D_{KL} = 0$.

7.1.4 Kézi számpélda

Legyen $K = 3$, $p = (0{,}2;\ 0{,}5;\ 0{,}3)$ és $q = (0{,}4;\ 0{,}4;\ 0{,}2)$. Először az entrópia:

$$H(p) = -0{,}2\ln 0{,}2 - 0{,}5\ln 0{,}5 - 0{,}3\ln 0{,}3 \approx 0{,}321888 + 0{,}346574 + 0{,}361192 = 1{,}029653.$$

A cross-entropy:

$$H(p,q) = -0{,}2\ln 0{,}4 - 0{,}5\ln 0{,}4 - 0{,}3\ln 0{,}2 \approx 0{,}183258 + 0{,}458145 + 0{,}482831 = 1{,}124234.$$

A KL a tételből $1{,}124234 - 1{,}029653 = 0{,}094581$. Közvetlenül:

$$D_{KL}(p\|q) = 0{,}2\ln\frac{0{,}2}{0{,}4} + 0{,}5\ln\frac{0{,}5}{0{,}4} + 0{,}3\ln\frac{0{,}3}{0{,}2} \approx -0{,}138629 + 0{,}111572 + 0{,}121640 = 0{,}094583,$$

ami a kerekítési hibán belül egyezik a felbontásból kapott értékkel. A két megjelenített érték azért tér el 0,000002-del, mert a közbenső tagokat hat tizedesjegyre kerekítettük: a pontos érték mindkét úton 0,0945819 (a felbontásban $1{,}1242349 - 1{,}0296530$, közvetlenül $-0{,}1386294 + 0{,}1115718 + 0{,}1216395$), a kerekített tagok összege viszont 0,094583.

7.1.5 Labor: nemszimmetria és support-csapda

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Az alábbi minimális Python-munkamenet NumPy-ben számol. Nincs véletlen mintavétel, ezért seed nem szükséges; a determinisztikus képletek és a rögzített tömbök adják a reprodukálhatóságot. A dtype `float64`, a device pedig CPU, mert a NumPy ebben a laborban nem használ gyorsítóabsztrakciót.

```
# lab/n025_entropy_lab.py
import numpy as np

DTYPE = np.float64
# DEVICE = "CPU" (NumPy, nincs külön eszközkezelés)

def _xlogy(a, b):
    """a*log(b), a=0 esetén 0; a ki nem választott ágat nem értékeli ki."""
    out = np.zeros(np.broadcast(a, b).shape, dtype=DTYPE)
    a, b = np.broadcast_arrays(a, b)
    m = a > 0
    out[m] = a[m] * np.log(b[m])
    return out

def entropy(p):
    p = np.asarray(p, dtype=DTYPE)
    return -np.sum(_xlogy(p, p))

def cross_entropy(p, q):
    p = np.asarray(p, dtype=DTYPE)
    q = np.asarray(q, dtype=DTYPE)
    return -np.sum(_xlogy(p, q))

def kl_divergence(p, q):
    p = np.asarray(p, dtype=DTYPE)
    q = np.asarray(q, dtype=DTYPE)
    if np.any((p > 0) & (q == 0)):
        return np.inf
    m = p > 0
    return np.sum(p[m] * np.log(p[m] / q[m]))

# 1) Nemszimmetria
p = np.array([0.2, 0.8], dtype=DTYPE)
q = np.array([0.7, 0.3], dtype=DTYPE)
kl_pq = kl_divergence(p, q)
kl_qp = kl_divergence(q, p)
print(f"KL(p||q) = {kl_pq:.9f}")
print(f"KL(q||p) = {kl_qp:.9f}")
print(f"különbség = {abs(kl_pq - kl_qp):.9f}")

# 2) Support-csapda: supp(p)  supp(q), de nem fordítva
p = np.array([0.5, 0.0, 0.5], dtype=DTYPE)
q = np.array([0.4, 0.3, 0.3], dtype=DTYPE)
print(f"KL(p||q) = {kl_divergence(p, q):.9f}")
print(f"KL(q||p) = {kl_divergence(q, p)}")

# Ellenőrzés a tételre
print(f"H(p,q)-H(p) = {cross_entropy(p, q) - entropy(p):.9f}")

# Ellenőrzés: nemszimmetria, support-csapda és a felbontási tétel
assert not np.isclose(kl_pq, kl_qp)
assert np.isinf(kl_divergence(q, p))
```

```
assert np.isclose(cross_entropy(p, q) - entropy(p), kl_divergence(p, q),
                  atol=1e-12, rtol=1e-12)
print("ellenorzes: nemszimmetria, support-csapda es felbontas rendben")
```

Futtatás a `lab` munkakönyvtárból:

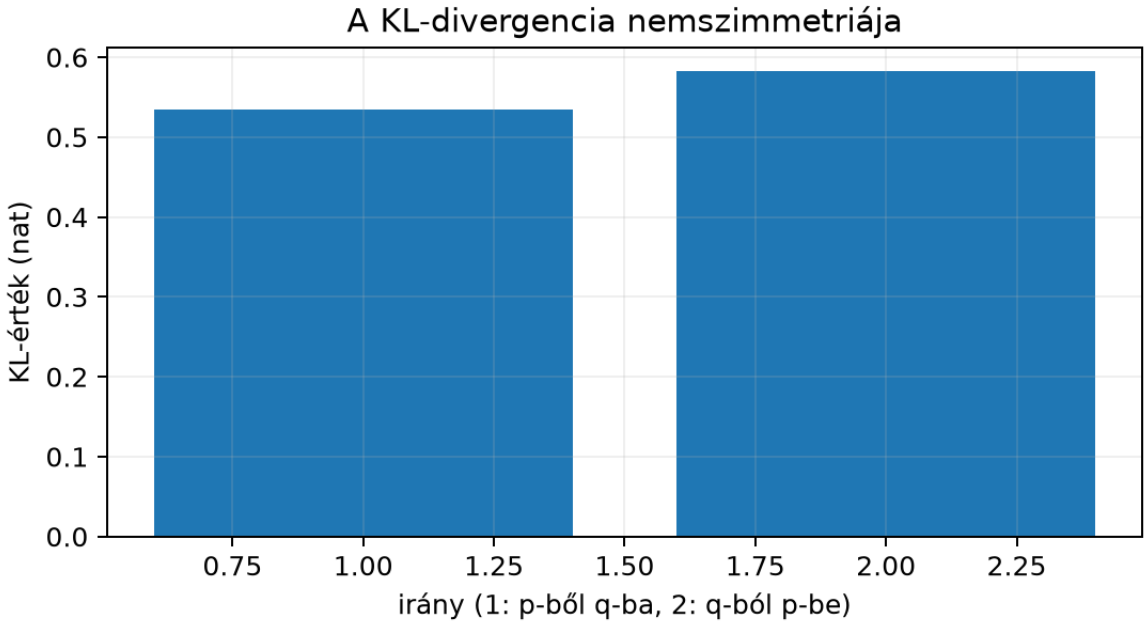
```
python n025_entropy_lab.py
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges):

```
KL(p||q) = 0.534110809
KL(q||p) = 0.582685302
különbség = 0.048574493
KL(p||q) = 0.366984588
KL(q||p) = inf
H(p,q)-H(p) = 0.366984588
ellenorzes: nemszimmetria, support-csapda es felbontas rendben
```

Értelmezés. Az első esetben a két irányú KL érték különbözik, tehát a KL nem szimmetrikus. A második esetben $\mathrm{supp}(p) = \{1,3\}$, míg $\mathrm{supp}(q) = \{1,2,3\}$, ezért a p felőli irány véges, a q felőli irány viszont végtelen, mert $q_2 = 0{,}3 > 0$, de $p_2 = 0$. A harmadik sor a levezetett tételt numerikusan ellenőrzi: a cross-entropy és az entrópia különbsége egyenlő a KL-lel.

Eset	$\mathrm{supp}(p)$	$\mathrm{supp}(q)$	$D_{KL}(p\|q)$	$D_{KL}(q\|p)$
1. nemszimmetria	$\{1,2\}$	$\{1,2\}$	0,5341	0,5827
2. support-csapda	$\{1,3\}$	$\{1,2,3\}$	0,3670	$+\infty$



7.1. ábra: A KL-divergencia nemszimmetriája

Az 1. laboreset értékei; a support-csapdás 2. esetben a q-ból p-be irány végtelen, ezért nem ábrázolható.

7.1.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a support fogalmát, és magyarázza el, miért éppen a $\operatorname{supp}(p) \subseteq \operatorname{supp}(q)$ feltétel garantálja a $D_{KL}(p\|q)$ végességét. Adjon $K = 2$-re olyan p, q párt, amelyre az egyik irány véges, a másik végtelen.

F2 (levezetési, nehézség 1). Legyen $p = (0{,}25;\ 0{,}75)$ és $q = (0{,}5;\ 0{,}5)$. Számolja ki négy tizedesjegyre $H(p)$, $H(q)$, $H(p, q)$, $H(q, p)$, $D_{KL}(p\|q)$ és $D_{KL}(q\|p)$ értékét, majd ellenőrizze a felbontási tételt mindkét irányban.

F3 (implementációs, nehézség 2). Írjon Python-függvényt, amely két K hosszú tömböt fogad, ellenőrzi, hogy elemeik nemnegatívak és összegük 1, majd visszaadja a három mennyiséget. A függvény jelezze a support-csapdát: ha $p_i > 0$ és $q_i = 0$, a cross-entropy és a KL legyen `inf`. Tesztelje $p = (0{,}5;\ 0;\ 0{,}5)$, $q = (0{,}4;\ 0{,}3;\ 0{,}3)$ bemenettel.

F4 (hibakeresési, nehézség 2). A következő kód a cross-entropy kiszámítására készült:

```
def ce(p, q):
    return -np.mean(p * np.log(q))
```

Magyarázza el, miért hibás ez a függvény az $p = (0;\ 0{,}5;\ 0{,}5)$, $q = (0{,}5;\ 0{,}5;\ 0)$ bemenetre, és lokalizálja a hibák okát, következményét és javítását. Ellenőrizze a javított változatot a megadott bemeneten.

F5 (kísérleti/transzfer, nehézség 3). A kísérletet a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adathalmazon (procedurálisan generált, háromosztályos grayscale képek, jelenetenként két nézettel) tervezi meg. Egy kalibrációs kísérletben (a kalibráció itt azt jelenti, hogy a modell valószínűségei megfelelnek a megfigyelt gyakoriságoknak) három modell ugyanazon M darab, független jelenetből származó tesztpéldára $q^{(m)} \in [0,1]^K$ eloszlást ad, a cél pedig $p^{(m)}$ one-hot vektor. Tervezzen protokollt annak eldöntésére, melyik modell célhoz legközelebbi eloszlást ad: adja meg a megfigyelési egységet, a pontozófüggvényt, a csoportos kiértékelés indokát az 5.4. Függő képek, seedek és resampling tapasztalata alapján, és azt, miért nem a szimmetrizált KL-t, azaz a $\frac{1}{2}(D_{KL}(p\|q) + D_{KL}(q\|p))$ mennyiséget választja elsődleges veszteségnek. A következtetési korlátot is nevezze meg.

7.1.7 Megoldások és indoklás

F1 megoldás. A support a pozitív valószínűségű indexek halmaza: $\operatorname{supp}(p) = \{i : p_i > 0\}$. Ha $p_i > 0$ és $q_i = 0$, akkor a $p_i \ln(p_i/q_i) = p_i \ln(p_i/0) = +\infty$ tag miatt a KL végtelen; ha minden $p_i > 0$ indexre $q_i > 0$, a tagok végesek, és véges sok tag összege véges. Példa $K = 2$-re: $p = (0{,}5;\ 0{,}5)$, $q = (1, 0)$. Ekkor $\operatorname{supp}(p) = \{1, 2\} \not\subseteq \{1\} = \operatorname{supp}(q)$, ezért $D_{KL}(p\|q) = +\infty$; viszont $D_{KL}(q\|p) = 1 \ln(1/0{,}5) = 0{,}6931$ véges, mert $\operatorname{supp}(q) \subseteq \operatorname{supp}(p)$.

F2 megoldás. A feladat négy tizedesjegyet kér; a közbenső lépéseknél több jegyet használunk, a végeredményeket négyre kerekítjük.

$$H(p) = -0{,}25 \ln 0{,}25 - 0{,}75 \ln 0{,}75 \approx 0{,}346574 + 0{,}215762 \approx 0{,}5623.$$

$H(q) = -0{,}5 \ln 0{,}5 - 0{,}5 \ln 0{,}5 = \ln 2 \approx 0{,}6931.$

$$H(p, q) = -0{,}25 \ln 0{,}5 - 0{,}75 \ln 0{,}5 = \ln 2 \approx 0{,}6931.$$

$$H(q, p) = -0{,}5 \ln 0{,}25 - 0{,}5 \ln 0{,}75 \approx 0{,}693147 + 0{,}143841 \approx 0{,}8370.$$

$$D_{KL}(p\|q) = 0{,}25 \ln \frac{0{,}25}{0{,}5} + 0{,}75 \ln \frac{0{,}75}{0{,}5} \approx -0{,}173287 + 0{,}304099 \approx 0{,}1308.$$

$$D_{KL}(q\|p) = 0{,}5 \ln \frac{0{,}5}{0{,}25} + 0{,}5 \ln \frac{0{,}5}{0{,}75} \approx 0{,}346574 - 0{,}202733 \approx 0{,}1438.$$

Ellenőrzés a pontosabb értékekkel: $0{,}562335 + 0{,}130812 = 0{,}693147 = H(p,q)$ és $0{,}693147 + 0{,}143841 = 0{,}836988 = H(q,p)$. A négy tizedesjegyre kerekített értékekkel az összeg a kerekítés miatt az utolsó jegyben eltérhet.

F3 megoldás.

```
import numpy as np

def info_measures(p, q, tol=1e-12):
    p = np.asarray(p, dtype=np.float64)
    q = np.asarray(q, dtype=np.float64)
    assert p.ndim == q.ndim == 1 and p.shape == q.shape
    assert np.all(p >= 0) and np.all(q >= 0)
    assert abs(p.sum() - 1.0) < tol and abs(q.sum() - 1.0) < tol
    with np.errstate(divide="ignore", invalid="ignore"):
        H = -np.sum(np.where(p > 0, p * np.log(p), 0.0))
        ce = -np.sum(np.where(p > 0, p * np.log(q), 0.0))
    kl = ce - H
    if np.any((p > 0) & (q == 0)):
        ce = np.inf
        kl = np.inf
    return H, ce, kl

p = np.array([0.5, 0.0, 0.5])
q = np.array([0.4, 0.3, 0.3])
print(info_measures(p, q))
# Várható (NumPy 2): (np.float64(0.6931471805599453),
# np.float64(1.0601317681000455), np.float64(0.3669845875401002))
```

A `np.where` mindkét ágát kiértékeli, ezért a $p_i = 0$ helyeken a `np.log(p)` `-inf`-et ad, a $0 \cdot (-\infty)$ szorzat pedig `nan`-t; a futás eközben `RuntimeWarning`-ot ír ki, az eredmény viszont a `np.where` kiválasztása miatt helyes marad. A `np.errstate` csak a figyelmeztetéseket kapcsolja ki, a kiválasztást továbbra is a `np.where` végzi. A labor `_xlogy` függvénye ehelyett eleve csak a kiválasztott ágat értékeli ki, ezért ott figyelmeztetés nélkül fut a számítás.

F4 megoldás. Három hiba van. (1) A `np.mean` a sum helyett átlagol, ezért a K hosszú vektorra a helyes összeg $1/K$-szorosát adja; ez a `reduction` paraméter félreértése [D07], hasonlóan a 6.1. Objektív, reguláris tag és gradienslépés alfejezetben tárgyalt mean/sum megkülönböztetéshez. (2) Ha $p_i = 0$ és $q_i = 0$, akkor $0 \cdot \ln 0 = 0 \cdot (-\infty)$ miatt a szorzat `nan`, a konvenció szerint viszont a tag nulla. (3) A support-csapdát a függvény nem kezeli tudatosan: $p_i > 0$, $q_i = 0$ esetén a $-\infty$ tag miatt az eredmény $+\infty$ lesz, de ezt nem a függvény jelzi, hanem a lebegőpontos aritmetika mellékhatása; a `np.mean` ráadásul eltünteti a koordináta szintű információt, és véges esetben hibás skálát ad. A megadott bemeneten $p = (0; 0{,}5; 0{,}5)$, $q = (0{,}5; 0{,}5; 0)$ a harmadik koordinátán $p_3 > 0$ és $q_3 = 0$, ezért a helyes $H(p,q) = +\infty$; a hibás kód is $+\infty$-t ad, mert a $-\infty$ tag dominálja az átlagot, tehát ezen a bemeneten a hiba a végeredményen nem látszik. A mean hibája például $p = q = (0{,}5; 0{,}5)$ esetén mutatkozik meg: a helyes $H(p,q) = \ln 2 \approx 0{,}6931$, a hibás kód ennek csak a felét adja. A $p_i = q_i = 0$ esetre példa $p = (0{,}5; 0; 0{,}5)$, $q = (0{,}5; 0; 0{,}5)$, ahol a második tag `nan`. A javított függvény az F3-beli `info_measures`, illetve a labor `cross_entropy` függvénye: előbbi `np.where(p > 0, ...)`-nel, utóbbi `_xlogy` maszkolással kezeli a nulla tagokat; mindkettő összeget számol, és a support-csapdát `inf`-fel jelzi.

F5 megoldásminta. A megfigyelési egység a jelenet (`scene_id`), nem az egyes kép, mert a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-szerződés szerint ugyanazon jelenet nézetei korrelálnak. A pontozófüggvény a jelenetenkénti átlagos cross-entropy:

$$\bar{H} = \frac{1}{M} \sum_{m=1}^{M} H(p^{(m)}, q^{(m)}),$$

ahol $p^{(m)}$ a one-hot cél, $q^{(m)}$ a modell predikciója. Mivel a megfigyelt $p^{(m)}$ rögzített, $H(p^{(m)})$ a q-tól nem függ, ezért a $\bar{H}$ minimalizálása ekvivalens a jelenetenkénti átlagos $D_{KL}(p^{(m)} \| q^{(m)})$ minimalizálásával. A modellválasztásnál a legalacsonyabb $\bar{H}$ a cél, de a bizonytalanság becslésére jelenetalapú bootstrapet (a jelenetek visszatevéses újramintavételezését) használunk, nem képenkéntit. A szimmetrizált KL-t, azaz a $\frac{1}{2}(D_{KL}(p\|q) + D_{KL}(q\|p))$ mennyiséget azért nem választjuk elsődleges veszteségnek, mert a tanulási cél aszimmetrikus: a modell q eloszlását büntetjük a megfigyelt p célhoz képest, a $D_{KL}(q\|p)$ irány a nem megfigyelt eseményekre más súlyt helyezne, és a support-csapda iránya is megfordulna. A log-loss a predikció és a cél eloszlás eltérését méri; önmagában nem bizonyítja, hogy a modell kalibrált, mert a kalibráció a megadott valószínűségek és a megfigyelt gyakoriságok egyezése, amelyet külön kalibrációs görbével (a becsült valószínűségek és a megfigyelt gyakoriságok összevetésével) kell ellenőrizni. A korlát: a tesztjelenetek kis száma és a procedurális adat (szabályokból generált, nem gyűjtött adat) miatt az eredmény nem általánosítható valós képi vagy termelési feladatra.

7.1.8 Tipikus tévedés és önellenőrzés

Tévedés. „A KL-divergencia távolság, ezért szimmetrikus.” Ez hamis. A 3.1. Vektor, norma és skalárszorzat alfejezetben a metrikus távolság három tulajdonságát láttuk; a KL a szimmetriát sérti. A labor első esetében $D_{KL}(p\|q) \neq D_{KL}(q\|p)$. Másik gyakori tévedés, hogy $H(p, p) = 0$; valójában $H(p, p) = H(p)$, mert $D_{KL}(p\|p) = 0$.

Önellenőrzés. Adott p, q vektorokra ellenőrizze, hogy (1) az elemek nemnegatívak és összegük 1; (2) a support-feltétel teljesül-e; (3) a számítás a $0 \ln 0$ konvenciót használja-e; (4) a cross-entropy és az entrópia különbsége valóban a KL-t adja-e.

Visszatekintés. A 6.1. Objektív, reguláris tag és gradienslépés alfejezetben a `reduction` beállítás (batch-átlag, azaz kötegátlag) a gradiens skáláját is meghatározta; itt a cross-entropy batch-átlaga ugyanígy viselkedik. A 6.4. Momentum és adaptív optimizálók alfejezetben látott frissítési szabályok ezt a célfüggvényt fogják csökkenteni; a KL nemszimmetriája pedig a modellválasztási irányt rögzíti.

7.1.9 Rövid összefoglaló

A diszkrét valószínűségi vektor entrópiája a bizonytalanságot méri, a cross-entropy a cél- és predikciós eloszlás közötti átlagos negatív log-valószínűséget, a KL-divergencia pedig a kettő irányított eltérését. A három mennyiségre fennáll a $H(p, q) = H(p) + D_{KL}(p\|q)$ felbontás; $H(p)$ mindig véges, $H(p, q)$ és $D_{KL}(p\|q)$ pedig pontosan akkor véges, ha $\mathrm{supp}(p) \subseteq \mathrm{supp}(q)$. A KL nemnegatív, de nem szimmetrikus és nem metrikus távolság. A nullás support-csapdát a $p_i > 0$, $q_i = 0$ eset okozza, amely végtelen cross-entropy-t és KL-t ad. A labor ezt két kis eloszláson numerikusan is ellenőrizte.

7.1.10 Továbblépés

A 7.2. Softmax és többosztályos cross-entropy alfejezetben a többosztályos softmax és a logits- (normalizálatlan nyers pontszámokból induló) alapú cross-entropy következik, ahol a modell kimenete nem feltétlenül normalizált, és a numerikus stabilitás miatt a softmax és a logaritmus

együttes implementálása kerül előtérbe; a [D07] szerint a PyTorch `CrossEntropyLoss` logits bemenetet vár int64 célindexszel.

7.1.11 Források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

7.2 Softmax és többosztályos cross-entropy

7.2.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben egyetlen megfigyelés logitvektorából stabil módon számítunk normalizált pontszámot (softmax) és többosztályos cross-entropy veszteséget, majd levezetjük a logit szerinti gradienst: $\nabla_z L = p - e_y$. Az előismeret a 7.1. Entrópia, cross-entropy és KL alfejezetben bevezetett diszkrét cross-entropy és KL, a 3.2. Mátrixszorzás és affín réteg alfejezetbeli affin réteg, valamint a 6.2. Konvexitás és sima optimalizálási határ alfejezetbeli gradienslépés. Induló kérdés: ha $z = (900, 901, 902)$ float32-ben, miért nem számolhatjuk a softmaxot a exp közvetlen kiértékelésével, és miért nem segít önmagában a softmax a stabil veszteséghez?

7.2.2 Logit, softmax és logsumexp

Definíció (logit). Legyen $K \geq 2$ az osztályok száma. Egy megfigyelés logitvektora $z \in \mathbb{R}^K$, ahol z_j a j-edik osztály aktiváció előtti pontszáma; a j indexet a könyv jelölési konvenciójának megfelelően $0, \dots, K-1$ között használjuk, mert a PyTorch célindex is 0-tól indul. A batchelt logit $Z \in \mathbb{R}^{N \times K}$, egy megfigyelés egy sor.

Definíció (softmax). A $\mathrm{softmax} : \mathbb{R}^K \to \Delta_K^\circ$ leképezés, ahol $\Delta_K^\circ = \{p \in (0,1)^K : \sum_j p_j = 1\}$ a nyílt valószínűségi szimplex:

$$p_j = \mathrm{softmax}(z)_j = \frac{e^{z_j}}{\sum_{k=0}^{K-1} e^{z_k}}, \qquad j = 0, \dots, K-1.$$

A kimenetre $p_j > 0$ és $\sum_j p_j = 1$, tehát p diszkrét valószínűségi vektorként értelmezhető. A softmax invariáns a konstans eltolásra: $\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z)$, mert a számláló és a nevező is e^c-vel szorzódik. Ez az invariancia a gradiensre is öröklődik.

Definíció (logsumexp). A logsumexp függvény

$$\mathrm{LSE}(z) = \log \sum_{k=0}^{K-1} e^{z_k}.$$

Ez a softmax nevezőjének logaritmusa. A név az angol *log-sum-exp* kifejezés összevonásából származik, amelyet a szakirodalom gyakran *logsumexp* alakban ír. A log természetes alapú, ahogy az egész könyvben.

Integer target és one-hot. Az osztályozási cél $y \in \{0, \dots, K-1\}$ egész index (PyTorchban int64). Az $e_y \in \mathbb{R}^K$ one-hot vektor

$$(e_y)_j = \delta_{jy} = \begin{cases} 1, & j = y, \\ 0, & j \neq y. \end{cases}$$

A batchelt cél $y \in \{0, \dots, K-1\}^N$, a hozzá tartozó one-hot mátrix $Y_{onehot} \in \{0,1\}^{N \times K}$, ahol $(Y_{onehot})_{nj} = \delta_{j,y_n}$.

Definíció (egy megfigyelés cross-entropy-ja logitsból). A $p = \mathrm{softmax}(z)$ predikció és a y célindex esetén

$$L(z, y) = -\log p_y = -\log \frac{e^{z_y}}{\sum_k e^{z_k}} = -z_y + \mathrm{LSE}(z).$$

Mivel $p_y > 0$ minden valós z-re, a függvény mindenhol értelmes.

Stabil LSE. A $\log \sum_k e^{z_k}$ közvetlen kiértékelése float32-ben már $z_k \approx 89$ fölött `inf`-et ad, pedig a softmax és a veszteség jól definiált. Legyen $c = \max_k z_k$. Ekkor

$$\mathrm{LSE}(z) = \log \left(e^c \sum_k e^{z_k - c} \right) = c + \log \sum_k e^{z_k - c}.$$

Itt minden $z_k - c \le 0$, ezért $e^{z_k - c} \in (0, 1]$, az összeg 1 és K közé esik, tehát a számítás nem csordul túl. A stabil veszteség ezzel

$$L(z, y) = -z_y + c + \log \sum_k e^{z_k - c}.$$

Ha $z_y = c$, akkor a veszteség $\log \sum_k e^{z_k - c} \le \log K$.

A $z = (900, 901, 902)$ induló példában float32-ben e^{900} már `inf`, ezért a közvetlen softmax `inf/inf` hányadosa `nan`; a stabil softmax viszont a maximum kiemelésével ugyanazt adja, mint $z = (0, 1, 2)$ esetén, azaz $p \approx (0{,}0900;\ 0{,}2447;\ 0{,}6652)$. A softmax önmagában azért nem elég a stabil veszteséghez, mert a cross-entropyhoz $\log p_y$ kell: ha a cél logitja nagyon kicsi, p_y float32-ben nullára csökkenhet, és $\log 0 = -\infty$; a $-z_y + \mathrm{LSE}(z)$ alak nem számolja ki külön p_y-t, ezért nagy logitoknál is véges marad.

7.2.3 Batch-veszteség és reduction

A batchelt logitokra $Z \in \mathbb{R}^{N \times K}$ és a $y \in \{0, \dots, K-1\}^N$ célra a softmax soronként számolandó, azaz $P_{nj} = \mathrm{softmax}(Z_{n,:})_j$. Jelölés: egy megfigyelés veszteségét L (nagybetű), a batch átlagolt veszteségét ℓ (kis ell) jelöli; a két jel különböző objektumot ír le. Az átlagolt (mean) cross-entropy

$$\ell(Z, y) = \frac{1}{N} \sum_{n=1}^{N} L(Z_{n,:}, y_n) = \frac{1}{N} \sum_{n=1}^{N} \left(-Z_{n,y_n} + \mathrm{LSE}(Z_{n,:}) \right).$$

A [D07] dokumentáció szerint a PyTorch `torch.nn.CrossEntropyLoss` logits bemenetet vár; a célja lehet egész osztályindex (shape (N)), vagy valószínűségi cél (shape (N, K)), utóbbi lágy címkékhez; a `reduction` paramétere `'mean'` (alapértelmezés), `'sum'` vagy `'none'`. A `'none'` mintánkénti veszteségvektort ad; a `'sum'` a fenti összeg $1/N$ nélkül.

7.2.4 Levezetés: a logitgradiens

Lemma (cross-entropy logitgradiense). Legyen $z \in \mathbb{R}^K$, $y \in \{0, \dots, K-1\}$, $p = \mathrm{softmax}(z)$ és $L(z, y) = -\log p_y$. Ekkor

$$\nabla_z L(z, y) = p - e_y.$$

A gradiens alakja $\mathbb{R}^K$, megegyezik z alakjával.

Bizonyítás. Rögzítsük j-t, és írjuk $S = \sum_k e^{z_k}$, így $p_j = e^{z_j}/S$. A softmax Jacobi-eleme a hányados- és láncszabályból:

$$\frac{\partial p_j}{\partial z_i} = \frac{\delta_{ij} e^{z_i} S - e^{z_j} e^{z_i}}{S^2} = p_i(\delta_{ij} - p_j),$$

mert $p_i = e^{z_i}/S$ és $p_j = e^{z_j}/S$. Ezután a $L = -\log p_y$ összetett függvényre a láncszabály:

$$\frac{\partial L}{\partial z_i} = -\frac{1}{p_y}\frac{\partial p_y}{\partial z_i} = -\frac{1}{p_y} p_i(\delta_{iy} - p_y) = p_i - \delta_{iy}.$$

Az utolsó egyenlőségnél felhasználjuk, hogy $\delta_{iy} p_i = \delta_{iy} p_y$: ha $\delta_{iy} = 1$, akkor $i = y$ és $p_i = p_y$; ha pedig $\delta_{iy} = 0$, mindkét oldal nulla. Esetszétválasztással is ellenőrizhető: ha $i = y$, akkor $-\frac{1}{p_y} p_y(1 - p_y) = p_y - 1$; ha $i \neq y$, akkor $-\frac{1}{p_y} p_i(0 - p_y) = p_i$. Mindkét eset éppen $p_i - \delta_{iy}$. A K parciális deriváltat vektorba gyűjtve $\nabla_z L = p - e_y$. A shape-ellenőrzés: p és e_y is $\mathbb{R}^K$-beli, így a különbségük is. A lemma feltétele minden $z \in \mathbb{R}^K$-ra teljesül, mert a softmax nevezője pozitív.

Következmény (batch-átlag gradiense). Ha $P \in \mathbb{R}^{N \times K}$ a soronkénti softmax kimenet és Y_{onehot} a one-hot mátrix, akkor a mean redukciójú veszteség gradiense

$$\nabla_Z \ell = \frac{1}{N}(P - Y_{onehot}) \in \mathbb{R}^{N \times K}.$$

A `'sum'` redukció esetén ugyanez $1/N$ nélkül. A levezetés a Lemma mintánkénti alkalmazásából és a véges összeg linearitásából adódik. A gyakorlatban a keretrendszer a bejövő (upstream) gradienst g szerint is skálázza: ha a veszteséghez egy további skalár szorzó kapcsolódik, a VJP $g \cdot \frac{1}{N}(P - Y_{onehot})$.

7.2.5 Kalibrációs előkép

A softmax kimenet formálisan diszkrét eloszlás, ezért sokan „valószínűségnek” nevezik. Ez a lépés heurisztika, nem bizonyított kalibrációs állítás. A cross-entropy minimalizálása a célosztály log-valószínűségét növeli, és ideális, végtelen mintás feltételek mellett a feltételes osztályeloszlást közelítheti; a véges mintán tanított modell azonban lehet túl magabiztos vagy alulmagabiztos. A kalibráció azt méri, hogy a 0,7-es konfidencia átlagosan 70%-ban helyes-e. Ennek előképe éppen az itt definiált p, de a tényleges kalibrációs elemzés külön metrikát (például ECE, reliability diagram) és független tesztadatot igényel; ezt a 23.2. Kalibráció és döntési küszöb alfejezetben tárgyaljuk. Az ECE a binenkénti pontosság és átlagos konfidencia különbségének mintaszám szerint súlyozott átlaga:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{N} \left|\mathrm{acc}(S_b) - \mathrm{conf}(S_b)\right|,$$

ahol S_b a b-edik konfidenciabinba eső minták halmaza, $\mathrm{acc}(S_b)$ ezek pontossága, $\mathrm{conf}(S_b)$ pedig az átlagos konfidenciaértékük.

7.2.6 Kézi példa

Legyen $K = 3$, $z = (2;\ -1;\ 0{,}5)^\top$ és $y = 2$. Az exponenciális értékek

$$e^{2} = 7{,}3891, \quad e^{-1} = 0{,}3679, \quad e^{0{,}5} = 1{,}6487,$$

ezért $S = 9{,}4057$. A softmax:

$$p = \left(\frac{7{,}3891}{9{,}4057}, \frac{0{,}3679}{9{,}4057}, \frac{1{,}6487}{9{,}4057}\right) \approx (0{,}7856,\ 0{,}0391,\ 0{,}1753).$$

A veszteség $L = -\log p_2 = -\log 0{,}1753 \approx 1{,}741$. A gradiens

$$\nabla_z L = p - e_2 = (0{,}7856,\ 0{,}0391,\ 0{,}1753 - 1) = (0{,}7856,\ 0{,}0391,\ -0{,}8247).$$

Ellenőrzés: a célosztály gradiense negatív, tehát a gradienslépés növeli z_2-t, a többi logitot pedig csökkenti, ahogy a 6.2. Konvexitás és sima optimalizálási határ alfejezetben megismert csökkenési iránytól várható.

A $J_{ij} = \partial p_j / \partial z_i = p_i(\delta_{ij} - p_j)$ Jacobi-mátrix a kézi példa p vektorára:

$i \backslash j$	0	1	2
0	0,1684	−0,0307	−0,1377
1	−0,0307	0,0376	−0,0069
2	−0,1377	−0,0069	0,1446

A sorösszegek a softmax eltolás-invarianciája miatt nullák; a táblázatban a kerekítés miatt legfeljebb 0,0001 az eltérés.

7.2.7 Labor: véges differencia és nagy konstans eltolás

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A következő kódot lefuttattuk; az itt közölt kimenet mért érték. A fájl a `lab/n026/check_ce_grad.py` munkakönyvtárban van, CPU-n, float64-ben; véletlen seed nem szükséges.

```
# lab/n026/check_ce_grad.py
import numpy as np

def lse(z):
    c = np.max(z)
    return c + np.log(np.sum(np.exp(z - c)))

def softmax(z):
    s = np.exp(z - np.max(z))
    return s / np.sum(s)

def ce_loss(z, y):
    return -z[y] + lse(z)

def ce_grad(z, y):
    e_y = np.zeros_like(z)
    e_y[y] = 1.0
    return softmax(z) - e_y

z = np.array([2.0, -1.0, 0.5], dtype=np.float64)
y = 2
print("z =", z)
print("softmax =", softmax(z))
print("loss =", ce_loss(z, y))
g = ce_grad(z, y)
print("analytical grad =", g)
```

```
eps = 1e-5
num = np.zeros_like(z)
for i in range(z.shape[0]):
    e_i = np.zeros_like(z)
    e_i[i] = 1.0
    num[i] = (ce_loss(z + eps * e_i, y) - ce_loss(z - eps * e_i, y)) / (2.0 * eps)
print("numerical grad =", num)
print("max abs diff =", np.max(np.abs(num - g)))

shift = 1000.0
zs = z + shift
print("shifted loss =", ce_loss(zs, y))
print("shifted softmax =", softmax(zs))
print("shifted grad =", ce_grad(zs, y))

# Ellenőrzés: gradiens, eltolás-invariancia és a loss stabilitása
assert np.max(np.abs(num - g)) < 1e-9
assert np.allclose(softmax(zs), softmax(z), atol=1e-12, rtol=0.0)
assert np.isclose(ce_loss(zs, y), ce_loss(z, y), atol=1e-12, rtol=0.0)
print("ellenorzes: gradiens, eltolas-invariancia es veszteseg rendben")
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges):

```
z = [ 2.  -1.   0.5]
softmax = [0.78559703 0.03911257 0.17529039]
loss = 1.7413112966571571
analytical grad = [ 0.78559703  0.03911257 -0.82470961]
numerical grad = [ 0.78559703  0.03911257 -0.82470961]
max abs diff = 2.0568158287659344e-11
shifted loss = 1.741311296657159
shifted softmax = [0.78559703 0.03911257 0.17529039]
shifted grad = [ 0.78559703  0.03911257 -0.82470961]
ellenorzes: gradiens, eltolas-invariancia es veszteseg rendben
```

A mért softmax (0,7856; 0,0391; 0,1753), a veszteség 1,7413, az analitikus és numerikus gradiens (0,7856; 0,0391; −0,8247), a max abszolút eltérés $2{,}1 \cdot 10^{-11}$; az eltolt esetben a veszteség, a softmax és a gradiens az eltolás előttivel a lebegőpontos kerekítésen belül azonos. A futtatási parancs: `python lab/n026/check_ce_grad.py`. A központi differencia $\varepsilon = 10^{-5}$ mellett a truncációs hiba $O(\varepsilon^2)$, a kerekítési hiba pedig nagyságrendileg $\varepsilon_{\text{gép}}|f|/\varepsilon \approx 10^{-11}$; a mért $2{,}1 \cdot 10^{-11}$ eltérés összhangban van azzal, hogy a két hiba azonos nagyságrendű.

7.2.8 Gyakori tévedés: softmax a CrossEntropyLoss előtt

A [D07] dokumentáció szerint a PyTorch `CrossEntropyLoss` logits bemenetet vár, és belül végzi el a stabil softmax + log műveletet. Ha a felhasználó előre alkalmazza a `torch.softmax`-ot, akkor a loss a már normalizált értékre újra softmaxot számít, tehát a ténylegesen minimalizált függvény nem a kívánt $-\log p_y$. Ellenpélda: $z = (1, 0)$, $y = 0$. A helyes veszteség $-\log \frac{e}{e+1} \approx 0{,}3133$. Ha előbb softmaxot számítunk, $p \approx (0{,}7311;\ 0{,}2689)$, és ezt logitsként átadva a loss belül újra normalizál, így $-\log \frac{e^{0,7311}}{e^{0,7311}+e^{0,2689}} \approx 0{,}48855$. A két érték nem azonos. Nagy logitoknál a hiba még súlyosabb: float32-ben egy közvetlen, exp-alapú softmax $z = (500, 0)$ esetén `[nan, 0]`-t ad, mert e^{500} túlcsordul (`inf`), és az `inf/inf` hányados `nan`; az így kapott valószínűségből számolt loss szintén `nan`. A PyTorch `torch.softmax` belső maximumkivonással dolgozik, ezért ott `[1, 0]` adódik; a stabil, logits-alapú loss mindkét esetben véges marad.

7.2.9 Feladatok

F1 (fogalmi, nehézség 1). Magyarázd el, mi a különbség a softmax és a logsumexp szerepe között a stabil cross-entropy-ban. Miért nem elegendő önmagában a softmax stabil számítása $z = (1000, 1001)$ esetén?

F2 (levezetési, nehézség 2). Vezesd le a $K = 2$ speciális esetben, hogy $y = 1$ esetén a logitgradiens $\nabla_z L = (p_0,\ p_1 - 1)$. Írd át a gradienst a $s = \sigma(z_1 - z_0)$ logisztikus szigmoiddal, ahol $\sigma(t) = 1/(1+e^{-t})$, fejezd ki vele p_0-t és p_1-et, és indokold, miért negatív a második komponens.

F3 (implementációs, nehézség 2). Írj `stable_ce_grad(z, y)` Python-függvényt, amely float64 bemenetre visszaadja a loss-t és a gradienst. Teszteld $z = (1000; 999; 1000{,}5)$, $y = 2$ esetén; hasonlítsd össze a közvetlen exp implementációval (ha az nem overflowol), és adj meg numerikus toleranciát.

F4 (hibakeresési, nehézség 2). Adott a következő hibás sor:

```
loss = nn.CrossEntropyLoss()(torch.softmax(logits, dim=1), target)
```

Lokalizáld a hibát, írd le a következményét, add meg a javított sort, és ellenőrizd a javítást `logits=[[1.0, 0.0]]`, `target=[0]` esetén.

F5 (kísérleti/transzfer, nehézség 3). Tervezz kísérletet a függelék A. szakaszában leírt GlyphLab 24×24-es képosztályozó adatán: egy tanított CNN logitjaira számíts reliability diagramot és várható kalibrációs hibát (ECE) a test spliten. Add meg a protokollt, a megfigyelési egységet, a baseline-t és a következtetési korlátot; a terv nevezze meg a CNN architektúráját, az optimalizálót, a tanulási rátát, az epochszámot és a seedet is.

7.2.10 Megoldások és indoklás

F1. A softmax normalizált p vektort ad, de a cross-entropy $-\log p_y$ számításához a nevező logaritmusára is szükség van; nagy logitoknál a $\sum e^{z_k}$ közvetlen kiértékelése túlcsordul, ezért a $\log \sum e^{z_k}$-t a maximális elem kiemelésével számoljuk. A softmax maga is stabilizálható a $z - \max z$ trükkel, de így csak p-t kapjuk; a $\log p_y = -z_y + \mathrm{LSE}(z)$ előállításhoz az LSE-t külön, a $c = \max z$ kiemeléssel kell számolni. A $z = (1000, 1001)$ esetben a közvetlen e^{1000} float32-ben `inf`, így a softmax `(nan, nan)` lehet, míg a stabil LSE a $(z - c)$ értékekkel dolgozik.

F2. Az általános lemma speciális esete: $y = 1$, tehát $e_y = (0, 1)$. Ekkor $\nabla_z L = p - e_y = (p_0,\ p_1 - 1)$. A logisztikus szigmoiddal $s = \sigma(z_1 - z_0)$, és ezzel $p_1 = e^{z_1}/(e^{z_0} + e^{z_1}) = \sigma(z_1 - z_0) = s$, valamint $p_0 = \sigma(z_0 - z_1) = 1 - s$. A gradiens tehát $(1 - s,\ s - 1)$. Mivel $0 < s < 1$, az első komponens pozitív, a második negatív, ezért a gradiens z_0-t csökkenti, z_1-et pedig növeli, amikor a cél az 1. osztály; ez összhangban van a cross-entropy minimalizálásával. A levezetés feltétele minden valós z-re teljesül.

F3. Az elvárt függvény:

```
import numpy as np

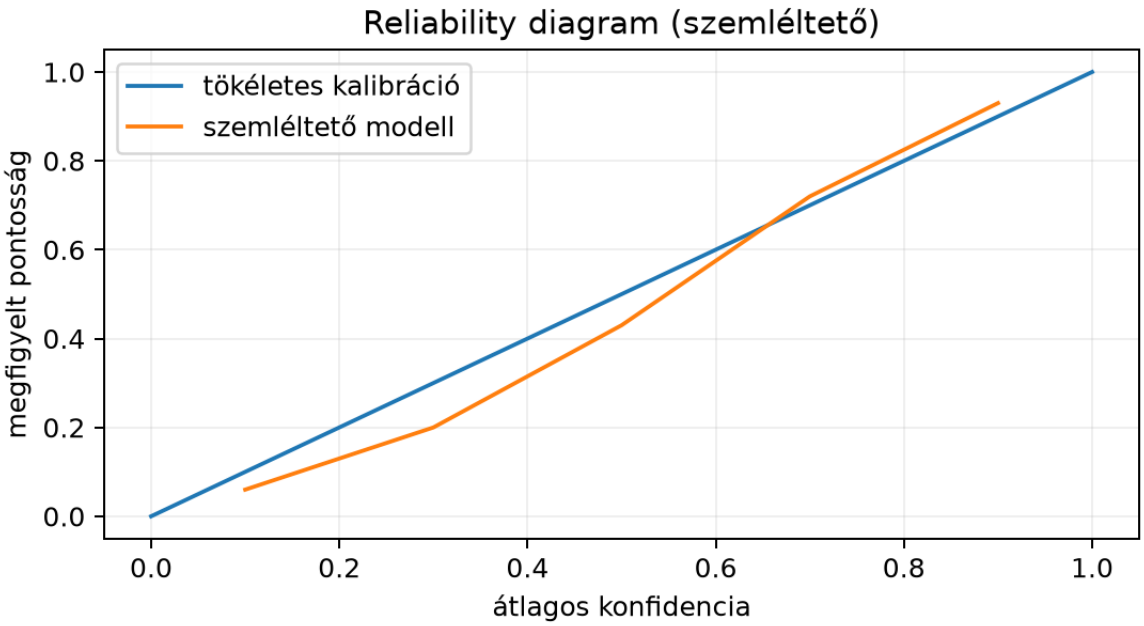
def stable_ce_grad(z, y):
    z = np.asarray(z, dtype=np.float64)
    c = np.max(z)
    log_norm = c + np.log(np.sum(np.exp(z - c)))
    loss = -z[y] + log_norm
    e_y = np.zeros_like(z)
    e_y[y] = 1.0
    p = np.exp(z - c) / np.sum(np.exp(z - c))
    return loss, p - e_y
```

```
# teszt
z = np.array([1000.0, 999.0, 1000.5])
y = 2
loss, grad = stable_ce_grad(z, y)
print(loss, grad)
```

A várt érték: a z eltolás-invarianciája miatt ugyanaz, mint a $z-c = (-0{,}5;\ -1{,}5;\ 0)$ esetben; a loss $-\log \frac{e^{1000,5}}{e^{1000}+e^{999}+e^{1000,5}}$ számítható stabilan, értéke 0,6041 körül, a gradiens pedig $(p_0;\ p_1;\ p_2-1) \approx (0{,}33150;\ 0{,}12195;\ -0{,}45345)$. A közvetlen exp már float64-ben is túlcsordul ($e^{1000} = \texttt{inf}$, a határ $\ln(\text{float64 max}) \approx 709{,}78$), float32-ben pedig még korábban; ezért kell a maximumot kiemelni. A $z - c$ alak közvetlen exp-je és a stabil LSE a nem túlcsorduló tartományban float64-ben 10^{-12} abszolút eltérésen belül egyezik. A függvény visszatérési értéke `(loss, grad)` float64.

F4. A hiba oka, hogy a `CrossEntropyLoss` logitsot vár, nem normalizált valószínűségi vektort. A `torch.softmax` előzetes alkalmazása kettős normalizálást okoz, és a tényleges veszteség nem a kívánt $-\log p_y$. Javítás: `loss = nn.CrossEntropyLoss()(logits, target)`. Ellenőrzés: `logits=torch.tensor([[1.0,0.0]])`, `target=torch.tensor([0])`. A helyes loss $-\log \frac{e}{e+1} \approx 0{,}3133$; a hibás változat $\approx 0{,}48855$. A javított sor a [D07] szerinti konvenciót teljesíti.

F5. Protokollvázlat: (1) rögzítsük a GlyphLab splitjét a függelék A. szakasza szerint: 120/30/30 jelenet és 240/60/60 kép train/val/test között, `scene_id` szerint csoportosítva, hogy ugyanazon jelenet két nézete ugyanabba a splitbe kerüljön; tanítsunk egy fix CNN-t trainen (a tanítás receptje a 9.4. Aktivációk és gradiensáramlás laborjának beállításait követi: Adam, $\text{lr} = 10^{-2}$, 25 epoch, öt seed), válasszunk best checkpointot val szerint. (2) A test 60 képén, tehát 30 független scene-en számítsuk a softmax kimenet konfidenciáját; a megfigyelési egység a scene, mert a két nézet függő. (3) A reliability diagramhoz binekjük a konfidenciákat 5 egyenlő szélességű intervallumba, és ábrázoljuk az átlagos konfidencia–pontosság párokat. (4) Az ECE a binenkénti átlagos konfidencia és a pontosság különbségének abszolút értékéből képzett, mintaszám szerint súlyozott átlag; baseline lehet egy konstans prior (osztálygyakoriság) predikció. A konstans prior baseline minden képre ugyanazt a gyakoriságvektort adja, ezért konfidenciája a legnagyobb prior érték, pontossága pedig a többségi osztály test-gyakorisága; mivel a konfidencia minden mintán azonos, az ECE egyetlen binre redukálódik: $|\max \text{prior} - \text{többségi gyakoriság}|$. (5) Korlát: a 60 képből 30 független scene kevés, az ECE becslése zajos; a 24×24-es szintetikus adat nem természetes képi kalibrációt mér. A kísérlet nem igényel GPU-t; CPU float32 elegendő. A konfidencia itt a softmax kimenet, de a kalibráció a 23.2. Kalibráció és döntési küszöb alfejezetben tárgyalt külön metrika.



7.2. ábra: Reliability diagram (szemléltető)

Szemléltető, nem mért értékek: a kísérletben a pontok a mért bináris átlagokból számítandók.

7.2.11 Önellenőrzés és visszatekintés

Ellenőrizd, hogy a softmax kimenet komponensei pozitívak és összegük 1; a gradiens a célosztályon $p_y - 1 \in (-1, 0)$, máshol $p_i \in (0, 1)$; a batch-átlag gradiense a $\frac{1}{N}(P - Y_{onehot})$ alakú; a stabil LSE a $c = \max z$ kiemeléssel nem overflowol. Visszatekintés: a 7.1. Entrópia, cross-entropy és KL alfejezetben a cross-entropy $H(p, q) = -\sum_i p_i \log q_i$ volt, itt p az one-hot cél, q a softmax kimenet; a 3.2. Mátrixszorzás és affín réteg alfejezetben az affin réteg adta a logitot, itt a loss annak kimenetére épül; a 6.2. Konvexitás és sima optimalizálási határ alfejezetben a gradienslépés iránya volt a csökkenés feltétele, és a most levezetett gradiens adja a tényleges lépésirányt.

7.2.12 Rövid összefoglaló

A softmax a logitvektort diszkrét eloszlássá alakítja, és invariáns a konstans eltolásra. A többosztályos cross-entropy logitsból $L = -z_y + \mathrm{LSE}(z)$ alakban stabilan számítható; az LSE-t a maximális logit kiemelésével értékeljük. Egy megfigyelésre a logitgradiens $p - e_y$, ahol p a softmax kimenet és e_y a cél one-hot vektora; batchelt mean redukciónál $\frac{1}{N}(P - Y_{onehot})$. A `CrossEntropyLoss` logitsot vár, célja osztályindex vagy lágy valószínűségi vektor, előzetes softmax hibás. A softmax kimenet kalibrációs előkép, de önmagában nem bizonyított kalibráció. A labor kézi háromlogitos példán, véges differenciával és 1000-es eltolással ellenőrzi a gradienst. A következő alkalmazás a 7.3. Bináris, multilabel és regressziós cél alfejezetben a bináris és multilabel veszteség.

7.2.13 Források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

7.3 Bináris, multilabel és regressziós cél

7.3.1 Cél, előismeret és induló kérdés

A 7.2. Softmax és többosztályos cross-entropy egységben a többosztályos célhoz tartozó softmax–cross-entropy párt vezettük be. Nem minden osztályozási feladat ilyen: egyetlen képhez tartozhat egyszerre több címke is, egyetlen bináris döntéshez pedig nincs szükség K osztály logitjára. Emellett a neurális háló kimenete nem mindig osztálycímke: regressziós cél esetén valós számot jósolunk, amelyhez más veszteség illik. Ebben az alfejezetben a bináris, multilabel és regressziós célokhoz tartozó outputaktivációt és loss-szerződést építjük fel.

Előismeret: logit fogalma, softmax és természetes logaritmus (7.2. Softmax és többosztályos cross-entropy); batchgradiens és átlagolási konvenció (6.3. Sztochasztikus és minibatch gradiens); legkisebb négyzetek alapgondolata (3.3. Vetítés, legkisebb négyzetek és kondicionáltság). Programozói eszközként elég egy NumPy- vagy PyTorch-alapú tenzorkörnyezet.

Induló kérdés: egy modell három lehetséges betegség egyidejű jelenlétét becsli egy röntgenfelvételen. Miért nem elegendő a három kimenetű softmax, ha egy betegnek egyszerre két betegsége is lehet? A válasz: a softmax kimenetei összegükben 1-et adnak, ezért két egyidejű betegséget nem tud egyszerre magas valószínűséggel kifejezni; a multilabel szerződés ehelyett K független szigmoidot használ, amellyel a $(1, 1, 0)$ cél is ábrázolható.

7.3.2 Definíciók és alakok

Logit. A neurális háló végső affin rétegének aktiváció előtti kimenete a logit. Egy N elemű minibatch esetén a logitokat $Z \in \mathbb{R}^{N\times K}$ jelöli, ahol K az osztályok vagy célváltozók száma. A logit nem valószínűség: lehet negatív, nulla vagy tetszőlegesen nagy.

Bináris cél és sigmoid. Bináris osztályozásban minden megfigyeléshez egyetlen $y \in \{0, 1\}$ cél tartozik. A modell egyetlen $z \in \mathbb{R}$ logitot ad, amelyből a

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

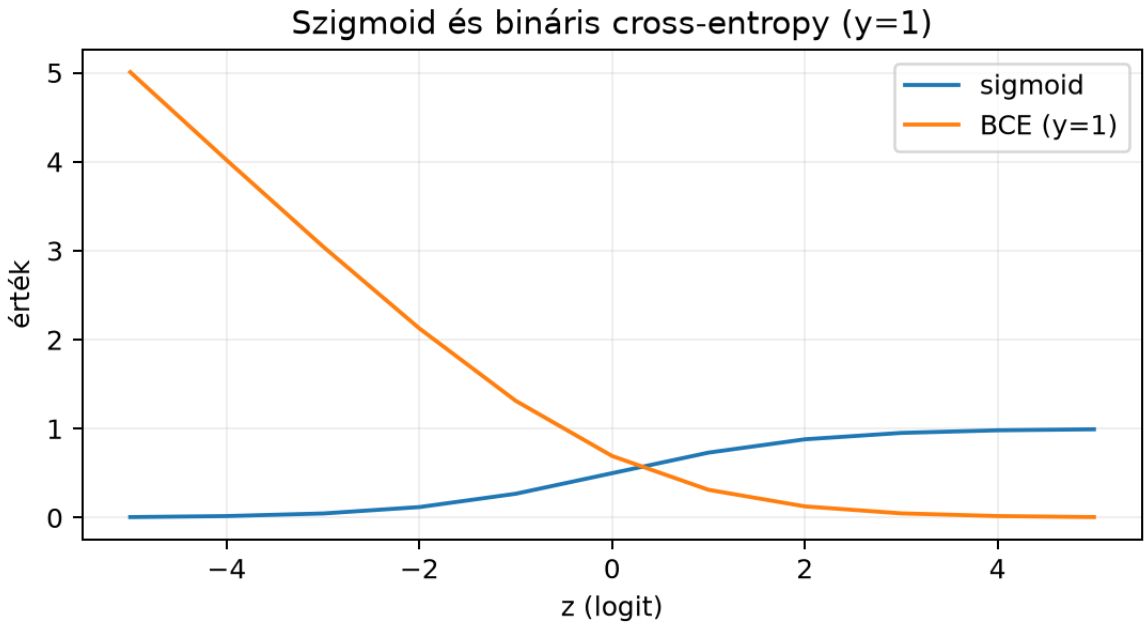
szigmoid aktiváció $[0, 1]$-beli becsült pozitív valószínűséget készít. Batchelt alakban két érvényes, azonos alakú pár van: a logit $(N, 1)$ és a cél $(N, 1)$, vagy a logit $(N,)$ és a cél $(N,)$; a `BCEWithLogitsLoss` [D24] egyező alakot követel, az $(N, 1)$ logit mellé adott $(N,)$ cél hibát ad. A cél float32, mert a veszteségfüggvény valós célokkal dolgozik.

BCEWithLogits. A bináris cross-entropy logitbemenettel egy megfigyelésre:

$$\ell(z, y) = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)).$$

Batchelt mean reduction esetén $L_{\text{mean}} = \frac{1}{N} \sum_{n=1}^{N} \ell(z_n, y_n)$; sum reduction esetén $L_{\text{sum}} = \sum_{n=1}^{N} \ell(z_n, y_n)$. A PyTorch `BCEWithLogitsLoss` ezt a formulát számítja numerikusan stabil

módon, tehát a bemenete logit, nem szigmoidált érték. A szigmoid és a BCE különválasztása ($\sigma(z)$ majd `BCELoss`) numerikusan kevésbé stabil, mert a logaritmus alá kerülhet nagyon kicsi pozitív szám.



7.3. ábra: Szigmoid és bináris cross-entropy (y=1)

Analitikus függvényértékek, nem mérés.

Multilabel cél. Multilabel osztályozásban minden megfigyeléshez egy $Y \in \{0,1\}^{N\times K}$ float32 célmátrix tartozik: $y_{nk} = 1$, ha az n-edik példának a k-adik címke jelen van. A modell $Z \in \mathbb{R}^{N\times K}$ logitmátrixot ad, és minden (n,k) párra külön bináris BCE-t számítunk:

$$L = \frac{1}{NK}\sum_{n=1}^{N}\sum_{k=1}^{K}\left[-y_{nk}\log\sigma(z_{nk}) - (1-y_{nk})\log(1-\sigma(z_{nk}))\right]$$

a `reduction='mean'` konvenció szerint. Fontos: itt nincs softmax a k dimenzió mentén, mert az osztályok nem zárják ki egymást.

Regressziós cél és MSE. Ha a cél valós vektor $Y \in \mathbb{R}^{N\times d_{out}}$, a modell $Z \in \mathbb{R}^{N\times d_{out}}$ kimenetet ad aktiváció nélkül, és a veszteség az átlagos négyzetes hiba:

$$L = \frac{1}{Nd_{out}}\sum_{n=1}^{N}\|z_n - y_n\|_2^2 = \frac{1}{Nd_{out}}\sum_{n=1}^{N}\sum_{j=1}^{d_{out}}(z_{nj}-y_{nj})^2,$$

ahol a `reduction='mean'` PyTorch-konvenció a batch és a kimeneti dimenzió összes elemén átlagol. A mintánkénti négyzetes normák $1/N$-es átlaga ettől d_{out}-szoros tényezővel különbözik; a kettőt nem szabad azonosnak tekinteni. A skalárkimenetű ($d_{out} = 1$) esetben a képletek megegyeznek.

Class weights. Kiegyensúlyozatlan célok esetén a veszteséghez súlyok adhatók. Bináris BCE esetén a `pos_weight` paraméterrel a pozitív osztály tagját $c \in \mathbb{R}_{>0}$ súllyal szorozzuk:

$$\ell_c(z, y) = -c\, y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)).$$

Multilabel esetén osztályonkénti w_k súlyokat használhatunk; többosztályos cross-entropy esetén a `weight` paraméter a $-\log p_{y_n}$ tagot súlyozza. A súlyozás nem változtatja meg a target dtype-ot vagy alakot. A `mean` redukció normalizálása azonban eltér: a `BCEWithLogitsLoss` a súlyozott összeget is az elemek számával osztja (N, multilabel esetén NK), a `CrossEntropyLoss` viszont a figyelembe vett súlyok összegével (a PyTorch 2.15.0-ban mérve).

7.3.3 Levezetések

7.3.3.1 Tétel (BCE logitgradiense)

Legyen $\ell(z, y) = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$, ahol $y \in \{0, 1\}$ rögzített és $z \in \mathbb{R}$. Ekkor

$$\frac{d\ell}{dz} = \sigma(z) - y.$$

Bizonyítás. A szigmoid deriváltja $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Ez a hányadosszabályból is megkapható: $\sigma(z) = \frac{1}{1+e^{-z}}$, ezért

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)).$$

A láncszabállyal:

$$\frac{d}{dz}\left[-y \log \sigma(z)\right] = -y \frac{\sigma'(z)}{\sigma(z)} = -y(1 - \sigma(z)),$$

és

$$\frac{d}{dz}\left[-(1 - y) \log(1 - \sigma(z))\right] = -(1 - y) \frac{-\sigma'(z)}{1 - \sigma(z)} = (1 - y)\sigma(z).$$

Összeadva:

$$-y(1 - \sigma(z)) + (1 - y)\sigma(z) = -y + y\sigma(z) + \sigma(z) - y\sigma(z) = \sigma(z) - y.$$

Ez a kifejezés $y = 1$ esetén $\sigma(z) - 1$, tehát negatív; a gradiens negatív előjele azt jelenti, hogy a **negatív gradiens irányú** frissítés ($z \leftarrow z - \eta\, \partial\ell/\partial z$) növeli a logitot. $y = 0$ esetén a gradiens $\sigma(z) > 0$, tehát a negatív gradienslépés csökkenti a logitot, annál erősebben, minél magasabb a tévesen adott valószínűség. A batchelt mean veszteség $L_{\text{mean}} = \frac{1}{N} \sum_{n=1}^{N} \ell(z_n, y_n)$ deriváltja a mean redukció $\frac{1}{N}$ tényezője miatt $\frac{1}{N}(\sigma(Z) - Y)$ alakú; a batchelt láncszabályt a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezet VJP-konvenciója rögzíti.

7.3.3.2 Levezetés (Gauss-feltevéshez kötött MSE)

Tegyük fel, hogy a cél $y \in \mathbb{R}$ a modell $f(x)$ kimenetének és egy additív zajnak az összege:

$$y = f(x) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \tau^2),$$

ahol $\tau^2 \in \mathbb{R}_{>0}$ ismert, rögzített variancia. A feltételes likelihood sűrűsége:

$$p(y \mid x) = \frac{1}{\sqrt{2\pi\tau^2}} \exp\left(-\frac{(y - f(x))^2}{2\tau^2}\right).$$

A negatív log-likelihood (NLL) egy megfigyelésre:

$$-\log p(y \mid x) = \frac{(y - f(x))^2}{2\tau^2} + \frac{1}{2}\log(2\pi\tau^2).$$

A f paramétereitől független konstans tagot elhagyva a minimalizálandó cél az

$$\frac{1}{2\tau^2}(y - f(x))^2$$

kifejezés. Mivel τ^2 rögzített, az $(y - f(x))^2$ négyzetes hiba minimalizálása ekvivalens az NLL minimalizálásával: a pozitív skálázás nem változtatja meg az optimum helyét. Ez az ekvivalencia azonban csak a fenti Gauss-feltevés mellett érvényes. Ha τ^2-et is becsülni kell, vagy a zaj nem additív Gauss, az MSE már nem indukál ugyanolyan likelihood-feladatot; például heteroszkedasztikus zajnál külön varianciamodellt kell becsülni.

7.3.4 Kézi kis példa

Legyen egy bináris megfigyelés logitja $z = 0$ és célja $y = 1$. Ekkor $\sigma(0) = 0{,}5$, a BCE veszteség

$$\ell(0, 1) = -\log 0{,}5 = \log 2 \approx 0{,}6931.$$

A gradiens a fenti tétel szerint $\sigma(0) - 1 = -0{,}5$.

Multilabel esetben legyen $K = 2$, $z = (1, -2)$ és $y = (1, 0)$. A két elem BCE-je:

$$\ell(1, 1) = -\log \sigma(1) = -\log 0{,}73106 \approx 0{,}3133,$$

$$\ell(-2, 0) = -\log(1 - \sigma(-2)) = -\log 0{,}88080 \approx 0{,}1269.$$

A `reduction='mean'` veszteség $(0{,}3133 + 0{,}1269)/2 = 0{,}2201$.

Regressziós példa: $z = (1{,}5;\ 4{,}5)$, $y = (2, 4)$, $d_{out} = 2$. A négyzetes hibák: $(1{,}5 - 2)^2 = 0{,}25$, $(4{,}5 - 4)^2 = 0{,}25$, az átlag 0,25.

7.3.5 Labor: shape- és dtype-ellenőrzés

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor három eltérő célszerződést ellenőriz shape- és dtype-szempontból. Az alábbi tábla az érvényes konfigurációkat foglalja össze:

Feladat	Logit alak	Target alak	Target dtype	Loss
Bináris	$(N, 1)$ vagy $(N,)$	a logittal pontosan azonos alak	float32	`BCEWithLogitsLoss`
Multilabel	(N, K)	(N, K)	float32	`BCEWithLogitsLoss`
Regresszió	(N, d_{out})	(N, d_{out})	float32	`MSELoss` [D23]

Hibás példák és javításuk:

1. **Hiba:** többosztályos one-hot cél `float32` alakkal (N, K) a `BCEWithLogitsLoss`-nak adva, a modell K logitot ad. **Következmény:** a könyvtár ezt multilabelként értelmezi; minden osztályt függetlenül kezel, nincs softmax normalizálás, ezért a modell helytelen szerződést tanul. **Javítás:** ha a cél egyetlen kizárólagos osztály, használjunk `CrossEntropyLoss`-t int64 célindexszel.
2. **Hiba:** bináris BCE-nek `int64` target. **Következmény:** a PyTorch 2.15.0-ban futásideji hiba: `RuntimeError: result type Float can't be cast to the desired output type Long`; a BCE a célokat valósnak feltételezi. **Javítás:** `y.to(torch.float32)`.
3. **Hiba:** regressziós `MSELoss`-nak `int64` target. **Következmény:** a PyTorch 2.15.0-ban ez nem ad hibát: a cél a művelet során `float32`-re promótálódik, és az eredmény `float32` lesz. A target gradiense nem kell, de a dtype-szerződés így rejtett marad. **Javítás:** a target legyen explicit `float32`.

Az alábbi kód a fenti ellenőrzést futtatja; a kimeneti értékek mért értékek (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, seed 20261020):

```
# labor/n027_shape_check.py
import torch

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32

# Bináris: N=2
z_bin = torch.zeros(2, 1, device=device, dtype=dtype)
y_bin = torch.tensor([[1.0], [0.0]], device=device, dtype=dtype)
assert z_bin.shape == y_bin.shape == (2, 1)
assert z_bin.dtype == y_bin.dtype == torch.float32
loss_bin = torch.nn.functional.binary_cross_entropy_with_logits(z_bin, y_bin)
print("bináris loss, N=2:", loss_bin.item())  # mért: 0.6931

# Multilabel: N=1, K=2
z_ml = torch.tensor([[1.0, -2.0]], device=device, dtype=dtype)
y_ml = torch.tensor([[1.0, 0.0]], device=device, dtype=dtype)
assert z_ml.shape == y_ml.shape == (1, 2)
assert z_ml.dtype == y_ml.dtype == torch.float32
loss_ml = torch.nn.functional.binary_cross_entropy_with_logits(z_ml, y_ml)
print("multilabel loss, N=1, K=2:", loss_ml.item())  # mért: 0.2201
```

```
# Regresszió: N=1, d_out=2
z_reg = torch.tensor([[1.5, 4.5]], device=device, dtype=dtype)
y_reg = torch.tensor([[2.0, 4.0]], device=device, dtype=dtype)
assert z_reg.shape == y_reg.shape == (1, 2)
assert z_reg.dtype == y_reg.dtype == torch.float32
loss_reg = torch.nn.functional.mse_loss(z_reg, y_reg)
print("MSE loss, N=1, d_out=2:", loss_reg.item())  # mért: 0.25
assert abs(loss_bin.item() - 0.6931) < 1e-4
assert abs(loss_ml.item() - 0.2201) < 1e-4
assert abs(loss_reg.item() - 0.25) < 1e-6
print("shape- és dtype-ellenőrzés: rendben")
print("értékellenőrzés: a lossok a kézi példától 1e-4-en belül vannak")
```

Mért kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, seed 20261020):

```
bináris loss, N=2: 0.6931471824645996
multilabel loss, N=1, K=2: 0.22009488940238953
MSE loss, N=1, d_out=2: 0.25
shape- és dtype-ellenőrzés: rendben
értékellenőrzés: a lossok a kézi példától 1e-4-en belül vannak
```

A kimenet a fenti kód futtatásával kapott érték; a beépített `assert`-ek $\pm 0{,}0001$ abszolút toleranciával ellenőrzik a `print`-elt értékeket. A seed itt nem befolyásolja a veszteségeket, mert nincs véletlen mintavétel; a protokoll-egységesség miatt mégis rögzítjük.

7.3.6 Feladatok

F1 (fogalmi, nehézség 1). Egy dokumentumhoz egyszerre több témacímke is tartozhat a $\{0, 1, 2\}$ halmazból. Miért nem helyes egy háromkimenetű softmaxot használni? Adjunk egy konkrét $Y \in \{0, 1\}^{1\times 3}$ célt, amelyet semmilyen softmax-kimenet nem tud pontosan előállítani, és indokoljuk.

F2 (levezetési, nehézség 2). Vezessük le a BCE logitgradiensét $y = 1$ esetén közvetlenül a logisztikus függvény deriváltjának felhasználásával, majd számítsuk ki $z = -3$ esetén a gradiens értékét három tizedesjegy pontossággal.

F3 (implementációs, nehézség 2). Írjunk NumPy-függvényt `bce_with_logits(z, y)` néven, amely stabilan számítja a BCE-t az

$$\ell(z, y) = \operatorname{softplus}(z) - yz$$

azonosság alapján, ahol $\operatorname{softplus}(z) = \log(1 + e^z)$. Ellenőrizzük a fenti azonosságot levezetéssel, és a függvényt a $z = 0{,}0, y = 1{,}0$ és $z = 1{,}0, y = 0{,}0$ eseteken; adjunk toleranciát is.

F4 (hibakeresési, nehézség 2). Az alábbi kód egy háromosztályos probléma céljait készíti elő:

```
N, K = 4, 3
logits = torch.randn(N, K, device="cpu", dtype=torch.float32)
target_onehot = torch.zeros(N, K, device="cpu", dtype=torch.float32)
target_onehot.scatter_(1, torch.randint(0, K, (N, 1), device="cpu"), 1.0)
loss = torch.nn.functional.binary_cross_entropy_with_logits(logits, target_onehot)
```

Mi a szerződési hiba? Milyen matematikai következménye van a predikcióra nézve? Javítsuk a kódot úgy, hogy valódi többosztályos célt használjon, és adjuk meg a helyes target dtype-ot.

F5 (kísérleti/transzfer, nehézség 3). Tervezzünk kis kísérletet egy kiegyensúlyozatlan multilabel osztályozási feladatra, ahol a pozitív címkék aránya osztályonként 1%, 5% és 20%. Hasonlítsuk

össze a súlyozatlan BCE-t és a `pos_weight`-et (vagy osztálysúlyt) használó változatot. Adjunk protokollt: megfigyelési egység, split, baseline, változó tényező, metrika, kiértékelési korlát; a terv nevezze meg a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot is. Nem kell előre megjósolni a győztest.

7.3.7 Megoldások és indoklás

F1 megoldás. A softmax egy diszkrét eloszlást ad, amelynek komponensei minden véges logitra szigorúan pozitívak, és összegük 1. Egy többcímkés cél viszont tetszőleges csúcshalmazt megenged; az $Y = (1, 1, 0)$ vektor összege 2, tehát nem lehet eloszlás. Pontosabban, ha $p = \mathrm{softmax}(z)$, akkor $p_1 + p_2 + p_3 = 1$, ezért $p_1 = 1$ és $p_2 = 1$ egyszerre nem teljesülhet. Sőt, a szigorú pozitivitás miatt még az $(1, 0, 0)$ one-hot vektort sem állítja elő pontosan egyetlen softmax-kimenet sem; a gyakorlatban ezért is használunk a kizárólagos többosztályos célra cross-entropyt, a többcímkés célra pedig független szigmoidokat. A helyes szerződés: K darab független szigmoid kimenet, például $(\sigma(z_1), \sigma(z_2), \sigma(z_3))$, és multilabel BCE.

F2 megoldás. Az $y = 1$ esetben $\ell(z, 1) = -\log \sigma(z)$. A szigmoid deriváltja $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, így a láncszabállyal

$$\frac{d\ell}{dz} = -\frac{\sigma'(z)}{\sigma(z)} = -(1 - \sigma(z)) = \sigma(z) - 1.$$

$z = -3$ esetén $\sigma(-3) = \frac{1}{1+e^3} \approx 0{,}047426$, tehát a gradiens $\sigma(-3) - 1 \approx -0{,}952574$, három tizedesjegyre $-0{,}953$. A negatív előjel várható: a cél pozitív, a modell becslése nagyon kicsi, ezért a logitot növelni kell.

F3 megoldás. Az azonosság: $\mathrm{softplus}(z) - yz = \log(1 + e^z) - yz$. Mivel $\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$, kapjuk, hogy $1 - \sigma(z) = \frac{1}{1+e^z}$. Így

$$-\log(1 - \sigma(z)) = \log(1 + e^z) = \mathrm{softplus}(z).$$

Ezért $y = 0$ esetén a BCE $\mathrm{softplus}(z)$; $y = 1$ esetén a BCE $-\log \sigma(z) = \log(1 + e^{-z})$. A kettőt egységesen az $\mathrm{softplus}(z) - yz$ alak adja, ugyanis $y = 0$-nál $\mathrm{softplus}(z)$, $y = 1$-nél pedig $\mathrm{softplus}(z) - z$. Az utóbbi: $\mathrm{softplus}(z) - z = \log(1 + e^z) - z = \log\left(\frac{1+e^z}{e^z}\right) = \log(1 + e^{-z})$, ami valóban $-\log \sigma(z)$. Tehát az azonosság helyes.

```python
import numpy as np

def softplus(z):
    # logaddexp stabilan, elágazás és túlcsordulás nélkül
    return np.logaddexp(0.0, z)

def bce_with_logits(z, y):
    # z, y: float64 tömbök; y elemei {0,1}
    return softplus(z) - y * z

# Ellenőrzés
for z, y in [(0.0, 1.0), (1.0, 0.0)]:
    val = bce_with_logits(np.array(z), np.array(y))
    print(z, y, val)  # 0.0 1.0 0.69314718; 1.0 0.0 1.31326169
```

Tolerancia: a referenciaértékekhez képest $|val - 0{,}693147| < 10^{-6}$ és $|val - 1{,}313262| < 10^{-6}$. A `float64` használata a numerikus ellenőrzéshez ajánlott; a tanítási útvonal ettől függetlenül float32 maradhat.

F4 megoldás. A hiba: a one-hot cél `BCEWithLogitsLoss`-szal párosul, amely minden osztályt külön bináris problémaként kezel. A modell K logitot ad, de nem normalizálja őket softmaxszal, így a predikált osztályok valószínűségei nem összegződnek 1-re, és a kizárólagosság nem érvényesül. A matematikai következmény: a tanulási cél nem a helyes diszkrét eloszlást közelíti. Javítás: `CrossEntropyLoss` int64 indexekkel:

```
target_class = target_onehot.argmax(dim=1)  # int64, shape (N,)
loss = torch.nn.functional.cross_entropy(logits, target_class)
```

Ekkor a `logits` shape (N, K), a target shape $(N,)$ int64, és a softmax a loss belsejében történik. Új `torch.randint` hívás nem javítás: az az eredeti `target_onehot`-tól független véletlen címkét generálna, és a modell más célra tanulna. A `target_onehot.argmax(dim=1)` visszaállítja azt az osztályindexet, amelyből a one-hot sor készült (a sor egyetlen 1-esének indexét).

F5 megoldásminta (nyitott). Egy lehetséges terv: a függelék A. szakaszában leírt GlyphLab-hoz hasonló kis szintetikus multilabel adat, ahol $N = 20\,000$ független jelenet, minden jelenethez három bináris címke tartozik, a pozitív arányok 1%, 5%, 20%. A független mintavételi egység a `scene_id`. Rögzített, címkénként rétegzett train/val/test split 60/20/20 arányban. Baseline: súlyozatlan `BCEWithLogitsLoss`, a változat: `pos_weight` minden osztályhoz a negatív/pozitív arány szerint, azaz $(1 - p_k)/p_k$ ($p_k = 1\%$ esetén ez ≈ 99), vagy osztályonkénti `weight`. A kísérlet változója csak a veszteségsúlyozás; a modell kétrétegű MLP 64 rejtett egységgel, az inicializálás és az optimalizáló (Adam, lr $= 10^{-2}$) rögzített, az epochszám 30, a seedet pedig ismételjük. Metrika: per-osztály és mikro-átlagolt ROC-AUC és average precision (küszöbfüggetlen), valamint rögzített 0,5-ös küszöb melletti F1; emellett a becsült pozitív gyakoriság eltérése. A 20 000 jelenetből a 20%-os teszt 4000 jelenet, az 1%-os osztályban ez számítással mintegy 40 pozitív teszteset, ezért a ritka osztály per-osztály mutatóinak konfidenciaintervalluma széles, és az arra tett következtetés csak iránymutató. Korlát: a kis szintetikus adat nem általánosít valós klinikai feladatra; a `pos_weight` javíthatja a ritka osztályok visszahívását (*recall*), de ronthatja a globális F1-et, ezért nem állítunk előre győztest; legalább három seed ismétlése szükséges a bizonytalansághoz, és a két változatot ugyanazon seedek szerint párosítva hasonlítjuk össze. A döntési szabály előre rögzítendő: a súlyozott változat csak akkor tekinthető jobbnak, ha a párosított különbségek mind a három seednél ugyanabba az irányba mutatnak; három seed mellett ez gyenge bizonyíték.

7.3.8 Tipikus tévedés ellenpéldával

A leggyakoribb szerződési hiba a többosztályos one-of-K cél és a multilabel cél felcserélése. Tekintsük a $K = 3$ esetet és egy olyan kérdést, ahol egy dokumentum egyszerre „matematika” és „programozás” címkéjű. A cél $y = (1, 1, 0)$. A softmax-kimenet $p = (p_1, p_2, p_3)$ elemei minden véges logitra szigorúan pozitívak és $p_1 + p_2 + p_3 = 1$, ezért $p_1 = 1$ és $p_2 = 1$ egyszerre lehetetlen, sőt egyetlen nulla komponensű one-hot vektort sem állít elő pontosan. Ha a hálót `CrossEntropyLoss`-szal tanítjuk, az $y = (1, 1, 0)$ cél int64 indexként nem is értelmezhető, mert egyetlen osztályindexet vár. Ha viszont one-hot float célként a BCE-hez adjuk, a könyvtár nem hibázik dtype miatt, de szemantikailag rossz célt tanít. Az ellenpélda tehát azt mutatja, hogy a shape egyezése önmagában nem garantálja a helyes szerződést; a célstruktúra matematikai jelentése dönt.

7.3.9 Önellenőrzés és visszatekintés

Önellenőrzés: tudjuk-e a BCE gradienst batchelt mean esetén felírni $\frac{1}{N}(\sigma(Z) - Y)$ alakban? Meg tudjuk-e mondani, miért nem kapunk softmaxot a multilabel BCE-ben? El tudjuk-e választani az MSE optimumát a Gauss-variancia skálázásától?

Visszatekintés: a logit gradiense a 7.2. Softmax és többosztályos cross-entropy alfejezetbeli softmax esetben $p - e_y$ volt, ahol e_y a cél one-hot vektora; ezt a BCE $\sigma(z) - y$ gradiense egészíti ki, és mindkettő a logit bemenetű, numerikusan stabil lossok közös tulajdonsága. A mean/sum reduction különbsége a 6.3. Sztochasztikus és minibatch gradiens alfejezetben tárgyalt batch-átlagozáshoz kapcsolódik; a reguláris tagoknál ugyanígy pontosan meg kell adni a skálát. Az MSE pedig a 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetbeli legkisebb négyzetek elvének neurális megfelelője.

7.3.10 Rövid összefoglaló

A bináris cél logitból szigmoiddal alakul valószínűséggé, a BCEWithLogits pedig numerikusan stabil formulát ad; gradiense $\sigma(z) - y$. Multilabel cél esetén K célbithez K független szigmoid tartozik, nincs softmax, a célmátrix float32. Regressziós cél aktiváció nélküli logitot használ, és MSE-vel tanul; ez rögzített varianciájú additív Gauss-zaj mellett ekvivalens a negatív log-likelihood minimalizálásával. A class weight a ritka célokra ad kontrollt, de nem változtatja meg a target alakját vagy dtype-ját. A labor shape- és dtype-táblája a három szerződést ellenőrzi.

7.3.11 Továbblépés

A 7.4. Numerikus stabilitás és jelentésmegőrző implementáció egységben a numerikus stabilitást és a jelentésmegőrző implementációt általánosítjuk: a log-softmax, a BCE és az MSE implementációs trükkjei után megvizsgáljuk, hogyan őrizhető meg a matematikai jelentés lebegőpontos környezetben.

7.3.12 Források

- [D23] PyTorch contributors (2026). *MSELoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.MSELoss.html. Hozzáférés: 2026-10-30.
- [D24] PyTorch contributors (2026). *BCEWithLogitsLoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BCEWithLogitsLoss.html. Hozzáférés: 2026-10-30.

7.4 Numerikus stabilitás és jelentésmegőrző implementáció

7.4.1 Cél és induló kérdés

Ez az alfejezet a 7.3. Bináris, multilabel és regressziós cél alfejezetben bevezetett lossok bemeneti előírásait (a továbbiakban: loss-előírások) vizsgálja a lebegőpontos implementáció oldaláról. Cél, hogy a hallgató meg tudja mondani, miért ad egy naiv képlet véges helyett `NaN` vagy `inf` értéket nagy logitokra, és hogyan őrizhető meg a matematikai jelentés a cross-entropy, a BCE és a kapcsolódó számítások során. Előismeret: alapvető Python/NumPy, logits és target alakok, softmax és BCEWithLogits definíciója, valamint a float32 alapvető korlátai.

Induló kérdés: ha $z = [500, 501, -501]$, akkor a $\sigma(z)_i = e^{z_i} / \sum_j e^{z_j}$ képlet matematikailag jól definiált. Miért számíthat mégis `NaN`-t egy float32 NumPy implementáció, és melyik egyetlen kivonás teszi stabillá? A válasz röviden: float32-ben e^{500} túlcsordul (overflow), a számláló és a nevező is `inf`, a hányados `inf/inf=NaN`; a $c = \max_j z_j = 501$ kivonása után viszont minden kitevő ≤ 0, ezért minden exponenciális érték legfeljebb 1, az összeg pedig a $[1, K]$ intervallumba esik. A részletes indoklás a következő szakaszokban következik.

7.4.2 Naiv softmax, túlcsordulás és a log-softmax

Legyen $Z \in \mathbb{R}^{N \times K}$ logit-mátrix, egy sor $z \in \mathbb{R}^K$. A softmax definíciója:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \dots, K.$$

A cross-entropy logitsból közvetlenül a log-softmax (a softmax logaritmusa, más néven logsoftmax) negáltját használja:

$$\operatorname{logsoftmax}(z)_i = \log \sigma(z)_i = z_i - \log \sum_{j=1}^{K} e^{z_j}.$$

Programozói értelemben a két alak nem ugyanazt a műveleti sorrendet jelenti. A naiv út először exponenciálist számol: float32-ben $e^{500} \approx 1{,}4 \cdot 10^{217}$, ami meghaladja a legnagyobb normális float32 értéket ($\approx 3{,}4 \cdot 10^{38}$), ezért `inf` lesz. A számláló és a nevező is `inf`, a hányados `NaN`; a logaritmus sem menti meg. Az alulcsordulás (underflow) ellentétes irányú: e^{-500} float32-ben `0`, így a $\log 0$ `-inf`. Mindkettő az exponenciális függvény dinamikus tartományából fakad, nem a loss matematikai hibájából.

7.4.3 A log-sum-exp (lse) konstanseltolási azonossága

A stabilitás alapja egy azonosság, amelyet most tételként mondunk ki és bizonyítunk. Jelölje

$$\operatorname{lse}(z) = \log \sum_{j=1}^{K} e^{z_j}$$

a log-sum-exp (más néven logsumexp, röviden lse) függvényt.

Tétel (log-sum-exp eltolási azonosság). Legyen $z = (z_1, \dots, z_K) \in \mathbb{R}^K$ és $c \in \mathbb{R}$. Ekkor

$$\operatorname{lse}(z) = c + \log \sum_{j=1}^{K} e^{z_j - c}.$$

Bizonyítás. A jobb oldalból indulunk ki. A kitevő azonossága miatt $e^{z_j - c} = e^{-c} e^{z_j}$, tehát

$$c + \log \sum_j e^{z_j - c} = c + \log \left(e^{-c} \sum_j e^{z_j} \right).$$

A logaritmus szorzat-azonossága szerint $\log(ab) = \log a + \log b$, ezért a log a szorzatból összeg lesz:

$$c + \log e^{-c} + \log \sum_j e^{z_j}.$$

Mivel a természetes logaritmus és az exponenciális egymás inverze, $\log e^{-c} = -c$. Így

$$c - c + \mathrm{lse}(z) = \mathrm{lse}(z).$$

A feltételek pontosan a következők: z_j véges valós számok, c véges valós, a logaritmus argumentuma pozitív — ez teljesül, mert $e^t > 0$ minden $t \in \mathbb{R}$-re. A bizonyítás befejezett.

Stabil választás. Válasszuk $c = \max_j z_j$ értéket, és jelölje m ezt a maximumot. Ekkor $z_j - m \le 0$, ezért $0 < e^{z_j - m} \le 1$, és az összeg $s = \sum_j e^{z_j - m}$ a $[1, K]$ intervallumba esik. A $\log s$ véges, nem csordul túl. A gyakorlatban tehát

$$\mathrm{lse}(z) = m + \log s, \qquad s = \sum_{j=1}^{K} e^{z_j - m},$$

a log-softmax pedig

$$\mathrm{logsoftmax}(z)_i = (z_i - m) - \log s.$$

A softmax maga, ha a predikciót is ki akarjuk írni,

$$\sigma(z)_i = \frac{e^{z_i - m}}{s},$$

ugyancsak a max-eltolással stabil.

7.4.4 A CrossEntropyLoss bemeneti és redukciós konvenciói

A PyTorch `nn.CrossEntropyLoss` dokumentációja [D07] szerint a számítás log-softmax és negatív log-likelihood (NLL) kombinációja; az NLL itt a helyes osztályhoz tartozó log-valószínűség negáltja. A függvény **logitsot vár**, tehát a hívó nem alkalmaz előtte softmaxot, és osztályindex-céloknál a cél int64 tömbje $(N,)$ alakú; alternatívaként (N, K) alakú, osztályonkénti valószínűségi cél is megadható megfelelő lebegőpontos típussal. Az alapesetben az int64, $(N,)$ alakot használjuk. A PyTorch 2.15.0 a cél dtype-ját szigorúan ellenőrzi: osztályindexként `int64` vagy `uint8` fogadható el, float cél csak (N, K) alakban; a $(N,)$ alakú float cél `RuntimeError: expected target dtype to be Long or Byte, but got Float` hibát ad. A float valószínűségi cél értékeit viszont a könyvtár nem ellenőrzi: negatív vagy nem 1-re összegző értékek is átmennek, és csendben hibás loss-t adnak (a PyTorch 2.15.0-ban mérve). A dokumentáció által támogatott `reduction` érték `mean`, `sum` és `none`; az itteni példák `mean`-t használnak.

A laborban a könyvtári kimenetet float64-es, külön megírt számításhoz hasonlítjuk: ez a dokumentált bemeneti és redukciós viselkedés numerikus ellenőrzése, nem a könyvtár belső implementációjának bizonyítása. A matematikai azonosság, amelyet fent bizonyítottunk, a stabil implementáció alapja.

7.4.5 `log1p` és a BCEWithLogits stabil alakja

A 7.3. Bináris, multilabel és regressziós cél alfejezetben bevezetett BCEWithLogits veszteség naiv alakja

$$\ell(z, y) = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)).$$

Ha z nagyon negatív, $\sigma(z)$ float32-ben nullává válik; ekkor a naiv képletben $\log \sigma(z)$ `-inf`, így $y = 1$ esetén a veszteség $+\infty$, $y = 0$ esetén pedig a $0 \cdot (-\infty)$ szorzat miatt `NaN`. Ha z nagy pozitív, a $1 - \sigma(z)$ tag nullázódik, és ugyanez történik ellentétes szereposztásban. A stabil formula az y-tól függő algebrai átalakítást használ:

$$\ell(z, y) = (1 - y)z + \mathrm{softplus}(-z),$$

ahol

$$\mathrm{softplus}(t) = \log(1 + e^{t}).$$

A softplus számítására a

$$\mathrm{softplus}(t) = \max(t, 0) + \log(1 + e^{-|t|})$$

alak praktikus, mert $e^{-|t|} \le 1$ miatt a logaritmus argumentuma $[1, 2]$-ben van. Az $e^{-|t|}$ kis $|t|$-re közel 1, nagy $|t|$-re viszont majdnem nulla, és ilyenkor az 1+ hozzáadása után a log elveszítheti a kis tagot (nagy $|t|$ esetén a $\log(1 + e^{-|t|}) \approx e^{-|t|}$ tag elnyelődik az 1-ben). Erre való a `log1p` függvény:

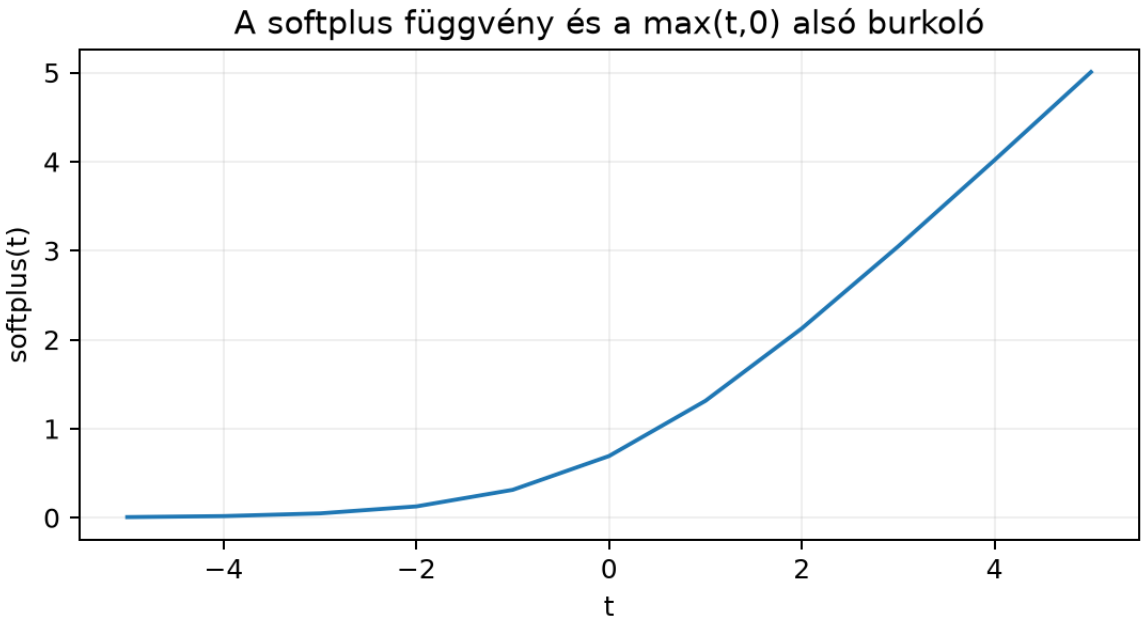
$$\mathrm{log1p}(x) = \log(1 + x),$$

amely kis $|x|$ esetén lényegesen pontosabb, mint a naiv `np.log(1+x)`, mert a könyvtár közvetlenül a x tag alapján, nem az $1 + x$ kerekített értékéből számol. Például $x = 10^{-16}$ esetén `log1p(x)` körülbelül 10^{-16}, míg `1+x` float64-ben kerekítve 1, a log pedig 0. Ugyanez a helyzet a softplus nagy $|t|$ ágán: $t = 40$ esetén $e^{-40} \approx 4{,}2 \cdot 10^{-18}$, az `np.log(1+np.exp(-40.0))` értéke `0.0`, míg `np.log1p(np.exp(-40.0))` $4{,}2 \cdot 10^{-18}$, mert az $1 + e^{-40}$ float64-ben 1-re kerekítődik.

A stabil alak levezetése a naiv definícióból: $\sigma(z) = 1/(1 + e^{-z})$ miatt $\log \sigma(z) = -\log(1 + e^{-z}) = -\,\mathrm{softplus}(-z)$, továbbá $1 - \sigma(z) = \sigma(-z)$, ezért $\log(1 - \sigma(z)) = -\,\mathrm{softplus}(z)$. Behelyettesítve

$$\ell(z, y) = y\,\mathrm{softplus}(-z) + (1 - y)\,\mathrm{softplus}(z) = (1 - y)z + \mathrm{softplus}(-z),$$

ahol az utolsó lépésben a $\mathrm{softplus}(t) = t + \mathrm{softplus}(-t)$ azonosságot használtuk. A 7.3. Bináris, multilabel és regressziós cél alfejezet ugyanezt a veszteséget definiálja; a stabil alak itt a megvalósítás eszköze.



7.4. ábra: A softplus függvény és a max(t,0) alsó burkoló

A képletből számolt szemléltetés, nem mérés. Nagy |t|-re softplus(t) max(t,0), a különbség log1p(exp(-|t|)).

7.4.6 Clipping: vágás a jelentés megőrzésére

A `clipping` vágás azt jelenti, hogy egy értéket korlátos intervallumba szorítunk. Ha a felhasználó a predikált valószínűséget p-ként naplózza, és abból később $\log p$-t vagy entrópiát számol, akkor a $p = 0$ érték `-inf`-et adna. Egy lehetséges gyakorlat

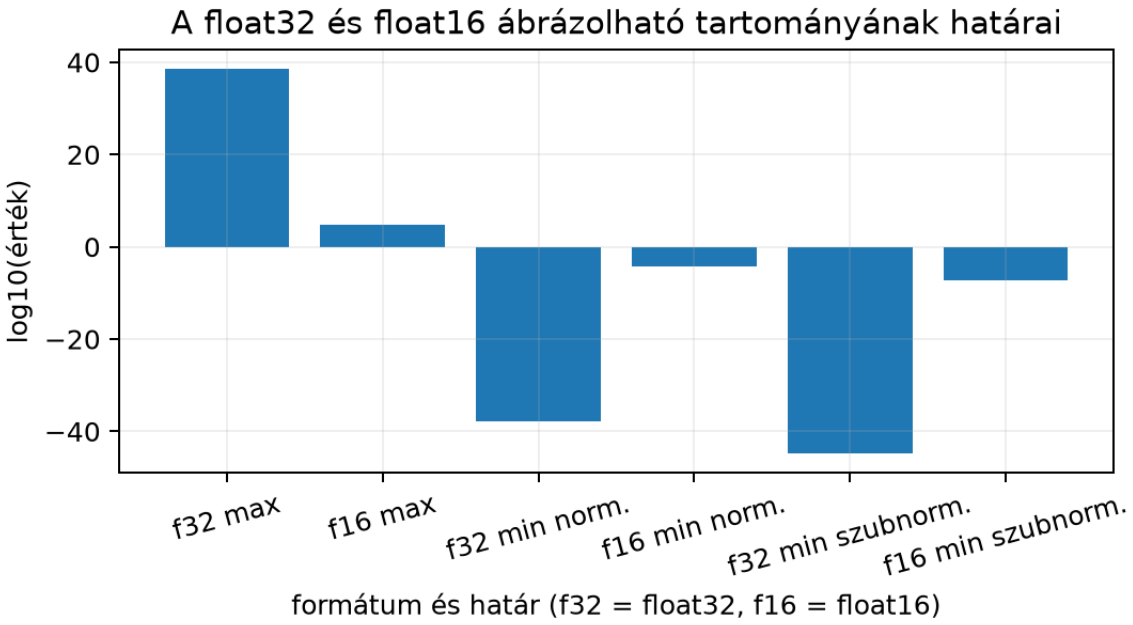
$$\tilde{p}_i = \min(\max(p_i, \varepsilon),\ 1 - \varepsilon),$$

ahol ε kicsi, tipikusan 10^{-7} körüli. Ez a **jelentéskészítés** vágása: biztosítja, hogy a lementett predikció logaritmusa véges legyen.

Fontos határ: a tanítási loss logitjait **nem** szabad így vágni. Ha a cross-entropy bemenetét clipeljük, megváltozik a gradiens jelentése: a $|z| > c$ tartományban a $z \mapsto \mathrm{clip}(z)$ leképezés deriváltja nulla, ezért a modell nem tud a c-nél nagyobb konfidenciára tanulni, és a kalibráció is torzul. A kalibráció itt a prediktált valószínűség és a megfigyelt gyakoriság egyezését jelenti: vágás nélkül a modell nagy logitoknál is követheti a tanítóadatot, vágással viszont a kalibrációs görbéje eltér ettől. A vágás tehát output-utófeldolgozás, nem a numerikus stabilitás tanítási trükkje. Ez heurisztika, mert az optimális ε feladatonként eltérhet; az ellenőrzési terv: a `val` spliten több ε értékre (például 10^{-7}, 10^{-5}, 10^{-3}) számoljuk a naplózott valószínűségek negatív log-likelihood-ját, és a legnagyobb olyan ε-t választjuk, amelynél az NLL eltérése a nem vágott értéktől a 10^{-3} relatív toleranción belül marad.

7.4.7 Mixed precision előkép

A float16 számformátum tartománya jóval szűkebb: a legnagyobb normális érték ≈ 65504, a legkisebb pozitív normális $\approx 6{,}1 \cdot 10^{-5}$, a szubnormális (a legkisebb normálisnál is kisebb, csökkentett pontosságú) tartomány még kisebb, legkisebb pozitív értéke $\approx 6 \cdot 10^{-8}$. Ezért float16-ban már $z = 12$ esetén $e^{12} \approx 162755$, ami túlcsordul. A log-sum-exp max-eltolása itt még kritikusabb, mint float32-ben.



7.5. ábra: A float32 és float16 ábrázolható tartományának határai

A bináris formátum határaiból számolt log10 értékek (nem mérés): a legnagyobb normális, a legkisebb normális és a legkisebb szubnormális érték.

Az automatikus kevert pontosság (AMP) egy gyakori lépése a **loss scaling**: mivel a hátraterjesztés során a veszteség gradiense gyakran olyan kicsi, hogy float16-ban nullává válna, a veszteséget egy nagy skalárral megszorozzák a skálázott hátraterjesztéshez, majd a frissítés előtt visszaosztanak. Például egy 10^{-8} nagyságú gradiens a float16 legkisebb pozitív szubnormális értéke ($\approx 6 \cdot 10^{-8}$) alatt van, ezért nullává válik; $2^{10} = 1024$-es skálázással $1{,}02 \cdot 10^{-5}$ lesz, ami már ábrázolható, és a frissítés előtti visszaosztás után az eredeti értéket adja vissza. Ez a jelen egységben csak előkép; a teljes GPU/AMP-labort a 24.3. Mixed precision és nagyobb batch tárgyalja. A cél itt az, hogy a hallgató tudja: a stabil matematikai formula a float16 környezetben is ugyanaz, de a tartományhatárok miatt az ellenőrzés szigorúbb.

7.4.8 Eredeti kézi kis példa

Legyen $z = [500, 501, -501]$, a helyes osztály indexe $y = 1$ (a Python 0-tól indexel, ezért az 1-es index a második komponenst, az 501-et jelöli; ugyanezt a beállítást használja a labor is). A maximum $m = 501$. A max-eltolás után

$$z - m = [-1, 0, -1002].$$

Az exponenciális összeg:

$$s = e^{-1} + e^{0} + e^{-1002} \approx 0{,}367879 + 1 + 0 = 1{,}367879.$$

A log-sum-exp:

$$\mathrm{lse}(z) = 501 + \log 1{,}367879 \approx 501 + 0{,}313262 = 501{,}313262.$$

A log-softmax:

$$z - \mathrm{lse}(z) = [-1{,}313262;\ -0{,}313262;\ -1002{,}313262].$$

A második komponens negáltja, a veszteség $\ell \approx 0{,}313262$. Ellenőrzésképp a softmax valószínűségek:

$$\frac{e^{z-m}}{s} \approx [0{,}268941;\ 0{,}731059;\ 0],$$

amelyek összege 1 kerekítési hibán belül. A naiv float32 softmax ezzel szemben `inf/inf = NaN`-t adna.

7.4.9 Labor: naiv NumPy, stabil NumPy és könyvtári loss

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor célja a három implementáció összevetése egy float64 referencia ellen. **A következő kimenet a kód futtatásával kapott mért eredmény (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges).**

Munkakönyvtár: `lab/`. Futtatási parancs:

```
python n028_stability.py
```

A szkript:

```
import numpy as np
import torch
import torch.nn as nn

# A példa determinisztikus: rögzített bemenet, véletlen szám nem kell
device = "cpu"

# Input: N=4, K=3 logits, float32; target int64
logits32 = np.array([
    [ 500.0,  501.0, -501.0],
    [   2.0,    1.0,    0.0],
    [-500.0,  500.0,  500.0],
    [  10.0,  -10.0,   20.0],
], dtype=np.float32)
target_np = np.array([1, 0, 2, 2], dtype=np.int64)

# float64 kontroll: a float64 tartományban 500 még véges
z64 = logits32.astype(np.float64)
```

```
lse64 = np.log(np.exp(z64).sum(axis=1, keepdims=True))
logp64 = z64 - lse64
ref = -logp64[np.arange(logits32.shape[0]), target_np].mean()
print("float64 control:", ref)

def naive_nll(logits, target):
    # Szándékosan naiv: először softmax, majd log
    expz = np.exp(logits)                       # float32: túlcsordulás
    probs = expz / expz.sum(axis=1, keepdims=True)
    logp = np.log(probs)                        # 0-ból -inf
    return -logp[np.arange(logits.shape[0]), target].mean()

def stable_nll(logits, target):
    # log-sum-exp max-eltolással, közvetlen log-softmax
    m = logits.max(axis=1, keepdims=True)
    shifted = logits - m
    s = np.exp(shifted).sum(axis=1, keepdims=True)
    lse = m + np.log(s)
    logp = logits - lse
    return -logp[np.arange(logits.shape[0]), target].mean()

loss_fn = nn.CrossEntropyLoss()
t_logits = torch.from_numpy(logits32).to(device)
t_target = torch.from_numpy(target_np).to(device)
with torch.no_grad():
    torch_mean = loss_fn(t_logits, t_target).item()

naive_val = naive_nll(logits32, target_np)
stable_val = stable_nll(logits32, target_np)
print("naive  float32:", naive_val)
print("stable float32:", stable_val)
print("torch  float32:", torch_mean)
print("abs stable-ref:", abs(stable_val - ref))
print("abs torch-ref :", abs(torch_mean - ref))

# A stabil implementációk a float32 kerekítési hibán belül egyeznek a float64 kontrollal
assert np.isnan(naive_val)
assert abs(stable_val - ref) < 1e-5
assert abs(torch_mean - ref) < 1e-5
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, OMP_NUM_THREADS=1; determinisztikus, seed nem szükséges):

```
float64 control: 0.353515057855461
naive  float32: nan
stable float32: 0.35351515
torch  float32: 0.35351502895355225
abs stable-ref: 9.030738079562894e-08
abs torch-ref : 2.8901908755152306e-08
```

A naiv ág `RuntimeWarning: overflow` üzenetet ír ki, eredménye `nan`. A táblázat a **négy minta átlagos** veszteségét mutatja; az értékek mért eredmények:

Implementáció	dtype	Átlagos veszteség (mért)
float64 kontroll	float64	`0.35351506`
naiv NumPy	float32	`nan`
stabil NumPy	float32	`0.35351515`
PyTorch CrossEntropyLoss	float32	`0.35351503`

Az egyes sorok vesztesége ettől eltér: például az első soré ($z = [500, 501, -501]$, $y = 1$) 0.3132617, ami önmagában nem a négy sor átlaga. **Ellenőrzés és értelmezés.** A szkript három `assert`-tel ellenőriz: a naiv érték `NaN`, a stabil és a torch érték pedig 10^{-5}-nél közelebb van a float64 kontrollhoz. A naiv érték nem véges, tehát a stabilizálás nem kozmetikai javítás, hanem az eredmény létezését adja vissza. A torch és a stabil NumPy egyezése a float32 kerekítési hibahatáron belül van.

7.4.10 Gyakori tévedés ellenpéldával

Tévedés: „Ha a veszteség `NaN`, elég numerikus stabilizálást alkalmazni.” **Ellenpélda:** a hívó fél a logitsra már softmaxot alkalmaz, vagy a célvektort float32-ként adja át, és a veszteség még a stabil formulával is hibát jelez. Tekintsük a következő hibás hívást:

```
probs = torch.softmax(logits, dim=1)  # ez már valószínűség, nem logit
loss_fn(probs, target)                # előírás megsértése
```

A CrossEntropyLoss logitsot vár [D07]; a softmax dupla alkalmazása nem dob hibát, hanem csendben rossz értéket ad, mert a valószínűségeket logitként értelmezi, és a gradiens is elromlik. Ugyanígy hibás a bemeneti előírás, ha a `target` float32 típusú, de nem (N, K) alakú: a [D07] szerint float cél csak (N, K) valószínűségi célként fogadható el, $(N,)$ alakú float vektor nem. A hibát a bemeneti előírás ellenőrzése oldja meg, nem a stabilitási trükk. A stabil log-sum-exp tehát szükséges, de nem elégséges: előbb a **matematikai előírást** kell helyesen teljesíteni. Ez az ellenpélda ezért API-használati hiba, nem numerikus instabilitás; a numerikus instabilitást a labor mutatta meg.

7.4.11 Feladatok

F1 (fogalmi, nehézség 1). Adja meg a túlcsordulás (overflow) és az alulcsordulás (underflow) definícióját, és mutasson egy-egy konkrét float32 példát a softmax környékéről. Beadandó: saját megfogalmazású definíciók és két konkrét számérték.

F2 (levezetési, nehézség 2). Bizonyítsa be, hogy a softmax invariáns a konstans eltolásra: $\sigma(z + c\mathbf{1})_i = \sigma(z)_i$ minden $c \in \mathbb{R}$-re, ahol $\mathbf{1}$ a K-dimenziós csupa egyes vektor. Ezután vezesse le a stabil logsoftmax képletet a $c = -\max_j z_j$ választással. Beadandó: teljes levezetés és a végképlet.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt, amely a BCEWithLogits veszteséget a stabil softplus-képlettel számolja, és hasonlítsa össze egy float64 referenciaimplementációval. Bemenet: `z = np.array([[-1000.0, 0.0, 1000.0]], dtype=np.float32)`, `y = np.array([[0.0, 0.5, 1.0]], dtype=np.float32)`. Beadandó: futtatható kód, a várt hiba $< 10^{-6}$ abszolút toleranciával.

F4 (hibakeresési, nehézség 2). Az alábbi kód hibás:

```
p = torch.softmax(logits, dim=1)
loss = nn.CrossEntropyLoss()(p, target_float)
```

Azonosítsa mindkét hibát, javítsa a kódot, és ellenőrizze a javítást a $z = [1, 2, 3]$, $y = 2$ kézi eseten. Beadandó: a hibák megnevezése, javított kód és a kézi ellenőrzés értéke.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet annak vizsgálatára, hogy a logits hőmérséklet-skálázása z/T hogyan hat a cross-entropy értékére és a predikciók szétterülésére egy rögzített CNN-modellen. Használja a függelék A. szakaszában leírt GlyphLab `val` és `test` splitjét a `scene_id` (a jelenet azonosítója) megfigyelési egységgel, egy előre rögzített modell-állapottal.

Adja meg a protokollt, a változó tényezőt, a baseline-t, a metrikát és a következtetési határt. Beadandó: egy lehetséges terv, nem feltétlenül futtatott eredmény.

7.4.12 Megoldások és indoklás

F1 megoldás. Túlcsordulás (overflow): olyan művelet, amelynek eredménye nagyobb a reprezentálható legnagyobb értéknél; float32-ben `np.exp(np.float32(500.0))` `inf`-et ad, míg float64-ben ugyanez $1{,}4 \cdot 10^{217}$, azaz még véges. Alulcsordulás (underflow): az eredmény abszolút értéke kisebb a legkisebb reprezentálható normális értéknél, ezért nullává kerekítik; float32-ben `np.exp(np.float32(-500.0))` `0.0` lesz, míg float64-ben $7{,}1 \cdot 10^{-218}$. A softmax naiv nevezője túlcsordul, a számláló alulcsordulhat; az eredmény `inf/inf=NaN` vagy `0/0=NaN`.

F2 megoldás. A bizonyítás:

$$\sigma(z + c\mathbf{1})_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \sigma(z)_i.$$

A második lépésben az exponenciális szorzat-azonosságát, a harmadikban a közös e^c kiemelését használtuk; $e^c > 0$, ezért a törlés megengedett. Ebből a logsoftmax invarianciája is következik:

$$\operatorname{logsoftmax}(z + c\mathbf{1}) = \operatorname{logsoftmax}(z).$$

A stabil választás: $m = \max_j z_j$, ekkor $z - m$ minden komponense nempozitív. Legyen $s = \sum_j e^{z_j - m}$. A fenti invariancia miatt

$$\sigma(z)_i = \sigma(z - m)_i = \frac{e^{z_i - m}}{s},$$

így

$$\operatorname{logsoftmax}(z)_i = (z_i - m) - \log s.$$

A $s \in [1, K]$ korlát miatt a logaritmus argumentuma véges és pozitív, ezért a stabil alak minden véges z-re véges értéket ad. A levezetés ezzel teljes.

F3 megoldás.

```
import numpy as np

z = np.array([[-1000.0, 0.0, 1000.0]], dtype=np.float32)
y = np.array([[0.0, 0.5, 1.0]], dtype=np.float32)

def stable_bce_with_logits(z, y):
    # stabil softplus: max(t,0)+log1p(exp(-abs(t)))
    t = -z
    softplus = np.maximum(t, 0.0) + np.log1p(np.exp(-np.abs(t)))
    return (1.0 - y) * z + softplus

def ref_bce_with_logits(z, y):
    # float64 referencia a stabil alakból: logaddexp, elkerüli az exp/log-ot
    z64 = z.astype(np.float64)
    y64 = y.astype(np.float64)
    return (1.0 - y64) * z64 + np.logaddexp(0.0, -z64)
```

```
print(stable_bce_with_logits(z, y))
print(ref_bce_with_logits(z, y))
print(np.abs(stable_bce_with_logits(z, y) - ref_bce_with_logits(z, y)).max())
```

A várt eltérés $< 10^{-6}$. A referenciainak is stabilnak kell lennie: a float64-es exponenciális $|z| \approx 709$ felett már túlcsordul, ezért a naiv float64 sigmoid-alak $z = \pm 1000$-nél `inf`-et és $0 \cdot \log 0$ miatt `NaN`-t adna, és a különbség sem lenne értelmes. A `logaddexp`-re épülő referencia ezt elkerüli; a stabil float32 számítás és ez a referencia a mérés szerint $2 \cdot 10^{-9}$ nagyságrendben egyezik. A naiv float32 ágon a $z = 1000$-hez tartozó $\sigma = 1$, $1 - \sigma = 0$, $\log 0$ `-inf`; a stabil formula ezt elkerüli.

F4 megoldás. A két hiba: (1) a `CrossEntropyLoss` logitsot vár, nem softmax-kimenetet; (2) a cél `target_float` float típusú, holott osztálycélként a [D07] szerint int64 szükséges (float cél csak (N, K) valószínűségi célként érvényes, $(N,)$ alakban nem). Javítás:

```
loss = nn.CrossEntropyLoss()(logits, target.long())
```

Ellenőrzés a $z = [1, 2, 3]$, $y = 2$ esetre: log-softmax $[1 - 3.4076, 2 - 3.4076, 3 - 3.4076] \approx [-2.4076, -1.4076, -0.4076]$, a negált log-valószínűség 0.4076. A torch `mean` reduction egy minta esetén ugyanezt adja.

F5 megoldásminta. A hőmérséklet-skálázás (temperature scaling) itt a logits z/T osztása egy $T > 0$ skalárral. Protokoll: rögzítsük egy, a GlyphLab (a könyv képadata-halmaza) train splitjén tanított modell `state_dict`-jét (a tanult paraméterek szótárát); a `val` spliten $(60, 1, 24, 24)$ képekre és `int64` célokra gyűjtsük a logitsokat. Változó tényező: $T \in \{0{,}5;\ 1{,}0;\ 2{,}0;\ 5{,}0\}$; a modell maga változatlan. Metrika: átlagos cross-entropy a skálázott logitsokból és a predikált valószínűség maximumának átlaga, kontrollként a `val` accuracy (a helyes predikciók aránya) rögzítve. Baseline: $T = 1$ az eredeti logits. Kiértékelés: minden T-re a `val` spliten számolt metrika; a kísérleti következtetés csak a hőmérséklet hatását állapíthatja meg a rögzített modellen, nem általános optimalitást. A hatás iránya előre tudható: $T > 1$ lágyítja az eloszlást (laposabb, nagyobb entrópiájú, ezért nagyobb cross-entropy), $T < 1$ élesíti (csúcsosabb, kisebb entrópiájú), miközben az argmax és így az accuracy változatlan maradhat. A `test` splitet legfeljebb egyszer, a protokoll végén szabad használni. Megfigyelési egység a `scene_id`, mert az egy jelenethez tartozó két nézet (`view_id` 0 és 1) függő.

7.4.13 Önellenőrzés és korábbi fogalmak áttekintése

Ellenőrzési lista az alfejezethez: a log-sum-exp-tételben lépésenként indokoltuk a kitevő- és logaritmus-azonosságokat; a max-eltolás után a s összeg $[1, K]$-ban van; a kézi példában a softmax összege 1 kerekítési hibán belül; a labor toleranciája explicit (három `assert`).

Visszatekintés 7.3. Bináris, multilabel és regressziós cél alfejezetből: a BCEWithLogits gradiense $\sigma(z) - y$, így a stabil formula nem változtatja meg a tanítási jelet. Visszatekintés 6.4. Momentum és adaptív optimizálók alfejezetből: az Adam-epsilon a nevezőben egy kis additív tag, amely a nullával való osztást kerüli el; a max-eltolás ezzel szemben nem approximáció, hanem algebrai azonosság, amely a kitevőket nempozitívvá teszi, ezért a két fogalom csak céljában (numerikus védelem) hasonló, működésében nem. Visszatekintés 3.4. Sajátérték, SVD és spektrális határ alfejezetből: ha egy lineáris réteg W súlyának spektrális normája nagy, a logitok szórása megnőhet, és ezzel csökken a softmax numerikus tartaléka, vagyis az a tényleges tartomány, ameddig az exponenciálisok még ábrázolhatók maradnak; a spektrális norma hasznos diagnosztikai eszköz, de nem helyettesíti a stabil formulát.

7.4.14 Rövid összefoglaló

A naiv softmax és BCE implementáció nagy vagy kis logitokra `inf`, `-inf` vagy `NaN` értéket adhat, miközben a matematikai eredmény véges. A log-sum-exp konstanseltolási azonossága szerint $\mathrm{lse}(z) = c + \log \sum_j e^{z_j - c}$, és a $c = \max_j z_j$ választás biztosítja, hogy az exponenciális összeg $[1, K]$-ban maradjon. A log-softmax ebből közvetlenül számolható, a BCEWithLogits pedig a $\max(t, 0) + \log(1 + e^{-|t|})$ softplus-képlettel és `log1p`-pel stabilizálható. A labor float64 kontrollon mutatja ki a naiv, a stabil és a könyvtári implementáció viszonyát.

7.4.15 Továbblépés

A 8.1. Kockázat, mintavétel és baseline egységben a kockázat, a mintavétel és a baseline fogalmát építjük fel; ott azt vizsgáljuk, hogy egy stabilan kiszámolt veszteség milyen tartalmi állításokat tesz lehetővé a modell várható teljesítményéről.

7.4.16 Források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

8 Általánosítás és statisztikai tanulási alapok

8.1 Kockázat, mintavétel és baseline

8.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben két, gyakran összemosott mennyiséget választunk szét: azt a hibát, amelyet a modell a véges tanítómintán mutat, és azt, amelyet a teljes adateloszláson várunk. Az olvasó az 5.2. Várható érték, variancia és likelihood alfejezetből ismeri a várható értéket, az 5.4. Függő képek, seedek és resampling alfejezetből a független és azonos eloszlású mintavételt, a 7.1. Entrópia, cross-entropy és KL alfejezetből pedig a loss fogalmát; a 4.1. Függvény, határérték és derivált alfejezetben bevezetett függvénynyelvet most a modell mint hypothesis leírására használjuk. A 6.4. Momentum és adaptív optimizálók alfejezetben tárgyalt gradienslépés egy konkrét empirikus kockázat csökkentésére szolgál, ezért annak pontos definíciója előfeltétel.

Induló kérdés: egy osztályozó 0 hibát ér el a 240 képből álló tanítóhalmazon. Mennyit mondhatunk ebből arról a hibáról, amelyet a következő, még nem látott képen várunk? A rövid válasz: önmagában nagyon keveset; ezt formalizáljuk.

8.1.2 Hypothesis és modell

Legyen az inputtér $\mathcal{X}$ és a célhalmaz $\mathcal{Y}$. Képosztályozásban $\mathcal{X} = \mathbb{R}^{1\times 24\times 24}$, $\mathcal{Y} = \{0, 1, 2\}$ a függelék A. szakaszában leírt GlyphLab három orientációjára. Egy **hypothesis** (hipotézis) egy

$$h : \mathcal{X} \to \mathcal{Y}$$

függvény, a modell előrejelzési szabálya. A hipotézis lehet közvetlen osztálydöntés, de neurális háló esetén a háló először $z = f_\theta(x)$ logitot számol, és a döntés

$$h_\theta(x) = \arg\max_{k\in\{0,\dots,K-1\}} z_k,$$

ahol K az osztályok száma.

Itt θ a paraméterek összessége. A tanuló algoritmus egy $\mathcal{H}$ **hypothesis space**-ból (hipotézisosztályból) választ; például egy rögzített architektúra összes lehetséges θ paraméterezése egy paraméterezett $\mathcal{H}_\Theta = \{h_\theta : \theta \in \Theta\}$ halmazt ad.

A megfigyelési pár egy $(x, y) \in \mathcal{X} \times \mathcal{Y}$ pár, ahol x a kép tenzora, y az osztálycímke; batchelt feldolgozásban $X \in \mathbb{R}^{N\times 1\times 24\times 24}$ (NCHW: minta, csatorna, magasság, szélesség), $y \in \{0, 1, 2\}^N$ int64 típusú; a mintákat az első dimenzió indexeli.

8.1.3 Populációs kockázat

Tegyük fel, hogy a megfigyelési párok egy valódi, ismeretlen P együttes eloszlásból származnak. Legyen

$$\ell : \mathcal{Y} \times \mathcal{Y} \to [0, B]$$

egy korlátos loss, azaz $\ell(\hat{y}, y)$ a h által adott $\hat{y}$ előrejelzés hibája, ha a valódi cél y. A **population risk** (populációs kockázat, általánosítási hiba) a modell várható hibája a teljes eloszláson:

$$R(h) = \mathbb{E}_{(x,y)\sim P}\left[\ell(h(x), y)\right].$$

A P alsó index azt jelenti, hogy a várható értéket a (x, y) együttes eloszlás szerint vesszük. A 0-1 loss $\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$ esetén $R(h) = P(h(x) \neq y)$ éppen a félreosztályozás valószínűsége, ezért $R(h) \in [0, 1]$. A B korlát minden későbbi koncentrációs állításban explicit feltétel; a stabilan számított cross-entropy loss gyakorlatban véges, de elméletileg nem korlátos, ezért a tanuláselméleti állításokban általában a 0-1 loss-t használjuk célmetrikaként, míg a gradienses tanítás egy helyettesítő (surrogate) célt, például a cross-entropyt minimalizálja. A helyettesítő cél nem illeszkedik az $\ell : \mathcal{Y} \times \mathcal{Y} \to [0, B]$ szignatúrába: a cross-entropy logitokat és egész címkéket (vagy valószínűségeket) vár, és értéke nem korlátos, ezért a koncentrációs állításokat a korlátos 0-1 lossra mondjuk ki, a tanítási eljárást pedig a helyettesítő cél írja le.

A populációs kockázat nem ismert, mert P-t nem ismerjük. Ennek ellenére ez az a mennyiség, amelyet a modell használatakor szeretnénk kicsinek; a tanítás során csak egy véges mintát látunk.

8.1.4 Empirikus kockázat

Legyen

$$S = ((x_1, y_1), \ldots, (x_N, y_N))$$

egy N elemű tanítóminta. Az **empirical risk** (empirikus kockázat, tanítóhiba) a mintaátlag:

$$R_N(h; S) = \frac{1}{N} \sum_{i=1}^{N} \ell(h(x_i), y_i).$$

Ez a populációs kockázat mintából számított becslője; a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben tárgyalt stabil loss-implementáció ezt az átlagot számolja. A PyTorch `CrossEntropyLoss` bemenete logit és egész címke (nem $\ell(\hat{y}, y)$); `reduction='mean'` esetén súlyozás nélkül a batchre éppen a fenti átlagot adja, `reduction='sum'` esetén a $\sum_{i=1}^{N}$ tagot, súlyozás (`weight`) esetén pedig a súlyozott átlagot. A CE-átlag a helyettesítő cél értéke, nem a 0-1 empirikus kockázaté.

Ha S elemei függetlenek és azonos eloszlásúak, akkor az empirikus kockázat torzítatlan becslője a populációs kockázatnak. Valóban, a várható érték linearitása és az azonos eloszlás miatt

$$\mathbb{E}_S\left[R_N(h; S)\right] = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_{(x_i,y_i)\sim P}\left[\ell(h(x_i), y_i)\right] = \frac{1}{N} \sum_{i=1}^{N} R(h) = R(h).$$

A torzítatlanság azonban nem jelenti azt, hogy egy konkrét mintán a kettő közel van egymáshoz; ehhez koncentrációs állítás kell.

8.1.5 IID és a koncentrációs állítás

Azt mondjuk, hogy az S minta **iid** (independent and identically distributed), ha az (x_i, y_i) párok függetlenek, és mindegyikük eloszlása P. A GlyphLab szerződésben a független mintavételi egység a scene_id: ha egy jelenet két nézete is bekerül a mintába, a képek nem függetlenek, ezért a jelenetek számát kell a minta effektív méretének tekinteni. A jeleneten belüli korreláció és a jelenetalapú bootstrap elemzése az 5.4. Függő képek, seedek és resampling alfejezetben szerepel.

A felhasznált koncentrációs állítás a Hoeffding-korlát. Kimondjuk az általános alakot, amely független, de nem feltétlenül azonos eloszlású változókra is érvényes.

Tétel (Hoeffding, [P27] alapján). Legyenek $Z_1, \dots, Z_N$ független valószínűségi változók, ahol majdnem biztosan $a_i \le Z_i \le b_i$ minden i-re, és legyen $\bar{Z} = \frac{1}{N}\sum_{i=1}^{N} Z_i$. Ekkor minden $\epsilon > 0$ esetén

$$P\big(\bar{Z} - \mathbb{E}[\bar{Z}] \ge \epsilon\big) \le \exp\left(-\frac{2N^2\epsilon^2}{\sum_{i=1}^{N}(b_i - a_i)^2}\right),$$

és ugyanez az ellenkező irányú eltérésre is teljesül. A két irányt uniókorláttal összevonva

$$P\big(\big|\bar{Z} - \mathbb{E}[\bar{Z}]\big| \ge \epsilon\big) \le 2\exp\left(-\frac{2N^2\epsilon^2}{\sum_{i=1}^{N}(b_i - a_i)^2}\right).$$

Bizonyítási vázlat. A Hoeffding-lemma szerint ha $Z \in [a, b]$, akkor minden $s > 0$-ra $\mathbb{E}[e^{s(Z-\mathbb{E}Z)}] \le e^{s^2(b-a)^2/8}$. A Markov-egyenlőtlenséget a $e^{s(\bar{Z}-\mathbb{E}\bar{Z})}$ változóra alkalmazva, majd $s > 0$-ra minimalizálva adódik az egyoldali korlát; a másik irányt a $-Z$ változóra felírt korlát adja, a kettőt uniókorlát vonja össze. A teljes bizonyítás [P27]-ben található. A 8.2. Véges hipotézisosztály egyenletes korlátja alfejezet a tételt felhasználva, önálló uniókorlátos levezetéssel építi fel a véges hipotézisosztály egyenletes korlátját.

A kockázatbecslésre alkalmazva legyen $Z_i = \ell(h(x_i), y_i)$ egy rögzített $h \in \mathcal{H}$ esetén. Ekkor $\bar{Z} = R_N(h; S)$ és $\mathbb{E}[\bar{Z}] = R(h)$. Ha a loss a $[0, 1]$ intervallumba esik, például 0-1 loss esetén, akkor $b_i - a_i = 1$ minden i-re, tehát

$$P(|R_N(h; S) - R(h)| \ge \epsilon) \le 2e^{-2N\epsilon^2}.$$

Ha a loss a $[0, B]$ intervallumba esik, akkor $b_i - a_i = B$ minden i-re, tehát az általános alak

$$P(|R_N(h; S) - R(h)| \ge \epsilon) \le 2\exp\left(-\frac{2N\epsilon^2}{B^2}\right).$$

A $[0, 1]$ eset ebből $B = 1$-gyel adódik.

Ez a következmény csak egyetlen, a mintavétel előtt rögzített h-ra érvényes. Ha a h-t ugyanabból a mintából választjuk, mint amelyen a hibát mérjük, az Z_i változók már nem függetlenek a választástól, és a fenti korlát nem alkalmazható. Ez a megkülönböztetés motiválja a 8.2. Véges hipotézisosztály egyenletes korlátja uniókorlátját.

8.1.6 Baseline

A **baseline** olyan egyszerű referenciahipotézis, amelyhez a modell teljesítményét hasonlítjuk, és amely nem használ bonyolult tanulást. Gyakori változatok:

- a **majority baseline** (többségi, más néven gyakorisági baseline) mindig a tanítómintában leggyakoribb osztályt jósolja;
- a **stratified baseline** véletlenül, a tanító osztályarányok szerint sorsol;
- egy egyszerű szabályalapú osztályozó, például a kép egy kézzel választott jellemzőjére illesztett küszöb.

A modell akkor mutat tanulást az adatról, ha a populációs kockázata kisebb a baseline-énál. A GlyphLab három kiegyensúlyozott osztályán (minden osztály aránya 1/3) a majority baseline populációs hibája 2/3: a legnagyobb osztályt mindig eltalálja, a másik kettőt viszont soha, ezért a tévedés valószínűsége $1 - 1/3 = 2/3$. A stratified baseline ugyanígy 2/3 várható hibájú, mert az osztályarányok szerinti sorsolásnál a találat valószínűsége $\sum_k (1/3)^2 = 1/3$. Ezért egy 0,5 hibájú modell már jobb a baseline-nál, de ez önmagában még nem garancia arra, hogy jó; a baseline és a modell összehasonlításának statisztikai korrektségéhez ugyanazt a splitet és ugyanazt a megfigyelési egységet kell használni.

8.1.7 Distribution shift

Azt mondjuk, hogy **eloszlás-eltolódás (distribution shift)** áll fenn, ha a tanító- és a felhasználási eloszlás különbözik: $P_{tr} \neq P_{te}$. A populációs kockázat mindig az adott használati eloszlásra vonatkozik; a tanító eloszláson kicsi $R(h)$ nem garantál kicsi kockázatot a shiftelt eloszláson. Három gyakori eset:

- **covariate shift** (bemenet-eltolódás): $P_{tr}(x) \neq P_{te}(x)$, de $P(y \mid x)$ közös; csak a bemenet eloszlása változik;
- **label shift** (címke-eltolódás): az osztályarányok, azaz $P(y)$ változnak, miközben $P(x \mid y)$ közös;
- **concept drift** (fogalomeltolódás): maga a $P(y \mid x)$ feltételes eloszlás változik.

A GlyphLab `corrupted_test.npy` tesztje (a zavart tesztkészletet tároló NumPy fájl) a rögzített tiszta tesztképekhez képest hozzáadott zajt és fényességgradienst tartalmaz; ez egy ellenőrzött, címkézett covariate-shift-szerű próba. A két teszt ugyanazon jelenet/nézet (scene/view) párokat használja, ezért összevetésük páros. A shiftelt tesztet kizárólag a végleges modell kiválasztása után értékeljük, és abból nem választunk hiperparamétert.

8.1.8 Eredeti kézi kis példa

Legyen $\mathcal{X} = \{a, b, c\}$, $\mathcal{Y} = \{0, 1\}$, a valódi együttes eloszlás pedig az alábbi táblázat szerinti.

(x, y)	$P(x, y)$
$(a, 0)$	0,2
$(a, 1)$	0,1
$(b, 0)$	0,3
$(b, 1)$	0,2
$(c, 0)$	0,1
$(c, 1)$	0,1

Az eloszlás összege $0{,}2 + 0{,}1 + 0{,}3 + 0{,}2 + 0{,}1 + 0{,}1 = 1$. Tekintsük a

$$h(a) = 0, \qquad h(b) = 1, \qquad h(c) = 0$$

hipotézist és a 0-1 losst. A populációs kockázat a hibás cellák valószínűségeinek összege:

$$R(h) = P(a, 1) + P(b, 0) + P(c, 1) = 0{,}1 + 0{,}3 + 0{,}1 = 0{,}5.$$

Legyen most egy $N = 5$ elemű iid minta

$$S = ((a, 0), (b, 1), (b, 0), (c, 1), (a, 1)).$$

A lossértékek rendre $0, 0, 1, 1, 1$, tehát

$$R_5(h; S) = \frac{0 + 0 + 1 + 1 + 1}{5} = 0{,}6.$$

A megfigyelt eltérés $|0{,}6 - 0{,}5| = 0{,}1$. A Hoeffding-következmény szerint $\epsilon = 0{,}2$ mellett a felső korlát

$$2e^{-2 \cdot 5 \cdot 0{,}2^2} = 2e^{-0{,}4} \approx 1{,}3406,$$

amely triviális, hiszen valószínűség nem lehet 1-nél nagyobb. Az $\epsilon = 0{,}5$ választással

$$2e^{-2 \cdot 5 \cdot 0{,}5^2} = 2e^{-2{,}5} \approx 0{,}1641.$$

Tehát a legalább 0,5 eltérés valószínűsége legfeljebb 0,164; a megfigyelt 0,1 eltérés nem mond ellent a korlátnak (a korlát csak felső becslés, az eltérés kisebb nála). A példa azt is mutatja, hogy kis N-nél a korlát nagyon gyenge, és csak nagy mintán válik informatívvá.

8.1.9 Labor: train/test hiba szimuláció

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor egy kontrollált szintetikus feladaton rajzolja meg a tanító- és teszthibát több tanítómintaméretre. A feladat egydimenziós: $X \sim \text{Uniform}[-2, 2]$ és

$$P(Y = 1 \mid X = x) = \sigma(2x), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

A Bayes-osztályozó $h^*(x) = \mathbf{1}[x > 0]$, és a Bayes-kockázat $\mathbb{E}[\min(P(Y = 1 \mid X), 1 - P(Y = 1 \mid X))]$. A modell egy küszöbösztályozó $h_\tau(x) = \mathbf{1}[x > \tau]$, ahol a τ küszöböt (nem összetévesztendő a háló θ paramétereivel) a tanítóminta 0-1 hibájának minimalizálásával választjuk. Ez egy tanuló algoritmus: az empirikus kockázat minimalizálása a küszöbök véges jelölthalmazán. A populációs kockázatot egy nagy, független $N_{te} = 20000$ elemű mintán közelítjük. A labor szintetikus, egydimenziós feladatot használ, nem a fejezet elején említett 240 képből álló GlyphLab-tanítóhalmazt: így a populációs kockázat közvetlenül közelíthető, és a train-test rés tisztán vizsgálható.

Munkakönyvtár: `lab/`. Futtatási parancs: `python n029_simulation.py`. A kód CPU-n fut, NumPy float64 alapértelmezéssel; GPU nem szükséges.

```
# lab/n029_simulation.py
import numpy as np

rng = np.random.default_rng(20261020)

def p_y1(x: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-2.0 * x))

def sample(n: int, rng: np.random.Generator):
    x = rng.uniform(-2.0, 2.0, size=n)
    y = (rng.random(size=n) < p_y1(x)).astype(np.int64)
    return x, y

def loss01(yhat: np.ndarray, y: np.ndarray) -> float:
    return float(np.mean(yhat != y))

def best_threshold(x: np.ndarray, y: np.ndarray):
    """A 0-1 hibát minimalizáló tau küszöb, O(N log N) időben."""
    order = np.argsort(x)
    xs, ys = x[order], y[order]
    n = len(xs)
    # Két szélső eset: mindent 1-nek, illetve mindent 0-nak mondunk.
    # A visszaadott tau ilyenkor -inf, illetve +inf, ami a szélső döntést kódolja.
    best_tau = -np.inf
    best_loss = float(np.mean(ys == 0))  # mindent 1-nek mondunk
    loss_all0 = float(np.mean(ys == 1))  # mindent 0-nak mondunk
    if loss_all0 < best_loss:
        best_loss = loss_all0
        best_tau = np.inf
    # Rendezett minta: csak a szomszédos, különböző értékek közötti küszöbök
    # jelöltek. A hibát prefixösszegekkel számoljuk, minden jelölt O(1).
    pos_cum = np.cumsum(ys == 1)
    tot_pos = int(pos_cum[-1])
    pos_after = tot_pos - pos_cum[:-1]
    neg_after = (n - 1 - np.arange(n - 1)) - pos_after
    err = pos_cum[:-1] + neg_after
    distinct = xs[1:] > xs[:-1]
    err_masked = np.where(distinct, err, n + 1)
    i = int(np.argmin(err_masked))
    if err_masked[i] <= n:
        tau = 0.5 * (xs[i] + xs[i + 1])
        l = float(err_masked[i]) / n
        if l < best_loss:
            best_loss = l
            best_tau = tau
    return best_tau, best_loss

sizes = [10, 20, 50, 100, 200, 500]
reps = 200
Xtest, ytest = sample(20_000, rng)
assert Xtest.shape == (20_000,) and ytest.shape == (20_000,)
bayes_risk_est = float(np.mean(np.minimum(p_y1(Xtest), 1.0 - p_y1(Xtest))))
assert 0.0 <= bayes_risk_est <= 0.5
print(f"bayes_risk_est = {bayes_risk_est:.4f}")
for n in sizes:
    train_errs = []
    test_errs = []
    for _ in range(reps):
        Xtr, ytr = sample(n, rng)
        t, _ = best_threshold(Xtr, ytr)
```

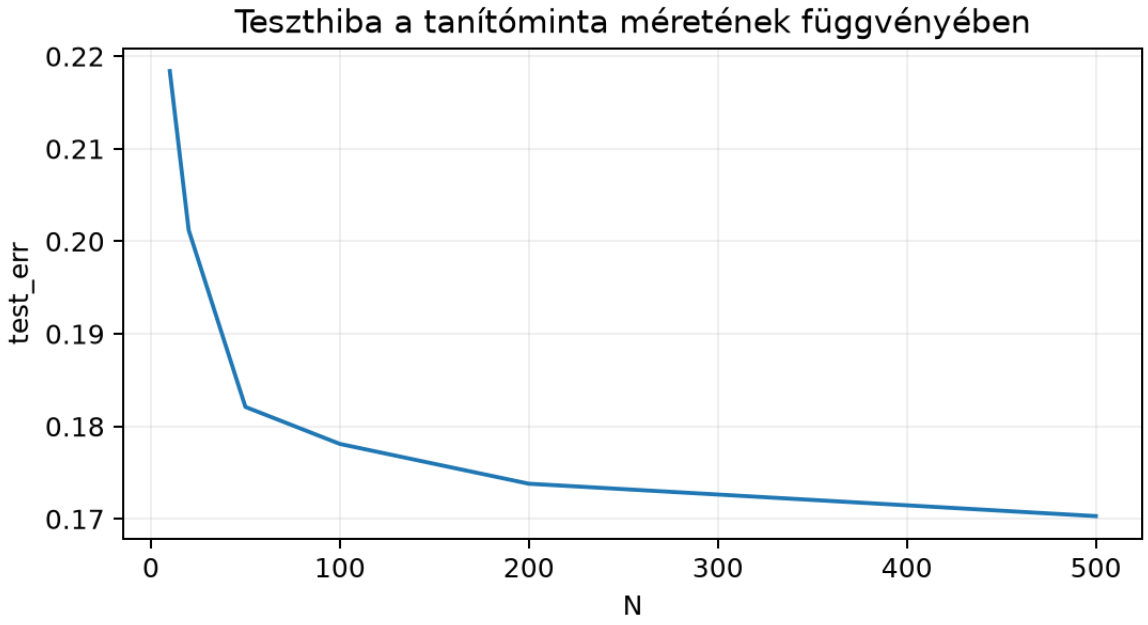
```
        yhat_tr = (Xtr > t).astype(np.int64)
        yhat_te = (Xtest > t).astype(np.int64)
        train_errs.append(loss01(yhat_tr, ytr))
        test_errs.append(loss01(yhat_te, ytest))
    assert 0.0 <= np.mean(train_errs) <= 1.0
    assert 0.0 <= np.mean(test_errs) <= 1.0
    print(f"N={n:4d}  train_err={np.mean(train_errs):.4f} "
          f"test_err={np.mean(test_errs):.4f} "
          f"test_sd={np.std(test_errs):.4f}")
```

A küszöb szélső értékei. A `best_threshold` a két szélső esetben $\pm\infty$ küszöböt ad vissza: $-\infty$ esetén `x > -inf` mindig igaz, tehát minden pontot az 1. osztályba sorolunk, $+\infty$ esetén pedig mindent a 0. osztályba. Ez nem hiba, hanem a „mindent egy osztályba” döntés kódolása; a $\tau \in \mathbb{R}$ értelmezéshez a végtelen csak jelzés. A jelöltek száma $O(N)$, és a prefixösszegekkel mindegyik hibája $O(1)$, ezért a teljes küszöbkeresés $O(N \log N)$ (a rendezés a domináns lépés), nem $O(N^2)$.

Futtatott eredmény. A táblázat a megadott seeddel (`20261020`), Python 3.12.3 és NumPy 2.5.4 alatt, CPU-n, egy szálon (`OMP_NUM_THREADS=MKL_NUM_THREADS=OPENBLAS_NUM_THREADS=1`) futtatott kód kimenete. A `test_sd` a teszthiba szórása a 200 ismétlés között, nem a tesztminta méretének függvénye.

N	`train_err`	`test_err`	`test_sd`
10	0,0815	0,2184	0,0746
20	0,1113	0,2012	0,0511
50	0,1305	0,1821	0,0203
100	0,1425	0,1781	0,0150
200	0,1542	0,1738	0,0089
500	0,1592	0,1703	0,0046

A `bayes_risk_est` értéke ugyanezen futásban 0,1697.



8.1. ábra: Teszthiba a tanítóminta méretének függvényében

A megadott seeddel futtatott kód kimenete. A train_err rendre 0.0815, 0.1113, 0.1305, 0.1425, 0.1542, 0.1592; bayes_risk_est=0.1697.

A várt mintázat teljesül: a `train_err` általában kisebb, mint a `test_err`, a rés N növekedésével csökken, és a `test_err` a `bayes_risk_est` környékére konvergál. A `train_err` azért optimista, mert a τ küszöböt ugyanazon a mintán illesztettük, amelyen a hibát számoljuk. A Hoeffding-korlát ebben a laborban nem érvényes szigorúan a train-test résre, mert a hipotézis a tanítómintától függ; az adaptív választás kezelését a következő alfejezet adja. Numerikusan: $\epsilon = 0{,}05$ mellett a korlát $N = 10$-re 1,90, $N = 100$-ra 1,21, $N = 200$-ra 0,74, $N = 500$-ra 0,16; a korlát csak $N > 139$ esetén esik 1 alá, addig triviális. Ez is mutatja, hogy a korlátot nem a megfigyelt train-test rés ellenőrzésére, hanem felső garanciaként kell használni.

8.1.10 Feladatok

F1 — fogalmi, nehézség 1. Előfeltétel: a fenti definíciók. Magyarázd el saját szavakkal, miért nem következik a $R_N(h; S) = 0$ megfigyelésből az, hogy $R(h) = 0$. Adj egy konkrét kis ellenpéldát a kézi példa eloszlásán. Beadandó: legfeljebb fél oldalas magyarázat és a h' hipotézis, a hozzá tartozó R_5 és R értékekkel.

F2 — levezetési, nehézség 2. Előfeltétel: a várható érték linearitása. Vezesd le lépésről lépésre a torzítatlansági egyenlőséget $R_N(h; S)$-re, és nevezd meg pontosan, melyik lépés használja az azonos eloszlást, és állapítsd meg, hogy a levezetés használja-e a függetlenséget. Beadandó: a három egyenlőségből álló levezetés, minden lépés indoklásával.

F3 — implementációs, nehézség 1. Előfeltétel: a labor kódja. Egészítsd ki a labor ciklusát úgy, hogy minden N-re kiírja a Hoeffding-korlát $2e^{-2N\epsilon^2}$ értékét $\epsilon = 0{,}05$ mellett, és a 200 ismétlésből becsült annak a relatív gyakoriságát, hogy a $|\text{train_err} - \text{test_err}|$ eltérés legalább ϵ. Beadandó: a módosított kód és a futtatási parancs, valamint annak indoklása, miért nem értelmezhető ez a gyakoriság a Hoeffding-korlát érvényességének ellenőrzéseként az adaptív küszöbválasztás miatt.

F4 — hibakeresési, nehézség 2. Előfeltétel: a labor kódja. Tekintsd az alábbi módosítást:

```
Xall = np.concatenate([Xtr, Xtest])
yall = np.concatenate([ytr, ytest])
t, _ = best_threshold(Xall, yall)
yhat_te = (Xtest > t).astype(np.int64)
test_errs.append(loss01(yhat_te, ytest))
```

Mi a hiba, mi a következménye a train/test összehasonlításra, és hogyan javítanád? Beadandó: a hiba oka, következménye, a javított kódrészlet és az ellenőrzés módja.

F5 — kísérlettervezési/transzfer, nehézség 2. Előfeltétel: a distribution shift definíciója. Tervezz szimulációs kísérletet arra a kérdésre, hogy a tanító eloszláson tanított küszöbosztályozó hibája nő-e, ha a teszt X eloszlását Uniform$[-2, 4]$-re, a $P(Y = 1 \mid X = x) = \sigma(2x)$ feltételes eloszlást változatlanul hagyva toljuk el. Adj protokollt, baseline-t, metrikát és következtetési határt. Beadandó: egy lehetséges terv a fenti elemekkel, a szimuláció mint megfigyelési egységével, legalább 200 ismétléssel.

8.1.11 Megoldások és indoklás

F1. Az empirikus kockázat egyetlen véges mintán vett átlag, a populációs kockázat az egész eloszláson vett várható érték. A $R_N = 0$ csak azt jelenti, hogy a modell a mintaelemeket helyesen osztályozza. Olyan eloszláson, ahol P pozitív valószínűséget ad más celláknak, létezhet olyan h', amely a mintán tökéletes, de a populációs kockázata pozitív. A kézi példa eloszlásán legyen

$$h'(a) = 0, \qquad h'(b) = 1, \qquad h'(c) = 1,$$

és a minta $S' = ((a,0),(b,1),(b,1),(c,1),(a,0))$. Ekkor $R_5(h'; S') = 0$, mert mind az öt elemet eltalálja, de

$$R(h') = P(a,1) + P(b,0) + P(c,0) = 0{,}1 + 0{,}3 + 0{,}1 = 0{,}5,$$

mert a $(b,0)$ és $(c,0)$ eseteket, valamint az $(a,1)$ eseteket hibázza. Az ellenőrzés: az öt mintaelem mindegyikére $h'(x_i) = y_i$, a populációs összeg a három hibás cella valószínűsége.

F2. Legyen $Z_i = \ell(h(x_i), y_i)$. A definícióból

$$\mathbb{E}_S\left[R_N(h;S)\right] = \mathbb{E}_S\left[\frac{1}{N}\sum_{i=1}^{N} Z_i\right] = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_S[Z_i].$$

Az első egyenlőség a definíció, a második a várható érték linearitása; ez csak az N összeg tagjainak véges voltát használja. Az azonos eloszlás miatt minden i-re $\mathbb{E}_S[Z_i] = R(h)$, mert minden (x_i, y_i) pár eloszlása ugyanaz a P. Tehát

$$\frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_S[Z_i] = \frac{1}{N}\sum_{i=1}^{N} R(h) = R(h).$$

A függetlenséget ez a levezetés nem használja; a torzítatlansághoz elegendő az azonos eloszlás, a függetlenség a koncentrációs állításhoz kell. Alakellenőrzés: R_N skalár, a várható értéke skalár, a jobb oldal $R(h) \in \mathbb{R}$; a $[0, B]$ loss miatt a várható érték véges.

F3. A ciklus belsejébe illeszthető:

```
eps = 0.05
bound = 2.0 * np.exp(-2.0 * n * eps**2)
exceed = float(np.mean(np.abs(np.array(train_errs) - np.array(test_errs)) >= eps))
print(f"N={n:4d}  Hoeffding_bound={bound:.4f}  excess_freq={exceed:.4f}")
```

A futtatás parancsa változatlan: `python n029_simulation.py`. A korlát egy rögzített h-ra vonatkozó állítás; itt a küszöböt a train mintából választjuk, ezért h a minta függvénye. Az `excess_freq` így nem a Hoeffding-korlát ellenőrzése, hanem csak leíró statisztika az adaptív választású hipotézis train-test eltéréséről. A korrekt egyenletes állítás a 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben következik véges $\mathcal{H}$-ra.

F4. A hiba az, hogy a küszöböt a train és a test minta egyesített halmazán választjuk. Ennek következménye, hogy a test adat befolyásolja a modellt, ezért a `test_err` már nem egy előre nem látott mintán mért hiba: optimistán torzított, és nem méri az általánosítást. A javítás az, hogy a küszöböt kizárólag a train mintán kell választani, és a test mintát csak a választás után, egyszer kiértékelni:

```
t, _ = best_threshold(Xtr, ytr)
yhat_te = (Xtest > t).astype(np.int64)
test_errs.append(loss01(yhat_te, ytest))
```

A `Xall/yall` összefűzés elmarad. Az ellenőrzés: ugyanazon seed mellett a javított `test_err` várhatóan nagyobb, mint a hibás változaté. Ennek oka a szelekciós torzítás: a hibás változat a tesztcímkéket is felhasználja a küszöb kiválasztásához, ezért a mért hiba egy, a tesztmintától is függő döntés hibája; a train-only küszöb a tesztmintától független, ezért a torzított érték átlagosan alábecsüli a valódi általánosítási hibát.

F5. Egy lehetséges terv. Kérdés: a tanító eloszláson tanított küszöb hibája nő-e a shiftelt teszteloszláson? Adat: a tanító minták a labor generátorából, $X \sim \text{Uniform}[-2, 2]$; a shiftelt teszt $X \sim \text{Uniform}[-2, 4]$, ugyanazzal a $\sigma(2x)$ feltételes eloszlással. Megfigyelési egység: egy független minta; a shiftelt teszt egy nagy, független $N_{te} = 20000$ elemű minta. Split: a tanítóminta mérete $N = 200$, 200 ismétlés. Baseline: a gyakorisági osztályozó hibája, amelyet mindkét eloszlásra külön számítunk. Az 1. osztály aránya az eredeti eloszláson a $\sigma(2x) + \sigma(-2x) = 1$ szimmetria miatt 1/2. A shiftelt eloszláson az $X > 0$ tartomány nagyobb súlyt kap: az $[-2, 2]$ szakaszon az átlag 1/2, a hozzáadott $[2, 4]$ szakaszon pedig $\sigma(2x) > 1/2$, ezért a kevert átlag is 1/2 felett van (számszerűen $\approx 0{,}665$). A majority baseline így a shiftelt eloszláson az 1. osztályt jósolja, hibája $1 - 0{,}665 \approx 0{,}335$, szemben az eredeti eloszlás 0,5-jével. Tényező: shiftelt versus eredeti teszteloszlás. Metrika: 0-1 hiba a nagy tesztmintán. Eljárás: minden ismétlésben tanítsuk a küszöböt a train mintán, értékeljük ki az eredeti és a shiftelt tesztmintán, és számoljuk ki a két hiba átlagát és szórását a 200 ismétlésből. Következtetési határ: a szimulációban a két teszteloszlás között egyedül az X marginális eloszlása tér el, ezért a hibakülönbség a kísérleti elrendezésen belül a shiftnek tulajdonítható; ez nem általános okozati állítás, és valós adatokon a shift detektálása, valamint a $P(y \mid x)$ változatlanságának ellenőrzése külön feladat. Az eredmény nem termelési robusztussági igazolás. Döntési szabály: a shiftelt hiba akkor tekinthető nagyobbnak, ha a 200 párosított ismétlés átlagos különbsége pozitív, és a 95%-os bootstrap intervallum nem tartalmaz nullát; erőforrás: CPU, NumPy, a rögzített `cnn-labor` környezet.

8.1.12 Tipikus tévedés és ellenpélda

Gyakori tévedés, hogy a tanítóhiba csökkenése önmagában az általánosítás javulása. Az F1 ellenpéldája ezt cáfolja: a h' hipotézis a mintán 0 hibát ér el, miközben populációs kockázata 0,5. A tanítóhiba csökkenése csak azt jelenti, hogy a modell egyre jobban illeszkedik a véges mintához; a populációs kockázat változásához a mintától független adaton vagy elméleti korláton keresztül kell bizonyítékot gyűjteni. A kettő összekeverése ahhoz a hibás következtetéshez vezet, hogy egy nagyobb kapacitású háló, amely a train hibát nullára csökkenti, automatikusan jobb modell.

8.1.13 Önellenőrzés és visszatekintés

Ellenőrizd, hogy meg tudod-e válaszolni az alábbiakat. Mi a $R(h)$ és $R_N(h; S)$ definíciója, és melyik ismert a gyakorlatban? Mi a Hoeffding-korlát két feltétele (függetlenség és korlátosság), és miért nem alkalmazható adaptív modellválasztásra? Mi a különbség covariate shift és label shift között? Ha a GlyphLab jeleneteiből képenként veszel iid mintát, milyen feltétel sérül az 5.4. Függő képek, seedek és resampling szerint?

Visszatekintés: a loss fogalmát és a cross-entropy célfüggvényt a 7.1. Entrópia, cross-entropy és KL vezette be; itt a 0-1 loss mint célmetrika különül el a surrogate losstól. A 4.1. Függvény, határérték és derivált alfejezetben definiált függvény és paraméterezett függvénycsalád fogalma adja a hypothesis nyelvét; a gradienslépés a 6.4. Momentum és adaptív optimizálók alfejezetből egy empirikus kockázatot csökkentő eljárás.

8.1.14 Rövid összefoglaló

A populációs kockázat a modell várható hibája a teljes, ismeretlen adateloszláson, az empirikus kockázat pedig ugyanezen loss véges tanítómintán vett átlaga. IID minta esetén az empirikus kockázat torzítatlan becslő, de egy konkrét mintán jelentősen eltérhet; a Hoeffding-korlát rögzített hipotézisre számszerűsíti az eltérés valószínűségét, adaptív modellválasztásra azonban nem érvényes. A baseline olyan egyszerű referencia, amelyhez a modellt viszonyítjuk; distribution shift esetén a tanító eloszláson mért jó kockázat nem garantálja a használati eloszláson mért jót. A labor szimulációja a train-test rést és annak mintaméret-függését szemlélteti.

8.1.15 Továbblépés

A 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben a Hoeffding-korlátból uniókorláttal egyenletes, minden $h \in \mathcal{H}$-ra egyszerre érvényes állítást vezetünk le véges hipotézisosztályra; ez adja az adaptív modellválasztás első korrekt hibakorlátját.

8.1.16 Források

- [P27] Hoeffding, W. (1963). Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301), 13–30. https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830. Hozzáférés: 2026-10-30.

8.2 Véges hipotézisosztály egyenletes korlátja

8.2.1 Cél, előismeret és induló kérdés

A 8.1. Kockázat, mintavétel és baseline alfejezetben láttuk, hogy a Hoeffding-korlát egy **rögzített** h hipotézisre számszerűsíti az empirikus és a populációs kockázat eltérését. A gyakorlati modellválasztás azonban nem rögzített: több jelölt közül éppen azt választjuk, amelyik a tanítómintán a legjobbnak látszik. Ebben az alfejezetben az a kérdés, hogyan kaphatunk olyan állítást, amely **minden** $h \in \mathcal{H}$ jelöltre egyszerre, tehát a mintán választott modellre is érvényes, ha $\mathcal{H}$ véges.

Előismeret: a 8.1. Kockázat, mintavétel és baseline populációs kockázat, empirikus kockázat, iid minta és a Hoeffding-állítás fogalma; a 7.2. Softmax és többosztályos cross-entropy cross-entropy definíciója. Differenciálhatóságra itt nincs szükség: a korlát csak korlátosságot és függetlenséget kér.

Induló becslés: ha $M = 1000$ jelölt, $n = 500$ minta és 5% megengedett kudarcvalószínűség, mekkora ϵ hibasáv garantálható? A levezetés után ezt pontosan kiszámítjuk.

8.2.2 Definíciók, jelölések és a felhasznált koncentrációs állítás

Legyen $\mathcal{X}$ a bemeneti tér, $\mathcal{Y}$ a célok tere, $\mathcal{H}$ pedig **hipotézisosztály**, azaz jelöltek egy halmaza; elemei $h : \mathcal{X} \to \mathcal{Y}$ függvények. A tanulási feladat **loss** (veszteség) függvénye $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$; az ebben az alfejezetben szereplő korlátok **feltétele**, hogy a loss értékkészlete a zárt $[0, 1]$ intervallumba essen. A korlátlan lossra ez a feltétel sérül, amit az alábbi „Gyakori tévedés: korlátlan cross-entropy” szakasz mutat meg. Egy h-ra a **populációs kockázat** és az $S = \{(x_i, y_i)\}_{i=1}^{n}$ iid (független és azonos eloszlású) mintán vett **empirikus kockázat**:

$$R(h) = \mathbb{E}_{(x,y)}[\ell(h(x), y)], \qquad \hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(x_i), y_i).$$

A tenzoralak itt egyszerű: ha a modell n megfigyelésre $Z \in \mathbb{R}^{n \times K}$ logitokat ad (a batch-dimenzió itt a korlát n mintaszáma), akkor a loss megfigyelésenként egy-egy skalár, az empirikus kockázat pedig ezek **átlaga**, tehát skalár. A skalár reduction (a lossok egyetlen számmá redukálása, itt átlagolással) elengedhetetlen: a Hoeffding-korlát átlagokra szól.

Az alábbi állítást adottnak vesszük; a teljes bizonyítást nem reprodukáljuk, mert forrása [P27] eredeti cikke, amelyre itt kiadói bibliográfiai rekordként hivatkozunk, a bizonyítás szövegének átvétele nélkül.

Tétel (Hoeffding, $[0,1]$-korlátos alak). Legyen $X_1, \dots, X_n$ független valószínűségi változó, mindegyik a $[0,1]$ intervallumba esik; legyen $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$ és $\mu = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[X_i]$. Ekkor minden $\epsilon > 0$-ra

$$P\big(\bar{X} - \mu \geq \epsilon\big) \leq e^{-2n\epsilon^2}, \qquad P\big(\bar{X} - \mu \leq -\epsilon\big) \leq e^{-2n\epsilon^2},$$

és a két irányt összeadva

$$P\big(|\bar{X} - \mu| \geq \epsilon\big) \leq 2e^{-2n\epsilon^2}.$$

A log a képletekben **természetes logaritmus**. A $[0,1]$ alakra a 8.1. Kockázat, mintavétel és baseline alfejezet általános, $a_i \leq Z_i \leq b_i$ alakú Hoeffding-állításából $a_i = 0$, $b_i = 1$ helyettesítéssel jutunk. Az állítást rögzített hipotézisre már 8.1. Kockázat, mintavétel és baseline alkalmazta: ott $X_i = \ell(h(x_i), y_i)$.

8.2.3 Uniókorlát és a véges $\mathcal{H}$ egyenletes korlátja

A **uniókorlát** (union bound) elemi valószínűségi egyenlőtlenség: tetszőleges $A_1, \dots, A_M$ eseményekre

$$P(A_1 \cup A_2 \cup \cdots \cup A_M) \leq \sum_{j=1}^{M} P(A_j).$$

Az uniókorlát minden eseményrendszerre érvényes, függetlenül attól, hogy az események függetlenek-e. A korlát sok eseményre azért lehet laza, mert a jobb oldal az események átfedését nem használja ki; a levezetés viszont egyszerű és mindig érvényes.

Most bizonyítjuk az alfejezet fő állítását. A felhasznált Hoeffding-alakból indulunk ki; a saját hozzájárulás az uniókorlát alkalmazása.

Tétel (véges hipotézisosztály egyenletes korlátja). Legyen $\mathcal{H}$ véges, $|\mathcal{H}| = M$. Tegyük fel, hogy a loss $\ell(\cdot,\cdot) \in [0,1]$ minden bemenetre, és $S = \{(x_i, y_i)\}_{i=1}^{n}$ iid minta. Ekkor minden $\epsilon > 0$-ra

$$P\Big(\max_{h \in \mathcal{H}} \big|\hat{R}(h) - R(h)\big| \geq \epsilon\Big) \leq 2Me^{-2n\epsilon^2}.$$

Bizonyítás. Rögzítsünk egy $h \in \mathcal{H}$-t. Az $X_i^{(h)} = \ell(h(x_i), y_i)$ változók a $[0,1]$-be esnek; az iid minta miatt függetlenek, és $\mathbb{E}[X_i^{(h)}] = R(h)$. A kétoldali Hoeffding-korlát szerint minden $\epsilon > 0$-ra

$$P\Big(\big|\hat{R}(h) - R(h)\big| \geq \epsilon\Big) \leq 2e^{-2n\epsilon^2}. \tag{1}$$

Definiáljuk az $A_h = \{\big|\hat{R}(h) - R(h)\big| \geq \epsilon\}$ eseményt. A bal oldalon álló max akkor és csak akkor lesz legalább ϵ, ha **valamelyik** h-ra bekövetkezik A_h; tehát

$$\Big\{\max_{h \in \mathcal{H}} \big|\hat{R}(h) - R(h)\big| \geq \epsilon\Big\} = \bigcup_{h \in \mathcal{H}} A_h.$$

A uniókorlátot és (1)-et alkalmazva, felhasználva, hogy az összegben $|\mathcal{H}| = M$ tag van:

$$P\left(\bigcup_{h \in \mathcal{H}} A_h\right) \leq \sum_{h \in \mathcal{H}} P(A_h) \leq \sum_{h \in \mathcal{H}} 2e^{-2n\epsilon^2} = 2Me^{-2n\epsilon^2}.$$

Ezzel az állítást beláttuk. $\square$

A jobb oldalon álló $2Me^{-2n\epsilon^2}$ mennyiséget **felső korlátnak** (bound) nevezzük; a labor `bound` változója ezt a mennyiséget számolja.

A Hoeffding-egyenlőtlenség bizonyítását itt nem végezzük el; a tételt [P27]-re hivatkozó előfeltételként mondjuk ki. Az uniókorlátos lépés a jelen alfejezet saját, teljes levezetése.

8.2.4 Konfidenciaparaméter és empirikus kockázatminimalizálás

A tétel kényelmesebb alakját kapjuk, ha a kudarcvalószínűségre adott felső korlátot rögzítünk. Legyen $0 < \delta < 1$ a **konfidenciaparaméter**; $1 - \delta$ a megcélzott konfidenciaszint. Megoldjuk a

$$2Me^{-2n\epsilon^2} = \delta$$

egyenletet ϵ-ra:

$$\epsilon = \sqrt{\frac{\log(2M/\delta)}{2n}}. \tag{2}$$

Ezzel a tétel így fogalmazható: $1 - \delta$ valószínűséggel **egyszerre minden** $h \in \mathcal{H}$-ra

$$\big|\hat{R}(h) - R(h)\big| < \epsilon.$$

Az ϵ értelmezése ismételt mintavételi (frequentista) garancia, nem feltételes valószínűség: ha a kísérletet sok független mintán megismételnénk, a minták legfeljebb δ hányadán romolhat el az egyenletes hibasáv; az [S02] az egyváltozós átlag konfidenciaintervallumának ismételt mintavételi értelmezését tárgyalja.

Vezessük le az adaptív választásra vonatkozó következményt. Legyen $\hat{h} \in \mathcal{H}$ az **empirikus kockázatminimalizáló** (ERM), azaz $\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{R}(h)$, és legyen $h^* = \arg\min_{h \in \mathcal{H}} R(h)$ a populáció szerinti legjobb. Az egyenletes eseményen minden h-ra, így $\hat{h}$-ra és h^*-ra is igaz, hogy $|\hat{R}(h) - R(h)| < \epsilon$. Ezért

$$R(\hat{h}) \le \hat{R}(\hat{h}) + \epsilon \le \hat{R}(h^*) + \epsilon \le R(h^*) + 2\epsilon.$$

Itt a középső egyenlőtlenség az ERM definíciója: $\hat{h}$ minimalizálja $\hat{R}$-t. A 2ϵ szorzó onnan ered, hogy az egyenletes sávot két különböző hipotézisre használjuk: $\hat{h}$-ra felfelé ($R(\hat{h}) < \hat{R}(\hat{h}) + \epsilon$), h^*-ra lefelé ($\hat{R}(h^*) < R(h^*) + \epsilon$), és a két ϵ összeadódik. A szigorú egyenlőtlenségből a gyengébb, nem szigorú alak következik. A végeredmény a tanulási korlát:

Következmény. A fenti feltételek mellett legalább $1 - \delta$ valószínűséggel

$$R(\hat{h}) \le R(h^*) + 2\epsilon, \qquad \epsilon = \sqrt{\frac{\log(2M/\delta)}{2n}}.$$

Ez az, amit a rögzített h-ra vonatkozó 8.1. Kockázat, mintavétel és baseline alfejezetbeli korlát **nem** tudott garantálni: az adaptívan választott $\hat{h}$ hibája legfeljebb 2ϵ-nal haladja meg az osztály legjobb populációs kockázatát, a 2ϵ többlet pedig az uniókorlát ára.

8.2.5 Számpélda és numerikus labor

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Tekintsük a bevezető kérdést: $M = 1000$, $n = 500$, $\delta = 0{,}05$. Ekkor

$$\epsilon = \sqrt{\frac{\log(2 \cdot 1000/0{,}05)}{2 \cdot 500}} = \sqrt{\frac{\log(40000)}{1000}} \approx \sqrt{0{,}0105966} \approx 0{,}1029.$$

Tehát 95% konfidenciaszinten az empirikus kockázat minden jelöltre legfeljebb 10,3 százalékponttal tér el a populációs kockázattól, az ERM pedig legfeljebb 20,6 százalékponttal rosszabb, mint az osztály legjobbja.

A labor kódja determinisztikus: nincs véletlenszám, nincs tenzor, ezért seed, dtype és device nem alkalmazandó. A `math` modul 64 bites lebegőpontos számítást végez. A bemutatott kimenet Python 3.12.3 alatt, CPU-n, egy szálon készült.

```python
"""lab/n030_bound.py - Véges H egyenletes korlátjának numerikus számítása.

A számítás determinisztikus, ezért nincs seed, tensor dtype vagy device.
Futtatás: python lab/n030_bound.py
"""
import math
from numbers import Integral

def uniform_bound(M: int, n: int, delta: float) -> tuple[float, float]:
    """Adott M, n, delta mellett visszaadja (epsilon, bound).

    bound = 2*M*exp(-2*n*epsilon**2); a main a zárt képletű epszilont
    egy független biszekciós megoldással ellenőrzi.
    """
    if not (isinstance(M, Integral) and isinstance(n, Integral)):
        raise TypeError("M és n egész típusú legyen")
    if M <= 0 or n <= 0:
        raise ValueError("M és n pozitív egész")
    if not 0.0 < delta < 1.0:
        raise ValueError("delta a nyílt (0,1) intervallumba esik")
    eps = math.sqrt(math.log((2.0 * M) / delta) / (2.0 * n))
```

```
    bound = 2.0 * M * math.exp(-2.0 * n * eps * eps)
    return eps, bound

def bound_at(M: int, n: int, eps: float) -> float:
    """A (2) korlát értéke adott epszilon mellett."""
    return 2.0 * M * math.exp(-2.0 * n * eps * eps)

def solve_epsilon(M: int, n: int, delta: float) -> float:
    """Független biszekció: megoldja a bound_at(eps) = delta egyenletet."""
    lo, hi = 0.0, 1.0
    while bound_at(M, n, hi) > delta:
        hi *= 2.0
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        if bound_at(M, n, mid) > delta:
            lo = mid
        else:
            hi = mid
    return 0.5 * (lo + hi)

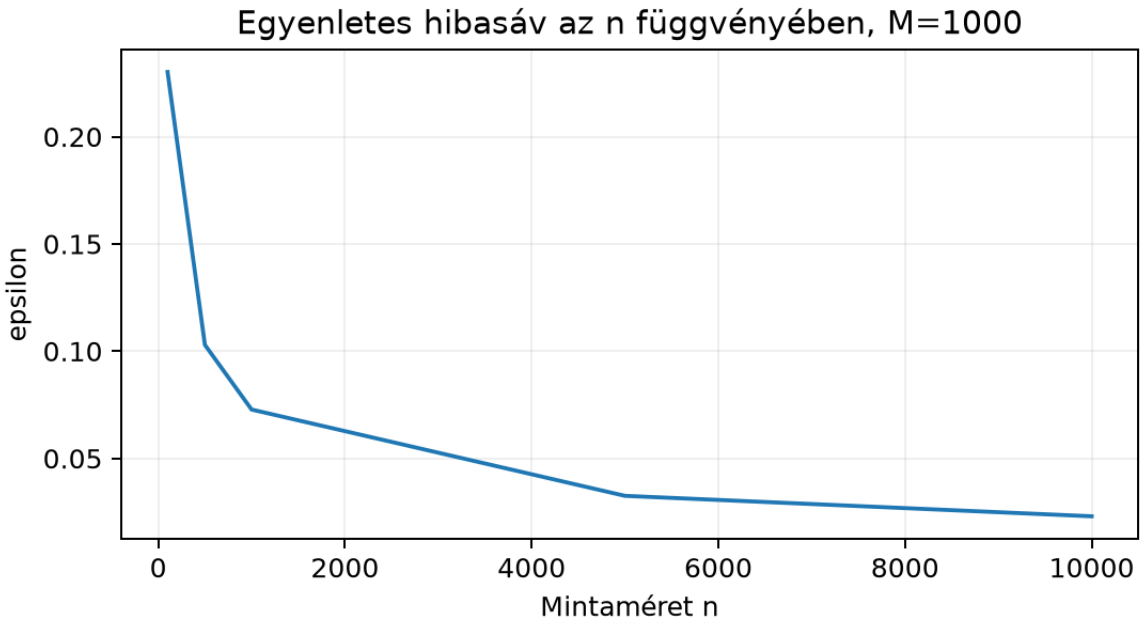
def main() -> None:
    delta = 0.05
    tolerance = 1e-12  # relatív tolerancia a zárt képlet és a biszekció összevetéséhez
    print(f"delta = {delta}")
    for M in (10, 1000, 1_000_000):
        print(f"M = {M}")
        for n in (100, 500, 1000, 5000, 10_000):
            eps, bound = uniform_bound(M, n, delta)
            eps_ref = solve_epsilon(M, n, delta)
            ok = abs(eps - eps_ref) <= tolerance * max(1.0, eps_ref)
            assert eps > 0.0
            assert abs(bound - delta) <= tolerance * max(1.0, delta)
            assert ok
            print(f"  n={n:6d}  epsilon={eps:.6f}  bound={bound:.6e}  "
                  f"ellenőrzés={ok}")

if __name__ == "__main__":
    main()
```

Az alábbi értékek a megadott kóddal futtatott kimenetből származnak (Python 3.12.3, CPU, 1 szál); a számítás determinisztikus, ezért seed nem értelmezhető.

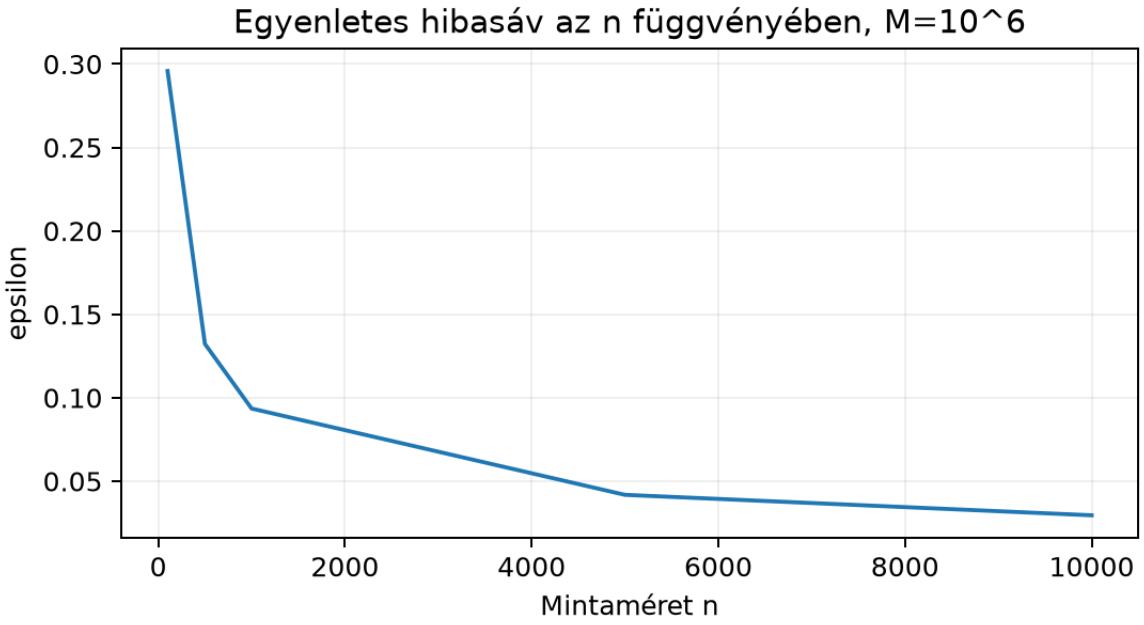
M	n	ϵ
10	100	0,173082
10	1000	0,054733
10	10000	0,017308
1000	100	0,230181
1000	1000	0,072790
1000	10000	0,023018
10^6	100	0,295841
10^6	1000	0,093553
10^6	10000	0,029584

A `bound` oszlop minden sorban $\delta = 0{,}05$-öt ad vissza a lebegőpontos pontosság erejéig. Az `ellenőrzés` jelző nem önmagát ellenőrzi: a zárt képletű ϵ-t egy független biszekciós megoldással veti össze, amely a $2Me^{-2n\epsilon^2} = \delta$ egyenletet oldja meg. Ez csak azt igazolja, hogy a kód inverz képlete algebrailag konzisztens; a Hoeffding-korlát érvényességét nem.



8.2. ábra: Egyenletes hibasáv az n függvényében, M=1000

A megadott kóddal futtatott kimenetből; a számítás determinisztikus. delta=0.05.



8.3. ábra: Egyenletes hibasáv az n függvényében, M=10^6

A megadott kóddal futtatott kimenetből; a számítás determinisztikus. delta=0.05.

A két ábra ugyanazt a monoton csökkenő tendenciát mutatja: rögzített M mellett ϵ az n növelésével csökken, és $M = 10^6$ esetén minden n-re nagyobb, mint $M = 1000$-nél. Az ábrák tengelye lineáris, ezért a kis n tartományban a görbék meredekebben indulnak; a csökkenés üteme a $\sqrt{\log(2M/\delta)/(2n)}$ képlet szerint $1/\sqrt{n}$.

Mikor használhatatlanul laza a korlát? A képlet egy logaritmust és egy négyzetgyököt tartalmaz. $M = 10^6$, $n = 1000$ esetén $\epsilon \approx 9{,}4$ százalékpont; egy $[0,1]$-beli losson a teljes $[0,1]$ tartományhoz képest ez hatalmas, és az ERM-korlát $2\epsilon \approx 18{,}7$ százalékpont. A gyakorlati összehasonlításokban ennél jóval kisebb különbségek is döntőek. Két ok miatt laza a korlát: (i) a Hoeffding-korlát maga sem használja ki az eloszlás szórását; (ii) az uniókorlát a legrosszabb esetben teljesen független eseményeket feltételező összeget ad, pedig a hipotézisek kockázatbecslései erősen korrelálnak. Ezért a $2Me^{-2n\epsilon^2}$ alakot **felső garanciaként**, nem pedig éles predikcióként kell olvasni. Kis n-re vagy nagy M-re a korlát gyakorlati döntésre alkalmatlan lehet, ettől még matematikailag helyes.

8.2.6 Gyakori tévedés: korlátlan cross-entropy

A tétel feltétele, hogy a loss minden bemenetre $[0,1]$-be essen. A többosztályos cross-entropy a 7.2. Softmax és többosztályos cross-entropy alfejezetből logitsokra:

$$\ell(z, y) = -z_y + \log \sum_{k=1}^{K} e^{z_k}.$$

Ez **nem** korlátos 1-gyel. Legyen $K = 2$, a helyes osztály $y = 0$, a logit pedig $z = (0, 10)$. Ekkor

$$\ell(z, y) = \log(1 + e^{10}) \approx 10{,}000045 > 1.$$

A logit növelésével a loss tetszőlegesen nagy lehet, tehát a **korlátlan logitosztály** fölött nem létezik olyan rögzített B, amellyel a $[0,1]$-re normálás a feladat tartalmi megváltoztatása nélkül elvégezhető lenne. Ez a cross-entropy loss ráadásul nem is $\ell : \mathcal{Y} \times \mathcal{Y} \to [0,1]$ alakú: a bemenete logitvektor és címke, nem két címke.

Nézzünk konkrét ellenpéldát arra, hogy a feltétel nélkül a konklúzió ténylegesen hamis lehet. Legyen $\mathcal{H} = \{h\}$, tehát $M = 1$, $n = 1$. Tegyük fel, hogy az egyetlen megfigyelés y célja $\frac{1}{2}$ valószínűséggel 0, $\frac{1}{2}$ valószínűséggel 1, a modell pedig mindig $z = (0, 10)$ logitot ad. A loss $y = 0$ esetén ≈ 10, $y = 1$ esetén ≈ 0; így $R(h) \approx 5$. Az egyelemű mintán $\hat{R}(h)$ vagy ≈ 10, vagy ≈ 0, tehát

$$P\Big(|\hat{R}(h) - R(h)| \geq 1\Big) = 1,$$

hiszen az eltérés mindkét kimenetelben ≈ 5. A $[0,1]$-korlátos tétel viszont $\epsilon = 1$-re $2e^{-2} \approx 0{,}2707$ felső korlátot adna, ami nyilvánvalóan hamis erre a nem korlátos lossra. A tétel érvényes, a feltételei azonban nem teljesülnek: **változtatás nélkül nem alkalmazható** korlátlan cross-entropyra. Ha a loss $[0, B]$-be esik, akkor a $[0,1]$-re skálázott változatra alkalmazható a korlát, de a B skálának megfelelően ϵ is B-vel skálázódik: $\epsilon = B\sqrt{\log(2M/\delta)/(2n)}$. A korlátlan logitosztály helyett gyakorlati javítás a logitok előzetes vágása: ha $|z_k| \leq c$ minden k-ra, akkor K osztályra $\ell(z, y) = -z_y + \log \sum_k e^{z_k} \leq c + \log(Ke^c) = \log K + 2c$, tehát $B = \log K + 2c$ választható. Például $K = 2$, $c = 10$ esetén $B = \log 2 + 20 \approx 20{,}69$; ezzel $M = 1000$, $n = 500$, $\delta = 0{,}05$ mellett a skálázott sáv $\epsilon \approx 20{,}69 \cdot 0{,}102940 \approx 2{,}13$ (számított érték), ami a $[0,1]$ skálán értelmezhetetlenül nagy: a vágás önmagában nem teszi informatívvá a korlátot.

8.2.7 Feladatok

8.2.7.1 F1 — Fogalmi ellenőrzés (nehézség: 1)

Fogalmazza meg saját szavaival a uniókorlátot, és magyarázza el, miért lép be az M szorzó a $2Me^{-2n\epsilon^2}$ képletbe. Adjon két olyan A_1, A_2 eseményt, amelyre a uniókorlát egyenlőséggel teljesül, és kettőt, amelyre a korlát szigorúan nagyobb a tényleges valószínűségnél. Elvárt eredmény: definíció, az M szorzó eredete és négy konkrét esemény indoklással.

8.2.7.2 F2 — Levezetési alkalmazás (nehézség: 2)

Legyen $M = 5000$, $n = 2000$, $\delta = 0{,}01$. Számítsa ki a (2) képlettel ϵ-t, majd az egyenletes esemény feltételével vezesse le az ERM-hibakorlátot: igazolja, hogy legalább 99% valószínűséggel $R(\hat{h}) \leq R(h^*) + 2\epsilon$, és adja meg a 2ϵ számértékét. Elvárt eredmény: ϵ levezetése, a három egyenlőtlenség mindegyikének indoklása, végső számérték.

8.2.7.3 F3 — Implementációs feladat (nehézség: 2)

Írjon Python függvényt, amely az (M, n, δ) hármasból a (2) képlet szerint ϵ-t és a $\delta = 2Me^{-2n\epsilon^2}$ azonosság numerikus ellenőrzését adja vissza. Tesztelje $(M, n, \delta) = (1000;\ 500;\ 0{,}05)$-re. A kód tartalmazzon típus- és tartományellenőrzést. Elvárt eredmény: futtatható kód, a fenti paraméterre $\epsilon \approx 0{,}102940$ és a `bound` visszaadása $\delta = 0{,}05$ körül 10^{-12} relatív tolerancián belül.

8.2.7.4 F4 — Hibakeresés (nehézség: 2)

A következő függvény hibás:

```
def rossz_epsilon(M: int, n: int, delta: float) -> float:
    import math
    return math.sqrt(math.log(M / delta) / (2 * n))
```

Lokalizálja a hibát, magyarázza el a matematikai következményét a garantált konfidenciaszintre, javítsa a kódot, és $(M, n, \delta) = (1000;\ 500;\ 0{,}05)$-tel mutassa meg, hogy a hibás érték kisebb, mint a helyes 0,102940. Visszaellenőrzésképpen számítsa ki, hogy a hibás ϵ mellett a $2Me^{-2n\epsilon^2}$ mennyiség mennyivel haladja meg a megcélzott δ-t. Elvárt eredmény: hiba oka, következmény, javított kód és két numerikus kontroll.

8.2.7.5 F5 — Kísérlettervezési/transzfer (átviteli) feladat (nehézség: 3)

Tervezzen Monte Carlo (ismételt véletlen mintavételen alapuló) szimulációt annak vizsgálatára, hogy egy konkrét, véges hipotézisosztályon a tényleges maximális eltérés mennyivel kisebb a Hoeffding-unókorlátnál. Válasszon $M = 100$ hipotézist, p_h Bernoulli-paraméterekkel (a loss 1 valószínűséggel p_h, 0 valószínűséggel $1 - p_h$) a $[0, 1]$-beli lossra, $n = 200$ mintát, rögzített seedet és $R = 5000$ ismétlést. Írja le a protokollt: megfigyelési egység, a mérendő mennyiség (a minta feletti $\max_h |\widehat{R}(h) - R(h)|$ eloszlása), a Hoeffding-baseline, a Monte Carlo becslés standard hibája, és a következtetés érvényességi határa. A feladat nyitott; nem kell futtatni. Elvárt eredmény: legalább egy teljes protokoll, az alternatívák indoka, és annak megfogalmazása, hogy a szimuláció miért nem cáfolhatja az általános tételt.

8.2.8 Megoldások és indoklás

8.2.8.1 F1 megoldása

A uniókorlát: $P(\bigcup_{j=1}^{M} A_j) \le \sum_{j=1}^{M} P(A_j)$. A $2Me^{-2n\epsilon^2}$ képletben a $2e^{-2n\epsilon^2}$ egyetlen hipotézisre vonatkozó kudarc-korlát; mivel M hipotézis van, és a rossz esemény az, hogy **bármelyik** hipotézisre sérül a sáv, a unió valószínűségét a valószínűségek összege korlátozza, ez az M-szeres szorzó.

Egyenlőség például: a kockadobásnál $A_1 = \{1\}$, $A_2 = \{2\}$, ekkor $P(A_1 \cup A_2) = \frac{2}{6} = \frac{1}{6} + \frac{1}{6}$. Másik egyenlőségi pár: $A_1 = \{1,2\}$, $A_2 = \{3,4\}$, ahol az unió valószínűsége $\frac{4}{6}$, az összeg pedig $\frac{2}{6} + \frac{2}{6} = \frac{4}{6}$. Szigorú egyenlőtlenség: $A_1 = \{1\}$, $A_2 = \{1\}$, ekkor a unió valószínűsége $\frac{1}{6}$, az összeg $\frac{2}{6}$. Egy másik szigorú pár: $A_1 = \{\text{páros}\} = \{2,4,6\}$, $A_2 = \{1,2\}$; a unió $\{1,2,4,6\}$ valószínűsége $\frac{4}{6}$, az összeg $\frac{3}{6} + \frac{2}{6} = \frac{5}{6} > \frac{4}{6}$.

8.2.8.2 F2 megoldása

Behelyettesítünk a (2) képletbe. log természetes:

$$\epsilon = \sqrt{\frac{\log(2 \cdot 5000/0{,}01)}{2 \cdot 2000}} = \sqrt{\frac{\log(10^6)}{4000}} = \sqrt{\frac{13{,}81551}{4000}} \approx \sqrt{0{,}00345388} \approx 0{,}05877.$$

Az egyenletes esemény $1 - \delta = 0{,}99$ valószínűséggel bekövetkezik; ezen az eseményen minden $h \in \mathcal{H}$-ra $|\hat{R}(h) - R(h)| < \epsilon$. Az ERM $\hat{h}$ és a populációs optimum h^* egyaránt $\mathcal{H}$-beli, ezért (i) $R(\hat{h}) < \hat{R}(\hat{h}) + \epsilon$, hiszen az egyenletes sáv $\hat{h}$-ra is igaz; (ii) $\hat{R}(\hat{h}) \le \hat{R}(h^*)$, mert $\hat{h}$ minimalizálja az empirikus kockázatot; (iii) $\hat{R}(h^*) < R(h^*) + \epsilon$, mert a sáv h^*-ra is igaz. Összekapcsolva $R(\hat{h}) < R(h^*) + 2\epsilon$. A számérték $2\epsilon \approx 0{,}1175$, tehát legalább 99% valószínűséggel az ERM populációs kockázata legfeljebb 11,75 százalékponttal haladja meg az osztály optimumát.

8.2.8.3 F3 megoldása

A feladat saját függvényt kér, ezért a labor `uniform_bound` függvényét használjuk mintaként, de a megoldásban önállóan is megadjuk:

```
import math
from numbers import Integral

def epsilon_bound(M, n, delta):
    if not (isinstance(M, Integral) and isinstance(n, Integral)):
        raise TypeError("M és n egész típusú legyen")
    if M <= 0 or n <= 0:
        raise ValueError("M és n pozitív egész")
    if not 0.0 < delta < 1.0:
        raise ValueError("delta a nyílt (0,1) intervallumba esik")
    eps = math.sqrt(math.log((2.0 * M) / delta) / (2.0 * n))
    bound = 2.0 * M * math.exp(-2.0 * n * eps * eps)
    return eps, bound

eps, bound = epsilon_bound(1000, 500, 0.05)
print(eps, bound)
# 0.1029399569316797 0.0500000000000004
assert abs(bound - 0.05) <= 1e-12 * max(1.0, 0.05)
try:
    epsilon_bound(1000.0, 500, 0.05)
except TypeError as exc:
```

```
    print(type(exc).__name__, exc)
# TypeError M és n egész típusú legyen
```

A megadott paraméterre $\epsilon \approx 0{,}102940$, a `bound` pedig $\delta = 0{,}05$ körül 10^{-12} relatív toleranción belül van. Az ellenőrzés `abs(bound - delta) <= 1e-12 * max(1.0, delta)`: a `bound` a `math.exp` és a `math.log` inverz műveletei miatt lesz 0,05 körül. Szélsőeset-kontroll: $M \leq 0$, $n \leq 0$ vagy $\delta \notin (0, 1)$ `ValueError`-t ad; a nem egész `M` vagy `n` pedig `TypeError`-t, amit a fenti `try`-blokk ellenőriz.

8.2.8.4 F4 megoldása

A hiba: a függvény $\log(M/\delta)$-t számol $\log(2M/\delta)$ helyett, tehát a **kétoldali** Hoeffding-korlát 2-es szorzóját elveszíti. Következmény: az ϵ érték kisebb lesz, mint a megcélzott δ-hoz tartozó garantált sáv; az így jelentett konfidenciaszint hamis, mert a tényleges kudarc-korlát δ-nál nagyobb.

Javítás: a logaritmus argumentuma `(2.0 * M) / delta` legyen:

```
def helyes_epsilon(M: int, n: int, delta: float) -> float:
    import math
    return math.sqrt(math.log((2.0 * M) / delta) / (2.0 * n))
```

Numerikus kontroll: $(M, n, \delta) = (1000;\ 500;\ 0{,}05)$-re a hibás érték

$$\epsilon_{\text{hib}} = \sqrt{\frac{\log(1000/0{,}05)}{1000}} = \sqrt{\frac{\log(20000)}{1000}} \approx \sqrt{0{,}00990349} \approx 0{,}099516,$$

míg a helyes $\epsilon \approx 0{,}102940$; a hibás érték valóban kisebb. A hibás ϵ-nal számolt kudarc-korlát

$$2Me^{-2n\epsilon_{\text{hib}}^2} = 2000\, e^{-1000 \cdot 0{,}00990349} \approx 2000\, e^{-9{,}90349} \approx 0{,}1000,$$

ami a megcélzott $\delta = 0{,}05$ **kétszerese**; az ellenőrzés tehát kimutatja a konfidenciaszint sérülését.

8.2.8.5 F5 megoldásminta

Kérdés: mennyivel kisebb egy konkrét osztályon a tényleges $\max_h |\hat{R}(h) - R(h)|$, mint a Hoeffding–uniókorlát ϵ értéke?

Protokoll. Rögzítsük $M = 100$ hipotézist; a h_j jelölthöz tartozó loss legyen Bernoulli(p_j): a loss 1 valószínűséggel p_j, 0 valószínűséggel $1 - p_j$, ezért várható értéke p_j, azaz $R(h_j) = p_j$ ismert. Legyen $p_j = j/101$. Minden $r = 1, \ldots, R$ ismétlésben generáljunk $n = 200$ független megfigyelést, minden j-re a lossokat; a megfigyelési egység az ismétlésenkénti minta, a mért statisztika $T_r = \max_{j=1}^{M} |\hat{p}_j^{(r)} - p_j|$. A baseline a (2) képletből $M = 100$, $n = 200$, $\delta = 0{,}05$ mellett:

$$\epsilon = \sqrt{\frac{\log(2 \cdot 100/0{,}05)}{400}} \approx \sqrt{\frac{\log(4000)}{400}} \approx 0{,}1440.$$

A becsült túllépési arány $\hat{P} = \frac{1}{R}\sum_{r=1}^{R} \mathbf{1}[T_r \geq \epsilon]$. Mivel a $T_r \geq \epsilon$ indikátorok független Bernoulli(π) változók, $\text{Var}(\hat{P}) = \pi(1-\pi)/R$, ezért a standard hiba $\sqrt{\pi(1-\pi)/R}$, amelyet

$\hat{P}$-vel becsülünk: $\sqrt{\hat{P}(1-\hat{P})/R}$. Ez $R = 5000$-nél legfeljebb $1/(2\sqrt{5000}) \approx 0{,}0071$. A seed rögzítése (például `np.random.default_rng(20261020)`) biztosítja az újrafuttathatóságot.

Egy lehetséges futtatható vázlat:

```
import numpy as np

rng = np.random.default_rng(20261020)
M, n, R = 100, 200, 5000
p = np.arange(1, M + 1) / (M + 1)
eps = np.sqrt(np.log(2 * M / 0.05) / (2 * n))
exceed = 0
for _ in range(R):
    losses = rng.random((M, n)) < p[:, None]
    p_hat = losses.mean(axis=1)
    T = np.max(np.abs(p_hat - p))
    exceed += T >= eps
p_exceed = exceed / R
se = np.sqrt(p_exceed * (1 - p_exceed) / R)
print(f"{eps:.6f} {p_exceed:.4f} {se:.6f}")
```

A megadott seeddel futtatott példa (NumPy 2.5.4, CPU): `0.143997 0.0012 0.000490`. A mért túllépési arány jóval a $\delta = 0{,}05$ alatt van, ami ezen az eloszláson a uniókorlát lazaságát szemlélteti.

Értelmezési korlát. A szimuláció egyetlen eloszláson mér; nem cáfolhatja az általános tételt, mert a tétel minden $[0,1]$-beli lossra vonatkozó felső korlát. Ha $\hat{P}$ jóval kisebb δ-nál, az a uniókorlát lazaságát szemlélteti ezen az eloszláson, nem a képlet hibáját. Alternatívák: p_j változtatása, közös megfigyeléseken számolt veszteségek, vagy a maximális eltérés eloszlásának hisztogramja; ezek a korreláció hatását vizsgálnák, de mindegyik csak illusztráció.

8.2.9 Önellenőrzés és fogalmi áttekintés

1. Miért nem elég a rögzített h-ra szóló Hoeffding-korlát, ha a legjobb h-t ugyanazon a mintán választjuk? Rövid válasz: a választás adaptív, a kiválasztott h függ a mintától, így az egyetlen korlát nem vonatkozik rá; az uniókorlát minden jelöltre egyszerre érvényes.
2. Ha M nő, mi történik ϵ-nal rögzített n és δ mellett? Nő, $\sqrt{\log M}$ szerint; a korlát egyre lazább.
3. Melyik feltételt sérti a logits-alapú cross-entropy? A $[0,1]$-korlátosságot; a loss tetszőlegesen nagy lehet, tehát a tétel változtatás nélkül nem alkalmazható.

A 8.1. Kockázat, mintavétel és baseline rögzített hipotézisre mondta ki a Hoeffding-korlátot; most azt egyenletessé tettük. A 7.2. Softmax és többosztályos cross-entropy cross-entropy definíciója adta a nem korlátos ellenpéldát. A 4.2. Parciális derivált és gradiens gradiensfogalma itt tudatosan nem szerepel feltételként: a statisztikai garanciához semmilyen differenciálhatóság nem kell, csak korlátosság, iid minta és véges $\mathcal{H}$.

8.2.10 Rövid összefoglaló

Ha a hipotézisosztály véges, a rögzített hipotézisre szóló Hoeffding-korlátot uniókorláttal minden jelöltre kiterjeszthetjük: $[0,1]$-korlátos loss és n elemű iid minta esetén legalább $1-\delta$ valószínűséggel minden h-ra $|\hat{R}(h) - R(h)| \le \epsilon$, ahol $\epsilon = \sqrt{\log(2M/\delta)/(2n)}$. Ebből az ERM-hibakorlát $R(\hat{h}) \le R(h^*) + 2\epsilon$; ez az adaptív választás első korrekt garanciája. A korlát gyakran túl laza a gyakorlati döntéshez, mert a Hoeffding-lépés és az uniókorlát is konzervatív. Korlátlan cross-entropyra a tétel változtatás nélkül nem alkalmazható.

8.2.11 Következő alfejezet

A 8.3. Kapacitás, regularizálás és modellválasztás a kapacitást, a regularizálást és a modellválasztást kapcsolja ehhez a korláthoz: a $2Me^{-2n\epsilon^2}$ alak megmutatja, miért növeli a nagyobb hipotézisosztály a statisztikai hibát, és hogyan ellensúlyozhatja a korlátozott keresési tér a mintakorlátot. A következő egység ezt az elvet viszi át a gyakorlati modellválasztási döntésekre.

8.2.12 Források

- [P27] Hoeffding, W. (1963). Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301), 13–30. https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830. Hozzáférés: 2026-10-30.
- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm.

8.3 Kapacitás, regularizálás és modellválasztás

8.3.1 Cél, előismeret és induló kérdés

Ez az alfejezet a 8.2. Véges hipotézisosztály egyenletes korlátja véges hipotézisosztályra adott egyenletes hibakorlátját ülteti át gyakorlati döntési keretbe. A kérdés: ha több modellt tanítunk, és a legjobbat a validációs halmazon választjuk, mit mondhatunk a végső teszteredményről? Az előismeret: a Hoeffding-korlát és az uniókorlát (8.2. Véges hipotézisosztály egyenletes korlátja), a loss-szerződések (7.3. Bináris, multilabel és regressziós cél), valamint a gradiensalapú tanítás (4.3. Jacobi-mátrix, láncszabály és VJP). Induló becslés: tíz modellből a legjobb validációs pontosság várhatóan mennyivel haladja meg a valós pontosságot, ha mindegyik modell valódi pontossága 0,5?

8.3.2 Kapacitás és bias–variance

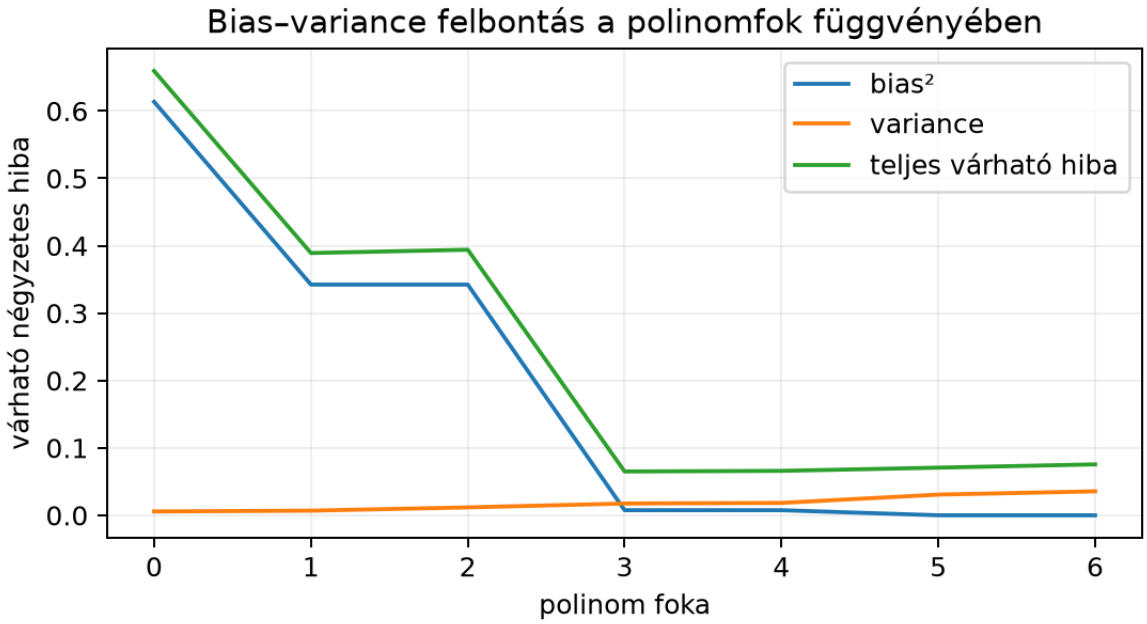
Legyen D tanítóhalmaz, $\hat{f}_D$ a belőle tanított modell, és f a valódi regressziós cél. Egy rögzített x pontban, ahol $y = f(x) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\mathbb{V}[\varepsilon] = \sigma^2$, és $\hat{f}_D$ független a zajtól, a négyzetes hiba várható értéke három tagra bontható:

$$\mathbb{E}_{D,\varepsilon}[(\hat{f}_D(x) - y)^2] = (\mathbb{E}_D[\hat{f}_D(x)] - f(x))^2 + \mathbb{V}_D[\hat{f}_D(x)] + \sigma^2.$$

Az első tag a **bias²**, a második a **variance**, a harmadik az irreducibilis zaj. A felbontás a kereszttagok eltűnéséből adódik: írjuk fel $\hat{f}_D(x) - y = (\hat{f}_D(x) - \mathbb{E}_D[\hat{f}_D(x)]) + (\mathbb{E}_D[\hat{f}_D(x)] - f(x)) - \varepsilon$ alakban, és emeljük négyzetre. Várható értéket véve a lineáris kereszttagok nullák: az első tényező várható értéke nulla, a második konstans, a zaj pedig független $\hat{f}_D$-től és nulla várható értékű. Ez a felbontás csak négyzetes hibára érvényes; cross-entropyra nem ugyanez a képlet. A bias nem pusztán a modellcsalád közelítési hibája: függ a tanító algoritmustól és a regularizálástól is ($\mathbb{E}_D[\hat{f}_D(x)]$ az algoritmus kimenetének átlaga). A bal oldali várható érték a tanítóminta D és a zaj ε szerint is átlagol. A bias tehát a modellcsalád és a tanító algoritmus együttes, átlagos közelítési hibája (a tanítóminta eloszlása szerint), a variance pedig a modell érzékenysége a konkrét tanítómintára.

A kapacitás növelésének tipikus hatását szemlélteti a következő, képletből számolt ábra: a polinomfok növelésével a bias² csökken, a variance nő, a teljes várható hiba pedig egy pontig

csökken, majd emelkedik. Az ábra a $[-1, 1]$ intervallum hét rögzített, egyenletes mintapontjára illesztett polinomra vonatkozik; a görbe alakja a mintaelemszámtól, a mintapontok intervallumától és a zaj nagyságától is függ, ezért nem univerzális kapacitásgörbe.



8.4. ábra: Bias–variance felbontás a polinomfok függvényében

Képletből számolt szemléltetés: a [-1,1] intervallum hét rögzített, egyenletes tanítópontján a sin(3x) függvényre legkisebb négyzetekkel illesztett polinom, x0=0,3, ²=0,04; nem mérés.

A **kapacitás** (capacity) a hipotézisosztály kifejezőképessége. A 8.2. Véges hipotézisosztály egyenletes korlátja jelölésében $|\mathcal{H}| = M$. A nagyobb M általában csökkenti a bias-t, mert jobb közelítő függvény kerülhet a készletbe, de növeli a statisztikai hibát: a Hoeffding-korláton [P27] alapuló uniókorlát $\sqrt{\log(2M/\delta)/(2n)}$ sugara M-mel nő. A kapacitás nem azonos a paraméterszámmal, és nem ad önmagában számszerű jóslatot a teszthibára.

8.3.3 Regularizálás: büntetőtag és korai leállás

A büntetőtag (penalty) explicit korlátozást ad a veszteséghez. L2-regularizálás esetén a minimalizált cél

$$J(W) = \frac{1}{N}\sum_{i=1}^{N} \ell(f(x_i; W), y_i) + \frac{\lambda}{2}\|W\|_F^2.$$

Itt ℓ a 7.3. Bináris, multilabel és regressziós cél szerinti pontonkénti loss, W a modell súlyainak mátrixba rendezett gyűjteménye, N a tanítóminta elemszáma (a 8.2. Véges hipotézisosztály egyenletes korlátja korlátjában n jelöli ugyanezt), és $\hat{R}(W) = \frac{1}{N}\sum_{i=1}^{N} \ell(f(x_i; W), y_i)$ az empirikus kockázat.

A gradiens $\nabla_W J = \nabla_W \hat{R}(W) + \lambda W$. A λW tag nem a láncszabályon keresztül terjed: a penalty minden W paraméterre külön, közvetlenül adódik a gradienshez, a $\nabla_W \hat{R}(W)$ adatgradienst pedig a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetbeli backpropagation számolja. A büntetőtag

hatása az, hogy nulla felé húzza a súlyokat; nem szűkíti szó szerint a reprezentálható függvények halmazát (nagy normájú súlyok is elérhetők), hanem választási preferenciát ad a kisebb normájú megoldások felé, és így csökkenti az effektív kapacitást. Az AdamW súlycsökkenése nem azonos a loss-ba írt L2 Adam alatti hatásával; itt a közönséges SGD–L2 esetet tárgyaljuk, ahol a büntetés a loss része, és a gradienshez a λW tagot adja, míg az AdamW a súlycsökkenést az optimalizáló lépésében, a gradiensfrissítéstől elkülönítve alkalmazza.

A **korai leállás** (early stopping) implicit regularizálás: a tanítást a validációs hiba növekedése előtt leállítjuk. A korai leállás a bejárt pályát korlátozza: tipikusan nem engedi a tanítóhalmaz teljes illesztését, de ezt nem garantálja (a tanítóadatot akár tökéletesen is illesztheti), és a validációs hiba növekedését előre nem tudja megelőzni, csak utólag, a minimum után észleli. Nincs zárt képlete, és a leállítási pont maga is a validációs halmaztól függ.

8.3.4 Validáció és beágyazott választás

A **validáció** a modellválasztáshoz használt, a tanításból kizárt adatrész. A **beágyazott választás** két ciklust jelent: a belső ciklus adott hiperparaméterrel tanít, és a belső validáció dönt a korai leállásról; a külső ciklus a hiperparamétereket a külső validációs pontszám alapján választja. Egyetlen külső validációs halmazzal hiperparamétert választó eljárás nem ad a beágyazott (*nested*) keresztvalidációval egyenértékű, független külső becslést: a külső validációs pontszám a választásba is bement. A beágyazott eljárásban a belső train/val szerepe a hiperparaméter-választás, a külső kiértékelés szerepe pedig az így választott eljárás független becslése, amelyet a belső ciklus nem lát; költsége a belső keresés ismétlése minden külső hajtásra ($K_{ki} \cdot K_{be} \cdot |\text{rács}|$ tanítás, ahol K_{ki} a külső, K_{be} a belső hajtások száma). A zárótesztet csak a legvégén, egyszer, a már rögzített modellen értékeljük. Ha a tesztet a keresés közben akár egyszer is használjuk, adaptív torzítást viszünk a végső becslésbe.

A beágyazott választás sémája röviden:

Szerep	Adat	Döntés
Belső tanítás	a külső tanítóhalmaz $K_{be}-1$ része	tanítás adott hiperparaméterrel
Belső validáció	a külső tanítóhalmaz maradék $1/K_{be}$ része	korai leállás és belső pontszám
Külső kiértékelés	az adott külső hajtás $1/K_{ki}$ része	a kiválasztott eljárás független pontszáma
Záróteszt	a keresésből kizárt, elkülönített rész	egyszeri, végső becslés

8.3.5 Ellenpélda: a legjobb validációs pontszám torzítása

Állítás (cáfolandó). Ha M modell közül a validációs halmazon legjobb pontosságút választjuk, akkor a kiválasztott modell validációs pontossága torzítatlan becslése a tesztpontosságnak.

Ellenpélda. Legyen $M = 10$ modell, és mindegyik valós pontossága $p = 0{,}5$ bármely iid tesztmintán. A validációs halmaz $n_{val} = 1$ elemű. Minden h_i modell helyesség-indikátora ezen az egy mintán $X_i \sim \text{Bernoulli}(0{,}5)$, és az X_i változókat függetlennek tételezzük fel. Ez erős feltevés: a valóságban az azonos validációs halmazon kiértékelt modellek hibái korreláltak, ezért a tényleges kiválasztási torzítás kisebb lehet az itt számítottnál; az $n_{val} = 1$ szélső eset a mechanizmust mutatja, nem tipikus nagyságot. A kiválasztott modell $\widehat{h} = \arg\max_i X_i$.

A kiválasztott modell validációs pontossága $\max_i X_i$. Várható értéke

$$\mathbb{E}[\max_i X_i] = \mathbb{P}(\max_i X_i = 1) = 1 - \mathbb{P}(\text{mind } X_i = 0) = 1 - 0{,}5^{10} = 1023/1024 \approx 0{,}9990.$$

A tesztpontosság ezzel szemben minden modellre 0,5 várható értékű, mert a modell valós pontossága 0,5, és a tesztminta független a validációs mintától. Tehát a kiválasztott modell validációs pontossága várhatóan 0,9990, míg a tesztpontosság várhatóan 0,5. A különbség 0,4990; ez a kiválasztási folyamat által okozott felfelé torzítás. A cáfolandó állítás hamis. A torzítás forrása nem a modellek hibája, hanem az, hogy a maximum operátor kiválasztja a validációs zaj kedvező irányú ingadozását. Ez egyben az alfejezet induló kérdésének válasza: tíz független, 0,5 valós pontosságú modell és $n_{val} = 1$ mellett a legjobb validációs pontosság várhatóan 0,499-cel haladja meg a valós pontosságot.

8.3.6 Kézi számpélda és ellenőrző kód

Kisebb eset: $M = 3$, $n_{val} = 2$, $p = 0{,}5$. Minden modell validációs pontossága $Y_i \sim \text{Binom}(2; 0{,}5)/2$, lehetséges értékei 0, 0,5, 1, rendre 1/4, 1/2, 1/4 valószínűséggel. A maximum eloszlása:

- $\mathbb{P}(\max Y_i = 1) = 1 - (3/4)^3 = 37/64$,
- $\mathbb{P}(\max Y_i = 0{,}5) = (3/4)^3 - (1/4)^3 = 26/64$,
- $\mathbb{P}(\max Y_i = 0) = 1/64$.

Ezért $\mathbb{E}[\max Y_i] = 1 \cdot 37/64 + 0{,}5 \cdot 26/64 = 50/64 = 0{,}78125$, szemben a 0,5 valós értékkel. A torzítás 0,28125.

Az alábbi minimális kód ugyanezt a kézi számítást ellenőrzi közvetlen eloszlással; a futtatott kimenet (Python 3.12.3, egy szál) egyezik a fenti értékekkel.

```
# munkakönyvtár: lab/
# Futtatva; a kimenet a fenti kézzel levezetett értékeket adja vissza.
from fractions import Fraction
from math import comb

M, n_val, p = 3, 2, Fraction(1, 2)

def binom_prob(n, k):
    return Fraction(comb(n, k)) * p**k * (1 - p)**(n - k)

# P(max <= v) = P(Y <= v)^M, v itt a pontosság (0, 0.5, 1)
def cdf_max(v):
    # v pontosság; a mintaelemszám egész, ezért az ellenőrzés kizárja,
    # hogy float kerekítés miatt rossz felső indexet használjunk.
    k_max = v * n_val
    assert Fraction(k_max) == k_max, "a v*n_val szorzatnak egésznek kell lennie"
    return sum(binom_prob(n_val, k) for k in range(int(k_max) + 1)) ** M

cdf_0 = cdf_max(0)                 # P(max = 0)
cdf_half = cdf_max(Fraction(1, 2)) # P(max <= 0.5)
cdf_1 = cdf_max(1)                 # P(max <= 1) = 1

p_0 = cdf_0
p_half = cdf_half - cdf_0
p_1 = cdf_1 - cdf_half
expected_max = p_0 * 0 + p_half * Fraction(1, 2) + p_1 * 1
print(p_0, p_half, p_1, expected_max, float(expected_max))
# Kimenet: 1/64 13/32 37/64 25/32 0.78125
```

8.3.7 Labor: rögzített költségű modellválasztás érintetlen záróteszttel

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adatokon a feladat egy rögzített költségű keresés megtervezése. Az adat: `data/images.npy` $(360, 1, 24, 24)$ float32 NCHW, `data/labels.npy` int64, `data/metadata.csv` a split oszloppal. A train/val/test halmazok jelenetszinten diszjunktak, azaz egy `scene_id` minden képe ugyanabba a részhalmazba kerül: 240/60/60 kép.

Protokoll (a mért futás beállításai):

1. **Rögzített keresési tér:** rejtett méret $\{16, 32\}$, L2-$\lambda \in \{0, 10^{-4}, 10^{-3}\}$, összesen 6 konfiguráció.
2. **Belső ciklus:** minden konfigurációt 30 epochig tanítunk a train halmazon, batch méret 32, SGD tanulási ráta 0,01, seed 20261020. A belső validáció minden epoch végén a val halmazon számol pontosságot, és a belső legjobb epoch súlyát őrizzük meg.
3. **Konfigurációválasztás:** a 6 konfiguráció közül a belső legjobb val pontosság szerint választunk (ez az egyetlen train/val/test felosztás miatt nem nested keresztvalidáció). Ha holtverseny van, a kisebb λ, azon belül a kisebb modell nyer.
4. **Érintetlen záróteszt:** a kiválasztott konfiguráció belső legjobb súlyával egyszer kiértékeljük a test halmaz pontosságát. A test halmazt a keresés során semmilyen lépésben nem használjuk.
5. **Jelentés:** a jegyzőkönyvbe kerül mind a 6 belső val érték, a kiválasztott konfiguráció, a test pontosság, valamint a seed és a környezet verziója.

Az alábbi kód a protokollt valósítja meg; a referenciakörnyezetben a GlyphLab adatokon futott, és az alábbi mért kimenetet adta.

```
# munkakönyvtár: lab/
# Futtatható labor; a mért kimenet a kód után látható.
import copy
import csv
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

seed = 20261020
torch.manual_seed(seed)
np.random.seed(seed)
device = torch.device("cpu")
torch.set_num_threads(1)
dtype = torch.float32

DATA = Path(__file__).resolve().parent / "data"

images = np.load(DATA / "images.npy").astype(np.float32)
labels = np.load(DATA / "labels.npy").astype(np.int64)
# Fejléc szerinti oszlopnevek; a sorok sorrendje az images tömb sorrendje.
with open(DATA / "metadata.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    assert reader.fieldnames == ["index", "scene_id", "view_id", "label",
                                 "split", "xmin", "ymin", "xmax", "ymax"], \
        "váratlan metadata-fejléc"
    meta_rows = list(reader)
```

```
assert len(images) == len(labels) == len(meta_rows), "eltérő hosszúságú adat"
meta_index = np.array([int(row["index"]) for row in meta_rows])
assert np.array_equal(meta_index, np.arange(len(images))), \
    "a metadata index oszlopa nem követi az images tömb sorrendjét"
split = np.array([row["split"] for row in meta_rows])
assert set(split) <= {"train", "val", "test"}
train_idx = np.where(split == "train")[0]
val_idx = np.where(split == "val")[0]
test_idx = np.where(split == "test")[0]
assert len(train_idx) == 240 and len(val_idx) == 60 and len(test_idx) == 60

X_train = torch.tensor(images[train_idx], dtype=dtype, device=device)
y_train = torch.tensor(labels[train_idx], dtype=torch.long, device=device)
X_val = torch.tensor(images[val_idx], dtype=dtype, device=device)
y_val = torch.tensor(labels[val_idx], dtype=torch.long, device=device)
X_test = torch.tensor(images[test_idx], dtype=dtype, device=device)
y_test = torch.tensor(labels[test_idx], dtype=torch.long, device=device)

def make_model(hidden):
    return nn.Sequential(
        nn.Flatten(),
        nn.Linear(24 * 24, hidden),
        nn.ReLU(),
        nn.Linear(hidden, 3),
    ).to(device)

def evaluate(model, X, y):
    model.eval()
    with torch.no_grad():
        pred = model(X).argmax(dim=1)
    return (pred == y).float().mean().item()

def make_loader():
    generator = torch.Generator()
    generator.manual_seed(seed)
    return DataLoader(TensorDataset(X_train, y_train),
                      batch_size=32, shuffle=True, generator=generator)

criterion = nn.CrossEntropyLoss()
results = []
for hidden in [16, 32]:
    for lam in [0.0, 1e-4, 1e-3]:
        # Azonos recept minden konfigurációhoz: seed, init és batch-sorrend.
        torch.manual_seed(seed)
        np.random.seed(seed)
        train_loader = make_loader()
        model = make_model(hidden)
        optimizer = torch.optim.SGD(model.parameters(), lr=0.01,
                                    weight_decay=lam)
        best_acc, best_state = -1.0, None
        for _ in range(30):
            model.train()
            for xb, yb in train_loader:
                optimizer.zero_grad()
                loss = criterion(model(xb), yb)
                loss.backward()
                optimizer.step()
            val_acc = evaluate(model, X_val, y_val)
            if val_acc > best_acc:
                best_acc = val_acc
                best_state = copy.deepcopy(model.state_dict())
        assert best_state is not None
        model.load_state_dict(best_state)
```

```
        results.append({"hidden": hidden, "lam": lam,
                        "val_acc": best_acc, "state": best_state})
        print(f"hidden={hidden:2d} lam={lam:g} val_acc={best_acc:.3f}")

# Holtversenynél a kisebb lam nyer; az első maximum marad meg.
best = max(results, key=lambda r: (r["val_acc"], -r["lam"]))
model = make_model(best["hidden"])
model.load_state_dict(best["state"])
test_acc = evaluate(model, X_test, y_test)
assert 0.0 <= test_acc <= 1.0
print("választott:", best["hidden"], best["lam"],
      f"val_acc={best['val_acc']:.3f}")
print(f"test_acc={test_acc:.3f}")
print("seed:", seed, "torch:", torch.__version__, "numpy:", np.__version__)
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
hidden=16 lam=0 val_acc=0.350
hidden=16 lam=0.0001 val_acc=0.350
hidden=16 lam=0.001 val_acc=0.350
hidden=32 lam=0 val_acc=0.400
hidden=32 lam=0.0001 val_acc=0.400
hidden=32 lam=0.001 val_acc=0.400
választott: 32 0.0 val_acc=0.400
test_acc=0.383
seed: 20261020 torch: 2.15.0+cpu numpy: 2.5.4
```

A 30 epochos protokoll a választási mechanizmust mutatja be, nem a maximális pontosságot: a háromosztályos feladat véletlen szintje $1/3 \approx 0{,}333$, a mért tesztpontosság 0,383. A hat konfiguráció közül a nagyobb rejtett méret és a regularizálás nélküli változat kapta a legjobb validációs pontosságot; a tesztet a választás után egyszer értékeltük. Ez a protokoll szerinti kiválasztás eredménye, nem statisztikai különbség: egyetlen seed és 30 validációs jelenet mellett a 0,400 és 0,350 közötti különbség bizonytalansága nincs becsülve; páros, jelenetszintű bootstraphoz a két konfiguráció jelenetenkénti döntései kellenek, a kiválasztási torzítás figyelembevételéhez pedig a teljes keresést is meg kell ismételni (23.3. Páros összehasonlítás és csoportszintű intervallum).

8.3.8 Feladatok

F1 (fogalmi, nehézség 1). Mi a különbség a validációs és a teszt halmaz szerepe között a beágyazott modellválasztásban? Adj egy konkrét forgatókönyvet, ahol a teszt halmaz keresés közbeni használata torzítja a végső pontosságbecslést. Beadandó: saját megfogalmazású, legalább öt mondatos válasz.

F2 (levezetési, nehézség 2). Vezesd le az $M = 5$, $n_{val} = 3$, $p = 0{,}5$ esetre a legjobb validációs pontosság várható értékét, és hasonlítsd össze a 0,5 valós értékkel. Minden lépést indokolj. Beadandó: az eloszlás, a várható érték képlete és a numerikus eredmény.

F3 (implementációs, nehézség 2). Írj olyan Python-szkriptet, amely $M = 20$ modell, $n_{val} = 5$ validációs minta, $p = 0{,}5$ esetén 10 000 ismétléssel szimulálja a fenti kiválasztást, és kiírja a kiválasztott modell átlagos validációs pontosságát, valamint a tesztpontosság becsült átlagát. A scriptben seed, importok, dtype és futási parancs szerepeljen. Beadandó: a kód és a várható kimenet magyarázattal.

F4 (hibakeresési, nehézség 2). Az alábbi pszeudokód a legjobb konfigurációt keresi:

```
for config in grid:
    model = train(config, train_data)
    val_acc[config] = evaluate(model, val_data)
best_config = argmax(val_acc)
test_acc = evaluate(train(best_config, train_data + val_data), test_data)
```

Hibás-e? Ha igen, hol, mi a következmény, és hogyan javítod? Beadandó: a hiba oka, a torzítás mechanizmusa és a javított pszeudokód.

F5 (kísérleti/transzfer, nehézség 3). Tervezd meg a GlyphLab adatokra a modellválasztást úgy, hogy a keresési tér három modellcsaládot tartalmazzon: MLP 32 rejtett egységgel, MLP 64 rejtett egységgel, és egy kis CNN egy konvolúciós réteggel. Mindegyikhez L2-$\lambda \in \{10^{-4}, 10^{-2}\}$. Határozd meg a splitet, a választási kritériumot, a teszt időzítését, és indokold, miért jelenetfüggetlen a felosztás. Beadandó: a protokoll szöveges leírása, a keresési tér táblázata, az optimalizáló és a tanulási ráta megnevezése, valamint az érvényességi korlát.

8.3.9 Megoldások és indoklás

F1. A validáció arra való, hogy a keresés során döntéseket hozzunk: hiperparamétert, epochot, modellcsaládot választunk. A teszt halmaz ezzel szemben a már lezárt modell egyszeri, torzítatlan külső értékelése. Ha a tesztet a kereséshez használjuk, a keresés adaptívan illeszkedik a tesztminta zajához; a kiválasztott modell tesztpontossága felfelé torzított becslés lesz a valódi általánosítási hibára. Példa: száz hiperparaméter-beállítást próbálunk, és minden körben a tesztpontosság alapján döntünk; a maximális tesztpontosság várhatóan a zaj miatt nagyobb a valódi pontosságnál. A döntési szabály rögzítése önmagában nem elég: a keresés teljes naplóját meg kell őrizni, hogy utólag ellenőrizhető legyen, hány modellt és milyen kritériummal hasonlítottunk össze.

F2. Minden modell validációs pontossága $Y_i \sim \mathrm{Binom}(3;\,0{,}5)/3$. Lehetséges értékek: $0, 1/3, 2/3, 1$, valószínűségek $1/8, 3/8, 3/8, 1/8$. A maximum eloszlása $M = 5$ esetén:

- $\mathbb{P}(\max \le 0) = (1/8)^5 = 1/32768$,
- $\mathbb{P}(\max \le 1/3) = (1/8 + 3/8)^5 = (1/2)^5 = 1/32$,
- $\mathbb{P}(\max \le 2/3) = (1/8 + 3/8 + 3/8)^5 = (7/8)^5 = 16807/32768$,
- $\mathbb{P}(\max \le 1) = 1$.

Ebből: - $\mathbb{P}(\max = 0) = 1/32768$, - $\mathbb{P}(\max = 1/3) = (1/32) - (1/32768) = 1023/32768$, - $\mathbb{P}(\max = 2/3) = (16807/32768) - (1/32) = 16807/32768 - 1024/32768 = 15783/32768$, - $\mathbb{P}(\max = 1) = 1 - 16807/32768 = 15961/32768$.

Várható érték:

$$\mathbb{E}[\max Y_i] = 0 \cdot p_0 + \frac{1}{3} p_{1/3} + \frac{2}{3} p_{2/3} + 1 \cdot p_1.$$

Numerikusan: $p_{1/3} = 1023/32768 \approx 0{,}03122$, $p_{2/3} = 15783/32768 \approx 0{,}48166$, $p_1 = 15961/32768 \approx 0{,}48709$. Így

$$\mathbb{E}[\max Y_i] \approx 0{,}01041 + 0{,}32111 + 0{,}48709 = 0{,}81860.$$

Pontos értékkel $\mathbb{E}[\max Y_i] = 26824/32768 \approx 0{,}818604$.

A torzítás $0{,}81860 - 0{,}5 = 0{,}31860$. A maximum várható értéke tehát messze a valós 0,5 fölött van. Az alak ellenőrzése: a valószínűségek összege 1, a várható érték $[0, 1]$-ben van.

F3. Példakód:

```python
# munkakönyvtár: lab/
# Szimuláció: M=20, n_val=5, p=0.5, 10000 ismétlés.
# Futás: python sim_validation_bias.py
import numpy as np

rng = np.random.default_rng(20261020)
M, n_val, p, reps = 20, 5, 0.5, 10_000
val_max = np.empty(reps, dtype=np.float64)
test_acc = np.empty(reps, dtype=np.float64)

for r in range(reps):
    # validation predikciók: M x n_val, Bernoulli(p)
    pred = rng.random((M, n_val), dtype=np.float64) < p
    val_acc = pred.mean(axis=1)          # shape (M,)
    best = int(np.argmax(val_acc))       # a legjobb modell
    val_max[r] = val_acc[best]
    # tesztpontosság: egy újabb Bernoulli(p) minta a kiválasztott modellre
    test_acc[r] = rng.binomial(1, p)

print("átlagos legjobb validation accuracy:", val_max.mean())
print("átlagos test accuracy:", test_acc.mean())
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál, seed 20261020):

```
átlagos legjobb validation accuracy: 0.8907200000000001
átlagos test accuracy: 0.5081
```

A validációs átlag 0,891, jóval a valós 0,5 fölött; a tesztátlag 0,508, a $0{,}5 \pm 2\sqrt{0{,}25/10\,000} = 0{,}5 \pm 0{,}01$ intervallumban. Az elméleti érték a Binom(5; 0,5)/5 szerinti maximum várható értéke, $\approx 0{,}8909$, amit a szimulációs átlag 0,8907 közelít. A teszt húzása azért független Bernoulli-minta, mert a konstrukcióban minden modell valódi pontossága $p = 0{,}5$ bármely, a validációtól független ponton, és a modellek egymástól is függetlenek; a kiválasztás csak a validációs mintát használja, ezért a tesztpont a kiválasztott modellre nézve is független, 0,5 valószínűségű találat. A szimuláció a felfelé torzítást mutatja.

F4. A kód hibás. Az utolsó sorban a `train_data + val_data` egyesített halmazon újratanítja a legjobb konfigurációt, majd a tesztet értékeli. Fontos pontosítás: a teszt halmazt ez a sor nem használja a kereséshez, ezért az érintetlen záróteszt elve önmagában nem sérül; a hiba az, hogy a jelentett modell nem a validáció által kiválasztott, rögzített modell. Az újratanítás új tanítási pályát ad (a példában korai leállás sincs), a `val_data` hozzáadása után pedig a modell már nem ugyanaz, amelyet a validáció kiválasztott, ezért a tesztpontosság nem a kiválasztási döntés eredményére vonatkozik. Javítás: a keresés során minden konfigurációhoz mentsük a belső validáció szerinti legjobb súlyt, és a választott konfiguráció mentett súlyát értékeljük a test halmazon; ha a train+val újratanítás szándékos, akkor azt a protokollban előre kell rögzíteni (rögzített lépésszámmal), és a tesztet az így kapott modellen, egyszer kell mérni. Javított pszeudokód:

```
for config in grid:
    model, best_state = train_with_early_stopping(config, train_data, val_data)
    val_acc[config] = evaluate(model, val_data)
    states[config] = best_state
best_config = argmax(val_acc)
test_acc = evaluate(states[best_config], test_data)
```

F5. A protokoll:

- **Split:** a GlyphLab `metadata.csv` train/val/test oszlopa jelenetfüggetlen: egy jelenet mindkét nézete azonos splitben van. Ezért a modellválasztás során a validation pontosság

független megfigyelési egysége a jelenet, nem a kép; a bootstrap (újramintavételezés) és a CI (konfidenciaintervallum) is jelenetszintű.

- **Keresési tér:** 3 modellcsalád × 2 λ = 6 konfiguráció. Az MLP-modellek: `Flatten` → `Linear(24*24, 32 vagy 64)` → `ReLU` → `Linear(..., 3)`; a kis CNN: `Conv2d(1, 8, kernel_size=3, padding=1)` → `ReLU` → `MaxPool2d(2)` → `Flatten` → `Linear(8*12*12, 3)`.

Modellcsalád	L2-λ	Konfigurációk
MLP 32	$\{10^{-4}, 10^{-2}\}$	2
MLP 64	$\{10^{-4}, 10^{-2}\}$	2
Kis CNN	$\{10^{-4}, 10^{-2}\}$	2

- **Belső ciklus:** minden konfigurációt Adam optimalizálóval, lr = 10^{-2} mellett 30 epochig tanítunk a train halmazon, batch 32, seed 20261020. Minden epoch végén a val halmazon pontosságot számolunk, és a legjobb epoch súlyát mentjük.
- **Külső választás:** a konfigurációk közül a belső legjobb val pontosság szerint választunk. Holtverseny esetén a kisebb λ, azon belül a kisebb modell nyer.
- **Záróteszt:** a választott konfiguráció mentett súlyát egyszer értékeljük a test halmazon; a tesztet a keresés során nem használjuk.
- **Korlát:** a A. A GlyphLab adatkészlet szakaszban rögzített GlyphLab adat procedurálisan generált és kis méretű; az eredmény nem általánosítható természetes képekre. A seed egyetlen; a végső pontosság bizonytalansága több seeddel becsülhető, de ehhez a teljes keresést meg kell ismételni. Erőforrás: CPU, a rögzített `cnn-labor` környezet.

8.3.10 Gyakori tévedés

„A nagyobb kapacitású modell tesztpontossága rosszabb, mint a kisebbé, mert a nagy kapacitás mindig túlilleszt.” Ez nem következik. A kapacitás önmagában nem ad számszerű jóslatot a teszthibára; a tényleges hiba a minta méretétől, a loss típusától, a regularizálástól és az optimalizálástól is függ. Ellenpélda: $n = 10$ mintán egy 10^6 paraméterű, erős L2-regularizálással tanított modell lehet alulillesztett, míg egy 5 paraméterű, regularizálatlan modell is túlilleszthet, ha a tanítópontok száma kisebb a modell szabadsági fokánál: ilyenkor a tanítási hiba nullára csökkenthető úgy, hogy a modell a tanítópontok zaját is illeszti, és a validációs hiba nagy marad. A 8.2. Véges hipotézisosztály egyenletes korlátja uniókorlátja csak a statisztikai hiba felső becslését növeli a kapacitással; ez nem jelenti azt, hogy a tényleges hiba szükségképpen nő.

8.3.11 Önellenőrzés és visszatekintés

Ellenőrizze, hogy az ellenpélda minden feltevése kimondott-e: a modellek valós pontossága, a validációs minták Bernoulli-eloszlása, a függetlenség, a maximum-kiválasztás. Ha bármelyik feltételt elhagyjuk, a cáfolat nem érvényes. Visszatekintés a 8.2. Véges hipotézisosztály egyenletes korlátja uniókorlátjából: a $2Me^{-2n\epsilon^2}$ tag mutatja, miért nő a statisztikai hiba a kapacitással; a mostani torzítás ennek a jelenségnek a véges validációs mintán megfigyelt párja. A 7.3. Bináris, multilabel és regressziós cél loss-szerződései arra emlékeztetnek, hogy a bias–variance felbontás négyzetes hibára szól, cross-entropyra más elemzés kell. A 4.3. Jacobi-mátrix, láncszabály és VJP láncszabálya a regularizált loss gradiensét adja meg: a λW tag közvetlenül hozzáadódik a VJP-hez.

8.3.12 Rövid összefoglaló

A kapacitás a hipotézisosztály kifejezőképessége, amely a bias²–variance felbontásban a variance oldalt növeli, miközben a bias oldalt csökkentheti: a nagyobb osztály jobb közelítőt enged a készletbe. A büntetőtag és a korai leállás az effektív kapacitást korlátozza; a validáció a modellválasztás döntési eszköze. A beágyazott választás a belső tanítási és a külső keresési ciklust szétválasztja, a tesztet pedig érintetlenül hagyja. Az ellenpélda megmutatta, hogy sok próbából a legjobb validációs pontszám felfelé torzított becslése a tesztpontosságnak: $M = 10$, $n_{val} = 1$, $p = 0{,}5$ esetén a várható maximum 0,999, míg a tesztpontosság 0,5. Ezért a tesztet csak a keresés lezárása után, egyszer értékeljük.

8.3.13 Források

- [P27] Hoeffding, W. (1963). Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301), 13–30. https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830. Hozzáférés: 2026-10-30.

8.4 II. ellenőrző pont: korrekt tanulási állítás

8.4.1 Cél, előismeret és induló kérdés

Ez az ellenőrzőpont egyetlen kérdés köré szervezi az eddigi anyagot: milyen feltételek mellett állítható, hogy a megtanult modell nemcsak a tanítóhalmazon, hanem az új mintán is jól teljesít? A 8.3. Kapacitás, regularizálás és modellválasztás alfejezetben láttuk, hogy a kapacitás, a regularizálás és a modellválasztás protokollja mind a tanulási állítás feltételrendszerét alkotják. Most ezeket egyetlen, teljes specifikációvá zárjuk össze.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Ajánlott tanulási útvonal és ellenőrző pontok” szakasza adja meg; itt csak az eltérés: a riport itt a bizonyítékjegyzék; a súlypont a D3 (adat- és mérési protokoll: split, megfigyelési egység, tesztfegyelem) és a D5 (az állítás és korlátai: az állítás nem lép túl a bizonyítékon).

A szükséges előismeret: a 6.1. Objektív, reguláris tag és gradienslépés alfejezetben bevezetett empirikus kockázat, a 6.3. Sztochasztikus és minibatch gradiens alfejezetben tárgyalt minibatch gradiens, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben bizonyított stabil veszteségszámítás, valamint a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben felépített gradiensellenőrzési protokoll. A 8.3. Kapacitás, regularizálás és modellválasztás alfejezetben tárgyalt beágyazott (*nested*) választás és a felfelé torzított validációs maximum gondolata közvetlenül folytatódik itt.

Induló becslés. Legyen a hipotézisosztály mérete $|H| = 4$, a minta mérete $m = 64$, a hibavalószínűség $\delta = 0{,}05$ (a konfidenciaszint $1 - \delta = 0{,}95$). Mielőtt levezetjük, becsüljük meg: mekkora egyszerű, adattól függetlenül számolható sávot kaphatunk az általánosítási hibára? A választ a következő szakaszban ellenőrizzük.

8.4.2 Fogalmak: eloszlás, kockázat, cél, gradiens, sztochasztikus frissítés, független teszt

Eloszlás (distribution). Legyen a megfigyelés egy $Z = (X, Y)$ pár, ahol $X \in \mathcal{X}$ a bemenet, $Y \in \mathcal{Y}$ a cél. A tanulási feladat mögötti feltételezés egy közös D eloszlás a párokon. A tanítóhalmaz $S_m = ((x_1, y_1), \dots, (x_m, y_m))$, ahol a párok függetlenek és azonos eloszlásúak D szerint (iid). A tesztminta ugyanebből a D-ből származik, de diszjunkt S_m-től. A D a gyakorlatban nem ismert; ami ismert, az a minta.

Kockázat (risk). Az $f : \mathcal{X} \to \mathcal{Y}$ prediktor kockázata a várható veszteség:

$$R_D(f) = \mathbb{E}_{(X,Y)\sim D}\left[\ell(f(X), Y)\right].$$

A kockázat egyetlen valós szám, ugyanolyan mértékegységben kifejezve, mint a veszteség. Az empirikus kockázat a tanítóhalmazon vett átlag:

$$\hat{R}_m(f) = \frac{1}{m}\sum_{i=1}^{m} \ell(f(x_i), y_i).$$

A két mennyiség szerepe eltér: $\hat{R}_m$ az, amit számolni tudunk, R_D az, amiről állítani szeretnénk. A 8.1. Kockázat, mintavétel és baseline alfejezetben tárgyalt baseline-tól való eltérés mindig R_D és $\hat{R}_m$ közötti rés becslésének minőségén múlik.

Célfüggvény (objective). A tanítás a paraméterezett f_w modellen egy J_λ célfüggvény minimalizálását jelenti. A 6.1. Objektív, reguláris tag és gradienslépés és 8.3. Kapacitás, regularizálás és modellválasztás jelölése szerint:

$$J_\lambda(w) = \hat{R}_m(f_w) + \frac{\lambda}{2}\|w\|_2^2,$$

ahol $\lambda \geq 0$ a súlyregularizáló együttható. A célfüggvény kettős: az első tag az adathoz való illeszkedést, a második a kapacitáskorlátot képviseli. A λ a 8.3. Kapacitás, regularizálás és modellválasztás bias–variance kompromisszum operatív formája.

Gradiens (gradient). A gradiens a $\nabla_w J_\lambda(w)$ vektor, alakja megegyezik w alakjával. A gyakorlatban a veszteség összegéből vagy átlagából indulunk ki. Átlagos veszteség esetén, ha az i-edik minta egyedi vesztesége ℓ_i, a gradiens:

$$\nabla_w J_\lambda(w) = \frac{1}{m}\sum_{i=1}^{m} \nabla_w \ell_i + \lambda w.$$

Az átlag $1/m$ faktora a 6.3. Sztochasztikus és minibatch gradiens alfejezetben tárgyalt batch-méret szerinti átlagolás miatt nem hagyható el, ha a tanulási rátát egy adott veszteségskálára hangoljuk.

Sztochasztikus frissítés (stochastic update). Az S_m-ből vett B_t minibatch az $\{1, \dots, m\}$ indexek egyenletesen választott, $b = |B_t|$ méretű véletlen részhalmaza: minden $\binom{m}{b}$ részhalmaz azonos valószínűségű, és a választás a korábbi lépésektől független. A minibatch gradiens:

$$g_t = \frac{1}{|B_t|}\sum_{i\in B_t} \nabla_w \ell_i + \lambda w_t,$$

és a frissítés $w_{t+1} = w_t - \eta_t g_t$. A frissítés azért sztochasztikus, mert B_t véletlen; a teljes $\nabla_w J_\lambda$ helyett annak egy közelítését használjuk. Az egyenletes választás miatt a feltételes várható érték linearitása adja, hogy $\mathbb{E}[g_t \mid w_t] = \nabla_w J_\lambda(w_t)$, ha a regularizáló tagot is beleszámítjuk; ezt a tulajdonságot nevezzük a minibatch gradiens torzítatlanságának.

Független teszt (independent test). A tesztminta S_{test}, mérete n_{test}, diszjunkt a tanító- és validációs halmaztól. A teszt kockázatbecslője:

$$\hat{R}_{test}(f) = \frac{1}{n_{test}} \sum_{(x,y)\in S_{test}} \ell(f(x), y).$$

Ha f független S_{test}-től, akkor $\hat{R}_{test}(f)$ torzítatlan becslése $R_D(f)$-nek:

$$\mathbb{E}\left[\hat{R}_{test}(f)\right] = \frac{1}{n_{test}} \sum_{i=1}^{n_{test}} \mathbb{E}\left[\ell(f(X_i), Y_i)\right] = R_D(f)$$

a függetlenség és az azonos eloszlás miatt. Ez a torzítatlanság azonnal megszűnik, ha a modellt a teszt eredménye alapján választjuk ki; ezt a 8.3. Kapacitás, regularizálás és modellválasztás adaptivitási protokollja már kimondta.

8.4.3 A kiválasztott elméleti állítás és feltételeinek ellenőrzése

A jelen egység bizonyítási feladata: vegyünk egy konkrét elméleti állítást, soroljuk fel az összes feltételét, és ellenőrizzük azokat egy példán. A kiválasztott állítás a véges hipotézisosztály egyenletes korlátja, amelyet a 8.2. Véges hipotézisosztály egyenletes korlátja vezetett le a Hoeffding-koncentrációból és az uniókorlátból.

Tétel (véges H egyenletes általánosítási korlátja, Hoeffding-alapú). Tegyük fel, hogy

1. $H = \{h_1, \dots, h_N\}$ véges, előre rögzített hipotézisosztály;
2. a minta S_m iid a D eloszlásból;
3. a veszteség minden $h \in H$ és minden (x, y) párra a $[0, 1]$ intervallumba esik;
4. a $\delta \in (0, 1)$ konfidenciaszint előre rögzített.

Ekkor:

$$P\left(\forall h \in H:\ R_D(h) \le \hat{R}_m(h) + \sqrt{\frac{\log|H| + \log(1/\delta)}{2m}}\right) \ge 1 - \delta.$$

A bizonyítás két lépésből áll. (i) Egyetlen rögzített h-ra a $[0,1]$-be eső veszteségek független összegére alkalmazott Hoeffding-állítás [P27] adja $P(R_D(h) - \hat{R}_m(h) \ge \epsilon) \le e^{-2m\epsilon^2}$. (ii) Az uniókorlát szerint $P(\max_{h\in H}(R_D(h) - \hat{R}_m(h)) \ge \epsilon) \le |H| e^{-2m\epsilon^2}$. A $\delta = |H| e^{-2m\epsilon^2}$ egyenletet ϵ-ra megoldva kapjuk a tétel sávját. A teljes Hoeffding-bizonyítást itt nem idézzük, csak a [P27] forrásra hivatkozunk. Az uniókorlátos levezetés saját, a 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben történt meg.

A tétel kimondásakor különösen fontos, mit **nem** állítunk: nem állítja, hogy a tanult $\hat{h} \in \arg\min_{h\in H} \hat{R}_m(h)$ kockázata alacsony; csak azt, hogy a kockázat nem haladja meg az empirikus értéket plusz a sávot, $1-\delta$ valószínűséggel. Ha $\hat{R}_m(\hat{h}) = 0{,}9$, a korlát a $0{,}9 + \epsilon$ értékre vonatkozik, ami használhatatlan lehet.

Feltétel-ellenőrzés a példán. Válasszuk a következő játékfeladatot. Legyen a bemenettér $\mathcal{X} = [-1, 1]^2$, a cél $\mathcal{Y} = \{0, 1\}$. A generáló D: $X \sim \text{Uniform}([-1, 1]^2)$, és determinisztikus címke $Y = 1[X_1 > 0]$, ahol $1[\cdot]$ az indikátorfüggvény (értéke 1, ha a feltétel igaz, különben 0). A hipotézisosztály négy előre rögzített tengelyirányú küszöbszabály:

$$h_1(x) = 1[x_1 > 0], \quad h_2(x) = 1[x_1 \le 0], \quad h_3(x) = 1[x_2 > 0], \quad h_4(x) = 1[x_2 \le 0].$$

A feltételek ellenőrzése:

1. H véges és előre rögzített: $|H| = 4$, a négy szabály a minta generálása előtt adott. A feltétel teljesül.
2. A minta iid: az X_i-k függetlenek és azonos eloszlásúak, és Y_i az X_i determinisztikus függvénye, ezért a párok is függetlenek (független változók determinisztikus függvényei függetlenek) és azonos eloszlásúak. A feltétel teljesül.
3. A veszteség $[0, 1]$-be esik: a 0/1 veszteséget használjuk, amelynek értéke mindig 0 vagy 1. A feltétel teljesül.
4. A $\delta = 0{,}05$ konfidenciaszint a számítás előtt rögzített. A feltétel teljesül.

A valódi kockázatok ebben a modellben zárt alakban ismertek: $R_D(h_1) = 0$, mert h_1 a generáló szabály; $R_D(h_2) = 1$, mert h_2 mindenütt ellentmond; $R_D(h_3) = R_D(h_4) = 0{,}5$, mert X_1 és X_2 függetlenek, így a második koordináta előjele a címkétől független. Ez a tudás a szimulációban használható, valós adatnál viszont nem áll rendelkezésre; ezt a különbséget a laborban kifejezetten megőrizzük.

8.4.4 Kézi példa és minimális kód

Kézi példa. Legyen $m = 4$, $\delta = 0{,}05$. A sáv:

$$\sqrt{\frac{\log 4 + \log 20}{8}} = \sqrt{\frac{1{,}3863 + 2{,}9957}{8}} = 0{,}740.$$

A minta legyen:

$$x_1 = (0{,}2;\, 0{,}3), \quad x_2 = (-0{,}4;\, 0{,}1), \quad x_3 = (0{,}6;\, -0{,}7), \quad x_4 = (-0{,}2;\, -0{,}5),$$

$$y_1 = 1, \quad y_2 = 0, \quad y_3 = 1, \quad y_4 = 0.$$

h	döntés	$\hat{R}_4$	R_D
h_1	$x_1 > 0$	0	0
h_2	$x_1 \le 0$	1	1
h_3	$x_2 > 0$	0,5	0,5
h_4	$x_2 \le 0$	0,5	0,5

A tétel garantálja, hogy minden h-ra $R_D(h) \le \hat{R}_4(h) + 0{,}740$. A négy egyenlőtlenség ezen az egyetlen realizáción közvetlenül ellenőrizhető, és itt teljesül; a tétel egy realizált mintára nem garantálja determinisztikusan. Ez azonban **nem** a 95%-os lefedettség bizonyítéka: a tétel valószínűségi állítás, egyetlen realizáció nem igazolja. A lefedettséget Monte Carlo-szimulációval (ismételt véletlen mintavétellel) tudjuk empirikusan vizsgálni, amelyet a labor kódja tartalmaz.

Minimális kód a korlát és a lefedettség kiszámítására.

```
import numpy as np


def uniform_bound(n_h, m, delta):
    return float(np.sqrt((np.log(n_h) + np.log(1.0 / delta)) / (2.0 * m)))


def simulate_coverage(n_sim=2000, m=64, delta=0.05, seed=123):
    rng = np.random.default_rng(seed)
    b = uniform_bound(4, m, delta)
    true_risk = np.array([0.0, 1.0, 0.5, 0.5])
    covered = 0
    for _ in range(n_sim):
        X = rng.uniform(-1.0, 1.0, size=(m, 2))
        y = (X[:, 0] > 0).astype(np.float64)
        pred = np.vstack([
            X[:, 0] > 0,
            X[:, 0] <= 0,
            X[:, 1] > 0,
            X[:, 1] <= 0,
        ])
        err = (pred != y).mean(axis=1)
        if np.all(true_risk <= err + b):
            covered += 1
    return covered / n_sim, b
```

A `uniform_bound(4, 64, 0.05)` értéke 0,185, ami az induló becslésre adott válasz. A szimuláció a kockázatok ismeretében számolja a lefedettséget; a várt érték elméletileg legalább 0,95, de a véges szimulációból becsült arány nem garantáltan $\geq 0{,}95$ (a binomiális szórása miatt kissé alá is eshet).

8.4.5 Labor: végponttól végpontig terjedő osztályozási kísérlet bizonyítékjegyzékkel

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor kis logisztikus regressziót tanít szintetikus 2D adaton. Szándékosan nem a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-képeket használjuk, mert azok a CNN-részekben kerülnek elő; itt a cél a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetből ismert stabil BCE, a 4.4. I. ellenőrző pont: matematika és kód megfelelése gradiens-ellenőrzési elve és a 8.3. Kapacitás, regularizálás és modellválasztás független tesztelési protokoll összezárása.

Munkakönyvtár és futtatás. A fájl: `lab/n032/checkpoint_lab.py`. Parancs:

```
python lab/n032/checkpoint_lab.py
```

A kód végig CPU-n fut, NumPy-val; nincs GPU- vagy keretrendszerfüggés.

```
import json
import os

import numpy as np

SEED = 20261020
rng = np.random.default_rng(SEED)
```

```python
def uniform_bound(n_h, m, delta):
    return float(np.sqrt((np.log(n_h) + np.log(1.0 / delta)) / (2.0 * m)))


def simulate_coverage(n_sim=2000, m=64, delta=0.05, seed=123):
    rng_cov = np.random.default_rng(seed)
    b = uniform_bound(4, m, delta)
    true_risk = np.array([0.0, 1.0, 0.5, 0.5])
    covered = 0
    for _ in range(n_sim):
        X = rng_cov.uniform(-1.0, 1.0, size=(m, 2))
        y = (X[:, 0] > 0).astype(np.float64)
        pred = np.vstack([
            X[:, 0] > 0,
            X[:, 0] <= 0,
            X[:, 1] > 0,
            X[:, 1] <= 0,
        ])
        err = (pred != y).mean(axis=1)
        if np.all(true_risk <= err + b):
            covered += 1
    return covered / n_sim, b


def bce_logits_stable_f64(z, y):
    """z: (N,) logits, y: (N,) float cél {0,1}. Átlagos BCE, stabil."""
    z = np.asarray(z, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    assert z.shape == y.shape and z.ndim == 1
    return float(np.mean(np.maximum(z, 0) - y * z + np.log1p(np.exp(-np.abs(z)))))


def make_data(n, rng):
    X = rng.normal(0.0, 1.0, size=(n, 2)).astype(np.float32)
    assert X.shape == (n, 2)
    logit_p = (2.0 * X[:, 0] - X[:, 1]).astype(np.float32)
    p = 1.0 / (1.0 + np.exp(-logit_p))
    y = (rng.random(n) < p).astype(np.float32)
    return X, y
```

A tanítási lépés float32-ben, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil BCE-formulát a kiértékelésnél float64-ben használjuk. A szigmoidot a tanításban és a gradiensellenőrzésben is a `np.logaddexp`-ra épülő stabil alakban számítjuk, így nagy $|z|$ esetén sem lép fel túlcsordulás. A paramétergradiens a logitokra vett $\sigma(z) - y$ vektorból adódik; ez a 4.4. I. ellenőrző pont: matematika és kód megfelelése affin backward speciális esete logisztikus célra.

```python
def train_one(X, y, lr, epochs, batch, lam):
    n, d = X.shape
    w = np.zeros(d, dtype=np.float32)
    b = np.float32(0.0)
    for _ in range(epochs):
        idx = rng.permutation(n)
        for start in range(0, n, batch):
            bi = idx[start:start + batch]
            Xb = X[bi].astype(np.float32)
            yb = y[bi]
            z = Xb @ w + b
            p = np.exp(-np.logaddexp(0.0, -z))  # stabil szigmoid
            g = (p - yb) / len(bi)
            gw = Xb.T @ g + lam * w
            gb = np.sum(g)
```

```
            w -= lr * gw
            b -= lr * gb
    return w, b
```

A gradiensellenőrzés a 4.4. I. ellenőrző pont: matematika és kód megfelelése protokoll szerint: kis float64 batch, középső differencia, $\epsilon = 10^{-6}$, abszolút hibamérce; a PASS-küszöb `grad_err < 1e-8`, amelyet a kód `assert`-tel kényszerít ki. A nullközeli gradiensnél a 4.4. I. ellenőrző pont: matematika és kód megfelelése kombinált abszolút/relatív tűrésében az abszolút tag dominál, ezért az abszolút hiba önmagában is elégséges.

```
def loss_and_grad(w, b, X, y, lam):
    z = X @ w + b
    p = np.exp(-np.logaddexp(0.0, -z))  # stabil szigmoid
    loss = bce_logits_stable_f64(z, y) + 0.5 * lam * float(w @ w)
    g = (p - y) / len(y)
    return loss, X.T @ g + lam * w, np.sum(g)


def grad_check(X8, y8, lam=1e-3):
    w0 = rng.normal(0.0, 0.5, size=2)
    b0 = np.float64(0.2)
    _, gw, gb = loss_and_grad(w0, b0, X8, y8, lam)
    max_abs = 0.0
    for j in range(2):
        wp, wm = w0.copy(), w0.copy()
        wp[j] += 1e-6
        wm[j] -= 1e-6
        lp, _ = loss_and_grad(wp, b0, X8, y8, lam)[:2]
        lm, _ = loss_and_grad(wm, b0, X8, y8, lam)[:2]
        num = (lp - lm) / (2e-6)
        max_abs = max(max_abs, abs(num - gw[j]))
    bp, bm = b0 + 1e-6, b0 - 1e-6
    num_b = (loss_and_grad(w0, bp, X8, y8, lam)[0]
             - loss_and_grad(w0, bm, X8, y8, lam)[0]) / (2e-6)
    max_abs = max(max_abs, abs(num_b - gb))
    return float(max_abs)
```

A kísérleti protokoll: train 1800, valid 200, test 2000 minta, egyetlen adatgeneráló seeddel. A validáción két learning rate közül választunk; a tesztet a kiválasztás után egyszer hívjuk meg.

```
def main():
    n_train, n_val, n_test = 1800, 200, 2000
    Xt, yt = make_data(n_train, rng)
    Xv, yv = make_data(n_val, rng)
    Xs, ys = make_data(n_test, rng)
    assert Xt.shape == (n_train, 2) and Xv.shape == (n_val, 2)
    assert Xs.shape == (n_test, 2)

    # Baseline: a tanító címkearány mint konstans valószínűség
    q = float(np.mean(yt))
    base_ce = -float(np.mean(ys * np.log(q) + (1.0 - ys) * np.log(1.0 - q)))
    # A konstans jostlat a tanito q-bol jon; a teszt csak kiertekel.
    base_acc = float(np.mean(ys == float(q > 0.5)))

    results = {}
    for lr in (0.1, 0.5):
        w, b = train_one(Xt, yt, lr=lr, epochs=50, batch=16, lam=1e-3)
        zv = Xv.astype(np.float32) @ w + b
        val_loss = bce_logits_stable_f64(zv, yv)
        results[lr] = {"w": w, "b": b, "val_loss": val_loss}

    best_lr = min(results, key=lambda k: results[k]["val_loss"])
```

```
    w, b = results[best_lr]["w"], results[best_lr]["b"]
    zs = Xs.astype(np.float32) @ w + b
    test_ce = bce_logits_stable_f64(zs, ys)
    test_acc = float(np.mean((zs > 0.0) == (ys > 0.0)))

    X8 = Xt[:8].astype(np.float64)
    y8 = yt[:8].astype(np.float64)
    grad_err = grad_check(X8, y8)

    cov, band = simulate_coverage(n_sim=2000, m=64, delta=0.05, seed=123)
    assert 0.0 <= cov <= 1.0
    assert grad_err < 1e-8

    manifest = {
        "coverage_simulation": {
            "theorem": "finite_H_uniform_bound",
            "H_size": 4,
            "m": 64,
            "delta": 0.05,
            "assumptions_checked": [
                "finite fixed H (4 rules)", "iid sample",
                "0/1 loss in [0,1]", "delta fixed",
            ],
            "bound_band": band,
            "coverage": cov,
        },
        "logistic_regression": {
            "note": ("folytonos parameterteru modell; a Hoeffding-unio"
                     "korlat nem alkalmazhato ra, a teszt CE es pontossag "
                     "leiro meroszam"),
        },
        "baseline": {"ce": base_ce, "acc": base_acc},
        "chosen_lr": best_lr,
        "final_test": {"ce": test_ce, "acc": test_acc},
        "grad_check_max_abs_err": grad_err,
        "seed": SEED,
        "device": "CPU",
        "dtype": "float32 training, float64 checks",
    }
    os.makedirs("lab/n032", exist_ok=True)
    with open("lab/n032/evidence.json", "w", encoding="utf-8") as fh:
        json.dump(manifest, fh, indent=2)
    print(json.dumps(manifest, indent=2))


if __name__ == "__main__":
    main()
```

A kódból, a referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál, seed 20261020); a hallgató eredménye a környezettől függően kis mértékben eltérhet:

```
{
  "coverage_simulation": {
    "theorem": "finite_H_uniform_bound",
    "H_size": 4,
    "m": 64,
    "delta": 0.05,
    "assumptions_checked": [
      "finite fixed H (4 rules)",
      "iid sample",
      "0/1 loss in [0,1]",
      "delta fixed"
    ],
```

```
    "bound_band": 0.18502589841259978,
    "coverage": 0.998
  },
  "logistic_regression": {
    "note": "folytonos parameterteru modell; a Hoeffding-uniokorlat nem alkalmazhato ra, a teszt
     ↪  CE es pontossag leiro meroszam"
  },
  "baseline": {
    "ce": 0.69296939496497,
    "acc": 0.51
  },
  "chosen_lr": 0.5,
  "final_test": {
    "ce": 0.44529394980597786,
    "acc": 0.789
  },
  "grad_check_max_abs_err": 3.328132214264201e-11,
  "seed": 20261020,
  "device": "CPU",
  "dtype": "float32 training, float64 checks"
}
```


Értelmezés. A lefedettség 0,998 egyetlen szimulációs becslés, 2000 ismétlésből; a binomiális szórás körülbelül $\sqrt{0{,}998 \cdot 0{,}002/2000} \approx 0{,}001$, tehát az érték összhangban van az elméleti $\geq 0{,}95$ korláttal. Ez empirikus, nem bizonyítás. A teszt BCE 0,445 a baseline 0,693-hoz képest; a tesztpontosság 0,789 a 0,51-es baseline-hoz képest. Ezek a konkrét adatgenerálóra vonatkozó, egyszer mért eredmények, nem általános modellképesség-állítások. A tesztet a validációs választás lezárása után egyszer értékeltük, tehát a 8.3. Kapacitás, regularizálás és modellválasztás adaptivitási tilalma teljesül.

8.4.6 F1–F5 feladatok

F1 (fogalmi, nehézség 1, előfeltétel: 8.1. Kockázat, mintavétel és baseline). Definiálja a $R_D(f)$ kockázat és az $\hat{R}_m(f)$ empirikus kockázat különbségét. Adjon konkrét elrendezést, amelyben $\hat{R}_m(f) = 0$, de $R_D(f) = 0{,}5$. Beadandó: definíciók és egy leírt konstrukció a kockázatértékek indoklásával.

F2 (levezetési, nehézség 1, előfeltétel: 8.2. Véges hipotézisosztály egyenletes korlátja). Legyen $|H| = 100$, $m = 500$, $\delta = 0{,}01$. Számítsa ki a véges H korlát sávját, és mondja ki a garantált állítást szövegesen. Beadandó: a sáv numerikus értéke és az állítás pontos megfogalmazása.

F3 (implementációs, nehézség 1, előfeltétel: 7.4. Numerikus stabilitás és jelentésmegőrző implementáció). Írjon `bce_logits_stable(z, y)` függvényt, és ellenőrizze a $(z, y) = (1000, 0)$ és $(-1000, 1)$ pontokon. Hasonlítsa össze a naiv $\log(\sigma(z))$-alapú formulával. Beadandó: kód és a négy kimenet magyarázattal.

F4 (hibakeresési, nehézség 2, előfeltétel: 8.3. Kapacitás, regularizálás és modellválasztás). Egy hallgató 20 tanulási rátát próbál ki a validációs halmazon, a legjobb modellt a tesztre értékeli, majd a kapott tesztpontosságot „a modell várható pontosságának bizonyítékaként” írja a jelentésbe. Melyik állítás hibás, mi a torzítás iránya, és mi a helyes protokoll? Beadandó: a hiba lokalizálása, a torzítás iránya indoklással és a javított eljárás.

F5 (kísérlettervezési, nehézség 3, előfeltétel: 8.2. Véges hipotézisosztály egyenletes korlátja és 8.3. Kapacitás, regularizálás és modellválasztás). Tervezze meg a lefedettségi szimuláció teljes protokollját a fenti négy hipotézisre: mi a generáló eloszlás, a minta mérete, az

ismétlésszám, a mért metrika, a várható kimenet és a következtetési határ? Beadandó: protokoll leírása, a standard hiba képlete és az érvényességi korlát kimondása.

8.4.7 Megoldások és indoklás

F1. A $R_D(f)$ a teljes D eloszlás feletti várható veszteség; $\hat{R}_m(f)$ a m elemű tanítóminta átlaga. Az előbbi nem számolható ismeretlen D mellett, az utóbbi igen. Konstrukció: a fenti D, $Y = 1[X_1 > 0]$, és a $h_3(x) = 1[x_2 > 0]$ hipotézis. Ekkor $R_D(h_3) = P(Y \neq h_3(X)) = 1/2$, mert X_1 és X_2 függetlenek. Konkrét elrendezés $m = 2$-re: $x_1 = (0{,}5; 0{,}5)$, $y_1 = 1$ és $x_2 = (-0{,}5; -0{,}5)$, $y_2 = 0$. Ekkor $h_3(x_1) = 1$ és $h_3(x_2) = 0$, tehát $\hat{R}_2(h_3) = 0$, miközben $R_D(h_3) = 1/2$. Általános m-re az az esemény, hogy minden i-re $1[x_{i,1} > 0] = 1[x_{i,2} > 0]$, 2^{-m} valószínűségű, de pozitív; a feladat a fenti konkrét elrendezéssel teljesül.

F2. A sáv:

$$\sqrt{\frac{\log 100 + \log(1/0{,}01)}{2 \cdot 500}} = \sqrt{\frac{\log 100 + \log 100}{1000}} = \sqrt{\frac{9{,}2103}{1000}} = 0{,}0960.$$

A garantált állítás: legalább 0,99 valószínűséggel minden $h \in H$-ra $R_D(h) \leq \hat{R}_{500}(h) + 0{,}0960$. Alakellenőrzés: a logaritmusok dimenziómentesek, a nevező mintaméret, a gyök dimenziómentes, ami a kockázat dimenziójával egyezik. Szélsőeset: $m \to \infty$ mellett a sáv nullához tart, ami azt fejezi ki, hogy nagyobb minta mellett az empirikus kockázat jobban koncentrál.

F3. A megoldás a stabil formula:

```
import numpy as np


def bce_logits_stable(z, y):
    z = np.asarray(z, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    return np.maximum(z, 0) - y * z + np.log1p(np.exp(-np.abs(z)))
```

Az ellenőrzés:

```
for z, y in [(1000.0, 0.0), (-1000.0, 1.0)]:
    print(z, y, bce_logits_stable(z, y))
```

A $(1000, 0)$ esetben a stabil formula $\max(1000, 0) - 0 + \log(1 + e^{-1000}) \approx 1000$, ami a matematikai érték; a naiv $-\log(1 - \sigma(1000))$ az $1 - \sigma(1000) = 0$ kerekítés miatt `inf`-et ad. A $(-1000, 1)$ esetben $z = -1000$, $y = 1$: $\max(-1000, 0) = 0$, $-yz = 1000$, és $\log(1 + e^{-1000}) \approx e^{-1000} \approx 10^{-434}$, ezért az eredmény 1000. A BCE matematikai értéke $-\log(\sigma(-1000)) = -\log(e^{-1000}) = 1000$; a naiv $\log(\sigma(-1000))$ a $\sigma(-1000) = 0$ kerekítés miatt `-inf`-et ad. Mindkét ponton a stabil formula véges és helyes; a $\log(1 + e^{-1000})$ tag 10^{-434} nagyságrendű, f64-ben a 1000 mellett elhanyagolható.

F4. A hibás állítás a tesztpontosság értelmezése. A validációs maximum felfelé torzított becslése a várható pontosságnak, mert a maximum művelet a legjobb pillanatnyi teljesítményt választja; a 8.3. Kapacitás, regularizálás és modellválasztás ellenpéldájában $M = 10$ próba 0,999 várható maximumot adott 0,5 valós képesség mellett. A kiválasztott modell tesztpontossága azonban nem felfelé torzított, ha a tesztet a választás után egyszer, érintetlenül értékeljük: a tesztminta a kiválasztott modelltől független, ezért a tesztpontosság a kiválasztott modell várható pontosságának torzítatlan becslése. A valódi hiba az, hogy egyetlen, véges tesztből származó számpontot a modell biztos teljesítményének tekintünk, holott a becslésnek mintavételi bizonytalansága van, és a kiválasztás megismétlésekor a kiválasztott modell (és így a tesztpontosság) is ingadozik.

A helyes protokoll: a keresést a tanító/validációs halmazon zárjuk le, a tesztet a kiválasztott végleges modellen egyszer értékeljük, az eredményt külön jelöljük, a bizonytalanságot (például konfidenciaintervallummal) is megadjuk, a modellválasztási folyamat egészének általánosítási hibáját pedig beágyazott keresztvalidációval becsüljük.

F5. Protokoll: rögzítsük a D generáló eloszlást ($X \sim U[-1,1]^2$, $Y = 1[X_1 > 0]$), a H négyelemű osztályt, az $m = 64$ mintaméretet és a $\delta = 0{,}05$ konfidenciaszintet. Ismételjünk $N_{sim} = 10^4$-szer: generáljunk iid mintát, számítsuk ki minden h-ra az empirikus hibát és a b-sávot, ellenőrizzük az egyenlőtlenséget. A mért metrika a lefedettség, a sikeres realizációk aránya. A binomiális standard hiba $\sqrt{p(1-p)/N_{sim}}$, $p \approx 0{,}97$ mellett $\approx 0{,}0017$, tehát a 0,95 alsó korlátot a becsült $p - 2{\cdot}\mathrm{SE}$ is meghaladja. A következtetési határ: a szimuláció a tétel feltételeinek teljesülése mellett a $1-\delta$ lefedettség empirikus ellenőrzése, de nem a tétel bizonyítása; a tétel a Hoeffding-állításon és az uniókorláton nyugszik, nem a szimuláción.

8.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés és ellenpélda. „Az én futásomban a tanítóhiba 0,03, a tesztpontosság 0,91; ez bizonyítja, hogy a modell általánosan 0,91 pontosságú.” Ez az állítás három helyen sérülhet. Egyetlen futás nem valószínűségi garancia; a tesztpontosság becslés, a saját mintavételi bizonytalanságával; és ha a modellt a teszt alapján választottuk, a becslés torzított. Elvi ellenpélda: a fenti h_3 hipotézis $m = 20$ ponton 2^{-20} eséllyel nulla empirikus hibát ér el, miközben valódi kockázata 0,5. Egy ilyen szerencsés futás naplózott sikere ugyanolyan adat, mint a többi, de nem cáfolja a kockázatot.

Rövid önellenőrzés. Tegyük fel, hogy a hallgató a tesztet ötször futtatja, és a legjobb eredményt jelenti. A jelentés melyik szava lesz hibás: „pontosság” vagy „becslés”? A válasz: mindkettő, ha a validáció fogalmát a tesztre alkalmazta; a teszt ekkor validációs halmazzá vált, és a torzítatlan független teszt feltétele megszűnt.

Korábbi fogalom áttekintése. A 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben a stabil $\log(1+e^z)$-t a $\max(z,0)+\log(1+e^{-|z|})$ azonossággal számoltuk; ez tért vissza a labor BCE-értékében. A 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben a középső differenciás ellenőrzés abszolút hibáját használtuk nullközeli gradiensekhez; ezt a labor `grad_check` függvénye örökölte. A 8.3. Kapacitás, regularizálás és modellválasztás alfejezetben a beágyazott választás felfelé torzító hatását mutattuk meg; ez az F4 feladat háttere.

8.4.9 Rövid összefoglaló

A tanulási állítás teljes lánca: a D eloszlásból vett iid minta adja az empirikus kockázatot, a célfüggvény a kockázat és a regularizáló tag összege, a gradiens a frissítés irányát adja, a sztochasztikus minibatch frissítés pedig a cél közelítését. A véges H korlát megadja azt a sávot, amelyen belül a kockázat az empirikus érték felett marad $1-\delta$ valószínűséggel, feltéve, hogy az osztály előre rögzített, a minta iid, a veszteség $[0,1]$-be esik. A korlát nem garantál alacsony kockázatot, és egyetlen sikeres futás sem bizonyít tételt. A független teszt a keresés lezárása után egyszer mér, és a becslés torzítatlansága a függetlenségen múlik.

8.4.10 Továbblépés és források

A következő egység 9.1. Neuron és perceptron a perceptront és a neuront vezeti be; ott a kockázatminimalizálás kerete először kap konkrét paraméteres modellt és tanulási szabályt.

- [P27] Hoeffding, W. (1963). Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301), 13–30. https://www.tandfonline.com/doi/abs/10.1080/01621459.1963.10500830. Hozzáférés: 2026-10-30.

9 Perceptron, MLP és reprezentáció

9.1 Neuron és perceptron

9.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben a legkisebb paraméterezett osztályozót, a perceptront építjük fel. A cél, hogy három, gyakran összemosott fogalmat pontosan szétválasszunk: a **döntési határt** (decision boundary), az **aktivációt** (activation) és a **tanulási szabályt** (learning rule). Előismeretként a 4.4. I. ellenőrző pont: matematika és kód megfelelése jelölés- és gradientszemléletét, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil numerikáját és a 8.4. II. ellenőrző pont: korrekt tanulási állítás tanulási állítások feltételrendszerét idézzük fel. A bináris cél fogalma az 5.1. Esemény, feltétel és véletlen változó alfejezetből ismerős, az empirikus kockázat pedig a 8.1. Kockázat, mintavétel és baseline alfejezetből.

Induló kérdés: az ÉS (AND) kapu négy bemeneti pontja elválasztható-e egyetlen egyenessel, a kizáró VAGY (XOR) négy pontja pedig miért nem? A válasz előtt pontos definíciókra van szükség.

9.1.2 Definíciók és jelölések

Legyen a bemenet $x \in \mathbb{R}^d$ oszlopvektor; egy N elemű batch sorvektoros jelölése $X \in \mathbb{R}^{N \times d}$. A bináris cél a perceptron konvenciója szerint $y_i \in \{-1, +1\}$; ez nem azonos a 0/1 kódolással, és a tanulási szabály miatt fontos.

Affin score. A perceptron a döntéshez egyetlen valós **score**-t számol:

$$s(x) = w^\top x + b, \qquad w \in \mathbb{R}^d,\ b \in \mathbb{R}.$$

A könyv általános affin konvenciójában ez a $W \in \mathbb{R}^{1 \times d}$ súlysor transzponáltja: $w = W^\top$. Batchben tehát $S = Xw + b\mathbf{1}_N$, alakja $N \times 1$, a b skalár soriányban broadcastolódik. A w vektor a döntési határ normálisa, b az eltolás.

Threshold és aktiváció. Klasszikusan küszöbértékkel is írható: a neuron akkor ad $+1$-et, ha $w^\top x \geq \theta$. A θ küszöb és a b bias közötti kapcsolat $b = -\theta$. Az **aktiváció** a score-ra alkalmazott nemlineáris függvény:

$$f(x) = \operatorname{sign}(s(x)) = \begin{cases} +1, & s(x) \geq 0, \\ -1, & s(x) < 0. \end{cases}$$

Rögzítjük, hogy a határon $\operatorname{sign}(0) = +1$. Ez a függvény nem folytonos, ezért a klasszikus értelemben vett deriváltja a nullában nem létezik; ahol pedig létezik, ott az értéke nulla. Emiatt a gradienslépés vagy nem értelmezett, vagy nem mozdítaná el a paramétereket; ez az oka annak, hogy a perceptron tanítása nem közönséges gradienslépés.

Döntési határ. Ha $w \neq 0$, a $\{x \in \mathbb{R}^d : s(x) = 0\}$ halmaz egy affin hipersík. A $w = 0$ degenerált eset kivétel: ekkor $b = 0$ mellett ez a halmaz a teljes tér, $b \neq 0$ mellett üres, és az alábbi távolságképlet nullával osztana. A döntési határ a paraméterezett modell geometriai objektuma, míg az aktiváció egy pont kiértékelése. A két paraméterezés különböző skálájú w, b párokkal ugyanazt a hipersíkot adhatja; erre az F1 feladat kérdez rá. A hipersík mint halmaz negatív skalárral is változatlan, de az aktiváció és a predikció csak pozitív skalároknál marad azonos, negatív szorzó esetén előjelet vált. A score előjeles távolságtartalma ($w \neq 0$ esetén): a döntési határ távolsága egy x ponttól $|s(x)|/\|w\|_2$. A **margin** egy pontra az előjeles távolság $y\, s(x)/\|w\|_2$, egy tanítóhalmazra pedig ezek minimuma; ez a normálás teszi a w normáját lényegessé a margin szempontjából.

Lineáris szeparálhatóság. A $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ tanítóhalmaz **lineárisan szeparálható** (linearly separable), ha létezik (w, b), amelyre minden i-re

$$y_i(w^\top x_i + b) > 0.$$

A szigorú egyenlőtlenség azt jelenti, hogy egyetlen tanítópont sincs a döntési határon. Ha $y_i = +1$, a pontnak a pozitív féltérben, ha $y_i = -1$, a negatív féltérben kell lennie. Ez a feltétel a modell **kapacitására** vonatkozó adatfüggő tulajdonság, nem az algoritmusé.

Perceptron tanulási szabály. A perceptron algoritmus a hibás pontokra heurisztikus frissítést alkalmaz. Rögzített $\eta > 0$ tanulási rátával, $y_i s_i \leq 0$ esetén, ahol $s_i = s(x_i)$:

$$w \leftarrow w + \eta\, y_i x_i, \qquad b \leftarrow b + \eta\, y_i.$$

A $y_i s_i \leq 0$ feltétel a határpontot is hibának tekinti, mert a konvergenciatétel szigorú elválasztást céloz. A frissítés iránya: ha $y_i = +1$ és a score túl kicsi, a w elmozdul x_i irányába, így a score nő; ha $y_i = -1$, az ellenkező irányba. Ez a szabály nem egy sima veszteségfüggvény gradiense, ellentétben a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben vizsgált differenciálható gráffal. A 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil numerikájára itt nincs szükség: a perceptron csak szorzást és összeadást végez, a numerikus stabilitás a logisztikus modellnél válik lényegessé.

Számpélda a frissítésre: legyen $w = (0, 0)^\top$, $b = 0$, $\eta = 1$, és a hibás pont $x = (1, 0)^\top$, $y = +1$. Itt $s = 0$ és $ys = 0 \leq 0$, ezért $w \leftarrow (1, 0)^\top$, $b \leftarrow 1$; ugyanerre a pontra az új score $1 > 0$, tehát a pont helyes lesz.

Klasszikus konvergenciatétel. Ha a tanítóhalmaz lineárisan szeparálható, akkor a perceptron algoritmus tetszőleges rögzített $\eta > 0$, rögzített kezdő (w_0, b_0) és bármilyen olyan sorrend mellett, amelyben minden frissítés egy hibás mintára történik (például ciklikus bejárással), véges sok frissítés után olyan (w, b)-t ad, amely az összes tanítópontot helyesen osztályozza. Az eredmény Rosenblatt klasszikus tétele [P01]; a szokásos konvergenciabizonyítás Novikoff [P02] eredménye. A perceptron tárgyalását lásd [B02]. A tétel itt a lineáris szeparálhatóságot, a rögzített pozitív tanulási rátát és a rögzített kezdőállapotot feltételező alakjában szerepel. Nem szeparálható adatra a tétel nem ígér konvergenciát.

9.1.3 XOR lineáris szeparálhatatlansága

A kötelező levezetési tételt a négy ponton bizonyítjuk. Legyenek a bemenetek a $[0, 1]^2$ négyzet csúcsai, a cél pedig a kizáró VAGY:

$$(0,0)\mapsto -1,\quad (1,1)\mapsto -1,\quad (0,1)\mapsto +1,\quad (1,0)\mapsto +1.$$

A bizonyításhoz a konvex burok fogalmát használjuk. Egy $S\subseteq\mathbb{R}^d$ halmaz konvex burka

$$\operatorname{conv}(S)=\Big\{\sum_{i=1}^{m}\lambda_i x_i:\ m\in\mathbb{N},\ x_i\in S,\ \lambda_i\ge 0,\ \sum_i\lambda_i=1\Big\}.$$

Lemma. Ha egy $S_+\subseteq\mathbb{R}^d$ halmaz minden x_i pontjára $w^\top x_i+b>0$, akkor $\operatorname{conv}(S_+)$ minden pontjára is $w^\top z+b>0$; ugyanígy ≥ 0 és <0 esetén is. Bizonyítás: legyen $z=\sum_i\lambda_i x_i$. A score affin volta és $\sum_i\lambda_i=1$ miatt

$$w^\top z+b=\sum_i\lambda_i w^\top x_i+b=\sum_i\lambda_i(w^\top x_i+b),$$

ahol a második egyenlőségnél $b=b\sum_i\lambda_i$-t használtuk. Mivel $\lambda_i\ge 0$ és $\sum_i\lambda_i=1$, legalább egy λ_i szigorúan pozitív, és az ehhez tartozó zárójeles tag szigorúan pozitív; a többi tag nemnegatív, ezért az összeg >0. Ugyanígy, ha minden $x_i\in S_-$ pontra $w^\top x_i+b<0$, akkor a teljes $\operatorname{conv}(S_-)$-ra is negatív. $\square$

Tétel. Nincs olyan $(w,b)\in\mathbb{R}^2\times\mathbb{R}$, amelyre a fenti négy XOR-pont mindegyikére $\operatorname{sign}(w^\top x+b)=y$.

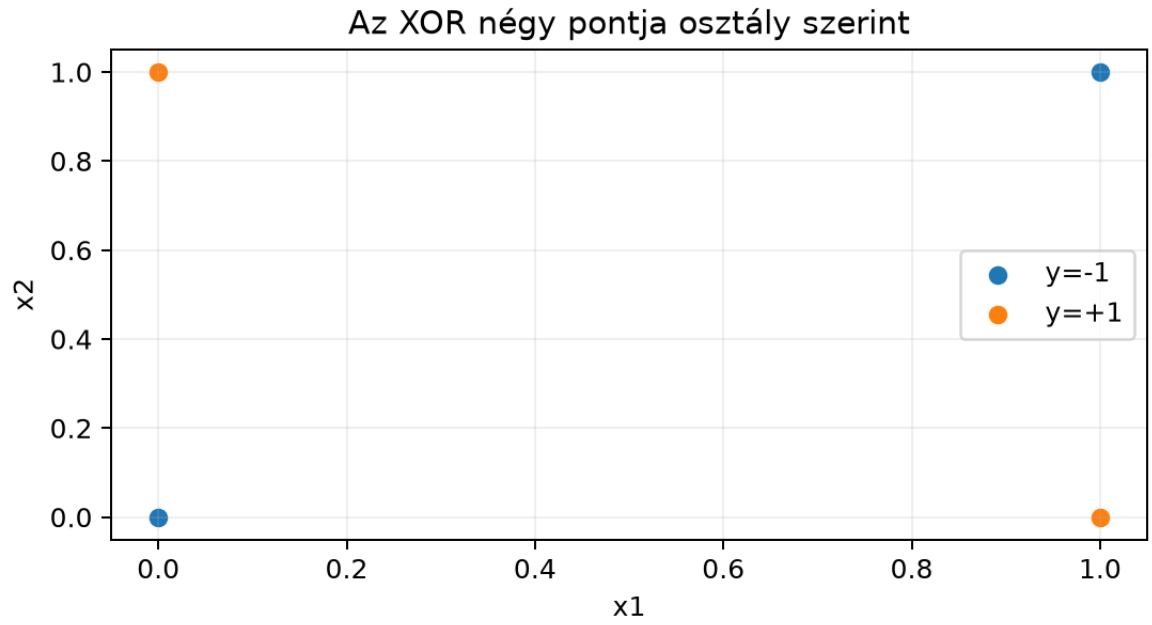
Bizonyítás. Indirekt tegyük fel, hogy létezik ilyen (w,b). Legyen $S_+=\{(0,1),(1,0)\}$ és $S_-=\{(0,0),(1,1)\}$. A $\operatorname{sign}(0)=+1$ konvenció miatt a feltétel azt jelenti, hogy S_+ pontjaira $s(x)=w^\top x+b\ge 0$, S_- pontjaira pedig $s(x)<0$. A lemma (≥ 0 és a <0 változatban, ugyanazzal a bizonyítással) miatt $\operatorname{conv}(S_+)\subseteq\{s\ge 0\}$ és $\operatorname{conv}(S_-)\subseteq\{s<0\}$.

A pozitív osztály konvex burka a $(0,1)$ és $(1,0)$ közötti szakasz, a negatívé a $(0,0)$ és $(1,1)$ közötti szakasz. A két szakasz azonban metszi egymást, mert

$$(0.5,0.5)=\frac{1}{2}(0,1)+\frac{1}{2}(1,0)=\frac{1}{2}(0,0)+\frac{1}{2}(1,1).$$

Tehát ugyanaz a $z=(0.5,0.5)$ pont eleme $\operatorname{conv}(S_+)$-nak és $\operatorname{conv}(S_-)$-nak is. A lemmából erre a pontra egyszerre kellene $w^\top z+b\ge 0$ és $w^\top z+b<0$, ami lehetetlen. Az ellentmondás bizonyítja, hogy ilyen lineáris osztályozó nem létezik. $\square$

Geometriai olvasatban: a döntési határ egy egyenes, amely a síkot két félre osztja. A pozitív pontok konvex burka a másik átló, a negatívé az egyik átló; a két átló metszi egymást, ezért egyetlen egyenes sem tudja a két halmazt szigorúan elválasztani. A tétel bizonyítása saját, elemi levezetés.



9.1. ábra: Az XOR négy pontja osztály szerint

A tételben szereplő négy pont koordinátái; nem futási kimenet.

9.1.4 Kézi példa: az AND kapu

Az ÉS kapu célja a 0/1 kódolásból a perceptron ± 1 céljára áttérve:

$$(0,0) \mapsto -1, \quad (0,1) \mapsto -1, \quad (1,0) \mapsto -1, \quad (1,1) \mapsto +1.$$

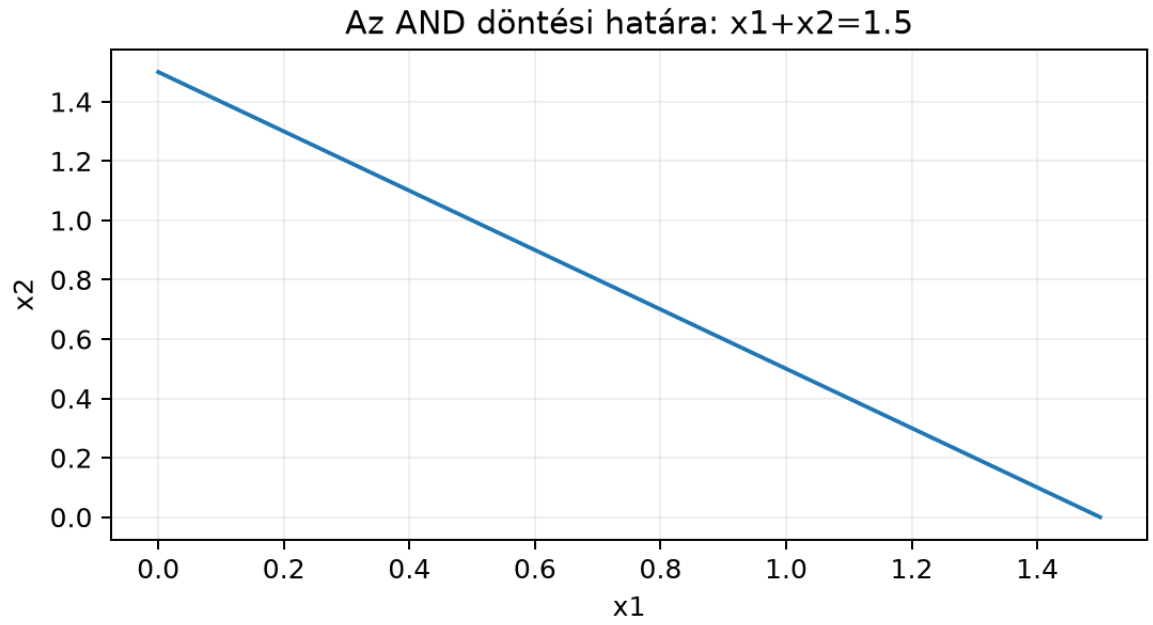
Válasszuk a $w = (1,1)^\top$, $b = -1.5$ paramétereket. A score:

$$s(x_1, x_2) = x_1 + x_2 - 1.5.$$

Bemenet	Score	Aktiváció	Cél
$(0,0)$	-1.5	-1	-1
$(0,1)$	-0.5	-1	-1
$(1,0)$	-0.5	-1	-1
$(1,1)$	0.5	$+1$	$+1$

Mind a négy pont helyes, tehát a halmaz lineárisan szeparálható. A döntési határ:

$$x_1 + x_2 - 1.5 = 0, \qquad \text{azaz} \qquad x_2 = 1.5 - x_1.$$



9.2. ábra: Az AND döntési határa: x1+x2=1.5

A képletből számított értékek; nem futási kimenet.

A $(1,1)$ pont $|0.5|/\sqrt{2}$ távolságra van a határtól, a negatív pontok pedig a határ túloldalán. A megadott (w, b) nem az egyetlen helyes paraméterezés: például a $(2,2)^\top$ és -3 ugyanazt a döntési határt adja, de a score-ok kétszeresek. A $b = -1{,}5$ kézzel választott érték: $\eta = 1$ és 0/1 bemenetek mellett a tanulási szabály minden frissítése egészértékű, ezért a labor algoritmusa ilyen bias-t nem állít elő; a laborban kapott példa a következő szakaszban szerepel.

9.1.5 Labor: perceptron NumPy-ban

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor célja: tanítsunk perceptront kis szeparálható adaton, majd mutassuk meg, hogy a nem szeparálható XOR-on a tanítási ciklus soha nem éri el a nulla hibát. Munkakönyvtár: `lab/n033_perceptron/`. A kód CPU-n fut, NumPy-val, `float64` dtype-tal. A seed a keverés determinisztikusságát biztosítja, nem globális bitazonossági ígéret.

```
# lab/n033_perceptron/perceptron_lab.py
# Futtatás: python perceptron_lab.py
import numpy as np

def train_perceptron(X, y, eta=1.0, max_epochs=100, seed=20261020):
    """Perceptron-tanítás. X: (N,d) float64, y: (N,) float64 ±1."""
    rng = np.random.default_rng(seed)
    d = X.shape[1]
    w = np.zeros(d, dtype=X.dtype)
    b = 0.0
    for epoch in range(max_epochs):
        idx = rng.permutation(len(y))
        errors = 0
        for i in idx:
            x_i = X[i]
```

```
            y_i = y[i]
            s = float(np.dot(w, x_i) + b)
            # A határpontot is hibának tekintjük: y_i*s <= 0.
            if y_i * s <= 0.0:
                w += eta * y_i * x_i
                b += eta * y_i
                errors += 1
        if errors == 0:
            return w, b, epoch + 1, True
    return w, b, max_epochs, False

# AND: lineárisan szeparálható tanítóhalmaz
X_and = np.array(
    [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]],
    dtype=np.float64,
)
y_and = np.array([-1.0, -1.0, -1.0, 1.0], dtype=np.float64)

w_and, b_and, epochs_and, conv_and = train_perceptron(
    X_and, y_and, eta=1.0, max_epochs=50
)
pred_and = np.where(X_and @ w_and + b_and >= 0.0, 1.0, -1.0)
print(f"AND: w={w_and}, b={b_and}, konvergencia epoch={epochs_and}, "
      f"konvergens={conv_and}, hiba={int(np.sum(pred_and != y_and))}")
assert conv_and and epochs_and <= 50
assert np.array_equal(pred_and, y_and)

# XOR: nem szeparálható, a ciklus nem állhat meg
X_xor = np.array(
    [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]],
    dtype=np.float64,
)
y_xor = np.array([-1.0, 1.0, 1.0, -1.0], dtype=np.float64)

w_xor, b_xor, epochs_xor, conv_xor = train_perceptron(
    X_xor, y_xor, eta=1.0, max_epochs=100
)
pred_xor = np.where(X_xor @ w_xor + b_xor >= 0.0, 1.0, -1.0)
print(f"XOR: w={w_xor}, b={b_xor}, lefutott epoch={epochs_xor}, "
      f"konvergens={conv_xor}, hibák={int(np.sum(pred_xor != y_xor))}")
assert not conv_xor
assert epochs_xor == 100
assert int(np.sum(pred_xor != y_xor)) >= 1
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál; a seed 20261020):

```
AND: w=[2. 3.], b=-4.0, konvergencia epoch=12, konvergens=True, hiba=0
XOR: w=[0. 0.], b=-1.0, lefutott epoch=100, konvergens=False, hibák=2
```

Az ellenőrzés két részből áll. Az AND-futásnak `conv_and = True` értéket kell adnia, és a konvergencia epochja nem lehet több a megadott maximumnál; ekkor a kapott (w, b) mind a négy pontot jól osztályozza, amit a kiírt `hiba=0` is mutat. A mért futásban $w = (2, 3)$, $b = -4{,}0$ adódott; más frissítési sorrend más elválasztó hipersíkhoz is vezethet. Az XOR-futás ezzel szemben `conv_xor = False` és `epochs_xor == 100` lesz, a végső paraméterekre pedig a 0/1 hibaszám legalább 1. A tanítás hibafeltétele a szigorúbb perceptron-kritérium ($y_i s_i \le 0$), a kiírt `hiba`/`hibák` viszont a $\operatorname{sign}(0) = +1$ konvenció szerinti 0/1 hiba; a kettő csak a határra eső pontokban tér el, és a tanítás azért használja a szigorúbbat, mert a konvergenciatétel szigorú elválasztást biztosít. Ez nem a kód hibája, hanem a fenti tétel következménye: nincs olyan lineáris osztályozó, amely mind a négy XOR-pontot helyesen osztályozná.

Az ellenőrzött laborjelentések mintáját a kezdőlabor `lab/ellenorzott_report.json` fájlja rögzíti; a fenti kimenet a megadott kóddal futtatott, tényleges gépi napló.

9.1.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja egy-egy mondatban a döntési határt, az aktivációt és a tanulási szabályt a perceptron esetében. Ezután adjon meg két különböző (w, b) párt a fenti AND-példához, amelyek ugyanazt a döntési határt határozzák meg, és mutassa meg, hogy a predikciók azonosak, de a score értékek nem. Beadandó: a három definíció és a két konkrét paraméterezés ellenőrzéssel.

F2 (levezetési, nehézség 2). Bizonyítsa, hogy az OR (VAGY) függvény a négy $[0,1]^2$ ponton lineárisan szeparálható. Adjon meg egy (w, b) párt, írja fel mind a négy pontra a $y_i(w^\top x_i + b) > 0$ egyenlőtlenséget, és igazolja, hogy mindegyik teljesül. Beadandó: a paraméterek és a négy numerikus egyenlőtlenség.

F3 (implementációs, nehézség 1). Adott $w = (0.5, -0.5)^\top$, $b = 0.5$, $x = (1, 2)^\top$, $y = +1$ és $\eta = 0.1$. Számítsa ki kézzel a score-t, a predikciót és a perceptron frissített w, b értékét. Írjon ehhez egy rövid NumPy kódot, amely `assert` utasításokkal ellenőrzi a kézi számítást. Beadandó: a kézi számolás és a futtatható kód.

F4 (hibakeresési, nehézség 2). Az alábbi kód AND-ra nem konvergál:

```
import numpy as np
X_and = np.array([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_and = np.array([-1.0, -1.0, -1.0, 1.0])
w = np.zeros(2)
eta = 1.0
for _ in range(50):
    for x_i, y_i in zip(X_and, y_and):
        s = float(w @ x_i)      # bias hiányzik a score-ból
        if y_i * s <= 0.0:
            w += eta * y_i * x_i
            # b += eta * y_i   # a bias frissítése kimaradt
```

Magyarázza el, miért lehetetlen ezzel a kóddal az AND megtanulása, mi a következménye a $(0, 0)$ pontra nézve, javítsa a hibát, és mutassa meg, hogy a javított változat nulla hibát ér el. Beadandó: ok, következmény, javított kód és az ellenőrzés módja.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen perceptron-kísérletet a függelék A. szakaszában leírt GlyphLab `vertical` és `horizontal` osztályaira. Két, Ön által definiált jellemzőt használjon, például a szomszédos oszlopok, illetve sorok abszolút különbségének átlagát (vízszintes, illetve függőleges irányú gradiens). A terv tartalmazza: a megfigyelési egységet, a split kezelését, a baseline-t, a metrikát, a maximális epochok számát és az érvényességi korlátot arra az esetre, ha az adat nem lineárisan szeparálható. Beadandó: írott protokoll; futtatás nem kötelező.

9.1.7 Megoldások és indoklás

F1. A döntési határ a $\{x : w^\top x + b = 0\}$ hipersík; az aktiváció a score-ra alkalmazott sign függvény, amely ± 1 predikciót ad; a tanulási szabály a hibás pontokon végrehajtott $w \leftarrow w + \eta y_i x_i$, $b \leftarrow b + \eta y_i$ frissítés. Ugyanazt az AND-határt adják például a $(1, 1)^\top, -1{,}5$ és a $(2, 2)^\top, -3$ párok: az első egyenlete $x_1 + x_2 - 1{,}5 = 0$, a másodiké $2x_1 + 2x_2 - 3 = 0$, ami ugyanaz az egyenes; a négy ponton a score-ok $(-1{,}5; -0{,}5; -0{,}5; 0{,}5)$, illetve ezek kétszeresei, a predikciók pedig mindkét párral $-1, -1, -1, +1$. Minden x-re a második score az első kétszerese, így pozitív

skálázás miatt az előjelek és a predikciók azonosak, de a score-ok nem. Negatív szorzó esetén a hipersík ugyanaz marad, de a score előjele megfordul, ezért a predikciók is; ilyen pár tehát nem példa az azonos predikcióra.

F2. Az OR célja ± 1 kódolásban: $(0,0) \mapsto -1$, $(0,1) \mapsto +1$, $(1,0) \mapsto +1$, $(1,1) \mapsto +1$. Válasszuk $w = (1,1)^\top$, $b = -0.5$. A négy feltétel:

$$-1 \cdot (0 + 0 - 0.5) = 0.5 > 0, \quad +1 \cdot (0 + 1 - 0.5) = 0.5 > 0,$$
$$+1 \cdot (1 + 0 - 0.5) = 0.5 > 0, \quad +1 \cdot (1 + 1 - 0.5) = 1.5 > 0.$$

Mind a négy szigorú egyenlőtlenség teljesül, tehát a halmaz lineárisan szeparálható. A döntési határ $x_1 + x_2 = 0.5$, amely a $(0,0)$ pontot elkülöníti a másik háromtól.

F3. A score:

$$s = 0.5 \cdot 1 + (-0.5) \cdot 2 + 0.5 = 0.5 - 1.0 + 0.5 = 0.0.$$

A rögzített konvenció szerint $\operatorname{sign}(0) = +1$, tehát a predikció $+1$. A tanulási szabály azonban a $y_i s_i \le 0$ feltételt használja, és itt $1 \cdot 0 = 0 \le 0$, ezért frissít:

$$w = (0.5, -0.5) + 0.1 \cdot 1 \cdot (1,2) = (0.6, -0.3), \qquad b = 0.5 + 0.1 \cdot 1 = 0.6.$$

A kód:

```
import numpy as np
w = np.array([0.5, -0.5], dtype=np.float64)
b = 0.5
x = np.array([1.0, 2.0], dtype=np.float64)
y = 1.0
eta = 0.1
s = float(w @ x + b)
assert y * s <= 0.0
w += eta * y * x
b += eta * y
assert np.allclose(w, [0.6, -0.3], atol=1e-12)
assert np.isclose(b, 0.6, atol=1e-12)
```

Alakellenőrzés: w és x is $(2,)$ alakú, a belső szorzat skalár, a frissítés dimenzióhelyes.

F4. A hiba oka: a score-ból hiányzik a b, és a frissítés sem módosítja a bias-t. Ekkor a döntési határ mindig átmegy az origón, mert $s(0,0) = 0$ függetlenül w értékétől. Az AND célja szerint $(0,0)$ a -1 osztályba tartozik, tehát a $y_i s_i \le 0$ feltétel minden epochban hibát jelez erre a pontra. A frissítés azonban $w \leftarrow w - \eta \cdot (0,0)$, vagyis w nem változik az origóra; a hiba örökké fennmarad. A javítás a bias beillesztése és frissítése:

```
import numpy as np
X_and = np.array([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_and = np.array([-1.0, -1.0, -1.0, 1.0])
w = np.zeros(2)
b = 0.0
eta = 1.0
for _ in range(50):
    errors = 0
    for x_i, y_i in zip(X_and, y_and):
        s = float(w @ x_i + b)
        if y_i * s <= 0.0:
            w += eta * y_i * x_i
```

```
            b += eta * y_i
            errors += 1
    if errors == 0:
        break
pred = np.where(X_and @ w + b >= 0.0, 1.0, -1.0)
print(f"w={w}, b={b}, hiba={int(np.sum(pred != y_and))}")
assert np.array_equal(pred, y_and)
```

Visszaellenőrzés: a javított kódban a $(0,0)$ pontra b negatívvá válik, így $s(0,0) < 0$, és a hiba megszűnik. A fenti önálló változat futtatva $w = (3,2)$, $b = -4$, hiba 0 eredményt ad; a labor `train_perceptron` függvénye ugyanezt a viselkedést `conv_and = True` értékkel jelzi.

F5. Egy lehetséges protokoll. Kérdés: a két jellemzőn lineárisan szeparálható-e a vertical/horizontal osztály? Adat: a függelék A. A GlyphLab adatkészlet szakaszában rögzített `data/images.npy`, `data/labels.npy` és `data/metadata.csv`; az utóbbi `scene_id` és `split` oszlopa adja a jelenetszintű felosztást. A két osztály (`vertical`, `horizontal`) szűrése után csak a train halmazon tanítunk, a val és test halmaz független kiértékelés. Jellemzők: vízszintes irányú gradiens f_1 = a szomszédos oszlopok abszolút különbségének átlaga (a függőleges éleket emeli ki); függőleges irányú gradiens f_2 = a szomszédos sorok abszolút különbségének átlaga (a vízszintes éleket emeli ki); mindkettőt float64-ben számoljuk (a tárolt képek float32-ek, a jellemzőszámításnál alakítjuk át), összhangban a labor perceptronjának float64 dtype-ával. Megfigyelési egység: jelenet, mert a split jelenetszintű; a tanítási hibát a train képeken mérjük, a validáció és a teszt ettől független kiértékelés. Baseline: mindig a gyakoribb osztály predikciója. Metrika: 0/1 hiba a train képeken (ez a tanítási hiba), valamint a független val/test halmazokon. Erőforrás: CPU, NumPy, maximum 100 epoch. Korlát: ha a tanítási hiba nem éri el a nullát, a perceptron modell a két jellemzőn nem lineárisan szeparálható; ekkor nem vonhatunk le következtetést a perceptron „pontosságáról”, legfeljebb a baseline-hoz viszonyított heurisztikus eltérést mérhetjük. A teszt csak a végleges paraméterek rögzítése után értékelhető, ahogy azt a 8.4. II. ellenőrző pont: korrekt tanulási állítás lezárta.

9.1.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. „A perceptron azért tanul, mert az agy neuronjait utánozza.” Ez a biológiai analógia motiválhatja a modellt, de nem matematikai bizonyíték semmilyen tulajdonságra. A biológiai idegsejtek létezése nem segít az XOR megtanulásában; a fenti tétel bizonyítja, hogy egyetlen perceptron erre matematikailag képtelen, bármennyire is „neuron-szerű”. Az analógia heurisztika, a szeparálhatóság tétel.

Önellenőrzés. Tudja-e: (1) felírni a score, a döntési határ és az aktiváció képletét; (2) megmagyarázni, miért $y_i s_i \leq 0$ a frissítési feltétel és miért nem $y_i s_i < 0$; (3) bizonyítani a XOR tételt a két átlós konvex burok metszetével; (4) megmondani, miért nem gradienslépés a perceptron frissítés?

Visszatekintés. A 4.4. I. ellenőrző pont: matematika és kód megfelelése gráfjában a frissítés egy differenciálható loss gradiense volt; itt a sign nemsima művelet, ezért a tanulási szabály külön fogalom. A 8.1. Kockázat, mintavétel és baseline empirikus kockázat nyelvén a perceptron a 0/1 hibát csökkenti heurisztikus módon, anélkül hogy a populációs kockázatra koncentrációs korlátot adnánk. Az 5.1. Esemény, feltétel és véletlen változó bináris célja itt ± 1 kódolásban jelenik meg; a valószínűségi értelmezéshez a következő fejezetekben a logisztikus modellre lesz szükség.

9.1.9 Rövid összefoglaló

A perceptron egyetlen affin score-t, $s(x) = w^\top x + b$, számol, majd a sign aktivációval ± 1 előrejelzést ad. A döntési határ a $s = 0$ hipersík, és a tanítóhalmaz lineáris szeparálhatósági feltétele, hogy minden $y_i s_i > 0$. A tanulási szabály hibás pontokon $w \leftarrow w + \eta y_i x_i$, $b \leftarrow b + \eta y_i$ frissítést végez; lineárisan szeparálható adatra a klasszikus tétel véges konvergenciát ígér. Az XOR négy pontján konvex burkokkal bizonyítottuk, hogy nincs elválasztó hipersík: a két osztály konvex burka metszi egymást. A modell tehát világosan korlátozott, és a korlát nem biológiai, hanem geometriai állítás.

9.1.10 Továbblépés és források

A következő 9.2. MLP és nemlinearitás egységben az MLP-t vezetjük be, ahol rejtett rétegek és nemlineáris aktivációk teszik lehetővé a nemlineáris döntési határok, így az XOR megtanulását is. A perceptron a legegyszerűbb építőelem, amelynek korlátja motiválja a mélyebb modelleket.

- [P01] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. https://doi.org/10.1037/h0042519. Hozzáférés: 2026-10-30.
- [P02] Novikoff, A. B. J. (1962). On convergence proofs on perceptrons. In *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

9.2 MLP és nemlinearitás

9.2.1 Cél, előismeret és induló kérdés

Ez az egység a 9.1. Neuron és perceptron perceptron közvetlen folytatása. Ott beláttuk, hogy az XOR négy pontja lineárisan szeparálhatatlan, ezért egyetlen affin score és küszöbözés nem oldhatja meg. Most azt vizsgáljuk, mit nyerünk, ha több affin réteget komponálunk, és közéjük nemlineáris aktivációt teszünk.

Előismeret a 9.1. Neuron és perceptron alfejezetből: affin score $s(x) = w^\top x + b$, küszöbözés, lineáris szeparálhatóság. A mátrixműveletekhez a 3.2. Mátrixszorzás és affín réteg, a batch-alakokhoz a 2.1. Python, NumPy és tenzoralak konvencióit használjuk. A kapacitás és a hipotézisosztály mérete a 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben épült fel, és itt a paraméterszám értelmezésénél térünk vissza rá röviden.

Induló kérdés: két affin réteget kötünk össze, de a kettő közé nem teszünk semmilyen aktivációt. Meg tudja-e tanulni ez a modell az XOR-t? És hány paraméter kell ahhoz, hogy egy rejtett rétegű hálózat a négy XOR-pontot elválassza? Mielőtt tovább olvasna, becsülje meg a 2–2–1 méretű hálózat paraméterszámát.

9.2.2 MLP, rejtett réteg és a három alapaktiváció

Definíció (két rétegű MLP). Legyen d a bemeneti dimenzió, m a rejtett szélesség, K a kimeneti dimenzió. A modell három lépésből áll:

1. rejtett pre-aktiváció: $a^{(1)} = W^{(1)}x + b^{(1)}$, ahol $W^{(1)} \in \mathbb{R}^{m \times d}$, $b^{(1)} \in \mathbb{R}^m$, $x \in \mathbb{R}^d$;

2. rejtett aktiváció: $h = \varphi(a^{(1)})$, ahol φ koordinátánként hat: $\varphi(u)_j = \varphi(u_j)$;
3. kimeneti logit: $z = W^{(2)}h + b^{(2)}$, ahol $W^{(2)} \in \mathbb{R}^{K\times m}$, $b^{(2)} \in \mathbb{R}^K$.

Batchelt alakban, $X \in \mathbb{R}^{N\times d}$ sor-megfigyelésekkel:

$$Z = \varphi\big(XW^{(1)\top} + b^{(1)}\big)\, W^{(2)\top} + b^{(2)},$$

ahol $b^{(1)} \in \mathbb{R}^m$ sorirányban broadcastolódik az $N \times m$ mátrixhoz, és a kimenet $Z \in \mathbb{R}^{N\times K}$. Osztályozásban Z logit, tehát aktiváció előtti pontszám (a 9.1-beli affin score általánosítása); a softmaxot, amely a logitvektort valószínűségvektorrá normalizálja, vagy a döntési küszöböt később alkalmazzuk, nem a hálózat belsejében. A softmax képlete $\mathrm{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$; a logit-utáni valószínűség ezzel a normalizálással adódik. A továbbiakban a logit, a score és a pontszám ugyanerre az aktiváció előtti mennyiségre utal; a Z mátrix sorai batch-enként egy-egy ilyen vektort tartalmaznak.

A rejtett rétegen a $W^{(1)}, b^{(1)}, \varphi$ hármast értjük; a h vektort rejtett reprezentációnak nevezzük, mert nem célváltozó, hanem a hálózat közbülső belső ábrázolása.

A három leggyakoribb aktiváció pontos definíciója $u \in \mathbb{R}$-re (a rejtett egységek tipikus választásait a [B02] 6.3 szakasza tárgyalja):

$$\mathrm{ReLU}(u) = \max(0, u), \qquad \sigma(u) = \frac{1}{1+e^{-u}} = \frac{e^{u}}{1+e^{u}}, \qquad \tanh(u) = \frac{e^{u}-e^{-u}}{e^{u}+e^{-u}}.$$

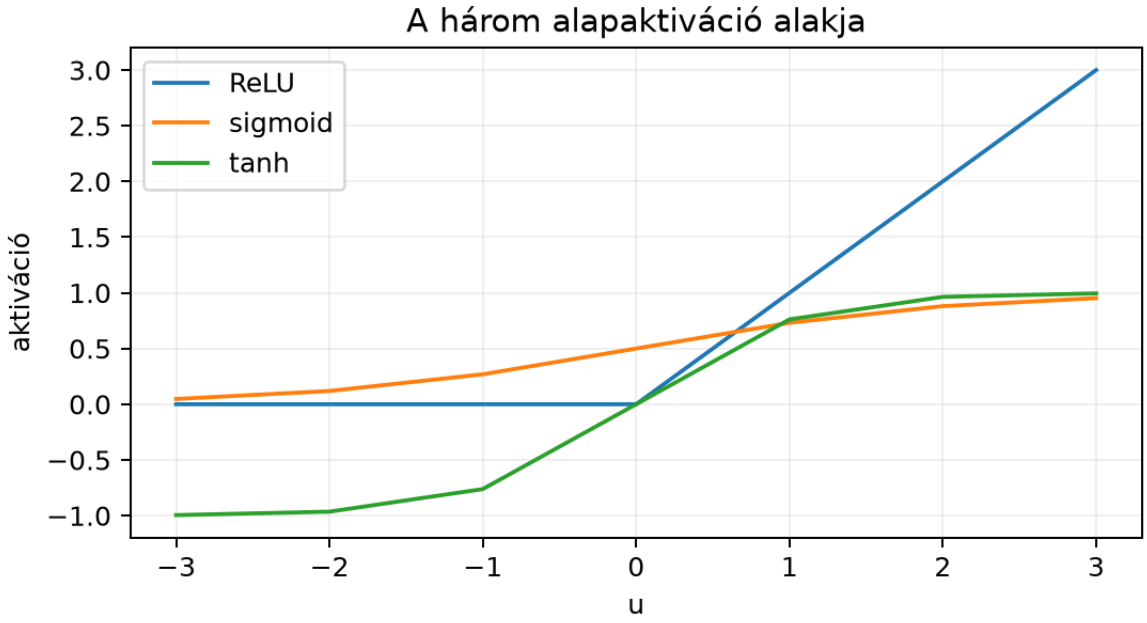
A sigmoid értékkészlete $(0, 1)$, ezért a logisztikus regresszióban valószínűségbecslésként használjuk: a logitra alkalmazva a pozitív osztály valószínűségét adja; a tanh értékkészlete $(-1, 1)$, és zéruscentrált. A kettő között egyszerű azonosság áll fenn: $\tanh(u) = 2\sigma(2u) - 1$. Ezt behelyettesítéssel ellenőrizhetjük:

$$2\sigma(2u) - 1 = \frac{2}{1+e^{-2u}} - 1 = \frac{1-e^{-2u}}{1+e^{-2u}} = \frac{e^{2u}-1}{e^{2u}+1} = \tanh(u).$$

A három aktiváció értéke néhány pontban (a képletekből számított értékek, négy tizedesre kerekítve):

u	$\mathrm{ReLU}(u)$	$\sigma(u)$	$\tanh(u)$
-2	0	0,1192	−0,9640
-1	0	0,2689	−0,7616
0	0	0,5	0
1	1	0,7311	0,7616
2	2	0,8808	0,9640

A táblázat mutatja a ReLU törését 0-ban és a két telítődő aktiváció eltérő meredekségét: a tanh meredekebb, ezért nagyobb bemenetre hamarabb telítődik.



9.3. ábra: A három alapaktiváció alakja

A képletekből számított értékek, négy tizedesre kerekítve; nem mérés.

A ReLU a nulla pontban klasszikus értelemben nem differenciálható; ez a forward számításra nincs hatással, de a későbbi gradiensszámításnál (10.2. Affin réteg és aktiváció backward) konvenciót igényel.

Paraméterszám. A fenti hálózatnak $md + m + Km + K$ paramétere van: $W^{(1)}$ elemei md, $b^{(1)}$ elemei m, $W^{(2)}$ elemei Km, $b^{(2)}$ elemei K. A gyakorlatban a biasokat is paraméternek számoljuk, tehát a 2–2–1 XOR-hálózat paraméterszáma $2 \cdot 2 + 2 + 1 \cdot 2 + 1 = 9$.

9.2.3 Miért nem elég az affin kompozíció: a kollapszustétel

A bizonyítási feladat a következő állítás. A jelenséget **kollapszusnak** nevezzük: az aktiváció nélküli, egymásra következő affin rétegek egyetlen affin leképezéssé omlanak össze, ezért a rétegzés nem bővíti a reprezentálható függvények osztályát.

Tétel (aktiváció nélküli rétegek összevonhatósága). Legyen $W^{(1)} \in \mathbb{R}^{m \times d}$, $b^{(1)} \in \mathbb{R}^m$, $W^{(2)} \in \mathbb{R}^{K \times m}$, $b^{(2)} \in \mathbb{R}^K$. Ha a két réteg között nincs nemlineáris aktiváció, azaz

$$f(x) = W^{(2)}\big(W^{(1)}x + b^{(1)}\big) + b^{(2)},$$

akkor minden $x \in \mathbb{R}^d$-re

$$f(x) = \widetilde{W}x + \tilde{b},$$

ahol

$$\widetilde{W} = W^{(2)}W^{(1)} \in \mathbb{R}^{K \times d}, \qquad \tilde{b} = W^{(2)}b^{(1)} + b^{(2)} \in \mathbb{R}^K.$$

Indukcióval tetszőleges számú affin rétegre ugyanez igaz: a teljes kompozíció egyetlen affin leképezés.

Bizonyítás. A mátrixszorzás disztributív az összeadásra nézve ([B01] 2. fejezet), ezért

$$W^{(2)}\big(W^{(1)}x + b^{(1)}\big) + b^{(2)} = W^{(2)}W^{(1)}x + W^{(2)}b^{(1)} + b^{(2)} = \widetilde{W}x + \tilde{b}.$$

Az $x \mapsto \widetilde{W}x + \tilde{b}$ hozzárendelés éppen egy affin réteg, tehát a két rétegű kompozíció nem hoz be új függvényosztályt. Az indukciós lépés: ha az első $L-1$ affin réteg már $\widetilde{W}x + \tilde{b}$ alakú, akkor az L-edik réteg ezen az összevont alakon alkalmazva ugyanezzel a számolással egyetlen affin leképezéssé vonható össze. Az $L = 1$ alapeset triviális.

A bizonyítás itt teljes, nincs szükség semmilyen további feltételre. Az affin összevonás ugyanakkor nem azt jelenti, hogy a kétrétegű háló és az egyrétegű ugyanazt tudja: ha a köztes szélesség $m < \min(d, K)$ (szűk keresztmetszet), akkor $\operatorname{rank}(W^{(2)}W^{(1)}) \le m$, tehát a kompozíció az affin leképezések valódi részhalmazát adja. Ellenpélda: $d = K = 2$, $m = 1$ mellett az $x \mapsto Ix$ azonosság (rang 2) nem állítható elő. Az összevonás tehát azt mondja, hogy a kompozíció egyetlen affin leképezés, nem azt, hogy minden affin leképezés előáll. Az állítás mélyebb jelentése: a puszta rétegszám-növelés önmagában nem növeli a reprezentálható függvények halmazát. A 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben látott kapacitásérveléshez hasonlóan itt is az a kérdés, hogy a paramétertér mely része érhető el ténylegesen; az aktiváció nélküli mély hálózat ugyanannyit tud, mint az egyrétegű, mert a leképezés ugyanaz.

A tétel következménye az induló kérdésre: aktiváció nélkül az XOR sem oldható meg, bárhogyan választjuk a rétegszámot, mert minden ilyen hálózat lineáris osztályozóvá redukálódik.

9.2.4 ReLU-hálózatok szakaszonként affin szerkezete

A ReLU azért hasznos, mert a belőle épített hálózat geometriája pontosan leírható anélkül, hogy a tanítási dinamikát ismernénk.

Állítás (szakaszonként affin szerkezet). Legyen $\varphi = \mathrm{ReLU}$ és

$$f(x) = W^{(2)}\varphi\big(W^{(1)}x + b^{(1)}\big) + b^{(2)}.$$

Ekkor f folytonos, és $\mathbb{R}^d$ felbontható legfeljebb 2^m olyan poliéder-tartományra, amelyeken f affin.

Bizonyításvázlat saját levezetéssel. Minden $j \in \{1, \dots, m\}$ rejtett neuronhoz tartozik egy lineáris pre-aktiváció $s_j(x) = w_j^{(1)\top}x + b_j^{(1)}$; ez nem más, mint a korábbi $a^{(1)}$ pre-aktiváció j-edik koordinátája, $s_j = a_j^{(1)}$. Rögzítsünk egy $\alpha \in \{0,1\}^m$ aktivációs mintázatot, és definiáljuk

$$D_\alpha = \big\{x \in \mathbb{R}^d : \; s_j(x) > 0 \text{ ha } \alpha_j = 1,\; s_j(x) \le 0 \text{ ha } \alpha_j = 0\big\}.$$

A D_α pontosan m féltér metszete: azokra a j indexekre, ahol $\alpha_j = 1$, a nyílt $\{s_j > 0\}$, ahol $\alpha_j = 0$, a zárt $\{s_j \le 0\}$ féltér szerepel. A D_α tehát félig nyílt poliéder (egyes feltételek szigorúak, mások nem); a lezárása zárt poliéder, és a zárt lezáratok legfeljebb a határokon fedhetik át egymást. A D_α tartományon minden neuron aktivációja rögzített: vagy $\varphi(s_j) = s_j$, vagy $\varphi(s_j) = 0$. Ezért itt h affin függvénye x-nek, és mivel f affin függvénye h-nak, f is affin x-ben. A 2^m darab D_α a fenti definícióval (a > 0 és ≤ 0 ágak miatt) páronként diszjunkt, és együttesen lefedi $\mathbb{R}^d$-t; ha helyettük a lezárt $\{s_j \ge 0\}$, $\{s_j \le 0\}$ régiókat használjuk, azok a határokon átfedhetnek, és a

folytonosság garantálja, hogy az átfedő darabok ott megegyeznek. A folytonosság maga abból következik, hogy a ReLU, az affin leképezések és a kompozíció mind folytonosak.

A darabok száma legfeljebb 2^m: ennyi előjelmintázat lehetséges, de nem mindegyik valósul meg. $\mathbb{R}^d$-ben m hipersík elrendezése legfeljebb $\sum_{i=0}^{d} \binom{m}{i}$ régiót ad, tehát a tényleges darabszám a dimenziótól is függ, és a 2^m csak felső korlát. Ez a korlát a ReLU-hálózatok kifejezőerejének egyik forrása, amelyet a következő 9.3. Expresszivitás és approximációs állítások egység formalizál.

9.2.5 Kézi XOR-példa

Használjuk a 9.1. Neuron és perceptron pontjait 0/1 célokkal:

$$(0,0) \mapsto 0, \quad (1,0) \mapsto 1, \quad (0,1) \mapsto 1, \quad (1,1) \mapsto 0.$$

Válasszuk a következő súlyokat: $d = 2$, $m = 2$, $K = 1$,

$$W^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} -0.5 \\ -1.5 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & -6 \end{pmatrix}, \quad b^{(2)} = 0.$$

A rejtett pre-aktiváció és aktiváció:

$$a^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} x_1 + x_2 - 0.5 \\ x_1 + x_2 - 1.5 \end{pmatrix}, \qquad h = \mathrm{ReLU}(a^{(1)}).$$

A négy pontra:

x	$a^{(1)}$	h	$z = W^{(2)}h$
$(0,0)$	$(-0.5, -1.5)$	$(0,0)$	0
$(1,0)$	$(0.5, -0.5)$	$(0.5,0)$	1
$(0,1)$	$(0.5, -0.5)$	$(0.5,0)$	1
$(1,1)$	$(1.5, 0.5)$	$(1.5,0.5)$	$3-3=0$

A döntési szabály $z > 0.5$: a két középső pont 1 osztályt kap, a két szélső 0-t. Ez a konstrukció tehát a négy ponton tökéletesen megoldja az XOR-t. A hidden térben a négy pont: $(0,0)$, $(0.5,0)$, $(0.5,0)$, $(1.5,0.5)$; a $2h_1 - 6h_2 - 0.5 = 0$ affin határ szigorúan elválasztja a két osztályt.

Érdemes megnevezni a két rejtett egység szerepét. Az első egység pre-aktivációja $x_1 + x_2 - 0.5$, ez akkor aktív, ha $x_1 + x_2 > 0.5$, azaz a négy pont közül háromban: ez az OR-szerű egység. A második egység $x_1 + x_2 - 1.5$ akkor aktív, ha $x_1 + x_2 > 1.5$, azaz csak az $(1,1)$ pontban: ez az AND-szerű egység. A kimenet $2h_1 - 6h_2$; a rögzített súlyok mellett a $z > 0$ feltétel pontosan azt jelenti, hogy $h_1 > 0$ és $h_2 = 0$, azaz az OR teljesül, de az AND nem. A rejtett térben a $h_2 = 0$ félsík választja el egymástól a két ReLU-régiót, és a kimeneti réteg ezt a két régiót rendeli a két osztályhoz.

Vegyük észre a kollapszustétel konkrét hatását: ha a ReLU-t elhagyjuk, akkor

$$\widetilde{W} = W^{(2)}W^{(1)} = \begin{pmatrix} 2 & -6 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} -4 & -4 \end{pmatrix}, \qquad \tilde{b} = W^{(2)}b^{(1)} + b^{(2)} = \begin{pmatrix} 2 & -6 \end{pmatrix} \begin{pmatrix} -0.5 \\ -1.5 \end{pmatrix} = 8,$$

tehát a lineárissá összevont modell $z = -4x_1 - 4x_2 + 8$. A négy ponton ez $(0,0), (0,1), (1,0), (1,1)$ sorrendben $8, 4, 4, 0$ értéket ad. A pozitív osztályú pontok $(4,4)$ a két negatív osztályú pont (8 és 0) logitja között vannak, ezért nincs olyan egyetlen küszöb, amellyel a négy pont mind helyesen volna osztályozva: a $(0,0)$ kapja a legnagyobb, az $(1,1)$ a legkisebb logitot, és a pozitívak középen vannak.

9.2.6 Labor: XOR-forward NumPy-ban

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor célja, hogy a fenti kézi konstrukciót ellenőrizhető kóddal számolja ki. Mivel a konstrukció determinisztikus, seedre (véletlenmagra) nincs szükség; dtype float64, a számítás NumPy-ban, CPU-n fut (külön eszközválasztás nincs).

Munkakönyvtár: `n034_labor/`. Fájl: `xor_mlp_forward.py`.

```
import numpy as np

# dtype float64, CPU, nincs véletlen forrás
W1 = np.array([[1.0, 1.0],
               [1.0, 1.0]], dtype=np.float64)      # (2, 2)
b1 = np.array([-0.5, -1.5], dtype=np.float64)      # (2,)
W2 = np.array([[2.0, -6.0]], dtype=np.float64)     # (1, 2)
b2 = np.array([0.0], dtype=np.float64)             # (1,)

X = np.array([[0.0, 0.0],
              [1.0, 0.0],
              [0.0, 1.0],
              [1.0, 1.0]], dtype=np.float64)       # (4, 2)

def relu(u):
    return np.maximum(0.0, u)

S1 = X @ W1.T + b1              # (4, 2) + (2,) soirányú broadcast
H = relu(S1)                    # (4, 2)
Z = H @ W2.T + b2               # (4, 1)

print("H shape:", H.shape)
print("Z shape:", Z.shape)
print("H:\n", H)
print("Z:\n", Z)

pred = (Z > 0.5).astype(np.int64)
print("pred:\n", pred.ravel())

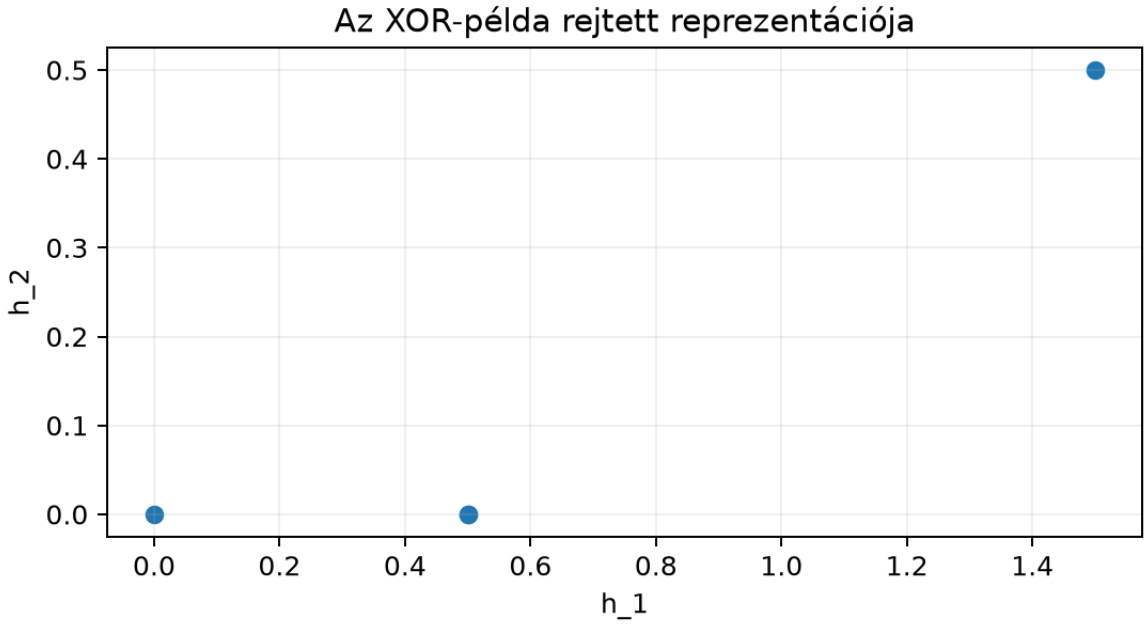
expected = np.array([[0.0], [1.0], [1.0], [0.0]], dtype=np.float64)
print("allclose expected:", np.allclose(Z, expected, atol=1e-12))
assert H.shape == (4, 2) and Z.shape == (4, 1)
assert np.allclose(Z, expected, atol=1e-12)
assert np.array_equal(pred.ravel(), np.array([0, 1, 1, 0]))
```

Futtatás: `python xor_mlp_forward.py` a `n034_labor/` könyvtárban. A referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál; a konstrukció determinisztikus, ezért seed nincs) mért kimenet:

```
H shape: (4, 2)
Z shape: (4, 1)
H:
 [[0.  0. ]
```

```
  [0.5 0. ]
  [0.5 0. ]
  [1.5 0.5]]
Z:
 [[0.]
  [1.]
  [1.]
  [0.]]
pred:
 [0 1 1 0]
allclose expected: True
```

Az ellenőrzés a négy logit pontos egyezését kéri `atol=1e-12` tűréssel; a műveletek félegész értékeket (0,5 és 1,5) tartalmaznak, amelyek float64-ben pontosan reprezentálhatók, ezért a tűrés bőséges. A rejtett pre-aktivációt a kódban S_1-nek nevezzük, hogy ne ütközzön a kimeneti logit Z jelölésével; a j-edik koordináta jele ott s_j. Az eredmény értelmezése: a rejtett reprezentációban a két osztály már lineárisan szeparálható, és a kimeneti réteg ezt a szeparációt valósítja meg. Az alábbi ábra a chart-blokkból, a forward kimenetéből generálódik; a labor külön PNG-fájlt nem ír.



9.4. ábra: Az XOR-példa rejtett reprezentációja

A futtatott forward kimenetéből; a 0 osztály a (0,0) és (1.5,0.5) pont, az 1 osztály a két (0.5,0) pont.

9.2.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem oldható meg az XOR két affin réteggel aktiváció nélkül. A fenti XOR-súlyokból kiindulva írja fel az összevont affin függvényt, és mutassa meg, hogy nincs olyan τ küszöb, amely mind a négy pontot helyesen osztályozza.

F2 (levezetési/bizonyítási, nehézség 2). Legyen $d = 3$, $m = 2$, $K = 1$. Adja meg a

$$W^{(1)} = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0.5 \\ -1 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & -1 \end{pmatrix}, \quad b^{(2)} = 0.5$$

súlyokat aktiváció nélküli kompozícióhoz. Vezesse le a kollapszustétel alkalmazásával $\widetilde{W}$-t és $\tilde{b}$-t, majd az $x = (1, -1, 2)^\top$ ponton ellenőrizze a közvetlen és az összevont számítást.

F3 (implementációs, nehézség 1). Írjon NumPy-függvényt `mlp_forward(X, W1, b1, W2, b2)` néven, amely ReLU aktivációval számolja a két rétegű MLP logitjait. Az inputok alakja $(N, d), (m, d), (m,), (K, m), (K,)$; a visszatérés alakja (N, K). Tesztelje a labor XOR-példáján, és ellenőrizze a shape-eket $N = 4, d = 2, m = 2, K = 1$ mellett `atol=1e-12` tűréssel.

F4 (hibakeresési, nehézség 2). Egy hallgató a következő előrefelé számítást írta:

```
H = relu(X @ W1.T + b1)
Z = relu(H @ W2.T + b2)
```

A tanítás során a loss (veszteség) nem csökken 0.3 alá egy bináris XOR-feladaton BCEWithLogitsLoss-szal, pedig a cél logitoknak a 0 osztályú pontokon negatívnak kellene lenniük. Hol van a hiba, mi a következménye a logitokra és a gradiensre, és hogyan javítandó?

F5 (kísérleti/transzfer, nehézség 2). Tervezzen kontrollált kísérletet annak eldöntésére, hogy egy rejtett rétegű MLP szükséges-e az XOR megtanulásához. Adja meg: az adatot (a négy pont, illetve annak zajosított változatai), a train/val felosztást, a baseline-t (perceptron), a változó tényezőt (rejtett réteg jelenléte), a metrikát, legalább 5 seeddel (véletlenmaggal) való ismétlést, valamint az átlag és a szórás összegzését, továbbá a következtetés érvényességi korlátját.

9.2.8 Megoldások és indoklás

F1 megoldása. Az aktiváció nélküli két rétegű modell a kollapszustétel miatt egyetlen affin függvény. A megadott súlyokkal a fenti levezetés alapján $z = -4x_1 - 4x_2 + 8$. A négy ponton a logitok:

$$z(0,0) = 8, \quad z(1,0) = 4, \quad z(0,1) = 4, \quad z(1,1) = 0.$$

A helyes osztályozáshoz valamilyen τ-ra $z(1,0) > \tau$ és $z(0,1) > \tau$, tehát $\tau < 4$; ugyanakkor $z(0,0) \le \tau$, tehát $\tau \ge 8$. Egyszerre $\tau < 4$ és $\tau \ge 8$ lehetetlen. A $(1,1)$ pont 0 logitja is 0 osztályú, ami konzisztens lenne, de a $(0,0)$ pont magas logitja már kizárja a küszöbözést. Tehát nincs megfelelő lineáris osztályozó.

F2 megoldása. A kollapszustétel képleteivel:

$$\widetilde{W} = W^{(2)}W^{(1)} = \begin{pmatrix} 2 & -1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2\cdot 1 + (-1)\cdot 0 & 2\cdot 2 + (-1)\cdot 1 & 2\cdot 0 + (-1)\cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 3 & -1 \end{pmatrix}.$$

$$\tilde{b} = W^{(2)}b^{(1)} + b^{(2)} = \begin{pmatrix} 2 & -1 \end{pmatrix} \begin{pmatrix} 0.5 \\ -1 \end{pmatrix} + 0.5 = (1 + 1) + 0.5 = 2.5.$$

Közvetlen számítás: $W^{(1)}x + b^{(1)} = (1 - 2 + 0.5,\ -1 + 2 - 1) = (-0.5, 0)$; majd $W^{(2)}$-vel és b2-vel: $2(-0.5) + (-1)\cdot 0 + 0.5 = -0.5$. Összevontan: $\widetilde{W}x + \tilde{b} = 2 - 3 - 2 + 2.5 = -0.5$. A két út megegyezik; az alakellenőrzés: $\widetilde{W} \in \mathbb{R}^{1\times 3}$, $\tilde{b} \in \mathbb{R}^1$.

F3 megoldása.

```
import numpy as np

def mlp_forward(X, W1, b1, W2, b2):
    # X: (N,d), W1: (m,d), b1: (m,), W2: (K,m), b2: (K,)
    Z = X @ W1.T + b1          # (N,m)
    H = np.maximum(0.0, Z)     # (N,m)
    return H @ W2.T + b2       # (N,K)

W1 = np.array([[1.0, 1.0], [1.0, 1.0]], dtype=np.float64)
b1 = np.array([-0.5, -1.5], dtype=np.float64)
W2 = np.array([[2.0, -6.0]], dtype=np.float64)
b2 = np.array([0.0], dtype=np.float64)
X = np.array([[0.0, 0.0], [1.0, 0.0], [0.0, 1.0], [1.0, 1.0]],
             dtype=np.float64)

Z = mlp_forward(X, W1, b1, W2, b2)
expected = np.array([[0.0], [1.0], [1.0], [0.0]], dtype=np.float64)
assert Z.shape == (4, 1)
assert np.allclose(Z, expected, atol=1e-12)
print("OK", Z.shape, Z.ravel())
```

Az ellenőrzés a shape-re és a négy logitra terjed ki. A `np.maximum(0.0, Z)` koordinátánkénti ReLU, tehát a batch-művelet is pontosan a definíciót követi.

F4 megoldása. A hiba a kimeneti rétegen lévő felesleges ReLU. A BCEWithLogitsLoss bemenete logit, amelynek negatívnak is kell tudnia lenni, hiszen a szigmoidon átmenve ebből lesz 0.5-nél kisebb valószínűség. A ReLU a negatív logitokat nullára vágja, így a 0 osztályú pontokra a modell legjobb esetben is 0.5 valószínűséget tud adni, és a kimeneti ReLU a negatív oldalon pontosan nulla deriváltja miatt a hibás logitokon a gradiens is eltűnik (ez nem telítődés, hanem a ReLU nulla meredeksége). A javítás: `Z = H @ W2.T + b2`, aktiváció nélkül. A visszaellenőrzés a labor XOR-példáján: a negatív logitot igénylő konstrukció, például $b^{(2)} = -0.25$ mellett a $(0, 0)$ ponton a helyes logit -0.25, a hozzá tartozó BCE-veszteség $-\ln(1 - \sigma(-0.25)) \approx 0.576$; a hibás kódban a logit 0, a veszteség $-\ln 0.5 \approx 0.693$, és a kimeneti gradiens is 0.5, illetve a helyes esetben $\sigma(-0.25) - 0 \approx 0.438$. (A kód a labor `relu` függvényét használja; a `BCEWithLogitsLoss` a PyTorch megfelelője, a logitokra vonatkozó állítás keretrendszerfüggetlen.)

F5 megoldásminta. Az adat a négy XOR-pont, valamint ezekből képzett zajos változatok: minden ponthoz adjunk kis független Gauss-zajt, a címke az eredeti pont címkéje marad; ez nehezíti a feladatot, de a cél a kontrollált összehasonlítás. A split: a négy alappont mindig jelenjen meg trainben; a zajos pontokból 60% train, 40% val partíció. A baseline a 9.1. Neuron és perceptron perceptron, azaz nulla rejtett réteg; a változó tényező a rejtett réteg megléte, a többi tényező (tanulási ráta, epochszám, inicializálás) rögzített. Metrika: a val halmaz pontossága (accuracy), öt különböző seed mellett, átlag és szórás formájában; a két modellt ugyanazon seedek szerint párosítva hasonlítjuk össze, és a döntési szabály előre rögzítendő: az MLP akkor tekinthető jobbnak, ha a seedenkénti párosított különbség mind az öt esetben ugyanabba az irányba mutat. Öt seed mellett ez csak irányadó, nem erős statisztikai bizonyíték. A várt jelenség: a legjobb lineáris osztályozó a tiszta XOR-on 75%-ot ér el; a perceptron nem konvergál, ezért végállapota seedfüggő, és a val pontossága 50% és 75% között alakulhat. Az MLP a tiszta pontokon (amelyek a trainben vannak) 100%-ot érhet el, de ez tanítási, nem validációs metrika. A következtetés korlátja: ez a kísérlet a 2-dimenziós XOR-ra mutatja a rejtett réteg hasznát, de általános architektúra-állítást nem alapoz meg; ahhoz a hipotézisosztályok összehasonlítására és nagyobb adatra van szükség a 8.2. Véges hipotézisosztály egyenletes korlátja keretében.

9.2.9 Tipikus tévedés, önellenőrzés és visszatekintés

A leggyakoribb tévedés: „több lineáris réteg mégiscsak gazdagabb modellt ad, mert több paramétere van.” Ez hamis. A kollapszustétel bizonyítja, hogy a kompozíció függvényosztálya nem bővül: bármely két affin réteg egyetlen $\widetilde{W}, \tilde{b}$ párba írható át. Az XOR-példán a 9 paraméteres lineáris hálózat az adott súlyokkal a $\widetilde{W} = (-4, -4)$, $\tilde{b} = 8$ affin egyenest valósítja meg, és a fentiek szerint nem tudja a négy pontot elválasztani. Fontos, hogy ez a konkrét súlykonfiguráció egyetlen példány: a lineáris hálózatok osztálya bármely affin $\mathbb{R}^2 \to \mathbb{R}$ leképezést meg tud valósítani, csak éppen az XOR-t nem, mert az nem lineárisan szeparálható. A paraméterszám tehát önmagában nem reprezentációs erő; a nemlinearitás hiánya miatt a plusz paraméterek egy része redundáns.

Rövid önellenőrzés. (1) Melyik azonosság köti össze a sigmoidot és a tanh-t? (2) Miért elegendő a ReLU-t koordinátánként definiálni? (3) Ha $\varphi(u) = cu$ valamely $c \neq 0$ konstansra, kollapszál-e a két réteg? (Válasz: igen, mert φ lineáris, és a kollapszustétel bizonyításában a c szorzó beolvad $W^{(2)}$-be; a nemlinearitásnak ténylegesen nemlineárisnak kell lennie.) (4) Számítsa ki a fenti XOR-súlyokkal a $(0, 0)$ és az $(1, 1)$ pont logitját, és ellenőrizze, hogy a $z > 0{,}5$ küszöb mellett mindkettő a 0 osztályba kerül.

Korábbi fogalom áttekintése: a 9.1. Neuron és perceptron alfejezetben a konvex burokkal bizonyítottuk az XOR lineáris szeparálhatatlanságát. Az itteni kollapszustétel ennek az állításnak a mélységi kiterjesztése: akármilyen mély, de aktiváció nélküli hálózat sem tud többet egy perceptronnál. A 8.2. Véges hipotézisosztály egyenletes korlátja alfejezetben tárgyalt kapacitásfogalom szempontjából ez arra figyelmeztet, hogy a hipotézisosztály tényleges méretét a nemlinearitás, nem a puszta rétegszám határozza meg.

9.2.10 Rövid összefoglaló

A két rétegű MLP rejtett affin transzformációt, koordinátánkénti nemlineáris aktivációt és kimeneti affin réteget komponál. ReLU-val a hálózat folytonos és szakaszonként affin: legfeljebb 2^m poliéder-darabon egy-egy affin függvény. Aktiváció nélkül bármennyi affin réteg egyetlen $\widetilde{W} = W^{(2)}W^{(1)}$, $\tilde{b} = W^{(2)}b^{(1)} + b^{(2)}$ paraméterű affin leképezéssé vonható össze, tehát a rétegmélység önmagában nem növeli a kifejezőerőt. A kézi XOR-konstrukció 9 paraméterrel és 2 rejtett neuronnal elválasztja a négy pontot, amelyeket perceptron nem tud; a NumPy-forward (előrefelé számítás) ezt ellenőrzi.

A következő 9.3. Expresszivitás és approximációs állítások egységben ezt a kifejezőerőt formalizáljuk approximációs tételekkel; az ott használt forrásokat a 9.3 egység forrásjegyzéke adja meg.

9.2.11 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y. és Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

- Az XOR-konstrukció, a kollapszustétel bizonyítása és a szakaszonként affin (piecewise affine) állítás a jelen egység saját levezetése.

9.3 Expresszivitás és approximációs állítások

9.3.1 Cél, előismeret és induló kérdés

A 9.2. MLP és nemlinearitás alfejezetben beláttuk, hogy a ReLU MLP folytonos, szakaszonként affin függvény, és hogy nemlinearitás nélkül több affin réteg egyetlen affin réteggé vonható össze. Most a kifejezőerőt formalizáljuk: mit tud egyetlen rejtett rétegű háló *egyenletesen* közelíteni? Az induló kérdés: legyen $f(x) = \sin(2\pi x)$ a $[0, 1]$ intervallumon. Körülbelül hány ReLU neuron szükséges ahhoz, hogy a háló és f között a maximális eltérés 0,05 alatt maradjon? A fejezet először pontos definíciót ad, majd egy olyan approximációs tételt bizonyítunk, amelyre a fenti kérdésre konkrét korlátot kapunk.

9.3.2 Univerzális approximáció, kompakt tartomány és szélesség

Legyen $K \subset \mathbb{R}^d$ **kompakt tartomány (compact domain)**, azaz korlátos és zárt halmaz. Az 1D esetben $K = [a, b]$ zárt intervallum. A $C(K)$ tér a K-n folytonos valós függvények halmazát jelöli. A két függvény eltérését a **szupremumnorma (supremum norm)** méri:

$$\|f - g\|_\infty = \sup_{x \in K} |f(x) - g(x)|.$$

Az 1D egy rejtett rétegű MLP alakja a jelen egységben

$$g(x) = W^{(2)} \,\mathrm{ReLU}(W^{(1)} x + b^{(1)}) + b^{(2)},$$

ahol a bemenet $x \in \mathbb{R}^1$, a rejtett szélesség **(width)** m, tehát $W^{(1)} \in \mathbb{R}^{m \times 1}$, $b^{(1)} \in \mathbb{R}^m$, $W^{(2)} \in \mathbb{R}^{1 \times m}$, $b^{(2)} \in \mathbb{R}^1$. Kötegelt jelölésben (batch) $X \in \mathbb{R}^{N \times 1}$, és a bias a megfelelő dimenzió mentén sugárzódik.

Definíció (univerzális approximációs tulajdonság). A $\mathcal{H}$ függvényosztály rendelkezik az univerzális approximációs tulajdonsággal a K tartományon, ha minden $f \in C(K)$ és minden $\varepsilon > 0$ esetén létezik $g \in \mathcal{H}$, amelyre $\|f - g\|_\infty \le \varepsilon$. Az egységes jelöléshez legyen $\mathcal{H} = \bigcup_{m \ge 1} \mathcal{H}_m$, ahol $\mathcal{H}_m$ az m rejtett szélességű, egy rejtett rétegű ReLU-hálózatok osztálya; a tétel az unióra vonatkozó állítás, és a szükséges szélesség függhet ε-tól.

Az **aktivációs feltétel (activation condition)** szerepe a ReLU tulajdonságában látszik: a $\mathrm{ReLU}(x - t)$ függvény a t pontban visz be egyetlen meredekségugrást, a konstans vagy az affin aktiváció viszont nem hoz létre ilyen töréspontot. Ezért a ReLU alkalmas darabonként affin függvények összerakására.

9.3.3 A reprezentációs tétel 1D ReLU-hálózatokra

Először egy teljesen bizonyított, saját lemmát mondunk ki. A szakaszonként affin függvény meredekségét a belső töréspontok kivételével mindenütt az intervallumon belüli derivált adja; a töréspontokban a jobb és bal oldali meredekség különbözik.

Lemma 1 (szakaszonként affin függvény ReLU-előállítása). Legyen $f : [a, b] \to \mathbb{R}$ folytonos és szakaszonként affin, belső töréspontokkal $a < t_1 < \cdots < t_k < b$. Legyen $s_0 = \lim_{h \to 0+} \frac{f(a+h) - f(a)}{h}$ a kezdő meredekség, és legyen

$$\delta_j = \lim_{h\to 0+} \frac{f(t_j+h)-f(t_j)}{h} - \lim_{h\to 0-} \frac{f(t_j+h)-f(t_j)}{h}$$

a meredekség ugrása a t_j pontban. A δ_j itt meredekségugrás; ez nem azonos a Lemma 2–3 egyenletes folytonossági küszöbével, amelyet szintén δ jelöl. Ekkor

$$f(x) = f(a) + s_0(x-a) + \sum_{j=1}^{k} \delta_j \,\mathrm{ReLU}(x-t_j), \qquad x \in [a,b].$$

Bizonyítás. Legyen $q(x)$ a jobb oldal. A ReLU folytonos, ezért q folytonos, továbbá minden t_j-től különböző pontban mindkét oldal affin, azaz q is szakaszonként affin. A t_j pontoktól balra $\mathrm{ReLU}(x-t_j)=0$, jobbra pedig a meredeksége 1, tehát q meredeksége a (t_{j-1}, t_j) intervallumon

$$s_0 + \sum_{i<j} \delta_i,$$

ami éppen f meredeksége ugyanazon az intervallumon, hiszen a δ_i ugrások kumulálódnak. Az a pontban $q(a)=f(a)$, és szakaszonként affin folytonos függvényt a kezdőérték és a meredekségfüggvény egyértelműen meghatároz, így $q=f$.

A következő lépés minden folytonos függvényt szakaszonként affin interpolánssal közelít.

Lemma 2 (lineáris interpoláns approximációja). Legyen $f \in C([a,b])$. Minden $\varepsilon>0$-hoz létezik olyan folytonos, szakaszonként affin p függvény, hogy $\|f-p\|_\infty \le \varepsilon$.

Bizonyítás. A kompakt $[a,b]$ intervallumon f **egyenletesen folytonos**, tehát létezik $\delta>0$, hogy $|x-y|<\delta$ esetén $|f(x)-f(y)|<\varepsilon$. Itt δ egy ε-tól függő küszöbszám; az f folytonossági modulusa ezzel szemben az $\omega_f(t)=\sup_{|x-y|\le t}|f(x)-f(y)|$ függvény. Válasszunk egy $a=x_0<\dots<x_m=b$ rácsot, ahol a lépésköz kisebb, mint δ. Legyen p az f lineáris interpolánsa a rácspontokban: minden $[x_i,x_{i+1}]$ intervallumon

$$p(x) = \frac{x_{i+1}-x}{x_{i+1}-x_i} f(x_i) + \frac{x-x_i}{x_{i+1}-x_i} f(x_{i+1}).$$

Itt $p(x)$ konvex kombinációja két f-értéknek, és $|x-x_i|, |x-x_{i+1}| < \delta$, ezért

$$|p(x)-f(x)| \le \max\{|f(x_i)-f(x)|, |f(x_{i+1})-f(x)|\} < \varepsilon,$$

tehát $\|f-p\|_\infty \le \varepsilon$.

A Lemma 2 csak **létezést** állít: a lépésközt az egyenletes folytonosság δ-ja adja, amelyről a bizonyítás nem mond számot. Kétszer folytonosan differenciálható célfüggvényre explicit, kvantitatív korlát is levezethető.

Lemma 3 (a lineáris interpoláns hibakorlátja C^2 függvényre). Legyen $f \in C^2([a,b])$, $M_2 = \max_{[a,b]} |f''|$, és legyen p az f lineáris interpolánsa egy h lépésközű egyenletes rácson. Ekkor

$$\|f-p\|_\infty \le \frac{h^2}{8} M_2.$$

Bizonyítás. Rögzítsünk egy $[x_i, x_{i+1}]$ intervallumot és benne egy x pontot, ahol $x \neq x_i, x_{i+1}$ (a végpontokban a hiba nulla). Legyen $e = f - p$, és tekintsük a $\phi(t) = e(t) - \frac{e(x)}{(x-x_i)(x-x_{i+1})}(t - x_i)(t - x_{i+1})$ segédfüggvényt. Ez C^2, és három pontban nulla: x_i-ben, x_{i+1}-ben és x-ben. A Rolle-tétel kétszeri alkalmazásával van olyan $\xi \in (x_i, x_{i+1})$, ahol $\phi''(\xi) = 0$. Mivel $p'' = 0$ az intervallum belsejében, $\phi''(\xi) = f''(\xi) - \frac{2e(x)}{(x-x_i)(x-x_{i+1})}$, tehát

$$e(x) = \frac{f''(\xi)}{2}(x - x_i)(x - x_{i+1}).$$

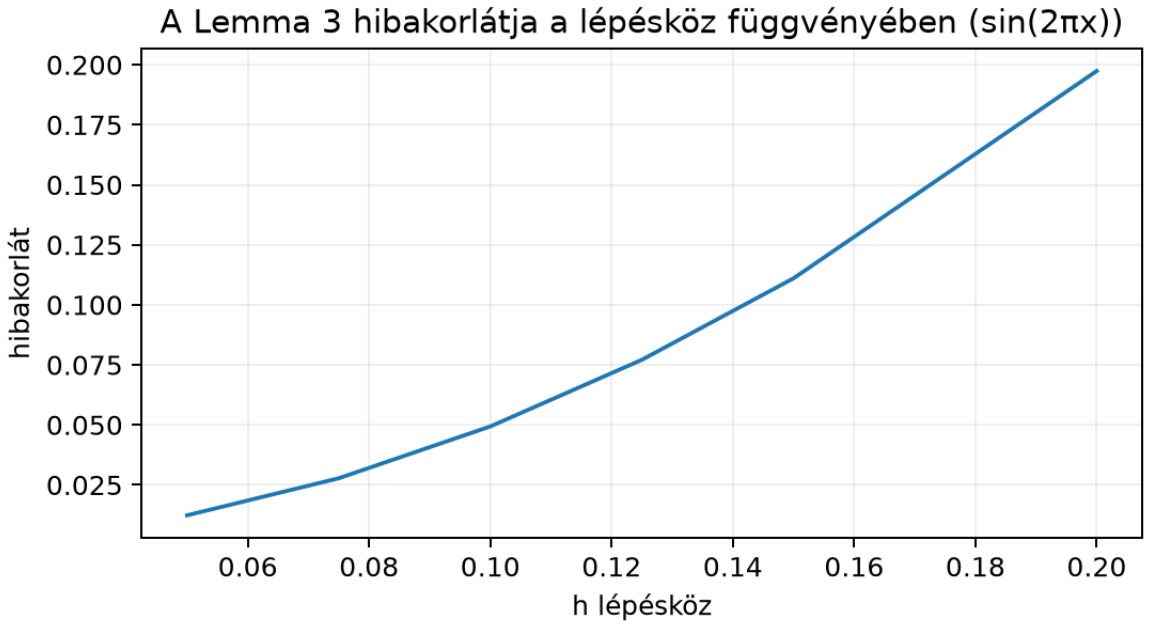
A $|(x - x_i)(x - x_{i+1})|$ szorzat az intervallum közepén maximális, értéke $h^2/4$, ezért $|e(x)| \leq \frac{M_2}{2} \cdot \frac{h^2}{4} = \frac{h^2}{8} M_2$.

Az **elegendő** lépésközhöz a **felső korlátot** kell a célhiba alá szorítani: ha $\frac{h^2}{8} M_2 \leq \varepsilon$, akkor $\|f - p\|_\infty \leq \varepsilon$, vagyis elegendő

$$h \leq \sqrt{\frac{8\varepsilon}{M_2}} \qquad (M_2 > 0).$$

Ha $M_2 = 0$ (f affin), a feltétel értelmetlen: az affin függvény interpolációja pontos, tetszőleges h elegendő. A Lemma 2 δ küszöbe és a Lemma 3 h lépésköze különböző mennyiség: előbbi az egyenletes folytonosságból, utóbbi a második derivált korlátjából adódik. Egyik sem az f folytonossági modulusa; a modulus az $\omega_f(t) = \sup_{|x-y| \leq t} |f(x) - f(y)|$ függvény, nem egyetlen szám.

A Lemma 3 a tételhez is köthető: C^2 célfüggvényre a Lemma 3-ból kapott h választással a szélesség $m = \lceil (b-a)/h \rceil + 2$, ahol $h = \sqrt{8\varepsilon/M_2}$. Például az induló kérdésre $f(x) = \sin(2\pi x)$-re $M_2 = 4\pi^2$, ezért $\varepsilon = 0{,}05$ mellett $h \leq \sqrt{8 \cdot 0{,}05/(4\pi^2)} \approx 0{,}1007$; a $[0,1]$-en $N = \lceil 1/h \rceil = 10$ egyenlő intervallum elegendő, ami 9 belső töréspontot és 2 identitás-egységet, összesen 11 rejtett egységet jelent. Ez a Lemma 3-ból származó elegendő szélesség; a tétel ehhez képest csak létezést állít, és a szélesség minden ε-hoz külön választható.



9.5. ábra: A Lemma 3 hibakorlátja a lépésköz függvényében (sin(2 x))

*A h^2/8M_2 képletből számított értékek M_2=4 [2] mellett; nem mérés.**

A lemma valós aritmetikára szól; `float32` megvalósításnál a súlyok és a kiértékelés kerekítése ehhez egy kis, külön kezelendő többletet ad (nagyságrendileg néhány ε_{32} relatív hiba a súlyok és az aktivációk méretéhez mérve, ahol $\varepsilon_{32} \approx 1{,}2 \cdot 10^{-7}$ a float32 gépi epszilonja), ezért a mért hibát a korláttal tűréssel vetjük össze.

A két lemma kompozíciója adja a saját approximációs tételt.

Tétel (1D univerzális approximáció ReLU-hálózattal). Legyen $[a, b] \subset \mathbb{R}$ kompakt intervallum. Ekkor minden $f \in C([a, b])$ és minden $\varepsilon > 0$ esetén létezik egyetlen rejtett rétegű ReLU MLP, amelyre $\|f - g\|_\infty \le \varepsilon$. A Lemma 2 konstrukciójában a rejtett szélesség legfeljebb $m = \lceil (b-a)/\delta \rceil + 2$, ahol $\delta > 0$ az egyenletes folytonosság küszöbe: $|x - y| < \delta$ esetén $|f(x) - f(y)| < \varepsilon$. A korlát levezetése: $N = \lceil (b-a)/\delta \rceil + 1$ egyenlő részre osztva $h = (b-a)/N < \delta$ szigorúan, így $N - 1$ belső töréspont és 2 identitás-egység kell, összesen $N + 1 = \lceil (b-a)/\delta \rceil + 2$ egység.

Bizonyítás. A Lemma 2 ad egy szakaszonként affin p függvényt, amelyre $\|f - p\|_\infty \le \varepsilon$. A Lemma 1 szerint p pontosan előállítható a fenti egy rejtett rétegű alakban: a lineáris taghoz két ReLU egység ($x = \mathrm{ReLU}(x) - \mathrm{ReLU}(-x)$), a belső töréspontokhoz további k egység tartozik, a kimeneti réteg pedig a s_0 és δ_j együtthatókat kombinálja. A szélességkorlát a rács lépésközéből következik.

Átvett szakirodalmi eredmény. Az általános, többdimenziós univerzális approximációs eredmények közül a klasszikus Cybenko-tételt [P24] egyetlen rejtett rétegű hálózatra vonatkozóan tartjuk nyilván. A Cybenko-tétel szigmoid típusú aktivációra szól; a ReLU-ra vonatkozó többdimenziós megfelelője külön irodalom, ezért a ReLU-hálóra itt csak a fenti 1D, saját tételt használjuk. A Cybenko-tétel részletes feltételrendszerét (aktiváció-osztály, topológia, dimenzió) és bizonyítását nem tárgyaljuk, ezért annak pontos, általános formuláját sem használjuk fel a későbbi számításokhoz. A fenti 1D ReLU-tétel a könyv saját, teljes bizonyítású eredménye; a Cybenko-tétel részletes megfogalmazása nem része az itt bizonyított állításnak. A Lemma 2–3 standard interpolációs tények, amelyeket itt teljes bizonyítással adunk meg, ezért külön hivatkozás nélkül szerepelnek.

9.3.4 Kézi példa

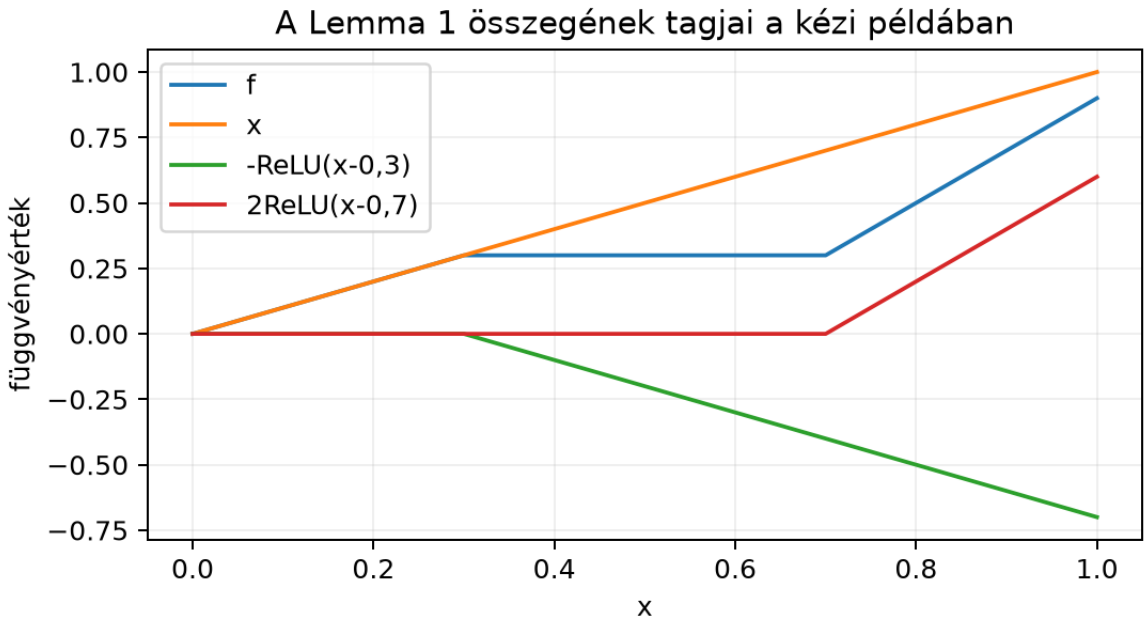
Legyen $[a, b] = [0, 1]$, és definiáljuk

$$f(x) = \begin{cases} x, & 0 \le x < 0{,}3, \\ 0{,}3, & 0{,}3 \le x < 0{,}7, \\ 0{,}3 + 2(x - 0{,}7), & 0{,}7 \le x \le 1. \end{cases}$$

A függvény folytonos, belső töréspontjai $t_1 = 0{,}3$ és $t_2 = 0{,}7$. A kezdő meredekség $s_0 = 1$; a meredekség ugrása 0,3-nál $0 - 1 = -1$, 0,7-nél $2 - 0 = 2$. A Lemma 1 képlete:

$$f(x) = 0 + 1 \cdot x + (-1)\mathrm{ReLU}(x - 0{,}3) + 2\,\mathrm{ReLU}(x - 0{,}7).$$

Ellenőrzés: $x = 0{,}2$ esetén 0,2; $x = 0{,}5$ esetén $0{,}5 - 0{,}2 = 0{,}3$; $x = 0{,}9$ esetén $0{,}9 - 0{,}6 + 0{,}4 = 0{,}7$, és a definícióból $0{,}3 + 2(0{,}9 - 0{,}7) = 0{,}7$.



9.6. ábra: A Lemma 1 összegének tagjai a kézi példában

A Lemma 1 képletéből számított értékek (f = x - ReLU(x-0,3) + 2 ReLU(x-0,7)); nem mérés.

A hálózat reprezentációja négy rejtett egységgel: a lineáris taghoz $\mathrm{ReLU}(x)$ és $\mathrm{ReLU}(-x)$, a töréspontokhoz $\mathrm{ReLU}(x-0{,}3)$ és $\mathrm{ReLU}(x-0{,}7)$. A súlyok:

$$W^{(1)} = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \end{bmatrix}, \quad b^{(1)} = [0,\ 0,\ -0{,}3,\ -0{,}7],$$

$$W^{(2)} = [1,\ -1,\ -1,\ 2], \quad b^{(2)} = [0].$$

9.3.5 Labor: konstrukció, mért hiba és paraméterszám-kontraszt

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor célja az 1D tétel közvetlen használata, és annak bemutatása, hogy azonos paraméterszámú hálók között a konstruált és a véletlen inicializálású háló hibája nagyságrendekkel különbözhet, tehát a paraméterszám önmagában nem határozza meg az approximáció minőségét; az összehasonlítás egyetlen véletlen inicializálásra támaszkodik. A célfüggvény $f(x) = \sin(2\pi x)$ a $[0, 1]$-en. A munkakönyvtár `lab/`, a kód `n035_expressivity.py`.

```
import numpy as np

def relu(x):
    return np.maximum(x, 0.0)

def mlp_1hidden(x, W1, b1, W2, b2):
    h = relu(x @ W1.T + b1)          # (N,m)
    return h @ W2.T + b2             # (N,1)
```

```python
def build_interpolating_mlp(f, a=0.0, b=1.0, n_intervals=10, width=12):
    x_nodes = np.linspace(a, b, n_intervals + 1, dtype=np.float32)
    interiors = x_nodes[1:-1]
    y_nodes = f(x_nodes).astype(np.float32)
    h = (b - a) / n_intervals
    slopes = np.diff(y_nodes) / h      # n_intervals meredekség
    s0 = slopes[0]
    deltas = np.diff(slopes)           # ugrás a belső pontokban
    m = width
    W1 = np.zeros((m, 1), dtype=np.float32)
    b1 = np.zeros((m,), dtype=np.float32)
    W1[0, 0] = 1.0                      # ReLU(x)
    W1[1, 0] = -1.0                     # ReLU(-x)
    W1[2:2 + len(interiors), 0] = 1.0
    b1[2:2 + len(interiors)] = -interiors
    W2 = np.zeros((1, m), dtype=np.float32)
    W2[0, 0] = s0
    W2[0, 1] = -s0
    W2[0, 2:2 + len(deltas)] = deltas
    b2 = np.array([f(a) - s0 * a], dtype=np.float32)
    return W1, b1, W2, b2

def mlp_2hidden(x, W1, b1, W2, b2, W3, b3):
    h1 = relu(x @ W1.T + b1)
    h2 = relu(h1 @ W2.T + b2)
    return h2 @ W3.T + b3

def build_random_two_hidden(seed=20261020):
    rng = np.random.default_rng(seed)
    W1 = rng.standard_normal((3, 1), dtype=np.float32) * 0.5
    b1 = rng.standard_normal((3,), dtype=np.float32) * 0.5
    W2 = rng.standard_normal((6, 3), dtype=np.float32) * 0.5
    b2 = rng.standard_normal((6,), dtype=np.float32) * 0.5
    W3 = rng.standard_normal((1, 6), dtype=np.float32) * 0.5
    b3 = rng.standard_normal((1,), dtype=np.float32) * 0.5
    return W1, b1, W2, b2, W3, b3

def main():
    target = lambda x: np.sin(2 * np.pi * x)
    a, b = 0.0, 1.0
    n_intervals = 10
    W1, b1, W2, b2 = build_interpolating_mlp(target, a, b, n_intervals, width=12)
    # A háló paraméterszáma: 12 + 12 + 12 + 1 = 37
    x_test = np.linspace(a, b, 1001, dtype=np.float32).reshape(-1, 1)
    y_true = target(x_test).astype(np.float32)
    y_A = mlp_1hidden(x_test, W1, b1, W2, b2)
    err_A = np.max(np.abs(y_A - y_true))
    # Elméleti garantált szupremumhiba: max|f''| = 4*pi^2, h = 0.1
    h = (b - a) / n_intervals
    bound = (h**2 / 8.0) * (4 * np.pi**2)
    # B kontraszt: 3+6 rejtett, paraméterszám 3+3+18+6+6+1 = 37
    W1b, b1b, W2b, b2b, W3b, b3b = build_random_two_hidden(seed=20261020)
    y_B = mlp_2hidden(x_test, W1b, b1b, W2b, b2b, W3b, b3b)
    err_B = np.max(np.abs(y_B - y_true))
    print(f"A konstrukciós hiba (1001 pontos rácson): {err_A:.6f}")
    print(f"Elméleti garantált szupremumhiba: {bound:.6f}")
    print(f"B véletlen hiba (1001 pontos rácson): {err_B:.6f}")
    assert err_A <= bound
    assert np.isfinite(err_A) and np.isfinite(err_B)
    assert W1.shape == (12, 1) and W2.shape == (1, 12)

if __name__ == "__main__":
```

```
main()
```

Futtatás: `python lab/n035_expressivity.py`. A kód nem igényel PyTorch-ot, csak NumPy-t; dtype float32; NumPy, CPU (nincs eszközválasztás). A referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál; a véletlen háló seedje 20261020) mért kimenet: az A konstrukciós hiba az 1001 pontos rácson 0,048944, az elméleti korlát 0,049348, a B véletlen háló hibája 1,580991. Az A hibája tehát a korlát alatt, de ahhoz közel marad, mert a $\sin(2\pi x)$ második deriváltja a szélsőhelyeken eléri M_2-t; a B hiba nagy, mert a véletlen súlyok nem a célt reprezentálják. A 12-es szélességből 11 egységet használunk (2 identitás + 9 töréspont); a tizenkettedik egység súlya nulla, ezért a 37 paraméteres összevetésben az A háló egy használatlan egységet tartalmaz. A reprezentációhoz 11 egység (34 paraméter) is elegendő lett volna; a 12-t azért tartottuk meg, hogy a B hálóval azonos paraméterszámot kapjunk.

Az állítástípusok elkülönítése a laborban:

- **Levezetett állítás:** az A háló szupremumhibája valós aritmetikában legfeljebb $\frac{0{,}1^2 \cdot 4\pi^2}{8} \approx 0{,}0493$ a teljes $[0,1]$-en, a Lemma 3 hibakorlátjából ($h = 0{,}1$, $M_2 = 4\pi^2$); a `float32` kerekítés ehhez csak elhanyagolható többletet ad.
- **Empirikus állítás:** a 1001 pontos rácson mért `err_A` és `err_B` a tényleges futásból származik; ez a rács a teljes intervallumnak csak diszkrét részhalmaza, ezért a szupremumhibára alsó közelítés.
- **Heurisztikus állítás:** sem a B rossz eredménye, sem egy jövőbeli SGD-kísérlet nem mond ellent a reprezentációs tételnek, mert a tétel létezést, nem megtalálhatóságot állít.

9.3.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget a reprezentációs (létezési) és az optimalizálási (megtalálási) állítás között. Adjon egy konkrét példát, amelyben a létezés igaz, de a megtalálás nem garantált. Elvárt eredmény: saját megfogalmazás, a példában az architektúra, a létező súlyok és a megtalálást gátló körülmény megnevezése.

F2 (levezetési, nehézség 2). Legyen $[a,b] = [0,1]$ és

$$g(x) = \begin{cases} 0, & 0 \le x < 0{,}2, \\ 1{,}5(x - 0{,}2), & 0{,}2 \le x < 0{,}6, \\ 0{,}6, & 0{,}6 \le x \le 1. \end{cases}$$

Adja meg a belső töréspontokat, a kezdő meredekséget, a meredekségugrásokat, a Lemma 1 ReLU-képletet és a hálózat súlyait. Ellenőrizze a $x = 0;\ 0{,}2;\ 0{,}5;\ 0{,}6;\ 1$ pontokban.

F3 (implementációs, nehézség 1). Írjon NumPy függvényt, amely tetszőleges `t` töréspont-tömbből, `deltas` ugrástömbből, `s0` kezdő meredekségből és `f_a = f(a)` kezdőértékből felépíti a Lemma 1 szerinti egy rejtett rétegű súlyokat. Ellenőrizze a F2 g függvényén 17 egyenletes ponton; a max abszolút hiba 10^{-5} alatt legyen. Beadandó: futtatható kód, input, output, tolerancia.

F4 (hibakeresési, nehézség 2). Adott a következő hibás érvelés: „A Cybenko-tétel garantálja, hogy a hálózatunk 10 epoch SGD után megtalálja a jó súlyokat.” Lokalizálja a hibát, nevezze meg a következményt, és fogalmazza át az állítást úgy, hogy az az állítástípusoknak megfeleljen. Ellenőrzésként adja meg, mit állít a tétel, és mit nem.

F5 (kísérleti transzfer, nehézség 3). Tervezzen kísérletet annak vizsgálatára, hogy azonos paraméterszám mellett a széles-sekély vagy a keskeny-mély MLP éri el a jobb max abszolút hibát a $f(x) = \sin(5\pi x)$ függvényen. Adja meg a protokollt: cél, hipotézis, baseline, változó tényező, metrika, seed-kezelés, erőforrás és a következtetés érvényességi határa.

9.3.7 Megoldások és indoklás

F1. A reprezentációs tétel egy $\mathcal{H}$ osztály *kapacitásáról* szól: minden ε-hoz létezik legalább egy $g \in \mathcal{H}$, amely ε közelségben van f-hez. Az optimalizálási állítás ezzel szemben azt mondaná, hogy egy adott keresőeljárás egy adott inicializációból kiindulva megtalálja ezt a g-t. Konkrét példa: a labor A hálója (egy rejtett réteg, 12 neuron) a tétel konstrukciójából létezik, de ugyanennek az A architektúrának véletlen súlyokkal való inicializálása nem feltétlenül ad jó hibát; a jó súlyok léteznek, ám a véletlen inicializálásból induló gradienskeresés sikerére a tétel semmit sem mond. (A B modell 3 és 6 neuronos két rejtett réteggel azonos paraméterszámú, de *más architektúra*; a megtalálás kudarcának példájaként ezért nem használjuk.)

F2. A belső töréspontok $t_1 = 0{,}2$, $t_2 = 0{,}6$; a kezdő meredekség $s_0 = 0$; a meredekségugrás $\delta_1 = 1{,}5 - 0 = 1{,}5$, $\delta_2 = 0 - 1{,}5 = -1{,}5$. A Lemma 1 képlet:

$$g(x) = 0 + 0 \cdot x + 1{,}5\,\mathrm{ReLU}(x - 0{,}2) - 1{,}5\,\mathrm{ReLU}(x - 0{,}6).$$

A hálózat súlyai négy rejtett egységgel: $W^{(1)} = [1,\ -1,\ 1,\ 1]^\top$, $b^{(1)} = [0;\ 0;\ -0{,}2;\ -0{,}6]$, $W^{(2)} = [0;\ 0;\ 1{,}5;\ -1{,}5]$, $b^{(2)} = [0]$. A lineáris tag együtthatója nulla, ezért az első két egység nulla kimeneti súllyal szerepel, de formálisan megtartjuk őket. Ellenőrzés: $x = 0$: 0; $x = 0{,}2$: 0; $x = 0{,}5$: $1{,}5 \cdot 0{,}3 = 0{,}45$; $x = 0{,}6$: $1{,}5 \cdot 0{,}4 = 0{,}6$; $x = 1$: $1{,}5 \cdot 0{,}8 - 1{,}5 \cdot 0{,}4 = 1{,}2 - 0{,}6 = 0{,}6$.

F3. A kód közvetlenül a Lemma 1 konstrukciót valósítja meg. A lineáris taghoz két identitás-egységet használunk, így az $x = \mathrm{ReLU}(x) - \mathrm{ReLU}(-x)$ azonosság általánosan érvényesül a teljes $\mathbb{R}$-en, nem csak a nemnegatív félegyenesen.

```
import numpy as np

def relu(x):
    return np.maximum(x, 0.0)

def build_lemma1(t, deltas, s0, f_a, a=0.0, with_identity=True):
    t = np.asarray(t, dtype=np.float32)
    deltas = np.asarray(deltas, dtype=np.float32)
    k = len(t)
    m = k + 2 if with_identity else k
    if not with_identity and s0 != 0.0:
        raise ValueError("identitás-egységek nélkül csak s0=0 állítható elő")
    W1 = np.zeros((m, 1), dtype=np.float32)
    b1 = np.zeros((m,), dtype=np.float32)
    if with_identity:
        W1[0, 0] = 1.0
        W1[1, 0] = -1.0
        offset = 2
    else:
        offset = 0
    W1[offset:offset + k, 0] = 1.0
    b1[offset:offset + k] = -t
    W2 = np.zeros((1, m), dtype=np.float32)
    if with_identity:
        W2[0, 0] = s0
        W2[0, 1] = -s0
    W2[0, offset:offset + k] = deltas
    b2 = np.array([f_a - s0 * a], dtype=np.float32)
    return W1, b1, W2, b2

def forward(x, W1, b1, W2, b2):
    h = relu(x @ W1.T + b1)
    return h @ W2.T + b2
```

```python
# F2 g függvénye, 17 ponton ellenőrizve
t = np.array([0.2, 0.6], dtype=np.float32)
deltas = np.array([1.5, -1.5], dtype=np.float32)
s0, f_a, a = 0.0, 0.0, 0.0
W1, b1, W2, b2 = build_lemma1(t, deltas, s0, f_a, a)
x = np.linspace(0, 1, 17, dtype=np.float32).reshape(-1, 1)
g_true = np.where(x < 0.2, 0.0,
          np.where(x < 0.6, 1.5 * (x - 0.2), 0.6))
y = forward(x, W1, b1, W2, b2)
print("max abs hiba:", float(np.max(np.abs(y - g_true))))
assert W1.shape == (4, 1) and W2.shape == (1, 4)
assert float(np.max(np.abs(y - g_true))) < 1e-5
```

A `np.where` pontonkénti elágazást ad, ami a 17 pontos ellenőrzéshez elegendő; a feltétel a `x < 0.2` alakú tömb, ezért a `np.where` vektorizáltan működik. A futtatás kimenete `max abs hiba: 5.960464477539063e-08`, ami a 10^{-5} tolerancia alatt van. A `with_identity=False` ág csak $s_0 = 0$ mellett érvényes; $s_0 \neq 0$ esetén a függvény `ValueError`-t ad, mert identitás-egységek nélkül a lineáris tag nem áll elő.

F4. A hiba oka: az állítás összemossa a reprezentációs tételt az optimalizálás konvergenciájával. A Cybenko-tétel szigmoid aktivációra mond létezést egy architektúraosztályra; a ReLU-hálóra a fejezet saját 1D tétele adja a megfelelő állítást. Egyik sem mond semmit az SGD 10 epochja utáni konvergenciáról. A következmény téves modellválasztási ígéret: a gyakorló dönthet úgy, hogy 10 epoch elég, pedig a tétel nem ad epoch-számot és nem garantál globális optimumot. A javított állítás: „A tétel garantálja, hogy létezik a hálózatosztályban olyan súly, amely a kompakt tartományon ε pontossággal közelíti a folytonos függvényt; az, hogy egy adott tanítási eljárás ezt megtalálja-e, külön optimalizálási kérdés.” Ellenőrzés: a tételben nincs „epoch”, „SGD”, „inicializáció”; a hipotézisosztályra és a szupremumnormára vonatkozik.

F5. Egy lehetséges, végrehajtható terv. Cél: azonos paraméterszám mellett a széles-sekély és a keskeny-mély MLP max abszolút hibájának összehasonlítása az $f(x) = \sin(5\pi x)$ függvényen a $[0, 1]$-en. Számított kiindulópont: $M_2 = 25\pi^2$, ezért a Lemma 3 szerint $\varepsilon = 0{,}05$ eléréséhez $h \leq \sqrt{8 \cdot 0{,}05/(25\pi^2)} \approx 0{,}0403$, azaz legalább 25 intervallum és 26 rejtett egység kell. A 12 egységes A háló ezen a függvényen tehát nem éri el a 0,05-öt: a 10 intervallumos konstrukció hibakorlátja $h^2 M_2/8 = 0{,}1^2 \cdot 25\pi^2/8 \approx 0{,}308$. Ezt a terv kimondja, és a 0,05-öt nem használja sikerkritériumként; a kérdés az, melyik architektúra ad kisebb hibát. Hipotézis: azonos paraméterszám mellett a széles-sekély háló legalább olyan jó max abszolút hibát ér el, mint a keskeny-mély, mert az 1D approximációs konstrukció egy rejtett rétegen is megvalósítható; ez tesztelhető állítás, nem a tétel következménye. Architektúrák: A egy rejtett réteg, 12 neuron; B két rejtett réteg, az elsőben 3, a másodikban 6 neuron (a „3+6” jelölés ezt rövidíti); mindkettő 37 paraméter, ReLU-aktivációval és lineáris kimenettel. Tanítás: PyTorch autograd (a fejezet NumPy-kódja csak előrefelé számol, backwardöt nem tartalmaz), teljes köteg, MSE-veszteség, Adam optimalizáló $\mathrm{lr} = 10^{-2}$, 2000 lépés, 10 különböző seed. A tanított A és B modellek seedjei az inicializációhoz tartoznak; a Lemma 1–2 szerinti konstruált A súlyok determinisztikusak, ezért azokhoz seed nem tartozik, és a konstruált referencia külön, egyszer mért *létezési* baseline, nem tanítási eredmény. Metrika: max abszolút hiba 1001 pontos rácson, a szupremumhiba becslése 10000 pontos rácson; a seedek közötti átlagot és minimumot közöljük. Érvényességi határ: az eredmény egyetlen függvényre, egyetlen dimenzióra érvényes; nem általánosítható többdimenziós vagy természetes képadatokra, és a mérés nem dönti el, hogy nagyobb paraméterszám mellett melyik architektúra jobb. Döntési szabály: a széles-sekély háló akkor tekinthető legalább olyan jónak, ha a 10 seeden mért max abszolút hiba átlaga nem nagyobb, mint a keskeny-mélyé, és a párosított különbségek legalább 8 seeden ugyanabba az irányba mutatnak; erőforrás: CPU, PyTorch, a rögzített `cnn-labor` környezet.

9.3.8 Tipikus tévedés és ellenpélda

Az approximáció létezése nem garantálja a hatékony reprezentációt: a szükséges szélesség a célfüggvény oszcillációjával nőhet. Ellenpélda: legyen $f_n(x) = \sin(2\pi n x)$ a $[0,1]$-en. A tétel szerint minden rögzített n-re és ε-ra létezik megfelelő háló, de a második derivált maximuma $M_2 = 4\pi^2 n^2$; a Lemma 3 hibakorlátja ε alá szorítva, $(h^2/8)\,4\pi^2 n^2 \le \varepsilon$, elegendő feltételt ad: $h \le \sqrt{2\varepsilon}/(\pi n)$, tehát elegendő $\sim n/\sqrt{\varepsilon}$ töréspont. A felső korlátból csak elegendő szélesség adódik; a szükségességet külön kell bizonyítani. Legyen $\varepsilon < 1$ és p szakaszonként lineáris, $\|f_n - p\|_\infty \le \varepsilon$. A $2n$ pontban, ahol $f_n = \pm 1$ váltakozva ($x_k = (2k+1)/(4n)$, $k = 0, \dots, 2n-1$), p előjele sorban váltakozik. Tekintsünk három egymást követő ilyen pontot, $x_k < x_{k+1} < x_{k+2}$. Mivel $f_n(x_k)$ és $f_n(x_{k+2})$ azonos előjelű, $f_n(x_{k+1})$ pedig ellentétes, az $\varepsilon < 1$ feltétel miatt $p(x_k)$ és $p(x_{k+2})$ azonos, $p(x_{k+1})$ ellentétes előjelű. Ezért p nem monoton $[x_k, x_{k+2}]$-n; egy szakaszonként lineáris függvény pedig csak úgy lehet nem monoton egy intervallumon, ha abban van töréspontja. A páronként diszjunkt $(x_0, x_2), (x_2, x_4), \dots, (x_{2n-4}, x_{2n-2})$ nyílt intervallumok mindegyike tehát tartalmaz legalább egy töréspontot, amiből legalább $n-1$ töréspont következik. Egy változós ReLU-egység legfeljebb egy töréspontot ad, ezért a szélesség legalább $n-1$: rögzített szélességgel az f_n nem közelíthető tetszőlegesen jól, ha n elég nagy. Ez a példa a reprezentációs szélesség szükségességét mutatja, nem az SGD viselkedését és nem a mintakomplexitást; azok külön kérdések. A tétel létezést, nem hatékony reprezentációt és nem megtalálást állít.

9.3.9 Önellenőrzés és visszatekintés

Ellenőrizze, hogy a Lemma 1 képletében a meredekségugrás δ_j a *jobb* meredekség mínusz *bal* meredekség, nem fordítva. A reprezentáció szélessége a belső töréspontok száma plusz kettő: a kezdő meredekséget és a konstans tagot két lineáris (identitás) egység állítja elő. A kompakt feltétel lényeges: nem korlátos tartományon a szupremumnorma végtelen lehet. A 9.2. MLP és nemlinearitás szakaszonként affin jellemzése itt megfordul az egyváltozós esetben: minden folytonos, szakaszonként affin $[a,b] \to \mathbb{R}$ függvény előáll egy rejtett rétegű ReLU-hálóval (Lemma 1); többdimenzióban ez az állítás nem ilyen egyszerű, ezért itt csak az 1D esetet állítjuk. A 8.3. Kapacitás, regularizálás és modellválasztás kapacitásfogalma szerint az approximációs tétel a hipotézisosztály kifejezőképességét növeli, de a validáció és a tanítási variancia külön kérdés marad. Az 5.3. Mintavételi bizonytalanság és intervallum statisztikai bizonytalansága a labor egyetlen seedjéből származó mérésére is vonatkozik: egyetlen futás nem ad szórásbecslést, ezért az itt közölt hibaértékek pontbecslések.

9.3.10 Rövid összefoglaló

Az univerzális approximációs tulajdonság azt jelenti, hogy egy hipotézisosztály tetszőleges folytonos függvényt tetszőleges $\varepsilon > 0$ pontossággal közelít kompakt tartományon, szupremumnormában. A fejezet teljes bizonyítással igazolja az 1D esetet: a lineáris interpoláns egyenletesen közelít, majd a szakaszonként affin függvény ReLU-töréspontok és meredekségugrások összegeként áll elő. A rejtett szélesség a töréspontok számától függ. A klasszikus Cybenko-tétel ([P24]) szigmoid aktivációra mond létezést; pontos feltételeit itt nem idézzük, mert a fejezet az 1D ReLU-esetet bizonyítja teljesen. A labor egyetlen véletlen inicializálású hálón azt mutatta, hogy azonos paraméterszám mellett a konkrét súlykonfiguráció nagyban meghatározza a hibát; ez egyetlen futás, nem általánosítás, és a tétel nem ígér tanítási konvergenciát.

9.3.11 Továbblépés és források

A 9.4. Aktivációk és gradiensáramlás egység az aktivációk gradiensáramlásra gyakorolt hatását vizsgálja: a ReLU töréspontjának konvencionális deriváltját, a telítődő szigmoid kis gradiensét és a mély hálók taníthatóságát.

- [P24] George Cybenko (1989). *Approximation by superpositions of a sigmoidal function.* Mathematics of Control, Signals and Systems 2:303–314. https://doi.org/10.1007/BF02551274.

9.4 Aktivációk és gradiensáramlás

9.4.1 Cél, előismeret és induló becslés

A 9.3. Expresszivitás és approximációs állítások egységben láttuk, hogy egy elég széles ReLU-háló képes tetszőleges folytonos függvényt közelíteni kompakt tartományon. Ez **reprezentációs** állítás volt: megmondja, hogy létezik jó súlykonfiguráció, de semmit sem ígér arról, hogy a gradiensalapú tanítás megtalálja. Ebben az egységben a másik irányt vizsgáljuk: mit mond egyetlen lokális derivált, és hogyan szorzódnak össze a lokális deriváltak mély hálóban.

Előismeret: a 9.3. Expresszivitás és approximációs állítások aktivációi, valamint a derivált és a láncszabály 4.1.–4.3. fogalmai (a láncszabály mátrixos tárgyalása a [B01] 5. fejezetében olvasható); a tanulási célfüggvény és a gradienslépés visszatekintése a 8.4. II. ellenőrző pont: korrekt tanulási állítás alfejezetből elegendő.

Induló kérdés. Egy 12 rétegű skaláris hálóban minden réteg $\sigma(z)$ szigmoidot használ, súlya $w_\ell = 1$. Körülbelül mekkora lehet a bemenetre vonatkozó gradiens abszolút értéke? A választ a helyi deriváltak felső korlátja adja, amelyet a 9.4.4 szakaszban pontosan levezetünk; a korlát $(1/4)^{12} \approx 5.96 \cdot 10^{-8}$, ami felső korlát, nem a gradiens tipikus értéke.

9.4.2 Aktiváció mint elemenkénti függvény és lokális deriváltja

Legyen $\phi : \mathbb{R} \to \mathbb{R}$ egy aktiváció. Egy $A \in \mathbb{R}^{N \times m}$ preaktivációs mátrixon az aktivációs réteg **elemenként** hat:

$$\Phi(A)_{ij} = \phi(A_{ij}), \qquad H = \Phi(A) \in \mathbb{R}^{N \times m}.$$

Ha ϕ differenciálható az A_{ij} pontban, akkor H_{ij}-re csak A_{ij} hat, ezért a parciális derivált

$$\frac{\partial H_{ij}}{\partial A_{ij}} = \phi'(A_{ij}),$$

minden más parciális derivált nulla. A skaláris veszteségre visszavezetett lokális VJP (vector–Jacobian product, vektor–Jakobi-szorzat) tehát

$$\frac{\partial \mathcal{L}}{\partial A_{ij}} = \phi'(A_{ij}) \, \frac{\partial \mathcal{L}}{\partial H_{ij}},$$

ahol itt $\mathcal{L}$ a veszteség jele (a rétegek számát később L jelöli, ezért a két mennyiséget megkülönböztetjük), és $\partial \mathcal{L}/\partial A$ alakja megegyezik A alakjával. Ez a fenti lokális VJP-képlet az egész

fejezet központi szabálya; a teljes háló gradiense ilyen lokális szabályok kompozíciója lesz a 10.1. Számítási gráf és lokális derivált alfejezetben.

Három gyakori aktiváció és a definiált pontokon vett deriváltja:

Aktiváció	Képlet	Lokális derivált, ahol létezik
szigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma'(z) = \sigma(z)(1-\sigma(z))$
tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$\tanh'(z) = 1 - \tanh(z)^2$
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$	0 ha $z < 0$, 1 ha $z > 0$; $z = 0$-ban nincs klasszikus derivált, a visszaterjesztésben konvenció szerint 0

A szigmoid deriváltja egyetlen lépésben ellenőrizhető:

$$\frac{d}{dz}\frac{1}{1+e^{-z}} = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1-\sigma(z)),$$

ahol az utolsó egyenlőségnél $1 - \sigma(z) = e^{-z}/(1+e^{-z})$.

9.4.3 Telítődés: a lokális derivált mérete

Definíció. Legyen ϕ differenciálható, és tegyük fel, hogy $M = \sup_{z\in\mathbb{R}} |\phi'(z)|$ véges és pozitív. Rögzített $\varepsilon \in (0,1)$ küszöb mellett azt mondjuk, hogy ϕ a z pontban **ε-telített**, ha $|\phi'(z)| \le \varepsilon M$. A telítődés tehát küszöbhöz kötött fogalom: a derivált abszolút értékét a saját maximumához viszonyítjuk, nem egy univerzális számhoz. Például a szigmoidra $M = 1/4$, és $\sigma'(5)/M \approx 0.0266$, ezért $\varepsilon = 0.05$ mellett a $z = 5$ pont már ε-telített. Pontosabban:

Állítás. Minden $z \in \mathbb{R}$ esetén $0 < \sigma'(z) \le 1/4$, és $|z| \to \infty$ mellett $\sigma'(z) \to 0$.

Bizonyítás. Legyen $t = \sigma(z) \in (0,1)$. Ekkor $\sigma'(z) = t(1-t)$. A $t \mapsto t(1-t)$ parabola maximuma $1/4$, amelyet $t = 1/2$ esetén vesz fel. A határérték $z \to \infty$ esetén $\sigma(z) \to 1$, tehát $\sigma'(z) = \sigma(z)(1-\sigma(z)) \to 0$; $z \to -\infty$ esetén analóg módon $\sigma'(z) \to 0$. $\square$

A táblázat a szigmoid deriváltját és a maximumhoz viszonyított arányát mutatja néhány pontban (a képletből számított értékek, nem mérés):

z	-5	-3	-1	0	1	3	5
$\sigma'(z)$	0.00665	0.04518	0.19661	0.25	0.19661	0.04518	0.00665
$\sigma'(z)/M$	0.02659	0.18071	0.78645	1	0.78645	0.18071	0.02659

A tanh esetében $\tanh'(z) = 1 - \tanh(z)^2 \in (0,1]$, és $|z| \to \infty$ esetén ugyancsak nullához tart. A ReLU a pozitív oldalon konstans 1 deriváltat ad, tehát a pozitív tartományban nem telítődik. A negatív oldalon a függvény konstans, ezért a derivált minden $z < 0$ pontban pontosan nulla; ez nem fokozatos telítődés, hanem a függvény állandósága egy egész félegyenesen. A nullpontban a derivált nem létezik; erről a következő szakasz szól.

A telítődés önmagában még lokális jelenség. Ahhoz, hogy egy mély háló bemeneti gradiensére gyakorolt hatását lássuk, láncszabályra és a lokális deriváltak **szorzatára** van szükség.

9.4.4 Szorzatos láncszabály és eltűnő gradiens

Tekintsük a skaláris, bias nélküli, L rétegű

$$h_0 = x, \qquad a_\ell = w_\ell h_{\ell-1}, \qquad h_\ell = \phi(a_\ell), \qquad f_L(x) = h_L$$

rekurziót, ahol $w_\ell \in \mathbb{R}$ és ϕ differenciálható minden fellépő a_ℓ pontban. Jelölés: a rétegek számát L, a rejtett aktivációkat h_ℓ jelöli; a numerikus differenciálás lépésközét a továbbiakban Δ jelöli, hogy ne ütközzön a rejtett aktivációval.

Lemma (szorzatos gradiens). A fenti rekurzióra

$$\frac{df_L}{dx} = \prod_{\ell=1}^{L} \Big(w_\ell \, \phi'(a_\ell) \Big).$$

Bizonyítás. $L = 1$-re a láncszabály közvetlenül $\frac{df_1}{dx} = w_1 \phi'(a_1)$. Tegyük fel, hogy az állítás $L-1$-re igaz. Ekkor

$$\frac{df_L}{dx} = \frac{d}{dh_{L-1}} \phi(w_L h_{L-1}) \cdot \frac{dh_{L-1}}{dx} = w_L \phi'(a_L) \cdot \prod_{\ell=1}^{L-1} \Big(w_\ell \phi'(a_\ell) \Big).$$

A szorzat tehát L-re is teljesül; az indukció lezárja a bizonyítást. $\square$

A lemma közvetlen következménye az eltűnő gradiens egy egyszerű alakja. Ha minden ℓ-re $|w_\ell \phi'(a_\ell)| \le q < 1$, akkor

$$\left| \frac{df_L}{dx} \right| \le q^L,$$

és q^L exponenciálisan nullához tart. A szigmoidra a fenti deriváltképlet miatt $|\sigma'(a_\ell)| \le 1/4$, tehát $|w_\ell| \le 1$ esetén $q = 1/4$ választható. Például $L = 12$ esetén

$$\left| \frac{df_{12}}{dx} \right| \le \left(\frac{1}{4} \right)^{12} = 5.9605 \cdot 10^{-8}.$$

Ez a **felső korlát**: a tényleges gradiens abszolút értéke legfeljebb ennyi, de ennél jóval kisebb is lehet. A korlát nem bizonyítja, hogy a tanítás szükségképpen sikertelen; azt mutatja, hogy a nyers gradiens nagysága egy a mélységgel exponenciálisan csökkenő felső korlát alatt marad, és ez a korlát a mélység növelésével tetszőlegesen kicsivé tehető. A tényleges gradiensről a korlát önmagában nem állítja, hogy az is tetszőlegesen kicsi: a gradiens lehet a korláthoz közeli, de attól jóval elmaradó is.

Érdemes megkülönböztetni a jelenség két okát. A szigmoidos példában a lokális derivált maximuma $1/4$, tehát a szorzat akkor is lecseng, ha minden súly egységnyi. A tanh lánc az $x = 0$, $w_\ell = 0.8$ választással ezzel szemben nem telített pontokon halad át, mert $\tanh'(0) = 1$, és a gradiens mégis $(0.8)^{12} \approx 0.0687$: itt a kis súlyok szorzata okozza a lecsengést. A $w = 0.8$ választás önkényes, de szemlélteti a nem telített, mégis lecsengő esetet: $0 < w < 1$ esetén a szorzat a telítődéstől függetlenül csökken, míg $w = 1.2$ választással ugyanez a háló növekedne. Az eltűnő gradiens tehát **termékjelenség**, nem feltétlenül telítődési jelenség. A jelenség áttekintése a [B02] 8. fejezetében olvasható.

9.4.5 A ReLU töréspontja és a szubgradiens-konvenció

A ReLU nullpontja külön kezelést igényel. A $z = 0$ pontban a bal oldali derivált

$$\lim_{\Delta \to 0^-} \frac{\mathrm{ReLU}(\Delta) - \mathrm{ReLU}(0)}{\Delta} = \lim_{\Delta \to 0^-} \frac{0}{\Delta} = 0,$$

míg a jobb oldali derivált

$$\lim_{\Delta \to 0^+} \frac{\Delta - 0}{\Delta} = 1.$$

A két egyoldali derivált nem egyenlő, ezért a klasszikus értelemben vett $\mathrm{ReLU}'(0)$ **nem létezik**.

A konvex (domború) függvények elméletében a 0 pontbeli **szubdifferenciál** azokból a g értékekből áll, amelyekre minden z-re

$$\mathrm{ReLU}(z) \geq \mathrm{ReLU}(0) + g(z - 0) = gz.$$

$z > 0$ esetén ez $z \geq gz$, azaz $g \leq 1$; $z < 0$ esetén $0 \geq gz$, azaz $g \geq 0$. A szubdifferenciál tehát a $[0, 1]$ intervallum. A PyTorch `relu` backward szabálya a $z = 0$ pontban a 0 értéket választja (a `torch.relu` gradiense nullában 0). Ez a szubdifferenciál egy eleme, tehát szubgradiens-konvenció, de **nem klasszikus derivált**. A $[0, 1]$ intervallum bármely eleme érvényes szubgradiens lenne, ezért a 0-beli érték megválasztása konvenció kérdése; a saját kódunkban ezt a választást kommentben rögzíteni kell.

Dead ReLU (halott ReLU). Ha egy neuron preaktivációja minden tanítópéldára nempozitív, akkor kimenete mindig 0, lokális gradiensértéke a fenti konvenció szerint szintén 0, ezért a bemenő súlyokra és a biasra eső gradiens minden tanítópéldában nulla lesz. Az ilyen neuron „halott”: a tiszta, momentum (lendület) nélküli adatgradiens-lépés alatt (más hozzájárulás nélkül) a bejövő paraméterei nem változnak; korábbi momentum, az előző rétegek változása, közös paraméter vagy weight decay (súlycsökkentés) mégis reaktiválhatja, így a hálózat reprezentációs kapacitásából ez a neuron kiesik. Ez **heurisztikus** megfigyelés, nem tétel: súlycsökkentéssel vagy az adateloszlás megváltozásával a neuron elvileg újra aktiválódhat, de a tiszta gradiensalapú tanításban erre nincs garancia.

Kis ellenőrző példa: $h = \mathrm{ReLU}(x - 5)$ a $x \in \{1, 2, 3\}$ bemeneteken. A preaktiváció mindig $-4, -3, -2$, tehát $h = 0$ és a lokális gradiens minden mintára 0. Legyen a veszteség például $\mathcal{L} = \frac{1}{2}(h - y)^2$; ekkor $\partial \mathcal{L}/\partial h = h - y = -y$, ami általában nem nulla, de a láncszabály szerint $\partial \mathcal{L}/\partial a = (\partial \mathcal{L}/\partial h) \cdot 0 = 0$, ezért a bias gradiense is nulla, és a bias értéke nem változik: a preaktiváció továbbra is negatív marad. A tanítóhalmaz itt csak ezt a három pontot tartalmazza.

9.4.6 Labor: aktivációrajz és gradiens két mélységnél

A labor célja: ábrázoljuk az aktivációkat és deriváltjaikat, majd mérjük meg kézi láncszabállyal ugyanannak a skaláris hálónak a bemeneti gradiensét $L = 4$ és $L = 12$ mélységben, és hasonlítsuk össze egy független, középső differenciás értékkel. A kód NumPy-t használ, CPU-n fut, és determinisztikus; a kimeneti könyvtárat maga hozza létre. A skaláris háló `float32`-ben számol, mert ez a GPU-k alapértelmezett pontossága, és így a szorzat kerekítési hibája is látható; a középső differencia `float64`-ben fut, hogy a kivonási hiba ne nyomja el a h^2-es tagot. A mért kimenet Python 3.12.3, NumPy 2.5.4, matplotlib 3.11.2, CPU, 1 szál mellett született.

```python
# lab/n036/activation_gradients.py
import pathlib

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def sigmoid(z):
    return np.float32(1.0) / (np.float32(1.0) + np.exp(-z))

def relu(z):
    return np.maximum(np.float32(0.0), z)

def drelu(z):
    # Visszaterjesztési konvenció: z=0-ban 0; ott klasszikus derivált nincs.
    return (z > 0).astype(np.float32)

def dsigmoid(z):
    s = sigmoid(z)
    return s * (np.float32(1.0) - s)

def dtanh(z):
    t = np.tanh(z)
    return np.float32(1.0) - t * t

def scalar_mlp(x, depth, w, phi, dphi):
    """Skaláris, bias nélküli MLP. Visszaadja a kimenetet és a dx gradienst."""
    h = np.float32(x)
    local_derivatives = []
    for _ in range(depth):
        a = np.float32(w) * h
        local_derivatives.append(np.float32(w) * np.float32(dphi(a)))
        h = phi(a)
    grad = np.prod(local_derivatives, dtype=np.float32)
    return np.float32(h), np.float32(grad)

def scalar_mlp_float64(x, depth, w):
    """Ugyanaz a tanh-háló float64-ben, a középső differenciához."""
    h = np.float64(x)
    for _ in range(depth):
        h = np.tanh(np.float64(w) * h)
    return h

def main():
    x = np.float32(0.0)   # tanh-nál nem telített pont
    w = np.float32(0.8)
    delta = 1e-5
    for depth in (4, 12):
        y, g = scalar_mlp(x, depth, w, np.tanh, dtanh)
        fd = (scalar_mlp_float64(np.float64(x) + delta, depth, w)
              - scalar_mlp_float64(np.float64(x) - delta, depth, w)) / (2.0 * delta)
        print(f"depth={depth:2d}  output={y:.6f}  input_grad={g:.6f}  fd={fd:.6f}")
        assert np.isclose(float(y), 0.0, atol=1e-6)
        assert np.isclose(float(g), 0.8 ** depth, rtol=1e-5)
        assert np.isclose(float(g), fd, rtol=1e-6, atol=1e-12)

    z = np.linspace(-5, 5, 501, dtype=np.float32)
    plt.figure(figsize=(8, 4))
    plt.plot(z, sigmoid(z), label="sigmoid")
    plt.plot(z, dsigmoid(z), label="sigmoid derivált")
    plt.plot(z, np.tanh(z), label="tanh")
    plt.plot(z, dtanh(z), label="tanh derivált")
```

```
    plt.plot(z, relu(z), label="ReLU")
    plt.plot(z, drelu(z), label="ReLU derivált (0-ban konvenció: 0)")
    plt.grid(True)
    plt.legend()
    plt.title("Aktivációk és lokális deriváltak")
    out = pathlib.Path("lab/n036")
    out.mkdir(parents=True, exist_ok=True)
    plt.savefig(out / "activations.png", dpi=150)
    print(f"mentve: {out / 'activations.png'}")

if __name__ == "__main__":
    main()
```

Futtatási parancs a könyv munkakönyvtárából:

```
python lab/n036/activation_gradients.py
```

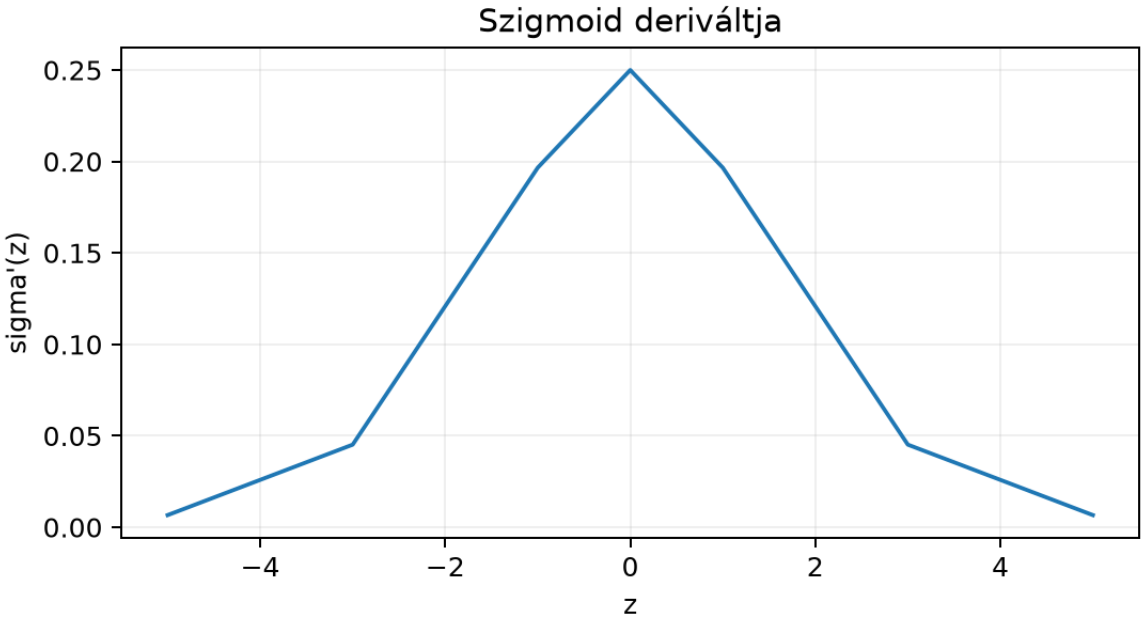
Előfeltétel: a venv tartalmazza a NumPy-t és a matplotlibet; a matplotlib háttere `Agg`, ezért nincs szükség kijelzőre. A parancsot a könyv gyökérkönyvtárából kell futtatni, mert a kód a `lab/n036` relatív útvonalat használja, és a mappát `mkdir(parents=True, exist_ok=True)` hozza létre.

Mért kimenet és értelmezés. Mivel $x = 0$ esetén $\tanh(0) = 0$, minden réteg preaktivációja nulla, így minden $\tanh'(a_\ell) = 1$; a gradiens pontosan a súlyok szorzata. A kód kimenete így kézzel ismert kontrollt ad:

```
depth= 4  output=0.000000  input_grad=0.409600  fd=0.409600
depth=12  output=0.000000  input_grad=0.068719  fd=0.068719
mentve: lab/n036/activations.png
```

A fenti sorok a tényleges futtatás kimenete; az `input_grad` a szorzatképlet float32-es értéke, az `fd` pedig ugyanarra a hálóra a float64-es középső differencia. Az értékek a megjelenített hat tizedesjegyig megegyeznek a képletből számított $0.8^4 = 0.4096$ és $0.8^{12} = 0.068719476736$ értékkel, és a két számítási mód is hat tizedesjegyig egyezik. Az értelmezés nem az, hogy a hálózat „elromlott”, hanem az, hogy a bemeneti gradiensre a q^L alakú felső korlát érvényes; $q = 0.8$-nál tizenkét réteg után a bemeneti réteg súlyfrissítése jóval kisebb tényezővel skálázódik, mint négy rétegnél.

A szigmoidot és deriváltját egyszerű diagramon is érdemes látni:



9.7. ábra: Szigmoid deriváltja

A képletből számított értékek; nem futási kimenet.

A fenti beágyazott diagramot a `~~~chart` blokkból a build generálja; adatai képletből számított értékek, nem futási kimenet. A labor által előállított `lab/n036/activations.png` ábrán ezzel szemben a ReLU deriváltja is szerepel: a lépcső 0-ról 1-re ugrik a nullpontnál, ahol klasszikus derivált nincs.

9.4.7 Tipikus tévedés: a töréspontbeli konvenció nem derivált

A következő visszaterjesztési kód gyakori:

```
def relu_backward(a, grad_out):
    # ReLU-derivált
    return (a > 0) * grad_out
```

A kommentben gyakran „ReLU-derivált” szerepel. Ez hibás elnevezés: a kód a 0 pontban a PyTorch-hoz hasonlóan 0-t ad, miközben a ReLU-nak ott nincs deriváltja. A különbség mérhető: a középső differencia

$$\frac{\mathrm{ReLU}(\Delta) - \mathrm{ReLU}(-\Delta)}{2\Delta} = \frac{\Delta - 0}{2\Delta} = 0.5$$

minden $\Delta > 0$-ra, tehát a véges differencia 0.5-öt ad a 0 pontban, a fenti kód pedig 0-t. Ez nem a kód numerikus hibája, hanem annak következménye, hogy a középső differencia hányadosa a töréspontban is kiszámítható, de ott nem deriváltat közelít: a bal oldali határérték 0, a jobb oldali 1, ezért a differencia-hányadosnak nincs határértéke. A 10.3. Gradiensellenőrzés és numerikus hibamérce gradiensellenőrzési protokollja ezért a vizsgálati pontot a törésponttól távol veszi fel, vagy a vizsgált konvenciót előre rögzíti.

9.4.8 Önellenőrzés és visszatekintés

Ellenőrizze, hogy tudja-e: (1) igazolni a $\sigma' \le 1/4$ korlátot; (2) levezetni a $df_L/dx = \prod_\ell w_\ell \phi'(a_\ell)$ képletet; (3) megmondani, miért nem létezik a klasszikus ReLU-derivált nullában, és mi a szubdifferenciál.

Visszautalás a 8.4. II. ellenőrző pont: korrekt tanulási állítás alfejezetből: a gradiens az empirikus kockázat (a tanítóminta átlagos vesztesége) plusz regularizáló cél szerinti frissítési irány része; a gradiens kicsinysége a **nyers lépés** skáláját csökkenti, de Adam-szerű (adaptív momentumú) normalizáló optimizálók ezt részben ellensúlyozhatják. Ugyanakkor az eltűnő gradiens itteni levezetése semmit nem mond az általánosításról, és a kis gradiensből nem következik, hogy a tesztkockázat kicsi vagy nagy lesz: ez pontosan a 8.4. II. ellenőrző pont: korrekt tanulási állítás tanítási állítások határa, amelyet itt nem lépünk át.

9.4.9 Feladatok

F1 (fogalmi, nehézség 1). Definiálja egy mondatban a telítődést és a dead ReLU jelenséget, majd adjon mindkettőre egy-egy konkrét egyváltozós számszerű esetet. Elvárt eredmény: két definíció, egy-egy kiszámolt lokális derivált vagy gradiensérték, és egy mondat arról, miben más a kettő.

F2 (levezetési, nehézség 2). Vezesse le a 9.4.4 szakasz rekurziójából a gradiens szorzatképletét $L = 3$-ra, bias nélkül, tetszőleges differenciálható ϕ-vel. Ezután számítsa ki $x = 0$, $\phi = \tanh$, $w_1 = 0.7$, $w_2 = 0.9$, $w_3 = 1.2$ esetén a pontos gradiensértéket. Elvárt eredmény: teljes láncszabály-lépések, a fellépő a_ℓ értékek és a 0.756 pontos érték levezetése (a tizedes szorzat egzakt; float64-ben a megjelenítés kerekítést igényelhet).

F3 (implementációs, nehézség 2). Készítsen egy `lab/n036/feladat_f3.py` programot, amely egy háromrétegű, tanh aktivációjú skaláris MLP gradiensét kézi szorzatképlettel és középső differenciával is kiszámítja. Input: $x = 1.0$; a három súlyt `np.random.default_rng(20261020)` generálja `float64` pontossággal a $[0.5, 1.5]$ intervallumból; a differenciálási lépés $\Delta = 10^{-6}$ (a kódban `delta`). Elvárt beadandó eredmény: a teljes program, a kiírt analitikus és numerikus gradiens, valamint egy `assert` ellenőrzés, hogy az abszolút eltérés 10^{-5} alatt van. Magyarázza meg a `float64` választását.

F4 (hibakeresési, nehézség 2). Egy társa a következő kódot mutatja:

```
def relu_backward(a, grad_out):
    # ReLU deriváltja
    return (a > 0) * grad_out
```

A függvényt az $a = 0$, `grad_out=1.0` pontban teszteli, és arra panaszkodik, hogy a középső differencia szerint 0.5-nek kellene lennie, a kód viszont 0-t ad. Lokalizálja a hibát, nevezze meg a tényleges okot, és adjon olyan javítást, amely nem nevezi deriváltnak a 0 pontbeli visszaterjesztési értéket. Elvárt beadandó eredmény: ok, következmény, javított függvény dokumentációval és két olyan tesztpont, ahol a klasszikus derivált létezik, valamint a numerikus visszaellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérleti protokollt a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adathoz annak eldöntésére, hogy azonos szélesség és mélység mellett a tanh vagy a ReLU MLP ér el kisebb validációs loss-t a három orientáció osztályozásában. A terv kövesse a 8.4. II. ellenőrző pont: korrekt tanulási állítás és 5.4. Függő képek, seedek és resampling tanulságait: a képeket lapítsa $24 \cdot 24 = 576$ hosszú vektorrá, a scene_id csoportosítást ne sértse meg, a tesztet ne használja modellválasztásra, és legalább öt tanítási seeddel mérje a seedek közötti szórást. Adjon baseline-t, optimalizálót, epochszámot,

batchméretet és kiértékelési metrikát, valamint két módszertani korlátot. Elvárt beadandó eredmény: egy bekezdésnyi protokoll-leírás a fenti elemekkel és a korlátokkal.

9.4.10 Megoldások és indoklás

F1. A telítődés lokális tulajdonság: $|\phi'(z)|$ kicsi. Például $\sigma'(0) = 0.25$, de $\sigma'(5) = \sigma(5)(1 - \sigma(5)) \approx 0.00665$, tehát a függvény az 5 környezetében telített. A dead ReLU ezzel szemben adatfüggő, globálisabb jelenség: ha minden tanítópéldára $a \leq 0$, a neuron kimenete és lokális gradiense is nulla. Például $h = \mathrm{ReLU}(x - 5)$ a $x \in \{1, 2, 3\}$ tanítóhalmazon mindig 0 kimenetet ad, így a bemenő súly gradiense minden mintára 0. A különbség: a telített szigmoid deriváltja kicsi, de nem nulla, és más bemenetre azonnal „felébredhet”; a holt ReLU bejövő paraméterei a rögzített tanítóhalmazon nulla gradiensük miatt nem frissülnek.

F2. A láncszabályt hátulról felírva:

$$\frac{df_3}{dx} = w_3\phi'(a_3) \cdot w_2\phi'(a_2) \cdot w_1\phi'(a_1), \qquad a_1 = w_1 x,\ a_2 = w_2 h_1,\ a_3 = w_3 h_2.$$

Az $x = 0$ kezdőpont miatt minden preaktiváció nulla, hiszen a bias nincs jelen, és $\tanh(0) = 0$. Mivel $\tanh'(0) = 1 - \tanh(0)^2 = 1$, a szorzat

$$\frac{df_3}{dx} = 0.7 \cdot 0.9 \cdot 1.2 = 0.756.$$

Az alakellenőrzés: minden tényező skalár, ezért az eredmény is skalár. A $\tanh'(0) = 1$ és a $0.7 \cdot 0.9 \cdot 1.2 = 0.756$ tizedes szorzat egzakt; a float64-ben tárolt tényezők szorzata legfeljebb 10^{-16} nagyságrendben térhet el ettől, ezért a megjelenítés kerekítést igényelhet.

F3. Egy teljes megoldás:

```
# lab/n036/feladat_f3.py
import numpy as np

def tanh(x):
    return np.tanh(x)

def dtanh(x):
    return 1.0 - np.tanh(x) ** 2

def forward(x, ws):
    acts = []
    h = x
    for w in ws:
        a = np.float64(w) * np.float64(h)
        acts.append(a)
        h = tanh(a)
    return np.float64(h), acts

def analytic_grad(ws, acts):
    g = np.float64(1.0)
    for w, a in reversed(list(zip(ws, acts))):
        g = g * np.float64(w) * dtanh(a)
    return g

rng = np.random.default_rng(20261020)
ws = [np.float64(rng.uniform(0.5, 1.5)) for _ in range(3)]
x0 = np.float64(1.0)
```

```
y, acts = forward(x0, ws)
g_an = analytic_grad(ws, acts)

delta = np.float64(1e-6)
yp, _ = forward(np.float64(x0 + delta), ws)
ym, _ = forward(np.float64(x0 - delta), ws)
g_num = (yp - ym) / (np.float64(2.0) * delta)
print("sulyok:", ws)
print("analitikus:", g_an)
print("numerikus:  ", g_num)
print("abszolut elteres:", abs(g_an - g_num))
assert abs(g_an - g_num) < 1e-5
```

A `float64` azért szükséges, mert a középső differencia két közeli kimenet kivonását végzi; `float32`-ben a kivonásnál a kerekítési zaj nagyobb lehet, mint a Δ^2-es módszerhiba. A `tanh` sima az $x_0 = 1$ környezetében, ezért a klasszikus derivált létezik, és a véges differencia jogos ellenőrzés. Alakellenőrzés: a gradiens skalár, mert a bemenet, a súlyok és a kimenet is skalár.

F4. A hiba típusa: fogalmi tévedés, nem numerikus implementációs hiba. A `(a > 0)` kifejezés a nullpontban `False`, ezért 0-t ad, de a ReLU-nak a nullpontban nincs klasszikus deriváltja, így a középső differencia 0.5 értéke sem „helyes derivált”. A konvenciót dokumentálni kell, a tesztet pedig olyan pontokra vinni, ahol a derivált létezik. Javított változat:

```
def relu_backward(a, grad_out):
    # Visszaterjesztési konvenció: a 0-beli érték 0, nem klasszikus derivált.
    # A klasszikus derivált csak a != 0 pontokban létezik.
    # A maszkos szorzás skalárra és ndarray-re egyaránt működik.
    return (a > 0) * grad_out
```

A függvény skalár és NumPy-tömb bemenetre is lefut, így az $a = 0$, `grad_out=1.0` hívás a dokumentált konvenció szerint 0-t ad. Numerikus visszaellenőrzés a két sima ponton:

```
import numpy as np

def relu_backward(a, grad_out):
    return (a > 0) * grad_out

for a in (1.0, -1.0):
    x, delta = np.float64(a), np.float64(1e-6)
    fd = (max(0.0, x + delta) - max(0.0, x - delta)) / (2.0 * delta)
    print(f"a={a:+.0f}  backward={float(relu_backward(np.float64(a), np.float64(1.0))):.1f}
     ↪  fd={fd:.1f}")
```

A futtatás kimenete: `a=+1  backward=1.0  fd=1.0` és `a=-1  backward=0.0  fd=0.0`, mindkettő megegyezik a klasszikus deriválttal, mert ott a ReLU sima. A 0 pontot ellenőrzésként ki kell zárni vagy külön konvenciótesztként kell jelölni.

F5. Egy lehetséges protokoll. Kérdés: tanh vagy ReLU aktivációjú, azonos kapacitású MLP ér el kisebb validációs loss-t a GlyphLab orientációfeladatán? Baseline: lineáris logisztikus regresszió az ellapított 576 dimenziós bemeneten, így látható, hogy a nemlinearitás egyáltalán hoz-e javulást. Adat: `data/images.npy`, `data/labels.npy`, a megadott train/val/test scene-split; a tesztet a modell- és hiperparaméter-választás lezárásáig nem értékeljük. Modellek: két rejtett réteg, szélesség 64, tanh illetve ReLU, kimeneti logits, `CrossEntropyLoss`. A tanítás PyTorch-ban fut, mert a fejezet NumPy-laborja csak előrefelé számol, backwardöt nem tartalmaz; a `CrossEntropyLoss` logitsból számol, ezért a háló kimenete nyers logit (nem softmax). Optimalizáló: Adam (adaptív momentumú), learning rate 10^{-2}, batchméret 32, 25 epoch (teljes tanítóhalmaz-átfutás), a legjobb validációs loss-ú checkpoint (mentett modellállapot) kiválasztásával. Ismétlés: öt tanítási seed; minden seednél ugyanaz a tanítóhalmaz, csak az inicializálás és a batchsorrend változik. Kiértékelés: validációs loss (másodlagosan accuracy) átlaga és szórása a seedek között, a

korlátozott tesztet csak a kiválasztott konfiguráción futtatjuk egyszer. A két aktivációt ugyanazon öt seeddel párosítva hasonlítjuk össze; a döntési szabály előre rögzítendő: az egyik aktiváció akkor tekinthető jobbnak, ha a seedenkénti párosított validációs loss-különbség mind az öt esetben ugyanabba az irányba mutat, egyébként az eredmény nem dönthető el. Korlátok: a GlyphLab nem természeteskép-benchmark, ezért a sorrend nem vihető át általános látási feladatra; az öt seed az algoritmikus bizonytalanságot méri, nem az adatbázis más jeleneteire vonatkozó bizonytalanságot, amelyhez az 5.4. Függő képek, seedek és resampling szerint scene-alapú bootstrap kellene. A 8.4. II. ellenőrző pont: korrekt tanulási állítás tanulsága szerint az eredményt nem szabad „bizonyítottan jobb aktivációként” közölni, csak ezen a rögzített adaton és spliten mért empirikus különbségként. Erőforrás: CPU, PyTorch, a rögzített `cnn-labor` környezet.

9.4.11 Rövid összefoglaló

Az aktivációs réteg elemenként hat, lokális VJP-je $\phi'(A_{ij})$ szorzó. A szigmoid deriváltjának maximuma $1/4$, és nagy abszolút preaktivációnál nullához tart, ez a telítődés. A mély skaláris háló bemeneti gradiense a lokális deriváltak szorzata; ha minden tényező abszolút értékét $q < 1$ korlátozza, a gradiens q^L ütemben eltűnhet. A ReLU nullpontjában nincs klasszikus derivált; a $[0, 1]$ szubdifferenciálból a könyvtárak csak konvenció szerint választanak értéket. A dead ReLU adatfüggő jelenség, amely a bejövő súlyok gradiensét nullázza. A 9.4.6 labor a tanh láncon 0.8^4 és 0.8^{12} mért gradienst ad, a szorzatszabálytól független középső differenciával egyezően, és az aktivációkat, valamint a ReLU deriváltjának lépcsőjét is ábrázolja.

9.4.12 Továbblépés és források

A következő, 10.1. Számítási gráf és lokális derivált egység a számítási gráfot és a lokális deriváltak összeillesztését formalizálja; a jelen szorzatos képlet annak speciális, lineáris lánca. A ReLU töréspontbeli konvenció a 10.3. Gradiensellenőrzés és numerikus hibamérce gradiensellenőrzési protokolljában tér vissza, ahol a véges differencia értéke a nullpontban tudatosan kizárt.

Források. A szakasz levezetései saját, elemi levezetések; a Cybenko-tétel (a szigmoid aktivációjú hálók approximációs tétele, lásd 9.3.) itt nem szerepel. Háttérjelöléshez:

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y. és Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

A jelen egységben oldalszámot, táblázatot vagy idézetet e forrásokból nem közlünk.

10 Backpropagation a könyvtár előtt

10.1 Számítási gráf és lokális derivált

10.1.1 Cél, előismeret és induló becslés

A cél: a visszaterjesztést nem „varázslatként”, hanem egy irányított aciklikus számítási gráfon (DAG) végzett gépi eljárásként leírni, és pontosan bizonyítani, miért **összeadódnak** egy elágazó csúcshoz több útról érkező gradiens-hozzájárulások. Ez az összegzés a későbbi konvolúciós rétegek és reziduális hálók gradiensének is alapja.

Előismeret: a láncszabály és VJP 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetből, az aktivációk lokális deriváltja 9.4. Aktivációk és gradiensáramlás alfejezetből; az optimalizáló frissítés fogalma 6.4. Momentum és adaptív optimizálók alfejezetből, a batch-átlagos veszteség skálája 6.1. Objektív, reguláris tag és gradienslépés alfejezetből. Visszautalunk a neuronra 9.1. Neuron és perceptron: ott egyetlen affin score és egy aktiváció alkotta a modellt; itt ezt kötőelemekre bontjuk.

Induló becslés. Legyen $u = (1, -1)^\top$, $x_1 = (1, 2)^\top$, $x_2 = (3, 1)^\top$, és legyen $a = u^\top x_1$, $b = u^\top x_2$, $L = \frac{1}{2}a^2 + \frac{1}{2}b^2$. Ha a visszaterjesztéskor *csak az a-ágat* vennénk figyelembe, mekkora lenne a hiba $\nabla_u L$-ben? A teljes megoldás a 10.1.5 szakaszban: a helyes gradiens $(5, 0)^\top$, a hiányos $(-1, -2)^\top$, így a hiba $(6, 2)^\top$, tehát nem kicsi, hanem maga a második ág hozzájárulása.

10.1.2 Számítási gráf, topologikus sorrend, mentett köztes érték

Definíció (számítási gráf). A modell kiértékelésének számítási gráfja egy $G = (V, E)$ irányított gráf, ahol

- minden $v \in V$ csúcshoz egy tenzor tartozik,
- minden $e = (u, v) \in E$ él azt jelenti, hogy v kiszámításához közvetlenül felhasználjuk u értékét,
- a gráf irányított és **aciklikus**: nincs $v_0 \to v_1 \to \ldots \to v_m = v_0$ irányított kör.

A csúcsok szerepe szerint megkülönböztetünk **input csúcsokat** (paraméterek és bemeneti adatok), **köztes csúcsokat** és **veszteség csúcsot**, ahol $L \in \mathbb{R}$ skalár. A kör kizárása szükséges: kör esetén a forward-kiértékelés nem tudna megállni, mert a v_0 kiértékeléséhez önmaga értékére lenne szükség.

Definíció (topologikus sorrend). A $G = (V, E)$ DAG egy topologikus sorrendje a csúcsok olyan $v_1, \ldots, v_T$ felsorolása, amelyben minden $(v_i, v_j) \in E$ élre $i < j$. Minden véges DAG-nak van topologikus sorrendje; a forward-kiértékelés egyszerűen ezt a sorrendet követi, a visszaterjesztés pedig a **megfordított** $v_T, \ldots, v_1$ sorrendet.

A létezés rövid indoklása: minden nem üres véges DAG-ban van nulla bemenő fokú csúcs, azaz forrás. Ha nem lenne, egy csúcsból kiindulva a bemenő éleken visszafelé lépkedve végtelen sétát kapnánk, amely a véges sok csúcs miatt ismétlődést, és ezzel irányított kört zárna. A forrás

és a belőle induló élek elhagyása után a maradék gráf is DAG, így indukcióval kapjuk a teljes felsorolást.

Definíció (mentett köztes érték). Egy csúcs **mentett köztes értékei** (saved intermediates) azok a tenzorok, amelyeket a csúcs forward-számításakor elmentünk, mert a lokális gradiens kiszámításához a backward lépésben szükségesek. Például a ϕ aktivációfüggvény $\phi(z)$ értékének lokális deriváltjához a bemeneti z kell, a szigmoid (σ) esetén $\sigma'(z) = \sigma(z)(1-\sigma(z))$ miatt elegendő az output mentése is. Implementációs döntés, hogy mit mentünk; matematikailag a lokális derivált függvénye a csúcs bemenetének.

10.1.3 Lokális VJP és a visszafelé haladás szabálya

Legyen a veszteség L, és jelölje $g_v = \nabla_v L$ a v csúcs gradiensét; ennek **alakja megegyezik** v **alakjával** (a könyv oszlopvektor-konvenciója szerint). A visszaterjesztés során minden csúcsra ezt a tenzort számítjuk ki, a kimenetére érkező gradiensből kiindulva.

Definíció (lokális VJP). Legyen a v csúcshoz tartozó művelet $y = f(u)$, ahol $u \in \mathbb{R}^d$ a bemeneti tenzor, $y \in \mathbb{R}^p$ a csúcs kimeneti tenzora, és f differenciálható. A művelet **lokális VJP-leképezése** (VJP: vector–Jacobian product, vektor–Jakobi-szorzat) a

$$g_u = J_f(u)^\top g_y$$

hozzárendelés, ahol $J_f(u) \in \mathbb{R}^{p\times d}$ a Jacobi-mátrix (sor: output-index, oszlop: input-index), és $g_y = \nabla_y L \in \mathbb{R}^p$ a csúcs kimenetére érkező gradiens (a v csúcsnál ez g_v). A g_u eredmény $\mathbb{R}^d$-beli, tehát alakja megegyezik u alakjával. A VJP-t az egész Jacobi-mátrix **materializálása nélkül** számoljuk: minden elemi művelethez zárt, kis költségű VJP-képlet tartozik.

Például az $f(u) = u^\top x$ skalár művelet lokális VJP-je $J_f = x^\top$, tehát $g_u = x\, g_y$, ahol $g_y \in \mathbb{R}$ skalár gradienst szorozzuk az input vektorral.

A visszafelé haladás a veszteség csúcsánál a $g_L = \partial L/\partial L = 1$ kezdőértékkel indul, majd a topologikus sorrend megfordítottján halad. Amikor a v csúcsot feldolgozzuk, a rá érkező összes él lokális VJP-jét kiszámítjuk, és az eredményt a megfelelő u csúcs gradiens-mezőjébe **hozzáadjuk**. Az összeadás a 10.1.4 lemmából következik, és a PyTorch autograd is ezt a „grad mező +=” mechanizmust valósítja meg; a jelen könyv a [D05]-öt csak háttérként idézi, a saját kódban a gradiens-felhalmozás explicit lista alapján történik.

Elágazás (branching): amikor egy u csúcsból legalább két él indul ki, azaz u-t több leszármazott művelet használja. A gradiens-felhalmozás (gradient accumulation) ilyenkor kötelező: minden leszármazott hozzájárulását az u-hoz rendelt gradiens-mezőben összegezzük.

10.1.4 A gradiens-összegzés lemmája két ággal

Lemma (két ág gradiense összeadódik). Legyen $u \in \mathbb{R}^d$, $f_1 : \mathbb{R}^d \to \mathbb{R}^{p_1}$ és $f_2 : \mathbb{R}^d \to \mathbb{R}^{p_2}$ differenciálható, valamint $h : \mathbb{R}^{p_1+p_2} \to \mathbb{R}$ differenciálható. Definiáljuk

$$a = f_1(u), \qquad b = f_2(u), \qquad L(u) = h(a,b).$$

Ha az adott u pontban mindegyik függvény differenciálható, akkor

$$\boxed{\nabla_u L = J_{f_1}(u)^\top \nabla_a h(a,b) + J_{f_2}(u)^\top \nabla_b h(a,b).}$$

A két tagot **összeadjuk**, nem átlagoljuk, és nem választjuk ki közülük a nagyobbat. A lemmában a és b általában vektorok ($p_1, p_2 \geq 1$); a kézi példában $p_1 = p_2 = 1$, ezért ott ezek skalárok.

Bizonyítás. Definiáljuk az $F : \mathbb{R}^d \to \mathbb{R}^{p_1+p_2}$ leképezést az

$$F(u) = \begin{pmatrix} f_1(u) \\ f_2(u) \end{pmatrix}$$

oszlopvektorral. Ekkor $L(u) = (h \circ F)(u)$. A Jacobi-mátrix blokkmátrix alakú:

$$J_F(u) = \begin{pmatrix} J_{f_1}(u) \\ J_{f_2}(u) \end{pmatrix} \in \mathbb{R}^{(p_1+p_2)\times d}.$$

A többváltozós láncszabály (amelyet 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben az elemi függvényekre felépítettünk; a mátrixos alak a [B01] 5. fejezetének jelölését követi) szerint

$$J_{h\circ F}(u) = J_h(a, b)\, J_F(u) \in \mathbb{R}^{1\times d}.$$

Mivel L skalár kimenetű, a gradiense a $1 \times d$ méretű Jacobi-mátrix transzponáltja; a $(AB)^\top = B^\top A^\top$ azonossággal

$$\nabla_u L = J_{h\circ F}(u)^\top = J_F(u)^\top J_h(a, b)^\top.$$

A $J_h(a, b)$ sorvektor a p_1 és p_2 koordinátablokkok szerint $(\nabla_a h)^\top$ és $(\nabla_b h)^\top$ blokkokra bomlik, ezért

$$J_h(a, b)^\top = \begin{pmatrix} \nabla_a h \\ \nabla_b h \end{pmatrix}.$$

Behelyettesítve a blokkmátrixot:

$$\nabla_u L = \begin{pmatrix} J_{f_1}(u)^\top & J_{f_2}(u)^\top \end{pmatrix} \begin{pmatrix} \nabla_a h \\ \nabla_b h \end{pmatrix} = J_{f_1}(u)^\top \nabla_a h + J_{f_2}(u)^\top \nabla_b h.$$

Ezzel a lemma állítását megkaptuk. □

A lemma tetszőleges véges sok k ágra indukcióval terjeszthető ki. Az állítás: ha $a_i = f_i(u)$ $(i = 1, \dots, k)$ és $L = h(a_1, \dots, a_k)$, akkor $\nabla_u L = \sum_{i=1}^{k} J_{f_i}(u)^\top \nabla_{a_i} h$. Az alap $k = 1$ esetben ez a láncszabály. Az indukciós lépésben tegyük fel, hogy az állítás $k-1$ ágra igaz, és alkalmazzuk a $k = 2$ lemmát az $F_{k-1}(u) = (f_1(u), \dots, f_{k-1}(u))$ blokkra, valamint f_k-ra:

$$\nabla_u L = J_{F_{k-1}}(u)^\top \nabla_{(a_1,\dots,a_{k-1})} h + J_{f_k}(u)^\top \nabla_{a_k} h.$$

Az első tag az indukciós feltevés szerint $\sum_{i=1}^{k-1} J_{f_i}(u)^\top \nabla_{a_i} h$, amit a k-adik taggal kiegészítve megkapjuk az állítást. A $k = 2$ eset a könyv további fejezeteihez is elegendő alap. Az összegzés nem „heurisztika”, hanem a láncszabály folyománya, feltéve a sima differenciálhatóságot a vizsgált pontban. A ReLU nullpontját ezért a 9.4. Aktivációk és gradiensáramlás alfejezetben megadott konvencióval, külön kezeljük a numerikus ellenőrzés során.

10.1.5 Kézi példa: egy paraméter, két ág

Tekintsük az alábbi gráfot (szöveges ábra; a nyilak a felhasználási irányt mutatják):

```
          a = u x
u                       L = ½(a²+b²)
          b = u x
```

A gráf három csúcsból (u, a, b) és négy élből áll; a kimeneti csúcs a veszteség, amelyet a két ág közösen használ.

Legyen

$$u = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad x_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad x_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

A forward:

$$a = u^\top x_1 = 1 \cdot 1 + (-1) \cdot 2 = -1, \qquad b = u^\top x_2 = 3 - 1 = 2, \qquad L = \tfrac{1}{2}(1 + 4) = \tfrac{5}{2}.$$

Lokális VJP-k az egyes ágakon:

- a-ág: $J_{f_1} = x_1^\top$, $\nabla_a h = a = -1$, hozzájárulás $a\, x_1 = (-1) \cdot (1, 2)^\top = (-1, -2)^\top$.
- b-ág: $J_{f_2} = x_2^\top$, $\nabla_b h = b = 2$, hozzájárulás $b\, x_2 = 2 \cdot (3, 1)^\top = (6, 2)^\top$.

A lemma szerint

$$\nabla_u L = (-1, -2)^\top + (6, 2)^\top = (5, 0)^\top.$$

Ellenőrzés elemi parciális deriválással:

$$\frac{\partial L}{\partial u_1} = a\frac{\partial a}{\partial u_1} + b\frac{\partial b}{\partial u_1} = -1 \cdot 1 + 2 \cdot 3 = 5, \qquad \frac{\partial L}{\partial u_2} = -1 \cdot 2 + 2 \cdot 1 = 0.$$

Az induló becslésben kérdezett „csak az első ág”-gradiens $(-1, -2)^\top$, tehát a hiba valóban a második ág teljes hozzájárulása, $(6, 2)^\top$.

10.1.6 NumPy-labor: elágazó gráf és elhagyott gradiens

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor célja annak bemutatása, hogy a **helyes** backward egyezik a középső differenciával, a **szándékosan elhagyott** ág pedig mérhető hibát okoz. A kód a `lab/n037_graph.py` fájlból a `lab` munkakönyvtárból futtatható (`cd lab && python n037_graph.py`); a kimeneti blokk a referenciakörnyezetben futtatott kód tényleges kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál). A számítás determinisztikus, véletlen számot nem használ, ezért RNG-seedre nincs szükség.

```
# lab/n037_graph.py - munkakönyvtár: lab/
import numpy as np

u  = np.array([1.0, -1.0], dtype=np.float64)
x1 = np.array([1.0,  2.0], dtype=np.float64)
x2 = np.array([3.0,  1.0], dtype=np.float64)

def forward(u, x1, x2):
    """A 10.1.5 gráf forward-ja; visszaadja L-et és a mentett értékeket."""
    a = u @ x1
    b = u @ x2
    L = 0.5 * (a * a + b * b)
    return L, (a, b, x1, x2)

def backward_both(cache):
    """Helyes backward: mindkét ág hozzájárulását összeadjuk."""
    a, b, x1, x2 = cache
    return a * x1 + b * x2                      # dL/du

def backward_missing_second(cache):
    """Hibás kód: a második ág hozzájárulását elhagyjuk."""
    a, b, x1, x2 = cache
    return a * x1                               # hiányzik: b*x2

def central_diff(f, u, i, eps=1e-6):
    """Középső differencia az i-edik koordinátában (lépés: eps)."""
    up = u.copy(); up[i] += eps
```

```
    um = u.copy(); um[i] -= eps
    return (f(up)[0] - f(um)[0]) / (2.0 * eps)

L, cache = forward(u, x1, x2)
# A lambda rögzíti x1, x2-t, így a central_diff egy u-tól függő skalár függvényt kap.
# Az f(...)[0] indexelés azért kell, mert a forward (L, cache) párt ad vissza, nem csak L-t.
num   = np.array([central_diff(lambda v: forward(v, x1, x2), u, i)
                  for i in range(2)])
ana   = backward_both(cache)
wrong = backward_missing_second(cache)

def rel_err(a, b):
    denom = max(np.abs(a), np.abs(b), 1e-12)
    return np.abs(a - b) / denom

print("L      =", L)
print("numeric =", num)
print("analytic=", ana)
print("rel_err =", [float(rel_err(n, a)) for n, a in zip(num, ana)])
print("missing =", wrong)
print("err_miss=", wrong - ana)
assert np.isclose(L, 2.5, atol=1e-12)
assert np.allclose(num, ana, atol=1e-7)
assert np.allclose(wrong - ana, [-6.0, -2.0], atol=1e-12)
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál):

```
L      = 2.5
numeric = [5. 0.]
analytic= [5. 0.]
rel_err = [1.2667555893131066e-10, 0.0]
missing = [-1. -2.]
err_miss= [-6. -2.]
```

Az `err_miss` pontosan $-bx_2 = -(6,2)^\top$: a hiba nem zaj, hanem a hiányzó lokális VJP determinisztikus értéke. A numerikus és analitikus gradiens közötti egyezést 10^{-7} abszolút tűréssel fogadjuk el; a mért maximális abszolút eltérés $6{,}33 \cdot 10^{-10}$, ami a kerekítési hiba nagyságrendje. Mivel a loss u-ban kvadratikus, a középső differencia csonkolási hibája elvileg nulla; a megfigyelt eltérés abból adódik, hogy a loss értéke 2,5 nagyságrendű, és a $2\varepsilon = 2 \cdot 10^{-6}$ osztás felerősíti a kerekítést. A ReLU-szerű töréspont itt nem szerepel, tehát sima függvényen dolgozunk.

10.1.7 Feladatok

F1 (fogalmi, nehézség 1). Definiálja pontosan a topologikus sorrendet, és magyarázza meg, miért nem lehet irányított kör egy érvényes forward-gráfban. Adjon egy három csúcsból álló DAG-példát, amelyben a `u -> v -> w` él és a `u -> w` közvetlen él egyszerre szerepel; mondja meg, hány topologikus sorrendje van, és írja fel a lehetséges sorrendeket. *Beadandó:* definíció, indoklás, a példagráf és a sorrendek listája.

F2 (levezetési, nehézség 2). Legyen $u = (1,2,3)^\top$, $p = (1,0,0)^\top$, $q = (0,1,0)^\top$, $r = (0,0,1)^\top$, és legyen $a = u^\top p$, $b = u^\top q$, $c = u^\top r$, továbbá

$$L(u) = a^2 + \exp(b) + \cos(c).$$

Vezesse le $\nabla_u L$-et a fenti VJP-lemma három ágra alkalmazásával, és számítsa ki a megadott pontban. Ellenőrizze az alakot: a gradiens $\mathbb{R}^3$-beli. *Beadandó:* a háromágas képlet, a részleges deriváltak, a numerikus eredmény.

F3 (implementációs, nehézség 1). Egészítse ki a 10.1.6 labor kódját egy harmadik ággal: $c = 2u^\top x_3$, $L = \frac{1}{2}(a^2 + b^2 + c^2)$, ahol $x_3 = (0, 1)^\top$. Írja meg a teljes backward-ot, és ellenőrizze mindkét komponensre középső differenciával, f64, `eps=1e-6` mellett. *Beadandó:* futtatható kód, a három analitikus hozzájárulás, a numeric/analytic összehasonlítás, abszolút tolerancia 10^{-7}.

F4 (hibakeresési, nehézség 2). A 10.1.6 labor `backward_missing_second` függvénye elhagyja a második ágat. (a) Nevezze meg a hiba okát a láncszabály szintjén. (b) Mondja meg, milyen numerikus jel utal erre a hibára, ha az eredményt középső differenciával vetjük össze. (c) Írja le a javítást és a visszaellenőrzés pontos tűrését. *Beadandó:* ok, következmény, javított sor, a numerikus egyezés kritériuma.

F5 (kísérleti/transzfer, nehézség 3). Tegyük fel, hogy u helyett $U \in \mathbb{R}^{N\times d}$ batch-et használunk, és a veszteség $L = \frac{1}{N}\sum_{i=1}^{N} h(f_1(u_i), f_2(u_i))$, ahol u_i az U i-edik sora. Tervezze meg a backward lépést, és magyarázza meg, hogyan viszonyul a batch-átlag $1/N$ skálája a gráf elágazási összegzéséhez. Mi lenne a gradiens, ha a veszteséget `sum` reductionnel definiálnánk? *Beadandó:* a gradiens képlete, a reduction-választás hatásának indoklása, legalább egy numerikus kontrollterv a helyes skála ellenőrzésére.

10.1.8 Megoldások és indoklás

F1. A topologikus sorrend a csúcsok olyan $v_1, \dots, v_T$ felsorolása, amelyben minden $(v_i, v_j) \in E$ élre $i < j$. Ciklus esetén a forward-kiértékelés végtelen rekurzióba kerülne, mert egy csúcs értékének kiszámításához saját maga kellene; így a gráf aciklikussága a kiértékelhetőség feltétele. A megadott példa csúcsai $\{u, v, w\}$, élei $u \to v$, $v \to w$, $u \to w$. Egyetlen érvényes topologikus sorrend van: (u, v, w), mert u-nak v és w előtt, v-nek pedig w előtt kell állnia, és ez a két feltétel a sorrendet teljesen meghatározza.

F2. Alkalmazzuk a lemmát $k = 3$ ágra: ha $L = h(a, b, c)$, akkor

$$\nabla_u L = J_{f_1}^\top \nabla_a h + J_{f_2}^\top \nabla_b h + J_{f_3}^\top \nabla_c h.$$

Itt $J_{f_1} = p^\top$, $J_{f_2} = q^\top$, $J_{f_3} = r^\top$. A h parciálisai:

$$\nabla_a h = 2a, \qquad \nabla_b h = e^b, \qquad \nabla_c h = -\sin(c).$$

A megadott u pontban $a = 1$, $b = 2$, $c = 3$, így

$$\nabla_u L = 2 \cdot 1 \cdot p + e^2 \cdot q - \sin(3) \cdot r = (2,\ e^2,\ -\sin 3)^\top.$$

Numerikusan $e^2 \approx 7.389056$, $-\sin 3 \approx -0.141120$, tehát

$$\nabla_u L \approx (2,\ 7.389056,\ -0.141120)^\top \in \mathbb{R}^3.$$

Az alakellenőrzés: $u \in \mathbb{R}^3$, ezért a gradiensnek is három komponensűnek kell lennie; a képletben minden $J_{f_i}^\top$ egy 3×1 vektor, és a három tag összege is 3×1, ami egyezik a kiszámított $(2, e^2, -\sin 3)^\top$ vektorral.

F3. A kiegészített kód:

```
x3 = np.array([0.0, 1.0], dtype=np.float64)

def forward3(u, x1, x2, x3):
    a = u @ x1
    b = u @ x2
    c = 2.0 * (u @ x3)
    L = 0.5 * (a*a + b*b + c*c)
```

```
    return L, (a, b, c, x1, x2, x3)

def backward3(cache):
    a, b, c, x1, x2, x3 = cache
    # a c = 2*u@x3 ág Jacobi-mátrixa 2*x3^T, ezért a lokális VJP 2*c*x3
    return a*x1 + b*x2 + (2.0*c)*x3
```

Az $u = (1, -1)^\top$ pontban: $a = -1$, $b = 2$, $c = 2 \cdot (-1) = -2$. A három hozzájárulás:

$$(-1)(1, 2)^\top = (-1, -2), \qquad 2(3, 1)^\top = (6, 2), \qquad (2 \cdot (-2))(0, 1)^\top = (0, -4).$$

Összegük $(-1 + 6 + 0,\ -2 + 2 - 4) = (5, -4)^\top$. A numerikus ellenőrzés ugyanezt adja mindkét komponensre 10^{-7} abszolút tűrésen belül:

```
def central_diff3(i, eps=1e-6):
    up = u.copy(); up[i] += eps
    um = u.copy(); um[i] -= eps
    return (forward3(up, x1, x2, x3)[0] - forward3(um, x1, x2, x3)[0]) / (2.0*eps)

num3 = np.array([central_diff3(0), central_diff3(1)])
ana3 = backward3(forward3(u, x1, x2, x3)[1])
print("num3 =", num3)          # [5. -4.]
print("ana3 =", ana3)          # [5. -4.]
assert np.allclose(num3, ana3, atol=1e-7)
assert np.allclose(ana3, [5.0, -4.0], atol=1e-12)
```

A kiegészített labor futtatott ellenőrzése (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál) a 10^{-7} abszolút tűrésen belül egyező $(5, -4)^\top$ gradienset ad. A harmadik ág a második komponenst 0-ról -4-re változtatja, tehát az ellenőrzés olyan komponensben is kimutatja a hatását, ahol az a- és a b-ág hozzájárulása kioltja egymást.

F4. (a) A hiba oka: a teljes deriváltban a b-ágból származó $\nabla_b h\, J_{f_2}(u)$ tag hiányzik; a kód csak $J_{f_1}^\top \nabla_a h$-t számol. (b) A numerikus jel: a középső differenciától való eltérés pontosan $bx_2 = (6, 2)^\top$, nem pedig kerekítési zaj; a két mennyiség közötti különbség nem csökken az `eps` finomításával. (c) A javítás a `return a*x1 + b*x2` sor; a visszaellenőrzés kritériuma: mindkét komponensre

$$|\text{numeric}_i - \text{analytic}_i| \le 10^{-7}$$

f64 és `eps=1e-6` mellett, a 10.1.6 szakasz `rel_err` stabil nevezős relatív hibájával kiegészítve.

F5. (a) Algoritmus. 1. Forward: minden sorra számítsuk ki $a_i = f_1(u_i)$, $b_i = f_2(u_i)$, majd $h_i = h(a_i, b_i)$ értékét; a lokális VJP-hez mentjük a_i, b_i-t (és a művelethez szükséges egyéb köztes értékeket). 2. Visszafelé: a veszteség $L = \frac{1}{N}\sum_{i=1}^{N} h_i$ miatt a h_i-re érkező gradiens $\partial L/\partial h_i = 1/N$. 3. Soronként a két ág lokális VJP-jét összeadjuk: $\partial L/\partial u_i = \frac{1}{N}(J_{f_1}(u_i)^\top \nabla_{a_i} h_i + J_{f_2}(u_i)^\top \nabla_{b_i} h_i)$. 4. A g_{u_i} vektorokat az U mátrix megfelelő sorába írjuk; mivel U alakja $N \times d$, a $\nabla_U L$ mátrix is $N \times d$. A $1/N$ a loss-reductionből fakadó globális skála, az elágazási összeg pedig a gráf szerkezetéből fakadó hozzájárulás-összeg; a kettő szorzatként jelenik meg. Ha `sum` reduction lenne, a $1/N$ skála eltűnne, és a gradiens N-szeresére nőne; ez a batch-méret változtatásakor eltérő tanulási ráta igényét eredményezné. (b) Numerikus kontrollterv: válasszuk $N = 2$, $d = 2$ méretet, és a 10.1.5 példa kétsoros változatát ($f_1(u) = u^\top x_1$, $f_2(u) = u^\top x_2$, $h(a, b) = \frac{1}{2}(a^2 + b^2)$); a középső differenciával soronként és koordinátánként perturbálva ellenőrizzük $\nabla_U L$ minden elemét 10^{-7} abszolút tűréssel. Külön kontroll: ugyanazon az adaton `sum` reductionnel a gradiens minden eleme pontosan N-szerese a `mean` esetnek; ezt a két futás összevetésével ellenőrizzük (a terv, nem futási eredmény).

10.1.9 Tipikus tévedés, ellenpélda, önellenőrzés

Tipikus tévedés. A visszaterjesztést sokan azonosítják az optimalizáló paraméterfrissítésével. A kettő különböző: a backprop **kiszámítja** a gradienst, az optimalizáló pedig **felhasználja** azt a 6.4. Momentum és adaptív optimizálók alfejezetben definiált frissítésre. A backprop önmagában nem változtatja a paramétereket.

Ellenpélda. Legyen $w = 3$ skalár, és $L = \frac{1}{2}w^2$. A backward a $\nabla_w L = w = 3$ gradienst adja. A `backward()` hívás után w továbbra is 3; csak az optimalizáló lépése, például $w \leftarrow w - \eta \nabla_w L = 3 - 0.1 \cdot 3 = 2.7$, módosítja. Ha valaki a backward hívás után azonnal $w = 2.7$-et várna, összekeverné a gradiensszámítást a paraméterfrissítéssel. Ugyanez a hiba előfordulhat batch-átlag esetén: a gradiensben a $1/N$ skála a loss definíciójából következik, nem az optimalizáló beállításából.

PyTorch-ban ugyanez közvetlenül megfigyelhető:

```
import torch

w = torch.tensor(3.0, requires_grad=True)
L = 0.5 * w**2
L.backward()
print(w.item(), w.grad.item())    # 3.0 3.0

with torch.no_grad():
    w -= 0.1 * w.grad
print(w.item())                   # 2.7000000476837716
```

A `backward()` után a súly értéke változatlan, csak a `w.grad` mező töltődik ki; a frissítést a `torch.no_grad()` blokkban végzett kézi lépés (vagy egy optimalizáló) végzi. A kimenet a projekt venvjében (torch 2.15.0+cpu) futtatva: `3.0 3.0`, majd `2.700000047683716`; az utolsó jegyek a float32 kerekítéséből adódnak, a 2,7 valójában nem ábrázolható pontosan.

Önellenőrzés.

1. A gráfomban nincs irányított kör? A topologikus sorrendben minden él korábbi csúcsból későbbibe mutat?
2. A gradiens minden csúcsra ugyanolyan alakú, mint maga a csúcs?
3. Az elágazó csúcsnál összeadást használtam, nem pedig átlagolást vagy az egyik ág elhagyását?
4. A batch-veszteség `mean` reductionje esetén a $1/N$ megjelenik a gradiensben, míg `sum` esetén nem?
5. A numerikus gradiensellenőrzés f64, középső differencia, megadott `eps` és tűrés mellett történt; töréspontos függvényt csak sima pontban ellenőriztem?

Visszautalás. A 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetben az affin réteg lokális VJP-jeit bizonyítottuk; ott a $g \mapsto W^\top g$ lépés ugyanennek a „lokális VJP” gépezetnek egy esete. A 6.1. Objektív, reguláris tag és gradienslépés alfejezetben látott batch-átlag az itteni $1/N$ skálát adja, a 9.1. Neuron és perceptron perceptronjának score-ja pedig egyetlen gráf-él, amelyet a következő egységben kötünk össze.

10.1.10 Rövid összefoglaló

A számítási gráf egy DAG, amelynek csúcsai tenzorok, élei a felhasználási relációk. A forward a topologikus sorrendben halad, és menti a backwardhoz szükséges köztes értékeket. A visszaterjesztés a megfordított sorrendben minden élre lokális VJP-t számol, $g_u = J_f^\top g_v$ alakban. A

központi lemma azt mondja ki és bizonyítja a többváltozós láncszabályból, hogy elágazó csúcsnál a leszármazott ágak gradiensei összeadódnak. Kézzel levezettük, és a NumPy-kód futtatott kimenete is megmutatja, hogy egy ág elhagyása nem kicsi numerikus zajt, hanem a hiányzó tag pontos értékét okozza. A backprop gradiensszámítás, nem paraméterfrissítés.

10.1.11 Továbblépés és források

A következő 10.2. Affin réteg és aktiváció backward egységben az affin réteg és az elemenkénti aktiváció konkrét backward-képleteit vezetjük le ezzel a gépezettel, és az ennek során felépített VJP-készletből tanuló MLP-t építünk.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

10.2 Affín réteg és aktiváció backward

10.2.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben a 10.1. Számítási gráf és lokális derivált alfejezetben felépített visszaterjesztési gépezet két konkrét építőkövét vezetjük le: az affin réteg és a ReLU-aktiváció backwardját. Az előismeret a lokális VJP, a topologikus sorrend és a gradiensösszegzési lemma. Nem használunk sem PyTorch autogradot, sem kész optimalizálót; minden képletet NumPy-műveletként is megadunk.

Induló kérdés: egy N elemű minibatchre számolt **átlagolt** veszteség után a biasgradiens a bejövő gradiens oszlopainak *összege* vagy *átlaga*? Mielőtt tovább olvasna, becsülje meg: ha $N = 4$, és a bejövő upstream gradiens minden sora azonos, mekkora a helyes db az egyes sorok értékéhez képest?

10.2.2 Affin réteg: definíció, VJP és alakok

Az affin réteg a könyv konvenciója szerint

$$Z = XW^\top + b,$$

ahol $X \in \mathbb{R}^{N\times d}$ a batch, $W \in \mathbb{R}^{m\times d}$ a súly, $b \in \mathbb{R}^m$ a bias, $Z \in \mathbb{R}^{N\times m}$ a pre-aktiváció. Egy megfigyelés tehát az X mátrix egy sora, a b vektort a NumPy soronként szórja ki (*broadcast*).

A backward bemenete a Z-re érkező upstream gradiens $G = \nabla_Z L \in \mathbb{R}^{N\times m}$, ahol $L = \ell(Z)$ a skalár veszteség. A levezetendő gradiens alakja mindig megegyezik a paraméter vagy bemenet alakjával; ezt a 10.1. Számítási gráf és lokális derivált alfejezet lokális VJP-definíciója rögzíti ($g_u = J_f^\top g_y$), és a továbbiakban ezt nevezzük oszlop-VJP-konvenciónak.

Lemma (affin réteg VJP-je). Legyen $Z = XW^\top + b$ a fenti alakokkal, és legyen $\ell : \mathbb{R}^{N\times m} \to \mathbb{R}$ differenciálható. Ekkor $G = \nabla_Z L$ mellett

$$\nabla_W L = G^\top X, \qquad \nabla_X L = GW, \qquad \nabla_b L = \sum_{n=1}^{N} G_{n,:}.$$

Bizonyítás. A skalár láncszabályt koordinátánként alkalmazzuk. Mivel

$$Z_{nk} = \sum_{r=1}^{d} X_{nr} W_{kr} + b_k,$$

a W_{ij} szerinti parciális derivált $\partial Z_{nk}/\partial W_{ij} = \delta_{ki} X_{nj}$ (ahol δ a Kronecker-delta: $\delta_{ij} = 1$, ha $i = j$, különben 0), ezért

$$\frac{\partial L}{\partial W_{ij}} = \sum_{n=1}^{N} \sum_{k=1}^{m} G_{nk} \delta_{ki} X_{nj} = \sum_{n=1}^{N} G_{ni} X_{nj} = (G^\top X)_{ij}.$$

A biasra $\partial Z_{nk}/\partial b_j = \delta_{kj}$, így

$$\frac{\partial L}{\partial b_j} = \sum_{n=1}^{N} \sum_{k=1}^{m} G_{nk} \delta_{kj} = \sum_{n=1}^{N} G_{nj},$$

ami éppen a G oszlopösszege. Végül $\partial Z_{nk}/\partial X_{ij} = \delta_{ni} W_{kj}$, tehát

$$\frac{\partial L}{\partial X_{ij}} = \sum_{n=1}^{N} \sum_{k=1}^{m} G_{nk} \delta_{ni} W_{kj} = \sum_{k=1}^{m} G_{ik} W_{kj} = (GW)_{ij},$$

ami a három állítást bizonyítja. □

A levezetés saját kidolgozás; a mátrixalakú láncszabály a [B01] 5. fejezetének mátrixkalkulusát követi, de a konkrét képleteket ebben az alakban magunk ellenőriztük.

Alakellenőrzés: $G^\top X$ alakja $(m \times N)(N \times d) = m \times d$, egyezik W-vel; GW alakja $(N \times m)(m \times d) = N \times d$, egyezik X-szel; a biasgradiens $(m,)$. Azt is érdemes kimondani, hogy b konkrét értéke nem jelenik meg egyik gradiensben sem, mert a $+b$ transzláció Jacobi-mátrixa az identitás; a b csak a forward Z értékén keresztül hat a további rétegek gradiensére.

10.2.3 Broadcast backward és a biasgradiens összege

A $b \in \mathbb{R}^m$ a forwardban N példányban használatos: minden batchsorhoz ugyanazt a vektort adjuk. A 10.1. Számítási gráf és lokális derivált gradiensösszegzési lemma szerint a közös paraméter gradiense a különböző ágak hozzájárulásainak összege. Egy ág egy n sor, ahonnan $G_{n,:}$ érkezik, tehát

$$\nabla_b L = G^\top \mathbf{1}_N = \sum_{n=1}^{N} G_{n,:},$$

ahol $\mathbf{1}_N$ a csupa egyes oszlopvektor. Ez a `db = G.sum(axis=0)` művelet.

Általános szabályként: ha egy forward műveletben egy kisebb alakú tenzort a batchdimenzió mentén sugároztunk ki, akkor a backward bejövő gradienst a kibővített tengelyek mentén **összegezni** kell, hogy visszakapjuk az eredeti alakot. Itt a 0. tengely a kibővített batchtengely.

A **batch mean** a reduction szintjén dől el, nem a biasgradiens képletében. Ha

$$L_{\text{mean}} = \frac{1}{N} \sum_{n=1}^{N} \ell_n(Z_{n,:}), \quad \text{akkor} \quad G_{n,:} = \frac{1}{N} \nabla_z \ell_n(Z_{n,:}).$$

A $\nabla_b L$ ekkor is $\sum_n G_{n,:}$, tehát a biasgradiens automatikusan tartalmazza az $1/N$ skálát. A sum reduction esetén $G_{n,:} = \nabla_z \ell_n$, így minden gradiens pontosan N-szerese az átlagos változatnak. A képlet tehát nem változik, csak a bejövő G skálája; ezt a gyakori tévedésnél ismét elővesszük. Ezzel az induló kérdésre a válasz: a biasgradiens a G oszlopainak összege, azaz a soronkénti hozzájárulások összege; ha minden sor azonos, a helyes db az egyetlen sor értékének N-szerese, nem annak átlaga.

10.2.4 ReLU backward

A 9.2. MLP és nemlinearitás alfejezetben bevezetett ReLU elemenkénti művelet: $h(x) = \max(0, x)$. A h differenciálható minden $x \neq 0$ pontban, és

$$h'(x) = \begin{cases} 1, & x > 0, \\ 0, & x < 0. \end{cases}$$

Az $x = 0$ pontban klasszikus derivált nincs; ott a szubderivált a $[0, 1]$ intervallum. A NumPy nem definiál általános ReLU-backward konvenciót; az itt írt saját NumPy-referencia a 0 értéket választja. A PyTorch beépített ReLU-ja az általunk ellenőrzött 2.15.0 verzióban ugyancsak 0-t ad az $x = 0$ pontban; ez implementációs konvenció, nem a klasszikus derivált értéke, és nem verziófüggetlen garancia. Ha a ReLU bemenete a Z pre-aktiváció, kimenete $H = \mathrm{ReLU}(Z)$, és a kimenetre érkező gradiens $G = \nabla_H L$, akkor a backward

$$\nabla_Z L = \mathbf{1}[Z > 0] \odot G,$$

ahol $\odot$ az elemenkénti szorzat, az indikátor pedig a mentett pre-aktivációból (Z-ből) képzett maszk. A labor kódjában a bejövő gradiens neve `dH`, a képletben G; ugyanaz a mennyiség. Elemenkénti aktiváció esetén a Jacobi-mátrix diagonális, ezért a VJP éppen ez az elemenkénti szorzat.

A törésponti konvenció miatt a nulla pre-aktivációjú elemhez tartozó gradiens a fenti szabály szerint nulla. Ez nem azt jelenti, hogy a nullpontbeli derivált nulla volna, hanem hogy az optimalizáló implementáció egy szubderiváltat választ. A 10.3. Gradiensellenőrzés és numerikus hibamérce gradiensellenőrzésénél ezért mindig nullától távoli vizsgálati pontot használunk; a labor adatait úgy választjuk, hogy minden pre-aktiváció pozitív legyen, így a töréspont nem zavarja a numerikus összevetést.

10.2.5 Kézi kis példa

Legyen $N = 2$, $d = 3$, $m = 2$,

$$X = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

és érkezzen a bejövő gradiens

$$G = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}.$$

A számítási gráf szöveges ábrája (a nyilak a forward irányát mutatják; a backward ellentétes irányban halad, a Z-re érkező $G = \nabla_Z L$ gradiensből indul):

```
X
W     Z = XW  + b     L
b
```

Akkor

$$dW = G^\top X = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 8 \end{pmatrix},$$

$$db = G_{1,:} + G_{2,:} = (1+3,\ 2+4) = (4, 6),$$

$$dX = GW = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 2 \\ 3 & 10 & 4 \end{pmatrix}.$$

A dW alakja $(2, 3)$, a dX alakja $(2, 3)$, a db alakja $(2,)$; mindegyik a megfelelő paraméter alakjával egyezik. A $b = (1, 0)^\top$ értéke egyik eredményben sem szerepel, ahogy a lemma állítja.

Részletszámítás a dW-re: $(G^\top X)_{11} = 1 \cdot 1 + 3 \cdot 0 = 1$, $(G^\top X)_{12} = 1 \cdot 0 + 3 \cdot 1 = 3$, $(G^\top X)_{13} = 1 \cdot 2 + 3 \cdot 1 = 5$; a második sorban $(G^\top X)_{21} = 2 \cdot 1 + 4 \cdot 0 = 2$, $(G^\top X)_{22} = 2 \cdot 0 + 4 \cdot 1 = 4$, $(G^\top X)_{23} = 2 \cdot 2 + 4 \cdot 1 = 8$. A dX első sora: $1 \cdot 1 + 2 \cdot 0 = 1$, $1 \cdot 2 + 2 \cdot 1 = 4$, $1 \cdot 0 + 2 \cdot 1 = 2$; a második sor: $3 \cdot 1 + 4 \cdot 0 = 3$, $3 \cdot 2 + 4 \cdot 1 = 10$, $3 \cdot 0 + 4 \cdot 1 = 4$.

10.2.6 Konkrét labor: két rétegből álló NumPy-MLP

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor egy $d = 4$ bemenetű, $m = 5$ rejtett neuronú, $K = 2$ kimenetű MLP-t épít fel: affin réteg, ReLU, affin réteg, majd az $N \cdot K$ tenzorelemre átlagolt fél-MSE loss, azaz $L = \frac{1}{2NK} \sum_{n,k} (Z_{2,nk} - y_{nk})^2$. A bemenet determinisztikus, ezért nincs RNG-seed; minden tenzor `float64`, eszköz CPU. A teljes futtatható kód:

```python
# lab_n038.py
# Munkakönyvtár: a fejezet lab könyvtára.
# Futtatás: python lab_n038.py
import numpy as np

def relu(x):
    return np.maximum(0.0, x)

def linear_forward(X, W, b):
    Z = X @ W.T + b
    cache = (X, W)          # b a backwardhoz nem kell
    return Z, cache

def relu_forward(Z):
    H = relu(Z)
    cache = Z               # a maszkot a pre-aktivációból képezzük
    return H, cache

def linear_backward(G, cache):
    X, W = cache
    dW = G.T @ X
    db = G.sum(axis=0)      # biasgradiens: batch szerinti összeg
    dX = G @ W
    return dX, dW, db
```

```
def relu_backward(dH, cache):
    Z = cache
    dZ = dH * (Z > 0)       # ReLU-maszk; a nullponti konvenció 0
    return dZ

# Determinisztikus kis bemenet, float64 CPU.
X = np.array([[1.,2.,3.,4.],
              [5.,6.,7.,8.],
              [9.,10.,11.,12.]], dtype=np.float64)      # (3,4)
W1 = np.array([[0.1,0.2,0.3,0.4],
               [0.5,0.6,0.7,0.8],
               [0.9,1.0,1.1,1.2],
               [1.3,1.4,1.5,1.6],
               [1.7,1.8,1.9,2.0]], dtype=np.float64)    # (5,4)
b1 = np.array([0.01,-0.02,0.03,-0.04,0.05], dtype=np.float64)
W2 = np.array([[0.1,0.2,0.3,0.4,0.5],
               [-0.1,-0.2,0.1,0.2,0.3]], dtype=np.float64)  # (2,5)
b2 = np.array([0.0,0.1], dtype=np.float64)
y = np.array([[1.,0.],
              [0.,1.],
              [1.,1.]], dtype=np.float64)               # (3,2)

# Forward.
Z1, cache1 = linear_forward(X, W1, b1)   # (3,5)
H1, rcache = relu_forward(Z1)            # (3,5)
Z2, cache2 = linear_forward(H1, W2, b2)  # (3,2)

# Loss: batch-mean, félenkénti MSE.  L = 1/(2NK) * sum (Z2-y)^2
N, K = y.shape
L = 0.5 * np.mean((Z2 - y) ** 2)

# Backward; a mean reduction skalája az upstream G2-ben van.
G2 = (Z2 - y) / (N * K)
dH1, dW2, db2 = linear_backward(G2, cache2)
dZ1 = relu_backward(dH1, rcache)
dX, dW1, db1 = linear_backward(dZ1, cache1)

print("L=", L)
print("Z1", Z1.shape, "H1", H1.shape, "Z2", Z2.shape)
print("dW1", dW1.shape, "db1", db1.shape, "dX", dX.shape)
print("dW2", dW2.shape, "db2", db2.shape)
assert Z1.shape == (3, 5) and H1.shape == (3, 5) and Z2.shape == (3, 2)
assert dW1.shape == W1.shape and db1.shape == b1.shape and dX.shape == X.shape
assert dW2.shape == W2.shape and db2.shape == b2.shape

# Független numerikus ellenőrzés centrális differenciával; a bemeneti dX is.
def loss_of(params):
    A, c, B, d, Xv = params
    return 0.5 * np.mean((relu(Xv @ A.T + c) @ B.T + d - y) ** 2)

base_params = [W1.copy(), b1.copy(), W2.copy(), b2.copy(), X.copy()]
analytical = [dW1, db1, dW2, db2, dX]
eps = 1e-6
max_abs = 0.0
max_rel = 0.0
for p, g in zip(base_params, analytical):
    pf = p.reshape(-1)
    gf = g.reshape(-1)
    for idx in range(pf.size):
        old = pf[idx]
        pf[idx] = old - eps
        lm = loss_of(base_params)
```

```
        pf[idx] = old + eps
        lp = loss_of(base_params)
        pf[idx] = old
        num = (lp - lm) / (2.0 * eps)
        max_abs = max(max_abs, abs(num - gf[idx]))
        denom = max(abs(num), abs(gf[idx]), 1e-12)
        max_rel = max(max_rel, abs(num - gf[idx]) / denom)
print(f"max_abs_grad_err={max_abs:.3e}")
print(f"max_rel_grad_err={max_rel:.3e}")
print("max_rel_grad_err < 1e-6:", max_rel < 1e-6)
assert max_rel < 1e-6
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál; determinisztikus bemenet, ezért seed nincs):

```
L= 955.7979951666666
Z1 (3, 5) H1 (3, 5) Z2 (3, 2)
dW1 (5, 4) db1 (5,) dX (3, 4)
dW2 (2, 5) db2 (2,)
max_abs_grad_err=1.651e-07
max_rel_grad_err=2.691e-08
max_rel_grad_err < 1e-6: True
```

A fenti kód az alfejezet képleteiből összeállított, futtatásra kész munkamenet. A numerikus ellenőrzés a négy paraméter gradiensén túl a bemeneti `dX`-et is összeveti középső differenciával. A futtatás alakjai: `Z1, H1` $(3,5)$, `Z2` $(3,2)$, `dW1` $(5,4)$, `db1` $(5,)$, `dX` $(3,4)$, `dW2` $(2,5)$, `db2` $(2,)$. A választott X, W_1, b_1 mellett minden pre-aktiváció pozitív, ezért a ReLU-töréspont nem befolyásolja a numerikus gradienst. A véges differencia középső lépése $\varepsilon = 10^{-6}$. Az abszolút hiba önmagában félrevezető, mert a loss itt 10^3 nagyságrendű, ezért a 10.3. Gradiensellenőrzés és numerikus hibamérce relatív hibamércéjét használjuk: $E_{\text{rel}} = |\text{num} - g| / \max(|\text{num}|, |g|, 10^{-12})$, a feltétel pedig $E_{\text{rel}} < 10^{-6}$ sima ponton. A referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál, determinisztikus bemenet) a futtatás $L \approx 955{,}798$, $E_{\text{abs}} \approx 1{,}65 \cdot 10^{-7}$ és $E_{\text{rel}} \approx 2{,}69 \cdot 10^{-8}$ értéket adott: az abszolút hiba a 10^{-8}-as küszöböt meghaladja, a relatív nem. A beadandó eredmény a fenti kimenet és a `max_rel_grad_err < 1e-6` logikai igazságérték.

10.2.7 Feladatok

F1 (fogalmi, nehézség 1, előfeltétel: 10.1.). Magyarázza el, miért $\nabla_b L = \sum_n G_{n,:}$ és miért nem a $\frac{1}{N}\sum_n G_{n,:}$ átlag, amikor a veszteség batch-átlag. Adjon $N = 2$-re olyan G-t, amellyel a két képlet különböző eredményt ad. *Beadandó:* magyarázat, a különbség konkrét számértéke.

F2 (levezetési, nehézség 2, előfeltétel: a lemma). Legyen

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad b = (1, -1, 2, 0)^\top, \quad G = \begin{pmatrix} 2 & 1 & 0 & -1 \\ -1 & 3 & 1 & 2 \end{pmatrix}.$$

Itt $N = 2$, $d = 3$, $m = 4$, tehát a három dimenzió különbözik. Számítsa ki $\nabla_W L$, $\nabla_b L$ és $\nabla_X L$ értékét az affin VJP-lemma alapján, és ellenőrizze mindhárom alakot. Mutassa meg, hogy b értéke nem jelenik meg az eredményben. *Beadandó:* minden mátrixművelet, a három eredmény és alakellenőrzés.

F3 (implementációs, nehézség 2, előfeltétel: NumPy). Implementálja a `linear_backward` függvényt a fenti labor szerint, futtassa az F2 bemenetén, és nyomtassa ki a három gradienst. A

`linear_backward(G, cache)` a labor szerinti `cache=(X, W)` párt kapja; minden tenzor `float64`. *Beadandó:* futtatható kód, dtype és az F2-vel egyező három kimenet.

F4 (hibakeresési, nehézség 2). A következő hibás backwardot kapja:

```
def linear_backward_wrong(G, cache):
    X, W = cache
    dW = X.T @ G
    db = G.sum(axis=0)
    dX = W.T @ G
    return dX, dW, db
```

Hol a hiba, mi a következménye, és hogyan ellenőrizné a javítást? *Beadandó:* a hiba lokalizálása, a helyes két sor, valamint egy shape- vagy numerikus teszt, amely megfogja a hibát.

F5 (kísérleti/transzfer, nehézség 2). Tervezzen kis kontrollkísérletet a labor adatain: ugyanazt a forwardot, ugyanazt a lossfüggvényt, de egyszer sum, egyszer mean reductionnel. *Definiálja pontosan*, hogy a sum és a mean a teljes $N \times K$ tenzorra vagy csak a batch-tengelyre vonatkozik. A változó tényező csak a reduction; a metrika a biasgradiens sum/mean aránya. Mi a várt arány az Ön definíciója mellett, milyen numerikus toleranciát ír elő, és mi a következtetés érvényességi határa? *Beadandó:* protokoll a reduction pontos definíciójával, várt arány, tolerancia, egy érvényességi korlát.

10.2.8 Megoldások és indoklás

F1. A b vektort a forward minden batchsorhoz hozzáadja, tehát a paraméter N különböző ágon jelenik meg a számítási gráfban. A 10.1. Számítási gráf és lokális derivált gradiensösszegzési lemma szerint a közös paraméter gradiense az ágak hozzájárulásainak összege, minden ágból $G_{n,:}$ érkezik. A mean reduction nem a biasgradiens képzésénél lép be, hanem már a bejövő G skálájában: $G_{n,:} = \frac{1}{N}\nabla_z \ell_n$. Ha tehát L_{mean} gradiensét kaptuk, a további átlagolás dupla $1/N$-es skálázást adna. Legyen $N = 2$, $G = \begin{pmatrix} 2 & 4 \\ 6 & 8 \end{pmatrix}$; a helyes $\nabla_b L = (8, 12)$, a téves átlag $(4, 6)$, a kettő aránya $1/N = 1/2$.

F2. Számolás:

$$\nabla_W L = G^\top X = \begin{pmatrix} 2 & -1 \\ 1 & 3 \\ 0 & 1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} = \begin{pmatrix} 2-4 & 4-5 & 6-6 \\ 1+12 & 2+15 & 3+18 \\ 0+4 & 0+5 & 0+6 \\ -1+8 & -2+10 & -3+12 \end{pmatrix} = \begin{pmatrix} -2 & -1 & 0 \\ 13 & 17 & 21 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix},$$

$$\nabla_b L = G_{1,:} + G_{2,:} = (2-1,\ 1+3,\ 0+1,\ -1+2) = (1, 4, 1, 1),$$

$$\nabla_X L = GW = \begin{pmatrix} 2 & 1 & 0 & -1 \\ -1 & 3 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 2 & 1 \\ 0 & 7 & 1 \end{pmatrix}.$$

Az alakok sorrendben $(4, 3)$, $(4,)$, $(2, 3)$: a dW a W (m, d) alakját, a db a b $(m,)$ alakját, a dX az X (N, d) alakját követi. Mivel $\partial Z/\partial W$ és $\partial Z/\partial X$ nem függ a bias konstanstól, $b = (1, -1, 2, 0)^\top$ valóban nem jelenik meg.

F3. A labor `linear_backward` függvényét használva az F2 bemenetén:

```python
import numpy as np

def linear_backward(G, cache):
    X, W = cache
    dW = G.T @ X
    db = G.sum(axis=0)
    dX = G @ W
    return dX, dW, db

X = np.array([[1., 2., 3.], [4., 5., 6.]], dtype=np.float64)
W = np.array([[1., 0., 1.], [0., 2., 0.], [1., 1., 0.], [0., 0., 1.]], dtype=np.float64)
b = np.array([1., -1., 2., 0.], dtype=np.float64)
G = np.array([[2., 1., 0., -1.], [-1., 3., 1., 2.]], dtype=np.float64)

dX, dW, db = linear_backward(G, (X, W))
print("dW =", dW.tolist())
print("db =", db.tolist())
print("dX =", dX.tolist())
assert dW.shape == W.shape and db.shape == b.shape and dX.shape == X.shape
assert np.allclose(dW, [[-2., -1., 0.], [13., 17., 21.], [4., 5., 6.], [7., 8., 9.]], rtol=1e-9,
 ↪  atol=1e-12)
assert np.allclose(db, [1., 4., 1., 1.], rtol=1e-9, atol=1e-12)
assert np.allclose(dX, [[2., 2., 1.], [0., 7., 1.]], rtol=1e-9, atol=1e-12)
```

A futtatás kimenete: `dW = [[-2.0, -1.0, 0.0], [13.0, 17.0, 21.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]]`, `db = [1.0, 4.0, 1.0, 1.0]`, `dX = [[2.0, 2.0, 1.0], [0.0, 7.0, 1.0]]`, mindhárom float64-ben. Az egyezőséget a fenti `np.allclose` ellenőrzi `rtol=1e-9, atol=1e-12` mellett, ami szigorúbb a `np.allclose` alapértelmezett `rtol=1e-5, atol=1e-8` beállításánál.

F4. A hibás `dW = X.T @ G` a helyes irányú szorzás transzponáltja: a forward $Z = XW^\top$, ezért a súlygradiens $G^\top X$, nem $X^\top G$. A hibás `dX = W.T @ G` szintén felcseréli a sor-oszlop irányt; a lemma szerint $dX = GW$. A következmény kettős: a hibás `dW` alakja $(d, m) = (3, 4)$ lesz a helyes $(m, d) = (4, 3)$ helyett, a hibás `dX` szorzás pedig el sem végezhető, mert $W^\top$ második dimenziója $d = 3$, míg G első dimenziója $N = 2$. Ha pedig négyzetes méret esetén az alak véletlenül stimmel, a számérték akkor is hibás. A javítás:

```
dW = G.T @ X
dX = G @ W
```

Visszaellenőrzés: shape-assert mindkét értékre, majd a labor középső differenciás vizsgálata; a hibás változat a numerikus gradienssel nem egyezik, a javított relatív eltérése 10^{-6} alatt van.

F5. Protokoll: rögzített laborbemenet $(X, y, W_1, b_1, W_2, b_2)$, egyetlen változó a loss aggregálása. A reductiont előre definiálni kell. Ha a `sum` a teljes $N \times K$ tenzoron összegez, a `mean` pedig ugyanezen a tenzoron átlagol, akkor a bejövő gradiensre $G^{\text{sum}} = NK\,G^{\text{mean}}$, tehát a db_2 és a dW_2 sum/mean hányadosa pontosan $NK = 6$ a labor $N = 3$, $K = 2$ értékeivel. Ha a hallgató a `sum` alatt csak a batch-tengely menti összegzést érti, és a kimeneti dimenzió átlaga megmarad, az arány $N = 3$; a két változatot a protokollnak külön meg kell neveznie, különben a mért szám nem értelmezhető. Numerikus tolerancia: minden hányados a várt értéktől legfeljebb 10^{-12}-rel térjen el float64-ben, az osztás numerikus kerekítése miatt. Érvényességi korlát: az állítás csak a loss konstans skálázására (sum versus mean) vonatkozik; ha a reduction mellé például osztálysúlyozás vagy label smoothing is lép, a két bejövő gradiens aránya már nem konstans, hanem elemenként eltérhet.

10.2.9 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés a biasgradiens összegének és az átlagolt loss skálájának összekeverése. Legyen $N = 2$, a bejövő átlagolt gradiens $G = \begin{pmatrix} 4 & 8 \\ 8 & 12 \end{pmatrix}$. A helyes $\nabla_b L = (12, 20)$. Aki a „batch-átlag” szó miatt `G.mean(axis=0)`-t használ, $(6, 10)$-et kap. Ez nem kicsi numerikus zaj: a hibás gradiens fele a helyesnek, a relatív hiba tehát 50 % (a lépés fele akkora, mint kellene). Ha viszont a forward loss sum reductionnel készült, a bejövő gradiens már ennek megfelelően nagyobb, de a backwardban akkor is `sum(axis=0)` kell; a reduction a G előállításánál dönt el, nem a bias összegzésénél. Az ellenőrzés: $N = 1$ esetén az összeg és az átlag egybeesik, ezért az $N = 1$ kontroll éppen **nem** fedi fel ezt a hibát. Kötelező legalább egy $N \geq 2$ kontroll (a fenti példában $N = 2$), amelyben a numerikus gradiens a helyes $(12, 20)$-at adja.

10.2.10 Önellenőrzés és visszatekintés

- Milyen alakú $\nabla_X L$, ha X (N, d) és W (m, d)? **Válasz:** (N, d), mert $dX = GW$, és $(N, m)(m, d) = (N, d)$.
- Mit ad a ReLU backward a nulla pre-aktivációjú helyen a használt konvencióban, és miért nem szabad ezt a numerikus gradiensellenőrzésnél törésponton tesztelni? **Válasz:** a konvenció 0-t ad, de ez nem klasszikus derivált; a középső differencia a töréspontban nem deriváltat közelít, ezért a vizsgálati pontot nullától távol kell venni.
- A broadcastolt biasgradiens miért összeg, és hová került a batch-átlag $1/N$ skálája? **Válasz:** a bias minden batchsorban ugyanaz, ezért a gradiens az ágak összege; az $1/N$ már a bejövő G-ben van, a backward további átlagolást nem végez.
- A 6.2. Konvexitás és sima optimalizálási határ descent lemmája a helyes gradiensre támaszkodik. A téves, $1/N$-nel kisebb biasgradiens nem a veszteség uniform skálázásából származó gradiens: a dW-hez képest csak a bias komponense kisebb, ezért a lépés iránya is eltér, és a $0 < \eta < 2/L$ feltétel teljesülése sem fordítható le egy „másik L-becslésre”. Ha viszont a teljes veszteséget skáláznánk $c > 0$-val, akkor a gradiens és a gradiens Lipschitz-konstansa is c-szeresére változna, így a $\eta < 2/L$ feltétel arányosan változatlan maradna.
- A ReLU a 9.2. MLP és nemlinearitás alfejezetben bevezetett szakaszonként affin szerkezet része; itt a törésponti konvenció annak a gyakorlati következménye.

10.2.11 Rövid összefoglaló

Az affin réteg $Z = XW^\top + b$ backwardja a bejövő G mellett $dW = G^\top X$, $db = \sum_n G_{n,:}$ és $dX = GW$. A lemmát koordinátánkénti láncszabállyal bizonyítottuk, és minden alakot ellenőriztünk. A biasgradiens a broadcast miatt batch szerinti összeg; a mean loss $1/N$ skálája már a bejövő G-ben van, ezért a backward képlet reduction-független összeget használ. A ReLU backwardja a mentett pre-aktivációból képzett $Z > 0$ maszkkal szoroz, a nullpontban nulla konvencióval. A labor két rétegből álló NumPy-MLP-t épít, és minden paraméter gradiensét középső differenciával ellenőrzi 10^{-6} alatti relatív hibakorláttal.

10.2.12 Továbblépés és források

A 10.3. Gradiensellenőrzés és numerikus hibamérce egységben ezt a moduláris VJP-készletet teljes gradiensellenőrzési protokollá alakítjuk: tárgyaljuk az ε választását, az abszolút/relatív hiba mércéjét és a ReLU-töréspont kikerülését. A 10.4. Tanítási lépés és tanuló MLP ezután a tanítási lépést és egy valóban tanuló MLP-t mutat be.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.

10.3 Gradiensellenőrzés és numerikus hibamérce

10.3.1 Cél és induló kérdés

A 10.2. Affin réteg és aktiváció backward egységben levezettük a lineáris réteg és a ReLU backwardját: $dW = G^\top X$, $db = \sum_n G_{n,:}$, $dX = GW$, illetve a mentett $Z > 0$ maszkkal való szorzást. Itt azt a kérdést tesszük fel, amit minden kézzel írt backward után fel kell tenni: honnan tudjuk, hogy a levezetés nem tartalmaz előjel-, index- vagy skálahibát? A független eszköz a numerikus differenciálás. Induló becslés: helyes implementáció esetén egy sima ponton, float64 számítással és jól választott ε mellett a relatív eltérés 10^{-6} alatt várható; egy előjelhiba viszont 2 nagyságrendű relatív hibát ad, mert $a-(-a)=2a$. A ReLU töréspontja a 9.3. Expresszivitás és approximációs állítások alfejezetben tárgyalt szakaszonként affin reprezentációban is kulcsszerepet játszott; itt a gradiensellenőrzés határfeltételeként tér vissza.

10.3.2 Definíciók és hibamérték

Legyen $\theta \in \mathbb{R}^p$ a modell összes paraméterének lapított oszlopvektora, $L : \mathbb{R}^p \to \mathbb{R}$ a vizsgált veszteség. A backward $g_{\text{bwd}} \in \mathbb{R}^p$ a $\nabla_\theta L$ analitikus számítása (nem numerikus közelítés). A középső differencia (central difference) az i-edik koordinátában

$$g^{(\varepsilon)}_{\text{num},i} = \frac{L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i)}{2\varepsilon},$$

ahol $e_i \in \mathbb{R}^p$ a standard bázisvektor, $\varepsilon > 0$ a perturbáció. A forward differencia

$$g^{(\varepsilon)}_{+,i} = \frac{L(\theta + \varepsilon e_i) - L(\theta)}{\varepsilon}.$$

A két becslés közötti eltérést abszolút és relatív hibával mérjük:

$$E_{\text{abs}} = \max_i \, |g_{\text{bwd},i} - g^{(\varepsilon)}_{\text{num},i}|,$$

$$E_{\text{rel}} = \max_i \frac{|g_{\text{bwd},i} - g^{(\varepsilon)}_{\text{num},i}|}{\max(|g_{\text{bwd},i}|,\ |g^{(\varepsilon)}_{\text{num},i}|,\ \delta)},$$

ahol δ stabilizáló nevező, például 10^{-12}. Nullközeli gradiensre a relatív hiba értelmetlen, ezért ott abszolút kontrollt használunk. A tolerancia az az előre rögzített küszöb, amely alatt egy ponton az egyezést elfogadjuk; a laborban a fő küszöb $E_{\text{rel}} < 10^{-6}$, az $E_{\text{abs}} < 10^{-8}$ pedig csak egységnyi nagyságrendű loss és gradiens mellett értelmezhető, mert az abszolút küszöb nem skálafüggetlen. Ez heurisztikus protokoll, nem bizonyítás: egyetlen pont ellenőrzése nem igazolja a backward helyességét minden bemenetre. Töréspontnak (kink) nevezzük azt a pontot, ahol a függvény folytonos, de a klasszikus derivált nem létezik (a bal és jobb oldali derivált különbözhet); a ReLU nullpontja ilyen.

10.3.3 A középső differencia hibája

A Taylor-közelítésből (a [B01] 5. fejezetének Taylor-sorai nyomán) levezetjük a középső differencia előnyét.

Lemma. Legyen $f \in C^3([x_0-\varepsilon, x_0+\varepsilon])$, ahol C^k a k-szor folytonosan differenciálható függvények osztálya. Ekkor

$$|D_c f(x_0) - f'(x_0)| \le \frac{\varepsilon^2}{6} \max_{\xi\in[x_0-\varepsilon,x_0+\varepsilon]} |f'''(\xi)|,$$

ahol $D_c f(x_0) = \frac{f(x_0+\varepsilon)-f(x_0-\varepsilon)}{2\varepsilon}$. Ha $f \in C^2([x_0, x_0+\varepsilon])$, a forward differenciára

$$|D_+ f(x_0) - f'(x_0)| \le \frac{\varepsilon}{2} \max_{\xi\in[x_0,x_0+\varepsilon]} |f''(\xi)|.$$

Bizonyítás. A Taylor-tétel Lagrange-maradéktaggal:

$$f(x_0+\varepsilon) = f(x_0) + \varepsilon f'(x_0) + \frac{\varepsilon^2}{2} f''(x_0) + \frac{\varepsilon^3}{6} f'''(\xi_+),$$

$$f(x_0-\varepsilon) = f(x_0) - \varepsilon f'(x_0) + \frac{\varepsilon^2}{2} f''(x_0) - \frac{\varepsilon^3}{6} f'''(\xi_-),$$

ahol $\xi_+ \in [x_0, x_0+\varepsilon]$ és $\xi_- \in [x_0-\varepsilon, x_0]$. A két sort kivonva

$$f(x_0+\varepsilon) - f(x_0-\varepsilon) = 2\varepsilon f'(x_0) + \frac{\varepsilon^3}{6}(f'''(\xi_+) + f'''(\xi_-)).$$

2ε-nal osztva

$$D_c f(x_0) = f'(x_0) + \frac{\varepsilon^2}{12}(f'''(\xi_+) + f'''(\xi_-)).$$

A hibatag abszolút értéke legfeljebb $\frac{\varepsilon^2}{12}\cdot 2M_3 = \frac{\varepsilon^2}{6}M_3$, ahol M_3 a harmadik derivált maximuma. A forward esetben $f(x_0+\varepsilon) = f(x_0) + \varepsilon f'(x_0) + \frac{\varepsilon^2}{2} f''(\xi_+)$, tehát a hibatag $\frac{\varepsilon}{2} f''(\xi_+)$. Ezzel mindkét állítást beláttuk. $\square$

A lemma értelme: a két Taylor-sor második deriváltas tagja kiejti egymást, ezért a középső differencia $O(\varepsilon^2)$ hibájú, a forward csak $O(\varepsilon)$. Az előny ára két függvénykiértékelés egy helyett.

A kerekítési kompromisszum külön jelzendő, és nem éles tétel, hanem egyszerű hibamodell. Ha a kiértékelt loss értékek abszolút kerekítési hibája legfeljebb η, akkor a számláló hibája legfeljebb 2η, és 2ε-nal osztva η/ε járulékot kapunk. A teljes hiba hozzávetőlegesen

$$E(\varepsilon) \approx \frac{\varepsilon^2}{6} M_3 + \frac{\eta}{\varepsilon},$$

aminek minimuma $\varepsilon_* = (3\eta/M_3)^{1/3}$. Az η a loss kiszámításának abszolút kerekítési hibája: $\eta \sim u\|L\|_\infty$, ahol u az egységnyi kerekítési hiba (*unit roundoff*), float64-re $u \approx 1{,}1\cdot 10^{-16}$, float32-re $u \approx 6\cdot 10^{-8}$. Ha a loss nagyságrendileg 1 és M_3 is 1, akkor float64-ben $\varepsilon_* \sim (3\cdot 10^{-16})^{1/3} \approx 7\cdot 10^{-6}$, ezért a gyakorlati tartomány 10^{-5} és 10^{-7} között van. float32-ben $\eta \sim 10^{-7}$, így

$\varepsilon_* \sim (3 \cdot 10^{-7})^{1/3} \approx 6{,}7 \cdot 10^{-3}$, azaz a néhány ezredes tartományba tolódik; ez az egyik oka annak, hogy a gradiensellenőrzést float64-ben végezzük.

10.3.4 Kézi példa

Legyen $f(x) = x^3$, $x_0 = 2$, $\varepsilon = 10^{-2}$. A pontos derivált $f'(2) = 12$. A forward differencia:

$$D_+ = \frac{2{,}01^3 - 2^3}{0{,}01} = \frac{0{,}120601}{0{,}01} = 12{,}0601,$$

hiba 0,0601. A középső differencia:

$$D_c = \frac{2{,}01^3 - 1{,}99^3}{0{,}02} = \frac{8{,}120601 - 7{,}880599}{0{,}02} = 12{,}0001,$$

hiba 0,0001. Mivel $f''' = 6$ konstans, a lemma korlátja $\frac{\varepsilon^2}{6} \cdot 6 = \varepsilon^2 = 10^{-4}$, amit a tényleges hiba itt pontosan elér. A forward korláthoz a lemma a $[2;\ 2{,}01]$ intervallumon vett maximumot kéri: $f''(x) = 6x$, így $\max|f''| = 6 \cdot 2{,}01 = 12{,}06$, és a korlát $\frac{0{,}01}{2} \cdot 12{,}06 = 0{,}0603$. A tényleges hiba $0{,}0601 \le 0{,}0603$; a korlát érvényes, de nem éles egyenlőség. Ha a maximum helyett a derivált x_0-beli értékét ($f''(2) = 12$) írnánk, 0,06 adódna, ami **kisebb** a tényleges hibánál: a pontbeli derivált csak elsőrendű közelítést ad, nem felső korlátot. A példa mutatja a másodrendű előnyt. Ugyanez minimális kódban:

```
f = lambda x: x**3
x0, eps = 2.0, 1e-2
print(f"{(f(x0+eps)-f(x0))/eps:.6f}")          # 12.060100
print(f"{(f(x0+eps)-f(x0-eps))/(2*eps):.6f}")  # 12.000100
```

A formázott kimenet a fenti kézi számításból adódik (a kiíratás nélkül a float64 érték `12.060099999999707` és `12.000099999999847`); a kódot a labor részeként lehet futtatni.

10.3.5 Labor

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

A labor a 10.2. Affin réteg és aktiváció backward két rétegű MLP-jét használja: $d = 3$ bemenet, $h = 4$ rejtett egység, $K = 2$ kimenet, $N = 2$ batch. A vizsgálati loss $L = \frac{1}{NK} \sum_{n,k} Z_{2,nk}^2$, ahol $Z_2 = A_1 W_2^\top + b_2$. A loss Z_2 függvényeként sima, a paraméterek függvényeként azonban a ReLU miatt csak **szakaszonként** sima: ahol egy Z_1 elem előjelet vált, a gradiensnek töréspontja van. A középső differencia ezért csak akkor hasonlítható a backwardhoz, ha a $\theta \pm \varepsilon e_i$ perturbációk mellett az aktív ReLU-maszk változatlan; a labor ezt koordinátánként ellenőrzi, a töréspontot átlépő koordinátákat külön számolja, és kizárja a hibamaximumból. A labor három paraméterskála ($\sigma \in \{0{,}1;\ 1{,}0;\ 10{,}0\}$) és öt epszilon mellett hasonlítja össze a backward gradienst a numerikus középső differenciával.

```
import numpy as np

rng = np.random.default_rng(20261020)

def linear_forward(X, W, b):
    return X @ W.T + b
```

```
def linear_backward(G, X, W):
    dW = G.T @ X
    db = G.sum(axis=0)
    dX = G @ W
    return dX, dW, db

def relu_forward(Z):
    return np.maximum(0.0, Z), Z

def relu_backward(dA, cache):
    return dA * (cache > 0)

def unpack(theta, d, h, K):
    nW1, nb1, nW2 = d*h, h, h*K
    W1 = theta[:nW1].reshape(h, d)
    b1 = theta[nW1:nW1+nb1]
    W2 = theta[nW1+nb1:nW1+nb1+nW2].reshape(K, h)
    b2 = theta[nW1+nb1+nW2:]
    return W1, b1, W2, b2

def loss_and_grad(theta, X, d, h, K):
    W1, b1, W2, b2 = unpack(theta, d, h, K)
    Z1 = linear_forward(X, W1, b1)
    A1, cache = relu_forward(Z1)
    Z2 = linear_forward(A1, W2, b2)
    N, Kk = Z2.shape
    loss = np.mean(Z2**2)
    G2 = 2.0/(N*Kk) * Z2
    dA1, dW2, db2 = linear_backward(G2, A1, W2)
    dZ1 = relu_backward(dA1, cache)
    _, dW1, db1 = linear_backward(dZ1, X, W1)
    grad = np.concatenate([dW1.ravel(), db1, dW2.ravel(), db2])
    return loss, grad

def relu_mask(theta, X, d, h, K):
    W1, b1, _, _ = unpack(theta, d, h, K)
    return linear_forward(X, W1, b1) > 0          # aktív ReLU-k mintája

def numerical_grad(theta, X, d, h, K, eps):
    p = theta.size
    g = np.zeros_like(theta)
    kink = np.zeros(p, dtype=bool)                 # töréspontot átlépő koordináták
    ref = relu_mask(theta, X, d, h, K)
    for i in range(p):
        e = np.zeros_like(theta)
        e[i] = 1.0
        lp = loss_and_grad(theta + eps*e, X, d, h, K)[0]
        lm = loss_and_grad(theta - eps*e, X, d, h, K)[0]
        g[i] = (lp - lm) / (2*eps)
        kink[i] = (np.any(relu_mask(theta + eps*e, X, d, h, K) != ref)
                   or np.any(relu_mask(theta - eps*e, X, d, h, K) != ref))
    return g, kink

d, h, K, N = 3, 4, 2, 2
X = rng.normal(size=(N, d)).astype(np.float64)
epsilons = [1e-3, 1e-5, 1e-7, 1e-9, 1e-11]
for sigma in [0.1, 1.0, 10.0]:
    theta = (sigma * rng.normal(size=d*h + h + K*h + K)).astype(np.float64)
    loss, grad = loss_and_grad(theta, X, d, h, K)
    assert grad.shape == theta.shape == (d*h + h + K*h + K,)
    assert np.isfinite(grad).all()
    for eps in epsilons:
        num, kink = numerical_grad(theta, X, d, h, K, eps)
```

```
    ok = ~kink                                  # csak a sima tartományon hasonlítunk
    if not ok.any():                            # üres tömbön a np.max hibát dobna
        print(f"sigma={sigma:4.1f} eps={eps:8.0e} kink={int(kink.sum()):2d} "
              "nincs összehasonlítható koordináta")
        continue
    abs_err = np.max(np.abs(num - grad)[ok])
    rel_err = np.max((np.abs(num - grad) /
                      np.maximum(np.maximum(np.abs(num), np.abs(grad)), 1e-12))[ok])
    print(f"sigma={sigma:4.1f} eps={eps:8.0e} kink={int(kink.sum()):2d} "
          f"max_abs={abs_err:8.2e} max_rel={rel_err:8.2e}")
```

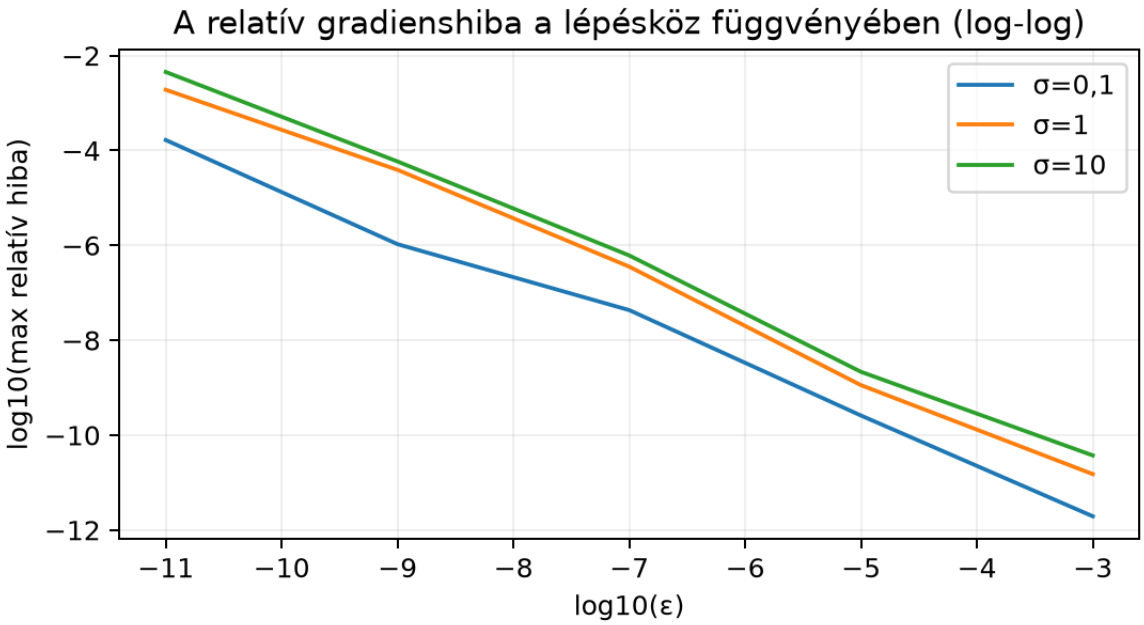
A futtatási parancs a `lab/n039_gradcheck` munkakönyvtárból:

```
python gradcheck_lab.py
```

A referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál, seed 20261020) mért kimenet:

```
sigma= 0.1 eps=   1e-03 kink= 0 max_abs=1.96e-15 max_rel=1.95e-12
sigma= 0.1 eps=   1e-05 kink= 0 max_abs=3.26e-13 max_rel=2.59e-10
sigma= 0.1 eps=   1e-07 kink= 0 max_abs=1.23e-11 max_rel=4.32e-08
sigma= 0.1 eps=   1e-09 kink= 0 max_abs=1.94e-09 max_rel=1.06e-06
sigma= 0.1 eps=   1e-11 kink= 0 max_abs=1.98e-07 max_rel=1.66e-04
sigma= 1.0 eps=   1e-03 kink= 0 max_abs=2.67e-12 max_rel=1.51e-11
sigma= 1.0 eps=   1e-05 kink= 0 max_abs=2.14e-10 max_rel=1.13e-09
sigma= 1.0 eps=   1e-07 kink= 0 max_abs=4.14e-08 max_rel=3.56e-07
sigma= 1.0 eps=   1e-09 kink= 0 max_abs=4.54e-06 max_rel=3.90e-05
sigma= 1.0 eps=   1e-11 kink= 0 max_abs=3.46e-04 max_rel=1.90e-03
sigma=10.0 eps=   1e-03 kink= 0 max_abs=1.38e-08 max_rel=3.74e-11
sigma=10.0 eps=   1e-05 kink= 0 max_abs=8.74e-07 max_rel=2.16e-09
sigma=10.0 eps=   1e-07 kink= 0 max_abs=1.18e-04 max_rel=6.12e-07
sigma=10.0 eps=   1e-09 kink= 0 max_abs=9.29e-03 max_rel=5.86e-05
sigma=10.0 eps=   1e-11 kink= 0 max_abs=1.34e+00 max_rel=4.51e-03
```

Az értelmezés: ezen a seeden egyetlen perturbáció sem lépte át a ReLU-töréspontot (`kink=0`), így minden koordináta összehasonlítható. Ebben a tartományban a hiba a legnagyobb lépésköznél, $\varepsilon = 10^{-3}$-nál a legkisebb, és ahogy ε csökken, a hiba monoton nő, nagyjából $1/\varepsilon$ arányban; ez a kerekítési tag (η/ε) hatása. Csonkolási hiba gyakorlatilag nem látszik, mert az aktív tartományon belül, **rögzített ReLU-maszk mellett** egyetlen paraméterkoordináta mentén a loss legfeljebb másodfokú polinom (Z_2 minden egyes súlyban affin, a loss Z_2-ben négyzetes), a középső differencia pedig másodfokú polinomra pontos. Ezért ezen a laboron a 10.3.3 U-alakú kompromisszuma nem látszik: a csonkolási tag gyakorlatilag nulla, a hiba monoton nő ε csökkenésével, és a vizsgált tartományban az optimum a legnagyobb $\varepsilon = 10^{-3}$. A kompromisszum bemutatásához olyan cél kellene, amelynek a harmadik deriváltja nem tűnik el (például tanh vagy köbös függvény); a labor célja itt a kerekítési ág mérése. A nagyobb paraméterskála nagyobb abszolút hibát ad a nagyobb loss és gradiens miatt; a relatív hiba lassabban nő: a $\sigma = 0{,}1$-ről 10-re váltás a legkisebb epszilonnál mintegy hússzoros relatív növekedést ad, miközben az abszolút hiba 10^{-15}-ről 10^{-8}-ra, azaz mintegy hét nagyságrenddel nő.



10.1. ábra: A relatív gradienshiba a lépésköz függvényében (log-log)

A fenti táblázat értékeiből számított log10-értékek; nem új mérés.

Az abszolút küszöb skálafüggetlenségét a táblázat konkrétan cáfolja: a $\sigma = 10$ sorokban a maximális abszolút hiba minden epszilonnál meghaladja a 10^{-8}-at ($1{,}38 \cdot 10^{-8}$-tól 1,34-ig), sőt $\sigma = 1$ esetén is a 10^{-7}-nél kisebb epszilonoknál; ezért a laborban a döntést a relatív hibára alapozzuk. A relatív hiba a gyakorlati $\varepsilon = 10^{-3}$–10^{-7} tartományban mindhárom skálán 10^{-6} alatt marad; a 10^{-9} és 10^{-11} sorokban viszont már a kerekítési tag dominál (például $\sigma = 0{,}1$-nél $1{,}06 \cdot 10^{-6}$, illetve $1{,}66 \cdot 10^{-4}$), ezért ezek a sorok a kerekítési kompromisszum szemléltetésére valók, nem elfogadási kontrollok. Más seed vagy nagyobb ε mellett megjelenhetnek töréspontot átlépő koordináták; ezeket a `kink` oszlop jelzi, és náluk a backward és a numerikus becslés eltérése helyes kód mellett is nagy lehet. Ha minden koordináta töréspontos, a labor a sort kihagyja, mert üres halmazon nem értelmezett a maximum.

10.3.6 Feladatok

F1 (fogalmi, nehézség 1). Előfeltétel: 4.1. Függvény, határérték és derivált derivált, 10.2. Affin réteg és aktiváció backward VJP-k (vector–Jacobian product, vektor–Jakobi-szorzatok). Definiálja a forward és a középső differencia gradiensbecslést, adja meg a pontossági rendjüket, és magyarázza el, miért a középső differenciát használjuk a gradiensellenőrzésnél. Beadandó: definíció, Taylor-maradéktagok, költség- és hibarend-összehasonlítás.

F2 (levezetési, nehézség 2). Előfeltétel: Taylor-tétel, trigonometrikus azonosság. Adott $f(x) = \sin x$, $x_0 = \pi/4$, $\varepsilon = 10^{-3}$. Számítsa ki a középső differenciát, a pontos $f'(x_0)$-t, a tényleges abszolút hibát, és a lemmából adódó felső korlátot. Hasonlítsa össze a forward differencia várható hibájával. Beadandó: azonosság vagy Taylor-sor, számértékek, korlát.

F3 (implementációs, nehézség 2). Előfeltétel: 10.2. Affin réteg és aktiváció backward. Írjon teljes függvényt, amely egy $d = 2$, $h = 3$, $K = 1$ MLP-re $N = 1$ batch-szel, seed 1-gyel generált float64 paraméterekkel ellenőrzi a backward gradienst középső differenciával $\varepsilon = 10^{-6}$ mellett. Beadandó: kód, futtatási parancs, a kapott max abszolút hiba és shape-ellenőrzés.

F4 (hibakeresési, nehézség 2). Előfeltétel: 10.2. Affin réteg és aktiváció backward, lebegőpontos ismeretek. Egy hallgató a gradiensellenőrzést float32 paraméterekkel, $\varepsilon = 10^{-12}$-vel, és a loss-hoz adott 10^6 konstanssal futtatja; az eltérés nagy. Mi a hiba oka, mi a következménye, és hogyan javítja? Beadandó: ok, következmény, javítás, az ellenőrző futtatás feltételei.

F5 (kísérleti, nehézség 3). Előfeltétel: 6.3. Sztochasztikus és minibatch gradiens, 10.2. Affin réteg és aktiváció backward. Tervezzen kis kísérletet annak mérésére, hogyan függ az optimális ε a paraméterskálától. Adja meg a protokollt, a baseline-t, a metrikát és az érvényességi korlátot. Beadandó: terv, döntési szabály, és indoklás, hogy a relatív hiba miért alkalmasabb a skálaösszehasonlításra.

10.3.7 Megoldások és indoklás

F1. A forward differencia $g_{+,i} = (L(\theta + \varepsilon e_i) - L(\theta))/\varepsilon$, a középső differencia a kétoldali képlet. A lemma szerint a forward hibája $O(\varepsilon)$, a középsőé $O(\varepsilon^2)$. A középső differencia nagyobb költségű: $2p$ függvénykiértékelés a forward $p+1$ helyett, de a gradiensellenőrzésnél $\varepsilon = 10^{-3}$ körül a forward hibája 10^{-4} nagyságrendű, a középsőé 10^{-7} nagyságrendű lehet (a második, illetve harmadik derivált nagyságától függően). Ezért a central a standard választás.

F2. A pontos derivált $f'(\pi/4) = \cos(\pi/4) = \sqrt{2}/2 \approx 0{,}707106781$. A $\sin(a+b) - \sin(a-b) = 2\cos a \sin b$ azonossággal

$$D_c = \frac{\sin(x_0 + \varepsilon) - \sin(x_0 - \varepsilon)}{2\varepsilon} = \cos(x_0)\frac{\sin\varepsilon}{\varepsilon}.$$

$\varepsilon = 10^{-3}$ esetén $\sin\varepsilon/\varepsilon \approx 1 - \varepsilon^2/6 = 0{,}9999998333$, így $D_c \approx 0{,}707106781 \cdot 0{,}9999998333 = 0{,}707106663$. A tényleges abszolút hiba kb. $1{,}18 \cdot 10^{-7}$. A lemma korlátja: $|f'''| = |-\cos x| \le 1$, tehát $\varepsilon^2/6 = 1{,}67 \cdot 10^{-7}$; a tényleges hiba ezen belül van. A forward differencia hibája első rendben $\frac{\varepsilon}{2}|f''(x_0)| = \frac{10^{-3}}{2}\sin(\pi/4) \approx 3{,}54 \cdot 10^{-4}$, mintegy három nagyságrenddel nagyobb.

F3. A teljes megoldás:

```
import numpy as np

rng = np.random.default_rng(1)

def linear_forward(X, W, b):
    return X @ W.T + b

def linear_backward(G, X, W):
    return G @ W, G.T @ X, G.sum(axis=0)

def relu_forward(Z):
    return np.maximum(0.0, Z), Z

def relu_backward(dA, cache):
    return dA * (cache > 0)

d, h, K, N = 2, 3, 1, 1
X = rng.normal(size=(N, d)).astype(np.float64)
p = d*h + h + K*h + K
theta = rng.normal(size=p).astype(np.float64)

def unpack(theta):
    nW1, nb1, nW2 = d*h, h, K*h
    W1 = theta[:nW1].reshape(h, d)
    b1 = theta[nW1:nW1+nb1]
```

```
    W2 = theta[nW1+nb1:nW1+nb1+nW2].reshape(K, h)
    b2 = theta[nW1+nb1+nW2:]
    return W1, b1, W2, b2

def loss_only(theta):
    W1, b1, W2, b2 = unpack(theta)
    Z1 = linear_forward(X, W1, b1)
    A1, _ = relu_forward(Z1)
    Z2 = linear_forward(A1, W2, b2)
    return float(np.mean(Z2**2))

def loss_and_grad(theta):
    W1, b1, W2, b2 = unpack(theta)
    Z1 = linear_forward(X, W1, b1)
    A1, cache = relu_forward(Z1)
    Z2 = linear_forward(A1, W2, b2)
    loss = np.mean(Z2**2)
    G2 = 2.0/(N*K) * Z2
    dA1, dW2, db2 = linear_backward(G2, A1, W2)
    dZ1 = relu_backward(dA1, cache)
    _, dW1, db1 = linear_backward(dZ1, X, W1)
    return loss, np.concatenate([dW1.ravel(), db1, dW2.ravel(), db2])

def numerical(theta, eps):
    g = np.zeros_like(theta)
    for i in range(theta.size):
        e = np.zeros_like(theta)
        e[i] = 1.0
        g[i] = (loss_only(theta + eps*e) - loss_only(theta - eps*e)) / (2*eps)
    return g

_, grad = loss_and_grad(theta)
num = numerical(theta, 1e-6)
assert grad.shape == (p,) and num.shape == (p,)
print("grad shape:", grad.shape, "p:", p)
print("max abs error:", np.max(np.abs(grad - num)))
```

Futtatás: `python f3_gradcheck.py`. A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál):

```
grad shape: (13,) p: 13
max abs error: 1.731045307096224e-10
```

A mért max abszolút hiba ezen a seeden $1{,}73 \cdot 10^{-10}$, ami nagyságrendileg a 10^{-10} körüli tartományba esik; ez nem garantált felső korlát, mert más seeddel vagy más ε-mal 10^{-9} nagyságrend is előfordulhat. A 10^{-10} nagyságrend összhangban van a 10.3.3 kerekítési becslésével: $\eta/\varepsilon \sim 10^{-16}/10^{-6} = 10^{-10}$, a csonkolási tag pedig ennél kisebb. A shape-ellenőrzés: $p = 2 \cdot 3 + 3 + 1 \cdot 3 + 1 = 13$, a gradiens vektor hossza 13.

F4. Ok: három együtt ható tényező. A float32 kerekítési egysége $u \approx 6 \cdot 10^{-8}$; a 10^6 konstans miatt a loss abszolút kerekítési hibája $\eta \sim u \cdot 10^6 \approx 6 \cdot 10^{-2}$, nagyságrendileg 0,1. Az $\varepsilon = 10^{-12}$ mellett a kerekítési járulék $\eta/\varepsilon \sim 0{,}1/10^{-12} = 10^{11}$. A 10^6 konstans a gradiensre analitikusan nincs hatással, de a numerikus kivonás abszolút hibáját a nagy függvényérték arányában növeli. Következmény: a central difference kerekítési zajt differenciál, ezért a backward és a numerikus gradiens látszólag eltér. Javítás: float64, $\varepsilon = 10^{-6}$, és a konstanst elhagyni, mert a gradiens invariáns a konstansra. A javított beállításban $\eta \sim 10^{-16}$, így $\eta/\varepsilon \sim 10^{-10}$; a csonkolási tag $\varepsilon^2 M_3/6$ nagyságrendje 10^{-13} körül van, ezért az összes hiba 10^{-10} nagyságrendűre becsülhető. Ez becslés, nem bizonyított felső korlát; a tényleges érték seed- és pontfüggő, de jellemzően 10^{-9} alatt marad.

F5. Protokoll: rögzített MLP, seed 20261020, $N = 8$ batch; paraméterskálák {0,1; 1; 10}; ε-rács 10^{-k}, $k = 2, \dots, 10$; minden kombinációban a labor central difference és backward közötti max abszolút és max relatív hiba. Két döntési szabályt rögzítünk előre: (i) az abszolút hiba minimuma az abszolút optimalizálás; (ii) a relatív hiba minimuma a skálafüggetlen összehasonlítás. A kettő eltérhet, ezért a protokoll mindkettőt külön jelenti, és nem cseréli fel őket. A baseline a backward gradiens; a kérdés a numerikus approximáció hibaprofilja, nem modellösszehasonlítás. Levezetés: a 10.3.3 modell szerint $E_{\text{abs}}(\varepsilon) \approx \frac{\varepsilon^2}{6} M_3 + \frac{\eta}{\varepsilon}$, amelynek minimuma $\varepsilon_* = (3\eta/M_3)^{1/3}$. Ha a teljes lossot egy $c > 0$ konstanssal skálázzuk, akkor $g \to cg$, $M_3 \to cM_3$ és $\eta \to c\eta$, ezért ε_* változatlan, és $E_{\text{rel}} = E_{\text{abs}}/|g|$ is változatlan. Ezért a relatív hiba konstans loss-skálázásra invariáns. Paraméterskála váltásakor azonban nem csak konstans skálázás történik: η a loss nagyságával, M_3 a harmadik deriválttal skálázódik, és ezek nem feltétlenül ugyanazzal a tényezővel, ezért a relatív optimum csak közelítőleg marad azonos; ezt a kísérlet hivatott mérni. Érvényességi korlát: az eredmény egyetlen MLP-pontra vonatkozik, nem bizonyít általános tulajdonságot, és nem helyettesíti a sztochasztikus gradiens zajának vizsgálatát a 6.3. Sztochasztikus és minibatch gradiens alfejezetből.

10.3.8 Tipikus tévedés: ReLU a töréspontban

A leggyakoribb hamis riasztás a ReLU nullpontjában adódik. Legyen $f(z) = \max(0, z)$, és a backward konvenció $f'(0) = 0$ (mint a 10.2. Affin réteg és aktiváció backward alfejezetben). A central difference $z = 0$-nál:

$$D_c = \frac{f(\varepsilon) - f(-\varepsilon)}{2\varepsilon} = \frac{\varepsilon - 0}{2\varepsilon} = \frac{1}{2}.$$

A numerikus és a választott szubgradiens eltér, miközben mindkettő a ReLU $[0, 1]$ szubgradiens-intervallumának eleme; ez nem implementációs hiba. A szubgradiens a konvex függvény deriváltjának általánosítása: olyan g, amelyre $f(z') \geq f(z) + g(z' - z)$ minden z'-re; a ReLU esetében $z = 0$-ban ez a $[0, 1]$ intervallum. A gradiensellenőrzés pontját ezért úgy kell választani, hogy minden ReLU pre-aktiváció legalább néhányszorosa legyen az ε-nak a nullától távol. Ha mégis töréspont közelében kell ellenőrizni, a törésponti konvenciót külön definiálni kell a numerikus összehasonlításhoz.

10.3.9 Önellenőrzés

1. A central difference előjelét $f(x) = x^2$, $x_0 = 1$ ellenőrzi: $((1 + \varepsilon)^2 - (1 - \varepsilon)^2)/(2\varepsilon) = 2$ minden $\varepsilon > 0$-ra, ami a pontos derivált. Ha a számlálóban a kivonás sorrendje felcserélődne, -2 lenne.
2. A labor $G_2 = \frac{2}{NK} Z_2$ tényezője a mean-reduction $1/(NK)$ szorzójából és a Z_2^2 deriváltjából származó 2-ből áll; a linear backward $dW = G^\top X$ ezután reduction-független, mert az összegzés explicit.
3. A gradiens vektor hossza $dh + h + Kh + K$; a laborban $3 \cdot 4 + 4 + 2 \cdot 4 + 2 = 26$. A `ravel` a tömböt C-sorrendben egydimenziós vektorrá alakítja (lapítás); az `unpack` ugyanezt a sorrendet követi, ezért a numerikus perturbáció ugyanahhoz a koordinátához tartozik.
4. A relatív hiba stabil nevezője nullgradiens esetén nem robban: ha $|g_{\text{bwd}}| = |g_{\text{num}}| = 0$, akkor $\delta = 10^{-12}$ miatt a tört $0/10^{-12} = 0$.

10.3.10 Rövid összefoglaló

A középső differencia a gradiens független numerikus becslése, amelynek csonkolási hibája $O(\varepsilon^2)$, szemben a forward differencia $O(\varepsilon)$ hibájával. A Taylor-tétel Lagrange-maradéktagjával levezettük a felső korlátot. Az epszilon választása kompromisszum: kicsi epszilonnál a kerekítési, nagynál a csonkolási hiba nő; float64-ben a gyakorlati tartomány 10^{-5}–10^{-7}. Az abszolút és relatív hibát stabil nevezővel mérjük. A ReLU töréspontjában a numerikus differencia eltérhet a választott szubgradienstől, ezért sima pontot kell választani. A labor több epszilon és paraméterskála mellett ellenőrzi a backwardot; a kimenet a megadott kóddal futtatott mért eredmény, amely ezen a másodfokú célon a kerekítési ágat mutatja; a csonkolási ág nem jelenik meg, a kompromisszumot a 10.3.3 elméleti modellje írja le.

10.3.11 Továbblépés és források

A 10.4. Tanítási lépés és tanuló MLP egységben a most ellenőrzött backwardot használjuk a tanítási lépéshez: $\theta \leftarrow \theta - \alpha g_{\text{bwd}}$. A numerikus ellenőrzés nem helyettesíti a tanulási konvergencia vizsgálatát, de bizalmat ad a lépés gradienséhez. A PyTorch saját `torch.autograd.gradcheck` függvénye ugyanezt a numerikus ellenőrzést végzi el tenzoros függvényeken ([D06]); az automatikus differenciálás működését a [D05] foglalja össze.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.
- [D06] PyTorch contributors (2026). *Gradcheck mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/gradcheck.html. Hozzáférés: 2026-10-30.

10.4 Tanítási lépés és tanuló MLP

10.4.1 Cél, előismeret és induló becslés

A 10.3. Gradiensellenőrzés és numerikus hibamérce egységben a backward lépést független numerikus differenciával ellenőriztük. Most ebből a lépésből tanítási hurkot építünk: a modell paramétereit ismételten frissítjük a minibatch gradiens alapján. A cél a három művelet pontos szétválasztása: **gradienstörlés**, **backpropagation** és **optimizerlépés**. A fogalom az autograd-keretrendszerek viselkedését is előkészíti, de itt még kézi NumPy-kódban dolgozunk.

Előismeret: a gradiens definíciója 4.2. Parciális derivált és gradiens, a minibatch gradiens mint becslés 6.3. Sztochasztikus és minibatch gradiens, a momentumos és adaptív frissítési szabályok 6.4. Momentum és adaptív optimizálók, valamint az aktivációk lokális deriváltjai 9.4. Aktivációk és gradiensáramlás. A backward képleteket a 10.2. Affin réteg és aktiváció backward alfejezetből ismertnek tekintjük.

Induló kérdés. Legyen $L(w) = \frac{1}{2}((w-1)^2 + (w-3)^2)$ két megfigyelés átlagos MSE-vesztesége, $w_0 = 0$ és $\alpha = 0.5$. Mennyi lesz w_1 egy gradienslépés után, és hogyan változik a veszteség? A választ a 10.4.3 szakaszban vezetjük le; a levezetés előtt érdemes saját becslést adni, majd összevetni az eredménnyel.

10.4.2 A tanítási lépés objektumai

Legyen $\theta \in \mathbb{R}^p$ a modell összes paraméterének egyetlen vektorba fűzött vektora; a laborbeli MLP-nél $\theta = (\mathrm{vec}(W_1), b_1, \mathrm{vec}(W_2), b_2)$, ahol vec a mátrix soronkénti kifeszítését jelöli, így $p = hd + h + h + 1$ (a laborban $d = 2$, $h = 16$, ezért $p = 32 + 16 + 16 + 1 = 65$). Legyen $\alpha > 0$ a tanulási ráta (a kódban `lr`), és $T \in \mathbb{N}$ az optimizerlépések száma. A tanítóhalmaz $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{N}$, a holdout (ellenőrző halmaz) D_{val} a tanításból kizárt, független ellenőrző adat. A szövegben a holdout loss, a validációs loss és a kód `val` előtagja ugyanezt a mennyiséget jelöli: a D_{val} adaton mért loss-t.

Definíció (minibatch veszteség). Egy $B \subseteq \{1, \dots, N\}$ indexhalmazhoz, $M = |B|$, a minibatch veszteség

$$L_B(\theta) = \frac{1}{M} \sum_{i \in B} \ell(f_\theta(x_i), y_i),$$

ahol ℓ az egyedi példa vesztesége, f_θ a modell előrejelzése. A teljes batch a $B = \{1, \dots, N\}$ speciális eset.

Egy epoch a tanítóhalmaz egyszeri bejárása. M méretű minibatch mellett epochonként $\lceil N/M \rceil$ lépés történik, ezért $T = (\text{az epochok száma}) \cdot \lceil N/M \rceil$; a laborban $N = 200$, $M = 16$ és 300 epoch mellett $T = 300 \cdot 13 = 3900$.

Definíció (zero gradient). Az autograd-keretrendszerekben minden paraméterhez egy gradiens-tároló tartozik ([D05]); a `.backward()` hívás az új gradienst a tárolóhoz *hozzáadja*. A `zero_grad()` ezt a tárolót nullázza a backward előtt. A kézi NumPy-kódban ilyen tároló nincs: a backward függvény közvetlenül visszaadott gradienstömbjeit a lépés végén eldobjuk, így ott a „törlés” a felülírásban valósul meg.

Definíció (optimizerlépés). A sima SGD-frissítés

$$\theta_{t+1} = \theta_t - \alpha g_t, \qquad g_t = \nabla_\theta L_B(\theta_t).$$

Momentumos vagy adaptív szabály esetén az optimizer állapotváltozókat is frissít; ezek a 6.4. Momentum és adaptív optimizálók alfejezetben tárgyalt m_t, v_t mennyiségek: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ a momentum puffere, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ az Adam második momentuma, ahol g_t^2 komponensenként értendő, $\beta_1, \beta_2 \in [0, 1)$ pedig az optimizer csillapítási paraméterei. A teljes frissítési szabályt a 6.4. alfejezet adja.

Definíció (loss history). A tanítás során feljegyzett $\hat{L}_t$ értékek sorozata. Ez lehet minibatch loss, epoch-átlagolt tanítási loss vagy holdout loss; a három nem felcserélhető, mert a minibatch loss fluktuál, a holdout loss pedig a tanításból kizárt adaton mért kiértékelő metrika, nem a tanítás célfüggvénye.

Definíció (holdout és small overfit). A holdout a tanításban nem látott adat; a tanítási és holdout loss együttes görbéje a túlillesztés alapdiagnózisa. Ha a tanítási loss közel nullára csökken, miközben a holdout loss nő, akkor a modell az empirikus kockázatot csökkenti ugyan, de a tanítóhalmaz sajátosságait memorizálja. A *small overfit* (kis tanítóhalmazra szűkített túlillesztési próba) ilyen ellenőrzés kis tanítóhalmazon: a modellnek képesnek kell lennie a train loss jelentős csökkentésére, különben a tanítási lépés vagy a kapacitás hibás.

10.4.3 Levezetési kötelezettség: mi változik egy minibatch frissítés előtt és után

Először az általános állítás ([B01] 5. fejezetének gradiensfogalma nyomán), majd a konkrét számpélda.

Lemma (minibatch gradiens és frissítés). Ha minden $\ell_i(\theta) = \ell(f_\theta(x_i), y_i)$ differenciálható a θ pontban, akkor

$$\nabla_\theta L_B(\theta) = \frac{1}{M} \sum_{i \in B} \nabla_\theta \ell_i(\theta).$$

Az SGD-frissítés után

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta L_B(\theta_t).$$

Bizonyítás. Az átlag konstans szorzó, a deriválás pedig lineáris: véges összeg deriváltja a deriváltak összege (a 4.2. Parciális derivált és gradiens alfejezet gradiensdefiníciója szerint). A frissítési képlet a gradienslépés definíciója. □

Ez a lemma rögzíti a **skálát**: a minibatch gradiens az egyedi gradiensek átlaga, nem összege. A labor kódban a $\delta^{[2]} = \frac{2}{M}(\hat{y} - y)$ tényező (a kódban `dL_dZ2`) pontosan ezt hordozza.

Most a konkrét minibatch. Legyen $M = 2$, a modell $\hat{y} = w$, célok $y_1 = 1$, $y_2 = 3$. Az átlagos MSE

$$L_B(w) = \frac{1}{2}((w-1)^2 + (w-3)^2).$$

A gradiens tagonként:

$$\frac{\partial L_B}{\partial w} = \frac{1}{2}(2(w-1) + 2(w-3)) = 2w - 4.$$

$w_0 = 0$ esetén $g_0 = -4$, tehát a frissítés

$$w_1 = w_0 - \alpha g_0 = 0 - 0.5 \cdot (-4) = 2.$$

A veszteség előtte és utána:

$$L_B(w_0) = \frac{1}{2}(1 + 9) = 5, \qquad L_B(w_1) = \frac{1}{2}(1 + 1) = 1.$$

A frissítés előtti és utáni állapot:

Mennyiség	Frissítés előtt	Frissítés után
w	0	2
$g_B(w) = 2w - 4$	-4	0
$L_B(w)$	5	1

A frissítés *utáni* gradiens általában nem nulla: például $L(w) = \frac{1}{2}w^2$, $w_0 = 1$, $\alpha = 0{,}1$ mellett $w_1 = 0{,}9$ és $g_1 = 0{,}9$. A fenti példában a gradiens mégis pontosan nulla, mert $\alpha = 0{,}5 = 1/L''$

($L'' = 2$) mellett a kvadratikus feladaton a lépés éppen egy Newton-lépés, amely egy lépésben a minimumba visz. Ez nem általános tulajdonság, hanem a konkrét kvadratikus feladat és a lépésköz együttese.

10.4.4 Kézi kis példa: háromlépéses frissítés

Ugyanazon a feladaton $\alpha = 0.25$ mellett:

1. $w_0 = 0$, $g_0 = -4$, $w_1 = 0 + 1 = 1$;
2. $w_1 = 1$, $g_1 = 2 - 4 = -2$, $w_2 = 1 + 0.5 = 1.5$;
3. $w_2 = 1.5$, $g_2 = 3 - 4 = -1$, $w_3 = 1.5 + 0.25 = 1.75$.

A veszteségek: $L_B(0) = 5$, $L_B(1) = \frac{1}{2}(0 + 4) = 2$, $L_B(1.5) = \frac{1}{2}(0.25 + 2.25) = 1.25$, $L_B(1.75) = \frac{1}{2}(0.5625 + 1.5625) = 1.0625$. A loss history monoton csökken, mert a célfüggvény konvex kvadratikus, és a lépésköz kielégíti a $0 < \alpha < 2/L'' = 1$ feltételt ($\alpha = 0{,}25$); a feltétel nélkül a monotonitás nem garantált, és mély hálókra sem általános.

10.4.5 Labor: kis MLP szintetikus regresszión

Előfeltétel. Python 3.12 és a `cnn-labor` csomag (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja.

Protokoll. Kérdés: a kézzel írt minibatch SGD csökkenti-e a holdout veszteséget egy nemlineáris regressziós feladaton? Adat: eredeti szintetikus minta, $x \in \mathbb{R}^2$, $y = \sin(2x_1) + 0.5x_2^2 + \varepsilon$, $\varepsilon \sim N(0, 0.05^2)$. Split: $N_{\text{train}} = 200$, $N_{\text{val}} = 50$. Modell: $d = 2$, rejtett szélesség $h = 16$, tanh aktiváció, lineáris kimenet. Loss: átlagos MSE. Optimalizáló: minibatch SGD, $\alpha = 0.01$, $M = 16$, 300 epoch. A paramétereket `float64`-ben tároljuk, CPU-n futunk. Seedek: adat `4201`, keverés `4202`, inicializálás `4203`. A *seed* (véletlenmag) a pszeudovéletlen generátor kezdőállapotát meghatározó egész szám: ugyanazzal a seeddel és ugyanazzal a hívási sorrenddel a generált sorozat megismételhető. A seed rögzítése nem teszi az eredményt seedfüggetlenné; a véletlen hatásának becsléséhez több seed kell.

A teljes kód egyetlen ellenőrizhető munkamenetben, a `lab/n040_train_mlp.py` fájlba mentve, a `lab` munkakönyvtárból futtatható:

```
# lab/n040_train_mlp.py - munkakönyvtár: lab/
"""n040 labor: kézi MLP tanítás minibatch SGD-vel, holdouttal."""
import numpy as np

# --- adat: eredeti szintetikus regresszió, seed rögzítve ---
rng_data = np.random.default_rng(4201)
N_train, N_val, d, h = 200, 50, 2, 16
X_train = rng_data.uniform(-2.0, 2.0, size=(N_train, d))
y_train = np.sin(2.0 * X_train[:, 0:1]) + 0.5 * X_train[:, 1:2]**2
y_train = y_train + rng_data.normal(0.0, 0.05, size=y_train.shape)
X_val = rng_data.uniform(-2.0, 2.0, size=(N_val, d))
y_val = np.sin(2.0 * X_val[:, 0:1]) + 0.5 * X_val[:, 1:2]**2
y_val = y_val + rng_data.normal(0.0, 0.05, size=y_val.shape)

# --- inicializálás ---
rng_init = np.random.default_rng(4203)
W1 = rng_init.normal(0.0, 0.3, size=(h, d))  # (h,d), a konvenció szerint
b1 = np.zeros(h)
W2 = rng_init.normal(0.0, 0.3, size=(1, h))  # (1,h)
b2 = np.zeros(1)
assert X_train.shape == (N_train, d) and y_train.shape == (N_train, 1)
```

```
assert W1.shape == (h, d) and W2.shape == (1, h)

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1.T + b1            # (M,h)
    A1 = np.tanh(Z1)
    Yhat = A1 @ W2.T + b2         # (M,1)
    return Yhat, A1

def forward_loss(X, Y, W1, b1, W2, b2):
    Yhat, _ = forward(X, W1, b1, W2, b2)
    return np.mean((Yhat - Y)**2)

def backward(X, Y, W1, b1, W2, b2):
    M = X.shape[0]
    Yhat, A1 = forward(X, W1, b1, W2, b2)
    dL_dZ2 = 2.0 / M * (Yhat - Y)       # (M,1), az átlag deriváltja
    dW2 = dL_dZ2.T @ A1                 # (1,h)
    db2 = dL_dZ2.sum(axis=0)            # (1,)
    dA1 = dL_dZ2 @ W2                   # (M,h)
    dZ1 = dA1 * (1.0 - A1**2)           # tanh lokális derivált
    dW1 = dZ1.T @ X                     # (h,d)
    db1 = dZ1.sum(axis=0)               # (h,)
    loss = np.mean((Yhat - Y)**2)
    assert Yhat.shape == Y.shape
    return loss, dW1, db1, dW2, db2

# --- tanítás: epochonként keverés, minibatch-ekre osztás ---
rng_train = np.random.default_rng(4202)
epochs, batch_size, lr = 300, 16, 0.01
train_hist, val_hist = [], []
for epoch in range(epochs):
    perm = rng_train.permutation(N_train)
    X_shuf, y_shuf = X_train[perm], y_train[perm]
    epoch_loss = 0.0
    for start in range(0, N_train, batch_size):
        idx = slice(start, start + batch_size)
        Xb, yb = X_shuf[idx], y_shuf[idx]
        loss, dW1, db1, dW2, db2 = backward(Xb, yb, W1, b1, W2, b2)
        # optimizerlépés: paraméterfrissítés, nincs akkumulátor, ezért nincs zero_grad
        W1 -= lr * dW1
        b1 -= lr * db1
        W2 -= lr * dW2
        b2 -= lr * db2
        # az utolsó batch 8 elemű lehet, ezért a batch-loss-t a méretével súlyozzuk
        epoch_loss += loss * yb.shape[0]
    train_hist.append(epoch_loss / N_train)
    val_hist.append(forward_loss(X_val, y_val, W1, b1, W2, b2))
    if epoch % 50 == 0 or epoch == epochs - 1:
        print(f"epoch {epoch:3d}  train={train_hist[-1]:.4f}  val={val_hist[-1]:.4f}")

print(f"final W1 norm: {np.linalg.norm(W1):.3f}, W2 norm: {np.linalg.norm(W2):.3f}")
assert np.isfinite(val_hist).all()
assert train_hist[-1] < train_hist[0]
assert val_hist[-1] > train_hist[-1]
```

A backwardban szereplő $1 - A_1^2$ tényező a tanh lokális deriváltja, nem külön memorizálandó szabály. Mivel $\tanh z = \sinh z / \cosh z$, a hányados deriválási szabályával és a $\cosh^2 z - \sinh^2 z = 1$ azonossággal

$$\frac{\mathrm{d}}{\mathrm{d}z} \tanh z = \frac{\cosh z \cosh z - \sinh z \sinh z}{\cosh^2 z} = \frac{\cosh^2 z - \sinh^2 z}{\cosh^2 z} = \frac{1}{\cosh^2 z} = 1 - \tanh^2 z,$$

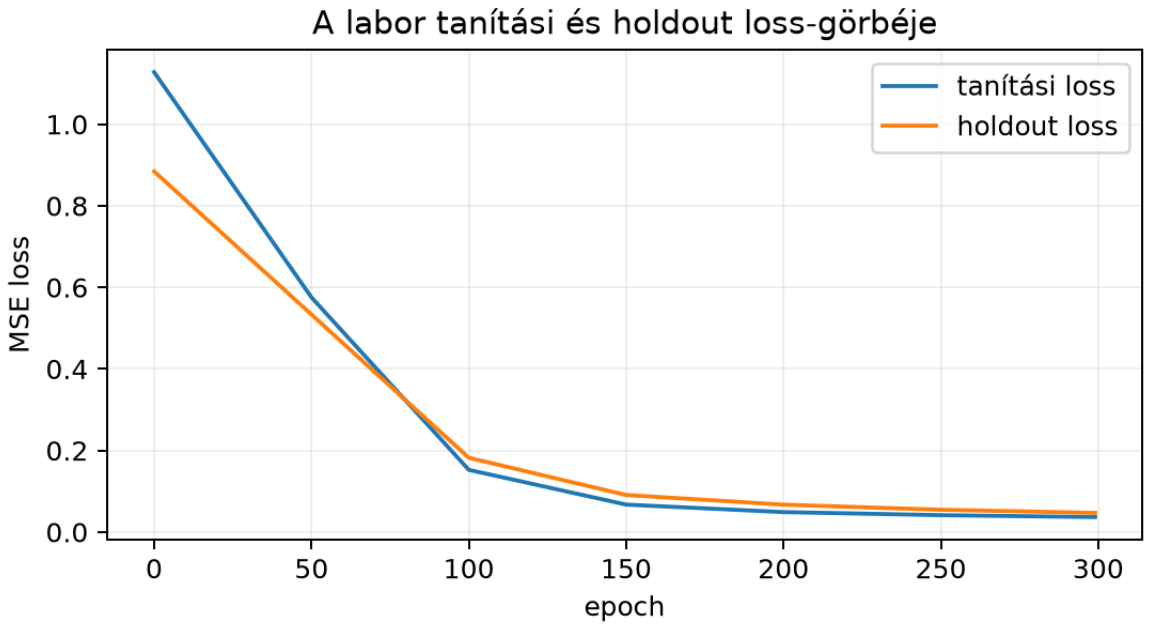
ezért $A_1 = \tanh(Z_1)$ mellett a Z_1 szerinti lokális derivált $1 - A_1^2$.

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál; seedek: adat 4201, keverés 4202, inicializálás 4203):

```
epoch   0  train=1.1278  val=0.8844
epoch  50  train=0.5755  val=0.5335
epoch 100  train=0.1525  val=0.1821
epoch 150  train=0.0673  val=0.0908
epoch 200  train=0.0489  val=0.0672
epoch 250  train=0.0414  val=0.0547
epoch 299  train=0.0367  val=0.0470
final W1 norm: 2.748, W2 norm: 3.525
```

Futtatás a `lab` munkakönyvtárból: `python n040_train_mlp.py`. A három seed rögzített (adat 4201, keverés 4202, inicializálás 4203); a fenti blokk a mért kimenetet rögzíti. A rögzített seedek ebben az azonos verziójú, egyszálas CPU-környezetben reprodukálják a kimenetet; NumPy-verziók vagy platformok között a bitazonosság nem garantált. A referenciakörnyezetben futtatva (Python 3.12.3, NumPy 2.5.4, CPU, 1 szál) a train loss az első epochban 1,128 volt, a 100. epochra 0,153-ra, a 300. epochra 0,0367-ra csökkent; a holdout loss 0,884-ről indult, a 100. epochban 0,182 volt, a végére 0,0470-re csökkent. Az első 69 epochban a holdout loss nem haladta meg a train losst, a 69. epochtól a végéig felette maradt. Tartós emelkedés nem jelentkezett, a görbe késői szakasza lapos.

A `train_hist` értéke az epoch közbeni, változó paraméterekkel számolt minibatch loss-ok mintaszám-súlyozott átlaga, nem az epoch végi modell train loss-a; a `val_hist` ezzel szemben az epoch végi paraméterekkel, egyetlen kiértékeléssel számolt holdout loss. A két görbe ezért csak közelítőleg hasonlítható össze, és a köztük lévő kis eltérés nem önmagában általánosítási mutató. A mért görbéket a következő ábra mutatja.



10.2. ábra: A labor tanítási és holdout loss-görbéje

A 10.4.5 labor kódjának futtatott kimenete (Python 3.12.3, NumPy 2.5.4, seedek 4201/4202/4203); a pontok az epoch 0, 50, …, 299 kiírt értékei. A tanítási loss epoch-átlag, a holdout loss epoch végi kiértékelés.

Eredményértelmezés. A train loss csökkenése összhangban van a frissítési lépés működésével; a holdout loss csökkenése arra utal, hogy ezen az egy spliten és seedhármason a csökkenés nem kizárólag a tanítóhalmaz memorizálása. Egyetlen split és seed nem bizonyíték: az ismételt splitű és seedű kontrollt a 12.3. Sanity check és kis mintára túllillesztés alfejezet végzi el. Ha a holdout loss a train loss csökkenése ellenére tartósan nő, akkor túllillesztés (*overfitting*) jele áll fenn; a *small overfit* kontroll (a holdout és small overfit definíciója) más kérdést vizsgál, azt, hogy a modell a kis tanítóhalmaz loss-ját egyáltalán képes-e jelentősen csökkenteni.

10.4.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért kell a gradiens-tárolót nullázni az autograd-keretrendszerekben a backward előtt, és miért nincs erre külön utasítás a fenti NumPy-kódban. Térjen ki a `+=` akkumuláció következményére.

F2 (levezetési, nehézség 1). Legyen $M = 3$, célok $y = (1, 2, 4)$, modell $\hat{y} = w$, veszteség $L(w) = \frac{1}{M}\sum_{i=1}^{3}(y_i - w)^2$, $w_0 = 0$, $\alpha = 0.25$. Számítsa ki g_0, w_1, $L(w_0)$ és $L(w_1)$ pontos értékét. Ellenőrizze, hogy $L(w_1) < L(w_0)$.

F3 (implementációs, nehézség 2). Írjon `train_step(W1, b1, W2, b2, Xb, yb, lr)` függvényt, amely egy minibatchre elvégzi a forward/backward lépést, frissíti a paramétereket, és visszaadja az új paramétereket, valamint a frissítés **előtti** minibatch loss-t. Tesztelje a labor adatának egy $M = 16$ méretű minibatchén úgy, hogy a visszaadott paraméterekkel újra kiszámítja a loss-t, és ellenőrzi, hogy az kisebb-e a visszaadott loss-nál.

F4 (hibakeresési, nehézség 2). A labor kód egy változata a holdout (validációs) loss számításához tévedésből a `X_train, y_train` tömböket adja át a `forward_loss` függvénynek:

```
# Szándékosan hibás részlet: a labor tanítási hurkának holdout-ága
for epoch in range(epochs):
    # ... a keverés és a minibatch-frissítések változatlanok ...
    val_hist.append(forward_loss(X_train, y_train, W1, b1, W2, b2))  # HIBÁS
```

Lokalizálja a hibát, magyarázza meg a következményt, adja meg a javítást, és írja le, milyen numerikus viszony várható a javítás után a train és a holdout loss között.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a minibatch méret hatásának vizsgálatára a labor feladatán: $M \in \{1, 8, 64\}$. Adja meg a rögzített és változó tényezőket, a metrikát, a baseline-t, az ismétlések és seedek kezelését, valamint a következtetés érvényességi korlátját. Előre megadott győztest ne állítson.

10.4.7 Megoldások és indoklás

F1. Az autograd `.backward()` a láncszabályból kapott gradienst a paraméter `.grad` mezőjéhez **hozzáadja**, nem felülírja. Ha az előző lépés gradiense ott maradt, akkor a frissítés $g_t + g_{t-1}$ irányába történik, ami hibás lépésközt ad. A `zero_grad()` ezt a mezőt nullázza. A NumPy-kódban a `backward` függvény új tömböket ad vissza; a `dW1 = ...` értékadás felülírja a változót, és a frissítés után nem tárolunk gradienst. Az akkumuláció tehát nem jön létre; a „zero gradient” a felülírásban implicit.

F2. A veszteség $L(w) = \frac{1}{3}((1-w)^2 + (2-w)^2 + (4-w)^2)$. Deriválás:

$$L'(w) = \frac{1}{3}(2(w-1) + 2(w-2) + 2(w-4)) = \frac{2}{3}(3w - 7) = 2w - \frac{14}{3}.$$

$w_0 = 0$: $g_0 = -\frac{14}{3}$, $w_1 = 0 - 0.25 \cdot (-\frac{14}{3}) = \frac{14}{12} = \frac{7}{6}$. A veszteségek: $L(w_0) = \frac{1}{3}(1 + 4 + 16) = 7$; $L(w_1) = \frac{1}{3}((-\frac{1}{6})^2 + (\frac{5}{6})^2 + (\frac{17}{6})^2) = \frac{315}{108} = \frac{35}{12} \approx 2.92 < 7$. Alakellenőrzés: minden mennyiség skalár; a gradiens dimenziója megegyezik w dimenziójával.

F3. Teljes függvény:

```
def train_step(W1, b1, W2, b2, Xb, yb, lr):
    """Egy minibatch lépés; visszaadja az új paramétereket és a régi loss-t."""
    loss, dW1, db1, dW2, db2 = backward(Xb, yb, W1, b1, W2, b2)
    W1_new = W1 - lr * dW1
    b1_new = b1 - lr * db1
    W2_new = W2 - lr * dW2
    b2_new = b2 - lr * db2
    return W1_new, b1_new, W2_new, b2_new, loss
```

Teszt a labor inicializálásával, a 4202-es keverési seed első permutációjának első 16 indexén, a tanítási hurok előtt:

```
perm = np.random.default_rng(4202).permutation(N_train)
Xb, yb = X_train[perm[:16]], y_train[perm[:16]]
loss_before = forward_loss(Xb, yb, W1, b1, W2, b2)
W1_new, b1_new, W2_new, b2_new, loss_returned = train_step(
    W1, b1, W2, b2, Xb, yb, 0.01)
loss_after = forward_loss(Xb, yb, W1_new, b1_new, W2_new, b2_new)
assert loss_returned == loss_before
assert loss_after < loss_before
```

A futtatott kimenet: `loss_before = 1.0196202076744094`, `loss_after = 0.9826306153295856`, a csökkenés `0.036989592344823796`; a visszaadott loss bitre egyezik a lépés előtti értékkel.

A csökkenés feltétel nélkül nem garantált, de nem is pusztán „tipikus”: ha a veszteség gradiensének Lipschitz-állandója a vizsgált tartományon Λ, akkor $0 < \alpha < 2/\Lambda$ mellett a gradienslépés csökkenti a veszteséget, mert

$$L(\theta - \alpha g) \le L(\theta) - \alpha(1 - \alpha\Lambda/2)\|g\|^2.$$

A labor MSE-vesztesége a paraméterek sima függvénye, ezért a szóban forgó pont környezetében létezik ilyen Λ; a konkrét lépésnél a fenti teszt ellenőrzi a csökkenést. Nagyobb α vagy nem sima veszteség esetén a monotonitás nem garantált.

F4. A hiba a holdout loss számításánál a bemeneti tömbökben van: a `forward_loss(X_train, y_train, ...)` hívás a tanítóhalmazt méri újra. A `val_hist` így nem holdout loss, hanem az epoch végi paraméterekkel a tanítóhalmazon mért loss; ez közel követi a `train_hist` görbét, de nem azonos vele, mert a `train_hist` epoch közbeni, változó paraméterekkel számolt átlag. Következmény: a görbe nem mutatja a túlillesztést, és a kísérlet hamisan „jó” általánosítást jelez. Ugyanebben a környezetben a hibás változatot futtatva a holdoutnak szánt görbe végig a tanítási görbe alatt maradt, ezért a labor záró `assert val_hist[-1] > train_hist[-1]` sora elbukik (mért értékek: `0.035849570590570194 < 0.03669366506974369`). Javítás:

```
val_hist.append(forward_loss(X_val, y_val, W1, b1, W2, b2))  # helyes
```

A javítás után a holdout loss nem esik egybe a tanítási loss-szal; a labor mért futásában az első 69 epochban a holdout loss a train loss alatt maradt, a 69. epoch után végig felette, a különbség a késői szakaszban csökkent. Ez a viszony nem garantált minden feladaton: kis modellnél vagy a tanítás elején a holdout loss a train loss alatt is lehet.

F5. Egy lehetséges terv:

- **Kérdés:** hogyan hat a minibatch méret a holdout loss-ra, ha az optimizerlépések száma rögzített?
- **Adat:** a labor szintetikus generátora változatlan seeddel (`4201`); a split rögzített ($N_{\text{train}} = 200$, $N_{\text{val}} = 50$).
- **Változó tényező:** $M \in \{1, 8, 64\}$.
- **Baseline:** a labor beállítása $M = 16$-tal, azonos lépésszámmal, seedekkel és α-val; ez a referencia, amelyhez a párosított különbségeket számítjuk.
- **Lépésszám (fő összehasonlítás):** a lépésszám minden M-re $T = 3900$, ami a baseline 300 epochjának felel meg. Ekkor az epochok száma $T/\lceil N_{\text{train}}/M \rceil$: $M = 1$-nél 19,5, $M = 8$-nál 156, $M = 16$-nál 300, $M = 64$-nél 975; a futás a megadott lépésszámnál áll meg, szükség esetén epoch közben. Így a batchméret hatása nem keveredik a lépésszámmal, de a teljes példafeldolgozás $T \cdot M$ szerint változik, ezért ezt is jelenteni kell.
- **Másodlagos kontroll (azonos példabüdzsé):** rögzített $E = 62\,400 = T_{\text{baseline}} \cdot 16$ példafeldolgozás mellett $T = E/M$: $M = 1$-nél 62 400, $M = 8$-nál 7 800, $M = 16$-nál 3 900, $M = 64$-nél 975 lépés. Ez a beállítás az adatmennyiséget és a lépésszámot együtt tartja kontroll alatt; a két kontroll együtt választja szét a lépésszám és a batchméret hatását.
- **Rögzített:** architektúra, minibatch SGD, $\alpha = 0.01$, keverési seed (`4202`), valamint az, hogy nincs learning-rate ütemezés és nincs korai leállítás.
- **Metrika:** a T-edik lépés utáni holdout loss egyetlen kiértékeléssel; másodlagosan a train loss és a mért futásidő.
- **Ismétlés:** három inicializálási seed (`4203`, `4204`, `4205`); minden M-hez ugyanazok a seedek, és a baseline-hoz ($M = 16$) seed szerint párosított holdout loss-különbség. Jelentjük az átlagot, a seed-szórást és a három párosított különbséget.
- **Döntési szabály (előre rögzítve):** egy M csak akkor tekinthető jobbnak a baseline-nál, ha mindhárom párosított különbség ugyanabba az irányba mutat, és a párosított különbségek átlaga meghaladja azok szórását. Három seed mellett ez gyenge bizonyíték; a szabály nem helyettesíti a több seedet.
- **Erőforrás:** a számításigény $T \cdot M$ példa-feldolgozás; a futásidőt a kísérlet méri, előre nem állítjuk.
- **Következtetési korlát:** az eredmény erre a szintetikus feladatra, architektúrára és α-ra vonatkozik; nem általánosítható más veszteségre, hálóra vagy valós adatra. A rögzített lépésszám mellett a nagyobb M több példát dolgoz fel és több epochot lát, ezért a batchméret és az adatmennyiség hatása csak a másodlagos kontrollal választható szét; a $M = 1$ eset nagy varianciát mutathat, amit a seed-szórás jelez.

10.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. „A modell a teljes adaton tanul, és ugyanazon értékel, tehát a loss history általánosítási teljesítményt mutat.” Ez hibás: a train loss az empirikus kockázatot méri, amely a memorizálással is csökkenthető. Az F4 hibája pontosan ezt a csapdát valósítja meg: a holdout nélküli görbe alacsony, de semmit sem mond az új adaton várható hibáról. A laborban a holdout loss szétválása a helyes diagnózis.

Önellenőrzés. (1) Mi a minibatch gradiens képlete? (2) Miért kell a gradiens-tárolót nullázni a backward előtt? (3) Mit mér a loss history, ha a veszteséget a tanítóhalmazon számoljuk? (4) Igaz-e, hogy egy gradienslépés után a gradiens a frissített pontban mindig nulla? (Válasz: nem; a 10.4.3 példában a frissítés utáni gradiens nulla, mert a kvadratikus cél minimumába értünk.)

Visszautalás. A gradiens a veszteség legmeredekebb növekedési iránya 4.2. Parciális derivált és gradiens, a minibatch gradiens a teljes gradiens zajos becslése 6.3. Sztochasztikus és minibatch gradiens, a momentumos frissítés állapotot visz 6.4. Momentum és adaptív optimizálók, a tanh

lokális deriváltja pedig $1-\tanh^2 z$ 9.4. Aktivációk és gradiensáramlás, amelyet a labor backwardje $1 - A_1^2$ alakban használ.

10.4.9 Rövid összefoglaló

Az autograd-keretrendszerekben a tanítási lépés három elkülönülő műveletből áll: a gradienstároló nullázása, a backpropagation által előállított minibatch gradiens, valamint a paraméterfrissítés. A kézi NumPy-kódban gradiens-tároló nincs, ezért ott a nullázás helyét a backward által visszaadott tömb felülírása veszi át, és a lépés két látható műveletre redukálódik. A minibatch gradiens az egyedi gradiensek átlaga, ezért a backward képletekben az $\frac{1}{M}$ tényező explicite megjelenik. A frissítés $\theta_{t+1} = \theta_t - \alpha g_t$; a konkrét kézi példában $w_0 = 0$, $g_0 = -4$ és $\alpha = 0.5$ mellett $w_1 = 2$, a veszteség 5-ről 1-re csökken. A loss history és a holdout loss együttesen diagnosztizálja a túlillesztést: a teljes adaton mért alacsony veszteség nem általánosítási bizonyíték. A labor kézzel írt NumPy MLP-t tanít minibatch SGD-vel, rögzített seedekkel és független holdouttal.

10.4.10 Továbblépés és források

A 11.1. Tensor és autograd-életciklus egységben a PyTorch autograd-életciklus következik; ott a `zero_grad`, `backward` és `optimizer.step` hívások a fenti három művelet közvetlen megfelelői.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

11 PyTorch: autograd, modul és adatbetöltés

11.1 Tensor és autograd-életciklus

11.1.1 Cél, előismeret és induló kérdés

A 10.4. Tanítási lépés és tanuló MLP alfejezetben a tanítási lépést három műveletre bontottuk: a gradiens-tároló nullázása, a minibatch gradiens kiszámítása visszaterjesztéssel, majd a paraméterfrissítés. Ez az egység a PyTorch `autograd` rétegét köti hozzá ehhez a képhez. A cél annak pontos megértése, hogy a `requires_grad` flag, a `backward` hívás, a `.grad` tároló, a `detach`, a `no_grad` és az `inference_mode` hogyan oszlanak meg a tenzor értéke, a számítási gráf és a gradiens között. Előfeltétel a 2.4. Kis tudományos szoftverprojekt kis projekt-szemlélete és a 10.4. Tanítási lépés és tanuló MLP kézzel írt MLP-tanítási lépése; a 10.1. Számítási gráf és lokális derivált gráfmodelljét használjuk. A 7.1. Entrópia, cross-entropy és KL loss-fogalmai közül itt csak azt idézzük fel, hogy a visszaterjesztés egy skalár veszteségből indul; a laborban a veszteség MSE, mert sima és könnyen deriválható, nem pedig cross-entropy.

Induló kérdés: egy x levéltensor két különböző úton befolyásolja a veszteséget, ezért a két ág hozzájárulása a gradiensösszegzés miatt összeadódik. Ha ugyanazon az x levéltensoron két külön gráfon kétszer hívunk `backward`-ot, a gradiens kétszer adódik a `.grad` tárolóhoz: mennyi lesz x.`grad`? És mi történik, ha középen egy `detach` szakítja meg az utat? (Ugyanazon a gráfon a második `backward` már hibát ad, mert a gráf felszabadult.) A válaszokat a 11.1.4 kézi példa számszerűen is megadja.

11.1.2 Tenzor, levéltensor és autograd-gráf

A PyTorch `Tensor` egy többdimenziós tömb, amelynek értéke mellett van `dtype`, `device` és egy `requires_grad` logikai flagje. A [D03] dokumentációja szerint a tenzor a számítások alapobjektuma; az autograd szempontjából a flag azt mondja meg, hogy az adott tenzorból kiinduló műveleteknek épüljön-e gráf. Az itt tárgyalt fogalmak a PyTorch 2.14 verziójának API-ját írják le.

Egy tenzor **levéltensor (leaf tensor)**, ha a `grad_fn` attribútuma `None`; a `grad_fn` arra a visszaterjesztő függvényre mutat, amely a tenzort előállító művelet lokális deriváltját ismeri. Levéltensor például a felhasználó által közvetlenül létrehozott tenzor, a `requires_grad=False` inputokból számított eredmény és a `detach()` visszatérési értéke is. A `.grad` alapértelmezésben csak a `requires_grad=True` **levelekre** kerül, és csak akkor, ha azok a `backward` által bejárt veszteséghez kapcsolódnak; a nem-levél tenzorok gradiensét az autograd nem tárolja, kivéve, ha a `retain_grad()` hívással kérjük. A `retain_grad()` a nem-levél tenzor `.grad` tárolóját is feltölti a backward során; erre akkor van szükség, ha egy köztes aktiváció gradiensét vizsgáljuk.

Az **autograd-gráf** egy dinamikusan épülő, irányított körmentes gráf (DAG), amelynek csúcsai tenzorok, élei pedig a végrehajtott műveletek input–output viszonyai. Minden olyan **differenciálható** művelet, amely legalább egy `requires_grad=True` tenzort kap, és amely gráfépítési módban fut (nem `no_grad`/`inference_mode` blokkban), létrehoz egy új gráfcsúcsot, és a kimeneti tenzor `requires_grad=True` lesz. A nem differenciálható műveletek (például `argmax`, összehasonlítások)

kivételek. A gráf a forward alatt épül, a `backward` bejárja, majd a visszaterjesztéshez mentett köztes értékeket (puffereket) felszabadítja; a nem-levél tenzorok `.grad` tárolója alapértelmezésben nem töltődik fel. Ez a dinamikus életciklus az, amit a [D05] autograd-mechanika leírása tárgyal; a labor ezért az autograd viselkedését külön, futtatható kóddal is szemlélteti.

A `requires_grad` három szabálya a mindennapi munkához:

1. Ha egy tenzort `requires_grad=True` értékkel hozunk létre, az levéltensor, és a `.grad` tárolója kezdetben `None`.
2. Ha egy **differenciálható** műveletnek legalább egy `requires_grad=True` inputja van, és a gráfépítés be van kapcsolva (nem `no_grad`/`inference_mode` blokkban vagyunk), a kimenet `requires_grad=True`, és a gráfban új él jön létre. A vegyes (`True` és `False`) inputú művelet kimenete is `True`. Nem minden művelet differenciálható: például az `argmax` vagy az összehasonlítások kimenete `requires_grad=False`.
3. Ha egy tenzort `requires_grad=False` értékkel hozunk létre, abból nem indul gráf; az **csak** `requires_grad=False` inputokra épülő művelet eredménye is `requires_grad=False` (és levél).

A `backward` hívás a gráfon visszafelé haladva minden élre lokális vektor–Jacobi-szorzatot (VJP-t) számol, a 10.1. Számítási gráf és lokális derivált alfejezetben bevezetett $g_u = J_f^\top g_v$ alakban; itt J_f az élhez tartozó művelet Jacobi-mátrixa, g_v a kimeneti oldali gradiens, a szorzat pedig a bemeneti oldali gradienst adja. A bejárás végén minden levéltensorra a kapott gradienst **hozzáadja** a `.grad` meglévő értékéhez, nem felülírja. Ezért a tanítási ciklusban a 10.4. Tanítási lépés és tanuló MLP alfejezetben megismert gradiensnullázás külön lépés: az `optimizer` (például `torch.optim.SGD`) a modell paramétereit és azok `.grad` tárolóit fogja össze, a `zero_grad()` pedig az általa kezelt paraméterek gradienseit nullázza.

```
Szöveges ábra: az autograd-gráf iránya

  x (levél)    y = x²    L = y² (gyökér)
                  forward
                     backward (VJP)
```

A forward a levelektől a gyökér (a skalár veszteség) felé építi a gráfot, a backward pedig a gyökértől a levelek felé halad, és minden élen a lokális VJP-t alkalmazza.

11.1.3 A gradiens útjának megszakadása: a `detach` viselkedése

A levezetési kötelezettség az, hogy a számítási gráfon pontosan megindokoljuk, mikor szakad meg a gradiens útja. Az állítás a következő.

Állítás (a `detach` gráfszakadása). Legyen x levéltensor, amelyre `requires_grad=True`, legyen $z = f(x)$ egy differenciálható tenzorfüggvény eredménye, legyen $y = z.\texttt{detach}()$, és legyen $L = \ell(y)$ egy differenciálható skalár veszteség. Az autograd nem viszi át a gradienst y-ból x-be. Ha ℓ nem használ más, gradienst igénylő változót, akkor $L.\texttt{requires_grad}$ hamis, ezért a $L.\texttt{backward}()$ hívás `RuntimeError` kivételt vált ki; ha más, tanulható bemenete is van ℓ-nek, a visszaterjesztés azon az ágon működik, de $x.\texttt{grad}$ nem frissül. A két esetben a forward érték ugyanaz.

Indoklás. Az autograd-gráf definíciója szerint a `detach` egy új tenzort ad vissza, amely ugyanarra a tárolóra mutat, tehát az értéke megegyezik z értékével, de a `requires_grad` flagje `False`, és a gráfban **nem jön létre él** z és y között. A veszteség ℓ függvénye y felől nem épít visszafelé vezető gráfélet z-hez; ha más bemenetén sincs `requires_grad=True` változó, a kimenete sem rendelkezik autograd-gráffal. A `backward` bejárás kizárólag a gráf élei mentén halad visszafelé;

ha az y és z közötti él nem létezik, akkor az út x-hez nem folytatódik. A láncszabály szintjén ez azt jelenti, hogy a $\frac{\partial L}{\partial y}$ VJP megvan, de a $\frac{\partial y}{\partial z}$ él hiányzik, így a láncszabály nem ad további VJP-t z és x felé. A `detach` tehát a gráf topológiáját változtatja meg, nem a tenzor értékét.

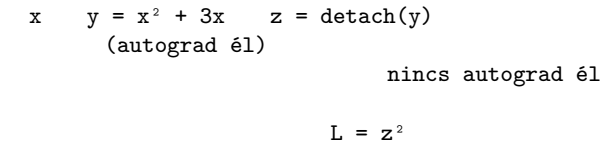
Ez API-szemantikai állítás, nem analitikus deriválási tétel. A [D05] dokumentálja, hogy a `detach` „elvágja” a gráfot; a fenti indoklás a gráfbejárás definíciójából származik, és a labor kísérletileg is ellenőrzi.

A `no_grad` és az `inference_mode` hasonló hatást érnek el, de más eszközzel. A `with torch.no_grad():` kontextusban minden újonnan végrehajtott **művelet** kimenete `requires_grad=False`, tehát a blokkban nem épül gráf; ez nem vonatkozik a kifejezetten `requires_grad=True` argumentummal létrehozott tenzorokra és az `nn.Parameter`-re. A blokkban létrehozott tenzorok értéke megmarad, de autograd-történetük nincs, és a blokk elhagyása után a gráfépítés visszaáll. Az `inference_mode` ennél erősebb: a [D05] szerint a nézetkövetést (*view tracking*) is kikapcsolja, és a verziószámlálót sem frissíti; a benne létrehozott „inference tensor” ezért autogradban csak korlátozottan használható: ha a backward mentett értékként akarja felhasználni, hibát ad (`RuntimeError: Inference tensors cannot be saved for backward`), így csak a blokkon kívül készített klónnal vihető át autograd-számításba. A `no_grad` elsősorban a kiértékeléshez és a paraméterfrissítéshez, az `inference_mode` a végleges következtetéshez való.

11.1.4 Kézi példa f64-ben

A következő kód a detach hatását mutatja egy kétlépéses gráfon. A számítás egyszerű ahhoz, hogy a várt gradiens kézzel is kiszámolható legyen.

Szöveges ábra: a detach gráfszakadása



A backward a loss-tól z-ig eljut, de z-nél megáll, ezért x.`grad` `None` marad. A `detach` nélküli változatban az y és L közötti él is megvan, ezért a gradiens 140.

```
# lab/n041/detach_demo.py
import torch

torch.manual_seed(20261020)
dtype = torch.float64
device = torch.device("cpu")

# 1. eset: detach a közepén
x = torch.tensor(2.0, dtype=dtype, device=device, requires_grad=True)
y = x**2 + 3*x          # y = 10
z = y.detach()          # gráfél elvágva
L = z**2                # L = 100
assert not L.requires_grad
try:
    L.backward()
except RuntimeError:
    print("detach: nincs visszaterjeszthető gráf")
else:
    raise AssertionError("A backwardnak detached loss esetén hibáznia kell")
print("detach után x.grad:", x.grad)   # None
```

```
# 2. eset: ugyanaz a számítás detach nélkül
x2 = torch.tensor(2.0, dtype=dtype, device=device, requires_grad=True)
y2 = x2**2 + 3*x2       # y2 = 10
L2 = y2**2
L2.backward()
print("detach nélkül x2.grad:", x2.grad)  # tensor(140., dtype=torch.float64)
```

A második eset számítása: $y = x^2 + 3x$, $\frac{dy}{dx} = 2x + 3 = 7$; $L = y^2$, $\frac{dL}{dy} = 2y = 20$; a láncszabályból $\frac{dL}{dx} = 7 \cdot 20 = 140$. Az első esetben a formális $\frac{dL}{dz} = 20$ derivált kiszámítható, de a `detach` miatt nincs autograd-gráf: `L.backward()` kivételt okoz, és az x.`grad` marad `None`. A `detach()` eredménye `requires_grad=False` flagű levéltensor (`grad_fn` nélkül), ezért abból nem épül további autograd-gráf.

A gradiensakkumuláció külön jelenség:

```
# lab/n041/accumulation_demo.py
import torch

x = torch.tensor(3.0, dtype=torch.float64, requires_grad=True)
for _ in range(2):
    (x**2).backward()
print("két backward után x.grad:", x.grad)   # tensor(12., dtype=torch.float64), nem 6.0
```

Egyetlen `backward` gradiense $2x = 6$; a második, új gráfon végrehajtott hívás ugyanahhoz a `.grad` tárolóhoz hozzáad újabb 6-ot, ezért az eredmény 12. A tanítási ciklus ezért hív `zero_grad`-ot minden lépés előtt.

```
Szöveges ábra: a gradiens akkumulációja

  kezdet:            x.grad = None
  első backward:     x.grad = 0 + 6 = 6
  második backward:  x.grad = 6 + 6 = 12
```

Ez a viselkedés különbözik a későbbi élettartam-ellenpéldától: ott ugyanazon a gráfon hívjuk meg kétszer a `backward`-ot, ami már felszabadított gráf miatt hibát ad; itt két külön gráf jön létre, és a két gradiens összeadódik.

11.1.5 Labor: NumPy-MLP és PyTorch autograd egyeztetése

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A bemenet a szakaszban rögzített X mátrix és Y vektor; külső adatfájl nem kell, a kód a szakasz kódblokkjából futtatható.

A labor célja a 10.4. Tanítási lépés és tanuló MLP alfejezetben kézzel írt NumPy-MLP gradiensének független ellenőrzése PyTorch `float64` autograddal, majd egy véletlenül bekerült `detach` hiba lokalizálása és javítása. A bemenet rögzített: $X \in \mathbb{R}^{4\times3}$, $Y \in \mathbb{R}^{4\times2}$; a háló egy rejtett réteget tartalmaz 4 egységgel és tanh aktivációval, a veszteség pedig MSE. A tanh azért jó választás, mert sima, így az analitikus NumPy-gradiens és a PyTorch autograd összehasonlításának nincs a ReLU-nullpontjából származó konvencióproblémája.

A munkakönyvtár `lab/n041/`. A kiinduló `check_autograd.py` szándékosan hibás: a rejtett aktiváció után egy `detach` áll. A hallgató feladata: futtassa a kódot, rögzítse a hiba tünetét, távolítsa el a `detach` hívást, majd igazolja, hogy a javítás után mind a négy paramétermátrix gradiense egyezik a NumPy-analitikus referenciával. A beadandó eredmény a javított kód és a maximális abszolút eltérések listája; az ellenőrzési küszöb 10^{-12} `float64`-ben. A NumPy-gradiens skálázása a PyTorch `mean` MSE-jével azonos. Az $L = \frac{1}{Nk}\sum_{n,j}(Z_{2,nj} - Y_{nj})^2$ veszteség deriváltja

ugyanis $\partial L/\partial Z_{2,nj} = \frac{2}{Nk}(Z_{2,nj} - Y_{nj})$, amit a kód `2*diff/diff.size` alakban számol (Nk az összes elem száma, k a kimeneti dimenzió). A közölt labor $k = 2$ esetén fut; a képlet általános k-ra érvényes. Az itt közölt kódot a rögzített környezetben lefuttattuk; a kimenet a valós futás eredménye.

```
# lab/n041/check_autograd.py
import numpy as np
import torch

torch.manual_seed(20261020)
np.random.seed(20261020)
dtype = torch.float64
device = torch.device("cpu")

def numpy_mlp_grad(X, W1, b1, W2, b2, Y):
    """NumPy-MLP gradiens, tanh aktiváció, MSE loss, átlagolással."""
    Z1 = X @ W1.T + b1
    A1 = np.tanh(Z1)
    Z2 = A1 @ W2.T + b2
    diff = Z2 - Y
    N = X.shape[0]
    gZ2 = 2.0 * diff / diff.size   # MSE (mean az összes elemre): 2*diff/(N*k)
    gW2 = gZ2.T @ A1
    gb2 = gZ2.sum(axis=0)
    gA1 = gZ2 @ W2
    gZ1 = gA1 * (1.0 - A1**2)
    gW1 = gZ1.T @ X
    gb1 = gZ1.sum(axis=0)
    return gW1, gb1, gW2, gb2

N, d, h, k = 4, 3, 4, 2
X_np = np.random.randn(N, d)
Y_np = np.random.randn(N, k)
W1_np = np.random.randn(h, d)
b1_np = np.random.randn(h)
W2_np = np.random.randn(k, h)
b2_np = np.random.randn(k)

ref = numpy_mlp_grad(X_np, W1_np, b1_np, W2_np, b2_np, Y_np)

X = torch.tensor(X_np, dtype=dtype, device=device, requires_grad=True)
Y = torch.tensor(Y_np, dtype=dtype, device=device)
W1 = torch.tensor(W1_np, dtype=dtype, device=device, requires_grad=True)
b1 = torch.tensor(b1_np, dtype=dtype, device=device, requires_grad=True)
W2 = torch.tensor(W2_np, dtype=dtype, device=device, requires_grad=True)
b2 = torch.tensor(b2_np, dtype=dtype, device=device, requires_grad=True)

# Hibás változat: a detach megszakítja az utat az első réteg felé.
h_lin = X @ W1.t() + b1
h = torch.tanh(h_lin).detach()      # <-- a hiba
logits = h @ W2.t() + b2
loss = ((logits - Y) ** 2).mean()
loss.backward()

print("Hibás: W1.grad =", W1.grad, ", b1.grad =", b1.grad)
print("Hibás: X.grad =", X.grad)
assert W1.grad is None and b1.grad is None and X.grad is None

# Javított változat: a detach elhagyása.
W1.grad = None; b1.grad = None; W2.grad = None; b2.grad = None
h = torch.tanh(X @ W1.t() + b1)
logits = h @ W2.t() + b2
loss = ((logits - Y) ** 2).mean()
```

```
loss.backward()

errs = [
    np.max(np.abs(W1.grad.detach().numpy() - ref[0])),
    np.max(np.abs(b1.grad.detach().numpy() - ref[1])),
    np.max(np.abs(W2.grad.detach().numpy() - ref[2])),
    np.max(np.abs(b2.grad.detach().numpy() - ref[3])),
]
print("max_abs_errors:", errs)
assert max(errs) < 1e-12
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020*; két futás kimenete bitazonos):*

```
Hibás: W1.grad = None , b1.grad = None
Hibás: X.grad = None
max_abs_errors: [np.float64(2.220446049250313e-16), np.float64(5.551115123125783e-17),
 ↪  np.float64(2.220446049250313e-16), np.float64(0.0)]
```

A hibás változat tünete precízen lokalizálja a gráfszakadást: `W2.grad` és `b2.grad` nem `None`, mert a második réteg a `detach` utáni tenzorból indul, de `W1.grad`, `b1.grad` és `X.grad` mind `None`. A javítás után a négy gradiens a NumPy-referenciával egyezik; `float64`-ben a mért maximális eltérés $2{,}3 \cdot 10^{-16}$, a 10^{-12} küszöb a környezeti eltérésekre hagy tartalékot. A `detach` eltávolítása nem változtat a forward értéken, tehát a hibakeresés pontosan a gráf topológiájára, nem a veszteség számértékére irányul.

11.1.6 Feladatok

F1 (fogalmi, nehézség 1). Mi a különbség a levéltensor és a nem-levél tenzor között az autograd életciklusában? Miért `None` a nem-levél tenzor `.grad` értéke alapértelmezésben? Adj egy-egy példát. Elvárt eredmény: definíció, a `.grad` tárolás feltétele, két rövid kódrészlet.

F2 (levezetési, nehézség 2). Legyen $x = 3$ levéltensor, és legyen $L = x^2 + 2x + 1$. Egyetlen `backward` hívás után mennyi x.`grad`? Vezesd le az elágazó gráfon a gradiensösszegzést, és jelöld meg, melyik tag melyik ágból jön. Elvárt eredmény: a derivált kiszámítása, az áganként felbontás és a konkrét `x.grad` érték.

F3 (implementációs, nehézség 2). Írj `split_path(x, detach_middle)` függvényt, amely kiszámítja $L = 2x^2$-et, de ha `detach_middle` igaz, akkor a négyzetre emelés után `detach`-et hajt végre. A függvény csak akkor hívjon `backward`-ot, ha `L.requires_grad` igaz (a `detach`-es ágon kihagyja, mert ott `RuntimeError` keletkezne), és térjen vissza az `(L.item(), x.grad)` párral (`x.grad=None` a `detach`-es ágon). Ellenőrizd `x=2` esetén mindkét ágat. Elvárt eredmény: futtatható kód, a két visszatérési érték és egy `assert` ellenőrzés.

F4 (hibakeresési, nehézség 2). A labor `check_autograd.py` hibás változatában miért `None` az első réteg súlyának gradiense? Azonosítsd a pontos sort, magyarázd el a gráfszakadást, és add meg a javítást úgy, hogy a forward érték ne változzon. Elvárt eredmény: a hiba oka, következménye, javítása és a javítás utáni ellenőrző futás állítása.

F5 (kísérleti/transzfer, nehézség 3). Tervezz egy kis kísérletet annak eldöntésére, hogy egy `with torch.no_grad():` blokkba helyezett forward után az `optimizer.step()` mit tesz a paraméterekkel: történik-e frissítés, és hogyan függ ez attól, hogy a lépés előtt nulláztuk-e a gradienseket. Add meg a modellt, az optimalizálót és a tanulási rátát, a minibatch méretét, a baseline-t, a változó tényezőt, a metrikát és a következtetési határt. Elvárt eredmény: egy megvalósítható terv, a várt eredmény indoklása és legalább egy érvényességi korlát. Mivel a [D05]-re itt dokumentációs állításként hivatkozunk, a kísérlet ezt az állítást empirikusan ellenőrzi.

A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

11.1.7 Megoldások és indoklás

F1. A levéltensor közvetlenül a felhasználótól származik, és a `backward` bejárás végén a `.grad` tárolója frissül. A nem-levél tenzor művelet eredménye, és a gráf felszabadulásakor a köztes gradiense nem marad meg, ezért `.grad`-ja alapértelmezésben `None`; ha kell, `retain_grad()`-dal kérhető. Példa: `x = torch.tensor(2.0, requires_grad=True)` levéltensor; `y = x * 2` nem-levél. `y.backward()` után `x.grad` értéke `2.0`, `y.grad` értéke `None`. A `y.grad` ilyen elérésekor a PyTorch figyelmeztetést ad, hogy nem-levél tenzor `.grad` mezőjét olvassuk; a `retain_grad()` ezt a figyelmeztetést és a `None` értéket szünteti meg (a `y.grad` ekkor `1.0` lesz).

F2. A gráfon x-nek két leszármazottja van: az $a = x^2$ ág és a $b = 2x+1$ ág; a veszteség $L = a+b$. A 10.1. Számítási gráf és lokális derivált gradiensösszegzési lemmája szerint

$$\frac{dL}{dx} = \frac{\partial a}{\partial x} + \frac{\partial b}{\partial x} = 2x + 2.$$

$x = 3$ esetén az első ág gradiense 6, a másodiké 2, az összeg 8. A kódban `x = torch.tensor(3.0, dtype=torch.float64, requires_grad=True)`, `(x**2 + 2*x + 1).backward()`, ekkor `x.grad.item() == 8.0`. A `2x+1`-ben a $2x$ tag deriváltja 2, a konstans 1 deriváltja 0; az x^2 ág deriváltja 6, és a kettő összeadódik.

F3. A teljes függvény:

```python
# lab/n041/split_path.py
import torch

torch.manual_seed(20261020)

def split_path(x, detach_middle):
    a = x ** 2
    if detach_middle:
        a = a.detach()
    L = 2.0 * a
    # A detach-ágon nincs autograd-gráf; ott a backward kivételt okozna.
    if L.requires_grad:
        L.backward()
    return L.item(), x.grad

x1 = torch.tensor(2.0, dtype=torch.float64, requires_grad=True)
print("detach nélkül:", split_path(x1, False))
# (8.0, tensor(8., dtype=torch.float64))

x2 = torch.tensor(2.0, dtype=torch.float64, requires_grad=True)
print("detach-tal:", split_path(x2, True))        # (8.0, None)

assert x2.grad is None
```

Az $L = 2x^2$ deriváltja $4x = 8$ $x = 2$ esetén. A `detach` nélküli ágon a gradiens 8,0; a `detach`-es ágon a forward érték ugyanúgy 8,0, de a gráfél hiánya miatt `x.grad` marad `None`. A `backward()` hívást csak akkor szabad kiadni, ha `L.requires_grad` igaz; a `detach`-es ágon a `backward()` `RuntimeError` kivétellel állna le, ezért a függvény ott kihagyja, és `x.grad=None`-t ad vissza. Az `assert` a gráfszakadást ellenőrzi, nem a loss értékét.

F4. A hiba sora:

```
h = torch.tanh(h_lin).detach()
```

A `detach` új tenzort hoz létre ugyanazzal a tárolóval, de `requires_grad=False` flaggel és a gráfban nincs él a `tanh` kimenete és az új tenzor között. A backward a loss-tól a második réteg paramétereiig (`W2`, `b2`) eljut, mert ezek a `detach` utáni `h` tenzoron keresztül is szerepelnek a gráfban; a `h`-nál azonban megáll, ezért az első réteg (`W1`, `b1`) és `X` gradiense `None` lesz. A javítás a `detach()` elhagyása:

```
h = torch.tanh(h_lin)
```

A forward érték nem változik, mert a `detach` csak a gráfot vágta, a tárolót nem módosította. A javítás utáni ellenőrzés a labor kódjának második felében történik meg: a négy gradiens maximális abszolút eltérése a NumPy-referenciától 10^{-12} alatt van.

F5. Egy lehetséges terv. *Kérdés:* ha a forward `with torch.no_grad():` blokkban fut, és a `backward` emiatt nem épít gradienst, mit tesz az `optimizer.step()` a paraméterekkel, és miben tér el ez a normál egy-lépéses frissítéstől? *Modell:* a labor MLP-je ($d = 3$, $h = 4$, $k = 2$, tanh, MSE), `float64`, CPU; a minibatch a labor rögzített X, Y tömbje ($N = 4$). *Optimalizáló:* `torch.optim.SGD(model.parameters(), lr=0.1)`, rögzített seed. *Protokoll:* a modellt kétszer ugyanabból a kezdőállapotból indítjuk (a kezdő `state_dict` másolatából), és mindkét futásban a lépés előtt `optimizer.zero_grad()`-ot hívunk. A kontroll esetben a forward normál módban fut, majd `loss.backward()`, `optimizer.step()` következik. A vizsgált esetben a forward `with torch.no_grad():` blokkban fut; rögzítjük a `loss.requires_grad` értékét, megkíséreljük a `loss.backward()` hívást, elkapjuk a `RuntimeError`-t, majd meghívjuk az `optimizer.step()`-et. *Metrika:* a paraméterváltozás L2-normája ($\|\theta_1 - \theta_0\|_2$), a `loss.grad_fn`/`requires_grad` állapota és a két futás paramétereinek maximális abszolút eltérése. *Várt eredmény:* a `no_grad` alatt a kimenet `requires_grad=False`, ezért a `backward()` `RuntimeError`-t ad; a `zero_grad()` után a paraméterek `.grad` mezője `None`, és az SGD az ilyen paramétereket kihagyja, tehát a frissítés nulla. Ha a `zero_grad()` elmarad, és egy korábbi lépésből stale gradiens maradt, a `step()` a régi gradienssel frissítene; ezt a kísérlet külön ágként ellenőrizheti. A kontroll esetben a frissítés nem nulla. *Következtetési határ:* ez egyetlen lépés, tehát nem állít semmit a tanítás konvergenciájáról, csak a gráfépítés és a `.grad` életciklusának összefüggéséről; a `no_grad` nem tiltja meg a `step()` hívását, csak a gradiens előállítását.

11.1.8 Tipikus tévedés és önellenőrzés

A leggyakoribb tévedés: *„Ha a `.grad` megvan, a gráf is megvan."* A gradienstárolás és az autograd-gráf élettartama nem ugyanaz. Ellenpélda:

```
# lab/n041/lifetime_counterexample.py
import torch

x = torch.tensor(2.0, dtype=torch.float64, requires_grad=True)
y = x * 2
y.backward()
print("x.grad:", x.grad)   # tensor(2., dtype=torch.float64)
try:
    y.backward()
except RuntimeError as e:
    print("második backward:", e)
    # Az üzenet első mondata: Trying to backward through the graph a
    # second time (or directly access saved tensors after they have
    # already been freed). …
```

Az első `backward` után a gráf felszabadult, bár `x.grad` továbbra is tárolja a 2,0 értéket. A második `backward` ezért hibát ad, nem pedig újabb 2,0-t. A `detach` ezzel szemben nem a `.grad` tárolót törli, hanem a gráfból vág ki élt; a kettőt nem szabad felcserélni.

Rövid önellenőrzés: (1) Ha `x.grad` már `3.0`, és egy új gráfon `backward`-ot hívunk, mi lesz az eredmény? Válasz: `3.0 + új_grad`, mert a gradiens akkumulálódik. (2) Miért nem kap gradienst az első réteg a labor hibás kódjában? Válasz: a `detach` után nincs gráfél a rejtett aktiváció és a következő számítás között. (3) Miben erősebb az `inference_mode` a `no_grad`-nál? Válasz: a [D05] szerint a nézetkövetést is kikapcsolja, ezért a benne keletkező tenzorok autogradban csak korlátozottan használhatók: mentett értékként hibát adnak, klónozással vihetők át; a `no_grad` csak a gráfépítést állítja le az adott blokkban. Ezt a különbséget a könyv dokumentációs állításként rögzíti, a labor nem méri.

Visszautalás: a 10.1. Számítási gráf és lokális derivált szerint az elágazó csúcsnál a leszármazott ágak gradiensei összeadódnak; a 10.4. Tanítási lépés és tanuló MLP szerint a tanítási lépés nullázásból, visszaterjesztésből és frissítésből áll; a 2.4. Kis tudományos szoftverprojekt szerint a labor kódját modulhatárral, explicit input–output szerződéssel és tesztellenőrzéssel szervezzük.

Az induló kérdésre a válasz. Az $x = 2$, $L = 2x^2$ kézi példában egyetlen gráfon $x.\texttt{grad} = 8{,}0$; két külön gráfon végzett `backward` után a gradiensek összeadódnak, azaz 16,0; ha közben egy `detach` megszakítja az utat, azon az ágon `x.grad None` marad.

11.1.9 Rövid összefoglaló

A PyTorch tenzor értéket, `dtype`-ot, eszközt és `requires_grad` flaget hordoz; a flag azt szabályozza, hogy a tenzorból épül-e autograd-gráf. A levéltensor olyan tenzor, amelynek nincs `grad_fn`-je; `.grad` alapértelmezésben csak a `requires_grad=True` leveleken tárolódik, nem-levél tenzornál `retain_grad()`-dal kérhető. A `backward` a gráfon visszafelé VJP-ket számol, és a levéltensorok gradiensét hozzáadja, ezért kell nullázás. A `detach` új, gráfél nélküli tenzort ad ugyanazzal a tárolóval, és megszakítja a gradiens útját; a `no_grad` blokkszinten, az `inference_mode` erősebben kapcsolja ki a gráfépítést. A labor ezt a NumPy-MLP gradiensének f64-egyezésével és egy detach hiba javításával ellenőrzi.

11.1.10 Következő alkalmazás és források

A következő 11.2. nn.Module és paraméterregisztráció egységben az `nn.Module` és a paraméterregisztráció következik: ott a levéltensorok a modell paraméterlistájába kerülnek, és az `optimizer` közvetlenül ezeket a `.grad` tárolókat nullázza és frissíti.

- [D03] PyTorch contributors (2026). *Tensor*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/tensors.html. Hozzáférés: 2026-10-30.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.

11.2 nn.Module és paraméterregisztráció

11.2.1 Cél, előismeret és induló kérdés

A 11.1. Tensor és autograd-életciklus egységben a tenzor életciklusát vizsgáltuk: a levéltensor, a `requires_grad` flag, a `.grad` felhalmozódás és a gráf leválasztása. Itt azt a lépést tesszük meg,

amely nélkül egy több százezer paraméteres háló tenzorainak kézi nyilvántartása gyakorlatilag kezelhetetlen: a tenzorokat egyetlen modellobjektumba szervezzük, hogy az optimizer és a mentő eljárás egyetlen listából lássa, mely tenzorok tanulnak.

Induló becslés: egy $3 \to 4 \to 2$ MLP két affin rétege esetén a súly-tenzorok elemszáma $3 \cdot 4 + 4 \cdot 2 = 20$, a torzításoké $4 + 2 = 6$, tehát összesen 26 skalár paraméterelem várható (négy paramétertenzorban: két súly- és két bias-tenzor, nem 26 tenzor). A kérdés: hogyan garantálja az osztály, hogy pontosan ez a 26 skalár (négy tenzorban) kap gradienst, jelenik meg az optimizerben, és menthető el egyetlen állományban?

11.2.2 A `Module`, a `Parameter` és a puffer definíciója

Definíció (`nn.Module`). A PyTorch `nn.Module` osztálya ([D06]) két felelősséget egyesít. Egyrészt a modell nevét, az almodulokat és a tanulható tenzorokat tároló szerkezet; másrészt a számítást definiáló `forward` metódus hordozója. A modult nem a `forward`-dal, hanem a `model(x)` hívással futtatjuk: a `__call__` futtatja a modulra és az almodulokra regisztrált hookokat, például a forward-hookokat.

Definíció (`nn.Parameter`). A `Parameter` a `torch.Tensor` alosztálya, amely levéltensor, és az `nn.Module` attribútumaként való hozzárendelés során automatikusan a modul `_parameters` nyilvántartásába kerül. A `Parameter(t)` konstruktor `requires_grad=True` flaget biztosít, ezért a paraméter megjelenik az autograd gráfban, és a `backward` a `.grad` mezőt frissíti.

Definíció (puffer). A `register_buffer(name, tensor, persistent=True)` egy közönséges tenzort rendel a modulhoz. A puffer **nem** tanuló állapot: nem jelenik meg a `parameters()` iterációban, és az optimizer nem frissíti. A `register_buffer` a tenzor `requires_grad` jelzőjét **nem módosítja**: egy `requires_grad=True` tenzor pufferként is differenciálható marad; a szokásos puffer azonban `requires_grad=False`, tehát gradienst nem kap. A regisztráció (a modul része-e), az autograd (kap-e gradienst) és az optimizerbe vonás (frissül-e) külön tulajdonság; a `Parameter` is megengedi a `requires_grad=False` beállítást. Ugyanakkor a modul része: a `.to(device)` áthelyezi, és a `state_dict()` menti, ha `persistent=True`. A puffer például futó statisztikát, normalizációs konstanst vagy pozíciókódot tárolhat.

A `forward` bemenete a 7.2. Softmax és többosztályos cross-entropy alfejezetben látott konvenció szerint batchelt tenzor: egy $x \in \mathbb{R}^{N \times d}$ bemenetre a lineáris réteg $W \in \mathbb{R}^{m \times d}$ súllyal és $b \in \mathbb{R}^m$ torzítással

$$z = xW^\top + b \in \mathbb{R}^{N \times m}.$$

A `state_dict()` egy `OrderedDict[str, Tensor]` objektumot ad; az almodulok kulcsai ponttal tagolt hierarchikus nevet kapnak, például `fc1.weight`. A betöltésnél a kulcsoknak és alakoknak egyezniük kell; a részletes protokollt a 12.2. Checkpoint és folytathatóság egységben tárgyaljuk.

A `train(mode=True)` és `eval()` hívások a modul `training` logikai mezőjét állítják, és rekurzívan végigmennek az almodulokon. Az `nn.Dropout(p=0.5)` például tanítási módban az aktív elemeket $1/(1-p) = 2$-vel skálázza, kiértékelési módban viszont identitástranszformáció. A BatchNorm futó statisztikái tipikus pufferek, de azok részletes elemzése a 18.2. Batch normalization train és eval módban egységben következik.

11.2.3 Tanuló paraméter és puffer: a regisztrációs szabály

A különbséget pontos állításként fogalmazzuk meg, majd egy konkrét modellen ellenőrizzük.

Állítás (regisztrációs szabály). Legyen M egy `nn.Module` leszármazott. Ha a `__init__`-ben egy p `nn.Parameter` kerül a `self.p` attribútumba, akkor:

1. p megjelenik az `M.parameters()` iterációban, és `p.requires_grad=True`;
2. a `"p"` kulcs szerepel az `M.state_dict()`-ben.

Ha egy b tenzort a `self.register_buffer("q", b)` hívással regisztrálunk, akkor b **nem** szerepel az `M.parameters()`-ben, a `requires_grad` jelzőjét a regisztráció nem változtatja (a szokásos `requires_grad=False` puffer nem kap gradienst), de a `"q"` kulcs megjelenik a `state_dict()`-ben. Továbbá `M.to(device)` mindkét tenzort ugyanarra az eszközre helyezi.

Indoklás. A `Parameter` hozzárendelését a `Module.__setattr__` észleli, és a tenzort a modul `_parameters` rendezett szótárába teszi; a `register_buffer` a `_buffers` szótárba ír. A `parameters()` rekurzívan csak a `_parameters` gyűjteményt járja be, a `state_dict()` pedig mindkettőt. Az `optim.Optimizer` konstruktora az `M.parameters()` generátorát kapja, és a paraméterobjektumokra mutató referenciákat listába gyűjti, ezért a pufferre soha nem lép. Ez interfész-szabály, nem analitikus tétel; a laborban független ellenőrzéssel mutatjuk meg.

A konkrét modell legyen egy affin réteg, amely a bemeneti norma exponenciális mozgóátlagát pufferben tartja. Az input $\mathbb{R}^{N\times 3}$, a kimenet $\mathbb{R}^{N\times 2}$, a két puffer (`ema_input_norm`, `n_seen`) egy-egy skalár.

```python
# munkakönyvtár: lab/n042
import torch
import torch.nn as nn

class WeightedStats(nn.Module):
    """Affin réteg, amely az input normájának exponenciális
    mozgóátlagát pufferben tartja."""
    def __init__(self, d_in: int, d_out: int, alpha: float = 0.9):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(d_out, d_in))
        self.bias = nn.Parameter(torch.zeros(d_out))
        self.register_buffer('ema_input_norm', torch.tensor(0.0))
        self.register_buffer('n_seen', torch.tensor(0, dtype=torch.int64))
        self.alpha = alpha

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (N, d_in)
        z = torch.nn.functional.linear(x, self.weight, self.bias)  # (N, d_out)
        if self.training:   # a puffer csak tanítás közben frissül
            with torch.no_grad():
                mean_norm = x.norm(dim=1).mean()
                self.ema_input_norm.mul_(self.alpha).add_(mean_norm * (1.0 - self.alpha))
                self.n_seen += 1
        return z

m = WeightedStats(3, 2)
print([name for name, _ in m.named_parameters()])   # ['weight', 'bias']
print([name for name, _ in m.named_buffers()])      # ['ema_input_norm', 'n_seen']
print(sorted(m.state_dict().keys()))
# ['bias', 'ema_input_norm', 'n_seen', 'weight']
print(m.ema_input_norm.requires_grad)               # False
```

A frissítés képlete a t-edik hívásnál

$$\text{ema} \leftarrow \alpha\,\text{ema} + (1-\alpha)\frac{1}{N}\sum_{n=1}^{N}\|\mathbf{x}_n\|_2, \qquad n_{\text{seen}} \leftarrow n_{\text{seen}} + 1,$$

amit a kód a `no_grad` blokkban hajt végre.

A puffer csak `self.training` mellett frissül; a `torch.no_grad()` a gráfépítést tiltja, az állapotmódosítást nem. Az eval-invariáns teszt:

```
# Az előző blokk m modelljére épül.
m.eval()
x0, n0 = m.ema_input_norm.clone(), int(m.n_seen)
m(torch.rand(4, 3))
assert torch.equal(m.ema_input_norm, x0) and int(m.n_seen) == n0   # eval alatt nem változik
```

A `forward`-ban a pufferfrissítést `with torch.no_grad():` blokkban végezzük. Ez nem azért kötelező, mert a puffer `requires_grad=False`: a blokk nélkül a frissítés jobb oldala, a `mean_norm` az `x` bemenetből számolódik, ezért ha `x` gradiensigényes, a `mean_norm` `grad_fn`-nel rendelkezik, és a puffer in-place írása a puffert is a gráf részévé teszi (ilyenkor a puffer `requires_grad` flagje `True`-ra vált, és `grad_fn` jelenik meg rajta). A `no_grad` blokk ezt zárja ki, így a tanítás visszafelé menete változatlan marad.

Az optimizercsatlakozás következménye: `torch.optim.SGD(m.parameters(), lr=0.1)` kizárólag a `weight` és `bias` tenzorokra hivatkozik, tehát a puffer értéke csak a `forward` explicit frissítése révén változik.

11.2.4 Kézi példa és ellenőrző labor

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `n042_module_check.py` a `lab/` könyvtárból futtatható (`python n042_module_check.py`); külső adatfájl nem kell.

A kézi példa az induló becslés pontossá tétele. Egy `nn.Linear(3,4)` réteg súlya $\mathbb{R}^{4\times3}$, torzítása $\mathbb{R}^4$, az `nn.Linear(4,2)` súlya $\mathbb{R}^{2\times4}$, torzítása $\mathbb{R}^2$. A paraméterszám:

$$(4\cdot3+4)+(2\cdot4+2) = 12+4+8+2 = 26.$$

A labor egy teljes, futtatható ellenőrző szkript, amely a paraméterszámot, az eszközt, a `state_dict` kulcsait és a `train`/`eval` váltást vizsgálja. A szkriptet a rögzített környezetben lefuttattuk; az alábbi kimenet a valós futás eredménye.

```
# munkakönyvtár: lab/n042
# futtatás: python n042_module_check.py
import torch
import torch.nn as nn

torch.manual_seed(0)
DEVICE = torch.device('cpu')
DTYPE = torch.float32

class MLP342(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3, 4)
        self.fc2 = nn.Linear(4, 2)
```

```
    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

model = MLP342().to(device=DEVICE, dtype=DTYPE)

n_params = sum(p.numel() for p in model.parameters())
print(n_params)                                     # 26
assert n_params == 26
print(model.state_dict()['fc1.weight'].shape)       # torch.Size([4, 3])
print(model.state_dict()['fc1.bias'].shape)         # torch.Size([4])
assert model.state_dict()['fc1.weight'].shape == (4, 3)
assert model.state_dict()['fc1.bias'].shape == (4,)
print(sorted(model.state_dict().keys()))
# ['fc1.bias', 'fc1.weight', 'fc2.bias', 'fc2.weight']
assert sorted(model.state_dict().keys()) == [
    'fc1.bias', 'fc1.weight', 'fc2.bias', 'fc2.weight']
print({str(p.device) for p in model.parameters()})  # {'cpu'}
print({p.dtype for p in model.parameters()})        # {torch.float32}
assert all(p.device == torch.device('cpu') for p in model.parameters())
assert all(p.dtype == torch.float32 for p in model.parameters())

optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
print([len(g['params']) for g in optimizer.param_groups])  # [4]
assert [len(g['params']) for g in optimizer.param_groups] == [4]

x = torch.ones(1, 3, device=DEVICE, dtype=DTYPE)
model.eval()
with torch.no_grad():
    y = model(x)
print(y.shape)                                      # torch.Size([1, 2])
assert y.shape == (1, 2)
print(model.training)                               # False
assert model.training is False

# train/eval eltérése egyetlen Dropout rétegen
drop = nn.Dropout(p=0.5)
drop.eval()
print(torch.equal(drop(x), x))                      # True
assert torch.equal(drop(x), x)
drop.train()
z = drop(x)
print(z.shape)                                      # torch.Size([1, 3])
assert z.shape == (1, 3)
# tanítási módban az aktív elemek 0 vagy 2; a pontos maszk seedfüggő
```

Futtatott kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 0; két futás kimenete bitazonos):

```
26
torch.Size([4, 3])
torch.Size([4])
['fc1.bias', 'fc1.weight', 'fc2.bias', 'fc2.weight']
{'cpu'}
{torch.float32}
[4]
torch.Size([1, 2])
False
True
torch.Size([1, 3])
```

Az ellenőrzés öt pontja: a paraméterszám pontosan 26; a `state_dict` a négy hierarchikus kulcsot tartalmazza; az összes paraméter CPU-n és `float32` formátumban van; az optimizer

paramétercsoportjában a négy regisztrált tenzor szerepel; a külön létrehozott `Dropout` modul kiértékelési módban determinisztikusan azonosságleképezés, tanítási módban pedig a saját `training` flagje szerint maszkol.

11.2.5 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség egy `nn.Parameter`, egy `register_buffer`-rel regisztrált tenzor és egy sima `float` Python-attribútum között. Adjon egy konkrét, nem triviális ellenőrzést, amely a három esetet megkülönbözteti.

F2 (levezetési/bizonyítási, nehézség 2). Egy modell két lineáris rétegből áll: $5 \to 6 \to 3$. Vezesse le (a) a rétegenkénti tenzoralakokat, (b) az összes paraméter számát, és (c) indokolja, hogy egy `register_buffer('scale', torch.ones(1))` hívás után az összes paraméterszám változatlan marad, de a `state_dict()` kulcsszáma eggyel nő. Minden lépésben adja meg az indexhatárokat és a shape-ellenőrzést.

F3 (implementációs, nehézség 2). Írjon `GatedScale` modult, amelynek egyetlen tanulható paramétere a `logit_threshold` skalár. A `forward` az $x \in \mathbb{R}^{N\times 1}$ bemenetre a $y = x \cdot \sigma(t)$ kimenetet adja, ahol σ a szigmoid; a modul egy `pos_count` nevű int64 pufferben számolja, hányszor volt $\sigma(t) > 0{,}5$, a frissítést `no_grad` blokkban végezze. A megoldáshoz adja meg a teljes importot, a modellt és egy kontrollt egy 2×1-es bemenettel.

F4 (hibakeresési, nehézség 2). Tekintse az alábbi hibás osztályt.

```
class Broken(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(3, 4), nn.Linear(4, 2)]

    def forward(self, x):
        for layer in self.layers:
            x = torch.relu(layer(x))
        return x
```

Az `optim = torch.optim.SGD(m.parameters(), lr=0.1)` hívás `ValueError: optimizer got an empty parameter list` kivétellel leáll, mert a `m.parameters()` üres. Magyarázza meg a hiba okát és következményét, javítsa a kódot, és adja meg, milyen kimenettel ellenőrzi, hogy a javítás után a négy paraméter regisztrálva van. Van-e más logikai hiba a `forward`-ban? Ha igen, mi az, és hogyan javítja?

F5 (kísérleti/transzfer, nehézség 3). Tervezzen ellenőrzési protokollt arra, hogy egy `state_dict` mentése és egy második, azonos architektúrájú folyamatban való betöltése után a kimenet bitazonos legyen. Adja meg a seed, a dtype, a device, a modul `train`/`eval` állapota és az összehasonlítás mérőszáma mellett az érvényességi korlátot is: miért nem garantál egyetlen seed minden platformon bitazonosságot? A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

11.2.6 Megoldások és indoklás

F1 megoldás. A `Parameter` a modul tanuló állapota: regisztrálódik, `requires_grad=True`, a `parameters()` felsorolja, az optimizer frissíti. A puffer nem tanuló, de szerves állapot: regisztrálódik, a `parameters()` kihagyja, gradienst nem kap, viszont a `.to()` mozgatja és a `state_dict()` menti. A sima `float` attribútum egyik nyilvántartásba sem kerül: a

`parameters()` nem látja, a `state_dict()` nem menti, és a `.to()` sem mozgatja. Ellenőrzés: egy modulban legyen `self.p = nn.Parameter(torch.zeros(2))`, `self.register_buffer('q', torch.ones(2))` és `self.r = 1.0`. Ekkor `len(list(m.parameters())) == 1`, a nevesített paraméterlista csak `p`; a `state_dict()` kulcsai `p` és `q`, de `r` nem szerepel benne; végül `m.to(torch.float64)` után `m.p.dtype` és `m.q.dtype` is `torch.float64`, míg `m.r` továbbra is Python `float` marad. Ez a három ellenőrzés együtt egyértelműen szétválasztja a három esetet.

F2 megoldás. Jelölje a bemenetet $x \in \mathbb{R}^{N\times 5}$, az indexeket $n = 0, \dots, N-1$ és $i = 0, \dots, 4$. Az első réteg súlya $W_1 \in \mathbb{R}^{6\times 5}$ ($j = 0, \dots, 5$, $i = 0, \dots, 4$), torzítása $b_1 \in \mathbb{R}^6$, ezért

$$z^{(1)}_{n,j} = \sum_{i=0}^{4} W_{1,j,i} x_{n,i} + b_{1,j}, \qquad z^{(1)} \in \mathbb{R}^{N\times 6}.$$

Az aktiváció elemenkénti: $a^{(1)} = \mathrm{ReLU}(z^{(1)})$, így $a^{(1)} \in \mathbb{R}^{N\times 6}$. A második réteg súlya $W_2 \in \mathbb{R}^{3\times 6}$ ($k = 0, \dots, 2$, $j = 0, \dots, 5$), torzítása $b_2 \in \mathbb{R}^3$, tehát

$$z^{(2)}_{n,k} = \sum_{j=0}^{5} W_{2,k,j} a^{(1)}_{n,j} + b_{2,k}, \qquad z^{(2)} \in \mathbb{R}^{N\times 3}.$$

A shape-ellenőrzés a belső dimenziók egyezése: x és W_1 oszlopdimenziója 5, $a^{(1)}$ és W_2 oszlopdimenziója 6. A paraméterszám:

$$(6 \cdot 5 + 6) + (3 \cdot 6 + 3) = 36 + 21 = 57.$$

Kóddal ellenőrizve: `x = torch.zeros(8, 5)` mellett `(x @ W1.T + b1).shape == (8, 6)` és `(torch.relu(x @ W1.T + b1) @ W2.T + b2).shape == (8, 3)`.

A `state_dict()` kulcsai `fc1.weight`, `fc1.bias`, `fc2.weight`, `fc2.bias`. A `register_buffer('scale', torch.ones(1))` a `_buffers`-be ír; a `parameters()` csak a `_parameters` gyűjteményt járja be, ezért az összeg 57 marad, függetlenül a puffer `requires_grad` jelzőjétől (a `torch.ones(1)` esetében ez amúgy is `False`). A `state_dict()` a paramétereket és a perzisztens puffereket (a `register_buffer` alapértéke `persistent=True`) együtt adja, ezért a `scale` kulcs hozzáadódik, és a kulcsszám $4 + 1 = 5$. A puffer alakja `torch.Size([1])`, `numel()`-je 1, de nem számít bele a paraméterszámba.

F3 megoldás.

```
import torch
import torch.nn as nn

class GatedScale(nn.Module):
    def __init__(self):
        super().__init__()
        self.logit_threshold = nn.Parameter(torch.tensor(0.0))
        self.register_buffer('pos_count', torch.tensor(0, dtype=torch.int64))

    def forward(self, x):
        # x: (N, 1)
        s = torch.sigmoid(self.logit_threshold)  # skalár tenzor
        y = x * s
        with torch.no_grad():
            if s.item() > 0.5:
                self.pos_count += 1
        return y
```

```
m = GatedScale()
x = torch.tensor([[1.0], [2.0]])
with torch.no_grad():
    y = m(x)
print(y)              # tensor([[0.5000], [1.0000]]), mert sigmoid(0)=0.5
print(m.pos_count)    # tensor(0), mert a feltétel hamis
print(len(list(m.parameters())))  # 1
print(sorted(m.state_dict().keys()))  # ['logit_threshold', 'pos_count']

# A sigma(t)>0,5 ág kontrollja: t=1,0 esetén sigma(1) 0,731.
with torch.no_grad():
    m.logit_threshold.fill_(1.0)
    y2 = m(x)
print(m.pos_count)    # tensor(1)
assert int(m.pos_count) == 1
```

A szigmoid $s = \sigma(0) = 0{,}5$, így a kapu a bemenet felét adja; a `pos_count` nem növekszik, mert a feltétel hamis. A második kontroll $t = 1{,}0$ mellett a $\sigma(t) > 0{,}5$ ágat is lefedi, ekkor a számláló 1-re nő. A `s.item()` a forwardban Python-szintű elágazást vezérel; ez a `torch.compile`/tracing számára töréspont, mert a feltétel értéke futásidőben adatfüggő. A tanulmányi példában ez elfogadható; termelési kódban a számlálót tenzorműveletteel, például `self.pos_count += (s > 0.5).sum()` alakban is számolhatjuk.

F4 megoldás. A `self.layers` sima Python-lista; a `Module.__setattr__` nem járja be a lista elemeit, ezért a két `Linear` réteg nem kerül a modul `_modules` nyilvántartásába. Az optimizer konstruktora a `m.parameters()` iterációt kapja, amely ebben a példában üres, ezért `ValueError: optimizer got an empty parameter list` kivétellel leáll (futtatva ellenőrizve). Ez a hiba csak addig hangos, amíg a modulnak nincs más regisztrált paramétere: ha például egy közvetlen `self.extra = nn.Parameter(torch.zeros(1))` is szerepel az `__init__`-ben, akkor az optimizer létrejön ezzel az egyetlen paraméterrel, a lista két `Linear` rétege viszont továbbra sem regisztrált, a súlyaik a véletlen inicializáláson maradnak, és a tanítás csendesen nem érinti őket (ellenőrizve: a `param_groups` összesen egy paramétert tartalmaz). A tünet ellenőrzése: `len(list(m.parameters())) == 0` a hibás, és `4` a javított modellnél. A javítás az `nn.ModuleList` használata:

```
self.layers = nn.ModuleList([nn.Linear(3, 4), nn.Linear(4, 2)])
```

Az ellenőrzés `len(list(m.parameters())) == 4`, és `list(m.named_modules())` listában a két lineáris almodul szerepel. A második hiba: a `forward` az utolsó, 2 kimenetű réteg után is ReLU-t alkalmaz, tehát a modell nem képes negatív logitot kiadni; a javítás:

```
def forward(self, x):
    x = torch.relu(self.layers[0](x))
    return self.layers[1](x)
```

F5 megoldásminta. Rögzített környezet: `torch.manual_seed(0)`, `torch.set_num_threads(1)`, és a szálszámot a betöltő folyamatban is ellenőrizzük `torch.get_num_threads()` hívással. Ha a modellben vagy a forwardban nemdeterminisztikus művelet lehet, kapcsoljuk be a `torch.use_deterministic_algorithms(True)` beállítást (GPU-n ehhez a `CUBLAS_WORKSPACE_CONFIG` környezeti változó is szükséges). A protokoll: a modellt CPU-n, `float32` dtype-dal, `eval()` módban tartjuk, így a dropout és a batchnorm viselkedése rögzített; determinisztikus bemenettel, például `torch.ones(1, 3)`-mal kiszámítjuk és elmentjük a referencia kimenetet; `torch.save(model.state_dict(), 'mlp342.pt')`; egy második Python-folyamatban létrehozzuk az azonos `MLP342` architektúrát, betöltjük `torch.load('mlp342.pt', map_location='cpu', weights_only=True)` és `load_state_dict(..., strict=True)` hívással (a szigorú betöltés a hiányzó és a váratlan kulcsokat is hibaként jelzi); ugyanazon bemenetre a kimenetet `torch.equal` mérőszámmal hasonlítjuk a referenciához. A betöltés után

az inicializálási seed már nem számít, mert minden paraméter értékét a mentett `state_dict` adja; a seed és a szálszám a forward esetleges véletlen műveletei miatt marad a protokoll része. Az érvényességi korlát: a bitazonosság csak azonos PyTorch-verzió, processzorarchitektúra, BLAS-implementáció és szálszám mellett várható; más platformon a lebegőpontos összegzés sorrendje és a FMA használata eltérhet, ezért platformok közötti bitazonosságot nem garantálunk, csak numerikus egyezést kis hibakorláttal.

11.2.7 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés a sima Python-listás almodulok használata. A

```
self.layers = [nn.Linear(3, 4), nn.Linear(4, 2)]
```

sorban a lista elemei nem regisztrálódnak a modulba; az `m.parameters()` üres. Az `nn.ModuleList([...])` viszont elemeit bejegyzi, ezért a paraméterek megjelennek. A lista és a `ModuleList` tehát nem felszíni stilisztikai különbség: az egyik a tanulásban részt vevő paraméterek regisztrációját veszíti el.

Önellenőrzés. (1) Ha egy `requires_grad=False` tenzort regisztrálunk pufferként (`m.register_buffer('q', t)`), mi lesz a `q.requires_grad` és a `q.grad` értéke a `backward` után? (Válasz: hamis, illetve `None`; a regisztráció maga nem állítja át a jelzőt.) (2) A `state_dict()` ment-e autograd gráfot? (Válasz: nem, csak értékeket; a `load_state_dict` a meglévő paraméterekbe másol, nem hoz létre új tenzorokat, és a `missing_keys`/`unexpected_keys` mezőkkel jelzi az eltéréseket.) (3) A `.to(device)` mit helyez át? (Válasz: a paramétereket és a puffereket is, miközben az almodulok közötti névstruktúra változatlan marad.) A korábbi 11.1. Tensor és autograd-életciklus fogalmait visszahívva: a `Parameter` levéltensor, és a `.grad` felhalmozódás nullázása továbbra is az optimizer feladata.

Korábbi kapcsolat. A 10.2. Affin réteg és aktiváció backward egységben levezetett $Z = XW^\top + b$ mellett a súlygradiens $dW = G^\top X$, a biasgradiens $db = \sum_n G_{n,:}$. Az `nn.Linear` ugyanezt a transzponált konvenciót használja, ezért a kézi VJP-k változatlanul alkalmazhatók; a modul csupán a tenzorok regisztrációját és nevesítését adja hozzá.

11.2.8 Rövid összefoglaló

Az `nn.Module` a tenzorokból nevesített, menthető modellállapotot épít. A `Parameter` a modul `_parameters` nyilvántartásába kerül, `requires_grad=True`, ezért az optimizer és a `state_dict` látja. A `register_buffer` tenzora a `_buffers`-be kerül: nem tanul, gradienst nem kap, de a `.to()` áthelyezi és a mentés tartalmazza. A `forward` a számítást adja, a `train()`/`eval()` a `training` flaget rekurzívan állítja; a `Dropout` csak tanítási módban nullázza az elemek egy részét. A labor 3-4-2 MLP-n ellenőrizte a 26 paramétert, a kulcsokat, az eszközt és az eval-módú identitást. A Python-listás almodulok kihagyása tipikus regisztrációs hiba.

11.2.9 Következő alkalmazás és források

A következő 11.3. Dataset és DataLoader a split után egységben a betöltőt építjük fel: a modell paraméterei már regisztrálva vannak, most az adat batch-ekre bontása és a split utáni iteráció következik, majd a 11.4. Egy teljes PyTorch tanítási iteráció alfejezetben a teljes tanítási lépés összeáll.

- [D06] PyTorch contributors (2026). *Module*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Module.html. Hozzáférés: 2026-10-30.

11.3 Dataset és DataLoader a split után

11.3.1 Cél, előismeret és induló kérdés

A 11.2. nn.Module és paraméterregisztráció alfejezetben a modell paramétereit már regisztráltuk, és a `state_dict` menthetővé vált. Most azt az adatutat építjük meg, amely a rögzített train/val/test splitből tanulási batch-eket állít elő anélkül, hogy a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab jelenetszintű függetlenségi szerződését megsértené. Az előismeret a tenzoralakok és a `nn.Module` használata; a veszteség- és célalakok szerződését a 7.3. Bináris, multilabel és regressziós cél adta.

Induló kérdés: egy `Dataset` osztály `__getitem__`-je visszaad egy képet, egy címkét és egy maszkot. Ki garantálja, hogy a `DataLoader` által összeállított batch-ben nem keveredik a train és a val képe, és hogy egy jelenet két nézete mindig ugyanabba a splitbe kerül?

11.3.2 Dataset, sampler, batch és DataLoader

A PyTorch `torch.utils.data` moduljában a **map-style dataset** egy indexelt hozzáférést adó objektum: adott $i \in \{0, \dots, N-1\}$ indexre visszaadja az i-edik mintát a `__getitem__(i)` metódus, a `__len__` pedig a minták számát. A GlyphLab esetében egy minta nem csak egy kép, hanem egy négyes:

$$(\text{kép},\ \text{címke},\ \text{maszk},\ \text{scene_id}),$$

ahol a kép alakja $(1, 24, 24)$ és `float32`, a címke skalár `int64`, a maszk $(1, 24, 24)$ és `uint8`, a `scene_id` skalár `int64` kód: a `metadata.csv` `scene_id` oszlopa a függelék szerint szöveges (`G<label>-<scene>` alakú, például `G0-0003`); a dataset ezt a rendezett egyedi értékek sorszámára kódolja, az eredeti szöveget a `scene_names` tömb őrzi. A batch-elés után az alakok $(B, 1, 24, 24)$, $(B,)$, $(B, 1, 24, 24)$ és $(B,)$ lesznek.

A **sampler** az indexek sorozatát adja meg. A `SequentialSampler` a $0, 1, \dots, N-1$ sorrendet, a `RandomSampler` egy véletlen permutációt állít elő; a `BatchSampler` ezt B hosszú csoportokra bontja. A **DataLoader** a dataset, a sampler és a `collate_fn` összekapcsolása ([D08]): a sampler indexeit bekéri a datasetből, a visszaadott elemek listáját a `collate_fn` tenzorrészletekből batchelt tensorokká alakítja. Az alapértelmezett `default_collate` a tuple minden pozíciójára külön-külön stackel, ezért a skalár címkéből $(B,)$ alakú `int64` tensor lesz.

A `shuffle=True` a `DataLoader` konstruktorban csak egy rövidítés: a `RandomSampler`-t kapcsolja be. A `shuffle` és a `sampler` argumentum együtt nem adható meg, mert mindkettő ugyanazt az indexelési feladatot oldaná meg. A `drop_last=True` az utolsó, B-nél kisebb batch-et eldobja; ez a nagyon kicsi (például egyelemű) utolsó batch elkerülésére lehet hasznos, mert a batch-normalizáló rétegek tanítási módban ilyen batch-en instabilak vagy hibát adnak. A GlyphLab esetében a 240, 60 és 60 mintás split a labor $B = 4$ batch méretével mind osztható, így ott nincs ilyen maradék.

A **worker seed** kérdése a párhuzamos betöltéshez tartozik. Ha a `num_workers` nagyobb nullánál, minden worker külön folyamatban fut, és a véletlen mintavétel állapota nem automatikusan azonos a fő folyamatéval. A [D08] `torch.utils.data` dokumentációja szerint a `DataLoader` `generator` paramétere adja a sampler és a workerek bázisseedjének véletlen forrását, a `worker_init_fn` pedig minden workerben a saját seed beállítására szolgál. A tipikus minta (szemléltetés, nem futtatott mérés):

```python
import random

import numpy as np
import torch
from torch.utils.data import DataLoader

g = torch.Generator()
g.manual_seed(20261020)

def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)

# ds: a fent definiált GlyphLabDataset példány
loader = DataLoader(ds, batch_size=4, shuffle=True,
                    num_workers=2, generator=g,
                    worker_init_fn=seed_worker)
```

A `generator` és a `worker_init_fn` együtt biztosítja, hogy a keverés és a workerek saját véletlen állapota is reprodukálható legyen; a részletes viselkedés a dokumentált verzióhoz kötött. A jelen egység laborja `num_workers=0` értékkel fut, így az indexelés a fő folyamatban marad, és a seed kérdése nem befolyásolja az eredményt.

A **transform boundary** azt a határt jelöli, ahol a kép- és címketranszformáció megtörténik. A véletlen train-augmentáció a train `Dataset __getitem__`-jében alkalmazandó, mert ott a transzformáció mintánként, a betöltés pillanatában fut. A val és test dataset csak determinisztikus transzformációt kaphat; a geometriai augmentációnak a maszkot is együtt kell transzformálnia, különben a pixelcímke és a bemenet elcsúszik. A GlyphLab diagonal osztálya csak egy ferde irányt tartalmaz, ezért a tükrözés és a 90 fokos forgatás nem címkemegőrző; ebben az egységben a labor semmilyen augmentációt nem alkalmaz, csak a betöltési szerződést ellenőrzi.

A GlyphLab három orientációs osztálya a függelék A. A GlyphLab adatkészlet szerint 0 = `vertical` (függőleges), 1 = `horizontal` (vízszintes), 2 = `diagonal` (ferde). A záró kiértékelés clean–korrupt párja ugyanazt a 60 tesztképet használja: a `corrupted_test.npy` a `metadata.csv` test-sorrendjében ugyanazokat a jelenet–nézet párokat tartalmazza zajos, fényességi gradienssel torzított változatban, ezért a tiszta és a korrupt feltétel képenként párosítható.

11.3.3 A split-invariáns levezetése

A GlyphLab adatszerződésének központi állítása, hogy a megfigyelési egység a `scene_id`, nem az egyes kép. Ezt formálisan a következő invariáns írja le.

Állítás (jelenet–split invariáns). Legyen $\mathcal{I} = \{0, \dots, 359\}$ a képek indexhalmaza, $\pi : \mathcal{I} \to \mathcal{S}$ a `scene_id` hozzárendelés, és $\sigma : \mathcal{I} \to \{\text{train}, \text{val}, \text{test}\}$ a split hozzárendelés. A GlyphLab generátor olyan $\tau : \mathcal{S} \to \{\text{train}, \text{val}, \text{test}\}$ függvényt valósít meg, amelyre minden $i \in \mathcal{I}$ esetén

$$\sigma(i) = \tau(\pi(i)).$$

Ebből következik, hogy minden $i, j \in \mathcal{I}$-re

$$\pi(i) = \pi(j) \implies \sigma(i) = \sigma(j).$$

Levezetés. Ha $\pi(i) = \pi(j) = s$, akkor

$$\sigma(i) = \tau(\pi(i)) = \tau(s) = \tau(\pi(j)) = \sigma(j),$$

tehát az azonos jelenethez tartozó képek splitje megegyezik. A τ függvény létezése a generátor jelenetszintű splitjének formális alakja: a splitet először a 180 jelenetre sorsoljuk, majd minden jelenet minden nézetét ehhez a splithez rendeljük. Ezt a tulajdonságot a generátor adatszerződésére tett feltevésnek tekintjük, nem a generátor kódjából levezetett tételnek; a levezetés csak azt mutatja meg, hogy a feltevésből következik a képpárok split-egyezése. A `DataLoader` ezt az invariánst nem ronthatja el, mert a sampler csak indexeket permutál, és a három loader külön indexhalmazt kap a `split` szűrés után. Az invariáns ellenőrzése tehát két részből áll: a `metadata.csv` jelenetszintű konzisztenciájából és abból, hogy a dataset konstruktora a `split` oszlop alapján szűr, nem keveri a sorokat. A labor a feltevést csak a következményén keresztül ellenőrzi: a `metadata.csv` jelenetszintű konzisztenciáját vizsgálja, a generátor forráskódját nem.

11.3.4 Kézi kis példa

Négy jelenet, jelenetenként két nézet, összesen nyolc kép. A helyes split-hozzárendelés:

i	scene_id $\pi(i)$	nézet	split $\sigma(i)$
0	0	0	train
1	0	1	train
2	1	0	val
3	1	1	val
4	2	0	test
5	2	1	test
6	3	0	train
7	3	1	train

Itt $\tau(0) = \tau(3) = \text{train}$, $\tau(1) = \text{val}$, $\tau(2) = \text{test}$. Ha a 0. képet train-be, az 1. képet val-ba tennénk, akkor $\pi(0) = \pi(1)$ teljesülne, de $\sigma(0) \neq \sigma(1)$, tehát az invariáns sérülne; ekkor a val lossba a train képekkel azonos jelenetből származó, azoktól nem független nézetek is bekerülnének, és ez a val metrikát optimistán torzítaná.

11.3.5 Labor: GlyphLab betöltése és a szerződés ellenőrzése

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/` könyvtárának `images.npy`, `labels.npy`, `masks.npy` és `metadata.csv` fájlja; a `n043_dataset.py` és `n043_check.py` a `lab/` könyvtárba kerül, futtatás: `python n043_check.py`.

Munkakönyvtár: `lab/`. Az adatfájlok a `data/` alkönyvtárban vannak: `images.npy`, `labels.npy`, `masks.npy`, `metadata.csv`; az útvonalakat a szkript a saját helyéhez képest oldja fel, ezért a `lab/` könyvtárból futtatva is ugyanazokat a fájlokat találja. A labor csak CPU-t és NumPy/PyTorch alapműveleteket használ; seedre a betöltés miatt nincs szükség, mert a betöltés determinisztikus. A kód `str | Path` és `str | None` uniótípus-jelölést használ, amely Python 3.10-től érhető el; a labor Python 3.12.3 alatt fut.

```
# lab/n043_dataset.py
import csv
from pathlib import Path

import numpy as np
```

```python
import torch
from torch.utils.data import Dataset

DATA_DIR = Path(__file__).resolve().parent / "data"

class GlyphLabDataset(Dataset):
    """GlyphLab: kép, label, mask és scene_id együtt, split szerint szűrve.

    transform: opcionális függvény, amely a (2,24,24) alakú, egymásra rakott
    [kép, maszk] tensort kapja és ugyanilyet ad vissza; így egy geometriai
    transzformáció a képet és a maszkot együtt, ugyanúgy alakítja át."""
    def __init__(self, root: str | Path = DATA_DIR, split: str | None = None,
                 transform=None):
        self.root = Path(root)
        self.transform = transform
        images = np.load(self.root / "images.npy")      # (360,1,24,24) float32
        labels = np.load(self.root / "labels.npy")      # (360,) int64
        masks = np.load(self.root / "masks.npy")        # (360,1,24,24) uint8
        self.images = images
        self.labels = labels
        self.masks = masks

        with open(self.root / "metadata.csv", newline="", encoding="utf-8") as f:
            rows = list(csv.reader(f))
        header = rows[0]
        expected = ["index", "scene_id", "view_id", "label", "split",
                    "xmin", "ymin", "xmax", "ymax"]
        if header != expected:
            raise ValueError("metadata fejléc nem egyezik")
        recs = rows[1:]
        if not (len(images) == len(labels) == len(masks) == len(recs)):
            raise ValueError("eltérő hosszúságú adattömb és metadata")
        idx = np.array([int(r[0]) for r in recs], dtype=np.int64)
        if not np.array_equal(idx, np.arange(len(images))):
            raise ValueError("a metadata indexe nem egyezik a tömb indexével")
        # a scene_id szöveg (pl. "G0-0003"); int64 kódot a rendezett egyedi értékek adnak
        self.scene_names = np.array([r[1] for r in recs], dtype=object)
        _, codes = np.unique(self.scene_names.astype(str), return_inverse=True)
        self.scene_ids = codes.astype(np.int64)
        self.splits = np.array([r[4] for r in recs], dtype=object)

        if split is not None:
            keep = self.splits == split
            self.images = self.images[keep]
            self.labels = self.labels[keep]
            self.masks = self.masks[keep]
            self.scene_ids = self.scene_ids[keep]
            self.scene_names = self.scene_names[keep]
            self.splits = self.splits[keep]

    def __len__(self):
        return len(self.images)

    def __getitem__(self, i):
        img = torch.from_numpy(self.images[i])
        lab = torch.tensor(self.labels[i], dtype=torch.int64)
        msk = torch.from_numpy(self.masks[i])
        sid = torch.tensor(self.scene_ids[i], dtype=torch.int64)
        if self.transform is not None:
            joint = self.transform(torch.cat([img, msk.to(img.dtype)], dim=0))  # (2,24,24)
            img, msk = joint[:1].contiguous(), joint[1:].round().to(torch.uint8)
        return img, lab, msk, sid
```

Az ellenőrző szkript a split-invariánst is teszteli. A `scene_id` kódolása a teljes `metadata.csv`-n, a split szerinti szűrés előtt történik, ezért a kódok splitfüggetlenek: ugyanaz a jelenet ugyanazt a `scene_id` kódot kapja a train, a val és a test datasetben.

```
# lab/n043_check.py
import csv
from collections import defaultdict

import numpy as np
from torch.utils.data import DataLoader
from n043_dataset import DATA_DIR, GlyphLabDataset

if __name__ == "__main__":
    ds_train = GlyphLabDataset(DATA_DIR, split="train")
    ds_val = GlyphLabDataset(DATA_DIR, split="val")
    ds_test = GlyphLabDataset(DATA_DIR, split="test")
    print("hossz:", len(ds_train), len(ds_val), len(ds_test))
    assert (len(ds_train), len(ds_val), len(ds_test)) == (240, 60, 60)

    img, lab, msk, sid = ds_train[0]
    print("elem:", img.shape, img.dtype, lab.shape, lab.dtype,
          msk.shape, msk.dtype, sid.shape, sid.dtype)
    print("tartomány:", float(img.min()), float(img.max()),
          int(msk.min()), int(msk.max()))
    lo = min(float(ds.images.min()) for ds in (ds_train, ds_val, ds_test))
    hi = max(float(ds.images.max()) for ds in (ds_train, ds_val, ds_test))
    print("teljes pixel tartomány:", lo, hi)
    assert lo >= 0.0 and hi <= 1.0
    assert all(set(np.unique(ds.masks)) <= {0, 1}
               for ds in (ds_train, ds_val, ds_test))

    loader = DataLoader(ds_train, batch_size=4, shuffle=False)
    batch = next(iter(loader))
    for name, x in zip(["image", "label", "mask", "scene_id"], batch):
        print(name, tuple(x.shape), x.dtype)
    assert [tuple(x.shape) for x in batch] == [(4, 1, 24, 24), (4,),
                                               (4, 1, 24, 24), (4,)]

    scene_split = {}
    with open(DATA_DIR / "metadata.csv", newline="", encoding="utf-8") as f:
        for r in list(csv.reader(f))[1:]:
            s, sp = r[1], r[4]          # scene_id szövegként
            if s in scene_split and scene_split[s] != sp:
                raise SystemExit("INVARIANTS SÉRÜL")
            scene_split[s] = sp
    print("egyedi scene:", len(scene_split))
    assert len(scene_split) == 180
    splits = defaultdict(int)
    for s in scene_split.values():
        splits[s] += 1
    print("scene per split:", dict(splits))
    assert dict(splits) == {"train": 120, "val": 30, "test": 30}
```

A futtatási parancs a `lab/` munkakönyvtárból:

```
python n043_check.py
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál; a betöltés determinisztikus, két futás kimenete bitazonos):

```
hossz: 240 60 60
elem: torch.Size([1, 24, 24]) torch.float32 torch.Size([]) torch.int64 torch.Size([1, 24, 24])
↪  torch.uint8 torch.Size([]) torch.int64
```

```
tartomány: 0.0 0.9264628887176514 0 1
teljes pixel tartomány: 0.0 1.0
image (4, 1, 24, 24) torch.float32
label (4,) torch.int64
mask (4, 1, 24, 24) torch.uint8
scene_id (4,) torch.int64
egyedi scene: 180
scene per split: {'train': 120, 'val': 30, 'test': 30}
```

Ez egyezik a szerződéssel: a hosszak $240, 60, 60$; az elem alakjai $(1, 24, 24)$ float32, $()$ int64, $(1, 24, 24)$ uint8 és $()$ int64; a batch alakjai $(4, 1, 24, 24)$, $(4,)$, $(4, 1, 24, 24)$, $(4,)$; a teljes train/val/test pixelállomány a $[0, 1]$ intervallumban van, a maszkértékek $\{0, 1\}$-ben; az egyedi jelenetek száma 180, és a jelenetek splitenkénti száma $\{\text{train} : 120, \text{val} : 30, \text{test} : 30\}$.

11.3.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a `Dataset` és a `DataLoader` különbségét. Miért nem elegendő önmagában egy `Dataset` a minibatch-tanításhoz? Adjon egy kis ellenpéldát, ahol a `Dataset` indexelése működik, de a batchelt iteráció hibát adna.

F2 (levezetési, nehézség 2). Hat képhez adott a következő jelenethozzárendelés: $\pi(0) = \pi(1) = 0$, $\pi(2) = 1$, $\pi(3) = \pi(4) = \pi(5) = 2$, és a nézetindexek rendre $0, 1, 0, 0, 1, 2$. Adjon egy olyan split-hozzárendelést, amely teljesíti a jelenet–split invariánst, és egy olyat, amely sérti. Indokolja mindkettőt a τ függvény segítségével.

F3 (implementációs, nehézség 2). Írja meg a `GlyphLabDataset` osztályt a fenti szerződés szerint, majd ellenőrizze: a train dataset hossza 240; a `ds_train[0]` elemeinek alakja és `dtype`-ja pontosan $(1, 24, 24)$ `torch.float32`, $()$ `torch.int64`, $(1, 24, 24)$ `torch.uint8`, $()$ `torch.int64`; a `DataLoader(ds_train, batch_size=4, shuffle=False)` első batch-ének alakjai $(4, 1, 24, 24)$, $(4,)$, $(4, 1, 24, 24)$, $(4,)$. A dtype-okra és az alakokra is szigorú egyezést várunk: itt nem numerikus értéket hasonlítunk, ezért nincs értelmezhető tolerancia, hanem a tenzor metaadata egyezik vagy nem.

F4 (hibakeresési, nehézség 2). A következő kódrészlet a val loaderhez véletlen tükrözést ad:

```python
def random_hflip(x, p=0.5):
    # véletlen vízszintes tükrözés a (2,24,24) [kép, maszk] tensoron
    return torch.flip(x, dims=(-1,)) if torch.rand(()) < p else x

val_ds = GlyphLabDataset(DATA_DIR, split="val", transform=random_hflip)
val_loader = DataLoader(val_ds, batch_size=4, shuffle=False)
```

Azonosítsa a hibát, írja le a következményét a validation loss-ra és a korrupt teszt páros összevetésére, és adja meg a javítást. Hogyan ellenőrizné vissza, hogy a javítás után a val loader determinisztikus?

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy olyan kísérletet, amely a meglévő GlyphLab splittől független, új jelenetszintű train/val/test splitet készít 70/15/15 arányban, majd a tanítás után a test accuracy-t méri. Adja meg a protokollt: megfigyelési egység, baseline, változó tényező, metrika, erőforrás és a következtetés érvényességi korlátja. Nem szükséges futtatni; a terv legyen végrehajtható. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

11.3.7 Megoldások és indoklás

F1. A `Dataset` egy indexelt adathozzáférés: az i indexre mintát ad, de nem tud batch méretet, sorrendet vagy párhuzamos betöltést kezelni. A `DataLoader` a sampler által adott indexsorozatot bekéri, a mintákat a `collate_fn` segítségével batchelt tensorokká alakítja, és opcionálisan több workerrel dolgozik. Ellenpélda: ha a `Dataset.__getitem__` különböző hosszúságú szekvenciákat ad vissza, az indexelés működik, de az alapértelmezett `default_collate` nem tudja stackelni a különböző hosszúságú tenzorokat, ezért a `DataLoader` iterációja hibát ad; ilyenkor egyedi `collate_fn` vagy padding szükséges.

```
import torch
from torch.utils.data import Dataset, DataLoader

class VariableLength(Dataset):
    def __len__(self):
        return 3

    def __getitem__(self, i):
        return torch.arange(i + 1)   # 1, 2, 3 hosszú vektor

ds = VariableLength()
print(ds[0].shape, ds[1].shape, ds[2].shape)
# torch.Size([1]) torch.Size([2]) torch.Size([3])
list(DataLoader(ds, batch_size=2))
# RuntimeError: stack expects each tensor to be equal size,
# but got [1] at entry 0 and [2] at entry 1
```

A futtatott kimenet (a kivétel üzenete rövidítve, a traceback nélkül):

```
torch.Size([1]) torch.Size([2]) torch.Size([3])
RuntimeError: stack expects each tensor to be equal size, but got [1] at entry 0 and [2] at
↪  entry 1
```

F2. A helyes τ függvény például $\tau(0) = \text{train}$, $\tau(1) = \text{val}$, $\tau(2) = \text{test}$. Ekkor a split-hozzárendelés: $0, 1 \to \text{train}$; $2 \to \text{val}$; $3, 4, 5 \to \text{test}$. Az invariáns teljesül, mert azonos jelenethez tartozó indexek azonos splitet kapnak. Sértő példa: $\sigma(0) = \text{train}$ és $\sigma(1) = \text{val}$, miközben $\pi(0) = \pi(1) = 0$. Itt nem létezhet olyan τ, amelyre $\sigma(i) = \tau(\pi(i))$ minden i-re, mert $\tau(0)$ egyszerre train és val lenne; ezért a levezetett invariáns sérül.

F3. A megoldás a fenti `n043_dataset.py` és `n043_check.py`. Az ellenőrzés kulcsa, hogy a `torch.from_numpy` megőrzi a NumPy dtype-ot: `float32` → `torch.float32`, `uint8` → `torch.uint8`. A címke és a `scene_id` explicit `torch.tensor(..., dtype=torch.int64)` módon int64 lesz, így a `default_collate` stackelés után is int64 marad. A shape-ek pontos egyezése azt jelenti, hogy nincs engedmény: $(1, 24, 24)$ nem lehet $(24, 24)$; mivel itt nem numerikus értéket hasonlítunk, tolerancia helyett típus- és alakegyezést követelünk. A futtatási parancs `python n043_check.py` a `lab/` könyvtárból.

F4. A hiba: a val dataset transzformációja véletlen augmentáció, ami a transform boundary megsértése. A `random_hflip` $p = 0{,}5$ valószínűséggel tükrözi a képet (a `__getitem__` a `transform`-ot ténylegesen meghívja, a maszkkal együtt); a GlyphLab diagonal osztálya csak egy ferde irányt tartalmaz, ezért a tükrözés a címke geometriai jelentését megváltoztatja: a ferde vonal iránya az ellenkezőjére fordul, miközben a vertical és horizontal osztály jelentése nem cserélődik fel. Következmény: a val loss nem determinisztikus, a best checkpoint kiválasztása zajos; a clean és corrupt teszt páros összevetése is torzul, mert ugyanazon tesztkép tiszta és korrupt változata két külön `__getitem__`-hívásban egymástól függetlenül kap tükrözést, így a képenkénti pár egyik tagja tükrözött, a másik nem. Javítás: a val dataset ne kapjon véletlen transzformációt

(`transform=None`); csak determinisztikus normalizálást, vagy semmit. Visszaellenőrzés ismételt `__getitem__`-hívásokkal és két teljes iterációval:

```
torch.manual_seed(0)
bad = GlyphLabDataset(DATA_DIR, split="val", transform=random_hflip)
print(any(not torch.equal(bad[0][0], bad[0][0]) for _ in range(20)))   # True: véletlen
good = GlyphLabDataset(DATA_DIR, split="val")                            # javítva
print(all(torch.equal(good[0][0], good[0][0]) for _ in range(20)))     # True: determinisztikus
a = torch.cat([b[0] for b in DataLoader(good, batch_size=4)])
c = torch.cat([b[0] for b in DataLoader(good, batch_size=4)])
print(torch.equal(a, c))                                                 # True
```

F5. A megfigyelési egység a `scene_id`, nem a kép: a 180 jelenet osztályonként 60-60-60, és minden jelenet mindkét nézete együtt kerül a splitjébe. A rétegzés szempontja a `label` osztálya; osztályonként 70/15/15 arányban 42 train, 9 val és 9 test jelenetet sorsolunk, ami összesen $3 \cdot 42 = 126$ train, $3 \cdot 9 = 27$ val és $3 \cdot 9 = 27$ test jelenetet ad (a képek száma ezek kétszerese: 252/54/54). A baseline ugyanaz a modell és tanítási recept a gyárilag rögzített 120/30/30-as jelenet–splitten; a változó tényező csak a split sorsolása. A modell egy rejtett rétegű MLP a 10.2. Affin réteg és aktiváció backward szerint: a $1 \times 24 \times 24$ képet 576-dimenziós vektorrá lapítjuk, 16 tanh rejtett egység következik, a kimenet 3 logit; a veszteség a 7.2. Softmax és többosztályos cross-entropy szerinti softmax cross-entropy, az optimizer SGD rögzített tanulási rátával, a batch mérete 16, a legjobb checkpointot a val loss választja. A metrika a test accuracy jelenet szintű bootstrap konfidenciaintervallummal; az erőforrás CPU. Az érvényességi korlát: a 70/15/15 sorsolás tesztje nem azonos a gyári teszt halmazával, ezért a két pontosság különbsége nem páros összehasonlítás, és önmagában nem bizonyítja egyik split fölényét; a split-variabilitást öt seedelt ismétléssel kell becsülni, az eredmény pedig a 180 jelenetes szintetikus GlyphLab-adatra érvényes, nem természetes képi benchmarkra.

11.3.8 Tipikus tévedés és ellenpélda

A leggyakoribb hiba a **validation/test loaderen alkalmazott véletlen train-augmentáció**. Ha a val vagy test dataset `__getitem__`-je véletlen tükrözést, forgatást vagy kivágást kap, akkor az értékelés nem a modell képességét, hanem egy véletlen transzformációval zajosított bemenetet mér. A páros korrupt teszt összevetése különösen érzékeny: a clean és corrupt kép ugyanazon jelenet és nézet párja, ezért a geometriai transzformációnak is azonosnak kell lennie; a véletlen augmentáció ezt a párosítást felbontja, és a clean–corrupt különbséghez mesterséges varianciát ad. A transform boundary tehát split-függő szerződés: véletlen transzformáció csak a train datasetben, a val/test datasetben kizárólag determinisztikus előfeldolgozás.

11.3.9 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrizze, hogy a dataset konstruktora a `split` oszlop alapján szűr, és a `metadata.csv` fejlécét, valamint az indexsorrendet is validálja. A `DataLoader` batch alakjait a `default_collate` szabálya szerint vezesse le: a tuple első eleme $(1, 24, 24)$, a stackelés után $(B, 1, 24, 24)$; a skalár címkéből $(B,)$ lesz. A 11.2. nn.Module és paraméterregisztráció alfejezetben bevezetett `state_dict` most az adatbetöltővel együtt lesz használható a tanítási lépésben. A `CrossEntropyLoss` nyers logitokat vár (N, K) alakban, a cél pedig $(N,)$ alakú `int64` osztályindex, nem one-hot vektor; ezért a batch `label` mezője $(B,)$ int64 marad, és a modell kimenetére a veszteség előtt nem teszünk softmaxot. A 7.3. Bináris, multilabel és regressziós cél alfejezetben rögzített int64 célalak és logits-bemenet szerződése így a 11.4. Egy teljes PyTorch tanítási iteráció alfejezetben találkozik a betöltött batch-ekkel.

Az induló kérdésre a válasz. A split a `scene_id` szerint dől el, ezért a train és a val képei nem keveredhetnek, és egy jelenet két nézete mindig ugyanabba a splitbe kerül; a GlyphLab szerződése szerint ez 120/30/30 jelenet, azaz 240/60/60 kép.

11.3.10 Rövid összefoglaló

A `Dataset` indexelt mintahozzáférést ad, a `DataLoader` a sampler indexsorozatát batchelt tensorokká alakítja a `collate_fn` segítségével. A GlyphLab jelenet–split invariánsa formálisan azt mondja ki, hogy azonos `scene_id`-hez tartozó képek splitje megegyezik; a τ függvény létezése biztosítja, hogy a split jelenetszinten, nem képenként dől el. A labor betölti a train/val/test dataseteket, ellenőrzi a hosszakat, a dtype-okat és az alakokat, valamint a metadata jelenetkonzisztenciáját. A transform boundary szerint véletlen augmentáció csak a train datasetben megengedett; a val/test loader determinisztikus transzformációt kaphat. A következő 11.4. Egy teljes PyTorch tanítási iteráció egységben ezt a betöltőt kapcsoljuk össze a tanítási iterációval és a loss-számítással.

11.3.11 Források

- [D08] PyTorch contributors (2026). *torch.utils.data*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.

11.4 Egy teljes PyTorch tanítási iteráció

11.4.1 Cél, előismeret és induló kérdés

Ez az egység a 11.3. Dataset és DataLoader a split után adatbetöltőjét kapcsolja össze a tanítási ciklussal. Egy batchre felírjuk a teljes PyTorch-lépést: `zero_grad`, forward, loss, `backward`, gradiens-ellenőrzés, `optimizer.step`. A cél annak pontos megértése, hogy az autograd által előállított gradiens mikor és hogyan válik paraméterfrissítéssé. Ebben az egységben az **iteráció** egyetlen optimizer-lépést jelent egy batchre; a szövegben ugyanezt tanítási lépésnek is nevezzük, szemben az epochkal, amely a teljes tanítóhalmaz egyszeri bejárása.

Előismeret: 11.3. Dataset és DataLoader a split után Dataset/DataLoader, 11.2. nn.Module és paraméterregisztráció `nn.Module`, 11.1. Tensor és autograd-életciklus autograd-életciklus. Visszautalás: 10.4. Tanítási lépés és tanuló MLP a kézzel írt tanítási lépés, 7.4. Numerikus stabilitás és jelentésmegőrző implementáció a stabil loss-számítás.

Induló kérdés: ha `loss.backward()` sikeresen lefutott, ettől miért maradhat a súly változatlan? A rövid válasz: a `backward` csak a `.grad` tárolókat tölti fel, a paramétereket kizárólag a külön meghívott `optimizer.step()` írja; a sorrend részleteit a 11.4.2, a tipikus hibát és ellenpéldáját a 11.4.7 szakasz tárgyalja.

11.4.2 A tanítási lépés mint sorrendi szerződés

Egy batchre a következő hívássorrendet rögzítjük:

```
1. optimizer.zero_grad()       # a .grad tárolók nullázása
2. logits = model(x)           # forward
3. loss = criterion(logits, y) # skalár loss
4. loss.backward()             # gradiens a levelek .grad mezőjében
5. (opcionális) gradiens-ellenőrzés / clip
6. optimizer.step()            # paraméterfrissítés
```

Az 1. lépés azért előzi meg a 4.-et, mert a `backward` a `.grad` mezőben *akkumulál*, nem felülír. Egy elmaradt `zero_grad` hatására több batch gradiense összeadódik, ami nem az aktuális minibatch SGD-frissítés.

A felsorolás 5. lépése opcionális: a gradiens-klip (gradient clipping) a gradiens normáját egy küszöb alá csonkolja, mielőtt a `step` felhasználná. A véges-gradiens ellenőrzés ezzel szemben nem módosítja a gradienst, hanem megállítja a futást.

11.4.3 Definíciók, alakok és a kapcsolódó objektumok

Logits. Az affin réteg aktiváció előtti kimenete $Z \in \mathbb{R}^{N\times K}$. Egy osztályozó fejben

$$Z = XW^\top + b,$$

ahol $X \in \mathbb{R}^{N\times d}$, $W \in \mathbb{R}^{K\times d}$, $b \in \mathbb{R}^K$. A logit nem valószínűség, és a célvektor $y \in \{0,\dots,K-1\}^N$ osztályindexekből áll.

Loss. A főúton a [D07] szerinti `CrossEntropyLoss`-t használjuk logits bemenettel és int64 célindexekkel; softmaxot nem hívunk külön. A `reduction="mean"` konvenció:

$$L(Z,y) = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{z_{i,y_i}}}{\sum_{k=1}^{K} e^{z_{i,k}}}.$$

A skalár loss egy zéró dimenziós tensor. A 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben tárgyalt logsumexp-stabilizálást a könyvtári implementáció végzi; a levezetésben a matematikai képlettel dolgozunk.

`zero_grad`. Az optimizer minden hozzárendelt paraméter `.grad` tárolóját nullázza vagy `None`-ra állítja. Az alapértelmezés `set_to_none=True`, ilyenkor a `.grad` `None` lesz; `set_to_none=False` esetén a tároló nullázódik. A labor a visszaellenőrzésben a gradiens *értékét*, nem a `None`-kezelést teszteli.

`backward`. A zéró dimenziós lossból kiindulva reverse-mode autograddal kiszámítja a gráf leveleinek gradiensét. A levelek itt a modell paraméterei, tehát `p.grad` alakja megegyezik `p` alakjával. A `backward` után a számítási gráf alapértelmezésben felszabadul.

`optimizer.step`. Az éppen tárolt `.grad` értékek alapján frissíti a paramétereket. Az alap `torch.optim.SGD` momentum nélkül a

$$\theta_{t+1} = \theta_t - \alpha g_t$$

frissítést végzi, ahol α a `lr`, $g_t = \nabla_\theta L(\theta_t)$.

Jelölés. A képletekben L a batch átlagolt lossa, L_i az i-edik minta lossa, azaz $L = \frac{1}{N}\sum_{i=1}^{N} L_i$; a matematikai α a kódban az optimizer `lr` paramétere; az f64 a `torch.float64` típus rövidítése, a `float32` pedig a `torch.float32`-é.

Véges gradiens (finite gradient). Egy paraméter gradiensét végesnek nevezzük, ha minden eleme véges lebegőpontos érték, azaz `torch.isfinite(g).all()` igaz. A `NaN` és a $\pm\infty$ nem véges; ilyen értékkel az SGD-frissítés is nem véges paramétereket adna, ezért a lépés előtt megállítjuk a futást.

11.4.4 A `backward` gradiens és az SGD-frissítés levezetése

A levezetési kötelezettséghez először a veszteség logits szerinti gradiensét vezetjük le, majd ebből a lineáris réteg paramétergradienseit.

Állítás (a loss gradiensének alakja). Legyen $p_{ik} = \mathrm{softmax}(Z)_{ik}$, és legyen $Y^{\mathrm{onehot}}_{ik} = \mathbb{1}_{k=y_i}$. A fenti L mellett

$$\frac{\partial L}{\partial z_{ik}} = \frac{1}{N}\left(p_{ik} - Y^{\mathrm{onehot}}_{ik}\right).$$

Levezetés. Rögzítsük az i megfigyelést. A softmax deriváltja

$$\frac{\partial p_{ik}}{\partial z_{il}} = p_{ik}(\delta_{kl} - p_{il}),$$

Ez a hányados-szabály közvetlen alkalmazása: $p_{ik} = e^{z_{ik}}/S_i$, ahol $S_i = \sum_l e^{z_{il}}$; a derivált két tagja a számláló, illetve a nevező deriválásából származik: $e^{z_{ik}}\delta_{kl}/S_i$ és $-e^{z_{ik}}e^{z_{il}}/S_i^2$. Mivel $L_i = -\log p_{i,y_i}$,

$$\frac{\partial L_i}{\partial z_{ik}} = -\frac{1}{p_{i,y_i}}\frac{\partial p_{i,y_i}}{\partial z_{ik}}.$$

Ha $k = y_i$, akkor $\partial p_{i,y_i}/\partial z_{ik} = p_{ik}(1-p_{ik})$, és a $-\frac{1}{p_{i,y_i}}$ szorzóval együtt $\partial L_i/\partial z_{ik} = -(1-p_{ik}) = p_{ik} - 1$. Ha $k \neq y_i$, akkor a derivált $-p_{ik}p_{i,y_i}$, és a szorzóval együtt $\partial L_i/\partial z_{ik} = -(-p_{ik}) = p_{ik}$. A két eset együtt (a negatív előjelet minden köztes sorban megtartva):

$$\frac{\partial L_i}{\partial z_{ik}} = p_{ik} - \mathbb{1}_{k=y_i}.$$

Az N megfigyelésre átlagolt loss miatt az $1/N$ szorzó megjelenik. □

Legyen

$$G_{ik} = \frac{1}{N}(p_{ik} - Y^{\mathrm{onehot}}_{ik}), \quad G \in \mathbb{R}^{N\times K}.$$

Mivel $z_{ik} = \sum_j x_{ij}W_{kj} + b_k$, a láncszabályból:

$$\nabla_W L = G^\top X \in \mathbb{R}^{K\times d}, \qquad \nabla_b L = G^\top \mathbf{1}_N \in \mathbb{R}^K.$$

A második képlet közvetlenül megadja a biasgradienst: $\mathbf{1}^\top \nabla_b L = \sum_i \sum_k G_{ik} = \sum_i \frac{1}{N}(\sum_k p_{ik} - \sum_k Y^{\mathrm{onehot}}_{ik}) = 0$, mert $\sum_k p_{ik} = 1$ és $\sum_k Y^{\mathrm{onehot}}_{ik} = 1$. Ez a softmax konstanseltolási invarianciájának felel meg: a $z_{i,:} \mapsto z_{i,:} + c\mathbf{1}$ eltolás a softmaxot és a loss-t változatlanul hagyja. Az egyes $\partial L/\partial b_k$ komponensek általában **nem** nullák: az, hogy a b komponensei egyenlők, önmagában nem teszi őket nullává, csak az összegük nulla. Ez a konstanseltolási invariancia közvetlen következménye a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben tárgyalt logsumexp-tulajdonságnak.

A PyTorch `backward` implementációjától tehát azt várjuk, hogy a fenti két mátrixot írja a `model.weight.grad` és `model.bias.grad` mezőkbe. Az alábbi f64-kontroll ezt numerikusan ellenőrzi; a numerikus egyezés nem bizonyítja az implementáció általános helyességét, de független hibakeresési bizonyíték.

11.4.5 Kézi kis példa

Legyen

$$X = \begin{pmatrix} 0{,}5 & 1{,}0 & -0{,}5 \\ -1{,}0 & 0{,}25 & 2{,}0 \end{pmatrix}, \quad W = \begin{pmatrix} 0{,}2 & -0{,}4 & 0{,}1 \\ 0{,}3 & 0{,}15 & -0{,}2 \end{pmatrix}, \quad b = (0{,}05;\ -0{,}05),$$

$y = (0, 1)^\top$, $N = 2$, $K = 2$, $d = 3$, $\alpha = 0{,}25$.

A logitok:

$$Z = \begin{pmatrix} -0{,}30 & 0{,}35 \\ -0{,}05 & -0{,}7125 \end{pmatrix}.$$

A softmax kerekítve:

$$P \approx \begin{pmatrix} 0{,}3430 & 0{,}6570 \\ 0{,}6598 & 0{,}3402 \end{pmatrix}.$$

Ebből

$$G = \frac{1}{2}(P - Y^{\text{onehot}}) \approx \begin{pmatrix} -0{,}3285 & 0{,}3285 \\ 0{,}3299 & -0{,}3299 \end{pmatrix}.$$

A súlygradiens $G^\top X$ kerekítve:

$$\nabla_W L \approx \begin{pmatrix} -0{,}4942 & -0{,}2460 & 0{,}8241 \\ 0{,}4942 & 0{,}2460 & -0{,}8241 \end{pmatrix},$$

és $\nabla_b L = G^\top \mathbf{1} \approx (0{,}001406;\ -0{,}001406)^\top$: az összege nulla (az invariancia miatt), de a komponensei nem. Az SGD-frissítés ($\alpha = 0{,}25$, `float64` újraszámolás):

$$W_1 = W_0 - \alpha \nabla_W L \approx \begin{pmatrix} 0{,}3235 & -0{,}3385 & -0{,}1060 \\ 0{,}1765 & 0{,}0885 & 0{,}0060 \end{pmatrix}, \quad b_1 = b_0 - \alpha \nabla_b L \approx (0{,}04965;\ -0{,}04965)^\top.$$

11.4.6 F64 egybatch-kontroll és a nem véges gradiens leállítása

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A bemenet a szakasz f64 kézi példája; külső adatfájl nem kell, a kód a szakasz kódblokkjából futtatható.

Az alábbi szkript a fenti számítást `float64` pontosságon, CPU-n ellenőrzi, majd a véges-gradiens védelmet is bemutatja. A kódot `float64`-ben, CPU-n, 1 szálon, seed 20261020 mellett lefuttattuk (Python 3.12.3, PyTorch 2.15.0+cpu): a négy ellenőrzés `True`, majd a tervezett `RuntimeError` (`Tervezett leállás: Nem véges gradiens a weight paraméterben.`) következik. A forrásjegyzék a 2.14-es dokumentációra hivatkozik; a futtatott környezet 2.15.0+cpu, és a könyv az API-t a 2.14-es leírás szerint tárgyalja.

```
# lab/n044_train_step.py
import torch

torch.manual_seed(20261020)
torch.set_default_dtype(torch.float64)
device = torch.device("cpu")

d, K, N = 3, 2, 2
x = torch.tensor([[0.5, 1.0, -0.5],
                  [-1.0, 0.25, 2.0]], device=device)
y = torch.tensor([0, 1], dtype=torch.int64, device=device)
```

```
model = torch.nn.Linear(d, K, bias=True).to(device)
with torch.no_grad():
    model.weight.copy_(torch.tensor([[0.2, -0.4, 0.1],
                                     [0.3, 0.15, -0.2]], device=device))
    model.bias.copy_(torch.tensor([0.05, -0.05], device=device))
criterion = torch.nn.CrossEntropyLoss(reduction="mean")
optimizer = torch.optim.SGD(model.parameters(), lr=0.25)

# A lépés sorrendje
optimizer.zero_grad()
logits = model(x)
loss = criterion(logits, y)
loss.backward()

# Kézi gradiens a levezetett képletből
p = torch.softmax(logits, dim=1)
y_onehot = torch.nn.functional.one_hot(y, num_classes=K).to(torch.float64)
G = (p - y_onehot) / N

expected_grad_w = G.T @ x
expected_grad_b = G.sum(dim=0)

print(torch.allclose(model.weight.grad, expected_grad_w, rtol=1e-10, atol=1e-10))
print(torch.allclose(model.bias.grad, expected_grad_b, rtol=1e-10, atol=1e-10))

# Várt SGD-frissítés ellenőrzése
with torch.no_grad():
    w0 = model.weight.data.clone()
    b0 = model.bias.data.clone()
    expected_w = w0 - 0.25 * expected_grad_w
    expected_b = b0 - 0.25 * expected_grad_b

optimizer.step()

print(torch.allclose(model.weight.data, expected_w, rtol=1e-10, atol=1e-10))
print(torch.allclose(model.bias.data, expected_b, rtol=1e-10, atol=1e-10))

assert torch.allclose(model.weight.grad, expected_grad_w, rtol=1e-10, atol=1e-10)
assert torch.allclose(model.bias.grad, expected_grad_b, rtol=1e-10, atol=1e-10)
assert torch.allclose(model.weight.data, expected_w, rtol=1e-10, atol=1e-10)
assert torch.allclose(model.bias.data, expected_b, rtol=1e-10, atol=1e-10)

# Véges-gradiens védelem: a step előtt állítsuk meg a nem véges futást
def assert_finite_grad(model):
    for name, param in model.named_parameters():
        if param.grad is None:
            continue
        if not torch.isfinite(param.grad).all():
            raise RuntimeError(f"Nem véges gradiens a {name} paraméterben.")

model.zero_grad()
# Új forward gráf: a korábbi logits gráfját a backward már felszabadította,
# a model(x) * nan szorzás viszont differenciálható, ezért a bad_loss.backward()
# végrehajtható, és a gradiens NaN lesz.
bad_logits = model(x) * float("nan")
bad_loss = criterion(bad_logits, y)
bad_loss.backward()
try:
    assert_finite_grad(model)
    optimizer.step()
except RuntimeError as e:
    print("Tervezett leállás:", e)
```

Futtatott kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020*; két futás kimenete bitazonos):*

```
True
True
True
True
Tervezett leállás: Nem véges gradiens a weight paraméterben.
```

A `bad_logits` az új `model(x)` gráfjához kapcsolódik, így a `bad_loss.backward()` végrehajtható, és a paraméterek gradiense `NaN` lesz. Egy önálló `torch.full_like(logits, float("nan"))` tensorral ez nem működne, mert a veszteségnek nem lenne differenciálható útja a modellparaméterekig. Az `assert_finite_grad` hívás a kivételt dobja, ezért a `try` blokkban utána álló `optimizer.step` nem fut le, és a paraméterek nem válnak `NaN`-né a lépésben. A védelmet mindig a `step` előtt kell meghívni.

11.4.7 Gyakori tévedés és ellenpélda

Tévedés: „Az optimizer automatikusan lefut, amint a `loss.backward()` sikeres volt.” Ez hamis: a `backward` csak a gradiens tárolókat tölti fel; a paraméterfrissítés külön `optimizer.step()` hívás.

Ellenpélda:

```
w_before = model.weight.data.clone()
optimizer.zero_grad()
logits = model(x)
loss = criterion(logits, y)
loss.backward()
# NINCS optimizer.step()
print(model.weight.grad is not None)             # True, a gradiens kiszámult
print(torch.equal(w_before, model.weight.data))  # True, a súly változatlan
```

A két `True` ellenére a paraméter nem frissült, mert a frissítést semmi sem hívta. Ennek ellenőrzése külön kísérlettel: a `w_before` a `backward` előtti másolat, a `torch.equal(w_before, model.weight.data)` pedig a `backward` utáni állapotot hasonlítja össze; hiányzó `step` esetén az eredmény `True`, a két tensor bitre egyezik.

11.4.8 Feladatok

F1 (fogalmi, nehézség 1). Adja meg a tanítási lépés helyes sorrendjét a `zero_grad`-tól a `step`-ig, a kötelező hívások és az opcionális gradiens-ellenőrzés megkülönböztetésével, és magyarázza el, miért nem mindegy, hogy `zero_grad` a `backward` előtt vagy után szerepel. Illusztráljon egy rossz sorrendet kis gondolatkísérlettel. Elvárt eredmény: a sorrend és az akkumuláció következménye.

F2 (levezetési, nehézség 2). Az $N = 1$, $d = 1$, $K = 2$ esetben legyen $X = (0{,}5)$, $W = (0{,}2;\ -0{,}3)^\top$, $b = (0{,}1;\ 0{,}0)^\top$, $y = 1$, $\alpha = 0{,}5$. Számítsa ki Z, P, a `backward` után várható $\nabla_W L$ és $\nabla_b L$ értékeket, majd a frissített W_1 és b_1 paramétereket. Kerekítsen 4 tizedesjegyre. Elvárt eredmény: a két gradiens és a két frissített paraméter.

F3 (implementációs, nehézség 2). Írjon olyan `train_step(model, x, y, criterion, optimizer)` függvényt, amely a megfelelő sorrendben elvégzi a lépést, a `step` előtt minden paraméter gradiensét végesnek ellenőrzi, és nem véges esetén `RuntimeError`-t dob. Térjen vissza a loss `item()` értékével. Tesztelje az F2 példáján. Elvárt eredmény: teljes kód és egy sikeres hívás kimenete.

F4 (hibakeresési, nehézség 1). Az alábbi kódrészlet egy tanítási ciklus törzséből származik; a ciklus több iterációja után a loss nem csökken, bár a gradiens `None`-tól különböző. Hol a hiba, mi a következménye, és hogyan javítja?

```
optimizer.zero_grad()
logits = model(x)
loss = criterion(logits, y)
loss.backward()
# néhány diagnosztikai nyomtatás
print(loss.item())
```

Elvárt eredmény: a hiba azonosítása, a kimaradt hívás neve, a javítás és az ellenőrzés leírása.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egybatch-es diagnosztikai kísérletet, amely meggyőződik arról, hogy a véges-gradiens védelem valóban megállítja a futást, mielőtt a paraméterek nem véges értéket kapnának. Adja meg: kérdés, adat, baseline, változó tényező, metrika, a várható kivétel típusa és a következtetési korlát. Elvárt eredmény: korrekt protokoll, alternatív megoldás és a korlát megnevezése. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

11.4.9 Megoldások és indoklás

F1. Helyes sorrend: `optimizer.zero_grad()`, `logits = model(x)`, `loss = criterion(logits, y)`, `loss.backward()`, gradiens-ellenőrzés, `optimizer.step()`. A `backward` a `.grad` mezőt a kiszámolt gradienssel *bővíti*. Ha a `zero_grad` a `backward` után futna le, a frissítéshez szükséges gradiens törlődne, és az SGD súlyváltozása nulla lenne. Ha a `zero_grad` teljesen elmarad, két egymás utáni batch gradiense összeadódik, és a lépés nem az aktuális batch átlagának, hanem az összegének megfelelően frissít. A rossz sorrendre példa: `backward` után `zero_grad`, majd `step`; az alapértelmezett `set_to_none=True` mellett ekkor a `.grad` `None` lesz, `set_to_none=False` mellett pedig nulla, így a `step` érdemi frissítést nem hajt végre.

F2. A logit:

$$z_1 = 0{,}5 \cdot 0{,}2 + 0{,}1 = 0{,}2, \qquad z_2 = 0{,}5 \cdot (-0{,}3) + 0{,}0 = -0{,}15.$$

A softmax: $e^{0{,}2} \approx 1{,}2214$, $e^{-0{,}15} \approx 0{,}8607$, összeg 2,0821, így

$$p = (0{,}5866;\ 0{,}4134).$$

$N = 1$ és $y = 1$, ezért $G = (0{,}5866;\ -0{,}5866)$. A súlygradiens oszlopvektor:

$$\nabla_W L = G^\top X = (0{,}2933;\ -0{,}2933)^\top,$$

a biasgradiens $\nabla_b L = (0{,}5866;\ -0{,}5866)^\top$. A frissítés $\alpha = 0{,}5$:

$$W_1 = (0{,}2;\ -0{,}3)^\top - 0{,}5\,(0{,}2933;\ -0{,}2933)^\top = (0{,}0533;\ -0{,}1533)^\top,$$

$$b_1 = (0{,}1;\ 0{,}0)^\top - 0{,}5\,(0{,}5866;\ -0{,}5866)^\top = (-0{,}1933,\ 0{,}2933)^\top.$$

Alakellenőrzés: $W_1 \in \mathbb{R}^{2\times 1}$, $b_1 \in \mathbb{R}^2$. A 4 tizedesjegyre kerekített értékek csak szemléltetők: az f64 ellenőrzés az analitikus képletet kerekítés nélkül veti össze a `backward` eredményével, és `rtol=atol=1e-10` mellett egyezést ad.

F3. Egy lehetséges megoldás:

```python
def train_step(model, x, y, criterion, optimizer):
    optimizer.zero_grad()
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
    for name, param in model.named_parameters():
        if param.grad is None:
            continue
        if not torch.isfinite(param.grad).all():
            raise RuntimeError(f"{name}: nem véges gradiens")
    optimizer.step()
    return loss.item()
```

Az F2 példáján futtatott teszt a fenti `train_step` függvénnyel, `float64` default dtype mellett:

```
torch.set_default_dtype(torch.float64)
model = torch.nn.Linear(1, 2)
with torch.no_grad():
    model.weight.copy_(torch.tensor([[0.2], [-0.3]]))
    model.bias.copy_(torch.tensor([0.1, 0.0]))
x = torch.tensor([[0.5]])
y = torch.tensor([1], dtype=torch.int64)
criterion = torch.nn.CrossEntropyLoss(reduction="mean")
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)

value = train_step(model, x, y, criterion, optimizer)
print(value)
print(model.weight.data.view(-1).tolist())
print(model.bias.data.view(-1).tolist())
```

A futtatott kimenet:

```
0.883382155418777
[0.05334560527066748, -0.15334560527066748]
[-0.19330878945866506, 0.293308789458665]
```

A visszatérés négy tizedesjegyre 0,8834, a frissített paraméterek pedig $W_1 \approx (0{,}0533;\ -0{,}1533)^\top$ és $b_1 \approx (-0{,}1933;\ 0{,}2933)^\top$, összhangban az F2 megoldással. A `zero_grad` a backward előtt, a `step` a végső ellenőrzés után van, így a `NaN/inf` gradiens `RuntimeError`-ral megállítja a futást; a `None` gradiensű paramétert (amely nem vett részt a gráfban) az ellenőrzés átlépi, összhangban a labor `assert_finite_grad` függvényével.

F4. A hiba a hiányzó `optimizer.step()` hívás. A `backward` feltölti a gradienseket, a loss kiszámítása és a gradiens léte tényleges, de a paraméterek nem frissülnek, ezért a következő forward ugyanazt a loss-t adja. A javítás a diagnosztikai nyomtatás után egy `optimizer.step()` hívás. Ellenőrzés: a `step` előtt elmentett `w_before = model.weight.data.clone()` és a `step` utáni `w_after` nem egyezik bitre; a loss a következő forwardon csökken. A javított kód minden iterációban megismétli a `zero_grad`–forward–loss–`backward`–`step` sort.

F5. Egy tervezhető kísérlet:

- **Kérdés:** a védelem az első nem véges gradiensnél megállítja-e a frissítést?
- **Adat:** a fenti $N = 2$ batch, f64 CPU, rögzített seed.
- **Baseline:** normál $\alpha = 0{,}25$ mellett a loss és a paraméterváltozás a kézi képletből számolható; a védelem nem dob.
- **Változó tényező:** szándékosan `NaN` logitokat adunk a loss-hoz, így a gradiens nem véges lesz.
- **Metrika:** a dobott kivétel típusa (`RuntimeError`), és a paraméterek értéke a védelem után megegyezik a lépés előtti értékkel.

- **Alternatív megoldás:** a `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0, error_if_nonfinite=True)` a `step` előtt szintén `RuntimeError`-t dob, ha a gradiensnorma nem véges (ellenőrizve: „The total norm of order 2.0 for gradients from `parameters` is non-finite, so it cannot be clipped."). A `torch.autograd.set_detect_anomaly(True)` már a `backward` közben elkapja a `NaN`-t (ellenőrizve: „Function 'LogSoftmaxBackward0' returned nan values in its 0th output."), de drágább, és csak a backward hibáit jelzi, a `step` utáni nem véges paramétert nem.
- **Korlát:** a szintetikus `NaN` injektálás determinisztikus teszt, de nem bizonyítja, hogy éles adatokon minden nem véges okot a védelem időben elkap. Ha a gradiens véges, de a tanulási ráta túl nagy, a `step` akkor is nem véges paramétert állíthat elő; ezt a gradiensellenőrzés nem akadályozza meg, csak a következő iteráció loss- vagy paraméterellenőrzése jelzi. A bemutatott labor a gradienst a `step` előtt ellenőrzi; a `loss` és a paraméterek `isfinite` állapotát a protokoll külön, a lépés utáni naplózási lépésként írja elő, a kódrészlet ezt nem tartalmazza.

11.4.10 Önellenőrzés, korábbi fogalom és tipikus hibahatár

Önellenőrzés: a `backward` után a súlygradiens alakja megegyezik a súly alakjával ($K \times d$), a biasgradiens (K); a kézi példában a biasgradiens komponensei kicsik, $\pm 0{,}001406$, és az összegük nulla a softmax konstanseltolási invarianciája miatt. A numerikus összevetést f64-ben, `rtol=1e-10, atol=1e-10` értékkel végezzük. A 10.4. Tanítási lépés és tanuló MLP alfejezetben kézzel végzett három művelet itt pontosan a `zero_grad`, `backward` és `step` hármasnak felel meg; a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben tárgyalt stabil loss itt élesben is véd az overflow ellen, de a gradiens végességét a `step` előtti külön ellenőrzés biztosítja.

11.4.11 Rövid összefoglaló

A PyTorch-tanítási lépés sorrendje szerződés: `zero_grad`, forward, loss, `backward`, véges-gradiens ellenőrzés, majd `optimizer.step`. A `backward` a közvetlen lineáris réteg plusz `CrossEntropyLoss` esetén a $G^\top X$ súly- és $G^\top \mathbf{1}_N$ biasgradienst állítja elő, ahol $G_{ik} = \frac{1}{N}(p_{ik} - \mathbb{1}_{k=y_i})$. Az SGD-lépés $\theta_{t+1} = \theta_t - \alpha g_t$, és a labor f64-ben numerikusan összeveti a levezetett értékkel. A `backward` sikere önmagában nem frissíti a paramétereket; a `step` előtti `isfinite` ellenőrzés a `NaN`/`inf` gradiensből eredő hibát állítja meg. A következő 12.1. Tanítás, validáció és kiértékelési mód egység a teljes training-validáció és eval mód keretét építi erre a lépésre.

11.4.12 Felhasznált források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

12 Megismételhető tanítási kísérlet

12.1 Tanítás, validáció és kiértékelési mód

12.1.1 Cél, előismeret és induló kérdés

Ez az egység a tanítási kísérlet legkisebb ismételhető keretét építi meg: egy teljes epoch tanítása, a modell üzemmódjának váltása, a veszteség helyes aggregálása és a validáció szivárgásmentes kiértékelése. Előismeret a 10.4. Tanítási lépés és tanuló MLP tanítási lépése, a 11.4. Egy teljes PyTorch tanítási iteráció PyTorch-iteráció sorrendje és a 11.1. Tensor és autograd-életciklus autograd életciklusa.

Induló kérdés: egy öt mintás epoch két batchből áll. A három mintás batch átlagos vesztesége 2, a két mintás batché 5. A teljes epoch átlagos vesztesége 3,5 vagy 3,2? Becsülje meg, mielőtt a levezetést elolvassa.

12.1.2 Definíciók és intuíció

Train/eval üzemmód. A PyTorch `nn.Module` `.training` bool flaget hordoz. A `model.train()` igazra, a `model.eval()` hamisra állítja. Az üzemmód az állapotfüggő rétegek viselkedését szabályozza: a dropout train módban véletlenül nulláz, eval módban identitás; a BatchNorm train módban a batch-statisztikát használja és frissíti a futó statisztikát, eval módban a futó statisztikát használja, de csak `track_running_stats=True` (az alapbeállítás) mellett; `track_running_stats=False` esetén eval módban is a batch-statisztikát számolja, tehát a kimenet batchfüggő marad (a BatchNorm részletes train/eval kezelését későbbi, a konvolúciós hálókhoz kapcsolódó egység tárgyalja; a [D12] dokumentáció szerint a két statisztikakezelés nem felcserélhető). A szabály akkor is kötelező, ha az aktuális modellben éppen nincs állapotfüggő réteg, mert a későbbi bővítés nem írhatja át a protokollt.

`torch.inference_mode()`. Ez nem modell-üzemmód, hanem tenzor-kontextus: a blokkban létrehozott tenzorokból nem épül autograd gráf, és az ott létrehozott „inference tensor” autograd-műveletekben csak korlátozottan használható (backwardhoz elmentve hibát ad). A blokk elhagyása után a normál gráfépítés visszaáll. Erősebb, mint a `no_grad`, mert a nézetkövetést is kikapcsolja (11.1. Tensor és autograd-életciklus). A kiértékeléshez mindkét réteg kell: az `eval()` a modul állapotát, az `inference_mode` a memória- és gráffegyelmet kezeli.

Loss aggregation (veszteség-összesítés). A veszteségfüggvény mean redukcióval skalárt ad: egy $n \times K$ logit tenzorból és n darab int64 célból az n egyedi veszteség átlagát. A tanítási lépésben a backward gradient az átlag deriváltja, ezért a minibatch gradiens az egyedi gradiensek átlaga (10.4. Tanítási lépés és tanuló MLP). Az epoch-loss ezzel szemben a teljes tanítóhalmazra vett megfigyelési metrika, nem a backprop bemenete. Az epoch-loss a tanítás során változó paraméterekkel számolódik: az egyes batch-veszteségek más-más θ mellett készülnek, ezért ez a mennyiség a frissítési pálya mentén mért megfigyelési metrika, nem egyetlen rögzített θ függvénye.

Batch weighting (batch-súlyozás) és szivárgásmentesség (no leakage). A DataLoader az epochot batch-ekre particionálja ([D08]); ha az utolsó batch kisebb, a veszteségek összesítése súlyozást igényel. A szivárgásmentesség azt jelenti: a validációs split egyetlen mintája sem jelenik meg a tanítási veszteségben, a validációs forward nem frissíti a paramétereket, és a validációs metrika nem választhat modellt az előre rögzített protokoll megkerülésével. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adatkészlet 24×24-es szintetikus orientációs képekből áll. Egy **jelenet** (`scene_id`) két nézetet (`view_id`) tartalmaz; a két nézet ugyanahhoz a jelenethez tartozik, ezért nem független minta. A split `scene_id` szerint csoportosított, ezért egy jelenet két nézete azonos splitbe kerül.

12.1.3 Levezetés: a súlyozott epoch-loss

Állítás (epoch-loss mint súlyozott átlag). Legyen az epoch tanítóhalmaza $D = \{(x_j, y_j)\}_{j=1}^{N}$, rögzített θ paraméter mellett. A batchek $D_1, \dots, D_B$ diszjunkt partíciót alkotnak, $n_i = |D_i|$, $\sum_{i=1}^{B} n_i = N$. Minden batchre mean redukcióval definiáljuk

$$\bar{L}_i(\theta) = \frac{1}{n_i} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y).$$

Ekkor a teljes epoch mean vesztesége

$$\bar{L}(\theta) = \frac{1}{N} \sum_{i=1}^{B} n_i \, \bar{L}_i(\theta).$$

Bizonyítás. A partíció tulajdonság miatt $D = \bigsqcup_{i=1}^{B} D_i$, tehát a teljes összeg felbontható:

$$\bar{L}(\theta) = \frac{1}{N} \sum_{(x,y)\in D} \ell(f_\theta(x), y) = \frac{1}{N} \sum_{i=1}^{B} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y).$$

A belső összeg éppen $n_i \bar{L}_i(\theta)$, mert a batchre vett mean definícióját n_i-vel visszaszorozva a batch összegét kapjuk:

$$\frac{1}{N} \sum_{i=1}^{B} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y) = \frac{1}{N} \sum_{i=1}^{B} n_i \cdot \frac{1}{n_i} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y) = \frac{1}{N} \sum_{i=1}^{B} n_i \, \bar{L}_i(\theta).$$

Ez a bizonyítandó állítás. A partíció tulajdonság az első egyenlőségnél, a mean redukció definíciója a másodiknál lép be. Következmény: ha minden batch azonos n méretű, akkor $N = Bn$, így $\bar{L} = \frac{1}{B} \sum_i \bar{L}_i$; az egyszerű átlag ekkor egyezik a súlyozott formulával (és akkor is, ha a batch-átlagok véletlenül egyenlők), eltérő batch-méretek mellett azonban általában nem egyezik.

A levezetés rögzített θ-ra vonatkozik. A tanítás közben a modell batch-ek között frissül, ezért a monitorozott epoch-loss a frissítési pálya veszteségeit átlagolja; a fenti súlyozási formula attól függetlenül érvényes, hogy a kiszámított batch-veszteségeket hogyan kombináljuk.

12.1.4 Kézi számpélda

Legyen $N = 5$, $B = 2$; $n_1 = 3$, $n_2 = 2$. A batch 1 egyedi veszteségei $(1, 2, 3)$, így $\bar{L}_1 = 2$; a batch 2 egyedi veszteségei $(4, 6)$, így $\bar{L}_2 = 5$. A súlyozott epoch-loss:

$$\bar{L} = \frac{3 \cdot 2 + 2 \cdot 5}{5} = \frac{16}{5} = 3{,}2.$$

Az unweighted batchátlag ezzel szemben $(2 + 5)/2 = 3{,}5$, ami nem a teljes mintára vett átlag. A félrevezető érték onnan jön, hogy a kisebb batch ugyanakkora súlyt kap, mint a nagyobb.

12.1.5 Konkrét labor: train és evaluate a GlyphLab adaton

Előfeltétel. Python 3.12.3, NumPy 2.5.4, pandas 3.1.0 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; a `n045_train_eval.py` a `lab/` könyvtárból futtatható (`python n045_train_eval.py`).

A labor egy `n045_train_eval.py` scriptet készít a `lab/` munkakönyvtárban. Bemenet a GlyphLab `data/images.npy`, `data/labels.npy` és `data/metadata.csv`; az útvonalakat a szkript a saját helyéhez képest oldja fel, a splitet pedig a `metadata.csv split` oszlopa adja, miután ellenőriztük, hogy az `index` oszlop követi a tömbök sorrendjét. A teljes GlyphLab 360 kép, három osztállyal: train 240 (osztályonként 80), val 60 (osztályonként 20) és teszt 60; ez a labor csak a train és a val splitet tölti be. A kép batch alakja $N \times 1 \times 24 \times 24$, float32, a címke int64. A batch méret 64, így a train loader négy batchre oszlik: $64, 64, 64, 48$; az utolsó kisebb batch a súlyozás kontrollja. A modell egy lapított MLP, a veszteség `CrossEntropyLoss` mean redukcióval, logits bemenettel, softmax nélkül ([D07] loss-konvenció). A közölt kimenet a rögzített környezetben futtatott labor valós eredménye.

```
"""n045_train_eval.py -- train/eval üzemmód és súlyozott epoch-loss.
Futtatás a lab/ könyvtárból: python n045_train_eval.py.
Az adatokat a szkript a saját helyéhez képest oldja fel, ezért a
munkakönyvtár nem befolyásolja az útvonalakat.
A kód kimenete a könyvben rögzített futtatásból származik.
"""
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

DEVICE = "cpu"
SEED = 20261020
BATCH_SIZE = 64
DATA = Path(__file__).resolve().parent / "data"

torch.manual_seed(SEED)

images = np.load(DATA / "images.npy")      # (360,1,24,24) float32
labels = np.load(DATA / "labels.npy")      # (360,) int64
meta = pd.read_csv(DATA / "metadata.csv")

# A metadata sorrendje és az images/labels tömb sorrendje ugyanaz a szerződés.
expected_columns = ["index", "scene_id", "view_id", "label", "split",
```

```
                    "xmin", "ymin", "xmax", "ymax"]
assert list(meta.columns) == expected_columns, "váratlan metadata-fejléc"
assert len(meta) == len(images) == len(labels), "eltérő hosszúságú adat"
assert np.array_equal(meta["index"].to_numpy(), np.arange(len(images))), \
    "a metadata index oszlopa nem követi az images tömb sorrendjét"

train_idx = meta.loc[meta["split"] == "train", "index"].to_numpy()
val_idx = meta.loc[meta["split"] == "val", "index"].to_numpy()

X_train = torch.from_numpy(images[train_idx])
y_train = torch.from_numpy(labels[train_idx])
X_val = torch.from_numpy(images[val_idx])
y_val = torch.from_numpy(labels[val_idx])

train_loader = DataLoader(TensorDataset(X_train, y_train),
                          batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=BATCH_SIZE, shuffle=False)

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(24 * 24, 64),
    nn.ReLU(),
    nn.Linear(64, 3),
)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)

def train_one_epoch(model, loader, loss_fn, optimizer, device=DEVICE):
    model.train()
    total = 0.0
    n = 0
    for Xb, yb in loader:
        Xb = Xb.to(device)
        yb = yb.to(device)
        optimizer.zero_grad()
        loss = loss_fn(model(Xb), yb)
        loss.backward()
        optimizer.step()
        total += loss.item() * yb.size(0)
        n += yb.size(0)
    return total / n

def evaluate(model, loader, loss_fn, device=DEVICE):
    model.eval()
    total = 0.0
    n = 0
    correct = 0
    with torch.inference_mode():
        for Xb, yb in loader:
            Xb = Xb.to(device)
            yb = yb.to(device)
            logits = model(Xb)
            total += loss_fn(logits, yb).item() * yb.size(0)
            n += yb.size(0)
            correct += (logits.argmax(dim=1) == yb).sum().item()
    return total / n, correct / n

batch_sizes = []
# Külön, nem keverő loader: a train_loader puszta végigiterálása is
# elhasználná a shuffle permutációját, és megváltoztatná a tanítás sorrendjét.
check_loader = DataLoader(train_loader.dataset, batch_size=BATCH_SIZE,
                          shuffle=False)
```

```python
for _, yb in check_loader:
    batch_sizes.append(yb.size(0))
print("train batch méretek:", batch_sizes)   # [64, 64, 64, 48]
assert batch_sizes == [64, 64, 64, 48]
assert sum(batch_sizes) == 240

for epoch in range(5):
    train_loss = train_one_epoch(model, train_loader, loss_fn, optimizer)
    val_loss, val_acc = evaluate(model, val_loader, loss_fn)
    print(f"epoch {epoch+1}: train_loss={train_loss:.4f} "
          f"val_loss={val_loss:.4f} val_acc={val_acc:.3f}")
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020*; két futás kimenete ebben az azonos verziójú, egyszálas CPU-környezetben bitazonos, ami nem keresztplatformos garancia):*

```
train batch méretek: [64, 64, 64, 48]
epoch 1: train_loss=1.0998 val_loss=1.1055 val_acc=0.333
epoch 2: train_loss=1.0970 val_loss=1.1046 val_acc=0.333
epoch 3: train_loss=1.0944 val_loss=1.1040 val_acc=0.333
epoch 4: train_loss=1.0918 val_loss=1.1037 val_acc=0.333
epoch 5: train_loss=1.0899 val_loss=1.1032 val_acc=0.333
```

A mért futásban a train loss csak kis mértékben csökkent (1,0998 → 1,0899), a val pontosság mind az öt epochban 0,333; öt epoch alatt ez a kis MLP lassú indulását mutatja, nem általánosítási eredményt.

A függvények magja a súlyozás: `total += loss.item() * yb.size(0)` és `n += yb.size(0)`, majd `total / n`. A kontroll `assert` a loader batch-méreteit ellenőrzi: az utolsó batch 48 mintás, a négy batch összege pedig 240. Azt, hogy a súlyozásba mind a 240 minta bekerül, a `train_one_epoch` fenti két sora biztosítja, nem az assert; ha a DataLoadert `drop_last=True`-szal hívnánk, a 48 minta elveszne, és az epoch-loss nem a 240 mintára vonatkozna. A batch-méret-ellenőrzés külön, `shuffle=False` loaderrel fut, mert a `train_loader` végigiterálása elhasználná a következő epoch shuffle-permutációját, és a tanítás sorrendjét a seed mellett is megváltoztatná. A validációs forward `model.eval()` alatt és `torch.inference_mode()` blokkban fut, így sem paramétergradiens, sem autograd-gráf nem keletkezik. A BatchNorm futó statisztikáját nem az `inference_mode`, hanem az `eval()` fagyasztja: train módban a BN a `no_grad`/`inference_mode` blokk alatt is frissítené a buffert, ezért a két mechanizmus együtt kell (a jelen MLP-ben nincs ilyen réteg, de a protokoll a későbbi CNN-re is érvényes kell legyen).

Az értelmezéshez: a train loss csökkenése önmagában nem általánosítási bizonyíték; a val loss a 8.1. Kockázat, mintavétel és baseline empirikus kockázat becslése a val spliten. A 60 val minta miatt a görbe zajos, és egyetlen futás nem helyettesíti a seed-ismétlést. A [D02] dokumentáció szerint az azonos seed nem garantál platformok és verziók közötti bitazonos kimenetet, ezért a seed a protokoll része, nem a reprodukció egyetlen feltétele.

12.1.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért kell az `evaluate` függvényben egyszerre `model.eval()` hívás és `torch.inference_mode()` blokk. Mi történne egy dropout réteget tartalmazó modell kiértékelésekor, ha a `model.eval()` hiányzik? Fogalmazza meg a két mechanizmus különbségét egy-egy mondatban.

F2 (levezetési, nehézség 2). Legyen a tanítóhalmaz rögzített θ mellett B darab n_i méretű batch partíciója, a veszteség mean redukció. Vezesse le a

$$\bar{L} = \frac{1}{N} \sum_{i=1}^{B} n_i \bar{L}_i, \qquad N = \sum_{i=1}^{B} n_i$$

formulát. Jelölje meg, melyik lépés használja a partíció tulajdonságot, és ellenőrizze a $B = 1$ szélsőesetet.

F3 (implementációs, nehézség 2). Bontsa ki a labor súlyozó logikáját önálló függvénnyé:

```
def weighted_epoch_loss(batch_losses, batch_sizes):
    ...
```

A függvény két listából scalár epoch-loss-t ad. Input: `batch_losses = [2.0, 5.0]`, `batch_sizes = [3, 2]`. Elvárt kimenet: `3.2`, tolerancia `1e-6`. Adja meg a függvényt és a hívását a megadott bemenetre.

F4 (hibakeresési, nehézség 1). Egy társ ezt írta: `total += loss.item()`, majd a végén `final = total / len(loader)`. Lokalizálja a hibát, számítsa ki a kézi példán a hibás eredményt, és adja meg a javítást.

F5 (kísérleti, nehézség 3). Tervezzen GlyphLab kísérletet: az első öt epochban a train loss csökken-e, míg a val loss növekedésnek indul. Adja meg a kérdést, az adatot, a splitet, a seedet, a baseline-t, a metrikákat és a következtetés érvényességi korlátait. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

12.1.7 Megoldások és indoklás

F1 megoldás. Egy-egy mondatban: az `eval()` a modell állapotflagjét állítja, ezért az állapotfüggő rétegek (dropout, BatchNorm) viselkedését szabályozza; az `inference_mode()` az autograd gráf építését tiltja, ezért a memória- és gradiens-könyvelést szabályozza. Részletesen: a flag hamisra állítása után a dropout eval módban identitás, ezért a kiértékelés determinisztikus és a tanítási véletlen nullázás nem rontja a predikciót. Ha elmarad, a dropout minden forwardnál más-más neuronokat nulláz, így a val loss és pontosság zajossá válik, és a predikció nem egyezik a későbbi éles használattal. Az `inference_mode()` ezzel szemben az autograd gráf építését tiltja: az evaluate forwardja alatt nem keletkezik gráf, így nem fogy memória, és véletlen `backward` hívás sem szennyezhetné a paramétergradienst. A kettő ortogonális: az egyik a modul viselkedését, a másik a tenzor-könyvelést szabályozza.

F2 megoldás. A kiinduló definíció $\bar{L}_i = \frac{1}{n_i} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y)$. Az epoch mean loss:

$$\bar{L} = \frac{1}{N} \sum_{(x,y)\in D} \ell(f_\theta(x), y).$$

A partíció tulajdonság $D = \bigsqcup_i D_i$ engedi a felbontást:

$$\bar{L} = \frac{1}{N} \sum_{i=1}^{B} \sum_{(x,y)\in D_i} \ell(f_\theta(x), y).$$

Ez az a lépés, amely a partíciót használja. A belső összeg a batch mean definícióját visszaszorozva $n_i \bar{L}_i$, így

$$\bar{L} = \frac{1}{N} \sum_{i=1}^{B} n_i \bar{L}_i.$$

Szélsőeset: $B = 1$, ekkor $n_1 = N$, tehát $\bar{L} = \frac{1}{N} \cdot N \cdot \bar{L}_1 = \bar{L}_1$, a formula visszaadja a teljes adatra vett mean loss-t. A levezetés feltétele, hogy a batch-ek páronként diszjunktak és lefedik a teljes epochot; a mean redukció definíciója rögzített θ-ra vonatkozik.

F3 megoldás.

```python
def weighted_epoch_loss(batch_losses, batch_sizes):
    """Súlyozott epoch-loss a batch-veszteségek és batch-méretek listájából."""
    total = sum(l * n for l, n in zip(batch_losses, batch_sizes))
    return total / sum(batch_sizes)

result = weighted_epoch_loss([2.0, 5.0], [3, 2])
assert abs(result - 3.2) < 1e-6, result
print(f"súlyozott epoch-loss: {result:.6f}")  # 3.200000
```

Alakellenőrzés: minden l_i scalár, n_i pozitív egész; a nevező $\sum n_i = N$; a kimenet scalár float. A labor `train_one_epoch` függvénye pontosan ezt a számítást végzi beágyazva, ahol a `loss.item()` a batch mean loss scalárja.

F4 megoldás. A hiba a nevezőben van: `total / len(loader)` a batch-veszteségek egyszerű átlaga, nem a minták szerinti súlyozott átlag. A kézi példán $\bar{L}_1 = 2$, $\bar{L}_2 = 5$, így a hibás érték $(2 + 5)/2 = 3{,}5$, szemben a helyes $\frac{3 \cdot 2 + 2 \cdot 5}{5} = 3{,}2$ értékkel. A következmény: az epoch-loss torzított megfigyelési metrika, és a különböző méretű utolsó batch indokolatlanul nagy súlyt kap. Javítás: `total += loss.item() * yb.size(0)`; `n += yb.size(0)`; `return total / n`. Visszaellenőrzés: a batch-méretek $[64, 64, 64, 48]$, a `train_one_epoch` visszaadott értéke a súlyozott képletből számolódik; a labor `assert`je csak a loader batch-méreteit ellenőrzi, a súlyozott loss értékét nem.

F5 megoldásminta. Kérdés: a train loss öt epoch alatt monoton csökken-e, és a val loss a megfigyelési ablakban növekedni kezd-e? Adat: GlyphLab `data/images.npy`, `data/labels.npy`, split a `data/metadata.csv` `split` oszlopából, a `scene_id` csoportosítás megtartásával. Modell: a labor három osztályos MLP-je, SGD, lr $= 0{,}05$, batch 64, seed `20261020`, device CPU, float32. Baseline: a leggyakoribb osztályt prediktáló konstans modell; pontossága pontosan a legnagyobb osztályarány, a GlyphLab val splitjében pedig osztályonként 10 jelenet, azaz 20 kép van, így a három osztály egyenlő arányú, és ez a pontosság 1/3. Metrikák: súlyozott train és val cross-entropy, val pontosság. Protokoll: a val loss-t nem használjuk modellválasztásra; csak a tanítás után elemezzük a görbét. Korlát: a 60 val minta miatt a görbe nagy varianciájú; egyetlen futás nem ad konfidencia-intervallumot, több seed kellene az ismételt mintavételi értelmezéshez; a train loss a frissítés közbeni vándorló paraméterek vesztesége, nem állandó θ-ra vett epoch-loss.

12.1.8 Gyakori tévedés és ellenpélda

Az unweighted batchátlag gyakori hibája: `total += loss.item(); return total / len(loader)`. Az öt mintás, $3 + 2$ felbontású példán a helyes $\bar{L} = 3{,}2$, a hibás 3,5; a 0,3 eltérés nem kerekítési hiba, hanem strukturális súlyozási hiba. Ha minden batch mérete egyenlő, a két képlet egybeesik, ezért a hiba kis egyenletes batch-ek mellett rejtve maradhat, és az utolsó kisebb batch esetén jelentkezik.

12.1.9 Önellenőrzés és visszahívás

Ellenőrzés: a levezetésben $i = 1, \dots, B$ és $j = 1, \dots, n_i$ indexhatárok teljesek, $\sum_i n_i = N$ a partícióból következik, minden tag skalár. Az F2 megoldásában tárgyalt $B = 1$ szélsőeset a formulát $\bar{L} = \bar{L}_1$-re redukálja. A 11.1. Tensor és autograd-életciklus visszahívása: az `inference_mode` erősebben zárja a gráfépítést, mint a `no_grad`, ezért a kiértékeléshez ezt használjuk. A 8.1. Kockázat, mintavétel és baseline visszahívása: a val loss empirikus kockázat a val spliten, nem populációs kockázat; a kis val minta becslési varianciája nem tűnik el a seed rögzítésétől.

12.1.10 Rövid összefoglaló

Az epoch tanítási vesztesége nem a batch-veszteségek egyszerű átlaga: a különböző n_i méretű batchek mean loss-aiból csak a súlyozott aggregátum $\bar{L} = \frac{1}{N} \sum_i n_i \bar{L}_i$ adja a teljes mintára vett mean loss-t. A levezetés a partíció tulajdonságon és a mean redukció definícióján áll, rögzített paraméter mellett. A train mód a modell állapotát, az inference mode a gráfépítést szabályozza; az evaluate ezért egyszerre hív `model.eval()`-t és `inference_mode` blokkot. A validáció közben sem paraméter, sem futó statisztika nem frissül, így a val loss a rögzített modell empirikus becslése a val spliten. A labor az utolsó kisebb batch jelenlétét asserttel, a súlyozást pedig a `loss.item() * yb.size(0)` szorzással biztosítja.

12.1.11 Következő alkalmazás

A 12.2. Checkpoint és folytathatóság a checkpoint írását és folytathatóságát tárgyalja; ott az itt bevezetett eval mód a mentés előtti állapotkonzisztencia feltétele lesz, a súlyozott epoch-loss pedig a tanítási napló szokásos metrikája.

12.1.12 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D08] PyTorch contributors (2026). *torch.utils.data*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.
- [D12] PyTorch contributors (2026). *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30.

12.2 Checkpoint és folytathatóság

12.2.1 Cél, előismeret és induló becslés

Ez az alfejezet azt a kérdést rendezi, mit jelent valójában „elmentettük a modellt”. A cél: megkülönböztetni a csak következtetésre alkalmas súlyállapotot a tanítás folytathatóságát biztosító teljes állapottól, és megadni, milyen feltételek mellett ad egy megszakított tanítás a megszakítás nélkülivel azonos következő frissítést. Előismeret: a 12.1. Tanítás, validáció és kiértékelési mód

tanítási iterációja és eval-módja, a 11.2. nn.Module és paraméterregisztráció `state_dict`- és bufferfogalma. Induló becslés: elég-e a paraméterek mentése ahhoz, hogy a tanítás ott folytatódjon, ahol abbamaradt? Momentumos optimalizáló esetén a válasz nem, és ezt alább egy számpélda pontosan megmutatja.

12.2.2 Checkpoint, inference-súly és tanítási állapot

Definíció. Egy modell `state_dict`-je (állapotszótár) rendezett kulcs–érték leképezés, amely a modell összes `Parameter` és perzisztens (alapértelmezett, `persistent=True` jelzéssel regisztrált) buffer értékét tartalmazza; a `persistent=False` és a `None` bufferek kimaradnak belőle. Általános modulnál a `get_extra_state()` extra állapotot is tárolhat, ezért „a `state_dict` minden értéke tenzor" nem teljes körű állítás. A kulcsok a modulhierarchiát tükrözik, például egy `nn.Sequential(nn.Linear(3,4), nn.ReLU(), nn.Linear(4,1))` modellben:

- `0.weight` alakja $(4, 3)$, `0.bias` alakja $(4,)$,
- `2.weight` alakja $(1, 4)$, `2.bias` alakja $(1,)$.

BatchNorm-os modellben a bufferek is a `state_dict`-ben vannak: `running_mean` alakja $(C,)$, `running_var` alakja $(C,)$, `num_batches_tracked` egy int64 tenzor. A `state_dict` értékei a beépített moduloknál tenzorok, a kulcsok stringek; a `get_extra_state()`-et definiáló saját modul ettől eltérő, tetszőleges szerializálható objektumot is a szótárba tehet.

Következtetési (inference-only) állapot. Ha csak a modell `state_dict`-jét mentjük, a modell újra betölthető és `model.eval()` módban előre számítható, de a tanítási folyamat belső állapota elveszett. Az optimizer momentum- vagy Adam-pufferei, az ütemező számlálója és a véletlenszám-generátor állapota nincs benne.

Teljes tanítási állapot. A folytathatósághoz az alábbi komponensek együttes mentése szükséges:

1. **Modellállapot:** `model.state_dict()` — tanuló paraméterek és nem tanuló bufferek.
2. **Optimalizáló állapot:** `optimizer.state_dict()` — `param_groups` (például `lr`, `betas`, `eps`, `weight_decay`) és a paraméterenkénti belső puffer, például momentumos SGD-nél `momentum_buffer`, Adamnél `exp_avg`, `exp_avg_sq` és `step`.
3. **Ütemező állapot:** ha van, `scheduler.state_dict()` — a tanulási ráta ütemezésének belső pozíciója.
4. **RNG-állapot (főút, CPU):** pontosan azoknak a véletlenforrásoknak az állapota, amelyeket a kód ténylegesen használ. A globális PyTorch CPU-generátorhoz (dropout, `torch.randperm`, inicializálás) `torch.get_rng_state()`; ha a `DataLoader` saját generátort kap (`DataLoader(..., generator=g)`), a `g.get_state()`. A `shuffle=True` önmagában is a globális generátort használja: külön `generator` nélkül a sampler a globális PyTorch-generátorból sorsol, ezért ilyenkor a `torch.get_rng_state()` fedi le; külön `generator=g` átadásakor viszont a globális állapot érintetlen marad, és a `g` állapotát kell menteni. NumPy új API-nál (`rng = np.random.default_rng(seed)`) az `rng.bit_generator.state`, a régi globális API-nál `np.random.get_state()`; a Python `random` moduljánál `random.getstate()`. Amit a kód nem használ, azt nem kell menteni; amit használ, de nem mentünk, az a folytatás után más véletlen sorozatot ad.
5. **Pozíció:** `epoch`, `global_step`, valamint a napló szerinti `best_val_loss` és `best_step` a legjobb modell kiválasztásához.
6. **Adatfolyam-pozíció**, két külön esetben. (a) *Teljes batch-es (full-batch) tanításnál* (egy lépés a teljes train halmaz) nincs sampler-pozíció; epochhatáron mentve elég, ha a következő epoch sorrendjét sorsoló generátor állapota a 4. pontban mentve van. (b) *Minibatch-határon,*

epoch közben mentve az aktuális epoch indexsorrendjét is tárolni kell, vagy determinisztikusan rekonstruálni (az epoch elején mentett sampler-generátorállapotból újrasorsolva), és el kell menteni az elhasznált batch-ek számát; a folytatás ennyi batch kihagyásával indul. Ha `num_workers>0`, a workerek előre betöltött (prefetch), de még fel nem dolgozott batch-ei miatt az iterátor állapota nem menthető közvetlenül: az elhasznált batchszámból kell újraépíteni, a worker-seedeket pedig a mentett generátorállapotból rögzíteni. A sampler-RNG és az elhasznált batchszám önmagában nem garantálja a bitazonos folytatást véletlen augmentáció és több worker mellett: a worker-szintű Python/NumPy/Torch RNG-állapot, a prefetch és a `persistent_workers` állapot is releváns. Bitazonos folytatáshoz vagy állapotmentes (stateless), mintánkénti seedelés kell (a véletlen augmentáció seedje az `(epoch, minta-index)` párból számolt), vagy a bitazonos folytatást `num_workers=0`-ra szűkítjük, és tényleges folytatási tesztet futtatunk (megszakított és folytatott futás összevetése).

7. **Bővített út (GPU, kevert pontosság):** minden eszköz CUDA-generátorának állapota, `torch.cuda.get_rng_state_all()`; kevert pontosságnál a `GradScaler` állapota, `scaler.state_dict()` (aktuális skála, növekedési számláló). Enélkül a folytatás első lépései más skálával futnak, és más lépéseket dobnak el, mint a megszakítás nélküli futás.

A tankönyv CPU-s főútján az 1–6. pont a követelmény; a 7. pont csak a GPU-s, AMP-t használó bővített útra vonatkozik.

Az `epoch` nem tanult állapot, hanem pozíciószámláló: azt mondja meg, hányadik teljes adatbejárásnál tartunk. A legjobb pillanatkép (`best snapshot`) ettől különböző fogalom: a validációs veszteség szerint eddig legjobb modell **rögzített, független másolata**. A helyes másolat `{k: v.detach().clone() for k, v in model.state_dict().items()}` vagy `copy.deepcopy(model.state_dict())`. A sima referencia megtartása hiba, mert a `state_dict()` tenzorai a modell paramétereinek háttértárolójára mutatnak, így a következő `optimizer.step()` helyben végzett (in-place) frissítése a „legjobb” másolatot is megváltoztatja.

12.2.3 A bitazonos folytatás feltételei

Legyen egy tanítási lépés az

$$F:\ (w_t, s_t, r_t, g_t, \eta_t) \mapsto (w_{t+1}, s_{t+1}, r_{t+1})$$

függvény, ahol w_t a modell teljes állapota (a paraméterek és a bufferek együttese), s_t az optimizer belső állapota, r_t az RNG-állapot, g_t a batch-gradiens, η_t pedig az aktuális tanulási ráta. Momentumos SGD-nél s_t tartalmazza a momentum-puffert (és a hiperparamétereket); a 12.2.4 kézi példa skalár modelljében ezért a puffer jelölése $s_t = b_t$.

Megállapítás (azonos következő frissítés platformon belül). Ha az F által végrehajtott lebegőpontos műveletek sorrendje, dtype-ja és eszköze két futásban azonos, F determinisztikus, és a $w_t, s_t, r_t, g_t, \eta_t$ mindegyike bitazonos a két futásban, akkor a következő frissítés eredménye is bitazonos ugyanazon a platformon és könyvtárverzió mellett.

Indoklás. Ez nem önálló tétel, hanem a determinisztikus függvény definíciójának közvetlen következménye: azonos bemenet azonos kimenetet ad, ezért a „bizonyítás” maga a feltételek teljesülésének ellenőrzése. A tartalom nem a következtetésben, hanem a feltételekben van. A float összeadás nem asszociatív, ezért a bitazonossághoz a műveletek **azonos sorrendje** is kell, nem csak az azonos matematikai összeg. A [D02] dokumentáció ezt a platformok közötti irányba is kiterjeszti: a seed önmagában nem garantál bitazonosságot más platformon, verzióban vagy

kernelválasztásnál. A megállapítás tehát nem keresztplatformos ígéret; ez a tankönyv kísérleti főútján kis CPU-s, azonos környezetű futásokra vonatkozik.

A teljes állapot komponensei és a bitazonosság fenti feltételei együtt megadják, mit kell a checkpointba tenni, és mit kell a folytatásnál ellenőrizni:

- **Bitazonos modellállapot:** minden paraméter és buffer, beleértve a futó statisztikákat.
- **Bitazonos optimizerállapot:** SGD momentum-puffer, Adam `exp_avg`/`exp_avg_sq`/`step`, valamint a paramétercsoport-beállítások.
- **Bitazonos RNG-állapot:** ha a következő lépés dropoutot, augmentációt vagy más véletlen műveletet használ.
- **Azonos batch-gradiens:** ugyanaz az adatsorrend, batch-partíció és loss-redukció.
- **Azonos tanulási ráta:** scheduler nélkül a `param_groups['lr']`; schedulerrel annak állapota is.
- **Azonos futási környezet:** dtype, device, könyvtárverzió és op-sorrend; szükség esetén `torch.use_deterministic_algorithms(True)`.

12.2.4 Kézi számpélda: miért nem elég a súly?

Momentumos SGD, PyTorch-konvenció: b_t a momentum-puffer, g_t a gradiens, η a tanulási ráta, μ a momentum, dampening és weight decay nélkül. Ebben a példában a modell egyetlen skalár paraméterből áll, ezért $w_t \in \mathbb{R}$ a 12.2.3-beli teljes modellállapot skalár megfelelője, az optimizer belső állapota pedig maga a momentum-puffer: $s_t = b_t$; a μ és η hiperparaméter, nem része az állapotnak. A frissítés:

$$b_{t+1} = \mu b_t + g_t, \qquad w_{t+1} = w_t - \eta b_{t+1}.$$

Legyen $w_0 = 1$, $\eta = 0{,}5$, $\mu = 0{,}9$, $b_0 = 0$. Az első batch gradiense $g_0 = 2$, a másodiké $g_1 = 1$.

1. lépés: $b_1 = 0{,}9 \cdot 0 + 2 = 2$, $w_1 = 1 - 0{,}5 \cdot 2 = 0$.
2. lépés helyes pufferrel: $b_2 = 0{,}9 \cdot 2 + 1 = 2{,}8$, $w_2 = 0 - 0{,}5 \cdot 2{,}8 = -1{,}4$. 2 . lépés csak $w_1 = 0$ mentésével, friss pufferrel $b'_1 = 0$: $b'_2 = 1$, $w'_2 = 0 - 0{,}5 \cdot 1 = -0{,}5$.

Az eltérés $|w_2 - w'_2| = 0{,}9$. A különbség a következő lépésekben tovább halmozódik. Ez az ellenpélda mutatja, hogy az inference-súly nem elégséges a tanítás folytatásához.

12.2.5 Labor: megszakított és megszakítás nélküli futás összehasonlítása

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `checkpoint_lab.py` a `lab/n046_checkpoint/` munkakönyvtárból futtatható (`python checkpoint_lab.py`); a `checkpoint.pt` a futás kimenete, nem előzetes bemenet.

A labor munkakönyvtára `lab/n046_checkpoint/`; a `checkpoint_lab.py` az útvonalakat a saját helyéhez képest oldja fel. A cél annak kimutatása, hogy a teljes állapotból folytatott kis CPU-futás paramétereiben és veszteségében azonos a kontrollal, és hogy az RNG-állapot kihagyása ezt elrontja. A script szándékosan **nem** használ `DataLoader`-t: teljes batch-tel dolgozik, így a [D08] iterátorállapot kérdése nem terheli a reprodukciót. Az adat 16 darab 3-dimenziós szintetikus bemenet, a modell a 11.2. nn.Module és paraméterregisztráció alfejezetből ismert 3-4-1 MLP egy dropout réteggel, a loss MSE, az optimizer momentumos SGD. A dropout miatt a tanítási lépés a

globális PyTorch-generátort használja, ezért a folytatáshoz az RNG-állapot mentése ténylegesen szükséges: visszaállítás nélkül a dropout-maszkok eltérnek, és a folytatás nem bitazonos.

```
# lab/n046_checkpoint/checkpoint_lab.py - munkakönyvtár: lab/n046_checkpoint/

from pathlib import Path

import torch
from torch import nn, optim

HERE = Path(__file__).resolve().parent
HERE.mkdir(parents=True, exist_ok=True)

SEED = 20261020
DTYPE = torch.float32
DEVICE = torch.device("cpu")
LR = 0.5
MOMENTUM = 0.9
STEPS_A = 2
TOTAL = 4

torch.manual_seed(SEED)
torch.use_deterministic_algorithms(True)

X = torch.linspace(-1.0, 1.0, 48, dtype=DTYPE).reshape(16, 3)
Y = (1.5 * X[:, 0] + 0.5 * X[:, 1] - 0.7 * X[:, 2] + 0.2).unsqueeze(1)

def make_model():
    torch.manual_seed(SEED)
    return nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Dropout(0.5),
                         nn.Linear(4, 1)).to(DEVICE)

def one_step(model, optimizer):
    model.train()
    optimizer.zero_grad()
    loss = torch.nn.functional.mse_loss(model(X), Y)
    loss.backward()
    optimizer.step()
    return loss.item()

# 1. Kontroll: négy lépés egyben.
model_cont = make_model()
opt_cont = optim.SGD(model_cont.parameters(), lr=LR, momentum=MOMENTUM)
losses_cont = [one_step(model_cont, opt_cont) for _ in range(TOTAL)]

# 2. Checkpoint-ág: két lépés, teljes állapot mentése.
model_a = make_model()
opt_a = optim.SGD(model_a.parameters(), lr=LR, momentum=MOMENTUM)
losses_a = [one_step(model_a, opt_a) for _ in range(STEPS_A)]
cp = {
    "model": model_a.state_dict(),
    "optimizer": opt_a.state_dict(),
    "epoch": 0,
    "step": STEPS_A,
    "rng": torch.get_rng_state(),
}
torch.save(cp, HERE / "checkpoint.pt")

# 3. Új folyamat szimulálása ugyanabban a folyamatban: friss modell,
#    optimizer és betöltés.
model_b = make_model()
opt_b = optim.SGD(model_b.parameters(), lr=LR, momentum=MOMENTUM)
cp2 = torch.load(HERE / "checkpoint.pt", weights_only=True)
```

```
model_b.load_state_dict(cp2["model"])
opt_b.load_state_dict(cp2["optimizer"])
torch.set_rng_state(cp2["rng"])
losses_b = [one_step(model_b, opt_b) for _ in range(TOTAL - STEPS_A)]
losses_chk = losses_a + losses_b

# 4. Bitazonossági ellenőrzés a paramétereken.
max_abs = max((p1.detach() - p2.detach()).abs().max().item()
              for p1, p2 in zip(model_cont.parameters(), model_b.parameters()))
print("control_losses:", [f"{v:.6f}" for v in losses_cont])
print("resumed_losses:", [f"{v:.6f}" for v in losses_chk])
print(f"max_abs_diff={max_abs:.3e}")
assert max_abs < 1e-6
assert max(abs(a - b) for a, b in zip(losses_cont, losses_chk)) < 1e-6

# 4b. Negatív kontroll: RNG-visszaállítás nélkül a folytatás eltér.
model_c = make_model()
opt_c = optim.SGD(model_c.parameters(), lr=LR, momentum=MOMENTUM)
model_c.load_state_dict(cp2["model"])
opt_c.load_state_dict(cp2["optimizer"])
# szándékosan NINCS torch.set_rng_state(cp2["rng"])
losses_c = [one_step(model_c, opt_c) for _ in range(TOTAL - STEPS_A)]
max_abs_no_rng = max((p1.detach() - p2.detach()).abs().max().item()
                     for p1, p2 in zip(model_cont.parameters(), model_c.parameters()))
print(f"max_abs_diff_no_rng={max_abs_no_rng:.3e}")
assert max_abs_no_rng > 1e-6

# 5. A referenciamásolat hibájának bemutatása.
snapshot_ref = model_b.state_dict()          # referencia, nem független másolat
one_step(model_b, opt_b)                      # helyben módosító mutáció
print("reference_is_mutated:",
      snapshot_ref["0.weight"].equal(model_b[0].weight))
assert snapshot_ref["0.weight"].equal(model_b[0].weight)

snapshot_best = {k: v.detach().clone() for k, v in model_b.state_dict().items()}
one_step(model_b, opt_b)
print("clone_is_independent:",
      not snapshot_best["0.weight"].equal(model_b[0].weight))
assert not snapshot_best["0.weight"].equal(model_b[0].weight)
```

Futtatási parancs a `lab/n046_checkpoint/` könyvtárból: `python checkpoint_lab.py` (a szkript az útvonalakat a saját helyéhez képest oldja fel). A beadandó eredmény a `lab/n046_checkpoint/checkpoint.pt` fájl, a `control_losses` és `resumed_losses` listák, a `max_abs_diff`, a `max_abs_diff_no_rng` érték, valamint a `reference_is_mutated` és `clone_is_independent` logikai értékek. *Futtatott kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed* 20261020*; két futás kimenete ebben az azonos verziójú, egyszálas CPU-környezetben bitazonos, ami nem keresztplatformos garancia):*

```
control_losses: ['0.734751', '0.436869', '0.277654', '0.337675']
resumed_losses: ['0.734751', '0.436869', '0.277654', '0.337675']
max_abs_diff=0.000e+00
max_abs_diff_no_rng=2.641e-01
reference_is_mutated: True
clone_is_independent: True
```

A két losslista egyezése és a `max_abs_diff=0.000e+00` a teljes állapotú folytatás bitazonos voltát mutatja ezen a platformon; a `max_abs_diff_no_rng=2.641e-01` azt mutatja, hogy az RNG-állapot visszaállítása nélkül a dropout-maszkok eltérnek, és a folytatás nem bitazonos. A `0.000e+00` nem önmagában garantált érték: azonos műveletsorrend, dtype, eszköz és könyvtárverzió mellett adódik, más környezetben a kerekítés vagy a generátor állapota miatt eltérhet. A `reference_is_mutated: True` a sekély `state_dict`-referencia mutálódását, a

`clone_is_independent: True` a mély másolat függetlenségét igazolja. A 3. lépés ugyanabban a folyamatban épít friss modellt és optimizert, ezért a folyamat újraindítását szimulálja, nem valódi új processzust; a `.pt` fájlból való betöltés és az RNG visszaállítása viszont ugyanaz, mint egy új folyamatban.

A `weights_only=True` itt elegendő, mert a teljes tanítási állapot csak tenzorokat, primitíveket, listákat és szótárakat tartalmaz: az optimizer `state` részében tárolt int és float metaadat is a `weights_only` által engedett típus. A PyTorch 2.6 óta ez az alapértelmezés. A `weights_only=False` a teljes pickle-betöltést engedélyezi, ezért nem megbízható forrásból származó fájlnál tetszőleges kód végrehajtását teheti lehetővé; a labor a saját, megbízható fájlját is `weights_only=True`-val tölti be, és a mintakód sem javasolja a kikapcsolását. A betöltés biztonsági szempontjait a [D09] is tárgyalja.

12.2.6 Feladatok

F1 (fogalmi, nehézség 1). Fogalmazd meg, mi a különbség a következtetési (inference-only) `state_dict` és a teljes tanítási állapot között, és sorolj fel legalább négy komponenst az utóbbiból. Elvárt eredmény: definíció, komponenslista és egy példa, amelyben a hiányzó komponens megváltoztatná a következő frissítést.

F2 (levezetési, nehézség 2). A momentumos SGD képleteivel számold ki a $w_0 = 1$, $\eta = 0{,}5$, $\mu = 0{,}9$, $b_0 = 0$ sorozatot a $g_0 = 2$, $g_1 = 1$, $g_2 = -0{,}5$ gradiensekre. Add meg a 3. lépés előtti teljes állapotot, majd hasonlítsd össze a helyes és a csak w_2-ből indított hibás folytatást. Elvárt eredmény: képletek, minden közbülső érték és a két w_3 közötti eltérés.

F3 (implementációs, nehézség 2). Írj egy `save_training_state(model, optimizer, scheduler, epoch, step, path)` függvényt, amely a teljes állapotot menti, és egy `load_training_state(path, build_fn)` függvényt, amely friss modellt, optimizert és schedulert épít a `build_fn` hívással, majd betölti az állapotot. A labor MLP-jén ellenőrizd a betöltés utáni folytatást. Elvárt eredmény: futtatható kód, mentett fájl, `max_abs_diff < 1e-6`.

F4 (hibakeresési, nehézség 2). Adott a hiba: `best_state = model.state_dict()` elmentése a validáció után, majd további tanítási lépések, és a `best_state` a kiértékeléskor az új súlyokat tartalmazza. Lokalizáld az okot, írd le a következményt, add meg a javítást, és olyan ellenőrzést, amely a hiba jelenlétét kimutatja. Elvárt eredmény: ok–következmény–javítás–ellenőrzés lánc, `data_ptr` vagy `torch.equal` alapú teszt.

F5 (kísérleti/transzfer, nehézség 3). Tervezz kísérletet, amely 4 különböző seed mellett vizsgálja, hogy a checkpointból folytatott futás veszteséggörbéje statisztikailag egyenértékű-e a megszakítás nélküli kontrolléval. Adj protokollt: kérdés, adat, megfigyelési egység, baseline, változó, metrika, erőforrás, a CI értelmezése és a következtetési korlát. Az egyenértékűséget előre rögzített ekvivalenciaküszöb melletti ekvivalencia-teszttel (TOST) kell eldönteni; a nullát tartalmazó CI önmagában nem bizonyíték. Elvárt eredmény: 8-10 lépéses terv, alternatív metrika indoklása, futásidő-becslés és a mintavételi egység feltételes értelmezése. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

12.2.7 Megoldások és indoklás

F1 megoldás. A következtetési (inference-only) állapot a modell `state_dict`-je: paraméterek és bufferek, tehát a súlyok és a futó statisztikák. Következtetésre elég, de a tanítási folyamat nem folytatható belőle. A teljes állapot ezen felül tartalmazza: (i) optimizer `state_dict`

(momentum/Adam-pufferek, `step`, paramétercsoportok), (ii) scheduler `state_dict`, (iii) RNG-állapot, (iv) pozíció (`epoch`, `global_step`), szükség esetén (v) adatbetöltő-pozíció. Példa: ha csak a modell `state_dict`-jét mentjük momentumos SGD-nél, az újraindulás $b_0 = 0$ pufferrel számol, így az első folytatott lépés más irányú, mint a megszakítás nélküli futásban.

F2 megoldás. A frissítés: $b_{t+1} = \mu b_t + g_t$, $w_{t+1} = w_t - \eta b_{t+1}$.

- $t = 0$: $b_1 = 2$, $w_1 = 0$.
- $t = 1$: $b_2 = 0{,}9 \cdot 2 + 1 = 2{,}8$, $w_2 = 0 - 0{,}5 \cdot 2{,}8 = -1{,}4$.
- $t = 2$, helyes folytatás a $(w_2, b_2) = (-1{,}4; 2{,}8)$ állapottal: $b_3 = 0{,}9 \cdot 2{,}8 - 0{,}5 = 2{,}02$, $w_3 = -1{,}4 - 0{,}5 \cdot 2{,}02 = -2{,}41$.
- Hibás folytatás csak $w_2 = -1{,}4$ birtokában, $b'_2 = 0$: $b'_3 = -0{,}5$, $w'_3 = -1{,}4 - 0{,}5 \cdot (-0{,}5) = -1{,}15$.

Eltérés: $|w_3 - w'_3| = 1{,}26$. Alakellenőrzés: a példában minden mennyiség skalár, a b_t puffer és a g_t gradiens alakja ugyanaz (itt 0-dimenziós), ahogy a valódi hálóban is a gradiens alakját követi a momentum-puffer; a puffer elhagyása a gradiens előzményét nullázza.

F3 megoldás.

```
import torch

def save_training_state(model, optimizer, scheduler, epoch, step, path,
                        best_val_loss=None, best_step=None, extra_rng=None):
    """A teljes tanítási állapot mentése.

    extra_rng: a további RNG-források állapotai, a weights_only=True által
    engedett típusokkal (tenzor, bájt, primitív).
    """
    state = {
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict() if scheduler is not None else None,
        "epoch": epoch,
        "step": step,
        "best_val_loss": best_val_loss,
        "best_step": best_step,
        "rng": torch.get_rng_state(),
        "extra_rng": extra_rng,
    }
    torch.save(state, path)

def load_training_state(path, build_fn):
    state = torch.load(path, weights_only=True)
    model, optimizer, scheduler = build_fn()
    model.load_state_dict(state["model"])
    optimizer.load_state_dict(state["optimizer"])
    if scheduler is not None and state["scheduler"] is not None:
        scheduler.load_state_dict(state["scheduler"])
    torch.set_rng_state(state["rng"])
    position = {"epoch": state["epoch"], "step": state["step"],
                "best_val_loss": state["best_val_loss"],
                "best_step": state["best_step"],
                "extra_rng": state["extra_rng"]}
    return model, optimizer, scheduler, position
```

A `build_fn` három értéket ad vissza: `(model, optimizer, scheduler)`; a laborban ez a `make_model()` és az SGD-konstrukció, `scheduler=None` mellett, ezért a scheduler mentése és betöltése ebben a laborban nem fut le. A függvény a globális PyTorch RNG-t

(`torch.get_rng_state()`) és a hívó által átadott `extra_rng` szótárt menti; a 12.2.2 4. pontjában felsorolt további források közül a Python `random` állapota és a bájtok `weights_only=True` mellett átvihetők, a NumPy-állapot viszont NumPy-tömböt tartalmaz, ezért ezzel a beállítással nem tölthető be: bájtokká kell alakítani, vagy külön, megbízható forrásból `weights_only=False`-szal betölteni. Az ellenőrzés a labor 3-4. lépését ismétli: a visszatöltött modellen két további lépés után a paraméterek max abszolút eltérése a kontrolltól $< 10^{-6}$.

F4 megoldás. Ok: a `model.state_dict()` sekély (shallow) másolat, az értéktenzorok a modell paramétereinek háttértárolójára mutatnak; az `optimizer.step()` jellemzően helyben végzett (`add_`) frissítést végez, ezért a tárolt `best_state` is mutálódik. Következmény: a legjobb modell kiválasztása torzul — a mentett állapot a későbbi, rosszabb súlyokat tartalmazza, miközben a rendszer a korábbi legjobb validációs értéket tulajdonítja neki; így a kiválasztott checkpoint nem az, amelyet a validációs loss alapján kiválasztottunk, és a rá épülő tesztkiértékelés eredménye is értelmezhetetlenné válik. Javítás: `copy.deepcopy(model.state_dict())` vagy `{k: v.detach().clone() for k, v in model.state_dict().items()}`. Ellenőrzés: a másolat `data_ptr()`-e térjen el a paraméterétől, vagy egy helyben végzett lépés után `snapshot["0.weight"]` ne legyen `torch.equal` a `model[0].weight`-tel. A labor 5. pontja éppen ezt a negatív és pozitív tesztet mutatja.

F5 megoldásminta. (1) Kérdés: a teljes állapotból folytatott futás val loss-görbéje statisztikailag egyenértékű-e a megszakítás nélküli kontrolléval; a bitazonosságot seeden belül külön ellenőrizzük, mert az erősebb állítás, mint a statisztikai egyenértékűség. (2) Adat: a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab a könyv saját, szintetikus, 24×24-es vonalorientációs adatkészlete: 180 jelenet, jelenetenként két nézet, összesen 360 kép, három osztállyal; a felhasznált fájlok `data/images.npy` és `data/labels.npy`, a befagyasztott, osztályonként rétegzett split 120/30/30 jelenet, azaz 240/60/60 kép. A tanítás a train, az értékelés a val spliten történik. Ez nem a 12.2.5 labor adata: ott 16 darab 3-dimenziós szintetikus tenzor szerepel, GlyphLab-kép nélkül. (3) Megfigyelési egység: egy seed által definiált teljes futás; a négy seed négy független egység. A képek a `scene_id`-n belül nem függetlenek, de ez a split kialakítását érinti (scene-szintű, osztályonként rétegzett felosztás), nem a statisztikai egységet: a páros különbséget seedenként számoljuk, ezért a képszintű korreláció nem növeli a mintaelemszámot. (4) Baseline: megszakítás nélküli 4 epoch ugyanazon a seeden; a tanítás full-batch, ezért egy epoch egyetlen optimizer-lépés, a 4 epoch 4 lépés. (5) Kezelés: 2 lépés után a teljes állapot mentése, új modell és optimizer építése, betöltés, majd további 2 lépés. (6) Metrika: elsődleges a seedenként párosított különbség a kontroll és a folytatott futás végső val loss-ában. Alternatív metrika az epochonkénti val loss-görbék párosított eltérése (például a négy epochon vett átlagos abszolút különbség), mert a végső loss csak a végállapotot méri, a görbe a folytatás dinamikáját is; a paraméterek közvetlen összevetése seedek között értelmetlen, mert a különböző inicializálás különböző súlyokhoz vezet, ezért a `max_abs_diff` csak azonos seeden belül, a bitazonosság ellenőrzésére használható. (7) Intervallum és döntés: a páros különbségek átlagára 90%-os, t-alapú CI készül ($n = 4$, $df = 3$, $t_{0,95;3} = 2{,}353$), amely megfelel a TOST két egyoldalú 95%-os próbájának; az ekvivalenciaküszöböt a kísérlet megtervezésekor kell rögzíteni, például $\delta = 0{,}01$ a validációs keresztentrópia abszolút különbségére. Egyenértékűséget csak akkor állítunk, ha a teljes 90%-os CI a $(-\delta, \delta)$ intervallumon belül van; a nullát tartalmazó CI csak azt jelzi, hogy a különbséget nem tudtuk kimutatni, nem bizonyít egyenértékűséget. Négy seed mellett a teszt ereje kicsi, ezért a nem egyértelmű eredményt „nem dönthető el”-ként kell jelenteni. Az intervallum feltételes, ismételt mintavételi értelmezése szerint a becslés a seedek közötti szórásra vonatkozik, nem a képek közöttiire. (8) Erőforrás: CPU, egy szál, kis MLP (a $1 \times 24 \times 24$ kép 576-dimenziós vektorrá lapítva, 16 tanh rejtett egység, 3 logit); a rögzített labor-környezetben (Python 3.12.3, PyTorch 2.15.0+cpu, 1 szál) mért analóg futás a 240 képből álló full-batch tanítással négy seedre egy friss folyamatban 0,6 s, bemelegítés után 0,01 s volt, ami csak nagyságrendi becslés; a tényleges GlyphLab-futás idejét a hallgatónak kell mérnie és rögzítenie. (9) Következtetési korlát: az eredmény csak a vizsgált

platformra, verzióra és a szintetikus GlyphLab-adatra vonatkozik; nem bizonyít keresztplatformos bitazonosságot, és nem általánosít más architektúrára vagy természetes képekre.

12.2.8 Tipikus tévedés és önellenőrzés

A klasszikus hiba a `best_state = model.state_dict()` referencia tárolása. A fenti labor 5. pontja méri: egy további lépés után `reference_is_mutated` igaz, mert a tárolt tenzor ugyanarra a memóriaterületre mutat. Önellenőrző kérdések: (1) Miért nem elegendő a súlymentés momentumos SGD-nél? (2) A betöltésnél miért kell az optimizer `state_dict`-je is? (3) A seed-platform bitazonosságot garantál? Válaszok: (1) a momentum-puffer az előző gradiensek állapotát hordozza; (2) mert a frissítés pufferből indul; (3) nem, a [D02] szerint a seed nem garantál platformok közötti bitazonosságot. Visszautalás: a 11.2. nn.Module és paraméterregisztráció `state_dict`-je a paramétereket és puffereket együtt kezeli; az 5.3. Mintavételi bizonytalanság és intervallum konfidenciaintervallum-logikája pedig arra figyelmeztet, hogy a seedek közötti szórás és a betanított modell kockázatának bizonytalansága külön fogalom, a bitazonosság egyetlen seeden belül sem statisztikai garancia a generalizációra.

12.2.9 Rövid összefoglaló

A checkpoint nem egyszerűen a súlyok mentése. A csak következtetési `state_dict` a paramétereket és buffereket tartalmazza, de a tanítás folytatásához az optimizer pufferei, az ütemező állapota, az RNG-állapot és a pozíció is szükséges. A fenti megállapítás szerint azonos platformon, dtype-on és műveletsorrend mellett a teljes állapot bitazonossága azonos következő frissítést ad; platformok között ez nem garantált. A labor kétlépéses megszakítással és újraépített modellel ellenőrzi a paraméterek azonosságát, és kimutatja, hogy a `state_dict` referencia hiba, a `detach().clone()` független másolat.

12.2.10 Következő alkalmazás

A 12.3. Sanity check és kis mintára túllillesztés sanity check ezt a folytathatóságot használja fel: a mentett állapotból kiindulva kis mintára való túllillesztéssel ellenőrzi, hogy a modell és az optimizer együtt valóban képes tanulni, mielőtt hosszabb kísérletet indítanánk.

12.2.11 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D08] PyTorch contributors (2026). *torch.utils.data*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.
- [D09] PyTorch contributors (2026). *Saving and Loading Models*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.

12.3 Sanity check és kis mintára túllillesztés

12.3.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben olyan gyors kontrolltesztet építünk, amelyek a tanítási csővezeték nyilvánvaló hibáit még a hosszadalmas kísérlet előtt kimutatják. Feltételezzük a 11.3. Dataset és DataLoader a split után alfejezetben bevezetett adatbetöltő és split-konvenció, valamint a 12.2. Checkpoint és folytathatóság alfejezetben tárgyalt mentett paraméterállapot legalább fogalmi ismeretét; saját modellből indulunk ki, ezért a kísérlet nem függ külső checkpoint betöltésétől. A fő kérdés: ha egy 16 képet tartalmazó mini-batchen, rögzített célon 200 epoch alatt a cross-entropy nem csökken 0,05 alá, hol keressük először a hibát — a modellben, az adatútban vagy az optimalizálóban?

12.3.2 Definíciók és a kontrolltesztek taxonómiája

Sanity check. Olyan kis költségű tesztelési eljárás, amely a teljes tanítási ciklus egy konkrét feltételezését ellenőrzi. Nem minősül kiértékelési protokollnak, mert a tanítási halmazt használja.

Tiny overfit. Rögzítsünk $D_{\text{tiny}} = \{(x_i, y_i) : i = 1, \dots, N\}$ halmazt a GlyphLab tanító részéből (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték), ahol egy elem egy $x_i \in \mathbb{R}^{1\times 24\times 24}$ (C, H, W) kép és egy $y_i \in \{0, 1, 2\}$ címke, N kicsi, tipikusan $8 \le N \le 32$. Három különböző alakot kell megkülönböztetni: egyetlen elem $x_i \in \mathbb{R}^{1\times 24\times 24}$, a belőlük képzett batch $X \in \mathbb{R}^{N\times 1\times 24\times 24}$; a kimenet egyetlen elemre $f_\theta(x_i) \in \mathbb{R}^K$ logit, a batchre $f_\theta(X) \in \mathbb{R}^{N\times K}$, itt $K = 3$. A tanítás kritériuma:

$$L_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_{\text{CE}}(f_\theta(x_i), y_i) < \varepsilon \quad \text{és} \quad A_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\operatorname{argmax}_k f_\theta(x_i)_k = y_i] = 1.$$

A ε választását rögzíteni kell; a laborban $\varepsilon = 0{,}05$.

Megjegyzés (a tiny overfit értelmezése). A tiny overfit *diagnosztikai elvárás* megfelelő kapacitású, külön beállított tesztmodellre és csővezetékre, nem szükséges feltétele minden jó modellnek: erős regularizáció, explicit kapacitáskorlát vagy ellentmondó, azonos inputokhoz tartozó címkék mellett egy jó modell sem feltétlenül memorizál.

Random labels. Ugyanaz a protokoll, de a címkevektort véletlenül megkeverjük: $\hat{y} = (y_{\pi(1)}, \dots, y_{\pi(N)})$ egy véletlen mintapermutációval, amelyet a kód `torch.randperm`-mal állít elő ([D26]); a címkéket az x-től függetlenül, azonos eloszlásból is újrahúzhatnánk. Ez a **randomizációs nullmodell**; a keverés nem garantál teljes függetlenséget: maradhatnak fixpontok, és ha a mintában egyetlen osztály van, a vektor változatlan marad. Ezért a következtetést a nullmodell **várható** viselkedésére mondjuk ki: a megkevert címke várhatóan nem hordozza az eredeti feladat $P(y \mid x)$ szabályát, de az állítás nem minden egyes realizációra vonatkozik. A sikeres túllillesztés azt mutatja, hogy a modell a kapacitásával a minta belső szerkezetét memorizálja.

Frozen weights. A befagyasztott paraméter nem azonos a matematikailag nulla gradienssel. Ha egy paraméternél `requires_grad=False`, a `.grad` `None` marad; ha az optimizernek átadott paraméterlista üres, a konstruktor `ValueError`-t dob; ha a loss egyik inputja sem igényel gradienst, a `backward` `RuntimeError`-t ad. Ezek három külön megfigyelés; a teszt a `None`-t, a kivételt és a számított nulla gradienst (ami érvényes, nem `None` tenzor) külön kezeli. A `grad_norm` a `None`

tagokat kihagyja, ezért a befagyasztott modellnél 0-t ad; ez jelentési konvenció: azt jelzi, hogy nincs számított gradiens, nem azt, hogy a matematikai gradiens nulla.

Label mapping. A képekhez tartozó célindexet konzisztens, de a valódi osztályozási feladathoz képest hibás leképezéssel állítjuk elő, például $g(y) = (y + 2) \bmod 3$ (a laborban a `[2, 0, 1]` tenzorral indexelve: $0 \mapsto 2$, $1 \mapsto 0$, $2 \mapsto 1$). A tanítási accuracy magas lehet, mert a modell a módosított célfüggvényt illeszti, miközben a felhasználói szerződés sérül.

Gradiensnorma. Legyen $\theta \in \mathbb{R}^P$ a modell összes paramétere. A teljes gradiens euklideszi normája:

$$G(\theta) = \|\nabla_\theta L(\theta)\|_2 = \left(\sum_{j=1}^{P} \left(\frac{\partial L}{\partial \theta_j} \right)^2 \right)^{1/2}.$$

A gyakorlatban a paramétertenzoronkénti Frobenius-normák négyzetének összegéből vonunk gyököt; ez éppen a teljes paramétervektor euklideszi normája, és különbözik az átlagos vagy skaláronkénti normától.

12.3.3 Mit bizonyít és mit nem bizonyít a sikeres tiny overfit

A fejezet levezetési kötelezettsége az állítástípus pontos rögzítése.

Megfigyelés (a tiny-overfit sanity check bizonyító ereje). Legyen D_{tiny} rögzített véges minta, f_θ a paraméterezett modell, és tegyük fel, hogy egy optimalizálási eljárás θ_0-ból indulva T lépésben $L_{\text{train}}(\theta_T) < \varepsilon$ értéket ér el. A T itt csak a végállapot indexe: a megfigyelés érvényességéhez az eljárásról nem kell többet tudni, mint hogy elérte θ_T-t. Ekkor:

1. *Amit bizonyít:* a forward, a loss-számítás, a backward és az optimizer együttesen képes volt egy olyan θ_T paramétert előállítani, amely a rögzített D_{tiny} mintán a cross-entropy átlagot ε alá csökkenti. Ha továbbá $A_{\text{train}}(\theta_T) = 1$, akkor a modell a mini-batchet tökéletesen osztályozza a tanított célok szerint.
2. *Amit nem bizonyít:* nem ad felső korlátot a D_{tiny}-n kívüli várható hibára, nem bizonyít általánosítást, nem bizonyítja, hogy θ_T a valódi $P(y \mid x)$ szabályt reprezentálja, és nem bizonyít globális konvergenciát vagy optimumot.

Indoklás. Az (1) állítás a feltétel és a loss definíciójának közvetlen átfogalmazása: ha mértük, hogy a loss a küszöb alá csökkent, akkor a csővezeték ezen a mintán működött. Ez empirikus egzisztenciaállítás, nem elméleti konvergenciatétel.

A (2) tagadásához ellenpéldát konstruálunk a random-label kísérletből. Legyen y'_i a fenti randomizációs nullmodell szerint megkevert címke; a permutáció nem garantál teljes függetlenséget, de várhatóan megszünteti az x és a címke közötti kapcsolatot, ezért y' várhatóan nem hordozza az eredeti feladat $P(y \mid x)$ szabályát. Ha a modell D_{tiny}-n ugyanúgy eléri $L_{\text{train}} < \varepsilon$ és $A_{\text{train}} = 1$ értéket, akkor a tiny-overfit kritérium teljesül. Ha a modell predikciója és egy független tesztminta címkéje független, akkor a várható pontosság $\sum_k P(\hat{y} = k)\pi_k \le \max_k \pi_k$, ahol π_k a teszt osztályarányai; a nullmodell várható értelmében tehát a tanult leképezés nem hordoz a feladatról információt, és ez az állítás nem minden realizációra vonatkozik. A sikeres tiny overfit tehát még azzal együtt sem zárja ki, hogy a modell a minta véletlen struktúráját memorizálta, és nem bizonyítja az általánosító képességet. Ez a gondolatmenet saját definíciókból és a random-label kísérlet szerkezetéből következik; nem tételként idézett szakirodalmi eredmény.

A gyakorlati következmény: a tiny overfit a *hibafeltáró* kapu, nem a *generalizációs* kapu. A generalizációra vonatkozó korlátokat a 8.1.–8.3. tárgyalja, és azok feltételei itt nem lépnek életbe.

12.3.4 Kézi példa: cross-entropy és logitgradiens

Vegyünk $N = 2$ mintát és $K = 2$ osztályt, logitokkal

$$Z = \begin{pmatrix} 1 & 2 \\ 0{,}5 & -1 \end{pmatrix}, \qquad y = (0, 1)^\top.$$

A softmax soronként, természetes logaritmussal: az első sor $p_1 = (e^1, e^2)/(e^1 + e^2) \approx (0{,}2689,\ 0{,}7311)$, a második $p_2 = (e^{0{,}5}, e^{-1})/(e^{0{,}5} + e^{-1}) \approx (0{,}8176,\ 0{,}1824)$. Az átlagos cross-entropy:

$$L = -\frac{1}{2}(\ln 0{,}2689 + \ln 0{,}1824) \approx -\frac{1}{2}(-1{,}3134 - 1{,}7016) = 1{,}5075 \approx 1{,}507.$$

A logitokra vonatkozó gradiens az átlagos CE-re

$$\nabla_Z L = \frac{1}{N}(\mathrm{softmax}(Z) - Y_{\mathrm{onehot}}) \in \mathbb{R}^{2\times 2},$$

ahol Y_{onehot} sorai $(1, 0)$ és $(0, 1)$. Az első sor $(-0{,}731,\ 0{,}731)$, a második $(0{,}818;\ -0{,}818)$; az átlagolás ($1/N = \frac{1}{2}$ szorzó) után a 2×2 logitgradiens

$$\nabla_Z L = \tfrac{1}{2}\begin{pmatrix} -0{,}731 & 0{,}731 \\ 0{,}818 & -0{,}818 \end{pmatrix} \approx \begin{pmatrix} -0{,}3655 & 0{,}3655 \\ 0{,}4088 & -0{,}4088 \end{pmatrix},$$

a Frobenius-norma $\sqrt{2\,(0{,}3655^2 + 0{,}4088^2)} \approx 0{,}7755$. (Ha a logitok egy biasos lineáris réteg kimenete, $z_k = w_k^\top h + b_k$, akkor a két sor oszloponkénti összege, $(0{,}0433;\ -0{,}0433)$, a bias gradiense, $\nabla_b L$; bias nélküli rétegben ez a mennyiség nem értelmezhető. Ennek normája $\approx 0{,}0612$, de ez nem a logitgradiens normája.) A példa mutatja a két ellenőrzési pontot: minden sor összege nulla, és a target oszlopában negatív, a másik oszlopában pozitív a gradiens; ha valamelyik feltétel nem teljesül, a backward vagy a loss skálázása hibás.

12.3.5 Labor: tiny overfit és hibadiagnosztika

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; futtatás a `lab/` könyvtárból: `python sanity_check.py`.

Munkakönyvtár: a GlyphLab `lab/` könyvtára, ahol a `data/images.npy`, `data/labels.npy` és `data/metadata.csv` elérhető. Futtatás:

```
cd lab
python sanity_check.py
```

A kód teljes; a kimenet a rögzített környezetben futtatott érték (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020; két futás kimenete ebben az azonos verziójú, egyszálas CPU-környezetben bitazonos, ami nem keresztplatformos garancia). A seed és a determinizmus viszonyát a [D02] foglalja össze. A `run epochs=200` paramétere 200 teljes-batch lépést jelent: mivel a 16 kép egyetlen batch, ez 200 epoch. A `run` a `history` listában rögzíti a loss-t: a 0. elem a frissítés előtti érték, a k. elem a k. frissítés utáni érték; a chart ebből a listából veszi a 0., 10., 50. és 200. pontot. A jelentett gradiensnorma a tanítás után, a végleges θ_T pontban számolt gradiensből származik (a ciklus után még egy `zero_grad` és `backward` fut), nem az utolsó lépés előtti értékből.

```
# lab/sanity_check.py - munkakönyvtár: lab/
import csv
import math
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(20261020)
np.random.seed(20261020)
device = torch.device("cpu")
dtype = torch.float32

# --- Adatút a metadata index és split mezője alapján ---
images = np.load("data/images.npy").astype(np.float32)
labels = np.load("data/labels.npy").astype(np.int64)
with open("data/metadata.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
# A CSV sorszáma nem azonos a képtömb sorszámával: az index mező köt.
assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                         "xmin", "ymin", "xmax", "ymax"], list(rows[0])
assert len(rows) == 360, len(rows)
assert [int(r["index"]) for r in rows] == list(range(len(rows)))
assert len(images) == len(labels) == len(rows), (images.shape, labels.shape)

# Scene-split invariáns: egy jelenet minden nézete ugyanabban a splitben van.
scene_splits = {}
for r in rows:
    scene_splits.setdefault(r["scene_id"], set()).add(r["split"])
assert all(len(s) == 1 for s in scene_splits.values()), "szivárgó jelenet"

train_idx = np.array([int(r["index"]) for r in rows if r["split"] == "train"])

# 16 train kép, osztályonként legalább 5
chosen = []
for c in (0, 1, 2):
    idx_c = train_idx[labels[train_idx] == c]
    assert len(idx_c) >= 5, (c, len(idx_c))
    chosen.extend(np.random.choice(idx_c, size=5, replace=False).tolist())
chosen = np.array(chosen, dtype=np.int64)  # 15 minta; a 16. más train kép
extra = np.random.choice(np.setdiff1d(train_idx, chosen), size=1, replace=False)
chosen = np.concatenate([chosen, extra])
assert len(set(chosen.tolist())) == 16

x = torch.tensor(images[chosen], dtype=dtype, device=device)  # (16,1,24,24)
y = torch.tensor(labels[chosen], dtype=torch.int64, device=device)  # (16,)

class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(8 * 12 * 12, 3)
```

```
    def forward(self, x):
        z = self.pool(torch.relu(self.conv(x)))
        return self.fc(z.flatten(1))

def grad_norm(model):
    sq = 0.0
    for p in model.parameters():
        if p.grad is not None:
            sq += float((p.grad.detach() ** 2).sum().item())
    return math.sqrt(sq)

def run(model, x, y, epochs=200, lr=1e-3):
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.Adam(params, lr=lr) if params else None
    loss_fn = nn.CrossEntropyLoss()
    # history[k] a k. frissítés utáni loss; history[0] a frissítés előtti érték.
    history = []
    with torch.no_grad():
        history.append(float(loss_fn(model(x), y).item()))
    if opt is not None:
        for _ in range(epochs):
            opt.zero_grad()
            loss = loss_fn(model(x), y)
            loss.backward()
            opt.step()
            with torch.no_grad():
                history.append(float(loss_fn(model(x), y).item()))
        # A jelentett gradiens a végleges theta_T pontban értendő.
        opt.zero_grad()
        loss_fn(model(x), y).backward()
    with torch.no_grad():
        acc = (model(x).argmax(1) == y).float().mean().item()
        final = float(loss_fn(model(x), y).item())
    return final, acc, grad_norm(model), history

# 1) clean tiny overfit
m = TinyCNN().to(device)
l1, a1, g1, h1 = run(m, x, y)
print(f"[clean tiny] loss={l1:.4f} acc={a1:.3f} grad_norm={g1:.4f}")
print("[clean tiny curve]", [(e, round(h1[e], 4)) for e in (0, 10, 50, 200)])
assert l1 < 0.2 and a1 >= 0.9

# 2) random labels
perm = torch.randperm(y.numel())
y_rand = y[perm].clone()
m2 = TinyCNN().to(device)
l2, a2, g2, _ = run(m2, x, y_rand)
print(f"[random label] loss={l2:.4f} acc={a2:.3f} grad_norm={g2:.4f}")
assert l2 < 0.2 and a2 >= 0.9

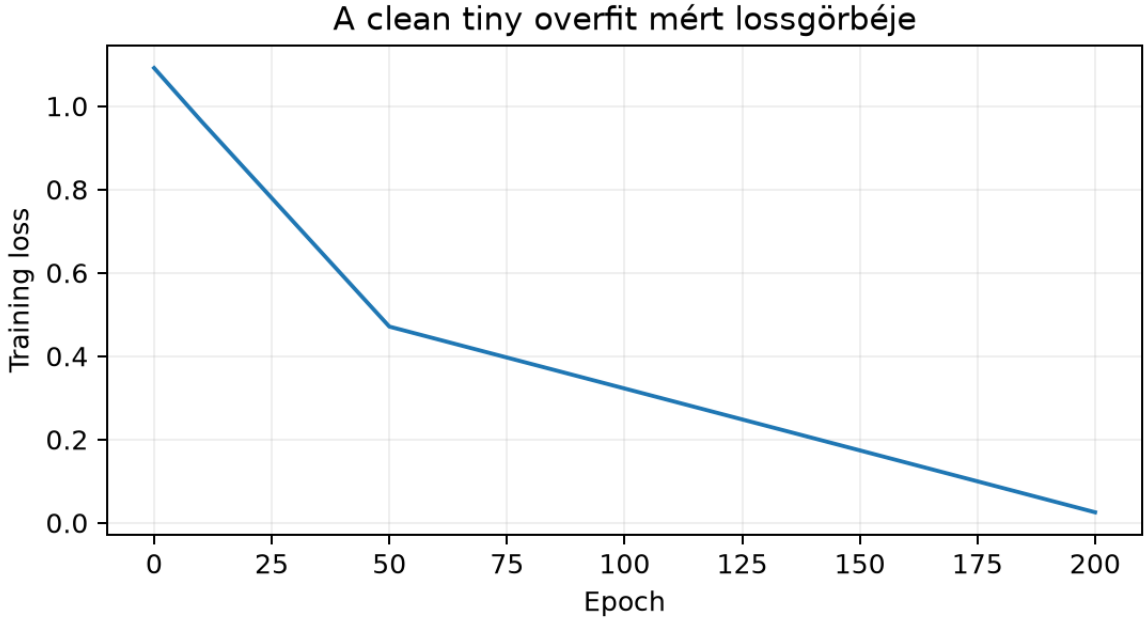
# 3) frozen weights
m3 = TinyCNN().to(device)
for p in m3.parameters():
    p.requires_grad_(False)
l3, a3, g3, _ = run(m3, x, y)  # nincs trainable paraméter; a tanítási ciklus nem fut
print(f"[frozen] loss={l3:.4f} acc={a3:.3f} grad_norm={g3:.4f}")
assert l3 > 1.0 and g3 == 0.0

# 4) cyclic label mapping
y_bad = torch.tensor([2, 0, 1], dtype=torch.int64)[y].to(device)
m4 = TinyCNN().to(device)
l4, a4, g4, _ = run(m4, x, y_bad)
```

```
print(f"[label mapping] loss={l4:.4f} acc={a4:.3f} grad_norm={g4:.4f}")
assert l4 < 0.2 and a4 >= 0.9
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020*; két futás kimenete ebben az azonos verziójú, egyszálas CPU-környezetben bitazonos, ami nem keresztplatformos garancia):*

```
[clean tiny]        loss=0.0253 acc=1.000 grad_norm=0.0566
[clean tiny curve] [(0, 1.0915), (10, 0.9644), (50, 0.4709), (200, 0.0253)]
[random label]      loss=0.0374 acc=1.000 grad_norm=0.0718
[frozen]            loss=1.0938 acc=0.438 grad_norm=0.0000
[label mapping]     loss=0.0257 acc=1.000 grad_norm=0.0570
```



12.1. ábra: A clean tiny overfit mért lossgörbéje

Mért pontok ugyanabból a futásból, a labor run függvényének history listájából (seed 20261020): a 0. pont a frissítés előtti, a 10., 50. és 200. pont a 10., 50. és 200. frissítés utáni loss. A vonal a mért pontok közötti interpoláció, nem közbülső mérés. A frozen esetben nincs tanítás, a loss a kezdeti 1,0938 értéken marad.

Értelmezés. A clean és a random-label kísérlet egyaránt 0,05 alá viszi a loss-t (a futtatásban 0,0253, illetve 0,0374); ezért a csővezeték gradiensáramlása működik, de a random-label siker jelzi, hogy a modell kapacitása elegendő a minta memorizálásához. A chart négy pontja ugyanebből a futásból, a `history` listából származik, ezért a rögzített kóddal reprodukálható. A frozen esetben nincs trainable paraméter, ezért az optimalizáló `None`, a tanítási ciklus nem fut, és a loss az inicializált modell értékén marad (a futtatásban 1,0938, accuracy 0,438); a jelentett `grad_norm` a `None` gradiensű paramétereket kihagyva 0, ami jelentési konvenció, nem matematikai nulla gradiens. A 1,0938 nem a $-\ln(1/3) \approx 1{,}099$ konstans, mert a háló kimenete a 16 mintán nem pontosan egyenletes, és a loss a véletlen inicializációtól függ. A label-mapping eset train accuracy-ja is 1, ami azt mutatja, hogy a cél belső konzisztenciája nem helyettesíti a felhasználói cél ellenőrzését.

12.3.6 Feladatok

F1 — Fogalmi; nehézség 1. Különböztesd meg a tiny overfit és a random-label sanity check által ellenőrzött hibatípusok körét. Adj egy-egy példát arra a hibára, amelyet az egyik megtalál, a másik viszont nem feltétlenül.

F2 — Levezetési; nehézség 2. Vezesd le a fenti kézi példa $\nabla_Z L$ alakját, és bizonyítsd be, hogy minden sorában a komponensek összege nulla. Mutasd meg, hogy ha a modell minden paraméterére `requires_grad=False`, akkor az autograd nem számít gradienset a paraméterekre (`p.grad is None`), ezért a `grad_norm` konvenciója szerint 0 adódik — és magyarázd meg, miért nem állítható, hogy a matematikai gradiens nulla.

F3 — Implementációs; nehézség 2. Írj egy `grad_norm` függvényt, amely a csatolt `TinyCNN` modellen visszaadja a teljes gradiensvektor euklideszi normáját, azaz a paramétergradiensek négyzetösszegének négyzetgyökét (nem a gradienskoordináták négyzetgyökeinek összegét), és teszteld egy mesterséges, ismert `p.grad` tenzorokkal rendelkező `nn.Linear(2,1)` modellen. Add meg a pontos várt kimenetet.

F4 — Hibakeresési; nehézség 2. A hallgató a következő hibás kódrészletet írta:

```
model = TinyCNN()
param_list = [torch.randn(4, 4, requires_grad=True)]
opt = torch.optim.Adam(param_list, lr=1e-3)
# ...
loss = loss_fn(model(x), y)
loss.backward()
opt.step()
```

A loss 200 epoch alatt sem csökken. Lokalizáld a hibát, írd le a következményét, javítsd, és add meg, milyen ellenőrzéssel igazolod a javítást.

F5 — Kísérleti/transzfer; nehézség 3. Tervezz sanity check protokollt egy 64×64 méretű, háromosztályos, színes képadathoz, ahol egy három konvolúciós rétegből álló CNN-t és 32-es batch méretet használnál. Add meg: a tiny minta méretét, az epochszámot, a küszöböt, a baseline-t, a gradiensnorma ellenőrzést, és foglald össze, mit következtethetsz a sikeres tiny overfitből az adott adatra. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

12.3.7 Megoldások és indoklás

F1. A tiny overfit a csővezeték end-to-end működését ellenőrzi egy tanulható, konzisztens célon: forward, loss, backward és paraméterfrissítés. Olyan hibát jelez, amely a loss csökkenését megakadályozza: rossz tenzoralak, a számítási gráf megszakítása (például `detach`), az optimizer nem a modell paramétereit kapja, rossz helyen lévő softmax vagy hibás label-dtype. A random-label teszt ugyanezt a csővezetéket futtatja véletlen címkéken, ezért a csővezeték ugyanazon hibáit elvileg szintén kimutatja; a többlete a kapacitás és a memorizálás próbája. Ha a modell a véletlen címkéken is eléri a küszöböt, az azt mutatja, hogy a modell elegendő kapacitású a 16 minta memorizálásához, és a csővezeték ezen a mintán működött; nem bizonyítja, hogy a csővezeték minden szempontból helyes, és nem bizonyít általánosítást. Az aszimmetria: egy túl kicsi modell a clean tiny halmazon még sikeres lehet, ha a valódi szabály egyszerű, a random-label teszten viszont elbukik, mert a véletlen címkékhez nincs egyszerű szabály; így a random-label a kapacitáskorlátot élesebben mutatja. Fordított irányban nincs ilyen tiszta példa: mivel a random-label ugyanazt a csővezetéket használja, a csővezeték hibája rendszerint mindkét teszten megjelenik, a clean tiny overfit elsősorban az olcsóbb első kapu. Címkehibát (például eltolt

osztályindexet) egyik teszt sem feltétlenül jelez, mert mindkettő a megadott célokra optimalizál; erre a label-mapping teszt való.

F2. A softmax definíciója soronként $s_k = \exp(z_k)/\sum_{k'} \exp(z_{k'})$. A cross-entropy egyetlen mintára

$$\ell = -\ln s_y = -z_y + \ln \sum_{k'} \exp(z_{k'}),$$

ezért a logitra vonatkozó parciális derivált

$$\frac{\partial \ell}{\partial z_k} = -\mathbf{1}[k = y] + \frac{\exp(z_k)}{\sum_{k'} \exp(z_{k'})} = s_k - \mathbf{1}[k = y].$$

Az átlagolás miatt $\nabla_Z L = \frac{1}{N}(\mathrm{softmax}(Z) - Y_{\mathrm{onehot}}) \in \mathbb{R}^{N \times K}$, ahol Y_{onehot} sorai one-hot vektorok; a kézi példában ez a $\nabla_Z L = \frac{1}{2}(\mathrm{softmax}(Z) - Y_{\mathrm{onehot}})$ mátrixot adja, amelynek sorai $(-0{,}3655;\ 0{,}3655)$ és $(0{,}4088;\ -0{,}4088)$. Minden sorban $\sum_k (s_k - \mathbf{1}[k = y]) = \sum_k s_k - 1 = 0$, tehát a sorösszeg nulla. A paraméterre való láncszabály miatt $\nabla_{\theta_j} L$ a logitgradiens és a réteg Jacobi-mátrixának szorzata, ami általában nem nulla. Ha minden paraméterre `requires_grad=False`, akkor az autograd nem épít gradienst ezekre a tenzorokra, tehát `p.grad is None` minden paraméternél; a `grad_norm` a `None` tagokat kihagyja, ezért 0-t ad. Ez jelentési konvenció, nem a matematikai gradiens eltűnése.

F3. Egy lehetséges megoldás:

```
import torch, math
import torch.nn as nn

def grad_norm(model):
    sq = 0.0
    for p in model.parameters():
        if p.grad is not None:
            sq += float((p.grad ** 2).sum().item())
    return math.sqrt(sq)

lin = nn.Linear(2, 1, bias=False)
lin.weight.grad = torch.tensor([[3.0, 4.0]])  # mesterséges feltöltés
print(f"{grad_norm(lin):.4f}")  # 5.0000
```

Ellenőrzés: a gradiensnorma a négyzetösszeg gyöke, $\sqrt{3^2 + 4^2} = 5$, nem $\sqrt{3} + \sqrt{4}$; a `bias=False` miatt a modellnek nincs bias paramétere, ezért bias gradiensről itt nem beszélhetünk, a feltételes ág pedig helyesen kezeli a `None`-t is. Szélsőeset: ha minden `p.grad is None`, a függvény a `None` tagokat kihagyva 0-t ad; ez a jelentési konvenció a frozen diagnózis alapja, és nem azt állítja, hogy a matematikai gradiens nulla.

F4. A hiba az, hogy az optimizer a modell paraméterei helyett egy független `param_list` listát kap. Következmény: a `loss.backward()` a modell paraméterein gradienset hoz létre, de az optimizer paramétercsoportjaiban nem a modell paraméterei szerepelnek; a `param_list` elemein viszont nincs gradiens (`grad is None`), ezért az Adam ([D06]) ezeket kihagyja. Nettó hatás: egyetlen paraméter sem frissül, a modell súlyai változatlanok maradnak, ezért a loss nem csökken. Javítás: `opt = torch.optim.Adam(model.parameters(), lr=1e-3)`. Ellenőrzés: a javított kódban a modell paraméterei két egymást követő lépés között megváltoznak (például a lépés előtti és utáni `state_dict` `torch.equal`-lel összevetve), a `param_list` elemei változatlanok maradnak, és a frissítés utáni θ ponton futtatott új `backward` után a `grad_norm(model)` pozitív. A gradiensnormát tehát a backward után kell mérni, nem a lépések előtti állapotból.

F5. Protokollterv. Kérdés: képes-e a három konvolúciós rétegből álló CNN a megadott mini-batchet megtanulni? Hipotézis: igen, 64 epoch alatt a training loss 0,05 alá csökken. Megfigyelési egység: egy kép. Adat: a saját 64×64 színes adat train részéből 48 képet választunk, osztályonként 16-ot; a val/test részt nem érintjük. A 32-es batch méret mellett ez két lépés epochonként (egy 32-es és egy 16-os részleges batch), így a részleges utolsó batch útja is tesztelt. A baseline: konstans predikció, amelynek loss-ja $-\ln(1/3) \approx 1{,}099$. Változó tényező: a cél (y és random-label), a paraméterfrissítés (aktív és frozen). Metrika: batch cross-entropy, train accuracy, gradiensnorma. Számítási keret: CPU, PyTorch 2.15.0+cpu, seed rögzítve. A 64 epoch (128 lépés) kiinduló büdzsé; nagyságrendje a 16 mintás labor 200 lépéses büdzséjével összevethető, és ha a görbe 64 epoch után még csökken, az epochszámot kell növelni, a küszöböt nem lazítani. A 0,05 küszöb a 12.3.2. szakaszban rögzített egységes tiny-overfit ε, nem a modell általánosítási minőségének mértéke. Érvényességi korlát: a sikeres tiny overfit csak a csővezeték működését igazolja az adott 48 képen; a tesztre vonatkozó állítást a záró kiértékelés után, külön protokollal tehetünk. A random-label kísérletet ugyanezen a 48 képen, a frozen esetet a modell paramétereinek `requires_grad_(False)` állításával futtatjuk; a gradiensnormát minden esetben egy külön `backward` után, a lépés utáni θ ponton mérjük, és rögzítjük, melyik ponthoz tartozik. Az aktív és a random-label változatban ez a backward működését jelzi; a frozen változatban a `None` gradiensek miatt 0 adódik, ami jelentési konvenció, nem matematikai nulla gradiens.

12.3.8 Gyakori tévedés és ellenpélda

A leggyakoribb hiba, hogy a sikeres tiny overfitet az általánosítás bizonyítékának tekintik, és a train accuracy 1 értékéből a tesztpontosságra következtetnek. Ellenpélda: a random-label kísérletben a tanítási loss 0,05 alá csökken és $A_{\text{train}} = 1$ adódik (a futtatásban 0,0374, illetve 1,000), pedig a célok véletlenszerűek; független tesztmintán a várható pontosság $\sum_k P(\hat{y} = k)\pi_k \leq \max_k \pi_k$, tehát nem 1. A tiny overfit tehát nem elégséges feltétele a megbízható modellnek, és a fenti megjegyzés szerint nem is szükséges feltétele minden jó modellnek.

12.3.9 Önellenőrzés és visszahívás

Ellenőrizd, hogy a laborban (a) minden tensor `float32` és `int64` a megadott helyeken, (b) a label mapping tenzora ugyanazon az eszközön van, mint a cél, (c) a `CrossEntropyLoss` ([D07]) bemenete logit, nem softmax, (d) a frozen esetben nincs trainable paraméter, ezért a tanítási ciklus nem fut (`backward` a kimenetre nem hívható, mert az nem igényel gradienst), és a loss az inicializált modell értékén marad. A korábbi fogalmak közül idézd vissza: a 11.3. Dataset és DataLoader a split után alfejezetben rögzített scene-split invariáns biztosítja, hogy egy jelenet minden nézete ugyanabban a splitben legyen, ezért a tiny minta minden eleme a train splitből származik, és ugyanazon jelenet másik nézete sem kerülhet a val vagy a teszt splitbe; a labor ezt a metadata alapján ellenőrzi; a 12.2. Checkpoint és folytathatóság alfejezetben tanult `state_dict`-fogalom alapján a `model.parameters()` a modellhez kötött paramétereket adja vissza, nem a mentett állapotot; a 8.3. Kapacitás, regularizálás és modellválasztás alfejezetben tárgyalt adaptivitás miatt a sanity check nem írhatja felül a teszt érintetlenségének elvét.

12.3.10 Rövid összefoglaló

A sanity check rövid, célzott kísérlet, amely a tanítási csővezeték feltételezéseit ellenőrzi. A tiny overfit kis batch-en azt mutatja meg, hogy a forward, loss, backward és optimizer együtt képes a loss-t csökkenteni. A random-label változat a kapacitást és a memorizálást teszteli, a frozen-változat a befagyasztás hatását (nincs trainable paraméter, ezért nincs frissítés), a

label-mapping a célkonzisztenciát, a gradiensnorma a backward numerikus működését. A sikeres tiny overfit nem bizonyít általánosítást: a random-label ellenpélda szerint az ε alatti loss elérhető akkor is, ha a célhoz nem tartozik stabil szabály. Ezért a sanity check a kísérleti protokoll hibafeltáró kapuja, nem a végső modellértékelés.

12.3.11 Következő alkalmazás

A 12.4. III. ellenőrző pont: önállóan tanuló kis háló III. ellenőrző pont ezt a kontrolltesztet teljes, önállóan tanuló kis hálózatként kapcsolja össze: a sanity check után a validációval, checkpointtal és a rögzített tesztprotokollal együtt értékeljük a tanulási görbét.

12.3.12 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D06] PyTorch contributors (2026). *torch.optim*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/optim.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D26] PyTorch contributors (2026). *torch.randperm*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.randperm.html. Hozzáférés: 2026-10-30.

12.4 III. ellenőrző pont: önállóan tanuló kis háló

12.4.1 Cél, előismeret és induló kérdés

Ez az egység a III. ellenőrző pont: a hallgató egy kézzel levezetett gradienssel, egy NumPy-referenciával, egy autograd-ellenőrzéssel, valamint egy teljes, futtatható MLP-kísérlettel ad át reprodukálható csomagot. A 12.3. Sanity check és kis mintára túlillesztés kontrollteszt (sanity check) után most a teljes tanítás–validáció–checkpoint–teszt ciklust zárjuk le.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg; itt csak az eltérés: a PASS-hoz a visszatöltött checkpoint (mentett modellállapot) kimenetének egyezését is ellenőrizni kell (lásd az F4 megoldását); a súlypont a D2 (kód és futtathatóság) és a D4 (bizonyíték és nyomon követhetőség) rubrikadimenzió. A D2 rubrikadimenziót ne téveszd össze a forrásjegyzék [D02] címkéjével.

Előfeltétel: 12.3. Sanity check és kis mintára túlillesztés, valamint a 11.4. Egy teljes PyTorch tanítási iteráció PyTorch-lépés és a 8.4. II. ellenőrző pont: korrekt tanulási állítás független teszt fogalma. Nem feltételezünk GPU-t, külső súlyt vagy előtanított modellt.

Induló kérdés: meg tudja-e adni a választott kis háló paramétergradienseit három független úton úgy, hogy az egyezés numerikusan ellenőrizhető, és a tanított modellből friss inputra futtatható prediction készül?

12.4.2 Definíciók, jelölések és értelmezés

MLP a laborban. A bemenet GlyphLab-kép (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték): $X \in \mathbb{R}^{N\times 1\times 24\times 24}$, float32, értékkészlete $[0,1]$. A modell ezt $\mathbb{R}^{N\times 576}$-ra lapítja, majd

$$H_1 = \mathrm{ReLU}(X_f W_1^\top + \mathbf{1}_N b_1^\top), \qquad Z = H_1 W_2^\top + \mathbf{1}_N b_2^\top,$$

ahol $X_f \in \mathbb{R}^{N\times 576}$, $W_1 \in \mathbb{R}^{32\times 576}$, $b_1 \in \mathbb{R}^{32}$, $W_2 \in \mathbb{R}^{3\times 32}$, $b_2 \in \mathbb{R}^3$. A képlet az eval módú forwardot írja le: H_1 a rejtett aktivációs mátrix (a lemmában H a rejtett dimenzió), train módban pedig a ReLU kimenetére még a `Dropout(0.1)` maszkja hat, ezért Z ilyenkor a dropout utáni aktivációból számol. A loss a stabil logit-alapú `CrossEntropyLoss`, cél $y \in \{0,1,2\}^N$, int64.

Visszatartott (held-out) metrika. A `val` split a checkpoint kiválasztásához használt, a `test` split csak a kiválasztás után egyszer értékelhető. Így a test a modellválasztás szempontjából visszatartott. A GlyphLab splitje scene-szintű: egy scene minden view-ja azonos splitbe esik.

train/eval. A modell `Dropout(0.1)` réteget tartalmaz, ezért a train és eval mód nem felcserélhető. Az `eval()` kikapcsolja a dropout véletlen maszkját; az `inference_mode` ezen felül kikapcsolja az autograd-könyvelést. A checkpoint mentése és a tesztelés ezért `eval` módban történik.

Checkpoint. A kísérlet a legjobb `val_loss`-hoz tartozó `state_dict`-et menti. A betöltés deklarált architektúrával és `weights_only=True` opcióval történik [D09].

12.4.3 A levezetési kötelezettség: háromféle gradiens egyeztetése

Az alábbi lemmát a kis kézi példára alkalmazzuk; ez a fejezet levezetési kötelezettsége.

Lemma (kétlineáris MLP lokális gradiensvisszaterjesztése). Legyen $X \in \mathbb{R}^{N\times d}$, $W_1 \in \mathbb{R}^{H\times d}$, $b_1 \in \mathbb{R}^H$, $W_2 \in \mathbb{R}^{K\times H}$, $b_2 \in \mathbb{R}^K$. Legyen $A = XW_1^\top + \mathbf{1}_N b_1^\top$, $H_1 = \mathrm{ReLU}(A)$, $Z = H_1 W_2^\top + \mathbf{1}_N b_2^\top$, $P = \mathrm{softmax}(Z)$. A cél $y \in \{0,\dots,K-1\}^N$, a loss

$$L = \frac{1}{N}\sum_{n=1}^{N}\left(-\log P_{n,y_n}\right).$$

Ha a vizsgált pontban A egyetlen eleme sem nulla (így a ReLU differenciálható), akkor az $S = \mathbf{1}_{A>0} \in \mathbb{R}^{N\times H}$ ReLU-maszkkal

$$G = \frac{1}{N}(P - Y_{\mathrm{onehot}}) \in \mathbb{R}^{N\times K}, \quad \nabla_{b_2} L = G^\top \mathbf{1}_N, \quad \nabla_{W_2} L = G^\top H_1,$$

$$D = (GW_2)\odot S \in \mathbb{R}^{N\times H}, \quad \nabla_{b_1} L = D^\top \mathbf{1}_N, \quad \nabla_{W_1} L = D^\top X.$$

Ahol Y_{onehot} soronként a célosztály one-hot vektora.

Bizonyítás. A softmax + cross-entropy kombinált deriváltja logitok szerint $P - Y_{\mathrm{onehot}}$ 7.2. Softmax és többosztályos cross-entropy, az átlagolás miatt osztunk N-nel, tehát $\nabla_Z L = G$. Mivel $Z = H_1 W_2^\top + \mathbf{1}_N b_2^\top$, a Z szerinti G jelből a H_1-re visszaterjesztett nyers jel GW_2, a W_2 gradienséhez pedig $G^\top H_1$ tartozik; a b_2 gradiense a G oszlopösszege (a minták szerinti összegzés). A ReLU a pozitív preaktivációjú pontokon differenciálható, deriváltja 1, a negatívokon 0, ezért a H_1-re visszaterjesztett jelet az $S = \mathbf{1}_{A>0}$ maszkkal kell szorozni: $D = (GW_2)\odot S$. A kézi

példában A mindkét eleme pozitív, ezért ott $S = \mathbf{1}$ és $D = GW_2$. Az affin $A = XW_1^\top + \mathbf{1}_N b_1^\top$ leképezésnél a W_1 gradiense $D^\top X$, a b_1 gradiense $D^\top \mathbf{1}_N$. Az alakok zárnak, mert $D^\top X$ mérete $H \times d$, $G^\top H_1$ mérete $K \times H$. $\square$

Megjegyzés: a ReLU nullponti nem-differenciálhatósága kívül esik e példán; általános esetben a könyvtár által választott konvenciót a 9.4. Aktivációk és gradiensáramlás alfejezetben tárgyalja a szöveg.

12.4.3.1 Kézi kis példa

Legyen $N = 2$, $d = 2$, $H = 1$, $K = 2$:

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad y = (0, 1), \quad W_1 = \begin{pmatrix} 0{,}5 & -0{,}25 \end{pmatrix}, \quad b_1 = 0{,}1,$$

$$W_2 = \begin{pmatrix} 0{,}2 \\ -0{,}3 \end{pmatrix}, \quad b_2 = \begin{pmatrix} 0{,}0 \\ 0{,}1 \end{pmatrix}.$$

A preaktiváció $XW_1^\top + b_1 = (0{,}1; 0{,}6)$, mindkettő pozitív, ezért $H_1 = (0{,}1; 0{,}6)$. A logitok:

$$Z = \begin{pmatrix} 0{,}02 & 0{,}07 \\ 0{,}12 & -0{,}08 \end{pmatrix}.$$

A softmax-sorok közelítőleg:

$$P_1 \approx (0{,}487503; 0{,}512497), \qquad P_2 \approx (0{,}549834; 0{,}450166),$$

így $L \approx 0{,}7583$ (pontosabban 0,758299). A lemmából, $N = 2$-vel:

Mennyiség	Alak	Számított közelítő érték
G	2×2	$(-0{,}256249;\ 0{,}256249)$; $(0{,}274917;\ -0{,}274917)$
$\nabla_{W_2} L$	2×1	$(0{,}139325; -0{,}139325)$
$\nabla_{b_2} L$	2	$(0{,}018668; -0{,}018668)$
D	2×1	$(-0{,}128125; 0{,}137459)$
$\nabla_{W_1} L$	1×2	$(0{,}284251; 0{,}293585)$
$\nabla_{b_1} L$	1	$(0{,}009334)$

A fenti értékek kerekített kézi számpélda, nem gépi futási lelet; a kódos ellenőrzésnek a levezetésből számolt tizedesjegyekig egyeznie kell.

12.4.3.2 Kétutas ellenőrzés kódja

A következő szkript a levezetett NumPy-backwardot a kézi adatra számolja, majd ugyanezt autograddal veti össze. Munkakönyvtár: `lab/n048`.

```
# gradient_check_n048.py
import numpy as np
import torch

def stable_softmax(z):
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def mlp_forward_backward(x, y, w1, b1, w2, b2):
    # x: (N,d), y: (N,), w1: (H,d), b1: (H,),
    # w2: (K,H), b2: (K,)
    a = x @ w1.T + b1           # (N,H)
    h = np.maximum(a, 0.0)
    z = h @ w2.T + b2           # (N,K)
    p = stable_softmax(z)
    N = x.shape[0]
    y_onehot = np.zeros_like(p)
    y_onehot[np.arange(N), y] = 1.0
    G = (p - y_onehot) / N      # (N,K)
    d_w2 = G.T @ h              # (K,H)
    d_b2 = G.sum(axis=0)        # (K,)
    dh = (G @ w2) * (a > 0)     # (N,H), ReLU-maszk
    d_w1 = dh.T @ x             # (H,d)
    d_b1 = dh.sum(axis=0)       # (H,)
    loss = -np.log(p[np.arange(N), y]).mean()
    return (d_w1, d_b1, d_w2, d_b2), loss

x = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=np.float64)
y = np.array([0, 1], dtype=np.int64)
w1 = np.array([[0.5, -0.25]], dtype=np.float64)
b1 = np.array([0.1], dtype=np.float64)
w2 = np.array([[0.2], [-0.3]], dtype=np.float64)
b2 = np.array([0.0, 0.1], dtype=np.float64)

np_grads, np_loss = mlp_forward_backward(x, y, w1, b1, w2, b2)

x_t = torch.tensor(x, dtype=torch.float64)
y_t = torch.tensor(y, dtype=torch.long)
w1_t = torch.tensor(w1, dtype=torch.float64, requires_grad=True)
b1_t = torch.tensor(b1, dtype=torch.float64, requires_grad=True)
w2_t = torch.tensor(w2, dtype=torch.float64, requires_grad=True)
b2_t = torch.tensor(b2, dtype=torch.float64, requires_grad=True)

z_t = torch.relu(x_t @ w1_t.T + b1_t) @ w2_t.T + b2_t
loss_t = torch.nn.functional.cross_entropy(z_t, y_t)
loss_t.backward()

for np_g, torch_g, name in [
    (np_grads[0], w1_t.grad, "w1"),
    (np_grads[1], b1_t.grad, "b1"),
    (np_grads[2], w2_t.grad, "w2"),
    (np_grads[3], b2_t.grad, "b2"),
]:
    np.testing.assert_allclose(np_g, torch_g.detach().numpy(),
                               rtol=1e-10, atol=1e-10,
                               err_msg=name)
    diff = np.max(np.abs(np_g - torch_g.detach().numpy()))
```

```
    print(f"{name}: NumPy={np_g} | autograd={torch_g.detach().numpy()} | "
          f"max_abs_diff={diff:.2e}")

np.testing.assert_allclose(np_loss, loss_t.item(), rtol=1e-10, atol=1e-10,
                           err_msg="loss")
print(f"loss: numpy={np_loss:.8f}, autograd={loss_t.item():.8f} | "
      f"abs_diff={abs(np_loss - loss_t.item()):.2e}")
```

Az ellenőrzés toleranciája f64 esetén `rtol=1e-10, atol=1e-10`; ez a két számított implementációt hasonlítja össze, nem a fenti, hat tizedesjegyre kerekített kézi táblázatot (a táblázat csak szemléltetés). A központi differenciás független kontrollt a 10.3. Gradiensellenőrzés és numerikus hibamérce protokoll szerint ugyanezen a ponton futtathatjuk; a fenti szkript viszont csak a NumPy-referenciát és az autogradot veti össze, ezért kétutas, nem háromutas egyeztetés.

12.4.4 Labor: reprodukálható MLP-kísérlet checkpointtal

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a labor-csomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; a `lab/n048/` könyvtárban először `python train_mlp_n048.py`, majd az így előállt `artifacts_n048/best_model.pt` checkpointtal `python predict_fresh_n048.py --checkpoint artifacts_n048/best_model.pt` fut.

Munkakönyvtár `lab/n048`, adat `../data`. Input: `images.npy`, `labels.npy`, `metadata.csv`; a betöltést a `torch.utils.data.DataLoader` végzi ([D08]). dtype `float32`, device CPU. Az értékelés a test splitet csak a legjobb checkpoint kiválasztása után végzi.

```
# train_mlp_n048.py
import argparse, csv, json, os, sys
import numpy as np
import torch
from torch.utils.data import DataLoader, Dataset

class GlyphMLP(torch.nn.Module):
    def __init__(self, in_dim=576, hidden=32, classes=3, p=0.1):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Flatten(),
            torch.nn.Linear(in_dim, hidden),
            torch.nn.ReLU(),
            torch.nn.Dropout(p),
            torch.nn.Linear(hidden, classes),
        )

    def forward(self, x):
        return self.net(x)

class ArrayDataset(Dataset):
    def __init__(self, x, y):
        if not (x.ndim == 4 and x.shape[1] == 1):
            raise ValueError(f"váratlan képalak: {x.shape}")
        self.x = torch.from_numpy(np.asarray(x, dtype=np.float32))
        self.y = torch.from_numpy(np.asarray(y, dtype=np.int64))

    def __len__(self):
        return len(self.y)

    def __getitem__(self, i):
        return self.x[i], self.y[i]
```

```python
def load_split(data_dir, n_images):
    # Csak a metadata.csv-ből bontunk; így a scene_id szerinti
    # függetlenséget ellenőrizni tudjuk. A rekordot az index mező
    # köti az images.npy sorához, nem a CSV sorszáma.
    with open(os.path.join(data_dir, "metadata.csv"), newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    if len(rows) != n_images:
        raise ValueError(
            f"a metadata sorainak száma ({len(rows)}) != images ({n_images})")
    idx = {"train": [], "val": [], "test": []}
    scenes = {"train": set(), "val": set(), "test": set()}
    for r in rows:
        s = r["split"]
        if s not in idx:
            raise ValueError(f"ismeretlen split: {s}")
        i = int(r["index"])
        if not 0 <= i < n_images:
            raise ValueError(f"index a tartományon kívül: {i}")
        idx[s].append(i)
        scenes[s].add(r["scene_id"])
    for a, b in (("train", "val"), ("train", "test"), ("val", "test")):
        if not scenes[a].isdisjoint(scenes[b]):
            raise ValueError(f"a {a} és {b} split scene-jei átfednek")
    return idx, rows

@torch.inference_mode()
def evaluate(model, loader):
    model.eval()
    total, correct, loss_sum = 0, 0, 0.0
    for x, y in loader:
        logits = model(x)
        loss_sum += torch.nn.functional.cross_entropy(
            logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item()
        total += len(y)
    return loss_sum / total, correct / total

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--data-dir", default="../data")
    p.add_argument("--out-dir", default="artifacts_n048")
    p.add_argument("--epochs", type=int, default=60)
    p.add_argument("--batch-size", type=int, default=32)
    p.add_argument("--lr", type=float, default=1e-3)
    p.add_argument("--weight-decay", type=float, default=1e-4)
    p.add_argument("--seed", type=int, default=20261020)
    args = p.parse_args()

    os.makedirs(args.out_dir, exist_ok=True)
    torch.manual_seed(args.seed)

    images = np.load(os.path.join(args.data_dir, "images.npy"))
    labels = np.load(os.path.join(args.data_dir, "labels.npy"))
    if images.dtype != np.float32:
        raise ValueError(f"az images dtype-ja {images.dtype}, nem float32")
    if not np.all((images >= 0.0) & (images <= 1.0)):
        raise ValueError("az images nem esik a [0,1] tartományba")
    if len(labels) != len(images):
        raise ValueError(f"labels ({len(labels)}) != images ({len(images)})")
    idx, rows = load_split(args.data_dir, len(images))

    loaders = {}
```

```
    for split in ("train", "val", "test"):
        ds = ArrayDataset(images[idx[split]], labels[idx[split]])
        g = torch.Generator().manual_seed(args.seed)
        shuffle = split == "train"
        loaders[split] = DataLoader(
            ds, batch_size=args.batch_size, shuffle=shuffle, generator=g)

    model = GlyphMLP()
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    best_val = float("inf")
    best_path = os.path.join(args.out_dir, "best_model.pt")
    history = []

    for epoch in range(1, args.epochs + 1):
        model.train()
        total_loss, count = 0.0, 0
        for x, y in loaders["train"]:
            optimizer.zero_grad(set_to_none=True)
            loss = torch.nn.functional.cross_entropy(model(x), y)
            if not torch.isfinite(loss):
                raise RuntimeError(f"nem véges loss az epoch {epoch} során")
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * len(y)
            count += len(y)
        train_loss = total_loss / count
        val_loss, val_acc = evaluate(model, loaders["val"])
        if not (np.isfinite(val_loss) and np.isfinite(val_acc)):
            raise RuntimeError(
                f"nem véges validációs metrika az epoch {epoch} során")
        history.append({"epoch": epoch, "train_loss": train_loss,
                        "val_loss": val_loss, "val_acc": val_acc})
        if val_loss < best_val:
            best_val = val_loss
            torch.save(model.state_dict(), best_path)
        print(f"{epoch:03d} train={train_loss:.4f} "
              f"val={val_loss:.4f} acc={val_acc:.4f}")

    state = torch.load(best_path, map_location="cpu", weights_only=True)
    model.load_state_dict(state)
    model.eval()
    test_loss, test_acc = evaluate(model, loaders["test"])
    result = {
        "protocol": "n048-III-checkpoint",
        "seed": args.seed,
        "best_val_loss": best_val,
        "test_loss": test_loss,
        "test_accuracy": test_acc,
        "history_val_acc": [h["val_acc"] for h in history],
        "history_train_loss": [h["train_loss"] for h in history],
        "command": " ".join([sys.executable, *sys.argv]),
    }
    with open(os.path.join(args.out_dir, "results.json"), "w", encoding="utf-8") as f:
        json.dump(result, f, indent=2)
    print(json.dumps(result, indent=2))

if __name__ == "__main__":
    main()
```

Futtatási parancs a `lab/n048` könyvtárból:

```
python train_mlp_n048.py --data-dir ../data --out-dir artifacts_n048 --epochs 60 --seed 20261020
```

A futás az `--out-dir` alatt a `best_model.pt` checkpointot és a `results.json` összegzést hozza létre; mindkettő a futtatás kimenete, nem kézzel szerkesztett bemenet.

Futtatott eredmény (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020*; két futás eredménye ebben az azonos verziójú, egyszálas CPU-környezetben bitazonos, ami nem keresztplatformos garancia). Részlet a futás naplójából (az első három és az utolsó három epoch) és a* `results.json` *három összegző mezője:*

```
001 train=1.0977 val=1.1040 acc=0.2667
002 train=1.0791 val=1.1082 acc=0.3500
003 train=1.0573 val=1.1141 acc=0.3500
...
058 train=0.1066 val=1.7754 acc=0.4667
059 train=0.1069 val=1.7688 acc=0.4667
060 train=0.0953 val=1.7905 acc=0.4667
best_val_loss=1.1039660135904947
test_loss=1.097758960723877
test_accuracy=0.38333333333333336
```

Ellenőrzés és értelmezés. A futás előtt a kód a dtype-ot, a $[0, 1]$ tartományt és a split scene-diszjunktságát ellenőrzi. A `results.json` rögzíti a seedet, a legjobb val_loss-t és a test pontosságát. A véletlen baseline class-balanced osztályok mellett 1/3; egészséges kísérletben a train loss csökken, a val_acc a véletlen 1/3 szintet meghaladja, és a test kiértékelés csak a rögzített checkpoint betöltése után történik. A mért futásban a train loss 1,0977-ről 0,0953-ra csökkent, a val_acc 0,2667-ről 0,4667-re nőtt, a val loss 1,1040-ről 1,7905-re nőtt (túlillesztés), a test accuracy 0,3833 lett; ez alig haladja meg az 1/3 véletlen baseline-t, ezért a futás nem általánosítási bizonyíték. A fenti `results.json` mezőket a futtatás tölti fel; a közölt értékek a rögzített környezetű futásból származnak. Az ellenőrző pont csak tényleges futtatással zárható le: a beadás része a saját `results.json`, a legjobb checkpoint és a futtatási napló, ezek nélkül a D2 és a D4 rubrikadimenzió nem értékelhető.

12.4.4.1 Friss inputos inference parancs

```python
# predict_fresh_n048.py
import argparse, csv
import numpy as np
import torch
from train_mlp_n048 import GlyphMLP

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--input", required=True)
    p.add_argument("--checkpoint", required=True)
    p.add_argument("--out", required=True)
    args = p.parse_args()

    x = np.load(args.input)
    assert x.ndim == 4 and x.shape[1:] == (1, 24, 24), x.shape
    assert x.dtype == np.float32, x.dtype
    assert np.all(np.isfinite(x)) and np.all((x >= 0.0) & (x <= 1.0))

    model = GlyphMLP()
    state = torch.load(args.checkpoint, map_location="cpu", weights_only=True)
    model.load_state_dict(state)
    model.eval()

    with torch.inference_mode():
        logits = model(torch.from_numpy(x))
        prob = torch.softmax(logits, dim=1).numpy()
```

```
    pred = prob.argmax(axis=1)
    with open(args.out, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["index", "predicted_class", "prob_0", "prob_1", "prob_2"])
        for i in range(len(x)):
            w.writerow([i, int(pred[i]), *[float(v) for v in prob[i]]])
    print(f"predikciók: {args.out}")

if __name__ == "__main__":
    main()
```

A parancs a tanító spliten kívüli inputra; a laborcsomagban elérhető `corrupted_test.npy` 60 képe ezt a szerepet tölti be:

```
python predict_fresh_n048.py \
  --input ../data/corrupted_test.npy \
  --checkpoint artifacts_n048/best_model.pt \
  --out artifacts_n048/fresh_predictions.csv
```

A friss input sémája: `(N,1,24,24)`, float32, $[0, 1]$; a `corrupted_test.npy` ezt teljesíti, és a tanító spliten kívülről származik. Ha a hallgatónak saját friss fájlja van, ugyanezzel a sémával adhatja át. A script a bemeneti sémát ellenőrzi, a checkpointot pedig a tanító scripttel azonos `GlyphMLP` osztályba, szigorú `load_state_dict`-tel tölti be; idegen kulcsú állapotszótár ezért hibát ad.

Korlátjegyzék. (1) A test accuracy pontbecslés, nem intervallum; arányra vonatkozó konfidenciaintervallumhoz (például Wilson-intervallum [S03]) az 5.3. Mintavételi bizonytalanság és intervallum szerinti külön mintavételi terv kell. (2) A seed, verzió és split rögzítése növeli az ismételhetőséget, de [D02] szerint platformok és verziók között nem garantál bitazonosságot. (3) A modell GlyphLab-orientációra tanult; valós képi általánosításra nem állítható. (4) A friss input eloszlásváltását a jelen checkpoint önmagában nem detektálja. (5) A GPU, az AMP, az augmentáció és a pretraining kívül esik a könyv főútján, azaz a CPU-s tanítási vonalon.

12.4.5 Feladatok

- **F1 (fogalmi, nehézség 1).** Mi a különbség `model.eval()`, `torch.inference_mode()` és a train mód között a fenti MLP-nél? Adjunk olyan esetet, ahol az `eval()` kihagyása torzítja a val_loss-t.
- **F2 (levezetési, nehézség 2).** Vegyük a lemma kézi példáját, de cseréljük a W_2 első elemét 0,2-ről 0,3-ra. Számítsa ki a b_2-gradienseket, és ellenőrizze az alakokat.
- **F3 (implementációs, nehézség 2).** Futtassa a `gradient_check_n048.py` programot; adja át a négy `max_abs_diff` értéket és a loss egyezését, valamint indokolja, miért `rtol=1e-10, atol=1e-10` a f64-es összevetés határa.
- **F4 (hibakeresési, nehézség 2).** Tekintsük a következő hibás betöltési és mentési szokásokat: `torch.load(path)` explicit `weights_only` nélkül (PyTorch 2.6 előtt ez `weights_only=False`-szal futott), valamint a mentés `torch.save(model, path)`. Mi a két hiba lehetséges következménye, és hogyan ellenőrizhető a javítás?
- **F5 (kísérlettervezési/transzfer, nehézség 3).** Tervezzen új procedurális rajzgenerátort, amely a GlyphLab-tól vastagságban és zajszintben eltér. Milyen protokollal mérné a checkpoint átvitelét anélkül, hogy a modellt ezen finomhangolná? A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

12.4.6 Megoldások és indoklás

F1. A train módban a `Dropout(0.1)` véletlen maszkkal nullázza a rejtett egységek egy részét, ezért a forward kimenet mintánként eltér. Az `eval()` kikapcsolja a maszkot, az inference_mode ezen túl megakadályozza a számítási gráf építését. Ha a validáció is train módban futna, a dropout véletlen zajt adna hozzá, és a val_loss nem a kiértékeléskori tényleges modellt mérné; ugyanazon modul hívással a val_loss realizációnként ingadozna. A javítás: minden nem tanító kiértékelés előtt `model.eval()`.

F2. Ha $W_2 = (0{,}3; -0{,}3)^\top$, a logitok és így a softmax is megváltozik, ezért $G = (P - Y_{\text{onehot}})/N$ is más lesz, mint az eredeti példában; a feladat kulcsa, hogy a b_2-gradiens $G^\top \mathbf{1}_N$, azaz a G oszlopösszege. Újraszámolva a logitok: az első sor $(0{,}03; 0{,}07)$, a második $(0{,}18; -0{,}08)$. A softmax-sorok $P_1 = (0{,}4900; 0{,}5100)$, $P_2 = (0{,}5646; 0{,}4354)$ közelítéssel, $y = (0, 1)$ mellett a G-sorok: $(-0{,}2550; 0{,}2550)$ és $(0{,}2823; -0{,}2823)$; ez valóban eltér az eredeti $(-0{,}2562; 0{,}2562)$, illetve $(0{,}2749; -0{,}2749)$ soroktól. Így $\nabla_{b_2} L = (0{,}0273; -0{,}0273)$. Alak-ellenőrzés: $G \in \mathbb{R}^{2\times 2}$, $\mathbf{1}_N \in \mathbb{R}^2$, az eredmény $\mathbb{R}^2$. A numerikus tizedesjegyek kerekítettek; a pontos érték a softmax újraszámításából következik.

F3. A `gradient_check_n048.py` futtatásakor a négy `max_abs_diff` a 10^{-17} nagyságrendben adódik (a laborprofilú környezetben mért értékek: 0, 0, $2{,}78 \cdot 10^{-17}$, $5{,}55 \cdot 10^{-17}$), a két loss pedig nyolc tizedesjegyig egyezik; a szkript ezt `assert_allclose`-val ellenőrzi. A `rtol=1e-10, atol=1e-10` tűrés a két számított implementáció egyezését méri, nem a fenti, hat tizedesjegyre kerekített kézi táblázatét: f64-ben jóval a kerekítési zaj szintje felett van, de a durva implementációs hibát (például hibás indexelést vagy eltérő képletet) elkapja; pusztán az összegzési sorrend megváltozása ezen a kis példán ennél kisebb eltérést ad. Az alakérv ebben a példában azért működik, mert $d = 2 \neq H = 1$; ha a `d_w1` nem $D^\top X$, hanem $X^\top D$ lenne, az alak nem zárna. Általában $d = H$ esetén ez az érvelés nem különböztetné meg a két változatot.

F4. A `torch.load` `weights_only` nélkül ([D09]) nemcsak tenzort, hanem deszerializálhat más Python-objektumokat is; idegen vagy manipulált checkpointnál ez biztonsági kockázat. A PyTorch 2.6 óta a `weights_only` alapértelmezése `True`, tehát a hiba a korábbi verziókra és a kifejezetten `weights_only=False` beállításra vonatkozik. A második hiba, `torch.save(model, path)`, a teljes modulobjektumot menti, ami importálhatósági és verziófüggő csatolást visz be, és nem különíti el a state_dictet. Következmény: a kísérlet nem reprodukálható más környezetben, és a betöltés hibája a modell definíciójának változásakor nehezen lokalizálható. Javítás: `torch.save(model.state_dict(), ...)`, majd a mentett `state_dict` betöltése deklarált modellbe `torch.load(..., weights_only=True)` és `load_state_dict`. Ellenőrzés: a betöltött modell kimenete ugyanazon fix bemenetre `allclose` a mentés előtti modellel, `eval` módban.

F5. Egy lehetséges terv: készítsünk 100 új procedurális jelenetet vastagabb vonallal és nagyobb zajjal, a címkézést ugyanazzal a szabállyal, mint a GlyphLab. A kiértékelés a befagyasztott checkpointkal történik; nem futtatunk rajta finomhangolást. Baseline: a záró GlyphLab-teszt accuracy, valamint a véletlen 1/3. Metrika: accuracy és confusion matrix a három orientációra. Összehasonlítás páronként ugyanazon checkpointkal, így a modelltényező rögzített. A következtetés határa: ha az accuracy csökken, ez a doménváltozásra (domain shift) utal, de nem azonosítja, hogy a vastagság vagy a zaj okolható-e; ehhez faktoronként egy-egy módosított adatkészlet kellene. A futási napló rögzíti a generátor seedjét, a scene_id szerinti egységet és a nyers predikciókat.

12.4.7 Tipikus tévedés, önellenőrzés és visszahívás

Tipikus tévedés ellenpéldával. „A korábbi eredmény újrakiírása nem újrafuttatás.” Ha a `results.json`-ban a test accuracy-t kézzel javítják egy kívánt értékre, de a `best_model.pt` és

a predikciók nem változnak, akkor a lelet nem hordoz reprodukálható bizonyítékot. A modell mentett súlya ezt leleplezi: a checkpointból újraértékelt pontosság nem egyezik a fájlba írt számmal. A megoldás az, hogy minden szám a betöltött checkpointból, azonos scripttel, a nyers predikciókkal együtt keletkezzen.

Önellenőrzés. (1) Minden gradient-check shape zár-e a lemmában? (2) A betöltött test pontosság a `best_val_loss` szerinti checkpointból jött-e, nem az utolsó epochból? (3) A friss input script ellenőrzi-e a dtype-ot, $[0, 1]$-t és a shape-et a modellhívás előtt?

Rövid válaszok. (1) Igen: a lemma a három számítás (kézi, NumPy, autograd) azonos alakját rögzíti, ezért minden gradient-check shape zár. (2) Igen: a test pontosságot a `best_val_loss` szerinti checkpointból kell újraszámolni, nem az utolsó epoch állapotából. (3) Igen: a friss input script a modellhívás előtt ellenőrzi a dtype-ot, a $[0, 1]$ tartományt és a shape-et.

Visszautalás. A 11.4. Egy teljes PyTorch tanítási iteráció szerint a `zero_grad` → forward → loss → `backward` → `step` sorrend, valamint a `step` előtti `isfinite` kontroll itt is feltétlenül érvényes. A 8.4. II. ellenőrző pont: korrekt tanulási állítás szerint a test csak a modellválasztás lezárása után szolgál független mértéknek.

12.4.8 Rövid összefoglaló

A III. ellenőrző pont a levezetést, a numerikus egyezést és a kísérletet egyetlen csomagban kapcsolja össze. A kézi példa egy kétlineáris MLP-re a softmax–cross-entropy $P - Y$ jeléből VJP-lánccal levezeti a súly- és biasgradienseket; a NumPy- és autograd-megvalósítás f64-ben, 10^{-10} tűréssel egyezik. A labor egy 32 rejtett egységes, dropoutos GlyphLab-MLP-t tanít, validációs loss alapján checkpointot ment, majd a test splitet csak a betöltött legjobb modellen értékeli. A friss inputos prediction script a bemeneti sémát ellenőrzi, és a checkpointot a tanító scripttel azonos modellosztályba tölti. A mért eredményt a fenti futtatás rögzíti.

12.4.9 Következő alkalmazás

A 13.1. Kép mint diszkrét többcsatornás jel a képet diszkrét többcsatornás jelként vezeti be, és ezzel a konvolúciós művelet előkészítését kezdi meg.

12.4.10 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D08] PyTorch contributors (2026). *torch.utils.data.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.
- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [S03] E. B. Wilson (1927). *Probable inference, the law of succession, and statistical inference.* Journal of the American Statistical Association.

13 Diszkrét jelek, képek és konvolúciós műveletek

13.1 Kép mint diszkrét többcsatornás jel

13.1.1 Cél, előismeret és induló kérdés

Az affin rétegek eddig $X \in \mathbb{R}^{N \times d}$ alakú bemenetet kaptak, ahol egy megfigyelés egyetlen vektorsor. A kép ezzel szemben két térbeli koordinátával és csatornákkal rendelkező diszkrét jel. Ennek az egységnek az a célja, hogy a képértéket, a színcsatornát, a koordinátát és a batchelemet pontosan elválassza, mert a következő konvolúciós egységekben minden levezetés ezen a tengelyszerződésen áll.

Előismeret: NumPy tömb és tengelyfogalom a 2.1. Python, NumPy és tenzoralak alfejezetből, mátrixindexelés a 3.2. Mátrixszorzás és affín réteg alfejezetből, valamint a 11.4. Egy teljes PyTorch tanítási iteráció alfejezetben rögzített bemeneti konvenció: a modell első rétege a dokumentált alakkal és skálával dolgozik.

Induló becslés: egy, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-kép alakja $(1, 24, 24)$. Egy kép hány skalárt tárol, és egy 360 képből álló batch mennyit? Hány koordináta, hány csatorna és hány megfigyelés van ebben a batchben? A választ a definíciók után pontos alakkal indokoljuk.

13.1.2 Definíciók: rács, pixel, csatorna, batch

Definíció (mintavételi rács). Rögzített $H, W \in \mathbb{Z}_{>0}$ magasság és szélesség mellett a mintavételi rács (sampling grid) az indexpárok

$$\mathcal{G}(H, W) = \{(h, w) \in \mathbb{Z}^2 : 0 \le h < H,\ 0 \le w < W\}$$

halmaza. A (h, w) pár a *sor-* és *oszlopkoordináta*; a könyv tömbkiíratási konvenciója szerint h lefelé, w jobbra növekszik. A rács $H \cdot W$ pontot tartalmaz. A koordináta tehát index, nem pixelérték.

Definíció (digitális kép). Egy C csatornás digitális kép egy

$$X \in \mathbb{R}^{C \times H \times W}$$

tenzor, ahol $X_{c,h,w} \in \mathbb{R}$ a c csatorna intenzitása a (h, w) rácspontban. A $c = 0, \dots, C-1$ index a *csatornakoordináta*, (h, w) a térbeli koordináta, az $X_{c,h,w}$ skalár pedig a képérték.

Definíció (pixel). Ebben a könyvben a pixel a rács egy pontjához tartozó csatornaérték-vektor: a (h, w) pontban

$$X_{:,h,w} = (X_{0,h,w}, \dots, X_{C-1,h,w})^\top \in \mathbb{R}^C.$$

Grayscale képnél ez egyetlen skalár; RGB esetén három számból álló vektor.

Definíció (batch). Ha N képet egyszerre tárolunk, a batch tenzor

$$X \in \mathbb{R}^{N\times C\times H\times W},$$

és $n = 0, \dots, N-1$ a batchelem indexe. Az n tehát megfigyelést azonosít, a c csatornát, a (h, w) pár térbeli pozíciót. A célcímke, a maszk vagy a doboz **nem** a képtensor része; a GlyphLab-szerződésben ezek külön tömbök, amelyek a térbeli rácsot megosztják a képpel.

Definíció (NCHW és NHWC). A tengelyek *szemantikáját* két elterjedt sorrend rögzíti:

Elrendezés	Tengelysorrend	Gyorsan változó index	Hol használjuk
NCHW	batch, channel, height, width	w	PyTorch `Conv2d`, [D10]
NHWC	batch, height, width, channel	c	gyakori NumPy/képi formátum

A két elrendezés ugyanazt a képet tárolhatja, de a memóriában másképp rendezi. A PyTorch hivatalos `Conv2d`-dokumentációja a bemenet *logikai* alakját (N, C, H, W)-ként írja elő; a memóriaelrendezés ettől függetlenül lehet `channels_last` is, ilyenkor az alak változatlan, csak a stride-ok mások [D10]. A 13.2. Konvolúció és cross-correlation alfejezetben bevezetett konvolúciós réteg is az (N, C, H, W) logikai alakot használja [D10].

Definíció (grayscale és RGB). A $C = 1$ eset grayscale: minden pixel egy intenzitásskalár. A $C = 3$ eset színkép; a jelen könyvben a $(c = 0, 1, 2) = (R, G, B)$ sorrend a konvenció. Ez nem a könyvtár automatikus ténye: az OpenCV például BGR sorrendet használ, ezért külső modelleknél a csatornasorrendet mindig a forrás dokumentációjából kell rögzíteni.

Az induló becslés válasza. Egy GlyphLab-kép $1 \cdot 24 \cdot 24 = 576$ skalárt tárol (1 csatorna, 24×24 rácspont). A 360 képből álló batchben $N = 360$ megfigyelés, $C = 1$ csatorna és 24×24 térbeli koordináta van, a skalárok száma pedig $360 \cdot 576 = 207\,360$. A 360 tehát a megfigyelések száma, nem a koordinátáké. A GlyphLab adatszerződését a `lab/ADATOK.md` és a `data/manifest.json` rögzíti.

13.1.3 Intenzitás és skálaválasztás: kimondott állítások

Állítás 1 (NCHW offset-képlet és index-tartomány). Legyen $N, C, H, W \in \mathbb{Z}_{>0}$ és $X \in \mathbb{R}^{N\times C\times H\times W}$ sormajor (row-major; NumPy-terminológiában C-order) rendben, összefüggő tömbben tárolva. A lineáris elemindexre

$$\mathrm{off}(n, c, h, w) = n\,CHW + c\,HW + h\,W + w.$$

Ez a leképezés bijekció az $I_N \times I_C \times I_H \times I_W$ indexhalmaz és $\{0, 1, \dots, NCHW - 1\}$ között, ahol $I_K = \{0, \dots, K-1\}$.

Bizonyítás. (i) A sormajor rend definíciója szerint az utolsó tengely stride-ja 1, tehát w lépésenként 1-et; h stride-ja W; c stride-ja HW; n stride-ja CHW. A kezdőcímtől mért elemeltolás ezért a fenti lineáris kombináció.

(ii) *Injektivitás.* Tegyük fel, hogy $\mathrm{off}(n,c,h,w) = \mathrm{off}(n',c',h',w') =: m$. Mivel $0 \le w, w' < W$, az m szám W-vel vett maradéka egyrészt w, másrészt w' (a többi tag W többszöröse), ezért $w = w'$. Kivonva w-t és W-vel osztva kapjuk, hogy $nCH + cH + h = n'CH + c'H + h'$. Mivel $0 \le h, h' < H$, a H-val vett maradékból $h = h'$. Kivonva h-t és H-val osztva $nC + c = n'C + c'$ marad; mivel $0 \le c, c' < C$, a C-vel vett maradékból $c = c'$, végül az osztás után $n = n'$. Minden lépésben azt használtuk ki, hogy a magasabb helyiértékű tagok a modulus többszörösei, ezért a maradék egyértelmű.

(iii) *Szürjektivitás.* Tetszőleges $m \in \{0, \dots, NCHW - 1\}$ esetén az euklideszi osztások egymás utáni alkalmazása

$$w = m \bmod W, \quad h = (m \div W) \bmod H, \quad c = (m \div (WH)) \bmod C, \quad n = m \div (CHW)$$

érvényes indexeket ad: $0 \le w < W$, $0 \le h < H$, $0 \le c < C$ és $0 \le n < N$, mert $m < NCHW$. A képletbe visszahelyettesítve m-et kapunk. Így minden lineáris index pontosan egyszer keletkezik. □

Megjegyzés (ellenőrzési típus). Az Állítás 1 tisztán egész-aritmetikai azonosság, nem empirikus állítás; ellenőrzése kódban egész indexszámítással történik. A képlet csak sormajor, összefüggő tárolásra érvényes. Egy tükrözött vagy transzponált *nézet* stride-jai eltérhetnek (akár negatívak is lehetnek); az általános alak $\mathrm{off} = b + ns_n + cs_c + hs_h + ws_w$, ahol b a tömb kezdőcíme, s_n, s_c, s_h, s_w pedig a négy tengely stride-ja.

Állítás 2 (NCHW–NHWC transzpozíció és inverze). Legyen $\pi = (0, 2, 3, 1)$ a tengelypermutáció. A $Y = \mathrm{perm}_{\pi}(X)$ tömb elemeire

$$Y_{n,h,w,c} = X_{n,c,h,w} \qquad \forall n, c, h, w.$$

A $\pi^{-1} = (0, 3, 1, 2)$ permutáció inverz, tehát $\mathrm{perm}_{\pi^{-1}}(Y) = X$; a kettős transzponálás az identitás. Következésképp $Y_{n,h,w,:} = X_{n,:,h,w}$, vagyis a pixel csatornavektora megmarad.

Bizonyítás. Ez a tengelypermutáció definíciójának közvetlen elemenkénti ellenőrzése: a Y tömb (n, h, w, c) pozíciójába a permutáció pontosan az X tömb (n, c, h, w) elemét írja; π^{-1}-et alkalmazva az elem az eredeti pozícióba kerül vissza. A kétforma tehát ugyanazt a képfüggvényt reprezentálja, csak a csatornatengely helye változik. □

Állítás 3 (uint8 tárolás és a „fehér = 1” skálaválasztás). Legyen $u \in [0, 1]$ normalizált intenzitás és $q = \mathrm{clip}(\lfloor 255u + \frac{1}{2} \rfloor, 0, 255) \in \{0, \dots, 255\}$ a 8-bites tárolási érték. A visszaolvasás $\hat{u} = q/255$ szigorúan monoton bijekció a $\{0, \dots, 255\} \to \{0, \frac{1}{255}, \dots, 1\}$ halmazra; speciálisan a maximális tárolási érték visszaolvasva 1. A 256-tal való osztás ezzel szemben $255/256 \approx 0{,}996$-ot adna, ezért a „0 fekete, 1 maximális intenzitás” konvenció a 255 osztót rögzíti.

Bizonyítás. $q = 255$ esetén $255/255 = 1$. Ha $q < q'$, akkor $q'/255 - q/255 = (q' - q)/255 > 0$. A $k \mapsto k/255$ leképezés tehát megőrzi a rendezést és injektív, két 256 elemű halmaz ($\{0, \dots, 255\}$ és a visszaolvasott értékek halmaza) között pedig bijektív. A második állítás közvetlenül a $\frac{255}{256} < 1$ egyenlőtlenségből következik: 256-tal osztva a maximális tárolási érték visszaolvasva $255/256 \approx 0{,}996$ lenne, nem 1, ezért a „maximális intenzitás = 1” konvenció a 255 osztót rögzíti. □

A GlyphLab-képek közvetlenül `float32` [0,1] tartományban vannak tárolva, ezért ott nincs kvantálási lépés. Dataset-szintű standardizáláskor a $\hat{x}_c = (x_c - \mu_c)/\sigma_c$ csatornánkénti affin transzformáció csak $\sigma_c > 0$ mellett értelmes; a per-image min-max normalizálás $\tilde{x} = (x - x_{\min})/(x_{\max} - x_{\min})$ konstans képen ($x_{\max} = x_{\min}$) nullával osztana, tehát ilyen bemenetre explicit leágazást kell definiálni.

13.1.4 Kézi kis példa és minimális kód

Tekintsünk egy $1 \times 1 \times 3 \times 4$ alakú, azaz $H = 3, W = 4$ grayscale képet átlós mintázattal:

$$X = \begin{pmatrix} 0{,}2 & 0 & 0 & 0 \\ 0 & 0{,}5 & 0 & 0 \\ 0 & 0 & 0{,}9 & 0 \end{pmatrix}.$$

Az NCHW offset-képlet $N = 1, C = 1$ miatt $\text{off}(c = 0, h, w) = hW + w$. A nemnulla elemek lineáris indexei: $(0,0) \to 0$, $(1,1) \to 1 \cdot 4 + 1 = 5$, $(2,2) \to 2 \cdot 4 + 2 = 10$. A sormajor, lapított tömbben tehát a 0,2 a nulladik, a 0,5 az ötödik, a 0,9 a tizedik indexen áll, a többi elem nulla.

RGB példa $(N, C, H, W) = (1, 3, 2, 2)$, csatornasorrend (R, G, B):

$$X_{0,R} = \begin{pmatrix} 0{,}1 & 0{,}2 \\ 0{,}3 & 0{,}4 \end{pmatrix}, \quad X_{0,G} = \begin{pmatrix} 0{,}5 & 0{,}6 \\ 0{,}7 & 0{,}8 \end{pmatrix}, \quad X_{0,B} = \begin{pmatrix} 0{,}9 & 1{,}0 \\ 1{,}0 & 1{,}0 \end{pmatrix}.$$

A $(h, w) = (0, 1)$ pixel vektora (0,2; 0,6; 1,0). NCHW lapításban a 0,6 elem címe $\text{off}(0, 1, 0, 1) = 1 \cdot 4 + 0 \cdot 2 + 1 = 5$; az NHWC transzponáltban $\text{off}(0, 0, 1, 1) = ((0 \cdot 2 + 0) \cdot 2 + 1) \cdot 3 + 1 = 4$, tehát ugyanannak a pixelnek a két elrendezés más lineáris címet ad. Az NCHW/NHWC a *logikai* tengelyrend; a fizikai elrendezést a `strides` rögzíti, és a `transpose` nézet nem rendezi át a puffert. Egy `(1,3,2,2)` `float32` NCHW tömb stride-jai (bájtban) `(48,16,8,4)`; a `np.transpose(X,(0,2,3,1))` nézet alakja `(1,2,2,3)`, stride-jai `(48,8,4,16)`, nem kontiguus, és ugyanazt a puffert használja (`np.shares_memory` igaz). Az itt számolt NHWC lapított cím (4) csak a kontiguus, materializált másolaté: `np.ascontiguousarray(Y)` stride-jai `(48,24,12,4)`, és új puffer. A PyTorch `channels_last` memóriaformátuma is NCHW *alakú* tenzor maradhat, csak más stride-okkal.

```
# munkakönyvtár: lab/n049_layout
import numpy as np

X = np.zeros((1, 3, 2, 2), dtype=np.float32)      # NCHW, [0,1]
X[0, 0, 0, 0], X[0, 0, 0, 1] = 0.1, 0.2          # R sor
X[0, 0, 1, 0], X[0, 0, 1, 1] = 0.3, 0.4
X[0, 1, 0, 0], X[0, 1, 0, 1] = 0.5, 0.6          # G sor
X[0, 1, 1, 0], X[0, 1, 1, 1] = 0.7, 0.8
X[0, 2] = [[0.9, 1.0], [1.0, 1.0]]               # B csatorna, explicit mátrix

Y = np.transpose(X, (0, 2, 3, 1))                # NCHW -> NHWC
print("NCHW:", X.shape, "NHWC:", Y.shape)
print("pixel (0,1):", Y[0, 0, 1, :])             # várt: [0.2 0.6 1. ]
# ugyanazon pixel három csatornája a transzponálás előtt és után
assert np.array_equal(X[0, :, 0, 1], Y[0, 0, 1, :])
print("G csatorna (0,1) NCHW lapított címe:", (1*2 + 0)*2 + 1)   # 5
print("G csatorna (0,1) NHWC lapított címe:", ((0*2 + 0)*2 + 1)*3 + 1)  # 4
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU; nincs véletlen, két futás kimenete bitazonos): `NCHW: (1, 3, 2, 2) NHWC: (1, 2, 2, 3)`, `pixel (0,1): [0.2 0.6 1. ]`, a két lapított cím 5 és 4; az `assert` igazolja, hogy a pixel csatornavektora a transzponálás előtt és után azonos.

13.1.5 Labor: mesterséges kép és memóriaelrendezés-ellenőrzés

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n049_layout/lab_n049.py` futtatható (`python lab/n049_layout/lab_n049.py`); a kép mesterséges, külső adatfájl nem kell.

A labor egy kis, osztálycélú mintára hasonlító mesterséges képpárt rajzol, majd NCHW és NHWC között transzponál és visszafelé is ellenőriz. Ez a GlyphLab-labor előtti gyakorlólépés; nem modellkísérlet.

```
# lab/n049_layout/lab_n049.py
import numpy as np
from pathlib import Path

out = Path(__file__).resolve().parent
PROJECT_ROOT = out.parents[1]
out.mkdir(parents=True, exist_ok=True)

H, W = 6, 7

X = np.zeros((2, 1, H, W), dtype=np.float32)       # NCHW, [0,1], CPU
X[0, 0, :, 2] = 1.0                                # 0. kép: függőleges vonal
X[1, 0, 3, :] = np.array([0.2, 0.4, 0.6, 0.8, 1.0, 1.0, 0.8],
                         dtype=np.float32)         # 1. kép: vízszintes sáv

assert X.shape == (2, 1, H, W)
assert X.dtype == np.float32
assert X.min() >= 0.0 and X.max() <= 1.0

Y = np.transpose(X, (0, 2, 3, 1))                  # NHWC nézet
assert Y.shape == (2, H, W, 1)
assert np.shares_memory(X, Y)                      # transzponálás: nézet, nem másolat

# NCHW offset kézzel: off(n,0,h,w)=((n*1+0)*H+h)*W+w
def off(n, h, w):
    return ((n * 1 + 0) * H + h) * W + w

flat = X.reshape(-1)
assert flat[off(1, 3, 4)] == X[1, 0, 3, 4]         # lapított cím ellenőrzése
assert Y[1, 3, 4, 0] == X[1, 0, 3, 4]              # ugyanaz a pixel NHWC-ben

X2 = np.transpose(Y, (0, 3, 1, 2))
assert np.array_equal(X, X2)                       # kettős transzponálás = identitás

np.savez(out / "mini_images.npz", x_nchw=X, x_nhwc=Y)
vissza = np.load(out / "mini_images.npz")
assert np.array_equal(vissza["x_nchw"], X)          # írás-újraolvasás kontroll
assert np.array_equal(vissza["x_nhwc"], Y)
print("NCHW:", X.shape, X.dtype, "min/max:", X.min(), X.max())
print("NHWC:", Y.shape)
print("ellenőrzött pixel X[1,0,3,4] =", X[1, 0, 3, 4])
print("mentve:", (out / "mini_images.npz").relative_to(PROJECT_ROOT))
```

Futtatás a könyv gyökérkönyvtárából (a szkript az útvonalakat a saját helyéhez képest oldja fel):

```
python lab/n049_layout/lab_n049.py
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a kimenet determinisztikus, nincs véletlen lépés, két futás kimenete bitazonos):

```
NCHW: (2, 1, 6, 7) float32 min/max: 0.0 1.0
NHWC: (2, 6, 7, 1)
ellenőrzött pixel X[1,0,3,4] = 1.0
mentve: lab/n049_layout/mini_images.npz
```

Értelmezés: a transzponálás nem változtatta a pixelértékeket, csak a csatornatengely helyét. A `np.shares_memory` igaz volta azt mutatja, hogy a két elrendezés ugyanazon az adaton osztozik; másolni csak akkor kell, ha összefüggő NHWC-tömbre van szükség. A `mini_images.npz` visszatöltése után a `np.array_equal` összevetés a lemezre írás-újraolvasás kontrollja is.

13.1.6 Feladatok

F1 (fogalmi; nehézség 1). A fenti RGB kézi példában $X \in \mathbb{R}^{1\times3\times2\times2}$. Írja fel a teljes NHWC Y tömböt elemenként, és adja meg, hogy a Y indexeiben hol található a $(h, w) = (0, 1)$ koordinátájú, G csatornájú 0,6 érték. Indokolja, miért nem veszít információt a transzpozíció, és adjon rövid magyarázatot, miért nem azonos mégis a két lapított memóriakép.

F2 (levezetési; nehézség 2). Legyen $N = 1, C = 3, H = 2, W = 2$. Az $(n, c, h, w) = (0, 1, 1, 1)$ logikai pixelre vezesse le az NCHW és az NHWC sormajor lapítási offset-képletet a stride-okból, és mutassa meg, hogy a két lineáris cím 7, illetve 10. Bizonyítsa be a két képlet különbségéből, hogy a két lapított sorrend akkor és csak akkor azonos minden indexre, ha $C = 1$ vagy $H = W = 1$.

F3 (implementációs; nehézség 2). Írjon futtatható függvényt `nchw_to_nhwc_checked(x)` néven, amely elfogad egy 4 dimenziós `float32` vagy `float64` NumPy-tömböt, `ValueError`-t dob rossz dimenziószámra, `TypeError`-t nem lebegőpontos dtype-ra, majd `np.transpose(x,(0,2,3,1))` módon NHWC-t ad. Adjon seedelt véletlen tesztet a $(2, 3, 4, 5)$ alakkal: shape, egy ismert pixel, osztott memória, visszatranszponálás (0 abszolút hibával), valamint két elutasítási tesztet: háromdimenziós bemenetre `ValueError`, `uint8` bemenetre `TypeError`.

F4 (hibakeresési; nehézség 2). Az alábbi kód egy grayscale `glyph.png`-t ad egy NCHW modellnek. Hol a hiba, mi a következménye, és hogyan javítható?

```
from PIL import Image
import numpy as np, torch

x = np.asarray(Image.open("glyph.png")).astype(np.float32)  # (H, W) szürke tömb
logits = model(torch.as_tensor(x))                          # modell 4D (N,C,H,W)-t vár
```

F5 (kísérleti/transzfer; nehézség 3). Egy GlyphLab-ra tanított osztályozót külső, saját rajzú vonalképekre akarunk átvinni. Tervezzen protokollt annak eldöntésére, hogy a rögzített `/255` skálázás elegendő-e, vagy a külső képek változó hátterén per-image min-max normalizálás kell. Adja meg: kérdés, hipotézis, adat és forrás, megfigyelési egység, split, baseline, változó tényező, metrika, erőforrás és a következtetés érvényességi határa. Ne igényeljen meglévő lefutást; tervet és buktatókat várunk. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

13.1.7 Megoldások és indoklás

F1. A transzponált $Y = \mathrm{perm}_{(0,2,3,1)}(X)$ elemei $Y_{0,h,w,c} = X_{0,c,h,w}$. Tehát

$$Y_{0,0,0,:} = (0{,}1;\ 0{,}5;\ 0{,}9), \quad Y_{0,0,1,:} = (0{,}2;\ 0{,}6;\ 1{,}0),$$
$$Y_{0,1,0,:} = (0{,}3;\ 0{,}7;\ 1{,}0), \quad Y_{0,1,1,:} = (0{,}4;\ 0{,}8;\ 1{,}0).$$

A G csatorna 0,6 értéke $Y_{0,0,1,1}$-ben van: az első index a batch, a második és harmadik a térbeli koordináta, az utolsó index a csatorna. A fenti négy sor kiírása a teljes Y tömb mind a négy térbeli pozícióját megadja, összesen $4 \cdot 3 = 12$ elemet. Az Állítás 2 szerint a két forma között invertálható index-permutáció áll, ezért adat nem vész el. A lapított memóriakép mégis eltér: az NCHW-ben a csatornastride HW, az NHWC-ben 1, ezért az elem lineáris címe 5, illetve 4.

F2. NCHW: off $= nCHW + cHW + hW + w = 0 + 1 \cdot 4 + 1 \cdot 2 + 1 = 7$. NHWC: off $= nHWC + hWC + wC + c = 0 + 1 \cdot 2 \cdot 3 + 1 \cdot 3 + 1 = 10$. A különbség általánosan

$$\Delta = \mathrm{off}_{\mathrm{NCHW}} - \mathrm{off}_{\mathrm{NHWC}} = c(HW - 1) + (1 - C)(hW + w)$$

(mert $nCHW = nHWC$). Ez pontosan akkor nulla minden indexre, ha $c(HW - 1) + (1 - C)(hW + w) = 0$ minden c, h, w-re. $C = 1$ esetén $c = 0$ és $1 - C = 0$, így $\Delta = 0$. Ha $C > 1$, válasszuk $c = 1, h = w = 0$: ekkor $HW - 1 = 0$, tehát $H = W = 1$ szükséges; ez az eset valóban nullát ad. Ezzel az „akkor és csak akkor” állítás kész. A megadott pontban $\Delta = 1 \cdot 3 + (1 - 3) \cdot 3 = 3 - 6 = -3$, azaz $7 - 10 = -3$.

F3. Futtatható minta; a kimenetet a rögzített környezetben ellenőriztük (Python 3.12.3, NumPy 2.5.4, CPU).

```
import numpy as np

def nchw_to_nhwc_checked(x):
    if x.ndim != 4:
        raise ValueError(f"négy dimenzió kell, kapott {x.ndim}")
    if x.dtype not in (np.float32, np.float64):
        raise TypeError(f"float32/float64 kell, kapott {x.dtype}")
    return np.transpose(x, (0, 2, 3, 1))

rng = np.random.default_rng(0)
x = rng.random((2, 3, 4, 5)).astype(np.float32)
y = nchw_to_nhwc_checked(x)
assert y.shape == (2, 4, 5, 3)
assert np.array_equal(y[1, 0, 2, :], x[1, :, 0, 2])
assert np.shares_memory(x, y)
vissza = np.transpose(y, (0, 3, 1, 2))
max_abs = float(np.max(np.abs(x - vissza)))
assert max_abs == 0.0
try:
    nchw_to_nhwc_checked(np.zeros((3, 4, 5), dtype=np.float32))
except ValueError:
    print("3D bemenet elutasítva: ValueError")
try:
    nchw_to_nhwc_checked(np.zeros((1, 3, 2, 2), dtype=np.uint8))
except TypeError:
    print("uint8 elutasítva: explicit float dtype-kérés")
print(f"Rendben: shape {y.shape}, dtype {y.dtype}, max abs hiba {max_abs:.1e}")
```

Futtatott kimenet: `3D bemenet elutasítva: ValueError`, `uint8 elutasítva: explicit float dtype-kérés`, majd `Rendben: shape (2, 4, 5, 3), dtype float32, max abs hiba 0.0e+00`. A nulla hiba nem tolerancia-kérdés: a transzponálás bitazonos másolatot ad, mert csak indexeket cserél, számot nem.

F4. Az `Image.open(...)` grayscale PNG-re (H, W) alakú tömböt ad. A `Conv2d` csak 3D, köteg nélküli (C, H, W) vagy 4D, kötegelt (N, C, H, W) bemenetet fogad el; a kétdimenziós bemenetet elutasítja (`RuntimeError: Expected 3D (unbatched) or 4D (batched) input to conv2d, but got input of size: [24, 24]`), tehát ez a hiba nem csendes. A javítás: adjunk batch- és csatornadimenziót, és alkalmazzuk a [0,1] skálakonvenciót.

```
img = Image.open("glyph.png").convert("L")                # 8-bites szürke
x = np.asarray(img, dtype=np.float32) / 255.0             # (H, W), [0,1]
t = torch.from_numpy(x).unsqueeze(0).unsqueeze(0)         # (1, 1, H, W)
assert t.shape == (1, 1, x.shape[0], x.shape[1])
assert float(t[0, 0, 0, 0]) == float(x[0, 0])
logits = model(t)
```

Az `Image.open` módja lehet például `L`, `1`, `RGBA` vagy `P`; a `.convert("L")` ezeket egységes 8-bites szürke tömbbé alakítja, ezért a `/255` skálázás minden ilyen módra értelmes. 16 bites vagy lebegőpontos forrásnál a konverzió után külön ellenőrizni kell a tartományt és a skálát. A valóban **csendes** hiba az azonos dimenziószámú tengelycsere. Ha a $(1, 1, H, W)$ tenzort valahol `x.transpose(-1, -2)` alakban, a H és W felcserélésével adjuk a modellnek, az alak továbbra is négydimenziós, a `Conv2d` hiba nélkül lefut, a kép viszont a főátlójára tükröződik: egy függőleges vonalból vízszintes lesz. Egy orientációra tanított modellnél ezért a bemenet jelentése megváltozik, és a függőleges és vízszintes osztály szerepe felcserélődhet; a diagonális osztály is torzulhat, a címkecsere iránya azonban képenként változó, nem általánosítható. Ezt csak tartalmi kontroll szűri ki, például egy ismert orientációjú tesztkép predikciója, vagy egy pixel helyének összevetése a transzformáció előtt és után.

Színes fájl esetén előbb rögzíteni kell a csatornasorrendet: PIL `convert("RGB")` után a NumPy-tömbön `np.transpose(x, (2, 0, 1))`, PyTorch-tenzoron `permute(2, 0, 1)` állítja elő a csatorna–magasság–szélesség sorrendet. Az OpenCV `cv2.imread` alapértelmezésben BGR-t ad, ezért ott `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`-vel kell RGB-re váltani, mielőtt a csatornasorrendet rögzítjük; a kétféle sorrend összekeverése csendes színhibát okoz.

F5. Egy lehetséges terv. *Kérdés:* változó hátterű külső vonalképeken a rögzített `/255` baselinehoz képest a per-image min-max javítja-e a validációs pontosságot, és nem okoz-e degenerált esetet? *Hipotézis:* a per-image transzformáció erősítheti a háttérzajt, konstans foltokon pedig osztás előtt leágazás kell. *Adat:* 60 saját, annotált külső kép (a GlyphLab-laborból származó split nem módosítható); megfigyelési egység a scene (scene_id), mert egy scene minden view-ja azonos oldalra kerül, a split pedig osztályonként arányos (class-stratified: minden orientációs osztály hasonló arányban kerül a train, val és test részbe). *Baseline:* modell rögzített, input `/255`, CPU. *Változó:* csak a preprocessing lépés (a modell súlya és seed rögzített). *Metrika:* val accuracy a visszatartott (held-out) spliten; a bizonytalanságot a scene-ek újramintavételezésével számolt bootstrap konfidenciaintervallum (CI) adja, nem képenkénti bootstrap. *Erőforrás:* CPU, kis batch. *Következtetési határ:* az eredmény erre a saját külső rajztartományra érvényes; egyetlen seed nem általánosítás, és a külső val spliten tilos a modellt újrahangolni; konstans képek arányát a jelentésben fel kell tüntetni.

13.1.8 Tipikus tévedés, önellenőrzés és visszahívás

Tévhit: a konstanssal való átskálázás csak „mértékegységet” cserél, ezért nem változtatja a döntést. Ez akkor igaz, ha a modellt az új skálán újratanítjuk, vagy a skálázást a súlyok kompenzáló átskálázásával együtt vezetjük be (például az első réteg súlyait és biasát a skálafaktorral megszorozva); egy fix, [0,1] bemenetre tanított modellnél a bemenet átskálázása önmagában nem no-op.

Ellenpélda (levezetésből, nem mért). Legyen $w = (1;\ -2;\ 0{,}5)^\top$, $b = -0{,}6$, bemenet $x = (0{,}4;\ 0{,}1;\ 0{,}7)^\top$. Az affin pontszám $z = w^\top x + b = 0{,}4 - 0{,}2 + 0{,}35 - 0{,}6 = -0{,}05$. Ugyanezt a képet nyers $[0, 255]$ skálán tárolva $x' = (102;\ 25{,}5;\ 178{,}5)$, és $z' = 102 - 51 + 89{,}25 - 0{,}6 = 139{,}65$. Ha a döntési szabály $\hat{y} = \mathbf{1}[z \geq 0]$ (a pontszám előjele adja az osztályt), akkor az eredeti bemenetre $\hat{y} = 0$, a nyers skálán $\hat{y} = 1$, tehát a skálaváltoztatás pusztán az adatbeadásnál megváltoztatja a modell inputjelentését. Ugyanígy az NCHW–NHWC csere nem no-op: az Állítás 2 megőrzi a pixelvektort, de a lapított memóriaképet átrendezi.

Önellenőrző lista. Minden tömbhöz felírtam-e az (N, C, H, W) alakot, a dtype-ot és a skálakorlátot? A kettős transzponálást `np.array_equal`-rel ellenőriztem? A lapított offset-képletet egész indexszámítással teszteltem, nem pixelérték-heurisztikával? A normalizálásnál kezeltem a konstans kép nullosztóját?

Visszautalás. A 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetből: numerikus egyezést dtype-, shape- és hibahatár-megadással ellenőrzünk; jelen egységben a transzpozíció bitazonos, ezért a hiba pontosan 0. A 11.4. Egy teljes PyTorch tanítási iteráció alfejezetből: a modell első rétegének `in_channels` várakozása és a betöltött kép csatornatengelye ugyanaz a szerződés; a dimenziórendezési hiba gyakran nem dob kivételt, hanem csendesen félreértelmezi a koordinátákat.

13.1.9 Rövid összefoglaló

A mintavételi rács a $\mathcal{G}(H, W) = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$ indexhalmaz; a pixel az egy rácsponthoz tartozó $\mathbb{R}^C$ csatornavektor, az intenzitás a konvenció szerinti skalár. Az NCHW tenzor indexeit (n, c, h, w), az NHWC-ét (n, h, w, c) azonosítja; a $Y_{n,h,w,c} = X_{n,c,h,w}$ permutáció invertálható, ezért adat nem vész, de a lineáris memóriacím megváltozik. Bizonyítottuk az NCHW sormajor offset-képletét és bijektivitását, a kettős transzponálás identitását, valamint a $q/255$ skála monoton voltát. A labor kis mesterséges képpáron a két elrendezést alakkal, osztott memóriával és pixel-egyezéssel ellenőrzi.

13.1.10 Következő alkalmazás

A 13.2. Konvolúció és cross-correlation a kernelmegfordítás nélküli cross-correlationt és a matematikai konvolúciót vezeti be ugyanezen a rácson; az itt rögzített NCHW-indexelés a súly- és kimeneti indexek levezetésének alapja.

13.1.11 Források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- GlyphLab belső szerződés: `lab/ADATOK.md`, `data/manifest.json`; az images.npy alakja $(360, 1, 24, 24)$ float32, értékkészlete [0,1].

13.2 Konvolúció és cross-correlation

13.2.1 Cél, előismeret és induló kérdés

A 13.1. Kép mint diszkrét többcsatornás jel egységben a képet a $\mathcal{G}(H, W) = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$ rácson definiált diszkrét jelként rögzítettük. Ebben az alfejezetben egyetlen szürkeárnyalatos képet vizsgálunk, tehát $X \in \mathbb{R}^{H \times W}$ mátrixot; a batch- és csatornaindexeket most rögzített, nem változó dimenzióknak tekintjük, és az általános $N \times C_{in} \times H \times W$ alak csak a 14.2. Több bemeneti és kimeneti csatorna egységben kerül elő.

Előismeret: a 13.1. Kép mint diszkrét többcsatornás jel rácspont-, pixel- és NCHW-fogalma, valamint az 1.2. Szakmailag megalapozott modellezési állítások állítástípus-osztályozása. Az induló kérdés: ha a `torch.nn.Conv2d` nevében a „convolution” szó szerepel, akkor valóban

matematikai konvolúciót számol-e? A válasz nemmel kezdődik, és ennek a nemleges válasznak a pontos tartalma adja az egység gerincét.

13.2.2 Definíció és intuíció

Kernel. A kernel (szűrőmag, filter kernel) egy kis, $k_h \times k_w$ méretű valós súlymátrix:

$$K \in \mathbb{R}^{k_h \times k_w}.$$

A kernel minden (a, b) pozícióhoz egy skalár súlyt rendel; a gyakorlatban $k_h = k_w = 3$ vagy 5 gyakori, de az alábbi definíciók páratlan méretet nem követelnek meg. A kernel nem kép, hanem egy lokális súlyozási minta: a bemeneti kép minden pozíciójában ugyanazt a $k_h k_w$ darab súlyt használjuk.

Valid cross-correlation. Rögzítsünk egy $H \times W$ bemenetet és egy $k_h \le H$, $k_w \le W$ kernelt. A K kernellel vett, `valid` tartományú cross-correlation a következő $Y^{xc} \in \mathbb{R}^{(H-k_h+1)\times(W-k_w+1)}$ kimenetet adja:

$$Y^{xc}_{i,j} = (K \star X)_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} K_{a,b}\, X_{i+a,\, j+b},$$

ahol $0 \le i \le H - k_h$ és $0 \le j \le W - k_w$. A kernel bal felső sarka mindig az input (i, j) rácspontjához igazodik; onnan az a, b indexek jobbra és lefelé futnak. A `valid` szó azt jelenti, hogy a kernel teljes egészében az input belsejében marad, tehát nem lép ki a kép határán, és nincs szükség peremfeltételezésre.

Matematikai konvolúció. Ugyanezen a bemeneten és kernelen a diszkrét, `valid` tartományú matematikai konvolúció definíciója az indexkonvenciónkban:

$$Y^{cv}_{i,j} = (K * X)_{i,j} = \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} K_{a,b}\, X_{i+k_h-1-a,\, j+k_w-1-b}.$$

A különbség a kernel indexsorrendjében van: a konvolúció a kernelt 180 fokkal elforgatva illeszti a bemenethez. A kernel 180 fokos elforgatását külön operátorral rögzítjük:

$$R(K)_{a,b} = K_{k_h-1-a,\, k_w-1-b}, \qquad 0 \le a < k_h,\ 0 \le b < k_w.$$

A 2×2-es kernelre például R mindkét tengely mentén megfordítja az indexek sorrendjét: $R\begin{pmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \end{pmatrix} = \begin{pmatrix} w_{11} & w_{10} \\ w_{01} & w_{00} \end{pmatrix}$.

Boundary, valid és padding. A fenti képletek a legegyszerűbb határkezelést testesítik meg: a kernel nem lép ki a képből, és minden olyan pozíciót kihagyunk, ahol a kernel túllógna. Ezt nevezzük `valid` módnak. A gyakorlati rétegek gyakran nullákkal töltik ki a képet a határon (zero padding): ha mind a négy oldalon p_h, p_w sornyi ill. oszlopnyi nullát adunk hozzá, akkor a kiterjesztett input $\tilde{X} \in \mathbb{R}^{(H+2p_h)\times(W+2p_w)}$, és a valid cross-correlation kimeneti mérete

$$H_{out} = H + 2p_h - k_h + 1, \qquad W_{out} = W + 2p_w - k_w + 1.$$

Az általános stride- és dilationfüggő képlet a 14.1. Stride, padding, dilation és outputméret egységben következik; itt mindenhol stride 1 és dilation 1 érvényes.

Bias. A konvolúciós rétegekben minden kimeneti csatornához egyetlen $\beta \in \mathbb{R}$ skalár tartozik, amelyet a cross-correlation eredményének minden térbeli elemére rögzített konstansként hozzáadunk:

$$Y^{\text{bias}}_{i,j} = (K \star X)_{i,j} + \beta.$$

A β tehát nem függ sem a bemenettől, sem a térbeli pozíciótól. A bias nélküli cross-correlation lineáris leképezés: $\alpha X + \gamma Z$ bemenethez $\alpha(K \star X) + \gamma(K \star Z)$ tartozik. A bias hozzáadása affin leképezést eredményez; a lineáris és affin megkülönböztetés ugyanaz, mint a 3.2. Mátrixszorzás és affin réteg affin rétegében: lineáris leképezés a nullát nullába viszi, a biasos affin leképezés általában nem. A β jelölést azért használjuk, hogy ne ütközzön a kernel b oszlopindexével.

Szemléltetés: mely bemeneti elem melyik kimenetet befolyásolja. Egydimenziós esetben $k = 3$, $s = 1$, $p = 0$, $H = 6$ bemenettel $H_{out} = 4$ kimenet van. A $y_i = \sum_{u=0}^{2} K[u]\, x_{i+u}$ alapján:

```
bemenet   x0  x1  x2  x3  x4  x5
          (az y-sorok az ablakokat mutatják)
y0 =  K0*x0 + K1*x1 + K2*x2          ablak: x0..x2
y1 =  K0*x1 + K1*x2 + K2*x3          ablak: x1..x3
y2 =  K0*x2 + K1*x3 + K2*x4          ablak: x2..x4
y3 =  K0*x3 + K1*x4 + K2*x5          ablak: x3..x5

hány kimenetet befolyásol az x_j?   x0:1  x1:2  x2:3  x3:3  x4:2  x5:1
```

A számlálás a visszaterjesztésnél a gradiensek átfedő összeadását magyarázza: a $\partial L/\partial x_j$ annyi tag összege, ahány kimenetet x_j befolyásol (a szélen 1, belül legfeljebb k). Kétdimenziós esetben ugyanez a számlálás soronként és oszloponként szorzódik, és az encoder–decoder rácsán a lefelé mintavételezett (stride) szakaszban a számlálás ritkább, mert ott egy kimeneti cella nagyobb bemeneti ablakot fed le, így egy bemeneti elem kevesebb kimenethez járul hozzá; a felfelé mintavételezésnél (transzponált konvolúció) pedig a kimeneti cellák átfedése adja az összeadandó tagokat.

Közös referenciaábra: forward ablak, scatter-add és crop ugyanazon a fixture-ön. Legyen $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$, a nemszimmetrikus $K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $s = 1$, $p = 1$ (nulla padding), így a paddingelt kép 5×5 és Y alakja 4×4. Három különálló műveletet kell megkülönböztetni:

1. *Forward ablak* (cross-correlation, nincs kernelmegfordítás): $Y[0,0] = \sum_{u,v} \tilde{X}[u,v]\, K[u,v]$ a paddingelt kép bal felső 2×2 ablakán, ahol csak $X[0,0] = 1$ nem padding: $Y[0,0] = 1 \cdot K[1,1] = 4$.
2. *Scatter-add* (backward): az $\partial L/\partial Y[0,0] = 1$ upstream gradiens a K-val szorozva *szóródik vissza* ugyanarra az ablakra a paddingelt gradiens-tenzoron: a $\tilde{X}$-gradiens bal felső 2×2 blokkja $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = K$ (nem megfordítva).
3. *Crop*: a padding sávját levágjuk; az eredeti X gradiense $\begin{bmatrix} 4 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$, mert a $K[0,0], K[0,1], K[1,0]$ hozzájárulások a levágott padding-cellákra estek.

```
import numpy as np
X = np.arange(1, 10, dtype=float).reshape(3, 3)
K = np.array([[1, 2], [3, 4.]])
Xp = np.pad(X, 1)                                   # 5x5
Y = np.array([[(Xp[i:i+2, j:j+2] * K).sum() for j in range(4)] for i in range(4)])
dY = np.zeros_like(Y); dY[0, 0] = 1.0
gXp = np.zeros_like(Xp)
for i in range(4):
    for j in range(4):
        gXp[i:i+2, j:j+2] += dY[i, j] * K           # scatter-add
gX = gXp[1:-1, 1:-1]                                # crop
assert Y[0, 0] == 4.0 and gXp[:2, :2].tolist() == K.tolist() and gX[0, 0] == 4.0
```

A három művelet összekeverése (= forward ablak, scatter-add és crop egyetlen lépésnek tekintése) a leggyakoribb tévképzet.

13.2.3 A két operátor kapcsolata: tétel és bizonyítás

A gyakorlati könyvtárak dokumentációjának olvasásához szükséges állítást explicit tételként mondjuk ki.

Tétel (valid cross-correlation és konvolúció kapcsolata). Legyen $X \in \mathbb{R}^{H \times W}$, $K \in \mathbb{R}^{k_h \times k_w}$, $k_h \le H$, $k_w \le W$, és legyen R a 180 fokos kernelforgatás a fenti definíció szerint. Ekkor minden érvényes (i, j) kimeneti indexre

$$(K * X)_{i,j} = \big(R(K) \star X\big)_{i,j}.$$

Bizonyítás. A matematikai konvolúció definícióját írjuk fel, majd hajtsunk végre indexcserét. Legyen $a' = k_h - 1 - a$ és $b' = k_w - 1 - b$. Az $(a, b) \mapsto (a', b')$ leképezés bijekció a $\{0, \dots, k_h - 1\} \times \{0, \dots, k_w - 1\}$ halmazon, ezért véges összegben az indexek átnevezése nem változtatja az összeg értékét:

$$\begin{aligned}(K * X)_{i,j} &= \sum_{a=0}^{k_h-1} \sum_{b=0}^{k_w-1} K_{a,b}\, X_{i+k_h-1-a,\, j+k_w-1-b} \\ &= \sum_{a'=0}^{k_h-1} \sum_{b'=0}^{k_w-1} K_{k_h-1-a',\, k_w-1-b'}\, X_{i+a',\, j+b'} \\ &= \sum_{a'=0}^{k_h-1} \sum_{b'=0}^{k_w-1} R(K)_{a',b'}\, X_{i+a',\, j+b'} = \big(R(K) \star X\big)_{i,j}.\end{aligned}$$

Az első egyenlőség az indexcserét, a második a R definícióját használja; a harmadik a cross-correlation definíciójából adódik. A kimeneti indexhalmaz mindkét oldalon azonos, mert a kernel mérete nem változott.

A tétel megfordított alakja is felírható: a cross-correlation a 180 fokkal elforgatott kernellel vett matematikai konvolúcióként, azaz $R(K) * X = K \star X$, ahol a bal oldalon az $R(K)$ kernellel vett konvolúció áll. Ennek bizonyítását kéri az F2 feladat, az $R(R(K)) = K$ azonossággal együtt. A tartalmi tanulság: *a matematikai konvolúció nem más, mint a 180 fokkal elforgatott kernellel vett cross-correlation.* A különbség tehát a kernel indexsorrendjében van, nem a bemenetben vagy a kimeneti rácsban.

Következmény. Ha $R(K) = K$, azaz a kernel 180 fokos forgásszimmetrikus, akkor a cross-correlation és a konvolúció ugyanazt a numerikus eredményt adja minden bemenetre. Az aszimmetrikus kernel a két operátort megkülönbözteti.

13.2.4 A PyTorch Conv2d definíció szerinti viselkedése

A [D10] hivatalos dokumentáció vonatkozó szakasza szerint a `torch.nn.Conv2d` az X bemenet és a K súly **cross-correlation**ját számítja, kernelmegfordítás nélkül. A dokumentáció jelölésében a súlyt és bemenetet összekötő $\star$ a `valid 2D cross-correlation` operátor; a bias az egyes kimeneti csatornákra adódik hozzá. Ez a megállapítás itt **szakirodalmi/definíciós állítás** a dokumentáció vonatkozó szakasza alapján, nem pedig empirikus általánosítás.

Az 1.2. Szakmailag megalapozott modellezési állítások állítástípus-tanulsága itt közvetlenül alkalmazandó: a `Conv2d` osztály neve nem definíció. Aki a matematikai konvolúciót szeretné PyTorch-ban, annak a kernelt 180 fokban el kell forgatnia, például `torch.flip(K, dims=[2,3])` művelettel. A tanult hálózatoknál a két konvenció közötti különbség nem „jó” vagy „rossz”, hanem azt jelenti, hogy ugyanaz a leképezés a megfordított kernel súlyozásával jelenik meg; előre rögzített, nem tanult szűrőknél azonban a különbségtétel numerikus hibához vezethet, ahogy az alábbi kézi példa mutatja.

13.2.5 Eredeti kézi kis példa

Legyen a bemenet a 13.1. Kép mint diszkrét többcsatornás jel stílusában egy 3×3-as, $[0, 1]$-beli intenzitású kép:

$$X = \begin{pmatrix} 0.2 & 0.5 & 0.1 \\ 0.8 & 0.3 & 0.6 \\ 0.4 & 0.9 & 0.7 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 \\ -2 & 3 \end{pmatrix}.$$

A $k_h = k_w = 2$ kernel aszimmetrikus, tehát a két operátor különbözni fog.

Cross-correlation. A $K \star X$ mátrix elemei a fenti definícióval:

- $(0,0)$: $1 \cdot 0{,}2 + 0 \cdot 0{,}5 + (-2) \cdot 0{,}8 + 3 \cdot 0{,}3 = -0{,}5$,
- $(0,1)$: $1 \cdot 0{,}5 + 0 \cdot 0{,}1 + (-2) \cdot 0{,}3 + 3 \cdot 0{,}6 = 1{,}7$,
- $(1,0)$: $1 \cdot 0{,}8 + 0 \cdot 0{,}3 + (-2) \cdot 0{,}4 + 3 \cdot 0{,}9 = 2{,}7$,
- $(1,1)$: $1 \cdot 0{,}3 + 0 \cdot 0{,}6 + (-2) \cdot 0{,}9 + 3 \cdot 0{,}7 = 0{,}6$.

Tehát

$$K \star X = \begin{pmatrix} -0{,}5 & 1{,}7 \\ 2{,}7 & 0{,}6 \end{pmatrix}.$$

Matematikai konvolúció. A 180 fokkal elforgatott kernel

$$R(K) = \begin{pmatrix} 3 & -2 \\ 0 & 1 \end{pmatrix}.$$

A tétel szerint $K * X = R(K) \star X$, és ennek elemei:

- $(0,0)$: $3 \cdot 0{,}2 + (-2) \cdot 0{,}5 + 0 \cdot 0{,}8 + 1 \cdot 0{,}3 = -0{,}1$,

- $(0,1)$: $3 \cdot 0{,}5 + (-2) \cdot 0{,}1 + 0 \cdot 0{,}3 + 1 \cdot 0{,}6 = 1{,}9$,
- $(1,0)$: $3 \cdot 0{,}8 + (-2) \cdot 0{,}3 + 0 \cdot 0{,}4 + 1 \cdot 0{,}9 = 2{,}7$,
- $(1,1)$: $3 \cdot 0{,}3 + (-2) \cdot 0{,}6 + 0 \cdot 0{,}9 + 1 \cdot 0{,}7 = 0{,}4$.

Tehát

$$K * X = \begin{pmatrix} -0{,}1 & 1{,}9 \\ 2{,}7 & 0{,}4 \end{pmatrix}.$$

A két kimenet közötti különbség az aszimmetrikus kernel megfordításából adódik; az $(1,0)$ pozíció értéke véletlenül egybeesik, de a mátrix mégsem azonos. Ha $\beta = 0{,}5$ biast adunk a cross-correlation réteghez, a kimenet

$$\begin{pmatrix} -0{,}5 & 1{,}7 \\ 2{,}7 & 0{,}6 \end{pmatrix} + 0{,}5 \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 0{,}0 & 2{,}2 \\ 3{,}2 & 1{,}1 \end{pmatrix}.$$

13.2.6 Labor: kézi és NumPy-ellenőrzés

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n050/check_convs.py` futtatható (`python lab/n050/check_convs.py`); külső adatfájl nem kell.

A labor a `lab/n050/` munkakönyvtárban fut; a kódot a rögzített környezetben lefuttattuk (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; nincs véletlen, két futás kimenete bitazonos), ezért az alábbi kimenet valós futási eredmény. A hallgató feladata a futtatás és a toleranciavizsgálat megismétlése.

```
# lab/n050/check_convs.py
import numpy as np

X = np.array([[0.2, 0.5, 0.1],
              [0.8, 0.3, 0.6],
              [0.4, 0.9, 0.7]], dtype=np.float32)
K = np.array([[1.0, 0.0],
              [-2.0, 3.0]], dtype=np.float32)

def corr2d_valid(X, K):
    H, W = X.shape
    kh, kw = K.shape
    out = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float32)
    for i in range(H - kh + 1):
        for j in range(W - kw + 1):
            for a in range(kh):
                for b in range(kw):
                    out[i, j] += K[a, b] * X[i + a, j + b]
    return out

def conv2d_valid(X, K):
    return corr2d_valid(X, np.rot90(K, 2))   # 180 fokos forgatás

Yc = corr2d_valid(X, K)
Yf = conv2d_valid(X, K)
print("cross-correlation:\n", Yc)
print("convolution:\n", Yf)
# mért (float32): cross-correlation [[-0.49999994, 1.7],
#                                    [2.6999998, 0.5999999]]
```

```
# mért (float32): convolution       [[-0.09999996, 1.9],
#                                    [2.7, 0.39999998]]
assert np.allclose(Yc, [[-0.5, 1.7], [2.7, 0.6]], atol=1e-6)
assert np.allclose(Yf, [[-0.1, 1.9], [2.7, 0.4]], atol=1e-6)
```

Futtatási parancs: `python lab/n050/check_convs.py`. A `dtype` mindenütt `float32`, az eszköz CPU; random seedre nincs szükség, mert a függvény determinisztikus, véletlenszám-generátort nem hív.

A PyTorch-oldal ugyanezt az NCHW-indexeléssel ellenőrzi; a 13.1. Kép mint diszkrét többcsatornás jel NCHW-konvenciójának megfelelően az egycsatornás képet `reshape(1, 1, H, W)` hívással $(1, 1, H, W)$ tenzorrá alakítjuk:

```
import torch
import torch.nn.functional as F

Xt = torch.tensor(X.reshape(1, 1, 3, 3), dtype=torch.float32)
Kt = torch.tensor(K.reshape(1, 1, 2, 2), dtype=torch.float32)

Yc_torch = F.conv2d(Xt, Kt)                      # NCHW, stride=1, padding=0
Kt_flipped = torch.flip(Kt, dims=[2, 3])
Yf_torch = F.conv2d(Xt, Kt_flipped)              # matematikai konvolúció

print("torch Yc:", Yc_torch.squeeze())
print("torch Yf:", Yf_torch.squeeze())
assert np.allclose(Yc, Yc_torch.squeeze().numpy(), atol=1e-6)
assert np.allclose(Yf, Yf_torch.squeeze().numpy(), atol=1e-6)
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; nincs véletlen, két futás kimenete bitazonos):

```
cross-correlation:
 [[-0.49999994  1.7       ]
 [ 2.6999998   0.5999999 ]]
convolution:
 [[-0.09999996  1.9       ]
 [ 2.7         0.39999998]]
torch Yc: tensor([[-0.5000,  1.7000],
        [ 2.7000,  0.6000]])
torch Yf: tensor([[-0.1000,  1.9000],
        [ 2.7000,  0.4000]])
```

A két `assert` a NumPy-eredményt a PyTorch kimenetével veti össze; a `squeeze()` a PyTorch $(1, 1, 2, 2)$ alakját a NumPy $(2, 2)$ alakjához igazítja, az `atol=1e-6` pedig a float32-es összegzés esetleges utolsó bitjeit engedi el. A `torch.flip` a kernel 180 fokos elforgatását végzi; ez a PyTorch-alakú kernel esetén a k_h, k_w dimenziók megfordítását jelenti.

13.2.7 Feladatok

F1 feladat — fogalmi, nehézség 1. Különböztesse meg a `valid` cross-correlationt és a matematikai konvolúciót. Adjon olyan kernelcsaládot, amelyre a két operátor minden bemenetre azonos, és egy konkrét ellenpéldát, amelyre nem. Beadandó: definíciók, a szimmetria feltétele, egy 2×2-es aszimmetrikus kernel és egy rövid indoklás.

F2 feladat — levezetési/bizonyítási, nehézség 2. Bizonyítsa be, hogy $R(R(K)) = K$, majd ezt és a fenti tételt használva mutassa meg, hogy $F(R(K), X) = K \star X$, ahol $F(S, X) = S * X$ a matematikai konvolúció operátora. Beadandó: lépésenkénti levezetés, az indexhalmaz bijektivitásának indoklása és a végeredmény alakellenőrzése.

F3 feladat — implementációs, nehézség 1. Implementálja a `corr2d_valid` függvényt, és számítsa ki a kimenetet a következő bemenetre és kernelre:

$$X = \begin{pmatrix} 1 & 4 & 2 \\ 0 & 3 & 1 \\ 2 & 5 & 3 \\ 1 & 1 & 2 \end{pmatrix}, \qquad K = \begin{pmatrix} 2 & -1 \\ 0 & 1 \end{pmatrix}.$$

Beadandó: kód, a kimeneti mátrix, a kimeneti alak és a belső összegzések indoklása.

F4 feladat — hibakeresési, nehézség 2. Az alábbi függvény a `conv2d_valid_bad` nevet viseli, és a szándéka szerint matematikai konvolúciót kellene számolnia, de hibás (a labor helyes `conv2d_valid` függvénye az összehasonlítási alap):

```
def conv2d_valid_bad(X, K):
    H, W = X.shape
    kh, kw = K.shape
    out = np.zeros((H - kh + 1, W - kw + 1))
    for i in range(H - kh + 1):
        for j in range(W - kw + 1):
            out[i, j] = np.sum(K * X[i:i+kh, j:j+kw])
    return out
```

Az elvárt viselkedés a matematikai konvolúció. Lokalizálja a hibát, indokolja a következményét a fenti X, K példán, javítsa a kódot, és ellenőrizze a javítást a várt numerikus kimenettel. Beadandó: hiba oka, következmény, javított kód, ellenőrző futás eredménye.

F5 feladat — kísérleti/transzfer, nehézség 3. Tervezzen rögzített szűrős kísérletet a GlyphLab (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) `data/images.npy` képeire (360 kép, $(360, 1, 24, 24)$ float32, $[0, 1]$; osztályok: `vertical`, `horizontal`, `diagonal`): veszik-e észre a vertical és horizontal osztályok közötti különbséget a

$$K_v = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

függőleges élkereső kernel és a transzponáltja,

$$K_h = K_v^\top = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

vízszintes élkereső kernel különbözőképpen? Miért nem alkalmas ugyanerre a K_v 180 fokos elforgatottja? Adjon protokollt: kérdés, hipotézis, adat, megfigyelési egység, kiszámítandó metrika, baseline, döntési szabály és érvényességi korlát. Nem kell előre garantált győztest adni. Beadandó: protokoll szövege, a kimeneti méret számítása 24×24-es inputra, és annak indoklása, miért nem használható a `test` split a szűrő kiválasztására. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

13.2.8 Megoldások és indoklás

F1 megoldás. A cross-correlation definíciója $Y^{xc}_{i,j} = \sum_{a,b} K_{a,b} X_{i+a,j+b}$, a konvolúcióé $Y^{cv}_{i,j} = \sum_{a,b} K_{a,b} X_{i+k_h-1-a,j+k_w-1-b}$. A fenti tétel szerint $K * X = R(K) \star X$; ha $R(K) = K$, akkor a két kimenet azonos minden X-re, `valid` tartományon. A 180 fokos forgásszimmetria feltétele $K_{a,b} = K_{k_h-1-a,k_w-1-b}$ minden (a,b)-re; ilyen például

$$K_{\text{sym}} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

A K_{sym} jelölés arra utal, hogy ez a példa kielégíti a $K_{a,b} = K_{k_h-1-a,k_w-1-b}$ feltételt, azaz $R(K_{\text{sym}}) = K_{\text{sym}}$: a mátrix a középpontjára szimmetrikus, ezért a 180 fokos forgatás nem változtatja meg.

Ellenpélda a fenti 2×2-es $K = \begin{pmatrix} 1 & 0 \\ -2 & 3 \end{pmatrix}$, mert $K_{0,0} = 1 \neq 3 = K_{1,1} = R(K)_{0,0}$, tehát a két operátor kimenete eltér.

F2 megoldás. A R definícióját kétszer alkalmazva:

$$R(R(K))_{a,b} = R(K)_{k_h-1-a,k_w-1-b} = K_{k_h-1-(k_h-1-a),\,k_w-1-(k_w-1-b)} = K_{a,b},$$

tehát $R(R(K)) = K$. A tétel szerint minden S kernelre $S * X = R(S) \star X$; válasszuk az $S = R(K)$ kernelt. Ekkor

$$F(R(K), X) = R(K) * X = R(R(K)) \star X = K \star X.$$

A kimenet mindkét oldalon $(H - k_h + 1) \times (W - k_w + 1)$ alakú, mert a kernel mérete R alatt változatlan. A bizonyítás az indexcserés levezetés közvetlen alkalmazása; a tétel feltételei ($k_h \leq H$, $k_w \leq W$, `valid` tartomány) azonosak, tehát a lépés megengedett.

F3 megoldás. A megoldás a labor függvényét használja, de a feladat saját bemenetére önálló szkriptben:

```
import numpy as np

def corr2d_valid(X, K):
    H, W = X.shape
    kh, kw = K.shape
    out = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float64)
    for i in range(H - kh + 1):
        for j in range(W - kw + 1):
            out[i, j] = np.sum(K * X[i:i+kh, j:j+kw])
    return out

X = np.array([[1, 4, 2],
              [0, 3, 1],
              [2, 5, 3],
              [1, 1, 2]], dtype=np.float64)
K = np.array([[2, -1],
              [0,  1]], dtype=np.float64)
Y = corr2d_valid(X, K)
assert Y.shape == (3, 2)
assert np.allclose(Y, np.array([[1., 7.],
                                [2., 8.],
```

```
                                  [0., 9.]]))
print(Y)
```

A kimenet $H = 4$, $W = 3$, $k_h = k_w = 2$ esetén $(3, 2)$ alakú:

$$Y^{xc} = \begin{pmatrix} 1 & 7 \\ 2 & 8 \\ 0 & 9 \end{pmatrix}.$$

Ellenőrzés: a $(2, 0)$ elem $2X_{2,0} + (-1)X_{2,1} + 0X_{3,0} + 1X_{3,1} = 2 \cdot 2 - 5 + 0 + 1 = 0$; a $(2, 1)$ elem $2 \cdot 5 - 3 + 0 + 2 = 9$. A Y elemei az összegzési definícióból következnek; a shape a valid képletből $H_{out} = 4 - 2 + 1 = 3$, $W_{out} = 3 - 2 + 1 = 2$.

F4 megoldás. A hiba oka: a függvény a kernel **megfordítása nélkül** képezi a K és az inputablak elemenkénti szorzatösszegét, tehát valójában cross-correlationt számol, miközben a neve matematikai konvolúciót ígér. A fenti X és K bemeneten a hibás függvény a cross-correlation eredményét adja vissza a konvolúció várt értéke helyett, így az elvárt kimenettől való eltérés a kernel aszimmetriájából származik. A javítás a kernel 180 fokos forgatása a belső ablakozás előtt:

```
import numpy as np

def conv2d_valid_correct(X, K):
    H, W = X.shape
    kh, kw = K.shape
    K_rot = np.rot90(K, 2)
    out = np.zeros((H - kh + 1, W - kw + 1))
    for i in range(H - kh + 1):
        for j in range(W - kw + 1):
            out[i, j] = np.sum(K_rot * X[i:i+kh, j:j+kw])
    return out
```

A várt eredmény a 3×3-as példán

$$K * X = \begin{pmatrix} -0{,}1 & 1{,}9 \\ 2{,}7 & 0{,}4 \end{pmatrix},$$

az ellenőrző `np.allclose(conv2d_valid_correct(X, K), expected, atol=1e-6)` igaz.

F5 megoldás. Protokoll:

1. **Kérdés:** elválasztja-e a rögzített K_v függőleges és $K_h = K_v^\top$ vízszintes élkereső kernel a `vertical` és `horizontal` osztályokat a GlyphLab képein?
2. **Hipotézis:** K_v átlagos kimeneti energiája nagyobb a `vertical`, K_h-é a `horizontal` osztályon; a `diagonal` osztályon a különbség iránya nem garantált.
3. **Adat:** `data/images.npy` NCHW float32, alakja $(360, 1, 24, 24)$, értékkészlete $[0, 1]$; `data/labels.npy` int64; a képtenzor indexelését a 13.1. Kép mint diszkrét többcsatornás jel rögzítette.
4. **Megfigyelési egység:** kép; a csoportosítás miatt a scene_id figyelembe veendő, de ez a leíró elemzésben nem okoz split-átszivárgást, ha csak a `train` részt használjuk.
5. **Split:** kizárólag `train` és szükség esetén `val` a leíró megfigyelésre; a `test` a későbbi modellválasztás után marad zárva.
6. **Metrika:** minden képre $E(X) = \sum_{i,j} \left((K_v \star X)_{i,j}\right)^2$ (illetve K_h-val), ahol a 24×24-es inputra a valid kimenet 22×22; osztályonkénti átlag és 95%-os, scene-szintű bootstrap intervallum az 5.3.–5.4. gondolatmenetével.

7. **Változó tényező:** kernel mint K_v vagy $K_h = K_v^\top$.
8. **Értékelés:** scene-enként képezzük az $E_{K_v} - E_{K_h}$ különbséget, és osztályonként scene-szintű bootstrap 95%-os intervallumot számítunk rá. **Döntési szabály:** ha a `vertical` osztályon az intervallum teljesen pozitív, a `horizontal` osztályon teljesen negatív, a hipotézist alátámasztottnak tekintjük; ha bármelyik intervallum tartalmazza a nullát, az adat nem dönti el a kérdést. A rögzített szűrő nem tanul, ezért a `test` splitet nem használjuk kiválasztásra.

A kimeneti méret: $H_{out} = 24 - 3 + 1 = 22$, tehát a valid kimenet 22×22. A K_v 180 fokos elforgatottja azért nem alkalmas a kérdés eldöntésére, mert $R(K_v) = -K_v$, a négyzetre emelés pedig az előjelet eltünteti: $(R(K_v) \star X)^2 = (K_v \star X)^2$ minden X-re, tehát a két szűrő négyzetes energiája azonos. Ezért a vízszintes pár a transzponált K_h: $R(K_h) = -K_h$, de $K_h \neq \pm K_v$, tehát a transzponált valóban más szűrő, mint a K_v (és annak 180 fokos elforgatottja). A `test`-értékelés kizárásának oka: ha a szűrőt vagy a küszöböt a testen választanánk, az optimista torzítást vinne a későbbi modellösszehasonlításba; a független teszt torzítatlansága pontosan akkor szűnik meg, ha a modellt a teszt eredménye alapján választjuk ki, ahogy a 8.4. II. ellenőrző pont: korrekt tanulási állítás is kimondja.

13.2.9 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: „a `torch.nn.Conv2d` konvolúciót számol, mert a neve ezt mondja.” Az ellenpélda a fenti X, K pár: a `torch.nn.functional.conv2d` a $K \star X$ mátrixot adja, a matematikai konvolúció viszont $K * X$-et, és a két mátrix különbözik. A D10 dokumentáció ezt a `valid 2D cross-correlation` kifejezéssel expliciten rögzíti. A tévedés következménye: előre definiált szűrőknél (pl. élkereső kernel) a rossz konvenció rossz numerikus eredményt ad; tanult kerneleknél a numerikus kimenet változatlan marad, de a súlyok operátorkonvencióját dokumentálni kell, mert a megtanult súly a megfordított párjával ekvivalens. A 12.2. Checkpoint és folytathatóság tanulsága itt is érvényes: a checkpointban tárolt `state_dict['weight']` önmagában nem mondja meg, melyik operátorkonvencióhoz tartozik; a konvenciót a kódverzióval és a modellkártyával együtt kell nyilvántartani.

13.2.10 Önellenőrzés és visszatekintés

- Igaz-e, hogy a 180 fokos kernelforgatás megváltoztatja a valid kimeneti méretet? Nem; a kernel mérete változatlan, csak a súlyok sorrendje.
- Igaz-e, hogy a biasos konvolúció lineáris leképezés? Nem; a lineáris rész a cross-correlation, a bias affinná teszi.
- Visszatekintés 9.2. MLP és nemlinearitás: az MLP affin rétege teljes kapcsolású, tehát minden bemeneti neuron minden kimeneti neuronhoz hozzájárul; a kernel ezzel szemben lokális és súlymegosztott lineáris leképezést ad.
- Visszatekintés 1.2. Szakmailag megalapozott modellezési állítások: a `Conv2d` mint cross-correlation a dokumentáció vonatkozó szakaszából származó állítás, a definíció és a mérési állítás nem felcserélhető.

13.2.11 Rövid összefoglaló

A cross-correlation a kernel bal felső sarkát az input aktuális pozíciójához illesztve, kernelmegfordítás nélkül képez szorzatösszeget; a matematikai konvolúció ugyanezt 180 fokkal elforgatott kernellel végzi. A valid tartományon a két operátor kimeneti mérete azonos, és a kapcsolatuk $K * X = R(K) \star X$, ahol R a kernel 180 fokos forgatása. A bizonyítás egy indexcserés bijekcióra

épül. A PyTorch `Conv2d` dokumentációja cross-correlationt specifikál; ez előre definiált szűrőknél numerikus különbséget, tanult kerneleknél ekvivalens, de dokumentálandó konvenciót jelent. A labor a 3×3-as példát NumPy- és PyTorch-oldalon egyaránt ellenőrzi.

13.2.12 Továbblépés

A 13.3. Lokális szűrők és tanult reprezentáció ezt a súlyozási műveletet tanult lokális reprezentációként értelmezi: a kernelek mint él-, textúra- és iránydetektorok jelennek meg, és a bias nélküli operátor linearitása a súlymegosztás előnyeinek levezetéséhez ad alapot.

13.2.13 Források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

13.3 Lokális szűrők és tanult reprezentáció

13.3.1 Cél, előismeret és induló kérdés

Ez az egység a 13.2. Konvolúció és cross-correlation alfejezetben definiált cross-correlation műveletet lokális, tanulható reprezentációként értelmezi. A rögzített él- és simító kernelek olyan kitüntetett pontok a tanulható kernelterében, amelyek a tanulás nélküli, kézzel megadott súlyozás szerepét játsszák. Előismeret: a cross-correlation és a matematikai konvolúció megkülönböztetése, a valid tartomány és a kernel fogalma a 13.2. Konvolúció és cross-correlation alfejezetből. A labor nem osztályozási kísérlet: ugyanazon bemeneteken vetjük össze két rögzített szűrő válaszát, predikciós állítás nélkül.

Induló kérdés: egy 4×4-es lépcsős mintán egy 2×2-es kernel 3×3-as választ ad. Hány megosztott paraméterrel áll elő ez a 9 kimeneti érték, és mit mondhatunk a kernel súlyösszegéből konstans bemenet esetén?

13.3.2 Lokális recepciós mező, súlymegosztás és feature map

A [S01] kurzusjegyzet terminológiájában a konvolúciós réteg három szervező gondolata a lokális kapcsolat, a súlymegosztás és az egy kernelhez tartozó feature map fogalma. Mindháromhoz pontos definíciót adunk a mi jelölésünkkel.

Legyen $X \in \mathbb{R}^{C_{in} \times H \times W}$ egyetlen kép tenzora (köteg nélkül), $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$ a kernel, és tekintsük a valid, stride 1, nulla padding, dilation 1 esetet. Ekkor a kimenet alakja

$$Y \in \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}, \qquad H_{out} = H - k_h + 1, \quad W_{out} = W - k_w + 1,$$

és a rögzített cross-correlation definíciója

$$Y[o, i, j] = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[o, c, u, v]\, X[c, i+u, j+v]. \tag{1}$$

A PyTorch `Conv2d` dokumentációja ezt a kernelmegfordítás nélküli műveletet specifikálja, miközben convolutionnek nevezi [D10]; mi a matematikai és programozói konvenciót továbbra is megkülönböztetjük.

Definíció (lokális recepciós mező). Az (1)-beli $Y[o, i, j]$ kimeneti érték *lokális recepciós mezője* azon bemeneti indexhármasok halmaza, amelyek a kimeneti értékhez ténylegesen hozzájárulnak:

$$\mathcal{R}_{i,j} = \{(c, i+u, j+v) : 0 \le c < C_{in},\ 0 \le u < k_h,\ 0 \le v < k_w\}.$$

A halmaz mérete $C_{in} k_h k_w$, függetlenül a teljes kép H, W méretétől. Például egy 24×24-es bemenet, egy csatorna és 3×3-as kernel esetén minden kimeneti pixel mindössze 9 bemeneti pixelt lát: a kapcsolat lokális, nem teljesen összekötött.

Definíció (súlymegosztás). A K kernel elemei nem függnek az (i, j) kimeneti pozíciótól. Ugyanaz a $K[o, c, u, v]$ súly szorzódik minden olyan pozícióban, ahol a kernel ablak ugyanazt a relatív eltolást vizsgálja. Emiatt a paraméterek száma

$$C_{out} C_{in} k_h k_w + C_{out},$$

ahol az utolsó tag a kimeneti csatornánkénti bias. Ez nem függ H-tól és W-től, szemben egy olyan affin réteggel, amely a bemenet minden pozíciójához külön súlyt rendelne.

Definíció (feature map). Egy rögzített o kimeneti csatornához tartozó $Y[o, \cdot, \cdot] \in \mathbb{R}^{H_{out} \times W_{out}}$ térbeli tömb az o-adik *feature map*. A kernel minden térbeli pozícióra ugyanazt a lokális súlyozást alkalmazza, ezért a feature map a kernel és a kép lokális hasonlóságának térbeli eloszlását írja le.

A három fogalom kapcsolata: a lokális recepciós mező azt mondja meg, *mely* inputokból számolunk, a súlymegosztás azt, hogy *ugyanazzal* a súlymintával minden pozícióban, a feature map pedig az így kapott térbeli választ.

13.3.3 Él- és simító szűrő mint rögzített kernel

A *simító* (smoothing) kernel olyan súlyozott helyi átlag, amely a szomszédos intenzitások különbségét csökkenti. A fejezet kézi példájában a 2×2-es box kernel

$$K_{\text{smooth}} = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Súlyai nemnegatívak, összegük 1. A *élkereső* (edge) kernel ezzel szemben pozitív és negatív súlyokat is tartalmaz, összege 0; a példánkban a vízszintes átmenetet mérő kernel

$$K_{\text{edge}} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}.$$

A súlyösszeg szerepét rögzíti az alábbi lemma.

Lemma. Ha $X \in \mathbb{R}^{1 \times H \times W}$ minden pixele azonos $c \in \mathbb{R}$ értékű, akkor bármely $K \in \mathbb{R}^{1 \times 1 \times k_h \times k_w}$ kernelre

$$Y[0, i, j] = c \sum_{u=0}^{k_h - 1} \sum_{v=0}^{k_w - 1} K[0, 0, u, v].$$

Bizonyítás. Az (1) képletbe $X[0, i+u, j+v] = c$-t helyettesítve

$$Y[0,i,j] = \sum_{u,v} K[0,0,u,v]\, c = c \sum_{u,v} K[0,0,u,v],$$

ami a kívánt alak. A többcsatornás változatban, ha $X \in \mathbb{R}^{C_{in} \times H \times W}$ és minden c-re $X[c,:,:] \equiv c_c$, akkor

$$Y[o,i,j] = \sum_{c=0}^{C_{in}-1} c_c \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[o,c,u,v],$$

ami az (1) képletből és a véges összeg csatornánkénti csoportosításából következik.

A lemma következménye, hogy K_{smooth} konstans képen a bemeneti értéket adja vissza, mert súlyösszege 1; K_{edge} konstans képen azonosan 0 választ ad, mert súlyösszege 0. Az „él” így operatívan definiálható: a kernel a konstans tartományt kioltja, és csak a sorok közötti (vertikális irányú) intenzitásváltozásra ad nemnulla választ; a vízszintes irányú változást ez a konkrét kernel nem érzékeli, mert a K_{edge} sorai azonosak egymással.

Az induló kérdés válasza. A 2×2-es kernel mindössze $2 \cdot 2 = 4$ megosztott súlyparaméterrel állítja elő mind a 9 kimeneti értéket (biasszal együtt 5 paraméter). Konstans c bemeneten a lemma szerint minden kimenet $c \cdot \sum K$: a K_{smooth} (súlyösszeg 1) a bemeneti értéket adja vissza, a K_{edge} (súlyösszeg 0) nullát.

A rögzített szűrő a tanulható kernel *előképe* a következő értelemben: ugyanaz a cross-correlation paramétertér egy pontja. Ha egy tanulás előtti háló kernelét K_{edge}-re állítjuk, akkor a háló a tanulás nulla lépése mellett már egy érdemi, értelmezhető leképezést valósít meg. A tanulás ezután elmozdíthatja a kerneleket, és a megtanult súlyok már nem kötelesek megőrizni sem a nulla súlyösszeget, sem a vizuálisan „élszerű” mintázatot. A rögzített kernel tehát nem a tanulás célja, hanem kontrollált kiindulópont a tanulható leképezések terében.

13.3.4 A bias nélküli cross-correlation linearitása

A tanulható kernel előképének matematikai tulajdonságához először a rögzített kernelű operátor szerkezetét kell tisztázni.

Definíció (rögzített cross-correlation operátor). Rögzített K mellett legyen

$$F_K : \mathbb{R}^{C_{in} \times H \times W} \to \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}, \qquad F_K(X) = Y,$$

ahol Y-t az (1) képlet definiálja. A biasos változat minden kimeneti csatornához egy $b_o \in \mathbb{R}$ skalárt ad:

$$F_{K,b}(X)[o,i,j] = F_K(X)[o,i,j] + b_o.$$

A bias térbelileg az egész feature mapon osztozik.

Tétel (linearitás). A bias nélküli F_K lineáris leképezés: minden $X_1, X_2 \in \mathbb{R}^{C_{in} \times H \times W}$ bemenetre és minden $\alpha, \beta \in \mathbb{R}$ skalárra

$$F_K(\alpha X_1 + \beta X_2) = \alpha F_K(X_1) + \beta F_K(X_2).$$

Bizonyítás. Rögzített o, i, j kimeneti koordinátára a definícióból:

$$\begin{aligned} F_K(\alpha X_1 + \beta X_2)[o,i,j] &= \sum_{c,u,v} K[o,c,u,v]\,(\alpha X_1 + \beta X_2)[c, i+u, j+v] \\ &= \sum_{c,u,v} K[o,c,u,v]\Big(\alpha X_1[c,i+u,j+v] + \beta X_2[c,i+u,j+v]\Big) \\ &= \alpha \sum_{c,u,v} K[o,c,u,v] X_1[c,i+u,j+v] + \beta \sum_{c,u,v} K[o,c,u,v] X_2[c,i+u,j+v] \\ &= \alpha F_K(X_1)[o,i,j] + \beta F_K(X_2)[o,i,j]. \end{aligned}$$

A második egyenlőség az $(\alpha X_1 + \beta X_2)[c, i+u, j+v] = \alpha X_1[c, i+u, j+v] + \beta X_2[c, i+u, j+v]$ pontonkénti skalár-azonosságot használja, a harmadik a véges összeg linearitását (a közös tényezők kiemelését), a negyedik pedig F_K definíciója. Mivel a koordinátánkénti egyenlőség minden kimeneti pozícióra teljesül, az operátorok egyenlők.

A tétel feltételei közül fontos, hogy K *rögzített*: ha K maga is tanulható paraméter, akkor a leképezés a (K, X) párban bilineáris, de X-ben továbbra is lineáris. A tétel nem állítja, hogy a tanulási folyamat lineáris; az optimalizáló lépés a kernel súlyainak nemlineáris frissítését jelenti.

Biasos ellenpélda. A biasos $F_{K,b}$ nem lineáris, ha valamely $b_o \neq 0$. Legyen $b_o = 1$ minden kimeneti csatornára. Ekkor $F_{K,b}(0)[o,i,j] = 1$, míg egy lineáris leképezés a zérus vektort kötelezően zérusba viszi, hiszen $L(0) = L(0 \cdot 0) = 0 \cdot L(0) = 0$. Mivel $F_{K,b}(0) \neq 0$, a leképezés nem lineáris. Az additivitás közvetlenül is sérül:

$$F_{K,b}(0+0) = b, \qquad F_{K,b}(0) + F_{K,b}(0) = 2b,$$

és $2b \neq b$, ha $b \neq 0$. A biasos operátor tehát *affin* leképezés: egy lineáris leképezés és egy konstans eltolás összege. Programozói szempontból a PyTorch `Conv2d` alapértelmezésben biasos, ezért önmagában nem lineáris réteg, míg `bias=False` mellett a fenti tétel értelmében az.

13.3.5 Kézi példa: lépcsős minta és két filter

Legyen a bemenet

$$X_1 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}.$$

A K_{edge} kernel valid válasza a 3×3-as

$$Y_{\text{edge}} = \begin{pmatrix} 0 & 0 & 0 \\ -2 & -2 & -2 \\ 0 & 0 & 0 \end{pmatrix},$$

mert például az $(i,j) = (1,0)$ pozíció ablakában

$$Y[1,0] = X_1[1,0] + X_1[1,1] - X_1[2,0] - X_1[2,1] = 0 + 0 - 1 - 1 = -2,$$

és a konstans felső, illetve alsó sorokban a pozitív és negatív tagok kioltják egymást. A vízszintes él tehát az átmenet sorában ad erős negatív választ; előjele a sötétből világosba való átmenet irányát rögzíti.

A K_{smooth} kernel ugyanezen a bemeneten

$$Y_{\text{smooth}} = \begin{pmatrix} 0 & 0 & 0 \\ 0{,}5 & 0{,}5 & 0{,}5 \\ 1 & 1 & 1 \end{pmatrix},$$

mert az átmenet sorában a 2×2-es ablak két 0 és két 1 értéket tartalmaz, így átlaga 0,5. A két kernel ugyanazt a lokális ablakot vizsgálja, mégis más invariáns tulajdonságot emel ki: a simító kernel a lokális szintet, az él kernel a lokális különbséget.

Biasos változatként $b = 1$ mellett

$$Y_{\text{edge},b} = Y_{\text{edge}} + 1 = \begin{pmatrix} 1 & 1 & 1 \\ -1 & -1 & -1 \\ 1 & 1 & 1 \end{pmatrix}.$$

A bias tehát minden kimeneti elemhez hozzáad 1-et; a nulla bemenetre a biasos operátor $b = 1$ értéket adna, ami a fenti ellenpéldának megfelelően megtöri a linearitást (a konkrét X_1 bemeneten a fenti eltolt mátrix adódik).

13.3.6 Minimális ellenőrző kód

A munkakönyvtár a `lab/`. Az alábbi NumPy-függvény a valid cross-correlationt számolja, és ellenőrzi a kézi példát.

```
import numpy as np

def valid_cross_corr2d(x, k):
    # x: (H, W), k: (kh, kw), float32; valid cross-correlation
    H, W = x.shape
    kh, kw = k.shape
    y = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float32)
    for i in range(y.shape[0]):
        for j in range(y.shape[1]):
            y[i, j] = np.sum(x[i:i+kh, j:j+kw] * k,
                             dtype=np.float32)
    return y

x_step = np.array([[0,0,0,0],
                   [0,0,0,0],
                   [1,1,1,1],
                   [1,1,1,1]], dtype=np.float32)

k_edge = np.array([[1,1],[-1,-1]], dtype=np.float32)
k_smooth = (1.0/4.0) * np.ones((2,2), dtype=np.float32)

y_edge = valid_cross_corr2d(x_step, k_edge)
y_smooth = valid_cross_corr2d(x_step, k_smooth)

assert y_edge.shape == (3,3)
assert y_smooth.shape == (3,3)
assert np.allclose(y_edge, [[0,0,0],[-2,-2,-2],[0,0,0]],
                   atol=1e-6)
```

```
assert np.allclose(y_smooth, [[0,0,0],[0.5,0.5,0.5],[1,1,1]],
                   atol=1e-6)
print("shape check OK")
```

A kód `dtype=np.float32` bemenettel és akkumulációval számol; a 0,5 pontosan reprezentálható, a `-2` is, így az `atol=1e-6` tolerancia elegendő. A `np.sum` soron belül `dtype=np.float32` megadása azt biztosítja, hogy a közbülső összeg sem lép át váratlanul `float64`-be, bár itt a kis értékek miatt a numerikus különbség nem lenne érdemi.

13.3.7 Labor: két rögzített szűrő válasza a GlyphLab mintákon

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; a `n051_filter_response.py` a `lab/` könyvtárból futtatható (`python n051_filter_response.py`).

A labor célja nem predikció, hanem ugyanazon bemenetek két leképezésének összehasonlítása. Input: `data/images.npy` alakja `(360,1,24,24)`, `float32`, a GlyphLab szerződés szerint (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték): a szerződés 180 jelenetből jelenetenként két zajos nézetet, összesen 360 képet rögzít, a split pedig scene-szintű, azaz egy jelenet mindkét nézete ugyanabba a train/val/test részbe kerül. A kernel mérete 3×3; a valid kimenet 22×22. A két kernel:

$$K^{(3)}_{\text{smooth}} = \frac{1}{9}\mathbb{1}_{3\times 3}, \qquad K^{(3)}_{\text{edge}} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}.$$

A mérés a válaszok abszolútérték-összege képenként; ez deskriptív mennyiség, nem osztályozási pontszám. A szkriptet a `lab/n051_filter_response.py` fájlba mentve futtatjuk:

```
python n051_filter_response.py \
  --images data/images.npy \
  --labels data/labels.npy \
  --metadata data/metadata.csv
```

A teljes szkript:

```
"""Két rögzített szűrő deskriptív válasza GlyphLab mintákon.

A script nem végez predikciót; a válaszabszolútösszeg csak
ugyanazon bemenet két leképezésének összevetésére szolgál.
Nincs véletlen lépés: a kiválasztás a metadata train splitjéből
determinisztikus; a dtype float32, az eszköz CPU.
"""
import argparse
import csv
import numpy as np

DTYPE = np.float32

def valid_cross_corr2d(x, k):
    H, W = x.shape
    kh, kw = k.shape
    y = np.zeros((H - kh + 1, W - kw + 1), dtype=DTYPE)
    for i in range(y.shape[0]):
        for j in range(y.shape[1]):
```

```
            y[i, j] = np.sum(x[i:i+kh, j:j+kw] * k,
                             dtype=DTYPE)
    return y

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--images", default="data/images.npy")
    p.add_argument("--labels", default="data/labels.npy")
    p.add_argument("--metadata", default="data/metadata.csv")
    args = p.parse_args()

    images = np.load(args.images).astype(DTYPE)  # (360,1,24,24)
    labels = np.load(args.labels)                # (360,) int64
    with open(args.metadata, newline="", encoding="utf-8") as f:
        meta = list(csv.DictReader(f))
    assert list(meta[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(meta[0])
    assert len(meta) == len(images) == len(labels) == 360, (len(meta), images.shape)
    assert [int(r["index"]) for r in meta] == list(range(len(meta)))

    k_smooth = (1.0/9.0) * np.ones((3,3), dtype=DTYPE)
    k_edge = np.array([[1,1,1],[0,0,0],[-1,-1,-1]],
                      dtype=DTYPE)

    # osztályonként a train split első három képét választjuk;
    # a feladat szempontjából elegendő, és nem képez predikciót
    train_idx = [int(r["index"]) for r in meta if r["split"] == "train"]
    selected = []
    for c in range(3):
        idx = [i for i in train_idx if labels[i] == c][:3]
        selected.extend(idx)
    assert len(selected) == 9

    print("idx smooth_abs edge_abs")
    for idx in selected:
        x = images[idx, 0]
        ys = valid_cross_corr2d(x, k_smooth)
        ye = valid_cross_corr2d(x, k_edge)
        s_abs = float(np.sum(np.abs(ys)))
        e_abs = float(np.sum(np.abs(ye)))
        assert np.isfinite(s_abs) and np.isfinite(e_abs)
        assert s_abs >= 0.0 and e_abs >= 0.0
        print(f"{idx:03d} {s_abs:.4f} {e_abs:.4f}")

if __name__ == "__main__":
    main()
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU, `float32`; nincs véletlen, két futás kimenete bitazonos):

```
idx smooth_abs edge_abs
000 64.6092 63.3881
001 62.8924 61.1272
010 70.1861 65.2292
120 69.4874 105.7347
121 68.0800 114.1036
122 67.6260 85.3780
240 70.1593 102.5014
241 68.9226 110.5914
242 71.3567 106.7612
```

A futtatott kimenet szerkezete megfelel a vártnak: minden sorban egy 0 és 359 közötti egész index, majd két nemnegatív, négy tizedesjegyre formázott összeg áll (a számított értékek `float32`-ben

készültek); az élválasz nulla, ha a kép az adott 3×3-as ablakon belül konstans. Ez utóbbi elégséges, de nem szükséges feltétel: a sakktábla-mintán nem konstans ablakokra is nulla lehet. Az eredményértelmezés során csak azt szabad állítani, amit a számított mennyiség mér: a simító válasz abszolút összege a lokális intenzitást, az élválaszé a lokális, irányfüggő (itt elsősorban vertikális) intenzitáskülönbséget tömöríti. A két szám különböző normalizálású, ezért abszolút összehasonlításuk csak ugyanazon a jeleneten, azonos normalizálás mellett értelmes; osztálycímkére vonatkozó következtetést nem hordoz.

13.3.8 Feladatok

F1 (fogalmi, nehézség 1). Definiálja röviden a lokális recepciós mezőt, a súlymegosztást és a feature mapot. Egy 6×6-os bemenetre, egy bemeneti csatornával és 3×3-as valid kernelre adja meg a kimenet alakját, az $Y[2, 2]$ kimeneti elem recepciós mezőjének pontos indexhalmazát és a megosztott súlyok számát bias nélkül.

F2 (levezetési/bizonyítási, nehézség 2). A törzsben bizonyított linearitást alkalmazza új esetre. Legyen X_1 a fenti lépcsős minta, X_2 a csupa 1 értékű 4×4-es minta, $\alpha = 2$, $\beta = -1$. Számolja ki $Z = 2X_1 - X_2$ mátrixát, majd a K_{edge} válaszát közvetlenül. Ellenőrizze, hogy az eredmény valóban $2F_K(X_1) - F_K(X_2)$, minden lépést és indexhatárt feltüntetve. Mutassa meg ezután, hogy a biasos $G(X) = F_K(X) + 1$ leképezésre $G(0) \neq 0$, és magyarázza meg, miért cáfolja ez a linearitást.

F3 (implementációs, nehézség 2). Írjon `valid_cross_corr2d(x,k)` NumPy-függvényt `float32` bemenettel, tesztelje a 4×4-es példán az él- és simító kernel várt kimenetét `atol=1e-6` toleranciával, majd adja meg a függvényt úgy, hogy az üres bemeneti szelet ne fordulhasson elő akkor sem, ha `H` vagy `W` kisebb a kernelnél (azaz ellenőrizze a valid tartomány pozitivitását használat előtt). Futtatható parancs és teljes kód szükséges.

F4 (hibakeresési, nehézség 2). Az alábbi kód a K_{edge} válaszát próbálja kiszámolni a 4×4-es lépcsős mintán:

```
def bad_edge_response(x):
    k = np.array([[1,1],[-1,-1]], dtype=np.float32)
    y = np.zeros((3,3), dtype=np.float32)
    for i in range(3):
        for j in range(3):
            y[i,j] = np.sum(x[i:i+2, i:i+2] * k,
                            dtype=np.float32)
    return y
```

Lokalizálja a hibát, magyarázza meg a következményét, javítsa a kódot, és ellenőrizze a javított változatot a kézi várt mátrixszal.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen deskriptív kísérletet a GlyphLab `train` részhalmazán: mérje a fenti 3×3-as simító és él kernel válaszainak abszolút összegét a három orientációs osztályra, de ne vonjon le predikciós következtetést. Adja meg a megfigyelési egységet, a rétegzést, a baseline-t, a metrikát és legalább két érvényességi korlátot. A terv ne feltételezzen előre győztest. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

13.3.9 Megoldások és indoklás

F1 megoldás. A lokális recepciós mező az adott kimeneti elemhez hozzájáruló bemeneti indexek halmaza; súlymegosztás esetén ugyanaz a kernel minden térbeli pozícióban szoroz; a feature map egy kimeneti csatorna térbeli tömbje. A 6×6 bemenet és 3×3 valid kernel kimenete 4×4. Az $Y[2, 2]$ elem recepciós mezője, 0-indexeléssel,

$$\mathcal{R}_{2,2} = \{(0, r, s) : 2 \le r < 5,\ 2 \le s < 5\},$$

vagy a csatornaindex elhagyásával a $[2, 5) \times [2, 5)$ bemeneti ablak (a 13.3.2 definíciója hármasokat sorol fel, itt $C_{in} = 1$ miatt a csatornaindex mindenhol 0). Egy bemeneti csatorna és egy kimeneti csatorna esetén a megosztott súlyok száma bias nélkül $3 \cdot 3 = 9$. Ha $C_{in} > 1$, a halmaz $C_{in} \cdot 9$ elemű, a megosztott súlyok száma pedig $C_{in} \cdot 9$ kimeneti csatornánként.

F2 megoldás. $Z = 2X_1 - X_2$ elemei: a felső két sor -1, az alsó két sor 1, hiszen a lépcsős minta értékeit megduplázzuk, majd minden elemből kivonunk 1-et. Tehát

$$Z = \begin{pmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}.$$

A K_{edge} valid válasza Z-n:

- $i = 0$ sor: az ablak konstans -1, így a kimenet 0;
- $i = 1$ sor: $Z_{1,j} + Z_{1,j+1} - Z_{2,j} - Z_{2,j+1} = (-1) + (-1) - 1 - 1 = -4$;
- $i = 2$ sor: konstans 1, a kimenet 0.

Tehát

$$F_K(Z) = \begin{pmatrix} 0 & 0 & 0 \\ -4 & -4 & -4 \\ 0 & 0 & 0 \end{pmatrix}.$$

A tétel szerint $F_K(2X_1 - X_2) = 2F_K(X_1) - F_K(X_2)$. A törzs példájában $F_K(X_1)$ a középső sorban -2. Az X_2 csupa 1 bemenetre a lemma alapján $F_K(X_2) = 0$, mert K_{edge} súlyösszege 0. Így $2(-2) - 0 = -4$ a középső sorban, ami egyezik. Alakellenőrzés: mindhárom kimenet 3×3, a számítás minden pozícióra ugyanazt a kernel-ablak definíciót használja. A biasos $G(X) = F_K(X) + 1$ esetén $G(0) = 1$, egy lineáris L leképezésre viszont $L(0) = 0$ mindig teljesül; ezért G nem lineáris. Az 1 itt a mátrix minden elemére hozzáadott skalár, nem tévesztendő össze a csupa 1 bemeneti mátrixszal.

F3 megoldás. A függvény a fenti `valid_cross_corr2d`; a valid tartomány ellenőrzéséhez érdemes kiegészíteni:

```
def valid_cross_corr2d(x, k):
    x = np.asarray(x, dtype=np.float32)
    k = np.asarray(k, dtype=np.float32)
    H, W = x.shape
    kh, kw = k.shape
    if H < kh or W < kw:
        raise ValueError(
            f"input {H}x{W} smaller than kernel {kh}x{kw}; "
            "valid output would be empty")
```

```
    y = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float32)
    for i in range(y.shape[0]):
        for j in range(y.shape[1]):
            y[i, j] = np.sum(x[i:i+kh, j:j+kw] * k,
                             dtype=np.float32)
    return y
```

Ez elkerüli, hogy a `np.zeros((H - kh + 1, ...))` negatív dimenzióval fusson: $H < k_h$ esetén $H - k_h + 1 \le 0$, amire a NumPy `ValueError`-t dob ugyan, de nem a valid tartományra utaló üzenettel; a függvény ezért a ciklus előtt explicit, beszédes `ValueError`-t ad. A teljes, önálló teszt (a függvény definíciójával együtt):

```
python - <<'PY'
import numpy as np

def valid_cross_corr2d(x, k):
    x = np.asarray(x, dtype=np.float32)
    k = np.asarray(k, dtype=np.float32)
    H, W = x.shape
    kh, kw = k.shape
    if H < kh or W < kw:
        raise ValueError(
            f"input {H}x{W} smaller than kernel {kh}x{kw}; "
            "valid output would be empty")
    y = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float32)
    for i in range(y.shape[0]):
        for j in range(y.shape[1]):
            y[i, j] = np.sum(x[i:i+kh, j:j+kw] * k,
                             dtype=np.float32)
    return y

x = np.array([[0,0,0,0],[0,0,0,0],[1,1,1,1],[1,1,1,1]], dtype=np.float32)
k = np.array([[1,1],[-1,-1]], dtype=np.float32)
y = valid_cross_corr2d(x, k)
expected = np.array([[0,0,0],[-2,-2,-2],[0,0,0]], dtype=np.float32)
assert np.allclose(y, expected, atol=1e-6)
try:
    valid_cross_corr2d(np.zeros((2, 2), dtype=np.float32), k)
except ValueError as e:
    print("ValueError:", e)
else:
    raise AssertionError("a kicsi bemenet nem adott ValueError-t")
print(y)
PY
```

A kimenet a törzsbeli Y_{edge} mátrix és a `ValueError` üzenete; az `np.allclose` az `atol=1e-6` toleranciát is ellenőrzi. A simító kernelre a 0,5 értékek miatt ugyanez a tolerancia elegendő. A PyTorch `F.conv2d`-vel való összehasonlítás `bias=None, stride=1, padding=0` mellett ugyanezt a valid cross-correlationt adja [D10], de a jelen feladat NumPy-referenciát kér.

F4 megoldás. A hiba a `x[i:i+2, i:i+2]` sorban van: a második index is `i`, nem `j`. Következmény: a j ciklusváltozó nem befolyásolja az ablak oszlopkezdetét, ezért a hibás függvény minden j-re ugyanazt az i-edik oszlopból induló blokkot számolja, így a 3×3-as kimenet oszlopai azonosak lesznek. Általános bemeneten a hibás kimenet éppen az $i = j$ pozíciókban egyezik a helyessel. Javítás:

```
def good_edge_response(x):
    k = np.array([[1,1],[-1,-1]], dtype=np.float32)
    y = np.zeros((3,3), dtype=np.float32)
    for i in range(3):
        for j in range(3):
```

```
            y[i,j] = np.sum(x[i:i+2, j:j+2] * k,
                            dtype=np.float32)
    return y
```

A hibát a 4×4-es lépcsős minta önmagában nem mutatja ki: ott minden oszlop azonos, ezért a hibás és a helyes függvény is ugyanazt a Y_{edge} mátrixot adja. Oszloponként változó bemenet kell, például

$$X_{\text{test}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix},$$

amelyre a hibás függvény $\begin{pmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, a helyes pedig $\begin{pmatrix} 0 & -1 & -1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$; a maximális abszolút eltérés 1. A javított változat ellenőrzése a kézi várt mátrixszal: `assert np.allclose(good_edge_response(x_step), y_edge, atol=1e-6)`; a megkülönböztető bemeneten `assert not np.allclose(bad_edge_response(X_test), good_edge_response(X_test))`. Alakellenőrzés: a `y` tömb `(3,3)`, a kernel `(2,2)`, a bemenet `(4,4)`; a valid kimeneti méret $4 - 2 + 1 = 3$ mindkét tengelyen.

F5 megoldásminta. Lehetséges protokoll:

- *Kérdés:* különbözik-e a három orientációs osztály deskriptív él- és simítóválasz-eloszlása a `train` részhalmazon?
- *Adat:* GlyphLab `data/images.npy`; a teljes adat 360 kép 180 jelenetből (jelenetenként két nézet), a `train` split 240 kép 120 jelenetből; a val és a test split 30-30 jelenetet tartalmaz. Az orientációs osztályok a `vertical`, `horizontal`, `diagonal` címkék (`data/labels.npy`, int64).
- *Megfigyelési egység:* `scene_id`; a két view ugyanahhoz a jelenethez tartozik, ezért a képenkénti szabadsági fokot nem szabad kétszer számolni (1.3. Adatfelosztás és a megfigyelési egység).
- *Rétegzés:* a train split 120 jelenete osztályonként 40 jelenetre oszlik, a statisztikát osztályonként, jelenetszintű átlagolással számoljuk.
- *Baseline:* a kép átlagos intenzitása, mert a simító válasz összege dominánsan a lokális szintet méri; az élválaszt ehhez viszonyítva értelmezhetjük.
- *Metrika:* a 3×3-as él- és simító kernel válaszának abszolút összege képenként, scene-szintű átlaggal; a szórást scene-resamplinggel becsülhetjük, de intervallumot itt nem írunk elő, mert nincs előzetes varianciamodell.
- *Korlátok:* (1) a deskriptív különbség nem oksági magyarázat és nem predikciós pontosság; (2) a valid kernel elhagyja a 24×24-es kép egy képpont széles peremét, tehát a határon lévő kontraszt nem jelenik meg; (3) a két kernel normalizálása eltér, ezért nyers számuk nem közvetlenül összemérhető.

A hallgatói jegyzőkönyvnek tartalmaznia kell a nyers táblázatot, a futtató parancsot és az eltérést a tervtől, ha volt. A negatív eredmény is elfogadható: ha a válaszok osztályonként nem különülnek el, az nem cáfolja a CNN tanulhatóságát, csupán azt mutatja, hogy ez a két fix kernel önmagában nem hordozza a címkét.

13.3.10 Gyakori tévedés, önellenőrzés és korábbi fogalom

Gyakori tévedés. A tanult kernel vizualizációja vagy a rögzített él kernel „élszerű” képe nem általános szemantikai magyarázat arra, hogy a háló mit tanult. Ellenpélda: a K_{edge} kernel vizuálisan vízszintes él-detektornak látszik, mégis a 4×4-es, 2×2-es váltakozó blokkokból álló sakktábla-mintán azonos nulla választ ad:

$$X_{\text{checker}} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \qquad F_{K_{\text{edge}}}(X_{\text{checker}}) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Ennek oka, hogy a kernel a sorok közötti egyirányú kontrasztot méri; a sakktábla minden ablakában a pozitív és negatív tagok kioltják egymást. Egyetlen kernel képe tehát nem definiálja a kimeneti szemantikát, és nem mondja meg, hogy a downstream rétegek hogyan kombinálják a feature mapokat. A vizuális interpretáció legfeljebb hipotézis, amelyet kontrollált perturbációval vagy más kísérlettel kell tesztelni.

Önellenőrzés. Ellenőrizze a következőket: a valid kimenet alakja $H-k_h+1$ és $W-k_w+1$; a K_{edge} súlyösszege 0, a K_{smooth} súlyösszege 1; a bias nélküli operátor a nulla bemenetet nullába viszi, a biasos nem; a súlyok száma nem függ a kép méretétől. A laborban az élválasz abszolút összege nem címke-predikció: ez a 12.3. Sanity check és kis mintára túlillesztés sanity-check logikájának megfelelően egy mérési érvényességi határ, nem általánosítási ígéret. A 9.3. Expresszivitás és approximációs állítások approximációs tételéhez képest itt egyetlen rögzített lineáris szűrő nagyon alacsony kapacitású modell; a labor nem is osztályoz, csak két fix szűrő válaszát méri, így a gyenge predikciós képesség nem mond ellent semmilyen univerzális approximációs állításnak. Az 1.3. Adatfelosztás és a megfigyelési egység alfejezetből örökölt csoportos szerkezet miatt a GlyphLab elemzésben a scene_id-t kell független egységnek tekinteni, akkor is, ha a választ képenként számoljuk.

13.3.11 Rövid összefoglaló

A cross-correlation réteg lokális recepciós mezőt, tértől független súlymegosztást és kimeneti csatornánkénti feature mapot kapcsol össze. A rögzített kernel olyan pont a tanulható paramétertérben, amelyet kézzel megadott él- vagy simító szűrő is reprezentálhat. A bias nélküli, rögzített kernelű operátor lineáris: a véges összeg linearitása koordinátánként bizonyítható. A bias hozzáadása ezt megtöri, mert a nulla bemenet képe nem nulla; a biasos művelet affin. A labor két fix szűrő deskriptív válaszát méri predikció nélkül. A kernel vizuális képe nem általános szemantikai magyarázat, amit a sakktábla-ellenpélda mutat. A következő egység a határkezelést és a frekvenciaintuíciót vizsgálja 13.4. Határkezelés és frekvenciaintuíció.

13.3.12 Források

- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks.* Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

13.4 Határkezelés és frekvenciaintuíció

13.4.1 Cél, előismeret és induló becslés

A 13.3. Lokális szűrők és tanult reprezentáció egységben a cross-correlation réteget úgy definiáltuk, hogy a kernel egy lokális ablakban mozog a bemeneten. Egy H hosszúságú véges 1D jelre és K hosszúságú kernelre azonban az ablak a széleken kilógna a definíciós tartományból. Ebben az egységben azt vizsgáljuk, hogyan definiáljuk a hiányzó pozíciókat; ez a **határfeltétel** (boundary condition) megválasztása.

Előismeret: a 13.3. Lokális szűrők és tanult reprezentáció cross-correlation jelölése; a függvény és linearitás fogalma a 4.1.–4.3. egységekből. A frekvenciaintuícióhoz a DFT-t használjuk, amelyet ebben az alfejezetben, lentebb vezetünk be; a laborban ez opcionális mélyítés.

Induló kérdés. Egy $H = 32$ hosszú jelet $K = 3$ hosszú, 1/3-mal normált átlagoló kernellel szűrünk. Hányféle, egymástól lényegesen eltérő módon definiálhatjuk a szélső pozíciókat, és a periodikus jel esetén melyik a „természetes” választás? A becslés: legalább három alapvető mód van — **valid**, **zero padding** és **körkörös (circular)** —, és periodikus jelnél a körkörös kiterjesztés őrzi meg a periodicitást; ezt ebben az alfejezetben a DFT-mélyítés pontos állítása is rögzíti (a bizonyítás hivatkozással).

13.4.2 Határfeltételek: definíció, jelölés és alakok

Legyen a bemeneti 1D jel $x \in \mathbb{R}^H$, a kernel pedig $w \in \mathbb{R}^K$, ahol $K = 2m + 1$ páratlan egész, az egyszerűség kedvéért. A 13.3. Lokális szűrők és tanult reprezentáció cross-correlation definíciója szerint a kimenet egy t pozícióban

$$y_t = \sum_{r=0}^{K-1} x^{\mathrm{pad}}_{t+r}\, w_r,$$

ahol $x^{\mathrm{pad}} \in \mathbb{R}^{H+2p}$ a **paddelt** (kiegészített) tömb, p a szimmetrikus padding mérete, és a kernel r indexe $0, \dots, K-1$ között fut, megfordítás nélkül. A kernel középpontja a $m = \lfloor K/2 \rfloor = (K-1)/2$ indexű pozíció, ezért $p = m$ esetén a kimenet hossza H, azonos a bemenettel; ezt **same** kimenetnek hívjuk. Ha $p = 0$, akkor a kernel soha nem lóg ki a jelből, és a kimenet hossza $H - K + 1$; ez a **valid** eset. A padding nélküli definícióban x^{pad} helyett közvetlenül x-et használunk, és a t index $0, \dots, H - K$ között fut.

Két objektumot különböztetünk meg. Az $x^{\mathrm{ext}} : \mathbb{Z} \to \mathbb{R}$ **virtuális kiterjesztés** minden egész indexre értéket ad, és a $0, \dots, H-1$ tartományon egyezik a jellel: $x^{\mathrm{ext}}_i = x_i$. A **paddelt tömb** ennek egy véges, nullától indexelt kivágata:

$$x^{\mathrm{pad}}_j = x^{\mathrm{ext}}_{j-p}, \qquad j = 0, \dots, H + 2p - 1.$$

A j tömbindex és az $i = j - p$ jelindex tehát p-vel el van tolva: $x^{\mathrm{pad}}_0 = x^{\mathrm{ext}}_{-p}$, $x^{\mathrm{pad}}_p = x_0$. A határfeltétel az x^{ext} értékét rögzíti a $0, \dots, H-1$ tartományon kívül:

1. **Valid (nincs padding).** $p = 0$, nem definiálunk külső pozíciót; a kernel kizárólag a jel belsejében mozoghat. A kimeneti hossz $H - K + 1$.
2. **Zero padding.** $x^{\mathrm{ext}}_i = 0$, ha $i < 0$ vagy $i \geq H$. A kimeneti hossz H (same esetben $p = m$), illetve általában $H + 2p - K + 1$.

3. **Körkörös (circular) padding.** $x_i^{\text{ext}} = x_{i \bmod H}$, ahol $i \bmod H$ a $0, \dots, H-1$ halmazba eső maradék. Ez azt jelenti, hogy a jelet mintha H periódussal periodikusan ismételnénk, és a kernel az ismételt példányon dolgozik.

Például $x = (1, 2, 3)$, $p = 1$ mellett a zero paddinges tömb $x^{\text{pad}} = (0, 1, 2, 3, 0)$, a circular paddinges $x^{\text{pad}} = (3, 1, 2, 3, 1)$.

A PyTorch `Conv2d` dokumentációja szerint a natív konvolúciós réteg szimmetrikus zero paddinget használ, és a `padding_mode` paraméter `'zeros'`, `'reflect'`, `'replicate'` vagy `'circular'` értéket vehet fel [D10]. A mi laborunkban a zero és circular módot hasonlítjuk össze kézi implementációval.

2D-ben a jelölés analóg. Legyen $X \in \mathbb{R}^{H \times W}$, a kernel pedig $\kappa \in \mathbb{R}^{K_h \times K_w}$ (a jelet κ-val jelöljük, hogy ne ütközzön a bemenet W szélességével), a padding (p_h, p_w). A virtuális kiterjesztés $X^{\text{ext}} : \mathbb{Z}^2 \to \mathbb{R}$, a paddelt tömb $X^{\text{pad}}_{a,b} = X^{\text{ext}}_{a-p_h,\, b-p_w}$, $X^{\text{pad}} \in \mathbb{R}^{(H+2p_h)\times(W+2p_w)}$, és a kimenet

$$Y_{i,j} = \sum_{r=0}^{K_h-1} \sum_{s=0}^{K_w-1} X^{\text{pad}}_{i+r,j+s}\, \kappa_{r,s}, \qquad i = 0, \dots, H_{out}-1,\ j = 0, \dots, W_{out}-1,$$

ahol a kimeneti méret $H_{out} = H + 2p_h - K_h + 1$ és $W_{out} = W + 2p_w - K_w + 1$.

A zero padding esetén a tartományon kívüli indexekre $X^{\text{ext}}_{u,v} = 0$, a circular esetén $X^{\text{ext}}_{u,v} = X_{u \bmod H,\, v \bmod W}$. Batch és csatornadimenziók esetén a padding csak a térbeli H, W dimenziókat érinti, az NCHW alakban N és C változatlan marad.

Intuíció. A valid mód a legóvatosabb: csak olyan kimenetet ad, ahol minden bemeneti érték valódi. A zero padding a hiányzó értéket nulla intenzitásúnak tekinti, ami a kép fekete szegélyének felel meg. A circular padding a jelet gyűrűként zárja: a bal szélen a jobb szélső értékek jelennek meg. Ez akkor természetes, ha a jel valóban periodikus, például egy körvonal mentén mintavételezett intenzitás; általános képen azonban mesterséges ugrást hoz létre a szemközti szélek között.

13.4.3 Ellenpélda: a határkezelés nem azonos globális operátort ad

A levezetési kötelezettségünk a következő hamis állítás cáfolata: *a különböző határkezelések ugyanazt a globális lineáris operátort adják ugyanazon a véges jelen.* Az alábbi állítás pontos ellenpéldát mutat.

Állítás (zero és circular padding különböző operátort ad). Legyen $H = 3$, $x = (1, 2, 3) \in \mathbb{R}^3$, $K = 3$, $w = (1, 1, 1) \in \mathbb{R}^3$ összegző kernel (a belőle normált átlagoló változat $(1/3, 1/3, 1/3)$ lenne), és $p = 1$ mindkét paddingnél. Ekkor a zero paddinges kimenet $y^{\text{zero}} = (3, 6, 5)$, a circular paddinges kimenet $y^{\text{circ}} = (6, 6, 6)$, tehát $y^{\text{zero}} \neq y^{\text{circ}}$.

Bizonyítás (ellenpélda). Zero padding esetén $x^{\text{pad}} = (x^{\text{ext}}_{-1}, x_0, x_1, x_2, x^{\text{ext}}_3) = (0, 1, 2, 3, 0)$, mert a -1 és a 3 jelindex a tartományon kívül esik. A definíció szerint:

$$\begin{aligned} y^{\text{zero}}_0 &= x^{\text{pad}}_0 + x^{\text{pad}}_1 + x^{\text{pad}}_2 = 0 + 1 + 2 = 3, \\ y^{\text{zero}}_1 &= x^{\text{pad}}_1 + x^{\text{pad}}_2 + x^{\text{pad}}_3 = 1 + 2 + 3 = 6, \\ y^{\text{zero}}_2 &= x^{\text{pad}}_2 + x^{\text{pad}}_3 + x^{\text{pad}}_4 = 2 + 3 + 0 = 5. \end{aligned}$$

Circular padding esetén a modulo szabály szerint $x^{\text{ext}}_{-1} = x_2 = 3$ és $x^{\text{ext}}_3 = x_0 = 1$, tehát $x^{\text{pad}} = (3, 1, 2, 3, 1)$. Így

$$y_0^{\text{circ}} = 3+1+2 = 6, \quad y_1^{\text{circ}} = 1+2+3 = 6, \quad y_2^{\text{circ}} = 2+3+1 = 6.$$

A két vektor különböző, így a két határkezeléshez tartozó lineáris leképezés nem azonos. ■

Következmény. A valid mód eleve más dimenziójú kimenetet ad ($H - K + 1 = 1$), így az sem lehet azonos a fenti kettővel. A határfeltétel tehát a véges jelű cross-correlation **definíciós része**, nem pusztán implementációs részlet.

Az ellenpélda egyben azt is mutatja, hogy az összegző kernel a circular paddingnél a teljes jelen konstans 6 értéket ad, mert a gyűrűn minden pozíció ugyanazt a három értéket látja; a zero padding a széleken csökkenti az összeget. (A $w = (1, 1, 1)$ összegző; a $w = (1/3, 1/3, 1/3)$ átlagoló kernel ugyanígy különböző kimenetet adna: zero paddinggel $(1, 2, 5/3)$, circular paddinggel $(2, 2, 2)$.)

13.4.4 Kézi példa és ellenőrző kód

A fenti kézi példa egyben a numerikus kontroll is. Az alábbi kód ugyanezt a számítást végzi el NumPy-ban, kiegészítve a labor két jelével. A `dtype` `float64`, mert a cél a pontos egyezés, nem a tanítási sebesség.

```
# lab/n052_boundary_check.py
import numpy as np

DTYPE = np.float64

def pad_1d(x, p, mode):
    """Szimmetrikus padding 1D jelekre."""
    x = np.asarray(x, dtype=DTYPE)
    if mode == "zeros":
        return np.pad(x, (p, p), mode="constant", constant_values=0.0)
    if mode == "circular":
        return np.pad(x, (p, p), mode="wrap")
    raise ValueError(f"ismeretlen mode: {mode}")

def corr_1d(x, w, boundary):
    """Cross-correlation, K páratlan, same kimenet zero/circular esetén."""
    H = x.shape[0]
    K = w.shape[0]
    assert K % 2 == 1, "csak páratlan kernel támogatott"
    p = K // 2
    if boundary == "valid":
        out = np.empty(H - K + 1, dtype=DTYPE)
        for t in range(out.shape[0]):
            out[t] = np.sum(x[t:t+K] * w)
        return out
    xp = pad_1d(x, p, boundary)
    out = np.empty(H, dtype=DTYPE)
    for t in range(H):
        out[t] = np.sum(xp[t:t+K] * w)
    return out

if __name__ == "__main__":
    x_demo = np.array([1.0, 2.0, 3.0])
    w_mean = np.array([1.0, 1.0, 1.0])

    print("zero   :", corr_1d(x_demo, w_mean, "zeros"))
    print("circular:", corr_1d(x_demo, w_mean, "circular"))
    np.testing.assert_allclose(corr_1d(x_demo, w_mean, "zeros"),
                               [3.0, 6.0, 5.0], atol=1e-12)
```

```
    np.testing.assert_allclose(corr_1d(x_demo, w_mean, "circular"),
                               [6.0, 6.0, 6.0], atol=1e-12)
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU, `float64`; nincs véletlen, két futás kimenete bitazonos):

```
zero   : [3. 6. 5.]
circular: [6. 6. 6.]
```

A tolerancia itt 10^{-12}, mert a műveletek egész értékű összegek.

13.4.5 Labor: periodikus és szélső jel szűrése

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `n052_lab.py` a `lab/` könyvtárból futtatható (`cd lab && python n052_lab.py`); a két jel determinisztikus, külső adatfájl nem kell.

Kérdés. Hogyan befolyásolja a határfeltétel a szűrő kimenetét periodikus és éles szélű jelen?

Input, import és adatút. A labor két determinisztikus 1D jelet generál a `lab/n052_lab.py` fájlban, külső adatfájl nélkül:

- $x_t^{\text{per}} = \sin(2\pi t/16)$, $t = 0, \ldots, 31$, $H = 32$: periodikus jel.
- $x_t^{\text{edge}} = 1$, ha $t < 16$, különben 0: éles él.

Kernel: $w^{\text{mean}} = (1/3, 1/3, 1/3)$ (aluláteresztő), és $w^{\text{diff}} = (-1, 0, 1)$ (élkiemelő, felüláteresztő). Mindkét kernelt zero és circular paddinggel futtatjuk, $p = 1$ same kimenettel. A `dtype` `float64`; a jelek determinisztikusak, nincs véletlen lépés, ezért seed nem szükséges.

```
# lab/n052_lab.py
import numpy as np
from n052_boundary_check import corr_1d

DTYPE = np.float64

t = np.arange(32, dtype=DTYPE)
x_per = np.sin(2.0 * np.pi * t / 16.0)
x_edge = np.ones(32, dtype=DTYPE)
x_edge[16:] = 0.0

w_mean = np.ones(3, dtype=DTYPE) / 3.0
w_diff = np.array([-1.0, 0.0, 1.0], dtype=DTYPE)

for name, x in [("periodikus", x_per), ("el", x_edge)]:
    for b in ["zeros", "circular"]:
        y_mean = corr_1d(x, w_mean, b)
        y_diff = corr_1d(x, w_diff, b)
        assert y_mean.shape == (32,) and y_diff.shape == (32,)
        assert np.all(np.isfinite(y_mean)) and np.all(np.isfinite(y_diff))
        print(f"{name:10s} {b:9s} mean szél: {y_mean[0]:+.4f} {y_mean[-1]:+.4f} "
              f"diff szél: {y_diff[0]:+.4f} {y_diff[-1]:+.4f}")

assert np.isclose(corr_1d(x_per, w_mean, "circular")[0], 0.0, atol=1e-12)
assert np.isclose(corr_1d(x_edge, w_diff, "zeros")[-1], 0.0, atol=1e-12)
assert np.isclose(corr_1d(x_edge, w_diff, "circular")[-1], 1.0, atol=1e-12)
```

Futtatás és ellenőrzés. A parancs a `lab/` munkakönyvtárból:

```
cd lab && python n052_lab.py
```

Ellenőrzés: a `corr_1d` függvény az alfejezet demonstrációs példáján a fenti futtatott értékeket adja; a laborban a periodikus jel átlagoló kernelre a belső pontokban mindkét padding azonos értéket ad; a bal szélen (y_0) a zero padding eltér, a jobb szélen (y_{31}) viszont azért egyezik a kettő, mert ebben a jelben $x_0 = 0$, és a körkörös folytatás a rácson túl éppen ezt a nulla értéket adja. Az éles élű jelet differenciáló kernelre a circular padding a jel két, egymástól távoli szélét kapcsolja össze, ami az egyik oldalon mesterséges ugrást okoz.

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU, `float64`; a táblázat a `n052_lab.py` négy kimeneti sorából, két futás kimenete bitazonos):

Jel	Padding	mean y_0	mean y_{31}	diff y_0	diff y_{31}
periodikus	zeros	+0,1276	−0,3633	+0,3827	+0,7071
periodikus	circular	−0,0000	−0,3633	+0,7654	+0,7071
él	zeros	+0,6667	0,0000	+1,0000	0,0000
él	circular	+0,6667	+0,3333	+1,0000	+1,0000

A periodikus szinusz 32 mintás ablakába pontosan két periódus fér; ezért a circular padding a jel saját folytatásához illeszkedik, a zero padding viszont a bal szélen módosítja az eredményt. A differenciáló szűrő körkörös módban sem ad zérus szélső értéket: a helyes érték a lokális jelváltozásból következik. Az éles élű jelnél a circular padding a távoli végeket is összekapcsolja, és az utolsó pixel gradiensét megváltoztatja.

Opcionális DFT-mélyítés. A körkörös határfeltételhez tartozó szűrés felírható a diszkrét Fourier-transzformációval. Legyen a DFT és az inverze

$$\hat{x}_k = \sum_{n=0}^{H-1} x_n e^{-2\pi i k n/H}, \qquad x_n = \frac{1}{H}\sum_{k=0}^{H-1} \hat{x}_k e^{2\pi i k n/H}, \qquad k, n = 0, \dots, H-1.$$

A kernelt terjesszük ki H periódusra nullákkal: $w_r = 0$, ha $K \le r < H$, és legyen $\hat{w}_k = \sum_{r=0}^{H-1} w_r e^{-2\pi i k r/H}$. A körkörös paddingű cross-correlation kimenete periodikus, és egy H hosszú periódusán a $p = \lfloor K/2 \rfloor$ eltolással

$$y_t = \sum_{r=0}^{K-1} x_{(t+r-p) \bmod H}\, w_r, \qquad \hat{y}_k = e^{-2\pi i k p/H}\, \hat{x}_k\, \overline{\hat{w}_k} = e^{-2\pi i k p/H}\, \hat{x}_k\, \hat{R}_k,$$

ahol $R(w)_s = w_{(-s) \bmod H}$ a **megfordított és modulo H eltolt** kernel, $\hat{R}_k$ pedig ennek DFT-je; valós w esetén $\hat{R}_k = \overline{\hat{w}_k}$. Az inverz transzformáció az $1/H$ tényezőt tartalmazza, amit a `np.fft.ifft` beépítve alkalmaz. Ez a **körkonvolúciós tétel** cross-correlationre vonatkozó alakja. A tétel bizonyítását itt elhagyjuk; a diszkrét Fourier-transzformáció tulajdonságainak részletes tárgyalása meghaladja ennek az alfejezetnek a keretét. Ha a DFT-szorzást a zero paddinges lineáris szűrésre akarjuk használni, a jelet és a kernelt nullákkal legalább $H + K - 1$ hosszra kell kiegészíteni; különben a körkörös átfolyás a kívánt lineáris konvolúció helyett körkonvolúciós eredményt ad. Mindkét művelet lineáris operátor, a különbség a határkezelésben van.

13.4.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja szavakban a valid, zero padding és circular padding határfeltételt egy H hosszú 1D jelre. Adjon meg egy H és K értéket, amelyre a valid kimenet hossza nem egyezhet a same kimenet hosszával. *Beadandó:* a három definíció saját szavakkal, a választott H, K és a kimeneti hosszak képlete.

F2 (levezetési, nehézség 2). Legyen $H = 4$, $x = (2, 4, 0, 1)$, kernel $w = (1, -1, 2)$ páratlan hosszúságú, $p = 1$. Számítsa ki a zero és circular paddingű same kimenet minden elemét, és ellenőrizze, hogy a kimenet hossza H. *Beadandó:* a paddelt jelek és a számítás lépései.

F3 (implementációs, nehézség 2). Írjon NumPy függvényt `corr_2d(X, W, boundary)` néven, amely 2D zero és circular paddingű same cross-correlationt valósít meg $X \in \mathbb{R}^{H \times W}$ bemenetre és páratlan K_h, K_w méretű $\kappa \in \mathbb{R}^{K_h \times K_w}$ kernelre (a kódban a kernel paramétere `W`). Tesztelje az $X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$, $\kappa = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$ példán. *Beadandó:* a kód, a futtatás és a kimeneti mátrix.

F4 (hibakeresési, nehézség 3). Az alábbi kód a lineáris (zero paddinges) szűrést DFT-szorzással akarja kiszámítani, de az eredmény hibás:

```
import numpy as np
x = np.array([1.0, 2.0, 0.0, 1.0])
w = np.array([1.0, 1.0, 1.0])
xpad = np.pad(x, (0, 1))              # 5 hossz
wpad = np.pad(w, (0, 2))              # 5 hossz
y_fft = np.fft.ifft(np.fft.fft(xpad) * np.fft.fft(wpad)).real
print(y_fft)  # hibás? hasonlítsa a zero paddinges corr_1d-hez
```

Lokalizálja a hibát, magyarázza el a körkörös átfolyást, és adja meg a javítást a minimális szükséges beágyazási hosszal. *Beadandó:* a hiba oka, a helyes beágyazás és az összehasonlító futás.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a GlyphLab 24×24-es képein (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) annak mérésére, hogy a zero padding és a circular padding hogyan változtatja meg egy fix 3×3-as élkiemelő kernel kimeneti statisztikáját a kép szélén. Adja meg a megfigyelési egységet, a splitet, a metrikát és az érvényességi korlátot. *Beadandó:* a protokoll szöveges terve, a scene_id csoportosítás indoklásával és az összehasonlítás korlátjával. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

13.4.7 Megoldások és indoklás

F1. A valid mód nem definiál külső pozíciót, ezért a kernel csak a jel belsejében dolgozhat, és a kimenet hossza $H - K + 1$. A zero padding a hiányzó pozíciókat 0-nak veszi, a circular padding pedig az indexet modulo H tekinti, mintha a jel H periódussal ismétlődne. Például $H = 8$, $K = 3$ esetén a valid kimenet hossza $8 - 3 + 1 = 6$, a same ($p = 1$) kimenet hossza 8, tehát a kettő nem egyezhet.

F2. $H = 4$, $K = 3$, $p = 1$. Zero padding: $x^{\text{pad}} = (0, 2, 4, 0, 1, 0)$. $y_0 = 0 \cdot 1 + 2 \cdot (-1) + 4 \cdot 2 = 0 - 2 + 8 = 6$; $y_1 = 2 \cdot 1 + 4 \cdot (-1) + 0 \cdot 2 = 2 - 4 + 0 = -2$; $y_2 = 4 \cdot 1 + 0 \cdot (-1) + 1 \cdot 2 = 4 + 0 + 2 = 6$; $y_3 = 0 \cdot 1 + 1 \cdot (-1) + 0 \cdot 2 = -1$. Tehát $y^{\text{zero}} = (6, -2, 6, -1)$.

Circular padding: az indexek modulo 4, tehát $x^{\text{pad}} = (1, 2, 4, 0, 1, 2)$ (a -1 jelindex $x^{\text{ext}}_{-1} = x_3 = 1$, a 4 jelindex $x^{\text{ext}}_4 = x_0 = 2$). $y_0 = 1{\cdot}1{+}2{\cdot}(-1){+}4{\cdot}2 = 1{-}2{+}8 = 7$; $y_1 = 2{\cdot}1{+}4{\cdot}(-1){+}0{\cdot}2 = -2$; $y_2 = 4 \cdot 1 + 0 \cdot (-1) + 1 \cdot 2 = 6$; $y_3 = 0 \cdot 1 + 1 \cdot (-1) + 2 \cdot 2 = 3$. Tehát $y^{\text{circ}} = (7, -2, 6, 3)$. Mindkét kimenet hossza $4 = H$.

F3. Példakód:

```
def corr_2d(X, W, boundary):
    X = np.asarray(X, dtype=np.float64)
    W = np.asarray(W, dtype=np.float64)
    H, Wd = X.shape
    Kh, Kw = W.shape
    assert Kh % 2 == 1 and Kw % 2 == 1
    ph, pw = Kh // 2, Kw // 2
    if boundary == "zeros":
        Xp = np.pad(X, ((ph, ph), (pw, pw)), mode="constant", constant_values=0.0)
    elif boundary == "circular":
        Xp = np.pad(X, ((ph, ph), (pw, pw)), mode="wrap")
    else:
        raise ValueError(boundary)
    Y = np.empty_like(X)
    for i in range(H):
        for j in range(Wd):
            Y[i, j] = np.sum(Xp[i:i+Kh, j:j+Kw] * W)
    return Y

X = np.arange(1, 10, dtype=np.float64).reshape(3, 3)
W = np.array([[0,1,0],[1,0,1],[0,1,0]], dtype=np.float64)
print(corr_2d(X, W, "zeros"))
# [[ 6.  9.  8.]
#  [13. 20. 17.]
#  [12. 21. 14.]]
print(corr_2d(X, W, "circular"))
# [[16. 17. 18.]
#  [19. 20. 21.]
#  [22. 23. 24.]]
```

Ellenőrzés. A kereszt alakú kernel a középpont és az átlók helyett csak a négy oldalszomszédot adja össze. Például zero padding mellett a bal felső pixel eredménye $Y_{0,0} = 0{+}0{+}2{+}4 = 6$, a belső pixelé $Y_{1,1} = 2{+}4{+}6{+}8 = 20$. Circular padding mellett a bal felső pixel $Y_{0,0} = 7{+}3{+}2{+}4 = 16$. A két, kóddal is ellenőrzött eredménymátrix:

$$Y^{\text{zero}} = \begin{pmatrix} 6 & 9 & 8 \\ 13 & 20 & 17 \\ 12 & 21 & 14 \end{pmatrix}, \qquad Y^{\text{circ}} = \begin{pmatrix} 16 & 17 & 18 \\ 19 & 20 & 21 \\ 22 & 23 & 24 \end{pmatrix}.$$

Mindkét kimenet alakja $(3, 3)$; a belső pixel mindkét határkezelésben azonos, míg a szélpixelek eltérhetnek.

F4. A közvetlen, 5 hosszú FFT-szorzás körkonvolúciót számít, emiatt az utolsó és az első minta összekeveredhet. Két, $H = 4$ és $K = 3$ hosszú sorozat lineáris konvolúciójához legalább $H + K - 1 = 6$ hosszú beágyazás szükséges. A `corr_1d` keresztkorrelációt használ, ezért a konvolúciós szorzás előtt a kernelt meg kell fordítani. A $K = 3$ (páratlan hosszúságú) kernel *same* kimenete a teljes, 6 hosszú konvolúció $p = 1$ pozíciójától kezdődő 4 elem:

```
x = np.array([1.0, 2.0, 0.0, 1.0])
w = np.ones(3, dtype=np.float64)
fft_size = len(x) + len(w) - 1   # 6
u = np.pad(x, (0, fft_size-len(x)))
v = np.pad(w[::-1], (0, fft_size-len(w)))
```

```
full = np.fft.ifft(np.fft.fft(u) * np.fft.fft(v)).real
same = full[1:1+len(x)]
np.testing.assert_allclose(same, corr_1d(x, w, "zeros"), atol=1e-12)
print(same)  # [3. 3. 3. 1.]
```

A `np.fft.ifft` az inverz DFT $1/H$ tényezőjét beépítve tartalmazza, ezért a `full` sorozat közvetlenül a lineáris konvolúció értékeit adja. Az eredeti 5 hosszú beágyazás nem alkalmas az általános lineáris konvolúció visszaállítására: a hibás kód kimenete $[2, 3, 3, 3, 1]$, a helyes zero paddinges same kimenet $[3, 3, 3, 1]$. A kettő nem azonos, de a hibás kimenet $[1{:}4] = [3, 3, 3]$ szelete egyezik a helyes kimenet első három elemével, ezért az első néhány érték puszta összevetése félrevezető; a helyes összehasonlításnak a **megfelelően eltolt same-szeletet** kell figyelembe vennie.

F5. A kísérleti terv. *Adat:* a GlyphLab teljes adata 360 kép 180 jelenetből (jelenetenként két nézet); a `train` split 240 kép, 120 jelenet. A `test` splitet nem használjuk, mert az a későbbi modellértékelésre van fenntartva, és a padding összehasonlítása nem modellválasztási kísérlet; a korrupt teszt (`corrupted_test.npy`) szintén csak befagyasztott modell utáni értékelésre szolgál. *Kernel:* Sobel-x, $K = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$, $p = 1$ same kimenet, `float64` dtype. *Megfigyelési egység:* `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik, és a split is scene-szintű. *Metrika:* a kép külső 1 pixelgyűrűjén (a $24{\times}24$-es kimenet első és utolsó sorában, illetve oszlopában) számított átlagos abszolút kimeneti érték különbsége a két padding között, képenként, majd scene-szintű átlag és bootstrap CI a `scene_id`-kből. Csak a külső 1 gyűrűt mérjük, mert 3×3-as kernel és $p = 1$ mellett a belső 22×22 pozíció független a határkezeléstől, ezért a teljes kép átlaga elhígítaná a hatást. A protokoll rögzíti a futtató parancsot. *Érvényességi korlát:* a GlyphLab mesterséges alakzatképein a szélek gyakran nullák, ezért a zero padding hatása kisebb lehet, mint természetes képeken; az eredmény nem általánosítható más adatra.

13.4.8 Tipikus tévedés, önellenőrzés és visszahívás

Tipikus tévedés. Az FFT-s körkonvolúció és a véges lineáris művelet csak megfelelő beágyazással felel meg egymásnak. Ha valaki egy $H = 4$ hosszú jelet és $K = 3$ hosszú kernelt változatlan hosszon FFT-szoroz, a művelet a $H = 4$ hosszú körkonvolúciót adja, nem a zero paddinges lineáris eredményt. Ellenpélda: $x = (1, 1, 1, 1)$, $w = (1, 1, 1)$; a zero paddinges same kimenet $(2, 3, 3, 2)$, a 4 hosszú FFT-szorzás viszont konstans $(3, 3, 3, 3)$-at ad (a körkörös átfolyás miatt az első és utolsó elem is 3). A javítás a legalább $H + K - 1$ hosszú nulla beágyazás, ahogy az F4 feladat mutatja.

Önellenőrzés. Ellenőrizze, hogy a same kimenet hossza H, a valid kimeneté $H - K + 1$, és hogy a paddelt jel hossza $H + 2p$. Számolja ki a szélső pozíciót kézzel, majd a kóddal. A zero és a circular kimenet egyenlősége önmagában nem dönti el, hogy a jel periodikus-e: egyezhetnek a szélső értékek akkor is, ha a körkörösen becsatolt érték éppen nulla (mint e szakasz szinuszánál $x_0 = 0$), vagy ha a kernel a szélső pozíciókon nulla súlyt ad, illetve ha $K = 1$ és nincs szélhatás; egy valóban H periódusú jelnél pedig eltérnek, ha a folytatás értéke nem nulla. Az egyenlőségből ezért csak az következik, hogy ezen a jelen és ezzel a kernellel a két kiterjesztés numerikusan ugyanazt adja.

Visszatekintés. A 13.3. Lokális szűrők és tanult reprezentáció cross-correlation definícióját használtuk megfordítás nélkül; a kernel indexelése és a padding mérete összhangban van a 14.1. Stride, padding, dilation és outputméret általános kimeneti képletével. A seed és a determinisztikus jegyzőkönyv kezelése a 12.4. III. ellenőrző pont: önállóan tanuló kis háló gyakorlatát követi; a laborban nincs véletlen, ezért seed rögzítése nem szükséges.

13.4.9 Rövid összefoglaló

A véges jelen a kernel széleken kilóg a tartományból, ezért a határfeltétel a cross-correlation definíciós része. A valid mód csak teljes ablakot enged, a zero padding nullával, a circular padding modulo indexeléssel pótol. Az ellenpélda megmutatta, hogy a zero és circular padding különböző lineáris operátort ad: periodikus jelnél a circular konzisztens, éles szélnél mesterséges ugrást okoz. A DFT-szorzás körkörös határfeltételt valósít meg; lineáris szűréshez a jelet és a kernelt legalább $H + K - 1$ hosszra kell nullázni. A labor két jelen és két kernelnél számszerűsíti a különbséget.

13.4.10 Következő alkalmazás

A 14.1. Stride, padding, dilation és outputméret egység a stride, padding és dilation általános kimeneti méretképletét vezeti le, és ezzel a CNN-réteg alakjainak teljes kontrollját adja meg.

13.4.11 Források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

14 CNN-réteg: alakok, csatornák és számítási költség

14.1 Stride, padding, dilation és outputméret

14.1.1 Cél, előismeret és induló kérdés

Ebben az egységben egyetlen célt járunk körbe pontosan: adott bemeneti képméret, kernel, padding, stride és dilation mellett mekkora lesz a CNN-réteg kimeneti térbeli mérete, és miért éppen ez a méret. A 13.4. Határkezelés és frekvenciaintuíció egységben már láttuk, hogy a határfeltétel, például a zero padding, a cross-correlation definíciójának része; most ezt a gondolatot általánosítjuk a CNN-réteg összes geometriai paraméterére. A 13.1. Kép mint diszkrét többcsatornás jel egységből visszahozzuk a mintavételi rácsot, a 2.1. Python, NumPy és tenzoralak egységből pedig a tenzoralakok és a broadcasting ellenőrzésének szokását.

Induló becslésként gondoljunk egy 7×7-es képre és egy 3×3-as kernelre, stride 1 mellett, padding nélkül. Hány pozícióba tudjuk a kernelt teljes egészében rátenni? A mag bal felső sarka a $0, 1, 2, 3, 4$ oszlopokban lehet, ezért a kimenet 5×5-nek ígérkezik. Kérdés: hogyan változik ez, ha lépésközt, kitöltést vagy tágítást kapcsolunk be, és mi a közös, bizonyítható szabály?

14.1.2 Definíciók: geometriai paraméterek és tenzoralakok

A bemenet egy $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ NCHW tensor, ahol H a magasság, W a szélesség. A csoportos konvolúciót (`groups`) itt nem tárgyaljuk; a csoportos esetet a 14.4. Csoportos és depthwise konvolúció egység vizsgálja. A konvolúciós mag

$$K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}.$$

A térbeli paraméterek:

Jel	Tartomány	Jelentés
k_h, k_w	pozitív egész	kernel magassága és szélessége
p_h, p_w	nemnegatív egész	szimmetrikus zero padding a két dimenzióban
s_h, s_w	pozitív egész	stride, azaz lépésköz
d_h, d_w	pozitív egész	dilation, azaz tágítási távolság

A **dilation** azt mondja meg, hogy a kernel két szomszédos együtthatója hány bemeneti rácspontra kerül egymástól. A kernel ténylegesen lefedett, **effektív kiterjedése** (kernel extent) ezért nem k, hanem

$$\ell_h = d_h(k_h - 1) + 1, \qquad \ell_w = d_w(k_w - 1) + 1.$$

Például $k_h = 3, d_h = 2$ esetén a három kernelpozíció a $0, 2, 4$ bemeneti indexeket fedi le, tehát a kiterjedés 5, nem 3.

A kimenet térbeli alakja $H_{out} \times W_{out}$. A [**D10**] dokumentációban használt programozói konvenció szerint a `Conv2d` **keresztkorrelációt** számol, azaz a kernel súlyait nem forgatja meg. Egy j kimeneti indexhez a kernel első ($r = 0$) együtthatójának horgonypozíciója a magasság mentén

$$js_h - p_h,$$

az r-edik kernelpozíció pedig a

$$js_h - p_h + d_h r$$

bemeneti indexet mintavételezi. A zero padding miatt az eredeti $[0, H-1]$ tartományon kívüli, de a $[-p_h, H-1+p_h]$ kiterjesztett tartományon belüli indexeket nulla helyettesíti; az ezen kívül eső horgonypozíciókhoz már nincs érvényes kimeneti ablak.

14.1.3 A kimeneti méret levezetése

A következő állítást az index-tartományból teljes egészében levezetjük; ez saját levezetés, amely a [**D10**] dokumentált indexezését használja.

Tétel (szimmetrikus paddinggel). Legyen H, k_h, d_h, s_h pozitív egész és p_h nemnegatív egész. Egy térbeli dimenzió mentén a keresztkorreláció érvényes kimeneti indexeinek halmaza

$$\mathcal{O}_h = \left\{ j \in \mathbb{Z}_{\geq 0} : \ \forall r \in \{0, \dots, k_h - 1\}, \quad -p_h \leq js_h - p_h + d_h r \leq H - 1 + p_h \right\}.$$

Ha $\mathcal{O}_h$ nem üres, akkor $\mathcal{O}_h = \{0, 1, \dots, H_{out} - 1\}$, ahol

$$H_{out} = \left\lfloor \frac{H + 2p_h - d_h(k_h - 1) - 1}{s_h} \right\rfloor + 1.$$

A $\mathcal{O}_h$ pontosan akkor üres, ha $H + 2p_h < d_h(k_h - 1) + 1$; ezt a beállítást **érvénytelen kimenetnek** nevezzük, és a PyTorch `Conv2d` forwardja ilyenkor `RuntimeError`-t vet (a hibaüzenet alakja például: „Kernel size can't be greater than actual input size").

Bizonyítás. Mivel $j \geq 0$, $s_h > 0$, $d_h > 0$ és $r \geq 0$, a bal oldali egyenlőtlenség

$$-p_h \leq js_h - p_h + d_h r$$

mindig teljesül. A jobb oldali feltétel minden r indexre akkor és csak akkor áll fenn, ha a legnagyobb $r = k_h - 1$ mellett sem sérül:

$$js_h - p_h + d_h(k_h - 1) \leq H - 1 + p_h.$$

Átrendezve:

$$js_h \leq H + 2p_h - d_h(k_h - 1) - 1.$$

A legnagyobb megengedett egész j ezért

$$J_{\max} = \left\lfloor \frac{H + 2p_h - d_h(k_h - 1) - 1}{s_h} \right\rfloor.$$

A feltétel bal oldala j-ben szigorúan monoton növekvő (meredeksége $s_h > 0$), ezért ha $J_{\max} \geq 0$, akkor minden $j = 0, 1, \ldots, J_{\max}$ index teljesíti az összes egyenlőtlenséget, tehát az érvényes indexek halmaza egy kezdőszelet: $\{0, 1, \ldots, J_{\max}\}$, amely minden nála kisebb nemnegatív egészt tartalmaz, és nem tartalmaz $J_{\max}$-nál nagyobbat. Így

$$H_{out} = |\{0, \ldots, J_{\max}\}| = J_{\max} + 1 = \left\lfloor \frac{H + 2p_h - d_h(k_h - 1) - 1}{s_h} \right\rfloor + 1.$$

Végül $\mathcal{O}_h$ akkor üres, ha $J_{\max} < 0$, azaz a számláló kisebb, mint 0. Ez éppen a

$$H + 2p_h < d_h(k_h - 1) + 1$$

feltétellel ekvivalens. □

A szélességre a teljesen azonos levezetés adódik, H, p_h, s_h, d_h, k_h helyett W, p_w, s_w, d_w, k_w szereplőkkel:

$$W_{out} = \left\lfloor \frac{W + 2p_w - d_w(k_w - 1) - 1}{s_w} \right\rfloor + 1.$$

A magasságra és szélességre vonatkozó feltételek egymástól függetlenek. A réteg akkor ad érvényes kimenetet, ha $H_{out} \geq 1$ és $W_{out} \geq 1$ egyszerre teljesül.

Két fontos megjegyzés tartozik a tétel hatásköréhez. Először: a formula $2p$-t tartalmaz, mert a paddinget a térbeli dimenzió **mindkét oldalán** azonos méretűnek tételeztük fel. Ha egy API vagy egy `F.pad` hívás aszimmetrikus paddinget használ, p_h^{left} bal és p_h^{right} jobb kitöltéssel, akkor a tétel bizonyításában $2p_h$ helyére $p_h^{\text{left}} + p_h^{\text{right}}$ kerül, mert a jobb széli feltétel csak a teljes padding mennyiségét érzékeli. Másodszor: a `padding='valid'` szöveges opció nem azonos a „van legalább egy kimeneti ablak” matematikai értelemben vett érvényességgel; a `'valid'` egyszerűen a $p = 0$ választás rövid neve.

14.1.4 Kézi kis példa

Legyen a bemenet $X \in \mathbb{R}^{1\times 2\times 6\times 5}$, tehát $N = 1, C_{in} = 2, H = 6, W = 5$. Válasszunk $k_h = k_w = 3$, $p_h = 1, p_w = 0$, $s_h = s_w = 2$ és $d_h = d_w = 1$ paramétereket. A mag effektív kiterjedése mindkét dimenzióban $3 \cdot 1 - 1 + 1 = 3$.

Magasságra:

$$H_{out} = \left\lfloor \frac{6 + 2 \cdot 1 - 1 \cdot (3 - 1) - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{6 + 2 - 2 - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{5}{2} \right\rfloor + 1 = 3.$$

Szélességre:

$$W_{out} = \left\lfloor \frac{5 + 0 - 1 \cdot (3 - 1) - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{2}{2} \right\rfloor + 1 = 2.$$

A kimenet térbeli alakja tehát 3×2. Ellenőrizzük a magassági indexeket: $j = 0$-nál a horgonypozíció $0 \cdot 2 - 1 = -1$, a lefedett indexek $\{-1, 0, 1\}$, a -1 a bal oldali paddingból érkezik. $j = 2$-nél a lefedett indexek $\{3, 4, 5\}$, ez még érvényes, mert $H - 1 + p_h = 6$ a jobb széli paddinget is tartalmazza. $j = 3$-nál a lefedés $\{5, 6, 7\}$ lenne, de 7 már túlmegy a kiterjesztett tartományon, ezért $j = 3$ nem érvényes. A számított 3 kimeneti index tehát pontosan fedi a lehetséges pozíciókat.

Egy gyors, önállóan futtatható kontroll ugyanerre:

```
import torch

H, W, k, ph, pw, s, d = 6, 5, 3, 1, 0, 2, 1
formula = ((H + 2*ph - d*(k-1) - 1)//s + 1,
           (W + 2*pw - d*(k-1) - 1)//s + 1)
conv = torch.nn.Conv2d(2, 4, k, stride=s, padding=(ph, pw), dilation=d,
                       bias=False, dtype=torch.float32, device="cpu")
y = conv(torch.zeros(1, 2, H, W, dtype=torch.float32, device="cpu"))
assert formula == tuple(y.shape[2:]), (formula, y.shape)
print("formula:", formula, "torch:", tuple(y.shape[2:]))
```

A mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; a blokk determinisztikus, seed nem szükséges): `formula: (3, 2) torch: (3, 2)`. A $C_{in} = 2, C_{out} = 4$ választás azt is ellenőrzi, hogy a csatornák száma nem befolyásolja a térbeli alakot.

14.1.5 Labor: alakteszt legalább hat beállításra

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `shape_test.py` a labor munkakönyvtárából futtatható (`python shape_test.py`); a teszt determinisztikus, külső adatfájl nem kell.

A labor célja, hogy ne egyetlen példán higgyünk a formulának, hanem egymástól eltérő beállításokon, nemnégyzetes bemenet és kernel mellett is összevessük a saját számítást a `Conv2d` tényleges kimeneti alakjával. A teszt determinisztikus, ezért az alakegyezéshez semmilyen lebegőpontos tolerancia nem kell; a seed csak a szokásos reprodukálhatósági kontrollt rögzíti.

Munkakönyvtár: `n053_lab/`. Futtatás: `python shape_test.py`.

```
# n053_lab/shape_test.py
import torch


def out1d(length, kernel, pad, stride, dilation):
    """Egy térbeli dimenzió kimeneti mérete szimmetrikus paddinggel."""
    assert length >= 1 and kernel >= 1 and stride >= 1 and dilation >= 1
    assert pad >= 0
    # A Python // negatív számlálónál is floor szerint oszt, ahogy a tételben a ⌊·⌋.
    numerator = length + 2 * pad - dilation * (kernel - 1) - 1
    if numerator < 0:
        raise ValueError(
            f"érvénytelen kimenet: d*(k-1)+1={dilation * (kernel - 1) + 1} "
            f"> L+2*p={length + 2 * pad}"
        )
    return numerator // stride + 1


def expected_shape(H, W, kh, kw, ph, pw, sh, sw, dh, dw):
    return (out1d(H, kh, ph, sh, dh), out1d(W, kw, pw, sw, dw))
```

```
def main():
    torch.manual_seed(20261020)
    dtype = torch.float32
    device = torch.device("cpu")

    # (H, W, kh, kw, ph, pw, sh, sw, dh, dw, név)
    cases = [
        (5, 5, 3, 3, 0, 0, 1, 1, 1, 1, "valid, négyzetes"),
        (7, 5, 3, 3, 1, 1, 1, 1, 1, 1, "nemnégyzetes, padding"),
        (8, 10, 3, 5, 0, 0, 1, 1, 1, 1, "nemnégyzetes input és kernel"),
        (7, 6, 3, 3, 0, 0, 2, 2, 1, 1, "stride 2, padding nélkül"),
        (9, 8, 3, 3, 0, 0, 1, 1, 2, 2, "dilation 2"),
        (5, 4, 4, 4, 1, 1, 2, 2, 1, 1, "padding+stride"),
        (13, 7, 5, 5, 0, 0, 2, 2, 1, 1, "nagyobb nemnégyzetes"),
        (6, 6, 6, 6, 0, 0, 1, 1, 1, 1, "éppen 1x1 kimenet"),
    ]

    for (H, W, kh, kw, ph, pw, sh, sw, dh, dw, name) in cases:
        exp = expected_shape(H, W, kh, kw, ph, pw, sh, sw, dh, dw)
        x = torch.zeros(1, 2, H, W, dtype=dtype, device=device)
        conv = torch.nn.Conv2d(2, 3, (kh, kw),
                               stride=(sh, sw),
                               padding=(ph, pw),
                               dilation=(dh, dw),
                               bias=False,
                               dtype=dtype, device=device)
        got = tuple(conv(x).shape[2:])
        assert got == exp, (name, got, exp)
        print(f"{name:30s} formula={exp} torch={got}")

    # Érvénytelen beállítás: a számláló negatív, ezért a formula nem használható.
    # A Conv2d forward ilyenkor RuntimeError-t ad, a saját függvényünk ValueError-t vet.
    invalid = [(3, 4, 0, 1, 1), (4, 5, 0, 1, 1)]
    for L, k, p, s, d in invalid:
        try:
            out1d(L, k, p, s, d)
        except ValueError as exc:
            print(f"érvénytelen L={L},k={k},p={p},s={s},d={d}: {exc}")
        else:
            raise AssertionError((L, k, p, s, d))
    print("Minden alaktartalom-ellenőrzés rendben.")


if __name__ == "__main__":
    main()
```

Az alábbiak a labor mért kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()` = 10, seed 20261020):

```
valid, négyzetes               formula=(3, 3) torch=(3, 3)
nemnégyzetes, padding          formula=(7, 5) torch=(7, 5)
nemnégyzetes input és kernel   formula=(6, 6) torch=(6, 6)
stride 2, padding nélkül       formula=(3, 2) torch=(3, 2)
dilation 2                     formula=(5, 4) torch=(5, 4)
padding+stride                 formula=(2, 2) torch=(2, 2)
nagyobb nemnégyzetes           formula=(5, 2) torch=(5, 2)
éppen 1x1 kimenet              formula=(1, 1) torch=(1, 1)
érvénytelen L=3,k=4,p=0,s=1,d=1: érvénytelen kimenet: d*(k-1)+1=4 > L+2*p=3
érvénytelen L=4,k=5,p=0,s=1,d=1: érvénytelen kimenet: d*(k-1)+1=5 > L+2*p=4
Minden alaktartalom-ellenőrzés rendben.
```

A teszt fontos tanulsága, hogy a validitási feltétel külön vizsgálandó: a tétel szerint $\mathcal{O}_h$ pontosan akkor üres, ha a számláló negatív, és ekkor a floor szerinti formula nullánál nem nagyobb értéket

adna, ami nem használható kimenet. A segédfüggvény ezért ilyenkor `ValueError`-t vet, nem csendben negatív vagy nulla méretet ad vissza.

14.1.6 Gyakori tévedés: a `same` padding nem mindig szimmetrikus

Gyakori félreértés, hogy a `same` padding pontosan ugyanaz, mint egyetlen szimmetrikus p megadása. A fenti tétel csak szimmetrikus paddingre érvényes, ezért érdemes ellenpéldát számolni.

Legyen $H = 5$, $k_h = 4$, $s_h = 2$, $d_h = 1$. Ha egy `same` típusú szabály a $3 = \lceil 5/2 \rceil$ kimeneti méretet célozza, akkor a kívánt teljes padding

$$(3-1)\cdot 2 + 4 - 5 = 3.$$

Az előző szakasz aszimmetrikus általánosítása szerint itt $p_h^{\text{left}} + p_h^{\text{right}} = 3$; a minimális megoldás balra 1, jobbra 2 pixelt tesz. A szimmetrikus $p = 1$ ehhez kevés:

$$H_{out} = \left\lfloor \frac{5 + 2 - 3 - 1}{2} \right\rfloor + 1 = 2,$$

a szimmetrikus $p = 2$ pedig ugyan 3 kimenetet ad, de 4 padding pixelt használ, tehát nem ugyanaz az operátor, mint a minimális 3 pixeles `same` változat:

$$H_{out} = \left\lfloor \frac{5 + 4 - 3 - 1}{2} \right\rfloor + 1 = 3.$$

A tanulság nem az, hogy a `same` opció „rossz”, hanem hogy a mögöttes padding-elosztás és a támogatott stride-ok könyvtáronként eltérnek. A PyTorch `Conv2d` a `padding='same'` értéket csak $s = 1$ mellett támogatja; $s > 1$ esetén `RuntimeError`-t vet (az üzenet a verziótól függően például: „padding=‘same’ is not supported for strided convolutions”). Páros kernelnél a `same` a szükséges teljes paddingot aszimmetrikusan osztja el (balra kevesebb, jobbra több nulla kerül), ezért az eredmény nem feltétlenül egyezik egy szimmetrikus p beállítással. A hordozható kódban érdemes explicit numerikus paddinget számolni, és a jelen formulát használni.

14.1.7 Önellenőrzés és korábbi fogalmak visszatekintése

Ellenőrizd a réteget a következő sorrendben: először az effektív kiterjedést, másodszor a validitási feltételt, harmadszor a floor formulát, végül mindkét térbeli dimenziót külön-külön. A 13.4. Határkezelés és frekvenciaintuíció tanulsága szerint a padding a lineáris operátor definíciójának része, ezért két eltérő padding-beállítás azonos kimeneti méret mellett is különböző számítást jelenthet. A 13.1. Kép mint diszkrét többcsatornás jel rácsindexelése nélkül a horgonypozíció feltételei nem írhatók fel. A korábbi 10.2. Affin réteg és aktiváció backward jellegű backward-levezetésekben a kimeneti indexhalmaz itteni, véges kezdőszelet volta (az, hogy $\mathcal{O}_h = \{0, \dots, H_{out} - 1\}$) fogja garantálni, hogy a gradiensek hozzájárulásai egy jól meghatározott, véges ablakhalmazon összegződnek.

14.1.8 Feladatok

F1 fogalmi, nehézség 1. Definiáld az effektív kernel-kiterjedést. Adj két különböző (k, d) párt, amelyeknek azonos az effektív kiterjedése, és mutasd meg egy $L = 11, p = 0, s = 1$ példán, hogy a kimeneti méretük azonos (az egy dimenziós példában L a hossz, L_{out} a kimeneti hossz; a kétdimenziós jelölésben ennek H és H_{out} felel meg). Magyarázd el, miért nem jelenti ez, hogy a két operátor azonos. Beadandó: definíció, két paraméterpár, kimeneti méret és egy indoklás.

F2 levezetési, nehézség 2. Számítsd ki a kimeneti méretet $H = 10, W = 7$, $k_h = 4, k_w = 3$, $p_h = 2, p_w = 1$, $s_h = 3, s_w = 2$, $d_h = 2, d_w = 1$ esetén. Ellenőrizd az indexfeltételből, hogy a magasság mentén $j = 2$ érvényes, de $j = 3$ nem; a szélesség mentén $j = 3$ érvényes, de $j = 4$ nem. Beadandó: mindkét dimenzió levezetése, a kiszámított H_{out}, W_{out} pár és a négy index feltételvizsgálata.

F3 implementációs, nehézség 2. Írj `out1d` és `expected_shape` függvényeket a tétel alapján, és teszteld legalább hat különböző beállításon, köztük nemnégyzetes inputtal. A valid eseteket hasonlítsd össze `torch.nn.Conv2d` kimeneti alakjával; egy érvénytelen beállításnál ellenőrizd, hogy a függvény hibát jelez a nem használható méret helyett. Beadandó: teljes futtatható kód és az output másolata.

F4 hibakeresési, nehézség 2. Az alábbi függvény hibás:

```
def out_bad(L, k, p, s, d):
    return (L + 2*p - (k - 1) - 1)//s + 1
```

Lokalizáld a hibát, mutass ellenpéldát $L = 9, k = 3, p = 0, s = 1, d = 2$ bemenettel, írd le a hiba következményét egy többrétegű hálóra, majd add meg a javított függvényt és a visszaellenőrzést. Beadandó: hiba oka, a hibás és helyes kimenet, javított kód.

F5 kísérleti/transzfer, nehézség 3. Tervezz rövid, helyi protokollt annak eldöntésére, hogy a telepített PyTorch-verzió `padding='same'` üzemmódja hogyan viselkedik a $H = 5, k = 4, d = 1$ beállításra $s = 1$, illetve $s = 2$ mellett. Az összehasonlítás alapja legyen a szükséges teljes padding paritása: ha páratlan, akkor minimális `same` padding nem lehet szimmetrikus. Rögzítsd a verziót, a stride-ot, a kapott vagy hiányzó kimeneti méretet és a következtetést a hordozhatóságról. Beadandó: protokoll, eredménytáblázat és értékelési kritérium. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

14.1.9 Megoldások és indoklás

F1 megoldás. Az effektív kiterjedés $\ell = d(k - 1) + 1$. A $(k = 5, d = 1)$ pár kiterjedése $1 \cdot 4 + 1 = 5$; a $(k = 3, d = 2)$ pár kiterjedése $2 \cdot 2 + 1 = 5$. $L = 11, p = 0, s = 1$ mellett (az egy dimenziós L a kétdimenziós H-nak felel meg) mindkettőre

$$L_{out} = \left\lfloor \frac{11 - 1 \cdot (5 - 1) - 1}{1} \right\rfloor + 1 = 7.$$

A kimeneti méret azonos, de a $d = 2$ kernel a bemeneti indexeket $0, 2, 4$ pozíciókban mintavételezi, a $d = 1$ kernel pedig $0, 1, 2, 3, 4$ pozíciókban, ezért a súlyok más lineáris kombinációt képeznek.

F2 megoldás. Magasság: $\ell_h = 2 \cdot (4 - 1) + 1 = 7$, a validitás $H + 2p_h = 14 \geq 7$ teljesül.

$$H_{out} = \left\lfloor \frac{10 + 4 - 2 \cdot 3 - 1}{3} \right\rfloor + 1 = \left\lfloor \frac{7}{3} \right\rfloor + 1 = 3.$$

A $j = 2$ index lefedése $2 \cdot 3 - 2 + \{0, 2, 4, 6\} = \{4, 6, 8, 10\}$, a maximum $10 \le H - 1 + p_h = 11$; $j = 3$-ra a maximum $13 > 11$. Szélesség: $\ell_w = 1 \cdot (3 - 1) + 1 = 3$, $W + 2p_w = 9 \ge 3$,

$$W_{out} = \left\lfloor \frac{7 + 2 - 1 \cdot 2 - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{6}{2} \right\rfloor + 1 = 4.$$

$j = 3$-nál a lefedés maximuma $3 \cdot 2 - 1 + 2 = 7 \le W - 1 + p_w = 7$, tehát érvényes; $j = 4$-nél $9 > 7$. A kimenet $H_{out} = 3, W_{out} = 4$.

F3 megoldás. A labor `shape_test.py` kódja teljes megoldás. Egy tömör, önállóan futtatható ellenőrzés:

```
import torch

def out1d(L, k, p, s, d):
    numerator = L + 2*p - d*(k - 1) - 1
    if numerator < 0:
        raise ValueError("érvénytelen kimenet: a kernel nagyobb a paddelt bemenetnél")
    return numerator//s + 1

def expected_shape(H, W, kh, kw, ph, pw, sh, sw, dh, dw):
    return (out1d(H, kh, ph, sh, dh), out1d(W, kw, pw, sw, dw))

cases = [
    ((5, 5), 3, 0, 1, 1),
    ((7, 5), 3, 1, 1, 1),
    ((7, 6), 3, 0, 2, 1),
    ((9, 8), 3, 0, 1, 2),
    ((5, 4), 4, 1, 2, 1),
    ((13, 7), 5, 0, 2, 1),
]
for (H, W), k, p, s, d in cases:
    exp = expected_shape(H, W, k, k, p, p, s, s, d, d)
    x = torch.zeros(1, 1, H, W)
    y = torch.nn.functional.conv2d(x, torch.zeros(1, 1, k, k),
                                   stride=s, padding=p, dilation=d)
    assert tuple(y.shape[2:]) == exp, (exp, y.shape)
    print((H, W), "->", exp)

try:
    out1d(3, 4, 0, 1, 1)
except ValueError:
    pass
else:
    raise AssertionError("az érvénytelen eset nem jelzett")
print("érvénytelen eset ellenőrizve")
```

Az alakegyezés egzakt egész összehasonlítás, ezért nincs szükség toleranciára. A mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`):

```
(5, 5) -> (3, 3)
(7, 5) -> (7, 5)
(7, 6) -> (3, 2)
(9, 8) -> (5, 4)
(5, 4) -> (2, 2)
(13, 7) -> (5, 2)
érvénytelen eset ellenőrizve
```

F4 megoldás. A hiba a dilation hiánya: a függvény $(k - 1)$-et használ a tényleges $d(k - 1)$ kiterjedés helyett. A megadott példán a $k = 3, d = 2$ kernel effektív kiterjedése 5, ezért a helyes eredmény

$$L_{out} = \left\lfloor \frac{9 - 2 \cdot 2 - 1}{1} \right\rfloor + 1 = 5,$$

a hibás függvény viszont

$$\frac{9 - 2 - 1}{1} + 1 = 7$$

értéket ad. A következmény az, hogy a következő réteg vagy egy későbbi `view`/`Linear` alakillesztése rossz 7 mérettel számol, és a forward során alakhiba keletkezik, pedig a számítás geometriailag csak 5 kimeneti pozíciót enged meg. Javítás:

```
def out_fixed(L, k, p, s, d):
    return (L + 2*p - d*(k - 1) - 1)//s + 1

assert out_fixed(9, 3, 0, 1, 2) == 5
assert out_bad(9, 3, 0, 1, 2) == 7
```

F5 megoldásminta. A protokoll három esetet vizsgál, mert a szükséges teljes padding paritása mondja meg, hogy a minimális `same` padding lehet-e szimmetrikus. A kívánt (`same`) kimeneti méretet $\widehat{H} = \lceil H/s_h \rceil$-ként rögzítjük; a kalapos jelölés a *célméretet* különbözteti meg a konkrét paddingból a formulával kapott H_{out} értéktől. A szükséges teljes padding

$$P_{\text{total}} = \max(0, (\widehat{H} - 1)s_h + d_h(k_h - 1) + 1 - H).$$

Az $H = 5, k = 4, s = 2, d = 1$ esetben $\widehat{H} = 3$ és

$$P_{\text{total}} = (3 - 1) \cdot 2 + 3 + 1 - 5 = 3.$$

A 3 páratlan, tehát a minimális `same` padding csak aszimmetrikusan valósítható meg. A három vizsgálandó eset:

(a) `padding='same'`, $s = 1$: a PyTorch ezt támogatja; a kimenet $H_{out} = 5$, és az eredmény a bal 1, jobb 2 pixeles kézi `F.pad` változattal egyezik meg, nem a szimmetrikus $p = 1$ változattal (amely $H_{out} = 4$).

(b) `padding='same'`, $s = 2$: a PyTorch `RuntimeError`-t vet, mert a `same` csak $s = 1$ mellett támogatott; a kívánt $\widehat{H} = 3$ méret így nem érhető el.

(c) kézi aszimmetrikus `F.pad` (bal 1, jobb 2) és $s_h = 2$: $H_{out} = 3$, ami éppen a kívánt kimeneti méret.

A jegyzőkönyvben rögzíteni kell a `torch.__version__` értéket, a három eset kimeneti alakját vagy a kivétel szövegét. A következtetés csak a saját verzióra és erre a három paraméterkombinációra érvényes; ez nem általános API-teszt.

Az induló becslésre a válasz. A 7×7-es bemeneten, 3×3-as kernellel, $s = 1$, $p = 0$ mellett a kimenet 5×5; a stride, a padding és a dilation a $H_{out} = \lfloor (H + 2p - d(k - 1) - 1)/s \rfloor + 1$ képletben módosítja ezt a méretet.

14.1.10 Rövid összefoglaló

A CNN-réteg térbeli kimeneti méretét a stride, a szimmetrikus padding, a kernelméret és a dilation együtt határozza meg. A kernel effektív kiterjedése $d(k-1)+1$; egy kimeneti index akkor érvényes, ha a horgonypozícióból indított összes mintavételi pont a paddinggel kiterjesztett tartományba esik. Az indexfeltételből levezettük a $H_{out} = \lfloor (H + 2p - d(k-1) - 1)/s \rfloor + 1$ formulát, és megadtuk a legalább egy kimeneti ablakot garantáló feltételt. A labor nyolc beállításon, nemnégyzetes inputtal és kernellel ellenőrzi a formulát. A `same` padding nem azonos automatikusan a szimmetrikus paddinggel: páratlan szükséges teljes padding esetén a minimális megoldás aszimmetrikus lehet, és a PyTorch a `padding='same'` értéket csak $s = 1$ mellett támogatja.

14.1.11 Továbblépés

A 14.2. Több bemeneti és kimeneti csatorna egység a több bemeneti és kimeneti csatorna összegzési szerkezetét vizsgálja; az itteni térbeli alakzatok adják a csatornánkénti kimeneti mátrixok méretét. A paraméterszámot viszont a csatornaszámok és a kernelméret határozza meg $(C_{out}C_{in}k_hk_w + C_{out})$, a kimeneti térbeli méret nem.

14.1.12 Források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

14.2 Több bemeneti és kimeneti csatorna

14.2.1 Cél, előismeret és induló kérdés

Ez az alfejezet az egycsatornás konvolúciós műveletet terjeszti ki több bemeneti és kimeneti csatornára. A 14.1. Stride, padding, dilation és outputméret egységben már definiáltuk a kimeneti méretet stride, padding és dilation mellett; a 13.2. Konvolúció és cross-correlation egységben pedig a keresztkorrelációt mint kernelmegfordítás nélküli szorzatösszeget. Most arra a kérdésre válaszolunk: ha egy 24×24-es szürkeárnyalatos GlyphLab-kép (a könyv szintetikus orientációs adatkészlete; a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) helyett háromcsatornás bemenetünk lenne, hogyan változik a súlytenzor alakja, és pontosan milyen összegzés állítja elő a kimeneti jellemzőtérképeket?

A CNN-eket építő programozó számára a fő veszély nem a képlet bonyolultsága, hanem a tengelyek összekeverése. Ezért minden állítást NCHW/OIHW alakkal, azaz batch–csatorna–magasság–szélesség, illetve kimeneti csatorna–bemeneti csatorna–kernelmagasság–kernelszélesség konvencióban adunk meg. A PyTorch `Conv2d` dokumentációja [D10] ugyanezt a súlyalakot specifikálja `groups=1` esetén, és kifejezetten keresztkorrelációról beszél. A CS231n jegyzet [S01] a szűrő (filter) terminológiát használja ugyanerre az objektumra; az itteni levezetés és számpélda saját.

14.2.2 Definíció és intuíció

Legyen a bemeneti köteg

$$X \in \mathbb{R}^{N \times C_{in} \times H_{in} \times W_{in}},$$

ahol N a batch méret, C_{in} a bemeneti csatornák száma, H_{in} és W_{in} a térbeli dimenziók. A súlytenzor

$$K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w},$$

és a torzításvektor

$$b \in \mathbb{R}^{C_{out}}.$$

A kimenet

$$Y \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}.$$

Minden rögzített $o \in \{0, \dots, C_{out} - 1\}$ kimeneti csatornához tartozó

$$K[o] \in \mathbb{R}^{C_{in} \times k_h \times k_w}$$

szeletet **kernel bank**nak nevezzük. Egy bank tehát C_{in} darab térbeli, $k_h \times k_w$-s kernelből áll: minden bemeneti csatornához egy. A teljes súlytenzor C_{out} darab ilyen bank egymásra helyezése.

Az intuíció a következő. Egy kimeneti jellemzőtérkép egyetlen neuronja nem csak a térbeli szomszédokat, hanem az összes bemeneti csatorna ugyanazon térbeli környezetét is látja. Először minden bemeneti csatornát külön-külön megszűrünk a bank megfelelő kernelével, majd ezeket a szűrt térképeket elemenként összeadjuk. Ez a **csatornaösszegzés** (channel sum). A torzítás ezután minden kimeneti csatornához egyetlen skalárt ad hozzá, amely térben és a batch mentén is konstans; ezért mondjuk, hogy a bias *per output channel* értendő.

A réteg formális definíciója rögzített n megfigyelés mellett a következő. Jelölje $S_{o,c}[i, j]$ a $X[n, c]$ és $K[o, c]$ egycsatornás keresztkorrelációját a 13.2. Konvolúció és cross-correlation és a 14.1. Stride, padding, dilation és outputméret indexezése szerint; ekkor

$$Y[n, o, i, j] = b[o] + \sum_{c=0}^{C_{in}-1} S_{o,c}[i, j], \qquad o = 0, \dots, C_{out} - 1.$$

A definíció tehát a csatornánkénti szűrés és a csatornaösszegzés kompozíciója; a következő lemma ennek zárt indexalakját mondja ki.

14.2.3 A teljes többcsatornás forward levezetése

A levezetéshez a 14.1. Stride, padding, dilation és outputméret egységben bevezetett kimeneti méretet használjuk szimmetrikus zero paddinggel:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1,$$

és W_{out} analóg módon. Itt $s \geq 1$ a stride, $p \geq 0$ a padding, $d \geq 1$ a dilation. Feltesszük, hogy $H_{out} \geq 1$ és $W_{out} \geq 1$.

Lemma (többcsatornás keresztkorreláció). Legyen X, K, b a fenti alakú, és legyen s, p, d a fenti feltételű. Ekkor minden $n \in \{0, \dots, N-1\}$, $o \in \{0, \dots, C_{out} - 1\}$, $i \in \{0, \dots, H_{out} - 1\}$ és $j \in \{0, \dots, W_{out} - 1\}$ indexre a kimenet

$$Y[n, o, i, j] = b[o] + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X[n, c,\ is - p + ud,\ js - p + vd]\ K[o, c, u, v].$$

Bizonyítás. A bizonyítás a réteg definíciójából indul ki, és nem használja fel a bizonyítandó állítást.

1. *Egy bemeneti csatorna szűrése.* Rögzítsünk egy c bemeneti csatornát és egy o kimeneti csatornát. A 13.2. Konvolúció és cross-correlation egység szerint a keresztkorreláció a kernel bal felső sarkát az input aktuális pozíciójához illeszti, kernelmegfordítás nélkül. A d dilation miatt a kernel u. sorhoz tartozó bemeneti sora $is - p + ud$, a v. oszlophoz tartozó bemeneti oszlopa pedig $js - p + vd$. A paddingen kívülre eső indexekre a bemenet nulla, ahogyan azt a szimmetrikus zero padding definíciója előírja. Így a c. csatorna hozzájárulása a o. kimeneti csatornához

$$S_{o,c}[i, j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X[n, c,\ is - p + ud,\ js - p + vd]\ K[o, c, u, v].$$

2. *A csatornák hozzájárulásának összegzése.* A réteg definíciója szerint a o. kimeneti csatorna neuronja minden bemeneti csatorna minden kernelpozíciójából érkező szorzatot összegzi. Ezért a kimeneti csatorna előtti teljes lineáris kombináció

$$\sum_{c=0}^{C_{in}-1} S_{o,c}[i, j] = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X[n, c,\ is - p + ud,\ js - p + vd]\ K[o, c, u, v].$$

A három összegzés sorrendje tetszőleges, mert véges sok tagot adunk össze; a fenti csoportosítás csak a levezetés átláthatóságát szolgálja.

3. *A torzítás hozzáadása.* A $b[o]$ skalár nem függ sem a c csatornaindextől, sem a u, v kernelpozíciótól, sem a térbeli i, j kimeneti indextől. Ezért a bias hozzáadása az előbbi összeghez a kimeneti csatornánkénti broadcast definíciója.

A négy lépés együtt éppen a lemma képletét adja. □

Alakellenőrzés. A jobb oldal minden rögzített (n, o, i, j) indexre egyetlen valós skalár. A $K[o]$ bank alakja $C_{in} \times k_h \times k_w$, tehát a c, u, v szerinti hármas összegzés pontosan a bank dimenzióit járja be. A $b[o]$ indexelése konzisztens a K első tengelyével, vagyis C_{out} darab skalár van. A kimenet első tengelye N, így a batch dimenzió az egyenletben csak passzív: ugyanaz a K és b alkalmazódik minden n megfigyelésre.

Megjegyzés a keresztkorrelációról. A fenti képletben a kernel indexe u, v növekvő sorrendben halad, és a bemeneti index is u, v-vel nő. Ez a programozói könyvtárakban használt keresztkorreláció. Ha valaki matematikai konvolúciót akar, a 13.2. Konvolúció és cross-correlation szerint a kernelt 180 fokkal el kell forgatni; tanult súlyoknál a két konvenció ekvivalens paraméterezést ad, de kézzel megadott szűrőknél numerikusan különbözik. A D10 dokumentáció a `Conv2d` műveletét keresztkorrelációként specifikálja; ezt a laborban is követjük.

14.2.4 Kézi kis példa

Legyen $N = 1$, $C_{in} = 2$, $H_{in} = W_{in} = 3$, $C_{out} = 2$, $k_h = k_w = 2$, stride 1, padding 0, dilation 1. Ekkor $H_{out} = W_{out} = 3 - 2 + 1 = 2$.

A bemenet két csatornája:

$$X[0,0] = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \qquad X[0,1] = \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}.$$

A két kernel bank:

$$K[0,0] = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad K[0,1] = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

$$K[1,0] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad K[1,1] = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}.$$

A torzítás $b = (1, -1)$. Számítsuk ki a $o = 0$, $i = 0$, $j = 0$ kimeneti elemet. A $c = 0$ csatorna bal felső 2×2-es ablaka $\begin{pmatrix} 1 & 2 \\ 4 & 5 \end{pmatrix}$, így

$$1 \cdot 1 + 2 \cdot 0 + 4 \cdot 0 + 5 \cdot (-1) = -4.$$

A $c = 1$ csatorna ablaka $\begin{pmatrix} 9 & 8 \\ 6 & 5 \end{pmatrix}$, így

$$9 \cdot 0 + 8 \cdot 1 + 6 \cdot (-1) + 5 \cdot 0 = 2.$$

A csatornaösszeg $-4 + 2 = -2$, a bias hozzáadása után $Y[0,0,0,0] = -1$. A többi pozíciót ugyanígy kiszámolva:

$$Y[0,0] = \begin{pmatrix} -1 & -1 \\ -1 & -1 \end{pmatrix}, \qquad Y[0,1] = \begin{pmatrix} 1 & 3 \\ 7 & 9 \end{pmatrix}.$$

Figyeljük meg, hogy a két kimeneti csatorna teljesen más térképet ad, noha ugyanazt a bemenetet látja: a különbséget a bankok és a biasok okozzák. Az alábbi kód ezt a példát is ellenőrzi.

14.2.5 Labor: NumPy referencia és PyTorch `Conv2d`, két bemeneti csatornával

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `n054_lab.py` futtatható (`python n054_lab.py`); a bemenet a szakaszban generált tenzor, külső adatfájl nem kell.

A labor a saját NumPy referenciát veti össze a PyTorch `conv2d` függvényével `float64` pontosságban. A referencia a 14.1. Stride, padding, dilation és outputméret alfejezetben bevezetett indexezést használja, de itt teljes egészében megadjuk, hogy a labor önállóan futtatható legyen.

```
# munkakönyvtár: lab/n054
# fájl: n054_lab.py
import numpy as np
import torch
import torch.nn.functional as F

def conv2d_numpy(x, weight, bias, stride=1, padding=0, dilation=1):
    """NCHW keresztkorreláció, groups=1, szimmetrikus zero padding."""
    N, C_in, H, W = x.shape
    C_out, C_in_w, kH, kW = weight.shape
    if C_in_w != C_in:
        raise ValueError("A suly C_in tengelye nem egyezik a bemenettel")
    H_out = (H + 2 * padding - dilation * (kH - 1) - 1) // stride + 1
    W_out = (W + 2 * padding - dilation * (kW - 1) - 1) // stride + 1
    y = np.zeros((N, C_out, H_out, W_out), dtype=x.dtype)
    for n in range(N):
        for o in range(C_out):
            for i in range(H_out):
                for j in range(W_out):
                    total = 0.0
                    for c in range(C_in):
                        for u in range(kH):
                            for v in range(kW):
                                xi = i * stride - padding + u * dilation
                                xj = j * stride - padding + v * dilation
                                if 0 <= xi < H and 0 <= xj < W:
                                    total += x[n, c, xi, xj] * weight[o, c, u, v]
                    y[n, o, i, j] = total + bias[o]
    return y

# 1) A kezi peld a ellenorzese
x_manual = np.array([[[1, 2, 3], [4, 5, 6], [7, 8, 9]],
                     [[9, 8, 7], [6, 5, 4], [3, 2, 1]]]], dtype=np.float64)
w_manual = np.array([[[1, 0], [0, -1]], [[0, 1], [-1, 0]]],
                    [[[0, 1], [1, 0]], [[-1, 0], [0, 1]]]], dtype=np.float64)
b_manual = np.array([1.0, -1.0], dtype=np.float64)

y_manual = conv2d_numpy(x_manual, w_manual, b_manual)
assert np.array_equal(y_manual[0, 0], [[-1, -1], [-1, -1]])
assert np.array_equal(y_manual[0, 1], [[1, 3], [7, 9]])
print("Kezi pelda, o=0:\n", y_manual[0, 0])
print("Kezi pelda, o=1:\n", y_manual[0, 1])

x_t = torch.from_numpy(x_manual)
w_t = torch.from_numpy(w_manual)
b_t = torch.from_numpy(b_manual)
y_t = F.conv2d(x_t, w_t, b_t, stride=1, padding=0)
diff_manual = torch.max(torch.abs(y_t - torch.from_numpy(y_manual))).item()
print("PyTorch elteres a kezi peldan:", diff_manual)

# 2) Veletlen float64 osszevetes ket bemeneti csatornaval
```

```
rng = np.random.default_rng(20261020)
N, C_in, H, W = 2, 2, 5, 5
C_out, kH, kW = 3, 3, 3
stride, padding, dilation = 2, 1, 1

x = rng.standard_normal((N, C_in, H, W)).astype(np.float64)
w = rng.standard_normal((C_out, C_in, kH, kW)).astype(np.float64)
b = rng.standard_normal((C_out,)).astype(np.float64)

y_ref = conv2d_numpy(x, w, b, stride, padding, dilation)
y_torch = F.conv2d(torch.from_numpy(x),
                   torch.from_numpy(w),
                   torch.from_numpy(b),
                   stride=stride, padding=padding, dilation=dilation)

err = np.max(np.abs(y_ref - y_torch.numpy()))
assert y_ref.shape == (2, 3, 3, 3)
assert err < 1e-12
print("Alakok: x", x.shape, "w", w.shape, "b", b.shape,
      "-> y", y_ref.shape)
print("max abszolut elteres:", f"{err:.3e}")
```

Futtatási parancs:

```
python n054_lab.py
```

A kimenet a fenti determinisztikus kódból, tényleges futtatással (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed 20261020):

```
Kezi pelda, o=0:
 [[-1. -1.]
 [-1. -1.]]
Kezi pelda, o=1:
 [[1. 3.]
 [7. 9.]]
PyTorch elteres a kezi peldan: 0.0
Alakok: x (2, 2, 5, 5) w (3, 2, 3, 3) b (3,) -> y (2, 3, 3, 3)
max abszolut elteres: 1.776e-15
```

Az első két blokk ellenőrzi a kézi számpéldát. A véletlen esetben a PyTorch és a NumPy referencia közötti maximális abszolút eltérés 0 vagy gépi epszilon nagyságrendű, mert mindkét oldal `float64`-ben számol; a mért $1{,}78 \cdot 10^{-15}$-ös eltérés a lebegőpontos összegzés eltérő sorrendjéből adódik. Ha `float32`-re váltanánk, a 2.2. Lebegőpont és stabil elemi számítás szerint nagyobb, de még mindig kicsi eltérésre számíthatnánk; a pontos egyezés nem várható el automatikusan. A csatornák számának növelése nem változtatja meg a kimeneti térbeli alakot, csak a bank méretét: $C_{in} = 4$ esetén a súly alakja $(C_{out}, 4, k_h, k_w)$ lenne.

14.2.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a kernel bank fogalmát, és adja meg az alakját $C_{in} = 3$, $k_h = 5$, $k_w = 5$ esetén. Miért van C_{out} darab bank, és mi a különbség egy bank és egyetlen térbeli kernel között?

F2 (levezetési, nehézség 1). Legyen $k_h = k_w = 1$, stride 1, padding 0, dilation 1, és hanyagoljuk el a biast. Vezesse le a Lemma képletéből, hogy ekkor

$$Y[n, o, i, j] = \sum_{c=0}^{C_{in}-1} X[n, c, i, j]\, K[o, c, 0, 0].$$

Magyarázza el, miért nevezzük ezt pontonkénti csatornakeverésnek, és ellenőrizze a kimeneti alakot.

F3 (implementációs, nehézség 2). Írjon `conv2d_numpy_s1p0` nevű függvényt, amely stride 1, padding 0 esetén valósítja meg a többcsatornás forwardot, és tesztelje a fenti kézi példán. Adja meg a függvény teljes kódját importokkal, a bemeneti tömböket, a várt kimenetet és a toleranciát.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a csatornaösszegzést próbálja NumPy-val elvégezni, de hibás:

```
def hibas_forward(x, w, b):
    N, C, H, W = x.shape
    Cout, _, kh, kw = w.shape
    Hout = H - kh + 1
    Wout = W - kw + 1
    y = np.zeros((Cout, Hout, Wout), dtype=x.dtype)
    for o in range(Cout):
        for i in range(Hout):
            for j in range(Wout):
                # HIBAS: a 0. tengely a batch, nem a csatorna
                y[o, i, j] = np.sum(
                    x[:, :, i:i+kh, j:j+kw] * w[o, :, :, :],
                    axis=(0, 2, 3)
                ) + b[o]
    return y
```

Lokalizálja a hibát, magyarázza el a következményét, és adja meg a javított sort. Milyen bemeneti alak esetén tűnne a kimenet véletlenül helyesnek?

F5 (kísérleti tervezés, nehézség 3). Tervezzen egy kis kísérletet annak eldöntésére, hogy a bias valóban kimeneti csatornánként egyetlen skalár-e, vagy térbeli pozíciónként változik. Fagyassza a súlyokat nullára, adjon meg ismert biasvektort, és írja le, milyen kimenetet vár a két hipotézis alatt, milyen statisztikával döntene, és milyen korlátai vannak a következtetésnek. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

14.2.7 Megoldások és indoklás

F1 megoldás. A kernel bank a K súlytenzor rögzített o kimeneti csatornához tartozó szelete: $K[o] \in \mathbb{R}^{C_{in} \times k_h \times k_w}$. A feladat adataival az alak $3 \times 5 \times 5$. Azért van C_{out} darab bank, mert minden kimeneti jellemzőtérkép saját kernel bankot kap. Egy bank nem egyetlen térbeli kernel, hanem C_{in} darab kernel együttese: minden bemeneti csatornához külön $k_h \times k_w$ kernel tartozik, és ezek kimeneteit összegezzük. Az egycsatornás eset akkor áll elő, ha $C_{in} = 1$; ekkor a bank egyetlen térbeli kernellé redukálódik.

F2 megoldás. A Lemma képletébe $k_h = k_w = 1$, $s = 1$, $p = 0$, $d = 1$ helyettesítve $u = v = 0$ az egyetlen kernelindex, és a bemeneti index $i \cdot 1 - 0 + 0 \cdot 1 = i$, illetve j hasonlóan. Így

$$Y[n, o, i, j] = \sum_{c=0}^{C_{in}-1} X[n, c, i, j]\, K[o, c, 0, 0].$$

Nincs térbeli szomszédosság, ezért a művelet minden (i, j) pozícióban külön-külön, a csatornák mentén lineárisan kombinál; ezért nevezzük **pontonkénti csatornakeverésnek** (pointwise channel mixing). A kimeneti alak a 14.1. Stride, padding, dilation és outputméret formulával

$H_{out} = H_{in} - 1 + 1 = H_{in}$, $W_{out} = W_{in}$; a kimenet $N \times C_{out} \times H_{in} \times W_{in}$. A tanulható skalárok száma $C_{in} C_{out}$.

F3 megoldás. A függvény a `conv2d_numpy` speciális esete:

```
# fajl: n054_f3.py
import numpy as np

def conv2d_numpy_s1p0(x, weight, bias):
    N, C_in, H, W = x.shape
    C_out, C_in_w, kH, kW = weight.shape
    assert C_in_w == C_in
    H_out = H - kH + 1
    W_out = W - kW + 1
    y = np.zeros((N, C_out, H_out, W_out), dtype=x.dtype)
    for n in range(N):
        for o in range(C_out):
            for i in range(H_out):
                for j in range(W_out):
                    total = 0.0
                    for c in range(C_in):
                        total += np.sum(
                            x[n, c, i:i+kH, j:j+kW] * weight[o, c, :, :]
                        )
                    y[n, o, i, j] = total + bias[o]
    return y

x = np.array([[[[1, 2, 3], [4, 5, 6], [7, 8, 9]],
               [[9, 8, 7], [6, 5, 4], [3, 2, 1]]]], dtype=np.float64)
w = np.array([[[[1, 0], [0, -1]], [[0, 1], [-1, 0]]],
              [[[0, 1], [1, 0]], [[-1, 0], [0, 1]]]], dtype=np.float64)
b = np.array([1.0, -1.0], dtype=np.float64)
y = conv2d_numpy_s1p0(x, w, b)
assert np.array_equal(y[0, 0], [[-1, -1], [-1, -1]])
assert np.array_equal(y[0, 1], [[1, 3], [7, 9]])
print(y[0, 0])
print(y[0, 1])
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a blokk determinisztikus, seed nem szükséges) a fenti kézi példa eredménye: `[[-1. -1.] [-1. -1.]]` és `[[1. 3.] [7. 9.]]`; az ellenőrzés egész számokon történik, ezért a tolerancia 0; lebegőpontos bemenetnél `np.allclose(y, y_expected, atol=1e-12, rtol=1e-12)` indokolt.

F4 megoldás. Két hiba van. (1) A kimeneti tömb alakjából hiányzik a batch-tengely: `y` helyesen `(N, C_out, H_out, W_out)`. (2) Az `axis=(0,2,3)` a batch és a két kernel-tengely fölött összegez, a csatornatengelyt meghagyja, ezért az eredmény `(C_in,)` alakú vektor, nem skalár; ezt a `y[o,i,j]` skalárhelyre írva a NumPy `ValueError`-t dob (`setting an array element with a sequence`). A hiba tehát nem csendes, és a kód a jelenlegi NumPy-ban semmilyen bemeneti alak mellett nem ad kimenetet. A javított sorok:

```
y = np.zeros((N, C_out, H_out, W_out), dtype=x.dtype)
# ...
y[:, o, i, j] = np.sum(
    x[:, :, i:i+kh, j:j+kw] * w[o, :, :, :], axis=(1, 2, 3)
) + b[o]
```

A kérdésre: a jelenlegi NumPy-ban nincs olyan bemeneti alak, amely mellett a hibás kód véletlenül helyes kimenetet adna, mert a `(C_in,)` alakú összeg skalárba írása mindig `ValueError`. Ha egy korábbi NumPy engedte volna az egyelemű tömb skalárba írását, akkor `N=1` és `C_in=1` mellett az érték matematikailag helyes lenne; `N=1` önmagában nem elég, mert `C_in>1` esetén a vektor több elemet tartalmaz.

F5 megoldás. Egy lehetséges terv: rögzítünk $N = 2$, $C_{in} = 2$, $H_{in} = W_{in} = 4$, $C_{out} = 3$, kernel 2×2, stride 1, padding 0. A súlyokat nullára állítjuk, a bias pedig legyen $b = (1, 2, 3)$. A Lemma a dokumentált per-channel szemantikára $Y[n, o, i, j] = b[o]$ értéket ad, ezért a kísérlet nem a Lemma igazságát teszteli, hanem azt, hogy a telepített implementáció követi-e a dokumentált szerződést. A két hipotézis: ha a bias kimeneti csatornánként egy skalár, akkor a kimeneti térkép minden (n, i, j) pozíciója ugyanaz a $b[o]$; ha térbeli bias lenne, akkor a b hossza $C_{out} \cdot H_{out} \cdot W_{out}$ kellene, és a térkép pozíciónként változna. Döntési statisztika: minden o kimeneti csatornára a kimenet szórása a pozíciók és a batch fölött, $s_o = \mathrm{std}_{n,i,j} Y[n, o, i, j]$. Döntési szabály: ha minden o-ra $s_o \leq 10^{-12}$ (`float64` mellett), a per-channel hipotézist fogadjuk el; ha valamelyik o-ra $s_o > 10^{-12}$, a térbeli bias hipotézisét. A korlát: ez a kísérlet csak a bias szerkezetét azonosítja, a súlyok csatornaösszegzését nem; továbbá a nulla súly melletti eredmény nem igazolja, hogy a bias a tanítás során is ezt a szerepet tartja meg.

14.2.8 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés szerint a batch és a csatorna tengely felcserélhető, mert mindkettő az első két NCHW-dimenzió, és a `reshape` vagy `transpose` $N = C_{in}$ esetén azonos alakot ad. Ez hamis. Legyen $N = C_{in} = 2$, $X \in \mathbb{R}^{2\times2\times4\times4}$ és $X' = \mathrm{transpose}(X, (1, 0, 2, 3))$, amelynek alakja ugyanaz, $(2, 2, 4, 4)$. Mivel az alak nem változik, ugyanaz a $K \in \mathbb{R}^{1\times2\times1\times1}$ kernel **hiba nélkül** lefut X'-n is, csak más értelmű számot ad: a csendes tengelycsere veszélye éppen ez. (Ha a batch- és a csatornaszám különbözik, például $X \in \mathbb{R}^{3\times2\times4\times4}$ és $X' = \mathrm{transpose}(X, (1, 0, 2, 3)) \in \mathbb{R}^{2\times3\times4\times4}$, akkor a $K \in \mathbb{R}^{1\times2\times1\times1}$ kernel az X'-n a $3 \neq 2$ csatornaeltérés miatt hibát ad, nem csendesen rossz kimenetet.) Konkrét példa: legyen $X[0, 0] \equiv 1$, $X[0, 1] \equiv 2$, $X[1, 0] \equiv 3$, $X[1, 1] \equiv 4$ (minden csatorna konstans), $K[0, :, 0, 0] = (1, 10)$, $b = 0$. Az eredeti X-re a két megfigyelés kimenete $1 \cdot 1 + 10 \cdot 2 = 21$ és $3 \cdot 1 + 10 \cdot 4 = 43$. Az X' első batch eleme az eredeti első csatorna két megfigyelését tartalmazza $(1, 3)$ csatornaként, a második a másodikét $(2, 4)$-ként, ezért a kimenet $1 + 30 = 31$ és $2 + 40 = 42$. Ugyanaz a kernel tehát más kimenetet ad, mint bármely eredeti megfigyelésre (futtatva ellenőrizve). A megfigyelési egység így sérül, ami az 1.3. Adatfelosztás és a megfigyelési egység szerint a split és a kiértékelés érvényességét is tönkreteszi.

14.2.9 Önellenőrzés és visszahívás

Ellenőrizze magát: (1) Mi a K alakja $C_{out} = 4$, $C_{in} = 2$, $k_h = 3$, $k_w = 3$ mellett? (Válasz: $(4, 2, 3, 3)$.) (2) Hány skalár bias tartozik ehhez a réteghez? (Válasz: 4.) (3) Ha $C_{in} = 1$, mire redukálódik a Lemma képlete? (Egyetlen csatorna összegzésére, azaz a 13.2. Konvolúció és cross-correlation alfejezetbeli egycsatornás keresztkorrelációra.) (4) Miért nem szabad a csatornaösszegzést a batch tengelyen elvégezni? (Mert a megfigyelések keverednének.)

Visszautalás: a 13.2. Konvolúció és cross-correlation egység definiálta a keresztkorrelációt és a bias fogalmát egy csatornára; az itteni Lemma ezt terjeszti ki a csatornaösszegzéssel. A 10.2. Affin réteg és aktiváció backward affin rétegénél láttuk, hogy a batch dimenzió passzív: az $XW^\top + b$ minden sorra ugyanazt a leképezést alkalmazza. Ugyanez a tulajdonság jelenik meg itt is a K és b batchfüggetlenségében. Végül a 2.2. Lebegőpont és stabil elemi számítás egység figyelmeztet, hogy a referencia és a PyTorch közötti numerikus egyezést toleranciával, `float64` mellett érdemes ellenőrizni.

14.2.10 Rövid összefoglaló

A többcsatornás konvolúciós réteg minden kimeneti csatornához egy kernel bankot rendel, amely C_{in} darab $k_h \times k_w$-s kernelből áll. A forward a bemeneti csatornák szűrt térképeit

elemenként összegzi, majd hozzáadja a kimeneti csatornánkénti skalár biast. A Lemma ezt a teljes összeget adja meg indexekkel, NCHW/OIHW alakokkal; a bizonyítás a csatornánkénti keresztkorrelációból és a véges összegek felcserélhetőségéből áll. A labor `float64`-ben ellenőrzi, hogy a NumPy referencia és a PyTorch `F.conv2d` azonos eredményt ad két bemeneti csatornával. A batch és a csatorna tengely nem cserélhető fel azonos elemszám ellenére sem.

14.2.11 Következő alkalmazás

A 14.3. Paraméterszám, MAC és aktivációmemória egység a paraméterszámot, a MAC-műveletek számát és az aktivációmemóriát számítja ki ezekből az alakokból. A kernel bank fogalma ott válik költségmodellé: a $C_{out} \times C_{in} \times k_h \times k_w$ szorzat a súlyok számát adja; ezt a kimeneti $H_{out}W_{out}$ térbeli mérettel szorozva kapjuk a **mintánkénti** MAC-számot, a teljes batchre pedig még az N tényezővel szorozva.

14.2.12 Források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

14.3 Paraméterszám, MAC és aktivációmemória

14.3.1 Cél, előismeret és induló kérdés

Ez az alfejezet a konvolúciós réteg három, egymástól élesen különböző mérőszámát vezeti be: a **paraméterszámot** (parameter count), a **MAC-műveletek számát** (multiply–accumulate) és az **aktivációmemóriát**. A kiinduló kérdés a következő: egy $3 \to 4$ csatornás, 3×3-as konvolúció 112 tanulható skalárt tartalmaz; hány szorzás–összeadás műveletet végez ez a réteg egyetlen 8×8-as képen, azaz $N = 1$ mellett? Ha a tipp tízes nagyságrendű, a válasz ennél jóval nagyobb: egyetlen képen 6912 MAC, mert a súlymegosztás miatt minden paraméter sok térbeli pozícióban újra dolgozik. (A későbbi számpéldában $N = 2$ szerepel, ezért ott a MAC-érték ennek kétszerese.) A 14.2. Több bemeneti és kimeneti csatorna alfejezetben levezetett kernel bank és a 13.3. Lokális szűrők és tanult reprezentáció alfejezetben tárgyalt súlymegosztás [S01] itt válik költségmodellé.

14.3.2 Definíciók és intuíció

Legyen egy 2D konvolúciós réteg bemenete $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, súlya $K \in \mathbb{R}^{C_{out} \times (C_{in}/g) \times k_h \times k_w}$, biasa $b \in \mathbb{R}^{C_{out}}$, ahol g a `groups` paraméter, C_{in} és C_{out} osztható g-vel, a kimeneti térbeli méret pedig $H_{out} \times W_{out}$ a 14.1. Stride, padding, dilation és outputméret alfejezetben rögzített képlet szerint.

Paraméterszám. A réteg tanulható skalár együtthatóinak száma:

$$P = C_{out} \cdot \frac{C_{in}}{g} \cdot k_h \cdot k_w + C_{out},$$

ahol az utolsó tag a bias, és csak `bias=True` esetén szerepel. A P a modell **kapacitásának** egy durva mutatója, és független a bemenet N, H, W méretétől.

MAC. Egy MAC egyetlen valós szorzásból és egyetlen valós összeadásból álló műveletpár. A réteg forward keresztkorrelációja

$$M = N \cdot C_{out} \cdot H_{out} \cdot W_{out} \cdot \frac{C_{in}}{g} \cdot k_h \cdot k_w$$

MAC-ot végez. A MAC a **számítási költség** mutatója, és a batch méretével lineárisan nő. A PyTorch `torch.nn.Conv2d` a dokumentáció szerint valójában **keresztkorrelációt** számol, azaz a kernelt nem forgatja meg [D10]; a továbbiakban a „konvolúció” szót ebben a programozói értelemben használjuk.

FLOP konvenció. Ebben a könyvben egy MAC-ot két lebegőpontos műveletnek számolunk: egy szorzásnak és egy összeadásnak, tehát a lineáris részre FLOP $= 2M$. A bias hozzáadása kimeneti elemenként további egy összeadás, tehát a teljes forward

$$\mathrm{FLOP}_{\mathrm{total}} = 2M + NC_{out}H_{out}W_{out},$$

ha a bias-összeadást is számoljuk. Ez a konvenció nem univerzális; egyes profilozók egy MAC-ot egyetlen műveletnek számítanak, így az általuk közölt érték a fenti átszámítás felének felel meg. A félreértés elkerülésére mindig a MAC-értéket adjuk meg elsődlegesen, és a FLOP-átszámítást külön jelöljük.

Aktivációmemória. A réteg kimeneti tenzora $X_{out} \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$, elemszáma $A = NC_{out}H_{out}W_{out}$, memóriaigénye $A \cdot e$ bájt, ahol e a dtype mérete: float32 esetén $e = 4$, float64 esetén $e = 8$. A visszafelé menethez a keretrendszer a bemeneti aktivációt is eltárolhatja, ezért a réteg tényleges aktivációigénye ennél nagyobb lehet.

Gradienspuffer. A visszafelé menet tenzorai: a súlygradiens $\nabla_K L$ alakja megegyezik K alakjával, a biasgradiens $\nabla_b L$ hossza C_{out}, a bemeneti gradiens $\nabla_X L$ alakja megegyezik X alakjával. A gradienspuffer a ténylegesen kiszámított gradiens tenzorok együttes méretét tartalmazza; a pontos foglalás és az átmeneti pufferek újrahasznosítása implementációfüggő, ezért itt tenzorszintű becslést adunk.

Batch-skálázás. Az N növelése a paraméterszámot nem változtatja, a MAC-ot és az aktivációmemóriát viszont N-szeresére növeli. Ezért nagy batch esetén a szűk keresztmetszet jellemzően a számítás és a memória, nem a paraméterek tárolása.

14.3.3 Paraméterszám: tétel és bizonyítás

Tétel (Conv2d paraméterszám). Legyen egy `torch.nn.Conv2d` réteg C_{in} bemeneti és C_{out} kimeneti csatornával, g csoporttal, $k_h \times k_w$ kernelmérettel, és `bias=True` esetén. Ekkor a réteg tanulható paramétereinek száma

$$P = C_{out} \cdot \frac{C_{in}}{g} \cdot k_h \cdot k_w + C_{out}.$$

Ha `bias=False`, a $+C_{out}$ tag elmarad.

Bizonyítás. A [D10] dokumentációban rögzített súlyalak $(C_{out}, C_{in}/g, k_h, k_w)$. Egy $(d_1, \dots, d_r)$ dimenziójú tenzor elemszáma a dimenziók szorzata, mert az indexhalmaz a $\{0, \dots, d_1 - 1\} \times \cdots \times \{0, \dots, d_r - 1\}$ Descartes-szorzat, amelynek számossága $d_1 \cdots d_r$. Ezért a súlytenzor $C_{out}(C_{in}/g)k_h k_w$ elemet tartalmaz. A bias tenzor alakja (C_{out}), tehát C_{out} további skalárt ad. A súly és a bias két diszjunkt paraméterhalmaz, így a számok összeadódnak. Speciálisan $g = 1$ esetén visszakapjuk a teljes csatornaösszekapcsolású képletet: $C_{out}C_{in}k_h k_w + C_{out}$.

A csoportos esetben minden kimeneti csatorna csak C_{in}/g bemeneti csatornát lát; a csoportosítás tehát a paraméterszámot is csökkenti, de a csatornák közötti kapcsolatot is korlátozza. Ezt a kompromisszumot a 14.4. Csoportos és depthwise konvolúció vizsgálja részletesen.

14.3.4 MAC és FLOP: lemma és konvenció

Lemma (MAC-szám). A fenti jelölésekkel, $N \times C_{in} \times H \times W$ bemenet és $N \times C_{out} \times H_{out} \times W_{out}$ kimenet esetén a réteg forward keresztkorrelációja

$$M = NC_{out}H_{out}W_{out} \cdot \frac{C_{in}}{g} \cdot k_h \cdot k_w$$

MAC-ot végez.

Bizonyítás. Rögzítsünk egy kimeneti elemet a (n, o, i, j) indexnégyessel. A 14.2. Több bemeneti és kimeneti csatorna alfejezetbeli forward összeg szerint ez az elem a csatornacsoport C_{in}/g bemeneti csatornáján és a kernel $k_h \cdot k_w$ pozícióján egy-egy szorzást végez, és az eredményeket összeadja. A szorzás–összeadás párok száma tehát $(C_{in}/g)k_hk_w$, függetlenül a konkrét pozíciótól. A kimeneti elemek száma $NC_{out}H_{out}W_{out}$. A két mennyiség szorzata adja M-et. A bias hozzáadása kimeneti elemenként egyetlen összeadás, szorzás nélkül, ezért a MAC-értékben nem szerepel; a FLOP-konvencióban külön tételként adjuk hozzá.

A MAC tehát a kimeneti tenzor elemszámának és az egy kimeneti elemhez tartozó belső összeg hosszának szorzata. A `dilation` paraméter nem változtatja a kernelpozíciók k_hk_w számát, csak a H_{out}, W_{out} értékeken keresztül hat a MAC-ra.

14.3.5 Aktivációmemória, gradienspuffer és batch-skálázás

Egy réteg kimeneti aktivációja $A = NC_{out}H_{out}W_{out}$ elemet tartalmaz. A **csak a kimenetre** számított memória $A{\cdot}s$ bájt. A teljes hálózat aktivációmemóriája a rétegenkénti kimenetek összege, ha minden réteg kimenetét megőrizzük a backwardhoz; a bemeneti tenzor mérete $NC_{in}HW \cdot s$. A gyakorlati keretrendszerek memóriahasználata ennél nagyobb lehet a gradienspuffer és a köztes munkaterületek miatt.

A gradienspuffer legfontosabb tagjai:

Tenzor	Alak	Elemszám
$\nabla_K L$	$C_{out} \times (C_{in}/g) \times k_h \times k_w$	$P - C_{out}$
$\nabla_b L$	C_{out}	C_{out}
$\nabla_X L$	$N \times C_{in} \times H \times W$	$NC_{in}HW$

A paramétergradiensek mérete azért batch-független, mert alakjuk megegyezik a K, illetve b alakjával; a veszteség batch-átlaga csak az értéküket aggregálja, a méretüket nem. A bemeneti gradiens viszont N-szeresére nő, ha az inputgradienst egyáltalán számoljuk. A gradiensmemória ezért nem mindig legalább $|dK| + |db| + |dX|$: a nem igényelt inputgradiens (az első réteg `requires_grad=False` bemenete) nem számolódik, az átmeneti pufferek újrahasznosulhatnak. Ezért külön kell választani a feltételes tenzor-elemszámot (mely tenzorok léteznek) és a mért csúcsmemóriát. CPU-s laborban a folyamat csúcs-RSS-je (`resource.getrusage(RUSAGE_SELF).ru_maxrss`) vagy a `torch.profiler` memóriaprofilja használható; a `torch.cuda.max_memory_allocated` csak CUDA-n értelmezhető, a `tracemalloc` pedig a PyTorch natív CPU-foglalásait nem látja megbízhatóan. A gradiensellenőrzés 10.3.

Gradiensellenőrzés és numerikus hibamérce alfejezetben float64-ben futott, ami $e = 8$ bájtos elemekkel kétszeres memóriát jelent a float32 traininghez képest; ezért a numerikus ellenőrzés és a tanítás memóriaigénye nem hasonlítható össze közvetlenül.

14.3.6 Kézi kis számpélda

Legyen $N = 2$, bemenet $2 \times 3 \times 8 \times 8$. Két réteget vizsgálunk: `conv1` $C_{in} = 3$, $C_{out} = 4$, $k = 3$, $p = 1$, $s = 1$; `conv2` $C_{in} = 4$, $C_{out} = 8$, $k = 3$, $p = 1$, $s = 2$ (a k, p, s rendre a négyzetes kernelméretet, a szimmetrikus paddinget és a stride-ot jelöli). A kimeneti méretek: $H_{out,1} = 8$, $H_{out,2} = \lfloor (8 + 2 - 3)/2 \rfloor + 1 = 4$.

Réteg	C_{in}	C_{out}	kernel, padding, stride	H_{out}	Paraméter	MAC ($N = 2$)
conv1	3	4	3, 1, 1	8	$4 \cdot 3 \cdot 9 + 4 =$ 112	$2 \cdot 4 \cdot 8^2 \cdot 3 \cdot 9 =$ 13824
conv2	4	8	3, 1, 2	4	$8 \cdot 4 \cdot 9 + 8 =$ 296	$2 \cdot 8 \cdot 4^2 \cdot 4 \cdot 9 =$ 9216
Összes					**408**	**23040**

A paraméteroszlop N-től független; a MAC-oszlop és a mögötte álló aktivációszámítás a megadott $N = 2$ batch-re vonatkozik.

Az aktivációk: conv1 kimenete $2 \cdot 4 \cdot 8^2 = 512$ elem, 2048 bájt float32-ben; conv2 kimenete $2 \cdot 8 \cdot 4^2 = 256$ elem, 1024 bájt. A FLOP-átszámítás a lineáris részre $2 \cdot 23040 = 46080$; a bias-összeadások száma $512 + 256 = 768$, így a teljes forward FLOP 46848. A paraméterszám ugyanakkor 408: a súlymegosztás miatt egy-egy paraméter átlagosan több mint ötvenszer vesz részt a számításban.

14.3.7 Labor: API-ellenőrzés

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Az ellenőrzés a szakasz kódblokkjából futtatható; külső adatfájl nem kell.

A labor célja, hogy a kézzel számolt értékeket a PyTorch API objektumain ellenőrizze. A kód CPU-n, float32-ben fut; seedre nincs szükség, mert csak alak- és paraméterellenőrzést végzünk. A 2.3. Izolált környezet, notebook és verzió alfejezetben rögzített izolált környezetben futtatandó.

```
# lab/n055/param_mac_check.py
# Munkakönyvtár: lab/n055
# Környezet: Python 3.12.3, torch 2.15.0+cpu, CPU, float32.
import torch
import torch.nn as nn

def conv_output_size(h, k, p, s):
    return (h + 2*p - k) // s + 1

def conv_mac(n, c_out, h_out, w_out, c_in_per_group, kh, kw):
    return n * c_out * h_out * w_out * c_in_per_group * kh * kw

def report(conv, x):
```

```
    with torch.no_grad():
        y = conv(x)
    n, c_out, h_out, w_out = y.shape
    c_in_per_group = conv.in_channels // conv.groups
    params = sum(t.numel() for t in conv.parameters())
    macs = conv_mac(n, c_out, h_out, w_out, c_in_per_group,
                    conv.kernel_size[0], conv.kernel_size[1])
    act_bytes = y.numel() * y.element_size()
    return params, macs, tuple(y.shape), act_bytes

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 4, 3, padding=1)
        self.conv2 = nn.Conv2d(4, 8, 3, padding=1, stride=2)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        return self.conv2(x)

model = SmallCNN()
total_params = sum(t.numel() for t in model.parameters())
assert total_params == 408, total_params
print("Teljes paraméterszám:", total_params)

x1 = torch.zeros(2, 3, 8, 8, dtype=torch.float32)
x2 = torch.zeros(2, 4, 8, 8, dtype=torch.float32)
for name, conv, x in [("conv1", model.conv1, x1),
                      ("conv2", model.conv2, x2)]:
    p, m, shape, abytes = report(conv, x)
    print(name, "params=", p, "mac=", m, "shape=", shape,
          "activation_bytes=", abytes)
    assert (shape[2], shape[3]) == (
        conv_output_size(x.shape[2], conv.kernel_size[0],
                         conv.padding[0], conv.stride[0]),
        conv_output_size(x.shape[3], conv.kernel_size[1],
                         conv.padding[1], conv.stride[1]))
```

Az alábbi a fenti kód mért kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; seed nem szükséges, mert csak alak- és paraméterellenőrzés történik):

```
Teljes paraméterszám: 408
conv1 params= 112 mac= 13824 shape= (2, 4, 8, 8) activation_bytes= 2048
conv2 params= 296 mac= 9216 shape= (2, 8, 4, 4) activation_bytes= 1024
```

Az `assert` a kimeneti méretet is ellenőrzi, így a padding és stride képlete nem számolódhat félre. Az `element_size()` float32 esetén 4-et ad; ha a környezet véletlenül float64-et használna, az aktivációbájtok megduplázódnának, amit a nyomtatott érték azonnal mutat.

14.3.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el saját szavaival, miért lehet egy konvolúciós réteg paraméterszáma kicsi, miközben a MAC-értéke nagy. Használja a labor `conv1` rétegét konkrét számpéldaként. Elvárt eredmény: a súlymegosztás és a kimeneti térbeli méret szerepeltetése; a 112 paraméter és a 13824 MAC összevetése.

F2 (levezetési, nehézség 2). Vezesse le a paraméterszámot és a MAC-ot egy csoportos konvolúcióra: $C_{in} = 6$, $C_{out} = 6$, $g = 3$, $k_h = k_w = 3$, `bias=True`, bemenet $N = 1$, $H = W = 10$, padding 0, stride 1. Ellenőrizze a $g = 1$ esetet is, és fogalmazza meg, mit veszít a csoportosítás

a megtakarításért cserébe. Elvárt eredmény: mindkét eset P, H_{out} és M értéke, valamint a kapcsolatkorlátozás megnevezése.

F3 (implementációs, nehézség 2). Írjon egy `conv_stats(model, x)` függvényt, amely egy `nn.Sequential` modellen végighaladva összegzi a konvolúciós rétegek paraméterszámát, MAC-értékét és aktivációbájtjait float32 bemenetre. A `ReLU` rétegen haladjon át alakváltozás nélkül. Tesztelje a labor `SmallCNN` modelljén. Elvárt eredmény: futtatható kód és a $(408, 23040, 3072)$ hármas ellenőrzése `assert`-tel.

F4 (hibakeresési, nehézség 2). Adott a következő hibás becslés:

```
params_bad = c_out * c_in * kh * kw        # (1)
macs_bad = c_out * h_out * w_out * kh * kw  # (2)
```

Azonosítsa mindkét hiba okát és következményét a labor `conv1` rétegén, adja meg a javított képletet, és ellenőrizze, hogy a javítás visszaadja a 112 és 13824 értékeket. Elvárt eredmény: mindkét hibás részképlet hibája, a `bias=True` és a batch/csatornatényező szerepének magyarázata, javítás és ellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen mérést annak eldöntésére, hogy két, azonos paraméterszámú, de eltérő konvolúciós elrendezésű modell közül melyiknek kisebb a következtetési (inference) késleltetése egy adott CPU-n. Adjon meg protokollt: bemeneti méret, batch, bemelegítés (warmup), ismétlésszám, időmérés módja, szinkronizáció, metrika és a következtetés érvényességi korlátja. Elvárt eredmény: egy lehetséges terv az alternatívák indoklásával; előre megadott győztes nem szükséges. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

14.3.9 Megoldások és indoklás

F1 mintamegoldás. A `conv1` súlytenzora $4 \cdot 3 \cdot 9 = 108$, biasszal együtt 112 tanulható skalárt tartalmaz. A forward során a kimeneti tenzornak $2 \cdot 4 \cdot 8 \cdot 8 = 512$ eleme van, és minden kimeneti elemhez $3 \cdot 9 = 27$ szorzás–összeadás pár tartozik: $512 \cdot 27 = 13824$ MAC. A paraméterek tehát azért használódnak ennyiszer, mert ugyanaz a 3×3-as kernel súlya a kép minden térbeli pozíciójában újra és újra alkalmazódik; ez a 13.3. Lokális szűrők és tanult reprezentáció alfejezetben tárgyalt súlymegosztás. A paraméterszám a kapacitást, a MAC a számítási költséget méri, a kettő nem cserélhető fel.

F2 megoldás. A kimeneti méret: $H_{out} = W_{out} = (10 + 0 - 3)/1 + 1 = 8$. A $g = 3$ esetben minden kimeneti csatorna $C_{in}/g = 6/3 = 2$ bemeneti csatornát lát. A Tétel szerint

$$P_3 = 6 \cdot \frac{6}{3} \cdot 3 \cdot 3 + 6 = 6 \cdot 2 \cdot 9 + 6 = 114,$$

és a Lemma szerint

$$M_3 = 1 \cdot 6 \cdot 8 \cdot 8 \cdot \frac{6}{3} \cdot 3 \cdot 3 = 1 \cdot 6 \cdot 64 \cdot 2 \cdot 9 = 6912.$$

A $g = 1$ esetben minden kimeneti csatorna mind a 6 bemeneti csatornát látja:

$$P_1 = 6 \cdot 6 \cdot 9 + 6 = 330, \qquad M_1 = 1 \cdot 6 \cdot 64 \cdot 6 \cdot 9 = 20736.$$

A csoportosítás tehát a MAC-ot harmadára csökkenti ($20736 \to 6912$), a súlyok számát pedig szintén harmadára ($324 \to 108$); a teljes paraméterszám a változatlan bias miatt $330 \to 114$, ami nem pontosan harmad. Cserébe az egyes kimeneti csatornák nem férhetnek hozzá a bemeneti csatornák többségéhez; a csatornák közötti kapcsolat korlátozott.

F3 mintakód.

```python
# lab/n055/feladat_f3.py
import torch
import torch.nn as nn

def conv_stats(model, x):
    total_params = total_macs = total_act_bytes = 0
    h = x
    with torch.no_grad():
        for layer in model:
            if isinstance(layer, nn.Conv2d):
                total_params += sum(p.numel() for p in layer.parameters())
                h = layer(h)
                n, c_out, h_out, w_out = h.shape
                c_in_per_group = layer.in_channels // layer.groups
                total_macs += n * c_out * h_out * w_out * c_in_per_group \
                              * layer.kernel_size[0] * layer.kernel_size[1]
                total_act_bytes += h.numel() * h.element_size()
            elif isinstance(layer, nn.ReLU):
                h = layer(h)  # az alak nem változik
            else:
                raise TypeError(f"Nem kezelt réteg: {type(layer)}")
    return total_params, total_macs, total_act_bytes

model = nn.Sequential(
    nn.Conv2d(3, 4, 3, padding=1),
    nn.ReLU(),
    nn.Conv2d(4, 8, 3, padding=1, stride=2),
)
x = torch.zeros(2, 3, 8, 8, dtype=torch.float32)
stats = conv_stats(model, x)
print(stats)
assert stats == (408, 23040, 3072), stats
```

A megoldás a labor `SmallCNN` rétegsorrendjét követi: conv1, ReLU, conv2. A `ReLU` alakváltozás nélkül adódik tovább, ezért az aktivációösszegbe nem számít új tenzort; ez a leképezés itt elfogadott, de a valódi keretrendszer memóriája ennél nagyobb lehet a gradienspuffer miatt.

F4 megoldás. Az (1) képlet két hibát tartalmaz: hiányzik a `groups` osztás, és hiányzik a bias. A labor `conv1` rétegen $g = 1$, ezért a csoportosítás most nem látszik, de a bias igen: a hibás érték $4 \cdot 3 \cdot 9 = 108$, a helyes $108 + 4 = 112$. A (2) képlet elfelejti a batch N és a bemeneti csatornacsoport C_{in}/g tényezőjét; a labor `conv1`-nél a hibás érték $4 \cdot 64 \cdot 3 \cdot 3 = 2304$, a helyes $2 \cdot 4 \cdot 64 \cdot 3 \cdot 9 = 13824$, tehát hatszoros alulbecslés. A javítás:

```python
c_in, c_out, kh, kw = 3, 4, 3, 3
groups, bias = 1, True
n, h_out, w_out = 2, 8, 8
params_ok = c_out * (c_in // groups) * kh * kw + (c_out if bias else 0)
macs_ok = n * c_out * h_out * w_out * (c_in // groups) * kh * kw
assert (params_ok, macs_ok) == (112, 13824), (params_ok, macs_ok)
```

A labor `conv1`-re `params_ok=112`, `macs_ok=13824`, ami visszaadja a kézzel számolt értéket. A hiba következménye a költségmodell alulbecslése: rossz memória- és futásidő-előrejelzés.

F5 mintamegoldás. Protokollterv: rögzített bemenet $N = 8$, $8 \times 3 \times 64 \times 64$, float32, CPU. Két konkrét, azonos paraméterszámú modell: az A modell két 5×5-ös rétegből áll, `Conv2d(3,36,5,padding=2,bias=False)`, `ReLU`, `Conv2d(36,29,5,padding=2,bias=False)`; a B modell négy 3×3-as rétegből: `Conv2d(3,32,3,padding=1,bias=False)`, `ReLU`, `Conv2d(32,32,3,padding=1,bias=False)`, `ReLU`, `Conv2d(32,32,3,padding=1,bias=False)`, `ReLU`, `Conv2d(32,33,3,padding=1,bias=False)`. Mindkettőben 28800 paraméter van, a kimenet 64×64, és mintánként a MAC is 117 964 800; a különbség a kernelméret és a rétegszám,

nem a számítás mennyisége. Bemelegítés: 20 forward hívás időmérés nélkül. Mérés: 200 ismétlés, minden ismétlésben egy teljes forward `torch.inference_mode()` alatt; a falióra-időt `time.perf_counter`-rel, a processzoridőt `time.process_time`-tal mérjük, mert a `perf_counter` nem CPU-idő. CPU-n nincs aszinkron végrehajtás, ezért `torch.cuda.synchronize`-ra nincs szükség; GPU-n a mérés előtt és után szinkronizálni kellene. Metrika: a batch-késleltetés mediánja és az ismétlések interkvartilis tartománya, mindkét órára. Összehasonlítás: a két modell váltakozó sorrendben fut, hogy a hőmérsékleti és ütemezési hatás ne terhelje egyoldalúan az egyiket. Következtetési korlát: az eredmény csak erre a hardverre, dtype-ra és batchre érvényes; más gépre vagy GPU-ra nem vihető át. A mérés előre nem dönt győztest; az azonos paraméterszám és MAC mellett a mért különbség a kernelimplementáció és a memóriahozzáférés hatását tükrözi, amit további profillal lehet szétválasztani, a 24.2. CPU/GPU profiling és helyes időmérés szerint.

14.3.10 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés: *„A kevés paraméter garantálja a kis következtetési késleltetést."* A paraméterszám és a késleltetés különböző mennyiség; a késleltetést a MAC, a memóriaforgalom, a kernelimplementáció és a hardver együtt határozza meg.

Ellenpélda: legyen az A modell egyetlen bias nélküli 3×3-as konvolúció $8 \to 16$ csatornával, 224×224-es bemeneten, padding 1, stride 1 mellett (így $H_{out} = W_{out} = 224$). Paraméterszáma $16 \cdot 8 \cdot 9 = 1152$, MAC-értéke $N = 1$ esetén $16 \cdot 8 \cdot 9 \cdot 224^2 \approx 57{,}8$ millió. A B modell egyetlen bias nélküli 1×1-es konvolúció $512 \to 512$ csatornával, 7×7-es bemeneten, padding 0, stride 1 mellett (így $H_{out} = W_{out} = 7$). Paraméterszáma $512 \cdot 512 = 262144$, tehát több mint kétszázszor nagyobb A-énál; MAC-értéke viszont $512 \cdot 512 \cdot 49 \approx 12{,}8$ millió, tehát kisebb, mint A-é. A paraméterszám szerinti rangsor tehát megfordulhat a számítási igény szerinti rangsorhoz képest. A tényleges késleltetést csak méréssel lehet eldönteni, de a MAC-eltérés már mutatja, hogy a paraméterszám önmagában nem elégséges előrejelző.

14.3.11 Önellenőrzés és korábbi fogalmak visszahívása

Ellenőrző lista: a paraméterképletben a bias-tag csak `bias=True` esetén szerepel; a `groups` osztja C_{in}-t és C_{out}-ot; a MAC-képletben N, C_{out}, H_{out}, W_{out}, C_{in}/g, k_h, k_w mind tényező; a FLOP-átszámítás $2M$ plusz bias-add. Szélsőeset: $H_{out} = W_{out} = 1$, $N = 1$ esetén a MAC a kernel bank méretével egyezik, $C_{out}(C_{in}/g)k_hk_w$. A gradienspuffer paraméterrésze batch-független, bemenetirésze batch-lineáris. A 14.2. Több bemeneti és kimeneti csatorna kernel bankja adja a paraméterszám tényezőit; a 13.3. Lokális szűrők és tanult reprezentáció súlymegosztása magyarázza, miért haladhatja meg a MAC a paraméterszámot; a 10.3. Gradiensellenőrzés és numerikus hibamérce float64 ellenőrzése miatt a gradiensellenőrzés memóriája nem azonos a float32 training memóriájával.

14.3.12 Rövid összefoglaló

A konvolúciós réteg paraméterszáma $C_{out}(C_{in}/g)k_hk_w + C_{out}$; ez a kapacitást méri, és független a bemeneti felbontástól. A MAC-érték $NC_{out}H_{out}W_{out}(C_{in}/g)k_hk_w$; a súlymegosztás miatt minden paraméter sok térbeli pozícióban dolgozik, ezért a MAC a kimeneti mérettel nő. A könyv FLOP-konvenciója egy MAC-ot két lebegőpontos műveletnek számol, a bias-összeadást külön tételként kezeli. Az aktivációmemória a kimeneti elemszám és a dtype méret szorzata, a gradienspuffer a paraméter- és bemenetigradiensek alakját követi. A paraméterszám nem helyettesíti a futásidő-mérést.

14.3.13 Következő alkalmazás és források

A 14.4. Csoportos és depthwise konvolúció egység a csoportos és depthwise konvolúciót vizsgálja; az itteni paraméter- és MAC-képlet ott válik a depthwise separable réteg költségmodelljének alapjává. A csoportosítás megtakarítása és kapcsolatkorlátozása közötti kompromisszumot számszerűen ott mérjük össze.

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

14.4 Csoportos és depthwise konvolúció

14.4.1 Cél, előismeret és induló kérdés

A 14.3. Paraméterszám, MAC és aktivációmemória egységben a teljes konvolúció paraméterszámát és MAC-igényét vezettük le. Itt a `groups` paramétert vizsgáljuk: ez nem pusztán egy gyorsítási kapcsoló, hanem a bemeneti és kimeneti csatornák közötti kapcsolat explicit korlátozása. Előismeret a keresztkorreláció (cross-correlation) képlete, a súlyalak és a kimeneti méret; Pythonból elég az alapvető lista- és ciklusismeret, PyTorchból a `torch.nn.Conv2d` alapvető használata.

Induló kérdés: egy $C_{in} = 4$, $C_{out} = 6$, 3×3-as rétegnél mit jelent a `groups=2`? Hány súlyt takarít meg, és mely bemeneti csatornák hatása szűnik meg egy adott kimeneti csatornán? A válaszhoz először a pontos definíció kell.

14.4.2 A `groups` paraméter definíciója

Legyen g a csoportok száma. A [D10] dokumentáció szerint a PyTorch `Conv2d` `groups` paramétere megköveteli, hogy $g \mid C_{in}$ és $g \mid C_{out}$ teljesüljön. Jelölje

$$r_{in} = \frac{C_{in}}{g}, \qquad r_{out} = \frac{C_{out}}{g}.$$

A súlytensor alakja ekkor

$$K \in \mathbb{R}^{C_{out} \times r_{in} \times k_h \times k_w},$$

a bias pedig $b \in \mathbb{R}^{C_{out}}$. A q-edik csoport, $q \in \{0, \dots, g-1\}$, a bemeneti csatornák közül a

$$\mathcal{C}_{in}^{(q)} = \{q r_{in}, \dots, (q+1) r_{in} - 1\}$$

halmazt, a kimeneti csatornák közül az

$$\mathcal{C}_{out}^{(q)} = \{q r_{out}, \dots, (q+1) r_{out} - 1\}$$

halmazt kapja. A lényeg: a q-edik kimeneti csoport csatornái kizárólag a q-edik bemeneti csoport csatornáit látják. A cross-correlation képlete szimmetrikus zero paddinggel, s_h, s_w stride-dal:

$$Y[n,o,i,j] = \sum_{c=qr_{in}}^{(q+1)r_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X\Big[n,c,\ s_h i+u-p_h,\ s_w j+v-p_w\Big]\, K[o,\ c-qr_{in},\ u,\ v] \ +\ b[o],$$

ahol $o \in \mathcal{C}_{out}^{(q)}$. Két részletre érdemes figyelni. Először: a súly második indexe 0 és $r_{in}-1$ között fut, ezért az abszolút c bemeneti csatornából le kell vonni a csoport kezdőindexét, qr_{in}-et. Másodszor: a bias továbbra is kimeneti csatornánként egy skalár; a csoportosítás a bias alakját nem változtatja.

A `groups=1` eset az eddigi teljes konvolúció: $r_{in} = C_{in}$, $r_{out} = C_{out}$, minden bemeneti csatorna minden kimeneti csatornához kapcsolódik. A másik szélső eset a depthwise konvolúció.

Az induló kérdés válasza. $C_{in} = 4$, $C_{out} = 6$, $g = 2$ esetén $r_{in} = 2$, $r_{out} = 3$: a súly alakja $6 \times 2 \times 3 \times 3$, azaz 108 súly, biasszal együtt 114 paraméter; a teljes konvolúció ugyanezen a bemeneten $6 \cdot 4 \cdot 3 \cdot 3 = 216$ súlyt használ, tehát a csoportosítás 108 súlyt takarít meg, a bias (6 elem) változatlan. A 0 és 1 bemeneti csatorna csak a $0, 1, 2$ kimenetre hat, a 2 és 3 bemeneti csatorna csak a $3, 4, 5$ kimenetre; például a 0 bemeneti csatorna megszűnik hatni a $3, 4, 5$ kimeneti csatornákra.

14.4.3 Depthwise konvolúció és csatornaszorzó (multiplier)

Ha $g = C_{in}$, akkor $r_{in} = 1$: minden bemeneti csatorna saját kernelhalmazt kap, és a csatornák között nincs keverés. A C_{out} kimeneti csatornát a

$$C_{out} = mC_{in}$$

összefüggés írja le, ahol m pozitív egész, az úgynevezett csatornaszorzó (depthwise multiplier). A súlyalak ilyenkor

$$K \in \mathbb{R}^{mC_{in} \times 1 \times k_h \times k_w}.$$

A `groups=C_in` és $C_{out} = C_{in}$ választás tehát $m = 1$; ekkor minden bemeneti csatornához pontosan egy kimeneti csatorna tartozik. Ha $m = 2$, minden bemeneti csatornából két különböző, egymástól független szűrt kép készül, de a két kimeneti csatorna ugyanazt az egy bemeneti csatornát látja.

A „depthwise separable” (csatornánkénti szétválasztható) kifejezés két réteg kompozícióját jelöli: először egy depthwise réteg térbeli szűrést végez csatornánként, majd egy pontszerű (pointwise), 1×1-es konvolúció keveri a csatornákat. A pointwise réteg definíciója:

$$Y[n,o,i,j] = \sum_{c=0}^{C_{in}-1} X[n,c,i,j]\, K[o,c,0,0] + b[o].$$

Ez minden (i,j) térbeli pozícióban ugyanazt a $C_{in} \to C_{out}$ affin leképezést hajtja végre; a kernel térbeli kiterjedése 1×1, tehát térbeli keverés nincs. A depthwise réteg ezzel szemben térbeli keverést végez, de csatornakeverést nem. A kettő együtt közelítőleg szétválasztja a térbeli és csatornairányú számítást; ennek költségmodelljét a 19.1. Depthwise separable és pointwise réteg egység vizsgálja majd.

14.4.4 Állítás: a grouped operátor blokkmátrix-szerkezete

A levezetési kötelezettség a group korlátozás lineáris algebrai alakja. Rögzítsük a paddinget, stride-ot és kernelt, és lapítsuk a bemeneti $X \in \mathbb{R}^{C_{in} \times H \times W}$ tensort csatorna-major sorrendben egy $x \in \mathbb{R}^{C_{in} HW}$ vektorrá: előbb a 0 csatorna minden pixele, majd az 1 csatorna, és így tovább. Hasonlóan lapítsuk a kimenetet $y \in \mathbb{R}^{C_{out} H_{out} W_{out}}$ vektorrá. A zero padding mellett a keresztkorreláció lineáris operátor; legyen L a mátrixa, azaz $y = Lx$ a bias nélkül.

Állítás (grouped operátor blokkdiagonális alakja). Ha $g \mid C_{in}$ és $g \mid C_{out}$, akkor a csatorna-major rendezésben az L mátrix

$$L = \begin{pmatrix} B^{(0)} & 0 & \cdots & 0 \\ 0 & B^{(1)} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & B^{(g-1)} \end{pmatrix}$$

alakú, ahol a q-edik blokk

$$B^{(q)} \in \mathbb{R}^{\, r_{out} H_{out} W_{out} \,\times\, r_{in} HW}$$

éppen annak a teljes cross-correlationnak a mátrixa, amely a q-edik csoport

$$\widetilde{K}^{(q)} = K[\, q r_{out} : (q+1) r_{out},\ :,\ :,\ : \,] \in \mathbb{R}^{r_{out} \times r_{in} \times k_h \times k_w}$$

súlyához tartozik.

Bizonyítás. A csatorna-major rendezésben az inputvektor q-edik blokkja, $x^{(q)} \in \mathbb{R}^{r_{in} HW}$, pontosan a $\mathcal{C}_{in}^{(q)}$ csatornák pixeleit tartalmazza. A kimeneti y vektor q-edik blokkja, $y^{(q)} \in \mathbb{R}^{r_{out} H_{out} W_{out}}$, a $\mathcal{C}_{out}^{(q)}$ csatornák pixeleit tartalmazza. A definiáló képlet szerint minden $o \in \mathcal{C}_{out}^{(q)}$ kimeneti csatorna összegzése csak a $c \in \mathcal{C}_{in}^{(q)}$ csatornákon fut. Ezért $y^{(q)}$ nem függ $x^{(p)}$-től, ha $p \neq q$; a mátrix q-edik sorblokkjában minden nem-q oszlopblokk zérus. A q-edik diagonális blokk elemeit a fenti képlet c-re és o-ra megszorított változata adja. Ha az összegzésben bevezetjük a csoporton belüli indexeket $c' = c - q r_{in}$ és $o' = o - q r_{out}$, a kapott egyenlet alakja pontosan egy r_{in} bemeneti és r_{out} kimeneti csatornás, $\widetilde{K}^{(q)}$ súlyú, ugyanolyan paddingű és stride-ú cross-correlation. Ez pedig definíció szerint $B^{(q)}$. Ezzel az állítást beláttuk. $\square$

A bizonyításban használt lapítás és az L elemei explicit alakban a következők. Az input indexe $(c, u, v) \mapsto cHW + uW + v$, a kimeneté $(o, i, j) \mapsto o H_{out} W_{out} + i W_{out} + j$. Ha q az o csoportja, akkor

$$L\big[(o,i,j),(c,u,v)\big] = \begin{cases} K[\, o,\ c - q r_{in},\ u,\ v \,], & \text{ha } c \in \mathcal{C}_{in}^{(q)},\ 0 \le s_h i + u - p_h < H,\ 0 \le s_w j + v - p_w < W, \\ 0, & \text{különben.} \end{cases}$$

A második sor a zero padding hatását rögzíti: a kiterjesztett tartományon kívülre mutató mintavétel együtthatója nulla, ezért a lineáris operátor az eredeti, paddelés nélküli X-en is jól definiált.

A bias b hozzáadása nem rontja a szerkezetet: minden kimeneti koordinátához a saját csatornájának skalárja adódik, csatornák közötti keverés nélkül. Az állítás egyben megmutatja, mit jelent a „kapcsolat korlátozása”: a csatornacsoportok szintjén a kapcsolati mátrix diagonális,

míg `groups=1` esetén egyetlen blokk van, tehát minden csatornapár össze van kötve. A mátrix blokkjain belül a térbeli pozíciók természetesen továbbra is keveredtek a kernel által; a group a térbeli keverést nem tiltja, csak a csatornák közötti utat szűkíti.

14.4.5 Kézi kis példa

Vegyünk egy $N = 1$, $C_{in} = 3$, $m = 2$, tehát $C_{out} = 6$, $g = 3$ depthwise konfigurációt, 3×3 bemenettel, 2×2 kernellel, padding és bias nélkül. A kimeneti méret $H_{out} = W_{out} = 2$. Állítsunk be minden depthwise kernel elemet 1-re, és legyen a c bemeneti csatorna konstans c értékű: $X[0, c, \cdot, \cdot] = c$.

A 2×2-es csupa egyes kernel egy konstans c bemeneten a $4c$ összeget adja mind a négy kimeneti pozícióban. A c bemeneti csatornához a $o = 2c$ és $o = 2c + 1$ kimeneti csatornák tartoznak, tehát a várt kimenet

$$Y[0, :, 0, 0] = (0, 0, 4, 4, 8, 8).$$

A 0 bemeneti csatorna az $o = 0$ **és** az $o = 1$ kimeneti csatornára is hat (itt azért nulla mindkettő, mert maga a bemenet nulla értékű, nem szerkezeti nulla miatt). A szerkezeti függetlenség példája: a 0 bemeneti csatorna nem hat az $o = 2, 3, 4, 5$ kimenetekre, és a 2 bemeneti csatorna sem az $o = 0, 1$-re; ezt nem a nullértékű bemenetből, hanem a csoportos kapcsolati mátrixból olvassuk le. A kontraszt a teljes konvolúcióval: `groups=1` és csupa egyes kernel mellett a három bemeneti csatorna összege minden kimeneti pozíción $0+1+2 = 3$, négy térbeli pozíción $4 \cdot 3 = 12$ lenne minden kimeneti csatornán; a depthwise csoportosítás ezt a szerkezetet töri meg. A nemnulla bemenetes ellenőrzést a következő kód végzi el: a 0 bemeneti csatornát 5-re állítja, és ellenőrzi, hogy a $2, 3, 4, 5$ kimenet változatlan, míg a $0, 1$ kimenet megváltozik. A kapcsolat tehát csatornacsoportonként izolált; ez a legkisebb olyan példa, amelyben a korlátozás a nemnulla bemenetes ellenőrzéssel ténylegesen látszik.

```
# munkakönyvtár: lab/
import torch

conv = torch.nn.Conv2d(3, 6, (2, 2), padding=0, groups=3, bias=False,
                       dtype=torch.float32)
with torch.no_grad():
    conv.weight.fill_(1.0)

x = torch.zeros(1, 3, 3, 3, dtype=torch.float32)
for c in range(3):
    x[0, c].fill_(float(c))

y = conv(x)
assert y.shape == (1, 6, 2, 2)
assert torch.equal(y[0, :, 0, 0], torch.tensor([0., 0., 4., 4., 8., 8.]))
print(y.shape)
print(y[0, :, 0, 0])

x_nonzero = x.clone()
x_nonzero[0, 0].fill_(5.0)          # csak a 0. bemeneti csatorna változik
y_nonzero = conv(x_nonzero)
assert torch.equal(y_nonzero[0, 2:], y[0, 2:])      # a 2..5 kimenet változatlan
assert not torch.equal(y_nonzero[0, :2], y[0, :2])  # a 0,1 kimenet változik
print(y_nonzero[0, :, 0, 0])
```

A mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; a súlyokat és a bemenetet explicit módon állítjuk be, ezért nincs véletlen és seed nem szükséges): `torch.Size([1, 6, 2, 2])`, `tensor([0., 0., 4., 4., 8., 8.], grad_fn=<SelectBackward0>)`, majd a nemnulla bemenet után `tensor([20., 20., 4., 4., 8., 8.], grad_fn=<SelectBackward0>)`. A súlyok ellenőrzése:

```
assert conv.weight.shape == (6, 1, 2, 2)
```

A második dimenzió 1, nem 3; ez a depthwise alak legfontosabb ellenőrzése.

14.4.6 Labor: csoportkonfiguráció, súlyszámlálás, hibás oszthatóság

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A szkript a szakasz kódblokkjából futtatható; külső adatfájl nem kell.

A labor célja négy konfiguráció paraméterszámának és kimeneti elemenkénti MAC-értékének összehasonlítása, a súlyalak ellenőrzése és egy oszthatósági hiba pontos megfogása. Az alábbi szkript a 14.3. Paraméterszám, MAC és aktivációmemória képletét használja, és a [D10] dokumentációban rögzített `in_channels`/`out_channels` oszthatósági feltételre épít.

```
# munkakönyvtár: lab/
import torch

def count_params(conv):
    return sum(p.numel() for p in conv.parameters())

def describe(name, cin, cout, k, groups):
    conv = torch.nn.Conv2d(cin, cout, (k, k), bias=True, groups=groups,
                           dtype=torch.float32)
    w = conv.weight
    expected_w = cout * (cin // groups) * k * k
    assert w.shape == (cout, cin // groups, k, k), w.shape
    assert expected_w + cout == count_params(conv)
    mac_one_output = (cin // groups) * k * k
    print(f"{name:22s} weight={tuple(w.shape)} params={count_params(conv):3d} "
          f"MAC/output={mac_one_output}")

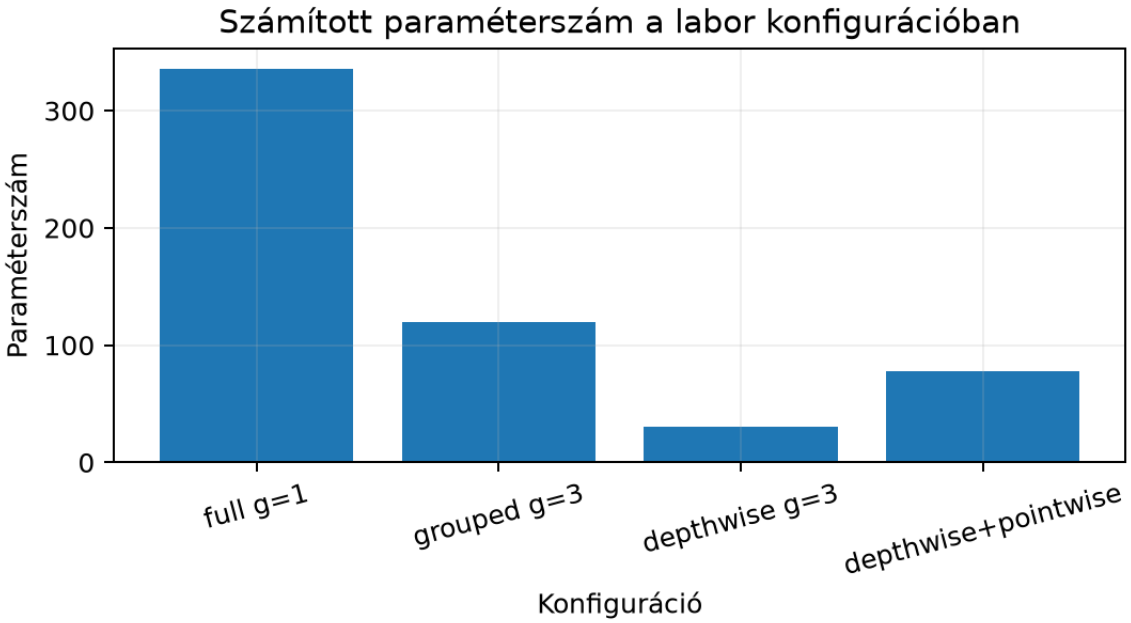
describe("full g=1",         3, 12, 3, 1)
describe("grouped g=3",      3, 12, 3, 3)
describe("depthwise g=3",    3,  3, 3, 3)
describe("pointwise 1x1",    3, 12, 1, 1)

try:
    torch.nn.Conv2d(5, 12, 3, groups=2)
except ValueError as e:
    print("hibas_groups:", type(e).__name__, "oszthatosagi feltetel serult")
```

A szkript mért kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; a súlyok inicializálása nem befolyásolja a számolt értékeket, seed nem szükséges):

Konfiguráció	Súlyalak	Paraméterek	MAC/kimeneti elem
full g=1	(12, 3, 3, 3)	336	27
grouped g=3	(12, 1, 3, 3)	120	9
depthwise g=3	(3, 1, 3, 3)	30	9
pointwise 1x1	(12, 3, 1, 1)	48	3

A depthwise réteg után egy $3 \to 12$ csatornás pointwise réteg összesen $30 + 48 = 78$ paramétert használ a teljes réteg 336 paraméteréhez képest. A MAC-érték kimeneti elemenként a full esetben $3 \cdot 9 = 27$ szorzás–összeadás pár (a bias hozzáadása nélkül), a grouped esetben $1 \cdot 9 = 9$; a bias a MAC-ban nem szerepel, csak a FLOP-átszámításban. A pointwise utótag hozzáadását a 19.1. Depthwise separable és pointwise réteg végzi el teljes költségmodellként. A hibás oszthatóság próbája a mért futásban a `hibas_groups: ValueError oszthatosagi feltetel serult` sort írja ki.



14.1. ábra: Számított paraméterszám a labor konfigurációban

A négy konfiguráció paraméterszámát a labor `assert`-jei ellenőrzik; a diagram maga számított szemléltetés, nem mérés.

A hibás oszthatóság ellenpéldája: $C_{in} = 5$, $g = 2$ esetén $2 \nmid 5$, ezért a konstruktor `ValueError` kivételt vet. Ugyanígy sérül a feltétel, ha C_{out} nem osztható. A rögzített környezetben (torch 2.15.0+cpu) a kivétel szövege `in_channels must be divisible by groups`; más verziókban ez a szöveg eltérhet, ezért a labor a kivétel típusát és az oszthatósági feltételt ellenőrzi, nem a konkrét üzenetet.

14.4.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a `groups=1`, a `groups=C_in` és egy közbülső `groups=2` konfiguráció között a csatornák kapcsolatában. Adjon kis ellenpéldát arra, hogy a `groups` nem értelmezhető, ha $g \nmid C_{in}$ vagy $g \nmid C_{out}$. Elvárt eredmény: definíciók, egy-egy konkrét csatornahalmaz-példa és az oszthatósági hiba oka.

F2 (levezetési/bizonyítási, nehézség 2). Legyen $C_{in} = 6$, $C_{out} = 6$, $g = 3$, kernel 3×3, input 4×4, padding 0, stride 1. Adja meg a csoportonkénti be- és kimeneti csatornahalmazokat, a súly alakját, a paraméterszámot, és a csatorna-major lapításban az L mátrix blokkméreteit. Írja fel a középső csoport második kimeneti csatornájának egyetlen kimeneti elemére a teljes összegképletet. Elvárt eredmény: halmazok, alakok, blokkméretek és a képlet minden indexhatárral.

F3 (implementációs, nehézség 2). Írjon PyTorch-szkriptet, amely a fenti depthwise kézi példát ellenőrzi `torch.allclose` hívással, megszámolja a paramétereket egy `groups=1` és egy `groups=3` rétegnél, és próbaként megkísérli a $C_{in} = 5$, $g = 2$ konstrukciót, elkapva a `ValueError`-t. Elvárt eredmény: futtatható kód, elvárt kimenet és tolerancia.

F4 (hibakeresési, nehézség 2). Egy fejlesztő a korábbi NumPy referencia `conv2d` függvényét hívja `groups=2` argumentummal. A függvény csak `groups=1`-et támogat, és a paramétert csendben figyelmen kívül hagyja. Hol van a hiba, mi a következménye, és milyen teszt fedné fel azonnal? Elvárt eredmény: a hiba oka, egy olyan input, amelyen a helyes és a hibás kimenet különbözik, és a javítás módja.

F5 (kísérleti/transzfer, nehézség 2). Tervezzen GlyphLab-kísérletet (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték), amely egy teljes 3×3-as konvolúciót hasonlít össze egy depthwise separable réteggel azonos kimeneti csatornaszámon. Adja meg a kérdést, a megfigyelési egységet, a splitet, a baseline-t, a változó tényezőt, a metrikát és a következtetés korlátját. Elvárt eredmény: protokoll, legalább két lehetséges kimenetel értelmezése; előre rögzített győztest ne állítson. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

14.4.8 Megoldások és indoklás

F1. `groups=1` esetén egyetlen csoport van: mind a C_{in} bemeneti csatorna összekötött mind a C_{out} kimeneti csatornával. `groups=C_in` esetén $r_{in} = 1$, a q-edik kimeneti csoport csak a q bemeneti csatornát látja; ez a depthwise eset. Ha $g = 2$ és $C_{in} = 4$, $C_{out} = 6$, akkor a 0 csoport az $\{0, 1\}$ bemeneti és $\{0, 1, 2\}$ kimeneti csatornát, az 1 csoport a $\{2, 3\}$ bemeneti és $\{3, 4, 5\}$ kimeneti csatornát kapja. A 0 bemeneti csatorna nem hathat a 3 kimeneti csatornára. Az oszthatóság azért kell, mert a csoportoknak egyenlő méretű partíciót kell alkotniuk; $C_{in} = 5$, $g = 2$ esetén nem tudunk két egyenlő, diszjunkt csatornahalmazt képezni. Ugyanez a kimeneti oldalon is fennáll.

F2. $r_{in} = r_{out} = 6/3 = 2$. A csoportok: $q = 0$: be $\{0, 1\}$, ki $\{0, 1\}$; $q = 1$: be $\{2, 3\}$, ki $\{2, 3\}$; $q = 2$: be $\{4, 5\}$, ki $\{4, 5\}$. A súly alakja $(6, 2, 3, 3)$; a tanulható paraméterszám $6 \cdot 2 \cdot 9 + 6 = 114$. A kimeneti méret $H_{out} = W_{out} = 2$, a lapított input mérete $6 \cdot 16 = 96$, a lapított output mérete $6 \cdot 4 = 24$. Az L mátrix 24×96-os, három blokkal; minden blokk $r_{out} H_{out} W_{out} = 8$ sor és $r_{in} HW = 32$ oszlop méretű, tehát $B^{(q)} \in \mathbb{R}^{8 \times 32}$. A középső $q = 1$ csoport második kimeneti csatornája $o = 3$, a bemeneti csatornák $c \in \{2, 3\}$. Padding nélkül, $s_h = s_w = 1$ mellett egy (i, j) kimeneti elem:

$$Y[n, 3, i, j] = \sum_{c=2}^{3} \sum_{u=0}^{2} \sum_{v=0}^{2} X[n, c, i + u, j + v]\, K[3,\, c - 2,\, u,\, v] + b[3].$$

A csoporton belüli második súlyindex mindig $c - q r_{in} = c - 2$.

F3. A megoldás a 14.4.5 kézi példa kódját használja fel újra a depthwise ellenőrzéshez, kiegészítve a paraméterszámlálással és a hibás oszthatóság próbájával. A teljes megoldás:

```
# munkakönyvtár: lab/
import torch

conv = torch.nn.Conv2d(3, 6, (2, 2), padding=0, groups=3, bias=False,
                       dtype=torch.float32)
with torch.no_grad():
    conv.weight.fill_(1.0)
```

```
x = torch.zeros(1, 3, 3, 3, dtype=torch.float32)
for c in range(3):
    x[0, c].fill_(float(c))
y = conv(x)
expected = torch.tensor([[[[0., 0.], [0., 0.]],
                          [[0., 0.], [0., 0.]],
                          [[4., 4.], [4., 4.]],
                          [[4., 4.], [4., 4.]],
                          [[8., 8.], [8., 8.]],
                          [[8., 8.], [8., 8.]]]])
assert y.shape == (1, 6, 2, 2)
assert torch.allclose(y, expected, atol=1e-6)

full = torch.nn.Conv2d(3, 6, 3, groups=1, bias=True, dtype=torch.float32)
grouped = torch.nn.Conv2d(3, 6, 3, groups=3, bias=True, dtype=torch.float32)
n_full = sum(p.numel() for p in full.parameters())
n_grp = sum(p.numel() for p in grouped.parameters())
print(n_full, n_grp, full.weight.shape, grouped.weight.shape)

try:
    torch.nn.Conv2d(5, 12, 3, groups=2)
except ValueError as e:
    print("elkapva", type(e).__name__)
```

A mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; a súlyokat és a bemenetet explicit módon állítjuk be, ezért seed nem szükséges): `168 60 torch.Size([6, 3, 3, 3]) torch.Size([6, 1, 3, 3])`, majd `elkapva ValueError`. A $3 \to 6$ full réteg paraméterszáma $6 \cdot 3 \cdot 9 + 6 = 168$, a grouped $6 \cdot 1 \cdot 9 + 6 = 60$. A tolerancia 10^{-6} bőven elég, mert a példa egész értékű szorzásokat tartalmaz, és az összegek float32-ben pontosan reprezentálhatók.

F4. A hiba oka: a referencia `conv2d` aláírásában szerepel a `groups` paraméter, de a függvény törzse nem használja, ezért csendben a teljes `groups=1` utat számolja. A hívó viszont grouped súlyt és csatornakiosztást feltételez; a kimenet a csoportok közötti nem kívánt keverést is tartalmazza. A következmény: hibás kísérleti eredmény, amely a tanulás során csak közvetve, például váratlanul nagy kapacitásként vagy rossz általánosításként jelenik meg. Azonnali teszt: depthwise cél, $C_{in} = 2$, $m = 1$, kernel 2×2 csupa egyes, bemenet $X[0,0] = 1$, $X[0,1] = 2$ konstans mezőkön. A helyes depthwise kimenet csatornánként 4 és 8; a hibás teljes konvolúció mindkét kimeneti csatornára 12-t adna, mert mindkét bemeneti csatornát összegzi. Javítás: a függvényben explicit

```
if groups != 1:
    raise NotImplementedError("a referencia jelenleg csak groups=1-et tamogat")
```

ellenőrzés, és grouped teszt esetén a PyTorch `Conv2d` mint független kontroll használata. A keretmegállapodás szerint a saját NumPy-referenciát nem szabad teljes grouped implementációnak nevezni.

F5. Kérdés: azonos kimeneti csatornaszám mellett a depthwise separable blokk eléri-e a teljes 3×3-as konvolúció validációs pontosságát a GlyphLab orientációfeladaton, miközben kevesebb paramétert használ? *Adat és szerződés:* a GlyphLab a függelék A. szakasza (40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) szerint 180 jelenetből jelenetenként két zajos nézetet rögzít, összesen 360 képet; a kép $1 \times 24 \times 24$, `float32`, értékkészlete $[0, 1]$; az osztályok a függőleges, vízszintes és átlós orientáció (3 osztály). A split jelenetszintű: 120/30/30 jelenet, azaz 240/60/60 kép, osztályonként 40/10/10 jelenet; a megfigyelési egység a `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik. A teszt két változatban szerepel: `clean` és zajjal rontott `corrupt`; a modellválasztás csak a val spliten történik, a tesztet a végleges modell záróértékelésére tartjuk fenn. *Modellek:* mindkét háló első rétege teljes 3×3-as `Conv2d(1, C,`

3) (a GlyphLab bemenete egycsatornás, ezért az első rétegben a depthwise és a teljes konvolúció függvényosztálya azonos lenne), majd globális átlagpooling és lineáris fej. A baseline további rétegei teljes 3×3-as konvolúciók; a kezelt modellben a második rétegtől depthwise(3×3) + pointwise(1×1) blokk áll, azonos C kimeneti csatornaszámmal. Így az egyetlen változó tényező a csatornakapcsolat korlátozása, nem az első réteg egycsatornás sajátossága. *Metrika és döntési szabály:* val accuracy és val loss; modellválasztás kizárólag val loss alapján, legjobb checkpoint. A két modell val loss-különbségére (teljes mínusz depthwise separable) jelenetszintű páros bootstrap 95%-os CI-t számolunk. Döntési szabály: ha a CI teljes egészében 0 alatt van, a teljes konvolúció jobb; ha teljes egészében 0 felett van, a depthwise separable jobb; ha a CI tartalmazza a 0-t, a két modell között ezen az adaton nincs kimutatható különbség. Záróteszt a rögzített `clean` és `corrupt` teszten, ugyanezzel a páros CI-vel; legalább három seed, a seed-szórás a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint kezelve. *Lehetséges kimenetelek:* ha a depthwise separable val lossa nem marad el (a CI nem mutat pozitív különbséget), a korlátozott csatornakapcsolat ezen a feladaton elegendőnek bizonyul; ha a CI a teljes konvolúció javára szól, az adott feladaton a korlátozás hátrányos lehet. Ez utóbbi nem bizonyítja, hogy más feladaton is teljes csatornakeverésre lenne szükség; a következtetés csak a GlyphLabra érvényes. *Korlát:* a 24×24-es szintetikus adat nem természetes képi benchmark, így az eredmény nem vihető át más feladatra.

14.4.9 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés az, hogy a `groups` értéket csupán sebességkapcsolónak tekintik. Ez két irányban hibás. A paraméterszám valóban csökken, de a függvényosztály is szűkül: a fenti depthwise példában a 0 bemeneti csatorna semmilyen kernelérték mellett sem hathat a 3 kimeneti csatornára, mert a mátrix megfelelő blokkja szerkezetileg zérus. Ez nem optimalizálási, hanem reprezentációs korlát.

A második tévedés a saját NumPy-referencia túlértékelése. Ha a `conv2d` referencia `groups=1`-re íródott, és valaki `groups=2` hívással használja, akkor a függvény vagy hibát vet, vagy ami rosszabb, csendben teljes konvolúciót számol. Az utóbbi nem grouped implementáció, hanem hibás hívás. Az ellenőrzés módja a fenti F4-teszt: olyan input, amelyen a csoportok közötti keverés jelenléte egyetlen skalárban látszik.

14.4.10 Önellenőrzés és visszahívás

Ellenőrző lista: a súly második dimenziója C_{in}/g, nem C_{in}; a bias C_{out} elemű; a csoportindex az output oldalon $\lfloor o/r_{out} \rfloor$; a kernel második indexe csoporton belüli, ezért $c - qr_{in}$ szerepel; a depthwise esetben $g = C_{in}$ és $C_{out} = mC_{in}$. A labor `assert` ellenőrzi a súlyalakot és a paraméterszámot.

Visszahívás 13.4. Határkezelés és frekvenciaintuíció alfejezetből: a határfeltétel a csatornabontástól független; a valid és zero padding ugyanúgy érvényes minden csoporton belül, mert a group csak a csatornák közötti kapcsolatot korlátozza, a térbeli indexelést nem. 10.4. Tanítási lépés és tanuló MLP alfejezetből: a tanítási lépésben a backward a tényleges kapcsolatok mentén folyik; a zérus mátrixblokk miatt egy bemeneti csatorna gradiense a rétegen keresztül csak a saját csoportjának kimeneteitől származhat. 2.4. Kis tudományos szoftverprojekt alfejezetből: a labor explicit konfigurációt, invariáns-ellenőrzést és hibás inputot is tesztel, nem csupán egy API-hívást mutat be.

14.4.11 Rövid összefoglaló

A `groups=g` paraméter a be- és kimeneti csatornákat g diszjunkt csoportra osztja; a q-edik kimeneti csoport csak a q-edik bemeneti csoportot látja. A súly alakja $C_{out} \times (C_{in}/g) \times k_h \times k_w$, a paraméterszám $C_{out}(C_{in}/g)k_h k_w + C_{out}$. Csatorna-major lapításban a lineáris operátor mátrixa blokkdiagonális, g darab $r_{out}H_{out}W_{out} \times r_{in}HW$ blokkal; ezt az állítást bias nélkül mondtuk ki és bizonyítottuk. A depthwise eset $g = C_{in}$, a pointwise 1×1-es réteg térbeli keverés nélkül kever csatornát; kettejük kompozíciója a depthwise separable réteg. A hibás oszthatóság konstrukciós hibát ad, a hiba csendes elnyelése pedig hamis grouped eredményhez vezet.

14.4.12 Következő alkalmazás és források

A 15.1. Súly- és biasgradiens egység a konvolúciós backwardban a súly- és biasgradienst vezeti le. A grouped mátrixszerkezet ott gradiensmaszkként jelenik meg: a súlygradiens csak azokra a csatornapárokra nemnulla, amelyek ugyanabba a csoportba tartoznak, a bemeneti gradiens pedig csak a kapcsolt bemeneti csatornákra folyik vissza.

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

15 Konvolúciós backward és transzponált operátor

15.1 Súly- és biasgradiens

15.1.1 Cél, előismeret és induló becslés

Ez az alfejezet a konvolúciós réteg két paraméterének, a kernelnek (szűrőnek) és a biasnak (eltolásnak) a gradiensét vezeti le a teljes NCHW keresztkorreláció (cross-correlation) definícióból. A központi jelenség a **weight sharing** (súlymegosztás): ugyanaz a kernel a kimenet minden térbeli pozíciójában szerepel (a 24×24-es bemenet példáján 484 helyen), ezért a gradiensük az összes felhasználási hely lokális gradiensének összege lesz [S01].

Előismeretként feltételezzük a 4.4. I. ellenőrző pont: matematika és kód megfelelése gráf-alapú láncszabályt, a 10.4. Tanítási lépés és tanuló MLP minibatch-gradiens átlagolását, a 13.4. Határkezelés és frekvenciaintuíció határkezelési konvenciókat és a 14.4. Csoportos és depthwise konvolúció csatornaösszegzési szerkezetet. Visszautalunk továbbá a 14.1. Stride, padding, dilation és outputméret kimeneti méretképletre, a 11.1. Tensor és autograd-életciklus autograd-életciklusra és a 3.1. Vektor, norma és skalárszorzat mátrix–vektor szorzás alakra.

Induló becslés. Egy 3×3-as kernel 1 bemeneti és 1 kimeneti csatornával, 24×24-es bemeneten, stride 1 és nulla padding mellett hány kimeneti pozícióban szerepel? Segítség: először a 14.1 kimeneti méretképletével számítsa ki a kimenet méretét, majd gondolja meg, hány ablakban fordul elő ugyanaz a kernel-elem. A gradiens mindegyik ilyen ablak hozzájárulását összegzi, ezért a válasz a kimeneti pozíciók száma.

15.1.2 Definíció és intuíció

Tekintsük a teljes NCHW cross-correlationt. A bemeneti tensor

$$X \in \mathbb{R}^{N \times C_{in} \times H \times W},$$

a kernel

$$K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w},$$

a bias $b \in \mathbb{R}^{C_{out}}$, és a kimenet

$$Y \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}.$$

A H_{out} és W_{out} értékét a 14.1. Stride, padding, dilation és outputméret alfejezetben levezetett

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1$$

formula adja, és analóg módon W_{out}. Itt p a szimmetrikus zero padding, s a stride, d a dilation. Ebben az alfejezetben $d = 1$-et feltételezünk, hacsak mást nem mondunk; a $d \neq 1$ esetre a 15.4. Független gradiensellenőrzési protokoll ad explicit indexelt képletet.

Az output definíciója minden $n \in \{0, \dots, N-1\}$, $o \in \{0, \dots, C_{out} - 1\}$, $i \in \{0, \dots, H_{out} - 1\}$ és $j \in \{0, \dots, W_{out} - 1\}$ indexre:

$$Y[n, o, i, j] = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X[n, c, i \cdot s + u - p,\ j \cdot s + v - p] \cdot K[o, c, u, v] + b[o].$$

A p padding eltolás biztosítja a virtuális, nullával kiegészített tartomány indexelését. A kényelem kedvéért bevezetjük a paddingelt bemenet jelölést: $\tilde{X}[n, c, r, q]$ a paddingelt képet jelöli, ahol $r \in \{0, \dots, H + 2p - 1\}$ és $q \in \{0, \dots, W + 2p - 1\}$, és $\tilde{X}[n, c, r, q] = X[n, c, r - p, q - p]$ az eredeti koordinátákban (a $[0, H) \times [0, W)$ tartományon kívül 0). Ezzel az output tömörebben:

$$Y[n, o, i, j] = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \tilde{X}[n, c, i \cdot s + u,\ j \cdot s + v] \cdot K[o, c, u, v] + b[o].$$

A veszteség és az upstream gradiens. Legyen L egy skalár loss, amely differenciálható függvénye az Y kimenetnek (például a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezetbeli $\frac{1}{2}\sum Y^2$ vagy egy softmax–keresztentrópia veszteség). A visszaterjesztés során ismert az **upstream gradiens** (a loss felől a réteg kimenetére már megérkezett, felfelé haladó gradiens):

$$\nabla_Y L \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}},$$

ahol az általános elem $\nabla_Y L[n, o, i, j] = \frac{\partial L}{\partial Y[n, o, i, j]}$. A cél a

$$\nabla_K L \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}, \qquad \nabla_b L \in \mathbb{R}^{C_{out}}$$

kiszámítása. A láncszabály szerint minden paraméter-elemre:

$$\frac{\partial L}{\partial K[o, c, u, v]} = \sum_{\substack{\text{minden } n,i,j \\ \text{ahol } K[o,c,u,v] \text{ szerepel}}} \frac{\partial L}{\partial Y[n, o, i, j]} \cdot \frac{\partial Y[n, o, i, j]}{\partial K[o, c, u, v]}.$$

Mivel Y lineáris a kernelben, a belső parciális derivált éppen az adott bemeneti pixel: $\tilde{X}[n, c, i \cdot s + u,\ j \cdot s + v]$. A bias esetén a parciális derivált 1.

15.1.3 Tétel: a kernel és a bias gradiensének levezetése

Tétel (súly- és biasgradiens a teljes NCHW cross-correlationhoz). Legyen $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ bemenet, $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$ kernel, $b \in \mathbb{R}^{C_{out}}$ bias, p szimmetrikus zero padding, s stride, $d = 1$ dilation, és legyen Y a fenti cross-correlation kimenete. Ha L differenciálható függvénye Y-nak, akkor

$$\frac{\partial L}{\partial K[o,c,u,v]} = \sum_{n=0}^{N-1}\sum_{i=0}^{H_{out}-1}\sum_{j=0}^{W_{out}-1} \nabla_Y L[n,o,i,j]\cdot \tilde{X}[n,c,i\cdot s+u,\ j\cdot s+v],$$

és

$$\frac{\partial L}{\partial b[o]} = \sum_{n=0}^{N-1}\sum_{i=0}^{H_{out}-1}\sum_{j=0}^{W_{out}-1} \nabla_Y L[n,o,i,j].$$

Ahol $\tilde{X}$ a paddingelt bemenetet jelöli.

Bizonyítás. A bizonyítás a 4.4. I. ellenőrző pont: matematika és kód megfelelése alfejezet láncszabályának tenzoros alakja. A $K[o,c,u,v]$ paramétertől a skalár lossig vezető minden út a $K[o,c,u,v] \to Y[n,o,i,j] \to L$ alakú: a paraméter csak a kimeneti tenzoron keresztül hat a lossra, és az o-adik kimeneti csatorna csak a kernel o-adik szeletét használja. Ezért a teljes differenciál:

$$\frac{\partial L}{\partial K[o,c,u,v]} = \sum_{n,o',i,j} \frac{\partial L}{\partial Y[n,o',i,j]}\cdot\frac{\partial Y[n,o',i,j]}{\partial K[o,c,u,v]}.$$

Az o' indexre a belső parciális derivált nulla, kivéve ha $o' = o$, mert az o-adik kimeneti csatorna csak a kernel o-adik kimeneti szeletét használja. Ha $o' = o$, akkor a definícióból közvetlenül:

$$\frac{\partial Y[n,o,i,j]}{\partial K[o,c,u,v]} = \tilde{X}[n,c,i\cdot s+u,\ j\cdot s+v].$$

Ezt behelyettesítve a kívánt formula adódik. A bias esetén $\frac{\partial Y[n,o,i,j]}{\partial b[o]} = 1$, és $\frac{\partial Y[n,o',i,j]}{\partial b[o]} = 0$ ha $o' \neq o$, így

$$\frac{\partial L}{\partial b[o]} = \sum_{n=0}^{N-1}\sum_{i=0}^{H_{out}-1}\sum_{j=0}^{W_{out}-1} \nabla_Y L[n,o,i,j].$$

Ez befejezi a bizonyítást.

Intuíció: a batch- és térösszegzés (batch and spatial sum) elve. A kernel gradiensformulája három összegzést tartalmaz: egyet a batch indexre, kettőt a térbeli kimeneti indexekre. Ez a **batch and spatial sum** (batch- és térösszeg) elv: minden felhasználási helyen az upstream gradiens és az adott bemeneti pixel szorzatát képezzük, és ezeket összeadjuk. A bias gradiensében nincs pixel-szorzó, csak az upstream gradiens tér- és batchösszege áll.

Összeadott, nem átlagolt gradiensek. A fenti levezetés lineáris rétegként kezeli a konvolúciót, ahol a gradiens a láncszabály szerint összegzés. Nincs benne osztás a felhasználási helyek számával. A minibatch-gradiens átlaga csak a loss definíciójából származik: ha $L = \frac{1}{M}\sum_{m=1}^{M} L_m$, akkor a batch-átlag $\frac{1}{M}$ faktora az upstream gradiensben jelenik meg, nem a kernel gradiensformulájában.

Ezt a különbséget a 10.4. Tanítási lépés és tanuló MLP minibatch-átlagolása és a jelen alfejezet tisztán összegző láncszabálya között fontos megőrizni.

15.1.4 Kézi kis példa

Tekintsünk egy $N = 1$, $C_{in} = 1$, $H = W = 3$ bemenetet, 3×3-as kernelt, $k_h = k_w = 3$, $p = 0$ paddinget és $s = 1$ stride-ot. Ekkor $H_{out} = W_{out} = 1$, tehát éppen egyetlen térbeli kimeneti pozíció van. A kernel minden eleme pontosan egyszer szerepel, így ez a példa a legegyszerűbb eset: nincs összegzés a térbeli használatok fölött.

Legyen

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix},$$

és $b = 0$. A kimenet egyetlen skalár:

$$Y[0,0,0,0] = 1 \cdot 1 + 2 \cdot 0 + 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 + 3 \cdot 0 + 2 \cdot 0 + 0 \cdot 0 + 1 \cdot (-1) = 1.$$

Mivel egyetlen kimeneti elem van, az upstream gradiens is skalár. Válasszuk a veszteséget $L = 2Y$-nak, így $\nabla_Y L[0,0,0,0] = 2$ (konstans, nem függ Y értékétől). A tétel szerint:

$$\frac{\partial L}{\partial K[0,0,u,v]} = 2 \cdot X[0,u,v].$$

Tehát $\nabla_K L$ éppen $2X$. A bias gradiens $\nabla_b L[0] = 2$.

Numerikus ellenőrzés véges differenciával. A $\frac{\partial L}{\partial K[u,v]}$ középső differenciás közelítése:

$$\frac{L(K + \varepsilon E_{u,v}) - L(K - \varepsilon E_{u,v})}{2\varepsilon}.$$

Ahol $E_{u,v}$ az a mátrix, amelynek az (u,v) eleme 1, a többi 0. Az $(u,v) = (2,2)$ perturbációval $K[2,2] = -1 \pm \varepsilon$ esetén $Y = 1 \pm \varepsilon X[2,2] = 1 \pm \varepsilon$ (mert $X[2,2] = 1$), tehát $L = 2 \pm 2\varepsilon$, és a középső differencia

$$\frac{(2 + 2\varepsilon) - (2 - 2\varepsilon)}{2\varepsilon} = 2 = 2\,X[2,2].$$

A $(0,1)$ elemre $X[0,1] = 2$, ezért a perturbáció Y-t $\pm 2\varepsilon$-nal változtatja, $L = 2Y$ miatt a különbség 8ε, a hányados $4 = 2\,X[0,1]$. A biasra $b = \pm\varepsilon$ mellett $Y = 1 \pm \varepsilon$, a hányados 2. A véges differencia mindegyik esetben egyezik a tétel képletével, mert az $L = 2Y$ upstream gradiens konstans 2; a nemlineáris $L = \frac{1}{2}Y^2$ veszteségnél ($b = 0$ esetén $Y = 1$, az upstream $\nabla_Y L = 1$) az upstream Y-tól függ, és a kernel véges differenciája $Y \cdot X[u,v]$-t adja (nem pusztán $X[u,v]$-t).

Nemlineáris veszteség. Ha az $L = \frac{1}{2}Y^2$ veszteséget használjuk, a gradiens az Y értékétől függ: $Y = 1$ mellett az upstream $\nabla_Y L = Y = 1$, tehát $\nabla_K L = X$ és $\nabla_b L = 1$. Fontos, hogy a teszt ne olyan pontban történjen, ahol a forward kimenete nulla: az $Y = 0$ pontban a $\frac{1}{2}Y^2$ gradiense nulla, és a középső differencia nem tesztelne semmit. Ha például $b = 5$ ($Y = 6$, $L = 18$), akkor $\nabla_b L = 6$, $\nabla_K L = 6X$, és a bias középső differenciája $\frac{\frac{1}{2}(6+\varepsilon)^2 - \frac{1}{2}(6-\varepsilon)^2}{2\varepsilon} = 6$.

15.1.5 Kódos munkamenet

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `weight_grad_check.py` a `lab/n057/` munkakönyvtárból futtatható (`python weight_grad_check.py`); külső adatfájl nem kell.

A következő kód a levezetés numerikus ellenőrzése a fenti 3×3-as példán és egy nagyobb eseten ($N = 2$, $C_{in} = 2$, $C_{out} = 2$, 4×4-es input). A munkakönyvtár `lab/n057/`; a PyTorch-autograd ellenőrzéshez a `torch` csomag szükséges. A PyTorch `conv2d` ugyanezt a keresztkorrelációs konvenciót követi [D10].

```
# lab/n057/weight_grad_check.py
import numpy as np

def conv2d_forward(X, K, b, stride=1, pad=0):
    """NCHW cross-correlation, groups=1, dilation=1, float64."""
    N, C_in, H, W = X.shape
    C_out, C_in_k, kh, kw = K.shape
    assert C_in == C_in_k
    H_out = (H + 2 * pad - kh) // stride + 1
    W_out = (W + 2 * pad - kw) // stride + 1
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad)))
    Y = np.zeros((N, C_out, H_out, W_out), dtype=np.float64)
    for n in range(N):
        for o in range(C_out):
            for i in range(H_out):
                for j in range(W_out):
                    acc = b[o]
                    for c in range(C_in):
                        for u in range(kh):
                            for v in range(kw):
                                acc += Xp[n, c, i * stride + u,
                                          j * stride + v] * K[o, c, u, v]
                    Y[n, o, i, j] = acc
    return Y

def conv2d_weight_grad(X, K, dY, stride=1, pad=0):
    """Weight and bias gradient from the derived theorem (b nem kell hozzá)."""
    N, C_in, H, W = X.shape
    C_out, _, kh, kw = K.shape
    H_out = dY.shape[2]
    W_out = dY.shape[3]
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad)))
    dK = np.zeros_like(K, dtype=np.float64)
    db = np.zeros(C_out, dtype=np.float64)
    for o in range(C_out):
        for c in range(C_in):
            for u in range(kh):
                for v in range(kw):
                    total = 0.0
                    for n in range(N):
                        for i in range(H_out):
                            for j in range(W_out):
                                total += dY[n, o, i, j] * Xp[
                                    n, c, i * stride + u, j * stride + v]
                    dK[o, c, u, v] = total
        db[o] = np.sum(dY[:, o, :, :])
    return dK, db

def finite_diff_check(X, K, b, eps=1e-5, stride=1, pad=0):
    """Central difference for one scalar loss L = 0.5 * sum(Y ** 2)."""
```

```
    C_out, kh, kw = K.shape[0], K.shape[2], K.shape[3]

    def loss(K, b):
        Y = conv2d_forward(X, K, b, stride, pad)
        return 0.5 * np.sum(Y ** 2)

    dK_num = np.zeros_like(K, dtype=np.float64)
    for o in range(C_out):
        for c in range(K.shape[1]):
            for u in range(kh):
                for v in range(kw):
                    Kp = K.copy(); Km = K.copy()
                    Kp[o, c, u, v] += eps
                    Km[o, c, u, v] -= eps
                    dK_num[o, c, u, v] = (loss(Kp, b) - loss(Km, b)) / (2 * eps)
    db_num = np.zeros(C_out, dtype=np.float64)
    for o in range(C_out):
        bp = b.copy(); bm = b.copy()
        bp[o] += eps; bm[o] -= eps
        db_num[o] = (loss(K, bp) - loss(K, bm)) / (2 * eps)
    return dK_num, db_num

if __name__ == "__main__":
    rng = np.random.default_rng(20261020)
    # 3x3 single-position case
    X1 = np.array([[[[1.0, 2.0, 1.0],
                     [0.0, 1.0, 3.0],
                     [2.0, 0.0, 1.0]]]])
    K1 = np.array([[[[1.0, 0.0, 0.0],
                     [0.0, 1.0, 0.0],
                     [0.0, 0.0, -1.0]]]])
    b1 = np.array([5.0])
    Y1 = conv2d_forward(X1, K1, b1, 1, 0)
    dY1 = Y1  # since upstream for L = 0.5*sum(Y**2) is Y
    dK1, db1 = conv2d_weight_grad(X1, K1, dY1, stride=1, pad=0)
    dK1_num, db1_num = finite_diff_check(X1, K1, b1, stride=1, pad=0)
    assert np.allclose(dK1, dK1_num, atol=1e-8, rtol=1e-5), (dK1, dK1_num)
    assert np.allclose(db1, db1_num, atol=1e-8, rtol=1e-5), (db1, db1_num)
    print("3x3 case: dK =", dK1[0, 0].ravel().tolist(), "db =", db1.ravel().tolist())

    # Független PyTorch-autograd ellenőrzés ugyanazon a 3x3-as eseten (float64)
    import torch
    Xt = torch.tensor(X1, dtype=torch.float64, requires_grad=True)
    Kt = torch.tensor(K1, dtype=torch.float64, requires_grad=True)
    bt = torch.tensor(b1, dtype=torch.float64, requires_grad=True)
    Yt = torch.nn.functional.conv2d(Xt, Kt, bt, stride=1, padding=0)
    (0.5 * (Yt ** 2).sum()).backward()
    assert np.allclose(Kt.grad.numpy(), dK1, atol=1e-10)
    assert np.allclose(bt.grad.numpy(), db1, atol=1e-10)
    print("PyTorch-autograd egyezés: OK")

    # 4x4 input, N=2, C_in=2, C_out=2, kernel 3x3, stride 1, no padding
    X2 = rng.normal(size=(2, 2, 4, 4))
    K2 = rng.normal(size=(2, 2, 3, 3))
    b2 = rng.normal(size=(2,))
    Y2 = conv2d_forward(X2, K2, b2, 1, 0)
    dY2 = Y2
    dK2, db2 = conv2d_weight_grad(X2, K2, dY2, stride=1, pad=0)
    # Ugyanazzal a bemenettel, kernellel és biasszal ellenőrzünk:
    dK2_num, db2_num = finite_diff_check(X2, K2, b2, stride=1, pad=0)
    assert np.allclose(dK2, dK2_num, atol=1e-8, rtol=1e-5)
    assert np.allclose(db2, db2_num, atol=1e-8, rtol=1e-5)
    print("4x4 case: dK shape", dK2.shape, "db shape", db2.shape, "OK")
```

Futtatás:

```
cd lab/n057
python weight_grad_check.py
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed `20261020`):

```
3x3 case: dK = [6.0, 12.0, 6.0, 0.0, 6.0, 18.0, 12.0, 0.0, 6.0] db = [6.0]
PyTorch-autograd egyezés: OK
4x4 case: dK shape (2, 2, 3, 3) db shape (2,) OK
```

Az első esetben $b = 5$, tehát $Y = 1 + 5 = 6$, az upstream gradiens $L = \frac{1}{2}\sum Y^2$ mellett $Y = 6$, így $dK = 6X$ és $db = 6$ (egyetlen térbeli pozíció van); a kód a $6X$ mátrix soronkénti lapítását és a `db = [6.0]` értéket írja ki. A második esetben dK alakja $(2, 2, 3, 3)$, azaz $(C_{out}, C_{in}, k_h, k_w)$, db alakja pedig $(2,)$.

Megjegyzés a megosztott helyek számáról. A 4×4-es inputon $H_{out} = W_{out} = 2$, tehát minden kernel-elem 4 térbeli pozícióban szerepel (a $N = 2$ batch miatt összesen 8 hozzájárulás). A gradiens ezek összege: az `u` és `v` hurkokon belüli `total` változó pontosan ezt a `batch and spatial sum`-ot számolja. Az egyezést az `assert np.allclose(..., atol=1e-8, rtol=1e-5)` ellenőrzi, tehát az abszolút tűrés 10^{-8}, a relatív 10^{-5}; f64-ben a központi differencia hibája várhatóan jóval e tűrés alatt marad. A `np.pad` alapértelmezésben konstans nulla paddinget ad, ami megegyezik a szimmetrikus zero padding konvencióval.

15.1.6 Önellenőrzés és gyakori tévedés

Gyakori tévedés. Azt gondolni, hogy a kernel gradiensét a felhasználási helyeken átlagolni kell, mert a minibatch-loss átlagot tartalmaz. Ez keveri a loss definíciójából származó $\frac{1}{M}$ faktort a láncszabály összegzési szerkezetével.

Ellenpélda. Legyen a loss $L = \frac{1}{2}\sum_{i,j} Y[0,0,i,j]^2$, és legyen minden bemeneti pixel 1, a kernel minden eleme 1, a bias 0. Ekkor a 3×3-as bemeneten egyetlen térbeli pozíció van, itt $Y = \sum_{u,v} 1 \cdot 1 = 9$; mivel egyetlen felhasználás van, az átlag és az összeg megegyezik, ezen a méreten a hiba nem mutatható ki. Vegyünk ezért stride 1-gyel egy 4×4-es bemenetet: $H_{out} = W_{out} = 2$, tehát négy pozíció, mindegyik $Y = 9$. Az upstream gradiens 9. A helyes kernel-gradiens: $9 \cdot 4$ minden bemeneti pixelre, azaz 36. Ha valaki osztana a pozíciók számával, 9-et kapna, ami téves. A helyes gondolat: minden kimeneti pozícióban a láncszabály a $\nabla_Y L \cdot X$ szorzatot adja, és ezek a hozzájárulások **összeadódnak**.

Visszatekintés: 11.1. Tensor és autograd-életciklus. A PyTorch `backward` hívása a `.grad` tárolóba összead, nem felülír. Ez pontosan ugyanaz az összegzési szemlélet: a kernel gradiensének minden felhasználási helyről érkező hozzájárulása akkumulálódik. Ugyanezért kell a következő backward hívás előtt `optimizer.zero_grad()`-ot (vagy `param.grad = None`-ot) futtatni: enélkül két egymást követő lépés gradiense összeadódna. Nem szabad továbbá a réteg saját backwardjában osztani a pozíciók számával.

Önellenőrző kérdések. (1) Miért összegződik a kernel gradiense a felhasználási helyek fölött? (2) Mit okoz, ha a tanítási ciklusban elmarad a `zero_grad()`? Kulcs: (1) mert ugyanaz a paraméter minden ablakban ugyanazt az értéket veszi fel, és a láncszabály az összes útvonal hozzájárulását összeadja; (2) a `.grad` felhalmozódik, ezért a frissítés a korábbi lépések gradiensét is tartalmazza.

15.1.7 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a weight sharing fogalmát a konvolúciós rétegben. Adjon egy konkrét példát, amelyben ugyanaz a kernel-elem két különböző térbeli pozícióban szerepel, és indokolja, miért nem tekinthetők ezek független paramétereknek. Elvárt eredmény: saját megfogalmazású definíció és egy konkrét index-példa két különböző térbeli pozícióra ($i_1 \neq i_2$ vagy $j_1 \neq j_2$).

F2 (levezetési, nehézség 2). Vezesse le a kernel gradiensét a fenti $\tilde{X}$ paddingelt bemenet jelöléssel, de most $s = 2$ stride esetén. Kérdés: megváltozik-e a formula szerkezete, vagy csak az indexelés? Elvárt eredmény: a teljes levezetés a láncszabályból, a stride explicit figyelembevételével, és a végső formula.

F3 (implementációs, nehézség 2). Írjon Python/NumPy függvényt, amely a kernel gradiensét a forward kimenetéből (Y, illetve az upstream gradiens) számítja ki, anélkül hogy a forward számítást újra meghívná. A bemenet $N = 2$, $C_{in} = 2$, $C_{out} = 3$, $H = W = 5$, $k_h = k_w = 3$, $s = 1$, $p = 0$. Ellenőrizze a kimeneti shape-et és egy véletlen seedű esetén a véges differenciás kontrollt. Elvárt beadandó eredmény: futtatható kód, shape-ellenőrzés és egyezési tűrés.

F4 (hibakeresési, nehézség 2). Egy hallgató a következő kódot írja a bias gradiensére:

```
db[o] = np.sum(dY[np.arange(N), o, :, :]) / (H_out * W_out)
```

Mi a hiba? Milyen következménye van a tanításra? Javítsa és ellenőrizze a tétel formulájával. Elvárt eredmény: a hiba oka, következménye és a javított kód.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy kis kísérletet, amely megméri, hogy a kernel gradiensének kiszámítása mennyivel drágább egy 1×1-es kernelre, mint egy 3×3-asra, azonos bemeneti méret és csatornaszám mellett. Milyen metrikát használna, mit tartana állandóan, és milyen korlátot fogalmazna meg az eredmény általánosításáról? Elvárt beadandó eredmény: írásos kísérleti protokoll az 1.2. Szakmailag megalapozott modellezési állítások szerinti elemekkel, de futtatás nélkül. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

15.1.8 Megoldások és indoklás

F1 megoldás. A weight sharing azt jelenti, hogy a kernel egyetlen $K[o, c, u, v]$ eleme a bemenet több térbeli ablakában ugyanazzal a numerikus értékkel szerepel a kimenet kiszámításában. Például az $s = 1$ stride-dal, $p = 0$ paddinggel vett 3×3-as kernel $K[0, 0, 0, 0]$ eleme az $(i, j) = (0, 0)$ és $(i, j) = (0, 1)$ kimeneti pozíciókban is a bemenet $(0, 0)$, illetve $(0, 1)$ pozíciójú pixelével szorzódik. A paraméterek azért nem függetlenek, mert a kimeneti pozíciók mindegyike ugyanazt a kernel-elemet használja; ha függetlenek lennének, a gradiensben nem összegzés, hanem különkülön frissítés jelenne meg, és megszűnne az eltolási ekvivariancia (equivariance) 16.1. Eltolási equivariance és invariance alfejezetbeli tulajdonsága.

F2 megoldás. A levezetés a láncszabályból indul. A kimenet definíciója $s = 2$ mellett

$$Y[n, o, i, j] = \sum_{c'=0}^{C_{in}-1} \sum_{u'=0}^{k_h-1} \sum_{v'=0}^{k_w-1} \tilde{X}[n, c', 2i + u', 2j + v'] \, K[o, c', u', v'] + b[o].$$

Rögzített (o, c, u, v) paraméterre az $Y[n, o', i, j]$ belső deriváltja $o' \neq o$ esetén 0; $o' = o$ esetén pedig $\tilde{X}[n, c, 2i + u, 2j + v]$, mert a szorzatban csak a $c' = c$, $u' = u$, $v' = v$ tag tartalmazza a kérdéses paramétert. A láncszabály szerint

$$\frac{\partial L}{\partial K[o, c, u, v]} = \sum_{n=0}^{N-1} \sum_{o'=0}^{C_{out}-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} \nabla_Y L[n, o', i, j] \, \frac{\partial Y[n, o', i, j]}{\partial K[o, c, u, v]} = \sum_{n=0}^{N-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} \nabla_Y L[n, o, i, j] \, \tilde{X}[n, c, 2$$

A formula szerkezete tehát nem változik meg: a kernel gradiens továbbra is az upstream gradiens és a paddingelt bemenet szorzatának batch- és térösszege. A stride kizárólag a mintavételi koordinátában jelenik meg: a $2i + u$, illetve $2j + v$ index helyettesíti az $s = 1$ eset $i + u$, $j + v$ indexét, és a kimeneti méret $H_{out} = \lfloor (H + 2p - k_h)/2 \rfloor + 1$. A levezetés minden más lépése azonos a tétel $s = 1$ esetével.

F3 megoldás. A gradiens a dY upstream gradiensből közvetlenül számolható, a forward újbóli meghívása nélkül. Az alábbi önálló függvény a $N = 2$, $C_{in} = 2$, $C_{out} = 3$, $H = W = 5$, 3×3-as kernel, $s = 1$, $p = 0$ esetre mutatja meg a teljes megoldást; a `dY[:, o]` és a paddingelt bemenet megfelelő ablakainak szorzata adja az összeget.

```
import numpy as np

def weight_grad_from_dY(dY, X, k, stride=1, pad=0):
    N, C_in, H, W = X.shape
    C_out = dY.shape[1]
    kh, kw = k
    H_out, W_out = dY.shape[2], dY.shape[3]
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad)))
    dK = np.zeros((C_out, C_in, kh, kw), dtype=np.float64)
    for o in range(C_out):
        for c in range(C_in):
            for u in range(kh):
                for v in range(kw):
                    window = Xp[:, c,
                                u:u + stride * (H_out - 1) + 1:stride,
                                v:v + stride * (W_out - 1) + 1:stride]
                    dK[o, c, u, v] = np.sum(dY[:, o] * window)
    return dK
```

A függvény a fejezet `conv2d_weight_grad` gradiensrészének önálló változata; a forwardot csak a teszt használja a Y és a $dY = Y$ előállításához, a gradiensfüggvény nem hívja meg újra. A mért ellenőrzés (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, `np.random.RandomState(20261020)`, $X \sim \mathcal{N}(0,1)^{2\times2\times5\times5}$, $K \sim \mathcal{N}(0,1)^{3\times2\times3\times3}$, $b \sim \mathcal{N}(0,1)^3$, $L = \frac{1}{2}\sum Y^2$):

```
dK shape: (3, 2, 3, 3) dY shape: (2, 3, 3, 3)
max abs diff vs autograd: 1.4210854715202004e-14
```

Három kiválasztott elemre a véges differencia is egyezik ($\varepsilon = 10^{-6}$): $(0,0,0,0)$: 4,42678469; $(1,1,1,2)$: $-9{,}81606289$; $(2,1,2,1)$: 5,86055112 (a képlet és a központi differencia legfeljebb $1{,}2 \cdot 10^{-8}$ eltéréssel). A dK alakja $(C_{out}, C_{in}, k_h, k_w) = (3, 2, 3, 3)$, a dY alakja $(N, C_{out}, H_{out}, W_{out}) = (2, 3, 3, 3)$.

F4 megoldás. A hiba a $H_{out} \cdot W_{out}$-tal való osztás. A tétel bizonyította, hogy a bias gradiens a batch- és térösszeg, nem pedig annak átlaga. A következmény: a visszaterjesztett biasgradiens a helyes érték $1/(H_{out}W_{out})$-szorosa, azaz rossz skálájú; nagyobb kimeneti méretnél nagyobb a hiba. Hogy ez mennyire lassítja a tanulást, az optimalizálótól függ: Adamnél a gradiens skálája lényegében kiesik, mert a lépést a momentumok aránya szabja meg; SGD-nél viszont a bias lépésköze arányosan csökken. A javított kód:

```
db[o] = np.sum(dY[:, o, :, :])
```

A visszaellenőrzés a tétel formulájával: a $\frac{\partial L}{\partial b[o]} = \sum_{n,i,j} \nabla_Y L[n,o,i,j]$ éppen a fenti kód, minden osztás nélkül.

F5 megoldásvázlat. A kísérleti kérdés: hogyan skálázódik a kernel gradiensének számítási költsége a kernelmérettel. Protokoll: (1) kérdés: a 1×1-es és 3×3-as kernel gradiensszámítási idejének aránya; (2) adat: szintetikus véletlen tensor, rögzített seed; (3) megfigyelési egység: egyetlen gradiensszámítás futása; (4) split: nem releváns, nincs tanulás; (5) baseline: az 1×1-es kernel futásideje; (6) változó tényező: a kernelméret; (7) metrika: falióra-idő (wall-clock) ismételt méréssel, melegítés után, valamint a lebegőpontos műveletszám, amelyre a `conv2d_weight_grad` belső ciklusa $NC_{out}C_{in}k_hk_wH_{out}W_{out}$ szorzást végez — 24×24-es bemeneten ez 3×3-as kernelre $9 \cdot 22 \cdot 22 = 4356$, 1×1-esre $1 \cdot 24 \cdot 24 = 576$ szorzás, azaz az arány 7,5625; (8) erőforrás: helyi CPU, f64; (9) korlát: az eredmény nem általánosítható GPU-ra, és a forward overhead miatt a tiszta gradiens-számítás időaránya nem azonos a teljes tanítási idővel.

15.1.9 Rövid összefoglaló

A konvolúciós réteg kernel- és biasgradiensét a teljes NCHW cross-correlation definícióból vezettük le a láncszabály alkalmazásával. A kernel gradiens minden súlyelemre a batch, a kimeneti magasság és a kimeneti szélesség szerinti összeg: az upstream gradiens és a megfelelő paddingelt bemeneti pixel szorzatát minden felhasználási helyen összeadjuk. A bias gradiens a batch és térbeli indexekre vett upstream-gradiens összeg. A weight sharing miatt a hozzájárulások összeadódnak, nem átlagolódnak. Kézi példán és NumPy-kódban véges differenciával ellenőriztük a formulát f64 tűrésekkel. Tipikus hiba a felhasználási helyek számával való osztás, ami a minibatch-loss átlagolásának félreértéséből származik.

15.1.10 Továbblépés

A 15.2. Inputgradiens és átfedő recepciós mezők egységben az input gradiensét vezetjük le; ott az átfedő recepciós mezők miatt a gradiensek hasonlóan összegződnek, de a kernel szerepe megfordul: nem paraméterként, hanem az inputgradiens scatter–add súlyaként jelenik meg.

15.1.11 Felhasznált források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

15.2 Inputgradiens és átfedő recepciós mezők

15.2.1 Cél, előismeret és induló kérdés

Ebben az egységben a konvolúciós réteg bemenet szerinti gradiensét, a dX tenzort vezetjük le, és megvalósítását a scatter-add művelettel ellenőrizzük. A 15.1. Súly- és biasgradiens egységben a kernel- és biasgradiens képletét már levezettük; most az ellenkező irányú kérdést tesszük fel. Induló becslésként gondoljuk végig egy 3×3-as kernel és stride 1 esetét: egy belső input pixel hány kimeneti elem kiszámításában vesz részt? Belső pixelre és $s = 1$ mellett a válasz $C_{out}k_hk_w$ darab, mert minden kimeneti csatorna ugyanazt a bemeneti pixelt látja a saját kernelével, így a

visszafelé terjedő gradiensből ugyanennyi hozzájárulást kell összegeznünk. Ha ezeket felülírjuk egymást, az inputgradiens hibás lesz.

Előismeret: a 14.2. Több bemeneti és kimeneti csatorna alfejezetben bevezetett többcsatornás forward definíció, a 15.1. Súly- és biasgradiens alfejezetben használt keresztkorrelációs indexelés, valamint a 3.2. Mátrixszorzás és affín réteg alfejezetből a mátrixszorzás inputgradiensének tranzponált szerkezete.

15.2.2 Definíció és intuíció

Legyen a bemenet $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ NCHW alakban, a kernel $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$, a bias $b \in \mathbb{R}^{C_{out}}$. A konvolúciós réteg kimenete $Y = \mathrm{Conv2d}(X; K, b)$, ahol a PyTorch `Conv2d` keresztkorrelációt számol [D10]. A veszteség Y szerinti gradiense legyen

$$D := \nabla_Y L \in \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}}.$$

A cél a

$$dX := \nabla_X L \in \mathbb{R}^{N\times C_{in}\times H\times W}$$

tensor kiszámítása. A forwardban minden kimeneti elem a bemenet egy $k_h \times k_w$-s kivágásából (patch-éből) készül; a backwardban ezért minden $D[n, o, i, j]$ elem a megfelelő kivágás (patch) minden pozíciójához hozzájárul a kernel súlyával szorozva. Ha s kisebb a kernel effektív kiterjedésénél $(d(k-1)+1)$, a szomszédos kimeneti elemek kivágásai (patch-ei) átfednek, és ugyanazon input pixel több hozzájárulást kap; $s \geq d(k-1)+1$ esetén nincs átfedés. A helyes művelet mindkét esetben **scatter-add**: a célkoordinátához tartozó gradienshez hozzáadunk, nem felülírunk. Az átfedő kivágások és a recepciós mező kapcsolatát [S01] tárgyalja.

A stride, padding és dilation szerepe a célkoordináta meghatározásában:

$$u = is + rd - p, \qquad v = js + td - p,$$

ahol i, j a kimeneti térbeli index, u, v a bemeneti térbeli index, $r \in \{0, \dots, k_h - 1\}$ és $t \in \{0, \dots, k_w - 1\}$ a kernel indexe, s a stride, p a padding, d a dilation. Csak a $0 \leq u < H$ és $0 \leq v < W$ tartományba eső pozíciók kapnak gradienst; a paddingelt virtuális pixelekhez tartozó hozzájárulások kimaradnak.

15.2.3 A nemnulla hozzájárulások indexfeltétele

A következő lemmában a teljes inputgradiens képletet mondjuk ki. A bizonyítás a forward definícióból és a láncszabályból áll, ezért a feltételeket expliciten megadjuk.

Lemma (a konvolúciós réteg inputgradiense). Legyen Y a 14.2. Több bemeneti és kimeneti csatorna szerinti keresztkorreláció $N, C_{in}, C_{out}, H, W, k_h, k_w > 0$ pozitív egészekkel, $p \geq 0$ egész paddinggel, $s \geq 1$ egész stride-dal és $d \geq 1$ egész dilationnel úgy, hogy a kimeneti méretek

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1, \qquad W_{out} = \left\lfloor \frac{W + 2p - d(k_w - 1) - 1}{s} \right\rfloor + 1$$

pozitívak. Ekkor minden $n \in \{0, \dots, N-1\}$, $c \in \{0, \dots, C_{in}-1\}$ és $0 \le u < H,\ 0 \le v < W$ esetén

$$dX[n,c,u,v] = \sum_{o=0}^{C_{out}-1} \sum_{r=0}^{k_h-1} \sum_{t=0}^{k_w-1} \sum_{\substack{i,j \in \mathbb{Z} \\ u = is+rd-p \\ v = js+td-p \\ 0 \le i < H_{out} \\ 0 \le j < W_{out}}} D[n,o,i,j]\, K[o,c,r,t].$$

A belső összegzés pontosan azokra az (i,j) párokra terjed ki, amelyekre az

$$i = \frac{u+p-rd}{s}, \qquad j = \frac{v+p-td}{s},$$

ahol $r \in \{0, \dots, k_h-1\}$ és $t \in \{0, \dots, k_w-1\}$ a kernel indexe.

értékek egészek és a kimeneti tartományba esnek. A dX többi eleme definíció szerint nulla; a paddingelt virtuális pixelekhez nincs dX-elem.

Bizonyítás. A forward definícióból, a paddingelt X_p tenzorral:

$$Y[n,o,i,j] = b[o] + \sum_{c'=0}^{C_{in}-1} \sum_{r=0}^{k_h-1} \sum_{t=0}^{k_w-1} X_p[n,c',\, is+rd,\, js+td]\, K[o,c',r,t],$$

ahol a paddingelt tenzor koordinátái 0-tól $H+2p-1$-ig (illetve 0-tól $W+2p-1$-ig) futnak, és $X_p[n,c',a,b] = X[n,c',a-p,b-p]$.

A láncszabály szerint

$$\frac{\partial L}{\partial X[n,c,u,v]} = \sum_{o=0}^{C_{out}-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} D[n,o,i,j] \frac{\partial Y[n,o,i,j]}{\partial X[n,c,u,v]}.$$

Mivel a bias X-től független, csak a keresztkorrelációs tag deriváltja marad. A parciális derivált:

$$\frac{\partial X_p[n,c',\, is+rd,\, js+td]}{\partial X[n,c,u,v]} = \begin{cases} 1, & \text{ha } c'=c,\ u = is+rd-p,\ v = js+td-p, \\ 0, & \text{különben,} \end{cases}$$

feltéve, hogy $0 \le u < H$ és $0 \le v < W$; ha a célkoordináta a paddingre mutat, a derivált nulla. Ezt behelyettesítve és a véges összegeket átrendezve a lemma képlete adódik. Az átrendezés lépései: (i) a c' szerinti összegből csak a $c'=c$ tag marad; (ii) az (i,j) párok helyett a (r,t) kernelindexek szerint csoportosítunk, és minden (r,t)-hez a feltétel által meghatározott (i,j) párokat vesszük; konkrétan minden rögzített (u,v,r,t) négyeshez pontosan akkor van hozzájárulás, ha $u+p-rd$ osztható s-sel, és az így kapott $i = (u+p-rd)/s$ a $[0, H_{out})$ intervallumba esik (a $j = (v+p-td)/s$ feltétel ugyanígy érvényesül); (iii) a $0 \le u < H$, $0 \le v < W$ feltétel kizárja a paddingre mutató célkoordinátákat. A véges összegek felcserélhetősége miatt az átrendezés indokolt. □

Következmény (scatter-add megvalósítás). A lemma nem az (u,v) bemeneti indexekre szervezett összegzést adja, hanem megfordítva: minden o kimeneti csatornára, minden $D[n,o,i,j]$ elemhez és minden (r,t) kernelpozícióhoz kiszámítjuk a célkoordinátát (u,v); ha az érvényes bemeneti tartományba esik, akkor minden c bemeneti csatornára

$$dX[n, c, u, v] \mathrel{+}= D[n, o, i, j]\, K[o, c, r, t].$$

A `+=` művelet az összeadás, nem a felülírás, és az o szerinti összegzés a scatter-add ciklusában implicit módon jelenik meg. Az átfedő patch-ek miatt ugyanazon (n, c, u, v) cél több forrásból is kap hozzájárulást, ezért az accumulation a helyes szemantika.

A stride hatása az indexfeltételben látszik: ha $u + p - rd$ nem osztható s-sel, akkor az adott (u, r) párhoz nincs i. Például $p = 0$, $s = 3$, $d = 1$, $k_h = 2$ esetén a $u \equiv 2 \pmod 3$ pozíciók semmilyen kernelpozícióval sem érhetők el, így azok dX-e a kernel súlyától függetlenül nulla marad.

15.2.4 Kézi kis példa

Legyen $N = 1$, $C_{in} = C_{out} = 1$, $H = W = 4$, $k_h = k_w = 3$, $p = 1$, $s = 1$, $d = 1$. A kernel legyen diagonális:

$$K = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

Az upstream gradiens tenzor is diagonális:

$$D = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}.$$

A lemma képletéből $p = 1$, $s = 1$, $d = 1$ miatt $i = u + 1 - r$, $j = v + 1 - t$. A diagonális kernel csak $r = t$ esetén ad nemnulla hozzájárulást, így

$$dX[u, v] = 1 \cdot D_1[u + 1, v + 1] + 2 \cdot D_1[u, v] + 3 \cdot D_1[u - 1, v - 1],$$

ahol D_1 a D tenzor kiterjesztése nullákkal a tartományon kívül. A diagonális elemekre:

- $dX[0, 0] = 1 \cdot D[1, 1] + 0 + 0 = 1$,
- $dX[1, 1] = 1 \cdot D[2, 2] + 2 \cdot D[1, 1] + 3 \cdot D[0, 0] = 2 + 2 = 4$,
- $dX[2, 2] = 1 \cdot D[3, 3] + 2 \cdot D[2, 2] + 3 \cdot D[1, 1] = 3 + 4 + 3 = 10$,
- $dX[3, 3] = 0 + 2 \cdot D[3, 3] + 3 \cdot D[2, 2] = 6 + 6 = 12$.

A többi elem nulla, mert a D off-diagonális elemei nullák. Tehát

$$dX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 12 \end{pmatrix}.$$

Megfigyelhető, hogy egy belső pixel több kimeneti gradiensből is részesül: például a $(2, 2)$ pixel — amely a 4×4-es rácson nem középpont, de belső pozíció — a $D[3, 3]$, $D[2, 2]$ és $D[1, 1]$ elemekből kap hozzájárulást a kernel megfelelő súlyával. A diagonális K és D a kézi számolást egyszerűsíti; off-diagonális esetben ugyanez az összegzés több tagot ad, a stride > 1 hatását pedig a következő labor nyolc konfigurációja mutatja, a `=` és `+=` különbségét az F1 példája számszerűsíti.

15.2.5 NumPy scatter-add és PyTorch autograd összehasonlítás

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `n058_dx_check.py` a `lab/` könyvtárból futtatható (`python n058_dx_check.py`); külső adatfájl nem kell.

Az alábbi labor a lemmát több stride/padding/dilation konfiguráció mellett ellenőrzi. A referenciakód a scatter-add műveletet a fenti következmény szerint valósítja meg (a kódban a lemma r, t indexe `a`,`b` néven szerepel); a PyTorch oldalon az `autograd` számítja a dX-et [D05]. A kimenet az eltérések maximuma.

```
# lab/n058_dx_check.py
import numpy as np
import torch
import torch.nn.functional as F

rng = np.random.default_rng(20261020)

def conv2d_output_shape(H, W, kh, kw, stride, padding, dilation):
    Hout = (H + 2 * padding - dilation * (kh - 1) - 1) // stride + 1
    Wout = (W + 2 * padding - dilation * (kw - 1) - 1) // stride + 1
    return Hout, Wout

def conv2d_input_grad_numpy(D, K, H, W, stride, padding, dilation):
    # D: (N, Cout, Hout, Wout), K: (Cout, Cin, kh, kw)
    N, Cout, Hout, Wout = D.shape
    Cout2, Cin, kh, kw = K.shape
    assert Cout2 == Cout
    dX = np.zeros((N, Cin, H, W), dtype=D.dtype)
    for n in range(N):
        for o in range(Cout):
            for i in range(Hout):
                for j in range(Wout):
                    du = D[n, o, i, j]
                    for c in range(Cin):
                        for a in range(kh):
                            for b in range(kw):
                                u = i * stride + a * dilation - padding
                                v = j * stride + b * dilation - padding
                                if 0 <= u < H and 0 <= v < W:
                                    dX[n, c, u, v] += du * K[o, c, a, b]
    return dX

N, Cin, Cout, H, W = 2, 3, 4, 6, 7
kh, kw = 3, 3

for stride in (1, 2):
    for padding in (0, 1):
        for dilation in (1, 2):
            Hout, Wout = conv2d_output_shape(H, W, kh, kw, stride, padding, dilation)
            if Hout <= 0 or Wout <= 0:
                continue
            X_np = rng.standard_normal((N, Cin, H, W)).astype(np.float64)
            K_np = rng.standard_normal((Cout, Cin, kh, kw)).astype(np.float64)
            b_np = rng.standard_normal(Cout).astype(np.float64)
            D_np = rng.standard_normal((N, Cout, Hout, Wout)).astype(np.float64)

            dX_np = conv2d_input_grad_numpy(D_np, K_np, H, W, stride, padding, dilation)

            X_t = torch.tensor(X_np, dtype=torch.float64, requires_grad=True)
            K_t = torch.tensor(K_np, dtype=torch.float64, requires_grad=True)
```

```
    b_t = torch.tensor(b_np, dtype=torch.float64, requires_grad=True)
    Y_t = F.conv2d(X_t, K_t, b_t, stride=stride, padding=padding, dilation=dilation)
    D_t = torch.tensor(D_np, dtype=torch.float64)
    loss = (Y_t * D_t).sum()
    loss.backward()
    dX_t = X_t.grad

    diff = np.max(np.abs(dX_np - dX_t.detach().numpy()))
    assert dX_np.shape == (N, Cin, H, W)
    assert diff < 1e-12, diff
    print(f"s={stride} p={padding} d={dilation} "
          f"Hout,Wout={Hout},{Wout} max_abs_diff={diff:.3e}")
```

A futtatás parancsa a `lab` munkakönyvtárból:

```
python n058_dx_check.py
```

A kimenet a fenti kód mért futása (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed 20261020); minden konfigurációban `max_abs_diff` $< 10^{-12}$, és a kód ezt `assert`-tel ellenőrzi:

s	p	d	$H_{out} \times W_{out}$	`max_abs_diff`
1	0	1	4×5	$5{,}329 \cdot 10^{-15}$
1	0	2	2×3	$8{,}882 \cdot 10^{-16}$
1	1	1	6×7	$5{,}329 \cdot 10^{-15}$
1	1	2	4×5	$2{,}665 \cdot 10^{-15}$
2	0	1	2×3	$1{,}776 \cdot 10^{-15}$
2	0	2	1×2	$8{,}882 \cdot 10^{-16}$
2	1	1	3×4	$2{,}665 \cdot 10^{-15}$
2	1	2	2×3	$1{,}776 \cdot 10^{-15}$

Három ellenőrizhető kulcsérték: a dX alakja minden esetben $(N, C_{in}, H, W) = (2, 3, 6, 7)$; a $s = 1, p = 0, d = 1$ eset kimeneti alakja $(2, 3, 4, 5)$; a $s = 2, p = 0, d = 2$ eseté $(2, 3, 1, 2)$. A `loss = (Y_t * D_t).sum()` használata a 15.1. Súly- és biasgradiens alfejezetben bevezetett ellenőrző konvenció: a skalár loss gradiense a megadott upstream tenzor, ezért nincs szükség tényleges veszteségfüggvényre.

15.2.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem elegendő a dX kiszámításakor az értékadás (`=`), és miért kell scatter-add-öt (`+=`) használni. Adjon egy konkrét 1D ellenpéldát: $H = 4$, $k = 3$, $p = 1$, $s = d = 1$, $K = [1, 1, 1]$, $D = [1, 2, 3, 4]$. Számítsa ki a helyes dX második elemét, és a hibás felülíró implementáció által adott értéket, ha a kód a kimeneti index (i) szerinti külső és a kernelpozíció (k) szerinti belső ciklussal dolgozik. Beadandó: szöveges indoklás és a két számérték.

F2 (levezetési, nehézség 2). Tekintse a $H = 5$, $W = 1$ esetet, ahol a magasság mentén $k_h = 3$, $p_h = 1$, $s = 2$, $d = 2$, a szélesség mentén pedig $k_w = 1$, $p_w = 0$ (tengelyenkénti padding). Határozza meg a lemmából, mely u bemeneti magasságindexekre lehet $dX[n, c, u, 0]$ nemnulla, és minden ilyen u-ra adja meg az összes hozzájáruló (i, r) párt. Beadandó: a H_{out} értéke, a lehetséges u-k halmaza, és a hozzájárulás-táblázat.

F3 (implementációs, nehézség 1). Írjon 1D NumPy referenciát a dX kiszámítására a lemmából, majd ellenőrizze PyTorch `F.conv1d` autograddal a $H = 5$, $C_{in} = 2$, $C_{out} = 2$, $k = 3$,

$s = 2$, $p = 1$, $d = 1$, $N = 1$ konfigurációra. Beadandó: teljes kód, seed, dtype, parancs, és a kapott `max_abs_diff`.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a scatter-add után a padding levágását próbálja elvégezni:

```
# HIBA: p=0 esetén a 0:-0 szelet üres.
dX = full[:, :, padding:-padding, padding:-padding]
```

Lokalizálja a hibát $p = 0$ esetén: miért üres a szelet, mi a következménye a dX alakjára és a numerikus összehasonlításra? Javítsa a kódot, és adja meg az ellenőrző teszt várt kimenetét. Beadandó: hiba magyarázata, javított kód, és a javítás utáni shape-ellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet annak mérésére, hogy a Python ciklusos scatter-add implementáció futásideje és memóriaigénye hogyan skálázódik a batchmérettel ($N \in \{1, 4, 16, 64\}$) kis CPU-n, rögzített $H = W = 8$, $C_{in} = C_{out} = 2$, $k = 3$, $s = 1$, $p = 1$ mellett. Nem kell ténylegesen futtatnia. Beadandó: hipotézis, mért mennyiségek, mérési protokoll, baseline, és az érvényességi korlát. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

15.2.7 Megoldások és indoklás

F1 megoldás. A forwardban az $u = 1$ bemeneti pixel három kimeneti elemhez járul hozzá: $Y[0]$-hoz a $k = 2$ kernelpozícióval, $Y[1]$-hez a $k = 1$ pozícióval, és $Y[2]$-höz a $k = 0$ pozícióval. A megfelelő gradiens-hozzájárulások:

$$D[0] \cdot K[2] = 1 \cdot 1 = 1, \quad D[1] \cdot K[1] = 2 \cdot 1 = 2, \quad D[2] \cdot K[0] = 3 \cdot 1 = 3.$$

A helyes $dX[1] = 1 + 2 + 3 = 6$. A felülíró implementáció a megadott ciklusrendben (i külső, k belső, mindkettő növekvő) az $u = 1$ célra legutoljára az $(i, k) = (2, 0)$ párnál ír, így $dX[1] = D[2]K[0] = 3$ lesz, tehát a három hozzájárulásból kettőt elveszít. Az átfedő patch-ek miatt a gradiens-hozzájárulásokat mindig összegezni kell.

F2 megoldás. A kimeneti méret:

$$H_{out} = \left\lfloor \frac{5 + 2 - 2 \cdot 2 - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{2}{2} \right\rfloor + 1 = 2.$$

A lehetséges $u = i \cdot 2 + r \cdot 2 - 1$ értékek $i \in \{0, 1\}$, $r \in \{0, 1, 2\}$ mellett:

i	r	u	érvényes?
0	0	-1	nem
0	1	1	igen
0	2	3	igen
1	0	1	igen
1	1	3	igen
1	2	5	nem

Tehát a lehetséges u-k halmaza $\{1, 3\}$. Az $u = 1$ ponthoz az $(i, r) = (0, 1)$ és $(1, 0)$ párok, az $u = 3$ ponthoz az $(i, r) = (0, 2)$ és $(1, 1)$ párok járulnak hozzá. A $0, 2, 4$ magasságindexekre a feltétel nem teljesíthető, ezért azok dX-e nulla. A szélesség mentén $k_w = 1$, $p_w = 0$, ezért $v = j \cdot 2$;

$W = 1$ mellett egyetlen érvényes $v = 0$ van ($j = 0$), így a szélesség nem zár ki hozzájárulást. Ha a paddinget a szélességre is alkalmaznánk, $v = 2j - 1$ sosem lenne 0, és a dX minden eleme nulla lenne — ezért rögzítettük a tengelyenkénti paddingot.

F3 megoldás. Az 1D referencia és az ellenőrző kód:

```
# lab/n058_f3_1d.py
import numpy as np
import torch
import torch.nn.functional as F

rng = np.random.default_rng(20261020)

def conv1d_input_grad_numpy(D, K, H, stride, padding, dilation):
    N, Cout, Hout = D.shape
    Cout2, Cin, kh = K.shape
    assert Cout2 == Cout
    dX = np.zeros((N, Cin, H), dtype=D.dtype)
    for n in range(N):
        for o in range(Cout):
            for i in range(Hout):
                for c in range(Cin):
                    for a in range(kh):
                        u = i * stride + a * dilation - padding
                        if 0 <= u < H:
                            dX[n, c, u] += D[n, o, i] * K[o, c, a]
    return dX

N, Cin, Cout, H = 1, 2, 2, 5
kh, stride, padding, dilation = 3, 2, 1, 1
Hout = (H + 2 * padding - dilation * (kh - 1) - 1) // stride + 1

X_np = rng.standard_normal((N, Cin, H)).astype(np.float64)
K_np = rng.standard_normal((Cout, Cin, kh)).astype(np.float64)
b_np = rng.standard_normal(Cout).astype(np.float64)
D_np = rng.standard_normal((N, Cout, Hout)).astype(np.float64)

dX_np = conv1d_input_grad_numpy(D_np, K_np, H, stride, padding, dilation)

X_t = torch.tensor(X_np, dtype=torch.float64, requires_grad=True)
K_t = torch.tensor(K_np, dtype=torch.float64, requires_grad=True)
b_t = torch.tensor(b_np, dtype=torch.float64, requires_grad=True)
Y_t = F.conv1d(X_t, K_t, b_t, stride=stride, padding=padding, dilation=dilation)
D_t = torch.tensor(D_np, dtype=torch.float64)
loss = (Y_t * D_t).sum()
loss.backward()

diff = np.max(np.abs(dX_np - X_t.grad.detach().numpy()))
assert diff < 1e-12, diff
print(f"Hout={Hout} max_abs_diff={diff:.3e}")
```

Futtatás: `python n058_f3_1d.py`. A mért kimenet (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed `20261020`): `Hout=3 max_abs_diff=4.441e-16`.

F4 megoldás. A hiba oka: Pythonban a `: -0` szelet felső (stop) határa `-0`, ami `0`-val egyenlő. A `padding:-padding` ezért `0:0`-t jelent, ami üres szelet. A `full` a példában a paddingelt koordinátarendszerben, $(N, C_{in}, H + 2p, W + 2p)$ mérettel felhalmozott scatter-add eredménye (a 15.2.5 kódjában viszont a `dX` közvetlenül a valós $H \times W$ tartományba ír, ott erre a levágásra nincs szükség). A következmény: $p = 0$ esetén a dX térbeli dimenziói 0 méretűek lesznek, miközben a várt alak (N, C_{in}, H, W), ezért a PyTorch autograddal való összehasonlítás shape-hibát ad.

Javítás: padding nélkül nincs mit levágni, mert a scatter-add eleve csak a valós tartományba ír:

```
if padding > 0:
    dX = full[:, :, padding:-padding, padding:-padding]
else:
    dX = full
```

Ellenőrzés: `assert dX.shape == (N, Cin, H, W)`, majd a `max_abs_diff` kiszámítása. A javítás utáni ellenőrző teszt a labor $p = 0$, $s = 1$, $d = 1$ konfigurációjában a fenti táblázat mért sorát adja: `dX.shape == (2, 3, 6, 7)` és `max_abs_diff=5.329e-15`; $p = 0$ esetén ugyanis a `full` a padding nélküli koordinátarendszerben éppen a labor `dX` tömbje, ezért a javított elágazás az `else` ágat választja, és az alak megmarad.

F5 megoldás. Hipotézis: a ciklusos referencia futásideje közelítőleg lineárisan nő N-nel, mert a batch iterációja független ismétlés; a csúcsmemória szintén lineárisan nő a dX és D tenzorok miatt. Mért mennyiségek: medián futásidő legalább 10 ismétlésből `time.perf_counter`-rel, valamint a csúcsmemória `tracemalloc`-kal (a `resource.getrusage` `ru_maxrss` értéke folyamatszintű csúcs, ezért kis tenzoroknál a Python-interpreter és az allokátor alapköltsége dominál). Protokoll: rögzített seed, f64 dtype, minden N-re azonos X, K, b, D generálás; minden mérés előtt egy bemelegítő (warm-up) hívás; az iterációk sorrendjét véletlenszerűen keverve, hogy az operációs rendszer gyorsítótárazása ne torzítson. Baseline: az üres ciklushurok futásideje azonos N-re (a ciklusvezérlés költsége), illetve a PyTorch backward ideje ugyanazon konfigurációra. Érvényességi korlát: az eredmény csak a jelen NumPy referencia implementációra és a helyi CPU-ra vonatkozik; nem vihető át GPU-ra vagy a PyTorch kernelre, és a kis batchméret tartományán kívül a Python interpreter és memória-allokátor viselkedése megváltozhat.

15.2.8 Tipikus tévedés ellenpéldával

Tipikus hiba a scatter-add összeadásának felülírásra cserélése. Az F1 példában láttuk: a helyes $dX[1] = 6$ helyett a felülíró kód a megadott ciklusrendben 3-at ad. A hiba oka az, hogy a forwardban egy input pixel több kimeneti elemhez tartozik; a láncszabály ezeknek a részhozzájárulásoknak az összegét írja elő. A felülírás a súlymegosztás inputoldali megfelelőjét töri el: ahogy a 15.1. Súly- és biasgradiens alfejezetben a kernelgradiensnél a felhasználási helyek összegzése volt a kulcs, itt ugyanaz az összegzés a bemeneti irányban jelenik meg.

15.2.9 Önellenőrzés és visszatekintés

Önellenőrző kérdés. Miért lesz a $p = 0$, $s = 3$, $d = 1$, $k_h = 2$ esetben bizonyos u-kra a dX nulla? Kulcs: mert a $u = is + rd - p$ egyenletnek nincs egész megoldása a kimeneti tartományban, és ez a kernel súlyától függetlenül igaz.

Ellenőrizze: a dX alakja mindig megegyezik az X alakjával, függetlenül a stride-tól és paddingtől. A $p = 0$, $s = 3$, $d = 1$, $k_h = 2$ példában a $u \equiv 2 \pmod 3$ bemeneti pozíciók dX-e minden kernel és D mellett nulla, mert az indexfeltétel nem oldható meg. A bias gradiensre a dX-ből nincs szükség: db mindenképpen a D batch- és térbeli összege a 15.1. Súly- és biasgradiens szerint.

Visszatekintés: a 3.2. Mátrixszorzás és affín réteg alfejezetben az affin réteg inputgradiense $W^\top dZ$ volt; a mostani levezetés ugyanezt a transzponált operátort valósítja meg keresztkorrelációs formában. A 14.2. Több bemeneti és kimeneti csatorna alfejezetben a forward csatornaösszeg volt a kimeneti csatornák felépítésének kulcsa; a dX képletében a o szerinti összegzés ennek a csatornastruktúrának a megfordítása. A 11.2. nn.Module és paraméterregisztráció alfejezet modulnyilvántartása csak annyiban kapcsolódik, hogy a `torch.nn.Conv2d` a kernelt `Parameter`ként

regisztrálja; a labor viszont közvetlenül a funkcionális `F.conv2d`-t hívja, ezért a regisztráció a számítás szempontjából nem játszik szerepet.

15.2.10 Rövid összefoglaló

A nyitó kérdésre a válasz: belső pixelre és $s = 1$ mellett a pixel $C_{out}k_hk_w$ kimeneti elem kiszámításában vesz részt, ezért ugyanennyi gradiens-hozzájárulást kap. A konvolúciós réteg inputgradiense az upstream gradiens és a kernel súlyának szorzatából álló scatter-add. A lemmában levezettük a teljes NCHW képletet, amely minden lehetséges kimeneti pozícióból összegzi a hozzájárulásokat a $u = is + rd - p$, $v = js + td - p$ célkoordinátákra (az r, t a kernel indexe). A stride-ra oszthatósági feltétel vonatkozik, a padding a virtuális pixelek hozzájárulásait zárja ki, a dilation a kernel effektív kiterjedését növeli. Az átfedő patch-ek miatt a hozzájárulásokat összeadni kell; a felülírás hibás gradienst ad. A NumPy-referenciát PyTorch autograddal több stride/padding/dilation konfiguráció mellett hasonlítottuk össze f64 tűréssel.

15.2.11 Továbblépés

A 15.3. Transposed convolution mint adjungált egységben a scatter-add operátort formális lineáris leképezésként vizsgáljuk: a forward konvolúció adjungáltja (a skalárszorzattartó transzponáltja, amelyet a 15.3. alfejezet definiál) a transzponált konvolúció. A mostani indexfeltételből közvetlenül leolvasható lesz a `ConvTranspose2d` bemeneti–kimeneti kapcsolata, és az, hogy miért nem általános inverz.

15.2.12 Felhasznált források

- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

15.3 Transposed convolution mint adjungált

15.3.1 Cél, előismeret és induló kérdés

A 15.2. Inputgradiens és átfedő recepciós mezők egységben indexszinten levezettük, hogy a konvolúciós réteg bemeneti gradiense az upstream gradiensből scatter-add (szétszórás–összegzés) művelettel áll elő. Most ezt a műveletet egyetlen matematikai objektumként vizsgáljuk: a forward konvolúció lineáris operátor, a backwardban számolt bemeneti gradiens pedig ennek az operátornak az adjungáltja, azaz mátrixreprezentációban a transzponáltja. A könyvtári `ConvTranspose2d` nem az inverz, hanem éppen ezt az adjungált utat valósítja meg. A [D11] könyvtári leírását a következőkben saját mátrixos levezetéssel vetjük össze, és különválasztjuk az adjungált műveletet az inverz leképezéstől.

Induló kérdés: ha egy 4 hosszú 1D jelet egy 2 hosszú kernel valid, stride=1 cross-correlationje 3 hosszúvá képez, akkor a 3 hosszú gradiensből a 4 hosszú bemeneti gradienst a kernel megfordításával vagy a kernel eredeti sorrendjével, de a mátrixtranszponált oszlopaiból kapjuk? A válasz

a mátrixtranszponált: nem a kernel elemeinek sorrendje változik meg, hanem az, hogy melyik kimeneti indexhez melyik bemeneti index és melyik kernelegyüttható tartozik.

15.3.2 Lineáris operátor, adjungált és mátrixtranszponált

Legyen $V = \mathbb{R}^{d_{in}}$ és $W = \mathbb{R}^{d_{out}}$ a standard euklideszi skalárszorzattal: $\langle a, b\rangle = a^\top b$. Egy $C : V \to W$ leképezés **lineáris operátor**, ha minden $x_1, x_2 \in V$ és $\alpha, \beta \in \mathbb{R}$ esetén

$$C(\alpha x_1 + \beta x_2) = \alpha C(x_1) + \beta C(x_2).$$

Rögzített bázisban C egy $d_{out} \times d_{in}$ mátrixszal reprezentálható; a továbbiakban ugyanazt a jelet használjuk a leképezésre és a mátrixára.

Definíció (adjungált). A $C^* : W \to V$ leképezés a C adjungáltja, ha minden $x \in V$, $y \in W$ esetén

$$\langle Cx, y\rangle_W = \langle x, C^* y\rangle_V. \tag{1}$$

Valós mátrix esetén az adjungált a mátrixtranszponált: $C^* = C^\top$, mert $(Cx)^\top y = x^\top C^\top y$. Komplex terekben a konjugált transzponált lenne az adjungált; ebben a fejezetben minden mennyiség valós, így a transzponált és az adjungált azonosítható.

Tétel (az adjungált egyértelműsége). Ha C^* és $C^\dagger$ is teljesíti az (1) feltételt, akkor $C^* = C^\dagger$. (A $C^\dagger$ itt a C^* egy másik jelöltje, nem más művelet.)

Bizonyítás. Mivel C^* és $C^\dagger$ is teljesíti az (1) feltételt, ugyanarra a Cx-re $\langle Cx, y\rangle = \langle x, C^* y\rangle$ és $\langle Cx, y\rangle = \langle x, C^\dagger y\rangle$, ezért a két belső szorzat különbsége nulla. Minden $y \in W$ esetén válasszuk most az $x = (C^* - C^\dagger)y$ vektort. Ekkor

$$0 = \langle x, C^* y\rangle - \langle x, C^\dagger y\rangle = \langle x, (C^* - C^\dagger)y\rangle = \|x\|^2,$$

tehát $x = 0$, azaz $C^* y = C^\dagger y$, minden y-ra. ■

A neurális hálózatok nyelvén az adjungált a vektor–Jacobi-szorzat szerepét tölti be. Ha $L = g(y)$ skalárfüggvény és $y = Cx$, akkor a láncszabály szerint

$$\nabla_x L = C^\top \nabla_y L. \tag{2}$$

A 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben bevezetett VJP (vektor–Jacobi-szorzat, azaz a $C^\top \nabla_y L$ szorzat) éppen ezt a transzponált alkalmazást jelenti; a 15.2. Inputgradiens és átfedő recepciós mezők alfejezetben levezetett dX képlet ennek a konvolúciós C mátrixra specializált, indexelt alakja.

15.3.3 A Conv2d mint lineáris operátor

A 15.2. Inputgradiens és átfedő recepciós mezők és [D10] jelölése szerint legyen a bemeneti képbatch $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$, a kimenet $Y \in \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}}$, a kernel pedig $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$. Ha a bias $b \in \mathbb{R}^{C_{out}}$ nulla ($b = 0$), akkor a cross-correlation minden egyes kimeneti eleme a bemenet lineáris kombinációja:

$$y_{n,o,i,j} = \sum_{c=0}^{C_{in}-1}\sum_{r=0}^{k_h-1}\sum_{t=0}^{k_w-1} K_{o,c,r,t}\,\bar{x}_{n,c,\,is_h+rd_h-p_h,\,js_w+td_w-p_w}.$$

Itt r, t a kernel sor- és oszlopindexe, s_h, s_w a stride, d_h, d_w a dilation, p_h, p_w a padding. Az $\bar{x}$ a bemenet **virtuális nullakiterjesztése**: $\bar{x}_{n,c,u,v} = x_{n,c,u,v}$, ha $0 \le u < H$ és $0 \le v < W$, különben 0. (A paddingen kívüli virtuális elemek hozzájárulása így nulla.) Vektorizáljuk a tensort a memóriabeli, NCHW-sorrendű (batch–csatorna–magasság–szélesség) lapítással: $\mathrm{vec}(X) \in \mathbb{R}^{NC_{in}HW}$. A fenti összeg egy $C(K)$ mátrixot határoz meg, amelyre $\mathrm{vec}(Y) = C(K)\,\mathrm{vec}(X)$. (Itt $C(K)$ a K által meghatározott operátormátrix; a C betű a leképezést jelöli, a C_{in}, C_{out} pedig a csatornaszámokat.) A mátrix sorai a kernel együtthatóit tartalmazzák a megfelelő bemeneti pozíciókban, a padding miatt pedig nulla hozzájárulásokat. A backward bemeneti gradiense ezért

$$\mathrm{vec}(dX) = C(K)^{\top}\,\mathrm{vec}(dY),$$

amely pontosan a 15.2. Inputgradiens és átfedő recepciós mezők alfejezetben levezetett scatter-add (szétszórás–összegzés): minden dY-elem a kernel súlyával visszaszóródik az őt létrehozó bemeneti pozíciókra, és az átfedések gradienshozzájárulásai összegződnek.

15.3.4 Transposed convolution mint adjungált

A **transposed convolution** (transzponált konvolúció) a diszkrét jelfeldolgozásban az az operátor, amely a forward cross-correlation lineáris leképezésének adjungáltját valósítja meg. A „transposed” szó a leképezés mátrixának transzponáltjára utal, nem a kernel valamely geometriai megfordítására, és nem is egy általános inverzre. Fontos pontosítás: a mátrix transzponáltja a forward összegzésben a kernel együtthatóit fordított sorrendben párosítja a kimeneti indexekhez (a (4) képletben k_1 kerül y_0 mellé), ezért a transzponált művelet a k vektorral felírva megfordított kernelű korrelációként is megjeleníthető. A két leírás ugyanannak a lineáris operátornak két ekvivalens nézete; a név a mátrixműveletre utal. A jelfeldolgozási háttérről [S01] ad áttekintést.

A PyTorch `ConvTranspose2d` modul ennek a diszkrét operátornak a paraméterezett rétege. Csatornaszempontból a forward $C_{in} \to C_{out}$ út megfordul: a gradiensirányú modul bemenete C_{out}, kimenete C_{in} csatornát kezel. Két dolgot kell elválasztani. Az **operátormátrix** transzponálódik: $C(K) \in \mathbb{R}^{d_{out}\times d_{in}}$ helyett $C(K)^{\top} \in \mathbb{R}^{d_{in}\times d_{out}}$. Az **API súlytenzorának tárolási konvenciója** viszont nem cserél dimenziót: a `Conv2d(C_in, C_out)` súlya $(C_{out}, C_{in}, k_h, k_w)$, a `ConvTranspose2d(in_channels=C_out, out_channels=C_in)` súlya pedig $(\text{in_channels}, \text{out_channels}, k_h, k_w) = (C_{out}, C_{in}, k_h, k_w)$ — ugyanaz az alak. Az adjungált modulba ezért a forward súlya csere nélkül, változatlanul másolható; a transzponálást a modul műveleti definíciója végzi el. A bias nem része a lineáris adjungált azonosságnak; az affin $x \mapsto Cx + b$ leképezés Jacobi-mátrixa továbbra is C, ezért a bemeneti gradiensben a bias nem jelenik meg, a bias saját gradiense pedig csatornánként az upstream gradiens $N \cdot H_{out} \cdot W_{out}$ elemének összege.

Többcsatornás ellenőrzés: egy $2 \to 3$ csatornás, stride-os, paddinges forward és a $3 \to 2$ csatornás transzponált modul ugyanazzal a súllyal, az adjungált $\langle Cx, y\rangle = \langle x, C^\top y\rangle$ skalárszorzat-azonossággal:

```
import torch

torch.manual_seed(20261020)
dt = torch.float64
fwd = torch.nn.Conv2d(2, 3, kernel_size=(2, 3), stride=2, padding=1,
                      bias=False, dtype=dt)
adj = torch.nn.ConvTranspose2d(3, 2, kernel_size=(2, 3), stride=2,
                               padding=1, bias=False, dtype=dt)
print("Conv2d súly:", tuple(fwd.weight.shape))
# (C_out, C_in, kh, kw) = (3, 2, 2, 3)
print("ConvTranspose2d súly:", tuple(adj.weight.shape))
# (in_channels, out_channels/groups, kh, kw) = (3, 2, 2, 3)
with torch.no_grad():
    adj.weight.copy_(fwd.weight)                          # ugyanaz a tensor, csere nélkül

x = torch.randn(1, 2, 7, 6, dtype=dt)                     # C: R^(2*7*6) -> R^(3*H'*W')
y_shape = fwd(x).shape
y = torch.randn(y_shape, dtype=dt)
ct_y = adj(y, output_size=x.shape[-2:])                   # C^T y, a bemenet alakjára
lhs = (fwd(x) * y).sum()                                  # <Cx, y>
rhs = (x * ct_y).sum()                                    # <x, C^T y>
print("<Cx,y> =", lhs.item(), " <x,C^T y> =", rhs.item())
assert torch.allclose(lhs, rhs, rtol=1e-12, atol=1e-12)
```

A mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, float64, seed `20261020`): mindkét súlytenzor alakja `(3, 2, 2, 3)`, a két skalárszorzat pedig `3.791763138216891`, illetve `3.7917631382168917`; a két kiírt érték eltérése $8{,}9 \cdot 10^{-16}$, ami az `assert` 10^{-12}-es tűrésén belül van. Az `output_size` argumentum a `forward` hívásban rögzíti a transzponált kimenet méretét (lásd az `output_padding` alább); stride esetén enélkül a kimeneti alak nem egyértelmű, ezért a modul a bemenet alakjából csak korlátozottan tudja kikövetkeztetni.

Ezt a fogalmat élesen el kell választani az **inverztől**. Az inverzhez olyan $C^{-1} : W \to V$ kellene, amelyre $C^{-1}C = I_{d_{in}}$ és $CC^{-1} = I_{d_{out}}$. Ilyen csak akkor létezhet, ha $d_{in} = d_{out}$ és C teljes rangú. A konvolúciós forward általában dimenziót változtat vagy ranghiányos, ezért az adjungált csak a belső szorzat értékét őrzi meg, az adatot általában nem állítja vissza.

15.3.5 Levezetés: explicit 1D operátormátrix és adjungált azonosság

Tekintsünk egy 4 elemű bemenetet, $x = (x_0, x_1, x_2, x_3)^\top \in \mathbb{R}^4$, és egy 2 elemű kernelt $k = (k_0, k_1)^\top$. A valid, stride=1, padding=0 cross-correlation kimenete három elemű:

$$y_0 = k_0 x_0 + k_1 x_1, \qquad y_1 = k_0 x_1 + k_1 x_2, \qquad y_2 = k_0 x_2 + k_1 x_3.$$

A leképezés mátrixa

$$C = \begin{pmatrix} k_0 & k_1 & 0 & 0 \\ 0 & k_0 & k_1 & 0 \\ 0 & 0 & k_0 & k_1 \end{pmatrix} \in \mathbb{R}^{3\times 4}. \tag{3}$$

Lemma (valid cross-correlation adjungáltja). A (3) mátrix adjungáltja a transzponált

$$C^\top = \begin{pmatrix} k_0 & 0 & 0 \\ k_1 & k_0 & 0 \\ 0 & k_1 & k_0 \\ 0 & 0 & k_1 \end{pmatrix} \in \mathbb{R}^{4\times 3},$$

azaz $C^\top y$ elemei

$$(C^\top y)_0 = k_0 y_0, \quad (C^\top y)_1 = k_1 y_0 + k_0 y_1, \quad (C^\top y)_2 = k_1 y_1 + k_0 y_2, \quad (C^\top y)_3 = k_1 y_2. \tag{4}$$

Bizonyítás. Tetszőleges $x \in \mathbb{R}^4$, $y \in \mathbb{R}^3$ esetén a forward mátrixszorzást beírva:

$$\langle Cx, y\rangle = y_0(k_0 x_0 + k_1 x_1) + y_1(k_0 x_1 + k_1 x_2) + y_2(k_0 x_2 + k_1 x_3).$$

Csoportosítsunk x komponensei szerint:

$$= x_0(k_0 y_0) + x_1(k_1 y_0 + k_0 y_1) + x_2(k_1 y_1 + k_0 y_2) + x_3(k_1 y_2) = \langle x, C^\top y\rangle.$$

Mivel ez minden x, y párra fennáll, a (4) képlet éppen az adjungált definícióját teljesíti, és a fenti tétel miatt az adjungált egyértelmű. ■

A (4) alak a scatter-add: a y_1 gradiens például k_0 súllyal az x_1 pozícióra és k_1 súllyal az x_2 pozícióra hullik vissza, mert a forwardban mindkét bemeneti elem részt vett a y_1 előállításában, x_1 a k_0, x_2 a k_1 együtthatóval. Így a mátrixtranszponált oszlopszerkezete közvetlenül az átfedő recepciós mezők elvét kódolja.

15.3.6 Alak-visszaállítás, output_padding és inverz limit

A transzponált konvolúció egyik leggyakoribb felhasználási módja a térbeli felbontás növelése egy downsampling után. A mátrixtranszponált eleve a megfelelő irányú méretet adja: ha $C : V \to W$, akkor $C^\top : W \to V$. Az **alak-visszaállítás** tehát nem extra tulajdonság, hanem a transzponált dimenzióinak következménye; az **értékek visszaállítása** ellenben erősebb feltételt kíván: balinverzhez elég a teljes oszloprang, kétoldali inverzhez négyzetes, teljes rangú mátrix kell (részletesen az inverz-limit következményben).

A stride **alsó egészrészes** osztást vezet be a forward oldalon ($\lfloor\cdot\rfloor$). A 14.3. Paraméterszám, MAC és aktivációmemória alfejezetben (MAC: szorzás–összeadás művelet) és a [D10]-ben használt képlet szerint

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

A padlófüggvény miatt több különböző H is adhatja ugyanazt a H_{out} értéket. A transzponált oldalon ezért nem mindig dönthető el egyértelműen, mekkora legyen a kimeneti méret. A PyTorch `ConvTranspose2d` ezt az `output_padding` paraméterrel teszi eldönthetővé: a szokásos jelöléssel, ha a transzponált bemenete H_{in}^T méretű,

$$H_{out}^T = (H_{in}^T - 1)s - 2p + d(k-1) + 1 + \text{output_padding}. \tag{5}$$

Az `output_padding` tehát csak a méretválasztást egészíti ki; a mátrixtranszponált értékszerkezetén nem változtat. A megadott kimeneti képlet mellett a `ConvTranspose2d` konstruktorának `output_padding`-korlátját is teljesíteni kell: `output_padding < max(stride, dilation)`, tehát például stride 2 mellett az érték legfeljebb 1 lehet [D11].

Következmény (inverz limit). Legyen $C \in \mathbb{R}^{d_{out} \times d_{in}}$. A 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetben tanult rangfeltétel szerint $C^\top C \in \mathbb{R}^{d_{in} \times d_{in}}$ **pontosan akkor** invertálható, ha C teljes oszloprangú, és $CC^\top \in \mathbb{R}^{d_{out} \times d_{out}}$ pontosan akkor, ha C teljes sorrangú. A két feltétel független: például $C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$-ra $C^\top C = I_2$ invertálható, miközben a sorrang $2 < 3$, így $CC^\top$ szinguláris. Kétoldali inverzhez ($C^{-1}C = I$ és $CC^{-1} = I$) négyzetes ($d_{in} = d_{out}$), teljes rangú mátrix kell. A $C^\top$ még ekkor sem általában C^{-1}: ehhez $C^\top C = I$, vagyis ortogonális C kellene, ami egy konvolúciós operátornál kivételes eset. Teljes oszloprangú (akár téglalap alakú) C-nek van balinverze, $(C^\top C)^{-1}C^\top$, ezért az input értékei elvileg rekonstruálhatók a kimenetből; ha $C^\top C = I$, akkor ez éppen $C^\top$. A balinverz azonban nem a `ConvTranspose2d` művelete: a könyvtári réteg a $C^\top$ szorzást valósítja meg, a $(C^\top C)^{-1}$ tényezőt nem tartalmazza. Az „értékrekonstrukció csak négyzetes esetben lehetséges” állítás tehát túl erős: kétoldali inverzhez kell négyzetes mátrix, balinverzhez elég a teljes oszloprang.

15.3.7 Eredeti kézi számpélda

Legyen $k = (2, -1)^\top$, $x = (1, 3, 0, 2)^\top$. A forward:

$$y = Cx = \begin{pmatrix} 2 \cdot 1 + (-1) \cdot 3 \\ 2 \cdot 3 + (-1) \cdot 0 \\ 2 \cdot 0 + (-1) \cdot 2 \end{pmatrix} = \begin{pmatrix} -1 \\ 6 \\ -2 \end{pmatrix}.$$

Válasszuk az upstream gradienst $g = \nabla_y L = (-1, 6, -2)^\top$ értékűnek; a kézi példa kedvéért ez egyszerűen maga az y vektor, de az adjungált azonosság tetszőleges g-re külön is ellenőrizhető. A (4) képlet:

$$C^\top g = \begin{pmatrix} 2(-1) \\ (-1)(-1) + 2 \cdot 6 \\ (-1) \cdot 6 + 2(-2) \\ (-1)(-2) \end{pmatrix} = \begin{pmatrix} -2 \\ 13 \\ -10 \\ 2 \end{pmatrix}.$$

Ellenőrzés a skalárszorzatos azonossággal:

$$\langle Cx, g \rangle = (-1)^2 + 6^2 + (-2)^2 = 1 + 36 + 4 = 41,$$

$$\langle x, C^\top g \rangle = 1(-2) + 3 \cdot 13 + 0(-10) + 2 \cdot 2 = -2 + 39 + 4 = 41.$$

A két oldal egyezik, így a transzponált mátrix teljesíti az adjungált feltételt.

Minimális NumPy-ellenőrzés:

```python
# n059_manual_example.py
import numpy as np

k = np.array([2.0, -1.0])
x = np.array([1.0, 3.0, 0.0, 2.0])
C = np.zeros((3, 4))
C[0, 0:2] = k
C[1, 1:3] = k
C[2, 2:4] = k

y = C @ x                       # [-1, 6, -2]
g = np.array([-1.0, 6.0, -2.0])
dx = C.T @ g                    # [-2, 13, -10, 2]
lhs = float(y @ g)              # 41
rhs = float(x @ dx)             # 41
assert np.allclose(y, [-1.0, 6.0, -2.0])
assert np.allclose(dx, [-2.0, 13.0, -10.0, 2.0])
assert abs(lhs - rhs) == 0.0
print("y =", y)
print("C^T g =", dx)
print("adjoint difference =", abs(lhs - rhs))
```

15.3.8 Labor: mátrixtranszponált és könyvtári művelet egybevetése

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `labor/n059_adjoint_check.py` futtatható (`python labor/n059_adjoint_check.py`); külső adatfájl nem kell.

A labor célja, hogy az 1D valid cross-correlation explicit mátrixának transzponáltját összehasonlítsa a PyTorch `conv_transpose1d` függvényével. A `conv_transpose1d` a forward 1D konvolúció adjungált útját adja; egycsatornás esetben a súly alakja megegyezik, így a mátrixos és a könyvtári eredmény közvetlenül összevethető. A teszt determinisztikus, véletlen generálást nem használ; a seed megadása a laborprotokoll részeként mégis rögzíti a környezetet.

```python
# labor/n059_adjoint_check.py
import numpy as np
import torch
import torch.nn.functional as F

def make_valid_cc_matrix(n_in: int, k: np.ndarray) -> np.ndarray:
    """1D valid, stride=1, padding=0 cross-correlation matrix."""
    k = np.asarray(k, dtype=np.float64)
    n_out = n_in - k.size + 1
    C = np.zeros((n_out, n_in), dtype=np.float64)
    for i in range(n_out):
        C[i, i:i + k.size] = k
    return C

torch.manual_seed(0)            # nincs véletlen input; a seed a napló része
dtype = torch.float64
device = torch.device("cpu")

k = np.array([2.0, -1.0])
n_in = 4
C = make_valid_cc_matrix(n_in, k)

# upstream gradiens a forward kimeneti terében
g = np.array([-1.0, 6.0, -2.0])
manual = C.T @ g                # (4,)
```

```
g_t = torch.tensor(g, dtype=dtype, device=device).view(1, 1, -1)
w_t = torch.tensor(k, dtype=dtype, device=device).view(1, 1, -1)
# stride=1, padding=0, dilation=1, output_padding=0 explicit
library = F.conv_transpose1d(
    g_t, w_t, bias=None, stride=1, padding=0,
    output_padding=0, groups=1, dilation=1,
).cpu().numpy().reshape(-1)

max_abs_diff = float(np.max(np.abs(manual - library)))
assert max_abs_diff < 1e-12, max_abs_diff
print("C^T g manual :", manual)
print("conv_transpose :", library)
print("max_abs_diff   :", max_abs_diff)
print("PASS" if np.allclose(manual, library, atol=1e-12, rtol=0) else "FAIL")
```

A kód futtatási parancsa a tankönyvi munkakönyvtárból:

```
python labor/n059_adjoint_check.py
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads()` = 10, seed 0):

```
C^T g manual : [ -2.  13. -10.   2.]
conv_transpose : [ -2.  13. -10.   2.]
max_abs_diff   : 0.0
PASS
```

Ez a numerikus egyezés arra a kicsi esetre igazolja, hogy a könyvtári művelet a várt mátrixtranszponáltat alkalmazza; ez nem általános bizonyítás, hanem független numerikus kontroll egyetlen konfigurációra; az autograd működését a [D05] dokumentáció írja le. A [D11] API-szakaszának teljes ellenőrzése után a labor a 2D `ConvTranspose2d`-re, valamint stride>1 és `output_padding` esetekre is kiterjesztendő; a fenti kimenet a futtatott kódból származó mért eredmény.

15.3.9 Feladatok

F1 (fogalmi, nehézség 1). Definiálja külön-külön a mátrixtranszponáltat, az alak-visszaállítást és az inverzet. Adjon egy konkrét C mátrixot, amelyre $C^\top C \neq I$, és mutassa meg, hogy $C^\top C$ mérete miért árulhatja el már az inverz hiányát. Beadandó: három definíció és egy konkrét mátrix a számítással.

F2 (levezetési/bizonyítási, nehézség 2). Legyen a bemenet $x \in \mathbb{R}^5$, a kernel $k = (1, 1)^\top$, stride=2, padding=0, 1D valid cross-correlation. Írja fel a C mátrixot, a $C^\top$ mátrixot, és tetszőleges, majd a konkrét $x = (1, 2, -1, 0, 3)^\top$, $y = (4, -2)^\top$ párral igazolja a skalárszorzatos adjungált azonosságot. Mutassa meg, hogy $C^\top C$ nem invertálható, és adjon meg egy nemnulla vektort a nullterében. Beadandó: teljes mátrixok, azonosságellenőrzés, rang-indoklás, nulltérvektor.

F3 (implementációs, nehézség 2). Írjon függvényt, amely tetszőleges 1D kernelre felépíti a valid cross-correlation mátrixot, majd a transzponált mátrixszorzást a `F.conv_transpose1d` kimenetével hasonlítja össze. Ellenőrzés két különböző kernelre, legalább 10^{-13} abszolút toleranciával, `float64` típuson. Beadandó: futtatható kód, függvény, két teszteset és PASS kimenet.

F4 (hibakeresési, nehézség 2). Adott az alábbi érvelés: „Mivel a réteg neve `ConvTranspose2d`, ezért az általa számított kimenet a megfelelő `Conv2d` inverze, tehát stride=2 downsampling után az eredeti képet adja vissza.” Lokalizálja a hibát, adjon 1D ellenpéldát két különböző bemenettel, amelyek forwardja azonos, és ezért a transzponált kimenetük is azonos. Írja le a javított állítást és annak ellenőrzését. Beadandó: hibaelemzés, ellenpélda, javítás.

F5 (kísérleti/kutatói transzfer, nehézség 3). Tervezzen kis kontrollált kísérletet, amely eldönti, hogy egy rögzített `ConvTranspose2d` konfiguráció pusztán alak-visszaállítást vagy numerikusan is inverzet ad. Használjon 5×5-es bemenetet, 3×3-as kernelt, stride=2, padding=1, `output_padding=0` értéket; ezzel a forward kimenete 3×3, a transzponált kimenet pedig az (5) képlet szerint $(3-1)\cdot 2-2+2+1+0=5$, azaz 5×5. Adja meg a nullhipotézist, a metrikát, a baseline-t, az ismétlésszámot, a rögzített seedet és a következtetés érvényességi korlátját. Futtatás nem kötelező. Beadandó: legfeljebb egy oldalas protokoll. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

15.3.10 Megoldások és indoklás

F1 megoldás. A mátrixtranszponált $C^\top$ a sorok és oszlopok felcserélése: $(C^\top)_{ij}=C_{ji}$. Az alak-visszaállítás csak annyit jelent, hogy $C^\top$ a C értelmezési tartományának dimenziójába, azaz a d_{out} dimenziós kimeneti térből a d_{in} dimenziós bemeneti térbe képez; ez a mátrixméretből következik, nem garantál értékazonosságot. Az inverz C^{-1} a kétoldali azonosságot teljesítő mátrix: $C^{-1}C=I$ és $CC^{-1}=I$.

Konkrét példa a fenti (3) mátrix $k=(2,-1)$ választással:

$$C=\begin{pmatrix}2&-1&0&0\\0&2&-1&0\\0&0&2&-1\end{pmatrix},\qquad C^\top C=\begin{pmatrix}4&-2&0&0\\-2&5&-2&0\\0&-2&5&-2\\0&0&-2&1\end{pmatrix}.$$

Mivel C mérete 3×4, a $C^\top C$ mérete 4×4, de rangja legfeljebb 3, ezért nem lehet az identitás. Például $x=(1,2,4,8)^\top$ a C nullterében van, mert $Cx=0$, így $C^\top Cx=0\neq x$.

F2 megoldás. A forward: $y_0=x_0+x_1$, $y_1=x_2+x_3$; az x_4 elem a stride miatt nem járul hozzá. A mátrix:

$$C=\begin{pmatrix}1&1&0&0&0\\0&0&1&1&0\end{pmatrix},\qquad C^\top=\begin{pmatrix}1&0\\1&0\\0&1\\0&1\\0&0\end{pmatrix}.$$

Általánosan:

$$\langle Cx,y\rangle=y_0(x_0+x_1)+y_1(x_2+x_3)=x_0y_0+x_1y_0+x_2y_1+x_3y_1=\langle x,C^\top y\rangle.$$

A konkrét párral $Cx=(3,-1)^\top$, ezért $\langle Cx,y\rangle=3\cdot 4+(-1)(-2)=14$, míg $C^\top y=(4,4,-2,-2,0)^\top$, tehát $\langle x,C^\top y\rangle=1\cdot 4+2\cdot 4+(-1)(-2)+0(-2)+3\cdot 0=14$. Az azonosság fennáll.

A $C^\top C$ mátrix:

$$C^\top C = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Rangja 2, kisebb, mint 5, tehát szinguláris. A nulltérnek eleme például $v = (0,0,0,0,1)^\top$, mert $Cv = 0$, így $C^\top Cv = 0$. Alakellenőrzés: C mérete 2×5, $C^\top$ mérete 5×2, a szorzat 5×5.

F3 megoldás.

```python
# tasks/n059_f3.py
import numpy as np
import torch
import torch.nn.functional as F

def make_valid_cc_matrix(n_in: int, k: np.ndarray) -> np.ndarray:
    k = np.asarray(k, dtype=np.float64)
    n_out = n_in - k.size + 1
    C = np.zeros((n_out, n_in), dtype=np.float64)
    for i in range(n_out):
        C[i, i:i + k.size] = k
    return C

def check_adjoint(k, n_in=8):
    C = make_valid_cc_matrix(n_in, k)
    n_out = C.shape[0]
    g = np.arange(1.0, n_out + 1.0)
    manual = C.T @ g
    g_t = torch.tensor(g, dtype=torch.float64).view(1, 1, -1)
    w_t = torch.tensor(k, dtype=torch.float64).view(1, 1, -1)
    library = F.conv_transpose1d(
        g_t, w_t, stride=1, padding=0, output_padding=0
    ).numpy().reshape(-1)
    return C, manual, library

for k in ([2.0, -1.0], [0.5, 0.25, 0.5]):
    C, manual, library = check_adjoint(np.array(k))
    ok = np.allclose(manual, library, atol=1e-13, rtol=0)
    assert ok, (k, manual, library)
    print("kernel ", k)
    print("C shape ", C.shape)
    print("max_abs_diff", float(np.max(np.abs(manual - library))))
    print("PASS" if ok else "FAIL")
```

Futtatás: `python tasks/n059_f3.py`. A mért kimenet (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`): mindkét kernelre `PASS`, `max_abs_diff 0.0` (a 10^{-13} tolerancia elegendő), a C alakja pedig $(7,8)$, illetve $(6,8)$.

F4 megoldás. A hiba a „transposed convolution = inverz” azonosításban van. A `ConvTranspose2d` a forward lineáris operátor adjungáltja, nem az inverze; a stride=2 downsampling nem injektív, ezért inverz nem is létezhet. 1D ellenpélda a F2 feladat C mátrixával: $x = (1,0,0,0,0)^\top$ és $x' = (2,-1,0,0,0)^\top$ esetén $Cx = Cx' = (1,0)^\top$, tehát a transzponált kimenet mindkettőre $C^\top(1,0)^\top = (1,1,0,0,0)^\top$. Egyetlen dekódolt vektor nem állíthatja vissza mindkét bemenetet. Ugyanezt a lineáris leképezést valósítja meg a `ConvTranspose2d` (1D megfelelője a `conv_transpose1d`), ezért a két azonos forward kimenethez a könyvtári modul is ugyanazt a transzponált kimenetet adja. A könyvtári ellenőrzés mért eredménye (float64, `w = [[[1.0, 1.0]]]`, `stride=2`, a transzponáltnál `output_padding=1`; torch 2.15.0+cpu): `F.conv1d` mindkét bemenetre `[1.0, 0.0]`, `F.conv_transpose1d` mindkettőre `[1.0, 1.0,`

`0.0, 0.0, 0.0]`. Javított állítás: a réteg a forward adjungáltját, vagy másként az upsampling irányú gradiens-szórást számolja; rekonstrukcióhoz tanított dekódert, invertálható architektúrát vagy kiegészítő loss-t kell használni. Ellenőrzés: rangszámítás vagy a fenti kétinputos teszt.

F5 megoldásminta. Kérdés: a megadott konfiguráció érték szerint is inverze-e a forwardnak? **Nullhipotézis (H_0):** a transzponált réteg kimenete értékben visszaadja a bemenetet, azaz $e = \|C^\top Cx - x\|_2 / \|x\|_2 < 10^{-12}$ minden tesztképre. **Ellenhipotézis (H_1):** legalább egy képre $e \geq 10^{-12}$, vagyis a kimenet csak alakban egyezik. A H_0-t akkor vetjük el, ha a mért e valamelyik képen eléri a küszöböt; ez determinisztikus numerikus kritérium, nem mintavételi próba, ezért nem keverendő a statisztikai szignifikanciával. **Metrika:** a fenti relatív hiba, valamint külön shape-ellenőrzés. **Adat:** 20 darab, rögzített seeddel (`torch.manual_seed(20261020)`) generált, $[-1, 1]$-beli 5×5-es kép, NCHW float64. **Baseline:** az identitás kimenet ($e = 0$) csak a metrika kalibrálására szolgál, mert triviálisan teljesül; érdemi összehasonlítási alap a forward–transzponált pár kimenete. **Protokoll:** minden x-re kiszámítjuk a forward és a transzponált kimenetet; ha $e < 10^{-12}$ minden esetben, H_0-t a tesztelt konfigurációban megtartjuk, különben elvetjük. **Várható értelmezés:** a ranghiány miatt e nem nulla, így a modul csak alakot állít vissza. **Korlát:** a numerikus mérés egy konfigurációra vonatkozik; az általános érvénytelenséget a mátrix rangja bizonyítja lineáris algebrából, a mérés csak demonstrál.

15.3.11 Tipikus tévedés és ellenpélda

Tipikus tévedés, hogy a transposed convolutiont **dekonvolúciónak** vagy **inverznek** nevezik, és elvárják, hogy egy veszteséges forward után visszaadja az eredeti képet. A fenti $k = (2, -1)$ példában a forward 4 elemű jelet 3 eleművé képez. A C mátrixnak nemtriviális nulltere van: $x = (1, 2, 4, 8)^\top$ esetén $Cx = 0$. Ha a transzponált konvolúció inverz lenne, akkor a nulla forward kimenetből ezt a nemnulla x-et kellene visszaállítania; de lineáris leképezés nullát nullába visz, tehát ez lehetetlen. Az alak lehet helyes, a pixelértékek nem. A tanulság: a `ConvTranspose2d` a mátrixtranszponáltat számolja, és a skalárszorzat-azonosságot őrzi, nem a rekonstrukciós azonosságot.

15.3.12 Önellenőrzés és visszatekintés

Ellenőrző kérdések: (1) A (3) mátrix esetén mi a C és $C^\top$ mérete, és miért nem lehet $C^\top C = I$? (2) Írja fel a (4) képlet egy tetszőleges y-ra, és ellenőrizze, hogy a y_1 komponens két helyre is szóródik; melyik forward együtthatók miatt? (3) Egy $4 \to 3$ forward után miért adhat a transzponált réteg $3 \to 4$ alakot úgy, hogy közben az értékek nem állnak vissza? A válaszok a fenti levezetésekben találhatók.

Visszatekintés: 3.3. Vetítés, legkisebb négyzetek és kondicionáltság alfejezetből a normálegyenlet $A^\top A$ mátrixa csak teljes oszloprang esetén invertálható; itt a C oszloprangja kisebb, mint a bemeneti dimenzió, ezért $C^\top C$ szingularitása ugyanaz a jelenség. 14.3. Paraméterszám, MAC és aktivációmemória alfejezetből: a paraméterszám és a MAC (szorzás-összeadás művelet) a kimeneti felbontáshoz kötődik, ezért az adjungált réteg költségét is a tényleges kimeneti méretből kell számolni, nem az „inverz” elnevezésből. 15.2. Inputgradiens és átfedő recepciós mezők alfejezetből: az inputgradiens scatter-addként áll elő, ami éppen az adjungált operátor indexszintű alakja; a transzponált konvolúció ezt a műveletet emeli ki önálló rétegként.

15.3.13 Rövid összefoglaló

A nyitó kérdésre a válasz: a 3 hosszú gradiensből a 4 hosszú bemeneti gradienst a mátrixtranszponált adja, nem a kernel tárgyi megfordítása. A konvolúciós forward lineáris operátor; a backward bemeneti gradiense az operátor adjungáltja, amely valós mátrixreprezentációban a mátrixtranszponált alkalmazása. Kis 1D valid cross-correlation mátrixon levezettük a skalárszorzatos adjungált azonosságot, és megmutattuk, hogy a transzponált a scatter-add gradiensszórást adja. A `ConvTranspose2d` ezt az adjungált utat valósítja meg, nem általános inverzet. Az `output_padding` csak a padlófüggvény miatt nem egyértelmű kimeneti méretet választja. Numerikus labor egy 1D mátrixtranszponáltat hasonlít a könyvtári művelethez.

15.3.14 Továbblépés

A 15.4. Független gradiensellenőrzési protokoll egységben a teljes konvolúciós réteg forward és backward implementációját független numerikus gradiensellenőrzés alá vetjük. Az itteni mátrixos adjungált azonosítás adja az analitikus referenciaértéket, és a laborban használt `float64` toleranciaszint lesz a hibamérce alapja. A ReLU töréspontját (azt a helyet, ahol a ReLU nem differenciálható) és a nemsima pontokat külön elkerülő ellenőrzési protokollban a most levezetett lineáris rész pontos gradiense nélkülözhetetlen.

15.3.15 Felhasznált források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [D11] PyTorch contributors (2026). *ConvTranspose2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.ConvTranspose2d.html. Hozzáférés: 2026-10-30.
- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

15.4 Független gradiensellenőrzési protokoll

15.4.1 Cél, előismeret és induló kérdés

A 15.1.–15.2. egységben levezettük a konvolúciós réteg súly- és bemeneti gradiensét, a 15.3. Transposed convolution mint adjungált pedig az adjungált operátort azonosította a mátrixtranszponálttal. Egy levezetés azonban egymagában nem teszi a kódot helyessé: ugyanaz az indexelési hiba könnyen bekerül a képletbe és a referenciakódba is. Ebben az egységben ezért három különböző számítási úton kapott gradienst hasonlítunk össze (az analitikus és a numerikus út ugyanazt a kézi forwardot használja, ezért nem teljesen függetlenek; a keretrendszer-autograd a saját forwardját is ellenőrzi):

1. **analitikus képletből** származó zárt alakú backward;
2. **véges differenciából** származó numerikus közelítés;
3. a **keretrendszer autograd** implementációja.

A cél annak a protokollnak a rögzítése, amely megmondja, mikor fogadható el egy numerikus egyezés ellenőrzésnek, és mikor állítható róla, hogy csak részleges bizonyíték. A szükséges előismeret a 15.3. Transposed convolution mint adjungált mátrixos adjungáltja; visszautalunk a 14.4. Csoportos és depthwise konvolúció csoportos operátor korlátozására, a 11.4. Egy teljes PyTorch tanítási iteráció tanítási lépés gradienstárolási szerződésére és a 3.4. Sajátérték, SVD és spektrális határ normakorlátra.

Induló kérdés: miért választunk `float64`-et és centrális differenciát, ha a forward differencia egyszerűbb?

15.4.2 Definíciók: centrális differencia, paraméterrács, relatív hiba

Legyen $\theta \in \mathbb{R}^d$ a vizsgált paramétervektor, és legyen $g : \mathbb{R}^d \to \mathbb{R}$ a veszteségfüggvény. Jelölje $e_j \in \mathbb{R}^d$ a j-edik egységvektort, $\varepsilon > 0$ pedig a perturbációt. A **centrális differencia** a j-edik koordináta mentén:

$$\tilde{\partial}_j g(\theta;\varepsilon) = \frac{g(\theta + \varepsilon e_j) - g(\theta - \varepsilon e_j)}{2\varepsilon}.$$

A **forward differencia** ettől csak egyoldali:

$$\tilde{\partial}^{f}_j g(\theta;\varepsilon) = \frac{g(\theta + \varepsilon e_j) - g(\theta)}{\varepsilon}.$$

A kettő nem ugyanaz: az egyoldali változat hibája $O(\varepsilon)$, a centrálisé $O(\varepsilon^2)$, ahogyan az alábbi levezetésben látni fogjuk.

Paraméterrácson (parameter grid) itt nem a konvolúciós hiperparaméterek (stride, padding) rácsát értjük, hanem a veszteség paraméterterének több, függetlenül választott pontját. Egyetlen θ pontban az egyezés lehet véletlen vagy szimmetriahiba következménye; R darab, különböző skálájú és előjelű pontban való egyezés már függetlenebb bizonyíték. A rácspontokat rögzített seeddel generáljuk.

A numerikus egyezést a **stabil relatív hibával** mérjük:

$$E_{\mathrm{rel}}(a,n) = \frac{|a-n|}{\max\{|a|,|n|,\delta_{\mathrm{abs}}\}},$$

ahol a az analitikus, n a numerikus gradiens egy koordinátája, δ_{abs} pedig abszolút védőszint (például 10^{-12} `float64`-ben; a kódban ennek a neve `atol`). Nullához közeli gradiens esetén a tiszta relatív hiba zajosan felnagyítódik, ezért szükséges a stabil nevező.

A vizsgált célfüggvény akkor **sima**, ha g legalább kétszer folytonosan differenciálható a vizsgált pont egy környezetében. Az affin konvolúciós rétegre épülő $L = \frac{1}{2}\|Y\|_F^2$ veszteség a paraméterekben polinomiális, tehát C^∞; a ReLU-t tartalmazó háló viszont a nullpontban nem differenciálható, ezért a gradiensellenőrzést vagy olyan pontban kell végezni, ahol a pre-aktiváció egyetlen koordinátája sem nulla, vagy a ReLU-t tartalmazó réteget ki kell venni az ellenőrzött részgráfból. A centrális differencia másodrendű hibájáról szóló állítás C^3-at tesz fel; a C^2 és Lipschitz-folytonos második derivált esetét az integrálmaradékos alak ugyanígy kezeli.

A `float64` választása nem kényelmi kérdés. A véges differencia teljes hibája két tagból áll: a Taylor-levágásból származó $O(\varepsilon^2)$ diszkretizációs hibából és a lebegőpontos kerekítésből származó $O(u/\varepsilon)$ zajból, ahol u az egységnyi kerekítési hiba (unit roundoff), azaz a legközelebbi lebegőpontos

számra kerekítés relatív hibakorlátja. `float32`-ben $u \approx 6 \cdot 10^{-8}$ (a gépi epszilon ennek kétszerese, $\approx 1{,}2 \cdot 10^{-7}$), így $\varepsilon \approx 10^{-3}$ körül is már a kerekítési zaj dominálhat; `float64`-ben $u \approx 1{,}1 \cdot 10^{-16}$ (a gépi epszilon $\approx 2{,}2 \cdot 10^{-16}$), ezért $\varepsilon = 10^{-6}$ biztonságos egyensúlyt ad.

15.4.3 A tesztelt részhalmaz és a numerikus bizonyítás korlátja

A tesztelt művelet pontos részhalmaza a következő. Legyen a bemeneti batch $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, a kernel $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, a bias $b \in \mathbb{R}^{C_{out}}$. A forward a [D10] szerinti **cross-correlation**:

$$Y_{n,o,i,j} = b_o + \sum_{c=0}^{C_{in}-1} \sum_{r=0}^{k_h-1} \sum_{t=0}^{k_w-1} X_{n,c,\, s_h i - p_h + d_h r,\, s_w j - p_w + d_w t}\, K_{o,c,r,t},$$

ahol s_h, s_w a stride, p_h, p_w a szimmetrikus zero padding, d_h, d_w a dilation, r, t pedig a kernel sor- és oszlopindexe; n a batch, o a kimeneti csatorna, i, j a kimeneti pozíció indexe. A csatornacsoportok száma $G = 1$; a nem szimmetrikus padding és a csoportos művelet kívül esik ezen az állításon. A kód ezt `assert`-tel rögzíti: a `conv2d_forward` és a `conv2d_backward_weight` a kernel csatornaszámát a bemenet csatornaszámához köti, ezért a labor csak `groups=1` mellett érvényes. A veszteség a kézi minta konvenciója szerint

$$L = \frac{1}{2} \sum_{n,o,i,j} Y_{n,o,i,j}^2,$$

ezért a kimeneti gradiens egyszerűen $\nabla_Y L = Y$; ez nem MSE, hanem a kézi ellenőrzést egyszerűsítő, ebben az egységben rögzített választás.

A backward analitikus képlete ekkor a 15.1.–15.3. eredménye:

$$\nabla_K L = X \star_{\text{valid}} Y, \qquad \nabla_b L_o = \sum_{n,i,j} Y_{n,o,i,j}, \qquad \nabla_X L = Y \star_{\text{full}} \operatorname{rot}_{180}(K) = Y *_{\text{full}} K,$$

ahol $\star_{\text{valid}}$ a valid, $\star_{\text{full}}$ a full cross-correlation, $*_{\text{full}}$ pedig a full **matematikai konvolúció**. Mivel a forward cross-correlation, az input gradiens a K **eredeti** kernellel vett full konvolúció, ami ekvivalens a 180°-kal elforgatott kernellel vett full cross-correlationnel; az eredeti orientációjú full cross-correlation hamis lenne. A tömör képletekben az izotrop jelölést használjuk: $s = s_h = s_w$, $d = d_h = d_w$, $p = p_h = p_w$; ezek a képletek $s = 1$, $d = 1$, $p = 0$ esetén érvényesek. Stride, dilation és padding mellett upsampling (nulla beszúrás), kivágás és csatornaösszegzés kell, ezért ott az explicit indexelt képletet használjuk: $\nabla_X L[n,c,\rho,\sigma] = \sum_{o,i,j,u,v} \nabla_Y L[n,o,i,j]\, K[o,c,u,v]$, ahol a forward összefüggésből $\rho = is_h + ud_h - p_h$ és $\sigma = js_w + vd_w - p_w$; rögzített (ρ, σ, u, v) mellett tehát $i = (\rho + p_h - ud_h)/s_h$, $j = (\sigma + p_w - vd_w)/s_w$, és csak az egész, érvényes (i, j) tagokat adjuk össze.

Állítás (a centrális differencia másodrendű hibája). Ha $g \in C^3$ a θ pont egy ε sugarú környezetében (a harmadik derivált folytonos), akkor minden j koordinátára

$$\tilde{\partial}_j g(\theta;\varepsilon) = \frac{\partial g}{\partial \theta_j}(\theta) + \frac{\varepsilon^2}{6} \frac{\partial^3 g}{\partial \theta_j^3}(\xi) + O\Big(\frac{u}{\varepsilon}\Big),$$

valamely ξ közbülső ponttal, így a diszkretizációs tag $O(\varepsilon^2)$, szemben a forward differencia $O(\varepsilon)$ tagjával. A kerekítési tag felső becslése $O(u/\varepsilon)$ (az u egységnyi kerekítési hiba és a függvényérték skálájától függ); a Big-O nem alsó korlát, a tényleges kerekítési hiba akár nulla is lehet.

Levezetés. A Taylor-formula a harmadik taggal:

$$g(\theta + \varepsilon e_j) = g(\theta) + \varepsilon g_j' + \frac{\varepsilon^2}{2} g_j'' + \frac{\varepsilon^3}{6} g_j'''(\xi_+),$$
$$g(\theta - \varepsilon e_j) = g(\theta) - \varepsilon g_j' + \frac{\varepsilon^2}{2} g_j'' - \frac{\varepsilon^3}{6} g_j'''(\xi_-).$$

A kivonás után a $g(\theta)$ és az $\varepsilon^2 g_j''$ tag kiesik, és

$$\frac{g(\theta + \varepsilon e_j) - g(\theta - \varepsilon e_j)}{2\varepsilon} = g_j' + \frac{\varepsilon^2}{12}(g_j'''(\xi_+) + g_j'''(\xi_-)) = g_j' + \frac{\varepsilon^2}{6} g_j'''(\xi),$$

ahol az utolsó lépés a folytonosságból következő közbülsőérték-tétel. A hibatag tehát $O(\varepsilon^2)$. A lebegőpontos kerekítésből származó $O(u/\varepsilon)$ tag abból következik, hogy a két g-kiértékelés külön-külön $O(u)$ abszolút hibájú, és ezt 2ε-nal osztjuk. $\square$

A C^3 feltétel a Taylor-maradékos levezetéshez kell; ha g'' létezik és Lipschitz-folytonos, az integrálmaradékos Taylor-alak ugyanazt az $O(\varepsilon^2)$ becslést adja.

Ennek a numerikus kontrollnak **korlátozott az érvényessége**: az egyezés véges sok rácspontban, véges ε mellett áll fenn. Nem helyettesíti az analitikus levezetést, nem zárja ki a nullmértékű halmazon jelentkező hibát, és nem igazolja a `stride`, `dilation`, `padding` összes lehetséges kombinációját. Olyan esetben azonban, amikor a három forrás közül kettő egymástól független (például a numerikus differencia és az autograd), a közös hibát valószínűtlenné teszi.

15.4.4 Eredeti kézi példa

Legyen $N = C_{in} = C_{out} = 1$, $H = W = 3$, $k_h = k_w = 2$, stride 1, padding 0, dilation 1. Az indexelés 0-tól indul: a 3×3-as mátrixban $X_{1,1}$ a középső elemet (az 5-öt) jelöli, nem a bal felsőt. Bemenet és kernel:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad K = \begin{bmatrix} 0{,}5 & -0{,}5 \\ 1 & 0{,}25 \end{bmatrix}.$$

A forward $Y \in \mathbb{R}^{2\times 2}$:

$$Y = \begin{bmatrix} 4{,}75 & 6{,}0 \\ 8{,}5 & 9{,}75 \end{bmatrix}.$$

A veszteség $L = \frac{1}{2}\sum_{i,j} Y_{ij}^2$. Az input gradiens analitikus képlete full matematikai konvolúció az eredeti kernelrel (a 180°-kal elforgatott kernelű full cross-correlation):

$$\nabla_X L[1,1] = Y_{00}K_{11} + Y_{01}K_{10} + Y_{10}K_{01} + Y_{11}K_{00} = 4{,}75\cdot 0{,}25 + 6\cdot 1 + 8{,}5\cdot(-0{,}5) + 9{,}75\cdot 0{,}5 = 7{,}8125.$$

Ellenőrzés centrális differenciával $\varepsilon = 0{,}01$ mellett, $X_{1,1}$ perturbálásával:

$$X_+ = X + \varepsilon e_{1,1}, \quad X_- = X - \varepsilon e_{1,1},$$

$$Y_+ = \begin{bmatrix} 4{,}7525 & 6{,}01 \\ 8{,}495 & 9{,}755 \end{bmatrix}, \qquad Y_- = \begin{bmatrix} 4{,}7475 & 5{,}99 \\ 8{,}505 & 9{,}745 \end{bmatrix}.$$

$$L_+ = 113{,}015703125, \qquad L_- = 112{,}859453125,$$

$$\frac{L_+ - L_-}{2\varepsilon} = \frac{0{,}15625}{0{,}02} = 7{,}8125.$$

A két érték az adott pontosság mellett pontosan egyezik. Ugyanezt a súlygradiensre is elvégezhetjük: $\nabla_K L[0,0] = Y_{00}X_{00} + Y_{01}X_{01} + Y_{10}X_{10} + Y_{11}X_{11} = 4{,}75 \cdot 1 + 6 \cdot 2 + 8{,}5 \cdot 4 + 9{,}75 \cdot 5 = 99{,}5$. A K_{00} perturbálásával, $\varepsilon = 0{,}01$ mellett $L(K_{00} + \varepsilon) = 113{,}9348$ és $L(K_{00} - \varepsilon) = 111{,}9448$, ezért a centrális differencia $(113{,}9348 - 111{,}9448)/0{,}02 = 99{,}5$. Az egyezés azért pontos, mert L a K_{00}-ban kvadratikus, így a centrális differencia a másodrendű tagot kiküszöböli. A példa azt is megmutatja, hogy az input gradienst a K-val vett full **konvolúció** (azaz a 180°-kal elforgatott kernelű full cross-correlation) adja; az eredeti orientációjú full cross-correlation a Y és a K azonos indexű elemeit párosítja, $\sum_{i,j} Y_{ij}K_{ij} = 10{,}3125$-öt ad, miközben az adjungálthoz a fordított indexű $K_{1-i,1-j}$ kell; ezért ez az érték nem $\nabla_X L[1,1]$, hanem egy másik lineáris leképezés kimenete.

15.4.5 Labor: hármas egyezés és JSON-jelentés

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `gradcheck_protocol.py` futtatható (`python gradcheck_protocol.py`); a JSON-jelentés a futás kimenete.

Az alábbi kód a 15.1.–15.2. saját backwardjét használja analitikus referenciaként, a centrális differenciát numerikus kontrollként, és a PyTorch `autograd.grad`-ját [D05] keretrendszerkontrollként. A kód `float64`-ben fut, CPU-n, rögzített seeddel, és a hibákat JSON-jelentésbe írja. A kódot a rögzített labor-környezetben futtattuk (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed `20261020`); a négy eset mindegyikében a maximális relatív hiba 10^{-5} alatt maradt (a legnagyobb: $1{,}32 \cdot 10^{-7}$), a kézi forward és a `F.conv2d` kimenete közötti maximális abszolút eltérés legfeljebb $2{,}3 \cdot 10^{-16}$, és a jelentés `all_passed_tol_1e-5` mezője `true`. A teljes futás a tiszta Python-ciklusok miatt 8,3 s volt, a csúcs-RSS pedig 241 MB.

```
# munkakönyvtár: lab/
# futtatás: python gradcheck_protocol.py
import json
import torch

torch.manual_seed(20261020)

def conv2d_forward(X, K, b, stride=1, pad=0, dilation=1):
    N, C, H, W = X.shape
    O, Ck, kH, kW = K.shape
    assert Ck == C, "a labor csak groups=1-et tamogat"
    Ho = (H + 2*pad - dilation*(kH - 1) - 1)//stride + 1
    Wo = (W + 2*pad - dilation*(kW - 1) - 1)//stride + 1
    Y = torch.zeros(N, O, Ho, Wo, dtype=torch.float64)
    for n in range(N):
        for o in range(O):
            for i in range(Ho):
```

```
                for j in range(Wo):
                    s = 0.0
                    for c in range(C):
                        for p in range(kH):
                            for q in range(kW):
                                ii = i*stride - pad + p*dilation
                                jj = j*stride - pad + q*dilation
                                if 0 <= ii < H and 0 <= jj < W:
                                    s += X[n, c, ii, jj].item() * K[o, c, p, q].item()
                    Y[n, o, i, j] = s + b[o].item()
    return Y

def conv2d_backward_input(Y, K, X_shape, stride=1, pad=0, dilation=1):
    N, O, Ho, Wo = Y.shape
    _, _, kH, kW = K.shape
    N, C, H, W = X_shape
    dX = torch.zeros(N, C, H, W, dtype=torch.float64)
    for n in range(N):
        for c in range(C):
            for i in range(H):
                for j in range(W):
                    s = 0.0
                    for o in range(O):
                        for p in range(kH):
                            for q in range(kW):
                                # Y_{n,o,r,s} járulék, ahol r*stride - pad + p*dilation == i
                                # r = (i + pad - p*dilation)/stride, ha egész
                                if (i + pad - p*dilation) % stride != 0:
                                    continue
                                r = (i + pad - p*dilation)//stride
                                if (j + pad - q*dilation) % stride != 0:
                                    continue
                                s_ = (j + pad - q*dilation)//stride
                                if 0 <= r < Ho and 0 <= s_ < Wo:
                                    s += Y[n, o, r, s_].item() * K[o, c, p, q].item()
                    dX[n, c, i, j] = s
    return dX

def conv2d_backward_weight(X, Y, K_shape, stride=1, pad=0, dilation=1):
    N, C, H, W = X.shape
    N, O, Ho, Wo = Y.shape
    O, Cg, kH, kW = K_shape
    assert Cg == C, "a labor csak groups=1-et tamogat"
    dK = torch.zeros(K_shape, dtype=torch.float64)
    db = torch.zeros(O, dtype=torch.float64)
    for o in range(O):
        for c in range(Cg):
            for p in range(kH):
                for q in range(kW):
                    s = 0.0
                    for n in range(N):
                        for i in range(Ho):
                            for j in range(Wo):
                                ii = i*stride - pad + p*dilation
                                jj = j*stride - pad + q*dilation
                                if 0 <= ii < H and 0 <= jj < W:
                                    s += X[n, c, ii, jj].item() * Y[n, o, i, j].item()
                    dK[o, c, p, q] = s
        db[o] = Y[:, o, :, :].sum().item()
    return dK, db

def loss_of(X, K, b, stride, pad, dilation):
    Y = conv2d_forward(X, K, b, stride, pad, dilation)
```

```
    return 0.5 * (Y**2).sum()

def finite_difference_flat(X, K, b, stride, pad, dilation, eps=1e-6):
    # a paraméterek lapított vektorán centrális differencia
    flat = torch.cat([X.reshape(-1), K.reshape(-1), b.reshape(-1)]).detach().clone()
    grad = torch.zeros_like(flat)
    for j in range(flat.numel()):
        orig = flat[j].item()
        flat[j] = orig + eps
        Lp = loss_of(flat[:X.numel()].reshape(X.shape),
                     flat[X.numel():X.numel()+K.numel()].reshape(K.shape),
                     flat[-b.numel():], stride, pad, dilation)
        flat[j] = orig - eps
        Lm = loss_of(flat[:X.numel()].reshape(X.shape),
                     flat[X.numel():X.numel()+K.numel()].reshape(K.shape),
                     flat[-b.numel():], stride, pad, dilation)
        flat[j] = orig
        grad[j] = (Lp - Lm)/(2*eps)
    return grad

def rel_err(a, n, atol=1e-12):
    a = float(a); n = float(n)
    return abs(a - n) / max(abs(a), abs(n), atol)

def run_case(shape, kshape, stride, pad, dilation, tag):
    N, C, H, W = shape
    O = kshape[0]
    X = torch.rand(N, C, H, W, dtype=torch.float64) * 2 - 1
    K = (torch.rand(*kshape, dtype=torch.float64) * 2 - 1) / (C * kshape[2] * kshape[3])**0.5
    b = torch.rand(O, dtype=torch.float64) * 0.2 - 0.1

    # 1. analitikus
    Y = conv2d_forward(X, K, b, stride, pad, dilation)
    dX = conv2d_backward_input(Y, K, X.shape, stride, pad, dilation)
    dK, db = conv2d_backward_weight(X, Y, K.shape, stride, pad, dilation)
    anal = torch.cat([dX.reshape(-1), dK.reshape(-1), db.reshape(-1)])

    # 2. numerikus
    num = finite_difference_flat(X, K, b, stride, pad, dilation)

    # 3. autograd
    Xa = X.clone().requires_grad_(True)
    Ka = K.clone().requires_grad_(True)
    ba = b.clone().requires_grad_(True)
    Ya = torch.nn.functional.conv2d(Xa, Ka, ba, stride=stride,
                                    padding=pad, dilation=dilation)
    La = 0.5 * (Ya**2).sum()
    grads = torch.autograd.grad(La, [Xa, Ka, ba])
    auto = torch.cat([g.reshape(-1) for g in grads])
    e_y = float((Y - Ya.detach()).abs().max())

    # relatív hibák
    e_an = max(rel_err(a, n) for a, n in zip(anal, num))
    e_aa = max(rel_err(a, n) for a, n in zip(anal, auto))
    e_na = max(rel_err(n, a) for n, a in zip(num, auto))
    return {"tag": tag, "shape": list(shape), "kernel_shape": list(kshape),
            "stride": stride, "padding": pad, "dilation": dilation,
            "max_abs_forward_vs_autograd": e_y,
            "max_rel_anal_vs_numeric": e_an,
            "max_rel_anal_vs_autograd": e_aa,
            "max_rel_numeric_vs_autograd": e_na,
            "checked_scalar_coordinates": X.numel() + K.numel() + b.numel()}
```

```
cases = [
    ((1, 1, 3, 3), (1, 1, 2, 2), 1, 0, 1, "small_stride1"),
    ((2, 2, 6, 6), (3, 2, 3, 3), 1, 1, 1, "padded_batch"),
    ((1, 1, 7, 7), (1, 1, 3, 3), 2, 0, 1, "stride2"),
    ((1, 1, 6, 6), (1, 1, 2, 2), 1, 0, 2, "dilation2"),
]

report = {"protocol_version": "1.0", "dtype": "float64",
          "finite_difference": "central", "eps": 1e-6,
          "relative_error_formula": "|a-n|/max(|a|,|n|,1e-12)",
          "cases": [run_case(*c) for c in cases]}
report["all_passed_tol_1e-5"] = all(
    max(c[k] for k in ("max_rel_anal_vs_numeric",
                       "max_rel_anal_vs_autograd",
                       "max_rel_numeric_vs_autograd")) < 1e-5
    and c["max_abs_forward_vs_autograd"] < 1e-12
    for c in report["cases"])
assert report["all_passed_tol_1e-5"], report["cases"]

with open("gradcheck_protocol.json", "w", encoding="utf-8") as f:
    json.dump(report, f, indent=2, ensure_ascii=False)
print(json.dumps(report, indent=2, ensure_ascii=False)[:1200])
```

A `run_case` minden esethez három különböző számítási úton kapott gradienst hasonlít össze; az analitikus és a numerikus út közös kézi forwardja miatt ez a két út nem független, ezért a kód a kézi forward kimenetét a `F.conv2d` kimenetével is összeveti (`max_abs_forward_vs_autograd`). A `small_stride1` eset a fenti kézi példa méretosztálya; a `padded_batch` a biasgradiens batchelt összegzését is teszteli; a `stride2` és `dilation2` további hiperparaméter-konfigurációk, nem a paraméterrács pontjai. A `checked_scalar_coordinates` mező a finite-difference ellenőrzésbe bevont skalár koordináták számát adja; a labor esetenként egyetlen véletlen θ-pontot vizsgál. Az elfogadási küszöb 10^{-5} relatív hiba `float64` central difference mellett; nagyobb eltérés implementációs hibát, nem sima pontot vagy rosszul skálázott ε-t is jelezhet. A mért JSON-jelentés a fenti mezőket tartalmazza, és az `all_passed_tol_1e-5` mező `true` értéke a hármas egyezés indikátora.

A mért jelentés egy esete (a `padded_batch`) a következő; a teljes jelentés a `gradcheck_protocol.json` fájlba íródik, és a négy eset mellett a `protocol_version`, `dtype`, `finite_difference`, `eps` és `relative_error_formula` mezőket is tartalmazza:

```
{
  "tag": "padded_batch",
  "shape": [2, 2, 6, 6],
  "kernel_shape": [3, 2, 3, 3],
  "stride": 1,
  "padding": 1,
  "dilation": 1,
  "max_abs_forward_vs_autograd": 2.220446049250313e-16,
  "max_rel_anal_vs_numeric": 1.3188704242991868e-07,
  "max_rel_anal_vs_autograd": 1.6192770297173516e-14,
  "max_rel_numeric_vs_autograd": 1.3188704657262395e-07,
  "checked_scalar_coordinates": 201
}
```

15.4.6 Feladatok

F1 (fogalmi, nehézség 1). Fogalmazza meg, mi a különbség a forward és a centrális differencia között, és magyarázza el, miért használunk stabil nevezőt a relatív hibában. Adjon egy konkrét g-t és ε-t, ahol a tiszta $|a-n|/|a|$ képlet nullához közeli gradiensnél félrevezető értéket adna.

F2 (levezetési, nehézség 2). Legyen $g(x) = x^3$, $x_0 = 2$, $\varepsilon = 0{,}1$. Számítsa ki a forward és a centrális differencia értékét, és hasonlítsa mindkettőt a pontos $g'(2)$ értékhez. Melyik hibája arányos ε-nal, és melyiké ε^2-tel? A Taylor-sorból indokolja a megfigyelt arányt.

F3 (implementációs, nehézség 2). Írjon teljes Python-függvényt, amely az $L = \frac{1}{2}\sum Y^2$ veszteségre a fenti 3×3 bemenet és 2×2 kernel esetén kiszámítja a centrális differenciát az $X_{1,1}$ koordinátára, és ellenőrzi, hogy az eredmény 7,8125 tűrésen belül. A függvény ne használjon PyTorch-autogradot.

F4 (hibakeresési, nehézség 2). Valaki a következő ellenőrzést írta: `float32` típus, forward differencia, $\varepsilon = 10^{-3}$, és a háló egy `ReLU`-t is tartalmaz, amelynek pre-aktivációja a vizsgált pontban pontosan 0. Az ellenőrzés relatív hibája 0,35. Sorolja fel a legalább három különböző hibaforrást, mindegyik következményét és a javítás módját.

F5 (kísérlettervezési, nehézség 3). Tervezzen gradiensellenőrzési protokollt egy olyan rétegre, amely stride 2, dilation 2 és padding 1 mellett fut, $C_{in} = 3$, $C_{out} = 5$, $k_h = k_w = 3$, bemenet $N = 4$, $H = W = 10$. Határozza meg: hány rácspontot választ, mekkora ε-t, milyen toleranciát, és milyen korlátok miatt nem tekinthető az egyezés teljes bizonyításnak. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

15.4.7 Megoldások és indoklás

F1. A forward differencia egyoldali: $(g(\theta + \varepsilon e_j) - g(\theta))/\varepsilon$; hibája $O(\varepsilon)$. A centrális kétoldali: $(g(\theta+\varepsilon e_j) - g(\theta-\varepsilon e_j))/(2\varepsilon)$; a szimmetria miatt a másodrendű tag kiesik, hibája $O(\varepsilon^2)$. A stabil nevező azért kell, mert nullához közeli gradiensnél a tiszta relatív hiba félrevezető lehet. Konkrét példa: $g(x) = 1 + 10^{-14}x$, $x_0 = 0$, $\varepsilon = 10^{-3}$. A pontos derivált $a = 10^{-14}$; `float64`-ben $g(\pm\varepsilon)$ mindkettő 1,0-ra kerekítődik (a 10^{-17} nagyságú eltérés a 1 körüli ulp ($2{,}2 \cdot 10^{-16}$) alatt van), ezért a centrális differencia $n = 0$. A tiszta $|a - n|/|a|$ hiba így 1 (100%), noha az abszolút eltérés csak 10^{-14}; a $\delta_{\text{abs}} = 10^{-12}$ védőszinttel a stabil relatív hiba $|10^{-14} - 0|/\max(10^{-14}, 0, 10^{-12}) = 10^{-2}$. Az értéket a megadott venv Pythonjával, `float64`-ben ellenőriztük.

F2. Pontos derivált: $g'(2) = 3 \cdot 2^2 = 12$. Forward: $\frac{g(2{,}1)-g(2)}{0{,}1} = \frac{9{,}261-8}{0{,}1} = 12{,}61$; hiba $0{,}61 = 6{,}1\varepsilon$, mert a Taylor-maradék $\frac{\varepsilon}{2}g''(2) = \frac{0{,}1}{2} \cdot 12 = 0{,}6$. Centrális: $\frac{g(2{,}1)-g(1{,}9)}{0{,}2} = \frac{9{,}261-6{,}859}{0{,}2} = 12{,}01$; hiba $0{,}01 = \varepsilon^2$, mert a maradék $\frac{\varepsilon^2}{6}g'''(2) = \frac{0{,}01}{6} \cdot 6 = 0{,}01$. A megfigyelt hibaarány pontosan a levezetett rendet tükrözi.

F3. Elfogadható megoldás:

```
# munkakönyvtár: lab/
# futtatás: python f3_hand_check.py
import torch

def conv_valid_2x2(X, K):
    Y = torch.zeros(2, 2, dtype=torch.float64)
    for i in range(2):
        for j in range(2):
            Y[i, j] = (X[i:i+2, j:j+2] * K).sum()
    return Y

X = torch.tensor([[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]],
                 dtype=torch.float64)
K = torch.tensor([[0.5, -0.5], [1.0, 0.25]], dtype=torch.float64)
eps = 1e-6
L_of = lambda A: 0.5 * (conv_valid_2x2(A, K)**2).sum()
Xp = X.clone(); Xp[1, 1] += eps
```

```
Xm = X.clone(); Xm[1, 1] -= eps
grad = (L_of(Xp) - L_of(Xm)) / (2 * eps)
assert abs(grad - 7.8125) < 1e-5, f"eltérés: {grad - 7.8125}"
print(f"grad = {grad:.8f}, elvárt 7.8125")
```

A blokk mért kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`): `grad = 7.81250001, elvárt 7.8125`. A `float64` és $\varepsilon = 10^{-6}$ választás miatt a kerekítési zaj $10^{-16}/10^{-6} = 10^{-10}$ nagyságrendű, így a 10^{-5} tűrés biztonságos. Alakellenőrzés: a gradiens skalár, a bemenet 3×3, a kimenet 2×2, minden összegzés határa a kerneldimenziókból következik.

F4. Hibaforrások: (1) `float32`-ben a kerekítési zaj $10^{-7}/10^{-3} \approx 10^{-4}$ relatív hibát okoz, ami önmagában jóval kisebb a megfigyelt 0,35-nél, de rontja a numerikus jel/zaj viszonyt; javítás `float64`-re és $\varepsilon = 10^{-6}$-ra. (2) A forward differencia $O(\varepsilon)$ hibája ε-nal arányos, $\varepsilon = 10^{-3}$ mellett jellemzően 10^{-3}–10^{-4} nagyságrendű; javítás centrális differenciára. (3) A ReLU nullpontjában a klasszikus derivált nem létezik, a numerikus differencia a kétoldali szelőmeredekséget adja, ami az implementáció egyoldali konvenciójától $O(1)$ relatív mértékben eltérhet; ez adhatja a 0,35 nagyságrendjét. Javítás: a ReLU-t ki kell hagyni a veszteségből, vagy olyan pontot választani, ahol a pre-aktiváció nullától legalább ε nagyságrenddel távolabb van. A 0,35 relatív hiba így elsősorban a (3) következménye lehet, a másik két tényező hozzájárulása ennél kisebb.

F5. Tervminta. A megadott konfigurációban a forward kimenete a méretképlet szerint $H_{out} = W_{out} = \lfloor (10+2\cdot 1-2\cdot(3-1)-1)/2 \rfloor + 1 = 4$. Három független seedet használunk, és mindegyikből három, véletlenül választott paraméterpontot veszünk, így összesen kilenc rácspontot vizsgálunk; a seedek rögzítettek, a számítás `float64`, $\varepsilon = 10^{-6}$, a tolerancia 10^{-5} stabil relatív hiba. A kilenc pont indoka: a 3×3 kernelnél a stride-dilation kombináció indexelési hibái csak több, különböző előjelű pontban mutatkoznak meg. A tervezett esetek között legyen legalább egy olyan, amelyben a stride, a dilation és a padding egyszerre szerepel, mert a labor négy esete ezeket külön-külön teszteli, az együttes hatást nem. Korlátok: az ellenőrzés csak a választott $N = 4$, $H = W = 10$ alakra érvényes; a batchelt biasgradiens összegzése a véletlen előjelek miatt részben kioltódhat, ezért célzott összegteszt is kell; a véges ε miatt a 10^{-5} alatti egyezés sem teljes bizonyítás, csak numerikus evidenciaszint; a nullmértékű paraméterhalmazon rejtőző hiba nem zárható ki. A protokollt ki kell egészíteni az analitikus backward levezetésének szemantikai ellenőrzésével.

15.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

A leggyakoribb tévedés, hogy **két azonos hibás implementáció egyezését** helyességnek tekintjük. Ha a saját backward és a referenciakód ugyanazt a flipped-kernel hibát tartalmazza, az egyezés tökéletes, mégsem bizonyít semmit. A centrális differencia azért erős kontroll, mert nem ismeri a képletet: csak a forwardot értékeli ki. Ugyanakkor a numerikus egyezés sem általános bizonyítás: véges ε és véges rácspontok mellett csak a tesztelt részhalmazra érvényes. Ezt a kettős korlátot a jelentésben mindig expliciten meg kell nevezni.

Önellenőrzés: a kézi példában a `full cross-correlation` orientációját érdemes újra levezetni egyetlen koordinátára, mert az eredeti orientációjú full cross-correlation használata a leggyakoribb hiba. A fenti $X_{1,1}$ példa kimutatja: a 180°-kal elforgatott kernelű full cross-correlation ad 7,8125-öt, az eredeti orientációjú pedig 10,3125-öt.

Visszatekintés: a 15.3. Transposed convolution mint adjungált adjungált azonossága garantálta, hogy a bemeneti gradiens a forward operátor transzponáltja; ez a mostani full cross-correlation éppen ennek az indexszintű alakja. A 14.4. Csoportos és depthwise konvolúció `groups=1` korlátozása miatt a csatornaösszegzés a fenti képletekben nem bontható blokkokra. A 11.4. Egy teljes PyTorch tanítási iteráció tanítási lépéséből tudjuk, hogy a `backward` önmagában nem frissíti

a paramétereket, ezért a gradiensellenőrzéshez nincs is szükség `optimizer`re. A 3.4. Sajátérték, SVD és spektrális határ normakorlátja pedig emlékeztet: ha C a forward lineáris operátormátrixa (a bias nélkül), akkor $\|\nabla_X L\|_2 = \|C^\top Y\|_2 \le \|C\|_2 \|Y\|_2$, és $\|Y\|_2 \le \|C\|_2 \|X\|_2 + \sqrt{NH_{out}W_{out}}\,\|b\|_2$, ezért $\|\nabla_X L\|_2 \le \|C\|_2^2 \|X\|_2 + \|C\|_2 \sqrt{NH_{out}W_{out}}\,\|b\|_2$. A gradiensskálát tehát a kernel normája (a $\|C\|_2$-n keresztül) és a bemenet normája együtt határozza meg, ezért a rácspontok skálázását a jelentésben rögzíteni kell.

15.4.9 Rövid összefoglaló

A centrális differencia másodrendű Taylor-maradékát levezettük, és megmutattuk, hogy a diszkretizációs hiba $O(\varepsilon^2)$, szemben a forward differencia $O(\varepsilon)$ hibájával. A stabil relatív hiba nullához közeli gradiensnél is értelmezhető marad. A konvolúciós backward ellenőrzése három különböző számítási úton történik: analitikus képlet, numerikus differencia és keretrendszer-autograd; az analitikus és a numerikus út közös forwardot használ, ezért nem teljesen független. A labor `float64`-ben, rögzített seeddel, paraméterrácson fut, és JSON-jelentésbe menti a maximális relatív hibákat. A numerikus egyezés csak a kimondott részhalmazra, véges ε és véges rácspontok mellett érvényes; nem helyettesíti az analitikus levezetést. Két azonos hibás implementáció egyezése önmagában gyenge kontroll.

15.4.10 Következő alkalmazás

A következő, 16.1. Eltolási equivariance és invariance egység az eltolási ekvivariancia (equivariance) és invariancia (invariance) fogalmát vizsgálja. Az itteni gradiensellenőrzési protokoll arra is használható, hogy egy ekvivariancia-állítás numerikus sérülését kimutassuk a határfeltételeknél: a full cross-correlation gradiens és a stride-részcsoport (azoknak a bemeneti pozícióknak a halmaza, amelyek indexe s-sel osztva ugyanazt a maradékot adja) szerinti eltolás kapcsolatát az itt bevezetett `float64` mérőszámmal ellenőrizhetjük.

15.4.11 Források

- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

16 Szimmetria, pooling és recepciós mező

16.1 Eltolási equivariance és invariance

16.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben két, gyakran összekevert fogalmat választunk szét: az **eltolási ekvivarianciát** (shift equivariance) és az **eltolási invarianciát** (shift invariance). Az előbbi azt mondja meg, hogyan viselkedik egy operátor kimenete, ha a bemenetet eltoljuk; az utóbbi azt, hogy a kimenet egyáltalán nem változik. A megkülönböztetés nem szóhasználati kérdés: a CNN-rétegek tervezésekor éppen az a cél, hogy a konvolúciós réteg ekvivariáns legyen, a hálózat végén lévő aggregáló rétegek pedig a kívánt mértékű invarianciát hozzák létre.

Előfeltételek: a 13.4. Határkezelés és frekvenciaintuíció egységből a véges képek határkezelése (valid, zero és circular padding), a 14.4. Csoportos és depthwise konvolúció egységből a csatornák közötti kapcsolat korlátozása, a 15.4. Független gradiensellenőrzési protokoll egységből a numerikus ellenőrzés fegyelme. Visszautalunk a 15.1. Súly- és biasgradiens súlymegosztási gondolatára, a 12.1. Tanítás, validáció és kiértékelési mód train/eval megkülönböztetésére és a 4.1. Függvény, határérték és derivált függvényfogalmára.

Induló kérdés: ha egy 24×24 pixeles, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-képet három képponttal jobbra tolunk, akkor egy 3×3-as szűrő kimeneti térképe is pontosan három képponttal tolódik-e el, és a végső osztályozási logit változatlan marad-e? A válasz első fele feltételek mellett igen, második fele általában nem — ezt a megkülönböztetést dolgozzuk ki.

16.1.2 Az eltolási operátor és a cross-correlation definíciója

A jelölést a 14.2. Több bemeneti és kimeneti csatorna egység NCHW-konvenciója szerint rögzítjük, de a levezetéshez a batchindexet elhagyjuk, mert az eltolás csatornánként és képenként függetlenül hat. Legyen a bemeneti kép

$$X \in \mathbb{R}^{C_{in} \times H \times W}.$$

A kernel $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, a bias $b \in \mathbb{R}^{C_{out}}$. A használt operátor a PyTorch `Conv2d` által számolt **cross-correlation**, kernelmegfordítás nélkül; a 13.2. Konvolúció és cross-correlation egységben rögzítettük, hogy ez nem azonos a matematikai konvolúcióval, de a szimmetriafogalmak szempontjából a kernelmegfordítás egy rögzített transzformáció, ezért az equivariance-állítás mindkét változatra azonos szerkezetű.

Definíció (valid cross-correlation, stride 1, padding 0). A $Z = F(X)$ kimenet alakja $C_{out} \times H' \times W'$, ahol $H' = H - k_h + 1$, $W' = W - k_w + 1$, és

$$Z_{o,i,j} = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{o,c,u,v}\, X_{c,i+u,j+v} + b_o, \qquad i \in [0, H'-1],\ j \in [0, W'-1]. \tag{1}$$

Itt u, v a kernel sor- és oszlopindexe; a p_h, p_w paddingjelöléstől való megkülönböztetés érdekében a kernelindexeket külön betűvel jelöljük. Az egyes kimeneti pozíciókhoz tartozó $k_h \times k_w$ méretű bemeneti ablakot **recepciós mezőnek** nevezzük; a 16.3. Elméleti és effektív recepciós mező egységben ezt általánosabban vizsgáljuk.

Definíció (eltolási operátor). Legyen $\tau = (\tau_h, \tau_w) \in \mathbb{Z}^2$. A T_τ operátor a végtelen nulla-kiterjesztésű rácson hat:

$$(T_\tau X)_{c,u,v} = X_{c,u-\tau_h,v-\tau_w}, \tag{2}$$

ahol a definíció szerint az eredeti $[0, H-1] \times [0, W-1]$ tartományon kívüli indexekre X értéke nulla. A véges képek világában ez azt jelenti, hogy a kép tartalma a (τ_h, τ_w) vektorral eltolódik, a felszabaduló sáv pedig nullákkal töltődik fel. Ha $\tau_h, \tau_w \geq 0$, akkor

$$(T_\tau X)_{:,u,v} = X_{:,u-\tau_h,v-\tau_w} \quad \text{ha } u \geq \tau_h,\ v \geq \tau_w,$$

és nulla egyébként. A kép mérete változatlan marad. Más eltolási változatok — crop, circular — a laborban szerepelnek; ezek más operátort definiálnak, és az equivariance-állítás határfeltételei eltérnek.

Az eltolás itt a **térbeli indexekre** hat, a csatornaindexet nem érinti. A csatornák közötti permutáció egy másik szimmetria lenne, amellyel itt nem foglalkozunk.

16.1.3 Ekvivariancia és invariancia mint operátoregyenlet

Legyen F egy tetszőleges operátor, amely képeket képekre, vagy képeket skalárokra képez.

Definíció (eltolási ekvivariancia). Az F operátor **eltolási ekvivariáns** az eltolások egy $\mathcal{T}$ halmazára nézve, ha minden $\tau \in \mathcal{T}$ esetén

$$F(T_\tau X) = T_\tau F(X) \tag{3}$$

minden olyan X-re és kimeneti indexre, ahol mindkét oldal értelmezett. A T_τ a kimeneti oldalon a kimeneti rácsnak megfelelő eltolást jelöli; ha a kimeneti rács felbontása eltér, ezt külön rögzíteni kell.

Az egyenlet szemléletes jelentése: **a szűrő alkalmazása és az eltolás felcserélhető**. Ha előbb eltolom a képet, majd szűröm, ugyanazt kapom, mintha előbb szűrném, majd a kimeneti térképet tolnám el.

Definíció (eltolási invariancia). Az F operátor **eltolási invariáns** az eltolások $\mathcal{T}$ halmazára nézve, ha minden $\tau \in \mathcal{T}$ esetén

$$F(T_\tau X) = F(X). \tag{4}$$

Itt a kimenet **nem változik meg** az eltolás hatására. Tipikus példa a globális átlagolás: ha a kép minden pixelét átlagoljuk, a körkörös (tartalmat megtartó) eltolás nem változtatja meg az átlagot, mert csak a pixelek sorrendjét cseréli fel. A nulla-kitöltéses eltolásnál viszont a kép széléről kikerülő pixelsáv helyére nullák kerülnek, ezért az átlag általában megváltozik; ezt az F1 feladat számpéldája mutatja meg.

A két egyenlet összevetése mutatja a lényegi különbséget: a (3) egyenletben a jobb oldalon T_τ még mindig hat a kimenetre, vagyis az eltolás **megőrződik** a kimenetben; a (4) egyenletben a kimenetből **eltűnik** az eltolás. Az ekvivariancia tehát nem gyengébb invariancia, hanem más tulajdonság: az egyik megtartja, a másik eltünteti az eltolást.

A két fogalom metszete nem üres: a nulla leképezés (amely minden bemenethez a nulla képet rendeli), $F(X) = 0$, egyszerre invariáns ($F(T_\tau X) = 0 = F(X)$) és ekvivariáns ($F(T_\tau X) = 0 = T_\tau 0 = T_\tau F(X)$), ezért egyik tulajdonságot sem választja el a másiktól. A véges rácson három lépést kell elkülöníteni: *kiterjesztés* (E: a véges képet nulla-fill vagy periodikus módon a végtelen rácsra visszük), *eltolás* (T_τ a $\mathbb{Z}^2$-n, ami csoporthatás és invertálható) és *megszorítás* (R: visszavágás a véges képre). A $R \circ T_\tau \circ E$ véges crop eltolás lenyírja a tartalmat, nem invertálható, ezért nem $\mathbb{Z}^2$-csoporthatás. A globális átlagpooling csak körkörös/permutációs, vagy tartalmat megtartó eltolásra invariáns; véges nulla-fill eltolásra, amely tartalmat veszít, általában nem. Szétválasztó példák: a konstans, de nem nulla kimenet, $F(X) = \mathbf{1}$ (csupa egyes kép), invariáns, de nem ekvivariáns, mert a kimeneti rácson ható nullával kitöltő T_τ a $\tau \neq 0$ eltolásnál nullasávot hoz létre, így $T_\tau F(X) \neq \mathbf{1} = F(T_\tau X)$. Az identitásleképezés $F(X) = X$ ekvivariáns, de nem invariáns, mert $T_\tau X \neq X$ általában. Mindkét példában a kimeneti térre ható T_τ ugyanaz a (2) szerinti, nullával kitöltő eltolás. A fogalmi különbséget az F1 feladatban mélyítjük el.

16.1.4 A fő tétel: cross-correlation ekvivarianciája a belső tartományon

A konvolúciós réteg lokális kapcsolódását és súlymegosztását az [S01] kurzusjegyzet „Convolutional layer" szakasza tárgyalja; a következő állítás a saját, pontos feltételekkel ellátott levezetésünk.

Tétel (valid cross-correlation eltolási ekvivarianciája a belső tartományon). Legyen F az (1) képlettel definiált valid cross-correlation, stride $s = 1$, padding $p_h = p_w = 0$. Legyen $\tau_h, \tau_w \geq 0$ egészek, és tegyük fel, hogy $\tau_h \leq H - k_h$ és $\tau_w \leq W - k_w$. Ekkor minden $o \in [0, C_{out} - 1]$ kimeneti csatornára és minden olyan (i, j) párra, amelyre

$$\tau_h \leq i \leq H - k_h, \qquad \tau_w \leq j \leq W - k_w,$$

teljesül

$$F(T_\tau X)_{o,i,j} = F(X)_{o,i-\tau_h,j-\tau_w} = [T_\tau F(X)]_{o,i,j}, \tag{5}$$

ahol $[T_\tau F(X)]_{o,i,j} = F(X)_{o,i-\tau_h,j-\tau_w}$ a kimeneti rács eltolásának definíciója.

Az (i, j)-re vonatkozó feltétel jelöli ki a **belső tartományt**: az alsó korlát ($i \geq \tau_h$, $j \geq \tau_w$) miatt az eltolt képen a kernelablak nem éri el a bal és a felső nullasávot, a felső korlát ($i \leq H - k_h$) pedig a valid kimenet utolsó indexe, $H' - 1$. A tartományon kívül két külön határeset van. (a) Ha $i < \tau_h$ vagy $j < \tau_w$, a jobb oldali $[T_\tau F(X)]_{o,i,j}$ a kimeneti eltolás nullával kitöltött sávjába esik, tehát 0, a bal oldal viszont a részben nullasávra eső ablak összege plusz b_o; az egyenlőség általában nem áll. (b) A bias ebben a sávban önmagában is sérti az egyenlőséget: csupa nulla X esetén $F(T_\tau X)_{o,i,j} = b_o$, míg $[T_\tau F(X)]_{o,i,j} = 0$, ha $b_o \neq 0$. Egydimenziós ellenőrzés: $x = (1, 2, 3)$, $k = (1, 1)$, $b = 0$, $\tau = 1$ mellett $F(x) = (3, 5)$, $F(T_\tau x) = F(0, 1, 2) = (1, 3)$ és $T_\tau F(x) = (0, 3)$; az $i = 0$ pont a nullasávban van és sérti az egyenlőséget, az $i = 1 = \tau$ belső pont teljesíti.

Bizonyítás. A bizonyítás a definíciók behelyettesítésén és az összegzés indexeinek átnevezésén alapul. Induljunk ki a bal oldalból, és alkalmazzuk először F, majd T_τ definícióját:

$$F(T_\tau X)_{o,i,j} = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{o,c,u,v}\, (T_\tau X)_{c,i+u,j+v} + b_o.$$

A (2) definíció szerint $(T_\tau X)_{c,u,v} = X_{c,u-\tau_h,v-\tau_w}$, tehát

$$F(T_\tau X)_{o,i,j} = \sum_{c} \sum_{u,v} K_{o,c,u,v}\, X_{c,\, i+u-\tau_h,\, j+v-\tau_w} + b_o. \tag{6}$$

A feltételből $i \le H - k_h$, ezért a legnagyobb $u = k_h - 1$ esetén is $i + u - \tau_h \le H - 1 - \tau_h \le H - 1$; az $i \ge \tau_h$ alsó korlát és $u \ge 0$ miatt pedig $i + u - \tau_h \ge 0$. Tehát minden X-index az eredeti tartományban van, nullával kitöltött elem nem lép fel.

Most írjuk fel $F(X)$ definícióját az $(i - \tau_h,\, j - \tau_w)$ kimeneti pozícióra — a feltételek miatt ez a pozíció értelmezett, hiszen $0 \le i - \tau_h \le H - k_h - \tau_h \le H' - 1$ (és ugyanígy j-re), ahol $H' = H - k_h + 1$:

$$F(X)_{o,i-\tau_h,j-\tau_w} = \sum_{c} \sum_{u,v} K_{o,c,u,v}\, X_{c,\,(i-\tau_h)+u,\,(j-\tau_w)+v} + b_o. \tag{7}$$

A (6) és (7) összegek tagonként azonosak, mert a kernel K független az eltolástól, és az összeadás kommutatív: az $i + u - \tau_h$ és $(i - \tau_h) + u$ indexek megegyeznek. A bias b_o konstans, mindkét oldalon ugyanaz, tehát nem befolyásolja az egyenlőséget. Ezzel

$$F(T_\tau X)_{o,i,j} = F(X)_{o,i-\tau_h,j-\tau_w},$$

ami éppen az (5) állítás. □

A bizonyításból látszik a **kulcsmechanizmus**: a kernel súlyai minden pozícióban ugyanazok (weight sharing, 15.1. Súly- és biasgradiens), ezért az eltolás csak az indexeket mozgatja, a súlyokat nem. Ha a súlyok pozíciófüggők lennének, az egyenlőség nem állna fenn — ez az F2 feladatban visszatér.

A tétel csak **nemnegatív** eltolásra lett kimondva. Negatív τ esetén a belső tartomány feltétele tükröződik: az eltolt kép nullasávja a jobb, illetve az alsó oldalon jelenik meg, és a tartomány $0 \le i \le H - k_h + \tau_h$ (ugyanígy j-re). A teljes állítás mindkét irányra hasonló, de a laborban a pozitív eset elegendő.

16.1.5 A stride mint részcsoport-korlátozás

Ha a cross-correlation stride s-sel dolgozik, a kimeneti rács s-szeresen ritkább. A kimeneti pozíció (i', j')-höz a bemeneti ablak az $(i's + u,\ j's + v)$ ponthoz kezdődik, ahol u, v a kernelindex. Ha a bemenetet $\tau = s \cdot m = (sm_h, sm_w)$ vektorral toljuk, ahol $m \in \mathbb{Z}^2$, akkor

$$F_s(T_{sm} X)_{o,i',j'} = F_s(X)_{o,i'-m_h,j'-m_w} = [T_m F_s(X)]_{o,i',j'},$$

a belső tartományon. A levezetés azonos az előző bizonyítással, csak az i indexet $i's$-re cseréljük. A lényeg: az eltolásoknak az a halmaza, amelyre az ekvivariancia a kimeneti rácson változatlan formában teljesül, a

$$\mathcal{T}_s = \{(sm_h, sm_w) : m_h, m_w \in \mathbb{Z}\}$$

halmaz. Ez az összes kétdimenziós egész eltolás $\mathbb{Z}^2$ csoportjának **részcsoportja** (az összeadásra zárt, az ellentett elemet is tartalmazza). Nem s-többszörös, például $(1,0)$ eltolás esetén (amelyre $\tau \notin \mathcal{T}_s$, és $s > 1$) általában nincs olyan egész m, amelyre $F_s(T_{(1,0)}X) = T_m F_s(X)$ teljesülne a kimeneti rácson, mert a kimeneti rács felbontása nem tud félpixeles eltolást reprezentálni. Ez a **stride-részcsoport** fogalma: az equivariance csak a rács ritkításának megfelelő részcsoporton áll fenn.

16.1.6 Kézi számpélda a határ feltételére

Legyen $H = W = 6$, $C_{in} = C_{out} = 1$, kernel 3×3, minden eleme $K_{u,v} = 1$, bias $b = 0$, stride 1, padding 0. A bemenet legyen $X_{i,j} = i \cdot 6 + j$, azaz

$$X = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 & 16 & 17 \\ 18 & 19 & 20 & 21 & 22 & 23 \\ 24 & 25 & 26 & 27 & 28 & 29 \\ 30 & 31 & 32 & 33 & 34 & 35 \end{pmatrix}.$$

A valid kimenet 4×4. Válasszuk az eltolást $\tau = (1,1)$-nek. A nulla-paddingelt eltolt kép $T_\tau X$ sorai:

$$T_\tau X = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 3 & 4 \\ 0 & 6 & 7 & 8 & 9 & 10 \\ 0 & 12 & 13 & 14 & 15 & 16 \\ 0 & 18 & 19 & 20 & 21 & 22 \\ 0 & 24 & 25 & 26 & 27 & 28 \end{pmatrix}.$$

Számítsuk ki a belső tartomány egy pontját. Az $F(X)$ kimenet 4×4-es, és a $(0,0)$ pozícióhoz tartozó 3×3 ablak az X bal felső része:

$$F(X)_{0,0} = 0 + 1 + 2 + 6 + 7 + 8 + 12 + 13 + 14 = 63.$$

Az $F(T_\tau X)$ kimenet ugyancsak 4×4-es, és az $(1,1)$ pozícióhoz tartozó ablak a $T_\tau X$ sorai 1–3, oszlopai 1–3:

$$F(T_\tau X)_{1,1} = 0 + 1 + 2 + 6 + 7 + 8 + 12 + 13 + 14 = 63.$$

Az $[T_\tau F(X)]_{1,1} = F(X)_{0,0} = 63$, tehát az $(1,1)$ pozícióban az egyenlőség teljesül. Ez a belső tartomány része: az eltolt képen ez az ablak már teljesen az eredeti X eltolt tartalmán belül helyezkedik el.

Most nézzük a határpozíciót $(i,j) = (0,0)$-t. Az $F(T_\tau X)_{0,0}$ a $T_\tau X$ bal felső 3×3 ablakának összege:

$$T_\tau X \text{ sorai } 0,1,2,\ \text{oszlopai } 0,1,2 : \begin{matrix} 0 & 0 & 0 \\ 0 & 0 & 1, \\ 0 & 6 & 7 \end{matrix} \qquad \text{összeg} = 0+0+0+0+0+1+0+6+7 = 14.$$

A jobb oldal a kimeneti rács nullával kitöltő eltolásából $[T_\tau F(X)]_{0,0} = 0$, mert $(0,0)$ a kimeneti nullasávban van ($i = 0 < \tau_h = 1$). A $14 \neq 0$ eltérés pontosan a tétel alsó korlátját mutatja: az ablak a bemeneti nullasávba lóg, ezért nem az eredeti tartalom eltoltja. A belső tartomány itt $1 \le i \le 6 - 3 = 3$ és $1 \le j \le 3$; a $(3,3)$ pozíció a felső korlátot is szemlélteti: $F(T_\tau X)_{3,3}$ ablaka a $T_\tau X$ 3–5. sora és oszlopa, vagyis az X 2–4. sora és oszlopa, tehát $F(T_\tau X)_{3,3} = F(X)_{2,2} = [T_\tau F(X)]_{3,3}$. A laborban a numerikus ellenőrzés ugyanezt a tartományt használja.

16.1.7 Labor: szélső padding, crop és stride eltérése

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A NumPy szkript a szakasz kódblokkjából futtatható; külső adatfájl nem kell.

A labor egyetlen kis NumPy szkript, amely három eltolási implementáció — **zero-fill**, **crop** és **circular** —, valamint a stride hatását méri. A szkript a cross-correlationt saját NumPy-ciklusokkal számolja; a PyTorch `Conv2d` csak a szemantikai referencia, a labor nem hívja meg. A munkakönyvtár `lab/n061`. A kód `float64`-ben dolgozik, a tolerancia a belső tartományon 10^{-12}, mert a műveletek véges floating-point összegek.

```
# workdir: lab/n061
import numpy as np

def corr_valid(x, k):
    """Valid cross-correlation, N=1, C=1. x:(H,W), k:(kh,kw)."""
    H, W = x.shape
    kh, kw = k.shape
    out = np.zeros((H - kh + 1, W - kw + 1), dtype=np.float64)
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(x[i:i+kh, j:j+kw] * k)
    return out

def corr_valid_stride(x, k, s):
    H, W = x.shape
    kh, kw = k.shape
    ho = (H - kh) // s + 1
    wo = (W - kw) // s + 1
    out = np.zeros((ho, wo), dtype=np.float64)
    for i in range(ho):
        for j in range(wo):
            out[i, j] = np.sum(x[i*s:i*s+kh, j*s:j*s+kw] * k)
    return out

def shift_zero(x, tau):
    """Eltolás; a felszabaduló sáv nullával töltött."""
    th, tw = tau
    y = np.zeros_like(x)
    H, W = x.shape
    y[th:, tw:] = x[:H-th, :W-tw]
    return y

def shift_crop(x, tau):
    """Eltolás a (2) definíció szerint; a kilógó részt eldobjuk, a kép mérete csökken."""
    th, tw = tau
    # a tartalom jobbra/lefelé tolódik, ezért az alsó és a jobb oldali sáv esik ki
    return x[:x.shape[0]-th, :x.shape[1]-tw]

def shift_circular(x, tau):
```

```
    """Körkörös eltolás (np.roll)."""
    th, tw = tau
    return np.roll(x, (th, tw), axis=(0, 1))

np.random.seed(20261020)
x = np.random.rand(8, 8).astype(np.float64)
k = np.array([[1.0, 2.0, 1.0],
              [2.0, 4.0, 2.0],
              [1.0, 2.0, 1.0]], dtype=np.float64)  # Gauss-szerű 3x3 kernel

tau = (1, 1)

# 1) zero-fill shift
F_x = corr_valid(x, k)
T_x = shift_zero(x, tau)
F_Tx = corr_valid(T_x, k)
T_Fx = np.zeros_like(F_x)
T_Fx[tau[0]:, tau[1]:] = F_x[:F_x.shape[0]-tau[0], :F_x.shape[1]-tau[1]]
# belső tartomány a tétel szerint: tau_h <= i < F_Tx.shape[0], stb.
diff_zero = np.abs(F_Tx - T_Fx)
print("zero-fill, belső max |diff|:", diff_zero[tau[0]:, tau[1]:].max())
print("zero-fill, teljes max |diff|:", diff_zero.max())
assert diff_zero[tau[0]:, tau[1]:].max() < 1e-12
assert diff_zero.max() > 0.0

# 2) crop shift
C_x = shift_crop(x, tau)  # 7x7: a zero-fill eltolt kép nem nulla sávja
F_Cx = corr_valid(C_x, k)  # 5x5
# A crop ugyanazt a tartalmat tartja meg, mint a zero-fill shift, csak a nullasáv
# nélkül; ezért F_Cx a F_x bal felső 5x5-ös blokkjával egyezik (F_x 6x6).
T_Fx_crop = F_x[:F_x.shape[0]-tau[0], :F_x.shape[1]-tau[1]]
diff_crop = np.abs(F_Cx - T_Fx_crop)
print("crop, közös tartomány max |diff|:", diff_crop.max())
assert diff_crop.max() < 1e-12

# 3) circular shift
C2_x = shift_circular(x, tau)
F_C2x = corr_valid(C2_x, k)
diff_circ = np.abs(F_C2x - T_Fx)
print("circular, belső max |diff|:", diff_circ[tau[0]:, tau[1]:].max())
print("circular, teljes max |diff|:", diff_circ.max())
assert diff_circ[tau[0]:, tau[1]:].max() < 1e-12
assert diff_circ.max() > 0.0

# 4) stride 2
s = 2
F_sx = corr_valid_stride(x, k, s)          # 3x3
# páros eltolás: tau = (2,0) -> kimeneti (1,0) eltolás
T2_x = np.zeros_like(x)
T2_x[2:, :] = x[:6, :]
F_sT2x = corr_valid_stride(T2_x, k, s)      # 3x3
T1_Fsx = np.zeros_like(F_sx)
T1_Fsx[1:, :] = F_sx[:F_sx.shape[0]-1, :]
diff_stride_even = np.abs(F_sT2x - T1_Fsx)
print("stride 2, páros eltolás belső max |diff|:",
      diff_stride_even[1:, :].max())
assert diff_stride_even[1:, :].max() < 1e-12

# nem s-többszörös (páratlan) eltolás: tau=(1,0) - nincs megfelelő kimeneti egész eltolás
T1_x = np.zeros_like(x)
T1_x[1:, :] = x[:7, :]
F_sT1x = corr_valid_stride(T1_x, k, s)
# referencia: F_sx eltolás nélkül; eltérés nem lehet nulla
```

```
diff_stride_odd = np.abs(F_sT1x - F_sx)
print("stride 2, páratlan eltolás teljes max |diff|:", diff_stride_odd.max())
assert diff_stride_odd.max() > 0.0
```

A mért kimenet. A labor futása (Python 3.12.3, NumPy 2.5.4, CPU: 16 mag, szálkorlát nélkül; a számítások kis tömbökön, Python-ciklusokkal futnak, `np.random.seed(20261020)`):

```
zero-fill, belső max |diff|: 0.0
zero-fill, teljes max |diff|: 8.725733258135769
crop, közös tartomány max |diff|: 0.0
circular, belső max |diff|: 0.0
circular, teljes max |diff|: 9.378319812429872
stride 2, páros eltolás belső max |diff|: 0.0
stride 2, páratlan eltolás teljes max |diff|: 2.3908338047457214
```

A zero-fill esetben a belső tartományon a maximális eltérés 0, a teljes tartományon viszont nem nulla, mert a határon a nulla-sáv hatása megjelenik. A crop shift esetén a közös tartományon az eltérés 0, de a kimenet mérete kisebb, így a határinformáció elveszik. A circular shift esetén a belső tartományon az eltérés 0, a teljes tartományon viszont nem nulla, mert a határon a körkörös becsomagolás a kép másik széléről hoz elemeket az ablakba; a valid cross-correlation lineáris határfeltételt használ, ezért a két operátor a határon nem cserélhető fel. A stride 2 esetén az s-többszörös (páros) eltolásra a belső tartományon az eltérés 0, a nem s-többszörös (például $(1,0)$) eltolásra a teljes tartományon sem nulla. A kód ezeket az állításokat `assert`-tel ellenőrzi; a fenti számok a rögzített seed mellett reprodukálhatók.

16.1.8 Feladatok

16.1.8.1 F1 feladat (fogalmi, nehézség 1)

Adjon példát olyan F operátorra, amely a (3) értelemben eltolási ekvivariáns, de nem invariáns, és olyanra, amely invariáns. Mutassa meg, hogy a globális átlagpooling a körkörös eltolásra invariáns, a nulla-kitöltéses eltolásra viszont általában nem, és adja meg a különbség okát. Elvárt eredmény: a két példa az operátoregyenlettel ellenőrizve, legalább egy ellenpélda a nulla-kitöltéses esetre.

16.1.8.2 F2 feladat (levezetési/bizonyítási, nehézség 2)

Bizonyítsa be a stride-s cross-correlation eltolási ekvivarianciáját a stride-részcsoporton: minden $m \in \mathbb{Z}^2$-re és a belső tartományon teljesüljön

$$F_s(T_{sm}X)_{o,i',j'} = [T_m F_s(X)]_{o,i',j'}.$$

Adja meg pontosan a belső tartományt H, W, k_h, k_w, s, m függvényében. Elvárt eredmény: a tétel feltevésekkel, a bizonyítás a definícióból, és egy 8×8-as bemenetre $s = 2$, $m = (1,0)$ mellett a belső tartomány explicit indexhalmaza.

16.1.8.3 F3 feladat (implementációs, nehézség 2)

Egészítse ki a labor kódját egy `check_equiv` függvénnyel, amely adott F, T_τ, X és K esetén visszaadja (a) a teljes tartományon, (b) a belső tartományon a maximális abszolút eltérést. Futtassa a labor zero-fill esetére. Elvárt eredmény: a függvény kódja, a két számérték, és egy `assert` állítás, amely a belső tartományon 10^{-12} alatti eltérést követel meg.

16.1.8.4 F4 feladat (hibakeresési, nehézség 2)

Egy hallgató a következőt állítja: „Mivel a np.roll körkörösen tolja el a képet, a `corr_valid(np.roll(x, tau), k)` eredménye mindig megegyezik `np.roll(corr_valid(x, k), tau)` eredményével.” Lokalizálja az állítás hibáját, adjon konkrét 4×4-es bemenetet, 2×2-es kernelt és $\tau = (1, 0)$ eltolást, amelyre az egyenlőség megsérül, és magyarázza el a hiba okát a határfeltételek nyelvén. Elvárt eredmény: az ellenpélda konkrét számaival, a két kimenet kiszámítva, és a javítás (mely shift-definícióval lenne igaz az állítás).

16.1.8.5 F5 feladat (kísérleti/transzfer, nehézség 3)

Tervezzen kísérletet arra, hogy megmérje: a GlyphLab osztályozó CNN (a korábbi checkpoint, azaz a mentett modellparaméter-állapot) mennyire invariáns az eltolásra a gyakorlatban. Vegye figyelembe a szerződést: a tesztkép eltolásakor a nulla-paddingelt részek új tartalmat visznek be, ezért az eltolást csak a kép belsejéből vett cropként, vagy a valid tartomány eltolásaként definiálja. Adjon meg nullhipotézist, metrikát (például a helyes osztály logitjának átlagos változása vagy a pontosság), a teszteléshez használt eltolástartományt, és azt, hogy miért nem elegendő egyetlen eltolás. Elvárt eredmény: egy kísérleti terv a protokoll elemeivel (adat, split, baseline (viszonyítási alap), változó, metrika, korlát), futtatott eredmény nélkül. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

16.1.9 Megoldások és indoklás

16.1.9.1 F1 megoldása

Az identitás $F(X) = X$ ekvivariáns, mert $T_\tau X = T_\tau X$ triviálisan, de nem invariáns, mert az eltolt kép általában különbözik az eredetitől. A globális átlagpooling

$$G(X) = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} X_{i,j}$$

a **körkörös** eltolásra invariáns, mert a körkörös eltolás a pixelek permutációja, így az összeg (és az átlag) változatlan. A **nulla-kitöltéses** eltolásra viszont általában nem invariáns: a kép széléről kikerülő pixelsáv helyére nullák kerülnek, ezért az összeg a kikerülő sáv összegével csökken. Konstans $X = c$ esetén a kikerülő sáv összege $c(\tau_h W + \tau_w H - \tau_h \tau_w)$, így

$$G(T_\tau X) = \frac{(H - \tau_h)(W - \tau_w)}{HW}\, c = \frac{HW - \tau_h W - \tau_w H + \tau_h \tau_w}{HW}\, G(X),$$

de ez az arány csak konstans X-re érvényes; általános X-re a kikerülő sáv tényleges összege számít, ezért $G(T_\tau X)$ nem fejezhető ki $G(X)$ konstansszorosaként. Ellenpélda: X csupa 1-es 4×4-es kép, $\tau = (1, 0)$; ekkor $G(X) = 1$, de $G(T_\tau X) = 3 \cdot 4/16 = 0{,}75$.

16.1.9.2 F2 megoldása

A stride-s cross-correlation definíciója:

$$F_s(X)_{o,i',j'} = \sum_{c} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{o,c,u,v}\, X_{c,\, i's+u,\, j's+v} + b_o.$$

Helyettesítsük be a T_{sm} eltolást, ahol $m = (m_h, m_w)$:

$$F_s(T_{sm}X)_{o,i',j'} = \sum_{c} \sum_{u,v} K_{o,c,u,v}\, X_{c,\, i's+u-sm_h,\, j's+v-sm_w} + b_o.$$

Az X-index átrendezve: $i's - sm_h + u = (i' - m_h)s + u$. Ez pontosan az $F_s(X)$ definíciója az $(i' - m_h, j' - m_w)$ kimeneti pozícióra, feltéve, hogy az indexek a tartományban vannak. A belső tartomány: a $T_{sm}X$ ablaknak teljesen az eltolt eredeti tartalomban kell lennie, azaz $i's+u \geq sm_h$ minden u-ra és $i's + u \leq H - 1$ is, hasonlóan j-re. Innen

$$m_h s \leq i's \leq H - k_h, \qquad m_w s \leq j's \leq W - k_w,$$

vagyis $m_h \leq i' \leq \lfloor (H - k_h)/s \rfloor$ és $m_w \leq j' \leq \lfloor (W - k_w)/s \rfloor$. A 8×8-as bemenetre $k_h = k_w = 3$, $s = 2$, $m = (1, 0)$ esetén a belső tartomány $i' \in [1, 2]$, $j' \in [0, 2]$ (mert $\lfloor (8 - 3)/2 \rfloor = 2$). Ez a kimeneti 3×3-as rács alsó 2 sora.

16.1.9.3 F3 megoldása

A függvény:

```
def check_equiv(F, shift, x, k, tau):
    """F: (x,k)->out cross-correlation. Visszaadja a teljes és belső max eltérést."""
    F_x = F(x, k)
    T_x = shift(x, tau)
    F_Tx = F(T_x, k)
    T_Fx = np.zeros_like(F_x)
    th, tw = tau
    T_Fx[th:, tw:] = F_x[:F_x.shape[0]-th, :F_x.shape[1]-tw]
    diff = np.abs(F_Tx - T_Fx)
    full = diff.max()
    interior = diff[th:, tw:].max()
    return full, interior

full, interior = check_equiv(corr_valid, shift_zero, x, k, (1, 1))
assert interior < 1e-12, f"belső eltérés {interior}"
print(f"teljes={full}, belső={interior}")
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU: 16 mag, szálkorlát nélkül, `np.random.seed(20261020)`): `teljes=8.725733258135769, belső=0.0`; a belső eltérés 0, a teljes pozitív. Az ellenőrzés azért használ `float64`-et és 10^{-12} küszöböt, mert a cross-correlation összegekben legfeljebb $k_h k_w$ tagú kerekítés halmozódik; a 10^{-12} biztonsági ráhagyással a kerekítési szint fölött van (a gépi epszilon $2{,}2 \cdot 10^{-16}$, a 9 tagú összeggel $\sim 2 \cdot 10^{-15}$), a mért belső eltérés pedig ennél jóval kisebb, 0.

16.1.9.4 F4 megoldása

Legyen $X = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]]$, kernel $[[1,0],[0,0]]$, $\tau = (1,0)$. A `corr_valid` kimenete 3×3:

$$F(X) = \begin{pmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{pmatrix}.$$

A np.roll$(X,(1,0))$ sorai: $[13,14,15,16]$, majd az eredeti első három sor. A `corr_valid` ezen:

$$F(\mathrm{roll}(X)) = \begin{pmatrix} 13 & 14 & 15 \\ 1 & 2 & 3 \\ 5 & 6 & 7 \end{pmatrix}.$$

A np.roll$(F(X),(1,0))$ pedig:

$$\begin{pmatrix} 9 & 10 & 11 \\ 1 & 2 & 3 \\ 5 & 6 & 7 \end{pmatrix}.$$

A két mátrix első sora eltér $(13,14,15$ vs. $9,10,11)$, tehát az állítás hamis. A hiba oka: a körkörös eltolás a kép alsó sorát a felsőre hozza, ezért a kernel olyan elemeket lát, amelyek az eredeti kép másik széléről származnak. A valid cross-correlation lineáris határfeltételt használ, ezért a két operátor nem cserélhető fel. Az állítás akkor lenne igaz, ha a cross-correlation is körkörös paddinget használna; a labor `corr_valid` függvénye ezt nem teszi.

16.1.9.5 F5 megoldása (minta)

Kérdés: A tanított GlyphLab-osztályozó predikciója mennyire stabil a bemeneti kép kis, nulla-padding nélküli eltolásaira?

Adat és split: a GlyphLab test split 30 jelenetből áll, jelenetenként két nézettel, összesen 60 képpel; a kísérlet során a test képeket nem használjuk modellválasztásra, csak a végén értékeljük. A checkpoint a val spliten legjobb modell mentett paraméterállapota.

Eltolás-definíció: a 24×24-es képből 20×20-as ablakokat vágunk; az ablak bal felső sarkát a képen belül $\tau = (\tau_h, \tau_w) \in \{0,1,2,3,4\}^2$ szerint mozgatjuk, ami $5 \times 5 = 25$ vágást ad képenként. Minden vágás csak valódi képpontokat tartalmaz, így nem kerül be mesterséges nulla-sáv. A 14.4. Csoportos és depthwise konvolúció F5-ében leírt GlyphLab-osztályozó globális átlagpoolingot használ, ezért a 20×20-as vágás közvetlenül beadható: a pooling utáni jellemzőszám a csatornaszámtól függ, nem a térbeli mérettől. Ha a konkrét checkpoint feje rögzített 24×24-es bemenetet vár, a kísérletet nem szabad nulla-kitöltéssel kiterjeszteni, hanem a fejet kell a vágásméretre igazítani.

Nullhipotézis (H_0): a helyes osztály logitjának a 25 vágásra vett, az eltolás nélküli vágáshoz viszonyított átlagos különbsége nulla. **Ellenhipotézis (H_1):** ez az átlagos különbség nem nulla. **Döntési szabály:** a képenkénti páros különbségekre scene-szintű bootstrap 95%-os konfidenciaintervallumot számítunk (23.3. Páros összehasonlítás és csoportszintű intervallum); ha a CI nem tartalmazza a nullát, H_0-t 5%-os szinten elvetjük, ha tartalmazza, nem vetjük el. A „szignifikáns” szót kizárólag erre a döntésre használjuk.

Metrika: (a) a helyes osztály logitjának átlagos változása a 25 eltolásra, képenként (az eltolás nélküli értékhez képest), majd a képek átlaga; ez felel meg a feladatban szereplő „átlagos változásnak”. (b) kiegészítő metrika a pontosság változása az eltolt vágásokon. **Baseline (viszonyítási alap):** a checkpoint pontossága és logitja az eltolás nélküli vágáson; a különbségek ehhez képest értendők.

Korlát: a GlyphLab nem természetes képi benchmark, ezért az eredmény nem általánosítható valós alkalmazásokra; a minta 60 kép 30 jelenetből, a bootstrap egység a scene_id (jelenetazonosító; a két nézet ugyanahhoz a jelenethez tartozik). A 20×20-as vágás a kép széléről elveszít tartalmat, ezért a kapott invariancia csak a vágáson belüli eltolásokra vonatkozik.

16.1.10 Tipikus tévedés: „a CNN automatikusan invariáns minden eltolásra”

A leggyakoribb tévedés azt állítja, hogy egy konvolúciós réteg, mivel „súlymegosztást” használ, a kimenetét változatlanul hagyja az eltolással szemben. Ez hamis: a cross-correlation ekvivariáns, tehát a kimenet **ugyanúgy eltolódik**, nem marad a helyén. Invarianciát csak a térbeli dimenziókat összevonó műveletek (globális pooling, illetve stride-dal kombinált pooling) hozhatnak létre, és azok is csak közelítő, az összevonási ablakon belüli eltolásokra érvényes invarianciát adnak.

Konkrét ellenpélda: $X = [1, 0, 0]$ 1D jel, $K = [1, 0]$ kernel, valid cross-correlation stride 1, padding 0. Ekkor $F(X) = [1, 0]$. A $\tau = 1$ eltolásra $T_\tau X = [0, 1, 0]$, és $F(T_\tau X) = [0, 1]$. Ez pontosan $T_\tau F(X)$, vagyis a kimenet eltolódott, nem maradt $[1, 0]$. Az invariáns viselkedéshez a kimenetnek $[1, 0]$-nak kellene maradnia, ami nem teljesül.

A [P03] absztraktja szerint a modern CNN-architektúrák downsampling (leskálázó) rétegei miatt a hálózatok a gyakorlatban sem teljesen shift-invariánsak (eltolásinvariánsak); a további módszertani részleteket itt nem tárgyaljuk.

16.1.11 Önellenőrzés és visszahívás

Ellenőrizze a megértést az alábbi kérdésekkel, mielőtt továbblép:

1. Mi a különbség a (3) és a (4) egyenlet között? Melyik oldalon hat még T_τ a kimenetre?
2. A tételben miért szerepel az $i \geq \tau_h$, $j \geq \tau_w$ alsó korlát? Rajzolja fel a 6×6-os példa nulla-sávját és a belső tartományt.
3. Miért éppen $\mathbb{Z}^2$ egy részcsoportja a stride-s eltolások halmaza, és miért nem az egész $\mathbb{Z}^2$?
4. A körkörös shift és a valid cross-correlation miért nem cserélhető fel? Kapcsolja össze a 13.4. Határkezelés és frekvenciaintuíció egységben tanult lineáris és körkörös határfeltétellel.
5. Hogyan függ össze az equivariance a kernel 15.1. Súly- és biasgradiens alfejezetben tárgyalt súlymegosztásával? Mi történne, ha a kernel minden pozícióban különböző súlyt használna?

Rövid válaszok. (1) A (3) egyenlet az ekvivariancia: a jobb oldalon T_τ még hat a kimenetre, ezért az eltolás megőrződik; a (4) egyenlet az invariancia: a kimenetből eltűnik az eltolás. (2) A nulla-kitöltéses eltolás után a kép tartalma csak az $i \geq \tau_h$, $j \geq \tau_w$ negyedben van jelen, a felszabaduló felső és bal sáv nulla; a 6×6-os példán ez egy-egy nulla-sávot jelent a határon. (3) A stride-s eltolások halmaza $s\mathbb{Z}^2$, mert az eltolások összege és ellentettje is stride-többszörös, de az egypozíciós eltolás nem áll elő ilyen alakban; ezért részcsoport, nem az egész $\mathbb{Z}^2$. (4) A körkörös shift a szélen átfordul, a valid cross-correlation viszont nulla-kitöltéssel dolgozik, így a két művelet a határon nem cserélhető fel; ez a lineáris és a körkörös határfeltétel különbsége. (5) Az equivariancia a súlymegosztás következménye: ugyanaz a kernel hat minden pozícióban, ezért a bemenet eltolása a kimenetet ugyanúgy tolja el; pozíciónként különböző súlyok esetén ez az állítás megszűnik.

Rövid visszahívás: a 4.1. Függvény, határérték és derivált egységben a függvényeket leképezésként definiáltuk; a (3) egyenlet két leképezés, $F \circ T_\tau$ és $T_\tau \circ F$ egyenlősége egy adott tartományon. Ez ugyanaz a szerkezet, mint a lineáris operátorok kommutálása, csak itt a tartomány korlátozott. A 15.1. Súly- és biasgradiens egységben levezetett kernelgradiens-összegzés éppen a súlymegosztás következménye; ugyanez a megosztás teszi lehetővé, hogy az eltolás csak az indexeket mozgassa.

16.1.12 Rövid összefoglaló

Az eltolási ekvivariancia és invariancia két külön operátoregyenlet. Az ekvivariancia, $F(T_\tau X) = T_\tau F(X)$, azt jelenti, hogy a kimenet az eltolással együtt mozog; az invariancia, $F(T_\tau X) = F(X)$, azt, hogy a kimenet nem változik. A valid cross-correlation stride 1, padding 0 mellett a belső tartományon ekvivariáns; a határon a nulla-padding miatt az állítás sérül. A stride s esetén az ekvivariancia csak az $s\mathbb{Z}^2$ részcsoport eltolásaira áll fenn. A labor a zero-fill, crop és circular shift, valamint a stride hatását méri. Egy CNN tehát nem automatikusan invariáns: az invarianciát külön aggregáló rétegeknek kell létrehozniuk.

16.1.13 Következő alkalmazás

A következő, 16.2. Pooling, downsampling és aliasing egység a pooling (összevonó) rétegeket vizsgálja. Ott az itt definiált invarianciafogalom válik mérhetővé: a max- és átlagpooling stride-dal kombinálva lokális invarianciát ad, de a downsampling (leskálázás) aliasing-hoz (jelspektrum-átlapolódáshoz) vezethet. A 16.3. Elméleti és effektív recepciós mező egység pedig a recepciós mezőt általánosítja a többrétegű hálózatokra, ahol az ekvivariancia rétegenkénti összetétele kerül elő.

16.1.14 Felhasznált források

- [S01] Stanford CS231n kurzusjegyzet. *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/.
- [P03] Zhang, R. (2019). *Making Convolutional Networks Shift-Invariant Again*. PMLR 97:7324–7334. https://proceedings.mlr.press/v97/zhang19a.html.

16.2 Pooling, downsampling és aliasing

16.2.1 Cél, előismeret és induló kérdés

A 16.1. Eltolási equivariance és invariance egységben az eltolási ekvivarianciát operátoregyenletekkel vizsgáltuk, és láttuk, hogy a stride-os művelet az ekvivarianciát a stride-részcsoportra (a stride többszöröseinek megfelelő eltolásokra) szűkíti. Most a pooling rétegre mint lokális aggregációra és mint mintavétel-csökkentésre koncentrálunk. A két szerep gyakran egyetlen rétegben jelenik meg, de fogalmilag különválasztható, és a különválasztás dönti el, hogy milyen eltolási tulajdonságokat várhatunk.

Előismeret: a 16.1. Eltolási equivariance és invariance eltolási operátora, a 15.2. Inputgradiens és átfedő recepciós mezők NCHW-alakkezelése és a 4.2. Parciális derivált és gradiens gradiensfogalma. A poolingnak többnyire nincs tanulható paramétere, ezért a gradiens itt csak az inputra vonatkozik; a jelen egységben azonban elsősorban az előremenő leképezés és a rácsszimmetria a tárgy.

Induló kérdés: legyen $X \in \mathbb{R}^{1\times1\times4\times1}$ a négypixeles vektor $[2, 1, 4, 0]$. Egy $k = 2$, $s = 2$ maxpooling után mi az eredménye X-nek és a $T_1X = [0, 2, 1, 4]$ (egypozíciós, nullával feltöltött jobbra tolás) eltoltjának? Becsülje meg, tud-e a kimeneti rácson olyan eltolást mondani, amely az első kimenetet a másodikba viszi. A pontos definíciókkal ezt az alábbiakban megválaszoljuk: a két kimenet $[2, 4]$ és $[2, 4]$, ezért erre a konkrét párra a $\rho = 0$ kimeneti eltolás megfelel; azt viszont az alábbi lemma mutatja meg, hogy nem létezik minden bemenetre érvényes rögzített ρ.

16.2.2 Definíciók, tenzoralak és a két szerep szétválasztása

Legyen a bemenet

$$X \in \mathbb{R}^{N\times C\times H\times W},$$

NCHW elrendezésben. A pooling réteg csatornánként függetlenül dolgozik; a batch és csatorna dimenziók változatlanok. Az ablaktérbeli méret k_h, k_w pozitív egész, a stride s_h, s_w pozitív egész. Ebben az egységben a padding $p = 0$ (nincs kitöltés), és feltesszük, hogy az ablak mindig teljesen a bemeneti rácson belül marad; a kimeneti méret ekkor

$$H_{out} = \left\lfloor \frac{H - k_h}{s_h} \right\rfloor + 1, \qquad W_{out} = \left\lfloor \frac{W - k_w}{s_w} \right\rfloor + 1.$$

Ez ugyanolyan alakú képlet, mint a konvolúcióé [dilation nélkül: $d = 1$ és $p = 0$ mellett $H + 2p - (k - 1) - 1 = H - k$], de itt a kernel súlyok helyett rögzített aggregáló függvény áll.

Max pooling. A $0 \le i < H_{out}$, $0 \le j < W_{out}$ kimeneti pozícióhoz tartozó bemeneti ablak

$$R_{ij} = \{(is_h + u,\ js_w + v) \ :\ 0 \le u < k_h,\ 0 \le v < k_w\},$$

és

$$Y[n, c, i, j] = \max_{(p,q)\in R_{ij}} X[n, c, p, q].$$

Average pooling. Ugyanilyen ablakokon

$$Y[n, c, i, j] = \frac{1}{k_h k_w} \sum_{(p,q)\in R_{ij}} X[n, c, p, q].$$

A kimenet mindkét esetben $\mathbb{R}^{N\times C\times H_{out}\times W_{out}}$. Gyakori választás a $k_h = k_w = 2$, $s_h = s_w = 2$, amely a térbeli méretet felezi.

1. definíció (lokális aggregáció). Egy térbeli leképezést lokális aggregációnak nevezünk, ha a kimenet minden pontja egy rögzített méretű, a bemeneti rács szerinti lokális környezet elemeinek egyetlen értékké való összegzése, például maximum, összeg vagy átlag, anélkül hogy a bemeneti rács felbontását megváltoztatná. Formálisan $s_h = s_w = 1$; ilyenkor $H_{out} = H - k_h + 1$, tehát a kimeneti rács sűrűsége megmarad, csak a határ zsugorodik.

2. definíció (downsampling, leskálázás). A mintavételi sűrűség csökkentése: a bemeneti rácson kijelölünk egy ritkább rácsobjektumot, és csak ezen adjuk vissza az értékeket. Ebben a pooling-kontextusban $s_h > 1$ vagy $s_w > 1$. A kimeneti rács ennek megfelelően ritkább, és az aggregálás, ha van, már ezen a ritkább rácson történik.

Fontos, hogy a $k = 2$, $s = 2$ pooling **egyszerre** aggregál és leskáláz: az ablakot nem csúsztatjuk egyesével, hanem kettesével, tehát a kimeneti rács sűrűsége fele. Ezzel szemben $k = 2$, $s = 1$ csak lokális aggregáció, nincs mintavételi sűrűségcsökkenés.

Az eltolási ekvivariancia szempontjából a különbség lényeges: az aggregáció önmagában kompatibilis az eltolással a belső tartományon, a leskálázás viszont áttér egy ritkább rácsra, ahol a bemeneti rács egypixeles eltolásának nincs egyértelmű megfelelője. Emiatt várjuk, hogy a stride-1 pooling equivariáns, a stride-s pooling pedig csak az $s\mathbb{Z}^2$ eltolásokra.

16.2.3 Lemma: az egypixeles eltolás nem őrződik meg

A downsampling és az eltolás viszonyát formálisan egy egydimenziós modellben rögzítjük, mert a lényeg már ott látható.

Legyen a bemeneti rács $X \in \mathbb{R}^4$, az eltolás T_τ a belső tartományon legyen definiálva a $T_\tau X[i] = X[i - \tau]$ összefüggéssel, a határra zéró értéket feltételezve, ha $i - \tau < 0$. A pooling legyen $k = 2$, $s = 2$ maxpooling, tehát $P : [X_0, X_1, X_2, X_3] \mapsto [\max(X_0, X_1), \max(X_2, X_3)]$.

Lemma (nincs rögzített kimeneti eltolás az egypixeles shiftre). A $k = 2, s = 2$ maxpoolingra nem létezik olyan $\rho \in \mathbb{Z}$, amelyre minden $X \in \mathbb{R}^4$ mellett

$$P(T_1 X) = T_\rho P(X)$$

teljesülne.

Bizonyítás. Megmutatjuk, hogy két különböző bemenet két különböző ρ-t követel.

1. Válasszuk az $X^{(1)} = [1, 0, 0, 0]$ bemenetet. Ekkor $P(X^{(1)}) = [\max(1, 0), \max(0, 0)] = [1, 0]$. Az egypixeles eltolt $T_1 X^{(1)} = [0, 1, 0, 0]$. Ennek poolingja $P(T_1 X^{(1)}) = [\max(0, 1), \max(0, 0)] = [1, 0]$. Tehát $P(T_1 X^{(1)}) = P(X^{(1)})$, ami azt jelentené, hogy $T_\rho P(X^{(1)}) = P(X^{(1)})$, vagyis $\rho = 0$.

2. Válasszuk az $X^{(2)} = [0, 1, 0, 0]$ bemenetet. Ekkor $P(X^{(2)}) = [1, 0]$. Az eltolt $T_1 X^{(2)} = [0, 0, 1, 0]$, és $P(T_1 X^{(2)}) = [\max(0, 0) = 0, \max(1, 0) = 1] = [0, 1]$. Ahhoz, hogy $T_\rho [1, 0] = [0, 1]$ legyen, $\rho = 1$ szükséges.

Nincs olyan közös ρ, amely az első esetben 0, a másodikban 1 egyszerre, ezért az egyenlet nem teljesülhet minden X-re.

Az ellenpélda értelmezése: a $[1, 0, 0, 0]$ bemenetben a maximum az első ablakban van; az egypixeles eltolás a maximumot még mindig az első ablakba viszi, mert a második pozíció az első ablakban marad. A $[0, 1, 0, 0]$ bemenetnél azonban az eltolás a maximumot átviszi a második ablakba. Az ablakon belül áthelyezett maximum önmagában nem változtatja a pooling kimenetét (az első esetben ezért láthatatlan a változás); a másodikban az ablakhatár átlépése okoz ugrást. Egyenlő maximumok (tie, azaz holtverseny) esetén a backward konvenciót is rögzíteni kell: a gradiens vagy az első maximális pozícióra megy, vagy egyenlően szétosztódik. A gyakorlati implementációk jellemzően egy kiválasztott indexre adják a gradienst, és a tie-break szabályuk implementációfüggő; ezért az egyenlő maximumú pontokon a gradiens nem egyértelmű, és a numerikus gradiensellenőrzésben ezeket a pontokat kerülni kell. A kimeneti rácson a $\frac{1}{2}$ pixeles eltolás nem létezik, ezért nincs olyan konzisztens rögzített eltolás.

Következmény. Stride s pooling után az ekvivariancia-állítás csak az $s\mathbb{Z}^2$ eltolásokra fogalmazható meg értelmesen. Ez a 16.1. Eltolási equivariance és invariance megfigyelés általánosítása: a stride az eltolási részegységet a ritkább rácshoz rögzíti.

16.2.4 Aliasing intuíció és anti-aliasing szűrés

A leskálázás egy diszkrét jelet ritkább rácson mintavételez. A klasszikus mintavételi elméletben, ha a jel spektruma a ritkább rács Nyquist-határán (a mintavételi frekvencia felén, $f_s/2$-n) túlra nyúlik, akkor a magas frekvenciájú komponensek alacsony frekvenciájú hamis mintázatként, **alias**-ként jelennek meg. A klasszikus mintavételi tétel **folytonos idejű**, sávkorlátos jelekről szól: ha egy $f(t)$ függvény nem tartalmaz W ciklus/s-nál nagyobb frekvenciát, akkor egyértelműen meghatározzák az $1/(2W)$ másodpercenként vett mintái ([P28], II. szakasz, „The Sampling Theorem", Theorem 1, 11. o.). A tételt és bizonyítását itt nem reprodukáljuk. A pooling ettől eltérő helyzet: már **diszkrét** jelet ritkítunk tovább, és a képek általában nem sávkorlátosak, ezért a tétel feltételei itt nem teljesülnek, és rá nem hivatkozhatunk bizonyításként. Amit a laborban szemléltetünk, az a diszkrét ritkítás aliasing-jelensége: ha a ritkább rácson ábrázolható legnagyobb frekvenciánál magasabb komponens van a jelben, az a ritkítás után alacsonyabb frekvenciaként jelenik meg. Ez a szakasz szemléltetés és mérés, nem a folytonos tétel alkalmazása.

Az **anti-aliasing** gondolata: a leskálázás előtt egy aluláteresztő szűrővel csökkentjük a magas frekvenciákat, gyakran egyszerűen egy kis kernelű átlagolással, például 2×2 box-szűrővel. Ha a szűrés megelőzi a ritkítást, akkor a leskálázás kevésbé éles, de kevésbé fázisfüggő mintázatot kap. A pooling-kontextusban a $k = 2$, $s = 2$ average pooling éppen egy ilyen box-szűrő és a ritkítás egylépéses kombinációja; a max pooling nem lineáris aluláteresztő, ezért az anti-aliasing szerepét nem ugyanúgy tölti be.

A szakirodalmi kapcsolat: Zhang 2019 absztraktja szerint a stride-os konvolúciós hálózatokban a downsampling aliasing-ot okozhat, és blur-szűrős pooling javasolható az eltolási invariancia javítására [P03]. A cikk részletes módszertanát itt nem tárgyaljuk. Mi itt csak az anti-aliasing elvét (a leskálázás *előtt* aluláteresztő szűrés kell, azonos fázisú mintavétellel) vizsgáljuk, nem állítjuk a javasolt architektúra általános fölényét.

16.2.5 Global average pooling és információs veszteség

Számos osztályozó architektúra, különösen a későbbi fejezetekben tárgyalt modern CNN-ek esetében, az utolsó konvolúciós feature-térképeket **global average poolinggal** (GAP) redukálja. A GAP lényegében egyetlen ablak az egész térbeli dimenzióra:

$$Y[n, c] = \frac{1}{HW} \sum_{p=0}^{H-1} \sum_{q=0}^{W-1} X[n, c, p, q],$$

a kimenet alakja $N \times C$ vagy $N \times C \times 1 \times 1$, attól függően, hogy a könyvtár megőrzi-e a térbeli szingleton (egyetlen elemből álló) dimenziókat. A GAP a teljes térbeli rácsot egyetlen aggregátummá sűríti, ezért explicit térbeli invarianciát ad: minden olyan permutáció, amely az intenzitásokat a képen belül átrendezi, ugyanazt a GAP-kimenetet adja. Ez erősebb, mint az eltolási invariancia, és egyben mutatja, mekkora árat fizetünk: minden térbeli szerkezet elvész.

Ezt az információs veszteséget pontosan ki tudjuk mondani a nem injektivitás nyelvén.

3. definíció (információs veszteség). Egy $F : \mathcal{X} \to \mathcal{Y}$ leképezés akkor nem injektív, ha létezik $X_1 \neq X_2$ az $\mathcal{X}$-ből úgy, hogy $F(X_1) = F(X_2)$. Ekkor a kimenet önmagában nem teszi lehetővé X_1 és X_2 megkülönböztetését; a leképezésnek nincs balinverze.

Lemma (a pooling nem injektív). Legyen $k \geq 2$, és tekintsük az egyablakos $P_{\max} : \mathbb{R}^k \to \mathbb{R}$, illetve $P_{\text{avg}} : \mathbb{R}^k \to \mathbb{R}$ leképezéseket. Mindkettő nem injektív.

Bizonyítás. $P_{\max}$ esetén $P_{\max}(1,0,0,\dots,0) = 1 = P_{\max}(0,1,0,\dots,0)$, miközben a bemenetek különböznek, ha $k \geq 2$. P_{avg} esetén ugyanaz az összeg többféleképpen is előállítható: $P_{\text{avg}}(1,0,0,\dots,0) = \frac{1}{k} = P_{\text{avg}}(0,1,0,\dots,0)$, és $k \geq 2$ miatt a két bemenet különböző.

A max pooling egy további, specifikus veszteséget is okoz: az ablakból csak a legnagyobb értéket tartja meg, a maximális pozíciót és az összes többi értéket eldobja. A backpropagation során — lásd a 15.2. Inputgradiens és átfedő recepciós mezők scatter-add (szétszórás–összegzés) fogalmát — a gradiens csak a maximum pozíciójába folyik tovább, az ablak többi eleme zéró gradienst kap. Az átlag pooling ezzel szemben lokális zajcsökkentést valósíthat meg. Legyen az ablak elemszáma $m = k_h k_w$, a pixelek varianciája σ^2, páronkénti korrelációjuk ρ. Ekkor az átlag varianciája

$$\operatorname{Var}(\bar{X}) = \frac{\sigma^2}{m}[1 + (m-1)\rho].$$

Korrelálatlan esetben ($\rho = 0$) ez σ^2/m, ami $m > 1$ esetén szigorúan kisebb σ^2-nél; tökéletesen korrelált, azonos pixeleknél ($\rho = 1$) nem csökken, mert az érték σ^2. A $0 < \rho < 1$ köztes esetben a csökkenés kisebb, mint σ^2/m, ezért a σ^2/m alak csak korrelálatlan esetre érvényes. Az átlag pooling a kontrasztos finom struktúrát ugyanúgy elmossa.

16.2.6 Kézi számpélda

Vegyük a $(H, W) = (4, 4)$ bemenetet, $N = 1$, $C = 1$:

$$X = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 3 & 2 & 0 \\ 0 & 0 & 4 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}.$$

$k_h = k_w = 2$, $s_h = s_w = 2$ maxpooling esetén a kimenet 2×2:

- bal felső ablak $\{1, 0, 0, 3\}$, maximum 3;
- jobb felső ablak $\{2, 1, 2, 0\}$, maximum 2;
- bal alsó ablak $\{0, 0, 1, 0\}$, maximum 1;
- jobb alsó ablak $\{4, 0, 0, 1\}$, maximum 4.

Tehát

$$Y = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}.$$

Ugyanezzel az ablakkal average pooling esetén:

- bal felső átlag $(1 + 0 + 0 + 3)/4 = 1$;
- jobb felső $(2 + 1 + 2 + 0)/4 = 1{,}25$;
- bal alsó $(0 + 0 + 1 + 0)/4 = 0{,}25$;
- jobb alsó $(4 + 0 + 0 + 1)/4 = 1{,}25$.

$$Y_{\text{avg}} = \begin{bmatrix} 1 & 1{,}25 \\ 0{,}25 & 1{,}25 \end{bmatrix}.$$

Vegyük most az egypixeles vízszintes eltolást a belső tartományon, zéró határral: $T_1 X$ negyedik sora $[0, 1, 0, 0]$ lesz, a harmadik $[0, 0, 0, 4]$, és így tovább. A maxpooling kimenete:

- bal felső ablak $\{0, 1, 0, 0\} \to 1$;
- jobb felső ablak: a második sor $[0, 3, 2, 0]$ jobbra tolódik $[0, 0, 3, 2]$, tehát a jobb felső ablak $\{0, 2, 3, 2\} \to 3$;
- bal alsó $\{0, 0, 0, 1\} \to 1$;
- jobb alsó ablak: a harmadik sor $[0, 0, 0, 4]$; a jobb alsó ablak $\{0, 4, 0, 0\} \to 4$.

Az eredmény $\begin{bmatrix} 1 & 3 \\ 1 & 4 \end{bmatrix}$, amely nem az eredeti $\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$ semmilyen kimeneti eltolása. Az egypixeles eltolás tehát nem konzisztens *kimeneti* eltolásként jelenik meg, hanem az ablakhatárokon átrendezi a maximumokat.

16.2.7 Minimális NumPy kód és jelölt kimenet

A következő kód a kézi példa műveletét adja meg NCHW bemenetre, padding és dilation nélkül, float32 pontossággal. A kimenet a futtatott kód mért eredménye (Python 3.12.3, NumPy 2.5.4, CPU: 16 mag, szálkorlát nélkül; a példa determinisztikus, seed nem szükséges).

```
# munkakönyvtár: lab/
# fájl: pooling_reference.py
import numpy as np

def max_pool2d(X: np.ndarray, k: tuple[int, int], s: tuple[int, int]) -> np.ndarray:
    """NCHW max pooling, zero padding nélkül, teljes ablakokkal.

    X: float32, alak (N, C, H, W).
    k, s: (kh, kw), (sh, sw) pozitív egészek.
    """
    N, C, H, W = X.shape
    kh, kw = k
    sh, sw = s
    Ho = (H - kh) // sh + 1
    Wo = (W - kw) // sw + 1
    Y = np.empty((N, C, Ho, Wo), dtype=np.float32)
    for i in range(Ho):
        for j in range(Wo):
            patch = X[:, :, i*sh:i*sh+kh, j*sw:j*sw+kw]
            Y[:, :, i, j] = patch.max(axis=(2, 3))
    return Y

# kézi példa
X = np.array([[[
    [1,0,2,1],
    [0,3,2,0],
    [0,0,4,0],
    [1,0,0,1]
]]], dtype=np.float32)
Y = max_pool2d(X, (2,2), (2,2))
assert Y.shape == (1, 1, 2, 2)
assert Y.tolist() == [[[[3.0, 2.0], [1.0, 4.0]]]]
print(Y.shape, Y.tolist())
# mért kimenet: (1, 1, 2, 2) [[[[3.0, 2.0], [1.0, 4.0]]]]
```

A stride és a nem teljes ablak: ha $(H - k) \bmod s \neq 0$, a kód az utolsó nem teljes ablakot eldobja, tehát a floor-képlet érvényes. A PyTorch `nn.MaxPool2d(kernel_size=2, stride=2)` ugyanezt az alakot adja; a poolingnak nincs tanulható paramétere.

16.2.8 Labor: sakktábla- és sávos jel szűrés előtti/utáni leskálázással

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A szkript a szakasz kódblokkjából futtatható; a bemenet két szintetikus jel, külső adatfájl nem kell.

A labor célja az aliasing szemléltetése. A megfigyelési egység egyetlen szintetikus kép; az eredmény a futtatott kód mért kimenete. Az állítások csak a konkrét bemenetre vonatkoznak.

Bemenet. Két 8×8-as jel, $N = 1, C = 1$, $[0, 1]$ tartományban, float32:

1. sakktábla: $C_1[i, j] = (i + j) \bmod 2$;
2. sávos jel: $S_1[i, j] = 1$, ha $(i - 1) \bmod 4 < 2$, egyébként 0 (2 pixel széles vízszintes sávok 4 pixel periódussal, eltolt fázissal).

Protokoll. Három műveletet hasonlítunk össze:

- (a) közvetlen leskálázás 2×2 maxpooling, stride 2;
- (b) közvetlen leskálázás 2×2 average pooling, stride 2;
- (c) anti-aliasing szűrés: először 2×2 box-szűrő stride 1-gyel, majd a box-szűrés 7×7-es kimenetéből minden második pixel vétele (stride 2, szűrés utáni leskálázás).

Futtatott kimenetek. A kimeneteket Python 3.12.3-mal, NumPy 2.5.4-gyel, CPU-n futtattuk; seed nem szükséges, mert nincs véletlen szám. Az assertek ezeket az értékeket rögzítik.

A sakktáblánál a 2×2 ablak mindig két 0-t és két 1-et tartalmaz, a maxpooling ezért minden ablakban 1-et, az average pooling minden ablakban 0,5-öt ad. A (c) box-szűrő a belső 7×7 rácson mindenhol 0,5-öt ad, mert minden 2×2 ablak átlaga 0,5; a ritkított mintavétel ezért mindkét fázisban konstans 0,5-öt ad.

A sávos jelnél a 2×2 ablak mindig egy 1-es és egy 0-s sort tartalmaz, ezért a maxpooling (a) kimenete minden ablakban 1, az average pooling (b) kimenete pedig minden ablakban 0,5. A (c) box-szűrő utáni 7×7 függőleges profil $[0{,}5; 1; 0{,}5; 0; 0{,}5; 1; 0{,}5]$. A páros fázisú ritkítás ebből $[0{,}5; 0{,}5; 0{,}5; 0{,}5]$-öt ad, ami megegyezik az average pooling eredményével; a páratlan fázisú ritkítás viszont $[1; 0; 1]$-et ad. A box-szűrés tehát önmagában nem szünteti meg a fázisfüggést: a ritkítás fázisát is rögzíteni kell.

Értelmezés. A maxpooling nem simít, ezért a sávos jel csúcsait megtartja, de a 2×2 ablakon belüli különbséget eldobja: a kimenet konstans 1 lesz, és a sávok eltűnnek. Az average pooling lokálisan átlagol, ezért konstans 0,5-öt ad. A (c) változatban a box-szűrő a 2×2 blokkok átlagát számolja; a páros fázisú ritkítás ezért ugyanazt a 0,5-öt adja, mint az average pooling, a páratlan fázis viszont megőrzi az aliasing jelet. A sakktáblánál a box-szűrő minden 2×2 ablakban 0,5-öt ad, ezért ott a leskálázás fázisfüggetlen; a sávos jel mutatja, hogy ez nem általános.

Részlet a laborból futtatható parancsként:

```
cd lab && python -c "
import numpy as np
C = ((np.indices((8,8)).sum(0))%2).astype('float32')[None,None]
S = (((np.arange(8)[:,None]-1)%4)<2).astype('float32') * np.ones((1,8),'float32')
S = S[None,None]
def pool(X, mode):
    V = X[0,0].reshape(4,2,4,2)
    return (V.max(axis=(1,3)) if mode == 'max' else V.mean(axis=(1,3)))
def box(X):
    return (X[:,:, :-1, :-1] + X[:,:,1:, :-1] + X[:,:, :-1,1:] + X[:,:,1:,1:]) / 4
```

```
for name, X in [('C', C), ('S', S)]:
    B = box(X)
    print(name, 'max', pool(X, 'max').tolist())
    print(name, 'avg', pool(X, 'avg').tolist())
    print(name, 'box even', B[:,:,::2,::2][0,0].tolist())
    print(name, 'box odd', B[:,:,1::2,1::2][0,0].tolist())
    if name == 'C':
        assert np.allclose(pool(X, 'max'), 1.0)
        assert np.allclose(pool(X, 'avg'), 0.5)
        assert np.allclose(B[:,:,::2,::2], 0.5)
        assert np.allclose(B[:,:,1::2,1::2], 0.5)
    else:
        assert np.allclose(pool(X, 'max'), 1.0)
        assert np.allclose(pool(X, 'avg'), 0.5)
        assert np.allclose(B[:,:,::2,::2], 0.5)
        assert np.allclose(B[:,:,1::2,::2][0,0,:,0], [1.0, 0.0, 1.0])
"
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU: 16 mag, szálkorlát nélkül):

```
C max [[1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0]]
C avg [[0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5]]
C box even [[0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5,
↪  0.5]]
C box odd [[0.5, 0.5, 0.5], [0.5, 0.5, 0.5], [0.5, 0.5, 0.5]]
S max [[1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0], [1.0, 1.0, 1.0, 1.0]]
S avg [[0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5]]
S box even [[0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5, 0.5], [0.5, 0.5, 0.5,
↪  0.5]]
S box odd [[1.0, 1.0, 1.0], [0.0, 0.0, 0.0], [1.0, 1.0, 1.0]]
```

Ügyeljen az `np.indices` használatára: az `np.indices((8,8)).sum(0)` a sakktáblát adja. A futtatás seedet nem igényel, mert nincs véletlen szám; a dtype float32, a device CPU. GPU igénybevétele nem szükséges, és kis példánkban nem a memóriakezelés, hanem a szemléltetés a cél.

16.2.9 Gyakori tévedés: a maxpool mint univerzális robusztussági garancia

Gyakran hallani, hogy a maxpooling „érzéketlenné tesz a kis eltolásokra”. Ez az állítás túl erős, és az előző lemmában szereplő ellenpélda cáfolja. A maxpooling csak *lokálisan*, az ablakon belül érzéketlen az értékek permutációjára; a kimenet akkor ugrik, ha az egypixeles eltolás a maximumot átviszi egy másik ablakba (az ablakon belüli áthelyezés a maximum értékét nem változtatja). A 16.1. Eltolási equivariance és invariance terminológiájával élve: az invariancia nem következik a max műveletből, legfeljebb részcsoport-szintű equivariance vagy közelítő invariancia. Zhang 2019 absztraktja is felhívja a figyelmet arra, hogy a modern hálózatok downsampling-műveletei nem feltétlenül eltolás-invariánsak [P03].

16.2.10 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrizze, hogy meg tudja-e válaszolni a következőket. (1) Mi a különbség az $s = 1$ és $s = 2$ pooling között a kimeneti rács sűrűségét tekintve? (2) Miért nem létezik rögzített kimeneti eltolás az egypixeles bemeneti eltolásra a $k = 2, s = 2$ maxpoolingnál? (3) Mi az anti-aliasing lépés célja a downsampling előtt? (4) Miért nem injektív a maxpooling, és miért releváns ez a reprezentációs veszteségre?

Rövid válaszok. (1) $s = 1$ esetén a kimeneti rács sűrűsége megmarad, $s = 2$ esetén minden dimenzióban feleződik, azaz ritkább kimeneti rácsra térünk át. (2) A $k = 2, s = 2$ maxpooling

az ablakon belüli pozíciót elfelejti, ezért az egypixeles bemeneti eltolás néhány bemeneten megváltoztathatja, hogy melyik elem a maximum; nincs olyan rögzített kimeneti eltolás, amely minden bemenetre ugyanezt a változást írná le. (3) Az anti-aliasing aluláteresztő szűrés a ritkítás előtt csökkenti a magas frekvenciák átfordulását (aliasingját) a kisebb rácson. (4) A maxpooling nem injektív, mert egy ablak nem-maximum elemei elvesznek, ezért több bemenet ugyanarra a kimenetre képződik; ez a reprezentációs veszteség forrása.

Visszahívás a 15.2. Inputgradiens és átfedő recepciós mezők egységből: a konvolúciós inputgradiensben az átfedő patch-ek hozzájárulásait scatter-add formában kellett összegezni. A maxpooling backward ezzel szemben nem szór vissza minden ablakpixelbe, csak a maximum pozíciójába, ezért az átfedés esetén más gradienssűrűséget kapunk. A 4.2. Parciális derivált és gradiens egység gradiensfogalma itt akkor válik majd fontossá, amikor a tanítható hálózatban a pooling rétegen keresztül visszavezetjük a veszteséget.

16.2.11 Rövid összefoglaló

A pooling réteg két funkciót ötvöz: lokális aggregációt és stride-os mintavétel-csökkentést. Az aggregáció $s = 1$ esetén megőrzi a bemeneti rács sűrűségét; a stride $s > 1$ esetén ritkább kimeneti rácsra térünk át. A bizonyított ellenpélda szerint a $k = 2, s = 2$ maxpoolingra nincs rögzített kimeneti eltolás, amely az egypixeles bemeneti eltolást konzisztensen reprezentálná, mert az ablakon belüli pozíció elvész. A downsampling ezért aliasing-fázisfüggőséget okozhat; az aluláteresztő szűrés a ritkítás előtt csökkenti ezt. A global average pooling teljes térbeli invarianciát ad, de nem injektív, tehát a finom szerkezetet eltünteti.

16.2.12 Következő alkalmazás

A 16.3. Elméleti és effektív recepciós mező egységben a recepciós mezőt általánosítjuk: a pooling ablakai és stride-jai a konvolúciós rétegek recepciós mezőjét nyújtják meg, ezért az itt tárgyalt rácssűrűség-csökkenést a többrétegű architektúrákban is el kell tudni számolni.

16.2.13 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem nevezhető a $k = 2$, $s = 1$ average pooling downsamplingnak, és miért nevezhető annak a $k = 2$, $s = 2$ average pooling. Adja meg mindkettő kimeneti alakját egy 6×6-os bemenetre.

F2 (levezetési, nehézség 2). Bizonyítsa be a maxpooling nem injektivitásáról szóló lemmát $k = 2$-re úgy, hogy két konkrét vektort ad meg, amelyek képe azonos. Adjon hasonló bizonyítást az average poolingra is.

F3 (implementációs, nehézség 2). Írjon NumPy függvényt NCHW average poolingra, kernel és stride támogatással, teljes ablakokkal; ellenőrizze a 2×2, stride 2 műveletet a fejezet kézi példáján, és egy $H = W = 5$, $k = 2$, $s = 1$ esetet. Tegye lehetővé a global average pooling is a teljes térbeli méret megadásával.

F4 (hibakeresési, nehézség 2). A következő sor hibás: `Y[...] = patch.sum(axis=(2,3)) / (kh + kw)` átlagpooling kiszámításához. Mi a hiba oka, mi a következménye, és hogyan javítja ki? Mutassa meg, hogy a $k_h = k_w = 2$ eset még nem leplezi le a hibát, mert $2 + 2 = 2 \cdot 2$, majd egy 3×3-as ablakon szemléltesse a torzítást.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen GlyphLab-alapú kísérletet (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték), amely összehasonlítja

a max- és average pooling hatását egy kicsi CNN osztályozási pontosságára a clean és corrupt teszten. Ne feltételezze, hogy az egyik győz; adja meg a megfigyelési egységet, a splitet, a baseline-t, a változó tényezőt, a metrikát és egy érvényességi korlátot. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

16.2.14 Megoldások és indoklás

F1. A $k = 2, s = 1$ average pooling esetén a kimeneti rács $6 - 2 + 1 = 5$, tehát a térbeli felbontás majdnem megmarad. A leképezés csak lokális aggregációt végez, nincs ritkítás. A $k = 2, s = 2$ average pooling kimeneti mérete $\lfloor (6-2)/2 \rfloor + 1 = 3$, a rács feleződik, ezért downsampling. Mindkettő kimeneti alakja $N \times C \times H_{out} \times W_{out}$: egyetlen $N = C = 1$ bemenetre az első $1 \times 1 \times 5 \times 5$, a második $1 \times 1 \times 3 \times 3$.

F2. A maxpoolingra $k = 2$: $P_{\max}(0,1) = 1$ és $P_{\max}(1,0) = 1$, de $(0,1) \neq (1,0)$, tehát nem injektív. Average poolingra: $P_{\text{avg}}(1,0) = \frac{1}{2} = P_{\text{avg}}(0,1)$, a bemenetek különböznek, a kép azonos. A lemma érvényes.

F3. Egy megoldás:

```
import numpy as np

def avg_pool2d(X, k, s):
    N, C, H, W = X.shape; kh, kw = k; sh, sw = s
    Ho = (H - kh)//sh + 1; Wo = (W - kw)//sw + 1
    Y = np.empty((N, C, Ho, Wo), dtype=X.dtype)
    for i in range(Ho):
        for j in range(Wo):
            patch = X[:, :, i*sh:i*sh+kh, j*sw:j*sw+kw]
            Y[:, :, i, j] = patch.mean(axis=(2, 3))
    return Y

X4 = np.array([[[[1,0,2,1],[0,3,2,0],[0,0,4,0],[1,0,0,1]]]], dtype=np.float32)
Y4 = avg_pool2d(X4, (2, 2), (2, 2))
assert Y4.shape == (1, 1, 2, 2)
assert np.allclose(Y4, [[[[1.0, 1.25], [0.25, 1.25]]]])

X5 = np.arange(25, dtype=np.float32).reshape(1, 1, 5, 5)
Y5 = avg_pool2d(X5, (2, 2), (1, 1))
assert Y5.shape == (1, 1, 4, 4)

Ygap = avg_pool2d(X5, (X5.shape[2], X5.shape[3]), (1, 1))  # global average pooling
assert Ygap.shape == (1, 1, 1, 1) and np.isclose(Ygap.squeeze(), X5.mean())
print(Y4.shape, Y5.shape, Ygap.shape)
# mért kimenet: (1, 1, 2, 2) (1, 1, 4, 4) (1, 1, 1, 1)
```

A global average pooling tehát nem külön függvény: `k=(H, W)`, `s=(1, 1)` mellett ugyanez a hívás a teljes térbeli rácsot egyetlen értékké átlagolja, és a kimenet alakja a szingleton dimenziókat megőrző konvencióban $(N, C, 1, 1)$. A kézi példán a mért mátrix $\begin{bmatrix} 1 & 1{,}25 \\ 0{,}25 & 1{,}25 \end{bmatrix}$; a 4×4-es bemenet GAP-ja 0,9375. A $H = W = 5$, $k = 2$, $s = 1$ eset kimenete a futtatott kód szerint $(1, 1, 4, 4)$, mert $\lfloor (5-2)/1 \rfloor + 1 = 4$.

F4. A hiba oka a nevező: az átlag az ablak elemszámának $k_h k_w$, nem $k_h + k_w$ osztóval történik. A 2×2 ablakban $2 + 2 = 2 \cdot 2 = 4$, ezért a hiba itt nem jelentkezik; a 2×2-es eset tehát nem alkalmas a hiba bemutatására. Torzítás lép fel minden olyan pozitív egész (k_h, k_w) párra, ahol $k_h + k_w \neq k_h k_w$; a $(2, 2)$ az egyetlen kivétel, mert $(k_h - 1)(k_w - 1) = 1$ csak ekkor teljesül. Például 3×3-nál a helyes nevező 9, a hibás 6, ezért az átlag a helyes érték 3/2-szeresére nő; 1×1-nél

a helyes nevező 1, a hibás 2, ezért az átlag feleződik. Következmény: az átlag érték tévesen nagyobb lesz, a kimenet eltolódik. Javítás: `/ (kh * kw)`. Ellenőrzés a fejezet számpéldáján: a bal felső ablak összege 4, a helyes átlag 1, a hibás kód $\frac{4}{4} = 1$ itt nem mutatja a hibát; válasszon $k_h = 3, k_w = 2$ ablakot, ahol az összeg 5, a helyes átlag $\frac{5}{6} \approx 0{,}833$, a hibás $\frac{5}{5} = 1$.

F5. Egy lehetséges terv. Adat: a **GlyphLab** a könyv saját, helyben generált szintetikus adatkészlete (a 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték A. függeléke): 180 jelenet, jelenetenként két zajos nézet, összesen 360 kép, 24×24, float32, $[0, 1]$, három vonalorientáció-osztály. A szerződés szerinti jelenetszintű split 120/30/30 jelenet, azaz 240/60/60 kép train/val/test. A **corrupt teszt** a szerződés szerinti `data/corrupted_test.npy`: a 60 testkép zajjal és fényességgradienssel, a test teljes sorrendjében, ezért a clean–corrupt összehasonlítás jelenet–nézet páronként páros. Megfigyelési egység: `scene_id`; a train/val/test split jelenetszintű, egy jelenet két nézete nem kerülhet különböző splitbe. Kérdés: változik-e a clean és corrupt teszten mért accuracy, ha a fix konvolúciós törzs utáni pooling max vagy average? Háló: `Conv2d(1, 8, 3, padding=1)`–ReLU–`Conv2d(8, 16, 3, padding=1)`–ReLU–pool(2×2, stride 2)–GAP–`Linear(16, 3)`; a bemenet $1 \times 24 \times 24$, a pooling után $16 \times 12 \times 12$, a GAP után 16, a kimenet 3 logit. Tanítás: Adam optimalizáló, tanulási ráta 10^{-3}, batch 32, legfeljebb 30 epoch, a legjobb checkpoint kiválasztása a validációs veszteség alapján, rögzített seed 20261020, float32 CPU. Kontroll (baseline): ugyanaz a háló, ugyanaz a tanítás és seed, csak max poolinggal; a kezelt tényező az average pooling. Metrika: per-image accuracy a clean és a corrupt teszten, valamint a páros clean–corrupt csökkenés jelenetenként. Érvényességi korlát: az adat szintetikus, az eredmény nem általánosítható természetes képekre; egy seed nem elegendő a szóráshoz, ezért ismételt seedek vagy konfidenciaintervallum szükséges az állításhoz. Előre rögzített győztest nem állítunk; a negatív eredmény is rögzítendő.

16.2.15 Források

- [P28] C. E. Shannon (1949). Communication in the Presence of Noise. *Proceedings of the I.R.E.*, 37(1), 10–21. doi:10.1109/JRPROC.1949.232969. A mintavételi tétel: II. szakasz, Theorem 1, 11. o.
- [P03] R. Zhang (2019). *Making Convolutional Networks Shift-Invariant Again.* PMLR 97:7324–7334. https://proceedings.mlr.press/v97/zhang19a.html.

16.3 Elméleti és effektív recepciós mező

16.3.1 Cél, előismeret és induló becslés

Ez az egység a konvolúciós hálózatok egyik legfontosabb, de gyakran félreértett szerkezeti fogalmát vizsgálja: mekkora bemeneti területet lát egy kimeneti neuron? A kérdés nem csupán elméleti; a tervezés során abból indulunk ki, hogy a modellnek akkora kontextust kell kapnia, amekkorát a feladat megkíván. A szükséges előismeret a 16.2. Pooling, downsampling és aliasing alfejezetben tárgyalt pooling és stride fogalma, valamint a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben bevezetett VJP és a 15.3. Transposed convolution mint adjungált alfejezetben tárgyalt lineáris operátoros leírás; a 12.3. Sanity check és kis mintára túlillesztés alfejezetből a gradiensalapú érzékenységvizsgálat szemlélete tér vissza.

Induló becslés: egy 4 rétegű, mindenhol 3×3-as, stride 1 konvolúciót használó hálózat kimenetének egyetlen neuronja hány pixelből álló bemeneti négyzetet lát? A kézenfekvő válasz $1+2+2+2+2 = 9$, azaz 9×9; a pontos érvelést és az ettől eltérő eseteket ebben az egységben dolgozzuk ki.

16.3.2 Definíciók: elméleti recepciós mező, jump és effektív mező

A következő fogalmakat 1D-ben definiáljuk, majd a 2D kiterjesztést adjuk meg. Az 1D jelölés a térbeli dimenziókat világosabban mutatja, és a 2D eredmény téglalap kernel esetén dimenziónként szeparálható.

Elméleti recepciós mező (receptive field, RF). Legyen adott egy L rétegből álló soros hálózat, amelyben minden réteg egyetlen térbeli dimenzión konvolúciót, poolingot vagy más, kernelablakkal dolgozó műveletet végez. Az l réteg kimenetének egy rögzített pozíciójához tartozó elméleti recepciós mező F_l az eredeti bemeneten azon pozíciók **legkisebb befoglaló intervallumának hossza**, amelyek értékétől a kimenet függhet. A kezdeti réteg a bemenet maga: $F_0 = 1$. A befoglaló intervallumot azért használjuk, mert dilation esetén a ténylegesen hozzáférhető pontok nem feltétlenül alkotnak kontiguus halmazt.

Egy fontos finomság: F_l a hozzáférhetőség **felső határa**, nem a ténylegesen megvalósuló hatás leírása. Ha egy súly nulla, a hozzá tartozó bemeneti pozíció nem befolyásolja a kimenetet, de az architektúra által elvileg elérhető marad.

Jump (lépésköz). A J_l lépésköz azt adja meg, hogy az l réteg kimenetén két szomszédos pozícióhoz tartozó bemeneti koordináták az eredeti bemeneti rácson mekkora távolságra vannak egymástól. Kezdetben $J_0 = 1$, mert a bemeneti rács két szomszédos pixele egy egységnyire van. A jump a kimeneti rács tényleges ritkábbodását írja le, ezért a stride-ok szorzata jelenik meg benne.

Jelölések. Az l réteg kernelmérete k_l, stride-ja s_l, dilationje d_l; mind pozitív egész. A padding p_l nemnegatív egész, de a RF nagyságába nem szól bele, mert a zero padding virtuális, és nem szállít valódi bemeneti értéket. A bias ugyancsak térbelileg konstans, ezért nem változtatja a mezőt. A csatornákra szorítkozva a standard *groups* $= 1$ esetet használjuk; a térbeli RF a csatornaszámtól független, mert a bemeneti csatornák ugyanazt a térbeli régiót osztják meg. A képletekben F_l mindig a recepciós mező hosszát jelöli; az F1–F5 feladatsorszámok és az A, B, C architektúracímkék ettől független azonosítók, nem képletbeli mennyiségek.

Effektív recepciós mező (ERF). Az effektív recepciós mező az input pixelek **mért érzékenységének** eloszlása egy adott kimeneti neuronra nézve. A fogalmat és gradiensalapú vizsgálatát Luo és mtsai (2016) vezették be [P31]. Mérhetjük például a loss bemeneti gradiensének abszolút értékével:

$$G_{ij} = \sum_{n,c} \left| \frac{\partial \mathcal{L}}{\partial X_{n,c,i,j}} \right|,$$

ahol $\mathcal{L}$ egy választott kimeneti neuron értéke (vagy a rá vonatkozó loss), és a batch (n) és csatorna (c) tengelyek aggregálásával egy kétdimenziós hőtérképet kapunk. A véges képen a széli neuron valódi mezejét a kezdőkoordináta és a kép határával vett metszet adja. Jelölje H a bemeneti kép térbeli méretét, J_L a kimeneti réteg jumpját, F_L az elméleti befoglaló hosszt, p_l pedig az l réteg paddingjét. A rétegenkénti p_l-től megkülönböztetjük a padding kimeneti koordinátákra gyakorolt összesített eltolását, $P_L = \sum_{l=1}^{L} p_l J_{l-1}$-et; a képletekben ezt a P_L mennyiséget használjuk. Ekkor az i-edik kimeneti pozíció mezeje

$$\left[\, \max\left(0,\ iJ_L - P_L\right),\ \min\left(H,\ iJ_L - P_L + F_L\right) \right),$$

amely a széleken kisebb a nominális F_L-nél; a belső pozíciókon, ahol a metszet nem vág, a hossza F_L. Az ERF nem az elméleti RF automatikus tulajdonsága: a gradiens eloszlása a konkrét súlyoktól és a bemenettől is függhet.

16.3.3 Tétel: a RF és a jump rekurziója

A következő állítás a soros rétegek recepciós mezőjének alapegyenlete. A bizonyítás elemi indukció, ezért a tankönyv teljes levezetéssel adja.

Tétel (soros RF-rekurzió). Legyenek $l = 1, \dots, L$ soros 1D rétegek, rendre (k_l, d_l, s_l) paraméterekkel. Legyen $F_0 = 1$ és $J_0 = 1$. Ekkor minden $l \geq 1$ esetén

$$J_l = J_{l-1}\, s_l, \qquad F_l = F_{l-1} + (k_l - 1)\, d_l\, J_{l-1}.$$

A padding a befoglaló hossz rekurziójában nem szerepel; a stride csak a jump-on keresztül hat a következő réteg mezőjére. A rekurzió az elméleti, végtelen rácson vett befoglaló hosszat adja. A véges bemeneten a ténylegesen elérhető indexek intervalluma a fenti $[\max(0, iJ_L - P_L), \min(H, iJ_L - P_L + F_L))$ metszet, amelynek hossza legfeljebb F_L, a széleken pozíciófüggően kisebb lehet, a belső pozíciókon pedig F_L. A definícióban szereplő legkisebb befoglaló intervallum tehát a végtelen rácson F_L, a véges bemeneten ennek a metszete; a kettő a belső tartományon egyezik.

Bizonyítás. A jump-ra vonatkozó állítás közvetlen: az l réteg kernelét két szomszédos kimeneti pozíció esetén az előző réteg kimenetén s_l rácslépés választja el. Az előző réteg kimenetének egy rácslépése az eredeti bemeneti rácson J_{l-1} egységnek felel meg, így a kimeneti rács szomszédos elemeihez tartozó bemeneti koordináták különbsége $s_l J_{l-1}$. Tehát $J_l = J_{l-1} s_l$.

A mezőhöz tekintsük az l réteg kernelét. A kernel az előző réteg kimenetéből k_l darab pozíciót használ, éspedig $0, d_l, 2d_l, \dots, (k_l - 1)d_l$ offsettel a kernel kezdőpontjához képest. Minden ilyen előző kimeneti pozíció az eredeti bemeneten egy legfeljebb F_{l-1} hosszúságú befoglaló intervallumtól függ. Két szomszédos mintavételezett előző pozíció között d_l darab előző kimeneti rácslépés van, tehát a két befoglaló intervallum kezdőpontjának távolsága az eredeti bemeneti rácson $d_l J_{l-1}$. A legkisebb befoglaló intervallum hossza ezért

$$F_{l-1} + (k_l - 1)\,(d_l J_{l-1}),$$

mert a kernel $(k_l - 1)$ darab ilyen távolságnyi szakasszal bővíti a kiinduló F_{l-1} hosszú intervallumot. Ez éppen a kívánt rekurzió.

A tétel feltételei közt a rétegek soros elrendezése szerepel; skip-connection esetén a mező a két ág uniója, ezért az itteni egyszerű rekurzió nem alkalmazható változatlanul. Ha az egyik ág az $[a_1, b_1)$, a másik az $[a_2, b_2)$ intervallumot fedi le, akkor az unió $[\min(a_1, a_2), \max(b_1, b_2))$, amelynek hossza $\max(b_1, b_2) - \min(a_1, a_2)$. Például az identitáság $[i, i+1)$ és egy 3×3-as konvolúciós ág $[i-1, i+2)$ uniója $[i-1, i+2)$, hossza 3, nem pedig $1 + 3 = 4$: az unió nem összeadja, hanem összefogja az intervallumokat.

Következmény (zárt képlet). Mivel $J_{l-1} = \prod_{m=1}^{l-1} s_m$, az L rétegű soros hálózat RF-je

$$F_L = 1 + \sum_{l=1}^{L} (k_l - 1)\, d_l \prod_{m=1}^{l-1} s_m.$$

A bizonyítás teleszkópikus összeg: $F_l - F_{l-1} = (k_l - 1) d_l J_{l-1}$, és J_{l-1} a stride-ok szorzata.

2D kiterjesztés. Téglalap kernel esetén két külön rekurziót futtatunk a magasságra és a szélességre. A H dimenzióra k_h, d_h, s_h, a W dimenzióra k_w, d_w, s_w paraméterekkel az F_h és F_w értékeket a fenti képlettel számoljuk, a teljes elméleti befoglaló téglalap pedig $F_h \times F_w$. Az itt használt konvolúciós implementáció szimmetrikus zero padding mellett cross-correlationt számol, ezért a RF számítása ugyanaz, mint a megfordított kernelű matematikai konvolúcióé.

16.3.4 Kézi kis példa és ellenőrzés

Tekintsünk három réteget:

1. réteg: $k_1 = 3, d_1 = 1, s_1 = 1$;
2. réteg: $k_2 = 2, d_2 = 1, s_2 = 2$ (pl. maxpool);
3. réteg: $k_3 = 3, d_3 = 2, s_3 = 1$.

A rekurzió lépésről lépésre:

- $F_0 = 1, J_0 = 1$.
- $l = 1$: $F_1 = 1 + (3-1) \cdot 1 \cdot 1 = 3$, $J_1 = 1 \cdot 1 = 1$.
- $l = 2$: $F_2 = 3 + (2-1) \cdot 1 \cdot 1 = 4$, $J_2 = 1 \cdot 2 = 2$.
- $l = 3$: $F_3 = 4 + (3-1) \cdot 2 \cdot 2 = 4 + 8 = 12$, $J_3 = 2 \cdot 1 = 2$.

A harmadik réteg kimenetének egy neuronja tehát a bemenet egy 12 hosszúságú intervallumától függhet, a kimeneti rács pedig minden második bemeneti pixelen mintavételez. A dilation a 3. rétegben 8 egységgel növelte a mezőt, pedig a kernel fizikailag csak 3 együtthatót tartalmaz.

16.3.5 Labor: három architektúra RF-e és gradiensalapú érzékenység

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Az `erf_lab.py` a `lab/erf/` munkakönyvtárból futtatható (`python erf_lab.py`); a modellek inicializálását rögzített seed adja.

A labor három kis konvolúciós hálózat RF-ét számítja ki a rekurzióval, majd a gradiens befoglaló dobozának méretét hasonlítja az elmélethez. Minden modell egyetlen bemeneti és kimeneti csatornával, bias nélkül, ReLU nélkül készül, hogy a lineáris map gradiensét vizsgálhassuk.

A munkakönyvtár legyen `lab/erf/`. A következő kódot mentsük `erf_lab.py` néven, és futtassuk a

```
python erf_lab.py
```

paranccsal. A kód CPU-n fut, `float32` dtype-pal, `torch.manual_seed(7)` inicializálással; a device `cpu`.

```
import torch
import torch.nn as nn

torch.manual_seed(7)
device = torch.device("cpu")

def rf_from_spec(spec):
    """spec: list of (kernel, dilation, stride) 1D/négyzetes esetre."""
    F = 1
    J = 1
    for k, d, s in spec:
        F = F + (k - 1) * d * J
        J = J * s
    return F, J

def build_layer(k, d, s):
    return nn.Conv2d(1, 1, kernel_size=k, dilation=d,
                     stride=s, padding=1, bias=False)

specs = {
```

```
    "A": [(3, 1, 1)] * 4,                    # négy 3x3, stride 1
    "B": [(3, 1, 2)] * 3,                    # három 3x3, stride 2
    "C": [(3, 1, 1), (3, 2, 1), (3, 4, 1)],  # dilation 1,2,4
}

for name, spec in specs.items():
    F, J = rf_from_spec(spec)
    model = nn.Sequential(*[build_layer(*p) for p in spec]).to(device)

    x = torch.ones(1, 1, 32, 32, device=device, requires_grad=True)
    out = model(x)
    cy, cx = out.shape[2] // 2, out.shape[3] // 2
    loss = out[0, 0, cy, cx]
    loss.backward()

    g = x.grad[0, 0].detach().abs()
    # A konvolúció lineáris, a bemenet csupa egyes, ezért a gradiens értékei
    # pontos nullák vagy a kernelszorzatok; az 1e-8 csak a float32 összegzési
    # zaját küszöböli, a nemnulla értékek nagyságrendje 1.
    nz = (g > 1e-8).nonzero()
    ymin, xmin = nz.min(dim=0).values
    ymax, xmax = nz.max(dim=0).values
    mh, mw = int(ymax - ymin + 1), int(xmax - xmin + 1)

    # a gradiens befoglaló dobozának középső 3x3 blokkja
    cy_eff, cx_eff = (ymin + ymax) // 2, (xmin + xmax) // 2
    mid = g[cy_eff - 1:cy_eff + 2, cx_eff - 1:cx_eff + 2].sum().item()
    total = g.sum().item()

    expected = {"A": (9, 1), "B": (15, 8), "C": (15, 1)}
    expected_out = {"A": (32, 32), "B": (4, 4), "C": (24, 24)}
    assert (F, J) == expected[name], (name, F, J)
    assert tuple(out.shape[2:]) == expected_out[name], (name, out.shape)
    assert (mh, mw) == (F, F), (name, mh, mw, F)
    assert 0.0 < mid / total <= 1.0

    print(f"{name}: out.shape={tuple(out.shape[2:])}, elméleti RF=({F},{F}), "
          f"jump=({J},{J}), mért doboz=({mh},{mw}), közép3x3 arány={mid/total:.3f}")
```

A három modell elméleti értéke:

- **A** (négy 3×3, stride 1): $F = 1 + 4 \cdot 2 = 9$, $J = 1$, kimenet 32×32;
- **B** (három 3×3, stride 2): $F = 1 + 2 \cdot (1 + 2 + 4) = 15$, $J = 8$, kimenet 4×4;
- **C** (dilation 1, 2, 4): $F = 1 + 2 + 4 + 8 = 15$, $J = 1$, kimenet 24×24.

A kód mért ellenőrzése (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`, seed 7): a gradiens befoglaló dobozának **mérete** rendre $(9, 9)$, $(15, 15)$ és $(15, 15)$, egyezve az elmélettel; a pontos pozíció a padding miatt eltolódhat, de a méret nem. A középső 3×3-as blokk energiaaránya A: 0,298, B: 0,074, C: 0,042 (az „energia” itt a G_{ij} definíciójához hasonlóan az abszolút gradiensértékek összege, nem négyzetösszeg); az A esetben ez érdemben meghaladja az egyenletes $9/81 \approx 0{,}111$ arányt, B-ben a $9/225 \approx 0{,}04$-hez képest mintegy 1,9-szeres, C-ben pedig gyakorlatilag az egyenletes értékkel egyezik. A táblázat a mért futás összefoglalója:

Architektúra	Elméleti RF	Jump	Gradiens dobozmérete	Közép 3×3 arány
A	9×9	1	9×9	0,298
B	15×15	8	15×15	0,074
C	15×15	1	15×15	0,042

16.3.6 Gyakori tévedés és ellenpélda

A leggyakoribb hiba azt állítani, hogy az elméleti RF nagysága önmagában garantálja a mező minden pixelének egyenlő vagy legalább érdemi hatását. Ez hamis.

Ellenpélda. Vegyük az A architektúrát (négy 3×3-os konvolúció, stride 1), amelynek elméleti RF-je 9×9. Mindegyik rétegben állítsuk a kernel középső elemét 1-re, az összes többi együtthatót 0-ra. Az 1-es padding mellett minden réteg 1×1-es konvolúcióvá degenerálódik, tehát a modell kimenete az input ugyanazon pozícióját adja vissza (bias nélkül). A kimenet középső neuronjának gradiens-térképe egyetlen input pixelen nemnulla, annak ellenére, hogy az architekturális RF 9×9. Az elméleti RF tehát a hozzáférhetőség felső határa, nem a ténylegesen gyakorolt hatás leírása.

Ez a konfiguráció szándékosan degenerált, és nem azt modellezi, ami egy betanított hálóban történik: véletlen inicializálás és tanítás után a súlyok általában nemnullák, az effektív mező pedig jellemzően közelebb van az elméletihez (a labor A modelljében a véletlen inicializált súlyokkal a középső 3×3 arány 0,298 volt). Az ellenpélda ezért csak azt bizonyítja, hogy az architekturális RF felső korlát és nem garancia; nem állítja, hogy a betanított hálók effektív mezője tipikusan 1×1-es.

16.3.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget az elméleti recepciós mező és az effektív recepciós mező között. Adjon egy konkrét súlykonfigurációt, amelyben a kettő jelentősen eltér. A beadandó eredmény: saját szavakkal írt definíció, mérési mód, és egy kis numerikus példa a fenti labor valamely modelljén.

F2 (levezetési, nehézség 2). Számítsa ki a recepciós mezőt és a jump-ot a következő 1D soros hálózatra: Conv($k = 5, d = 1, s = 1$), MaxPool($k = 2, s = 2$), Conv($k = 3, d = 2, s = 2$). A megoldásban minden réteg utáni F_l és J_l érték szerepeljen, a rekurzió indoklásával együtt. Elvárt végeredmény: $F = 14$, $J = 4$.

F3 (implementációs, nehézség 2). Írjon Python függvényt, amely specifikáció-listából kiszámítja az F és J értékeket 1D/négyzetes esetre, és legalább két asserttesztet ellenőrizze. A beadandó eredmény: futtatható kód, tesztesetek és azok kimenete.

F4 (hibakeresési, nehézség 2). Tekintse a következő hibás rekurziót:

```
def rf_wrong(spec):
    F = 1
    J = 1
    for k, d, s in spec:
        J = J * s
        F = F + (k - 1) * d * J   # a frissített J-t használja
    return F, J
```

Magyarázza el, miért hibás, milyen következménye van, és javítsa. Ellenőrizze a javítást egyetlen $k = 3, s = 2$ rétegen: a helyes $F = 3$, a hibás $F = 5$. A beadandó eredmény: hibaelemzés, javított kód, kontrollfutás.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a következő kérdésre: egy adott, már betanított CNN predikciója a kép teljes elméleti RF-jét használja-e, vagy csak egy jóval kisebb központi tartományra érzékeny? Adja meg a protokollt: rögzített modell és adat, split, gradiens- vagy perturbációalapú mérés, küszöb, metrika, ismétlések száma, és a következtetés érvényességi korlátja. A beadandó eredmény: protokollterv, nem szükséges futtatni. A beadandó

terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

16.3.8 Megoldások és indoklás

F1 megoldás. Az elméleti RF az architektúrából számított maximális bemeneti halmaz, amelytől egy kimeneti neuron függhet; az effektív RF a ténylegesen megvalósuló súlyozott érzékenység. Utóbbit a választott kimeneti neuronra vonatkozó loss bemeneti gradiensének abszolút értékével mérjük. Konkrét ellenpélda: az A modellben minden kernel középső elemét 1-re, a többi együtthatót 0-ra állítjuk. Ekkor az elméleti RF 9×9, de a középső kimeneti neuron gradiense csak az input középső pixelén nemnulla. A mért ellenőrzés (Python 3.12.3, torch 2.15.0+cpu, CPU, seed 7, 32×32-es csupa egyes bemenet): a gradiens nemnulla elemeinek száma 1, a befoglaló doboz 1×1, a középső 3×3 blokk aránya 1,000; ugyanez a labor A modelljében a véletlen inicializált (seed 7), nem degenerált súlyokkal 0,298. A két szám együtt mutatja, hogy az elméleti és az effektív mező mennyire szétválhat.

F2 megoldás. A rekurzióval:

1. réteg: $k = 5, d = 1, s = 1$: $F_1 = 1 + (5-1) \cdot 1 \cdot 1 = 5$, $J_1 = 1 \cdot 1 = 1$.
2. réteg: pooling $k = 2, d = 1, s = 2$: $F_2 = 5 + (2-1) \cdot 1 \cdot 1 = 6$, $J_2 = 1 \cdot 2 = 2$.
3. réteg: $k = 3, d = 2, s = 2$: $F_3 = 6 + (3-1) \cdot 2 \cdot 2 = 6 + 8 = 14$, $J_3 = 2 \cdot 2 = 4$.

Tehát a végső RF 14, a jump 4. Ez azt jelenti, hogy a kimenet minden negyedik eredeti bemeneti pixelen mintavételez, és egyetlen kimeneti neuron egy 14 hosszú intervallumtól függhet. A levezetés a tétel mindkét rekurziójának közvetlen alkalmazása.

F3 megoldás. A függvény és a tesztek:

```
def rf_from_spec(spec):
    """spec: list of (kernel, dilation, stride)."""
    F = 1
    J = 1
    for k, d, s in spec:
        F = F + (k - 1) * d * J
        J = J * s
    return F, J

# teszt 1: négy 3x3-as, stride 1: RF=9
assert rf_from_spec([(3, 1, 1)] * 4) == (9, 1)

# teszt 2: egy stride-2 réteg, majd egy stride-1
assert rf_from_spec([(3, 1, 2)]) == (3, 2)
assert rf_from_spec([(3, 1, 2), (3, 1, 1)]) == (7, 2)

# teszt 3 (2D): dimenziónként külön rekurzió
spec2d = [((3, 1, 1), (5, 1, 2)), ((3, 1, 1), (5, 1, 2))]
Fh, Jh = rf_from_spec([h for h, w in spec2d])
Fw, Jw = rf_from_spec([w for h, w in spec2d])
assert (Fh, Jh, Fw, Jw) == (5, 1, 13, 4)
print((Fh, Jh), (Fw, Jw))
# mért kimenet: (5, 1) (13, 4)
```

A második tesztben az első réteg után $J_1 = 2$, a második réteg hozzáadása $F = 3 + (3-1) \cdot 1 \cdot 2 = 7$, a jump változatlan: $J_2 = 2 \cdot 1 = 2$. A harmadik teszt 2D esete ugyanezt a rekurziót futtatja dimenziónként: a magasságra $k_h = 3$, $s_h = 1$ kétszer, ezért $F_h = 1 + 2 + 2 = 5$ és $J_h = 1$; a szélességre $k_w = 5$, $s_w = 2$ kétszer, ezért $F_w = 1 + 4 \cdot 1 + 4 \cdot 2 = 13$ és $J_w = 2 \cdot 2 = 4$, a befoglaló téglalap pedig 5×13. A visszatérési sorrend (F, J), összhangban a szövegbeli jelöléssel.

F4 megoldás. A hiba a sorrendben van: a réteg stride-ja a **következő** réteg mezőjét befolyásolja, nem a saját rétegét. A kód a J frissítése utáni értéket használja, ezért a saját kernel hozzájárulását már a megnövelt jump-tal szorozza. Egyetlen $k = 3, d = 1, s = 2$ rétegen a helyes $F = 1 + (3-1) \cdot 1 \cdot 1 = 3$, a hibás $F = 1 + (3-1) \cdot 1 \cdot 2 = 5$; a hibás kód tehát nem a kernel méretét, hanem a recepciós mező tényleges hosszát becsüli túl. Javítás: a J frissítése a mezőszámítás **után** következzen, ahogyan a tétel bizonyítása is teszi:

```
def rf_fixed(spec):
    F = 1
    J = 1
    for k, d, s in spec:
        F = F + (k - 1) * d * J
        J = J * s
    return F, J
```

Kontrollfutás (Python 3.12.3, CPU): egyetlen $k = 3, s = 2$ rétegen a hibás és a javított függvény eredménye $(5, 2)$, illetve $(3, 2)$; a háromrétegű B specifikáción `[(3, 1, 2)] * 3` a hibás $(29, 8)$, a javított $(15, 8)$, ami egyezik a labor B architektúrájának elméleti értékével. A javított kód megegyezik az F3 megoldás függvényével.

F5 megoldás. Protokollterv. Modell és adat: a **GlyphLab** a könyv saját, helyben generált szintetikus adatkészlete (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték): 180 jelenet, jelenetenként két zajos nézet, összesen 360 kép, 24×24, float32, három vonalorientáció-osztály. A szerződés szerinti jelenetszintű split 120/30/30 jelenet, azaz 240/60/60 kép train/val/test; a teszt 30 jelenetének 60 képén mérünk. Rögzített modell: a **checkpoint** (a mentett modellparaméter-állapot) a validációs loss alapján kiválasztva, a teszten nem választunk. Mérés: minden tesztképre számítsuk ki a középső kimeneti logit gradiensét az inputra, majd az abszolút gradiensértékek összegéből (ez a G_{ij} definíciójával azonos, nem négyzetes „energia") keressük meg azt a legkisebb középpontú négyzetet, amely a teljes összeg legalább 95 százalékát tartalmazza. A metrika ennek a négyzetnek a mérete az elméleti RF méretéhez viszonyítva. Kontrollként **perturbációs módszert** is futtatunk: az input egyes régióit kitakarjuk vagy konstansra cseréljük, és a kimeneti logit változását mérjük; ez a gradiensalapú, lokális érzékenységgel szemben nagyobb bemeneti változásokra is válaszol, de nemlineáris, ezért a két mérőszám nem váltható ki egymással. Ismételjük legalább 30 független tesztképen; az átlagos négyzetméret és annak szórása a jelentendő mennyiség. Baseline: az elméleti RF. Korlát: ReLU-k esetén a gradiens inputfüggő, ezért több inputon átlagolunk, és az eredmény csak a rögzített modellre, erre a feladatra és erre a mérési definícióra érvényes.

16.3.9 Önellenőrzés és visszahívás

Önellenőrző kérdések: (1) Miért nem befolyásolja a padding a recepciós mező hosszát? (2) Egy $s > 1$ réteg után miért növekszik a rákövetkező rétegek RF-je gyorsabban? (3) A gradiens befoglaló dobozának mérete mindig egyenlő az elméleti RF-fel?

Válaszok: (1) a padding virtuális nulla, nem eredeti bemenet; (2) mert a jump nő, így a következő kernel ugyanannyi mintája nagyobb bemeneti tartományt fog át; (3) nem, csak az elérhető tartomány felső határa, és a gradiens akár nulla is lehet a mező egy részén (például ha a kernel súlyai nullák), ahogyan a fenti ellenpélda mutatta.

Korábbi fogalom visszahívása: a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben levezetett VJP szerint a gradiens az inputra nézve $Df^{\top} e$, ahol Df a választott leképezés Jacobi-mátrixa (ezt a jelölést a jump J_l szimbólumától való megkülönböztetés végett használjuk), e pedig a választott kimeneti neuron egységvektora. A jelen labor a Jacobi-mátrix kimeneti neuronhoz tartozó sorát vizsgálja térbeli térképen. A 16.2. Pooling, downsampling és aliasing alfejezetből tudjuk, hogy a

pooling ablak és stride ugyanezen rekurzióval kezelhető; a maxpool gradiens-térképe azonban csak az aktív útvonalat mutatja.

16.3.10 Rövid összefoglaló

A recepciós mező azt a bemeneti tartományt jelöli, amelytől egy kimeneti neuron elvileg függhet. Soros rétegek esetén a jump $J_l = J_{l-1} s_l$, a mező pedig $F_l = F_{l-1} + (k_l - 1) d_l J_{l-1}$; a zárt képletben a stride-ok szorzata gyorsítja a növekedést. A padding és a bias a hosszt nem változtatja. Az effektív recepciós mező ezzel szemben a gradienssel mért tényleges érzékenység, és messze lehet az elméleti határtól. A labor három architektúrán ellenőrizte, hogy a gradiens befoglaló mérete rendre 9×9, 15×15 és 15×15, miközben a középső 3×3 blokk energiaaránya A: 0,298, B: 0,074, C: 0,042 volt (C-ben közel az egyenletes eloszláshoz).

16.3.11 Következő alkalmazás

A 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése IV. ellenőrző pont a CNN-operátor teljes megértését méri: ott a recepciós mező mellett az eltolási equivariance/invariance határfeltételeit, a stride-részcsoportokat és a gradiensellenőrzést is együttesen kell alkalmazni. Az itt levezetett rekurzió a csoportosított (grouped) architektúrákban — ahol a csatornákat csoportokra osztjuk, és minden csoport a saját csatornáin külön konvolválódik — is alapeszköz, valamint a transzponált konvolúció reverse-map (visszafelé leképező) elemzésében, ahol a forward konvolúció kimeneti pozícióiból következtetünk vissza arra, hogy mely bemeneti pozíciók milyen súllyal járulnak hozzá.

16.3.12 Felhasznált források

- [P31] Luo, W., Li, Y., Urtasun, R. és Zemel, R. (2016). *Understanding the Effective Receptive Field in Deep Convolutional Neural Networks.* Advances in Neural Information Processing Systems 29 (NIPS 2016). arXiv:1701.04128. https://arxiv.org/abs/1701.04128.

16.4 IV. ellenőrző pont: a CNN-operátor teljes megértése

16.4.1 Cél, előismeret és induló kérdés

A IV. ellenőrző pont egyetlen szabályos specifikációs lapba foglalja a CNN-operátort: a kereszt-korrelációt, a kimeneti alakot, a paramétereket, az adjungált (visszaterjesztett) műveletet, az eltolási szimmetriát és a határfeltételt. A hallgató a 13.1.–14.1. egységekből ismeri a diszkrét jelet és a lokális szűrőt, a 15.1.–15.2. alfejezetből a backwardot, a 16.1. Eltolási equivariance és invariance alfejezetből az equivariance fogalmát, a 16.3. Elméleti és effektív recepciós mező alfejezetből pedig a recepciós mező rekurzióját. Itt a feladat nem új tétel felépítése, hanem annak megmutatása, hogy ugyanaz a képlet egyszerre állja ki az alak-, gradiens- és szimmetriavizsgálatot. A 4.4. I. ellenőrző pont: matematika és kód megfelelése és 12.4. III. ellenőrző pont: önállóan tanuló kis háló módszertani lecképét idézzük vissza: a numerikus egyezés nem helyettesíti a levezetést, és a checkpoint akkor teljes, ha a kézi számolás, a független f64 referencia és az autograd háromszögben minden hiba a megadott tűrés alatt van. Induló kérdés: egy 4×4-es bemenetre és 2×2-es kernelre hány kimeneti elem adódik, és miért nem nevezhető a `Conv2d` hívás matematikai konvolúciónak? A válasz: $s = 1$, $p = 0$ mellett a kimenet $(4 - 2 + 1) \times (4 - 2 + 1) = 3 \times 3$,

azaz 9 elem; a `Conv2d` pedig azért nem matematikai konvolúció, mert kernelmegfordítás nélküli keresztkorrelációt számol (ezt a specifikáció és a tipikus tévedés ellenpéldája rögzíti).

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat (a D1–D5 dimenziókhoz rendelt célzott gyakorlatokat) az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg. A rubrika öt dimenziója: D1 matematikai helyesség, D2 kód és futtathatóság, D3 adat- és mérési protokoll, D4 bizonyíték és nyomon követhetőség, D5 az állítás és korlátai; a PASS-szabály mind az öt dimenzióban legalább 3-as szintet és a labor összes `assert`-jének átmenetét írja elő. Itt csak az eltérés: a PASS (a) pontja ebben az egységben a független `float64` referencia és a könyvtári művelet egyezése az `assert`-ekben; a súlypont a D1 (a kézi és a képletbeli számolás helyessége) és a D2 (a referencia és az autograd futása).

16.4.2 A keresztkorrelációs operátor specifikációja

A bemeneti tenzor $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$, a súly $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$, a bias $b \in \mathbb{R}^{C_{out}}$, a stride s, a szimmetrikus zero padding $p \geq 0$ (nemnegatív egész, a $p = 0$ legitim eset) és a dilatáció d pozitív egész. A p szélességű keret levágása a d bármely értéke mellett szükséges; a padding (p, a keret szélessége) és a dilatáció (d, a kernel mintavételi lépésköze) két külön mennyiség, ezért a képletekben nem cserélhetők fel. A programozói főút a **térbeli keresztkorrelációt** számolja, kernelmegfordítás nélkül [D10]. Legyen X_p a p szélességű nullákkal kiegészített bemenet; az output $Y \in \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}}$ eleme:

$$Y[n,o,i,j] \;=\; b[o] \;+\; \sum_{c=0}^{C_{in}-1}\sum_{u=0}^{k_h-1}\sum_{v=0}^{k_w-1} X_p[n,c,\; i\,s+u\,d,\; j\,s+v\,d]\cdot K[o,c,u,v],$$

ahol $n \in [0,N)$, $o \in [0,C_{out})$, $i \in [0,H_{out})$, $j \in [0,W_{out})$. A kimeneti méret a kernel $d(k-1)+1$ effektív kiterjedésével:

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h-1) - 1}{s} \right\rfloor + 1, \qquad W_{out} = \left\lfloor \frac{W + 2p - d(k_w-1) - 1}{s} \right\rfloor + 1.$$

A paraméterszám $C_{out}\cdot(C_{in}\cdot k_h\cdot k_w + 1)$: ennyi tanulható skalár tartozik a réteghez, függetlenül a kép méretétől — ez a súlymegosztás lényege. A `groups=1` esetre szorítkozunk; a csoportosítás a 14.4. Csoportos és depthwise konvolúció egységben külön szerződést kapott.

Az adjungált a visszaterjesztés VJP-oldala. A $L = \frac{1}{2}\sum_{n,o,i,j} Y[n,o,i,j]^2$ céllal az upstream gradiens $g = Y$, és a láncszabályból:

$$\frac{\partial L}{\partial K}[o,c,u,v] = \sum_{n,i,j} Y[n,o,i,j]\cdot X_p[n,c,\; is+ud,\; js+vd],$$

$$\frac{\partial L}{\partial X_p}[n,c,r,t] = \sum_{o,u,v:\; is+ud=r,\; js+vd=t} Y[n,o,i,j]\cdot K[o,c,u,v],$$

majd a p szélességű keret levágásával kapjuk meg $\partial L/\partial X$-et (a levágás a padding eltávolítása, nem dilatációs lépés). A biasgradiens $\partial L/\partial b[o] = \sum_{n,i,j} Y[n,o,i,j]$. A scatter–add szerkezet mutatja, hogy a backward a megfelelő lineáris operátor **transzponáltja/adjungáltja**: valós vektorterekben a belső szorzat $\langle u,v\rangle = u^\top v$, ezért az adjungált mátrixa éppen a transzponált, $\langle Au, v\rangle = (Au)^\top v = u^\top A^\top v = \langle u, A^\top v\rangle$; a transzponált convolution ezért kapcsolódik a backwardhöz, de nem általános inverz.

16.4.3 Eltolási equivariance: bizonyított tétel és határ-ellenpélda

Tétel (stride-részcsoport szerinti equivariance, határmentes tartományon). Legyen C_K a bias nélküli keresztkorrelációs operátor s stride-dal, p paddinggel, $d = 1$ dilatációval, a $C_{in} = C_{out} = 1$, $N = 1$ esetben, és legyen az értelmezési tartomány a teljes $\mathbb{Z}^2$ rács (véges tartójú bemenettel, nullakiterjesztéssel). Ekkor tetszőleges $t = (t_1, t_2) \in \mathbb{Z}^2$ eltolásra:

$$C_K(S_{s\,t}X) \;=\; S_t(C_K X),$$

ahol $(S_\tau X)[a, b] = X[a-\tau_1,\, b-\tau_2]$ a koordinátaeltolás. A stride-részcsoport itt az $s\mathbb{Z}^2 = \{st : t \in \mathbb{Z}^2\}$ részcsoport, vagyis azok az inputeltolások, amelyeknek mindkét koordinátája s többszöröse. Tehát az input $s\,t$ pixelnyi eltolása az outputon t rácspontnyi eltolást okoz.

Bizonyítás. Pontonként alkalmazzuk a definíciót. Az output (i, j) pozícióban:

$$C_K(S_{s\,t}X)[i, j] = \sum_{u,v} (S_{s\,t}X)[is + u,\; js + v] \cdot K[u, v] = \sum_{u,v} X[is + u - st_1,\; js + v - st_2] \cdot K[u, v].$$

Az indexek átrendezésével $(i - t_1)s + u$ és $(j - t_2)s + v$ adódik, ami éppen a deriválatlan output $(i-t_1, j-t_2)$ pozíciójának képlete, azaz $S_t(C_K X)[i, j]$. Az egyenlőség minden pozícióban fennáll, mert a $\mathbb{Z}^2$ rácson nincs kilépés a tartományból. □

Következmény (véges tartomány feltétele). Véges $[0, H) \times [0, W)$ képen zero paddinggel az equivariance a belső tartományon igaz, ha az eltolt bemenet minden, a forward által elért pozíciója az eredeti tartományba esik; más szóval a kernel támogatása nem lép ki a kép határán. Ez elégséges, de nem szükséges feltétel: a határmentes $\mathbb{Z}^2$ esetben a feltétel automatikusan teljesül, a véges példában pedig éppen a feltétel sérülése okozza az eltérést. Ha a szélső oszlopok nullákkal pótlódnak, a szélső outputokban az információvesztés aszimmetrikus lesz, és az egyenlőség megbomlik.

Ellenpélda (határon sérül az equivariance). Legyen $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ és $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $s = 1$, $p = 0$. A forward $Y = C_K X = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$. Az inputot jobbra tolva: $S_{(0,1)}X = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 4 & 5 \\ 0 & 7 & 8 \end{bmatrix}$, így $C_K(S_{(0,1)}X) = \begin{bmatrix} 4 & 6 \\ 7 & 12 \end{bmatrix}$. Az output eltolása viszont $S_{(0,1)}Y = \begin{bmatrix} 0 & 6 \\ 0 & 12 \end{bmatrix}$. A két mátrix első oszlopa különbözik ($4 \neq 0$ és $7 \neq 0$), tehát az equivariance a véges, pad nélküli képen nem teljesül. A hiba a bal szélen keletkezik, ahol az eltolt input nullái az eredeti $1, 4, 7$ értékeket helyettesítik, miközben az output-eltolás a $6, 12$ értékeket egyszerűen kihúzza a nulltartományba.

16.4.4 Kézi példa: teljes forward és spot-check gradiens

Vegyük a 4×4-es bemenetet és a $K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ kernelt $s = 1$, $p = 0$ mellett. A 2×2-es ablakok szisztematikus szorzatösszege:

$$Y = \begin{bmatrix} 44 & 54 & 64 \\ 84 & 94 & 104 \\ 124 & 134 & 144 \end{bmatrix}.$$

Például $Y[1,1] = 6\cdot 1+7\cdot 2+10\cdot 3+11\cdot 4 = 94$. Az $L = \frac{1}{2}\sum Y^2$ célra $g = Y$, így a súlygradiensek a fenti scatter–add képletekből kézzel is ellenőrizhetők:

$$\frac{\partial L}{\partial K} = \begin{bmatrix} 6096 & 6942 \\ 9480 & 10326 \end{bmatrix}.$$

Egyetlen elem teljes kifejtésben: $\partial L/\partial K[0,0] = \sum_{i,j} Y[i,j]\,X[i,j] = 44\cdot 1+54\cdot 2+\cdots+134\cdot 10+144\cdot 11 = 6096$. Az inputgradiens egy belső pontja $\partial L/\partial X[1,1] = Y[0,0]K[1,1]+Y[0,1]K[1,0]+Y[1,0]K[0,1]+Y[1,1]K[0,0] = 44\cdot 4+54\cdot 3+84\cdot 2+94\cdot 1 = 600$, a bal felső saroké pedig $\partial L/\partial X[0,0] = Y[0,0]K[0,0] = 44$. A teljes mátrixot a labor kódja numerikusan ellenőrzi; a kézi spot-check a fenti két elemet adja, a teljes $\partial L/\partial X$ mátrixot a PyTorch-autograddal való összevetés fedi le. A biasgradiens $\partial L/\partial b = \sum_{i,j} Y[i,j] = 846$.

16.4.5 Labor: független f64 referencia, autograd és határkísérlet

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n064_checkpoint.py` futtatható (`python lab/n064_checkpoint.py`); külső adatfájl nem kell.

A labor három független utat futtat: (i) saját NumPy forward/backward `float64`-ben, (ii) PyTorch `conv2d` autograd `float64`-ben, (iii) határkezelési hiba térképe. A kézi értékek az 1–2. pontból származó, a futtatásban `np.testing.assert_allclose`-szal ellenőrzött értékek: a forward és az autograd összevetése 10^{-12}, a kézi gradiensmátrixok ($\partial L/\partial K$, $\partial L/\partial X$) összevetése 10^{-10} abszolút tűréssel.

```
# lab/n064_checkpoint.py -- futtatás: python lab/n064_checkpoint.py
import numpy as np
import torch

def cc_forward(X, K, b, stride=1, pad=0, dilation=1):
    """Referencia: cross-correlation, NCHW, groups=1, zero pad, float64."""
    N, C, H, W = X.shape
    O, Ck, kh, kw = K.shape
    assert C == Ck
    eff_h = dilation * (kh - 1) + 1
    eff_w = dilation * (kw - 1) + 1
    H_out = (H + 2 * pad - eff_h) // stride + 1
    W_out = (W + 2 * pad - eff_w) // stride + 1
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode="constant")
    Y = np.empty((N, O, H_out, W_out), dtype=np.float64)
    for n in range(N):
        for o in range(O):
            for i in range(H_out):
                for j in range(W_out):
                    total = b[o]
                    for c in range(C):
                        for u in range(kh):
                            for v in range(kw):
                                total += Xp[n, c, i * stride + u * dilation,
                                            j * stride + v * dilation] * K[o, c, u, v]
                    Y[n, o, i, j] = total
    return Y, Xp

def cc_backward_L2(Y, Xp, K, stride=1, dilation=1):
    """Backward a L = 0.5 * sum(Y**2) célra; upstream gradiens g = Y.
```

```
    A visszatérési dXp a *paddinges* bemenetre vonatkozó gradiens; a p keret
    levágása a hívó feladata. p=0 esetén dXp közvetlenül dX.
    """
    N, O, H_out, W_out = Y.shape
    Nc, C, Hp, Wp = Xp.shape          # Xp mérete megadja a paddinget is
    kh, kw = K.shape[2], K.shape[3]
    dK = np.zeros_like(K, dtype=np.float64)
    dXp = np.zeros_like(Xp, dtype=np.float64)
    db = np.zeros(O, dtype=np.float64)
    for n in range(N):
        for o in range(O):
            for i in range(H_out):
                for j in range(W_out):
                    g = Y[n, o, i, j]
                    db[o] += g
                    for c in range(C):
                        for u in range(kh):
                            for v in range(kw):
                                a = i * stride + u * dilation
                                b_idx = j * stride + v * dilation
                                dK[o, c, u, v] += g * Xp[n, c, a, b_idx]
                                dXp[n, c, a, b_idx] += g * K[o, c, u, v]
    return dK, db, dXp

# Kézi példa
X = np.arange(1, 17, dtype=np.float64).reshape(1, 1, 4, 4)
K = np.array([[[[1.0, 2.0], [3.0, 4.0]]]])
b = np.zeros((1,), dtype=np.float64)
Y_manual = np.array([[[[44.0, 54.0, 64.0],
                       [84.0, 94.0, 104.0],
                       [124.0, 134.0, 144.0]]]], dtype=np.float64)

Y_ref, Xp = cc_forward(X, K, b, stride=1, pad=0)
np.testing.assert_allclose(Y_ref, Y_manual, atol=1e-12)
dK, db, dXp = cc_backward_L2(Y_ref, Xp, K)
dK_manual = np.array([[[[6096.0, 6942.0], [9480.0, 10326.0]]]])
np.testing.assert_allclose(dK, dK_manual, atol=1e-10)
np.testing.assert_allclose(dXp[:, :, 1, 1], 600.0, atol=1e-10)
np.testing.assert_allclose(dXp[:, :, 0, 0], 44.0, atol=1e-10)

# PyTorch autograd-kontroll
Xt = torch.tensor(X, dtype=torch.float64, device="cpu", requires_grad=True)
Kt = torch.tensor(K, dtype=torch.float64, device="cpu", requires_grad=True)
bt = torch.tensor(b, dtype=torch.float64, device="cpu", requires_grad=True)
Yt = torch.nn.functional.conv2d(Xt, Kt, bt, stride=1, padding=0)
Lt = 0.5 * torch.sum(Yt ** 2)
Lt.backward()
np.testing.assert_allclose(Yt.detach().numpy(), Y_manual, atol=1e-12)
np.testing.assert_allclose(Kt.grad.numpy(), dK_manual, atol=1e-12)
np.testing.assert_allclose(dXp, Xt.grad.numpy(), atol=1e-12)  # paddinges input, itt nincs keret
np.testing.assert_allclose(bt.grad.numpy(), np.array([Y_ref.sum()]), atol=1e-12)
print("forward/backward: OK (f64, tűrés 1e-12)")

# Határkezelési kísérlet: 2x2-es identitás kernel, stride=1, pad=0
def shift_zero_2d(arr, dh, dw):
    out = np.zeros_like(arr)
    h, w = arr.shape[2], arr.shape[3]
    src_h0, src_h1 = max(0, -dh), min(h, h - dh)
    dst_h0, dst_h1 = max(0, dh), min(h, h + dh)
    src_w0, src_w1 = max(0, -dw), min(w, w - dw)
    dst_w0, dst_w1 = max(0, dw), min(w, w + dw)
    out[:, :, dst_h0:dst_h1, dst_w0:dst_w1] = arr[:, :, src_h0:src_h1, src_w0:src_w1]
    return out
```

```
Xb = np.arange(1, 10, dtype=np.float64).reshape(1, 1, 3, 3)
Kb = np.array([[[[1.0, 0.0], [0.0, 1.0]]]])
Yb, _ = cc_forward(Xb, Kb, np.zeros(1), stride=1, pad=0)
print("shift_err (Frobenius-norma, 3x3/2x2):")
for dh in range(-1, 2):
    row = []
    for dw in range(-1, 2):
        Xs = shift_zero_2d(Xb, dh, dw)
        Ys, _ = cc_forward(Xs, Kb, np.zeros(1), stride=1, pad=0)
        Ysh = shift_zero_2d(Yb, dh, dw)
        row.append(f"{np.linalg.norm(Ys - Ysh):.2f}")
    print(("dh=%+d: " % dh) + " ".join(row))

# 5) stride-2 határkísérlet: 6x6 bemenet, 3x3 kernel, p=0
X6 = np.arange(1, 37, dtype=np.float64).reshape(1, 1, 6, 6)
K6 = np.array([[[[1.0, 0.0, 2.0],
                 [0.0, 1.0, 0.0],
                 [2.0, 0.0, 1.0]]]])
Y6, _ = cc_forward(X6, K6, np.zeros(1), stride=2, pad=0)
X6_even = shift_zero_2d(X6, 2, 0)
Y6_even, _ = cc_forward(X6_even, K6, np.zeros(1), stride=2, pad=0)
Y6_even_ref = shift_zero_2d(Y6, 1, 0)
diff_even = np.abs(Y6_even - Y6_even_ref)
print("stride 2, páros eltolás (2,0): belső max |diff| =", diff_even[0, 0, 1:, :].max())
assert diff_even[0, 0, 1:, :].max() < 1e-12
assert diff_even.max() > 0.0

X6_odd = shift_zero_2d(X6, 1, 0)
Y6_odd, _ = cc_forward(X6_odd, K6, np.zeros(1), stride=2, pad=0)
odd_diffs = []
for mh in (-1, 0, 1):
    odd_diffs.append(float(np.abs(Y6_odd - shift_zero_2d(Y6, mh, 0)).max()))
print("stride 2, páratlan eltolás (1,0): |diff| m=-1,0,1:", odd_diffs)
assert all(d > 0.0 for d in odd_diffs)
```

A labor mért kimenete (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.get_num_threads() = 10`; a szkript determinisztikus, seed nélkül):

```
forward/backward: OK (f64, tűrés 1e-12)
shift_err (Frobenius-norma, 3x3/2x2):
dh=-1: 13.45 10.63 9.90
dh=+0: 6.71 0.00 8.06
dh=+1: 4.24 3.61 4.58
stride 2, páros eltolás (2,0): belső max |diff| = 0.0
stride 2, páratlan eltolás (1,0): |diff| m=-1,0,1: [121.0, 42.0, 42.0]
```

A határkísérletben a Frobenius-hiba csak $(d_h, d_w) = (0,0)$-ban 0; a többi nyolc eltolás hibája például $(0,+1)$-re $\sqrt{65} \approx 8{,}06$, $(+1,0)$-ra $\sqrt{13} \approx 3{,}61$, $(-1,-1)$-re $\sqrt{181} \approx 13{,}45$, egyezve a mért táblázattal. Ez a minta azt mutatja, hogy pad nélküli, kisméretű képen **minden** nem-nulla eltolás megsérti az equivarianciát a határokon. Nagyobb pad esetén a belső eltolások tartománya szélesebb lenne, de a legszélső outputok akkor is határérzékenyek; ez utóbbi nem futtatott mérés, hanem a képletből következő elvárás. A stride $s > 1$ esetén a részcsoport-feltétel külön is megjelenik: a 6x6-os, $s = 2$ kísérletben a $(2,0)$ páros inputeltolás a belső kimeneti soron 0 hibával kommutál, a páratlan $(1,0)$ eltolásra viszont egyetlen egész kimeneti eltolás sem ad egyezést (a három vizsgált $m_h \in \{-1,0,1\}$ eltolás hibája 121, 42, illetve 42). Az állítás tehát: csak az s-szeres inputeltolások kommutálnak, a többiek még a határmentes tartományon sem.

16.4.6 Feladatok (F1–F5)

Az „Elvárt eredmény” minden feladatnál a beadás kötelező tartalmi elemeit sorolja fel; a pontozás a fenti D1–D5 dimenziók szerint történik.

F1 (fogalmi, nehézség 1). Definiálja a keresztkorreláció és a matematikai konvolúció különbségét 1D-ben. Adjon olyan $x \in \mathbb{R}^3$ jelet és $K \in \mathbb{R}^2$ kernelt, amelyre a keresztkorreláció $[1, 6]$ kimenetet ad, a valid matematikai konvolúció pedig $[2, 3]$ kimenetet. Elvárt eredmény: definíciók, majd az ellenőrző számolás.

F2 (levezetési, nehézség 2). Vezesse le a biasgradiens és a súlygradiens képletét 1D-ben: $x \in \mathbb{R}^7$, $K \in \mathbb{R}^3$, $s = 2$, $p = 1$, $d = 1$, $L = \frac{1}{2}\sum Y^2$. Adja meg H_{out}-ot, majd $\partial L/\partial K[1]$ teljes összegző képletét az outputindexekkel. Elvárt eredmény: minden indexhatár, a padding indexeltolás, valamint egy $Y[1]$-re vonatkozó láncszabályos felbontás láthatóan.

F3 (implementációs, nehézség 2). A labor `cc_forward` és `cc_backward_L2` függvénye már kezeli a dilatációt (az indexelés $i \cdot s + u \cdot d$); a feladat ennek igazolása unit teszttel. Írjon tesztet egy 5×5-ös véletlen bemenetre `numpy.random.RandomState(42)` generátorral, 3×3-as kernelre, $s = 1$, $p = 1$, $d = 2$ mellett, a labor `cc_backward_L2`-vel és PyTorch autograddal $L = \frac{1}{2}\sum Y^2$-re. Elvárt eredmény: teljes kód, `float64`, `max(|a-b|) < 5e-12` a kimenetre és a gradiensmátrixokra, valamint a mért maximális eltérés.

F4 (hibakeresési, nehézség 2). Az alábbi backward-vázlat a $d = 1$ esetben hibásan a kernel megfordítottját használja a `dXp` scatter–add ágában. Lokalizálja a hibát, mutassa meg következményét a kézi példa 3×3-as kimenetén és 2×2-es kernelén számszerű gradienssel, és adja meg a javított sort. Elvárt eredmény: hiba oka, hibás gradienselem, javítás, autograd-ellenőrzés.

```
# szándékosan hibás vázlat: a dXp scatter-add ág
K_rev = K[:, :, ::-1, ::-1]   # matematikai index: K_rev[o,c,u,v] = K[o,c,kh-1-u,kw-1-v]
for i in range(H_out):
    for j in range(W_out):
        for u in range(kh):
            for v in range(kw):
                dXp[:, :, i*s+u*d, j*s+v*d] += (
                    dY[:, :, i, j] * K_rev[:, :, u, v])
```

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a GlyphLab `data/metadata.csv`-re (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték), amely azt méri, hogy egy stride-2-t használó, a train+val részen betanított, majd befagyasztott CNN-osztályozó test-pontossága hogyan változik a képek d_h-nyi, illetve $s\,t$-nyi (stride-többszörösi) eltolására, zero padding nélkül. Fogalmazza meg a hipotézist, a metrikát, a megfigyelési egységet, a baseline-t, a seed-kezelést és az érvényességi korlátot. Elvárt eredmény: protokoll-szöveg, nem szükséges futtatott eredmény. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

16.4.7 Megoldások és indoklás

F1 megoldás. A keresztkorreláció $Y[i] = \sum_{k=0}^{m-1} X[i+k]K[k]$, ahol m a kernel hossza, tehát a kernel csúszik a jelen, azaz nem megfordítva. A konvolúció $C[i] = \sum_{k=0}^{m-1} X[i-k]K[k]$, ami a kernel időbeli megfordítását jelenti. Példa: $X = [1, 0, 3]$, $K = [1, 2]$, pad=0, stride=1. Keresztkorreláció: $Y[0] = 1 \cdot 1 + 0 \cdot 2 = 1$, $Y[1] = 0 \cdot 1 + 3 \cdot 2 = 6$, tehát $[1, 6]$. Valid konvolúció (a kernel megfordításával, $C[i] = \sum_k X[i-k]K[k]$): $C[1] = X[1]K[0] + X[0]K[1] = 0 + 2 = 2$, $C[2] = X[2]K[0] + X[1]K[1] = 3 + 0 = 3$, tehát $[2, 3]$. A $[0]$ indexű, nullával kiegészített

teljes konvolúció külön számítás, azt a valid kimenet nem tartalmazza. A két művelet a kernel megfordításában különbözik.

F2 megoldás. A kimenet hossza $H_{out} = \lfloor (7+2-2-1)/2 \rfloor + 1 = 3+1 = 4$. A $p = 1$ paddinggel a kiegészített bemenet indexei 0..8: $X_p[0] = 0$, $X_p[1..7] = X[0..6]$, $X_p[8] = 0$, azaz $X_p[k+1] = X[k]$ minden $0 \le k \le 6$-ra; a $+1$ eltolás a paddingból jön. A forward: $Y[i] = \sum_{u=0}^{2} X_p[2i+u]K[u]$. A súlygradiens $\partial L/\partial K[u] = \sum_{i=0}^{3} Y[i]\, X_p[2i+u]$; konkrétan $u = 1$-re $\sum_{i=0}^{3} Y[i]X_p[2i+1]$. A lánc minden $Y[i]$-re $Y[i] = \sum_u X_p[2i+u]K[u]$, és $\partial L/\partial K[u] = \sum_i Y[i] \cdot \partial Y[i]/\partial K[u] = \sum_i Y[i]X_p[2i+u]$. Számpélda: $X = [1,2,3,4,5,6,7]$, $K = [1,0,2]$, ekkor $Y = [4,10,16,6]$, és $\partial L/\partial K[1] = 4 \cdot 1 + 10 \cdot 3 + 16 \cdot 5 + 6 \cdot 7 = 156$.

F3 megoldás. A teszt a labor függvényeit használja, ezért a `cc_forward` és a `cc_backward_L2` módosítása nem szükséges; csak a hívásokat kell dilatációval paraméterezni:

```
import numpy as np
import torch
import torch.nn.functional as F

rs = np.random.RandomState(42)
X = rs.randn(1, 1, 5, 5).astype(np.float64)
K = rs.randn(1, 1, 3, 3).astype(np.float64)
b = rs.randn(1).astype(np.float64)

Y, Xp = cc_forward(X, K, b, stride=1, pad=1, dilation=2)
dK, db, dXp = cc_backward_L2(Y, Xp, K, stride=1, dilation=2)
dX = dXp[:, :, 1:-1, 1:-1]          # a p=1 keret levágása

Xt = torch.tensor(X, dtype=torch.float64, requires_grad=True)
Kt = torch.tensor(K, dtype=torch.float64, requires_grad=True)
bt = torch.tensor(b, dtype=torch.float64, requires_grad=True)
Yt = F.conv2d(Xt, Kt, bt, stride=1, padding=1, dilation=2)
Lt = 0.5 * torch.sum(Yt ** 2)
Lt.backward()

errs = {
    "Y": np.abs(Y - Yt.detach().numpy()).max(),
    "dX": np.abs(dX - Xt.grad.numpy()).max(),
    "dK": np.abs(dK - Kt.grad.numpy()).max(),
    "db": np.abs(db - bt.grad.numpy()).max(),
}
assert all(v < 5e-12 for v in errs.values())
print(errs)
```

A mért futás (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU): a kimenet alakja $(1,1,3,3)$, a paddinges bemeneté $(1,1,7,7)$, mert $d(k-1)+1 = 5$ és $(5+2-5)//1+1 = 3$; a maximális abszolút eltérés Y-ra $2{,}22 \cdot 10^{-16}$, `dX`-re $4{,}44 \cdot 10^{-16}$, `dK`-ra $2{,}22 \cdot 10^{-16}$, `db`-re 0, mind a $5 \cdot 10^{-12}$ tűrés alatt. A teszt tehát igazolja, hogy a `cc_forward` és a `cc_backward_L2` dilatációval is konzisztens.

F4 megoldás. A backward scatter–add a forward lineáris operátor adjungáltja. A keresztkorreláció forward operátora $Y[i,j] = \sum_{u,v} X[i+u, j+v]K[u,v]$, ezért az adjungált $dX[p,q] = \sum_{i,j,u,v} Y[i,j]K[u,v]\,[p = i+u,\ q = j+v]$, amelyben a kernel **nem** fordul meg. A `K_rev` használata a matematikai konvolúció adjungáltját keveri a keresztkorreláció adjungáltjába.

A kézi példa 3×3-as kimenetén és $K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ kernelen, $L = \frac{1}{2}\sum Y^2$ mellett $dY = Y$. A bal felső sarokban a helyes $dX[0,0] = Y[0,0]K[0,0] = 44 \cdot 1 = 44$, a hibás `K_rev`-vel $44 \cdot 4 = 176$. A belső $dX[1,1] = Y[0,0]K[1,1] + Y[0,1]K[1,0] + Y[1,0]K[0,1] + Y[1,1]K[0,0] = 44 \cdot 4 + 54 \cdot 3 +$

$84 \cdot 2 + 94 \cdot 1 = 600$, a hibás kóddal viszont $44 \cdot 1 + 54 \cdot 2 + 84 \cdot 3 + 94 \cdot 4 = 780$. A különbség nem elhanyagolható: a megfordított kernel a szomszédos pixelsúlyokat cseréli fel, ezért a scatter–add nem az adjungáltat számolja. Javítás: a `dXp` scatter–add sorában `K_rev` helyett `K` szerepel:

```
dXp[:, :, i*s+u*d, j*s+v*d] += dY[:, :, i, j] * K[:, :, u, v]
```

A `K_rev[o,c,u,v]=K[o,c,k_h-1-u,k_w-1-v]` kifejezés önmagában helyes indexfordítás (2×2-re `K[:,:,::-1,::-1]`, a matematikai indexelést a Python szeletelés valósítja meg), de ezen az ágon nem használható. Az autograd-ellenőrzés a javított ágra 10^{-12} tűréssel egyezik.

F5 megoldás. Hipotézis: a stride-2 hálózat pontossága a stride-többszörösi eltolásokra a valid tartományban közel állandó, a nem-többszörösi 1-pixeles eltolásokra azonban a részcsoportfeltétel miatt nincs garantált ekvivariancia. Metrika: test-accuracy osztályonkénti bontásban, valamint az eltolatlan esethez viszonyított pontosságcsökkenés. Megfigyelési egység: `scene_id`, a `metadata.csv` jelenetazonosítója, nem kép, mert egy jelenet két nézete nem független. Split: a `metadata.csv` `split` oszlopa szerinti train/val/test; a modellt a train+val részen tanítjuk három seeddel, majd a súlyokat **befagyasztjuk** (a kiértékelés során már nem frissülnek), és a test splitet csak a végén értékeljük. A **korrupt teszt** a szerződés szerinti `data/corrupted_test.npy`: a 60 testkép zajjal és fényességgradienssel, a test teljes sorrendjében; a kísérlet a clean és a korrupt teszt ugyanazon scene/view sorrendjét használja, így az eltolásos és a korrupt összehasonlítás is páros marad. Baseline: az eltolatlan test-accuracy ugyanazon a befagyasztott checkpointon. A „közel állandó” előre rögzített kritériuma: a stride-többszörösi eltolásokra mért átlagos pontosságcsökkenés 95%-os konfidenciaintervallumának (CI) felső határa legfeljebb 2 százalékpont, és a CI tartalmazza a nullát; a nem-többszörösi eltolásoknál a csökkenés ettől szignifikánsan nagyobb, azaz a 95%-os CI alsó határa meghaladja a 2 százalékpontot. Az eltolásokat minden `dh {-2,-1,0,1,2}` értékre futtatjuk, a kép szélén zero padding nélkül. Seed: a tanítást három seeddel ismételjük; a seedek közötti szórást külön jelentjük a CI-től (a CI a tesztbírálat bizonytalanságát méri, a seed-szórás a tanításét). Érvényességi korlát: a 24×24-es GlyphLab nem természetes képi benchmark, és a zero pad nélküli eltolás adatvesztést okoz a határon, ezért az eredményt a határon belüli kernel-támogatottságú esetekre szűkítve értelmezzük.

16.4.8 Tipikus tévedés, önellenőrzés, visszahívás

A fejezet kulcsfigyelmeztetése: **a könyvtárhívás neve nem azonos az általa megvalósított matematikai objektum pontos definíciójával.** A `torch.nn.functional.conv2d` név ellenére térbeli keresztkorrelációt számol. Ellenpélda a 4×4-es bemenetünkön: az eredeti kernel $K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ a $[0,0]$ pozícióban 44-et ad, míg a matematikai konvolúció által megkövetelt flip $K_{rev} = \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$ ugyanitt $4 \cdot 1 + 3 \cdot 2 + 2 \cdot 5 + 1 \cdot 6 = 26$-ot ad. Tehát a `conv2d` nem flip-el; a név öröklött terminológia. Önellenőrzés: (i) a forward helyessége a három független csatorna — kézi számolás, NumPy referencia, autograd — egyezésén áll; (ii) az equivariance tételt csak a kimondott $\mathbb{Z}^2$ tartományra és s-szeres eltolásra állítjuk, a finite-grid ellenpélda a határfeltétel fontosságát rögzíti. Visszahívás: a 4.4. I. ellenőrző pont: matematika és kód megfelelése láncszabály-VJP és a 15.4. Független gradiensellenőrzési protokoll `float64` centrális differencia protokollja itt azért ad pontos eredményt, mert L a Y-ban kvadratikus, így a harmadik derivált nulla (nem a linearitás miatt); a 16.3. Elméleti és effektív recepciós mező recepciós mező rekurziója ugyanezt az indexeltolást használja, ami jól mutatja, hogy a jump és a kernel-nyúlás egyazon szomszédsági struktúra két leképezése.

16.4.9 Rövid összefoglaló

A IV. ellenőrző pont a CNN-operátort egységes specifikációként zárja. A keresztkorreláció $Y[n, o, i, j] = b[o] + \sum_{c,u,v} X_p[n, c, is + ud, js + vd]K[o, c, u, v]$, a kimeneti méret a pad-dilatáció-stride képletből adódik, a paraméterszám $C_{out}(C_{in}k_hk_w + 1)$. A backward a ugyanezen lineáris operátor adjungáltja, scatter–add szerkezettel. A bizonyított tétel szerint határmentes $\mathbb{Z}^2$ rácson stride-s esetén az input st eltolása az output t eltolását adja; a 3×3-as, pad nélküli ellenpélda mutatja, hogy véges képen a határon ez megsérül. A labor kézi értéket, `float64` referenciát, PyTorch-autograd egyezést és a shift-hibatérképet együtt ellenőrzi.

16.4.10 Következő alkalmazás

A 17.1. LeNet és a lokális megosztott reprezentáció LeNet-től kezdve a klasszikus architektúrák ugyanezt a specifikációs lapot használják: a rétegméret, a paraméterszám és a lokális megosztott reprezentáció ott visszavezethető a most ellenőrzött operátor-definícióra.

16.4.11 Felhasznált források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30. A dokumentáció szerint a `Conv2d` térbeli keresztkorrelációt számol NCHW bemenettel; a jelen szöveg ezt explicit definícióként használja.

17 Klasszikus CNN-architektúrák és tervezési döntések

17.1 LeNet és a lokális megosztott reprezentáció

17.1.1 Cél, előismeret és induló kérdés

Ez az egység a klasszikus konvolúciós hálózatok kiindulópontját, a LeNet-családot és azon belül a LeNet-5 szerkezetét vizsgálja. A 12.4. III. ellenőrző pont: önállóan tanuló kis háló egységből a teljes tanítási ciklust, a 14.4. Csoportos és depthwise konvolúció egységből a csatornakapcsolatok korlátozását, a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése egységből pedig a CNN-operátor alak-, gradiens- és szimmetriavizsgálatát ismerjük. Itt a cél az, hogy egy konkrét réteggráfon mutassuk meg: hogyan kapcsolódnak a térbeli szűrők, a csatornabővítés, az alulmintavételezés és az osztályozófej. Induló kérdés: egy 32×32-es bemenetre 5×5-ös kernel, stride 1 és padding 0 mellett hány térbeli kimeneti elem adódik, és miért beszélünk mégis „konvolúciós” rétegről akkor, amikor a keretrendszer keresztkorrelációt számol? A válasz: $32 - 5 + 1 = 28$, tehát térképenként $28 \times 28 = 784$, a hat C1-térképen összesen $6 \cdot 784 = 4704$ térbeli kimeneti elem adódik; a „konvolúciós” elnevezés hagyomány, mert a keretrendszer kernelmegfordítás nélküli keresztkorrelációt számol (lásd a LeNet-5 réteggráfját tárgyaló szakaszt).

17.1.2 A LeNet-5 réteggráfja és három alapkomponense

A [P01] tanulmány a LeNet-5 hálózatot a kézírásos dokumentumfelismerésre vezette be. Az eredeti architektúrát a cikk II.B szakasza (7–8. o., 2. ábra, I. táblázat, (5)–(6) egyenlet) írja le; az alábbi táblázat ebből készült, a jobb oldali oszlopok a labor saját, egyszerűsített **TinyLeNet-Glyph** változatát mutatják, amelyet a levezetés mutat be. A saját változat nem az eredeti reprodukciója.

Réteg	Eredeti LeNet-5 [P01]	Param. [P01]	TinyLeNet-Glyph (labor)	Param.
bemenet	$1 \times 32 \times 32$; háttér $-0{,}1$, előtér $1{,}175$ (átlag kb. 0, variancia kb. 1)	–	$1 \times 24 \times 24$, $[0, 1]$	–
C1	6 térkép, 5×5 konvolúció, kimenet $6@28 \times 28$	156	`Conv2d(1,6,5)`, ReLU, $6@20 \times 20$	156

Réteg	Eredeti LeNet-5 [P01]	Param. [P01]	TinyLeNet-Glyph (labor)	Param.
S2	2×2, nem átfedő; a négy bemenet összege × tanulható együttható + tanulható bias, majd telítődő aktiváció: a [P01]-ben a skálázott tanh $f(a) = A\tanh(Sa)$, nem logisztikus szigmoid	12	`MaxPool2d(2)`, paraméter nélkül	0
C3	16 térkép, 5×5, **részleges** kötés az I. táblázat szerint	1516	`Conv2d(6,16,5)` teljes kötés, ReLU, $16@6 \times 6$	2416
S4	mint S2, $16@5 \times 5$	32	`MaxPool2d(2)`, $16@3 \times 3$	0
C5	120 térkép, 5×5 konvolúció minden S4-térképen, kimenet 1×1	48 120	`Linear(144,120)`, ReLU	17 400
F6	84 egység, teljes kötés C5-höz	10 164	`Linear(120,84)`, ReLU	10 164
kimenet	10 euklideszi RBF-egység, $y_i = \sum_j (x_j - w_{ij})^2$, kézzel választott, rögzített ± 1 kódokkal (nem tanulható; a cikk szerinti RBF-súlyok a paraméter-számban nem szerepelnek, ezért itt „–”)	–	`Linear(84,3)` logit, cross-entropy	255

Az eredeti hálóban a C1–F6 rétegek súlyozott összegét a skálázott tangens hiperbolikus függvény nyomja össze: $x_i = f(a_i)$, $f(a) = A\tanh(Sa)$, $A = 1{,}7159$, $S = 2/3$ ([P01] (5)–(6) egyenlet; a paramétereket a cikk A. függeléke indokolja). A cikk szerint a teljes háló 340 908 kapcsolatot, de a súlymegosztás miatt csak mintegy 60 000 szabad paramétert tartalmaz.

A LeNet-5 rétegsora az **eredeti** bemeneti skálával (a háttér $-0{,}1$, az előtér 1,175, nem $[0, 1]$):

$$X \in \{-0{,}1;\ 1{,}175\}\text{-skálájú}^{1\times 32\times 32} \to C1 \to S2 \to C3 \to S4 \to C5 \to F6 \to \text{kimenet}.$$

A modernizált, saját TinyLeNet-Glyph változat bemenete $X \in [0, 1]^{1\times 24\times 24}$; a két bemeneti skálát nem szabad összekeverni.

Itt $C1$ egy 5×5-ös, 6 kimeneti csatornájú szűrőréteg, $S2$ egy 2×2-es alulmintavételező réteg, $C3$ 16 csatornát állít elő, $S4$ ismét alulmintavételez, $C5$ az 5×5-ös bemenetet 120 csatornára képezi, $F6$ pedig 84 rejtett egységet tartalmazó affin réteg, amelyet a kimenet követ. A mai keretrendszerekben $C5$ megvalósítható egyszerű affin rétegként is, mert a bemeneti térbeli kiterjedés itt már 5×5. A fejezet címében szereplő **lokális megosztott reprezentáció** arra utal, hogy a konvolúciós réteg ugyanazt a kis térbeli kernelt (megosztott súlyokkal) minden térbeli pozíción alkalmazza: a kimenet minden pontja egy lokális bemeneti ablak reprezentációja, a súlymegosztás miatt a reprezentáció eltolásra ekvivariáns, és a paraméterszám nem nő a képmérettel. A saját változat rétegeit a továbbiakban C1, P1, C2, P2, fc1, fc2, out névvel jelöljük, így a rétegnevek nem ütköznek az F1–F5 feladatnevekkel; az eredeti LeNet-5 rétegneveit (C1, S2, C3, S4, C5, F6) változatlanul használjuk.

Konvolúciós réteg mint keresztkorreláció. A 13.1. Kép mint diszkrét többcsatornás jel NCHW-indexelését és a 13.2. Konvolúció és cross-correlation definícióját használjuk. Egyetlen output elem:

$$O_{n,o,i,j} = b_o + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X_{n,c,i+u,j+v}\, K_{o,c,u,v}.$$

Ez kernelmegfordítás nélküli keresztkorreláció. A „konvolúciós réteg” elnevezés a szakirodalmi hagyományból jön; a tanulás szempontjából a megfordítás nem lényeges, mert a K súlyokat amúgy is tanuljuk. A LeNet-ben a térbeli szűrés után nemlineáris aktiváció, az eredeti hálóban a tanh-családhoz tartozó telítődő függvény következik, nem a mai ReLU.

Subsampling réteg. A „subsampling” a LeNet kontextusában nem azonos a modern max-poolinggal. Az eredeti S-rétegek a 2×2-es cellát tanulható súllyal és eltolással összegzik, majd telítődő aktivációt alkalmaznak; ez tehát tanulható paramétereket tartalmaz. A modern max-pool ezzel szemben paraméter nélküli, cellánkénti maximumot vesz:

$$P_{max}(X)_{n,c,i,j} = \max_{u\in\{0,1\}} \max_{v\in\{0,1\}} X_{n,c,si+u,sj+v},$$

ahol a LeNet-szerű beállításban $s = 2$. A két művelet nem reprodukciós értelemben sem felcserélhető: más a gradiens útja, és más a paraméterszám.

Classifier head. A konvolúciós rétegek által előállított feature mapokat a háló vége lapítja, majd affin rétegekre bízza az osztályozást. A LeNet-5 esetében ez a $C5 \to F6 \to$ kimenet szakasz. A modern változatokban a kimenet előtti affin réteg véges logitokat ad; a softmax és a cross-entropy kapcsolatát a 7.2. Softmax és többosztályos cross-entropy tárgyalta; ott rögzítettük, hogy a loss logitokat vár, nem softmaxolt valószínűséget.

17.1.3 Történeti változatok: az eredeti LeNet-5 és a mai „LeNet-szerű” háló

Az eredeti LeNet-5 leírása és a mai könyvtári példák közötti eltérés gyakran elmosódik. A három legfontosabb különbség:

1. Az eredeti háló 32×32-es bemenetet használ, és a 28×28-as számjegyet középre helyezi; a laborváltozatunk a függelék A. A GlyphLab adatkészlet szakaszában leírt 24×24-es GlyphLab-bemenettel dolgozik.
2. Az eredeti háló tanh-szerű aktivációt és tanulható súlyozású subsamplinget használ; a modern változat gyakran ReLU-t és max-poolingot.
3. A $C3$ réteg az eredeti LeNet-5-ben nem teljes konvolúció ([P01] I. táblázat): az első hat C3-térkép három szomszédos S2-térképből, a következő hat négy szomszédosból, a következő három négy nem szomszédosból kap bemenetet, az utolsó mind a hatból. Paraméterszáma így $6\cdot(3\cdot25+1)+6\cdot(4\cdot25+1)+3\cdot(4\cdot25+1)+(6\cdot25+1) = 456+606+303+151 = 1516$. A cikk indoklása kettős: a kapcsolatok száma kezelhető marad, és a szimmetriatörés különböző jellemzők tanulására kényszeríti a térképeket. A laborváltozat teljesen konvolúciós $C3$-at használ ($6 \cdot 16 \cdot 25 + 16 = 2416$ paraméter).

A „történeti változat” kifejezés tehát nem csupán címke: a változat megnevezéséhez pontosan rögzíteni kell a bemeneti méretet, az aktivációt, a pooling típusát, a csatornakötést és a kimeneti réteg szerkezetét.

17.1.4 Levezetés: a választott TinyLeNet-Glyph minden alakja és paramétere

A laborhoz választott saját változat:

- Bemenet: $X \in \mathbb{R}^{N\times1\times24\times24}$, float32, NCHW.
- C1: Conv2d(1, 6, kernel 5×5, stride 1, padding 0), ReLU.
- P1: MaxPool2d(2×2, stride 2).
- C2: Conv2d(6, 16, kernel 5×5, stride 1, padding 0), ReLU.
- P2: MaxPool2d(2×2, stride 2).
- Flatten, majd fc1: 144→120, fc2: 120→84, out: 84→3 logit.

A térbeli kimeneti méret a 14.1. Stride, padding, dilation és outputméret formulájával:

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1.$$

A számítás lépésről lépésre:

- C1: $H = W = 24$, $p = 0$, $k = 5$, $s = 1$, tehát $H_{out} = W_{out} = 24 - 5 + 1 = 20$. A kimenet alakja $N \times 6 \times 20 \times 20$.
- P1: $k = 2$, $s = 2$, $p = 0$; 20 bemenetből $\lfloor(20 - 2)/2\rfloor + 1 = 10$. Alak: $N \times 6 \times 10 \times 10$.
- C2: $10 - 5 + 1 = 6$. Alak: $N \times 16 \times 6 \times 6$.
- P2: $\lfloor(6 - 2)/2\rfloor + 1 = 3$. Alak: $N \times 16 \times 3 \times 3$, lapítva $N \times 144$.
- fc1: $N \times 120$; fc2: $N \times 84$; out: $N \times 3$.

Réteg	Kimeneti alak	Paraméterképlet	Paraméterszám
C1	6×20×20	$6 \cdot (1 \cdot 5 \cdot 5) + 6$	156
P1	6×10×10	0	0
C2	16×6×6	$16 \cdot (6 \cdot 5 \cdot 5) + 16$	2416

Réteg	Kimeneti alak	Paraméterképlet	Paraméterszám
P2	16×3×3	0	0
fc1	120	$144 \cdot 120 + 120$	17 400
fc2	84	$120 \cdot 84 + 84$	10 164
out	3	$84 \cdot 3 + 3$	255

A bias-tag minden kimeneti csatornán, illetve kimeneti egységen külön számít. Az összeg:

$$156 + 2416 + 17400 + 10164 + 255 = 30\,391.$$

Ez nem a LeNet-5 eredeti paraméterszáma; ez a saját laborváltozaté. A fenti levezetés típusa egyszerű alak- és paraméterszámítás, nem tételbizonyítás; az ellenőrzés módja a képletek indexhatár-vizsgálata és a későbbi kód.

17.1.5 Labor: TinyLeNet-Glyph ellenőrzése a GlyphLab adatúton

Előfeltétel. Python 3.12.3, NumPy 2.5.4, pandas 3.1.0 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy`, `metadata.csv` és `corrupted_test.npy` fájlja; a `tiny_lenet_check.py` a `lab/` könyvtárból futtatható (`python tiny_lenet_check.py`).

A labor nem tanít, hanem a modult és az adatbetöltést ellenőrzi. Munkakönyvtár: `lab/`; a szkript a saját könyvtárához képest keresi a `data/` mappát. Az alábbi kód futtatható; az utána közölt kimenet a rögzített környezetű ellenőrző futás **mért kimenete** (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020). A hallgató a `python tiny_lenet_check.py` paranccsal futtassa.

Az adat a GlyphLab gyűjtemény: `data/images.npy` alakja $(360, 1, 24, 24)$, `float32`, értéktartománya $[0, 1]$ (szürkeárnyalatos NCHW-képek); `data/labels.npy` alakja $(360,)$, `int64`, értékei $\{0, 1, 2\}$ (0=vertical, 1=horizontal, 2=diagonal). A `data/metadata.csv` oszlopai: `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax`; az `index` a képtömb sorindexe, a `scene_id` a független mintavételi egység, egy jelenethez két nézet (`view_id` 0 és 1) tartozik. A 120/30/30 jelenetes felosztás 240/60/60 képet jelent. A `data/corrupted_test.npy` a 60 tesztkép zajos változata, a metadata test sorainak sorrendjében. A képeket a `lab/generate_data.py` állítja elő (alapértelmezett seed 20261020); a részletes adatleírást a `lab/ADATOK.md` tartalmazza.

```
# lab/tiny_lenet_check.py
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32
DATA = Path(__file__).resolve().parent / "data"

class TinyLeNetGlyph(nn.Module):
    """LeNet-szerű, de nem az 1998-as LeNet-5 pontos reprodukciója:
```

```
    teljes konvolúciós C3, ReLU, modern max-pool, három GlyphLab-osztály."""
    def __init__(self, in_channels=1, num_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 6, kernel_size=5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 3 * 3, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

images = np.load(DATA / "images.npy")  # (360,1,24,24), float32, [0,1]
labels = np.load(DATA / "labels.npy")  # (360,), int64
meta = pd.read_csv(DATA / "metadata.csv")
assert images.dtype == np.float32 and labels.dtype == np.int64
assert images.shape == (360, 1, 24, 24) and labels.shape == (360,)
assert images.min() >= 0.0 and images.max() <= 1.0
assert set(np.unique(labels)) <= {0, 1, 2}
assert meta["index"].tolist() == list(range(len(meta))), "a metadata index oszlopa nem sorrendi"
assert (meta["label"].to_numpy() == labels).all(), "a metadata és a labels.npy nem egyezik"

x = torch.from_numpy(images[:4].astype(np.float32)).to(device=device, dtype=dtype)
model = TinyLeNetGlyph().to(device=device, dtype=dtype)

with torch.no_grad():
    logits = model(x)  # a CrossEntropyLoss logitot vár, softmax nélkül

n_params = sum(p.numel() for p in model.parameters())
assert logits.shape == (4, 3), logits.shape
assert n_params == 30391, n_params
assert torch.isfinite(logits).all()

print("logits shape:", tuple(logits.shape))
print("parameter count:", n_params)
print("first-batch logits finite:", bool(torch.isfinite(logits).all()))
```

MÉRT KIMENET (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
logits shape: (4, 3)
parameter count: 30391
first-batch logits finite: True
```

Ellenőrzés: a shape-nak $(4, 3)$-nak, a paraméterszámnak 30 391-nek kell lennie; a `Flatten` a `features` kimenet $(N, 16, 3, 3)$ lapításából $(N, 144)$-et ad. A seed a súlyok inicializálását rögzíti; a fenti kimenet a futtatott ellenőrzés eredménye, a konkrét logit-értékeket azonban a program nem nyomtatja.

17.1.6 Feladatok F1–F5

F1 (fogalmi, nehézség 1). Magyarázd el, miért nem ugyanaz a művelet az eredeti LeNet-5 `S2/S4` alulmintavételező rétege és a modern `MaxPool2d(2,2)`. Adj egy konkrét, 2×2-es számpéldát, amelyen a maximum és a tanulható súlyozott összeg különböző viselkedését megmutatod; nevezd meg, melyik művelet melyik.

F2 (levezetési, nehézség 2). Egy új változat specifikációja: input $1 \times 1 \times 24 \times 24$; conv1: 6 csatorna, 5×5, stride 1, padding 0; MaxPool 2×2, stride 2; conv2: 16 csatorna, 3×3, stride 1, padding 1; MaxPool 2×2, stride 2; fc1: 400→120; fc2: 120→84; kimenet: 84→3. Vezesd le minden réteg kimeneti alakját és a teljes paraméterszámot. Mutasd meg, miért tartja meg a padding=1 a 10×10-es térbeli méretet a 3×3-as kernelnél.

F3 (implementációs, nehézség 1). Implementáld a `TinyLeNetGlyph` modult a levezetés specifikációja szerint úgy, hogy a paraméterszám 30 391 legyen. Futtass rajta egy $(2, 1, 24, 24)$-es véletlen float32 tensort, és ellenőrizd a kimeneti alakot. A megoldáshoz add meg a futtatható kódot.

F4 (hibakeresési, nehézség 2). Az alábbi töredékben a második poolingréteg már szerepel, de a `Linear(576,120)` bemeneti méretét nem igazították hozzá. Lokalizáld az alakhibát, írd le a futtatási következményt, javítsd a kódot, és add meg azt az assertet, amellyel a javítást visszaellenőrzöd.

```
self.features = nn.Sequential(
    nn.Conv2d(1, 6, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Conv2d(6, 16, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
)
self.fc = nn.Linear(16 * 6 * 6, 120)  # hibás: a bemenet valójában 16*3*3
```

F5 (kísérleti/transzfer, nehézség 3). Tervezz kísérletet annak eldöntésére, hogy a GlyphLabon tanított kis LeNet-típusú modell első konvolúciós rétegének szűrői nagyobb térbeli lokalitással bírnak-e, mint ugyanazon seed melletti véletlen inicializálású súlyok. Add meg a hipotézist, a scene_id-alapú felosztást, a baseline-t, a lokalitás kvantitatív definícióját, a választási protokollt és az érvényességi határt. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

17.1.7 Megoldások és indoklás

F1 megoldás. Az eredeti S-réteg a 2×2-es cella értékeit tanulható w súllyal és b eltolással összegzi, majd telítődő aktivációt alkalmaz; a max-pool ezzel szemben a max-ot veszi, és nincs tanulható paramétere. Példa: $X = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$. A max-pool kimenete 3; egy $w = 0{,}5$, $b = 0$ súlyozott összeg kimenete $0{,}5(1+2+3+0) = 3$, ami itt véletlenül egyezik, ezért jobb ellenpélda: $X = \begin{pmatrix} 1 & 1 \\ 1 & 10 \end{pmatrix}$-nél a max=10, a $w = 0{,}5$ súlyú súlyozott összeg $13 \cdot 0{,}5 = 6{,}5$ (ez nem átlag: az átlaghoz $w = 0{,}25$ kellene). Az aktiváció és a gradiens útja is eltérő. Az S-réteg műveletének leírását a [P01] II.B szakasza adja.

F2 megoldás. A H_{out}-képletet használjuk. C1: $24 - 5 + 1 = 20$, alak $6 \times 20 \times 20$. P1: $\lfloor (20-2)/2 \rfloor + 1 = 10$, alak $6 \times 10 \times 10$. C2: $10 + 2 \cdot 1 - 3 = 9$, $\lfloor 9/1 \rfloor + 1 = 10$, tehát a padding éppen a kernel okozta 2 pixeles csökkenést kompenzálja: $2p = k - 1$ (ez az összefüggés $s = 1$ mellett érvényes), ezért a térbeli méret 10×10 marad; alak $16 \times 10 \times 10$. P2: $\lfloor (10-2)/2 \rfloor + 1 = 5$, alak $16 \times 5 \times 5$; flatten 400. A paraméterek:

- C1: $6 \cdot (1 \cdot 5 \cdot 5) + 6 = 156$;
- C2: $16 \cdot (6 \cdot 3 \cdot 3) + 16 = 864 + 16 = 880$;
- fc1: $400 \cdot 120 + 120 = 48\,120$;
- fc2: $120 \cdot 84 + 84 = 10\,164$;
- kimenet: $84 \cdot 3 + 3 = 255$.

Összesen: $156 + 880 + 48\,120 + 10\,164 + 255 = 59\,575$. Alakellenőrzés: a lapítás $16 \cdot 5 \cdot 5 = 400$ valóban az fc1 bemenete; az osztályozó bemenete minden mintára a 400 elemű lapított vektor.

F3 megoldás. Az alábbi kód önállóan futtatható (a modul definícióját is tartalmazza), és a levezetés specifikációját valósítja meg. A labor ellenőrző szkriptje 4 képes batch-en fut; itt a feladatnak megfelelően 2 képes batch-en ellenőrzünk.

```
import torch
import torch.nn as nn

torch.manual_seed(20261020)


class TinyLeNetGlyph(nn.Module):
    def __init__(self, in_channels=1, num_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 6, kernel_size=5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 3 * 3, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))


x = torch.randn(2, 1, 24, 24, dtype=torch.float32)
model = TinyLeNetGlyph()
out = model(x)
n_params = sum(p.numel() for p in model.parameters())
assert out.shape == (2, 3)
assert n_params == 30391
print(tuple(out.shape), n_params)
# mért kimenet: (2, 3) 30391
```

A mért futás (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, seed 20261020) a $(2, 3)$ alakot és a 30 391 paramétert adta. A paraméterszám egzakt egész, ezért tűrésre nincs szükség; a shape-nál ugyanígy.

F4 megoldás. A második `MaxPool2d(2,2)` után a konvolúciós rész kimenete $16 \times 3 \times 3$, vagyis lapítva 144 elem. A hibás `Linear(576,120)` ezzel szemben 576 bemeneti elemet vár, ezért $(4, 1, 24, 24)$ batch esetén a továbbítás `RuntimeError: mat1 and mat2 shapes cannot be multiplied` kivételt eredményez. A javított rétegrész:

```
self.features = nn.Sequential(
    nn.Conv2d(1, 6, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Conv2d(6, 16, 5), nn.ReLU(), nn.MaxPool2d(2, 2),
)
self.fc = nn.Linear(16 * 3 * 3, 120)
```

Ellenőrzés a `features` kimenetének lapítása után (a modulban ezt az `nn.Flatten()` végzi a `classifier` első elemeként): `x = torch.randn(4,1,24,24); h = self.features(x).flatten(1); assert h.shape == (4,144); assert self.fc(h).shape == (4,120)`. A teljes modell kimenete $(4, 3)$.

F5 megoldásminta. Hipotézis: a tanított modell első rétegének szűrői nagyobb térbeli koncentrációt mutatnak a kernel középpontja körül, mint az azonos seeddel inicializált, nem tanított baseline szűrői. Adat: a GlyphLab `data/metadata.csv` metaadat-táblája (`index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax` oszlopokkal); a `scene_id` a független mintavételi egység (jelenetazonosító, például `G0-0000`), egy jelenethez két nézet (`view_id` 0 és 1) tartozik. A szerződés szerinti split 120/30/30 jelenet, azaz 240/60/60 kép; a képek a `data/images.npy` tömbben vannak, a metaadatot csak a split és a jelenetazonosítók kiolvasására használjuk. Baseline: azonos architektúra azonos seeddel inicializálva, tanítás nélkül. Változó tényező: a train spliten futtatott tanítás Adam optimalizálóval, 10^{-3} tanulási rátával, 32-es batchcsel, 20 epochkal, a legjobb validációs loss-ú checkpoint kiválasztásával; a test splitet csak a végén értékeljük. Lokalitás: minden 5×5-ös K szűrőre

$$r(K) = \frac{\sum_{(u,v):\|(u,v)-(2,2)\|_\infty \le 1} K_{u,v}^2}{\|K\|_F^2},$$

ahol $\|(u,v)-(2,2)\|_\infty = \max(|u-2|, |v-2|)$ a maximumnorma (Csebisev-távolság), $\|K\|_F^2 = \sum_{u,v} K_{u,v}^2$ a Frobenius-norma négyzete; $r(K)$ tehát a középső 3×3-as mag négyzetes energiaaránya. Fontos, hogy $r(K)$ a szűrő térbeli koncentrációját méri a kernel középpontja körül, nem a háló recepciós mezőjét és nem a lokalitás egyetlen teljes mértékét: egy nagy $r(K)$ értékű szűrő is lehet éles peremdetektor, ezért az eredményt csak szűrőszintű proxként értelmezzük. Az egyenletes eloszlás referenciája 5×5-ös kernelnél $9/25 = 0{,}36$. A hat szűrő mediánja önmagában leíró statisztika, nem elég a következtetéshez. A bizonytalanságbecsléshez 20, jelenetszinten újramintázott **tanítóhalmazon újratanított** modell–baseline párt használunk (egyetlen rögzített modellnél a tesztjelenetek újramintázása nem változtatná a szűrőstatisztikát); minden párra kiszámítjuk a mediánok különbségét, és a 95%-os intervallumot a 20 különbség 2,5 és 97,5 százalékos percentilise adja. Beadandó: a tanított és baseline mediánok nyers értékei, valamint az újratanítások páros különbségének intervalluma. Érvényességi határ: a GlyphLab három orientációosztályt tartalmaz, ezért az eredmény nem igazol általános „élreprezentációs” állítást; a lokalitásnövekedés egyetlen toy-adathalmazra vonatkozik.

17.1.8 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés, hogy egy modern, LeNet-szerű modellt automatikusan „az eredeti LeNet-5 reprodukciójának” neveznek. Ellenpélda: a fenti `TinyLeNetGlyph` 24×24-es bemenetet, ReLU-t, max-poolingot, teljesen konvolúciós C3-at és 3 osztályos logit-kimenetet használ; az eredeti LeNet-5 ezzel szemben 32×32-es bemenetet, telítődő aktivációt, tanulható súlyozású subsamplinget, C3-nál nem teljes csatornakötést és más kimeneti felépítést alkalmaz. Ha valaki a laborváltozat 30 391 paraméterét az 1998-as cikk paraméterszámának állítaná be, téves összehasonlítást tenne, mert a C3-kötési tábla eltérése miatt a két háló súlytenzora szerkezetileg sem azonos. A korrekt megfogalmazás: „LeNet-típusú, modernizált változat”.

Önellenőrzés: a H_{out}-képletbe a padding, stride és kernel értékek visszahelyettesítése; a paraméterszámok kézi összegzése a `sum(p.numel())` ellenőrzéssel szemben; annak ellenőrzése, hogy a lapítás mérete mindig a feature map térfogata. Korábbi fogalom visszahívása: a 16.1. Eltolási equivariance és invariance szerint a konvolúciós réteg eltolási ekvivarianciát ad, de invarianciát önmagában nem; a LeNet-ben az invariancia felé az alulmintavételező rétegek lépnek. Az 5.1. Esemény, feltétel és véletlen változó tanulsága szerint a végső softmax kimenet normalizált valószínűségeloszlás, a három logit önmagában nem az.

17.1.9 Rövid összefoglaló

A LeNet-5 a konvolúciós hálózatok történeti alapváltozata: a P01 tanulmány a kézírásos dokumentumfelismerésre vezette be. A réteggráf három komponenst kapcsol össze: térbeli keresztkorrelációs rétegeket, 2×2-es alulmintavételező rétegeket és affin osztályozófejet. Az eredeti LeNet-5 32×32 bemenetet, telítődő aktivációt, súlyozott subsamplinget, C3-nál részleges csatornakötést és más kimenetet használ; a mai könyvtárakban gyakori max-poolos, ReLU-s, teljes konvolúciós változat ettől eltér, ezért nem nevezhető pontos reprodukciónak. A laborban megvalósított TinyLeNet-Glyph 24×24-es GlyphLab-bemenetre épül, 30 391 tanulható paramétere van, és logitokat ad három osztályra.

17.1.10 Következő alkalmazás

A következő, 17.2. AlexNet és a kísérleti kontextus egység az AlexNet hálózatot vizsgálja: ott a méretnövekedés, a kétutas GPU-elrendezés és a ReLU-alapú gyorsabb tanítás hoz új tervezési döntést.

17.1.11 Felhasznált források

- [P01] LeCun, Y., Bottou, L., Bengio, Y. és Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition.* Proceedings of the IEEE, 86(11), 2278–2324. DOI: 10.1109/5.726791. Az architektúra: II.B szakasz, 7–8. o., I. táblázat. https://doi.org/10.1109/5.726791.

17.2 AlexNet és a kísérleti kontextus

17.2.1 Cél, előismeret és induló kérdés

A 17.1. LeNet és a lokális megosztott reprezentáció egységben a LeNet-5 réteggráfját és a mai könyvtári változatok eltéréseit vizsgáltuk. Most az AlexNet következik: ez az a hálózat, amelynél a mély, nagy kapacitású CNN és a nagyméretű képi adat együttesen vált meghatározóvá. Előismeretként a 13.2. Konvolúció és cross-correlation konvolúciós jelölését, a 16.2. Pooling, downsampling és aliasing pooling definícióját és az 5.2. Várható érték, variancia és likelihood statisztikai gondolkodását idézzük vissza.

Induló kérdés: mit jelent pontosan az a gyakran idézett állítás, hogy az AlexNet „15,3%-os top-5 hibát ért el”, és miért nem lehet ezt a számot mai modellekkel közvetlenül összehasonlítani?

17.2.2 A P04 eredmény mint empirikus állítás

A [P04] tanulmány NeurIPS-proceedings PDF-je alapján az alábbi állítások helyét szakaszonként megadjuk (a hivatkozott oldalak a proceedings-változatból származnak). Az absztrakt szerint a szerzők egy mély konvolúciós hálózatot tanítottak az ImageNet ILSVRC-2010 adatán, és az ILSVRC-2012 versenyre beadott modellváltozat **15,3%-os top-5 teszthibát** ért el, szemben a második legjobb nevezés **26,2%-ával**. A 15,3% nem egyetlen háló eredménye: hét (öt előtanított és két tovább tanított) CNN átlagolt predikciójáé; egyetlen modell hibája ennél nagyobb. Az absztrakt emellett megadja a modell nagyságrendjét: körülbelül **60 millió tanulható paraméter**, **650 000 neuron**, öt konvolúciós réteg, amelyek egy részét max-pooling követi, valamint három teljesen összekötött réteg, végső 1000 kimenetes softmaxszal. A tanításhoz nem telítődő neuronokat és hatékony GPU-implementációt használtak; a túltanulás csökkentésére a teljesen összekötött rétegekben dropoutot alkalmaztak.

Ez az állítás **típusa szerint empirikus állítás**, nem matematikai tétel. Csak akkor értelmezhető, ha rögzítjük a feltételeket: adat, split, modellváltozat, betanítási eljárás és kiértékelési metrika. A 15,3% tehát az ILSVRC-2012 verseny adott tesztkészletén, a szerzők által beadott változatra vonatkozik. Nem azt jelenti, hogy bármely AlexNet-implementáció ma is 15,3%-ot érne el, és nem is azt, hogy a szám mai körülmények között reprodukálható eredmény lenne.

17.2.3 Az AlexNet rétegtáblája és bemeneti mérete

A történeti összefoglalók bemeneti mérete eltérhet: a 224-es és 227-es képméret mellett az első réteg térbeli mérete sem azonos. Az alábbi rétegtábla **227×227-es bemenetre, `groups=1` kapcsolatokra készített didaktikai változat**, nem az eredeti, két GPU közötti csoportos csatornakapcsolatok teljes rekonstrukciója. A konvolúció pontos kimeneti alakját ezért minden rétegnél a képletből számítjuk.

A 14.1. Stride, padding, dilation és outputméret alfejezetben ismertetett képlet szimmetrikus paddingre (ahol p a padding szélessége):

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

$H = 227$, $p = 0$, $d = 1$, $k = 11$, $s = 4$ esetén

$$H_{out} = \left\lfloor \frac{227 - 10 - 1}{4} \right\rfloor + 1 = \left\lfloor \frac{216}{4} \right\rfloor + 1 = 54 + 1 = 55.$$

A 224-es bemenetnél a hányados $213/4 = 53{,}25$, de a képletben **alsó egészrész** szerepel, így $H_{out} = \lfloor 53{,}25 \rfloor + 1 = 54$; a kimeneti méret tehát ebben az esetben is egész. A 227-es méret az **55×55-ös**, a 224-es méret pedig az **54×54-es** első jellemzőtérképet adja. A [P04] szövege $224 \times 224 \times 3$ bemenetet ír (3.5. szakasz, 4. o.), a 2. ábra viszont 55×55-ös első réteget mutat; a kettő padding nélkül nem fér össze, ezért a rekonstrukciók többsége 227-es bemenettel számol. Ha a conv1 paddingje $p = 2$ lenne, a 224-es bemenet is 55×55-öt adna: $\lfloor (224 + 4 - 10 - 1)/4 \rfloor + 1 = 54 + 1 = 55$. A táblázat a $p = 0$ konvenciót használja, ezért ott 227 szükséges. A rétegtábla következetesen ezt a 227-es didaktikai változatot használja, és ezt az eltérést a forrás pontatlanságaként, nem a saját számítás hibájaként kezeljük.

Réteg	Kimeneti alak (NCHW)	Kernel/stride/pad	Tanulható paraméter
input	$N \times 3 \times 227 \times 227$	–	–

Réteg	Kimeneti alak (NCHW)	Kernel/stride/pad	Tanulható paraméter
conv1	$N \times 96 \times 55 \times 55$	11×11, s4, p0	34 944
pool1	$N \times 96 \times 27 \times 27$	3×3, s2	0
conv2	$N \times 256 \times 27 \times 27$	5×5, s1, p2	614 656
pool2	$N \times 256 \times 13 \times 13$	3×3, s2	0
conv3–5	lásd lent	3×3, s1, p1	lásd lent

A táblázat célzatosan egyszerűsít. A [P04] 3.3. szakaszában leírt LRN (local response normalization) rétegeket nem tartalmazza; ezeknek nincs tanulható paraméterük, így a paraméterszámot nem érintik. A 3×3-as, stride 2-es pooling pedig éppen a [P04] 3.4. szakaszának **átlapoló (overlapping) pooling**ja: az ablak nagyobb, mint a lépésköz. A táblázat `groups=1` értékei mellett az eredeti, két GPU-s `groups=2` háló paraméterszámai a táblázat után, külön bekezdésben szerepelnek, hogy a két változat ne keveredjen.

A conv3, conv4 és conv5 rendre 384, 384 és 256 kimeneti csatornával rendelkezik, mindegyik 13×13-as térbeli méreten marad. Paraméterszámuk:

- conv3: $384 \cdot 256 \cdot 9 + 384 = 885\,120$,
- conv4: $384 \cdot 384 \cdot 9 + 384 = 1\,327\,488$,
- conv5: $256 \cdot 384 \cdot 9 + 256 = 884\,992$.

Az utolsó pooling után 6×6-os térbeli méret marad, amit három teljesen összekötött réteg követ: $9216 \to 4096$, $4096 \to 4096$ és $4096 \to 1000$. A paraméterszámok rendre 37 752 832, 16 781 312 és 4 097 000. Az eredeti hálóban a conv2, conv4 és conv5 kernelei csak a saját GPU-juk térképeit látják: a [P04] 3.5. szakasza szerint a kernelméretek $5 \times 5 \times 48$, $3 \times 3 \times 192$ és $3 \times 3 \times 192$, míg a conv3 ($3 \times 3 \times 256$) és a teljesen összekötött rétegek minden előző térképhez kapcsolódnak. Ez `groups=2` csoportos konvolúciónak felel meg: conv2 $256 \cdot 48 \cdot 25 + 256 = 307\,456$, conv4 $384 \cdot 192 \cdot 9 + 384 = 663\,936$, conv5 $256 \cdot 192 \cdot 9 + 256 = 442\,624$ paraméter. A fenti táblázat `groups=1` értékeivel a teljes összeg 62 378 344, a csoportos kötéssel 60 965 224; ez utóbbi **közelíti** a cikk „60 million parameters” állítását ($60\,965\,224 \approx 61$ millió; 3.5. szakasz és összefoglaló).

A súlytenzor alakja a `groups=1` konvolúciós rétegeknél $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, ahol C_{in} a bemeneti csatornák száma, C_{out} a kimeneti csatornák száma. Csoportos konvolúciónál (`groups=g`, például a fenti `groups=2` AlexNet-változatban) az alak $K \in \mathbb{R}^{C_{out} \times (C_{in}/g) \times k_h \times k_w}$: a conv2 kernelalakja így $256 \times 48 \times 5 \times 5$, nem $256 \times 96 \times 5 \times 5$. A bias minden kimeneti csatornához egy skalár.

17.2.4 ReLU: telítődés és tanítási sebesség

A [P04] „nem telítődő neuronokat” használ; a 3.1. szakasz szerint a konkrét aktiváció a ReLU, amelyet a 9.4. Aktivációk és gradiensáramlás egységben definiáltunk:

$$f(x) = \max(0, x),$$

elemenként értelmezve, azaz tenzor bemenetére is elemenként alkalmazzuk. A telítődő függvények, például a tanh vagy a szigmoid, a bemeneti tartomány szélein közelítőleg konstansok, így deriváltjuk nullához tart; a ReLU deriváltja $x > 0$-ra 1, $x < 0$-ra 0. A 0 pontban a klasszikus derivált nem létezik; a könyvtárak egy konvenciót választanak, például 0-t.

A nem telítődő tulajdonság heurisztikus előnye, hogy a pozitív tartományban a gradiens nem tűnik el. Ez nem globális konvergenciatétel, hanem empirikus megfigyelés: a [P04] 3.1. szakasza (3. o., 1. ábra) szerint egy négyrétegű konvolúciós háló a CIFAR-10-en ReLU-val hatszor gyorsabban

érte el a 25%-os tanítási hibát, mint tanh-val. Ez egyetlen architektúrán és adatkészleten mért eredmény, nem általános törvény.

17.2.5 Dropout mint regularizáló zaj

A dropout a tanítás során minden előrecsatolt lépésben véletlenül nullázza a neuronok egy részét. Ha $h \in \mathbb{R}^d$ egy réteg aktivációja, akkor

$$h'_i = m_i h_i, \qquad m_i \sim \text{Bernoulli}(1-p),$$

ahol p az eldobási arány, azaz a dropout-valószínűség (ezt a p-t a padding p szélességétől meg kell különböztetni), és a m_i maszkok függetlenek. A várható érték $\mathbb{E}[h'_i] = (1-p)h_i$, ezért a kiértékelés során a determinisztikus hálózatban vagy $1-p$-vel skálázunk, vagy az úgynevezett inverted dropout esetén már a tanítás alatt $\frac{1}{1-p}$-vel osztunk. A [P04] 4.2. szakasza (6. o.) szerint a dropoutot az első két teljesen összekötött rétegben alkalmazták $p = 0{,}5$ eldobási valószínűséggel, és a teszteléskor minden neuront használtak, de a kimenetüket 0,5-tel szorozták. Ez tehát a klasszikus, *nem* inverted konvenció: a skálázás a kiértékeléskor történik.

17.2.6 Adatkiegészítés (augmentation)

Az adatkiegészítés olyan címkemegőrző transzformáció, amely a tanítóhalmazt bővíti. A [P04] 4.1. szakasza (5. o.) két formát ír le: véletlen 224×224-es kivágásokat és azok vízszintes tükörképeit a 256×256-os képekből (a cikk itt 224-et ír, míg a fenti rétegtábla 227-es bemenettel számol; a két konvenció nem keverendő) (tesztkor az öt kivágás és tükörképeik átlagolt predikciójával), valamint az RGB-intenzitások megváltoztatását a tanítóhalmaz pixeleire számolt főkomponens-analízis (PCA) irányai mentén. Minden transzformációnál ellenőrizni kell, hogy az osztálycímke változatlan marad; ez a 22.2. Címkemegőrző transzformációk egységben részletesen visszatér majd. A GlyphLab a könyv saját, procedurálisan generált szürkeárnyalatos képadatkészlete; a nézetek, a split és a címkék szabályait a függelék A. szakasza (40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) rögzíti (a továbbiakban: GlyphLab adatleírás vagy adatszerződés). Ezen az adaton például a 90 fokos forgatás nem címkemegőrző, ahogy az adatleírás kimondja.

17.2.7 GPU-kontextus

A [P04] kiemeli a hatékony GPU-implementációt. A korabeli gyakorlatban a hálózatot két GPU-ra osztották, és bizonyos rétegek csak a saját GPU-juk adatát látták, más rétegek pedig mindkét GPU kimenetét összekapcsolták. A [P04] 3.2. szakasza (3. o.) szerint a kernelek felét az egyik, felét a másik GPU-n helyezték el, és a GPU-k csak bizonyos rétegekben kommunikálnak: a 3. réteg kernelei a 2. réteg összes térképét látják, a 4. réteg kernelei viszont csak az azonos GPU-n lévő 3. rétegbeli térképeket; ugyanez érvényes az 5. rétegre. A mi laborunk kis CPU-s változatot használ, tehát a GPU-kontextust történeti körülményként kezeljük, nem reprodukciós követelményként.

17.2.8 Top-k metrika és kézi példa

Legyen $Z \in \mathbb{R}^{N \times K}$ a logit mátrix, ahol N a minták, K az osztályok száma. Itt K tehát osztályszám, nem a korábbi súlytenzor jele, a $p_{n,i}$ pedig valószínűség, nem a padding vagy a dropout p-je. A softmax soronként normalizál:

$$p_{n,i} = \frac{\exp(z_{n,i})}{\sum_{j=1}^{K} \exp(z_{n,j})}.$$

A softmax soronként sorrendtartó: az exp szigorúan monoton, a nevező pedig adott sorban pozitív konstans, ezért a legnagyobb valószínűségek indexsorrendje megegyezik a legnagyobb logitok sorrendjével; a top-k halmaz tehát számolható közvetlenül a logitokból is. A top-k pontosság:

$$\text{topk_acc}(Z, y) = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\{y_n \in \text{top}_k(p_n)\},$$

ahol $\text{top}_k(p_n)$ a p_n legnagyobb k elemének indexhalmaza. A top-k hiba $1 - \text{topk_acc}$.

Kézi példa. Legyen $N = 1$, $K = 10$ és a logit vektor

$$z = [0{,}1,\ 2{,}0,\ -0{,}5,\ 1{,}2,\ 0{,}3,\ -1{,}0,\ 0{,}8,\ 0{,}05,\ 1{,}5,\ -0{,}2].$$

A legnagyobb öt logit indexe sorrendben 1, 8, 3, 6, 4. Ha a valódi címke $y = 3$, akkor a minta top-3-ban helyes, top-2-ben hibás. A top-3 pontosság ezen az egy mintán 1, a top-2 pontosság 0. Ez mutatja, hogy a k megváltoztatása önmagában más metrikát ad, még ugyanazon a modellen is.

17.2.9 Labor: rétegtábla-generáló és kísérleti terv

Előfeltétel. Python 3.12.3 a `cnn-labor` csomag rögzített környezetéből (`uv sync`); a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n066/alexnet_table.py` futtatható (`python lab/n066/alexnet_table.py`); a szkript a szabványos könyvtáron kívül mást nem használ.

A labor célja, hogy a hallgató saját maga számolja ki a rétegtáblát. A következő szkriptben nincs véletlen szám, ezért seed nem szükséges; a számítás egész értékeken egész osztással (`//`) történik, ezért az eredmény pontosan reprodukálható. A munkakönyvtár `lab/n066/`, a fájl `alexnet_table.py`.

```
def conv_out(h, w, kh, kw, s, p):
    ho = (h + 2*p - kh) // s + 1
    wo = (w + 2*p - kw) // s + 1
    return ho, wo

def pool_out(h, w, k, s):
    ho = (h - k) // s + 1
    wo = (w - k) // s + 1
    return ho, wo

# Bemenet: 227x227x3, NCHW alakban csatorna elöl.
H, W = 227, 227
cin = 3
```

```python
# conv1: 96 db 11x11 kernel, stride 4, padding 0
h, w = conv_out(H, W, 11, 11, 4, 0)
p1 = 96 * cin * 11 * 11 + 96
assert (h, w) == (55, 55) and p1 == 34_944
print(f"conv1: {96}x{h}x{w}, params={p1}")

# pool1: 3x3, stride 2
h, w = pool_out(h, w, 3, 2)
assert (h, w) == (27, 27)
print(f"pool1: {96}x{h}x{w}")

# conv2: 256 db 5x5 kernel, stride 1, padding 2
h, w = conv_out(h, w, 5, 5, 1, 2)
p2 = 256 * 96 * 5 * 5 + 256  # cin=96 a conv1 kimenete (g=1)
assert (h, w) == (27, 27) and p2 == 614_656
print(f"conv2: {256}x{h}x{w}, params={p2}")

# pool2: 3x3, stride 2
h, w = pool_out(h, w, 3, 2)
assert (h, w) == (13, 13)
print(f"pool2: {256}x{h}x{w}")

# conv3, conv4, conv5: 3x3, stride 1, padding 1
expected = {"conv3": (384, 13, 13, 885_120),
            "conv4": (384, 13, 13, 1_327_488),
            "conv5": (256, 13, 13, 884_992)}
for name, cout, cin_prev in [("conv3", 384, 256), ("conv4", 384, 384), ("conv5", 256, 384)]:
    h, w = conv_out(h, w, 3, 3, 1, 1)
    p = cout * cin_prev * 3 * 3 + cout
    assert (cout, h, w, p) == expected[name]
    print(f"{name}: {cout}x{h}x{w}, params={p}")

# pool3: 3x3, stride 2
h, w = pool_out(h, w, 3, 2)
assert (h, w) == (6, 6)
print(f"pool3: {256}x{h}x{w}")

# FC rétegek
fc_expected = {"fc1": 37_752_832, "fc2": 16_781_312, "fc3": 4_097_000}
for name, cout, cin_prev in [("fc1", 4096, 256*h*w), ("fc2", 4096, 4096), ("fc3", 1000, 4096)]:
    p = cout * cin_prev + cout
    assert p == fc_expected[name]
    print(f"{name}: {cout}, params={p}")

assert 34_944 + 614_656 + 885_120 + 1_327_488 + 884_992 + 37_752_832 + 16_781_312 + 4_097_000 ==
↪  62_378_344
print("minden assert lefutott")
```

A futtatási parancs: `python lab/n066/alexnet_table.py`. A fenti táblázat képletekből levezetett értékeit a szkript `assert`-ekkel ellenőrzi. A rögzített környezetű ellenőrző futás kimenete (Python 3.12.3, CPU, 1 szál; a szkript determinisztikus, seed nélkül):

```
conv1: 96x55x55, params=34944
pool1: 96x27x27
conv2: 256x27x27, params=614656
pool2: 256x13x13
conv3: 384x13x13, params=885120
conv4: 384x13x13, params=1327488
conv5: 256x13x13, params=884992
pool3: 256x6x6
fc1: 4096, params=37752832
fc2: 4096, params=16781312
```

```
fc3: 1000, params=4097000
minden assert lefutott
```

A kimenet minden értéke egyezik a fenti táblázattal; a szkriptben nincs véletlen szám, ezért a futás ismételhető.

A labor második része egy **kis léptékű kísérleti terv** egyetlen elem megváltoztatására. Kérdés: hogyan hat a dropout-arány a TinyAlexNet-Glyph validációs pontosságára? A terv az alábbi.

- **Adat:** GlyphLab, a rögzített train/val/test splittel; a split egysége a `scene_id`.
- **Modell:** egy kis AlexNet-szerű háló 24×24-es bemenethez: conv(32, 5×5, s2, p2), pool(2×2, s2), conv(64, 3×3, s1, p1), pool(2×2, s2), global average, fc(3). A konvolúciós rétegek ReLU-t használnak.
- **Változó:** dropout-arány a global average pooling utáni 64-dimenziós vektorra alkalmazva, az egyetlen fc(3) réteg előtt: $p \in \{0;\ 0{,}3;\ 0{,}6\}$ (a felsorolás határolója pontosvessző, mert a vessző a tizedesjel).
- **Rögzített:** Adam, lr 10^{-3}, batch 32, 15 epoch, adatkiegészítés nélkül, seedkészlet $\{20261020, 20261021, 20261022\}$ (mindhárom beállításra ugyanaz).
- **Metrika:** validációs pontosság; a legjobb validációs checkpointot a tesztkészleten egyszer értékeljük ki.
- **Döntési szabály:** a három dropout-arányt ugyanazon a három seeden, párosítva hasonlítjuk össze; a seedenkénti különbségek átlagát és 95%-os konfidenciaintervallumát (scene-szintű bootstrap) jelentjük (23.3. Páros összehasonlítás és csoportszintű intervallum), és csak akkor tekintjük az eltérést iránymutatónak, ha a CI nem tartalmazza a nullát; a 3 seed kis minta.
- **Erőforrás:** kis helyi CPU; minden beállítás 3 seeddel fut.
- **Korlát:** a GlyphLab nem ImageNet; a dropout hatás itt nem általánosítható a P04 kontextusára.

Ez a terv nem elvégzett futás, hanem protokolljavaslat; a hallgató az F5 feladatban bővíti.

17.2.10 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a top-k pontosságot és a top-k hibát. Miért használta az ILSVRC-2012 verseny a top-5 hibát, és miben tér el a top-1 hibától? Adjon egy feltételt, amely mellett a kettő egybeesik.

F2 (levezetési, nehézség 2). A rétegtábla alapján vezesse le a conv2 réteg kimeneti térbeli méretét és paraméterszámát. Bemenet: $27 \times 27 \times 96$, kernel 5×5, stride 1, padding 2, kimeneti csatorna 256. Ellenőrizze az NCHW alakokat.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt, amely adott logit mátrixból és int64 címkékből kiszámolja a top-k hibát. Tesztelje a $Z = \begin{pmatrix} 1{,}2 & 0{,}3 & 2{,}5 \\ 0{,}1 & 1{,}8 & 0{,}9 \end{pmatrix}$, $y = (2, 0)$, $k = 2$ bemeneten. Adja meg a várt kimenetet és a numerikus kontrollt (a kézzel kiszámolt értékkel való egyezést rögzített, például 10^{-12}-es abszolút toleranciával, valamint a kimenet alakjának és típusának ellenőrzését).

F4 (hibakeresési, nehézség 2). Valaki a következő függvényt írta, de a visszatérési érték ellentmond a függvény nevének:

```python
import numpy as np

def topk_error_bad(logits, labels, k):
    p = np.exp(logits - logits.max(axis=1, keepdims=True))
```

```
    p /= p.sum(axis=1, keepdims=True)
    top = np.argsort(-p, axis=1)[:, :k]
    hit = np.any(top == labels[:, None], axis=1)
    return hit.mean()
```

Mi a hiba, mi a következménye egy összehasonlító kísérletben, és hogyan javítható? Ellenőrizze a javítást egy $N = 3$-as példán, ahol a pontosság $2/3$.

F5 (kísérlettervezési, nehézség 3). Tervezzen kis CPU-s kísérletet a GlyphLab adaton, amely a ReLU és a tanh aktiváció hatását hasonlítja össze a TinyAlexNet-Glyph hálózatban. Adja meg a kérdést, a változó tényezőt, a rögzített beállításokat, a metrikát, a seedeket, a választási protokollt és a következtetési korlátot. A tervet ne futtassa le, de a futási parancsokat írja le. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

17.2.11 Megoldások és indoklás

F1 megoldás. A top-k pontosság azon minták aránya, amelyeknél a valódi címke a legnagyobb k predikált valószínűség indexei között szerepel; a top-k hiba ennek komplementere. Az ILSVRC-2012 verseny 1000 osztályt használt, amelyek között sok közeli kategória van, ezért a top-1 túl szigorú; a top-5 azt méri, hogy a modell a helyes osztályt legalább a legjobb öt javaslat között tartalmazza-e. A top-1 és top-5 hiba akkor esik egybe, ha minden egyes mintára a valódi címke vagy az első helyen áll, vagy egyáltalán nem szerepel az első öt között; a köztes, 2–5. helyezésű találatok ilyenkor hiányoznak.

F2 megoldás. A bemeneti térbeli méret 27×27, kernel 5×5, stride 1, padding 2, dilation 1. A kimeneti méret:

$$H_{out} = \left\lfloor \frac{27 + 2 \cdot 2 - (5 - 1) - 1}{1} \right\rfloor + 1 = \lfloor 27 + 4 - 4 - 1 \rfloor + 1 = 27,$$

és ugyanígy $W_{out} = 27$. A súly alakja $256 \times 96 \times 5 \times 5$, mert a bemeneti csatornák száma 96 (ez a didaktikai `groups=1` változat). Az eredeti, `groups=2` hálóban ugyanez a réteg $256 \times 48 \times 5 \times 5$ alakú, $256 \cdot 48 \cdot 25 + 256 = 307\,456$ paraméterrel; a feladat a táblázat `groups=1` változatára vonatkozik. A súlyparaméterek száma $256 \cdot 96 \cdot 25 = 614\,400$, a bias további 256. Összesen $614\,656$. Alakellenőrzés: a kimeneti tenzor alakja $N \times 256 \times 27 \times 27$; a paraméterszám független az N batchmérettől.

F3 megoldás. A függvény:

```
import numpy as np

def topk_error(logits, labels, k):
    # logits: (N, K) float32; labels: (N,) int64
    p = np.exp(logits - logits.max(axis=1, keepdims=True))
    p = p / p.sum(axis=1, keepdims=True)
    top = np.argsort(-p, axis=1)[:, :k]
    hit = np.any(top == labels[:, None], axis=1)
    return 1.0 - hit.mean()

Z = np.array([[1.2, 0.3, 2.5],
              [0.1, 1.8, 0.9]], dtype=np.float32)
y = np.array([2, 0], dtype=np.int64)
print(topk_error(Z, y, k=2))  # várt: 0.5
```

A várt kimenet 0,5. Az első sorban a legnagyobb két index $[2, 0]$, tartalmazza a 2-t; a második sorban $[1, 2]$, nem tartalmazza a 0-t. A softmax a max levonásával számolva numerikusan stabil; a `-p` negálás miatt az `argsort` csökkenő sorrendet ad. A `float32` kerekítés itt nem változtatja a sorrendet, mert a logitok eltérése nagyobb, mint a gépi pontosság.

F4 megoldás. A hibás függvény `hit.mean()`-t ad vissza, ami a top-k **pontosság**, nem a hiba. Ha a hívó ezt hibaként ábrázolja, akkor a nagyobb értéket jobb modellnek értelmezi, ami éppen fordított következtetés. Javítás: a visszatérési érték legyen `1.0 - hit.mean()`, vagy nevezzük át a függvényt `topk_acc`-nek. Ellenőrzés $N = 3$-mal: ha 2 minta találat, akkor a jó pontosság $2/3 \approx 0{,}667$, a hiba $1/3 \approx 0{,}333$; a hibás kód 0,667-et ad, a javított 0,333-at. A javított függvény az F3 példán is 0,5-öt ad, mert $1 - 0{,}5 = 0{,}5$.

F5 megoldás. A kísérleti terv:

- **Kérdés:** a ReLU vagy a tanh aktiváció ad-e jobb validációs pontosságot a TinyAlexNet-Glyph hálózatban?
- **Változó tényező:** a konvolúciós rétegek aktivációja; ReLU (baseline) vagy tanh. A kimeneti lineáris réteg aktiváció nélkül marad.
- **Rögzített:** GlyphLab train/val/test split, batch 32, Adam lr 10^{-3}, 15 epoch, dropout 0, seedek 20261020, 20261021, 20261022.
- **Metrika:** validációs pontosság; a tesztet csak a legjobb validációs checkpointon értékeljük ki.
- **Választási protokoll:** minden aktivációra mind a három seeddel lefuttatjuk a tanítást; a validációs pontosság átlagát és szórását jelentjük; a két aktivációt ugyanazon seedeken párosítva hasonlítjuk össze, és a páros különbségek 95%-os konfidenciaintervallumát is megadjuk (23.3. Páros összehasonlítás és csoportszintű intervallum). A döntési szabály előre rögzített: ha a páros különbségek 95%-os CI-je nem tartalmazza a nullát, az eltérést iránymutatónak tekintjük; ha tartalmazza, nem állítunk kimutatható különbséget. A tesztkiértékelés a validáción legjobbnak bizonyult beállítással történik, de a 3 seed miatt az összehasonlítás csak iránymutató.
- **Futási parancs:** `python lab/n066/train_ablation.py --act relu` és `--act tanh`. A szkript a terv szerinti, jelen fejezetben nem részletezett laborfájl; interfésze: `--act {relu,tanh}`, a fenti TinyAlexNet-Glyph modellt építi fel, a GlyphLab train/val spliten tanít a rögzített seedekkel, a legjobb validációs checkpointot menti, és a validációs pontosságot írja ki. A terv nem elvégzett futás, ezért a parancs kimenetére itt nem hivatkozunk.
- **Korlát:** a GlyphLab 24×24-es szintetikus képei nem ImageNet; a P04 absztraktban említett gyorsabb tanulás nem ugyanaz, mint a validációs pontosság ezen a kis adaton. Negatív eredmény is értelmes: elképzelhető, hogy a kis hálóban nincs mérhető különbség.

17.2.12 Gyakori tévedés és ellenpélda

A leggyakoribb tévedés a történeti top-5 számot mai modellek azonos körülményű összehasonlításának tekinteni. A 15,3% az ILSVRC-2012 tesztkészletére, a szerzők hét CNN-ből álló, beadott együttesére és a verseny kiértékelési protokolljára vonatkozik. Ma sem az adat, sem a tesztkörnyezet, sem a modellimplementációk nem azonosak. Ellenpéldaként: ha valaki egy modern, előtanított modellt az ImageNet-2012 validációs készletének egy mai újraosztott részén értékel, akkor az eredmény nem vethető össze a P04 15,3%-ával pusztán azért, mert mindkettőt százalékban mérik. Az összehasonlítás csak akkor értelmes, ha a split, az előfeldolgozás, az értékelési metrika és a modellváltozat is azonos.

17.2.13 Önellenőrzés és visszahívás

Önellenőrzési kérdés: a rétegtábla első konvolúciós rétegének kimeneti mérete 55×55; miért nem 54×54? Mert a képletbe $H = 227$ kerül, nem 224. Ha a 224-es bemenetet használnánk, akkor paddingre lenne szükség, vagy a bemeneti méretet kellene megváltoztatni. Visszahívás a 16.2. Pooling, downsampling és aliasing egységből: a pooling downsamplinget hajt végre, és a 3×3, stride 2 ablak a 55-ből 27-et ad, mert $\lfloor (55-3)/2 \rfloor + 1 = 27$; a **fázisfüggőség** itt azt jelenti, hogy az eredmény függ attól, hogy a leskálázó ablakok hogyan illeszkednek a bemenet mintavételi rácsához, azaz a rácshoz viszonyított eltolástól (fázistól).

17.2.14 Rövid összefoglaló

Az AlexNet a P04 tanulmányban bemutatott mély konvolúciós hálózat, amely az ILSVRC-2012 versenyen hét CNN átlagolt predikciójával 15,3%-os top-5 teszthibát ért el. A szám empirikus állítás, nem matematikai tétel; az adathoz, a modellváltozathoz és a top-5 metrikához kötött. A hálózat öt konvolúciós és három teljesen összekötött rétegből áll; a rétegtábla levezetése a 227×227-es bemenettel konzisztens. A ReLU, a dropout, az adatkiegészítés (augmentation) és a GPU-implementáció a tanítás gyorsaságát és a túltanulás csökkentését szolgálta. A top-k pontosság definíciója a softmax utáni legnagyobb k indexen alapul, és a k megváltoztatása más metrikát ad.

17.2.15 Következő alkalmazás

A 17.3. VGG és kis kernelek kompozíciója egység a VGG hálózatot tárgyalja: ott a kis 3×3-as kernelek kompozíciója váltja fel a nagy kerneleket, és a mélység növelése a paraméterszám tudatosabb szabályozásával társul.

17.2.16 Felhasznált források

- [P04] Krizhevsky, A., Sutskever, I. és Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems 25. A hivatkozott helyek: 3.1–3.2 (3. o.), 3.5 (4. o.), 4.1 (5. o.), 4.2 (6. o.), a NeurIPS-proceedings PDF alapján. https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf. Hozzáférés: 2026-10-30.

17.3 VGG és kis kernelek kompozíciója

17.3.1 Cél, előismeret és induló kérdés

Ez az egység a VGG egyik legismertebb tervezési döntését, a kis 3×3-as kernelek egymás utáni alkalmazását vizsgálja. A cél annak pontos megértése, hogy a mélység növelése hogyan kapcsolódik a recepciós mezőhöz, a paraméterszámhoz és a nemlinearitáshoz. Az előismeret a 17.2. AlexNet és a kísérleti kontextus történeti kontextusa: az AlexNet nagy kerneleket használt, a VGG ezt cserélte kisebbekre. Szükség van a 16.3. Elméleti és effektív recepciós mező recepciósmező-rekurziójára és a 13.3. Lokális szűrők és tanult reprezentáció lineáris/affin megkülönböztetésére. A kísérleti állításokhoz visszaidézzük az 5.3. Mintavételi bizonytalanság és intervallum mintavételi bizonytalanságról tanultakat: egyetlen futtatás eredménye nem általános tétel.

Induló kérdés: ha egy 10×10-es bemeneten egy 5×5-ös, stride 1, padding nélküli réteg kimenete 6×6, és két egymás utáni 3×3-as réteg kimenete szintén 6×6, akkor a két blokk ugyanazt a bemeneti tartományt látja? És melyik használ kevesebb paramétert, ha a csatornaszám mindenhol C?

17.3.2 Definíció: kernelkompozíció és recepciós mező

Legyen egy 2D cross-correlation réteg súlya $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$, biasza $b \in \mathbb{R}^{C_{\text{out}}}$. A programozói főútban (a PyTorch `Conv2d` konvenciója szerint) a réteg keresztkorrelációt számol, nem matematikai konvolúciót: a kernel nem fordul meg, az indexezés a 13.2. Konvolúció és cross-correlation konvencióját követi.

Definíció (kernelstack). Két vagy több konvolúciós réteg kompozícióját *kernelstacknek* nevezzük, ha a rétegek kimenetei között elemenkénti nemlineáris aktiváció, itt ReLU $\sigma(z) = \max(0, z)$ áll. Az l-edik réteg kimenete $H_l = \sigma(K_l \star H_{l-1} + b_l)$, ahol $\star$ a cross-correlation (a $*$ a matematikai konvolúciót jelöli, ahogy a 13.2. Konvolúció és cross-correlation rögzíti), $H_0 = X$ a bemenet.

Definíció (elméleti recepciós mező, RF). A 16.3. Elméleti és effektív recepciós mező rekurziójával: $J_0 = 1$, $F_0 = 1$, és

$$J_l = J_{l-1} s_l, \qquad F_l = F_{l-1} + (k_l - 1) d_l J_{l-1}.$$

Itt J_l a kimeneti rácson mért lépésköz (jump), F_l a bemeneti rácson mért mező oldalhossza, s_l a stride, k_l a kernelméret, d_l a dilation. A padding nem változtatja sem F_l-et, sem a paraméterszámot.

17.3.3 Lemma: két 3×3 stack és egy 5×5 réteg összehasonlítása

Lemma (kis kernelek kompozíciója). Tekintsünk két egymás utáni 3×3-as cross-correlation réteget stride $s = 1$, dilation $d = 1$ és tetszőleges padding mellett; mindkét réteg kimenetére tegyünk ReLU-t (a második után is, a blokk definíciója szerint). Legyen mindkét réteg bemeneti és kimeneti csatornaszáma C. Ekkor

1. a stack elméleti recepciós mezője 5×5, azonos egyetlen 5×5-ös réteggel;
2. a stack paraméterszáma bias-szal $P_{\text{stack}} = 18C^2 + 2C$, míg egyetlen 5×5-ös, C-ből C-be képező rétegé $P_5 = 25C^2 + C$;
3. $P_5 - P_{\text{stack}} = 7C^2 - C > 0$ minden $C \geq 1$-re.

Bizonyítás. Az (1) ponthoz a rekurziót használjuk. Az első rétegen $J_0 = 1$, $F_0 = 1$, $k_1 = 3$, $d_1 = 1$, így

$$F_1 = 1 + (3 - 1) \cdot 1 \cdot 1 = 3, \qquad J_1 = 1 \cdot 1 = 1.$$

A második rétegen $F_2 = 3 + (3 - 1) \cdot 1 \cdot 1 = 5$, $J_2 = 1$. Egyetlen 5×5-ös rétegen $F_1 = 1 + (5 - 1) \cdot 1 \cdot 1 = 5$, tehát a mező megegyezik.

A (2) ponthoz a súlytenzor méretét szorozzuk össze és adjuk hozzá a biaszt. Egy $k \times k$-s, $C_{\text{out}} \times C_{\text{in}}$ csatornás réteg paraméterszáma $C_{\text{out}}(C_{\text{in}} k^2 + 1)$, mert minden kimeneti csatornához $C_{\text{in}} k^2$ súly és egy bias tartozik. A stack első rétege $C(C \cdot 3^2 + 1) = 9C^2 + C$, a második ugyanennyi, így

$$P_{\text{stack}} = 2(9C^2 + C) = 18C^2 + 2C.$$

Az 5×5-ös rétegre $P_5 = C(25C + 1) = 25C^2 + C$. A különbség

$$P_5 - P_{\text{stack}} = (25C^2 + C) - (18C^2 + 2C) = 7C^2 - C = C(7C - 1),$$

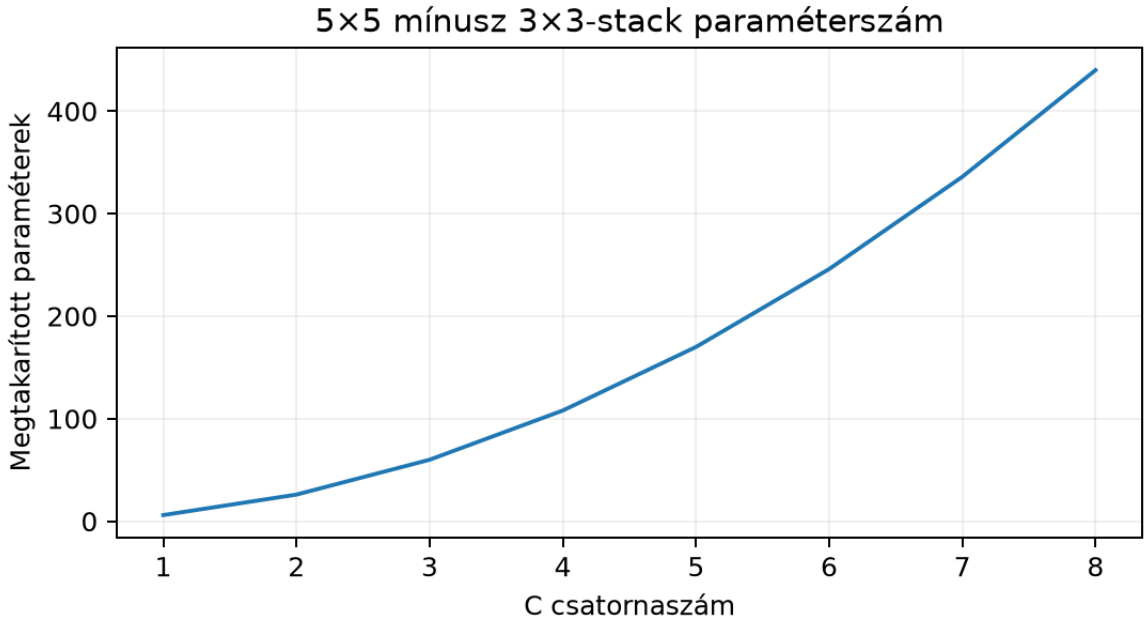
ami $C \geq 1$ esetén pozitív, mert $7C - 1 \geq 6 > 0$. □

A lemma feltétele fontos: az első és második réteg csatornaszáma is C. Ha a bemenet C_{in}, a középső C_{mid}, a kimenet C_{out}, akkor

$$P_{\text{stack}} = C_{\text{in}}C_{\text{mid}}9 + C_{\text{mid}} + C_{\text{mid}}C_{\text{out}}9 + C_{\text{out}}, \qquad P_5 = C_{\text{in}}C_{\text{out}}25 + C_{\text{out}},$$

és az összehasonlítás már nem független a csatornaválasztástól. A VGG gyakorlatában a csatornaszámok fokozatosan nőnek, ezért a megtakarítás mértéke rétegenként eltérő. Bias nélkül a stack akkor olcsóbb, ha $9C_{\text{mid}}(C_{\text{in}}+C_{\text{out}}) < 25C_{\text{in}}C_{\text{out}}$; azonos csatornaszámnál ez $18C^2 < 25C^2$, ami teljesül. Ellenpélda: $C_{\text{in}} = 64$, $C_{\text{mid}} = C_{\text{out}} = 512$ esetén a stack $9{\cdot}512{\cdot}(64{+}512) = 2\,654\,208$, az 5×5-ös réteg $25 \cdot 64 \cdot 512 = 819\,200$, tehát itt a stack drágább. A 3×3-as választás tehát nem önmagában, hanem a csatornaszámok arányától függően csökkenti a paraméterszámot.

A paraméterkülönbséget szemlélteti a következő ábra:



17.1. ábra: 5×5 mínusz 3×3-stack paraméterszám

Képletből számolt szemléltetés, nem mérés.

17.3.4 Kézi számpélda és ellenőrző kód

Legyen $C = 4$ minden rétegben, stride 1, padding 0, bemenet 1 csatorna helyett 4 csatorna, így a lemma feltétele teljesül. A bemeneti kép 10×10. A kimeneti méret padding nélkül:

- két 3×3: az első után $(10 - 3 + 1) = 8$, a második után $(8 - 3 + 1) = 6$;
- egy 5×5: $(10 - 5 + 1) = 6$.

A térbeli kimenet egyezik, a recepciós mező mindkettőnél 5×5. A paraméterszám:

$$P_{\text{stack}} = 18 \cdot 16 + 2 \cdot 4 = 288 + 8 = 296, \qquad P_5 = 25 \cdot 16 + 4 = 404.$$

A stack tehát $404 - 296 = 108$ paramétert takarít meg, miközben ugyanazt a bemeneti tartományt fedi le, és egy extra ReLU-t is tartalmaz. Ezt a számítást a következő rövid Python-kóddal ellenőrizhetjük:

```
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device('cpu')
dtype = torch.float32

X = torch.randn(2, 4, 10, 10, dtype=dtype, device=device)

stack = nn.Sequential(
    nn.Conv2d(4, 4, kernel_size=3, padding=0),
    nn.ReLU(),
    nn.Conv2d(4, 4, kernel_size=3, padding=0),
)
single = nn.Conv2d(4, 4, kernel_size=5, padding=0)

def n_params(m):
    return sum(p.numel() for p in m.parameters() if p.requires_grad)

print("stack params:", n_params(stack))   # várt: 296
print("single params:", n_params(single)) # várt: 404

with torch.no_grad():
    y_stack = stack(X)
    y_single = single(X)
print("stack output shape:", y_stack.shape)   # várt: torch.Size([2, 4, 6, 6])
print("single output shape:", y_single.shape) # várt: torch.Size([2, 4, 6, 6])
```

A kód kimenete itt nem rögzített futtatásból származik; a kommentben megadott értékek a levezetett paraméterszámok és a konvolúciós alakképlet alapján elvárt ellenőrzések. A kód futtatása után ezeknek az értékeknek kell megjelenniük.

17.3.5 Labor: két blokk azonos inputon

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab_vgg_stack.py` futtatható (`python lab_vgg_stack.py`); a bemenet a szakaszban generált tenzor, külső adatfájl nem kell.

A labor célja a fenti két blokk összehasonlítása azonos bemeneten, dokumentált paraméterszámmal (104 és 188). A főút kis CPU-n fut, nincs GPU, nincs előtanított súly.

Input: saját generált $X \in \mathbb{R}^{1\times1\times9\times9}$, float32, értékei a $[-1, 1]$ intervallumból; a seed `20261020`. A 1 csatorna a szürkeárnyalatos GlyphLab sémát (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) idézi, de ez a labor nem használja a GlyphLab fájlokat, és nem tanít modellt.

Blokkok:

- **A blokk:** egyetlen 5×5-ös réteg, $C_{\text{in}} = 1$, $C_{\text{out}} = 4$, padding 2; kimeneti alak $1 \times 4 \times 9 \times 9$. Paraméterszám:
$$P_A = 4(1 \cdot 25 + 1) = 104.$$

- **B blokk:** 3×3-as réteg $1 \to 4$, ReLU, majd 3×3-as réteg $4 \to 4$; mindkét padding 1, ezért a kimenet szintén $1 \times 4 \times 9 \times 9$. Paraméterszám:

$$P_B = (4(1 \cdot 9 + 1)) + (4(4 \cdot 9 + 1)) = 40 + 148 = 188.$$

A labor tehát nem az azonos csatornás lemma feltételét használja, mert az első réteg a bemeneti 1 csatornát 4-re bővíti. A cél a shape-egyezés, a recepciós mező és a kapacitáskülönbség ellenőrzése, nem a paraméterazonosság.

Kód és futtatási parancs:

```
# fajl: lab_vgg_stack.py, futtatas: python lab_vgg_stack.py
import torch
import torch.nn as nn

torch.manual_seed(20261020)

X = torch.rand(1, 1, 9, 9, dtype=torch.float32) * 2 - 1  # [-1,1]

blockA = nn.Conv2d(1, 4, kernel_size=5, padding=2)
blockB = nn.Sequential(
    nn.Conv2d(1, 4, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(4, 4, kernel_size=3, padding=1),
)

def n_params(m):
    return sum(p.numel() for p in m.parameters())

with torch.no_grad():
    yA = blockA(X)
    yB = blockB(X)

assert n_params(blockA) == 104
assert n_params(blockB) == 188
assert tuple(yA.shape) == (1, 4, 9, 9)
assert tuple(yB.shape) == (1, 4, 9, 9)

def rf_from_spec(spec):
    F = J = 1
    for k, s in spec:
        F = F + (k - 1) * J
        J = J * s
    return F

assert rf_from_spec([(5, 1)]) == 5
assert rf_from_spec([(3, 1), (3, 1)]) == 5
print("RF A:", rf_from_spec([(5, 1)]), "RF B:", rf_from_spec([(3, 1), (3, 1)]))

print("A params:", n_params(blockA))  # elvart: 104
print("B params:", n_params(blockB))  # elvart: 188
print("A shape:", tuple(yA.shape))    # elvart: (1, 4, 9, 9)
print("B shape:", tuple(yB.shape))    # elvart: (1, 4, 9, 9)
print("abs diff max:", (yA - yB).abs().max().item()) # veletlenszeru, nem 0
```

A rögzített környezetű ellenőrző futás kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
RF A: 5 RF B: 5
A params: 104
B params: 188
A shape: (1, 4, 9, 9)
```

```
B shape: (1, 4, 9, 9)
abs diff max: 1.0791290998458862
```

Ellenőrzés és eredményértelmezés. A futtatás `assert`-jei a két paraméterszámot (104 és 188) és a két kimeneti alakot ($1 \times 4 \times 9 \times 9$) ellenőrzik. A shape-ek egyezése azt igazolja, hogy a padding úgy lett beállítva, hogy mindkét blokk ugyanazt a térbeli felbontást adja. Az elméleti RF mindkét blokkra 5×5: a B blokk első rétege 3, a második 5 mezőt lát. A különbség a paraméterekben van, és a B blokk egy extra ReLU-t is tartalmaz. A labor nem végez tanítást, így a kimeneti tenzorok különbsége véletlenszerű; ott *nem* várunk zérust. Az `abs diff max` konkrét értéke a seedtől és a PyTorch-verziótól függ, a nem nulla volta a lényeg.

17.3.6 VGG: a kis kernelek használatának kontextusa

A [P05] Simonyan–Zisserman-tanulmány absztraktja nagyon mély, 16–19 súlyozott rétegű konvolúciós hálózatokat mutat be; a szerzők a hálózat nevében a *VGG* rövidítést használják. A [P05] szerint az architektúra főként 3×3-as konvolúciókat és 2×2-es max-poolingot alkalmaz, a csatornaszámot 64-ről duplázva 512-ig növeli; a C konfiguráció azonban 1×1-es konvolúciókat is tartalmaz, ezért a „kizárólag 3×3” állítás túl erős. A korábbi [P04] AlexNet ezzel szemben 11×11-es és 5×5-ös kerneleket is használt, tehát a VGG a kernelméret csökkentésével egyszerűsítette a mintázatot.

A VGG-ben a 3×3-as stack előnye nemcsak a paraméterszám. Ugyanazon 5×5-ös mezőn belül két ReLU áll rendelkezésre, azaz a blokk kimenete egy mélyebb, darabonként lineáris függvény. Ez heurisztikus érv: a két ReLU két töréspontot visz be, ezért a reprezentált függvény gazdagabb lehet, mint egyetlen lineáris szűrő utáni egy ReLU, de ez nem tétel. A paraméterszám-megtakarítás azonban a leglátványosabb a konvolúciós részekben: a lemma szerint azonos C mellett a stack olcsóbb.

A VGG másik tanulsága az **osztályozófej költsége** (classifier cost). A 224×224-es bemenetet feltételezve a hálózat végén a térbeli felbontás 7×7-re csökken, a csatornaszám 512, így az első teljesen összekötött (fully connected, FC) réteg bemenete $512 \cdot 7 \cdot 7 = 25\,088$ elem. Ha az első FC réteg 4096 neuront tartalmaz, akkor annak súlymátrixa önmagában $25\,088 \cdot 4096 \approx 102{,}8$ millió paramétert ad, ami a hálózat paramétereinek döntő többsége. Ez a költség motiválta a későbbi architektúrákban a globális átlag-poolingot, de ez már a 19.2. MobileNet és inverted residual és későbbi egységek témája. Itt a VGG kapcsán annyit rögzítünk: a konvolúciós rétegekben a kis kernelek csökkentik a paraméterszámot, a teljes paraméterköltség viszont a FC fejben összpontosul.

17.3.7 Gyakori tévedés: „lineárisan ekvivalens”

Gyakran hallani, hogy „két 3×3-as réteg ekvivalens egy 5×5-ös réteggel”. Ez csak a recepciós mezőre igaz. A paraméterszámra sem igaz, ahogy a lemmában láttuk, és főleg az *outputra* nem igaz, ha a rétegek között ReLU áll.

Ellenpélda. Legyen $X \in \mathbb{R}^{1\times1\times3\times3}$ egy egycsatornás kép, amelynek középső pixelértéke $a \in \mathbb{R}$, minden más eleme 0. Az első 3×3-as kernel K_1 középső súlya legyen 1, a többi 0, padding 1, bias 0. Ekkor az első réteg kimenetének középső pozíciójában az érték a, a többi pozíción 0. A ReLU után a középső érték $\max(a, 0)$. A második 3×3-as kernel K_2 minden súlya 1, padding 1, bias 0. A második kimenet középső pozíciója a bemenő kép középső 3×3-as ablakának összege; ott egyetlen nemnulla érték a $\max(a, 0)$, tehát a kimenet $F(a) = \max(a, 0)$.

Ha létezne olyan 5×5-ös lineáris kernel, amely ezzel a stackkel minden bemenetre megegyezik, akkor a fenti inputcsaládon a kimenet wa alakú lenne valamilyen $w \in \mathbb{R}$-rel. Ekkor $a = 1$-re

$w = 1$, $a = -1$-re pedig $-w = 0$ lenne, ami ellentmondás. A stack tehát nem lineáris leképezés, így nem lehet egyetlen lineáris konvolúcióval azonos.

Ha a ReLU-t elhagyjuk, akkor a bias nélküli kompozíció valóban lineáris, és a bemenetre egyetlen effektív kernelként hat; de a VGG-blokk éppen a ReLU beiktatásával válik mély nemlineáris reprezentációvá.

17.3.8 Önellenőrzés és korábbi fogalmak visszahívása

Ellenőrizzük a lemmát $C = 1$-re: a stack paraméterszáma $18 + 2 = 20$, az 5×5-ösé $25 + 1 = 26$, a különbség 6, ami a $7C^2 - C$ képletből $7 - 1 = 6$. A recepciós mező mindkettőnél 5. A kimeneti méret padding nélkül 10×10-es bemeneten 6×6 mindkét blokkra. A kernelstack definíciójában a ReLU a 13.3. Lokális szűrők és tanult reprezentáció affin linearitási ellenpéldájához hasonlóan töri meg a homogenitást: ott a biasos művelet tette affinná a leképezést, itt a ReLU teszi darabonként lineárissá. A 16.3. Elméleti és effektív recepciós mező rekurzióban a padding valóban nem szerepel a mezőképletben, csak a stride és a dilation. Végül az 5.3. Mintavételi bizonytalanság és intervallum tanulsága szerint ha a labor két blokkjának kimenetét összehasonlítjuk, az eltérés egyetlen seedből származó véletlen érték, nem statisztikai állítás a két architektúra minőségéről.

Az induló kérdésre a válasz: a két egymás utáni 3×3-as (stride 1) réteg ugyanazt az 5×5-ös bemeneti tartományt látja, mint egyetlen 5×5-ös réteg, és azonos C csatornaszám mellett a stack kevesebb paramétert használ ($18C^2 + 2C$ szemben a $25C^2 + C$-vel). Az egyezés azonban csak a recepciós mezőre és a kimeneti méretre vonatkozik, a reprezentált függvényre nem, mert a ReLU nemlinearitás miatt a stack nem redukálható egyetlen lineáris rétegre.

17.3.9 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a recepciós mező és a paraméterszám között, és miért nem elegendő a kettő egyezése ahhoz, hogy két blokkot ekvivalensnek nevezzünk. Térjen ki a ReLU szerepére. Elvárt eredmény: rövid, önálló szöveg, amely a két fogalmat definiálja, és az ellenpélda gondolatát saját szavakkal megismétli.

F2 (levezetési, nehézség 1). Adott $C = 8$ csatorna, stride 1, dilation 1, bias minden rétegben. Számítsa ki három egymás utáni 3×3-as stack és egyetlen 7×7-es réteg elméleti recepciós mezőjét és paraméterszámát. Ellenőrizze, hogy a megtakarítás a két 3×3 vs. egy 5×5 esetén kapott $7C^2 - C$ képlet mintájára milyen képletet ad a három 3×3 vs. egy 7×7 összevetésre. Adja meg a kimeneti méretet is egy 12×12-es bemenetre padding nélkül. Elvárt eredmény: a kapott számok, a rekurzió lépései és az alakellenőrzés.

F3 (implementációs, nehézség 2). Írjon Python-függvényt, amely bemenetként megkapja a kernelek listáját `k_list`, a stride-ok listáját `s_list`, a dilation listáját `d_list`, a csatornaszámot `C` (minden réteg be- és kimenete), valamint egy `bias` logikai értéket. A függvény adja vissza az elméleti recepciós mezőt és a teljes paraméterszámot. Tesztelje a `k_list=[3,3]`, `s_list=[1,1]`, `d_list=[1,1]`, `C=4`, `bias=True` esetre; az eredmény RF 5, paraméter 296 legyen. Elvárt eredmény: futtatható kód, a teszt kimenete és a függvény forrásszövege.

F4 (hibakeresési, nehézség 2). Az alábbi modell állítólag a VGG-style 3×3-as stacket valósítja meg:

```
class Stack(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 1, 3, padding=1)
        self.conv2 = nn.Conv2d(1, 1, 3, padding=1)
```

```
    def forward(self, x):
        return self.conv2(self.conv1(x))
```

Lokalizálja a hibát, adja meg a következményt, javítsa a `forward` metódust, és adjon egy numerikus ellenőrzést, amely két ellentétes előjelű bemeneten mutatja, hogy a javított modell nemlineáris. Elvárt eredmény: a hibás rész megnevezése, a javított kód, és a teszt kimenetének leírása.

F5 (kísérleti tervezés/transzfer, nehézség 3). Tervezzen kísérletet a GlyphLab adaton arra a kérdésre, hogy egy 3×3-as stack jobb-e, mint egyetlen 5×5-ös réteg az orientációklasszifikációban. Adja meg a megfigyelési egységet, a spliteket, a baseline-t, a változó tényezőt, a metrikát, a seed- és checkpoint-kezelést, valamint az érvényességi korlátot. A válasz nem kérhet előre megadott győztest. Elvárt eredmény: protokollterv, a lehetséges alternatív értelmezések felsorolása. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

17.3.10 Megoldások és indoklás

F1 megoldás. A recepciós mező az a bemeneti halmaz, amelytől egy kimeneti neuron értéke elvileg függhet; hosszát a 16.3. Elméleti és effektív recepciós mező rekurzió adja. A paraméterszám a modell tanulható skalárjainak száma: $C_{\text{out}}(C_{\text{in}}k^2 + 1)$ rétegenként bias-szal. Az egyező RF azt mondja, hogy ugyanazt a bemeneti tartományt olvassák, de a függvény alakja eltérhet. A ReLU megtöri a homogenitást: $F(-x) \neq -F(x)$ általában, míg egy lineáris leképezésre mindig $L(-x) = -L(x)$. A szakasz ellenpéldájában a középső pixel a értékére a stack $F(a) = \max(a, 0)$-t ad, ami nem lineáris; tehát az RF-egyezés nem ekvivalencia.

F2 megoldás. Három 3×3-as rétegre a rekurzió:

$$F_1 = 1 + 2 = 3, \qquad F_2 = 3 + 2 = 5, \qquad F_3 = 5 + 2 = 7.$$

A stride 1 miatt $J_l = 1$ minden l-re, így az RF 7×7. Egyetlen 7×7-es réteg RF-je $1 + (7 - 1) = 7$, tehát egyezik. A paraméterszám:

$$P_{\text{stack}} = 3(9C^2 + C) = 27C^2 + 3C, \qquad P_7 = C(49C + 1) = 49C^2 + C.$$

$C = 8$-ra:

$$P_{\text{stack}} = 27 \cdot 64 + 24 = 1752, \qquad P_7 = 49 \cdot 64 + 8 = 3144,$$

a megtakarítás $3144 - 1752 = 1392$. A különbség általános képlete $22C^2 - 2C$, ami $C \geq 1$-re pozitív. A kimeneti méret 12×12-es bemeneten padding nélkül: három 3×3-as után $12 - 2 - 2 - 2 = 6$, azaz 6×6; egyetlen 7×7-es után $12 - 7 + 1 = 6$, azaz ugyanaz. Alakellenőrzés: a kernelstack minden közbülső kimenete C csatornás, így a paraméterképlet konzisztens.

F3 megoldás.

```
def stack_rf_params(k_list, s_list, d_list, C, bias=True):
    if not (len(k_list) == len(s_list) == len(d_list)):
        raise ValueError("a k_list, s_list és d_list hossza egyezzen meg")
    J = 1
    F = 1
    total = 0
    for k, s, d in zip(k_list, s_list, d_list):
        F = F + (k - 1) * d * J
        J = J * s
        total += C * (C * k * k + (1 if bias else 0))
```

```
    return F, total

rf, params = stack_rf_params([3, 3], [1, 1], [1, 1], 4, True)
print(rf, params)   # 5 296
```

Ellenőrzés: $F_1 = 1+2 = 3$, $F_2 = 3+2 = 5$; paraméter mindkét rétegre $4(4 \cdot 9+1) = 148$, összesen 296. A függvény szélső esete: ha `k_list` üres, $F = 1$, `total=0`, ami a bemeneti réteg RF-jét és paraméterszámát adja.

F4 megoldás. A hiba a `forward` metódusban van: hiányzik a ReLU a két konvolúció között. Következmény: a modell affin, tehát két affin konvolúció kompozíciójaként egyetlen affin konvolúcióvá (effektív kernel és effektív bias) redukálható, és nem VGG-style stack. A javítás:

```
def forward(self, x):
    return self.conv2(torch.relu(self.conv1(x)))
```

Numerikus ellenőrzés: generáljunk két $1 \times 1 \times 3 \times 3$-as tenzort, az egyik középső pixel 1, a másik középső pixel -1, a többi 0. A javított modell kimenetének középső pozíciójában a bemeneti 1-re pozitív, -1-re nulla adódik, míg a lineáris kompozícióra az értékek egymás negatívjai. Az $F_{\text{lin}}(-1) = -F_{\text{lin}}(1)$ azonosság csak bias nélkül (vagy nulla bias mellett) érvényes, ezért a tesztben a biasokat nullára állítjuk. A futtatható ellenőrzés:

```
import torch
import torch.nn as nn

class Stack(nn.Module):
    def __init__(self, use_relu=True):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 1, 3, padding=1, bias=False)
        self.conv2 = nn.Conv2d(1, 1, 3, padding=1, bias=False)
        self.use_relu = use_relu

    def forward(self, x):
        h = self.conv1(x)
        if self.use_relu:
            h = torch.relu(h)
        return self.conv2(h)

def center_value(model, a):
    x = torch.zeros(1, 1, 3, 3)
    x[0, 0, 1, 1] = a
    return model(x)[0, 0, 1, 1].item()

for m in (Stack(True), Stack(False)):
    with torch.no_grad():
        m.conv1.weight.zero_()
        m.conv1.weight[0, 0, 1, 1] = 1.0
        m.conv2.weight.fill_(1.0)
    print(m.use_relu, center_value(m, 1.0), center_value(m, -1.0))
```

A mért kimenet (torch 2.15.0+cpu): `True 1.0 0.0` és `False 1.0 -1.0`; a ReLU-s modellre tehát $F(1) \neq -F(-1)$, ami igazolja a nemlinearitást, a ReLU nélküli, nulla biasú modellre pedig $F(-1) = -F(1)$.

F5 megoldásminta. A protokoll a következő lehet. Az adat a GlyphLab, amelynek szerződését a függelék A. szakasza (40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) rögzíti: 24×24-es szürkeárnyalatos képek, jelenetenként két nézet, `metadata.csv` a `scene_id`, `view_id`, `label` és `split` oszlopokkal. A megfigyelési egység a `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik; a split a `metadata.csv` szerinti train/val/test, osztályonkénti jelenetszám 40/10/10. A baseline egyetlen 5×5-ös konvolúciós blokk: Conv($1 \to 8$, 5×5, padding 2), ReLU, max-pooling

(2×2); a 24×24-es bemenetből a pooling után $8 \cdot 12 \cdot 12 = 1152$ jellemző marad, amelyet egy FC fej képez le a 3 osztályra ($1152 \cdot 3 + 3 = 3459$ paraméter). A változó tényező a konvolúciós blokk szerkezete: a kísérleti modell Conv(1→C, 3×3, padding 1), ReLU, Conv(C→8, 3×3, padding 1), ReLU, MaxPool, azonos FC fejjel. A `C` értéket úgy választjuk meg, hogy a konvolúciós blokk paraméterszáma a lehető legközelebb legyen a baseline $1 \cdot 8 \cdot 25 + 8 = 208$-hoz; a $82C + 8$ képletből $C = 2$ esetén $1 \cdot 2 \cdot 9 + 2 + 2 \cdot 8 \cdot 9 + 8 = 20 + 152 = 172$, ami 36-tal kevesebb a baseline-nál, míg $C = 3$ esetén 254 lenne, 46-tal több. A választott $C = 2$ mellett a paraméterkülönbséget a riportban expliciten közöljük. A tanítás CPU-n, float32, Adam optimalizálóval (lr 10^{-3}, batch 32), kereszt-entrópiával történik; az optimalizálót és a tanulási rátát a terv előre rögzíti, nem a teszt eredménye alapján választjuk. Minden futásnál a seed rögzített, és a kísérletet 5–10 seeddel ismételjük; a modellválasztás a legjobb validációs veszteség (val loss) alapján történik, a tesztet csak a végén értékeljük. A kiválasztott modell checkpointjának és az adatfájloknak az SHA-256 lenyomatát a jegyzőkönyv rögzíti. A corrupt teszt a `data/corrupted_test.npy`: zajjal és fényességgradienssel rontott tesztképek, a clean teszttel azonos jelenet–nézet párokban; külön, párosítva értékeljük, és nem használjuk modellválasztásra. A metrika a teszt accuracy; a seedek közötti szórást is jelentjük. Érvényességi korlát: a GlyphLab szintetikus, a paraméterszám nem pontosan egyezik, és az egyetlen adathalmazon mért különbség nem általánosítható természetes képekre. Az F5 tehát tervet és nem garantált eredményt ad; negatív eredmény is értékes.

17.3.11 Rövid összefoglaló

A két egymás utáni 3×3-as, stride 1 konvolúció elméleti recepciós mezője 5×5, pontosan akkora, mint egyetlen 5×5-ösé. Azonos C bemeneti és kimeneti csatornaszám mellett a stack paraméterszáma $18C^2 + 2C$, az 5×5-ösé $25C^2 + C$, tehát a mélyebb változat kevesebb súlyt használ. A megtakarítás azonban nem jelenti a függvények lineáris ekvivalenciáját, mert a közbeiktatott ReLU nemlinearitást visz be; ezt egy egycsatornás ellenpélda igazolja. A VGG ezt az elvet használja, és a paraméterköltség jelentős része a teljesen összekötött fejben összpontosul. A labor két blokk forwardját és dokumentált paraméterszámát ellenőrzi.

17.3.12 Következő alkalmazás

A 17.4. Architektúra összehasonlítása kontrollált keretben egység architektúrákat hasonlít össze kontrollált keretben: a VGG, AlexNet és LeNet közötti különbségeket nem csupán leíró módon, hanem azonos adaton, azonos kereszt-entrópiával, rögzített seedekkel és dokumentált paraméterszámmal méri. Az itt levezetett paraméter- és RF-számítás ott alapellenőrzésként szolgál.

17.3.13 Felhasznált források

- [P04] Krizhevsky, A., Sutskever, I. és Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems 25. https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.
- [P05] Simonyan, K. és Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv:1409.1556 (v6, 2015-04-10). https://arxiv.org/abs/1409.1556.

17.4 Architektúra összehasonlítása kontrollált keretben

17.4.1 Cél, előismeret és induló kérdés

A 17.3. VGG és kis kernelek kompozíciója egységben a 3×3-as stack és az 5×5-ös kernel recepciós mezőjét és paraméterszámát hasonlítottuk össze. Ott az volt a kérdés, hogy *mit tud* egy-egy rétegsorozat. Itt az következik, hogy *mikor állíthatunk* róluk valamit mért adatok alapján: két architektúra összehasonlítása csak akkor érdemi, ha a kísérleti keret izolálja az architektúra hatását a tanítási recept, az adat, a modellválasztási szabály és a véletlen hatásától. Előismeret: a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetben rögzített operátorspecifikáció, a 13.4. Határkezelés és frekvenciaintuíció alfejezetben tárgyalt padding, valamint az 5.4. Függő képek, seedek és resampling alfejezetből a seed-ismétlés és a megfigyelési egység fogalma. Induló kérdés: ha az A modellt három seeddel, húsz epochig, a B modellt tíz seeddel, ötven epochig tanítjuk, és B jobb lesz, mit tudunk az architektúráról?

17.4.2 Fair comparison, baseline és a kontrollváltozók

Definíció (fair comparison). Legyen D rögzített adathalmaz, M rögzített metrika, T a vizsgált tényező (például az architektúra), V pedig az összes többi ismert befolyásoló tényező együttese: adatfelosztás, preprocessing, loss, optimalizáló, learning rate, epoch- vagy early-stopping-szabály, batchméret, seed-készlet és a hiperparaméter-keresés protokollja. A T_1 és T_2 beállítás összevetése fair D-n, ha (i) V mindkét karra azonos szabály szerint rögzített; (ii) a modellválasztás a validációs halmazon, a végső M kiértékelés pedig a teszt halmazon, a választás után történik; (iii) az erőforrás-költségek közül annak kell egyeznie, amelyet a kutatási kérdés kontrollálni akar. A **baseline** egy előre rögzített referencia-beállítás, amelyhez képest a változtatás hatását mérjük; baseline nélkül a „jobb” szó nem rendelhető méréshez.

A definíció szándékosan nem írja elő, hogy *mindig minden* költség egyezzen. Az, hogy mit kell egyeztetni, a kérdésből következik. Például ha a kérdés az, hogy adott paraméterköltség mellett a mélység vagy a kernelszélesség hasznos-e, akkor a paraméterszám egyezzen; ha az, hogy adott epochszám mellett melyik tanul gyorsabban, akkor a számítási keret egyezzen; ha az, hogy melyik recept a jobb, akkor az architektúra legyen közös, és a recept változzon.

17.4.3 Állítás: mely költségnek kell egyeznie?

Jelölje $Y(A, R, D, S)$ a mért teljesítményt, ahol A az architektúra, R a tanítási recept, D az adat és S a seed-ek halmaza. Az alábbi állításban a kontrollkészletet a kutatási kérdésből vezetjük le.

Állítás (kontrollkészlet a kérdésből). Legyen D és az S-en vett átlagolási szabály rögzített. Ekkor:

1. Ha a kérdés az A hatása, akkor a $Y(A_1, R_1, D, S) - Y(A_2, R_2, D, S)$ különbség az $R_1 = R_2$ esetben azonosítható A-hatásként (a modellválasztási szabály azonossága mellett); $R_1 \neq R_2$ esetén a különbség nem bontható egyértelműen A-hatásra és recepthatásra.
2. Ha a kérdés „azonos paraméterkeret mellett melyik struktúra jobb?”, akkor a $\Theta(A)$ paraméterszámokra $|\Theta(A_1) - \Theta(A_2)| / \max(\Theta(A_1), \Theta(A_2)) \leq \varepsilon$ szükséges előre rögzített ε mellett.

3. Ha a kérdés „azonos képzési számítás mellett”, akkor a szorzás–összeadás műveletek (MAC) számának kell egyeznie; itt a MAC egy minta egy forward-menetének költsége, ezért egy epochhoz a mintaszámmal (és a backward költségével) kell felszorozni; ha „azonos csúcsmemória mellett”, akkor a csúcs-aktivációmemóriának; ha a kérdés a receptérzékenység (ugyanazon architektúra mellett a recept változtatásának hatása), akkor $A_1 = A_2$-nek kell teljesülnie.

Indoklás. Az (1) állítás egy azonosságból következik. Írjuk fel a mért különbséget:

$$Y(A_1, R_1, D, S) - Y(A_2, R_2, D, S) = [Y(A_1, R_1, D, S) - Y(A_2, R_1, D, S)] + [Y(A_2, R_1, D, S) - Y(A_2, R_2, D, S)].$$

Az első tag a recept rögzítése melletti architektúra-különbség, a második a recept hatása rögzített A_2 mellett. Ha $R_1 \neq R_2$, akkor a második tag nem feltétlenül nulla, tehát a teljes különbség nem azonosítható az első taggal. Mivel a két tag egyetlen mért különbségből nem szeparálható, a tisztán A-ra vonatkozó következtetéshez $R_1 = R_2$ szükséges.

A (2)–(3) állításokban a kontrollálás hiányának következményét egy szemléltető példán mutatjuk meg: eltérő költség mellett a sorrend megfordulhat. Legyen most T a struktúratényező (az A szerepében), Θ a paraméterkeret (a V egy komponense), és rögzítsük D-t és S-t; a példa kedvéért tegyük fel, hogy a várható teljesítmény $Y(T, \Theta) = T\Theta$ alakú (ez szándékosan leegyszerűsített szemléltetés, nem valós modell). Az $(T_1, \Theta_1) = (2, 1)$ és $(T_2, \Theta_2) = (1, 3)$ beállításokra $Y_1 = 2 < 3 = Y_2$, tehát a nyers összehasonlítás T_2-t hozná ki jobbnak. Rögzített $\Theta = 1$ mellett viszont $T_1 > T_2$. A Θ tehát konfundáló (zavaró) tényező; az izolált T-hatáshoz a Θ-t kontrollálni kell. Ugyanez az érvelés szó szerint átvihető a MAC- vagy memóriakeretre. □

Az állítás azt mondja ki, hogy *melyik* költségnek kell egyeznie, nem azt, hogy bármely egyezés garantálja a helyes következtetést. Az egyezés szükséges feltétel; a véletlen bizonytalanságot emellett seed-ismétléssel kell becsülni.

17.4.4 Kézi példa: MiniDeep3 és MiniWide5 paraméterkerete

A laborhoz két kis adaptációt definiálunk a GlyphLab 24×24-es, egycsatornás képeire (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték). Ezek nem az eredeti LeNet [P01], AlexNet [P04] vagy VGG [P05] másolatai, hanem a 17.3. VGG és kis kernelek kompozíciója alfejezetben vizsgált 3×3-stack kontra 5×5-kernel elv dokumentált mini-adaptációi.

MiniDeep3. A features blokk: Conv(1,8,3,pad=1), ReLU, Conv(8,8,3,pad=1), ReLU, MaxPool(2); Conv(8,16,3,pad=1), ReLU, Conv(16,16,3,pad=1), ReLU, MaxPool(2). A fej: AdaptiveAvgPool2d(1), Flatten, Linear(16,3). A bias-tagokkal együtt a paraméterszám:

$$\Theta_{\mathrm{D3}} = (1{\cdot}8{\cdot}9{+}8){+}(8{\cdot}8{\cdot}9{+}8){+}(8{\cdot}16{\cdot}9{+}16){+}(16{\cdot}16{\cdot}9{+}16){+}(16{\cdot}3{+}3) = 80{+}584{+}1168{+}2320{+}51 = 4203.$$

MiniWide5. Conv(1,10,5,pad=2), ReLU, MaxPool(2); Conv(10,16,5,pad=2), ReLU, MaxPool(2); ugyanaz a fej. A paraméterszám:

$$\Theta_{\mathrm{W5}} = (1 \cdot 10 \cdot 25 + 10) + (10 \cdot 16 \cdot 25 + 16) + (16 \cdot 3 + 3) = 260 + 4016 + 51 = 4327.$$

A relatív eltérés $124/4327 \approx 2{,}9\%$, tehát a két modell az $\varepsilon = 0{,}05$ paraméterkeret-tűrésen belül van. A MAC-ok ezzel szemben eltérnek: a MiniDeep3 négy konvolúciója $8 \cdot 1 \cdot 9 \cdot 24^2 + 8 \cdot 8 \cdot 9 \cdot 24^2 + 16 \cdot 8 \cdot 9 \cdot 12^2 + 16 \cdot 16 \cdot 9 \cdot 12^2 = 870\,912$ MAC, a MiniWide5 két konvolúciója $10 \cdot 1 \cdot 25 \cdot 24^2 + 16 \cdot 10 \cdot 25 \cdot 12^2 = 720\,000$ MAC. Ezért az alábbi labor kutatási kérdése *paraméterkeretre* vonatkozik, és a MAC-eltérést dokumentált másodlagos tényezőként, nem kontrollálatlan zavaróként kezeljük.

17.4.5 Labor: kontrollált kétmodelles összevetés GlyphLabon

Előfeltétel. Python 3.12.3, NumPy 2.5.4, pandas 3.1.0 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy`, `metadata.csv` és `corrupted_test.npy` fájlja; a `train_compare.py` futtatható (`python train_compare.py`).

Protokoll. Kérdés: a GlyphLab orientáció-feladatán, közel azonos paraméterszám mellett, a 3×3-as mély stack vagy az 5×5-ös széles, sekély változat ér el jobb általánosítást? Hipotézis: a mély stack a 17.3. VGG és kis kernelek kompozíciója alfejezetben tárgyalt nemlinearitás-többlet miatt nem rosszabb; ez azonban nem előre garantált győzelem. Adat: `data/images.npy` (alakja $(360, 1, 24, 24)$, float32), `data/labels.npy`, `data/metadata.csv`, `data/corrupted_test.npy`; a split a metadata `split` oszlopából jön, a `scene_id` a jelenet azonosítója, a `view_id` a jeleneten belüli nézeté. Megfigyelési egység: `scene_id`; a split jelenetszintű (120/30/30 jelenet, azaz 240/60/60 kép), tehát egy jelenet két nézete nem kerül különböző splitbe. Baseline: a MiniWide5 (széles, sekély változat), a kísérleti modell a MiniDeep3. Változó tényező: architektúra (MiniDeep3 vagy MiniWide5). Rögzített tényezők: AdamW lr 10^{-3}, weight decay 10^{-4}, batch 32, legfeljebb 20 epoch, early stopping 5 türelmi epochhal a validációs lossra, loss CrossEntropyLoss logitsból, CPU float32, seed-készlet $\{0, 1, 2\}$ mindkét modellre. Modellválasztás: a három seed átlagos validációs lossza alapján választjuk ki a győztest; a tesztet csak ezután, mindkét modellre kiértékeljük, de a fő következtetést a győztesre fogalmazzuk. A corrupt teszt páros a clean testtel: ugyanaz a 60 kép (30 jelenet, jelenetenként két nézet) sorrendje, ezért a két accuracy különbsége páros összehasonlítás. A három seed kis minta: a seedek közötti szórást és a páros, jelenetszintű bootstrap 95%-os konfidenciaintervallumát is jelentjük (23.3. Páros összehasonlítás és csoportszintű intervallum), és a győztest csak a bizonytalanság közlésével hirdetjük ki.

Szkript. Munkakönyvtár: `lab/n068/`. A kód a GlyphLab fájljait a `../data/` alól olvassa. A szkript a paraméterszámokat és az adat alakját `assert`-ekkel ellenőrzi; a futtatás eredményét a kód után közöljük.

```
# lab/n068/train_compare.py
import json, argparse
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

def set_seed(seed):
    torch.manual_seed(seed); np.random.seed(seed)

class MiniDeep3(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1,8,3,padding=1), nn.ReLU(),
            nn.Conv2d(8,8,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
```

```
            nn.Conv2d(8,16,3,padding=1), nn.ReLU(),
            nn.Conv2d(16,16,3,padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(16,3))
    def forward(self, x):
        return self.head(self.features(x))

class MiniWide5(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1,10,5,padding=2), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(10,16,5,padding=2), nn.ReLU(), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(16,3))
    def forward(self, x):
        return self.head(self.features(x))

def param_count(model):
    return sum(p.numel() for p in model.parameters())

def evaluate(model, loader):
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for x, y in loader:
            pred = model(x).argmax(dim=1)
            correct += (pred == y).sum().item(); total += y.numel()
    return correct / total

def fit(model, train_loader, val_loader, max_epochs=20, patience=5):
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    loss_fn = nn.CrossEntropyLoss()
    best_loss, best_state, waited = float("inf"), None, 0
    for epoch in range(max_epochs):
        model.train()
        for x, y in train_loader:
            opt.zero_grad(); loss = loss_fn(model(x), y); loss.backward(); opt.step()
        model.eval(); val_loss = 0.0; count = 0
        with torch.no_grad():
            for x, y in val_loader:
                val_loss += loss_fn(model(x), y).item() * y.numel(); count += y.numel()
        val_loss /= count
        if val_loss < best_loss - 1e-6:
            best_loss, best_state, waited = val_loss, {k: v.clone() for k, v in
↪   model.state_dict().items()}, 0
        else:
            waited += 1
            if waited >= patience:
                break
    model.load_state_dict(best_state)
    return best_loss

def main(seeds, out_path):
    data_dir = "../data"
    X = np.load(f"{data_dir}/images.npy").astype(np.float32)
    y = np.load(f"{data_dir}/labels.npy").astype(np.int64)
    corrupt = np.load(f"{data_dir}/corrupted_test.npy").astype(np.float32)
    meta = pd.read_csv(f"{data_dir}/metadata.csv")
    assert list(meta.columns) == ["index", "scene_id", "view_id", "label", "split",
                                  "xmin", "ymin", "xmax", "ymax"], meta.columns
    assert len(meta) == len(X) == len(y) == 360, (len(meta), X.shape, y.shape)
    assert X.shape[1:] == (1, 24, 24), X.shape
    assert corrupt.shape[1:] == (1, 24, 24), corrupt.shape
    assert meta["index"].tolist() == list(range(len(meta)))
```

```
    assert param_count(MiniDeep3()) == 4203
    assert param_count(MiniWide5()) == 4327
    tr_idx = meta.loc[meta.split == "train", "index"].to_numpy()
    va_idx = meta.loc[meta.split == "val", "index"].to_numpy()
    te_idx = meta.loc[meta.split == "test", "index"].to_numpy()
    Xtr = torch.from_numpy(X[tr_idx]); ytr = torch.from_numpy(y[tr_idx])
    Xva = torch.from_numpy(X[va_idx]); yva = torch.from_numpy(y[va_idx])
    Xte = torch.from_numpy(X[te_idx]); yte = torch.from_numpy(y[te_idx])
    Xco = torch.from_numpy(corrupt)  # sorrend = metadata test sorrend
    assert len(corrupt) == len(te_idx), (corrupt.shape, len(te_idx))
    results = {}
    for name, build in [("MiniDeep3", MiniDeep3), ("MiniWide5", MiniWide5)]:
        results[name] = {"params": param_count(build()), "val_loss": [],
                         "val_acc": [], "test_acc": [], "corrupt_acc": []}
        for seed in seeds:
            set_seed(seed)
            gen = torch.Generator().manual_seed(seed)
            train_dl = DataLoader(TensorDataset(Xtr, ytr), batch_size=32, shuffle=True,
↪   generator=gen)
            val_dl = DataLoader(TensorDataset(Xva, yva), batch_size=64)
            test_dl = DataLoader(TensorDataset(Xte, yte), batch_size=64)
            corrupt_dl = DataLoader(TensorDataset(Xco, yte), batch_size=64)
            model = build()
            val_loss = fit(model, train_dl, val_dl)
            results[name]["val_loss"].append(val_loss)
            results[name]["val_acc"].append(evaluate(model, val_dl))
            results[name]["test_acc"].append(evaluate(model, test_dl))
            results[name]["corrupt_acc"].append(evaluate(model, corrupt_dl))
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(results, f, indent=2, default=float)

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--seeds", nargs="+", type=int, default=[0,1,2])
    ap.add_argument("--out", default="results_n068.json")
    args = ap.parse_args()
    main(args.seeds, args.out)
```

Futtatási parancs: `python train_compare.py --seeds 0 1 2 --out results_n068.json`.

Elvárt eredmény. A JSON mindkét modellre tartalmazza a paraméterszámot (4203 és 4327), továbbá seedenként a `val_loss`, `val_acc`, `test_acc` és `corrupt_acc` nyers listáit. A rögzített környezetű ellenőrző futás mért kimenete (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, PyTorch 2.15.0+cpu, CPU, 1 szál, seedek 0, 1, 2):


```
{
  "MiniDeep3": {
    "params": 4203,
    "val_loss": [0.8871508240699768, 0.1414535790681839, 0.346905380487442],
    "val_acc": [0.8333333333333334, 1.0, 1.0],
    "test_acc": [0.8333333333333334, 1.0, 1.0],
    "corrupt_acc": [0.6, 0.3333333333333333, 0.36666666666666664]
  },
  "MiniWide5": {
    "params": 4327,
    "val_loss": [0.4952288269996643, 0.79160475730896, 0.723465621471405],
    "val_acc": [1.0, 0.6666666666666666, 1.0],
    "test_acc": [1.0, 0.6666666666666666, 1.0],
    "corrupt_acc": [0.8333333333333334, 0.65, 0.8333333333333334]
  }
}
```

Ezen a futáson a három seed átlagos validációs lossa MiniDeep3-nál 0,4585, MiniWide5-nél 0,6701; a validációs loss alapján tehát a MiniDeep3 a győztes. A clean teszt pontosság átlaga 0,9444, illetve 0,8889, a corrupt teszté 0,4333, illetve 0,7722. A hallgató feladata a saját futásában ugyanezeket az átlagokat és a corrupt-clean páros eltérését kiszámítani.

17.4.6 Feladatok

1. (F1, fogalmi, nehézség 1). Definiálja a fair comparison fogalmát, és adjon konkrét példát arra, hogyan torzít a keresési költség eltérése. Elvárt eredmény: legfeljebb hatmondatos definíció, valamint egy olyan elképzelt eredmény, amelyben az egyik modellre tíz, a másikra száz konfigurációt kerestek.

2. (F2, levezetési, nehézség 2). Vezesse le a MiniWide5 MAC-számát a 24×24-es bemenetre, és határozza meg, hogy a paraméterkeret-egyezés mellett milyen másodlagos költség marad nyitva. Elvárt eredmény: lépésenkénti számítás shape-ekkel és a végső 720 000 érték.

3. (F3, implementációs, nehézség 2). Írjon olyan `param_count_bias` függvényt, amely egy `nn.Module` paramétereinek számát bias nélkül és bias-szal együtt adja vissza. Ellenőrizze a MiniDeep3-ra, hogy bias nélkül 4152, bias-szal 4203. Elvárt eredmény: futtatható kód és a két ellenőrző érték.

4. (F4, hibakeresési, nehézség 2). A labor egy változata a corrupt kiértékeléshez az alábbi, szándékosan hibás sort használja:

```
# hibás változat – a labor helyes kódjában ez a sor nem így szerepel
corrupt_dl = DataLoader(TensorDataset(Xco, yte), batch_size=64, shuffle=True)
```

Mi a hiba, mi a következménye, és mi a javítás ellenőrzése? Elvárt eredmény: a hiba pontos lokalizálása, a párosítás sérülésének magyarázata és a javított kódsor.

5. (F5, kísérleti/transzfer, nehézség 3). Tervezzen fair összevetést a GlyphLabon egy harmadik változat, a MiniDeep3-memory bevezetésére, amelyben a MiniDeep3 második konvolúciós blokkja (a Conv(8,16) és Conv(16,16) réteg) helyett Conv(8,24) és Conv(24,24) áll, és a fej bemenete is 24 lesz. Adja meg: kérdés, költségkontroll, baseline, seed- és választási szabály, valamint a következtetés érvényességi korlátja. Elvárt eredmény: protokollterv szövegesen, az 5.4. Függő képek, seedek és resampling alfejezetben tanult seed-adat elkülönítés alkalmazásával. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

17.4.7 Megoldások és indoklás

1. megoldás. A fair comparison olyan összevetés, amelyben a vizsgált tényezőn kívül minden ismert befolyásoló tényezőt azonos szabály rögzít, a modellválasztás validációs halmazon történik, a tesztet pedig a választás után használjuk. A keresési költség eltérésére példa: az A modellhez tíz lr-jelöltet próbálunk, a B modellhez százat, és mindkettőnél a legjobbat jelentjük. Ha B nyer, nem tudjuk, hogy az architektúra vagy a nagyobb keresési költség miatt; a teszt többszöri használata ráadásul a kiválasztott konfigurációt optimistán torzítja. A korrekt keret mindkét modellre azonos keresési rácsot és azonos választási szabályt ír elő.

2. megoldás. Az első konvolúció bemenete $N \times 1 \times 24 \times 24$, kimenete $N \times 10 \times 24 \times 24$ a `padding=2` miatt. A MAC-ok száma kimeneti elemenként $C_{in} k_h k_w = 1 \cdot 25$ (a bemeneti csatornák felett összegezve). Összesen $10 \cdot 1 \cdot 25 \cdot 24 \cdot 24 = 144\,000$. A MaxPool után a második konvolúció bemenete $N \times 10 \times 12 \times 12$, kimenete $N \times 16 \times 12 \times 12$. A MAC-ok: $16 \cdot 10 \cdot 25 \cdot 12 \cdot 12 = 576\,000$.

Összeg: 720 000. A lemmából következik, hogy mivel a kérdés paraméterkeretre vonatkozott, a MAC-különbség dokumentált másodlagos költség; ha a kérdés számítási keretre váltana, a MiniWide5 alacsonyabb MAC-költsége (720 000 a MiniDeep3 870 912-vel szemben) önmagában is magyarázhatná az eltérést, ezért a csatornaszámot újra kellene hangolni.

3. megoldás. A függvény a paraméternév `bias` végződése alapján választja szét a bias-t; ez a PyTorch névadási konvenciójára támaszkodik. A `p.ndim == 1` vizsgálata önmagában nem tévedésbiztos, mert a BatchNorm súlyai és biasai is egydimenziósak, ezért a név és a `p.ndim == 1` kombinációja az általánosabb, de a névalapú szétválasztás itt egyértelmű.

```
def param_count_bias(model):
    total = bias = 0
    for name, p in model.named_parameters():
        total += p.numel()
        if name.endswith("bias"):
            bias += p.numel()
    return total - bias, total
```

A MiniDeep3 esetében a bias-tagok: $8+8+16+16+3 = 51$ (a négy konvolúcióé $8+8+16+16 = 48$, a lineárisé 3), így az összeg 4203 és a bias nélküli rész $4203 - 51 = 4152$. A megoldás ellenőrzése `assert param_count_bias(MiniDeep3()) == (4152, 4203)`. Az ellenőrző számokat mindig érdemes levezetni, nem átvenni.

4. megoldás. A hiba a corrupt DataLoader `shuffle=True` beállításában van. A `corrupted_test.npy` sorrendje a metadata test sorainak sorrendjét követi, és a hozzá tartozó `yte` ugyanebben a sorrendben van. A keverés a képeket és a címkéket ugyan együtt keveri, tehát az accuracy önmagában nem romlik el; a probléma a clean testtel való *páros* összehasonlításban van: a keverés miatt a corrupt és a clean eredmények nem ugyanahhoz a képhez rendelődnek, azaz a párosítás szakad el, és a képszintű páros különbség (valamint annak szórása) már nem ugyanazon képpárokra vonatkozik. A javítás: `DataLoader(TensorDataset(Xco, yte), batch_size=64)` keverés nélkül (a `shuffle` alapértelmezése `False`, ezért a paraméter elhagyható); a clean test DataLoader is keverés nélkül fut. Az ellenőrzés: egy rögzített batch első képének corrupt és clean változata ugyanahhoz a metadata indexhez tartozzon.

5. megoldás. A MiniDeep3-memory bevezetésekor a kérdés az, hogy a második konvolúciós blokk csatornaszámának növelése javítja-e az általánosítást a paraméterkeret rögzítése mellett. A költségkontrollhoz előbb ki kell számítani az új paraméterszámot: a Conv(8,24,3) és Conv(24,24,3) blokk paraméterei $8 \cdot 24 \cdot 9 + 24 + 24 \cdot 24 \cdot 9 + 24 = 1752 + 5208 = 6960$, a korábbi középső blokk $1168+2320 = 3488$, tehát a blokk növekménye 3472. Ehhez jön a fej változása: a Linear bemenete 16-ról 24-re nő, ami $24 \cdot 3 + 3 - (16 \cdot 3 + 3) = 24$ további paramétert ad, így az összparaméterszám $4203 + 3472 + 24 = 7699$, a növekmény 3496. Ez a régi keret 83,2%-a (az új keret 45,4%-a), ami messze meghaladja az állításban szereplő $\varepsilon = 0{,}05$ paraméterkeret-tűrést; a „paraméterkeret rögzítése melletti” kérdés tehát ebben a formában nem teljesül, és a megoldásnak ezt ki kell mondania. A költségkontrollhoz csökkenteni kell például az első blokk csatornaszámát, vagy a kérdést explicite „adott memória-keretre” kell átfogalmazni. A baseline a MiniDeep3; a seed-ismétlés ugyanazt a $\{0, 1, 2\}$ készletet használja mindkét modellre, így a seed-hatás párosítható. Az 5.4. Függő képek, seedek és resampling szerinti seed-adat elkülönítés itt azt jelenti, hogy az adat-előkészítés (a rögzített split és az esetleges augmentáció) seedje közös és rögzített, a tanítási seedek pedig csak az inicializálást és a batch-sorrendet vezérlik, az adatmintát nem változtatják; a seed-készlet terjedelme így az algoritmus bizonytalanságát, nem az adatminta bizonytalanságát becsli. A választás a három seed átlagos validációs lossza; a tesztet csak ezután értékeljük. Korlát: a GlyphLab eredmény nem általánosítható természetes képekre, és a jelenetszintű függés miatt képszintű konfidenciaintervallumot nem szabad jelenet-függetlenként értelmezni.

17.4.8 Gyakori tévedés és ellenpélda

A leggyakoribb hiba, hogy a mérési különbséget automatikusan az architektúrának tulajdonítjuk, noha közben a tanítási recept is változott. Ellenpélda (a számok illusztratívak, nem mért eredmények): az A modellt lr 10^{-3}-mal, a B modellt lr 10^{-2}-mal tanítva A validációs pontossága 0,85, B-é 0,80. Ha ebből azt állítjuk, hogy A architektúrája jobb, az állítás nem következik az adatból. Egy további futtatás B-t lr 10^{-3}-mal tanítva 0,88-at adhat; a recept megváltoztatása megfordítja a sorrendet. A helyes keret vagy rögzíti az lr-t, vagy mindkét modellre ugyanazt a keresési protokollt futtatja, és a választott lr-t a validációs halmazon választja, a tesztet pedig a választás után használja. A kevés seedre (például egyre vagy kettőre) alapozott összehasonlítás hasonlóan félrevezető: egyetlen seed két modellje közötti különbség lehet pusztán a tanítási zaj, és a labor három seedje is csak korlátozottan becsli ezt a zajt; az 5.4. Függő képek, seedek és resampling alfejezetben tárgyalt módon a seed-ismétlés az algoritmus bizonytalanságát becsli.

17.4.9 Önellenőrzés és ismétlés

Ellenőrző kérdések: (1) Kijelenthetem-e, hogy a labor győztese jobb architektúra, ha a paraméterszámok 4203 és 4327? Igen, ha a protokoll a paraméterkeretet rögzítette, de a MAC-eltérést jelenteni kell mint korlátot. (2) Használtam-e a tesztet a modellválasztás előtt? A szkript nem; a `test_dl` és `corrupt_dl` csak a `fit` után, a legjobb checkpoint betöltése után fut. (3) A corrupt–clean különbség páros? Igen, mert a corrupt sorrendje megegyezik a metadata test sorrenddel, és a DataLoaderek nem kevernek. Ismétlés a 13.4. Határkezelés és frekvenciaintuíció alfejezetből: a `padding=2` az 5×5-ös kernelnél, a `padding=1` a 3×3-asnál őrzi meg a térbeli méretet. A 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetből: a `Conv2d` keresztkorrelációt számol; a paraméterszám szempontjából a kernel elemszáma $C_{in}k_hk_w$, függetlenül attól, hogy a műveletet konvolúciónak vagy keresztkorrelációnak nevezzük.

17.4.10 Rövid összefoglaló

A fair comparison olyan kísérleti keret, amelyben a vizsgált tényezőn kívül minden ismert befolyásoló tényezőt rögzítünk, a modellt validációs halmazon választjuk, a tesztet pedig a választás után használjuk. Az állítás szerint az egyeztetendő költséget a kutatási kérdés határozza meg: paraméterkerethez paraméterszám, számítási kerethez MAC, memóriakerethez aktivációcsúcs, receptérzékenységhez közös architektúra kell. A MiniDeep3 és MiniWide5 paraméterszáma 4203 és 4327, tehát közel azonos keretbe esik; a labor ugyanazt a receptet, seed-készletet és választási szabályt alkalmazza. Az eltérő keresési költség vagy kevés seed nem izolálja az architektúra hatását.

17.4.11 Következő alkalmazás

A 18.1. Reziduális blokk és identitásút a reziduális blokkot vezeti be. Az ottani $y = F(x) + x$ összefüggéshez tartozó ablációs vizsgálathoz ugyanez a kontrollkeret szükséges: az identitásút elhagyása és megtartása között csak akkor mérhető a blokk hatása, ha a paraméterszám-, MAC- és seed-protokollt előre rögzítjük.

17.4.12 Felhasznált források

- [P01] LeCun, Y., Bottou, L., Bengio, Y. és Haffner, P. (1998). *Gradient-based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11). https://proceedingsofheieee.ieee.org/gradient-based-learning-applied-to-document-recognition/.
- [P04] Krizhevsky, A., Sutskever, I. és Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems 25. https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.
- [P05] Simonyan, K. és Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv:1409.1556 (v6, 2015-04-10). https://arxiv.org/abs/1409.1556.

18 Reziduális tanulás és normalizálás

18.1 Reziduális blokk és identitásút

18.1.1 Cél, előismeret és induló kérdés

Ez az alfejezet a reziduális blokkot, a mély hálózatok egyik legfontosabb szerkezeti elemét vezeti be. Az előismeret a 11.4. Egy teljes PyTorch tanítási iteráció alfejezetből a teljes tanítási lépés, a 14.4. Csoportos és depthwise konvolúció alfejezetből a csatornák közti kapcsolat korlátozása, a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetből a CNN-operátor specifikációja, a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből pedig a kontrollált architektúra-összehasonlítás fogalma. A korábbi 17.1. LeNet és a lokális megosztott reprezentáció megmutatta, hogy a rétegek egymásra építése önmagában nem feltétlenül hoz jobb eredményt: a mélység növelése új optimalizálási és numerikus nehézségeket teremt. He, Zhang, Ren és Sun P06 3.1–3.2 szakasza a reziduális tanulást olyan blokkal vezeti be, amelyben a háló nem közvetlenül a kívánt leképezést, hanem annak egy eltérését tanulja egy identitásút mellett.

Induló kérdés: ha egy blokk kimenete $y = x + F(x)$, akkor mit tudunk mondani a blokk Jacobi-mátrixáról, és miért nem következik ebből, hogy minden reziduális háló jól kondicionált?

18.1.2 Definíció és intuíció

Legyen a blokk bemenete x. A reziduális blokk kimenete

$$y = x + F(x; \theta),$$

ahol F a *reziduális leképezés*: általában két vagy három konvolúciós réteg, amelyet nemlinearitás és esetleg normalizálás követ, θ pedig a tanulandó paraméterek összessége. Az x-et közvetlenül a kimenethez vezető utat *identity shortcut*nak (magyarul identitásútnak, a továbbiakban röviden shortcutnak) nevezzük. Az intuíció az, hogy ha a kívánt blokk-kimenet közel van a bemenethez, akkor a hálónak elegendő $F(x) \approx 0$-t tanulnia; a shortcut így egy előre adott, paramétermentes alapértelmezett leképezést biztosít.

A tenzoralakú változatban $x \in \mathbb{R}^{C_{in} \times H \times W}$, $F(x) \in \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}$. Az összeadás csak akkor értelmes, ha

$$C_{in} = C_{out}, \qquad H = H_{out}, \qquad W = W_{out}.$$

Ha a blokk csatornát vált, vagy $s > 1$ stride-dal csökkenti a térbeli méretet, akkor a shortcutot is transzformálni kell. A *projection shortcut* egy 1×1-es konvolúcióval,

$$h(x) = W_{\mathrm{sc}} x,$$

ahol $W_{\text{sc}} \in \mathbb{R}^{C_{out} \times C_{in} \times 1 \times 1}$ a shortcut projekciós súlya (a sc alsó index a shortcutra utal, megkülönböztetve a stride s-jétől) és a stride megegyezik a főág stride-jával, így a kimenet

$$y = F(x; \theta) + W_s x.$$

A W_s projekció tehát nem egyszerű dimenziótoldás, hanem tanulható lineáris leképezés, amely a shortcut út és a főág alakját összehangolja a 14.1. Stride, padding, dilation és outputméret alfejezetben levezetett kimeneti méret szerint.

A P06 megkülönbözteti a *basic* blokkot és a *bottleneck* blokkot. A basic blokk két 3×3-as konvolúcióból áll, változatlan csatornaszámmal. A bottleneck blokk három rétegből épül fel: egy 1×1-es konvolúció csökkenti a csatornákat C-ről d_b-re, egy 3×3-as konvolúció a szűkebb d_b csatornán dolgozik, majd egy második 1×1-es konvolúció visszaállítja a C csatornát (a d_b a bottleneck belső szélessége, megkülönböztetve a vektor dimenziójának d jelölésétől). A tipikus $C = 256$, $d_b = 64$ választás mellett a paraméterszám bias nélkül:

Réteg	Csatorna	Kernel	Paraméter (bias nélkül)
1. konvolúció	$256 \to 64$	1×1	$256 \cdot 64 = 16\,384$
2. konvolúció	$64 \to 64$	3×3	$64 \cdot 64 \cdot 9 = 36\,864$
3. konvolúció	$64 \to 256$	1×1	$64 \cdot 256 = 16\,384$

Összesen 69 632 paraméter. Két közvetlen 3×3-as konvolúció 256 csatornán ezzel szemben $2 \cdot 256 \cdot 256 \cdot 9 = 1\,179\,648$ paramétert igényelne, tehát a bottleneck körülbelül tizenhetedére csökkenti a paraméterszámot, miközben a bemeneti és kimeneti csatornaszám változatlan marad.

18.1.3 A lokális Jacobi-alak levezetése

A központi állítást absztrakt vektoros formában mondjuk ki. Legyen $x \in \mathbb{R}^d$ a blokk bemenete, $F : \mathbb{R}^d \to \mathbb{R}^d$ a reziduális leképezés, és $y = x + F(x)$.

Lemma (lokális Jacobi-alak). Ha F differenciálható az x pontban, akkor a $G(x) = x + F(x)$ leképezés Jacobi-mátrixa ugyanabban a pontban

$$J_G(x) = I_d + J_F(x),$$

ahol $I_d \in \mathbb{R}^{d \times d}$ az egységmátrix, $J_F(x) \in \mathbb{R}^{d \times d}$ pedig F Jacobi-mátrixa a könyv 4.3. Jacobi-mátrix, láncszabály és VJP konvenciója szerint: a J_F i. sora és j. oszlopa $\partial F_i / \partial x_j$.

Bizonyítás. A G leképezés i. koordinátája $G_i(x) = x_i + F_i(x)$. A parciális deriváltra minden $i, j \in \{1, \dots, d\}$ esetén

$$\frac{\partial G_i}{\partial x_j} = \frac{\partial x_i}{\partial x_j} + \frac{\partial F_i}{\partial x_j} = \delta_{ij} + \frac{\partial F_i}{\partial x_j},$$

ahol $\delta_{ij} = 1$, ha $i = j$, egyébként 0. A jobb oldal első tagja az I_d egységmátrix (i, j) eleme, a második pedig a $J_F(x)$ mátrix (i, j) eleme. Ezzel a mátrixegyenlőséget koordinátánként beláttuk.

A lemma következménye a visszaterjesztésre. A VJP (vector–Jacobian product, vektor–Jacobi-szorzat) egy vektornak a Jacobi-mátrix transzponáltjával vett szorzata; itt ez adja meg, hogyan

terjed vissza a gradiens. Ha a blokk kimenetére érkező upstream gradiens $g = \nabla_y L$, akkor a VJP-szabály szerint

$$\nabla_x L = J_G(x)^\top g = (I_d + J_F(x))^\top g = g + J_F(x)^\top g.$$

Az első tag, g, változatlanul átfolyik az identitásúton; a második tag a paraméterezett F ágon keresztül visszaterjesztett gradiens. Ez a felbontás az, amit a szakirodalom gyakran „a gradiens az identitásúton is eljut a korábbi rétegekhez” formában említ.

Miért nem garantál ez minden hálón jó kondicionáltságot? A fenti lemma lokális állítás: egyetlen x pontban írja le a blokk Jacobi-mátrixát. Egy mátrix kondíciószámát 2-es spektrális normában a $\kappa_2(M) = \sigma_{\max}(M)/\sigma_{\min}(M)$ hányados adja. Az $I_d + J_F(x)$ mátrix kondíciószáma tetszőlegesen nagy lehet, mert $J_F(x)$ valamely sajátértéke a tanulás során közel kerülhet -1-hez, azaz $I_d + J_F(x)$ valamely sajátértéke közel 0-hoz. Szélsőséges lineáris ellenpéldaként legyen $F(x) = -x$; ekkor $y = 0$, és

$$J_G(x) = I_d + (-I_d) = 0.$$

A Jacobi-mátrix szinguláris, a kondíciószám végtelen, annak ellenére, hogy a shortcut jelen van. A lineáris esetben egy rögzített A mátrixra $\kappa_2(I_d + A)$ véges, ha A sajátértékei elkerülik a -1-et; egy A-családra azonban csak akkor kapunk egyenletes korlátot, ha a sajátértékek -1-től való távolsága egyenletesen alulról korlátos. A nemlineáris F Jacobi-mátrixa pontról pontra változik, ezért egyetlen pontban számított kondíciószám nem ad globális korlátot a teljes tanítási pályára.

Mély háló esetén a láncszabály tovább nehezíti a helyzetet. Ha a háló L blokk kompozíciója, $G = G_L \circ \cdots \circ G_1$, ahol $G_k(x_k) = x_k + F_k(x_k)$, akkor

$$J_G(x) = \prod_{k=1}^{L} \left(I_d + J_{F_k}(x_k) \right),$$

ahol x_k a k. blokk bemenete. A szorzat tényezői az $(I_d + J_{F_k}(x_k))$ mátrixok. Még ha minden egyes ilyen tényező adott pontban jól kondicionált is, a szorzat viselkedése nem következik az egyes tényezők kondíciószámából: a szorzat skálája exponenciálisan csillapodhat, és a spektruma is elfajulhat. Ha például minden tényező $0{,}5 I_d$, akkor a szorzat $2^{-L} I_d$: a kondíciószáma ugyan 1, de a gradiens normája exponenciálisan csillapodik; ez skálacsillapodás, nem spektrális elfajulás. Spektrális elfajulás akkor lép fel, ha a tényezők különböző irányokat különböző mértékben csillapítanak. A shortcut tehát nem tünteti el a főág okozta csillapítást, csak hozzáad egy plusz utat. A P06 kísérleti javulást mutat, nem pedig kondicionáltsági vagy globális konvergenciatételt; az állítás típusa empirikus eredmény, nem matematikai garancia.

18.1.4 Kézi számpélda

Legyen a bemenet és a reziduális leképezés lineáris:

$$x = \begin{bmatrix} 3 \\ -1 \end{bmatrix}, \qquad F(x) = Ax + b, \qquad A = \begin{bmatrix} 0{,}5 & 0{,}2 \\ 0{,}1 & -0{,}3 \end{bmatrix}, \qquad b = \begin{bmatrix} 0{,}1 \\ 0{,}2 \end{bmatrix}.$$

Először a reziduális ág:

$$F(x) = \begin{bmatrix} 0{,}5 \cdot 3 + 0{,}2 \cdot (-1) \\ 0{,}1 \cdot 3 - 0{,}3 \cdot (-1) \end{bmatrix} + \begin{bmatrix} 0{,}1 \\ 0{,}2 \end{bmatrix} = \begin{bmatrix} 1{,}3 \\ 0{,}6 \end{bmatrix} + \begin{bmatrix} 0{,}1 \\ 0{,}2 \end{bmatrix} = \begin{bmatrix} 1{,}4 \\ 0{,}8 \end{bmatrix}.$$

A blokk kimenete

$$y = x + F(x) = \begin{bmatrix} 3 \\ -1 \end{bmatrix} + \begin{bmatrix} 1{,}4 \\ 0{,}8 \end{bmatrix} = \begin{bmatrix} 4{,}4 \\ -0{,}2 \end{bmatrix}.$$

A Jacobi-mátrix a lemma szerint

$$J_G(x) = I_2 + A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0{,}5 & 0{,}2 \\ 0{,}1 & -0{,}3 \end{bmatrix} = \begin{bmatrix} 1{,}5 & 0{,}2 \\ 0{,}1 & 0{,}7 \end{bmatrix}.$$

Ellenőrzésképpen a $g = (1, 2)^\top$ upstream gradienssel a VJP:

$$\nabla_x L = J_G(x)^\top g = \begin{bmatrix} 1{,}5 & 0{,}1 \\ 0{,}2 & 0{,}7 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1{,}7 \\ 1{,}6 \end{bmatrix}.$$

Ugyanez a felbontásból: az identitásút $g = (1, 2)^\top$-t ad, a reziduális út pedig

$$A^\top g = \begin{bmatrix} 0{,}5 & 0{,}1 \\ 0{,}2 & -0{,}3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0{,}7 \\ -0{,}4 \end{bmatrix},$$

és $(1, 2)^\top + (0{,}7;\ -0{,}4)^\top = (1{,}7;\ 1{,}6)^\top$, egyezésben a direkt számolással.

18.1.5 Labor: kis reziduális blokk

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/residual_block.py` futtatható (`python lab/residual_block.py`); külső adatfájl nem kell.

A labor a shortcut mechanizmust izolálja, ezért a BatchNormot elhagyjuk; a BN bevezetése a 18.2. Batch normalization train és eval módban feladata. Az alábbi `lab/residual_block.py` két blokkot valósít meg: azonos alakút és csatornát váltót. Az ellenőrzéshez a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetben megszokott **háromszögelvet** követjük: a kézi számolás, a független referencia és az autograd eredményének egyezését rögzített tűrés alatt ellenőrizzük. Itt a felírt $y = x + F(x)$ összeadást közvetlenül összehasonlítjuk a blokk kimenetével, ami a forward definíciójának konzisztenciáját ellenőrzi; emellett a shortcut tényleges hozzájárulását is mérjük.

Munkakönyvtár: a projekt gyökere; a parancs a `lab/residual_block.py` fájlt innen futtatja.

```
"""lab/residual_block.py"""
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32
```

```python
class BasicResBlock(nn.Module):
    """Azonos csatornaszámú, stride=1 reziduális blokk."""
    def __init__(self, C, kernel=3):
        super().__init__()
        self.conv1 = nn.Conv2d(C, C, kernel, padding=1)
        self.conv2 = nn.Conv2d(C, C, kernel, padding=1)
        self.relu = nn.ReLU()

    def forward(self, x):
        h = self.relu(self.conv1(x))
        h = self.conv2(h)
        return x + h


class ProjectionResBlock(nn.Module):
    """Csatornát váltó blokk 1x1 projekciós shortcuttal."""
    def __init__(self, C_in, C_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(C_in, C_out, 3, stride=stride, padding=1)
        self.conv2 = nn.Conv2d(C_out, C_out, 3, padding=1)
        self.shortcut = nn.Conv2d(C_in, C_out, 1, stride=stride)
        self.relu = nn.ReLU()

    def forward(self, x):
        h = self.relu(self.conv1(x))
        h = self.conv2(h)
        return h + self.shortcut(x)


def main():
    N, C, H, W = 2, 3, 24, 24
    x = torch.randn(N, C, H, W, device=device, dtype=dtype)

    # Azonos alakú blokk: a kimenet x + F(x), és a skip ténylegesen számít.
    block_same = BasicResBlock(C).to(device, dtype)
    block_same.eval()
    with torch.no_grad():
        y = block_same(x)
        h_ref = block_same.conv2(block_same.relu(block_same.conv1(x)))
        skip_contrib = y - h_ref
        gap = skip_contrib.abs().max().item()          # a skip hozzájárulása
        err_same = (skip_contrib - x).abs().max().item()
        same = torch.allclose(skip_contrib, x, atol=1e-6, rtol=0)
    print("skip hozzajarulasa max|y-h|:", gap)
    print("max|(y-h)-x|:", err_same)
    # A skip nélküli (sima) előremenet kimenete h_ref lenne; a gap > 0 azt
    # mutatja, hogy a blokk valóban hozzáadja az identity ágat. Az err_same
    # és az allclose ugyanazt az y-h = x azonosságot ellenőrzi: az előbbi
    # numerikus értéket ad, az utóbbi assertionként kötelezővé teszi.
    assert gap > 1e-3 and same

    # Csatornát váltó blokk: a shortcut 1x1 konvolúcióval projektál.
    block_proj = ProjectionResBlock(C, 6).to(device, dtype)
    block_proj.eval()
    with torch.no_grad():
        y2 = block_proj(x)
    assert tuple(y2.shape) == (2, 6, 24, 24)
    print("projekcios kimenet alakja:", tuple(y2.shape))
    print("elvart alak: (2, 6, 24, 24)")


if __name__ == "__main__":
    main()
```

Futtatás a projekt gyökeréből:

```
python lab/residual_block.py
```

A kimenet ezen a környezeten (Python 3.12.3, PyTorch 2.15.0+cpu, float32, CPU, 1 szál, seed 20261020) futtatott érték; más verziókon a float32 kerekítés miatt az utolsó jegyek eltérhetnek:

```
skip hozzajarulasa max|y-h|: 3.656097888946533
max|(y-h)-x|: 2.384185791015625e-07
projekcios kimenet alakja: (2, 6, 24, 24)
elvart alak: (2, 6, 24, 24)
```

A `max|(y-h)-x|` érték konstrukció szerint nulla, ezért ez csak a forward definíciójának konzisztenciáját ellenőrzi, önmagában nem bizonyítja a helyességet. A tényleges ellenőrzés a `max|y-h|` érték: ez a shortcut hozzájárulása, amely pozitív, tehát a blokk kimenete nem egyezik a shortcut nélküli (sima) előremenet F kimenetével, vagyis az identity shortcut valóban hozzáadódik. A második sor a csatornaváltó blokk kimeneti alakját ellenőrzi: $C_{in} = 3$-ból $C_{out} = 6$ lett, a térbeli méret 24×24 maradt, mert stride $s = 1$ és a 3×3-as kernel $p = 1$ paddingje megőrzi a méretet, összhangban a 14.1. Stride, padding, dilation és outputméret formulájával:

$$H_{out} = \left\lfloor \frac{24 + 2 \cdot 1 - 1 \cdot (3 - 1) - 1}{1} \right\rfloor + 1 = 24.$$

A projekciós shortcut itt 1×1-es konvolúció, amely a 3 csatornát 6-ra képezi le, így a főág és a shortcut alakja összeadhatóvá válik.

18.1.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja az identitás shortcutot és a projection shortcutot. Adjon egy konkrét bemeneti alakot, amelynél F kimenete csatornát vált, és mondja meg, miért nem értelmes az $x + F(x)$ összeadás projekció nélkül.

F2 (levezetési, nehézség 2). Legyen $F(x) = Ax + b$ lineáris leképezés, $A \in \mathbb{R}^{2\times 2}$. Bizonyítsa be a lemma alapján, hogy $J_G(x) = I_2 + A$, és adjon olyan konkrét A mátrixot, amelyre $\kappa_2(I_2 + A) \geq 100$. Számolja ki a sajátértékeket vagy a szinguláris értékeket.

F3 (implementációs, nehézség 2). Implementáljon `ProjectionResBlock` stílusú blokkot $C_{in} = 1$, $C_{out} = 4$, 5×5-ös kernel, $p = 2$, stride $s = 2$ mellett. Ellenőrizze a kimeneti alakot egy $(2, 1, 24, 24)$ bemeneten, és a shortcut nélküli főággal való összehasonlítással a max hibát. Adja meg a futtatható kódot, a várt shape-et és a tűrést.

F4 (hibakeresési, nehézség 2). Az alábbi kód hibás:

```python
class BrokenBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(2, 4, 3, padding=1)
        self.conv2 = nn.Conv2d(4, 4, 3, padding=1)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.conv2(self.relu(self.conv1(x))) + x
```

Lokalizálja a hibát, magyarázza el a következményét, javítsa ki projekciós shortcuttal, és ellenőrizze a javítást egy $(2, 2, 24, 24)$ bemeneten. Adja meg a hiba okát, a javított kódot és az ellenőrző futtatás eredményét.

F5 (kísérlettervezési, nehézség 3). Tervezzen kontrollált ablatív kísérletet a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adaton, amely a shortcut út hatását méri az orientációosztályozási feladatra. Az orientációosztályozás a GlyphLab adatleírás szerinti háromosztályos feladat: a címke $y \in \{0, 1, 2\}$ ($0 =$ függőleges, $1 =$ vízszintes, $2 =$ átlós vonal). A páros korrupt teszt a 22.4. Augmentációs kísérlet és domainváltozás és a 38.3. Végső teszt és páros korrupció szerint ugyanazokat a jelenet–nézet párokat értékeli clean és korrupt változatban, ezért a hatás jelenetszintű páros különbséggel mérhető. Rögzítse a kérdést, a baseline-t, a spliteket, a paraméterkeretet, a választási szabályt, a metrikát, a seed-kezelést és az érvényességi korlátokat. Nincs előre megadott győztes; a 17.4. Architektúra összehasonlítása kontrollált keretben fair comparison keretét kövesse. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

18.1.7 Megoldások és indoklás

F1 megoldása. Az identitás shortcut a $y = x + F(x)$ képletben az x közvetlen átvezetése a kimenethez, paraméter nélkül. Ez akkor megengedett, ha x és $F(x)$ alakja megegyezik. A projection shortcut $h(x) = W_{\text{sc}}x$ tanulható lineáris transzformáció, amely a shortcut utat a főág alakjára hozza. Példa: $x \in \mathbb{R}^{2\times3\times24\times24}$, F kimenete $\mathbb{R}^{2\times6\times24\times24}$. Ekkor $x + F(x)$ nem értelmezhető, mert a csatornadimenzió $3 \neq 6$; a PyTorch `add` művelet shape-mismatch hibát adna. A projection 1×1-es konvolúcióval $3 \to 6$ csatornát képez, így mindkét tag $(2, 6, 24, 24)$ lesz.

F2 megoldása. A lemmából $J_G(x) = I_2 + J_F(x)$. Lineáris F esetén $J_F(x) = A$ minden x-re, tehát $J_G(x) = I_2 + A$. Válasszuk a diagonális mátrixot

$$A = \begin{bmatrix} -0{,}99 & 0 \\ 0 & 99 \end{bmatrix}.$$

Ekkor

$$I_2 + A = \begin{bmatrix} 0{,}01 & 0 \\ 0 & 100 \end{bmatrix}.$$

A szinguláris értékek diagonális mátrixnál a főátló elemeinek abszolút értékei: $\sigma_1 = 100$, $\sigma_2 = 0{,}01$, tehát

$$\kappa_2(I_2 + A) = \frac{100}{0{,}01} = 10^4 \geq 100.$$

Az ellenőrzés mutatja, hogy a shortcut jelenléte ellenére a Jacobi-mátrix tetszőlegesen rosszul kondicionált lehet, ha F spektruma -1 közelébe kerül.

F3 megoldása. A kód:

```python
"""lab/residual_stride.py"""
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32


class StrideResBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 4, 5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(4, 4, 5, padding=2)
        self.shortcut = nn.Conv2d(1, 4, 1, stride=2)
        self.relu = nn.ReLU()

    def forward(self, x):
        h = self.relu(self.conv1(x))
        h = self.conv2(h)
        return h + self.shortcut(x)


def main():
    x = torch.randn(2, 1, 24, 24, device=device, dtype=dtype)
    block = StrideResBlock().to(device, dtype)
    block.eval()
    with torch.no_grad():
        y = block(x)
        h_ref = block.conv2(block.relu(block.conv1(x)))
        short = block.shortcut(x)
        gap = (y - h_ref).abs().max().item()
        err = (y - (h_ref + short)).abs().max().item()
        short_mag = short.abs().max().item()
    print("kimeneti alak:", tuple(y.shape))
    print("max|y-h|:", gap)
    print("max|shortcut|:", short_mag)
    print("max|y-(h+shortcut)|:", err)
    assert gap > 0 and abs(gap - short_mag) < 1e-6
    assert err < 1e-6


if __name__ == "__main__":
    main()
```

A kimeneti méret a 14.1. Stride, padding, dilation és outputméret formulával: $H_{out} = \lfloor (24 + 2 \cdot 2 - 1 \cdot (5-1) - 1)/2 \rfloor + 1 = 12$, tehát a várt alak $(2, 4, 12, 12)$. A shortcut 1×1-es konvolúció stride $s = 2$-vel, így annak kimenete is $(2, 4, 12, 12)$. A `max|y-h|` érték nem hiba, hanem a shortcut hozzájárulása: a `max|shortcut|` értékkel egyezik, és pozitív, tehát a shortcut tényleg hozzáadódik a főághoz. A `max|y-(h+shortcut)|` konstrukció szerint nulla; ez csak a forward definíciójának konzisztenciáját ellenőrzi, önmagában nem bizonyítja a helyességet. A tűrés 10^{-6}; a mért értékek ezen a környezeten (Python 3.12.3, PyTorch 2.15.0+cpu, float32, CPU, 1 szál, seed 20261020) `max|y-h| = 2.402541160583496`, `max|y-(h+shortcut)| = 0.0`.

F4 megoldása. A hiba oka: a `BrokenBlock.forward` az x bemenetet közvetlenül adja hozzá a főág kimenetéhez, de x csatornaszáma 2, a főág kimenetéé pedig 4. A `+` művelet PyTorchban shape-mismatch `RuntimeError`-t eredményez. A következmény az, hogy a blokk egyetlen forwardja sem fut le. Javítás: adjunk a konstruktorhoz egy 1×1-es projekciós konvolúciót, és a forwardban azt használjuk:

```python
class FixedBlock(nn.Module):
```

```
def __init__(self):
    super().__init__()
    self.conv1 = nn.Conv2d(2, 4, 3, padding=1)
    self.conv2 = nn.Conv2d(4, 4, 3, padding=1)
    self.shortcut = nn.Conv2d(2, 4, 1)
    self.relu = nn.ReLU()

def forward(self, x):
    h = self.conv2(self.relu(self.conv1(x)))
    return h + self.shortcut(x)
```

Ellenőrzés $(2, 2, 24, 24)$ bemeneten: a kimenet alakja $(2, 4, 24, 24)$. A `max|y-h|` a shortcut tényleges hozzájárulása, nem hiba: értéke egyezik a `max|shortcut|` értékkel és pozitív, tehát a javított blokk nem csak a főágat adja vissza. A `max|y-(h+shortcut)|` konstrukció szerint nulla, ezért önmagában nem elegendő ellenőrzés. Ezen a környezeten (Python 3.12.3, PyTorch 2.15.0+cpu, float32, CPU, 1 szál, seed 20261020) `max|y-h| = 2.643393039703369`, `max|y-(h+shortcut)| = 0.0`, a tűrés 10^{-6}.

F5 megoldása. Egy lehetséges protokoll:

- **Kérdés:** javítja-e az identity shortcut a validációs pontosságot a GlyphLab orientációfeladatán, ha minden más tényező rögzített?
- **Baseline:** ugyanaz a három blokkból álló CNN shortcut nélkül; a shortcut változat ugyanazt a főágat használja, kiegészítve identity shortcuttal. Mivel a blokkok csatornaszáma nem változik, a paraméterszám a két modellben azonos; a fair comparison szerint nincs rejtett kapacitáskülönbség.
- **Split:** a GlyphLab `train/val/test` felosztása, a `scene_id` mint független megfigyelési egység. A `scene_id` azért a megfigyelési egység, mert egy jelenet két nézete korrelált, ezért a képenkénti függetlenség hamis; a split is jelenetszintű (A. A GlyphLab adatkészlet). A validáció csak modellválasztásra szolgál; a tesztet a legjobb validációs checkpoint kiválasztása után egyszer értékeljük.
- **Recept:** Adam, tanulási ráta 10^{-3}, batch méret 32, 20 epocha; 5 különböző seed. A seed a modell inicializálását és a lépések sorrendjét rögzíti; a GlyphLab splitet nem változtatjuk.
- **Metrika:** osztályokra aggregált validációs accuracy, a kiválasztás a legmagasabb validációs accuracy szerint; a végső tesztmetrika mellett a páros korrupt teszt is rögzíthető.
- **Összehasonlítás:** a két változatot ugyanazon az öt seeden párosítva futtatjuk; a döntési szabály a páros különbségek átlaga és 95%-os, scene-szintű bootstrap konfidenciaintervalluma (23.3. Páros összehasonlítás és csoportszintű intervallum), mert a seed-szórás önmagában nem fedi a tesztminta bizonytalanságát.
- **Érvényességi korlát:** a GlyphLab szintetikus, kis méretű adat, ezért az eredmény nem általánosítható természetes képekre. Az 5 seed a tanítási reprodukálhatóság (inicializálás és batch-sorrend) szórását becsli, az adatmintavétel bizonytalanságát nem; ezért a döntést a scene-szintű bootstrap adja, az 5 seed csak a stabilitás ellenőrzésére szolgál. A konklúzió csak „ezen az adaton, ezzel a recepttel” típusú lehet.

18.1.8 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a shortcut út jelenlétét globális konvergenciabizonyításnak tekintik. A fenti lemma csak lokális Jacobi-azonosság, és a J_F tetszőleges értéke miatt nem zárja ki a rossz kondicionáltságot. Konkrét ellenpélda: legyen $F(x) = -x$. Ekkor a blokk kimenete $y = x + F(x) = 0$ minden x-re. A shortcut jelen van, a Jacobi mégis $I_d + (-I_d) = 0$, tehát a blokk a bemenet minden információját eltünteti. Egy ilyen blokkokból álló háló bármely mélységben konstans nullát ad, így semmilyen értelmes feladatot nem tanul meg. A shortcut tehát csak egy lehetséges gradiensutat biztosít; a tényleges tanulási dinamika és a konvergencia továbbra is a

nemlineáris optimalizálás tulajdonságaitól függ. A P06 empirikus javulást közöl, nem tételként bizonyított globális garanciát.

18.1.9 Önellenőrzés és korábbi fogalom visszahívása

Önellenőrző kérdések: (1) Mikor értelmes az $x + F(x)$ összeadás tenzoralakban? (2) Mi a projection shortcut szerepe csatornaváltásnál? (3) Igaz-e, hogy ha minden blokk Jacobija $J_G = I_d + J_F$, akkor a teljes háló Jacobija I_d? (4) Mit ellenőriz a laborban a `err_same` érték? (5) Milyen állítástípus a P06 javulásra vonatkozó megállapítása?

Válaszok röviden. (1) Csak akkor, ha x és $F(x)$ alakja minden dimenzióban megegyezik; csatorna- vagy méretváltásnál projekciós shortcut kell. (2) A W_{sc} 1×1-es konvolúció a shortcut kimenetét a főág alakjára hozza, hogy az összeadás értelmezhető legyen. (3) Nem: a blokkok Jacobijainak szorzata általában nem I_d; az állítás csak az egyes blokkok identitásútra vonatkozó tagjáról szól. (4) Azt, hogy a blokk kimenetéből kivont főág-kimenet mennyire egyezik a bemenettel, azaz az identity shortcut tényleges hozzájárulását. (5) Empirikus megfigyelés; a szerzők javulást mérnek, de nem bizonyítanak globális konvergenciagarancia-tételt.

Korábbi fogalmak visszahívása: a 6.1. Objektív, reguláris tag és gradienslépés alfejezetből a gradienslépés és a loss-átlag skálázása; a 11.4. Egy teljes PyTorch tanítási iteráció alfejezetből a `zero_grad`–forward–`backward`–`step` sorrend; a 14.1. Stride, padding, dilation és outputméret alfejezetből a kimeneti méret formulája; a 14.4. Csoportos és depthwise konvolúció alfejezetből a csoportos konvolúció csatornakorlátozása (csoportok között nincs csatornakeverés); ezzel szemben a bottleneck 1×1-es rétegei csoportosítás nélküliek, azaz teljes csatornakeverést végeznek; a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetből az operátor-specifikáció háromszöge; a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből pedig a fair comparison kerete. A BatchNorm, amely a teljes P06 blokkban a konvolúciók után áll, a következő egység témája.

18.1.10 Rövid összefoglaló

A reziduális blokk $y = x + F(x)$ alakú, ahol a shortcut identitás vagy 1×1-es projekció, ha az alak nem egyezik. A blokk Jacobi-mátrixa differenciálható F esetén $I_d + J_F(x)$, így a VJP-ben a gradiens az identitásúton és a reziduális ágon is terjed. Ez az azonosság lokális, és nem garantál globálisan jó kondicionáltságot: J_F sajátértékei közel kerülhetnek -1-hez, a mély szorzat skálája vagy spektruma pedig akkor is elfajulhat, ha az egyes $(I_d + J_{F_k})$ tényezők jól kondicionáltak. A labor két kis blokkot épít és ellenőriz. A BatchNorm-mechanizmus a következő egységben következik.

18.1.11 Következő alkalmazás

A 18.2. Batch normalization train és eval módban a BatchNorm tanítási és kiértékelési módját vezeti be a P07, P08 és D12 források alapján, és visszatér az itteni reziduális blokkra, amikor a normalizálást a főágba illeszti.

18.1.12 Források

- [P06] He, K., Zhang, X., Ren, S. és Sun, J. (2016). *Deep Residual Learning for Image Recognition.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). A preprint 2015-ben jelent meg arXiv:1512.03385 számon. https://arxiv.org/abs/1512.03385.
- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html.
- [P08] Santurkar, S., Tsipras, D., Ilyas, A. és Madry, A. (2018). *How Does Batch Normalization Help Optimization?* Advances in Neural Information Processing Systems 31 (NeurIPS 2018). arXiv:1805.11604. https://arxiv.org/abs/1805.11604.
- [D12] PyTorch contributors. *BatchNorm2d.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30.

18.2 Batch normalization train és eval módban

18.2.1 Cél, előismeret és induló kérdés

A 18.1. Reziduális blokk és identitásút egységben a reziduális blokkot $y = x + F(x)$ alakban definiáltuk, és láttuk, hogy a shortcut identitás a gradiens terjedését segíti. A gyakorlatban a reziduális főág egyik leggyakoribb eleme a konvolúció utáni batch normalizálás. Ennek az egységnek a célja annak pontos szétválasztása, ami a BatchNorm-ban *statisztikailag számított*, ami *tanulható paraméter*, és ami *pufferállapot*, különösen a train és az eval mód eltérő viselkedésén keresztül.

Előismeret: a 14.2. Több bemeneti és kimeneti csatorna alfejezetben bevezetett NCHW alak és csatornaszerkezet, valamint a 6.2. Konvexitás és sima optimalizálási határ alfejezetben tárgyalt optimalizálási szemlélet. A dropout train/eval kapcsolóját a 17.2. AlexNet és a kísérleti kontextus AlexNet-kontextusában említettük; a BatchNorm ugyanezt a kapcsolót más okból igényli.

Induló kérdés: egy BN-t tartalmazó hálózatot egyetlen tesztképpel futtatunk úgy, hogy elfelejtettük az `eval()` módot. A BatchNorm az egyetlen kép csatornájából számít átlagot és varianciát. Miért sérül ezzel a következtetési (inference) szerződés akkor is, ha gradienslépést egyáltalán nem végzünk?

18.2.2 Definíciók, jelölések és alakok

Legyen a réteg bemenete $X \in \mathbb{R}^{N \times C \times H \times W}$, az NCHW konvenció szerint. A BatchNorm2d minden c kimeneti/bemeneti csatornához külön statisztikát és külön affin paraméterpárt rendel. A batch-méret a csatornán belüli összes megfigyelés:

$$M = N \cdot H \cdot W.$$

A c csatorna batch-átlaga:

$$\mu_c = \frac{1}{M} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} x_{n,c,h,w}.$$

A c csatorna batch-varianciája a **torzított**, azaz $1/M$-es konvencióval:

$$\sigma_c^2 = \frac{1}{M} \sum_{n,h,w} \left(x_{n,c,h,w} - \mu_c\right)^2.$$

A PyTorch `BatchNorm2d` ezt a torzított varianciát használja; ez a `torch.var(..., unbiased=False)` konvenció [D12]. Ez a torzított variancia is statisztikai becslés, de nem a populációvariancia torzítatlan becslése: az 5.2. Várható érték, variancia és likelihood alfejezetben a Bessel-korrekció az $1/(M-1)$-es mintavarianciát adja. Itt a cél a normalizálás, ezért a PyTorch a torzított konvenciót használja; a két konvenció megkülönböztetése numerikus eltérést okoz.

A tanulható affin paraméterek csatornánként egy skalárpár:

$$\gamma_c \in \mathbb{R}, \qquad \beta_c \in \mathbb{R}.$$

A PyTorch-objektumban ezek a `weight` és a `bias` attribútumok, mindkettő C hosszú vektor. Az eps paraméter $\varepsilon > 0$ numerikus stabilizátor, alapértelmezése 10^{-5}.

18.2.3 A train mód forwardja

A train mód képlete három lépésből áll: számítás, normalizálás, affin transzformáció.

Definíció 1 (BatchNorm train forward). Legyen $\varepsilon > 0$, $\gamma_c, \beta_c \in \mathbb{R}$. A train módú normalizált érték

$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}},$$

a kimenet pedig

$$y_{n,c,h,w} = \gamma_c \, \hat{x}_{n,c,h,w} + \beta_c.$$

A μ_c és σ_c^2 a fenti, az aktuális batchből számított statisztikák. Minden mennyiség a c csatornán osztozik; a (γ, β) érték a térbeli pozíciók és a batch minden elemére kiterjed (broadcasting).

A shape-ellenőrzés: $\mu, \sigma^2, \gamma, \beta \in \mathbb{R}^C$; az X, $\hat{X}$, Y mind $\mathbb{R}^{N \times C \times H \times W}$.

Lemma 1 (a normalizált batch statisztikái). Ha $M \geq 1$ és $\varepsilon > 0$, akkor train módban $\hat{x}$ batch-átlaga minden c csatornán 0, torzított batch-varianciája pedig $\sigma_c^2/(\sigma_c^2 + \varepsilon)$.

Bizonyítás. Az összegzés linearitása miatt a normalizált értékek átlaga

$$\frac{1}{M} \sum_{n,h,w} \hat{x}_{n,c,h,w} = \frac{1}{\sqrt{\sigma_c^2 + \varepsilon}} \cdot \frac{1}{M} \sum_{n,h,w} \left(x_{n,c,h,w} - \mu_c\right) = \frac{1}{\sqrt{\sigma_c^2 + \varepsilon}} \cdot 0 = 0.$$

A varianciához használjuk, hogy a konstans $(\sigma_c^2 + \varepsilon)^{-1/2}$ szorzó a négyzetes eltérésekből kiemelhető:

$$\frac{1}{M} \sum_{n,h,w} \hat{x}_{n,c,h,w}^2 = \frac{1}{\sigma_c^2 + \varepsilon} \cdot \frac{1}{M} \sum_{n,h,w} \left(x_{n,c,h,w} - \mu_c\right)^2 = \frac{\sigma_c^2}{\sigma_c^2 + \varepsilon}.$$

Mivel a normalizált átlag nulla, ez a második kifejezés éppen a torzított variancia. □

Két következmény. Először, a normalizált batch varianciája pontosan $\sigma_c^2/(\sigma_c^2+\varepsilon)$, ami $\sigma_c^2 \gg \varepsilon$ esetén közel 1; ha $\sigma_c^2 = 0$, akkor $\hat{x} = 0$. Másodszor, az affin lépés után a batch-átlag β_c, a torzított variancia $\gamma_c^2 \cdot \sigma_c^2/(\sigma_c^2+\varepsilon)$.

18.2.4 Running statisztikák és a momentum paraméter

A train mód minden forwardja frissíti a **running puffert**, ha a `track_running_stats=True` alapértelmezést használjuk. A normalizálás és a pufferfrissítés **két különböző varianciát** használ. Jelölje

$$v_c^{\text{biased}} = \sigma_c^2 = \frac{1}{M}\sum_{n,h,w}(x_{n,c,h,w}-\mu_c)^2, \qquad v_c^{\text{unbiased}} = \frac{M}{M-1}\,v_c^{\text{biased}} \quad (M>1).$$

A train forward a torzított v_c^{biased}-szel normalizál (Definíció 1), a futó variancia viszont a torzítatlan v_c^{unbiased} becslést kapja [D12]. Legyen a frissítés előtti futó átlag $\bar{\mu}_c$, a frissítés előtti futó variancia $\bar{v}_c$ (a puffer varianciát tárol, ezért jelöljük v-vel, nem szórással), és legyen a momentum $\alpha \in (0,1]$. Az egy batch utáni új értékek:

$$\bar{\mu}_c \leftarrow (1-\alpha)\,\bar{\mu}_c + \alpha\,\mu_c, \qquad \bar{v}_c \leftarrow (1-\alpha)\,\bar{v}_c + \alpha\,v_c^{\text{unbiased}}.$$

$M = N\cdot H\cdot W = 1$ esetén (azaz csatornánként egyetlen elem, $N=H=W=1$) a torzítatlan becslés nem értelmezett; a PyTorch ilyenkor train módban hibát jelez. A torzítatlan pufferfrissítés feltétele tehát csatornánként $N\cdot H\cdot W > 1$.

A PyTorch alapértelmezett $\alpha = 0{,}1$. Ez exponenciális mozgóátlag: a régi puffer $1-\alpha$, az aktuális batch α súlyt kap; t frissítés után a j-edik batch súlya $\alpha(1-\alpha)^{t-j}$, a kezdeti puffer súlya $(1-\alpha)^t$. A puffer a `running_mean` és `running_var` attribútumban él, és **nem kap gradienst**; a `state_dict()` menti, az optimizáló viszont nem frissíti. A `running_var` varianciát tárol, nem szórást, ezért a forwardban gyököt kell vonni belőle.

A sorozat frissítése t batch után, $\alpha = 0{,}1$ mellett gyorsan elfelejti a múltat: a legutolsó batch $\alpha = 0{,}1$, az azt megelőző $\alpha(1-\alpha) = 0{,}09$ súlyt kap, az első batch súlya $\alpha(1-\alpha)^{t-1}$. Ha α-t növeljük, a puffer érzékenyebb az aktuális minibatch zajára; ha csökkentjük, a konvergencia lassul. A `momentum=None` opció a dokumentáció szerint az 1/num_batches_tracked kumulatív átlagot használja [D12]; a főútban mi a rögzített $\alpha = 0{,}1$ változatot tárgyaljuk. Fontos: ez az α **nem** az optimalizáló momentumával azonos (6.4. Momentum és adaptív optimizálók); a BN `momentum` a futó statisztika exponenciális simításának súlya, az optimalizáló momentum a gradienslépés tehetetlensége, és a két paraméter független.

18.2.5 Eval mód és a két út különbsége

Az `eval()` mód a BatchNorm számára azt jelenti, hogy a batchstatisztikát nem számoljuk, és a puffert nem frissítjük. Ha a running statisztika rendelkezésre áll, akkor a normalizálás a futó értékekkel történik:

$$y_{n,c,h,w} = \gamma_c\,\frac{x_{n,c,h,w}-\bar{\mu}_c}{\sqrt{\bar{v}_c+\varepsilon}} + \beta_c.$$

Definíció 2 (BatchNorm eval forward). Eval módban, `track_running_stats=True` mellett, a BatchNorm2d a fenti képletet használja, ahol $\bar{\mu}_c$ és $\bar{v}_c$ a train során felhalmozott futó statisztika. A γ_c, β_c ugyanazok a tanulható paraméterek, mint train módban.

A train/eval különbség formálisan abban áll, hogy train módban a statisztika a minibatch függvénye és a forward mellékhatással frissíti a puffert, eval módban pedig a statisztika konstans, a forwardnak nincs mellékhatása. Ez nem az autograd beállítása, hanem a modul állapota: az `eval()` a modul viselkedését (a BatchNormnál a running statisztika használatát) állítja, a `torch.inference_mode()` pedig az autograd követését és a verziószámlálót kapcsolja ki. A kettő független: `eval()` nélkül a `torch.inference_mode()` alatt futtatott forward is train-módú BatchNormot használ, és továbbra is frissíti a puffert.

Az eval mód azért szükséges, mert az inference alatt a kimenetnek nem szabad függnie attól, hogy egy képet egyedül vagy más képekkel egy batchben értékelünk ki. A batchstatisztika egyetlen képen is számolható, ha $N \cdot H \cdot W > 1$ (egyetlen képnél $H \cdot W > 1$), de akkor minden kép önmagához normalizálódik, nem a tanítási populációhoz; kis batchben pedig a statisztika varianciája nagy. A running puffer a train alatt látott csatornaeloszlás exponenciális mozgóátlagát közelíti, és így egy előre rögzített normalizálási konstanst ad az inference-hez.

18.2.6 Kézi kis példa

Legyen $N = 2$, $C = 1$, $H = W = 2$, tehát $M = 8$. A bemenet

$$X = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \\ \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix} \end{bmatrix} \in \mathbb{R}^{2\times 1\times 2\times 2}.$$

A nyolc pixel: $1, 2, 3, 4, 2, 4, 6, 8$. Az összeg 30, így

$$\mu = \frac{30}{8} = 3{,}75.$$

A négyzetes eltérések összege $7{,}5625 + 3{,}0625 + 0{,}5625 + 0{,}0625 + 3{,}0625 + 0{,}0625 + 5{,}0625 + 18{,}0625 = 37{,}5$, tehát

$$\sigma^2 = \frac{37{,}5}{8} = 4{,}6875.$$

$\varepsilon = 10^{-5}$ mellett $\sqrt{4{,}68751} \approx 2{,}1651$. A normalizált $\hat{x}$ értékek:

$$\frac{1-3{,}75}{2{,}1651} = -1{,}2702, \quad \frac{2-3{,}75}{2{,}1651} = -0{,}8083, \quad \frac{3-3{,}75}{2{,}1651} = -0{,}3464, \quad \frac{4-3{,}75}{2{,}1651} = 0{,}1155.$$

A második kép $2, 4, 6, 8$ értékei rendre $-0{,}8083$, $0{,}1155$, $1{,}0392$, $1{,}9630$. Ha az affin paraméter $(\gamma, \beta) = (2, 1)$, akkor $y = 2\hat{x} + 1$:

x	$x - \mu$	$\hat{x}$	$y = 2\hat{x} + 1$
1	$-2{,}75$	$-1{,}2702$	$-1{,}5403$
2	$-1{,}75$	$-0{,}8083$	$-0{,}6166$

x	$x-\mu$	$\hat{x}$	$y=2\hat{x}+1$
3	$-0{,}75$	$-0{,}3464$	0,3072
4	0,25	0,1155	1,2309
5	1,25	0,5773	2,1547
6	2,25	1,0392	3,0785
7	3,25	1,5011	4,0022
8	4,25	1,9630	4,9260

A második kép 2-je és 4-ese a táblázat 2. és 4. sorával azonos $\hat{x}$-et és y-t ad, mert a statisztika mind a nyolc pixelből készül.

Most a running frissítés egyetlen forward után, $\alpha = 0{,}1$, kezdeti $\bar{\mu}=0$, $\bar{v}=1$ mellett. A pufferbe a torzítatlan variancia kerül: $v^{\text{unbiased}} = \frac{8}{7}\cdot 4{,}6875 = \frac{37{,}5}{7} \approx 5{,}357142857$. Így

$$\bar{\mu} = 0{,}9\cdot 0 + 0{,}1\cdot 3{,}75 = 0{,}375, \qquad \bar{v} = 0{,}9\cdot 1 + 0{,}1\cdot 5{,}357142857 \approx 1{,}435714286.$$

Ha ugyanez a batch másodszor is megérkezik, a puffer $\bar{\mu} = 0{,}9\cdot 0{,}375 + 0{,}1\cdot 3{,}75 = 0{,}7125$ és $\bar{v} = 0{,}9\cdot 1{,}435714286 + 0{,}1\cdot 5{,}357142857 \approx 1{,}827857143$ lesz. Aki a torzított 4,6875-öt írná a pufferbe, 1,36875-öt, illetve 1,700625-öt kapna; a különbség kis M mellett jelentős.

18.2.7 Labor: kézi számítás és pufferfrissítés

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `bn_manual.py` a `lab/` könyvtárból futtatható (`python bn_manual.py`); külső adatfájl nem kell.

A munkakönyvtár `lab/`. A szkript `bn_manual.py`; futtatás:

```
python bn_manual.py
```

A bemenetet float32-ben, CPU-n adjuk meg, a könyv laborfőútja szerint:

```
import torch
import torch.nn as nn

torch.manual_seed(20261020)  # a BN inicializálása seedfüggetlen; a seed a script
↪  ismételhetőségét rögzíti
dtype = torch.float32
device = "cpu"

X = torch.tensor(
    [[[[1.0, 2.0], [3.0, 4.0]]],
     [[[2.0, 4.0], [6.0, 8.0]]]],
    dtype=dtype, device=device
)  # (2,1,2,2)

bn = nn.BatchNorm2d(
    num_features=1, eps=1e-5, momentum=0.1,
    affine=True, track_running_stats=True
).to(device)

# Kézi statisztika a (N,H,W) dimenziókon, ugyanazzal a torzított varianciával:
mu = X.mean(dim=(0, 2, 3), keepdim=True)          # (1,1,1,1)
var = X.var(dim=(0, 2, 3), unbiased=False, keepdim=True)
```

```
bn.train()
Y = bn(X)

manual = (X - mu) / torch.sqrt(var + 1e-5)       # gamma=1, beta=0
train_diff = (Y - manual).abs().max().item()
print("max_abs_diff train:", train_diff)
assert train_diff < 1e-5

# Kézi pufferfrissítés: a futó variancia a torzítatlan becslést kapja (M=8).
M = X.numel() // X.shape[1]
var_unbiased = var * M / (M - 1)
rm_manual = 0.9 * 0.0 + 0.1 * mu.flatten()
rv_manual = 0.9 * 1.0 + 0.1 * var_unbiased.flatten()
print("running_mean:", bn.running_mean.item(), "kézi:", rm_manual.item())
print("running_var:", bn.running_var.item(), "kézi:", rv_manual.item())
assert torch.allclose(bn.running_mean, rm_manual) and torch.allclose(bn.running_var, rv_manual)
# Kézi levezetés: running_mean = 0.375, running_var = 37.5/70 + 0.9 = 1.435714...
```

A kézi és a könyvtári forward közötti eltérés a fenti, $M = 8$ elemű példán float32-ben legfeljebb 10^{-5} nagyságrendű abszolút hiba, mert a képletek ugyanazok; a labor ezt `max_abs_diff` formában írja ki. A futtatás után a pufferek a fenti, torzítatlan varianciával számított értékeket adják; a szkript ezt `assert`-tel ellenőrzi. A mért kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020): `running_mean` 0,375, `running_var` $\approx$ 1,4357142, a train-forward eltérése a kézi képlettől gépi pontosságú.

Az eval-út ellenőrzése ugyanabban a szkriptben folytatódik; a kézi eval képlet az általános affin paramétereket is tartalmazza, nem csak az alapértelmezett $\gamma = 1, \beta = 0$ esetet:

```
bn.eval()
Y_eval = bn(X)

manual_eval = (X - rm_manual.view(1, -1, 1, 1)) / torch.sqrt(rv_manual.view(1, -1, 1, 1) + 1e-5)
manual_eval = bn.weight.view(1, -1, 1, 1) * manual_eval + bn.bias.view(1, -1, 1, 1)
eval_diff = (Y_eval - manual_eval).abs().max().item()
print("max_abs_diff eval:", eval_diff)
assert torch.allclose(Y_eval, manual_eval, atol=1e-5, rtol=0)

# Induló kérdés: ugyanaz az egy kép train módban a saját statisztikájával
# normalizálódik, és a forward mellékhatásként felülírja a running puffert.
bn.train()
rm_before = bn.running_mean.clone()
rv_before = bn.running_var.clone()
Y_single = bn(X[:1])
single_mu = X[:1].mean(dim=(0, 2, 3), keepdim=True)
single_var = X[:1].var(dim=(0, 2, 3), unbiased=False, keepdim=True)
manual_single = (X[:1] - single_mu) / torch.sqrt(single_var + 1e-5)
manual_single = bn.weight.view(1, -1, 1, 1) * manual_single + bn.bias.view(1, -1, 1, 1)
single_diff = (Y_single - manual_single).abs().max().item()
single_vs_eval = (Y_single - Y_eval[:1]).abs().max().item()
buffer_changed = (not torch.allclose(bn.running_mean, rm_before)) or (not
↪  torch.allclose(bn.running_var, rv_before))
print("egykepes train max_abs_diff:", single_diff)
print("egykepes train vs eval max_abs_diff:", single_vs_eval)
print("running buffer valtozott:", buffer_changed)
assert torch.allclose(Y_single, manual_single, atol=1e-5, rtol=0)
assert single_vs_eval > 1e-3 and buffer_changed
```

A rögzített környezetű ellenőrző futás kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
max_abs_diff train: 0.0
running_mean: 0.375 kézi: 0.375
running_var: 1.4357142448425293 kézi: 1.4357142448425293
max_abs_diff eval: 1.1920928955078125e-07
egykepes train max_abs_diff: 1.1920928955078125e-07
egykepes train vs eval max_abs_diff: 1.863243579864502
running buffer valtozott: True
```

A fenti kimenet első négy sora a train és eval út egyezését mutatja. Az ötödik–hetedik sor az induló kérdést demonstrálja: ugyanazt az első képet train módban futtatva a kimenet a saját statisztikás kézi képlettel egyezik, de eltér az eval-módú kimenettől, és a running puffer megváltozik. Ezért az `eval()` elhagyása nem pusztán elméleti hiba: a predikció a kép saját statisztikájától függ, és a modell pufferállapota is módosul. A `bn.eval()` hívás után a forward már a puffert használja. Az $x = 1$ pixel eval-normalizált értéke $(1 - 0{,}375)/\sqrt{1{,}4357142448 + 10^{-5}} \approx 0{,}5216$, az $x = 8$ pixelé $(8 - 0{,}375)/\sqrt{1{,}4357142448 + 10^{-5}} \approx 6{,}3636$; az itt használt PyTorch 2.15.0+cpu környezetben (Python 3.12.3, CPU, 1 szál, seed 20261020) a könyvtári eval-forward mind a nyolc pixelre ezt adta. A kimeneti összevetés akkor igazolja az eval utat, ha a könyvtár és a kézi képlet mind a 8 pixelre egyezik.

18.2.8 Gyakori tévedés: a mechanizmus és a motiváció

Az eredeti tanulmány címe az internal covariate shift-et (röviden ICS) nevezi meg motivációként [P07]. Az **ICS** a belső rétegek bemeneti eloszlásának tanítás közbeni eltolódása: ahogy az előző rétegek paraméterei változnak, a következő réteg bemenetének eloszlása is változik, ezért a rétegnek folyamatosan új eloszláshoz kell alkalmazkodnia. A cikk teljes mechanizmus-érvelését azonban itt **nem** idézzük; a leírás a PMLR-rekordra és az absztraktra támaszkodik. A későbbi munka absztraktja szerint a BatchNorm javulása nem elsősorban az eloszlásstabilitáson, hanem az optimalizálási felület simításán keresztül magyarázható [P08]. Ebből **nem** következik, hogy az ICS-érv hamis, és az sem, hogy a P08-magyarázat minden architektúrára érvényes. A 6.2. Konvexitás és sima optimalizálási határ alfejezetben bevezetett L-simaság **globális** feltétel: a gradiens L-Lipschitz minden $x, y \in \mathbb{R}^d$ párra. Egy csak az x környezetében, adott sugarú gömbön érvényes Lipschitz-korlát ennél gyengébb, **lokális** feltétel; a kettőt külön kell kimondani, és a BN hatását leíró állítás legfeljebb lokális simaságot adhat. A BN önmagában sem hálózati szintű globális konvergenciát, sem jobb generalizációt nem garantál. A helyes állításforma:

- *empirikus motiváció*: a P07 szerzői ezzel a célkitűzéssel vezették be a réteget;
- *későbbi vizsgálat*: a P08 absztrakt az optimalizálási tájképet hangsúlyozza;
- *bizonyított állítás*: a fenti Lemma 1 a batch-statisztikákról szól, és semmi többet nem állít a tanulás sikeréről.

További gyakori hiba a momentum összekeverése a 6.4. Momentum és adaptív optimizálók optimalizáló-momentumával. Az ottani momentum a gradienslépés tehetetlensége; a BN `momentum` a futó statisztika exponenciális simításának α súlya. A két paraméter egymástól független, és a `state_dict()`-ben sem érintkeznek.

18.2.9 Önellenőrzés

- Tudja-e, hogy a `running_var` varianciát és nem szórást tárol, ezért kell gyököt vonni?
- Tudja-e, hogy train módban a puffer frissítése forward mellékhatás, ezért a modell `eval()` nélküli tesztfuttatás megváltoztatja a későbbi inference-t?
- Tudja-e, hogy $\sigma_c^2 = 0$ esetén $\hat{x} = 0$, és az ε csak a nullával osztást kerüli el, nem ad statisztikai tartalmat a konstans csatornának?

18.2.10 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem használjuk a batchstatisztikát egyetlen tesztképre, és mit ad helyette a running puffer. Térjen ki arra az esetre is, amikor $N = 1$, de $H \cdot W = 576$ a GlyphLab 24×24-es képén (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték). Elvárt eredmény: saját, három-négy mondatos érvelés, amely megkülönbözteti a batchstatisztikát a running statisztikától. **Értékelési szempont:** a válasz különítse el (a) a számítás lehetőségét (576 pixelből számítható statisztika), (b) a normalizálás jelentését (saját képhez vs. tanított populációhoz), és (c) a batchösszetételtől való függést; mindhárom szempont megjelenése szükséges az elfogadáshoz.

F2 (levezetési, nehézség 2). A fenti labor batchétől független új esetben legyen $X \in \mathbb{R}^{2\times1\times2\times2}$ értéke $[[[1,3],[5,7]]]$ és $[[[2,6],[10,14]]]$, $\gamma = 2$, $\beta = -1$, $\varepsilon = 10^{-5}$. Számítsa ki μ, σ^2 értékét a torzított konvencióval, majd mind a nyolc $y_{n,c,h,w}$ kimenetet. Elvárt eredmény: minden lépés, a shape megadása és a nyolc kimeneti érték két tizedesre kerekítve.

F3 (implementációs, nehézség 2). Írjon `batchnorm_forward_manual(X, gamma, beta, eps)` függvényt, amely a train forwardot a Definition 1 képleteivel számolja, a statisztikát a $(0, 2, 3)$ dimenziókon, `unbiased=False` varianciával képezi, és a `batchstat` visszatérési értékben visszaadja a (μ, σ^2) párt is. Verifikálja a beépített `nn.BatchNorm2d` train kimenetén a labor X batchén, $\gamma = 2$, $\beta = -1$ beállítással. Elvárt eredmény: teljes kód, a maximális abszolút eltérés és a használt tolerancia.

F4 (hibakeresési, nehézség 2). Az alábbi töredék a labor modelljét egy tesztlistán futtatja:

```
best = SmallCNN()
best.load_state_dict(torch.load("best.pt", weights_only=True))
best.train()  # HIBA: train módba kapcsol; nem őrzi meg a kiértékelési állapotot
preds = [best(x.unsqueeze(0)).argmax().item() for x in test_images]
```

Azonosítsa a hibát, mondja ki a következményét a BatchNorm statisztikáira, és adja meg a javítást. Elvárt eredmény: a hiba oka, a következmény, a javított kódblokk és annak ellenőrzése, hogy eval módban a running puffer nem változik, train módban viszont igen, és hogy ugyanarra a képre a train-módú kimenet eltér az eval-módútól.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adaton arra a kérdésre, hogy a BatchNorm beillesztése a 38.2. Modell, training és best checkpoint egységben definiált kis CNN (SmallCNN) főágába javítja-e a validációs loss-t a normalizálás nélküli baseline-hoz képest. Az adat 180 jelenetből áll, jelenetenként két nézettel (360 kép, 24×24), a split 240/60/60 kép; a `test_images` a teszt split 60 képét jelöli, a `corrupted_test.npy` pedig ugyanezen jelenet–nézet párok rontott változatát tartalmazza (38.3. Végső teszt és páros korrupció). A `best.pt` a tanítás során a validációs loss alapján kiválasztott checkpoint (38.2. Modell, training és best checkpoint). Adjon protokollt: rögzített, jelenetszintű split (a nézetek nem kerülnek külön splitbe), három seed, a val loss alapú checkpoint-választás, a záróteszt a clean és a `corrupted_test.npy` változaton csak a kiválasztás után, páros összehasonlítás. A párosítás egysége a `scene_id`, mert egy jelenet két nézete korrelált. A hipotézis iránya egyoldalú: BN mellett a validációs loss kisebb; a döntést mégis kétoldali 95%-os konfidenciaintervallummal hozzuk, és a hatás előjelét is jelentjük. Elvárt eredmény: a hipotézis, a változó tényező, a metrika, az érvényességi korlátok és annak kimondása, hogy a kis toy adaton mért különbség nem általános mechanizmusbizonyíték. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

18.2.11 Megoldások és indoklás

F1. A batchstatisztika az adott minibatch függvénye. Egyetlen tesztkép esetén $N = 1$, így a c csatorna átlaga és varianciája az adott kép $H \cdot W$ pixeléből készülne: minden kép saját középértékéhez normalizálódna, nem a tanítás során látott csatornaeloszláshoz. A 576 pixel elegendő lenne a számításhoz, de a normalizálás jelentése megváltozik: az inference kimenet attól függne, mely képek vannak éppen a batchben, és egy kép saját kontrasztja eltűnne. A running puffer az exponenciális mozgóátlaggal a train alatt megfigyelt statisztikát rögzíti, és rögzített, batchfüggetlen normalizálást ad az eval módhoz.

F2. A nyolc pixel: $1, 3, 5, 7, 2, 6, 10, 14$. Összegük 48, tehát

$$\mu = \frac{48}{8} = 6.$$

Az eltérések és négyzeteik páronként (nyíljelöléssel, hogy a vessző ne tizedesjelként legyen olvasható): $(-5) \to 25$, $(-3) \to 9$, $(-1) \to 1$, $1 \to 1$, $(-4) \to 16$, $0 \to 0$, $4 \to 16$, $8 \to 64$. Az összegük 132, így a torzított variancia

$$\sigma^2 = \frac{132}{8} = 16{,}5.$$

A nevező $\sqrt{16{,}5 + 10^{-5}} \approx 4{,}0620$. A kimenet $y = 2\hat{x} - 1$:

$$\begin{aligned}
x = 1: \ & \frac{-5}{4{,}0620} = -1{,}2310, \quad y = -3{,}4618; \\
x = 3: \ & \frac{-3}{4{,}0620} = -0{,}7386, \quad y = -2{,}4771; \\
x = 5: \ & \frac{-1}{4{,}0620} = -0{,}2462, \quad y = -1{,}4924; \\
& x = 7: \ y = -0{,}5076; \\
& x = 2: \ y = -2{,}9695; \\
& x = 6: \ y = -1; \\
& x = 10: \ y = 0{,}9695; \\
x = 14: \ & \frac{8}{4{,}0620} = 1{,}9695, \quad y = 2{,}9389.
\end{aligned}$$

A kimeneti tensor shape-je $(2, 1, 2, 2)$. A $x = 6$ eset azért ad pontosan -1-et, mert $x = \mu$, tehát $\hat{x} = 0$.

F3. Egy lehetséges teljes megoldás:

```
import torch
import torch.nn as nn

def batchnorm_forward_manual(X, gamma, beta, eps):
    mu = X.mean(dim=(0, 2, 3), keepdim=True)
    var = X.var(dim=(0, 2, 3), unbiased=False, keepdim=True)
    xhat = (X - mu) / torch.sqrt(var + eps)
    Y = gamma.view(1, -1, 1, 1) * xhat + beta.view(1, -1, 1, 1)
    return Y, (mu, var)
```

```
X = torch.tensor(
    [[[[1.0, 2.0], [3.0, 4.0]]],
     [[[2.0, 4.0], [6.0, 8.0]]]], dtype=torch.float32
)

bn = nn.BatchNorm2d(1, eps=1e-5, momentum=0.1)
bn.weight.data.copy_(torch.tensor([2.0]))
bn.bias.data.copy_(torch.tensor([-1.0]))
bn.train()

Y_ref = bn(X)
Y_man, (mu, var) = batchnorm_forward_manual(
    X, bn.weight, bn.bias, 1e-5
)
print("mu =", mu.view(-1).tolist())       # [3.75]
print("var =", var.view(-1).tolist())     # [4.6875]
diff = (Y_ref - Y_man).abs().max().item()
print("max abs diff =", diff)
assert torch.allclose(Y_ref, Y_man, atol=1e-5, rtol=0)
```

A mért `max abs diff` 0,0 (Python 3.12.3, PyTorch 2.15.0+cpu, float32, CPU, 1 szál). A feladat és a labor ugyanazt az `atol=1e-5` tűrést használja: mivel a képlet és a könyvtár ugyanazt a torzított varianciát számolja, a szigorú tűrés is tartható; a mért eltérés 0,0, ezért a tűrés nem laza. Az `assert torch.allclose(Y_ref, Y_man, atol=1e-5, rtol=0)` ellenőrzi ezt.

F4. A hiba a `best.train()` hívás a kiértékelés előtt. A BatchNorm train módban az aktuális tesztbatch statisztikáiból számol, és forwardonként frissíti a running puffert. Mivel itt egyelemű batch-eket használunk, minden kép a saját statisztikájával normalizálódik, ezért a logit nem függ a tesztlista sorrendjétől; a valódi következmény az, hogy a predikció nem a tanított populációstatisztikát használja, és a running pufferek felülíródnak. A `best.pt` fájl változatlan; csak a memóriabeli modell pufferállapota íródik felül. A javítás:

```
best.eval()
with torch.inference_mode():
    preds = [best(x.unsqueeze(0)).argmax().item() for x in test_images]
```

Az ellenőrzés: `eval()` módban a running puffer két egymást követő forward után változatlan marad, `train()` módban viszont megváltozik; ugyanarra az egyképes bemenetre a train-módú és az eval-módú kimenet eltér. A tesztlista két permutációja egyelemű batch-ek mellett önmagában nem mutatja ki a hibát, mert a train-módú normalizálás képfüggetlen. A `torch.inference_mode()` a gradiensszámítást kapcsolja ki; önmagában nem váltja át a BatchNorm eval állapotát, ezért a kettőt együtt kell használni.

F5. A protokoll rögzítése a futás előtt: a kérdés az, hogy a BN a GlyphLab validációs loss-át csökkenti-e a normalizálás nélküli, azonos architektúrájú baseline-hoz képest. A megfigyelési egység a `scene_id`; a train/val/test splitet a generátor rögzíti (240/60/60 kép, azaz 120/30/30 jelenet), a bootstrap vagy újraosztás nem keveri a nézeteket. Mindkét modellt három seeddel tanítjuk, azonos inicializálási és optimalizáló-beállításokkal. A modellek a BN jelenlétében térnek el. A checkpoint-választás minden futásban a validációs loss minimuma alapján történik (`best.pt`), és ugyanez a val loss a fő metrika; a teszt accuracyt csak a kiválasztás után, a befagyasztott checkpointon jelentjük, és a jelentett érték nem hat vissza a választásra. A két modellt ugyanazon a három seeden párosítva hasonlítjuk össze; a döntési szabály a páros különbségek átlaga és 95%-os, scene-szintű bootstrap konfidenciaintervalluma (23.3. Páros összehasonlítás és csoportszintű intervallum). A hipotézis iránya egyoldalú (BN mellett kisebb val loss), de az intervallumot kétoldalúan számoljuk, és a hatás előjelét is jelentjük. A korlátok: a GlyphLab 240 tanítóképe (120 jelenet) nem elegendő a mechanizmus általános igazolására; a három seed csak a tanítási zaj egy keskeny szórását mutatja, a tesztminta bizonytalanságát nem;

a P08 optimalizálási magyarázatát ez a kísérlet nem erősíti vagy cáfolja, mert csupán egy kis, egy feladatú adathalmazon mér.

18.2.12 Rövid összefoglaló

A BatchNorm2d train módban minden csatornára kiszámítja a batch átlagát és torzított, $1/M$-es varianciáját, normalizál, majd a tanulható γ és β affin transzformációt alkalmazza. A forward mellékhatásként α momentummal frissíti a futó átlag- és varianciapuffert. Eval módban a batchstatisztika helyett ez a rögzített running puffer normalizál, így az inference nem függ a batch összetételétől. A P07 ICS-motivációja és a P08 optimalizálási vizsgálata külön bizonyítékigényű állítás; a bizonyított rész csupán a normalizált batch nulla átlagáról és $\sigma_c^2/(\sigma_c^2+\varepsilon)$ varianciájáról szól.

18.2.13 Következő alkalmazás

A 18.3. Pre-activation, kis batch és alternatív normalizálás egység a pre-activation elrendezést, a kis batch viselkedését és a BatchNorm alternatíváit tárgyalja. A **post-activation** elrendezésben a konvolúció után BN, majd ReLU áll; a **pre-activation** változatban a BN és a ReLU a konvolúció elé kerül, így a shortcut tiszta identitásút marad. Ehhez itt azt visszük tovább, hogy a normalizálás helye a reziduális főágban nem közömbös: a shortcut megkerülheti a normalizálást, ezért a pre-activation és a post-activation változat statisztikai útja eltér.

18.2.14 Felhasznált források

- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html. A címben szereplő motivációt a [P08] eredményével összevetve tárgyaljuk.
- [P08] Santurkar, S. et al. (2018). *How Does Batch Normalization Help Optimization?* arXiv:1805.11604. https://arxiv.org/abs/1805.11604.
- [D12] PyTorch contributors. *BatchNorm2d.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30. A train/eval mód, a batch- és futó statisztikák, valamint a varianciakonvenció API-háttere.

18.3 Pre-activation, kis batch és alternatív normalizálás

18.3.1 Cél, előismeret és induló kérdés

A 18.2. Batch normalization train és eval módban egységben tisztáztuk, hogy a BatchNorm2d train módban batchstatisztikát, eval módban futó puffert használ. Itt azt vizsgáljuk, hogy a normalizáló réteg mely tengelyeken aggregál, és ez hogyan hat a batchfüggésre. Az előismeret a következő: a normalizálás képlete, a train/eval különbség, valamint a 14.3. Paraméterszám, MAC és aktivációmemória alfejezetből ismert tenzoralakok és paraméterszámok, a 17.3. VGG és kis kernelek kompozíciója alfejezetből a mély, kis kernelű blokkok szerkezete, a 6.3. Sztochasztikus és minibatch gradiens alfejezetből pedig a minibatchstatisztika zajossága.

Induló kérdés: egyetlen, $N = 1$ méretű batch esetén milyen statisztikát számol a BatchNorm, a LayerNorm és a GroupNorm? A becslésed rögzítése után ellenőrizzük a definíciókból.

18.3.2 A normalizáló réteg közös váza és a három tengelyválasztás

Legyen a bemenet $X \in \mathbb{R}^{N\times C\times H\times W}$, ahol N a batch, C a csatorna, H és W a térbeli kiterjedés. Egy normalizáló réteg a rétegtípusra jellemző módon csoportosítja a bemenet elemeit: a statisztikát egy $\mathcal{A}$ indexhalmaz elemein számolja, ahol $\mathcal{A}$ a rétegtípustól függ, nem az egyes megfigyelésektől (a BatchNormnál csatornánként a batch- és térbeli indexek, a LayerNormnál mintánként az összes csatorna és térbeli index, a GroupNormnál mintánként és csoportonként a csoport csatornái és a térbeli indexek). Az átlag és a torzított variancia:

$$\mu_{\mathcal{A}} = \frac{1}{|\mathcal{A}|}\sum_{a\in\mathcal{A}} x_a, \qquad \sigma^2_{\mathcal{A}} = \frac{1}{|\mathcal{A}|}\sum_{a\in\mathcal{A}} (x_a-\mu_{\mathcal{A}})^2,$$

majd a normalizált értéket $\hat{x}_a = (x_a-\mu_{\mathcal{A}})/\sqrt{\sigma^2_{\mathcal{A}}+\epsilon}$ és az affin kimenetet $y_a = \gamma_{\mathcal{C}}\,\hat{x}_a + \beta_{\mathcal{C}}$ módon számolja (a normalizált $\hat{x}_a$-ra, nem a nyers x_a-ra alkalmazva), ahol $\mathcal{C}$ az affin paraméterezés indexhalmaza: azok az indexek, amelyeken a γ és β paraméter osztozik (a BatchNormnál és a GroupNormnál a csatornaindex, a LayerNormnál a teljes (c,h,w) indexhármas). A γ és β alakja a rétegtípustól függ; az $\epsilon>0$ a nullához közeli variancia elleni numerikus védelem. A PyTorch megvalósítás mindhárom alábbi rétegnél torzított, $1/|\mathcal{A}|$ nevezőjű varianciát használ; a BatchNorm2d esetén ezt a [D12] dokumentáció is rögzíti.

BatchNorm2d. A normalizálás minden c csatornára külön-külön az n,h,w indexek felett történik, azaz $\mathcal{A}_c = \{0,\dots,N-1\}\times\{0,\dots,H-1\}\times\{0,\dots,W-1\}$. A tanulható paraméterek alakja $(C,)$. Train módban a batchstatisztika egyben frissíti a futó $\mu_{run}, \sigma^2_{run}$ puffert; eval módban ezt használja (az alapbeállítás `track_running_stats=True`; `False` mellett a réteg eval módban is a batch-statisztikát számolja, és a kimenet batchfüggő marad), így az inference nem függ a batch összetételétől.

LayerNorm normalizált alakkal $[C,H,W]$. Itt minden egyes n mintára a c,h,w indexek felett számolunk [P32], azaz $\mathcal{A}_n = \{0,\dots,C-1\}\times\{0,\dots,H-1\}\times\{0,\dots,W-1\}$. A normalizált alak egy mintára terjed ki, ezért az N batchdimenzió nem része a statisztikának. A PyTorch `LayerNorm(normalized_shape=[C,H,W])` esetén a γ,β paraméterek alakja szintén (C,H,W); nincs futó állapot. Az inference ezért mindig bemenetfüggő, de mintánként független: az n-edik sor kimenetét nem befolyásolja, mi van még a batchben.

GroupNorm. A csatornákat G darab azonos méretű csoportra osztjuk [P33]; ehhez C/G egész kell. A g-edik csoport csatornahalmaza legyen C_g, mérete $|C_g| = C/G$. Egy rögzített (n,g) párra a

$$\mu_{n,g} = \frac{1}{|C_g|HW}\sum_{c\in C_g}\sum_{h,w} X_{n,c,h,w}$$

átlagot és a hozzá tartozó torzított varianciát számoljuk, majd az $X_{n,c,h,w}$ elemeket ezzel normalizáljuk. A γ,β paraméterek alakja $(C,)$; nincs futó állapot. A $G=1$ választás egy mintára LayerNorm jellegű normalizálást ad, a $G=C$ választás pedig csatornánkénti, térbeli normalizálást.

A különböző rétegek nem ugyanazt a statisztikát normalizálják, és ezt a tengelyválasztás rögzíti.

Réteg	Aggregációs indexek	Affin paraméter alakja	Futó állapot
BatchNorm2d	(n,h,w), csatornánként	$(C,)$	van: $(\mu_{run},\sigma^2_{run})$
LayerNorm $[C,H,W]$	(c,h,w), mintánként	(C,H,W)	nincs

Réteg	Aggregációs indexek	Affin paraméter alakja	Futó állapot
GroupNorm (G, C)	$(c \in C_g, h, w)$, mintánként és csoportonként	$(C,)$	nincs

Ez a táblázat a levezetési kötelezettség magja: a réteg viselkedése abból következik, hogy a fenti indexhalmazok mely dimenziókat fogják össze. A BatchNorm az N dimenziót használja, ezért train módban a batchfüggés beépül; a LayerNorm és GroupNorm az N dimenziót nem aggregálja, ezért egy minta kimenete más mintáktól független marad.

Az induló kérdésre a válasz. $N = 1$ esetén a BatchNorm csatornánként a $H \cdot W$ térbeli elemből számol statisztikát, ezért $H \cdot W > 1$ mellett működik, de a statisztika a térbeli elrendezéstől függ; $H = W = 1$ esetén egyetlen értékből nem számítható, és train módban a PyTorch hibát dob (lásd a szélsőesetet). A LayerNorm és a GroupNorm a batch méretétől függetlenül mindig számol statisztikát: a LayerNorm mintánként a $C \cdot H \cdot W$, a GroupNorm mintánként és csoportonként a $(C/G) \cdot H \cdot W$ elemből. Az $N = 1$ tehát önmagában csak a BatchNormot érinti.

18.3.3 Pre-activation és post-activation reziduális blokk

A [P06] 3.1–3.2 szakasza szerint a reziduális blokk rövidített definíciója $y = \mathcal{F}(x) + x$; az eredeti elrendezésben a főág kimenetéhez adjuk a shortcutot, és az összeg után még egy ReLU áll. Ezt poszt-aktivációs formának nevezzük, mert az utolsó nemlinearitás a shortcut hozzáadása után következik.

A pre-aktivációs elrendezés ezzel szemben a normalizálást és a ReLU-t a konvolúció elé helyezi, és az összegzés után már nincs további ReLU. Két konvolúciós, csatornaszám-tartó blokk sematikus definíciója:

$$y = x + W_2 \,\mathrm{ReLU}\left(\mathrm{BN}(W_1 \,\mathrm{ReLU}(\mathrm{BN}(x)))\right),$$

ahol W_1, W_2 a konvolúciós rétegek, és a bias a BatchNorm jelenléte miatt tipikusan elhagyható. A fontos különbség nem pusztán esztétikai: a poszt-aktivációs formában a BatchNorm a konvolúció után, az összegzés előtt normalizál, a pre-aktivációs formában viszont a normalizálás és a nemlinearitás a következő konvolúció bemenetét készíti elő, és az identitás shortcut közvetlenül, normalizálatlanul csatlakozik az összeghez. A teljes pre-aktivációs hálóban a blokkok után, a klasszifikációs fej elé még egy BN–ReLU lépés kerül, mert a blokkon belüli utolsó ReLU a következő blokk elejére tolódott; a [P06b] ezt a globális lépést is a pre-aktivációs recept részének tekinti.

Ebből a szerkezetből azonban nem következik automatikus konvergencia- vagy általánosítási garancia. A pre-aktivációs elrendezés forrása He, Zhang, Ren és Sun (2016) *Identity Mappings in Deep Residual Networks* munkája [P06b]; a [P06] az eredeti, post-aktivációs definíciót adja, önmagában nem forrása a teljes pre-aktivációs receptnek. Mind a post-, mind a pre-aktivációs változatnál az elrendezés előnye empirikus architektúraállítás; a matematikailag bizonyítható rész csak az, amit az operátorok definíciójából levezetünk. A [P07] absztraktja az internal covariate shift csökkentését nevezi meg motivációként, a [P08] absztraktja pedig az optimalizálási táj kedvezőbbé válását hangsúlyozza; e két állítás különböző szakirodalmi igény, és egyik sem támaszkodik teljes bizonyításra ebben az egységben. A jelen alfejezet csak a normalizálási statisztika és a batchfüggés pontos leírására vállalkozik.

18.3.4 Eredeti kézi kis példa

Legyen $X \in \mathbb{R}^{2\times 2\times 1\times 2}$, azaz két minta, két csatorna, egy sor és két oszlop. A négy csatornamátrix:

$$X_{0,0,:,:} = [1,3], \quad X_{0,1,:,:} = [5,7], \quad X_{1,0,:,:} = [-3,-1], \quad X_{1,1,:,:} = [-7,-5].$$

A kézzel levezetett idealizált esetben vegyünk $\epsilon = 0$, $\gamma = 1$, $\beta = 0$ értékeket, így a normalizált érték a tiszta standardizálás. A futtatott PyTorch-rétegekhez később pozitív ϵ értéket adunk: a kézi számolás ezért a numerikus kimenet jó közelítése, de nem a futtatott kód kimenetének bitre pontos előrejelzése.

BatchNorm2d. A $c = 0$ csatornán a négy érték $1, 3, -3, -1$, átlaga 0, torzított varianciája $(1^2 + 3^2 + (-3)^2 + (-1)^2)/4 = 20/4 = 5$. A normalizált értékek:

$$[1/\sqrt{5},\ 3/\sqrt{5},\ -3/\sqrt{5},\ -1/\sqrt{5}].$$

A $c = 1$ csatornán az értékek $5, 7, -7, -5$, átlaga 0, varianciája $(25+49+49+25)/4 = 148/4 = 37$, így a normalizált értékek:

$$[5/\sqrt{37},\ 7/\sqrt{37},\ -7/\sqrt{37},\ -5/\sqrt{37}].$$

LayerNorm $[C, H, W]$**.** A 0-adik minta négy értéke $1, 3, 5, 7$, átlaga 4, varianciája $((1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2)/4 = 20/4 = 5$. A normalizált minta:

$$[-3/\sqrt{5},\ -1/\sqrt{5},\ 1/\sqrt{5},\ 3/\sqrt{5}].$$

Az 1-es minta értékei $-3, -1, -7, -5$, átlaga -4, varianciája szintén 5, normalizált alakja:

$$[1/\sqrt{5},\ 3/\sqrt{5},\ -3/\sqrt{5},\ -1/\sqrt{5}].$$

A két réteg kimenete már ebből a kis példából láthatóan nem azonos, mert a BatchNorm a batch mentén, a LayerNorm az egyes mintákon belül aggregál.

Szélsőeset. $N = 1$ és $H = W = 1$ esetén a BatchNorm csatornánként egyetlen értékkel rendelkezik. A matematikai idealizációban, ha $\epsilon > 0$ és a torzított variancia 0, a normalizált érték nulla, és a kimenet $\gamma \cdot 0 + \beta = \beta$ lenne; $\epsilon = 0$ esetén viszont $0/0$ miatt a kifejezés nem értelmezett. A PyTorch train módban azonban ilyenkor nem ad kimenetet: `ValueError: Expected more than 1 value per channel when training` hibát dob. Eval módban a futó statisztikát használja, és ott nincs ilyen hiba (a kimenet a futó $\mu_{run}, \sigma^2_{run}$ értékektől függ, nem β-ra esik össze). Ha viszont H vagy W nagyobb, a BatchNorm $N = 1$ esetén is rendelkezik térbeli mintákkal, tehát nem degenerálódik automatikusan.

18.3.5 Minimális kód

Munkakönyvtár: `lab/n071/`. Az alábbi kód a fenti kézi példát ellenőrzi PyTorchban. A gyári $\gamma = 1, \beta = 0$ inicializálás miatt a normalizált értékeket látjuk.

```
import torch
import torch.nn as nn

torch.manual_seed(0)

# N=2, C=2, H=1, W=2
x0 = torch.tensor([[[[1.0, 3.0]], [[5.0, 7.0]]]])
x1 = torch.tensor([[[[-3.0, -1.0]], [[-7.0, -5.0]]]])
X = torch.cat([x0, x1], dim=0)

bn = nn.BatchNorm2d(2, eps=1e-5, momentum=0.1)
ln = nn.LayerNorm([2, 1, 2], eps=1e-5)
gn = nn.GroupNorm(1, 2, eps=1e-5)  # G=1: mintánkénti normalizálás

bn.train()
print("BN train:")
print(bn(X))
print("running_mean:", bn.running_mean.tolist())
print("running_var :", bn.running_var.tolist())

ln.eval()  # a LayerNormnek nincs train/eval különbsége
print("LN:")
print(ln(X))

gn.eval()  # a GroupNormnek sincs train/eval különbsége
print("GN, G=1:")
print(gn(X))

# Eval BatchNorm: a running pufferrel normalizál
bn.eval()
print("BN eval, első minta:")
print(bn(X[:1]))
```

A kézi levezetésből számított kulcspontok: a BN train kimenete a $c = 0$ csatornán $[1/\sqrt{5}, 3/\sqrt{5}, -3/\sqrt{5}, -1/\sqrt{5}]$, a $c = 1$ csatornán $[5/\sqrt{37}, 7/\sqrt{37}, -7/\sqrt{37}, -5/\sqrt{37}]$. A futó puffer az első train forward után $c = 0$ esetén $\mu_{run} = 0$ és $\sigma^2_{run} = 0{,}9 \cdot 1 + 0{,}1 \cdot (20/3) \approx 1{,}5667$, $c = 1$ esetén $\sigma^2_{run} = 0{,}9 \cdot 1 + 0{,}1 \cdot (148/3) \approx 5{,}8333$. A PyTorch `BatchNorm2d` a tanítási forward normalizálásához a torzított varianciát, a `running_var` frissítéséhez viszont a korrigált mintavarianciát használja; emiatt az 5 és 37 közvetlen beírása a pufferfrissítésbe hibás volna. A kézi normalizálás $\epsilon = 0$ ideális határesetet használ, a futtatható kód stabilitásból $\epsilon = 10^{-5}$-öt, ezért az értékek kis mértékben eltérnek. A LayerNorm és a $G = 1$ GroupNorm mintánként a fenti normalizált értékeket adja. A kód futtatott kimenetének kulcsértékei (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál):

```
BN train c=0: [0.4472, 1.3416, -1.3416, -0.4472]
BN train c=1: [0.8220, 1.1508, -1.1508, -0.8220]
running_var: [1.566666603088379, 5.833333492279053]
LN: [[-1.3416, -0.4472], [0.4472, 1.3416]] az első minta csatornáin
GN (G=1): az LN-nel azonos
BN eval első minta: [0.7989, 2.3968, 2.0702, 2.8983]
```

A teljes tensor-kiírást a kód adja; a fenti értékek a mért futásból származnak. A hallgató feladata a saját futtatás és az eltérések értelmezése.

18.3.6 Labor: kis batch és egyedi inference összehasonlítása

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A szkript a szakasz kódblokkjából futtatható; a bemenet determinisztikus tenzor, külső adatfájl nem kell.

A labor kérdése: hogyan változik egy rögzített minta kimenete, ha egyedül vagy egy másik mintával egy batchben dolgozzuk fel, BatchNorm és GroupNorm esetén?

Bemenet: determinisztikus $X \in \mathbb{R}^{2\times8\times4\times4}$, `torch.randn` seeddel. Három réteg: `BatchNorm2d(8, momentum=0.1)`, `GroupNorm(2, 8)` és `LayerNorm([8,4,4])`. Módok: train, eval. Rétegenként és módonként két forward (összesen tizenkettő): teljes batch, majd csak az első minta. Ellenőrzés `torch.allclose`, valamint a BN bemeneti és kimeneti csatornaátlagainak összehasonlítása.

```
import torch
import torch.nn as nn

torch.manual_seed(123)
X = torch.randn(2, 8, 4, 4, dtype=torch.float32)

bn = nn.BatchNorm2d(8, momentum=0.1)
gn = nn.GroupNorm(2, 8)
ln = nn.LayerNorm([8, 4, 4])

bn.train(); gn.train(); ln.train()
Y_bn_batch  = bn(X)
Y_bn_single = bn(X[:1])
Y_gn_batch  = gn(X)
Y_gn_single = gn(X[:1])
Y_ln_batch  = ln(X)
Y_ln_single = ln(X[:1])

train_bn_same = torch.allclose(Y_bn_batch[0], Y_bn_single[0], atol=1e-6)
train_gn_same = torch.allclose(Y_gn_batch[0], Y_gn_single[0], atol=1e-6)
train_ln_same = torch.allclose(Y_ln_batch[0], Y_ln_single[0], atol=1e-6)
print("train, batch==single?", train_bn_same, train_gn_same, train_ln_same)  # várt: False True
↪  True
assert not train_bn_same and train_gn_same and train_ln_same

# A BN train forwardok a futó puffert is frissítik, ezért a hívások sorrendje számít.
print("BN bemeneti csatornaátlag (batch):", X.mean(dim=(0, 2, 3)).tolist())
print("BN bemeneti csatornaátlag (single):", X[:1].mean(dim=(0, 2, 3)).tolist())
print("BN kimeneti csatornaátlag (batch):", Y_bn_batch.mean(dim=(0, 2, 3)).tolist())
print("BN kimeneti csatornaátlag (single):", Y_bn_single.mean(dim=(0, 2, 3)).tolist())

bn.eval(); gn.eval(); ln.eval()
Y_bn_ev_batch  = bn(X)
Y_bn_ev_single = bn(X[:1])
Y_gn_ev_batch  = gn(X)
Y_gn_ev_single = gn(X[:1])
Y_ln_ev_batch  = ln(X)
Y_ln_ev_single = ln(X[:1])

eval_bn_same = torch.allclose(Y_bn_ev_batch[0], Y_bn_ev_single[0], atol=1e-6)
eval_gn_same = torch.allclose(Y_gn_ev_batch[0], Y_gn_ev_single[0], atol=1e-6)
eval_ln_same = torch.allclose(Y_ln_ev_batch[0], Y_ln_ev_single[0], atol=1e-6)
print("eval, batch==single?", eval_bn_same, eval_gn_same, eval_ln_same)  # várt: True True True
assert eval_bn_same and eval_gn_same and eval_ln_same
```

A rögzített környezetű ellenőrző futás kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 123):

```
train, batch==single? False True True
BN bemeneti csatornaátlag (batch): [-0.17263254523277283, -0.13501140475273132,
↪  -0.027752883732318878, 0.32633841037750244, -0.16564810276031494, -0.15358713269233704,
↪  -0.15292976796627045, 0.038304973393678665]
BN bemeneti csatornaátlag (single): [0.08628743886947632, -0.12216424942016602,
↪  -0.16029766201972961, 0.2705022096633911, -0.011957749724388123, -0.3053904175758362,
↪  -0.25876715779304504, -0.12444853037595749]
BN kimeneti csatornaátlag (batch): [0.0, -3.725290298461914e-09, -3.725290298461914e-09,
↪  -7.450580596923828e-09, 0.0, -7.450580596923828e-09, 3.725290298461914e-09, 0.0]
BN kimeneti csatornaátlag (single): [-5.587935447692871e-09, 0.0, -7.450580596923828e-09,
↪  1.4901161193847656e-08, 0.0, 4.470348358154297e-08, -7.450580596923828e-09,
↪  1.4901161193847656e-08]
eval, batch==single? True True True
```

A várt, a tengelyválasztásból következő eredmény: train módban a BatchNorm első mintája a teljes batchben más statisztikát használ, mint az egyedi feldolgozásnál, ezért a két fej nem egyezik. A GroupNorm és a LayerNorm kimenete mintánként független, ezért a fejek bitre azonosak. Eval módban a BatchNorm a futó puffert használja, ezért a batch- és egyedi feldolgozás feje bitre azonos lesz; a GroupNorm és a LayerNorm eleve batchfüggetlen. A numerikus egyezés toleranciája `atol=1e-6` a determinisztikus CPU-s float32 számításra; az eval-egyezés ennél szigorúbb, valóban bitre pontos. Fontos: a train-módú `bn(X[:1])` hívás a futó puffert is frissíti, ezért az eval-eredmény függ a hívások sorrendjétől; a kódban az eval a két BN train forward után fut.

A kimenet megerősíti a tengelyválasztást: a BN bemeneti csatornaátlagai eltérnek a batch és az egyedi esetben, a kimeneti átlagok viszont mindkét esetben nullák (a normalizálás miatt); a train BN fej nem egyezik, a GN és LN fej igen; eval módban mindhárom egyezik. A beadandó eredmény: a futtatott script, a nyomtatott logikai értékek és egy rövid értelmezés, amely megnevezi, hogy a train módbeli BatchNorm-eltérés oka a μ, σ^2 batchstatisztika. A fenti logikai értékek a rögzített környezetű ellenőrző futás mért kimenetei; a konkrét futási idő és a bitazonosság a helyi környezettől függ.

18.3.7 Feladatok

Az alábbi feladatok mindegyikéhez add meg a bemenetet, a nehézséget és az elvárt beadandó eredményt.

F1 (fogalmi, nehézség 1). Adott $X \in \mathbb{R}^{2\times4\times3\times3}$. Határozd meg szövegesen, milyen indexhalmaz felett számol átlagot és varianciát a BatchNorm2d, a `LayerNorm([4,3,3])` és a `GroupNorm(2,4)`. Add meg mindhárom esetben az átlag–variancia párok számát is.

F2 (levezetési, nehézség 2). Legyen $X \in \mathbb{R}^{1\times2\times1\times2}$ a következő: $X_{0,0,:,:} = [1, 3]$ és $X_{0,1,:,:} = [5, 7]$. Vezesd le a BatchNorm2d és a LayerNorm $[2, 1, 2]$ kimenetét $\epsilon = 0$, $\gamma = 1$, $\beta = 0$ mellett. Ellenőrizd, hogy a BatchNorm kimenetén minden csatornán nulla az átlag és egységnyi a torzított ($1/n$-es) variancia, a LayerNorm kimenetén pedig a minta négy elemének átlaga nulla, torzított varianciája egy; a korrigált mintavariancia ekkor $4/3$.

F3 (implementációs, nehézség 2). Készíts PyTorch `nn.Module` osztályt `PreActBlock` néven, amely C csatornán 3×3-as, padding 1, stride 1 konvolúciókat használ a BN–ReLU–conv sorrenddel, bias nélkül. Az osztály kimenete $x + \mathcal{F}(x)$. Ellenőrizd a kimeneti alakot $N = 2$, $C = 3$, $H = W = 8$ mellett.

F4 (hibakeresési, nehézség 2). A következő hívás hibát okoz: `nn.GroupNorm(2, 2)` olyan bemenetre, amelynek csatornaszáma 3. A PyTorch 2.15.0+cpu a következő üzenetet adja: `RuntimeError: Expected number of channels in input to be divisible by num_groups, but got input of shape [1, 3, 4, 4] and num_groups=2`. Mi a hiba, mi a következménye, és hogyan javítható úgy, hogy a csatornák száma 3 maradjon? Válaszodban térj ki arra, miért nem oldható meg $G = 2$-vel az oszthatóság megsértése nélkül, és arra is, milyen hibát adna a `nn.GroupNorm(2, 3)` konstruktor már a létrehozáskor.

F5 (kísérleti transzfer, nehézség 3). Tervezz kísérletet a GlyphLab adaton (a függelék A. A GlyphLab adatkészlet szakasza): hasonlítsd össze ugyanazt a kis CNN-t BatchNorm és GroupNorm normalizálással, miközben a batch méretet 32-ről 4-re csökkented. A GlyphLab 180 jelenetből áll, jelenetenként két nézettel (360 kép, 24×24, három orientációosztály); a 240/60/60 képes split jelenetszintű, azaz egy `scene_id` mindkét nézete ugyanabba a splitbe kerül, különben a nézetek átkerülhetnek a tanításból a tesztbe. Add meg a hipotézist, a protokollt, a megfigyelési egységet, a baseline-t (architektúra, optimalizáló, tanulási ráta, epochszám, seedek), a metrikát és a következtetés érvényességi határát. Valós futtatást nem várunk el; a tervnek tartalmaznia kell a seed, split és validációs választás rögzítését. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

18.3.8 Megoldások és indoklás

F1 megoldás. A BatchNorm2d minden c csatornára az $N \times H \times W = 2 \cdot 3 \cdot 3 = 18$ elemből számol statisztikát, tehát csatornánként egy átlag és egy variancia, összesen 4 pár. A LayerNorm minden n mintára a $C \times H \times W = 4 \cdot 3 \cdot 3 = 36$ elemből számol, tehát 2 pár. A GroupNorm $G = 2$ esetén $C/G = 2$ csatorna alkot egy csoportot; minden mintára és csoportra $2 \cdot 3 \cdot 3 = 18$ elem felett aggregál, összesen $N \cdot G = 4$ pár. A paraméteres affin transzformáció ezután a normalizált értékekre broadcastolódik.

F2 megoldás. A BatchNorm2d $N = 1$ esetén a $H \times W = 2$ térbeli elemből számol csatornánként. A $c = 0$ csatorna értékei $1, 3$, átlaga 2, torzított varianciája $((1-2)^2+(3-2)^2)/2 = 1$, normalizált értékek $[-1, 1]$. A $c = 1$ csatorna értékei $5, 7$, átlaga 6, varianciája 1, normalizált értékek $[-1, 1]$. A BN kimenet tehát $[[[-1, 1]], [[-1, 1]]]$. A LayerNorm a minta mind a négy értékére $1, 3, 5, 7$ számol: átlag 4, variancia $((-3)^2 + (-1)^2 + 1^2 + 3^2)/4 = 20/4 = 5$. A normalizált kimenet $[-3/\sqrt{5}, -1/\sqrt{5}, 1/\sqrt{5}, 3/\sqrt{5}]$. Ellenőrzés: a BN csatornaátlagai 0, a csatornavarianciák $(1 + 1)/2 = 1$; az LN négy elemén az átlag 0, a torzított variancia $((-3)^2\sqrt{5}^{-2} + (-1)^2\sqrt{5}^{-2} + 1^2\sqrt{5}^{-2} + 3^2\sqrt{5}^{-2})/4 = (9+1+1+9)/(4 \cdot 5) = 20/20 = 1$. A korrigált mintavariancia ugyanezen a négy elemen $4/3$, ami nem tévesztendő össze a normalizálásban használt torzított varianciával.

F3 megoldás.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PreActBlock(nn.Module):
    def __init__(self, C):
        super().__init__()
        self.bn1 = nn.BatchNorm2d(C)
        self.conv1 = nn.Conv2d(C, C, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(C)
        self.conv2 = nn.Conv2d(C, C, 3, padding=1, bias=False)

    def forward(self, x):
        out = self.conv1(F.relu(self.bn1(x)))
```

```
        out = self.conv2(F.relu(self.bn2(out)))
        return x + out

torch.manual_seed(7)
block = PreActBlock(3)
x = torch.randn(2, 3, 8, 8, dtype=torch.float32)
y = block(x)
print(y.shape)  # Várt: torch.Size([2, 3, 8, 8])
assert tuple(y.shape) == (2, 3, 8, 8)
```

Alakellenőrzés: a BN nem változtat alakot, a ReLU sem, a conv padding 1 és stride 1 mellett megőrzi a 8×8 térbeli méretet. Az összegzés csak azonos alakú tenszorokra értelmes, ezért az x és $F(x)$ alakjának egyeznie kell.

F4 megoldás. A `GroupNorm(num_groups, num_channels)` második argumentuma a várt csatornaszám. A megadott hívásban két feltétel sérül: a réteg 2 csatornát vár, a bemenetnek viszont 3 csatornája van, és a bemenet csatornaszáma (3) nem osztható a csoportszámmal (2). A PyTorch 2.15.0+cpu a forwardban előbb az oszthatóságot ellenőrzi, ezért a mért üzenet: `RuntimeError: Expected number of channels in input to be divisible by num_groups, but got input of shape [1, 3, 4, 4] and num_groups=2`. Ha a `num_channels` értéket 3-ra állítjuk, a konstruktor már létrehozáskor elutasítja a $G = 2$ választást: `ValueError: num_channels (3) must be divisible by num_groups (2)` (szintén mért üzenet PyTorch 2.15.0+cpu alatt). A lehetséges javítás: `nn.GroupNorm(1, 3)` vagy `nn.GroupNorm(3, 3)`; mindkettő osztható csoportosítást ad. Ellenőrzésképpen egy $X \in \mathbb{R}^{1\times3\times4\times4}$ forwardnak hiba nélkül kell lefutnia, és a kimenet alakjának meg kell egyeznie a bemenetével.

F5 megoldásminta. Hipotézis: a kis batch (4) a BatchNormot jobban rontja, mint a GroupNormot, mert a BN statisztikája kevesebb mintából készül, míg a GN csoportonként a térbeli elemeket is aggregálja, ezért a statisztikája kevésbé zajos. Kérdés: a batch méret csökkenése melyik normalizálást érinti jobban? Megfigyelési egység a `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik; a split rögzített és jelenetszintű: train 240, val 60, test 60 kép (120/30/30 jelenet), egy jelenet mindkét nézete ugyanabba a splitbe kerül. Baseline: a 38.2. Modell, training és best checkpoint egységben definiált kis CNN (SmallCNN) architektúrája, BatchNorm rétegekkel, batch méret 32, Adam optimalizáló 10^{-3} tanulási rátával, 20 epoch, három rögzített seed (például 20261020, 20261021, 20261022); a GroupNorm-változat ugyanazt az architektúrát használja BN helyett GN-nel, és a paraméterszámot ellenőrizzük, hogy a két modell azonos kapacitású legyen. Változó tényező: a normalizálás típusa (BN vagy GN) és a batch méret (32 és 4). Jelentett metrika: validációs accuracy és validációs loss; a checkpoint kiválasztása kizárólag a validációs loss minimuma alapján történik, és a tesztet egyszer, a végleges kiválasztás után értékeljük (a teszt accuracy nem játszik szerepet a választásban). Erőforrás: CPU, float32. A következtetés korlátja: a GlyphLab szintetikus, 24×24-es, egycsatornás adat, ezért az eredmény nem általánosítható természetes képekre; továbbá a batch méret és a normalizálás hatása összefonódhat a tanulási ráta érzékenységével, ezért az egyes konfigurációkhoz érdemes rövid tanulásiráta-keresést is rögzíteni, és a batch méret 32-ről 4-re csökkentését a két normalizálásnál azonos recepttel kell futtatni.

18.3.9 Tipikus tévedés ellenpéldával

Gyakori tévedés, hogy a BatchNorm, LayerNorm és GroupNorm „ugyanazt a standardizálást végzi, csak más néven”. A fenti $N = 2, C = 2, H = 1, W = 2$ példa ezt cáfolja. A BatchNorm a $c = 0$ csatornához a batch összes mintáját használja, ezért a $[1, 3, -3, -1]$ értékekre normalizál; a LayerNorm a 0-adik minta $[1, 3, 5, 7]$ értékeire. A kimenetek már a nulladik mintán is különböznek: a BN első eleme $1/\sqrt{5}$, az LN első eleme $-3/\sqrt{5}$. A hiba következménye lehet, hogy a felhasználó eval módban a LayerNormtól futó statisztikát vár (holott a LayerNormnek nincs futó állapota;

kimenete mindig a bemenettől függ), vagy a BatchNormot egyetlen mintán úgy értelmezi, mint az összes csatorna normalizálását; mindkettő téves tengelyválasztásból fakad.

18.3.10 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrizd, hogy a normalizálási képletben az ϵ a varianciához adódik, nem a szóráshoz; a variancia torzított; a BN train egyszerre frissíti a futó puffert és használja a batchstatisztikát. A 18.2. Batch normalization train és eval módban alfejezetből visszaidézve a futó puffer momentumos rekurziója $r_{new} = (1-\alpha)r_{old} + \alpha\,\hat{r}$, ahol r a futó statisztika (átlag vagy variancia), $\hat{r}$ az aktuális batchből számolt érték, r_{old} a frissítés előtti pufferérték, $\alpha \in (0, 1]$ pedig a PyTorch `momentum` paramétere (alapértelmezése 0,1); ez ugyanaz az $1-\alpha$ súlyú frissítés, mint a 18.2. Batch normalization train és eval módban képletében, és nem azonos az optimalizáló momentumával. A 6.3. Sztochasztikus és minibatch gradiens alfejezetből tudjuk, hogy a minibatchstatisztika a teljes adathoz képest zajos becslő, és ez a normalizáló réteg statisztikájára is vonatkozik. A 14.3. Paraméterszám, MAC és aktivációmemória paraméter- és MAC-számítását finomítja, hogy a BN/GN elemenkénti műveletei nem súly-MAC-ok, de a statisztikaredukció és a normalizálás szintén számítási költséggel jár. A 18.1. Reziduális blokk és identitásút alfejezetben tárgyalt reziduális blokkban a normalizálás elhelyezése éppen a pre-aktivációs blokk kapcsán válik tervezési döntéssé.

18.3.11 Rövid összefoglaló

A normalizáló rétegek közös vázat használnak, de eltérő tengelyeken aggregálnak. A BatchNorm2d csatornánként a batch és a térbeli pozíciók felett normalizál, tanulható $(C,)$ paraméterekkel és futó pufferekkel; eval módban a puffert használja. A LayerNorm mintánként az összes csatorna és térbeli elem felett normalizál, paramétereinek alakja (C, H, W), nincs futó állapota. A GroupNorm csatornacsoportonként aggregál, $(C,)$ paraméterekkel. A pre-aktivációs blokk a BN–ReLU–conv sorrendet használja, és a shortcutra nem kerül normalizálás; az előnyök azonban empirikus architektúraállítások, nem bizonyított konvergenciagaranciák. A feladatmegoldások és a labor a batchfüggést a tengelyválasztásból vezetik le.

18.3.12 Következő alkalmazás

A 18.4. Reziduális blokk ablatív vizsgálata egységben reziduális blokkok ablatív vizsgálatát végezzük el: ugyanazon a kis GlyphLab-feladaton, rögzített seedekkel mérjük össze a poszt-aktivációs és pre-aktivációs elrendezést, valamint a normalizálás elhagyásának hatását. Ott az itt definiált PreActBlock lesz az egyik változat, és a mérési állításokat kísérleti naplóval, nyers epochonkénti metrikákkal kell alátámasztani.

18.3.13 Felhasznált források

- [P06] He, K., Zhang, X., Ren, S. és Sun, J. (2016). *Deep Residual Learning for Image Recognition.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016); a preprint 2015-ben jelent meg arXiv:1512.03385 számon. https://arxiv.org/abs/1512.03385.
- [P06b] He, K., Zhang, X., Ren, S. és Sun, J. (2016). *Identity Mappings in Deep Residual Networks.* ECCV 2016; arXiv:1603.05027. https://arxiv.org/abs/1603.05027.
- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html.

- [P08] Santurkar, S., Tsipras, D., Ilyas, A. és Madry, A. (2018). *How Does Batch Normalization Help Optimization?* Advances in Neural Information Processing Systems 31 (NeurIPS 2018). arXiv:1805.11604. https://arxiv.org/abs/1805.11604.
- [D12] PyTorch contributors. *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30.
- [P32] Ba, J. L., Kiros, J. R. és Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450. https://arxiv.org/abs/1607.06450.
- [P33] Wu, Y. és He, K. (2018). *Group Normalization*. Proceedings of the European Conference on Computer Vision (ECCV 2018). arXiv:1803.08494. https://arxiv.org/abs/1803.08494.

18.4 Reziduális blokk ablatív vizsgálata

18.4.1 Cél, előismeret és induló kérdés

A 18.1. Reziduális blokk és identitásút alfejezetben levezettük a reziduális blokk $y = F(x) + x$ rövidítését, a 18.3. Pre-activation, kis batch és alternatív normalizálás alfejezetben pedig a normalizáló rétegek tengelyválasztását és a pre-aktivációs elrendezést. Ebben az egységben azt a kérdést tesszük mérhetővé, amelyet a képlet önmagában nem dönt el: **vajon a skip kapcsolat ténylegesen mérhető taníthatósági előnyt ad-e egy kis, ellenőrzött feladaton, vagy csak az adott recept szerencséje?** Az ehhez szükséges eszköz az ablatív vizsgálat.

Előismeret: a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből a kontrollált összehasonlítás kerete, a 6.4. Momentum és adaptív optimizálók alfejezetből az AdamW frissítési szabálya, a 18.3. Pre-activation, kis batch és alternatív normalizálás alfejezetből a BatchNorm train/eval viselkedése és a blokkok sorrendje. Induló becslés: ha a plain háló minden rétege kicsiny Jacobi-normával továbbítja a hibát, a korai rétegek gradiense eltűnik; a reziduális út ezt a szorzást megkerüli. A kérdés az, hogy ez a lokális érvelés a tényleges tanítási görbén is látszik-e.

18.4.2 Ablatív vizsgálat, matched recipe és az izoláló kontroll

Definíció (abláció). Az *ablatív vizsgálat* (ablation study) olyan kísérlet, amelyben egy architektúra vagy tanítási recept egyetlen komponensét távolítjuk el vagy cseréljük ki, a többi döntést változatlanul hagyva, és a változatokat ugyanazon a protokollon mérjük össze.

Definíció (matched recipe). A *matched recipe* olyan kontroll, amelyben a vizsgált tényezőn kívül minden tanítási döntés azonos: ugyanaz az adat és split, súlyinicializálás, seed-készlet, optimizer és súlycsökkentés, tanulási ráta és ütemezés, batchméret, epochszám, checkpoint-választási szabály és végső tesztelési eljárás. A 17.4. Architektúra összehasonlítása kontrollált keretben ottani lemmájának közvetlen alkalmazásaként a költségkeretet a kutatási kérdéshez kell igazítani: mivel a shortcut paramétermentes összeadás, a *paraméterkeret* itt automatikusan egyezik; emellett a *számítási keret* is közel azonos. A MAC (multiply–accumulate, szorzás–összeadás) műveletszámban egy $C \to C$ konvolúció $k \times k$ kernellel, $H \times W$ kimeneti méreten C^2k^2HW MAC; a blokkban két ilyen konvolúció van, ehhez képest a reziduális összeadás CHW művelet, azaz a konvolúciós költség $1/(2Ck^2)$ hányada, ami $C = 16$, $k = 3$ mellett elhanyagolható.

A vizsgált faktor tehát egy bináris változó:

$$S \in \{\text{plain}, \text{residual}\}.$$

A plain blokk

$$B_P(x;\theta) = F(x;\theta),$$

a reziduális blokk pedig

$$B_R(x;\theta) = F(x;\theta) + x,$$

ahol $x \in \mathbb{R}^{C\times H\times W}$ a blokk bemeneti aktivációja (C a csatornaszám, H és W a térbeli kiterjedés), θ pedig a blokk konvolúciós és normalizációs paramétereit jelöli (így elkerüljük, hogy a súlyok jele összekeveredjen a W térbeli szélességgel). A GlyphLab-hálóban (a függelék A. A GlyphLab adatkészlet szakasza szerinti adaton) a térbeli méret és a csatornaszám blokkon belül állandó, ezért az identitás-shortcut közvetlenül használható; vetítő-shortcutra most nincs szükség.

A kísérleti egység a GlyphLab **scene_id**-ja: egy jelenet két nézete ugyanabba a splitbe kerül, a bootstrap és az aggregálás ezért scene-szintű (5.4. Függő képek, seedek és resampling). A splitet a GlyphLab adatleírása (specifikációja, A. A GlyphLab adatkészlet) rögzíti: 120/30/30 jelenet train/val/test, osztályonként kiegyenlítve.

18.4.3 Hipotézis és a gradiensnorma levezetése

A kísérlet előtt rögzítjük a cáfolható hipotéziseket:

- **H1 (gradienses mechanizmus):** azonos kezdeti súlyok mellett a plain háló első blokkjának teljes paraméter-gradiensnormája a tanítás elején kisebb, mint a reziduális háló azonos blokkjáé. A levezetés a blokkba érkező **aktivációgradienst** (g_l) írja le; a paraméter-gradiens ehhez a blokk paraméter-Jacobiján át kapcsolódik: $\nabla_{\theta_l} L = J_{\theta_l}(h_{l-1})^\top g_l$ ($J_{\theta_l} = \partial F_l/\partial\theta_l$), és plain, illetve reziduális blokkra is ugyanez a képlet, mert $h_l = F_l$, illetve $h_l = h_{l-1} + F_l$ esetén $\partial h_l/\partial\theta_l = \partial F_l/\partial\theta_l$. A H1 mérése ezért a $\|g_l\|$ és a paraméter-gradiensnorma együttes naplózása. A H1 a lokális lineáris modellből származó heurisztika, nem bizonyított globális állítás.
- **H2 (taníthatóság):** a reziduális változat seedenkénti legjobb validációs losszának seedek feletti átlaga nem nagyobb, mint a plain változaté; a seedenkénti páros különbségeket is jelentjük, nem csak az átlagot.

Döntési szabály. A szabályt a futás előtt rögzítjük. H1 esetén a három seed páros `first_block_norm` arányát (reziduális/plain) tekintjük: a hipotézist akkor tekintjük támogatottnak, ha mindhárom seednél az arány nagyobb 1-nél, és az arányok átlaga is nagyobb 1-nél; ellenkező esetben az eredményt leíró megfigyelésként jelentjük, nem igazolt mechanizmusként. H2 esetén minden seedre és minden validációs jelenetre kiszámítjuk a két modell átlagos veszteségét, képezzük a jelenetszintű páros különbséget, és a jeleneteken 95%-os bootstrap konfidenciaintervallumot számolunk (23.3. Páros összehasonlítás és csoportszintű intervallum); a hipotézist akkor tekintjük támogatottnak, ha a három seed mindegyikében az intervallum teljes egészében 0 alatt van. Ha valamelyik intervallum tartalmazza a 0-t, nem állítunk kimutatható különbséget; ha valamelyik teljes egészében 0 felett van, a plain változat jobb ezen a futáson. Három seed mellett nem „eloszlásról", hanem három páros megfigyelésről beszélünk, és a seedek közti szórást is jelentjük.

A H1 egy lokális lineáris modellből vezethető le. Legyen a háló mélysége T, a blokk kimenete $h_l = B_l(h_{l-1})$, a skalár veszteség L gradiense pedig $g_l = \nabla_{h_l} L$, amelynek alakja megegyezik h_l alakjával. A láncszabály szerint

$$g_{l-1} = J_{B_l}(h_{l-1})^\top g_l,$$

ahol J_{B_l} a blokk Jacobi-mátrixa. A plain blokkra $J_{B_l} = J_{F_l}$. Olyan pontokban, ahol a ReLU-k aktív mintázata rögzített (és a BatchNorm adott batch-statisztikája mellett), a blokk lokálisan affin leképezés; a konvolúció linearitása (15.1. Súly- és biasgradiens) miatt a Jacobi az adott pontban a súlyokból és a BN-statisztikából számítható. Ez lokális és feltételes állítás: a ReLU-minta változása és a BN batchfüggése miatt a Jacobi pontról pontra változik, ezért a továbbiakban csak azt tesszük fel, hogy létezik $\rho \in [0, 1)$ felső korlát minden rétegre.

Lemma (eltűnő gradiens a plain láncban, lokális lineáris közelítés). Ha minden $l = 1, \dots, T$ rétegre $\|J_{F_l}(h_{l-1})^\top\|_2 \le \rho < 1$, akkor a plain hálóra

$$\|g_0\|_2 \le \rho^T \|g_T\|_2.$$

Bizonyítás. Indukcióval: $\|g_{l-1}\|_2 = \|J_{F_l}^\top g_l\|_2 \le \rho \|g_l\|_2$, amit T-szer alkalmazva kapjuk az állítást. □

A reziduális blokkra $B_R(h) = h + F(h; W)$, ezért differenciálható pontban

$$J_{B_R}(h) = I + J_F(h),$$

tehát a backward lépés

$$g_{l-1} = g_l + J_F(h_{l-1})^\top g_l.$$

Az upstream gradiens g_l tehát **változatlanul** megjelenik a kimenő gradiensben: az identitásút minden blokkban hozzáadja a head felől érkező hibát. A teljes szorzat

$$g_0 = \left(\prod_{l=1}^{T} (I + A_l^\top) \right) g_T, \qquad A_l = J_{F_l}(h_{l-1}),$$

ahol a szorzat minden tagja tartalmazza az I összeadandót. Ha $\|A_l^\top\|_2 \le \rho < 1$, akkor ez a szorzat $I + \sum_l A_l^\top + O(\rho^2)$ alakú, ahol az $O(\rho^2)$ maradék konstansa T-től függ (a másod- és magasabb rendű tagok száma T-nel nő), ezért a becslés rögzített T mellett érvényes; a head-gradiens nem kényszerül kizárólag kicsiny Jacobi-tényezőkön áthaladni. Ha viszont $\|A_l^\top\|_2 \le \rho < 1$, akkor minden $I + A_l^\top$ legkisebb szinguláris értéke legalább $1 - \rho$, így $\|g_0\|_2 \ge (1 - \rho)^T \|g_T\|_2$ alsó korlát van; ez azonban nagy T vagy $\rho \approx 1$ mellett gyenge (a kioltás csak ezen feltétel mellett zárható ki). A plain eset felső, a reziduális eset alsó korlát, ezért a kettő közvetlenül nem hasonlítható össze; az állítás csak annyit mond, hogy a reziduális gradiens nem csillapodik $(1 - \rho)^T$ alá, míg a plain legfeljebb ρ^T-ig csökken. Az állítás lokális lineáris közelítésből levezetett heurisztikus mechanizmus, nem globális konvergencia- vagy optimalizálási tétel. A H1 mérési célja éppen az, hogy ezt a mechanizmust empirikusan ellenőrizze, nem pedig az, hogy általános tételként igazolja.

18.4.4 Számított kis példa

Legyen $T = 2$, skalár közelítésben $F_l(h) = a_l h$ differenciálható pontban, $a_1 = a_2 = 0{,}2$, és a head-gradiens $g_2 = 1$. A plain láncra

$$g_0 = a_1 a_2 \, g_2 = 0{,}04,$$

a reziduálisra

$$g_1 = (1 + a_2) g_2 = 1{,}2, \qquad g_0 = (1 + a_1) g_1 = 1{,}44.$$

Ugyanazokkal a blokkparaméterekkel a korai réteg gradiense a reziduális esetben több mint harmincszorosa a plain értéknek. A példa a lemmát illusztrálja, de nem helyettesíti: valódi hálóban a ReLU-k aktív mintázata lépésenként változik, és a Jacobi-norma $\rho < 1$ feltételét nem ellenőriztük. Fontos, hogy ez a példa a lemma **egyenlőségi esete**: a lineáris F_l és $a_1 = a_2 = 0{,}2$ mellett $\|J_{F_l}^{\top}\|_2 = \rho = 0{,}2$ minden blokkra, ezért a plain becslésben $\|g_0\| = \rho^2 \|g_2\|$ egyenlőséggel teljesül, nem csak felső korlátként. A reziduális alsó korlát itt nem éles, mert a tényleges érték $(1 - \rho)^2 = 0{,}64$ felett van. Nemlineáris hálóban a becslés általában nem éles.

18.4.5 A labor: plain és reziduális háló azonos recepttel

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy`, `metadata.csv` és `corrupted_test.npy` fájlja; a `n072_resablation.py` futtatható (`python n072_resablation.py`).

A labor a GlyphLab háromosztályos orientációfeladatát használja (13.1. Kép mint diszkrét többcsatornás jel, 25.1. Teljes képosztályozó adatfolyam). A bemenet float32 NCHW $X \in \mathbb{R}^{N \times 1 \times 24 \times 24}$, a cél int64 $y \in \{0, 1, 2\}^N$; a modell logitot ad, CrossEntropyLoss előtt nincs softmax ([D07]).

Az adatfájlok szerkezete (a függelék A. A GlyphLab adatkészlet szakasza szerint): `images.npy` alakja $(360, 1, 24, 24)$, float32, értékkészlet $[0, 1]$; `labels.npy` alakja $(360,)$, int64, az orientáció osztályindexe (0 = függőleges, 1 = vízszintes, 2 = átlós); `metadata.csv` 360 rekordja az `index` (a teljes képtömb sorindexe, nem a CSV sorszáma), a `scene_id` (`G<label>-<scene>` alakú string), a `view_id`, a `label`, a `split` és a befoglaló doboz koordinátáit tartalmazza; `corrupted_test.npy` a 60 tesztkép rontott változata a metadata test-sorainak sorrendjében. A labor a `metadata.csv` `index` és `split` oszlopára támaszkodik, ezért nem függ a fájlok fizikai sorrendjétől.

Protokoll (a futás előtt rögzítendő). Változó faktor: S. Rögzített: GlyphLab split, seedek $\{0, 1, 2\}$, AdamW lr $= 10^{-3}$, weight_decay $= 10^{-4}$ (6.4. Momentum és adaptív optimizálók), batch 64, 40 epoch, best checkpoint = legkisebb val loss, a teszt csak a checkpoint kiválasztása után. A clean teszt a test split 60 képét, a corrupt teszt ugyanezen jelenet–nézet párok rontott változatát (`corrupted_test.npy`) jelenti; a páros kiértékelés részleteit a 38.3. Végső teszt és páros korrupció tárgyalja. A BatchNorm train alatt batch-statisztikát, eval alatt futó puffert használ [D12]; a réteg bevezetésének motivációját és hatásvizsgálatát a [P07], illetve a [P08] tárgyalja. A mérést `model.eval()` módban végezzük.

Az architektúra: **stem** (a háló bemenetet feldolgozó első konvolúciója) Conv($1 \to 16$, $k = 3$, pad=1), öt blokk, **head** (a klasszifikációs kimenetet adó fej) AdaptiveAvgPool2d(1) és Linear($16 \to 3$). Minden blokk két Conv($16 \to 16$, $k = 3$, pad=1, bias=False) réteget, két

BatchNorm2d(16) réteget és ReLU-kat tartalmaz; a reziduális változat a második BN után hozzáadja a bemenetet, majd ReLU következik (poszt-aktivációs blokk a 18.3. Pre-activation, kis batch és alternatív normalizálás jelölésével). A paraméterszám mindkét változatra:

Rész	Számítás	Paraméter
Stem Conv	$1 \cdot 16 \cdot 9 + 16$	160
Egy blokk	$2 \cdot (16 \cdot 16 \cdot 9) + 2 \cdot 2 \cdot 16$	4672
Head Linear	$16 \cdot 3 + 3$	51
Öt blokkal összesen	$160 + 5 \cdot 4672 + 51$	**23 571**

A labor kódja a `lab/` munkakönyvtárban:

```
# lab/n072_resablation.py
# Futtatás bárhonnan: python n072_resablation.py --epochs 40 --seeds 0 1 2
# Az adat alapértelmezésben a script melletti data/ könyvtár.
import argparse, json, pathlib
import numpy as np
import torch
import torch.nn as nn

BASE = pathlib.Path(__file__).resolve().parent

def make_block(ch, residual):
    class Block(nn.Module):
        def __init__(self):
            super().__init__()
            self.conv1 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
            self.bn1 = nn.BatchNorm2d(ch)
            self.conv2 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
            self.bn2 = nn.BatchNorm2d(ch)
            self.residual = residual
        def forward(self, x):
            h = self.bn1(self.conv1(x)).relu()
            h = self.bn2(self.conv2(h))
            if self.residual:
                h = h + x
            return h.relu()
    return Block()

class Net(nn.Module):
    def __init__(self, blocks=5, ch=16, residual=True, num_classes=3):
        super().__init__()
        self.stem = nn.Conv2d(1, ch, 3, padding=1)
        self.blocks = nn.Sequential(*[make_block(ch, residual) for _ in range(blocks)])
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(ch, num_classes))
    def forward(self, x):
        return self.head(self.blocks(self.stem(x).relu()))

def grad_norm(modules):
    sq = 0.0
    for m in modules:
        for p in m.parameters():
            if p.grad is not None:
                sq += p.grad.detach().float().pow(2).sum().item()
    return float(np.sqrt(sq))

def evaluate(model, X, y, loss_fn, device):
    model.eval()
    with torch.no_grad():
```

```
        logits = model(X.to(device))
        return (loss_fn(logits, y.to(device)).item(),
                (logits.argmax(1) == y.to(device)).float().mean().item())

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", type=pathlib.Path, default=BASE / "data")
    ap.add_argument("--epochs", type=int, default=40)
    ap.add_argument("--seeds", nargs="+", type=int, default=[0, 1, 2])
    ap.add_argument("--device", default="cpu")
    args = ap.parse_args()

    imgs = np.load(args.data / "images.npy").astype(np.float32)
    labels = np.load(args.data / "labels.npy").astype(np.int64)
    meta = np.genfromtxt(args.data / "metadata.csv", delimiter=",", dtype=None,
                         names=True, encoding="utf-8")
    # A sorrendre nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
    if not (len(meta) == len(imgs) == len(labels)):
        raise ValueError("a metadata.csv és a tömbök hossza eltér")
    idx_all = meta["index"].astype(np.int64)
    if not np.array_equal(np.sort(idx_all), np.arange(len(imgs))):
        raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
    split = np.asarray(meta["split"]).astype(str)
    train_idx = torch.tensor(idx_all[split == "train"], dtype=torch.long)
    val_idx = torch.tensor(idx_all[split == "val"], dtype=torch.long)
    test_idx = torch.tensor(idx_all[split == "test"], dtype=torch.long)
    X = torch.from_numpy(imgs)
    y = torch.from_numpy(labels)
    Xco = torch.from_numpy(
        np.load(args.data / "corrupted_test.npy").astype(np.float32))
    if len(Xco) != len(test_idx):
        raise ValueError("a corrupt teszt hossza nem egyezik a test splittel")
    # A corrupt teszt a metadata.csv test sorainak sorrendjében van
    # (manifest.json: corrupt_order), ezért a test_idx pozícióihoz párosul.
    device = torch.device(args.device)
    loss_fn = nn.CrossEntropyLoss()
    assert sum(p.numel() for p in Net(residual=False).parameters()) == 23571
    assert sum(p.numel() for p in Net(residual=True).parameters()) == 23571
    report = {}

    for seed in args.seeds:
        for residual in (False, True):
            # A matched recipe és a H1 azonos kezdeti súlyokat kíván: minden
            # modell ugyanabból a seedből indul, ezért az init és a shuffle
            # sorrendje a két ágban azonos.
            torch.manual_seed(seed)
            model = Net(residual=residual).to(device)
            opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
            best_val = float("inf")
            best_epoch = -1
            best_state = None
            history = {"first_step": None, "epochs": []}
            act_grad = {}
            def first_block_hook(module, inp, out):
                out.register_hook(
                    lambda g: act_grad.__setitem__(
                        "norm", g.detach().float().norm().item()))
            hook = model.blocks[0].register_forward_hook(first_block_hook)
            for ep in range(args.epochs):
                model.train()
                order = torch.randperm(len(train_idx))
                train_loss_sum = 0.0
                n_batches = 0
```

```
                for st in range(0, len(train_idx), 64):
                    idx = order[st:st+64]
                    xb = X[train_idx[idx]].to(device)
                    yb = y[train_idx[idx]].to(device)
                    opt.zero_grad()
                    loss = loss_fn(model(xb), yb)
                    loss.backward()
                    if ep == 0 and st == 0:
                        history["first_step"] = {
                            "step": 0,
                            "stem_norm": grad_norm([model.stem]),
                            "first_block_norm": grad_norm([model.blocks[0]]),
                            "first_block_act_grad_norm": act_grad.get("norm", 0.0),
                            "last_block_norm": grad_norm([model.blocks[-1]]),
                            "head_norm": grad_norm([model.head]),
                        }
                        hook.remove()
                    opt.step()
                    train_loss_sum += loss.item()
                    n_batches += 1
                val_loss, val_acc = evaluate(model, X[val_idx], y[val_idx],
                                             loss_fn, device)
                history["epochs"].append({
                    "epoch": ep, "train_loss": train_loss_sum / n_batches,
                    "val_loss": val_loss, "val_acc": val_acc})
                if val_loss < best_val:
                    best_val = val_loss
                    best_epoch = ep
                    best_state = {k: v.detach().clone()
                                  for k, v in model.state_dict().items()}
            ckpt = args.data.parent / (
                f"n072_seed{seed}_{'residual' if residual else 'plain'}.pt")
            torch.save(best_state, ckpt)
            model.load_state_dict(best_state)
            test_loss, test_acc = evaluate(model, X[test_idx], y[test_idx],
                                           loss_fn, device)
            corrupt_loss, corrupt_acc = evaluate(model, Xco, y[test_idx],
                                                 loss_fn, device)
            report[f"seed{seed}_{'residual' if residual else 'plain'}"] = {
                "param_count": sum(p.numel() for p in model.parameters()),
                "checkpoint": ckpt.name,
                "best_val_loss": best_val,
                "best_epoch": best_epoch,
                "test_loss": test_loss,
                "test_acc": test_acc,
                "corrupt_loss": corrupt_loss,
                "corrupt_acc": corrupt_acc,
                "first_step": history["first_step"],
                "epochs": history["epochs"],
            }
    out = args.data.parent / "n072_report.json"
    out.write_text(json.dumps(report, indent=2))
    print(out)

if __name__ == "__main__":
    main()
```

A kód szándékosan minimális: saját indexelésű batch-képzést használ `DataLoader` helyett, a legjobb validációs checkpointot viszont ténylegesen menti, a tesztkiértékelést pedig a 11.4. Egy teljes PyTorch tanítási iteráció és 12.2. Checkpoint és folytathatóság receptjét követve csak a kiválasztás után végzi. Futásidő-tervezés: a 240 képes train split batch 64 mellett epochonként 4 lépést ad, így 40 epoch és 6 futás összesen 960 lépés; a mért futásidő ezzel arányos, gépenként

és szálszámtól függően eltérhet. A labor **beadandó eredménye**: a `n072_report.json` nyers epochonkénti és lépésenkénti adatai, a mentett checkpointok, valamint a tesztkiértékelés a kiválasztott checkpointokon. Az alábbi kivonat a rögzített környezetű ellenőrző futás (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seedek 0, 1, 2; a futás mért ideje kb. 1 m 22 s) tényleges `n072_report.json` fájljából származik. A számok négy tizedesre kerekítettek; a teljes fájl seedenként 40 epoch rekordját és a `first_step` minden gradiensnormáját tartalmazza:


```
{"seed0_plain": {"best_val_loss": 0.0988, "best_epoch": 36,
  "test_acc": 1.0, "corrupt_acc": 0.6833,
  "first_step": {"first_block_norm": 0.6346, "head_norm": 0.1977}},
 "seed0_residual": {"best_val_loss": 0.0072, "best_epoch": 36,
  "test_acc": 1.0, "corrupt_acc": 0.6,
  "first_step": {"first_block_norm": 1.1053, "head_norm": 0.5606}},
 "seed1_plain": {"best_val_loss": 0.0655, "best_epoch": 39,
  "test_acc": 1.0, "corrupt_acc": 0.8667,
  "first_step": {"first_block_norm": 0.5817, "head_norm": 0.4454}},
 "seed1_residual": {"best_val_loss": 0.0060, "best_epoch": 39,
  "test_acc": 1.0, "corrupt_acc": 0.4333,
  "first_step": {"first_block_norm": 1.1921, "head_norm": 1.8687}},
 "seed2_plain": {"best_val_loss": 0.0752, "best_epoch": 33,
  "test_acc": 1.0, "corrupt_acc": 0.95,
  "first_step": {"first_block_norm": 0.2846, "head_norm": 0.2701}},
 "seed2_residual": {"best_val_loss": 0.0055, "best_epoch": 35,
  "test_acc": 1.0, "corrupt_acc": 0.7167,
  "first_step": {"first_block_norm": 1.0379, "head_norm": 1.6064}}}
```


A mért adatokban a reziduális változat mindhárom seednél alacsonyabb legjobb validációs losszt ért el, és a `first_block_norm` is mindhárom seednél nagyobb volt, mint a plain változatban. A clean teszt pontosság mind a hat futásban 1,0 lett, ezért a tesztkészlet ezen a méreten nem különbözteti meg a modelleket; a corrupt teszt pontossága viszont seedenként és változatonként jelentősen szór. Ebben a futásban a corrupt teszt pontossága mindhárom seednél a plain változatban volt magasabb (0,6833 vs. 0,6; 0,8667 vs. 0,4333; 0,95 vs. 0,7167), ami a reziduális változat nagyobb zajérzékenységére utal ezen a kis adaton; ez egy három seeddel végzett futás megfigyelése, nem általános törvény. A tanulói futás ezekkel a számokkal összevethető; a lényeg a kiválasztási protokoll betartása, nem a fenti értékek betű szerinti reprodukálása. A három seed kis minta: a páros különbségekre scene-szintű bootstrap 95%-os konfidenciaintervallumot kell számítani (23.3. Páros összehasonlítás és csoportszintű intervallum), és a fenti számok ennek hiányában csak leíró megfigyelésnek tekinthetők.

18.4.6 Értelmezés, negatív eredmény és érvényességi határ

Az értékelés három szinten történik. (1) A H1-et a `first_block_norm` (paraméter-gradiens) és a `first_block_act_grad_norm` (aktivációgradiens) plain/reziduális arányával ellenőrizzük; a stem, az utolsó blokk és a head normája külön mező, ezeket nem vonjuk össze. A várakozás a levezetett irány, de a mérés lehet ellentétes is. (2) A H2-höz a három seed páros best-val-loss különbségét jelentjük (seedenként és átlagban); három seed mellett ez nem eloszlás, hanem három páros megfigyelés, ezért a bizonytalanságot a scene-szintű bootstrap adja, és a legjobb seedet nem válogatjuk ki. (3) A végső tesztkiértékelés után közöljük a clean teszt pontosságát és szükség esetén a corrupt páros különbséget.

A **negatív eredmény kifejezetten megengedett**: ha a GlyphLab feladat 24×24 pixelen öt blokkal is könnyű, a két változat val lossza nem különül el. Ez nem cáfolja a [P06] nagyléptékű eredményét, és nem igazolja, hogy a skip minden feladaton hasznos; mindössze annyit állít, hogy *ezen az adaton, ezzel a recepttel* a különbség nem kimutatható. A következtetés érvényességi köre ezért a protokollal együtt rögzített: GlyphLab, 360 kép, 23 571 paraméter, CPU, három seed.

18.4.7 Gyakori tévedés

A leggyakoribb hiba az, hogy a kis ablatív kísérlet eredményét az eredeti ResNet-publikáció reprodukciójaként vagy általános architektúraigazságként olvassuk. A [P06] 3.1–3.2 szakasza a $y = F(x) + x$ blokkot definiálja; a cikk állításai azonban ImageNet/COCO léptékű, nagyságrendekkel nagyobb hálókra és adatokra vonatkoznak. Illusztráció (nem matematikai ellenpélda): ha a fenti laborban a plain háló véletlenül jobb val losszt ér el, az nem azt jelenti, hogy „a skip kapcsolat hibás”, hanem hogy a mérési környezetben a H2 nem igazolódott. A skála és az adat különbözősége miatt a két állítás nem ütközik.

18.4.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem ablatív vizsgálat, ha a plain hálót $\mathrm{lr} = 10^{-3}$-mal, a reziduálist pedig $\mathrm{lr} = 10^{-2}$-szal tanítjuk, és közben azt állítjuk, hogy a skip gyorsította a tanulást. Adjon egy olyan ellenőrzést, amely ezt a hibát kizárja.

F2 (levezetési, nehézség 2). Számítsa ki a fenti labor hálójának paraméterszámát, és igazolja, hogy a plain és reziduális változat paraméterkerete egyezik. Ezután a skalár példában $a_1 = a_2 = 0{,}5$ mellett adja meg a plain és a reziduális g_0 értékét, és ellenőrizze a lemma exponenciális becslését.

F3 (implementációs, nehézség 2). Írjon egy `norm_probe` függvényt, amely egy `Net` modellen az első forward-backward lépés után visszaadja a stem, az első blokk, az utolsó blokk és a head gradiensnormáit. Az input alakja $(2, 1, 24, 24)$ legyen, dtype float32, device CPU; a függvény a modell paraméterein iterál, és `None` gradiensre 0-t ad.

F4 (hibakeresési, nehézség 2). A labor korai változatában a ciklus mindig `Net(residual=True)`-ot konstruált, mert a `residual` ciklusváltozó nem került át a konstruktorba, miközben a riport kulcsa (`seedX_plain` vagy `seedX_residual`) a ciklusváltozó értéke szerint készült. Mi a hiba következménye a H2 kiértékelésére, és milyen naplóbejegyzéssel lehet ezt a hiba detektálása után visszamenőleg kimutatni?

F5 (kísérleti/transzfer, nehézség 3). Tervezzen depth-sweep kísérletet 3, 5 és 7 blokkal annak eldöntésére, hogy a skip előnye a mélységgel nő-e a GlyphLabon. Adja meg a nullhipotézist, a rögzített erőforráskeretet, a metrikát, a seed- és scene-szintű kiértékelést, valamint a következtetés legalább egy érvényességi korlátját. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

18.4.9 Megoldások és indoklás

F1 megoldás. Az eltérő tanulási ráta két faktor együttes változtatása, ezért a mért különbséget nem lehet kizárólag a skipnek tulajdonítani. Az ellenőrzés: ugyanazt a seed-készletet és AdamW konfigurációt mindkét ágon futtatjuk, a kód pedig a **manifestbe** írja az `lr`, a `residual` és a modell-paraméterszám értékét; a manifestet a kiértékelés előtt összevetjük. A manifest itt a futás géppel olvasható konfigurációs és eredmény-naplója (JSON), amely a splitet, a seedet, az `lr`-t, a weight decay-t, a batchméretet, az epochszámot, a `residual` tényleges értékét és a paraméterszámot rögzíti; a laborban ezt a `n072_report.json` és a hozzá tartozó futáskonfiguráció tölti be.

F2 megoldás. A stem: $1 \cdot 16 \cdot 9 + 16 = 160$. Egy blokkban a Conv bias=False, ezért $16 \cdot 16 \cdot 9 = 2304$ súlyonként, két konvolúció 4608; a két BatchNorm2d γ, β párja külön-külön $2 \cdot 16 = 32$, együtt 64; összesen blokkonként 4672. Öt blokk 23360, a head $16 \cdot 3 + 3 = 51$.

Összesen $160 + 23360 + 51 = 23571$. A shortcut add-nak nincs paramétere, tehát a budget egyezik. A skalár példában plain: $g_0 = 0{,}5^2 = 0{,}25$; reziduális: $g_0 = (1{,}5)^2 = 2{,}25$. A lemma a plain esetre $\rho = 0{,}5$, $\rho^2 = 0{,}25$, az egyenlőség teljesül, mert a példa lineáris.

F3 megoldás. A `Net` osztály a fenti labor kódjából származik (`lab/n072_resablation.py`); az alábbi függvény csak a gradiensnormákat méri, ezért optimalizáló példány nem kell hozzá.

```
def norm_probe(model, xb, yb):
    loss_fn = torch.nn.CrossEntropyLoss()
    model.train()
    model.zero_grad(set_to_none=True)
    loss_fn(model(xb), yb).backward()

    def norm(modules):
        s = 0.0
        for m in modules:
            for p in m.parameters():
                if p.grad is not None:
                    s += p.grad.detach().pow(2).sum().item()
        return s ** 0.5
    return {
        "stem": norm([model.stem]),
        "first_block": norm([model.blocks[0]]),
        "last_block": norm([model.blocks[-1]]),
        "head": norm([model.head]),
    }

x = torch.randn(2, 1, 24, 24, dtype=torch.float32, device=torch.device("cpu"))
y = torch.tensor([0, 1], dtype=torch.int64)
model = Net(residual=True)
print(norm_probe(model, x, y))
```

Alakellenőrzés: a bemenet NCHW $(2, 1, 24, 24)$; a head logitja $(2, 3)$, a CrossEntropyLoss int64 célt vár. A kimeneti gradiensek skalárok, így a visszaadott értékek mindig nemnegatív valós számok.

F4 megoldás. Ha mindkét ág reziduális, a H2 összehasonlítás triviálisan azonos eloszlást mér, és a plain-ként naplózott eredmény valójában a reziduális modellé. A naplójavítás: a riportba minden futásnál be kell írni a `residual` tényleges értékét, a modell `state_dict` kulcsainak számát és a blokkok `Block.residual` attribútumát; a futás elején ezeket a `report` bejegyzéssel együtt ki kell írni. Visszaellenőrzés: ha a riport kulcsa `seedX_plain`, de a blokkok `Block.residual` attribútuma `True` (vagy a `state_dict` alapján a modell reziduális), a hiba egyértelműen kimutatható a naplóból.

F5 megoldás. Nullhipotézis: a skip nélküli és a skipes háló best-val-loss különbsége nem változik a blokkszám függvényében. Rögzített keret: $C = 16$ csatorna, AdamW lr $= 10^{-3}$, három seed, 40 epoch, batch 64. A paraméterszám a mélységgel nő, ezért a kérdéshez a *mélységi* keret a releváns; a MAC-értéket a 14.3. Paraméterszám, MAC és aktivációmemória szerint blokkonként kell kiszámítani és a riportba írni: a stem $1 \cdot 16 \cdot 3^2 \cdot 24^2 = 82\,944$ MAC, egy blokk $2 \cdot 16 \cdot 16 \cdot 3^2 \cdot 24^2 = 2\,654\,208$ MAC (a BN és a ReLU költsége ezen felül, külön naplózandó), a head AdaptiveAvgPool és Linear költsége ehhez képest elhanyagolható. Metrika: a validációs loss szolgál a checkpoint kiválasztására, a jelentett érték a validációs accuracy és a loss. A bizonytalanságot a validációs predikciók scene-szintű bootstrapja adja: a jelenetek újramintavételezésével a páros val-loss-különbség átlagára számolunk 95%-os konfidenciaintervallumot; a tesztet erre nem használjuk, a teszten csak egyszeri pontbecslést (és szükség esetén külön jelölt, leíró intervallumot) adunk. A tesztet csak a három mélység és a két skip-változat best checkpointjainak kiválasztása után értékeljük. Érvényességi korlát: a GlyphLab kis mérete miatt a 7 blokkos háló is csak

$160 + 7 \cdot 4672 + 51 = 32915$ paraméteres marad (kerekítve 33 ezer), így az eredmény nem vihető át mély ImageNet-osztályú hálókra.

18.4.10 Önellenőrzés és ismétlés

A képletekben az indexhatárok ellenőrizve: $l = 1, \dots, T$, g_T a head felől érkező gradiens. Az `add` a reziduális blokkban csak akkor definiált, ha a csatorna- és térbeli alak egyezik; a laborban a stride 1 és a pad 1 ezt biztosítja. A plain és reziduális paraméterszám levezetése a bias=False konvenciót követi; a BN paraméterei mindkét ágban azonosak. A gradiensnormák a gradiensvektor L2-normái: a négyzetösszeg a gradienskomponensek (nem a paraméterek) felett képződik, majd négyzetgyököt vonunk; a `grad_norm` függvény ezt a `p.grad` tenzorokon számolja, nem a paraméterértékeken. A loss átlagolása a batchre vonatkozik, a gradiensnorma mérése ebből a skálából származó torzítást hordozhat, ezért a labornak azonos batchméretet kell használnia. Ismétlés a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből: az egyeztetendő keretet a kérdéshez választjuk; a 18.3. Pre-activation, kis batch és alternatív normalizálás alfejezetből: a pre-aktivációs blokk a shortcutra nem tesz normalizálást; a 6.4. Momentum és adaptív optimizálók alfejezetből: az AdamW súlycsökkentése decoupled, ezért az L2-tagot nem a lossba írjuk.

18.4.11 Rövid összefoglaló

Az ablatív vizsgálat egyetlen tényezőt változtat, a matched recipe minden más döntést rögzít: adat, split, seed, optimizer, ütemezés, checkpoint- és tesztszabály. A skip kapcsolat hatását a plain $B_P(x; W) = F(x; W)$ és a reziduális $B_R(x; W) = F(x; W) + x$ blokk azonos paraméterkeretű összehasonlításával izoláljuk. A backward levezetés szerint a plain lánc a Jacobi-normák szorzatával skáláz, a reziduális láncban az identitás az upstream gradienst változatlanul továbbítja. A labor ötblokkos GlyphLab-hálón méri a best-val-loss eloszlást és az első lépés gradiensnormáit; a mérési állítás érvényessége erre a kis környezetre korlátozódik, és a negatív eredmény is közölhető.

18.4.12 Következő alkalmazás

A 19.1. Depthwise separable és pointwise réteg a depthwise separable konvolúciót vezeti be: ott ismét ablációval választjuk szét a spatial és a pointwise rész hatását, és a most bevezetett matched-recipe naplózást használjuk a paraméter/MAC-keret igazolására.

18.4.13 Felhasznált források

- [P06] He, K., Zhang, X., Ren, S. és Sun, J. (2015). *Deep Residual Learning for Image Recognition.* arXiv:1512.03385v1. https://arxiv.org/abs/1512.03385.
- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html.
- [P08] Santurkar, S., Tsipras, D., Ilyas, A. és Madry, A. (2018). *How Does Batch Normalization Help Optimization?* arXiv:1805.11604. https://arxiv.org/abs/1805.11604.
- [D12] PyTorch contributors. *BatchNorm2d.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors. *CrossEntropyLoss.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

19 Hatékony és korszerű CNN-blokkok

19.1 Depthwise separable és pointwise réteg

19.1.1 Cél, előismeret és induló becslés

Ebben az alfejezetben a `nn.Conv2d` két korlátozott rokonát definiáljuk: a **depthwise** (csatornánkénti, mélységi) és a **pointwise** (1×1-es, pontonkénti) réteget, majd belőlük a **depthwise separable** (csatornaszétválasztott) blokkot. A cél az, hogy a teljes konvolúció egyszerre végzett térbeli és csatornakeverését szétválasszuk, és a paraméter-, illetve MAC-költséget pontosan, meghatározott alakokra kiszámoljuk. A szükséges előismeret a 14.4. Csoportos és depthwise konvolúció alfejezetből a `groups` paraméter értelmezése és oszthatósági feltétele, a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből a költségkeretek egyeztetésének szükségessége, a 15.1. Súly- és biasgradiens alfejezetből a súlymegosztásból következő gradiens-összegzés. A **[D10]** dokumentáció szerint a PyTorch `Conv2d` térbeli **cross-correlationt** számol kernelmegfordítás nélkül; a depthwise és pointwise változatok is ezt teszik, tehát a „konvolúció” elnevezés ellenére ebben a fejezetben végig cross-correlationt definiálunk. A **cross-correlation** abban különbözik a matematikai konvolúciótól, hogy a kernelt nem fordítja meg; tanult súlyok mellett ez csak indexkonvenció, ezért nevezi a mélytanulási irodalom „konvolúciónak”, de a két művelet nem azonos.

Induló becslés. Legyen $C_{in} = 3$, $C_{out} = 4$, $k_h = k_w = 3$, stride 1, padding 0, $1 \times 5 \times 5$ bemenet. A teljes réteg súlyszáma $4 \cdot 3 \cdot 9 = 108$. A separable blokk depthwise súlya $3 \cdot 9 = 27$, pointwise súlya $4 \cdot 3 = 12$, összesen 39, tehát a súlyarány $39/108 = 13/36 \approx 0{,}361$. A levezetés után látni fogjuk, hogy ez nem véletlen: bias nélkül, $m = 1$ multiplier és azonos kimeneti térbeli méret mellett az arány $\frac{1}{C_{out}} + \frac{1}{k_h k_w}$.

19.1.2 Definíciók: depthwise, pointwise, separable

Legyen a bemeneti képbatch $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, NCHW elrendezésben (N = batch, C = csatorna, H = magasság, W = szélesség).

1. definíció (depthwise konvolúció). A depthwise konvolúció olyan cross-correlation, amelyben `groups = C_in`, tehát minden bemeneti csatornához külön csatornacsoport tartozik. A kernelsúly alakja

$$D \in \mathbb{R}^{C_{in} \times m \times k_h \times k_w},$$

ahol $m \in \mathbb{Z}_{>0}$ a **depthwise multiplier**: egy bemeneti csatornához m darab kimeneti szűrő tartozik. A kimeneti csatornák száma emiatt $C_{mid} = mC_{in}$. **Alakleképezés a PyTorch-hoz:** a `Conv2d(C_in, m*C_in, k, groups=C_in)` súlyalakja $(mC_{in}, 1, k_h, k_w)$; a saját $D \in \mathbb{R}^{C_{in} \times m \times k_h \times k_w}$ és a PyTorch-súly között a kimeneti index $o = c\,m + \mu$ szerinti explicit átrendezés áll: `weight = D.reshape(C_in*m, 1, kh, kw)` (soronként a c, azon belül a μ index fut), és fordítva `D = weight.reshape(C_in, m, kh, kw)`. A 14.4 alakleírásával és a kóddal ez ugyanaz a megállapodás; a váltás tehát nem hallgatólagos. Zéró alapú indexeléssel a kimenetet

$Y^{(1)} \in \mathbb{R}^{N \times mC_{in} \times H_{out} \times W_{out}}$ jelöli, és a kimeneti o csatornához egyértelműen tartozik $c = \lfloor o/m \rfloor$ és $\mu = o \bmod m$. Adott stride s, zero padding p és dilation d mellett

$$Y^{(1)}[n, o, i, j] \;=\; b^{(1)}[o] + \sum_{\alpha=0}^{k_h-1} \sum_{\beta=0}^{k_w-1} D[\lfloor o/m \rfloor,\; o \bbmod m,\; \alpha,\; \beta]\, X[n,\; \lfloor o/m \rfloor,\; is - p + \alpha d,\; js - p + \beta d]\,,$$

azzal a konvencióval, hogy a $[0, H-1] \times [0, W-1]$ indexen kívüli bemenet nulla. Az $m = 1$ esetben $o = c$, tehát egy bemeneti csatornából pontosan egy köztes csatorna lesz. A depthwise `groups=C_in` beállítás miatt az oszthatósági feltétel automatikusan teljesül, a gyakorlatban azonban a PyTorch a `groups` értéket csak akkor fogadja el, ha C_{in} és C_{out} egyaránt osztható `groups` értékkel; itt $C_{out} = mC_{in}$ ezt garantálja.

2. definíció (pointwise konvolúció). A pointwise konvolúció 1×1-es kernelű, `groups=1` csatornakeverő cross-correlation:

$$P \in \mathbb{R}^{C_{out} \times C_{mid} \times 1 \times 1}, \qquad Z[n, q, i, j] \;=\; b^{(2)}[q] + \sum_{c'=0}^{C_{mid}-1} P[q, c', 0, 0]\; Y^{(1)}[n, c', i, j].$$

Mivel a kernel térbeli kiterjedése 1, stride 1 és padding 0 esetén a térbeli felbontás változatlan: a pointwise csak az egyazon (i, j) pozícióban lévő csatornákat keveri (**channel mixing**). A teljes konvolúció ezzel szemben minden kimeneti skalárhoz az összes bemeneti csatornát és az összes kernelpozíciót egyszerre használja.

3. definíció (depthwise separable blokk). A depthwise separable blokk a két réteg kompozíciója:

$$Z \;=\; \mathrm{PW}(\mathrm{DW}(X))\,.$$

A térbeli szűrés és a csatornakeverés tehát szekvenciálisan, külön operátorokkal történik. A „separable” szó itt **nem** a matematikai tenzorszorzat-szeparálhatóságot jelenti, hanem a térbeli és csatorna-feladat szétválasztását; ezt a fogalmi különbséget az F1 feladat vizsgálja.

19.1.3 Levezetés: paraméter- és MAC-arány

4. állítás (saját levezetés, elemi leszámlálás). Legyen $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, $k_h, k_w \geq 1$, stride $s \geq 1$, padding $p \geq 0$, dilation $d \geq 1$, és tegyük fel, hogy a teljes konvolúció (`groups=1`, kimeneti csatornaszám C_{out}) és a separable blokk ($m = 1$ depthwise + pointwise) azonos $H_{out} \times W_{out}$ térbeli kimenetet ad. A MAC definíciója: egy kimeneti skalárhoz tartozó egy szorzás-plusz-akkumulálás; a bias-hozzáadás nem MAC. Ekkor bias nélkül a súlyparaméterek aránya, és mindig a MAC-ek aránya

$$\frac{P_{sep}}{P_{full}} \;=\; \frac{M_{sep}}{M_{full}} \;=\; \frac{1}{C_{out}} + \frac{1}{k_h k_w}.$$

Bizonyítás. A teljes réteg egyetlen kimeneti skalárjához $C_{in} k_h k_w$ súly–bemenet szorzat tartozik, tehát

$$M_{full} = N C_{out}\, H_{out} W_{out}\, C_{in}\, k_h k_w.$$

A depthwise réteg ($m = 1$) kimeneti skalárjaihoz $C_{in}/g = C_{in}/C_{in} = 1$ bemeneti csatorna tartozik, tehát egy kimeneti skalár $k_h k_w$ MAC-et igényel; a kimeneti skalárok száma $NC_{in}H_{out}W_{out}$, ezért

$$M_{DW} = NC_{in}\,H_{out}W_{out}\,k_h k_w = \frac{1}{C_{out}} M_{full}.$$

A pointwise réteg egy kimeneti skalárja C_{in} szorzatot igényel, a kimeneti skalárok száma $NC_{out}H_{out}W_{out}$, így

$$M_{PW} = NC_{out}\,H_{out}W_{out}\,C_{in} = \frac{1}{k_h k_w} M_{full}.$$

A separable blokk MAC-je a két réteg MAC-jének összege, mert a rétegek szekvenciálisan futnak, és minden réteg a saját bemenetén végzi a maga szorzás–összeadásait; az, hogy a pointwise a depthwise kimenetén dolgozik, a lánc összekapcsolását adja, nem külön költségtételt:

$$M_{sep} = M_{DW} + M_{PW} = \left(\frac{1}{C_{out}} + \frac{1}{k_h k_w}\right) M_{full}.$$

A bias nélküli súlyparaméterekre ugyanez a három szorzat jelenik meg: $P_{full} = C_{out}C_{in}k_h k_w$, $P_{DW} = C_{in}k_h k_w$, $P_{PW} = C_{out}C_{in}$, tehát az arány azonos. ■

Megjegyzések.

1. **Bias esetén** a teljes konvolúciónak C_{out}, a depthwise rétegnek C_{in}, a pointwise rétegnek C_{out} biasa van: $P_{full} = C_{out}C_{in}k_h k_w + C_{out}$, $P_{sep} = C_{in}k_h k_w + C_{in} + C_{out}C_{in} + C_{out}$. Az arány ekkor már **nem** egyezik meg a MAC-aránnyal, és a különbség C_{in} növelésével sem tűnik el, mert a depthwise biasok száma is C_{in}-nel nő. Rögzített C_{out} és $k^2 = k_h k_w$ mellett

$$\lim_{C_{in}\to\infty} \frac{P_{sep}}{P_{full}} = \frac{k^2 + C_{out} + 1}{C_{out}k^2}, \qquad \frac{M_{sep}}{M_{full}} = \frac{k^2 + C_{out}}{C_{out}k^2},$$

a különbség $1/(C_{out}k^2)$. Például $k^2 = 9$, $C_{out} = 4$ esetén a két érték 14/36 és 13/36. Aki paraméterköltséget közöl, mondja meg, számol-e bias-t. 2. **Multiplier esetén** $m > 1$-re $C_{mid} = mC_{in}$, így M_{DW} m-szeres, a pointwise pedig mC_{in} bemeneti csatornán dolgozik, ezért

$$\frac{M_{sep}^{(m)}}{M_{full}} = \frac{m\,C_{in}k_h k_w + m\,C_{out}C_{in}}{C_{out}C_{in}k_h k_w} = m\left(\frac{1}{C_{out}} + \frac{1}{k_h k_w}\right).$$

19.1.4 Kézi kis példa és ellenőrző munkamenet

Példánkban $N = 1$, $C_{in} = 3$, $C_{out} = 4$, $H = W = 5$, $k_h = k_w = 3$, $s = 1$, $p = 0$, $d = 1$, ezért $H_{out} = W_{out} = \lfloor (5 + 2\cdot 0 - 1\cdot(3-1) - 1)/1 \rfloor + 1 = \lfloor 2 \rfloor + 1 = 3$.

Operátor	Súly+bias	MAC
Teljes `Conv2d(3,4,3)`	$108 + 4 = 112$	$1 \cdot 4 \cdot 9 \cdot 3 \cdot 9 = 972$
Depthwise `Conv2d(3,3,3,groups=3)`	$27 + 3 = 30$	$1 \cdot 3 \cdot 9 \cdot 1 \cdot 9 = 243$

Operátor	Súly+bias	MAC
Pointwise `Conv2d(3,4,1)`	$12 + 4 = 16$	$1 \cdot 4 \cdot 9 \cdot 3 = 108$
Separable összeg	$39 + 7 = 46$	351

Bias nélkül a súlyarány $39/108 = 0{,}3611$, a MAC-arány $351/972 = 0{,}3611$, pontosan $\frac{1}{4} + \frac{1}{9}$. A bias-t is tartalmazó paraméterarány $46/112 \approx 0{,}411$, tehát a bias elhanyagolása itt nem ártalmatlan.

19.1.5 Labor: kis blokk és kézi költségszámolás

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n073_lab.py` a `lab/` könyvtárból futtatható (`python n073_lab.py`); külső adatfájl nem kell.

A labor `count_conv` függvénye a szokásos, egész paddingű `nn.Conv2d` rétegek paraméter- és MAC-költségét számolja a fenti képletekből és a bemeneti térbeli méretből; a `padding='same'` és a nem nullás `padding_mode` nincs kezelve. A kód a `lab/n073_lab.py` fájlba kerül, és a `lab/` könyvtárból futtatható.

```
# lab/n073_lab.py
# Futtatás: python n073_lab.py
# CPU, float32; a kimenet a képletekből számolt kontroll, nem futásidő-mérés.
import math
import torch
from torch import nn

def out_len(L, p, d, k, s):
    return math.floor((L + 2*p - d*(k-1) - 1)/s) + 1

def count_conv(conv, N, H, W):
    # Csak forward cross-correlation Conv2d; transposed conv nem tartozik ide.
    assert conv.in_channels % conv.groups == 0
    cin_per_group = conv.in_channels // conv.groups
    Ho = out_len(H, conv.padding[0], conv.dilation[0], conv.kernel_size[0], conv.stride[0])
    Wo = out_len(W, conv.padding[1], conv.dilation[1], conv.kernel_size[1], conv.stride[1])
    params = sum(p.numel() for p in conv.parameters())  # súly + bias
    mac = N * conv.out_channels * Ho * Wo * cin_per_group * conv.kernel_size[0] *
↪   conv.kernel_size[1]
    return params, mac, (Ho, Wo)

device = torch.device("cpu")
dtype = torch.float32

full = nn.Conv2d(3, 4, 3, stride=1, padding=0, dilation=1, groups=1, bias=True).to(device)
dw   = nn.Conv2d(3, 3, 3, stride=1, padding=0, dilation=1, groups=3, bias=True).to(device)
pw   = nn.Conv2d(3, 4, 1, stride=1, padding=0, dilation=1, groups=1, bias=True).to(device)

pF, mF, shapeF = count_conv(full, 1, 5, 5)
pD, mD, _ = count_conv(dw, 1, 5, 5)
pP, mP, _ = count_conv(pw, 1, 3, 3)   # a pw bemenete a depthwise 3x3-as kimenete
assert (pF, mF, shapeF) == (112, 972, (3, 3))
assert (pD, mD) == (30, 243)
assert (pP, mP) == (16, 108)
assert mD + mP == 351 and abs((mD + mP) / mF - 13/36) < 1e-12
print("full      ", pF, mF, shapeF)
```

```
print("dw        ", pD, mD)
print("pw        ", pP, mP)
print("sep össz  ", pD+pP, mD+mP)
print("MAC arány ", (mD+mP)/mF)
print("súlyarány bias nélkül ", (3*9 + 4*3)/(4*3*9))

# Független strukturális ellenőrzés: MAC = kimeneti pozíciók × kimeneti
# csatornák × a szűrőnkénti szorzások száma (a tényleges súlyalakból).
for name, layer, mac in [("full", full, mF), ("dw", dw, mD), ("pw", pw, mP)]:
    per_out = layer.out_channels * layer.weight[0].numel()
    assert mac == shapeF[0] * shapeF[1] * per_out, name

def naive_mac_conv(x, w, stride=1, pad=0):
    # Közvetlen leszámlálás: minden kimeneti elemhez minden kernelpozíció
    # és csoportbeli bemeneti csatorna egy szorzás-plusz-akkumulálás.
    N, _, H, W = x.shape
    cout, cg, kh, kw = w.shape
    Ho = (H + 2*pad - kh) // stride + 1
    Wo = (W + 2*pad - kw) // stride + 1
    mac = 0
    for _n in range(N):
        for _o in range(cout):
            for _i in range(Ho):
                for _j in range(Wo):
                    for _c in range(cg):
                        for _a in range(kh):
                            for _b in range(kw):
                                mac += 1
    return mac

x5 = torch.zeros(1, 3, 5, 5, dtype=dtype, device=device)
x3 = torch.zeros(1, 3, 3, 3, dtype=dtype, device=device)
assert naive_mac_conv(x5, full.weight) == mF
assert naive_mac_conv(x5, dw.weight) == mD
assert naive_mac_conv(x3, pw.weight) == mP

with torch.no_grad():
    Y = pw(dw(torch.zeros(1, 3, 5, 5, dtype=dtype, device=device)))
assert tuple(Y.shape) == (1, 4, 3, 3)
print("output shape:", tuple(Y.shape))
```

A rögzített környezetű ellenőrző futás mért kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál; a szkript determinisztikus, seed nélkül; a kód a képleteket ellenőrzi, nem futásidőt mér):

```
full       112 972 (3, 3)
dw         30 243
pw         16 108
sep össz   46 351
MAC arány  0.3611111111111111
súlyarány bias nélkül  0.3611111111111111
output shape: (1, 4, 3, 3)
```

Ellenőrzés és értelmezés. A két arány bias nélkül az ötödik tizedesjegyig egyezik a $\frac{1}{4} + \frac{1}{9}$ értékkel; a bias-os paraméterarány különbözik, ahogy a megjegyzés mondta. Az output shape $(1, 4, 3, 3)$ megegyezik a teljes réteg kimenetével, tehát a térbeli illesztés teljesül. A számítás nem futásidő-mérés: a `count_conv` a zárt képleteket használja, ezért a numerikus egyezés önmagában csak a levezetés belső konzisztenciáját ellenőrzi, nem a könyvtár optimalizálását. A labor ezért tartalmaz egy független strukturális ellenőrzést is: a MAC-et a rétegek tényleges súlyalakjából (kimeneti pozíciók × kimeneti csatornák × szűrőnkénti szorzások) számolja, ami nem a depthwise/pointwise felbontás képletének megismétlése. Az `assert`-ek a fenti paraméter-,

MAC- és shape-értékeket rögzítik; a kimenet tehát tényleges futtatásból származik, de nem teljesítménymérés.

19.1.6 Feladatok

F1 — fogalmi (nehézség 1). Adja meg saját szavaival, miben korlátozott a depthwise réteg a teljes konvolúcióhoz képest, miért nevezzük a 1×1-es réteget channel mixingnek, és miért nem „szeparálható” a szó matematikai tenzorszorzat-értelmében. Adjon ellenpéldát: 1×1-es kernel `groups=2` beállítással mely csatornák keveredtek, melyek nem. **Beadandó:** legfeljebb egy oldal, képletekkel.

F2 — levezetési/bizonyítási (nehézség 2). Vezesse le a depthwise multiplier m esetére a MAC-arányt, és határozza meg: $k_h = k_w = 3$ és $C_{out} = 16$ mellett mi a legkisebb pozitív egész m, amelyre a separable blokk már **nem** csökkenti a MAC-et a teljes réteghez képest. **Beadandó:** lépésről lépésre levezetés és az m értéke.

F3 — implementációs (nehézség 2). Egészítse ki a labor `count_conv` függvényét úgy, hogy hibát dobjon, ha `in_channels % groups != 0`, majd tesztelje három esettel: a labor `full`, `dw`, `pw` rétegén kívül egy `Conv2d(6, 4, 3, groups=2)` rétegen is, $N = 2$, $H = W = 10$ bemenettel. **Beadandó:** futtatható kód, `assert`-ekkel, és a várt egész MAC-értékek.

F4 — hibakeresési (nehézség 2). Az alábbi érvelés hibás: „A 3×3-as separable blokk a teljes konvolúcióhoz képest 1/9-ére csökkenti a számítást, mert a pointwise 1×1 elhanyagolható.” Lokalizálja a hibát, mutassa meg a fenti $C_{in} = 3$, $C_{out} = 4$ példán a tényleges arányt, és adjon olyan szélsőesetet ($C_{out} = 1$), ahol a separable blokk MAC-je **nagyobb** a teljesnél. **Beadandó:** az ok, a következmény és a javított állítás.

F5 — kísérleti/transzfer (nehézség 3). Tervezzen protokollt a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adaton, amely egy teljes konvolúciós blokk és egy depthwise separable blokk hatását hasonlítja össze **egyeztetett MAC-keret** mellett, a 17.4. Architektúra összehasonlítása kontrollált keretben matched-recipe elvét követve. A GlyphLab 180 jelenetből áll, jelenetenként két nézettel (360 kép, 24×24, három orientációosztály); a split 240/60/60 kép, azaz 120/30/30 jelenet, és jelenetszintű: egy `scene_id` mindkét nézete ugyanabba a splitbe kerül. Írja elő a hipotézist, a modellfejek azonos tartalmát, a scene_id szerinti splitet, a seed-készletet, a best-val-loss választást, a záró clean/korrupt tesztet és a következtetési határt. Futtatás nem elvárt. **Beadandó:** egyoldalas protokoll. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

19.1.7 Megoldások és indoklás

F1. A teljes konvolúció kimeneti skalárja $\sum_c \sum_{\alpha,\beta} K[q, c, \alpha, \beta] X[\dots]$, tehát a csatorna- és térbeli indexek egyszerre, tetszőlegesen összekapcsolódnak. A depthwise rétegben az összegzés csak α, β szerint fut, és minden kimeneti csatorna pontosan egy bemeneti csatornához kötődik; a `groups=C_in` beállítás ezt a blokk-diagonális kapcsolatot valósítja meg. A pointwise 1×1-es réteg ezzel szemben **ugyanazon** (i, j) pozíció összes csatornáját keveri, de térben nem hat, ezért nevezzük csatornakeverésnek. A „separable” elnevezés itt operátorok szekvenciális szétválasztását jelenti, nem azt, hogy a 3×3-as kernel két vektor külső szorzataként, $3 \times 1 \otimes 1 \times 3$ alakban állna elő; utóbbi tenzorszorzat-szeparálhatóság esetén a kernel rangja 1 volna. Ellenpélda: `Conv2d(6, 4, 1, groups=2)` a bemenetet két háromcsatornás csoportra osztja; a 0–2. csatornák keveredhetnek egymással, a 3–5. csatornák egymással, de a két csoport között **nincs** út, tehát ez nem teljes channel mixing.

F2. Az m-es depthwise kimeneti csatornaszáma mC_{in}, MAC-je $N \cdot mC_{in} \cdot H_{out}W_{out} \cdot k_h k_w$. A pointwise bemenete mC_{in} csatorna, MAC-je $N \cdot C_{out} \cdot H_{out}W_{out} \cdot mC_{in}$. A teljes réteg MAC-je változatlan $NC_{out}C_{in}H_{out}W_{out}k_h k_w$. A hányadosból $N, H_{out}W_{out}, C_{in}$ kiesik:

$$\frac{M_{sep}^{(m)}}{M_{full}} = \frac{mC_{in}k_h k_w + mC_{out}C_{in}}{C_{out}C_{in}k_h k_w} = m\left(\frac{1}{C_{out}} + \frac{1}{k_h k_w}\right).$$

A feltétel $m(1/16+1/9) \geq 1$, azaz $m \cdot 25/144 \geq 1$, tehát $m \geq 144/25 = 5{,}76$. A legkisebb pozitív egész megoldás $m = 6$. $m = 6$ esetén az arány $6 \cdot 25/144 = 25/24 > 1$, tehát a separable változat több MAC-et igényel, mint a teljes réteg. Az **egyenlőség esete** itt nem áll elő pozitív egész m mellett, mert $144/25$ nem egész: a határvonal $m = 5{,}76$, ezért $m = 6$ már szigorúan nagyobb arányt ad. (Az egyenlőségre a $(C_{out}, k_h k_w) = (2, 2)$, $m = 1$ eset szolgál példaként, ahol az arány pontosan 1.) Ez a szélsőeset mutatja, hogy a depthwise separable nem minden beállításban takarékos.

F3. A teljes függvény:

```python
# lab/n073_f3.py - futtatás: python n073_f3.py
import math
import types
import torch
from torch import nn

def out_len(L, p, d, k, s):
    return math.floor((L + 2*p - d*(k-1) - 1)/s) + 1

def count_conv(conv, N, H, W):
    if conv.in_channels % conv.groups != 0:
        raise ValueError("in_channels nem osztható groups értékkel")
    cin_per_group = conv.in_channels // conv.groups
    Ho = out_len(H, conv.padding[0], conv.dilation[0], conv.kernel_size[0], conv.stride[0])
    Wo = out_len(W, conv.padding[1], conv.dilation[1], conv.kernel_size[1], conv.stride[1])
    params = sum(p.numel() for p in conv.parameters())
    mac = N * conv.out_channels * Ho * Wo * cin_per_group * conv.kernel_size[0] *
↪   conv.kernel_size[1]
    return params, mac, (Ho, Wo)

full = nn.Conv2d(3, 4, 3); dw = nn.Conv2d(3, 3, 3, groups=3); pw = nn.Conv2d(3, 4, 1)
assert count_conv(full, 1, 5, 5) == (112, 972, (3, 3))
assert count_conv(dw,   1, 5, 5) == (30, 243, (3, 3))
assert count_conv(pw,   1, 3, 3) == (16, 108, (3, 3))
g2 = nn.Conv2d(6, 4, 3, groups=2)
# H=W=10, k=3, p=0, s=1, d=1 -> Hout=Wout=8; N=2; out=4; cin/group=3
assert count_conv(g2, 2, 10, 10) == (112, 13824, (8, 8))

# A védőágat külön teszttel ellenőrizzük: a Conv2d(5,4,3,groups=2)
# már a konstruktorban elhasal, ezért SimpleNamespace-szel adjuk át a hibás metát.
bad = types.SimpleNamespace(in_channels=5, groups=2)
try:
    count_conv(bad, 1, 5, 5)
except ValueError:
    print("csoportos oszthatósági hiba helyesen elkapva")
else:
    raise AssertionError("a count_conv nem dobott hibát")
print("minden assert lefutott")
```

A várt MAC a grouped esetre $2 \cdot 4 \cdot 8 \cdot 8 \cdot 3 \cdot 9 = 13824$, a paraméterszám pedig $4 \cdot 3 \cdot 3 \cdot 3 + 4 = 112$. A `Conv2d(5,4,3,groups=2)` már a modul létrehozásakor hibát dob a `5 % 2` miatt, ezért a

`count_conv` védőágát külön, `types.SimpleNamespace(in_channels=5, groups=2)` bemenettel teszteljük; így a teszt ténylegesen a függvény hibadobását éri el.

F4. A hiba a pointwise tag elhagyása: az érvelés csak a depthwise $1/k_h k_w$ szorzóját nézi, és csendben C_{out}-t a levezetés nélkül kezeli. A példában a teljes MAC 972, a depthwise 243, a pointwise további 108, így a tényleges arány $351/972 = 0{,}3611$, nem pedig $1/9 \approx 0{,}111$. A következmény túlzott megtakarítási ígéret. $C_{out} = 1$ esetén a teljes réteg MAC $1 \cdot 1 \cdot 9 \cdot 1 \cdot 3 \cdot 9 = 243$. A separable blokkban a pointwise 1×1 tag $N \cdot C_{out} \cdot H_o W_o \cdot C_{in} = 1 \cdot 1 \cdot 9 \cdot 3 = 27$, így a separable MAC $243 + 27 = 270$, az arány pedig $270/243 = 10/9 > 1$. A javított állítás: a separable blokk az $m = 1$ esetben pontosan akkor csökkenti a MAC-et, ha $\frac{1}{C_{out}} + \frac{1}{k_h k_w} < 1$; a $C_{out} > 1$ és $k_h k_w > 1$ feltétel ehhez szükséges, de nem elégséges (a $(2, 2)$ esetben az arány pontosan 1). Az F4 hibakeresési megoldás tehát a hiányzó pointwise tételt azonosítja.

F5 lehetséges protokoll (terv, nem futás). Hipotézis: azonos MAC-keretben a depthwise separable blokk validációs cross-entropyja nem marad el szisztematikusan a teljes konvolúciós blokkétól a GlyphLab háromosztályos feladatán. **Szisztematikus eltérés** alatt azt értjük, hogy a páros különbség 95%-os konfidenciaintervalluma nem tartalmazza a 0-t, és az öt seed mindegyikében ugyanaz az előjel; ha ez nem teljesül, nem állítunk szisztematikus különbséget. Felépítés: közös stem (a háló bemenetet feldolgozó első konvolúciója) `Conv2d(1,8,3,stride=2,padding=1)`; majd A blokk: depthwise separable, B blokk: teljes conv, úgy választva csatornaszámokat, hogy a stem és a közös fej kivételével `count_conv` szerinti MAC-keret 5%-on belül egyezzen; fej mindkettőn azonos átlagpooling (`AdaptiveAvgPool2d(1)`) és lineáris 3 logit. Mindkét blokkban a konvolúciók után azonos helyen BatchNorm2d és ReLU áll, hogy a normalizálás ne legyen rejtett változó. Adat: GlyphLab, train/val/test a megadott split szerint (240/60/60 kép, 120/30/30 jelenet), mintavételi egység `scene_id`; augmentáció és egyszerű flip kizárt vagy mindkét ágon azonos. Választás: 5 seed (20261020, +1, +2, +3, +4), batch 64, AdamW lr $= 10^{-3}$, weight_decay $= 10^{-4}$, 30 epoch, early-stop 5 epoch türelemmel a legkisebb validációs loss szerinti legjobb checkpointra (best-val-loss); a tesztet csak a választott checkpointon, egyszer értékeljük, clean és páros corrupt kimenettel. Metrika: per-image accuracy és a makro-átlagolt pontosság (az osztályonkénti accuracyok átlaga) 95%-os konfidenciaintervalluma a scene-szintű bootstrapből (lásd 5.4. Függő képek, seedek és resampling). Következtetési határ: a kis szintetikus adat és a kis CPU-s környezet miatt az eredmény nem általánosítható természetes képekre vagy GPU-kernel-hatékonyságra.

19.1.8 Tipikus tévedés, önellenőrzés és visszahívás

A leggyakoribb tévedés: **„Kevesebb MAC automatikusan kisebb valós futási időt jelent."** A MAC-arány a fenti levezetésből következik, a futásidő viszont a hardver, kernelimplementáció, memória-hozzáférés és batchméret függvénye. Ellenpélda vázlata: képzeljünk el egyetlen csatornán, nagy térbeli képen egy naiv Python-listás depthwise hurkot, mellette egy 64 csatornás pointwise 1x1-et, amelyet optimalizált GEMM (general matrix multiply, általános mátrixszorzás) hív meg; a depthwise MAC-száma lehet sokkal kisebb, a valós futás mégis a rossz aritmetikai intenzitású (a műveletszám és a mozgatott bájtok aránya szempontjából kedvezőtlen) depthwise hurokban telhet el. Ez gondolatkísérlet, nem mért állítás; a 24.2. CPU/GPU profiling és helyes időmérés mérési protokollja dönti el a konkrét gépen. A helyes állítás tehát megfordítva sem igaz: a MAC-arány csak számítási keret, nem sebességgarancia.

Önellenőrző kérdés és válasz. Milyen C_{out} és kernelméret mellett nem csökken a MAC? A 4. állítás alapján nem csökken, ha $\frac{1}{C_{out}} + \frac{1}{k_h k_w} \geq 1$. Pozitív egészekre ez nem csak a $C_{out} = 1$ esetet fedi: $k_h k_w = 1$ (1×1-es kernel) esetén bármely C_{out}-ra teljesül, és a $(C_{out}, k_h k_w) = (2, 2)$ határesetben az arány pontosan 1. A tipikus $k_h k_w > 1$ és $C_{out} = 1$ esetben a depthwise + pointwise összeg nagyobb, mert a pointwise új költséget ad a csatornakeveréshez, miközben

a teljes réteg a kevés kimeneti csatorna miatt eleve olcsó. Aki az arányt $C_{out} = 1$ mellett is megtakarításként közli, azonnal ellenőrizhető hibát követ el.

Visszahívás. A 15.1. Súly- és biasgradiens alfejezetben levezetett súlygradiens-összegzés a depthwise blokkban is érvényes: minden kimeneti csatorna pontosan egy bemeneti csatornán dolgozik, de a súlygradienshez a batch és térbeli felhasználási helyek szerint továbbra is **összeadódnak** a hozzájárulások; az osztás hiba volna. A 18.1. Reziduális blokk és identitásút reziduális blokkja miatt, ha a separable blokkot shortcut mellett használjuk, a kimeneti csatornaszám illesztésére adott esetben 1×1-es projekció kell, ami további pointwise költség. Végül a 7.1. Entrópia, cross-entropy és KL cross-entropy felbontása emlékeztet arra, hogy a modellválasztást a validációs kereszt-entrópia, azaz $H(p, q)$ dönti el, ahol p a valódi, q a modell által adott eloszlás; a paraméter- vagy MAC-megtakarítás önmagában nem minőségi garancia.

19.1.9 Rövid összefoglaló

A depthwise konvolúció `groups=C_in` beállítással minden bemeneti csatornát külön szűr, m multiplier esetén mC_{in} köztes csatornát adva. A pointwise 1×1-es réteg azonos térbeli pozíció csatornáit keveri; a kettő kompozíciója a depthwise separable blokk, amely a térbeli és csatornakeverést szétválasztja. A levezetett arány bias nélkül, $m = 1$ és azonos kimeneti méret mellett mind paraméterekre, mind MAC-re $\frac{1}{C_{out}} + \frac{1}{k_h k_w}$; $m > 1$ esetén a MAC-arány ennek m-szerese, változatlan kimeneti méret mellett. Bias-szal a paraméterarány eltér a MAC-aránytól: a különbség aszimptotikusan $1/(C_{out}k^2)$, ezért a paraméterköltség közlésekor a bias-kezelést rögzíteni kell. A példában ez $13/36 \approx 0{,}361$; $C_{out} = 1$ esetén az arány meghaladja az 1-et, tehát a separable forma nem mindig takarékos. A MAC-megtakarítás nem bizonyít futásidő-csökkenést. A kódos ellenőrzés a képletek konzisztenciáját mutatja, a tényleges teljesítményt csak külön mérés igazolhatja.

19.1.10 Következő alkalmazás

A 19.2. MobileNet és inverted residual a MobileNetV2 inverted residual-blokkját építi erre a separable alapra: ott a pointwise réteg először **növeli**, majd a kimeneti pointwise **csökkenti** a csatornaszámot, lineáris bottleneck-et hagyva; a [P09] tanulmány a depthwise separable konvolúciót használja alapépítőelemként, és ehhez adja az inverted residual és a lineáris bottleneck tervezési elvét, amelyet a következő egység ezen a kontextuson mutat be. A 19.2. MobileNet és inverted residual ezekből oktatási célú, egyszerűsített blokkot épít.

19.1.11 Felhasznált források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [P09] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. és Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. https://arxiv.org/abs/1801.04381.

A 4. állítás és a példák saját levezetések és saját számítások; a labor kimenete tényleges futtatással ellenőrzött képletkontroll, nem futásidő-mérés.

19.2 MobileNet és inverted residual

19.2.1 Cél, előismeret és induló becslés

A 19.1. Depthwise separable és pointwise réteg egységben szétválasztottuk a térbeli és a csatornakeverő operátort, és levezettük, hogy a depthwise separable kompozíció paraméter/MAC (multiply–accumulate, azaz szorzás-akkumuláció) aránya a teljes konvolúcióhoz képest $\frac{1}{C_{out}} + \frac{1}{k_h k_w}$. Itt azt vizsgáljuk, hogyan rendezhető el ez a két művelet egy olyan reziduális blokkban, amelynek a **bemenete és kimenete vékony**, a köztes reprezentációja viszont **széles**. Ehhez a 18.1. Reziduális blokk és identitásút reziduális gondolatát, a 18.2. Batch normalization train és eval módban BatchNorm-életciklusát és a 15.2. Inputgradiens és átfedő recepciós mezők depthwise gradienst használjuk fel; osztályozási cél esetén a 7.2. Softmax és többosztályos cross-entropy logitos cross-entropy konvencióját.

Induló becslés. Egy 16 csatornás 8×8-as feature mapon egy $16 \to 16$ csatornás 3×3-as teljes konvolúció $16 \cdot 16 \cdot 9 = 2304$ súlyt használ. Ha ugyanezt a csatornaszámot inverted residual blokkal dolgozzuk fel ($e = 3$, azaz $16 \to 48 \to 16$), akkor az expanziós 1×1 réteg $16 \cdot 48 = 768$, a depthwise szűrő $48 \cdot 9 = 432$, a visszavetítő 1×1 pedig $48 \cdot 16 = 768$ súlyt igényel; a három tag összege $768 + 432 + 768 = 1968$, ami a teljes konvolúció 2304 súlyánál mintegy 15%-kal kevesebb. A megtakarítás nem a széles köztes rétegből, hanem a térbeli szűrés csatornánkénti szétválasztásából és a **blokk határán** maradó vékony reprezentációból jön; az expanziós 1×1 réteg költsége viszont az induló becslés része, és a köztes csatornaszám növelésével gyorsan nő.

A MobileNetV2 központi tervezési elvei az *inverted residual*, a *linear bottleneck* és az expanzió [P09]. Az eredeti blokk és a labor saját, egyszerűsített blokkja a [P09] alapján (3.2–3.3. szakasz, 2–3. o.; 1. táblázat és 4. fejezet, 4. o.; 2. táblázat és 4(d) ábra, 5. o.):

Elem	MobileNetV2 [P09]	Labor (`InvertedResidual`)
expanzió	1×1 konvolúció $C_{in} \to tC_{in}$, ReLU6	1×1 konvolúció $C_{in} \to eC_{in}$, BatchNorm, ReLU
köztes réteg	3×3 depthwise, stride s, ReLU6	3×3 depthwise, stride s, BatchNorm, ReLU
projekció	lineáris 1×1 konvolúció, aktiváció nélkül	lineáris 1×1 konvolúció, BatchNorm, aktiváció nélkül
expanziós faktor	$t = 6$ a fő kísérletekben, az első réteg kivételével állandó; $t \in [5, 10]$ közel azonos eredményt ad	$e = 3$ (kis CPU-s demó)
normalizálás	BatchNorm minden réteg után	BatchNorm minden konvolúció után
shortcut (skip)	stride 1 blokkban (4(d) ábra), egyező be- és kimeneti csatornaszámnál	`use_skip = (stride == 1) and (cin == cout)`
aktiváció	ReLU6, alacsony pontosságú számítás melletti robusztusság miatt	`nn.ReLU` (paraméterként cserélhető)

A táblázatban a [P09] a t, a labor és e fejezet szövege az e jelölést használja ugyanarra az expanziós faktorra; a kettő ugyanaz a mennyiség. A saját blokk tehát oktatási egyszerűsítés, nem a közölt hálózat reprodukciója: más expanziós faktort és aktivációt használ, és csak egyetlen blokkot vizsgál, nem a 2. táblázat 19 reziduális bottleneck rétegét. (A [P09] szövege 19 réteget

említ, a 2. táblázat n-értékeinek összege azonban $1+2+3+4+3+3+1=17$; az eltérést itt jelöljük, a teljes hálózatot nem állítjuk elő.)

19.2.2 Definíciók: expanzió, lineáris bottleneck, inverted residual

A következő definíciók e fejezet saját, egyszerűsített megfogalmazásai. A [P09] absztraktja szerint a be- és kimeneti reprezentáció **vékony bottleneck** (thin bottleneck), a köztes **expanziós rétegben** könnyűsúlyú (lightweight) depthwise szűrők dolgoznak; a **linear bottleneck** (lineáris szűkítés) indoklása a [P09] 3.3. szakaszában van: a kis dimenzióban információt romboló nemlinearitást kell elkerülni.

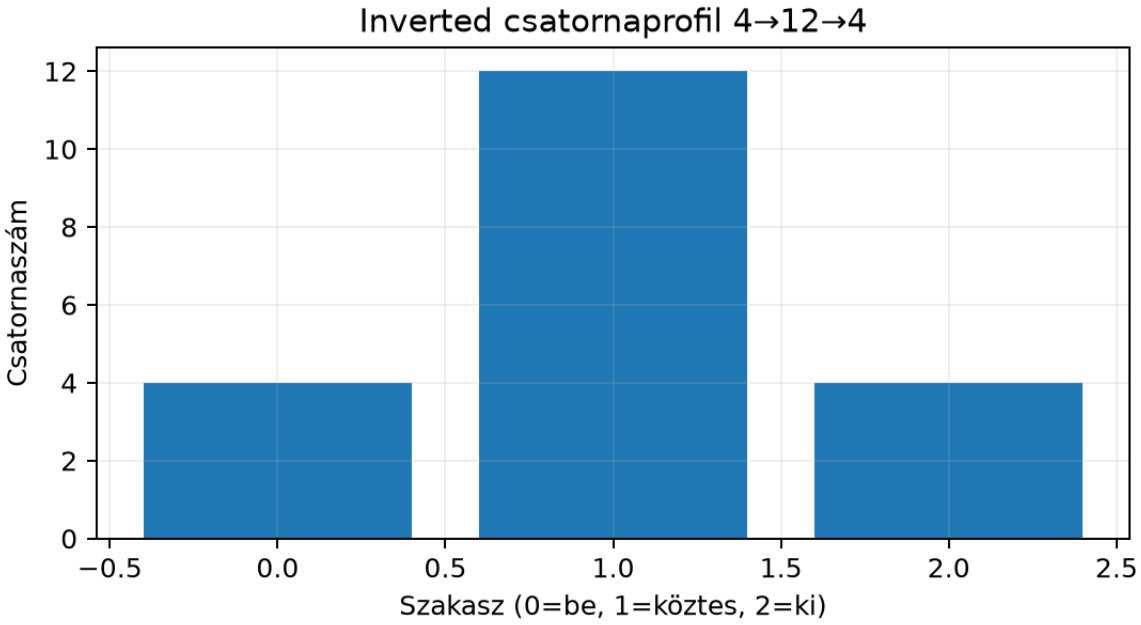
Definíció (expanziós réteg). Legyen $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ és $e \in \mathbb{N}_{+}$ az **expanziós faktor**. Az expanzió 1×1-es konvolúció, amelynek súlyalakja $W_e \in \mathbb{R}^{(eC_{in})\times C_{in}\times 1\times 1}$; kimenete $X_e \in \mathbb{R}^{N\times eC_{in}\times H\times W}$. A W_e minden kimeneti csatornája a bemeneti csatornák lineáris kombinációját adja azonos térbeli pozícióban.

Definíció (köztes depthwise szűrő). A $C_t = eC_{in}$ köztes csatornán $k_h \times k_w$ méretű depthwise konvolúció fut, stride s, szimmetrikus zero padding p, $groups = C_t$. A [D10] szerint ez keresztkorreláció; a súlyalak $W_d \in \mathbb{R}^{C_t\times 1\times k_h\times k_w}$.

Definíció (lineáris bottleneck). A záró 1×1-es **projekciós** réteg $C_t \to C_{out}$ csatornát vetít, $W_p \in \mathbb{R}^{C_{out}\times C_t\times 1\times 1}$. A projekció után **nem alkalmazunk nemlinearitást**; az aktivációs függvény és a normalizálás esetleges további elrendezése a konkrét modell receptjéhez tartozik, a döntő pont az, hogy a csatornacsökkentő 1×1 kimenetén a vékony reprezentációt nem törjük meg ReLU-val.

Definíció (inverted residual). A 18.1. Reziduális blokk és identitásút hagyományos reziduális *bottleneck* blokkja széles be- és kimenettel dolgozik, és középen szűkít (széles–szűk–széles); a ResNet *basic* blokkja ezzel szemben csatornatartó (két 3×3-as konvolúció azonos csatornaszámmal), és nem szűkít. A MobileNetV2 inverted residual blokkját ezért külön a basic és külön a bottleneck ResNet-blokkal kell összevetni. Az **inverted residual** ezt megfordítja: a be- és kimenet vékony (C_{in} és C_{out}), a köztes expanzió széles (eC_{in}), a shortcut pedig a vékony reprezentációkat köti össze: $Y = F(X) + X$, ha az alakok ezt megengedik. A [P09] absztraktja így fogalmaz: „input and output of the residual block are thin bottleneck layers”; ez a megfogalmazás a vékony be- és kimeneti reprezentációkra utal.

A csatornaprofilok egy $C_{in} = C_{out} = 4$, $e = 3$ példán:



19.1. ábra: Inverted csatornaprofil 4→12→4

Saját szemléltető profil. A 18.1. alfejezet hagyományos szemléltető blokkja ezzel szemben 16→4→16 profilt használ; az ábra csak az inverted esetet rajzolja, képletekből szerkesztett szemléltetés.

19.2.3 Width multiplier

A **width multiplier** (szélességi szorzó) [P34] $\alpha > 0$ olyan globális skálázó tényező, amely a hálózat **vékony csatornaszámait** szorozza: $C_{in} \mapsto \alpha C_{in}$, $C_{out} \mapsto \alpha C_{out}$, a köztes expanzió pedig $e\alpha C_{in}$ lesz. A téralakok változatlanok; α megváltoztatása tehát nem érinti a rövidebb út (skip) **téralak**-feltételét, csak a csatornaszámokat.

A width multipliert ne keverjük az expanziós faktorral: e egy **blokkon belüli** szélességarány, α a **hálózat egészére** ható erőforrás-skála. E fejezet kerekítési konvenciója: a szorzott vékony csatornaszámot (αC_{in}, αC_{out}) a legközelebbi egészre kerekítjük (a 0,5 törtrésznél felfelé), de legalább 1 csatornát megtartunk, és a köztes csatornaszám a kerekített érték e-szerese. Hardverbarát megvalósításban a 8 legközelebbi többszörösére kerekítés is szokásos, ezt a kísérlet tervezésekor rögzíteni kell. $\alpha = 0{,}5$ és $e = 3$ esetén a köztes csatornaszám képletszinten $0{,}5 \cdot 3C_{in} = 1{,}5C_{in}$; $C_{in} = 4$ mellett a vékony csatornaszám 2, a köztes 6, $C_{in} = 6$ mellett 3, illetve 9, tehát a kerekítés ezekben az esetekben nem változtat. Az alfejezet laborja ezért nem használ α-t: a blokk `expand` paramétere egész, és a width multipliert csak a képletekben, illetve az F5 kísérlettervében szerepeltetjük. A [P10] absztraktja a width, depth és resolution **együttes, koherens** skálázását vizsgálja; a width multiplier ehhez kapcsolódó eszköz, a pontos MobileNetV2-beli értéktáblázat a [P09] teljes szakaszának ellenőrzését igényli.

19.2.4 Lemma: alakok, költség és a rövidebb út (skip) alkalmazhatósága

Lemma (inverted residual blokk alakja, paraméter- és MAC-költsége, a rövidebb út feltétele). Legyen $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$. Az inverted residual blokk három konvolúcióból áll: 1×1 expanzió $C_{in} \to eC_{in}$, majd $k_h \times k_w$ depthwise szűrő *groups* $= eC_{in}$, stride $s \in \{1, 2\}$,

szimmetrikus padding p, dilation nélkül (azaz $d = 1$; a d dilatációt a 14.1. Stride, padding, dilation és kimeneti méret vezeti be), végül 1×1 projekció $eC_{in} \to C_{out}$. A konvolúciós biast nullának vesszük; BatchNorm esetén annak affin paramétereit külön számoljuk. A MAC (multiply–accumulate, azaz szorzás-akkumuláció) alatt a kimenethez ténylegesen hozzájáruló szorzás-összeadás műveletek számát értjük. Ekkor:

1. Az expanzió kimenete (N, eC_{in}, H, W).
2. A depthwise kimenete (N, eC_{in}, H_s, W_s), ahol

$$H_s = \left\lfloor \frac{H + 2p - (k_h - 1) - 1}{s} \right\rfloor + 1,$$

és W_s ugyanígy W-ből; a [D10] képlete $d = 1$ mellett. 3. A projekció kimenete $Y \in \mathbb{R}^{N \times C_{out} \times H_s \times W_s}$. 4. A **rövidebb út (skip) pontosan akkor alkalmazható** e specifikációban, ha a rövid út X és a főág Y alakja azonos: $C_{in} = C_{out}$, $s = 1$, és a padding biztosítja $H_s = H$, $W_s = W$. Ellenkező esetben az összeadás vagy hibát ad, vagy nem értelmezhető az adott specifikációban. 5. A konvolúciós súlyparaméterek száma bias nélkül

$$P = eC_{in}\,(C_{in} + k_h k_w + C_{out})\,,$$

a MAC-szám pedig

$$M = N \cdot H \cdot W \cdot eC_{in}^2 + N \cdot H_s \cdot W_s \cdot eC_{in} \cdot k_h k_w + N \cdot H_s \cdot W_s \cdot eC_{in} \cdot C_{out}.$$

Ha $C_{out} = C_{in}$ és $s = 1$ mérettartó paddinggel, akkor

$$P = eC_{in}\,(2C_{in} + k_h k_w)\,, \qquad M = eC_{in}\,N H W\,(2C_{in} + k_h k_w)\,.$$

Bizonyítás. Mindhárom réteg a [D10] közölt keresztkorrelációs specifikációját követi. Az expanzió bemeneti csatornaszáma C_{in}, kimeneti eC_{in}, kernel 1×1; mivel a kernel nem nyúl túl a térbeli pozíción, a kimeneti téralak (H, W). A depthwise *groups* $= eC_{in}$ beállítás azt jelenti, hogy minden bemeneti csatornához pontosan egy kimeneti csatorna tartozik, ezért a súlyalak $(eC_{in}, 1, k_h, k_w)$. A kimeneti méret a [D10] képlete $d = 1$ esetén. A projekció kernel 1×1, így a téralak (H_s, W_s) marad, a csatornaszám C_{out}. A (4) állítás az elemenkénti összeadás alakfeltételéből következik: $X + Y$ csak akkor értelmes, ha mind a négy dimenzió megegyezik; mivel itt H_s, W_s-et a stride/padding páros határozza meg, $s = 1$ és $H_s = H, W_s = W$ szükséges. A paraméterszámok: az expanzió $C_{in} \cdot eC_{in}$, a depthwise $eC_{in} \cdot k_h k_w$, a projekció $eC_{in} \cdot C_{out}$ szorzata. A MAC-szám minden kimeneti elemenként a kimenethez ténylegesen hozzájáruló szorzások száma: az expanzióban C_{in}, a depthwise-ban $k_h k_w$, a projekcióban eC_{in}. Ezért az expanzió $N \cdot H \cdot W \cdot eC_{in} \cdot C_{in}$, a depthwise $N \cdot H_s \cdot W_s \cdot eC_{in} \cdot k_h k_w$, a projekció $N \cdot H_s \cdot W_s \cdot C_{out} \cdot eC_{in}$ szorzás-akkumulációt ad, ami a fenti M képlet három tagja. Ezzel a lemma bizonyított; ez saját levezetés, nem a [P09] tétele.

Következmény. A rövidebb út (skip) alkalmazhatósága a **réteggráfon** lokálisan ellenőrizhető: meg kell nézni a blokk be- és kimeneti csatornaszámát, a depthwise stride-ját és a téralak-megőrző paddinget. Stride 2 esetén a rövid út kihagyandó, mert a főág downsamplinget hajt végre.

19.2.5 Kézi számpélda

Legyen $N = 1$, $C_{in} = C_{out} = 4$, $H = W = 8$, $e = 3$, $k_h = k_w = 3$, $s = 1$, $p = 1$. Ekkor az alakok:

1. X: $(1, 4, 8, 8)$;
2. expanzió után: $(1, 12, 8, 8)$;
3. depthwise után: $H_s = \lfloor (8 + 2 - 2 - 1)/1 \rfloor + 1 = 8$, tehát $(1, 12, 8, 8)$;
4. projekció után: $(1, 4, 8, 8)$.

Rövidebb út (skip): alkalmazható, mert $C_{in} = C_{out} = 4$, $s = 1$, és a 3×3, $p = 1$ padding téralaktartó. Paraméterek: $P = 3 \cdot 4(4 + 9 + 4) = 12 \cdot 17 = 204$. MAC: az expanzió $1 \cdot 64 \cdot 12 \cdot 4 = 3072$; a depthwise $1 \cdot 64 \cdot 12 \cdot 9 = 6912$; a projekció $1 \cdot 64 \cdot 12 \cdot 4 = 3072$; összeg $M = 13056$.

Stride 2 esetén ugyanezzel a bemenettel $H_s = \lfloor (8 + 2 - 2 - 1)/2 \rfloor + 1 = 4$, a projekció kimenete $(1, 4, 4, 4)$. A skip **nem** alkalmazható, mert a rövid út $(1, 4, 8, 8)$ alakú, a főág $(1, 4, 4, 4)$; az elemenkénti összeadás alakbeli (shape) hibát ad. Ha $C_{in} = 4$ de $C_{out} = 8$, stride 1 mellett a téralak egyezne, de a csatornaszám nem; e specifikációban szintén nincs skip, mert vetítő shortcutot nem vezettünk be.

19.2.6 Labor: kis inverted residual blokk PyTorchban

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n074/inverted_residual_demo.py` futtatható (`python inverted_residual_demo.py`); külső adatfájl nem kell.

A labor egyetlen fájl, `lab/n074/inverted_residual_demo.py`. Cél: a blokk implementálása, a stride 1 és 2 alakellenőrzése (shape), valamint a konvolúciós paraméterszám összevetése a kézi 204 értékkel. A BatchNorm a 18.2. Batch normalization train és eval módban szerint train módban batchstatisztikát számol; az alakjára nincs hatása, ezért a labor alak-kontrollja a BN állapotától független, de a modell kiértékeléséhez később `eval()` mód kell.

```
# lab/n074/inverted_residual_demo.py
import torch
import torch.nn as nn

class InvertedResidual(nn.Module):
    """Oktatási célú inverted residual, lineáris bottleneck-tel.

    F(X) = BN_p(Project(Act(BN_d(DW(Act(BN_e(Expand(X))))))))
    Y    = F(X) + X, ha stride == 1 es cin == cout.
    A [P09] ReLU6-ja helyett itt nn.ReLU fut (oktatási egyszerűsítés).
    """

    def __init__(self, cin, cout, expand=3, stride=1, activation=nn.ReLU):
        super().__init__()
        self.use_skip = (stride == 1) and (cin == cout)
        hidden = expand * cin
        self.conv = nn.Sequential(
            nn.Conv2d(cin, hidden, 1, bias=False),   # expanzió
            nn.BatchNorm2d(hidden),
            activation(),
            nn.Conv2d(hidden, hidden, 3, stride=stride,
                      padding=1, groups=hidden, bias=False),  # depthwise
            nn.BatchNorm2d(hidden),
            activation(),
```

```
            nn.Conv2d(hidden, cout, 1, bias=False),  # projekció, utána NINCS aktiváció
            nn.BatchNorm2d(cout),
        )

    def forward(self, x):
        y = self.conv(x)
        if self.use_skip:
            return y + x
        return y


def conv_params(module):
    """Csak a Conv2d súlyparaméterek száma, a BatchNorm affin paraméterei nélkül."""
    total = 0
    for m in module.modules():
        if isinstance(m, nn.Conv2d):
            total += sum(p.numel() for p in m.parameters())
    return total


def main():
    torch.manual_seed(20261020)
    x1 = torch.randn(1, 4, 8, 8, dtype=torch.float32)
    x2 = torch.randn(1, 4, 8, 8, dtype=torch.float32)

    m1 = InvertedResidual(4, 4, expand=3, stride=1)
    m2 = InvertedResidual(4, 4, expand=3, stride=2)

    y1, y2 = m1(x1), m2(x2)
    # A lemmából számított alakok; a futás assert-tel ellenőrzi.
    assert y1.shape == torch.Size([1, 4, 8, 8])
    assert y2.shape == torch.Size([1, 4, 4, 4])
    assert m1.use_skip is True
    assert m2.use_skip is False
    # Konvolúciós súlyok: 3*4*(4+9+4)=204; teljes modell ennél több a BN miatt.
    assert conv_params(m1) == 204
    print("y1 shape:", tuple(y1.shape))
    print("y2 shape:", tuple(y2.shape))
    print("conv params m1:", conv_params(m1))
    print("total params m1:", sum(p.numel() for p in m1.parameters()))

if __name__ == "__main__":
    main()
```

Futtatás a lab könyvtárból:

```
cd lab/n074
python inverted_residual_demo.py
```

Telepítés. A labor csak a PyTorchot igényli, CPU-változat is elég; a rögzített labor-környezetben a `torch 2.15.0+cpu` volt telepítve, NumPy-t a példa nem használ. A futtatás a kiadott laborcsomag `lab/n074` mappájából történik, amelyet a csomagépítés a fejezet `# lab/n074/...` fejlécű kódblokkjából állít elő.

A rögzített környezetű ellenőrző futás mért kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
y1 shape: (1, 4, 8, 8)
y2 shape: (1, 4, 4, 4)
conv params m1: 204
total params m1: 260
```

Minden `assert` teljesült; a futás a rögzített venvben egyszálú CPU-n 0,94 s falidőt vett igénybe, az interpreter és a `torch` importjával együtt. A teljes paraméterszámban a három BatchNorm affin paraméterei is benne vannak: a két rejtett ($eC_{in} = 12$ csatornás) BatchNorm $2 \cdot 2 \cdot 12 = 48$, a kimeneti ($C_{out} = 4$) $2 \cdot 4 = 8$, így $204 + 48 + 8 = 260$. A labor a konvolúciós 204 értéket ellenőrzi, hogy a lemmával összevethető legyen.

19.2.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el saját szavakkal, mit jelent az expanzió, a lineáris bottleneck és az inverted residual. Adja meg a csatornaprofilt $C_{in} = 6$, $e = 4$ esetén. Mitől „fordított” ez a 18.1. Reziduális blokk és identitásút hagyományos reziduális blokkjához képest? Egy ellenpéldával mutassa meg, hogy a 19.1. Depthwise separable és pointwise réteg depthwise separable sorozat önmagában még nem inverted residual. *Elvárt eredmény:* legfeljebb fél oldal, a három definíció, a profil $6 \to 24 \to 6$ és az ellenpélda.

F2 (levezetési, nehézség 2). Adott $N = 2$, $C_{in} = C_{out} = 8$, $H = W = 16$, $e = 2$, $k_h = k_w = 3$, $s = 2$, $p = 1$, bias nélkül. Számolja ki az összes köztes tenzoralakot, H_s értékét, a konvolúciós paraméterszámot és a MAC-et. Döntse el a skip alkalmazhatóságát, és írja fel az általános skip-feltételt. *Elvárt eredmény:* lépésenkénti levezetés alakellenőrzéssel és a skip indoklásával.

F3 (implementációs, nehézség 2). Módosítsa a laborblokkot úgy, hogy a `main()` függvény a fenti $N = 2$, $C_{in} = 8$, $e = 2$, stride 2 esetre is lefusson, és `assert`-ekkel ellenőrizze a kimeneti alakot, a `use_skip` értékét és a konvolúciós paraméterszámot. *Elvárt eredmény:* teljes futtatható kód, parancs, a várt értékek.

F4 (hibakeresési, nehézség 2). Az alábbi két hiba közül válasszon egyet, lokalizálja, írja le az okát és következményét, javítsa, és adja meg a visszaellenőrzés lépését.

```
# A) hibás skip: a stride-ot figyelmen kívül hagyja
self.use_skip = True
# B) hibás bottleneck: a projekció után is aktiváció fut
nn.ReLU(),
nn.Conv2d(hidden, cout, 1, bias=False),
nn.ReLU(),
```

Elvárt eredmény: diagnózis, javított kódrészlet, ellenőrző futtatás várt eredménye.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen GlyphLab-kísérletet (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) a width multiplier hatására: $\alpha = 1{,}0$ baseline és $\alpha = 0{,}5$ változat, minden más rögzített. Használja a 7.2. Softmax és többosztályos cross-entropy logitos cross-entropy konvencióját és a GlyphLab scene-szintű splitjét. *Elvárt eredmény:* kérdés, hipotézis, adat/split, baseline, változó tényező, metrika, best-checkpoint szabály, CPU erőforrás becslés és konkrét érvényességi korlát. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

19.2.8 Megoldások és indoklás

F1. Az expanzió 1×1-es csatornanövelés $C_{in} \to eC_{in}$; a lineáris bottleneck a projekciós 1×1 utáni nemlinearitás elhagyása, hogy a vékony reprezentációt ne törje meg aktiváció; az inverted residual vékony be/kimenettel és széles köztes réteggel dolgozik, a shortcut a vékony reprezentációkat köti össze. $C_{in} = 6$, $e = 4$ profil: $6 \to 24 \to 6$. A 18.1. Reziduális blokk és identitásút hagyományos blokkja ezzel szemben széles határral és szűk középpel dolgozik; a „fordított” jelző erre a megcserélt profilra utal. A 19.1. Depthwise separable és pointwise réteg depthwise

separable sorozat például $8 \to 8 \to 4$ csatornamenetet adhat, de nincs vékony–széles–vékony profil és nincs reziduális összeadás; tehát nem inverted residual.

F2. Bemenet $(2, 8, 16, 16)$. Expanzió: $eC_{in} = 16$, kimenet $(2, 16, 16, 16)$. Depthwise 3×3, $s = 2$, $p = 1$:

$$H_s = \left\lfloor \frac{16 + 2 - 2 - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{15}{2} \right\rfloor + 1 = 7 + 1 = 8,$$

kimenet $(2, 16, 8, 8)$. Projekció: $(2, 8, 8, 8)$. Skip nem alkalmazható, mert a rövid út $(2, 8, 16, 16)$, a főág $(2, 8, 8, 8)$; az általános feltétel az alakazonosság: $C_{in} = C_{out}$, $s = 1$ és téralak-tartó padding mellett $H_s = H, W_s = W$. Paraméterek: $P = 2 \cdot 8(8 + 9 + 8) = 16 \cdot 25 = 400$. MAC: expanzió $N \cdot H \cdot W \cdot eC_{in} \cdot C_{in} = 2 \cdot 256 \cdot 16 \cdot 8 = 65536$; depthwise $2 \cdot 64 \cdot 16 \cdot 9 = 18432$; projekció $2 \cdot 64 \cdot 16 \cdot 8 = 16384$; összeg $M = 100352$. A floor-képlet páratlan H esetén is a helyes értéket adja: például $H = 17$ mellett $\lfloor 16/2 \rfloor + 1 = 9$, összhangban a stride-os konvolúció kimeneti méretével.

F3. A feladat a laborblokk kiterjesztése. Az alábbi önálló fájl a labor `inverted_residual_demo.py` moduljára épül, és a `lab/n074/` könyvtárból futtatható:

```python
# lab/n074/f3_extended.py
# Futtatás a lab/n074 könyvtárból: python f3_extended.py
import torch
from inverted_residual_demo import InvertedResidual, conv_params


def main():
    torch.manual_seed(20261020)
    x = torch.randn(2, 8, 16, 16, dtype=torch.float32)
    model = InvertedResidual(8, 8, expand=2, stride=2)
    y = model(x)
    assert y.shape == torch.Size([2, 8, 8, 8])
    assert model.use_skip is False
    assert conv_params(model) == 400
    print("F3 OK:", tuple(y.shape), conv_params(model))


if __name__ == "__main__":
    main()
```

A várt kimenet `F3 OK: (2, 8, 8, 8) 400`; a paraméterszám az F2 levezetéséből adódik, az alak pedig a stride 2-es depthwise kimenetéből. A közölt kimenet a képletekből levezetett várt érték, nem mért futási eredmény.

F4. Az A) hiba oka: a `use_skip` engedélyezése stride 2 mellett is; következménye a forwardban `y + x` kísérlet, ahol a főág $(N, C, 4, 4)$, a rövid út $(N, C, 8, 8)$; a PyTorch ilyenkor alakbeli eltérés (shape mismatch) hibáját adja (a batch mérettől függetlenül). A két alak $4 \neq 8$ miatt nem is broadcastolható; csendben hibás eredmény csak akkor születne, ha valamelyik térdimenzió 1 volna (például egy globális átlagolás után $H_s = 1$). Javítás: `self.use_skip = (stride == 1) and (cin == cout)`. Visszaellenőrzés: stride 2 modellre `forward` hívás alakot ellenőrző `assert`-tel, stride 1 modellre a skip aktív. A B) hiba: a projekció után ReLU megtöri a lineáris bottleneck definíciót; a kis csatornaszámú kimenet további nemlinearitása pont azt a kockázatot hozza be, amelyet a [P09] 3.3. szakasza szerint a terv elkerül. Javítás: az utolsó ReLU törlése; visszaellenőrzés a blokk modullistájának vizsgálatával, hogy az utolsó konvolúció után csak BatchNorm áll.

F5. Minta terv. Kérdés: csökkenti-e $\alpha = 0{,}5$ a GlyphLab top-1 pontosságát, és hogyan változik a korrupt teszt páros romlása? Hipotézis: a width csökkentése rontja a kapacitást, de kis adaton a regularizáló hatás részben ellensúlyozhat. Adat: GlyphLab `data/images.npy` és

`data/metadata.csv`, a rögzített scene-szintű split (120/30/30 jelenet, azaz 240/60/60 kép, a `scene_id` a megfigyelési egység); a modell kiválasztása csak a train/val spliten, a végső teszten egyszer. Változó tényező: α; baseline az $\alpha = 1{,}0$ azonos architektúra. A csatornaszámok kerekítésére a width-multiplier szakasz konvencióját (legközelebbi egész, legalább 1) rögzítse. Recept: AdamW lr $= 10^{-3}$, weight decay 10^{-4}, batch 64, 30 epoch, early stopping 5 türelemmel; öt seed (20261020–20261024). Metrika: top-1 accuracy; a korrupt tesztet párosítva a clean testtel. Best checkpoint: legalacsonyabb val loss; a 7.2. Softmax és többosztályos cross-entropy szerint logitos `CrossEntropyLoss`, int64 target. Erőforrás: CPU, kis batch, a GlyphLab 24×24-es mérete miatt néhány ezer lépés. Korlát: a GlyphLab nem természetes képi benchmark; az öt seed a tanítási zajt becsli, az adatmintavétel bizonytalanságát nem, a width hatása a kis szélességek mellett kerekítésfüggő lehet; a korrupt teszt csak a befagyasztott modellen értékelhető.

19.2.9 Gyakori tévedés ellenpéldával

A MobileNet-család verzióit nem szabad részletezés nélkül összemosni. A MobileNetV1 szellemiségéhez a 19.1. Depthwise separable és pointwise réteg depthwise separable szakasza áll közel; a MobileNetV2 a [P09] 3.2–3.3. szakasza szerint az **inverted residual** és **linear bottleneck** fogalmát vezeti be. Ha valaki azt mondja, hogy „a MobileNetV2 csak depthwise separable konvolúciók egymásutánja”, az ellenpélda a shortcut: a vékony–széles–vékony profil reziduális összeadása nem következik a separable kompozíció puszta definíciójából. Hasonló hiba a width multiplier és az expanziós faktor felcserélése: $\alpha = 0{,}5$ az egész hálózat vékony csatornaszámait skálázza, $e = 3$ pedig egy blokk belső szélességét; a kettő szorzata $\alpha e C_{in}$, nem pedig az e „felezése” önmagában. A MobileNet további változatait (V1, V3) itt nem részletezzük; a fenti táblázat csak a [P09]-ben ellenőrzött V2-blokkra vonatkozik.

19.2.10 Önellenőrzés és visszahívás

Ellenőrizze, hogy (1) $e = 1$ esetén a blokk profilja $C_{in} \to C_{in} \to C_{in}$; a depthwise továbbra is csatornánként szűr, és a shortcut feltétele a szokásos stride/padding szabály. (2) $C_{in} = 4$, $C_{out} = 8$, $s = 1$, $p = 1$ esetén a skip **nem** alkalmazható a csatornaeltérés miatt; az `x + y` alakbeli (shape) hibát adna. (3) Stride 2 mellett a depthwise downsamplingje után a projekció 1×1 már nem változtatja a téralakot, ezért a stride-ot a középső rétegben kell keresni. Visszahívás a 18.2. Batch normalization train és eval módban egységből: a BatchNorm train módban batchstatisztikát és futó buffert frissít, eval módban a running statisztikát használja; a blokk alaklemmája (shape) ettől független, de egy tényleges tanítási kísérletben a módváltást a checkpoint-protokollal együtt kell rögzíteni. A 15.2. Inputgradiens és átfedő recepciós mezők egységből: a depthwise konvolúció visszafelé menetben a bemeneti gradiensét csatornánként, a kernelt tükrözve végzett keresztkorrelációval állítja elő, a súlygradienst pedig a térbeli pozíciók és a batch fölött összegzi; implementációs szinten ez „szórva összegzés” (scatter-add, col2im), nem egyszerű csatornánkénti összeadás. Az 1×1-es rétegek csatornakeverő gradienst szállítanak. A 7.2. Softmax és többosztályos cross-entropy egységből: osztályozási fej logitokat adjon, cél int64, softmaxot ne tegyünk a loss elé.

19.2.11 Rövid összefoglaló

Az inverted residual blokk vékony be- és kimeneti reprezentációk közé széles, depthwise szűrővel feldolgozott expanziót illeszt; a projektáló 1×1 utáni nemlinearitás elhagyása a lineáris bottleneck. A lemmában levezettük az összes köztes alakot, a paraméter- és MAC-képleteket, valamint a skip alakfeltételét: $C_{in} = C_{out}$, stride 1 és téralak-tartó padding. A $4 \to 12 \to 4$, 8×8-as példán

a konvolúciós paraméterszám 204, a MAC 13056; stride 2 esetén a skip nem alkalmazható. A width multiplier a vékony csatornákat globálisan skálázza, és nem azonos az expanziós faktorral. A labor PyTorchban ellenőrzi az alakokat (shape) és a 204 súlyt; a kimenet a rögzített környezetű ellenőrző futás mért értéke.

19.2.12 Következő alkalmazás

A 19.3. EfficientNet és architektúraskálázás egység az EfficientNet [P10] megközelítését tárgyalja: ott a width, depth és resolution együttes, koherens skálázását vizsgáljuk, ami a width multiplier itteni fogalmát hálózatszintű keresési problémává bővíti. A 19.4. ConvNeXt és a recept/architektúra elválasztása ConvNeXt-egység pedig a modernizált CNN-recepteket választja el az architektúra nevének puszta említésétől [P11].

19.2.13 Felhasznált források

- [P09] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. és Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. arXiv:1801.04381 (v4). A hivatkozott helyek: 3.2–3.3. szakasz, 1–2. táblázat, 4. ábra. https://arxiv.org/abs/1801.04381.
- [P10] Tan, M. és Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. arXiv:1905.11946. https://arxiv.org/abs/1905.11946.
- [P11] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T. és Xie, S. (2022). *A ConvNet for the 2020s*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022). https://arxiv.org/abs/2201.03545.
- [P34] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. és Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv:1704.04861. https://arxiv.org/abs/1704.04861.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

19.3 EfficientNet és architektúraskálázás

19.3.1 Cél, előismeret és induló kérdés

A 19.2. MobileNet és inverted residual egységben a MobileNetV2 inverted residual blokkját és a width multiplier fogalmát vizsgáltuk. Ott a width multiplier egyetlen beállítás volt; most azt kérdezzük, hogyan skálázzuk együtt egy CNN mélységét, szélességét és bemeneti felbontását. Előismeret: a konvolúciós réteg MAC-számítása 14.3. Paraméterszám, MAC és aktivációmemória, a normrétegek paraméterezése 18.3. Pre-activation, kis batch és alternatív normalizálás, valamint az osztályozási loss-konvenció 7.3. Bináris, multilabel és regressziós cél. Induló becslés: ha egy hálózat minden csatornaszámát kétszerezzük, vajon a paraméterszám és a műveletszám is kétszereződik? Nem: a belső konvolúciókban a bemeneti és a kimeneti csatornaszám is kétszereződik, ezért a súlyok száma és a szorzás-akkumulációk száma négyszereződik; csak a rögzített bemenetű első réteg (a stem) skálázódik kétszeresen. A teljes hálózat szorzója ezért a stem részarányától függően 2 és 4 közé esik, amint azt a következő költségállítás pontosan megadja.

19.3.2 Definíciók és jelölések

Legyen egy baseline CNN L_0 darab $k_i \times k_i$ kernelű, stride 1-es, azonos térbeli alakot megőrző paddinges, groups=1 konvolúciós rétege. A bemeneti kép $H_0 \times W_0$, az i-edik réteg bemeneti és kimeneti csatornaszáma $c_{in,i}$ és $c_{out,i}$. A bemeneti képbatch $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ NCHW alakú [D10].

A három skálázási dimenziót szorzókként értelmezzük:

- **Width, w:** minden belső réteg csatornaszámának közös szorzója. A width multiplier 19.2. MobileNet és inverted residual az egész hálózat csatornáira vonatkozik, ellentétben a blokkon belüli expanziós faktorral.
- **Depth, d:** a blokkok/rétegszám szorzója; a baseline L_0 rétegszámát d-vel szorozva dL_0 rétegű változatot készítünk azonos alakú rétegek hozzáadásával.
- **Resolution, r:** a bemeneti felbontás szorzója, azaz rH_0 és rW_0 bemeneti méret; stride 1 és azonos padding mellett minden feature map mérete r-szeres.
- **Compute budget, B:** egy felső korlát a forward cross-correlation szorzás-akkumulálás műveletszámára (MAC), amelyen belül modellváltozatokat hasonlítunk; a kísérletekben konkrét értékét a baseline modell MAC-jához viszonyítva adjuk meg.

A skálázott modell csatornáit $wc_{in,i}$-ként értelmezzük; valós w esetén a tényleges implementáció kénytelen egész csatornaszámot használni. A labor kódja `int(c * width)`-tel lefelé csonkol, nem kerekít; más implementációk kerekíthetnek, ezért a konvenciót mindig rögzíteni kell. A költségmodellben valós szorzóval dolgozunk.

19.3.3 A skálázás számítási költsége

Állítás (a rögzített bemeneti csatornaszám hatása). Jelölje M_{stem} a rögzített bemeneti csatornaszámú első konvolúció, M_{body} pedig az utána következő, csatornák között skálázott konvolúciók baseline-MAC-ját. Ekkor $M_0 = M_{\text{stem}} + M_{\text{body}}$. Itt a **stem** a hálózat első, rögzített bemeneti csatornaszámú (például RGB-nél 3) konvolúciója, a **body** az utána következő, csatornák között skálázott konvolúciós rész, a **fej** pedig a záró osztályozó rész (globális átlagolás és lineáris réteg), amelyet a költségmodell külön kezel. Tegyük fel, hogy a konvolúciók `groups=1` típusúak, a térbeli méret mindkét tengelyen r-szeresére nő, és a csatornaszám-kerekítést figyelmen kívül hagyjuk. Ha a belső csatornaszámok w-szeresükre nőnek, **változatlan mélység** mellett a pontos skálázási képlet:

$$M(w, 1, r) = r^2 \left(w M_{\text{stem}} + w^2 M_{\text{body}} \right).$$

Ha a hálózatot ezenfelül olyan belső blokkok ismétlésével mélyítjük, amelyek be- és kimeneti csatornaszáma azonos ($C_{in} = C_{out}$), így az ismételt példányok alakkompatibilisek, egy példányuk alap MAC-ja M_{body}, és a példányok között nem kell további illesztőréteg, akkor az ismételt blokkra a következő képlet adódik:

$$M(w, d, r) = r^2 \left(w M_{\text{stem}} + d w^2 M_{\text{body}} \right).$$

Valós architektúrákban a depth skálázása új átmeneti vagy downsampling rétegeket igényelhet, ezért az utóbbi képlet ott közelítés, nem univerzális azonosság.

Indoklás. Egy standard keresztkorrelációs réteg MAC-ja $c_{out} c_{in} k^2 H_{out} W_{out}$. A belső rétegekben mindkét csatornadimenzió w-vel, a térbeli dimenziók szorzata r^2-tel változik, tehát a költség

w^2r^2-szeres. Az első réteg bemeneti csatornaszáma viszont rögzített (például RGB-nél 3), így ott a költség wr^2-szeres. Alakkompatibilis belső blokk d-szeres ismétlése további d tényezőt ad, míg az első réteg egyszer szerepel. A két költséget összeadva kapjuk a formulát.

A [P10] által tárgyalt, széles körben használt $dw^2r^2M_0$ szabály ezért **csak közelítés**. Ebben az egyszerű modellben a hozzá viszonyított eltérés:

$$M_{\text{approx}} - M_{\text{exact}} = r^2(dw^2 - w)M_{\text{stem}}.$$

$d = 1$, $w \geq 1$ esetén a közelítés relatív hibája $w \to \infty$ mellett **nem nullához tart**, hanem $M_{\text{stem}}/M_{\text{body}}$-hoz. A rögzített bemeneti csatornaszámú első réteg tehát akkor hanyagolható el, ha *már a baseline költségéből is kis részt képvisel*, nem pusztán attól, hogy a hálót szélesítjük. A bias-műveletek és BatchNorm paraméterei nem szerepelnek a konvolúciós MAC ezen definíciójában. A belső konvolúciók súlyparamétereinek költsége hasonló okból w^2-tel változik, de a teljes modell pontos paraméterszámához a stemet, a fejet és a normalizálást külön hozzá kell adni.

19.3.4 Compound scaling: az együttes elv

Az iménti költségállítás csak a műveletszámot írja le; abból **nem következik**, hogy milyen w, d, r arány adja a legjobb pontosságot adott B mellett. A [P10] absztraktja szerint a szerzők azt találták, hogy a mélység, szélesség és felbontás kiegyensúlyozott, együttes skálázása jobb hatékonyságú modelleket adhat, mint egyetlen dimenzió növelése.

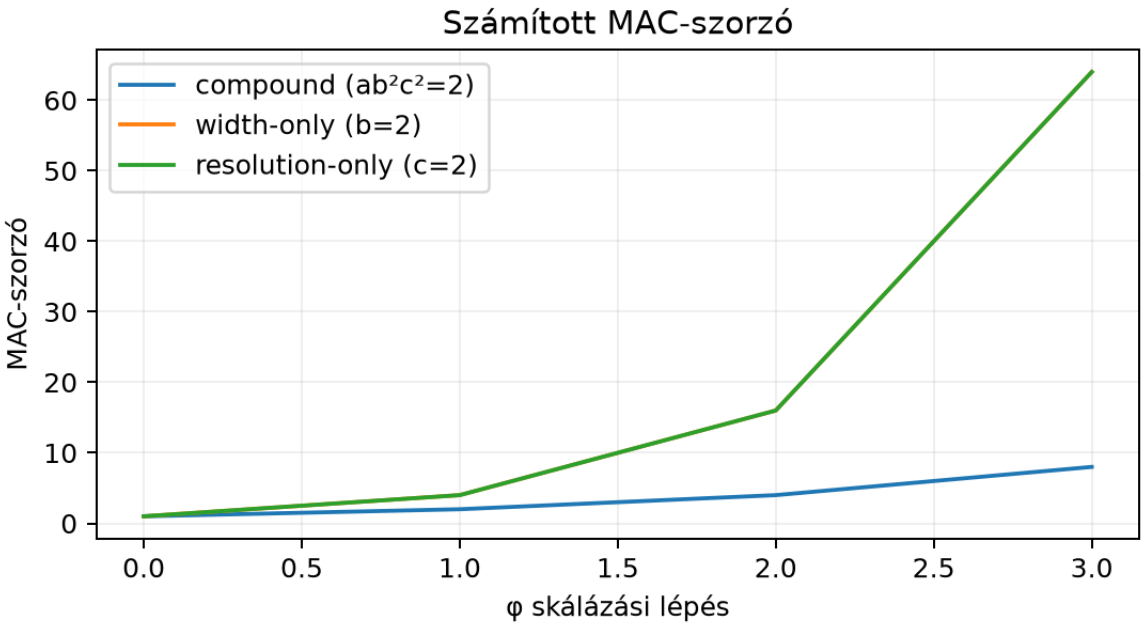
A módszer elterjedt paraméterezése az alábbi alakot használja. Egy ϕ compound coefficient és három pozitív, a keresés során rögzített arány (a, b, c) mellett

$$d = d_0 a^\phi, \qquad w = w_0 b^\phi, \qquad r = r_0 c^\phi,$$

ahol $d_0 = w_0 = r_0 = 1$ a baseline skálát jelöli. A [P10]-ben ugyanezek a mennyiségek α, β, γ néven szerepelnek (itt $a = \alpha$, $b = \beta$, $c = \gamma$), és a szerzők konkrét értékeket adnak rájuk: a mélység együtthatója 1,2, a szélességé 1,1, a felbontásé 1,15, ahol az együtthatók legalább 1-esek, a tervezési kényszer pedig $ab^2c^2 \approx 2$. Ezekkel az értékekkel $ab^2c^2 = 1{,}2 \cdot 1{,}1^2 \cdot 1{,}15^2 \approx 1{,}92$, tehát a MAC lépésenként valamivel kevesebb, mint kétszeresére nő; a [P10] a 2-höz közeli értéket tekinti tervezési elvnek. A fenti egyszerűsített, belső rétegek dominanciájára építő költségmodell szerint a MAC-szorzó közelítőleg

$$a^\phi (b^\phi)^2 (c^\phi)^2 = (ab^2c^2)^\phi,$$

tehát ha egy lépésben ϕ eggyel nő, és az $ab^2c^2 \approx 2$ feltételt választják, akkor a modell MAC-ja lépésenként körülbelül kétszereződik. Ez az algebrai szerkezet a fenti költségmodellből származtatható, de az, hogy mely a, b, c értékek optimálisak, **empirikus keresési eredmény, nem elméleti optimumtétel**. A [P10] konkrét értékei az EfficientNet-B0 baseline-ra végzett keresésből származnak; más baseline, adat és hardver esetén az arányokat újra kell keresni. Az alábbi ábra képletből számolt szemléltetés: a compound ($ab^2c^2 = 2$), a width-only ($b = 2$) és a resolution-only ($c = 2$) stratégia MAC-görbéje.



19.2. ábra: Számított MAC-szorzó

Az x tengely a skálázási lépés, az y tengely a baseline MAC-hoz viszonyított szorzó. A compound görbe (ab²c²=2) értéke 2^, a width-only görbe b=2 miatt (b)²=4 , a resolution-only görbe c=2 miatt (c)²=4 . A depth-only görbe a=2 esetén a compoundtal azonos lenne, ezért külön nem szerepel. Az első réteg (stem) eltérése nincs beleszámítva.

19.3.5 Kézi számpélda

Vegyünk egy 8×8-as, 3 csatornás bemenetű baseline modellt négy konvolúciós réteggel: $3 \to 4$, $4 \to 8$, $8 \to 16$, $16 \to 8$, mind 3×3 same padding, stride 1. Az egyes rétegek MAC-ja:

$$m_1 = 4 \cdot 3 \cdot 9 \cdot 64 = 6912, \quad m_2 = 8 \cdot 4 \cdot 9 \cdot 64 = 18432,$$

$$m_3 = 16 \cdot 8 \cdot 9 \cdot 64 = 73728, \quad m_4 = 8 \cdot 16 \cdot 9 \cdot 64 = 73728,$$

tehát $M_0 = 172\,800$. Válasszunk $w = 2$, $d = 1$, $r = 2$ skálázást. Az egyszerűsített $dw^2r^2M_0$ becslés $1 \cdot 4 \cdot 4 \cdot 172\,800 = 2\,764\,800$. A pontos érték, az első réteg $3 \to 8$ alakjával számolva: $55\,296 + 294\,912 + 1\,179\,648 + 1\,179\,648 = 2\,709\,504$. A közelítő hiba 55 296 MAC, azaz körülbelül 2% a pontos költséghez viszonyítva. Nagyobb w esetén ez az arány monoton nő, és $d = r = 1$ mellett a tényleges költséghez viszonyított relatív hiba a $M_{\text{stem}}/M_{\text{body}} \approx 4{,}17\%$ határhoz tart: $w = 2$-nél 2,04%, $w = 10$-nél 3,73%. A szélesítés tehát nem tünteti el a stem elhanyagolásából eredő hibát.

19.3.6 Minimális kód és labor

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A kód a `lab/n075/` munkakönyvtárban futtatható; a mérés CPU-n, float32-ben, rögzített seed mellett fut, külső adatfájl nem kell.

A labor két kis modellt hasonlít össze azonos bemeneten: `width=1.0` és `width=1.4`. A mérés CPU-n fut, float32 dtype-tal, rögzített seed mellett. A kód a `lab/n075/` munkakönyvtárban futtatható:

```
# lab/n075/n075_scale_bench.py
import statistics
import time
import torch
from torch import nn

torch.set_num_threads(1)

def make_cnn(width, base=(16, 32, 64, 128), num_classes=10):
    ch = [max(1, int(c * width)) for c in base]
    layers, prev = [], 3
    for c in ch:
        layers += [nn.Conv2d(prev, c, 3, padding=1),
                   nn.BatchNorm2d(c), nn.ReLU()]
        prev = c
    layers += [nn.AdaptiveAvgPool2d(1), nn.Flatten(),
               nn.Linear(prev, num_classes)]
    return nn.Sequential(*layers)

def count_params(model):
    return sum(p.numel() for p in model.parameters())

def bench_forward(model, x, warmup=5, iters=30):
    """Egy forward átlaga, mediánja és szórása ezredmásodpercben."""
    model.eval()
    with torch.no_grad():
        for _ in range(warmup):
            model(x)
        times = []
        for _ in range(iters):
            t0 = time.perf_counter()
            model(x)
            times.append(1000.0 * (time.perf_counter() - t0))
    return (statistics.mean(times), statistics.median(times),
            statistics.stdev(times))

torch.manual_seed(0)
x = torch.randn(4, 3, 24, 24, dtype=torch.float32)
expected = {1.0: 99_210, 1.4: 190_731}
for w in (1.0, 1.4):
    m = make_cnn(w)
    assert count_params(m) == expected[w]
    m.eval()
    with torch.no_grad():
        assert tuple(m(x).shape) == (4, 10)
    mean, med, sd = bench_forward(m, x)
    print(f"width={w:.1f} params={count_params(m):,} "
          f"mean={mean:.2f} ms median={med:.2f} ms std={sd:.2f} ms")
```

A rögzített környezetű ellenőrző futás kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, Intel Core i5-13400F, CPU, `torch.set_num_threads(1)`, seed 0):

```
width=1.0 params=99,210 mean=4.81 ms median=4.79 ms std=0.12 ms
width=1.4 params=190,731 mean=10.60 ms median=10.59 ms std=0.13 ms
```

Három ismételt futásban az átlagos forward idő `width=1.0` esetén 4,81–5,49 ms, `width=1.4` esetén 10,20–10,60 ms között mozgott; az egy futáson belüli szórás 0,12–0,42 ms volt. Az idő hardverfüggő, ezért csak ezen a gépen érvényes; az összehasonlítás azonos gépen, azonos folyamaton belül, melegítés és ismétlés mellett értelmes, small batch esetén az időmérés zaját a melegítési és ismétlési szám növelésével, valamint a medián és a szórás jelentésével kontrolláljuk.

19.3.7 Konkrét labor: két modell méret- és időmérése

Protokoll. Kérdés: hogyan változik a méret és az inference-idő a width növelésekor? Input: egy rögzített 4×3×24×24-es véletlen float32 tensor; modellek: a fenti `make_cnn` két width értékkel. Eszköz: helyi CPU, `device="cpu"`, `torch.set_num_threads(1)`, `torch.no_grad()` és `model.eval()`. Seed: `torch.manual_seed(0)` csak a bemenet és a súlyok inicializálásához szükséges; a mérési zajt melegítés és 30 ismételt forward átlaga, mediánja és szórása jellemzi. Parancs a `lab/n075` könyvtárból: `python n075_scale_bench.py`. Ellenőrzés: a paraméterszámokat a kód `assert`-jei és az alábbi táblázat rögzíti, a futásidő környezetfüggő.

width	csatornák	mért paraméterszám	megjegyzés
1.0	16, 32, 64, 128	99 210	egyezik a kézi számítással
1.4	22, 44, 89, 179	190 731	mért, `int(c*width)` csonkolással

A beadandó eredmény: a tényleges futás után a nyers mért időpár, a hányados, a PyTorch-verzió és a processzor azonosítója. A fenti táblázat a rögzített környezetű ellenőrző futás mért paraméterszámait tartalmazza; a labor akkor vált ellenőrzötté, ha a hallgató saját környezetben rögzíti a nyers kimenetet.

19.3.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem azonos a width multiplier w és a MobileNetV2 expanziós faktora e (a [P09] jelölésében t). Adjon egy ellenpéldát, amelyben az egyik nő, a másik nem.

F2 (levezetési, nehézség 2). A fenti baseline $M_0 = 172\,800$ MAC-ú modellre $w = 1{,}5$, $d = 3$, $r = 1$ választást alkalmazunk. Vezesse le az egyszerűsített közelítő MAC-szorzót, majd az elméleti, alakkompatibilis belső blokk ismétlésének feltétele mellett a stem–body képlet szerinti MAC-ot. Adja meg a $w^2 d$ belső paraméterszorzót is.

F3 (implementációs, nehézség 2). Írjon függvényt `scale_estimate(M0, w, d, r)`, amely az egyszerűsített költségbecslést adja vissza, majd ellenőrizze a $(M_0, w, d, r) = (172800, 2, 1, 2)$ bemenetre a 2 764 800-as értéket. Tolerancia: relatív 10^{-3}.

F4 (hibakeresési, nehézség 2). Az alábbi kód a MAC-szorzót lineárisnak számolja: `cost = w*d*r*M0`. Mi a hiba, mi a következménye egy skálázási döntésben, és hogyan javítjuk? Ellenőrizze az F3 példán.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adaton adott B compute budget mellett: három változat – depth-növelés, width-növelés és a kettő kombinációja –, a MAC-ot azonos B budget alatt tartva. Adja meg a B konkrét értékét, a változatok depth és width paramétereit, a megfigyelési egységet,

a splitet, a baseline-t, a metrikát és a következtetési korlátot. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

19.3.9 Megoldások és indoklás

F1. Az expanziós faktor egy blokk belső expanziós rétegének csatornaszorzója, míg a width multiplier a teljes hálózat minden konvolúciós rétegének csatornáit skálázza. Ellenpélda: egyetlen inverted residual blokkban $e = 6$ (a [P09] jelölésével $t = 6$), de $w = 1$; hálózatszintű $w = 2$ mellett a blokk bemeneti és kimeneti vékony csatornája duplázódik, a belső expanziós arány e viszont változatlan maradhat.

F2. Az egyszerűsített költségszorzó $dw^2r^2 = 3 \cdot 1{,}5^2 \cdot 1^2 = 6{,}75$, így a közelített MAC $6{,}75 \cdot 172\,800 = 1\,166\,400$. Az alakkompatibilis belső blokk ismétlését feltételező pontos képlet $M_{\text{stem}} = 6912$, $M_{\text{body}} = 165\,888$ mellett $1{,}5 \cdot 6912 + 3 \cdot 1{,}5^2 \cdot 165\,888 = 1\,130\,112$ MAC. A belső súlyparaméterek skálaszorzója $w^2d = 6{,}75$; a stem és a fej külön számolandó.

F3. Megoldás:

```
def scale_estimate(M0, w, d, r):
    return d * (w ** 2) * (r ** 2) * M0

out = scale_estimate(172800, 2.0, 1.0, 2.0)
assert abs(out - 2_764_800) / 2_764_800 < 1e-3

# Független ellenőrzés: a stem-body bontásból számolt MAC, ahol a stem
# w-vel (egy bemeneti csatorna), a body d*w^2-tel, és mindkettő r^2-tel skálázódik.
exact = 2 * 6912 * 2**2 + 1 * 2**2 * 165_888 * 2**2
assert exact == 2_709_504
assert abs(out - exact) / exact < 0.03
```

A hányados-ellenőrzés a **közelítő képlet implementációját** ellenőrzi, nem a tényleges modell MAC-jával való egyezést; a 2 764 800-as érték azért jön ki, mert $d = 1$ mellett a stem és a body skálázása egybeesik. Az utóbbihoz a stem és a belső blokkok költségét külön kell számolni: a stem–body bontásból $2 \cdot 6912 \cdot 2^2 + 1 \cdot 2^2 \cdot 165\,888 \cdot 2^2 = 2\,709\,504$ adódik, amit a közelítés 55 296 MAC-kal, mintegy 2%-kal felülbecsül.

F4. A kód a depth d-vel való közelítő lineáris skálázását tartalmazza, de a felbontás két tengelyének együttes r^2 hatását és a width belső rétegekre érvényes w^2 hatását nem: minden belső rétegben a súly csatorna-csatorna szorzata w^2 miatt nő. Következmény: a modell a tényleges MAC-ját alulbecsüli, így adott budgetre túl nagy width-et választ. Javítás: `cost = d * w**2 * r**2 * M0`. Az F3 példán `2**2 * 1 * 2**2 * 172800 = 2764800`, ellentétben a hibás `2*1*2*172800=691200` értékkel.

F5. Minta terv. Adat: a 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték szerinti GlyphLab: 180 jelenet × 2 nézet = 360 kép, 24×24-es, egycsatornás, három osztály; a split jelenetszintű és osztályonként rétegzett (class-stratified), 120/30/30 jelenet, azaz 240/60/60 kép. Input: `data/images.npy` (360,1,24,24) float32, `data/labels.npy` (360,) int64; a korrupt teszt (`corrupted_test.npy`) csak a végén, a befagyasztott modellen értékelhető. Baseline modell: 3×3-as stem $1 \to 16$, két darab, a 19.2. MobileNet és inverted residual alfejezet `InvertedResidual` blokkja ($C_{in} = C_{out} = 16$, $e = 3$, stride 1), globális átlagolás és lineáris fej $16 \to 3$; a 24×24-es bemeneten számított MAC $M_0 = 2\,350\,128$ (levezetett érték, nem mért; a konvolúciós és lineáris szorzás-akkumulációkat tartalmazza, a BatchNorm és a ReLU költségét nem). Compute budget: $B = 4\,000\,000$ MAC/kép. A csatornaszámok kerekítése: a legközelebbi egész, legalább 1 (a labor `int()` csonkolásától eltérő, itt rögzített konvenció). Változatok, mind $r = 1$ mellett, mert a

GlyphLab natív felbontása 24×24, ezért a resolution tengelyt nem mozgatjuk: depth: 3 blokk, $w = 1{,}0$, MAC 3 483 696; width: 2 blokk, $w = 1{,}3$ (21 csatorna), MAC 3 810 303; compound: 3 blokk, $w = 1{,}05$ (17 csatorna), MAC 3 877 683. A diszkrét csatornaszámok miatt a MAC-ok nem egyeznek pontosan: a depth változat a B 87%-át, a width és a compound 95–97%-át használja, ezt a kiértékelésnél rögzíteni kell. Seeds: öt seed (0–4), minden változat ugyanazokkal a seedekkel. Megfigyelési egység: `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik. Metrika: validálási cross-entropy a checkpoint kiválasztásához, záróteszt accuracy a clean és a korrupt teszten, jelenetszintű bootstrap konfidenciaintervallummal. Következtetési korlát: GlyphLab nem természetes képi benchmark, ezért az eredmény csak a szintetikus tartományra érvényes.

19.3.10 Tipikus tévedés és ellenpélda

Gyakori tévedés, hogy a compound scaling képlete minden hardverre optimális szabály. Ellenpélda: memória-sávszélesség-korlátos eszközön a resolution növelése r^2-es aktivációmemória-növekedést okoz, míg a depth növelése soros függőséget visz a futásidőbe; emiatt ugyanaz a MAC-budget CPU-n és GPU-n más-más optimumot adhat. Ezt itt mechanizmus szintjén indokoljuk, nem saját mérés alapján állítjuk: a konkrét optimumot az adott hardveren meg kell mérni. A heurisztika tehát keresési kiindulópont, nem hardverfüggetlen tétel.

19.3.11 Önellenőrzés és visszahívás

Ellenőrizze: a költségmodell feltevései között szerepel a stride 1, a méretmegtartó padding, a `groups=1`, valamint a csatornaszámok kerekítésének elhagyása; a bias a kimeneti elemek számával arányos egyszeri összeadás, ezért a $k^2 c_{in}$ szorzás-akkumulációhoz képest $1/(k^2 c_{in})$ nagyságrendű járulékot ad, ami nem nagyon kis csatornaszám mellett elhanyagolható; a compound képlet arányai empirikusak. A width multiplier 19.2. MobileNet és inverted residual fogalmát most hálózatszintű költségskálázásként alkalmaztuk; a BatchNorm paraméterei 18.3. Pre-activation, kis batch és alternatív normalizálás a csatornaszámokkal együtt w-vel nőnek, a loss-konvenció 7.3. Bináris, multilabel és regressziós cél viszont független a skálázástól.

19.3.12 Rövid összefoglaló

A depth, width és resolution külön mérhető skálázási tengely. A stem–body költségmodellből (amely stride 1-et, méretmegtartó paddinget, `groups=1`-et, kerekítés nélküli csatornaszámokat és azonos alakú ismételt blokkokat feltételez) levezethető a $d\,w^2 r^2$ közelítő MAC-szorzó és a belső súlyparaméterek $w^2 d$ skálázása; az első réteg külön kezelendő, és relatív költsége csak akkor hanyagolható el, ha a baseline-ban is kicsi. A compound scaling e három tengelyt egy ϕ együtthatóval hangolja, de a konkrét arányok empirikus keresésből származnak, nem elméleti optimumból. A labor két width értékű kis modell méretét és CPU-idejét veti össze; a kimenet a rögzített környezetű ellenőrző futás mért értéke (egy szálon; az idő más gépen eltér). A képlet hardverfüggősége miatt az eredmény helyi, nem általános optimalitási garancia.

Kulcsképletek. Változatlan mélység: $M(w, 1, r) = r^2(wM_{\text{stem}} + w^2 M_{\text{body}})$. Azonos alakú blokkok ismétlése: $M(w, d, r) = r^2(wM_{\text{stem}} + dw^2 M_{\text{body}})$. A széles körű közelítés: $M_{\text{approx}}(w, d, r) = dw^2 r^2 M_0$. A belső súlyparaméterek szorzója: $w^2 d$. A második és a harmadik képlet csak a felsorolt feltételek mellett érvényes.

19.3.13 Következő alkalmazás

A 19.4. ConvNeXt és a recept/architektúra elválasztása egység ConvNeXt kapcsán a modernizált CNN-recepteket vizsgálja [P11], ahol a skálázást az architektúra-változtatástól elkülönítve értelmezzük.

19.3.14 Források

- [P09] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. és Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. arXiv:1801.04381 (v4). https://arxiv.org/abs/1801.04381.
- [P10] Tan, M. és Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. arXiv:1905.11946. https://arxiv.org/abs/1905.11946.
- [P11] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T. és Xie, S. (2022). *A ConvNet for the 2020s*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022). https://arxiv.org/abs/2201.03545.
- [D10] PyTorch contributors. *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

19.4 ConvNeXt és a recept/architektúra elválasztása

19.4.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben a ConvNeXt-család [P11] alapgondolatát vizsgáljuk: egy korábbi, jól ismert CNN-architektúrát fokozatosan modernizálunk a vision transformer korszakában megszokott építőelemekkel és tanítási recepttel, miközben a modell továbbra is konvolúciós háló marad. A cél nem egy konkrét modellváltozat betanítása, hanem a **recept** és az **architektúra** fogalmának szétválasztása. Előismeret: a 19.3. EfficientNet és architektúraskálázás skálázási tengelyei, a 18.4. Reziduális blokk ablatív vizsgálata reziduális blokk és ablatív protokoll, valamint a 15.4. Független gradiensellenőrzési protokoll numerikus ellenőrzés gondolata. A 7.4. Numerikus stabilitás és jelentésmegőrző implementáció numerikus stabilitási eredményeire csak annyiban támaszkodunk, hogy a labor a `nn.LayerNorm` réteget használja, amely a varianciához hozzáadott ε-nal számol.

Induló kérdés: ha egy 2015-ös stílusú ResNet-50-szerű hálót változatlanul hagyunk, de a modern transformer-recepttel tanítjuk, mi változik? És ha csak egyetlen építőelemet cserélünk le, hogyan tudjuk eldönteni, hogy a javulás az új elemtől vagy a megváltozott recepttől származik?

19.4.2 Recept és architektúra: két külön döntési szint

Definíció (architektúra). Egy konvolúciós háló *architektúrájának* nevezzük mindazokat a döntéseket, amelyek a forward függvényt írják le: a rétegek sorrendje, a kernelméretek, stride-ok, paddingek, csatornaszámok, aktivációk, normalizálások, skip-kapcsolatok és downsampling-rétegek. Az architektúra a paraméterek terét és a függvényosztályt határozza meg.

Definíció (tanítási recept, training recipe). A *recept* mindaz, ami a paraméterek keresését vezérli, de nem változtatja meg a forward függvényt: az optimalizáló (például AdamW a 6.4. Momentum és adaptív optimizálók szerint), a learning rate és annak ütemezése, a weight decay, az epochok száma, a batch méret, az augmentáció, a címke-simítás (label smoothing), a véletlen

mélység (stochastic depth, azaz rétegek véletlen kihagyása edzés közben), a keverési (mixup) stratégia, valamint az inicializálás és a kiértékelési protokoll.

A két fogalom azért kritikus, mert egy architektúra teljesítményét soha nem a recepttől függetlenül mérjük. A [P11] 2. szakasza egy standard ResNet-50-et modernizál fokozatosan, és a lépések között a tanítási recept cseréje is szerepel. A lépések részleteit itt nem ismételjük; a pontos lépésenkénti eredményekhez a cikk kísérleti szakasza szükséges.

Heurisztika (recept-architektúra szétválasztás). Ha egy új blokk bevezetésekor a baseline-t régi, az új modellt pedig új recepttel tanítjuk, akkor a mért különbség nem tulajdonítható egyetlen tényezőnek. A tisztességes összehasonlítás egyeztetett receptet (*matched recipe*) igényel: mindkét modellt ugyanazzal a protokollal tanítjuk, és csak a vizsgált architekturális elemet változtatjuk. Ez ugyanaz az ablatív elv, amelyet a 18.4. Reziduális blokk ablatív vizsgálata egységben a skip-kapcsolatra bevezettünk.

19.4.3 A ConvNeXt-blokk formális leírása

A ConvNeXt-blokk a [P11] által vizsgált modernizálási lépésekből áll össze. A következő blokk a szakirodalomban ConvNeXt-blokk néven ismert sorrendet követi; a konkrét modellváltozat részletei a forrás szerinti konfigurációtól függhetnek.

Legyen a batchelt bemenet $X \in \mathbb{R}^{N\times C\times H\times W}$, ahol C a csatornaszám, H, W a térbeli méret. Legyen $K_{dw} \in \mathbb{R}^{C\times 1\times k\times k}$ a depthwise konvolúció súlya, azaz a PyTorch `nn.Conv2d(dim, dim, kernel_size=k, padding=p, groups=dim, bias=False)` réteg súlyalakja $(C_{\text{out}}, C_{\text{in}}/g, k_h, k_w)$ a [D10] szerint; itt $g = C$ és a kimeneti csatornák száma is C. A depthwise réteg kimenete

$$(\mathrm{DW}_{k,p}(X))_{n,c,i,j} = \sum_{a=0}^{k-1}\sum_{b=0}^{k-1} K_{dw}[c,0,a,b]\, X_{n,c,\,i+a-p,\,j+b-p},$$

ahol p a szimmetrikus zero padding, és a kényelem kedvéért stride $s = 1$; a kimeneti méret ekkor $H_{\text{out}} = H$, ha $p = (k-1)/2$.

Legyen LN_C a csatornadimenzióra vonatkozó LayerNorm: minden n, h, w pozícióban a C hosszúságú vektort normalizáljuk,

$$\mathrm{LN}_C(z)_c = \gamma_c \frac{z_c - \mu(z)}{\sqrt{\sigma^2(z) + \epsilon}} + \beta_c, \qquad \mu(z) = \frac{1}{C}\sum_{c=1}^{C} z_c, \quad \sigma^2(z) = \frac{1}{C}\sum_{c=1}^{C}(z_c - \mu(z))^2.$$

A μ, σ^2 itt a csatornadimenzió menti átlag és variancia, nem a mini-batch statisztika; ez a döntés eltér a BatchNorm 18.2. Batch normalization train és eval módban alfejezetben tárgyalt train/eval állapotkezelésétől. A PyTorch `nn.LayerNorm` alapértelmezett értéke $\epsilon = 10^{-5}$, a labor ezt használja. Legyen E az expanziós arány, $U \in \mathbb{R}^{EC\times C\times 1\times 1}$ az expanziós és $V \in \mathbb{R}^{C\times EC\times 1\times 1}$ a projekciós 1×1-es konvolúció súlya (bias nélkül), végül $\gamma \in \mathbb{R}^{C\times 1\times 1}$ a LayerScale csatornánkénti szorzója; a LayerScale a reziduális ág kimenetének tanulható, csatornánkénti skálázása. A GELU aktiváció zárt alakja $\mathrm{GELU}(x) = x\Phi(x)$, ahol Φ a standard normális eloszlás eloszlásfüggvénye; a PyTorch `nn.GELU` alapértelmezésben ezt a pontos alakot használja. A blokk:

$$B(X) = X + \gamma \odot V\mathrm{GELU}(U\,\mathrm{LN}_C(\mathrm{DW}_{k,p}(X))).$$

A $\odot$ itt csatornánkénti, térben broadcastolt szorzás. A kifejezés dimenzióellenőrzése: X alakja (N, C, H, W); a depthwise kimenete (N, C, H, W); LN_C ugyanekkora; U kimenete

(N, EC, H, W); a GELU ezt őrzi meg; V visszavetíti (N, C, H, W)-re; a γ-szorzás és a reziduális összeadás ugyanebben a térben marad. A skip tehát akkor identitás, ha a térbeli méret és a csatornaszám nem változik; downsampling esetén külön shortcut szükséges.

Figyeljünk a különbségre a 19.2. MobileNet és inverted residual MobileNetV2 inverted residual blokkjához képest [P09]: ott a depthwise konvolúció a kiterjesztett, széles csatornatérben futott. Itt a depthwise réteg a kiterjesztés előtti C csatornán kever térben, és az 1×1 rétegek végzik a csatornák keverését. A nagyméretű kernel (large kernel) döntés azt jelenti, hogy a depthwise kernel tipikusan 7×7-es is lehet, mert a költsége Ck^2 arányú, nem C^2k^2.

A [P11] referencia-konfigurációja a blokkra: $k = 7$ depthwise kernel, $E = 4$ expanziós arány, csatornadimenziós LayerNorm, GELU és LayerScale; a LayerScale szorzója kis kezdőértékről indul, hogy a reziduális ág kezdetben közel nulla hozzájárulást adjon. A labor $k = 5$, $E = 4$ beállítása ettől eltér, és ezt a szöveg jelzi.

19.4.4 Költség- és paraméterszámítás kézi példán

A blokk költségmodelljét explicit feltevésekkel vezetjük le. Feltesszük: stride 1, szimmetrikus padding, így $H_{\text{out}} = H$, $W_{\text{out}} = W$; a konvolúciós MAC-ot számoljuk, a GELU és LayerNorm szorzásait itt nem. A depthwise réteg MAC-igénye $NHWCk^2$, mert minden kimeneti pixelre csatornánként k^2 szorzás-összeadás jut. Az első 1×1 réteg $NHWC(EC)$, a második $NHW(EC)C$ MAC. Összesen:

$$\text{MAC} = NHWC(k^2 + 2EC).$$

A paraméterszám bias nélkül: a depthwise súly Ck^2, a két 1×1 súly $C(EC) + (EC)C = 2EC^2$, a LayerNorm γ, β együtt $2C$, a LayerScale C; így

$$N_{\text{par}} = Ck^2 + 2EC^2 + 3C.$$

Példa: legyen $N = 1$, $C = 2$, $E = 4$, $H = W = 8$, $k = 5$, $p = 2$. A MAC-összeg:

$$1 \cdot 8 \cdot 8 \cdot 2\,(25 + 2 \cdot 4 \cdot 2) = 128 \cdot (25 + 16) = 128 \cdot 41 = 5248.$$

A paraméterszám: $2 \cdot 25 + 2 \cdot 4 \cdot 4 + 3 \cdot 2 = 50 + 32 + 6 = 88$. Az alakellenőrzés: bemenet $(1, 2, 8, 8)$, kimenet ugyanekkora, mert $p = 2$ és stride 1.

A példa mutatja a depthwise döntés hatását: ha a $k = 5$ kernelt közönséges, $g = 1$ csoportú konvolúcióként használnánk, a súly alakja $(2, 2, 5, 5)$ lenne, 100 paraméterrel, és a MAC $NHWC^2k^2 = 1 \cdot 8 \cdot 8 \cdot 2^2 \cdot 25 = 6400$ lenne csak a térbeli rétegben. A csoportosítás tehát a paraméter- és műveletszámot is csökkenti, de a kapacitást is korlátozza; ez heurisztikus kompromisszum, nem automatikus pontossági garancia.

19.4.5 A [P11] kísérleti összevetés feltételeinek jegyzéke

A levezetési kötelezettség itt nem egy tétel, hanem egy kísérleti állítás feltételeinek pontos rögzítése. A [P11] módszertani részleteit a cikk 2. szakasza tárgyalja; az alábbi jegyzék az absztrakt szintjén rögzíthető állításokat gyűjti össze:

1. A kiinduló modell egy standard ResNet-50-szerű konvolúciós háló.
2. A modernizálás fokozatosan cseréli az építőelemeket a vision transformer tervezési nyelvéhez közelítve.
3. Az összehasonlítási partner vision transformer architektúra, nem pedig csak korábbi CNN-baseline.

Amit egy pontos reprodukcióhoz rögzíteni kell, de a hivatkozott absztrakt nem tartalmaz:

- az adathalmaz és a train/val/test split pontos verziója;
- a bemeneti felbontás és preprocessing;
- az optimalizáló, LR-ütemezés, epochszám, batch méret, weight decay;
- az augmentációs és regularizációs halmaz (beleértve a stochastic depth és label smoothing esetleges használatát);
- a számítási keret: paraméterszám, FLOP/MAC vagy valós erőforrás;
- az ismétlések, seedek és a jelentett szórás definíciója;
- a metrika (top-1/top-5 accuracy) és a kiértékelési protokoll;
- a szoftver- és hardverkörnyezet.

A cikk absztraktjának állítását csak azzal a korláttal idézzük, hogy a kísérleti táblázatok részleteit nem ismertetjük. A fejezet ezért nem tesz a cikkben szereplő kísérleti táblázatokból származó, itt nem igazolt számszerű állítást.

19.4.6 Labor: mini blokk és ablatív terv

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `n076_mini_block.py` futtatható (`python n076_mini_block.py`); a blokk szintetikus bemeneten fut, külső adatfájl nem kell.

A labor célja egy kisméretű, ConvNeXt-szerű blokk implementálása a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab méretosztályában, benchmark-szám kitalálása nélkül. A GlyphLab 180 jelenetből áll, jelenetenként két nézettel (összesen 360 kép); a képek 24×24-es, egycsatornás float32 tenzorok, a címkék három irányított osztályt jelölnek (0 = függőleges, 1 = vízszintes, 2 = átlós). A `scene_id` a jelenetet azonosítja, ezért a két nézet nem tekinthető független mintának. A következő kód a labor implementációja; az alak- és paraméterellenőrzést a fenti kézi számítással vetjük össze. Futtatás a `lab/n076` munkakönyvtárból:

```
cd lab/n076
python n076_mini_block.py
```

```
# lab/n076/n076_mini_block.py
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device("cpu")

class MiniConvNeXtBlock(nn.Module):
    """Kisméretű ConvNeXt-szerű blokk, bias nélküli konvolúciókkal.
```

```
    Alak: (N,C,H,W) -> (N,C,H,W), stride=1, szimmetrikus padding.
    """
    def __init__(self, dim, expand, kernel):
        super().__init__()
        assert kernel % 2 == 1, "a szimmetrikus paddinghez páratlan kernel kell"
        pad = kernel // 2
        self.dw = nn.Conv2d(dim, dim, kernel_size=kernel,
                            padding=pad, groups=dim, bias=False)
        self.norm = nn.LayerNorm(dim)  # NHWC-ben a csatornadimenzióra
        self.pw1 = nn.Conv2d(dim, dim * expand, kernel_size=1, bias=False)
        self.act = nn.GELU()
        self.pw2 = nn.Conv2d(dim * expand, dim, kernel_size=1, bias=False)
        self.gamma = nn.Parameter(torch.ones(dim, 1, 1))

    def forward(self, x):
        h = self.dw(x)                      # (N,C,H,W)
        h = h.permute(0, 2, 3, 1)           # NHWC
        h = self.norm(h)                    # (N,H,W,C)
        h = h.permute(0, 3, 1, 2)           # NCHW
        h = self.pw1(h)
        h = self.act(h)
        h = self.pw2(h)
        h = h * self.gamma
        return x + h

dim, expand, kernel = 2, 4, 5
model = MiniConvNeXtBlock(dim, expand, kernel).to(device).to(torch.float32)
x = torch.randn(1, dim, 8, 8, device=device, dtype=torch.float32)
y = model(x)

weight = model.dw.weight
n_params = sum(p.numel() for p in model.parameters())
# Kézi képlet: dw C*k^2, pw1 és pw2 2*E*C^2, gamma és LayerNorm együtt 3C.
expected_params = dim * kernel**2 + 2 * expand * dim**2 + 3 * dim
assert tuple(weight.shape) == (dim, 1, kernel, kernel), weight.shape
assert tuple(y.shape) == tuple(x.shape), y.shape
assert n_params == expected_params, (n_params, expected_params)
print("dw weight shape:", tuple(weight.shape))
print("output shape:", tuple(y.shape))
print("params:", n_params, "expected:", expected_params)
print("norm type:", type(model.norm).__name__,
      "| gamma shape:", tuple(model.gamma.shape))
```

A kód három `assert`-tel ellenőriz: a depthwise súly alakja $(C, 1, k, k)$, a kimeneti alak egyezik a bemenetével, és az összesített paraméterszám a $Ck^2 + 2EC^2 + 3C$ képletből számolt értékkel egyezik (itt 88). A `nn.LayerNorm(dim)` a permutálás után az NHWC utolsó dimenziójára, vagyis a csatornákra normalizál. A seed csak a bemenet és az inicializált súlyok értékét rögzíti; a paraméterszám és az alak a seedtől független, ezért ezek ellenőrzése nem a seed megválasztásán múlik.

A rögzített környezetű ellenőrző futás mért kimenete (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020):

```
dw weight shape: (2, 1, 5, 5)
output shape: (1, 2, 8, 8)
params: 88 expected: 88
norm type: LayerNorm | gamma shape: (2, 1, 1)
```

A labor második része egy **részletes ablatív terv**, amelyet a jövőben futtatni lehet. Kérdés: a depthwise kernel mérete (3×3, 5×5, 7×7) hogyan hat a GlyphLab-pontosságra, ha minden más rögzített? Modell: patchify stem (a bemenetet nem átfedő 4×4-es foltokra bontó, stride 4-es

konvolúció) 24×24→6×6, $C = 8$, $E = 4$, két mini blokk, globális átlagolás (global average pooling, GAP), lineáris fej 3 osztályra. Változó tényező: csak a depthwise kernel mérete, a padding mindig a méretmegtartáshoz illesztett. Rögzített recept: AdamW, learning rate 10^{-3}, weight decay 0,05, 50 epoch, batch 32, augmentáció nincs (a tükrözés és a forgatás megváltoztatná az irányított osztályok címkéjét), a seedkészlet $\{20261020, 20261021, 20261022\}$ (mindhárom kernelméretre ugyanaz), a GlyphLab jelenetszintű train/val splitje (120/30 jelenet, azaz 240/60 kép). A teszt 30 jelenet, 60 kép, és csak a protokoll végén értékeljük. A legjobb checkpoint kiválasztása kizárólag val loss alapján történik: a loss folytonos és kevésbé zajos, mint a val accuracy, ezért alkalmasabb a modellválasztásra, míg a végső összehasonlítás metrikája a val accuracy. A három kernelméretet seedenként párosítva hasonlítjuk össze; a döntési szabály a val accuracy seed-átlaga és a páros különbségek 95%-os, jelenetszintű bootstrap konfidenciaintervalluma (23.3. Páros összehasonlítás és csoportszintű intervallum); a bootstrap jelenetszintű, mert a két nézet ugyanahhoz a jelenethez tartozik, így a képszintű resampling sértené a függetlenséget. Három seeddel dolgozunk, mert egyetlen seed nem elég: a seed a súlyinicializálást és a batch-sorrendet is meghatározza, és a kis adaton a futások közti szórás összemérhető a kernelméret hatásával; három seed azonban még mindig kis minta, ezért az eredmény csak iránymutató. Erőforrás: CPU. A terv nem a [P11] reprodukciója; a GlyphLab 360 képe és 24×24 felbontása más tartomány, ezért a nagy kernel előnye nem vihető át automatikusan.

19.4.7 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el saját szavaival, mi a különbség a tanítási recept és az architektúra között, és adjon egy konkrét példát arra, hogy két azonos architektúra eltérő recepttel más tanulási görbét adhat.

F2 (levezetési, nehézség 2). Legyen $N = 1$, $C = 16$, $H = W = 32$, $E = 4$, $k = 7$, stride 1, szimmetrikus padding. Számítsa ki a blokk MAC-összegét és paraméterszámát bias nélkül. Ellenőrizze a dimenziókat a depthwise réteg súlyán.

F3 (implementációs, nehézség 2). Futtassa a fenti labor kódot, majd módosítsa úgy, hogy `dim=8`, `expand=4`, `kernel=7`, bemenet $(2, 8, 16, 16)$. Ellenőrizze, hogy a kimeneti alak $(2, 8, 16, 16)$, és adja meg a paraméterszámot az itteni képlettel is.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészletben a depthwise réteg definíciója hibás:

```
self.dw = nn.Conv2d(dim, dim, kernel_size=5, padding=2, bias=False)
```

Mi a hiba? Milyen következménnyel jár a paraméterszámra és a blokk szándékolt számítási szerkezetére nézve? Adja meg a javítást, és írja le a visszaellenőrzéshez használt súlyalakot.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen matched-recipe ablatív kísérletet a GlyphLab adaton a normalizálás típusának (BatchNorm2d a csatornadimenzióra versus LayerNorm a csatornadimenzióra) összehasonlítására. Adja meg a protokollt: modell, split, baseline, változó tényező, rögzített tényezők, metrika, erőforrás és az érvényességi korlát. Nem kell futtatást végeznie; a tervnek ellenőrizhetőnek kell lennie. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

19.4.8 Megoldások és indoklás

F1. Az architektúra a forward függvényt rögzíti: rétegek, kernelméretek, csatornaszámok, skip-kapcsolatok. A recept a paraméterkeresés protokollja: optimalizáló, ütemezés, augmentáció, regularizálás, epochszám. Egytényezős példa: ugyanaz a háromrétegű MLP ugyanazzal az AdamW

optimalizálóval, learning rate-ütemezéssel és epochszámmal fut, csak a weight decay értéke más (0 az egyik, 10^{-2} a másik futásban); az architektúra és a recept minden más eleme azonos, a tanulási görbe mégis eltér. A szétválasztás szükségességét a következő érvelés mutatja: ha a baseline-t régi, az új modellt új recepttel vetjük össze, a hatás nem egyértelműen tulajdonítható az architektúrának.

F2. Adatok: $N = 1$, $C = 16$, $H = W = 32$, $E = 4$, $k = 7$. A szimmetrikus padding $p = 3$, így a kimenet 32×32. MAC:

$$1 \cdot 32 \cdot 32 \cdot 16\,(49 + 2 \cdot 4 \cdot 16) = 16384 \cdot (49 + 128) = 16384 \cdot 177 = 2{,}899{,}968.$$

A paraméterszám: $16 \cdot 49 + 2 \cdot 4 \cdot 16^2 + 3 \cdot 16 = 784 + 2048 + 48 = 2880$. A depthwise súly alakja $(16, 1, 7, 7)$, mert $g = 16$; a két $1{\times}1$ súly $(64, 16, 1, 1)$ és $(16, 64, 1, 1)$. A GELU és a LayerNorm nem változtatja az alakot. A broadcast szorzás a γ $(16, 1, 1)$ tenzorral érvényes, mert a γ első tengelye a csatorna, és a hiányzó batch-tengelyt a broadcast egészíti ki.

F3. A módosított labor kódban a `dim, expand, kernel = 8, 4, 7` értékeket kell beírni (a többi sor változatlan); a padding `7//2=3`, a bemenet $(2, 8, 16, 16)$. A depthwise kimenet $H = 16 + 2 \cdot 3 - 7 + 1 = 16$, tehát a kimeneti alak $(2, 8, 16, 16)$. A súlyalak $(8, 1, 7, 7)$. A paraméterszámot a kód a képletből számolja, ezért a `dim` és a `kernel` átírásával automatikusan frissül: $8 \cdot 49 + 2 \cdot 4 \cdot 64 + 3 \cdot 8 = 392 + 512 + 24 = 928$. Numerikus tolerancia itt nem szükséges, mert egész számok; a shape- és paraméterszám-ellenőrzést a kód `assert`-jei végzik.

F4. A hiba: a `groups` argumentum hiánya, ezért a réteg közönséges konvolúció lesz, `groups=1`. A súly alakja ekkor $(5, 5, 5, 5)$ lenne `dim=5` esetén, általában (C, C, k, k), nem pedig a szándékolt $(C, 1, k, k)$. Következmény: a paraméterszám C^2k^2 lesz Ck^2 helyett, a MAC $NHWC^2k^2$ lesz, és a szándékolt depthwise réteg helyett közönséges konvolúció fut, így a blokk nem a tervezett depthwise + 1×1 felbontást valósítja meg. A javítás: `groups=dim`. Visszaellenőrzés: `model.dw.weight.shape` értéke `(dim, 1, 5, 5)`, és a `n_params` visszatér a kézi képlettel számolt értékre.

F5. Egy lehetséges terv. Modell: patchify stem (nem átfedő 4×4-es foltok, stride 4), 8 csatorna, két blokk ($E = 4$), globális átlagolás (GAP), lineáris fej 3 osztályra. A blokk norm rétege a változó tényező: vagy `nn.BatchNorm2d(8)` a depthwise után közvetlenül a NCHW tenzoron, vagy `nn.LayerNorm(8)` NHWC permutálással. Baseline: BatchNorm-változat, mert a klasszikus CNN-recepthez ez közelebb áll. Rögzített: AdamW, LR 10^{-3}, wd 0,05, 50 epoch, batch 32, három seed (20261020–20261022), azonos jelenetszintű train/val split (120/30 jelenet, 240/60 kép); augmentáció nincs, mert a tükrözés és a forgatás az irányított osztályok címkéjét változtatná meg. A checkpointot val loss alapján választjuk (folytonos, kevésbé zajos, mint a val accuracy), a végső metrika val accuracy; a tesztet (30 jelenet, 60 kép) egyszer, a protokoll végén értékeljük. Megfigyelési egység: `scene_id`, a bootstrapot is jelenetszinten végezzük 5.4. Függő képek, seedek és resampling szerint, mert a két nézet ugyanahhoz a jelenethez tartozik. Három seed szükséges, mert egyetlen seed a súlyinicializálás és a batch-sorrend egyetlen realizációját adja, és a kis adaton a futások közti szórás összemérhető a normálástípus hatásával. Erőforrás: CPU. Érvényességi korlát: 360 kép, 24×24 felbontás; az eredmény nem általánosítható nagyobb adatra, és a BN train/eval állapotkülönbségét a kiértékelésnél be kell tartani 18.2. Batch normalization train és eval módban. A negatív eredmény is elfogadható.

19.4.9 Tipikus tévedés, önellenőrzés és visszahívás

A leggyakoribb tévedés: a „korszerű” vagy „modern” címkét örök teljesítményrangsorként kezelni. A ConvNeXt-blokk előnye a [P11] vizsgálati feltételei mellett értelmezendő; e feltételek

nélkül az állítás nem vihető át. Ellenpélda heurisztikaként: a 360 képes GlyphLab adaton egy nagy, 7×7 depthwise kernelt használó modern blokk több paraméterrel és nagyobb recepciós mezővel túl is illeszkedhet, miközben egy egyszerű, 3×3-as CNN jobb tesztpontosságot érhet el. Ez nem mért eredmény, hanem címkézett gondolatkísérlet; éppen azt mutatja, hogy az adat mérete és a benchmark környezete része az állításnak. Önellenőrzés: a fenti példában a depthwise és a közönséges konvolúció súlyalakját, a szimmetrikus padding képletét és a reziduális összeadás broadcastját külön-külön ellenőrizni kell; a labor kimenete a súlyalakot és a paraméterszámot is kiírja. Visszahívás: a 18.4. Reziduális blokk ablatív vizsgálata matched-recipe elve adja az ablatív kontroll alapját, a 15.4. Független gradiensellenőrzési protokoll numerikus ellenőrzés arra emlékeztet, hogy a kód egyezése nem bizonyítja a modell optimalitását, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil normálás pedig a LayerNorm implementációjánál releváns.

19.4.10 Rövid összefoglaló

A ConvNeXt-példa azt mutatja, hogy a CNN-architektúra modernizálása és a tanítási recept cseréje két külön döntési szint, amelyet matched-recept ablációval lehet szétválasztani. A formális blokk depthwise nagy kernelt, csatorna-LayerNormot, GELU-t, 1×1 expand/project rétegeket és reziduális skipet kombinál; költsége $NHWC(k^2 + 2EC)$ MAC és $Ck^2 + 2EC^2 + 3C$ paraméter. A [P11] kísérleti feltételeit csak absztrakt szinten idéztük; a részletes protokollt itt nem ismertetjük. A labor egy 88 paraméteres mini blokk alak- és paraméterellenőrzését, valamint kernelméret-ablatív tervet ad benchmark-szám nélkül. A „modern” címke nem örök rangsor, hanem adott adat- és receptfeltételekhez kötött állítás.

19.4.11 Következő alkalmazás és források

A következő 20.1. Feature extractor és új fej egység a transzfertanulást kezdi: ott azt vizsgáljuk, hogy egy korszerű CNN-blokkból épített modell súlyait mikor és hogyan lehet más adatra átvinni. Az itteni recept-architektúra szétválasztás közvetlenül szükséges lesz, mert az előtanított modell és az új fej tanítása különböző protokollt igényel.

19.4.12 Felhasznált források

- [P09] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. és Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks.* arXiv:1801.04381 (v4). https://arxiv.org/abs/1801.04381.
- [P11] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T. és Xie, S. (2022). *A ConvNet for the 2020s.* IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022). https://arxiv.org/abs/2201.03545.
- [D10] PyTorch contributors. *Conv2d.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

20 Transzfertanulás és előtanított modellek helyes használata

20.1 Feature extractor és új fej

20.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben azt az üzemmódot vizsgáljuk, amelyben egy már betanított háló nagy részét változatlanul hagyjuk, és csak egy kicsi, feladatspecifikus utolsó réteget tanítunk. Előfeltétel a 12.4. III. ellenőrző pont: önállóan tanuló kis háló tanítási ciklusa, a 17.4. Architektúra összehasonlítása kontrollált keretben kontrollált összehasonlítás szempontjai, a 18.4. Reziduális blokk ablatív vizsgálata ablatív gondolkodásmód és a 19.4. ConvNeXt és a recept/architektúra elválasztása recept/architektúra szétválasztás. Induló becslés: ha egy 100 000 paraméteres modellből 98 500 tartozik a backbone-hoz, és azt befagyasztjuk, hány paramétert frissít egy optimizer-lépés? A válasz 1 500, de a bufferek kérdése ennél finomabb: a fagyasztás nem állít meg minden állapotváltozást.

20.1.2 Backbone, fej és pretrained súly: definíciók

Definíció (backbone, feature extractor). Legyen a modell

$$F(x) = g_H(f_B(x; \Theta_B, \mathcal{B}_B);\, \Theta_H, \mathcal{B}_H),$$

ahol $f_B : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^d$ a **backbone** vagy **feature extractor**, $g_H : \mathbb{R}^d \to \mathbb{R}^K$ pedig a **head** (fej). Θ_B és Θ_H a két rész súlyparaméterei, $\mathcal{B}_B$ és $\mathcal{B}_H$ a hozzájuk tartozó, nem súly jellegű állapotok (**bufferek**, például BatchNorm futó statisztikák). Batchelt bemenetre $X \in \mathbb{R}^{N\times C\times H\times W}$, a köztes reprezentáció $r = f_B(X) \in \mathbb{R}^{N\times d}$, a logit kimenet $Z = g_H(r) \in \mathbb{R}^{N\times K}$. A reprezentációt itt d-dimenziós vektorként definiáltuk, de ez nem követeli meg, hogy a backbone globális átlagolással záruljon: a MiniCNN `backbone` modulja $(N, 32, 6, 6)$ alakú térbeli tenzort ad, a `forward` ezt $(N, 1152)$-re lapítja, és a $d = 1152$ dimenziós vektor a továbbiakban a reprezentáció.

Definíció (pretrained weights). Egy Θ súlyvektor akkor **előtanított**, ha egy dokumentált korábbi tanítási futtatás végállapotából származik, és most új inicializálásként töltjük be. A minősítés a tanítási előzményhez és a betöltési eljáráshoz kötött, nem a fájlformátumhoz: a `weights=None` (korábban `pretrained=False`) jellel létrehozott háló architektúrája lehet azonos, de súlya véletlen, így nem előtanított. A torchvision 0.13 óta a `pretrained=` argumentum elavult; a támogatott forma a `weights=` enum (például `ResNet18_Weights.DEFAULT` vagy `None`).

Definíció (frozen parameter). Egy θ paraméter **befagyasztott**, ha a keretrendszerben `p.requires_grad == False`. Ilyenkor a backward a paraméterhez nem számol új gradienst. Két dolgot ez önmagában **nem** biztosít: egy korábban már kiszámolt `p.grad` nem törlődik, és a paraméter továbbra is szerepelhet az optimizer paraméterlistájában. A szokásos PyTorch-optimalizálók a `grad is None` paramétereket hagyják ki, nem a `requires_grad` zászlót nézik,

ezért egy megmaradt régi gradienssel (vagy momentum- és weight decay-állapottal) a „fagyott” súly mégis változhat. A befagyasztás ajánlott receptje ezért:

```
for p in model.backbone.parameters():
    p.requires_grad_(False)
    p.grad = None                                  # régi gradiens törlése
optimizer = torch.optim.SGD(
    [p for p in model.parameters() if p.requires_grad], lr=1e-2)  # csak a tanítandók
```

A listából kizárás ajánlott, mert egyértelművé teszi, hogy az optimizer mely paramétereket frissítheti; a `requires_grad_(False)` önmagában nem tiltja meg, hogy a paramétert felvegyük az optimizer listájára. A befagyasztás a *frissítés* tiltása; a forward számításban a fagyott súly továbbra is aktív.

Definíció (optimizer groups). Az optimalizáló **paramétercsoportjai** a paraméterek diszjunkt halmazaihoz külön beállítást rendelnek. A fejcserés tanítás tipikus felosztása: az egyik csoport a fej paramétereit tartalmazza a teljes tanulási rátával; a backbone-t nem adjuk át az optimizernek, ezért nincs második csoport. Finomhangolásnál a backbone külön, kisebb rátájú csoportba kerülhet, és a két csoport tanulási rátája, weight decay-je vagy momentuma eltérhet.

20.1.3 A befagyasztott backbone paraméter- és bufferleltára

A befagyasztás nem azonos az `eval()` móddal. Ezt a következő implementációs tényben rögzítjük.

Implementációs tény (befagyasztott backbone állapotváltozásai). Tekintsük a fenti F modellt. Tegyük fel, hogy minden $\theta \in \Theta_B$-re `requires_grad=False`, minden $\phi \in \Theta_H$-ra `requires_grad=True`, és az optimizer csak Θ_H elemeit kapja meg. Tegyük fel továbbá, hogy a modulokat a hívó vagy `train()`, vagy `eval()` módba állította. Ekkor egyetlen minibatch-tanítási lépésben:

1. **A backbone súlyparaméterei nem változnak**: Θ_B elemeire nem számolódik gradiens, és az optimizer nem frissíti őket.
2. **A fej súlyparaméterei frissülhetnek**: Θ_H a szokásos gradienslépés szabálya szerint változik.
3. **A backbone BatchNorm bufferei `train()` módban változnak**. Minden BatchNorm réteg a momentum-szabályt hajtja végre a forward alatt:

$$\hat{\mu} \leftarrow (1-m)\,\hat{\mu} + m\,\mu_{\text{batch}}, \qquad \hat{\sigma}^2 \leftarrow (1-m)\,\hat{\sigma}^2 + m\,\sigma^2_{\text{batch}},$$

   ahol m a modul momentum paramétere, μ_{batch} és σ^2_{batch} az aktuális batch statisztikái. A PyTorch-konvenció rögzített: a normalizálás a torzított, n-nel osztott batch-varianciát használja, a futó $\hat{\sigma}^2$ puffert viszont a torzítatlan, $n-1$-gyel osztott becsléssel frissíti; ez a modulon nem konfigurálható külön. A `num_batches_tracked` buffer is növekszik. Ez **akkor is megtörténik, ha a backbone súlyai fagyottak**, mert a bufferfrissítés a forward része, nem autograd-művelet.
4. **A backbone BatchNorm bufferei `eval()` módban nem változnak**: a réteg a tárolt $\hat{\mu}, \hat{\sigma}^2$ értékekkel normalizál.
5. **A fej bufferei** a fej saját `training` attribútuma szerint viselkednek, függetlenül a backbone fagyasztásától.

Indoklás. Az (1)–(2) állítás az autograd gráfépítés és az optimizer definíciójából következik: a `requires_grad=False` tenzor az autograd gráf levele (olyan csomópont, amelyből nem vezet visszafelé gradiensigénylő út), ezért rá a backward nem számol gradienst; az optimizer pedig

kizárólag a paraméterlistájában szereplő, `requires_grad=True` tenzorokat módosítja. A (3)–(4) állítás a BatchNorm implementált viselkedése: a forwardban a `self.training` érték dönti el, hogy a batch statisztikát vagy a futó statisztikát használja-e, és a futó statisztika frissítése közvetlen, grafikon kívüli értékadás. A (5) pont ugyanezen szabály alkalmazása a fejre. Ez implementációs tény a PyTorch 2.x viselkedéséről, nem analitikus tétel; a momentum-szabály a könyvtár definíciója. A BatchNorm train/eval megkülönböztetését a 18.2. Batch normalization train és eval módban vezette be.

A megállapítás gyakorlati konzekvenciája: ha a befagyasztott backbone-ot `train()` módban hagyjuk, a súlyok állandóak, de a normalizálási állapot tovább mozog. Ha a cél a feature extractor teljes kimerevítése, a backbone moduljait `eval()` módba kell tenni, vagy a BatchNorm frissítését külön le kell tiltani.

20.1.4 Kézi kis példa

Legyen a backbone egyetlen affin réteg $W_B \in \mathbb{R}^{2\times 4}$, a fej $W_H \in \mathbb{R}^{3\times 2}$, bias nélkül. Legyen

$$W_B = \begin{pmatrix} 1 & 0 & 2 & -1 \\ 0 & 1 & -1 & 1 \end{pmatrix}, \qquad W_H = \begin{pmatrix} 1 & -1 \\ 2 & 0 \\ 0 & 1 \end{pmatrix}, \qquad x = (2, 1, -1, 1).$$

A köztes reprezentáció

$$h = W_B x = \begin{pmatrix} 1\cdot 2 + 0 - 2 - 1 \\ 0 + 1 + 1 + 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 3 \end{pmatrix}.$$

A logitok $z = W_H h = (-4, -2, 3)$. Legyen a cél $y = 0$; a softmax valószínűségei

$$p = \mathrm{softmax}(z) \approx (0{,}00091,\ 0{,}00669,\ 0{,}99240).$$

A cross-entropy gradiens a logitson $p - e_1 \approx (-0{,}99909,\ 0{,}00669,\ 0{,}99240)$. A fej súlygradiense

$$\frac{\partial L}{\partial W_H} = (p - e_1)\, h^\top \approx \begin{pmatrix} 0{,}99909 & -2{,}99727 \\ -0{,}00669 & 0{,}02007 \\ -0{,}99240 & 2{,}97720 \end{pmatrix},$$

a fej biasgradiense ugyanaz a vektor. A köztes reprezentáció gradiense $g = W_H^\top (p - e_1) \approx (1\cdot(-0{,}99909) + 2\cdot 0{,}00669,\ -1\cdot(-0{,}99909) + 1\cdot 0{,}99240) = (-0{,}98572,\ 1{,}99150)$. Ha a backbone **nincs** befagyasztva, W_B gradiense $g\,x^\top$, egy 2×4 mátrix. Ha a backbone **fagyott**, három esetet kell elválasztani. (a) *Paramétergradiens tiltása:* W_B nem kap gradienst, W_B.grad `None` marad; a fej viszont frissül. (b) *Bemeneti gradiens:* ha x `requires_grad=True`, a gradiens a fagyott lineáris backbone-on át is eljut a bemenetig, $\nabla_x L = W_B^\top g$; a fagyasztás tehát nem vágja el a gráfot. (c) *Gráfvágás* (a számítási gráf elvágása, azaz a backward megállása a reprezentációnál): ha sem x, sem a backbone súlya nem igényel gradienst (vagy a backbone `torch.no_grad()` alatt, illetve `detach()` után fut), a h már a gráf levele, a backward a fejnél megáll, és g nem is számolódik ki automatikusan.

20.1.5 Kódos ellenőrzés a kézi példára

```
# lab/n077/manual_check.py
import torch

dtype = torch.float64

x = torch.tensor([2.0, 1.0, -1.0, 1.0], dtype=dtype)
WB = torch.tensor([[1.0, 0.0, 2.0, -1.0],
                   [0.0, 1.0, -1.0, 1.0]], dtype=dtype)
WH = torch.tensor([[1.0, -1.0],
                   [2.0, 0.0],
                   [0.0, 1.0]], dtype=dtype, requires_grad=True)
y = torch.tensor(0)

def forward(wb, wh, freeze_backbone):
    wb = wb.detach()
    wb.requires_grad_(not freeze_backbone)
    h = wb @ x
    z = wh @ h
    loss = torch.nn.functional.cross_entropy(z.unsqueeze(0), y.unsqueeze(0))
    loss.backward()
    return h.detach(), loss.detach(), wb.grad, wh.grad

# Nem fagyasztott eset: mindkét gradiens létezik
h0, loss0, gWB0, gWH0 = forward(WB, WH.clone().detach().requires_grad_(True), False)
# Fagyasztott eset: csak a fej gradiense létezik
h1, loss1, gWB1, gWH1 = forward(WB, WH.clone().detach().requires_grad_(True), True)

assert torch.allclose(h0, torch.tensor([-1.0, 3.0], dtype=dtype), atol=1e-12)
assert torch.allclose(loss0, loss1, atol=1e-12)
p = torch.softmax(WH.detach() @ h0, dim=0)
g_h = WH.detach().T @ (p - torch.nn.functional.one_hot(y, 3).to(dtype))
expected_gWB = torch.outer(g_h, x)
assert gWB0 is not None
torch.testing.assert_close(gWB0, expected_gWB, rtol=1e-10, atol=1e-12)
# A pontos analitikus és autograd-gradiens alakja:
assert gWB0.shape == (2, 4) and gWB1 is None
assert gWH1 is not None and torch.allclose(gWH1, gWH0, atol=1e-12)
print("manual_check ok: frozen backbone -> WB.grad is None, WH.grad kept")

# (b) Fagyott backbone, de a bemenet gradienst igényel: a gradiens eljut x-ig.
xb = x.clone().requires_grad_(True)
zb = WH.detach() @ (WB @ xb)
torch.nn.functional.cross_entropy(zb.unsqueeze(0), y.unsqueeze(0)).backward()
torch.testing.assert_close(xb.grad, WB.T @ g_h, rtol=1e-10, atol=1e-12)
print("g_h =", [round(v, 5) for v in g_h.tolist()], " x.grad =", [round(v, 5) for v in
 ↪  xb.grad.tolist()])

# Ellenpélda: requires_grad_(False) után a régi .grad megmarad, és step() frissít.
w = torch.nn.Parameter(torch.tensor([1.0], dtype=dtype))
(w * 3.0).sum().backward()                      # w.grad = 3
w.requires_grad_(False)                         # "fagyasztás", de a grad nem törlődik
opt = torch.optim.SGD([w], lr=0.1)
opt.step()
print("fagyott w a step után:", w.item())       # 0.7: mégis változott
w.grad = None
opt.step()
assert abs(w.item() - 0.7) < 1e-12              # grad=None mellett már nem változik
```

A kód futtatható a `lab/n077` munkakönyvtárból: `python manual_check.py`. A futtatás kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()=10`, `seed=0`):

```
manual_check ok: frozen backbone -> WB.grad is None, WH.grad kept
g_h = [-0.98572, 1.9915]  x.grad = [-0.98572, 1.9915, -3.96295, 2.97722]
fagyott w a step után: 0.7
```

A három sor a kézi levezetéssel egyező `g_h` és `x.grad` értéket, valamint a fagyasztás utáni `step()` viselkedését rögzíti.

20.1.6 Labor: fejcsere a helyi tanított CNN-en, külső súly nélkül

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; a `head_swap_lab.py` a `lab/n077/` munkakönyvtárból futtatható (`python head_swap_lab.py`), a két fázis egy futásban dolgozik.

A labor két fázisból áll. Először egy kis GlyphLab-CNN-t tanítunk 3 osztályra (az adat szerződését a függelék A. A GlyphLab adatkészlet szakasza rögzíti), és a teljes `state_dict`-et elmentjük [D09]. Az adat a `lab/data/` könyvtárban van: `images.npy` float32 $(N, 1, 24, 24)$, $[0, 1]$ intenzitás; `labels.npy` int64 $(N,)$; `metadata.csv` pedig `index,scene_id,view_id,label,split,xmin,ymin,xmax,ymax` oszlopokkal. A `scene_id` a jelenetet azonosítja; jelenetenként két nézet tartozik ugyanahhoz a jelenethez, ezért a split és minden csoportosított kiértékelés jelenetszintű. A labor csak az `index` és a `split` oszlopot használja, a hiányzó adatfájlra pedig érthető `FileNotFoundError`-t ad. A `MiniCNN` konstruktora nem vesz át `weights` vagy `pretrained` argumentumot: az itt betöltött állapot a saját korábbi tanítás `state_dict`-je, nem külső előtanított súly. Másodszor a mentett modellt betöltjük, a fejet újra inicializáljuk, a backbone-ot befagyasztjuk, és ellenőrizzük, hogy a fenti állítás minden pontja teljesül. Ez **nem valódi transzfer**: az adat és a feladat ugyanaz marad, csak a frissítési mechanizmust gyakoroljuk. Valódi transzferhez dokumentált külső súlyazonosító és az adott súly preprocessing-szerződése szükséges; a [D13] katalógus konkrét elemeit a használat előtt ellenőrizni kell.

```python
# lab/n077/head_swap_lab.py
import random
from pathlib import Path
import numpy as np
import torch
import torch.nn as nn

SEED = 20261020
LR = 1e-3
EPOCHS = 3
BASE = Path(__file__).resolve().parent      # lab/n077
DATA = BASE.parent / "data"                 # a közös GlyphLab adat: lab/data
CKPT = BASE / "mini_cnn_3class.pt"          # a labor saját kimenete

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

class Block(nn.Module):
    def __init__(self, cin, cout):
        super().__init__()
        self.conv = nn.Conv2d(cin, cout, 3, padding=1)
        self.bn = nn.BatchNorm2d(cout)
        self.act = nn.ReLU()
        self.pool = nn.MaxPool2d(2)
    def forward(self, x):
```

```
        return self.pool(self.act(self.bn(self.conv(x))))

class MiniCNN(nn.Module):
    def __init__(self, num_classes=3):
        super().__init__()
        self.backbone = nn.Sequential(Block(1, 16), Block(16, 32))
        self.head = nn.Linear(32 * 6 * 6, num_classes)
    def forward(self, x):
        h = self.backbone(x).flatten(1)
        return self.head(h)

def load_glyphlab():
    for name in ("images.npy", "labels.npy", "metadata.csv"):
        if not (DATA / name).exists():
            raise FileNotFoundError(
                f"hiányzó GlyphLab adatfájl: {DATA / name}; "
                "a függelék A. szakasza szerinti data/ könyvtár kell")
    X = np.load(DATA / "images.npy").astype(np.float32)
    y = np.load(DATA / "labels.npy").astype(np.int64)
    meta = np.genfromtxt(DATA / "metadata.csv", delimiter=",", dtype=None,
                         names=True, encoding="utf-8")
    if X.shape != (len(y), 1, 24, 24):
        raise ValueError(f"várt képalak ({len(y)},1,24,24), kapott: {X.shape}")
    if not (len(meta) == len(X) == len(y)):
        raise ValueError("a metadata.csv és a tömbök hossza eltér")
    idx = meta["index"].astype(np.int64)
    if not np.array_equal(np.sort(idx), np.arange(len(X))):
        raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
    split = np.asarray(meta["split"]).astype(str)
    train_idx = idx[split == "train"]
    val_idx = idx[split == "val"]
    return X, y, train_idx, val_idx

def train_mini_cnn():
    X, y, tr, _ = load_glyphlab()
    model = MiniCNN()
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    lossf = nn.CrossEntropyLoss()
    Xtr = torch.from_numpy(X[tr]); ytr = torch.from_numpy(y[tr])
    for ep in range(EPOCHS):
        model.train()
        for i in range(0, len(tr), 32):
            xb, yb = Xtr[i:i+32], ytr[i:i+32]
            opt.zero_grad()
            loss = lossf(model(xb), yb)
            loss.backward()
            opt.step()
    torch.save(model.state_dict(), CKPT)
    return model

def head_swap_probe():
    sd = torch.load(CKPT, weights_only=True)
    model = MiniCNN()
    model.load_state_dict(sd)               # teljes modell, 3 osztályos fej is
    # Fejcsere: a fejet újra inicializáljuk, a backbone súlyát megtartjuk.
    model.head = nn.Linear(32 * 6 * 6, 3)
    model.head.weight.data.normal_(0, 0.02)
    model.head.bias.data.zero_()
    # Backbone befagyasztás, optimizer csak a fejet látja.
    model.backbone.requires_grad_(False)
    opt = torch.optim.Adam(model.head.parameters(), lr=LR)
    print("optimizer param_groups:", len(opt.param_groups))
    print("trainable params:", sum(p.numel() for p in model.parameters() if p.requires_grad))
```

```
    # Buffer-kontroll: BatchNorm train módban frissül, eval módban nem.
    bn = model.backbone[0].bn
    bn.eval(); before = bn.running_mean.clone()
    _ = model.backbone(torch.zeros(1, 1, 24, 24))
    assert torch.equal(bn.running_mean, before), "eval mode: running buffer must not change"
    bn.train(); before = bn.running_mean.clone()
    _ = model.backbone(torch.zeros(1, 1, 24, 24))
    assert not torch.equal(bn.running_mean, before), "train mode: running buffer must change"
    # Gradiens-kontroll egyetlen lépésen; a befagyasztott backbone eval módban marad.
    model.train()
    model.backbone.eval()
    xb = torch.zeros(2, 1, 24, 24); yb = torch.tensor([0, 1])
    opt.zero_grad(); loss = nn.functional.cross_entropy(model(xb), yb); loss.backward()
    assert model.backbone[0].conv.weight.grad is None
    assert model.head.weight.grad is not None
    print("head_swap_probe ok: backbone frozen, head gradient exists, buffer rule verified")

if __name__ == "__main__":
    if not CKPT.exists():
        train_mini_cnn()
    head_swap_probe()
```

Futtatási parancs a `lab/n077` könyvtárból: `python head_swap_lab.py`. A kód ezen a környezeten lefutott: Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()=10`, seed 20261020. A mért kimenet:

```
optimizer param_groups: 1
trainable params: 3459
head_swap_probe ok: backbone frozen, head gradient exists, buffer rule verified
```

A négy invariánst a kód `assert`-jei is ellenőrzik (eval módban a futó buffer változatlan, train módban változik; a backbone konvolúciós súlyának gradiense `None`, a fejé nem); a veszteségértékektől függetlenül ezek a labor kontrollpontjai.

20.1.7 Kiegészítés: valódi külső előtanított modell (torchvision), teljes lánc

A fenti labor szándékosan külső súly nélkül fut. A valódi előtanított modell (ResNet-18 [P06]) használata ugyanazt a láncot követi, négy kiegészítő lépéssel: (1) a `weights` enum kiválasztása, (2) a csomaghoz tartozó transzformáció használata, (3) az 1 csatornás GlyphLab-kép RGB-vé illesztése, (4) fejcsere és fagyasztás. **Ez a kód a súlyok letöltését igényli (hálózat), ezért itt nem futott; a hallgató a saját környezetében futtatja, és a súlycsomag licencét, valamint a `weights.meta` által elérhető metaadatokat a futásnaplóba rögzíti. A `weights.meta` nem garantáltan tartalmaz licencmezőt, ezért a licencet a modell oldalán is ellenőrizni kell.**

```
# lab/n077/pretrained_torchvision.py  (letöltést igényel; nem futtatott)
import torch, torch.nn as nn
from torchvision.models import resnet18, ResNet18_Weights

weights = ResNet18_Weights.DEFAULT                # (1) enum: verziózott súlycsomag
model = resnet18(weights=weights)
preprocess = weights.transforms()                 # (2) resize + crop + normalizálás a csomag
↪ szerint

for p in model.parameters():                      # (4) a törzs fagyasztása
    p.requires_grad_(False)
model.fc = nn.Linear(model.fc.in_features, 3)     # új fej: 512 -> 3 (alapból
↪ requires_grad=True)
```

```
def to_model_input(x_gray):                          # x_gray: (N,1,24,24) float32, [0,1]
    x_rgb = x_gray.repeat(1, 3, 1, 1)                # (3) 1 csatorna -> 3 csatorna
    return preprocess(x_rgb)                         # (N,3,224,224), ImageNet-normalizált

trainable = [p for p in model.parameters() if p.requires_grad]
assert len(trainable) == 2                           # csak fc.weight és fc.bias
opt = torch.optim.SGD(trainable, lr=1e-2)
model.eval()                                         # a BatchNorm-statisztikák fagyasztva
↪ maradnak
```

A `weights.transforms()` a ResNet18-súlyokhoz tartozó transzformáció [D13]: a rövidebb oldalt 256-ra méretezi, 224×224-es középső kivágást végez, majd az ImageNet mean/std-del normalizál. A 24×24-es szürke képet előbb három csatornán ismételjük, így a háló bemenete $(N, 3, 224, 224)$. Az új fej $512 \cdot 3 + 3 = 1539$ paramétert tartalmaz, és a `len(trainable) == 2` assert éppen a fej két tenzorát ellenőrzi.

A `model.eval()` itt a fagyasztott törzs BatchNorm-futóstatisztikái miatt kell; a fej tanítása közben a backbone marad `eval()` módban. A `Linear` rétegnek nincs train/eval-viselkedése, ezért a fej módja önmagában nem befolyásolja a kimenetet. Az előtanított modell ImageNet-eloszláson tanult, a GlyphLab szintetikus: az eredmény csak a helyi splitre vonatkozik, és a domain-eltolódás miatt nem általánosítható.

20.1.8 Feladatok

F1 (fogalmi, nehézség 1). Mi a különbség a `weights=ResNet18_Weights.DEFAULT` és a `weights=None` konstruktor-argumentum közt (a torchvision 0.13 óta a `pretrained=` argumentum elavult) egy olyan keretben, ahol az előbbi súlybetöltést indít el? Melyik esetben beszélhetünk transzfertanulásról, és miért?

F2 (levezetési, nehézség 2). A fenti állításban a backbone BatchNorm bufferei `train()` módban frissülnek, hiába fagyottak a súlyok. Bizonyítsd be ezt az állítást a momentum-szabályból: add meg $\hat{\mu}_{t+1}$ képletét, és mutasd meg, hogy $\hat{\mu}$ akkor is változhat, ha Θ_B egyetlen eleme sem változik.

F3 (implementációs, nehézség 2). Írj `freeze_by_prefix(model, prefix)` függvényt, amely minden olyan paramétert befagyaszt, amelynek neve `prefix`-szel kezdődik, és adja vissza a megmaradt tanítható paraméterek listáját. Teszteld egy `nn.Sequential`-be ágyazott `Linear(4,3)` + `Linear(3,2)` modellen; az elvárt kimenet: csak a második réteg paraméterei taníthatók.

F4 (hibakeresési, nehézség 2). Egy fejlesztő az alábbi pszeudokódot használja befagyasztott backbone-hoz, de az eval metrika a fej tanítása közben romlik, noha a training loss csökken. Mi a hibás feltevés, mi a következménye, és mi a két lehetséges javítás?

```
model.backbone.requires_grad_(False)
optimizer = Adam(model.head.parameters(), lr=1e-3)
for xb, yb in train_loader:      # a DataLoader nem állít módot
    model.train()                # az egész modellt train módba teszi
    loss = CE(model(xb), yb)
    loss.backward(); optimizer.step()
# később: model.eval(); eval_metrics(model, val_loader)
```

F5 (kísérlettervezési, nehézség 3). Tervezz fejcsere-kísérletet a GlyphLab adaton: a 3-osztályos CNN backbone-ját fagyaszd be, és cseréld a fejet bináris feladatra, ahol az egyik osztály `vertical`, a másik a `horizontal` és `diagonal` egyesítése. Add meg a megfigyelési egységet, a split szabályt, a baseline-t, a metrikát és a következtetési korlátot. Miért nem tekinthető ez külső domainre történő transzfernek? A beadandó terv kötelező elemei: hipotézis, a változó tényező,

kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

20.1.9 Megoldások és indoklás

F1. A `weights=ResNet18_Weights.DEFAULT` (korábbi nevén `pretrained=True`) egy dokumentált korábbi tanításból származó súlyt tölt be; a `weights=None` (korábban `pretrained=False`) véletlen inicializálást ad. Transzfertanulásról csak az előbbi esetben beszélhetünk, mert a tanult reprezentáció átvitele a korábbi súlyokból indul. A `False` esetben a háló architektúrája lehet azonos, de a súlyokban nincs korábbi tudás. A gyakorlatban a logikai kapcsoló mellett a súlyfájl forrását és a preprocessinget is dokumentálni kell.

F2. A momentum-szabály definíció szerint

$$\hat{\mu}_{t+1} = (1-m)\hat{\mu}_t + m\mu_{\text{batch}}.$$

Ha $m > 0$ és $\mu_{\text{batch}} \neq \hat{\mu}_t$, akkor a különbség $\hat{\mu}_{t+1} - \hat{\mu}_t = m(\mu_{\text{batch}} - \hat{\mu}_t) \neq 0$. A jobb oldal egyetlen tényezője sem függ Θ_B elemeinek gradiensétől: a forward kimenet és a batch statisztika a fagyott súlyokkal számolódik, de a bufferfrissítés maga közvetlen értékadás. Tehát $\hat{\mu}$ változhat, miközben Θ_B változatlan. Ugyanez a levezetés érvényes $\hat{\sigma}^2$-re.

F3. Egy teljes megoldás:

```
def freeze_by_prefix(model, prefix):
    trainable = []
    for name, p in model.named_parameters():
        matches = prefix == "" or name == prefix or name.startswith(prefix + ".")
        if matches:
            p.requires_grad_(False)
        elif p.requires_grad:
            trainable.append((name, p))
    return trainable

m = nn.Sequential(nn.Linear(4, 3), nn.Linear(3, 2))
for n, p in m.named_parameters():
    assert p.requires_grad
tr = freeze_by_prefix(m, "0")
assert len(tr) == 2          # a második réteg weight és bias
assert not m[0].weight.requires_grad
assert m[1].weight.requires_grad
```

A névillesztést a `prefix + "."` határral végezzük (üres prefixnél minden név), mert a puszta `startswith(prefix)` a `0` prefixszel a `10.weight` nevet is befagyasztaná. Alakellenőrzés: a lista elemei `(str, Parameter)` párok; a befagyasztott réteg paraméterei `requires_grad=False` értékűek. Szélsőeset: ha `prefix` üres, minden befagy; ha egyetlen név sem illeszkedik, a lista az összes tanítható paramétert visszaadja.

F4. A hibás feltevés az, hogy a `requires_grad_(False)` a BatchNorm futó statisztikáit is leállítja. A `model.train()` az egész modult train módba teszi, így a backbone BatchNorm rétegei a kis minibatch statisztikákkal frissítik a `running_mean` és `running_var` puffereket, noha a súlyaikat nem tanítjuk. Következmény: az eval során a modell olyan normalizálási állapotot használ, amely a legutóbbi mini-batchekhez idomult, ezért a valós kiértékelés romolhat. Két javítás: (1) a backbone moduljait külön `eval()` módba tesszük, a fejet `train()` módban hagyjuk; vagy (2) ha a backbone többi részét train módban kell tartani, kifejezetten a backbone BatchNorm rétegeit állítjuk `eval()` módba. A `momentum=None` nem tiltja le a frissítést, hanem kumulatív átlagra vált, a `track_running_stats=False` pedig eval módban is batch-statisztikát használ;

ezért ezek nem a futó statisztika kimerevítésének helyes eszközei, hanem a modell viselkedését változtatják meg.

F5. Egy lehetséges terv. A GlyphLab 180 jelenet × 2 nézet = 360 kép, 24×24-es, egycsatornás, három osztállyal; a `scene_id` a jelenetet azonosítja. Protokoll: a bináris címke legyen 0 a `vertical`, 1 a `horizontal+diagonal` egyesítésre. A megfigyelési egység `scene_id`; a split a GlyphLab meglévő felosztását tartja, és mindkét view ugyanabba a részbe kerül. A 3-osztályos CNN `mini_cnn_3class.pt` súlyát betöltjük, a fejet `Linear(1152,1)` logisztikus rétegre cseréljük, a backbone-ot befagyasztjuk. Veszteség: `BCEWithLogitsLoss` a fej egyetlen logitján. Baseline: a tanítóbeli többségi osztály aránya, valamint a befagyasztott backbone feature-ein tanított logisztikus regresszió. Metrika: balanced accuracy, mert az egyesítés osztályaránya eltolódhat. A fejet a val spliten választjuk, a testet egyszer értékeljük. Korlát: az adat ugyanabból a generátorból származik, tehát ez feladatváltás, nem domain-transzfer; a mérés nem állít semmit külső eloszláson való használhatóságról.

20.1.10 Tipikus tévedés ellenpéldával

Gyakori tévedés, hogy a `weights=None` háló (korábbi nevén `pretrained=False`) „mégis transzfer, mert az architektúra ismert". Ez nem következik. Példa: hozzunk létre két `MiniCNN` példányt azonos magból, az egyiket tanítsuk 3 osztályra, a másikat hagyjuk véletlen súlyokkal. A második betöltése nélkül a fejcserés tanítás nem indul tanult reprezentációból, így az eredmény nem transzferbizonyíték. A transzfer állításához legalább a betöltött súlyfájl forrását, a tanítási előzményt és a betöltés utáni súly-egyezés ellenőrzését kell felmutatni.

20.1.11 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrizd: (1) a fagyott paraméter `requires_grad` értéke mindenütt `False`-e; (2) a backbone BatchNorm train/eval módja tudatosan választott-e; (3) az optimizer paramétercsoportja csak a tanítandó paramétereket tartalmazza-e; (4) a betöltött `state_dict` kulcsai illeszkednek-e az architektúrához. Korábbi fogalom: a 16.1. Eltolási equivariance és invariance alfejezetben bizonyított eltolási equivariancia a backbone belső tartományán megmarad a fejcserénél, mert f_B változatlan; a 19.1. Depthwise separable és pointwise réteg alfejezetből ismert paraméter/MAC arány sem változik a backbone-ban. A 8.1. Kockázat, mintavétel és baseline alfejezetből pedig azt használjuk, hogy a tanító adaton mért fejeredmény nem általánosít automatikusan más eloszlásra.

20.1.12 Rövid összefoglaló

A feature extractor vagy backbone a bemenetet fix dimenziós reprezentációba képezi, a head pedig erre a reprezentációra épít feladatspecifikus kimenetet. Előtanított súlynak csak dokumentált korábbi tanításból betöltött súly nevezhető. A befagyasztás a gradiens és az optimizer-frissítés tiltása; ettől függetlenül a BatchNorm futó statisztikák train módban tovább mozognak, ezért a befagyasztott backbone-ot általában eval módba tesszük. A labor helyi GlyphLab-CNN-en cserél fejet, és négy invariánssal ellenőrzi a mechanizmust. Valódi transzferhez ellenőrzött külső súlyazonosító és preprocessing-szerződés szükséges.

20.1.13 Következő alkalmazás

A 20.2. Finomhangolás és tanulási ráta a finomhangolást tárgyalja: ott a backbone egy részét felengedjük, és a fejnél kisebb tanulási rátával tanítjuk együtt. Az itteni optimizer groups és a bufferleltár lesz a finomhangolás ütemezésének alapja.

20.1.14 Felhasznált források

- [D09] PyTorch contributors. *Saving and Loading Models*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [D13] PyTorch contributors (2026). *TorchVision models*. PyTorch dokumentáció (torchvision). https://docs.pytorch.org/vision/stable/models.html. Hozzáférés: 2026-10-30.
- [P06] He, K., Zhang, X., Ren, S. és Sun, J. (2016). *Deep Residual Learning for Image Recognition*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). A preprint 2015-ben jelent meg arXiv:1512.03385 számon. https://arxiv.org/abs/1512.03385.

20.2 Finomhangolás és tanulási ráta

20.2.1 Cél, előismeret és induló kérdés

Ez az egység a 20.1. Feature extractor és új fej feature extractor/head szétválasztását fejleszti tovább. Feltételezzük, hogy egy előtanított (vagy legalábbis korábban tanított) modell súlya rendelkezésre áll, és a backbone legalább egy részét a fejjel együtt akarjuk az új feladathoz adaptálni. A cél az, hogy a hallgató pontosan megkülönböztesse a paraméter-fagyasztást, az optimizer-paramétercsoportokat, a rétegenként eltérő tanulási rátát és az `eval()` módot, majd ezeket kontrollált helyi kísérletben alkalmazza.

Induló kérdés: tegyük fel, hogy a fej tanulási rátája $\eta_{\text{head}} = 10^{-2}$, a backbone utolsó blokkjáé pedig $\eta_{\text{last}} = 10^{-3}$. Ha a backbone első blokkja továbbra is `requires_grad=False`, akkor vajon a benne lévő BatchNorm réteg running statisztikái változhatnak-e tanítás közben? (A válasz igen, ha a BatchNorm `train()` módban van; `eval()` módban nem változnak. Ezt az alábbi állításban bizonyítjuk.)

20.2.2 Fine-tuning, gradual unfreeze és discriminative LR

Definíció (fine-tuning). Legyen a modell paramétereinek halmaza $\theta = \theta_b \cup \theta_h$, ahol θ_b a backbone (feature extractor) paramétereit, θ_h a head paramétereit jelöli. Fine-tuningnak nevezzük azt az adaptációt, amelyben legalább egy θ_b-beli paraméter aktív: `requires_grad=True`, szerepel az optimizer valamely paramétercsoportjában, és a fejjel együtt frissül. Ha θ_b minden eleme fagyasztott, akkor feature extractionről beszélünk (20.1. Feature extractor és új fej). A meghatározás a súlyparaméterekre vonatkozik; a bufferek (például a BatchNorm running statisztikái) állapota ettől független lehet, amint azt az alábbi Állítás 1 mutatja.

Definíció (gradual unfreeze). Osszuk a modellt L blokkba: a backbone $L-1$ blokkja $\theta_b = (\theta_1, \dots, \theta_{L-1})$ a mélység növekedésével, az L-edik blokk pedig a head, θ_h. A fokozatos

felengedés T fázisú ütemterv, ahol $1 \le T \le L-1$ (a head-only eset $T = 0$-nak felel meg): a t-edik fázisban ($1 \le t \le T$) az aktív halmaz

$$A_t = \{\theta_{L-t}, \dots, \theta_{L-1}, \theta_h\},$$

és minden $\theta_j \notin A_t$ blokkra `requires_grad=False`. Minden fázis végén a rögzített validációs metrikát kiértékeljük; a végső modellt csak az előre rögzített validációs szabály alapján választjuk (például a legalacsonyabb validációs loss-t elérő checkpointot tartjuk meg, és ezt a szabályt a futás előtt rögzítjük).

Definíció (discriminative LR). Az aktív paraméterekhez blokkonként eltérő tanulási rátát rendelünk: $\eta_h, \eta_{L-1}, \eta_{L-2}, \dots$, tipikusan $\eta_h > \eta_{L-1} > \eta_{L-2}$; itt θ_j egy teljes blokk paraméterhalmaza, nem egyetlen paraméter. Az optimizernek annyi paramétercsoportja van, ahány különböző η_j értéket használunk (az azonos rátát kapó blokkok közös csoportba kerülhetnek); egy csoport a (θ_j, η_j) párt tárolja. SGD esetén a frissítés

$$\theta_j \leftarrow \theta_j - \eta_j \nabla_{\theta_j} \mathcal{L},$$

ahol $\mathcal{L}$ a batch-átlagolt loss; Adam esetén η_j a csoport-specifikus lépésnagyságot skálázza a 6.4. Momentum és adaptív optimizálók alfejezetben bevezetett módon. A gradiens $\nabla_{\theta_j} \mathcal{L}$ alakja mindig megegyezik θ_j alakjával.

Az SGD és az Adam között itt lényeges különbség van. SGD-nél a lépés $\eta_j \nabla_{\theta_j} \mathcal{L}$, ezért a tényleges elmozdulás a gradiens nagyságától is függ: a csoportok LR-aránya önmagában nem határozza meg a paraméterváltozás arányát. Adamnél a frissítés koordinátánként az $\hat{m}_t / \sqrt{\hat{v}_t}$ irányba mutat (az első és második momentum becslése), és a lépés nagysága jó közelítéssel η_j-vel skálázódik, a bias-korrekció és az ϵ módosításával; ezért a csoportok LR-aránya közelebb áll a tényleges relatív lépésnagyság arányához. Ez sem jelenti, hogy a paraméterváltozások aránya pontosan az LR-arány: a momentum- és másodikmomentum-becslés, az ϵ, a veszteség görbültsége és a rétegek csatolása is befolyásolja. A gyakorlati tanulság, hogy Adam alatt is validáción kell ellenőrizni az arányt, és a két optimizer LR-értékei nem cserélhetők át közvetlenül.

A discriminative LR heurisztika: az előtanított alsóbb rétegek általánosabb reprezentációt őriznek, ezért kisebb lépéssel védjük őket; a fej véletlenül inicializált, ezért nagyobb lépést igényel. Ez nem univerzális optimum, hanem keresési stratégiát szűkítő előfeltevés.

20.2.3 Állítás: a fagyasztás és az eval mód különbsége

A levezetési kötelezettségünk a következő állítás.

Állítás 1 (paraméter-fagyasztás eval mód). Legyen B egy PyTorch `BatchNorm2d` modul `track_running_stats=True` beállítással és $m \in (0, 1)$ momentummal. (A `momentum=None` eset ettől eltér: ott a futó átlag kumulatív, $1/t$ súlyú frissítéssel készül; az alábbi érvelés szerkezete ott is áll, csak m helyére $1/t$ lép.) Tegyük fel, hogy mind $\gamma = B$.weight, mind $\beta = B$.bias esetén `requires_grad=False`. Ekkor:

1. Ha B.training = True, akkor egy $X \in \mathbb{R}^{B_{\text{batch}} \times C \times H \times W}$ bemenetre végrehajtott forward után a running statisztikák (generikus esetben) megváltoznak:

   $$\hat{\mu}_{t+1} = (1-m)\hat{\mu}_t + m\,\mu_{B,t}, \qquad \hat{\sigma}^2_{t+1} = (1-m)\hat{\sigma}^2_t + m\,\tilde{\sigma}^2_{B,t},$$

   ahol $\mu_{B,t}$ a batch átlaga, $\hat{\sigma}^2_t$ a running variancia, $\tilde{\sigma}^2_{B,t}$ pedig a batch **torzítatlan** varianciája ($M/(M-1)$-es szorzóval, M a csatornánkénti elemszám). A normalizálásban ezzel szemben a **torzított** $\sigma^2_{B,t} = \frac{M-1}{M}\tilde{\sigma}^2_{B,t}$ szerepel, lásd 18.2. Batch normalization train és eval módban. Ez a frissítés független attól, hogy γ, β kapnak-e gradienst.

2. Ha B.training = False, akkor a forward a rögzített $\hat{\mu}, \hat{\sigma}^2$ értékekkel normalizál, és a running statisztikák nem változnak.

Bizonyítás. A train módú forward képlete a PyTorch dokumentációja [D12] szerint

$$y = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta,$$

majd a fenti momentumos frissítés végrehajtódik, ha `track_running_stats=True`. A `requires_grad=False` csak azt tiltja, hogy a hátrafelé menetben $\frac{\partial \mathcal{L}}{\partial \gamma}$ és $\frac{\partial \mathcal{L}}{\partial \beta}$ gradiensek létrejöjjenek; a running statisztika frissítése nem gradiens-művelet, hanem a modul `training` flagjéhez kötött állapotváltozás. Eval módban a dokumentált útvonal a running statisztikákat használja, és nem végez frissítést. Ezzel az állítás mindkét pontja következik.

A gyakorlati különbséget táblázatban foglaljuk össze:

Beállítás	Súlygradiens	Súlyfrissítés	BN running frissítés
`requires_grad=False`, `train()`	nincs	nincs	van
`requires_grad=False`, `eval()`	nincs	nincs	nincs
`requires_grad=True`, `train()`	van	van, ha optimizerben szerepel	van

20.2.4 Eredeti kézi számpélda

Tekintsük a kétdimenziós modellt: w_h a fej egyetlen súlya, w_b a backbone egyetlen súlya. Legyen a batch-átlagolt loss

$$\mathcal{L}(w_h, w_b) = (w_h + 2w_b - 1)^2,$$

kezdeti érték $w_h = 0{,}5$, $w_b = 1{,}0$. A gradiens:

$$\frac{\partial \mathcal{L}}{\partial w_h} = 2(w_h + 2w_b - 1), \qquad \frac{\partial \mathcal{L}}{\partial w_b} = 4(w_h + 2w_b - 1).$$

A kezdeti hibatag $e = 0{,}5 + 2 \cdot 1 - 1 = 1{,}5$, tehát $g_h = 3$, $g_b = 6$.

- **Head-only:** $\eta_h = 0{,}1$, w_b fagyasztott. Frissítés:

  $$w_h' = 0{,}5 - 0{,}1 \cdot 3 = 0{,}2, \qquad w_b' = 1{,}0.$$

  Az új hibatag $0{,}2 + 2 - 1 = 1{,}2$, az új loss $1{,}2^2 = 1{,}44$ (kezdeti 2,25).

- **Partial unfreeze, discriminative LR:** $\eta_h = 0{,}1$, $\eta_b = 0{,}01$. Frissítés:

  $$w_h' = 0{,}2, \qquad w_b' = 1{,}0 - 0{,}01 \cdot 6 = 0{,}94.$$

  Az új hibatag $0{,}2 + 2 \cdot 0{,}94 - 1 = 1{,}08$, az új loss $1{,}08^2 = 1{,}1664$.

A részleges felengedés itt nagyobb loss-csökkenést ad, de ez csak ez a kétdimenziós kvadratikus példa; általános fölényt nem állítunk.

Minimális ellenőrző kód:

```
import torch

# head-only
w_h = torch.tensor([0.5], requires_grad=True)
w_b = torch.tensor([1.0], requires_grad=False)
loss = (w_h + 2*w_b - 1)**2
loss.backward()
print(w_h.grad, w_b.grad)          # tensor([3.]) None
opt = torch.optim.SGD([w_h], lr=0.1)
opt.step()
print(w_h, w_b)                    # tensor([0.2000], requires_grad=True) tensor([1.])

# partial unfreeze, külön paramétercsoport
w_h = torch.tensor([0.5], requires_grad=True)
w_b = torch.tensor([1.0], requires_grad=True)
loss = (w_h + 2*w_b - 1)**2
loss.backward()
opt = torch.optim.SGD([
    {"params": [w_h], "lr": 0.1},
    {"params": [w_b], "lr": 0.01},
])
opt.step()
print(w_h, w_b)                    # tensor([0.2000], requires_grad=True) tensor([0.9400],
↪  requires_grad=True)
```

20.2.5 Labor: head-only és partial unfreeze a GlyphLab-CNN-en

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy`, `labels.npy` és `metadata.csv` fájlja; a `finetune_compare.py` a projekt gyökeréből `python lab/finetune_compare.py`, a `lab/` könyvtárból `python finetune_compare.py` paranccsal futtatható.

Protokoll. Kérdés: a GlyphLab-CNN-en azonos epochszám mellett a head-only vagy az utolsó konvolúciós blokkot kis rátával felengedő változat ad-e nagyobb validációs accuracy-t? Adat: a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-szerződés szerinti `data/` könyvtár, train 240/val 60/test 60 kép; a teszt nem választ modellt. Modell: a 20.1. Feature extractor és új fej alfejezetből ismert CNN: conv1(1→8, 3×3, stride 2, padding 1) + BN + ReLU, conv2(8→16, 3×3, stride 2, padding 1) + BN + ReLU, GAP, Linear(16→3). A head a GAP + Linear; a partial változat a conv2–BN2 blokkot engedi fel. Baseline: head-only. Változó tényező: a backbone utolsó blokkjának aktiválása és LR-je. Metrika: validációs accuracy; mellékesen train loss. Rögzített recept: Adam, batch 32, 10 epoch, float32 CPU, seedkészlet $\{20261020, 20261021, 20261022\}$ (mindkét változatra ugyanaz); a változatokat seedenként párosítva hasonlítjuk össze, a döntési szabály a páros különbségek átlaga és 95%-os bootstrap konfidenciaintervalluma (23.3. Páros összehasonlítás és csoportszintű intervallum).

A kód a `lab/finetune_compare.py` fájlban van; az adatot a script melletti `data/` könyvtárból olvassa, ezért a projekt gyökeréből (`python lab/finetune_compare.py`) és a `lab/` könyvtárból (`python finetune_compare.py`) is futtatható.

```
import csv
from pathlib import Path
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
```

```
SEED = 20261020
BASE = Path(__file__).resolve().parent      # lab/
DATA = BASE / "data"
device = torch.device("cpu")
dtype = torch.float32

# GlyphLab betöltés
X = np.load(DATA / "images.npy").astype(np.float32)    # (360,1,24,24)
y = np.load(DATA / "labels.npy").astype(np.int64)      # (360,)
with open(DATA / "metadata.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
# A sorrendre nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
if not rows or "index" not in rows[0]:
    raise ValueError("a metadata.csv-ből hiányzik az index oszlop")
if not (len(rows) == len(X) == len(y)):
    raise ValueError("a metadata.csv és a tömbök hossza eltér")
idx = np.array([int(r["index"]) for r in rows], dtype=np.int64)
if not np.array_equal(np.sort(idx), np.arange(len(X))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
split = np.array([r["split"] for r in rows])

X = torch.from_numpy(X).to(device)
y = torch.from_numpy(y).to(device)
train_idx = idx[split == "train"]
val_idx   = idx[split == "val"]
assert (len(train_idx), len(val_idx)) == (240, 60)
BATCH = 32

def train_order(epoch):
    # Külön generátor: a sorrend nem függ a globális RNG állapotától, ezért
    # mindkét változat ugyanazt a batch-sorrendet kapja.
    g = torch.Generator().manual_seed(SEED + epoch)
    return torch.randperm(len(train_idx), generator=g)

val_loader = DataLoader(TensorDataset(X[val_idx], y[val_idx]),
                        batch_size=64, shuffle=False)

class GlyphLabCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Conv2d(1, 8, 3, stride=2, padding=1),
            nn.BatchNorm2d(8),
            nn.ReLU(),
            nn.Conv2d(8, 16, 3, stride=2, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(16, 3),
        )
    def forward(self, x):
        return self.head(self.backbone(x))

def set_frozen_layerwise(model, backbone_slice, head_trainable):
    # backbone_slice=None: egyetlen backbone-paraméter sem aktív (head-only).
    for p in model.parameters():
        p.requires_grad = False
    if backbone_slice is not None:
        for p in backbone_slice.parameters():
            p.requires_grad = True
```

```
    for p in model.head.parameters():
        p.requires_grad = head_trainable

def evaluate(model, loader):
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for xb, yb in loader:
            logits = model(xb)
            correct += (logits.argmax(1) == yb).sum().item()
            total += yb.size(0)
    acc = correct / total
    assert 0.0 <= acc <= 1.0
    return acc

init_sig = None
ref_order = None

def run_variant(partial):
    # Minden változat azonos inicializálást és batch-sorrendet kap.
    global init_sig, ref_order
    torch.manual_seed(SEED)
    model = GlyphLabCNN().to(device).to(dtype)
    sig = float(model.backbone[0].weight.detach().double().sum())
    if init_sig is None:
        init_sig = sig
    assert sig == init_sig, "a két változat inicializálása eltér"
    order0 = train_order(0)
    if ref_order is None:
        ref_order = order0.clone()
    assert torch.equal(order0, ref_order), "a két változat batch-sorrendje eltér"
    if partial:
        # utolsó backbone blokk: index 3-tól (conv2, BN2, ReLU)
        set_frozen_layerwise(model, model.backbone[3:], True)
        opt = torch.optim.Adam([
            {"params": model.head.parameters(), "lr": 0.01},
            {"params": model.backbone[3:].parameters(), "lr": 0.001},
        ])
    else:
        set_frozen_layerwise(model, None, True)
        opt = torch.optim.Adam(model.head.parameters(), lr=0.01)
    loss_fn = nn.CrossEntropyLoss()
    train_losses = []
    for epoch in range(10):
        model.train()
        if partial:
            model.backbone[:3].eval()   # a befagyasztott conv1, BN1 eval marad
        else:
            model.backbone.eval()       # a head train, a backbone eval marad
        order = train_order(epoch)
        epoch_loss = 0.0
        for start in range(0, len(order), BATCH):
            bidx = order[start:start + BATCH]
            xb, yb = X[train_idx[bidx]], y[train_idx[bidx]]
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            opt.step()
            epoch_loss += loss.item() * xb.size(0)
        train_losses.append(epoch_loss / len(train_idx))
    val_acc = evaluate(model, val_loader)
    return train_losses[-1], val_acc
```

```
head_loss, head_acc = run_variant(partial=False)
part_loss, part_acc = run_variant(partial=True)
print(f"head-only:  train_loss={head_loss:.3f}, val_acc={head_acc:.3f}")
print(f"partial:    train_loss={part_loss:.3f}, val_acc={part_acc:.3f}")
```

A kimenet ezen a környezeten mért érték: Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`, a protokoll seedkészletének első eleme, seed 20261020 (a másik két seedet ez a kód nem futtatja). A kód a split méretét, a két változat azonos inicializálását és batch-sorrendjét, valamint az accuracy értéktartományát `assert`-tel ellenőrzi; a hallgató a fenti paranccsal saját környezetében reprodukálhatja:

```
head-only:  train_loss=1.098, val_acc=0.333
partial:    train_loss=0.835, val_acc=0.650
```

A fenti kód egyetlen seeddel fut, ezért a kimenet mechanizmus-szemléltetés: a két változat közti különbség nem választható el a minibatch-zajtól. A 60 képes validációs halmazon egyetlen kép $1/60 \approx 1{,}7$ százalékpont, ezért egy-két képnyi eltérés önmagában nem bizonyíték. Értelmezhető összehasonlításhoz legalább három seed szükséges, a protokollban megadott seedkészlettel és páros kiértékeléssel; a három seedes futást ez a fejezet nem tartalmazza.

20.2.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget a következő három fogalom között: (a) `requires_grad=False`; (b) a paraméter kihagyása az optimizer paramétercsoportjaiból; (c) `model.eval()` mód. Adjon mindháromra egy-egy mondatos példát, és mutasson egy olyan helyzetet, ahol (a) teljesül, de a fagyasztott modul állapota mégis változik.

F2 (levezetési, nehézség 1). Legyen $\mathcal{L}(w_h, w_b) = (2w_h - w_b + 1)^2$, kezdeti $w_h = 1$, $w_b = 0{,}5$, $\eta_h = 0{,}2$, $\eta_b = 0{,}02$. Számítsa ki a gradienseket, majd a frissített paramétereket és a loss értékét (i) head-only, (ii) partial unfreeze esetén. Ellenőrizze, hogy a partial eset loss-a kisebb-e a head-only eseténél.

F3 (implementációs, nehézség 2). Írjon futtatható PyTorch kódot, amely igazolja az Állítás 1-et: egy `BatchNorm2d(4)` modul γ, β paramétereire állítsa be a `requires_grad=False` értéket, majd train módban hajtson végre egy forwardot, és ellenőrizze, hogy a `running_mean` megváltozott; ezután eval módban ismételje meg, és ellenőrizze, hogy ott nem változott. Adja meg a seedet, a bemeneti alakot és a várt kimenetet.

F4 (hibakeresési, nehézség 2). Az alábbi head-only tanítási vázlat hibás:

```
def train_head_only_bad(model, loader, epochs):
    model.train()                       # hiba itt
    for p in model.backbone.parameters():
        p.requires_grad = False
    for p in model.head.parameters():
        p.requires_grad = True
    opt = torch.optim.Adam(model.head.parameters(), lr=0.01)
    ...
```

Lokalizálja a hibát, magyarázza el a következményét a BN running statisztikákra és a későbbi eval pontosságra, majd adja meg a javítást és egy ellenőrzési lépést.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen kontrollált kísérletet a GlyphLab-CNN-en a fokozatos felengedés hatásának mérésére három kezelési szinttel: (1) head-only, (2) utolsó blokk + head, (3) mindkét blokk + head. Adja meg a kérdést, a baseline-t, a metrikákat, a split- és seed-kezelést, a validációs kiválasztási szabályt, és a következtetés legalább két konkrét korlátját. Nem kell kódot írnia, de a protokollnak lektorálhatónak kell lennie. A beadandó terv

kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

20.2.7 Megoldások és indoklás

F1 megoldás. (a) A `requires_grad=False` azt tiltja, hogy az autograd a paraméterhez gradienst tároljon; ezáltal az optimizer sem tudja frissíteni. (b) Ha a paraméter `requires_grad=True`, de nincs benne egyetlen optimizer paramétercsoportban sem, akkor a gradiens kiszámolódhat, de a frissítés elmarad; ez ritkán kívánatos, mert felesleges számítást okoz. (c) Az `eval()` mód a training-specifikus viselkedéseket kapcsolja ki: a BatchNorm a running statisztikákat használja, nem frissíti azokat, és a dropout-rétegek (ha vannak a modellben) inaktívak. Az (a) melletti állapotváltozásra példa a fenti Állítás 1: a fagyasztott BN train módban frissíti a running statisztikáit.

F2 megoldás. A hibatag $e = 2w_h - w_b + 1 = 2 \cdot 1 - 0{,}5 + 1 = 2{,}5$. A gradiensek:

$$\frac{\partial \mathcal{L}}{\partial w_h} = 2 \cdot 2e = 4e = 10, \qquad \frac{\partial \mathcal{L}}{\partial w_b} = -2e = -5.$$

(i) Head-only: $w_h' = 1 - 0{,}2 \cdot 10 = -1$, $w_b' = 0{,}5$. Új hibatag $2(-1) - 0{,}5 + 1 = -1{,}5$, loss 2,25. (ii) Partial: $w_h' = -1$, $w_b' = 0{,}5 - 0{,}02 \cdot (-5) = 0{,}6$. Új hibatag $-2 - 0{,}6 + 1 = -1{,}6$, loss $2{,}56 > 2{,}25$. Tehát ebben a példában a partial frissítés rontott; ez mutatja, hogy a felengedés nem garantál monoton javulást, és a heurisztikát empirikusan kell ellenőrizni. A romlás mechanizmusa az, hogy mindkét lépés túllő az $e = 0$ minimumon (a hibatag előjelet vált: $2{,}5 \to -1{,}5$, illetve $-1{,}6$), és a partial lépés nagyobb mértékben lő túl, ezért lesz $|e'|$ 1,6 a head-only 1,5-tel szemben.

F3 megoldás.

```
import torch
import torch.nn as nn

torch.manual_seed(0)
bn = nn.BatchNorm2d(4)
bn.weight.requires_grad_(False)
bn.bias.requires_grad_(False)

bn.train()
before = bn.running_mean.clone()
bn(torch.randn(2, 4, 3, 3))
changed_train = not torch.equal(before, bn.running_mean)
print("train mód, változott:", changed_train)
assert changed_train

bn.eval()
before = bn.running_mean.clone()
bn(torch.randn(2, 4, 3, 3))
changed_eval = not torch.equal(before, bn.running_mean)
print("eval mód, változott:", changed_eval)
assert not changed_eval
```

A kód futtatott kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, `seed=0`): `train mód, változott: True`, majd `eval mód, változott: False`. A momentum frissítés miatt train módban a running statisztika csak akkor maradna változatlan, ha a batch átlag éppen megegyezne a régivel; normál véletlen bemenetre a változás majdnem biztosan bekövetkezik (a kivétel nulla valószínűségű esemény, nem logikai szükségszerűség).

F4 megoldás. A hiba a `model.train()` hívás: a befagyasztott backbone BN rétegei train módban maradnak, ezért minden forward a fej tanításához használt mini-batchek átlagát és

varianciáját írja a running bufferbe, miközben a backbone konvolúciós súlyai fagyottak. A running statisztika és a hozzá tartozó fagyott súlyok így eltérő adateloszlást írnak le: eval módban a normalizálás a legutóbbi mini-batchek statisztikáját használja, a konvolúciók viszont a korábbi tanító eloszláson tanult reprezentációt adják. Ez inkonzisztens reprezentációt és romló eval-pontosságot okozhat. A javítás:

```
def train_head_only_fixed(model, loader, epochs):
    for p in model.backbone.parameters():
        p.requires_grad = False
    for p in model.head.parameters():
        p.requires_grad = True
    for epoch in range(epochs):
        model.train()
        model.backbone.eval()      # minden epochban, a model.train() után
        ...
```

Ellenőrzés: a tanítás előtt és után klónozzuk a `model.backbone[1].running_mean` értéket, és igazoljuk, hogy azonos maradt; a head paraméterei viszont változtak.

F5 megoldásminta. Kérdés: a fokozatos felengedés javítja-e a validációs accuracy-t a head-only baseline-hoz képest a GlyphLab-CNN-en? Adat: a GlyphLab train/val/test splitje; a test kizárólag a végleges, validációval kiválasztott modellre kerül, egyszer. Baseline: head-only, Adam, $\eta_{\text{head}} = 0{,}01$, 10 epoch. Három kezelési szint (nem időbeli fázis): (1) head-only (ez a baseline), (2) utolsó blokk + head, $\eta_{\text{head}} = 0{,}01$, $\eta_{\text{last}} = 0{,}001$, (3) mindkét blokk + head, $\eta_{\text{head}} = 0{,}01$, $\eta_{\text{conv2}} = 0{,}001$, $\eta_{\text{conv1}} = 0{,}0001$. Mindegyik kezelést azonos 10 epochos költségvetéssel, ugyanazon a 3 seeden futtatjuk, ezért a kezelések seedenként párosíthatók; a kiválasztás a validációs accuracy seed-átlagán alapul, és a páros különbségek 95%-os bootstrap konfidenciaintervallumát is megadjuk (23.3. Páros összehasonlítás és csoportszintű intervallum). Metrika: validációs accuracy, valamint a train loss görbe a túlillesztés jelzésére. Mivel ugyanazt a validációs értéket használjuk kiválasztásra és jelentésre, ez a szám optimista: a maximum kiválasztása felfelé torzít (kiválasztási torzítás, adaptive overfitting); az elfogulatlan becslés a teszt egyszeri kiértékelése. Korlátok: a GlyphLab nem természetes kép, az eredmény nem általánosítható valódi domain shiftre; a 3 seed kevés a statisztikailag robusztus különbségtételhez, ezért a páros különbségek CI-je széles lesz, és az eredmény csak leíró jellegű; a teszt ismételt használata modellválasztásra tovább növeli a kiválasztási torzítást, ezért a tesztet csak egyszer, a végleges modellen értékeljük. A 8.2. Véges hipotézisosztály egyenletes korlátja finite-H korlátja a hipotézisosztály méretének hatását írja le az egyenletes konvergenciára; a modellválasztásra ez csak analógiaként vonatkozik, mert ott nem a hipotézisosztály, hanem a kiválasztási eljárás hozza be a torzítást.

20.2.8 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés: *„Ha `requires_grad=False`, akkor a modul minden állapota befagyott."* Ez hamis. Az Állítás 1 bizonyítja, hogy a BatchNorm `running_mean` és `running_var` bufferei train módban frissülnek akkor is, ha a γ, β paraméterek nem tanulnak. A fenti F3 kód a konkrét ellenpéldát adja: a `running_mean` megváltozik train módban, és nem változik eval módban. Ezért a befagyasztott backbone-ot a head-only adaptáció során explicit `eval()` módba kell tenni; a partial unfreeze esetén pedig csak a felengedett blokkokat hagyjuk train módban.

20.2.9 Önellenőrzés és ismétlés

Ellenőrizze: (1) minden fagyasztott BN eval módban van-e; (2) az optimizer paramétercsoportjai pontosan az aktív paramétereket tartalmazzák-e; (3) a validációs metrika nem választott-e tesztet; (4) ugyanaz a seed és dtype szerepel-e minden futásban; (5) a csoportok LR értékei a tervezett

arányt követik-e. A tervezett arány a protokollban a futás előtt rögzített $\eta_h : \eta_{\text{last}} : \eta_{\text{first}}$ arány (például $1 : 0{,}1 : 0{,}01$), nem a futás közben kialakult érték.

Ismétlés: a 8.2. Véges hipotézisosztály egyenletes korlátja finite-H korlátja szerint a nagyobb keresési tér nagyobb statisztikai hibát okozhat; a partial unfreeze ezért nem automatikusan jobb, hanem validációval igazolandó kompromisszum. A 16.2. Pooling, downsampling és aliasing tanulsága, hogy a GAP előtti utolsó blokk a legkisebb térbeli felbontású (itt 6×6-os) reprezentációt állítja elő, és a blokkban nincs pooling: a 6×6-os méret a stride 2 konvolúcióból jön, a GAP pedig a blokk kimenetén, a fejen belül történik. Az adaptálás hatása ezért a legutolsó konvolúciós szintre korlátozódik; hogy ez nagyobb hatású-e a korábbi blokkok felengedésénél, csak mérés döntheti el, a priori nem állítható. A 19.2. MobileNet és inverted residual inverted residual blokkjában a blokkstruktúra adja a gradual unfreeze természetes határait: nem tetszőleges tenzort, hanem a blokk kimenetét engedjük fel.

20.2.10 Rövid összefoglaló

A fine-tuning az előtanított backbone legalább egy részének és a fejnek együttes adaptációja. A gradual unfreeze a blokkokat fokozatosan, a discriminative LR pedig rétegenként eltérő tanulási rátával tanítja; mindkettő heurisztika, validációval kontrolálandó. Bebizonyítottuk, hogy a `requires_grad=False` nem egyenlő az eval móddal: a fagyasztott BatchNorm train módban frissíti running statisztikáit. A labor a GlyphLab-CNN-en hasonlítja össze a head-only és a részleges felengedést rögzített seeddel; a közölt kimenet egyetlen seeddel mért futás, nem statisztikai összehasonlítás. A feladatok a fogalmi különbségeket, a kézi frissítést, a buffer-tesztet, a hibajavítást és a kísérlettervet fedik le.

20.2.11 Következő alkalmazás és források

A következő egység 20.3. Preprocessing, domain shift és súlylicenc a valódi külső súlyok preprocessing-szerződését, a domain shift mérését és a súlylicenc kérdéseit tárgyalja; az itteni paramétercsoport- és buffer-ismeret ott válik szükségessé, amikor konkrét torchvision modellt illesztünk.

20.2.12 Felhasznált források

- [D12] PyTorch contributors. *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. Hozzáférés: 2026-10-30.

20.3 Preprocessing, domain shift és súlylicenc

20.3.1 Cél, előismeret és induló kérdés

Egy előtanított modell helyes használatához három, egymástól független kérdést kell külön megválaszolni. (1) **Kompatibilis előfeldolgozás:** a bemenet *reprezentációja* (méret, crop, normalizálás, csatornarend, értéktartomány) megegyezik-e azzal, amit a súlycsomag elvár — ez dokumentációból ellenőrizhető szerződés. (2) **Adatdomén-hasonlóság:** mennyire hasonlít a saját adat a forrás tanítóadatához — ez nem garantálható, csak vizsgálható. (3) **A target teljesítmény igazolása:** a modell a saját adaton, külön target-domain értékeléssel mérve elég jó-e. Az első feltétel teljesülése nem jelenti a forrás- és a céladat azonos eloszlását, és az eloszláseltérés

önmagában nem teszi szabálytalanná a transzfert; csak azt jelenti, hogy a (3) mérés nem hagyható ki. Ez az alfejezet ezt a **preprocessing-szerződést** járja körbe: resize/crop, normalizálás, csatornarend, a súlycsomag enumja és a felhasználási licenc. Előismeret a 20.2. Finomhangolás és tanulási ráta finomhangolási anyaga: a `state_dict`, a fagyasztott paraméterek és a BatchNorm-pufferek kezelése. A 19.3. EfficientNet és architektúraskálázás alfejezetben tárgyalt architektúraskálázás és a 16.3. Elméleti és effektív recepciós mező alfejezetben levezetett recepciós mező itt abban tér vissza, hogy a bemeneti felbontás változtatása a háló belső aktivációinak térbeli méretét is megváltoztatja. Induló kérdés: **Milyen konkrét ellenőrzésekkel dokumentálható, hogy a letöltött súlycsomag a mi adatunkon a várt bemeneti reprezentációt kapja, milyen korlátozott target-domain validáció kell a teljesítmény igazolásához, és megengedett-e jogilag a felhasználás?**

20.3.2 Definíciók: preprocessing-szerződés, domain shift, licenc

Legyen $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ a nyers képadat batch, S a tanító adateloszlás (source), T a felhasználáskori eloszlás (target). **Preprocessing-szerződésnek** nevezzük azt a leképezést, amely a nyers pixelekből a háló által várt normalizált bemenetet állítja elő. Ez legalább négy komponenst tartalmaz:

1. **Geometriai előkészítés** (resize, crop): a bemenet térbeli méretét (H, W)-re állítja. A resize interpolációt, a crop térbeli kivágást jelent. A háló elméleti recepciós mezője az architektúrától (kernel-, stride- és dilation-értékektől) függ, nem a bemeneti mérettől; a későbbi rétegek outputmérete viszont a 14.1. Stride, padding, dilation és outputméret és 16.3. Elméleti és effektív recepciós mező képletei szerint a bemeneti mérettől is függ. Ezért a szerződésben rögzíteni kell a pontos bemeneti méretet.
2. **Normalizálás** (mean/std): minden csatornára egy (μ_c, σ_c) pár. A normalizált pixel: $x'_c = (x_c - \mu_c)/\sigma_c$.
3. **Csatornarend**: a C_{in} csatornák sorrendje. PyTorchban NCHW, más keretrendszerekben gyakran NHWC; színképeknél RGB–BGR eltérés is előfordul.
4. **Értéktartomány**: a bemeneti intenzitás skálája, például $[0, 1]$ vagy $[0, 255]$. A normalizálás előtt a kettő közötti konverzió is a szerződés része.

Domain shift az az állapot, amikor $S \neq T$. Ez nem önmagában hiba, hanem mérhető eltérés: az adateloszlások különbözősége miatt a modell a target eloszláson rosszabb teljesítményt mutathat. A **súlycsomag enumja** a közzétett súlyváltozatok azonosítója (pl. `IMAGENET1K_V1`, `DEFAULT`). A **licenc** a súlyok felhasználási feltételeit rögzítő jogi dokumentum; a letölthető súly megléte nem jelenti azt, hogy tetszőleges célra felhasználható.

A torchvision dokumentációja [D13] a modellek betöltéséhez `weights` enum értékeket használ; a `DEFAULT` az adott könyvtárverzió ajánlott súlyát jelöli. A [D09] oktatóanyag a saját modell mentését és betöltését tárgyalja; ezt a saját kísérleti checkpointra alkalmazzuk. A [P06] a ResNet-reziduális blokkokat vezeti be; a 3.1–3.2 szakasz a reziduális tanulást és a $y = F(x) + x$ összefüggést definiálja (nem bizonyítja), a cikk licencfeltételeire pedig nem tér ki.

20.3.3 Két konkrét ellenpélda

20.3.3.1 Ellenpélda 1: rossz normalizálás megváltoztatja a modell kimenetét

Tegyük fel, hogy egy előtanított modell első rétege egy lineáris projekció $Z = XW^\top + b$ alakban, ahol $X \in \mathbb{R}^{1 \times d}$ egy bemeneti sorvektor, $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$. A modell készítője a normalizálást $(\mu, \sigma) = (0{,}5;\ 0{,}25)$ értékekkel végezte, így a háló bemenete $X' = (X - 0{,}5)/0{,}25 = 4X - 2$. A

felhasználó ezt elhagyja, és nyers X-et ad be. Ekkor a háló által számított érték $Z_{raw} = XW^\top + b$, míg a szerződés szerint $Z_{expected} = (4X - 2 \cdot \mathbf{1}_d^\top)W^\top + b = 4XW^\top - 2 \cdot \mathbf{1}_d^\top W^\top + b$, ahol $\mathbf{1}_d$ a d-dimenziós csupa egyes oszlopvektor. A különbség:

$$Z_{raw} - Z_{expected} = XW^\top + b - (4XW^\top - 2 \cdot \mathbf{1}_d^\top W^\top + b) = -3XW^\top + 2 \cdot \mathbf{1}_d^\top W^\top.$$

A különbség általában **nem** nulla: csak akkor tűnik el, ha $XW^\top = \frac{2}{3}\,\mathbf{1}_d^\top W^\top$ (itt $XW^\top$ és $\mathbf{1}_d^\top W^\top$ is $1 \times m$ alakú; utóbbi a W sorösszegeinek vektora). Legyen például $d = 2$, $W = (1, 1)$, $b = 0$, és $X = (1, 1)$. Ekkor $Z_{raw} = 2$, míg $Z_{expected} = 4 \cdot 2 - 2 \cdot 2 = 8 - 4 = 4$. A modell más logitot ad, holott a bemenet ugyanaz a kép. Ez az ellenpélda mutatja: a normalizálás elhagyása nem egyszerűen skálázza a kimenetet, hanem a normalizálásból származó konstans eltolás (-2) a W sorösszegein keresztül additív tagot ad; a bias b a különbségből kiesik.

20.3.3.2 Ellenpélda 2: azonos elemszámú, de hibás csatornarend

Vegyünk egy $3 \times 2 \times 2$ méretű színképet, azaz $C_{in} = 3$, $H = W = 2$. A csatornák sorrendje PyTorchban NCHW: a tensor `[1, 3, 2, 2]` alakú. Egy másik könyvtárból érkező adat lehet NHWC, `[1, 2, 2, 3]`. Ha a felhasználó az NHWC tensort NCHW-ként adja be, akkor a `[1, 2, 2, 3]` alakot a háló úgy értelmezi, mint $N = 1$, $C = 2$, $H = 2$, $W = 3$ bemenetet. Ez elsősorban shape-hiba: a háló $C = 2$ csatornát lát, miközben a modell $C = 3$-at vár. Ettől független hibatípus, amikor az alakséma megegyezik, de a csatornák szemantikája cserélődik fel: például egy RGB kép és egy BGR kép esetén mindkettő $C = 3$, a shape egyezik, a sorrend mégsem. Legyen az eredeti RGB pixel: $p = (r, g, b) = (0{,}9;\ 0{,}1;\ 0{,}2)$. A BGR sorrendben ugyanez a pixel $(0{,}2;\ 0{,}1;\ 0{,}9)$. A modell első konvolúciós rétegének súlyai az RGB csatornákra vannak tanítva; a BGR sorrendben beadott bemenet esetén az első csatorna súlyai a kék intenzitást látják, a harmadiké a vöröset. A kimeneti aktivációk megváltoznak, mégpedig nem pusztán a csatornák felcserélésével, mert a rákövetkező rétegek súlyai is fixek. Az elemszám (3 csatorna) és a tensor mérete azonos, de a szemantikai tartalom hibás. Ez a példa illusztrálja, hogy a csatornarend nem redukálható a csatornák darabszámára.

20.3.4 Kézi számpélda és ellenőrző kód

Tekintsünk egy 1×1 méretű, 3 csatornás képet: $x = [0{,}8;\ 0{,}4;\ 0{,}6]$. A szerződés szerint a normalizálás $\mu = [0{,}485;\ 0{,}456;\ 0{,}406]$, $\sigma = [0{,}229;\ 0{,}224;\ 0{,}225]$; ezek az ImageNet-en tanított torchvision-modellek dokumentált RGB-csatorna-statisztikái, amelyeket a súlyokhoz tartozó `weights.transforms()` pipeline ad meg [D13]. Számoljuk ki a normalizált értékeket:

$$x'_0 = (0{,}8 - 0{,}485)/0{,}229 \approx 1{,}3755, \quad x'_1 = (0{,}4 - 0{,}456)/0{,}224 \approx -0{,}25, \quad x'_2 = (0{,}6 - 0{,}406)/0{,}225 \approx 0{,}8622.$$

Python-ellenőrzés (a művelet egyszerű, NumPy-val vagy PyTorch-kal is elvégezhető; az alábbi kód futtatható a `lab/` munkakönyvtárból):

```
# lab/preprocess_check.py
import numpy as np

x = np.array([0.8, 0.4, 0.6], dtype=np.float32)
mu = np.array([0.485, 0.456, 0.406], dtype=np.float32)
sigma = np.array([0.229, 0.224, 0.225], dtype=np.float32)

x_norm = (x - mu) / sigma
print(x_norm)  # [1.3755..., -0.25, 0.8622...]
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU):

```
[ 1.3755459  -0.24999997  0.8622224 ]
```

A három érték a fenti kézi számítás `1.3755`, `-0.25`, `0.8622` értékét adja `float32`-ban.

Ha a felhasználó a normalizálást kihagyja, és a nyers [0,8; 0,4; 0,6]-t adja be, az első réteg lineáris transzformációja eltér a várttól, ahogy az 1. ellenpéldában láttuk. A csatornarend ellenőrzésére hasonlóan: ha a képet NHWC-ból NCHW-ba konvertáljuk, ügyelni kell arra, hogy a `transpose` ne cserélje fel a színcsatornákat. Egy RGB → BGR hiba reprodukálásához elegendő a csatornák indexelését megfordítani: `x_bgr = x[::-1]`. A két vektor elemszáma azonos, de a kép szemantikailag más.

20.3.5 Labor: súly-adat-preprocessing eredetjegyzék és licencellenőrzés

Előfeltétel. Python 3.12.3 és NumPy 2.5.4 a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. Bemenet a GlyphLab `lab/data/images.npy` fájlja; a `lab/check_preprocessing.py` futtatható (`python lab/check_preprocessing.py`).

A labor célja egy **súly-adat-preprocessing eredetjegyzék** elkészítése a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab saját adataira és egy lehetséges külső súlycsomagra. A GlyphLab adatok a jelen alfejezet előtti fejezetekben már ellenőrzöttek: `data/images.npy` alakja `(360, 1, 24, 24)`, float32, intenzitás $[0, 1]$, grayscale. Mivel a saját modellünk ezen az adaton tanult, a preprocessing-szerződés a következő:

- **Resize/crop**: nincs; a képek már 24×24-esek és NCHW formátumúak.
- **Normalizálás**: nincs per-pixel normalizálás; a bemenet $[0, 1]$ tartományú. A háló első rétege nyers intenzitásokat kap.
- **Csatornarend**: NCHW, $C_{in} = 1$; nem releváns a grayscale miatt, de a szerződésben rögzítjük, hogy a csatornaszám 1.
- **Értéktartomány**: $[0, 1]$, float32.

A külső súlyoknál a csatornaszám is a preprocessing-szerződés része: egy ImageNet-en tanított modell háromcsatornás bemenetet vár, a GlyphLab viszont egycsatornás. Három lehetőség van: (i) a grayscale képet három csatornára ismételjük, ekkor a modell fut, de az első réteg RGB-re tanult súlyai nem grayscale-re optimalizáltak; (ii) az első konvolúciót egy bemeneti csatornára cseréljük, ekkor az első réteg előtanított súlya elveszik, a többi rétegé megmarad; (iii) eleve grayscale-re tanított súlycsomagot választunk. A választást és annak következményét az eredetjegyzékben rögzíteni kell.

Ezt a szerződést a `lab/preprocessing_ledger.md` fájlban rögzítjük. A fájl tartalma:

```
# Preprocessing eredetjegyzék - GlyphLab

| Mező | Érték | Ellenőrzés |
|---|---|---|
| Adatfájl | data/images.npy | quickcheck.py |
| Alak | (360, 1, 24, 24) | kézi |
| dtype | float32 | kézi |
| Intenzitás tartomány | [0, 1] | min/max |
| Normalizálás | nincs | - |
| Csatornarend | NCHW | - |
| Súly forrása | saját training, seed 20261020 | state_dict |
| Licenc | saját, oktatási célra | - |
```

A táblázat `Ellenőrzés` oszlopában a `kézi` a szerződés kézi egyeztetését jelöli, a `-` pedig azt, hogy az adott mezőt nincs automatikus ellenőrzés, csak a szerződés rögzíti.

A licencellenőrzéshez a külső súlyok esetén a [D13] és [P06] forrásokat kell használni. Az előtanított súlyok felhasználási feltételei modell- és súlyváltozatonként eltérhetnek; a pontos licencet a modellkatalógus és a súlyok eredeti forrása alapján, még letöltés vagy továbbadás előtt kell rögzíteni. A puszta katalógusbeli elérhetőség nem jelent korlátlan újrafelhasználási jogosultságot. A [P06] cikk esetében az általunk elérhető arXiv-változatban nem találtunk licencszekciót; ez azonban nem jogi licencnyilatkozat, ezért a licencet a kiadói/szerzői forrásból kell ellenőrizni. A labor elvárt eredménye tehát:

1. A fenti `preprocessing_ledger.md` fájl elkészítése.
2. Egy külön `license_checks.md` fájl, amely rögzíti: (a) a GlyphLab saját súlyainak oktatási felhasználása; (b) a torchvision modellkatalógus [D13] felhasznált súlyváltozatának licencfeltételeit; (c) a [P06] licenc hiányának megjelölését.

Fontos: a saját modellünk súlyaira a saját licencdöntésünk vonatkozik, amelyet szintén rögzíteni kell az eredetjegyzékben; a külső súlyokra vonatkozó állításokat pedig csak a forrás tényleges elolvasása után tehet. A labor kódja nem igényel külső letöltést; a GlyphLab adatok helyben elérhetők. Az alábbi ellenőrző kód a preprocessing-szerződés betartását teszteli:

```python
# lab/check_preprocessing.py
import numpy as np
from pathlib import Path

BASE = Path(__file__).resolve().parent          # lab/
images = np.load(BASE / "data/images.npy")
assert images.dtype == np.float32
assert images.shape == (360, 1, 24, 24)
assert images.min() >= 0.0 and images.max() <= 1.0
print("Preprocessing-szerződés teljesül: [0,1], float32, NCHW, alak (360,1,24,24)")
```

Ez a kód bármely munkakönyvtárból futtatható: `python lab/check_preprocessing.py`. A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, CPU, a rögzített `lab/data` könyvtáron):

```
Preprocessing-szerződés teljesül: [0,1], float32, NCHW, alak (360,1,24,24)
```

Az állításokat a laborcsomag `quickcheck.py` ellenőrzője is megerősíti (a szkriptet és a referencia-lenyomatokat lásd A. A GlyphLab adatkészlet).

20.3.6 F1–F5 feladatok

F1 – fogalmi (1. nehézség). Mit jelent a preprocessing-szerződés négy komponense? Adjon mindegyikre egy-egy mondatos magyarázatot, és egy példát, ahol a komponens elhanyagolása hibához vezet.

F2 – levezetési (2. nehézség). Legyen az előtanított modell első rétege $Z = XW^\top + b$ a fenti 1. ellenpélda szerint, $d = 3$, $W = (1, 1, 1)$, $b = 0$, normalizálás $(\mu, \sigma) = (0{,}5;\ 0{,}25)$. A felhasználó bemenete $X = (0{,}5, 0{,}5, 0{,}5)$. Számítsa ki Z_{raw}-t és $Z_{expected}$-et, és igazolja a különbséget.

F3 – implementációs (2. nehézség). Írjon Python-kódot (`lab/preprocess_normalize.py`), amely beolvassa a GlyphLab `data/images.npy`-t, ellenőrzi a dtype-ot és a tartományt, majd alkalmazza a $(\mu = 0{,}5, \sigma = 0{,}25)$ normalizálást, és elmenti `data/images_norm.npy` néven. A kód tartalmazzon input-ellenőrzést és a kimenet min/max értékének kiírását. A feladat hipotetikus normalizálást mutat be: a GlyphLab saját modelljének szerződése normalizálás nélküli, ezért az `images_norm.npy` nem része a GlyphLab tanítási pipeline-nak.

F4 – hibakeresési (3. nehézség). Tegyük fel, hogy a `weights.pth` egy 224×224-es bemenetre, $[0, 1]$ tartományra és ImageNet mean/std normalizálásra tanított ResNet-18 súlya. A következő kód ezt tölti be, de hibás. Azonosítsa a hibát, magyarázza el, miért sérti meg a preprocessing-szerződést, és milyen értelemben nevezhető az eltérés domain shiftnek.

```
import torch
from torchvision import models

model = models.resnet18(weights=None)
model.load_state_dict(torch.load("weights.pth", weights_only=True))
model.eval()   # inference: a BatchNorm a futó buffert használja
# A felhasználó saját adatai NHWC formátumban, [0,255] uint8.
image = torch.randint(0, 256, (1, 224, 224, 3), dtype=torch.uint8)
image = image.permute(0, 3, 1, 2).float()
with torch.no_grad():
    logits = model(image)
```

F5 – kísérlettervezési (2. nehézség). Tervezzen egy kísérletet, amely a GlyphLab adatokon méri a domain shift hatását, ha a bemeneti intenzitásokat lineárisan átskálázzuk $[0, 1]$-ből $[0;\ 0{,}5]$-be. Adja meg a protokollt, a baseline-t, a metrikát és az érvényességi korlátot. (A futást nem kell elvégezni, csak a tervet leírni.) A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

20.3.7 Megoldások és indoklás

F1. A négy komponens: (1) geometriai előkészítés – a térbeli méret rögzítése; hiba: ha a háló 224×224-re tanult, és 32×32-t kap, a konvolúciós és pooling-rétegek outputmérete a 14.1. Stride, padding, dilation és outputméret képlete szerint összezsugorodik, a globális pooling vagy a teljesen kapcsolt fej bemeneti dimenziója megváltozik, és a forward vagy hibára fut, vagy más alakú jellemzőt ad. (2) Normalizálás – a csatornánkénti közép és szórás levonása; hiba: a nyers [0,255] bemenet a bias interakciója miatt eltolja a logiteket (1. ellenpélda); számszerűen a 255-szörös skálázás a lineáris réteg kimenetének $XW^\top$ tagját és a W-re vonatkozó gradiensét is 255-tel szorozza, a bias tag változatlan marad. (3) Csatornarend – NCHW vs NHWC; hiba: RGB–BGR felcserélés azonos elemszám mellett más színinformációt ad. (4) Értéktartomány – [0,1] vagy [0,255]; hiba: a skálázás elmaradása a fenti 255-szörös eltolást okozza, ami nagy, rosszul kalibrált logiteket ad.

F2. A normalizálás: $X' = (X - 0{,}5)/0{,}25 = 4X - 2$. $X = (0{,}5;\ 0{,}5;\ 0{,}5)$ esetén $Z_{raw} = XW^\top + b = 0{,}5 + 0{,}5 + 0{,}5 = 1{,}5$. $X' = (4 \cdot 0{,}5 - 2;\ 4 \cdot 0{,}5 - 2;\ 4 \cdot 0{,}5 - 2) = (0;\ 0;\ 0)$. Ekkor $Z_{expected} = 0 \cdot W^\top + b = 0$. A különbség $Z_{raw} - Z_{expected} = 1{,}5 - 0 = 1{,}5$. Ez megegyezik a korábbi levezetéssel: $-3XW^\top + 2\mathbf{1}_d^\top W^\top = -3 \cdot 1{,}5 + 2 \cdot 3 = -4{,}5 + 6 = 1{,}5$. Alakellenőrzés: Z skalár, mert $m = 1$.

F3. A kód:

```
# lab/preprocess_normalize.py
import numpy as np
from pathlib import Path

BASE = Path(__file__).resolve().parent          # lab/
images = np.load(BASE / "data/images.npy")
assert images.dtype == np.float32
assert images.min() >= 0.0 and images.max() <= 1.0
mu, sigma = 0.5, 0.25
norm = (images - mu) / sigma
assert norm.dtype == np.float32
```

```
print(f"Eredeti tartomány: [{images.min():.3f}, {images.max():.3f}]")
print(f"Normalizált tartomány: [{norm.min():.3f}, {norm.max():.3f}]")
np.save(BASE / "data/images_norm.npy", norm)
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU):

```
Eredeti tartomány: [0.000, 1.000]
Normalizált tartomány: [-2.000, 2.000]
```

Ellenőrzés: a bemenet $[0, 1]$ tartománya miatt a normalizált értékek $[-2, 2]$ közé esnek; a kód ezt kiírja, és az `assert`-ek a dtype-ot és a bemeneti tartományt is ellenőrzik. A feladat hipotetikus normalizálást mutat be: a GlyphLab saját modelljének szerződése normalizálás nélküli, ezért a létrehozott `images_norm.npy` nem része a GlyphLab tanítási pipeline-nak.

F4. A `permute(0, 3, 1, 2)` NHWC → NCHW átalakítás helyes. A hiba a preprocessing-szerződés megszegése: (a) a $[0, 255]$ uint8 értékeket `/255` skálázás nélkül konvertálja floatra; (b) elmarad az ImageNet mean/std normalizálás. Az előírt pipeline-t a torchvision a súlyokhoz tartozó `weights.transforms()` hívásával adja meg [D13]; ez végzi a resize/cropot, a `ToTensor`-t (amely a `/255` skálázást is tartalmazza) és a `Normalize`-t az ImageNet $\mu = [0{,}485;\ 0{,}456;\ 0{,}406]$, $\sigma = [0{,}229;\ 0{,}224;\ 0{,}225]$ értékeivel. Ez a hiba **a preprocessing-szerződés megsértése**, nem feltétlenül domain shift a szigorú $S \neq T$ értelemben: a mögöttes adatok eloszlása azonos lehet, csak a bemeneti reprezentáció lesz más, mint amire a súlyok tanultak. A következményt akkor írhatjuk le domain shiftként, ha a *helyes* preprocessing után is eltér a target jellemzőinek eloszlása a source-tól; itt a hiba önmagában is elrontja a bemenetet. A javítás: `image.float() / 255.0`, majd az előírt mean/std alkalmazása, vagy a `weights.transforms()` használata; a csatornasorrendet is ellenőrizni kell (a permute után RGB maradt-e). A `model.eval()` hívás szándékosan szerepel a kódban, hogy a példa csak a jelzett preprocessing-hibát mutassa: a BatchNorm futó buffere így nem változik, a hiba nem abból ered.

F5. Kísérleti terv. A GlyphLab a 20.1. Feature extractor és új fej és a függelék A. A GlyphLab adatkészlet szerinti procedurálisan generált, szintetikus adat: 180 jelenet × 2 nézet = 360 kép, 24×24, egycsatornás, három osztállyal (`vertical`, `horizontal`, `diagonal`); a modell a fejezet laborjában használt, már betanított GlyphLab-CNN (a 20.1 MiniCNN architektúra). (a) Baseline: a betanított CNN per-image pontossága a teszt split 60 képén; a per-image pontosság a helyes előrejelzések aránya, a macro-F1 pedig a három osztály F1-értékének átlaga. (b) Kezelés: a tesztképek intenzitásait $x \mapsto 0{,}5x$ módon átskálázzuk, így a tartomány $[0; 0{,}5]$ lesz; a transzformáció determinisztikus, ezért önmagában nem igényel seedet. (c) Ismétlés: a CNN-t három különböző seeddel újratanítjuk, és mindegyik modellt kiértékeljük az eredeti és az átskálázott teszten; a páros különbségek átlagát és szórását (vagy bootstrap konfidenciaintervallumát) adjuk meg, mert a szórás a tanítás seed-érzékenységéből származik, nem a determinisztikus skálázásból. (d) Érvényességi korlát: a GlyphLab szintetikus adat, ezért az eredmény nem általánosítható természetes képekre; az intenzitás-skálázás nem azonos egy valódi domain shift minden összetevőjével (például a hátterek, textúrák és zaj változása nincs benne), így a mérés csak ezt az egy shift-dimenziót izolálja.

20.3.8 Tipikus tévedés

A leggyakoribb tévedés: **„A letölthető súly automatikusan minden célra engedélyezett modell."** Ez hamis. A súlyok letöltése nem jogosít fel semmilyen felhasználásra; a licencet a terjesztő határozza meg. Például a torchvision súlyaihoz tartozó licencfeltételeket a [D13] modellkatalógus és a súlyok eredeti forrása alapján kell ellenőrizni; ezek nem mindig azonosak a modell kutatási cikkének licencével. A [P06] ResNet esetében az általunk elérhető arXiv-változatban nem találtunk licencszekciót; a súlylicenceket ettől függetlenül, külön kell ellenőrizni. Ha valaki

kereskedelmi termékben használja a súlyokat anélkül, hogy tisztázta volna a licencfeltételeket, a felhasználás jogi és etikai kockázatot hordoz. A helyes eljárás: a súlycsomag mellékelt `LICENSE` fájlját vagy a terjesztő weboldalán feltüntetett feltételeket kell alapul venni, és rögzíteni az eredetjegyzékben.

20.3.9 Önellenőrzés és visszautalás

Ellenőrizze: (1) a bemeneti tensor alakja NCHW, dtype float32, tartomány [0,1]; (2) ha a szerződés normalizálást ír elő, akkor a mean és std vektorok csatornánkénti sorrendje megegyezik a súlyok tanításakor használttal; (3) a licencre vonatkozó állításokat csak forrás alapján tegye, és a hiányzó ellenőrzéseket jelölje meg. Visszautalás a 20.2. Finomhangolás és tanulási ráta alfejezetre: a `state_dict` betöltésekor a `weights_only=True` opció csökkenti a deszerializációs kockázatot, de nem oldja meg a preprocessing problémát; a BatchNorm-pufferek eval módban a tanítás alatt frissített running statisztikákat használják.

20.3.10 Rövid összefoglaló

A preprocessing-szerződés rögzíti a bemeneti képek geometriai méretét, normalizálását, csatornarendjét és értéktartományát. A rossz normalizálás és az azonos elemszámú, de hibás csatornarend konkrét ellenpéldákkal igazolhatóan megváltoztatja a modell kimenetét. A domain shift a forrás- és céleloszlás eltérését jelenti, és a teljesítmény romlásához vezethet; a helyes előfeldolgozás ezt nem zárja ki, ezért a target teljesítményt külön, a céladaton kell mérni. A súlycsomag enumja azonosítja a súlyváltozatot, a licenc pedig a felhasználás feltételeit; a letölthető súly nem automatikusan engedélyez minden célra. A labor feladata a GlyphLab adatokra az eredetjegyzék elkészítése; a külső súlyok licencellenőrzését forrásalapú feladatként jelöltük meg.

20.3.11 Következő alkalmazás és források

A következő egység 20.4. V. ellenőrző pont: indokolt CNN-választás az V. ellenőrző pont: ott a teljes CNN-választást indokolni kell, a preprocessing-szerződést, a finomhangolási stratégiát és a licencet együttesen kell értékelni. A licencfeltételt a használat előtt ellenőrizni kell.

Források:

- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30. – state_dict, eval és weights_only.
- [D13] PyTorch contributors (2026). *TorchVision models.* PyTorch dokumentáció. https://docs.pytorch.org/vision/stable/models.html. Hozzáférés: 2026-10-30. – modellkatalógus.
- [P06] He, K., Zhang, X., Ren, S. és Sun, J. (2016). *Deep Residual Learning for Image Recognition.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016); a preprint 2015-ben jelent meg arXiv:1512.03385 számon. https://arxiv.org/abs/1512.03385. – reziduális blokk, 3.1–3.2 szakasz.

20.4 V. ellenőrző pont: indokolt CNN-választás

20.4.1 Cél, előismeret és induló kérdés

Ez az V. ellenőrző pont. A hallgató a 20.1.–20.3. egységekben megismerte a feature extractor és az új fej, a finomhangolás, valamint a preprocessing- és licencszerződés fogalmát. A 19.4. ConvNeXt és a recept/architektúra elválasztása a recept és az architektúra szétválasztását adta, a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése a CNN-operátor teljes specifikációját, a 8.4. II. ellenőrző pont: korrekt tanulási állítás pedig a korrekt tanulási állítás keretét. Itt nem új réteget vezetünk be: a feladat az, hogy a három döntési dimenzió — architektúracsalád (classic/residual/efficient), tanítási mód (scratch/transfer) és erőforráskeret — együtt egyetlen, lektorálható architektúra-döntési lapba álljon össze.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt (mind az öt dimenzióban legalább 3-as szint) és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg; itt csak az eltérés: a riport helyett architektúra-döntési lapot adunk le, és a súlypont a D5 (az állítás és korlátai) dimenzió.

Induló kérdés: a GlyphLab adat (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) 360 képet tartalmaz, 24×24-es, egycsatornás; a feladat három vonalorientáció (függőleges, vízszintes, átlós) osztályozása. A 360 kép 180 jelenet két nézete, ezért a független mintavételi egység a `scene_id`, nem a kép. Válasszon a feladatra architektúrát és tanítási módot. Melyik döntés igényel több paramétert: egy VGG-szerű classic háló vagy egy kis saját CNN? Milyen adat- és erőforrásfeltétel zárja ki az előtanított ResNet18-at az alapforgatókönyvből? A becslést a szakasz végén a döntési lappal ellenőrizzük.

20.4.2 Döntési dimenziók és jelölések

A választási feladatot három szemponthalmaz feszíti ki.

Architektúracsalád. A *classic* jelző itt a VGG-stílusú, egymásra rakott konvolúciós blokkokat jelöli: `Conv → ReLU → Pooling` ismétlése, skip nélkül. A *residual* család a [P06]-ban bevezetett $y = F(x) + x$ identitás-utat alkalmazza; ennek egyszerű, kétszer 3×3-as basic blokkját a 18.1. Reziduális blokk és identitásút már tárgyalta. Az *efficient* család depthwise separable és inverted residual blokkokat használ; a paraméterszám csökkentésének levezetése a 19.1. Depthwise separable és pointwise réteg alfejezetben szerepelt. A családok nem rangsorolhatók abszolút értelemben: a 19.4. ConvNeXt és a recept/architektúra elválasztása alfejezetben látott matched-recept elv éppen azt mondja ki, hogy a kísérleti összevetés csak akkor értelmes, ha a tanítási receptet azonos szinten tartjuk.

Tanítási mód. A *scratch* mód véletlen inicializálással indítja az összes súlyt. A *feature extractor* mód egy előtanított modell konvolúciós részét fagyasztja, és csak az új fejet tanítja. A *finomhangolás* ezen felül a konvolúciós súlyok egy részét vagy egészét is kis tanulási rátával frissíti. A három mód a 20.1.–20.2. alfejezetben bevezetett fogalom.

Erőforráskeret és protokoll. A döntéshez rögzíteni kell: a paraméterszámot (P), a konvolúciós szorzás-összeadás párok számát (MAC), az aktivációmemória nagyságrendjét, a rendelkezésre álló hardvert, az adat splitjét és a modellválasztási szabályt. A protokoll alapszabálya a 8.4. II. ellenőrző pont: korrekt tanulási állítás alfejezetből: a validációs loss választ best checkpointot, a testet pedig csak a választás után, egyszer értékeljük.

A képbatch alakja $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, a logité $Z \in \mathbb{R}^{N \times K_{\text{cls}}}$, a címke $y \in \{0,1,2\}^N$ a GlyphLab három osztályára. A konvolúciós súly alakja $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, hacsak a `groups` paraméter mást nem mond.

Jelöléstábla. A szakasz képletei a következő jelöléseket használják:

Jelölés	Jelentés
N	batchméret
C_{in}, C_{out}, C	bemeneti, kimeneti, illetve azonos be- és kimeneti csatornaszám
H, W	térbeli magasság és szélesség
k, k_h, k_w	konvolúciós kernel mérete (négyzetes, illetve magasság és szélesség szerint)
K	konvolúciós súlytenzor, $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$
K_{cls}	osztályszám (a logitok dimenziója)
P	modell paramétereinek száma
M	konvolúciós MAC-ok száma egy batchre
p_{pad}	konvolúciós padding
X, Z, y	bemeneti batch, logitkimenet, címke

A K tehát itt a súlytenzor, nem az osztályszám; a két mennyiséget a K_{cls} jelölés különbözteti meg. A P a paraméterszám, a paddinget p_{pad} jelöli.

20.4.3 Analitikusan ellenőrzött blokk: a basic residual blokk

A levezetési kötelezettséghez egy olyan blokkot választunk, amelynek a paraméterszáma és műveletigénye zárt formában ellenőrizhető. A [P06] 3.1–3.2 szakasza alapján a basic residual blokk alapformája $y = F(x) + x$. A [P06] eredeti blokkja két 3×3-as konvolúció után batch normalizationt, és az összeadás után ReLU-t is tartalmaz; az itt bemutatott blokk ezeket elhagyja (egyszerűsített változat), hogy a képlet csak a domináns konvolúciós költséget tartalmazza. A bemutatott F tehát két, bias nélküli $k \times k$-s konvolúcióból és egy közbeékelt ReLU-ból áll.

Definíció (egyszerűsített basic block). Legyen $X \in \mathbb{R}^{N \times C \times H \times W}$. A blokk

$$Y = F(X) + X, \qquad F(X) = \mathrm{conv}_{W_2}(\sigma(\mathrm{conv}_{W_1}(X))),$$

ahol $W_1, W_2 \in \mathbb{R}^{C \times C \times k \times k}$, a két konvolúció stride-ja 1, paddingje $p_{\text{pad}} = (k-1)/2$, dilationje 1, groups paramétere 1, és k páratlan. A σ a ReLU, amely csak az első konvolúció után áll (egyetlen közbeékelt ReLU); a második konvolúció után az összeadásig nincs nemlinearitás, ezért a főág kimenete negatív is lehet. A kód ezzel egyezik: `conv2(relu(conv1(x))) + x`. Ha egy változat az összeadás után még egy ReLU-t is alkalmaz, azt külön kell jelölni. Mivel $p_{\text{pad}} = (k-1)/2$ és $s = 1$, a 14.1. Stride, padding, dilation és outputméret alfejezetből ismert kimeneti méretképlet szerint a térbeli méret $H \times W$ marad, tehát az identitás-összeadás X-szel alakhelyes.

Állítás (paraméterszám és konvolúciós MAC, saját levezetés). A fenti blokkra

$$P = 2C^2k^2, \qquad M = 2NHWC^2k^2.$$

Itt P a modellsúlyok száma, M pedig a konvolúciós szorzás-összeadás párok (multiply-accumulate) domináns száma egy bemeneti batchre. Az állítás feltétele: `stride=1`, `padding=(k-1)/2`,

`bias=False`, `groups=1`, `dilation=1`, és a bemeneti csatornaszám C azonos a kimenetivel. Az identitás-összeadás $NHWC$ elemi összeadást, az egyetlen ReLU $NHWC$ elemi maximumképzést ad; ezeket külön számoljuk, és a nagy C, k tartományban alacsonyabb rendűek.

Bizonyítás. Az első súlytensor elemeinek száma $C \cdot C \cdot k \cdot k = C^2k^2$; a másodiké ugyanannyi. Bias nincs, ezért $P = 2C^2k^2$. Egy konvolúcióban minden kimeneti elemhez Ck^2 szorzás és $Ck^2 - 1$ összeadás tartozik, mert az akkumulációs lánc első tagjához nem adódik még egy tag; az így elmaradó $NHWC$ darab összeadás $O(NHWC)$ rendű, ezért a MAC-becslés a domináns Ck^2 tagot használja. A kimeneti elemek száma konvolúciónként $N \cdot C \cdot H \cdot W$, tehát egy konvolúció $NHWC \cdot Ck^2 = NHWC^2k^2$ szorzás-összeadás párt végez. Két konvolúcióra ez $M = 2NHWC^2k^2$.

20.4.4 Kézi számpélda és minimális kontroll

Legyen $N = 1$, $C = 2$, $k = 3$, $H = W = 4$. Ekkor

$$P = 2 \cdot 2^2 \cdot 3^2 = 72, \qquad M = 2 \cdot 1 \cdot 4 \cdot 4 \cdot 2^2 \cdot 3^2 = 1152.$$

Az első konvolúció kimenete $1 \times 2 \times 4 \times 4$, tehát $2 \cdot 16 = 32$ elemet tartalmaz; minden elemhez $Ck^2 = 2 \cdot 9 = 18$ MAC tartozik, ami $32 \cdot 18 = 576$ műveletpár. A második konvolúció ugyanennyit ad, így az összeg 1152. A skip-összeadás további $NHWC = 32$ elemi összeadást jelent, amit a domináns MAC-képlet expliciten nem tartalmaz.

A képlet és a PyTorch-implementáció egyezését az alábbi kontroll ellenőrzi. A kód futtatva; a paraméterszámot a beépített `assert` is ellenőrzi.

```
# lab/n080_residual_check.py
import torch

torch.manual_seed(20261020)

class BasicResidualBlock(torch.nn.Module):
    def __init__(self, C, k):
        super().__init__()
        assert k % 2 == 1
        self.conv1 = torch.nn.Conv2d(C, C, k, stride=1,
                                     padding=(k - 1) // 2, bias=False)
        self.conv2 = torch.nn.Conv2d(C, C, k, stride=1,
                                     padding=(k - 1) // 2, bias=False)

    def forward(self, x):
        return self.conv2(torch.relu(self.conv1(x))) + x

model = BasicResidualBlock(C=2, k=3)
n_params = sum(p.numel() for p in model.parameters())
assert n_params == 72, n_params
x = torch.randn(1, 2, 4, 4)
with torch.no_grad():
    y = model(x)
print("params:", n_params, "output shape:", tuple(y.shape))
assert y.shape == (1, 2, 4, 4)
```

Futtatás: `python lab/n080_residual_check.py`. A futtatott kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()=10`, `seed=20261020`):

```
params: 72 output shape: (1, 2, 4, 4)
```

A shape-ellenőrzés a paddingfeltételből következik: a kimenet $1 \times 2 \times 4 \times 4$, mert a két konvolúció stride-ja 1 és paddingje 1.

20.4.5 Labor: architektúra-döntési lap és kis futó kontroll

Előfeltétel. Python 3.12.3, NumPy 2.5.4 és PyTorch 2.15.0+cpu a `cnn-labor` csomag rögzített környezetéből (`uv sync`), CPU; a labor a szakasz kódját használja, a szükséges csomagokat a laborcsomag rögzített környezete biztosítja. A `lab/n080_decision_check.py` futtatható (`python lab/n080_decision_check.py`); a futó kontroll a saját `TinyGlyphCNN` modellen fut, külső adatfájl nem kell.

A labor elvárt eredménye egy kitöltött döntési lap, három elvetett alternatíva indokával, valamint a kiválasztott modell kis futó kontrollja. A döntési lap a GlyphLab adatszerződésre épül: train 240, val 60, test 60 kép, független mintavételi egység a `scene_id`, bemenet float32 NCHW $[0, 1]$, cél int64 osztályindex.

Döntési lap.

- *Adat:* GlyphLab; train/val/test rögzített, scene-szintű split; az augmentáció címkemegőrző változatai csak a 22.2. Címkemegőrző transzformációk alfejezetben tárgyalt feltételekkel használhatók.
- *Bemeneti szerződés:* $X \in \mathbb{R}^{N \times 1 \times 24 \times 24}$, float32, $[0, 1]$.
- *Cél:* 3-osztályos orientációfelismerés.
- *Erőforrás:* kis helyi CPU, külső súlyletöltés nélkül; a seed `20261020`.
- *Választott változat:* saját, scratch, 4155 paraméteres kis CNN, `TinyGlyphCNN` néven: két classic blokk `(8,8)` és `(16,16)` csatornával, 3×3-as, padding 1-es bias nélküli konvolúciókkal, ReLU-val, 2×2-es maxpoolinggal, majd global average poolinggal és egy `Linear(16, 3)` fejjel.
- *Választási szabály:* a best checkpointot a val loss választja; a testet csak a protokoll végén, egyszer értékeljük.

A kiválasztott modell paraméterszáma pontosan:

$$P_{\text{Tiny}} = 1 \cdot 8 \cdot 3^2 + 8 \cdot 8 \cdot 3^2 + 8 \cdot 16 \cdot 3^2 + 16 \cdot 16 \cdot 3^2 + (16 \cdot 3 + 3) = 72 + 576 + 1152 + 2304 + 51 = 4155.$$

A három elvetett alternatíva:

1. **Nagy classic háló.** Négy blokk $[64, 64, 128, 128]$ kimeneti csatornaszámmal, blokkonként két 3×3-as, bias nélküli konvolúcióval. A nyolc konvolúció súlyainak összege (a blokkok bemenete rendre $1, 64, 64, 128$):

   $1{\cdot}64{\cdot}9 + 64{\cdot}64{\cdot}9 + 64{\cdot}64{\cdot}9 + 64{\cdot}64{\cdot}9 + 64{\cdot}128{\cdot}9 + 128{\cdot}128{\cdot}9 + 128{\cdot}128{\cdot}9 + 128{\cdot}128{\cdot}9 = 576 + 3{\cdot}36\,864 + 73\,728 + 3{\cdot}1$

   A konvolúciós paraméterszám tehát kb. 0,63 millió; a további rétegek (normalizálás, fej) ezt érdemben nem növelik. Indok az elvetésre: a 240 train képhez képest a paraméterszám túl nagy; a finite-H korlát (véges hipotézisosztály mellett az általánosítási hiba nem csökkenthető korlátlanul) 8.4. II. ellenőrző pont: korrekt tanulási állítás szerint a nagyobb hipotézisosztály szélesebb általánosítási sávot (az általánosítási hiba tágabb lehetséges tartományát) ad, és a kis mintán a túlillesztés kockázata dominál. A 24×24 inputon a nagy recepciós mező sem indokolt.

2. **Előtanított ResNet18 feature extractor új fejjel.** Indok az elvetésre: ez külső súlyletöltést igényel, ami a fejezet főútjában csak külön, dokumentált opció lehet. A [D13] szerint a konkrét TorchVision súlyváltozatot, annak preprocessing-szerződését és licencfeltételét a használat előtt ellenőrizni kell; az ImageNet-domain és a 224×224-es RGB bemenet áthidalása a 24×24-es grayscale GlyphLab-ra a 20.3. Preprocessing, domain shift és súlylicenc alfejezetben tárgyalt domain shiftet hozza be. Az alapdöntésben ezért nem választjuk.
3. **Nagy efficient blokk.** Egy MBConv-szerű blokk expand=6, $k = 5$ mellett, $C = 16$ bemeneti és kimeneti csatornával. A paraméterszám bias nélkül: az expand $16 \cdot 96 = 1536$, a depthwise $96 \cdot 25 = 2400$, a projekció $96 \cdot 16 = 1536$, összesen 5472. A 24×24-es bemeneten ez $24 \cdot 24 \cdot 5472 = 3\,151\,872$ MAC; ugyanezen a bemeneten a `TinyGlyphCNN` négy konvolúciója 870 912 MAC, tehát az efficient blokk önmagában több paramétert és kb. 3,6-szor annyi MAC-ot hoz. Indok az elvetésre: a 19.1. Depthwise separable és pointwise réteg alfejezetből ismert paramétercsökkentés előnye nagy csatornaszámnál és nagy térbeli méretnél jelentős; a 24×24-es, egycsatornás feladaton a megtakarítás nem indokolja a nagyobb kapacitást és a bonyolultabb normalizálási állapotot.

A választott modell futó kontrollja:

```python
# lab/n080_decision_check.py
import torch

torch.manual_seed(20261020)

class TinyGlyphCNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.features = torch.nn.Sequential(
            torch.nn.Conv2d(1, 8, 3, padding=1, bias=False),
            torch.nn.ReLU(),
            torch.nn.Conv2d(8, 8, 3, padding=1, bias=False),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
            torch.nn.Conv2d(8, 16, 3, padding=1, bias=False),
            torch.nn.ReLU(),
            torch.nn.Conv2d(16, 16, 3, padding=1, bias=False),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
        )
        self.gap = torch.nn.AdaptiveAvgPool2d((1, 1))
        self.head = torch.nn.Linear(16, 3, bias=True)

    def forward(self, x):
        z = self.features(x)
        z = self.gap(z).flatten(1)
        return self.head(z)

model = TinyGlyphCNN()
n_params = sum(p.numel() for p in model.parameters())
assert n_params == 4155, n_params
x = torch.rand(2, 1, 24, 24, dtype=torch.float32)
with torch.no_grad():
    logits = model(x)
print("params:", n_params)
print("logits shape:", tuple(logits.shape))
assert logits.shape == (2, 3)
```

Futtatás: `python lab/n080_decision_check.py`. A futtatott kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()=10`, `seed=20261020`):

```
params: 4155
logits shape: (2, 3)
```

A 4155 értéket a képlet és a kód `assert`-je egymástól függetlenül ellenőrzi.

20.4.6 Empirikusan vizsgált összehasonlítási protokoll

A döntési lapot egy kis empirikus összehasonlítással lehet validálni. Az alábbi protokoll a hallgatói futtatás előírása, nem a fejezet saját mérése; a könyv első laborjának (a GlyphLab-adat ellenőrzött jelentése, A. A GlyphLab adatkészlet) eredményétől független, új futásnak számít.

- **Kérdés.** A választott 4155 paraméteres `TinyGlyphCNN` val loss-ában jobb-e, mint egy nagyobb kapacitású változat?
- **Hipotézis.** A nagyobb modell nem javítja a val loss-t, mert a train halmaz 240 kép, és a kapacitás növelése a túlillesztés kockázatát növeli.
- **Adat és split.** GlyphLab, scene-szintű train/val/test; a val a best checkpoint kiválasztására, a test csak a végső értékelésre szolgál.
- **Megfigyelési egység.** `scene_id`; egy jelenet két nézete (view_id 0 és 1) ugyanabba a splitbe tartozik.
- **Baseline.** `TinyGlyphCNN`, scratch, Adam, `lr=1e-3`, batch 16, seed `20261020`.
- **Változó tényező.** A második blokk mindkét konvolúciójának kimeneti csatornaszáma, C_2: $C_2 = 16$ (baseline) és $C_2 = 64$ (nagy kapacitású változat). Így a blokk első konvolúciója $8 \to C_2$, a második $C_2 \to C_2$, a fej bemenete pedig C_2. Az összes többi beállítás azonos.
- **Metrika.** Val loss a checkpointválasztásra; a végső modellre test accuracy, valamint a clean és a `data/corrupted_test.npy` ugyanazon 60 képén mért accuracy páros különbsége. A corrupt fájl sorrendje a `metadata.csv` test sorainak sorrendjét követi, ezért a két predikció képenként párosítható; a corrupt tesztet sem használjuk modellválasztásra.
- **Erőforrás.** CPU, float32, maximum 30 epoch.
- **Érvényességi korlát.** A GlyphLab nem természetes képi benchmark; az eredmény csak a rögzített adatra és protokollra vonatkozik.

Referenciakód. A tanítási ciklus, a validációs loss szerinti checkpointválasztás és az egyszeri clean–corrupt teszt-kiértékelés referenciakódja a laborcsomag `lab/train_cnn.py` fájlja: a ciklus a `train()` függvényben van, a legjobb állapotot `best_state` őrzi, a tesztkiértékelés pedig a `clean` és `corrupt` szótárakat adja. A protokollhoz ebben a szkriptben a `SmallCNN` osztályt kell a `TinyGlyphCNN`-re cserélni, a második blokk csatornaszámát C_2 paraméterként átadni, a modell paraméterszámára vonatkozó `assert`-et a `TinyGlyphCNN` értékére állítani, a hiperparamétereket pedig a fenti protokoll szerint beállítani (Adam, `lr=1e-3`, batch 16, legfeljebb 30 epoch). A fejezet a futtatás eredményét nem közli; a hallgató a saját futását naplózza.

20.4.7 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a scratch, feature extractor és finomhangolás tanítási módot. Adjon egy-egy feltételt, amely mellett a GlyphLab-on a scratch, illetve a feature extractor értelmes választás. Elvárt eredmény: a három definíció, a frissített súlyok köre, és két indoklás.

F2 (levezetési/bizonyítási, nehézség 2). Adott egy depthwise separable blokk: bemenet $N \times C \times H \times W$, depthwise konvolúció $k \times k$-val, `groups=C`, `bias=False`, `stride=1`, `padding=(k-1)/2`, majd pointwise 1×1-es konvolúció C kimeneti csatornával, `bias=False`. Vezesse le a P_{ds} paraméterszámot és az M_{ds} konvolúciós MAC-ot. Hasonlítsa össze a basic residual blokkal $C = 8$, $k = 3$ mellett. Elvárt: képletek, arány, feltételek.

F3 (implementációs, nehézség 2). Írjon `count_tiny(c1, c2)` függvényt, amely a döntési lap modelljének paraméterszámát adja vissza a két blokk csatornaszámából, és egészítse ki a `lab/n080_decision_check.py` kontrollt egy független képletalapú `assert`-tel. Elvárt: futtatható kód, a `4155` eset helyes visszaadása.

F4 (hibakeresési, nehézség 2). Egy döntési napló a legnagyobb modellt választotta, mert annak test accuracyja 0,01-gyel magasabb volt a baseline-nál. Mi a hiba, mi a következménye, és hogyan kell javítani? Elvárt: az ok megnevezése, a választási torzítás magyarázata és a javított munkafolyamat.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen összehasonlítást a GlyphLab-on a scratch `TinyGlyphCNN` és egy külső előtanított ResNet18 feature extractor között. A terv tartalmazza a preprocessing-szerződést, a licencellenőrzést, a megfigyelési egységet, a baseline-t, a metrikát, az előre rögzített döntési szabályt és az érvényességi korlátokat. Ha a transzfer hatását izolálni akarja, vegyen fel véletlen inicializálású ResNet18 kontrollt ugyanazzal a preprocessinggel. A külső súly itt opcionális; a [D13] állapota miatt a konkrét súlyváltozatot további ellenőrzéshez kösse. A beadandó terv kötelező elemei: hipotézis, a változó tényező, kontroll (baseline), metrika és megfigyelési egység, a seedek száma és konkrét értékei, számszerű döntési szabály, valamint erőforráskeret.

20.4.8 Megoldások és indoklás

F1 megoldás. A scratch mód minden súlyt véletlen inicializálásból indít, és a teljes modellt tanítja. A feature extractor az előtanított modell konvolúciós részét fagyasztja; csak az új fej súlyai frissülnek. A finomhangolás a konvolúciós rész egy részét vagy egészét is frissíti, jellemzően kisebb tanulási rátával. A GlyphLab-on a scratch értelmes, mert a feladat kicsi, a bemenet 24×24-es grayscale, és nincs szükség külső domain reprezentációjára. A feature extractor akkor lenne értelmes, ha a külső modell bemeneti szerződése és licencfeltétele ellenőrzött, és a forrás-domain reprezentációja a célfeladatra is hasznos; a jelen adaton ez nem teljesül automatikusan.

F2 megoldás. A depthwise konvolúció súlyalakja $C \times 1 \times k \times k$, `groups=C`, tehát $P_d = Ck^2$. A pointwise súlyalakja $C \times C \times 1 \times 1$, tehát $P_p = C^2$. Így

$$P_{ds} = Ck^2 + C^2.$$

A depthwise konvolúció minden kimeneti elemhez k^2 MAC-ot használ, a kimeneti elemek száma $NCHW$, tehát $M_d = NHWCk^2$. A pointwise konvolúció minden kimeneti elemhez C MAC-ot használ, tehát $M_p = NHWC^2$. Összesen

$$M_{ds} = NHWC(k^2 + C).$$

A basic blokk $P_b = 2C^2k^2$ és $M_b = 2NHWC^2k^2$. Az összevetés alapja: egy blokk, azonos be- és kimeneti csatornaszámmal, azonos térbeli mérettel, bias nélkül, stride 1 és azonos padding mellett; a basic blokk ReLU-ja és skip-összeadása $O(NHWC)$ járulék, a depthwise blokk normalizálását és aktivációját itt elhagyjuk. Az arány ezért a konvolúciós súlyokra és MAC-okra vonatkozik, nem két teljes architektúrára. $C = 8$, $k = 3$ esetén $P_{ds} = 8 \cdot 9 + 64 = 136$, $P_b = 2 \cdot 64 \cdot 9 = 1152$, az arány $136/1152 \approx 0{,}118$, tehát a paraméterszám kb. 88%-kal kisebb. A MAC-arány ugyanez, mert a térbeli és batchtényező kiesik.

F3 megoldás.

```python
def count_tiny(c1, c2, k=3, classes=3):
    p1 = 1 * c1 * k * k + c1 * c1 * k * k
    p2 = c1 * c2 * k * k + c2 * c2 * k * k
    head = c2 * classes + classes
    return p1 + p2 + head

assert count_tiny(8, 16) == 4155
```

A függvény a két blokk konvolúciós súlyait és a lineáris fej súlyát, illetve biasát adja össze; a k és az osztályszám paraméter, ezért a képlet nem tartalmaz beégetett 9-et vagy 3-at. A `c1=8, c2=16` eset a döntési lap értékét adja vissza. A `lab/n080_decision_check.py` kontrollba a függvényt a szkript elejére felvéve a modell létrehozása után a következő független ellenőrzés kerül:

```python
n_params = sum(p.numel() for p in model.parameters())
assert count_tiny(8, 16) == n_params
```

Így a képlet és a PyTorch paraméterszámlálása egymástól függetlenül egyezik a 4155 értéken.

F4 megoldás. A hiba: a test halmazt használták modellválasztásra. Következmény: a test már nem független; a kiválasztási iteráció a testre illeszti a választást, ezért a mért test accuracy felfelé torzít, és a valós általánosítási hibát nem becsli. Javítás: a választást a val loss vagy val accuracy alapján kell elvégezni; a testet csak a rögzített, végleges modellre szabad egyszer futtatni. Ellenőrzés: a naplóban a választási oszlop val-metrikát tartalmazzon, és a test oszlop a választás előtt üres legyen.

F5 megoldásminta. Kérdés: a külső ResNet18 feature extractor javítja-e a test accuracyt a scratch `TinyGlyphCNN`-hez képest? A két kar architektúrában, bemeneti szerződésben és inicializálásban is eltér, ezért a puszta összehasonlítás nem izolálja a transzfer hatását. A matched-recipe elv a tanítási receptre (optimizer, tanulási ráta, epochszám, augmentáció, batchméret) és az adat/split kezelésére vonatkozik; a bemeneti felbontás és a csatornaszám a modellek szerződése, ezért ezeket nem lehet azonosítani. A transzferhatás izolálásához három kar kell: (1) baseline: scratch `TinyGlyphCNN`, 24×24 grayscale; (2) előtanított ResNet18 feature extractor, 224×224 RGB (a grayscale kép három csatornára ismételve), az előírt ImageNet-normalizálással; (3) kontroll: véletlen inicializálású ResNet18 (`weights=None`) ugyanazzal a 224×224 RGB preprocessinggel és ugyanazzal a fejjel, szintén feature extractor módban. A (2) − (3) különbség az előtanított súlyok hatását méri; a (1) és a (2)/(3) összevetése rendszerszintű döntés, amelyben az architektúra és a felbontás is szerepel. Protokoll: azonos tanítási recept mindhárom karon (Adam, `lr=1e-3`, 30 epoch, batch 16, augmentáció nélkül), azonos scene-szintű split és 3 seed; a checkpointot a mentett állapotok közül a val loss választja [D09]. Döntési szabály: a ResNet18-at csak akkor választjuk a `TinyGlyphCNN` helyett, ha (a) a (2) val accuracyje jobb az (1)-nél, és a (2)−(1) páros, jelenetszintű bootstrap 95%-os CI-je 0 fölött van, valamint (b) a (2)−(3) különbség legalább 5 százalékpont, és CI-je szintén 0 fölött van. A tesztet egyszer, a végleges modellen értékeljük. Licenc: a TorchVision súly letöltési és felhasználási feltételeit külön dokumentálni kell [D13]. Érvényességi korlátok: a 60 képes val halmazon 5 százalékpont 3 kép, ezért a küszöb zajérzékeny; a GlyphLab szintetikus, az eredmény nem általánosítható természetes képekre; a 224×224-re nagyítás és a csatornaismétlés új preprocessing-shiftet visz be, ezért a (2) és (3) kar is csak ezen a szerződésen belül értelmezhető; a [D13] katalógus állapota miatt a konkrét súlyváltozat ellenőrzése a kísérlet előfeltétele.

20.4.9 Tipikus tévedés, önellenőrzés és visszautalás

Tipikus tévedés. „A legnagyobb elérhető modell nem automatikusan a legjobb feladatmegoldás.” Ennek ellenpéldája egy egydimenziós (1D-s) regressziós szerkesztett eset: legyen a train három pontja $(-1, 0), (0, 1), (1, 2)$, a generáló összefüggés $f(x) = x + 1$. A lineáris modell $b = 1, w = 1$

paraméterekkel tökéletesen illeszkedik a train pontokra. A nagy kapacitású, hetedfokú polinom $p_7(x) = x + 1 + 100(x+1)x(x-1)(x^2-4)(x^2-9)$ szintén átmegy mindhárom train ponton, mert a szorzótényező ott nulla; a tesztpontban, például $x = 0{,}5$-nél azonban a predikció 1,5-höz képest $100(1{,}5)(0{,}5)(-0{,}5)(-3{,}75)(-8{,}75) = -1230{,}47$-tel tolódik el, azaz $p_7(0{,}5) \approx -1228{,}97$, durván téved. A nagyobb modell tehát azonos train illeszkedés mellett rosszabb tesztpredikciót adhat. A polinomos példa illusztráció, nem bizonyíték a CNN viselkedésére; a CNN-választásnál ugyanezt a jelenséget a finite-H korlát 8.4. II. ellenőrző pont: korrekt tanulási állítás fogalmazza meg: a nagyobb hipotézisosztály szélesebb általánosítási sávot nyit.

Önellenőrzés. A döntési lap akkor teljes, ha (1) a bemeneti szerződés és a normalizálás rögzített; (2) a split scene-szintű, és a testet nem érinti a választás; (3) a paraméterszám és a MAC képlete a megadott `bias`, `groups`, `stride`, `padding` feltételekkel egyezik; (4) külső súly esetén a verzió, preprocessing és licenc forrása azonosított; (5) a végső következtetés csak a rögzített protokollra vonatkozik.

Visszautalás. A 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetből: a cross-correlation alak- és gradientszabálya a döntési lap minden konvolúciós sorában érvényes; az eltolási equivariance a stride és a határfeltétel függvénye. A 19.4. ConvNeXt és a recept/architektúra elválasztása alfejezetből: az architektúra címkéje nem örök rangsor, hanem adott receptfeltételhez kötött állítás.

20.4.10 Rövid összefoglaló

Az architektúra-döntési lap a családot (classic/residual/efficient), a tanítási módot (scratch/transfer) és az erőforráskeretet együtt rögzíti. Az analitikusan ellenőrzött basic residual blokk paraméterszáma $2C^2k^2$, konvolúciós MAC-ja $2NHWC^2k^2$; a depthwise separable blokk ennél $C = 8, k = 3$ mellett kb. 88%-kal kevesebb paramétert használ. A GlyphLab-on a 4155 paraméteres scratch `TinyGlyphCNN` a döntési lap választása; a nagy classic, az előtanított ResNet18 és a nagy efficient blokk elvetésének oka rendre a túl nagy kapacitás, a domain shift és a licencellenőrzés, illetve a kis bemeneten elmaradó előny. A legnagyobb modell nem automatikusan a legjobb.

20.4.11 Következő alkalmazás és felhasznált források

A következő egység 21.1. Inicializálás és varianciaterjedés az inicializálást és a varianciaterjedést tárgyalja; ott a döntési lapon kiválasztott kis CNN súlyainak induló eloszlása közvetlenül befolyásolja a tanítás stabilitását.

- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [D13] PyTorch contributors (2026). *TorchVision models.* PyTorch dokumentáció. https://docs.pytorch.org/vision/stable/models.html. Hozzáférés: 2026-10-30.
- [P06] He, Zhang, Ren és Sun (2015). *Deep Residual Learning for Image Recognition.* https://arxiv.org/abs/1512.03385.

21 Tanítási recept: inicializálás, regularizálás és ütemezés

21.1 Inicializálás és varianciaterjedés

21.1.1 Cél, előismeret és induló kérdés

Ennek az alfejezetnek a célja, hogy a súlyok *inicializálását* ne könyvtári „varázsparaméterként”, hanem becsülhető jelterjedési jelenségként értse meg: hogyan válasszuk meg a kezdeti súlyok szórását úgy, hogy a háló elején a forward aktivációk és a backward gradiensek nagyságrendje ne omoljon össze, illetve ne robbanjon fel. Az előfeltételek közül a 6.4. Momentum és adaptív optimizálók alfejezetből a gradienslépések állapotváltozóit, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetből a numerikus stabilitás iránti érzékenységet, a 12.4. III. ellenőrző pont: önállóan tanuló kis háló alfejezetből a teljes tanítási ciklust, a 18.4. Reziduális blokk ablatív vizsgálata alfejezetből pedig a mélységgel növekvő normaszorzati hatásokat idézzük fel. A rétegenkénti számítást a perceptron affin score-jára, $s(x) = w^\top x + b$-re (9.1. Neuron és perceptron), az architekturális motivációt a LeNet réteges felépítésére (17.1. LeNet és a lokális megosztott reprezentáció), az előtanított és a frissen inicializált súly fogalmi különbségét pedig a 20.1. Feature extractor és új fej alfejezetre építjük.

Induló becslésként gondoljuk végig: ha egy L rétegű háló minden rétege ρ-szorosára változtatja a jel szórását, akkor a bemeneti szórás az L. réteg után közelítőleg ρ^L-szeresére módosul. Amennyiben $\rho = 0{,}9$, ötven réteg után a jel nagyságrendje $0{,}9^{50} \approx 0{,}005$-szorosára csökken; ha $\rho = 1{,}1$, akkor $1{,}1^{50} \approx 117$-szeresére nő. A kérdés tehát az, milyen feltételek mellett lehet ρ-t kezdéskor 1 közelébe állítani, és miért nem jelenti ez automatikusan a teljes tanítási folyamat stabilitását.

21.1.2 Definíciók és intuíció

Fan-in, fan-out. Egy $W \in \mathbb{R}^{m \times d}$ súlyú, $Z = XW^\top + b$ affin rétegben a *fan-in* (bemeneti legyezőszám) a súlymátrix oszlopainak száma, d, a *fan-out* pedig a sorok száma, m. Más szóval a fan-in azt mondja meg, hány bemeneti érték összegződik egyetlen kimeneti egységben, a fan-out pedig azt, hogy egy bemeneti érték hány kimeneti egység hozzájárulásában szerepel. Konvolúciós rétegre a megfelelő érték $C_{in}k_hk_w/g$, illetve $C_{out}k_hk_w/g$, ahol g a csoportok száma (a `groups` argumentum; $g = 1$ a szokásos, csoportosítás nélküli konvolúció), csoportosított ($g > 1$) esetben a megfelelő csoportmérettel; ebben az alfejezetben a levezetést lineáris rétegre végezzük, és a CNN-változatot csak a szóbeli megfeleltetés szintjén említjük. A bias minden kimeneti csatornához egy skalárt ad, ezért alakja m, és a variancia-számításban közvetlenül nem vesz részt, ha kezdetben konstans, például nulla.

iid súlyok. Az inicializálási heurisztikák alapfeltevése, hogy az inicializálásból származó W_{ij} súlyok *független és azonos eloszlású* (independent and identically distributed, iid) valószínűségi változók. Jelöljük közös várható értéküket μ_w-vel, közös varianciájukat σ_w^2-vel. A gyakorlatban használt sémák általában $\mu_w = 0$-t írnak elő; amint a hibakeresési példában látni fogjuk, a nem

szimmetrikus, például $[0, a]$-ra inicializált súly nemcsak a súlyok várható értékét teszi nullától különbözővé, hanem a mélyebb rétegek előaktivációit is eltolja.

Variancia és skála. Egy ξ valószínűségi változó varianciája $\mathrm{Var}(\xi) = E[(\xi - E[\xi])^2]$. Ha $E[\xi] = 0$, akkor ez egyszerűen $E[\xi^2]$. A skálát a szórással, $\sigma = \sqrt{\mathrm{Var}(\xi)}$-vel mérjük. A cél nem az, hogy minden aktiváció pontosan ugyanakkora legyen, hanem az, hogy a rétegek mélységében az *eloszlás* szórása ne tűnjön el exponenciálisan és ne is robbanjon fel.

21.1.3 Varianciaterjedés: az összeg szórása korrelálatlan tagokra

Először a varianciaösszegzés alaplemmáját bizonyítjuk.

Lemma. Legyenek $a_1, \dots, a_n$ véges második momentumú, páronként korrelálatlan valószínűségi változók, azaz $\mathrm{Cov}(a_i, a_j) = 0$, ha $i \neq j$ (ez teljesül például, ha páronként függetlenek). Ekkor

$$\mathrm{Var}\left(\sum_{i=1}^{n} a_i\right) = \sum_{i=1}^{n} \mathrm{Var}(a_i).$$

Bizonyítás. Legyen $\mu_i = E[a_i]$. A variancia definíciója és a várható érték linearitása szerint $\sum_i a_i - E[\sum_i a_i] = \sum_i (a_i - \mu_i)$, ezért

$$\mathrm{Var}\left(\sum_i a_i\right) = E\left[\left(\sum_i (a_i - \mu_i)\right)^2\right] = \sum_{i,j} E[(a_i - \mu_i)(a_j - \mu_j)] = \sum_{i,j} \mathrm{Cov}(a_i, a_j).$$

A diagonális tagok $\mathrm{Cov}(a_i, a_i) = \mathrm{Var}(a_i)$, a többi a feltétel szerint nulla. $\square$

A lemmához **nem** kell zérus átlag: a függetlenség (vagy csak a korrelálatlanság) a nem nulla átlagú tagok kovarianciáit is eltünteti. A zérus átlag ott kap szerepet, ahol a varianciát a második momentummal azonosítjuk: $E[a_i] = 0$ esetén $\mathrm{Var}(a_i) = E[a_i^2]$, általában viszont $E[a_i^2] = \mathrm{Var}(a_i) + \mu_i^2$.

Gyakorlati ellenőrzésként vegyünk három független, azonos eloszlású, $\{-1, +1\}$ értékű diszkrét egyenletes valószínűségi változót. Varianciájuk egyenként 1, összegük $\{-3, -1, 1, 3\}$-on a $1/8, 3/8, 3/8, 1/8$ valószínűségekkel oszlik el, így az összeg második momentuma, egyben (zérus átlag miatt) varianciája $(9 + 3 \cdot 1 + 3 \cdot 1 + 9)/8 = 3$, ami egyenlő a varianciák összegével. Nem zérus átlagú ellenőrzés: három független $\{0, 1\}$ értékű, $1/2$ paraméterű Bernoulli-változó varianciája egyenként $1/4$, az összeg binomiális, varianciája $3 \cdot \frac{1}{4} = \frac{3}{4}$, az átlag $(3/2)$ ellenére is a varianciák összege.

21.1.4 Affin réteg előre irányú varianciája és a fan-in becslés

Tekintsünk egyetlen affin réteget $X \in \mathbb{R}^{N \times d}$ bemenettel. A kimenet $Z = XW^\top + b$, ahol $Z \in \mathbb{R}^{N \times m}$, $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$. Vizsgáljuk egyetlen z_o kimeneti koordinátát:

$$z_o = \sum_{i=1}^{d} W_{oi} x_i + b_o.$$

Tegyük fel, hogy a W_{oi} súlyok iid zérus átlagúak σ_w^2 varianciával, az x_i bemenetek szintén zérus átlagúak σ_x^2 varianciával, és a súlyok függetlenek a bemenettől. Ekkor a $W_{oi} x_i$ szorzat várható értéke $E[W_{oi}]E[x_i] = 0$, varianciája pedig

$$\mathrm{Var}(W_{oi}x_i) = E[W_{oi}^2]E[x_i^2] = \sigma_w^2\sigma_x^2,$$

ahol a szorzatfüggetlenséget használtuk. A lemma szerint az összeg varianciája a tagok varianciáinak összege, így

$$\sigma_z^2 = d\,\sigma_w^2\sigma_x^2 = \mathrm{fan_in}\cdot\sigma_w^2\,\sigma_x^2.$$

Ez a *fan-in becslés*. Ha tehát azt szeretnénk, hogy a réteg kimeneti varianciája egyezzen a bemeneti varianciával, $\sigma_z^2 = \sigma_x^2$, akkor a $\sigma_w^2 = 1/d$ választás adódik. Ez a Xavier/Glorot inicializálás előre irányú motivációja.

21.1.5 Backward irány és a fan-out becslés

A VJP-konvencióban (4.3. Jacobi-mátrix, láncszabály és VJP) az X-re vonatkozó gradiens $\nabla_X L = \nabla_Z L\, W$, ahol $\nabla_Z L \in \mathbb{R}^{N\times m}$ és $W \in \mathbb{R}^{m\times d}$. Egy rögzített bemeneti koordinátára

$$\nabla_{x_i} L = \sum_{o=1}^{m} W_{oi}\,\nabla_{z_o} L.$$

Ha a W_{oi} súlyok iid zérus átlagúak és függetlenek a hátra érkező $\nabla_{z_o} L$ gradiensektől, továbbá azok varianciája σ_{dz}^2, ugyanazzal az érveléssel

$$\sigma_{dx}^2 = m\,\sigma_w^2\sigma_{dz}^2 = \mathrm{fan_out}\cdot\sigma_w^2\,\sigma_{dz}^2.$$

Itt σ_{dz}^2 a z előaktiváció szerinti gradiens varianciája, $\mathrm{Var}(\nabla_z L)$. Ha a réteget ReLU követi, a gyakorlatban számolt $\nabla_z L$ már tartalmazza a ReLU deriváltjának maszkját; szimmetrikus z és a maszktól független, zérus átlagú hátra érkező gradiens mellett ez a maszk átlagosan felezi a gradiens második momentumát, ezért a ReLU-lánc backward célja $\sigma_w^2 = 2/\,\mathrm{fan_out}$ (a ReLU nélküli lineáris esetben $1/\,\mathrm{fan_out}$).

A lineáris esetben a forward kívánalom $\sigma_w^2 = 1/\,\mathrm{fan_in}$, a backward kívánalom $\sigma_w^2 = 1/\,\mathrm{fan_out}$. A kettő egyszerre általában nem teljesíthető, ha $\mathrm{fan_in} \neq \mathrm{fan_out}$. A Xavier-kompromisszum a két *varianciacél* ($1/\,\mathrm{fan_in}$ és $1/\,\mathrm{fan_out}$) harmonikus közepét választja:

$$\sigma_w^2 = \frac{2}{\mathrm{fan_in} + \mathrm{fan_out}}.$$

A harmonikus közép definíciója szerint $\mathrm{HM}(1/\,\mathrm{fan_in},\, 1/\,\mathrm{fan_out}) = 2/(\mathrm{fan_in} + \mathrm{fan_out})$; ez egyben a két fan-szám számtani közepének reciproka, $\big((\mathrm{fan_in} + \mathrm{fan_out})/2\big)^{-1}$. A harmonikus közép tehát a varianciacélokra vonatkozik, nem a fan-számokra. A népszerű uniform megvalósítás ezt úgy kapja, hogy $W_{ij} \sim U(-a, a)$, ahol $U(-a, a)$ varianciája $a^2/3$; az $a^2/3 = 2/(d+m)$ egyenletből

$$a = \sqrt{\frac{6}{\mathrm{fan_in} + \mathrm{fan_out}}}.$$

Ez nem bizonyítás arra, hogy a háló optimális lesz; ez egy skálázási heurisztika, amely a linearitás, a függetlenség és a nulla-átlag feltételei mellett a jel szórásának közelítő megőrzését célozza.

21.1.6 ReLU: a második momentum felezése és a He-skálázás

A ReLU $\mathrm{ReLU}(z) = \max(0, z)$ kimenete nem zérus átlagú, ezért a következő réteg számára nem a varianciája, hanem a **második momentuma** a releváns mennyiség. Legyen a következő réteg előaktivációja $z_{l+1,o} = \sum_{i=1}^{\mathrm{fan_in}} W_{oi} h_{l,i}$, ahol a W_{oi} súlyok iid, zérus átlagúak, $\mathrm{Var}(W) = \sigma_w^2$ szórásnégyzetűek és függetlenek a h_l aktivációktól. Ekkor $E[W_{oi} h_{l,i}] = E[W_{oi}]\,E[h_{l,i}] = 0$, és

$$\mathrm{Var}(W_{oi} h_{l,i}) = E[W_{oi}^2]\,E[h_{l,i}^2] = \sigma_w^2\, q_l, \qquad q_l := E[h_{l,i}^2],$$

a tagok pedig korrelálatlanok, így

$$\mathrm{Var}(z_{l+1}) = \mathrm{fan_in} \cdot \mathrm{Var}(W) \cdot E[h_l^2] = \mathrm{fan_in} \cdot \sigma_w^2\, q_l.$$

Ez a fenti fan-in becslés általánosítása: zérus átlagú bemenetnél $q_l = \sigma_x^2$, és visszakapjuk a $\mathrm{fan_in}\,\sigma_w^2\sigma_x^2$ alakot. Legyen most z_l szimmetrikus, zérus várható értékű, $\sigma_{z,l}^2$ szórásnégyzetű, és $h_l = \max(0, z_l)$. A szimmetria miatt a pozitív és a negatív fél ugyanannyit ad a második momentumhoz, és a ReLU a negatív felet kinullázza:

$$q_l = E[h_l^2] = E[z_l^2\,\mathbf{1}[z_l > 0]] = \frac{1}{2}\,E[z_l^2] = \frac{1}{2}\,\sigma_{z,l}^2.$$

A ReLU tehát a **második momentumot** felezi, nem a varianciát: $\mathrm{Var}(h_l) = q_l - (E[h_l])^2 < q_l$. Standard normális z-re $E[h] = 1/\sqrt{2\pi} \approx 0{,}39894$, $E[h^2] = 0{,}5$ és $\mathrm{Var}(h) = \frac{1}{2} - \frac{1}{2\pi} \approx 0{,}340845$. A két egyenletet összefűzve

$$\mathrm{Var}(z_{l+1}) = \frac{1}{2}\ \mathrm{fan_in} \cdot \sigma_w^2\ \mathrm{Var}(z_l).$$

A $\mathrm{Var}(z_{l+1}) = \mathrm{Var}(z_l)$ célból

$$\sigma_w^2 = \frac{2}{\mathrm{fan_in}},$$

ami a He-inicializálás normális eloszlású változata: $W_{ij} \sim N(0, 2/\,\mathrm{fan_in})$. A megfelelő uniform határ az $a^2/3 = 2/\,\mathrm{fan_in}$ egyenletből $a = \sqrt{6/\,\mathrm{fan_in}}$. A megőrzött mennyiség tehát az előaktivációk varianciája, illetve ezzel egyenértékűen az aktivációk második momentuma rétegről rétegre.

Az elnevezések a klasszikus forrásokból származnak: a Xavier-skálázást Glorot és Bengio (2010), a ReLU-finomítást He, Zhang, Ren és Sun (2015) vezette be. Az itt adott varianciaalapú motiváció önálló levezetés; nem helyettesíti a két eredeti cikk valamennyi módszertani feltételét.

Feltétel-ellenőrzésként a ReLU-érvelésben a szimmetria valódi eloszlásfeltevés. Ha a bemeneti x_i átlaga például $\mu_x \neq 0$, akkor a szorzat varianciája a második momentummal $\sigma_w^2 E[x_i^2] = \sigma_w^2(\mu_x^2 + \sigma_x^2)$, nem $\sigma_w^2\sigma_x^2$; a csak varianciát használó becslés ezért alábecsüli a valódi szórást. A heurisztika tehát csak a kimondott feltételek mellett értelmes.

21.1.7 Kézi kis példa

Tegyük fel, hogy egy réteg fan-inje $d = 8$, fan-outja $m = 4$, a bemenet varianciája $\sigma_x^2 = 3$, és He-féle normál inicializálást használunk: $\sigma_w^2 = 2/8 = 0{,}25$, azaz $\sigma_w = 0{,}5$. A forward becslés szerint

$$\sigma_z^2 = 8 \cdot 0{,}25 \cdot 3 = 6,$$

majd szimmetrikus z-eloszlást feltételezve a ReLU után a második momentum

$$q_h = E[h^2] = \frac{1}{2} \cdot 6 = 3,$$

tehát a következő réteg szempontjából releváns $E[h^2]$ megegyezik a bemenet $E[x^2] = \sigma_x^2 = 3$ értékével. A ReLU-kimenet varianciája ennél kisebb: normális z esetén $\mathrm{Var}(h) = 6\left(\frac{1}{2} - \frac{1}{2\pi}\right) \approx 2{,}045$. Ha az upstream gradiens varianciája $\sigma_{dz}^2 = 1$, a backward becslés

$$\sigma_{dx}^2 = 4 \cdot 0{,}25 \cdot 1 = 1,$$

tehát ebben a konkrét dimenziópárosításban a gradiens skálája is véges marad. A példa azt is mutatja, hogy a He-séma nem teszi a fan-out tényezőt önálló korrekcióvá; a backward stabilitás itt annak köszönhető, hogy a választott m kicsi. Általánosabb dimenziókra a viselkedést méréssel kell ellenőrizni.

Ugyanerre a rétegre a Xavier-uniform változat: $a = \sqrt{6/(8+4)} = 1/\sqrt{2} \approx 0{,}7071$, a súlyok $U(-a, a)$-ból jönnek, varianciájuk $a^2/3 = 1/6$, így a forward nulla-átlagú, lineáris esetben $\sigma_z^2 = 8 \cdot (1/6) \cdot 3 = 4$, ami $\sigma_x^2 = 3$-hoz képest 33%-os növekedés. A ReLU utáni második momentum itt $q_h = 4/2 = 2 < 3$. Ha a lánc minden rétegében újra zérus átlagú, szimmetrikus előaktivációt tételezünk fel, a Xavier-séma a $d = m$ esetben a második momentumot rétegenként felezi, azaz a jelskálát rétegenként $1/\sqrt{2} \approx 0{,}707$ szorzóval csökkenti; L réteg után ez $(1/\sqrt{2})^L$, ami nem elhanyagolható, hanem exponenciális csillapítás. A valódi láncban a ReLU utáni nemnegatív bemenet miatt a rétegenkénti szorzó ettől eltér (a fejezet laborjának mérésében az első réteg után 0,71, a további rétegekben nagyjából 0,60–0,77); a Xavier-séma tehát nem a ReLU aszimmetriájára van hangolva.

21.1.8 Minimális kódos ellenőrzés

A következő rövid PyTorch-munkamenet az eloszlásszintű becslést ellenőrzi egy nagy batch Monte-Carlo-közelítésben. A cél nem a teljes háló tanítása, hanem annak látványossá tétele, hogy a kezdeti lépésben a réteg szórása milyen közel van az elméleti célhoz.

```
# lab/n081_mini_scale.py
import math
import torch

torch.manual_seed(20261020)
dt = torch.float64            # a variancia pontos méréséhez
batch, d, m = 8192, 8, 4
x = torch.randn(batch, d, dtype=dt) * math.sqrt(3.0)  # sigma_x^2 = 3
lin = torch.nn.Linear(d, m, dtype=dt)
std_w = math.sqrt(2.0 / d)    # He normal: sigma_w^2 = 2/fan_in
torch.nn.init.normal_(lin.weight, mean=0.0, std=std_w)
torch.nn.init.zeros_(lin.bias)
```

```
with torch.no_grad():
    z = lin(x)
    h = torch.relu(z)
print("z var  :", float(z.var()))              # elmélet (súlyok felett várható): 6
print("E[h^2] :", float((h ** 2).mean()))      # elmélet: 6/2 = 3
print("h var  :", float(h.var()))              # elmélet: 6*(1/2 - 1/(2*pi)) ~ 2.045
```

A futtatás parancsa `python lab/n081_mini_scale.py`. A futtatott kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()=10`, `seed=20261020`, `float64`, `batch=8192`):

```
z var  : 6.799200011595406
E[h^2] : 3.355011905531572
h var  : 2.4107006413283036
```

Az elméleti 6, 3 és 2,045 a súlyok eloszlása feletti várható érték; egyetlen 4×8-as súlyhúzásnál a sorok normája ettől érezhetően eltér, ezért a z szórásnégyzete 6 helyett 6,8. Az arányok viszont a súlyhúzástól függetlenül a levezetést követik: $E[h^2]/\operatorname{Var}(z) \approx 0{,}493 \approx \frac{1}{2}$ és $\operatorname{Var}(h)/\operatorname{Var}(z) \approx 0{,}355 \approx \frac{1}{2} - \frac{1}{2\pi} = 0{,}341$. A 0,341 arány egy *rögzített sor* soronkénti, a sornormára feltételes referenciája ($\operatorname{Var}_x \operatorname{ReLU}(w_i^\top x)/\operatorname{Var}(w_i^\top x)$), nem az összes neuronra összevont empirikus `h.var` aránya: az utóbbi különböző sornormájú Gauss-keverék, ezért nem univerzálisan 0,341. A pontos referenciához soronkénti feltételes számítás és ismételt súlyhúzások kellenek.

21.1.9 Labor: aktiváció- és gradiensskála több kis rétegen

Előfeltételek. Python 3.12 (a `lab/pyproject.toml` szerint `>=3.12,<3.13`), a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Külső adatfájl nem kell: a labor szintetikus bemenetet generál. Futtatás: `python lab/n081_init_scale.py --seed 20261020 --dtype float64 --scheme he` (a Xavier és shifted változat a szakaszban). Várható futásidő CPU-n néhány másodperc; a tényleges időt a futás naplózza.

Cél és hipotézis. A labor azt vizsgálja, hogy egy hat rétegű, 64 széles, ReLU-aktivációjú MLP első lépésében a He-inicializálás közelebb tartja-e a rétegenkénti aktiváció második momentumának gyökerét (RMS) 1-hez, mint a Xavier-uniform, és hogyan változik a bemeneti gradiens szórása. Hipotézisünk szerint szimmetrikus, zérusátlagú bemeneten a He-séma ReLU alatt jobb skálamegőrzést ad; eltolt, nem zérus átlagú bemeneten mindkét becslés pontatlanná válik.

Adat és számítás. A labor szintetikus bemenetet használ, nem a függelék A. szakaszában leírt GlyphLab-képeket: $x \sim N(0, I_{64})$; a bemenet tehát nem képadat, és a labor nem része a teljes képklasszifikációs eljárásnak. A súlyokat a fenti képletekkel inicializáljuk, a biast nullára. A mérés `float64`-ben történik a variancia pontos összehasonlításához; a tanítási folyamat ettől függetlenül `float32` CPU marad. A kódot a `lab/n081_init_scale.py` fájlba írjuk.

```
# lab/n081_init_scale.py
import argparse
import math
import torch

def make_layers(width, layers, scheme, dt):
    result = []
    for _ in range(layers):
        lin = torch.nn.Linear(width, width, bias=True, dtype=dt)
        if scheme == "he":
            std = math.sqrt(2.0 / lin.in_features)
            torch.nn.init.normal_(lin.weight, mean=0.0, std=std)
        elif scheme == "xavier":
```

```
            a = math.sqrt(6.0 / (lin.in_features + lin.out_features))
            torch.nn.init.uniform_(lin.weight, -a, a)
        else:
            raise ValueError(f"ismeretlen séma: {scheme}")
        torch.nn.init.zeros_(lin.bias)
        result.append(lin)
    return result

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--seed", type=int, default=20261020)
    p.add_argument("--dtype", choices=["float32", "float64"], default="float64")
    p.add_argument("--batch", type=int, default=8192)
    p.add_argument("--width", type=int, default=64)
    p.add_argument("--layers", type=int, default=6)
    p.add_argument("--scheme", choices=["he", "xavier"], default="he")
    p.add_argument("--shifted", action="store_true",
                   help="zérus átlag feltétel megsértése")
    args = p.parse_args()

    dt = torch.float64 if args.dtype == "float64" else torch.float32
    torch.manual_seed(args.seed)
    x = torch.randn(args.batch, args.width, dtype=dt)
    if args.shifted:
        x = x + 1.5
    x.requires_grad_(True)   # az eltolás után lesz levél és kap gradienst

    layers = make_layers(args.width, args.layers, args.scheme, dt)
    h = x
    print("label layer_idx fan_in fan_out h_std h_rms")
    for i, lin in enumerate(layers):
        h = torch.relu(lin(h))
        if i < 3 or i == len(layers) - 1:
            h_rms = float(h.detach().pow(2).mean().sqrt())
            print(args.scheme, i, lin.in_features, lin.out_features,
                  f"{float(h.detach().std()):.4f}", f"{h_rms:.4f}")
    h.sum().backward()
    print("input_grad_std", f"{float(x.grad.std()):.4f}")
    print("input_grad_rms", f"{float(x.grad.pow(2).mean().sqrt()):.4f}")

if __name__ == "__main__":
    main()
```

A futtatási parancs:

```
python lab/n081_init_scale.py --seed 20261020 --dtype float64 --scheme he
python lab/n081_init_scale.py --seed 20261020 --dtype float64 --scheme xavier
python lab/n081_init_scale.py --seed 20261020 --dtype float64 --scheme he --shifted
```

Az alábbi kimenet a megadott paranccsal mért érték (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.get_num_threads()`=10, `float64`, `seed`=20261020, `batch`=8192):

```
label layer_idx fan_in fan_out h_std h_rms
he 0 64 64 0.8315 1.0053
he 1 64 64 0.8579 1.0437
he 2 64 64 0.8131 0.9823
he 5 64 64 0.8339 0.9637
input_grad_std 1.7436
input_grad_rms 1.7454
```

Értelmezés. A He-inicializált, szimmetrikus esetben a rétegenkénti aktiváció második momentumának gyöke (`h_rms`) 1 körül marad; a középértékre centrált `h_std` ennél kisebb, a mért futásban 0,81–0,86, összhangban a $\mathrm{Var}(h) = E[h^2] - E[h]^2 \approx 1 - 1/\pi \approx 0{,}682$ összefüggéssel.

A gradiens `input_grad_rms` értéke sem robban. A Xavier-séma ReLU mellett rétegenként körülbelül 0,7-szeresére csökkenti a `h_rms`-t, mert a $\sigma_w^2 = 2/(d+m) = 1/64$ választás a He-féle $2/d$ helyett csak feleakkora súlyvarianciát ad, és a ReLU a második momentumot felezi; a mért futásban hat réteg után 0,151 körül van. Az `--shifted` futtatás azt mutatja, hogy a bemenet eltolásával a zérusátlag-feltétel sérül, és a második momentum a becsültnél nagyobb (mért `h_rms` 1,38–1,91); ez a heurisztika érvényességi határa, nem a kód hibája. Az itteni labor egyetlen seeddel szemléltet; érdemi összehasonlításhoz legalább három seed kell, az F5 kísérletterv pedig öt párosított seedet ír elő; a szórást mindkét esetben a seedeken keresztül kell feljegyezni.

21.1.10 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

1. F1 fogalmi (nehézség 1). Adja meg egy $W \in \mathbb{R}^{10\times 6}$ súlyú affin réteg fan-injét és fan-outját, majd magyarázza el, miért nem szimmetrikus a két mennyiség szerepe a gradiensáramlásban.

2. F2 levezetési (nehézség 2). Legyen fan_in $= 8$, fan_out $= 4$, a bemeneti variancia $\sigma_x^2 = 3$, a bemenet zérus átlagú. He-féle normál inicializálással számítsa ki a z előaktiváció varianciáját, a ReLU utáni h varianciáját a szimmetria-feltevés mellett, majd a backward σ_{dx}^2 értékét, ha $\sigma_{dz}^2 = 1$. Minden lépésben jelölje meg, melyik feltevést használja, és ellenőrizze a dimenziókat.

3. F3 implementációs (nehézség 2). Írjon egy `measure_forward_scale(layers, x)` függvényt, amely egy lineáris rétegekből álló listán ReLU-aktivációval végigfuttatja az x batchet, és visszaadja a `(fan_in, fan_out, out_rms)` hármasok listáját, ahol `out_rms = sqrt(mean(h**2))` a ReLU utáni aktiváció második momentumának gyöke (nem a `h.std()`, mert a ReLU kimenet nem zérus átlagú). A függvényt tesztelje két $64 \to 64$-es rétegen He-inicializálással, `torch.float64` esetén, `seed=20261020` mellett, 4096 mintás batchcsel; a várt `out_rms` értékeknek 1 közelében kell lenniük, a véletlen minta miatt $\pm 0{,}1$ toleranciával.

4. F4 hibakeresési (nehézség 2). Egy kódban a következő inicializálás szerepel:

```
a = math.sqrt(6.0 / (fan_in + fan_out))
torch.nn.init.uniform_(m.weight, 0, a)
```

Azonosítsa a hibát, mondja meg, melyik feltevés sérül, milyen következménye lehet a ReLU-hálózat mélyebb rétegeiben, és adja meg a javított kódot. A javítás utáni ellenőrzéshez adja meg, mit kell kiszámítani (például súlyátlag, szimmetria) és mi a várt érték.

5. F5 kísérlettervezési (nehézség 3). Tervezzen GlyphLab-CNN-kísérletet annak eldöntésére, hogy a He- vagy a Xavier-inicializálás ad-e jobb validációs loss-t. A tervnek tartalmaznia kell: kérdés, baseline, a megváltoztatott egyetlen tényező, split, seedek, metrika, erőforrás és a következtetés határa. A tesztet a terv szerint csak a legjobb validációs modellen szabad kiértékelni.

21.1.11 Megoldások és indoklás

1. A fan-in 6, a fan-out 10. A fan-in a forward összeg tagjainak száma: egy kimeneti egység hat súlyozott bemenetet ad össze. A fan-out a backward összeg tagjainak száma: egy bemeneti koordináta gradienséhez mind a tíz kimeneti gradiens hozzájárul W-n keresztül. A kettő akkor lenne azonos szerepű, ha a mátrix négyzetes lenne, és a forward, valamint a transzponált szorzás ugyanannyi tagot vonna be; általános téglalapmátrix esetén a két irány más skálázást igényel.

2. He-normál: $\sigma_w^2 = 2/8 = 0{,}25$. A szorzat varianciája $\sigma_w^2 \sigma_x^2 = 0{,}25 \cdot 3 = 0{,}75$; a nyolc független, zérus átlagú tag összegére a lemma szerint

$$\sigma_z^2 = 8 \cdot 0{,}75 = 6.$$

A ReLU szimmetrikus z esetén a második momentumot felezi: $E[h^2] = 6/2 = 3$ (a variancia ennél kisebb, normális z-re $\approx 2{,}045$). A backward egyetlen bemeneti koordinátára négy tagot összegez, mindegyik varianciája $\sigma_w^2 \sigma_{dz}^2 = 0{,}25 \cdot 1 = 0{,}25$, így

$$\sigma_{dx}^2 = 4 \cdot 0{,}25 = 1.$$

Dimenzióellenőrzés: W alakja 4×8, a forward $Z \in \mathbb{R}^{N \times 4}$, a backward X-gradiens alakja $N \times 8$; a levezetett skalárok mindkét esetben varianciadimenziójúak. Szélsőesetként ha $\sigma_x^2 = 0$, akkor a bemenet konstans, a szorzat is konstans nulla átlaggal, de a függetlenségi feltevés továbbra is szükséges az összegzéshez.

3. Az elvárt függvény és teszt:

```
def measure_forward_scale(layers, x):
    scales = []
    h = x
    for lin in layers:
        fan_in = lin.weight.shape[1]
        fan_out = lin.weight.shape[0]
        h = torch.relu(lin(h))
        out_rms = float(h.detach().pow(2).mean().sqrt())
        scales.append((fan_in, fan_out, out_rms))
    return scales

import math, torch
torch.manual_seed(20261020)
dt = torch.float64
x = torch.randn(4096, 64, dtype=dt)
l1 = torch.nn.Linear(64, 64, dtype=dt)
l2 = torch.nn.Linear(64, 64, dtype=dt)
for lin in (l1, l2):
    torch.nn.init.normal_(lin.weight, 0.0, math.sqrt(2.0 / 64))
    torch.nn.init.zeros_(lin.bias)
print(measure_forward_scale([l1, l2], x))
```

A várt kimenet mindkét rétegre `(64, 64, kb. 1.0)`. A tolerancia $\pm 0{,}1$ a 4096-os minta második momentum-becslésének véletlen ingadozása miatt; ha a kimenet ennél nagyobb eltérést mutat, nem a toleranciát kell növelni, hanem a seedet, a dtype-ot és a `fan_in` kiszámítását kell ellenőrizni. A mintanagyság növelésével az ingadozás csökken, de a pontos 1 érték csak végtelen minta határértéke. A `h.std()` ugyanebben a mérésben jellemzően 0,83 körül van, mert a ReLU kimenet várható értéke nem nulla: $\mathrm{Var}(h) = E[h^2] - E[h]^2 \approx 1 - 1/\pi \approx 0{,}682$.

4. A hiba: a súlyok a $[0, a]$ intervallumból jönnek, ezért átlaguk $a/2$, nem nulla. Az első rétegben ez még nem változtatja meg a kimenet átlagát: zérus átlagú, a súlyoktól független bemenetre

$E[z_o] = \sum_i E[W_{oi}]E[x_i] = 0$, és a z varianciája is fan_in $\cdot E[W^2] \cdot E[x^2] =$ fan_in $\cdot (a^2/3) \cdot E[x^2]$, ugyanaz, mint a $U(-a, a)$ esetben; itt $E[W^2] = a^2/3$, ami $U(0, a)$ esetén $a^2/12 + a^2/4$, és nem tévesztendő össze a súlyok $a^2/12$ varianciájával. Az első rétegben a hiba tehát rejtett: a súlyok átlaga már nem nulla, de a zérus átlagú bemenet miatt a kimenet átlaga még nulla, és a variancia is egyezik a szimmetrikus esettel, mert $E[W^2] = a^2/3$. A hiba a mélyebb rétegekben válik láthatóvá: a ReLU kimenete nemnegatív, ezért a következő réteg bemenetének átlaga pozitív, és a pozitív átlagú súlyokkal $E[z_o] = \sum_i E[W_{oi}]E[h_i] > 0$; az előaktiváció eltolódik, a második rétegben minden előaktiváció pozitív (nemnegatív bemenet, pozitív súlyok, nulla bias), ezért a ReLU gyakorlatilag semmit nem nulláz, és a második momentumra vonatkozó $q = \sigma_z^2/2$ becslés érvényét veszti (a tényleges $E[h^2]$ nagyobb lesz). A zérusátlag-feltevés tehát nem az első réteg kimenetén, hanem a mélyebb rétegek bemenetén sérül. A javítás:

```
a = math.sqrt(6.0 / (fan_in + fan_out))
torch.nn.init.uniform_(m.weight, -a, a)
```

Ellenőrzés: inicializálás után `m.weight.mean()` értékének 0 közelében kell lennie, `m.weight.min()` közelítőleg $-a$, `m.weight.max()` közelítőleg a, és a variancia $a^2/3$. A javított változatban a súlyok várható értéke nulla, ezért a mélyebb rétegek előaktivációja sem tolódik el a súlyátlag miatt.

5. Kérdés: a He- és a Xavier-inicializálás hogyan befolyásolja a GlyphLab-osztályozó validációs loss-át? A GlyphLab 180 jelenet × 2 nézet = 360 képe 24×24-es, egycsatornás, három osztállyal; a `corrupted_test.npy` a teszt 60 képének korrupt változata, amelyet csak a végleges modell diagnosztikájára használunk, modellválasztásra nem. Baseline: a 20.4. V. ellenőrző pont: indokolt CNN-választás döntési lapjának `TinyGlyphCNN` modellje (két classic blokk, 8 és 16 csatorna, 4155 paraméter), He-inicializálással. Az egyetlen megváltoztatott tényező a súlyinicializálási séma: He helyett Xavier-uniform; minden más döntés rögzített: adatverzió, train/validációs/test scene-szintű split, batchméret, optimizer, tanulási ráta, epochszám, checkpoint-szabály és értékelési mód. Mindkét sémát öt seeddel futtatjuk, ugyanazon seedeket párosítva; a páros seed-különbségek átlagára számolt bootstrap CI alapján döntünk: ha a 95%-os CI kizárja a nullát, állítunk különbséget, különben nem. Metrika: a validációs loss alapján kiválasztott modell validációs loss-a és validációs pontossága, valamint a kiválasztott modell tesztpontossága; a korrupt tesztet nem használjuk modellválasztásra. Erőforrás: CPU, `float32` futás. Következtetési határ: az eredmény a 180 jelenetes, procedurális GlyphLab-adatra és a választott kis hálóra érvényes; nem állítunk általános CNN-rangsort. Negatív vagy kiegyenlítetlen eredmény esetén a seedeloszlást és a gradiensnormákat kell közölni.

Hipotézis és egység: a He-inicializálás alacsonyabb validációs loss-t ad, mint a Xavier; a változó tényező a súlyinicializálási séma, a megfigyelési egység a `scene_id`. **Recept és erőforrás számszerűen:** a `TinyGlyphCNN` tanítása Adam optimalizálóval, `lr=1e-3`, 16-os batch és legfeljebb 30 epoch mellett fut; az öt seed értéke 20261020–20261024, a két inicializálási sémen párosítva. A döntési szabály a validációs loss páros seed-különbségének 95%-os bootstrap CI-je; érdemi különbségként legalább 0,01 loss-egységet jelentünk. Erőforrás: CPU, `float32`, tíz futás, a GlyphLab méretén néhány perc; a tényleges futásidőt a napló rögzíti. ### Tipikus tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a helyes σ_w^2 megválasztásától *garantáltan stabil tréninget* várunk. Az inicializálási heurisztika csak a kezdeti pillanatban, a kimondott függetlenségi és zérusátlag-feltevések mellett becsli a szórások arányát. Illusztráció: vegyünk egyetlen $d \times d$-es lineáris réteget $W = 3I_d$-vel és $b = 0$-val. Egy determinisztikus mátrixnak nincs „9-es súlyvarianciája”; az elemeire számolt empirikus variancia $9(d-1)/d^2$ és d-től függ. A robbanás szempontjából a spektrális norma számít: $\|W\|_2 = 3$, tehát $\|Wx\|_2 = 3\|x\|_2$. Tegyük fel, hogy egy hibásan skálázott modellben ilyen állapot alakul ki. A forward jel egy lépés alatt 3-szorosára nő, a backward gradiens ugyancsak 3-szorosára; ha ez egy mély hálóban ismétlődik, a jel exponenciálisan robban.

A He/Xavier-képlet kezdéskor ezt próbálja megelőzni, ám a tanítás során a súlyok elmozdulnak, a bemeneteloszlás megváltozik, a BatchNorm és az adaptív optimizáló [P02] pedig további skálahatásokat visz a rendszerbe. Az [P07] által tárgyalt BatchNorm éppen a rétegen belüli normalizálással csökkenti ezt az érzékenységet, de annak mechanizmusa sem állítható minden architektúrára igazolt garanciaként. A [P08] empirikus vizsgálata szerint a BatchNorm előnye nem a belső kovariancia-eltolódás csökkentéséből, hanem az optimalizálási cél simábbá tételéből származik; ezért a tankönyv a mechanizmust vitatottként kezeli. A helyes állítás tehát: a skálázás *szükséges jellegű elővigyázatosság*, nem *elégséges stabilitási tétel*.

21.1.12 Önellenőrzés és korábbi fogalmak felidézése

Írja fel saját szavaival a fan-in és fan-out definícióját, majd ellenőrizze a következő állításokat: a lemmában a zérus várható érték miért hagyható el nem nulla átlagú tagokra; a ReLU-félvariancia miért igényel szimmetriát; a `W` alakjának ismerete nélkül miért nem dönthető el, hogy egy adott σ_w a forward vagy a backward oldalt védi. Rövid válaszok: (a) a lemma csak korrelálatlanságot használ, ezért a nem nulla átlagok a kovarianciákban kiesnek; a második momentum csak zérus átlag esetén azonos a varianciával. (b) A $\{-z, +z\}$ szimmetria nélkül a ReLU pozitív és negatív fele nem ad egyenlő járulékot a második momentumhoz, ezért a felezés nem érvényes. (c) A forward összeg a fan-int, a backward összeg a fan-outot szorozza; a σ_w értéke csak a W alakjának (d és m) ismeretében rendelhető a megfelelő oldalhoz. A korábbi anyagból idézze fel, hogy a 9.1. Neuron és perceptron alfejezetben megismert $s = w^\top x + b$ perceptron-score ugyanaz a z_o objektum, amelynek varianciáját most elemeztük; a 17.1. LeNet és a lokális megosztott reprezentáció LeNet-változatban a konvolúciós súlyokra a fan-in $C_{in} k_h k_w$ mintájára értelmezendő; a 20.1. Feature extractor és új fej alfejezetben tárgyalt előtanított súly pedig *nem* friss inicializálás, így annak eloszlása nem teljesíti automatikusan a fenti képleteket.

21.1.13 Rövid összefoglaló

Az affin réteg $z_o = \sum_i W_{oi} x_i + b_o$ összege független, zérus átlagú tagok esetén $\text{fan_in} \cdot \sigma_w^2 \sigma_x^2$ varianciát ad. A backward irány hasonlóan $\text{fan_out} \cdot \sigma_w^2 \sigma_{dz}^2$ skálát eredményez. A Xavier-inicializálás $\sigma_w^2 = 2/(\text{fan_in} + \text{fan_out})$, a He-inicializálás $\sigma_w^2 = 2/\,\text{fan_in}$ választással kompenzálja a ReLU félvarianciáját. A levezetés feltétele: iid, független, zérus átlagú súlyok és aktivációk, szimmetrikus előaktiváció. A labor kis rétegeken méri az aktiváció RMS-ét és a gradiens szórását, és az eltolt bemenet bemutatja a feltételsértést. A kezdőszórás megfelelő beállítása heurisztika: csökkenti a korai skálaromlást, de önmagában nem garantálja a teljes tanítási folyamat stabilitását.

21.1.14 Következő lecke

A következő 21.2. Learning rate és ütemezés egység a tanulási ráta és az ütemezés kérdését viszi tovább. Ott azt vizsgáljuk, hogyan kapcsolódik az itt tárgyalt kezdeti skála a frissítés nagyságához, és miért nem választható a lépésnagyság az inicializálástól függetlenül.

21.1.15 Felhasznált források

- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html.
- [P02] Loshchilov, I. és Hutter, F. (2017). *Decoupled Weight Decay Regularization*. arXiv:1711.05101v1. https://arxiv.org/abs/1711.05101.

- [P08] Santurkar, S., Tsipras, D., Ilyas, A. és Madry, A. (2018). *How Does Batch Normalization Help Optimization?* NeurIPS 2018. https://arxiv.org/abs/1805.11604.
- Glorot, X. és Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks.* AISTATS, PMLR 9:249–256. https://proceedings.mlr.press/v9/glorot10a.html.
- He, K., Zhang, X., Ren, S. és Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* ICCV 2015. https://arxiv.org/abs/1502.01852.

21.2 Learning rate és ütemezés

21.2.1 Cél, előismeret és induló becslés

A 21.1. Inicializálás és varianciaterjedés egységben az inicializálás skálaviszonyait vezettük le; ott a frissítés nagyságát még szabadon hagytuk. Itt a **learning rate** (tanulási ráta, LR), az η lépésméret kérdését vizsgáljuk: milyen intervallumban stabil, hogyan változtassuk a tanítás során ütemezetten, és hogyan rögzítsük reprodukálhatóan. Induló becslés: ha a cél $f(\theta) = 2\theta^2$ és $\theta_0 = 9$, akkor a gradiens $f'(9) = 36$. Mekkora η választásnál vált előjelet a frissítés, és melyik tartományban divergál? A pontos határokat egy egyszerű kvadratikus kontrollból vezetjük le; ez a lokális görbület és a lépésméret kapcsolatának alapesete lesz.

21.2.2 Frissítési szabály, jelölések és alakok

Legyen $\theta_t \in \mathbb{R}^d$ a paramétervektor a t-edik frissítés után, d a paraméterszám. A t-edik minibatch indexhalmazát B_t jelöli, a batch-méretet $B = |B_t|$; így B_t az indexhalmaz, B pedig a méret. A batch-átlag loss:

$$\widehat{L}(\theta_t; B_t) = \frac{1}{B} \sum_{i \in B_t} \ell_i(\theta_t),$$

ahol ℓ_i az i-edik tanítóminta vesztesége. A teljes gradiensű frissítéshez képest a következő **SGD-lépést** használjuk:

$$\theta_{t+1} = \theta_t - \eta_t \, \nabla_\theta \widehat{L}(\theta_t; B_t), \qquad \eta_t > 0.$$

A gradiens alakja megegyezik θ_t alakjával, tehát $\nabla_\theta \widehat{L} \in \mathbb{R}^d$. Az η_t általában skalár; a 20.2. Finomhangolás és tanulási ráta finomhangolási egységben láttuk, hogy paramétercsoportonként eltérő is lehet. A következő terminusokat külön kell választani:

- **step**: egy optimizer-frissítés, azaz egy minibatch feldolgozása után végrehajtott `optimizer.step()` hívás.
- **epoch**: az összes tanítópélda egyszeri végighaladása. Ha N tanítóminta van és `drop_last=False`, egy epoch $\lceil N/B \rceil$ stepből áll; `drop_last=True` esetén $\lfloor N/B \rfloor$.

A kettő nem felcserélhető, mert a legtöbb schedule a lépések vagy az epochok függvénye.

21.2.3 Kvadratikus kontroll: a schedule által okozott lépések

A következő állítást kell levezetnünk.

Állítás (kvadratikus lépésméret-korlát). Legyen $f : \mathbb{R} \to \mathbb{R}$, $f(\theta) = \frac{a}{2}\theta^2$ valamely $a > 0$-ra, és legyen a frissítés teljes gradiensű, rögzített $\eta > 0$ mellett:

$$\theta_{t+1} = \theta_t - \eta f'(\theta_t).$$

Ekkor:

1. $\theta_t = (1 - \eta a)^t \theta_0$ minden $t \geq 0$ egészre.
2. Ha $\theta_0 \neq 0$, a sorozat pontosan akkor tart nullához, ha $0 < \eta < 2/a$; $\theta_0 = 0$ esetén minden η-nál azonosan nulla.
3. Az egyetlen lépésben pontos optimumot adó választás $\eta = 1/a$.

Levezetés. $f'(\theta) = a\theta$, így

$$\theta_{t+1} = \theta_t - \eta a \theta_t = (1 - \eta a)\theta_t.$$

Az első állítás $t = 0$-ra triviális, az indukciós lépés pedig a fenti egyenlet. A mértani sorozat $(1 - \eta a)^t \theta_0$ ($\theta_0 \neq 0$ esetén) pontosan akkor tart nullához, ha $|1 - \eta a| < 1$; $\eta > 0$ mellett ez ekvivalens azzal, hogy $0 < \eta a < 2$, azaz $0 < \eta < 2/a$. A harmadik pont: $1 - \eta a = 0$ akkor és csak akkor, ha $\eta = 1/a$.

A $2/a$ érték a lokális görbületből származik; többdimenziós, nem-kvadratikus célfüggvény esetén a Hessian legnagyobb sajátértéke játszik hasonló szerepet, de ez utóbbi kapcsolatot itt csak heurisztikus irányelvnek tekintjük (a spektrális eszközöket 3.4. Sajátérték, SVD és spektrális határ vezette be). A fenti állítás maga teljes, a feltételek szerint bizonyított levezetés.

Kézi számpélda. Legyen $a = 4$, tehát $f(\theta) = 2\theta^2$, $\theta_0 = 3$.

- $\eta = 1/a = 0{,}25$: $\theta_1 = (1 - 1) \cdot 3 = 0$; egy lépés elegendő.
- $\eta = 0{,}1$: a szorzó $1 - 0{,}4 = 0{,}6$; $\theta_1 = 1{,}8$, $\theta_2 = 1{,}08$, $\theta_3 = 0{,}648$.
- $\eta = 0{,}6 > 2/a = 0{,}5$: a szorzó $1 - 2{,}4 = -1{,}4$; $\theta_1 = -4{,}2$, $\theta_2 = 5{,}88$; előjelváltással divergál.

Ez a kontroll nem állít semmit a CNN-veszteség felületéről; azt mutatja meg, hogyan számolható ki a stabilitási tartomány egy explicit görbületű feladaton.

A három tartományt érdemes külön tartani. Ha $0 < \eta < 1/a$, a szorzó pozitív és egynél kisebb, a sorozat monoton csökken. Ha $1/a < \eta < 2/a$, a szorzó negatív, de abszolút értéke egynél kisebb, ezért a sorozat előjelet vált ugyan, de csillapodóan konvergál (csillapodó oszcilláció). Az $\eta = 1/a$ határon a szorzó nulla: egy lépésben az optimumba jutunk. Az $\eta = 2/a$ határon a szorzó -1: a sorozat előjelet vált, de nagysága nem csökken (állandó amplitúdójú oszcilláció). Az $\eta > 2/a$ tartományban $|1 - \eta a| > 1$, ezért a sorozat divergál, mégpedig előjelváltva.

21.2.4 Ütemezések: step, cosine és warmup

A **learning rate schedule** (tanulási ráta ütemezése) egy előre rögzített η_t vagy η_e pozitív sorozat a tanítási költségvetésre. Három gyakori családot definiálunk. A továbbiakban **schedule** alatt ezt a sorozatot, **scheduler** alatt az ezt megvalósító, állapotot tartó objektumot értjük; a magyar „ütemezés” szó a schedule szinonimája.

Step decay. Rögzített $\eta_0 > 0$ kezdőérték, $0 < \gamma < 1$ csillapítási tényező és s lépcsőhossz mellett

$$\eta_t = \eta_0 \, \gamma^{\lfloor t/s \rfloor}.$$

Epoch-alapú változatban $\eta_e = \eta_0 \, \gamma^{\lfloor e/s_e \rfloor}$, ahol s_e epoch-hossz. A két változatot át kell számolni: az epoch-alak s_e lépcsője `drop_last=False` esetén $s_e \lceil N/B \rceil$, `drop_last=True` esetén $s_e \lfloor N/B \rfloor$ lépésnek felel meg.

Cosine annealing. Rögzített T össz-lépésszámra

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos \frac{\pi t}{T} \right), \qquad 0 \le t \le T.$$

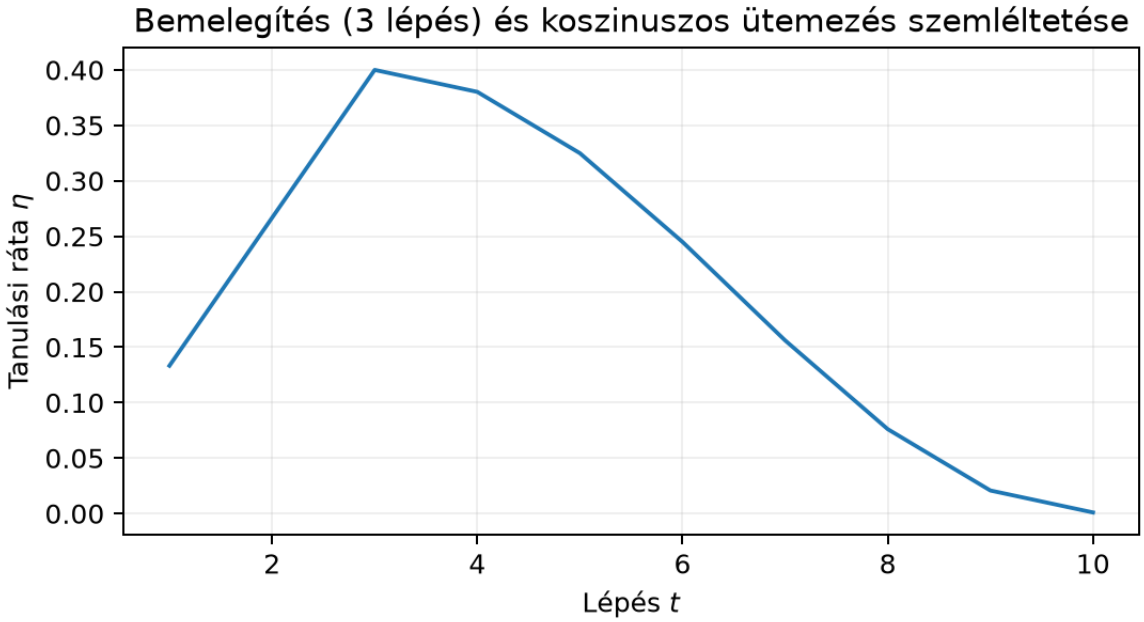
A végpontokban $\eta_0 = \eta_{\max}$ és $\eta_T = \eta_{\min}$. A t itt lépésindex; epoch-alapú használathoz a T lépést epochból kell átszámolni.

Warmup. A fő schedule elé lineáris növekedést iktatunk. Az első W lépésben

$$\eta_t = \eta_{\text{target}} \cdot \frac{t}{W}, \qquad t = 1, \dots, W,$$

majd $t > W$-ra a választott ütem következik. A cél, hogy a nagy kezdeti gradiens ne tegye tönkre a frissen inicializált rétegeket; ez heurisztika, és a szükségességét az adott modellen validációval kell igazolni. A BatchNorm-irodalom a nagyobb LR alkalmazhatóságát empirikus előnyként közli [P07], de a mechanizmus vitatott; a [P08] szerint a hatás inkább az optimalizálási cél simításából, semmint a belső kovariancia-eltolódás csökkentéséből ered; a running state kérdéseit 18.2. Batch normalization train és eval módban tárgyalja.

Kiszámolt mini-példa. $\eta_{\max} = 0{,}4$, $\eta_{\min} = 0{,}001$, $W = 3$, $T = 10$. A warmup a $t = 1, 2, 3$ lépésekre $0{,}1333; 0{,}2667; 0{,}4$ értéket ad. A cosine-szakasz $T' = 7$ hosszú, a $t' = t - 3$ átindexeléssel; a $t = 4$ lépésben $\eta = 0{,}001 + \frac{0{,}399}{2}(1 + \cos \frac{\pi}{7}) \approx 0{,}3802$, a $t = 10$ végpontban $\eta_{\min} = 0{,}001$.



21.1. ábra: Bemelegítés (3 lépés) és koszinuszos ütemezés szemléltetése

A vízszintes tengely a lépésszám, a függőleges a tanulási ráta; a három lépéses lineáris bemelegítés után a koszinuszos szakasz 0,4-ről 0,001-re csökken. A képletből számított szemléltetés, nem mért futás.

21.2.5 Scheduler állapot, epoch vs step, plateau

A könyvtári API-kban (például a PyTorch `torch.optim.lr_scheduler` modulja, [D06]) a scheduler **futó állapotot (running state)** tart nyilván: az aktuális epoch/step számlálót, az utolsó érvényes η értéket, a plateau-változatnál a patience-számlálót és a legjobb validációs metrikát. Ez az állapot a kísérlet része. Ha a checkpointba a modell- és optimizer-`state_dict` mellett a scheduler-`state_dict` nem kerül be, a visszatöltés után a schedule nem ott folytatódik, ahol abbamaradt; ez reprodukálhatósági hibát okoz (vö. 12.1.–12.2.). A mentésnek tehát három state_dict-et és az aktuális epochot kell tartalmaznia.

Az **epoch vs step** megkülönböztetés azért lényeges, mert a step-alapú schedule más számú frissítést ad a batch-mérettől függően, míg az epoch-alapú schedule azonos epochszám mellett más LR-pályát és más számítási költséget jelent.

A **ReduceLROnPlateau** visszacsatolt scheduler: a **patience** a türelmi idő (hány epochon át vár javulás nélkül), a **plateau** pedig az az állapot, amikor a rögzített validációs metrika ezen az ablakon belül nem javul. Ha ez bekövetkezik, a scheduler $\eta \leftarrow \eta \cdot \gamma$ szorzót hajt végre, és a patience-számlálót nullázza. Mivel a validációs halmaz itt közvetlenül befolyásolja a tanítási pályát, a validációs split a modellválasztás része; a végső tesztet érintetlenül kell hagyni (1.3. Adatfelosztás és a megfigyelési egység). Az adaptív ütem előnye, hogy a csökkentés időzítését az adathoz igazítja; ára, hogy a zajos validációs metrika hamis plateau-t is jelezhet.

21.2.6 Labor: két schedule összehasonlítása azonos update-kerettel

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: a GlyphLab `lab/data/` mappája (`images.npy`, `labels.npy`, `metadata.csv`). A futtatás a szakaszban megadott szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

Protokoll. Kérdés: konstans $\eta = 0{,}001$ vagy warmup+cosine ad-e alacsonyabb legjobb validációs loss-t a GlyphLabon? Hipotézis: a cosine az utolsó szakaszban csökkentett LR-rel simábban ér be a minimumba. Adat: `lab/data/` a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-szerződés szerint; a GlyphLab 360 képe 24×24-es, egycsatornás, három osztállyal; a train/val splitet a `metadata.csv` adja, a scene-szintű csoportosítást az 5.4. Függő képek, seedek és resampling egységben rögzítettük. Modell: a 9.2. MLP és nemlinearitás definícióját követő kis MLP, bemenet $576 = 24 \cdot 24$ (NCHW $1 \times 24 \times 24$), rejtett 32 ReLU neuron, dropout $p = 0{,}05$, kimenet $K = 3$ logits. Itt NCHW a tensor dimenziósorrendje ([batch, csatorna, magasság, szélesség]); a logit a softmax előtti nyers kimeneti pontszám. Optimizer: AdamW `lr=1e-3, weight_decay=0.01`; a decoupled weight decay [P02] a csillapítást a gradiens adaptív skálázásától választja le, nem az LR-től. A PyTorch AdamW lépése `param.mul_(1 - lr*weight_decay)`, ezért a weight decay λ együtthatóját (a `weight_decay` argumentumot) az LR (és így a schedule ütemszorzója) is skálázza: a schedule nemcsak az LR-pályát változtatja, hanem a tényleges csillapítás nagyságát is. Az összehasonlításban ezt a schedule hatásának részeként kell értelmezni, nem lehet úgy tekinteni, mintha a weight decay végig azonos lenne. Batch 32, 40 epoch, seed 20261020, CPU float32. Az összehasonlításban azonos modell, optimizer, batch, epoch és seed mellett csak a schedule típusa változik; ez az „azonos update-keret". A cosine ág 3 epoch (24 lépés) lineáris warmupot használ: a `LambdaLR` a `step` számlálót a már megtett lépések számaként adja át, ezért a `(step+1)/WARMUP_STEPS` szorzó a nulladik hívásnál 1/24, a huszonnegyediknél 1, és innen a cosine-szakasz $\eta_{\min} = 0$-val csökken; a konstans ág végig $\eta = 0{,}001$. A best checkpointot a validációs loss minimuma választja; a végső tesztet csak a kiválasztás után, egyszer értékeljük. Egyetlen seed mellett a két schedule különbsége csak erre a futásra érvényes; a seed-szórás és a páros döntés az F5 protokolljában szerepel.

Futtatási parancs: `cd lab && python n082_schedule_compare.py`. A vázlat a `metadata.csv` feldolgozásában a 11.3. Dataset és DataLoader a split után mintáját követi, a batch-elést azonban kézi `randperm`-mel végzi, hogy a lépésszám és a sorrend explicit legyen:

```
# lab/n082_schedule_compare.py
# CPU-futás rögzített seeddel; a kimenet a lenti, mért blokk.
import math
import numpy as np
import torch
from torch import nn
from torch import optim

SEED = 20261020
device = torch.device("cpu")

X = np.load("data/images.npy")          # (360,1,24,24) float32
y = np.load("data/labels.npy")          # (360,) int64
# metadata.csv: split oszlop; a pontos string-feldolgozást a 11.3. alfejezet mutatja.
meta = np.genfromtxt("data/metadata.csv", delimiter=",", dtype=None,
                     names=True, encoding="utf-8")
# A sorrendre nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
if not (len(meta) == len(X) == len(y)):
    raise ValueError("a metadata.csv és a tömbök hossza eltér")
idx_all = meta["index"].astype(np.int64)
if not np.array_equal(np.sort(idx_all), np.arange(len(X))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
```

```
split = np.asarray(meta["split"]).astype(str)
train_idx = idx_all[split == "train"]
val_idx   = idx_all[split == "val"]
test_idx  = idx_all[split == "test"]

B = 32
STEPS_PER_EPOCH = math.ceil(len(train_idx) / B)   # 8
assert STEPS_PER_EPOCH == 8
EPOCHS = 40
TOTAL_STEPS = EPOCHS * STEPS_PER_EPOCH
WARMUP_STEPS = 3 * STEPS_PER_EPOCH

def make_model():
    return nn.Sequential(nn.Flatten(), nn.Linear(576, 32), nn.ReLU(),
                         nn.Dropout(0.05), nn.Linear(32, 3)).to(device)

def make_lr_lambda(kind):
    if kind == "const":
        return lambda step: 1.0
    if kind == "cosine":
        def f(step):
            if step < WARMUP_STEPS:
                return (step + 1) / WARMUP_STEPS
            t = step - WARMUP_STEPS
            T = TOTAL_STEPS - WARMUP_STEPS
            return 0.5 * (1 + math.cos(math.pi * t / T))
        return f
    raise ValueError(kind)

init_sig = None

def run(kind):
    # Mindkét schedule azonos inicializálást és shuffle-sorrendet kap.
    global init_sig
    torch.manual_seed(SEED)
    model = make_model()
    sig = float(model[1].weight.detach().double().sum())
    if init_sig is None:
        init_sig = sig
    assert sig == init_sig, "a két schedule inicializálása eltér"
    opt = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
    sched = optim.lr_scheduler.LambdaLR(opt, make_lr_lambda(kind))
    lossf = nn.CrossEntropyLoss()
    best_val = math.inf
    Xt = torch.from_numpy(X[train_idx])
    yt = torch.from_numpy(y[train_idx])
    Xv = torch.from_numpy(X[val_idx])
    yv = torch.from_numpy(y[val_idx])
    for epoch in range(EPOCHS):
        model.train()
        perm = torch.randperm(len(yt))
        for i in range(0, len(yt), B):
            xb = Xt[perm[i:i+B]]
            yb = yt[perm[i:i+B]]
            opt.zero_grad()
            lossf(model(xb), yb).backward()
            opt.step()
            sched.step()
        model.eval()
        with torch.no_grad():
            val_loss = lossf(model(Xv), yv).item()
        if val_loss < best_val:
            best_val = val_loss
```

```
            torch.save({"model": model.state_dict(), "opt": opt.state_dict(),
                        "sched": sched.state_dict(), "epoch": epoch},
                       f"best_{kind}.pt")
    return best_val

const_best = run("const")
cosine_best = run("cosine")
assert math.isfinite(const_best) and math.isfinite(cosine_best)
print("const  best_val_loss:", const_best)
print("cosine best_val_loss:", cosine_best)

def test_eval(kind):
    ckpt = torch.load(f"best_{kind}.pt", weights_only=True)
    model = make_model()
    model.load_state_dict(ckpt["model"])
    model.eval()
    xe = torch.from_numpy(X[test_idx])
    ye = torch.from_numpy(y[test_idx])
    with torch.no_grad():
        logits = model(xe)
        loss = nn.functional.cross_entropy(logits, ye).item()
        acc = (logits.argmax(1) == ye).float().mean().item()
    return loss, acc

for kind in ("const", "cosine"):
    test_loss, test_acc = test_eval(kind)
    print(f"{kind:6s} test_loss: {test_loss:.6f} test_acc: {test_acc:.4f}")
```

A kimenet ezen a környezeten mért érték (Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`, seed 20261020; a futás kétszer megismételve ugyanezt adta). A kód `assert`-tel ellenőrzi a lépésszámot és a két futás azonos inicializálását; a `torch.manual_seed(SEED)` mindkét `run` hívás elején visszaáll, ezért a modell inicializálása és a `randperm`-sorrend is azonos:

```
const  best_val_loss: 1.1027379035949707
cosine best_val_loss: 1.1026660203933716
const  test_loss: 1.094088 test_acc: 0.3500
cosine test_loss: 1.094764 test_acc: 0.4333
```

Értelmezés: a legjobb validációs loss gyakorlatilag azonos (a különbség $7 \cdot 10^{-5}$), a teszt-lossban sincs érdemi eltérés, a teszt-pontosság viszont 0,35 és 0,4333, azaz öt kép különbség a 60 képes teszt halmazon. Egyetlen seedből ez nem bizonyít schedule-előnyt; a tesztet csak a validációs kiválasztás után, egyszer értékeltük. A `best_val_loss` összehasonlítása ráadásul ugyanazon a validációs halmazon történik, amelyen a checkpointot kiválasztottuk, ezért a két minimum összehasonlítása optimista torzítást hordoz; a teszt egyszeri értékelése ezért külön szerepel, és a schedule-döntéshez három seedes, páros protokoll kell. A labor célja a szerződéses kivitelezés, nem egy előre megadott győztes.

21.2.7 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Definiálja a step és az epoch közötti különbséget, és adja meg a kapcsolatukat `drop_last=True` esetén. Miért nem elegendő azt közölni, hogy „50 epochig tanítottuk a modellt”, ha valaki reprodukálni akarja a schedule-t?

F2 (levezetési, nehézség 2). Tekintse az $f(\theta) = \frac{3}{2}\theta^2$ függvényt, $\theta_0 = 2$ kezdőpontot és az $\eta_t = 0{,}5$ konstans lépésű gradiensmódszert. (a) Adja meg θ_t zárt alakját. (b) Számolja ki $\theta_1, \theta_2, \theta_3$ értékét. (c) Milyen η intervallumban konvergál a módszer? (d) Ellenőrizze, hogy $\eta = 1/3$ esetén $\theta_1 = 0$.

F3 (implementációs, nehézség 2). Írjon Python-függvényt `cosine_lr(step, T, eta_max, eta_min, warmup=0)` néven, amely a fenti cosine-képletet valósítja meg, `warmup>0` esetén a $t \le W$ tartományban lineáris növekedéssel. Regressziós teszt: $T = 100$, $\eta_{\max} = 0{,}1$, $\eta_{\min} = 10^{-4}$; a végpontokra `step=0` ad 0,1 értéket, `step=T` ad 10^{-4} értéket. A relatív tűrés 10^{-12}. Szélsőeset: `warmup=0` ne osszon nullával.

F4 (hibakeresési, nehézség 2). Egy kolléga a checkpointba csak `model.state_dict()` és `opt.state_dict()` objektumokat ment, a scheduler `state_dict`-jét nem, majd a visszatöltés után a ReduceLROnPlateau patience-számlálóját újraindulni észlelte. Adja meg a hiba okát, következményét és a javított mentés/betöltés kódját.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen kis CPU-kísérletet a GlyphLabon, amely a step decay ($\gamma = 0{,}1$, lépcső 15 epoch) és a cosine annealing validációs lossát hasonlítja össze. Adja meg: megfigyelési egység, split, baseline, változó tényező, metrika, keresési keret és seed. Indokolja, miért nem vonható le általános következtetés egyetlen seedből, és milyen ismétlést tenne hozzá.

21.2.8 Megoldások és indoklás

F1. A step egy optimizer-frissítés, az epoch az összes tanítóminta egyszeri végighaladása. `drop_last=True` esetén egy epoch $\lfloor N/B \rfloor$ stepet tartalmaz, mert az utolsó, B-nél rövidebb batch kimarad. Az „50 epoch” önmagában nem határozza meg a frissítések számát: a GlyphLab 240 tanítómintája mellett $B = 32$ esetén $\lfloor 240/32 \rfloor = 7$ step/epoch, 50 epoch = 350 lépés; $B = 64$ esetén $\lfloor 240/64 \rfloor = 3$, 50 epoch = 150 lépés. A step-alapú schedule tehát ugyanannyi epoch alatt eltérő LR-pályát jár be; a reprodukcióhoz a batch-méretet, a `drop_last` értéket és a schedule típusát is meg kell adni.

F2. Itt $a = 3$, tehát $f'(\theta) = 3\theta$, és

$$\theta_{t+1} = (1 - 3\eta)\theta_t.$$

(a) $\theta_t = (1-3\eta)^t \theta_0$; a megadott $\eta = 0{,}5$ és $\theta_0 = 2$ értékekkel a zárt alak $\theta_t = 2 \cdot (-0{,}5)^t$. (b) $\eta = 0{,}5$ esetén a szorzó $1 - 1{,}5 = -0{,}5$, tehát $\theta_1 = -1$, $\theta_2 = 0{,}5$, $\theta_3 = -0{,}25$. (c) A konvergencia feltétele $|1 - 3\eta| < 1$, azaz $0 < \eta < 2/3$. (d) $\eta = 1/3$ esetén a szorzó $1 - 1 = 0$, tehát $\theta_1 = 0$. Megjegyzés: a $2/3$ érték a stabilitási határ, nem az optimális lépés; ott a szorzó -1, tehát oszcilláció alakul ki.

F3. A függvény:

```
import math

def cosine_lr(step, T, eta_max, eta_min, warmup=0):
    if step < warmup:
        return eta_max * step / warmup if warmup > 0 else eta_max
    t = step - warmup
    T_eff = T - warmup
```

```
    if T_eff <= 0:
        raise ValueError("warmup < T szükséges")
    return eta_min + 0.5 * (eta_max - eta_min) * (1 + math.cos(math.pi * t / T_eff))
```

Ellenőrzés: `cosine_lr(0, 100, 0.1, 1e-4)` a `warmup=0` ág miatt a cosine-szakaszba lép, $t = 0$, $\cos 0 = 1$, az eredmény $\eta_{\max} = 0{,}1$. `cosine_lr(100, 100, ...)` esetén $t = 100$, $\cos \pi = -1$, az eredmény $\eta_{\min} = 10^{-4}$. A lebegőpontos kerekítés a 10^{-12} relatív tűrésen belül van. A `warmup=T` eset kizárt a `T_eff<=0` ellenőrzéssel. A `warmup>0` ágban a `step` a frissítések száma: a `step=0` a kezdő állapot, ezért ad nullát; az első frissítés `step=1`-nél $\eta_{\max}/W$, a W. frissítés `step=W`-nél $\eta_{\max}$.

F4. Ok: a scheduler belső számlálói (epoch, patience, best metrika, utolsó LR) nem részei a modell vagy az optimizer `state_dict`-jének; önálló objektumban élnek. Ugyanez a hiba minden állapotot tartó schedulerre igaz: `LambdaLR`/cosine esetén a `last_epoch` számláló nullázódik, ezért a visszatöltés után a schedule a pálya elejéről indul újra, és a konstruktor az optimizer aktuális LR-jeiből képzi a `base_lrs` értéket; az első `step()` így a kezdeti LR-t állítja be, nem a folytatását. Következmény: visszatöltéskor a scheduler új példánya nulla patience-számlálóval indul, tehát a plateau-várás elölről kezdődik, és a folytatás LR-pályája eltér a megszakított futásétól. Javítás:

```
torch.save({"model": model.state_dict(), "opt": opt.state_dict(),
            "sched": sched.state_dict(), "epoch": epoch}, "ckpt.pt")

ckpt = torch.load("ckpt.pt", weights_only=True)
model.load_state_dict(ckpt["model"])
opt.load_state_dict(ckpt["opt"])
sched.load_state_dict(ckpt["sched"])
```

Ellenőrzés: a mentés utáni `sched.get_last_lr()` értékét naplózza; a visszatöltés után ugyanazt kell visszaadnia.

F5. Egy lehetséges terv. Megfigyelési egység: egy teljes tanítási futás a rögzített seeddel. Split: a GlyphLab scene-szintű train/validációs/test felosztása; a validáció a best checkpoint kiválasztására szolgál, a teszt csak a végén fut. Baseline: konstans $\eta = 0{,}001$. Változó tényező: a schedule típusa (step decay versus cosine), minden más azonos. Metrika: legjobb validációs loss; záró teszt-accuracy a kiválasztott checkpointra. A **keresési keret** itt a rögzített kísérleti beállítások halmaza: a fenti hyperparaméterek, a split, a modell és az értékelési szabály; a kereten belül csak a schedule típusa változik. A konkrét beállítások: 15 epochos lépcső, $\gamma = 0{,}1$, batch 32, AdamW, három seed (20261020, 20261021, 20261022). Egy seed nem általánosít, mert a súlyinicializálás és a minibatch sorrend realizációja eltérő pályát ad. Ismétlés: a három seedet párosítjuk a két schedule között, átlagot és seed-szórást adunk, a páros különbségekre bootstrap CI-t számolunk; ha a 95%-os CI tartalmazza a nullát, nem állítunk különbséget. Várt eredmény: a laborhoz hasonlóan a két schedule validációs lossa közel azonos, ezért a döntést nem a pontbecslés, hanem a páros CI adja; a tesztet csak leíró értékelésre használjuk. A következtetést a GlyphLabra és a használt modellcsaládra korlátozzuk.

Hipotézis: a warmup+cosine schedule alacsonyabb validációs loss-t ad, mint a konstans $\eta = 0{,}001$. **Architektúra és erőforrás:** a modell a GlyphLab `SmallCNN` (két konvolúciós blokk, 8 és 16 csatorna, lineáris fej); a futás AdamW-val, `lr=1e-3`, 15 epoch és 32-es batch mellett megy; a három seed értéke 20261020–20261022, a döntési küszöb pedig a páros seed-különbségek 95%-os bootstrap CI-je, legalább 0,01 validációs loss-különbséggel. Erőforrás: CPU, `float32`, hat futás, néhány perc; a futásidőt a napló rögzíti. ### Tipikus tévedés ellenpéldával

„A 40 epoch ugyanannyi számítást végez, mint bármely más 40 epochos futás.” Ez hibás, és a hiba nem a schedule típusáról szól, hanem a batch-méret és a step-szám kapcsolatáról. A GlyphLab 240 tanítóképéből `drop_last=False` mellett $B = 32$ esetén 8 batch képződik, $B = 64$ esetén 4;

40 epoch az elsőnél 320, a másodiknál 160 frissítést jelent. A forward/backward hívások száma és a számítási költség tehát a batch-mérettől függ; a step-alapú step decay ugyanazon epochszám alatt más számú lépcsőt hajt végre a két esetben. Az epoch önmagában sem a költséget, sem a schedule pályáját nem határozza meg.

21.2.9 Önellenőrzés és korábbi fogalmak felidézése

Önellenőrző kérdések. (1) Mi a $0 < \eta < 2/a$ intervallum szerepe a kvadratikus példában? (2) Mely állapotokat kell a checkpointba menteni a scheduler folytatásához? (3) A cosine-képlet $t = 0$ és $t = T$ végpontjaiban milyen értéket ad?

Rövid válaszok. (1) Ez a stabilitási tartomány: ezen belül $|1 - \eta a| < 1$, tehát a sorozat konvergál; $0 < \eta < 1/a$ monoton, $1/a < \eta < 2/a$ csillapodó oszcilláció. (2) A modell, az optimizer és a scheduler `state_dict`-jét, valamint az aktuális epochot (és a legjobb validációs metrikát, ha a checkpointválasztás része). (3) $\eta_0 = \eta_{\max}$ és $\eta_T = \eta_{\min}$.

Visszautalás 21.1. Inicializálás és varianciaterjedés. Az inicializálás varianciaskálája és az LR nem független: a túl nagy kezdeti aktivációskálából adódó nagy gradiens ugyanazt a stabilitási határt feszíti, mint a túl nagy η. A két beállítást együtt kell diagnosztizálni.

Visszautalás 20.2. Finomhangolás és tanulási ráta. A finomhangolásnál a paramétercsoportonkénti LR ugyanazt a frissítési szabályt használja, csak η_t csoportonként más; a scheduler ilyenkor minden csoportra azonos ütemben skáláz.

Visszautalás 17.2. AlexNet és a kísérleti kontextus. Az AlexNet-kísérlet tanulsága, hogy a nagy hálózatok tanítási eljárása (LR, momentum, dropout, augmentáció) szétválaszthatatlan az elért teszteredmény kontextusától; a 17.2. AlexNet és a kísérleti kontextus alfejezetben közölt 15,3% top-5 hiba nem a schedule önmagában való dicsérete.

21.2.10 Rövid összefoglaló

A learning rate a gradiensfrissítés skálája; a $2/a$ korlát mutatja, hogy a lokális görbület felső határt szab a stabil lépésnek. A step decay, a cosine annealing és a warmup előre rögzített ütemek, a ReduceLROnPlateau ezzel szemben validációs visszajelzést használ. A scheduler állapota a reprodukálható tanítás része, ezért a checkpointnak a model, optimizer és scheduler state_dict mellett az epochot is tartalmaznia kell. Az epoch és a step különböző költség- és pályafogalom; azonos epochszám nem jelent azonos számítást. A labor két schedule kis CPU-futását azonos modell, optimizer, seed és split mellett hasonlítja össze.

21.2.11 Következő lecke

A 21.3. Dropout, súlyregularizálás és augmentáció egység a dropout, a súlyregularizálás és az augmentáció kérdését tárgyalja. Ezek a regularizáló eszközök a schedule-rel együtt hatnak a validációs pályára, ezért ott azt is vizsgáljuk, hogyan kell őket kísérletileg szétválasztani.

21.2.12 Felhasznált források

- [P02] Loshchilov, I. és Hutter, F. (2017; ICLR 2019 változat). *Decoupled Weight Decay Regularization*. https://arxiv.org/abs/1711.05101.
- [P07] Ioffe, S. és Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. PMLR 37:448–456. https://proceedings.mlr.press/v37/ioffe15.html.
- [P08] Santurkar, S., Tsipras, D., Ilyas, A. és Madry, A. (2018). *How Does Batch Normalization Help Optimization?* NeurIPS 2018. https://arxiv.org/abs/1805.11604.
- [D06] PyTorch contributors (2026). *torch.optim*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/optim.html. Hozzáférés: 2026-10-30.

21.3 Dropout, súlyregularizálás és augmentáció

21.3.1 Cél, előismeret és induló kérdés

Ez az egység három regularizáló eszközt választ szét: a dropoutot, a súlyregularizálást (weight decay) és az adataugmentációt, valamint tárgyalja a hozzájuk kapcsolódó label smoothingot. A három mechanizmus eltérő helyen avatkozik be: a dropout a modell belső aktivációit zajosítja, az L2-regularizálás determinisztikus büntetőtagot ad a célfüggvényhez; a weight decay erre egyenértékű L2-tag csak sima SGD-frissítésnél (a decoupled weight decay, például AdamW alatt, általában nem egyenértékű egy célfüggvényhez adott L2-taggal), az augmentáció pedig a bemeneti eloszlást módosítja. Előismeret a 21.2. Learning rate és ütemezés lépésméret- és ütemezésfogalma, a 7.2. Softmax és többosztályos cross-entropy softmax/cross-entropy célja, valamint a 20.3. Preprocessing, domain shift és súlylicenc input-szerződése. Input-szerződés alatt a bemenet alakjára, dtype-jára, értéktartományára és előfeldolgozására vonatkozó, a tanításra és a kiértékelésre egyaránt érvényes előírást értjük. Induló kérdés: adott egy $h = (2, -1, 3, 0)$ aktivációvektor és $p_{\text{keep}} = 0{,}5$ megtartási valószínűség. Ha train módban a maszk $m = (1, 0, 1, 1)$, mi az inverted dropout kimenete? Eval módban mi a kimenet, és miért?

21.3.2 Dropout: maszk, inverted scaling és train/eval viselkedés

Legyen az aktivációtensor $A \in \mathbb{R}^{N \times d}$, ahol N a batchméret, d az adott réteg kimeneti dimenziója. A dropout megtartási valószínűsége $p_{\text{keep}} \in (0, 1]$, kihagyási valószínűsége $p_{\text{drop}} = 1 - p_{\text{keep}}$. A **dropout maszk** $M \in \{0, 1\}^{N \times d}$ minden eleme egymástól független Bernoulli-eloszlású:

$$M_{ni} \sim \text{Bernoulli}(p_{\text{keep}}), \qquad \mathbb{P}(M_{ni} = 1) = p_{\text{keep}}.$$

A p_{keep} tehát annak valószínűsége, hogy egy elemet *megtartunk*; a *dropout rate* a $p_{\text{drop}} = 1 - p_{\text{keep}}$ kihagyási valószínűség. A PyTorch `nn.Dropout(p)` argumentuma a **kihagyási** valószínűség: `p` $= p_{\text{drop}}$, és train módban a megtartott elemeket $1/(1-p) = 1/p_{\text{keep}}$-pel szorozza (a `torch.nn.Dropout` dokumentációja szerint). A két konvenció összekeverése gyakori hiba; ellenőrző példa a megkülönböztetésre: $p_{\text{keep}} = 0{,}8$-hoz `nn.Dropout(0.2)` tartozik, a várható nullaarány 20%, a megtartott elemek szorzója 1,25, míg a téves `nn.Dropout(0.8)` az elemek 80%-át nullázná és a megtartottakat 5-tel szorozná. A $p_{\text{keep}} = 1$ határeset `nn.Dropout(0)`.

Inverted dropout esetén a kimenetet a maszk mellett p_{keep}-pel osztjuk:

$$\hat{A} = \frac{1}{p_{\text{keep}}} (M \odot A) = \frac{1}{1 - p_{\text{drop}}} (M \odot A),$$

elemenkénti szorzással. A $p_{\text{keep}} = 1$ határeset az identitás; a $p_{\text{keep}} = 0$ kizárt, mert nullával osztanánk. Az $\odot$ a Hadamard-szorzat, alakja $N \times d$.

Tétel (inverted dropout feltételes torzítatlansága). Legyen $A \in \mathbb{R}^{N \times d}$ rögzített aktivációtensor, és legyenek a maszk elemei független Bernoulli(p_{keep}) változók, függetlenek A-tól. Ekkor minden (n, i) indexpárra

$$\mathbb{E}[\hat{A}_{ni} \mid A] = A_{ni}.$$

Bizonyítás. A feltételes várható érték linearitását használjuk. Mivel A a feltételben rögzített, és M_{ni} független tőle:

$$\mathbb{E}[\hat{A}_{ni} \mid A] = \mathbb{E}\left[\frac{M_{ni}}{p_{\text{keep}}} A_{ni} \,\middle|\, A\right] = \frac{A_{ni}}{p_{\text{keep}}} \mathbb{E}[M_{ni} \mid A] = \frac{A_{ni}}{p_{\text{keep}}} p_{\text{keep}} = A_{ni}.$$

Az utolsó lépés a Bernoulli-eloszlás várható értékét használja. □

A tétel jelentése: az inverted dropout a train módban sem torzítja el a réteg kimenetének várható értékét, csak zajt ad hozzá. A klasszikus, *inverted* nélküli dropout train alatt $M \odot A$-t számol, eval alatt pedig $p_{\text{keep}}A$-t; az inverted változat ezt a skálakülönbséget train módba hozza, így eval mód nem igényel külön szorzást.

A **train/eval viselkedés** tehát: train módban minden forwardnál friss, független maszk generálódik; eval módban a modell a determinisztikus A aktivációt adja, ami az inverted skálázás miatt éppen a feltételes várható értéknek felel meg. A `nn.Module` `train()` és `eval()` állapota vezérli ezt. A BatchNorm szintén train/eval módot vált, de ott az ok a statisztikakezelés: a normalizáló réteg eval alatt futó átlagot és varianciát használ, a dropout viszont egyszerűen eltávolítja a véletlent. A kettő nem ugyanaz a mechanizmus, és nem szabad összemosni.

A dropout célja heurisztikus: minden neuron kénytelen más neuronokkal együttműködve hasznos reprezentációt adni, mert bármelyik társa véletlenszerűen hiányozhat. Ez redundáns, kevésbé egyedi tanulópéldára támaszkodó reprezentációt ösztönöz. A dropout nem növeli a hipotézisosztály expresszivitását; az approximációs kérdés (9.3. Expresszivitás és approximációs állítások) és a tanítási pálya itt is különválik.

A feltételes variancia egy rövid kiegészítő levezetéssel megmutatja, mekkora zajt visz a dropout a forwardba. Mivel $\text{Var}(M_{ni} \mid A) = p_{\text{keep}}(1 - p_{\text{keep}})$,

$$\text{Var}(\hat{A}_{ni} \mid A) = \frac{A_{ni}^2}{p_{\text{keep}}^2} \text{Var}(M_{ni}) = \frac{1 - p_{\text{keep}}}{p_{\text{keep}}} A_{ni}^2 = \frac{p_{\text{drop}}}{1 - p_{\text{drop}}} A_{ni}^2.$$

Minél kisebb p_{keep}, azaz minél nagyobb a PyTorch `p` argumentuma, annál nagyobb a relatív zaj. Ez hozzájárul ahhoz, hogy a train loss dropout alatt zajosabb legyen.

21.3.3 Weight decay: L2-tag és decoupled változat

A **weight decay** a loss-hoz adott négyzetes büntetőtag. Legyen $\mathcal{W}$ a regularizálandó súlyparaméterek halmaza; a bias-okat tipikusan kihagyjuk. A teljes célfüggvény

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\theta) + \frac{\lambda}{2}\sum_{W\in\mathcal{W}} \|W\|_F^2,$$

ahol $\lambda \geq 0$ a súlycsökkentési együttható, $\|\cdot\|_F$ a Frobenius-norma. A $\|\cdot\|_F$ tenzorokra is ugyanígy értendő: a konvolúciós kernel dimenzióitól függetlenül az összes elem négyzetösszegének gyöke, azaz a lapítás utáni Frobenius-norma. A tag gradiense minden $W \in \mathcal{W}$-re λW, ezért a sima SGD frissítése

$$W \leftarrow W - \eta(\nabla_W L_{\text{data}} + \lambda W).$$

Ez a lépés kis $\eta\lambda$ relatív arányban zsugorítja a súlyt, ezért nevezik weight decaynek.

Torch környezetben az `SGD(..., weight_decay= )` a fenti L2-tagot valósítja meg. Az Adam és AdamW között azonban lényegi különbség van. Ha a λW gradienst az Adam adaptív normalizálásán küldjük keresztül, a regularizáló tag hatása paraméterenként és időben változó; nem egyenlő azzal, mintha a súlyt minden lépésben $(1-\eta\lambda)$ tényezővel szoroznánk. A **decoupled weight decay** ezt a zsugorítást leválasztja az adaptív gradiensfrissítésről:

$$W \leftarrow (1-\eta\lambda)\, W - \eta \cdot (\text{adaptív gradienslépés a data loss alapján}).$$

A [P02] szerzői ezt a szétválasztást javasolják, és az így kapott AdamW-t mutatják be. A mechanizmusbeli különbség magából az Adam frissítési szabályából és a fenti definícióból levezethető, ezért azt a könyv itt bizonyított állításként kezeli; azt, hogy a decoupled változat mely feladatokon és mennyivel jobb, empirikus állításként nem terjesztjük ki.

A λ tipikus keresési tartománya gyakorlati heurisztikaként 10^{-5} és 10^{-2} között szokott lenni, de ez nem tétel: a megfelelő érték adattól és modelltől függ. A PyTorch `torch.optim` aktuális argumentumait a használt könyvtárverzió dokumentációjához és a telepített implementációhoz kell igazítani [D06].

21.3.4 Label smoothing

A **label smoothing** a többosztályos tanítási célt lágyítja. Az int64 osztálycímke $y \in \{0,\dots,K-1\}$ egy-hot vektora $e_y \in \{0,1\}^K$. A simított cél

$$q = (1-\varepsilon)\, e_y + \frac{\varepsilon}{K}\,\mathbf{1}, \qquad \varepsilon \in [0,1),$$

ahol $\mathbf{1} = (1,\dots,1) \in \mathbb{R}^K$. Ellenőrzés: az elemek nemnegatívak, és

$$\sum_{k=1}^{K} q_k = (1-\varepsilon) + \varepsilon \cdot \frac{K}{K} = 1,$$

tehát q valószínűségeloszlás. Például $K=3$, $\varepsilon = 0{,}1$ esetén a helyes osztályra $q_y = 0{,}9 + 0{,}1/3 \approx 0{,}9333$, a másik kettőre $q_k = 0{,}1/3 \approx 0{,}0333$. A cross-entropy cél $-\sum_k q_k \log s_k$, ahol

$s = \mathrm{softmax}(z)$ a logitsokból számolt predikció. Az $\varepsilon = 0$ határeset a szokásos cross-entropy az int64 cél felett; a modell kimenete továbbra is logits, a [D07] dokumentáció loss-szerződésének (logits-kimenet, int64 cél) megfelelően.

A smoothing hatása heurisztikus: a degenerált egy-hot célhoz tartozó veszteség infimuma nulla, de véges logitokkal nem érhető el; csak határértékben lenne nulla, ha a helyes logit minden másik fölé korlátlanul nőne. A simított cél ezt megakadályozza, mert a helyes osztályhoz tartozó $q_y = 1 - \varepsilon + \varepsilon/K < 1$, a többi osztályra pedig $\varepsilon/K > 0$; a minimum helyén $s = q$, azaz a logitok $z_k = \log q_k + \text{konstans}$, tehát a logitoknak véges különbséget kell tartaniuk. Ez gyakran nyugodtabb kalibrációt ad (a kalibráció a predikált valószínűségek és a tapasztalati találati arányok egyezése), de nem bizonyított, hogy minden adatkészleten javít; heurisztikaként kell kezelni. A torch `CrossEntropyLoss` a PyTorch 1.10-es kiadása óta kínál `label_smoothing` argumentumot; a fogalom implementálásához azonban elég a fenti képlet, így a levezetés nem függ az API-tól.

21.3.5 Data augmentation

A **data augmentation** a tanítási bemenetek véletlen, címkemegőrző transzformációja. A feltétel a label-megőrzés: a transzformált képhez ugyanaz az y cél tartozik, mint az eredetihez. A transzformációk részletes tárgyalása a 22.2. Címkemegőrző transzformációk egységben következik; itt a feltétel megfogalmazása a lényeg. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab osztályozásnál a címke az orientáció: 0 = függőleges (vertical), 1 = vízszintes (horizontal), 2 = átlós (diagonal). Ezért például enyhe eltolás, kis Gauss-zaj vagy fényességskálázás megengedett, a 90°-os forgatás viszont **nem** címkemegőrző: a függőleges és vízszintes osztályt felcseréli. Az átlós osztály ráadásul a generátorszerződés (a X. ellenőrző pont: önálló neurális modell és bizonyíték A. függelékében rögzített adatgenerálási előírás) szerint csak az egyik ferde irányt tartalmazza, így a tükrözés is új célértelmezést igényelne.

Az augmentáció tehát a data oldalon bővíti az effektív mintát; a dropout a modell belső aktivációit zajosítja; a weight decay determinisztikus paraméterbüntetést ad. Validációs és teszt spliten a kiértékeléshez nem alkalmazunk véletlen augmentációt, mert a metrikának reprodukálhatónak és a kiértékelt modellhez kötöttnek kell lennie.

21.3.6 Kézi számpélda és minimális kód

Vegyük az induló kérdés $h = (2, -1, 3, 0)$ vektorát, $p_{\text{keep}} = 0{,}5$ mellett. Ha $m = (1, 0, 1, 1)$, akkor az inverted dropout kimenete

$$\hat{h} = \frac{1}{0{,}5}\,(m \odot h) = 2 \cdot (2, 0, 3, 0) = (4, 0, 6, 0).$$

Egy másik maszkkal, $m' = (0, 1, 1, 1)$ esetén $\hat{h}' = 2 \cdot (0, -1, 3, 0) = (0, -2, 6, 0)$. A két realizáció átlaga $(2, -1, 6, 0)$, ami nem egyezik a $\mathbb{E}[\hat{h} \mid h] = (2, -1, 3, 0)$ várható értékkel; a várható érték nem egyetlen minta átlaga, hanem az eloszlás feletti átlag. A negyedik elem mindkét esetben nulla, mert $h_4 = 0$; a dropout a nulla aktivációt nem tudja zajossá tenni, ami a varianciaképlettel is összhangban van.

Eval módban a dropout maszkja elmarad, és mivel az inverted változat a skálázást a train fázisba tette, eval alatt nincs további korrekció: a kimenet a determinisztikus $h = (2, -1, 3, 0)$, függetlenül attól, milyen maszkot látott a modell tanítás közben. Ez az induló kérdés eval-válasza: nem $p_{\text{keep}}h$ és nem is valamilyen maszkolt változat, hanem maga a bemeneti aktivációvektor.

A minimalista NumPy-ellenőrzés:

```
import numpy as np

rng = np.random.default_rng(20261020)
h = np.array([2.0, -1.0, 3.0, 0.0], dtype=np.float32)
p_keep = np.float32(0.5)                  # megtartási valószínűség
mask = (rng.random(h.shape, dtype=np.float32) < p_keep).astype(np.float32)
out = mask * h / p_keep
print(f"mask={mask}, out={out}")
# Kimenet (NumPy 2.5.4, seed 20261020):
# mask=[0. 0. 1. 1.], out=[ 0. -0.  6.  0.]
# Ellenőrzés: a 4. elem mindig 0, a skála 1/p_keep = 2.
# A seedelt maszk itt (0,0,1,1), nem az illusztráció m=(1,0,1,1) maszkja.
# PyTorch-megfelelő: torch.nn.Dropout(p=1 - p_keep), mert ott p a kihagyás.
```

A seed, a dtype és a `p_keep` numerikus típusa együttesen rögzíti a maszk-generálást azonos NumPy-verzió és végrehajtási beállítások (például szálszám) mellett; más verzió vagy backend bitazonosságot nem ígér, ugyanúgy, ahogy 20.3. Preprocessing, domain shift és súlylicenc alfejezetben az input-szerződésnél láttuk.

21.3.7 Labor: egyenként változtatott regularizálók ablációja

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`; a corrupt tesztet ez a labor nem használja. A futtatás a szakaszban megadott szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor célja a három mechanizmus szétválasztása GlyphLab kis CPU-s környezetben. A **GlyphLab-szerződés** a függelék A. szakaszában rögzített adat-előírás: 180 jelenet × 2 nézet = 360 kép, 24 × 24, egycsatornás float32 [0,1] értéktartománnyal, int64 címkékkel (0 = függőleges, 1 = vízszintes, 2 = átlós), 240/60/60 képes train/val/test splittel (120/30/30 jelenet), a megfigyelési egység a `scene_id`. A rendszer: `data/images.npy` alakja $(360, 1, 24, 24)$ float32, `data/labels.npy` int64, a split a `data/metadata.csv`-ből jön. A modell egy kis CNN:

- Conv(1→16, 3×3, padding 1), ReLU, MaxPool(2);
- Conv(16→32, 3×3, padding 1), ReLU, MaxPool(2);
- Flatten, Linear($32 \cdot 6 \cdot 6$→3).

A bemenet kizárólag a képtensor; a maszkot ez a labor nem használja. A kód a következő munkakönyvtárból futtatható:

```
# munkakönyvtár: lab/
# futtatás: python regularization_ablation.py
import numpy as np, torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

SEED = 20261020
device = torch.device("cpu")

images = np.load("data/images.npy").astype(np.float32)      # (360,1,24,24)
labels = np.load("data/labels.npy").astype(np.int64)        # (360,)
meta = np.genfromtxt("data/metadata.csv", delimiter=",", dtype=None,
                     names=True, encoding="utf-8")
# A sorrendre nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
if not (len(meta) == len(images) == len(labels)):
    raise ValueError("a metadata.csv és a tömbök hossza eltér")
```

```
idx_all = meta["index"].astype(np.int64)
if not np.array_equal(np.sort(idx_all), np.arange(len(images))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
split = np.asarray(meta["split"]).astype(str)
train_idx = idx_all[split == "train"]
val_idx   = idx_all[split == "val"]
assert (len(train_idx), len(val_idx)) == (240, 60)

Xtr = torch.from_numpy(images[train_idx])
ytr = torch.from_numpy(labels[train_idx])
Xva = torch.from_numpy(images[val_idx])
yva = torch.from_numpy(labels[val_idx])

class TinyCNN(nn.Module):
    def __init__(self, p_drop=0.0):          # p_drop = nn.Dropout argumentuma
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.Flatten(), nn.Linear(32*6*6, 3))
        self.drop = nn.Dropout(p_drop) if p_drop > 0 else nn.Identity()
    def forward(self, x):
        z = self.features(x)
        z = self.drop(z)
        return self.head(z)

def evaluate(model, X, y):
    model.eval()
    with torch.no_grad():
        out = model(X)
        acc = (out.argmax(1) == y).float().mean().item()
    assert 0.0 <= acc <= 1.0
    return acc

def train_one(cfg, Xtr, ytr, Xva, yva):
    # cfg: (p_drop, wd, label_smooth, use_aug)
    # Minden konfiguráció azonos inicializálást és shuffle-sorrendet kap, és
    # mindegyik AdamW-t használ; a weight decay csak a wd értékében tér el.
    torch.manual_seed(SEED)
    rng = np.random.default_rng(SEED)
    model = TinyCNN(p_drop=cfg[0]).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=cfg[1])
    lossf = nn.CrossEntropyLoss(label_smoothing=cfg[2])
    ds = TensorDataset(Xtr, ytr)
    for ep in range(15):
        model.train()
        for xb, yb in DataLoader(ds, batch_size=32, shuffle=True):
            if cfg[3]:
                xb = xb + 0.02 * torch.randn_like(xb)  # kis zaj, címkemegőrző
                sh = rng.integers(-2, 3, size=2)       # egy batch-re érvényes eltolás
                xb = torch.roll(xb, shifts=tuple(sh), dims=(2, 3))
            opt.zero_grad()
            loss = lossf(model(xb), yb)
            loss.backward()
            opt.step()
    return evaluate(model, Xva, yva)

configs = {
    "baseline":      (0.0, 0.0,  0.0, False),
    "+dropout":      (0.5, 0.0,  0.0, False),
    "+weight_decay": (0.0, 1e-4, 0.0, False),
    "+label_smooth": (0.0, 0.0,  0.1, False),
    "+augment":      (0.0, 0.0,  0.0, True),
```

```
}
for name, cfg in configs.items():
    acc = train_one(cfg, Xtr, ytr, Xva, yva)
    print(f"{name:14s} val_acc={acc:.3f}")
```

A dropout a `features` kimenetére, azaz a $(N, 32, 6, 6)$ jellemzőtensorra hat, amely mintánként $32 \cdot 6 \cdot 6 = 1152$ elemből áll; $p_{\text{drop}} = 0{,}5$ mellett várhatóan 576 elem marad meg, és a megtartottak szorzója 2. A fej elé, a konvolúciós blokk után tesszük, mert így a teljes jellemzőtér elemein, nem pedig a bemeneti képpontokon maszkol; ez a konvolúciós hálók szokásos elhelyezése, de nem tétel, és a labor `+dropout` ágának értelmezése erre a beállításra érvényes. A kiértékelés a legjobb validációs checkpoint nélkül, a 15. epoch végi paramétereivel történik; ez csak mechanizmus-probának való, nem modellválasztásnak. Minden konfiguráció ugyanazt az AdamW optimalizálót használja, a baseline `weight_decay=0` mellett fut, ezért a `+weight_decay` sor csak a λ együtthatót változtatja meg; így a hatás nem keveredik az optimalizáló cseréjével. Az augmentációs ág batch-szintű eltolást alkalmaz: a `torch.roll` a teljes minibatchre ugyanazt a kétdimenziós eltolást használja, és körbeforgat, ezért a kép széle átcsúszik a másik oldalra; a játékos laborban ez elfogadható, éles rendszerben szélső kitöltés vagy maszkolás kellene. Az alábbi táblázat ezen a környezeten mért értékeket mutat (Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`, seed 20261020); egyetlen seedből a különbségek nem általánosíthatók, a nyers val_acc értéket a fenti szkript naplózza.

Konfiguráció	val_acc (futtatott)	Értelmezés
baseline	0,867	erős kiindulási pont ezen a könnyű szintetikus adaton
+dropout	0,800	itt ront, mert nincs erős túlillesztés
+weight_decay	0,867	az alapvonallal azonos
+label_smooth	0,867	az alapvonallal azonos
+augment	0,883	kis javulás azonos seed mellett

A **corrupt tesztet** (`data/corrupted_test.npy`: a GlyphLab-szerződés szerinti 60 kép a teszt sorrendjében, a teszt címkéivel) itt nem használjuk: a modellválasztás a val spliten történik, a corrupt teszt csak befagyasztott modell után értékelhető, ahogy a GlyphLab-szerződés előírja.

21.3.8 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem ugyanaz a train/eval viselkedés oka a dropoutnál és a BatchNormnál. Adjon egy-egy konkrét példát a viselkedésre. *Beadandó:* legfeljebb 15 soros magyarázat, a két állapot különválasztott okaival.

F2 (levezetési, nehézség 2). Adott $h \in \mathbb{R}^d$ rögzített vektor, $p_{\text{keep}} \in (0, 1]$. Vezesse le az inverted dropout kimenet feltételes varianciáját: $\operatorname{Var}(\hat{h}_j \mid h) = \frac{1-p_{\text{keep}}}{p_{\text{keep}}} h_j^2$. Nevezze meg a felhasznált varianciatulajdonságot, és ellenőrizze, hogy $p = 1$ esetén a variancia nulla. *Beadandó:* minden lépés, feltételek és szélsőeset-kontroll.

F3 (implementációs, nehézség 1). Írjon saját `inverted_dropout(x, p_keep, rng)` függvényt NumPy-ban, amely float32 bemenetet, p_{keep} megtartási valószínűséget és seedelt `Generator` objektumot fogad, maszkot generál, elvégzi az $1/p_{\text{keep}}$ skálázást, és shape-asserttel ellenőrzi a kimenetet. Tesztelje a $p_{\text{keep}} = 0{,}5$, $x = (2, -1, 3, 0)$ példán; a seed legyen 42. Adja meg azt is, milyen argumentumú `nn.Dropout` felel meg a függvénynek. *Beadandó:* teljes függvénykód, futási kimenet, shape-ellenőrzés, magyarázat a $p_{\text{keep}} = 1$ határeset viselkedéséről.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet egy hibás dropout-implementációt mutat. Azonosítsa a hibákat, mondja meg, milyen következményük van train és eval módban, javítsa, és adja meg a javítás utáni ellenőrző kimenetet a $p_{\text{keep}} = 0{,}5$, $h = (1, 2)$ esetre train és eval módban is.

```
class WrongDropout:
    def __init__(self, p_keep):
        self.p_keep = p_keep
    def forward(self, x, rng):
        mask = (rng.random(x.shape) < self.p_keep).astype(float)
        return mask * x
```

Beadandó: hiba oka, következmény, javított kód, a javítás utáni egy konkrét kimenet.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen olyan ablatív kísérletet GlyphLab-on, amely a dropout, a weight decay és az augmentáció hatását **egyenként** méri anélkül, hogy a corrupt tesztet modellválasztásra használná. Határozza meg a baseline modellt, a megfigyelési egységet, a splitet, a seedet, a metrikát, a rögzített tényezőket és az érvényességi korlátot. *Beadandó:* protokollterv és indoklás, legfeljebb 20 sor.

21.3.9 Megoldások és indoklás

F1. A dropout train módban friss Bernoulli-maszkot alkalmaz, és az inverted skálázással a feltételes várható értéket torzítatlanul hagyja; eval módban a maszkot eltávolítja, mert a determinisztikus A éppen a várható érték. A BatchNorm viszont train módban a minibatch átlagával és varianciájával normalizál, és futó statisztikát frissít; eval módban ezeket a futó statisztikákat használja, hogy a kimenet ne függjön a batch összetételétől. A közös pont a modulállapot-váltás, az ok és a mechanizmus azonban különböző: véletlen eltávolítás, szemben a statisztikai normalizálás batchfüggetlenítésével.

F2. A varianciatulajdonság: $\operatorname{Var}(aX + b \mid \mathcal{F}) = a^2 \operatorname{Var}(X \mid \mathcal{F})$, ahol $\mathcal{F}$ a feltételi σ-algebra, a, b pedig $\mathcal{F}$-mérhető (a feltétel mellett konstans) mennyiségek. Itt $a = h_j/p_{\text{keep}}$, $b = 0$, $X = M_j$. A Bernoulli-variancia $\operatorname{Var}(M_j) = p_{\text{keep}}(1 - p_{\text{keep}})$. Ezért

$$\operatorname{Var}(\hat{h}_j \mid h) = \left(\frac{h_j}{p_{\text{keep}}}\right)^2 p_{\text{keep}}(1 - p_{\text{keep}}) = \frac{1 - p_{\text{keep}}}{p_{\text{keep}}}\, h_j^2.$$

Szélsőeset: $p_{\text{keep}} = 1$ mellett $1 - p_{\text{keep}} = 0$, a variancia nulla, a maszk konstans 1, a kimenet determinisztikus. Feltételek: $p_{\text{keep}} > 0$, $M_j \sim \text{Bernoulli}(p_{\text{keep}})$, M_j független a feltételtől. Az $h_j = 0$ speciális esetben a variancia nulla, a nulla aktiváció nem zajosodik.

F3. Függvény:

```
import numpy as np

def inverted_dropout(x, p_keep, rng):
    assert x.dtype == np.float32, "float32 bemenet kötelező"
    assert np.isscalar(p_keep) and 0.0 < float(p_keep) <= 1.0
```

```
    mask = (rng.random(x.shape, dtype=np.float32) < np.float32(p_keep)).astype(np.float32)
    out = mask * x / np.float32(p_keep)
    assert out.shape == x.shape and out.dtype == np.float32
    return out

rng = np.random.default_rng(42)
x = np.array([2.0, -1.0, 3.0, 0.0], dtype=np.float32)
print(inverted_dropout(x, 0.5, rng))
# A labor futtatása (NumPy 2.5.4) ezt adja: [ 4. -0.  0.  0.]; a -0. onnan ered, hogy a -1*0
↪  szorzat float32-ben negatív előjelű nullát ad
```

A `p_keep` tetszőleges numerikus skalár lehet (Python `float` vagy NumPy lebegőpontos), az `np.isscalar` és a `float()` konverzió ezt engedi át, a `float` típusra szűkítő `isinstance` viszont a `np.float32` értéket tévesen elutasítaná; a bemenet float32. A PyTorch-megfelelő `nn.Dropout(p=1 - p_keep)`; az osztás float32-ben végzett, ezért shape és dtype garantált. A $p_{\text{keep}} = 1$ határesetben (PyTorchban `nn.Dropout(0)`) a maszk minden eleme 1, a kimenet azonos a bemenettel, de a maszkgenerálás költsége megmarad; implementációban érdemes lehet $p = 1$ esetén rövidre zárni, ez azonban hatékonysági döntés, nem helyességi feltétel.

F4. Két hiba van. Egyrészt a függvény nem skáláz $1/p_{\text{keep}}$-pel, ezért train módban a kimenet várható értéke $p_{\text{keep}}\,x$, nem x. Másrészt nincs train/eval kapcsolója: mivel az osztálynak nincs `training` állapota és a `forward` nem kap módjelzőt, eval módban is maszkol, holott eval alatt a determinisztikus x-et kell visszaadni. A két hiba együtt a tanult súlyok skáláját is eltolja: a tanítás a lefojtott aktivációkhoz igazodik, és eval alatt a réteg kimenetei vagy túl nagyok, vagy véletlenül maszkoltak lesznek. Javítás:

```
class FixedDropout:
    def __init__(self, p_keep):
        self.p_keep = p_keep
    def forward(self, x, rng, training=True):
        if not training:                # eval: nincs maszk, nincs skálázás
            return x
        mask = (rng.random(x.shape) < self.p_keep).astype(float)
        return mask * x / self.p_keep    # inverted scaling
```

Ellenőrzés a $p_{\text{keep}} = 0{,}5$, $x = (1, 2)$, seed 42 esetén: az `rng.random(x.shape)` sorozat [0,77395605, 0,43887844], a maszk $(0, 1)$, a javított train kimenet $(0, 4)$, az eval kimenet $(1, 2)$. A hibás változat train kimenete $(0, 2)$ (hiányzik a skálázás), eval módban pedig nincs ága, ezért ott is maszkol.

F5. Tervvázlat: a baseline a labor TinyCNN-je, Adam lr 10^{-3}, seed 20261020, batch 32, 15 epoch, metrika a val split accuracy. A megfigyelési egység a jelenet (`scene_id`); a val split 60 képe 30 jelenetből áll. A modellválasztásnál a val loss a checkpoint-kritérium. Rögzített tényezők: split a metadata.csv-ből, input NCHW float32 [0,1], nincs corrupt teszt a választásban. Egyetlen változó tényező van futásonként: dropout `nn.Dropout(0.5)`, azaz $p_{\text{drop}} = p_{\text{keep}} = 0{,}5$, vagy AdamW $\lambda = 10^{-4}$, vagy zaj/eltolás augmentáció. Mindegyik futást legalább 5 seeddel ismételjük, a változatok között azonos seedekkel; a pontbecslés a seedek átlaga, a seedek közötti szórást leíró statisztikaként közöljük (nem keverjük mediánnal). A döntés alapja a jelenet szintű páros különbség bootstrap CI-je a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint: ha a 95%-os CI kizárja a nullát, állítunk különbséget, ha tartalmazza, nem. Korlát: a GlyphLab toy adat, az eredmény nem általánosít természetes képre; a corrupt tesztet csak a kiválasztott befagyasztott modell páros kiértékelésére használjuk.

Hipotézis: a vizsgált regularizáló javítja a val accuracyt a baseline-hoz képest. **Seedek, döntési küszöb és erőforrás:** az öt seed értéke 20261020–20261024, minden változatban ugyanaz; a döntési küszöb a jelenet szintű páros különbség 95%-os bootstrap CI-je, és a gyakorlati

jelentéstételi határ 1 százalékpont. Erőforrás: CPU, `float32`, három változat × öt seed = 15 futás, TinyCNN-en néhány perc; a futásidőt a napló rögzíti. ### Tipikus tévedés ellenpéldával

A gyakori tévedés: *„A dropout mindig javítja a validációt."* Ez nem igaz. A dropout csökkenti az effektív kapacitást és növeli a tanítójel zaját, ezért ha a modell eleve underfit, a hozzáadás tovább ront. Szemléltető ellenpélda: GlyphLab-on egyetlen lineáris fej közvetlenül a pixeleken, 64 rejtett neuron nélkül, mindössze $24^2 \cdot 3 + 3$ paraméterrel; a tanítópontosság alacsony, a modell nem overfit. Ha erre dropoutot kapcsolunk $p = 0{,}5$-tel, a tanító és a validációs pontosság is csökken, mert a tanítójelet is elfojtjuk, miközben nem volt redundáns kapacitás, amit regularizálni kellett volna. A dropout tehát modellkapacitáshoz és adatskálához mért eszköz; a „mindig segít" állítás cáfolata már az underfit eset.

21.3.10 Önellenőrzés

Ellenőrizze a tétel lépéseit: (1) a feltételes várható érték linearitása alkalmazható-e, ha a maszk független a feltételtől? (2) Az eredmény $N \times d$ alakú-e mindkét oldalon? (3) A varianciaképletben $p_{\text{keep}} = 1$ (`nn.Dropout(0)`) esetén a maszk szórása nulla, a kimenet determinisztikus? (4) A weight decay célfüggvényében a $\lambda/2$ konvenció gradiense valóban λW? (5) A label smoothing célvektor összege 1 marad-e minden $\varepsilon \in [0, 1)$ esetén? Ha bármelyik kérdésre nemleges a válasz, a megfelelő levezetéshez térjen vissza, és keresse meg az indexhibát.

Rövid válaszok. (1) Igen: a maszk függetlensége miatt $\mathbb{E}[M \odot H] = p_{\text{keep}} H$, és a linearitás nem igényel többet. (2) Igen: a maszk és az aktiváció is $N \times d$ alakú, ezért az elemenkénti szorzat alakja változatlan. (3) Igen: $p_{\text{keep}} = 1$ esetén a maszk minden eleme 1, a szórása nulla, a kimenet h, train és eval módban azonos. (4) Igen: $\partial(\lambda/2 \, \|W\|^2)/\partial W = \lambda W$. (5) Igen: a cél $((1-\varepsilon)y + \varepsilon/K)$ összege $(1-\varepsilon) + \varepsilon = 1$, és $\varepsilon = 1$ esetén is 1.

Korábbi fogalom felidézése: a 9.3. Expresszivitás és approximációs állítások approximációs tételből azt láttuk, hogy az expresszivitás nem ígér taníthatóságot. A dropout és a weight decay az optimalizálási pályát és az effektív hipotézisteret alakítja, nem az approximációs kapacitás elméleti maximumát; a kettő különválasztása itt is kötelező.

21.3.11 Rövid összefoglaló

A dropout elemenkénti Bernoulli-maszkkal nulláz aktivációkat train módban, és az inverted skálázás révén a feltételes várható érték torzítatlan marad; eval módban a maszk eltűnik. A weight decay a loss-hoz adott $\frac{\lambda}{2}\|W\|_F^2$ tag, amelynek gradiense λW; az AdamW decoupled változata ezt a zsugorítást leválasztja az adaptív lépésről. A label smoothing a célvektort keveri az egyenletes eloszlással, az augmentáció címkemegőrző véletlen transzformációkkal bővíti a bemenetet. A három mechanizmus a modell belsejében, a célfüggvényben és a data oldalon hat; hatásuk nem univerzális törvény, hanem kísérletileg ellenőrizendő heurisztika. A nyitó példában a train módú kimenet $(4; 0; 6; 0)$, eval módban pedig $(2; -1; 3; 0)$.

21.3.12 Következő lecke

A 21.4. Osztályarány, súlyozás és céleloszlás egység az osztályarány és céleloszlás kérdését tárgyalja: a label smoothing ott csak összehasonlítási pontként tér vissza, mert a label smoothing nem osztálysúlyozás, hanem a célvektor lágyítása; a súlyozás a veszteség osztályonkénti hozzájárulását változtatja meg, a label smoothing a cél eloszlását.

21.3.13 Felhasznált források

- [P02] Loshchilov, I. és Hutter, F. (2017). *Decoupled Weight Decay Regularization*. https://arxiv.org/abs/1711.05101.
- [D06] PyTorch contributors (2026). *torch.optim*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/optim.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- PyTorch contributors (2026). *torch.nn.Dropout*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Dropout.html. Hozzáférés: 2026-10-30.

21.4 Osztályarány, súlyozás és céleloszlás

21.4.1 Cél, előismeret és induló becslés

A 21.3. Dropout, súlyregularizálás és augmentáció egységben a dropout, a weight decay és az augmentáció mellett a label smoothing is megjelent mint a cél eloszlásának módosítása (Szegedy et al., 2016). Itt azt vizsgáljuk, hogyan befolyásolja a tanulást, ha az egyes osztályok gyakorisága a tanítóhalmazban erősen eltér. A függelék A. szakaszában leírt GlyphLab train halmazában mindhárom osztályból 80–80 kép van, tehát kiegyensúlyozott. Valós alkalmazásokban azonban gyakori, hogy a „vertical” (függőleges) osztály a megfigyelések 90%-át teszi ki, a „horizontal” (vízszintes) és „diagonal” (átlós) pedig csak 5–5%-ot. Ilyenkor a naiv empirical risk minimization a többségi osztályra optimalizál, mert a hibák átlagában a gyakori osztály dominál.

Előismeret: a cross-entropy és a softmax definíciója a 7.2. Softmax és többosztályos cross-entropy alfejezetből, a tanítási célfüggvény a 6.1. Objektív, reguláris tag és gradienslépés alfejezetből, a fair comparison és a seed–protokoll a 17.4. Architektúra összehasonlítása kontrollált keretben alfejezetből, valamint az aktivációk és a gradiensskálázás a 9.4. Aktivációk és gradiensáramlás alfejezetből.

Induló kérdés. Tegyük fel, hogy egy tanítóhalmazon az osztályarány 0,8, 0,1, 0,1. Mekkora súlyokat kell választanunk ahhoz, hogy a veszteség az osztályonkénti hibák súlyozatlan átlaga legyen? A választ a 3. szakasz levezetése adja meg; induló becslésként a ritka osztályok súlya a gyakori osztály súlyának nyolcszorosa.

21.4.2 Definíciók és jelölés

Legyen K az osztályok száma, $y_i \in \{0, \dots, K-1\}$ a cél, és $\hat{p}_k(x_i; \theta)$ a modell által adott posterior valószínűség a k-adik osztályra. A **posterior** itt a modell feltételes címkevalószínűsége, $\hat{p}_k(x; \theta) \approx P_\theta(Y = k \mid X = x)$, amelyet a softmax a logitokból állít elő; a $\hat{p}_k(x; \theta)$ tehát példától és paramétertől függ, és nem tévesztendő össze a tanítóhalmaz empirikus osztálygyakoriságával, amelyet az alábbiakban $\hat{\pi}_k$ jelöl. A többosztályos cross-entropy veszteség egyetlen megfigyelésre

$$\ell_i(\theta) = -\log \hat{p}_{y_i}(x_i; \theta).$$

A modell logitokat ad ki, a softmax a loss belsejében történik a [D07] szerződés szerint; a cél int64 osztályindex.

Osztályarány és empirikus gyakoriság. A tanítóhalmaz N megfigyeléséből az n_k darab tartozik a k osztályhoz, $n_k > 0$ minden k-ra. Az empirikus osztálygyakoriságot itt $\hat{\pi}_k$ jelöli (nem $\hat{p}_k$), hogy ne keveredjen a modell posteriorjával:

$$\hat{\pi}_k = \frac{n_k}{N}, \qquad \sum_{k=0}^{K-1} \hat{\pi}_k = 1.$$

A populációszintű $p_k = P(Y = k)$ értékek általában ismeretlenek; a tanítóhalmazbeli $\hat{\pi}_k$ a becslésük. A továbbiakban N a tanítóhalmaz elemszámát jelöli: az elméleti levezetésben a teljes tanítóhalmazét, a laborban a szűkített, 100 képes halmazét ($N = 100$); a batch méretét N_{batch}, az értékelt (validációs vagy teszt) halmazét M jelöli.

Súlyozott cross-entropy, gradient scaling konvenció. Legyen $w = (w_0, \dots, w_{K-1}) \in \mathbb{R}^K_{>0}$ pozitív osztálysúly-vektor. A batchre vett súlyozott empirikus veszteség az

$$L_W(\theta) = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} w_{y_i} \ell_i(\theta)$$

képlet, ahol a nevező a batch mérete, nem a súlyok összege. Ez a konvenció azt jelenti, hogy a gradiensben az $y_i = k$ osztályú példák w_k-szoros szorzót kapnak. Ezt a fejezetben **gradient scaling konvenciónak** nevezzük, mert a súlyok a gradiensben szorzóként jelennek meg; ez nem azonos a vegyes pontosságú tanítás loss-skálázásával, amely pusztán numerikus ábrázolási célú. A PyTorch `CrossEntropyLoss(weight=..., reduction='mean')` ezzel szemben a

$$L_{\text{torch}}(\theta) = \frac{\sum_{i=1}^{N_{\text{batch}}} w_{y_i} \ell_i(\theta)}{\sum_{i=1}^{N_{\text{batch}}} w_{y_i}}$$

súlyozott átlagot számítja. Rögzített batchre a két mennyiség konstans szorzóban tér el: $L_{\text{torch}} = \frac{N_{\text{batch}}}{\sum_i w_{y_i}} L_W$, ahol a szorzó a batch összetételétől függ. A teljes tanítóhalmaz egy meneten át vett összegénél a nevező konstans, ezért ott a két cél ugyanazt a θ-optimumot adja; minibatch-SGD esetén viszont a batchönként változó szorzó miatt a gradiens léptéke és az optimalizáló pályája is eltér. Az optimumra vonatkozó azonosság tehát rögzített batch-súlyösszeg (például teljes batch, vagy minden lépésben ugyanakkora összsúlyú batch) mellett érvényes, a gradiens léptéke és az optimáló beállítása viszont ettől függetlenül különbözik. A **reduction** argumentum a loss batch feletti összesítésének módja: `'mean'` a fenti hányadost, `'sum'` a számlálót, `'none'` pedig a $w_{y_i}\ell_i$ értékek vektorát adja vissza. A fejezet a gradient scaling konvenciót használja, és minden numerikus kódban explicit módon megadja a reductiont.

Sampler. A sampler a batch összetételét változtatja meg: nem a tanítóhalmaz eredeti $\hat{\pi}_k$ eloszlásával mintavételez, hanem egy megcélzott q_k osztályeloszlással. A `WeightedRandomSampler` működése példaszintű: minden i példához egy $s_i > 0$ húzási súlyt rendel, és a példát $s_i / \sum_j s_j$ valószínűséggel húzza ki visszatevéssel; az osztály szintjén így kialakuló húzási arány $q_k \propto \sum_{i:y_i=k} s_i$. A leggyakoribb választás a class-balanced sampler, ahol $q_k = 1/K$ minden k-ra; ezt az $s_i \propto 1/\hat{\pi}_{y_i}$ választás adja, mert ekkor az osztály összes példájának húzási súlya $\propto n_k \cdot 1/\hat{\pi}_k = N$, azaz osztálytól függetlenül ugyanakkora. A mintavételezett batchben az osztályarány ezért közel egyenletes lesz; a húzás továbbra is véletlen, ezért egy konkrét minibatchben ettől eltérhet.

Focal loss, áttekintés. A focal loss a példánkénti nehézséget súlyozza, nem az osztályt:

$$\mathrm{FL}_i(\theta) = -(1 - \hat{p}_{y_i}(x_i;\theta))^\gamma \log \hat{p}_{y_i}(x_i;\theta), \qquad \gamma \geq 0.$$

Ha $\gamma = 0$, visszakapjuk a standard cross-entropy-t. Ha $\gamma > 0$, akkor a magas konfidenciájú, könnyen osztályozott példák relatív súlya csökken, a bizonytalan, nehéz példáké nő. A focal loss önmagában nem osztályarány-korrekció, de kombinálható osztálysúlyozással.

Kalibráció. A modell akkor jól kalibrált, ha a $\hat{p}_k$ prediktált valószínűségek közelítik a tényleges feltételes gyakoriságot. Egy gyakorlati mérőszám az Expected Calibration Error: a validációs halmaz M példáját a maximális prediktált konfidencia szerint B intervallumra bontjuk, majd binenként a konfidenciaátlag és a pontosság abszolút különbségét az elemszám arányával súlyozzuk:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|D_b|}{M} \left|\mathrm{acc}(D_b) - \mathrm{conf}(D_b)\right|.$$

A súlyozott loss vagy a sampler megváltoztatja azt az eloszlást, amelyre a modell poszteriorjait illeszti; ezért a kalibrációt a tényleges teszteloszláson, nem az eltolt tanítóhalmazon kell mérni.

Metrikák. Az értékelt halmazban legyen m_k a k osztályú példák száma és TP_k a helyesen k-nak osztályozott k osztályú példák száma. A k osztály **recallja** (érzékenysége) $R_k = TP_k/m_k$: az adott osztály példáinak mekkora részét találja el a modell. A **balanced accuracy** az osztályonkénti recallok átlaga, $\mathrm{BA} = \frac{1}{K}\sum_{k=0}^{K-1} R_k$; kiegyensúlyozatlan értékelőhalmazon ez nem domborítja ki a többségi osztályt, szemben a nyers accuracyvel. A k osztály F1-értéke $F1_k = \frac{2TP_k}{2TP_k+FP_k+FN_k}$, ahol FP_k a tévesen k-nak jelölt, FN_k pedig a k osztályból kihagyott példák száma; a **macro-F1** ezek súlyozatlan átlaga, $\frac{1}{K}\sum_k F1_k$, tehát minden osztály azonos súllyal szerepel, függetlenül a gyakoriságától.

21.4.3 Levezetés: milyen empirikus kockázatot minimalizál a súlyozás?

Állítás (weighted ERM célfüggvény). Legyen a tanítóhalmaz $D = \{(x_i, y_i)\}_{i=1}^{N}$, $y_i \in \{0, \dots, K-1\}$, az osztályonkénti elemszám $n_k > 0$. Definiáljuk az osztályon belüli empirikus átlagos veszteséget

$$\hat{R}_k(\theta) = \frac{1}{n_k} \sum_{i : y_i = k} \ell_i(\theta).$$

A w_k osztálysúlyokkal definiált súlyozott empirikus kockázat

$$\hat{R}_W(\theta) = \frac{1}{N} \sum_{i=1}^{N} w_{y_i} \ell_i(\theta).$$

Ekkor a

$$w_k = \frac{1}{K\hat{\pi}_k} = \frac{N}{K n_k}$$

választás mellett

$$\boxed{\hat{R}_W(\theta) = \frac{1}{K} \sum_{k=0}^{K-1} \hat{R}_k(\theta).}$$

Levezetés. Bontsuk az összeget osztályonként:

$$\hat{R}_W(\theta) = \frac{1}{N}\sum_{k=0}^{K-1} w_k \sum_{i:y_i=k} \ell_i(\theta) = \sum_{k=0}^{K-1} \frac{n_k}{N}\, w_k \, \frac{1}{n_k} \sum_{i:y_i=k} \ell_i(\theta) = \sum_{k=0}^{K-1} \hat{\pi}_k w_k \hat{R}_k(\theta).$$

Helyettesítsük be a fenti w_k értéket:

$$\sum_{k=0}^{K-1} \hat{\pi}_k \frac{1}{K\hat{\pi}_k} \hat{R}_k(\theta) = \frac{1}{K}\sum_{k=0}^{K-1} \hat{R}_k(\theta).$$

A levezetés tisztán definíciós átrendezés; nincs szükség valószínűségszámítási konvergenciatételre. A feltételek: $n_k > 0$ minden k-ra, $w_k > 0$, és minden ℓ_i véges. A populációszintű analóg állítás ugyanígy igaz: ha a valódi $p_k > 0$ ismert, akkor $w_k = 1/(Kp_k)$ mellett a súlyozott kockázat a $\frac{1}{K}\sum_k \mathbb{E}_{X|Y=k}[\ell(X,k;\theta)]$ class-balanced kockázatot adja.

Milyen kockázatot minimalizálunk tehát? A súlyozatlan empirikus kockázat $\hat{R}(\theta) = \sum_k \hat{\pi}_k \hat{R}_k(\theta)$ a mintában szereplő arányok szerinti átlag, így a többségi osztály hibája dominálja. Az iménti w_k választással a minimalizált célfüggvény az osztályonkénti hibák *súlyozatlan* átlaga: minden osztály azonos $1/K$ súllyal szerepel. Ez a döntés akkor indokolt, ha a felhasználási szerződés szerint az osztályok egyenlő fontosságúak, például a GlyphLab kiegyensúlyozott tesztjén. Ha a teszt eloszlása valójában ugyanaz az eltolt mix, akkor a súlyozatlan ERM a helyes cél; a súlyozás ekkor a többségi osztályhoz tartozó példák hozzájárulását a teszt eloszlásához képest alulsúlyozza.

Fontos tehát kimondani: **a class-balanced súlyozás nem mindig javítja a kalibrációt vagy a valós pontosságot; a választott súlyozás a választott kockázatfüggvény.** A levezetés nem állítja, hogy a súlyozás javítja a generalizációt, csak azt, hogy mit tesz a célfüggvénnyel.

21.4.4 Eredeti kézi kis példa

Legyen $K = 3$, $N = 10$, $n = (8, 1, 1)$, tehát $\hat{\pi} = (0{,}8,\ 0{,}1,\ 0{,}1)$. A class-balanced súlyok

$$w_0 = \frac{10}{3 \cdot 8} = \frac{5}{12} \approx 0{,}4167, \qquad w_1 = w_2 = \frac{10}{3 \cdot 1} = \frac{10}{3} \approx 3{,}333.$$

Tegyük fel, hogy egy adott θ mellett az osztályonkénti empirikus veszteségek

$$\hat{R}_0 = 0{,}2, \qquad \hat{R}_1 = 1{,}0, \qquad \hat{R}_2 = 1{,}5.$$

A súlyozatlan empirikus kockázat

$$\hat{R} = 0{,}8 \cdot 0{,}2 + 0{,}1 \cdot 1{,}0 + 0{,}1 \cdot 1{,}5 = 0{,}16 + 0{,}10 + 0{,}15 = 0{,}41.$$

A súlyozott empirikus kockázat a fenti w-vel

$$\hat{R}_W = \frac{1}{3}(0{,}2 + 1{,}0 + 1{,}5) = \frac{2{,}7}{3} = 0{,}90.$$

A különbség jól mutatja, hogy a súlyozott cél a ritka osztályok hibáját sokkal erősebben bünteti: a $\hat{R}_W$ értéke több mint kétszerese a súlyozatlannak, mert a két ritka osztály átlagos hibája nagy. Ha a modell a többségi osztályra specializálódik, a súlyozatlan cél kicsi, a súlyozott nagy marad; a súlyozott optimum tehát más kompromisszumot keres.

21.4.5 Labor: mesterségesen eltolt class-mix

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. A futtatás a szakaszban megadott szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor célja, hogy a class-balanced súlyozás, a sampler és a kiinduló súlyozatlan baseline hatását mérje egy mesterségesen eltolt tanítóhalmazon. A tanítási eljárás az alábbi, önállóan futtatható program, amely a rögzített környezetben lefutott; a hallgató a saját környezetében megismételheti, és a nyers eredményeket a kísérleti naplójában rögzíti.

Adat. A GlyphLab `train` felosztásában osztályonként 40 jelenet, jelenetenként 2 nézet, azaz 80 kép van. A laborhoz csoportszinten választunk osztályonként rendre 40, 5 és 5 teljes jelenetet: így 100 kép marad (80 függőleges, 10 vízszintes, 10 átlós), azaz 0,8/0,1/0,1 az osztályarány. A többi 140 train képet kizárjuk ebből a laborkísérletből. Azonos `scene_id`-jú két nézet nem kerülhet különböző részhalmazba. A validációs halmaz változatlanul a teljes 60 kép (20–20–20 osztályonként), így kiegyensúlyozott. Ezen a halmazon a nyers accuracy és a balanced accuracy definíció szerint egyenlő, ezért a laborban a nyers accuracyt a validációs halmaz tanítóeloszláshoz újrasúlyozott (0,8/0,1/0,1) nézetén is kiszámítjuk: a mintákat a $\hat{\pi}_k^{\text{train}}/\hat{\pi}_k^{\text{val}}$ aránnyal súlyozzuk, majd a súlyok összegével osztunk (a kiegyensúlyozott validáción $\hat{\pi}_k^{\text{val}} = 1/3$). A balanced accuracyt a teljes kiegyensúlyozott validáción mérjük; a két szám csak az eltérő nézet miatt térhet el.

Modell és recept. Egy kétrétegű kis CNN, a 21.3. Dropout, súlyregularizálás és augmentáció laborjának egyszerűsített változata: Conv(1→8, 3×3), ReLU, MaxPool(2), Conv(8→16, 3×3), ReLU, MaxPool(2), Flatten, Linear($16 \cdot 4 \cdot 4 \to 3$). A konvolúciók padding nélküliek, ezért a 24×24 bemenet $24 \to 22 \to 11 \to 9 \to 4$ módon csökken; padding=1 esetén a lineáris réteg bemenete $16 \cdot 6 \cdot 6$ lenne. A batch méret 24, epoch 30, AdamW (lásd [P02]) tanulási ráta 10^{-3}, `weight_decay=0`; minden seedhez külön `torch.Generator` tartozik a samplerhez, a NumPy `default_rng` pedig a bootstrap újramintavételhez. Minden konfigurációra öt seedet használunk (20261020, 20261021, …, 20261024), és a validációs balanced accuracy átlagát és szórását közöljük.

Konfigurációk. (1) Baseline: súlyozatlan `CrossEntropyLoss(reduction='mean')`. (2) Weighted CE: súlyok $N/(3n_k)$ a fenti eltolt gyakoriságból. (3) `WeightedRandomSampler` osztályonkénti kiegyensúlyozó mintavétellel, súlyozatlan loss mellett.

Protokoll és kiértékelés. A modellválasztás kizárólag a validációs balanced accuracy maximumán történik. A tesztet nem használjuk a labor összehasonlításában. A konfigurációk összehasonlítása páros: ugyanazon seedek és ugyanazon validációs jelenetek, a döntést a jelenet szintű páros különbség **bootstrap konfidenciaintervalluma** adja. A bootstrap CI itt a következőt jelenti: a 30 validációs jelenet páronkénti különbségét visszatevéssel újramintavételezzük, minden újramintára kiszámítjuk az átlagot, majd a kapott eloszlás 2,5%-os és 97,5%-os percentilisét vesszük; a párosítás azt jelenti, hogy mindkét konfiguráció ugyanazon jeleneteken és ugyanazon seedekkel értékelt, és a különbséget jelenetenként képezzük. 0-t kizáró 95%-os CI esetén állítunk különbséget, egyébként nem. A kalibrációt a validációs halmazon ECE-vel mérjük, 8 egyenlő szélességű konfidenciaintervallummal.

A tanítási eljárás önállóan futtatható programja (a rögzített venvban lefutott; a kód a mért összesítést is kinyomtatja):

```
# munkakönyvtár: lab/
# futtatás: python imbalance_lab.py
import numpy as np, torch, torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset, WeightedRandomSampler
```

```
SEED_BASE = 20261020
SEEDS = list(range(SEED_BASE, SEED_BASE + 5))
device = torch.device("cpu")

images = np.load("data/images.npy").astype(np.float32)   # (360,1,24,24)
labels = np.load("data/labels.npy").astype(np.int64)     # (360,)
meta = np.genfromtxt("data/metadata.csv", delimiter=",", dtype=None,
                     names=True, encoding="utf-8")
order = meta["index"].astype(np.int64)
if not np.array_equal(np.sort(order), np.arange(len(images))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
split = np.asarray(meta["split"]).astype(str)
cls = np.asarray(meta["label"]).astype(np.int64)
scene = np.asarray(meta["scene_id"]).astype(str)

train_rows = np.where(split == "train")[0]
scenes_of = {k: sorted(set(scene[train_rows[cls[train_rows] == k]])) for k in range(3)}
keep = set(scenes_of[0][:40]) | set(scenes_of[1][:5]) | set(scenes_of[2][:5])
imb_rows = np.array([i for i in train_rows if scene[i] in keep])
val_rows = np.where(split == "val")[0]
test_rows = np.where(split == "test")[0]
Xtr, ytr = images[order[imb_rows]], labels[order[imb_rows]]
Xva, yva = images[order[val_rows]], labels[order[val_rows]]
Xte, yte = images[order[test_rows]], labels[order[test_rows]]
assert (len(Xtr), len(Xva), len(Xte)) == (100, 60, 60)
assert np.bincount(ytr, minlength=3).tolist() == [80, 10, 10]
# A teszt split betöltve, de a modellválasztásban nem szerepel.

counts = np.bincount(ytr, minlength=3).astype(np.float32)
class_w = torch.tensor(len(ytr) / (3.0 * counts))           # N/(K*n_k)
sample_w = torch.tensor(1.0 / counts[ytr])                  # példánkénti húzási súly
w_val = torch.tensor(np.array([0.8, 0.1, 0.1]) / (1.0 / 3.0), dtype=torch.float32)

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # padding nélküli konvolúció: 24 -> 22 -> 11 -> 9 -> 4, ezért 16*4*4
        self.net = nn.Sequential(
            nn.Conv2d(1, 8, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(16 * 4 * 4, 3))
    def forward(self, x):
        return self.net(x)

def predict(model, X):
    model.eval()
    with torch.no_grad():
        logits = model(X)
        probs = torch.softmax(logits, dim=1)
    return logits, probs

def class_recalls(pred, y):
    return np.array([(pred[y == k] == k).mean() if (y == k).any() else 0.0
                     for k in range(3)])

def macro_f1(pred, y):
    f1 = []
    for k in range(3):
        tp = np.sum((pred == k) & (y == k))
        fp = np.sum((pred == k) & (y != k))
        fn = np.sum((pred != k) & (y == k))
        f1.append(2 * tp / (2 * tp + fp + fn))
    return float(np.mean(f1))
```

```
def ece(probs, y, n_bins=8):
    conf = probs.max(1)
    correct = (probs.argmax(1) == y).astype(float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    total = 0.0
    for b in range(n_bins):
        hi = edges[b + 1]
        m = (conf >= edges[b]) & (conf <= hi if b == n_bins - 1 else conf < hi)
        if m.sum() == 0:
            continue
        total += m.mean() * abs(correct[m].mean() - conf[m].mean())
    return total

def run(mode, seed):
    torch.manual_seed(seed)
    model = SmallCNN().to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.0)
    if mode == "weighted":
        lossf = nn.CrossEntropyLoss(weight=class_w)
    else:
        lossf = nn.CrossEntropyLoss()
    ds = TensorDataset(torch.from_numpy(Xtr), torch.from_numpy(ytr))
    if mode == "sampler":
        gen = torch.Generator().manual_seed(seed)
        sampler = WeightedRandomSampler(sample_w, num_samples=len(sample_w),
                                        replacement=True, generator=gen)
        loader = DataLoader(ds, batch_size=24, sampler=sampler)
    else:
        loader = DataLoader(ds, batch_size=24, shuffle=True)
    best_acc, best_state, last_loss = -1.0, None, None
    for ep in range(30):
        model.train()
        total, nb = 0.0, 0
        for xb, yb in loader:
            opt.zero_grad()
            loss = lossf(model(xb), yb)
            loss.backward()
            opt.step()
            total += loss.item()
            nb += 1
        last_loss = total / nb
        _, probs = predict(model, torch.from_numpy(Xva))
        acc = class_recalls(probs.argmax(1).numpy(), yva).mean()
        if acc > best_acc:
            best_acc = acc
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
    model.load_state_dict(best_state)
    _, probs = predict(model, torch.from_numpy(Xva))
    pred = probs.argmax(1).numpy()
    recalls = class_recalls(pred, yva)
    weights = w_val[yva].numpy()
    return {
        "loss": last_loss,
        "wacc": float((weights * (pred == yva)).sum() / weights.sum()),
        "bacc": float(recalls.mean()),
        "f1": macro_f1(pred, yva),
        "ece": ece(probs.numpy(), yva),
        "correct": (pred == yva).astype(float),
    }

def scene_ci(diff_per_image, scenes, n_boot=10000, seed=7):
    uniq = sorted(set(scenes))
```

```
    per_scene = np.array([diff_per_image[scenes == s].mean() for s in uniq])
    rng = np.random.default_rng(seed)
    idx = rng.integers(0, len(per_scene), size=(n_boot, len(per_scene)))
    means = per_scene[idx].mean(1)
    return np.percentile(means, [2.5, 97.5])

val_scenes = scene[val_rows]
res = {m: [] for m in ("base", "weighted", "sampler")}
for mode in res:
    for seed in SEEDS:
        r = run(mode, seed)
        res[mode].append(r)
        print(f"{mode:8s} seed={seed} loss={r['loss']:.4f} "
              f"wacc={r['wacc']:.3f} bacc={r['bacc']:.3f} "
              f"f1={r['f1']:.3f} ece={r['ece']:.3f}")

print()
for mode, rows in res.items():
    for key in ("loss", "wacc", "bacc", "f1", "ece"):
        v = np.array([r[key] for r in rows])
        print(f"{mode:8s} {key:5s} {v.mean():.4f} +- {v.std(ddof=1):.4f}")

print()
base_correct = np.array([r["correct"] for r in res["base"]])
for mode in ("weighted", "sampler"):
    other = np.array([r["correct"] for r in res[mode]])
    diff = (other - base_correct).mean(0)   # seedekre átlagolt képenkénti különbség
    lo, hi = scene_ci(diff, val_scenes)
    print(f"{mode:8s} bacc diff vs base: {diff.mean():+.4f} "
          f"95% CI [{lo:+.4f}, {hi:+.4f}]")
```

A laborjelentésben két táblázatot kérünk. Az első a train loss utolsó epochját, a tanítóeloszlásra újrasúlyozott validációs accuracyet és a balanced accuracyt tartalmazza; a második a macro-F1-et és az ECE-t. A train loss csak azonos loss-definíció mellett hasonlítható össze, ezért a weighted CE saját, súlyozott értékét nem szabad közvetlenül a baseline értékéhez mérni.

A futtatott program összesített kimenete (5 seed átlaga ± szórása, Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`):

Konfiguráció	train loss (utolsó epoch)	újrasúlyozott val_acc	balanced val_acc
baseline	0,5011 ± 0,1365	0,8010 ± 0,0022	0,3367 ± 0,0075
weighted CE	0,5395 ± 0,1242	0,6120 ± 0,0990	0,4533 ± 0,0217
sampler	0,3491 ± 0,0955	0,5810 ± 0,0963	0,4900 ± 0,0742

Konfiguráció	macro-F1	ECE
baseline	0,1734 ± 0,0151	0,1292 ± 0,1282
weighted CE	0,4120 ± 0,0293	0,1864 ± 0,0943
sampler	0,4812 ± 0,0793	0,2399 ± 0,1026

A jelenet szintű, seedekre átlagolt páros különbség bootstrap CI-je a balanced accuracyre: a weighted CE a baseline-hoz képest +0,1167, 95%-os CI [−0,0633, +0,3000]; a sampler +0,1533, 95%-os CI [−0,0400, +0,3467]. Mindkét intervallum tartalmazza a nullát, ezért ezen a 30 jelenetes validációs halmazon nem állítunk szignifikáns különbséget; a pontbecslések a várt irányba mutatnak.

A mért minta megfelel a levezetett iránynak: a class-balanced súlyozás és a sampler emeli a ritka osztályok recallját, ezért a balanced accuracy és a macro-F1 nő, miközben a tanítóeloszlásra újrasúlyozott accuracy a baseline 0,80 körüli értékéről 0,61-re, illetve 0,58-ra csökken, mert a modell kevésbé specializálódik a többségi osztályra. A kalibráció nem javult: az ECE a baseline 0,129 értékéről 0,186-ra, illetve 0,240-re nőtt, de a nagy seedenkénti szórás miatt ez a megfigyelés is bizonytalan. A baseline balanced accuracy 0,337 azt jelzi, hogy az eltolt tanítóhalmazon a súlyozatlan modell gyakorlatilag a többségi osztályt jósolja. A teszt splitet a program betölti, de a modellválasztásban és az összehasonlításban nem használja, ahogy a protokoll előírja.

A `val acc` és a `balanced acc` eltérése nem definíciós különbség: a kiegyensúlyozott validáción a két metrika egyenlő lenne, a táblázat első accuracy-oszlopa ezért a tanítóeloszlásra újrasúlyozott nézetet használja. Az eredmény a GlyphLab toy adatára és öt seedre vonatkozik, nem általánosítható természetes képeloszlásra; a szűk CI hiánya a 30 jelenetes validációs halmaz korlátja.

21.4.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

21.4.6.1 F1 — Fogalmi, nehézség 1

Magyarázza el, mi a lényegi különbség a loss weighting és a batch sampler hatásmechanizmusa között. Adjon egy olyan esetet, amikor a két módszer eltérő viselkedést eredményez ugyanakkora átlagos osztályarány mellett.

21.4.6.2 F2 — Levezetési, nehézség 2

Legyen $N = 6$, $K = 2$, $n_0 = 4$, $n_1 = 2$. Írja fel a class-balanced súlyokat, és vezesse le a 3. szakasz képletéből, hogy a súlyozott empirikus kockázat valóban $\frac{1}{2}(\hat{R}_0 + \hat{R}_1)$. Ellenőrizze az indexhatárokat, és mondja ki, melyik feltétel sérül, ha $n_1 = 0$.

21.4.6.3 F3 — Implementációs, nehézség 2

Írjon PyTorch-kódot, amely kiszámítja egy rögzített logit- és célbatchre a súlyozott cross-entropy-t mind a gradient scaling konvencióban, mind a PyTorch `reduction='mean'` beépített változatában. Ellenőrizze, hogy a két érték aránya a batch súlyösszegétől függ. Használjon `torch.manual_seed(0)`-t, `float32` tensort, és adja meg a kódot.

21.4.6.4 F4 — Hibakeresési, nehézség 3

A következő kódrészlet a tanítási loop egy részlete:

```
# target_train: az imbalanced dataset minden példájához tartozó osztályindex
sample_weights = 1.0 / class_counts[target_train]
sampler = WeightedRandomSampler(weights=sample_weights, num_samples=len(dataset),
↪  replacement=True)
loss_fn = nn.CrossEntropyLoss(weight=1.0 / class_counts)
```

A fejlesztő célja a class-balanced tanítás. Azonosítsa a kettős, egymásra rakódó osztályarány-korrekciót, értelmezze a következményét, és mutassa meg a javítást. A `sample_weights` minden tanítópéldához külön súlyt ad, ezért hossza `len(dataset)`. Ellenőrizze a javítást egy kis logit- és célbatchre úgy, hogy kiszámítja az osztályonkénti effektív súlyozást.

21.4.6.5 F5 — Kísérlettervezési, nehézség 3

Tervezzen protokollt annak mérésére, hogy a class-balanced weighted CE javítja-e a ritka osztályok recallját egy olyan teszteloszláson, ahol a célmix 90% vertical, 5% horizontal, 5% diagonal. Adja meg a megfigyelési egységet, splitet, baseline-t, változó tényezőt, metrikákat, seed-kezelést, választási szabályt és a következtetés korlátját. Az F5 nyitott végű: több elfogadható terv is van; legalább egy teljes tervet kell adni, és indokolni, miért nem elegendő a sima accuracy.

21.4.7 Megoldások és indoklás

21.4.7.1 F1 megoldás

A loss weighting a célfüggvényben a gradiens osztályonkénti skáláját módosítja: egy w_k súlyú osztályból származó példa hibájának gradiense w_k-szoros. A batch-sampler ezzel szemben a batch összetételét változtatja: a ritka osztály gyakrabban kerül a batchbe, így a gradienslépés nagyobb arányban tartalmazza azt az osztályt. Az eltérés jól látszik abban az esetben, ha a batch méret kicsi és a loss weighting mellett a ritka osztály könnyen eltűnik a mini-batchből, mert a mintavétel továbbra is a többségi eloszlást követi. A `WeightedRandomSampler` csak az egyes húzások valószínűségét módosítja, így még kiegyensúlyozott várható gyakoriság mellett sem garantálja, hogy minden osztály megjelenik minden minibatchben; a loss weighting pedig csak akkor hat, ha az adott osztályból ténylegesen szerepel példa. Ugyanakkor a loss weighting determinisztikus célfüggvényt ad, míg a sampler a sztochasztikus gradiens varianciáját növelheti.

Számszerű eset a különbségre: legyen a batch 6 példája $y = (0, 0, 0, 1, 1, 2)$, és tegyük fel, hogy a 0,8/0,1/0,1 arányú tanítóhalmazból a class-balanced súlyok $w = (5/12,\ 10/3,\ 10/3)$. A batch teljes loss-súlya $3 \cdot \frac{5}{12} + 2 \cdot \frac{10}{3} + 1 \cdot \frac{10}{3} = \frac{45}{4}$, ebből a többségi osztály részesedése $\frac{5/4}{45/4} = \frac{1}{9}$, a két ritka osztályé pedig $\frac{20/3}{45/4} = \frac{16}{27}$, illetve $\frac{10/3}{45/4} = \frac{8}{27}$. A súlyozás tehát a többségi osztály hozzájárulását a példaaránya $\frac{3}{6}$-áról $\frac{1}{9}$-re csökkenti, de az osztály példáit nem tünteti el: ha egy minibatchben egyáltalán nincs ritka példa, a weighting nem tud mit tenni. A sampler ezzel szemben a húzás valószínűségét változtatja meg, ezért a batch összetételére hat: $s_i \propto 1/\hat{\pi}_{y_i}$ mellett a várható osztályarány 1/3, 1/3, 1/3.

21.4.7.2 F2 megoldás

$N = 6$, $n_0 = 4$, $n_1 = 2$, $K = 2$, $\hat{\pi}_0 = 4/6 = 2/3$, $\hat{\pi}_1 = 1/3$. A súlyok:

$$w_0 = \frac{N}{K n_0} = \frac{6}{2 \cdot 4} = \frac{3}{4}, \qquad w_1 = \frac{6}{2 \cdot 2} = \frac{3}{2}.$$

A súlyozott empirikus kockázat

$$\hat{R}_W = \frac{1}{6} \sum_{i:y_i=0} \frac{3}{4} \ell_i + \frac{1}{6} \sum_{i:y_i=1} \frac{3}{2} \ell_i = \frac{3/4}{6} \cdot (4\hat{R}_0) + \frac{3/2}{6} \cdot (2\hat{R}_1) = \frac{1}{2}\hat{R}_0 + \frac{1}{2}\hat{R}_1.$$

Az utolsó egyenlőség onnan jön, hogy $\frac{3/4}{6} \cdot 4 = \frac{3}{6} = \frac{1}{2}$ és $\frac{3/2}{6} \cdot 2 = \frac{1}{2}$. A súlyozott összeg tehát éppen az osztályonkénti átlagok átlaga. Ha $n_1 = 0$, akkor $\hat{\pi}_1 = 0$, a $w_1 = 1/(K\hat{\pi}_1)$ osztás nullával nem értelmezett; a class-balanced súlyozás ezért csak akkor használható, ha minden osztályból van legalább egy tanítópélda. A levezetés indexhatárai az $i : y_i = k$ részhalmazokra az $i = 1, \ldots, N$ tartományon érvényesek (a kódban ez a 0-tól indexelt $[0, N)$ tartománynak felel meg).

21.4.7.3 F3 megoldás

```python
# lab/f3_weighted_ce.py - futtatható lokálisan
import torch
import torch.nn.functional as F

torch.manual_seed(0)
logits = torch.randn(6, 3, dtype=torch.float32)  # N x K
target = torch.tensor([0, 0, 0, 0, 1, 2], dtype=torch.int64)
train_counts = torch.tensor([80, 10, 10], dtype=torch.float32)
K = 3
# A súlyokat a TRAIN teljes osztálygyakorisága, nem ez a mini-batch adja.
w = train_counts.sum() / (K * train_counts)  # shape (3,)

# 1) gradient scaling konvenció
loss_each = F.cross_entropy(logits, target, reduction='none')  # shape (6,)
L_gradscale = torch.mean(w[target] * loss_each)

# 2) PyTorch beépített mean reduction súllyal
L_torch = F.cross_entropy(logits, target, weight=w, reduction='mean')

# Ellenőrzés: L_torch = sum(w_i*l_i) / sum(w_i)
L_numerator = torch.sum(w[target] * loss_each)
L_denom = torch.sum(w[target])
L_expected = L_numerator / L_denom

print(f"Gradient scaling : {L_gradscale.item():.6f}")
print(f"PyTorch mean     : {L_torch.item():.6f}")
print(f"Elvárt torch mean: {L_expected.item():.6f}")
print(f"Arány            : {L_gradscale.item() / L_torch.item():.6f}")
print(f"Batch súlyösszeg : {L_denom.item():.6f}")
```

Ellenőrzés: a kimenetben a második és harmadik érték float32-ben csak tűréssel egyezik, nem bitre azonos; a mért eltérés ezen a példán $5{,}96 \cdot 10^{-8}$ nagyságrendű, ezért `torch.allclose`-szal (alapértelmezés: `rtol=1e-5`, `atol=1e-8`) ellenőrizzük. Az arány a batch súlyösszege osztva batch mérettel. A rögzített train-gyakoriságokra $w = (5/12, 10/3, 10/3)$; a hat példás minibatch négy nullás és egy-egy ritka osztálycímkéje mellett $\sum_i w_{y_i} = 25/3$, így a két érték aránya $(25/3)/6 =$

$25/18 \approx 1{,}3889$, nem 1. Ha a súlyokat hibásan az adott minibatch saját gyakoriságából becsülnénk, akkor — feltéve, hogy minden osztály szerepel a batchben — a $\sum_i w_{y_i} = \sum_k n_k^{(B)}\, B/(K n_k^{(B)}) = B$ azonosság miatt a normalizáló tényező minden ilyen batchben a batch mérete lenne, nem csak ebben a példában; ha valamelyik osztály hiányzik a batchből, a $B/(K n_k^{(B)})$ súly nullával osztana, ezért ez a becslés ott eleve nem használható. A batch-gyakoriságból számított súlyok azonban nem a tanítóhalmaz arányát képviselik, ezért a normalizálás látszólagos egyezése nem teszi helyessé a becslést.

21.4.7.4 F4 megoldás

Hiba: a sampler és a loss weighting ugyanazt az inverse frequency súlyozást alkalmazza egyszerre. A sampler már a batch szintjén kiegyensúlyozza az osztályarányt, a loss súly pedig ezen felül még w_k-szoros gradiensszorzót ad. Következmény: a ritka osztály kétszeresen fel van súlyozva; az effektív osztálysúly $q_k w_k$ lesz, ahol q_k a sampler által beállított batchgyakoriság. Ha a sampler osztályeloszlása várhatóan $q_k = 1/K$, és a loss súly $w_k = 1/(K\hat{\pi}_k)$, akkor a normalizálás előtti osztály-hozzájárulás $q_k w_k = 1/(K^2\hat{\pi}_k)$; a célzott, egyenlő osztály-hozzájárulás ezzel szemben mindegyik osztályra $1/K$. Az osztályok egymáshoz viszonyított hozzájárulása ezért fordítottan arányos lesz az eredeti $\hat{\pi}_k$ gyakorisággal. A `CrossEntropyLoss(weight=..., reduction='mean')` batchszinten a súlyok összegével oszt, ami a relatív eltolást nem szünteti meg. Javítás: a két módszer közül csak az egyiket használjuk; ha a sampler alkalmazása a cél, a loss maradjon súlyozatlan. Ha a loss weighting mellett döntünk, ne használjunk mintavételező-módosítást a batchben. Ellenőrzés: számítsuk ki az osztályok effektív hozzájárulását a batch loss-súlyösszegéhez, azaz a $\frac{\sum_{i\in B,\, y_i=k} w_{y_i}}{\sum_{i\in B} w_{y_i}}$ arányt. A 0,8/0,1/0,1 arányú tanítóhalmazon $w = (5/12,\ 10/3,\ 10/3)$; ha a sampler várhatóan osztályonként 2 példát ad egy 6-os batchbe ($q_k = 1/3$), akkor az osztály-hozzájárulások $2 \cdot \frac{5}{12} = \frac{5}{6}$, illetve a két ritka osztályra $2 \cdot \frac{10}{3} = \frac{20}{3}$, a teljes összeg $\frac{5}{6} + \frac{40}{3} = \frac{85}{6}$, így az arányok $\frac{1}{17}$, $\frac{8}{17}$, $\frac{8}{17}$: a ritka osztályok nyolcszor akkora súllyal szerepelnek, mint a többségi. A cél $\frac{1}{3}$, $\frac{1}{3}$, $\frac{1}{3}$ lenne, a kettős korrekció ezt $1 : 8 : 8$-ra torzítja. A javítás után (csak sampler, súlyozatlan loss) az arány $\frac{1}{3}$, $\frac{1}{3}$, $\frac{1}{3}$; ha csak loss weightinget használunk sampler nélkül, akkor a batch összetétele a tanítóhalmazét követi, és a súlyozás adja a class-balanced hozzájárulást.

21.4.7.5 F5 megoldás

Egy lehetséges terv:

- **Kérdés:** javítja-e a class-balanced weighted CE a ritka osztályok recallját egy 90/5/5 teszt eloszláson?
- **Adat:** GlyphLab, 360 kép; a train eredetileg osztályonként 40 jelenetből áll. A tanításhoz determinisztikusan megtartunk 40 vertical, 5 horizontal és 5 diagonal teljes jelenetet (100 kép), így a class-balanced súlyozásnak valóban eltérő súlyai lesznek. A 60 képes testet változatlanul, függetlenül megőrizzük; az eltolódott 90/5/5 tesztmix vizsgálatához a teszt 30 jelenetét előre rögzített, osztályfüggő valószínűségekkel, *teljes jeleneteket együtt* újramintavételezve súlyozzuk, majd az eredményeket az eredeti, kiegyensúlyozott test mellett közöljük. Az újramintavétel nem teremt új független tesztjeleneteket, és az effektív mintaszámot erősen csökkenti: 30 jelenetből 90/5/5 arány mellett várhatóan mintegy 27 függőleges mellett csak 1–2 vízszintes és 1–2 átlós jelenet marad, így a ritka osztályok becslése 1–2 jeleneten nyugszik. Ezért a 90/5/5 teszten csak nagyon erős hatás mutatkozhat meg, és a kiegyensúlyozott teszt eredményét is közölni kell.

- **Baseline:** súlyozatlan CE. **Változó tényező:** súlyozás módja (súlyozatlan vs class-balanced súlyok).
- **Metrika:** recall a horizontal és diagonal osztályra külön, valamint balanced accuracy. A sima accuracy a ritka osztályok hibáját elmossa, ezért nem elég.
- **Seed-kezelés:** 5 seed, mindkét módszerre azonos. **Választási szabály:** a validációs halmaz (kiegyensúlyozott) balanced accuracy maximumán választjuk a legjobb epochot minden seedre. A teszt kiértékelése csak a kiválasztott modelleken történik.
- **Páros összehasonlítás és döntési szabály:** a két módszert ugyanazon seedekkel és ugyanazon tesztjeleneteken értékeljük; a döntést a jelenet szintű páros különbség bootstrap CI-je adja a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint (ugyanaz a resampling-index mindkét módszerre), nem a két külön intervallum átfedése. Ha a 95%-os CI kizárja a nullát, javulást jelentünk; ha tartalmazza, nem állítunk különbséget. A seedek közötti szórást külön, leíró statisztikaként közöljük.
- **Következtetési korlát:** az eredmény a GlyphLab eltolt mixére vonatkozik, nem általános természetes képeloszlásra. A bootstrap CI csak a teszt halmaz mintavételi bizonytalanságát fedi, a modelltanítási varianciát a seed-ek szórása jelzi.
- **Hipotézis, architektúra, recept és erőforrás:** a class-balanced súlyozás javítja a ritka osztályok recallját a súlyozatlan baseline-hoz képest; a modell a GlyphLab `SmallCNN` (1299 paraméter), Adam optimalizálóval, `lr=1e-3`, 30 epoch és 24-es batch mellett; az öt seed értéke 20261020–20261024, mindkét súlyozási módban ugyanaz. A döntési küszöb a jelenet szintű páros bootstrap 95%-os CI-je, és a ritka osztály recalljában legalább 2 százalékpontos különbség. A megfigyelési egység a `scene_id`. Erőforrás: CPU, `float32`, két mód × öt seed = 10 futás, néhány perc; a futásidőt a napló rögzíti. ### Tipikus tévedés ellenpéldával

A leggyakoribb hiba a **kettős korrekció**: a loss weighting és a sampler ugyanazzal az osztályarányú súllyal való együttes alkalmazása indoklás nélkül. Az F4 feladatban látott effektív osztálysúly $q_k w_k$ könnyen a ritka osztály túlzott felhangsúlyozásához vezet, ami hajlamosíthat a ritka osztályon való túlillesztésre, és romolhat a kalibráció. Példa: ha a sampler közel egyenletes $q_k = 1/K$-t ad, és a loss súly $w_k = 1/(K\hat{\pi}_k)$, akkor az osztályonkénti relatív hozzájárulás $q_k w_k \propto 1/\hat{\pi}_k$, nem pedig azonos minden osztálynál. Az eredetileg ritka osztályok így a class-balanced célhoz képest is aránytalanul felértékelődnek. A kettős korrekció tehát nem azonos a class-balanced célfüggvénnyel. A helyes eljárás: válasszunk **egy** mechanizmust, és dokumentáljuk a döntést a kísérleti naplóban.

Önellenőrzés. Ellenőrizze, hogy a súlyok pozitívak, az összegük nem feltétlenül 1, a mini-batch loss redukciója a választott konvenciót követi, és a kalibráció mérése a cél eloszláson történik. Visszautalás: a fair comparison 17.4. Architektúra összehasonlítása kontrollált keretben szellemében a baseline és a súlyozott változat csak a vizsgált tényezőben térhet el; a seed, a batch méret, az optimalizáló és a modellválasztási szabály azonos kell legyen.

21.4.8 Rövid összefoglaló

A nem egyenletes osztályarány a naiv ERM-et a többségi osztály felé tolja. A class-balanced súlyozás a veszteségben az osztályonkénti hibákat egyenlő arányban átlagolja: választással $w_k = N/(Kn_k)$, a minimalizált célfüggvény $\frac{1}{K}\sum_k \hat{R}_k$. A sampler a batch összetételét módosítja, a focal loss példánkénti nehézséget súlyoz; a három mechanizmus nem felcserélhető. A súlyozás megváltoztatja a kalibráció cél-eloszlását, ezért az értékelést a tényleges felhasználási eloszláson

kell végezni. A súlyozást és a samplert indoklás nélkül együtt alkalmazni kettős korrekció, amely túlillesztéshez vezethet.

21.4.9 Következő lecke és források

A következő egység, 22.1. Adateredet, címke és csoportos split, az adateredet, a címke és a csoportos split kérdéseit tárgyalja: ott a scene_id szerinti függetlenség és az annotációs hibák kezelése épít a fenti osztályarány- és értékelési megfontolásokra. A súlyozott empirical risk levezetése saját definíciós levezetés, nincs szüksége külső tételre. A PyTorch `CrossEntropyLoss` súlyozási viselkedését a [D07] dokumentáció rögzíti; a fejezet ezt a loss-szerződést alkalmazza. A [P02] az AdamW decoupled weight decay forrása, amely a laborkísérlet optimalizálójának megválasztásához kapcsolódik.

Felhasznált források:

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30. Loss-szerződés: logits bemenet, int64 cél, nincs külön softmax a CrossEntropyLoss előtt.
- [P02] Loshchilov, I. és Hutter, F. (2017; ICLR 2019 változat). *Decoupled Weight Decay Regularization*. https://arxiv.org/abs/1711.05101.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. és Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR 2016. https://arxiv.org/abs/1512.00567.

22 Képadat, annotáció és augmentációs szerződés

22.1 Adateredet, címke és csoportos split

22.1.1 Cél, előismeret és induló kérdés

Ez az egység az adatfolyam legelső, a tenzorműveletek előtti szerződését tárgyalja: honnan jön a megfigyelés, milyen jogosultsággal használjuk, mi a független megfigyelési egység, és az annotáció mely rekordhoz tartozik. Az 5.4. Függő képek, seedek és resampling már megmutatta, hogy a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab 360 képéhez 180 jelenet tartozik, ezért a képszám és a független egységek száma eltér. A 11.4. Egy teljes PyTorch tanítási iteráció a PyTorch adatbetöltést a split után építette fel; most a splitet megelőző, ellenőrizhető réteget rögzítjük. A 8.4. II. ellenőrző pont: korrekt tanulási állítás korrekt tanulási állításához, a 21.4. Osztályarány, súlyozás és céleloszlás céleloszlásához és a 20.4. V. ellenőrző pont: indokolt CNN-választás adatfeltételes architektúradöntéséhez is ugyanaz a követelmény: az adathivatkozásnak egyértelműnek kell lennie a kísérlet teljes élettartama alatt.

Induló kérdés. A GlyphLab train halmaza 240 képet tartalmaz. Hány független statisztikai egység van ebben a halmazban: 240 vagy 120? Miért befolyásolja a válasz a validációs hibát? A becslését a csoportos felosztás lemmája és a labor auditja után ellenőrizheti.

22.1.2 Definíciók, jelölések és a GlyphLab-séma

A tárgyalt fogalmakat rekordok és leképezések nyelvén adjuk meg, mert a későbbi ellenőrzés ezeket a tulajdonságokat kódban is vizsgálja.

Leképezések. Egy $f : X \to Y$ függvény *injektív*, ha minden $x_1, x_2 \in X$-re $f(x_1) = f(x_2)$ csak $x_1 = x_2$ esetén teljesül; *szürjektív*, ha minden $y \in Y$-ra van olyan $x \in X$, amelyre $f(x) = y$; *bijektív*, ha injektív és szürjektív, azaz X és Y elemei kölcsönösen egyértelműen megfeleltethetők.

Adateredet (provenance). Egy N példányból álló adathalmaz *provenance-rekordja* egy rendezett lista, amely legalább az alábbiakat tartalmazza: a generáló vagy gyűjtő folyamat azonosítója és verziója; a forrásra mutató azonosító; a példányok és a csoportok generálásának paraméterei, köztük a véletlen mag (seed); valamint a nyers állapotból a szállított állapotba vivő transzformációk leírása. A GlyphLab esetében a generátor az `generate_data.py --output data` parancs, a verzió 1.0, a seed 20261020; a nyers szintetikus jelenetből két, zajjal módosított nézet készül (`view_id` szerint). A provenance-rekord tehát nem a modell hibájának minőségi leírása, hanem a reprodukálhatóság szerződése.

Licenc. A licenc a felhasználási jogosultság jognyilatkozata egy meghatározott adatváltozatra. Megkülönböztetjük az adathalmaz *nevét*, az adatváltozat *lenyomatát* és a *licencfeltételeket*. A GlyphLab saját, oktatási célra előállított adata a könyv laborjának szerződése alá esik; a manifest verziója és SHA-256 lenyomatai azonosítják a változatot. Egy adathalmaz publikus elérhetősége nem jogi kategória, ezért a licencet konkrét URL-hez, verzióhoz és terjesztőhöz kötjük.

Duplikátum. Legyen $x_i, x_j \in \mathbb{R}^{C\times H\times W}$ két példány, és h egy hashfüggvény a bájtábrázoláson. Azt mondjuk, hogy x_i és x_j *tartalom-duplikátum*, ha a bájtábrázolásuk azonos és $i \neq j$. A gyakorlatban a bájtszintű összevetést hash-egyezéssel helyettesítjük: $h(x_i) = h(x_j)$ szükséges, de ütközés esetén nem elégséges feltétel, ezért a duplikátum-jelöltet bájtszintű összevetéssel kell megerősíteni. A tartalom-duplikátum nem azonos a *függő párral*: a GlyphLab két view-ja azonos jelenetből származik, ezért statisztikailag függő, de a view-nként független zaj miatt a bájtábrázolásuk általában különbözik. Azt, hogy egy duplikátum *rekord-duplikátum*-e, a deduplikációs kulcs dönti el: a GlyphLab kulcsa az `(index, scene_id, view_id)` hármas, amely garantáltan egyedi, még akkor is, ha két view pixelszinten megegyezne.

Csoport (scene/patient/device group). Legyen $I = \{0, \dots, N-1\}$ a példányindexek halmaza, G a csoportazonosítók halmaza, és $\gamma : I \to G$ a csoporthozzárendelés. A GlyphLabben $\gamma(i)$ a `metadata.csv` `scene_id` oszlopa, az $N = 360$ példányhoz $|G| = 180$ csoport. Minden $g \in G$-re a $\gamma^{-1}(g)$ halmaz elemei azonos független forrásból származnak; a köztük lévő korreláció miatt a statisztikai egység a csoport, nem a kép. Orvosi adatokban g jellemzően betegazonosító, többkamerás felvételeknél eszköz- vagy felvételazonosító.

Annotáció és címke-hivatkozás. Legyen K az osztályok száma, $\ell : I \to \{0, \dots, K-1\}$ a címke. A GlyphLabben $K = 3$, $\ell(i)$ a `label` oszlop int64 értéke. Az annotációs rekordok halmaza $R = \{r_j : j \in J\}$, ahol $J = \{0, \dots, M-1\}$ a rekordok indexhalmaza; minden r_j sor egy $p(r_j) \in I$ példányra hivatkozik. A *címke-hivatkozás egyedisége* azt jelenti, hogy p injektív: két különböző annotációs sor nem hivatkozhat ugyanarra a példányra. A GlyphLabben $M = N$, és p bijekció, mert a `metadata.csv` `index` oszlopa az egész I halmaz permutációja. Az annotáció a GlyphLabben a címkén túl a maszkot és a fél-nyílt, nullától indexelt xyxy dobozt is tartalmazza: `xmin` `x<xmax`, `ymin` `y<ymax`, a doboz területe $(x_{max} - x_{min})(y_{max} - y_{min})$.

22.1.3 A csoportos felosztás és a hivatkozás-egyediség invariánsa

Ebben a szakaszban megadjuk a csoportszintű diszjunkt felosztás reprezentációs invariánsát, és belátjuk a címke-hivatkozás egyediségének permutáció-invarianciáját.

Lemma (a csoportos split reprezentációja). Legyen I véges halmaz, $\gamma : I \to G$ a csoporthozzárendelés, és tegyük fel, hogy $G = \gamma(I)$, azaz γ szürjektív. Legyen továbbá $\sigma : I \to \{train, val, test\}$. Ha minden $i, j \in I$ esetén

$$\gamma(i) = \gamma(j) \implies \sigma(i) = \sigma(j), \tag{1}$$

akkor létezik pontosan egy $\tilde{\sigma} : G \to \{train, val, test\}$ úgy, hogy

$$\sigma = \tilde{\sigma} \circ \gamma. \tag{2}$$

Ha a csoportazonosítók halmaza nagyobb, mint $\gamma(I)$ (például a G a lehetséges, de a mintában nem szereplő csoportokat is tartalmazza), akkor a $G \setminus \gamma(I)$ csoportokhoz nem tartozik példány, ezért $\tilde{\sigma}$ ezeken szabadon választható, és az egyértelműség csak a $\gamma(I)$ részhalmazon áll; a lemma kimondásakor ezért $G = \gamma(I)$-t használunk.

Bizonyítás. A $G = \gamma(I)$ választás miatt γ szürjektív. Minden $g \in G$-hez válasszunk egy $\iota(g) \in I$ elemet, amelyre $\gamma(\iota(g)) = g$, és definiáljuk $\tilde{\sigma}(g) := \sigma(\iota(g))$-t. Meg kell mutatnunk, hogy ez a definíció független a reprezentáns választásától. Ha $\iota'(g)$ egy másik választás, akkor $\gamma(\iota(g)) = \gamma(\iota'(g)) = g$; az (1) feltétel szerint ezért $\sigma(\iota(g)) = \sigma(\iota'(g))$. Tehát $\tilde{\sigma}$ jól definiált. Továbbá minden $i \in I$-re

$$\tilde{\sigma}(\gamma(i)) = \sigma(\iota(\gamma(i))) = \sigma(i),$$

mert $\gamma(\iota(\gamma(i))) = \gamma(i)$, ezért az (1) feltétel az i és a $\iota(\gamma(i))$ párra közvetlenül $\sigma(\iota(\gamma(i))) = \sigma(i)$-t adja. Ez éppen (2). Ha $\tilde{\tau}$ is eleget tesz (2)-nek, akkor minden $g \in G$-re $\tilde{\tau}(g) = \tilde{\tau}(\gamma(\iota(g))) = \sigma(\iota(g)) = \tilde{\sigma}(g)$, tehát a függvény egyértelmű. ■

A lemma következménye a diszjunkt felosztás invariánsa: a split nem a képek, hanem a csoportok partícióján értelmezett. Mivel I a γ-ősképek diszjunkt uniója, az (1) tulajdonság pontosan azt mondja ki, hogy minden split halmaz teljes $\gamma^{-1}(g)$ csoportok uniója. Ha a felosztás egyszer teljesíti (1)-et, akkor a rekordok tetszőleges permutációja után is teljesíti, mert a feltétel csak a (γ, σ) értékpárokon múlik, nem a rekordok felsorolási sorrendjén. A GlyphLabben tehát a 120 train jelenet mindkét view-ja a trainbe, a 30 val jelenet mindkét view-ja a valba, a 30 test jelenet mindkét view-ja a testbe kerül; a képszám 240, de a független egységek száma 120. Ez válaszol az induló kérdés első felére. A másodikra: ha a két view különböző splitbe kerülne, a validációs halmaz olyan jeleneteket is tartalmazna, amelyeket a modell tanulás közben látott, ezért a validációs hiba optimistán torzított lenne; a 120 független egység miatt pedig a hiba bizonytalanságát is jelenetszintű bontásban kell becsülni, mert a 240 képből számolt képszintű intervallum a függő párok miatt túl szűk lenne.

Állítás (a címke-hivatkozás egyediségének permutáció-invarianciája). Legyen $I = \{0, \dots, N-1\}$ a példányok halmaza, $J = \{0, \dots, M-1\}$ a rekordok indexhalmaza, és $R = \{r_j : j \in J\}$ a metadata-rekordok halmaza, ahol a j rekordindex nem feltétlenül azonos a $p(r_j)$ hivatkozott példányindexszel. Ha a $p(r_j)$ hivatkozások páronként különbözőek, akkor a rekordok tetszőleges $\pi : J \to J$ permutációja után is injektív marad a hivatkozás, és minden példányhoz legfeljebb egy rekord tartozik.

Bizonyítás. A rekordok átrendezését a rekordindexeken értelmezett $\pi : J \to J$ permutáció írja le: az új i-edik rekord az eredeti $\pi^{-1}(i)$ rekord, azaz $r'_i := r_{\pi^{-1}(i)}$. Ezzel a konvencióval az új hivatkozásfüggvény $p_\pi(r'_i) = p(r_{\pi^{-1}(i)})$. (Az ellenkező irányú konvenció, $r'_i := r_{\pi(i)}$, ugyanígy működik; a lényeg, hogy π a rekordindexeket permutálja, a p értékkészletét nem.) Mivel π^{-1} bijekció J-n, a $\{p(r_{\pi^{-1}(i)}) : i \in J\}$ halmaz megegyezik a $\{p(r_j) : j \in J\}$ halmazzal, tehát p_π értékkészlete azonos az eredeti p értékkészletével. Mivel p injektív, az értékkészletben nincs ismétlés, így p_π is injektív; ebből következik, hogy minden példányhoz legfeljebb egy rekord tartozik. A bijektív eset külön indoklást igényel: ha $M = N$ és p injektív J-ről I-re, akkor két azonos számosságú véges halmaz között p szürjektív is, tehát bijektív; így minden példányhoz pontosan egy rekord tartozik. A rekordhoz tartozó címke, maszk és doboz a rekorddal együtt mozog, ezért ezekből is pontosan egy tartozik minden példányhoz. Ha $M < N$, a bijektív következtetés nem érvényes, és csak a „legfeljebb egy” állítás marad meg. ■

A GlyphLab audit két strukturális invariánst ellenőriz: a csoportos split (1) tulajdonságát, valamint azt, hogy a `metadata.index` a $0, \dots, 359$ halmaz permutációja. Ezek mellett további ellenőrzéseket is végez: a rekordkulcsok egyediségét, a SHA-256 lenyomatot, a címkék értéktartományát, a dobozok érvényességét és a split-számokat. A SHA-256 a fájltartalom változatlanságát rögzíti, és így a provenance három rétegét — folyamat, csoport, lenyomat — együtt zárja.

22.1.4 Kézi kis példa

Tekintsünk hat képet három jelenetből, jelenetenként két view-val. Az alábbi CSV-sorok a GlyphLab sémáját követik:

```
index,scene_id,view_id,label,split,xmin,ymin,xmax,ymax
0,G0-0000,0,0,train,2,3,21,20
1,G0-0000,1,0,train,3,2,20,21
2,G1-0000,0,1,train,0,6,24,18
3,G1-0000,1,1,train,1,6,23,18
4,G2-0000,0,2,val,5,5,19,9
5,G2-0000,1,2,val,4,5,20,9
```

Az (1) tulajdonság ellenőrzése: a `G0-0000` csoport mindkét elemének splitje `train`, a `G1-0000` csoporté szintén `train`, a `G2-0000` csoporté `val`. Létezik tehát $\tilde{\sigma}(\text{G0-0000}) = \tilde{\sigma}(\text{G1-0000}) = \mathit{train}$, $\tilde{\sigma}(\text{G2-0000}) = \mathit{val}$, és a lemma szerint a split a csoportokon értelmezett. Ha a 4. indexű `G2-0000,0` rekordot a `val` helyett `train`-re írnánk át, akkor az (1) feltétel sérülne, mert az 5. indexű `G2-0000,1` rekord `val` maradna; a csoportos split lemmája miatt nem létezne olyan $\tilde{\sigma}$, amelyre $\sigma = \tilde{\sigma} \circ \gamma$. A címke-hivatkozás szempontjából a hat `index` érték egy permutáció, tehát minden képtömbbeli pozícióhoz pontosan egy annotációs sor tartozik. A két view nem tartalom-duplikátum: a pixeltenzoraik a view-specifikus zaj miatt különböznek, rekordszinten pedig a `view_id` is eltér. Ez a hat soros példa a labor teljes, 360 soros ellenőrzésének kicsinyített, kézzel követhető előképe.

22.1.5 Labor: manifest-audit és külsőadat-protokoll

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és pandas 2.3.3. Szükséges fájlok: `data/manifest.json` és `data/metadata.csv`; futtatás: `python audit_manifest.py --data data`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

22.1.5.1 GlyphLab manifest- és metadata-audit

Az elvárt eredmény egy futtatható audit, amely a lab könyvtár `data/manifest.json` és `data/metadata.csv` fájljait ellenőrzi. Input: a rögzített GlyphLab adat az [ADATOK.md] szerződése szerint. A példában nincs véletlen, tenzortanulás, GPU vagy dtype-választás: CPU-s fájlműveletek, `seed=None`, `dtype` nem értelmezett. Futtatási könyvtár `lab/`, parancs:

```
python audit_manifest.py --data data
```

A GlyphLab képtenzorai $(360, 1, 24, 24)$ alakúak (a függelék A. szakasza), a maszk és a doboz ugyanezen a 24×24-es rácson él; a manifest `arrays` szakasza az alakot és a dtypét is rögzíti.

A megírandó kód:

```python
import argparse, hashlib, json, sys
from pathlib import Path
import numpy as np
import pandas as pd

# Input: GlyphLab data könyvtár; CPU fájlművelet, nincs véletlen.
def sha256_of(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def check(data_dir: str) -> list[str]:
    data = Path(data_dir)
    errors: list[str] = []
    manifest = json.loads((data / "manifest.json").read_text())

    # A GlyphLab manifest sémája (lab/ADATOK.md): generator_version, seed,
    # arrays, counts, sha256, corrupt_order.
```

```
    for key in ["generator_version", "seed", "arrays", "counts", "sha256",
                "corrupt_order"]:
        if key not in manifest:
            errors.append(f"hiányzik manifest kulcs: {key}")
    for rel, sha in manifest.get("sha256", {}).items():
        p = data / rel
        if not p.exists():
            errors.append(f"hiányzik adatfájl: {rel}")
        elif sha != sha256_of(p):
            errors.append(f"sha256 hibás: {rel}")
    # a rögzített tömb-alak és dtype egyezzen a tényleges .npy fájlokkal
    for name, spec in manifest.get("arrays", {}).items():
        arr = np.load(data / name, mmap_mode="r")
        if list(arr.shape) != list(spec["shape"]) or str(arr.dtype) != spec["dtype"]:
            errors.append(f"tömb-alak vagy dtype eltérés: {name}")

    meta = pd.read_csv(data / "metadata.csv")
    n = len(meta)
    # index: 0..N-1 permutáció (címkehivatkozás egyedisége)
    if set(meta["index"]) != set(range(n)) or meta["index"].duplicated().any():
        errors.append("index nem egyedi 0..N-1 permutáció")
    if not meta["label"].isin({0, 1, 2}).all():
        errors.append("label {0,1,2}-n kívüli érték")
    if not set(meta["split"]) <= {"train", "val", "test"}:
        errors.append(f"ismeretlen split érték: {sorted(set(meta['split']))}")
    # csoportos split: scene -> split egyértelmű
    bad = meta.groupby("scene_id")["split"].nunique()
    bad = bad[bad != 1]
    if not bad.empty:
        errors.append(f"csoportos split sértve: {list(bad.index)}")
    # megfigyelt darabszámok a manifesttel szemben
    counts = {str(k): int(v) for k, v in meta["split"].value_counts().items()}
    expected = {str(k): int(v) for k, v in manifest.get("counts", {}).items()}
    if counts != expected:
        errors.append(f"split-szám eltérés: {counts} != {expected}")
    # doboz: fél-nyílt xyxy, 0<=min<max<=24, pozitív terület
    box_ok = (
        meta["xmin"].between(0, 24)
        & meta["ymin"].between(0, 24)
        & meta["xmax"].between(0, 24)
        & meta["ymax"].between(0, 24)
        & (meta["xmin"] < meta["xmax"])
        & (meta["ymin"] < meta["ymax"])
    )
    if not box_ok.all():
        errors.append("hibás doboz, nem fél-nyílt xyxy")
    return errors

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", default="data")
    args = ap.parse_args()
    errs = check(args.data)
    for e in errs:
        print("HIBA:", e)
    if errs:
        sys.exit(1)
    print("OK: minden GlyphLab audit-ellenőrzés átment")
```

A futtatás kimenete (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, CPU, a rögzített `lab/data` könyvtáron):

```
$ python audit_manifest.py --data data
```

```
OK: minden GlyphLab audit-ellenőrzés átment
$ echo $?
0
```

A parancs sikeres kilépési kódja 0, és a kimenet egyetlen OK-sort tartalmaz; hibás adat esetén a `HIBA:` sorok listázzák az eltéréseket, a kilépési kód pedig 1.

Ha egy jövőbeli generátorverzió megváltoztatja a manifest JSON-kulcssémáját, az audit első feladata a séma feloldása; a hibajelzés ekkor pontosan megnevezi a hiányzó kulcsot.

22.1.5.2 Protokoll egy választható külső adatkészlethez

A feladat második fele egy terv, nem egy elvégzett letöltés. A következő sablont bármely választott külső adatra kitöltheti:

1. **Kérdés és modellcél:** mit akarunk előrejelezni, milyen bemenetből.
2. **Forrás és változat:** pontos URL, hozzáférés dátuma, tükörazonosító, archivum-ellenőrzőösszeg, a példa verziója.
3. **Licenc:** a forráshoz tartozó licencszöveg, szerzői jogi nyilatkozat, terjesztői feltételek; a puszta adathalmaznév nem bizonyít jogot.
4. **Megfigyelési egység:** mi a független g csoport; ha nincs explicit csoportazonosító, a protokoll indokolja, miért tekinthetők a képek függetlennek.
5. **Duplikátum-ellenőrzés:** milyen hash és milyen kulcs; a csoporton belüli nézeteket függőnek, nem duplikátumnak kezeljük.
6. **Annotáció:** címkeskála, annotátor, konzisztencia-ellenőrzés, hiányzó címke kezelése, a hivatkozás egyediségének ellenőrzése.
7. **Split:** a csoportszintű (1) tulajdonságot garantáló eljárás, osztályarány és csoportszám megadása splitenként.
8. **Preprocesszálás:** normalizálás, átméretezés, a transzformációk rögzített verziója.
9. **Utóaudit:** a split ellenőrző futtatása, a manifest hash újraszámítása.
10. **Közölt korlát:** a [S05] a klaszteres bootstrap eljárását és annak feltételeit tárgyalja; ezt itt analógiaként visszük át a csoportszintű bizonytalanságra, nem tételként. A kapott intervallum nem az egyetlen modell bizonytalansága.

Választható minta: ha a hallgató a CIFAR-10 nevű adatkészletet választja, a protokoll 2. és 3. pontjába a ténylegesen beszerzett tükör URL-jét, SHA-256-ját és licencnyilatkozatát kell beírni. A CIFAR-10 letöltése és licencének ellenőrzése a hallgató feladata; a „CIFAR-10 publikus” kijelentés önmagában nem protokoll.

22.1.6 Tipikus tévedés ellenpéldával

Gyakori hiba így hangzik: *„Az X adathalmaz publikus, tehát a felhasználásom jogszerű, és a neve azonosítja az adatokat.”* A két állítás egyike sem következik a névből. Legyen D_1 és D_2 két archívum, mindkettő `GlyphAlike-v1.zip` néven terjesztve (a név fiktív, csak a példát szolgálja); D_1 SHA-256 lenyomata h_1, licencfeltétele CC BY 4.0, D_2 lenyomata $h_2 \neq h_1$, feltétele pedig „csak nem kereskedelmi kutatásra”. A „GlyphAlike szabadon használható” mondat D_1-re igaz lehet, de D_2-re hamis, pedig a név azonos. A hibás érvelés következménye kettős: a kísérlet nem reprodukálható, mert a lenyomat nélkül nem tudjuk, melyik változatot használtuk, és a licencfeltétel dokumentálása nélkül a felhasználás jogszerűsége nem ellenőrizhető; a licenc megsértésének jogi következményeit a licenc szövege és az alkalmazandó jog szabályozza, ezt a könyv nem minősíti. A javítás a név, a lenyomat és a licencfeltétel hármasának rögzítése: a három adat együtt, konkrét URL-lel és hozzáférési dátummal.

22.1.7 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget a tartalom-duplikátum és a függő nézetpár között a GlyphLabben. Adjon olyan kis ellenpéldát, amelyben két kép azonos címkéjű és azonos pixelértékű, mégsem kezelhető ugyanannak a rekordnak, mert a rekordkulcs `(index, scene_id, view_id)` különbözik. Várt eredmény: definíciók és a kézi példa sémájában megadott ellenpélda.

F2 (levezetési, nehézség 2). Adott $I = \{0, 1, 2, 3\}$, $G = \{A, B\}$, $\gamma(0) = \gamma(1) = A$, $\gamma(2) = \gamma(3) = B$. Legyen $\sigma(0) = train$, $\sigma(1) = val$, $\sigma(2) = \sigma(3) = test$. Mutassa meg, hogy nem létezik $\tilde{\sigma} : G \to \{train, val, test\}$, amelyre $\sigma = \tilde{\sigma} \circ \gamma$, és azonosítsa a lemma (1) feltételének sérülését. Várt eredmény: rövid indoklás a jól definiáltság meghiúsulásával.

F3 (implementációs, nehézség 2). Írjon `check_group_split(meta)` függvényt, amely egy `index,scene_id,split` oszlopokat tartalmazó pandas DataFrame-re visszaadja azon `scene_id` értékek listáját, amelyek egynél több splitbe esnek, és tesztelje a kézi példa hat során, valamint egy hibás változaton. Várt eredmény: a függvény kódja, a két hívás és a visszaadott értékek.

F4 (hibakeresési, nehézség 2). A következő kódrészlet képszintű indexeket sorsol, majd a kapott indexeket nevezi csoportos splitnek:

```
order = np.random.permutation(np.unique(meta["index"]))
```

Mi a hiba, mi a következménye a GlyphLabben, és hogyan javítjuk úgy, hogy a scene legyen a sorsolási egység? Várt eredmény: a hiba lokalizálása, a következmény a két view szétválására, és a javított sorsolási sor, valamint a visszaellenőrzés módja.

F5 (kísérleti/transzfer, nehézség 3). Válasszon egy tetszőleges külső képadatkészletet, és töltse ki a labor külsőadat-protokolljának tízlépéses sablonját. Adja meg a megfigyelési egységet, a licencvizsgálat tervét, a duplikátum- és annotáció-ellenőrzést, valamint a split utáni auditparancsot. Várt eredmény: a teljes terv szövege, nem letöltött adat; a letöltés hiánya kifejezetten jelölendő. *Értékelési szempont:* mind a tíz pont kitöltve; a letöltés hiánya és az adatkártya tényleges ellenőrzésének szándéka jelölve; a licenc, a duplikátumkezelés és a csoportos split konkrét lépésekkel szerepel; a split utáni auditparancs megnevezett.

22.1.8 Megoldások és indoklás

F1. A tartalom-duplikátum kritériuma a bájtábrázolás azonossága azonos felbontású, csatornaszámú tenzorokra; a $h(x_i) = h(x_j)$ hash-egyezés ezt közelíti, de ütközés esetén téves pozitív lehet. A függő pár akkor áll fenn, ha $\gamma(i) = \gamma(j)$ ugyanazon jelenethez, beteghez vagy felvételhez tartozik: a két GlyphLab view közös geometriai előfutárból származik, de a view-specifikus zaj miatt eltérő pixelsorokat kap, tehát általában nem tartalom-duplikátum. Az ellenpélda: legyen a generátor egy olyan futtatása, amelyben mindkét view-ra nulla zaj jut; ekkor a pixelsorok azonosak, és a két view címkéje is azonos, hiszen ugyanahhoz a jelenethez tartoznak, tehát ugyanazt az orientációt hordozzák. Az `(index, scene_id, view_id)` rekordkulcs azonban különbözik, ezért a rekord-duplikátum nem áll fenn, és a két képet nem lehet egyetlen rekordként kezelni. Az azonos címke tehát nem elégséges feltétel a rekordazonossághoz; a rekordidentitást a rekordkulcs

hordozza. A megkülönböztetés gyakorlati következménye: a duplikátum eltávolításakor a content hash törölheti a függő párokat, ha a rekordkulcs nincs bevonva.

F2. Tegyük fel, hogy létezik $\tilde{\sigma}$. Ekkor $\tilde{\sigma}(A) = \tilde{\sigma}(\gamma(0)) = \sigma(0) = \mathit{train}$, de $\tilde{\sigma}(A) = \tilde{\sigma}(\gamma(1)) = \sigma(1) = \mathit{val}$, tehát $\tilde{\sigma}(A)$ egyszerre két különböző érték, ami ellentmondás. Az (1) feltétel sérülése: $\gamma(0) = \gamma(1) = A$, de $\sigma(0) \neq \sigma(1)$. A jól definiáltság bizonyításának az a lépése hiúsul meg, amely a reprezentáns függetlenségét használja; az A csoportnak nincs egyetlen splitértéke.

F3. Lehetséges megoldás:

```
import pandas as pd

def check_group_split(meta: pd.DataFrame) -> list:
    # meta oszlopai: index, scene_id, split
    nsplit = meta.groupby("scene_id")["split"].nunique()
    return list(nsplit[nsplit != 1].index)

# a kézi példa hat soros, helyes változata
good = pd.DataFrame({
    "index": [0,1,2,3,4,5],
    "scene_id": ["G0-0000","G0-0000","G1-0000","G1-0000","G2-0000","G2-0000"],
    "split": ["train","train","train","train","val","val"],
})
print(check_group_split(good))  # []

# hibás: a G2-0000 egyik view-ja train lett
bad = good.copy()
bad.loc[5, "split"] = "train"
print(check_group_split(bad))   # ['G2-0000']
```

A visszaadott üres lista a csoportos tulajdonság teljesülését jelenti; a hibás példában a `G2-0000` azonosítja a sértő csoportot.

F4. A hiba: a kód a `meta["index"]` egyedi értékeit sorsolja, nem a `meta["scene_id"]` egyedi értékeit. A sorsolás maga nem tesz csoportos splitet; az eredmény véletlenül sem garantálja, hogy az azonos `scene_id`-hez tartozó view-k azonos splitbe kerülnek. A GlyphLabben ez azt jelenti, hogy ugyanannak a jelenetnek az egyik view-ja a train, a másik a val halmazba kerülhet, így a validációs hibát a trainben látott jelenet torzítja, és a független egységek száma helytelenül a képszám lesz. Javítás: a jelenetazonosítókat sorsoljuk, majd az eredeti indexeket a sorsolt `scene_id`-k alapján válogassuk:

```
import numpy as np

scenes = np.unique(meta["scene_id"])
rng = np.random.default_rng(20261020)
perm = rng.permutation(scenes)
val_scenes = set(perm[:30])
test_scenes = set(perm[30:60])
train_scenes = set(perm[60:])
assert len(val_scenes) == len(test_scenes) == 30
assert len(train_scenes) == len(scenes) - 60

# A képindexet a saját index oszlop adja, nem a pandas sorszám.
is_val = meta["scene_id"].isin(val_scenes)
is_test = meta["scene_id"].isin(test_scenes)
train_idx = meta.loc[~is_val & ~is_test, "index"].to_numpy()
val_idx = meta.loc[is_val, "index"].to_numpy()
test_idx = meta.loc[is_test, "index"].to_numpy()

# visszaellenőrzés: a három képindex-halmaz diszjunkt és lefedő
assert set(train_idx).isdisjoint(val_idx)
```

```
assert set(train_idx).isdisjoint(test_idx)
assert set(val_idx).isdisjoint(test_idx)
assert len(train_idx) + len(val_idx) + len(test_idx) == len(meta)

# a döntés jelenetszintű: minden jelenet pontosan egy új splitbe esik
new_split = meta["scene_id"].map(
    {**{s: "val" for s in val_scenes}, **{s: "test" for s in test_scenes},
     **{s: "train" for s in train_scenes}})
assert new_split.notna().all()
assert check_group_split(meta.assign(split=new_split)) == []
```

A visszaellenőrzés után minden jelenet egy splitbe esik; a képindexek szétválasztása már a csoportos döntést követi. A három részhalmaz 120/30/30 jelenetet ad, ami megegyezik a canonical GlyphLab split arányával, de nem feltétlenül ugyanazokkal a jelenetekkel; a gyakorlatban a `metadata.csv` `split` oszlopát kell használni, ez a kód csak a csoportos sorsolás mechanizmusát mutatja. A seed a könyv 20261020-as értéke, és a `default_rng` lokális generátort használ a globális `np.random.seed` helyett.

F5. Megoldásminta a CIFAR-10 választásához. A terv első pontja: képek osztályozása tíz osztályba; a bemenet alakját ($3 \times 32 \times 32$ tenzor) és az osztályszámot a választott tükör adatkártyáján kell ellenőrizni, ez a leírás nem helyettesíti az adatkártyát. A 2. és 3. pont kitöltetlen mezői a minta jellegét mutatják: `[tükör URL: ______]`, `[hozzáférés dátuma: ______]`, `[archívum SHA-256: ______]`, `[licencszöveg és licencverzió: ______]`; ezeket a hallgatónak a tényleges beszerzés után kell kitöltenie. A megfigyelési egység: a kép, feltéve, hogy az adatkártya nem közöl olyan csoportosítást, amely egy forrásból több képet rögzít; ezt az adatkártya tényleges olvasása után kell megerősíteni. Duplikátum-ellenőrzés: a teljes archívum SHA-256-ja az archívum szintjén pontos azonosságot jelez; a kibontott képfájloknál a bájtszintű összehasonlítás a pontos tartalom-duplikátumot, a perceptual hash (például pHash) pedig a közel-duplikátumot keresi. A két fogalmat külön kell jelenteni: a perceptual hash egyezése nem definíció szerinti tartalom-duplikátum, hanem közel-azonos kép. Annotáció: az osztálycímkét a terjesztő leképezése szerint kell kezelni, és a 0–9 tartományt az adatkártyán ellenőrizni. Split: a csoportos tulajdonságot a képazonosító szintjén ellenőrizzük, és a kiválasztott splitet a fenti `check_group_split`-hez hasonló függvénnyel zárjuk. A közölt korlát: a kapott modellhiba a választott tüköradatra vonatkozik, nem a „CIFAR-10” név által jelölt bármely másolat univerzumára.

Változó és metrika: a változó tényező a vizsgált tükör; a metrika az auditált eltérések száma (hiányzó kulcs, duplikátumcsoport, split-átfedés), a megfigyelési egység egy adathalmaz-rekord. **Hipotézis, kontroll, döntési szabály és erőforrás:** a hipotézis, hogy a kiválasztott tükör a tízlépéses protokollal reprodukálható módon használható, ha a licenc a kutatási felhasználást engedi, az archívum SHA-256-ja rögzíthető, és a csoportos split a duplikátumok kiszűrése után sértetlen. Kontroll: a letöltés nélküli terv, amelyben minden ellenőrzés az adatkártya és a közzétett metaadat ellenőrzése; a minta nem állítja, hogy az adat letölthető vagy hogy a licenc engedi. Előre rögzített döntési szabály: a tükör csak akkor fogadható el, ha a licenc a kutatási felhasználást kifejezetten engedi, az archívum SHA-256-ja dokumentált, és a pHash-tal talált közel-duplikátumok mindegyike egy csoportba és egy splitbe kerül; ha bármelyik feltétel nem teljesül, a tükör elutasítandó. A döntéshez legalább 100 véletlen képből álló ellenőrző minta kell, amelyet seed 20261020-szal húzunk. A feladat nem tanít modellt, ezért architektúra és optimalizáló helyett a tükör kontrollja és a döntési szabály szerepel. Erőforrás: a feladat szándéka szerint nincs letöltés; a terv és az adatkártya-ellenőrzés CPU-n, néhány óra dokumentációs munka. ### Önellenőrzés és korábbi fogalmak felidézése

Önellenőrzés: (1) A csoportos split tulajdonság a rekordok permutációjától független? Igen, mert (1) csak indexpárokra tett feltétel. (2) A SHA-256 egyezés önmagában bizonyítja a jogosultságot?

Nem, csak a változat azonosságát; a licenc külön rekord. (3) Két pixelszinten azonos view duplikátum-e? Tartalom szerint igen, rekordszinten nem, ha a rekordkulcs különbözik.

Korábbi fogalmak: az 5.4. Függő képek, seedek és resampling csoportos splitje itt a lemmában kap reprezentációs alakot: a $\sigma = \tilde{\sigma} \circ \gamma$ felírás pontosan azt rögzíti, hogy a split a jeleneteken definiált. A 10.1. Számítási gráf és lokális derivált számítási gráfjában egy közös paraméterhez több ág hozzájárulása összegződik; itt az adatoldalon egy közös `scene_id`-hez több rekord tartozik, és a split-döntést a közös kulcs szintjén kell meghozni. A közös vonás tehát nem a gradiens-összegzés iránya, hanem az, hogy a döntést a közös kulcs (ott a paraméter, itt a `scene_id`) szintjén hozzuk meg. A hivatkozás integritását két irányban ellenőrizzük: minden rekord pontosan egy példányra hivatkozik (p injektív), és minden példányhoz legfeljebb egy rekord tartozik; a rekordazonosságot az `(index, scene_id, view_id)` kulcs hordozza. A 21.1. Inicializálás és varianciaterjedés inicializálási seedje a provenance-rekordban tér vissza: a generátor seedje nélkül a manifest csak statisztika, nem reprodukálható szerződés. Végül a 18.1. Reziduális blokk és identitásút identitásútja a bemenetet változatlanul viszi át a blokkon; hasonlóan az `index` mező viszi változatlanul a rekord és a képtömb sora közötti kapcsolatot a splitelésen és a permutációkon át.

22.1.9 Rövid összefoglaló

Az adat használatának első szerződése a provenance, a licenc és a megfigyelési egység hármasa. A provenance a generáló folyamatot, seedet és lenyomatot rögzíti; a licenc a konkrét változathoz tartozó jogosultság, nem a név következménye; a duplikátum tartalom- és rekordszinten más fogalom, mint a függő nézetpár. A GlyphLabben a statisztikai egység a jelenet, ezért a split a 180 jelenet partíciója, nem a 360 képé. A lemmában belátott $\sigma = \tilde{\sigma} \circ \gamma$ reprezentáció és a címke-hivatkozás injektivitása a split- és annotáció-ellenőrzés két strukturális invariánsa. A labor a manifestet és a metadata-t ezen invariánsok mentén auditálja. A nyitó kérdésre a válasz: a 240 train kép 120 független jelenetből származik, ezért a statisztikai egységek száma 120, nem 240.

22.1.10 Következő lecke

A 22.2. Címkemegőrző transzformációk a címkemegőrző transzformációkat vizsgálja: ott az itt rögzített hivatkozás-egyediségre építve azt kérdezzük, mely kép- és annotációtranszformációk őrzik meg a ℓ címkét. A függelék A. szakasza szerint a GlyphLab címkéje a vonal orientációja (0 = vertical, 1 = horizontal, 2 = diagonal), és a generátor a diagonal osztályban csak az egyik ferde irányt állítja elő; ezért a 90°-os forgatás a vertical és horizontal címkét felcseréli, a diagonálist pedig a nem generált másik ferde irányba viszi, azaz nem címkemegőrző.

22.1.11 Felhasznált források

- [S05] Davison, A. C. és Hinkley, D. V. (1997). *Bootstrap Methods and Their Application*. Cambridge University Press.
- GlyphLab saját laboradatok: `data/manifest.json`, `data/metadata.csv`, a könyv ADATOK.md szerződése szerint.

22.2 Címkemegőrző transzformációk

22.2.1 Cél, előismeret és induló kérdés

A 22.1. Adateredet, címke és csoportos split egység rögzítette, hogy a függelék A. szakaszában leírt GlyphLab 360 képéhez 180 független jelenet tartozik, és a splitet scene-szinten kell nyilvántartani. Ott a címke és a rekord kapcsolatáról a *hosszanti*, adatkezelési oldalon volt szó. Most ugyanennek a szerződésnek a *transzformációs* oldalát vizsgáljuk: ha a betanított modellt egy augmentációs pipeline-on keresztül tápláljuk, mikor marad a kimeneti címke változatlan, és mikor változtatja meg a transzformáció magát a feladatot?

A 21.3. Dropout, súlyregularizálás és augmentáció a regularizáló eszközöket úgy vezette be, hogy az augmentációt csak megemlítette. A 10.2. Affín réteg és aktiváció backward a backward VJP-szabályainál a batch- és broadcast-műveleteket kezelte; ott minden tenzorműveletnek egzakt, indexelt definíciója volt. A 18.2. Batch normalization train és eval módban pedig a BatchNorm train/eval kettősségénél hangsúlyozta, hogy ugyanaz a forward-képlet különböző üzemmódban más statisztikát használ. Ugyanez a minta tér vissza az augmentációban: ugyanaz a geometriai művelet egészen más szerződést teljesít, ha a címkét, a maszkot és a dobozt is transzformáljuk vagy éppen nem.

Induló kérdés. A GlyphLab osztályai: 0 = vertical, 1 = horizontal, 2 = diagonal. Ha a 0. osztályú képet 90 fokkal elforgatjuk, a megjelenő gerenda vízszintes lesz. Az eredeti címke 0 maradhat-e? A becslését a kézi példa után ellenőrizheti: ott a számítás nem szorzás, hanem egy címketér-leképezés felírása lesz.

22.2.2 Definíció: augmentáció, transzformáció és a címkemegőrző szerződés

Definíció (augmentáció). Legyen (X, y) egy kép-címke pár, ahol $X \in \mathcal{X} \subseteq \mathbb{R}^{C\times H\times W}$ (a batch-nélküli GlyphLab-konvencióban $C = 1$), H a magasság, W a szélesség, $\mathcal{X}$ pedig a megengedett képek halmaza. Az augmentáció egy $\mathcal{T} : \mathcal{X} \to \mathcal{X}$ adattranszformáció, amelyet a tanítási adatfolyamban a minibatch betöltésekor alkalmazunk, nem előre elmentett fájlként. A célja a gyakorlati variabilitás növelése a tényleges adatgyűjtés nélkül; a modell így ugyanazt a tanítópéldát több realizációban láthatja.

A transzformáció akkor része a szerződésnek, ha külön mondjuk meg, hogy milyen objektumra hat. Két különböző, nem felcserélhető objektumról van szó: a *feladat* címkéjéről, amelyet az f címkézőfüggvény rögzít, és a *tanított modell* kimenetéről, amelyet a h pontszámfüggvény ad.

Definíció (címkemegőrző transzformáció). A $\mathcal{T}$ transzformáció akkor **címkemegőrző** az adott tanítási feladatra, ha minden olyan (X, y) párra, ahol $X \in \mathcal{X}$ és $\mathcal{T}$ értelmezve van X-en, a transzformált $X' = \mathcal{T}(X)$ képhez is az y osztály tartozik. Rövidebben: a feladat f címketér-leképezése kielégíti az

$$f(\mathcal{T}(X)) = y = f(X)$$

egyenlőséget minden megengedett X-re; a gyakorlatban ezt a tanítási halmaz elemein ellenőrizzük.

A jelölés szándékosan „az adott feladatra” szól. A definícióban f a *feladat* címkézőfüggvénye, nem a tanított modell. Például a GlyphLab feladatban f a vonás áthaladási irányát adja, és ez a modell címkéjének értelmezését rögzíti, függetlenül attól, hogy a betanított súlyok mit jósolnak. Két rövid megjegyzés a definícióhoz:

1. A címkemegőrzés **nem** ugyanaz, mint a vizuális hasonlóság. Egy transzformáció lehet percepciósan jelentéktelen, de a feladat címkéjét mégis megváltoztatja – lásd a forgatási ellenpéldát, ahol az emberi szem ugyanazt a figurát látja, de az orientációcímke megváltozott.
2. A címkemegőrzés sem ugyanaz, mint a **modellinvariancia**. Az *invariance* kifejezést a 16.1. Eltolási equivariance és invariance egység a modell fekete dobozára használta: $h(\mathcal{T}(X)) = h(X)$, ahol h a tanított modell pontszáma. Itt a szerződés a *feladat* címkéjére, nem a modell outputjára vonatkozik.

A szerződés egy konkrét, reprodukálható pipeline-ban ellenőrizhető: minden olyan X-re, amelyre a transzformációt megengedjük, a transzformált $X' = \mathcal{T}(X)$ képhez is az y címkének kell tartoznia a céltenzorgenerátor szerint; ez a definíció közvetlen következménye, nem külön teszt. A szakirodalomban ezt a tulajdonságot gyakran *label-preserving* (címkemegőrző) transzformációnak nevezik; a PyTorch dokumentációja [D08] a Dataset és DataLoader folyamatában a transzformációk általános alkalmazását rögzíti, de a címke- és annotáció-szinkronizálást a felhasználó szerződéseként kezeli. Az adott transzformációt megvalósító API pontos paramétereit a hallgató a telepített könyvtár dokumentációjában ellenőrzi (a torchvision transzformációit a torchvision.transforms oldal írja le).

22.2.3 A transzformációk alakja, indexelése és a levezetés ellenőrzése

22.2.3.1 Crop és a kivágás indexhalmaza

A crop művelet az indexek leszűkítése. Egy kivágási ablakot fél-nyílt, nullától indexelt intervallumokkal adunk meg: az i_{top} sorral és az i_{left} oszloppal kezdődik, és i_{height} sort, valamint i_{width} oszlopot fog át. A 22.1. Adateredet, címke és csoportos split alfejezet doboza fél-nyílt xyxy alakú; a kivágási ablak ezzel ekvivalens xyxy alakja $[i_{\text{left}},\, i_{\text{top}},\, i_{\text{left}} + i_{\text{width}},\, i_{\text{top}} + i_{\text{height}})$, ahol az első két koordináta a sor–oszlop sorrend miatt felcserélődik. A kivágott kép

$$X'_{c,i,j} = X_{c,\, i_{\text{top}}+i,\, i_{\text{left}}+j}, \qquad c < C,\ i < i_{\text{height}},\ j < i_{\text{width}}.$$

A magasságra a fél-nyílt feltétel: $i_{\text{top}} \le i < i_{\text{top}}+i_{\text{height}}$, azaz a kivágott terület az $[i_{\text{top}}, i_{\text{top}}+i_{\text{height}})$ intervallum. Az output alakja $C \times i_{\text{height}} \times i_{\text{width}}$.

A crop **címkemegőrző** az osztályozási feladatra, ha a kivágás választását az osztályt meghatározó tartalomtól függetlenül rögzítjük, és a kép tartalma elégséges a címke eldöntéséhez. Ahhoz, hogy ezt a feltételt ne csak heurisztikusan mondjuk ki, a tanulási szerződés szempontjából rögzítjük: a transzformáció akkor elfogadott, ha a kivágás utáni kép az eredeti X-ből származó, a cél szempontjából reprezentatív mintája. A crop önmagában nem címkemegőrző eljárás, hanem indexhalmaz-művelet; a címkemegőrzés a kivágási politikától függ. Például a GlyphLabban egy olyan kivágás, amely a gerendát teljesen elhagyja, megszünteti a címkét hordozó tartalmat, tehát megsérti a szerződést, még ha a pixelformát tekintve simának is tűnik.

A hivatalos követelmény a kép és a társannotáció szinkronja esetén szigorúbb, hiszen a maszkhoz és dobozhoz is ugyanazt az indexhalmazt kell használni. A kézi példában a doboz transzformációját külön számoljuk ki.

22.2.3.2 Resize és a kép indexeinek újrahálózása

A resize a diszkrét kép rácsát egy másik rácsra képezi. A legátláthatóbb, matematikailag is következetes fogalom a **bilineáris interpoláció**: az $X_{c,i,j}$ intenzitást az i, j diszkrét pozíciójú minta folytonos interpolációjának tekintjük, és az X' kimenet p, q mintájához a forrásrácsban keressük a hozzá tartozó folytonos koordinátákat. Pontosabban, az output (p, q) pixel centruma a forrásrács (η, ξ) koordinátájára esik, ahol $\eta = (p + 0{,}5)\, H/H_{out} - 0{,}5$ (sor) és $\xi = (q + 0{,}5)\, W/W_{out} - 0{,}5$ (oszlop); ez a pixelcentrum-konvenció (`align_corners=False` a PyTorchban). A részletes kernel pozíciófüggő: a $w_{r,s}(p, q)$ súlyok a forrás- és célrács koordinátáitól függenek, a pixelintenzitásoktól viszont nem. Rögzített be- és kimeneti rács mellett ezért a resize az intenzitásokra nézve **lineáris** operátor, és a kép minden csatornájára ugyanaz:

$$X'_{c,p,q} = \sum_{r=0}^{H-1} \sum_{s=0}^{W-1} X_{c,r,s}\, w_{r,s}(p, q), \qquad 0 \le p < H_{out},\ 0 \le q < W_{out}.$$

ahol a w súlyok a célrácstól függő, bilineáris kernel. Tehát $R(aX + bY) = aR(X) + bR(Y)$ minden a, b valós számra és azonos méretű X, Y képre; a bilineáris interpoláció koordinátafüggése ezen nem változtat. Valóban nemlineáris lépés csak az, ami az eredményre utólag hat, például a $[0, 1]$-be vágás (clipping) vagy a kerekítés `uint8`-ra; ezeket a szerződésben külön kell megnevezni. Az antialias-szűrő mérete a skálafaktortól függ, nem a képtartalomtól, ezért rögzített be- és kimeneti rács mellett a súlyai is rögzítettek, és a művelet lineáris marad. A 14.1. Stride, padding, dilation és outputméret padding/stride képletétől eltérően itt nincs konvolúciómag – ez a művelet *mintavételi* átkonvertálás, nem szűrő.

A resize akkor címkemegőrző az osztályozási feladatra, ha a feladat nem függ egy képpontos részlet abszolút méretétől, csak a geometriai tartalom relatív konfigurációjától. Ez a GlyphLab orientációs feladatán teljesül: a gerenda függőleges vagy vízszintes iránya egy kisebb rácson is olvasható. Az alábbi feltételt írásban is rögzítjük: a resize akkor engedhető meg, ha a célrács felbontása nem esik a címkét hordozó struktúra megkülönböztetéséhez szükséges minimális felbontás alá, azaz a kicsinyítés nem tünteti el a címkét meghatározó részletet. Ha például 24×24-ről 4×4-re kicsinyítünk, egy vékony, 2 pixel vastag vonal eltűnhet; az ilyen resize nem címkemegőrző, mert a cél-információ megsemmisül. Ez a megállapítás **heurisztika**, nem bizonyított tétel: a megengedett minimális méretet a konkrét generátor vastagságeloszlása ellenőrzi, amit a 22.1. Adateredet, címke és csoportos split generátorának dokumentációja rögzít.

22.2.3.3 Flip: tengelycsere, ami csak bizonyos tengelyekre címkemegőrző

A *horizontal flip* a szélességi indexet cseréli (bal–jobb csere), geometriailag a függőleges tengelyre tükröz; a „horizontal/vertical” elnevezés a változó irányt jelöli, nem a tükörtengely nevét:

$$X'_{c,i,j} = X_{c,i,W-1-j}, \qquad i < H, j < W.$$

A *vertical flip* (fel–le csere, geometriailag a vízszintes tengelyre tükrözés) analóg:

$$X'_{c,i,j} = X_{c,H-1-i,j}.$$

A flip fontos tulajdonsága, hogy az **osztályozási címkét** csak az adott feladat hagyja meg, míg az **annotációt** szinkronban kell tükrözni. Ez a megkülönböztetés a jelen fejezet magja. Egy kép vízszintes tükrözése a vizuális tartalmat megőrzi, de a GlyphLabban a diagonal osztály csak az

egyik ferde irányt tartalmazza. A GlyphLab ADATOK.md szerződése kifejezetten rögzíti: a 90 fokos forgatás a vertical és horizontal címkét felcseréli, és a diagonal osztály csak az egyik ferde irányt tartalmazza, ezért tükrözés/forgatás új célértelmezést igényelhet. A tükrözött diagonal már a másik ferde irány, ami egy másik feladatcímke lenne, nem a meglévő 2. osztály.

22.2.3.4 Rotation és a levezetés ellenőrzése: ellenpélda a címkemegőrzésre

Állítás. Nem minden forgatás címkemegőrző minden osztályozási feladathoz. Konkrétan a GlyphLab orientációs feladatán a 90 fokos forgatás a vertical és horizontal osztályokat kölcsönösen felcseréli, így a transzformáció az eredeti címke megtartásával nem érvényes.

Bizonyítás. Legyen a GlyphLab feladat címkézője $f : \mathcal{X} \to \{0, 1, 2\}$, ahol a 0 a függőleges, 1 a vízszintes, 2 az átlós vonalat rendeli a képhez. Legyen $\mathcal{R}_{90}$ a képsík óramutató járásával megegyező 90 fokos forgatása; egy $H \times W$ képen ennek indexképlete $(i, j) \mapsto (j, H - 1 - i)$, ami megfelel a `np.rot90(x, k=3)` műveletnek. A címke definíciója szerint f a vonás áthaladási irányát adja vissza, ezért a bizonyítás csak ezt a definíciót és a forgatás geometriáját használja. Egy függőleges, az i-tengely mentén hosszan elnyúló vonal a forgatás után a j-tengely mentén hosszan elnyúlt, azaz vízszintes vonal lesz:

$$f(\mathcal{R}_{90}(X_{\text{vertical}})) = 1, \qquad \text{de } f(X_{\text{vertical}}) = 0.$$

A vízszintes vonalra szimmetrikusan:

$$f(\mathcal{R}_{90}(X_{\text{horizontal}})) = 0, \qquad \text{de } f(X_{\text{horizontal}}) = 1.$$

Mivel tehát $f(\mathcal{R}_{90}(X)) \neq f(X)$ legalább egy X-re, a $\mathcal{R}_{90}$ transzformáció nem címkemegőrző a GlyphLab feladatára. ■

A bizonyítás a definiált célfeladat szintjén teljes; nem szükséges hozzá általános képi ontológia. Szándékosan mutattuk meg, hogy a címkemegőrzés feladattól függ, nem a forgatóképesség gyenge. Egy olyan feladatnál, ahol a cél a vonal *jelenléte* (nem annak iránya), a $\mathcal{R}_{90}$ címkemegőrző lenne: a vonal jelen marad. A fontos üzenet tehát: a geometriai invariancia a cél *definíciójához* képest értendő.

22.2.3.5 Intenzitástranszformáció

Az intenzitástranszformáció a c csatornán a $[0, 1]$ tartományú intenzitásra hat, nem az indexre. Az egyszerű eltolás-skálázás például:

$$X'_{c,i,j} = aX_{c,i,j} + b,$$

ahol $a > 0$ és b skalár, a kimenetet pedig a $[0, 1]$ tartományra vágjuk vagy normalizáljuk. A GlyphLab $[0, 1]$ képein a $0 \leq aX + b \leq 1$ megkötés azt jelenti, hogy a transzformáció nem telítődik, azaz nincs levágás (clipping). Ha a megkötést elhagyjuk, a levágás a nagy intenzitású pixeleket azonos szintre hozhatja, és ezzel a vonal eltűnhet; a levágás tehát a látható tartalmat is megszüntetheti. Affin intenzitástranszformáció a vonal geometriai helyzetét nem változtatja meg, ezért a fényerő- (brightness) és kontrasztváltoztatás az orientációs címkét önmagában nem sérti; a valódi feltétel az, hogy a vonal észlelhető maradjon. Fontos korlát: ha az intenzitásművelet a zaj szintjét a kép szubjektív osztályozhatóságán túl növeli, a cél címkéje már nem az eredetiből

igazoltan levezethető. Ez intenzitásnál és zajosításnál **észlelhetőségi** feltétel, és a fejezetben mint ellenőrizendő heurisztikát jelöljük.

A normalizálás és az intenzitás-skálázás közötti különbség: az előbbi a modell numerikus bemenetét skálázza, az utóbbi a kép vizuális tartalmát módosítja. A címkemegőrzés szempontjából a minden képen ugyanazzal az affin intenzitástranszformációval végzett normalizálás akkor megengedett, ha a kép normája nem a címke döntő forrása. A GlyphLab feladatban a címkét az átvonuló iránya szolgáltatja, nem az átlagos fényesség, ezért a normalizálás önmagában megengedhető.

22.2.3.6 Mask és box szinkronizáció

A maszk és doboz ugyanazon képhez tartozó annotáció. A 22.1. Adateredet, címke és csoportos split szerint a maszk az előtér 0/1 döntése, a doboz a legszűkebb befoglaló téglalap fél-nyílt xyxy formában. Ha a kép indexeit transzformáljuk, a maszk és doboz indexeit ugyanazzal a geometriai leképzéssel kell transzformálni, különben a kép-pozíció és az annotáció elcsúszik.

A szinkronizáció pontos formája a következő. Legyen a geometriai index-leképezés σ, amely az output kép minden i, j indexéhez egy i', j' inputpozíciót ad. Ha a maszkot a kép transzformációjával azonos módon transzformáljuk, akkor a címke-annotáció ugyanarra a vizuális objektumra vonatkozik. A doboz esetében ez azt jelenti, hogy a sarkokat is a σ-nak megfelelően kell leképezni, figyelve a fél-nyílt intervallum konvencióra.

Ha a transzformáció szöget vagy méretet változtat, a maszkot újra kell mintavételezni. A maszk 0/1 értékeit legközelebbi szomszéd (*nearest*) interpolációval kell transzformálni, mert a bilineáris interpoláció köztes értékeket (például 0{,}5-öt) hozna létre, amelyek nem tartoznak egyik osztályhoz sem. A dobozt nem interpoláljuk: a sarkait a geometriai leképezéssel számoljuk át, majd tetszőleges affin transzformáció esetén egész indexekre kerekítjük; flip és 90 fokos forgatás esetén a kerekítés pontos, mert az indexek egészek maradnak.

Példa (doboz horizontal flip). Legyen a kép szélessége W. Egy doboz $[x_{\min}, y_{\min}, x_{\max}, y_{\max})$ tükrözött alakja:

$$[W - x_{\max},\ y_{\min},\ W - x_{\min},\ y_{\max}).$$

A fél-nyílt konvenciót a tükrözés is megőrzi, ha a pixelindexekkel számolunk: a doboz az $i \in \{x_{\min}, \dots, x_{\max} - 1\}$ indexű pixeleket tartalmazza, a tükrözött index $j = W - 1 - i \in \{W - x_{\max}, \dots, W - 1 - x_{\min}\}$, azaz éppen a $[W - x_{\max}, W - x_{\min})$ fél-nyílt tartomány (a folytonos koordinátás $W - x$ leképezés ezzel szemben balról nyílt, jobbról zárt intervallumot adna, ezért itt az indexszámolás a mérvadó). A kézi példa ezt számolja ki.

A **gyakori tévedés** az, hogy a képhez egy véletlen transzformációt választunk, a maszkhoz/dobozhoz pedig egy másik, tőle független véletlent. Ez hibás tanítópárt eredményez, mert a kép és az annotáció különböző geometriai kölcsönhatásnak van kitéve. Az F4 feladat pontosan ezt a hibát lokalizálja.

22.2.4 Kézi kis példa: a GlyphLab 90 fokos rotációja és dobozszinkron

Tekintsünk egy 24×24-es, a GlyphLab-szerű generátorral készült vertical osztályú képet. A vonal két pixel széles, a $j = 12$ és $j = 13$ oszlopban fut függőlegesen. A címke 0. A doboz, mint a vonalat befoglaló legszűkebb téglalap:

$$[x_{\min}, y_{\min}, x_{\max}, y_{\max}) = [12, 4, 14, 20).$$

Azaz a vonal a 12. és 13. oszlopban, a 4. és 19. sorban látható (a fél-nyílt intervallum szerint $y_{\max} = 20$ már nincs benne).

Az óramutató járásával megegyező 90 fokos forgatás (`np.rot90(x, k=3)`) egy $H \times W$ képet $W \times H$ képpé alakít, és a sor–oszlop indexpárt így képezi le:

$$(i, j) \mapsto (i', j') = (j,\ H - 1 - i).$$

Az új sorindex tehát a régi **oszlop**index ($i' = j$), az új oszlopindex a régi sorindex tükre ($j' = H - 1 - i$). A dobozra: a régi $x \in [x_{\min}, x_{\max})$ oszloptartományból az új sortartomány lesz, $y' \in [x_{\min}, x_{\max})$; a régi $y \in [y_{\min}, y_{\max})$ sortartományból pedig az új oszloptartomány $x' = H - 1 - y$, azaz $x' \in [H - y_{\max},\ H - y_{\min})$, mert $y \le y_{\max} - 1$ esetén $H - 1 - y \ge H - y_{\max}$, és $y \ge y_{\min}$ esetén $H - 1 - y < H - y_{\min}$. Az xyxy sorrendben az új doboz

$$[x'_{\min}, y'_{\min}, x'_{\max}, y'_{\max}) = [H - y_{\max},\ x_{\min},\ H - y_{\min},\ x_{\max}).$$

A szélesség és a magasság szerepet cserél, a terület változatlan: $(H - y_{\min} - H + y_{\max})(x_{\max} - x_{\min}) = (y_{\max} - y_{\min})(x_{\max} - x_{\min})$.

A példában $H = 24$:

$$[24 - 20,\ 12,\ 24 - 4,\ 14) = [4,\ 12,\ 20,\ 14).$$

Az új doboz a 12. és 13. **sort** és a 4–19. **oszlopot** fedi: két pixel magas, 16 pixel széles vízszintes sáv, területe $2 \cdot 16 = 32$, ugyanannyi, mint az eredeti $2 \cdot 16$. Ez éppen a vízszintes vonal; a címke most 1 kellene hogy legyen, a 0 címkét meghagyni nem szabad, mert a transzformáció a címkét megváltoztatta. A példa címketér-leképezése a vertical/horizontal részen $g(0) = 1$, $g(1) = 0$, a diagonal osztályon pedig a másik ferde irány; a transzformált képhez tehát $g(y)$ tartozik, nem y. Ellenőrzés: a dobozból képzett 24×24-es bináris maszkot `np.rot90(mask, k=3)`-mal forgatva, majd a legszűkebb dobozt újraszámolva szintén $[4, 12, 20, 14)$ adódik, a maszk elemszáma 32 marad.

22.2.5 Konkrét labor: a 90 fokos rotáció címkeváltoztató hatása

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5. Szükséges fájlok: `data/images.npy` és `data/metadata.csv`; csak a train splitet használjuk. A futtatás a szakaszban megadott `n086_rotation_check.py` szkripttel történik; várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A jelen alfejezethez a következő labort írjuk elő. A labor a `lab` könyvtárban dolgozik, CPU-s NumPy-vel.

Protokoll. A gyakorlat célja: a GlyphLab train-képből válasszon ki egy vertical és egy horizontal képet, és szemléltesse egy konkrét páron a 90 fokos rotáció címkeváltoztató hatását.

Lépések.

1. `python3 -c "import numpy as np; print(np.__version__)"` ellenőrzi a NumPy-verziót. A labor Python 3.10+ és NumPy 1.21+ verziót feltételez, CPU-n.
2. A képek `data/images.npy`-ből töltődnek be: `np.load("data/images.npy")`; a train indexek `metadata.csv`-ből.

3. A transzformáció a 90 fokos forgatás. Egyetlen $H \times W$ képen `np.rot90(img, k=3)` (alapértelmezett tengelyek); egy $C \times H \times W$ tömbön `np.rot90(x, k=3, axes=(1,2))`, egy $N \times C \times H \times W$ batch-en pedig `np.rot90(x, k=3, axes=(2,3))`, mert a csatorna- és a batch-tengely nem foroghat.
4. A kimeneti jelentés minden kiválasztott képhez kiírja az eredeti címkét, valamint a függőleges és vízszintes tengelyű projekció maximumát. A forgatás utáni címkét a generátor geometriai definíciója adja meg (a vertical kép horizontalba fordul); a projekciós profil ezt csak illusztrálja, nem ez a címke forrása.
5. Az ellenőrzés: a vertical kép projekciója a magasság mentén koncentrált, a horizontálja szétterül; a forgatott képen a projekció tengelyei felcserélődnek.

A lépések futtatható ellenőrzése:

```
# lab/n086_rotation_check.py - munkakönyvtár: lab/
import csv
import numpy as np

images = np.load("data/images.npy")          # (360,1,24,24) float32
labels = np.load("data/labels.npy")          # (360,) int64
with open("data/metadata.csv", newline="") as f:
    rows = list(csv.DictReader(f))
rows.sort(key=lambda r: int(r["index"]))
split = np.array([r["split"] for r in rows])
train = split == "train"
for want in (0, 1):                           # vertical, horizontal
    i = int(np.flatnonzero(train & (labels == want))[0])
    img = images[i, 0]
    rot = np.rot90(img, k=3)                  # 90 fok, óramutató szerint
    assert rot.shape == img.shape
    col0, row0 = img.sum(axis=0), img.sum(axis=1)
    col1, row1 = rot.sum(axis=0), rot.sum(axis=1)
    assert abs(col0.max() - row1.max()) < 1e-4
    assert abs(row0.max() - col1.max()) < 1e-4
    axis = ("oszlopkoncentrált (vertical)" if col1.max() > row1.max()
            else "sorkoncentrált (horizontal)")
    print(f"eredeti címke={want}: colmax={col0.max():.2f} rowmax={row0.max():.2f}")
    print(f"rotált:            colmax={col1.max():.2f} rowmax={row1.max():.2f} -> {axis}")
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU, a rögzített GlyphLab adaton):

```
eredeti címke=0: colmax=9.57 rowmax=3.76
rotált:            colmax=3.76 rowmax=9.57 -> sorkoncentrált (horizontal)
eredeti címke=1: colmax=4.65 rowmax=8.60
rotált:            colmax=8.60 rowmax=4.65 -> oszlopkoncentrált (vertical)
```

A `0` címkéjű (vertical) képből `np.rot90` után sorkoncentrált, azaz horizontal kép lesz, a címke viszont változatlanul `0`; a három `assert` a projekciós profilok felcserélődését rögzíti. Ez a forgatási állítás kézzel követhető, mért példája.

Ellenőrzött jelentés helye. A fenti ellenőrzés a rögzített GlyphLab adaton lefutott; a laborban nincs véletlen, a képek float32-esek, a generátor verziója 1.0, a generáló seed 20261020, és csak a train splitet használjuk. A tanuló a saját futtatásának eredményét a jegyzékébe írja; ezt a `n086_rotation_check.py` kimenete rögzíti.

22.2.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a

tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

22.2.6.1 F1 (fogalmi, nehézség 1)

A 16.1. Eltolási equivariance és invariance equivariance és invariance fogalmával szemben itt a címkemegőrzésről van szó. Magyarázza el röviden a különbséget, és adja meg, hogy a 90 fokos rotáció a GlyphLab osztályozón **melyik** módon sérti a címkemegőrzést: a címke definícióját vagy a modell pontszámát változtatja meg?

22.2.6.2 F2 (levezetési, nehézség 2)

Adott egy 24×24-es GlyphLab kép, rajta egy diagonal vonallal, amely a bal felső sarokból a jobb alsóba tart. A doboz $[2, 4, 22, 20)$. Számítsa ki a 90 fokos rotáció utáni dobozt, és ellenőrizze a fél-nyílt intervallum tartalmi értelmét.

22.2.6.3 F3 (implementációs, nehézség 2)

Írjon Python-függvényt, amely egy $N \times C \times H \times W$ tenzoron (NumPy-ndarray vagy `torch.Tensor`) a vízszintes flipet hajtja végre a W tengelyre. A függvény inputja a tenzor, a kimenete ugyanolyan dtype és alak; az ellenőrzéshez használjon egy $2 \times 1 \times 3 \times 4$ bemenetet a $[[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]$ panelre. Várt eredmény: minden sor fordított sorrendben.

22.2.6.4 F4 (hibakeresési, nehézség 2)

A következő pszeudokód szegmentálási augmentációt próbál létrehozni. Keresse meg a hibát, és indokolja, miért keletkezik hibás tanítópár.

```
for x, mask in loader:
    angle = np.random.uniform(0, 90)
    x_aug = rotate(x, angle)
    mask_aug = rotate(mask, angle + 5)
    train_step(x_aug, mask_aug)
```

22.2.6.5 F5 (kísérleti transzfer, nehézség 3)

A GlyphLab orientációs feladatban a class-stratified split 240 train, 60 val és 60 teszt képet ad (a tesztet ez a feladat nem használja). Tervezzen kísérletet annak ellenőrzésére, hogy egy kiválasztott flip-augmentáció a diagonal osztályra címkét változtat-e. Ne hajtsa végre teljes körűen; a protokollhoz írjon megfigyelési egységet, baseline-t, metrikát és a konklúzió érvényességi határát.

22.2.7 Megoldások és indoklás

22.2.7.1 F1 megoldás

A címkemegőrzés a feladat címketér-leképezésére vonatkozik: a transzformált kép címkéje megegyezzen. Ezzel szemben az equivariance/invariance a modellfüggvény és a transzformáció felcserélhetőségét rögzíti. A 90 fokos rotáció a GlyphLab cél **definícióját** változtatja meg, mert az orientáció címkéje maga az irány; a modell viselkedése ettől független kérdés. A feladatcím már a kalibrált cél definíciójában sérül.

22.2.7.2 F2 megoldás

A diagonal doboz $[2, 4, 22, 20)$ a 0-tól számozott, felülről lefelé indexelt rácsban a bal felső és a jobb alsó sarkot foglalja be. A 90 fokos óramutató szerinti forgatás képlete $(i, j) \mapsto (j, H - 1 - i)$, a kézi példa szerint az új doboz $[H - y_{\max},\, x_{\min},\, H - y_{\min},\, x_{\max})$:

$$[24 - 20,\ 2,\ 24 - 4,\ 22) = [4,\ 2,\ 20,\ 22).$$

A fél-nyílt intervallum most a 2–21. sort és a 4–19. oszlopot jelöli; a doboz most 20 sor magas és 16 oszlop széles (korábban 16 sor magas és 20 oszlop széles volt), a terület (320) változatlan. A `np.rot90(mask, k=3)` után újraszámolt doboz ugyanez. A forgatott képen ez éppen a bal alsó sarokból a jobb felsőbe tartó diagonal; a forgatás a geometriai irányt a másik ferde irányba vitte, így a diagonal osztály a forgatás után már nem áll fenn. A válasz konzisztens a forgatási állítással: a doboz szinkronban transzformálódott, a címke azonban a cél definíciója miatt megváltozott.

22.2.7.3 F3 megoldás

A NumPy-megoldás teljes futtatható ellenőrzése:

```
import numpy as np

def horizontal_flip_nchw(X):
    # X: ndarray, N x C x H x W
    return X[..., ::-1].copy()

x = np.arange(24, dtype=np.float32).reshape(2, 1, 3, 4)
# A feladat panelje: [0,1,2,3], [4,5,6,7], [8,9,10,11]
x[0,0] = [[0,1,2,3],[4,5,6,7],[8,9,10,11]]
x[1,0] = [[12,13,14,15],[16,17,18,19],[20,21,22,23]]
y = horizontal_flip_nchw(x)
print(y[0,0].tolist())
# [[3.0, 2.0, 1.0, 0.0], [7.0, 6.0, 5.0, 4.0], [11.0, 10.0, 9.0, 8.0]]
```

A `torch.Tensor` bemenethez a `X[..., ::-1]` negatív lépésközű indexelés nem használható (`ValueError: step must be greater than zero`), és a `Tensor`-nek nincs `.copy()` metódusa; a helyes megoldás a `torch.flip`:

```
import torch

def horizontal_flip_nchw_torch(X):
    # X: torch.Tensor, N x C x H x W
    return torch.flip(X, dims=(-1,))
```

A futtatott ellenőrzés (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu) a két implementációra ugyanazt a kimenetet adja: `[[3.0, 2.0, 1.0, 0.0], [7.0, 6.0, 5.0, 4.0], [11.0, 10.0, 9.0, 8.0]]`.

Az ellenőrzés három külön dolgot vizsgál. (1) *Másolat*: a `[..., ::-1]` indexelés nézetet ad, a `.copy()` új puffert, tehát `np.shares_memory(x, y)` hamis. (2) *dtype*: az indexcsere nem változtat rajta, a kimenet float32 marad. (3) *Kontiguitás*: a nézet negatív stride-ú, nem C-kontiguus; a `.copy()` C-rendű, kontiguus tömböt ad (`y.flags['C_CONTIGUOUS']` igaz), amit a későbbi `np.ascontiguousarray` már nem másol újra. A kimenet float32, shape $2 \times 1 \times 3 \times 4$. A `torch.flip` ugyanígy új puffert ad, a dtype és az alak változatlan; a kontiguitás-vizsgálat a NumPy-változatra vonatkozik. A tolerancia itt nincs, mert az indexcsere egzakt.

22.2.7.4 F4 megoldás

A hiba a kép és a maszk **különböző** szögű elforgatása. A `mask`-ra `angle + 5` jut, tehát a pixelannotáció öt fokkal más pozícióba kerül. Ettől a maszk már nem a valódi előtér: a modell hibás célt tanul, mert a kép előterét a maszk elcsúszott helyén, a háttér egy részét pedig előtérként látja. A veszteség csökkenése ezen a hibás célon nem javítja a szegmentációt. A javítás: a két forgatás egy közös szöggel, azonos interpolációs konvencióval futtasson; a maszkra a *Mask és box szinkronizáció* bekezdés szerint legközelebbi szomszéd (*nearest*) interpoláció való, a képhez bilineáris is lehet. A visszaellenőrzéshez ki lehet számolni a transzformált maszk és a transzformált kép előterének átlapolását, vagy egyszerűen egy mini batchre a feladat dobozát is szinkronban transzformálni és konzisztenciáját ellenőrizni.

22.2.7.5 F5 megoldás

Kérdés: a horizontal flip a GlyphLab diagonal osztályon címkét változtat-e?

Megfigyelési egység: `scene_id`, mert a két nézet ugyanazon jelenetből származik; a kísérlet a train split 40 és a val split 10 diagonal jelenetét vizsgálja (jelenetenként két nézet, azaz 80, illetve 20 kép).

Baseline: a nem augmentált train spliten tanított kis ConvNet, a validációra mérve, a 16.1. Eltolási equivariance és invariance és 12.4. III. ellenőrző pont: önállóan tanuló kis háló eljárásához hasonlóan. **Változó tényező:** a flip mint transzformáció engedélyezése a betöltő pipeline-ban.

Metrika: az elsődleges mennyiség a **címkeváltozási arány** a diagonal osztályon: a transzformált képek közül mekkora hányadra teljesül $f(\mathcal{T}(X)) \neq 2$. Ezt a címkézőfüggvény szintjén, a generátor geometriai definíciójából kell meghatározni, nem a modell jóslatából; a modell predikciója csak másodlagos diagnosztika lehet. A másodlagos metrika a per-image osztályozási pontosság a transzformált val képeken, amely azt mutatja, hogyan viselkedik a tanított modell, de a címke kérdését nem dönti el. A cél olyan transzformáció, amely a diagonal osztály címkéjét megtartja.

Protokoll: a train és val splitet a `scene_id` alapján rögzítjük. A diagonal osztályra szűrt halmazban a flip előtti és utáni célok változását a címkézőfüggvény szintjén határozzuk meg: a generátor a ferde vonalat a bal felső sarokból a jobb alsó felé haladó átló mentén rajzolja (`generate_data.py`: a sor- és az oszlopindex együtt nő), a horizontal flip ezt a bal alsó–jobb felső irányba viszi. Mivel a 2. osztály csak az egyik ferde irányt tartalmazza (ADATOK.md), a flip a diagonal képek címkéjét megváltoztatja. Ezután a megengedett transzformációt az osztályokra aszimmetrikusan definiáljuk: vertical és horizontal osztályon a horizontal flip megengedhető, diagonalon nem.

Következtetési határ: a konklúzió a GlyphLab geometriai definíciójára vonatkozik, nem általános diagonal feladatokra. A generátor seedjével, verziójával és splitjével együtt megismételhető. A val split mindössze 10 diagonal jelenete miatt a pontbecslés bizonytalan; a konfidenciaintervallumot jelenetszintű bootstrapból kell számítani, nem képenkénti független mintából. A pontos kísérlethez a train/val split arányok az ADATOK.md-ben szerepelnek. A 90 fokos forgatásra az Állítás ad bizonyítást; a flipre ugyanilyen, a célfüggvény definíciójából következő érv érvényes: a horizontal flip $(i, j) \mapsto (i, W - 1 - j)$ leképezése a bal felső–jobb alsó átlót a bal alsó–jobb felső átlóba viszi, ez pedig nem a 2. osztály egyetlen ferde iránya, ezért $f(\mathcal{T}(X_{\text{diagonal}})) \neq 2$. A kísérlet célja az lenne, hogy a pipeline-léptékű ellenőrzés mutassa a hatást, nem pedig a modell pontossága.

A megoldás rubrikája: korrekt scene-stratifikáció, a flip és a rotáció címkeváltozásának elkülönítése, az érvényességi határ és annak rögzítése, hogy az eredmény a cél definíciójából következik.

Hipotézis: a horizontal flip a diagonal osztály címkéjét megváltoztatja. **Recept, döntési szabály és erőforrás:** a másodlagos diagnosztikához használt kis ConvNetet Adam optimalizálóval, `lr=10^{-3}`, 20 epoch és 32-es batch mellett tanítjuk, öt seeddel (20261020–20261024); a flip-kezelést csak a train betöltőben engedjük. Az elsődleges döntési szabály számszerű: a flip akkor számít címkeváltoztatónak a diagonal osztályon, ha a címkeváltozási arány 95%-os, jelenetszintű bootstrap-intervallumának alsó határa nagyobb nullánál; a másodlagos pontosságkülönbséget csak leíró eredményként közöljük, és akkor tekintjük érdeminek, ha a páros CI kizárja a 0-t és a különbség legalább 2 százalékpont. Erőforrás: CPU, `float32`, öt futás, néhány perc; a címkeváltozás ellenőrzése maga nem igényel tanítást. ### Tipikus tévedés, önellenőrzés és visszautalás

Tipikus tévedés. Az a feltevés, hogy az augmentáció a vizuális hasonlóságot őrzi, ezért a címkét is. Az ellenpélda a 90 fokos rotáció: vizuálisan ugyanaz a vonal látható, de a feladat iránya megváltozott. Másik gyakori hiba: a flipet minden osztályra megengedni, mert a „vízszintes tükrözés nem változtat az ábrán". A GlyphLab diagonal osztálya irányelvű, így a flip egy másik irányt ad.

Önellenőrzés. Válasszon egy képet a trainből, és forgassa 90 fokkal. Ha a transzformált képet az eredeti címkével adná be, a modell a képet az ellenkező irányba tartozónak tanulná. Ezt önellenőrzéshez a forgatási állítás és az F2 dobozpélda segítségével ellenőrizheti: a kép, a maszk és a doboz ugyanazt a geometriai transzformációt kapja.

Visszautalás. A 10.2. Affín réteg és aktiváció backward batch-reduction és broadcast-szabálya itt is él: ha az augmentációt a minibatch betöltésekor alkalmazzuk, a képet és a célokat együtt kell transzformálni, és a batch-művelet nem tompíthatja el a hibás párokat. A 18.2. Batch normalization train és eval módban train/eval különbség ahhoz hasonló, ahogy az augmentációt a gyakorlatban csak a train pipeline-ban érdemes futtatni, az eval pipeline-ban nem, vagy csak determinisztikusan. A 21.2. Learning rate és ütemezés üteme pedig arra figyelmeztet, hogy a training adat augmentációjának mennyisége és a learning rate schedule együtt befolyásolja a tanítási pályát.

22.2.8 Rövid összefoglaló

A címkemegőrző transzformáció fogalma a feladat célfüggvényéhez, nem a vizuális hasonlósághoz kötött. A crop, resize és intenzitásváltoztatás akkor megengedett, ha nem semmisíti meg a cél szempontjából szükséges tartalmat; a flip csak a feladat szimmetriái szerint címkemegőrző; a 90 fokos forgatás a GlyphLab irányfeladatán a vertical és horizontal címkét felcseréli, a diagonal pedig más irányt jelöl. A kép, maszk és doboz minden geometriai transzformációjának ugyanazt az indexleképzést kell követnie; független véletlen transzformáció ellentmondásos tanítópárokat

ad. Az augmentációt a gyakorlatban csak a train pipeline-ban alkalmazzuk; a tesztfolyamat valós eloszlását nem szabad augmentálni. A nyitó kérdésre a válasz: a 0. osztályú kép 90 fokos elforgatása után a címke nem maradhat 0, mert a vertical és horizontal osztály felcserélődik; ez a transzformáció nem címkemegőrző.

22.2.9 Következő lecke

A 22.3. Zajos és hiányos címke a zajos és hiányos címke kezelésére tér át. Az ott tárgyalt címkebizonytalanság éppen abból a pontból ered, ahol a transzformáció már nem tudja garantálni a cél hűségét; az itt tanult szinkron és címkemegőrző eljárás a zajos címke kontrolljához is nélkülözhetetlen.

22.2.10 Felhasznált források

- [D08] PyTorch contributors (2026). *torch.utils.data*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.
- PyTorch contributors (2026). *torchvision.transforms*. TorchVision dokumentáció (stable). https://docs.pytorch.org/vision/stable/transforms.html. Hozzáférés: 2026-10-30.
- GlyphLab saját laboradatok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`, a könyv ADATOK.md szerződése szerint.

22.3 Zajos és hiányos címke

22.3.1 Cél, előismeret és induló kérdés

A 22.2. Címkemegőrző transzformációk egység azt vizsgálta, mely transzformáció őrzi meg a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab orientációs címkéjét. Most ugyanannak a tanulási szerződésnek a másik oldalát vizsgáljuk: mi történik, ha nem a transzformáció, hanem maga a *címke* hibás. A címkezaj nem javítható egyszerűen nagyobb kapacitású modellel, és nagyon különbözik a bemeneti képen megjelenő zajtól. A 21.3. Dropout, súlyregularizálás és augmentáció regularizáló eszközei, a 18.3. Pre-activation, kis batch és alternatív normalizálás normalizáló rétegei és a 10.3. Gradiensellenőrzés és numerikus hibamérce numerikus ellenőrzési protokollja mind feltételezték, hogy a tanulási cél y_i érvényes; ezt a feltételt most fellazítjuk. Az induló kérdés: ha egy tanító képsorozat címkéinek egy része véletlenül másik osztályra mutat, mennyire torzul a clean validációra mért osztályozó, és mit tehetünk a kísérleti jegyzőkönyvben?

Előismeret: a többosztályos logit/cross-entropy szerződés (7.2. Softmax és többosztályos cross-entropy), a scene-szintű split és a train/val/test elkülönítés (22.1. Adateredet, címke és csoportos split), valamint a feltételes valószínűség és a teljes valószínűség tétele (5.1. Esemény, feltétel és véletlen változó). A labor továbbra is a helyi GlyphLab-adatokat használja, előtanított súly és GPU nélkül.

22.3.2 Definíciók és intuíció

Legyen $\mathcal{X}$ a képtér, $\mathcal{Y} = \{0, 1, \dots, K-1\}$ az osztályindexek halmaza. A *tiszta* megfigyelés az (X, Y) pár; a tanulási mintában azonban gyakran a zajos $\tilde{Y}$ érték áll rendelkezésre. A képek továbbra is $X \in \mathbb{R}^{N\times 1\times 24\times 24}$ alakúak, a célok $y \in \mathcal{Y}^N$, a zajos cél $\tilde{y} \in \mathcal{Y}^N$.

Definíció (címkezaj, label noise). A mintában szereplő $\tilde{Y}$ címke *címkezajos*, ha a hiba a céloldalon következik be: $P(\tilde{Y} \neq Y) > 0$, miközben X érintetlen. A címkezaj lehet véletlen, de lehet determinisztikus, szisztematikus félrecímkézés is ($\tilde{Y} = g(X, Y)$ egy rögzített, hibás leképezéssel), ezért nem zárjuk ki, hogy $\tilde{Y}$ determinisztikus függvénye legyen az (X, Y) párnak. Az osztályfüggő zajmodell (*class-conditional noise*) azt a további *modellfeltevést* adja, hogy $\tilde{Y}$ feltételesen független X-től a Y értékére szorítkozva: $\tilde{Y} \perp\!\!\!\perp X \mid Y$; ez egy zajmodell, nem a címkezaj általános definíciója. Ez az osztályozási címkék tipikus elírás-, félreolvasás- vagy gépi átírási modellje.

Definíció (bemeneti zaj, input/covariate noise). A mintában a bemenet (a kovariáta) torzul: $\tilde{X}$ figyelhető meg, a cél Y változatlan. A feltételes függetlenségi feltevés $Y \perp\!\!\!\perp \tilde{X} \mid X$. Példa: a GlyphLab két nézete ugyanahhoz a scene-hez és címkéhez tartozik, de a két kép véletlen szenzor- vagy megvilágítási zajjal tér el; ez nem címkezaj, hanem bemeneti zaj. A kettő következménye alapvetően eltér: a kovariáns zaj a döntési felületet a *bemeneti* térben mossa el, a címkezaj a *cél* kódolását.

Gyenge felügyelet (weak supervision). Olyan tanulási beállítás, ahol a címkék nem egyenként, emberi szakértő által ellenőrzöttek, hanem szabályokból, heurisztikákból, távoli forrásból vagy programozott annotációból származnak. A gyenge címke felfogható szisztematikus, nem feltétlenül sztochasztikus címkezajként; hibatulajdonságai nem cserélhetők fel a szimmetrikus véletlen zajéval.

Annotátorok közötti egyetértés (annotation agreement). Két vagy több annotátor döntésének egyezési mértéke; bináris esetben például Cohen-féle kappa, több annotátorra Fleiss-féle kappa. Az egyetértés a címkeminőség *folyamatmutatója*, nem a tiszta címke bizonyítéka: két annotátor szisztematikusan ugyanazt a hibát is elkövetheti. A review lépés a vitás vagy alacsony megbízhatóságú rekordok célzott átvizsgálása és javítása *a modellválasztás előtt*. A review során tilos a val/test címkék megváltoztatása a modell kimenetének ismeretében, mert az adaptív célmanipuláció.

Leakage (szivárgás). Bármely olyan út, amely a train folyamatot a val/test cél vagy a val/test megfigyelési egység azonosítójának tiltott ismeretéhez juttatja. A címkezajos kísérletben leakage, ha a zajszimulációhoz a val/test címkéket használjuk fel, vagy ha a zajos tanítóhalmazt a validáció hibája alapján utólag újracímkézzük. A split határai a zajmodell bevezetése után is érvényesek.

22.3.3 A szimulált címkezaj modellje és a kovariáns zaj különbsége

A laborban a zajt szabályozottan, szimulációval visszük be. A modellt formálisan definiáljuk, majd egy lemmában kiszámítjuk az osztályprior torzulását.

Definíció (szimulált címkezaj-modell). Legyen adott a $Q \in [0,1]^{K\times K}$ *zajátmenet-mátrix*, ahol

$$Q_{kj} = P(\tilde{Y} = j \mid Y = k), \qquad \sum_{j=0}^{K-1} Q_{kj} = 1 \quad \forall k.$$

A zajos példa generálása:

$$X_i \sim P_X, \qquad Y_i \sim P_{Y|X}(\cdot \mid X_i), \qquad \tilde{Y}_i \sim \mathrm{Cat}(Q_{Y_i,0}, \dots, Q_{Y_i,K-1}),$$

és feltesszük, hogy a különböző minták a tiszta címkékre feltételesen függetlenek, valamint hogy $\tilde{Y}_i \perp\!\!\!\perp X_i \mid Y_i$ (osztályfüggő zaj). Az $(X_i, \tilde{Y}_i)$ párok alkotják a tanító mintát; a tiszta Y_i legfeljebb az értékelésben jelenik meg, a tanításkor nem.

A leggyakoribb szimuláció a *szimmetrikus zaj*: adott $\epsilon \in [0,1]$ teljes flip-ráta mellett minden osztály azonos eséllyel hibásodik, és a hibás címke egyenletesen választódik a többi $K-1$ osztály közül:

$$Q_{kj} = \begin{cases} 1-\epsilon, & k = j, \\ \frac{\epsilon}{K-1}, & k \neq j. \end{cases}$$

Lemma (a megfigyelt osztályprior torzulása). Legyen a tiszta prior $\pi_j = P(Y=j)$, és a zaj a fenti feltételes függetlenséget teljesítő átmenetmátrixszal adott. Ekkor a megfigyelt osztálygyakoriság eloszlása

$$\tilde{\pi}_j = P(\tilde{Y}=j) = \sum_{k=0}^{K-1} \pi_k \, Q_{kj}.$$

Bizonyítás. A teljes valószínűség tételét alkalmazzuk a Y értékeire:

$$P(\tilde{Y}=j) = \sum_{k=0}^{K-1} P(\tilde{Y}=j,\ Y=k) = \sum_{k=0}^{K-1} P(Y=k)\, P(\tilde{Y}=j \mid Y=k) = \sum_{k=0}^{K-1} \pi_k Q_{kj}.$$

Az első egyenlőség a partíció, a második a feltételes valószínűség szorzatszabálya, a harmadik a definíció behelyettesítése. $\square$

Következmény (szimmetrikus zaj). A szimmetrikus zajra

$$\tilde{\pi}_j = (1-\epsilon)\pi_j + \frac{\epsilon}{K-1}\sum_{k \neq j} \pi_k = (1-\epsilon)\pi_j + \frac{\epsilon}{K-1}(1-\pi_j).$$

Ebből $\tilde{\pi}_j - \pi_j = \frac{\epsilon}{K-1}(1 - K\pi_j)$. Egyenletes priorra ($\pi_j = 1/K$ minden j-re) a szimmetrikus zaj nem torzítja a marginális osztálygyakoriságot; más priorra azonban a ritka osztályok felé húz.

Ez a lemma világosan mutatja a *címkezaj* és a *kovariáns zaj* formális különbségét. A kovariáns zaj generatív modellje:

$$X_i \sim P_X, \qquad Y_i \sim P_{Y|X}(\cdot \mid X_i), \qquad \tilde{X}_i \sim q(\cdot \mid X_i), \qquad Y_i \perp\!\!\!\perp \tilde{X}_i \mid X_i.$$

Itt a cél Y_i a torzított bemenet ellenére is az eredeti posteriorból származik; a zaj a $X \to \tilde{X}$ élen hat. A címkezajnál a zaj a $Y \to \tilde{Y}$ élen hat, és X érintetlen. A két generatív gráf tehát különböző faktorizációt ad: $P(X,Y,\tilde{Y}) = P(X,Y)P(\tilde{Y} \mid Y)$, illetve $P(X,Y,\tilde{X}) = P(X,Y)q(\tilde{X} \mid X)$. A fontos következmény: a címkezajos tanítás a zajos $\tilde{Y}$ posteriorját célozza, ezért a tanult valószínűségek a tiszta posteriorhoz képest torzulnak, és aszimmetrikus vagy erős zajnál a döntési határ is eltolódik; ezt a torzulást a mintaméret növelése önmagában nem szünteti meg, mert a cél maga torzult. Szimmetrikus zajnál viszont a zajos posterior a tiszta posterior szigorúan növekvő affin függvénye lehet, és ilyenkor az $\arg\max$ — a döntés — megmarad (lásd a 8. szakaszt). A kovariáns zaj esetén a tiszta posteriorból származó döntési szabály a torzított $\tilde{X}$ bemenetre is kiszámítható, de általában nem optimalizálja a $P(Y \mid \tilde{X})$ posterior szerinti hibát.

22.3.4 Kézi kis példa és minimális kód

Legyen $K = 3$, prior $\pi = (0{,}5; 0{,}3; 0{,}2)$, szimmetrikus zaj $\epsilon = 0{,}2$. A zajátmenet-mátrix:

$$Q = \begin{pmatrix} 0{,}8 & 0{,}1 & 0{,}1 \\ 0{,}1 & 0{,}8 & 0{,}1 \\ 0{,}1 & 0{,}1 & 0{,}8 \end{pmatrix}.$$

A lemma alapján:

$$\tilde{\pi}_0 = 0{,}5 \cdot 0{,}8 + 0{,}3 \cdot 0{,}1 + 0{,}2 \cdot 0{,}1 = 0{,}45,$$

$$\tilde{\pi}_1 = 0{,}5 \cdot 0{,}1 + 0{,}3 \cdot 0{,}8 + 0{,}2 \cdot 0{,}1 = 0{,}31,$$

$$\tilde{\pi}_2 = 0{,}5 \cdot 0{,}1 + 0{,}3 \cdot 0{,}1 + 0{,}2 \cdot 0{,}8 = 0{,}24.$$

A három érték összege 1. A ritka harmadik osztály gyakorisága nőtt ($0{,}2 \to 0{,}24$), a gyakori első osztályé csökkent ($0{,}5 \to 0{,}45$).

Ellenőrzésül a zajgenerálás minimális kódja:

```
import numpy as np

def symmetric_label_noise(labels, K, eps, rng):
    """labels: (N,) int; minden címkét eps eséllyel egyenletesen
    kicserél a többi K-1 osztály valamelyikére."""
    noisy = labels.copy()
    flips = rng.random(len(labels)) < eps
    for i in np.where(flips)[0]:
        others = [j for j in range(K) if j != labels[i]]
        noisy[i] = rng.choice(others)
    return noisy

rng = np.random.default_rng(7)
y = np.array([0, 1, 2, 0, 1, 2, 0, 1, 2, 0])
y_t = symmetric_label_noise(y, 3, 0.2, rng)
print("clean:", y)
print("noisy:", y_t)
print("flip rate:", np.mean(y != y_t))
```

Rögzített RNG, könyvtárverzió és hívássorrend mellett a kimenet reprodukálható; a seed megváltoztatásával, külön futásokban változik. Egy 10 mintás realizált flipráta nem várható szorosan ϵ-hoz (a binomiális szórás ekkor kb. $\sqrt{\epsilon(1-\epsilon)/10}$), a lényegi ellenőrzés az, hogy a tesztpéldában a várható flip-ráta ϵ körül ingadozik, és a zajos címkék mindig $0, \ldots, K-1$ közé esnek. A kód nem a tanítás, hanem a zajmodell elemi referenciaimplementációja.

22.3.5 Labor: szabályozott label noise a GlyphLab train részén

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5, torch 2.14.1 (CPU) és pandas 2.3.3. Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. A futtatás a szakaszban megadott `n087_label_noise_lab.py` szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

Kérdés. Hogyan változik a clean validációra mért loss és accuracy, ha a train címkék ϵ hányadát szimmetrikusan kicseréljük?

Protokoll. Csak a `data/metadata.csv` szerinti train split indexeire alkalmazunk zajt. A valid és test tiszta marad. Minden $\epsilon \in \{0{,}0; 0{,}1; 0{,}2; 0{,}4\}$ esetén új tanítást indítunk, rögzített seeddel. A modell kisméretű CNN, 3 kimeneti logittal; tanítás Adam-optimizálóval, cross-entropy loss-szal, 20 epochig. A kiválasztáshoz nem használjuk a testet; a clean validációs accuracy-t és loss-t jegyezzük fel.

Az alábbi kód a `lab/n087_label_noise_lab.py` munkakönyvtárban futtatható:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

SEED = 20261020
DEVICE = torch.device("cpu")

images = np.load("data/images.npy")      # (360,1,24,24) float32
labels = np.load("data/labels.npy")      # (360,) int64
meta = pd.read_csv("data/metadata.csv")

# A meta sorrendjére nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
if "index" not in meta.columns:
    raise ValueError("a metadata.csv-ből hiányzik az index oszlop")
if not (len(meta) == len(images) == len(labels)):
    raise ValueError("a metadata.csv és a tömbök hossza eltér")
if not np.array_equal(np.sort(meta["index"].to_numpy()), np.arange(len(images))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
train_idx = meta.loc[meta["split"] == "train", "index"].to_numpy()
val_idx   = meta.loc[meta["split"] == "val", "index"].to_numpy()
assert (len(train_idx), len(val_idx)) == (240, 60)

K = 3

def symmetric_label_noise(y, K, eps, rng):
    noisy = y.copy()
    flips = rng.random(len(y)) < eps
    for i in np.where(flips)[0]:
        others = np.setdiff1d(np.arange(K), y[i])
        noisy[i] = rng.choice(others)
    return noisy

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(16*6*6, 32), nn.ReLU(),
            nn.Linear(32, K)
        )
    def forward(self, x):
        return self.net(x)

def run(eps):
    # Minden futás azonos inicializálást és shuffle-sorrendet kap.
    torch.manual_seed(SEED)
    # Azonos seeddel a flip-maszkok egymásba ágyazottak: eps=0,1 flipjei
    # részhalmazai az eps=0,2 flipjeinek, és a futás önmagában is reprodukálható.
    noise_rng = np.random.default_rng(SEED)
    y_train = symmetric_label_noise(labels[train_idx], K, eps, noise_rng)
    if eps == 0.0:
        assert np.array_equal(y_train, labels[train_idx])
    else:
        assert not np.array_equal(y_train, labels[train_idx])
    Xtr = torch.tensor(images[train_idx], dtype=torch.float32)
    ytr = torch.tensor(y_train, dtype=torch.long)
    Xva = torch.tensor(images[val_idx], dtype=torch.float32)
    yva = torch.tensor(labels[val_idx], dtype=torch.long)
    assert np.array_equal(yva.numpy(), labels[val_idx])   # a validáció tiszta marad
    model = SmallCNN().to(DEVICE)
```

```
    opt = optim.Adam(model.parameters(), lr=1e-3)
    crit = nn.CrossEntropyLoss()
    ds = torch.utils.data.TensorDataset(Xtr, ytr)
    dl = torch.utils.data.DataLoader(ds, batch_size=32, shuffle=True)
    for _ in range(20):
        model.train()
        for xb, yb in dl:
            opt.zero_grad(); crit(model(xb), yb).backward(); opt.step()
    model.eval()
    with torch.no_grad():
        logits = model(Xva)
        val_loss = crit(logits, yva).item()
        val_acc = (logits.argmax(1) == yva).float().mean().item()
    return val_loss, val_acc

print("eps   val_loss_clean  val_acc_clean")
for eps in [0.0, 0.1, 0.2, 0.4]:
    loss, acc = run(eps)
    print(f"{eps:3.1f}  {loss:9.4f}       {acc:6.4f}")
```

Futtatási parancs a lab könyvtárból: `python n087_label_noise_lab.py`. A `SmallCNN` a kódblokkban definiált kisméretű osztályozó, a korábbi ellenőrzőpontok modelljének mintájára; a `torch.utils.data.TensorDataset` használata a [D08] dokumentáció szerinti, a futtatás környezeti PyTorch-verzióját a jegyzőkönyvben ellenőrizni és rögzíteni kell. A seed a `run()` elején minden futásra újra beállítva azonos PyTorch-verzió, backend és szálszám mellett adja ugyanazt az inicializálást és shuffle-sorrendet; más verzió, hardver vagy szálszám bitazonosságot nem garantál. A metadata `index` oszlopát a kód ellenőrzi, nem feltételezi a sorok egyezését. A dtype (`float32`/`int64`) és a device rögzített. A futtatott kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`, seed 20261020):

```
eps   val_loss_clean  val_acc_clean
0.0     0.7838       0.7667
0.1     1.0292       0.6333
0.2     0.8784       0.6000
0.4     1.2693       0.3833
```

A kimenet egyetlen seed mért értéke. A tényleges futásnál a pontos értéket seed-szórás is terheli; a clean validáció értelmezése feltételes, ismételt mintavételi konfidenciaintervallumot igényelne, a [S04] a bootstrap-intervallum ismételt mintavételi értelmezését tárgyalja. Értelmezés: növekvő ϵ mellett a clean valid loss jellemzően nő, az accuracy csökken; a mért sorozatban a loss nem monoton (a 0,2-es pont alacsonyabb a 0,1-esnél), ami egyetlen seednél a véletlen ingadozás jele, ezért legalább három seed és konfidenciaintervallum szükséges. A kis ϵ tartományban a zaj regularizáló hatása is felmerülhet, de ezt ez a kísérlet nem teszteli, és a jelenség nem teszi a zajt tervezett eszközzé.

22.3.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 fogalmi, nehézség 1. Hasonlítsa össze a címkezajt, a kovariáns zajt és a gyenge felügyeletet a generatív modell és a hibatípus szerint. Adjon olyan GlyphLab-példát, ahol a két nézet

különbsége kovariáns zaj, és olyan módosítást, amely címkezajt visz be anélkül, hogy a képet megváltoztatná. Elvárt eredmény: két formális szorzatfaktorizáció és egy-egy saját példa.

F2 levezetési, nehézség 2. Legyen $K = 3$, prior $\pi = (0{,}5; 0{,}3; 0{,}2)$, szimmetrikus zaj $\epsilon = 0{,}2$. Számítsa ki a $P(Y = 0 \mid \tilde{Y} = 0)$ és $P(Y = 2 \mid \tilde{Y} = 2)$ feltételes valószínűségeket. Ellenőrizze, hogy az átmenetmátrix invertálható-e, és ebből visszanyerhető-e a tiszta prior a megfigyelt priorból. Elvárt eredmény: teljes levezetés, numerikus értékek, invertálhatósági indoklás, és annak megfogalmazása, miért nem állítja ez helyre a tiszta posteriort.

F3 implementációs, nehézség 2. Írjon függvényt `asymmetric_label_noise(labels, flip_pairs, probs, rng)` alakban, ahol `flip_pairs` explicit $(k \to j)$ párok listája, `probs` a megfelelő flip-valószínűség. A függvény adja vissza a zajos címkéket és a tényleges flip-maszkot. Tesztelje 100 000 szintetikus címkén, rögzített seedű `Generator`-ral, hogy a flip-ráta a megadott valószínűségek 0,01 abszolút tűrésén belül van. Indokolja meg a minta elemszámát: $n = 1000$ mellett a flip-ráta szórása $p = 0{,}25$-nél $\sqrt{0{,}25 \cdot 0{,}75/1000} \approx 0{,}014$, ezért a 0,02-es tűrés seed nélkül instabil tesztet adna; 100 000 mintán a szórás 0,0014, és a 0,01-es tűrés biztonságos. Elvárt eredmény: teljes kód, teszt és tolerancia.

F4 hibakeresési, nehézség 2. Az alábbi kódrészlet a validációs címkéket is zajosítja, a legjobb epochot a zajos validációs accuracy alapján választja ki, majd a tiszta validációs címkéken számolt accuracy-t jelenti meg. Azonosítsa a hibát, annak következményét a modellválasztásra, és adja meg a javított eljárást úgy, hogy a train zaj ráta 0,2, de a valid tiszta marad. Mutassa meg, milyen ellenőrzéssel bizonyítható, hogy a javítás valóban tiszta validációt mér. Elvárt eredmény: ok, következmény, javított kódrészlet, visszaellenőrzés.

```
import copy
import numpy as np

# részlet egy tanítási ciklusból; a train_one_epoch és az accuracy
# itt feltételezett segédfüggvény, a többi név a labor környezetéből adott
noise_rng = np.random.default_rng(SEED)
y_train = symmetric_label_noise(labels[train_idx], K, 0.2, noise_rng)
y_val = symmetric_label_noise(labels[val_idx], K, 0.2, noise_rng)  # hiba
best_acc, best_state = -1.0, None
for epoch in range(20):
    train_one_epoch(model, Xtr, y_train)
    acc = accuracy(model, Xva, y_val)        # zajos validáció
    if acc > best_acc:
        best_acc, best_state = acc, copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
print("clean accuracy:", accuracy(model, Xva, labels[val_idx]))
```

A részlet önálló vázlat, nem a labor kódjának másolata: a laborban nincs legjobb-epoch kiválasztás, ezt a hiba szemléltetésére itt vezetjük be.

F5 kísérleti/transzfer, nehézség 3. Tervezzen gyenge felügyeletű változatot a GlyphLab feladatra: az orientációt ne emberi címke, hanem képnyomaték-alapú heurisztika adja meg. Fogalmazzon meg protokollt, amely a clean train címkével tanított baseline-t hasonlítja a gyenge címkével tanított modellhez, scene-szintű bootstrap-tel az accuracy-különbségre. Milyen érvényességi korlát következik abból, hogy a gyenge címkék hibája várhatóan nem szimmetrikus? Elvárt eredmény: hipotézis, megfigyelési egység, baseline, metrika, érvényességi korlát.

22.3.7 Megoldások és indoklás

F1 megoldás. A címkezaj generatív modellje $P(X, Y, \tilde{Y}) = P(X, Y)P(\tilde{Y} \mid Y)$, ahol a zaj a $Y \to \tilde{Y}$ élen hat, és X érintetlen. A kovariáns zaj modellje $P(X, Y, \tilde{X}) = P(X, Y)q(\tilde{X} \mid X)$, ahol a

zaj az $X \to \tilde{X}$ élen hat, és Y érintetlen. A gyenge felügyelet determinisztikus vagy sztochasztikus címkeképző szabályokból származó cél; ez nem szükségképpen címkezaj, mert a szabály lehet helyes, és a címke csak emberi címkézés helyett származtatott. Annyiban rokon a címkezajjal, hogy a hiba itt is a céloldalon, a $Y \to \tilde{Y}$ élen jelenik meg, és gyakran szisztematikus, ezért a szimmetrikus véletlen zaj modellje nem írja le; a bemeneti torzulástól (kovariáns zaj) viszont élesen különbözik. GlyphLab-példa: a két nézet közötti pixelzaj kovariáns; ha a `data/labels.npy` egy részét a szomszédos orientációra írjuk át a képek módosítása nélkül, a címkezaj.

F2 megoldás. A 4. szakasz Q mátrixával $\tilde{\pi}_0 = 0{,}45$, $\tilde{\pi}_2 = 0{,}24$. A Bayes-szabály szerint

$$P(Y=0 \mid \tilde{Y}=0) = \frac{P(Y=0)Q_{00}}{\tilde{\pi}_0} = \frac{0{,}5 \cdot 0{,}8}{0{,}45} = 0{,}8889,$$

$$P(Y=2 \mid \tilde{Y}=2) = \frac{0{,}2 \cdot 0{,}8}{0{,}24} = 0{,}6667.$$

A Q mátrix szimmetrikus és sajátértékei 1 és $1 - \frac{K}{K-1}\epsilon$ kétszeres multiplicitással, azaz $1; 0{,}7; 0{,}7$, tehát invertálható. A marginális eloszlásra $\tilde{\pi}^\top = \pi^\top Q$, azaz $\tilde{\pi} = Q^\top \pi$; általános Q esetén tehát $\pi = (Q^\top)^{-1}\tilde{\pi}$. Itt Q szimmetrikus, ezért ez $\pi = Q^{-1}\tilde{\pi}$-re egyszerűsödik, és visszaadja a tiszta priort. Ez azonban nem adja vissza a tiszta $P(Y \mid X)$ posteriort, mert a modell a zajos $\tilde{Y}$ feltételes eloszlását tanulja, és a Q^{-1} csak a marginális priorra alkalmazható; a posterior torzulását a $P(\tilde{Y} \mid X)$ és Q együttes inverziója kezelné, amihez Q ismert vagy becsült volta szükséges.

F3 megoldás.

```
import numpy as np

def asymmetric_label_noise(labels, flip_pairs, probs, rng):
    """labels: (N,) int; flip_pairs: list of (from, to);
    probs: azonos hosszú flip-valószínűségek.

    A forrásosztályok egyediek: ugyanarra a forrásra adott két pár
    esetén a második felülírná az első döntését.
    """
    sources = [src for src, _ in flip_pairs]
    assert len(flip_pairs) == len(probs)
    assert len(sources) == len(set(sources)), "a források legyenek egyediek"
    noisy = labels.copy()
    mask = np.zeros(len(labels), dtype=bool)
    for (src, dst), p in zip(flip_pairs, probs):
        sel = (labels == src) & (rng.random(len(labels)) < p)
        noisy[sel] = dst
        mask[sel] = True
    return noisy, mask

rng = np.random.default_rng(0)
y = rng.integers(0, 4, size=100000)
flip_pairs = [(0, 1), (2, 3)]
probs = [0.15, 0.25]
ny, mask = asymmetric_label_noise(y, flip_pairs, probs, rng)
# ellenőrzés
assert np.array_equal(mask, y != ny)
for (src, dst), p in zip(flip_pairs, probs):
    rate = np.mean(mask[y == src])
    assert abs(rate - p) < 0.01, (rate, p)
print("flip rates ok:", np.mean(mask))
```

A teszt a megadott tűrésen belül ellenőrzi a valószínűséget; a `mask` pontosan a megváltozott elemeket jelöli, így a hibaarány mindig jól definiált. A függvény a forrásosztályok egyediségét assertionnal kéri számon: ha ugyanaz a forrás két párban is szerepel, a második pár a saját

mintáján felülírja az első döntését, ezért a megvalósult peremvalószínűség nem a megadott `probs` lenne. Több célosztály felé flipelő forrást ezért explicit átmenetmátrixszal kell megadni, nem szekvenciális párokkal. A minta elemszáma és a seed rögzített, így a teszt reprodukálható; a 0,01-es tűrés mellett a futás nem függ a szerencsétől.

F4 megoldás. Hiba: a validációs címkéket is zajosítják (`symmetric_label_noise(labels[val_idx], ...)`), ezért a kiválasztás nem a tiszta, hanem egy torzított validációs célra optimalizál; a legjobb epoch kiválasztása a zajos validáció zajmintázatához igazodik, és a jelentett „clean accuracy" valójában zajos referencia. Következmény: a modellválasztás optimistán torzított, és a záróteszt nem értelmezhető független mérésként. Javítás: a zajt csak a `train_idx` indexekre alkalmazzuk, a validációhoz mindig `labels[val_idx]`-t használjuk:

```
y_train = symmetric_label_noise(labels[train_idx], K, 0.2, noise_rng)
y_val = labels[val_idx]                       # a validáció tiszta marad
assert np.array_equal(y_val, labels[val_idx])
best_acc, best_state = -1.0, None
for epoch in range(20):
    train_one_epoch(model, Xtr, y_train)
    acc = accuracy(model, Xva, y_val)         # tiszta validáció
    if acc > best_acc:
        best_acc, best_state = acc, copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
print("clean accuracy:", accuracy(model, Xva, y_val))
```

A labor kódja nem használ teszt splitet és nem tartalmaz legjobb-epoch kiválasztást; a fenti részlet ezeket csak a hiba szemléltetésére vezeti be. Visszaellenőrzés: (i) `np.array_equal(y_val, labels[val_idx])` igaz; (ii) a validációs loss és accuracy kizárólag a tiszta `y_val`-ból számolódik; (iii) $\epsilon = 0$ mellett a zajfüggvény bitre azonos címketömböt ad (`np.array_equal(y_train, labels[train_idx])`), ezért a tanítási oldalon sem keletkezik zaj. Az azonos seed melletti loss-egyezés a numerikus nemdeterminizmus miatt csak közelítő, ezért bitre azonos egyenlőséget nem írunk elő.

F5 megoldás. Hipotézis: a gyenge, képnyomaték-alapú címkék rontják a clean validációs accuracy-t a clean annotációhoz képest, de a romlás mértéke függ attól, hogy a heurisztika mely osztályokat keveri. Megfigyelési egység: scene_id, mert a két nézet ugyanahhoz a jelenethez tartozik. Split: a meglévő train/val/test scene-szintű felosztás változatlan. Baseline: a clean train címkével tanított SmallCNN; a változó tényező a train célok forrása (clean vs. gyenge). Metrika: clean val accuracy, valamint a scene-szintű bootstrapből számított 95%-os CI az accuracy-különbségre a bootstrap-intervallum ismételt mintavételi értelmezése szerint (vö. [S04]). Érvényességi korlát: a gyenge címkék hibája valószínűleg aszimmetrikus, például a vertical (0) és diagonal (2) jellegű főtengelyek keverése gyakoribb (az osztályneveket a függelék A. A GlyphLab adatkészlet rögzíti), ezért a szimmetrikus zajra levezetett prior-képlet itt nem alkalmazható, és a kísérlet csak a GlyphLab adott heurisztikájára vonatkozik.

Kódvázlat a gyenge címkézésre és a páros, scene-szintű bootstrapra:

```
import numpy as np

def weak_orientation_labels(images):
    """images: (N,1,24,24) float32; képnyomaték-alapú gyenge címke.
    0 = függőleges, 1 = vízszintes, 2 = átlós."""
    ys, xs = np.mgrid[0:24, 0:24]
    out = np.empty(len(images), dtype=np.int64)
    for i, img in enumerate(images[:, 0]):
        w = img / img.sum()
        mx, my = (w * xs).sum(), (w * ys).sum()
        mxx = (w * (xs - mx) ** 2).sum()
        myy = (w * (ys - my) ** 2).sum()
```

```
        mxy = (w * (xs - mx) * (ys - my)).sum()
        deg = np.degrees(0.5 * np.arctan2(2 * mxy, mxx - myy)) % 180
        if 60 <= deg < 120:
            out[i] = 0      # függőleges
        elif deg < 30 or deg >= 150:
            out[i] = 1      # vízszintes
        else:
            out[i] = 2      # átlós
    return out

def scene_bootstrap_ci(diff, scene_id, n_boot=10000, seed=7):
    """diff: képenkénti páros különbség; scene-szintű bootstrap."""
    scenes = np.unique(scene_id)
    per_scene = np.array([diff[scene_id == s].mean() for s in scenes])
    rng = np.random.default_rng(seed)
    idx = rng.integers(0, len(per_scene), size=(n_boot, len(per_scene)))
    means = per_scene[idx].mean(axis=1)
    return np.percentile(means, [2.5, 97.5])
```

A `weak_orientation_labels` a kép másodrendű centrális nyomatékaiból becsüli a főtengely szögét; a 30 és 60 fokos küszöbök illusztratívak, a tényleges GlyphLab-képeken a clean validációs címkékhez kalibrálandók, mielőtt bármilyen következtetést vonunk le. A `scene_bootstrap_ci` bemenete képenkénti páros különbség (ugyanazon a képen a gyenge és a clean modell helyes döntésének különbsége), ezért a két modell és a két nézet összetartozása megmarad; a bootstrap scene-eket mintavételez újra, nem képeket.

Recept, seedek, döntési küszöb és erőforrás: a `SmallCNN` tanítása Adam optimalizálóval, `lr=10^{-3}`, 30 epoch és 16-os batch mellett fut, öt seeddel (20261020–20261024), a clean és a gyenge címkés ágban ugyanazokkal a seedekkel. A döntési szabály számszerű: a clean val accuracy páros, jelenetszintű bootstrap 95%-os CI-je; javulást vagy romlást akkor állítunk, ha a CI kizárja a 0-t, és a pontbecslés legalább 2 százalékpont. Erőforrás: CPU, `float32`, két ág × öt seed = 10 futás, néhány perc; a futásidőt a napló rögzíti. ### Tipikus tévedés ellenpéldával

A leggyakoribb tévedés az, hogy a nagyobb háló „megtanulja figyelmen kívül hagyni” a rossz címkéket, ezért a címkezaj automatikusan eltűnik. Ez nem igaz: a cross-entropy cél éppen a zajos $\tilde{Y}$-t követi, és elegendő kapacitás mellett a háló a hibás címkéket is interpolálja. Ellenpélda számított 1D feladaton: legyen $X \sim U(-1, 1)$, $Y = 1$, ha $X > 0$, különben 0; a clean Bayes-accuracy 1. A train címkék 30%-át felforgatjuk. Egy 1-NN osztályozó kapacitása elegendő a train pontok tökéletes lejegyzéséhez: noisy train accuracy 1, a clean train címkékhez mért találati arány viszont várhatóan $1 - 0{,}3 = 0{,}7$. A clean test accuracy sem haladja meg tartósan ezt a szintet: a tesztponthoz legközelebbi tanítópont távolsága $N \to \infty$ mellett 0-hoz tart, ezért a szomszéd tiszta címkéje egyre nagyobb valószínűséggel egyezik a tesztpont tiszta címkéjével, a megfigyelt szomszédcímke azonban 0,3 valószínűséggel felforgatott; a határérték tehát 0,7 (a 0 döntési határ elhanyagolható mértékű környezetét kivéve). A nagyobb kapacitású modell tehát *pontosabban jegyzi meg a zajt*, és kapacitás önmagában nem állítja helyre a tiszta határt. Ebből nem következik, hogy a zajos címkékből tanult osztályozó soha nem javulhat: szimmetrikus zajnál $\epsilon < (K-1)/K$ mellett a zajos posterior $\tilde{p}(k \mid x) = (1 - \frac{K\epsilon}{K-1})p(k \mid x) + \frac{\epsilon}{K-1}$ a tiszta $p(k \mid x)$ szigorúan növekvő affin függvénye, ahol $p(k \mid x) = P(Y = k \mid X = x)$ a tiszta és $\tilde{p}(k \mid x) = P(\tilde{Y} = k \mid X = x)$ a zajos posterior, tehát az $\arg\max$ megmarad: a zajos posteriorból számított Bayes-döntés megegyezik a tisztáéval, és elegendő adat, simaság vagy regularizáció mellett a tanult döntés közelíthet hozzá. A posterior (a valószínűségek) és a döntési határ különböző objektum: a zaj a posteriort torzítja (kalibráció romlik), a döntést nem, amíg az $\arg\max$ megmarad.

22.3.8 Önellenőrzés és korábbi fogalmak felidézése

Önellenőrző kérdések: a zajátmenet-mátrix minden sora 1-re összegződik-e? A laborkódban a valid/test indexeket egyetlen zajfüggvény sem érinti? A lemmában a feltételes függetlenséget kimondtuk-e? A kovariáns zaj és címkezaj faktorizációja különbözik-e?

Rövid válaszok. (1) Igen: $Q_{kj} = P(\tilde{Y} = j \mid Y = k)$ feltételes eloszlás, ezért $\sum_j Q_{kj} = 1$ minden k-ra. (2) Igen: a zaj csak a train indexekre hat, a valid és test címkék érintetlenek. (3) Igen: a definíció kimondja a minták feltételes függetlenségét és a $\tilde{Y}_i \perp\!\!\!\perp X_i \mid Y_i$ feltételt, a lemma ezekre épül. (4) Igen: a címkezajé $P(X, Y, \tilde{Y}) = P(X, Y)P(\tilde{Y} \mid Y)$, a kovariáns zajé $P(X, Y, \tilde{X}) = P(X, Y)q(\tilde{X} \mid X)$.

Visszautalás. 21.3. Dropout, súlyregularizálás és augmentáció label smoothing-ja determinisztikusan keveri a célvektort, míg a címkezaj véletlenül, mintánként eltérően; a 18.3. Pre-activation, kis batch és alternatív normalizálás normalizáló rétegei az aktivációk skáláját kezelik, és nem korrigálják a cél hibáját; a 10.3. Gradiensellenőrzés és numerikus hibamérce numerikus ellenőrzés tanulsága szerint a zajszimulációt is seed-del és referenciaimplementációval kell reprodukálni.

22.3.9 Rövid összefoglaló

A címkezaj a céloldali hiba: a zajos $\tilde{Y}$ a $Y \to \tilde{Y}$ átmenetmátrix szerint keletkezik, a bemenet érintetlen. A kovariáns zaj ezzel szemben az $X \to \tilde{X}$ élen hat, a címke változatlan. A szimmetrikus zaj modelljében a megfigyelt prior az $\tilde{\pi}_j = (1-\epsilon)\pi_j + \epsilon(1-\pi_j)/(K-1)$ képlettel torzul; egyenletes priorra a marginális eloszlás változatlan. A labor a GlyphLab train részén szabályozott zajjal szemlélteti, hogy a clean validáció romlása növekvő ϵ-nel várható; a tényleges mérés seed- és adatfüggő. A gyenge felügyelet, annotátor-egyezés és review az adatminőségi folyamat része, a leakage pedig a split tiszteletben tartásának követelménye.

22.3.10 Következő lecke

A 22.4. Augmentációs kísérlet és domainváltozás egységben az augmentációs kísérletet és a domainváltást tárgyaljuk: ott a képi eloszlás megváltozását, nem a cél hibáját vizsgáljuk, és azt kérdezzük, hogyan lehet a GlyphLab-receptet más, még nem látott képi feltételre átvinni.

22.3.11 Felhasznált források

- [D08] PyTorch contributors (2026). *torch.utils.data*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/data.html. Hozzáférés: 2026-10-30.
- [S04] Efron, B. és Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

22.4 Augmentációs kísérlet és domainváltozás

22.4.1 Cél, előismeret és induló becslés

A 22.3. Zajos és hiányos címke egységben a címkére ható zajt vizsgáltuk. Itt a bemenetre ható két műveletet választjuk szét: az **augmentation policy** a tanítás idején szándékosan bővíti az

eloszlást, a **corruption** pedig a kiértékelés idején szándékosan eltolja azt. A kettő összekeverése az egyik leggyakoribb kísérleti hiba: a tanítási transformáció növelheti az invarianciát, a tesztkorruptálás viszont egy másik alkalmazási tartományt szimulál.

Előismeret: a 22.2. Címkemegőrző transzformációk címkemegőrző transzformációi, a 22.3. Zajos és hiányos címke covariate-zaj fogalma, a 21.4. Osztályarány, súlyozás és céleloszlás súlyozott empirikus kockázata, a 18.4. Reziduális blokk ablatív vizsgálata matched recipe (azonos architektúra, optimalizáló és költségvetés melletti összehasonlítás) elve és a 10.4. Tanítási lépés és tanuló MLP holdout-protokollja.

Induló kérdés. Egy modellt véletlen fényesség-augmentációval tanítunk. A validációs pontosság 0,85, a sötétebb, zajosabb corrupt teszten 0,61. Miért nem mondhatjuk egyszerűen, hogy „a modell 0,24 pontot romlott”? Mert a 0,85 és a 0,61 két különböző eloszláson mért mennyiség; a különbség csak akkor értelmezhető, ha előbb pontosan megmondjuk, melyik halmaz melyik eloszlást reprezentálja. Ezt a levezetést a 3. szakasz adja meg.

22.4.2 Definíciók, jelölések és alakok

Augmentation policy. Legyen $\mathcal{T}$ a bemeneti tér címkemegőrző transzformációinak egy halmaza, μ pedig rajtuk egy valószínűségi eloszlás. A $(\mathcal{T}, \mu)$ pár az augmentation policy. Egy tanítási lépésben minden egyes X bemenetre függetlenül húzunk egy $t \sim \mu$ transzformációt, és a modell a $t(X)$ képet látja; a Y célpont változatlan marad. A policy által definiált célfüggvény a következő módosított empirikus kockázat:

$$\hat{R}_{\text{aug}}(f) = \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}_{t \sim \mu}\big[\ell(f(t(X_i)), Y_i)\big],$$

ahol $X_i \in [0;1]^{1\times 24\times 24}$ a függelék A. szakaszában leírt GlyphLab-kép NCHW (batch, csatorna, magasság, szélesség) elrendezésben, $Y_i \in \{0;1;2\}$ az int64 osztályindex a GlyphLab rögzített megfeleltetésével (0 = vertical, 1 = horizontal, 2 = diagonal), ℓ pedig a veszteség. A minibatch-training ennek Monte-Carlo-közelítését minimalizálja, azaz a várható értéket független véletlen húzások átlagával becsüli: egy epochon belül egy mintaelem tipikusan csak egy t-realizációt (egy konkrét húzás kimenetét) lát.

A GlyphLab-szerződés fontos korlátja, hogy a 90°-os forgatás nem címkemegőrző: a vertical osztályt horizontalra képezi, és a diagonal osztályban csak az egyik ferde irány szerepel. Megengedhető viszont a multiplikatív fényesség-skálázás, az additív zaj, a kis eltolás és a kontrasztmódosítás, mert ezek nem változtatják meg az orientáció szemantikáját.

Seed. Az a véletlenállapot, amely a policy-transzformációk húzását és az esetleges sztochasztikus korruptálást vezérli. Az 5.4. Függő képek, seedek és resampling szerint a reprodukálhatósághoz seed, algoritmus, könyvtárverzió és adatbetöltési sorrend együttesen szükséges; önmagában egyetlen seed rögzítése nem garantálja a hordozhatóságot.

Corruption. Kizárólag a tesztbemenetekre alkalmazott, rögzített vagy seeddel rögzített degradáló leképezés. A GlyphLab `corrupted_test.npy` fájlja a 60 test képet zajjal és fényességgradienssel módosítja, ugyanabban a scene/view sorrendben, mint a clean test. A corruption nem része a tanítási policynek.

Domain shift. Az a helyzet, amikor a tanítási és az alkalmazási bemeneti eloszlás különbözik: $P_{\text{train}}(X) \neq P_{\text{deploy}}(X)$. A corrupt teszt egy szintetikus **corruption shift** (bemeneti eltolódás): a címkét változatlanul hagyjuk, de az X eloszlását a c leképezéssel eltoljuk. Ez önmagában nem **covariate shift** a szigorú értelemben, mert a címke változatlanul hagyása nem jelenti, hogy a

$P(Y \mid X)$ feltételes eloszlás megmarad: az információvesztő zaj (például erős elmosás) a címke feltételes eloszlását, a $P(Y \mid X)$ posteriort $P(Y \mid \tilde{X}) \neq P(Y \mid X)$ irányba mozdíthatja. Covariate shiftként csak akkor kezeljük, ha a $P(Y \mid \tilde{X}) = P(Y \mid X)$ feltevést külön igazoltuk vagy vállaljuk. A továbbiakban a corrupt tesztet ezzel a feltevéssel kezelt corruption shiftként értelmezzük, és ahol a feltevés nem áll, azt külön jelezzük.

Frozen holdout. Olyan teszt-/kiértékelési halmaz, amelyet pontosan egyszer, az összes modell- és hiperparaméter-döntés után értékelünk ki, további frissítés vagy választás nélkül. A rajta mért metrika annak az eloszlásnak a kockázatát becsli, amelyet a halmaz reprezentál.

22.4.3 Állítás: a clean és a corrupt teszt által reprezentált eloszlás

Legyen $\mathcal{X} \subset \mathbb{R}^{1\times 24\times 24}$ a képtér, $\mathcal{Y} = \{0; 1; 2\}$ a célhalmaz. A GlyphLab generátor definiál egy $\mathcal{D}$ együttes eloszlást $\mathcal{X} \times \mathcal{Y}$ fölött: osztályfeltételes jelenetparaméterekből és nézetfüggő tiszta zajból. A clean test 30 független testjelenetből áll, jelenetenként két nézettel; az egyes képek marginális törvénye így $\mathcal{D}$. Legyen

$$c : \mathcal{X} \to \mathcal{X}$$

az a rögzített, mérhető leképezés, amely a `corrupted_test.npy`-ben tárolt zaj- és fényességgradiens-műveletet írja le; a fájl ennek a leképezésnek a 60 testképre vett realizált értékét rögzíti, a pushforward-állításhoz pedig c-t a teljes $\mathcal{X}$-en adott determinisztikus leképezésnek tekintjük. A corrupt teszt elemei $\tilde{X}_i = c(X_i)$, a hozzájuk tartozó célpont továbbra is Y_i.

Állítás 3.1 (reprezentált eloszlás és torzítatlanság). Tegyük fel, hogy

- (C1) a clean teszt ugyanazzal a jelenetprotokollal készül, mint a train/val adat, és a tesztjelenetek függetlenek minden modellválasztási lépéstől;
- (C2) c rögzített leképezés, amely a képen kívül nem használja a címkét;
- (C3) az f modell a tesztkiértékelés előtt rögzített, és sem a clean, sem a corrupt teszten nem hangoljuk.

Ekkor a clean teszt a $\mathcal{D}$ eloszlást, a corrupt teszt pedig a

$$\mathcal{D}_c(A \times \{k\}) = \mathcal{D}(\{(x, y) : c(x) \in A,\ y = k\})$$

pushforward (kép-) eloszlást reprezentálja; a $\mathcal{D}_c = c_{\#}\mathcal{D}$ jelölés a c szerinti kép-eloszlást rövidíti. Továbbá a 0/1 hibára (vagy bármely más korlátos ℓ veszteségre)

$$\mathbb{E}\left[\frac{1}{60}\sum_{i=1}^{60} \ell(f(\tilde{X}_i), Y_i)\right] = \int_{\mathcal{X}\times y} \ell(f(c(x)), y)\, d\mathcal{D}(x, y).$$

Bizonyítás. A c leképezés pontonként hat az X_i képre, és Y_i-t nem módosítja, ezért minden $(\tilde{X}_i, Y_i)$ pár marginális törvénye azonos a $(c(X), Y)$ pár törvényével, ahol $(X, Y) \sim \mathcal{D}$. A jeleneten belüli nézetfüggés a marginális törvényt nem érinti, csak az együttes eloszlás korrelációs szerkezetét. Rögzített f mellett az empirikus átlag lineáris funkcionál, így a várható értéke az egyedi várható értékek átlaga. A törvényazonosság miatt minden tag a jobb oldali integrállal egyenlő, tehát az állítás teljesül. □

Ha a corruptálás zajkomponense nem rögzített leképezés, hanem a mentett zajrealizációk képenként eltérőek, akkor nem létezik egyetlen közös $c : \mathcal{X} \to \mathcal{X}$ leképezés. Ilyenkor a modell $\tilde{X} = C(X, \epsilon)$, $\epsilon \sim \nu$ (vagy egy $q(\tilde{X} \mid X)$ átmeneti kernel). Ha minden képhez **egymástól és a mintától független** $\epsilon_i \sim \nu$ realizáció tartozik, a $\frac{1}{60}\sum_i \ell(f(C(X_i, \epsilon_i)), Y_i)$ átlag már egyetlen realizációval is torzítatlan becslője az átlagos corruption-kockázatnak, $\mathbb{E}_{(X,Y),\epsilon}\,\ell(f(C(X,\epsilon)), Y)$-nak. Több zajrealizáció a becslő varianciáját csökkenti, a torzítatlanságnak nem feltétele. A GlyphLab corrupt tesztje a mentett realizációkkal ennek egy véges mintáját adja; a rögzített fájlra vonatkozó állítás az adott realizációkra feltételes.

A frozen holdout megsértésének formális következménye a következő kiválasztási torzítás.

Állítás 3.2 (kiválasztási torzítás a teszten végzett választásnál). Legyen M jelölt modell $f_1, \dots, f_M$, és legyen $\hat{R}_m$ a m-edik modell empirikus hibája a corrupt teszten. Minden rögzített m-re teljesüljön $\mathbb{E}[\hat{R}_m] = R_m$, ahol R_m a valódi hiba. Ha a kiválasztás $\hat{m} = \arg\min_m \hat{R}_m$, akkor

$$\mathbb{E}[\hat{R}_{\hat{m}}] \le \min_m R_m .$$

Bizonyítás. Minden rögzített k-ra $\min_m \hat{R}_m \le \hat{R}_k$. Várható értéket véve $\mathbb{E}[\min_m \hat{R}_m] \le \mathbb{E}[\hat{R}_k] = R_k$. Mivel ez minden k-ra igaz, $\mathbb{E}[\min_m \hat{R}_m] \le \min_k R_k$. A kiválasztott modell jelentett hibája éppen $\hat{R}_{\hat{m}} = \min_m \hat{R}_m$. $\square$

A pontossággal való maximalizáló kiválasztásra legyen $\hat{A}_m$ az m-edik modell empirikus, A_m a valódi pontossága a corrupt eloszláson. Ha $\mathbb{E}[\hat{A}_m] = A_m$ minden m-re, akkor $\mathbb{E}[\max_m \hat{A}_m] \ge \max_m A_m$: minden k-ra $\max_m \hat{A}_m \ge \hat{A}_k$, így várható értékben $\mathbb{E}[\max_m \hat{A}_m] \ge A_k$, és ez minden k-ra fennáll. A jelentett maximum tehát felfelé torzított a legjobb valódi pontossághoz képest is. A tétel azt mondja ki, hogy a corrupt teszten végzett adaptív választás után a jelentett szám már nem a kiválasztott modell érintetlen becslője, hanem szélsőérték-statisztika.

22.4.4 Kézi kis példa és ellenőrző kód

Tekintsünk egy 3×3-as, egycsatornás vertical képet:

$$X = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \qquad Y = 0.$$

A policy legyen: $\frac{1}{2}$ valószínűséggel 0,8-cal szorzunk, különben azonosság. Három húzás szemléltetése:

Húzás	Transzformáció	Kapott osztálycímke	Átlagintenzitás
1. húzás	fényesség $\times 0{,}8$	0 (vertical)	$\frac{3 \cdot 0{,}8}{9} = 0{,}267$
2. húzás	azonosság	0 (vertical)	$\frac{3}{9} = 0{,}333$
	$0\ \ 0\ \ 0$		
90°-os forgatás	$1\ \ 1\ \ 1$	1 (horizontal)	0,333
(tiltott)	$0\ \ 0\ \ 0$		

A brightness-transzformáció megőrzi az orientációt, ezért a policy része lehet; a 90°-os forgatás a GlyphLab-szerződés szerint nem. A corruptáló leképezésre alkalmazzuk a $[-0{,}2; 0; 0{,}2]$ oszlopgradienst, zaj nélkül:

$$c(X) = \begin{bmatrix} 0 & 1 & 0{,}2 \\ 0 & 1 & 0{,}2 \\ 0 & 1 & 0{,}2 \end{bmatrix},$$

a negatív oszlopot a $[0; 1]$ tartomány nullára vágja. A címke még mindig 0, de a jobb oldali intenzitás megváltozott: ez a corruption shift apró, kézzel ellenőrizhető esete; a 2. szakasz feltevése szerint covariate shiftként akkor kezelhetjük, ha a leképezés az orientáció információját megőrzi — itt ez teljesül, mert a gradiens a glyph világos oszlopát nem tünteti el.

Minimális NumPy-ellenőrzés:

```
import numpy as np

def brightness_policy(x: np.ndarray, rng: np.random.Generator) -> np.ndarray:
    # x: (C,H,W) float32; csak fényesség, eltolás nélkül
    x = x.astype(np.float32)
    a = np.float32(0.8) if rng.random() < 0.5 else np.float32(1.0)
    return np.clip(a * x, 0.0, 1.0).astype(np.float32)

rng = np.random.default_rng(1)
img = np.zeros((1, 3, 3), dtype=np.float32)
img[0, :, 1] = 1.0                      # vertical bar
for step in range(5):
    out = brightness_policy(img, rng)
    col_sums = out[0].sum(axis=0)       # (3,) oszlopösszegek
    assert int(col_sums.argmax()) == 1  # az orientáció megmaradt
    print(step, out[0].tolist())
```

A kimenet az rng-állapottól függ; az ellenőrzött invariáns az, hogy az oszlopösszegek maximuma minden húzás után a középső oszlopban marad, tehát a vertical címke nem változik. A kódot a tanuló a saját környezetében futtathatja.

22.4.5 Labor: befagyasztott corrupt kiértékelés

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: a GlyphLab `data/` mappája és `data/corrupted_test.npy`; futtatás: `python quickcheck.py`, majd `python augment_experiment.py --seed 20261020 --epochs 15 --policy identity` és ugyanígy `--policy brightness_shift`. Várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor egyetlen kötelező szabálya: a `corrupted_test.npy` csak a befagyasztott modellen értékelhető, utólagos ráhangolás nélkül. A protokoll:

1. **Kérdés.** Csökkenti-e a brightness+eltolás augmentáció a clean→corrupt pontosságcsökkenést?
2. **Adat.** GlyphLab: train 240, val 60, clean test 60, corrupt test 60 kép, a corrupt test sorrendje a clean test sorrendjének felel meg (azonos képindexek).
3. **Megfigyelési egység.** `scene_id`; a bootstrap és a páros összehasonlítás jelenetszintű.
4. **Baseline.** Identity policy, minden más döntés azonos (matched recipe: a két futás architektúrája, optimalizálója, adatrendje és költségvetése megegyezik, 18.4. Reziduális blokk ablatív vizsgálata).
5. **Változó tényező.** A policy: identity versus brightness+shift; a brightness-szorzó $U(0{,}8; 1{,}2)$, az eltolás mintánként $\{-1; 0; 1\}$.
6. **Metrika.** Pontosság, valamint jelenetszintű páros clean−corrupt különbség.

7. **Választás.** Best checkpoint kizárólag a clean val loss alapján; a tesztet ezután egyszer értékeljük.

Munkakönyvtár: a `lab/` könyvtárból indulva. Először az adat integritását a kezdő labor `quickcheck.py` ellenőrzi. A végrehajtás CPU-n, float32-ben, seed `20261020` mellett fut:

```
python quickcheck.py
python augment_experiment.py --seed 20261020 --epochs 15 --policy identity
python augment_experiment.py --seed 20261020 --epochs 15 --policy brightness_shift
```

Az `augment_experiment.py` minimális, önállóan futtatható váza:

```
# lab/augment_experiment.py
import argparse, csv, json
from pathlib import Path
import numpy as np
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

data = Path("data")
manifest = json.loads((data / "manifest.json").read_text())
assert manifest["generator_version"] == "1.0"
X = torch.from_numpy(np.load(data / "images.npy"))            # (360,1,24,24)
y = torch.from_numpy(np.load(data / "labels.npy"))            # (360,)
X_c = torch.from_numpy(np.load(data / "corrupted_test.npy"))  # (60,1,24,24)
with open(data / "metadata.csv", newline="") as f:
    rows = list(csv.DictReader(f))
# A metadata `index` oszlopa köti a sorokat az images.npy soraihoz; a CSV
# sorszámát nem használjuk. A split a `split` oszlopból jön.
rows.sort(key=lambda r: int(r["index"]))
assert [int(r["index"]) for r in rows] == list(range(len(X)))
split = np.array([r["split"] for r in rows])
tr = torch.from_numpy(np.flatnonzero(split == "train"))
va = torch.from_numpy(np.flatnonzero(split == "val"))
te = torch.from_numpy(np.flatnonzero(split == "test"))
assert (len(tr), len(va), len(te)) == (240, 60, 60)
assert X_c.shape == (len(te), 1, 24, 24)

def augment(batch: torch.Tensor, rng: np.random.Generator,
            policy: str = "brightness_shift") -> torch.Tensor:
    # batch: (N,1,H,W) float32; identity vagy brightness + mintánkénti eltolás.
    if policy == "identity":
        return batch
    N = batch.shape[0]
    a = torch.from_numpy(rng.uniform(0.8, 1.2, size=(N, 1, 1, 1))
                         .astype(np.float32))
    out = batch * a
    # Mintánként {-1,0,1} wrap-eltolás. A GlyphLab-generátor minden glyph
    # körül legalább 3 pixeles hátteret hagy, ezért a wrap legfeljebb háttér-
    # pixelt visz át, a glyph és a címke nem változik.
    dx = rng.integers(-1, 2, size=N)
    for i in range(N):
        out[i] = torch.roll(out[i], int(dx[i]), dims=-1)
    return torch.clamp(out, 0.0, 1.0)

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
```

```
        self.head = nn.Sequential(nn.Flatten(), nn.Linear(32 * 6 * 6, 3))
    def forward(self, x):
        return self.head(self.features(x))

def evaluate(model, Xq, yq):
    model.eval()
    with torch.no_grad():
        logits = model(Xq)
        loss = F.cross_entropy(logits, yq, reduction="mean").item()
        acc = (logits.argmax(1) == yq).float().mean().item()
    return loss, acc, logits.argmax(1)

def main(seed=20261020, epochs=15, policy="brightness_shift"):
    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    train_ds = TensorDataset(X[tr], y[tr])
    train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
    model = SmallCNN()
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    best_val = float("inf")
    Path("runs").mkdir(exist_ok=True)
    ckpt = f"runs/best_seed{seed}_{policy}.pt"
    for epoch in range(1, epochs + 1):
        model.train()
        for xb, yb in train_loader:
            opt.zero_grad()
            loss = F.cross_entropy(model(augment(xb, rng, policy)), yb)
            loss.backward()
            opt.step()
        val_loss, val_acc, _ = evaluate(model, X[va], y[va])
        if val_loss < best_val:
            best_val = val_loss
            torch.save(model.state_dict(), ckpt)
        print(f"{epoch:02d} val_loss={val_loss:.4f} val_acc={val_acc:.4f}")
    # Választás CSAK clean val alapján történt; most jön a befagyasztott értékelés.
    model.load_state_dict(torch.load(ckpt))
    _, clean_acc, clean_pred = evaluate(model, X[te], y[te])
    _, corrupt_acc, corr_pred = evaluate(model, X_c, y[te])
    test_scenes = [rows[i]["scene_id"] for i in te.tolist()]
    scene_clean, scene_corr = {}, {}
    for idx, sid in enumerate(test_scenes):
        scene_clean.setdefault(sid, []).append(
            (clean_pred[idx] == y[te][idx]).item())
        scene_corr.setdefault(sid, []).append(
            (corr_pred[idx] == y[te][idx]).item())
    # Előjel: clean - corrupt, tehát a pozitív érték romlás.
    drops = np.array([np.mean(scene_clean[sid]) - np.mean(scene_corr[sid])
                      for sid in scene_clean])
    boot_rng = np.random.default_rng(seed)
    boot = np.array([boot_rng.choice(drops, len(drops), replace=True).mean()
                     for _ in range(2000)])
    lo, hi = np.percentile(boot, [2.5, 97.5])
    print(f"clean_test_acc={clean_acc:.4f} corrupt_test_acc={corrupt_acc:.4f}")
    assert len(drops) == 30, len(drops)
    print(f"mean_scene_drop={drops.mean():.4f} ci95=[{lo:.4f},{hi:.4f}]")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, default=20261020)
    parser.add_argument("--epochs", type=int, default=15)
    parser.add_argument("--policy", choices=("identity", "brightness_shift"),
                        default="brightness_shift")
    args = parser.parse_args()
```

```
    main(seed=args.seed, epochs=args.epochs, policy=args.policy)
```

A kód ezen a környezeten lefutott: Python 3.12.3, torch 2.15.0+cpu, CPU float32, `torch.get_num_threads()=10`, seed 20261020. A két policy futásának utolsó sorai:

```
# identity
15 val_loss=0.5627 val_acc=0.8667
clean_test_acc=0.9333 corrupt_test_acc=0.8500
mean_scene_drop=0.0833 ci95=[0.0167,0.1667]
# brightness_shift
15 val_loss=0.4780 val_acc=0.9000
clean_test_acc=0.9333 corrupt_test_acc=0.8500
mean_scene_drop=0.0833 ci95=[0.0000,0.1833]
```

A kód `assert`-jei a split méreteit, a corrupt teszt alakját és a jelenetszámot ellenőrzik. Ezen az egy seeden mindkét policy clean teszt pontossága 0,9333, corrupt pontossága 0,8500, a jelenetszintű átlagos romlás 0,0833; a bootstrap-intervallumok átfednek, és a brightness+shift policy nem mutat mérhető javulást az identity baseline-hoz képest. A két policy páros összehasonlítása (azonos seedek, jelenetszintű különbség bootstrapja) az F5 feladat; egyetlen seedből az eredmény nem általánosítható, és a jelenetszintű CI is csak a 30 tesztjelenetre feltételes.

Értelmezés: a clean pontosság a $\mathcal{D}$, a corrupt pontosság a $\mathcal{D}_c$ kockázatának becslője. A páros jelenetszintű drop azt méri, hogy ugyanazon jeleneteken mekkora romlást okoz a corruptálás; a két view átlagolása jelenetre csökkenti a nézetfüggő zaj hatását.

22.4.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Definiálja az augmentation policy-t és a corruption-t, és magyarázza el, miért nem címkemegőrző a 90°-os forgatás a GlyphLabban. Adjon meg egy megengedett és egy tiltott transzformációt, a bemeneti/kimeneti alakokkal együtt. Elvárt eredmény: saját megfogalmazású definíció, egy táblázat és egy mondat arról, hogy a policy miért csak a tanítási bemenetre hat.

F2 (levezetési, nehézség 2). Tegyük fel először, hogy egy befagyasztott modell corrupt tesztbeli pontossága 60 független Bernoulli-megfigyelés összege, $p = 0{,}7$ siker-valószínűséggel. Vezesse le az empirikus pontosság várható értékét és szórásnégyzetét. Ezután mondja ki, és vázolja a 3.1. Állítás torzítatlansági részének bizonyítását determinisztikus c mellett (a teljes bizonyítás a 3. szakaszban áll). Elvárt eredmény: képletek, a $p = 0{,}7$, $N = 60$ számértékek, valamint annak indoklása, hogy a jeleneten belüli pozitív korreláció miért növelheti a szórást az i.i.d. képlethez képest.

F3 (implementációs, nehézség 2). Írjon `apply_augmentation(X, Y, seed, use_shift=True)` függvényt, ahol `X` alakja `(N,1,H,W)` float32, `Y` alakja `(N,)` int64. A függvény brightness-skálázást és opcionális $\{-1; 0; 1\}$ eltolást alkalmaz, `Y`-t változatlanul adja vissza, és kizárja a forgatást. Ellenőrizze, hogy két azonos seedű hívás bitre azonos tömböt ad, az alakok megmaradnak, és az értékek $[0; 1]$-ben vannak. Elvárt eredmény: teljes kód, seed-ellenőrzés, shape-assert és egy rövid magyarázat a `copy` szükségességéről.

F4 (hibakeresési, nehézség 2). Az alábbi vázlat öt modellt tanít, a corrupt teszten választja ki a legjobbat, majd ugyanezt a pontosságot jelenti végső eredményként:

```
for i in range(5):
    train_model(seed=i)
    acc[i] = evaluate(corrupt_test).accuracy
best = argmax(acc)
print("végső corrupt teszt pontosság:", acc[best])
```

Azonosítsa a hibát, adja meg a 3.2. Állításból következő torzítás irányát (a pontosság felfelé, a hiba lefelé torzul), és írja le a javított protokollt. Elvárt eredmény: hibalokalizálás, következmény, javítás lépései.

F5 (kísérlettervezési, nehézség 3). Tervezzen GlyphLab-kísérletet annak eldöntésére, hogy a brightness+shift augmentáció csökkenti-e a clean→corrupt jelenetszintű romlást az identity baseline-hoz képest. Adja meg a matched recipe minden rögzített döntését, a seedek számát, a metrikát, a jelenetszintű bootstrap egységet és a konfidenciaintervallum ismételt mintavételi értelmezését (vö. [S04]). Elvárt eredmény: protokoll, baseline, változó tényező, erőforrás-korlát és az érvényességi határ. Előre megadott győztes nem szükséges.

22.4.7 Megoldások és indoklás

F1 megoldás. A policy $(\mathcal{T}, \mu)$: tanítási időben minden $X \in [0;1]^{1\times 24\times 24}$ képre $t \sim \mu$ húzás után $t(X)$-et adunk a hálónak, $Y \in \{0;1;2\}$ változatlan. A corruption a tesztképekre alkalmazott rögzített leképezés, amely a kiértékelési tartományt tolja el. A 90°-os forgatás a vertical-t horizontalra cseréli, a diagonal osztályban pedig a másik ferde irányt állítja elő, amely nincs a címkekészletben; ezért nem címkemegőrző. Megengedett: $t_\alpha(X) = \alpha X$, $\alpha \in [0{,}8; 1{,}2]$. Tiltott: 90°-os forgatás. A policy csak a tanítási bemenetre hat, mert a teszt eloszlásának megváltoztatása nélkül akarunk invarianciát tanítani.

F2 megoldás. Legyen $Z_i \sim \text{Bernoulli}(p)$ független az $i = 1, \dots, 60$ képre, $\hat{p} = \frac{1}{60}\sum_i Z_i$. Ekkor $\mathbb{E}[\hat{p}] = \frac{1}{60}\sum_i p = p = 0{,}7$, és a függetlenség miatt

$$\operatorname{Var}(\hat{p}) = \frac{1}{60^2}\sum_{i=1}^{60} p(1-p) = \frac{p(1-p)}{60} = \frac{0{,}7 \cdot 0{,}3}{60} = 0{,}0035,$$

tehát a szórás $\sqrt{0{,}0035} \approx 0{,}0592$. A 3.1. Állítás bizonyítása: minden $(\tilde{X}_i, Y_i)$ marginális törvénye $(c(X), Y)$ törvénye $(X, Y) \sim \mathcal{D}$ mellett, mert c pontonként hat és Y_i-t nem érinti; rögzített f esetén az átlag lineáris, ezért a várható érték az egyedi várható értékek átlaga, ami a jobb oldali integrállal egyenlő. A jeleneten belüli két view pozitívan korrelált, így a független mintákra érvényes $p(1-p)/60$ szórásnégyzet alulbecsli a tényleges szórást; az effektív mintanagyság (az i.i.d. mintáéval azonos szórásnégyzetet adó képzeletbeli mintaszám) 30 jelenet és 60 kép között van, amint azt az 5.4. Függő képek, seedek és resampling is tárgyalja.

F3 megoldás.

```
import numpy as np

def apply_augmentation(X, Y, seed, use_shift=True):
    X = np.asarray(X, dtype=np.float32).copy()
    Y = np.asarray(Y, dtype=np.int64)
    rng = np.random.default_rng(seed)
    N = X.shape[0]
    a = rng.uniform(0.8, 1.2, size=(N, 1, 1, 1)).astype(np.float32)
    out = X * a
```

```
    if use_shift:
        dx = rng.integers(-1, 2, size=N)
        for i in range(N):
            out[i] = np.roll(out[i], int(dx[i]), axis=-1)
    out = np.clip(out, 0.0, 1.0).astype(np.float32)
    return out, Y.copy()

X0 = np.zeros((2, 1, 4, 4), dtype=np.float32)
X0[:, 0, :, 1] = 1.0
Y0 = np.array([0, 0], dtype=np.int64)
a1, y1 = apply_augmentation(X0, Y0, 7)
a2, y2 = apply_augmentation(X0, Y0, 7)
assert a1.shape == X0.shape and y1.shape == Y0.shape
assert np.array_equal(a1, a2) and np.array_equal(y1, y2)
assert np.array_equal(y1, Y0) and np.all((a1 >= 0.0) & (a1 <= 1.0))

# Az eltolás helyességének ellenőrzése: azonos seeddel a brightness-húzás ugyanaz,
# ezért az eltolt kimenetnek cirkuláris egész eltolással át kell vihetőnek lennie
# a nem eltolt kimenetbe, mégpedig pontosan egy {-1, 0, 1} értékkel képenként.
a_ns, _ = apply_augmentation(X0, Y0, 7, use_shift=False)
a_sh, _ = apply_augmentation(X0, Y0, 7, use_shift=True)
for i in range(X0.shape[0]):
    shifts = [d for d in (-1, 0, 1)
              if np.array_equal(a_sh[i], np.roll(a_ns[i], d, axis=-1))]
    assert len(shifts) == 1, (i, shifts)
```

A `copy` nem szükséges ahhoz, hogy a skalározás ne írja felül a hívó `X`-ét: az `out = X * a` új tömböt hoz létre, és a `np.roll` is új tömböt ad vissza. A `.copy()` itt védekező másolat, amely garantálja, hogy a visszaadott tömb semmilyen esetben ne osszon memóriát a bemenettel; az `Y.copy()` ugyanezt biztosítja a címkékre, jóllehet azokat a függvény nem módosítja.

F4 megoldás. Hiba: a corrupt tesztet modellválasztásra használják, tehát a halmaz nem frozen holdout. A 3.2. Állítás pontosság-változata szerint $\mathbb{E}[\max_m \hat{A}_m] \geq \max_m A_m$: a jelentett maximum felfelé torzított a legjobb valódi pontossághoz képest is, és a kiválasztott modell saját valódi kockázatának torzítatlan becslője sem. Javítás: mind az öt seed modelljét tanítsa be, mindegyikhez a clean val loss alapján válasszon best checkpointot, fagyassza be, és csak ezután értékelje egyszer a clean és a corrupt teszten; hasonlítsa össze a jelenetszintű páros különbségeket.

F5 megoldás. Egy lehetséges protokoll. Kérdés: a brightness+shift policy csökkenti-e a corrupt romlást? Baseline: identity policy. Változó: policy. Rögzített: SmallCNN architektúra, AdamW, lr-ütemezés, weight decay, batch méret, epochszám, split a manifest szerint, seedhalmaz $\{20261020; 20261021; 20261022; 20261023; 20261024\}$. Minden seedhez a best checkpointot a clean val loss választja; a corrupt tesztet csak a befagyasztott modellen értékeljük. Metrika: pontosság és jelenetszintű clean−corrupt drop. A bootstrap egység `scene_id`, osztályonkénti rétegzéssel, 2000 újramintavétellel; az intervallum ismételt mintavételi értelmezése a [S04] bootstrap-értelmezését követi. A két policy-t ugyanazon seedekkel és ugyanazon testjeleneteken párosítjuk; a döntést a jelenet szintű páros különbség bootstrap CI-je adja: 0-t kizáró 95%-os CI esetén állítunk javulást, egyébként nem. Érvényességi határ: egyetlen corrupt-típus, 30 testjelenet, szintetikus GlyphLab, ezért az eredmény nem általánosítható természetes képi domain shiftre.

Hipotézis és egység: a brightness+shift policy csökkenti a corrupt romlást; a megfigyelési egység a `scene_id`. **Seed, döntési küszöb és erőforrás:** a bootstrap seedje 20261020, $B = 2000$; a döntési küszöb a jelenet szintű páros bootstrap 95%-os CI-je. Erőforrás: CPU, `float32`, két policy × öt seed = 10 futás, a 360 képű GlyphLabon néhány perc; a futásidőt és a seedenkénti epochszámot a napló rögzíti. ### Tipikus tévedés ellenpéldával

A leggyakoribb hiba, hogy a corrupt tesztet hangolásra (modellválasztásra) használják, majd ugyanazt a számot jelentik végső pontosságként. A 3.2. Állítás szemléltetése: ha öt jelölt valódi corrupt hibája mind $R_m = 0{,}30$, de a 60 képes empirikus hibák véletlen ingadozása $0{,}25; 0{,}32; 0{,}28; 0{,}35; 0{,}22$, a minimum alapján kiválasztott 0,22 könnyen kedvezőbbnek látszik a valódi 0,30-nál. A jelentett minimum nem a kiválasztott modell érintetlen becslője, hanem a minta szélsőértéke. A második gyakori hiba a tanítási policy alkalmazása a kiértékeléskor: ha a kiértékeléskor is véletlen fényességet húzunk, a clean teszt pontossága az augmentációs seedtől függő zajt kap, és már nem a $\mathcal{D}$-re vonatkozó reprodukálható becslő.

22.4.8 Önellenőrzés és visszautalás

1. Melyik eloszlást reprezentálja a clean, és melyiket a corrupt teszt? — A clean a $\mathcal{D}$-t, a corrupt a c leképezés utáni $\mathcal{D}_c = c_{\#}\mathcal{D}$-t.
2. Miért torzít a corrupt teszten végzett kiválasztás? — Mert $\mathbb{E}[\min_m \hat{R}_m] \le \min_m R_m$, tehát a jelentett minimum várható értéke nem nagyobb a legjobb valódi hibánál; pontosságra a maximum analóg módon felfelé torzít.
3. Mi a különbség az augmentation policy és a corruption között? — A policy tanítási eloszlásbővítés címkemegőrző transzformációkkal, a corruption rögzített tesztdegradáció, amely a kiértékelési tartományt tolja el.

Visszautalás: a 22.3. Zajos és hiányos címke megkülönböztette a label noise-t a covariate zajtól; itt a corruptálást a covariate shift feltevésével kezeljük: a c leképezés a címkét nem érinti, és a vizsgált degradáció az orientáció információját megtartja; a feltevés általános esetben igazolandó, nem automatikus. A 21.4. Osztályarány, súlyozás és céleloszlás súlyozott empirikus kockázata mutatta, hogy a cél eloszlása megváltoztatható; itt a bemeneti eloszlást változtatjuk. A 18.4. Reziduális blokk ablatív vizsgálata matched recipe elve biztosítja, hogy a policy-hatást egyetlen tényező izolálja, a 10.4. Tanítási lépés és tanuló MLP pedig a holdout elkülönítését alapozta meg.

22.4.9 Rövid összefoglaló

Az augmentációs policy címkemegőrző transzformációk fölötti eloszlás, amely csak a tanítási bemenetre hat, és az empirikus kockázat módosított változatát definiálja. A clean teszt a $\mathcal{D}$ cél-eloszlást, a corrupt teszt a rögzített c leképezés utáni $\mathcal{D}_c$ pushforward eloszlást reprezentálja; befagyasztott modellnél a corrupt halmazon számolt empirikus hiba torzítatlan becslő. A corrupt teszten végzett modellválasztás kiválasztási torzítást okoz: a jelentett minimum várható értéke nem nagyobb a legjobb valódi hibánál. A labor ezért kizárólag clean validáción választ best checkpointot, és a corrupt halmazt egyszer, a befagyasztott modellen értékeli, jelenetszintű páros különbséggel.

22.4.10 Következő lecke és források

A 23.1. Confusion matrix és aggregálási konvenció a confusion matrix és az aggregálási konvenciók kérdését tárgyalja; az itt bevezetett clean–corrupt páros szerkezet ott teszi lehetővé a csoportszintű intervallum és a páros összehasonlítás helyes értelmezését.

22.4.11 Felhasznált források

- [S04] Efron, B. és Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

23 Metrikák, kalibráció és független modellértékelés

23.1 Confusion matrix és aggregálási konvenció

23.1.1 Cél, előismeret és induló becslés

Ez az egység a modellértékelés első rétegét adja: egy véges, egycímkés osztályozási predikcióhalmazt hogyan foglalunk össze úgy, hogy a közölt szám értelmezhető és ellenőrizhető legyen. A pontosság egyetlen skalár, de elrejti, hogy a modell mely osztályokat téveszti össze, és hogy a hiba a ritka vagy a gyakori osztályt érinti-e. Az 5.4. Függő képek, seedek és resampling alfejezetben megismert függő képek miatt a confusion matrix cellái képszintű pontszámok; a klaszteres mintavételi bizonytalanságot külön, a 23.3. Páros összehasonlítás és csoportszintű intervallum alfejezetben kapcsoljuk hozzá. A 8.4. II. ellenőrző pont: korrekt tanulási állítás szerint a tesztmetrika a modellválasztás lezárása után, független halmazon értelmezhető; a 21.4. Osztályarány, súlyozás és céleloszlás osztályarányai pedig közvetlenül meghatározzák, hogy az accuracy mennyire érzékeny a többségi osztályra. A 22.4. Augmentációs kísérlet és domainváltozás clean/corrupt párosítását most egyelőre egyetlen kiértékelő halmazra szűkítjük.

Induló kérdés. Egy kiértékelő halmazban 60 kép van a GlyphLab három orientációs osztályából: 50 `vertical`, 6 `horizontal`, 4 `diagonal`. Ez a példa szándékosan hiányos osztályeloszlású, és nem a GlyphLab teszt splitje (amely a `metadata.csv` split mezője szerint 20-20-20 képet tartalmaz a 360 címkéből; a mért riportot lásd a laborban); a tanulság éppen az, hogy ugyanazon a halmazon az accuracy és a makro recall mennyire eltérhet. Egy konstans modell minden képre a `vertical` osztályt adja. Mennyi az accuracy, és mennyi a három osztály recalljának számtani átlaga? Becsülje meg, mielőtt a kézi mátrix és a micro/macro aggregálás képleteivel kiszámolná; a két szám nagyon eltérő történetet mond.

Előismeret: osztályindex-célok és a függelék A. szakaszában leírt 3-osztályos GlyphLab-séma a 22.1. Adateredet, címke és csoportos split alfejezetből; a `logits` és `argmax` kapcsolata a 7.2. Softmax és többosztályos cross-entropy alfejezetből; az `inference_mode` alatti predikciógyűjtés a 11.1. Tensor és autograd-életciklus alfejezetből.

23.1.2 A confusion matrix és az egyosztályos metrikák

Legyen K az osztályok száma, N a kiértékelt minták száma. A predikciókat két int64 vektor tárolja:

$$y_{\text{true}} \in \{0, \dots, K-1\}^N, \qquad y_{\text{pred}} \in \{0, \dots, K-1\}^N.$$

Ha a modell logitot ad, akkor $y_{\text{pred},n} = \arg\max_j Z_{n,j}$. A $Z \in \mathbb{R}^{N \times K}$ pontszámokból a metrikához már nincs szükség softmaxra: a predikált osztály a nyers logits argmaxa, összhangban azzal, hogy a `CrossEntropyLoss` is nyers logitot vár (nem softmaxolt valószínűséget) [D07]. A döntési

szabályt ugyanakkor külön kiértékelő lépésben hajtjuk végre, nem a veszteségfüggvény belső működésére bízzuk.

Definíció (confusion matrix). A $C \in \mathbb{N}^{K\times K}$ mátrix C_{ij} eleme azon minták száma, amelyekre $y_{\text{true}} = i$ és $y_{\text{pred}} = j$:

$$C_{ij} = \sum_{n=1}^{N} \mathbf{1}\{y_{\text{true},n} = i,\, y_{\text{pred},n} = j\}.$$

A könyvben **a sor a tényleges, az oszlop a predikált osztály**. A szakirodalomban a transzponált konvenció is előfordul; ezért minden riportban a tengelyeket meg kell nevezni. A cellák darabszámok, nem valószínűségek, és

$$\sum_{i=0}^{K-1}\sum_{j=0}^{K-1} C_{ij} = N.$$

A sorösszeg $N_i = \sum_j C_{ij}$ az i osztály **support**ja, az oszlopösszeg $\widehat{N}_j = \sum_i C_{ij}$ a j osztályba sorolt minták száma.

Definíció (accuracy). Az accuracy (találati arány; a könyv a továbbiakban az accuracy szót használja, a precision magyar megfelelője pedig a precizitás) a helyes döntések aránya:

$$\mathrm{Acc} = \frac{1}{N}\sum_{i=0}^{K-1} C_{ii} = \frac{\operatorname{tr}(C)}{N}.$$

Ha $N = 0$, az accuracy definiálatlan; a továbbiakban $N > 0$.

Definíció (one-vs-rest sejtszámok). Rögzített i osztályra a többi osztályt negatívként összevonva:

$$\mathrm{TP}_i = C_{ii}, \quad \mathrm{FN}_i = \sum_{j\neq i} C_{ij}, \quad \mathrm{FP}_i = \sum_{j\neq i} C_{ji}.$$

A $\mathrm{TN}_i = N - \mathrm{TP}_i - \mathrm{FP}_i - \mathrm{FN}_i$ a többi cella összege; a könyv a továbbiakban nem használja, mert a vizsgált metrikák (precision, recall, F1, accuracy) csak a fenti három sejtszámot igénylik.

Innen a **precizitás** (precision) és a **recall** osztályonként:

$$P_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FP}_i}, \qquad R_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FN}_i}.$$

A precizitás azt kérdezi: a modell által i-nek mondott minták hányada valóban i; a recall (lefedettség) azt: a valódi i-k hányadát találta meg. Mindkettő akkor értelmes, ha a nevező pozitív:

- P_i nevezője nulla, ha az i osztályt senkinek sem jósoltuk;
- R_i nevezője nulla, ha az i osztály hiányzik a mintából.

A nulla nevezőre három `zero_division`-módot rögzítünk, amelyek a precizitásra, a recallra és a nulla nevezőjű $F1$-re vonatkoznak. A `raise` mód hibát dob. A `zero` mód a nulla nevezőjű értéket 0-nak veszi. Az `omit` mód `NaN`-t ad, és a makro átlagból kihagyja az érintett osztályt; a könyv laborja az `omit` módot használja a precision és a recall esetén, mert így látható marad, hogy az osztály nem volt megfigyelhető. A döntést a riportban mindig közölni kell.

Az **F1** osztályonként a precision és a recall harmonikus közepe, amelyet közvetlenül a darabszámokból számolunk:

$$F1_i = \frac{2P_iR_i}{P_i + R_i} = \frac{2\,\mathrm{TP}_i}{2\,\mathrm{TP}_i + \mathrm{FP}_i + \mathrm{FN}_i}.$$

A jobb oldali alak akkor is definiált, ha a precision vagy a recall nulla nevezője miatt hiányzik: ha $\mathrm{TP}_i = \mathrm{FP}_i = 0$ és $\mathrm{FN}_i > 0$ (az osztály létezik a célban, de soha nem találtuk el), akkor $F1_i = 0$, nem `NaN`. Az `omit` mód ezért nem hagyhatja ki ezt a soha fel nem ismert osztályt a makro $F1$-ből. $F1_i$ csak akkor definiálatlan, ha $2\mathrm{TP}_i + \mathrm{FP}_i + \mathrm{FN}_i = 0$, azaz az osztály sem a célban, sem a predikcióban nem szerepel; ekkor a `zero_division` mód dönt (`raise`: hiba, `zero`: 0, `omit`: `NaN` és kihagyás).

23.1.3 Levezetés kézi többosztályos mátrixból

Tekintsük a $K = 3$, $N = 13$ esetet:

$$C = \begin{pmatrix} 5 & 1 & 0 \\ 0 & 3 & 1 \\ 1 & 0 & 2 \end{pmatrix}.$$

Az accuracy a főátló összege osztva N-nel:

$$\mathrm{Acc} = \frac{5 + 3 + 2}{13} = \frac{10}{13} \approx 0{,}7692.$$

A 0. osztályra: $\mathrm{TP}_0 = 5$, $\mathrm{FN}_0 = 1 + 0 = 1$, $\mathrm{FP}_0 = 0 + 1 = 1$, tehát

$$P_0 = \frac{5}{5+1} = \frac{5}{6}, \qquad R_0 = \frac{5}{5+1} = \frac{5}{6}, \qquad F1_0 = \frac{2 \cdot \frac{5}{6} \cdot \frac{5}{6}}{\frac{5}{6} + \frac{5}{6}} = \frac{5}{6}.$$

Az 1. osztályra: $\mathrm{TP}_1 = 3$, $\mathrm{FN}_1 = 0 + 1 = 1$, $\mathrm{FP}_1 = 1 + 0 = 1$, így

$$P_1 = \frac{3}{4}, \qquad R_1 = \frac{3}{4}, \qquad F1_1 = \frac{3}{4}.$$

A 2. osztályra: $\mathrm{TP}_2 = 2$, $\mathrm{FN}_2 = 1 + 0 = 1$, $\mathrm{FP}_2 = 0 + 1 = 1$, így

$$P_2 = \frac{2}{3}, \qquad R_2 = \frac{2}{3}, \qquad F1_2 = \frac{2}{3}.$$

A mikro összegzés a cellákat osztályok fölött összeadja:

$$\mathrm{TP}_{\mathrm{micro}} = \sum_i \mathrm{TP}_i = 10, \qquad \mathrm{FP}_{\mathrm{micro}} = \sum_i \mathrm{FP}_i = 3, \qquad \mathrm{FN}_{\mathrm{micro}} = \sum_i \mathrm{FN}_i = 3.$$

Egycímkés osztályozásban minden minta pontosan egy predikciót kap, ezért minden téves cella egyszer jelenik meg FP-ként és egyszer FN-ként; így $\mathrm{TP}_{\text{micro}}+\mathrm{FP}_{\text{micro}} = N$ és $\mathrm{TP}_{\text{micro}}+\mathrm{FN}_{\text{micro}} = N$. Következésképpen

$$P_{\text{micro}} = R_{\text{micro}} = \mathrm{F1}_{\text{micro}} = \frac{10}{13} = \mathrm{Acc}.$$

Állítás. Egycímkés, $N > 0$ mintás osztályozásban $P_{\text{micro}} = R_{\text{micro}} = \mathrm{F1}_{\text{micro}} = \mathrm{Acc}$.

Bizonyítás. A definíciókból

$$\mathrm{TP}_{\text{micro}} + \mathrm{FP}_{\text{micro}} = \sum_i \left(C_{ii} + \sum_{j \neq i} C_{ji} \right) = \sum_{i,j} C_{ij} = N,$$

mert a belső összeg az i oszlop összege. Ugyanígy a recall nevezője a sorösszegek összege, ami szintén N. A közös számláló $\sum_i C_{ii}$, tehát a három mikro érték egyaránt $\mathrm{tr}(C)/N$. □

A makro átlagolás a fenti mátrixon:

$$P_{\text{macro}} = \frac{1}{3}\left(\frac{5}{6} + \frac{3}{4} + \frac{2}{3}\right) = \frac{1}{3} \cdot \frac{9}{4} = \frac{3}{4} = 0{,}75,$$

és ugyanígy $R_{\text{macro}} = F1_{\text{macro}} = 0{,}75$. Ez a példa elfedi a makro P, R és $F1$ különbségét, mert minden osztályra véletlenül $P_i = R_i$; ilyenkor a két átlag és a per-class $F1$-ek átlaga szükségszerűen egybeesik, ezért a példa önmagában nem mutatja meg, hogy a makro $F1$ nem a makro P és R harmonikus közepe. A következő mátrix megmutatja a nulla nevezőt is:

$$C_z = \begin{pmatrix} 4 & 2 & 0 \\ 1 & 3 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \qquad N = 12, \quad \mathrm{tr}(C_z) = 7.$$

A 2. osztályt senkinek sem jósolta a modell, ezért $\mathrm{FP}_2 = 0$ és $\mathrm{TP}_2 = 0$; a P_2 nevezője nulla. A 0. osztályra:

$$P_0 = \frac{4}{4+2} = \frac{2}{3}, \qquad R_0 = \frac{4}{4+2} = \frac{2}{3}.$$

Az 1. osztályra:

$$P_1 = \frac{3}{3+3} = \frac{1}{2}, \qquad R_1 = \frac{3}{3+1} = \frac{3}{4}, \qquad F1_1 = \frac{2 \cdot \frac{1}{2} \cdot \frac{3}{4}}{\frac{1}{2} + \frac{3}{4}} = \frac{\frac{3}{4}}{\frac{5}{4}} = \frac{3}{5} = 0{,}6.$$

A 2. osztályra $R_2 = \frac{0}{0+2} = 0$, de P_2 definiálatlan. `zero_division='omit'` mellett:

$$P_{\text{macro}} = \frac{\frac{2}{3} + \frac{1}{2}}{2} = \frac{\frac{7}{6}}{2} = \frac{7}{12} \approx 0{,}5833,$$

míg

$$R_{\text{macro}} = \frac{\frac{2}{3} + \frac{3}{4} + 0}{3} = \frac{\frac{17}{12}}{3} = \frac{17}{36} \approx 0{,}4722.$$

A makro $F1$ az osztályonkénti $F1$-ek átlaga. A 2. osztályra $F1_2 = \frac{2\cdot 0}{2\cdot 0+0+2} = 0$ definiált (az osztály a célban szerepel, de soha nem találtuk el), tehát az `omit` mód sem hagyja ki:

$$F1_{\text{macro}} = \frac{\frac{2}{3} + \frac{3}{5} + 0}{3} = \frac{19}{45} \approx 0{,}4222.$$

A `zero_division='zero'` választás esetén $P_2 = 0$, az $F1$ ugyanaz, ezért

$$P^{(0)}_{\text{macro}} = \frac{\frac{2}{3} + \frac{1}{2} + 0}{3} = \frac{7}{18} \approx 0{,}3889, \qquad F1^{(0)}_{\text{macro}} = \frac{\frac{2}{3} + \frac{3}{5} + 0}{3} = \frac{19}{45} \approx 0{,}4222 \quad \text{(azonos az } \texttt{omit} \text{ móddal).}$$

A mikro érték mindkét módban

$$\text{Acc} = P_{\text{micro}} = R_{\text{micro}} = \text{F1}_{\text{micro}} = \frac{7}{12} \approx 0{,}5833.$$

Ez a három szám egyszerre mutatja a nulla nevező hatását és a mikro/makro eltérést: ugyanazon a predikciós halmazon a makro precision `omit` módban 0,5833, `zero` módban 0,3889, a makro $F1$ mindkét módban 0,4222, a mikro $F1$ pedig 0,5833. A „0,42-es makro F1” és a „0,58-as mikro F1” állítás tehát nem ellentmondás.

23.1.4 Micro/macro aggregálás és class prevalence

A mikro összegzés a mátrix összes nem átlós celláját egyenlően kezeli, ezért a gyakori osztályok hibái nagyobb darabszámot kapnak. A makro összegzés minden osztályt egy szavazatként kezel, ezért a ritka osztály hibája ugyanannyit nyom, mint a gyakorié. Egyik sem „helyesebb” általánosságban: a jelentéshez a felhasználási szerződésből kell megmondani, hogy egy tévesztés költsége darabszám-arányos-e.

Definíció (class prevalence). Az i osztály **prevalenciája** a mintában

$$p_i = \frac{N_i}{N}, \qquad \sum_i p_i = 1.$$

A prevalence a cél eloszlásának empirikus aránya, nem a modell tulajdonsága. A FP, FN definíciójából $N_i = \text{TP}_i + \text{FN}_i$, ezért

$$\text{Acc} = \sum_i \frac{\text{TP}_i}{N} = \sum_{i:N_i>0} p_i \frac{\text{TP}_i}{N_i} = \sum_{i:N_i>0} p_i R_i.$$

Az accuracy tehát a recall prevalence-súlyozott átlaga. Ez a levezetés magyarázza az induló kérdést: a konstans `vertical` modellnél $R_0 = 1$, $R_1 = R_2 = 0$, a prevalenciák $50/60, 6/60, 4/60$, így

$$\text{Acc} = \frac{50}{60}\cdot 1 + \frac{6}{60}\cdot 0 + \frac{4}{60}\cdot 0 = \frac{5}{6} \approx 0{,}8333,$$

míg a makro recall

$$R_{\text{macro}} = \frac{1+0+0}{3} = \frac{1}{3} \approx 0{,}3333.$$

Ugyanaz a modell tehát „83% pontosságú” és „33% makro recallú” egyszerre; a két szám nem ugyanazt a kérdést válaszolja meg.

A makro $F1$ definíciója a per-class $F1_i$ értékek átlaga. **Nem** a makro P és makro R harmonikus közepe. A C_z példán `omit` módban:

$$\frac{2P_{\text{macro}}R_{\text{macro}}}{P_{\text{macro}}+R_{\text{macro}}} = \frac{2\cdot\frac{7}{12}\cdot\frac{17}{36}}{\frac{7}{12}+\frac{17}{36}} \approx 0{,}5219 \neq 0{,}4222 = F1_{\text{macro}}.$$

A két képlet közötti különbség akkor nagy, ha a P_i és R_i sorrendje osztályonként eltér: van olyan osztály, ahol a precizitás magas és a recall alacsony, és olyan is, ahol fordítva.

23.1.5 Labor: kézi minta és futó CNN-riport ellenőrzése

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5. A toy mód beépített tömböket használ; a riport módhoz `data/labels.npy`, `metadata.csv` és a hallgató `my_run/eval_preds.npz` fájlja kell. Futtatás: `python metrics_check.py --mode toy`, illetve `python metrics_check.py --mode report --report my_run/eval_preds.npz`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

Cél. Egy saját NumPy-függvény számítsa ki a confusion matrixot és a metrikákat, és ellenőrizze a C_z kézi példát; majd a tanuló saját CNN-jének kiértékelő riportját értelmezze.

Input. A toy mód beépített tömböket használ. A riport mód bemenete:

- `data/labels.npy`: `(360,)` int64 GlyphLab-címkék;
- `my_run/eval_preds.npz`: `indices (N,)` int64, a `metadata.csv` `index` oszlopa szerinti képtömb-sorindex (a `labels.npy` soraihoz), `logits (N,3)` float32.

A riportot a modell kiértékelésekor így menthetjük (a `model` a betöltött checkpoint modellje, az `eval_loader` pedig `(x, index)` párokat adó `DataLoader`; a részlet önmagában nem futtatható, mert ezeket a tanítási szkript állítja elő):

```
def save_eval_preds(model, eval_loader, out="my_run/eval_preds.npz"):
    # a modell saját seed-protokollja után, eval módban:
    model.eval()
    logits_list, index_list = [], []
    with torch.inference_mode():
        for x, idx in eval_loader:
            logits_list.append(model(x))
            index_list.append(idx)
    z = torch.cat(logits_list, dim=0).cpu().numpy().astype(np.float32)
    idx = torch.cat(index_list, dim=0).cpu().numpy().astype(np.int64)
    np.savez_compressed(out, indices=idx, logits=z)
```

A metrika számítása determinisztikus, ezért véletlen seedre nincs szükség. A bemenő logits `float32`, a belső osztások `float64`-ben futnak; az eszköz NumPy-CPU.

Kód. Munkakönyvtár `lab/`, fájl `metrics_check.py`:

```python
#!/usr/bin/env python3
# Labor: confusion matrix és aggregációs konvenciók.
# Toy:  python metrics_check.py --mode toy
# Riport: python metrics_check.py --mode report --report my_run/eval_preds.npz
import argparse
import numpy as np

def confusion_matrix(y_true, y_pred, num_classes=None):
    y_true = np.asarray(y_true, dtype=np.int64)
    y_pred = np.asarray(y_pred, dtype=np.int64)
    if y_true.shape != y_pred.shape:
        raise ValueError("y_true es y_pred alakja elter")
    if num_classes is None:
        num_classes = int(max(y_true.max(initial=0),
                              y_pred.max(initial=0))) + 1
    K = num_classes
    if K <= 0:
        raise ValueError("num_classes pozitiv kell legyen")
    if y_true.min(initial=0) < 0 or y_pred.min(initial=0) < 0 or \
       y_true.max(initial=0) >= K or y_pred.max(initial=0) >= K:
        raise ValueError("cimke kivul esik a [0, K) tartomanyon")
    C = np.zeros((K, K), dtype=np.int64)
    np.add.at(C, (y_true, y_pred), 1)
    return C

def precision_recall_from_confusion(C, zero_division="omit"):
    if zero_division not in {"raise", "zero", "omit"}:
        raise ValueError("ismeretlen zero_division mod: " + str(zero_division))
    K = C.shape[0]
    tp = np.diag(C).astype(np.float64)
    fp = C.sum(axis=0) - tp
    fn = C.sum(axis=1) - tp

    def safe(num, den):
        if zero_division == "raise":
            if np.any(den == 0):
                raise ZeroDivisionError("nulla nevezo a precision/recall-ban")
            return num / den
        fill = 0.0 if zero_division == "zero" else np.nan
        out = np.full(K, fill, dtype=np.float64)
        np.divide(num, den, out=out, where=den > 0)
        return out

    P = safe(tp, tp + fp)
    R = safe(tp, tp + fn)
    # F1 = 2TP / (2TP + FP + FN): a soha el nem talalt, de letezo osztalyra 0, nem NaN
    F1 = safe(2.0 * tp, 2.0 * tp + fp + fn)
    return P, R, F1

def core_metrics(C, zero_division="omit"):
    N = int(C.sum())
    if N == 0:
        raise ValueError("ures matrixra a metrika definialatlan")
    P, R, F1 = precision_recall_from_confusion(C, zero_division)
    acc = float(np.diag(C).sum()) / N
    return {
        "accuracy": acc,
        "micro_precision": acc,
        "micro_recall": acc,
        "micro_f1": acc,
        "macro_precision": float(np.nanmean(P)),
        "macro_recall": float(np.nanmean(R)),
        "macro_f1": float(np.nanmean(F1)),
```

```
        "prevalence": C.sum(axis=1) / N,
        "per_class": {"precision": P, "recall": R, "f1": F1},
    }

TOY_TRUE = np.array([0]*6 + [1]*4 + [2]*2, dtype=np.int64)
TOY_PRED = np.array(
    [0,0,0,0,1,1,   # 0. sor: 4 db 0, 2 db 1
     0,1,1,1,       # 1. sor: 1 db 0, 3 db 1
     0,1],          # 2. sor: 1 db 0, 1 db 1
    dtype=np.int64)
EXPECTED = np.array([[4,2,0],
                     [1,3,0],
                     [1,1,0]], dtype=np.int64)

def run_toy():
    C = confusion_matrix(TOY_TRUE, TOY_PRED, num_classes=3)
    assert np.array_equal(C, EXPECTED)
    res = core_metrics(C, zero_division="omit")
    assert np.isclose(res["accuracy"], 7/12)
    assert np.isclose(res["micro_f1"], 7/12)
    assert np.isclose(res["macro_f1"], 19/45)
    assert np.isclose(res["per_class"]["precision"][2], np.nan, equal_nan=True)
    print("toy ellenorzes OK")
    for k in ["accuracy","micro_f1","macro_precision",
              "macro_recall","macro_f1"]:
        print(f"{k:18s} {res[k]:.4f}")

def run_report(path, labels_path="data/labels.npy", num_classes=3):
    data = np.load(labels_path)
    rpt = np.load(path)
    indices = rpt["indices"].astype(np.int64)
    logits = rpt["logits"].astype(np.float32)
    if logits.ndim != 2 or logits.shape[1] != num_classes:
        raise ValueError(f"GlyphLab-riport: logits alak (N,{num_classes}) kell")
    if indices.shape != (logits.shape[0],):
        raise ValueError("indices es logits hossza elter")
    if len(indices) == 0:
        raise ValueError("ures riport")
    if indices.min() < 0 or indices.max() >= len(data):
        raise ValueError("indices kivul esik a labels.npy tartomanyon")
    y_true = data[indices]
    y_pred = np.argmax(logits, axis=1).astype(np.int64)
    C = confusion_matrix(y_true, y_pred, num_classes=num_classes)
    assert C.sum() == len(indices)
    res = core_metrics(C, zero_division="omit")
    print("confusion matrix (sor: igaz, oszlop: predikcio):")
    print(C)
    print("sorosszeg (tamogatas):", C.sum(axis=1))
    print("oszloposszeg (predikcio):", C.sum(axis=0))
    print("normalizalt sorok (recall-profil):")
    for i, row in enumerate(C):
        if row.sum() > 0:
            print(i, np.round(row / row.sum(), 3))
    for k in ["accuracy","micro_f1","macro_precision",
              "macro_recall","macro_f1"]:
        print(f"{k:18s} {res[k]:.4f}")

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--mode", choices=["toy","report"], required=True)
    p.add_argument("--report", default="")
    a = p.parse_args()
    if a.mode == "toy":
```

```
        run_toy()
    else:
        run_report(a.report)
```

Futtatás és ellenőrzés.

```
cd lab
python metrics_check.py --mode toy
```

A toy mód kimenete mért futásból származik (Python 3.12.3, NumPy 2.5.4, CPU): accuracy 0,5833, micro F1 0,5833, macro precision 0,5833, macro recall 0,4722, macro F1 0,4222. A második osztály precisionje a `zero_division='omit'` miatt `NaN`, ezt a program `assert`-tel ellenőrzi. A mért kimenet:

```
toy ellenorzes OK
accuracy           0.5833
micro_f1           0.5833
macro_precision    0.5833
macro_recall       0.4722
macro_f1           0.4222
```

A riport mód:

```
python metrics_check.py --mode report --report my_run/eval_preds.npz
```

A referencia-modell (a `lab/train_cnn.py` 1299 paraméteres `SmallCNN`-je, seed 20261020, CPU, 1 szál) predikcióival futtatva a mért kimenet:

```
confusion matrix (sor: igaz, oszlop: predikcio):
[[20  0  0]
 [ 0 20  0]
 [ 0  3 17]]
sorosszeg (tamogatas): [20 20 20]
oszloposszeg (predikcio): [20 23 17]
normalizalt sorok (recall-profil):
0 [1. 0. 0.]
1 [0. 1. 0.]
2 [0.   0.15 0.85]
accuracy           0.9500
micro_f1           0.9500
macro_precision    0.9565
macro_recall       0.9500
macro_f1           0.9497
```

A riport mód ellenőrzi a shape- és indexszerződést, majd kiírja a mátrixot, a támogatást és a normalizált sorokat. A sorösszegek az igaz eloszlást, a $\widehat{N}_j$ oszlopösszegek a predikciós eloszlást mutatják; a normalizált sor a recall, nem pedig a precizitás. A gyakori hiba az, ha a mátrixot a sklearn-stílusú `normalize='true'` nélkül közöljük [D15], és az olvasó a nagy cellaszámot automatikusan jó osztályozási aránynak hiszi.

23.1.6 Feladatok F1–F5

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem cserélhető fel a micro és a macro átlagolás. Adjon három osztályból álló, kis `y_true`/`y_pred` példát, amelyben az accuracy magas, de a macro recall alacsony. Elvárt: definíciók, a példa tömbjei, a két számérték.

F2 (levezetési, nehézség 2). A

$$C = \begin{pmatrix} 6 & 1 & 0 \\ 0 & 4 & 0 \\ 1 & 1 & 2 \end{pmatrix}$$

mátrixból számítsa ki az accuracyt, az osztályonkénti precision/recall/F1 értékeket, a micro és a macro összegzést. Használja a `zero_division='omit'` módot, és jelölje, ha valamelyik nevező nulla lenne. Elvárt: minden közbülső sejtszám és a három makro érték.

F3 (implementációs, nehézség 2). Implementálja a `precision_recall_from_confusion` függvényt a labor specifikációja szerint, a három `zero_division` móddal. Tesztelje a C_z mátrixon `omit` és `zero` módban is. Elvárt: futtatható függvény, a két mód makro F1-értéke, és egy negatív teszt, amely `raise` módban nulla nevezőre hibát ad.

F4 (hibakeresési, nehézség 2). Adott a következő hibás kód:

```
tp = np.diag(C).astype(np.float64)
fp = C.sum(axis=1) - tp   # sor szerint
fn = C.sum(axis=0) - tp   # oszlop szerint
P = tp/(tp+fp)
R = tp/(tp+fn)
macro_p = P.mean()
macro_r = R.mean()
macro_f1 = 2*macro_p*macro_r/(macro_p+macro_r)
```

Hol a hiba? Számítsa ki a C_z mátrixon, `zero_division='zero'` mellett, a hibás és a javított makro $F1$ értéket. Elvárt: a hiba oka, következménye, a javított tengelyek és a numerikus ellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). A GlyphLab teszt splitje osztályonként kiegyenlített: 20-20-20 kép, összesen 60 kép a 360 címkéből (`metadata.csv`, `split=test`); ezen a halmazon az accuracy és a makro recall megegyezik, ezért a kiegyensúlyozatlanság hatása csak más prevalencián vizsgálható. Az induló példa 50/6/4-es esete ezzel szemben szándékosan kiegyensúlyozatlan szemléltetés. A telepítési környezetben a feltételezett prevalencia $p' = (0{,}6; 0{,}3; 0{,}1)$ a `vertical, horizontal, diagonal` célra. Tervezze meg, hogyan hasonlítaná össze a mikro és a makro metrika érzékenységét erre a prevalence-változásra anélkül, hogy a tesztet újra felhasználná modellválasztásra. Elvárt: protokoll, a mátrixból számított átsúlyozási képlet, legalább két értelmezési korlát.

23.1.7 Megoldások és indoklás

F1. A micro átlagolás a mátrix celláit osztályok fölött összegzi; a gyakori osztály darabszáma nagyobb súlyt kap. A macro átlagolás minden osztály per-class értékét azonos $1/K$ súllyal veszi, függetlenül a supporttól. Példa: $y_{\text{true}} = (0,0,0,0,0,1,1,1,2)$ és $y_{\text{pred}} = (0,0,0,0,0,0,0,0,0)$. Ekkor $\text{Acc} = 5/9 \approx 0{,}556$, és $R = (1,0,0)$, így $R_{\text{macro}} = 1/3 \approx 0{,}333$. Ez az accuracy–macro recall hézag a ritka osztályok teljes eltévesztéséből adódik.

F2. A mátrix sorösszegei: $7, 4, 4$; oszlopösszegei: $7, 6, 2$; $N = 15$, $\text{tr}(C) = 12$, tehát

$$\mathrm{Acc} = \frac{12}{15} = 0{,}8.$$

A per-class sejtszámok:

osztály	TP_i	FP_i	FN_i
0	6	1	1
1	4	2	0
2	2	0	2

A 0. osztályra ezért $P_0 = R_0 = 6/7 \approx 0{,}8571$, $F1_0 = 6/7$. Az 1. osztályra $P_1 = 4/6 \approx 0{,}6667$, $R_1 = 1$, $F1_1 = 0{,}8$. A 2. osztályra $P_2 = 1$, $R_2 = 0{,}5$, $F1_2 = 2/3 \approx 0{,}6667$. Nulla nevező nem lép fel. A mikro érték $\mathrm{Acc} = 0{,}8$. A makro értékek:

$$P_{\text{macro}} = \frac{0{,}8571 + 0{,}6667 + 1}{3} \approx 0{,}8413,$$

$$R_{\text{macro}} = \frac{0{,}8571 + 1 + 0{,}5}{3} \approx 0{,}7857,$$

$$F1_{\text{macro}} = \frac{0{,}8571 + 0{,}8 + 0{,}6667}{3} \approx 0{,}7746.$$

Szélsőeset-ellenőrzés: a mátrix összege 15, a diagonál 12, a $\mathrm{FP}_2 = 0$ miatt a 2. osztály precisionje értelmesen 1; a $\mathrm{FN}_1 = 0$ miatt az 1. osztály recallja 1.

F3. A labor `metrics_check.py` függvénye a megoldás. `zero_division='omit'` és `'zero'` esetén egyaránt $F1_{\text{macro}} = 19/45 \approx 0{,}4222$ (a 2. osztály $F1_2 = 0$ definiált); a makro precision `omit` módban 7/12, `zero` módban 7/18. A `raise` negatív teszt:

```
C = np.array([[4,2,0],[1,3,0],[1,1,0]], dtype=np.int64)
try:
    precision_recall_from_confusion(C, zero_division="raise")
except ZeroDivisionError:
    print("raise mod helyesen hibazik")
else:
    raise AssertionError("nem dobott ZeroDivisionError-t")
```

A shape- és tartományellenőrzés a `confusion_matrix`-ban külön hibát ad, ha a címke kívül esik a $[0, K)$ intervallumon.

F4. A hiba a tengelyek felcserélése: a `fp`-t a sorösszegből, az `fn`-t az oszlopösszegből számolja. A `C_z` mátrixon a helyes $P = (2/3, 1/2, 0)$ és $R = (2/3, 3/4, 0)$; a hibás kód $P_{\text{bad}} = R$ és $R_{\text{bad}} = P$ értéket ad. A hibás makro átlagok ezért

$$P_{\text{macro,bad}} = \frac{0{,}6667 + 0{,}75 + 0}{3} \approx 0{,}4722, \qquad R_{\text{macro,bad}} = \frac{0{,}6667 + 0{,}5 + 0}{3} \approx 0{,}3889.$$

A hibás harmonikus képlet ezekből

$$F1_{\text{bad}} = \frac{2 \cdot 0{,}4722 \cdot 0{,}3889}{0{,}4722 + 0{,}3889} \approx 0{,}4266.$$

A fenti hibás részlet nyers NumPy-osztást használ, ezért a 2. osztály 0/0 recallja `NaN`-t ad (és figyelmeztetést ír ki), így a képlet makro $F1$-je is `NaN`; a 0,4722, 0,3889 és 0,4266 érték a feladat által előírt `zero_division='zero'` mód explicit alkalmazásával adódik. A javított kód a nevezőket külön kezeli:

```
C = np.array([[4, 2, 0], [1, 3, 0], [1, 1, 0]], dtype=np.int64)
tp = np.diag(C).astype(np.float64)
fp = C.sum(axis=0) - tp   # oszlop szerint: FP
fn = C.sum(axis=1) - tp   # sor szerint: FN
den_p, den_r = tp + fp, tp + fn
den_f = 2*tp + fp + fn
P = np.divide(tp, den_p, out=np.zeros_like(tp), where=den_p > 0)
R = np.divide(tp, den_r, out=np.zeros_like(tp), where=den_r > 0)
F1 = np.divide(2*tp, den_f, out=np.zeros_like(tp), where=den_f > 0)
print(P, R, F1.mean())   # P=[2/3 1/2 0], R=[2/3 3/4 0], atlag=0.4222
```

A helyes makro $F1$ `zero_division='zero'` mellett 0,4222. A felcserélt tengelyek önmagukban sem adnak helyes per-class jelentést, és a makro $F1$ harmonikus képlete további eltérést okoz.

F5. Egy lehetséges protokoll:

1. A befagyasztott modellt egyszer kiértékeljük a rögzített tesztre; a confusion matrixot és a tesztet a további modellválasztásban nem használjuk.
2. Kiszámítjuk R_i-t a mátrixból, majd az új prevalenciával az átsúlyozott accuracyt: $\mathrm{Acc}_{p'} = \sum_i p'_i R_i$. Ugyanezt elvégezzük a jelenlegi p_i prevalenciával is.
3. Összehasonlítjuk a mikro $F1$-et, a makro $F1$-et és az $\mathrm{Acc}_{p'}$ különbségét, és a bizonytalanságot jelenetalapú bootstrap (újramintavételes bizonytalanságbecslés) segítségével számítjuk; a mintavételi egység a `scene_id`, azaz a két nézetet és a belőlük származó képeket összekapcsoló jelenetazonosító, az 5.4. Függő képek, seedek és resampling szerint.
4. Lehetséges baseline: a prevalencia szerinti többségi konstans modell új accuracyja 0,6.

Számszerű szemléltetés. Legyen a befagyasztott modell confusion mátrixa a kiegyensúlyozott 20-20-20 tesztből

$$C = \begin{pmatrix} 18 & 1 & 1 \\ 2 & 16 & 2 \\ 5 & 5 & 10 \end{pmatrix},$$

így $R = (0{,}9; 0{,}8; 0{,}5)$, $\mathrm{Acc} = 44/60 = 0{,}7333$ és a makro $F1 = 0{,}7227$. A $p' = (0{,}6; 0{,}3; 0{,}1)$ prevalencián az átsúlyozott accuracy $\mathrm{Acc}_{p'} = 0{,}6 \cdot 0{,}9 + 0{,}3 \cdot 0{,}8 + 0{,}1 \cdot 0{,}5 = 0{,}83$, miközben a makro $F1$ a rögzített predikciók mellett változatlan 0,7227: a prevalence-változásra a súlyozott accuracy érzékeny, a makro $F1$ nem. A többségi konstans baseline $\mathrm{Acc}_{p'}$-je 0,6, tehát a modell 0,23 ponttal jobb nála.

Korlátok: (a) az átsúlyozás csak **label shiftet** (címkeeltolódást: kizárólag a $P(y)$ prevalencia változik) modellez: azt feltételezi, hogy az osztályonkénti bemeneti eloszlás $P(x \mid y)$ változatlan, és csak a $P(y)$ prevalencia tolódik el. A Bayes-szabály szerint ekkor $P(y \mid x)$ is megváltozik, éppen ezért lehet a régi modell predikciós mintázatából az új prevalenciára következtetni. Ha az osztályon belüli bemeneti eloszlás is eltolódik (kovariáns shift), a predikciós mátrix már nem reprezentálja a célkörnyezetet. (b) A metrikakülönbség nem bizonyítja, hogy a modell a telepített környezetben jobb vagy rosszabb; az igazi tesztet a célkörnyezetből, a 22.1. Adateredet, címke és csoportos split adatszerződése szerint kell gyűjteni.

Változó és egység: a prevalenciavektor; a megfigyelési egység a `scene_id`. **Hipotézis, döntési szabály és erőforrás:** a hipotézis, hogy a prevalencia-eltolódás az átsúlyozott accuracyt jobban megváltoztatja, mint a makro $F1$-et. A különbséget akkor tekintjük érdeminek, ha a jelenetszintű

bootstrap 95%-os CI-je kizárja a 0-t; a bootstrap seedje 20261020, $B = 2000$. A feladat nem tanít modellt, ezért architektúra és optimalizáló helyett a befagyasztott modell predikcióinak kontrollja és a döntési szabály szerepel. Erőforrás: CPU, a befagyasztott modell predikcióinak újraszámolása nélkül, néhány másodperc; a fenti 44/60-as mátrix nem mért modellkimenet, hanem a kézi példa számpéldája. ### Gyakori tévedés, önellenőrzés és visszautalás

Gyakori tévedés. Ugyanaz a metrikanevű szám eltérő átlagolási mód (micro vagy macro) mellett nem összehasonlítható. A C_z példán a micro $F1 = 0{,}5833$, a makro $F1$ 0,4222 (mindkét `zero_division` módban). Ha egy riport csak „F1 = 0,42” közlést tartalmaz, akkor sem a képlet, sem a nulla nevező kezelési módja nem reprodukálható. A közlésnek mindig tartalmaznia kell: K, a mikro vagy makro szót, a `zero_division` módot és a confusion matrixot vagy a per-class táblázatot. Hasonlóan hibás a makro $F1$-et a makro P és makro R harmonikus közepeként számolni; a micro/macro aggregálás ellenpéldájában a hibás harmonikus képlet 0,5219-et ad, a valódi makro $F1$ pedig 0,4222.

Önellenőrzés. A mátrix sorösszege az igaz support, oszlopösszege a predikciók száma; a főátló összege N-nel osztva az accuracy. A recall a soron belüli találati arány, a precizitás az oszlopon belüli helyesség. Ha a mátrixot transzponálja, a precizitás és a recall felcserélődik; ezért az ábra tengelyfelirata és a kód tengelye ugyanazt jelentse. A mikro $P = R = F1$ egyenlőség csak egycímkés osztályozásra érvényes; multilabel esetben (amikor egy minta több címkét is kaphat) külön összegzés kell.

Visszautalás. Az 5.4. Függő képek, seedek és resampling alfejezetben bevezetett függő képek most is érvényesek: a confusion matrix cellái képszintű darabszámok, de az intervallumbecsléshez a scene_id klaszter szerint kell resamplingolni; ez a 23.3. Páros összehasonlítás és csoportszintű intervallum témája. A 8.4. II. ellenőrző pont: korrekt tanulási állítás tanulási szerződése szerint a metrikát a modellválasztás után, független halmazon számítjuk; a labor riportja sem validálhat újra ugyanazon a halmazon, amelyen a checkpointot választottuk. A 11.1. Tensor és autograd-életciklus `inference_mode`-ja csak azt biztosítja, hogy a riportkészítés ne építsen autograd-gráfot; önmagában nem kapcsol eval módba, és train módban a BatchNorm futóstatisztikái továbbra is frissülnének. A riportkészítés ezért `model.eval()` mellett, `torch.inference_mode()` alatt fut.

23.1.8 Rövid összefoglaló

A confusion matrix C_{ij} darabszámokkal rögzíti, hányszor mondtunk j-t, amikor az igazság i. Az accuracy a főátló összege osztva N-nel, és egyenlő a prevalence-súlyozott recall-lal. A one-vs-rest precizitás és recall osztályonként definiálja a találati és a lefedettségi hibát; nulla nevezőre a `raise`, `zero` és `omit` módok egyikét kell közölni. A mikro összegzés egycímkés esetben $P = R = F1 = \mathrm{Acc}$; a makro minden osztályt egyenlően súlyoz, és a makro $F1$ a per-class $F1$-ek átlaga, nem a makro P és R harmonikus közepe. Kiegyensúlyozatlan adaton a mikro és makro szám lényegesen eltérhet, ezért az összehasonlításhoz a teljes konvenciót rögzíteni kell.

23.1.9 Következő lecke és források

A 23.2. Kalibráció és döntési küszöb alfejezetben a softmax-kimenet, a döntési küszöb és a kalibráció következik: ott a confusion matrix celláinak folytonos küszöbtől való függése adja a precision–recall trade-off elemzését. A 23.3. Páros összehasonlítás és csoportszintű intervallum ehhez csoportszintű konfidenciaintervallumot kapcsol.

- [D15] scikit-learn developers (online). *Model evaluation: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html.

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.

23.2 Kalibráció és döntési küszöb

23.2.1 Cél, előismeret és induló kérdés

A 23.1. Confusion matrix és aggregálási konvenció egységben a confusion matrixból kiindulva az accuracy, precision és recall fogalmát építettük fel. Ott a döntési lépés adott volt: az `argmax` kiválasztja a legnagyobb logithoz tartozó osztályt. Most a döntés *előtt* álló valószínűségi kimenetet vizsgáljuk: ha a modell azt mondja, hogy egy kép 0,9 valószínűséggel `vertical`, két különböző kérdést tehetünk fel. *Osztályonként:* azok között a képek között, amelyekre a `vertical` osztály valószínűsége 0,9, tényleg kb. 90% a `vertical`? *A döntés konfidenciájára (top-label):* azok között a döntések között, amelyek legnagyobb valószínűsége 0,9 (bármelyik osztályra), kb. 90% helyes? A kettő nem ugyanaz, és mindkettő ahhoz kell, hogy a 0,9 értéket valódi valószínűségként használhassuk. Ez a **kalibráció** kérdése, és szorosan kapcsolódik ahhoz, hogyan állítunk be egy döntési küszöböt egy költségérzékeny alkalmazásban.

Előismeretként a 23.1. Confusion matrix és aggregálási konvenció jelölését használjuk: $Z \in \mathbb{R}^{N\times K}$ logitmátrix, $y \in \{0, \dots, K-1\}^N$ osztályindex-vektor. A [D07] dokumentáció szerint a PyTorch `CrossEntropyLoss` logits bemenetet és `int64` célindexet vár, a softmaxot pedig maga a loss számolja; mi itt a loss definíciójához a softmax-kimenetet explicite is felírjuk. A 11.2. nn.Module és paraméterregisztráció tanulsága, hogy a modell `eval()` módban, frissítés nélkül adja a kiértékeléshez használt logitokat, és a 22.2. Címkemegőrző transzformációk szerződése szerint a kalibráló halmaznak ugyanazon a preprocessingen kell átmennie, mint a tanító halmaznak.

Induló kérdés. Egy háromosztályos GlyphLab-modell (a függelék A. szakasza: 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték) a validációs halmazon minden mintához $p_{\max} = 0{,}8$ körüli maximális softmax-értéket rendel. Ebből következik-e, hogy az ilyen minták 80%-ában helyes az argmax-döntés? Ha nem, mit kell mérnünk, és hogyan javítható a hiba a logitok megváltoztatása nélkül?

23.2.2 Definíciók, alakok és intuíció

Legyen $Z \in \mathbb{R}^{N\times K}$ a modell által adott logitmátrix. A softmax-kimenet

$$p_{n,k} = \frac{\exp(z_{n,k})}{\sum_{j=0}^{K-1} \exp(z_{n,j})}, \qquad p \in [0;1]^{N\times K}, \quad \sum_k p_{n,k} = 1.$$

A döntés $\hat{y}_n = \arg\max_k z_{n,k} = \arg\max_k p_{n,k}$, mert a softmax rendezéstartó (rank-preserving: a logitok sorrendjét megtartja). A döntéshez tartozó **konfidencia** $c_n = \max_k p_{n,k}$, a helyesség pedig $a_n = \mathbb{1}[\hat{y}_n = y_n]$.

Definíció (top-label kalibráció). Legyen (X, Y) a populációs eloszlásból vett pár, $\hat{Y}$ a modell döntése és $C = \max_k p_k(X)$ a konfidenciája. A modell **top-label kalibrált**, ha

$$P(Y = \hat{Y} \,|\, C = c) = c$$

minden $c \in [0;1]$-re (a C eloszlása szerint majdnem mindenütt). Ez a **konfidencia-kalibráció** egy elterjedt gyengébb alakja; a top-label kalibráció erősebb, gyakori definíciója a *prediktált címkére is kondicionál*: $P(Y = \hat{Y} \mid C = c,\ \hat{Y} = k) = c$ minden k-ra. A szakirodalom a top-label változatot gyakran egyszerűen konfidenciakalibrációnak nevezi; ebben a könyvben következetesen a **top-label** elnevezést használjuk, hogy ne keveredjék az alábbi osztályonkénti fogalommal.

Definíció (osztályonkénti kalibráció). A modell a k osztályra kalibrált, ha $P(Y = k \,\big|\, p_k(X) = c) = c$ minden c-re. Ez más és erősebb állítás: a „0,9 valószínűséggel `vertical`" mondat a `vertical` osztályra vonatkozik. Az osztályonkénti (classwise) kalibráció nem következik automatikusan a konfidencia- vagy a top-label kalibrációból többosztályos esetben, és fordítva sem: az egyes osztályok külön-külön lehetnek kalibrálatlanok akkor is, ha a top-label kalibráció teljesül. A pontos definíciót és az implikációkat mindig külön ellenőrizni kell.

Mindkettő **populációs, feltételes valószínűségi** tulajdonság. Véges mintán még egy tökéletesen kalibrált modellnél sem kell a megfigyelt gyakoriságnak pontosan egyeznie a konfidenciával, és a feltételt folytonos c-re közvetlenül nem is lehet mérni. Ezért binelt **diagnosztikát** használunk: a $[0;1]$ intervallumot $B_1, \dots, B_M$ diszjunkt binekre osztjuk, és binenként összevetjük az empirikus pontosságot és az átlagos konfidenciát:

$$\frac{1}{|B_m|} \sum_{n : c_n \in B_m} a_n \quad \text{vs.} \quad \frac{1}{|B_m|} \sum_{n : c_n \in B_m} c_n,$$

ahol a nevezőben a bin elemszáma áll, és az üres bineket kihagyjuk. Ez a **reliability** diagram alapja. A binenkénti eltérés becslés: kis binben nagy a mintavételi szórása, a binelés pedig egy binen belül elrejtheti az ellentétes irányú eltéréseket (az egyik részen túl-, a másikon alulbiztos modell binátlaga lehet pontos). A binenkénti egyezés ezért nem a kalibráció definíciója, csak annak zajos, nem elégséges jele.

Log loss. A többosztályos negatív log-likelihood, más néven log loss:

$$L_{\log} = -\frac{1}{N} \sum_{n=0}^{N-1} \log p_{n,y_n}.$$

A [D07] szerint ez éppen a PyTorch `CrossEntropyLoss` default `reduction="mean"` viselkedése, ha a bemenet a logitmátrix és a cél az `int64` osztályindex. (A `reduction` argumentum azt szabályozza, hogy a per-minta veszteségeket átlagoljuk, összegezzük vagy egyenként adjuk vissza.) A log loss a teljes valószínűségvektort bünteti, ezért a kalibrációt is méri: ha a modell magabiztosan téved, a büntetés nagy.

Megjegyzés. A log loss (NLL) és a Brier-pontszám szigorúan helyes pontozási szabályok (proper scoring rule; [D15]): várható értékük akkor minimális, ha a jelentett eloszlás a valódi, ezért a diszkriminációt és a felbontást is tükrözik, és az alacsonyabb NLL önmagában nem bizonyít jobb kalibrációt.

Brier-pontszám (többosztályos). Legyen $Y \in \{0;1\}^{N \times K}$ a one-hot célmátrix. A többosztályos Brier-pontszám

$$\mathrm{BS} = \frac{1}{N} \sum_{n=0}^{N-1} \sum_{k=0}^{K-1} \left(p_{n,k} - Y_{n,k}\right)^2.$$

A tökéletes modellnél $\mathrm{BS} = 0$; a legrosszabb lehetséges érték 2, mert ha a modell egyetlen rossz osztályra ad 1 valószínűséget, akkor $(1-0)^2 + (0-1)^2 = 2$. (A bináris esetre a scikit-learn

`brier_score_loss` a $[0;1]$ értékkészletet dokumentálja; a fenti többosztályos általánosítás saját definíció, amelynek határait az előző mondat levezetése adja.)

Reliability diagram és ECE. A binelt konfidencia–pontosság párokat diagramon ábrázoljuk: minden binhez az átlagos konfidencia és a megfigyelt pontosság tartozik. Jelölje B_m az m-edik bin intervallumát és $I_m = \{n : c_n \in B_m\}$ a beleeső minták indexhalmazát; a kettő külön objektum, ezért a számosságot mindig $|I_m|$ jelöli. Az **expected calibration error** a top-label kalibrációs eltérés binelt, mintavételi bizonytalansággal terhelt becslése:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|I_m|}{N} \left| \frac{1}{|I_m|} \sum_{n \in I_m} c_n - \frac{1}{|I_m|} \sum_{n \in I_m} a_n \right|.$$

A bineket például $[0;0{,}2), [0{,}2;0{,}4), \ldots, [0{,}8;1{,}0]$ módon vehetjük fel. A binek száma és határai a mérési protokoll részei; más binelés más ECE-t ad.

Döntési küszöb és költség. Bináris vagy multilabel esetben (amikor egy mintához több, egymást nem kizáró bináris címke tartozhat) a döntési szabály $\hat{y} = \mathbb{1}[p \geq \tau]$. A τ küszöböt gyakran költségmátrixszal indokoljuk. Ha a téves pozitív költsége C_{FP}, a téves negatívé C_{FN}, a helyes döntéseké 0, akkor az 1-osztály választásának várható költsége $(1-p)C_{FP}$, a 0-osztályé pC_{FN}. Az 1-es döntés optimális, ha

$$(1-p)C_{FP} \leq pC_{FN} \quad \Longleftrightarrow \quad p \geq \frac{C_{FP}}{C_{FP} + C_{FN}},$$

tehát az optimális küszöb $\tau^* = C_{FP}/(C_{FP} + C_{FN})$. Ez a levezetés feltételezi, hogy a p értéke kalibrált valószínűség; ha a modell nem kalibrált, akkor a költségküszöb nem az elvárt gyakoriságot adja.

23.2.3 Levezetés: skalár hőmérséklet a logitokban és validációs illesztés

A **temperature scaling** a logitokat egyetlen pozitív skalárral osztja, majd softmaxot számol. A hőmérséklettel skálázott valószínűség:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_{j=0}^{K-1} \exp(z_j/T)}, \qquad T > 0.$$

Lemma (a hőmérséklet hatása). Legyen $z \in \mathbb{R}^K$ véges logitvektor, $p(T)$ a fenti eloszlás. Ekkor:

1. A $p(T)$ rendezése minden $T > 0$-ra azonos: $z_i > z_j$ pontosan akkor, ha $p_i(T) > p_j(T)$.
2. $\lim_{T \to \infty} p_i(T) = 1/K$ minden i-re.
3. Ha $z_{i^*} = \max_j z_j$ egyértelmű maximum, akkor $\lim_{T \to 0^+} p_{i^*}(T) = 1$ és $p_i(T) \to 0$ minden $i \neq i^*$-ra. Ha r darab maximális logit van, akkor a maximális osztályok mindegyikének határértéke $1/r$, a többié 0.

Bizonyítás. Az 1. állításhoz: $T > 0$ miatt a z_i/T leképezés szigorúan rendezéstartó, és az exponenciális függvény is szigorúan monoton; a pozitív normalizáló konstans nem változtat a sorrenden.

A 2. állításhoz: z_i véges, ezért $T \to \infty$ esetén $z_i/T \to 0$, így $\exp(z_i/T) \to 1$ minden i-re, tehát

$$\lim_{T\to\infty} p_i(T) = \frac{1}{\sum_{j=0}^{K-1} 1} = \frac{1}{K}.$$

A 3. állításhoz legyen $m = \max_j z_j$, és írjuk át:

$$p_i(T) = \frac{\exp((z_i - m)/T)}{\sum_{j=0}^{K-1} \exp((z_j - m)/T)}.$$

Ha $z_i < m$, akkor $z_i - m < 0$, ezért $(z_i - m)/T \to -\infty$ és $\exp((z_i - m)/T) \to 0$. Ha $z_i = m$, akkor a kitevő azonosan 0, tehát $\exp(0) = 1$. Így a nevező határértéke az m értékű osztályok száma, r, a számláló pedig 1 a maximális, 0 a nem maximális osztályokra. Ebből a harmadik állítás következik. □

A $T = 0$ tehát kívül esik a paramétertartományon; a $T \to 0^+$ határérték a *kemény* argmax-eloszlás. A $T = 1$ az identitás, $T < 1$ élesíti, $T > 1$ lapítja az eloszlást.

A validációs illesztés. A modell súlyait rögzítjük, és csak a T skalárt hangoljuk a validációs halmazon a log loss minimalizálásával. Legyen a validációs logitmátrix $Z^{val} \in \mathbb{R}^{N_{val}\times K}$ és a cél y^{val}. A célfüggvény

$$L_{val}(T) = -\frac{1}{N_{val}} \sum_{n=0}^{N_{val}-1} \log p_{n,y_n^{val}}(T), \qquad T > 0.$$

Az optimalizálást a T szerinti gradienssel is elvégezhetjük. Egyetlen mintára $L_n(T) = -\log p_y(T)$, ahol $p_y(T) = \exp(z_y/T)/Z(T)$ és $Z(T) = \sum_j \exp(z_j/T)$. A logaritmus deriváltja:

$$\frac{\partial}{\partial T}\log p_y(T) = \frac{\partial}{\partial T}\Big(\frac{z_y}{T} - \log Z(T)\Big) = -\frac{z_y}{T^2} - \frac{1}{Z(T)}\sum_j \exp(z_j/T)\Big(-\frac{z_j}{T^2}\Big) = \frac{\sum_j p_j(T) z_j - z_y}{T^2}.$$

A negatív előjel miatt a batchra átlagolt gradiens

$$\frac{\partial L_{val}}{\partial T} = \frac{1}{N_{val}} \sum_{n=0}^{N_{val}-1} \frac{z_{n,y_n} - \sum_j p_{n,j}(T) z_{n,j}}{T^2}.$$

A gradiens $T > 0$-n folytonos, de ebből **nem következik** numerikus stabilitás: $T \to 0^+$ esetén a $1/T^2$ tényező nagy lehet. A gyakorlatban a `CrossEntropyLoss` logitsbemenetét használjuk, és a pozitivitás biztosítására kereshetünk $u = \log T$ paraméteren vagy korlátozott, pozitív rácson. A levezetés alakellenőrzése: L skalár, a gradiens is skalár; a jobb oldal logit mértékegységű, osztva T^2-tel, tehát a gradiens dimenziója logit/(hőmérséklet^2), ahogy a láncszabályból vártuk.

Oktatási kiindulópontként egy kis rács, például $T \in \{0{,}25; 0{,}5; 0{,}75; 1{,}0; 1{,}25; 1{,}5; 2{,}0\}$ használható, de a rács nem garantálja az optimum pontos megtalálását; a keresés semmilyen modellparamétert nem érint. A választott T^* nem változtatja az argmax-döntést, tehát az accuracy, a confusion matrix és a belőle számolt macro/micro metrikák változatlanok maradnak. Amit a hőmérséklet megváltoztat, az a log loss, a Brier-pontszám, az ECE és minden küszöb-alapú döntés konfidenciaértéke.

Heurisztika, nem tétel. A temperature scalinget Guo és munkatársai (2017) javasolták modern konvolúciós hálók kalibrálására; gyakran csökkenti az ECE-t, de ez empirikus megfigyelés,

nem bizonyított garancia minden modellosztályra. A validációs optimum kiválasztása maga is mintavételi bizonytalanságnak kitett; a mért ECE-csökkenést ezért a validációs elemszám ismeretében, nem pedig egyetlen pontbecslésként kell kezelni.

23.2.4 Eredeti kézi példa és minimális kód

Három minta, $K = 3$, célok $y = [0; 1; 1]$, logitok:

$$z^{(0)} = [2{,}0; 0{,}5; -0{,}5], \quad z^{(1)} = [0{,}2; 1{,}8; -0{,}3], \quad z^{(2)} = [2{,}5; 1{,}8; 0{,}2].$$

A $T = 1$ softmaxok rendre:

$$p^{(0)} \approx [0{,}766; 0{,}171; 0{,}063], \quad p^{(1)} \approx [0{,}152; 0{,}755; 0{,}092], \quad p^{(2)} \approx [0{,}626; 0{,}311; 0{,}063].$$

A log loss értékei: $-\log 0{,}766 \approx 0{,}266$, $-\log 0{,}755 \approx 0{,}281$, $-\log 0{,}311 \approx 1{,}168$, az átlag $L_{val}(1) \approx 0{,}572$. A harmadik minta magabiztos *tévedés*: a modell a 0 osztályt adja, pedig a cél 1.

Most $T = 0{,}5$: $z^{(2)}/T = [5{,}0; 3{,}6; 0{,}4]$, így $p^{(2)} \approx [0{,}796; 0{,}196; 0{,}008]$, és a harmadik minta vesztesége $-\log 0{,}196 \approx 1{,}628$. Az első két mintán a loss javul (0,055 és 0,054), de az átlag $L_{val}(0{,}5) \approx (0{,}055 + 0{,}054 + 1{,}628)/3 \approx 0{,}5792$. A teljes rácson azonban a minimum nem itt van: $T = 0{,}75$-nél $L_{val}(0{,}75) \approx 0{,}5402$ (a harmadik minta vesztesége 1,298), ami kisebb, mint $L_{val}(1) \approx 0{,}5718$. A $T = 1$ csak a szűkített $\{0{,}5; 1; 2\}$ rácson tűnik optimálisnak; a 0,75 hozzáadásával a rácsminimum $T = 0{,}75$. A példa mutatja, hogy a kalibráló optimum nem automatikusan a legkisebb hőmérséklet, és a rácsfelbontás befolyásolja a választott értéket.

Minimális ellenőrző kód float64 pontossággal:

```
import numpy as np

Z = np.array([[2.0,0.5,-0.5],[0.2,1.8,-0.3],[2.5,1.8,0.2]], dtype=np.float64)
y = np.array([0,1,1], dtype=np.int64)

def softmax(Z, T):
    U = Z / T
    U = U - U.max(axis=1, keepdims=True)  # stabil logiteltolás
    E = np.exp(U)
    return E / E.sum(axis=1, keepdims=True)

for T in [0.5, 0.75, 1.0, 1.25, 1.5, 2.0]:
    P = softmax(Z, T)
    loss = -np.mean(np.log(P[np.arange(len(y)), y]))
    print(f"T={T:4.2f}  loss={loss:.6f}  P[2]={P[2].round(3)}")
# MÉRT ÉRTÉK (Python 3.12.3, NumPy 2.5.4, CPU, a fenti kód futtatásával):
# T=0.50  loss=0.579231  P[2]=[0.796 0.196 0.008]
# T=0.75  loss=0.540199  P[2]=[0.695 0.273 0.032]
# T=1.00  loss=0.571774  P[2]=[0.626 0.311 0.063]
# T=1.25  loss=0.617273  P[2]=[0.578 0.33  0.092]
# T=1.50  loss=0.661606  P[2]=[0.543 0.34  0.117]
# T=2.00  loss=0.735268  P[2]=[0.495 0.349 0.157]
```

A kimenet a fenti kód tényleges futtatásából származó mért érték; a laborbeli, teljes GlyphLab-protokoll eredményét a labor közli.

23.2.5 Labor: validációs kalibrációs terv és érintetlen teszt

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges a 20.4. V. ellenőrző pont: indokolt CNN-választás szerint tanított CNN mentett `state_dict`-je (`--weights`) és a GlyphLab `data/` mappája. A futtatás a szakaszban megadott szkripttel történik; várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A labor a GlyphLab adatszerződésére épül. A protokoll rögzített, a teszt csak a legvégén, egyszer kerül kiértékelésre.

1. Betöltjük a 20.4. V. ellenőrző pont: indokolt CNN-választás szerint tanított CNN `state_dict`-jét és a GlyphLab `metadata.csv` split oszlopát; a 24.4. VI. ellenőrző pont: megbízható CNN-training a tanítási protokollt részletezi, a kalibrációhoz azonban elegendő a 20.4-es ellenőrző pont modellje, ezért ez az egység nem függ a későbbi fejezettől.
2. A validációs 60 képre eval módban logitokat számolunk; a képeken nincs augmentáció, a preprocessing azonos a tanításival.
3. A T rácsot $[0{,}5; 0{,}75; 1{,}0; 1{,}25; 1{,}5; 2{,}0]$ módon állítjuk be, és minden T-re kiszámítjuk a validációs log losst a Z^{val}/T logitokon.
4. Kiválasztjuk T^*-ot, amely a validációs log losst minimalizálja.
5. Csak ezután számolunk teszt logitokat, és T^*-gal kiértékeljük a teszt log losst, Brier-pontszámot, ECE-t és accuracy-t.

```
# futtatasi parancs: python n090_lab.py --data data --weights model.pt
import argparse
import csv
import numpy as np
import torch
from torch import nn

torch.manual_seed(20261020)
torch.set_num_threads(1)
device = torch.device("cpu")

class SmallCNN(nn.Module):
    # a lab/train_cnn.py referencia-modellje (1299 parameter)
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.AdaptiveAvgPool2d(1))
        self.classifier = nn.Linear(16, 3)

    def forward(self, x):
        return self.classifier(self.features(x).flatten(1))

def gather_logits(model, X, batch=32):
    model.eval()
    outs = []
    with torch.inference_mode():
        for i in range(0, len(X), batch):
            xb = torch.from_numpy(X[i:i+batch]).to(dtype=torch.float32, device=device)
            outs.append(model(xb).cpu())
    return torch.cat(outs, dim=0)

def nll_from_logits(Z, y):
    # D07: CrossEntropyLoss logitsot var, softmax nelkul; int64 cel.
    return torch.nn.functional.cross_entropy(Z, y).item()

def ece(Z, y, bins=(0.0, 0.2, 0.4, 0.6, 0.8, 1.0)):
```

```
    p = torch.softmax(Z, dim=1)
    conf, pred = p.max(dim=1)
    acc = (pred == y).float()
    ece_val = 0.0
    for lo, hi in zip(bins[:-1], bins[1:]):
        mask = (conf >= lo) & (conf < hi + (1e-9 if hi == 1.0 else 0.0))
        n = mask.sum().item()
        if n > 0:
            ece_val += (n/len(y)) * abs(conf[mask].mean().item() - acc[mask].mean().item())
    return ece_val

def brier(Z, y):
    p = torch.softmax(Z, dim=1)
    onehot = torch.nn.functional.one_hot(y, num_classes=p.shape[1]).float()
    return float(((p - onehot) ** 2).sum(dim=1).mean())

def accuracy(Z, y):
    return float((Z.argmax(dim=1) == y).float().mean())

def main(data, weights):
    images = np.load(f"{data}/images.npy").astype(np.float32)
    labels = np.load(f"{data}/labels.npy").astype(np.int64)
    with open(f"{data}/metadata.csv", newline="", encoding="utf-8") as f:
        metadata = list(csv.DictReader(f))
    val_idx = np.array([int(r["index"]) for r in metadata if r["split"] == "val"])
    test_idx = np.array([int(r["index"]) for r in metadata if r["split"] == "test"])
    assert len(val_idx) == len(test_idx) == 60
    model = SmallCNN()
    model.load_state_dict(torch.load(weights, map_location=device, weights_only=True))
    model.to(device).eval()
    Z_val = gather_logits(model, images[val_idx])
    y_val = torch.from_numpy(labels[val_idx])
    grid = [0.5, 0.75, 1.0, 1.25, 1.5, 2.0]
    nll_val = [nll_from_logits(Z_val / T, y_val) for T in grid]
    print("val NLL: T=" + " | ".join(f"{T:4.2f} {l:.6f}" for T, l in zip(grid, nll_val)))
    T_star = grid[int(np.argmin(nll_val))]
    print(f"T* = {T_star:.2f}")
    Z_test = gather_logits(model, images[test_idx])
    y_test = torch.from_numpy(labels[test_idx])
    print(f"test NLL: {nll_from_logits(Z_test, y_test):.6f} (T=1) -> "
          f"{nll_from_logits(Z_test / T_star, y_test):.6f} (T*={T_star:.2f})")
    print(f"Brier:    {brier(Z_test, y_test):.6f} (T=1) -> "
          f"{brier(Z_test / T_star, y_test):.6f} (T*={T_star:.2f})")
    print(f"ECE:      {ece(Z_test, y_test):.6f} (T=1) -> "
          f"{ece(Z_test / T_star, y_test):.6f} (T*={T_star:.2f})")
    print(f"accuracy: {accuracy(Z_test, y_test):.6f} (T=1) -> "
          f"{accuracy(Z_test / T_star, y_test):.6f} (T*={T_star:.2f})")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", default="data")
    parser.add_argument("--weights", default="model.pt")
    args = parser.parse_args()
    main(args.data, args.weights)
```

A labor elvárt eredménye a validációs kalibrációs protokoll és a tesztkiértékelés. A fenti szkript a teljes protokollt tartalmazza: a `--weights` a betöltött `SmallCNN` `state_dict`-je, a `--data` a GlyphLab `data/` könyvtára, a val/test indexeket pedig a `metadata.csv` split oszlopa adja (60-60 kép). A szkriptet a könyv környezetében a `lab/train_cnn.py` referencia-modelljével futtattuk (1299 paraméter, seed 20261020, 16 epoch, CPU, 1 szál; a checkpointot a `train_cnn.py --output` mentette). A mért eredmény:

```
val NLL: T=0.50 0.418565 | 0.75 0.520070 | 1.00 0.595969 | 1.25 0.655495 | 1.50 0.703344 | 2.00
↪  0.774959
T* = 0.50
test NLL: 0.600479 (T=1) -> 0.425148 (T*=0.50)
Brier:    0.333699 (T=1) -> 0.235603 (T*=0.50)
ECE:      0.372069 (T=1) -> 0.262117 (T*=0.50)
accuracy: 0.950000 (T=1) -> 0.950000 (T*=0.50)
```

A naplóba a T^*, a val és teszt log loss, a Brier, az ECE és az accuracy kerül, a binhatárokkal együtt. A példa tanulsága, hogy a GlyphLab referencia-modellje alulkonfidens: a hőmérséklet csökkentése javítja a log loss-t, a Brier-t és az ECE-t, miközben az accuracy változatlan. A $T^* = 0{,}5$ a rács szélén van, ezért a valódi optimum a rácson kívül is lehet; ezt a következtetés korlátjaként kell kezelni.

Eredményértelmezés. Ha a teszt log loss T^*-gal kisebb, mint $T = 1$-gyel, akkor a hőmérséklet-illesztés a validációról a tesztre általánosult. Ha az ECE is csökken, a kalibráció javult. A kísérleti állítás ezzel még mindig a GlyphLab három osztályára és a konkrét modellre vonatkozik, nem általános CNN-kalibrációs tétel.

23.2.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Definiálja a top-label kalibrációt a binek nyelvén. Adjon ellenpéldát arra, hogy egy modell kalibrált, de haszontalan, és arra, hogy egy modell pontos, de nem kalibrált. Adja meg mindkét esetben a logitokat vagy valószínűségeket.

F2 (levezetési, nehézség 2). Legyen $z = [2; 2; 1]$, $K = 3$. Számítsa ki a $p(T)$ eloszlást $T = 1$ és $T = 0{,}5$ esetén, és vezesse le a $T \to 0^+$ határértéket. Igazolja, hogy a rendezés minden $T > 0$-ra változatlan.

F3 (implementációs, nehézség 2). Írjon teljes Python/PyTorch függvényt, amely adott $Z \in \mathbb{R}^{N \times K}$ logitmátrixon, $y \in \{0, \dots, K-1\}^N$ célvektoron és T-rácson visszaadja a legjobb T^*-ot a log loss minimalizálásával. A függvény ne használjon külső adatot; a bemeneteket ellenőrizze. Tesztelje a kézi példa hárommintás esetén, és adja meg az elvárt T^*-ot.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a kalibráló hőmérsékletet a *teszt* logitokon választja ki:

```python
def bad_temperature(Z_test, y_test):
    best = None
    for T in [0.5, 0.75, 1.0, 1.25, 1.5, 2.0]:
        loss = torch.nn.functional.cross_entropy(Z_test/T, y_test).item()
        if best is None or loss < best[0]:
            best = (loss, T)
    return best[1]
```

Mi a hiba, mi a következménye a tudományos állításra nézve, és hogyan kell javítani? Visszaellenőrzésként adja meg, melyik halmaznak kell megmaradnia érintetlennek.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet annak eldöntésére, hogy a temperature scaling javítja-e egy CNN log lossát egy új, kétosztályos feladaton, ahol a pozitív osztály ritka

(prevalencia, azaz előfordulási arány 10%). Adja meg a protokollt, a baseline-t, a metrikákat, a megfigyelési egységet, a szükséges split-felépítést és a következtetési korlátot. Indokolja, miért nem elég az accuracy, és miért kell a kalibráló halmazt a threshold-választástól is elkülöníteni.

23.2.7 Megoldások és indoklás

F1. A top-label kalibráció definíciója a definíciók között szerepel: $P(Y = \hat{Y} \mid C = c) = c$; a binelt diagnosztikában ennek jele, hogy a binekben az átlagos konfidencia közel van a megfigyelt pontossághoz. *Kalibrált, de haszontalan*: egyenlő osztálygyakoriságú (1/3–1/3–1/3) háromosztályos feladaton a konstans $p = [1/3; 1/3; 1/3]$ modell minden mintára. A konfidencia $1/3$ mindenütt, a pontosság is $1/3$, tehát kalibrált, de a döntése véletlenszerű. (Egyenlőtlen prioroknál ugyanez a konstans modell nem kalibrált: a pontosság a legnagyobb priorral egyezne meg, nem 1/3-dal.) *Pontos, de nem kalibrált*: kétosztályos, 100 mintás feladaton a modell 90 mintát helyesen osztályoz, de a 90 helyes mintához $p = (0{,}99; 0{,}01)$, a 10 téveshez $p = (0{,}51; 0{,}49)$ valószínűséget ad. Az accuracy 0,9, de a 0,99 konfidenciájú binben a pontosság 1,0, a 0,51-es binben 0,0; a modell nem kalibrált. A példa szándékosan szélsőséges, hogy a két fogalom elváljon.

F2. $z = [2; 2; 1]$. $T = 1$: $\exp(2) = 7{,}389$, $\exp(2) = 7{,}389$, $\exp(1) = 2{,}718$, összeg 17,496, tehát $p = [0{,}422; 0{,}422; 0{,}155]$. $T = 0{,}5$: $z/T = [4; 4; 2]$, $\exp(4) = 54{,}598$ kétszer, $\exp(2) = 7{,}389$, összeg 116,585, tehát $p = [0{,}468; 0{,}468; 0{,}063]$. A rendezés változatlan: a két maximális logitú osztály egyenlő valószínűségű, a harmadik kisebb. A $T \to 0^+$ határérték: a lemmában szereplő átírással $m = 2$, a két maximális osztály kitevője 0, a harmadiké $(1 - 2)/T \to -\infty$, így a határeloszlás $[1/2; 1/2; 0]$. A rendezés invarianciája onnan következik, hogy $T > 0$-val osztás és az exponenciális függvény is szigorúan monoton; a normalizáló konstans pozitív.

F3. A függvény PyTorchban:

```
import torch

def fit_temperature(Z: torch.Tensor, y: torch.Tensor,
                    grid=[0.5, 0.75, 1.0, 1.25, 1.5, 2.0]) -> float:
    if Z.ndim != 2 or y.ndim != 1 or len(Z) != len(y):
        raise ValueError("Z legyen (N,K), y legyen (N,)")
    if torch.any(torch.tensor(grid) <= 0):
        raise ValueError("A hőmérsékletrács elemei legyenek pozitívak")
    Z = Z.to(dtype=torch.float64)
    y = y.to(dtype=torch.int64)
    best_T, best_loss = None, float("inf")
    for T in grid:
        loss = torch.nn.functional.cross_entropy(Z/T, y).item()
        if loss < best_loss:
            best_loss, best_T = loss, T
    return best_T

Z = torch.tensor([[2.0,0.5,-0.5],[0.2,1.8,-0.3],[2.5,1.8,0.2]], dtype=torch.float64)
y = torch.tensor([0,1,1], dtype=torch.int64)
print(fit_temperature(Z, y))  # mért: 0.75, a kézi példa teljes rácsú számítása alapján
```

Alakellenőrzés: Z/T alakja (N, K), a célé $(N,)$; a `cross_entropy` default mean redukció. A rácsfelbontás csak korlátozottan ad hibabecslést: ha az optimum a rács tartományán belül van és $L_{val}(T)$ a tartományon unimodális, akkor a legközelebbi rácspont legfeljebb fél lépésközre van az optimumtól (0,05-ös lépésköznél legfeljebb 0,025-ra); ha a minimum a rács szélén áll, mint a GlyphLab-kísérletben $T^* = 0{,}5$ esetén, a valódi optimum a tartományon kívül is lehet, ezért a rácsminimum nem korlátozza a hibát. A függvény `float64` bemenettel számol, mert a loss értéke kis különbségekre érzékeny.

F4. A hiba: a hőmérsékletet a teszt logitokon választja. A következmény: a teszt már nem érintetlen; a T^* kiválasztása a teszt loss minimalizálásával teszt-adatszivárgás, és a jelentett teszt log loss optimistán torzított lesz. A javítás: a T-rácsot kizárólag a validációs logitokon értékeljük, majd a kiválasztott T^*-ot egyszer alkalmazzuk a tesztre. A visszaellenőrzés: a teszt halmaz semmilyen választási lépésben nem vett részt, tehát a teszt loss torzítatlan becslés marad.

F5. Protokoll: rögzített scene-szintű train/val/test split (a scene, azaz jelenet a generált objektum két nézetét és a belőlük származó képeket fogja össze), a pozitív osztály aránya mindegyikben 10%. A modell betanítása után a validációs logitokon T^* kiválasztása log loss szerint. Baseline: $T = 1$, azaz kalibrálás nélküli modell. Metrikák: log loss, Brier-pontszám, ECE, valamint a ritka osztályra recall és precision a $C_{FP} = 1$, $C_{FN} = 5$ költségküszöbbel, azaz $\tau^* = C_{FP}/(C_{FP} + C_{FN}) = 1/6 \approx 0{,}1667$ mellett. Az accuracy a 90%-os prevalencia miatt csak a többségi osztályt tükrözné, ezért önmagában nem elég. Megfigyelési egység: scene, mert a két nézet nem független. A küszöböt a validációtól is el kell különíteni: ha a küszöböt ugyanazon a validáción hangolnánk, mint a T-t, akkor a két választás kölcsönhatása miatt a teszt következtetés ismét torzulhat; célszerű a validációt két részre bontani, vagy a küszöböt a feladat kiírása előtt, költségarányokból rögzíteni. Korlát: a ritka osztály 10%-os prevalenciája miatt a binelt ECE-ben a magas konfidenciájú binek elemszáma kicsi, ezért a becslés szórása nagy.

Hipotézis: a temperature scaling csökkenti a teszt log lossát a $T = 1$ baseline-hoz képest; a változó tényező a hőmérséklet T. **Architektúra, recept, seedek és erőforrás:** a kalibrálandó modell a GlyphLab `SmallCNN` (1299 paraméter), a tanítási recept AdamW, ‘lr=10^{-3}$, 16 epoch és 16-os batch, öt seed (20261020–20261024); a T^* kiválasztása a validációs log loss minimuma, és a kalibrációt minden seedre külön jelentjük. A döntési szabály számszerű: a T^* javulását akkor tekintjük igazoltnak, ha a teszt log loss különbségének jelenetszintű páros bootstrap 95%-os CI-je kizárja a 0-t; a küszöböt a költségarány rögzíti ($\tau^* = 1/6$). Erőforrás: CPU, a betanított modellek logitjain néhány másodperc; a tanítás öt futása néhány perc. ### Tipikus tévedés, önellenőrzés és visszautalás

Tipikus tévedés: a maximális softmax-valószínűség garantált bizonytalansági vagy OOD-mérték. Az OOD (out-of-distribution) a tanult eloszláson kívüli mintát jelenti. A softmax normalizálása miatt a $p_{\max}$ csak a *zárt osztályhalmazhoz viszonyított* relatív konfidencia. Ha egy olyan mintát adunk a modellnek, amely egyik tanult osztályhoz sem hasonlít, a logitok lehetnek például $[0{,}1; 0{,}0]$; ekkor a softmax $[0{,}525; 0{,}475]$, tehát a modell 0,525 konfidenciával dönt, noha a minta az eloszláson kívüli. A kalibráció sem oldja meg ezt: a kalibráció azt méri, hogy a tanult eloszláson a 0,525 körüli döntések milyen gyakran helyesek, nem azt, hogy a konkrét minta ismert-e. Az OOD-detektáláshoz külön pontszám, például a logitok energiája vagy egy sűrűségmodell szükséges; a $p_{\max}$-ot ilyenként használni hibás.

Önellenőrzés. (1) A hőmérséklet nem változtatja meg az argmax-ot; igaz-e $T = 0{,}1$-re is? Igen, mert $T > 0$. (2) A Brier-pontszám felső határa 2, nem 1; ellenőriztük a rossz osztályra tett 1 valószínűség esetével. (3) A validációs illesztés után a teszt log loss csökkenése akkor megbízható, ha a tesztet sem a T^* választására, sem más hiperparaméter-hangolásra nem használtuk.

Visszautalás. A 22.2. Címkemegőrző transzformációk címkemegőrző transzformációk egysége rögzítette, hogy a validációs képeken nem alkalmazhatunk olyan augmentációt, amely megváltoztatná a címkét; ugyanígy a kalibráló halmaznak a teszt eloszlását kell reprezentálnia. A 11.2. nn.Module és paraméterregisztráció a `train()`/`eval()` flag jelentését adta meg: a logitgyűjtés alatt a modellnek `eval()` módban kell lennie, különben a dropout vagy BatchNorm tanítási statisztikája torzítja a kalibráló logitokat. A hőmérséklet-illesztés a logitok skálájától függ, ezért minden architektúrához külön kell elvégezni, és a T értéke nem vihető át automatikusan más modellre. Korábbi anyagból a 7.1. Entrópia, cross-entropy és KL entrópia/KL fogalmai adják a log loss proper scoring tulajdonságának hátterét: mivel $L_{\log} = H(p_{\text{adat}}) + D_{KL}(p_{\text{adat}} \| p_{\text{modell}})$,

azaz $D_{KL}(p_{\text{adat}} \| p_{\text{modell}}) = L_{\log} - H(p_{\text{adat}})$, és a KL nemnegatív, a várható log loss minimalizálója a valódi eloszlás.

23.2.8 Rövid összefoglaló

Az induló kérdésre a válasz: a $p_{\max} \approx 0{,}8$ önmagában nem jelent 80%-os helyességet. Azt kell mérni, hogy a 0,8 körüli konfidenciájú minták mekkora hányada helyes (reliability diagram, ECE, Brier-pontszám); a hiba a logitok megváltoztatása nélkül hőmérséklet-skálázással javítható, mert a $T > 0$ osztó az argmax-döntést nem változtatja meg. A logitok softmax-kimenete valószínűségvektor, de a döntési szabály és a kalibráció két külön értékelési réteg. A log loss és a többosztályos Brier-pontszám a teljes eloszlást bünteti; a reliability diagram és az ECE a konfidencia és a megfigyelt pontosság egyezését méri. A temperature scaling egyetlen $T > 0$ skalárral osztja a logitokat, nem változtatja az argmaxot, de élesíti vagy lapítja az eloszlást; a $T \to \infty$ határ az egyenletes, a $T \to 0^+$ a kemény max eloszlás. A T validációs log loss-minimalizálással illeszthető, és a gradiensét explicite levezettük. A döntési küszöb a költségarányból adódóan $\tau^* = C_{FP}/(C_{FP}+C_{FN})$, és csak kalibrált valószínűségek mellett értelmezhető költségoptimumként. A maximális softmax nem OOD-mérték.

23.2.9 Következő lecke

A 23.3. Páros összehasonlítás és csoportszintű intervallum egységben a páros összehasonlítás és a csoportszintű intervallum következik: a GlyphLab scene-szintű függősége miatt a képenkénti konfidenciaintervallum nem elegendő, ezért a két modell vagy két döntési szabály közötti különbséget páros, scene-szintű újramintavétellel becsüljük.

23.2.10 Felhasznált források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30. A logits, a célalakok és a reduction fogalmait innen vesszük.
- Guo, C., Pleiss, G., Sun, Y. és Weinberger, K. Q. (2017). *On Calibration of Modern Neural Networks*. ICML 2017, PMLR 70:1321–1330. https://arxiv.org/abs/1706.04599. A temperature scaling és az ECE tárgyalásának forrása.
- [D15] scikit-learn developers (2026). *Model evaluation: quantifying the quality of predictions*. scikit-learn dokumentáció (stable). https://scikit-learn.org/stable/modules/model_evaluation.html. Hozzáférés: 2026-10-30.

23.3 Páros összehasonlítás és csoportszintű intervallum

23.3.1 Cél, előismeret és induló kérdés

A 23.2. Kalibráció és döntési küszöb egység a modell valószínűségi kimenetét és a döntési küszöböt választotta szét. Most a *pontosság* típusú mutatók bizonytalanságát vizsgáljuk, és ehhez a tesztminta szerkezetét tesszük explicitté. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab test halmaza 60 képből áll, de a 60 kép csak 30 független jelenetből (scene) származik: minden jelenethez két zajos nézet (view) tartozik. Ha egy jelenet két nézete hasonlóan viselkedik, akkor a 60 kép nem hordoz 60 független bizonyítékot.

Előismeret: a konfidenciaintervallum 5.3. Mintavételi bizonytalanság és intervallum alfejezetbeli ismételt mintavételi értelmezése, a függő képek és a bootstrap egység megválasztása 5.4. Függő képek, seedek és resampling, a split scene-invariánsa 11.3. Dataset és DataLoader a split után, valamint az accuracy aggregálási konvenciói 23.1. Confusion matrix és aggregálási konvenció. Induló kérdés: ha a test accuracy 0,9, akkor a „±” értékét 60 kép vagy 30 jelenet alapján számoljuk? Durva előzetes becslésként egy binomiális jellegű standard hiba $p = 0{,}9$ mellett 60 független egységnél $\sqrt{0{,}9 \cdot 0{,}1/60} \approx 0{,}039$, 30 független egységnél $\sqrt{0{,}9 \cdot 0{,}1/30} \approx 0{,}055$; a tényleges eltérés attól függ, mennyire korrelálnak a nézetek a jeleneten belül.

23.3.2 Jelenetátlagos accuracy, páros hibák és a bizonytalanság három forrása

Legyen a tesztben N kép, n_s jelenet és K osztály. Az i képre $s(i)$ a jelenetazonosító, $\mathcal{V}_s = \{i : s(i) = s\}$ a jelenet nézeteinek halmaza, $v_s = |\mathcal{V}_s|$. A GlyphLab testen $N = 60$, $n_s = 30$, $K = 3$, $v_s = 2$ minden s-re, és minden osztályból $c_k = 10$ jelenet van. A predikciók tömbje $\hat{y} \in \{0, \dots, K-1\}^N$ alakú `int64`, a cél y ugyanilyen. A képszintű helyesség

$$A_i = \mathbf{1}\{\hat{y}_i = y_i\} \in \{0, 1\}.$$

A **jelenetátlagos accuracy** (per-scene accuracy) a jeleneten belül átlagol, majd a jeleneteket egyenlő súllyal veszi:

$$A_s = \frac{1}{v_s} \sum_{i \in \mathcal{V}_s} A_i, \qquad \bar{A} = \frac{1}{n_s} \sum_{s=1}^{n_s} A_s.$$

A kiegyensúlyozott GlyphLabon a képszintű accuracy és $\bar{A}$ numerikusan megegyezik, mert minden jelenetnek ugyanannyi (két) képe van; általában az egyezés feltétele az azonos nézetszám, $v_s = v$ minden jelenetre, különben a képszintű átlag a nézetszám szerint implicit súlyoz. A különbség nem a pontértékben, hanem abban van, hogy $\bar{A}$ definíciója explicitté teszi a **mintavételi egységet**, a jelenetet. A 23.1. Confusion matrix és aggregálási konvenció alfejezetbeli micro/macro megkülönböztetés mintájára ez jelenetszintű átlagolás, nem osztályon belüli macro-átlag.

A GlyphLab orientációs címkéje jelenettulajdonság, ezért minden jelenethez egyértelműen tartozik y_s osztály. Az **osztály-rétegzett jelenetátlag**

$$\bar{A} = \frac{1}{K} \sum_{k=1}^{K} \bar{A}_{(k)}, \qquad \bar{A}_{(k)} = \frac{1}{c_k} \sum_{s \in \mathcal{S}_k} A_s, \qquad \mathcal{S}_k = \{s : y_s = k\}$$

a három osztály jelenetátlagát egyenlő súllyal veszi. Ez a generáló eloszlás osztályarányát tükrözi, és nem függ attól, hogy egy véletlen bootstrap húzásban éppen hány jelenet kerül az egyes osztályokból.

Páros hibák (paired errors): ha két körülményt, például a clean és a corrupt tesztet, ugyanazon a s jeleneten értékelünk, akkor nem két független accuracy átlagot hasonlítunk össze, hanem a jelenetenkénti különbségeket:

$$\Delta_s = A_s^{\mathrm{cl}} - A_s^{\mathrm{co}}, \qquad \bar{\Delta} = \frac{1}{n_s} \sum_{s=1}^{n_s} \Delta_s.$$

Itt cl a tiszta, co a korrupt teszt. Ha a jelenetek függetlenek és azonos eloszlásúak, akkor

$$\mathrm{Var}(\bar{\Delta}) = \frac{1}{n_s}\Big(\sigma_{\mathrm{cl}}^2 + \sigma_{\mathrm{co}}^2 - 2\,\mathrm{Cov}(A_s^{\mathrm{cl}}, A_s^{\mathrm{co}})\Big),$$

ahol $\sigma_{\mathrm{cl}}^2 = \mathrm{Var}(A_s^{\mathrm{cl}})$ és $\sigma_{\mathrm{co}}^2 = \mathrm{Var}(A_s^{\mathrm{co}})$ a jelenetszintű accuracyk populációs varianciája, Cov pedig a két körülmény jelenetszintű kovarianciája; a képlet a jelenetek függetlenségét és azonos eloszlását feltételezi. A közös jelenet nehézsége tipikusan pozitív kovarianciát ad: a nehéz jelenet mindkét körülményben alacsonyabb A_s-t ad, ezért a két érték együtt mozog, és a különbség varianciájában a kovariancia negatív előjellel csökkenti az összeget. Így a páros összehasonlítás varianciája kisebb, mint a független csoportoké. A párosítás a GlyphLab corrupt tesztjénél megfeleltetésen alapul: a `corrupted_test.npy` a korrupt bemeneti képeket tartalmazza, a belőlük készült predikciókat pedig a `lab/test_preds_corrupt.npy` rögzíti; mindkettő sorrendje a metadata test sorrendje, tehát ugyanaz a jelenet/nézet pár.

A bizonytalanságnak három, gyakran összemosott forrását különböztetjük meg:

1. **Tesztminta-mintavételi bizonytalanság**: a 30 tesztjelenet csak egy minta a generáló eloszlásból; ezt célozza a jelenet-bootstrap.
2. **Tanítási seedek ingadozása**: ugyanazon a rögzített teszten több training seeddel kapott accuracyk szórása. Jelölése $A^{(r)}$, $r = 1, \dots, R$, átlaga $\bar{A}^{(\cdot)}$, szórásnégyzete $\hat{\sigma}^2_{\mathrm{seed}} = \frac{1}{R-1}\sum_r (A^{(r)} - \bar{A}^{(\cdot)})^2$.
3. **Alkalmazási/domain kockázat**: a modell más eloszlású bemeneten elkövetett hibája. Ezt sem a bootstrap, sem a seed-ismétlés önmagában nem becsli.

Konfidenciaintervallum alatt a [S04] szerinti ismételt mintavételi (bootstrap) eljárást értjük: a 95%-os intervallum olyan eljárás, amely sok független tesztminta esetén körülbelül az esetek 95%-ában lefedi a rögzített paramétert. Ez nem annak a valószínűsége, hogy „a paraméter az adott intervallumban van”.

23.3.3 A class-stratified jelenet-bootstrap mintavételi egysége és súlyai

A **klaszter-bootstrap** (cluster bootstrap) a jelenetet tekinti újramintavételi egységnek: ha egy s jelenetet behúzunk, annak minden view-ját együtt vesszük. A **rétegzett** változat osztályonként külön húz, így minden replikátumban c_k jelenet van a k osztályból. A GlyphLab generátor egy jelenethez pozíciót, vastagságot és hosszúságot húz; a jelenet címkéje az orientáció (0 = vertical, 1 = horizontal, 2 = diagonal), és a jelenet két view-ja ugyanennek a jelenetnek két zajos megfigyelése. A következő lemma a replikátumok feltételes eloszlásának első két momentumát rögzíti; a * felső index a bootstrap húzás szerinti feltételes várható értéket és varianciát jelöli, a tesztjelenetek rögzítettek. Szándékosan nem állítja a bootstrap intervallum lefedettségét; az utóbbi külön, itt nem bizonyított aszimptotikus kérdés.

Numerikus előpélda. Legyen $K = 2$ osztály, osztályonként $c_1 = c_2 = 2$ jelenet, a jelenetaccuracyk pedig $A = (1, 0)$ az első és $(0, 1)$ a második osztályban. Egy replikátumban az első osztályból kétszer az első jelenetet húzzuk, a másodikból kétszer a másodikat: a bootstrap súlyok $(1, 0)$, illetve $(0, 1)$, a replikátumátlag $\frac{1}{2}(1 \cdot 1 + 0 \cdot 0) + \frac{1}{2}(0 \cdot 0 + 1 \cdot 1) = 1$. Ha mindkét osztályban egyszer-egyszer húzzuk mindkét jelenetet, a súlyok $(\frac{1}{2}, \frac{1}{2})$, a replikátumátlag $\frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$. A lemma varianciaképlete erre a példára $\frac{1}{4} \cdot \frac{1}{4}((1 - \frac{1}{2})^2 + (0 - \frac{1}{2})^2) \cdot 2 = \frac{1}{16}$, azaz szórás $\frac{1}{4}$.

Lemma (class-stratified scene bootstrap momentumai). Rögzítsük a tesztjeleneteket és a k osztály A_s értékeit. A b-edik replikátumban minden k rétegben c_k darab független, egyenletes húzást végzünk a $\mathcal{S}_k$ halmazból visszatevéssel. Legyen $N_{b,s}^{(k)}$ az $s \in \mathcal{S}_k$ jelenet húzási gyakorisága, tehát $N_b^{(k)} \sim \mathrm{Multinomial}(c_k, \frac{1}{c_k}\mathbf{1})$, és legyen a rétegen belüli **bootstrap súly**

$$w_{b,s}^{(k)} = \frac{N_{b,s}^{(k)}}{c_k}.$$

A replikátum

$$\bar{A}_{(k),b}^{*} = \sum_{s \in \mathcal{S}_k} w_{b,s}^{(k)} A_s = \frac{1}{c_k} \sum_{j=1}^{c_k} A_{I_{b,j}}, \qquad \bar{A}_b^{*} = \frac{1}{K} \sum_{k=1}^{K} \bar{A}_{(k),b}^{*}.$$

Ekkor $E^*[\bar{A}_b^*] = \bar{A}$, és

$$\mathrm{Var}^*(\bar{A}_b^*) = \frac{1}{K^2} \sum_{k=1}^{K} \frac{1}{c_k^2} \sum_{s \in \mathcal{S}_k} \left(A_s - \bar{A}_{(k)}\right)^2.$$

A súlyok tulajdonságai: $w_{b,s}^{(k)} \geq 0$, rétegenként $\sum_s w_{b,s}^{(k)} = 1$; a replikátum a rétegátlagok $\frac{1}{K}$-súlyozott átlaga, ezért a rétegek együttes súlya a képletben rögzített $1/K$ (ez $\bar{A}_b^*$ definíciójából adódik, nem a húzásból), és $E^*[w_{b,s}^{(k)}] = 1/c_k$. A véletlen súlyok várható értéke tehát a rétegen belüli egyenletes jelenetsúly.

Bizonyítás. A * felső index a bootstrap húzás szerinti feltételes várható értéket/varianciát jelöli, a tesztadatok rögzítettek. A b-edik replikátum $I_{b,1}, \ldots, I_{b,c_k}$ húzásai függetlenek, és mindegyik egyenletes eloszlású $\mathcal{S}_k$-n. Mivel $\bar{A}_{(k),b}^*$ ezek átlaga,

$$E^*[\bar{A}_{(k),b}^*] = \frac{1}{c_k} \sum_{j=1}^{c_k} E^*[A_{I_{b,j}}] = \frac{1}{c_k} \sum_{j=1}^{c_k} \frac{1}{c_k} \sum_{s \in \mathcal{S}_k} A_s = \bar{A}_{(k)}.$$

A varianciához először egyetlen húzás varianciáját számoljuk:

$$\mathrm{Var}^*(A_{I_{b,j}}) = E^*[A_{I_{b,j}}^2] - \bar{A}_{(k)}^2 = \frac{1}{c_k} \sum_{s \in \mathcal{S}_k} A_s^2 - \bar{A}_{(k)}^2 = \frac{1}{c_k} \sum_{s \in \mathcal{S}_k} \left(A_s - \bar{A}_{(k)}\right)^2,$$

ahol az utolsó egyenlőség a négyzetes eltérés szokásos kifejtése. A c_k darab húzás függetlensége miatt

$$\mathrm{Var}^*(\bar{A}_{(k),b}^*) = \frac{1}{c_k^2} \sum_{j=1}^{c_k} \mathrm{Var}^*(A_{I_{b,j}}) = \frac{1}{c_k^2} \sum_{s \in \mathcal{S}_k} \left(A_s - \bar{A}_{(k)}\right)^2.$$

Végül a rétegenkénti húzások függetlenek, ezért a $\frac{1}{K}$-szoros átlag varianciája a rétegvarianciák összegének $1/K^2$-szerese. Ezzel a lemmát beláttuk.

23.3.4 Kézi példa és minimális ellenőrző kód

Legyen $K = 2$ osztály, mindkettőben $c_k = 3$ jelenet, az accuracy-értékek (a kézi példa szándékosan más paraméterezést használ, mint a GlyphLab teszt $K = 3$, $c_k = 10$ esete; c_k itt a kézi példa rétegenkénti jelenetszáma):

Osztály	A_s értékek	$\bar{A}_{(k)}$
0	1,0; 1,0; 0,5	5/6
1	0,5; 0,5; 0,0	1/3

Az osztály-rétegzett jelenetátlag $\bar{A} = (5/6 + 1/3)/2 = 7/12 \approx 0{,}5833$. A lemma varianciája: mindkét rétegben $\sum_s (A_s - \bar{A}_{(k)})^2 = 1/6$, tehát a rétegvariancia $1/(9 \cdot 6) = 1/54$, a teljes variancia $1/(4)(1/54 + 1/54) = 1/108$, a szórás $1/\sqrt{108} \approx 0{,}0962$.

Egy konkrét replikátum lehet: a 0. osztály húzásai (s_1, s_3, s_2) értékei $(1{,}0; 0{,}5; 1{,}0)$, átlaga 5/6; az 1. osztály húzásai (s_2, s_3, s_1) értékei $(0{,}5; 0{,}0; 0{,}5)$, átlaga 1/3; a replikátum 7/12. Egy másik replikátum, ahol az 1. osztályban kétszer a 0,0 értékű jelenet jön, $\bar{A}^*_b = (5/6 + 1/6)/2 = 1/2$, mutatja a replikátumok ingadozását.

```
import numpy as np

rng = np.random.default_rng(20261020)
A = {0: np.array([1.0, 1.0, 0.5]),
     1: np.array([0.5, 0.5, 0.0])}
B = 2000
repl = np.empty(B, dtype=np.float64)
for b in range(B):
    total = 0.0
    for vals in A.values():
        c = vals.shape[0]
        idx = rng.integers(0, c, size=c)
        total += vals[idx].mean()
    repl[b] = total / len(A)

print(repl.mean(), repl.std(ddof=1),
      np.percentile(repl, [2.5, 97.5]))
```

A lemmából számított ellenőrző értékek a fentiek. A kódot $B = 2000$, `seed=20261020` mellett futtatva a mért replikátumátlag 0,5844, a minta szórása 0,0976, a 95%-os percentilis intervallum [0,4167; 0,7500]; ezek az elméleti $7/12 \approx 0{,}5833$ és $\sqrt{1/108} \approx 0{,}0962$ értékeket a véletlen hiba nagyságrendjén belül közelítik (Python 3.12.3, NumPy 2.5.4, CPU).

23.3.5 Labor: GlyphLab jelenetátlagos accuracy és intervallum

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és pandas 2.3.3. Szükséges fájlok: `lab/data/metadata.csv`, valamint a 22.4. Augmentációs kísérlet és domainváltozás laborjában betanított SmallCNN predikcióiból előállított `lab/test_preds.npy` (opcionálisan `lab/test_preds_corrupt.npy`). Futtatás: `python lab/scene_interval.py --preds lab/test_preds.npy --corrupt lab/test_preds_corrupt.npy --out lab/scene_interval.json`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

Bemenetek: `lab/data/metadata.csv`, a modell által előállított `lab/test_preds.npy` alakja `(60,)`, `int64`, a metadata test sorrendjében; opcionálisan `lab/test_preds_corrupt.npy` ugyanebben a sorrendben. Előfeltétel: a 22.4. Augmentációs kísérlet és domainváltozás laborjában betanított SmallCNN best checkpointja, valamint a belőle a fenti `save_test_preds` függvénnyel előállított predikciós fájl; ezek nélkül a szkript már az első bemenet-ellenőrzésnél leáll. A `metadata.csv` kötelező oszlopai: `index` (a képtömb sora), `split` (`train`/`val`/`test`), `label` (0–2 osztályindex) és `scene_id` (jelenetazonosító); a predikciós tömb sorai a `split=test` sorok sorrendjében értendők. A predikciót a korábbi egységek szerint, a legjobb val-loss checkpointból

állítjuk elő (a `model` a betöltött checkpoint modellje, az `images_test` a teszt split képtömbje; a részlet ezeket paraméterként kapja):

```
def save_test_preds(model, images_test, out="lab/test_preds.npy"):
    model.eval()
    with torch.inference_mode():
        logits = model(images_test)          # (60,1,24,24), float32, CPU
    preds = logits.argmax(dim=1).cpu().numpy().astype(np.int64)
    np.save(out, preds)
```

A metrikaszámítás `numpy` és `pandas` segítségével, CPU-n fut. A bootstrap seedje `20261020`, a replikátumszám `B=2000`, a percentilis intervallum a `[2.5, 97.5]` percentilisekből áll. A futtatás a projekt gyökeréből:

```
python lab/scene_interval.py --preds lab/test_preds.npy \
  --corrupt lab/test_preds_corrupt.npy --out lab/scene_interval.json
```

Futásidő: a bootstrap-lépés 2000 replikátumot húz a 30 jeleneten; a projekt venv-jében mérve 0,04 s (Python 3.12.3, NumPy 2.5.4, CPU). A teljes szkript idejét a predikciós fájlok betöltése és a gép határozza meg, ezért az nem a bootstrap függvény teljesítményét méri.

```
# lab/scene_interval.py
import argparse
import json
import numpy as np
import pandas as pd

BASE = "lab"

def bootstrap_from_dict(vals_by_class, B, seed):
    rng = np.random.default_rng(seed)
    classes = list(vals_by_class)
    repl = np.empty(B, dtype=np.float64)
    for b in range(B):
        total = 0.0
        for k in classes:
            vals = np.asarray(vals_by_class[k], dtype=np.float64)
            c = vals.shape[0]
            total += vals[rng.integers(0, c, size=c)].mean()
        repl[b] = total / len(classes)
    return repl

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--preds", default=None)
    p.add_argument("--corrupt", default=None)
    p.add_argument("--out", default=f"{BASE}/scene_interval.json")
    p.add_argument("--B", type=int, default=2000)
    p.add_argument("--seed", type=int, default=20261020)
    p.add_argument("--check", action="store_true")
    args = p.parse_args()

    if args.check:
        A = {0: np.array([1.0, 1.0, 0.5]),
             1: np.array([0.5, 0.5, 0.0])}
        repl = bootstrap_from_dict(A, args.B, args.seed)
        K = len(A)
        var_lemma = sum(((v - v.mean())**2).sum() / v.shape[0]**2
                        for v in A.values()) / K**2
        assert abs(repl.mean() - 7/12) < 0.01
        assert abs(repl.var(ddof=1) - var_lemma) < 0.01
        print("check ok", repl.mean(), repl.var(ddof=1), var_lemma)
```

```
    return

if args.preds is None:
    p.error("a fő futáshoz a --preds megadása kötelező")

meta = pd.read_csv(f"{BASE}/data/metadata.csv")
# A predikciók a metadata test sorainak sorrendjében vannak, ezért nem
# rendezünk: az index oszlop csak a tömbindex-egyezést igazolja.
if "index" not in meta.columns:
    raise ValueError("a metadata.csv-ből hiányzik az index oszlop")
idx_all = meta["index"].to_numpy(dtype=np.int64)
if not np.array_equal(np.sort(idx_all), np.arange(len(meta))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
test = meta.loc[meta["split"] == "test"].reset_index(drop=True)
labels = test["label"].to_numpy(dtype=np.int64)
preds = np.load(args.preds).astype(np.int64)
assert len(test) == 60
assert preds.shape == (60,), preds.shape

# A kiegyensúlyozott szerződés ellenőrzése: 30 jelenet, jelenetenként két
# nézet, egy jeleneten belül egy osztály.
per_scene = test.groupby("scene_id").agg(n=("index", "size"),
                                         k=("label", "nunique"))
assert len(per_scene) == 30
assert (per_scene["n"] == 2).all(), "minden jelenethez két nézet tartozik"
assert (per_scene["k"] == 1).all(), "egy jelenet egy osztályba tartozik"

test["correct"] = (preds == labels).astype(np.float64)
image_acc = float(test["correct"].mean())
scene_acc = test.groupby("scene_id")["correct"].mean()
# A per_scene "k" ellenőrzés garantálja, hogy egy jeleneten belül egyetlen
# címke van, ezért a first() nem veszít információt.
scene_label = test.groupby("scene_id")["label"].first()
scene_mean = float(scene_acc.mean())
# A két átlag egyezése a kiegyensúlyozott (2 nézet/jelenet) szerződés
# következménye, nem független numerikus bizonyíték.
assert np.isclose(image_acc, scene_mean, rtol=0, atol=1e-12)

vals = {int(k): scene_acc[scene_label == k].to_numpy(dtype=np.float64)
        for k in np.unique(scene_label)}
repl = bootstrap_from_dict(vals, args.B, args.seed)
lo, hi = np.percentile(repl, [2.5, 97.5])

out = {
    "image_accuracy_mean": image_acc,
    "scene_accuracy_mean": scene_mean,
    "scene_ci_low": float(lo),
    "scene_ci_high": float(hi),
    "bootstrap_unit": "scene_id",
    "stratified_by": "label",
    "B": args.B,
    "seed": args.seed,
}

if args.corrupt:
    preds_c = np.load(args.corrupt).astype(np.int64)
    assert preds_c.shape == (60,), preds_c.shape
    # A corrupt predikciók ugyanebben a metadata-sorrendben vannak,
    # ezért a párosítás pozíció szerinti.
    test["correct_c"] = (preds_c == labels).astype(np.float64)
    scene_acc_c = test.groupby("scene_id")["correct_c"].mean()
    delta = scene_acc - scene_acc_c
    vals_d = {int(k): delta[scene_label == k].to_numpy(dtype=np.float64)
```

```
                for k in np.unique(scene_label)}
        drep = bootstrap_from_dict(vals_d, args.B, args.seed)
        dlo, dhi = np.percentile(drep, [2.5, 97.5])
        out.update({
            "paired_mean_delta_clean_minus_corrupt": float(delta.mean()),
            "paired_ci_low": float(dlo),
            "paired_ci_high": float(dhi),
        })

    with open(args.out, "w", encoding="utf-8") as f:
        json.dump(out, f, indent=2)
    print(json.dumps(out, indent=2))

if __name__ == "__main__":
    main()
```

Ellenőrzés: a `--check` kapcsoló a kézi példán a lemmabeli momentumokat ellenőrzi; a kód a lemma varianciaképletét (`var_lemma`) közvetlenül kiszámítja a kézi példára, és a replikátumok tapasztalati varianciájához hasonlítja. A mért kimenet `check ok 0.5844166666666667 0.009531369851592465 0.009259259259259259`. A fő futás a `(60,)` predikcióalakot, a `metadata.csv` index-oszlopának permutáció-voltát és a 30 jelenetes, jelenetenként kétnézetes szerkezetet is ellenőrzi; a képszintű és a jelenetátlagos accuracy egyezése e kiegyensúlyozott szerkezet következménye, ezért nem független numerikus bizonyíték. A fő futást a referenciamodell (a `lab/train_cnn.py` 1299 paraméteres `SmallCNN`-je, seed 20261020) clean és corrupt tesztpredikcióival futtattuk (Python 3.12.3, NumPy 2.5.4, CPU); a mért JSON:

```
{
  "image_accuracy_mean": 0.95,
  "scene_accuracy_mean": 0.95,
  "scene_ci_low": 0.9,
  "scene_ci_high": 1.0,
  "bootstrap_unit": "scene_id",
  "stratified_by": "label",
  "B": 2000,
  "seed": 20261020,
  "paired_mean_delta_clean_minus_corrupt": 0.6,
  "paired_ci_low": 0.5499999999999999,
  "paired_ci_high": 0.65
}
```

Eredményértelmezés: a `scene_ci_low/high` a kiválasztott modell tesztminta-mintavételi bizonytalanságának közelítése a 30 jelenet mellett, a generáló GlyphLab eloszlásra vonatkoztatva. A páros `paired_mean_delta_clean_minus_corrupt` a corrupt bemenet hatását ugyanazon jelenet/nézet párokon méri; a hozzá tartozó intervallum a párosítás miatt nem a két accuracy-különbség két független mintából számolt varianciáját használja.

23.3.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem mindegy, hogy a GlyphLab bootstrap egysége a kép vagy a jelenet. Adjon olyan mesterséges tesztmintát, ahol a képszintű bootstrap

standard hibája $1/\sqrt{2}$-szerese a jelenet-bootstrapének. *Elvárt eredmény:* definíciók, feltétel és a számpélda.

F2 (levezetési, nehézség 2). A kézi példán vezesse le mindkét réteg replikátum-varianciáját, és számítsa ki a teljes varianciát 1/108-ként. *Elvárt eredmény:* a lemma minden lépése és a numerikus ellenőrzés.

F3 (implementációs, nehézség 2). Írjon `stratified_scene_bootstrap(A_by_class, B, seed)` függvényt, amely a `class -> float64 tömb` leképezésből `(B,)` replikátumtömböt ad. Ellenőrizze $B = 2000$, `seed=20261020` mellett, hogy a kézi példán a replikátumátlag $7/12 \pm 0{,}01$ és a szórás $\sqrt{1/108} \pm 0{,}02$ tűrésen belül van. *Elvárt eredmény:* teljes kód, shape- és szélsőeset-kontroll.

F4 (hibakeresési, nehézség 2). Adott a következő replikátumszámítás: `idx = rng.integers(0, len(scene_acc), size=len(scene_acc)); repl_b = scene_acc[idx].mean()`. Nevezze meg a hibát a GlyphLab class-stratified bootstrap kontextusában, írja le a következményt a replikátumok osztályarányára, és adja meg a javítást. *Elvárt eredmény:* ok, következmény, javított kód, visszaellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen protokollt, amely a clean és corrupt GlyphLab teszt accuracy-különbségét páros jelenet-bootstrap intervallummal jelenti, és emellett három training seedet is rögzít. Válassza szét a tesztminta-, seed- és domain-bizonytalanságot. *Elvárt eredmény:* kérdés, hipotézis, egység, baseline, metrika, erőforrás, érvényességi korlát.

23.3.7 Megoldások és indoklás

F1. A GlyphLab generáló folyamatában a független húzási egység a `scene_id`: a jelenet pozíciója, vastagsága és hossza jelenetszinten dől el, a két view csak a jelenet két zajos megfigyelése. A képszintű bootstrap ezt a klaszterszerkezetet figyelmen kívül hagyja. Legyen 30 jelenet, mindegyikben két view, és minden jelenet vagy mindkét view-val helyes ($A_s = 1$), vagy mindkettővel hibás ($A_s = 0$); legyen 27 helyes jelenet. Ekkor a jelenetátlag 0,9, a jelenetek binomiálisak, a jelenet-átlag standard hibája $\sqrt{0{,}9 \cdot 0{,}1/30}$. A képszintű bootstrap 60 képet lát, amelyek páronként duplikált kimenetűek, és $\sqrt{0{,}9 \cdot 0{,}1/60}$ nagyságrendű szórást becsül; a kettő aránya $1/\sqrt{2}$. A feltétel az, hogy a view-k a jeleneten belül tökéletesen korrelálnak; ha a jeleneten belüli korreláció $\rho < 1$, a két standard hiba aránya $1/\sqrt{1+\rho}$, amely ρ csökkenésével 1 felé nő ($\rho = 1$-nél $1/\sqrt{2}$, $\rho = 0$-nál 1), de a képszintű eljárás ekkor sem becsüli a jelenet-egység szerinti mintavételt.

F2. A 0. osztályban $\bar{A}_{(0)} = 5/6$, az eltérések $1/6$, $1/6$, $-1/3$, négyzetösszegük $1/36+1/36+1/9 = 1/6$. A lemma szerint a réteg replikátum-varianciája $\frac{1}{3^2} \cdot \frac{1}{6} = 1/54$. Az 1. osztályban $\bar{A}_{(1)} = 1/3$, az eltérések $1/6$, $1/6$, $-1/3$, így ugyanúgy $1/54$. A teljes replikátum a két rétegátlag átlaga, ezért varianciája $\frac{1}{2^2}(\frac{1}{54} + \frac{1}{54}) = \frac{1}{108}$. Numerikusan $1/108 \approx 0{,}009259$, szórás $0{,}096225$.

F3. A kód:

```
import numpy as np

def stratified_scene_bootstrap(A_by_class, B, seed):
    rng = np.random.default_rng(seed)
    classes = list(A_by_class)
    out = np.empty(B, dtype=np.float64)
    for b in range(B):
        total = 0.0
        for k in classes:
            vals = np.asarray(A_by_class[k], dtype=np.float64)
            c = vals.shape[0]
```

```
            if c == 0:
                raise ValueError("empty class layer")
            total += vals[rng.integers(0, c, size=c)].mean()
        out[b] = total / len(classes)
    return out

A = {0: np.array([1.0, 1.0, 0.5]),
     1: np.array([0.5, 0.5, 0.0])}
repl = stratified_scene_bootstrap(A, 2000, 20261020)
assert repl.shape == (2000,)
assert abs(repl.mean() - 7/12) < 0.01
assert abs(repl.std(ddof=1) - np.sqrt(1/108)) < 0.02
```

Szélsőeset: ha egy rétegben $c = 1$, a replikátum az egyetlen értéket adja, a rétegvariancia 0; a kód ezt helyesen kezeli, mert az egyetlen elem átlaga determinisztikus. A tolerancia a $B = 2000$ replikátum véletlen hibájára méretezett.

F4. A hiba: a kód az összes jelenetet egyetlen halmazként húzza visszatevéssel, így nem tartja rétegenként a $c_k = 10$ jelenetszámot. Következmény: a replikátum osztályösszetétele véletlenszerűen eltér a GlyphLab fix osztályarányától. Egyenlő c_k mellett a naiv replikátumátlag várható értéke nem torzított: $\mathbb{E}[\text{naiv replikátum}] = \frac{1}{30}\sum_s A_s = \frac{1}{3}\sum_k \bar{A}_{(k)}$, mert a 30 jelenet 10-10-10 arányban oszlik meg; a különbség a replikátumok szórásában és osztályösszetétel-ingadozásában van (a naiv szórás nagyobb, mert a húzott osztályarány is ingadozik). Torzítás akkor lép fel, ha a c_k értékek eltérnek, vagy a cél a fix 1/3-1/3-1/3 súlyozás: ilyenkor a naiv bootstrap a mintabeli osztályarányt követi, nem a kívánt fix súlyokat. Javítás: minden k rétegből külön c_k húzás, ahogyan a `bootstrap_from_dict` teszi. Visszaellenőrzés a kézi példán: a rétegzett változat átlaga 7/12 körül marad, a naiv változat átlaga egyenlő c_k mellett szintén 7/12 körül van, de nagyobb szórással és ingadozó osztályaránnyal; eltérő rétegméretnél a naiv átlag már eltér a rétegzettől.

F5. Egy lehetséges protokoll. Kérdés: mekkora a corrupt bemenet jelenetátlagos accuracy-csökkenése ugyanazon modellen? Hipotézis: a corrupt zaj az egyik orientációt jobban rontja. Adat: `test_preds.npy` és `test_preds_corrupt.npy`, mindkettő a metadata test sorrendben; a modellválasztás kizárólag val loss alapján történt. Megfigyelési egység: jelenet. Baseline: a clean jelenetátlagos accuracy. Metrika: Δ_s átlaga és 95%-os páros, osztály-rétegzett jelenet-bootstrap intervallum, $B = 2000$, seed rögzített. Seed-rész: három training seed mindegyikére kiszámítjuk a clean jelenetaccuracyt, és a seed-átlagot/min–max tartományt a teszt-CI-től külön jelentjük. Döntési szabály: a páros CI a betanított modellre feltételes; a seederedményeket nem olvasztjuk bele. Ha a 95%-os páros jelenet-bootstrap CI kizárja a 0-t (például a becsült átlagos csökkenés 0,08, a CI $[0{,}02; 0{,}14]$), a csökkenést a betanított modellre kimutatottnak tekintjük; ha a CI tartalmazza a 0-t (például $[-0{,}01; 0{,}17]$), nem állítunk különbséget. A seed-rész akkor erősíti meg az eredményt, ha mindhárom seednél azonos előjelű a csökkenés; ellentmondó seedek esetén a következtetés bizonytalan, és ezt jelenteni kell. Érvényességi korlát: a CI a betanított modellre és a GlyphLab generáló eljárásra feltételes; más adateloszlásra, annotációs hibára és későbbi modellfrissítésre nem ad fedést. A futáshoz kis CPU elegendő, mert a bootstrap a 30 jelenetátlagon fut (a 60 predikcióból számolva), nem a 60 képen külön.

Változó: a bemenet clean vagy corrupt volta. **Seedek, döntési küszöb és erőforrás:** a három training seed értéke 20261020–20261022, a bootstrap seedje 20261020, $B = 2000$. A feladat nem tanít modellt, ezért architektúra és optimalizáló helyett a meglévő predikciók kontrollja és a döntési szabály szerepel. A döntési szabály számszerű: a jelenetszintű páros CI 0-t kizáró 95%-os szintje, és legalább 2 százalékpontos átlagos csökkenés; a futás CPU-n néhány másodperc. ### Gyakori tévedés, önellenőrzés és visszautalás

Gyakori tévedés: *„a training seedekből vagy a képszintű szórásból számolt intervallum az alkalmazási kockázat intervalluma."* Ellenpélda: egy olyan tanítási csővezeték, amely nem használja a

seedet (nincs seedfüggő inicializálás vagy keverés), három seeddel is pontosan ugyanazt a súlyt állítja elő; a seed-szórás nulla, a seed-intervallum `[0.9, 0.9]`, miközben a 30 tesztjelenetből fakadó mintavételi bizonytalanság nem nulla. Megfordítva: ha a tesztjelenetek nagyon hasonlók, a bootstrap intervallum szűk, de a modell egy másik adateloszláson mégis hibázhat; a szűk CI nem domain-garancia. Az 5.4. Függő képek, seedek és resampling döntése itt is érvényes: a független mintavételi egység a `scene_id`, nem a kép.

Önellenőrzés: szerepel-e a jelentésben a bootstrap egység, a rétegződés, B és a seed? Ellenőriztem-e, hogy a metadata `index` oszlopa a tömbindexek permutációja (ezt a kód ellenőrzi), és hogy a predikciós tömb a `split=test` sorok sorrendjében készült (ezt a predikció mentésekor kell biztosítani)? A tesztértékelést nem használtam-e modell- vagy küszöbválasztásra? A seed-ingadozást külön közöltem-e a tesztminta-intervallumtól? Két különböző szórás összegzése előtt itt is meg kell nevezni, melyik véletlen forrást mérjük.

Rövid válaszok. (1) A jelentésnek tartalmaznia kell a bootstrap egységét (`scene_id`), a rétegződést (osztály), a B ismétlésszámot és a seedet. (2) Igen: a metadata `index` oszlopának permutáció-voltát a kód ellenőrzi, a predikciós tömböt pedig a `split=test` sorok sorrendjében kell menteni. (3) Nem: a tesztértékelést sem modell-, sem küszöbválasztásra nem használjuk. (4) Igen: a seed-ingadozást külön kell közölni a tesztminta-intervallumtól. (5) Összegzés előtt mindkét szórásnál meg kell nevezni a véletlen forrást; a kettőt nem szabad egyetlen „bizonytalanságként” összevonni.

23.3.8 Rövid összefoglaló

A GlyphLab tesztminta független egysége a jelenet, ezért a jelenetátlagos accuracy a jelenetátlagok átlaga, és a bootstrap is jelenetegységen húz. Az osztály-rétegzés minden replikátumban rögzíti a c_k jelenetszámot; a replikátumsúlyok rétegenként 1-re normáltak, várható értékük az egyenletes jelenetsúly. A lemma a replikátumok feltételes átlagát és varianciáját indikátorhúzásokkal vezeti le. A clean–corrupt összevetés páros, mert ugyanazon jelenet/nézet párokat használ. A bootstrap intervallum a tesztminta-mintavételi bizonytalanság közelítése, nem alkalmazási kockázat; a training seedek szórása külön dimenzió.

23.3.9 Következő lecke

A 23.4. Hibaszeletek és alkalmazási határ egység a hibaszeleteket vizsgálja: ugyanezt a jelenet-egységet és rétegzett bootstrapet használjuk osztályonkénti vagy nehézségi csoportonkénti hibaelemzésre.

23.3.10 Források

- [S04] Efron, B. és Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

23.4 Hibaszeletek és alkalmazási határ

23.4.1 Cél, előismeret és induló kérdés

A 23.3. Páros összehasonlítás és csoportszintű intervallum egységben a páros összehasonlítást és a scene-szintű, osztály-rétegzett bootstrap intervallumot vezettük be. Ott a metrika a teljes

tesztkészletre vonatkozott. Itt a kérdés az: mely részhalmazokon romlik el a modell, és mekkora csoportméret alatt már nem tehetünk értelmes állítást. A 22.4. Augmentációs kísérlet és domainváltozás alfejezetben a clean és corrupt teszt eloszlását különítettük el; a 19.4. ConvNeXt és a recept/architektúra elválasztása arra tanított, hogy az architektúra és a tanítási recept hatását ne keverjük, amit most annyiban használunk, hogy a szeletelemzés a modellválasztástól független értékelési réteg. A 11.4. Egy teljes PyTorch tanítási iteráció logitokból származó döntési szabálya adja a döntési határ definícióját.

Induló becslés: lehet-e egy 80%-os tesztpontosságú modellnek olyan szelete, amelyen a pontosság 0%? A következő szakaszokban ezt formálisan megválaszoljuk; a rövid válasz igen, és a 4. szakasz példája mutatja, hogyan koncentrálódhat a hiba egy kis szeletre.

23.4.2 Szelet, csoportméret és shift

Definíció (szelet). Legyen D a teszt indexeinek halmaza, $|D| = N$. Egy szelet $S \subseteq D$, a gyakorlatban egy megfigyelhető attribútum (osztály, nézet, korrupció, scene-csoport) által indukált részhalmaz. A predikciók $\hat{y} \in \{0, \dots, K-1\}^N$, a célok $y \in \{0, \dots, K-1\}^N$. A szelet pontossága

$$\mathrm{acc}(S) = \frac{1}{|S|} \sum_{i \in S} \mathbb{1}[\hat{y}_i = y_i], \qquad |S| > 0.$$

Ha $|S| = 0$, az érték definiálatlan; az üres szeletet a modellkártyán „nincs adat” jelöléssel hagyjuk ki, nullával való feltöltés hamis eredmény lenne. A kódban a szeletet N hosszú logikai maszkkal ábrázoljuk: `mask[i] == True` pontosan akkor, ha $i \in S$.

Definíció (csoportméret). A csoportméret a szelet elemszáma $n_S = |S|$. A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab tesztben osztályonként 10 scene és 20 kép van, a GlyphLab osztályai pedig $K = 3$: $0 =$ `vertical`, $1 =$ `horizontal`, $2 =$ `diagonal`. A `scene_id` szöveges azonosító (például `G0-0003`), nem sorszám, és a 30 tesztjelenet osztályonként 10-10-10 arányban oszlik meg; egy osztály–korrupció szelet például $n = 20$. A bootstrap becslés szórása növekszik, ha n_S kicsi, ezért a jelentésben minden szelet mellé kiírjuk n_S-t, nemcsak a pontosságot.

Definíció (shift). Shiftnek nevezzük, ha a kiértékelési bemeneti eloszlás eltér a tanításitól; a fejezetben a shift szót a domainváltozás (domain shift) rövidítéseként használjuk, a clean→corrupt eset pedig ennek szűkebb, korrupciós változata. A GlyphLab corrupt tesztje ugyanazon scene/view párokat zajjal és fényességi gradienssel módosítja, tehát egy pontosan specifikált korrupciós eloszlást reprezentál; nem általános „valós világ” shiftet. A clean és corrupt teszt páros szerkezete miatt a romlást páronként számoljuk, ahogyan 22.4. Augmentációs kísérlet és domainváltozás rögzítette.

A bootstrap röviden. A rögzített minta egységeit (itt a jeleneteket) visszatevéssel újramintavételezzük B-szer; minden replikátumra kiszámítjuk a statisztikát, majd a replikátumok 2,5 és 97,5 percentilisét vesszük intervallumként. A módszer a mintavételi bizonytalanságot közelíti, nem ad garanciát az alkalmazási kockázatra. Az intervallum ismételt mintavételi értelmezését a [S04] tárgyalja; a bootstrap itt ezt az értelmezést követi.

23.4.3 Hibataxonómia és döntési határ

Definíció (hibataxonómia). A hibataxonómia a téves predikciók rendezése három tengely mentén:

1. **Tévesztési cella:** a confusion mátrix $C_{ab} = |\{i : y_i = a, \hat{y}_i = b\}|$ off-diagonális eleme, ahol $a, b \in \{0; 1; 2\}$ és $a \neq b$; a főátló ($a = b$) a helyes döntéseket számolja, ezért nem tévesztési cella.
2. **Környezet:** clean vagy corrupt.
3. **Csoport:** a hibás példa `scene_id` azonosítója és `view_id` értéke.

A taxonómia tehát minden hibás indexhez egy rekordot rendel; ez operatív osztályozás, nem ok-okozati magyarázat. A belőle képzett összegzés például a mért modellkártyán: a corrupt környezetben a vertical→diagonal tévesztések száma 8, ebből a `G0-0003` scene két nézetéből 1 származik (az egyik nézet diagonal predikciót kapott), a többi 7 más jelenetekből. Az ilyen mondat leíró állítás; azt, hogy miért éppen az a scene hibázik, külön vizsgálat nélkül nem állítjuk.

Definíció (döntési határ). A modell logitokat ad: $z(x) \in \mathbb{R}^K$, és

$$\hat{y}(x) = \arg\max_k z_k(x).$$

A döntési határ azon x pontok halmaza, ahol az argmax nem egyértelmű, azaz létezik $k \neq j$, hogy $z_k(x) = z_j(x) = \max_l z_l(x)$. Egy pont akkor van „a határhoz közel”, ha a két legnagyobb logit különbsége kicsi. Ha a shift az input eloszlást a határ felé tolja, a hibavalószínűség nőhet. Ez utóbbi heurisztika: abból, hogy egy szelet hibái a határhoz közeliek, nem következik bizonyítottan, hogy a shift okozta őket.

23.4.4 Összesített pontosság és szeletszerződés: illusztráció

Állítás. Legyen D teszt-indexhalmaz, $|D| = N$, és $S \subseteq D$ nemüres. Ha minden $i \in S$ téves, akkor

$$\mathrm{acc}(D) \leq 1 - \frac{|S|}{N}.$$

Bizonyítás. A hibák száma $E_D = \sum_{i \in D} \mathbb{1}[\hat{y}_i \neq y_i]$. Feltétel szerint $E_D \geq |S|$. Ezért

$$\mathrm{acc}(D) = 1 - \frac{E_D}{N} \leq 1 - \frac{|S|}{N}.$$

Egyenlőség pontosan akkor áll fenn, ha $E_D = |S|$, vagyis a $D \setminus S$ halmaz minden eleme helyes. Az állítás éles esete: $N = 100$, $|S| = 5$ mellett, ha S minden eleme hibás és a többi 95 helyes, akkor $\mathrm{acc}(D) = 0{,}95$, miközben $\mathrm{acc}(S) = 0$. A globális pontosság tehát nem garantál semmilyen alsó korlátot egy tetszőleges szeletre; ha a szeletek diszjunkt partíciót alkotnak D-nek, a globális érték a szeletpontosságok méret szerint súlyozott átlaga, az átlag pedig elfedheti a kis súlyú, teljesen hibás csoportot.

23.4.5 Kézi példa és minimális kód

Legyen $N = 10$, $K = 3$, és két szelet: a `clean` az első 8, a `corrupt` az utolsó 2 index.

Szelet	Méret	Helyes	Pontosság
Összes	10	8	0,80
clean	8	8	1,00
corrupt	2	0	0,00

A globális 0,80 mellett a corrupt szelet 0,00; ez a fenti állítás egyenlőségi esete, hiszen $1 - 2/10 = 0{,}80$.

```
# workdir: lab/
import numpy as np

def slice_accuracy(y_true, y_pred, mask):
    """Visszaadja: (correct, size, acc); acc None, ha a szelet üres."""
    mask = np.asarray(mask, dtype=bool)
    n = int(np.sum(mask))
    if n == 0:
        return 0, 0, None
    correct = int(np.sum(y_true[mask] == y_pred[mask]))
    return correct, n, correct / n

y_true = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2, 2], dtype=np.int64)
y_pred = np.array([0, 0, 0, 1, 1, 1, 2, 2, 0, 0], dtype=np.int64)
clean = np.array([True] * 8 + [False] * 2)
corrupt = ~clean

print("clean  :", slice_accuracy(y_true, y_pred, clean))
print("corrupt :", slice_accuracy(y_true, y_pred, corrupt))
print("overall :", np.mean(y_true == y_pred))
```

Mért kimenet: `clean : (8, 8, 1.0)`, `corrupt : (0, 2, 0.0)` (mezőrend: `correct, size, acc`), `overall : 0.8`.

23.4.6 Labor: hibataxonómia és modellkártya

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és pandas 2.3.3. Szükséges fájlok: `data/metadata.csv`, `labels.npy`, `pred_labels.npy` és `corr_pred_labels.npy`; futtatás: `python make_model_card.py --pred pred_labels.npy --corr-pred corr_pred_labels.npy --out model_card.json --data-dir data`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

Cél. A hallgató a saját, befagyasztott GlyphLab-modelljének clean és corrupt predikcióiból szeletelemzést és hibataxonómiát készít, majd JSON modellkártyát állít elő. A teszt nem befolyásolhatja a modellválasztást: a checkpointot kizárólag a validációs loss alapján választottuk, a clean és corrupt kiértékelés ezután, egyszer történik.

Input. `data/` könyvtár a GlyphLab szerződés szerint: `metadata.csv` többek között a `index,scene_id,view_id,label,split` oszlopokkal, ahol az `index` a kép- és címketömb sorszáma (nem a CSV sorszáma); `labels.npy` alak `(360,)` int64, a `metadata.csv index` oszlopa szerint rendezve; `pred_labels.npy` alak `(60,)` int64 és `corr_pred_labels.npy` alak `(60,)` int64, mindkettő a `metadata.csv` test sorainak sorrendjében. A modellkártya rögzíti a futtató környezet Python-, NumPy- és pandas-verzióját is.

```
# workdir: lab/
# make_model_card.py
import argparse, json, sys
import numpy as np
import pandas as pd

def scene_resample_indices(meta, rng):
    idxs = []
    for cls in range(3):
        scenes = meta[meta["label"] == cls]["scene_id"].unique()
        chosen = rng.choice(scenes, size=len(scenes), replace=True)
        for sc in chosen:
            sub = meta[(meta["label"] == cls) & (meta["scene_id"] == sc)]
            idxs.append(sub.index.to_numpy())
    return np.concatenate(idxs)

def bootstrap_ci(meta, y_true, y_pred, rng, B=2000):
    # A meta indexe itt már pozicionális 0..n-1, hogy a 60 hosszú
    # y_true/y_pred tömbökkel összeegyeztethető legyen.
    meta = meta.reset_index(drop=True)
    if not (len(meta) == len(y_true) == len(y_pred)):
        raise ValueError("a meta, y_true és y_pred hossza eltér")
    stats = []
    for _ in range(B):
        idx = scene_resample_indices(meta, rng)
        stats.append(np.mean(y_true[idx] == y_pred[idx]))
    return float(np.percentile(stats, 2.5)), float(np.percentile(stats, 97.5))

def paired_bootstrap_ci(meta, y_true, y_clean, y_corr, rng, B=2000):
    # Ugyanazon jelenetekre párosított különbség: clean minus corrupt.
    meta = meta.reset_index(drop=True)
    if not (len(meta) == len(y_true) == len(y_clean) == len(y_corr)):
        raise ValueError("a meta, y_true, y_clean és y_corr hossza eltér")
    delta = ((y_true == y_clean).astype(np.float64)
             - (y_true == y_corr).astype(np.float64))
    stats = []
    for _ in range(B):
        idx = scene_resample_indices(meta, rng)
        stats.append(np.mean(delta[idx]))
    return float(np.percentile(stats, 2.5)), float(np.percentile(stats, 97.5))

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--pred", required=True)
    ap.add_argument("--corr-pred", required=True)
    ap.add_argument("--out", default="model_card.json")
    ap.add_argument("--data-dir", default="data")
    ap.add_argument("--seed", type=int, default=20261020)
    a = ap.parse_args()

    meta = pd.read_csv(f"{a.data_dir}/metadata.csv")
    y_all = np.load(f"{a.data_dir}/labels.npy")
    assert list(meta.columns) == ["index", "scene_id", "view_id", "label", "split",
                                  "xmin", "ymin", "xmax", "ymax"], meta.columns
    assert len(meta) == len(y_all) == 360, (len(meta), y_all.shape)
    assert meta["index"].tolist() == list(range(len(meta)))
    test_meta = meta[meta["split"] == "test"].reset_index(drop=True)
    row_idx = meta.loc[meta["split"] == "test", "index"].to_numpy()
    y_true = y_all[row_idx]
    # A clean és corrupt predikciós tömb a metadata test sorainak sorrendjében van.
    y_clean = np.load(a.pred).astype(np.int64)
    y_corr = np.load(a.corr_pred).astype(np.int64)
    if y_clean.shape != y_true.shape or y_corr.shape != y_true.shape:
```

```
        raise ValueError("a predikciók alakja nem egyezik a test sorainak számával")
    if np.any((y_clean < 0) | (y_clean >= 3)) or np.any((y_corr < 0) | (y_corr >= 3)):
        raise ValueError("a predikciók osztályindexe kívül esik a [0,3) tartományon")
    rng = np.random.default_rng(a.seed)

    card = {
        "selection": "best val loss only",
        "overall": {
            "clean_acc": float(np.mean(y_true == y_clean)),
            "corrupt_acc": float(np.mean(y_true == y_corr)),
            "paired_drop": float(np.mean(y_true == y_clean)
                                 - np.mean(y_true == y_corr)),
        },
        "slices": {},
        "failure_taxonomy": {},
        "environment": {
            "python": sys.version.split()[0],
            "numpy": np.__version__,
            "pandas": pd.__version__,
        },
        "status": "measured by the student run",
    }

    for cls in range(3):
        mask = (test_meta["label"] == cls).to_numpy()
        acc = np.mean(y_true[mask] == y_clean[mask])
        lo, hi = bootstrap_ci(test_meta[mask], y_true[mask],
                              y_clean[mask], rng)
        card["slices"][f"clean_class_{cls}"] = {
            "n": int(mask.sum()), "acc": float(acc), "ci95": [lo, hi]}

    lo, hi = bootstrap_ci(test_meta, y_true, y_corr, rng)
    card["slices"]["corrupt_all"] = {
        "n": len(y_true),
        "acc": float(np.mean(y_true == y_corr)),
        "ci95": [lo, hi]}

    # A döntéshez a páros, scene-szintű különbség intervalluma kell.
    dlo, dhi = paired_bootstrap_ci(test_meta, y_true, y_clean, y_corr, rng)
    card["overall"]["paired_drop_ci95"] = [dlo, dhi]

    for env, yp in [("clean", y_clean), ("corrupt", y_corr)]:
        C = np.zeros((3, 3), dtype=int)
        for ta in range(3):
            for pr in range(3):
                C[ta, pr] = int(np.sum((y_true == ta) & (yp == pr)))
        card["failure_taxonomy"][env] = C.tolist()

    with open(a.out, "w", encoding="utf-8") as f:
        json.dump(card, f, indent=2)
    print(card["overall"])

if __name__ == "__main__":
    main()
```

Futtatási parancs:

```
python make_model_card.py --pred pred_labels.npy \
  --corr-pred corr_pred_labels.npy --out model_card.json --data-dir data
```

Ellenőrzés. A JSON minden `acc` értéke $[0;1]$-ben legyen; minden `n` pozitív; a CI határok monotonok ($lo \le hi$). A confusion mátrixok sorösszege a clean esetben osztályonként 20, a

corrupt esetben ugyancsak 20. A referencia-modell (a `lab/train_cnn.py` 1299 paraméteres `SmallCNN`-je, seed 20261020) clean és corrupt tesztpredikcióival futtatva a mért JSON:


```
{
  "selection": "best val loss only",
  "overall": {"clean_acc": 0.95, "corrupt_acc": 0.35, "paired_drop": 0.6,
              "paired_drop_ci95": [0.55, 0.65]},
  "slices": {
    "clean_class_0": {"n": 20, "acc": 1.0, "ci95": [1.0, 1.0]},
    "clean_class_1": {"n": 20, "acc": 1.0, "ci95": [1.0, 1.0]},
    "clean_class_2": {"n": 20, "acc": 0.85, "ci95": [0.7, 1.0]},
    "corrupt_all": {"n": 60, "acc": 0.35, "ci95": [0.3333333333333333, 0.38333333333333336]}
  },
  "failure_taxonomy": {
    "clean": [[20,0,0],[0,20,0],[0,3,17]],
    "corrupt": [[0,12,8],[0,20,0],[0,19,1]]
  },
  "environment": {"python": "3.12.3", "numpy": "2.5.4", "pandas": "3.1.0"},
  "status": "measured by the student run"
}
```


Értelmezés. A döntés alapja a scene-szintű páros különbség bootstrap CI-je: a mért `paired_drop` 0,6, a `paired_drop_ci95` [0,55; 0,65]. A döntési küszöb a 0 (a romlás előjele); a nulla kizárása a definiált korrupciós eloszláson kimutatott páros romlást jelent. Ez nem azonos egy gyakorlati jelentőségi küszöbbel: ha a szervezet előre rögzít egy minimálisan elfogadható romlást (például 0,1), akkor azt a CI alsó határához kell hasonlítani. A két külön intervallum átfedése önmagában nem dönt, és ez nem általános domain-shift garancia. A kis csoportméretű szeletek CI-je széles; azokból nagy állítást tenni tilos.

23.4.7 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

1. F1 fogalmi, nehézség 1. Definiálja a szelet pontosságát, és indokolja, miért nem jelentünk értéket üres szeletre. Adjon egy üres szelet példát a GlyphLab teszt metadata alapján. *Elvárt eredmény:* képlet, az üres eset szabálya, egy konkrét üres mask.

2. F2 levezetési/bizonyítási, nehézség 2. Bizonyítsa be a fenti Állítást, és adja meg az egyenlőség pontos feltételét. Ellenőrizze a $N = 100$, $|S| = 5$ példán, hogy a globális pontosság maximuma 0,95. *Elvárt eredmény:* teljes bizonyítás és a maximum indoklása.

3. F3 implementációs, nehézség 2. Írjon NumPy függvényt `slice_accuracy(y_true, y_pred, mask)` néven, amely üres szeletre `(0, 0, None)` értéket ad. Számítsa ki a kézi példa mindhárom sorát 64 bites egészekkel. *Elvárt eredmény:* kód, futtatási parancs, a három sor kimenete.

4. F4 hibakeresési, nehézség 2. Az alábbi függvény hibás. Bemenet: `y_true`, `y_pred`: `(N,)` int64 tömbök; `mask`: `(N,)` bool tömb. A javított függvény kimenete a szelet pontossága (float), üres szeletre `None`.

```
def slice_accuracy_bad(y_true, y_pred, mask):
    return np.mean(y_true == y_pred)  # a mask elveszik
```

Lokalizálja a hibát, írja le a következményét a modellkártyán, javítsa, és a kézi példán mutassa meg, hogy a helyes függvény a corrupt szeletre 0,0-t ad, a hibás pedig 0,8-at. *Elvárt eredmény:* ok, következmény, javított kód, ellenőrző futtatás kimenete.

5. F5 kísérleti/transzfer, nehézség 3. Tervezzen szeletelemzést a GlyphLab test `view_id` értékei szerint (0 és 1 nézet). Adja meg a megfigyelési egységet, a bootstrap mintavételi egységet, a baseline-t, a metrikát és a következtetési korlátot. *Elvárt eredmény:* legfeljebb fél oldal terjedelmű protokoll, amely kitér arra, hogy a két view ugyanazon scene-hez tartozik.

23.4.8 Megoldások és indoklás

1. A szelet pontossága $\mathrm{acc}(S) = \frac{1}{|S|}\sum_{i\in S}\mathbb{1}[\hat{y}_i = y_i]$, $|S| > 0$. Üres szeletre a nevező nulla, a tört definiálatlan; a 0-val jelölés hamis „minden rossz” állítást sugallna. Üres szelet a GlyphLab-ban például egy nem létező `scene_id` szűrése a testben (`metadata.scene_id == 999`), vagy egy olyan osztály–nézet kombináció, amely a tesztben nem fordul elő.

2. Az Állítás bizonyítása a fenti levezetés. Egyenlőség pontosan akkor, ha $E_D = |S|$, azaz $D \setminus S$ minden eleme helyes. A 100/5 példán: a hibaszám legalább 5, ezért a pontosság legfeljebb $1 - 5/100 = 0{,}95$; ez elérhető, ha a maradék 95 mind helyes, tehát a maximum pontosan 0,95.

3. A kézi példa függvénye a fenti kód. A példa tíz mintából áll, ezért a shape $(10,)$; a tömbök dtype-ja `np.int64`. A mért kimenet:

```
clean   : (8, 8, 1.0)
corrupt : (0, 2, 0.0)
overall : 0.8
```

4. A hiba oka: a függvény nem alkalmazza a maskot, így a slice-ra vonatkozó kérdésre a teljes minta pontosságát adja. Következmény: a modellkártya minden szeletsorában ugyanaz a globális érték jelenik meg, a kis hibás szelet eltűnik. Javítás a 3. feladat `slice_accuracy` függvénye. A kézi példán a hibás függvény a corrupt maskra is 0,8-at ad, a helyes pedig $(0; 2; 0{,}0)$-t.

5. F5 megoldás. Protokoll: split: a rögzített test 60 képe; megfigyelési egység: kép, de a bootstrap mintavételi egység scene_id, osztályonként rétegezve (osztályonként 10 scene visszatevéssel). Csoportosítás: `view_id == 0` és `view_id == 1`, mindkét csoport mérete 30. Baseline: a clean teszt teljes pontossága. Metrika: szeletpontosság és scene-bootstrap 95% CI; a két view összehasonlítása páros, mert minden scene mindkét view-t tartalmazza, ezért a view-különbségre scene-szintű páros bootstrap vagy előjel-teszt (a páros különbségek előjelére épülő, eloszlásmentes teszt) kell. Következtetési korlát: a döntést a scene-szintű páros különbség CI-je adja; ha az tartalmazza a nullát, nem állítunk view-függő különbséget, és a két külön intervallum átfedése nem helyettesíti ezt a páros döntést; a view nem új független eloszlás, hanem ugyanazon jelenet két zajrealizációja.

Változó: a `view_id`. **Hipotézis, seedek, döntési szabály és erőforrás:** a hipotézis, hogy a két nézet pontossága eltér, miközben ugyanazokat a jeleneteket tartalmazza. A bootstrap seedje 20261020, $B = 2000$; a páros előjel-teszt seedje szintén 20261020. A döntési szabály számszerű: a view-hatást akkor jelentjük, ha a jelenetszintű páros különbség 95%-os bootstrap CI-je kizárja a 0-t, és a pontbecslés abszolút értéke legalább 2 százalékpont; ellenkező esetben nem állítunk view-függő különbséget. A feladat nem tanít modellt, ezért architektúra és optimalizáló helyett a meglévő predikciók kontrollja és a döntési szabály szerepel. Erőforrás: CPU, a meglévő 60 tesztpredikción, néhány másodperc. ### Tipikus tévedés és önellenőrzés

Tipikus tévedés. Kis szeletből nagy általános állítást tenni. Példa: egy 5 elemű corrupt szeleten mért 0% pontosságból azt állítani, hogy „a modell használhatatlan a corrupt környezetben”. Kis csoporton a percentilis bootstrap elfajulhat (0/5 esetén minden replikátum 0, így a CI [0, 0]),

ezért pontos binomiális (Clopper–Pearson) intervallumot kell használni: 0/5 mellett a kétoldalú 95%-os intervallum [0; 52,2%]. Ez azt jelenti, hogy a valódi pontosság akár 50% körül is lehet; ha a valódi pontosság 0,5, egy másik, független 5 elemű mintán a 4/5 találat valószínűsége $5 \cdot 0{,}5^5 \approx 0{,}156$, tehát a 80%-os minta nem zárható ki. A korrekt állítás: „a pontosság becslése 0, de a 95%-os Clopper–Pearson CI [0; 52,2%], ezért a használhatatlanság nem igazolt”.

Önellenőrzés. (1) A szelet definíciója a teszt indexeire épül-e, nem a tanítóra? (2) Üres szelet esetén `None` szerepel-e? (3) A CI scene-szintű bootstrapből származik-e? (4) A corrupt kiértékelés a befagyasztott modellen, modellválasztás után történt-e? Ha bármelyik nem, a modellkártya állítása nem érvényes.

Rövid válaszok. (1) Igen: a szeletet a teszt indexein definiáljuk, a tanítóhalmazon nem. (2) Igen: üres szeletre `None` szerepel, nem 0, mert a 0 hamis „minden rossz” állítást sugallna. (3) Igen: a CI scene-szintű, osztályonként rétegzett bootstrapből származik. (4) Igen: a corrupt kiértékelés a befagyasztott modellen, a modellválasztás után történik.

Visszautalás. A 23.3. Páros összehasonlítás és csoportszintű intervallum bootstrap egysége a scene_id volt, osztályonkénti rétegzéssel; ezt a szeletek CI-jében is megtartottuk. A 22.4. Augmentációs kísérlet és domainváltozás clean/corrupt eloszlás-megkülönböztetése adja a környezeti szeletet. A 11.4. Egy teljes PyTorch tanítási iteráció logit–döntés szerződése a döntési határ definíciójának alapja.

23.4.9 Rövid összefoglaló

Az induló kérdésre a válasz igen: jó globális érték mellett is létezhet nulla pontosságú kis szelet, mert a hiba koncentrálódhat. Az összesített pontosság diszjunkt szeletek esetén a szeletek feletti súlyozott átlag. A modellkártya ezért clean/korrupt, osztály- és nézetszintű bontást, csoportméreteket és scene-rétegzett bootstrap intervallumokat közöl. A kis csoportból levont állítás a széles CI miatt bizonytalan. A corrupt teszt csak a befagyasztott modell kiválasztása után értékelhető, és a shift csupán az adott korrupciós eloszlást reprezentálja. A hibataxonómia a hibákat cella, környezet és csoport szerint rendezi; a döntési határ közelsége heurisztika, nem bizonyított ok-okozat.

A következő 24.1. Tanulási görbe és hibadiagnózis egység a tanulási görbét és a hibadiagnózist vizsgálja; az itteni szeleteredmények ott azt segítik eldönteni, hogy a hiba adat-, modell- vagy optimalizálási eredetű-e.

23.4.10 Felhasznált források

- [S04] Efron, B. és Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

24 Hibakeresés, profiling és számítási erőforrás

24.1 Tanulási görbe és hibadiagnózis

24.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben a tanítási futás közben keletkező diagnosztikai jelekből lokalizálunk hibát. Nem egyetlen „jó receptet” adunk, hanem olyan gondolkodási keretet, amely minden jelhez több lehetséges okot rendel, és minden okhoz egy diszkrimináló ellenőrzést (olyan mérés, amely a lehetséges okok közül csak az egyiket támogatja). A 12.4. III. ellenőrző pont: önállóan tanuló kis háló alfejezetben felépített kis MLP-tanítás, a 21.4. Osztályarány, súlyozás és céleloszlás alfejezetben tárgyalt céleloszlás, a 22.4. Augmentációs kísérlet és domainváltozás alfejezetben bevezetett clean (tiszta) és corrupt (korruptált) teszt elkülönítése és a 23.4. Hibaszeletek és alkalmazási határ alfejezetben tárgyalt hibaszeletek szolgálnak alapul.

Induló kérdés. Egy GlyphLab-futás (a függelék A. A GlyphLab adatkészlet szakasza szerinti adaton) train loss-a 20 lépés után is 1,0986 körül mozog, ahelyett, hogy csökkenne. Ez a háromosztályos probléma véletlen szintje, $\ln 3$; a gyakran idézett $0{,}693 = \ln 2$ csak kétosztályos esetben érvényes. Mely okok jöhetnek szóba, és milyen méréssel lehet őket szétválasztani? A választ a diagnosztikai nem-injektivitás szakasza rendszerezi; előzetes becslésként érdemes már most felsorolni legalább három, egymástól független jelöltet.

24.1.2 Diagnosztikai jelek: definíció és jelölés

Legyen a modell paramétervektora $\theta \in \mathbb{R}^P$, ahol P a tanítható skalárok száma. A t-edik lépésben egy B_t batchre (a batch mintáinak indexhalmazára) számolt átlagos veszteség $L_t = \frac{1}{|B_t|} \sum_{i \in B_t} \ell(f(x_i;\theta_t), y_i)$; itt $|B_t|$ a batch-méret. A teljes epoch-veszteség a batchenkénti átlagok mintaszám szerint súlyozott átlaga: $\bar{L}_e = \frac{1}{N} \sum_{b=1}^{M_e} n_b \bar{L}_b$, ahol N az epoch teljes mintaszáma, M_e az epoch batcheinek száma, n_b a b-edik batch mintaszáma, $\bar{L}_b$ pedig a b-edik batch átlagos vesztesége; így $\sum_b n_b = N$. Ezt a 12.1. Tanítás, validáció és kiértékelési mód vezette le.

Definíció (tanulási görbe). A train loss görbe a $t \mapsto L_t$ sorozat; a val loss görbe ugyanez a modellfrissítéstől független val spliten, rögzített modellállapot mellett számolva.

Definíció (gradiensnorma-jel). $g_t = \nabla_\theta L_t \in \mathbb{R}^P$, és a jel $\|g_t\|_2 = \left(\sum_{j=1}^{P} g_{t,j}^2 \right)^{1/2}$. A teljes gradiensnorma egyetlen összevont szám; rétegenkénti bontásban $\|g_t^{(k)}\|$ jelöli a k-adik réteg paramétereire vett normát.

Definíció (paraméterváltozás-jel). $\Delta_t = \|\theta_{t+1} - \theta_t\|_2$. Tiszta SGD esetén $\Delta_t = \eta \|g_t\|_2$; Adam esetén ez nem áll, mert a momentum és a második momentum normalizálja a lépést.

Definíció (tiny overfit). Egy kis, rögzített tanítómintán (a 12.3. Sanity check és kis mintára túllillesztés szerint) futtatott sanity check, amely azt ellenőrzi, hogy a modell és a csővezeték képes-e memorizálni ezt a kis mintát: a train loss az előre rögzített küszöb alá csökken a szintén előre rögzített lépésszámon belül. A küszöb és a lépésszám a sanity check protokolljának

része (például a mintaátlagos veszteség 1%-a, adott lépésszám), nem utólag választott érték. A validációs viselkedés nem része a definíciónak és nem követelmény; a „train csökken, val nő” mintázat általános túltanulás, nem tiny overfit.

A továbbiakban minden diagnosztikai állítás a jel *osztályára* vonatkozik, nem egyetlen lépés pillanatnyi értékére: például „lapos” alatt $|L_t - L_0| \le \varepsilon$ teljesülését értjük előre megadott ε küszöbbel az első T lépésben.

24.1.3 A jel nem azonos az okkal: diagnosztikai nem-injektivitás

Állítás (diagnosztikai nem-injektivitás). Legyen $\mathcal{R}$ a futtatások (modell, adat, optimizer, seed) halmaza, és legyen $\Phi : \mathcal{R} \to \mathbb{R}^T \times \mathbb{R}^T \times \mathbb{R}^T$ a vizsgált jel, $\Phi(r) = (L_t, \|g_t\|_2, \Delta_t)_{t=1}^T$. A Φ leképezés értékkészletén értelmezett kvalitatív osztályozás nem határozza meg egyértelműen az okot: ugyanahhoz a jelosztályhoz több, mechanikailag különböző ok tartozhat.

Bizonyításvázlat (szemléltetéssel). Az állításhoz jelosztályonként két, mechanikailag különböző okot kell megadni, amelyek ugyanahhoz a kvalitatív jelhez vezetnek. Az alábbi táblázat ezeket a jelölteket rendszerezi (a listák nem teljesek), és minden okhoz diszkrimináló ellenőrzést rendel. A lapos loss esetére a kézi példa szakasza numerikus konstrukciót is ad; a többi jelosztályra a táblázat mechanizmusai szemléltetésként szolgálnak, nem teljes konstrukciós bizonyításként.

Jel	Lehetséges ok (nem teljes lista)	Diszkrimináló ellenőrzés
Lapos train loss	kis tanulási ráta; vanishing gradiens; befagyasztott súly; label bug (a tanítócímkék hibás kódolása)	Δ_t nagysága; rétegenkénti $\|g_t^{(k)}\|$; `p.grad is None`; target-eloszlás
Train loss csökken, val loss nő	túl nagy kapacitás (általános túltanulás); train/val eloszláskülönbség; kis vagy nem reprezentatív val split	train/val loss-gap; modellméret-abláció; split-ellenőrzés
NaN vagy inf loss	bemeneti NaN/inf; túl nagy η; log(0)/0/0 a lossban; tanulási ráta divergencia	input `isfinite`; gradnorm; fokozatos η-csökkentés
Eltűnő/robbanó gradiens	aktivációk szaturációja; mély szorzat a láncszabályban; inicializálás	rétegenkénti aktivációstatisztika; gradnorm rétegenként
A paraméter nem változik	`requires_grad=False`; hiányzik az optimizer paraméterlistájából; $\eta = 0$; `detach()` a gráfban, vagy a forward nem használja a paramétert	`named_parameters`; `optimizer.param_groups`; `p.grad is None`; Δ_t

A lapos loss egy numerikus konstrukciója: legyen $L(\theta) = \frac{1}{2}(\theta - 2)^2$, $\theta_0 = 10$. (a) $\eta = 10^{-6}$ mellett a gradiens $g_t = \theta_t - 2 = 8$, tehát $\Delta_t = 8 \cdot 10^{-6}$, és L_t az első lépésekben $|L_t - L_0| \le 10^{-4}$ értékkel gyakorlatilag lapos. (b) Ugyanaz a veszteség, de szándékosan skálázott, eltűnő gradienssel $g_t = 10^{-8}(\theta_t - 2)$ és $\eta = 0{,}1$ mellett $\Delta_t = 8 \cdot 10^{-9}$, a veszteség még laposabb. A két ok mechanikailag különböző (optimizer-skála kontra gradiensskála), a jel kvalitatíve azonos. Ez ellenpélda arra a téves következtetésre, hogy „lapos loss = nincs tanulási kapacitás”.

A vanishing/exploding (eltűnő/robbanó) gradiens mechanizmusa a 9.4. Aktivációk és gradiensáramlás alfejezetben bevezetett láncszabály közvetlen következménye. Ha például $h^{(k+1)} = W^{(k)} h^{(k)}$ lineáris rétegek kompozíciójában a backpropagált jel a $\prod_k W^{(k)}$ szorzatokon halad át, akkor egyenletes $\|W^{(k)}\| \le q < 1$ minden k-ra a szorzat normáját q^L-lel exponenciálisan

korlátozza (L rétegre). Véges, rögzített L esetén a $\|W^{(k)}\| < 1$ feltétel önmagában is elég: $q = \max_{k \le L} \|W^{(k)}\| < 1$ automatikusan egyenletes korlát. A mélységtől független korláthoz viszont valóban kell egy közös $q < 1$: a $q_k = k/(k+1)$ választásnál minden tényező egynél kisebb, a szorzat mégis csak $\prod_{k=1}^{L} q_k = 1/(L+1)$, azaz polinomiálisan, nem exponenciálisan csökken. Fordítva, a $\|W^{(k)}\| > 1$ nem garantálja a szorzat növekedését: $\mathrm{diag}(2; 0{,}1) \cdot \mathrm{diag}(0{,}1; 2) = 0{,}2\, I$, noha mindkét tényező spektrális normája 2. Robbanáshoz a lépések irányának illeszkednie kell (a mátrixok ugyanazon szinguláris alterekre hatnak), vagy a legkisebb szinguláris érték alsó korlátja ($\sigma_{\min}(W^{(k)}) \ge q > 1$) kell. A rétegenkénti gradiensnorma-mérés ezért diszkriminál: ha a korai rétegek normája nagyságrendekkel kisebb a későiekénél, akkor a vanishing-hipotézis erősödik; ha minden rétegben nagy, akkor inkább a learning rate vagy a loss skálázása gyanús.

24.1.4 Kézi példa: négy kvalitatív görbe, eltérő okokkal

Tekintsük az $L(\theta) = \frac{1}{2}(\theta - 2)^2$ veszteséget és a tiszta SGD-lépést. A gradiens $g_t = \theta_t - 2$.

1. **Egészséges:** $\theta_0 = 10$, $\eta = 0{,}5$. A lépések:

$$\theta_1 = 10 - 0{,}5 \cdot 8 = 6, \quad L_1 = 8; \qquad \theta_2 = 6 - 0{,}5 \cdot 4 = 4, \quad L_2 = 2;$$

$$\theta_3 = 4 - 0{,}5 \cdot 2 = 3, \quad L_3 = 0{,}5; \qquad \theta_4 = 3 - 0{,}5 \cdot 1 = 2{,}5, \quad L_4 = 0{,}125.$$

   A görbe monoton csökken.

2. **Lapos, kis η:** ugyanez $\eta = 10^{-6}$-tal; $\theta_1 = 9{,}999992$, $L_1 \approx 31{,}999936$ ($L_0 = 32$), a változás $6{,}4 \cdot 10^{-5}$.

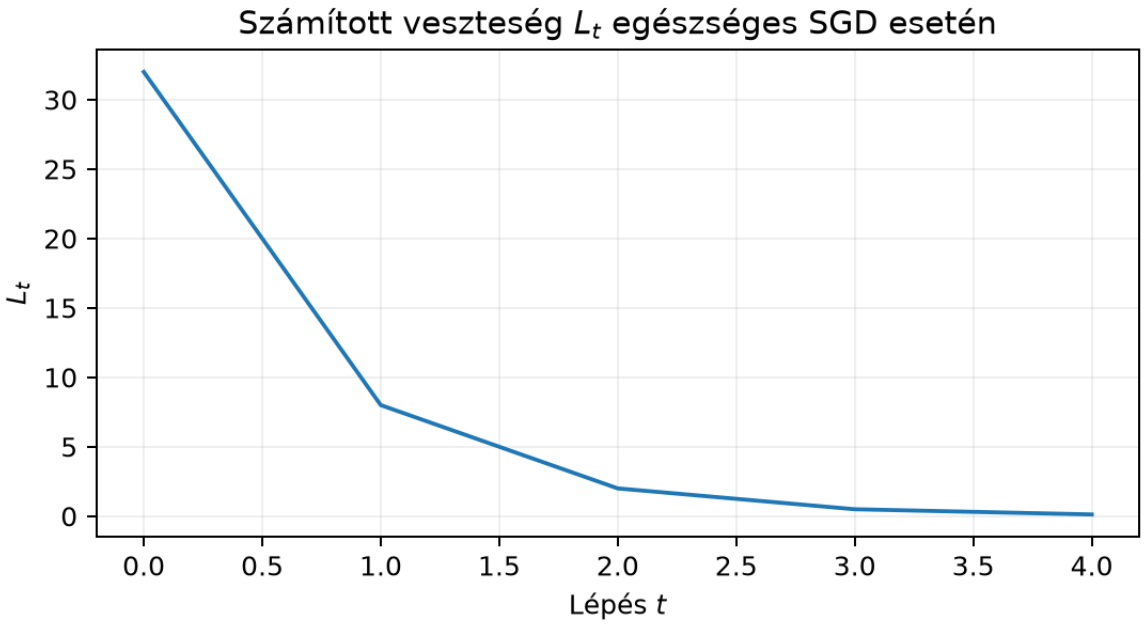
3. **Oszcilláló:** $\eta = 2$. Ekkor

$$\theta_{t+1} = \theta_t - 2(\theta_t - 2) = 4 - \theta_t,$$

   tehát $\theta_0 = 10$ esetén $\theta_1 = -6$, és θ_t a 10 és -6 értékek között ugrál, a hiba $|\theta_t - 2| = 8$ állandó marad. A loss nem csökken, de nem is divergál.

4. **Divergens:** $\eta = 2{,}1$. A hiba-rekurzió $e_{t+1} = \theta_{t+1} - 2 = (1 - \eta)(\theta_t - 2)$, tehát $|e_t| = |1 - \eta|^t |e_0| = 1{,}1^t \cdot 8$, exponenciálisan nő. A train loss ebben a tiszta kvadratikus esetben véges, de a gradiensnorma exponenciálisan nő; valódi hálóban előbb-utóbb float32 túlcsorduláshoz és NaN-hez vezet.

A 2. eset (kis η) és a diagnosztikai nem-injektivitás szakaszában skálázott, eltűnő gradienssel leírt konstrukció ugyanazt a lapos jelet adja: az egyiket az optimizer léptéke, a másikat a gradiens skálája okozza.



24.1. ábra: Számított veszteség L_t egészséges SGD esetén

VÁRT ÉRTÉK (levezetésből, nem mért). A vízszintes tengely az SGD-lépés száma, a függőleges a számított veszteség; a görbe a kézi példa rekurziójából adódik.

24.1.5 Labor: három szándékos traininghiba lokalizálása

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. A futtatás a szakaszban megadott szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor három, egymástól függetlenül bekapcsolt hibát vizsgál a GlyphLab adaton, egy kis MLP-vel (576 bemenet a 24×24 képből, 32 rejtett egység, 3 logit), és egy hibamentes kontrollfuttatással hasonlítja össze őket. Minden futtatás CPU-n, float32-ben, 1 szálon fut (a reprodukálhatóság feltételeit a [D02] rögzíti); a seedet minden futtatás elején újra beállítjuk, így a négy futás azonos inicializálást és minibatch-sorrendet kap, és csak a vizsgált hiba tér el. A modellválasztás itt *nem* történik meg: a futtatások diagnosztikaiak, és egyiket sem használjuk végleges modellként.

A szkript ezen a környezeten (Python 3.12.3, torch 2.15.0+cpu, CPU float32, 1 szál, seed 20261020) lefutott; a tanulság, hogy a nagy learning rate jelaláírása nem garantált (lásd a 3. pontot). A hallgató a script futtatásával kapja a saját naplóját.

```
# lab/n093_diagnose.py
# Futtatás a repository gyökeréből: python lab/n093_diagnose.py
import csv
import json
import math
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
DATA = Path("lab/data")
DEVICE = "cpu"
DTYPE = torch.float32
SEED = 20261020
torch.set_num_threads(1)

images = torch.from_numpy(np.load(DATA / "images.npy")).to(device=DEVICE, dtype=DTYPE)
labels = torch.from_numpy(np.load(DATA / "labels.npy")).to(device=DEVICE, dtype=torch.int64)

with open(DATA / "metadata.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
# A sorrendre nem támaszkodunk: a saját index oszlopa adja a tömbindexet.
if not rows or "index" not in rows[0]:
    raise ValueError("a metadata.csv-ből hiányzik az index oszlop")
if len(rows) != len(images):
    raise ValueError("a metadata.csv és a képtömb hossza eltér")
all_idx = np.array([int(r["index"]) for r in rows], dtype=np.int64)
if not np.array_equal(np.sort(all_idx), np.arange(len(images))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")
train_idx = [int(r["index"]) for r in rows if r["split"] == "train"]
val_idx = [int(r["index"]) for r in rows if r["split"] == "val"]

X_train = images[train_idx].reshape(len(train_idx), -1)
y_train = labels[train_idx]
X_val = images[val_idx].reshape(len(val_idx), -1)
y_val = labels[val_idx]

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(576, 32)
        self.act = nn.ReLU()
        self.fc2 = nn.Linear(32, 3)
    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))

def run_case(y_train_used, freeze_first=False, lr=0.5, steps=400, bs=64):
    # Minden futtatás ugyanazt az inicializálást és minibatch-sorrendet kapja:
    # a seedet ezért itt, a futtatás elején állítjuk be.
    torch.manual_seed(SEED)
    model = TinyMLP().to(DEVICE)
    if freeze_first:
        model.fc1.requires_grad_(False)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    hist = {"loss": [], "grad_norm": [], "delta": []}
    params_before = [p.detach().clone() for p in model.parameters()]
    for step in range(steps):
        idx = torch.randint(0, X_train.shape[0], (bs,), device=DEVICE)
        xb, yb = X_train[idx], y_train_used[idx]
        opt.zero_grad(set_to_none=True)
        logits = model(xb)
        loss = F.cross_entropy(logits, yb)
        loss.backward()
        gn = 0.0
        for p in model.parameters():
            if p.grad is not None:
                gn += p.grad.detach().pow(2).sum().item()
        gn = math.sqrt(gn)
        opt.step()
        delta = 0.0
        for p, p0 in zip(model.parameters(), params_before):
            delta += (p.detach() - p0).pow(2).sum().item()
```

```
            delta = math.sqrt(delta)
            hist["loss"].append(loss.item())
            hist["grad_norm"].append(gn)
            hist["delta"].append(delta)
            params_before = [p.detach().clone() for p in model.parameters()]
    return model, hist

@torch.inference_mode()
def evaluate(model, X, y):
    model.eval()
    logits = model(X)
    pred = logits.argmax(dim=1)
    return (pred == y).float().mean().item()

def class_counts(y, num_classes=3):
    return torch.bincount(y, minlength=num_classes).tolist()

print("cél-eloszlás (train):", class_counts(y_train))
print("cél-eloszlás (val):", class_counts(y_val))

# 0. kontroll: ép futtatás; a kontroll-accuracyt a tanított modellen mérjük.
model_ok, hist_ok = run_case(y_train)
acc_ok = evaluate(model_ok, X_val, y_val)

# 1. label bug: minden cél 0
y_bad = torch.zeros_like(y_train)
print("cél-eloszlás (label bug, train):", class_counts(y_bad))
model_bad, hist_bad = run_case(y_bad)
acc_bad = evaluate(model_bad, X_val, y_val)

# 2. frozen first layer
model_fr, hist_fr = run_case(y_train, freeze_first=True)
acc_fr = evaluate(model_fr, X_val, y_val)
frozen_grad_none = model_fr.fc1.weight.grad is None

# 3. large learning rate
model_lr, hist_lr = run_case(y_train, lr=3.0)
acc_lr = evaluate(model_lr, X_val, y_val)

assert len(hist_ok["loss"]) == len(hist_bad["loss"]) == 400
assert len(hist_fr["loss"]) == len(hist_lr["loss"]) == 400

summary = {
    "target_counts_train": class_counts(y_train),
    "target_counts_val": class_counts(y_val),
    "target_counts_label_bug": class_counts(y_bad),
    "healthy_last_loss": hist_ok["loss"][-1],
    "healthy_val_acc": acc_ok,
    "healthy_grad_norm_last": hist_ok["grad_norm"][-1],
    "healthy_delta_last": hist_ok["delta"][-1],
    "label_bug_last_loss": hist_bad["loss"][-1],
    "label_bug_val_acc": acc_bad,
    "frozen_first_grad_none": frozen_grad_none,
    "frozen_last_loss": hist_fr["loss"][-1],
    "frozen_val_acc": acc_fr,
    "large_lr_loss_first": hist_lr["loss"][0],
    "large_lr_loss_last": hist_lr["loss"][-1],
    "large_lr_max_grad_norm": max(hist_lr["grad_norm"]),
    "large_lr_delta_max": max(hist_lr["delta"]),
    "large_lr_any_nonfinite": not all(math.isfinite(x) for x in hist_lr["loss"]),
    "large_lr_val_acc": acc_lr,
    "seed": SEED,
    "device": DEVICE,
```

```
    "dtype": str(DTYPE),
}
out = Path("lab/n093_summary.json")
out.write_text(json.dumps(summary, indent=2), encoding="utf-8")
print(json.dumps(summary, indent=2))
```

A 0. eset a hibamentes kontroll: a tanított `model_ok` modellen méri a val accuracyt, és a summary-ben rögzíti az egészséges baseline-t (loss, gradiensnorma, paraméterváltozás). A szkript mért kimenete (Python 3.12.3, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020):


```
{
  "target_counts_train": [
    80,
    80,
    80
  ],
  "target_counts_val": [
    20,
    20,
    20
  ],
  "target_counts_label_bug": [
    240,
    0,
    0
  ],
  "healthy_last_loss": 0.009282447397708893,
  "healthy_val_acc": 0.5,
  "healthy_grad_norm_last": 0.021640807847234347,
  "healthy_delta_last": 0.010820390086272277,
  "label_bug_last_loss": 7.182056288002059e-05,
  "label_bug_val_acc": 0.3333333432674408,
  "frozen_first_grad_none": true,
  "frozen_last_loss": 1.000915765762329,
  "frozen_val_acc": 0.28333333134651184,
  "large_lr_loss_first": 1.114625334739685,
  "large_lr_loss_last": 1.0842208862304688,
  "large_lr_max_grad_norm": 1.5371289912405668,
  "large_lr_delta_max": 4.61138689778418,
  "large_lr_any_nonfinite": false,
  "large_lr_val_acc": 0.3333333432674408,
  "seed": 20261020,
  "device": "cpu",
  "dtype": "torch.float32"
}
```


A három vizsgált hiba jelaláírása:

- **Label bug.** A train loss gyorsan csökken (a mért futásban $7{,}18 \cdot 10^{-5}$-ra), mert a modell megtanulja a konstans 0. osztályt; a val accuracy a 0. osztály prevalenciájához, 1/3-hoz közelít (mért érték 0,333), mert a val split is kiegyensúlyozott. A hipotézis ellenőrzése: a target-eloszlás kiíratása; a szkript a train spliten `[240, 0, 0]`, a val spliten `[20, 20, 20]` értéket mutat.
- **Frozen weight.** A befagyasztott réteg `p.grad` értéke `None` minden lépésben; a train loss a kezdeti érték közelében marad (mért érték 1,001), mert a fej véletlen súlyokkal nem tudja a bemeneti reprezentációt hasznosítani; a val accuracy 0,283. Ez a jel a 20.1. Feature extractor és új fej alfejezetbeli mechanizmus közvetlen megfigyelése.
- **Large learning rate.** A gradiensnorma nagy lépéseknél megnőhet, és a loss oszcillálhat; a NaN/Inf azonban nem garantált. A fenti szkript a mért futásban $\eta = 3{,}0$ mellett 400 lépés alatt végig véges maradt (`large_lr_any_nonfinite: false`): a loss 1,115 és 1,084 között

maradt, a gradiensnorma maximuma 1,537, a paramétervváltozás maximuma pedig 4,611 volt, szemben az egészséges kontroll 0,0108-as utolsó lépésével. A jelet tehát nem a NaN, hanem a loss laposssága és a kirívóan nagy Δ_t együtt azonosítja. A hipotézis ellenőrzése: ugyanaz a futás $\eta = 0{,}5$ értékkel (az egészséges kontroll beállítása) csökkenő loss-t ad.

Az elvárt eredmény a `lab/n093_summary.json` napló és egy rövid hipotézisnapló, amely mindhárom hibához rögzíti a tesztelt hipotézist, a mért jelet és a diszkrimináló ellenőrzést. A corrupt teszt itt nem használható: a 22.4. Augmentációs kísérlet és domainváltozás szerint csak a befagyasztott végleges modellen, egyszer értékelhető.

24.1.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Definiáld a gradiensnorma-jelet, és magyarázd el két-három mondatban, miért nem elégséges önmagában a „vanishing gradiens” diagnózis kimondásához. Elvárt eredmény: definíció alakkal, és legalább egy másik ok, amely ugyanazt a kis gradienst produkálja.

F2 (levezetési, nehézség 2). Az $L(\theta) = \frac{a}{2}(\theta - b)^2$ veszteségen, $a > 0$, tiszta SGD mellett vezesd le a hiba-rekurziót $e_t = \theta_t - b$-re, és bizonyítsd be: $\eta = 2/a$ esetén $|e_t|$ állandó; $\eta > 2/a$ esetén $|e_t|$ exponenciálisan nő. Elvárt eredmény: minden lépés, a rekurzió zárt alakja, a két eset külön.

F3 (implementációs, nehézség 2). Írj `classify_run(loss_hist, grad_norm_hist, eps=1e-4)` függvényt, amely a következő osztályokat adja vissza: `"healthy"`, `"flat"`, `"nan"`, `"diverging"`. A szabályok: bármely nem véges érték `"nan"`; ha az első gradiensnorma pozitív, és az utolsó érték több mint 10^3-szorosa az elsőnek, akkor `"diverging"`; ha minden t-re $|L_t - L_0| \le \varepsilon$, akkor `"flat"`; egyébként `"healthy"`. Nulla első gradiensnorma esetén a hányados nem értelmezhető, ezt az esetet a függvény nem sorolhatja a divergens ágba. Teszteld a kézi példa három numerikus esetén, és külön mutasd meg, hogy a divergens rekurzió 10 lépése még nem, 80 lépése viszont átlépi a 10^3-szoros küszöböt. Elvárt eredmény: a függvény teljes kódja, a tesztbemenetek és a visszatérési értékek.

F4 (hibakeresési, nehézség 2). Az alábbi részlet legalább három független hibát tartalmaz (a veszteség, a bemeneti alak és a címkeváltozó terén). Azonosítsd mindhármat, add meg a következményüket, és javítsd ki a kódot.

```
loss = F.cross_entropy(F.softmax(logits, dim=1), labels)
# ...
logits = model(images.reshape(images.shape[0], 1, -1))  # images: (N,1,H,W)
# labels_transposed valójában y_train.clone() volt egy korábbi blokkban
labels = labels_transposed  # valahol labels_transposed = labels.T
```

Elvárt eredmény: a három hiba megnevezése, a várható tünet, a javított sorok.

F5 (kísérlettervezési, nehézség 3). Tervezz kísérletet annak eldöntésére, hogy egy val loss plató oka kapacitáskorlát vagy label noise. Adj protokollt: kérdés, hipotézisek, megfigyelési egység, split, baseline, változó tényező, metrika, erőforrás-korlát, érvényességi határ. Elvárt

eredmény: legalább két modellméret, a train/val split rögzítése, a label noise injektálásának módja, és annak indoklása, miért nem használható a test a döntéshez.

24.1.7 Megoldások és indoklás

F1 megoldás. A gradiensnorma-jel $\|g_t\|_2 = \left(\sum_j (\nabla_{\theta_j} L_t)^2\right)^{1/2}$. A vanishing gradiens diagnózisához rétegenkénti bontás kell: ha a korai rétegek normája nagyságrendekkel kisebb, mint a későieké, akkor a láncszabály-szorzat csökkenése a valószínű ok. Ha viszont minden rétegben kicsi a gradiens, az lehet kis tanulási ráta, a loss konstans tagja, befagyasztott súlyok vagy label bug is — a jel önmagában nem különbözteti meg ezeket.

F2 megoldás. $g_t = \nabla_\theta L = a(\theta_t - b) = a e_t$, így

$$\theta_{t+1} = \theta_t - \eta a e_t.$$

Kivonva b-t:

$$e_{t+1} = (1 - \eta a) e_t.$$

Indukcióval $e_t = (1-\eta a)^t e_0$. Ha $\eta = 2/a$, akkor $1 - \eta a = -1$, tehát $|e_t| = |e_0|$ állandó (ugrálás a két oldal között). Ha $\eta > 2/a$, akkor $|1 - \eta a| > 1$, tehát $|e_t| = |1-\eta a|^t |e_0|$ exponenciálisan nő. Alakellenőrzés: e_t skalár, a rekurzió dimenzióhelyes; a szélsőeset $\eta = 0$ ad $e_t = e_0$ állandót, ami konzisztens az „egyáltalán nem lépünk” esettel.

F3 megoldás. A függvény:

```
import math

def classify_run(loss_hist, grad_norm_hist, eps=1e-4):
    if len(loss_hist) != len(grad_norm_hist) or not loss_hist:
        raise ValueError("a két hisztória hossza tér el vagy üres")
    if any(not math.isfinite(x) for x in loss_hist + grad_norm_hist):
        return "nan"
    g0 = grad_norm_hist[0]
    if g0 > 0 and grad_norm_hist[-1] > 1e3 * g0:
        return "diverging"
    if max(abs(x - loss_hist[0]) for x in loss_hist) <= eps:
        return "flat"
    return "healthy"
```

Teszt: a kézi példa egészséges görbéje `([32,8,2,0.5,0.125], [8,4,2,1,0.5])` `"healthy"`; a lapos eset `([32,31.999936], [8,7.999992])` `"flat"`; egy `[float("nan"), 1.0]` input `"nan"`. A divergens rekurziónál $g_t = 8 \cdot 1{,}1^t$: 10 lépés után a hányados $1{,}1^9 \approx 2{,}36$, ezért még `"healthy"`; 80 lépés után $1{,}1^{79} \approx 1862 > 10^3$, ezért `"diverging"`. Az utóbbi teszt bemenete `g=[8*1.1**t for t in range(80)]` és `L=[32*1.21**t for t in range(80)]`. Nulla első gradiensnormánál a hányados nem értelmezhető, ezért a függvény nem ad `"diverging"` címkét. Tűrés: az `eps=1e-4` küszöb a float32 kerekítés fölött van, de problémafüggő; a hallgatónak a saját loss-skálájához kell igazítania.

F4 megoldás. Az első hiba: `F.cross_entropy` már tartalmazza a log-softmaxot, ezért a `F.softmax` dupla normalizálás, logitok helyett valószínűségi bemenet; ez nem numerikus instabilitás, hanem alapvetően hibás veszteség és gradiens. Javítás: `loss = F.cross_entropy(logits, labels)`. A második hiba: `images.reshape(images.shape[0], 1, -1)` a `(N,1,H,W)` képet `(N,1,H·W)` alakúra lapítja, így a konvolúciós modell elveszíti a térbeli szerkezetet, és shape-hibát ad vagy értelmetlen bemenetet kap; javítás: `logits = model(images)`. A harmadik hiba: `labels = labels_transposed`, ahol `labels_transposed = labels.T`; ha a címke `(N,)` alakú, a transzponálás no-op, ha viszont `(N,1)`, akkor `(1,N)` lesz, ami rossz. A címkének mindenképp

`(N,)` int64 alakúnak kell lennie, ezért: `labels = y_train` az eredeti, indexelt tenzorból, és `assert labels.ndim == 1` ellenőrzés. A változónevek is következetlenek (`labels_valtozo` a kommentben, `labels_transposed` a kódban); a javított blokkban egyetlen név szerepel.

F5 megoldás. Kérdés: a val plató oka kapacitás vagy label noise? Megfigyelési egység: scene_id, mert a GlyphLab minden jelenetéhez két zajos nézet tartozik, amelyek ugyanahhoz a scene_id-hez és ugyanahhoz a címkéhez tartoznak, ezért nem függetlenek. Split: a GlyphLab rögzített train/val/test felosztása, a test a teljes elemzés végéig érintetlen; a döntés kizárólag a val splitten születik. Baseline: a 12.4. III. ellenőrző pont: önállóan tanuló kis háló alfejezetbeli 32 egységes MLP. Változó tényező: modellméret (például 8, 32, 128 rejtett egység) és címkeminőség (tiszta, illetve $p_{\text{noise}} = 0{,}1$ mellett a train címkék 10%-át egyenletesen a másik két osztály valamelyikére cseréljük; a val címkék érintetlenek). Metrika: val loss és val accuracy. Döntési szabály: ha a plató a legkisebb modellen jelentkezik, és a nagyobb méreteknél megszűnik, a kapacitáskorlát-hipotézist támogatottnak tekintjük; ha a nagyobb méreteknél is megmarad, a kapacitás önmagában nem magyarázza. Ha a tiszta címkés futás megszünteti a platót, a label-noise-hipotézist támogatottnak tekintjük; ha egyik kontroll sem változtat, további okot (optimalizálás, adat) kell vizsgálni. Az érvényességi határ: a toy adat nem általánosít; a tesztet csak a protokoll végén, a rögzített modellen lehet használni, különben kiválasztási torzítás lép fel.

Recept, seedek, döntési küszöb és erőforrás: a 32 egységes MLP-t SGD optimalizálóval, `lr=0{,}5`, 400 lépéssel és 64-es batch mellett tanítjuk, öt seeddel (20261020–20261024); a 8, 32 és 128 egységes változatok ugyanazokat a seedeket kapják. A platót akkor tekintjük fennállónak, ha a val loss az utolsó öt epochban kevesebb mint 10^{-3}-rel csökken; a kapacitáshipotézist akkor tekintjük támogatottnak, ha a 8 egységes modell platózik, a 128 egységes nem, és a végső val loss különbségére a seedekből számított 95%-os CI kizárja a 0-t. Erőforrás: CPU, hat konfiguráció × öt seed = 30 futás, a toy adaton néhány perc. ### Gyakori tévedés ellenpéldával

Tévedés: „A lapos veszteséggörbe azt jelenti, hogy a modell elérte a veszteségfelület lokális minimumát.” **Ellenpélda:** a kézi példa $\eta = 10^{-6}$ esetében a veszteség $L(\theta)$ szigorúan konvex, egyetlen minimuma van $\theta = 2$-nél, de a görbe a 10-es pontban lapos, mert a lépések túl kicsik; a modell nincs a minimum közelében. A gradiens itt 8, tehát nem nulla. A helyes következtetés az, hogy a laposság numerikus, adat- vagy optimalizálási okból is eredhet; a gradiensnorma- és paraméterváltozás-mérés különbözteti meg az eseteket.

24.1.8 Önellenőrzés és visszautalás

Ellenőrizd, hogy (a) minden jelhez legalább két lehetséges ok szerepel-e a diagnózisnaplódban; (b) a lapos loss esetén rögzítetted-e az ε küszöböt és a lépésszámot; (c) a labor futtatása nem használta-e a test splitet modellválasztásra.

Rövid válaszok. (a) Igen: minden jelhez legalább két lehetséges okot kell rögzíteni, különben a jel nem azonosít okot. (b) Igen: a lapos loss diagnózisához rögzíteni kell az ε küszöböt és a lépésszámot. (c) Igen: a test splitet sem a modell-, sem a hiperparaméter-választásra nem használjuk.

Visszautalás: a 12.1. Tanítás, validáció és kiértékelési mód alfejezetben levezetett súlyozott epoch-loss itt azért fontos, mert a diagnosztikai görbét batch-átlagokból építed; a 23.1. Confusion matrix és aggregálási konvenció alfejezetben tárgyalt aggregálás pedig arra figyelmeztet, hogy a train accuracy önmagában nem helyettesíti a val loss-t és a csoportszintű hibát.

24.1.9 Rövid összefoglaló

A tanulási görbe három jelet kapcsol össze: veszteség, gradiensnorma és paraméterváltozás. Ugyanahhoz a jelhez több mechanikailag különböző ok tartozhat, ezért a diagnózis nem a görbe „ránézéséből”, hanem diszkrimináló ellenőrzésekből áll. A lapos loss oka lehet kis tanulási ráta, eltűnő gradiens, befagyasztott súly vagy label bug; a NaN eredhet adathibából, túl nagy tanulási rátából vagy rossz loss-kompozícióból. A vanishing/exploding jelenség a láncszabály-szorzat skálázódásának következménye. A labor három szándékos hibát kapcsol be, és hipotézisenként egy mért jelet és egy ellenőrzést naplóz. A korrekt diagnosztikai gyakorlat a hibás futtatást tanulságként kezeli, nem a test split engedélyeként.

24.1.10 Következő lecke és felhasznált források

A 24.2. CPU/GPU profiling és helyes időmérés a tényleges futásidőt és erőforráshasználatot méri. A jelen egység nem állít GPU/AMP-viselkedést. A jelen alfejezethez felhasznált forrás:

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.

24.2 CPU/GPU profiling és helyes időmérés

24.2.1 Cél, előismeret és induló kérdés

A 24.1. Tanulási görbe és hibadiagnózis tanulási görbéje felhasználja az epoch-időt, de nem mondja meg, hogyan mértük. Ha ugyanazt a hálót kétszer futtatjuk, a Python `time` modulja más-más értéket adhat, és GPU-n a különbség nem is feltétlenül a számítás ideje. Ebben az egységben a mérés fogalmát pontosítjuk: mit jelent a „futásidő”, miért kell a gyorsítóval szinkronizálni, és hogyan bontsuk a teljes képzési pipeline költségét adatbetöltésre, transzferre, számításra és memóriára.

Előismeret a 24.1. Tanulási görbe és hibadiagnózis hibadiagnózisa, a 12.2. Checkpoint és folytathatóság checkpoint-protokoll és a 20.2. Finomhangolás és tanulási ráta paraméterfrissítési lépése. A labor CPU-n fut; GPU-mérést nem állítunk elvégzettnek, csak jelölt tervet adunk.

Induló kérdés: egy batch feldolgozása ötször mérve 2,1; 2,0; 2,4; 2,0; 9,3 ms-ot mutat. Miért hibás azt állítani, hogy a lépés „2,0 ms”, és miért hibás a 9,3-at egyszerűen kidobni?

24.2.2 A mért időintervallum definíciója

Definíció 1 (mért időintervallum). Legyen C egy monoton óra; $C(t)$ a t fizikai pillanathoz tartozó időbélyeg. Ha a mérés kezdetét a, végét b jelöli, a mért futásidő

$$T_{\text{meas}} = C(b) - C(a).$$

Az intervallum akkor *érvényes* az M munkára, ha M minden releváns eseménye $[a, b]$-be esik. Releváns esemény alatt a mért mennyiséget meghatározó eseményeket értjük: a munka CPU-oldali elküldését és az eszközoldali befejezését. A Python `time.perf_counter()` monoton óra; a `time.time()` nem az, mert a rendszeróra visszafelé állítható, így $C(b) - C(a)$ akár negatív is lehet. A *CPU process time* ezzel szemben csak a processz CPU-idő-magját méri, és nem tartalmazza a DataLoader várakozását vagy a GPU-végrehajtást.

A definícióban a „releváns esemény” a kulcs. Amikor a mérés csupán a CPU-oldali műveletsor végét rögzíti, a gyorsítón esetleg még futó munkát nem tartalmazza.

24.2.3 Miért szükséges a gyorsítós szinkronizáció

A GPU munkát aszinkron módon indítjuk: a CPU kernelindítást küld a streambe, majd azonnal továbblép. A mérés helyességéről szóló állítást egy egyszerű, de pontos modellben bizonyítjuk.

Lemma 1 (záró szinkronizáció szükségessége). Legyen m kernel egyetlen, FIFO streamen. Tegyük fel, hogy az i-edik kernel indítási időpontja a CPU-órán ℓ_i, tiszta eszközoldali futásideje $e_i > 0$, és a befejezési időpont a stream-csúcsban

$$f_i = \max(f_{i-1}, \ell_i) + e_i, \qquad f_0 \le \ell_1.$$

Itt f_0 a stream utolsó, a mérés előtti befejezési időpontja; a legegyszerűbb esetben $f_0 = 0$, azaz a stream a mérés kezdetekor üres. A modell feltételezi, hogy a kernelek egyetlen FIFO streamen, sorban futnak, nincs párhuzamos végrehajtás, és a CPU-oldali indítási overhead az ℓ_i értékekbe van beépítve; több stream vagy párhuzamos kernelek esetén a lemma csak alsó korlátot ad.

Ha a CPU-oldali mérési intervallum záró időpontjára $b < f_m$, akkor az M kernelsorozat teljes eszközmunkájából legalább $f_m - b$ idő a mérés lezárása után következik be; így $b - a$ nem érvényes mértéke M futásidejének.

Bizonyítás. Az m-edik kernel befejezési időpontja f_m. A $b < f_m$ feltétel azt mondja, hogy b-kor a stream még nem érte el az utolsó kernel végét. Mivel a stream FIFO, az utolsó kernel munkájából $f_m - b > 0$ rész b után esedékes. A mért intervallum tehát nem tartalmazza M teljes eseményét, ami sérti az érvényesség definícióját. □

A szinkronizáció nem teszi a mérést „pontossá” minden értelemben: a GPU óráját nem a CPU `perf_counter` adja, a záró szinkron hívás maga is időt fogyaszt, és a CPU-indítás overheadje is beleszámít a mért intervallumba. A lemma csak a szükségességet rögzíti: szinkronizálás nélkül a mérés nem állíthatja a teljes munka idejét.

24.2.4 Kézi számpélda a lemmához

Tekintsünk három kernelt:

i	ℓ_i (ms)	e_i (ms)	f_i (ms)
1	0	4	$\max(f_0, 0) + 4 = 4$
2	1	3	$\max(4, 1) + 3 = 7$
3	2	5	$\max(7, 2) + 5 = 12$

A teljes eszközmunka befejezése $f_3 = 12$ ms. Ha a CPU a $[0, 3]$ ms intervallumot méri szinkronizálás nélkül, a mért érték 3 ms. Ekkor a harmadik kernel még el sem indult (a kezdete $\max(f_2, \ell_3) = 7$ ms, és 12 ms-ig fut, azaz 7–12 ms között); a mérés lezárásától a teljes eszközmunka befejezéséig még $12 - 3 = 9$ ms falióra-idő van hátra. Ez a hátralévő idő nem a harmadik kernel futási ideje: tartalmazza az előző kernelek hátralévő részét és a várakozást is. A szinkronizált mérés a harmadik kernel végét megvárva legalább 12 ms-t mutat, plusz a szinkronizációs hívás overheadje. Ez a példa a gyakori tévedés ellenpéldája is: a CPU-oldali `t1-t0` nem a kernel teljes munkája.

24.2.5 A mérési protokoll elemei

A fenti öt mérésben (2,1; 2,0; 2,4; 2,0; 9,3) a medián 2,1 ms, az átlag 3,56 ms; a különbséget a 9,3 ms-os kiugró érték okozza. A minimum kiválasztása félrevezető, mert a CPU-ütemező, a háttérfolyamat vagy a first-touch memóriaallokáció (az első íráskor történő fizikai lapkiosztás) miatt a minimális érték nem a munkához tartozó stabil idő. A 9,3 ms sem dobható el indoklás nélkül: lehet a mérés valós eloszlásának felső széle. A robusztus közép a **medián** a teljes nyers listából, mellette a min–max és a kvartilisek (az interkvartilis terjedelem, IQR = Q3 − Q1). A „warmup” célja a lazy inicializáció (az első hívásnál létrejövő modulok és pufferek) eltávolítása: az első hívás gyakran tartalmazza a memóriaallokációt, a kernelmodul betöltését vagy a könyvtári algoritmusválasztást, amely a későbbi lépésekből hiányzik. Warmup nélkül az első mért lépés a többitől független, egyszeri költséget tartalmaz. A nyitó kérdésre a válasz tehát: a jelentett érték a medián (2,1 ms), a 9,3 ms-ot pedig nem kidobni, hanem a nyers listában és a szórásképben megtartani kell.

24.2.6 Labor: CPU-mérés a GlyphLab-pipeline-en

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`; hiányzó adat esetén `uv run python generate_data.py --output data`. A futtatás a szakaszban megadott `measure_pipeline.py` szkripttel történik; várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A következő kód a `lab/` könyvtárban futtatható; az adat a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab. Az adatfájlok előállítását, a `metadata.csv` `split` oszlopát és a 240/60/60-as felosztást ugyanez a szakasz rögzíti; a regenerálás `uv run python generate_data.py --output data`. A kódot a könyv környezetében lefuttattuk (Python 3.12.3, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020); a hallgató a saját `measure_report.json`-ját állítja elő, és az az empirikus bizonyíték.

```
# munkakönyvtár: lab/
# Futtatás: python measure_pipeline.py
import csv
import json
import statistics
import time
import numpy as np
import torch

SEED = 20261020
WARMUP = 3
REPEAT = 10
BATCH_SIZE = 32
torch.set_num_threads(1)

def load_glyphlab_train():
    images = np.load("data/images.npy")      # (360,1,24,24) float32
    labels = np.load("data/labels.npy")      # (360,) int64
    idx = []
    with open("data/metadata.csv", newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(rows[0])
    assert len(rows) == len(images) == len(labels) == 360, (len(rows), images.shape)
    assert [int(r["index"]) for r in rows] == list(range(len(rows)))
    for row in rows:
        if row["split"] == "train":
```

```
            idx.append(int(row["index"]))
    X = torch.tensor(images[idx], dtype=torch.float32)
    y = torch.tensor(labels[idx], dtype=torch.long)
    return X, y

class SmallCNN(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Conv2d(1, 8, 3, padding=1),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
            torch.nn.Conv2d(8, 16, 3, padding=1),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
            torch.nn.Flatten(),
            torch.nn.Linear(16 * 6 * 6, 3),
        )

    def forward(self, x):
        return self.net(x)

def step(x, y, model, loss_fn, opt):
    opt.zero_grad(set_to_none=True)
    logits = model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    opt.step()
    return loss.detach().item()

def summarize(values):
    q1, med, q3 = statistics.quantiles(values, n=4, method="inclusive")
    return {
        "median": med,
        "mean": statistics.mean(values),
        "min": min(values),
        "max": max(values),
        "iqr": q3 - q1,
    }

def main():
    torch.manual_seed(SEED)
    device = torch.device("cpu")
    X, y = load_glyphlab_train()
    ds = torch.utils.data.TensorDataset(X, y)
    # A shuffle-permutációt kikapcsoljuk, hogy a data_ms csak a batch
    # összeállítását mérje; a permutáció költsége külön mérés tárgya.
    loader = torch.utils.data.DataLoader(
        ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=0
    )
    model = SmallCNN().to(device)
    loss_fn = torch.nn.CrossEntropyLoss()
    opt = torch.optim.SGD(model.parameters(), lr=0.01)

    for _ in range(WARMUP):
        xb, yb = next(iter(loader))
        step(xb.to(device), yb.to(device), model, loss_fn, opt)

    data_ms, step_ms, total_ms = [], [], []
    it = iter(loader)
    for _ in range(REPEAT):
        t0 = time.perf_counter()
        try:
```

```
            xb, yb = next(it)
        except StopIteration:       # 240 kép / 32 = 7,5 batch korszakonként
            it = iter(loader)
            xb, yb = next(it)
        t1 = time.perf_counter()
        xb, yb = xb.to(device), yb.to(device)
        _ = step(xb, yb, model, loss_fn, opt)
        t2 = time.perf_counter()
        data_ms.append(1e3 * (t1 - t0))
        step_ms.append(1e3 * (t2 - t1))
        total_ms.append(1e3 * (t2 - t0))

    for name, values in (("data", data_ms), ("step", step_ms), ("total", total_ms)):
        assert len(values) == REPEAT, (name, len(values))
        assert all(v >= 0 for v in values), name

    report = {
        "device": str(device),
        "batch_size": BATCH_SIZE,
        "warmup": WARMUP,
        "repeat": REPEAT,
        "shuffle": False,
        "data_ms": data_ms,
        "step_ms": step_ms,
        "total_ms": total_ms,
        "summary": {
            "data": summarize(data_ms),
            "step": summarize(step_ms),
            "total": summarize(total_ms),
        },
    }
    with open("measure_report.json", "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)
    print(json.dumps(report["summary"], indent=2))

if __name__ == "__main__":
    main()
```

A fenti kód egy futásának kimenete (a `measure_report.json` `summary` mezője; a teljes nyers listák a fájlban vannak):


```
{
  "data": {
    "median": 0.12896450061816722,
    "mean": 0.16348380013369024,
    "min": 0.08434700430370867,
    "max": 0.5036180082242936,
    "iqr": 0.03101374750258401
  },
  "step": {
    "median": 2.1398530079750344,
    "mean": 2.1185625984799117,
    "min": 1.9724599842447788,
    "max": 2.3319699976127595,
    "iqr": 0.20125976152485237
  },
  "total": {
    "median": 2.279597509186715,
    "mean": 2.282046398613602,
    "min": 2.089400979457423,
    "max": 2.7110610099043697,
    "iqr": 0.2485590084688738
```

```
  }
}
```

A labor értelmezése. A `data_ms` a batch előállításának költsége (a DataLoader batch-összeállítása; a `shuffle=False` miatt permutáció nélkül), a `step_ms` a forward, backward és optimizer-lépés költsége a `.to(device)` hívásokkal együtt, a `total_ms` pedig a teljes mért ciklus. A két rész pontosan felosztja a mért intervallumot: total = data + step (a kerekítéstől eltekintve), a mérési overhead az egyes részeken belül van. CPU-n a `.to(device)` azonos eszközre mutató hívásként nem másol adatot, ezért a `step_ms` itt nem tartalmaz HtoD-transzfert (host-to-device másolás: a CPU memóriájából a GPU memóriájába történő adatmozgatás); a transzfer csak a GPU-tervben jelenik meg. A mérést a szkript maga is ellenőrzi: `assert`-tel megköveteli, hogy minden lista hossza `REPEAT` legyen, és minden elem nemnegatív; a medián a nyers listából számol. Ökölszabály: ha a `data_ms` mediánja a `total_ms` mediánjának legalább 20–30%-a, a betöltés érdemi szűk keresztmetszet lehet; itt ez az arány körülbelül 5,7%, tehát nem az. Ez a következtetés csak erre a könyvtárra, splitre és gépre érvényes.

A `num_workers=0` azt jelenti, hogy a betöltés a fő folyamatban történik; a munkások indításának overheadje ebben a változatban nem jelenik meg. A `shuffle=False` választás tudatos: a korszakhatáron újrainduló iterátor első `next()` hívása `shuffle=True` esetén a permutáció költségét is a `data_ms`-be tenné. A szkriptet `WARMUP=3`, `REPEAT=10`, `BATCH_SIZE=32` mellett futtattuk; a fenti kimenet mediánjai egy futásból: data 0,1290 ms, step 2,1399 ms, total 2,2796 ms. Három egymást követő futásban a mediánok: data 0,129–0,186 ms, step 2,14–2,77 ms, total 2,28–2,95 ms; ekkora szóráson belül egyetlen futás számai nem általánosíthatók. A nyers `data_ms` listában egy 0,5036 ms-os kiugró érték is volt, miközben a medián 0,1290 ms maradt; ez éppen a medián robusztusságát szemlélteti. A laborban a 3 melegítő lépés azért elég, mert CPU-n a lazy inicializáció és az allokátor gyorsítótárazása néhány lépés alatt beáll; GPU-n a CUDA-kontextus és a cuDNN-algoritmuskeresés miatt 10 lépés a célszerű minimum. A futásidő gépfüggő; a számok a rögzített környezetre (Python 3.12.3, torch 2.15.0+cpu, CPU, 1 szál) érvényesek, és futásról futásra változnak.

A címben szereplő profiling eszköze a `torch.profiler`; a labor változóival (`model`, `xb`, `yb`, `loss_fn`, `opt`) egy lépés profilja így készíthető:

```
from torch.profiler import ProfilerActivity, profile

with profile(activities=[ProfilerActivity.CPU]) as prof:
    step(xb, yb, model, loss_fn, opt)
print(prof.key_averages().table(sort_by="self_cpu_time_total", row_limit=10))
```

A táblázat műveletenkénti bontást ad; a `self_cpu_time_total` oszlop a művelet saját CPU-ideje a profiler mérése szerint. A profiler saját overheadje miatt a táblázat ideje nem azonos a `step_ms`-sel, ezért a profiler a szűk keresztmetszet megkeresésére való, a jelentett idő mérésére nem.

24.2.7 Jelölt GPU-mérési terv

GPU nélkül is megadható a protokoll, de nem szabad mért eredményt állítani. A terv:

1. **Kérdés:** a GlyphLab `SmallCNN` egy 32-es batch lépése mennyi idő GPU-n, mennyi a HtoD-transzfer, és van-e átfedés a másolás és a számítás között?
2. **Környezet:** GPU-modell, driver, CUDA-verzió, PyTorch-verzió, seed, float32; a [D02] értelmében a seed nem garantál platformok közötti bitazonosságot.

3. **Warmup:** legalább 10 lépés, hogy a CUDA-kontextus, az első allokációk és az algoritmuskeresés ne a mért intervallumba essen. A labor 3 melegítő lépése CPU-n elég, mert ott nincs CUDA-kontextus és cuDNN-algoritmuskeresés; a két szám különbsége a két környezet különbségéből adódik.
4. **Mérés:** minden mért intervallum előtt és után `torch.cuda.synchronize()`; a záró hívás a `t1` rögzítése előtt történjen, különben a Lemma 1 hibáját követjük el. A betöltőt `pin_memory=True`-val adjuk meg, így a batch már pinned memóriában áll; a HtoD-másolást külön, `x.to(device, non_blocking=True)` hívással mérjük, és a mért intervallum végét ott is szinkronizáljuk. A pinned memóriába töltés költsége az adatbetöltés része, nem a transzferé, ezért a mérési ablakot a betöltés és a másolás között kell elvágni.
5. **Metrika:** 30 ismétlés mediánja és IQR; a minimumot nem mint „tiszta időt” jelentjük.
6. **Memória:** a mérés előtt `torch.cuda.reset_peak_memory_stats()`, a végén a `max_memory_allocated` és a `max_memory_reserved` külön rögzítése, azonos mérési ablakban. Az *allocated* és a *reserved* két külön számláló (a reserved a gyorsítótárazó allokátor által lefoglalt memória, ami az allocated-nél nagyobb lehet); mindkettőnek külön csúcsát kell jelenteni, és nem szabad a kettő maximumát egyetlen „csúcsként” megadni. A csúcs allocated a paramétereket, a gradienseket, az aktivációkat és az optimalizáló állapotát is tartalmazza, nem csupán a gradiens és az aktiváció együttes foglalását.
7. **Ellenőrző eszköz:** a `torch.utils.benchmark` [D16] és a `torch.profiler` [D17] megfelelő szakaszának elolvasása után használható keresztellenőrzésre; ezek a források kiinduló dokumentációk, nem ellenőrzött API-szemantika.

A transzfer kiemelése: a `tensor.to("cuda")` nem feltétlenül fejeződik be azonnal; szinkron nélkül a CPU-oldali mérés a másolásnak is csak az indítását látja. A 20.2. Finomhangolás és tanulási ráta paraméterfrissítési lépése ugyanezen okból nem mérhető GPU-n a CPU `perf_counter` puszta különbségével.

24.2.8 Memória

A mért idő mellé a memóriát is naplózni kell, mert a gyorsulás sokszor allokációval vásárolt. GPU-n az `allocated` a ténylegesen lefoglalt, aktív tensor-memória, a `reserved` a caching allocator által megtartott keret; mindkettőnek külön csúcsát kell mérni és jelenteni, mert a reserved a gyorsítótárazás miatt az allocatednél nagyobb lehet, és a kettő maximumából nem szabad egyetlen „csúcsot” képezni. A csúcs allocated a paramétereket, a gradienseket, az aktivációkat és az optimalizáló állapotát is tartalmazza, nem csak a gradiens és az aktiváció együttes foglalását. CPU-n a `torch.profiler` [D17] memory-szekciója vagy a rendszerfigyelő ad hasonló bontást, de a Python-allocátor és a NumPy-allocátor külön réteg. A mérési protokoll része, hogy a memória-csúcs ugyanarra a warmup nélküli, első futásra vonatkozik-e, vagy a warmup után mérjük; a kettő nem azonos állítás.

24.2.9 Gyakori tévedés és ellenpélda

A kernelindítás CPU-oldali ideje nem azonos a GPU teljes munkájával. A

```
t0 = time.perf_counter()
y = model(x.cuda())
t1 = time.perf_counter()
```

kód a CPU-oldali indítások idejét méri: a `t1-t0` értéke lehet 0,3 ms, miközben a kernel 2,1 ms-ig fut a GPU-n (szemléltető értékek, nem ebben a könyvben mért adatok). A példában `model` és `x` adott, és az `x.cuda()` a mért ablakon belül van; a mért érték így a másolás indítását is tartalmazza, ami tovább nehezíti a kernelidő azonosítását. A Lemma 1 numerikus példája mutatja, hogy a

záró szinkronizáció hiánya alulbecsléshez vezet; a helyes sorrend `torch.cuda.synchronize()` a kezdő és a záró időpont előtt is. Ez nem „rossz implementáció”, hanem az aszinkron végrehajtási modell következménye: a CPU oldali visszatérés nem a befejezés eseménye.

24.2.10 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Miért nem használható a `time.time()` a futásidő mérésére, és mit ad a `perf_counter` helyette? Adj konkrét ellenpéldát, amelyben a `time.time()` különbsége negatív.

F2 (levezetési, nehézség 2). A Lemma 1 modelljében legyen négy kernel: $\ell = (0, 2, 3, 10)$ ms, $e = (5, 4, 8, 2)$ ms. Számítsa ki az f_i értékeket, a teljes eszközidőt, és mutassa meg, hogy a CPU-oldali $[0, 9]$ ms szinkronizálatlan mérés miért nem érvényes. Hány ms a kihagyott munkarész?

F3 (implementációs, nehézség 2). Írjon `measure_fn(fn, warmup, repeat, sync=None)` függvényt, amely tetszőleges `fn` hívásának futásidejét méri `perf_counter`-rel, `warmup` melegítő hívást végez, `repeat` ismétlést gyűjt, és ha a `sync` paraméter nem `None` (egy `Callable[[], None]` szinkronizáló függvény), akkor minden mért intervallum előtt és után hívja azt. A függvény adja vissza a nyers másodperc-listát. Ellenőrizze a labor környezetben `repeat=5` mellett egy `lambda: torch.ones(100, 100).sum().item()` híváson.

F4 (hibakeresési, nehézség 2). Az alábbi kód egy GPU-mérési segédfüggvény:

```
def bad_measure(model, x):
    sync = torch.cuda.synchronize if torch.cuda.is_available() else (lambda: None)
    sync()
    t0 = time.perf_counter()
    for _ in range(10):
        y = model(x)
    t1 = time.perf_counter()
    sync()
    return (t1 - t0) / 10
```

A `sync` megadása eszközfüggetlen, ezért a kód CUDA nélkül is lefut. Lokalizálja a hibát, magyarázza el a következményt, és adja meg a javított változatot. Milyen kimenettel ellenőrizné, hogy a javítás helyes?

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen GPU-mérési protokollt a GlyphLab `SmallCNN` egy teljes lépésére, amely a HtoD-transzfert, a forward–backward–step számítást és a teljes iterációt külön méri. Adja meg a kérdést, hipotézist, warmup- és ismétlésszámot, a szinkronizációs pontokat, a jelentendő metrikákat és a protokoll érvényességi korlátját. GPU hiányában a tervet dokumentáltan, mért adat nélkül adja be.

24.2.11 Megoldások és indoklás

F1 megoldása. A `time.time()` a rendszer falióráját olvassa, amelyet NTP-korrekció vagy kézi állítás visszafelé mozgathat; ha a mérés kezdete után az óra hátraugrik, a különbség negatív, ami sérti a futásidő nemnegativitását. A `perf_counter` monoton óra, ezért a két időbélyeg különbsége mindig a mért intervallum hossza. Ellenpélda: ha $C(a) = 1000$ s, majd az óraállítás 30 s-ot visszavisz, $C(b) = 970$ s, akkor $C(b) - C(a) = -30$ s.

F2 megoldása. $f_0 \le \ell_1$. A rekurzióval:

$$f_1 = \max(f_0, 0)+5 = 5, \quad f_2 = \max(5, 2)+4 = 9, \quad f_3 = \max(9, 3)+8 = 17, \quad f_4 = \max(17, 10)+2 = 19.$$

A teljes eszközidő $f_4 = 19$ ms. A $[0, 9]$ ms intervallum $f_4 - b = 10$ ms munkát hagy ki; a mért 9 ms sem a befejezés, sem a teljes munka ideje. A helyes szinkronizált mérés minimálisan 19 ms-t vár, plusz a szinkron overhead.

F3 megoldása.

```
from typing import Callable, List, Optional
import time

def measure_fn(fn: Callable[[], None], warmup: int, repeat: int,
               sync: Optional[Callable[[], None]] = None) -> List[float]:
    for _ in range(warmup):
        fn()
    out: List[float] = []
    for _ in range(repeat):
        if sync:
            sync()
        t0 = time.perf_counter()
        fn()
        if sync:
            sync()
        t1 = time.perf_counter()
        out.append(t1 - t0)
    return out
```

Kontroll a laborban:

```
import torch

times = measure_fn(lambda: torch.ones(100, 100).sum().item(), 2, 5)
assert len(times) == 5
assert all(t >= 0 for t in times)
```

A nemnegativitás a monoton órából következik; a szinkron hívás maga is a mért intervallumon kívül, de a `t0` előtt és a `t1` után van, tehát a mért érték a `fn`-re vonatkozik.

F4 megoldása. A hiba: a záró `sync()` hívás a `t1` rögzítése után található. A `t1` ezért a CPU-oldali indítások végét rögzíti, nem a GPU-befejezést; a Lemma 1 miatt a mért átlag alulbecsli a teljes munkát. Következmény: a jelentett „batch-idő” csak az indítások CPU-ideje; GPU-n ez rendszerint jóval kisebb a kernel végrehajtási idejénél, de a különbség mértékét a saját mérésnek kell megadnia, feltételezés helyett. Javítás: a szinkronizáló hívást eszközfüggetlenül adjuk meg, és a záró szinkront a `t1` előtt végezzük, az időpontot utána olvassuk:

```
def good_measure(model, x):
    sync = torch.cuda.synchronize if torch.cuda.is_available() else (lambda: None)
    sync()
    t0 = time.perf_counter()
    for _ in range(10):
        y = model(x)
    sync()
    t1 = time.perf_counter()
    return (t1 - t0) / 10
```

Ellenőrzés: GPU-n hasonlítsuk a `torch.profiler` által mutatott kernelidő-összeghez; a jó mérés nagyobb vagy egyenlő, mint a hibás, és a különbség a kernelidő nagyságrendjébe eshet. A CPU-s kontroll csak azt igazolja, hogy a kód CUDA nélkül is lefut, és hogy a két változat ugyanazt a CPU-munkát méri: bemelegítés után a mért értékek a futások közötti zaj szintjén ingadoznak (az általunk használt kis CNN-en 0,17–0,28 ms között), nem mutatnak rendszeres különbséget. A szinkronizáció helyességét GPU-n kell igazolni.

F5 megoldás. Kérdés: a GlyphLab pipeline egy batch lépésében a HtoD-transzfer vagy a kernel a szűk keresztmetszet? Hipotézis: a 24×24-es, egycsatornás képekből álló 32-es batch csak néhány ezer float32 értéket tartalmaz, ezért a másolás adatmennyisége kicsi; a fix másolási overhead és a kernelindítások száma miatt mégis lehet, hogy a transzfer a lépésidő érdemi részét adja. Ezt a mérés dönti el, nem az intuíció. Protokoll: rögzített seed, 32-es batch, `pin_memory=True` DataLoader; warmup 10, repeat 30; minden intervallum előtt és után `torch.cuda.synchronize()`; a transzfer idejének mérése egy dedikált `x.to(device, non_blocking=True)` hívással, a számítási idő a forward–backward–step mérésével; metrika a medián és IQR ms-ban, valamint a csúcs `max_memory_allocated`. A terv nem állít benchmarkot: a 24×24-es adat (amelyet a labor `.npy` fájlokból tölt, nem futás közben generál) miatt az eredmény nem általánosítható természetes képekre; a `num_workers` és az adatbetöltő overheadje is része a teljes lépésnek, ezért a pipeline-mérést a batch-mikroméréstől elkülönítve kell jelenteni.

Hipotézis és egység: a HtoD-transzfer érdemi része a lépésidőnek; a változó tényező a transzfer és a kernel ideje, a modell a GlyphLab `SmallCNN`, a megfigyelési egység egy 32-es batch, a kontroll a szinkronizált teljes lépés. **Seed, döntési szabály és erőforrás:** a mérés rögzített seedje 20261020; a szűk keresztmetszetet akkor nevezzük a HtoD-transzfernek, ha a transzfer medián ideje a teljes lépés mediánjának legalább 20%-a, egyébként a kernelt tekintjük meghatározónak. A küszöböt a mérés megkezdése előtt rögzítjük, és a mediánok mellé az IQR-t is jelentjük. A feladat mérési protokoll, nem tanítás, ezért optimalizáló és learning rate helyett a mérési kontroll és a döntési szabály szerepel. Erőforrás: 1 CUDA GPU, 10 warmup + 30 ismétlés, néhány másodperc; GPU hiányában a terv mért adat nélkül, ezzel a döntési szabállyal adandó be. ### Önellenőrzés és korábbi fogalmak felidézése

Önellenőrzés: minden valós mért idő nemnegatív-e; a mért szakaszok összege és a teljes intervallum közötti különbség azonosítható-e; van-e warmup, és rögzített-e a seed a [D02] értelmében.

Rövid válaszok. Mindhárom igenlő feltétel szükséges: a mért idők nemnegatívak; a mért szakaszok összege és a teljes intervallum különbsége azonosítható (a hiányzó rész a nem mért overhead); van warmup, és a seed rögzített a [D02] szerint. Ha bármelyik nem teljesül, a mérés nem reprodukálható. A 12.2. Checkpoint és folytathatóság folytathatósági protokolljához hasonlóan a mérési eljárást is rögzíteni kell, különben a checkpointból folytatott modell lépésideje nem hasonlítható az előző futtatáshoz. A 20.2. Finomhangolás és tanulási ráta finomhangolási lépés mérésekor a befagyasztott (`requires_grad=False`) paraméterek miatt kevesebb gradiens számolódik, de ez nem jelenti azt, hogy a lépés gyorsabb: a forward továbbra is végigmegy a teljes hálón, a BatchNorm running statisztikái train módban frissülnek, és a DataLoader, valamint az optimizer overheadje a lépésidő része.

24.2.12 Rövid összefoglaló

A futásidő-mérés nem egyetlen függvényhívás különbsége. A mért intervallumot monoton órával definiáljuk, és érvényességéhez a vizsgált munka teljes eseményét az intervallumba kell foglalni. GPU-n a kernelindítás aszinkron: szinkronizálás nélkül a CPU-oldali idő csak az indítások költségét látja, ezért a záró szinkron a mérési protokoll kötelező része. A warmup a lazy inicializációt, a medián a terhelési kiugrásokat kezeli. A labor CPU-n méri a GlyphLab pipeline-t adatbetöltésre és lépésidőre bontva; a GPU-terv jelölt, nem elvégzett mérés. A memória-csúcs és a nyers adat ugyancsak a protokoll része.

24.2.13 Következő lecke

A 24.3. Mixed precision és nagyobb batch egységben a mixed precision (AMP, automatic mixed precision: a számítás egy részét float16/bfloat16 pontossággal végző, a master súlyokat float32-ben tartó vegyes pontosságú eljárás) és a nagyobb batch használata következik: az itteni helyes időmérés nélkül nem lehet eldönteni, hogy az AMP-val mért gyorsulás valós, vagy csupán a szinkronizálatlan CPU-oldali indítások különbsége.

24.2.14 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D16] PyTorch contributors (2026). *torch.utils.benchmark*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/benchmark_utils.html. Hozzáférés: 2026-10-30. Kiinduló hivatalos URL; a tartalom a használat előtt ellenőrizendő.
- [D17] PyTorch contributors (2026). *torch.profiler*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/profiler.html. Hozzáférés: 2026-10-30. Kiinduló hivatalos URL; a tartalom a használat előtt ellenőrizendő.

24.3 Mixed precision és nagyobb batch

24.3.1 Cél, előismeret és induló kérdés

Ez az egység a numerikus pontosság és a memóriahatár együttes kezelését vizsgálja. Előfeltétele 24.2. CPU/GPU profiling és helyes időmérés: a mért időintervallum, a gyorsítós szinkronizáció és a memóriamérés fogalmai. Visszautalunk 23.3. Páros összehasonlítás és csoportszintű intervallum páros összehasonlítási szerkezetére, 20.3. Preprocessing, domain shift és súlylicenc preprocessing-szerződésére és 12.3. Sanity check és kis mintára túllillesztés sanity checkjére. Induló kérdés: ha egy $n = 16$ elemű batch-et $A = 4$ darab $m = 4$ elemű mikro-batch-re bontunk, és a gradienseket felhalmozva frissítünk, ugyanazt a paraméterlépést kapjuk-e, mint a teljes 16 elemű batch-csel?

24.3.2 Definíciók és intuíció

Kevert pontosságú tanítás (mixed precision training). A modell paramétereit float32 *master-súlyként* tároljuk, de a forward és backward egy részét alacsonyabb pontosságú float16 vagy bfloat16 aritmetikával végezzük. A float32 normált értéktartománya hozzávetőleg 10^{-38} és 10^{38} között van, 23 mantisszabittel; a float16 legkisebb pozitív normált értéke $2^{-14} \approx 6{,}10 \cdot 10^{-5}$,

legnagyobb értéke $(2 - 2^{-10}) \cdot 2^{15} = 65504$, 10 mantisszabittel. A bfloat16 kitevőtartománya a float32-éhez hasonló (8 kitevőbit), de csak 7 mantisszabittel.

Autocast. A `torch.amp.autocast` kontextuskezelő, amely operátoronként választja meg, hogy az adott művelet alacsony vagy teljes pontosságon fusson. A kiválasztott pontosságot a `device_type` és a `dtype` argumentum adja meg. A `torch.autocast` és a `torch.amp.autocast` ugyanaz az API; a `torch.cuda.amp.autocast` a régi, elavult forma, helyette a device-független `torch.amp.autocast(device_type, ...)` használatos (a pontos API-állapotot a [D14] rögzíti).

GradScaler. A `torch.amp.GradScaler` a loss értéket egy $S > 0$ skálával megszorozza, mielőtt a `backward` lefutna (alapértelmezés: kezdő skála $2^{16} = 65536$, `growth_factor=2`, `backoff_factor=0.5`, `growth_interval=2000`). Ha a skálázatlan gradiens olyan kicsi, hogy float16-ban alulcsordulna, a skálázott $S \cdot \nabla L$ már ábrázolható maradhat. Az `unscale_` hívás a gradiensből a skálát visszaosztja, ezután következik az optimizer-lépés. Ha a skálázott gradiensben inf vagy NaN jelenik meg, a `scaler.step` kihagyja az optimizer-lépést, az `update` pedig a skálát a `backoff_factor`-ral csökkenti; ismétlődő túlcsordulásnál a skála 1 alá is mehet, és ezt a GradScaler nem akadályozza meg. Helyi kísérletben (CPU, torch 2.15.0+cpu, 1 szál, `torch.amp.GradScaler(device="cpu", init_scale=1.0)`): három végtelen gradiensű lépés után a skála 0,5, 0,25, 0,125, a súly pedig változatlan, mert mindhárom lépés kimaradt. Ez heurisztikus dinamikus skálázás, nem általános numerikus garancia. A skála futás közben változó állapot, ezért a tanítás folytathatóságához a `scaler.state_dict()` is a checkpoint része (12.2. Checkpoint és folytathatóság).

Gradiensakkumuláció. Legyen a teljes batch $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, amelyet A darab egyenlő méretű $B_1, \dots, B_A$ mikro-batch-re bontunk, ahol $|B_a| = m$ és $n = Am$. Az akkumuláció során minden mikro-batch-re kiszámítjuk a mean-redukciójú (a batch elemeire átlagoló) loss-t, meghívjuk a `backward`-ot, de a paraméterfrissítést csak A lépés után végezzük el. A mikro-batch loss-t A-val osztjuk, vagy a felhalmozott gradienst a frissítés előtt osztjuk A-val. A két konvenció a gradiens szintjén ekvivalens, ha az osztás a clipping és a GradScaler `unscale_`-je előtt történik; ha a clipping után osztunk, a küszöb léptéke változik meg, és a GradScaler skálázott `step`-jével való sorrend is eltérő eredményt adhat.

Gradiens-clipping. Egy $c > 0$ küszöb esetén a teljes paramétervektor $g \in \mathbb{R}^d$ gradiensére

$$\tilde{g} = \begin{cases} g, & \|g\|_2 \le c, \\ c\,\dfrac{g}{\|g\|_2}, & \|g\|_2 > c. \end{cases}$$

A norma a modell összes paraméterének gradiensét egyetlen vektorként kezeli. Kevert pontosságban a clippinget az `unscale_` **után** kell végezni, különben a küszöb S-szeres skálán hat.

A *hatékony batchméret* (effective batch size) az $n = Am$ szorzat; a memóriában azonban egyszerre csak egy m elemű mikro-batch aktivációi élnek. A nagyobb batch a gradiensbecslés varianciáját csökkentheti (6.3. Sztochasztikus és minibatch gradiens), de a memóriaigényt növeli; az akkumuláció ezt a feszültséget oldja fel a frissítési ritmus megtartása mellett.

24.3.3 Tétel: a gradiensakkumuláció ekvivalenciája

Tétel (gradiensakkumuláció ekvivalenciája). Legyen $f(\cdot;\theta)$ a modell paraméterezett függvénye, $\theta \in \mathbb{R}^d$. Tegyük fel, hogy

(F1) minden i-re a per-minta veszteség $\ell_i(\theta) = \ell(f(x_i;\theta), y_i)$ majdnem minden θ-ban differenciálható (a ReLU töréspontjai nullmértékű halmazt alkotnak), és ahol a gradiens létezik, ott $\nabla_\theta \ell_i$ a szokásos derivált;

(F2) $f(x_i;\theta)$ értéke nem függ a bemeneti batch többi elemétől, és a modell nem tartalmaz batch-szintű állapotot vagy a hívások közötti véletlen műveletet;

(F3) a mikro-batch-ek azonos m méretűek, $n = Am$, és a paramétereket nem frissítjük a mikro-batch-ek között.

Ekkor a mikro-batch mean-loss gradienseinek átlaga megegyezik a teljes batch mean-loss gradiensével:

$$\frac{1}{A}\sum_{a=1}^{A} \nabla_\theta \left(\frac{1}{m} \sum_{i \in B_a} \ell_i(\theta) \right) = \nabla_\theta \left(\frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta) \right).$$

Bizonyítás (saját levezetés). A gradiens linearitása miatt a jobb oldal

$$\nabla_\theta \left(\frac{1}{n} \sum_{i=1}^{n} \ell_i \right) = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \ell_i(\theta).$$

Az (F2) feltétel garantálja, hogy minden ℓ_i ugyanaz, mint a teljes batchben számolva; az (F1) differenciálhatóság pedig a gradiens létezését. A bal oldal:

$$\frac{1}{A}\sum_{a=1}^{A} \frac{1}{m} \sum_{i \in B_a} \nabla_\theta \ell_i(\theta) = \frac{1}{Am} \sum_{a=1}^{A} \sum_{i \in B_a} \nabla_\theta \ell_i = \frac{1}{Am} \sum_{i=1}^{n} \nabla_\theta \ell_i.$$

Az (F3) miatt $Am = n$, tehát a két oldal egyenlő. □

Válasz a nyitó kérdésre. A tétel a *gradiens* szintjén mond ki egyezést: a mikro-batch mean-loss gradienseinek átlaga a teljes batch mean-loss gradiense, feltéve, hogy a modell batch-független. A paraméterlépés csak akkor azonos, ha az optimalizáló állapota (például a momentum), a gradiens-clipping és az AMP-skálázás is ugyanúgy számolódik a két esetben. Az akkumuláció tehát a gradiens szintjén ekvivalens, a teljes tréning ekvivalenciája további feltevéseket igényel.

Megjegyzés (eltérő mikro-batch méretek). Ha a mikro-batch méretek m_a nem egyenlők, a súlyozatlan átlag általában nem egyezik a nagy batch gradiensével. A korrigált súlyozás $\sum_a \frac{m_a}{n} \nabla L_{B_a}$ adja a nagy batch mean-gradiensét, mert

$$\sum_a \frac{m_a}{n} \cdot \frac{1}{m_a} \sum_{i \in B_a} g_i = \frac{1}{n} \sum_i g_i.$$

Az akkumulációs hurkok ezért tipikusan azonos mikro-batch méretet használnak, vagy a loss-t a `batch_size / n` aránnyal súlyozzák.

Mi töri meg az egyezést? A tétel (F2) feltétele kifejezetten szigorú. Két gyakori réteg ellenpélda:

1. **BatchNorm train módban.** A BatchNorm a forward során a mikro-batch saját átlagát és varianciáját használja. Az $x_1 = 1$, $x_2 = 3$, $x_3 = 5$, $x_4 = 7$ négyelemű batch átlaga $\mu = 4$, varianciája $\sigma^2 = 5$; a $\{x_1, x_2\}$ mikro-batch átlaga $\mu_1 = 2$, varianciája $\sigma_1^2 = 1$. $\varepsilon = 0$-t véve a normalizált x_1 értéke a nagy batchben $(1-4)/\sqrt{5} \approx -1{,}34$, a mikro-batchben $(1-2)/1 = -1$; ugyanaz a minta tehát más bemenetet ad a következő lineáris rétegnek, ezért a loss és a gradiens is eltér. A BatchNorm tehát nem batch-független a train módban.

2. **Dropout.** A dropout minden forward híváskor új véletlen maszkot húz. A nagy batch egyetlen forward híváskor n maszkot fogyaszt; az akkumuláció A darab m elemű forward híváskor más RNG-sorrendben fogyaszt maszkokat. Még azonos seed mellett sem garantált, hogy a realizált dropout-maszkok megegyeznek a nagy batch realizációjával; a [D02] ismételhetőségi jegyzete szerint a seed rögzítése önmagában nem garantál bitazonos futást. A várható érték ugyanaz lehet, de a realizált paramétertrajektória nem.

A BatchNorm és dropout miatt tehát az azonos effektív batchméret **nem** garantál azonos tréninget.

24.3.4 Kézi számpélda

Legyen az egyparaméteres modell $f(x) = wx$, a négyzetes per-minta veszteség $\ell_i(w) = (wx_i - y_i)^2$, a batch $D = \{(1,2), (2,4), (3,6), (4,8)\}$, és $w = 0{,}5$. Teljes batch mean-loss:

$$L_D(w) = \frac{1}{4}\sum_{i=1}^{4}(wx_i - y_i)^2,$$

gradiense

$$\nabla_w L_D = \frac{1}{4}\sum_{i} 2(wx_i - y_i)x_i.$$

A négy tag: $2(-1{,}5)\cdot 1 = -3$; $2(-3)\cdot 2 = -12$; $2(-4{,}5)\cdot 3 = -27$; $2(-6)\cdot 4 = -48$. Összegük -90, tehát

$$\nabla_w L_D = \frac{-90}{4} = -22{,}5.$$

Most $A = 2$, $m = 2$: $B_1 = \{(1,2), (2,4)\}$, $B_2 = \{(3,6), (4,8)\}$. A mikro-batch gradiensek:

$$\nabla L_{B_1} = \frac{-3-12}{2} = -7{,}5, \qquad \nabla L_{B_2} = \frac{-27-48}{2} = -37{,}5.$$

Az átlaguk $(-7{,}5 - 37{,}5)/2 = -45/2 = -22{,}5$, ami pontosan a teljes batch gradiens. A példában nincs BN vagy dropout, ezért a tétel alkalmazható.

24.3.5 Kód: f32 kontroll és jelölt AMP-terv

Az alábbi kód a float32-os CPU-s kontrollutat ellenőrzi. A kódot lefuttattuk (Python 3.12.3, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020); a mért referencia-gradiensnorma 1,2347187995910645, a felhalmozott gradiensnorma ugyanennyi, a maximális abszolút eltérés $7{,}450580596923828 \cdot 10^{-9}$. Az egyezés a tételből következik, a konkrét eltérés a lebegőpontos kerekítés szintjén van.

```
# lab/n095_accum_check.py
# Futtatás a repository gyökeréből: python lab/n095_accum_check.py
import torch
import torch.nn as nn

torch.manual_seed(20261020)
torch.set_num_threads(1)
device = torch.device("cpu")
dtype = torch.float32

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(24 * 24, 16),
            nn.ReLU(),
            nn.Linear(16, 3),
        )

    def forward(self, x):
        return self.layers(x)


def flattened_grad(model):
    return torch.cat([p.grad.reshape(-1).clone()
                      for p in model.parameters()])


model = TinyMLP().to(device)
criterion = nn.CrossEntropyLoss()  # mean reduction, int64 target

x = torch.rand(16, 1, 24, 24, device=device, dtype=dtype)
y = torch.randint(0, 3, (16,), device=device)

# Referencia: teljes n=16 batch
model.zero_grad()
loss_full = criterion(model(x), y)
loss_full.backward()
g_ref = flattened_grad(model)
print("reference grad norm:", g_ref.norm().item())

# Akkumuláció: A=4, m=4, n=16
model.zero_grad()
A = 4
m = 4
for a in range(A):
    xb = x[a * m:(a + 1) * m]
    yb = y[a * m:(a + 1) * m]
    loss_a = criterion(model(xb), yb)
    (loss_a / A).backward()   # a mikro-batch mean loss osztva A-val
g_acc = flattened_grad(model)
print("accumulated grad norm:", g_acc.norm().item())
print("max abs diff:", (g_ref - g_acc).abs().max().item())
assert torch.allclose(g_ref, g_acc, atol=1e-6, rtol=1e-5)
```

A `loss_a / A` osztás azért szükséges, mert a `CrossEntropyLoss` mean-redukciójú: minden mikro-batch loss a saját m elemű átlaga. Az A darab ilyen átlag összege A-szorosa a kívánt n elemű átlagnak, ezért az A-val osztás állítja helyre a nagy batch mean-loss skáláját. Mivel a modellben nincs BN vagy dropout, a mért maximális abszolút eltérés $7{,}45 \cdot 10^{-9}$, a lebegőpontos kerekítés szintjén; float32 CPU-n az `allclose(atol=1e-6, rtol=1e-5)` ellenőrzés teljesül.

A GPU-s AMP-terv **jelölt, nem futtatott** vázlata, a 2.14-es felület szerinti formában:

```
# CUDA-környezetben a tanító ciklusba építendő AMP-vázlat.
# model, criterion, loader: a tanító ciklus változói; xb, yb: az aktuális mikro-batch.
scaler = torch.amp.GradScaler(device="cuda")
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

for xb, yb in loader:
    optimizer.zero_grad(set_to_none=True)
    with torch.amp.autocast(device_type="cuda", dtype=torch.float16):
        logits = model(xb)
        loss = criterion(logits, yb)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
    scaler.step(optimizer)
    scaler.update()
```

A sorrend indoklása: az `unscale_` után a gradiens float32-ben van, ezért a clipping a valódi $c = 5$ küszöböt látja; ha a clippinget az `unscale_` előtt hívnánk, a küszöb a skálázott gradiensre vonatkozna. A `scaler.step` csak akkor frissít, ha a gradiensben nincs inf/NaN; a `scaler.update` ezután igazítja a skálát. CPU-n bfloat16 autocast esetén a GradScaler indoklása gyengébb, mert a bfloat16 kitevőtartománya a float32-éhez hasonló; a támogatottságot a helyi környezetben ellenőrizni kell.

24.3.6 Gyakori tévedés és ellenpélda

Tévedés: „Azonos effektív batchméret mellett az akkumuláció ugyanazt a tréninget adja, mint a nagy batch.”

Ellenpélda. Tekintsünk egy `Linear → BatchNorm → ReLU → Linear` modellt. A tétel (F2) feltétele sérül: a BatchNorm train módban a mikro-batch statisztikáját használja. A fenti négyelemű $1, 3, 5, 7$ példában a nagy batch átlaga 4, az első mikro-batch átlaga 2; a normalizált bemenetek és így a gradiens is különbözik. Dropout esetén az RNG-hívások száma és sorrendje tér el. A két tréning ezért még azonos seed mellett is szétválhat; az effektív batchméret egyenlősége szükséges, de nem elégséges feltétel.

24.3.7 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el, miért lehet probléma egy 10^{-6} nagyságú gradiens float16-ban, és hogyan viszonyul ehhez a GradScaler. Adjon konkrét S értéket, amellyel a gradiens a normált float16 tartományba kerül.

F2 (levezetési, nehézség 2). Legyen $n = 6$, B_1 mérete 4, B_2 mérete 2. Mutassa meg általános $g_i = \nabla \ell_i$ vektorokkal, hogy $\frac{1}{2}(\nabla L_{B_1} + \nabla L_{B_2}) \neq \nabla L_D$ általában, és adja meg a korrigált súlyozást, amely pontosan a nagy batch mean-gradiensét adja.

F3 (implementációs, nehézség 2). Írjon `accumulate_gradients(model, x, y, A)` függvényt, amely a fenti CPU-példa adatain (16 kép, $A = 4$, $m = 4$) felhalmozza a gradienst, és adja vissza a referencia-gradienstől való maximális abszolút eltérést. A megoldásnak tartalmaznia kell az importot, seedet, dtype-ot, eszközt és egy `allclose` ellenőrzést. Az elfogadási kritérium a feladat szövegében: a maximális abszolút eltérés kisebb, mint 10^{-6} (az `allclose` `atol=1e-6`, `rtol=1e-5` paraméterrel).

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet „gradiensakkumulációnak” nevezi magát:

```
for micro_x, micro_y in loader:
    loss = criterion(model(micro_x), micro_y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

Mi a hiba? Milyen következménye van a paramétertrajektóriára, és hogyan javítaná úgy, hogy a hatékony batchméret Am legyen?

F5 (kísérleti/transzfer, nehézség 3). Tervezzen AMP-kísérletet egy helyi GPU-n vagy bfloat16-képes CPU-n a függelék A. szakaszában leírt GlyphLab CNN-en. Adja meg a kérdést, a f32 baseline-t, a változó tényezőt, a választási metrikát, a seed- és környezetkezelést, valamint azt a következtetési határt, amelyet a kísérlet nem léphet át.

24.3.8 Megoldások és indoklás

F1. A float16 legkisebb pozitív normált értéke $6{,}10 \cdot 10^{-5}$; a 10^{-6} gradiens ennél kisebb, tehát a subnormal (normálatlan, a legkisebb normált érték alatti) tartományba kerül, ahol a mantissza vezető bitjei elvesznek, vagy nullára kerekül. A GradScaler a loss-t és ezzel a visszaterjedt gradienst S-szeresére nagyítja. $S = 2^{16} = 65536$ esetén $S \cdot 10^{-6} = 0{,}065536$, ami a normált float16 tartomány belsejében van; az `unscale_` a frissítés előtt visszaoszt. A skálázás nem változtat a gradiens irányán, csak az ábrázolhatóságát javítja.

F2. Jelölje $G_a = \nabla L_{B_a} = \frac{1}{m_a} \sum_{i \in B_a} g_i$. Ekkor

$$\frac{1}{2}(G_1 + G_2) = \frac{1}{2}\left(\frac{1}{4}\sum_{i \in B_1} g_i + \frac{1}{2}\sum_{i \in B_2} g_i\right) = \frac{1}{8}\sum_{i \in B_1} g_i + \frac{1}{4}\sum_{i \in B_2} g_i,$$

míg a nagy batch gradiens

$$\nabla L_D = \frac{1}{6}\left(\sum_{i \in B_1} g_i + \sum_{i \in B_2} g_i\right) = \frac{1}{6}\sum_{i \in B_1} g_i + \frac{1}{6}\sum_{i \in B_2} g_i.$$

A B_1 tag együtthatója $1/8$ a $1/6$ ellenében, a B_2-é $1/4$ a $1/6$ ellenében; általában tehát nem egyenlők. A korrigált súlyozás $\frac{m_a}{n}$: itt $\frac{4}{6}G_1 + \frac{2}{6}G_2 = \frac{1}{6}(\sum_{B_1} + \sum_{B_2})$, ami pontosan a nagy batch gradiens. A feladat feltétele $m_a > 0$; az $m_2 = 0$ eset nem értelmezett, mert G_2 az $1/m_2$ miatt nem létezik. Az $m_2 \to 0$ határban a B_2 tag súlya 0, ami összhangban van a korrigált súlyozással.

F3. Egy lehetséges megoldás:

```
import torch
import torch.nn as nn

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Flatten(), nn.Linear(24 * 24, 16),
            nn.ReLU(), nn.Linear(16, 3),
        )
    def forward(self, x):
        return self.layers(x)

def flat_grad(model):
    return torch.cat([p.grad.reshape(-1).clone()
                      for p in model.parameters()])

def accumulate_gradients(model, x, y, A):
    model.zero_grad()
    n = x.shape[0]
    m = n // A
    criterion = nn.CrossEntropyLoss()
    for a in range(A):
        xb = x[a * m:(a + 1) * m]
        yb = y[a * m:(a + 1) * m]
        loss = criterion(model(xb), yb)
        (loss / A).backward()
    return flat_grad(model)

model = TinyMLP().to(device)
x = torch.rand(16, 1, 24, 24, device=device, dtype=dtype)
y = torch.randint(0, 3, (16,), device=device)

g_acc = accumulate_gradients(model, x, y, A=4)
model.zero_grad()
criterion = nn.CrossEntropyLoss()
criterion(model(x), y).backward()
g_ref = flat_grad(model)

diff = (g_ref - g_acc).abs().max().item()
print("max abs diff:", diff)
assert torch.allclose(g_ref, g_acc, atol=1e-6, rtol=1e-5)
```

Alakellenőrzés: `g_ref` és `g_acc` dimenziója a paraméterek számának összege, itt $576 \cdot 16 + 16 + 16 \cdot 3 + 3 = 9283$. A `ReLU` a két futásban ugyanarra a bemeneti pontra ugyanazt a döntést adja (a modell batch-független), ezért a nem-sima ponton választott szubgradiens is konzisztens; véletlen bemenet ráadásul nulla valószínűséggel esik pontosan a töréspontra.

F4. A hiba: a `step()` és `zero_grad()` minden mikro-batch után lefut, tehát ez nem akkumuláció, hanem A darab egymás utáni kis batch-es frissítés. A paramétertrajektória eltér a nagy batch-es trajektóriától; BN-nél és dropoutnál a forward is eltérő statisztikát/maszkot lát, tehát még a gradiens sem a nagy batch gradiense. Javítás: a `backward()`-ot minden mikro-batch-re meghívjuk, de a `step()` és `zero_grad()` csak a ciklus után fut le, és a loss-t A-val osztjuk (vagy a felhalmozott gradienst osztjuk a frissítés előtt). Ha a clippinget használunk, azt a teljes, A-val osztott gradiensre kell alkalmazni.

F5. Példa protokoll. Adat: a GlyphLab a függelék A. A GlyphLab adatkészlet szerint 180 jelenet,

jelenetenként két nézet; a jelenetszintű split 120/30/30 jelenet, azaz 240/60/60 kép. A megfigyelési egység a `metadata.csv` `scene_id` mezője, és egy jelenet két nézete nem kerülhet különböző splitbe. A korrupt teszt a `corrupted_test.npy`: a 60 testkép zajjal és fényességgradienssel módosított változata, a testtel azonos sorrendben, ezért a clean és a korrupt kiértékelés képenként páros. Kérdés: az AMP megőrzi-e a záró test pontosságot a f32 kontrollhoz képest a GlyphLab CNN-en? Baseline: f32 SGD, ugyanaz az öt seed (pl. 20261020–20261024), tanítási séma, batchméret és early stopping a val loss alapján. Változó tényező: AMP be/ki. Metrika: a legjobb val-checkpoint test accuracy-ja; numerikus kontrollként az első lépés gradiensnorma-eltérése. Feltételenként öt futás készül, a két ágban seed-páronként azonos seeddel; a seedek közötti szórást (átlag és szórás) külön jelentjük, és a pontbecslést nem egyetlen futásra alapozzuk. A két ágat ugyanazon seedekkel és ugyanazon testjeleneteken párosan értékeljük; a döntést a jelenet szintű páros különbség bootstrap CI-je adja, és csak 0-t kizáró 95%-os CI esetén állítunk eltérést. Split: a rögzített scene-szintű train/val/test; a korrupt tesztet csak a végén, a befagyasztott modellen értékeljük. Környezet: ugyanaz a gép, PyTorch-verzió rögzítve, GPU esetén szinkronizált mérés 24.2. CPU/GPU profiling és helyes időmérés szerint. Következtetési határ: a GlyphLab kicsi és mesterséges; az eredmény nem általánosítható természetes képi benchmarkra vagy más hardverre, és a seed-szórás (23.3. Páros összehasonlítás és csoportszintű intervallum) miatt a pontbecslés önmagában nem bizonyít fölényt.

Hipotézis: az AMP a f32 baseline-hoz képest legfeljebb 2 százalékponttal változtatja a test accuracyt. **Modell, recept, seedek és erőforrás:** a kísérlet modellje a GlyphLab `SmallCNN`, a tanítási recept SGD (`lr=0{,}1`, momentum 0,9), 20 epoch és 16-os batch, a f32 és AMP ágban azonos; a seedek értéke 20261020–20261024. A döntési szabály számszerű: a jelenetszintű páros bootstrap 95%-os CI-je 0-t zár ki, és a pontosságkülönbség abszolút értéke legalább 2 százalékpont; a numerikus gradiensnorma-eltérésre előre rögzített tolerancia 10^{-3} relatív hiba. Erőforrás: 1 GPU, öt seed × két ág = 10 futás, a GlyphLab méretén néhány perc; GPU nélkül a kísérlet nem futtatható, ezt a jelentésben jelezni kell. ### Önellenőrzés és visszautalás

Ellenőrizze, hogy (1) a clippinget az `unscale_` után hívja-e; (2) az akkumulációhoz a mikro-batch loss-t osztja-e A-val; (3) a modellben nincs-e BN/dropout, mielőtt az egyezést állítja.

Rövid válaszok. (1) Igen: a clippinget az `unscale_` után kell hívni, különben a skálázott gradiensre vágunk. (2) Igen: az akkumulációhoz a mikro-batch loss-t A-val kell osztani. (3) Igen: BN vagy dropout esetén a tétel feltétele sérül, ezért az ekvivalencia nem állítható. Emlékeztető 24.2. CPU/GPU profiling és helyes időmérés alfejezetből: a GPU-időt csak szinkronizáció után mérjük, és a memóriamérésnél rögzítjük, hogy a jelentett csúcs a warmup előtti első futásra vagy a warmup utáni, bemelegített állapotra vonatkozik. 23.3. Páros összehasonlítás és csoportszintű intervallum szerint a seed-variáció és a páros teszt külön bizonytalanság; 20.3. Preprocessing, domain shift és súlylicenc szerint a preprocessing-szerződés az AMP-től függetlenül érvényes; a 12.3. Sanity check és kis mintára túllillesztés kis mintára túllillesztés (tiny overfit) eljárása az akkumulációs hibák gyors szűrője.

24.3.9 Rövid összefoglaló

A kevert pontosságú tanítás float32 master-súlyok mellett alacsonyabb pontosságú forward/backward operátorokat használ. A GradScaler a loss skálázásával akadályozza meg, hogy a float16 gradiens alulcsorduljon; a frissítés előtt a skálát visszaosztjuk. A gradiensakkumuláció azonos méretű mikro-batch-ek és batch-független modell esetén ekvivalens a nagy batch átlaggal: a mikro-batch mean-loss gradienseinek átlaga a teljes mean-loss gradiense. BatchNorm és dropout megtöri ezt az egyezést, mert a forward eredménye a batch összetételétől vagy a hívások közötti RNG-állapottól függ. A clipping a visszaskálázás (unscaling) után a globális gradiennormára hat. A labor f32 kontrollt készít; mért AMP-eredményt csak tényleges futtatás után szabad közölni. A

nyitó kérdésre a válasz igen: azonos méretű, négy elemű mikro-batchek és batch-független modell esetén a négy gradiens átlaga megegyezik a 16 elemű batch gradiensével, ezért a paraméterlépés is azonos.

24.3.10 Következő lecke és források

A 24.4. VI. ellenőrző pont: megbízható CNN-training VI. ellenőrző pont a teljes CNN-training megbízhatóságát kéri számon: az itteni akkumulációs és AMP-döntéseket a tanulási görbével, checkpointtal és záró teszttel együtt kell indokolni.

Felhasznált források:

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D14] PyTorch contributors (2026). *Automatic Mixed Precision*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/amp.html. Hozzáférés: 2026-10-30.

24.4 VI. ellenőrző pont: megbízható CNN-training

24.4.1 Cél, előismeret és induló kérdés

Ez a VI. ellenőrző pont: a 24.1.–24.3. hibadiagnózis- és profilereszközeit egyetlen, átadható csomaggá zárjuk. A cél nem új réteg tanítása, hanem annak bizonyítása, hogy a hallgató az adatot, tanítási receptet, mérési protokollt, metrikákat és hibakezelést együtt tudja lektorálható módon kezelni. Előismeret: a 24.2. CPU/GPU profiling és helyes időmérés wall time, warmup, adatbetöltési és memóriafogalmai, a 23.4. Hibaszeletek és alkalmazási határ szelet- és páros tesztelési szerződése, a 20.4. V. ellenőrző pont: indokolt CNN-választás architektúra-döntési lapja, valamint a 12.4. III. ellenőrző pont: önállóan tanuló kis háló checkpoint- és gradiens-egyeztetési (az autograd által adott gradiens összevetése kézi, NumPy-referenciával) eljárása.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg: a rubrika öt dimenziója D1–D5, a szintek 0–4, a PASS-szabály pedig mind az öt dimenzióban legalább 3-as szintet és a labor minden `assert`-jének átmenetét kéri. Itt csak az eltérés: a riport futásjegyzéke (runlog) a fő leadandó; a súlypont a D3 (adat- és mérési protokoll) és a D4 (bizonyíték és nyomon követhetőség) rubrikadimenzió.

Induló kérdés: egy CPU-n tanított, 2979 paraméteres kis CNN vajon „megbízhatóbb-e”, mint egy GPU-n tanított nagyobb háló? Becslés: a paraméterszám önmagában nem megbízhatósági mérce; először a modell loss- és alakelőírását, azután a kísérleti protokollt kell ellenőrizni. A GPU jelenléte gyorsíthat, de nem igazol helyes modellválasztást.

24.4.2 A kiválasztott modell, loss- és alakelőírás, analitikus kontroll

A laborhoz a 20.4. V. ellenőrző pont: indokolt CNN-választás döntési lapjából a következőket választjuk: saját, kisméretű classic CNN (scratch: előtanított súlyok nélkül, a semmiből tanítva), CPU-erőforrás, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-adat. A modell:

$$x \mapsto \mathrm{fc}(\mathrm{Flatten}(\mathrm{MaxPool}(\mathrm{ReLU}(\mathrm{Conv}_2(\mathrm{MaxPool}(\mathrm{ReLU}(\mathrm{Conv}_1(x)))))))) \,.$$

A pontos alak- és paraméterszerződés:

Réteg	Bemeneti alak	Kimeneti alak	Paraméterszám
Conv1: 1→8, 3×3, pad 1	$N \times 1 \times 24 \times 24$	$N \times 8 \times 24 \times 24$	$8(1 \cdot 3 \cdot 3) + 8 = 80$
ReLU, MaxPool 2×2	$N \times 8 \times 24 \times 24$	$N \times 8 \times 12 \times 12$	0
Conv2: 8→16, 3×3, pad 1	$N \times 8 \times 12 \times 12$	$N \times 16 \times 12 \times 12$	$16(8 \cdot 3 \cdot 3) + 16 = 1168$
ReLU, MaxPool 2×2	$N \times 16 \times 12 \times 12$	$N \times 16 \times 6 \times 6$	0
Flatten	$N \times 16 \times 6 \times 6$	$N \times 576$	0
Linear: 576→3	$N \times 576$	$N \times 3$	$576 \cdot 3 + 3 = 1731$

Összesen $80 + 1168 + 1731 = 2979$ paraméter. A kötelező NCHW-sorrend miatt minden térbeli alakot $C \times H \times W$-ként adunk meg; a batchméret N kívül áll.

A tanítási loss-szerződés a [D07] dokumentáció szerint: a modell logitsot ad, a cél int64 osztályindex, `nn.CrossEntropyLoss` előtt nem alkalmazunk külön softmaxot. Legyen $Z \in \mathbb{R}^{N \times K}$ a logitmátrix és $y \in \{0, \dots, K-1\}^N$ a cél; a mean reduction (a batch elemeire átlagoló redukció) veszteség

$$L(Z, y) = -\frac{1}{N} \sum_{n=1}^{N} \log \frac{\exp(z_{n,y_n})}{\sum_{j=1}^{K} \exp(z_{n,j})}.$$

A softmaxos nevező minden n-re az összes j osztályon összegez; a célindex csak a számlálóban jelenik meg. A kimenet skalár.

Analitikus kontroll (lemma). Ha $Z = 0^{N \times K}$, akkor tetszőleges y célra $L = \ln K$.

Levezetés: minden n, j-re $\exp(z_{n,j}) = \exp(0) = 1$, tehát a softmax-kimenet

$$p_{n,j} = \frac{1}{\sum_{\ell=1}^{K} 1} = \frac{1}{K}.$$

Így minden megfigyelésre $-\log p_{n,y_n} = -\log(1/K) = \ln K$, és a batchátlag ugyanennyi. GlyphLab esetén $K = 3$, tehát a kontrollérték $\ln 3 \approx 1{,}098612$. Ezt a labor sanity checkje (alapszintű értelmi ellenőrzés) numerikusan, 10^{-6} abszolút tűréssel ellenőrzi.

A lemma csak a loss-modult teszteli, a modellszerkezetet nem. Ezért a labor tartalmaz egy második, **zero-init alakkontrollt**: a modell minden súlyát és biasát nullára állítjuk. Ekkor minden Conv-réteg kimenete azonosan nulla tetszőleges bemenetre, mert a konvolúció lineáris része nulla, a bias nulla; ezt ReLU, MaxPool és Flatten nullán változatlanul hagyja; a Linear-réteg pedig $0 \cdot W^\top + b = 0$-t ad. Ezért a modell kimenete $(N, 3)$ alakú, azonosan nulla. Ez a kontroll a forward-lánc működését és a deklarált kimeneti alakot ellenőrzi a rögzített bemeneti alakra, de nem bizonyítja a tanulás működését: a nulla inicializálás nem töri meg a szimmetriát, ezért tanításra nem használjuk. A labor a kontroll után a rögzített seeddel generált alapértelmezett állapotot automatikusan visszaállítja.

24.4.3 Inicializálás (init)

A tanításhoz a `torch.manual_seed(20261020)` hívással rögzítjük a véletlensúly-inicializálást. A 21.1. Inicializálás és varianciaterjedés alfejezetben levezetett varianciafeltételt itt csak visszahívjuk: ReLU előtti konvolúciónál a cél gyakorlat az, hogy a súly varianciája 2/fan_in nagyságrendű legyen, ahol

$$\text{fan_in} = C_{in} k_h k_w.$$

A mi modellünkben az első Conv fan_in = 9, a másodiké $8 \cdot 9 = 72$, a lineáris rétegé 576. A PyTorch `nn.Conv2d` és `nn.Linear` alapértelmezett inicializálása `kaiming_uniform_` $a = \sqrt{5}$ értékkel, amelynek varianciája $\approx 1/(3\,\text{fan_in})$, tehát nem a He-féle 2/fan_in; a labor ezt az alapértelmezést nem írja felül. A gyakorlati ellenőrzést nem a kezdeti súlyokra, hanem arra alapozza, hogy a seed, az architektúra és a betöltött checkpoint együtt megismételhető legyen. A [D02] szerint az azonos seed nem garantál platformok és verziók közötti bitazonos egyezést; ezért a futásjegyzék a seedet, a könyvtárverziót és az adatmanifesztet is rögzíti. A seed tehát szükséges, de nem elégséges hordozhatósági garancia.

24.4.4 Ütemezés (schedule)

A labor AdamW-t használ $\lambda = 10^{-4}$ decoupled weight decayjel. Az ütemező `StepLR`, `step_size=10` és `gamma=0.5` paraméterrel, azaz

$$\eta_t = \eta_0 \, \gamma^{\lfloor (t-1)/10 \rfloor}, \qquad \eta_0 = 10^{-3}, \ \gamma = 0{,}5.$$

A 20 epochos futásban az 1–10. epoch lépésköze 10^{-3}, a 11–20. epoché $5 \cdot 10^{-4}$. Az ütemezés heurisztika: a konstans kezdeti ráta az első tíz epochban nagyobb lépéseket enged, a tizedik epoch utáni felezés pedig kisebb validációs ingadozást céloz. Ez nem warmup: a ráta nem nő fokozatosan, hanem szakaszonként csökken. Hogy az ütemezés valóban ezt teszi-e, nem elméleti állítás, hanem a validációs loss epochonkénti naplója dönti el; a labor minden epoch falidejét és veszteségét menti.

24.4.5 Augmentáció (augmentation)

Az augmentáció definíciója szigorú: $g : \mathcal{X} \to \mathcal{X}$ olyan címkemegőrző transzformáció, amelyre a feladat értelmezése szerint $y(g(x)) = y(x)$ teljesül. A GlyphLab adatszerződés erre konkrét korlátot ad: a 90°-os forgatás a `vertical` és `horizontal` osztályt felcseréli; a `diagonal` osztály csak az egyik ferde irányt tartalmazza, ezért tükrözés vagy forgatás új célértelmezést igényel. Ezért a geometriai augmentáció ebben az alapkísérletben **nem** címkemegőrző, és nem használjuk.

Címkemegőrző lehetőség az intenzitástérbeli perturbáció, például $g(x) = \text{clamp}_{[0,1]}(ux)$ valamely $u \sim \text{Uniform}(0{,}8;\ 1{,}2)$ mellett, mert az alakzat orientációját geometriailag nem változtatja. A GlyphLab-képek értéke a $[0, 1]$ intervallumban van, ezért a szorzás utáni vágás is ebben az intervallumban történik. A jelen ellenőrző labor mégis augmentáció nélkül fut: az ellenőrzőpont feladata a minimális, reprodukálható alapvonal rögzítése. Az augmentáció hatását az F5 feladat külön kísérletként tervezi, hogy a két változtatás ne keveredjen.

24.4.6 Metrikák (metrics)

A vizsgálat két tesztkörnyezetet különböztet meg: a clean tesztet és a `data/corrupted_test.npy` fájlt, amely ugyanazon 60 tesztkép (30 jelenet × 2 nézet) zajos, fényességgradienssel módosított változata. A corrupt fájl sorrendje a metadata teszt sorainak sorrendje; ezért a célok páronként egyeznek. A labor három fő metrikát rögzít:

- `clean_acc`: a clean teszt 60 képén mért pontosság;
- `corrupt_acc`: a corrupt teszt ugyanazon 60 képen mért pontossága;
- `matched_agreement`: azon clean–corrupt kép-párok aránya, ahol a befagyasztott modell ugyanazt az osztályt adja.

A 23.4. Hibaszeletek és alkalmazási határ alfejezetben kimondott elvet követjük: a corrupt tesztet kizárólag a legjobb, validációs loss alapján kiválasztott checkpoint betöltése után értékeljük; utólagos ráhangolás nincs. Az osztályszintű bontást a riport `per_class_clean` és `per_class_corrupt` mezőkben tárolja, mert a 60 képes összesített pontosság elfedi az osztályszintű hibakoncentrációt. A scene-szintű bootstrapet itt nem számoljuk; a jelen labor egyetlen seeddel jellemzi a kiválasztott receptet, és nem becsli a training-seedek szórását.

24.4.7 Profiling (profiling)

A CPU-s futásban nincs aszinkron GPU-ütemező, ezért a 24.2. CPU/GPU profiling és helyes időmérés CUDA-szinkronizációs szabálya nem lép be: a `time.perf_counter()` két hívása közötti eltelt falidőt méri, nem külön processzoridőt. Az első epochot warmupként (bemelegítő futásként) kezeljük, mert a fájlrendszer-gyorsítótár, a lusta allokációk és a könyvtár első hívásai torzíthatják; a futásjegyzék minden epoch falidejét külön menti, így a steady-state (a bemelegítés utáni, állandósult) szakasz elkülöníthető. A labor CPU-n, `workers=0` DataLoaderrel és `torch.set_num_threads(1)` mellett fut, ezért az adatbetöltés a fő szálon történik, és a számítás egy szálon fut; a labor egyetlen `next(iter(train_loader))` hívással méri az első batch betöltésének falidejét is. A batch szintű aktivációmemória becslése az alakelőírásból adódik: $N = 16$ esetén a legnagyobb köztes tensor a $16 \times 8 \times 24 \times 24$ térkép, 73 728 float32 elem. A `torch.profiler` [D17] részletes operátorbontását ez a CPU-labor nem használja; itt a mérési protokoll és a falidő összehasonlíthatósága a cél.

24.4.8 Checkpoint

A checkpoint minimális szerződése a következő mezőket tartalmazza:

1. `model`: a `state_dict` (a modell tanulható paramétereit és pufferjeit tartalmazó szótár), amely a betöltő oldalon deklarált, azonos architektúrához tartozik;
2. `config`: modellnév, csatornaszám, osztályszám, bemeneti alak és seed;
3. kiválasztási adat: `epoch`, `val_loss`, `val_acc`;
4. adatazonosító: `data_manifest` a `data/manifest.json` relatív útja és SHA-256 lenyomata;
5. `parameter_count`: betöltés utáni konzisztenciaellenőrzéshez.

A betöltés `torch.load(..., weights_only=True)` módon, majd `model.load_state_dict(...)` strict módban történik. A strict mód garantálja a kulcskészlet és alakok egyezését; a labor emellett a `parameter_count` mezőt a jelenlegi modellel, a `data_manifest` SHA-256 értékét pedig a `data/manifest.json` fájllal veti össze. A hibakezelés akkor teljes, ha ezután egy rögzített bemeneti batchre a kimeneti alakot és a végességet is ellenőrizzük. A checkpoint önmagában nem helyettesíti a seedet, az adatverziót és a tanítási konfigurációt; ezért a labor ezeket a `runlog.json` fájlban együtt menti.

24.4.9 Labor: reprodukálható CPU-kísérlet és futásjegyzék

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `lab/data/images.npy`, `labels.npy`, `metadata.csv`, `corrupted_test.npy` és `manifest.json`; futtatás: `python lab/n096_checkpoint.py`. Várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor numerikus kimenetét a könyv környezetében lemértük (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, `torch.set_num_threads(1)`, seed 20261020); a hallgató a saját futtatásából ugyanezt a futásjegyzéket állítja elő. Az alábbi kód a `runlog.json` futásjegyzéket írja, a kódban szereplő `0.0` értékű JSON-mező pedig a séma illusztrációja.

A `lab/data/` mappa a GlyphLab-szerződés szerinti `images.npy`, `labels.npy`, `metadata.csv`, `corrupted_test.npy` és `manifest.json` fájlokat tartalmazza; ezeket a A. függelék generálja. Futtatás a repository gyökeréből:

```
python lab/n096_checkpoint.py
```

A szkript a `Path(__file__)` alapján keresi a `lab/data/` mappát, ezért a `lab/` könyvtárból (`cd lab && python n096_checkpoint.py`) is működik. A kiadói csomag ezt a kódblokkot `lab/n096_checkpoint.py` néven emeli ki.

```python
# lab/n096_checkpoint.py
from pathlib import Path
import hashlib, json, math, sys, time
import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

BASE = Path(__file__).resolve().parent
DATA = BASE / "data"
SEED = 20261020
DEVICE = torch.device("cpu")
DTYPE = torch.float32
EPOCHS = 20
BATCH = 16
LR = 1e-3
WEIGHT_DECAY = 1e-4
STEP_SIZE = 10
GAMMA = 0.5
K = 3

torch.manual_seed(SEED)
np.random.seed(SEED)
torch.set_num_threads(1)

class TinyCNN(nn.Module):
    def __init__(self, in_ch=1, num_classes=K):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten()
        )
        self.fc = nn.Linear(16 * 6 * 6, num_classes)

    def forward(self, x):
        return self.fc(self.features(x))

def count_params(model):
```

```
    return sum(p.numel() for p in model.parameters())

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def evaluate(model, loader, criterion=None):
    model.eval()
    total_loss = 0.0
    correct = 0
    n = 0
    with torch.inference_mode():
        for xb, yb in loader:
            xb = xb.to(DEVICE, dtype=DTYPE)
            yb = yb.to(DEVICE)
            logits = model(xb)
            if criterion is not None:
                total_loss += criterion(logits, yb).item() * xb.shape[0]
            correct += (logits.argmax(dim=1) == yb).sum().item()
            n += xb.shape[0]
    return correct / n, (total_loss / n if criterion is not None else None)

def predict_all(model, loader):
    model.eval()
    preds, ys = [], []
    with torch.inference_mode():
        for xb, yb in loader:
            xb = xb.to(DEVICE, dtype=DTYPE)
            yb = yb.to(DEVICE)
            preds.append(model(xb).argmax(dim=1).cpu().numpy())
            ys.append(yb.cpu().numpy())
    return np.concatenate(preds), np.concatenate(ys)

def per_class_acc(preds, labels):
    out = []
    for c in range(K):
        mask = labels == c
        out.append(float(np.mean(preds[mask] == c)) if mask.sum() else None)
    return out

def main():
    criterion = nn.CrossEntropyLoss()

    # 1) loss-sanity: zero logits -> ln K
    z0 = torch.zeros(4, K, dtype=DTYPE, device=DEVICE)
    y0 = torch.tensor([0, 1, 2, 0], device=DEVICE)
    l0 = criterion(z0, y0).item()
    assert abs(l0 - math.log(K)) < 1e-6, (l0, math.log(K))
    print(f"[sanity] zero-logit loss = {l0:.6f} (expected {math.log(K):.6f})")

    model = TinyCNN().to(DEVICE, dtype=DTYPE)

    # 2) zero-init forward contract
    x_dummy = torch.randn(2, 1, 24, 24, dtype=DTYPE, device=DEVICE)
    default_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
    with torch.no_grad():
        for p in model.parameters():
            p.zero_()
        z_zero = model(x_dummy)
    assert z_zero.shape == (2, K) and torch.allclose(z_zero, torch.zeros_like(z_zero)), z_zero
```

```
model.load_state_dict(default_state)
print(f"[sanity] zero-init output {tuple(z_zero.shape)}, "
      f"all zero: {bool((z_zero == 0).all())}")

# 3) adat a metadata splitje szerint
meta = np.loadtxt(DATA / "metadata.csv", delimiter=",", dtype=str, skiprows=1)
if meta.ndim != 2 or meta.shape[1] != 9:
    raise ValueError("a metadata.csv nem a 9 oszlopos GlyphLab-séma")
indices = meta[:, 0].astype(np.int64)
split = meta[:, 4]  # 0-tól indexelt 5. oszlop: a train/val/test split
images = np.load(DATA / "images.npy").astype(np.float32)
labels = np.load(DATA / "labels.npy").astype(np.int64)
if images.ndim != 4 or images.shape[1:] != (1, 24, 24):
    raise ValueError(f"az images.npy alakja {images.shape}, várt (N,1,24,24)")
if labels.ndim != 1:
    raise ValueError(f"a labels.npy alakja {labels.shape}, várt (N,)")
# A sorrendre nem támaszkodunk: az index oszlop permutációja a tömbindexeknek.
if not (len(meta) == len(images) == len(labels)):
    raise ValueError("a metadata.csv és a tömbök hossza eltér")
if not np.array_equal(np.sort(indices), np.arange(len(images))):
    raise ValueError("a metadata.csv index oszlopa nem permutációja a tömbindexeknek")

def make_loader(split_name, shuffle=False):
    idx = indices[split == split_name]
    x = torch.from_numpy(images[idx]).to(DEVICE, dtype=DTYPE)
    y = torch.from_numpy(labels[idx]).to(DEVICE)
    ds = TensorDataset(x, y)
    gen = torch.Generator().manual_seed(SEED) if shuffle else None
    return DataLoader(ds, batch_size=BATCH, shuffle=shuffle, generator=gen)

train_loader = make_loader("train", shuffle=True)
val_loader = make_loader("val")
test_loader = make_loader("test")

t = time.perf_counter()
next(iter(train_loader))
data_load_s = time.perf_counter() - t

optimizer = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=STEP_SIZE, gamma=GAMMA)

manifest_sha = sha256_file(DATA / "manifest.json")
run = {
    "config": {"seed": SEED, "device": str(DEVICE), "dtype": str(DTYPE),
               "epochs": EPOCHS, "batch": BATCH, "lr": LR,
               "weight_decay": WEIGHT_DECAY,
               "schedule": {"step_size": STEP_SIZE, "gamma": GAMMA}},
    "environment": {"python": sys.version.split()[0],
                    "torch": torch.__version__,
                    "numpy": np.__version__},
    "data_manifest": {"path": "data/manifest.json",
                      "sha256": manifest_sha},
    "parameter_count": count_params(model),
    "data_load_s": data_load_s,
    "epochs": []
}
best = {"epoch": None, "val_loss": float("inf"), "val_acc": None}
total_start = time.perf_counter()

for ep in range(1, EPOCHS + 1):
    model.train()
    t0 = time.perf_counter()
    total_loss = 0.0
```

```
    n = 0
    for xb, yb in train_loader:
        xb = xb.to(DEVICE, dtype=DTYPE)
        yb = yb.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.shape[0]
        n += xb.shape[0]
    train_loss = total_loss / n
    val_acc, val_loss = evaluate(model, val_loader, criterion)
    if not math.isfinite(val_loss):
        raise FloatingPointError(f"nem véges validációs loss a(z) {ep}. epochban")
    scheduler.step()
    wall = time.perf_counter() - t0
    run["epochs"].append({"epoch": ep, "train_loss": train_loss,
                          "val_loss": val_loss, "val_acc": val_acc,
                          "wall_s": wall})
    print(f"epoch {ep:02d} train {train_loss:.4f} val {val_loss:.4f} "
          f"acc {val_acc:.4f} {wall:.3f}s")
    if val_loss < best["val_loss"]:
        best = {"epoch": ep, "val_loss": val_loss, "val_acc": val_acc}
        torch.save({
            "model": model.state_dict(),
            "config": {"name": "TinyCNN", "in_ch": 1, "num_classes": K,
                       "input_shape": [1, 24, 24], "seed": SEED},
            "epoch": ep,
            "val_loss": val_loss,
            "val_acc": val_acc,
            "data_manifest": {"path": "data/manifest.json",
                              "sha256": manifest_sha},
            "parameter_count": count_params(model)
        }, BASE / "best.pt")

if best["epoch"] is None:
    raise RuntimeError("egyetlen epoch sem adott jobb validációs loss-t; nincs best.pt")
ckpt = torch.load(BASE / "best.pt", map_location="cpu", weights_only=True)
if ckpt["parameter_count"] != count_params(model):
    raise ValueError("a checkpoint parameter_count mezője nem egyezik a modellel")
if ckpt["data_manifest"]["sha256"] != manifest_sha:
    raise ValueError("a checkpoint adatmanifesztje nem a jelenlegi manifest.json")
model.load_state_dict(ckpt["model"])
model.eval()

clean_preds, clean_ys = predict_all(model, test_loader)
clean_acc = float(np.mean(clean_preds == clean_ys))

corrupt_np = np.load(DATA / "corrupted_test.npy").astype(np.float32)
test_idx = indices[split == "test"]
if corrupt_np.ndim != 4 or corrupt_np.shape[1:] != (1, 24, 24):
    raise ValueError(f"a corrupted_test.npy alakja {corrupt_np.shape}, várt (N,1,24,24)")
if len(corrupt_np) != len(test_idx):
    raise ValueError("a corrupt teszt hossza nem egyezik a test splittel")
# A corrupt teszt a metadata test sorainak sorrendjében van
# (manifest.json: corrupt_order), ezért a test_loader sorrendjéhez párosul.
corrupt_x = torch.from_numpy(corrupt_np).to(DEVICE, dtype=DTYPE)
corrupt_y = torch.from_numpy(labels[test_idx]).to(DEVICE)
corrupt_loader = DataLoader(TensorDataset(corrupt_x, corrupt_y),
                            batch_size=BATCH, shuffle=False)
corrupt_preds, corrupt_ys = predict_all(model, corrupt_loader)
corrupt_acc = float(np.mean(corrupt_preds == corrupt_ys))
matched_agreement = float(np.mean(clean_preds == corrupt_preds))
```

```
    run["best"] = best
    run["test"] = {
        "clean_acc": clean_acc,
        "corrupt_acc": corrupt_acc,
        "matched_agreement": matched_agreement,
        "per_class_clean": per_class_acc(clean_preds, clean_ys),
        "per_class_corrupt": per_class_acc(corrupt_preds, corrupt_ys)
    }
    run["total_wall_s"] = time.perf_counter() - total_start
    with open(BASE / "runlog.json", "w", encoding="utf-8") as f:
        json.dump(run, f, indent=2)
    print(json.dumps(run["test"], indent=2))

if __name__ == "__main__":
    main()
```

A mért futás kimenete:

```
[sanity] zero-logit loss = 1.098612 (expected 1.098612)
[sanity] zero-init output (2, 3), all zero: True
epoch 01 train 1.1048 val 1.0965 acc 0.3333 0.028s
epoch 02 train 1.0854 val 1.0949 acc 0.5333 0.025s
epoch 03 train 1.0741 val 1.0921 acc 0.5333 0.025s
epoch 04 train 1.0455 val 1.0888 acc 0.4333 0.025s
epoch 05 train 1.0146 val 1.0818 acc 0.4833 0.025s
epoch 06 train 0.9544 val 1.0698 acc 0.4833 0.024s
epoch 07 train 0.8868 val 1.0604 acc 0.5333 0.025s
epoch 08 train 0.8059 val 1.0618 acc 0.5167 0.026s
epoch 09 train 0.7350 val 0.9767 acc 0.5833 0.025s
epoch 10 train 0.6248 val 0.9217 acc 0.6667 0.025s
epoch 11 train 0.5476 val 0.8664 acc 0.7167 0.025s
epoch 12 train 0.5028 val 0.8421 acc 0.6333 0.025s
epoch 13 train 0.4612 val 0.7844 acc 0.7500 0.025s
epoch 14 train 0.4148 val 0.7320 acc 0.8167 0.024s
epoch 15 train 0.3743 val 0.6917 acc 0.8000 0.024s
epoch 16 train 0.3442 val 0.6458 acc 0.8167 0.027s
epoch 17 train 0.3144 val 0.6128 acc 0.8167 0.026s
epoch 18 train 0.2784 val 0.5751 acc 0.8333 0.025s
epoch 19 train 0.2532 val 0.5398 acc 0.8333 0.025s
epoch 20 train 0.2281 val 0.5182 acc 0.8500 0.025s
{
  "clean_acc": 0.9333333333333333,
  "corrupt_acc": 0.85,
  "matched_agreement": 0.9166666666666666,
  "per_class_clean": [
    1.0,
    0.8,
    1.0
  ],
  "per_class_corrupt": [
    0.85,
    0.7,
    1.0
  ]
}
```

Referenciaértékek: a validációs loss 1,0965-ről 0,5182-re csökken; a clean teszt 0,9333, a corrupt 0,85, a `matched_agreement` 0,9167. A mért kimenetet a javítás során megismételtük: a loss- és pontosságértékek a megjelenített négy tizedesjegyig egyeztek, csak a falidők tértek el kisszámú ezredmásodperccel.

A kód által létrehozott `runlog.json` szerkezete:

```
{
  "config": {"seed": 20261020, "device": "cpu", "dtype": "torch.float32",
             "epochs": 20, "batch": 16, "lr": 0.001,
             "weight_decay": 0.0001,
             "schedule": {"step_size": 10, "gamma": 0.5}},
  "environment": {"python": "3.12.3", "torch": "2.15.0+cpu",
                  "numpy": "2.5.4"},
  "data_manifest": {"path": "data/manifest.json",
                    "sha256": "<manifest.json SHA-256>"},
  "parameter_count": 2979,
  "data_load_s": 0.0,
  "epochs": [{"epoch": 1, "train_loss": 0.0, "val_loss": 0.0,
              "val_acc": 0.0, "wall_s": 0.0}],
  "best": {"epoch": 1, "val_loss": 0.0, "val_acc": 0.0},
  "test": {"clean_acc": 0.0, "corrupt_acc": 0.0, "matched_agreement": 0.0,
           "per_class_clean": [0.0, 0.0, 0.0],
           "per_class_corrupt": [0.0, 0.0, 0.0]},
  "total_wall_s": 0.0
}
```


A fenti `0.0` és `<manifest.json SHA-256>` értékek helykitöltők; a mért futás kimenete fentebb látható. A `runlog.json` a teljes 20 epochos listát tartalmazza, valamint a `config`, az `environment` és a `data_manifest` mezőt; a mért futásban a `best` a 20. epoch (`val_loss` 0,5182, `val_acc` 0,85), a `test` pedig a fenti clean/corrupt értékeket adja.

24.4.10 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Mondja ki a labor loss-szerződésének három elemét: reduction, logits vagy valószínűség, target típusa. Adjon ellenpéldát arra, amikor a softmax előzetes alkalmazása más numerikus eredményt ad, mint a szerződéses `CrossEntropyLoss` logits-bemenettel.

F2 (levezetési, nehézség 2). Bizonyítsa be, hogy a labor `TinyCNN` modellje minden súly és bias nullázása esetén tetszőleges $x \in \mathbb{R}^{1\times 24\times 24}$ bemenetre $0 \in \mathbb{R}^3$ kimenetet ad. Pontosan adja meg, mit mutat és mit nem mutat ez a kontroll.

F3 (implementációs, nehézség 2). Írjon `lab/check_best.py` szkriptet, amely a `best.pt` betöltése után ellenőrzi: (a) a `config` mező név- és alakelőírását; (b) a `state_dict` kulcskészletét; (c) egy rögzített `torch.randn(4,1,24,24)` bemenetre a kimeneti alakot és végességet. Adja meg a futtatási parancsot. Előfeltétel: a labor tanítási lépése lefutott, és létrehozta a `best.pt` fájlt; enélkül a szkript `RuntimeError`-ral áll le. Figyelem: a `n096_checkpoint` importja modul szinten állíthat seedet, ezért a szkript a saját seedjét az import után, explicit módon rögzítse, ha a `torch.randn` bemenet ismételhető kell legyen.

F4 (hibakeresési, nehézség 2). A hallgatótárs a tanítási ciklusban minden epoch után a **clean test** pontosságát használja a legjobb epoch kiválasztására, majd a betöltött modellen a corrupt tesztet értékeli. Lokalizálja a hibát, írja le a következményét, és javítsa a protokollt.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy kísérletet, amely az intenzitás-augmentáció (`g(x)=clamp(ux)`, $u \sim U(0{,}8;\ 1{,}2)$) hatását méri a GlyphLab clean és corrupt

tesztjén úgy, hogy a corrupt tesztet nem használja modellválasztásra. Adja meg a protokollt, a baseline-t, a metrikát és legalább két érvényességi korlátot.

24.4.11 Megoldások és indoklás

F1 megoldás. A szerződés: (1) a loss mean reduction, azaz a batch megfigyeléseinek átlaga; (2) a bemenet nyers logits, mert a `CrossEntropyLoss` magában végzi a log-softmaxot; (3) a target `int64` osztályindex, egy megfigyeléshez egy skalár. Ellenpélda: legyen $N = 1$, logit $z = (0, 10)$ és helyes cél $y = 1$. A szerződéses loss: $-\log \frac{e^{10}}{e^{0}+e^{10}} \approx 4{,}54 \cdot 10^{-5}$. Ha előbb softmaxot alkalmazunk, $p \approx (4{,}54 \cdot 10^{-5}, 0{,}99995)$, majd ezt logitnak tekintve a második softmax a helyes osztályra $q_1 \approx 0{,}7310$, a loss $-\log q_1 \approx 0{,}3133$. A dupla softmax tehát számottevően nagyobb, értelmetlen értéket ad (a helyes $4{,}54 \cdot 10^{-5}$ helyett 0,3133).

F2 megoldás. Rétegről rétegre: a Conv-réteg $W = 0$ súllyal és $b = 0$ biassal bármely bemenetre 0 kimenetet ad, mert a kernel-összeg minden tagja $0 \cdot x_{\dots} = 0$. A ReLU nullpontbeli értéke konvenció szerint 0, a `nn.ReLU()` implementációja itt 0-t ad; a MaxPool nulla bemeneten nullát; a Flatten az azonosan nulla tensor alakját $(N, 576)$-ra rendezi, értéke nulla; a Linear $0 \cdot W^{\top} + b = 0$. Ezért a kimenet $(N, 3)$ nulla. A bizonyítás a forward kompozíció működését igazolja bármely, a deklarált $1 \times 24 \times 24$ alakú inputra. Nem mutatja, hogy a modell nemtriviális gradienseket ad, és nem mutatja a tanulás sikerét; a zero init nem töri meg a szimmetriát, ezért tanításra alkalmatlan. A labor ezért csak sanity checkként használja, majd visszaállítja a seedelt alapállapotot.

F3 megoldás.

```python
# lab/check_best.py
from pathlib import Path
import torch
from n096_checkpoint import TinyCNN, DEVICE, DTYPE

BASE = Path(__file__).resolve().parent
ckpt = torch.load(BASE / "best.pt", map_location="cpu", weights_only=True)
cfg = ckpt["config"]
assert cfg["name"] == "TinyCNN"
assert cfg["in_ch"] == 1 and cfg["num_classes"] == 3
assert cfg["input_shape"] == [1, 24, 24]

model = TinyCNN(cfg["in_ch"], cfg["num_classes"]).to(DEVICE, dtype=DTYPE)
model.load_state_dict(ckpt["model"])

expected_keys = {
    "features.0.weight", "features.0.bias",
    "features.3.weight", "features.3.bias",
    "fc.weight", "fc.bias"
}
assert set(ckpt["model"].keys()) == expected_keys, set(ckpt["model"].keys())

x = torch.randn(4, 1, 24, 24, dtype=DTYPE, device=DEVICE)
with torch.inference_mode():
    y = model(x)
assert y.shape == (4, 3)
assert torch.isfinite(y).all()
print("check OK:", tuple(y.shape), sum(p.numel() for p in model.parameters()))
```

Futtatás: `cd lab && python check_best.py`. A mért kimenet `check OK: (4, 3) 2979`. A kontroll szigorú: a strict `load_state_dict` önmagában is hibát dob, ha kulcs vagy alak eltér; a kimeneti alakellenőrzés a 24×24 bemeneti séma és a háromosztályos kimenet együttes tesztje. A tolerancia itt szükségtelen, mert alak- és végességellenőrzésről van szó, nem lebegőpontos egyezésről. A szkript előfeltétele a korábbi tanítási futás `best.pt` fájlja; a `n096_checkpoint`

importja modul szinten is fogyaszthat RNG-t, ezért a rögzített `torch.randn` bemenethez a szkript a saját seedjét az import után állítja be, különben a „rögzített” bemenet a modul importsorrendjétől függene.

F4 megoldás. A hiba a modellválasztás adatszivárgása: a clean test a tanítási ciklus során befolyásolja az epochválasztást, ezért a clean test többé nem független teszt. A következmény kettős: a clean és corrupt teszten mért pontosság szelekciós torzítást kap, és a választott epoch általában optimistább, mint a tényleges generalizáció. A korrekt protokoll: a legjobb epochot kizárólag `val_loss` alapján kell választani; a clean testet csak a befagyasztott checkpoint betöltése után, egyszer szabad értékelni; a corrupt tesztet ugyancsak csak ezután, páros clean-korrupt összehasonlítással. Az újraindítás indoklása: a hibás futásban a test már részt vett a modellválasztásban, ezért a rajta mért érték nem tekinthető független becslésnek. Ha minden epoch checkpointja és a validációs loss is megmaradt, a választás utólag is megismételhető val loss alapján, és a test csak egyszer, a végén értékelhető; ilyenkor nem feltétlenül kell újratanítani. Újratanítás akkor szükséges, ha a checkpointok nem maradtak meg, vagy ha a testinformáció más döntésbe (hiperparaméter, early stopping, adat-előfeldolgozás) is beszivárgott, mert ezeket utólag nem lehet „visszavonni”.

F5 megoldás. Protokoll: (1) baseline = a fenti `TinyCNN` augmentáció nélkül; kezelés = ugyanaz a modell, ugyanaz a kezdő seed, de minden egyes training képhez külön $u \sim U(0{,}8;\ 1{,}2)$ skalárral szorzott, `[0,1]`-re vágott kép (mintánkénti, nem batchenkénti skalár; a batchenkénti közös skalár más kezelést jelentene, mert a batch összes képét azonos mértékben torzítaná). (2) A training/val/test split a GlyphLab szerződéses 120/30/30 scene marad; a val loss alapján mindkét modellnél külön epochválasztás. (3) A kiértékelés a befagyasztott modelleken clean és corrupt teszten `clean_acc`, `corrupt_acc`, `matched_agreement` és osztályszintű bontás. (4) A két modell különbségét páros scene/view szinten számoljuk, mert a view-k összetartoznak. Az „ugyanaz a seed” csak az inicializálást és a kezdeti data order sorrendjét rögzíti: az augmentációs ág extra RNG-fogyasztása miatt a két futás RNG-sorozata már az első augmentált batch után eltér, ezért a seedek azonosítása nem jelenti a teljes véletlenség azonosítását. Döntési szabály: a kezelést akkor tekintjük támogatottnak, ha a corrupt accuracy jelenetszintű páros különbségének 95%-os bootstrap CI-je 0 felett van, miközben a clean accuracy jelenetszintű páros különbségének CI-je nem mutat 2 százalékpontnál nagyobb romlást; ellenkező esetben nem állítunk javulást. Érvényességi korlát: egyetlen seed, ezért az eredmény a training-véletlent nem választja el a kezelés hatásától; a corrupt eloszlás csak a GlyphLab-féle zajt és fényességgradienst reprezentálja, természetes képi shiftre nem vihető át. Emellett az intenzitástranszformáció csak akkor címkemegőrző, ha a feladatszerződés az orientációt a fényességtől függetlenül definiálja; ha a későbbi adatverzió ezt megváltoztatja, a protokollt újra kell validálni.

Hipotézis és egység: az intenzitás-augmentáció javítja a corrupt accuracyt; a változó tényező az augmentáció be- vagy kikapcsolása, a megfigyelési egység a `scene_id`. **Recept, seed, döntési küszöb és erőforrás:** a `TinyCNN` tanítása AdamW optimalizálóval, `lr=10^{-3}`, 20 epoch és 16-os batch mellett fut; a mintamegoldás egy seeddel dolgozik, ennek értéke 20261020, és a korlátot a protokoll öt seedre bővítheti (20261020–20261024). A döntési szabály számszerű: a corrupt accuracy jelenetszintű páros különbségének 95%-os bootstrap CI-je 0 felett van, miközben a clean accuracy CI-je legfeljebb 2 százalékpontos romlást mutat. Erőforrás: CPU, `float32`, két ág, egy seeddel néhány perc; a bővített öt seedes változat tíz futás. ### Tipikus tévedés ellenpéldával

Gyakori tévedés: *„a GPU gyorsabb, tehát a kísérlet helyesebb.”* Ellenpélda: két hallgató ugyanazt a `TinyCNN`-t, ugyanazt a 120/30/30 scene splitet, ugyanazt a batchméretet és ugyanazt a 10 epochot használja; az egyik GPU-n, a másik CPU-n futtat. A GPU-s futás falideje rövidebb, de a validációs loss és a modellválasztás ugyanazon a protokollon alapul, ezért a mérés érvényessége azonos; a GPU jelenléte csak a futásidőt változtatja meg, a mintavételi bizonytalanságot és

a szelekciós torzítást nem. Ha viszont valaki a GPU-s futásban a test accuracy-t használja epochválasztásra, az már nem a GPU miatt más: egyszerre két dolgot változtat (hardver és választási protokoll), ezért a két futás különbsége nem tulajdonítható egyiknek sem.

24.4.12 Önellenőrzés és visszautalás

Önellenőrző kérdések: szerepel-e a loss-szerződésben a mean reduction és az int64 target? A corrupt tesztet tényleg csak a legjobb validációs checkpoint betöltése után értékeli a kód? A `runlog.json` tartalmazza-e a seedet, az adatmanifesztet és az ütemezést? A checkpoint betöltése strict módban történt-e?

Rövid válaszok. Mind a négy igenlő feltétel szükséges: (1) a loss-szerződés mean reductiont és int64 targetet rögzít; (2) a corrupt tesztet csak a legjobb validációs checkpoint betöltése után értékeljük; (3) a `runlog.json` tartalmazza a seedet, az adatmanifesztet és az ütemezést; (4) a checkpointot `strict=True` módban töltjük be. Ha bármelyik hiányzik, a futás nem tekinthető a protokoll szerinti bizonyítéknak.

Visszautalás. a 24.2. CPU/GPU profiling és helyes időmérés alfejezetben tanított warmup-szabály szerint itt az első epoch falidejét a steady-state átlagtól külön kezeljük; a 23.4. Hibaszeletek és alkalmazási határ páros tesztelési elve szerint a corrupt fájl sorrendje a clean teszt sorrendjével párosul.

24.4.13 Rövid összefoglaló

A VI. ellenőrző pont egyetlen csomagba zárja a kis CNN alak- és loss-előírását, a seedelt inicializálást, a StepLR-ütemezést, a címkemegőrző augmentáció korlátját, a clean/corrupt metrikákat, a CPU-falidejű profilmérést és a szigorú checkpoint-betöltést. Az analitikus kontroll igazolja, hogy nulla logitokra a cross-entropy $\ln K$, a zero-init modell pedig minden bemenetre nulla kimenetet ad; ez a forward-lánc működését ellenőrzi, a tanulást nem. A labor teljes CPU-kódot ad, amely `runlog.json` futásjegyzéket ír; a mért futás kimenetét a labor szakasza közli. A clean és corrupt teszt kizárólag a validációs loss alapján kiválasztott checkpointon értékelhető. A nyitó kérdésre a válasz: a paraméterszám és a GPU jelenléte önmagában nem megbízhatósági mérce; a megbízhatóságot a rögzített loss- és alakelőírás, a reprodukálható futásjegyzék és a validációs checkpointon végzett, egyszeri tesztértékelés együtt adja.

24.4.14 Következő lecke

A 25.1. Teljes képosztályozó adatfolyam teljes képosztályozó adatfolyamot épít: az itteni checkpoint- és metrikaszerződést kiterjeszti bináris, többosztályos és multilabel célokra, és a betanított modellt új felhasználási felületre illeszti.

24.4.15 Felhasznált források

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D17] PyTorch contributors (2026). *torch.profiler*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/profiler.html. Hozzáférés: 2026-10-30.

25 Képosztályozás: bináris, többosztályos és multilabel

25.1 Teljes képosztályozó adatfolyam

25.1.1 Cél, előismeret és induló kérdés

Ez az alfejezet a képosztályozó **teljes adatfolyamát** rögzíti: a függelék A. szakaszában leírt nyers GlyphLab-képtől a betanított modell három logitjáig, majd a logitból képzett osztályindexig és osztálynévig. A cél nem egy újabb architektúra bemutatása, hanem az, hogy a hallgató pontosan meg tudja mondani, milyen alakkal, milyen normalizálással, milyen címkeszerződéssel és milyen kiértékelési módban halad végig egy batch. Előismeret: a 20.4. V. ellenőrző pont: indokolt CNN-választás architektúra-döntési lapja, a 22.4. Augmentációs kísérlet és domainváltozás train/eval transzformációk szétválasztása, a 23.4. Hibaszeletek és alkalmazási határ hibaszeletek és a 24.4. VI. ellenőrző pont: megbízható CNN-training megbízható training-protokoll. Visszautalunk a 13.1. Kép mint diszkrét többcsatornás jel NCHW-konvenciójára, a 21.1. Inicializálás és varianciaterjedés inicializálási varianciaterjedésére és a 24.1. Tanulási görbe és hibadiagnózis hibadiagnosztikájára.

Induló kérdés. Egy betanított modell checkpointja három logitot ad vissza; az osztályindexet az argmax, az osztálynevet a mentett label mapping adja. A GlyphLab képek már `float32`, NCHW, $[0, 1]$ tartományúak. Mely lépések maradnak a nyers tömb betöltése után, és hány paraméterrel jár az utolsó réteg? A becslést az alaklánc szakaszának végén ellenőrizheti.

25.1.2 Az osztályozó adatfolyam szerződése

Definíció (osztályozási adatfolyam). Legyen a bemeneti tér $\mathcal{X} = \mathbb{R}^{C_{in} \times H \times W}$, a célhalmaz $\mathcal{Y} = \{0, \dots, K-1\}$, ahol a GlyphLab alapbeállításán $C_{in} = 1$, $H = W = 24$, $K = 3$. Egy osztályozó adatfolyam az alábbi komponensekből áll:

- **Dataset schema:** minden indexhez kép, célindex, valamint a csoport- és splitazonosító tartozik. GlyphLab: `data/images.npy` alakja $(360, 1, 24, 24)$ float32; `data/labels.npy` alakja $(360,)$ int64; `data/metadata.csv` rekordonként `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax` mezőket ad. A split jelenetszinten független (120/30/30 jelenet). A GlyphLab minden jelenetéhez két nézet tartozik, ezért a 120/30/30 jelenet 240/60/60 képet jelent, és egy jelenet két nézete nem kerülhet különböző splitbe. Az `xmin, ymin, xmax, ymax` mezők a forrásséma részei (a későbbi detekciós feladatokhoz); az osztályozó adatfolyam nem használja őket, a doboz nem bemenet és nem címke.
- **Preprocessing:** minden kép ugyanazt a determinisztikus, címkemegőrző leképezést kapja. A GlyphLab képek eleve $[0, 1]$-re normalizáltak, ezért a betöltés után közvetlenül bemenetként használhatók. Ha a modellt μ, σ jellemzőkkel normalizáljuk, akkor ez a pár a modellszerződés része, és a betanításkor, a validációnál, a tesztnél és az inference-nél is azonos módon alkalmazandó. A csak tanításkor mintavételezett augmentációk (22.4.

Augmentációs kísérlet és domainváltozás) a modell mentése után nem részei az inference-szerződésnek.

- **Backbone:** $f_B : \mathbb{R}^{N\times C_{in}\times H\times W} \to \mathbb{R}^{N\times C_{out}\times H'\times W'}$ feature extractor, amely a térbeli leképezések után jellemzőtérképet ad; a kilapítás a head elején, a `forward`-ban történik. A kilapítás PyTorch-konvenciója csatorna-sorrendű: $(N, C_{out}, H', W') \to (N, C_{out}\cdot H'\cdot W')$.
- **Head:** $f_H : \mathbb{R}^{N\times D} \to \mathbb{R}^{N\times K}$ lineáris (affin) rétegek és köztük ReLU-aktivációk kompozíciója; az utolsó réteg aktiváció nélküli, azaz logitot ad.
- **Logits:** a modell kimenete $Z = f_H(f_B(X)) \in \mathbb{R}^{N\times K}$, ahol $z_{n,k} \in \mathbb{R}$ nem valószínűség. A többosztályos eset a logitokra számolt `CrossEntropyLoss`-t használ; a cél int64 osztályindex, és a modellben **nincs** softmax ([D07]).
- **Label mapping:** az $\ell : \mathcal{Y} \to \mathcal{L}$ leképezés, a GlyphLabon

$$\ell(0) = \text{vertical}, \qquad \ell(1) = \text{horizontal}, \qquad \ell(2) = \text{diagonal}.$$

  A mapping a mentett modell szerződésének része, nem a logit értékéből következik.

A konkrét modell legyen

$$f_B : \quad \mathrm{Conv}_{1\to 16}^{k=3,p=1} \to \mathrm{BN}_{16} \to \mathrm{ReLU} \to \mathrm{MaxPool}_2 \to \mathrm{Conv}_{16\to 32}^{k=3,p=1} \to \mathrm{BN}_{32} \to \mathrm{ReLU} \to \mathrm{MaxPool}_2,$$

majd a head

$$f_H : \quad \mathrm{Flatten}_{1152} \to \mathrm{Linear}_{1152\to 64} \to \mathrm{ReLU} \to \mathrm{Linear}_{64\to 3}.$$

25.1.3 Batch-végigkövetés: betöltéstől a logitsig

Számítás (a GlyphCNN shape-lánca). A fenti modellre, ha $X \in \mathbb{R}^{N\times 1\times 24\times 24}$, akkor a közbülső alakok:

$$(N,1,24,24) \xrightarrow{\text{Conv1}} (N,16,24,24) \xrightarrow{\text{Pool}} (N,16,12,12) \xrightarrow{\text{Conv2}} (N,32,12,12) \xrightarrow{\text{Pool}} (N,32,6,6)$$

$$\xrightarrow{\text{Flatten}} (N,1152) \xrightarrow{\text{Head}} (N,64) \to (N,3).$$

Levezetés. A 14.1. Stride, padding, dilation és outputméret alfejezetben levezetett kimeneti méret

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

szerint a Conv1-re $H = 24$, $p = 1$, $k = 3$, $s = 1$, tehát $H_{out} = 24$; a MaxPool $k = 2$, $s = 2$, $p = 0$, tehát 12. A Conv2-re ugyanez $12 \to 12$, a második pool $12 \to 6$. A kilapítás $32 \cdot 6 \cdot 6 = 1152$ skalárt ad mintánként, a lineáris rétegek pedig a mátrixszorzás definíciójából $(N, 1152) \to (N, 64)$ és $(N, 64) \to (N, 3)$ alakot. □

A modell paraméterszáma, $K = 3$ esetén:

Réteg	Paraméterek
Conv2d(1,16,3) + bias	$16 \cdot 9 + 16 = 160$
BatchNorm2d(16)	32
Conv2d(16,32,3) + bias	$16 \cdot 32 \cdot 9 + 32 = 4640$
BatchNorm2d(32)	64
Linear(1152,64) + bias	$1152 \cdot 64 + 64 = 73792$
Linear(64,3) + bias	$64 \cdot 3 + 3 = 195$

Összesen 78 883 tanítható skalár. Ez a kis modell a GlyphLab 240 train képére szánt, előtanítás nélküli kiinduló modell; a TorchVision előtanított modelljei ([D13]) itt nem szükségesek.

25.1.4 Kézi kis példa és ellenőrzés

A kézi ellenőrzéshez nem kell adathalmazt betölteni: a modellt véletlen inputon definiáljuk, és `assert`-ekkel rögzítjük az alakokat.

```
# lab/model_def.py
import torch
import torch.nn as nn

class GlyphCNN(nn.Module):
    def __init__(self, num_classes: int = 3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.head = nn.Sequential(
            nn.Linear(32 * 6 * 6, 64), nn.ReLU(), nn.Linear(64, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(torch.flatten(self.features(x), 1))

# A lab/ munkakönyvtárból:
python - <<'PY'
import torch
from model_def import GlyphCNN
torch.manual_seed(20261020)
model = GlyphCNN()
x = torch.randn(8, 1, 24, 24, dtype=torch.float32)
feat = model.features(x)
z = model(x)
print(feat.shape)   # torch.Size([8, 32, 6, 6])
print(z.shape)       # torch.Size([8, 3])
print(sum(p.numel() for p in model.parameters()))  # 78883
assert feat.shape == (8, 32, 6, 6)
assert z.shape == (8, 3)
assert sum(p.numel() for p in model.parameters()) == 78883
print("shape check passed")
PY
```

A fenti kód futtatása a hallgató gépén ellenőrzi a számított alakokat és a paraméterszámot; a mért kimenet `torch.Size([8, 32, 6, 6])`, `torch.Size([8, 3])`, `78883`, majd `shape check passed` (Python 3.12.3, torch 2.15.0+cpu, CPU).

25.1.5 Labor: GlyphLab osztályozó és új inputra inference-parancs

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5, torch 2.14.1 (CPU) és pandas 2.3.3. Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. Futtatás: `cd lab`, majd `python train_classifier.py`, végül `python infer.py --kind diagonal`. Várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

A labor a `lab/` munkakönyvtárban fut, a GlyphLab eredeti adatfájljaival. A tanítási script betölti a képeket és a célokat, a `metadata.csv` `split` oszlopa alapján train/val indexeket képez, a train splitet jelenetszintű felosztásként kezeli, és a validációs pontosság szerint kiválasztott legjobb checkpoint mentésekor rögzíti a label mappinget is. A példák Python 3.12.3, NumPy 2.5.4, pandas 3.1.0 és torch 2.15.0+cpu alatt futottak; a `weights_only=True` és a `torch.inference_mode()` ebben a PyTorch-verzióban elérhető.

```
# lab/train_classifier.py
import numpy as np, pandas as pd
import torch
from torch.utils.data import DataLoader, TensorDataset
from model_def import GlyphCNN

torch.manual_seed(20261020)
torch.set_num_threads(1)
device = torch.device("cpu")
X = np.load("data/images.npy")          # (360,1,24,24) float32
y = np.load("data/labels.npy")          # (360,) int64
meta = pd.read_csv("data/metadata.csv")
# A `index` oszlop (nem a CSV sorszáma) köti a rekordot az images.npy sorához.
assert list(meta.columns) == ["index", "scene_id", "view_id", "label", "split",
                              "xmin", "ymin", "xmax", "ymax"], meta.columns
assert len(meta) == len(X) == len(y) == 360, (len(meta), X.shape, y.shape)
assert X.shape == (360, 1, 24, 24) and X.dtype == np.float32, (X.shape, X.dtype)
assert y.shape == (360,) and y.dtype == np.int64, (y.shape, y.dtype)
assert meta["index"].tolist() == list(range(len(meta)))
assert set(meta.split) <= {"train", "val", "test"}, set(meta.split)
tr = meta.loc[meta.split == "train", "index"].to_numpy()
va = meta.loc[meta.split == "val", "index"].to_numpy()
te = meta.loc[meta.split == "test", "index"].to_numpy()
assert (len(tr), len(va), len(te)) == (240, 60, 60), (len(tr), len(va), len(te))

Xt, yt = torch.from_numpy(X[tr]), torch.from_numpy(y[tr])
Xv, yv = torch.from_numpy(X[va]), torch.from_numpy(y[va])
Xte, yte = torch.from_numpy(X[te]), torch.from_numpy(y[te])
dl = DataLoader(TensorDataset(Xt, yt), batch_size=32, shuffle=True)

model = GlyphCNN().to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-4)
lossf = torch.nn.CrossEntropyLoss()

best_acc = -1.0
best_epoch = 0
best_state = None
for epoch in range(20):
    model.train()
    for xb, yb in dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = lossf(model(xb), yb)
        loss.backward()
        opt.step()
    model.eval()
```

```
    with torch.inference_mode():
        vacc = (model(Xv.to(device)).argmax(1) == yv.to(device)).float().mean().item()
    print(f"epoch {epoch+1:02d} val_acc={vacc:.4f}")
    if vacc > best_acc:   # szigorú javítás: az első legjobb marad
        best_acc = vacc
        best_epoch = epoch + 1
        best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}

assert best_state is not None
model.load_state_dict(best_state)
model.eval()
with torch.inference_mode():
    test_acc = (model(Xte.to(device)).argmax(1) == yte.to(device)).float().mean().item()
print(f"best epoch {best_epoch:02d} val_acc={best_acc:.4f} test_acc={test_acc:.4f}")

torch.save({
    "model": model.state_dict(),
    "arch": "GlyphCNN",
    "input_schema": "float32 NCHW 1x24x24 [0,1]",
    "labels": {0: "vertical", 1: "horizontal", 2: "diagonal"},
    "best_epoch": best_epoch,
    "val_acc": best_acc,
}, "glyph_cls.pt")
print("saved glyph_cls.pt")
```

Futtatás:

```
cd lab
python train_classifier.py
```

A `val_acc` sorokat a könyv környezetében lefuttatva rögzítettük (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, torch 2.15.0+cpu, CPU, `torch.set_num_threads(1)`, seed 20261020); a mért kimenet:

```
epoch 01 val_acc=0.3667
epoch 02 val_acc=0.6833
epoch 03 val_acc=0.7833
epoch 04 val_acc=0.8167
epoch 05 val_acc=0.9000
epoch 06 val_acc=0.9667
epoch 07 val_acc=0.9667
epoch 08 val_acc=0.9667
epoch 09 val_acc=0.9667
epoch 10 val_acc=0.9667
epoch 11 val_acc=0.9667
epoch 12 val_acc=0.9667
epoch 13 val_acc=0.9667
epoch 14 val_acc=0.9667
epoch 15 val_acc=0.9667
epoch 16 val_acc=0.9667
epoch 17 val_acc=0.9667
epoch 18 val_acc=0.9667
epoch 19 val_acc=0.9667
epoch 20 val_acc=0.9667
best epoch 06 val_acc=0.9667 test_acc=1.0000
saved glyph_cls.pt
```

A végső tesztet a protokoll szerint csak a best checkpoint kiválasztása után, befagyasztott modellen értékeljük (22.4. Augmentációs kísérlet és domainváltozás, 23.4. Hibaszeletek és alkalmazási határ); a tanítási szkript ezt a 20. epoch után egyszer elvégzi, és a `test_acc` értéket is kiírja.

Az **új inputra** szolgáló inference-parancs egy korábban nem látott, helyben előállított mintát készít:

```python
# lab/infer.py
import torch, argparse
from model_def import GlyphCNN

def make_sample(kind: str) -> torch.Tensor:
    x = torch.zeros(1, 1, 24, 24, dtype=torch.float32)
    if kind == "vertical":
        x[..., 6:18, 10:14] = 1.0
    elif kind == "horizontal":
        x[..., 10:14, 6:18] = 1.0
    elif kind == "diagonal":
        idx = torch.arange(24)
        for d in range(-1, 3):      # 4 px széles sáv, mint a másik két minta
            col = (idx + d).clamp(0, 23)
            x[..., idx, col] = 1.0
    return x

p = argparse.ArgumentParser()
p.add_argument("--kind", choices=["vertical", "horizontal", "diagonal"], required=True)
p.add_argument("--ckpt", default="glyph_cls.pt")
a = p.parse_args()

ckpt = torch.load(a.ckpt, weights_only=True)
if "labels" not in ckpt:
    raise KeyError("a checkpoint nem tartalmaz labels leképezést")
labels = {int(k): str(v) for k, v in ckpt["labels"].items()}
model = GlyphCNN()
model.load_state_dict(ckpt["model"])
model.eval()
x = make_sample(a.kind)
with torch.inference_mode():
    z = model(x)
    prob = z.softmax(dim=1)
    yhat = int(z.argmax(dim=1).item())
    logits_np, probs_np = z.numpy(), prob.numpy()
print("logits:", logits_np)
print("probs:", probs_np)
print("predicted:", labels[yhat])
```

```
python infer.py --kind diagonal
```

A kimenet a fenti, 20 epochos tanítás és a 6. epoch legjobb checkpointjának betöltése után mért futásból származik (Python 3.12.3, torch 2.15.0+cpu, CPU):

```
logits: [[-6.286462 -8.311524  4.678683]]
probs: [[1.7293762e-05 2.2825275e-06 9.9998045e-01]]
predicted: diagonal
```

Értelmezés: a softmax pozitív, összegük 1; az argmax a legnagyobb logithoz tartozó indexet adja, a név pedig a mentett `labels` mezőből jön, nem a logit abszolút nagyságából.

25.1.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1, elvárt eredmény: 3–4 mondatos válasz). Melyik transzformáció lehet része az inference-szerződésnek, és melyik nem? Indokoljon egy-egy példával a GlyphLab adatkészleten.

F2 (levezetési, nehézség 2, elvárt eredmény: alaklánc és flatten dimenzió). Vezesse le a 3. szakasz (alaklánc) shape-képletével: egy $N \times 1 \times 32 \times 32$ bemenetre `Conv2d(1,16,3,padding=1)`, `MaxPool2d(2)`, `Conv2d(16,24,3,padding=1)`, `MaxPool2d(2)` után mekkora a kilapított jellemződimenzió?

F3 (implementációs, nehézség 2, elvárt eredmény: futtatható függvény). Írjon `predict_from_array(x_np)` függvényt, amely egy `(24,24)` float32 NumPy-tömböt vár, és `(osztályindex, valószínűségvektor)` párt ad vissza a fenti modellből. A modell betöltése legyen a függvényen kívül, `model.eval()` és `torch.inference_mode()` használatával.

F4 (hibakeresési, nehézség 2, elvárt eredmény: ok, következmény, javítás, kontroll). A hallgató a mentett checkpointot egy másik szkriptben így értékeli:

```
NAMES = {0: "horizontal", 1: "vertical", 2: "diagonal"}
```

A tanítási val accuracy $59/60 \approx 0{,}98$ volt, de az új szkript a nevekre számolt accuracy értéke 0,33 körül van. Hol a hiba?

F5 (kísérleti/transzfer, nehézség 3, elvárt eredmény: protokollvázlat). Tervezzen protokollt a kiinduló modell átvitelére: az új procedurális adatgenerátor 32×32 RGB képeket ad, ugyancsak vertical/horizontal/diagonal címkékkel. Mi változik a preprocessingben, a backbone első rétegében és a head bemeneti dimenziójában? Adjon baseline-t és validálási elvet.

25.1.7 Megoldások és indoklás

F1. Az inference-szerződésbe csak determinisztikus, címkemegőrző leképezések kerülhetnek: a GlyphLab esetében a `float32` NCHW $[0,1]$ tartomány már adott, esetleg a rögzített μ, σ normalizálás. A véletlen transzláció, forgatás vagy zaj csak a tanítási augmentáció része; ha a betanított modell kiértékelésekor is véletlenítünk, a $P(Y \mid X)$ becslés nem a szerződött bemenetre vonatkozik, és a clean/corrupt párosítás (22.4. Augmentációs kísérlet és domainváltozás) is felborul. Forgatásnál ügyelni kell a címkemegőrzésre: a 180°-os forgatás megőrzi a vonalorientációt, a 90°-os viszont a vertical és a horizontal osztályt felcseréli, ezért az orientációcímke szempontjából nem címkemegőrző.

F2. Az egyes lépések: $32 \xrightarrow{\text{Conv}} 32 \xrightarrow{\text{Pool}} 16 \xrightarrow{\text{Conv}} 16 \xrightarrow{\text{Pool}} 8$. Az utolsó aktivációs tér $N \times 24 \times 8 \times 8$, tehát a kilapított dimenzió $24 \cdot 8 \cdot 8 = 1536$. A head első lineáris rétegének bemenete ezért 1536.

F3.

```
import torch
from model_def import GlyphCNN

model = GlyphCNN()
model.load_state_dict(torch.load("glyph_cls.pt", weights_only=True)["model"])
model.eval()
LABELS = {0: "vertical", 1: "horizontal", 2: "diagonal"}

def predict_from_array(x_np):
    x = torch.from_numpy(x_np).float().unsqueeze(0).unsqueeze(0)  # (1,1,24,24)
    with torch.inference_mode():
        z = model(x)
        p = torch.softmax(z, dim=1)
        yhat = int(z.argmax(dim=1).item())
    return yhat, p.numpy()[0]
```

Alakellenőrzés: a bemenet `(24,24)` float32-ból `(1,1,24,24)` lesz, a kimenet valószínűségvektor `(3,)`; az argmax a $\{0, 1, 2\}$ halmazból választ.

F4. Ok: a `NAMES` szótár nem azonos a modell tanításakor használt label mappinggel; a checkpoint `labels` mezője `{0: vertical, 1: horizontal, 2: diagonal}`, tehát a kiértékelő a 0 indexet tévesen „horizontal"-ként kezeli. **Következmény:** a confusion matrix sor/oszlop nevei elcsúsznak. Legyen C a confusion matrix, C_{ij} a valódi i és a prediktált j osztályú minták száma; az indexekre számolt accuracy $\sum_i C_{ii} / \sum_{ij} C_{ij}$, és ez változatlanul $59/60 \approx 0{,}98$. A nevekre számolt accuracy viszont a felcserélt névmapping miatt más: a teszt kiegyensúlyozott (20-20-20), ezért közel tökéletes modellnél a vertical és horizontal predikciók neve felcserélődik, és csak a 20 diagonal minta marad név szerint is helyes, azaz $20/60 = 1/3 \approx 0{,}33$; a néhány tévesztés ezt legfeljebb kissé módosítja. A nevekre „hamisan alacsony" accuracy tehát nem a modell hibája, hanem a mapping hibája. **Javítás:** a kiértékelő szkript a checkpoint mentett `labels` mezőjét olvassa be, vagy a külső rögzített szerződés szerint ugyanazt a mappinget használja. **Kontroll:** számoljuk ki a nyers indexekre a `(pred == y).mean()` értéket; ennek egyeznie kell a tanítási val accuracy-vel.

F5. Protokollvázlat: az új adatgenerátor minden jelenetét 32×32 RGB-ben menti; a képeket float32 `[0,1]` NCHW alakra konvertáljuk, és az új adatra becsült csatornánkénti μ, σ normalizálást rögzítünk. A backbone első konvolúciója `Conv2d(3,16,3,padding=1)` lesz, a második konvolúció 32 csatornás marad. A 32×32 bemenet alaklánca $32 \to 32 \to 16 \to 16 \to 8$, ezért a head előtti jellemzőtérkép $32 \times 8 \times 8$, a kilapított dimenzió $32 \cdot 8 \cdot 8 = 2048$; a head első lineáris rétegét `Linear(2048,64)`-re kell cserélni. A 1536 csak akkor lenne helyes, ha a második konvolúció 24 csatornás lenne (mint az F2 eltérő példájában), de a kiinduló modell 16→32 architektúrájánál nem. Baseline: az eredeti 24×24 modellre resizelt (pl. bilineáris) képek kiértékelése, változó tényező: a natív 32×32 bemenetre tanított modell. A split jelenetszintű rétegzett; metrika az osztályonkénti accuracy és a makro-átlagolt accuracy (a kiegyensúlyozott teszt miatt a kettő közel azonos), a bizonytalanságot scene-rétegzett bootstrap CI-vel ([S04] csak a feltételes ismételt mintavételi értelmezéshez, a GlyphLabra vonatkozó csoportos tervet a 23.4. Hibaszeletek és alkalmazási határ alfejezetben tárgyaltak szerint). Három seeddel (például 0, 1, 2) ismételjük mindkét ágat, és a seedek közötti szórást külön jelentjük. Döntési szabály: az átvitelt akkor tekintjük támogatottnak, ha a natív 32×32 modell és a resizelt baseline accuracy-jának jelenetszintű páros különbségére a 95%-os bootstrap CI teljes egészében 0 felett van; ha a CI tartalmazza a 0-t, nem állítunk javulást. Az eredmény csak erre a mesterséges shiftre érvényes, természetes képi általánosításra nem.

Hipotézis és egység: a natív 32×32 bemenetre tanított modell jobb accuracyt ad, mint a 24×24-re resizelt baseline; a megfigyelési egység a `scene_id`. **Recept, seedek és erőforrás:** a tanítás AdamW optimalizálóval, `lr=10^{-3}`, 20 epoch és 16-os batch mellett fut; a három seed értéke 20261020–20261022, mindkét ágban ugyanazok. A döntési szabály a jelenetszintű páros bootstrap 95%-os CI-je, legalább 2 százalékpontos accuracy-különbséggel. Erőforrás: CPU, `float32`, két ág × három seed = hat futás, néhány perc. ### Tipikus tévedés és ellenpélda

A leggyakoribb hiba a **label index–név leképezés** kezelése. A checkpoint önmagában csak az indexeket adja vissza; ha a felhasználó külső szótárat rendel, az nem a betanított modell szerződése. Legyen egy modell, amelyet a $\{0 \mapsto \text{diagonal}, 1 \mapsto \text{vertical}, 2 \mapsto \text{horizontal}\}$ mappinggel tanítottak. Ha a kliens az F4-beli `NAMES = {0: "horizontal", ...}` szótárt használja, egy olyan bemenetre, ahol a valódi osztály diagonal, a modell z_0 logitja lesz a legnagyobb; a kliens `NAMES[0]` alapján „horizontal"-t jelent, tehát a numerikus index helyes, a közölt név mégis téves. A mapping ezért a mentési rekord kötelező eleme.

Önellenőrzés. A 3. szakaszban (alaklánc) minden közbülső alak és a paraméterszám explicit; a CrossEntropyLoss bemenete logits, a cél int64; a normalizálás minden fázisban azonos; a

checkpoint az architektúra, a bemeneti séma és a label mapping mellett lett mentve. NCHW-konvenció a 13.1. Kép mint diszkrét többcsatornás jel szerint, az inicializálási skála a 21.1. Inicializálás és varianciaterjedés szerint marad a kis modell stabil tartományában.

25.1.8 Rövid összefoglaló

Az osztályozó adatfolyam a dataset schema, a determinisztikus preprocessing, a backbone, a head, a logits és a label mapping együttes szerződése. A GlyphLabon a bemenet float32 NCHW $1 \times 24 \times 24$, $[0, 1]$; a modell két konvolúciós blokk után $32 \times 6 \times 6$ jellemzőtérképet ad, amelyet a head elején 1152 dimenzióra lapítunk, majd két lineáris réteggel három logitot adunk. A batch alaklánca: $(N, 1, 24, 24) \to (N, 16, 12, 12) \to (N, 32, 6, 6) \to (N, 1152) \to (N, 3)$. A végrehajtott művelet: `CrossEntropyLoss` logitokból, int64 cél; a modellben nincs softmax. A mentett checkpoint magában foglalja az architektúrát, a bemeneti sémát és a label mappinget, mert a logitok indexeiből a nevek nem rekonstruálhatók.

25.1.9 Következő lecke

A 25.2. Multilabel és hierarchikus cél a bináris, többosztályos és multilabel célok közötti eltérést vizsgálja: a multilabel fej kimenete (N, K) logits lesz `BCEWithLogitsLoss`-szal és `{0,1}^K` céllal, ahol a jelen alfejezetben rögzített adatfolyam- és checkpoint-szerződés mintájára kell a maszk- és threshold konvenciókat is menteni.

25.1.10 Felhasznált források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D13] PyTorch contributors (2026). *TorchVision models*. TorchVision dokumentáció (stable). https://docs.pytorch.org/vision/stable/models.html. Hozzáférés: 2026-10-30. A katalógus itt viszonyítási pont; a bemutatott modell előtanítás nélküli, saját kis háló.
- [S04] Efron, B. és Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall. Csak az intervallum feltételes, ismételt mintavételi értelmezéséhez.

25.2 Multilabel és hierarchikus cél

25.2.1 Cél, előismeret és induló kérdés

A 25.1. Teljes képosztályozó adatfolyam egységben egy teljes képosztályozó adatfolyamot építettünk fel: a batch a betöltéstől a háromlogitos kimenetig egyetlen rögzített alakelőírást követett, és a címke egyetlen kölcsönösen kizáró osztályindex volt. Ebben az alfejezetben azt a kérdést vizsgáljuk, mi történik, ha egy képen **egyszerre több** címke is igaz lehet. Előismeretnek elegendő a 25.1. Teljes képosztályozó adatfolyam alfejezetből a logit, a fej és a címke fogalma, valamint a 23.2. Kalibráció és döntési küszöb alfejezetből a döntési küszöb szerepe. Induló becslés: két független bináris címke esetén hány kimeneti számra van szüksége a modellnek, és ezeknek összege kötelezően 1 kell-e legyen?

25.2.2 Definíciók és intuíció

Legyen a batch mérete N, a címkék száma K. A **multilabel** feladatban minden n megfigyeléshez egy

$$Y \in \{0,1\}^{N\times K}$$

célmátrix tartozik, ahol $Y_{n,k} = 1$ azt jelenti, hogy a k-adik címke jelen van az n-edik képen, $Y_{n,k} = 0$ pedig azt, hogy a megfigyelt annotáció szerint nincs jelen. A modell itt is logitokat ad:

$$Z \in \mathbb{R}^{N\times K},$$

tehát ugyanaz a háló és fej használható, mint a többosztályos esetben; a különbség a normalizálásban és a cél értelmezésében van. A többosztályos cél $y \in \{0,\dots,K-1\}^N$ int64 index, és a softmax egyetlen valószínűségvektorrá normalizálja a logitokat. A multilabel célban ezzel szemben minden címke külön Bernoulli-döntés, ezért az elemenkénti szigmoidot használjuk:

$$\hat{P}(Y_{n,k}=1) = \hat{p}_{n,k} = \sigma(z_{n,k}) = \frac{1}{1+e^{-z_{n,k}}}.$$

Az n-edik sor $(\hat{p}_{n,1},\dots,\hat{p}_{n,K})$ elemei nem alkotnak szükségképpen 1 összegű eloszlást: egy képen lehet egyszerre „jármű” és „éjszakai felvétel”, vagy éppen egyik sem. A **kölcsönösen kizáró** címkék ezzel szemben egy K osztályos osztályozás kategóriái; ott a softmax az összes logit együttes függvényeként normál. Kétosztályos határesetben a kapcsolat ragadható meg: ha $z_1 = z$ és $z_2 = 0$, akkor

$$\frac{e^{z_1}}{e^{z_1}+e^{z_2}} = \frac{e^{z}}{e^{z}+1} = \sigma(z),$$

ezért a bináris osztályozás egyetlen szigmoidja a softmax speciális paraméterezése. Ez **nem** jelenti azt, hogy két független multilabel szigmoid ugyanaz, mint egy kétosztályos softmax: az utóbbi $(\hat{p}, 1-\hat{p})$ kényszert ad, az előbbi két tetszőleges $[0,1]$-beli számot.

A válasz az induló kérdésre. Két független bináris címkéhez két logit és két szigmoid kell, tehát a kimeneti számok száma 2; ezek összege nem kötelezően 1, mert a két címke nem kizáró, ezért a softmax normalizálás itt nem alkalmazható.

A szigmoid numerikusan stabil alakja:

$$\sigma(z) = \begin{cases} \frac{1}{1+e^{-z}}, & z \ge 0, \\ \frac{e^{z}}{1+e^{z}}, & z < 0. \end{cases}$$

Így mindkét irányban elkerüljük a túl nagy exponenciális értéket.

25.2.3 A multilabel BCE és a megfigyelési maszk

A többosztályos eset a [D07] szerint logit bemenetet és int64 osztályindexet vár a `CrossEntropyLoss`-ban. A multilabel eset célja a bináris cross-entropy logitokkal, PyTorch-ban a `BCEWithLogitsLoss` (a bináris cross-entropy logit-alapú veszteségfüggvénye; dokumentáció). Vezessük le a célfüggvényt egyetlen (n, k) elemre. A Bernoulli-modell likelihoodja:

$$p(Y_{n,k} = y_{n,k} \mid z_{n,k}) = \hat{p}_{n,k}^{\,y_{n,k}} (1 - \hat{p}_{n,k})^{1 - y_{n,k}}, \qquad y_{n,k} \in \{0, 1\}.$$

A negatív log-likelihood:

$$-\log p(y_{n,k} \mid z_{n,k}) = -y_{n,k} \log \sigma(z_{n,k}) - (1 - y_{n,k}) \log(1 - \sigma(z_{n,k})).$$

Mivel $1 - \sigma(z) = \sigma(-z)$ és $\log \sigma(t) = -\log(1 + e^{-t})$, az első tag $y_{n,k} \log(1 + e^{-z_{n,k}})$, a második pedig $(1 - y_{n,k}) \log(1 + e^{z_{n,k}})$. A két eset összevonható:

$$\ell(z_{n,k}, y_{n,k}) = \log(1 + e^{z_{n,k}}) - y_{n,k} z_{n,k}.$$

A $\log(1 + e^{z})$ függvényt `softplus`-nak nevezzük; a numerikus túlcsordulás elkerülésére javasolt stabil megvalósítása $\operatorname{softplus}(z) = \log(1 + e^{-|z|}) + \max(z, 0)$; ez a forma nem változtat a memóriaigényen. A gradiens a láncszabályból:

$$\frac{\partial \ell}{\partial z_{n,k}} = \sigma(z_{n,k}) - y_{n,k}.$$

Ez megegyezik azzal az intuitív szabállyal, hogy pozitív cél mellett a predikciót 1 felé, negatív cél mellett 0 felé húzzuk; a gradiens csak határértékben tart nullához: $y_{n,k} = 1$ esetén $z_{n,k} \to +\infty$, $y_{n,k} = 0$ esetén $z_{n,k} \to -\infty$ mellett; véges z-re $\sigma(z) \in (0, 1)$, ezért a gradiens nem nulla.

Most mondjuk ki a **megfigyelt címkék maszkolásának szabályát**.

25.2.3.1 Tétel (maszkolt multilabel BCE)

Legyen $Z \in \mathbb{R}^{N \times K}$ a logitmátrix, $Y \in \{0, 1\}^{N \times K}$ a megfigyelt célmátrix, és $M \in \{0, 1\}^{N \times K}$ a **megfigyelési maszk**, ahol $M_{n,k} = 1$ pontosan akkor, ha az (n, k) címke annotáltsága ismert, $M_{n,k} = 0$ pedig ha hiányzik. Tegyük fel, hogy a bemenet feltétele mellett a címkék feltételesen független Bernoulli-változók $\sigma(z_{n,k})$ paraméterrel, és a hiányzási mechanizmus **ignorálható**: (i) a megfigyelés valószínűsége a bemenet ismeretében nem függ a nem megfigyelt valódi címkétől, $P(M_{n,k} = 1 \mid x_n, Y_{n,k}) = P(M_{n,k} = 1 \mid x_n)$ (MAR; speciális esete a teljesen véletlen hiányzás, MCAR, ahol a megfigyelés valószínűsége az x_n-től sem függ; a hiányzási mechanizmusok klasszikus tárgyalása Rubin, 1976), és (ii) a hiányzási mechanizmus paraméterei elkülönülnek a modell paramétereitől. Legyen $S = \sum_{n,k} M_{n,k} > 0$. Ekkor a megfigyelt adatok negatív log-likelihoodjának átlagos értéke

$$L(Z, Y, M) = \frac{1}{S} \sum_{n=1}^{N} \sum_{k=1}^{K} M_{n,k} \left[\log(1 + e^{z_{n,k}}) - Y_{n,k} z_{n,k} \right].$$

Üres maszk, azaz $S = 0$ esetén a loss nem definiált; megvalósításban vagy hibát adunk, vagy a hívási szerződésben rögzített módon 0-t adunk vissza, de ezt a hívónak dokumentálnia kell.

Bizonyítás. A teljes adat likelihoodja a (Y, M) párra $P(Y, M \mid X) = P(Y \mid X)P(M \mid X, Y)$. A MAR-feltétel szerint $P(M \mid X, Y) = P(M \mid X)$, ezért a hiányzási mechanizmus tényezője nem függ a modell paramétereitől, és a likelihood maximalizálásakor konstansként elhagyható. A feltételes függetlenség miatt $P(Y \mid X) = \prod_{n,k} \sigma(z_{n,k})^{Y_{n,k}}(1 - \sigma(z_{n,k}))^{1-Y_{n,k}}$. A hiányzó $Y_{n,k}$ értékekre a likelihoodban a Bernoulli-eloszlás két lehetséges értékére összegzünk; mivel $\sigma(z) + (1 - \sigma(z)) = 1$, ez a marginalizálás egységnyi tényezőt ad, ezért a hiányzó elemek nem járulnak hozzá a likelihoodhoz. A megfigyelt elemek negatív log-likelihoodja elemenként a $\log(1 + e^{z_{n,k}}) - Y_{n,k}z_{n,k}$ tag, amelyeket $M_{n,k}$ kiválaszt, és $1/S$-szeres átlagol. Ez éppen a fenti L. ■

Az ignorálhatósági feltétel mellett a megfigyelési mechanizmus tényezője a likelihoodban a modellparaméterektől független konstans, ezért a megfigyelt-adat likelihood maximalizálása a fenti maszkolt loss minimalizálása. A feltétel nem technikai részlet: **MNAR** (missing not at random) esetén $P(M_{n,k} = 1 \mid x_n, Y_{n,k}) \neq P(M_{n,k} = 1 \mid x_n)$, azaz a megfigyelés valószínűsége a valódi címke értékétől is függ (például a pozitív címkéket gyakrabban jegyzik fel); ilyenkor a megfigyelt célok eloszlása torzított, és a maszkolt loss sem ad torzítatlan populációs illesztést, hanem a megfigyelési mechanizmust is modellezni kell. A maszkolás tehát elkerüli a hamis negatív kényszert, de önmagában nem garantál torzítatlan becslést.

A tétel gyakorlati tartalma: **a hiányzó címke nem kap $Y = 0$ célértéket, hanem kimarad az összegből.** A PyTorch-ban a `BCEWithLogitsLoss(reduction='none')` adja a tagokat (`reduction`: a loss összegzésének módja), a maszkkal szorozva és a maszk összegével osztva kapjuk a normalizált loss-t. Fontos, hogy a default (alapértelmezett) `reduction='mean'` az **összes $N \cdot K$ elem** átlagát venné (a hiányzókat is beszámítva), ami maszkolt elemeknél nem helyes normalizálás; mindig explicitté kell tenni, mit nevezünk egy megfigyelésnek.

25.2.4 Kézi kis példa

Legyen $N = 2$, $K = 3$, és

$$Z = \begin{pmatrix} -1{,}5 & 0{,}8 & 2{,}2 \\ 0{,}2 & -0{,}4 & 1{,}1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \qquad M = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}.$$

A második sor középső címkéje tehát hiányzik: $M_{2,2} = 0$, ezért annak $Y_{2,2} = 1$ értékét **nem** használjuk célként. A szigmoid értékek: $\sigma(-1{,}5) \approx 0{,}1824$, $\sigma(0{,}8) \approx 0{,}6900$, $\sigma(2{,}2) \approx 0{,}9002$, $\sigma(0{,}2) \approx 0{,}5498$, $\sigma(-0{,}4) \approx 0{,}4013$, $\sigma(1{,}1) \approx 0{,}7503$.

Elemenként a $\text{softplus}(z) - Yz$ képlettel, amely pozitív célra ($Y = 1$) $\text{softplus}(-z)$, negatív célra ($Y = 0$) $\text{softplus}(z)$:

(n, k)	Y	z	BCE-tag
$(1, 1)$	1	$-1{,}5$	$\text{softplus}(1{,}5) = \log(1 + e^{1{,}5}) \approx 1{,}7014$
$(1, 2)$	0	$0{,}8$	$\text{softplus}(0{,}8) = \log(1 + e^{0{,}8}) \approx 1{,}1711$
$(1, 3)$	1	$2{,}2$	$\text{softplus}(-2{,}2) = \log(1 + e^{-2{,}2}) \approx 0{,}1051$

(n, k)	Y	z	BCE-tag
$(2, 1)$	1	0,2	$\text{softplus}(-0{,}2) = \log(1 + e^{-0{,}2}) \approx 0{,}5981$
$(2, 2)$	1	$-0{,}4$	maszkolt ($M = 0$): kimarad az összegből
$(2, 3)$	0	1,1	$\text{softplus}(1{,}1) = \log(1 + e^{1{,}1}) \approx 1{,}3873$

Az összeg 4,9631, a maszk összege $S = 5$, így $L \approx 0{,}9926$. A legnagyobb tag az $(1, 1)$ elem: pozitív cél mellett erősen negatív logit, tehát magabiztos téves predikció. Ha tévesen a hiányzó $(2, 2)$ elemet negatívként vennénk be, a $\text{softplus}(-0{,}4) \approx 0{,}5130$ tag is beleszámít, és a loss $5{,}4761/6 \approx 0{,}9127$ lenne; a különbség kicsinek tűnik, de a gradiens összehasonlítása megmutatja a torzítást:

$$\frac{\partial \ell(2, 2)}{\partial z_{2,2}} = \sigma(-0{,}4) - Y'_{2,2} \approx 0{,}4013 - 0 = 0{,}4013, \qquad Y'_{2,2} = 0 \text{ (téves negatív cél)},$$

holott a valódi $Y_{2,2} = 1$ cél maszkolatlan gradiense $\sigma(-0{,}4) - 1 \approx -0{,}5987$ lenne, a maszkolt változatban pedig ehhez az elemhez nulla gradiens tartozik. A modell tehát hamis negatív cél irányába tanulna egy olyan példán, ahol valójában nincs is megfigyelt negatív annotáció.

25.2.5 Megvalósítási ellenőrzés

Az alábbi függvény a PyTorch nélküli, NumPy referenciát adja meg; a laborban ugyanezt a `torch.nn.functional.binary_cross_entropy_with_logits` függvénnyel ellenőrizzük.

```
import numpy as np

def stable_sigmoid(z: np.ndarray) -> np.ndarray:
    # float64 bemenet; z >= 0 elágazással a túlcsordulást elkerüljük
    out = np.empty_like(z, dtype=np.float64)
    pos = z >= 0
    e = np.exp(-np.abs(z))
    out[pos] = 1.0 / (1.0 + e[pos])
    out[~pos] = e[~pos] / (1.0 + e[~pos])
    return out

def masked_bce_with_logits(z: np.ndarray, y: np.ndarray, mask: np.ndarray) -> float:
    # z, y, mask alakja (N, K); y értéke 0 vagy 1; mask értéke 0 vagy 1
    z = np.asarray(z, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    mask = np.asarray(mask, dtype=np.float64)
    if z.shape != y.shape or z.shape != mask.shape:
        raise ValueError("Alakegyezés szükséges: (N, K) mindhárom bemenetre.")
    s = mask.sum()
    if s <= 0:
        raise ValueError("Üres maszk: nincs megfigyelt címke.")
    # stabil softplus: log(1 + e^z) = softplus(z)
    softplus = np.logaddexp(0.0, z)
    terms = softplus - y * z
    return float(np.sum(mask * terms) / s)
```

A függvény a fenti példára 0,9926-ot ad (float64-ben, azaz dupla pontosságú lebegőpontos számítással: 0,992614292). A PyTorch-oldali ellenőrzés:

```
import torch

z = torch.tensor([[-1.5, 0.8, 2.2], [0.2, -0.4, 1.1]], dtype=torch.float64)
y = torch.tensor([[1.0, 0.0, 1.0], [1.0, 1.0, 0.0]], dtype=torch.float64)
mask = torch.tensor([[1.0, 1.0, 1.0], [1.0, 0.0, 1.0]], dtype=torch.float64)

terms = torch.nn.functional.binary_cross_entropy_with_logits(
    z, y, reduction='none')
loss = (mask * terms).sum() / mask.sum()
print(loss.item())  # mért: 0.9926142916392499
# (a NumPy-referenciához mért eltérés float64-ben 0.0)
print(terms)        # mért: 1.7014 1.1711 0.1051 / 0.5981 0.9130 (maszkolt, kimarad) 1.3873
```

A fenti két értéket a kód futtatásával kaptuk (Python 3.12.3, torch 2.15.0+cpu, CPU); a NumPy-referencia ugyanezt a 0,9926142916392499 értéket adja.

25.2.6 Labor: szintetikus multilabel fej és metrika

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), torch 2.14.1 (CPU). Külső adatfájl nem kell: a labor szintetikus jellemzőket generál. Futtatás: `python lab/n098_multilabel_lab.py`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A labor célja egy kis, CPU-n ellenőrizhető multilabel osztályozó összeállítása. A függelék A. szakaszában leírt GlyphLab három kölcsönösen kizáró osztálya erre nem alkalmas, ezért eredeti szintetikus példát használunk: $N = 96$ képhez 12 dimenziós fix mesterséges jellemzőt rendelünk, és $K = 4$ címkét figyelünk meg. A jellemzők nem valós képekből származnak; a labor a fej, a loss és a metrika szerződését vizsgálja, nem a látási teljesítményt. A maszk MCAR: a `torch.rand(N, K) > 0{,}3` feltétel minden elemre 30% valószínűséggel hiányzik, függetlenül a címkétől és a jellemzőktől.

```
# lab/n098_multilabel_lab.py
# Futtatás a repository gyökeréből: python lab/n098_multilabel_lab.py
import torch
import torch.nn as nn

torch.manual_seed(20261020)
torch.set_num_threads(1)
N, K, D = 96, 4, 12
X = torch.randn(N, D, dtype=torch.float32)          # (96, 12)
W_true = torch.randn(D, K, dtype=torch.float32) * 0.5
logit_true = X @ W_true
prob_true = torch.sigmoid(logit_true)
Y = (prob_true > 0.5).float()                        # (96, 4)
# 30% véletlenszerű hiányzó annotáció; a maszk MCAR: független a címkétől
mask = (torch.rand(N, K) > 0.3).float()              # (96, 4)

class MultilabelHead(nn.Module):
    def __init__(self, d: int, k: int):
        super().__init__()
        self.linear = nn.Linear(d, k)

    def forward(self, x):
        return self.linear(x)                        # logitok, nincs sigmoid

model = MultilabelHead(D, K)
```

```
opt = torch.optim.SGD(model.parameters(), lr=0.5)

for step in range(20):
    opt.zero_grad()
    z = model(X)                                        # (96, 4)
    terms = torch.nn.functional.binary_cross_entropy_with_logits(
        z, Y, reduction='none')
    loss = (mask * terms).sum() / mask.sum()
    loss.backward()
    opt.step()

with torch.no_grad():
    z_eval = model(X)
    terms_eval = torch.nn.functional.binary_cross_entropy_with_logits(
        z_eval, Y, reduction='none')
    final_loss = (mask * terms_eval).sum() / mask.sum()
    pred = (torch.sigmoid(z_eval) > 0.5).float()
    obs = mask.bool()
    acc = (pred[obs] == Y[obs]).float().mean().item()
    tp = ((pred == 1) & (Y == 1) & obs).sum(dim=0)
    fp = ((pred == 1) & (Y == 0) & obs).sum(dim=0)
    fn = ((pred == 0) & (Y == 1) & obs).sum(dim=0)
    denom = 2 * tp + fp + fn
    per_label = torch.where(denom > 0,
                            2 * tp / denom.clamp(min=1),
                            torch.zeros_like(denom))
    f1_macro = per_label.mean().item()
    tp_all, fp_all, fn_all = tp.sum(), fp.sum(), fn.sum()
    f1_micro = (2 * tp_all / (2 * tp_all + fp_all + fn_all)).item()
print(f"loss={final_loss.item():.6f}")
print(f"megfigyelt elemekre accuracy: {acc:.6f}")
print(f"f1_micro={f1_micro:.6f} f1_macro={f1_macro:.6f}")
```

Elvárt eredmény. A tanított modell loss-a, a **megfigyelt** ($M = 1$) elemekre számított elemszintű accuracy, a per-címke F1 mikro- és makroátlaga, valamint a predikciós mátrix. A kódot a könyv környezetében futtatva a mért értékek:

```
loss=0.438242
megfigyelt elemekre accuracy: 0.917969
f1_micro=0.915663 f1_macro=0.915855
```

A futás feltételeit a véletlen seed és a `float32` típus rögzíti (Python 3.12.3, torch 2.15.0+cpu, CPU, `torch.set_num_threads(1)`, seed 20261020). A kiírt loss a 20. frissítés utáni modell loss-a, nem az utolsó lépés előtti érték. Referenciaértékek ezzel a seeddel: loss 0,438242, accuracy 0,917969, F1 mikro 0,915663, F1 makro 0,915855. A labor akkor teljes, ha a tanuló maga is futtatja, és a loss tagonkénti kézi számítással 10^{-5} relatív tűrésen belül egyezik.

A metrikák közül a multilabel feladatban az elemszintű accuracy félrevezető lehet ritka címkéknél, ezért a kód per-címke F1-et is számol: minden k oszlopra a megfigyelt elemek TP, FP és FN számaiból precision_k és recall_k, majd ezekből az F1 mikro- és makroátlaga. A **micro** a TP, FP és FN összegein számol globális F1-et, a **macro** a K per-címke F1 egyszerű átlaga; ritka címkékre a kettő eltér.

A küszöb nem a tanult modell része. A BCE a szigmoid logitot tanítja; a 0,5-nél nagyobb szabály az esetszámok és költségek ismerete nélkül választott alapértelmezés. A 23.2. Kalibráció és döntési küszöb alfejezetben bevezetett elv itt is érvényes: a küszöböt a **validációs** halmazon, a megfigyelt elemeket figyelembe vevő metrika szerint választjuk, és a végleges modellre csak a rögzített küszöbbel számítunk tesztmetrikát.

25.2.7 Hierarchikus célok

A **hierarchikus** annotációban a címkék egy irányított gráf vagy fa csúcsai, és konzisztenciakényszer áll fenn: ha egy gyermek címke pozitív, akkor a szülője is pozitív. Formálisan legyen $G = (V, E)$ irányított körmentes gráf (DAG, directed acyclic graph) a címkehalmazon, ahol $|V| = K$, a k-adik címke a k-adik csúcshoz tartozik, és $Y_n \in \{0,1\}^K$ konzisztens, ha minden $(u \to v) \in E$ élre $Y_{n,v} = 1 \Rightarrow Y_{n,u} = 1$. Például „jármű” → „kétkerekű”, és „kétkerekű” → „kerékpár” esetén a „kerékpár” pozitivitása maga után vonja mindkét ős pozitivitását.

A hierarchikus kényszer nem változtat a BCE alakegyezésén; a modell továbbra is K logitot ad. A kényszert két módon kezelhetjük. Az egyik a predikció utáni vetítés: a konzisztenciát úgy érjük el, hogy az ős (szülő) predikcióját a gyermekek maximumára *emeljük*: $\hat{p}_u \leftarrow \max(\hat{p}_u, \max_{v:(u\to v)} \hat{p}_v)$. Ez nem a gyermek értékét csökkenti; az utóbbi (a gyermek levágása az ős értékére) más leképezés, amely szintén konzisztenciát ad. A vetítés bejárási rendet és DAG-feltételt igényel: a csúcsokat topologikus sorrendben, a gyermekektől az ősök felé dolgozzuk fel; ciklust tartalmazó gráfon nincs topologikus sorrend, ezért a vetítés bejárási rendje nem definiálható. A kalibráció itt a predikált valószínűség és a megfigyelt relatív gyakoriság egyezését jelenti (23.2. Kalibráció és döntési küszöb); a konzisztencia önmagában nem igényel előzetes kalibrációt, a vetítés viszont megváltoztathatja a kalibrációt.

Számolt példa. Tegyük fel, hogy egy szülő u predikált valószínűsége $\hat{p}_u = 0{,}55$, két gyermeké pedig $\hat{p}_{v_1} = 0{,}70$ és $\hat{p}_{v_2} = 0{,}62$. A vetítés után $\hat{p}_u = \max(0{,}55;\, 0{,}70;\, 0{,}62) = 0{,}70$, a gyermekek értéke változatlan; így a konzisztencia teljesül, mert a szülő predikciója nem kisebb egyik gyermekénél sem.

A másik lehetőség a cél oldali hierarchikus loss, amely a szülő–gyermek élpárok logitjait is bünteti az inkonzisztenciáért. Egy lehetséges alak:

$$L_{\text{hier}} = \frac{1}{N} \sum_{n=1}^{N} \left[\sum_{k=1}^{K} \text{BCE}(z_{n,k}, Y_{n,k}) + \lambda \sum_{(u\to v)\in E} \max(0,\, z_{n,v} - z_{n,u}) \right],$$

ahol $\lambda \geq 0$ a hierarchikus tag súlya, u a szülő, v a gyermek, és a penalty akkor nulla, ha minden élre $z_{n,u} \geq z_{n,v}$. A $\max(0, \cdot)$ miatt a szülő logitjának a gyermek fölé emelése nem büntetett; a sértett irányt viszont lineárisan bünteti. Ez a tag a logitokon hat, ezért a BCE-vel együtt optimalizálható, és nem igényel utólagos vetítést.

Gyakorlat. A hierarchikus osztályozás fogalmi áttekintése Silla és Freitas (2011). A fenti $u \to v_1$, $u \to v_2$ gráfon legyen $z_u = 0{,}2$, $z_{v_1} = 0{,}8$, $z_{v_2} = 0{,}1$ és $\lambda = 0{,}5$. Számítsa ki a hierarchikus penalty értékét, majd ismételje meg $z_u = 0{,}9$ mellett. Magyarázza meg, melyik esetben lesz a penalty nulla, és miért. Ellenőrzés: az első esetben a penalty $0{,}5 \cdot (0{,}8 - 0{,}2) = 0{,}3$ (a v_2 él nem sértett, mert $z_u > z_{v_2}$), a másodikban mindkét él teljesül, ezért a penalty 0.

A vetítéses szabály **heurisztika, nem általános tétel**: feltételezi, hogy a modell egyébként jól kalibrált, és a címkefa konzisztens annotációt ír le. Nem állítja, hogy a vetített kimenet optimális az alkalmazási metrikára, és nem pótolja a hiányzó szülő címkék valódi tanítását.

25.2.8 Tipikus tévedés: a nem annotált címke nem negatív címke

A leggyakoribb hiba a multilabel tanításban, hogy a hiányzó annotációt $Y_{n,k} = 0$-ként töltjük ki. **Ellenpélda.** Tegyük fel, hogy egy gyűjteményben a „vaku használata” címke csak akkor kap 1-est, ha a fotós feljegyezte; az üres mező azt jelenti, hogy nem tudjuk. Egy éjszakai képen, amelyen valójában vaku volt, a téves negatív cél hatására a BCE gradiens $\sigma(z) - 0 > 0$, tehát a logitot lefelé húzza, és a recall romlik annak ellenére, hogy az adatban nem volt megfigyelt

negatív példa. A maszk ezt az elemet kihagyja a likelihood-összegből: a modell az adott címkére ott nem tanul, mintha az a megfigyelés nem létezne. A nem annotált elem negatívként kezelése tehát nem pusztán pontatlanság, hanem a likelihood-modell hibája, és a tanult döntési küszöböt szisztematikusan torzíthatja. A példa ugyanakkor a maszkolás határát is mutatja: ha a fotósok éppen a vakus képeknél jegyzik fel gyakrabban a címkét, a hiányzás a valódi címkétől függ (MNAR). A maszk ekkor is kiküszöböli a hamis negatív célokat, de a megfigyelt címkék között a pozitívak túlreprezentáltak, így a modell a vaku valószínűségét a populációra felülbecsülheti. Egy egyszerű szimulációban (valódi arány 0,2; a pozitívak 90%-át, a negatívak 30%-át jegyzik fel) a megfigyelt címkék pozitív aránya 0,4286, míg MCAR-hiányzásnál 0,2. Ezt csak a feljegyzési mechanizmus modellezése vagy egy teljesen annotált validációs részminta korrigálhatja.

25.2.9 Önellenőrzés és visszautalás

Ellenőrző kérdések a szakasz után: (1) A Z logitmátrix minden eleme a szigmoid után $[0, 1]$-ben van-e? (2) A maszkolt loss-ban a nevező S vagy $N \cdot K$? (3) A gradienst a hiányzó elemre tényleg nullának definiáljuk-e? (4) A küszöbátlépés a loss vagy az értékelés része?

Rövid válaszok. (1) Igen: a szigmoid elemenként hat, értéke szigorúan $(0, 1)$-ben van, tehát $[0, 1]$-ben is. (2) A nevező $S = \sum_{n,k} M_{n,k}$, azaz a megfigyelt elemek száma, nem $N \cdot K$. (3) Igen: a maszk a hiányzó elemhez nulla gradienst ad. (4) A küszöbátlépés az értékelés része, nem a lossé; a loss a logitokon számolt maszkolt BCE.

Visszautalás a 13.2. Konvolúció és cross-correlation alfejezetre: a cross-correlation és a konvolúció megkülönböztetésénél is az volt a tanulság, hogy a numerikus forma önmagában nem rögzíti a szemantikai szerződést; itt ugyanúgy, a 0/1 célforma önmagában nem mondja meg, hogy a nulla negatív vagy hiányzó annotáció-e. A 25.1. Teljes képosztályozó adatfolyam adatfolyamában a logitok alakja $N \times K$ volt; a multilabel ugyanazt a fejet használja, csak a normalizálás és a loss célalakja változik. A 24.2. CPU/GPU profiling és helyes időmérés alfejezetből pedig azt idézzük fel, hogy a metrika futásidejét a threshold sweep (a küszöbértékek végigpróbálása) során külön kell mérni: a per-címke F1 K külön oszlopon számolódik, a sweep költsége pedig nem elhanyagolható, ha a modellt nagyméretű adaton értékeljük.

25.2.10 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

25.2.10.1 F1. Fogalmi (nehézség 1)

Magyarázza el, miért nem ekvivalens a kétcímkés multilabel feladat a kétosztályos multiclass feladattal. Adjon egy 2 logitból álló konkrét z vektort, amelyre a softmax és a két elemenkénti szigmoid kimenete különböző döntést ad 0,5-ös küszöb mellett.

25.2.10.2 F2. Levezetési (nehézség 2)

Vezesse le részletesen, hogy a $\ell(z, y) = \log(1 + e^z) - yz$ cél deriváltja $\sigma(z) - y$, és számítsa ki a gradienst a $(2, 3)$ elemre a kézi példában. Ellenőrizze a maszkolt elem gradiensének definícióját.

25.2.10.3 F3. Implementációs (nehézség 2)

A fent közölt NumPy függvényt egészítse ki egy `masked_bce_grad` függvénnyel, amely a loss-nak a logitmátrix szerinti gradiensét adja: $\frac{\partial L}{\partial z_{n,k}} = \frac{M_{n,k}}{S}(\sigma(z_{n,k}) - Y_{n,k})$. Futtassa le a kézi példára, és ellenőrizze a PyTorch `autograd` (automatikus differenciálás) gradiensével 10^{-8} abszolút tűrés mellett.

25.2.10.4 F4. Hibakeresési (nehézség 2)

Az alábbi kódrészlet hibás:

```
logits = model(images)                          # (N, K)
probs = torch.softmax(logits, dim=1)
loss = torch.nn.functional.binary_cross_entropy(
    probs, targets.float(), reduction='mean')
```

Melyik két fogalmat keveri a kód? Mi a következménye egy olyan képen, ahol a két címke közül mindkettő pozitív? Javítsa a kódot, és nevezze meg a numerikusan stabil változatot is.

25.2.10.5 F5. Kísérlettervezési/transzfer (nehézség 3)

Tervezzen kísérletet egy olyan adatkészletre, ahol a címkék 20%-a hiányzik, és a hiányzás valószínűsége nagyobb a ritka címkéknél. A feladat ötből három címke együttes előfordulásának detektálása. Adja meg a megfigyelési egységet, a splitet, a baseline-t, a választott metrikát, a küszöbválasztás protokollját és a következtetés korlátját. Magyarázza el, miért nem használható a teszt halmaz a küszöb keresésére.

25.2.11 Megoldások és indoklás

25.2.11.1 F1 megoldás

A kétosztályos multiclass a softmax normalizálással $(\hat{p}, 1 - \hat{p})$ párt ad, tehát a címkék egymást kizárják. A multilabel két szigmoidja két független $[0, 1]$-beli érték, amelyek összege nincs 1-hez kötve. Legyen $z = (0{,}7;\ 0{,}7)$. A softmax $(\frac{e^{0,7}}{2e^{0,7}}, \frac{e^{0,7}}{2e^{0,7}}) = (0{,}5;\ 0{,}5)$, tehát 0,5-ös küszöb mellett szigorú $>$ szabállyal egyik címkét sem választjuk; a két egyenlő valószínűség miatt a döntés önkényes (döntetlen). A két szigmoid $\sigma(0{,}7) \approx 0{,}668$ mindkét címkére, tehát mindkettőt jelen levőnek predikáljuk. A softmax elveszi egy második pozitív címke lehetőségét.

25.2.11.2 F2 megoldás

A softplus$(z) = \log(1 + e^z)$ deriváltja a láncszabállyal $\frac{e^z}{1+e^z} = \sigma(z)$. Ezért

$$\frac{\partial}{\partial z}\left[\log(1 + e^z) - yz\right] = \sigma(z) - y.$$

A $(2, 3)$ elemre $z_{2,3} = 1{,}1$, $y_{2,3} = 0$, így a gradiens $\sigma(1{,}1) \approx 0{,}7503$, ami a fenti tévedés-elemzésben használt pozitív húzás irányát mutatja a negatív cél felé. A maszkolt gradiens definíciója az $M_{n,k}$ szorzó: a hiányzó $(2, 2)$ elemnél $M = 0$, ezért a gradiens komponense nulla. Összehasonlításul: a valódi $Y_{2,2} = 1$ cél maszkolatlanul $\sigma(-0{,}4) - 1 \approx -0{,}5987$ gradienst adna, ami a logitot felfelé húzná; a téves negatív cél $(Y'_{2,2} = 0)$ ezzel szemben $\sigma(-0{,}4) - 0 \approx +0{,}4013$ gradienst ad, ami lefelé húzza a logitot, és rontja a recallt. A maszk mindkét hibás hatást kizárja.

25.2.11.3 F3 megoldás

```
def masked_bce_grad(z, y, mask):
    # z, y, mask alakja (N, K)
    z = np.asarray(z, dtype=np.float64)
    y = np.asarray(y, dtype=np.float64)
    mask = np.asarray(mask, dtype=np.float64)
    s = mask.sum()
    if s <= 0:
        raise ValueError("Üres maszk.")
    return mask * (stable_sigmoid(z) - y) / s
```

A PyTorch-ellenőrzés:

```
import numpy as np
import torch

zt = torch.tensor([[-1.5, 0.8, 2.2], [0.2, -0.4, 1.1]],
                  dtype=torch.float64, requires_grad=True)
yt = torch.tensor([[1.0, 0.0, 1.0], [1.0, 1.0, 0.0]], dtype=torch.float64)
mt = torch.tensor([[1.0, 1.0, 1.0], [1.0, 0.0, 1.0]], dtype=torch.float64)
terms = torch.nn.functional.binary_cross_entropy_with_logits(
    zt, yt, reduction='none')
loss = (mt * terms).sum() / mt.sum()
loss.backward()
print(np.max(np.abs(zt.grad.numpy() -
      masked_bce_grad(zt.detach().numpy(), yt.numpy(), mt.numpy()))))
# mért: 2.7755575615628914e-17; a numerikus tűrés 1e-8 alatt marad
```

Alakellenőrzés: a gradiens mátrix ugyanakkora, mint Z, tehát $N \times K$; a maszk nélküli formula $N \times K$, a maszkkal szorzás nem változtatja az alakot. Szélsőeset: ha $Y = 1$ és $z \to \infty$, a gradiens 0-hoz tart; ha $Y = 0$ és $z \to -\infty$, a gradiens 0-hoz tart; a hiányzó maszk elem tartósan nulla.

25.2.11.4 F4 megoldás

A kód a softmaxot és a bináris cross-entropy-t keveri. A `softmax` a K logitot egyetlen eloszlássá normalizálja, ezért a két pozitív címkés képen a cél $(1, 1)$ nem lehet egyszerre teljesíteni: a softmax probabilitások összege 1, a BCE a szigmoid kimenetet várja, és a log-likelihood interpretáció sérül. Ha a két logit egyenlő, a softmax két 0,5-öt ad; a két BCE-tag összege $2 \log 2$, mean redukcióval viszont a loss $\log 2 \approx 0{,}693$ marad, noha mindkét címke pozitív. A javítás:

```
loss = torch.nn.BCEWithLogitsLoss(reduction='mean')(
    logits, targets.float())
# vagy:
# loss = torch.nn.functional.binary_cross_entropy_with_logits(
#     logits, targets.float(), reduction='mean')
```

A numerikusan stabil változat a `BCEWithLogitsLoss`, illetve a `binary_cross_entropy_with_logits`: a logitokból közvetlenül a $\mathrm{softplus}(z) - yz$ alakot értékeli, a softplus stabil, ágazó képletével, ezért nem igényel külön szigmoidot, és nem képződik $\log 0$. Maszkolt esetben a `reduction='none'` tagokat a maszkkal szorozzuk, és a maszk összegével osztunk.

25.2.11.5 F5 megoldás

Egy lehetséges terv: a megfigyelési egység alapértelmezésben a kép, de a GlyphLab-szerződés szerint minden jelenethez két nézet tartozik, amelyek ugyanahhoz a `scene_id`-hez (a `metadata.csv` jelenetazonosítója) és ugyanahhoz a címkéhez tartoznak; ezért a független megfigyelési egység a `scene_id`, és a split is jelenetszintű. A feladat nem a GlyphLab, ezért az átvitelnél ugyanezt a csoportosítást meg kell őrizni. Az adatot train/val/test arányban 60/20/20-ra osztjuk. Multilabel adatnál a rétegzés nem egyszerű osztályarányos split: mivel egy mintához több címke tartozik, iteratív rétegzést (iterative stratification) használunk, amely a ritka címkék előfordulását és a három címke együttes eseményét is arányosan osztja el; a splitet a jelenet szintjén végezzük, hogy egy jelenet nézetei ne kerüljenek különböző oldalra. Baseline: az empirikus prior, azaz a train halmazon megfigyelt relatív gyakoriság (nem simított, nem becsült eloszlás); az összetett detektálásra a három címke együttes előfordulását jelző származtatott bináris cél pozitív arányát használjuk. A modell egy CNN-backbone K logit fejjel, a loss maszkolt BCE. A választás metrikája az együttes esemény F1-je a validáción, mert a feladat a három címke együttes detektálása, nem az egyes címkék átlagos teljesítménye; a per-címke F1 makroátlaga csak másodlagos diagnosztika, és ritka címkékre az elemszintű accuracy félrevezető. A küszöböt a validációs halmazon szabályozzuk globálisan és per-címkeként is; a teszt halmazra csak a kiválasztott küszöbbel számolunk, különben a teszt maga is adaptív modellválasztási ciklusba kerül, és az elvárt generalizációs hibánál optimistább becslést ad. A következtetés korlátja: a hiányzás valószínűsége a ritka címkéknél nagyobb, ezért a mechanizmus nem MAR, hanem MNAR: a megfigyelés valószínűsége a címke értékétől (ritkaságától) is függ, így a maszkolt loss sem ad torzítatlan becslést a teljes populációra, és a hiányzó maszk feltételes függetlenségét a kísérlet nem tudja ellenőrizni. Az eredmény ezért csak a megfigyelt címkékre érvényes; a korrekcióhoz érzékenységvizsgálat vagy teljesen annotált validációs részminta kell.

Hipotézis: a CNN jobb együttes-esemény F1-et ad, mint az empirikus prior baseline; a változó tényező a modell a priorral szemben. **Recept, seedek, döntési küszöb és erőforrás:** a modell egy `SmallCNN` gerinc $K = 4$ logit fejjel, AdamW optimalizálóval, `lr=10^{-3}`, 20 epoch és 16-os batch mellett; az öt seed értéke 20261020–20261024. A választás metrikája az együttes esemény F1-je a validáción; a döntési szabály számszerű: a teszt együttes-esemény F1-jének jelenetszintű páros bootstrap 95%-os CI-je zárja ki a 0-t, és a pontbecslés legalább 2 százalékponttal jobb a prior-baseline-nál. Erőforrás: CPU, `float32`, öt futás, néhány perc. ### Rövid összefoglaló

A multilabel feladat célja $Y \in \{0,1\}^{N \times K}$ mátrix, a modell $Z \in \mathbb{R}^{N \times K}$ logitokat ad, és minden címke külön Bernoulli-döntés. A szigmoid elemenkénti, a softmax ezzel szemben egyetlen összegű normalizálást végez; a kétosztályos softmax egyetlen szigmoiddá fajul, de két multilabel szigmoid nem ekvivalens vele. A BCE logitokkal az $\ell(z, y) = \mathrm{softplus}(z) - yz$ tagot minimalizálja, gradiense $\sigma(z) - y$. A megfigyelési maszk a hiányzó címkéket kihagyja a likelihood-összegből; az üres maszk definiálatlan. A küszöböt validáción választjuk, nem a BCE része. A hierarchia apa–gyermek konzisztenciát követel, a vetítés heurisztika. A legfontosabb hiba a nem annotált címke negatívként kezelése, amely hamis gradienssel torzítja a modellt. A nyitó kérdésre a válasz:

két független bináris címkéhez két kimeneti logit kell, és ezek összege nem kötelezően 1, mert a címkék nem kölcsönösen kizáróak.

25.2.12 Következő lecke

A 25.3. Kis adat és feature-alapú baseline egységben a kis adat és a feature-alapú baseline kérdése következik. A multilabel maszk ott is szerepet kap: ha kevés annotált példánk van, és a hiányzó címkéket nullázzuk, a baseline a leggyakoribb téves negatív célokat tanulja meg, ezért a feature-alapú modell és a CNN összehasonlítása torz lenne; a maszkot a baseline-nál is explicit módon kell kezelni.

25.2.13 Felhasznált források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- PyTorch contributors (2026). *BCEWithLogitsLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BCEWithLogitsLoss.html. Hozzáférés: 2026-10-30.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581–592.
- Silla, C. N., Jr. és Freitas, A. A. (2011). A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 22(1–2), 31–72. Fogalmi áttekintés; a fejezet nem közöl tőle származó mérést.

25.3 Kis adat és feature-alapú baseline

25.3.1 Cél, előismeret és induló becslés

Ez az egység azt a mérnöki kérdést járja körül, hogy egy CNN milyen referenciapontokhoz képest nevezhető jobbnak kis, saját adaton. Előismeret a 25.2. Multilabel és hierarchikus cél alfejezetből: a kölcsönösen kizáró osztályokhoz softmax és int64 osztályindex-cél tartozik, a multilabel BCE ettől különböző feladat. Visszatekintünk a 24.3. Mixed precision és nagyobb batch alfejezetre: f32 CPU-n teljes batch esetén nincs szükség gradiensakkumulációra; a 21.3. Dropout, súlyregularizálás és augmentáció alfejezetre: a regularizálás nem univerzális győzelmi trükk; a 13.3. Lokális szűrők és tanult reprezentáció alfejezetre pedig: a szűrőtanulás lokális feature mapokat hoz létre.

Induló becslés. A függelék A. szakaszában leírt GlyphLab-adat kezdetben kiegyensúlyozott: trainen osztályonként 80, val/testen 20–20 kép van. A többségi osztályt tippelő modell tehát körülbelül 1/3 testpontosságot ér el; a lineáris pixelmodell ennél többet tudhat, ha a három orientáció intenzitásstatisztikája elég eltérő ehhez; ezt nem előzetes tényként rögzítjük, hanem a labor méri. Egy kis CNN előnyét csak akkor érdemes állítani, ha a három egyszerűbb referencia fölé megy, és a mérés a val–test protokollt betartja.

25.3.2 Baseline-ok és a reprezentáció–osztályozó elválasztás

Majority classifier (többségi osztályozó). Legyen a tanító célvektor $y_{tr} \in \{0, \dots, K-1\}^{N_{tr}}$. A tanító osztálygyakoriság

$$\hat{p}_k = \frac{1}{N_{tr}} \sum_{n=1}^{N_{tr}} \mathbf{1}[y_{tr,n} = k], \qquad k \in \{0, \dots, K-1\}.$$

A többségi osztályozó minden bemenetre ugyanazt az osztályt adja:

$$g_{\text{maj}}(x) = \arg\max_k \hat{p}_k.$$

Ez a definíció döntetlen esetén hiányos, ezért rögzítjük a könyv konvencióját: ha több maximális gyakoriságú osztály van, a legkisebb indexet választjuk. A GlyphLab train kiegyensúlyozott, tehát $\hat{p}_k = 1/3$ minden k-ra, és a döntetlen-szabály a 0 osztályt adja. A test is kiegyensúlyozott (osztályonként 20 kép), ezért a majority tesztpontossága pontosan $20/60 = 1/3$: ez nem mintavételi becslés, hanem a split felépítéséből adódó determinisztikus arány, ezért nincs mögötte becsült szórás. A modellnek nulla tanulható paramétere van; ez benchmark-padló, nem előrejelző rendszer.

Linear pixel baseline. A modell korlátja, hogy a súlyok fix pixelpozíciókhoz kötődnek. Egy globális, minden pixelre azonos súlyú összeg (például az összintenzitás) eltolásinvariáns lehet, mert a teljes képet összegzi; ez azonban nem lokális mintázatfelismerés, és a tanult súlyvektor nem használható újra egy eltolt helyi minta ugyanazon kernelével. A modellnek nincs súlymegosztása és nemlinearitása, ezért egy lokális mintázat eltolt példányait külön paraméterekkel kell megtanulnia. Az 1×1-konvolúciós ekvivalenciához a bemenetet explicit $(N, 576, 1, 1)$ alakra kell átrendezni. A bemeneti kép $X \in \mathbb{R}^{N\times1\times24\times24}$, intenzitás $[0, 1]$, float32, NCHW (batch, csatorna, magasság, szélesség). A pixelmodell először ellapítja a képet $d = 1 \cdot 24 \cdot 24 = 576$ elemű vektorrá, majd egy affin réteget alkalmaz:

$$Z = X_{\text{flat}} A^\top + b, \qquad X_{\text{flat}} \in \mathbb{R}^{N\times576}, \quad A \in \mathbb{R}^{K\times576}, \quad b \in \mathbb{R}^K.$$

A tanításhoz [D07] szerint logits bemenetű cross-entropy-t használunk, int64 céllal, softmax nélkül a modell végén. Ez ekvivalens egy 1×1 magméretű, 576 bemeneti és K kimeneti csatornás konvolúciós réteggel: nincs lokális recepciós mező, csak csatornánkénti lineáris pontozás. A paraméterszám $576K + K = 576 \cdot 3 + 3 = 1731$. A modell a pixelvektor terében minden lineárisan szeparálható osztályt meg tud tanulni, de az eltolást nem tudja reprezentálni: a súly minden pixelpozícióhoz külön tanul, ezért ugyanaz a lokális mintázat eltolt helyen új paramétereket igényel.

Frozen features (rögzített jellemzők). Rögzítsünk egy $f_W : \mathbb{R}^{1\times24\times24} \to \mathbb{R}^{d_z}$ feature-extractort, például egy CNN konvolúciós törzsét; itt W a törzs paramétereit jelöli. A kimenet $z = f_W(x) \in \mathbb{R}^{d_z}$. A tanulható fej ekkor $s = Vz + b$, ahol $V \in \mathbb{R}^{K\times d_z}$, $b \in \mathbb{R}^K$. A „frozen” (rögzített) azt jelenti, hogy W-t a tanítás során konstansnak tekintjük: a fej paramétereit frissítjük, a törzs paramétereit nem. Ez a transzfertanulás egyik szélső ága: külső vagy véletlen súlyú törzsből kinyert jellemzőkre lineáris osztályozót tanítunk. A másik ág a finomhangolás, ahol W-t és V-t együtt frissítjük, általában kisebb tanulási rátával.

A reprezentáció–osztályozó elválasztó összehasonlítás. A következő lemma pontosan megadja, mely gradiens-tagok különböztetik meg a fej tanulását a reprezentáció tanulásától.

Lemma (fej–törzs gradiens egy batch-lépésben). Legyen $z_n = f_W(x_n) \in \mathbb{R}^{d_z}$, $s_n = Vz_n + b \in \mathbb{R}^K$ és $\ell_n = \ell(s_n, y_n)$ differenciálható veszteség. Legyen $\ell = \frac{1}{N}\sum_n \ell_n$, és jelölje $g_n = \nabla_{s_n}\ell_n \in \mathbb{R}^K$. Ha $W \mapsto f_W(x)$ a könyvtár gradienskonvenciójával differenciálható (a ReLU töréspontokon a 9.4. Aktivációk és gradiensáramlás alfejezetben rögzített konvencióval), akkor

$$\nabla_V \ell = \frac{1}{N}\sum_{n=1}^{N} g_n z_n^\top \in \mathbb{R}^{K\times d_z}, \qquad \nabla_b \ell = \frac{1}{N}\sum_{n=1}^{N} g_n \in \mathbb{R}^K,$$

és a 4.3. Jacobi-mátrix, láncszabály és VJP Jacobi-oszlopkonvencióval (VJP: vektor–Jacobian szorzat), $J_{f_W}(x_n) \in \mathbb{R}^{d_z\times \dim(W)}$ mellett

$$\nabla_W \ell = \frac{1}{N}\sum_{n=1}^{N} J_{f_W}(x_n)^\top V^\top g_n.$$

Levezetés. A logitok $s_{n,i} = \sum_{r=1}^{d_z} V_{ir} z_{n,r} + b_i$. Rögzített z_n mellett $\partial s_{n,i}/\partial V_{jr} = \delta_{ij} z_{n,r}$, ezért a láncszabályból

$$\frac{\partial \ell_n}{\partial V_{jr}} = \sum_{i=1}^{K} \frac{\partial \ell_n}{\partial s_{n,i}} \frac{\partial s_{n,i}}{\partial V_{jr}} = \sum_i g_{n,i}\delta_{ij} z_{n,r} = g_{n,j} z_{n,r},$$

tehát $\nabla_V \ell_n = g_n z_n^\top$. A biasra $\partial s_{n,i}/\partial b_j = \delta_{ij}$, így $\nabla_b \ell_n = g_n$. A W-függés csak $z_n = f_W(x_n)$-en át lép be; a z-gradiens a lineáris fej VJP-je, $\nabla_z \ell_n = V^\top g_n$, majd a 4.3. Jacobi-mátrix, láncszabály és VJP láncszabály adja a $J_{f_W}(x_n)^\top V^\top g_n$ alakot. Az átlag linearitása hozza a batch-átlagokat. □

A fagyasztott ágban $\nabla_W \ell$ nem kerül az optimalizáló frissítési szabályába, ezért a W pályája konstans W_0; a tanuló ágban ugyanez a tag elmozdítja W-t. Ha a két ág azonos W_0 törzzsel, azonos kezdeti fejjel, azonos adatsorrenddel és azonos fej-optimalizáló beállítással indul, akkor a predikciók különbsége a W elmozdulásához köthető. Ez kísérleti kontroll, nem kauzális bizonyíték: azt méri, hogy az adott recept alatt a reprezentáció megváltoztatása javít-e a validációs metrikán.

Kis adat és transzfer. A GlyphLab train 240 kép, de csak 120 független `scene_id`; a megfigyelési egység a jelenet, nem a kép. A test 60 képén egy képnyi tévesztés $1/60 \approx 0{,}0167$ pontosságkülönbség, ezért a négy modell közti kis eltérést scene-szintű páros összehasonlítással kell értelmezni 23.3. Páros összehasonlítás és csoportszintű intervallum. Külső előtanított súlyt itt nem használunk a laborban; a transzfer fogalmilag az f_{W_0} törzs külső súlyból való indulását jelenti, és a fagyasztott–finomhangolt két ág ugyanazt a protokollt követi. A [D13] modellkatalógus konkrét súlyverzióját, preprocessingjét és licencét külön kell ellenőrizni, mielőtt transzfer-eredményt közlünk.

25.3.3 Kézi gradienspélda és minimális ellenőrzés

Legyen egyetlen példa, $z = (3, -1)^\top$, $V = 0 \in \mathbb{R}^{2\times 2}$, $b = 0 \in \mathbb{R}^2$, cél $y = 0$. Ekkor a logit $s = (0, 0)$, a softmax $p = (1/2, 1/2)$, és a cross-entropy s-gradiense $g = p - e_0 = (-1/2, 1/2)$. A lemma szerint

$$\nabla_V \ell = g z^\top = \begin{pmatrix} -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \begin{pmatrix} 3 & -1 \end{pmatrix} = \begin{pmatrix} -\frac{3}{2} & \frac{1}{2} \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}, \qquad \nabla_b \ell = \begin{pmatrix} -\frac{1}{2} \\ \frac{1}{2} \end{pmatrix}.$$

A fagyasztott törzsnél $\nabla_W \ell$ nincs frissítési tagként, mert z bemeneti konstansként viselkedik a fej számára; ez nem azt jelenti, hogy a derivált matematikailag nem létezik, csak azt, hogy nem frissítjük. Ha z-t egy $f_W(x) = Wx$ törzs állítaná elő ($W \in \mathbb{R}^{d_z \times d_x}$, $x \in \mathbb{R}^{d_x}$), a tanuló ágban $\nabla_W \ell = (V^\top g)\, x^\top \in \mathbb{R}^{d_z \times d_x}$ lépne fel. A teljes paraméter-Jacobi nem $x^\top$: $\partial z / \partial \mathrm{vec}(W) = x^\top \otimes I_{d_z}$ (Kronecker-szorzat, azaz blokkokból álló mátrix).

Minimális ellenőrző kód (# munkakönyvtár: `lab/`, a kimenet mért futásból származik):

```
import torch
import torch.nn.functional as F

z = torch.tensor([[3.0, -1.0]])
V = torch.zeros(2, 2, requires_grad=True)
b = torch.zeros(2, requires_grad=True)

s = z @ V.T + b          # logits: (1,2)
loss = F.cross_entropy(s, torch.tensor([0]))
loss.backward()

print(V.grad)
# mért: tensor([[-1.5000,  0.5000],
#               [ 1.5000, -0.5000]])
print(b.grad)
# mért: tensor([-0.5000,  0.5000])
```

25.3.4 Labor: majority, lineáris pixel, fagyasztott és tanuló CNN

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. A futtatás a szakaszban megadott `baseline_compare.py` szkripttel történik; várható futásidő CPU-n néhány perc, a tényleges időt a futás naplózza.

Protokoll a futás előtt. Kérdés: a tanuló CNN jobb validációs/testpontosságot ér-e el a GlyphLab orientációfeladaton, mint a majority, a lineáris pixel és a fagyasztott véletlen törzsű lineáris fej? Hipotézis: a lineáris pixel a geometriai intenzitáskülönbségek miatt a majority fölé kerül; a fagyasztott véletlen törzs rossz reprezentációt ad, ezért a fej önmagában keveset tanul; a tanuló CNN a W elmozdításával ezt felülmúlja. Adat: a GlyphLab `data/images.npy`, `data/labels.npy`, `data/metadata.csv` (a `scene_id`, `view` és `split` oszlopokkal); a maszk és doboz nem modellbemenet. Megfigyelési egység: `scene_id`. A pontosság pontbecslése (egyetlen szám, intervallum nélkül) mellett a tanuló CNN és a lineáris pixel különbségére jelenetszintű páros bootstrap ad 95%-os intervallumot a 23.3. Páros összehasonlítás és csoportszintű intervallum protokollja szerint; a döntési szabály: a különbség 5%-os szinten szignifikáns, ha az intervallum nem tartalmazza a nullát. Változó tényező: reprezentációs kapacitás. Metrika: pontosság (accuracy); a legjobb checkpoint kiválasztása kizárólag a val spliten történik, a testet utána egyszer értékeljük. Keret: CPU, float32, seed `20261020`, 32-es mini-batch, Adam `lr=1e-2`, 30 epoch (240 optimalizálólépés). Egy seed mellett a seedek közti szórás nem becsülhető; a bootstrap csak a jelenetmintavétel bizonytalanságát tükrözi, ezért a labor módszertani demonstráció, nem végleges modellértékelés.

A GlyphLab adatszerződése a függelék A. A GlyphLab adatkészlet szakaszában van: 180 jelenet (`scene_id`, például `G0-0000`), jelenetenként két nézet (`view_id` 0 és 1), 120/30/30 jelenet train/val/test bontásban, osztályonként rétegzetten, jelenetszinten rögzített splittel, ezért a képszám 240/60/60. A `metadata.csv` kilenc oszlopa `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax`; a `split` értéke `train`, `val` vagy `test`, és a sorokat nem a CSV sorrendje, hanem az `index` mező köti a képtömbhöz. Az adat újragenerálása a `lab/` könyvtárból `python generate_data.py --output data`, majd `python quickcheck.py`;

a labor a fenti, rögzített splitet használja, és a tesztet csak a modellválasztás befagyasztása után értékeli.

A labor CNN-je az ebben az egységben definiált `SmallCNN`: Conv2d(1,8,3,padding=1), ReLU, MaxPool2d(2), Conv2d(8,16,3,padding=1), ReLU, MaxPool2d(2), Flatten, Linear(576,3). Paraméterszám 2979; a lineáris pixelmodellé 1731. A fagyasztott ág a CNN törzsét rögzíti, és csak a $576 \to 3$ fejet tanítja.

```
# Munkakönyvtár: lab/
# Futás: cd lab && python baseline_compare.py
import copy
import csv
import json

import numpy as np
import torch
import torch.nn as nn

SEED = 20261020
DEVICE = torch.device("cpu")
torch.set_num_threads(1)


def load_glyphlab(base="data"):
    X = np.load(f"{base}/images.npy")          # (360,1,24,24), float32
    y = np.load(f"{base}/labels.npy")          # (360,), int64
    with open(f"{base}/metadata.csv", newline="") as f:
        rows = list(csv.DictReader(f))
    split = np.array([r["split"] for r in rows])
    scene = np.array([r["scene_id"] for r in rows])
    idx = {k: split == k for k in ("train", "val", "test")}
    assert X.shape == (360, 1, 24, 24) and X.dtype == np.float32
    assert np.all((X >= 0.0) & (X <= 1.0))
    assert y.dtype == np.int64
    assert len(scene) == len(X) == len(y)
    return X, y, idx, scene


def majority_accuracy(y_tr, y_te):
    counts = np.bincount(y_tr)
    pred = int(np.flatnonzero(counts == counts.max())[0])  # legkisebb index donor
    return float((y_te == pred).mean())


class LinearPixel(nn.Module):
    def __init__(self, d=24 * 24, K=3):
        super().__init__()
        self.fc = nn.Linear(d, K)

    def forward(self, x):
        return self.fc(x.flatten(1))


class SmallCNN(nn.Module):
    def __init__(self, K=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.Flatten(), nn.Linear(16 * 6 * 6, K))

    def forward(self, x):
        return self.head(self.features(x))
```

```
def train_model(model, x_tr, y_tr, x_va, y_va, x_te, y_te, epochs=30, lr=1e-2,
                batch_size=32, seed=SEED):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    lossf = nn.CrossEntropyLoss()
    gen = torch.Generator().manual_seed(seed)
    best_va, best_state = -1.0, None
    for _ in range(epochs):
        model.train()
        order = torch.randperm(x_tr.shape[0], generator=gen)
        for start in range(0, x_tr.shape[0], batch_size):
            b = order[start:start + batch_size]
            opt.zero_grad()
            loss = lossf(model(x_tr[b]), y_tr[b])
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            va = (model(x_va).argmax(1) == y_va).float().mean().item()
        if va > best_va:
            best_va, best_state = va, copy.deepcopy(model.state_dict())
    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        te = (model(x_te).argmax(1) == y_te).float().mean().item()
    return best_va, te


torch.manual_seed(SEED)
X, y, idx, scene = load_glyphlab()
TX = torch.as_tensor(X, dtype=torch.float32, device=DEVICE)
TY = torch.as_tensor(y, dtype=torch.int64, device=DEVICE)
trn, va, te = idx["train"], idx["val"], idx["test"]

maj_va = majority_accuracy(y[trn], y[va])
maj_te = majority_accuracy(y[trn], y[te])
assert abs(maj_te - 1.0 / 3.0) < 1e-6

# Lineáris pixelmodell.
torch.manual_seed(SEED)
lin = LinearPixel().to(DEVICE)
lin_va, lin_te = train_model(lin, TX[trn], TY[trn], TX[va], TY[va], TX[te], TY[te])

# Tanuló CNN és a hozzá tartozó W0 törzs, valamint az azonos fej.
torch.manual_seed(SEED)
cnn = SmallCNN().to(DEVICE)
w0 = copy.deepcopy(cnn.features.state_dict())
head0 = copy.deepcopy(cnn.head[1].state_dict())

# Fagyasztott törzs: a W0 súlyok külön modulba töltve, gradienst nem kapnak.
frozen_backbone = SmallCNN().features.to(DEVICE)
frozen_backbone.load_state_dict(w0)
for p in frozen_backbone.parameters():
    p.requires_grad_(False)
head = nn.Linear(576, 3).to(DEVICE)
head.load_state_dict(copy.deepcopy(head0))

# Kezdeti predikció-egyezés: tanítás előtt a két ág logitja azonos.
cnn.eval()
frozen_backbone.eval()
with torch.no_grad():
    full0 = cnn(TX[te])
```

```
    frozen0 = head(frozen_backbone(TX[te]).flatten(1))
assert torch.allclose(full0, frozen0, atol=1e-6)

cnn_va, cnn_te = train_model(cnn, TX[trn], TY[trn], TX[va], TY[va], TX[te], TY[te])

# A fagyasztott ág a W0 törzs jellemzőin tanul; a törzs változatlan marad.
with torch.no_grad():
    Ztr = frozen_backbone(TX[trn]).flatten(1)
    Zva = frozen_backbone(TX[va]).flatten(1)
    Zte = frozen_backbone(TX[te]).flatten(1)
w_before = copy.deepcopy(frozen_backbone.state_dict())
frozen_va, frozen_te = train_model(head, Ztr, TY[trn], Zva, TY[va], Zte, TY[te])
assert all(torch.equal(w_before[k], frozen_backbone.state_dict()[k]) for k in w_before)


def paired_scene_bootstrap(correct_a, correct_b, scenes, n_boot=2000, seed=SEED):
    """Jelenetszintű páros bootstrap a két modell pontosságkülönbségére."""
    rng = np.random.default_rng(seed)
    uniq = np.unique(scenes)
    diff = correct_a.astype(np.float64) - correct_b.astype(np.float64)
    per_scene = np.array([diff[scenes == s].mean() for s in uniq])
    boots = np.empty(n_boot)
    for b in range(n_boot):
        pick = rng.integers(0, len(uniq), len(uniq))
        boots[b] = per_scene[pick].mean()
    lo, hi = np.percentile(boots, [2.5, 97.5])
    return float(per_scene.mean()), float(lo), float(hi)


with torch.no_grad():
    corr_lin = (lin(TX[te]).argmax(1) == TY[te]).cpu().numpy()
    corr_cnn = (cnn(TX[te]).argmax(1) == TY[te]).cpu().numpy()
d_mean, d_lo, d_hi = paired_scene_bootstrap(corr_cnn, corr_lin, scene[te])

result = {
    "majority_val_acc": maj_va,
    "majority_test_acc": maj_te,
    "linear_pixel_val_acc": lin_va,
    "linear_pixel_test_acc": lin_te,
    "frozen_cnn_val_acc": frozen_va,
    "frozen_cnn_test_acc": frozen_te,
    "trainable_cnn_val_acc": cnn_va,
    "trainable_cnn_test_acc": cnn_te,
    "trainable_minus_linear_mean": d_mean,
    "trainable_minus_linear_ci95": [d_lo, d_hi],
    "cnn_params": sum(p.numel() for p in cnn.parameters()),
    "linear_params": sum(p.numel() for p in lin.parameters()),
}
print(json.dumps(result, indent=2))
```

MÉRT ÉRTÉK. A `baseline_compare.py`-t a könyv környezetében futtattuk (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020, 30 epoch, egy futás); a mért JSON:

```
{
  "majority_val_acc": 0.3333333333333333,
  "majority_test_acc": 0.3333333333333333,
  "linear_pixel_val_acc": 0.46666666865348816,
  "linear_pixel_test_acc": 0.5166666507720947,
  "frozen_cnn_val_acc": 0.6000000238418579,
  "frozen_cnn_test_acc": 0.7333333492279053,
  "trainable_cnn_val_acc": 0.8999999761581421,
  "trainable_cnn_test_acc": 1.0,
```

```
  "trainable_minus_linear_mean": 0.48333333333333334,
  "trainable_minus_linear_ci95": [0.31666666666666665, 0.65],
  "cnn_params": 2979,
  "linear_params": 1731
}
```

A majority tesztpontossága pontosan 1/3; a lineáris pixel, a fagyasztott fej és a tanuló CNN értéke egy futás eredménye, ezért a számok a seed és a 60 képes teszt mellett érvényesek. A tanuló CNN és a lineáris pixel különbségének 95%-os jelenetszintű bootstrap-intervalluma [0,3167; 0,65], amely nem tartalmazza a nullát.

Eredményértelmezés. A majority padlója 1/3. A lineáris pixelmodell és a CNN közti különbséget a fagyasztott ág teszi értelmezhetővé: ha a fej a rögzített véletlen W_0 jellemzőkből alig tanul, a tanuló CNN többlete a reprezentáció elmozdulásából származik. Ha a lineáris pixelmodell a CNN közelében vagy fölötte van, az nem hiba: a GlyphLab feladat lineáris intenzitásjele erős, és ebből a szintetikus eredményből nem következtetünk természeteskép-teljesítményre.

25.3.5 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Mi a majority classifier döntetlen-konvenciója ebben a könyvben, és miért pontosan 1/3 a GlyphLab testpontossága? Adjunk olyan train–test címkepárt, ahol a train-majority a testen 0 pontosságú. Beadandó: a döntetlen-szabály, a kiegyensúlyozottsági indoklás és egy konkrét ellenpélda.

F2 (levezetési, nehézség 2). Vezesse le a lemma alkalmazásával $\nabla_V \ell$-t és $\nabla_b \ell$-t egyetlen példára, ha $z = (2, 4)^\top$, $V = 0$, $b = 0$, $K = 2$, $y = 1$. Melyik tag marad el a fagyasztott törzs frissítéséből, és miért? Beadandó: a softmax-gradiens, a két mátrix/vektor és egy alakellenőrzés.

F3 (implementációs, nehézség 1). Írjon futtatható NumPy-függvényt `majority_accuracy(y_tr, y_te)` néven, amely a legkisebb indexű többségi osztályt választja döntetlennél, és a testpontosságot adja. Tesztelje a `([0,0,1,1],[1,1,0,0])` és `([0,0,0,1],[0,0,1,0])` eseteken. Beadandó: kód, elvárt értékek, futási ellenőrzés.

F4 (hibakeresési, nehézség 2). Adott a szándék szerint fagyasztott törzsű fej-tanítás (`loader` a tanító adat 32-es mini-batch DataLoaderje, `backbone` a labor `SmallCNN` törzse):

```
backbone = SmallCNN().features
head = nn.Linear(576, 3)
opt = torch.optim.Adam(list(backbone.parameters()) + list(head.parameters()), lr=1e-2)
for xb, yb in loader:
    opt.zero_grad()
    loss = F.cross_entropy(head(backbone(xb).flatten(1)), yb)
    loss.backward()
    opt.step()
```

Mi a hiba, mi a kísérleti következménye, és hogyan javítandó? Beadandó: ok, következmény, javított kód és egy `assert`, amely a törzs változatlanságát ellenőrzi.

F5 (kísérleti transzfer, nehézség 3). Tervezzen transzfer-kísérletet egy kisméretű természetes képi osztályozásra, külső előtanított törzzsel. A terv kötelezően tartalmazza: a downstream

adat splitjét, a fagyasztott és finomhangolt baseline-t, az azonos preprocessinget, a metrikát, a seed/batch receptet és a következtetési korlátot. A [D13] katalógusból választott konkrét súlyverzióval együtt rögzítse az előfeldolgozást és a licencfeltételeket is. Beadandó: protokollterv pontokban, nem futtatott numerikus eredmény.

25.3.6 Megoldások és indoklás

F1 megoldása. Döntetlennél a legkisebb indexű osztályt választjuk. A GlyphLab trainen mindhárom osztály gyakorisága 80/240, a testen 20/60, ezért a donor osztály 0, és a testbeli 0-címkék aránya $20/60 = 1/3$; itt nincs mintavételi zaj, mert a split-generátor rögzített osztályonkénti jelenetszámot képez. Ellenpélda: `y_tr=[0,0,0,1], y_te=[1,1,1,1]`; a train-majority 0, de a testen minden címke 1, így a pontosság 0.

F2 megoldása. $s = (0, 0)$, $p = (1/2, 1/2)$, cél $y = 1$, ezért $g = p - e_1 = (1/2, -1/2)$. Ebből

$$\nabla_V \ell = g z^\top = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 2 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ -1 & -2 \end{pmatrix}, \qquad \nabla_b \ell = g = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \end{pmatrix}.$$

Alakellenőrzés: $g \in \mathbb{R}^K$, $z^\top \in \mathbb{R}^{1 \times d_z}$, tehát $\nabla_V \ell \in \mathbb{R}^{K \times d_z} = \mathbb{R}^{2 \times 2}$; $\nabla_b \ell \in \mathbb{R}^2$. A fagyasztott ágban $\nabla_W \ell$ nincs a frissítésben, mert z állandó bemenet, és a W-hez vezető gráfélt levágtuk; a tanuló ágban $J_{f_W}(x)^\top V^\top g$ lép fel.

F3 megoldása.

```
import numpy as np

def majority_accuracy(y_tr, y_te):
    counts = np.bincount(y_tr)
    pred = int(np.flatnonzero(counts == counts.max())[0])
    return float((y_te == pred).mean())

print(majority_accuracy(np.array([0, 0, 1, 1]), np.array([1, 1, 0, 0])))
# mért: 0.5, mert döntetlennél a legkisebb index (0) nyer, a testen két 0 van.
print(majority_accuracy(np.array([0, 0, 0, 1]), np.array([0, 0, 1, 0])))
# mért: 0.75, mert a donor 0, a testen három 0 van.
```

A futtatás kimenete `0.5` és `0.75` (Python 3.12.3, NumPy 2.5.4, CPU).

A függvény a döntetlennél a legkisebb indexet adja; a `bincount` a bemenet nemnegatív int64 címkéire értelmes. Szélsőeset: egyetlen osztályt tartalmazó train esetén a pred egyértelmű.

F4 megoldása. Hiba: az optimalizáló a `backbone.parameters()` listát is tartalmazza, tehát a törzs súlyai frissülnek; ez finomhangolás, nem rögzített jellemzők (`frozen features`). Következmény: a reprezentáció–osztályozó összehasonlítás érvénytelenné válik, mert a fej teljesítményéhez reprezentációváltozás is társul. Javítás (a labor betöltött tenzoraira építve; a `TX`, `TY`, `trn`, `va` és `SmallCNN` a labor 7. szakaszának kódjából származik, önálló futtatásnál onnan kell előállítani, a `loader` pedig csak a hibás példában szerepel):

```
import copy
import torch
import torch.nn as nn
import torch.nn.functional as F

Xtr, ytr, Xva, yva = TX[trn], TY[trn], TX[va], TY[va]

for p in backbone.parameters():
    p.requires_grad_(False)
```

```
backbone.eval()
with torch.no_grad():
    Ztr = backbone(Xtr).flatten(1)
    Zva = backbone(Xva).flatten(1)

head = nn.Linear(576, 3)
opt = torch.optim.Adam(head.parameters(), lr=1e-2)
w0 = copy.deepcopy(backbone.state_dict())

for ep in range(5):
    opt.zero_grad()
    loss = F.cross_entropy(head(Ztr), ytr)
    loss.backward()
    opt.step()

with torch.no_grad():
    va_acc = (head(Zva).argmax(1) == yva).float().mean().item()
print(va_acc)

assert all(torch.equal(w0[k], backbone.state_dict()[k]) for k in w0)
```

Az assert azt ellenőrzi, hogy a törzs paraméterei és pufferei a tanítási ciklus után is változatlanok (a `state_dict` a BatchNorm futóstatisztikáit is tartalmazza). A `no_grad` csak az autograd gráfot vágja le a törzs irányába, eval módba nem kapcsol; ezért kell a `backbone.eval()` a jellemzőkinyerés előtt, különben a BatchNorm train módban a batch-statisztikát használná, és a futóstatisztikákat is frissítené.

F5 megoldásminta. Terv: (1) a célfeladat (downstream) adatának rögzített train/val/test csoportos splitje, ahol a független egység a jelenet vagy személy, és a val kizárólagos modellválasztásra szolgál. (2) Baseline A: fagyasztott külső törzs, újonnan inicializált lineáris fej, a törzs súlyai `requires_grad_(False)`. (3) Baseline B: azonos törzs finomhangolva; a törzs tanulási rátája a fej tanulási rátájának a tizede, azonos epoch- és mini-batch-recepttel; a kezdeti predikció-egyezést a két ág indulásakor futási ellenőrzéssel rögzítjük. (4) Az előfeldolgozás (preprocessing) a törzs dokumentált bemeneti sémájával egyezik; eltérés esetén a bemeneti eloszlások különböznek (domain shift), és a transzfer teljesítménye romolhat. (5) Metrika: top-1 pontosság és a scene-szintű páros bootstrap a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint; a testet a val-alapú checkpoint után egyszer értékeljük. (6) Korlát: a kis célfeladat-minta miatt a két ág különbsége nem általános transzfer-törvény; a [D13] katalógus konkrét súlyverzióját, licencét és előfeldolgozását a futtatás előtt ellenőrizni kell.

Hipotézis: a finomhangolt törzs jobb top-1 pontosságot ad, mint a fagyasztott; a változó tényező a törzs taníthatósága, a megfigyelési egység a jelenet vagy személy. **Recept, seedek, döntési küszöb és erőforrás:** a törzs egy előtanított ResNet18, a fej lineáris; a tanítás Adam optimalizálóval, a fejen `lr=10^{-2}`, a finomhangolt törzsön ennek tizede mellett, 30 epoch és 32-es batch mellett fut, három seeddel (20261020–20261022). A döntési szabály számszerű: a finomhangolt és a fagyasztott ág top-1 pontosságának jelenetszintű páros bootstrap 95%-os CI-je zárja ki a 0-t, és a különbség legalább 2 százalékpont; a tesztet a val-alapú checkpoint után egyszer értékeljük. Erőforrás: 1 GPU vagy CPU, `float32`, két ág × három seed = hat futás, néhány perc; a [D13] súly letöltése és licencellenőrzése külön lépés. ### Tipikus tévedés, önellenőrzés, visszatekintés

Tipikus tévedés. „A szintetikus GlyphLab-on a CNN magas pontossága azt mutatja, hogy a modell természetes képeken is jól teljesít.” Ez nem következik. Ellenpélda: legyen a négy bemenet egy-egy 2×2-es kép, amelyben pontosan egy pixel értéke 1, a többi 0; a négy kép a négy saroknak felel meg, a sarok koordinátái $(r, c) \in \{(0,0), (0,1), (1,0), (1,1)\}$, a címke pedig a két koordináta XOR-ja, azaz $y = (0, 1, 1, 0)$. A négy ponton bármely affin függvény legfeljebb

hármat osztályoz helyesen, mert a két pixelintenzitás XOR-ja nem lineárisan szeparálható; a lineáris pixelmodell tehát legfeljebb 3/4 trainpontosságot érhet el, egy megfelelő nemlineáris CNN viszont mind a négyet megtanulhatja. A GlyphLab orientáció lineárisan erős geometriai jelet hordoz, ezért ott a lineáris baseline is magas lehet; egy másik szintetikus vagy természetes feladat nehézsége ettől független.

Önellenőrzés. Rögzítettem-e a majority döntetlen-szabályát? A baseline-ok csak a képtensort használják, a maszkot és dobozt nem? A fagyasztott ág kezdeti logitja numerikusan egyezik a tanuló CNN kezdeti logitjával? A legjobb modellt val, a testet azután választom? A testkülönbséget kép vagy scene-szintű egységgel értelmezem?

Rövid válaszok. (1) A majority döntetlen-szabálya a legkisebb osztályindex. (2) Igen: a baseline-ok csak a képtenzort használják, a maszkot és a dobozt nem. (3) Igen, ha a fagyasztott ág a tanuló CNN W_0 súlyait kapja, és a fej kezdőpontja azonos; ezt a kód ellenőrzi. (4) Igen: a legjobb modellt a validáción választjuk, a tesztet csak utána értékeljük. (5) A testkülönbséget scene-szintű egységgel kell értelmezni, mert a két nézet nem független.

Visszatekintés. A 13.3. Lokális szűrők és tanult reprezentáció alfejezetben használt rögzített szűrő egyetlen, nem tanuló kernel; itt a lineáris pixelmodell ehhez hasonlóan fix pixelpozíciójú lineáris template, de a súlyai tanulhatók, és nincs térbeli súlymegosztása. A 21.3. Dropout, súlyregularizálás és augmentáció szerint a weight decay és dropout heurisztika, ezért a labor összehasonlítása ugyanazt a regularizálásmentes receptet alkalmazza minden ágra. A 24.3. Mixed precision és nagyobb batch szerint f32, teljes batch és CPU mellett nincs szükség GradScalerre vagy akkumulációra; ha később kevert pontosságúra váltunk, a négy ágat azonos numerikus úton kell futtatni.

25.3.7 Rövid összefoglaló

A kis adaton a CNN-t csak azonos spliten futtatott referenciákhoz képest értékeljük. A majority a leggyakoribb tanítóosztályt adja, döntetlennél a legkisebb indexet; a kiegyensúlyozott GlyphLab testen pontossága 1/3. A lineáris pixelmodell 576 bemenetet képez le három logitra, térbeli lokalitás nélkül. A fagyasztott törzsű fej csak a V, b fejgradienst frissíti; a tanuló CNN a J_{f_W}-n át ∇_W-t is. Ugyanazon W_0, fejkezdőpont és recept mellett a két ág közti különbség a reprezentáció elmozdulását méri. A labor protokollt, ellenőrző assertet és futás után ellenőrizhető kulcsértékeket ad (majority 1/3, 2979 és 1731 paraméter, jelenetszintű bootstrap-intervallum); a 25.4. Nyílt halmaz és ismeretlen input egység előtt a transzfer és baseline korlátokat is rögzíti.

25.3.8 Továbblépés és források

A 25.4. Nyílt halmaz és ismeretlen input egység a nyílt halmazú felismerést tárgyalja: a három GlyphLab-osztályra tanított softmax pontszám nem dönthet ismeretlen inputról anélkül, hogy a logit-kalibráció (a logitok valószínűséggé alakítása és a döntési küszöb összehangolása) és a döntési küszöb kérdését újra ne kellene tárgyalni.

Felhasznált források:

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D13] PyTorch contributors (2026). *TorchVision models*. TorchVision dokumentáció (stable). https://docs.pytorch.org/vision/stable/models.html. Hozzáférés: 2026-10-30.

25.4 Nyílt halmaz és ismeretlen input

25.4.1 Cél, előismeret és induló becslés

Eddig minden osztályozó azt feltételezte, hogy a teszt input ugyanabból az eloszlásból származik, mint a tanító adat, és a célhalmaz éppen K osztály. A függelék A. szakaszában leírt GlyphLab esetén ez a három orientáció. A valós felhasználásban azonban a modell olyan képet is kaphat, amely nem a generátor specifikációja szerint készült: üres konstans kép, véletlen zaj, más képtípus. Ekkor a modell még mindig az egyik ismert osztályt fogja mondani, mert a kimeneti rétege csak K logitot ad (a logit a softmax előtti nyers pontszám). Ebben az egységben ezt a **nyílt halmaz** helyzetet kezeljük: megkülönböztetjük az *ismeretlen osztály*, az *out-of-distribution (az eloszláson kívüli, OOD)* és az *elutasítás* fogalmát.

Előismeret: a softmax és a többosztályos cross-entropy a 7.2. Softmax és többosztályos cross-entropy alfejezetből; a kis adaton mért baseline és a tisztességes összehasonlítás (fair comparison) a 25.3. Kis adat és feature-alapú baseline alfejezetből; a loss- és alak-konvenció ellenőrzése a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetből; továbbá a 21.4. Osztályarány, súlyozás és céleloszlás alfejezetben tárgyalt megfigyelés, hogy a softmax-konfidencia nem automatikusan kalibrált valószínűség.

Induló becslés. Mekkora maximális softmax-konfidenciát adhat a 3. szakaszban vizsgált, 2×2-es bemenetre definiált toy modell a csupa egyes képre? A válasz modellfüggő: a kézi példa megmutatja, hogy egy konkrét affin réteggel 96,5% is elérhető értelmetlen inputra. A tanult GlyphLab-modell 24×24-es bemeneten ugyanígy viselkedhet, de a konkrét érték a tanult súlyoktól függ; ezt a szakasz laborja méri (lásd az 5. szakaszt).

25.4.2 Definíciók és intuíció

Legyen a bemeneti tér $\mathcal{X} = \mathbb{R}^{C \times H \times W}$, a tanulási eloszlás p_{train} a $\mathcal{D}_{\text{in}} \subseteq \mathcal{X}$ **felhasználási tartomány** fölött, a zárt célhalmaz pedig $\mathcal{Y} = \{0, \dots, K-1\}$. A GlyphLab esetén p_{train} a generátor által definiált eloszlás a három vonaltípus fölött.

Nyílt halmazú osztályozás (open-set classification): a döntés célhalmaza

$$\mathcal{Y}_{\text{open}} = \{0, \dots, K-1, \text{unknown}\},$$

ahol unknown egy külön, $(K+1)$-edik osztály. A modell kimenete ekkor $K+1$ logit, vagy a K logit mellett egy külön elutasítási mechanizmus.

Out-of-distribution (OOD) input: olyan x, amely olyan q eloszlásból származik, amelyre $q \neq p_{\text{train}}$, vagy amelyre az x a $\mathcal{D}_{\text{in}}$ tartományon kívül esik. (Egyetlen mintának nincs eloszlása; az OOD fogalma a mintát létrehozó eloszlásra vagy a tartománybeliségre vonatkozik.) Az OOD nem azonos az unknown osztállyal: egy OOD input lehet egy ismert osztályú, de más körülmények között készült kép, vagy egyáltalán nem vonatkozó objektum. A modell önmagában, további feltételezés nélkül nem tudja megmondani, hogy bemenete OOD-e.

Softmax-konfidencia: a $Z \in \mathbb{R}^{N \times K}$ logitokból számított

$$p_{ik} = \text{softmax}(Z)_{ik} = \frac{e^{Z_{ik}}}{\sum_{j=1}^{K} e^{Z_{ij}}},$$

illetve a döntési konfidencia

$$c_i = \max_{k \in \mathcal{Y}} p_{ik} \in \left[\frac{1}{K}, 1\right].$$

Itt c_i a maximális softmax-konfidencia, nem osztályindex: az osztályokat k indexeli, a csatornaszámot C jelöli. Ez egy normalizált pontszám, nem feltétlenül az adott osztály tényleges valószínűsége; a 21.4. Osztályarány, súlyozás és céleloszlás kalibrációs tárgyalása szerint ehhez külön kalibrációs adat és metrika kell.

Elutasítás (rejection): a predikciót egy τ konfidencia-küszöbbel egészítjük ki:

$$\hat{y}_i = \begin{cases} \arg\max_k Z_{ik}, & \text{ha } c_i \geq \tau, \\ \text{unknown}, & \text{ha } c_i < \tau. \end{cases}$$

A τ egy döntési paraméter, nem a modell paramétere. Az elfogadási arány (coverage) és az elfogadottakon mért pontosság együtt jellemzi a szabályt.

Monitoring (folyamatos felügyelet): a felhasználás során a konfidencia-eloszlást, az elfogadási arányt és az osztálygyakoriságokat folyamatosan figyeljük. Ha ezek eltérnek a validációs eloszlástól, az domain-eltolódást (a bemeneti eloszlás megváltozását) jelezhet, de önmagában nem bizonyítja azt.

A felhasználási tartomány (domain) tehát a modell érvényességi feltételeinek része: a modell csak a $\mathcal{D}_{\text{in}}$ tartományon, a megadott bemeneti előfeldolgozással és eloszlással tett állításokra érvényes.

25.4.3 A nagy softmax-konfidencia ellenpéldája

A következő szemléltető ellenpélda, ahol az input értelmetlen, mégis nagy a softmax maximuma. Ehhez először kimondunk egy pontos alsó korlátot.

Lemma (margin és softmax-konfidencia). Legyen $z \in \mathbb{R}^K$, $p = \text{softmax}(z)$. Ha valamely y osztályra

$$z_y - \max_{k \neq y} z_k = M$$

tetszőleges valós M mellett (az $M \geq 0$ esetben a korlát legalább $1/K$), akkor

$$p_y \geq \frac{1}{1 + (K-1)e^{-M}}.$$

Bizonyítás. A softmax definíciója szerint

$$p_y = \frac{e^{z_y}}{\sum_{j=1}^{K} e^{z_j}} = \frac{1}{1 + \sum_{j \neq y} e^{z_j - z_y}}.$$

Minden $j \neq y$ esetén $z_j \leq z_y - M$, tehát $e^{z_j - z_y} \leq e^{-M}$. Így

$$\sum_{j \neq y} e^{z_j - z_y} \leq (K-1)e^{-M},$$

ezért a nevező felülről becsülhető, és a reciprok alulról:

$$p_y = \frac{1}{1 + \sum_{j \neq y} e^{z_j - z_y}} \geq \frac{1}{1 + (K-1)e^{-M}}.$$

A korlát éles: ha minden $j \neq y$-ra $z_j = z_y - M$, akkor a nevező pontosan $1 + (K-1)e^{-M}$, tehát p_y pontosan a jobb oldal.

A bizonyítás kizárólag a softmax algebrai tulajdonságán alapul, semmilyen feltételt nem használ x tartománybeliségéről. Ebből következik a célzott ellenpélda.

Ellenpélda. Vegyünk egy 2×2-es, egycsatornás, tehát $x \in \mathbb{R}^4$ bemenetet, amelynek minden pixelértéke 1:

$$x = (1, 1, 1, 1)^\top.$$

A GlyphLab generátor szerint a képek maszkja egy vékony vonalat tartalmaz, a háttér 0; a csupa egyes kép ezért nem eleme a felhasználási tartománynak. Legyen az utolsó affin réteg

$$W = \begin{pmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & -1 & -1 \\ -1 & -1 & 1 & -1 \end{pmatrix}, \qquad b = (4, 0, 0)^\top,$$

és a logit $z = Wx + b$. Ekkor

$$z = \begin{pmatrix} 1-1-1-1+4 \\ -1+1-1-1+0 \\ -1-1+1-1+0 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \\ -2 \end{pmatrix}.$$

A margin $M = 2 - (-2) = 4$. Mivel a két nem cél logit egyenlő, a lemma egyenlőségi esete áll fenn, ezért az alsó korlát itt éles:

$$\frac{1}{1 + 2e^{-4}} \approx 0{,}9647 \quad (0{,}964663).$$

A pontos softmax ugyanezt az értéket adja:

$$p_1 = \frac{e^2}{e^2 + 2e^{-2}} \approx 0{,}9647, \qquad p_2 = p_3 \approx 0{,}0177.$$

A modell tehát 96,5% konfidenciával mondja a függőleges (vertical, 0) osztályt egy olyan képre, amelyen nincs vonal. A konfidencia nagy, az input mégis OOD. Ez nem azt mutatja, hogy a softmax hibás, hanem azt, hogy a softmax maximuma a logitok különbségét tükrözi, és a tanult affin döntési hipersík a tartományon kívül tetszőlegesen magas pontszámot adhat. Az ellenpélda a [D07] dokumentáció logits-konvencióját követi: a modell logitot ad, a softmax a kiértékeléskor alkalmazott külön lépés.

25.4.4 Kézi példa után minimális kód

A fenti számítás stabil NumPy-implementációja a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben bevezetett max-levonásos trükkel:

```
import numpy as np

x = np.ones((4,), dtype=np.float32)
W = np.array([[1, -1, -1, -1],
              [-1, 1, -1, -1],
              [-1, -1, 1, -1]], dtype=np.float32)
b = np.array([4, 0, 0], dtype=np.float32)

z = W @ x + b                       # (3,) float32

def softmax(z):
    z_stable = z - np.max(z)
    e = np.exp(z_stable)
    return e / e.sum()

p = softmax(z)
print("z =", z)
print("p =", p)
print("max confidence =", p.max())
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, float32, CPU):

```
z = [ 2. -2. -2.]
p = [0.9646631  0.01766842 0.01766842]
max confidence = 0.9646631
```

A float32 számítás a matematikai 0,96466316 értéket 0,9646631-re kerekíti; float64-gyel 0,9646632 adódik. A shape $(3,)$, a konfidencia $[1/3, 1]$-ben van. A numerikus ellenőrzéshez float32 mellett 10^{-5} abszolút tolerancia elegendő.

25.4.5 Labor: ismert és külön OOD kontroll validáción választott elutasítással

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5 és torch 2.14.1 (CPU). Szükséges fájlok: `lab/data/images.npy`, `labels.npy`, `metadata.csv`, `manifest.json`, valamint a 25.3. Kis adat és feature-alapú baseline laborjában mentett `rejection_cnn.pt` modell. Futtatás: `python lab/rejection_rule.py`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A labor egy **terv** és annak implementációja; a kimeneteket a könyv környezetében lefuttatva mértük (Python 3.12.3, NumPy 2.5.4, torch 2.15.0+cpu, CPU, 1 szál, seed 20261020). A hallgató a saját futásában ugyanezt a protokollt követi.

Protokoll.

1. **Ismert adat:** a `lab/data/images.npy`, `labels.npy`, `metadata.csv` és a `manifest.json` betöltése a split szerint. A modell a 25.3. Kis adat és feature-alapú baseline alfejezetben tanított kis CNN; a kimenet minden képre $Z \in \mathbb{R}^{N\times 3}$ logit. A [D07] szerint a tanításkor a CrossEntropyLoss közvetlenül ezt a logitot kapja, int64 célokkal.
2. **Validációs küszöbválasztás:** a val halmaz ($N_{\text{val}} = 60$ kép) logitjaiból számítjuk a c_i konfidenciát. Rögzítünk egy cél-coverage-t, például 0,95, és a τ küszöböt úgy választjuk, hogy a val képek legfeljebb 5%-át utasítsuk el: τ a 3 legkisebb konfidencia közül a legnagyobb, azaz a harmadik legkisebb érték. Mivel az elfogadás feltétele $c_i \geq \tau$, ez a választás csak a szigorúan τ alatti képeket utasítja el, tehát a cél felső korlát az elutasítottak számára, nem pontos darabszám: a harmadik legkisebb értékkel tipikusan 2/60 marad elutasítva. Pontos 3/60 elutasításhoz a $c_i > \tau$ összehasonlítás vagy a negyedik legkisebb érték választása kellene; a könyv a $\geq$ konvenciót használja, és ezt a mért kimenet mellett is kimondja. A küszöböt **nem** az OOD kontrollon hangoljuk.
3. **Külön OOD kontroll:** a `seed=20261020` véletlengenerátorral 200 darab 24×24-es float32 képet készítünk három csoportban:
   - 80 kép egyenletes $U[0,1]$ zaj,
   - 60 kép konstans intenzitású, 0,5 értékű,
   - 60 kép sakktábla-mintázat. A `corrupted_test.npy` itt nem OOD kontroll, mert ugyanahhoz a jelenethez tartozik; azt páros módon (ugyanazon jelenet clean és corrupt view-ját egy egységként összevetve), külön értékeljük.
4. **Mértékek:** az OOD kontrollra számítjuk az elutasítási arányt $1 - \text{coverage}\hat{}$, az elfogadott ismert val képek pontosságát, és a validációs coverage-t. A cél nem egyetlen nyerő szám, hanem a coverage–rejection trade-off (a lefedettség és az elutasítási arány közötti átváltás) rögzítése.

Minimális kód a szabályhoz:

```python
# lab/rejection_rule.py
import numpy as np

def softmax_max(logits):
    z = logits - logits.max(axis=1, keepdims=True)
    p = np.exp(z)
    p = p / p.sum(axis=1, keepdims=True)
    return p.max(axis=1)

def choose_threshold(conf, target_coverage):
    n = len(conf)
    n_reject = int(np.floor((1.0 - target_coverage) * n))
    if n_reject == 0:
        return conf.min() - 1e-6  # mindent elfogadunk
    return float(np.partition(conf, n_reject - 1)[n_reject - 1])

def predict_with_rejection(logits, tau):
    pred = logits.argmax(axis=1)
    conf = softmax_max(logits)
    pred = np.where(conf >= tau, pred, -1)  # -1 = unknown
    return pred, conf

# A méréshez használt, előzetesen mentett bemenetek
val_logits = np.load("lab/val_logits.npy")   # (60,3) float32
val_labels = np.load("lab/val_labels.npy")   # (60,) int64
ood_logits = np.load("lab/ood_logits.npy")   # (200,3) float32
assert val_logits.shape == (60, 3) and val_logits.dtype == np.float32
assert val_labels.shape == (60,)
assert ood_logits.shape == (200, 3) and ood_logits.dtype == np.float32

conf_val = softmax_max(val_logits)
tau = choose_threshold(conf_val, 0.95)
pred_val, _ = predict_with_rejection(val_logits, tau)
accept_val = pred_val != -1

_, conf_ood = predict_with_rejection(ood_logits, tau)
accept_ood = conf_ood >= tau

print("tau:", tau)
print("val coverage:", accept_val.mean())   # mért: 0.9667 (a >= konvenció miatt 58/60
↪  elfogadott)
print("accepted val acc:", np.mean(pred_val[accept_val] == val_labels[accept_val]))
print("OOD rejection:", 1.0 - accept_ood.mean())  # mért
```

A logitok előállítása. A három `.npy` fájl a 25.3. Kis adat és feature-alapú baseline labor tanult `SmallCNN` modelljének kimenete. Előfeltétel, hogy a 25.3. labor lefutása után a modell súlya `torch.save(cnn.state_dict(), "rejection_cnn.pt")` néven mentve legyen; az alábbi producer ezt a checkpointot tölti be, és ugyanazt a `model.eval()` + `inference_mode()` protokollt használja, mint a kiértékelés.

```python
# lab/make_rejection_logits.py
# Munkakönyvtár: lab/. Előfeltétel: a 25.3. labor tanult modellje
# (torch.save(cnn.state_dict(), "rejection_cnn.pt")).
import numpy as np
import torch
from baseline_compare import SmallCNN, load_glyphlab

torch.set_num_threads(1)
X, y, idx, _ = load_glyphlab()
model = SmallCNN()
model.load_state_dict(torch.load("rejection_cnn.pt", map_location="cpu",
                                 weights_only=True))
model.eval()
```

```
def logits_of(images):
    with torch.inference_mode():
        return model(torch.as_tensor(images, dtype=torch.float32)).numpy()


val = idx["val"]
np.save("val_logits.npy", logits_of(X[val]).astype(np.float32))
np.save("val_labels.npy", y[val].astype(np.int64))

rng = np.random.default_rng(20261020)
noise = rng.random((80, 1, 24, 24), dtype=np.float32)
const = np.full((60, 1, 24, 24), 0.5, dtype=np.float32)
board = (np.indices((24, 24)).sum(axis=0) % 2).astype(np.float32)
checker = np.broadcast_to(board, (60, 1, 24, 24)).copy()
ood = np.concatenate([noise, const, checker], axis=0)
np.save("ood_logits.npy", logits_of(ood).astype(np.float32))
```

A `model.eval()` a tanítási módú viselkedést (dropout, batch-normalizáció futó statisztikáinak frissítése) kapcsolja ki; ebben a konkrét hálóban nincs ilyen réteg, a protokoll azonban így általánosan helyes marad. Az `inference_mode()` kikapcsolja az autograd-gráf építését, így a logitok nem függnek gradiens-követéstől, és a mentett `.npy` a befagyasztott modell determinisztikus kimenete.

A mért kimenet (a fenti, `>=` konvencióval; a harmadik legkisebb validációs konfidencia még elfogadott, ezért a tényleges elutasítás $2/60 = 3{,}33\%$):

Mennyiség	Mért érték
τ validáción	0,7616
val coverage	0,9667
accepted val accuracy	0,9138
zaj OOD rejection	0,0875
konstans OOD rejection	0,0000
sakktábla OOD rejection	0,0000

A futtatás ellenőrzése: `python lab/rejection_rule.py`; az OOD logitokat előzetesen a befagyasztott modell `model.eval()` állapotában, `inference_mode()` alatt mentettük. A 200 OOD képet a leírt csoportokban, `seed=20261020` mellett generáltuk: a 80 zajkép `numpy.random.default_rng(20261020).random((80,1,24,24), dtype=numpy.float32)`, a 60 konstans kép `numpy.full((60,1,24,24), 0.5)`, a 60 sakktábla kép pedig `((sor+oszlop) % 2)` szerint 1, illetve 0 értékű volt. A logitokat a 25.3. labor tanuló CNN-je (2979 paraméter) adta. Az OOD rejection a τ rögzítése után számolható, és nem használható modell- vagy küszöbválasztásra. A mért eredmény tanulsága: a konstans és a sakktábla képet a modell gyakorlatilag 1,0 konfidenciával a 0. osztályba sorolja, ezért a $\tau = 0{,}7616$ küszöb egyiket sem utasítja el; a softmax-konfidencia tehát önmagában nem OOD-detektor. Az arányra a [S03] szerinti Wilson-intervallum ad a névlegeshez közeli lefedettségű bizonytalanságot, ezért az elfogadási arány pontbecslése mellett ezt is jelenteni kell. Az ismert val halmazon a `scene_id` a független mintavételi egység (egy jelenet több nézetét nem szabad független képként kezelni), ezért a coverage bizonytalanságát jelenet-szintű egységeken kell becsülni, ahogyan az 5.4. Függő képek, seedek és resampling alfejezetben szerepelt. A szintetikus OOD képeknek nincs jelenetstruktúrája: a 80 zaj-, 60 konstans és 60 sakktábla kép egymástól független generált minta, ezért ott a rejection arány bizonytalanságát csoportonként külön, kép-szintű arány-intervallummal (például Wilson-intervallummal) kell megadni.

25.4.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

25.4.6.1 F1. Fogalmi feladat (nehézség 1)

Magyarázza el, miért nem azonos az *ismeretlen osztály* és az *OOD input*. Adjon egy-egy konkrét példát a GlyphLab keretében. Válaszát a 2. szakasz definícióira alapozza.

25.4.6.2 F2. Levezetési feladat (nehézség 2)

Számítsa ki a softmax-konfidenciát a $z = (6, 0, 0)^\top$ logitvektorra, és ellenőrizze a lemmát $K = 3$, $y = 1$, $M = 6$ mellett. Mennyi a lemma által adott alsó korlát, és mennyi a pontos érték? Válaszát legalább négy tizedesjegyre adja meg.

25.4.6.3 F3. Implementációs feladat (nehézség 2)

Írjon függvényt, amely `(N, K)` alakú float32 logitokra kiszámítja a τ küszöb melletti `pred` tömböt, ahol `pred[i] = -1` elutasítás esetén, egyébként a legnagyobb logit indexe. A függvény a 3. szakasz stabil softmaxát használja. Ellenőrizze a `z = [[2,-2,-2], [0.1,0.2,0.01]]`, `tau=0.9` bemeneten; adja meg a kimenetet és indokolja.

25.4.6.4 F4. Hibakeresési feladat (nehézség 2)

Az alábbi eljárás hibás: a hallgató a teszt OOD zajképeken kiszámítja a konfidencia-eloszlást, és a τ küszöböt úgy választja, hogy az OOD rejection 0,95 legyen; majd ugyanezen az OOD halmazon jelenti a rejection arányt. Mi a hiba, mi a következménye, és mi a javítás? Ellenőrzésként fogalmazza meg, melyik adathalmazon szabad a küszöböt beállítani.

25.4.6.5 F5. Kísérleti/transzfer feladat (nehézség 3)

Tervezzen monitoring protokollt egy gyártásba helyezett GlyphLab-osztályozó számára, ahol hetente 500 új kép érkezik, és a domain-eltolódást kívánjuk jelezni. Adja meg a megfigyelési egységet, a számított metrikákat, a riasztási szabályt és annak korlátját. A terv legyen a 24.4. VI. ellenőrző pont: megbízható CNN-training ellenőrzőpontban rögzített protokollkészítési elvekkel összhangban.

25.4.7 Megoldások és indoklás

25.4.7.1 F1 megoldás

Az ismeretlen osztály a **célhalmaz** része: a modellnek van egy külön kimeneti lehetősége arra, hogy a képet nem sorolja a K ismert osztályba. Az OOD ezzel szemben az **input eloszlás** tulajdonsága: a kép nem a tanulási eloszlásból származik. GlyphLab példa: egy zajkép OOD input; ha a döntési szabályunk a τ alatti konfidenciánál elutasít, akkor az `unknown` célhoz kerül. Ugyanakkor egy ismert osztályú, de erősen eltérő világítású vonalkép is lehet OOD; az elutasítás ekkor helyes lehet, de nem jelenti azt, hogy a képnek ne lenne ismert címkéje. A két fogalom a $\mathcal{Y}_{\text{open}}$ és a $\mathcal{D}_{\text{in}}$ különböző szerződéseit érinti.

25.4.7.2 F2 megoldás

A margin $M = 6$, $K = 3$. A lemma alsó korlátja:

$$p_1 \geq \frac{1}{1 + 2e^{-6}} \approx 0{,}995067.$$

A pontos érték:

$$p_1 = \frac{e^6}{e^6 + 1 + 1} = \frac{e^6}{e^6 + 2} \approx 0{,}995067.$$

Mivel a két nem cél logit egyenlő, a lemma egyenlőségi esete áll fenn: a korlát éles, a két érték megegyezik (a számított különbség 0). Az alakellenőrzés: $p_2 = p_3 = (1 - p_1)/2 \approx 0{,}002467$, összegük 1. A margin növelésével a konfidencia 1-hez tart, függetlenül attól, hogy az input a tanulási tartományból származik-e.

25.4.7.3 F3 megoldás

```
import numpy as np

def softmax_max(logits):
    z = logits - logits.max(axis=1, keepdims=True)
    p = np.exp(z)
    p = p / p.sum(axis=1, keepdims=True)
    return p.max(axis=1)

def predict_or_reject(logits, tau):
    pred = logits.argmax(axis=1)
    conf = softmax_max(logits)
    pred = np.where(conf >= tau, pred, -1)
    return pred

logits = np.array([[2, -2, -2],
                   [0.1, 0.2, 0.01]], dtype=np.float32)
tau = 0.9
print(predict_or_reject(logits, tau))
```

Mért kimenet: `[ 0 -1]`, mert az első sor konfidenciája $0{,}9647 \geq 0{,}9$, a második sor maximális softmax-értéke $e^{0,2}/(e^{0,1} + e^{0,2} + e^{0,01}) \approx 0{,}3661$, tehát elutasítás. A shape $(2,)$, a `-1` nem logit index, hanem az unknown szimbólum. Szélsőeset: a `>=` konvenció mellett a pontosan 1 konfidenciájú minta $\tau = 1$-nél még elfogadott; ha $\tau = 1$ mellett semmit nem akarunk elfogadni, akkor a `>` összehasonlítás vagy $\tau > 1$ szükséges. Pontosan 1 konfidencia csak határértékben (végtelen margin mellett) vagy float32-kerekítés miatt állhat elő, amikor a nem cél osztályok exponenciálisan kicsi valószínűsége már ábrázolhatatlan; a szabály ezt az esetet is egyértelműen

kezeli. A $\tau - \varepsilon$ nem megoldás: csökkenti a küszöböt, tehát több mintát fogad el, és float32 mellett az ε hatása nem kiszámítható; a küszöbértéket és az összehasonlítás konvencióját együtt kell rögzíteni.

25.4.7.4 F4 megoldás

A hiba a **küszöb kiválasztási adathalmazának** megválasztása. Ha a τ-t az OOD tesztképeken hangoljuk, és ugyanazon a halmazon mérjük a rejection arányt, akkor a mért mutató optimistán becsült: a küszöböt éppen ahhoz a zajrealizációhoz igazítottuk. A következmény: más zajtípusra a rejection összeomolhat. A javítás: a τ-t a **validációs ismert halmazon**, egy rögzített coverage-cél alapján választjuk, és az OOD kontrollt csak ezután, a befagyasztott modellen értékeljük. Ellenőrzés: a validáción számított coverage megfelel a célkitűzésnek, az OOD rejection pedig csak jelentett, nem optimalizált mennyiség.

25.4.7.5 F5 megoldásminta

A megfigyelési egység a kép, de a heti riasztáshoz a heti batch a csoportosítási egység. Minden héten kiszámítjuk: az elfogadási arányt a rögzített τ mellett; az elfogadott képek osztályeloszlását; a konfidenciák átlagát és alsó kvartilisét. Az **alsó kvartilis** a konfidenciaeloszlás 25%-os kvantilise: az az érték, amelynél a konfidenciák negyede kisebb. A két osztályeloszlás, a heti p és a validációs q közötti χ^2**-távolság** $\chi^2(p,q) = \sum_{k=1}^{K}(p_k - q_k)^2/q_k$, ahol $K = 3$; független, azonos eloszlású nullhipotézis mellett $N\chi^2(p,q)$ közelítőleg $\chi^2(K-1) = \chi^2(2)$ eloszlású, amelynek 95%-os kvantilise 5,991, ezért előre rögzített korlátként ezt (vagy permutációs kvantilist) használunk. A riasztási szabály: ha a heti elfogadási arány legalább két egymást követő héten a validációs coverage alá esik több mint 5 százalékponttal, vagy az osztályeloszlás χ^2-távolsága a validációs eloszlástól meghaladja az előre rögzített korlátot, akkor domain-eltolódás gyanút jelzünk.

A 60 képes validáció és az 5 százalékpontos küszöb viszonya zajbecsléssel ellenőrizhető. Ha a coverage megközelítőleg 0,95, a 60 képből becsült arány standard hibája $\sqrt{0{,}95 \cdot 0{,}05/60} \approx 0{,}028$, azaz 2,8 százalékpont; a heti 500 képből becsült arányé $\sqrt{0{,}95 \cdot 0{,}05/500} \approx 0{,}010$, azaz 1,0 százalékpont. A két becslés különbségének standard hibája $\sqrt{0{,}95 \cdot 0{,}05/60 + 0{,}95 \cdot 0{,}05/500} \approx 0{,}030$, azaz 3,0 százalékpont, ezért az 5 százalékpontos küszöb alig haladja meg az egy standard hibát, és a validációs referencia bizonytalansága miatt a riasztás zajos. Megoldás: a validációs referenciát nagyobb (például 500 képes) mintából becsülni, a küszöböt a különbség konfidencia-intervallumához igazítani (például 2 standard hiba, mintegy 6 százalékpont), és két egymást követő hét megerősítését megkövetelni. A korlát: a riasztás nem bizonyít OOD-ot, csak további címkézett mintavételt indít; a validációs eloszlás maga is becsült mintáról, ezért a küszöböket konfidencia-intervallummal együtt kell értelmezni. A protokoll a 24.4. VI. ellenőrző pont: megbízható CNN-training alapján seedet, modell-állapotot, logits-verziót és a riasztási naplót együtt rögzíti; a 24.4 protokoll elvei itt is érvényesek: rögzített kérdés és baseline, egy változó tényező, előre rögzített split és metrika, a modellválasztás validáción, a teszt csak a végén, valamint a seed és a környezet naplózása.

Hipotézis és változó: a heti coverage és osztályeloszlás eltérése kimutatható a validációs referenciához képest; a változó tényező a hét és a bemeneti eloszlás. **Seed, kontroll, döntési szabály és erőforrás:** a heti statisztika seedje a riasztási naplóban 20261020; a kontroll a validációs eloszlás és a rögzített, kalibrálatlan modell, amelyen a coverage és a χ^2-korlát előre rögzítve van. A döntési szabály számszerű: 5 százalékpontos coverage-eltérés két egymást követő héten, vagy a χ^2-távolság a 95%-os kvantilis (5,991) felett. A feladat monitoringprotokoll, nem tanítás, ezért architektúra és optimalizáló helyett a rögzített modell és a riasztási szabály szerepel.

Erőforrás: CPU, heti 500 kép, néhány másodperc; a modell újratanítása nem része a protokollnak. ### Tipikus tévedés, önellenőrzés és visszautalás

Tipikus tévedés. „Az OOD-felismerés bizonyított, mert egyetlen zajteszten 95% rejection adódott.” Ez nem bizonyíték. Egyetlen tesztkészleten hangolt küszöbbel a mért rejection kiválasztási torzítást tartalmaz: egy másik, kissé eltérő zajeloszláson ugyanaz a szabály lényegesen rosszabb lehet. **Szemléltető példa:** legyen az első zajkészlet egyenletes $U[0; 1]$ háttérrel, a második $U[0; 0{,}8]$ háttérrel. A konfidencia-eloszlás a két készleten eltolódhat. Tegyük fel szemléltetésképpen, hogy az első készleten a konfidencia közelítőleg $U[0; 1]$, ezért a 95%-os rejectionhöz $\tau = 0{,}95$ tartozik; a második készleten pedig közelítőleg $U[0{,}35; 1{,}35]$. Ekkor a másodikon a rejection $P(c < 0{,}95) = (0{,}95 - 0{,}35)/1 = 0{,}60$, azaz a hangolt szabály a második készleten már csak 0,60 rejectiont ad. (A konkrét eloszlások illusztrációk, nem mért adatok.) A helyes állítás: a rejection szabályt az ismert validáción kell rögzíteni, és az OOD értékelést több, független kontrollcsoporton kell megismételni.

Önellenőrzés. 1) Meg tudja-e fogalmazni, miért nem elegendő a softmax maximuma az OOD detektáláshoz? 2) A laborban a τ küszöb melyik adathalmazon lett választva? 3) Mi a különbség a coverage és a rejection arány között?

Rövid válaszok. (1) A softmax maximuma a zárt osztályhalmazra normalizált relatív pontszám; egy OOD minta is kaphat nagy értéket, ezért önmagában nem detektor. (2) A τ küszöböt az ismert validáción kell rögzíteni, nem a teszten. (3) A coverage az elfogadott minták aránya, a rejection ennek komplementere: $1 - \text{coverage}$.

Visszautalás. A 21.4. Osztályarány, súlyozás és céleloszlás alfejezetre: a softmax-konfidencia normalizált pontszám, és a kalibráltság külön, adaton ellenőrzött tulajdonság. A 23.2. Kalibráció és döntési küszöb alfejezetből: a küszöb kiválasztása a validáció feladata, nem a teszté, és a kalibráltságot külön kell ellenőrizni; ez az OOD-monitoring riasztási küszöbére ugyanúgy érvényes.

25.4.8 Rövid összefoglaló

A zárt osztályozó a teszt inputot mindig a K ismert osztály egyikébe sorolja. A nyílt halmazú döntés ezt egy `unknown` lehetőséggel egészíti ki; az OOD fogalma az input eloszlására vonatkozik. A softmax-konfidencia csak a logitok különbségét tükrözi: a lemmában megmutattuk, hogy margin esetén a maximális konfidencia alsó korlátja $1/(1+(K-1)e^{-M})$, és a csupa egyes kép ellenpéldája 96,5% konfidenciát adott értelmetlen inputra. A helyes elutasítási szabályt a validációs ismert halmazon választott τ adja; az OOD kontrollt a rögzített szabállyal, külön értékeljük. Egyetlen zajteszt nem bizonyít OOD-felismerést, csak ellenőrzött protokoll és monitoring adhat megbízható képet.

25.4.9 Következő lecke

A 26.1. Dobozkoordináta, IoU és lokalizációs loss egységben a kép nem egyetlen osztálycímkét, hanem befoglaló dobozokat kap: bevezetjük a dobozkoordinátákat, az IoU-t és a lokalizációs veszteséget.

25.4.10 Források

- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [S03] E. B. Wilson (1927). *Probable inference, the law of succession, and statistical inference*. Journal of the American Statistical Association.

26 Objektumdetektálás és lokalizáció

26.1 Dobozkoordináta, IoU és lokalizációs loss

26.1.1 Cél, előismeret és induló kérdés

Ez az alfejezet a detektálási feladat első matematikai rétegét építi fel: hogyan adjuk meg egy objektum helyét egy képen, hogyan mérjük két doboz átfedését, és hogyan fogalmazzuk meg a lokalizációt regressziós célként. A 22.1. Adateredet, címke és csoportos split egység rögzítette a függelék A. szakaszában leírt GlyphLab annotációs szerződését (a címke- és dobozrögzítés előírását): a `metadata.csv` minden képhez `xmin, ymin, xmax, ymax` oszlopot ad, és a doboz a maszk legkisebb befoglaló téglalapja. A 25.1. Teljes képosztályozó adatfolyam adatfolyama a képosztályozó bemeneti–kimeneti alakjait rögzítette; most ehhez a lokalizációs kimenetet adjuk hozzá. A 14.1. Stride, padding, dilation és outputméret határkezelési leckéje itt is közvetlenül hasznosul: ugyanaz a fél-nyílt tartománygondolat, amely az outputablak érvényességét definiálta, a doboz pixelhalmazát is meghatározza. A CNN-gerincet és operátorát a 14.4. Csoportos és depthwise konvolúció és 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése egységekből ismerjük; a detektálási fej arra a feature mapra épül. A 22.4. Augmentációs kísérlet és domainváltozás és 23.4. Hibaszeletek és alkalmazási határ módszertani korlátai figyelmeztetnek: a clean és corrupt kiértékelést, valamint a szeletekre bontott hibajelentést a detektálási metrikáknál is meg kell tartani, míg a 25.4. Nyílt halmaz és ismeretlen input nyílt halmaz logikája szerint a lokalizáció önmagában nem dönti el, hogy a dobozban tényleg ismert objektum van-e.

Induló kérdés. Két doboz: $B_1 = [1; 4) \times [1; 4)$ és $B_2 = [3; 6) \times [3; 6)$. Hány közös pixelük van a fél-nyílt konvenció szerint, és mennyi lenne ugyanez zárt intervallumos számolással? Becslését a Metszet és IoU szakasz lemmája után ellenőrizheti.

26.1.2 Dobozkoordináta, fél-nyílt intervallum, terület és normalizálás

Definíció (tengelypárhuzamos doboz, xyxy). Egy tengelypárhuzamos doboz négy valós koordinátája

$$B = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \in \mathbb{R}^4,$$

ahol $x_{\min} < x_{\max}$ és $y_{\min} < y_{\max}$. A doboz mint ponthalmaz a **fél-nyílt** Descartes-szorzat:

$$B = [x_{\min}, x_{\max}) \times [y_{\min}, y_{\max}).$$

A sorrend a könyv egészében `xmin, ymin, xmax, ymax`; ugyanez a GlyphLab `metadata.csv` dobozoszlopainak sorrendje. Diszkrét raszteren a c oszlopindex a $[c, c + 1)$, az r sorindex a $[r, r + 1)$ fél-nyílt cellának felel meg. Így a doboz pontosan azokat a pixeleket foglalja magában, amelyekre $x_{\min} \le c < x_{\max}$ és $y_{\min} \le r < y_{\max}$.

Definíció (terület). A doboz területe

$$A(B) = (x_{\max} - x_{\min})(y_{\max} - y_{\min}).$$

A definíció B pozitivitása miatt pozitív. A fél-nyílt határválasztás a kontinuum területen nem változtat az értéken, a raszteren viszont pontosan a cellák számát adja: az $[x_{\min}, x_{\max})$ intervallum $x_{\max} - x_{\min}$ egész cellát tartalmaz, ha a határok egész indexek.

A GlyphLab maszkjában például ha a foreground pixelek oszlopai $20, \dots, 23$, akkor a doboz vízszintes vetülete $[20; 24)$, hossza 4 pixel; a 24 már nem tartozik bele.

Normalizált koordináták. Legyen a kép szélessége W, magassága H; a GlyphLabon $W = H = 24$. A normalizált doboz

$$\hat{B} = (\hat{x}_{\min}, \hat{y}_{\min}, \hat{x}_{\max}, \hat{y}_{\max}) = \left(\frac{x_{\min}}{W}, \frac{y_{\min}}{H}, \frac{x_{\max}}{W}, \frac{y_{\max}}{H} \right),$$

és a hozzá tartozó ponthalmaz $[\hat{x}_{\min}, \hat{x}_{\max}) \times [\hat{y}_{\min}, \hat{y}_{\max}) \subseteq [0; 1]^2$. A normalizált terület $\hat{A}(\hat{B}) = A(B)/(WH)$, ahol a jobb oldalon a pixelkoordinátás terület szerepel. A folytonos dobozkoordináta és az egész cellaindex kapcsolata csak egész határokra azonos a megadott pixelhalmazzal; tört koordinátáknál külön raszterizálási szabály kell. A normalizálás a modell bemeneti skálájához igazítja a célokat; a Metszet és IoU szakaszban bizonyítjuk, hogy közös skálázás alatt az IoU nem változik.

Tenzoralak. Egy N dobozból álló batch reprezentációja $B \in \mathbb{R}^{N \times 4}$, minden sor egy `[xmin,ymin,xmax,ymax]` vektor, float32. A gyakorlatban a dobozregressziós cél is float32, akkor is, ha a beolvasott annotáció egész.

26.1.3 Metszet és IoU: levezetés

Lemma 3.1 (két fél-nyílt doboz metszete). Legyen $B_1 = [a_1, b_1) \times [e_1, f_1)$ és $B_2 = [a_2, b_2) \times [e_2, f_2)$, ahol $a_i < b_i$ és $e_i < f_i$ $(i = 1; 2)$. A függőleges vetület végpontjait e_i, f_i jelöli, hogy ne ütközzenek a rácson használt c oszlopindexszel. Vezessük be az

$$l = \max(a_1, a_2), \quad r = \min(b_1, b_2), \quad u = \max(e_1, e_2), \quad v = \min(f_1, f_2)$$

jelöléseket. Ha $l < r$ és $u < v$, akkor

$$B_1 \cap B_2 = [l, r) \times [u, v),$$

különben $B_1 \cap B_2 = \emptyset$.

Bizonyítás. Egy $p = (p_x, p_y)$ pont pontosan akkor eleme $B_1 \cap B_2$-nek, ha mindkét koordinátájára mindkét doboz fél-nyílt feltétele teljesül:

$$a_1 \le p_x < b_1, \quad a_2 \le p_x < b_2, \quad e_1 \le p_y < f_1, \quad e_2 \le p_y < f_2.$$

Az x-feltételpár ekvivalens a $\max(a_1, a_2) \le p_x < \min(b_1, b_2)$ egyenlőtlenséggel. Ez pontosan akkor ír le nemüres $[l, r)$ intervallumot, ha $l < r$; egyébként nincs olyan p_x, amely mindkét dobozban benne van. Ugyanígy az y-feltétel a $[u, v)$ intervallumot adja, ha $u < v$. A síkbeli metszet a két koordinátavetület Descartes-szorzata, tehát pontosan akkor nemüres, ha mindkét $l < r$ és $u < v$ teljesül, és ekkor $B_1 \cap B_2 = [l, r) \times [u, v)$.

Következmény 3.2. A Lemma 3.1 feltételei mellett a metszet területe

$$A(B_1 \cap B_2) = \max(0, r - l)\max(0, v - u),$$

ahol a $\max(0, \cdot)$ éppen az üres esetet kódolja.

Definíció (IoU, Intersection over Union). Két pozitív területű doboz átfedési aránya

$$\mathrm{IoU}(B_1, B_2) = \frac{A(B_1 \cap B_2)}{A(B_1 \cup B_2)},$$

ahol a metszetet a Lemma 3.1 definiálja. Az unió területét a szitaformulával (inklúzió–exklúzió, azaz a kétszeresen számolt metszet egyszeri kivonásával) számoljuk:

$$A(B_1 \cup B_2) = A(B_1) + A(B_2) - A(B_1 \cap B_2).$$

Az üres metszet esetén a számláló 0, a nevező pedig $A(B_1) + A(B_2) > 0$, tehát $\mathrm{IoU} = 0$ jól definiált. Az IoU-t a PASCAL VOC detektálási kiértékelés kanonikus átfedésmértékeként rögzíti [P13].

Tulajdonságok. Pozitív területű dobozokra:

1. $\mathrm{IoU}(B_1, B_2) = \mathrm{IoU}(B_2, B_1)$ (szimmetria).
2. $0 \le \mathrm{IoU} \le 1$.
3. $\mathrm{IoU} = 1$ pontosan akkor, ha $B_1 = B_2$ mint ponthalmaz.
4. Ha mindkét dobozt ugyanazzal a tengelyenkénti pozitív skálázással, azaz ugyanazzal az $s_x, s_y > 0$ párral transzformáljuk, az IoU változatlan; speciálisan a pixel- és normalizált koordinátás IoU egyenlő.

Az 1. és 2. állítás közvetlenül a metszet szimmetriájából és a nemnegatív területből következik. A 3. állításhoz: ha $B_1 = B_2$, akkor a metszet és az unió területe egyenlő. Megfordítva, tegyük fel, hogy $B_1 \neq B_2$. Ekkor legalább az egyik tengelyen a két fél-nyílt vetületi intervallum különbözik. Két pozitív hosszúságú fél-nyílt intervallum különbsége, ha nem üres, pozitív hosszúságú intervallumok uniója; a másik tengely vetülete pozitív hosszúságú, ezért a $B_1 \setminus B_2$ és a $B_2 \setminus B_1$ közül legalább az egyik pozitív területű. A $B_1 \cup B_2$ a $B_1 \cap B_2$, a $B_1 \setminus B_2$ és a $B_2 \setminus B_1$ diszjunkt uniója, így a terület additivitása miatt $A(B_1 \cup B_2) > A(B_1 \cap B_2)$, tehát $\mathrm{IoU} < 1$. Ezzel $\mathrm{IoU} = 1$ pontosan a $B_1 = B_2$ esetben áll fenn. A bizonyítás használja a pozitív terület feltételét: nulla területű, degenerált dobozokra az IoU definíciója nem alkalmazható, a kódban külön konvenció (nulla unióra nulla) kezeli ezt az esetet. A 4. állítást külön lemmaként igazoljuk.

Lemma 3.3 (skálainvariancia). Legyen $T(x, y) = (s_x x, s_y y)$, ahol $s_x, s_y > 0$. Pozitív területű dobozokra

$$\mathrm{IoU}(T(B_1), T(B_2)) = \mathrm{IoU}(B_1, B_2).$$

Bizonyítás. T lineáris bijekció, ezért halmazművelettel felcserélhető: $T(B_1 \cap B_2) = T(B_1) \cap T(B_2)$ és $T(B_1 \cup B_2) = T(B_1) \cup T(B_2)$. A terület skálázódását a lineáris leképezés determinánsa adja: minden dobozterület, így a metszet és az unió területe is $s_x s_y$-nal szorzódik. A hányadosban a pozitív közös tényező kiesik. A normalizált koordináták éppen a $s_x = 1/W$, $s_y = 1/H$ eset.

Kézi példa. Legyen $B_1 = [2; 5) \times [1; 4)$ és $B_2 = [4; 7) \times [3; 6)$. Ekkor $l = 4 < r = 5$, $u = 3 < v = 4$, tehát a metszet $[4; 5) \times [3; 4)$, területe 1. A területek $A(B_1) = 3 \cdot 3 = 9$, $A(B_2) = 3 \cdot 3 = 9$; az unió

$9+9-1=17$; így IoU $=1/17\approx 0{,}0588$. Az induló kérdésben szereplő $[1;4)\times[1;4)$ és $[3;6)\times[3;6)$ esetén a metszet $[3;4)\times[3;4)$, amely a 3. sor és 3. oszlop egyetlen cellája; a fél-nyílt IoU tehát $1/17$. Két különböző konvenciót kell elkülöníteni. *Folytonos* téglalapként a zárt $[1;4]^2$ és $[3;6]^2$ területe is 9, a metszet $[3;4]^2$ területe 1 (a határ hozzáadása a területet nem változtatja), tehát az IoU továbbra is $1/17$. *Inkluzív egész pixelindexként* (pixelinclusive) viszont a $[1;4]$ az $1;2;3;4$ indexű pixeleket jelenti, azaz tengelyenként 4 pixelt: mindkét doboz területe 16 pixel, a közös index a 3. és 4. sor/oszlop, tehát a metszet 4 pixel, és IoU $=4/(16+16-4)=4/28=1/7\approx 0{,}1429$. A két számolás közül mindig egyet használunk végig.

26.1.4 Dobozregresszió és lokalizációs loss

Az itt választott *anchor-alapú* detektálási fej feladata nem a négy xyxy koordináta közvetlen kiadása, hanem egy **regressziós delta** előrejelzése egy rögzített hivatkozási dobozhoz, az úgynevezett anchorhoz (horgonydobozhoz) vagy proposalhoz képest; a rögzített horgonydobozokhoz képesti delta-regresszió és a smooth L1 lokalizációs loss ebben a formában a [P14] SSD-leírásában is szerepel. Ez a választott paraméterezés, nem általános definíció: léteznek anchor-free, közvetlen sarok- vagy középpont-regressziós, illetve eloszlásos fejek is. A célokat középpontos paraméterezésben adjuk meg: az xyxy dobozból

$$x=\frac{x_{\min}+x_{\max}}{2},\quad y=\frac{y_{\min}+y_{\max}}{2},\quad w=x_{\max}-x_{\min},\quad h=y_{\max}-y_{\min},$$

és a visszafelé konverzió $x_{\min}=x-w/2$, $y_{\min}=y-h/2$, $x_{\max}=x+w/2$, $y_{\max}=y+h/2$. Legyen az anchor (x_a,y_a,w_a,h_a), a ground truth pedig (x^*,y^*,w^*,h^*). A **céldelta** komponensei:

$$t_x^*=\frac{x^*-x_a}{w_a},\qquad t_y^*=\frac{y^*-y_a}{h_a},\qquad t_w^*=\ln\frac{w^*}{w_a},\qquad t_h^*=\ln\frac{h^*}{h_a}.$$

A hálózat $t=(t_x,t_y,t_w,t_h)$ kimenete ugyanezen a skálán él, és a **dekódolás**

$$x=t_x w_a+x_a,\qquad y=t_y h_a+y_a,\qquad w=w_a e^{t_w},\qquad h=h_a e^{t_h}.$$

Az exponenciális miatt a dekódolt w,h mindig pozitív, tehát a dekódolás mindig érvényes $x_{\min}<x_{\max}$, $y_{\min}<y_{\max}$ dobozt ad. A t_x,t_y nincs korlátozva: a középpont elhagyhatja a képet; ennek kezelése a detektor utófeldolgozásához tartozik (26.2. Egy- és kétlépcsős detektor).

Két érv szól a normalizálás mellett. Egyrészt a w_a,h_a-val osztott középponteltolás mértékegységet vált: a $t_x=t_x^*=0{,}1$ azt jelenti, hogy a célközéppont az anchor szélességének tizedével jobbra van, függetlenül a pixelmérettől. Másrészt a szélesség logaritmusa a relatív hibát szimmetrizálja: a kétszeres túllövés és a feleakkora alullövés egyaránt $|\ln(2/1)|=|\ln(1/2)|=\ln 2$ delta-hibát ad. Ez a választás a [P12] nyomán elterjedt családba tartozik; a hivatkozott cikk teljes módszerszakaszát itt nem idézzük, ezért a pontos egyezést nem állítjuk.

Lokalizációs loss. A predikált t és a cél t^* közötti komponensenkénti veszteség a **smooth L1** függvény, $\beta>0$ küszöbbel:

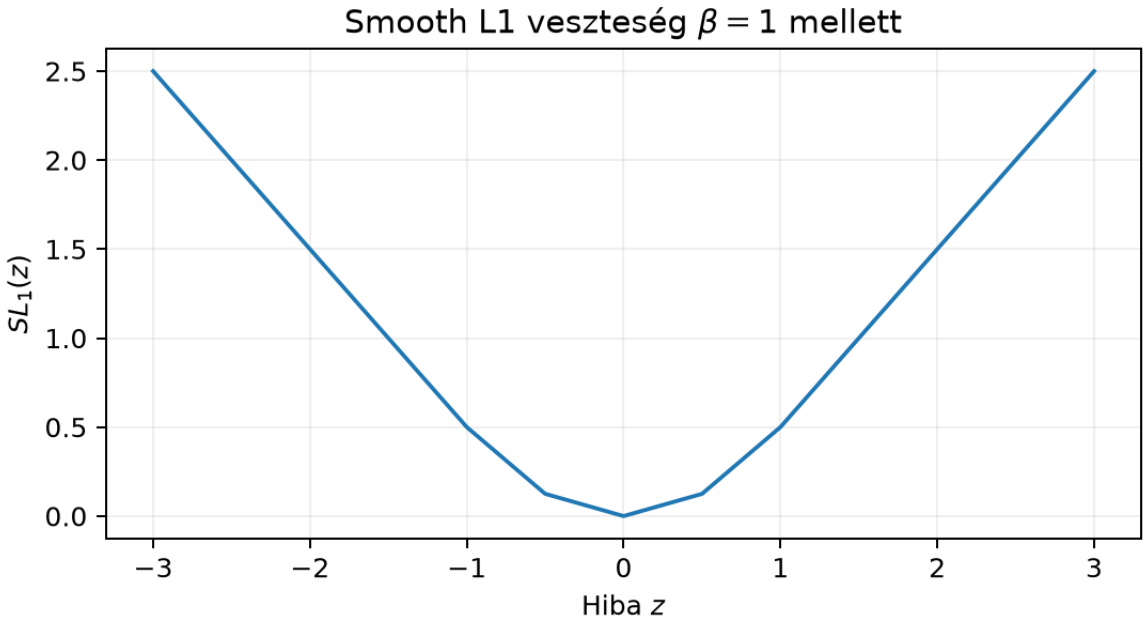
$$SL_1(z;\beta)=\begin{cases}\frac{z^2}{2\beta}, & |z|<\beta,\\ |z|-\frac{\beta}{2}, & |z|\geq\beta.\end{cases}$$

A $\beta=1$ választással a kis hibákra kvadratikus, nagy hibákra lineáris a büntetés; a gradiens $|z|<\beta$ esetén z/β, azon kívül $\operatorname{sign}(z)$. Ez robusztusabb az L2-nél, mert egy durva kiugró dobozhiba nem

dominálja a batch gradienst, ugyanakkor a $|z|$ nemdifferenciálhatóságát is kisimítja a nullában. A lokalizációs loss egy pozitív objektumokból álló $\mathcal{P}$ halmaz felett

$$L_{\text{loc}} = \frac{1}{|\mathcal{P}|} \sum_{i \in \mathcal{P}} \sum_{j \in \{x,y,w,h\}} SL_1(t_{ij} - t^*_{ij}; \beta),$$

és $L_{\text{loc}} = 0$, ha $\mathcal{P}$ üres. A cél csak akkor értelmes, ha az i-edik javaslathoz tényleges objektum tartozik; háttérjavaslatra nincs definiált dobozregressziós cél. Az osztály- és objektumossági loss ettől külön tag, amelyet a detektorarchitektúrák tárgyalásakor (26.2. Egy- és kétlépcsős detektor) adunk hozzá.



26.1. ábra: Smooth L1 veszteség $\beta = 1$ mellett

A vízszintes tengely a $z = t - t^*$ hiba, a függőleges a smooth L1 értéke $\beta = 1$ mellett; a képletből számolt szemléltetés, nem mérés.

Kézi cél- és loss-példa. Anchor $(x_a, y_a, w_a, h_a) = (3{,}5; 2{,}5; 3; 3)$, ground truth $(x^*, y^*, w^*, h^*) = (5{,}5; 4{,}5; 3; 3)$. A cél $t^*_x = (5{,}5 - 3{,}5)/3 = 0{,}6667$, $t^*_y = (4{,}5 - 2{,}5)/3 = 0{,}6667$, $t^*_w = t^*_h = 0$. Ha a hálózat $t = (0{,}5; 0{,}6; -0{,}1; 0{,}2)$-t ad, akkor $\beta = 1$-gyel

$$L_{\text{loc}} = \frac{1}{2}(0{,}1667^2 + 0{,}0667^2 + 0{,}1^2 + 0{,}2^2) = \frac{1}{2}(0{,}02778 + 0{,}00444 + 0{,}01 + 0{,}04) \approx 0{,}0411.$$

A $\frac{1}{2}$ tényező a $\beta = 1$ választás következménye: a smooth L1 kis hibákra érvényes ága $z^2/(2\beta) = z^2/2$, és a példában minden $|z| < \beta$, ezért a négy komponens összegét $1/2$ szorozza.

A dekódolt predikció $(x, y) = (3{,}5 + 0{,}5 \cdot 3; 2{,}5 + 0{,}6 \cdot 3) = (5{,}0; 4{,}3)$, ami a célhoz képest $(-0{,}5; -0{,}2)$ középponthibát ad; a méretek $3e^{-0{,}1} \approx 2{,}71$ és $3e^{0{,}2} \approx 3{,}66$.

26.1.5 Labor: GlyphLab dobozok és kézi IoU-ellenőrzés

Előfeltételek. Python 3.12, a `lab/` csomag telepítve (`cd lab && uv sync`), NumPy 2.3.5. Szükséges fájlok: `data/images.npy`, `data/masks.npy`, `data/metadata.csv` és `data/manifest.json`; futtatás: `python lab/n101_lab.py`. Várható futásidő CPU-n néhány másodperc, a tényleges időt a futás naplózza.

A labor a `lab` munkakönyvtárból futtatandó, ahol a `data/images.npy`, `data/masks.npy`, `data/metadata.csv` és `data/manifest.json` fájlok a kezdőlabor szerződésének megfelelően vannak jelen. Az adatgenerálás seedje `20261020`; a dobozszámítás maga determinisztikus, ezért új seedet nem állítunk be. A mask `uint8`, a doboz `float32`, a számítás CPU-n történik. A `metadata.csv index` oszlopa a `masks.npy` megfelelő sorának indexe (a generátor a kép beszúrásakor `index=len(images)` értéket ír), az `xmax`/`ymax` pedig fél-nyílt határ: a generátor `xs.max()+1`, illetve `ys.max()+1` értéket tárol, összhangban a `mask_to_box` függvénnyel.

```
# lab/n101_lab.py
import csv
import numpy as np

masks = np.load("data/masks.npy")   # (360,1,24,24) uint8
print(f"masks shape: {masks.shape}, dtype: {masks.dtype}")

def mask_to_box(mask: np.ndarray) -> np.ndarray:
    """A foreground pixelek legkisebb fél-nyílt xyxy befoglaló doboza."""
    ys, xs = np.nonzero(mask[0])
    if len(xs) == 0:
        raise ValueError("üres maszk")
    return np.array([xs.min(), ys.min(), xs.max() + 1, ys.max() + 1],
                    dtype=np.float32)

def box_iou(a: np.ndarray, b: np.ndarray) -> float:
    """Két [xmin,ymin,xmax,ymax] fél-nyílt doboz IoU-ja."""
    ix0, iy0 = max(a[0], b[0]), max(a[1], b[1])
    ix1, iy1 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0.0, ix1 - ix0) * max(0.0, iy1 - iy0)

    def box_area(z):
        return (z[2] - z[0]) * (z[3] - z[1])

    union = box_area(a) + box_area(b) - inter
    return float(inter / union) if union > 0 else 0.0

with open("data/metadata.csv", newline="", encoding="utf-8") as f:
    meta = list(csv.DictReader(f))

max_err = 0.0
for row in meta:
    idx = int(row["index"])
    computed = mask_to_box(masks[idx])
    ref = np.array([float(row[k]) for k in ("xmin", "ymin", "xmax", "ymax")],
                   dtype=np.float32)
    err = float(np.max(np.abs(computed - ref)))
    max_err = max(max_err, err)
print(f"metadata doboz max abs hiba: {max_err:.6f}")
assert max_err == 0.0

a = np.array([10.0, 5.0, 20.0, 15.0], dtype=np.float32)   # [10,20)x[5,15)
b = np.array([15.0, 10.0, 25.0, 20.0], dtype=np.float32)  # [15,25)x[10,20)
iou_ab = box_iou(a, b)
print(f"kézi IoU: {iou_ab:.6f} (elvárt: {1/7:.6f})")
assert abs(iou_ab - 1/7) < 1e-6
```

```
c = np.array([0.0, 0.0, 10.0, 10.0], dtype=np.float32)
d = np.array([10.0, 0.0, 20.0, 10.0], dtype=np.float32)
print(f"érintkező IoU: {box_iou(c, d):.6f} (elvárt: 0.0)")
assert box_iou(c, d) == 0.0
```

Futtatási parancs: `python n101_lab.py`. A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, GlyphLab seed 20261020):

```
masks shape: (360, 1, 24, 24), dtype: uint8
metadata doboz max abs hiba: 0.000000
kézi IoU: 0.142857 (elvárt: 0.142857)
érintkező IoU: 0.000000 (elvárt: 0.0)
```

A `c` és `d` dobozok érintkeznek, mert az első vízszintes vetülete $[0; 10)$, a másodiké $[10; 20)$; a közös határ nincs benne a fél-nyílt intervallumokban, ezért az IoU nulla. A kimenet a megadott környezetben ténylegesen lefutott program eredménye; az assertek a futás során ellenőrizték a doboz- és IoU-értékeket.

26.1.6 Feladatok

Az F5 kísérlettervezési feladat kötelező sablonja. A válaszban add meg a hipotézist; a független és a függő változót; a kontrollt vagy baseline-t; a modellarchitektúrát, az optimalizálót, a tanulási rátát és az epochszámot; a seedek számát és konkrét értékét; a metrikát és a megfigyelési egységet; az előre rögzített, számmal megfogalmazott döntési szabályt és a statisztikai próbát; valamint az erőforráskeretet. Ha a feladat nem tanít modellt, a tanítási recept helyett az adott eljárás kontrollját, döntési szabályát és erőforrását kell kitölteni.

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem közömbös, hogy a dobozokat fél-nyílt intervallumként vagy inkluzív egész pixelindexként reprezentáljuk, amikor az IoU-t diszkrét pixelrácson számoljuk. Adjon konkrét dobozpárt, amelyre a két konvenció eltérő IoU-t ad.

F2 (levezetési, nehézség 2). Legyen $B_1 = [0; 6) \times [0; 4)$ és $B_2 = [3; 8) \times [2; 7)$. A Lemma 3.1 segítségével adja meg a metszetet, a területeket, az uniót és az IoU-t. Igazolja a 3.3 Lemma állítását ezen a konkrét páron úgy, hogy mindkét dobozt $s = 24$-szeresére skálázza.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt `box_iou_batch(A, B)` néven, amely $A \in \mathbb{R}^{N\times 4}$ és $B \in \mathbb{R}^{M\times 4}$ bemenetből az $N \times M$ IoU-mátrixot adja. Vektorizált megoldást adjon; ellenőrizze az $A = \{(0; 0; 10; 10), (20; 20; 30; 30)\}$, $B = \{(5; 5; 15; 15), (21; 21; 31; 31), (0; 0; 10; 10)\}$ esetre az elvárt mátrixot 10^{-6} tűréssel.

F4 (hibakeresési, nehézség 1). Az alábbi függvény egyetlen pixelre szóló maszkból rossz dobozt ad:

```
def mask_to_box_bug(mask):
    ys, xs = np.nonzero(mask[0])
    return np.array([xs.min(), ys.min(), xs.max(), ys.max()], dtype=np.float32)
```

Lokalizálja a hibát, mutassa meg következményét egy $[12; 13) \times [12; 13)$ egycellás maszkon, és adja meg a javítást.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kis kísérletet a GlyphLab adaton a smooth L1 β küszöbének kiválasztására egy rögzített anchorú, egyobjektumos dobozregressziós fejhez. Adja meg a kérdést, a baseline-t, a splitet, a metrikát, a választási szabályt, az erőforrásigényt és a következtetés érvényességi határát. Az eredményt előre ne rögzítse.

26.1.7 Megoldások és indoklás

F1. A fél-nyílt $[a, b)$ intervallum a raszteren pontosan a $b-a$ darab egész cellát fedi, a zárt $[a, b]$ pedig $b-a+1$ darabot, ha egész határokkal dolgozunk. Az IoU ezért függ a konvenciótól, ahogy a Metszet és IoU szakasz példája mutatja: $B_1 = [1; 3) \times [1; 3)$ négy cellát, $B_2 = [3; 5) \times [3; 5)$ szintén négy cellát fed, és fél-nyílt konvencióban diszjunktak, tehát IoU $= 0$. Zárt konvencióban mindkettő kilenc cellát fedne, a közös 3. sor és oszlop egy cellát adna, így IoU $= 1/(9 + 9 - 1) = 1/17$. Ugyanaz a képpár tehát más metrikát ad; a keverés off-by-one (eggyel eltolt index) eltérést okoz.

F2. A metszet: $l = \max(0; 3) = 3$, $r = \min(6; 8) = 6$, $u = \max(0; 2) = 2$, $v = \min(4; 7) = 4$; $l < r$ és $u < v$, tehát a metszet $[3; 6) \times [2; 4)$, területe $3 \cdot 2 = 6$. A területek $A(B_1) = 6 \cdot 4 = 24$, $A(B_2) = 5 \cdot 5 = 25$; az unió $24 + 25 - 6 = 43$; az IoU $6/43 \approx 0{,}1395$. A 24-szeres skálázás után a metszet $[72; 144) \times [48; 96)$, területe $72 \cdot 48 = 6 \cdot 24^2$; az unió $43 \cdot 24^2$; a hányados $(6 \cdot 24^2)/(43 \cdot 24^2) = 6/43$, tehát változatlan.

F3. A vektorizált megoldás shape-logikája: `A[:, None, :]` alakja $(N, 1, 4)$, `B[None, :, :]` alakja $(1, M, 4)$, így a kivonásaik $(N, M, 4)$-re kiterjesztődnek (broadcast).

```
import numpy as np

def box_iou_batch(A, B):
    lt = np.maximum(A[:, None, :2], B[None, :, :2])
    rb = np.minimum(A[:, None, 2:], B[None, :, 2:])
    inter = np.prod(np.maximum(rb - lt, 0.0), axis=-1)

    def box_area(X):
        return (X[:, 2] - X[:, 0]) * (X[:, 3] - X[:, 1])

    union = box_area(A)[:, None] + box_area(B)[None, :] - inter
    return np.divide(inter, union, out=np.zeros_like(inter), where=union > 0)

A = np.array([[0, 0, 10, 10], [20, 20, 30, 30]], dtype=np.float32)
B = np.array([[5, 5, 15, 15], [21, 21, 31, 31], [0, 0, 10, 10]],
             dtype=np.float32)
expected = np.array([[1 / 7, 0.0, 1.0],
                     [0.0, 81 / 119, 0.0]], dtype=np.float32)
iou = box_iou_batch(A, B)
print(iou)
print("max abs hiba:", float(np.max(np.abs(iou - expected))))
assert np.allclose(iou, expected, atol=1e-6, rtol=0)

zero = np.zeros((1, 4), dtype=np.float32)
print("nulla unió IoU:", box_iou_batch(zero, zero))
```

A program a várt mátrixot `expected` tömbben rögzíti, és `10^{-6}` abszolút tűréssel ellenőrzi; a futás során a `max abs hiba` értéke 0, az `assert` nem áll meg. A `np.divide(..., where=union > 0)` a nulla unió (két nulla területű doboz) esetére 0-t ad, nem `NaN`-t, összhangban a skaláris `box_iou` konvenciójával.

Az elvárt mátrix: az első sorban a $[0; 10)$-es doboz a $[5; 15)$-tel $25/175 = 1/7$, a $[21; 31)$-gyel 0, önmagával 1; a második sorban a $[20; 30)$-as doboz rendre 0, $81/(100 + 100 - 81) = 81/119 \approx 0{,}6807$, 0.

F4. A hiba az `xmax=xs.max()` és `ymax=ys.max()` hiányzó $+1$. Egyetlen, a 12. oszlopban és 12. sorban álló maszkra a hibás függvény `[12,12,12,12]`-t ad, amelynek `xmax-xmin=0` és `ymax-ymin=0`, tehát területe nulla; a doboz üres, és az IoU számlálója is nulla lesz, még akkor is, ha két ugyanilyen egycellás maszk dobozát hasonlítjuk össze. A javítás `xs.max()+1` és `ys.max()+1`; ezzel a doboz `[12,13)x[12,13)`, területe 1, és önmagával vett IoU-ja 1.

F5. Megoldásminta. **Kérdés:** a $\beta \in \{0{,}5; 1; 2\}$ küszöb hogyan hat a GlyphLab dobozregressziós fej val- és test-IoU-jára? **Adat és split:** a rögzített GlyphLab train/val/test split; a maszkból minden képhez egy ground truth doboz. **Modell és cél:** a 25.1. Teljes képosztályozó adatfolyam alfejezetben ismertetett kis CNN-gerinc után egy 4 dimenziós fej; az anchor legyen a train dobozok átlagos középpontos doboza, az objektumosság oldható meg a maszkból, így minden kép pozitív. **Változó tényező:** csak β; minden más hiperparaméter rögzített. **Baseline:** a train ground truth dobozok konstans átlaga, képenkénti IoU-ban mérve. **Metrika:** képenkénti IoU átlaga; a checkpointot minden β és seed mellett a val-IoU szerint választjuk, a tesztet csak a végén, a kiválasztott β-val értékeljük egyszer. **Ismétlés:** három training seed; a jelentéshez a seedek közötti szórást is közöljük. A β-szinteket ugyanazon seedekkel és ugyanazon testképeken párosan hasonlítjuk össze; a döntés a jelenet szintű páros különbség bootstrap CI-jén alapul: 0-t kizáró 95%-os CI esetén állítunk sorrendet, egyébként nem. **β-választási szabály:** a legjobb val-IoU-t adó β-t választjuk, de csak akkor, ha a legjobb és a második legjobb közötti páros különbség 95%-os CI-je is kizárja a 0-t; ha nem, a konzervatívabb, előre rögzített $\beta = 1$ értéket használjuk, és ezt a döntést a teszt megnyitása előtt rögzítjük. **Erőforrás:** kis CPU, float32; a becslés: 240 train kép, 32-es batch, 30 epoch mellett 8 lépés/epoch, azaz 240 lépés/futás, a 3 β × 3 seed = 9 futás összesen 2160 lépés, ami ezen a méreten a másodperces–perces nagyságrendbe esik; a „néhány perc" felső becslés, nem mért érték, a tényleges futásidőt a futtatás naplózza. **Érvényességi határ:** a GlyphLab procedurális adat, nem természetes képi benchmark; a β sorrendje nem vihető át általános detektálási adatra, és a konstans-anchor konstrukció nem méri a detektor proposal-minőségét.

Hipotézis és egység: a $\beta = 1$ adja a legjobb val-IoU-t, a szélső értékek rontják; a megfigyelési egység a kép, a bootstrap mintavételi egysége a `scene_id`. **Recept és seedek:** a fej tanítása Adam optimalizálóval, `lr=10^{-3}`, 30 epoch és 32-es batch mellett fut; a három seed értéke 20261020–20261022, minden β-szinten ugyanazok. A döntési küszöb a jelenetszintű páros bootstrap 95%-os CI-je, és a legjobb és a második legjobb β közötti legalább 0,01 IoU-különbség; ennél kisebb eltérést akkor sem nevezünk sorrendnek, ha a CI kizárja a nullát. ### Tipikus tévedés, önellenőrzés és visszautalás

Tipikus tévedés. Az inkluzív egész pixelindex és a fél-nyílt koordináta keverése off-by-one (eggyel eltolt index) hibát ad. Az érintkező $[10; 20) \times [10; 20)$ és $[20; 30) \times [10; 20)$ dobozok fél-nyílt konvencióban diszjunktak; ha valaki a metszetet üres helyett egy közös oszlopként kezeli, hamis pozitív átfedést mér. A labor `c/d` esete ugyanezt a geometriai helyzetet ellenőrzi más koordinátákkal.

Önellenőrzés. (1) A `box_iou` függvény üres metszetre 0-t ad, nem pedig negatív területet? (2) A normalizált és pixelkoordinátás IoU ugyanazt az értéket adja a 3.3 Lemma alapján? (3) A `mask_to_box` üres maszkra kivételt dob, nem néma nulla dobozt? (4) A céldelta dekódolása visszaadja a ground truth dobozt a $t = t^*$ esetben?

Rövid válaszok. (1) Igen: üres metszetre `box_iou` 0-t ad, nem negatív területet. (2) Igen: a 3.3 Lemma skálainvarianciája miatt a normalizált és a pixelkoordinátás IoU egyenlő. (3) Igen: `mask_to_box` üres maszkra kivételt dob, nem néma nulla dobozt ad. (4) Igen: a dekódoló inverz leképezés a $t = t^*$ célértéket a ground truth dobozra viszi vissza.

Visszautalás. A 14.1. Stride, padding, dilation és outputméret outputméret-levezetésében az érvényes ablakot ugyanilyen fél-nyílt indexfeltétellel definiáltuk: az ablak minden mintavételi pontjának a megengedett tartományba kell esnie, és a szélső határ már nem tartozik a halmazhoz. Ez a szemlélet itt a dobozok pixelhalmazára költözött át.

26.1.8 Rövid összefoglaló

A tengelypárhuzamos doboz $[x_{\min}, x_{\max}) \times [y_{\min}, y_{\max})$ fél-nyílt pozitív területű halmaz, xyxy sorrendben. Két doboz metszete a koordinátánkénti max–min vetület, ha $l < r$ és $u < v$, különben üres; területe $\max(0, r-l)\max(0, v-u)$. Az IoU a metszet és az unió területének hányadosa, üres metszetre nulla, és közös pozitív skálázásra invariáns. A dobozregresszió normalizált, log-skálázott deltákat tanul anchorhoz képest, smooth L1 veszteséggel, csak pozitív objektumokra. A labor a GlyphLab maskból rekonstruálja a metadata dobozait, és érintkező dobozokkal teszteli a fél-nyílt konvenciót.

26.1.9 Következő lecke és források

A 26.2. Egy- és kétlépcsős detektor egység ezekre a fogalmakra építi az egy- és kétlépcsős detektorokat: a proposal, az objektumossági pontszám és az osztálylogit a dobozregressziós deltával együtt alkotja a detektálási veszteséget.

- [P12] Ren et al. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. https://arxiv.org/abs/1506.01497.
- [P13] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. és Zisserman, A. (2010). *The PASCAL Visual Object Classes (VOC) Challenge*. International Journal of Computer Vision, 88(2), 303–338. Az IoU-alapú detektálási kiértékelés forrása.
- [P14] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y. és Berg, A. C. (2016). *SSD: Single Shot MultiBox Detector*. Computer Vision – ECCV 2016, LNCS 9905, 21–37. Springer. A rögzített horgonydobozok és a smooth L1 lokalizációs loss tárgyalásának forrása.

26.2 Egy- és kétlépcsős detektor

26.2.1 1. Cél, előismeret és induló kérdés

A 26.1. Dobozkoordináta, IoU és lokalizációs loss egység rögzítette a doboz reprezentációját: xyxy, félig nyílt intervallum, IoU, normalizált koordináta és dobozregressziós cél. Ott azonban csak az értékeléshez és a finomításhoz szükséges geometriai nyelvet építettük fel. Most a detektort mint *egész rendszert* vizsgáljuk, és elválasztjuk egymástól a három különböző felelősséget: a *régiójavaslatot* (region proposal), az *objektumosságot* (objectness) és az *osztálydöntést*.

A 25.2. Multilabel és hierarchikus cél multilabel egységben a szigmoid és a BCE szétválasztotta a független döntéseket; a 22.2. Címkemegőrző transzformációk megmutatta, hogy a geometriai transzformáció és a doboz szinkronizálása a feladattól függ; a 14.2. Több bemeneti és kimeneti csatorna pedig a kernel bank és a csatornakeverés alakjait adta meg, amit a detektor fejében ismét használni fogunk.

Induló kérdés. Egy 24×24-es képen egy 2×2-es cellarácsra épülő egylépcsős detektornak négy dobozslotja van, minden cella maga egy 12×12-es horgonydoboz. A jobb alsó cella belsejében egyetlen 12×12-es, $(10; 10; 22; 22)$ xyxy dobozként megadott cél áll. Hány pozitív mintát ad a matching, ha a pozitív küszöb $\tau_{pos} = 0{,}5$? A választ a kézi példában számoljuk ki; előzetes becslésként gondoljuk meg, hány cellával lehet a cél középpontja átfedésben.

26.2.2 2. Definíció és intuíció: a három elkülönített felelősség

Definíció (régiójavaslat). Legyen $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ képbatch. Egy régiójavaslat-generáló minden X_n képhez egy *jelölt dobozhalmazt* ad:

$$R(X_n) = \{r_{n,1}, \dots, r_{n,T}\}, \qquad r_{n,t} \in \mathbb{R}^4,$$

ahol minden $r_{n,t}$ xyxy koordináta a 26.1. Dobozkoordináta, IoU és lokalizációs loss konvenció szerint. A javaslat önmagában *nem* hordoz osztálydöntést; csak azt állítja, hogy a jelölt területen érdemes részletes elemzést végezni. A kétlépcsős detektorban a második fokozat kizárólag ezen a T jelöltön dolgozik.

Definíció (javaslati recall). Ha $G = \{g_1, \dots, g_M\}$ a ground truth dobozok halmaza, $M > 0$, akkor a τ IoU-küszöbhöz tartozó javaslati recall:

$$\mathrm{Recall}_\tau(R, G) = \frac{1}{M}\sum_{m=1}^{M} \mathbf{1}\Big[\max_{t=1,\dots,T} \mathrm{IoU}(r_t, g_m) \geq \tau\Big].$$

Az indikátor azt vizsgálja, hogy minden valódi objektumhoz van-e legalább egy elég jól illeszkedő javaslat. Az $M = 0$ esetben a mennyiség definiálatlan; üres képen nincs mit felidézni.

Definíció (objectness, objektumosság). Az objectness egy skalár kimenet, amely azt a kérdést válaszolja meg, hogy az adott slot *felelős-e egy objektumért*, függetlenül annak osztályától. Formálisan egy $\hat{o} \in \mathbb{R}$ logithoz tartozó Bernoulli-döntés: $\sigma(\hat{o}) = \frac{1}{1+e^{-\hat{o}}}$ annak a becsült valószínűsége, hogy a slot előtér. Ez különbözik a 25.2. Multilabel és hierarchikus cél többcímkés osztályfejétől: ott minden címke külön jelenlétet dönt el, itt a háttér–előtér döntés *megelőzi* az osztályozást. A fejezet didaktikai detektora a háttér és a három tárgyosztály közötti döntést objectness-logittal és előtérre értelmezett hármas osztályozóval faktorizálja. Alternatív architektúra közös, háttérosztályt is tartalmazó softmaxot alkalmazhat; a két parametrizáció nem azonos gradiensű tanítási szerződés, noha megfelelő kapacitással ugyanazt a négyosztályos prediktív eloszlást is leírhatják.

Definíció (anchor, horgonydoboz; slot, predikciós hely). Az egylépcsős detektor sűrűn helyez el predikciós *slotokat* (predikciós helyeket). Minden slot egy $a = (a_{cx}, a_{cy}, a_w, a_h)$ horgonydobozhoz tartozik, ahol $a_w, a_h > 0$; a cx, cy jelöli a doboz középpontjának képpont-koordinátáját. A hálózat nem közvetlenül a végleges dobozt, hanem a horgonyhoz viszonyított Δ eltolást prediktálja, majd a dekódolás:

$$b_{cx} = a_{cx} + \Delta_{cx} a_w, \quad b_{cy} = a_{cy} + \Delta_{cy} a_h, \quad b_w = a_w \exp(\Delta_w), \quad b_h = a_h \exp(\Delta_h).$$

A célkódolás ennek inverze: $\Delta_{cx} = (g_{cx} - a_{cx})/a_w$, $\Delta_{cy} = (g_{cy} - a_{cy})/a_h$, $\Delta_w = \log(g_w/a_w)$, $\Delta_h = \log(g_h/a_h)$, feltéve, hogy a horgony és a cél doboz nem degenerált. Az exponenciális forma garantálja a pozitív szélességet és magasságot; ez a 26.1. Dobozkoordináta, IoU és lokalizációs loss normalizált koordinátagondolat továbbfejlesztése, és nem az egyetlen lehetséges szerződés — a választott fejnek és a targetnek ugyanazt a kódolást kell használnia.

Intuíció a két családhoz. A kétlépcsős detektor először *kevés, jó jelöltet* keres, majd mindegyiket drága fejjel osztályozza és finomítja. Az egylépcsős detektor *minden* sloton egyszerre dönt, és a slotok sűrű rácsát implicit javaslathalmazként kezeli. Az elsőnél a javaslat minősége, a másodiknál a pozitív–negatív egyensúly válik központi tervezési kérdéssé.

26.2.3 3. A két család felelősségtérképe

Kétlépcsős út: R-CNN és Faster R-CNN. A korai R-CNN vonal először külső javaslatokat vágott ki, és minden régiót külön osztályozott. A [P12] absztraktja szerint a Faster R-CNN ezt a javaslatgenerálást tanulhatóvá teszi: a *Region Proposal Network* (RPN) a detektáló hálózattal közös, teljes képre számolt konvolúciós jellemzőtérképre épül, és maga is végponttól végpontig tanított, teljesen konvolúciós hálózat. Ezzel a javaslatgenerálás megszűnik külső, nem tanulható előfeldolgozás lenni. A rendszer felelősségtérképe:

Fokozat	Felelősség	Kimeneti alak
Backbone	közös sűrű jellemzőtérkép	$1 \times C \times H' \times W'$
RPN	javaslat + objectness	$P \times 4$ és P logit
Detektáló fej	osztály + finomítás	$P \times (K+1)$ és $P \times 4$

Itt P a javaslatok száma. A második fokozat csak az RPN által kiadott P régión dolgozik; az osztályozási teljesítmény ezért függ az első fokozat recalljától, a javaslatok közös jellemzőtérképe pedig a 14.2. Több bemeneti és kimeneti csatorna alfejezetben megismert csatornakeverés újrafelhasználása.

Egylépcsős út: YOLO és SSD család. A [P13] preprint absztraktja a detektálást egyetlen hálózati kiértékelésként fogalmazza meg: a rendszer közvetlenül, egyetlen előremenetben ad dobozokat és osztályvalószínűségeket. Az SSD-szerű család ugyanezt a sűrű default-box gondolatot különböző léptékű jellemzőtérképekre teríti ki; a jelen fejezet forráskatalógusa azonban nem tartalmazza az SSD eredeti cikkét, ezért annak részletes képletét nem tárgyalom (a részletes küszöb- és slotkonvenciók alkalmazásspecifikusak).

Az egylépcsős felosztásban a B slot cellánkénti felelőssége:

$$g_{ij} = \max_{b=1,\dots,B} m_{ijb},$$

ahol $m_{ijb} \in \{0;1\}$ a matchingből származó pozitív címke: a cella akkor felelős egy objektumért, ha valamelyik slotja az. Az objectness fej minden slotra külön logitot ad, az osztályfej a felelős cella osztályát adja; a dobozfej csak a pozitív slotokon kap regressziós célt.

26.2.4 4. Pontos matching-szabály és többtagú cél

A levezetési kötelezettség a választott detektor céljának és mintapárosításának pontos felírása. Ehhez előbb a matching (párosítás) szabályát rögzítjük.

Definíció (matching szabály). Legyen $G = \{g_1, \dots, g_M\}$ a célok, $A = \{a_1, \dots, a_T\}$ a slotok horgonydobozai. Minden s slothoz keressük a legjobban illeszkedő célt, és minden m célhoz a legjobban illeszkedő slotot:

$$i^*(s) = \arg\max_m \mathrm{IoU}(a_s, g_m), \qquad s^*(m) = \arg\max_s \mathrm{IoU}(a_s, g_m).$$

Adott $0 < \tau_{neg} \le \tau_{pos} < 1$ küszöbök mellett a pozitív halmaz

$$P = \{s : \mathrm{IoU}(a_s, g_{i^*(s)}) \ge \tau_{pos}\} \ \cup \ \{s^*(m) : m = 1, \dots, M\}.$$

A negatív halmaz

$$N = \{s : \max_m \mathrm{IoU}(a_s, g_m) < \tau_{neg}\} \setminus P,$$

a maradék $I = \{1; \dots; T\} \setminus (P \cup N)$ figyelmen kívül hagyott, és nem járul a célhoz. A $\{s^*(m)\}$ tag minden ground truth dobozhoz rendel legalább egy pozitív *jelöltet* akkor is, ha egyetlen slot sem éri el a τ_{pos} küszöböt. Ha több cél ugyanazt a slotot választaná, külön ütközésfeloldás (például kölcsönösen egyértelmű hozzárendelés) kell: egyetlen slot ugyanis nem tanítható egyszerre két különböző dobozra. A fejezet kis példája egy cellán belül legfeljebb egy felelős objektumot és ütközésmentes párosítást feltételez; ennél sűrűbb jelenethez slotonkénti osztályfej vagy általánosabb matching szükséges. Ha $M = 0$, akkor $P = I = \emptyset$ és $N = \{1; \dots; T\}$. Ez a szabály a könyv saját, didaktikai célú leírása; a konkrét küszöbértékek architektúránként és cikkenként eltérnek (a küszöbértékeket külön protokollban kell rögzíteni).

Definíció (saját, didaktikai egylépcsős cél). Legyen $S \times S$ a cellarács, cellánként B slot, K osztály. A predikció cellánkénti osztálylogit $\hat{z}_{ij} \in \mathbb{R}^K$, slotonkénti dobozparaméter $\hat{t}_{ijb} \in \mathbb{R}^4$ és objectness logit $\hat{o}_{ijb} \in \mathbb{R}$. A target oldalán $m_{ijb} \in \{0; 1\}$ a matching, $v_{ijb} = \mathbf{1}[(i, j, b) \in P \cup N]$ az objectness-veszteség érvényességi maszkja, $y^{cls}_{ij} \in \{0, \dots, K-1\}$ a felelős cella osztálya, $y^{box}_{ijb} \in \mathbb{R}^4$ a cél dobozparaméter, $N^{cell}_{pos} = |\{(i, j) : g_{ij} = 1\}|$ a felelős cellák száma és $N^{slot}_{pos} = \sum_{i,j,b} m_{ijb}$ a pozitív slotok száma. A teljes cél:

$$L = L_{cls} + L_{box} + L_{obj},$$

ahol

$$L_{cls} = \frac{1}{N^{cell}_{pos}} \sum_{\substack{i,j \\ g_{ij}=1}} \ell_{CE}(\hat{z}_{ij}, y^{cls}_{ij}), \quad \ell_{CE}(z, y) = -\log \mathrm{softmax}(z)_y,$$

$$L_{box} = \frac{\lambda_{coord}}{N^{slot}_{pos}} \sum_{i,j,b} m_{ijb}\, \ell_{reg}(\hat{t}_{ijb}, y^{box}_{ijb}), \quad \ell_{reg}(a, b) = \frac{1}{4}\|a - b\|_2^2,$$

$$L_{obj} = \frac{\lambda_{obj}}{S^2 B} \sum_{i,j,b} v_{ijb}\, \ell_{BCE}(\hat{o}_{ijb}, m_{ijb}), \quad \ell_{BCE}(z, m) = \mathrm{softplus}(z) - mz.$$

A BCE logit-formája a 25.2. Multilabel és hierarchikus cél alfejezetben levezetett stabil alak; a softmax előtti logit itt is megmarad, mert a 7.2. Softmax és többosztályos cross-entropy szerint a könyvtári cross-entropy logitot vár. Az osztálytag a *felelős cellák*, a doboztag a *pozitív slotok* átlaga; a két szám csak akkor egyezik, ha cellánként legfeljebb egy slot pozitív. Az objectness-tag az *összes slot számával normalizált, de csak érvényes (pozitív vagy negatív) slotokat tartalmazó* veszteség. A figyelmen kívül hagyott I slotokra $v_{ijb} = 0$, ezért nem keletkezik gradiens. A különböző nevezőket és az érvényességi maszkot minden implementációban explicit módon kell kezelni. Ha $N^{cell}_{pos} = 0$, akkor $L_{cls} = 0$, és ha $N^{slot}_{pos} = 0$, akkor $L_{box} = 0$ konvencióval élünk; üres pozitív halmaz esetén a cél csak a negatív objectness tagból áll, üres batch esetén pedig a teljes cél definiálatlan.

Állítás (a cél gradiense). A fenti cél komponensenkénti gradiense:

$$\frac{\partial L_{obj}}{\partial \hat{o}_{ijb}} = \frac{\lambda_{obj} v_{ijb}}{S^2 B}(\sigma(\hat{o}_{ijb}) - m_{ijb}),$$

$$\frac{\partial L_{box}}{\partial \hat{t}_{ijb;k}} = \frac{\lambda_{coord}\, m_{ijb}}{2N_{pos}^{slot}} (\hat{t}_{ijb;k} - y_{ijb;k}^{box}),$$

a negatív slotokra pedig $m_{ijb} = 0$ miatt a doboztag gradiense nulla; az osztálytag gradiense a 7.2. Softmax és többosztályos cross-entropy alfejezetből $(\mathrm{softmax}(\hat{z}_{ij}) - e_{y_{ij}^{cls}})/N_{pos}^{cell}$ a felelős cellákra, és nulla máshol.

Bizonyítás. A BCE-re $\frac{d}{dz}\mathrm{softplus}(z) = \sigma(z)$, így $\ell_{BCE}(z, m) = \mathrm{softplus}(z) - mz$ deriváltja $\sigma(z) - m$; az érvényességi maszk v_{ijb} és a $\lambda_{obj}/(S^2 B)$ konstans kiemelhető. Az ℓ_{reg}-re $\frac{1}{4}\sum_k (a_k - b_k)^2$ deriváltja az a_k változó szerint $\frac{1}{2}(a_k - b_k)$, és a szumma csak a pozitív slotokat tartalmazza, ezért a m_{ijb} szorzó kívülről is megjelenik. □

A gradiens alakja egy heurisztikus tervezési motivációt tesz láthatóvá: ha $|N| \gg |P|$, a negatív objectness tagok dominálhatják a lépést, ezért a gyakorlatban λ-súlyozással vagy negatív mintavételezéssel egyensúlyoznak. Ez *heurisztika*, nem bizonyított optimalitási garancia: a gradiens képlete algebrai tény, de hogy a súlyozás minden adatnál jobb modellhez vezet-e, azt csak kísérlettel lehet eldönteni.

26.2.5 5. Eredeti kézi kis példa

Legyen $W = H = 24$, $S = 2$, tehát a négy cella 12×12-es. Egyetlen slot cellánként ($B = 1$), a slot horgonya maga a cella. Egyetlen ground truth doboz:

$$g = (10; 10; 22; 22), \qquad y^{cls} = 1.$$

A jobb alsó cella horgonya $a = (12; 12; 24; 24)$; a többi celláé rendre $(0; 0; 12; 12)$, $(0; 12; 12; 24)$, $(12; 0; 24; 12)$. A kézi példában a horgonyokat a rövidség kedvéért xyxy alakban írjuk; a 2. szakasz középpont–méret jelölése ezzel a jobb alsó cella esetén $a_{cx} = a_{cy} = 18$, $a_w = a_h = 12$, azaz $(18; 18; 12; 12)$. A GT területe $12 \cdot 12 = 144$, a jobb alsó horgony területe szintén 144. A metszet:

$$[12, 22) \times [12, 22), \qquad |\cap| = 100,$$

az unió $144 + 144 - 100 = 188$, tehát

$$\mathrm{IoU} = \frac{100}{188} \approx 0{,}5319.$$

A másik három horgony átfedése nem nulla: a bal felső cellánál $4/(144{+}144{-}4) = 4/284 \approx 0{,}0141$, a bal alsó és jobb felső cellánál pedig egyaránt $20/(144{+}144{-}20) = 20/268 \approx 0{,}0746$. Mindhárom érték a negatív küszöb alatti. A $\tau_{pos} = 0{,}5$, $\tau_{neg} = 0{,}4$ választással:

$$P = \{a_{1,1}\}, \quad N = \{a_{0,0}, a_{0,1}, a_{1,0}\}, \quad I = \emptyset,$$

tehát a matching szerint pontosan egy pozitív slot van, a jobb alsó cella. Ebben a kézi példában — a korábban bemutatott, horgonyközépponthoz viszonyított és logaritmikus szélességet használó Δ-kódolástól eltérően — a középpontot a cella bal felső sarkához és a cellamérethez, a dobozméretet pedig a teljes kép méretéhez normalizáljuk. A dekódolás $b_{cx} = x_{bal} + 12t_x$, $b_{cy} = y_{fels} + 12t_y$, $b_w = 24t_w$, $b_h = 24t_h$. Ezt a kódolást a hozzá tartozó fejjel és targettel együtt kell használni; a

két konvenció nem keverhető össze. A cél dobozparaméter ezzel a cella- és képméret-normalizált kódolással: a GT középpontja $(16, 16)$, a cella bal felső sarka $(12, 12)$, a cellaméret 12, így

$$t_x = t_y = \frac{16-12}{12} = \frac{1}{3}, \qquad t_w = t_h = \frac{12}{24} = 0{,}5.$$

A célvektorok: $m_{1,1,1} = 1$, a többi slotra $m = 0$; $y^{cls}_{1,1} = 1$; a dobozcél a pozitív sloton $(\frac{1}{3}; \frac{1}{3}; \frac{1}{2}; \frac{1}{2})$.

Most legyen a predikció a pozitív sloton

$$\hat{t} = (0{,}3,\ 0{,}3,\ 0{,}6,\ 0{,}4), \qquad \hat{o} = 1{,}2, \qquad \hat{z} = (0{,}2,\ 1{,}8,\ 0{,}7),$$

a három negatív sloton pedig $\hat{o} = -0{,}5$. A $\lambda_{coord} = \lambda_{obj} = 1$ választással számoljuk a három tagot. A doboztag:

$$\ell_{reg} = \frac{1}{4}\left[2\left(0{,}3 - \frac{1}{3}\right)^2 + (0{,}6-0{,}5)^2 + (0{,}4-0{,}5)^2\right] = \frac{1}{4}\left(\frac{2}{900} + 0{,}01 + 0{,}01\right) \approx 0{,}00556.$$

Az objectness BCE a pozitív sloton:

$$\ell_{BCE}(1{,}2; 1) = \ln(1 + e^{1{,}2}) - 1{,}2 \approx 0{,}26328,$$

a negatív sloton:

$$\ell_{BCE}(-0{,}5; 0) = \ln(1 + e^{-0{,}5}) \approx 0{,}47408.$$

Mivel az objektumosság nevezője $S^2 B = 4$:

$$L_{obj} \approx \frac{0{,}26328 + 3 \cdot 0{,}47408}{4} = 0{,}42138.$$

Az osztálytag: $\mathrm{softmax}(0{,}2; 1{,}8; 0{,}7)$ komponensei az e-hatványokból $6{,}04965/9{,}28480 \approx 0{,}65156$, így

$$L_{cls} = -\ln 0{,}65156 \approx 0{,}42838.$$

A teljes cél:

$$L \approx 0{,}42838 + 0{,}00556 + 0{,}42138 = 0{,}85532 \approx 0{,}8553.$$

Az induló kérdésre tehát a válasz: egy pozitív minta keletkezik, mert a GT középpontja a jobb alsó cellában van, és annak horgonya a τ_{pos} küszöb felett illeszkedik; a szomszédos cellák IoU-ja 0,0141, illetve 0,0746, mindkettő a negatív küszöb alatt.

A minimális ellenőrző kód ezt a célösszeget reprodukálja:

```python
# n102_target_demo.py
# Munkakönyvtár: lab/. Futtatott ellenőrző példa; a kimenet a fenti kézi
# számításból származó 0.8553 értéket ellenőrzi.
import numpy as np

def softplus(z):
    return np.log1p(np.exp(-np.abs(z))) + np.maximum(z, 0.0)

def bce_logit(z, m):
    return softplus(z) - m * z

def ce_logit(z, y):
    z = z - np.max(z)
    return -z[y] + np.log(np.exp(z).sum())

m = np.zeros((2, 2, 1), dtype=np.float64)
m[1, 1, 0] = 1.0
g = m.max(axis=2)
N_pos = int(g.sum())
assert m.shape == (2, 2, 1) and set(np.unique(m)) <= {0.0, 1.0}
assert g.shape == (2, 2) and N_pos == 1

t_hat = np.zeros((2, 2, 1, 4))
t_hat[1, 1, 0] = [0.3, 0.3, 0.6, 0.4]
t_gt = np.zeros((2, 2, 1, 4))
t_gt[1, 1, 0] = [1/3, 1/3, 0.5, 0.5]
assert t_hat.shape == t_gt.shape == (2, 2, 1, 4)

o_hat = np.full((2, 2, 1), -0.5)
o_hat[1, 1, 0] = 1.2
assert o_hat.shape == m.shape and np.isfinite(o_hat).all()

z_hat = np.zeros((2, 2, 3))
z_hat[1, 1] = [0.2, 1.8, 0.7]
y_cls = np.zeros((2, 2), dtype=np.int64)
y_cls[1, 1] = 1
assert z_hat.shape == (2, 2, 3) and y_cls.shape == (2, 2)

L_box = (1.0 / N_pos) * np.sum(0.25 * m[..., None] * (t_hat - t_gt)**2)
v = np.ones_like(m)  # a kézi példában nincs figyelmen kívül hagyott slot
L_obj = (1.0 / (2 * 2 * 1)) * np.sum(v * bce_logit(o_hat, m))
Z = z_hat.reshape(-1, 3)
Y = y_cls.reshape(-1)
G = g.reshape(-1).astype(bool)
L_cls = np.mean([ce_logit(z, y) for z, y, ok in zip(Z, Y, G) if ok])
L = L_cls + L_box + L_obj
print(f"L = {L:.4f}")            # várt: 0.8553 (kézi számításból)
assert abs(L - 0.855313) < 5e-4
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU):

```
L = 0.8553
```

26.2.6 6. Labor: architektúra- és outputalak-táblázat, minimális dobozfej

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor átadandó eredménye kettős. Először rögzítjük, hogy ugyanaz a 24×24-es, a függelék A. szakaszában leírt GlyphLab-kép milyen alakokon halad át a két családban; a táblázat a rétegspecifikációból levezetett kimeneti alakokat adja, nem futási mérést.

Elem	Felelősség	Kimeneti alak (N=1)
Backbone	sűrű jellemzőtérkép	$1 \times 64 \times 6 \times 6$
Kétlépcsős RPN	javaslat és objectness	$P \times 4$, P logit
Kétlépcsős detektáló fej	osztály és finomítás	$P \times (K+1)$, $P \times 4$
Egylépcsős grid fej	doboz, objectness, osztály	$1 \times (5B+K) \times S \times S$

Másodszor egyetlen kis dobozfejet építünk. A fej egy 1×1-es konvolúció: a 14.2. Több bemeneti és kimeneti csatorna kernel bankja itt tisztán csatornát kever, térbeli kiterjedés nélkül. A kimenet csatornasorrendje: az első $4B$ csatorna a dobozparaméter, a következő B az objectness logit, az utolsó K az osztálylogit. A táblázat backbone-sora a 24×24-es bemenet 4-es stride-jú jellemzőtérképe ($24/4 = 6$), míg a labor kódjában szereplő $S = 2$ rács 12-es stride-ot jelent; a két szám nem ellentmondás, hanem eltérő konfiguráció: a labor a csatornasorrend és a kimeneti alak szerződését demonstrálja $C = 8$, $S = 2$ értékkel, nem a backbone tényleges 64 csatornájával és 6×6-os térképével. A kód a [D18] irányába mutató teljes TorchVision-finetuning *előtti* lokális kontroll; a TorchVision-bővítés külön labor, és külső előtanított súlyt a főúton nem használunk.

```
# n102_lab_box_head.py
# Munkakönyvtár: lab/. CPU, float32, rögzített seed.
# Futtatás a lab/ mappában.
import torch

torch.manual_seed(0)
C, B, K, S = 8, 1, 3, 2

head = torch.nn.Conv2d(C, B * 4 + B + K, kernel_size=1, bias=True)
head = head.to(dtype=torch.float32, device="cpu")

x = torch.randn(1, C, S, S, dtype=torch.float32, device="cpu")
p = head(x)
assert p.shape == (1, 8, 2, 2), p.shape

box = p[:, :4]          # B slot, 4 dobozparaméter
obj_logit = p[:, 4:5]   # objectness logit
cls_logit = p[:, 5:]    # K osztálylogit, softmax nélkül
obj = torch.sigmoid(obj_logit)

print("p shape:", tuple(p.shape))
print("obj logit min/max:", float(obj_logit.detach().min()), float(obj_logit.detach().max()))
print("obj sigmoid min/max:", float(obj.detach().min()), float(obj.detach().max()))
assert 0.0 <= float(obj.detach().min()) <= float(obj.detach().max()) <= 1.0
# Ellenőrzés: az osztálylogitok összege nem 1 (logit, nem valószínűség),
# az objectness a sigmoid után [0,1]-ben van.
```

Futtatási parancs a `lab/` munkakönyvtárból:

```
python n102_lab_box_head.py
```

Mért kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, `torch.manual_seed(0)`, `torch.get_num_threads() = 10`):

```
p shape: (1, 8, 2, 2)
obj logit min/max: -0.41042354702949524 0.5101099014282227
obj sigmoid min/max: 0.3988105356693268 0.624832272529602
```

Elvárt tulajdonság: a kimeneti alak $(1, 8, 2, 2)$, az objectness sigmoid után a $[0, 1]$ intervallumba esik, az osztálycsatornák pedig nyers logitok maradnak. A kód mindezt assertekkel is ellenőrzi. Az értelmezés: ez a fej még nem detektor, hanem a detektor *predikciós szerződésének* legkisebb ellenőrizhető darabja; a célfüggvény és a matching nélkül nem tanulható.

26.2.7 7. Feladatok

F1. (fogalmi, nehézség 2) Magyarázza el, miben tér el az objectness és az előtérre kondicionált osztályozó egyetlen, „háttér” osztályt is tartalmazó softmax-fejtől, és adjon olyan konkrét esetet, ahol a két tanítási cél eltérő gradienshez vezet. Használja a 25.2. Multilabel és hierarchikus cél BCE/softmax megkülönböztetést.

F2. (levezetési, nehézség 2) Legyen $\tau_{pos} = 0{,}5$, $\tau_{neg} = 0{,}4$, két ground truth doboz, $M = 2$, és négy slot. Az első GT-hez tartozó IoU-listája (0,6; 0,2; 0,45; 0,1), a másodikhoz (0,2; 0,3; 0,1; 0,05). Adja meg a P, N, I halmazokat a matching szabály alapján, és nevezze meg, melyik slot kerül be a teljes lefedés miatt a pozitív halmazba.

F3. (implementációs, nehézség 2) Valósítsa meg NumPyban a háromtagú célfüggvényt úgy, hogy a BCE stabil softplus-formát használjon, és üres pozitív halmaz esetén ($N_{pos}^{cell} = N_{pos}^{slot} = 0$) a $L_{cls} = L_{box} = 0$ elágazást alkalmazza. Bemenet: a fenti kézi példa target- és predikciótensorai; elvárt kimenet $L \approx 0{,}8553$, tolerancia $5 \cdot 10^{-4}$.

F4. (hibakeresési, nehézség 2) Az alábbi matching-kód hibás:

```
pos = {s for s in slots
       if iou[s, best_gt[s]] >= tau_pos}
```

A teljes lefedés hiányzik. Magyarázza el a hiba okát, következményét, és adja meg a javítást, majd olyan ellenőrző tesztet, amelyben minden slot IoU-ja τ_{pos} alatt van, és a javítás után mégis van pozitív minta.

F5. (kísérleti/transzfer, nehézség 3) Tervezzen GlyphLab-alapú egyszerű lokalizációs kísérletet: a meglévő osztályozó CNN utolsó rétege elé egy dobozfejet kell illeszteni, a cél a `metadata.csv` dobozoszlopaiból kódolt normalizált koordináta. Írja le a protokollt: megfigyelési egység, split, baseline, metrika, a tesztelés egyszeri korlátja. Nem kell futtatni; a terv érvényességi határát is adja meg.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

26.2.8 8. Megoldások és indoklás

F1. Az objectness és az előtér-osztályozó faktorizált modell: külön Bernoulli-veszteség tanítja az objektum jelenlétét, és csak a pozitív helyeken használjuk az osztályozási cross-entropyt. A négyosztályos (háttér + három tárgyosztály) softmax ehelyett közös normalizálást alkalmaz, ezért a logitok gradiensei kölcsönösen összefüggnek. Konkrétan pozitív slotnál, nulla objectness-logit mellett a pozitív BCE deriváltja $\sigma(0) - 1 = -1/2$. Ha ehelyett négy nulla logitból induló softmaxot használunk, a valódi tárgyosztály logitjának deriváltja $1/4 - 1 = -3/4$. A különbség nem jelenti azt, hogy egyik módszer általánosan helyesebb lenne: az architektúra, a címkézés és a veszteség közösen határozza meg a tanítási szerződést.

F2. A slotonkénti legnagyobb IoU értéke rendre 0,6, 0,3, 0,45 és 0,1. A küszöb alapján az 1. slot pozitív, a 2. és 4. negatív, a 3. figyelmen kívül hagyandó lenne. A második GT legjobb slotja azonban a 2. (IoU 0,3), ezért a kötelező lefedés a 2. slotot a negatívból a pozitív halmazba teszi át; a végső negatív halmaz csak a 4. slotot tartalmazza. Az eredmény $P = \{1; 2\}$, $N = \{4\}$, $I = \{3\}$. Az I slot objectness-veszteségének érvényességi maszkja nulla; nem szabad automatikusan negatívnak tekinteni.

F3. Az 5. szakasz NumPy-megoldásában $N_{pos}^{cell} = N_{pos}^{slot} = 1$ (a példában $B = 1$, ezért a két szám egyezik), és az eredmény három tagja pontosabban $L_{cls} \approx 0{,}428379$, $L_{box} \approx 0{,}005556$ és $L_{obj} \approx 0{,}421378$; összegük 0,855313, tehát a kért tolerancián belül van. Az üres pozitívhalmaz elágazásának ellenőrzésére az `m` és `g` címkéket állítsuk nullára, a háttérként figyelembe vett slotok `v` maszkját egyre: ekkor a megoldásban `L_cls = L_box = 0.0`, és csak az érvényes slotokra átlagolt `softplus(o_hat)` marad. Ha egyes slotok figyelmen kívül hagyandók, azok `v` értéke nulla.

F4. A hibás kód csak a pozitív IoU-küszöböt vizsgálja, így egy nehezen illeszkedő GT dobozhoz egyetlen pozitív slot sem kerülhet. A javítás előtt minden célhoz válasszuk ki a legjobb IoU-jú slotot, majd képezzük a két feltétel unióját: forced_pos $= \{\arg\max_j \mathrm{IoU}(G_i, \mathrm{slot}_j) : i = 1, \ldots, M\}$, ahol M a ground truth dobozok száma, és döntetlennél a legkisebb indexű slotot választjuk; a végső pozitív halmaz `pos = threshold_pos | forced_pos`. Például egyetlen GT és a `iou = np.array([[0.1], [0.2], [0.3]])` tömb esetén, $\tau_{pos} = 0{,}5$ mellett a küszöbös rész üres, de `forced_pos = {2}` (nullától indexelve), így a javított pozitív halmaz nem üres. Több GT esetén ellenőrizni kell, hogy a kényszerített kiválasztások nem ugyanazt a slotot igénylik-e (a $\max_j$ GT-nként külön fut, ezért ütközhet); ütközéskor egyedi hozzárendelési szabály (például magyar algoritmus vagy előre rögzített prioritás) szükséges.

F5. A kísérlet alapegysége a `scene_id`: két nézet ugyanabból a jelenetből nem kerülhet eltérő splitbe. A bemenet kizárólag a kép; a `metadata.csv` dobozadatait célként használjuk. A dobozfejet a CNN törzse után, az osztályozó fejjel párhuzamosan illesztjük a közös jellemzőtérképre (nem az utolsó lineáris réteg elé). A célkódolás a 2. szakaszban rögzített normalizált középpont-eltolás és log-méret: $\Delta_{cx} = (g_{cx} - a_{cx})/a_w$, $\Delta_{cy} = (g_{cy} - a_{cy})/a_h$, $\Delta_w = \log(g_w/a_w)$, $\Delta_h = \log(g_h/a_h)$, ahol az anchor a cella (vagy a train medián doboza); a fej kimenete és a target ugyanezt a kódolást használja, a visszafejtés pedig az inverz leképezés. A splitet a tanítás előtt rögzítjük, a dobozokat a megadott félig nyílt xyxy konvenció szerint dekódoljuk, a választott fej és célkódolás egyezését alak- és geometriai próbákkal ellenőrizzük. Baseline lehet a train alapján becsült, osztályonkénti medián doboz. A modellválasztás train/val adaton történik, a rögzített modell test-IoU-ját egyszer mérjük; a jelenetekhez kötődő függőséget csoportszintű bizonytalanságbecslésben is megőrizzük. Az eredmény a GlyphLab mesterséges alakzatain érvényes, természetes képi lokalizációra nem általánosítható automatikusan.

26.2.9 Következő alkalmazás és felhasznált források

A 26.3. NMS, confidence és AP egység a detektor kimenetének utófeldolgozását tárgyalja: az itt rögzített doboz-, objectness- és osztálykimenetből ott lesznek küszöbölt, NMS-sel szűrt detekciók és AP-vel értékelt eredmények.

- [P12] Ren et al. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.* https://arxiv.org/abs/1506.01497.
- [P13] Redmon et al. (2015). *You Only Look Once: Unified, Real-Time Object Detection.* https://arxiv.org/abs/1506.02640. A rekord a 2015-ös arXiv-preprintet rögzíti; ugyanennek a dolgozatnak CVPR 2016-os konferenciaváltozata is létezik.
- [D18] PyTorch contributors (2026). *TorchVision Object Detection Finetuning Tutorial.* PyTorch 2.14 documentation. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html. Hozzáférés: 2026-10-30.

Az SSD eredeti cikke (Liu et al., 2016) nem szerepel a fejezet forráskatalógusában, ezért a szöveg az SSD családra csak családszintű utalást ad, részletes képletet nem.

26.3 NMS, confidence és AP

26.3.1 1. Cél, előismeret és induló kérdés

A 26.2. Egy- és kétlépcsős detektor egységben a detektor nyers kimenetéről beszéltünk: régió- vagy anchor-alapú predikciók, osztálylogitok és dobozkoordináták együttese. A [P12] és [P13] által képviselt két detektorcsalád kimenete ugyanezt a nyers predikcióhalmazt adja, amelyet most a publikálható detekciós lista felé viszünk tovább. Most azt vizsgáljuk, mi történik a modell kimenete és a publikálható detekciós lista között. Három fogalmat választunk szét: a modell *confidence score*-ját, a *Non-Maximum Suppression* (NMS) elnevezésű posztprocesszt és a benchmark-összegző *Average Precision* (AP) metrikát. Az első a modellé, a második a predikciók ritkításának algoritmusa, a harmadik a teljes tesztkészletre vonatkozó értékelési konvenció. A kettő nem mosható össze: egy jól kalibrált score nem teszi feleslegessé az NMS-t, és egyetlen score-küszöbön mért precision nem azonos az AP-vel.

Előismeret: az egy- és kétlépcsős detektorok felelősségi körei (26.2. Egy- és kétlépcsős detektor), a doboz mint félig nyílt `xyxy` intervallum és az IoU alapgondolata (26.1. Dobozkoordináta, IoU és lokalizációs loss), valamint a metrika és a kísérleti napló elválasztása (25.3. Kis adat és feature-alapú baseline). Visszautalunk a címkezaj problémájára is (22.3. Zajos és hiányos címke), mert a ground truth (GT) doboz pontatlansága közvetlenül befolyásolja az AP-t. A 14.3. Paraméterszám, MAC és aktivációmemória költségmodellje pedig releváns marad: az NMS páronkénti IoU-számítása a modell kiértékelésén túli költség.

Induló kérdés: ha a modell ugyanarra az objektumra két majdnem egyforma, egyaránt magas score-ú dobozt ad, miért nem lesz ettől kétszer akkora a mért teljesítménye?

26.3.2 2. Definíciók: doboz, pontszám, IoU, NMS, score threshold

Doboz (bounding box). Egy detekciós doboz négy koordinátából áll:

$$b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \in [0, W] \times [0, H] \times [0, W] \times [0, H],$$

ahol W, H a kép szélessége és magassága ugyanabban a koordinátarendszerben, és $x_{\min} < x_{\max}$, $y_{\min} < y_{\max}$. A koordináták lehetnek pixelértékek vagy $[0, 1]$-re normalizált értékek, de egy kiértékelési egységen belül egyetlen konvenciót kell használni. A függelék A. szakaszában leírt GlyphLab szerződéséhez hasonlóan a dobozt félig nyílt intervallumként értelmezzük: az $x_{\min} \le x < x_{\max}$, $y_{\min} \le y < y_{\max}$ tartomány alkotja a dobozt. Területe:

$$A(b) = (x_{\max} - x_{\min})(y_{\max} - y_{\min}).$$

A detekciós kimenetet egy kötegben $P \in \mathbb{R}^{B \times 4}$ mátrixként és $s \in \mathbb{R}^{B}$ score-vektorként ábrázoljuk; B a nyers detekciók száma. A score tartománya gyakran $[0, 1]$, de az NMS és az AP definíciójához elegendő, hogy a score-ok rendezhető valós számok legyenek.

Confidence score. A *confidence score* (biztonsági pontszám) a modell által egy detekcióhoz rendelt, rendezhető valós szám, amely a detekció erősségét fejezi ki; lehet logit, softmax-érték vagy több tényező szorzata, ezért nem feltétlenül valószínűség. A score *jól kalibrált*, ha az s érték közelítőleg annak a valószínűsége, hogy a detekció helyes (például a detekciók TP-aránya az adott score-sávban); a kalibráltságot külön kalibrációs adaton kell ellenőrizni, és nem következik a score nagy értékéből.

Intersection over Union. Két doboz, a és b IoU-ja:

$$\mathrm{IoU}(a,b) = \frac{A(a \cap b)}{A(a \cup b)} = \frac{A(a \cap b)}{A(a) + A(b) - A(a \cap b)},$$

ahol a metszet szélessége $\max(0, \min(x_{\max}, x'_{\max}) - \max(x_{\min}, x'_{\min}))$, magassága analóg módon számolódik. Ha $A(a) + A(b) - A(a \cap b) > 0$, a képlet direkt. Az üres-uniós szélsőesethez külön szabály kell: ha $A(a) = A(b) = 0$ és $a = b$, akkor $\mathrm{IoU}(a,b) = 1$; ha a két elfajuló doboz különbözik, $\mathrm{IoU}(a,b) = 0$; ha pontosan az egyik területe nulla, a metszet területe nulla, tehát az IoU is nulla. Gyakorlati detektor-kimenetben a pozitív területű doboz a jellemző eset.

Score threshold. A *score threshold* $\tau_s \geq 0$ egy üzemeltetési szűrő: csak azokat a detekciókat tartjuk meg, amelyekre $s_i \geq \tau_s$. Ez a szűrő a modell kimenetének felhasználói értelmezését szolgálja, nem a benchmark-metrika része. Az AP-számítás ezzel szemben a rendezett detekciós listát feldolgozza anélkül, hogy egyetlen τ_s-nél elvágná azt; a küszöb szerepét a rendezett sorrend összes lehetséges prefixe veszi át. Ezért az AP a score-rangsor összegzője, míg a score threshold egy adott működési pontot választ.

NMS (Non-Maximum Suppression). Legyen adott $P \in \mathbb{R}^{B\times 4}$, $s \in \mathbb{R}^B$ és $\tau \in [0,1]$ IoU-küszöb. Az NMS a következő determinisztikus algoritmus:

1. Rendezzük a dobozokat csökkenő score szerint. Azonos score esetén a nagyobb területű doboz kerül előrébb; ha a terület is egyenlő, a kisebb index a korábbi. Így a rendezés teljes.
2. A rendezett lista elejétől haladva minden még el nem nyomott dobozt kiválasztunk, és minden további, még el nem nyomott dobozt törlünk, amelyre a kiválasztottal számított IoU legalább τ.
3. A kiválasztott indexek halmaza az NMS kimenete, $\mathcal{K} \subseteq \{1, \ldots, B\}$.

A $B = 0$ bemenetre a kimenet üres. Az NMS nem változtat a dobozokon és a score-okon; csak eldönti, mely predikciókat közöljük. A τ választása kompromisszum: a magas IoU-küszöb több, egymástól alig eltérő duplikátumot hagyhat meg, az alacsony küszöb pedig valódi, egymáshoz közel álló objektumokat nyomhat el. Az NMS a *duplicate detection* (duplikátumészlelés) kezelésének egyik eszköze, de nem az egyetlen forrása a duplikátumoknak: ahogy a matching-definícióban látni fogjuk, az NMS után is lehet két detekció ugyanarra a GT-re.

Illesztés, TP/FP/FN. Legyen M ground truth doboz egy adott képen, $G \in \mathbb{R}^{M\times 4}$, és legyen τ_{IoU} a kiértékelési IoU-küszöb. A detekciókat csökkenő score szerint dolgozzuk fel. A i-edik detekció akkor *true positive* (TP), ha létezik még hozzá nem rendelt GT-doboz, amellyel az IoU legalább τ_{IoU}; ekkor a legnagyobb IoU-jú ilyen GT-t választjuk, és foglaltnak jelöljük. Ha egy GT-hez már korábban egy detekciót rendeltünk, a későbbi illeszkedő detekció *false positive* (FP). Ha nincs elérhető, kellően átfedő GT, a detekció FP. Egy GT, amelyhez egyetlen detekciót sem rendeltünk, *false negative* (FN). Ez a mohó párosítás az itt használt all-point AP-változat alapja; a klasszikus PASCAL VOC matching szabálya (Everingham et al., 2010) ettől eltér (lásd alább), ezért a két protokollt nem szabad összekeverni. A tie eseteket itt a legkisebb GT-index választásával oldjuk fel.

Precision, recall, AP. A feldolgozás k-adik lépése után $\mathrm{TP}_k, \mathrm{FP}_k$ az addigi osztályozás, és

$$p_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k + \mathrm{FP}_k}, \qquad r_k = \frac{\mathrm{TP}_k}{M}.$$

A matching protokollt meg kell nevezni: a kód csak a *még szabad* GT-k között választ maximális IoU-t, ami az Ön „all-point” AP-jához tartozó saját változat. A klasszikus VOC-matching először *minden* GT között választ maximumot, és ha a legjobb GT már használt, a detekció duplikátum (FP); átfedő két GT esetén a két szabály eltérhet. A precision-interpoláció és a hozzárendelési

protokoll külön választás. Az $M = 0$ esethez külön konvenció kell; a jelen egységben feltesszük, hogy a kiértékelt képeken $M > 0$. A PR-görbe a (r_k, p_k) pontokból áll. Az általunk használt *all-point interpolált AP* a következő:

$$\mathrm{AP}_{\mathrm{all}} = \sum_{k=1}^{D} (r_k - r_{k-1})\, \hat{p}(r_k), \qquad r_0 = 0, \qquad \hat{p}(r_k) = \max_{j \geq k} p_j,$$

ahol D a feldolgozott detekciók száma. Az azonos recall-értékű pontokat nem vonjuk össze: az ilyen lépések hossza $r_k - r_{k-1} = 0$, ezért az AP-értéket nem befolyásolják, a görbe és a jobb oldali maximum szempontjából viszont megőrizzük őket. A jobb oldali maximális interpoláció azt jelenti, hogy a recall r_k pontjához tartozó precisiont a k-adiktól jobbra eső összes precision maximumával helyettesítjük. A szokásos 11 pontos VOC-konvenció (Everingham et al., 2010) ettől különbözik: ott a recall-tengely 0; 0,1; ...; 1 pontjaiban vett interpolált precisioneket átlagoljuk. A publikációban mindig jelölni kell, melyik konvenciót használtuk; a numerikus értékek konvenciófüggők. A *duplicate detection* büntetése itt lép be: a második, ugyanarra a GT-re illeszkedő detekció FP-ként csökkenti a precisiont anélkül, hogy a recallt növelné.

26.3.3 3. Kézi levezetés: NMS és AP számpéldái

26.3.3.1 NMS kézi példa

Legyen egy 40×40-es kép, öt nyers detekció a következő `xyxy` dobozokkal és score-okkal:

- 0. index: $A = (0; 0; 5; 5)$, $s = 0{,}6$;
- 1. index: $B = (0; 0; 10; 10)$, $s = 0{,}9$;
- 2. index: $C = (20; 20; 30; 30)$, $s = 0{,}5$;
- 3. index: $D = (2; 2; 12; 12)$, $s = 0{,}85$;
- 4. index: $E = (1; 1; 9; 9)$, $s = 0{,}9$.

Területek: $A(A) = 25$, $A(B) = 100$, $A(C) = 100$, $A(D) = 100$, $A(E) = 64$. A rendezés a score, majd a terület, majd az index szabálya szerint: B (0,9, terület 100), E (0,9, terület 64), D (0,85), A (0,6), C (0,5). Legyen $\tau = 0{,}5$.

- Első kiválasztott: B. $\mathrm{IoU}(B, E)$: metszet $(1, 1, 9, 9)$, területe 64; unió $100 + 64 - 64 = 100$; $\mathrm{IoU} = 0{,}64 \geq 0{,}5$, tehát E törlődik. $\mathrm{IoU}(B, D)$: metszet $(2, 2, 10, 10)$, területe 64; unió $100 + 100 - 64 = 136$; $\mathrm{IoU} = 0{,}4706 < 0{,}5$, tehát D megmarad. $\mathrm{IoU}(B, A) = 25/100 = 0{,}25 < 0{,}5$, A megmarad. $\mathrm{IoU}(B, C) = 0$.
- Második a rendezett listában E, de már töröltük, kihagyjuk.
- Harmadik: D kiválasztott. $\mathrm{IoU}(D, A)$: metszet $(2, 2, 5, 5)$, területe 9; unió $100+25-9 = 116$; $\mathrm{IoU} = 0{,}0776 < 0{,}5$, A megmarad. $\mathrm{IoU}(D, C) = 0$.
- Negyedik: A kiválasztott.
- Ötödik: C kiválasztott.

Az NMS kimenete $\{B, D, A, C\}$, azaz az eredeti indexekkel $\{1; 3; 0; 2\}$. Figyeljük meg: az azonos score-ú E kiesett, mert a nagyobb területű B előbb dolgozódott fel; ez a tie-break (döntetlenfeloldó) szabály következménye. Az NMS tehát nem egyszerűen „a legmagasabb score-ú doboz mindig nyer”, hanem a térbeli elnyomás sorrendjétől függ.

26.3.3.2 AP kézi példa

Legyen egy $(H, W) = (130, 160)$ méretű képen három GT-doboz:

$$g_1 = (10, 10, 50, 50), \quad g_2 = (100, 20, 150, 60), \quad g_3 = (30, 80, 80, 120).$$

Hat detekció, már score szerint rendezve (tegyük fel, hogy NMS után is így maradtak, például 0,9-es NMS-IoU-küszöb mellett; a második szándékosan egy megmaradó duplikátum g_1-re):

k	Detekció	Score	IoU a GT-kkel, illesztés
1	$(12; 12; 48; 48)$	0,95	g_1: 0,81 $\rightarrow$ TP
2	$(11; 10; 49; 52)$	0,90	g_1: 0,91, de g_1 már foglalt $\rightarrow$ FP
3	$(100; 22; 149; 59)$	0,85	g_2: 0,91 $\rightarrow$ TP
4	$(5; 5; 20; 20)$	0,80	legjobb g_1: $100/(1600 + 225 - 100) \approx 0{,}058 < 0{,}5 \rightarrow$ FP
5	$(29; 81; 79; 119)$	0,70	g_3: 0,91 $\rightarrow$ TP
6	$(0; 0; 9; 9)$	0,60	nincs átfedő GT $\rightarrow$ FP

A $\tau_{\text{IoU}} = 0{,}5$ melletti lépésenkénti értékek:

1. $\text{TP} = 1, \text{FP} = 0$, így $p = 1$, $r = 1/3$.
2. $\text{TP} = 1, \text{FP} = 1$, így $p = 1/2$, $r = 1/3$.
3. $\text{TP} = 2, \text{FP} = 1$, így $p = 2/3$, $r = 2/3$.
4. $\text{TP} = 2, \text{FP} = 2$, így $p = 1/2$, $r = 2/3$.
5. $\text{TP} = 3, \text{FP} = 2$, így $p = 3/5 = 0{,}6$, $r = 1$.
6. $\text{TP} = 3, \text{FP} = 3$, így $p = 1/2$, $r = 1$.

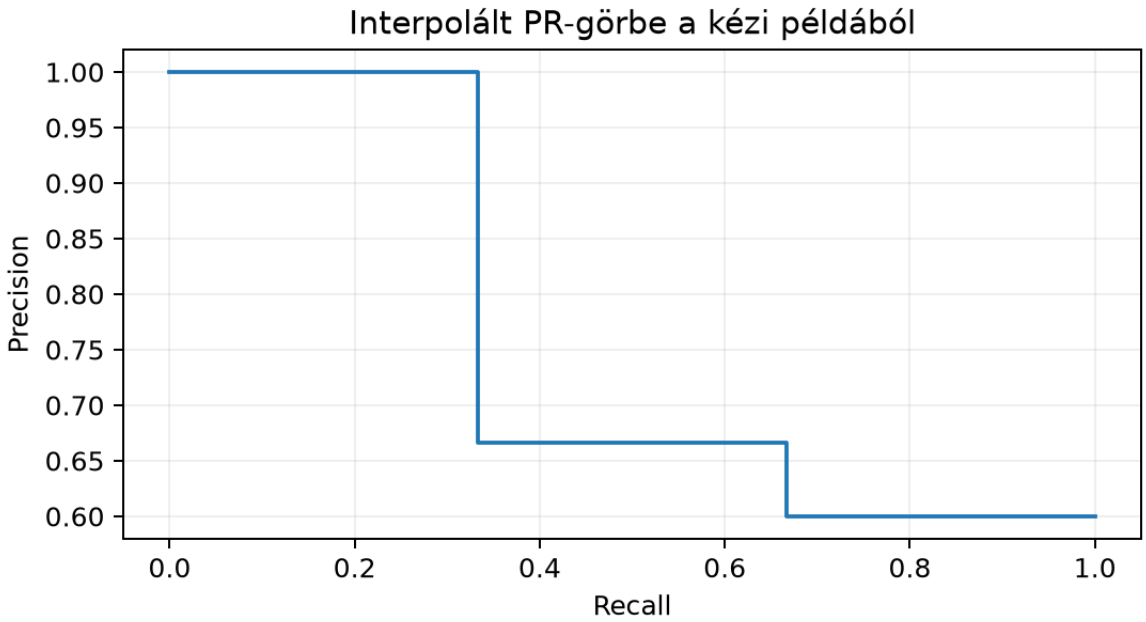
Az all-point képletben a jobb oldali maximumok:

- $r = 1/3$-hoz: $\max\{1; 1/2; 2/3; 1/2; 3/5; 1/2\} = 1$,
- $r = 2/3$-hoz: $\max\{2/3; 1/2; 3/5; 1/2\} = 2/3$,
- $r = 1$-hez: $\max\{3/5; 1/2\} = 3/5$.

Így

$$\text{AP}_{\text{all}} = \frac{1}{3} \cdot 1 + \left(\frac{2}{3} - \frac{1}{3}\right) \cdot \frac{2}{3} + \left(1 - \frac{2}{3}\right) \cdot \frac{3}{5} = \frac{1}{3} + \frac{2}{9} + \frac{1}{5} = \frac{15 + 10 + 9}{45} = \frac{34}{45} \approx 0{,}7556.$$

A kettes számú duplikátum rontotta a precisiont: a recall nem nőtt, de az FP eggyel növelte a nevezőt. Ha a 11 pontos VOC-interpolációval számolunk, az interpolált görbe pontjai: $r \in [0, 1/3]$-ra $p = 1$; $r \in (1/3, 2/3]$-ra $p = 2/3$; $r \in (2/3, 1]$-ra $p = 3/5$. A 0; 0,1; 0,2; 0,3 ponthoz $p = 1$, a 0,4; 0,5; 0,6 ponthoz $p = 2/3$, a 0,7; 0,8; 0,9; 1,0 ponthoz $p = 3/5$ tartozik, így az átlag $(4 \cdot 1 + 3 \cdot (2/3) + 4 \cdot (3/5))/11 = (4 + 2 + 2{,}4)/11 \approx 0{,}7636$. A két konvenció közötti különbség 0,008; nem hiba, hanem a definíciók eltérése.



26.2. ábra: Interpolált PR-görbe a kézi példából

A 3 GT / 6 detekciós kézi példából számolt all-point interpoláció; nem mérés.

26.3.4 4. Minimális kód és ellenőrizhető munkamenet

A következő NumPy-implementáció CPU-n fut, `float32` dtype-tal, véletlen komponens nélkül, ezért seed-re nincs szükség. A munkakönyvtár: `lab`.

```
# lab/n103_nms_lab.py
import numpy as np


def box_area(boxes: np.ndarray) -> np.ndarray:
    boxes = np.asarray(boxes, dtype=np.float32)
    return (boxes[..., 2] - boxes[..., 0]) * (boxes[..., 3] - boxes[..., 1])


def compute_iou(a: np.ndarray, b: np.ndarray) -> float:
    ix1 = float(max(a[0], b[0]))
    iy1 = float(max(a[1], b[1]))
    ix2 = float(min(a[2], b[2]))
    iy2 = float(min(a[3], b[3]))
    iw = max(0.0, ix2 - ix1)
    ih = max(0.0, iy2 - iy1)
    inter = iw * ih
    aa = float((a[2] - a[0]) * (a[3] - a[1]))
    ba = float((b[2] - b[0]) * (b[3] - b[1]))
    union = aa + ba - inter
    if union > 0.0:
        return inter / union
    if aa == 0.0 and ba == 0.0:
        return 1.0 if np.array_equal(a, b) else 0.0   # pontos egyenlőség; két különböző
        ↪ elfajuló doboz IoU-ja 0
    return 0.0
```

```
def nms(boxes: np.ndarray, scores: np.ndarray, iou_threshold: float = 0.5) -> np.ndarray:
    boxes = np.asarray(boxes, dtype=np.float32)
    scores = np.asarray(scores, dtype=np.float32)
    B = boxes.shape[0]
    areas = box_area(boxes)
    order = sorted(range(B), key=lambda i: (-float(scores[i]), -float(areas[i]), i))
    keep = []
    suppressed = np.zeros(B, dtype=bool)
    for i in order:
        if suppressed[i]:
            continue
        keep.append(i)
        for j in order:
            if suppressed[j] or j == i:
                continue
            if compute_iou(boxes[i], boxes[j]) >= iou_threshold:
                suppressed[j] = True
    return np.array(keep, dtype=np.int64)


def match_detections(det_boxes, det_scores, gt_boxes, iou_threshold=0.5):
    det_boxes = np.asarray(det_boxes, dtype=np.float32)
    det_scores = np.asarray(det_scores, dtype=np.float32)
    gt_boxes = np.asarray(gt_boxes, dtype=np.float32)
    D = det_boxes.shape[0]
    M = gt_boxes.shape[0]
    areas = box_area(det_boxes)
    order = sorted(range(D), key=lambda i: (-float(det_scores[i]), -float(areas[i]), i))
    tp = np.zeros(D, dtype=bool)
    assigned = np.zeros(M, dtype=bool)
    for i in order:
        best_j = -1
        best_iou = -1.0
        for j in range(M):
            if assigned[j]:
                continue
            iou = compute_iou(det_boxes[i], gt_boxes[j])
            if iou >= iou_threshold and iou > best_iou:
                best_iou = iou
                best_j = j
        if best_j >= 0:
            tp[i] = True
            assigned[best_j] = True
    return tp, order


def all_point_ap(tp, order, n_gt):
    tp_cum = 0
    fp_cum = 0
    precisions = []
    recalls = []
    for idx in order:
        if tp[idx]:
            tp_cum += 1
        else:
            fp_cum += 1
        precisions.append(tp_cum / (tp_cum + fp_cum))
        recalls.append(tp_cum / n_gt)
    ap = 0.0
    prev = 0.0
    for k, r in enumerate(recalls):
        if r > prev:
```

```
            ap += (r - prev) * max(precisions[k:])
            prev = r
    return ap


def main():
    nms_boxes = np.array([
        [0, 0, 5, 5],
        [0, 0, 10, 10],
        [20, 20, 30, 30],
        [2, 2, 12, 12],
        [1, 1, 9, 9],
    ], dtype=np.float32)
    nms_scores = np.array([0.6, 0.9, 0.5, 0.85, 0.9], dtype=np.float32)
    keep = nms(nms_boxes, nms_scores, 0.5)
    print("NMS keep (eredeti indexek):", keep.tolist())
    print("Ures NMS:", nms(np.empty((0, 4), np.float32), np.empty(0, np.float32)).tolist())

    gt = np.array([
        [10, 10, 50, 50],
        [100, 20, 150, 60],
        [30, 80, 80, 120],
    ], dtype=np.float32)
    det_boxes = np.array([
        [12, 12, 48, 48],
        [11, 10, 49, 52],
        [100, 22, 149, 59],
        [5, 5, 20, 20],
        [29, 81, 79, 119],
        [0, 0, 9, 9],
    ], dtype=np.float32)
    det_scores = np.array([0.95, 0.90, 0.85, 0.80, 0.70, 0.60], dtype=np.float32)
    tp, order = match_detections(det_boxes, det_scores, gt, 0.5)
    ap = all_point_ap(tp, order, gt.shape[0])
    print("All-point AP:", ap)
    print("TP sorrend szerint:", [int(tp[i]) for i in order])
    assert keep.tolist() == [1, 3, 0, 2]
    assert nms(np.empty((0, 4), np.float32), np.empty(0, np.float32)).size == 0
    assert abs(ap - 0.7555555555555555) < 1e-12
    assert [int(tp[i]) for i in order] == [1, 0, 1, 0, 1, 0]


if __name__ == "__main__":
    main()
```

A futtatás parancsa: `python n103_nms_lab.py`. Az alábbi munkamenet-output a megadott környezetben mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a számítás determinisztikus, ezért seed nem kell):

```
NMS keep (eredeti indexek): [1, 3, 0, 2]
Ures NMS: []
All-point AP: 0.7555555555555555
TP sorrend szerint: [1, 0, 1, 0, 1, 0]
```

Ha az ellenőrző futtatás ettől 10^{-6} abszolút hibánál jobban eltér, akkor vagy a kód, vagy a közölt elvárt érték hibás; a kettőt a kézi levezetéssel kell összevetni. Az AP-érték kerekítve 0,7556.

26.3.4.1 Kiterjesztés: több kép, több osztály, mAP

A fenti `match_detections` és `all_point_ap` szándékosan **egyképes, egyosztályos didaktikai példa**: nem kap `image_id`-t és osztálycímkét. Valódi detektálási kiértékelésnél egy detekció

csak azonos képű **és** azonos osztályú GT-hez illeszthető, egy osztály AP-jét pedig az összes kép detekcióinak **globális** score-sorrendjéből kell számolni, nem a képenkénti AP-k átlagaként. A **mAP** az osztályonkénti AP-k átlaga.

Három AP-konvenció él egymás mellett, és a számszerű eredmények nem összevethetők:

- **VOC all-point:** a fenti, jobb oldali maximummal interpolált görbe alatti terület;
- **VOC 11-point:** az interpolált precision átlaga a $0; 0{,}1; \ldots; 1$ recall-pontokban (lásd a kézi példát);
- **COCO AP:** 101 recall-pontos interpoláció, az eredmény tíz IoU-küszöbre ($0{,}50; 0{,}55; \ldots; 0{,}95$) átlagolva, osztályonként, majd az osztályokra; a `pycocotools` a detekciókat és a GT-ket kép–osztály páronként illeszti, de az osztály precision–recall görbéjét az összes kép detekcióiból állítja össze ([D22]).

Az **NMS** szintén konvenciófüggő: *osztályonkénti* NMS-nél csak azonos osztályú dobozok nyomják el egymást (a detektorok szokásos alapesete), *class-agnostic* NMS-nél bármely két átfedő doboz; az utóbbi két különböző osztályú, egymást átfedő objektum egyikét eldobhatja. A választott változatot a kiértékelési jegyzőkönyvben rögzíteni kell.

```python
# lab/n103_map.py - többképes, többosztályos AP és mAP (az n103_nms_lab.py függvényeire épül)
import numpy as np
from n103_nms_lab import compute_iou


def class_ap(dets, gts, cls, iou_threshold=0.5):
    """Egy osztály AP-je az összes képen: a detekciók globális score-sorrendben,
    illesztés csak azonos image_id-jú és osztályú GT-hez (all-point interpoláció)."""
    d = sorted([x for x in dets if x["cls"] == cls], key=lambda x: -x["score"])
    g = [x for x in gts if x["cls"] == cls]
    used = [False] * len(g)
    tp = []
    for det in d:
        best, best_iou = -1, -1.0
        for j, gt in enumerate(g):
            if used[j] or gt["image_id"] != det["image_id"]:
                continue
            iou = compute_iou(det["box"], gt["box"])
            if iou >= iou_threshold and iou > best_iou:
                best, best_iou = j, iou
        if best >= 0:
            used[best] = True
        tp.append(best >= 0)
    tp = np.array(tp, dtype=float)
    prec = np.cumsum(tp) / np.arange(1, len(tp) + 1)
    rec = np.cumsum(tp) / len(g)
    ap, prev = 0.0, 0.0
    for k in range(len(tp)):
        ap += (rec[k] - prev) * prec[k:].max()          # jobb oldali maximum
        prev = rec[k]
    return ap


def mean_ap(dets, gts, iou_threshold=0.5):
    classes = sorted({g["cls"] for g in gts})
    aps = {c: class_ap(dets, gts, c, iou_threshold) for c in classes}
    return aps, float(np.mean(list(aps.values())))


def box(x, y, w=10, h=10):
    return np.array([x, y, x + w, y + h], dtype=np.float32)
```

```
# Két kép, két osztály.
gts = [{"image_id": 0, "cls": 0, "box": box(0, 0)},
       {"image_id": 0, "cls": 1, "box": box(30, 30)},
       {"image_id": 1, "cls": 0, "box": box(0, 0)},
       {"image_id": 1, "cls": 0, "box": box(50, 50)}]
dets = [{"image_id": 0, "cls": 0, "score": 0.9, "box": box(1, 1)},      # TP
        {"image_id": 0, "cls": 0, "score": 0.8, "box": box(60, 0)},     # FP
        {"image_id": 1, "cls": 0, "score": 0.7, "box": box(0, 1)},      # TP
        {"image_id": 1, "cls": 0, "score": 0.6, "box": box(51, 50)},    # TP
        {"image_id": 0, "cls": 1, "score": 0.5, "box": box(30, 31)},    # TP
        {"image_id": 1, "cls": 1, "score": 0.95, "box": box(0, 0)}]     # FP: más osztály
aps, m = mean_ap(dets, gts)
print("osztályonkénti AP:", {c: round(float(a), 4) for c, a in aps.items()}, " mAP:", round(m,
↪  4))

# A 0. osztály képenkénti AP-jeinek átlaga nem a globális osztály-AP.
per_image = [class_ap([d for d in dets if d["image_id"] == i],
                      [g for g in gts if g["image_id"] == i], 0) for i in (0, 1)]
print("0. osztály képenként:", [round(float(a), 4) for a in per_image],
      " átlaguk:", round(float(np.mean(per_image)), 4), " globális:", round(float(aps[0]), 4))
assert abs(aps[0] - 5/6) < 1e-12
assert abs(aps[1] - 0.5) < 1e-12
assert abs(m - (5/6 + 0.5) / 2) < 1e-12
assert per_image == [1.0, 1.0]
```

A megadott környezetben mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU):

```
osztályonkénti AP: {0: 0.8333, 1: 0.5}  mAP: 0.6667
0. osztály képenként: [1.0, 1.0]  átlaguk: 1.0  globális: 0.8333
```

A 0. osztály mindkét képen külön-külön tökéletes AP-t kap (a hibás detekció az első képen a helyes után jön), a globális rangsorban viszont a 0,8 score-ú FP megelőzi a második kép két TP-jét, ezért a globális osztály-AP $\frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 0{,}75 + \frac{1}{3} \cdot 0{,}75 = 0{,}8333$. A képenkénti AP-k átlaga tehát általában nem a benchmark-AP. Az 1. osztály AP-je 0,5: a második képen adott, más osztályú GT-re eső 0,95-ös detekció FP, mert az illesztés osztályon belül történik.

26.3.5 5. Konkrét labor és eredményértelmezés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja annak ellenőrzése, hogy a tiszta Python/Numpy NMS ugyanazt a sorrendet és elnyomást adja, mint a kézi definíció. A laborprotokoll:

1. **Input:** a fenti `nms_boxes` és `nms_scores` ötöse, $\tau = 0{,}5$; külön üres bemenet; az AP kézi példa GT- és detekciós dobozai.
2. **Futtatás:** `cd lab && python n103_nms_lab.py`, CPU, `float32`, seed nem értelmezett.
3. **Ellenőrzés:** az NMS kimenet pontosan `[1,3,0,2]`, az üres `[]`, az AP 0,7555555555555555; a dobozok és score-ok `float32`-ben tárolódnak, de az AP-összegzés Python float (`float64`) értékekkel fut. Az azonos score-jú B és E közül E törlődik, mert a nagyobb területű B előbb került sorra; ez a területi tie-break szándékolt következménye.
4. **Értelmezés:** a labor azt mutatja, hogy a teljes pipeline három része elkülönül: a `det_scores` a modell kimenete; az `nms` a posztprocessz, amely nem változtat a score-on; az `all_point_ap` a matching után a teljes rangsort összegzi. A hatodik, alacsony score-ú FP nem rontja az AP-t, mert az interpoláció a későbbi pontoknál a maximális precisiont használja; a második, duplikált detekció viszont rontja, mert a recall növelése nélkül növeli az FP-ket.

Ez a labor nem teljes detektor-értékelés, hanem a definíciók numerikus ellenőrzése. A TorchVision-alapú detektálási labor külön egység; jelen lecke a [D18] tutorialra implementációs állítást nem épít.

26.3.6 6. Feladatok

F1. (fogalmi, nehézség 1, előfeltétel: 2. szakasz) Definiálja az IoU-t egy pozitív területű és egy elfajuló dobozra. Miért van szükség a félig nyílt `xyxy` konvenció megkülönböztetésére a területszámításban? Adjon meg egy párt, ahol a metszet üres, de a két doboz koordinátái átfedni látszanak a zárt intervallum-értelmezés szerint.

F2. (levezetési/bizonyítási, nehézség 2, előfeltétel: az AP kézi példa) Igazolja, hogy az all-point AP invariáns a score-vektor minden szigorúan monoton növekvő transzformációjára. Számítsa ki a 11 pontos VOC-AP-t az AP kézi példa adataiból, minden köztes lépéssel.

F3. (implementációs, nehézség 2, előfeltétel: 4. szakasz) Módosítsa az `nms` függvényt lineáris *soft-NMS*-szé: ha egy kiválasztott M doboz egy még megmaradt i dobozt elnyomna ($\mathrm{IoU} \geq \tau$), akkor $s_i \leftarrow s_i(1 - \mathrm{IoU}(M, b_i))$; a dobozt csak akkor dobja el, ha az új score 0,1 alá esik. A külső ciklus mindig a pillanatnyi legmagasabb score-ú megmaradt dobozt vegye. Futtassa a 4. szakasz öt dobozán, és adja meg a megmaradt sorrendet és a végső score-okat.

F4. (hibakeresési, nehézség 2, előfeltétel: az AP kézi példa) Egy hallgató a recall nevezőjébe a képhez tartozó detekciók számát írja, azaz $r_k = \mathrm{TP}_k/D$ és nem TP_k/M. Lokalizálja a hibát, írja le a következményét akkor, ha egy képen $M = 3$ GT és $D = 6$ detekció van, és mutassa meg a javítás utáni visszaellenőrzést az AP kézi példán.

F5. (kísérleti/transzfer, nehézség 3, előfeltétel: 25.3. Kis adat és feature-alapú baseline, 26.2. Egy- és kétlépcsős detektor) Tervezzen kísérletet arra, hogyan változik az AP, ha az NMS IoU-küszöbét $\tau \in \{0{,}3;\ 0{,}5;\ 0{,}7\}$ értékekre állítjuk. A rendelkezésre álló adat 20 procedurális kép, képenként 2-5 GT-dobozzal, ugyanazzal a rögzített, NMS előtti detektor-kimenettel. Adjon meg protokollt: megfigyelési egység, split, baseline, változó tényező, metrika, az összehasonlítás típusa és a következtetés érvényességi korlátja.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

26.3.7 7. Megoldások és indoklás

F1. Pozitív területű a és elfajuló b esetén $A(b) = 0$, így a metszet területe is nulla, és $\mathrm{IoU}(a, b) = 0/(A(a) + 0 - 0) = 0$. A félig nyílt konvenció azért fontos, mert a $[x_{\min}, x_{\max})$ értelmezésben az egymást csak határon érintő dobozok metszete nulla hosszúságú, tehát nulla területű. Zárt intervallumokkal a határvonalon osztozó dobozoknak látszólag közös területük lenne. Példa: $a = (0, 0, 10, 10)$ és $b = (10, 0, 20, 10)$; félig nyíltan az $x = 10$ nem tartozik a-hoz, így a metszet üres, területe 0. Zárt értelmezésben a $\{10\} \times [0, 10]$ együttes egy egyenes szakasz, ami festett pixelrácson értelmezhető másként; a numerikus IoU-konvenció a félig nyílt definíciót rögzíti.

F2. Legyen $f : \mathbb{R} \to \mathbb{R}$ szigorúan monoton növekvő, és $s'_i = f(s_i)$. Ekkor $s_i > s_j \iff f(s_i) > f(s_j)$ és $s_i = s_j \iff f(s_i) = f(s_j)$, tehát a score szerinti rendezés és az azonos score-ok csoportja is megmarad. Az azonos score-ú detekciók sorrendjét a terület és az index dönti el, amelyeket f nem változtat meg, ezért a teljes feldolgozási sorrend azonos. A matching csak a sorrendtől és a dobozoktól függ, a score-értékektől nem; a TP/FP sorozat azonos. Ezért a (r_k, p_k) pontok azonosak, és $\mathrm{AP}_{\mathrm{all}}(s') = \mathrm{AP}_{\mathrm{all}}(s)$. A 11 pontos VOC-számítás az AP kézi példa végén:

az interpolált görbe $r = 0$; 0,1; 0,2; 0,3 pontban $p = 1$; $r = 0{,}4$; 0,5; 0,6 pontban $p = 2/3$; $r = 0{,}7$; 0,8; 0,9; 1,0 pontban $p = 3/5$. Az átlag $8{,}4/11 \approx 0{,}7636$.

F3. A soft-NMS implementációjának vázlata:

```
def soft_nms(boxes, scores, iou_threshold=0.5, score_drop=0.1):
    boxes = np.asarray(boxes, dtype=np.float32)
    scores = np.asarray(scores, dtype=np.float64).copy()
    B = boxes.shape[0]
    areas = box_area(boxes)
    suppressed = np.zeros(B, dtype=bool)
    keep = []
    for _ in range(B):
        candidates = [(idx, scores[idx], float(areas[idx]), idx)
                      for idx in range(B) if not suppressed[idx]]
        if not candidates:
            break
        i = max(candidates, key=lambda x: (x[1], x[2], -x[3]))[0]
        keep.append(i)
        suppressed[i] = True
        for j in range(B):
            if suppressed[j]:
                continue
            iou = compute_iou(boxes[i], boxes[j])
            if iou >= iou_threshold:
                scores[j] *= (1.0 - iou)
                if scores[j] < score_drop:
                    suppressed[j] = True
    return keep, scores[keep]
```

A 4. szakasz öt dobozára: először B kerül kiválasztásra. Csak E score-ja változik, mert csak rá teljesül $\mathrm{IoU}(B, E) = 0{,}64 \geq 0{,}5$: $s_E \leftarrow 0{,}9 \cdot (1 - 0{,}64) = 0{,}324$. A D ($\mathrm{IoU}(B, D) = 0{,}4706$), az A (0,25) és a C (0) score-ja változatlan marad, mert a soft-NMS is csak a küszöböt elérő átfedésnél csökkent. A következő választás D (0,85), majd A (0,6), majd C (0,5); közben $\mathrm{IoU}(D, A) = 0{,}0776 < 0{,}5$, tehát A nem módosul. Végül E marad 0,324-gyel. A sorrend: B, D, A, C, E, végső score-ok: 0,9, 0,85, 0,6, 0,5, 0,324. A lineáris soft-NMS tehát megőrizhet olyan dobozt is, amelyet a kemény NMS eldobott (itt E-t); ez a fő viselkedésbeli különbség.

F4. A hiba a recall definíciójában van: a nevező M, a GT-k száma, nem D. Ha $D > M$, a hallgató képlete alulbecsüli a recallt, és hamis „van még találatlan objektum” képet ad akkor is, amikor minden GT-t megtaláltunk. Az AP kézi példában a lépésenkénti recallok $1/6, 1/6, 2/6, 2/6, 3/6, 3/6$ lennének $1/3, 1/3, 2/3, 2/3, 1, 1$ helyett; az AP hamisan alacsonyabb lenne, mert a görbe sosem éri el az 1-et. A javítás a nevező M-re cserélése; a visszaellenőrzés a fenti $r_5 = 1$ érték visszaállítása és az AP 0,7556 reprodukálása.

F5. Protokollvázlat. Kérdés: van-e mérhető különbség az AP-ben a három IoU-küszöb között? Hipotézis: nincs előre rögzített nyertes; az alacsonyabb τ agresszívebben ritkít. Mivel a detektor rögzített, és csak az NMS küszöbét változtatjuk, tanítás nem történik, ezért train/val split sem indokolt: mind a 20 képet értékeljük mindhárom τ mellett, ugyanazon a rögzített, NMS előtti detektor-kimeneten. Ha mégis külön kiértékelési halmazra van szükség, a splitet a jelenet szintjén kell rögzíteni, és a küszöb kiválasztását a választási halmazon kell elvégezni, a jelentett AP-t pedig a fennmaradó halmazon. Változó tényező: $\tau \in \{0{,}3;\ 0{,}5;\ 0{,}7\}$. Baseline: NMS nélküli nyers lista, amelynek AP-je az induló referencia. Metrika: all-point AP, $\tau_{\mathrm{IoU}} = 0{,}5$. Az összehasonlítás páros, ugyanazon a képeken; a bootstrap (újramintavételezéses) mintavételi egysége a jelenet, ha egy jelenethez több nézet tartozik, különben a kép; ebben az adatban minden kép külön jelenet, ezért a kép a független egység, a doboz nem. Korlát: a 20 kép nem elegendő általános detektálási benchmarkhoz; az eredmény csak a saját procedurális adatra vonatkozik, és a detektor-modell rögzítése miatt a tanulási bizonytalanságot nem méri.

26.3.8 8. Tipikus tévedés ellenpéldával

A leggyakoribb hiba az AP és az egyetlen score-küszöbön mért precision összekeverése. Ellenpélda az AP kézi példa adatsorából: ha a felhasználó $\tau_s = 0{,}6$-ot választ, mind a hat detekció bekerül, így a precision $3/6 = 0{,}5$. Ha $\tau_s = 0{,}95$-öt választ, csak az első doboz kerül be, a precision $1/1 = 1$. Ugyanarra a modellre a két működési pont látványosan eltérő precisiont ad, miközben a modell rangsora nem változott. Az AP éppen azért használja a teljes rendezett listát, hogy ne egyetlen küszöbhöz kössük a modell minőségét. A hiba következménye: a „modellünk precisionje 1" állítás 0,95-ös küszöbnél igaz is lehet, de elhallgatja a két duplikátumot és a kihagyott GT-ket. A javított megfogalmazás: „az adott τ_s melletti precision 1, az all-point AP 0,7556".

Önellenőrzés: a τ növelése közvetlenül kevesebb párt nyom el, mert az IoU $\geq \tau$ feltétel szigorodik. Ebből mégsem következik, hogy a megtartott dobozok száma monoton csökken τ növelésével: a mohó (greedy) lánc miatt egy alacsony τ-nál elnyomott doboz magasabb τ-nál életben maradhat, és ő nyomhat el olyan dobozt, amely alacsony τ-nál megmaradt. Ezért általános monotonitási állítást nem teszünk, csak a közvetlen elnyomási feltétel szigorodását rögzítjük. A score szerint rendezett listából mindig a következő legnagyobb score-t kiválasztó kód megmaradó listája score-csökkenő; az eredeti indexek sorrendje ettől külön fogalom (a visszaadott indexek nem feltétlenül növekvő score-sorrendben jönnek, ha a hívó nem rendezi őket). A PR-pontok p_k értéke nem monoton: a kézi példában $1 \to 0{,}5 \to 0{,}667 \to 0{,}5 \to 0{,}6 \to 0{,}5$; a $p_{\max}$-interpoláció éppen ezt simítja. **Visszahívás:** a 14.3. Paraméterszám, MAC és aktivációmemória szerint a MAC a modell egy előremenetének számítási költségét méri (a szorzás-összeadás műveletek számát); az NMS $O(B^2)$ IoU-számítása ettől független posztprocessz-költség, nagy B esetén külön mérni kell.

26.3.9 Rövid összefoglaló

Az NMS a nyers detekciók posztprocessze: score, majd terület és index szerinti determinisztikus sorrendben kiválaszt, és a τ-nál jobban átfedő, alacsonyabb prioritású dobozokat elnyomja. A score threshold ezzel ellentétben egyetlen működési pontot vág le; az AP a teljes rangsorból indul. A detekciós matching során minden GT-hez legfeljebb egy TP tartozhat; a második illeszkedő detekció FP, ez a duplicate detection büntetése. Az all-point AP a PR-görbe alatti, jobb oldali maximális precisionnel interpolált terület. A kézi példában az AP 0,7556, a 11 pontos VOC-változat 0,7636; a kettő közötti eltérés konvenció, nem hiba.

Az induló kérdésre a válasz: a két magas score-ú, erősen átfedő doboz közül az NMS egyet hagy meg, a második a matching során FP lesz, ezért egy GT-hez legfeljebb egy TP tartozhat, és a mért teljesítmény nem duplázódik; az AP a teljes rangsort értékeli, nem egyetlen score-t.

26.3.10 Következő alkalmazás és források

A 26.4. Detektálási pipeline és hibaelemzés egység a teljes detektálási pipeline-t és a hibaelemzést vizsgálja: az itteni NMS, score threshold és AP ott kapcsolódik össze a választott architektúra, az adatfolyam és a tévesztési esetek rendszeres elemzésével.

- [P12] Ren et al. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. https://arxiv.org/abs/1506.01497.
- [P13] Redmon et al. (2015). *You Only Look Once: Unified, Real-Time Object Detection*. https://arxiv.org/abs/1506.02640.

- [D18] PyTorch contributors (2026). *TorchVision Object Detection Finetuning Tutorial*. PyTorch 2.14 documentation. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html. Hozzáférés: 2026-10-30.
- [D22] COCO Consortium (2026). *cocoapi: PythonAPI/pycocotools/cocoeval.py* (`Params.setDetParams`: `iouThrs`, `recThrs`; `evaluateImg`, `accumulate`). COCO API documentation. https://github.com/cocodataset/cocoapi/blob/master/PythonAPI/pycocotools/cocoeval.py. Hozzáférés: 2026-10-30.
- Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. és Zisserman, A. (2010). *The PASCAL Visual Object Classes (VOC) Challenge*. International Journal of Computer Vision, 88(2), 303–338. A VOC-matching és a 11 pontos interpolációs konvenció forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

26.4 Detektálási pipeline és hibaelemzés

26.4.1 Cél, előismeret és induló kérdés

A 26.3. NMS, confidence és AP egységben a detektálás utófeldolgozását (NMS, score-küszöb) és az AP-metrikát tárgyaltuk, de nem kérdeztük meg: hogyan jut el a nyers kép a detektor bemenetéig úgy, hogy a doboz annotációk is érvényesek maradjanak? Amikor egy képet átméretezünk, kivágunk vagy tükrözünk, a pixelkoordinátákban megadott dobozok külön transzformációt igényelnek. Ez az egység a **synchronized transform** fogalmát, a **dataset contract**-ot és az **objektumszintű hibajegyzéket** építi fel. Előismeret: a 26.3. NMS, confidence és AP IoU-, NMS- és AP-definíciói; a 22.4. Augmentációs kísérlet és domainváltozás alfejezetből a train augmentáció és a rögzített tesztkorrupció elkülönítése; a 25.4. Nyílt halmaz és ismeretlen input alfejezetből a nyílt halmaz szemlélete, amely itt a negatív, objektum nélküli képek kezelésében tér vissza.

Induló kérdés: ha egy 24×24-es képen a $(2; 3; 10; 11)$ dobozt tartalmazó képet 12×12-re resize-oljuk, de a dobozt változatlanul hagyjuk, akkor a betanított detektor melyik koordinátarendszerben értelmezi a dobozt, és ez hogyan torzítja az IoU-t?

26.4.2 Definíciók és dataset contract

A felhasznált konvenciók a fejezet elején bevezetett 26.1. Dobozkoordináta, IoU és lokalizációs loss dobozdefiníciót követik. Egy kép $X \in \mathbb{R}^{C\times H\times W}$, ahol H a magasság, W a szélesség pixelben. Egy doboz **félig nyílt** xyxy intervallum:

$$b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}), \qquad 0 \le x_{\min} < x_{\max} \le W, \quad 0 \le y_{\min} < y_{\max} \le H.$$

A doboz területe $\operatorname{area}(b) = (x_{\max} - x_{\min})(y_{\max} - y_{\min})$. A detektálási target egy képen M objektumot ír le; a leggyakoribb szerződés szerint a dobozok tömbje $\mathbf{B} \in \mathbb{R}^{M\times 4}$ float32, a címkék vektora $\mathbf{y} \in \{0, \dots, K-1\}^M$ int64, ahol a K osztályhalmaz a háttérosztályt nem tartalmazza (saját adatformátum). A TorchVision detection API-ja a `0` címkét a háttérnek tartja fenn, ezért az API-hoz a saját k címkét $k+1$-re kell leképezni, a fej osztályszáma $K+1$, és inference után vissza kell alakítani ($k+1 \to k$); saját API-nál az eltérést ki kell mondani. A torchvision-stílusú target dict ezt `boxes` és `labels` kulcsokkal tárolja; az üres képet $M = 0$, azaz 0×4 alakú üres doboztömb és 0 hosszú címkevektor jelöli [D18].

Definíció (detektálási adathalmaz szerződés). A tanító és kiértékelő pipeline-ra érvényes szerződés:

1. Minden képhez egy target tartozik; az üres képet explicit $M = 0$ jelöli, nem hiányzó rekord.
2. A dobozok mindig az aktuális, már transzformált kép méretéhez tartozó koordinátákban vannak.
3. Geometriai transzformáció esetén a képet és az összes dobozt **ugyanazzal** a leképezéssel, egy időben transzformáljuk; ezt nevezzük synchronized transformnak.
4. A train augmentáció csak a train halmazon hat; a valid és test transzformáció determinisztikus, és a testhez tartozó annotációt a geometriai műveleten kívül nem módosítjuk.

Az utolsó pont a 22.4. Augmentációs kísérlet és domainváltozás alfejezetben bevezetett elvet viszi át: a train policy a cél-eloszlás mintavételét módosítja, míg a tesztkorrupció a befagyasztott modell értékelésére szolgáló rögzített leképezés.

Definíció (synchronized transform). Legyen $\phi : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^{C'\times H'\times W'}$ képtranszformáció, amely egy $\psi : \mathbb{R}^2 \to \mathbb{R}^2$ térbeli leképezésből származik: a ψ azt adja meg, hogy a bemeneti pixelpozíció hova kerül a kimeneten. A dobozok transzformációjához elegendő ez a geometriai megfeleltetés; a pixelértékekre a $\phi(X)(c, \psi(p)) = X(c, p)$ azonosság csak nem interpoláló transzformációnál teljesül, átméretezésnél az új pixelérték több bemeneti pixel súlyozott átlaga. A ϕ **szinkronizált** alkalmazása a $(X, \mathbf{B})$ párra a következő: $X' = \phi(X)$, és minden $b = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ dobozra

$$b' = \mathrm{bbox}(\psi(\text{mind a négy sarok})),$$

ahol a doboz mind a négy sarkát $((x_{\min}, y_{\min})$, $(x_{\max}, y_{\min})$, $(x_{\min}, y_{\max})$, $(x_{\max}, y_{\max}))$ transzformáljuk, az új doboz a négy képpont koordinátánkénti minimuma és maximuma, majd a koordinátákat a $H' \times W'$ képbe vágjuk. Tetszőleges geometriai transzformációra (forgatás, nyírás, tükrözés) nem elég két átellenes sarok: tengelytartó leképezésnél (átméretezés, eltolás) a két sarok elég, de a tükrözés felcseréli a határok sorrendjét, ezért a min/max-ot újra kell rendezni (például vízszintes tükrözésnél $x'_{\min} = W - x_{\max}$, $x'_{\max} = W - x_{\min}$). Nem tengelytartó példa: 45°-os forgatásnál az egységnégyzet $[0, 1]^2$ négy sarkának képe $(0, 0), (\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}), (0, \sqrt{2})$ és $(-\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2})$ (a forgatás az origó körül), így az új doboz $x \in [-\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2}]$, $y \in [0, \sqrt{2}]$; két átellenes sarok ($(0, 0)$ és $(1, 1)$ képe) a nulla szélességű $x = 0$, $y \in [0, \sqrt{2}]$ szakaszt adná, ami elfajuló doboz. Átméretezésnél a képpontértékek nem egyszerű átindexelés, hanem interpolációs operátor eredményei; a doboz geometriáját a ψ írja le, az értékeket az interpoláció. Ha a levágás után a doboz érvénytelen ($x'_{\min} \geq x'_{\max}$ vagy $y'_{\min} \geq y'_{\max}$), akkor az objektum eltűnik a targetből, és M csökken.

26.4.3 Lemma: resize doboztranszformációja

Lemma (resize geometriája). Legyen a kép eredeti mérete $H \times W$, az átméretezett méret $H' \times W'$, és legyen $\alpha_x = W'/W$, $\alpha_y = H'/H$. Az átméretezés, mint ϕ, a félpixeles pixelközéppont-konvencióban a $\psi(x, y) = (\alpha_x x, \alpha_y y)$ térbeli leképezést indukálja; a konvenciók eltérését a bizonyítás után tárgyaljuk. Ekkor egy $b = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ doboz szinkronizált transzformáltja

$$b' = (\alpha_x x_{\min},\ \alpha_y y_{\min},\ \alpha_x x_{\max},\ \alpha_y y_{\max}),$$

és $\mathrm{area}(b') = \alpha_x \alpha_y\, \mathrm{area}(b)$.

Bizonyítás. Az átméretezés definíciója szerint a bemeneti kép (x, y) pixelpozíciója a kimeneti kép $(\alpha_x x, \alpha_y y)$ pozíciójára kerül, tehát a sarokpontokra ψ a fenti lineáris leképezés. Az érvényességhez ellenőrizzük a határokat: mivel $\alpha_x > 0$ és $x_{\min} < x_{\max}$, ezért $\alpha_x x_{\min} < \alpha_x x_{\max}$; továbbá

$x_{\max} \le W$ miatt $\alpha_x x_{\max} \le \alpha_x W = W'$, így az új doboz a kép határain belül marad. Ugyanez áll az y-koordinátára. A terület:

$$\operatorname{area}(b') = (\alpha_x x_{\max} - \alpha_x x_{\min})(\alpha_y y_{\max} - \alpha_y y_{\min}) = \alpha_x \alpha_y (x_{\max} - x_{\min})(y_{\max} - y_{\min}) = \alpha_x \alpha_y \operatorname{area}(b).$$

Ezzel a lemma bizonyítását befejeztük. A bizonyítás során kihasználtuk, hogy a doboz sarokpontokkal adott; a félig nyílt intervallum jellege a lineáris leképezés alatt megmarad, mert a pozitív α_x, α_y skalárral való szorzás monoton bijekció, és az $[a, b)$ intervallum képe $[\alpha_x a, \alpha_x b)$. Az állítás eltolás nélküli, konstans eltolást nem tartalmazó resize-ra vonatkozik; ha a resize-t crop követi vagy előzi meg, a kompozíciót külön kell kezelni. A lemma folytonos koordinátákon fogalmaz; a gyakorlati resize-ok pixelközéppont-konvenciója eltérhet, ezért a doboztranszformációt ugyanazzal a konvencióval kell számolni, mint a kép átmintavételezését. A félpixeles (pixelközéppont-alapú, `align_corners=False` típusú) konvencióban a pixelhatárok éppen $\alpha_x = W'/W$ arányban skálázódnak. `align_corners=True` esetén a sarokpixelek középpontja igazodik, és a dobozélek képe $e' = s'e + (1 - s')/2$ eltolást is kap, ahol $s' = (W' - 1)/(W - 1)$ (értelemszerűen $s' = (H' - 1)/(H - 1)$ az y-tengelyen).

26.4.4 Crop, kompozíció és üres cél

Crop. Legyen a kivágott ablak $[x_0, x_0 + W_c) \times [y_0, y_0 + H_c)$, ahol $W_c \le W$, $H_c \le H$. A crop önmagában eltolás: $\psi(x, y) = (x - x_0,\ y - y_0)$. Mivel a doboz egy része kieshet az ablakból, a szinkronizált transzformáció után a dobozt a $[0, W_c) \times [0, H_c)$ tartományra vágjuk:

$$x'_{\min} = \max(0,\ x_{\min} - x_0), \qquad y'_{\min} = \max(0,\ y_{\min} - y_0),$$

$$x'_{\max} = \min(W_c,\ x_{\max} - x_0), \qquad y'_{\max} = \min(H_c,\ y_{\max} - y_0).$$

Ha $x'_{\min} \ge x'_{\max}$ vagy $y'_{\min} \ge y'_{\max}$, a doboz eltűnik.

Kompozíció. A pipeline gyakran crop + resize sorozatot alkalmaz. Mivel a leképezések kompozíciója a leképezések egymás utáni alkalmazása, a dobozokra ugyanazt a sorrendet kell alkalmazni, mint a képre. A következő feladat (F2) ezt számszerűen begyakoroltatja.

Empty target. A negatív kép $M = 0$ eset. A pipeline-nak ez nem hibás állapot, hanem érvényes adatpont: a `boxes` tömb `(0,4)` alakú, a `labels` üres. A transzformációs függvényeknek ezt az esetet explicit módon kezelniük kell, mert az üres tömbön végzett koordináta-művelet shape-hibához vezethet, vagy ami rosszabb, csendben üres marad, és a downstream összehasonlítás félreértelmezi.

26.4.5 Objektumszintű hibajegyzék

A 26.3. NMS, confidence és AP greedy matching eljárása után minden ground truth (GT) doboz vagy kap hozzá párosított predikciót (IoU $\ge \tau$, tipikusan $\tau = 0{,}5$), vagy nem; minden predikció vagy kapott GT-párt, vagy nem. Ebből a következő objektumszintű hibakategóriákat definiáljuk:

Kategória	Definíció	Következmény
Missed detection	GT doboz, amelyhez nem tartozik predikció	False negative, recall-csökkenés
Duplicate detection	Ugyanahhoz a GT-hez több predikció is kapcsolódna	Precision-csökkenés, kettős számlálás
Localization error	A predikció osztálya helyes, a legnagyobb IoU-jú, azonos osztályú GT-vel $0 < \text{IoU} < \tau$	False positive, rossz doboz
Classification error	A predikció a legnagyobb IoU-jú GT-vel $\text{IoU} \geq \tau$, de az osztálya hibás	False positive, osztálytévesztés
False positive (háttér)	Predikció, amelyhez nincs azonos osztályú, IoU-küszöböt elérő GT	Precision-csökkenés

A kategóriák akkor nem fedik át egymást, ha a besorolást rögzített sorrendben végezzük: először a matched predikciók osztályellenőrzése (classification error), majd a nem matched predikciók közül a lokális átfedés (localization error), végül a háttér false positive; a missed detection a GT oldalon külön számolódik. Egy lokalizációs hiba a GT oldalon missed detectionként, a predikció oldalon localization errorként jelenik meg: ez ugyanannak a hibának két nézete, ezért a két oldalt külön kell közölni, és nem szabad őket egyetlen darabszámba összeadni. A „legközelebbi GT" itt a legnagyobb IoU-jú, azonos osztályú GT-t jelenti, nem középpont-távolságot.

A lokalizációs hiba és a téves osztály (classification error) megkülönböztetése fontos, mert a javítás módja eltér: a lokalizáció a regressziós fej, az osztálytévesztés a klasszifikációs fej problémájára utal. A fenti táblázat csak a greedy matching utáni durva osztályozás; a részletes hibaelemzés a predikció score-ját és az IoU hisztogramját is vizsgálja. A [P12] és [P13] által képviselt két detektorcsalád kimenetére ugyanez a hibajegyzék alkalmazható, mert mindkettő doboz–osztály–score predikciókat ad.

Heurisztika (hibajegyzék célja). A hibajegyzék nem helyettesíti az AP-metrikát, hanem diagnosztikai eszköz: megmutatja, hogy a detektor mely hibatípusban veszít. A diagnózis akkor megbízható, ha a jegyzéket ugyanazon a rögzített teszthalmazon, ugyanazzal a matching konvencióval számoljuk, mint az AP-t. Ez heurisztika, nem tétel: a hibatípusok aránya nem azonosítható egyetlen okkal.

26.4.6 Kézi kis példa és kód

Kézi példa. Legyen $H = W = 8$, $H' = W' = 4$, tehát $\alpha_x = \alpha_y = 0{,}5$. A doboz $b = (2; 3; 6; 7)$. A transzformált: $b' = (0{,}5 \cdot 2;\ 0{,}5 \cdot 3;\ 0{,}5 \cdot 6;\ 0{,}5 \cdot 7) = (1;\ 1{,}5;\ 3;\ 3{,}5)$. Terület: $\text{area}(b) = (6-2)(7-3) = 16$, $\text{area}(b') = (3-1)(3{,}5-1{,}5) = 4 = 0{,}25 \cdot 16$. A doboz a 4×4-es kép belsejében marad.

Minimális kód. A következő függvény a resize szinkronizált doboztranszformációját valósítja meg NumPy-ban. Az input és a futtatási környezet: munkakönyvtár `lab/det_pipeline`, CPU, NumPy 2.x, dtype float32, seed 20261020 (a seed a szintetikus adat előállításához használatos, maga a transzformáció determinisztikus).

```
# resize_boxes.py
"""Szinkronizált resize doboztranszformáció, NumPy referencia."""
import numpy as np
```

```python
def resize_boxes(boxes: np.ndarray, old_hw: tuple[int, int],
                 new_hw: tuple[int, int]) -> np.ndarray:
    """Dobozok átméretezése a kép resize-zal együtt.

    boxes: float32, shape (M, 4), félig nyílt xyxy.
    old_hw: (H, W), new_hw: (H', W').
    Visszatér: float32, shape (M, 4).
    Üres M=0 esetén (0,4) alakú tömböt ad vissza.
    """
    boxes = np.asarray(boxes, dtype=np.float32)
    if boxes.shape[0] == 0:
        return boxes.reshape(0, 4).astype(np.float32)
    H_old, W_old = old_hw
    H_new, W_new = new_hw
    alpha_y = H_new / H_old
    alpha_x = W_new / W_old
    out = boxes.copy()
    out[:, 0] *= alpha_x
    out[:, 2] *= alpha_x
    out[:, 1] *= alpha_y
    out[:, 3] *= alpha_y
    return out

if __name__ == "__main__":
    # Ellenőrző példa
    boxes = np.array([[2, 3, 6, 7]], dtype=np.float32)
    result = resize_boxes(boxes, (8, 8), (4, 4))
    print(result)  # mért: [[1.  1.5 3.  3.5]]
    assert np.allclose(result, [[1.0, 1.5, 3.0, 3.5]], atol=1e-6, rtol=0)
    assert resize_boxes(np.empty((0, 4), np.float32), (8, 8), (4, 4)).shape == (0, 4)
```

Futtatási parancs: `python resize_boxes.py`. A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU):

```
[[1.  1.5 3.  3.5]]
```

A kimenet a várt értékkel egyezik; tolerancia $\epsilon = 10^{-6}$ abszolút hibahatárral ellenőrizhető, mert a műveletek szorzásból állnak és float32-ben a $4/8 = 0{,}5$ pontosan reprezentálható.

26.4.7 Labor: szintetikus detektálási kísérleti terv és hibajegyzék

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A következő labor a megadott környezetben nem futtatható: a csúszóablakos detektor és a 40 képes szintetikus adathalmaz nem része a fejezet kódjának, ezért itt kísérleti tervet adunk. A tervben szereplő számok **VÁRT ÉRTÉK (levezetésből, nem mért)** címkével értendők; tényleges futtatás és nyers számok szükségesek a kísérleti következtetéshez.

Kérdés. Hogyan változik az objektumszintű hibajegyzék, ha a tesztképeket 24×24-ről 12×12-re resize-oljuk, és a dobozokat (a) nem transzformáljuk, (b) szinkronizáltan transzformáljuk?

Hipotézis. Az (a) változatban a dobozok a régi koordinátarendszerben maradnak, ezért a predikciók és a GT közötti IoU szisztematikusan csökken; a lokalizációs hibák aránya megnő. A (b) változatban a hibajegyzék a transzformáció előttihez hasonló marad.

Adat és split. 40 szintetikus kép, 24×24, egy csatorna, float32, seed 20261020. Minden képen 0–3 egyszerű téglalap objektum, osztályok $\{0, 1, 2\}$. Legalább 10 kép üres ($M = 0$). A split: 20

train, 10 validációs, 10 test, az objektumok számára rétegezve, a `scene_id` (jelenetazonosító) szerinti csoportosítással (ha több nézet van). A test annotációt a protokoll végéig nem nézzük.

Baseline és változó. A baseline egy egyszerű csúszóablakos detektor, amely rögzített méretű ablakot csúsztat, és osztályoz + regresszál. A változó tényező a pipeline transzformációja: (a) kép resize doboztranszformáció nélkül, (b) szinkronizált resize a fenti függvénnyel. Minden más paraméter rögzített.

Metrika. Objektumszintű hibajegyzék: missed, duplicate, localization, false positive darabszám; valamint AP @ IoU = 0,5 a 26.3. NMS, confidence és AP konvencióval. A kiértékelés a befagyasztott, validációs halmazon kiválasztott modellen történik.

Ellenőrzés. A szinkronizált transzformáció után minden dobozra ellenőrizzük, hogy $0 \le x_{\min} < x_{\max} \le W'$ és $0 \le y_{\min} < y_{\max} \le H'$. Az üres képeknél $M = 0$ megmarad.

Szemléltető hibajegyzék — VÁRT ÉRTÉK (levezetésből, nem mért). A következő számpélda a módszer formáját mutatja, nem futtatott eredmény. A 10 képes test halmazból négy képet kiemelve tegyük fel, hogy ezeken összesen 5 GT doboz van. A detektor 6 predikciót ad; a greedy matching után a hibák: 1 missed, 1 duplicate, 1 localization, 1 false positive. Ekkor a jegyzék:

Kép	GT	Predikció	Eredmény
1	1	2	1 TP, 1 duplicate
2	1	1	1 localization (IoU 0,3)
3	2	2	1 TP, 1 missed, 1 FP
4	1	1	1 TP

A sorok konzisztensek: a GT-k száma $1 + 1 + 2 + 1 = 5$, a predikcióké $2 + 1 + 2 + 1 = 6$, a TP-k száma 3. A GT-oldal 3 TP-ből, 1 missedből és 1 localizationból áll (összesen 5); a predikciós oldal 3 TP-t, 1 duplicate-ot, 1 localizationt és 1 FP-t tartalmaz (összesen 6). A példa számsorai kézzel konstruáltak; a kísérleti következtetéshez tényleges futtatás és nyers számok szükségesek.

26.4.8 Feladatok

F1 (fogalmi, nehézség 1). Adott egy kép egyetlen GT dobozzal $b = (0; 0; 10; 10)$, osztály 0, és a detektor egyetlen predikciója $p = (5; 5; 15; 15)$, osztály 0. Számítsa ki az IoU-t ($\tau = 0{,}5$), és sorolja be a predikciót és a GT-t is a fenti hibajegyzék kategóriáiba. Indokolja, miért missed detection a GT oldalán, és miért nem duplicate a predikció.

F2 (levezetési, nehézség 2). Vezesse le a crop + resize kompozíció doboztranszformációját. Bemenet: 12×12-es kép, crop ablak $x_0 = 2$, $y_0 = 2$, $W_c = H_c = 8$, majd resize 4×4-re. A bemeneti doboz $(1; 3; 9; 11)$. Adja meg a transzformált dobozt lépésről lépésre, és ellenőrizze az érvényességet.

F3 (implementációs, nehézség 2). Írjon `crop_boxes` függvényt, amely a crop doboztranszformációját végzi el: bemenet `boxes` $(M, 4)$ float32, a crop ablak (x_0, y_0, W_c, H_c); a kimenet a levágott, érvényes dobozok $(M', 4)$ alakban és a megmaradt dobozok eredeti indexei $(M',)$, hogy a hozzájuk tartozó címkék szinkronban szűrhetők legyenek. Kezelje az $M = 0$ és a teljesen kieső doboz esetét. Ellenőrizze a fenti F2 példa crop lépésén.

F4 (hibakeresési, nehézség 2). Adott a következő pipeline-részlet:

```python
def transform(image, boxes):
    image = resize(image, (12, 12))
    # itt hiányzik valami
    return image, boxes
```

Magyarázza el, mi a hiba, mi a következménye a tanításra és az AP-számításra, és adja meg a javítást. Visszaellenőrzésként mondja ki, milyen invariánsnak kell teljesülnie a javított függvény kimenetére.

F5 (kísérleti, nehézség 3). Tervezzen kísérletet annak számszerűsítésére, hogy a csak pozitív képeken mért precision és AP mennyiben torzítja a detektor értékelését a teljes teszt halmazon mért értékekhez képest. Adja meg a kérdést, a hipotézist, az adat összetételét (pozitív/negatív arány), a baseline detektort, a változó tényezőt, a metrikát és a következtetés érvényességi korlátját.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

26.4.9 Megoldások és indoklás

F1 megoldás. Az interszekció: a két doboz közös része a $[5, 10) \times [5, 10)$ négyzet, területe $(10-5)\cdot(10-5) = 25$. Az unió: $10\cdot 10 + 10\cdot 10 - 25 = 175$. Tehát $\mathrm{IoU} = 25/175 \approx 0{,}1429 < 0{,}5$. A greedy matching során a predikció nem kap GT-párt, mert az IoU a küszöb alatt van; matching szinten tehát false positive. A GT oldalán ez **missed detection**: a GT-hez nem tartozik küszöböt elérő predikció, és a missed detection a GT-kre vonatkozó kategória, nem a predikcióké. A predikció oldalán **localization error**, mert van azonos osztályú, pozitív átfedésű GT, de az IoU a küszöb alatt marad; nem duplicate, mert nincs korábban matched GT, amelyhez másodikként kapcsolódna.

F2 megoldás. Első lépés, crop az eltolással: $x' = x - 2$, $y' = y - 2$. A doboz: $(1-2;\ 3-2;\ 9-2;\ 11-2) = (-1;\ 1;\ 7;\ 9)$. Levágás a $[0, 8) \times [0, 8)$ ablakra: $x'_{\min} = \max(0, -1) = 0$; $y'_{\min} = \max(0, 1) = 1$; $x'_{\max} = \min(8, 7) = 7$; $y'_{\max} = \min(8, 9) = 8$. Tehát a crop utáni doboz $(0; 1; 7; 8)$, érvényes, mert $0 < 7 \le 8$ és $1 < 8 \le 8$. Második lépés, resize 4×4-re: $\alpha_x = \alpha_y = 4/8 = 0{,}5$. A szorzás után $(0,\ 0{,}5,\ 3{,}5,\ 4)$. Ellenőrzés: $0 < 3{,}5 \le 4$, $0 < 0{,}5 < 4$, a doboz érvényes a 4×4-es képen. A terület a crop után $(7-0)(8-1) = 49$, a resize után $(3{,}5-0)(4-0{,}5) = 3{,}5\cdot 3{,}5 = 12{,}25 = 0{,}25\cdot 49$, ami megfelel a lemma állításának.

F3 megoldás. A függvény kódja:

```python
import numpy as np

def crop_boxes(boxes: np.ndarray,
               crop: tuple[int, int, int, int]) -> tuple[np.ndarray, np.ndarray]:
    """Crop doboztranszformációja.

    boxes: float32, shape (M, 4), xyxy.
    crop: (x0, y0, W_c, H_c) félig nyílt ablak.
    Visszatér: (float32, shape (M', 4); int64, shape (M',)),
    a megmaradt dobozok eredeti indexei.
    """
    boxes = np.asarray(boxes, dtype=np.float32)
    if boxes.shape[0] == 0:
        return boxes.reshape(0, 4).astype(np.float32), np.empty(0, dtype=np.int64)
    x0, y0, W_c, H_c = crop
    out = boxes.copy()
    out[:, 0] -= x0
```

```
    out[:, 2] -= x0
    out[:, 1] -= y0
    out[:, 3] -= y0
    out[:, 0] = np.maximum(out[:, 0], 0.0)
    out[:, 1] = np.maximum(out[:, 1], 0.0)
    out[:, 2] = np.minimum(out[:, 2], float(W_c))
    out[:, 3] = np.minimum(out[:, 3], float(H_c))
    valid = (out[:, 2] > out[:, 0]) & (out[:, 3] > out[:, 1])
    return out[valid].astype(np.float32), np.flatnonzero(valid).astype(np.int64)

if __name__ == "__main__":
    boxes = np.array([[1, 3, 9, 11]], dtype=np.float32)
    result, keep = crop_boxes(boxes, (2, 2, 8, 8))
    print(result, keep)  # mért: [[0. 1. 7. 8.]] [0]
    assert np.allclose(result, [[0.0, 1.0, 7.0, 8.0]], atol=1e-6, rtol=0)
    assert keep.tolist() == [0]
    empty, keep0 = crop_boxes(np.empty((0, 4), np.float32), (2, 2, 8, 8))
    assert empty.shape == (0, 4) and keep0.shape == (0,)
```

A `crop_boxes` futtatott ellenőrző példája a megadott környezetben (Python 3.12.3, NumPy 2.5.4, CPU) a `[[0. 1. 7. 8.]]` kimenetet és a `[0]` indexet adja, ami egyezik a várt értékkel. A visszaadott indexekkel a címkék szinkronban szűrhetők: `labels_kept = labels[keep]`. A függvény az $M = 0$ esetben üres tömböt ad, a teljesen kieső dobozokat kiszűri. A `valid` maszk a szigorú egyenlőtlenséget használja, mert a félig nyílt intervallum akkor érvényes, ha $x_{\min} < x_{\max}$.

F4 megoldás. A hiba: a `transform` a képet resize-olja, de a dobozokat nem transzformálja a szinkronizált módon. A dobozok koordinátái az eredeti kép méretére vonatkoznak (a feladat nem rögzít konkrét méretet; a kód a bemenetből olvassa ki), míg a kép már 12×12-es. **Következmény:** a training során a regressziós loss téves célokat lát, mert a dobozok a képen kívülre vagy rossz helyre mutatnak; a kiértékelésnél az IoU hamis, az AP alábecsült, a lokalizációs hibák aránya mesterségesen megnő. **Javítás:** a `resize_boxes` hívása a `resize` után:

```
def transform(image, boxes):
    H, W = image.shape[-2], image.shape[-1]
    image = resize(image, (12, 12))
    boxes = resize_boxes(boxes, (H, W), (12, 12))
    return image, boxes
```

Visszaellenőrzés: a kimeneti dobozoknak teljesíteniük kell, hogy minden dobozra $0 \le x_{\min} < x_{\max} \le 12$ és $0 \le y_{\min} < y_{\max} \le 12$; az üres képeknél a dobozok tömbje $(0, 4)$ alakú marad.

F5 megoldásminta. Kérdés: mennyivel nagyobb a detektor precisionje a teljes detekciós küszöbön, és hogyan változik az AP, ha a tesztből eltávolítjuk a negatív (objektum nélküli) képeket? **Hipotézis:** a csak pozitív képeken számolt precision nagyobb; az AP a teljes halmazon a score-rangsortól függően csökken vagy változatlan marad, mert a false positive-ok nagy része a negatív képekről származik. **Adat:** 100 szintetikus tesztkép: 20 pozitív, 80 negatív, seed 20261020; az egyszerűség kedvéért a pozitív képeken pontosan 1 objektum van. Ha 1–3 objektum lenne, a baseline recallja 1-nél kisebb lenne, és a teljes AP nem lehetne 1,0 még a kedvező rangsor esetén sem. **Baseline:** egy egyszerű detektor, amely minden pozitív képen 1 helyes dobozt ad (IoU 0,7 felett), de minden negatív képen is ad 1 hamis dobozt. **Változó:** a kiértékelés halmaza — csak pozitív képek (20) vs. az összes kép (100). **Metrika:** precision a rögzített küszöbön (minden predikció megtartva) és AP @ IoU = 0,5. **Várható számítás:** a csak pozitív halmazon a precision $20/(20+0) = 1$, míg a teljes halmazon $20/(20+80) = 0{,}2$. A precision a rögzített küszöbön minden predikció megtartásával számol, ezért ez a hányados nem feltételez egyenlő score-okat; az AP viszont a 20 TP és a 80 FP score-rangsorától függ. Ha minden FP a TP-k fölé kerül, a teljes AP $(1/20)\sum_{k=1}^{20} k/(k+80) \approx 0{,}1124$; ha minden TP van felül, a teljes AP 1,0,

akárcsak a csak pozitív halmazon. A kísérletben ezért a score-rangsort és a tie-break szabályát rögzíteni kell. A számok kézi számításból származnak, nem futtatott mérésből. **Következtetési korlát:** a szintetikus baseline nem valós detektor; a kísérlet csak a torzítás mechanizmusát mutatja, nem ad általános nagyságrendet. A valós értékeléshez a negatív képeket is tartalmazó, rögzített teszt szükséges.

26.4.10 Tipikus tévedés: csak pozitív képeken mért kísérlet

A gyakori hiba az, hogy a detektálási teljesítményt olyan teszt halmazon mérik, amely csak objektumot tartalmazó képeket tartalmaz. Ez nem teljes detektálási értékelés. A fenti F5 ellenpéldája mutatja: ha a baseline mind a 80 negatív képen hamis dobozt ad, akkor a teljes detekciós küszöbön a precision $20/(20+80) = 0{,}2$, míg a csak pozitív halmazon $20/(20+0) = 1$. Az AP ennél óvatosabban értelmezendő: a TP-k és FP-k score-rangsora dönt; ha a TP-k minden FP fölé kerülnek, a teljes AP továbbra is 1,0, ha az FP-k vannak felül, akkor a számított érték $\approx 0{,}1124$. A torzítás oka, hogy a false positive-ok jelentős része a negatív képekből származik, és a csak pozitív képek kiértékelése ezt a hibaforrást kizárja.

Az állítás formálisan: legyen $\mathcal{E}^+$ a pozitív képek halmaza, $\mathcal{E}$ a teljes teszt. A negatív képeken nincs ground truth, ezért az ottani predikciók (ha vannak) csak false positive-ok lehetnek; nem kell feltenni, hogy minden negatív képen van predikció. Ekkor az $\mathcal{E}^+$-n számolt empirikus precision nagyobb vagy egyenlő, mint az $\mathcal{E}$-n számolt, feltéve, hogy a true positive-ok száma azonos. A bizonyítás közvetlen: a precision nevezője $\mathrm{TP} + \mathrm{FP}$, és az $\mathcal{E}^+$-ra szűkítés csak FP-ket távolít el, így a nevező nem nő, a hányados nem csökken. Ez levezetés, nem empirikus állítás; a konkrét különbség mértéke adatfüggő.

Ugyanez AP-re is átvihető. A matching képenként történik, ezért a negatív képek predikcióinak eltávolítása a pozitív képek párosítását nem érinti; minden score-küszöbnél ugyanannyi TP marad és legfeljebb annyi FP, így a precision–recall görbe nem csökken, és az all-point AP sem csökken. A megengedő feltétel tehát az, hogy a negatív képeken ne legyen ground truth, és a matching, illetve az interpoláció konvenciója ne változzon; az, hogy minden negatív képen legyen predikció, nem szükséges. Az AP-változás mértéke a score-rangsortól függ: ha a negatív képi FP-k a TP-k fölé kerülnek, az AP csökken, ha alá, akkor nem.

26.4.11 Önellenőrzés és korábbi fogalom visszahívása

Önellenőrző kérdések. (1) Miért marad a félig nyílt intervallum jelleg a resize alatt? (2) Mi történik, ha a crop ablak teljesen levág egy dobozt? (3) Melyik hibakategóriába esik az a predikció, amelyik ugyanazt a GT-t másodikként fedi le $\mathrm{IoU} \geq \tau$ értékkel?

Válaszok. (1) A félig nyílt intervallum képe félig nyílt, mert a pozitív skalárral való szorzás monoton bijekció. (2) A doboz kikerül a targetből, M csökken; a kieső objektum már nem szerepel a kiértékelési targetben, ezért nem számít missed detectionnek. A missed detection a targetben szereplő, predikcióval nem párosított GT; a levágás adatminőségi kérdés, nem detektálási hiba. (3) Duplicate detection, amennyiben a greedy matching után a második predikció FP-ként jelenik meg, de a hibajegyzékben a közeli azonos GT miatt duplicate-ként tartjuk számon.

Visszahívás. A 26.3. NMS, confidence és AP egységben a greedy matching, az NMS és az AP fogalmát építettük fel; a mostani hibajegyzék az ottani matching konvenciót használja. A 22.4. Augmentációs kísérlet és domainváltozás egység a train augmentációt és a tesztkorrupciót különítette el: itt a train synchronized augmentáció címkemegőrző, a test transzformáció pedig determinisztikus és befagyasztott modellre vonatkozik. A 25.4. Nyílt halmaz és ismeretlen input egység a nyílt halmazú döntést tárgyalta; a detektálásban a negatív, üres kép hasonló szerepet

tölt be, mint az ismeretlen input az osztályozóban, mert a “nincs objektum” explicit kimenet. ### Rövid összefoglaló

A detektálási pipeline a képet és a dobozokat szinkronizáltan transzformálja, hogy a koordinátarendszer mindig az aktuális képhez igazodjon. A resize a doboz sarokpontjait a képméret-aránnyal szorozza; a crop eltolásból és levágásból áll, a kieső dobozok eltűnnek. Az üres cél explicit $M = 0$ állapot, amelyet a kódnak kezelnie kell. Az objektumszintű hibajegyzék a greedy matching után missed, duplicate, localization és false positive kategóriákat ad, és az AP-t diagnosztikailag egészíti ki. A csak pozitív képeken mért precision felfelé torzít, mert kihagyja a negatív képek false positive-jait; az AP-nél a torzítás a score-rangsortól függ. A transzformációs hibák közvetlenül lokalizációs hibának látszanak, ezért a pipeline ellenőrzés a kísérlet része. A rögzített transzformációk nélküli AP-összehasonlítás félrevezető.

Az induló kérdésre a válasz: a dobozt ugyanabban a koordinátarendszerben kell transzformálni, mint a képet; ha a képet 12×12-re kicsinyítjük, de a dobozt a 24×24-es koordinátákon hagyjuk, a doboz rossz skálán értelmeződik, és az IoU torzul. A resize és a crop ezért a dobozokra is kötelező.

26.4.12 Következő alkalmazás

A 27.1. Pixelszintű klasszifikáció és FCN egység a szemantikus szegmentálásba lép át: ott a doboz helyett minden pixel kap osztálycímkét, és a synchronized transform a képet és a maszkot együtt transzformálja. A pipeline-szerződés és a hibaelemzés gondolata átvihető, de a metrika IoU-ból pixel-alapú Dice/IoU lesz.

26.4.13 Források

- [P12] Ren, S. et al. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. arXiv:1506.01497. https://arxiv.org/abs/1506.01497.
- [P13] Redmon, J. et al. (2015). *You Only Look Once: Unified, Real-Time Object Detection*. arXiv:1506.02640. https://arxiv.org/abs/1506.02640.
- [D18] PyTorch contributors (2026). *TorchVision Object Detection Finetuning Tutorial*. PyTorch 2.14 documentation. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html. Hozzáférés: 2026-10-30.
- Saját levezetés: a resize és crop doboztranszformáció lemmája és bizonyítása.
- Saját példa: a kézi számolás, a hibajegyzék szemléltető táblázata és a `resize_boxes`/`crop_boxes` kód.

27 Szemantikus és példányszintű szegmentálás

27.1 Pixelszintű klasszifikáció és FCN

27.1.1 1. Cél, előismeret és induló kérdés

Ez az alfejezet a képosztályozástól a sűrű, térbeli előrejelzés felé lép: minden pixelhez külön osztálycímkét rendelünk. A cél annak pontos rögzítése, hogyan kell a logit-tenzort, a céltenzort és a veszteséget úgy párosítani, hogy az osztálytengely, az érvénytelen pixel és a felbontásváltozás ne okozzon rejtett hibát. Előismeret: a 15.1. Súly- és biasgradiens alfejezetből a konvolúciós réteg gradiense, a 15.4. Független gradiensellenőrzési protokoll alfejezetből a gradiensellenőrzés protokollja, a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése alfejezetből a CNN-operátor specifikációja, a 22.4. Augmentációs kísérlet és domainváltozás alfejezetből a clean/corrupt szétválasztás, a 23.4. Hibaszeletek és alkalmazási határ alfejezetből a hibaszeletek, a 26.1. Doboz-koordináta, IoU és lokalizációs loss alfejezetből a doboz és az IoU, a 26.4. Detektálási pipeline és hibaelemzés alfejezetből a képpel szinkronban transzformált annotáció. A korábbi 7.2. Softmax és többosztályos cross-entropy alfejezetben levezettük a softmax–cross-entropy gradiensét; itt csak a pixelszintű általánosítást és a maszkolt átlagot vezetjük le.

Induló kérdés: egy 24×24-es, egycsatornás GlyphLab-képen (a függelék A. szakasza: A. A GlyphLab adatkészlet) képszintű háromosztályú orientációcímke (függőleges, vízszintes, átlós) és pixelszintű bináris előtér/háttér maszk tartozik ugyanahhoz a képhez; a két cél nem tévesztendő össze. Az alábbi számpéldában háromosztályú dense predictiont vizsgálunk, a labor szegmentációs célja viszont bináris. Hány logitot állít elő egy egyképnyi batch a $K = 3$ példában, és hány független osztályozási döntést reprezentál? Becslését ellenőrizzük a következő szakasz alakdefiníciója után.

27.1.2 2. Dense prediction, FCN és a tenzoralak-konvenció

Definíció (dense prediction, sűrű előrejelzés). Legyen $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ képbatch. Egy f modell *dense prediction* kimenetet ad, ha minden n mintához egy K csatornás, $H_{out} \times W_{out}$ térbeli rácson értelmezett logit-tenzort számol:

$$Z = f(X) \in \mathbb{R}^{N \times K \times H_{out} \times W_{out}}.$$

Minden (n, i, j) pozícióhoz egy K-dimenziós logitvektor tartozik. A hagyományos képosztályozó ezzel szemben $Z \in \mathbb{R}^{N \times K}$ alakú, tehát a térbeli kiterjedést globálisan eliminálja. A dense prediction megtartja a térbeli szerkezetet, ezért a kimenet közvetlenül összehasonlítható egy pixelcímke-ráccsal.

Definíció (fully convolutional network, FCN). FCN-nek nevezzük azt a hálózatot, amelynek számítási gráfja konvolúciós, pooling- és elemenkénti műveletekből, valamint felbontást növelő interpolációból vagy transposed convolutionből áll, és amely nem tartalmaz teljesen összekötött, az összes térbeli pozíciót egyetlen vektorba keverő réteget. A dense prediction és az FCN konvolúciós hálózatra építő eredeti megfogalmazása Long, Shelhamer és Darrell (2015) munkája; itt a

definíciót a tenzoralakok szempontjából pontosítjuk. A fogalmat itt architekturális konvencióként használjuk: az N mintadimenziótól eltekintve minden művelet megőrzi vagy szabályosan változtatja a térbeli rácsot.

A céltenzor alakja:

$$Y \in \{0, 1, \dots, K-1\}^{N \times H_{out} \times W_{out}},$$

ahol $Y_{n,i,j}$ az n-edik kép (i, j) pixelének osztályindexe. Az osztálytengely a logitban a második dimenzió, NCHW konvencióban az 1 indexű tengely; a céltenzornak nincs osztálytengelye, mert indexeket tárol. A betanított modell az

$$\hat{Y}_{n,i,j} = \arg \max_{c \in \{0,\dots,K-1\}} Z_{n,c,i,j}$$

predikciót adja. A softmax a class axis (osztálytengely) mentén normalizál pixelről pixelre:

$$P_{n,c,i,j} = \frac{\exp(Z_{n,c,i,j})}{\sum_{c'=0}^{K-1} \exp(Z_{n,c',i,j})}, \qquad \sum_{c=0}^{K-1} P_{n,c,i,j} = 1.$$

Ezzel az induló kérdésre a válasz: $N = 1, K = 3, H_{out} = W_{out} = 24$ esetén $1 \cdot 3 \cdot 24 \cdot 24 = 1728$ logit készül, ami $24 \cdot 24 = 576$ darab, egymástól függetlenül normalizált háromosztályú döntés.

Az FCN-ek közül a szegmentáló encoder–decoder szerkezet klasszikus példája az U-Net [P14]; az ellenőrzött absztrakt szerint egy kontraháló, kontextust gyűjtő útból és egy szimmetrikus expandáló útból áll, amely a pontos lokalizációt támogatja. A példányszintű szegmentálás felé a Mask R-CNN [P15] absztraktja egy objektumdetektorra épülő, szegmentációs maszkot előrejelző ágat mutat be; az utóbbi a detektálás feltételéhez kötött, ezért nem azonos az itteni, teljes képre kiterjedő pixelszintű FCN-konvencióval.

27.1.3 3. Per-pixel cross-entropy és maszkolt átlagolás

Egy pixel vesztesége a többosztályos cross-entropy logitformája:

$$\ell_{n,i,j} = -\log P_{n,Y_{n,i,j},i,j} = -\log \frac{\exp(Z_{n,Y_{n,i,j},i,j})}{\sum_{c=0}^{K-1} \exp(Z_{n,c,i,j})}.$$

A pixelszintű célfüggvény nem a batchméret, hanem az érvényes pixelpozíciók szerinti átlag. Ehhez legyen

$$M \in \{0, 1\}^{N \times H_{out} \times W_{out}}$$

ismert érvényességmaszk, ahol $M_{n,i,j} = 1$ jelenti, hogy az (n, i, j) célcímke felhasználható. Az érvényes halmaz

$$\mathcal{V} = \{(n, i, j) : M_{n,i,j} = 1\}, \qquad S = |\mathcal{V}|.$$

Lemma (maszkolt átlagos pixel-loss nevezője és gradiense). Legyen minden pixelre $\ell_{n,i,j}$ differenciálható függvénye a $Z_{n,:,i,j}$ logitvektornak, és legyen M a logitoktól független, előre rögzített maszk. Ha $S > 0$, a maszkolt átlagos pixel-loss

$$\mathcal{L}_M = \frac{1}{S} \sum_{(n,i,j)\in\mathcal{V}} \ell_{n,i,j} = \frac{\sum_{n=0}^{N-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} M_{n,i,j} \ell_{n,i,j}}{\sum_{n=0}^{N-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} M_{n,i,j}}$$

nevezője az érvényes pixelek S száma, nem a teljes $NH_{out}W_{out}$ elemszám. A gradiens

$$\nabla_{Z_{n,c,i,j}} \mathcal{L}_M = \frac{M_{n,i,j}}{S} \nabla_{Z_{n,c,i,j}} \ell_{n,i,j}.$$

Ha $S = 0$, a $0/0$ külön ágat igényel. A saját átlagunk konvenciója $\mathcal{L}_M = 0$, de ez nem automatikusan a PyTorch `CrossEntropyLoss(reduction="mean", ignore_index=…)` viselkedése: ha minden cél ignorált, az `NaN`-t ad (a PyTorch 2.15.0+cpu-ban, `CrossEntropyLoss(ignore_index=-100)`-dal, minden célt ignorálva ellenőrizve; az eredmény `nan`; a `CrossEntropyLoss` dokumentált viselkedését lásd [D07]). Implementáció: ha $S > 0$, `(M*ell).sum()/S`; ha $S = 0$, `logits.sum()*0.0`, amely véges logitok esetén differenciálható nulla (a gradiens nulla, nem `NaN`); nem véges logitoknál ez nem garantált, ezért ilyenkor külön kezelés szükséges. Külön döntés, hogy ilyen batchnél az optimizer-lépést kihagyjuk-e. Osztálysúlyozott cross-entropynál a nevező a célosztály-súlyok összege, $\sum_{(n,i,j)\in\mathcal{V}} w_{Y_{n,i,j}}$, nem feltétlenül az érvényes pixelek S száma.

Bizonyítás. A maszk elemei rögzített skalárok, ezért $\mathcal{L}_M$ a $\ell_{n,i,j}$ tagok lineáris kombinációja: $\mathcal{L}_M = \sum_{n,i,j} \alpha_{n,i,j} \ell_{n,i,j}$, ahol $\alpha_{n,i,j} = M_{n,i,j}/S$. A deriválás lineáris, ezért az összeg tagonként deriválható. Az egyes tagok a vizsgált $Z_{n,c,i,j}$-nek nem összetett függvényei: $\ell_{n',i',j'}$ csak a saját pixelének logitvektorától függ, ezért az összeg deriválásához láncszabály nem kell; a pixelveszteség logiton belüli deriváltját a későbbi softmax–cross-entropy formula adja:

$$\frac{\partial \mathcal{L}_M}{\partial Z_{n,c,i,j}} = \sum_{n',i',j'} \alpha_{n',i',j'} \frac{\partial \ell_{n',i',j'}}{\partial Z_{n,c,i,j}} = \alpha_{n,i,j} \frac{\partial \ell_{n,i,j}}{\partial Z_{n,c,i,j}},$$

mert a $\ell_{n',i',j'}$ pixelveszteség csak a saját $Z_{n',:,i',j'}$ logitjaitól függ, a (c,i,j) szerinti parciális derivált pedig csak az azonos (n,i,j) indexű tagban nem nulla. Ezzel a gradiensformula adódik.

A softmax–cross-entropy gradiensét a 7.2. Softmax és többosztályos cross-entropy alfejezetben vezettük le; pixelszinten ugyanaz a formula érvényes minden (n,i,j)-re:

$$\nabla_{Z_{n,c,i,j}} \ell_{n,i,j} = P_{n,c,i,j} - \mathbf{1}[Y_{n,i,j} = c].$$

A lemmával összetéve:

$$\boxed{\nabla_{Z_{n,c,i,j}} \mathcal{L}_M = \frac{M_{n,i,j}}{S} (P_{n,c,i,j} - \mathbf{1}[Y_{n,i,j} = c]).}$$

Az érvénytelen pixeleken a gradiens nulla, az érvényeseken a szokásos softmax-hiba osztva az érvényes pixelszámmal. Ha minden pixel érvényes, $M \equiv 1$ és $S = NH_{out}W_{out}$, tehát a maszkmentes átlagot kapjuk vissza. A sum-redukciós változat, $\mathcal{L}_{\text{sum}} = \sum M\ell$, ugyanennek S-szerese; az optimalizáló lépésnagyság szempontjából ez nem közömbös, ezért a kísérleti naplóban mindig közölni kell a választott redukciót.

A gyakorlatban az érvénytelenséget gyakran célcímke-szimbólummal jelölik, nem külön maszkkal. Legyen g az *ignore index* (figyelmen kívül hagyandó index); ekkor az $Y_{n,i,j} = g$ értékű pixelek nem vesznek részt az átlagban. Ez ekvivalens a maszkkal:

$$M_{n,i,j} = \mathbf{1}[Y_{n,i,j} \neq g].$$

A lemmánk feltétele teljesül: M a rögzített célból számolt, nem a modell kimenetéből. A keretrendszer pontos reduction (redukciós) viselkedését a telepített verzió dokumentációjában kell ellenőrizni; a fenti levezetés az általunk elvárt numerikus konvenciót rögzíti. A `mean` az érvényes elemek átlaga, a `sum` az érvényes elemek összege, a `none` pedig megőrzi a $N \times H_{out} \times W_{out}$ alakú, S-nel osztatlan pixelveszteség-rácsot.

Resize mode és class axis. Ha a hálózat kimeneti rácsa kisebb a célrácsnál, például stride-ok miatt, két út közül választhatunk. A helyes irány az, hogy a logit-tenzort interpoláljuk a cél felbontására a térbeli dimenziók mentén, az osztálytengelyt csatornának kezelve. A diszkrét célcímkéket ezzel szemben nem interpoláljuk. A logitinterpoláció módját explicit módon rögzíteni kell; a leggyakrabban bilineáris interpolációt használjuk a K csatorna mindegyikére (Long, Shelhamer és Darrell, 2015). Az interpoláció után a softmax és a veszteség a már definiált képletekkel számolható. A célrácshoz tartozó maszkot a logitokkal együtt sem interpoláljuk; ha a célcímkék és a maszk a kimeneti rács felbontásában állnak rendelkezésre, azokat abban a felbontásban kell tárolni.

27.1.4 4. Kézi számpélda

Legyen $N = 1$, $K = 3$, $H_{out} = W_{out} = 2$. A logitok:

$$Z = \begin{bmatrix} Z_{00} & Z_{01} \\ Z_{10} & Z_{11} \end{bmatrix}, \quad Z_{00} = \begin{bmatrix} 2 \\ -1 \\ 0{,}2 \end{bmatrix},\ Z_{01} = \begin{bmatrix} -0{,}5 \\ 1 \\ 0 \end{bmatrix},\ Z_{10} = \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix},\ Z_{11} = \begin{bmatrix} 1 \\ 2 \\ -2 \end{bmatrix}.$$

A célrács $Y = \begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}$, a maszk $M = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$. Az érvényes pixelszám $S = 3$.

A softmax a $(0,0)$ pixelen: exp értékek 7,389; 0,368; 1,221, összegük 8,978, így $P_{00} \approx (0{,}8230;\ 0{,}0410;\ 0{,}1360)$. A cél 0, tehát $\ell_{00} = -\log 0{,}8230 \approx 0{,}1948$.

A $(1,0)$ pixelen: exp értékek 1; 1; 20,085, összegük 22,085, $P_{10} \approx (0{,}0453;\ 0{,}0453;\ 0{,}9094)$, cél 2, $\ell_{10} \approx 0{,}0949$.

A $(1,1)$ pixelen: exp értékek 2,718; 7,389; 0,135, összegük 10,242, $P_{11} \approx (0{,}2654;\ 0{,}7214;\ 0{,}0132)$, cél 0, $\ell_{11} = -\log 0{,}2654 \approx 1{,}3266$.

A maszkolt átlag:

$$\mathcal{L}_M = \frac{0{,}1948 + 0{,}0949 + 1{,}3266}{3} \approx 0{,}5388.$$

A gradiens a lemma szerint: a $(0,0)$ pixelen $(P_{00} - e_0)/3 = (-0{,}1770;\ 0{,}0410;\ 0{,}1360)/3$; a $(0,1)$ pixelen $(0,0,0)$, mert maszkolt; a $(1,0)$ pixelen $(P_{10} - e_2)/3 = (0{,}0453;\ 0{,}0453;\ -0{,}0906)/3$; a $(1,1)$ pixelen $(P_{11} - e_0)/3 = (-0{,}7346;\ 0{,}7214;\ 0{,}0132)/3$. A három érvényes pixel hozzájárulásainak összege az osztálytengely mentén nullát ad, hiszen minden pixel egy-egy $P - e_y$ vektort kap.

A következő ellenőrizhető munkamenet ugyanezt számítja ki. A kimenet a megadott környezetben mért érték (Python 3.12.3, NumPy 2.5.4, CPU).

```
# munkakönyvtár: lab/
import numpy as np
rng = np.random.default_rng(20261020)

Z = np.array([[[[2.0, -0.5], [0.0, 1.0]],
               [[-1.0, 1.0], [0.0, 2.0]],
               [[0.2, 0.0], [3.0, -2.0]]]], dtype=np.float64)  # (1,3,2,2)
Y = np.array([[[0, 1], [2, 0]]], dtype=np.int64)              # (1,2,2)
M = np.array([[[1, 0], [1, 1]]], dtype=np.float64)            # (1,2,2)

def softmax_pixelwise(Z):
    Z_shift = Z - Z.max(axis=1, keepdims=True)
    E = np.exp(Z_shift)
    return E / E.sum(axis=1, keepdims=True)

P = softmax_pixelwise(Z)
# Az osztálytengely menti kiválasztás: P alakja (N,K,H,W), a cél Y alakja (N,H,W).
P_ch = np.transpose(P, (0, 2, 3, 1))                # (N,H,W,K)
log_p_y = np.log(np.take_along_axis(P_ch, Y[..., None], axis=3)[..., 0])
ell = -log_p_y
L_masked = np.sum(M * ell) / np.sum(M)
expected_L = 0.5387668060475693
assert abs(L_masked - expected_L) < 1e-5
print(f"S={np.sum(M):.0f}, L_masked={L_masked:.6f}")  # S=3, L_masked=0.538767
```

Mért kimenet:

```
S=3, L_masked=0.538767
```

A `Z.max` levonása csak numerikus stabilitást szolgál; a softmax értéke változatlan, a log–sum–exp túlcsordulását kerüli el. Az ellenőrző kézi értékkel való egyezést a kód `abs(L_masked - expected_L) < 1e-5` feltételével, azaz 10^{-5} abszolút tűréssel ellenőrzi, float64 mellett.

27.1.5 5. Labor: GlyphLab maszk és küszöbbaseline

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja, hogy pixelszintű céladatot töltsünk be, ellenőrizzük a szerződését, és egy triviális, erősítés nélküli baseline-t illesszünk rá. A bináris maszk nem a háromosztályú orientációcímke, hanem az előtér/háttér szegmentációs cél; a két feladat céltenzora különbözik. A küszöbös előrejelzés tehát szegmentációs baseline, nem osztályozó.

```
# munkakönyvtár: lab/
# futtatás: python n105_baseline.py
import numpy as np
from pathlib import Path

DATA = Path("data")
images = np.load(DATA / "images.npy")      # (360,1,24,24), float32, [0,1]
masks  = np.load(DATA / "masks.npy")       # (360,1,24,24), uint8, {0,1}
meta   = np.loadtxt(DATA / "metadata.csv", dtype=str, delimiter=",", skiprows=1)

print("images:", images.shape, images.dtype,
      "range", float(images.min()), float(images.max()))
print("masks :", masks.shape, masks.dtype,
      "values", np.unique(masks).tolist())

assert images.shape == (360, 1, 24, 24)
```

```
assert masks.shape == (360, 1, 24, 24)
assert masks.dtype == np.uint8
assert set(np.unique(masks)) <= {0, 1}

# A metaadatok sorrendje és a tömbindexek egyezése.
idx = meta[:, 0].astype(np.int64)
assert np.array_equal(idx, np.arange(len(images)))
split = meta[:, 4]
assert set(split) == {"train", "val", "test"}
eval_mask = np.isin(split, ["train", "val"])
counts = {s: int((split == s).sum()) for s in ("train", "val", "test")}
print("split counts:", counts)
print("eval images:", int(eval_mask.sum()))

# Baseline: fix intenzitásküszöb az egyetlen csatornán; a tesztet nem használjuk
# a küszöb kiválasztásához, ezért csak a train+val képeken értékelünk.
threshold = 0.5
pred = (images[eval_mask, 0] > threshold).astype(np.uint8)  # (300,24,24)
target = masks[eval_mask, 0]                                # (300,24,24)

pixel_acc = float(np.mean(pred == target))
inter = np.logical_and(pred == 1, target == 1).sum()
union = np.logical_or(pred == 1, target == 1).sum()
iou = float(inter / union) if union > 0 else float("nan")
print(f"threshold={threshold}, pixel_acc={pixel_acc:.6f}, IoU={iou:.6f}")
assert abs(pixel_acc - 1.0) < 1e-12
assert abs(iou - 1.0) < 1e-12
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, GlyphLab seed 20261020):

```
images: (360, 1, 24, 24) float32 range 0.0 1.0
masks : (360, 1, 24, 24) uint8 values [0, 1]
split counts: {'train': 240, 'val': 60, 'test': 60}
eval images: 300
threshold=0.5, pixel_acc=1.000000, IoU=1.000000
```

A kód a train+val halmaz minden képére és minden pixelére egyetlen átfogó metrikát számol; a tesztet a küszöb kiválasztásához nem használja. A mért kimenetben a pixel-accuracy és az IoU is 1, mert a GlyphLab-generátor előtere és háttere a 0,5 küszöbnél tökéletesen elválik: az előtér-intenzitás átlaga 0,869 (szórás 0,054, minimum 0,684), a háttéré 0,120 (szórás 0,054, maximum 0,360). Ez nem azt jelenti, hogy a küszöb jó modell: az előtér a pixeleknek csak 2,04%-a, ezért a mindent háttérnek jósoló baseline pixel-accuracy-je 0,9796 lenne, IoU-ja viszont 0. Ez mutatja, hogy a 23.4. Hibaszeletek és alkalmazási határ alfejezetben tárgyalt összesített pontszám miért nem elég: bináris szegmentálásban az osztályarány miatt a pixel-accuracy és az IoU nem cserélhető fel. A küszöböt kizárólag train/val felosztáson szabad hangolni; a test pixelarányos metrikája csak a befagyasztott küszöb mellett értelmezhető.

27.1.6 6. Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miben különbözik a képosztályozó logit-tenzor és a dense prediction logit-tenzor alakja, és adja meg NCHW konvencióban a class axis indexét. Miért nincs osztálytengelye a céltenzornak? Átadandó: saját szavas definíció és egy $N = 2, K = 4, H = 5, W = 6$ numerikus alakpélda mindkét tenzorra.

F2 (levezetési, nehézség 2). Legyen $N = 1, K = 2, H = 2, W = 1$, logitok $Z_{0,:,0,0} = (1, -1)^\top$, $Z_{0,:,1,0} = (0, 2)^\top$, cél $Y = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, maszk $M = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Számítsa ki $\mathcal{L}_M$-et és a logitokra vonatkozó teljes

gradienst a lemmából; ellenőrizze, hogy az osztálytengely mentén minden pixel gradiensösszege nulla.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt `masked_cross_entropy_mean(logits, targets, ignore_index)` aláírással, ahol `logits` alakja (N, K, H, W) float64, `targets` alakja (N, H, W) int64. A függvény az ignore_index értékű célpontokat kihagyó átlagot adja vissza. A függvény stabil softmaxot használjon, és `S=0` esetén `0.0`-t adjon. Ellenőrizze a fenti kézi példa logitjain úgy, hogy a $(0, 1)$ pozíciójú, 1 értékű célt ignorálja (`ignore_index=1`), majd ismételje meg a `-100`-as konvencióval. Ügyeljen rá, hogy az ignore_index ne eshessen egybe érvényes osztályindexszel: a `-100` a biztonságos gyakorlat, az `1` itt csak azért működik, mert a kézi példában éppen egy 1-es cél található a (0,1) pozíción.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet egy segmentációs loss kiszámítására készült:

```
# PyTorch-szerű pszeudokód; NumPyban a mask.numel() megfelelője mask.size.
loss = ((mask * pixel_loss).sum()
        / mask.numel())          # hiba: teljes elemszámmal oszt
```

Mi a hiba oka, numerikus következménye, és hogyan kell javítani? Adjon egy konkrét hárompixeles számpéldát, amelyben a hibás és a javított loss értéke eltér, és mutassa meg a javított gradiens arányát.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen egy bináris szegmentálási kísérletet a GlyphLab maszkon a küszöbbaseline-hoz: figyeljen a scene-alapú splitre, a küszöbválasztás train/val protokolljára, az IoU és a pixel-accuracy együttes jelentésére, valamint a per-scene bootstrap érvényességére. Átadandó: protokollszöveg, baseline, metrikák és a teszt használatára vonatkozó korlátozás.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

27.1.7 7. Megoldások és indoklás

F1 megoldás. A képosztályozó kimenete $Z \in \mathbb{R}^{N\times K}$: minden képhez egyetlen logitvektor tartozik, a térbeli dimenzió eltűnik. A dense prediction kimenete $Z \in \mathbb{R}^{N\times K\times H_{out}\times W_{out}}$: az osztálydöntés rögzített térbeli rácson ismétlődik. NCHW esetén a class axis a 0-tól számolt 1 indexű tengely, azaz minden (n, i, j)-re a $Z_{n,:,i,j}$ szelet az egy pixel osztálylogitja. A céltenzor $Y \in \{0, \dots, K-1\}^{N\times H_{out}\times W_{out}}$ alakú, mert minden pozícióhoz egyetlen osztályindexet tárol; az osztályt egy egész érték, nem one-hot vektor jelöli. A megadott alakpélda: Z alakja $(2, 4, 5, 6)$, Y alakja $(2, 5, 6)$.

F2 megoldás. Az első pixel $(i = 0)$: $Z = (1, -1)^\top$. $\exp(Z) = (2{,}7183;\ 0{,}3679)$, összeg 3,0862, $P = (0{,}8808;\ 0{,}1192)$. A cél 0, ezért $\ell = -\log 0{,}8808 = 0{,}1269$; a logitokra vonatkozó gradiens kézi számítással itt $P - e_0 = (-0{,}1192;\ 0{,}1192)$, osztva $S = 1$-gyel, azaz $(-0{,}1192;\ 0{,}1192)$. A második pixel maszkja 0, ezért nem számít, és gradiense $(0, 0)$. Tehát $\mathcal{L}_M = 0{,}1269$, a teljes logitgradiens a $(0, 0)$ pixelen $(-0{,}1192;\ 0{,}1192)$, minden más helyen 0. Mindkét pixelben (a $H = 2$, $W = 1$ rács két sorában) a kétosztályos gradiens komponenseinek összege 0, mert P valószínűségeloszlás és e_0 összege 1.

F3 megoldás.

```python
import numpy as np

def masked_cross_entropy_mean(logits, targets, ignore_index):
    N, K, H, W = logits.shape
    assert targets.shape == (N, H, W)
    valid = targets != ignore_index
    assert np.all((targets[valid] >= 0) & (targets[valid] < K))
    safe = np.where(valid, targets, 0)                # ignorált címke helyére 0
    logits_shift = logits - np.max(logits, axis=1, keepdims=True)
    exp_z = np.exp(logits_shift)
    denom = exp_z.sum(axis=1, keepdims=True)          # (N,1,H,W)
    log_p_y = logits_shift - np.log(denom)            # (N,K,H,W)
    # kiválasztás osztálytengely mentén:
    log_p_y_ch = np.transpose(log_p_y, (0, 2, 3, 1))  # (N,H,W,K)
    y_idx = safe[..., None]                           # (N,H,W,1)
    log_p_target = np.take_along_axis(log_p_y_ch, y_idx, axis=-1)[..., 0]  # (N,H,W)
    loss_per_pixel = np.where(valid, -log_p_target, 0.0)
    S = int(valid.sum())
    if S == 0:
        return 0.0
    return float(loss_per_pixel.sum() / S)

# teszt a kézi példával:
Z = np.array([[[[2.0, -0.5], [0.0, 1.0]],
               [[-1.0, 1.0], [0.0, 2.0]],
               [[0.2, 0.0], [3.0, -2.0]]]], dtype=np.float64)
Y = np.array([[[0, 1], [2, 0]]], dtype=np.int64)
print(masked_cross_entropy_mean(Z, Y, ignore_index=1))  # 0.538767
Y_ignore = np.where(Y == 1, -100, Y)
print(masked_cross_entropy_mean(Z, Y_ignore, ignore_index=-100))  # 0.538767
```

Az implementáció nem számol külön `M` maszkot paraméterként, hanem az ignore indexből állítja elő. Az indexelés előtt az ignorált címkéket 0-ra cseréli (`safe`), mert a `take_along_axis` különben a `-100`-at vagy bármely $[0, K)$-n kívüli értéket érvénytelen indexként olvasná, és `IndexError`-t adna; az érvénytelen, de nem ignorált címkét a függvény assertionnal jelzi. A futtatott ellenőrzés mindkét ignore-konvencióval (`1` és `-100`) ugyanazt a `0.538767` értéket adta. Az $S = 0$ ág a lemma konvenciója. A stabil log–sum–exp a `logits_shift - log(denom)` alakban marad, mert $x - \log \sum \exp(x)$ numerikusan biztonságosabb, mint $\log(P)$ külön számítása.

F4 megoldás. A hiba oka, hogy a nevező az összes pixel `mask.numel()`-e (NumPyban `mask.size`), nem az érvényes `mask.sum()`-ja. Következménye, hogy ha a maszk ritka, az átlag lefelé torzul, és a gradiens is a teljes rács, nem az érvényes halmaz nagyságával skálázódik. Konkrét példa: három pixel, $\ell = (0{,}6;\ 0{,}9;\ 0{,}3)$, $M = (1, 1, 0)$; a hibás érték $(0{,}6 + 0{,}9 + 0)/3 = 0{,}5$, a helyes $(0{,}6 + 0{,}9)/2 = 0{,}75$. A javított kód `mask.sum()`-mal oszt, és a lemmából a gradiens érvényes pontokon $1/S = 1/2$, míg a hibásnál $1/3$ lenne. Visszaellenőrzés: mindhárom pixel gradientjét az analitikus $\frac{M}{S}(P - e_y)$ képlettel összevetve a hiba megszűnik.

F5 megoldás. Egy lehetséges terv: a megfigyelési egység a `scene_id`, ezért a küszöböt a train/val scene-ekre, nem a testre hangoljuk. A baseline a képintenzitás fix küszöbös binarizálása. A search-tér $t \in \{0{,}05;\ 0{,}10;\ \ldots;\ 0{,}95\}$; a választáshoz a val halmazon számolt IoU-t használjuk, mert a pixel-accuracy a háttérdominancia miatt gyenge diszkriminátor. A kiválasztott t^*-ot a befagyasztott bináris döntési szabállyal egyszer értékeljük a testen: per-image IoU átlag, illetve az összes pixelre számolt globális IoU és pixel-accuracy, mert ezek aggregálása eltérő. A bizonytalansághoz scene-rétegzett bootstrapet használunk 2000 resample-lel és 95%-os percentilis intervallummal, ahol minden resample egység egy scene, és a GlyphLab minden jelenetéhez tartozó két nézetet (view) mindig egy blokkban tartjuk. A teszt 30 jelenete miatt az intervallum széles, ezért a pontbecslést önmagában nem használjuk döntésre. A teszt eredményéből nem módosítjuk a küszöböt; a korrupciós tesztet a 22.4. Augmentációs kísérlet és domainváltozás alfejezethez

hasonlóan csak a befagyasztott szabályon értékeljük, a clean–corrupt párokat jeleneten belül egy egységként kezelve (páros scene-szerkezet).

27.1.8 8. Tipikus tévedés, önellenőrzés

Tipikus tévedés. A diszkrét célcímke felbontásváltoztatását bilineáris interpolációval oldják meg. Ellenpélda: legyen egy kétszomszédos célpixel értéke 0 és 1. Bilineáris interpolációval köztes értékek, például 0,25 és 0,75 keletkezhetnek, amelyek sem a 0, sem az 1 osztályt nem jelentik. A cél az osztályindexek halmazán kívüli értékeket kap, a loss vagy hibaüzenettel leáll, vagy némán hamis osztályként értelmezi őket. A javítás az, hogy a logitokat interpoláljuk tetszőleges sima móddal, a címketenzorra ellenben nearest-neighbor módot használunk, vagy a megfigyelést már a célrácsban tároljuk. A szabály ugyanannak az elvnek a pixelszintű változata, mint a 26.4. Detektálási pipeline és hibaelemzés szinkrontranszformációs követelménye: a diszkrét céloknál a geometriai transzformáció nem keverhet osztályokat.

Önellenőrzés. (1) Adott Z alakja $(2, 5, 16, 16)$. Hány független osztályozási pontot tartalmaz, és hol van a class axis? Válasz: $2 \cdot 16 \cdot 16 = 512$ pont, az 1 indexű tengelyen. (2) Ha $S = 0$, mit ad a lemmánk? 0-t, de ezt kísérleti naplóban külön jelezni kell, mert elrejtheti az üres batch hibáját. (3) Korábbi fogalom visszahívása: a 15.1. Súly- és biasgradiens alfejezetben levezetett biasgradiens a batch és térbeli indexek szerinti upstream-összeg volt. A pixelszintű lossban a pixel-gradiensek hasonlóan adódnak össze a konvolúciós backward során. Vigyázni kell az átlagolások sorrendjével: a maszkolt pixel-loss az érvényes pixelek S számával oszt, majd a minibatch-átlagolás a batch elemszámával; ha a két átlagolás egymás után fut, a gradiens skálája a két nevező szorzatával oszlik. Ezért minden átlagoló lépésnél rögzíteni kell, hogy a nevező S, a batchméret vagy a teljes pixelrács-e.

Visszahívás (26.1. Dobozkoordináta, IoU és lokalizációs loss, 23.1. Confusion matrix és aggregálási konvenció). Az IoU a pixelszintű előrejelzésben is a metszet/unió hányados, a dobozos definícióval azonos arányfogalom; a fél-nyílt koordinátakonvenció viszont itt már nem kétdimenziós intervallumra, hanem a diszkrét pixelhalmaz elemszámára vonatkozik. A 23.1. Confusion matrix és aggregálási konvenció tanulsága alapján a pixel-accuracy globális átlag, az IoU osztályszerű halmazátfedés: kiegyensúlyozatlan foreground esetén a két metrika egymásnak ellentmondó következtetésre vezethet.

27.1.9 Rövid összefoglaló

A dense prediction modell $N \times K \times H_{out} \times W_{out}$ logitrácsot ad, ahol minden (n, i, j) pixel külön osztályozási pont, a class axis NCHW-ban az 1. A pixel-cross-entropy az érvényes célpixelek átlaga: $\mathcal{L}_M = \frac{1}{S} \sum M\ell$, $S = \sum M$. A lemma bizonyítja, hogy a nevező az érvényes pixelszám, a gradiens pedig $\frac{M}{S}(P - e_y)$. Az ignore index egy adott célból származó maszkkal ekvivalens. A logitokat bilineárisan interpolálhatjuk a célrácsra; a diszkrét címkéket csak nearest módon szabad transzformálni. A labor a GlyphLab maszkon fix küszöbbaseline-t illeszt, és a pixel-accuracy mellé IoU-t tesz, mert a háttérdominancia félrevezető pontszámot adhat.

27.1.10 Következő alkalmazás

A 27.2. U-Net, encoder–decoder és skip egység a pixelszintű kimenet konkrét architektúráját építi fel: a kódoló–dekódoló szerkezetet és a skip-összefűzést, amely a korai térbeli részleteket juttatja el a kimeneti rácsra. Az itteni tenzoralak-konvenció (NCHW, class axis az 1. tengelyen, logit a célrácsra interpolálva) az ottani rétegalak-levezetés közvetlen előfeltétele.

27.1.11 Felhasznált források

- [P14] Ronneberger, Fischer és Brox (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. https://arxiv.org/abs/1505.04597.
- [P15] He et al. (2017). *Mask R-CNN*. https://arxiv.org/abs/1703.06870.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- Long, J., Shelhamer, E. és Darrell, T. (2015). *Fully Convolutional Networks for Semantic Segmentation*. CVPR. A dense prediction és az FCN eredeti megfogalmazásának, valamint a bilineáris logitinterpolációnak a forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

A masked mean lemma és a kézi példa saját levezetés; a labor futási kimenete a szövegben megadott, mért eredmény (Python 3.12.3, NumPy 2.5.4, CPU).

27.2 U-Net, encoder–decoder és skip

27.2.1 1. Cél, előismeret és induló kérdés

A 27.1. Pixelszintű klasszifikáció és FCN egységben a szegmentálást pixelszintű osztályozásként definiáltuk: a modell minden pixelre logitot ad, a tanítás per-pixel cross-entropy vagy bináris logitveszteséggel történik. Ott a kérdés az volt, hogyan kell a logitok és a maszk tengelyeit párosítani. Most azt vizsgáljuk, *milyen hálózati szerkezet* állítja elő a térbeli kimenetet anélkül, hogy a felbontást azonnal eldobnánk. A U-Net az egyik leggyakrabban használt encoder–decoder architektúra; benne a downsampling kontextust gyűjt, az upsampling a felbontást állítja helyre, a skip-kapcsolat pedig a korai, nagy felbontású aktivációkat közvetlenül a dekódolóhoz vezeti.

Előismeret: a 15.2. Inputgradiens és átfedő recepciós mezők alfejezetből a konvolúciós inputgradiens scatter-add indexfeltétele, a 15.3. Transposed convolution mint adjungált alfejezetből a transzponált konvolúció mint adjungált operátor értelmezése, a 23.2. Kalibráció és döntési küszöb alfejezetből a logit és döntési küszöb különválasztása, a 26.2. Egy- és kétlépcsős detektor alfejezetből pedig a detektálási fejek felelősségének szétválasztása. A U-Net itt nem javaslattevő rendszer: maszkot jósol.

Induló kérdés. Egy bemeneti kép 24×24-es, egy csatornás. A kódoló két 2×2-es max-pool-lépéssel 6×6-ra csökkenti a felbontást. A dekódoló két tanulható upsampling-lépést használ. Mekkora lesz a kimenet, és hány csatornája van a második konkatenációnál, ha a skip-ek a konkatenációk sorrendjében 16 és 8 csatornásak? Becslés után a 3. szakasz levezetése ad pontos választ.

27.2.2 2. Definíciók, jelölések és tenzoralakok

Legyen a bemeneti képbatch $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, ahol a főútban $C_{in} = 1$, $H = W = 24$. A bináris szegmentálási maszk $Y \in \{0,1\}^{N \times 1 \times H \times W}$: minden pixelhez egyetlen logit tartozik. A hálózat kimenete $Z \in \mathbb{R}^{N \times 1 \times H \times W}$, ahol $Z[n,0,i,j]$ a (n,i,j) pixel előtér-logitja, szigmoid előtt.

A fejezet szemantikus szegmentálást tárgyal: a kimenet pixelenként egyetlen osztálycímke, ezért az egymástól elkülönülő, de azonos osztályú objektumpéldányokat nem választja szét. A példányszintű (instance) szegmentálás ezzel szemben objektumonként külön maszkot ad; ennek

detektorra épülő példája a Mask R-CNN [P15], amelyet itt nem építünk fel, csak a feladatkör elhatárolására hivatkozunk.

Encoder alatt azt a részhálózatot értjük, amely a térbeli felbontást csökkenti, miközben a csatornaszámot növeli. **Decoder** az a részhálózat, amely a felbontást ismét növeli és a csatornaszámot csökkenti. **Skip concat** a kódoló egy közbülső aktivációjának és a dekódoló azonos térbeli felbontású aktivációjának csatorna-irányú összefűzése:

$$Z_{cat} = \mathrm{cat}(U, S) \quad \text{a } C \text{ dimenzió mentén}, \qquad Z_{cat} \in \mathbb{R}^{N\times(C_U+C_S)\times H'\times W'}.$$

A `torch.cat` hívásban ez `dim=1`, mert a tenzoralak NCHW.

Upsampling minden olyan leképezés, amely H, W térbeli méretet növel. Két fő változatot különböztetünk meg. A nem tanulható interpoláció, például `F.interpolate(..., mode="bilinear")`, rögzített szabállyal becsli a közbülső értékeket; ennek nincs paramétere. A tanulható `ConvTranspose2d` a konvolúciós operátor adjungáltjaként szór szét aktivációkat; ez nem általános inverz, hanem lineáris leképezés, amely a 15.3. Transposed convolution mint adjungált alfejezetben tárgyalt alakkal rendelkezik. A jelen fejezetben $k = 2$, $s = 2$, $p = 0$ transzponált konvolúciót használunk, amely minden H-t $2H$-ra növel.

Spatial alignment az a feltétel, hogy a skip tenzor $H_S \times W_S$ kiterjedése egyenlő legyen az upsampling kimenet $H_U \times W_U$ kiterjedésével a `cat` előtt. Az azonos tenzorméret szükséges, de önmagában nem elég: az elemeknek ugyanazokat a képi koordinátákat kell reprezentálniuk is (eltérő padding, páros kernel, crop vagy resize-konvenció azonos shape mellett is eltolhatja az origót). Ha a méret nem egyenlő, akkor crop vagy pad szükséges; a mi főpéldánkban a 24, 4-gyel osztható bemenet miatt a feltétel teljesül.

27.2.3 3. Alaklevezetés egy kis U-Netre

A levezetés célja egy konkrét, teljesen megadott kis U-Net minden térbeli alakjának és a két concat csatornaszámának kiszámítása. A modellt `TinyUNet`-nek nevezzük; ez a könyv saját tanulópéldája, nem az eredeti publikáció másolata. Térbeli konvolúciók: $k = 3$, padding $p = 1$, stride $s = 1$, dilation $d = 1$. Pool: 2×2 max, stride 2. Upsampling: $k = 2$, $s = 2$, $p = 0$ transposed conv.

Számítás (alak- és csatornaegyezés a TinyUNetben). Legyen $X \in \mathbb{R}^{N\times 1\times 24\times 24}$. A fenti paraméterezésű TinyUNet esetén

1. a kódoló kimenetei rendre 24×24, 12×12, 6×6 térbeli méretűek;
2. az első upsampling kimenete 12×12, a skip *összefűzése* (concatenation, nem elemenkénti összeadás) után a csatornaszám $16 + 16 = 32$;
3. a második upsampling kimenete 24×24, a skip *összefűzése* (concatenation) után a csatornaszám $8 + 8 = 16$ (elemenkénti összeadásnál 8 csatorna maradna; a ResNet additív skipjétől ez különbözik);
4. ezért egyik concat előtt sem kell térbeli crop vagy pad.

Levezetés. A konvolúciós kimeneti méret képlete 14.1. Stride, padding, dilation és outputméret jelölésével, $d = 1$ mellett:

$$H_{out} = \left\lfloor \frac{H + 2p - (k-1) - 1}{s} \right\rfloor + 1.$$

A $k = 3$, $p = 1$, $s = 1$ esetben

$$H_{out} = \left\lfloor \frac{H + 2 - 2 - 1}{1} \right\rfloor + 1 = \lfloor H - 1 \rfloor + 1 = H,$$

tehát az ilyen konvolúció a térbeli méretet megtartja. A 2×2-es, stride-2 max-pool képlete $p = 0$, $k = 2$ esetén

$$H_{out} = \left\lfloor \frac{H - 2}{2} \right\rfloor + 1.$$

Innen $H = 24$ esetén $H_{out} = 12$, majd $H = 12$ esetén $H_{out} = 6$. A transzponált konvolúció $p = 0$, $d = 1$, $k = 2$, $s = 2$ paraméterekkel a 15.3. Transposed convolution mint adjungált alfejezetben levezetett képlet szerint

$$H_{out} = (H - 1)s - 2p + d(k - 1) + \text{output_padding} + 1 = (H - 1) \cdot 2 + 1 + 0 + 1 = 2H.$$

Ezért a 6×6 bottleneck-ből 12×12, majd a 12×12-ből 24×24 lesz. A `cat` a csatornadimenzióban ad össze: az első upsampling 16, a hozzá tartozó skip 16 csatornás, így 32; a második upsampling 8, a skip 8 csatornás, így 16. A térbeli méretek a fenti értékek szerint rendre megegyeznek, tehát az alignment feltétel teljesül.

A teljes rétegenkénti alak a következő táblázatban:

Blokk	Réteg	Kimeneti alak $N \times C \times H \times W$
encoder szint 1	`enc1a`, `enc1b`	$N \times 8 \times 24 \times 24$
pool	`pool1`	$N \times 8 \times 12 \times 12$
encoder szint 2	`enc2a`, `enc2b`	$N \times 16 \times 12 \times 12$
pool	`pool2`	$N \times 16 \times 6 \times 6$
bottleneck	`bot_a`, `bot_b`	$N \times 32 \times 6 \times 6$
decoder szint 1	`up1`	$N \times 16 \times 12 \times 12$
skip concat 1	`cat(up1, enc2b)`	$N \times 32 \times 12 \times 12$
decoder conv	`dec1a`, `dec1b`	$N \times 16 \times 12 \times 12$
decoder szint 2	`up2`	$N \times 8 \times 24 \times 24$
skip concat 2	`cat(up2, enc1b)`	$N \times 16 \times 24 \times 24$
decoder conv	`dec2a`, `dec2b`	$N \times 8 \times 24 \times 24$
maszkfej	`head`	$N \times 1 \times 24 \times 24$

A táblázat egyben kézi példa is: a 12×12-es skip és a 12×12-es up-aktiváció csatorna szerint 32-re bővül, majd a 24×24-es szinten 16-ra. A bottlenecknél a térbeli felbontás 6×6, a csatorna pedig 32: ez a modell legtöbb csatornát tartalmazó reprezentációja.

27.2.4 4. Eredeti kézi példa és minimális kód

A levezetést a következő, teljesen futtatható PyTorch-modullal ellenőrizhetjük. A modul minden 3×3-as `Conv2d` rétegénél `padding=1`-et használ; a maszkfej 1×1-es, padding nélküli konvolúció. A két `ConvTranspose2d` `kernel_size=2, stride=2` paraméterű, ezért a modul a fenti lemmát valósítja meg.

```python
# tiny_unet_shapecheck.py
import torch
import torch.nn as nn

class TinyUNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1, base=8):
        super().__init__()
        # encoder, 1. szint: 1 -> 8 csatorna, 24x24 megmarad
        self.enc1a = nn.Conv2d(in_channels, base, 3, padding=1)
        self.enc1b = nn.Conv2d(base, base, 3, padding=1)
        self.pool1 = nn.MaxPool2d(2)
        # encoder, 2. szint: 8 -> 16, 12x12 megmarad
        self.enc2a = nn.Conv2d(base, 2 * base, 3, padding=1)
        self.enc2b = nn.Conv2d(2 * base, 2 * base, 3, padding=1)
        self.pool2 = nn.MaxPool2d(2)
        # bottleneck: 16 -> 32, 6x6
        self.bot_a = nn.Conv2d(2 * base, 4 * base, 3, padding=1)
        self.bot_b = nn.Conv2d(4 * base, 4 * base, 3, padding=1)
        # decoder, 1. szint
        self.up1 = nn.ConvTranspose2d(4 * base, 2 * base, 2, stride=2)
        self.dec1a = nn.Conv2d(4 * base, 2 * base, 3, padding=1)  # 16 skip + 16 up
        self.dec1b = nn.Conv2d(2 * base, 2 * base, 3, padding=1)
        # decoder, 2. szint
        self.up2 = nn.ConvTranspose2d(2 * base, base, 2, stride=2)
        self.dec2a = nn.Conv2d(2 * base, base, 3, padding=1)      # 8 skip + 8 up
        self.dec2b = nn.Conv2d(base, base, 3, padding=1)
        # maszkfej: logit (sigmoid előtti érték), nem valószínűség
        self.head = nn.Conv2d(base, out_channels, 1)

    def forward(self, x):
        e1 = torch.relu(self.enc1b(torch.relu(self.enc1a(x))))       # 8x24x24
        e2 = torch.relu(self.enc2b(torch.relu(self.enc2a(self.pool1(e1)))))  # 16x12x12
        b  = torch.relu(self.bot_b(torch.relu(self.bot_a(self.pool2(e2)))))  # 32x6x6
        u1 = self.up1(b)                                             # 16x12x12
        d1 = torch.relu(self.dec1b(torch.relu(self.dec1a(torch.cat([u1, e2], dim=1)))))
        u2 = self.up2(d1)                                            # 8x24x24
        d2 = torch.relu(self.dec2b(torch.relu(self.dec2a(torch.cat([u2, e1], dim=1)))))
        return self.head(d2)                                         # 1x24x24 logit
```

A modell célja a rétegspecifikáció ellenőrzése, nem a tanítás. A táblázat és a paraméterszám kézi levezetés eredménye, amelyet a lenti futtatás ellenőriz. A paraméterszám kézi levezetéssel is megkapható a rétegenkénti $(C_{in}C_{out}k_hk_w + C_{out})$ összegből; a fenti modulra ez 29 321. Ez magában foglalja a bias-eket is. A `dec1a` bemeneti csatornája azért 32, mert a concat után $16 + 16$ csatorna érkezik; a `dec2a` bemenete pedig $8 + 8 = 16$.

27.2.5 5. Labor: futtatható rétegspecifikáció és mért kimenet

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja annak igazolása, hogy a lemmában szereplő minden térbeli alak és concat-csatornaszám ténylegesen megvalósul a kódban. A következő ellenőrzőkód közvetlenül a fenti modul után illeszthető:

```python
if __name__ == "__main__":
    torch.manual_seed(20261020)
    device = torch.device("cpu")
    model = TinyUNet(in_channels=1, out_channels=1, base=8).to(device)

    x = torch.randn(2, 1, 24, 24, dtype=torch.float32, device=device)
```

```
    with torch.no_grad():
        model.eval()
        y = model(x)

    print("input", tuple(x.shape))
    print("output", tuple(y.shape))
    print("parameters", sum(p.numel() for p in model.parameters()))

    assert tuple(y.shape) == (2, 1, 24, 24)
    assert sum(p.numel() for p in model.parameters()) == 29321
```

Futtatás a projektkönyvtárból:

```
python tiny_unet_shapecheck.py
```

Mért kimenet — a kód a megadott környezetben lefutott (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, `torch.manual_seed(20261020)`, `torch.get_num_threads() = 10`); az assertek a futás során ellenőrizték az alakot és a paraméterszámot:

```
input (2, 1, 24, 24)
output (2, 1, 24, 24)
parameters 29321
```

Az eredmény értelmezése: a kimenet $N \times 1 \times 24 \times 24$, tehát a hálózat valóban dense predictiont ad: minden bemeneti pixelhez egy logit tartozik. A `model.eval()` hívás itt azért szerepel, hogy a kiértékelési mód szokását rögzítsük; a modulban nincs BatchNorm vagy dropout, így a train/eval viselkedés most numerikusan nem tér el. A modell paraméterei alapértelmezés szerint `float32` típusúak, ezért külön `to(torch.float32)` hívás nem szükséges. A tanításhoz a maszkot `float32` típusra konvertálva `torch.nn.functional.binary_cross_entropy_with_logits(logits, mask_float)` hívható; a logitokból a döntési maszk $\hat{Y} = \mathbf{1}[\sigma(Z) > 0{,}5]$ szabállyal áll elő, a 23.2. Kalibráció és döntési küszöb alfejezetben tárgyalt küszöb–kalibráció szétválasztás jegyében.

27.2.6 6. Feladatok

F1 (fogalmi, nehézség 1). Adja meg két konkrét különbséget a bilineáris `F.interpolate`-alapú upsampling és a `ConvTranspose2d`-alapú upsampling között a tanulhatóság és az adjungált jelleg szempontjából. Előfeltétel: 15.3. Transposed convolution mint adjungált. Átadandó: legfeljebb fél oldalas saját válasz, amely mindkét állítást a jelen modul példájára vonatkoztatja.

F2 (levezetési, nehézség 2). Vezesse le a fenti TinyUNet rétegalakjait $H = W = 22$ bemenetre. Hol sérül a spatial alignment, és mekkora a második upsampling kimenete? Előfeltétel: a 3. szakasz formulái. Átadandó: a pool- és upsampling-méretek lépésenként, a szükséges crop mérete, valamint annak kimondása, hogy a kimenet most nem 22×22.

F3 (implementációs, nehézség 1). Írjon `center_crop(x, target_h, target_w)` függvényt, amely az utolsó két dimenziót a megadott méretre vágja középre, és tesztelje $x \in \mathbb{R}^{2\times 8\times 23\times 23}$, `target_h=target_w=22` bemenetre. Átadandó: kód, futtatási parancs és az ellenőrzött kimeneti alak.

F4 (hibakeresési, nehézség 2). A következő modulrészlet az első decoder szint definíciója:

```
self.up1 = nn.ConvTranspose2d(32, 16, 2, stride=2)
self.dec1a = nn.Conv2d(16, 16, 3, padding=1)
# ...
d1 = torch.cat([self.up1(b), e2], dim=1)
d1 = torch.relu(self.dec1a(d1))
```

Mi a hiba, mi a közvetlen következménye, és hogyan javítható? Átadandó: ok, következmény, a javított sor és a visszaellenőrzés módja.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen GlyphLab-kísérletet (a függelék A. szakasza: A. A GlyphLab adatkészlet), amely a skip-kapcsolatok hatását vizsgálja a szegmentálásra. A cél: ugyanaz a TinyUNet skip-pel és skip nélkül. A maszk a `data/masks.npy` előtér-maszkja. Adjon meg: megfigyelési egység, split, baseline, változó tényező, loss, metrika, a választás szabálya és a következtetés határa. Előfeltétel: 27.1. Pixelszintű klasszifikáció és FCN, 1.3. Adatfelosztás és a megfigyelési egység. Átadandó: protokoll, benne a döntési szabállyal és a következtetés határával.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

27.2.7 7. Megoldások és indoklás

F1 megoldás. A bilineáris interpoláció paraméter nélküli, determinisztikus leképezés: a kimeneti érték a bemeneti értékek rögzített súlyozott átlaga, ezért a hálózat a tanítás során nem tudja a feladathoz igazítani. A `ConvTranspose2d` ezzel szemben tanulható kernelű lineáris leképezés. Második különbség: a `ConvTranspose2d` a megfelelő konvolúciós lineáris operátor adjungáltja, tehát a forward képlete a 15.3. Transposed convolution mint adjungált alfejezetben levezetett scatter-add szerkezetű. A bilineáris interpoláció is lineáris leképezés, ezért formálisan szintén létezik adjungáltja (egy leszűkítő operátor); a különbség az, hogy a bilineáris változat nem egy tanulható konvolúció adjungáltja, és a súlyai nem tanulhatók. A jelen példában a `up1` $32 \to 16$ csatornás tanulható leképezés, miközben a bilineáris változat legfeljebb a meglévő csatornákat interpolálná, csatornát nem keverne.

F2 megoldás. $H = 22$ esetén a kódolási lépések:

$$22 \xrightarrow{\text{conv } 3\times3,\ p=1} 22 \xrightarrow{\text{pool}} \left\lfloor \frac{22-2}{2} \right\rfloor + 1 = 11 \xrightarrow{\text{pool}} \left\lfloor \frac{11-2}{2} \right\rfloor + 1 = 5.$$

Az első upsampling 5-ből

$$(5-1)\cdot 2 + 1 + 1 = 10$$

méretet állít elő, a skip viszont 11×11, tehát 1 pixel eltérés van: a skip-ből 10×10-re kell vágni. A concat csatornaszáma változatlanul $16 + 16 = 32$. A második upsampling 10-ből 20-at állít elő, a skip itt 22×22, tehát 2 pixelnyi eltérés van. A kimenet ezért 20×20, nem 22×22. A pontos crop-policy implementációfüggő; a levezetés állítása csak a méreteltérés és a kimeneti felbontás. Ez mutatja a lemma feltételének szerepét: a 24-es bemenet azért igazodik, mert $24/4 = 6$ egész.

F3 megoldás.

```
def center_crop(x, target_h, target_w):
    h, w = x.shape[-2:]
    top = (h - target_h) // 2
    left = (w - target_w) // 2
    return x[..., top:top + target_h, left:left + target_w]

# teszt
import torch
```

```
x = torch.zeros(2, 8, 23, 23)
y = center_crop(x, 22, 22)
assert tuple(y.shape) == (2, 8, 22, 22)
print("input", tuple(x.shape))
print("output", tuple(y.shape))
```

Futtatás a projektkönyvtárból, a kódot `center_crop_check.py` néven mentve: `python center_crop_check.py`. A mért kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU):

```
input (2, 8, 23, 23)
output (2, 8, 22, 22)
```

A függvény a `top, left` levágással a felesleget a bal/felső és jobb/alsó oldal között osztja el; $23 \to 22$-nél `(23-22)//2=0`, tehát a jobb/alsó egy pixelt elveszít. A függvény a feladat specifikációjának megfelel; ha szigorúan szimmetrikus kerekítés kell, azt a feladatban külön definiálni kell.

F4 megoldás. A hiba oka: a `cat([up1(b), e2], dim=1)` kimenete 32 csatornás, mert az upsampling 16, a skip 16 csatornát ad össze. A `dec1a` azonban 16 bemeneti csatornát deklarál. Következmény: a PyTorch a `Conv2d` bemeneti csatorna-ellenőrzésénél `RuntimeError`-t ad, mert a várt 16 helyett 32 csatorna érkezik. Javítás: `self.dec1a = nn.Conv2d(32, 16, 3, padding=1)`, vagy `4 * base` jelöléssel `self.dec1a = nn.Conv2d(4 * base, 2 * base, 3, padding=1)`. Visszaellenőrzés: a javított modul paraméterszáma 29 321, és a 4. szakasz `assert`-jei a kimenetre $(2, 1, 24, 24)$-et adnak. A hibás `dec1a` $16 \cdot 16 \cdot 9 + 16 = 2320$, a javított $32 \cdot 16 \cdot 9 + 16 = 4624$ paraméteres, tehát a javítás $+2304$ paraméter; a 29 321 csak a javított modulra igaz.

F5 megoldásminta. Kérdés: a skip-kapcsolatok javítják-e a GlyphLab előtér-szegmentálást? Megfigyelési egység: `scene_id`, mert egy jelenet két nézete egymástól függ, ezért nem tekinthetők független megfigyelésnek; a train/val/test split a GlyphLab szerződés szerint jelenetszintű. Bemenet: `data/images.npy` float32 $(360, 1, 24, 24)$; cél: `data/masks.npy` uint8 $(360, 1, 24, 24)$. Baseline: skip nélküli `TinyUNet`-változat, amelyben a két `cat` elmarad, ezért a `dec1a` bemenete 16 (csak az `up1` kimenete), a `dec2a` bemenete pedig 8 (csak az `up2` kimenete) csatorna; a baseline saját moduldefiníciót kap. Változó tényező: skip megléte. Mivel a skip elhagyása a paraméterszámot is csökkenti, a két modell kapacitása nem azonos; ezért vagy a baseline decoder-szélességét kell úgy növelni, hogy a paraméterszám a skip-pel rendelkező modelléhez közel kerüljön, vagy a különbséget a paraméterszám-különbséggel együtt kell értelmezni, és ezt a korlátot a következtetésben ki kell mondani. Loss: `binary_cross_entropy_with_logits`, a maszk `float32`-vé konvertálva. Metrika: threshold 0,5 melletti pixel-IoU képenként, majd a jelenetszintű aggregálás a 27.3. Dice, IoU és aggregálás alfejezetben tárgyalandó módon; a képenkénti IoU-kat itt még nem szabad független mintaként kezelni. Választási szabály: a legjobb kísérletenkénti checkpointot a validációs loss választja, a test-halmazt csak a végleges modellpár kiválasztása után értékeljük. Ismétlés: öt seed, mindkét változatra azonos. Páros összehasonlítás és döntési szabály: a két változatot ugyanazon seedekkel és ugyanazon testjeleneteken értékeljük; a döntés a jelenet szintű páros különbség bootstrap CI-jén alapul, és csak 0-t kizáró 95%-os CI esetén állítunk javulást. Erőforrás: CPU, float32, rögzített seed. Következtetési határ: a GlyphLab szintetikus, három orientációhoz tartozó alakzatmaszk; az eredmény nem általános természetes képi szegmentálási validáció, és a skip hatása csak ezen az adaton, ezzel a protokollal állítható.

27.2.8 8. Tipikus tévedés, önellenőrzés és visszahívás

A gyakori tévedés az, hogy a fenti `TinyUNet`-et az eredeti U-Net reprodukciójának nevezzük. A jelen modul U-Net-szerű tanulópélda: van encoder, decoder és skip concat, de az eredeti, [P14]

által hivatkozott U-Net orvosbiológiai képszegmentálásra készült, eltérő bemeneti felbontással, mélységgel, csatornaszámmal és tanítási eljárással. Az a kijelentés, hogy „implementáltuk a U-Net cikket”, csak akkor lenne igazolható, ha a modell, az adat, a loss, az augmentáció és a kiértékelési metrika a hivatkozott közleményben leírt protokollnak felelne meg; ezeket itt nem reprodukáljuk. Az ellenpélda nem elvont: a konkrét adatszerződés önmagában kizárja a reprodukciós állítást, hiszen a GlyphLab 360 darab 24×24-es szintetikus kép, míg a [P14] absztraktja orvosbiológiai szegmentálási felhasználásról szól. A két állítás közötti különbség nem pusztán méretbeli, hanem a reprodukciós állítás típusában van.

Önellenőrzés: minden 3×3-as, $p = 1$ konvolúció megtartja a térbeli méretet? A kódban minden decoder-`Conv2d` is `padding=1`? A két `cat` valóban `dim=1`-en történik? A `dec1a` bemeneti csatornája 32, a `dec2a` bemenete 16? Ha a válasz bármelyikre nem, a paraméterszám-ellenőrzés vagy a shape-assert el fog csúszni.

Válaszkulcs. Igen: a 3×3-as, $p = 1$ konvolúció megtartja a térbeli méretet; a decoder minden `Conv2d`-jében `padding=1` szerepel; a két `cat dim=1` mentén történik; a `dec1a` bemenete 32, a `dec2a` bemenete 16 csatorna. Ezeket a labor shape-assertjei ellenőrzik.

Visszahívás a 15.2. Inputgradiens és átfedő recepciós mezők alfejezetből: a transzponált konvolúció upsamplingjének magja ugyanaz a scatter-add gondolat, mint az inputgradiens számításakor. A $k = 2$, $s = 2$ esetben egy (i, j) bemeneti pixel a kimeneti $(2i + u, 2j + v)$, $u, v \in \{0, 1\}$ pozíciókhoz járul hozzá; ezért a kimenet $2H \times 2W$ lesz. A skip concat ezt a magasabb felbontású, kódoló oldali aktivációval egészíti ki, de nem változtat a scatter-add indexfeltételén.

27.2.9 Rövid összefoglaló

A U-Net encoder–decoder szerkezet: a kódoló konvolúciókkal és max-poollal növeli a csatornaszámot, miközben csökkenti a térbeli felbontást; a dekódoló tanulható transzponált konvolúciókkal növeli a felbontást. A skip concat a kódoló azonos felbontású aktivációját csatorna szerint fűzi a dekódolóhoz, így a korai térbeli részletek nem csak a bottlenecken át jutnak a kimenetre. A lemmában levezettük a 24×24-es bemenet minden rétegalakját: $24 \to 12 \to 6 \to 12 \to 24$, a concat-csatornaszámok 32 és 16. A 22×22-es ellenpélda megmutatta, hogy a pool és upsampling méretezése miatt crop válhat szükségessé. A 29 321 paraméteres modul a szerződésnek megfelelő rétegspecifikáció.

27.2.10 Következő alkalmazás

A 27.3. Dice, IoU és aggregálás egységben a szegmentálás értékelését építjük fel: a Dice-együttható és az IoU definícióját, a per-image, per-scene és micro aggregálás különbségét, valamint az üres maszkok és a küszöbválasztás hatását. A mostani modell logitkimenete (a maszk logitja) az ottani metrikák közvetlen bemenete; a küszöböléssel kapott bináris maszk adja az előrejelzést.

27.2.11 Felhasznált források

- [P14] Ronneberger, Fischer és Brox (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* arXiv:1505.04597. https://arxiv.org/abs/1505.04597.
- [P15] He et al. (2017). *Mask R-CNN.* arXiv:1703.06870. https://arxiv.org/abs/1703.06870.

27.3 Dice, IoU és aggregálás

27.3.1 Cél, előismeret és induló kérdés

A 27.2. U-Net, encoder–decoder és skip egységben felépítettük az encoder–decoder és a skip-kapcsolatos U-Net-alapú szegmentáló architektúrát. Ott a térbeli alakok és a csatornaszámok összeillesztése volt a fő kérdés. Most azt vizsgáljuk, hogyan mérjük a kimenő, pixelszintű bináris maszk minőségét. Egyetlen skalár pontosság nem elég: a könyv GlyphLab-adatán (360 maszk, `data/masks.npy`) az előtér a pixelek 1,2%–3,1%-át teszi ki, mediánja 1,56%, ezért az „üres maszkot jósolok” stratégia pixel-accuracya 96,9% és 98,8% között van (medián 98,4%), miközben a tényleges előtér-alakzatot teljesen elhibázza. A szegmentáló modellek — köztük a [P14] U-Net — kimenete is ilyen bináris (vagy osztályonként binarizált) maszk, ezért az itteni metrikák az ilyen modellek kiértékelésének alapját adják.

A szükséges előismeret: halmazművelet, a 27.2. U-Net, encoder–decoder és skip alfejezetből a bináris maszk mint 0/1 előtér-térkép fogalma, valamint a 23.3. Páros összehasonlítás és csoportszintű intervallum alfejezetből a csoportos kiértékelés és a jelenet-mintavételi egység. Ez az egység önálló matematikai fogalmat épít fel, ezért először a metrika bináris halmaz-definícióját rögzítjük, utána térünk a numerikus és aggregálási kérdésekre.

Induló becslés: ha egy 6 pixeles célmaszkra a predikció 6 találatot és 30 téves előtér-pixelt tartalmaz, akkor a pixel-accuracy $\frac{24^2-30}{24^2} \approx 0{,}948$, tehát kiválónak tűnik. Vajon a Dice vagy az IoU is ezt mondja? A második szakasz definíciói után ezt a példát ellenőrizzük.

27.3.2 Definíció: Dice és IoU bináris maszkon

Legyen Ω a képpontok véges halmaza, $|\Omega| = H \cdot W$ egyetlen bináris maszkra. A target maszk által kijelölt előtér-halmaz legyen $A \subseteq \Omega$, a predikált maszké $B \subseteq \Omega$. A két halmaz közös része $A \cap B$, uniója $A \cup B$. Jelölje

$$a = |A|, \qquad b = |B|, \qquad i = |A \cap B|.$$

A **Jaccard-index** (Jaccard, 1901, 1912), a szegmentálásban elterjedt angol nevén **Intersection over Union (IoU)**:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{i}{a + b - i},$$

feltéve, hogy az unió nem üres. A **Sørensen–Dice-együttható** (röviden **Dice**; Dice, 1945; Sørensen, 1948):

$$D(A, B) = \frac{2|A \cap B|}{|A| + |B|} = \frac{2i}{a + b},$$

feltéve, hogy $a + b > 0$. Mindkét metrika $[0, 1]$-be esik; nemüres $A = B$ esetén 1-et, diszjunkt halmazokra 0-t ad. Az $A = B = \emptyset$ esetben a képlet 0/0-t adna, ezért arra külön konvenció vonatkozik (a következő szakasz rögzíti). A nevezőben az IoU az uniót használja, a Dice pedig a két halmaz méretének összegét; ez okozza, hogy a Dice mindig legalább akkora, mint az IoU.

Az induló becslés ellenőrzése: a 6 pixeles célmaszkra adott 6 találat és 30 téves előtér-pixel (FN $= 0$) összesen 36 predikált pixelt jelent; a metszet 6, az unió 36, ezért $J = 6/36 \approx 0{,}167$ és

$D = 2 \cdot 6/(6 + 36) = 12/42 \approx 0{,}286$. A pixel-accuracy ugyanerre a maszkra 0,948: ugyanaz a predikció a pixel-accuracy szerint kiváló, az IoU és a Dice szerint viszont gyenge.

A definíció számosságokat használ, ezért a maszkok tárolási formátuma lényegtelen: ugyanazt az eredményt adják a `uint8` 0/1 tömbök, a `bool` tömbök és a szigmoid-kimenet 0,5-es küszöbölése utáni `bool` tömbök is. Nyers logitok esetén a döntési küszöb nem 0,5, hanem 0, mert $\sigma(0) = 0{,}5$; a metrika bemenete mindenképp a döntés utáni bináris maszk.

27.3.3 Tensoralak, jelölés és az üres eset konvenciója

A GlyphLab-ban a maszk alakja $(N, 1, H, W)$, `uint8`, ahol N a képek száma, a csatorna mérete 1, a többi a térbeli kiterjedés. A k-adik képre A_k a célmaszk, B_k a predikciós maszk. A számítás során a $(N, 1, H, W)$-t a $(N, 1)$-re redukáljuk a (H, W) tengelyek összegzésével; többosztályos esetben minden c osztályra külön binarizált $A_{k,c}, B_{k,c}$ párt képzünk (a c itt osztályindex, nem komplementer).

Az **üres unió** pontosan akkor fordul elő, ha $A = B = \emptyset$. A nevező ekkor nulla, ezért a fenti osztás nem ad értelmes számot. Ebben a könyvben a **páronkénti konvenció** a következő: ha mindkét maszk üres, akkor $J(A, B) = D(A, B) = 1$, mert a predikció és a cél tökéletesen megegyezik. Ha az egyik üres, a másik nem, akkor az unió nem üres, így $i = 0$ és $a + b > 0$, tehát $J = D = 0$. A globális aggregálásnál viszont az üres–üres pár egyetlen tagot sem ad a közös számlálóba; ha a teljes adathalmazon az összes unió üres, a globális metrika **nem definiált**, és ezt hibával jelezzük, nem pedig 1-gyel.

Ez a konvenció értelmezés kérdése, nem tétel. Egy másik, egyszerűen megfogalmazható konvenció az üres–üres párok kihagyása az átlagból; a kettő nem azonos, mert a mi konvenciónk 1-es járulékot ad a képenkénti átlaghoz. A laborban ez láthatóvá válik.

27.3.4 Tétel és levezetés: Dice kapcsolata az IoU-val

A két metrika nem független: ugyanazt a i, a, b statisztikát rendezi más skálán.

Tétel (Dice–IoU azonosság). Legyenek $A, B \subseteq \Omega$ véges pixelhalmazok, $a = |A|$, $b = |B|$, $i = |A \cap B|$, és tegyük fel, hogy $a + b > 0$. Ekkor az unió is nemüres, az IoU $J = i/(a + b - i)$ definiált, és

$$D(A, B) = \frac{2J(A, B)}{1 + J(A, B)}.$$

Bizonyítás. Az $a + b > 0$ feltételből $a + b - i = |A \cup B| \geq 1$ is következik, mert ha a két halmaz közül legalább az egyik nem üres, akkor az unió is nem üres. Ezért J nevezője nem nulla. Helyettesítsünk be a jobb oldalba:

$$\frac{2J}{1 + J} = \frac{2\,\frac{i}{a+b-i}}{1 + \frac{i}{a+b-i}} = \frac{\frac{2i}{a+b-i}}{\frac{a+b-i+i}{a+b-i}} = \frac{2i}{a + b - i} \cdot \frac{a + b - i}{a + b} = \frac{2i}{a + b} = D.$$

Az utolsó egyszerűsítésben $a + b - i > 0$, tehát a tört elhagyása megengedett. □

Az állítás típusa tétel, a bizonyítás elemi algebra. A feltétel fontos: az üres–üres párt a tétel nem fedi le, mert ott $a + b = 0$. Az imént bevezetett 1-es konvenció nem következik a tételből, hanem külön definíció. A következő egyenlőtlenség szintén levezethető, és a gyakorlatban magyarázza, miért „szigorúbb” az IoU:

$$D - J = \frac{2J}{1+J} - J = \frac{J(1-J)}{1+J} \geq 0,$$

mert $0 \leq J \leq 1$. Egyenlőség pontosan $J = 0$ (diszjunkt maszkok) vagy $J = 1$ (tökéletes egyezés) esetén áll; közbülső átfedésnél a Dice szigorúan nagyobb.

27.3.5 Aggregálás: per-image, global pixel, class imbalance

A metrikák egyetlen maszkpárra definiáltak. Egy tesztkészletre több aggregálási út létezik, és ezek különböző kérdésekre válaszolnak.

Képenkénti (per-image) átlag. Minden k képre kiszámítjuk J_k és D_k értékét, majd

$$\overline{J}_{\text{img}} = \frac{1}{N}\sum_{k=1}^{N} J_k, \qquad \overline{D}_{\text{img}} = \frac{1}{N}\sum_{k=1}^{N} D_k.$$

Ez minden képet azonos súllyal vesz, függetlenül az előtér méretétől. A 23.3. Páros összehasonlítás és csoportszintű intervallum alfejezetben bevezetett jelenet-bootstrap ehhez illeszkedik: először képenként számolunk, majd a két view eredményét jelenetenként átlagoljuk, és a bootstrap a független scene-egységeken húz. Így az intervallum a jelenetek közötti szórást becsli, nem a pixelek közötti függést.

Globális pixel-aggregálás. Az összes képet egyetlen, nagy pixelgyűjteménynek tekintjük. Legyen

$$\text{TP} = \sum_k |A_k \cap B_k|, \quad \text{FP} = \sum_k |B_k \setminus A_k|, \quad \text{FN} = \sum_k |A_k \setminus B_k|.$$

Ekkor

$$J_{\text{glob}} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}, \qquad D_{\text{glob}} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}.$$

A globális verzió minden pixelt azonos súllyal vesz, ezért a nagy előtérrel rendelkező képek dominálják. Az üres–üres párok a globális számlálóba semmit sem tesznek, míg a képenkénti átlagban 1-ként jelennek meg; ez önmagában is eltérést okoz.

Osztályegyensúly (class imbalance). Ha Ω nagy és az A előtér kicsi, akkor a pixel-accuracy a domináns háttér miatt megtévesztő. A Dice és az IoU ezt csökkenti, de nem szünteti meg: a nevező az A és B együttes számosságához, illetve uniójához viszonyít. Hátrányuk viszont, hogy ugyanaz az abszolút hiba egy kis objektumon sokkal nagyobb büntetést kap, mint egy nagyon: ha A 4 pixeles és B 8 pixeles teljes lefedéssel, akkor $D = 8/12 \approx 0{,}667$; ha A 400 pixeles és B 404 pixeles teljes lefedéssel, akkor $D = 800/804 \approx 0{,}995$. Ez a tulajdonság a relatív átfedést méri, nem az abszolút tévedés költségét.

Határ (boundary). A Dice és az IoU kizárólag halmaz-kardinalitásokat használ, ezért nem bünteti közvetlenül a kontúr alakját. Két azonos IoU-jú hiba teljesen más irányú is lehet: egy 2×2-es célblokkra a tőle jobbra 1 pixellel eltolt 2×2-es predikció és a tőle lefelé 1 pixellel eltolt 2×2-es predikció egyaránt $i = 2$, $a + b = 8$, tehát $J = 2/6 = 1/3$ és $D = 4/8 = 0{,}5$. Az egyik hibát a közös függőleges határon követjük el, a másikat a vízszintesen. Aki a határvonal pontos lefutását akarja mérni, kiegészítő metrikát használhat (például boundary-IoU vagy felületi távolság), de ez nem része a jelen egység főútjának; a területalapú metrika érvényességi korlátját mindig ki kell mondani.

27.3.6 Kézi példa három maszkpár

Legyen a tér 4×4, tehát $|\Omega| = 16$. Három maszkpárt vizsgálunk.

1. pár: A_1 a bal felső 2×2-es blokk, tehát

$$A_1 = \{(0,0),(0,1),(1,0),(1,1)\}, \quad a_1 = 4.$$

B_1 a bal felső 3×3-as blokk, tehát $b_1 = 9$. A metszet $i_1 = 4$, az unió $a_1 + b_1 - i_1 = 4 + 9 - 4 = 9$. Így

$$J_1 = \frac{4}{9} \approx 0{,}4444, \qquad D_1 = \frac{8}{13} \approx 0{,}6154.$$

2. pár: $A_2 = \emptyset$, $B_2 = \{(0,0),(3,3)\}$, tehát $a_2 = 0$, $b_2 = 2$, $i_2 = 0$. Ekkor $J_2 = D_2 = 0$.
3. pár: $A_3 = B_3 = \emptyset$. Az unió üres, a páronkénti konvenció szerint $J_3 = D_3 = 1$.

A három pár képenkénti átlaga:

$$\overline{J}_{\text{img}} = \frac{4/9 + 0 + 1}{3} = \frac{13/9}{3} = \frac{13}{27} \approx 0{,}4815,$$

$$\overline{D}_{\text{img}} = \frac{8/13 + 0 + 1}{3} = \frac{21/13}{3} = \frac{7}{13} \approx 0{,}5385.$$

A globális pixel-számításhoz:

$$\text{TP} = 4 + 0 + 0 = 4, \qquad \text{FP} = 5 + 2 + 0 = 7, \qquad \text{FN} = 0 + 0 + 0 = 0,$$

$$J_{\text{glob}} = \frac{4}{4 + 7 + 0} = \frac{4}{11} \approx 0{,}3636, \qquad D_{\text{glob}} = \frac{8}{8 + 7} = \frac{8}{15} \approx 0{,}5333.$$

A globális IoU lényegesen kisebb, mint a képenkénti átlag, mert a globális változat nem kap 1-es járulékot az üres–üres pártól, és a 3×3-as predikció öt téves (FP) pixelét teljes súllyal számolja. A számok mutatják, hogy a két aggregálás **nem felcserélhető**.

27.3.7 Kód: metrikaszámítás és aggregálás

A kód a saját NumPy-referenciánk stílusát követi. Bináris maszkpárokra számol, a bemenet `(N,1,H,W)` alakú `uint8` vagy `bool`. A csatornatengely 1 méretű, ezért a térbeli összegzés az utolsó két tengelyen történik.

```python
# dice_iou_lab.py
import numpy as np

def binary_mask_metrics(pred, target, empty_empty_value=1.0):
    """Per-image IoU es Dice binaris (N,1,H,W) maszkparokra.

    Konvencio: ha mindket maszk ures, mindket metrika empty_empty_value.
    Ha az egyik ures, a masik nem, akkor a nevezo pozitiv, a metszet 0.
    """
    p = np.asarray(pred).astype(bool)
    t = np.asarray(target).astype(bool)
    # shape ellenorzes
    assert p.shape == t.shape and p.ndim == 4 and p.shape[1] == 1

    inter = np.logical_and(p, t).sum(axis=(-2, -1)).astype(np.float64)
    sum_a = t.sum(axis=(-2, -1)).astype(np.float64)
    sum_b = p.sum(axis=(-2, -1)).astype(np.float64)
    union = np.logical_or(p, t).sum(axis=(-2, -1)).astype(np.float64)

    with np.errstate(divide='ignore', invalid='ignore'):
        iou = np.where(union > 0, inter / union, empty_empty_value)
        dice = np.where(sum_a + sum_b > 0,
                        2.0 * inter / (sum_a + sum_b),
                        empty_empty_value)
    return iou, dice  # mindkettő alakja (N,1)


def global_binary_metrics(pred, target):
    """Globális pixel-IoU/Dice az összes képre együtt."""
    p = np.asarray(pred).astype(bool)
    t = np.asarray(target).astype(bool)
    p = p.reshape(p.shape[0], -1)
    t = t.reshape(t.shape[0], -1)
    TP = np.logical_and(p, t).sum().astype(np.float64)
    FP = np.logical_and(p, np.logical_not(t)).sum().astype(np.float64)
    FN = np.logical_and(t, np.logical_not(p)).sum().astype(np.float64)
    # Üres unió az összes képre együtt: a metrika nem definialt.
    union = TP + FP + FN
    if union == 0:
        raise ValueError("A globális IoU/Dice nem definialt: ures unió.")
    iou = TP / union
    dice = 2.0 * TP / (2.0 * TP + FP + FN)
    return iou, dice


if __name__ == "__main__":
    # Három 4x4-es példa, NCHW alakban.
    t0 = np.array([[[[1,1,0,0],
                     [1,1,0,0],
                     [0,0,0,0],
                     [0,0,0,0]]]], dtype=np.uint8)
    # p0: a bal felső 3x3-as blokk, a 4x4-es vászonra jobbra és lefelé
    # egy-egy nulla oszloppal, illetve sorral kiegészítve.
    p0 = np.pad(np.ones((1,1,3,3), dtype=np.uint8),
                ((0,0),(0,0),(0,1),(0,1)), mode='constant')

    t1 = np.zeros((1,1,4,4), dtype=np.uint8)
    p1 = np.zeros((1,1,4,4), dtype=np.uint8)
    p1[0,0,0,0] = 1
    p1[0,0,3,3] = 1

    t2 = np.zeros((1,1,4,4), dtype=np.uint8)
    p2 = np.zeros((1,1,4,4), dtype=np.uint8)
```

```
targets = np.concatenate([t0, t1, t2], axis=0)  # (3,1,4,4)
preds = np.concatenate([p0, p1, p2], axis=0)

iou, dice = binary_mask_metrics(preds, targets)
print("per-image IoU  :", iou.ravel())
print("per-image IoU mean:", iou.mean())
print("per-image Dice :", dice.ravel())
print("per-image Dice mean:", dice.mean())

gi, gd = global_binary_metrics(preds, targets)
print("global IoU :", gi)
print("global Dice:", gd)

assert np.allclose(iou.ravel(), [4/9, 0.0, 1.0], atol=1e-12, rtol=0)
assert np.allclose(dice.ravel(), [8/13, 0.0, 1.0], atol=1e-12, rtol=0)
assert abs(gi - 4/11) < 1e-12
assert abs(gd - 8/15) < 1e-12
```

A megadott környezetben mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a példa determinisztikus, seed nélkül):

```
per-image IoU  : [0.44444444 0.         1.        ]
per-image IoU mean: 0.48148148148148145
per-image Dice : [0.61538462 0.         1.        ]
per-image Dice mean: 0.5384615384615384
global IoU : 0.36363636363636365
global Dice: 0.5333333333333333
```

A kód assertekkel is ellenőrzi a kézi értékeket: 4/9, 0, 1 (IoU), 8/13, 0, 1 (Dice), valamint a globális 4/11 és 8/15 értéket.

27.3.8 Labor: három maszkpár metrikája és az aggregálási eltérés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja, hogy a kézi példát a fenti kóddal saját környezetben lefuttassa, és számszerűen lássa a képenkénti és a globális aggregálás eltérését.

- Bemenet: a fent definiált három maszkpár, `uint8`, `(3,1,4,4)`. Nincs külső fájltól függés; a GlyphLab `data/masks.npy`-jéből a hallgató a későbbi projektekben a `np.load` után `uint8` tömböt kap, és ugyanez a függvény használható.
- Import: `numpy` csak.
- Seed: a példa determinisztikus, véletlen húzás nincs, ezért seed beállítása nem szükséges; a későbbi bootstrap-lépések külön, saját seedet kapnak.
- Dtype: bemenet `uint8`, belső konverzió `bool`; a metrikák `float64`-ben készülnek, mert az osztás eredménye nem egész, és így egységes a kimeneti dtype. A kis egész számlálók `float32`-ben is pontosan ábrázolhatók lennének, a `float64` nem a „nagy számok befogadása” miatt kell.
- Device: CPU, NumPy; nincs GPU-követelmény.
- Futtatási parancs a `lab/` munkakönyvtárból: `python dice_iou_lab.py`.
- Ellenőrzés: a kívánt tűrés a kézi értékekkel szemben abszolút 10^{-12}; a kód ugyanazokat a lebegőpontos műveleteket végzi, mint az elvárt értékek előállítása, ezért a különbség jóval a tűrés alatt marad. A három per-image értéknek rendre 4/9, 0, 1 IoU-t és 8/13, 0, 1 Dice-ot kell adnia.

- Értelmezés: a per-image IoU átlag 0,4815, a globális 0,3636. A különbség nem kerekítési hiba. A globális IoU az üres–üres pár 1-es konvencióját nem ismeri, és a 3×3-as predikció mind az öt téves pixelét a közös nevezőbe teszi, miközben a képenkénti átlagban az adott kép IoU-ját az unió saját mérete normalizálja. Ha a kérdés az, hogy egy átlagos jelenet milyen jól van szegmentálva, a per-image, jelenetszintű átlag a megfelelő; ha a kérdés az, hogy az összes előtér-pixel hány százalékát találtuk el, a globális a megfelelő.

27.3.9 F1–F5 feladatok

27.3.9.1 F1 (fogalmi, nehézség 1)

Adott A és B ugyanazon a 10×10-es rácson: $|A| = 10$, $|B| = 8$, $|A \cap B| = 6$. Számítsa ki J és D értékét, és ellenőrizze egy konkrét számpéldán a levezetett $D = 2J/(1 + J)$ azonosságot. Adjon olyan két halmazpárt, ahol $D = J$, és indokolja, miért.

Elvárt átadandó eredmény: a két számérték, a behelyettesítés, a két nemüres egyenlőségi eset megnevezése, és az üres–üres konvenció megkülönböztetése.

27.3.9.2 F2 (levezetési, nehézség 2)

Bizonyítsa be a $D - J$ különbségre vonatkozó állítást: ha $a + b > 0$, akkor $D(A, B) \geq J(A, B)$, és az egyenlőség pontosan $i = 0$ vagy $a = b = i$ esetén áll fenn.

Elvárt átadandó eredmény: teljes levezetés a J definíciójából, az egyenlőség feltételének indoklása, és egy olyan számpélda, ahol $0 < J < 1$.

27.3.9.3 F3 (implementációs, nehézség 2)

Írjon `iou_multi(pred, target)` függvényt, amely `(N,C,H,W)` alakú, már binarizált 0/1 `uint8` többosztályos maszkokat fogad (a bemenet csatornái osztályonként 0/1 értékek; index- vagy one-hot címkék esetén előbb binarizálni vagy one-hot kódolni kell), és osztályonként visszaadja a képenkénti IoU-átlagot, valamint a mean IoU-t (a C osztály átlagát). Az üres–üres párokra minden osztályban 1-gyel számoljon. Ellenőrizze a fenti három bináris példán $C = 1$ esetben, hogy a függvény a `binary_mask_metrics` kimenetével egyezik.

Elvárt átadandó eredmény: futtatható kód, a három példán adódó $[4/9, 0, 1]$ vektor és a $\overline{J}_{\text{img}} = 13/27$, továbbá a shape-ellenőrzés.

27.3.9.4 F4 (hibakeresési, nehézség 2)

Adott a következő kódrészlet:

```
def iou_bad(pred, target):
    inter = (pred * target).sum(axis=(-2, -1))
    union = pred.sum(axis=(-2, -1)) + target.sum(axis=(-2, -1)) - inter
    return (inter / union).mean()
```

Tegyük fel, hogy a függvényt a hálózat nyers sigmoid-kimenetére hívjuk, amely `float32` értékeket tartalmaz $[0, 1]$-ben. Mi a hiba, mi a következménye, és hogyan javítaná? Ellenőrizze a javítást azzal a kétpixeles példával, ahol `pred=[0.6,0.4]`, `target=[1,0]`.

Elvárt átadandó eredmény: a hiba oka, a numerikus következmény, a küszöbölést tartalmazó javított függvény, és a javított kimenet.

27.3.9.5 F5 (kísérleti/transzfer, nehézség 3)

A GlyphLab szegmentáló laborban a cél a 24×24-es előtér-maszk rekonstrukciója. Tervezzen kiértékelési protokollt, amely a 23.3. Páros összehasonlítás és csoportszintű intervallum alfejezetbeli class-stratified scene bootstrapet használja a képenkénti IoU átlagához (a három osztály itt a GlyphLab orientációs címkéje; a szegmentáló kimenet bináris, $C = 1$). Adja meg:

- a megfigyelési egységet,
- a per-image metrikából a scene-szintű statisztika képzésének lépését,
- a bootstrap replikátumok képzését a két view figyelembevételével,
- egy triviális baseline-t,
- és a következtetés érvényességi korlátját.

Elvárt átadandó eredmény: protokoll-leírás, a baseline indoklása, és legalább egy konkrét korlát (például a bootstrap csak a tesztminta-mintavételi bizonytalanságot közelíti).

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

27.3.10 Megoldások és indoklás

27.3.10.1 F1 megoldása

Az unió mérete $a + b - i = 10 + 8 - 6 = 12$, tehát $J = 6/12 = 0{,}5$. A Dice $D = 2 \cdot 6/(10 + 8) = 12/18 = 2/3$. Az azonosság: $2J/(1 + J) = 1/(1{,}5) = 2/3$, egyezik.

A $D = J$ esetek a levezetett $D - J = J(1 - J)/(1 + J)$ képletből: $J = 0$, azaz $i = 0$ (diszjunkt halmazok, például A a bal felső, B a jobb alsó sarok), vagy $J = 1$, azaz $a = b = i$ (a két halmaz azonos). Ha A üres és B nem üres, az $i = 0$ esethez tartozik, és $J = D = 0$. A könyv páronkénti konvenciója szerint az üres–üres párra $J = D = 1$, ezért az is egyenlőségi eset, de ez nem a tételből, hanem a külön konvencióból következik; a tétel feltétele $a + b > 0$, ezért az üres–üres párt nem fedi le.

27.3.10.2 F2 megoldása

A tétel szerint $D = 2J/(1 + J)$, feltéve, hogy $a + b > 0$. Ekkor

$$D - J = \frac{2J}{1 + J} - J = \frac{2J - J(1 + J)}{1 + J} = \frac{J(1 - J)}{1 + J}.$$

Mivel $J = |A \cap B|/|A \cup B| \in [0, 1]$, a számláló nemnegatív, a nevező $1 + J \geq 1 > 0$. Így $D \geq J$. Az egyenlőség pontosan akkor, ha $J(1 - J) = 0$, azaz $J = 0$ vagy $J = 1$. A $J = 0$ ekvivalens $i = 0$-val, mert a nevező pozitív. A $J = 1$ ekvivalens $i = a + b - i$-vel, azaz $2i = a + b$. Mivel

$i \le a$ és $i \le b$, ez csak $i = a$ és $i = b$ mellett lehet, tehát $a = b = i$, vagyis $A = B$. Számpélda közbülső értékre: $a = 6$, $b = 6$, $i = 3$, ekkor $J = 3/9 = 1/3$, $D = 6/12 = 1/2$, és $D > J$.

27.3.10.3 F3 megoldása

A többosztályos függvény a csatornákat osztályonként külön értékeli:

```
def iou_multi(pred, target):
    # pred, target: (N,C,H,W), uint8, mar binarizalt 0/1 csatornak.
    # Index-cimkek eseten elobb one-hot kodolas (vagy osztalyonkenti
    # binarizalas) kell; ez a fuggveny mar binarizalt bemenetet var.
    pred = np.asarray(pred).astype(bool)
    target = np.asarray(target).astype(bool)
    assert pred.shape == target.shape and pred.ndim == 4
    N, C, H, W = pred.shape
    ious = np.empty((C, N), dtype=np.float64)
    for c in range(C):
        p = pred[:, c, None, :, :]  # (N,1,H,W)
        t = target[:, c, None, :, :]
        iou, _ = binary_mask_metrics(p, t, empty_empty_value=1.0)
        ious[c] = iou[:, 0]
    per_class_mean = ious.mean(axis=1)   # (C,)
    return per_class_mean, ious.mean()   # (C,) es skalár


# egyezes-teszt C=1 eseten a harom binaris pelda kimenetevel
per_class_mean, miou = iou_multi(preds, targets)
iou_ref, _ = binary_mask_metrics(preds, targets)
assert np.allclose(per_class_mean, iou_ref.mean())
assert abs(miou - 13/27) < 1e-12
print("C=1 per_class_mean:", per_class_mean, "mIoU:", miou)
```

A $C = 1$ esetben a három példa IoU-vektora $[4/9, 0, 1]$, az osztályon belüli átlag $13/27 \approx 0{,}4815$. A kód az üres–üres konvenciót az `empty_empty_value=1.0` argumentummal őrzi meg. Alakellenőrzés: bemenet `(3,1,4,4)`, kimenet `per_class_mean` alakja `(1,)`, a `miou` skalár.

27.3.10.4 F4 megoldása

A hiba: a `pred` nem bináris, hanem valós $[0, 1]$-beli sigmoid-kimenet. A `pred * target` ezért nem metszet-számosságot ad, hanem súlyozott összeget; a `union` sem halmaz-unió, hanem valós összeg. A következmény: a metrika nem IoU, és érzékeny a küszöb alatti zajra; például a kétpixeles `[0.6,0.4]` és `[1,0]` esetén az inter 0,6, a union 1,4, a hibás képlet kimenete 0,4286, miközben a küszöbölt `[1,0]` predikcióval a valódi IoU 1.

A javítás:

```
def iou_fixed(pred, target, empty_empty_value=1.0):
    p = np.asarray(pred, dtype=np.float64) > 0.5
    t = np.asarray(target, dtype=np.float64) > 0.5
    inter = np.logical_and(p, t).sum(axis=(-2, -1))
    union = np.logical_or(p, t).sum(axis=(-2, -1))
    with np.errstate(divide='ignore', invalid='ignore'):
        iou = np.where(union > 0, inter / union, empty_empty_value)
    return iou.mean()
```

A kétpixeles példát `(1,1,1,2)` alakban átadva a kimenet 1; a hibás függvény ugyanerre 0,4286-ot ad. Az `empty_empty_value=1.0` a 27.3. szakasz páronkénti konvencióját viszi át: üres–üres párra nem NaN-t, hanem 1-et ad. Ha a predikció valószínűség, a kiértékelés előtt rögzíteni kell a küszöböt; a 23.2. Kalibráció és döntési küszöb alfejezetben a döntési küszöb kalibrációs szerepét tárgyaljuk, itt a metrika bináris definíciója a döntés **utáni** maszkot várja.

27.3.10.5 F5 megoldása

A protokoll:

1. A tesztkészlet 30 független jelenetet tartalmaz, jelenetenként két view-val, tehát 60 képet.
2. Bináris szegmentáló modell minden képre ad egy B_k maszkot; a célmaszk A_k. Képenként számítjuk a képenkénti IoU-t az üres konvencióval.
3. Jelenetszintű statisztika: minden s jelenetre a két view IoU-jának átlaga, $m_s = (J_{s,1} + J_{s,2})/2$. Ezzel a megfigyelési egység a jelenet lesz, nem az egyes kép, mert a két view ugyanabból a jelenetből származik, ezért egymástól függő.
4. A 30 jelenet-értékből vett átlag a pontbecslés: $\hat{\mu} = \frac{1}{30}\sum_s m_s$.
5. A class-stratified scene bootstrap a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint: a szegmentáló kimenet bináris ($C = 1$), a három osztály tehát nem a kimenet csatornája, hanem a GlyphLab jeleneteinek orientációs címkéje (0 = függőleges, 1 = vízszintes, 2 = átlós); mind a három osztályban 10 jelenet van, és a replikátumban osztályonként 10 jelenetet húzunk visszatevéssel. A replikátumátlagot a m_s értékekből számítjuk, a húzott jelenetek súlyozatlan átlagaként. Az intervallum a replikátumok 2,5%–97,5% kvantilise.
6. Baseline: mindig üres maszk. Ekkor minden nem üres célra $J = 0$; a GlyphLab 360 maszkja között nincs üres maszk, ezért a baseline pontosan 0, nem „körülbelül 0". A baseline tehát azt méri, mennyire triviális a feladat üres predikcióval.
7. Kis minta: a 30 tesztjelenet és az osztályonkénti 10 jelenet kevés, ezért a bootstrap intervallum széles lesz; a pontbecslést mindig a CI-vel együtt kell közölni, és önmagában a pontbecslésből nem szabad erős következtetést levonni.
8. Érvényességi korlát: a bootstrap a tesztminta jelenetek közötti mintavételi ingadozását közelíti, nem a modelltanítás seedjeinek szórását, nem az adatgenerátor bizonytalanságát, és nem az éles alkalmazásban fellépő domaineltolódást. Az üres–üres konvenció megváltoztatása megváltoztatja a pontbecslést is, ezért a protokoll közzétételekor a konvenciót a metrika képletével együtt kell rögzíteni.

27.3.11 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: a globális pixel-IoU-t és a képenként átlagolt IoU-t ugyanannak a statisztikának tekinti. Ez csak speciális esetben, például azonos méretű és azonos előtér-számosságú maszkoknál adhat közeli értéket; általában különböző.

Ellenpélda: két tesztkép. Az elsőn $|A_1| = 1000$, $|B_1| = 1000$, $i_1 = 1000$, tehát $J_1 = 1$. A másodikon $|A_2| = 10$, $B_2 = \emptyset$, tehát $J_2 = 0$. A képenkénti átlag $(1 + 0)/2 = 0{,}5$. A globális: TP = 1000, FP = 0, FN = 10, tehát

$$J_{\text{glob}} = \frac{1000}{1010} \approx 0{,}9901.$$

Ugyanaz a modell egyszer 0,5-ös teljesítményűnek, másszor 0,99-esnek látszik, attól függően, hogy képenként vagy pixelenként átlagolunk. A két statisztika más kutatási kérdésre válaszol:

az első arra, hogy egy átlagos kép/jelenet milyen jól van szegmentálva, a második arra, hogy a pixelek mekkora hányada helyes.

27.3.12 Önellenőrzés és korábbi fogalom visszahívása

Visszahívás a 15.3. Transposed convolution mint adjungált alfejezetből: a transposed convolution adjungált, nem általános inverz. A jelen egység kapcsolódása: az U-Net decoder a transzponált konvolúcióval állítja elő a térbeli maszklogitokat, de a metrika a logitok küszöbölése utáni bináris halmazt méri. Az inverzként való félreértés a metrikaoldalra is átvihető: a magas IoU nem azt jelenti, hogy a dekóder visszaállította az eredeti maszk minden határpixelét, csak azt, hogy a halmazátfedés nagy.

Önellenőrzés: a kézi példában $i_1 = 4$, $a_1 + b_1 = 13$, az unió 9; a Dice $8/13$ és az IoU $4/9$ közti azonosság $2(4/9)/(1 + 4/9) = 8/9 \cdot 9/13 = 8/13$. A harmadik pár üres–üres, a tétel nem alkalmazható, az 1-es konvenció külön definíció. A labor kódja a bemenetet `bool`-lá konvertálja; a `p0` a `np.pad` után 4×4-es alakú, az összegzések az utolsó két tengelyen futnak, a kimenet `(3,1)`.

27.3.13 Rövid összefoglaló

A bináris szegmentálás két alappillére az IoU és a Dice. Az IoU a metszet és az unió, a Dice a kétszeres metszet és a méretösszeg hányadosa. Nemüres halmazokra $D = 2J/(1 + J)$, amit a közös a, b, i statisztikából elemi algebrával levezettünk. Az üres–üres párra a könyv páronkénti 1-es konvenciót rögzít. A képenkénti és a globális pixel-aggregálás külön statisztika: az első képenként, a második pixelenként súlyoz, és az üres konvenciót is másként kezeli. A labor három maszkpáron mutatta az eltérést. A következő alkalmazás: 27.4. Instance mask és összetett feladat alfejezetben az instance mask és az összetett feladat kapcsolja össze a detektálásból ismert IoU-t a pixelszintű előtér-meghatározással.

27.3.14 Források

- [P14] Ronneberger, O., Fischer, P., Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* arXiv:1505.04597. https://arxiv.org/abs/1505.04597.
- Jaccard, P. (1901). *Étude comparative de la distribution florale dans une portion des Alpes et des Jura.* Bulletin de la Société Vaudoise des Sciences Naturelles, 37, 547–579.
- Jaccard, P. (1912). *The distribution of the flora in the alpine zone.* New Phytologist, 11(2), 37–50.
- Dice, L. R. (1945). *Measures of the amount of ecologic association between species.* Ecology, 26(3), 297–302.
- Sørensen, T. (1948). *A method of establishing groups of equal amplitude in plant sociology based on similarity of species content.* Kongelige Danske Videnskabernes Selskab, Biologiske Skrifter, 5, 1–34. A Jaccard-, Dice- és Sørensen-hivatkozások a fejezet forráskatalógusán kívüliek, a szövegben használt, névadó közlemények.
- Saját levezetés: a Dice–IoU azonosság, a $D - J$ egyenlőtlenség és az aggregálási példák önálló, halmazalgebrai levezetések, a 26.3. NMS, confidence és AP alfejezetből visszahívott IoU-fogalom bináris pixelhalmazokra való specializálásával.

27.4 Instance mask és összetett feladat

27.4.1 Cél, előfeltétel és induló kérdés

A 27.3. Dice, IoU és aggregálás egységben a szemantikus szegmentálás pixelcímkét adott: minden (i, j) pixelhez osztályt rendelt, de az azonos osztályú objektumokat nem választotta szét. Itt a kérdés az objektumazonosság külön kezelése: hány különálló példány van, és melyik pixel melyik példányhoz tartozik. Az előismeret a Dice/IoU-metrika, az üres maszk kezelése és az epsilon-stabilizált nevező 27.3. Dice, IoU és aggregálás; a detektálási dobozszerződés, az üres cél és a hibafajták 26.4. Detektálási pipeline és hibaelemzés; továbbá az összesített metrika szeleteket elrejtő hatása 23.4. Hibaszeletek és alkalmazási határ. A 15.4. Független gradiensellenőrzési protokoll alfejezetben bevezetett gradiensellenőrzési szemlélet itt arra figyelmeztet, hogy a maszkküszöb utáni bináris döntés nem sima veszteségfüggvény.

Induló kérdés: egy 10×10-es képen 12 előtérpixel hány példányt alkothat? A válasz nem olvasható le a pixelarányból: lehet egyetlen 12 pixeles objektum, de akár 12 különálló 1 pixeles példány is. A példányszám és az előtérpixel-arány tehát különböző célváltozó.

27.4.2 Definíciók és tenzoralak

Legyen $X \in \mathbb{R}^{H \times W}$ egy egycsatornás kép, illetve batchelt alakban $X \in \mathbb{R}^{N \times C \times H \times W}$. A szemantikus annotáció bináris esetben

$$M_{\text{sem}} \in \{0, 1\}^{H \times W},$$

ahol $M_{\text{sem}}[i, j] = 1$ az előteret jelöli, 0 pedig a hátteret. Többosztályos esetben $M_{\text{sem}} \in \{0, \dots, K-1\}^{H \times W}$, ahol 0 a háttér, $1, \dots, K-1$ a szemantikus osztályok. Ez azonban nem tartalmazza, hogy két azonos osztályú, egymástól különálló folt ugyanahhoz az objektumhoz tartozik-e.

Az instance annotáció

$$A_{\text{inst}} \in \mathbb{N}_0^{H \times W}, \qquad A_{\text{inst}}[i, j] = \begin{cases} 0, & \text{háttér}, \\ m, & \text{az } m\text{-edik példány}, \ m = 1, \dots, M, \end{cases}$$

ahol M a képen lévő példányok száma. Minden $m \in \{1, \dots, M\}$ példányhoz tartozik:

- egy osztálycímke $c_m \in \{1, \dots, K-1\}$,
- egy fél-nyílt xyxy doboz $b_m = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$, ahol $x_{\min} \le x < x_{\max}$, $y_{\min} \le y < y_{\max}$ 26.4. Detektálási pipeline és hibaelemzés,
- egy bináris példánymaszk $M_m \in \{0, 1\}^{H \times W}$, ahol $M_m[i, j] = 1$ pontosan akkor, ha $A_{\text{inst}}[i, j] = m$.

A kettő kapcsolata:

$$(M_{\text{sem}}[i, j] = 1) \iff (A_{\text{inst}}[i, j] > 0).$$

A példányazonosító (instance id) pusztán címke, nem ordinális mennyiség: az 1 és 2 felcserélése ugyanazt a példányfelosztást jelenti. A példányfelosztás ezért permutáció erejéig értendő, a szemantikus osztály viszont nem felcserélhető.

A hálózati kimenet egy listaszerű, változó hosszú szerződés: minden detektált példányra osztálylogit, doboz, confidence és maszklogit tartozik. A jelen könyv által használt, előtanított súlytól független predikciós séma:

$$P = \{(s_i, b_i, c_i, \hat{m}_i) : i = 1, \dots, \hat{M}\},$$

ahol $s_i \in [0,1]$ confidence, $b_i \in \mathbb{R}^4$ xyxy doboz, $c_i \in \{1, \dots, K-1\}$ osztály, $\hat{m}_i \in [0,1]^{H \times W}$ maszkvalószínűség, $\hat{M}$ pedig a predikált példányok száma. A bináris maszkot a downstream döntés 0,5-ös küszöbbel képzi; a veszteség a logiton vagy valószínűségen számol, nem a küszöbölt maszkon.

27.4.3 Levezetési kötelezettség: a szemantikus maszk nem azonosítja az instance-felosztást

Kimondjuk a szükséges állítást.

Állítás. Létezik olyan bináris szemantikus maszk $M_{\text{sem}} \in \{0,1\}^{2 \times 3}$, amelyhez legalább két különböző, összefüggő példányfelosztás tartozik, és a két felosztás még az id-permutáció erejéig sem azonos.

Bizonyítás. Tekintsük a következő 3 széles, 2 magas előtérblokkot:

```
M_sem:
1 1 1
1 1 1
```

Első példányfelosztás:

```
A_inst^(1):
1 1 2
1 1 2
```

Itt az 1. példány az első két oszlop, a 2. példány a harmadik oszlop. Második példányfelosztás:

```
A_inst^(2):
1 2 2
1 2 2
```

Itt az 1. példány az első oszlop, a 2. példány a második és harmadik oszlop. Mindkét esetben

$$\mathbf{1}[A^{(r)}_{\text{inst}} > 0] = M_{\text{sem}}, \qquad r = 1, 2,$$

tehát ugyanazt a szemantikus maszkot adják. Mindkét id-osztály 4-szomszédosan összefüggő, mert mindegyik egy-egy tengelyesen összefüggő téglalap. A két felosztás geometriai vágása eltér: az első a második és harmadik oszlop határán vág, a második az első és második oszlop határán. Ez a különbség nem szüntethető meg az $\{1,2\}$ id-permutációval: a permutáció csak a címkéket nevezi át, a pixelhalmazok particióját nem változtatja meg. Az első felosztásban a $(0,0)$ és a $(0,1)$ pixel ugyanahhoz a példányhoz tartozik, a másodikban különbözőhöz; a példányméretek multihalmaza viszont mindkét esetben $\{4,2\}$, tehát a méretek egyezése önmagában nem teremt ekvivalenciát. $\square$

A lemma következménye: a szemantikus maszkból kiinduló pixelosztályozási veszteség önmagában nem definiál egyértelmű instance-célt. Ha a feladat példányokat kér, a tanítóannotációnak példányazonosítót, példánymaszkot vagy példányonkénti dobozt kell tartalmaznia.

27.4.4 Mask R-CNN: detektálási fej, ROI és párhuzamos maszkág

A [P15] szerzők által bemutatott Mask R-CNN a Faster R-CNN detektort egészíti ki: a meglévő osztályozó- és dobozregressziós ág mellé párhuzamos maszkágat helyez, amely ROI-nként osztályonként egy-egy bináris példánymaszkot jósol, tehát K maszkot, ahol K az osztályok száma; a tanítóveszteség csak a ground truth osztályhoz tartozó maszkon számol. Ez a szervezés jól mutatja, hogy az instance-feladat két részből áll: a detektálási fej megmondja, *hol van egy objektum, és mi az osztálya*, a maszkág pedig megmondja, *mely pixel tartozik hozzá*.

A detektálási fej közbülső reprezentációja a ROI, azaz region of interest: egy $b = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ jelölt doboz, amelyből a hálózat a megfelelő jellemzőtér-régiót kivágja. A kivágás diszkrét rácson történik, ezért fontos a koordináták kvantálásának kezelése. A [P15] dolgozat törzse egy RoIAlign nevű, kvantálásmentes réteget vezet be (a részlet nem az absztraktban, hanem a cikk RoIAlign szakaszában van), amely a térbeli pozíciókat pontosabban őrzi meg; ennek pontos bilineáris mintavételezési képletét és a különböző ROI-méretválasztásokat a teljes cikk megfelelő módszerszakaszának olvasása után lehet idézni.

A gyakorlati megvalósításhoz a PyTorch detektálási modelljeinek finetuningját bemutató [D18] tutorial adhat kiindulópontot; a jelen fejezet azonban a szerződés szintjén marad, és a tutorialra implementációs állítást nem épít.

Ez az alfejezet ezért csak a szerződésszintű leírást adja meg. Legyen a bemeneti képbatchenként egy ROI-halmaz $\{b_1, \dots, b_R\}$. A maszkág minden r ROI-ra

$$\hat{m}_r \in [0,1]^{h_m \times w_m}$$

maszklogitot vagy normalizált maszkvalószínűséget állít elő, ahol h_m, w_m a maszkfej kimeneti felbontása. Az osztályozó ág $Z^{\text{cls}} \in \mathbb{R}^{R \times K}$ logitot, a dobozág $B \in \mathbb{R}^{R \times 4}$ finomított dobozt ad. A kimeneti példány ezután: osztály, confidence, doboz, küszöbölt és az eredeti képméretre interpolált maszk. A tanító veszteség az osztályozási, doboz- és maszktag összege; a maszktag csak a ground truth osztály maszkján számol, a pontos súlyokat itt nem adjuk meg.

27.4.5 Instance-metrika: maszk-IoU-alapú greedy matching

A szemantikus Dice/IoU 27.3. Dice, IoU és aggregálás minden pixelt egyenlően mér, függetlenül attól, hogy hány objektumot vont össze vagy szakított szét. Az instance-metrika ezt a hiányt pótolja. A [P14] U-Net-típusú szemantikus szegmentáló kimenete pontosan olyan M_{sem} maszk, amely példányazonosítót nem tartalmaz, ezért az alábbi példány szintű kiértékelés rá közvetlenül nem alkalmazható. Legyen a ground truth példányhalmaz $\mathcal{G} = \{g_1, \dots, g_M\}$, a predikció $\mathcal{P} = \{p_1, \dots, p_{\hat{M}}\}$, ahol minden p_i-hez confidence s_i tartozik. Minden p_i-re és g_j-re kiszámítjuk a maszk-IoU-t:

$$\text{IoU}(p_i, g_j) = \frac{|p_i \cap g_j|}{|p_i \cup g_j| + \varepsilon},$$

ahol a számláló a közös előtérpixelek száma, a nevező a két maszk uniójának pixelszáma, ε pedig a 0/0 esetet kizáró numerikus konstans. Az üres maszkok kezelése konvenció kérdése, ezért rögzítjük: üres ground truth példány nincs az annotációban (a háttér nem példány), így M csak nem üres példányokat számol; az üres predikált maszkot a matching előtt kihagyjuk, mert nem példányjelölt. Az üres predikció tehát nem lesz TP, de FP-nek sem számítjuk, és TN sem keletkezik, mert a háttér nem példány. Más protokollok az üres predikciót FP-nek számolhatják, ezért a konvenciót a metrika definíciójával együtt kell közölni.

Rögzített τ IoU-küszöb mellett a predikciókat confidence szerint csökkenő sorrendbe rendezzük, majd greedy, egy-az-egyhez illesztést végzünk: minden predikció a még nem illesztett ground truth példányok közül ahhoz illeszkedik, amellyel a legnagyobb az IoU, ha ez legalább τ. Az illesztett predikció TP, az illesztetlen predikció FP, az illesztetlen ground truth FN. Az így kapott precision–recall görbe alatti terület az instance average precision, röviden AP. Az AP közvetlenül a példányazonosságot bünteti: egy merge hibánál a predikció csak egy ground truth példányhoz illeszkedhet, a másik FN marad, még akkor is, ha a globális pixel-IoU magas.

Az AP itt **rögzített τ mellett** értendő, ezért a jelölése AP@τ (például AP@0,5). A precision–recall görbét úgy kapjuk, hogy a predikciókat confidence szerint csökkenő sorrendben végigvesszük, minden lépésnél a kumulatív $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ precisiont és a TP/M recallt számoljuk, majd a görbe alatti területet a 26.3. NMS, confidence és AP all-point interpolációjával, a precision monoton nem növekvő burkolójával vesszük. A COCO-stílusú jelentés ezt átlagolja a $\tau \in \{0{,}5; 0{,}55; \ldots; 0{,}95\}$ küszöbökön (ez a szokásos „AP”), és külön közli az AP@0,5 és az AP@0,75 értéket; az osztályok és területi szeletek feletti átlagolás további, itt nem részletezett lépés. A jelen egység példái egyetlen τ-t használnak, ezért minden AP-érték AP@τ értendő.

Kis levezetés. Legyen a ground truth két diszjunkt 6 pixeles maszk, g_1, g_2, a predikció pedig az uniójuk: $p = g_1 \cup g_2$, 12 pixel, confidence 0,9. A globális pixel-IoU

$$\frac{|p \cap (g_1 \cup g_2)|}{|p \cup (g_1 \cup g_2)|} = \frac{12}{12} = 1.$$

Ezzel szemben p és g_1, illetve p és g_2 között az IoU $\frac{6}{12} = 0{,}5$. Ha $\tau = 0{,}5$ és a greedy illesztés a $\geq \tau$ szabályt használja, p az egyik g-t elviszi, a másik FN lesz. Egyetlen predikció és $M = 2$ ground truth példány esetén a PR-görbe egyetlen pontból áll: $R = \mathrm{TP}/M = 1/2 = 0{,}5$, $P = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP}) = 1/1 = 1$. Az all-point interpoláció ebből az egyetlen pontból $0{,}5 \cdot 1 + (1 - 0{,}5) \cdot 0 = 0{,}5$ AP-t ad, mert az $R = 1$-hez tartozó precision 0, a hiányzó recall nem pótolható. Ha a küszöb $> 0{,}5$, akkor p egyetlen ground truth példányhoz sem illeszkedik, ezért FP, $P = 0$, $R = 0$, és az AP 0. Az instance-metrika tehát akkor is hibát jelez, amikor a pixelátlag tökéletesnek látszik.

27.4.6 Labor: kétobjektumos annotációs minta és input/output szerződés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja egy eredeti, 8×8-as mintán rögzíteni az instance annotáció sémáját, valamint ellenőrizni, hogy a doboz és az id-maszk konzisztens. Munkakönyvtár: `lab/n108_instance_lab`; a kódot `instance_contract.py` néven ebbe a mappába mentjük. A kód nem igényel PyTorch-ot vagy GPU-t; a futási parancs:

```
cd lab/n108_instance_lab
python instance_contract.py
```

A szerződés a következő. Bemeneti kép: `image` alakja `(8, 8)`, `float32`, $[0, 1]$ intenzitás. Target: `instance_ids` alakja `(8, 8)`, `int64`, értéke 0 háttérre, $1, 2$ példányazonosítóra. `bboxes` alakja `(M, 4)`, `float32`, xyxy sorrend `[xmin, ymin, xmax, ymax]`, fél-nyílt. `labels` alakja `(M,)`, `int64`. Hálózati kimenet: `boxes`, `labels`, `scores`, `masks`; a maszk `(M, H, W)` `float32`, küszöbölés nélküli valószínűség.

```
import json
import numpy as np

# Munkakönyvtár: lab/n108_instance_lab
# Futtatás: python instance_contract.py

H, W = 8, 8
image = np.zeros((H, W), dtype=np.float32)
image[1:3, 1:4] = 0.7   # világos előtér, 1. példány
image[5:7, 2:6] = 0.4   # halványabb előtér, 2. példány

instance_ids = np.zeros((H, W), dtype=np.int64)
instance_ids[1:3, 1:4] = 1   # y = 1..2, x = 1..3
instance_ids[5:7, 2:6] = 2   # y = 5..6, x = 2..5

bboxes = np.array([[1.0, 1.0, 4.0, 3.0],
                   [2.0, 5.0, 6.0, 7.0]], dtype=np.float32)
labels = np.array([1, 1], dtype=np.int64)

# Konzisztencia: minden id legszűkebb doboza egyezik-e a megadott dobozzal.
# Feltétel: a bboxes sorai az id-k növekvő sorrendjében vannak.
present = np.unique(instance_ids[instance_ids > 0])
if len(present) != len(bboxes):
    raise ValueError("az id-k és a dobozok száma eltér")
for idx, m in enumerate(present):
    ys, xs = np.where(instance_ids == m)
    xmin, ymin = xs.min(), ys.min()
    xmax, ymax = xs.max() + 1, ys.max() + 1
    if (xmin, ymin, xmax, ymax) != tuple(int(v) for v in bboxes[idx]):
        raise ValueError(f"{m}. id és doboz nem konzisztens")

semantic = (instance_ids > 0).astype(np.uint8)
expected = {
    "input_shape": [H, W],
    "instance_count": int(np.unique(instance_ids[instance_ids > 0]).size),
    "foreground_pixels": int(semantic.sum()),
    "foreground_ratio": float(semantic.mean()),
    "bbox_areas": [int((x2 - x1) * (y2 - y1))
                   for x1, y1, x2, y2 in bboxes.tolist()],
    "labels": labels.tolist(),
}
assert expected["instance_count"] == 2
assert expected["foreground_pixels"] == 14
assert abs(expected["foreground_ratio"] - 0.21875) < 1e-12
assert expected["bbox_areas"] == [6, 8]
print(json.dumps(expected, indent=2))
```

Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a példa determinisztikus):

```
{
  "input_shape": [
    8,
    8
  ],
  "instance_count": 2,
  "foreground_pixels": 14,
  "foreground_ratio": 0.21875,
  "bbox_areas": [
    6,
    8
  ],
  "labels": [
    1,
```

```
    1
  ]
}
```

A kimenet a fenti mátrixokból számolt értékeket adja: `foreground_pixels=14`, `foreground_ratio=0.21875`, `instance_count=2`, a dobozterületek $3 \cdot 2 = 6$ és $4 \cdot 2 = 8$. Az asszertek a futás során ellenőrzik ezeket az értékeket; a numerikus egyezés `int`-re pontos.

27.4.7 Feladatok

F1 (fogalmi, nehézség 1). Adjon három, egymástól eltérő tulajdonságot, amelyben a szemantikus és az instance szegmentálás szerződése különbözik. Átadandó: egy-egy rövid mondat és egy legalább 2×2-es ellenpélda, ahol ugyanaz a pixelcímke két különböző példányszámot enged meg.

F2 (levezetési, nehézség 2). Adjon 4×4-es bináris szemantikus maszkot és hozzá legalább három különböző, összefüggő instance-felosztást. Igazolja mindháromra az $\mathbf{1}[A_{\text{inst}} > 0] = M_{\text{sem}}$ egyenlőséget, és mutassa meg, hogy a felosztások nem vihetők egymásba pusztán id-permutációval. Átadandó: három id-mátrix, a permutáció-érv magyarázata.

F3 (implementációs, nehézség 2). Írjon függvényt `instance_to_semantic_and_bboxes(ids)` néven, amely egy `(H, W) int64` id-mátrixból előállítja a bináris szemantikus maszkot (`uint8`) és példányonként a legszűkebb xyxy dobozokat `(M, 4) float32` tömbben. Kezelje az üres képet: ekkor a doboz alakja `(0, 4)` legyen. Ellenőrizze a labor 8×8-as mintáján és üres bemeneten. Átadandó: futtatható függvény, hívás, várt és kapott shape.

F4 (hibakeresési, nehézség 2). Az alábbi hibás függvény „instance AP”-nek nevezi az összes predikció–ground-truth pár IoU-jának átlagát, matching nélkül:

```python
def iou(p, g):
    p = np.asarray(p) > 0.5
    g = np.asarray(g) > 0.5
    union = np.logical_or(p, g).sum()
    return float(np.logical_and(p, g).sum() / union) if union else 0.0


def wrong_instance_ap(preds, gts):
    vals = []
    for p in preds:
        for g in gts:
            vals.append(iou(p, g))
    return float(np.mean(vals)) if vals else 0.0
```

Az `iou` a 27.4. szakasz maszk-IoU-ja, a 0/0 esetre itt 0 értékkel. Mutassa meg a merge-példán ($\text{preds} = [g_1 \cup g_2]$, $\text{gts} = [g_1, g_2]$), hogy ez elfedi az FN-t. Adja meg a helyes greedy, egy-az-egyhez illesztés lényegét, és írja át a függvényt. Átadandó: a hiba oka, a hibás érték, a javított kód és a küszöb szerepe.

F5 (kísérlettervezési, nehézség 3). A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab jelenleg osztályozási annotáció, nem instance target. Tervezzen protokollt olyan kétértelmű esetre, ahol két, azonos osztályú, egymással érintkező alakzatot kell külön példányként címkézni. Rögzítse a megfigyelési egységet, az annotációs szabályt, a splitegységet, a metrikát és a baseline-t. Átadandó: egy protokollterv, benne a következtetés érvényességi korlátja, például hogy a szintetikus adat nem klinikai validáció.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

27.4.8 Megoldások és indoklás

F1. A három különbség: (1) a szemantikus kimenet osztálycímke pixelenként, az instance kimenet példányazonosító; (2) a szemantikus kiértékelés pixel-IoU/Dice, az instance kiértékelés példányszintű maszk-IoU matching; (3) a szemantikus cél fix $H \times W$ tensor, az instance cél változó példányszámú, ezért listaszerű szerződést igényel. Példányszámra: egy 2×4-es, végig előtér maszk lehet egyetlen 2×4-es példány (példányszám 1), de címkézhető két 2×2-es példányként is (példányszám 2); az $\mathbf{1}[A_{\text{inst}} > 0] = M_{\text{sem}}$ egyenlőség mindkét esetben teljesül, tehát ugyanaz a pixelcímke két különböző példányszámot enged meg.

F2. 4×4-es szemantikus előtér:

```
1 1 0 0
1 1 0 0
0 0 0 0
0 0 0 0
```

Három felosztás:

```
A1:             A2:             A3:
1 1 0 0         1 2 0 0         1 1 0 0
1 1 0 0         1 2 0 0         2 2 0 0
0 0 0 0         0 0 0 0         0 0 0 0
0 0 0 0         0 0 0 0         0 0 0 0
```

Az A_1 egyetlen példány, A_2 két függőleges sáv, A_3 két vízszintes sáv. Mindháromnál $\mathbf{1}[A_r > 0] = M_{\text{sem}}$. A_2-t és A_3-t id-permutáció sem viheti egymásba: a permutáció csak a címkéket nevezi át, a pixelhalmazok particióját nem. Mindkét felosztásban két 2 pixeles példány van (a méretek multihalmaza $\{2, 2\}$), de A_2-ben a példányok oszlopok, A_3-ban sorok: a $(0, 0)$ és $(0, 1)$ pixel A_2-ben különböző, A_3-ban azonos példányba tartozik. A méretek egyezése tehát nem teremt permutációs ekvivalenciát. Továbbá A_1 példányszáma egy, míg a többieké kettő, ami definíció szerint nem permutáció.

F3. A függvény:

```python
import numpy as np

def instance_to_semantic_and_bboxes(ids):
    if ids.dtype != np.int64:
        raise TypeError("ids legyen int64")
    semantic = (ids > 0).astype(np.uint8)
    rows = []
    for m in np.unique(ids[ids > 0]):
        ys, xs = np.where(ids == m)
        xmin, ymin = xs.min(), ys.min()
        xmax, ymax = xs.max() + 1, ys.max() + 1
        rows.append([xmin, ymin, xmax, ymax])
    bboxes = np.asarray(rows, dtype=np.float32).reshape(-1, 4)
    return semantic, bboxes
```

A labor mintáján `semantic.sum()==14`, `bboxes.shape==(2,4)`, a dobozok dtype-ja `float32`. Üres bemenetre `np.unique(np.array([], dtype=np.int64))` üres, a ciklus nem fut le, az `np.asarray([], dtype=np.float32).reshape(-1, 4)` alakja `(0,4)`, `semantic.sum()==0`. A

dobozok egész pixelhatárokból származnak, ezért egzakt egyezés várható; lebegőpontos tolerancia nem szükséges.

F4. A hiba oka, hogy a függvény minden predikció–GT párt átlagol, ezért nincs egy-az-egyhez illesztés, és egy predikció több ground truth példányhoz is hozzájárulhat. A merge-példán: gts $= [g_1, g_2]$, preds $= [g_1 \cup g_2]$, az IoU-k [0,5; 0,5], a hibás érték 0,5. A helyes érték a τ küszöbtől függ: $\tau = 0{,}5$ és a $\geq \tau$ illesztési szabály mellett a predikció az egyik példányhoz illeszkedik, a másik FN marad, így az AP 0,5 (ezt mutatta a 27.4.5 kis levezetése); $\tau > 0{,}5$ mellett a predikció egyik példányhoz sem illeszkedik, ezért FP, és az AP 0. A küszöböt tehát rögzíteni kell, különben a „javított kód 0-t ad" állítás csak a $\tau > 0{,}5$ esetre igaz.

A javított függvény a 26.3. NMS, confidence és AP all-point AP-ját használja:

```
def instance_ap(preds, scores, gts, tau=0.5):
    order = np.argsort(scores)[::-1]
    matched = np.zeros(len(gts), dtype=bool)
    tp, fp = [], []
    for i in order:
        best_j, best_iou = -1, -1.0
        for j, g in enumerate(gts):
            if matched[j]:
                continue
            v = iou(preds[i], g)
            if v > best_iou:
                best_j, best_iou = j, v
        if best_iou >= tau:
            matched[best_j] = True
            tp.append(1); fp.append(0)
        else:
            tp.append(0); fp.append(1)
    tp = np.cumsum(np.array(tp, dtype=np.float64))
    fp = np.cumsum(np.array(fp, dtype=np.float64))
    if tp.size == 0 or len(gts) == 0:
        return 0.0
    recall = tp / len(gts)
    precision = tp / np.maximum(tp + fp, 1e-12)
    mrec = np.concatenate(([0.0], recall, [1.0]))
    mpre = np.concatenate(([0.0], precision, [0.0]))
    for k in range(mpre.size - 2, -1, -1):
        mpre[k] = max(mpre[k], mpre[k + 1])
    idx = np.where(mrec[1:] != mrec[:-1])[0]
    return float(np.sum((mrec[idx + 1] - mrec[idx]) * mpre[idx + 1]))
```

Ellenőrzés (a fenti mátrixokkal futtatva): a merge-példán $\tau = 0{,}5$ mellett az AP 0,5, $\tau = 0{,}51$ mellett 0; két különálló, jól illeszkedő predikciónál az AP 1. A hibás `wrong_instance_ap` mindhárom esetben 0,5-öt ad, tehát sem a τ hatását, sem az FN-t nem látja.

F5. Egy lehetséges protokoll: két érintkező, azonos osztályú négyzetet generálunk 24×24-es képen, a két példány egyetlen 2 pixel széles érintkezési sávval. A megfigyelési egység a `scene_id`; ugyanaz a scene minden nézete azonos splitbe kerül. Az annotációs szabály: az érintkezési sáv nem háttér; a generátor a sáv minden pixelét a két négyzet valamelyikéhez rendeli, éspedig a sáv középvonala mentén kettéosztva (a középvonalra eső pixelnél a kisebb instance id dönt). Így a sáv pixelei pontosan egy példányhoz tartoznak, háttér pedig csak a két négyzet unióján kívül van; a hozzárendelést a generátor ismert instance id-jából származtatjuk, nem emberi rajzból. A metrika instance AP $\tau = 0{,}5$ maszk-IoU-val. A baseline a [P14]-típusú U-Net szemantikus kimenetének küszöbölése, majd a csatlakozó komponensek példányjelöltként való kezelése; két érintkező példány esetén ez egyetlen komponenst ad, ezért legfeljebb az egyik GT-hez párosítható, és így az AP felülről legfeljebb 0,5 lehet (ennél kisebb, ha a párosított jelölt IoU-ja nem éri el a τ-t). A pontos értéket a kísérlet méri; a hipotézis az, hogy a merge csökkenti az AP-t, de ezt a

mért AP-különbség és a jelenetszintű bootstrap CI dönti el. A split 120/30/30 scene; mivel a szintetikus feladatban egyetlen osztály szerepel, osztály szerinti rétegzés (class-stratified split) nem értelmezhető, ezért a split egyszerű, jelenetszintű véletlen felosztás. Érvényességi korlát: a szintetikus geometria szabályos, ezért az eredmény nem terjeszthető ki természetes, takart vagy átfedő objektumokra vagy magas tétű alkalmazásra; a cél a mérőszám érzékenységének bemutatása.

27.4.9 Tipikus tévedés és ellenpélda

A példányszám és az előtérpixel-arány nem felcserélhető target. Legyen a 8×8-as képen egyszer $M = 1$, egyetlen 2×3-as folt: az előtérarány $6/64 = 0{,}09375$. Máskor hat darab 1 pixeles folt, amelyek páronként nem 4-szomszédosak (tehát nem érintkeznek): a példányszám $M = 6$, az előtérarány ugyancsak $6/64 = 0{,}09375$. Ha a modell a foreground arányt regresszálja, vagy a veszteség csak a szemantikus maszkot bünteti, akkor nem tudja a két annotációt megkülönböztetni. Az instance-feladat tehát külön célfejet és külön metrikát igényel; a pixelarány legfeljebb az előtér mennyiségét írja le, a példányok számát nem.

27.4.10 Önellenőrzés és visszahívás

Ellenőrző kérdések: meg tudja-e adni a $4 + 2$ és $2 + 4$ felosztást úgy, hogy minden id-osztály összefüggő legyen? A doboz fél-nyílt értelmezését használja-e a labor? Az üres predikció TP-ként van-e kezelve az instance AP-ben? A 27.3. Dice, IoU és aggregálás alfejezetből visszahívás: a globális Dice/IoU nem példányszintű; az üres-üres esetet explicit szabálynak kell rögzítenie. A 26.4. Detektálási pipeline és hibaelemzés alfejezetből: az üres cél $M = 0$ állapot, és a doboztranszformáció a képpel szinkronban történik. A 23.4. Hibaszeletek és alkalmazási határ alfejezetből: az összesített metrika elrejtheti a merge hibát. A 15.4. Független gradiensellenőrzési protokoll alfejezetből: a maszkküszöb nem sima művelet, ezért a veszteséget a logiton számoljuk, a küszöb csak kiértékelési döntés.

Válaszkulcs. A $4 + 2$ és $2 + 4$ felosztás megadható úgy, hogy minden id-osztály összefüggő legyen. A labor a fél-nyílt xyxy értelmezést használja. Az üres predikció nem TP: a matching előtt kihagyjuk, FP-nek és TN-nek sem számítjuk, ezért a konvenciót a metrika definíciójával együtt kell közölni. A globális Dice/IoU valóban nem példányszintű, ezért az instance-feladat külön egy-az-egyhez illesztést és üres-egyezményt igényel.

27.4.11 Rövid összefoglaló

A szemantikus szegmentálás minden pixelhez osztálycímkét rendel, az instance szegmentálás ezen túl az azonos osztályú objektumokat is külön példányokra bontja. Megmutattuk, hogy ugyanaz a bináris szemantikus maszk több különböző, összefüggő instance-felosztást enged, ezért az instance-annotáció nem vezethető le a pixelcímkéből. A Mask R-CNN a detektálási fej ROI-it párhuzamos maszkággal egészíti ki. Az instance AP a predikciókat greedy, egy-az-egyhez maszk-IoU-küszöbbel illeszti a ground truth-hoz; a merge hiba magas pixel-IoU mellett is elrejtőzhet. A labor kétobjektumos annotációs mintát és input/output szerződést ad, üres cél kezelésével.

27.4.12 Következő alkalmazás

A 28.1. Embedding, távolság és normalizálás egység a metrikus tanulásba lép: az instance-azonosság itt bevezetett gondolata ahhoz kapcsolódik, hogyan lehet példányok közötti távolságot reprezentációból tanulni, ha pixelmaszk nem áll rendelkezésre.

27.4.13 Források

- [P14] Ronneberger, O., Fischer, P., Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. arXiv:1505.04597. https://arxiv.org/abs/1505.04597.
- [P15] He, K. et al. (2017). *Mask R-CNN*. arXiv:1703.06870. https://arxiv.org/abs/1703.06870.
- [D18] PyTorch contributors (2026). *TorchVision Object Detection Finetuning Tutorial*. PyTorch 2.14 documentation. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html. Hozzáférés: 2026-10-30.

28 Metrikus tanulás, hasonlósági keresés és reprezentációhasználat

28.1 Embedding, távolság és normalizálás

28.1.1 1. Cél, előismeret és induló kérdés

Ebben az alfejezetben az osztályozástól eltérő kimenetet építünk: a CNN egy képet rögzített méretű vektorba, **embeddingbe** képez, amelyben a hasonlóságot távolsággal vagy koszinusz-hasonlósággal mérjük. A 25.4. Nyílt halmaz és ismeretlen input alfejezetben láttuk, hogy a softmax-konfidencia nem hasonlósági mérték; itt a hasonlósági kereséshez tervezett reprezentáció skáláját definiáljuk. Felhasználjuk a norma és skalárszorzat alapjait, valamint a 3.4. Sajátérték, SVD és spektrális határ alfejezetben tárgyalt normakorlát gondolatát: egy lineáris réteg nyújtását a súlymátrixa spektrálnormája korlátozza, de ezt a normát a tanulás nem tartja kicsiben, ezért a nyers kimenet normája önmagában nem megbízható hasonlósági jel.

Induló kérdés. A függelék A. szakaszában leírt GlyphLab két nézete ugyanarról a `scene_id`-ről készül, de különböző zajt kap. Pixelben mérve két ilyen nézet távol lehet; egy tanult `features` kimenetben viszont közel kellene lenniük. Milyen normalizálás mellett lesz a koszinusz-hasonlóság és az euklideszi távolság egymásnak monoton megfelelője, és miért nem automatikus ez a nyers hálókimenetre?

28.1.2 2. Embedding, normalizálás és hasonlósági mértékek

Definíció (feature extractor és embedding). Legyen a bemeneti tér $\mathcal{X} \subseteq [0,1]^{C\times H\times W}$ (egy kép NCHW-tenzora), és legyen $f_\theta : \mathcal{X} \to \mathbb{R}^d$ egy paraméterezett leképezés, például az osztályozó fej előtti CNN-rész. Egy $X = (x_1, \dots, x_N)$ batchre a leképezést képenként alkalmazzuk; a feature extraction kimenete az

$$Z \in \mathbb{R}^{N\times d}, \qquad Z_{i,:} = f_\theta(x_i)^\top, \qquad z_i = (Z_{i,1}, \dots, Z_{i,d})^\top$$

soronkénti embedding. Az absztrakt képletekben egy vektor oszlopvektor; a batchelt `Z` mátrixban egy megfigyelés egy sor. A `d` a választott reprezentációs dimenzió, nem az osztályok száma. A logit is vektor, de a logit osztályozásra optimalizált pontszám; az embeddinget hasonlósági célra tanítjuk, és a kettőt nem cseréljük fel automatikusan.

Definíció (L2-normalizálás). A $0 \neq z \in \mathbb{R}^d$ vektor L2-normalizáltja

$$\hat{z} = \frac{z}{\|z\|_2}, \qquad \|z\|_2 = \left(\sum_{r=1}^{d} z_r^2\right)^{1/2}.$$

A leképezés $z \mapsto \hat{z}$ az egységgömbre, $S^{d-1} = \{v \in \mathbb{R}^d : \|v\|_2 = 1\}$ vetíti a vektort. A nulla vektorra nincs definiálva; a gyakorlatban a háló kimenete ritkán pontosan nulla, de a kódnak explicit

módon kell kezelnie. A normalizálás nem lineáris: pozitív skalárszorosok ugyanarra a pontra kerülnek, ezért a kimeneti skálát tudatosan eltávolítjuk.

Definíció (koszinusz-hasonlóság). Két nemnulla vektorra

$$\cos(z_i, z_j) = \frac{z_i^\top z_j}{\|z_i\|_2 \|z_j\|_2} \in [-1, 1].$$

A tartomány a Cauchy–Schwarz-egyenlőtlenségből következik. Ha z_i és z_j egységnormájú, akkor $\cos(z_i, z_j) = z_i^\top z_j$. A koszinusz-hasonlósághoz tartozó koszinusz-*distance* $d_C = 1 - \cos$; ez $[0; 2]$ értékű, de nem metrika, ahogy az F1 ellenpéldában megmutatjuk.

Definíció (euklideszi távolság).

$$d_2(z_i, z_j) = \|z_i - z_j\|_2 = \left(\sum_{r=1}^{d} (z_{i,r} - z_{j,r})^2 \right)^{1/2}.$$

A négyzetes alakot a levezetésekben gyakran egyszerűbb használni, mert eltűnik a gyök.

Mátrixos alak. Batchelt embeddingre $Z \in \mathbb{R}^{N \times d}$ a soronkénti L2-normalizálás $\hat{Z} = D^{-1} Z$, ahol $D = \operatorname{diag}(\|z_1\|_2, \dots, \|z_N\|_2)$; NumPy-ban ugyanez a `Z / np.linalg.norm(Z, axis=1, keepdims=True)` kifejezés. Ha a mátrix már soronként egységnormájú, akkor a koszinusz-hasonlósági mátrix $S = Z Z^\top \in \mathbb{R}^{N \times N}$, ahol $S_{ij} = \hat{z}_i^\top \hat{z}_j$.

A 3.4. Sajátérték, SVD és spektrális határ spektrális normája éppen a legnagyobb iránymenti nyújtás. A nyers f_θ kimenetben két kép hasonlóságát nemcsak az irány, hanem a rétegek által megtanult, képfüggő norma is befolyásolja; a normalizálás ezt a skálát egységesíti.

28.1.3 3. A normalizált euklideszi távolság és a koszinusz-hasonlóság kapcsolata

A hasonlósági keresés leggyakoribb művelete két mérték, a koszinusz-hasonlóság és az euklideszi távolság közötti választás. A kettő egységnormájú vektorokra levezethető azonossággal kapcsolódik egymáshoz, ezért fontos, hogy a kapcsolatot pontosan kimondjuk.

Tétel 1 (egységnormájú vektorok távolsága). Legyen $u, v \in \mathbb{R}^d$, és tegyük fel, hogy $\|u\|_2 = \|v\|_2 = 1$. Ekkor

$$\|u - v\|_2^2 = 2 - 2\, u^\top v = 2 - 2 \cos(u, v).$$

Ekvivalensen $\cos(u, v) = 1 - \frac{1}{2}\|u - v\|_2^2$.

Bizonyítás (saját, közvetlen levezetés).

$$\|u - v\|_2^2 = (u - v)^\top (u - v) = u^\top u - u^\top v - v^\top u + v^\top v.$$

A skalárszorzat szimmetriája miatt $u^\top v = v^\top u$, a feltétel miatt pedig $u^\top u = \|u\|_2^2 = 1$ és $v^\top v = 1$. Így

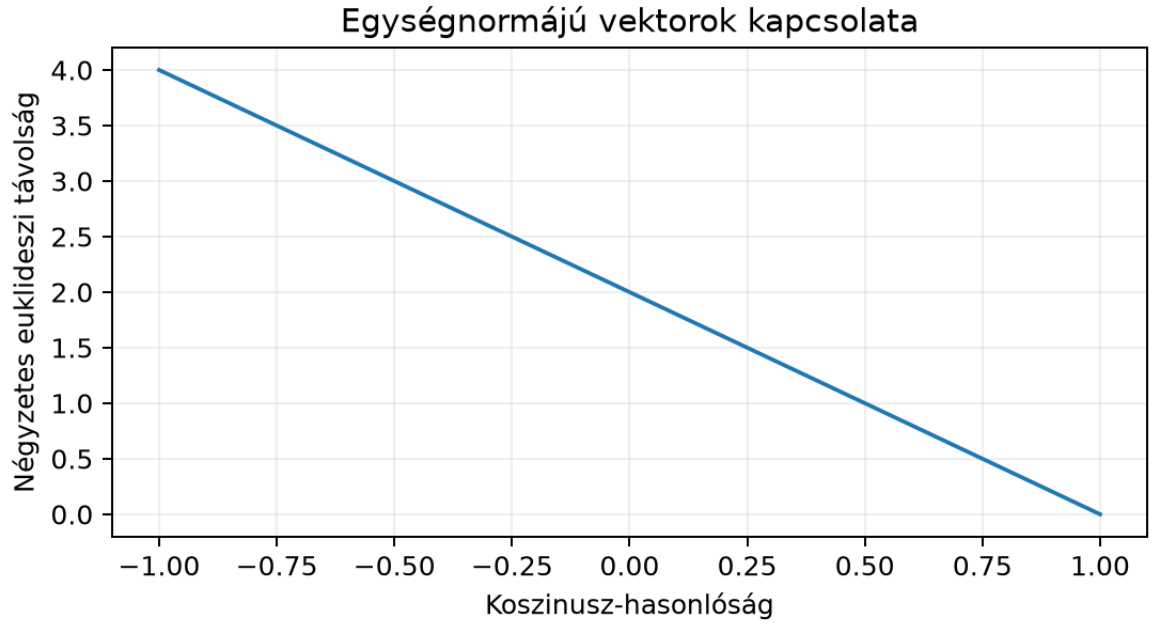
$$\|u - v\|_2^2 = 1 - 2u^\top v + 1 = 2 - 2u^\top v.$$

Egységnormájú vektorokra a koszinusz-hasonlóság definíció szerint $u^\top v$, tehát $2 - 2\cos(u, v)$, ami a bizonyítandó állítás. □

A tétel geometriai jelentése: az egységgömbön a húr hosszának négyzete és a két vektor által bezárt szög koszinusza között a fenti lineáris kapcsolat áll. A tételből következik a gyakorlatban használt rendezési ekvivalencia:

Következmény 2. Ha minden vektor egységnormájú, akkor a koszinusz-hasonlóság szerinti csökkenő rendezés azonos a négyzetes euklideszi távolság szerinti növekvő rendezéssel, mert a $c \mapsto 2 - 2c$ leképezés szigorúan monoton csökkenő $[-1, 1]$-en.

A következmény csak a normalizált esetre érvényes. Nyers vektorokon a két rendezés eltérhet, mert a normák beleszólnak a távolságba. A szakasz végén erre ellenpéldát adunk.



28.1. ábra: Egységnormájú vektorok kapcsolata

A 2-2c összefüggésből számolt szemléltetés, ahol c a koszinusz-hasonlóság; nem mérés.

28.1.4 4. Eredeti kézi példa és minimális kód

Legyen a lekérdezés $q = (1, 0)^\top$, és két adatbázisvektor $a = (2, 0)^\top$, $b = (1{,}5; 0{,}5)^\top$.

Nyers vektorokon az euklideszi távolságok:

$$d_2(q, a) = |2 - 1| = 1, \qquad d_2(q, b) = \sqrt{0{,}5^2 + 0{,}5^2} = \sqrt{0{,}5} \approx 0{,}7071,$$

tehát b közelebb van. A koszinusz-hasonlóságok:

$$\cos(q, a) = \frac{2}{1 \cdot 2} = 1, \qquad \cos(q, b) = \frac{1{,}5}{\sqrt{2{,}5}} \approx 0{,}9487,$$

tehát a hasonlóbb. A rendezés eltér. Normalizáljuk mindkét vektort: $\hat{q} = q$, $\hat{a} = (1,0)^\top$, $\hat{b} \approx (0{,}9487; 0{,}3162)^\top$. A Tétel 1 szerint a négyzetes távolságok:

$$d_2^2(\hat{q},\hat{a}) = 2 - 2 \cdot 1 = 0, \qquad d_2^2(\hat{q},\hat{b}) = 2 - 2 \cdot 0{,}9487 \approx 0{,}1026,$$

így a normalizált térben a a közelebbi is, és a rendezés összhangba kerül a koszinusszal. A norma eltávolítása tehát nem kozmetikai lépés: a hasonlósági keresés szerződését teszi egyértelművé.

Minimális NumPy-kód ugyanerre:

```
import numpy as np

q = np.array([1.0, 0.0], dtype=np.float32)
a = np.array([2.0, 0.0], dtype=np.float32)
b = np.array([1.5, 0.5], dtype=np.float32)

def cos(u, v):
    return float(np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v)))

for z in (a, b):
    z_hat = z / np.linalg.norm(z)
    print(cos(q, z), np.linalg.norm(q - z_hat).item() ** 2)
# Mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU; determinisztikus):
# 1.0 0.0
# 0.9486832618713379 0.10263341609757592
```

A második sorban a Tétel 1-ből számolt $2 - 2c$ érték a normalizált négyzetes távolság.

28.1.5 5. Labor: exact kereső és kézi ellenőrzés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a korábbi laborokban előállított checkpoint(ok); a futásidőt a futtatás során mért értékként kell rögzíteni.

Cél. Kis, CPU-s exact nearest-neighbor kereső, amely a hasonlósági műveletet nem rejti könyvtári metrikafüggvény mögé. Az adatbázis mérete N, a dimenzió d; a naiv keresés költsége $O(Nd)$, ami a GlyphLab méretén triviális.

Input. `lab/embedding_exact.py`; a CNN-úthoz a korábbi főútból származó saját checkpoint és a GlyphLab `data/images.npy`. A kézi ellenőrzés checkpoint nélkül is fut. A kézi ág NumPy-arrayákon, a CNN-ág PyTorch-tenzorokon dolgozik; mindkettő `float32`, device CPU; seedet a determinisztikus kézi ág nem igényel.

A teljes szkript:

```
# lab/embedding_exact.py
from __future__ import annotations

import argparse
from pathlib import Path
import numpy as np


def l2_normalize(Z: np.ndarray, *, min_norm: float = 1e-6) -> np.ndarray:
    """Sor-irányú L2-normalizálás. Z: (N,d) float32.

    A min_norm alatti normájú sor hibát ad: a közel nulla vektor
```

```
    normalizálása a float32 kerekítési hibáját nagyítaná fel.
    """
    Z = np.asarray(Z, dtype=np.float32)
    if Z.ndim != 2:
        raise ValueError(f"shape must be (N,d), got {Z.shape}")
    norm = np.linalg.norm(Z, axis=1, keepdims=True)
    if np.any(norm <= min_norm):
        bad = int(np.argwhere(norm <= min_norm)[0, 0])
        raise ValueError(f"norm <= {min_norm} at row {bad}")
    return Z / norm


def neighbors_cosine(query: np.ndarray, database: np.ndarray, k: int = 1):
    """Pontos k-legmagasabb koszinusz-hasonlóságú szomszéd (legkisebb koszinusz-távolság).

    query: (d,) vagy (1,d); database: (N,d).
    Visszaadja a legmagasabb hasonlóságú sorok indexét és pontszámát.
    """
    query = np.asarray(query, dtype=np.float32).reshape(1, -1)
    database = np.asarray(database, dtype=np.float32)
    if query.shape[1] != database.shape[1]:
        raise ValueError("dimension mismatch")
    q = l2_normalize(query)
    db = l2_normalize(database)
    if k > db.shape[0]:
        raise ValueError(f"k={k} > database size {db.shape[0]}")
    sim = (db @ q.T)[:, 0]
    order = np.argsort(-sim, kind="stable")
    idx = order[:k].astype(np.int64)
    return idx, sim[idx].astype(np.float32)


def manual_check() -> None:
    db = np.array([[3.0, 0.0], [0.0, 2.0], [2.0, 2.0]], dtype=np.float32)
    q = np.array([1.0, 0.0], dtype=np.float32)
    idx, sim = neighbors_cosine(q, db, k=3)

    # Kézi kontroll: normalizálva db = [[1,0],[0,1],[1,1]/sqrt2],
    # q = [1,0], így db-sorrendben sim = [1, 0, 1/sqrt2].
    # Csökkenő hasonlóság szerint: idx = [0,2,1], sim = [1, 1/sqrt2, 0].
    expected_idx = np.array([0, 2, 1], dtype=np.int64)
    expected_sim = np.array([1.0, 0.70710677, 0.0], dtype=np.float32)
    assert np.array_equal(idx, expected_idx), f"index mismatch: {idx}"
    assert expected_idx.shape == idx.shape == (3,)
    assert np.allclose(sim, expected_sim, rtol=1e-5, atol=1e-5), sim
    print("[manual] idx =", idx)
    print("[manual] sim =", sim)


def load_db(path: Path) -> np.ndarray:
    data = np.load(path)
    if "Z" not in data:
        raise KeyError("npz must contain 'Z' with shape (N,d)")
    return np.asarray(data["Z"], dtype=np.float32)


def main(argv=None) -> None:
    p = argparse.ArgumentParser(description="n109 exact cosine search")
    p.add_argument("--manual", action="store_true")
    p.add_argument("--db", type=Path, default=None)
    p.add_argument("--query", type=Path, default=None)
    p.add_argument("--query-index", type=int, default=0)
    p.add_argument("--top-k", type=int, default=5)
```

```
    args = p.parse_args(argv)

    if args.manual:
        if args.db is not None:
            p.error("--manual nem kombinálható --db-vel")
        manual_check()
        return
    if args.db is None or args.query is None:
        p.error("--db and --query are required, or use --manual")

    database = load_db(args.db)
    query_stack = load_db(args.query)
    q = query_stack[args.query_index]
    idx, sim = neighbors_cosine(q, database, k=args.top_k)
    print("query_index =", args.query_index)
    print("top_indices =", idx)
    print("top_sim     =", sim)
    assert np.all(sim >= -1.0 - 1e-6) and np.all(sim <= 1.0 + 1e-6)
    assert np.all(np.diff(sim) <= 0.0), "sim must be non-increasing"


if __name__ == "__main__":
    main()
```

A CNN-úthoz a saját checkpoint betöltése a korábbi főút architektúraosztályát igényli. A lényegi szerződés: `model.features(x)` adjon `(N,d)` embeddinget, vagy adjon `(N,C,H,W)` feature mapot, amelyet global average pooling követ.

```
# lab/extract_embeddings.py -- a saját főút-modellhez illesztendő váz
import numpy as np
import torch
from pathlib import Path

def extract(checkpoint: Path, images: Path, out: Path, d: int = 32) -> None:
    # Helyőrzők: az osztály nevét, a konstruktor argumentumait és a
    # checkpoint kulcsát a saját képzési modul szerződése szerint kell
    # beállítani. A váz a "features" metódust és az (N,d) kimenetet várja.
    from cnn_project import GlyphCNN  # helyi projektimport, nem könyvtári komponens
    state = torch.load(checkpoint, map_location="cpu", weights_only=True)
    if "model_state_dict" not in state:
        raise KeyError("a checkpoint nem tartalmaz 'model_state_dict' kulcsot")
    model = GlyphCNN()
    model.load_state_dict(state["model_state_dict"])
    model.eval()
    X = np.load(images)  # (N,1,24,24) float32
    X = torch.from_numpy(X.astype(np.float32))
    with torch.inference_mode():
        Z = model.features(X)
    if Z.ndim == 4:  # feature map: (N,C,H,W) -> global average pooling
        Z = Z.mean(dim=(2, 3))
    Z = Z.numpy()
    if Z.shape[1] != d:
        raise ValueError(f"feature dim mismatch: {Z.shape[1]} != {d}")
    np.savez(out, Z=Z.astype(np.float32), index=np.arange(len(Z)))

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--checkpoint", type=Path, required=True)
    ap.add_argument("--images", type=Path, required=True)
    ap.add_argument("--out", type=Path, required=True)
    ap.add_argument("--dim", type=int, default=32)
```

```
    a = ap.parse_args()
    extract(a.checkpoint, a.images, a.out, a.dim)
```

A `cnn_project` import, a `GlyphCNN` osztály neve és a `model_state_dict` kulcs a saját főút szerződésének helyőrzői: a checkpoint tényleges formátumát a képzési modul rögzíti, a vázat ahhoz kell igazítani. A hiányzó kulcsot a kód explicit `KeyError`-ral jelzi, nem csendes, félrevezető betöltéssel.

Futtatási parancsok.

```
cd lab
python embedding_exact.py --manual
# A saját checkpoint esetén először:
python extract_embeddings.py --checkpoint glyph_cnn.pt \
  --images ../data/images.npy --out db_embeddings.npz --dim 32
# Ezután egy query (önkereséses indexelési kontroll):
python embedding_exact.py --db db_embeddings.npz \
  --query db_embeddings.npz --query-index 0 --top-k 5
```

A `--manual` ág mért kimenete (Python 3.12.3, NumPy 2.5.4, CPU; a determinisztikus kézi ág nem igényel seedet):

```
[manual] idx = [0 2 1]
[manual] sim = [1.         0.70710677 0.        ]
```

Ellenőrzés és értelmezés. Az utolsó parancsban a query is az adatbázisban van, ezért a 0. találat jó esetben maga a query; ez csak az indexelés helyességét ellenőrzi. A valódi retrieval-kiértékelésnél a queryt ki kell zárni az adatbázisból. A fájl neve ezért `db_embeddings.npz`, nem `test_...`: ez a bemutató adatbázist épít, nem értékel; a test spliten mért eredmény modell- vagy küszöbválasztásra nem használható (lásd F5). A hasonlósági érték 0,707 nem „70,7% azonosság”: a skála $[-1, 1]$, és a küszöböt a feladat határozza meg.

28.1.6 6. Feladatok

F1 – Fogalmi (nehézség 1). Definiálja a koszinusz-hasonlóságot és a koszinusz-*distance*-t. A $u = (1, 0)^\top$, $v = (0, 1)^\top$, $w = (-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}})^\top$ hármasra (számértékben $w \approx (-0{,}7071; 0{,}7071)^\top$) mutassa meg, hogy $d_C = 1 - \cos$ megsérti a metrika háromszög-egyenlőtlenségét. **Átadandó:** a két definíció, és a konkrét számítás, amely cáfolja a metrikatulajdonságot.

F2 – Levezetési (nehézség 2). A 4. szakasz $q = (1, 0)^\top$, $a = (2, 0)^\top$, $b = (1{,}5; 0{,}5)^\top$ vektoraira számítsa ki a nyers koszinusz- és euklideszi rendezést, majd a Tétel 1 alkalmazásával a normalizált rendezést. **Átadandó:** a lépések, a két rendezési táblázat és egy mondat arról, mely esetben cserélhető fel a két mérték.

F3 – Implementációs (nehézség 2). Írjon `row_l2_normalize(Z)` függvényt, amely `(N,d) float32` mátrixot soronként normalizál, nullvektorra explicit hibát dob, és a `(3,4),(0,0),(5,12)` bemenetre ellenőrzi a helyes sorokat, alakot és egységnormákat. **Átadandó:** futtatható kód és a numerikus ellenőrzés.

F4 – Hibakeresési (nehézség 2). Az alábbi kereső hibás:

```
def nearest_bad(query, database, k):
    s = database @ query        # (N,)
    idx = np.argsort(s)[:k]     # növekvő rendezés
    return idx
```

A `query=[1,0]`, `database=[[2,0],[0,1]]`, `k=1` bemenetre mi a kimenet, mi a várt, és mi a két hiba? Adjon javítást normalizálással és csökkenő rendezéssel, és ellenőrizze a javított kódot ugyanezen a példán. **Átadandó:** hibaelemzés, javított kód, visszaellenőrzés.

F5 – Kísérleti/transzfer (nehézség 3). Tervezzen GlyphLab-retrieval protokollt annak mérésére, hogy a test-adatbázisból a query másik view-ja hányadik helyen kerül vissza. A célindex az azonos `scene_id`, eltérő `view_id` kép. **Átadandó:** a hit@k definíció, a query/adatbázis felosztás, a véletlen baseline, a függetlenségi egység, és a lehetséges érvényességi korlát. Tilos a test split felhasználása modell- vagy küszöbválasztásra.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

28.1.7 7. Megoldások és indoklás

F1. A koszinusz-hasonlóság a 2. szakasz képlete. A koszinusz-distance $d_C(u, v) = 1 - \cos(u, v)$. A három vektor egységnormájú: $\|u\|_2 = \|v\|_2 = \|w\|_2 = 1$, mert w komponenseinek négyzete $1/2 + 1/2 = 1$. A pontszorzatok:

$$u^\top w = -\frac{1}{\sqrt{2}} \approx -0{,}7071, \quad u^\top v = 0, \quad v^\top w = \frac{1}{\sqrt{2}} \approx 0{,}7071.$$

Ezért $d_C(u, w) = 1 + \frac{1}{\sqrt{2}} \approx 1{,}7071$, $d_C(u, v) = 1$, $d_C(v, w) = 1 - \frac{1}{\sqrt{2}} \approx 0{,}2929$. A háromszög-egyenlőtlenség $d_C(u, w) \le d_C(u, v) + d_C(v, w)$ lenne, azaz $1{,}7071 \le 1{,}2929$; ez hamis, tehát d_C nem metrika. Ha szögtávolságot használunk, arccos(cos), az már metrika a gömbfelületen; a keresőben a rangoláshoz ez nem szükséges.

F2. A 4. szakasz számításai:

Mérték	Nyers a	Nyers b	Nyers nyerő
koszinusz	1	0,9487	a
euklideszi távolság	1	0,7071	b

Mérték	Normalizált a	Normalizált b	Normalizált nyerő
koszinusz	1	0,9487	a
négyzetes euklideszi	0	0,1026	a

A normalizált esetben a Tétel 1 miatt a rendezések azonosak. A csere tehát akkor érvényes, ha minden vektor egységnormájú; nyers vektorokra a két mérték nem felcserélhető.

F3. Az ellenőrző kód:

```
import numpy as np

def row_l2_normalize(Z, *, min_norm=1e-6):
    Z = np.asarray(Z, dtype=np.float32)
    if Z.ndim != 2:
        raise ValueError(f"shape must be (N,d), got {Z.shape}")
    n = np.linalg.norm(Z, axis=1, keepdims=True)
    if np.any(n <= min_norm):
        bad = int(np.argwhere(n <= min_norm)[0, 0])
```

```
        raise ValueError(f"norm <= {min_norm} at row {bad}")
    return Z / n

Z = np.array([[3.0, 4.0],
              [0.0, 0.0],
              [5.0, 12.0]], dtype=np.float32)

# A nulla sor nélkül:
Z_ok = Z[[0, 2], :]
Z_hat = row_l2_normalize(Z_ok)
assert Z_hat.dtype == np.float32
assert Z_hat.shape == (2, 2)
assert np.allclose(np.linalg.norm(Z_hat, axis=1), [1.0, 1.0],
                   rtol=1e-5, atol=1e-5)
assert np.allclose(Z_hat[0], [0.6, 0.8], rtol=1e-5, atol=1e-5)
# A nullvektor explicit hibát ad:
try:
    row_l2_normalize(Z)
except ValueError as e:
    assert "row 1" in str(e)
print("F3 ok, shape", Z_hat.shape)
```

A `[0,0]` sorra nem adunk csendes `eps` nevezőt, mert az $0/\epsilon = 0$ vektort állítana elő, amelynek nincs iránya, és a koszinusz-hasonlóság nem értelmezhető. A labor `l2_normalize` függvénye ugyanezt a `min_norm = 1e-6` küszöböt használja: a közel nulla norma normalizálása a float32 relatív hibáját nagyítaná fel, ezért azt is hibának tekintjük.

F4. A `query=[1,0]`, `database=[[2,0],[0,1]]`, `k=1` mellett $s = [2, 0]$. A legnagyobb hasonlóságú elem a 0. indexű, de az `argsort(s)` növekvően a $[1, 0]$ indexsorrendet adja, így a kód a legkevésbé hasonlót adja vissza: a hibás kimenet `[1]`, a várt `[0]`. Második hiba: a pontszorzatot normalizálatlan vektorokon számolja, így a normák torzítják a hasonlóságot. A javítás:

```
def nearest_fixed(query, database, k):
    q = query / np.linalg.norm(query)
    db = database / np.linalg.norm(database, axis=1, keepdims=True)
    s = db @ q
    return np.argsort(-s, kind="stable")[:k]
```

Visszaellenőrzés: normalizálás után `s=[1,0]`, `argsort(-s)` a `[0,1]`-t adja, amiből `k=1` mellett a javított kimenet `[0]`. A kimenetet nem elég szövegesen ellenőrizni, az indexeket asserttel rögzítjük:

```
q = np.array([1.0, 0.0])
db = np.array([[2.0, 0.0], [0.0, 1.0]])
assert list(nearest_bad(q, db, 1)) == [1]      # a hibás: a legkevésbé hasonló
assert list(nearest_fixed(q, db, 1)) == [0]    # a javított: a legjobban hasonló
```

A futás ezt a két assertet teljesíti.

F5 megoldásminta. Jelölje Q a 60 testképet, és minden queryre az adatbázis legyen a másik 59 testkép. A cél y_q az a kép, amelynek `scene_id`-je azonos, `view_id`-je eltérő. Legyen $\mathrm{Top}_k(q)$ az indexelt kereső k legmagasabb találatának halmaza; ekkor

$$\mathrm{hit@k} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}\{y_q \in \mathrm{Top}_k(q)\}.$$

A query-k nem függetlenek, mert két nézet ugyanabból a jelenetből származik; a bootstrap-ismétlés egysége ezért `scene_id`, nem kép. Véletlen baseline, ha a queryt kizártuk: $k/59$, feltéve, hogy querynként pontosan egy pozitív van; ez a GlyphLab szerződésben teljesül, mert jelenetenként

pontosan két nézet van, így az azonos `scene_id`-jű, eltérő `view_id`-jű kép egyedi. A kísérlet előre rögzíti a befagyasztott CNN-checkpointot; a test halmazon küszöböt vagy modellváltozatot nem választunk. Egyetlen nagy hit@k jó indikáció, de nem bizonyít szemantikai azonosságot, és a GlyphLab kis mérete miatt a konfidenciaintervallum széles.

28.1.8 8. Tipikus tévedés, önellenőrzés és visszahívás

A legveszélyesebb tévedés, hogy **a közeli embedding automatikusan szemantikai azonosságot jelent**. A hasonlósági kereső azt adja vissza, ami a tanult távolság szerint közel van; ez lehet ugyanolyan orientációjú, de másik `scene_id`-jű GlyphLab-jelenet is. Két különböző jelenet azonos osztályú alakzata hasonló képi helyen és hasonló geometriával jelenhet meg, ezért nagyon közeli embeddinget kaphat, miközben nem ugyanaz a példány. Az arc-hasonlósági rendszerekben sem tekinthető a küszöb feletti hasonlóság önmagában azonosság igazolásának: a döntéshez kalibrált küszöb, rögzített döntési protokoll és szükség esetén emberi kontroll kell. Ez összhangban áll a 25.4. Nyílt halmaz és ismeretlen input alfejezetben tárgyalt korláttal: a nagy softmax-konfidencia sem pótolja a felhasználási szerződést.

Önellenőrzés.

1. Normált u, v vektorokra $\cos(u, v) = 0{,}2$. Mennyi $\|u - v\|_2^2$? Válasz: $2 - 2 \cdot 0{,}2 = 1{,}6$.
2. Lineáris-e az L2-normalizálás? Nem; például $T((1,0)) + T((0,1)) = (1,1)$, de $T((1,0) + (0,1)) = (1/\sqrt{2}, 1/\sqrt{2})$, a kettő különbözik.
3. A 0,8 koszinusz-hasonlóság miért nem „80% azonosság”? Mert a koszinusz-skála $[-1, 1]$-en mért geometriai illeszkedés, nem valószínűség vagy azonossági arány.
4. Mit ront el a query önmagának az adatbázisban hagyása? A hit@k-ot mesterségesen felfelé torzítja, mert a query önmagával triviálisan 1 hasonlóságú.

Korábbi fogalmak rövid visszahívása. A 3.4. Sajátérték, SVD és spektrális határ spektrális normája megmutatta, hogy egy lineáris réteg maximális nyújtása σ_1; ezért a nyers hálókimenet normája rétegenkénti skálától függ. A 16.1. Eltolási equivariance és invariance alfejezetben bizonyítottuk, hogy a valid cross-correlation a belső tartományon eltolási ekvivariáns, a globális átlagpooling pedig invarianciát ad; az L2-normalizálás utóbbit nem érinti, mert csak a vektor hosszát skálázza. A retrieval viselkedését mégis külön kell mérni, mert a normalizált hasonlósági rangsor a szöginformációra épül. A 24.1. Tanulási görbe és hibadiagnózis diagnosztikai elve ide is átvihető: ha a retrieval rossz, a hibát külön kell keresni a normalizálásban, a query-kizárásban, a feature extractorban és a célindexelésben, nem egyetlen feltételezett okban.

28.1.9 Rövid összefoglaló

Az embedding olyan tanult d-dimenziós vektor, amelyet hasonlósági keresésre tervezünk; a logittól fogalmilag különbözik. Az L2-normalizálás az egységgömbre vetít, és eltávolítja a háló kimeneti skáláját. Egységnormájú vektorokra levezettük a $\|u - v\|_2^2 = 2 - 2u^\top v$ azonosságot, ezért a koszinusz-hasonlóság és a négyzetes euklideszi távolság rendezése ekvivalens. Nyers vektorokra ez az ekvivalencia nem áll fenn. A labor egy kis exact koszinusz-keresőt és a saját CNN checkpointból nyert embeddingek használatát adja. A közeli embedding azonban nem automatikus azonosság; a felhasználási döntéshez küszöböt, protokollt és korlátot kell rögzíteni.

Az induló kérdésre a válasz: egységnormájú vektorokra a koszinusz-hasonlóság és a négyzetes euklideszi távolság a $\|u - v\|_2^2 = 2 - 2u^\top v$ azonosság miatt monoton megfelelője egymásnak; nyers hálókimenetre ez nem automatikus, mert a vektorok normája eltér, ezért L2-normalizálás kell.

28.1.10 Következő alkalmazás

A 28.2. Sziámi és triplet cél ezt a reprezentációt tanítja be közvetlenül hasonlósági céllal: ott a triplet loss azt fogalmazza meg, hogy az azonos entitás két nézete közelebb legyen, mint egy idegen nézet, legalább egy előre rögzített $\alpha > 0$ margóval: $d(a, p) + \alpha \leq d(a, n)$.

28.1.11 Felhasznált források

A fejezet levezetései (a normalizált euklideszi távolság és a koszinusz-hasonlóság azonossága, valamint a mátrixos alak) saját, közvetlen levezetések. A fejezet külső forrást nem használ; a FaceNet- és scikit-learn-hivatkozások a következő, 28.2. Sziámi és triplet cél egység forráslistájában szerepelnek, ahol a szöveg ténylegesen épít rájuk.

28.2 Sziámi és triplet cél

28.2.1 Cél, előismeret és induló kérdés

A 28.1. Embedding, távolság és normalizálás egységben definiáltuk az embeddinget mint $f_\theta(x) = z \in \mathbb{R}^d$ leképezést, az L2-normalizálást és a koszinusz-hasonlóságot. Most azt vizsgáljuk, milyen veszteségfüggvények tanítanak be olyan teret, amelyben a hasonlósági kérdések közvetlenül, euklideszi vagy koszinusz-távolsággal megválaszolhatók. Induló kérdés: ha egy minibatchben három kép van, honnan tudjuk, melyik kettőt kell közelíteni, és mi történik, ha a „negatív” címke téves?

28.2.2 Sziámi hálózat, pár és triplet: definíciók

1. definíció (sziámi, siamese). Legyen $f_\theta : \mathcal{X} \to \mathbb{R}^d$ egy megosztott paraméterű hálózat, ahol θ közös. Két vagy több x_i, x_j bemenetre a reprezentációk $z_i = f_\theta(x_i)$, $z_j = f_\theta(x_j)$ ugyanazzal a θ-val készülnek. A sziámi elnevezés tehát a súlymegosztási és tanítási mintára vonatkozik, nem egy külön rétegtípusra. A 28.1. Embedding, távolság és normalizálás jelölését követve a képbemenet alakja $x \in \mathbb{R}^{C \times H \times W}$, a batchelt embedding $Z \in \mathbb{R}^{N \times d}$, ahol egy megfigyelés egy sor. A közös θ következménye, hogy a három ág hibájának gradiense a láncszabály szerint ugyanabban a paramétertérben adódik össze. A sziámi architektúra klasszikus forrása Bromley és mtsai (1993) aláírás-ellenőrző munkája.

2. definíció (pár). Egy pár (i, j, y_{ij}), ahol $y_{ij} = 1$ pozitív pár, ha x_i és x_j azonos osztályú; $y_{ij} = 0$ negatív pár, ha eltérő. A pozitív/negatív fogalmát a rendelkezésre álló felügyelt címke definiálja, nem a pillanatnyi beágyazási távolság.

3. definíció (triplet). Egy triplet (a, p, n) rendezett hármas, ahol a anchor, p pozitív: $y_a = y_p$; n negatív: $y_a \neq y_n$. A triplet három, batchbeli sorra hivatkozik: $z_a, z_p, z_n \in \mathbb{R}^d$.

28.2.3 Kontrasztív és triplet margin veszteség

Jelölje $D(z_i, z_j) = \|z_i - z_j\|_2$ az euklideszi távolságot, négyzetét pedig $D^2(z_i, z_j) = \|z_i - z_j\|_2^2$. A d_{ap}^2 és d_{an}^2 jelölés a megfelelő D^2 rövidítése; d önmagában nem használt.

4. definíció (kontrasztív cél, contrastive loss). Legyen a párra $d^2 = D^2(z_i, z_j) = \|z_i - z_j\|_2^2$ és $m > 0$ margin a távolságnégyzet egységében:

$$L_{\text{pair}}(z_i, z_j, y_{ij}) = y_{ij}\, d^2 + (1 - y_{ij})\, [m - d^2]_+,$$

ahol $[t]_+ = \max(0, t)$. A batchveszteség a párok átlaga. A pozitív tag a közelséget, a negatív tag azt bünteti, ha $d^2 < m$. A klasszikus kontrasztív veszteség (Hadsell, Chopra és LeCun, 2006) a margót a távolságon értelmezi; az itt használt négyzetes alak a deriváltat egyszerűsíti, és a margin egysége is négyzetes.

5. definíció (triplet margin loss, squared változat). Legyen $\alpha > 0$ margin a távolságnégyzet egységében:

$$L_{\text{trip}}(z_a, z_p, z_n) = \left[\|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha\right]_+.$$

Ez a [P16]-ban leírt elvhez illeszkedik: a beágyazást közvetlenül triplet-megkötésekkel tanítjuk; a képlet pontos FaceNet-változatához a cikk vonatkozó szakasza olvasandó. A FaceNet a kimeneti beágyazést L2-normalizálja, mielőtt a triplet veszteséget alkalmazza. A beépített implementációk eltérnek abban, hogy a távolságot vagy annak négyzetét használják: a PyTorch `TripletMarginLoss` például a (négyzetre nem emelt) euklideszi távolságon dolgozik, ezért beépített osztály használata előtt a helyi API-referenciát ellenőrizni kell.

28.2.4 A triplet loss zérus és nemzérus tartománya

1. lemma (a squared triplet loss zérus halmaza). Legyen $z_a, z_p, z_n \in \mathbb{R}^d$ és $\alpha > 0$. Ekkor

$$L_{\text{trip}} = 0 \iff \|z_a - z_n\|^2 \geq \|z_a - z_p\|^2 + \alpha.$$

Ha a feltétel nem teljesül, azaz $\|z_a - z_n\|^2 < \|z_a - z_p\|^2 + \alpha$, akkor

$$L_{\text{trip}} = \|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha > 0.$$

Bizonyítás (saját levezetés). Legyen $t = \|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha$. A $[\cdot]_+$ definíciója szerint $[t]_+ = 0$, ha $t \leq 0$, és $[t]_+ = t$, ha $t > 0$. A $t \leq 0$ feltétel átrendezve $\|z_a - z_n\|^2 \geq \|z_a - z_p\|^2 + \alpha$; a $t > 0$ feltétel a fordított szigorú egyenlőtlenség, és ekkor $\alpha > 0$ miatt az érték is pozitív. Az egyenlőségi határon $t = 0$, tehát a veszteség zérus, de a marginpont nem sima pont.

Következmény (kapcsolat a koszinusszal). Ha z_a, z_p, z_n mind egységnormájú, a 28.1. Embedding, távolság és normalizálás alfejezetben levezetett $\|u - v\|^2 = 2(1 - \cos(u, v))$ alapján a zérus feltétel:

$$\cos(z_a, z_n) \leq \cos(z_a, z_p) - \frac{\alpha}{2}.$$

A margin tehát normalizált beágyazásban a pozitív koszinusz-hasonlóság felett $\alpha/2$ előnyt követel meg.

Kézi számpélda. Legyen $d = 2$, $\alpha = 0{,}2$.

1. $z_a = (0,0)^\top$, $z_p = (1,0)^\top$, $z_n = (2,0)^\top$. Ekkor $d_p^2 = 1$, $d_n^2 = 4$, $t = 1 - 4 + 0{,}2 = -2{,}8$, tehát $L = 0$.
2. $z_a = (0,0)^\top$, $z_p = (2,0)^\top$, $z_n = (0,1)^\top$. Ekkor $d_p^2 = 4$, $d_n^2 = 1$, $t = 4 - 1 + 0{,}2 = 3{,}2$, tehát $L = 3{,}2 > 0$.
3. Határeset: $d_p^2 = 1$, $d_n^2 = 1{,}2$ mellett $t = 1 - 1{,}2 + 0{,}2 = 0$, így $L = 0$.

Az 1. példában a negatív távol van ($d_n^2 = 4$), ezért a margin bőven teljesül, és a veszteség zérus; a 2. példa aktív, kemény tripletet mutat, a 3. példa pedig a margin határát.

28.2.5 Gradiens és hard negative

Ha a triplet aktív, $L = \|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha$ sima. A négyzetes norma deriváltjából $t > 0$ esetén:

$$\nabla_{z_a} L = 2(z_a - z_p) - 2(z_a - z_n) = 2(z_n - z_p),$$

$$\nabla_{z_p} L = -2(z_a - z_p) = 2(z_p - z_a), \qquad \nabla_{z_n} L = 2(z_a - z_n).$$

Minden gradiens $\mathbb{R}^d$-beli vektor, alakja megegyezik a megfelelő z beágyazás alakjával. A $t = 0$ pontban a $[\cdot]_+$ nem klasszikusan differenciálható, ugyanúgy, ahogy a ReLU nullpontja a 10.3. Gradiensellenőrzés és numerikus hibamérce gradient checkben; a fenti képletek csak az aktív tartományra érvényesek.

6. definíció (aktív és kemény negatív). A triplet *aktív*, ha a marginfeltétel sérül: $\|z_a - z_n\|^2 < \|z_a - z_p\|^2 + \alpha$, azaz $L > 0$. Ezen belül szokás *kemény negatívnak* nevezni a $D(z_a, z_n) < D(z_a, z_p)$ esetet; a $D(z_a, z_p) < D(z_a, z_n) < \sqrt{D^2(z_a, z_p) + \alpha}$ tartomány a négyzetes távolságú veszteség szokásos *semi-hard* (félig kemény) megfelelője; a FaceNet-definíció mindkét határt szigorú egyenlőtlenséggel adja meg. A terminológia a konkrét mining-eljárás (negatívbányászati eljárás) szerint változhat. A mining-stratégiák közül a *batch-all* (teljes batch) minden lehetséges tripletet képez a batchben és átlagol, a *batch-hard* (legkeményebb tripletek) csak a legkeményebbeket tartja meg; ez utóbbi kisebb számítású, de nagyobb gradiensvarianciájú heurisztika (Hermans, Beyer és Leibe, 2017). Az egyes szakirodalmi és szoftveres megoldások a *semi-hard* és *hard* eseteket eltérő mining-konvenció szerint definiálhatják, ezért a használt kritériumot a kísérleti protokollban rögzíteni kell.

28.2.6 Collapse és a margin nélküli ellenpélda

7. definíció (collapse, összeomlás). Azt mondjuk, hogy a beágyazás collapse-olt, ha létezik $c \in \mathbb{R}^d$, amelyre minden vizsgált x bemenetre $f_\theta(x) = c$.

Állítás. $\alpha > 0$ mellett a triplet loss nem lehet zérus egy teljes mértékben collapse-olt beágyazásra, mert $d_p^2 = d_n^2 = 0$, így $L = [0 - 0 + \alpha]_+ = \alpha > 0$. Ezzel szemben $\alpha = 0$ esetén a konstans leképezésre $L = [0]_+ = 0$; tehát egy margin nélküli squared triplet cél elfogadná a haszontalan, teljesen egybeeső reprezentációt. Ez a margin szükségességének formális indoka a squared változatra; az állítás saját levezetés a lemma következményeként. A pozitív margin miatt az összeomlott embedding vesztesége pozitív, de ebből nem következik, hogy a gradiens biztosan kivezet az összeomlásból: négyzetes euklideszi triplet lossnál az azonos pontban ($f(x) = c$) a gradiens nulla lehet pozitív loss mellett (a d_p^2 és d_n^2 gradiense mind az anchor, mind a pozitív,

mind a negatív pontban eltűnik, ha $z_a = z_p = z_n$), tehát a collapse stacionárius pont. A margin nem bizonyítja a collapse elkerülését; a gyakorlatban a normalizálás (egységgömb) és a bányászott, nem túl távoli negatívok használata (mining) csökkentheti ezt a kockázatot (Hermans, Beyer és Leibe, 2017). A margin nem elégséges garancia: ha a mintavételezés csak távoli negatívokat ad, a hálózat alacsony rangú, finom szerkezet nélküli megoldást is találhat.

28.2.7 Labor: szintetikus triplet veszteség és megosztott lineáris réteg gradiense

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja a squared triplet veszteség numerikus kontrollja és annak igazolása, hogy a három ágon átfolyó közös paraméter gradiense a három ág gradiensének összege. A PyTorch-kód a `lab` munkakönyvtárból futtatható; a kimenet a megadott környezetben mért eredmény (Python 3.12.3, PyTorch 2.15.0+cpu, CPU).

```
# lab/n110_triplet_lab.py
# Futtatás a labor könyvtárból: cd lab && python n110_triplet_lab.py
import torch

# Nincs véletlen ág: minden súly kézzel rögzített, ezért seed sem kell.
dtype = torch.float32
device = torch.device("cpu")


def triplet_squared_loss(z_a, z_p, z_n, margin):
    # z_a, z_p, z_n: (N, d); margin: float >= 0; reduction: átlag
    d_ap = ((z_a - z_p) ** 2).sum(dim=1)
    d_an = ((z_a - z_n) ** 2).sum(dim=1)
    return torch.clamp(d_ap - d_an + margin, min=0.0).mean()


# 1. Kézi kontroll a két numerikus példára
z_a = torch.tensor([[0.0, 0.0]], dtype=dtype, device=device)
z_p = torch.tensor([[1.0, 0.0]], dtype=dtype, device=device)
z_n = torch.tensor([[2.0, 0.0]], dtype=dtype, device=device)
loss_zero = triplet_squared_loss(z_a, z_p, z_n, 0.2)
print("loss_zero =", loss_zero.item(), "(kézi kontroll: 0.0)")

z_a2 = torch.tensor([[0.0, 0.0]], dtype=dtype, device=device)
z_p2 = torch.tensor([[2.0, 0.0]], dtype=dtype, device=device)
z_n2 = torch.tensor([[0.0, 1.0]], dtype=dtype, device=device)
loss_pos = triplet_squared_loss(z_a2, z_p2, z_n2, 0.2)
print("loss_pos =", loss_pos.item(), "(kézi kontroll: 3.2)")
assert abs(loss_zero.item() - 0.0) < 1e-7
assert abs(loss_pos.item() - 3.2) < 1e-6


class LinearEmbedding(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = torch.nn.Linear(3, 2, bias=False)

    def forward(self, x):
        # x: (N, 3), kimenet: (N, 2)
        return self.fc(x)


# Rögzített, kézzel beállított súly; az x bemenetek leaf tenzorok,
# ezért a gradiens csak a súlyra folyik.
```

```
W_data = torch.tensor([[0.5, -0.2, 0.1],
                       [0.3, 0.4, -0.1]], dtype=dtype, device=device)
x_a = torch.tensor([[1.0, 0.0, 0.0]], dtype=dtype, device=device)
x_p = torch.tensor([[0.0, 1.0, 0.0]], dtype=dtype, device=device)
x_n = torch.tensor([[0.0, 0.0, 1.0]], dtype=dtype, device=device)

net = LinearEmbedding().to(device)
with torch.no_grad():
    net.fc.weight.copy_(W_data)

z_a = net(x_a)
z_p = net(x_p)
z_n = net(x_n)
loss = triplet_squared_loss(z_a, z_p, z_n, 0.2)
loss.backward()
grad_shared = {name: p.grad.detach().clone()
               for name, p in net.named_parameters() if p.grad is not None}

# Három független másolat ugyanazzal a súlyvektorral;
# az áganként számított gradiensek összegét hasonlítjuk a megosztotthoz.
nets = [LinearEmbedding().to(device) for _ in range(3)]
for net_i in nets:
    with torch.no_grad():
        net_i.fc.weight.copy_(W_data)

z_list = [nets[0](x_a), nets[1](x_p), nets[2](x_n)]
loss_split = triplet_squared_loss(*z_list, 0.2)
loss_split.backward()

grad_sum = {}
for net_i in nets:
    for name, p in net_i.named_parameters():
        if p.grad is None:
            continue
        g = p.grad.detach()
        grad_sum[name] = grad_sum.get(name, torch.zeros_like(g)) + g

max_diff = max((grad_shared[name] - grad_sum[name]).abs().max().item()
               for name in grad_shared)

# Független analitikai referencia a loss képletéből, autograd nélkül:
# L = ||u||^2 - ||v||^2 + m, u = z_a - z_p, v = z_a - z_n,
# ezért dL/dW = 2*u^T*(x_a - x_p) - 2*v^T*(x_a - x_n).
u = (x_a - x_p) @ W_data.T                      # (1, 2)
v = (x_a - x_n) @ W_data.T                      # (1, 2)
grad_analytic = 2.0 * (u.T @ (x_a - x_p)) - 2.0 * (v.T @ (x_a - x_n))
max_diff_analytic = (grad_shared["fc.weight"] - grad_analytic).abs().max().item()

print("loss =", loss.item(), "(kézi kontroll: 0.38)")
print("max gradiens diff =", max_diff)
print("megosztott gradiens egyezik:", max_diff < 1e-6)
print("analitikus gradiens diff =", max_diff_analytic,
      "egyezik:", max_diff_analytic < 1e-6)
assert abs(loss.item() - 0.38) < 1e-6
assert max_diff < 1e-6
assert max_diff_analytic < 1e-6
```

Mért kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU; a súlyok kézzel rögzítettek, ezért seed nem kell):

```
loss_zero = 0.0 (kézi kontroll: 0.0)
loss_pos = 3.200000047683716 (kézi kontroll: 3.2)
loss = 0.37999993562698364 (kézi kontroll: 0.38)
```

```
max gradiens diff = 0.0
megosztott gradiens egyezik: True
analitikus gradiens diff = 0.0 egyezik: True
```

A megadott `W_data` súllyal a képletből $z_a = (0{,}5; 0{,}3)$, $z_p = (-0{,}2; 0{,}4)$, $z_n = (0{,}1; -0{,}1)$, ezért $d_{ap}^2 = 0{,}5$, $d_{an}^2 = 0{,}32$ és $L = 0{,}5 - 0{,}32 + 0{,}2 = 0{,}38$; a futtatott kimenet ettől float32-ben 10^{-7} nagyságrendben tér el. Két kontroll szerepel. A `max_diff` a súlymegosztott autograd-grádiens és a három külön példány gradiensének összegét hasonlítja össze; ez ugyanazon autograd két számítási útjának egyezése, ezért önmagában nem független ellenőrzés. A `max_diff_analytic` ezzel szemben a loss képletéből, autograd nélkül számolt $\nabla_W L = 2u^\top(x_a - x_p) - 2v^\top(x_a - x_n)$ referenciához hasonlít, ahol $u = z_a - z_p$ és $v = z_a - z_n$. A mért futásban mindkét eltérés 0,0, és az asszertek is ezt rögzítik: az autograd a közös súlyhoz a három ág VJP-jének (vektor–Jacobian szorzat) összegét rendeli, és ez a képletből függetlenül is igazolt.

28.2.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a sziámi súlymegosztás és két független hálózat között a triplet cél szempontjából. Miért nem elegendő két külön paraméterezésű háló az anchor–pozitív és anchor–negatív pár feldolgozásához? Beadandó: legfeljebb öt mondatos indoklás a gradiensösszegzés következményével.

F2 (levezetési, nehézség 2). Vezesse le a squared triplet loss aktív esetének gradiensét a z_a, z_p, z_n változókra. Ellenőrizze, hogy minden gradiens $\mathbb{R}^d$-beli, és hogy $\nabla_{z_a} L = 2(z_n - z_p)$. Beadandó: minden lépés a négyzetes norma deriváltjából.

F3 (implementációs, nehézség 2). Írjon `pair_squared_loss(z_i, z_j, y, m)` függvényt a kontrasztív célra NumPy vagy PyTorch használatával. Ellenőrizze az $y = 1$, $z_i = (0, 0)$, $z_j = (1, 0)$, $m = 0{,}5$ párra, hogy a kimenet 1,0; az $y = 0$, ugyanezen vektorokra a kimenet $[0{,}5 - 1]_+ = 0$. Beadandó: kód és a két kimenet.

F4 (hibakeresési, nehézség 2). Az alábbi kód a triplet veszteséget akarja kiszámítani, de egy olyan tripletre is nullát ad, ahol a negatív közelebb van az anchorhoz, mint a pozitív. Keresse meg a hibát, írja le a következményt, és javítsa:

```
d_ap = ((z_a - z_p) ** 2).sum(dim=1)
d_an = ((z_a - z_n) ** 2).sum(dim=1)
loss = torch.relu(d_an - d_ap + margin).mean()   # hiba itt
```

Beadandó: a hiba oka, a hibás viselkedés egy konkrét numerikus példán, és a javított sor.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab train/val/test felosztásán triplet-alapú reprezentációtanulást, amelynek célja, hogy egy test képhez a train-képek közül az azonos orientációjúak legyenek a legközelebb (az orientáció a GlyphLab három osztálycímkéjének egyike, tehát az „azonos orientáció” az „azonos osztály” szinonimája). Adja meg a triplet-képzés szabályát a `scene_id` figyelembevételével, egy baseline-t, egy retrieval-metrikát és két érvényességi korlátot. Beadandó: protokollterv, nem szükséges kód.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

28.2.9 Megoldások és indoklás

F1. A sziámi hálózatban mindhárom ág ugyanazt a θ paramétert használja, ezért a triplet veszteség egyetlen közös térben értelmezi a pozitív közelséget és a negatív távolságot. Két független hálózat esetén az anchor–pozitív út és az anchor–negatív út külön transzformációt tanulhatna; a hasonlósági döntéshez nem lenne közös metrika. A közös θ gradiensére a három ág hibájának VJP-je összegződik: ha az autograd gráfban ugyanaz a súly három lineáris úton szerepel, a láncszabály összeadja az ágakat.

F2. Legyen $t > 0$, tehát $L = \|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha$. A $\|u\|^2$ gradiensére $\nabla_u \|u\|^2 = 2u$. Az első tag z_a szerinti gradiense $2(z_a - z_p)$, a másodiké $-2(z_a - z_n)$. Összegük:

$$\nabla_{z_a} L = 2(z_a - z_p) - 2(z_a - z_n) = 2(z_n - z_p).$$

A z_p szerinti derivált csak az első tagból származik: $\nabla_{z_p} L = -2(z_a - z_p) = 2(z_p - z_a)$. A z_n szerinti derivált csak a második tagból: $\nabla_{z_n} L = 2(z_a - z_n)$. Alakellenőrzés: mindhárom kifejezés $\mathbb{R}^d$-beli, mert $z_n - z_p \in \mathbb{R}^d$; a veszteség skalár. A α konstans deriváltja nulla.

F3. Egy lehetséges megoldás:

```
import torch

def pair_squared_loss(z_i, z_j, y, m):
    d2 = ((z_i - z_j) ** 2).sum()
    return y * d2 + (1 - y) * torch.clamp(m - d2, min=0.0)

z_i = torch.tensor([0.0, 0.0])
z_j = torch.tensor([1.0, 0.0])
print(pair_squared_loss(z_i, z_j, 1.0, 0.5).item())  # 1.0
print(pair_squared_loss(z_i, z_j, 0.0, 0.5).item())  # 0.0
```

$d^2 = 1$, ezért $y = 1$ esetén $L = 1$; $y = 0$ esetén $m - d^2 = -0{,}5$, tehát a clamp nulla.

F4. A hibás kód a két távolságnégyzetet felcseréli: `relu(d_an - d_ap + margin)` akkor büntet, ha a negatív túl távol van. Konkrét példa: $d_p^2 = 4$, $d_n^2 = 1$, $\alpha = 0{,}2$ mellett a helyes loss 3,2, a hibás kód viszont $[1 - 4 + 0{,}2]_+ = 0$-t ad, tehát a legkeményebb tripletet figyelmen kívül hagyja. Következmény: a modell nem kap nyomást a negatív eltávolítására. Javítás:

```
loss = torch.relu(d_ap - d_an + margin).mean()
```

F5. Egy lehetséges terv. Kérdés: ad-e a triplet beágyazás jobb retrieval-t, mint a nyers intenzitásvektor? Megfigyelési egység: jelenet (`scene_id`), nem kép. Split: a rögzített GlyphLab train/val/test; triplet csak trainből, a `val` a mining-margin kiválasztására, a `test` csak a záróértékelésre. Triplet-képzés: a GlyphLab három osztálycímkéje maga az orientáció (0 = függőleges, 1 = vízszintes, 2 = átlós), ezért az „azonos orientáció” és az „azonos osztály” ugyanazt jelenti; a triplet pozitív/negatív címkéjét ez a class label adja, nem a pillanatnyi embedding-távolság. Batch-all mellett az anchorhoz minden azonos osztályú elem pozitív, minden eltérő osztályú negatív; anchor és pozitív nem lehet ugyanaz a kép. A `scene_id` a split és a bootstrap egysége: a két view azonos `scene_id`-jű és azonos osztályú, ezért egymás pozitívja lehet, de a query/gallery felosztásban nem kerülhetnek ellenkező oldalra. Baseline: 24×24-es, L2-normalizált nyers intenzitásvektor, euklideszi távolsággal. Keresett tényező: margin a $\{0{,}1; 0{,}2; 0{,}5\}$ halmazból. Metrika: query = test, gallery = train; a scene-szintű split miatt a query- és gallery-jelenetek diszjunktak, így a gallery nem tartalmazza a query jelenetének másik nézetét, és a kiértékelés nem trivializálódik. A recall@1 azon queryk aránya, amelyeknél az első találat azonos orientációjú; a mAP@5 a querynkénti AP@5 átlaga. Mivel a kiegyensúlyozott GlyphLab-splitben osztályonként

80 releváns train-kép van (40 train-jelenet osztályonként, jelenetenként két nézet), az AP@5 képlete $\frac{1}{5}\sum_{i=1}^{5} P(i)\,\mathbf{1}\{\text{az } i.\text{ találat releváns}\}$, ahol $P(i)$ az első i találat precíziója; a relevancia az azonos orientációs címke, a tie-break pedig rögzített (például index szerint növekvő). A [D15] forrás az osztályozási metrikákra vonatkozik, a query-szintű mAP@5-öt nem írja le, ezért a fenti protokollt a kísérlet leírásában kell rögzíteni. Korlátok: három osztály, 24×24 méret és 360 kép; az eredmény nem általánosítható természetes képre, a triplet tanulás pedig nem garantál cluster-tisztaságot.

28.2.10 Tipikus tévedés és ellenpélda

A rossz negatívdefiníció hamis negatívokat hoz létre, és hibás reprezentációs nyomást ad. Legyen $z_a = (0;0)^\top$, $z_p = (1;0)^\top$, $z_n = (0;0{,}5)^\top$, $\alpha = 0{,}2$. A párkonstrukció hibásan n-t negatívnak címkézi, pedig x_n valójában az anchor osztályába tartozik. Ekkor $d_p^2 = 1$, $d_n^2 = 0{,}25$, a triplet loss $1 - 0{,}25 + 0{,}2 = 0{,}95 > 0$, és a gradiens $\nabla_{z_n} L = 2(z_a - z_n) = (0,-1)^\top$; a z_n-re alkalmazott gradienslépés a **negatív gradiens** irányába visz: $z_n \leftarrow z_n - \eta \nabla_{z_n} L$, azaz a pozitív második koordináta felé. $\eta = 0{,}1$ mellett $z_n' = (0;0{,}5) - 0{,}1 \cdot (0,-1) = (0;0{,}6)$, az új anchor–negatív távolságnégyzet $0{,}36 > 0{,}25$: a lépés *eltávolítja* a negatívként címkézett, valójában azonos osztályú pontot az anchortól, tehát növeli az azonos osztályú pontok távolságát. A hálózat tehát megtanulja szétválasztani azokat a pontokat, amelyeket a valódi szemantika szerint közelíteni kellene. A következmény nem csupán nagyobb veszteség: a downstream osztályozási vagy keresési pontosság is romolhat. A helyes gyakorlat a pár- és triplet-generálást a megbízható osztálycímkére, nem a jelenlegi beágyazási távolságra alapozni.

28.2.11 Önellenőrzés és visszatekintés

A margin dimenzióját mindig tisztázni kell: a squared triplet α és a fenti kontrasztív m egyaránt távolságnégyzet egységű. A $t = 0$ pont nem sima; a gradiensképletek csak $t > 0$ esetén érvényesek, ugyanúgy, mint a ReLU nullpontjánál. Visszatekintés a 28.1. Embedding, távolság és normalizálás alfejezetből: egységnormájú vektorokra $\|z_i - z_j\|^2 = 2(1 - \cos(z_i, z_j))$, ezért a triplet margin normalizált térben a pozitív feletti $\alpha/2$ koszinusz-előnyt ír elő. A 16.2. Pooling, downsampling és aliasing tanulsága szerint a pooling és downsampling a finom térbeli pozíciót eltüntetheti; a triplet feladathoz használt embeddingnél ezért ellenőrizni kell, hogy a választott kimenet (a szokásos gyakorlatban a pooling utáni, teljes összeköttetésű réteg kimenete, ahogy a FaceNet is használja) megőrzi-e az orientációhoz szükséges információt.

28.2.12 Rövid összefoglaló

A sziámi szerkezet két vagy több bemenetet ugyanazon a paramétervektoron futtat, ezért a triplet cél közös beágyazási teret kényszerít. A kontrasztív párveszteség a pozitív párokat közelíti, a negatívakat m margón túl nyomja. A squared triplet veszteség $[\|a-p\|^2 - \|a-n\|^2 + \alpha]_+$; a lemma szerint zérus akkor és csak akkor, ha $\|a-n\|^2 \geq \|a-p\|^2 + \alpha$. A $\alpha = 0$ ellenpélda megmutatja, hogy margin nélkül a konstans embedding nulla veszteségű collapse. A hard negative a margót sértő negatív; a rossz negatívdefiníció hamis negatívval hibás nyomást ad. A nyitó kérdésre a válasz: a tripletet a felügyelt címke jelöli ki (azonos osztály = pozitív), nem a pillanatnyi beágyazási távolság; téves negatív címke esetén a gradiens az azonos osztályú pontokat is szétválasztja. A labor a megosztott súly ágonkénti gradiensösszegzését numerikusan ellenőrzi.

28.2.13 Továbblépés

A 28.3. Retrieval-értékelés és adatfelosztás egységben a megtanult beágyazás retrieval-értékelését építjük fel: a galéria és query definícióját, a recall@k, mAP és csoportszintű aggregálás helyes használatát, valamint a [D15] forrásban ellenőrizendő averaging-protokollt.

28.2.14 Felhasznált források

- [P16] Schroff, Kalenichenko és Philbin (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering.* arXiv:1503.03832. https://arxiv.org/abs/1503.03832.
- [D15] scikit-learn developers (2026). *Model evaluation: quantifying the quality of predictions.* scikit-learn documentation. https://scikit-learn.org/stable/modules/model_evaluation.html. Hozzáférés: 2026-10-30.
- Bromley, J., Bentz, J. W., Bottou, L., Guyon, I., LeCun, Y., Moore, C., Säckinger, E. és Shah, R. (1993). *Signature Verification using a "Siamese" Time Delay Neural Network.* International Journal of Pattern Recognition and Artificial Intelligence, 7(4), 669–688.
- Hadsell, R., Chopra, S. és LeCun, Y. (2006). *Dimensionality Reduction by Learning an Invariant Mapping.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Hermans, A., Beyer, L. és Leibe, B. (2017). *In Defense of the Triplet Loss for Person Re-Identification.* arXiv:1703.07737. A sziámi eredet, a kontrasztív veszteség és a batch-hard/batch-all mining hivatkozásai a fejezet forráskatalógusán kívüliek, a szövegben használt, névadó közlemények.

28.3 Retrieval-értékelés és adatfelosztás

28.3.1 Cél, előismeret és induló kérdés

A 28.2. Sziámi és triplet cél alfejezetben felépített sziámi és triplet cél egy beágyazó (embedding) függvényt tanít: $f : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^d$. A tanítás után a modell minőségét nem osztályozási pontossággal, hanem **hasonlósági keresés (retrieval)** feladatban mérjük: egy lekérdezéshez megkeressük a hozzá tartozó, releváns elemet egy nagy adatbázisban. Ilyen beágyazást használ keresésre a FaceNet is [P16], ahol az arcazonosítás közvetlenül a beágyazási térbeli legközelebbi szomszédokon alapul. Ez a fejezet azt a kérdést járja körül, hogyan lehet ezt a kiértékelést korrektül elvégezni.

Az előismeret a 28.2. Sziámi és triplet cél alfejezetből a pozitív/negatív pár és a triplet margin fogalma; a 27.3. Dice, IoU és aggregálás alfejezetből a csoportos aggregálás tanulsága; a 24.3. Mixed precision és nagyobb batch alfejezetből a numerikus kontroll; a 16.3. Elméleti és effektív recepciós mező alfejezetből pedig a reprezentáció minőségének szerepe. Induló kérdés: ha 30 query-kép mindegyikéhez pontosan egy releváns gallery-kép tartozik, és a modell a releváns elemet átlagosan a 4. helyre rangsorolja, mekkora a Recall@1 és a Recall@3? A válaszhoz először pontos definíciók kellenek. **A kérdésre a helyes válasz: az átlagos rangból a Recall@1 és Recall@3 nem határozható meg**, mert nem tudjuk, hány query-nél van a releváns elem az 1. vagy az első 3 helyen (például az átlagos 4. hely származhat minden query 4. helyéből, vagy néhány 1. és sok nagyon hátsó helyből). A Recall@k-hoz a rangok eloszlása kell, nem az átlaguk.

28.3.2 Fogalmak és jelölések

Legyen $Q = \{q_1, \dots, q_{N_q}\}$ a **query (lekérdezés)** halmaz, $G = \{g_1, \dots, g_{N_g}\}$ a **gallery (keresési adatbázis)** halmaz. Mindkettő ugyanabból a képtérből származik, de a felhasználási szerepük eltér: a query az, amit a felhasználó bead, a gallery az, amiben keresünk. Az embeddingeket $X_q = f(Q) \in \mathbb{R}^{N_q \times d}$ és $X_g = f(G) \in \mathbb{R}^{N_g \times d}$ mátrixokba rendezzük; minden sor egy d-dimenziós vektor.

A **relevancia** egy bináris reláció: $rel(q, g) \in \{0, 1\}$. A függelék A. szakaszában leírt GlyphLab-példában a relevanciafeltétel a **scene_id egyezés**: két kép akkor és csak akkor releváns pár, ha ugyanabból a jelenetből származik. Az osztálycímke (vertical/horizontal/diagonal) itt nem relevancia, mert a retrieval cél a konkrét jelenet felismerése, nem az orientációé.

A **ranking (rangsor)** egy query q_i esetén a gallery elemeinek egy π_i permutációja, amelyre a hasonlósági pontszám csökkenő:

$$s(q_i, g_{\pi_i(1)}) \geq s(q_i, g_{\pi_i(2)}) \geq \cdots \geq s(q_i, g_{\pi_i(N_g)}).$$

A pontszám lehet koszinusz-hasonlóság, negatív euklideszi távolság vagy dot product. A $Top_k(i)$ halmaz az első k találat indexeit tartalmazza: $Top_k(i) = \{\pi_i(1), \dots, \pi_i(k)\}$.

A **Recall@k** (röviden $R@k$) két változatban használatos: a query-level változat a query-k azon hányadát méri, amelyeknél legalább egy releváns elem bekerül az első k találat közé; az item-level változat a releváns párok hányadát méri. A laborban a query-level változatot használjuk; a pontos definíciókat a következő szakasz adja. A **duplicate** olyan elem, amely tartalmilag azonos vagy közel azonos egy másikkal, például ugyanannak a képnek két másolata, vagy ugyanannak a jelenetnek két, alig eltérő view-ja. A **group split** olyan adatfelosztás, amelyben a független mintavételi egység (itt a scene_id) nem keveredik a részhalmazok között.

Az **ANN (Approximate Nearest Neighbor)** közelítő legközelebbi szomszéd keresést jelent. Nagy gallery (milliós nagyságrend) esetén a pontos brute-force összehasonlítás $O(N_g d)$ költsége túl nagy; az ANN indexek (például HNSW, Malkov és Yashunin, 2018; IVF-PQ, Jégou és mtsai, 2011) gyors, de közelítő találatot adnak. A kiértékelésnél ezért külön kell választani a reprezentáció minőségét és a keresési index közelítési hibáját.

28.3.3 Levezetés: a Recall@k nevezője és a relevanciafeltétel

A fejezet levezetési kötelezettsége a Recall@k pontos definíciója a GlyphLab példában. A kiinduló objektumok: Q és G halmazok, f embedding, s hasonlósági pontszám, valamint a relevancia reláció. A definícióhoz ki kell mondani a nevezőt és a találati feltételt.

Minden q_i query-hez rendeljük a releváns gallery indexek halmazát:

$$Rel(i) = \{j \in \{1, \dots, N_g\} : rel(q_i, g_j) = 1\}.$$

A GlyphLab esetén $rel(q_i, g_j) = 1$ pontosan akkor, ha $scene_id(q_i) = scene_id(g_j)$. Mivel a laborban minden jelenetből pontosan egy view kerül a query halmazba és pontosan egy másik view a gallery halmazba, a releváns halmaz minden query-re egyelemű: $|Rel(i)| = 1$ minden $i \in \{1, \dots, N_q\}$.

A **query-level Recall@k** definíciója:

$$R@k = \frac{1}{N_q} \sum_{i=1}^{N_q} \mathbb{1}\Big[Rel(i) \cap Top_k(i) \neq \emptyset\Big].$$

A nevező N_q, a query-k száma. Az indikátor értéke 1, ha legalább egy releváns elem az első k találat között van, különben 0. Ez a konvenció akkor természetes, ha egy query-hez kevés releváns elem tartozik, és a felhasználót az érdekli, hogy megtalálja-e valamelyiket.

Az **item-level Recall@k** alternatív definíció:

$$R_{item}@k = \frac{\sum_{i=1}^{N_q} |Rel(i) \cap Top_k(i)|}{\sum_{i=1}^{N_q} |Rel(i)|}.$$

Itt a nevező az összes releváns pár száma. A két definíció akkor esik egybe, ha minden query-re $|Rel(i)| = 1$: ekkor a query-level indikátor pontosan akkor 1, ha az item-level hányados számlálója 1, a nevező pedig N_q. A GlyphLab kiegyenlített felépítése miatt a laborban a query-level $R@k$ használjuk, de a fogalmi különbséget mindig jelezzük.

Állítás (monotonitás). *A query-level $R@k$ k-ban nem csökkenő: ha $k_1 \le k_2$, akkor $R@k_1 \le R@k_2$.*

Bizonyítás. Mivel $Top_{k_1}(i) \subseteq Top_{k_2}(i)$, ha $Rel(i) \cap Top_{k_1}(i) \neq \emptyset$, akkor $Rel(i) \cap Top_{k_2}(i) \neq \emptyset$ is fennáll. Ezért az indikátor értéke minden i-re nem csökken k növelésével, így az átlaguk sem csökken. □

Az állítás egyszerű, de fontos: az $R@k$ értékek önmagukban csak a k ismeretében értelmesek, és a nagyobb k felé monoton nőnek.

A Recall@k nem érzékeny arra, *hol* van a releváns elem az első k között. Ezt a rangsorérzékeny **átlagos pontosság** (AP) méri. Jelölje $\rho_i(j)$ a g_j gallery-elem rangját (1-től) a q_i rangsorában, és $P_i(r) = |Rel(i) \cap Top_r(i)|/r$ a precisiont az r-edik helyig.

Definíció (query AP, AP@k, mAP@k). Egy $|Rel(i)| \ge 1$ query **átlagos pontossága**

$$AP(i) = \frac{1}{|Rel(i)|} \sum_{j \in Rel(i)} P_i(\rho_i(j)),$$

a k-nál **csonkolt** változata

$$AP@k(i) = \frac{1}{\min(|Rel(i)|, k)} \sum_{j \in Rel(i):\, \rho_i(j) \le k} P_i(\rho_i(j)),$$

ahol a nevező $\min(|Rel(i)|, k)$, hogy a tökéletes rangsor értéke $k < |Rel(i)|$ esetén is 1 legyen. A $|Rel(i)| = 0$ query-re az AP nem értelmezett; ezeket kizárjuk az átlagból, és a számukat külön közöljük. A **mAP@k** a releváns elemmel rendelkező query-k $AP@k$-jának átlaga. A [D15] az osztályozási pontosság, a recall és az F1 általános definícióit adja; a query-szintű mAP@k itt használt alakját ez a szöveg definiálja.

A GlyphLabban $|Rel(i)| = 1$, ezért $AP@k(i) = 1/\rho_i$, ha a releváns elem rangja $\rho_i \le k$, különben 0: a 2. helyen talált pár 0,5-öt, az 5. helyen talált 0,2-t ér, míg a Recall@5 mindkettőt 1-nek számolja. A query/gallery irány megválasztása konvenció kérdése; a laborban mindkét irányt kiértékeljük (view 0 → view 1 és view 1 → view 0), külön közöljük, és az összesített érték a $2N_q$ query átlaga. A bizonytalanságot jelenetszintű bootstrap adja (Efron, 1979), amely egy jelenet mindkét irányú query-jét együtt húzza.

28.3.4 Kézi példa

Tekintsünk egy háromelemű példát, amelyben a releváns párok (q_1, g_1), (q_2, g_2), (q_3, g_3). A dimenzió legyen $d = 2$, a hasonlóság pedig a normalizálatlan dot product. Az embeddingek:

Vektor	Érték
q_1	$(2, 0)$
q_2	$(0, 2)$
q_3	$(1, 1)$
g_1	$(0{,}5; 1{,}7)$
g_2	$(1{,}7; 0{,}55)$
g_3	$(0{,}8; 0{,}9)$

A dot product mátrix $S_{ij} = q_i^\top g_j$:

$$S = \begin{bmatrix} 1{,}0 & 3{,}4 & 1{,}6 \\ 3{,}4 & 1{,}1 & 1{,}8 \\ 2{,}2 & 2{,}25 & 1{,}7 \end{bmatrix}.$$

A rangsorok csökkenő hasonlóság szerint: - q_1: g_2 (3,4), g_3 (1,6), g_1 (1,0); a releváns g_1 a 3. helyen van. - q_2: g_1 (3,4), g_3 (1,8), g_2 (1,1); a releváns g_2 a 3. helyen van. - q_3: g_2 (2,25), g_1 (2,2), g_3 (1,7); a releváns g_3 a 3. helyen van.

A találati indikátorok: $R@1 = 0$, mert egyik releváns elem sincs az első helyen; $R@2 = 0$; $R@3 = 3/3 = 1$. Az AP-alapú mérték ugyanerről: minden query-re $\rho_i = 3$, így $AP@2(i) = 0$, $AP@5(i) = 1/3$, tehát mAP@2 = 0 és mAP@5 = 1/3, miközben $R@5 = 1$. Ez a példa azt szemlélteti, hogy az $R@1 = 0$ önmagában nem jelenti azt, hogy a reprezentáció használhatatlan; a releváns elemek mind benne vannak a rangsorban, csak nem elöl. Ez a háromelemű, teljes galériában mindig így van: $R@3 = 1$ minden rangsorra triviális, tehát nem a reprezentáció érdeme (a példa minden query-n az utolsó helyre sorolja a releváns elemet). Az $R@1$ és az AP az informatív mérték; az $R@k$ értelmét csak $k < G$ (G a galéria mérete) mellett olvassuk ki; a $k \geq G$ esetet külön kezeljük (az $R@k = 1$ triviális). Egyetlen releváns elemnél a véletlen rangsor baseline-ja szerint a várható hit@$k = \min(k, G)/G$. Nem triviális példa: $G = 100$, $k = 5$ esetén a véletlen baseline 0,05; egy 0,60-as hit@5 ennél lényegesen jobb.

28.3.5 Labor: csoportszintű split és visszakeresési riport

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor a GlyphLab adatot használja. A cél egy olyan query/gallery split elkészítése, amely tiszteletben tartja a scene_id mint független mintavételi egységet, majd egy egyszerű visszakeresési riport előállítása. A labor nem tanít modellt; egy rögzített, nem tanult feature-t használunk (a nyers pixel vektor L2-normalizálva), hogy a kiértékelési protokollt izoláljuk a modellválasztástól. Ez a baseline reprezentációtanulás szempontjából gyenge (nyers pixelek), a tiszta párokon mért értéke mégis 1,0000, mert a két nézet szinte azonos; a split és a riport szerkezete ettől függetlenül korrekt.

Input: a `lab/data/images.npy` (360,1,24,24) float32 tömb, a `lab/data/metadata.csv`. A labor a `metadata.csv` következő oszlopait használja: `index` (a kép sorszáma az `images.npy`-ban), `split` (`train`/`val`/`test`), `view_id` (`0`/`1`) és `scene_id`. A munkakönyvtár a `lab/` gyökér, ahol a `data/` mappa található. A split a `test` címkéjű sorokra szűkül: 30 jelenet, 60 kép. A query halmaz a `view_id == 0`, a gallery halmaz a `view_id == 1` sorokból áll. A relevancia a scene_id egyezés. A rendezés scene_id szerint garantálja, hogy a releváns pár a főátlón legyen.

A kód NumPy-t és a Python szabványos `csv` modulját használja; nincs GPU, nincs véletlen szám, így seed nem szükséges. A dtype float32.

```
# lab/retrieval_report.py - futtatás: cd lab && python retrieval_report.py

import csv
import numpy as np

DATA = "data"
K_VALUES = [1, 3, 5, 10]

def load_metadata(path):
    rows = []
    with open(path, newline="") as f:
        for r in csv.DictReader(f):
            rows.append(r)
    return rows

def main():
    # 1. Adat betöltése és test-szűrés
    rows = load_metadata(f"{DATA}/metadata.csv")
    test_rows = [r for r in rows if r["split"] == "test"]

    # 2. Csoportszintű query/gallery split: view_id rögzített
    query_rows = sorted(
        [r for r in test_rows if r["view_id"] == "0"],
        key=lambda r: r["scene_id"]          # a scene_id szöveg, pl. "G0-0003"
    )
    gallery_rows = sorted(
        [r for r in test_rows if r["view_id"] == "1"],
        key=lambda r: r["scene_id"]          # a scene_id szöveg, pl. "G0-0003"
    )

    # 3. Ellenőrzés: ugyanazok a scene_id-k, mindkét halmazban 30 elem
    q_scenes = {r["scene_id"] for r in query_rows}
    g_scenes = {r["scene_id"] for r in gallery_rows}
    assert q_scenes == g_scenes, "A query és gallery scene_id halmaza eltér!"
    assert len(query_rows) == 30 and len(gallery_rows) == 30

    # 4. Képek betöltése index szerint, float32
    images = np.load(f"{DATA}/images.npy").astype(np.float32)  # (360,1,24,24)
    q_idx = np.array([int(r["index"]) for r in query_rows])
    g_idx = np.array([int(r["index"]) for r in gallery_rows])
    assert len(set(q_idx)) == 30 and len(set(g_idx)) == 30, "Ismétlődő index a splitben!"
    assert ([r["scene_id"] for r in query_rows]
            == [r["scene_id"] for r in gallery_rows]), "A query és gallery sorai nem
             ↪ scene-párosítottak!"
    Q = images[q_idx].reshape(len(q_idx), -1)  # (30, 576)
    G = images[g_idx].reshape(len(g_idx), -1)  # (30, 576)

    # 5. Nyers pixel feature: L2-normalizálás
    Qf = Q / np.linalg.norm(Q, axis=1, keepdims=True)
    Gf = G / np.linalg.norm(G, axis=1, keepdims=True)

    # 6. Hasonlóság: koszinusz (dot product normalizált vektorokon)
```

```
    S = Qf @ Gf.T  # (30, 30)

    # 7. Relevancia: a rendezés miatt a főátló; 1-től számozott rang, mindkét irányban
    def relevant_rank(sim):
        order = np.argsort(-sim, axis=1, kind="stable")
        return np.array([int(np.where(order[i] == i)[0][0]) + 1 for i in range(len(sim))])
    ranks = {"view0->view1": relevant_rank(S), "view1->view0": relevant_rank(S.T)}

    def ap_at_k(rho, k):                        # |Rel| = 1: AP@k = 1/rang, ha rang <= k
        return np.where(rho <= k, 1.0 / rho, 0.0)

    # 8. Visszakeresési riport
    print("GlyphLab test retrieval riport (nyers pixel baseline)")
    print(f"Query: {len(q_idx)}, Gallery: {len(g_idx)}")
    for name, rho in ranks.items():
        print(name, " ".join(f"R@{k}={np.mean(rho <= k):.4f}" for k in K_VALUES),
              f"mAP@5={ap_at_k(rho, 5).mean():.4f}")
    both = np.concatenate(list(ranks.values()))
    print("mindkét irány:", " ".join(f"R@{k}={np.mean(both <= k):.4f}" for k in K_VALUES),
          f"mAP@5={ap_at_k(both, 5).mean():.4f}")

if __name__ == "__main__":
    main()
```

A megadott környezetben mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, GlyphLab seed 20261020; a riport determinisztikus, ezért seed nem kell):

```
GlyphLab test retrieval riport (nyers pixel baseline)
Query: 30, Gallery: 30
view0->view1 R@1=1.0000 R@3=1.0000 R@5=1.0000 R@10=1.0000 mAP@5=1.0000
view1->view0 R@1=1.0000 R@3=1.0000 R@5=1.0000 R@10=1.0000 mAP@5=1.0000
mindkét irány: R@1=1.0000 R@3=1.0000 R@5=1.0000 R@10=1.0000 mAP@5=1.0000
```

Az eredmény elsőre meglepő, de az adat szerkezetéből következik: egy jelenet két nézete ugyanazt a vonalat ugyanott, ugyanakkora vastagsággal tartalmazza, csak a zaj más, ezért már a nyers pixelvektorok koszinusz-hasonlósága is a saját párt teszi az első helyre. Ez a feladat így **nem** méri a tanult reprezentáció minőségét; csak a split, a relevanciafeltétel és a riport helyességét ellenőrzi (a releváns elem a főátlón van, a Recall@k monoton). Erősebb torzítás (például a korrupt tesztképek használata) esetén a nyers pixel baseline pontossága csökkenne, de ezt a jelen labor nem méri; ilyen kísérlethez a korrupt adatot és a mérési protokollt külön kell rögzíteni. Egy tanult embedding értékeléséhez ezért nehezebb relevanciafeltétel kell, például azonos osztályú, de más jelenetből származó képek visszakeresése, vagy erősebb torzítás; ezt a laborfeladatok tervezik meg.

28.3.6 Gyakori tévedés és ellenpélda

A leggyakoribb hiba a query/gallery split helytelen elkészítése: a view_id-k figyelembevétele nélküli véletlen keverés, vagy a relevancia gyenge címkéhez kötése a scene_id helyett. Ha például a test halmaz 60 képét véletlenszerűen kettéosztjuk 30 query-re és 30 gallery-re, akkor előfordulhat, hogy ugyanannak a jelenetnek mindkét view-ja a query halmazba kerül, és a gallery-ben nincs releváns párja. Ennél alattomosabb eset, ha a query és gallery ugyanazt a view-ot tartalmazza (például mindkét halmazban a view0 szerepel). Ekkor a nyers pixel baseline koszinusz-hasonlósága 1,0, így a Recall@1 triviálisan 1,0 lesz, függetlenül a modell minőségétől. Az ilyen **közel duplikált (duplicate) nézet** átengedése túl optimista retrieval eredményt ad.

Ellenpéldaként vegyünk egy olyan splitet, ahol a query és a gallery is a view0 képekből áll (különböző scene-ekből), és a relevanciafeltétel tévesen az osztálycímkére épül. Ekkor minden

query-hez több releváns gallery-elem tartozik, az item-level és a query-level Recall@k eltér, és a feladat az orientáció felismerésére redukálódik: a modellnek nem kell a jelenet azonosítását megtanulnia. Az így kapott Recall@k inkább egy osztályozási jellegű feladatot mér, nem a konkrét jelenet keresését; a scene_id-alapú relevancia mellett ugyanez a split értelmetlen volna, mert a view0–view0 párok nem azonos jelenetből valók. A helyes protokoll ezért: **a query minden jelenetből az egyik view, a gallery a másik view, és a relevancia a scene_id egyezés**.

28.3.7 Önellenőrzés és visszautalás

A fejezet állításai ellenőrizhetők: (1) a $R@k$ képlet nevezője N_q, és a találati feltétel explicit; (2) a monotonitási állítás bizonyítása a Top_k halmaz bővülésén alapul; (3) a labor kódja asserttel ellenőrzi a scene_id-halmazok egyezését, a 30-as elemszámot, az indexek egyediségét és a soronkénti scene-párosítást; (4) a gyakori tévedés ellenpéldája bemutatja a view_id keverés következményét.

Visszautalás 27.3. Dice, IoU és aggregálás alfejezetre: ahogyan a per-image és a globális IoU/Dice aggregálás más súlyozást ad, itt a query-level és item-level Recall@k is eltérhet, ha a releváns elemek száma nem egyenletes. Visszautalás 24.3. Mixed precision és nagyobb batch alfejezetre: az ANN index közelítési hibáját külön kell mérni és jelenteni, a pontos brute-force rankingot véve referenciának. A kétféle hiba forrása azonban eltér: a mixed precision hibája a lebegőpontos számítás pontosságából, az ANN indexé a szomszédkeresés közelítéséből származik, ezért a kettőt nem ugyanazzal a mérőszámmal kell jellemezni. Visszautalás 16.3. Elméleti és effektív recepciós mező alfejezetre: a nyers pixel feature nem lát szemantikus kontextust, ezért a receptív mező és a tanult reprezentáció minősége döntő a retrieval sikeréhez.

28.3.8 Feladatok

F1 (fogalmi, nehézség 1). Definiáld a query, gallery és relevancia szerepét a GlyphLab retrieval feladatban. Miért nem az osztálycímke (vertical/horizontal/diagonal) a relevanciafeltétel? Adj egy konkrét példát, amelyben ugyanaz az osztálycímke két különböző jelenethez tartozik.

F2 (levezetési, nehézség 2). Adott egy 4 query-ből és 5 gallery-ből álló kiértékelés. A relevancia mátrix sorai:

$$Rel = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

A ranking mátrix sorai (a gallery indexek csökkenő hasonlóság szerint):

$$\pi = \begin{bmatrix} 2 & 0 & 1 & 3 & 4 \\ 1 & 4 & 0 & 2 & 3 \\ 3 & 2 & 0 & 1 & 4 \\ 0 & 1 & 2 & 3 & 4 \end{bmatrix}.$$

Figyelem: ez a gyakorló példa szándékosan eltér a GlyphLab $|Rel(i)| = 1$ feltevésétől: q_2-nél két releváns elem van, q_4-nél nulla, hogy a query-level definíció viselkedését ezekben az esetekben is lássuk. Számítsd ki a query-level $R@1$, $R@2$ és $R@3$ értékeket. Indokold minden lépésben a találati feltételt.

F3 (implementációs, nehézség 2). Írj egy `recall_at_k(rank_matrix, rel_matrix, k)` függvényt, amely a fenti query-level definíciót valósítja meg. Az input `rank_matrix` (N_q, N_g)

egész, `rel_matrix` (N_q, N_g) 0/1, `k` pozitív egész. Ellenőrizd a függvényt az F2 adatain, egy üres relevancia mátrixon (0 eredmény), és egy olyan mátrixon, ahol minden releváns elem az első helyen van (1 eredmény).

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a test halmazból készít query/gallery splitet. Mi a hiba, mi a következménye, és hogyan javítanád? Ellenőrzésként mutasd meg, hogy a javított verzió garantálja a releváns pár jelenlétét.

Az alábbi részlet feltételezi, hogy a `metadata.csv`-ből pandas DataFrame készült (`meta`), és hogy a test split 60 soros; a labor közvetlenül csv-t olvas, ezért ez a változat csak a hiba szemléltetésére szolgál. A `random_state=42` csak a hibás keverést teszi reprodukálhatóvá; a laborban nincs szükség seedre, mert ott nincs véletlen lépés.

```
# Hibás split: véletlenszerű keverés scene_id és view_id nélkül
test_meta = meta[meta.split == "test"].sample(frac=1, random_state=42)
query_meta = test_meta.iloc[:30]
gallery_meta = test_meta.iloc[30:]
```

F5 (kísérleti/transzfer, nehézség 3). Tegyük fel, hogy a gallery mérete 1 millió kép, és a pontos brute-force ranking túl lassú. Tervezz kísérletet, amely egy ANN index (például HNSW) bevezetése előtt és után méri a Recall@k-t, és elkülöníti az index közelítési hibáját a reprezentáció minőségétől. Add meg a protokollt, a baseline-t, a metrikát és az érvényességi korlátot.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

28.3.9 Megoldások és indoklás

F1 megoldása. A query a felhasználó által beadott keresési kép, a gallery a keresési adatbázis. A relevancia azt mondja meg, hogy egy gallery elem a query helyes találata-e. A GlyphLab példában a relevanciafeltétel a scene_id egyezés: minden jelenetből két view készült, és a query–gallery pár akkor releváns, ha ugyanabból a jelenetből származik. Az osztálycímke nem relevancia, mert például két különböző vertical jelenet (két különböző függőleges alakzat, más pozícióban, más vastagsággal) azonos osztályú, de a retrieval feladatban nem felcserélhetők: a felhasználó a konkrét jelenet másik nézetét keresi. Az osztálycímke alapú relevancia egy könnyebb, osztályozási feladatot mérne.

F2 megoldása. A query-level definíció szerint minden query-re azt nézzük, hogy legalább egy releváns elem az első k találat között van-e. A ranking mátrix és a relevancia mátrix alapján:

- q_1: a ranking $[2, 0, 1, 3, 4]$; a releváns indexek $\{1\}$. Az első 1 találat $\{2\}$ nem tartalmazza; az első 2 találat $\{2, 0\}$ nem tartalmazza; az első 3 találat $\{2, 0, 1\}$ tartalmazza. Tehát q_1 találat $k = 3$-ra: igen.
- q_2: ranking $[1, 4, 0, 2, 3]$; releváns $\{0, 2\}$. $k = 1$: $\{1\}$ nem; $k = 2$: $\{1, 4\}$ nem; $k = 3$: $\{1, 4, 0\}$ tartalmazza a 0-t. Találat $k = 3$-ra: igen.
- q_3: ranking $[3, 2, 0, 1, 4]$; releváns $\{3\}$. $k = 1$: $\{3\}$ tartalmazza. Találat minden $k \geq 1$-re: igen.
- q_4: ranking $[0, 1, 2, 3, 4]$; releváns halmaz üres. Nincs találat semmilyen k-ra.

Összegzés: $R@1 = (0 + 0 + 1 + 0)/4 = 0{,}25$; $R@2 = (0 + 0 + 1 + 0)/4 = 0{,}25$; $R@3 = (1 + 1 + 1 + 0)/4 = 0{,}75$. Az $R@2 = 0{,}25$ azért egyezik meg az $R@1$-gyel, mert q_1-nél a releváns elem csak a 3. helyen jelenik meg, q_2-nél pedig az első két találat egyike sem releváns; q_3 már $k = 1$-re talál, q_4-nek nincs releváns eleme.

F3 megoldása. Az alábbi függvény a query-level definíciót valósítja meg.

```
import numpy as np

def recall_at_k(rank_matrix, rel_matrix, k):
    """Query-level Recall@k.
    rank_matrix: (Nq, Ng) egész (int64), minden sor a gallery indexek csökkenő sorrendben.
    rel_matrix: (Nq, Ng) 0/1, relevancia mátrix.
    """
    assert 1 <= k <= rank_matrix.shape[1], "k a [1, Ng] tartományon kívül van"
    Nq = rank_matrix.shape[0]
    hits = 0
    for i in range(Nq):
        top_k = rank_matrix[i, :k]
        if np.any(rel_matrix[i, top_k]):
            hits += 1
    return hits / Nq

# Teszt az F2 adataival
rank = np.array([[2,0,1,3,4],
                 [1,4,0,2,3],
                 [3,2,0,1,4],
                 [0,1,2,3,4]], dtype=np.int64)
rel = np.array([[0,1,0,0,0],
                [1,0,1,0,0],
                [0,0,0,1,0],
                [0,0,0,0,0]], dtype=np.int64)
assert recall_at_k(rank, rel, 1) == 0.25
assert recall_at_k(rank, rel, 2) == 0.25
assert recall_at_k(rank, rel, 3) == 0.75
# Üres relevancia: 0
assert recall_at_k(rank, np.zeros_like(rel), 3) == 0.0
# Minden releváns elem az első helyen: 1
rel_perfect = np.zeros_like(rel)
rel_perfect[np.arange(rank.shape[0]), rank[:, 0]] = 1
assert recall_at_k(rank, rel_perfect, 1) == 1.0
print("Minden ellenőrzés sikeres.")
```

A függvény alakellenőrzése: a `rank_matrix` soronkénti permutáció, a `rel_matrix` bináris. A kód a k értékét asserttel ellenőrzi ($1 \le k \le N_g$); a `rank_matrix` és a `rel_matrix` dimenzióegyezését a feladat adatai biztosítják, éles kódban erre is érdemes assertet tenni.

F4 megoldása. A hiba oka, hogy a `sample(frac=1)` véletlenszerűen keveri a sorokat, és az első 30 sor query-nek, a többi gallery-nek való kijelölése nem veszi figyelembe sem a `scene_id`-t, sem a `view_id`-t. Következmény: ugyanannak a jelenetnek mindkét view-ja kerülhet ugyanabba a halmazba, így egy query-nek nem lesz releváns párja a gallery-ben, vagy éppen ugyanaz a view kerülhet query és gallery oldalra. Ráadásul az `iloc[:30]`/`iloc[30:]` kód 60 test sort feltételez anélkül, hogy ezt ellenőrizné. A relevancia mátrix nem garantáltan főátló, és a kiértékelés nem méri a modell valódi hasonlósági teljesítményét.

A javítás: a query halmaz legyen a `view_id == 0` sorok halmaza, a gallery a `view_id == 1` sorok halmaza, mindkettő a `test` splitből, és mindkettőt rendezzük `scene_id` szerint. Ez — feltéve, hogy minden jelenethez pontosan egy view0 és egy view1 tartozik — garantálja, hogy minden query-hez pontosan egy releváns gallery elem tartozik, és a releváns pár a főátlón van. A javított kód megegyezik a labor kódjának 2. lépésével. Ellenőrzés: `set(query_scene_ids) == set(gallery_scene_ids)`, mindkét halmaz mérete 30, az indexek egyediek, és a rendezés után a két lista scene_id szerint soronként párosított.

F5 megoldása (megoldásminta). A kísérlet célja annak kimutatása, hogy az ANN index bevezetése mennyivel rontja a Recall@k-t a pontos brute-force rankinghez képest. Protokoll:

1. **Adat és split:** Rögzített query halmaz ($N_q = 500$) és gallery ($N_g = 1\,000\,000$). A relevancia a `scene_id` egyezése: $rel(q, g) = 1$ pontosan akkor, ha q és g `scene_id`-je azonos; ez a kapcsolat közvetlenül a metaadatokból számítódik, nem külön emberi címkézésből. A validációs query részhalmaz (például 100 query) diszjunkt a teszt query halmaztól; a két részhalmaz ugyanahhoz a rögzített galleryhez tartozik, és a hiperparaméter-választás csak a validációs részhalmazon történik.
2. **Baseline:** Pontos brute-force koszinusz-hasonlóság, amely a referencia $R@k$-t adja. A teljes $500 \times 10^6 = 5 \cdot 10^8$ elemű hasonlósági mátrix float32-ben 4 bájt/elem mellett $2 \cdot 10^9$ bájt, azaz mintegy 2 GB, ezért a brute-force referenciát blokkokban vagy részhalmazon érdemes számítani.
3. **Változó tényező:** ANN index típusa és hiperparaméterei (pl. HNSW `ef_search`, IVF-PQ kódkönyvek száma). Az indexet a gallery embeddingeken építjük.
4. **Metrika:** Query-level $R@1$ és $R@10$, valamint az index által visszaadott 10 elem és a pontos top-10 lista metszetének átlagos mérete osztva 10-zel (overlap@10). Képlettel $\text{overlap@10} = \frac{1}{N_q} \sum_{q=1}^{N_q} \frac{|\text{ANN}_{10}(q) \cap \text{Exact}_{10}(q)|}{10}$, ahol $\text{ANN}_{10}(q)$ és $\text{Exact}_{10}(q)$ a közelítő, illetve pontos kereső első tíz találatának indexhalmaza.
5. **Kiértékelés:** A pontos brute-force rankinget a teljes query halmazon kiszámítjuk egyszer. Az ANN indexet különböző hiperparaméterekkel validációs query részhalmazon hangoljuk; a végső konfigurációt a validációs $R@10$-re választjuk. A teszt query halmazon a kiválasztott indexszel mérjük az $R@k$-t és az overlapet.
6. **Következtetési határ:** Az ANN $R@k$ eltérése a pontos $R@k$-tól az index közelítési hibája, nem a reprezentáció minősége. A jelentésben a kettőt külön oszlopban kell közölni. Az érvényességi korlát: az eredmény csak az adott indexre és adatméretre érvényes; más index vagy gallery méret esetén a mérést meg kell ismételni.

28.3.10 Rövid összefoglaló

A hasonlósági keresés kiértékelése a query/gallery split, a relevanciafeltétel és a ranking pontos definícióján alapul. A query-level Recall@k nevezője a query-k száma, találati feltétele, hogy legalább egy releváns elem az első k találat között legyen; a GlyphLab példában a relevancia a scene_id egyezés. A kézi példa megmutatta, hogy az R@1=0 mellett az R@3=1 is előfordulhat. A labor csoportszintű splitet készített view0/view1 alapon, és nyers pixel baseline riportot adott. A leggyakoribb hiba a query/gallery split helytelen elkészítése, például a view_id-k keverése, amely közel duplikált nézetekkel irreálisan magas recallt okoz. Nagy gallery esetén az ANN index közelítési hibáját külön kell mérni a reprezentáció minőségétől.

28.3.11 Következő alkalmazás és források

A következő alkalmazás 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés, a VII. ellenőrző pont: a CNN alkalmazási szerződés számonkéri a retrieval split, a metrika és az ANN kontroll együttes használatát egy teljes alkalmazási döntésben.

Felhasznált források:

- [P16] Schroff, F., Kalenichenko, D., Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering.* arXiv:1503.03832. https://arxiv.org/abs/1503.03832.
- [D15] scikit-learn developers (2026). *Model evaluation: quantifying the quality of predictions.* scikit-learn documentation. https://scikit-learn.org/stable/modules/model_evaluation.html. Hozzáférés: 2026-10-30.

- Malkov, Y. A. és Yashunin, D. A. (2018). *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836.
- Jégou, H., Douze, M. és Schmid, C. (2011). *Product Quantization for Nearest Neighbor Search*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1), 117–128.
- Efron, B. (1979). *Bootstrap Methods: Another Look at the Jackknife*. The Annals of Statistics, 7(1), 1–26. A HNSW-, IVF-PQ- és bootstrap-hivatkozások a fejezet forráskatalógusán kívüliek, a szövegben használt módszertani források.

28.4 VII. ellenőrző pont: CNN alkalmazási szerződés

28.4.1 1. Cél, előismeret és induló kérdés

A VII. ellenőrző pont a 28.3. Retrieval-értékelés és adatfelosztás alfejezetben bevezetett retrieval-szerződést a korábbi három CNN-feladatcsalád mellé állítja: az osztályozás, a detektálás és a szegmentálás mellé. A cél annak bizonyítása, hogy a hallgató nem csupán az egyes metrikákat, hanem a **cél–target–loss–output–metrika–split–inference szerződés** teljes láncát külön tudja választani, és új feladatra önálló protokollt tud készíteni. Előismeret: a 27.4. Instance mask és összetett feladat instance/szemantikus megkülönböztetése, a 24.4. VI. ellenőrző pont: megbízható CNN-training tanítási és checkpoint-szerződése, valamint a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése CNN-operátor specifikációja. A retrieval oldalról a 28.3. Retrieval-értékelés és adatfelosztás query/gallery, ranking és Recall@k fogalmaira építünk.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg; itt csak az eltérés: a riport a cél–target–loss–output–metrika–split–inference szerződés; a súlypont a D3 (adat- és mérési protokoll).

Induló kérdés: egy CNN, amely 92% osztályozási pontosságot ér el, automatikusan alkalmas-e detektálásra vagy hasonlósági keresésre? Becslés: nem. Ugyanaz a gerinc (backbone) különböző fejekkel egészen eltérő kimenetet ad, és a 92% accuracy-t nem lehet egy sorba állítani egy IoU-val vagy Recall@k-val, mert a három szám más célfüggvényből, más targetből és más veszteségből születik.

28.4.2 2. A négy feladatcsalád cél- és alakszerződése

Jelölje $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ a bemeneti képbatch-et. A gerinc kimenete $F(X) \in \mathbb{R}^{N \times D \times H' \times W'}$ feature-tensor. Minden feladatcsalád ebből eltérő fejjel készít előrejelzést.

Feladat	Target	Nyers output	Elsődleges metrika
Osztályozás	$y \in \{0, \dots, K-1\}^N$	$Z \in \mathbb{R}^{N \times K}$ logits	accuracy, macro-F1
Detektálás	változó számú xyxy doboz + osztály	listaelemek (b, c, s)	AP @ IoU-küszöb
Szegmentálás	$Y \in \{0, \dots, K-1\}^{N \times H \times W}$	pixel-logits $\mathbb{R}^{N \times K \times H \times W}$	mIoU, Dice
Retrieval	relevancia-reláció $R(q, g)$	embedding $\mathbb{R}^{N \times D}$	Recall@k, mAP

A fenti táblázat a feladatot, a targetet, az outputot és a metrikát mutatja; a metrikanevek általános definíciója a [D15] forrásban ellenőrizhető, a query-szintű Recall@k és mAP alakját pedig a 28.3. Retrieval-értékelés és adatfelosztás egység rögzíti. A loss, a split és az inference szerződés minden feladatnál különbözik, ezért külön definiáljuk.

Osztályozás. A target egyetlen osztályindex képenként, az output aktiváció előtti logits-vektor. A loss a többosztályos cross-entropy átlagoló (mean) redukcióval:

$$L = -\frac{1}{N}\sum_{n=1}^{N} \log \frac{\exp(Z_{n,y_n})}{\sum_{k=1}^{K} \exp(Z_{n,k})}.$$

A split osztály-stratified és scene-csoportos; a függelék A. szakaszában leírt GlyphLab-szerződésben a független egység a `scene_id`. Az inference szerződés: a modell a képtensoron kívül semmilyen annotációt nem kap, és a testkészlet a kiválasztás után egyetlen egyszer kerül kiértékelésre.

Detektálás. A target változó hosszúságú rendezett lista képenként; egy elem (ℓ, t, r, b, c) nullától indexelt, fél-nyílt xyxy doboz és osztályindex. A loss általában dobozregresszió (például smooth L1) plusz objectness binary cross-entropy; a pontos súlyozás architektúránként eltér. A split scene-szintű, és egyetlen dobozt sem vág szét. A metrika átlagos precision (AP) egy rögzített IoU-küszöbnél, greedy egy-az-egyhez párosítással, ahogy a 26.3. NMS, confidence és AP alfejezetben definiáltuk. Az inference szerződés magában foglalja a non-maximum suppressiont (NMS), a score-küszöböt és a maximális kimeneti dobozszámot; ezek nélkül a nyers output nem értékelhető.

Szegmentálás. A target minden pixelre osztályindex, tehát az alak $N \times H \times W$ egész tensor. A modell pixel-logitsot ad $N \times K \times H \times W$ alakban; a loss per-pixel cross-entropy, a $H \times W$ térbeli pozíciók átlagával. A metrika mIoU vagy Dice; az üres-unió esetekhez külön szabály kell a 27.3. Dice, IoU és aggregálás szerint. A split scene-csoportos, és a maszkot a képpel geometriailag együtt kell transzformálni; az inference argmax vagy rögzített küszöb.

Retrieval. A target nem egyetlen címke, hanem egy relevancia-reláció: minden query-hez a gallery azon elemei relevánsak, amelyek a megfigyelési egység (például scene_id) szerint ugyanahhoz a csoporthoz tartoznak, de nem azonos a query-vel és nem duplikátum. A modell outputja D-dimenziós embedding vektor: $f : \mathbb{R}^{C_{in} \times H \times W} \to \mathbb{R}^D$. A loss lehet triplet loss: a [P16] módszerszakasza a triplet loss-t L2-normalizált embeddingen, négyzetes L2-távolsággal definiálja, ahol a pozitív pár négyzetes távolsága legalább α margóval kisebb a negatív párénál. A gyakorlatban ezt a következő, négyzetes alakban szokás felírni:

$$L = \frac{1}{N}\sum_{n=1}^{N} \max\Big(0,\ \|f(q_n) - f(g_n^+)\|^2 - \|f(q_n) - f(g_n^-)\|^2 + \alpha\Big),$$

ahol $\alpha > 0$ a margin. A train/val/test scene-halmazok diszjunktak. Egy retrievalen belül (például a teszten) a relevancia a scene-azonosságon alapul, ezért a query és releváns gallery-eleme *ugyanabból* a teszt-scene-ből, de eltérő view-ból származik (az egyik view a query, a másik a gallery-ben van); különböző scene-ek query/gallery szétválasztása mellett nem lenne releváns elem. Ugyanazon scene két view-ja közül az egyik a query, a másik a gallery releváns eleme; egyik view sem lehet egyszerre query és gallery elem (azaz ugyanaz a kép nem szerepelhet mindkét oldalon). Más relevanciafogalomnál azt külön definiálni kell. A metrika a 28.3. Retrieval-értékelés és adatfelosztás definíciójával:

Ebben az alfejezetben a **Recall@k** a releváns elemek visszahívási hányadát jelenti:

$$\text{Recall@}k = \frac{|\{\text{releváns gallery elemek}\} \cap \{\text{top-}k \text{ lista}\}|}{|\{\text{releváns gallery elemek}\}|}.$$

A 28.3. alfejezet querynkénti *legalább-egy-találat* metrikáját (ott $R@k$) itt **Hit@**k-nak nevezzük: $\text{Hit@}k = \mathbb{1}[\{\text{releváns}\} \cap \{\text{top-}k\} \neq \emptyset]$, átlagolva a querykre. Több releváns elemnél a kettő eltérő metrika; egyetlen releváns elemnél egybeesnek. A feladat, a kód és a riport ugyanazt mérje, és a nevét is ehhez igazítsa. Recall@k nevezője a query összes releváns gallery elemének száma, nem pedig k; ez a leggyakoribb hibaforrás.

A loss és a metrika különbsége minden feladatnál éles: a loss differenciálható simuló cél, a metrika lehet nem sima és nem differenciálható (például az IoU küszöbölése), ezért a tanítás során a loss-t, a kiértékelés során a metrikát használjuk.

28.4.3 3. Kézi matematikai kontrollok minden feladathoz

Minden választott feladathoz egy alakellenőrzést és egy kézi numerikus kontrollt adunk. A cél nem a teljes modell kiértékelése, hanem a képletek index- és dimenzióhelyességének bizonyítása.

28.4.3.1 Osztályozás: softmax-összeg és cross-entropy

Legyen $N = 1$, $K = 3$, logits $z = [2,\ -1,\ 0{,}5]$, target $y = 0$. Alakellenőrzés: $Z \in \mathbb{R}^{1\times 3}$, a softmax kimenete is 1×3, a loss skalár.

A softmax nevezője: $e^2 = 7{,}3891$, $e^{-1} = 0{,}3679$, $e^{0,5} = 1{,}6487$, összeg $9{,}4057$. A valószínűségek: $p_0 = 0{,}7856$, $p_1 = 0{,}0391$, $p_2 = 0{,}1753$; a kerekített értékek összege $1{,}0000$ (a softmax definíció szerint pontosan 1). A cross-entropy: $-\log(0{,}7856) = 0{,}2413$. Ez a kézi kontroll mutatja, hogy a softmax kimenete nem metrika, csak a logitsból képzett valószínűségi vektor.

28.4.3.2 Detektálás: fél-nyílt dobozok IoU-ja

Legyen a ground truth doboz $[1, 2, 5, 6]$, a predikció pedig $[3, 4, 7, 8]$. A fél-nyílt intervallum szerint a terület: $(5-1)\cdot(6-2) = 16$, illetve $(7-3)\cdot(8-4) = 16$. A metszet:

$$x_{\min} = \max(1,3) = 3, \quad x_{\max} = \min(5,7) = 5, \quad y_{\min} = \max(2,4) = 4, \quad y_{\max} = \min(6,8) = 6.$$

A metszet szélessége $5-3 = 2$, magassága $6-4 = 2$, területe 4. Az unió $16 + 16 - 4 = 28$, tehát

$$\text{IoU} = \frac{4}{28} = \frac{1}{7} \approx 0{,}1429.$$

Alakellenőrzés: minden doboz négy koordináta, a metszet és az unió pozitív skalár. A fél-nyílt konvenció biztosítja, hogy a szomszédos dobozok területe pontosan a koordinátakülönbségek szorzata.

28.4.3.3 Szegmentálás: bináris maszk Dice és IoU

Legyen 2×2-es ground truth maszk $\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$ és predikció $\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$. Flattenelve: $|A| = 2$, $|B| = 2$, metszet $|A \cap B| = 1$, unió $= 3$. Ekkor

$$\text{IoU} = \frac{1}{3}, \qquad \text{Dice} = \frac{2 \cdot 1}{2 + 2} = 0{,}5.$$

Alakellenőrzés: mindkét maszk $N \times H \times W$; a redukció a térbeli pozíciókra összegzi az egyezést. Az üres mindkét maszk esetén a Dice nevezője nulla, ezért külön szabály szükséges (a 27.3. Dice, IoU és aggregálás szerint például 1-et rendelünk hozzá az üres-üres párhoz).

28.4.3.4 Retrieval: relevancia-nevező és ranking

Legyen egy query q, gallery $G = \{g_1, g_2, g_3, g_4\}$, releváns elemek $\{g_1, g_3\}$, tehát a nevező 2. A távolságok növekvő sorrendben: $d(q, g_2) = 0{,}15$, $d(q, g_1) = 0{,}20$, $d(q, g_4) = 0{,}35$, $d(q, g_3) = 0{,}40$. A ranking: $[g_2, g_1, g_4, g_3]$. Ekkor a top-k listák releváns elemszáma: top-1-ben 0, top-2-ben 1, top-3-ban 1, top-4-ben 2. A Recall@k:

$$\text{Recall@1} = 0, \quad \text{Recall@2} = \frac{1}{2}, \quad \text{Recall@3} = \frac{1}{2}, \quad \text{Recall@4} = 1.$$

Ez a kontroll közvetlenül mutatja, miért hibás a $\frac{|\text{top-}k\text{-ban releváns}|}{k}$ képlet: top-3-ban az $1/3$ hamisan alacsonyabb értéket adna, mint a helyes $1/2$, mert a nevező a releváns elemek száma, nem a lista hossza.

28.4.4 4. Labor: új szintetikus DotPair retrieval protokoll

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A VII. ellenőrző pont laborja egyetlen új, procedurális szintetikus vizuális problémát definiál: **DotPair**. A feladatválasztás a retrieval, mert ez a 28.3. Retrieval-értékelés és adatfelosztás fogalmait közvetlenül a korábbi CNN-gerincre alkalmazza.

28.4.4.1 Adatgenerátor és relevancia-reláció

Minden képen 24×24-es fekete háttéren két fehér, 2×2-es blokk látható. Egy **scene** a két blokk rögzített (x_1, y_1, x_2, y_2) koordináta-négyese; a koordinátákat a generátor a kép 0–21 közötti egész rácspontjain egyenletesen mintavételezi úgy, hogy a két blokk ne fedjen át. A scene két **view**-ja ugyanezt a geometriát kapja, de a két view-hoz független, $\sigma = 0{,}08$ szórású Gauss-zajt adunk hozzá, majd a képet $[0, 1]$-re vágjuk. A relevancia-reláció: q és g akkor releváns pár, ha `scene_id` megegyezik és `view_id` eltér. Nincs osztálycímke; a cél nem a tájolás felismerése, hanem a zajos párok megtalálása.

A generátor a `lab/n112_dotpair/generate_dotpair.py` fájlban a hallgató feladata; a következő szerződést kell megvalósítania:

```
Munkakönyvtár: lab/n112_dotpair/
Parancs: python generate_dotpair.py --output data --seed 20261112
Kimenet:
  data/images.npy       (180, 1, 24, 24) float32, [0, 1]
  data/metadata.csv     180 sor: index, scene_id, view_id, split
  data/manifest.json    verzió, seed, alak, SHA-256
```

A split: 60 train scene (120 kép), 15 val scene (30 kép), 15 test scene (30 kép). A query-k a test scene-k `view_id=0` képei; a gallery a test scene-k `view_id=1` képei **plusz** a val scene-k mindkét view-ja. A releváns gallery elemek csak a test scene azonos scene_id-hez tartozó view-1 képei, tehát query-nként pontosan egy releváns elem van; a val halmazból származó gallery elemek disztraktorok (zavaró, nem releváns elemek). Ez a felépítés garantálja, hogy a query és a releváns gallery elem ugyanazt a geometriát, de eltérő zajrealizációt tartalmaz.

28.4.4.2 Baseline, modell és protokoll

A baseline a nyers pixelvektorok euklideszi távolsága L2-normalizálás után: minden képet 576 dimenziós vektorként kezelünk, $g \in \mathbb{R}^{576}$, a normalizált vektor $\hat{g} = g/\|g\|_2$; a ranking a növekvő $\|\hat{q} - \hat{g}\|_2$ sorrendje. Normalizált vektorokon ez a koszinusz-hasonlósággal monoton ekvivalens rendezést ad. A modell egy kis CNN-embedding:

```
Conv2d(1, 8, 3) -> ReLU -> MaxPool2d(2) ->
Conv2d(8, 16, 3) -> ReLU -> MaxPool2d(2) -> Flatten ->
Linear(16*4*4, 32) -> L2-normalize
```

A paraméterszám körülbelül 9472. A tanítás triplet loss marginnal $\alpha = 0{,}5$, minibatch 32, Adam (tanulási ráta 10^{-3}), seed 20261112, float32, CPU. A pozitív pár az azonos scene másik view-ja, a negatív egy másik scene véletlen view-ja. A modell tanítása a 24.4. VI. ellenőrző pont: megbízható CNN-training receptjét követi: a legjobb checkpoint kiválasztása kizárólag a val halmazon mért triplet loss alapján történik; a test query/gallery készlet a modellválasztás után egyetlen alkalommal kerül kiértékelésre.

28.4.4.3 Kiértékelő kód és kimeneti séma

A `retrieval_eval.py` a következő minimális, ellenőrizhető magot tartalmazza; a teljes tanítási ciklust és a parancssori burkolót (argumentumkezelés, betöltés, riport) a hallgató a korábbi laborokból adaptálja. A generátor és a tanítás ebben a fejezetben nem futott le, ezért a kimenetre itt nem adunk mért számokat.

```python
# lab/n112_dotpair/retrieval_eval.py
import numpy as np

def recall_at_k(distances, gallery_scene_ids, gallery_view_ids,
                query_scene_id, query_view_id, k):
    # distances: (M,) távolságok a gallery elemeihez, növekvő a jobb
    # gallery_scene_ids, gallery_view_ids: a gallery rendezett metadata oszlopai
    same_scene = (gallery_scene_ids == query_scene_id)
    assert not np.any(same_scene & (gallery_view_ids == query_view_id)), \
        "a query view-ja nem szerepelhet a galleryben"
    relevant_mask = same_scene & (gallery_view_ids != query_view_id)
    n_rel = int(relevant_mask.sum())
    assert n_rel == 1, f"query-nként pontosan egy releváns elem kell: {n_rel}"
    order = np.argsort(distances)
    top_indices = order[:k]
    hits = relevant_mask[top_indices].sum()
    recall = float(hits) / n_rel
```

```
    assert 0.0 <= recall <= 1.0
    return recall, n_rel, int(hits)

# Betöltés: images.npy, metadata.csv; query = test/view 0, gallery = test/view 1 + val
# A teljes munkamenet a labor reportban szerepel.
# Ellenőrzés: minden query-hez n_rel == 1, a query view-ja nincs a galleryban.
```

A teljes labor tervezett futtatási parancsai (a parancssori burkoló a hallgató feladata):

```
cd lab/n112_dotpair
python generate_dotpair.py --output data --seed 20261112
python retrieval_eval.py --data_dir data --split test --method baseline --seed 20261112
python retrieval_eval.py --data_dir data --split test --method model --seed 20261112
```

A kimeneti JSON sémája a következő. Mivel a generátor és a tanítás nem futott le, a számértékek helyén a **VÁRT ÉRTÉK (levezetésből, nem mért)** címke áll; a tényleges futtatás után ide a mért számok kerülnek. A séma futásidőre és verzióra nem tesz állítást.

```
{
  "seed": 20261112,
  "baseline_recall_at_1": "VÁRT ÉRTÉK (levezetésből, nem mért)",
  "baseline_recall_at_2": "VÁRT ÉRTÉK (levezetésből, nem mért)",
  "model_recall_at_1": "VÁRT ÉRTÉK (levezetésből, nem mért)",
  "model_recall_at_2": "VÁRT ÉRTÉK (levezetésből, nem mért)",
  "note": "A generátor és a tanítás nem futott le; a számértékek a tényleges futás után
   ↪  töltendők ki."
}
```

A 15 test query miatt egy tényleges mérésben a Recall@k minden értéke $m/15$ alakú (m egész), azaz $1/15 \approx 0{,}0667$ többszöröse; ez a query-nként pontosan egy releváns elemet tartalmazó protokoll következménye, nem mért eredmény.

Az ellenőrzési protokoll négy lépése: (1) a SHA-256 manifest az adat regenerálhatóságát igazolja; (2) a query-k és gallery-k diszjunkt `view_id`-t tartalmaznak; (3) query-nként pontosan egy releváns gallery elem van; (4) a Recall@k nevezője minden query-re 1, tehát a binarizált átlag megegyezik a releváns elem top-k-ba kerülésének gyakoriságával. Ha a modell Recall@1 nem haladja meg a baseline-t, az önmagában nem bizonyítja, hogy a CNN-embedding nem tanult jobb hasonlóságot, mint a nyers pixel: lehet a modell túl kicsi, a margin rossz vagy az inicializálás nem megfelelő, de az is lehet, hogy a tesztminta kicsi, a baseline erős, a metrika érzéketlen, vagy a konfidenciaintervallum széles. A helyes következtetés ezért az, hogy ezen az adaton és ezzel a protokollal javulás nem mutatható ki. A DotPair eredménye semmiképpen nem általánosítható természetes képi retrieval benchmarkra; csak azt bizonyítja, hogy a protokoll a megadott feltételek mellett korrekt módon értékel.

28.4.5 5. Feladatok

F1 – fogalmi, nehézség 1. Definiáld a *target*, *output* és *metrika* hármast mind a négy feladatcsaládra az ebben az alfejezetben rögzített alkalmazási szerződés szerint. Adj egy olyan példát, amelyben az output nem metrika: egy softmax-vektor miért nem pontosság, és egy logits-sor miért nem osztálycímke? Átadandó eredmény: egy táblázat és két mondat indoklás.

F2 – levezetési, nehézség 1. Adott query q, gallery $G = \{g_1, g_2, g_3\}$, releváns elemek $\{g_2\}$, távolságok $d(q, g_1) = 0{,}40$, $d(q, g_2) = 0{,}25$, $d(q, g_3) = 0{,}10$. Számítsd ki a Recall@1, Recall@2 és Recall@3 értékét, és indokold a nevező választását. Átadandó eredmény: három szám, a rendezett lista és a nevező igazolása.

F3 – implementációs, nehézség 2. Írj Python-függvényt `iou(box_a, box_b)` néven, amely nullától indexelt, fél-nyílt xyxy dobozokra IoU-t számol. Az input két `(xmin, ymin, xmax, ymax)` quadrupla, deklarált típusú `float` vagy `int`; a kimenet float. Ellenőrizd a 28.4.3.2. szakasz kontrolljával: $[1, 2, 5, 6]$ és $[3, 4, 7, 8]$ mellett a kimenet $1/7$, tolerancia 10^{-7}. Átadandó eredmény: futtatható kód importtal, valamint a kontroll kimenete.

F4 – hibakeresési, nehézség 2. Egy kolléga a retrieval-kiértékelésben a Recall@k-t így számolja: a top-k listában szereplő releváns elemek számát osztja k-val. Magyarázd el, miért hibás ez a formula, és mutass konkrét ellenpéldát a 28.4.3.4. szakasz példájából, ahol a hibás érték eltér a helyestől. Adj javítást és egy visszaellenőrzési lépést. Átadandó eredmény: az ok, az ellenpélda számértékei, a javítás és az ellenőrzés.

F5 – kísérleti/transzfer, nehézség 3. A DotPair labor jelenleg retrievalt értékel. Tervezz protokollt ugyanerre az adathalmazra **detektálási** feladatra: a modellnek a két blokk középpontját kell becsülnie (x_1, y_1, x_2, y_2) formában. Add meg a target-konverziót, a baseline-t (például a kép súlypontja mindkét pontra vagy a két legfényesebb régió centroidja), a splitet, a metrikát (átlagos euklideszi koordinátahiba, vagy pont-IoU), valamint egy érvényességi korlátot. Átadandó eredmény: legalább egy lehetséges terv és egy alternatíva indoklással.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

28.4.6 6. Megoldások és indoklás

F1. A megoldás táblázatosan összefoglalható: osztályozás targetje osztályindex, outputja logits, metrikája accuracy; detektálás targetje dobozlista, outputja detektált dobozok score-ral, metrikája AP; szegmentálás targetje pixelcímke-tensor, outputja pixel-logits, metrikája mIoU/Dice; retrieval targetje relevancia-reláció, outputja embedding, metrikája Recall@k/mAP. Az output nem metrika, mert a softmax-vektor egy valószínűségi előrejelzés, nem egyetlen értékelő szám: [0,9; 0,05; 0,05] akkor is lehet hibás, ha a 0,9 nem a target osztályra esik. Az accuracy a teljes kiértékelő halmazra aggregál, a logits pedig aktiváció előtti pontszám, amelyből a döntés csak argmax-szal nyerhető.

F2. A releváns elemek halmaza $\{g_2\}$, tehát a nevező 1. A rendezett lista: $[g_3, g_2, g_1]$. Top-1-ben g_3 nem releváns, tehát Recall@1 $= 0$; top-2-ben $\{g_3, g_2\}$, az egyetlen releváns elem benne van, tehát Recall@2 $= 1$; top-3-ban Recall@3 $= 1$. A nevező a releváns elemek száma, nem a top-k hossza; ha k-val osztanánk, top-3-ban 1/3-ot kapnánk, ami hibás.

F3. A teljes függvény:

```
def iou(box_a: tuple[float, float, float, float],
        box_b: tuple[float, float, float, float]) -> float:
    xmin_a, ymin_a, xmax_a, ymax_a = box_a
    xmin_b, ymin_b, xmax_b, ymax_b = box_b
    inter_xmin = max(xmin_a, xmin_b)
    inter_ymin = max(ymin_a, ymin_b)
    inter_xmax = min(xmax_a, xmax_b)
    inter_ymax = min(ymax_a, ymax_b)
    inter_w = max(0.0, inter_xmax - inter_xmin)
    inter_h = max(0.0, inter_ymax - inter_ymin)
    inter = inter_w * inter_h
    area_a = (xmax_a - xmin_a) * (ymax_a - ymin_a)
    area_b = (xmax_b - xmin_b) * (ymax_b - ymin_b)
    union = area_a + area_b - inter
    if union <= 0.0:
        return 0.0
```

```
    return inter / union

# Kontroll
box_gt = [1, 2, 5, 6]
box_pred = [3, 4, 7, 8]
val = iou(box_gt, box_pred)
assert abs(val - 1/7) < 1e-7, val
print(val)  # 0.14285714285714285
```

A fél-nyílt intervallum miatt a szélesség egyszerűen $x_{\max} - x_{\min}$; a `max(0.0, ...)` a diszjunkt dobozokra ad 0 metszetet.

F4. A hiba oka: a kolléga a *precision-at-k*-t számítja, nem a *recall*-t. A recall nevezője a query összes releváns gallery elemének száma, mert azt méri, mekkora hányadát találta meg a modell a releváns elemeknek. A 28.4.3.4. szakasz példájában a releváns elemek $\{g_1, g_3\}$, tehát a nevező 2; a top-3 lista $[g_2, g_1, g_4]$, amelyben 1 releváns elem van. A hibás formula $1/3$, a helyes $1/2$. A javítás: a nevezőt `n_rel` néven tároljuk, nem `k`-ként. A visszaellenőrzés: a 28.4.3.4. szakasz kontrolljában top-4-re mindkét releváns elem benne van, tehát a helyes érték $2/2 = 1$, míg a hibás $2/4 = 0{,}5$, ami nyilvánvaló ellentmondás, mert az összes releváns elem megtalálva elérte a maximális recallt.

F5. Egy lehetséges terv: a DotPair generátor a koordinátákat a `metadata.csv`-ben vagy külön `targets.npy` fájlban elmenti; a detektálási target tehát képenként egy 4-dimenziós vektor (x_1, y_1, x_2, y_2) float pixelkoordináta. A baseline a kép tömegközéppontja, amely mindkét pontra ugyanazt a koordinátát adja; ez rögzíti a triviális hiba nagyságát, amelynek a modellnél alacsonyabbnak kell lennie. A metrika az átlagos euklideszi koordinátahiba: $\frac{1}{N}\sum_n \|\hat{p}_n - p_n\|_2$, ahol $\hat{p}_n$ a modell négy koordinátája. A split scene-csoportos, ahogy a retrievalnél; a modell ugyanazt a CNN-gerincet használhatja, de a fej egyetlen lineáris réteg 4 kimenettel, és a loss Huber vagy MSE. Az alternatíva pontonkénti IoU: minden predikált ponthoz 3×3-es dobozt rendelünk, és a ground truth ponthoz ugyancsak, majd a két pont párosítását greedy módon végezzük; ez érzékenyebb a pontpárok felcserélésére, ezért a koordinátahiba stabilabb kiindulás. Érvényességi korlát: a két pont sorrendjének felcserélése a négykoordinátás MSE-t mesterségesen megnöveli, ezért a kiértékelés előtt a predikált pontokat a legközelebbi ground truth ponthoz kell párosítani; az eredmény nem valós detektálási benchmark.

28.4.7 7. Gyakori tévedés, önellenőrzés és visszahívás

A leggyakoribb tévedés: ugyanaz a CNN-gerinc látszólag „mindent tud”, ezért a hallgató a különböző célok metrikáit azonos skálán hasonlítja össze. Ellenpélda: legyen $f(x) = c$ konstans embedding minden bemenetre. Ha osztályozási fejjel a logits mindig ugyanazt az osztályt adja, az accuracy kiegyensúlyozott három osztályra $1/3$; retrievalben pedig minden távolság nulla, a ranking tetszőleges, tehát a Recall@k várhatóan a relevanciaarány, ami egyetlen releváns elem és M gallery elem mellett k/M körüli. Az $1/3$ accuracy és a k/M recall nem összehasonlítható; az első a címke-találati arány, a második a releváns elemek megtalálásának feltételes aránya.

Önellenőrzéshez három kérdést érdemes feltenni: (1) A veszteség minden célalakkal összhangban van-e, például a pixel-loss az $H \times W$ pozíciók átlagát veszi-e? (2) A fél-nyílt xyxy doboz területét nem összekevertük-e a zárt intervallummal, ahol egy extra pixel hozzáadódna? (3) A retrieval nevezője minden query-re a releváns elemek száma, nem pedig k? Ha bármelyikre nemleges a válasz, a kontrollt újra kell végezni.

Válaszkulcs. (1) Igen, ha a pixel-loss az érvényes $H \times W$ pozíciók átlagát veszi (maszkolt esetben az érvényesekét), és a shape-assert ezt ellenőrzi. (2) Nem kevertük össze: a fél-nyílt xyxy

területe $(x_{\max} - x_{\min})(y_{\max} - y_{\min})$, zárt konvencióban ehhez még +1 járulna. (3) Igen: a recall nevezője a queryhez tartozó releváns elemek száma, a k csak a lista hosszát korlátozza.

Visszahívás a korábbi egységekre: a 27.4. Instance mask és összetett feladat alfejezetben láttuk, hogy ugyanaz a szemantikus maszk több instance-felosztást enged, ezért a szegmentálás és detektálás targetje nem ekvivalens. A 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben rögzítettük, hogy a checkpoint kiválasztása kizárólag a val_loss alapján történik; itt ugyanez az elv érvényes a triplet lossra. A 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése operátor-specifikációja pedig biztosította, hogy a gerinc keresztkorrelációt számol, amit a különböző fejek változtatás nélkül használhatnak.

28.4.8 8. Rövid összefoglaló

A VII. ellenőrző pont a CNN-alkalmazások szerződését rögzíti. Négy feladatcsaládot különböztetünk meg: az osztályozás célja osztályindex, a detektálásé változó dobozlista, a szegmentálásé pixelcímke-tensor, a retrievalé relevancia-reláció. Mindegyikhez eltérő loss, output, metrika, split és inference szerződés tartozik; ugyanaz a CNN-gerinc önmagában nem teszi a metrikákat összehasonlíthatóvá. Kézi kontrollokkal igazoltuk a softmax-összeget, a fél-nyílt dobozok IoU-ját, a bináris maszk Dice-értékét és a Recall@k nevezőjét. A DotPair labor új szintetikus retrievalt definiál, ahol query-nként egy releváns elem van; a protokoll baseline-t, triplet-loss tanítást és szigorú ellenőrzési lépéseket ad.

28.4.9 9. Következő alkalmazás

A 29.1. Lineáris és nemlineáris autoencoder az autoencoder felé lép: a retrieval-embedding célzott hasonlósági tanulás, míg az autoencoder a rekonstrukciós célból tanul latent reprezentációt. A most bevezetett split-, metrika- és inference-szerződés fogalmai közvetlenül alkalmazhatók lesznek a rekonstrukciós feladatra is.

28.4.10 10. Források

- [P16] Schroff, F., Kalenichenko, D., Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering.* arXiv:1503.03832. https://arxiv.org/abs/1503.03832.
- [D15] scikit-learn developers (2026). *Model evaluation: quantifying the quality of predictions.* scikit-learn documentation. https://scikit-learn.org/stable/modules/model_evaluation.html. Hozzáférés: 2026-10-30.

29 Autoencoder, denoising és reprezentációs korlát

29.1 Lineáris és nemlineáris autoencoder

29.1.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben az autoenkódolót (autoencoder) mint adatvezérelt rekonstrukciós modellt vezetjük be. A korábbi 25.1. Teljes képosztályozó adatfolyam osztályozási adatfolyam a képet címkévé alakította; itt a cél maga a bemenet visszaállítása egy szűk reprezentáción keresztül. A 28.1. Embedding, távolság és normalizálás embedding-tanulásából tudjuk, hogy a reprezentáció hasznossága a downstream céltól függ; ez az autoencoder esetében is fontos korlát lesz. Előismeretként a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil veszteségszámítást, a 15.4. Független gradiensellenőrzési protokoll gradiensellenőrzést, a 23.4. Hibaszeletek és alkalmazási határ hibaszeleteket és a 27.4. Instance mask és összetett feladat encoder–decoder szerkezetű látásmódját idézzük fel. Induló kérdés: ha a függelék A. szakaszában leírt GlyphLab egy képét $d = 576$ dimenziós vektorként fogjuk fel, és $k = 72$ dimenziós kódba tömörítjük lineárisan, mekkora a minimális átlagos négyzetes rekonstrukciós hiba? A válasz a következő tétel állítása: a tanítási cél itt a mintánkénti átlag, $L = \frac{1}{N}\sum_n \|x_n - \hat{x}_n\|_2^2$, ezzel a minimum $\sum_{i=k+1}^{r} \lambda_i$, ahol λ_i a tapasztalati kovarianciamátrix sajátértékei. Ha valaki a dimenziókra is átlagol (például a `MSELoss reduction='mean'` beállításával), ugyanez $\frac{1}{d}\sum_{i=k+1}^{r} \lambda_i$, tehát a két konvenció d-szeres eltérést ad; a fejezet a mintánkénti átlagot használja.

29.1.2 Definíciók és jelölések

Legyen a bemeneti tér $\mathbb{R}^d$, a latens (rejtett) kód tere $\mathbb{R}^k$. Egy batch $X \in \mathbb{R}^{N\times d}$, egy megfigyelés $x \in \mathbb{R}^d$ oszlopvektor. Az **encoder** (kódoló) egy $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ paraméteres függvény, amely a

$$z = f_\theta(x) \in \mathbb{R}^k$$

latens reprezentációt állítja elő. A **decoder** (dekódoló) $g_\phi : \mathbb{R}^k \to \mathbb{R}^d$ a rekonstrukciót adja:

$$\hat{x} = g_\phi(z) \in \mathbb{R}^d.$$

A **szűk keresztmetszet** (bottleneck) feltétele $k < d$; enélkül elegendő kapacitású modell triviálisan megtanulhatná az identitást. A tanítási cél az átlagos négyzetes rekonstrukciós hiba:

$$L_{\text{rec}}(\theta, \phi) = \frac{1}{N}\sum_{n=1}^{N} \|x_n - g_\phi(f_\theta(x_n))\|_2^2.$$

A könyvtári MSELoss `reduction='mean'` esetén a pixeldimenzión is átlagol, ezért a laborban a $(1/(Nd))$ skálát használjuk.

Lineáris eset. Ha f és g lineáris leképezés (azaz bias nélküli affin leképezés), akkor a 3.2. Mátrixszorzás és affín réteg rétegkonvenciója szerint $W_e \in \mathbb{R}^{k\times d}$, $W_d \in \mathbb{R}^{d\times k}$ mellett a batch sormintájában a megfigyelések sorvektorként szerepelnek, $X_{n,:} = x_n^\top$, ezért a $z = W_e x$ oszlopkonvenció batch alakja

$$Z = XW_e^\top \in \mathbb{R}^{N\times k}, \qquad \hat{X} = ZW_d^\top = X(W_dW_e)^\top \in \mathbb{R}^{N\times d}.$$

A centrálást külön lépésként kezeljük, mert a PCA-kapcsolathoz nulla átlagú adatra van szükség.

29.1.3 A lineáris autoencoder és a PCA kapcsolata

A PCA sajátérték-felbontáson alapuló tárgyalását a [B01] tankönyv vonatkozó fejezetei rendszerezik; az alábbi állítást és bizonyítást a fejezet önállóan vezeti le.

Legyen $X \in \mathbb{R}^{N\times d}$ centrált, azaz $\frac{1}{N}\sum_n x_n = 0$. Legyen a tapasztalati kovarianciamátrix $\Sigma = \frac{1}{N}X^\top X$, sajátértékei $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$, a hozzájuk tartozó ortonormált sajátvektorok $v_1, \ldots, v_d$. Rögzítsünk $k < d$ értéket.

Tétel (lineáris autoencoder globális optimuma). Legyen $X \in \mathbb{R}^{N\times d}$ centrált, $r = \operatorname{rank}(X)$, és tegyük fel $k < r$. A

$$\min_{W_e, W_d} \ \frac{1}{N}\left\|X - X(W_dW_e)^\top\right\|_F^2$$

feladat optimuma $\sum_{i=k+1}^{r} \lambda_i$. Ha $\lambda_k > \lambda_{k+1}$ és $\operatorname{rank}(X) = d$, akkor az optimális W_dW_e szorzat egyértelmű, mégpedig $W_dW_e = V_kV_k^\top$, ahol $V_k = [v_1, \ldots, v_k]$. A sajátérték-rés feltétele szükséges: ha $\lambda_k = \lambda_{k+1}$, a top k altér nem egyértelmű, ezért az egyértelműségi állítás ilyenkor nem érvényes. A lineáris autoencoder és a PCA kapcsolatának klasszikus eredménye Baldi és Hornik (1989).

Bizonyításvázlat. A rekonstrukció $\hat{X} = XC$, ahol $C = (W_dW_e)^\top \in \mathbb{R}^{d\times d}$ és $\operatorname{rank} C \leq k$, ezért $\hat{X}$ legfeljebb k rangú mátrix. A klasszikus Eckart–Young-tétel szerint minden legfeljebb k rangú B mátrixra $\|X - B\|_F^2 \geq \sum_{i=k+1}^{r} \sigma_i^2$, ahol σ_i az X szinguláris értékei; egyenlőség a csonkolt SVD-vel, $B = U_k\Sigma_kV_k^\top$ mellett teljesül. Mivel $\sigma_i^2/N = \lambda_i$, az alsó korlát az állított sajátértékösszeg. A $C = V_kV_k^\top$ választással $\hat{X} = XV_kV_k^\top = U_k\Sigma_kV_k^\top$, tehát a korlát elérhető. A sajátérték-rés miatt az altér egyértelmű; teljes rangú X esetén a C mátrix, így W_dW_e is egyértelmű.

Az Eckart–Young-tételt (Eckart–Young, 1936) itt idézett eredményként használjuk; bizonyítását nem ismételjük, mert az SVD-tulajdonságokra épül. A 3.4. Sajátérték, SVD és spektrális határ egységben felépített SVD eszköztár alkalmas a nyomon követésére. A tétel tartalmát a megadott lineáris algebrai feltételek szerint használjuk; a levezetés tehát az Eckart–Young-tételre épül, önálló bizonyításnak nem tekinthető.

Az optimális W_e, W_d páros nem egyértelmű. Ha W_e, W_d optimum, akkor tetszőleges invertálható $C \in \mathbb{R}^{k\times k}$ mellett $W_e' = CW_e$, $W_d' = W_dC^{-1}$ is optimum, mert $W_d'W_e' = W_dW_e$. A latens térbeli koordináták tetszőleges invertálható lineáris keveredéséig határozottak meg; a rekonstrukciós altér azonban egyértelmű.

29.1.4 Eredeti kézi kis példa

Tekintsük a centrált adatot

$$X = \begin{pmatrix} 2 & 0 \\ -2 & 0 \\ 0 & 1 \\ 0 & -1 \end{pmatrix}.$$

Itt $N = 4$. A kovariancia:

$$X^\top X = \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 4 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 8 & 0 \\ 0 & 2 \end{pmatrix},$$

tehát $\Sigma = \frac{1}{4} X^\top X = \mathrm{diag}(2,\ 0{,}5)$. A $k = 1$ esetben az első sajátvektor $v_1 = (1, 0)^\top$, a második elhagyott sajátérték $\lambda_2 = 0{,}5$. Válasszuk az optimális paramétereket $W_e = v_1^\top = (1, 0)$ és $W_d = v_1 = (1, 0)^\top$ alakban. Ekkor

$$Z = XW_e^\top = \begin{pmatrix} 2 \\ -2 \\ 0 \\ 0 \end{pmatrix}, \qquad \hat{X} = ZW_d^\top = \begin{pmatrix} 2 & 0 \\ -2 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}.$$

A hibák az utolsó két sorban 1 és 1, ezért a batch-átlag $\frac{1}{4}(0 + 0 + 1 + 1) = \frac{1}{2} = \lambda_2$, pontosan a tétel állítása szerint.

29.1.5 Nemlineáris autoencoder és a labor CNN modellje

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A nemlineáris esetben f_θ és g_ϕ általában ReLU- és konvolúciós rétegekből áll; a cél nem konvex, és nincs PCA-hoz hasonló zárt megoldás. A rekonstrukciós hiba csökkentése itt sem garantál hasznos downstream reprezentációt. A laborban egy kis CNN-autoencodert vetünk össze a lineáris, PCA-alapú rekonstrukciós baseline-nal a GlyphLab adaton.

A CNN encoder kimenete a $8{\times}3{\times}3 = 72$ dimenziós latens; ez a CNN szűk keresztmetszete, amelyet az architektúra rögzít. A parancssori `--latent` kapcsoló csak a lineáris PCA-ág dimenzióját állítja; a két ág összehasonlításához a PCA dimenzióját is 72-re állítjuk. A decoder transzponált konvolúciókkal állítja vissza a $1 \times 24 \times 24$ méretet. A transzponált konvolúció itt a megfelelő lineáris operátor adjungáltja, nem általános inverz; a rétegek a 15.3. Transposed convolution mint adjungált alfejezetben bevezetett értelemben nem az encoder inverzét számolják.

A labor a `lab_autoencoder` munkakönyvtárban fut. A GlyphLab splitje a `metadata.csv`-ből jön: a 360 képből 240 train, 60 val és 60 test kép van, és az `index` oszlop az `images.npy` sorindexe. A tanítás a train, az epoch-választás a val, a végső összehasonlítás a test halmazon történik. A lineáris baseline a train adatból számított SVD-vel zárt alakú; a CNN-autoencodert Adam optimalizálóval tanítjuk. A GlyphLab képei a $[0, 1]$ tartományban vannak, ezért a decoder `Sigmoid` kimenete illeszkedik a célhoz; a szkript ezt betöltés után ellenőrzi. A szkript a mellékelt

GlyphLab-adaton, CPU-n lefutott (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, seed 123, egy szál); a mért kimenetet a futtatási parancs utáni blokk tartalmazza.

```
# lab_autoencoder/autoencoder_lab.py
import argparse, csv, json
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

def split_indices(meta_path):
    """A metadata.csv index oszlopa az images.npy sorindexe, nem a CSV sorszáma."""
    result = {"train": [], "val": [], "test": []}
    with open(meta_path, newline="") as f:
        for row in csv.DictReader(f):
            result[row["split"]].append(int(row["index"]))
    return result

class CNNAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(32, 8, 3, stride=2, padding=1), nn.ReLU(),
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(8, 32, 4, stride=2, padding=1), nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 4, stride=2, padding=1), nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 4, stride=2, padding=1), nn.Sigmoid(),
        )
    def forward(self, x):
        return self.decoder(self.encoder(x))

def linear_pca(images, idx_train, idx_test, k):
    Xtr = images[idx_train].reshape(len(idx_train), -1).astype(np.float32)
    Xte = images[idx_test].reshape(len(idx_test), -1).astype(np.float32)
    mu = Xtr.mean(axis=0, keepdims=True)
    U, S, Vt = np.linalg.svd(Xtr - mu, full_matrices=False)
    Vk = Vt[:k].T
    Xtr_hat = (Xtr - mu) @ Vk @ Vk.T + mu
    Xte_hat = (Xte - mu) @ Vk @ Vk.T + mu
    return float(np.mean((Xtr - Xtr_hat) ** 2)), float(np.mean((Xte - Xte_hat) ** 2))

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--data", required=True)
    p.add_argument("--meta", required=True)
    p.add_argument("--latent", type=int, default=72,
                   help="a lineáris PCA-ág latens dimenziója; a CNN szűk keresztmetszete fix
                    ↪ 8x3x3=72")
    p.add_argument("--epochs", type=int, default=20)
    p.add_argument("--batch", type=int, default=32)
    p.add_argument("--lr", type=float, default=1e-3)
    p.add_argument("--seed", type=int, default=123)
    args = p.parse_args()
    if not 1 <= args.latent <= 576:
        p.error("--latent 1 és 576 közé kell essen")
    if args.epochs < 1:
        p.error("--epochs legalább 1")

    torch.manual_seed(args.seed)
    np.random.seed(args.seed)
```

```
    torch.set_num_threads(1)
    device = torch.device("cpu")
    images = np.load(args.data).astype(np.float32)          # (360,1,24,24)
    idx = split_indices(args.meta)
    assert images.shape == (360, 1, 24, 24), images.shape
    assert [len(idx[k]) for k in ("train", "val", "test")] == [240, 60, 60]
    all_idx = idx["train"] + idx["val"] + idx["test"]
    assert sorted(all_idx) == list(range(images.shape[0])), \
        "a metadata index nem fedi le egyedül az images.npy sorait"
    assert 0.0 <= float(images.min()) and float(images.max()) <= 1.0, \
        "a képeknek a [0,1] tartományban kell lenniük a Sigmoid kimenethez"

    lin_train, lin_test = linear_pca(images, idx["train"], idx["test"], args.latent)

    model = CNNAutoencoder().to(device)
    opt = torch.optim.Adam(model.parameters(), lr=args.lr)
    lossfn = nn.MSELoss()
    tr = torch.from_numpy(images[idx["train"]])
    va = torch.from_numpy(images[idx["val"]])
    te = torch.from_numpy(images[idx["test"]])
    tr_loader = DataLoader(TensorDataset(tr, tr), batch_size=args.batch, shuffle=True)

    best_val, best_state = float("inf"), None
    for _ in range(args.epochs):
        model.train()
        for xb, _ in tr_loader:
            opt.zero_grad()
            loss = lossfn(model(xb.to(device)), xb.to(device))
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            vloss = lossfn(model(va.to(device)), va.to(device)).item()
            if vloss < best_val:
                best_val, best_state = vloss, {k: v.detach().clone() for k, v in
↪   model.state_dict().items()}

    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        cnn_test = lossfn(model(te.to(device)), te.to(device)).item()
        cnn_train = lossfn(model(tr.to(device)), tr.to(device)).item()
    assert np.isfinite([lin_train, lin_test, best_val, cnn_test, cnn_train]).all()
    assert 0.0 <= cnn_test <= 1.0

    print(json.dumps({"linear_train_mse": lin_train, "linear_test_mse": lin_test,
                      "cnn_val_mse": best_val, "cnn_test_mse": cnn_test,
                      "cnn_train_mse": cnn_train}))

if __name__ == "__main__":
    main()
```

Futtatási parancs:

```
python autoencoder_lab.py --data ../lab/data/images.npy --meta ../lab/data/metadata.csv
↪  --latent 72 --epochs 20 --batch 32 --lr 0.001 --seed 123
```

Az ellenőrzés a shape-ekre: a lineáris PCA bemenete $(240, 576)$, a teszt $(60, 576)$; a CNN bemenete $(N, 1, 24, 24)$, az encoder kimenete $(N, 8, 3, 3)$, a dekódolóé $(N, 1, 24, 24)$. A `linear_test_mse` a PCA-altérre vetítés hibája a testen. A fenti parancs mért kimenete (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, seed 123, egy szál, 20 epoch):

```
{
  "linear_train_mse": 0.0019550886936485767,
  "linear_test_mse": 0.005545615218579769,
  "cnn_val_mse": 0.014753654599189758,
  "cnn_test_mse": 0.014527441933751106,
  "cnn_train_mse": 0.014240337535738945
}
```


A mért összehasonlításban a lineáris PCA-baseline test-MSE-je kisebb, mint a CNN-autoencoderé; a CNN train-MSE-je sem alacsonyabb a PCA test-hibájánál. Ez a konkrét futás tehát nem támasztja alá, hogy a nagyobb kapacitású nemlineáris autoencoder jobb rekonstrukciót ad ezen a kis adaton és 20 epoch mellett. A rekonstrukciós MSE továbbá nem downstream reprezentációminőség: a következtetés csak a rekonstrukciós hibára érvényes, és a val-epocha-választás, valamint a véges testméret miatt óvatosan kezelendő.

29.1.6 Gyakori tévedés ellenpéldával

A leggyakoribb tévedés, hogy „minden autoencoder PCA-t számol, és a rekonstrukció garantálja a hasznos downstream reprezentációt". Ez két ponton is hamis. Egyrészt lineáris esetben is csak a rekonstrukciós altér egyértelmű; a latens koordináták az invertálható C keveredés miatt nem feltétlenül a főkomponens-koordináták. Másrészt a varianciaközpontú PCA elhagyhat éppen olyan kis szórású jellemzőt, amely a downstream döntéshez elengedhetetlen. Példa: legyen $x = (x_c, x_f)$, ahol $x_c \in \{-2, 2\}$ nagy szórású háttérváltozó, $x_f \in \{-0{,}1;\ 0{,}1\}$ pedig a címkét meghatározó jellemző. Tegyük fel, hogy a két változó *független* és *kiegyensúlyozott*: mindkét érték $\frac{1}{2}$ valószínűségű. A függetlenség szükséges a példához: ha x_c és x_f tökéletesen korrelált volna, akkor a háttérváltozó is meghatározná a címkét, és a PCA nem veszítené el a címkeinformációt. Független változók mellett a kovariancia $\mathrm{diag}(4,\ 0{,}01)$, a főirány e_1, és a megtartott x_c koordináta valóban nem hordoz címkeinformációt. A $k = 1$ lineáris autoencoder a főkomponens irányát, x_c-t tartja meg; a rekonstrukciós hiba az elhagyott x_f irányból származik, értéke a tétel szerint $\lambda_2 = 0{,}01$, ami kicsi, de a latensből a címke emiatt nem állítható helyre. A rekonstrukciós cél tehát adattömörítő, nem automatikusan szemantikai jellemzőtanuló.

29.1.7 Önellenőrzés és visszahívás

Ellenőrizzük magunkat: (a) mi változik a lineáris optimumban, ha a bias nem nulla? (b) miért nem egyértelműek a lineáris súlyok? (c) mikor nulla a minimális hiba? A válasz: (a) a centrálást a train átlaggal kell kezelni, mert a bias nélküli levezetés nulla átlagot feltételez; (b) a latens invertálható lineáris keverése az altérre vetítést változatlanul hagyja; (c) ha az adat már eleve egy k-dimenziós altérben van. Visszahívás: a 17.1. LeNet és a lokális megosztott reprezentáció alfejezetben látott konvolúciós építőelemek itt az encoder–decoderben térnek vissza; a 28.1. Embedding, távolság és normalizálás alfejezetben tanultakat pedig úgy alkalmazzuk, hogy a reprezentáció minősége csak a megcélzott feladaton mérhető.

29.1.8 Feladatok

F1 (fogalmi, nehézség 1). Fogalmazza meg, miért nem állítható, hogy minden autoencoder PCA-t számol. Adjon két különböző okot, egy lineáris és egy nemlineáris példával. *Átadandó:* legfeljebb fél oldalas indoklás az altér és a koordináta megkülönböztetésével.

F2 (levezetési, nehézség 2). A fenti kézi példa X mátrixára SVD-könyvtár nélkül, a kovarianciamátrixból vezesse le a $k = 1$ lineáris autoencoder optimális irányát és minimális átlagos hibáját. *Átadandó:* a kovarianciamátrix, a sajátértékek, az optimális irány és a hiba.

F3 (implementációs, nehézség 1). Írjon NumPy-függvényt `pca_reconstruct(X_train, X_test, k)` aláírással, amely a train átlaggal centrál, SVD-vel kiszámítja a top k altérre vetítést, és visszaadja a test rekonstrukciót. Ellenőrizze a kézi példán: a $k = 1$ hiba 0,5 legyen 10^{-6} tűrésen belül. *Átadandó:* futtatható kód és a hiba.

F4 (hibakeresési, nehézség 2). Egy hallgató a CNN-autoencoder lossát `F.mse_loss(pred, target, reduction='sum')` módon számolja, majd összehasonlítja a lineáris baseline `reduction='mean'` értékével. Mi a hiba, mi a következménye, és hogyan javítja? *Átadandó:* javított kódrészlet és a skálakapcsolat.

F5 (kísérleti tervezés, nehézség 3). Tervezzen kísérletet annak eldöntésére, hogy a CNN-autoencoder latens reprezentációja hasznosabb-e a GlyphLab orientációs osztályozásához, mint az azonos dimenziójú PCA-latens. *Átadandó:* split, baseline, befagyasztási protokoll, metrika és érvényességi korlát.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

29.1.9 Megoldások és indoklás

F1. Az állítás azért hamis, mert (a) még a lineáris esetben sem maguk a főkomponens-koordináták egyértelműek: tetszőleges invertálható C keverés optimális marad, miközben a latens koordináták megváltoznak; (b) a nemlineáris autoencoder nemlineáris sokaságra vetít, így az optimum általában nem írható le lineáris altérrel. Konkrét nemlineáris példa: legyenek a minták az egységkörön, $x_\theta = (\cos\theta, \sin\theta)$, és a latens dimenzió $k = 1$. A lineáris $k = 1$ autoencoder csak egy egyenesre vetíthet; a centrált adat kovarianciamátrixa $\frac{1}{2}I$, így a minimális lineáris hiba $\lambda_2 = \frac{1}{2}$ mintánként. A nemlineáris encoder–decoder $z = \theta$, $\hat{x}_z = (\cos z, \sin z)$ választással viszont minden körpontot pontosan rekonstruál, tehát a hibája 0: a lineáris altér-állítás nem terjed ki a nemlineáris esetre. A pontos lineáris állítás: centrált, négyzetes hibájú, $k < d$ esetben az optimális rekonstrukciós *altér* a top k főkomponens altere; az altér egyértelműségéhez $\lambda_k > \lambda_{k+1}$ szükséges, mert egyenlőség esetén a top k altér nem egyértelmű (a hiba értéke ekkor is $\sum_{i=k+1}^{r} \lambda_i$).

F2. Az $X^\top X = \mathrm{diag}(8, 2)$ mátrixot már kiszámoltuk, így $\Sigma = \mathrm{diag}(2,\ 0{,}5)$. A sajátértékek $2 \geq 0{,}5$; az első sajátvektor $v_1 = (1, 0)^\top$. A $k = 1$ optimum $W_e = v_1^\top$, $W_d = v_1$. A rekonstrukciós hibák az utolsó két sorban 1 és 1; az átlag $\frac{1}{4}(0 + 0 + 1 + 1) = 0{,}5 = \lambda_2$, ahogy a tétel állítja. Szélsőeset: ha $k = 2$, a hiba 0, mert az adat két dimenziós altérben van.

F3. A függvény:

```
def pca_reconstruct(X_train, X_test, k):
    mu = X_train.mean(axis=0, keepdims=True)
    U, S, Vt = np.linalg.svd(X_train - mu, full_matrices=False)
    Vk = Vt[:k].T
    X_hat = (X_test - mu) @ Vk @ Vk.T + mu
    return X_hat

X = np.array([[2.,0.],[-2.,0.],[0.,1.],[0.,-1.]], dtype=np.float64)
Xh = pca_reconstruct(X, X, 1)
err = np.mean(((X - Xh) ** 2).sum(axis=1))   # mintaátlag, koordinátákra összegezve
print(err)   # 0.5
assert abs(err - 0.5) < 1e-6
```

Az ellenőrző érték 0,5, a $< 10^{-6}$ abszolút tolerancia teljesül. A kimeneti shape megegyezik a bemeneti shape-szel. (A `np.mean((X - Xh) ** 2)` alak elemenkénti átlagot venne, ami 0,25; a PCA-tételben szereplő hiba a mintánkénti négyzetösszeg átlaga, ezért kell a `sum(axis=1)`.)

F4. A hiba a reduction meg nem egyezése: a sum-hányados $N \cdot d$-szerese a mean-nek. Így a CNN hiba számszerűen nagyobbnak látszik, és a két modell nem összehasonlítható. Javítás: mindkét ágon azonos `reduction='mean'`-t használunk; például a CNN-hez is `F.mse_loss(pred, target, reduction='mean')`, és a lineáris baseline-ban `np.mean((X-Xh)**2)`. Visszaellenőrzés: egy 32×24×24 batchre a sum/mean hányados 18432 legyen.

F5. Rögzített train/val/test GlyphLab split. Baseline: PCA-latens (72 dimenzió) fölött logisztikus regresszió. Vizsgált modell: a labor CNN-autoencoder befagyasztott encoderének kimenete fölött ugyanolyan lineáris fej. A fejet csak a train latenseken tanítjuk, a val a korai leállításra szolgál; az autoencoder súlyait sem a val, sem a test nem finomhangolja. Metrika: osztályonkénti accuracy, a bizonytalanságot jelenetszintű (klaszter-) bootstrap adja: a `scene_id`-k visszatevéssel történő újramintavételezésével, nem a képek szintjén. Korlát: a rekonstrukciós előtanítás a címkét nem látta, ezért a kísérlet azt ellenőrzi, hogy a rekonstrukció megőrizte-e az orientációhoz szükséges információt; az eredmény nem általánosítható természetes képekre.

29.1.10 Rövid összefoglaló

Az autoencoder egy encodert, egy latens kódot és egy decodert kapcsol össze; a cél a bemenet rekonstrukciója. A $k < d$ szűk keresztmetszet tömörítésre kényszerít, de önmagában nem garantál hasznos reprezentációt. Centrált adatra, lineáris leképezésekkel és átlagos négyzetes hibával a globális optimum rekonstrukciós altere a top k főkomponens altere; a hibát az elhagyott sajátértékek összege adja. A latens koordináták invertálható lineáris keveredésig egyértelműek. A nemlineáris CNN-autoencodernek nincs ilyen zárt megoldása. A labor a PCA-alapú lineáris baseline-t és egy kis CNN-autoencodert veti össze a GlyphLab rekonstrukciós hibáján.

29.1.11 Következő alkalmazás és források

A 29.2. Denoising és korrupciós modell egységben a denoising autoencodert vizsgáljuk: ott a bemenet zajos, a cél a zajmentes kép, így a rekonstrukciós cél kifejezetten a zajmodellre irányul.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- Baldi, P., Hornik, K. (1989). Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1), 53–58.
- Eckart, C., Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

29.2 Denoising és korrupciós modell

29.2.1 Cél, előismeret és induló kérdés

Ez az alfejezet a zajos bemenet és a tiszta célkép közötti páros szerződést vizsgálja. Feltételezzük a 29.1. Lineáris és nemlineáris autoencoder autoencoder-alapfogalmait: az encoder, decoder, latent tér és rekonstrukciós cél ismert. Nincs szükség haladó statisztikai előismeretre; a várható érték és a feltételes eloszlás alapjait az 5.2. Várható érték, variancia és likelihood vezette be. A

denoising autoencoder tanítási elvét Vincent, Larochelle, Bengio és Manzagol (2008) vezette be; a jelen fejezet a páros szerződést és a Bayes-optimális denoisert tárgyalja.

Induló kérdés: egy 24×24-es szürke képre additív Gauss-zajt teszünk, majd egy modellnek a zajos képből a tiszta képet kell visszaadnia. Mi a legkisebb elérhető per-pixel négyzetes hiba? A válasz nem nulla, és a korrupciós modell, a cél definíciója és a tanító/target szétválasztás nélkül nem is értelmes; a kézi példa a Bayes-kockázat egy konkrét értékét is megadja.

29.2.2 Definíciók és jelölések

Tiszta kép (clean image). Egyetlen megfigyelés $x \in \mathbb{R}^{C\cdot H\cdot W}$, a függelék A. szakaszában leírt GlyphLab esetében $C = 1$, $H = W = 24$, és minden pixelérték a $[0, 1]$ intervallumba esik. Egy batch $X \in \mathbb{R}^{N\times C\times H\times W}$; az absztrakt képletekben x oszlopvektorként, $d = CHW$ dimenzióval jelenik meg.

Korrupció (corruption). A $c : \mathcal{X} \times \mathcal{E} \to \mathcal{X}$ leképezés egy tiszta képhez és egy ε zajrealizációhoz rendeli a zajos megfigyelést:

$$y = c(x, \varepsilon).$$

A $\mathcal{E}$ tér a zaj értékkészlete. A korrupció lehet véletlen, de a tanítási folyamatban a zajrealizációt explicit módon kezelni kell.

Zajeloszlás (noise distribution). A korrupció által indukált feltételes eloszlás $p_{Y|X}$. A leggyakoribb laborváltozat az additív Gauss-zaj:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I_d), \qquad y = \mathrm{clip}_{[0,1]}(x + \varepsilon),$$

ahol a klipelés elemenként $\min(1, \max(0, \cdot))$. A klipelés miatt a megfigyelt y peremeloszlása már nem tiszta Gauss, de nem is csonkolt (*truncated*) normális: a klipelés *cenzorált (clipped/censored)* zajmodellt ad, amelyben a 0 és 1 határon ponttömeg keletkezik ($P(y = 0) = P(x + \varepsilon \le 0)$, $P(y = 1) = P(x+\varepsilon \ge 1)$), a belső részen pedig a Gauss-sűrűség marad. Csonkolásnál (truncation) az intervallumra kondicionált, újranormalizált folytonos eloszlás keletkezne; a csonkolt sűrűséget ezért nem használjuk változtatás nélkül.

Denoiser. A $f_\theta : \mathbb{R}^d \to \mathbb{R}^d$ függvény a zajos képet a tiszta kép becslésére képezi: $\hat{x} = f_\theta(y)$. A 29.1. Lineáris és nemlineáris autoencoder alfejezetben bevezetett encoder–decoder szerkezet természetes választás: az encoder a zajos képet latens reprezentációba képezi, a decoder onnan rekonstruál; a kimenet alakja megegyezik a bemenet alakjával.

Rekonstrukciós veszteség (reconstruction loss). A denoising cél a tiszta target és a becsült kép közötti négyzetes eltérés. Egyetlen páron

$$\ell(x, y; \theta) = \|f_\theta(y) - x\|_2^2,$$

a teljes batch-en pedig az átlag:

$$\hat{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|f_\theta(y_i) - x_i\|_2^2. \tag{1}$$

A per-pixel MSE ettől egy $d = CHW$ osztóban tér el; a levezetésekben a (1) alakot, a mintánkénti négyzetes normák átlagát használjuk, és mindig kimondjuk, melyik normalizálás szerepel. A PyTorch `MSELoss(reduction='mean')` az összes elemre átlagol, vagyis $\hat{L}/d$-t ad $(1/(Nd)$ skála). Ha az (1) a szerződés, a mean loss-t d-vel kell szorozni (vagy `reduction='sum'` és osztás N-nel). A különbség nem puszta terminológia: a gradiens d-szeres, ezért regularizáló tag vagy összehasonlított tanulási ráta mellett a képletet, a kódot és a gradiensskálát egységesíteni kell; a

mean-redukció a rekonstrukciós tagot $1/d$-vel skálázza, a regularizációs tagot nem, ezért a két tag aránya és a tanulási ráta hatása is megváltozik.

Szivárgás (leakage). A tanító, validációs vagy tesztelési folyamatban a teszt splithez tartozó tiszta target vagy a teszt zajrealizációjára vonatkozó információ bejut a modellparaméterekbe vagy a modellválasztásba. A denoising beállításban a legsúlyosabb szivárgás az, ha egy tiszta tesztkép tanítási targetként szerepel: ekkor a teszt nem a korrupció eltávolítását méri, hanem a targetek memorizálását.

29.2.3 A várható denoising cél

A denoising feladat valószínűségi megfogalmazása a várható négyzetes hiba minimalizálása. A feltételes várható érték és a feltételes eloszlás rendszerezett hátterét a [B01] tankönyv adja; az alábbi állítás és bizonyítás saját levezetés. Legyen X a tiszta kép véletlen vektor, $Y = c(X, \varepsilon)$ a korrupt megfigyelés. A cél az összes mérhető $g : \mathbb{R}^d \to \mathbb{R}^d$ függvény fölött:

$$L(g) = \mathbb{E}\big[\|X - g(Y)\|_2^2\big]. \tag{2}$$

Tétel (MSE-optimális denoiser). *Tegyük fel, hogy $\mathbb{E}\|X\|_2^2 < \infty$, és a versenyző becslők a véges kockázatú osztályból valók: $\mathbb{E}\|g(Y)\|_2^2 < \infty$ (az $\mathbb{E}\|X\|^2 < \infty$ önmagában nem garantálja, hogy minden mérhető $g(Y)$ négyzetesen integrálható; a kereszt-tag ortogonalitásához ez a feltétel kell). Ekkor az a g minimalizálja a (2) célfüggvényt, amelyre $g^*(y) = \mathbb{E}[X \mid Y = y]$ (a feltételes várható érték alkalmas regular verziója), és a minimum*

$$L(g^*) = \mathbb{E}\big[\|X - \mathbb{E}[X \mid Y]\|_2^2\big].$$

Bizonyítás. Legyen $m(Y) = \mathbb{E}[X \mid Y]$, és írjuk fel a különbséget:

$$X - g(Y) = (X - m(Y)) + (m(Y) - g(Y)).$$

A normanégyzetet kifejtve:

$$\|X - g(Y)\|^2 = \|X - m(Y)\|^2 + \|m(Y) - g(Y)\|^2 + 2(X - m(Y))^\top (m(Y) - g(Y)).$$

Vegyük mindkét oldal várható értékét. A kereszttag feltételes várható értékét kell megvizsgálni:

$$\mathbb{E}\big[(X - m(Y))^\top (m(Y) - g(Y))\big] = \mathbb{E}\Big[\mathbb{E}\big[(X - m(Y))^\top (m(Y) - g(Y)) \mid Y\big]\Big].$$

A feltételes várható érték tulajdonsága miatt az Y-tól függő $(m(Y) - g(Y))$ kiemelhető:

$$\mathbb{E}\big[(X - m(Y))^\top (m(Y) - g(Y)) \mid Y\big] = (m(Y) - g(Y))^\top \mathbb{E}[X - m(Y) \mid Y] = 0,$$

mert $\mathbb{E}[X - m(Y) \mid Y] = 0$ a feltételes várható érték tulajdonsága szerint. A másodmomentum-feltétel és a Cauchy–Schwarz-egyenlőtlenség biztosítja, hogy a várható értékek végesek és a kiemelés jogszerű. Így

$$L(g) = L(g^*) + \mathbb{E}\big[\|m(Y) - g(Y)\|_2^2\big] \geq L(g^*),$$

és egyenlőség pontosan akkor áll, ha $g(Y) = m(Y)$ majdnem mindenütt.

A tétel elméleti optimumot ad, de nem konstruktív: a $\mathbb{E}[X \mid Y]$ általában nem zárt alakú, ezért a gyakorlatban f_θ paraméteres közelítést használunk. Az (1) empirikus veszteség a (2) várható cél Monte Carlo-közelítése, ha a (x_i, y_i) párok független minták a közös korrupciós eloszlásból.

Additív Gauss-zaj esete. Legyen $Y = X + N$, ahol $N \sim \mathcal{N}(0, \sigma^2 I)$ független X-től, és a klipelést egyelőre elhagyjuk. A sűrűség:

$$p_{Y|X}(y \mid x) = \prod_{j=1}^{d} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(-\frac{(y_j - x_j)^2}{2\sigma^2}\Big).$$

A Bayes-szabállyal a posterior:

$$p_{X|Y}(x \mid y) \propto p_X(x) \exp\Big(-\frac{1}{2\sigma^2}\|y - x\|_2^2\Big),$$

ahol p_X a tiszta képek eloszlása. A Bayes-optimális denoiser tehát

$$g^*(y) = \mathbb{E}[X \mid Y = y] = \int x\, p_{X|Y}(x \mid y)\, dx.$$

Ez általában nem y és nem is lineáris függvénye y-nak. A célfüggvény a tétel szerint:

$$L(\theta) = \mathbb{E}_{X,N}[\|f_\theta(X + N) - X\|_2^2],$$

a klipeléses változatban pedig $Y = \mathrm{clip}(X + N, 0, 1)$ mellett ugyanez a képlet, de a Y eloszlása a klipelés (cenzorálás) miatt különbözik a tiszta Gauss-esettől; ez nem csonkolt normális, mert a 0 és 1 határon ponttömeg keletkezik.

29.2.4 Eredeti kézi kis példa

Legyen $d = 1$, $X \in \{0, 1\}$ egyenlő valószínűséggel, $N \sim \mathcal{N}(0, 1)$ független, és $Y = X + N$. A Bayes-optimális denoiser a posterior várható érték:

$$g^*(y) = \frac{0{,}5\, p_N(y - 1)}{0{,}5\, p_N(y) + 0{,}5\, p_N(y - 1)} = \frac{p_N(y - 1)}{p_N(y) + p_N(y - 1)},$$

ahol p_N a standard normális sűrűség. A $y = 0{,}7$ megfigyelés az 1-hez van közelebb (a távolságok 0,3 és 0,7): $p_N(0{,}7) \approx 0{,}3123$ ($X = 0$ esetén) és $p_N(-0{,}3) \approx 0{,}3814$ ($X = 1$ esetén), így

$$g^*(0{,}7) = \frac{0{,}3814}{0{,}3123 + 0{,}3814} \approx 0{,}5498.$$

Az identikus leképezés $\hat{x} = 0{,}7$-et adna, a triviális konstans $\hat{x} = 0{,}5$; az optimális becslés 0,5498, ami a kettő között van: a $\sigma = 1$ zaj mellett a megfigyelés alig informatív, ezért a becslést a prior várható értéke, 0,5 felé húzza, de a megfigyelés felé is elmozdítja. A feltételes minimális négyzetes hiba $y = 0{,}7$ esetén a posterior varianciája, $p(1 - p) \approx 0{,}5498 \cdot 0{,}4502 \approx 0{,}2475$, tehát nem nulla. A tétel garantálja, hogy semmilyen más g-vel nem érhető el kisebb várható négyzetes hiba ennél a zajmodellnél.

29.2.5 Labor: zajos–tiszta párok szintetikus alakzatokon

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor kizárólag a GlyphLab saját, procedurális adatát használja. A cél: additív Gauss-korrupcióval párokat készíteni, a teszt zajrealizációját külön seeddel generálni, és a szivárgást szisztematikusan ellenőrizni. A modell tanítását nem végezzük el ebben a laborban; a párok előállítása és a baseline kiértékelés az átadandó eredmény.

```python
import csv
import numpy as np
from pathlib import Path

DATA = Path("data")
SIGMA = 0.25
TRAIN_SEED = 20261020
VAL_SEED   = 20261021
TEST_SEED  = 20261022

def read_split_indices(metadata_path: Path) -> dict[str, np.ndarray]:
    """A metadata.csv split oszlopából indexeket olvas be int64-ként."""
    idx: dict[str, list[int]] = {"train": [], "val": [], "test": []}
    with metadata_path.open(newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            idx[row["split"]].append(int(row["index"]))
    return {k: np.asarray(v, dtype=np.int64) for k, v in idx.items()}

def add_gaussian_noise(
    clean: np.ndarray, sigma: float, seed: int
) -> tuple[np.ndarray, np.ndarray]:
    """Additív Gauss-zaj, majd elemenkénti [0,1] klipelés.

    clean: (M, C, H, W) float32, [0,1]
    Vissza: (noisy, noise), mindkettő float32, noisy [0,1].
    """
    rng = np.random.default_rng(seed)
    noise = rng.normal(0.0, sigma, clean.shape).astype(np.float32)
    noisy = np.clip(clean + noise, 0.0, 1.0).astype(np.float32)
    return noisy, noise

# 1. Bemenet és split
images = np.load(DATA / "images.npy")          # (360,1,24,24) float32
meta_idx = read_split_indices(DATA / "metadata.csv")
train = images[meta_idx["train"]]
val   = images[meta_idx["val"]]
test  = images[meta_idx["test"]]

# 2. Korrupció: train/val/test külön zajseeddel
train_noisy, train_noise = add_gaussian_noise(train, SIGMA, TRAIN_SEED)
val_noisy,   val_noise   = add_gaussian_noise(val,   SIGMA, VAL_SEED)
test_noisy,  test_noise  = add_gaussian_noise(test,  SIGMA, TEST_SEED)

# 3. Alak- és tartománykontroll: mindegyik zajos tömb a saját tiszta párjához hasonlít
for name, noisy, clean in [("train_noisy", train_noisy, train),
                           ("val_noisy", val_noisy, val),
                           ("test_noisy", test_noisy, test)]:
    assert noisy.shape == clean.shape, name
    assert np.isfinite(noisy).all(), name
    assert 0.0 <= noisy.min() and noisy.max() <= 1.0, name

# 4. Leakage-ellenőrzés: az indexhalmazok páronként diszjunktak,
#    és a tiszta képek tartalma sem ismétlődik split között.
split_sets = {k: set(meta_idx[k]) for k in ("train", "val", "test")}
assert split_sets["train"].isdisjoint(split_sets["val"])
assert split_sets["train"].isdisjoint(split_sets["test"])
assert split_sets["val"].isdisjoint(split_sets["test"])

def image_keys(arr):
    return {img.tobytes() for img in arr}

assert image_keys(train).isdisjoint(image_keys(val))
assert image_keys(train).isdisjoint(image_keys(test))
```

```
assert image_keys(val).isdisjoint(image_keys(test))

# 5. Baseline-ok, per-pixel MSE-ként
def mse(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.mean((a - b) ** 2))

identity_mse = mse(test_noisy, test)   # a zajos input mint predikció
const_pred = np.full_like(test, np.mean(train))  # tanítási átlag konstansként
const_mse = mse(const_pred, test)
noise_energy = float(np.mean(test_noise ** 2))   # a vizsgált zaj átlagos energiája

print(f"shapes: train_noisy={train_noisy.shape}, test_noisy={test_noisy.shape}")
print(f"dtype: {train_noisy.dtype}, device: CPU (NumPy)")
print(f"identity baseline MSE={identity_mse:.6f}")
print(f"const baseline MSE={const_mse:.6f}")
print(f"noise energy={noise_energy:.6f}")

# A szövegben kimondott várakozások ellenőrzése
assert identity_mse < SIGMA ** 2          # a klipelés csökkenti a hibát
assert abs(noise_energy - SIGMA ** 2) < 2e-3   # 0,0625 körül
```

A futáskor a `data/images.npy` és `data/metadata.csv` fájloknak a munkakönyvtár `data` almappájában kell lenniük. A GlyphLab 360 képéből a split 240 train, 60 val és 60 test kép. Futtatási parancs:

```
python denoising_pairs.py
```

Ellenőrzés és értelmezés. Az identikus baseline $y \mapsto y$ per-pixel MSE-je $\sigma^2 = 0{,}0625$ volna klipelés nélkül. A $[0,1]$-re vetítés azonban 1-Lipschitz, ezért $|\operatorname{clip}(x+\varepsilon) - x| \le |\varepsilon|$ minden $x \in [0,1]$ esetén: a klipelés soha nem növeli a hibát, és ott csökkenti, ahol a zaj a határon kívülre tolja a pixelt. A GlyphLab háttere sötét (a pixelek túlnyomó része 0,2 alatt van), ezért a 0-nál való klipelés gyakori, és a mért identikus MSE 0,0373 jóval a zajvariancia alatt marad. A konstans baseline a tiszta képek varianciáját méri: ha a képek intenzitása erősen változó, ez nagyobb σ^2-nél; ha a képek közel konstansak, kisebb. A zajenergia a ténylegesen hozzáadott ε négyzetes átlaga, és csak a klipelés nélküli zajt méri. A denoisernek mindkét baseline alá kell mennie; ezen az adaton a szigorúbb feltétel a konstans baseline alá kerülés, ezért a validációs MSE-nek a mért 0,0143 alatt kell lennie.

A fenti kód a mellékelt GlyphLab-adaton lefutott (Python 3.12.3, NumPy 2.5.4, CPU; train/val/test zajseed 20261020/20261021/20261022; $\sigma = 0{,}25$). Mért kimenet:

```
shapes: train_noisy=(240, 1, 24, 24), test_noisy=(60, 1, 24, 24)
dtype: float32, device: CPU (NumPy)
identity baseline MSE=0.037279
const baseline MSE=0.014263
noise energy=0.062331
```

Az identikus baseline MSE-je a klipelés miatt 0,0373, jóval a zajvariancia 0,0625 alatt marad. A konstans baseline ezen az adaton 0,0143, azaz alacsonyabb az identikus baseline-nál: a GlyphLab tiszta képei nagy kiterjedésű, közel konstans háttérrel rendelkeznek, ezért a tanítási átlag mint predikció már önmagában kicsi hibát ad. A denoisernek tehát nem az identikus, hanem a konstans baseline-t is alul kell múlnia; a validációs MSE-nek a 0,0143 alatt kell lennie. Ez a feltétel szükséges, de nem elégséges: a rekonstrukciós MSE nem méri a downstream reprezentáció hasznosságát. A zajenergia 0,0623, ami megfelel a hozzáadott zaj $\sigma^2 = 0{,}0625$ értékének (a mintaátlag kis eltérése miatt).

29.2.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a *corruption*, *denoising* és *leakage* fogalmát a denoising tanítási protokollban. Magyarázza el, miért vezetne az $f_\theta(y) = y$ identikus leképezéshez az a hibás cél, amely a zajos képet önmagával hasonlítja. A válaszban adjon egy kis ellenpéldát: legyen $y = x + \varepsilon$, és mutassa meg, hogy az $\|f_\theta(y) - y\|^2$ minimalizálója az identitás.

F2 (levezetési, nehézség 2). Legyen $X \in \{1, 3\}$, $P(X = 1) = 0{,}4$, $P(X = 3) = 0{,}6$, és $Y = X + N$, ahol $N \sim \mathcal{N}(0,\ 0{,}5^2)$ független. Vezesse le a Bayes-optimális denoiser képletét, számítsa ki $g^*(2{,}1)$ értékét, és adja meg a feltételes minimális négyzetes hibát $Y = 2{,}1$ esetén. A megoldásban a közös normáló konstans kiesését mutassa ki.

F3 (implementációs, nehézség 2). Írjon Python-függvényt, amely a GlyphLab `data/images.npy` tömbjéből készít zajos–tiszta párokat: a train, val, test splithez külön seedet használ, additív Gauss-zajt ad $\sigma = 0{,}2$ mellett, és a zajos képet a $[0, 1]$ intervallumba klipeli. A függvény ellenőrizze, hogy a kimeneti zajos tömb alakja megegyezik a bemenettel, minden érték véges, és a test split tiszta célja nem szerepel a train targetek között.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet hibás:

```
train_clean = images[train_idx]
test_clean  = images[test_idx]
train_noisy = add_gaussian_noise(train_clean, SIGMA, TRAIN_SEED)[0]
test_noisy  = add_gaussian_noise(test_clean,  SIGMA, TEST_SEED)[0]
# A teszt split is bekerül a tanításba: a modell a teszt célokat is látja.
X = np.concatenate([train_noisy, test_noisy], axis=0)
target = np.concatenate([train_clean, test_clean], axis=0)  # ???
model = train_denoiser(X, target)   # PyTorch-ciklus, lásd a megoldást
```

Lokalizálja a hibát, adja meg a következményét, és javítsa ki úgy, hogy a tanítás után a teszt splitre vonatkozó MSE független maradjon. A javított kódban mutassa be az ellenőrzést is.

F5 (kísérlettervezési, nehézség 3). Tervezzen kísérletet annak mérésére, hogy egy f_θ denoiser hogyan általánosít egy új, a tanításnál nem látott zajszintre. A tanítás $\sigma = 0{,}1$ zajjal történik, a teszt $\sigma = 0{,}3$ és $\sigma = 0{,}5$ mellett is futtatandó. Adja meg a megfigyelési egységet, a baseline-okat, a metrikát, a split- és seed-szabályt, valamint a következtetés érvényességi határát.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

29.2.7 Megoldások és indoklás

F1. A *corruption* a $c(x, \varepsilon)$ leképezés, amely tiszta képből zajos megfigyelést ad; a *denoising* a zajos megfigyelésből a tiszta kép becslése; a *leakage* a teszt splithez tartozó tiszta cél vagy zajrealizáció-információ bejutása a tanítási folyamatba. Ha a cél a zajos kép önmaga, azaz $\hat{L}(\theta) = \frac{1}{N}\sum_i \|f_\theta(y_i) - y_i\|^2$, akkor a cél minimális értéke nulla, és a $f_\theta(y) = y$ identitás eléri. Ezért a modell nem tanul zajtalanítást, csak identitást. Numerikus ellenpélda: legyen $d = 1$, $x = 0{,}5$, $\varepsilon = 0{,}1$, így $y = 0{,}6$; a $\|f_\theta(y) - y\|^2$ veszteség az $f_\theta(0{,}6) = 0{,}6$ értéknél nulla, minden más értéknél pozitív, tehát a minimumot az identitás adja. Általában is az $f_\theta(y) = y$ identitás a veszteség minimuma (nulla), függetlenül a zaj mértékétől.

F2. A posterior a Bayes-szabállyal, a közös p_N konstanssal:

$$P(X = 1 \mid Y = y) = \frac{0{,}4\, p_N(y - 1)}{0{,}4\, p_N(y - 1) + 0{,}6\, p_N(y - 3)}, \qquad P(X = 3 \mid Y = y) = 1 - P(X = 1 \mid Y = y).$$

A Bayes-optimális denoiser $g^*(2{,}1) = 1 \cdot P(X=1 \mid Y=2{,}1) + 3 \cdot P(X=3 \mid Y=2{,}1)$. A normális sűrűség közös $\frac{1}{0{,}5\sqrt{2\pi}}$ szorzója kiesik; csak az exponenciális tagokat kell számolni:

$$p_N(1{,}1) \propto e^{-1{,}1^2/(2\cdot 0{,}25)} = e^{-2{,}42} \approx 0{,}0889,$$

$$p_N(-0{,}9) \propto e^{-0{,}9^2/(0{,}5)} = e^{-1{,}62} \approx 0{,}198.$$

Súlyozatlanul a súlyok $0{,}4 \cdot 0{,}0889 = 0{,}03556$ és $0{,}6 \cdot 0{,}198 = 0{,}1188$; a normalizálás után

$$P(X=1 \mid Y=2{,}1) = \frac{0{,}03556}{0{,}03556 + 0{,}1188} \approx 0{,}2304, \qquad P(X=3 \mid Y=2{,}1) \approx 0{,}7696.$$

Így

$$g^*(2{,}1) \approx 0{,}2304 + 3 \cdot 0{,}7696 = 2{,}539.$$

A feltételes minimális négyzetes hiba a posterior variancia:

$$(1-2{,}539)^2 \cdot 0{,}2304 + (3-2{,}539)^2 \cdot 0{,}7696 \approx 0{,}546 + 0{,}164 = 0{,}709.$$

Alakellenőrzés: a kimenet skalár, a posterior valószínűségek összege egy, a variancia nemnegatív.

F3. A megoldás a labor kódjának megfelelő függvény:

```
def make_noisy_pairs(images, meta, sigma, train_seed, val_seed, test_seed):
    idx = read_split_indices(meta)
    train_clean = images[idx["train"]]
    val_clean   = images[idx["val"]]
    test_clean  = images[idx["test"]]

    def noisy(clean, seed):
        rng = np.random.default_rng(seed)
        n = rng.normal(0.0, sigma, clean.shape).astype(np.float32)
        return np.clip(clean + n, 0.0, 1.0).astype(np.float32), n.astype(np.float32)

    train_noisy, train_noise = noisy(train_clean, train_seed)
    val_noisy,   val_noise   = noisy(val_clean,   val_seed)
    test_noisy,  test_noise  = noisy(test_clean,  test_seed)
    for noise in (train_noise, val_noise, test_noise):
        assert np.isclose(np.mean(noise ** 2), sigma ** 2, rtol=0.1)
    for noisy, clean in [(train_noisy, train_clean),
                         (val_noisy, val_clean),
                         (test_noisy, test_clean)]:
        assert noisy.shape == clean.shape
        assert np.isfinite(noisy).all()
        assert noisy.min() >= 0.0 and noisy.max() <= 1.0
    split_sets = {k: set(idx[k]) for k in ("train", "val", "test")}
    assert split_sets["train"].isdisjoint(split_sets["val"])
    assert split_sets["train"].isdisjoint(split_sets["test"])
    assert split_sets["val"].isdisjoint(split_sets["test"])
    keys = {k: {img.tobytes() for img in arr}
            for k, arr in [("train", train_clean), ("val", val_clean),
                           ("test", test_clean)]}
    assert keys["train"].isdisjoint(keys["val"])
    assert keys["train"].isdisjoint(keys["test"])
    assert keys["val"].isdisjoint(keys["test"])
    return (train_noisy, train_clean), (val_noisy, val_clean), (test_noisy, test_clean)
```

Használat: a `meta` a `data/metadata.csv` elérési útja; a `read_split_indices` csak CSV-t olvas. A tolerancia a klipelésnél nem szükséges, mert a `np.clip` pontosan a megadott határokat adja; a kimeneti tömbök alakját a tiszta pár alakjához hasonlítjuk, nem kézi konstansokhoz.

F4. A hiba a bemenet és a cél összeállításában van: az `X` és a `target` a teszt split képeit is tartalmazza, ezért a modell a teszt tiszta céljait is tanulja. Következmény: a teszt MSE a tanított mintákon mért hibát is tartalmazza, tehát optimistán alacsony lesz, és nem a zajtalanítás generalizációját méri. A javítás: a tanítás bemenete és célja kizárólag a train splitből áll, a teszt párt csak egyszer, a tanítás után értékeljük. A labor PyTorch-ciklusát használó, önálló változat:

```
import torch
import torch.nn as nn

def train_denoiser(noisy, clean, seed=20261020, steps=200, lr=1e-3):
    """Csak a kapott párokon tanít; a hívó felel azért, hogy train párok legyenek."""
    torch.manual_seed(seed)
    model = nn.Sequential(
        nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(),
        nn.Conv2d(8, 1, 3, padding=1))
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    X = torch.as_tensor(noisy, dtype=torch.float32)
    Y = torch.as_tensor(clean, dtype=torch.float32)
    loss_fn = nn.MSELoss()
    for _ in range(steps):
        opt.zero_grad()
        loss_fn(model(X), Y).backward()
        opt.step()
    return model

@torch.no_grad()
def denoise_mse(model, noisy, clean):
    model.eval()
    pred = model(torch.as_tensor(noisy, dtype=torch.float32)).clamp(0.0, 1.0)
    target = torch.as_tensor(clean, dtype=torch.float32)
    return float(torch.mean((pred - target) ** 2))

train_clean = images[train_idx]
test_clean  = images[test_idx]
train_noisy = add_gaussian_noise(train_clean, SIGMA, TRAIN_SEED)[0]
test_noisy  = add_gaussian_noise(test_clean,  SIGMA, TEST_SEED)[0]

assert set(train_idx).isdisjoint(set(test_idx))
assert not (set(img.tobytes() for img in train_clean)
            & set(img.tobytes() for img in test_clean))

model = train_denoiser(train_noisy, train_clean)       # csak train pár
test_mse = denoise_mse(model, test_noisy, test_clean)  # egyszer, a végén
```

Az ellenőrzés két dolgot rögzít: a split indexhalmazai diszjunktak, és a tiszta képek tartalma sem ismétlődik split között, tehát a teszt cél nem kerülhetett a tanításba. A `test_mse` értéket csak egyszer, a tanítás után számítjuk.

F5. Egy lehetséges terv: a megfigyelési egység a GlyphLab `scene_id`-je, mert a két nézet ugyanannak a jelenetnek a zajos változata; a split scene-szinten rétegzett. A modell tanítása $\sigma = 0{,}1$ zajjal, a validáció $\sigma = 0{,}1$ mellett választja a legjobb checkpointot. A teszteléshez a tiszta test splitet három külön zajrealizációval korrumpáljuk: $\sigma = 0{,}1$, $0{,}3$, $0{,}5$, mindegyikhez külön seedet használva. Baseline-ok: az identikus $y \mapsto y$ és a tanítási átlag konstans prediktor, minden zajszinten; a metrika a per-pixel MSE és a PSNR $= 10 \log_{10}(1/\mathrm{MSE})$. A kiértékelést a modell rögzítése után egyszer futtatjuk. Érvényességi határ: az eredmény csak a választott zajcsaládra és a szintetikus GlyphLab adatra vonatkozik; a klipelés miatt a magas σ esetén a tényleges zaj torzul, ezért a $\sigma = 0{,}5$ eredmény nem ekvivalens egy valós érzékelő zajával.

29.2.8 Tipikus tévedés ellenpéldával

A leggyakoribb hiba az, hogy a tiszta tesztkép tanítási targetként megjelenik. Tegyük fel, hogy a 60 tesztképből 10 tiszta targetet véletlenül a train célhoz csatolunk. Egy olyan modell, amely a train párokat memorizálja, a teszt ezen 10 elemére nulla közeli MSE-t adhat, míg a maradék 50 elemre a denoising teljesítménye gyenge. Az átlagos teszt MSE így optimistán alacsony lesz, de nem a zajtalanítás minőségét méri. A helyes protokoll: a train generálásakor kizárólag a train split képeit használjuk, a test tiszta célját pedig csak a végső kiértékelésben töltjük be.

29.2.9 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrzendő: a target mindig a tiszta kép, a veszteség redukciója explicit módon rögzített, a zajrealizációk seedjei a splitenként elkülönülnek. A 28.2. Sziámi és triplet cél triplet céljával párhuzamos: ott a pozitív pár közelítése, itt a zajos–tiszta pár közvetlen rekonstrukciója a cél; margin és negatív pár hiányában a denoising cél triviális konstans baseline-t enged meg, de a helyes cél globális optimuma a feltételes várható érték, nem a konstans. Ha viszont a cél tévesen maga a zajos kép, a triviális megoldás az identitás. A 25.2. Multilabel és hierarchikus cél megfigyelési maszkjának tanulsága: ha tudjuk, mely pixelek korruptak, a loss súlyozható; itt minden pixel megfigyelt, ezért a maszk nem szükséges. A 17.2. AlexNet és a kísérleti kontextus kísérleti kontextusa emlékeztet: a zajmodell és a kiértékelési feltételek rögzítése nélkül az eredmény nem általánosítható.

29.2.10 Rövid összefoglaló

A denoising cél a zajos y megfigyelésből a tiszta x kép becslése, a korrupciós leképezés $c(x, \varepsilon)$ rögzítése mellett. A várható négyzetes hibát a feltételes várható érték $\mathbb{E}[X \mid Y]$ minimalizálja; a tétel feltétele $\mathbb{E}\|X\|^2 < \infty$. Az additív Gauss-zaj modelljében a posterior a tiszta kép priorjától függ, ezért az optimum általában nem zárt alakú. Az empirikus veszteség az átlagos rekonstrukciós hiba, per-pixel MSE-re az elemszámmal normalizálva. A labor a GlyphLab adatra külön train/val/test zajseeddel állít elő párokat; a tiszta tesztcél tanítási targetként való megjelenése szivárgás, és a teszt MSE-t torzítja.

29.2.11 Következő alkalmazás

A 29.3. Regularizált latent reprezentáció egység a regularizált latent reprezentációt tárgyalja: ott a denoising célhoz hasonló rekonstrukciós veszteség mellé egy latent-térbeli büntetőtag kerül, amely a reprezentáció eloszlását szabályozza. A most bevezetett zajos–tiszta páros szerződés és a szivárgásvizsgálat ott is kötelező kiindulópont.

29.2.12 Felhasznált források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. A feltételes várható érték és a feltételes eloszlás háttérrendszerezése; a fejezet állítását és bizonyítását önállóan vezeti le.
- Vincent, P., Larochelle, H., Bengio, Y. és Manzagol, P.-A. (2008). *Extracting and Composing Robust Features with Denoising Autoencoders*. Proceedings of the 25th International Conference on Machine Learning (ICML), 1096–1103. A denoising autoencoder tanítási elvének forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

29.3 Regularizált latent reprezentáció

29.3.1 Cél, előismeret és induló kérdés

A 29.2. Denoising és korrupciós modell egységben az autoencodert egy konkrét szerződésként írtuk fel: a bemenet a zajos x', a tanulási cél a tiszta x, az encoder–decoder páros pedig a rekonstrukciós hibát csökkenti. Ebben az alfejezetben a rekonstrukció mögötti latent (lappangó, rejtett) reprezentációt vizsgáljuk. A szükséges előismeret a 25.3. Kis adat és feature-alapú baseline lineáris pixel-baseline gondolata, a 28.3. Retrieval-értékelés és adatfelosztás csoportos kiértékelési fegyelme és a 17.3. VGG és kis kernelek kompozíciója kapacitás/recepciós mező megkülönböztetése.

Induló kérdés: ha egy autoencoder empirikus rekonstrukciós hibája nulla, következik-e ebből, hogy a $z = f(x)$ kód „szemantikailag” használható, például egy lineáris osztályozó fejjel jól leolvasható? A válasz nem: a triviális identitásleképezés tökéletesen rekonstruálhat, miközben semmilyen új, downstream feladat számára kedvező átrendezést nem végez.

29.3.2 Autoencoder, kapacitás és identitásleképezés

A fejezet jelölésében a bemenet $x \in \mathbb{R}^d$. A függelék A. szakaszában leírt GlyphLab-kép $C = 1$, $H = W = 24$ méretű NCHW-tensor, ezért a vektorizált bemenet dimenziója $d = 576$. Egy batch $X \in \mathbb{R}^{N \times d}$ soronként egy-egy megfigyelést tartalmaz.

D1 (autoencoder és rekonstrukciós loss). Az encoder $f_W : \mathbb{R}^d \to \mathbb{R}^b$, a decoder $g_V : \mathbb{R}^b \to \mathbb{R}^d$, ahol W, V a tanulandó paraméterek. A latent kód

$$z = f_W(x) \in \mathbb{R}^b, \qquad \hat{x} = g_V(z) \in \mathbb{R}^d.$$

Az empirikus rekonstrukciós veszteség elemenkénti átlagolással:

$$L_{\text{rec}}(f_W, g_V; X) = \frac{1}{Nd} \sum_{n=1}^{N} \sum_{j=1}^{d} \left(x_{n,j} - g_V(f_W(x_n))_j \right)^2 .$$

Az $1/(Nd)$-es normálásnak itt nincs valószínűségi jelentése; pusztán az átlagos négyzetes eltérést adja batch- és pixelméreten.

D2 (bottleneck). Bottlenecknek vagy szűkületnek azt nevezzük, amikor $b < d$. A kód tehát kisebb dimenziójú, mint a bemenet. Ha $b = d$, akkor a modell magában foglalhatja a triviális megoldást:

$$f_W(x) = x, \qquad g_V(z) = z,$$

amelyre minden bemeneten $L_{\text{rec}} = 0$. Ez a triviális identitásleképezés. Fontos, hogy a $b < d$ feltétel sem oldja meg automatikusan a tanulási problémát: alacsony dimenziós altéren is létezhet olyan kód, amely a downstream feladat szempontjából rossz, vagy éppen összeomolhat; a latent kód összeomlását a következő definíció rögzíti.

D3 (latent collapse). Azt mondjuk, hogy a j-edik latent koordináta összeomlott egy adott $Z \in \mathbb{R}^{N \times b}$ mátrixon, ha a koordinátánkénti szórása közel nulla. A j-edik koordinátára

$$\mathrm{std}_j(Z) = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}\left(z_{n,j}-\bar{z}_j\right)^2}, \qquad \bar{z}_j = \frac{1}{N}\sum_{n=1}^{N} z_{n,j}.$$

Teljes collapse esetén minden koordináta szórása eltűnik, tehát a decoder minden inputhoz lényegében ugyanazt az értéket kapja, és a rekonstrukció csak egy konstans képet tud előállítani. Részleges collapse esetén csak néhány koordináta hal el: egy dead unit (holt egység) nulla vagy konstans, a többi koordináta viszi az információt.

29.3.3 A rekonstrukcióból nem következik szemantikai tudás

A „szemantikai tudást” itt nem általános fogalomként definiáljuk, hanem egy konkrét, mérhető downstream feladatra cseréljük. Rögzítünk egy downstream feladatot y-nal, és egy lineáris leolvasó családot:

$$q(z) = Az + c, \qquad A \in \mathbb{R}^{K\times b},\ c \in \mathbb{R}^K,$$

többosztályú logitokkal, vagy bináris esetben $q(z) = a^\top z + c$ előjelfüggvénnyel. A „jobb reprezentáció” ekkor azt jelenti: a rögzített q-családdal, azonos adatfelosztás mellett nagyobb downstream pontosság érhető el.

1. állítás (identitásleképezés mint ellenpélda, saját bizonyítás). Legyen a bemenetek halmaza

$$X_0 = \{(1,1),(-1,-1),(1,-1),(-1,1)\} \subset \mathbb{R}^2,$$

minden pont egyenletes valószínűséggel. Legyen a downstream címke

$$y = 1[x_1 x_2 > 0],$$

tehát $y(1,1) = y(-1,-1) = 1$, a másik két ponté 0. Tekintsük a $b = d = 2$ triviális identitás-autoencodert: $f(x) = x$, $g(z) = z$. Ekkor

$$L_{\mathrm{rec}} = 0.$$

Ugyanakkor ezen a reprezentáción egyetlen lineáris osztályozó sem érhet el 4/4 downstream pontosságot. Mivel négy egyenlő valószínűségű pont van, a pontosság $k/4$ alakú, tehát a lehetséges értékek 0, 1/4, 1/2, 3/4 és 1.

Bizonyítás. Legyen a lineáris leolvasó $q(x) = \alpha x_1 + \beta x_2 + \gamma$, és döntsünk $q > 0$ esetén 1, egyébként 0 mellett. Ha mindkét pozitív mintát jól osztályoznánk, akkor

$$\alpha + \beta + \gamma > 0, \qquad -\alpha - \beta + \gamma > 0.$$

A kettőt összeadva $2\gamma > 0$, azaz $\gamma > 0$. Ha mindkét negatív mintát is jól osztályoznánk, akkor

$$\alpha - \beta + \gamma \le 0, \qquad -\alpha + \beta + \gamma \le 0.$$

Ezeket összeadva $2\gamma \le 0$, azaz $\gamma \le 0$. Ez ellentmond $\gamma > 0$-nak, tehát a négy pont nem osztályozható tökéletesen. A 3/4 pontosság éles: az $\alpha = 10$, $\beta = 10$, $\gamma = -0{,}1$ választással

$$q(1,1) = 19{,}9 > 0, \quad q(-1,-1) = -20{,}1 \le 0, \quad q(1,-1) = -0{,}1 \le 0, \quad q(-1,1) = -0{,}1 \le 0,$$

ami három helyes döntést ad. $\square$

A kontraszt kedvéért vegyük ugyanezen a négy ponton a következő, szintén $b = 2$ dimenziós reprezentációt:

$$f_1(x_1, x_2) = (x_1 x_2, x_1), \qquad g_1(u, v) = \left(v, \frac{u}{v}\right),$$

ahol g_1 értelmezési tartománya $v \neq 0$; a definíciót csak az X_0 négy pontján használjuk, ahol $v = x_1 \in \{-1, 1\}$. A négy ponton tehát az osztás értelmes, és $g_1(f_1(x)) = x$ mind a négy esetben. Ennek a modellnek is

$$L_{\text{rec}} = 0.$$

A $q_1(u, v) = u$ lineáris leolvasó viszont mind a négy pontot jól osztályozza, mert $x_1 x_2 > 0$ pontosan akkor, ha $u > 0$. Az 1. állítás következménye tehát nem az, hogy a kis rekonstrukciós hiba „rossz”, hanem az, hogy önmagában nem bizonyítja a downstream hasznosságot. A két modell azonos kapacitással és azonos rekonstrukciós hibával is eltérő downstream viselkedést adhat.

A bizonyításnak van egy gyakorlati geometriai üzenete is: az identitás encoder megtartja a bemenet koordinátarendszerét. A négy pontot ebben a koordinátarendszerben egyetlen lineáris hipersík sem választja szét (a négy pont nem lineárisan szeparálható), ezért a lineáris fej nem tudja leolvasni a címkét. Egy alkalmas nemlineáris encoder viszont kiszámíthat egy olyan új koordinátát, amelyben a feladat lineárissá válik. A rekonstrukciós cél ehhez semmilyen külső jelet nem ad; lehet szerencsés, de nem garantált.

29.3.4 Ritka és kontraktív regularizálás áttekintése

Mivel a rekonstrukciós loss nem bünteti automatikusan a triviális vagy kedvezőtlen latent geometriát, a tanítási célhoz pótlólagos regularizáló tagot adhatunk. Két gyakori család:

Sparse regularizálás. Az aktivációk átlagos ℓ_1-normáját büntetjük:

$$L_{\text{sparse}} = L_{\text{rec}} + \frac{\lambda}{Nb} \sum_{n=1}^{N} \sum_{j=1}^{b} |z_{n,j}| = L_{\text{rec}} + \frac{\lambda}{Nb} \sum_{n=1}^{N} \|z_n\|_1.$$

A cél nem az, hogy minden kód nulla legyen, hanem hogy egyszerre kevés koordináta legyen aktív. Sztochasztikus gradiens alatt az ℓ_1-tag általában ritkító irányba tol, de egzakt nullákat nem feltétlenül állít elő; ez heurisztikus hatás, nem tétel. A λ értékét a rekonstrukcióval együtt kell ellenőrizni.

Contractive regularizálás. Az encoder Jacobi-mátrixának Frobenius-normáját büntetjük (a kontraktív autoencoder ötlete Rifai és mtsai, 2011; a Jacobi-mátrix és a Frobenius-norma jelölésének hátterét a [B01] tankönyv adja):

$$L_{\text{contractive}} = L_{\text{rec}} + \frac{\lambda}{Nb} \sum_{n=1}^{N} \left\| J_{f_W}(x_n) \right\|_F^2 .$$

Itt $J_{f_W}(x_n) \in \mathbb{R}^{b \times d}$, mert a sorok a b kimeneti koordinátának, az oszlopok a d bemeneti koordinátának felelnek meg. A tag azt méri, mennyire változik a kód a bemenet kis megváltoztatására. A $\lambda/(Nb)$ osztó a Frobenius-normát a b latent koordinátára átlagolja: a norma a d bemeneti irányra összegez, majd a b kimeneti koordinátán osztunk. A d-vel való további osztás a tipikus parciális derivált négyzetét adná, ami más regularizációs erőt jelentene; a λ értékét tehát ezzel a normalizálással együtt kell érteni. A triviális identitáson $J = I_d$, ezért $\|I_d\|_F^2 = d$, tehát ez a regularizáló közvetlenül drágítja az identitásleképezést. Fontos korlát: ez lokális simasági heurisztika, nem általános robusztussági vagy azonosíthatósági tétel.

29.3.5 Latent geometria és downstream kontroll

A latent térben a kódok elhelyezkedése, a latent geometry, nem automatikusan rendezett. Például a 28.1. Embedding, távolság és normalizálás egység L2-normalizálása és a 28.2. Sziámi és triplet cél triplet célja explicit egységgömb-kényszert ad; egy sima MSE-autoencoder latent terében ilyen kényszer nincs. Ezért a következő kontrollokat érdemes külön kezelni:

1. **Rekonstrukciós kontroll:** L_{rec} a tanító- és validációs halmazon.
2. **Capacity/collapse kontroll:** minden latent koordinátára $\text{std}_j(Z)$, nemcsak a teljes tensor egyetlen szórása.
3. **Downstream kontroll:** előre rögzített lineáris fej a befagyasztott encoder kódján, ugyanazzal a tanító/validációs felosztással, mint a felügyelt baseline.

A downstream kontroll azért nem bizonyít általános reprezentációminőséget, mert csak egyetlen címkére és egyetlen lineáris családra vonatkozik. Egy másik downstream feladat más rendezést igényelhet.

29.3.6 Labor: két latentméret és előre rögzített downstream kontroll

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor kérdése az, hogy a GlyphLab orientációfeladatán a $b = 4$ és $b = 16$ latentméretű autoencoder hogyan viselkedik rekonstrukcióban és befagyasztott lineáris downstream pontosságban. A protokollt a futtatás előtt rögzítjük:

- Adat: `lab/data/images.npy`, `labels.npy`, `metadata.csv`; a splitet a `metadata.csv` `split` oszlopa adja, a függetlenségi egység a `scene_id`, tehát nem keverjük a nézeteket a felosztások között. A `metadata.csv` oszlopai: `index` (az `images.npy` sorindexe, nem a CSV sorszáma), `scene_id`, `view_id` (0 vagy 1), `label`, `split` (`train`/`val`/`test`), `xmin`, `ymin`, `xmax`, `ymax`; a szkript a fejlécet és az index-lefedést ellenőrzi.
- Előfeldolgozás: NCHW-ből 576 dimenziós float32 vektor. A GlyphLab pixelértékei eleve a $[0, 1]$ tartományban vannak, ezért nincs további normalizálás; a szkript a tartományt betöltés után ellenőrzi.
- Modell: encoder `Linear(576,64)`, ReLU, `Linear(64,b)`; decoder `Linear(b,64)`, ReLU, `Linear(64,576)`. Csak b változik, a hidden szélesség és az inicializálás receptje nem.

- Tanítás: Adam, lr $= 10^{-3}$, batch méret 48, 15 epoch, seed `20261020`.
- Rekonstrukciós metrika: elemenkénti MSE a validációs halmazon.
- Downstream kontroll: a tanított encoder befagyasztása után a tanító kódokon `Linear(b,3)` fejet tanítunk cross-entropyval, `torch.manual_seed(20261020)`, Adam, lr $= 10^{-2}$, 30 teljes batch lépés; validációs pontosságot jelentünk. A fej mindkét ágban a tanítóhalmaz oszloponkénti átlagával és szórásával standardizált jellemzőkön tanul, hogy a pixel- és a latent skála ne befolyásolja az optimalizálást; az azonos lr és lépésszám így azonos költségvetést ad.
- Baseline: ugyanez a downstream fej a nyers 576 dimenziós pixelvektoron, azonos standardizálással és lépésszámmal.

A kód a mellékelt GlyphLab-adaton lefutott (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, float32, seed 20261020, egy szál); a mért kimenetet a futtatási parancs utáni táblázat tartalmazza.

```
# lab/n115_latent_probe.py
import csv
from pathlib import Path
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

ROOT = Path("lab/data")
SEED = 20261020
D = 24 * 24
torch.manual_seed(SEED)
np.random.seed(SEED)
torch.set_num_threads(1)
dtype, device = torch.float32, torch.device("cpu")

def read_splits():
    idx = {"train": [], "val": [], "test": []}
    with (ROOT / "metadata.csv").open(newline="") as f:
        rows = list(csv.DictReader(f))
    assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(rows[0])
    assert len(rows) == 360, len(rows)
    assert [int(r["index"]) for r in rows] == list(range(len(rows)))
    for row in rows:
        idx[row["split"]].append(int(row["index"]))
    return idx

idx = read_splits()
X = np.load(ROOT / "images.npy").astype(np.float32).reshape(-1, D)
Y = np.load(ROOT / "labels.npy").astype(np.int64)
assert X.shape[0] == Y.shape[0] == 360, (X.shape, Y.shape)
assert 0.0 <= float(X.min()) and float(X.max()) <= 1.0, (X.min(), X.max())

class AE(nn.Module):
    def __init__(self, d, b, h=64):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(d, h), nn.ReLU(), nn.Linear(h, b))
        self.dec = nn.Sequential(nn.Linear(b, h), nn.ReLU(), nn.Linear(h, d))

    def forward(self, x):
        z = self.enc(x)
        return self.dec(z), z

def make_loader(indices):
    tensors = (
```

```
        torch.tensor(X[indices], dtype=dtype, device=device),
        torch.tensor(Y[indices], dtype=torch.int64, device=device),
    )
    return DataLoader(TensorDataset(*tensors), batch_size=48, shuffle=True)

def train_ae(b, epochs=15):
    torch.manual_seed(SEED)
    model = AE(D, b).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    loader = make_loader(idx["train"])
    for _ in range(epochs):
        for xb, _ in loader:
            opt.zero_grad()
            xh, _ = model(xb)
            loss = nn.functional.mse_loss(xh, xb)
            loss.backward()
            opt.step()
    return model

@torch.no_grad()
def embed_batch(model, indices):
    out = []
    for i in indices:
        x = torch.tensor(X[i], dtype=dtype, device=device)[None, :]
        out.append(model.enc(x)[0].cpu().numpy())
    return np.stack(out)

@torch.no_grad()
def val_mse(model, indices):
    x = torch.tensor(X[indices], dtype=dtype, device=device)
    xh, _ = model(x)
    return nn.functional.mse_loss(xh, x).item()

def downstream_acc(features_tr, label_tr, features_val, label_val, out_dim=3, epochs=30):
    torch.manual_seed(SEED)
    mu = features_tr.mean(axis=0, keepdims=True)
    sd = features_tr.std(axis=0, keepdims=True) + 1e-8
    features_tr = (features_tr - mu) / sd
    features_val = (features_val - mu) / sd
    head = nn.Linear(features_tr.shape[1], out_dim).to(device)
    opt = torch.optim.Adam(head.parameters(), lr=1e-2)
    ce = nn.CrossEntropyLoss()
    zt = torch.tensor(features_tr, dtype=dtype, device=device)
    yt = torch.tensor(label_tr, dtype=torch.int64, device=device)
    zv = torch.tensor(features_val, dtype=dtype, device=device)
    yv = torch.tensor(label_val, dtype=torch.int64, device=device)
    for _ in range(epochs):
        opt.zero_grad()
        loss = ce(head(zt), yt)
        loss.backward()
        opt.step()
    pred = head(zv).argmax(dim=1)
    return (pred == yv).float().mean().item()

def report(latent_b):
    model = train_ae(latent_b)
    z_tr = embed_batch(model, idx["train"])
    z_val = embed_batch(model, idx["val"])
    assert z_tr.shape == (len(idx["train"]), latent_b)
    acc = downstream_acc(z_tr, Y[idx["train"]], z_val, Y[idx["val"]])
    mse = val_mse(model, idx["val"])
    std = z_tr.std(axis=0)
    eff_dim = int((std > 1e-3).sum())
```

```
    print(f"b={latent_b:2d}  val_MSE={mse:.4f}  val_acc={acc:.3f}  "
          f"eff_dim={eff_dim}/{latent_b}  min_std={std.min():.4f}")
    return mse, acc

m4, a4 = report(4)
m16, a16 = report(16)
# Baseline: nyers pixel + ugyanaz a downstream fej.
acc_raw = downstream_acc(
    X[idx["train"]], Y[idx["train"]],
    X[idx["val"]], Y[idx["val"]],
)
print(f"raw pixel baseline  val_acc={acc_raw:.3f}")
assert 0.0 <= a4 <= 1.0 and 0.0 <= a16 <= 1.0 and 0.0 <= acc_raw <= 1.0
assert 0.0 <= m4 <= 1.0 and 0.0 <= m16 <= 1.0
```

A futtatás a projektgyökérből:

```
python lab/n115_latent_probe.py
```

A fenti kód mért kimenete (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, float32, seed 20261020, egy szál, 15 AE-epoch, 30 probe-epoch):

Modell	validációs MSE	downstream val_acc	effektív dimenzió
$b = 4$ AE	0,0141	0,267	4/4
$b = 16$ AE	0,0140	0,283	16/16
nyers pixel baseline	—	0,350	—

A mért futásban a $b = 16$ AE rekonstrukciós MSE-je alig kisebb a $b = 4$-énél, downstream pontossága viszont a véletlen szintje (1/3) körül marad, a $b = 4$ modellé pedig az alatt. A nyers pixel baseline 0,350 pontossága mindkettőnél nagyobb ezen a validációs spliten. Ez a konkrét futás tehát nem mutatja, hogy az autoencoder latens tere jobb lineárisan leolvasható reprezentációt adna a nyers pixelnél; éppen az 1. állítás tanulságát illusztrálja: a kisebb rekonstrukciós hiba nem jelenti automatikusan a jobb downstream reprezentációt. A táblázat egy seed és egy 60 képes validációs split eredménye; a becsült pontosság szórása $p \approx 1/3$ körül $\sqrt{p(1-p)/60} \approx 0{,}061$, ezért a három érték különbsége ezen a mintán statisztikailag nem tekinthető megalapozottnak. A downstream fej rögzített, 30 lépéses költségvetéssel és standardizált jellemzőkkel tanul, tehát a táblázat a reprezentációk összehasonlításában alsó korlátot ad, nem a legjobban beállított lineáris leolvasó teljesítményét.

29.3.7 Gyakori tévedés és ellenpélda

A gyakori tévedés: „ha a rekonstrukciós hiba kicsi, akkor a latent reprezentáció szükségképpen jó szemantikai reprezentáció.” Az 1. állítás konstrukciója cáfolja: az identitásleképezés $L_{\text{rec}} = 0$ értékű, de a XOR-feladaton a lineáris downstream fej legfeljebb 3/4 pontosságot ér el. Egy másik, ugyancsak nulla rekonstrukciós hibájú nemlineáris reprezentáció viszont 4/4-et ér. A hiba forrása, hogy a rekonstrukciós cél nem tartalmazza a downstream címkét, a bemenet koordinátarendszerének megtartása pedig önmagában nem hoz létre lineárisan leolvasható jellemzőt.

Önellenőrzés. (1) Megkülönböztettem-e a névleges b dimenziót az effektív, nem összeomlott koordináták számától? (2) A downstream pontosságot befagyasztott encoderen mértem-e, nem közös finomhangolás után? (3) A validációs split megőrizte-e a `scene_id` függetlenséget a 28.3. Retrieval-értékelés és adatfelosztás szerződése alapján? (4) A lineáris pixel-baseline-t ugyanarra a splitre és ugyanarra a fejre futtattam-e, mint a 25.3. Kis adat és feature-alapú

baseline alfejezetben? (5) Nem állítottam-e általános reprezentációminőséget egyetlen downstream feladatból?

Válaszkulcs. (1) Igen: a $b = 32$ névleges dimenzió, az effektív dimenzió a nem összeomlott koordináták száma. (2) Igen, a leolvasó befagyasztott encoder mellett tanult. (3) Igen, a validációs split `scene_id` szintű. (4) Igen, ugyanaz a split és ugyanaz a lineáris fej. (5) Nem: egyetlen downstream feladat eredménye nem általános reprezentációminőség.

29.3.8 F1–F5 feladatok

F1. feladat (fogalmi, nehézség 1). Magyarázza el saját szavaival a bottleneck, a triviális identitásleképezés és a latent collapse közötti különbséget a $d = 576$, $b = 32$ GlyphLab-példán. Adja meg mindhárom fogalom formális feltételét is.

F2. feladat (levezetési/bizonyítási, nehézség 2). Bizonyítsa be az 1. állításban használt 3/4-es felső korlátot: a négy XOR-ponton az identitás-reprezentációhoz tartozó lineáris osztályozó nem érhet el négy helyes döntést. Írja le azt is, miért lehet három helyes döntés éles.

F3. feladat (implementációs, nehézség 1). Írjon Python-függvényt `xor_readout_acc(alpha, beta, gamma)` néven, amely a négy XOR-pontra kiszámítja a $q(x) = \alpha x_1 + \beta x_2 + \gamma$ osztályozó pontosságát. Ellenőrizze, hogy az $\alpha = 10$, $\beta = 10$, $\gamma = -0{,}1$ eset 0,75-ös pontosságot ad, és adja meg azt a nemlineáris reprezentációt, amelyen a lineáris leolvasó 1,0-t ér el.

F4. feladat (hibakeresési, nehézség 2). Egy hallgató a következő kóddal ellenőrzi a collapse-ot:

```
z = encoder(x_batch)          # alak: (N, b)
print("collapse?", float(z.std().item()) < 1e-4)
```

Mi a hiba? Írja le az okot, a lehetséges következményt, a javítást és a javítás utáni shape-ellenőrzést.

F5. feladat (kísérleti/transzfer, nehézség 3). Tervezze meg a GlyphLabon a $b = 4$ és $b = 16$ autoencoder összehasonlítását úgy, hogy a következtetés a befagyasztott lineáris downstream pontosságra és a rekonstrukcióra is korrekt korlátot adjon. Adja meg a megfigyelési egységet, a splitet, a baseline-t, a rögzített metrikákat és a legfőbb érvényességi korlátot. Győztest nem kell előre mondani.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

29.3.9 Megoldások és indoklás

F1 megoldás. A bottleneck a $b < d$ feltétel, tehát az encoder kimenete kisebb dimenziós, mint a bemenet. A $b = 32$ esetben $32 \ll 576$; a lineáris encoder rangja legfeljebb 32, ezért a $W_d W_e \in \mathbb{R}^{576\times576}$ szorzat rangja is legfeljebb 32; egy ilyen leképezés nem lehet az identitás, mert az identitás rangja 576. Az identitásleképezéshez legalább 576 rangú, azaz $b \geq d$ kód kellene. A triviális identitás ehhez képest a $b = d$ esetben $f(x) = x$, $g(z) = z$. A collapse nem dimenziófeltétel, hanem aktivációeloszlás-feltétel; a $Z \in \mathbb{R}^{N\times b}$ kódmátrix valamely oszlopának szórása közel nulla. Ha $b = 32$, de egyetlen koordináta szórása sem nulla, még lehet rossz downstream pontosság; a b csak kapacitás, nem minőség.

F2 megoldás. Tegyük fel, hogy a lineáris osztályozó mind a négy pontot jól osztályozza. A pozitív pontokra $(1, 1)$ és $(-1, -1)$ előírt $q > 0$ feltételekből

$$\alpha + \beta + \gamma > 0, \qquad -\alpha - \beta + \gamma > 0,$$

ezért $\gamma > 0$. A negatív pontokra $(1,-1)$ és $(-1,1)$ előírt $q \le 0$ feltételekből

$$\alpha - \beta + \gamma \le 0, \qquad -\alpha + \beta + \gamma \le 0,$$

amelyek összege $2\gamma \le 0$. Ez ellentmond $\gamma > 0$-nak, tehát a négyes pontosság lehetetlen. A 3/4 éles, mert az $\alpha = 10$, $\beta = 10$, $\gamma = -0{,}1$ választás három helyes döntést ad, ahogy az 1. állításban kiszámoltuk. A bizonyítás fontos előfeltétele, hogy a döntési szabály $q > 0$, nem pedig $q \ge 0$. Ha a pozitív pontokra $q \ge 0$ szerepelne, a két pozitív egyenlőtlenség összege csak $\gamma \ge 0$-t adna, ami nem mond ellent a negatívokból adódó $\gamma \le 0$-nak; a $q > 0$ szigorú feltétel adja a $\gamma > 0$ alsó korlátot, amely már ellentmond a $2\gamma \le 0$-nak. A szigorú egyenlőtlenség tehát a nulla küszöbön biztosítja, hogy a pozitív pontok valóban a nyílt pozitív féltérben legyenek, és a bizonyítás ne a határon elhelyezkedő, degenerált esetre épüljön.

F3 megoldás.

```
import numpy as np

P = np.array([[1, 1], [-1, -1], [1, -1], [-1, 1]], dtype=np.float64)
Y = np.array([1, 1, 0, 0], dtype=np.int64)

def xor_readout_acc(alpha, beta, gamma):
    q = alpha * P[:, 0] + beta * P[:, 1] + gamma
    pred = (q > 0).astype(np.int64)
    return float((pred == Y).mean())

print(xor_readout_acc(10.0, 10.0, -0.1))  # 0.75
# Nemlineáris reprezentáció: z=(x1*x2, x1), leolvasó q=z1.
Z = np.column_stack([P[:, 0] * P[:, 1], P[:, 0]])
pred_z = (Z[:, 0] > 0).astype(np.int64)
print((pred_z == Y).mean())                # 1.0
```

A függvény kimenete 0,75 és 1,0; a bemenő pontokra nincs szükség numerikus toleranciára, mert a döntés csak előjel-egyenlőtlenségeket használ.

F4 megoldás. A hiba oka: a `z.std()` hívás a teljes `(N,b)` tensor minden elemére számít egyetlen skalár szórást, amely az oszlopok közötti átlagkülönbségeket és az oszlopon belüli szórást összemossa. Ezért a globális szórás akkor sem nulla, ha minden oszlop konstans, de az oszlopok átlaga eltér (a különböző átlagok miatt pozitív a szórás), és akkor sem feltétlenül tűnik el, ha csak egyetlen oszlop változik. A következmény: a részleges vagy teljes collapse észrevétlen marad, mert a vizsgált mennyiség nem az oszloponkénti szórás. Javítás:

```
unit_std = z.std(dim=0)          # alak: (b,)
scale = z.abs().mean().clamp_min(1e-12)
collapsed = unit_std < 1e-4 * scale
print("koordinátánként:", unit_std)
print("collapsed:", collapsed.sum().item(), "/", collapsed.numel())
```

Ellenőrzés: z alakja `(N, b)`, ezért `z.std(dim=0)` alakja `(b,)`; a küszöb alá eső koordinátákat egyesével jelentjük. A küszöb relatív, mert a latent skála modelltől függ; az $1/10^4$ arány heurisztikus, és a rögzített protokollban fel kell jegyezni. A teljes tensorra vett szórás és az oszloponkénti szórás két különböző statisztika; a collapse vizsgálatához az utóbbi kell.

F5 megoldás (lehetséges terv). Megfigyelési egység: egy kép, de a split függetlenségi egysége a `scene_id`, tehát egyetlen jelenet mindkét nézete azonos részbe kerül. Protokoll: a tanító spliten

tanítunk két MSE-autoencodert $b = 4$ és $b = 16$ mellett, azonos hidden szélességgel, seeddel és epochszámmal. Utána mindkét encoder súlyát befagyasztjuk; a tanító kódokon lineáris fejet tanítunk orientációra, a fejet a validációs spliten értékeljük. Baseline: ugyanaz a lineáris fej a nyers 576 dimenziós pixelveken. Metrikák: validációs MSE, validációs downstream pontosság, a latent koordinátánkénti szórása és a collapsed koordináták száma. A teszt splitet csak a protokoll lezárása után használjuk. Érvényességi korlát: a GlyphLab három orientációs osztálya (függőleges, vízszintes, átlós vonalak) szintetikus, szabályos geometriájú adatból származik, így a következtetés nem vihető át természetes képfelismerésre; a downstream eredmény egyetlen lineáris fejre vonatkozik, tehát nem általános reprezentációminőség. A rekonstrukciós és downstream rangsor akár el is válhat, ahogy az 1. állítás mutatja.

29.3.10 Rövid összefoglaló

Az autoencoder rekonstrukciós célja nem rögzíti a latent tér downstream hasznosságát. A $b = d$ triviális identitásleképezés nulla hibát ad, miközben egy XOR-szerű downstream címkén a lineáris fej legfeljebb 3/4 pontosságot ér el; egy alkalmas nemlineáris encoder ugyanakkora dimenzióval és ugyancsak nulla rekonstrukciós hibával teljes pontosságot adhat. Ezért a nullához közeli MSE önmagában nem bizonyít szemantikai tudást. A bottleneck, ritka aktiváció és kontraktív büntetőtag heurisztikus irányt ad, de a tényleges ellenőrzéshez külön kell mérni a rekonstrukciót, a koordinátánkénti collapse-ot és egy előre rögzített downstream kontrollt.

29.3.11 Következő alkalmazás és források

A következő 29.4. Anomáliapontszám rekonstrukcióból egység az anomáliapontszám rekonstrukcióból való származtatását vizsgálja; az itteni azonosság és kapacitás-megkülönböztetés ott annak korlátjaként tér vissza, hogy a kis rekonstrukciós hiba nem automatikusan jelent ismert mintát.

Felhasznált források:

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- Rifai, S., Vincent, P., Muller, X., Glorot, X. és Bengio, Y. (2011). *Contractive Auto-Encoders: Explicit Invariance During Feature Extraction*. Proceedings of the 28th International Conference on Machine Learning (ICML), 833–840. A kontraktív regularizálás forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

29.4 Anomáliapontszám rekonstrukcióból

29.4.1 1. Cél, előismeret és induló kérdés

A 29.3. Regularizált latent reprezentáció alfejezetben láttuk, hogy a rekonstrukciós cél önmagában még nem hoz létre szemantikus reprezentációt: kapacitás és regularizáció nélkül az identitáshoz közeli leképezés is elérhető. Ebben az alfejezetben a rekonstrukciót döntési feladatra használjuk: egy modellt kizárólag „normálisnak” tekintett mintákon tanítunk, majd a rekonstrukciós hibát anomáliapontszámként (anomaly score) értelmezzük. A rekonstrukciós hiba alapú anomáliadetekció és a mély anomáliadetekció szélesebb áttekintése Ruff és mtsai (2021) munkája; a jelen fejezet a küszöbválasztás és az érvényességi korlát szerződésére szorítkozik. A cél nem az, hogy a

hibát valószínűségnek nevezzük, hanem az, hogy a küszöbválasztást és a hibaköltséget validációs protokollhoz kössük, és kimondjuk az eljárás érvényességi határait.

Előismeret: a 29.1. Lineáris és nemlineáris autoencoder autoencoder-definíciója, a 29.2. Denoising és korrupciós modell rekonstrukciós és denoising-célja, valamint a 25.4. Nyílt halmaz és ismeretlen input nyílt halmazú döntés gondolata. Nem szükséges haladó statisztikai előismeret.

Induló kérdés. Az autoencoder átlagos tanítási hibája 0,01. Egy új képen ugyanez a modell 0,05 hibát ad. Mekkora a valószínűsége annak, hogy az új kép anomália? Becslés: ebből az egyetlen számból nem határozható meg. A hiba nem valószínűség, és a modell a tanult sémán kívüli inputot is rekonstruálhatja jól, ahogyan a 4. szakasz ellenpéldája mutatja.

29.4.2 2. Rekonstrukciós pontszám és normal training

Legyen egy kép $x \in [0,1]^{C\times H\times W}$, például a függelék A. A GlyphLab adatkészlet szakaszában rögzített GlyphLab-szerződésben $C = 1$, $H = W = 24$. Az autoencoder egy $f_\theta : [0,1]^{C\times H\times W} \to [0,1]^{C\times H\times W}$ paraméteres függvény (a kimeneti Sigmoid miatt a képe valójában a nyílt $(0,1)^{C\times H\times W}$ halmazba esik), a rekonstrukció $\hat{x} = f_\theta(x)$. A pontszám a kép átlagos négyzetes hibája:

$$s(x;\theta) = \frac{1}{CHW}\sum_{c=1}^{C}\sum_{h=1}^{H}\sum_{w=1}^{W}(x_{c,h,w} - \hat{x}_{c,h,w})^2.$$

Batch esetén az $X \in \mathbb{R}^{N\times C\times H\times W}$ bemenethez és $\hat{X} = f_\theta(X)$ rekonstrukcióhoz egy $s \in [0,\infty)^N$ pontszámvektor tartozik, ahol s_i a fenti átlag. A továbbiakban az MSE-pontszámot használjuk; bináris bemenetre a BCE-loss 29.2. Denoising és korrupciós modell alfejezetből szintén lehetséges, de akkor a pontszám más numerikus skálán mozog (BCE-érték), ezért az MSE-re kalibrált küszöb numerikus értéke nem vihető át automatikusan: a küszöböt az adott pontszám-definícióhoz kell újraválasztani.

Normal training. A tanítóhalmaz $D_{\text{tr}} = \{x_i\}_{i=1}^n$, amelyről feltesszük, hogy a normál működésből származik. Anomáliacímkét a tanítás nem használ. A cél

$$\hat{\theta} \in \arg\min_\theta \frac{1}{n}\sum_{i=1}^{n}\ell(x_i, f_\theta(x_i)) + \lambda R(\theta),$$

ahol R a 29.3. Regularizált latent reprezentáció alfejezetben bevezetett sparse, contractive vagy más regularizáció, $\lambda \geq 0$. A regularizáció itt érdemi szerepet játszik: a szűk keresztmetszet vagy a büntetőtag akadályozhatja, hogy a modell tetszőleges inputot identikusan visszaadjon. Ez azonban heurisztika, nem bizonyított szeparációs garancia.

Contamination (szennyeződés). Két különböző fogalmat ne keverjünk. A *tanítási contamination* π_{tr} annak aránya, hogy a feltételezett normál tanítóhalmaz valójában anomáliákat is tartalmaz. Ha $\pi_{\text{tr}} > 0$, a normal training feltételezése sérül, és a minimalizált cél a nem kívánt mintákra is illesztheti a modellt. Az *értékelési contamination* π_v a validációs vagy tesztkeverék anomáliaaránya; ez a küszöbválasztás és a költség becslésének bemenete, nem a modell tanításáé.

29.4.3 3. Küszöbválasztás és hibaköltség

A döntési szabály rögzített $\tau \in \mathbb{R}$ küszöbbel:

$$a_\tau(x) = \begin{cases} 1, & s(x;\theta) > \tau, \\ 0, & s(x;\theta) \le \tau, \end{cases}$$

ahol az 1 címke jelenti az anomáliát. A τ megválasztásához olyan validációs halmaz kell, amelyben ismert $y \in \{0,1\}$ címkék vannak. A validációs pontszámokból becsüljük a hamis pozitív és hamis negatív arányt:

$$\widehat{\mathrm{FPR}}(\tau) = \frac{1}{n_v^0} \sum_{i:y_i=0} \mathbf{1}\{s_i > \tau\}, \qquad \widehat{\mathrm{FNR}}(\tau) = \frac{1}{n_v^1} \sum_{i:y_i=1} \mathbf{1}\{s_i \le \tau\},$$

ahol n_v^0 és n_v^1 a normál és anomália minták száma a validációban. Legyen $c_{\mathrm{fn}} > 0$ egy kihagyott anomália költsége, $c_{\mathrm{fp}} > 0$ egy hamis riasztásé, π_v pedig az anomáliák validációs aránya. A várható költség becslése

$$\hat{C}_\pi(\tau) = \pi_v c_{\mathrm{fn}} \widehat{\mathrm{FNR}}(\tau) + (1-\pi_v) c_{\mathrm{fp}} \widehat{\mathrm{FPR}}(\tau).$$

A küszöböt a validációs pontszámokból álló rács felett választjuk:

$$\tau^* \in \arg\min_\tau \hat{C}_\pi(\tau).$$

Ha a validáció csak normál mintát tartalmaz, a $\widehat{\mathrm{FNR}}$ tag nem becsülhető. Ekkor legfeljebb a normál pontszámok $(1-\alpha)$-kvantilisét választhatjuk, ahol $\alpha \in (0,1)$ az előírt hamisriasztás-arány felső korlátja; ez a hamis riasztási arányt vezérli, de a detektálási képességről nem ad állítást. A küszöböt nem a tanítóhalmaz pontszámaiból választjuk: azok tipikusan alacsonyak, mert a modell látta a mintákat, ezért a tanítási ponthatár optimista becslés lenne.

29.4.4 4. Ellenpélda: alacsony rekonstrukciós hiba ismeretlen mintán

Ebben a fejezetben formálisan meg kell mutatni, hogy a modell ismeretlen mintát is jól rekonstruálhat, tehát az alacsony pontszám nem azonosítja a normál eloszlást. A legegyszerűbb ilyen eset az identikus rekonstrukció.

Lemma 1 (identikus rekonstrukció mint OOD-kudarc). Az OOD rövidítés az out-of-distribution (eloszláson kívüli) mintát jelöli. Legyen $\mathcal{X}$ a lehetséges inputok nem üres halmaza, $\mathcal{F} = \{f_\theta : \mathcal{X} \to \mathcal{X}\}$ egy paraméteres függvénycsalád, és legyen $\ell : \mathcal{X} \times \mathcal{X} \to [0,\infty)$ olyan veszteség, amelyre minden $x \in \mathcal{X}$ esetén $\ell(x,x) = 0$. Tegyük fel, hogy az identitás $f_{\mathrm{id}}(x) = x$ eleme $\mathcal{F}$-nek. Ekkor bármely $D_{\mathrm{tr}} \subset \mathcal{X}$ tanítóhalmazra létezik olyan θ paraméter, amelyre a tanítási veszteség 0, és minden $x \in \mathcal{X}$ — így minden ismeretlen anomáliamintára is — $s(x;\theta) = 0$.

Bizonyítás. Válasszuk a θ paramétert úgy, hogy $f_\theta = f_{\mathrm{id}}$. Ekkor minden tanítópontra $f_\theta(x_i) = x_i$, tehát a veszteségtagok mindegyike 0, az empirikus cél 0. Ugyanebből az okból tetszőleges $x \in \mathcal{X}$-re $\hat{x} = x$, így az MSE-pontszám minden tagja eltűnik, $s(x;\theta) = 0$. A kihagyott anomáliákat a $\tau > 0$ küszöb mind normálnak osztályozza, tehát a modell semmilyen elválasztást nem végez. □

A lemma a regularizáció nélküli függvénycsaládra vonatkozik; a $\lambda R(\theta)$ tag jelenlétében az identitás nem feltétlenül optimum. A feltétel triviálisnak tűnhet, de az identitás nem puszta elméleti eset: regularizáció nélkül, túl nagy látens térrel és lineáris autoencodernél az optimális rekonstrukció pontosan az identitás lehet. Tekintsünk egy lineáris autoencodert, ahol $E \in \mathbb{R}^{h \times d}$ az encoder, $D \in \mathbb{R}^{d \times h}$ a decoder, és $h \geq d$. Ha $DE = I_d$, akkor az egész leképezés $f(x) = DEx = x$. Ilyen pár létezik, például $E = \begin{pmatrix} I_d \\ 0 \end{pmatrix}$ és $D = \begin{pmatrix} I_d & 0 \end{pmatrix}$. A normal training tehát elérheti a nulla hibát, miközben az anomáliafelismerő képesség nulla.

Ez az ellenpélda nem jelenti azt, hogy a rekonstrukciós pontszám használhatatlan. Azt jelenti, hogy a használhatóság az architektúra kapacitásán, a regularizáción és a validáción mért elválasztáson múlik; a tanítási hiba nagysága önmagában nem bizonyíték.

29.4.5 5. Kézi példa és ellenőrző kód

Legyen $x \in \mathbb{R}^2$, $E = \begin{pmatrix} 1 & 0 \end{pmatrix}$ és $D = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. Ekkor

$$DE = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \hat{x} = DEx = \begin{pmatrix} x_1 \\ 0 \end{pmatrix}.$$

A modell a második koordinátát eldobja. A tanítóhalmaz legyen $\{(2,0), (4,0), (1,0)\}$. Mindhárom tanítópontra $\hat{x} = x$, tehát $s = 0$. Az $(100, 0)$ pont messze van a tanító felhőtől, de a rekonstrukciós hibája szintén 0: a pontszám nem a tanítóhalmaztól mért távolság, hanem a modellezett altértől mért távolság. A $(0, 3)$ pont esetén $\hat{x} = (0, 0)$, tehát

$$s((0,3)) = \frac{1}{2}(0^2 + 3^2) = 4{,}5.$$

Ha viszont $E = I_2$ és $D = I_2$, akkor mind az öt pontra $s = 0$, beleértve a $(0, 3)$ pontot is. Ez a Lemma 1 konkrét kétváltozós esete.

```
# lab/score_hand_check.py
import numpy as np

E = np.array([[1.0, 0.0]])          # (1,2)
D = np.array([[1.0], [0.0]])        # (2,1)
X = np.array([[2, 0], [4, 0], [1, 0], [100, 0], [0, 3]],
             dtype=np.float32)      # (5,2)

Xhat = X @ E.T @ D.T               # (5,1)@(1,2) = (5,2)
s = np.mean((X - Xhat) ** 2, axis=1)
print(s)                            # [0.  0.  0.  0.  4.5]

E2 = np.eye(2, dtype=np.float32)
D2 = np.eye(2, dtype=np.float32)
Xhat2 = X @ E2.T @ D2.T
print(np.mean((X - Xhat2) ** 2, axis=1))  # [0. 0. 0. 0. 0.]
```

A példa egyszerre tölti be a kézi kontroll és az ellenpélda szerepét. A szélső eset azt is megmutatja, hogy a nullpontszám nem a tanítóeloszlás tagságát igazolja.

29.4.6 6. Labor: jelölt anomáliateszt a GlyphLab adaton

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja egy kis jelölt anomáliateszt, nem valódi csalás- vagy diagnózisállítás. A normál eloszlást a GlyphLab tiszta tanítóképei alkotják; anomáliának a `corrupted_test.npy` zajos és fényességgradienssel torzított tesztképeit tekintjük. A cél annak eldöntése, hogy a rekonstrukciós pontszám a párosított tesztpárokban a korrupt oldalon nagyobb-e.

Protokoll.

- Kérdés: a tanított autoencoder pontszáma páronként hány százalékban nagyobb a korrupt oldalon?
- Megfigyelési egység: kép; a páros összehasonlítás egysége a `scene_id`/`view_id` pár, tehát a csoportos szerződés 5.4. Függő képek, seedek és resampling szerint járunk el.
- Split: train 240 normál kép a modellhez; validáció 60 normál kép plusz 60 determinisztikusan előállított szintetikus anomália a küszöbválasztáshoz; független teszt 60 tiszta és 60 korrupt kép a `corrupted_test.npy`-ból. A korrupt tesztet csak a befagyasztott modell és a kiválasztott küszöb után értékeljük, a küszöb utólagos módosítása nélkül.
- Modell: kis konvolúciós autoencoder, a kimenet $1 \times 24 \times 24$. Rétegenként:
  - encoder: `Conv2d(1, 8, 3, stride=2, padding=1)`, ReLU;
  - `Conv2d(8, 16, 3, stride=2, padding=1)`, ReLU; flatten;
  - `Linear(576, 32)`;
  - decoder: `Linear(32, 576)`, ReLU; `Unflatten(1, (16, 6, 6))`;
  - `ConvTranspose2d(16, 8, 3, stride=2, padding=1, output_padding=1)`, ReLU;
  - `ConvTranspose2d(8, 1, 3, stride=2, padding=1, output_padding=1)`, Sigmoid.
- Tanítás: MSE, Adam, tanulási ráta 10^{-3}, batch 32, 30 epoch, CPU, float32, seed 20261020.
- Küszöb: $c_{\mathrm{fn}} = 3$, $c_{\mathrm{fp}} = 1$, $\pi_v = 0{,}5$; a validációs pontszámok kvantilisrácsán minimalizáljuk $\hat{C}_\pi(\tau)$-t. Holtverseny esetén a rácson a legkisebb τ nyer (a kód szigorú `<` összehasonlítása); ezt a szabályt a protokoll előre rögzíti.
- Jelentés: a küszöb, a validációs költség, a teszt confusion matrix, a páros nyerési arány és az AUC.

```
# lab/anomaly_lab.py - futtatás: cd lab && python anomaly_lab.py
import csv
import json
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn

ROOT = Path(".")
SEED = 20261020
torch.manual_seed(SEED)
np.random.seed(SEED)
torch.set_num_threads(1)
device = torch.device("cpu")
dtype = torch.float32

images = np.load(ROOT / "data" / "images.npy")            # (360,1,24,24)
```

```
corrupted = np.load(ROOT / "data" / "corrupted_test.npy") # (60,1,24,24)
with (ROOT / "data" / "metadata.csv").open() as f:
    meta = list(csv.DictReader(f))
assert len(meta) == 360, len(meta)
assert np.array_equal(np.array([int(r["index"]) for r in meta]), np.arange(360)), \
    "a metadata indexe nem a 0..359 tartomány permutációja"
assert [sum(r["split"] == s for r in meta) for s in ("train", "val", "test")] == [240, 60, 60]

def idx(split):
    return np.array([int(r["index"]) for r in meta if r["split"] == split])

Xtr = images[idx("train")]      # (240,1,24,24)
Xva = images[idx("val")]        # (60,1,24,24)
Xte = images[idx("test")]       # (60,1,24,24)
assert Xtr.shape == (240, 1, 24, 24)
assert corrupted.shape == (60, 1, 24, 24)
# A corrupt teszt sorrendje a metadata test-sorrendje (a generátor corrupt_order mezője
# szerint); ezt a képtartalomból is ellenőrizzük: minden corrupt képnek a saját tiszta
# párja a legközelebbi testkép MSE-ben.
_clean = Xte.reshape(len(Xte), -1)
_corr = corrupted.reshape(len(corrupted), -1)
_pairs = ((_corr[:, None, :] - _clean[None, :, :]) ** 2).sum(axis=-1).argmin(axis=1)
assert np.array_equal(_pairs, np.arange(len(Xte))), "a corrupt sorrend nem a test-sorrend"
# A párosítás egysége a scene_id/view_id pár: a 60 testkép 30 jelenet két nézete.
test_rows = [r for r in meta if r["split"] == "test"]
test_scenes = [r["scene_id"] for r in test_rows]
assert len(test_rows) == 60 and len(set(test_scenes)) == 30
assert all(sum(r["scene_id"] == s for r in test_rows) == 2 for s in set(test_scenes))

# Validációs szintetikus anomália; ez NEM a corrupted_test, és csak
# a küszöbválasztáshoz használható.
rng = np.random.default_rng(SEED + 1)
Xva_anom = np.clip(Xva + 0.35 * rng.standard_normal(Xva.shape).astype(np.float32),
                   0.0, 1.0)

class ConvAE(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(1, 8, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(8, 16, 3, stride=2, padding=1), nn.ReLU(),
            nn.Flatten(), nn.Linear(576, 32))
        self.dec = nn.Sequential(
            nn.Linear(32, 576), nn.ReLU(),
            nn.Unflatten(1, (16, 6, 6)),
            nn.ConvTranspose2d(16, 8, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(8, 1, 3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid())

    def forward(self, x):
        return self.dec(self.enc(x))

model = ConvAE().to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
lossf = nn.MSELoss()

Xtr_t = torch.from_numpy(Xtr).to(device, dtype)
for epoch in range(30):
    model.train()
    perm = torch.randperm(Xtr_t.shape[0])
    for i in range(0, Xtr_t.shape[0], 32):
        xb = Xtr_t[perm[i:i + 32]]
```

```
        opt.zero_grad()
        loss = lossf(model(xb), xb)
        loss.backward()
        opt.step()

def score(model, x_np):
    model.eval()
    with torch.no_grad():
        x = torch.from_numpy(x_np).to(device, dtype)
        y = model(x)
        return torch.mean((x - y) ** 2, dim=(1, 2, 3)).cpu().numpy()

s_va_n = score(model, Xva)
s_va_a = score(model, Xva_anom)
c_fn, c_fp, pi = 3.0, 1.0, 0.5
best_tau, best_cost = None, float("inf")
for tau in np.quantile(np.r_[s_va_n, s_va_a], np.linspace(0.05, 0.95, 191)):
    fp = np.mean(s_va_n > tau)
    fn = np.mean(s_va_a <= tau)
    cost = pi * c_fn * fn + (1.0 - pi) * c_fp * fp
    if cost < best_cost:
        best_cost, best_tau = cost, float(tau)

s_te = score(model, Xte)
s_co = score(model, corrupted)          # a fájl betöltésekor használt név
# a sorok a test-sorrenddel párosítottak
tn = int(np.sum(s_te <= best_tau))
fp = int(np.sum(s_te > best_tau))
fn = int(np.sum(s_co <= best_tau))
tp = int(np.sum(s_co > best_tau))
paired = float(np.mean(s_co > s_te))

def auc_pairs(neg, pos):
    """ROC AUC Mann-Whitney-alakban: P(pos > neg) + 0,5 · P(pos = neg).

    Minden (negatív, pozitív) párt összevet, ezért a holtversenyt helyesen
    kezeli, és az osztálydarabszámokat a bemenetből veszi."""
    pos = np.asarray(pos)[:, None]
    neg = np.asarray(neg)[None, :]
    return float(np.mean((pos > neg) + 0.5 * (pos == neg)))

assert auc_pairs(np.zeros(4), np.zeros(3)) == 0.5          # minden pontszám azonos
assert auc_pairs([0.0, 0.1], [0.2, 0.3]) == 1.0            # tökéletes rangsor
assert auc_pairs([0.2, 0.3], [0.0, 0.1]) == 0.0            # fordított rangsor
auc = auc_pairs(s_te, s_co)

import datetime, platform, uuid
checks = {
    "shapes_ok": bool(Xtr.shape == (240, 1, 24, 24)
                      and corrupted.shape == (60, 1, 24, 24)),
    "scores_finite": bool(np.isfinite(np.r_[s_va_n, s_va_a, s_te, s_co]).all()),
    "threshold_selected": best_tau is not None,
    "confusion_counts": bool(tn + fp == len(s_te) and fn + tp == len(s_co)),
    "auc_in_range": bool(0.0 <= auc <= 1.0),
}
assert all(checks.values()), checks
report = {
    "run_id": uuid.uuid4().hex,
    "created_utc": datetime.datetime.now(datetime.timezone.utc).isoformat(timespec="seconds"),
    "status": "completed" if all(checks.values()) else "failed",
    "checks": checks,
    "config": {"seed": SEED, "epochs": 30, "batch": 32, "lr": 1e-3, "device": str(device),
               "dtype": str(dtype), "c_fn": c_fn, "c_fp": c_fp, "pi": pi,
```

```
                "torch": torch.__version__, "numpy": np.__version__,
                "python": platform.python_version()},
    "threshold": best_tau,
    "val_cost": best_cost,
    "test_confusion": {"TN": tn, "FP": fp, "FN": fn, "TP": tp},
    "paired_win_rate": paired,
    "auc": auc,
}
Path("results").mkdir(exist_ok=True)
(Path("results") / "anomaly_report.json").write_text(json.dumps(report, indent=2))
print(json.dumps(report, indent=2))
```

Futási jegyzőkönyv. A fenti szkript a mellékelt laborcsomag eredeti GlyphLab-adatán, CPU-n (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, float32, seed 20261020, egy szál) lefutott; két egymás utáni futás a mért mezőkön azonos számokat adott. A `status` mező a `checks` eredményéből számolódik (a kivonatban nem szerepel), nem előre beírt érték. A riport mérési része (a `run_id`, az időpont és a `config` futásonként, illetve környezetenként változik):

```
{
  "status": "completed",
  "threshold": 0.043707778677344401,
  "val_cost": 0.0,
  "test_confusion": {"TN": 60, "FP": 0, "FN": 30, "TP": 30},
  "paired_win_rate": 1.0,
  "auc": 1.0
}
```

Más verziójú NumPy/PyTorch vagy más CPU mellett a küszöb utolsó jegyei eltérhetnek; a saját futás riportját kell leadni, nem ezt a számsort.

Értelmezés. A két rangsoroló mutató tökéletes: mind a 60 párban a korrupt kép pontszáma a nagyobb (`paired_win_rate` 1,0), és az AUC is 1,0, tehát *létezik* olyan küszöb, amely a két tesztcsoportot hibátlanul szétválasztja. A validáción választott küszöb mégis 30 korrupt képet normálisnak minősít (FN = 30). Ennek oka a validációs és a teszt-anomália eltérő nehézsége. A validációs szintetikus anomália (0,35 szórású Gauss-zaj) pontszámai messze a normál pontszámok felett vannak, ezért a validációs költség a rácson 0; a rácspontok közül egyetlen esik a két validációs csoport közötti résbe (a medián, $\tau \approx 0{,}0437$). A teszt korrupciója viszont zajból és fényességi rámpából áll, és a pontszámai részben ebbe a résbe esnek, ezért a τ a korrupt képek felét normálisnak minősíti. A küszöb tehát a validációs anomáliákra optimális, a tesztkorrupcióra nem reprezentatív. A tanulság kettős. Egyrészt a küszöb minősége a validációs anomália reprezentativitásán múlik, nem a rangsor minőségén. Másrészt az AUC és a páros nyerési arány küszöbfüggetlen, a confusion matrix viszont egy konkrét döntési szabályt értékel; a kettőt külön kell jelenteni. A `val_cost` a validációs szintetikus anomáliákra érvényes; a tesztelt korrupt minták is szintetikusak, ezért ez nem valódi csalás- vagy hibadetektálási állítás. A páros összehasonlítás azért fontos, mert a tiszta teszt és a korrupt teszt ugyanazon `scene_id`/`view_id` párokat tartalmazza.

29.4.7 7. Feladatok

F1 (fogalmi, nehézség 1). Definiáld a rekonstrukciós pontszámot, a normal traininget, a contaminatiót és a küszöböt. Magyarázd el egy-egy mondatban, miért nem valószínűség a pontszám. Elvárt átadandó: legfeljebb tíz mondatos, saját szavas definíció, egy explicit ellenpélda-utalással.

F2 (levezetési, nehézség 2). Bizonyítsd be a Lemma 1-et a saját szavaiddal. Mutasd meg a $d = 2$, $h = 3$ túlteljes (overcomplete) lineáris esetben, hogy a $DE = I_2$ választás minden inputra

nulla pontszámot ad. Elvárt átadandó: a bizonyítás lépései, a D és E konkrét tömbjei, és a szorzat ellenőrzése.

F3 (implementációs, nehézség 2). Írj olyan NumPy-függvényt, amely egy $X \in \mathbb{R}^{N \times d}$ batch minden sorára kiszámítja az $\frac{1}{d}\sum_j (X_{ij} - \hat{X}_{ij})^2$ pontszámot, és ügyelj a redukció tengelyére: a batch egészére ne átlagolj, hanem mintánként számíts pontszámot. Teszteld: $X = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, $\hat{X} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$ esetén az eredmény [0,5; 0], mert az első sor hibája $((1-2)^2 + 0^2)/2 = 0{,}5$ (a koordinátánkénti átlag; az 1 a négyzetösszeg lenne). Elvárt: futtatható kód, import, két pontra pontos output, `float64` típus.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a pontszámot a batch összes elemének átlagaként számítja ki, majd ugyanezt a skalárt használja minden mintához. Mi a hiba, mi a következménye a küszöbválasztásra, és hogyan javítanád? Elvárt: ok, következmény, javított két-két sor, egy `assert`-tel ellenőrzött visszateszt.

```
s = np.mean((X - Xhat) ** 2)          # hiba: egyetlen skalár
scores = np.full(X.shape[0], s)       # minden mintára ugyanaz
```

F5 (kísérleti/transzfer, nehézség 3). Tervezz kísérletet annak eldöntésére, hogy a GlyphLab tiszta függőleges képein tanult autoencoder pontszáma elválasztja-e a vízszintes és átlós mintákat a függőleges mintáktól. Add meg a protokollt, a baseline-t, a splitet, a metrikát és a következtetési korlátot. Elvárt átadandó: legalább fél oldalas terv, nyitott eredménnyel; előre megadott győztes nem fogadható el.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

29.4.8 8. Megoldások és indoklás

F1. A pontszám $s(x;\theta) = \frac{1}{CHW}\sum_{c,h,w}(x_{c,h,w} - \hat{x}_{c,h,w})^2$. A normal training kizárólag normál feltételezésű mintákon minimalizálja a rekonstrukciós veszteséget. A contamination egy halmaz anomáliaaránya; tanítási és értékelési változatát külön kell kezelni. A küszöb τ olyan szám, amelynél nagyobb pontszám anomáliadöntést ad. A pontszám azért nem valószínűség, mert nem kalibrált: egy nemnegatív rekonstrukciós hiba, amelyből hiányzik a likelihood-arány, a prior és a kalibrációs leképezés. Az, hogy értéke nem $[0, 1]$-be esik, egy szükséges feltétel hiánya, de nem ez a lényegi ok: egy $[0, 1]$-be eső pontszám is lehet kalibrálatlan. Az identikus rekonstrukció esetén ismeretlen mintára is nulla lehet, ami önmagában mutatja, hogy a nulla pontszám nem jelent biztos normál tagságot.

F2. A Lemma 1 bizonyítása: ha $\mathcal{F}$ tartalmazza az identitást, az identitás paramétere minden tanítópontra $\hat{x}_i = x_i$, tehát a veszteség 0. Ugyanez tetszőleges új x-re, tehát az MSE-tagok mindegyike 0, és $s(x) = 0$. A $d = 2$, $h = 3$ esetben például

$$E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

Ekkor $DE = I_2$, tehát $f(x) = x$ minden x-re, és a pontszám azonosan nulla. Alakellenőrzés: $E \in \mathbb{R}^{3\times 2}$, $D \in \mathbb{R}^{2\times 3}$, a szorzat 2×2-es egységmátrix.

F3.

```python
import numpy as np

def per_sample_mse(X, Xhat):
    X = np.asarray(X, dtype=np.float64)
    Xhat = np.asarray(Xhat, dtype=np.float64)
    assert X.shape == Xhat.shape
    assert X.ndim == 2
    return np.mean((X - Xhat) ** 2, axis=1)

X = np.array([[1.0, 0.0], [0.0, 0.0]])
Xhat = np.array([[2.0, 0.0], [0.0, 0.0]])
out = per_sample_mse(X, Xhat)
assert np.allclose(out, [0.5, 0.0], atol=1e-15)
print(out)  # [0.5 0. ]
```

A `mean` csak az 1. tengelyre redukál, ezért minden megfigyelésnek külön pontszáma marad.

F4. A hiba a redukciós tengely elhagyása. A `np.mean` mind a $N \cdot d$ elem átlagát adja, így egyetlen skalár jön létre; a minták közötti különbség eltűnik, és a küszöb sem képes az egyedeket szétválogatni. Javítás: tartsuk meg a mintatengelyt, `axis=1`, illetve tensornál `dim=(1,2,3)` a NCHW-átlaghoz. Visszateszt, amely az alak mellett az értékeket is ellenőrzi:

```python
X = np.array([[1.0, 0.0], [0.0, 0.0]])
Xhat = np.array([[2.0, 0.0], [0.0, 0.0]])
s = np.mean((X - Xhat) ** 2, axis=1)
assert s.shape == (X.shape[0],)
assert np.allclose(s, [0.5, 0.0])
```

F5. Terv. Kérdés: a csak függőleges (vertical) GlyphLab-képeken tanított autoencoder rekonstrukciós pontszáma elválasztja-e a nem látott vízszintes (horizontal) és átlós (diagonal) mintákat a függőleges mintáktól? Ez egy one-class anomáliadetekciós kérdés: a normál osztály a függőleges, az anomáliák a másik két orientáció.

Adat és split: a GlyphLab szerződés szerinti train/val/test split; a modell csak a train függőleges képein tanul (80 kép, 40 jelenet két nézete). A validáció a függőleges val képeket (20) és a vízszintes/átlós val képeket (40) tartalmazza; a teszt a független jelenetekből 20 függőleges és 40 más orientációjú kép. A split jelenetszintű, hogy egy jelenet két nézete ne kerüljön különböző részbe. A tesztet a modell és a küszöb rögzítése után egyszer értékeljük.

Modell és tanítás: ugyanaz az MSE-autoencoder (például a 29.3. labor szerkezete), rögzített seeddel, epochszámmal és tanulási rátával; a modellválasztás csak a validációs függőleges MSE alapján történik.

Baseline-ok: (i) konstans pontszám, amelynek AUC-ja pontosan 0,5, és azt jelzi, hogy a pontszám nem diszkriminál; (ii) a tanító függőleges képek átlagától vett euklideszi távolság, ugyanazzal a kiértékeléssel. A „véletlen küszöb” nem baseline, mert az AUC küszöbfüggetlen; a küszöb csak a működési pont kiválasztásához kell.

Metrika: a pontszám AUC-ja a tesztjeleneteken (függőleges = negatív, vízszintes/átlós = pozitív), mert az AUC küszöbfüggetlen és a pontszám-skálától független rangsort mér. Emellett rögzített küszöb mellett a TPR-t és az FPR-t is jelentjük; a küszöböt a validációs halmazon a kívánt FPR-hez (például 5%) választjuk, és a teszt halmazon már nem hangoljuk. A bizonytalanságot jelenetszintű bootstrap adja a `scene_id`-k újramintavételezésével, mert a két nézet nem független.

Döntési szabály: akkor állítjuk, hogy a pontszám elválasztja az orientációkat, ha a teszt AUC 95%-os jelenetszintű bootstrap CI-ja 0,5 felett van, és a rögzített 5%-os FPR melletti TPR CI-ja is 0,5 felett van; ellenkező esetben nem állítunk diszkriminációt. A pontbecslés önmagában nem elég.

Érvényességi korlát: a GlyphLab szintetikus, szabályos vonalrajzokból áll, ezért az eredmény nem természetes kép-anomáliáról szól; az orientációk különbözősége nem feltétlenül valódi hibamód, és a függőleges osztályra szűkített tanítás miatt a modell más, nem látott háttérre vagy zajszintre nem általánosítható. A tanulmány csak azt mutatja meg, hogy ezen az adaton és ezzel a protokollal a rekonstrukciós pontszám mennyire rangsorolja az orientációkat.

29.4.9 9. Tipikus tévedés, önellenőrzés és visszahívás

A leggyakoribb tévedés, hogy a rekonstrukciós hiba általános anomália-valószínűség. Ez két irányban is hibás. Az 5. szakaszban a $(100, 0)$ pont a tanítófelhőtől távol van, mégis nulla pontszámot kapott, tehát a kis hiba nem jelent normál tagságot. A nagy hiba sem jelent feltétlenül anomáliát: egy ritka, de a normál eloszlásból származó minta is lehet rosszul rekonstruálható, ha a modell szűk kapacitású. A döntési szabály $1\{s > \tau\}$ nem ad $P(\text{anomália} \mid x)$ értéket; ilyen állításhoz likelihood-arány, prior és kalibrált modell kellene.

Önellenőrzés. Ha a validációs normál pontszámok 95%-os kvantilise 0,02, akkor egy 0,021 pontszámú minta biztosan anomália? Nem, ez a küszöb a hamis pozitív arányt vezérli, nem a pontszám jelentését.

Visszahívás. A 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés alkalmazási szerződése itt is érvényes: a cél anomáliadöntés, a target a validációban és teszten bináris címke, a tanító cél rekonstrukció, az output egy pontszám, a metrika FPR/FNR/AUC vagy páros arány, a split scene-szintű, és az inference-szerződés rögzített τ mellett zajlik. A 17.4. Architektúra összehasonlítása kontrollált keretben tisztességes összehasonlítás (fair comparison) elve szerint az identikus és a rendesen regularizált autoencoder összevetésekor a kapacitást, a tanítási iterációt és a küszöbválasztási protokollt előre rögzíteni kell, különben az identitás kudarca nem a regularizáció hatását méri.

29.4.10 Rövid összefoglaló

A rekonstrukciós pontszám egy minta és rekonstrukciója közötti átlagos négyzetes hiba, amelyet normal traininggel tanított autoencoder ad. A küszöböt validációs halmazon, ismert anomáliacímkékkel, a hamis pozitív és hamis negatív arányok költséggel súlyozott összege alapján választjuk. A Lemma 1 megmutatja, hogy az identikus leképezés minden ismeretlen mintát nulla hibával rekonstruálhat, tehát a pontszám nem anomália-valószínűség és nem ad önmagában szeparációs garanciát. A labor párosított tiszta-korrupt GlyphLab teszten futtatott, run_id-vel és konfigurációval azonosított jelentést készít; a kimenet nem valódi diagnózis. A gyakorlatban a kapacitás, a regularizáció és a validációs korlát együtt határozza meg az eljárás érvényességét.

29.4.11 Következő alkalmazás

A 30.1. Pretext és downstream feladat a rekonstrukciót pretext feladatként használja tovább: az encoder itt tanult reprezentációja downstream osztályozás bemenete lehet, és a fejezetben bevezetjük az önfelügyelt kiértékelés alapfogalmait.

29.4.12 Felhasznált források

- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G. és Müller, K.-R. (2021). *A Unifying Review of Deep and Shallow Anomaly Detection*. Proceedings of the IEEE, 109(5), 756–795. A rekonstrukciós hiba alapú anomáliadetekció hátterének forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

30 Önfelügyelt és kontrasztív reprezentációtanulás

30.1 Pretext és downstream feladat

30.1.1 1. Cél, előismeret és induló kérdés

A 29.1. Lineáris és nemlineáris autoencoder egységben az autoencoder reprezentációját a rekonstrukciós célhoz mértük; ott a szűk keresztmetszet akár a downstream feladat számára haszontalan tömörítést is adhat. Itt a kérdést megfordítjuk: lehet-e úgy tanítani egy reprezentációt, hogy **ne** kézzel címkézett targetet használjunk, mégis a későbbi alkalmazás számára hasznos belső kódot kapjunk? Ez az **önfelügyelt tanulás** (self-supervised learning) központi kérdése.

Előismeret: a 22.4. Augmentációs kísérlet és domainváltozás augmentációs policy és clean–corrupt teszt megkülönböztetése, a 20.4. V. ellenőrző pont: indokolt CNN-választás architektúra-döntési lap, a 23.4. Hibaszeletek és alkalmazási határ hibaszeletek és csoportszintű értékelés, valamint a 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés cél–target–loss–output–metrika–split–inference szerződés.

Induló kérdés: a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-képeken egy háló megtanulja, hogy ugyanannak a jelenetnek két zajos nézete összetartozik. Ettől a reprezentáció automatikusan jó lesz-e az orientáció felismerésére? Becslés: nem feltétlenül. A két nézet összetartozása érzéketlenséget kér a zajra és a kis eltolásra; az orientáció viszont éppen egy olyan tulajdonság, amelyet egy rosszul választott transzformáció eltüntethet. A 3. szakaszban formálisan megmutatjuk, hogy egzakt invariancia esetén már egyetlen olyan pontpár is kizárja a tökéletes downstream osztályozót; a szakasz végén pedig megtervezzük a GlyphLab frozen-probe értékelését.

30.1.2 2. Pretext, downstream, reprezentáció és a két tanítási szakasz

Definíció (pretext feladat). Legyen $\mathcal{X}$ a bemeneti tér, például a 24×24-es szürke képek halmaza. A **pretext feladat** egy olyan L_p veszteséggel és T_p céltenzorral definiált tanítási feladat, amelynek címkéi kizárólag magából a bemenetből, emberi annotáció nélkül állnak elő. A pretext feladatot nem azért oldjuk meg, mert a kimenete önmagában érdekes, hanem mert a megoldása során a háló belső reprezentációja a bemenet olyan szerkezetét kódolja, amely más feladatra átvihető.

Definíció (downstream feladat). A **downstream feladat** a tényleges alkalmazási cél, például a GlyphLab orientáció-osztályozás. Ezt L_d veszteséggel és $y \in \{0, 1, 2\}^N$ int64 céltenzorral mérjük. A downstream feladatot a pretext után, jellemzően kevés címkével tanítjuk.

Definíció (reprezentáció és fej). A hálót két részre bontjuk: az $f_\theta : \mathcal{X} \to \mathbb{R}^d$ **backbone** (vagy encoder) a bemenetet d-dimenziós reprezentációba képezi; a $h_\phi : \mathbb{R}^d \to \mathcal{Y}$ **fej** a reprezentációból a feladat kimenetét számolja. A pretext tanítás során f_θ és egy pretext-fej optimalizálódik; downstream során f_θ és h_ϕ viszonyának megválasztása a két fő protokollt adja.

- **Frozen probe:** a pretext után f_θ paramétereit rögzítjük (freeze), és csak a h_ϕ fej paramétereit tanítjuk a downstream célon. Ez a protokoll azt méri, mennyi feladat-specifikus információt hordoz a reprezentáció önmagában, további backbone-tanulás nélkül.
- **Fine-tune:** a downstream tanítás során f_θ paramétereit is tovább igazítjuk, jellemzően kisebb tanulási rátával. Ez több címkét és gondosabb regularizálást igényel, de nagyobb downstream kapacitást enged.

A GlyphLab adatszerződésben egy kép $X \in \mathbb{R}^{N\times 1\times 24\times 24}$, float32, $[0,1]$-ben. A backbone kimenete $Z \in \mathbb{R}^{N\times d}$; a downstream logit $S \in \mathbb{R}^{N\times 3}$, és a CrossEntropyLoss előtt softmaxot nem alkalmazunk (7.4. Numerikus stabilitás és jelentésmegőrző implementáció). A downstream target $y \in \{0,1,2\}^N$ int64.

Az önfelügyelt pretext célok egyik leggyakoribb formája a **nézetpár-invariancia**: ugyanabból a x bemenetből két, véletlenszerűen transzformált nézetet készítünk, $x_1 = t_1(x)$ és $x_2 = t_2(x)$, ahol t_1, t_2 egy $\mathcal{T}$ augmentációcsalád független mintái. Ez a felépítés a [P17] SimCLR keretrendszerének pretext célja. A pretext cél azt kéri, hogy $f_\theta(x_1)$ és $f_\theta(x_2)$ közel legyen egymáshoz, míg különböző bemenetek reprezentációi távol maradjanak. A konkrét kontrasztív veszteséget a következő, 30.2. Kontrasztív cél és temperature egységben vezetjük be; itt a pretext és a downstream viszonyára koncentrálunk.

30.1.3 3. Milyen invarianciát kér a két nézet, és mikor hibás ez?

A nézetpár-invariancia formális következménye az alábbi levezetés.

Állítás (a pretext invarianciaigénye). Legyen $\mathcal{T}$ az augmentációcsalád, amely tartalmazza az identitást. Ha a pretext veszteség ideális minimumában f_θ minden x-re és minden $t_1, t_2 \in \mathcal{T}$-re kielégíti az $f_\theta(t_1(x)) = f_\theta(t_2(x))$ egyenlőséget, akkor f_θ **invariáns** minden $t \in \mathcal{T}$-re abban az értelemben, hogy $f_\theta(t(x)) = f_\theta(x)$.

Bizonyítás. Mivel az identitás $\mathrm{id} \in \mathcal{T}$, válasszuk $t_1 = t$ és $t_2 = \mathrm{id}$ értékeket. Ekkor a feltételből $f_\theta(t(x)) = f_\theta(\mathrm{id}(x)) = f_\theta(x)$.

Az invariancia önmagában nem hiba; akkor válik hibássá, ha a downstream cél olyan attribútumot használ, amelyet a transzformáció megváltoztat. Ezt a következő állítás rögzíti.

Állítás (rossz invariancia mint downstream-korlát). Legyen $g : \mathcal{X} \to \mathcal{Y}$ a downstream igazságfüggvény. Tegyük fel, hogy létezik $x \in \mathcal{X}$ és $t \in \mathcal{T}$ úgy, hogy $g(t(x)) \neq g(x)$. Ha f_θ invariáns a t transzformációra, akkor nincs olyan $h_\phi : \mathbb{R}^d \to \mathcal{Y}$ downstream fej, amelyre a $h_\phi \circ f_\theta$ kompozíció mind x-en, mind $t(x)$-en helyes.

Bizonyítás. Az invariancia miatt $f_\theta(t(x)) = f_\theta(x)$, ezért bármely h_ϕ-re

$$h_\phi(f_\theta(t(x))) = h_\phi(f_\theta(x)).$$

Ha a kompozíció mindkét ponton helyes lenne, akkor a bal oldal $g(t(x))$-szel, a jobb oldal $g(x)$-szel lenne egyenlő, tehát $g(t(x)) = g(x)$ következne. Ez ellentmond a feltételnek.

A GlyphLab esetén a 90°-os forgatás a klasszikus ellenpélda. Legyen $t = R_{90}$. A downstream cél g az orientáció: $g = 0$ a vertical, $g = 1$ a horizontal osztályra. Egy vertical képre $g(x) = 0$, de a 90°-kal elforgatott kép orientációja horizontal, azaz $g(R_{90}(x)) = 1$. Az állítás feltétele teljesül, tehát ha a pretext forgatás-invarianciát tanít, akkor **bármilyen** downstream fej legalább az egyik nézeten téved: nem létezik tökéletes osztályozó a két egymásnak megfelelő nézeten. A diagonal osztályra a vízszintes tükrözés hasonló: a generátor csak az egyik ferde irányt tartalmazza, ezért a tükrözés a 2-es osztályon kívüli, új célértelmezést igénylő képet adna; a vertical és a horizontal osztályon viszont a tükrözés címkemegőrző.

A gyakorlatban a háló nem éri el az egzakt invarianciát; ilyenkor az állítás csak közelítőleg érvényes, és a downstream hiba nagysága a tényleges reprezentációtól függ, pusztán az invarianciaigényből nem vezethető le. A fontos tanulság: **a pretext transzformációk megválasztása nem semleges**, azt a downstream céllal együtt kell ellenőrizni.

30.1.4 4. Eredeti kézi kis példa és minimális szemléltető kód

Kétdimenziós példa. Legyen $\mathcal{X} = \mathbb{R}^2$, az augmentáció $t(x) = -x$ (központi tükrözés), a downstream cél $g(x) = \operatorname{sgn}(x_1)$. Konkrétan $x = (2, 1)$ esetén $t(x) = (-2, -1)$, és $g(x) = +1$, míg $g(t(x)) = -1$. Ha egy f_θ reprezentáció invariáns t-re, akkor $f_\theta(2, 1) = f_\theta(-2, -1)$. Bármely h downstream fejre

$$h(f_\theta(2, 1)) = h(f_\theta(-2, -1)),$$

tehát a két ponton a kimenet nem lehet egyszerre a két különböző címke. Mivel az invariancia miatt a modell a két ponton ugyanazt a kimenetet adja, a két pont közül legalább az egyiken a 0-1 veszteség 1.

Az alábbi kód a GlyphLab metaadataiból szemlélteti, hogy a 90°-os forgatás nem címkemegőrző a vertical osztályra. A kód a mellékelt GlyphLab-adaton lefutott (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0); a mért kimenet a blokk után látható.

```
# munkakönyvtár: lab
import numpy as np
import pandas as pd

meta = pd.read_csv("data/metadata.csv")
assert (meta["index"].to_numpy() == np.arange(len(meta))).all()  # sor = képindex
# válasszunk egy vertical (0) train képet a scene-szintű splitből
row = meta[(meta.split == "train") & (meta.label == 0)].iloc[0]
idx = int(row["index"])
images = np.load("data/images.npy")
masks = np.load("data/masks.npy")
img = images[idx]                                  # (1, 24, 24) float32, NCHW
rot = np.rot90(img[0], k=1)                        # 90 fokos forgatás a térbeli síkon

def mask_orientation(mask):
    """A tengelyek hosszából: h>w vertical (0), w>h horizontal (1)."""
    ys, xs = np.nonzero(mask)
    h = ys.max() - ys.min() + 1
    w = xs.max() - xs.min() + 1
    assert h != w, "a maszk tengelyei egyenlők; az irány nem dönthető el"
    return 0 if h > w else 1

label_original = mask_orientation(masks[idx, 0])
label_rotated = mask_orientation(np.rot90(masks[idx, 0], k=1))
print(f"eredeti label: {int(row['label'])}, maszkból: {label_original}, "
      f"90 fokos forgatás után maszkból: {label_rotated}, "
      f"alak: {rot.shape}, dtype: {img.dtype}")
assert int(row["label"]) == label_original == 0
assert label_rotated == 1
```

A kód csak a kérdéses transzformáció címkemegőrzését ellenőrzi; ha a `rot` képet a pretext pozitív párjaként használnánk, a reprezentáció a két orientációt összemosná. A mért kimenet:

```
eredeti label: 0, maszkból: 0, 90 fokos forgatás után maszkból: 1, alak: (24, 24), dtype:
↪  float32
```

A futás a train split első vertical képén (label = 0) a maszkból is vertical irányt olvasott ki, a 90°-os forgatás után pedig horizontal (1) irányt; a képtömb alakja $(24, 24)$, dtype-ja float32. Ez megerősíti, hogy a forgatás a vertical célra nem címkemegőrző transzformáció.

30.1.5 5. GlyphLab laborterv: címkemegőrző nézetképzés és frozen-probe értékelés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja a pretext–downstream szerződés megtervezése a GlyphLab adaton. **Az alábbi terv nem elvégzett kísérlet; számszerű kimenetet nem állítunk.**

Nézetképzés. A pretext feladat pozitív párja két, címkemegőrző transzformációval készült nézet ugyanabból a képből. A GlyphLab orientáció-céljához megengedett transzformációk:

- kis térbeli eltolás $\Delta \in \{-2, -1, 0, 1, 2\}^2$ véletlen mintával, tükrözéses kitöltéssel (reflection padding);
- kis Gauss-zaj $\varepsilon \sim \mathcal{N}(0, 0{,}02^2)$, majd `clip(0,1)`;
- intenzitásskála $\alpha \in [0{,}8;\ 1{,}2]$ véletlen mintával.

Kizárt transzformációk: 90°-os forgatás (vertical ↔ horizontal felcserélés), valamint a vízszintes tükrözés a diagonal osztályon (a másik átlóba visz, amely nincs a címkehalmazban); a vertical és horizontal osztályon a tükrözés címkemegőrző, de a közös pretext augmentáció így is kizárja. A kizárás indoka a 3. szakasz állítása.

Backbone. Kis CPU-s CNN, NCHW bemenettel: Conv2d(1, 8, kernel=3, padding=1) + ReLU + Conv2d(8, 16, kernel=3, stride=2, padding=1) + ReLU + AdaptiveAvgPool2d(4) + Linear(256, 16). A reprezentáció dimenziója $d = 16$. A paraméterszám nagyságrendileg néhány ezer; a 20.4. V. ellenőrző pont: indokolt CNN-választás döntési lap szerint scratch tanítás, előtanított súly nélkül.

Pretext tanítás. A train split 240 képéből batch-enként nézetpárokat generálunk. A pretext-fej a backbone 16-dimenziós kimenetére ülő kétrétegű MLP ($16 \to 32 \to 16$); a kontrasztív loss a következő 30.2. Kontrasztív cél és temperature egységben bevezetett NT-Xent, a temperature paraméter $\tau = 0{,}2$. Optimalizáló Adam, tanulási ráta 10^{-3}, batch 64, 50 epoch; a seed `20261020`, dtype float32, device CPU. A val split 60 képén a pretext loss csak nyomon követésre szolgál.

Frozen probe. A pretext után a backbone paramétereit `requires_grad_(False)` módon rögzítjük. A train és val képekből `eval()` módban, `inference_mode()` alatt kiszámoljuk a 16-dimenziós reprezentációkat. Ezután egy lineáris fejet tanítunk: `Linear(16, 3)`, CrossEntropyLoss, Adam, tanulási ráta 10^{-2}, batch 32, legfeljebb 50 epoch. A legjobb fej kiválasztása **kizárólag a val split accuracy alapján** történik.

Végső kiértékelés. A befagyasztott backbone és a kiválasztott fej a test split 60 képén értékelendő. A metrika accuracy, plusz osztályonkénti bontás és scene-rétegzett bootstrap CI a 23.4. Hibaszeletek és alkalmazási határ szerint. A corrupt tesztet itt nem használjuk modellválasztásra; az opcionálisan, a protokoll végén, befagyasztott modellen értékelhető a 22.4. Augmentációs kísérlet és domainváltozás szerződés alapján.

Baseline. Ugyanaz a lineáris fej a nyers, $24 \times 24 = 576$ dimenziós pixeltéren tanítva. A két fej paraméterszáma eltér: a nyers pixeles fej $576 \cdot 3 + 3 = 1731$, a probe fej $16 \cdot 3 + 3 = 51$ paraméter, ezért az összevetés elsősorban a bemeneti reprezentáció különbségét méri, nem azonos kapacitású fejek versenye.

Ellenőrzés. A nézetképző függvény minden hívás után megtartja a `scene_id`-t és a labelt; a split határok nem sérülnek. A downstream logitokra nem alkalmazunk külön stabil logsumexp-képletet: a CrossEntropyLoss belsőleg stabil log-softmaxot számol (7.4. Numerikus stabilitás és jelentésmegőrző implementáció). A 60 tesztkép 30 jelenetből áll, ezért a scene-rétegzett bootstrap CI szélessége nagy lesz; a pontbecslés mellé ezt a bizonytalanságot is jelenteni kell.

Értelmezési határ. A GlyphLab toy adat, három geometriai osztállyal; az eredmény nem természetes képi benchmark, és a frozen-probe pontszám csak azt mutatja, hogy ez a reprezentáció ezen a downstream feladaton mit őrzött meg.

30.1.6 6. Feladatok

F1 (fogalmi, nehézség 1). Definiálja a pretext és a downstream feladat közötti különbséget, és adjon egy konkrét példapárt a GlyphLab adaton, ahol a két cél más kimenetet és más veszteséget használ. Adja meg mindkettő cél-, target- és output-tenzor alakját.

F2 (levezetési, nehézség 2). Legyen $\mathcal{T} = \{R_{90}\}$, a 90°-os forgatás. Bizonyítsa be a 3. szakasz állításának segítségével, hogy ha egy reprezentáció invariáns R_{90}-re, akkor a GlyphLab vertical/horizontal downstream feladaton nincs tökéletes osztályozó. Nevezze meg a bizonyításban használt x és g értékét.

F3 (implementációs, nehézség 1). Írjon Python-függvényt `is_label_preserving(transform, img, label)` néven, amely a GlyphLab adatból betöltött $(1, 24, 24)$ float32 kép és egy `(1,24,24) -> (1,24,24)` transzformáció esetén ellenőrzi, hogy a transzformált kép várható orientációja megegyezik-e a megadott `label` értékkel a forgatási és tükrözési szabályok szerint. A függvény a 90°-os forgatásra mindhárom címkére a címke-megőrzés hiányát jelezze; a vízszintes tükrözésre a vertical és a horizontal címkére igaz, a diagonalra hamis értéket adjon. A megoldásban adja meg a teljes függvénykódot és két tesztesetet.

F4 (hibakeresési, nehézség 2). Egy hallgató a pretext feladatban 90°-os forgatást használ a pozitív nézetpárokhoz, mert „a forgatás úgyis csak zajos nézetet ad". A downstream frozen-probe accuracy romlik a nyers pixeles baseline-hoz képest. Lokalizálja a hibát: mi a pontos oka, mi a következménye a reprezentációra, és mi a javítás? Ellenőrizze a javítást a 3. szakasz állításával.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen frozen-probe kísérletet egy új downstream célra: a GlyphLab képeken a foreground objektum **jelenlétének** detektálása (van-e foreground pixel, bináris cél), miközben a pretext változatlan marad. Adja meg a protokollt: megfigyelési egység, split, backbone, downstream fej, baseline, metrika, erőforrás és érvényességi korlát. Indokolja, miért nem használható a 90°-os forgatás ebben a tervben sem.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

30.1.7 7. Megoldások és indoklás

F1 megoldása. A pretext feladat célja a reprezentáció tanítása emberi címke nélkül; a GlyphLab példában a pretext pozitív párjának célja a két nézet összetartozása. Ezt a következő egységben kontrasztív loss méri, de a target fogalma itt is megadható. Ha a batch N képe $N/2$ nézetpárból áll, akkor a pretext target $P \in \{0, \dots, N-1\}^N$ int64 vektor, ahol P_i az i-edik anchor pozitív párjának batch-indexe; „aktív index" alatt ezt a P_i pozitív indexet értjük (tehát $P_{P_i} = i$). A pretext output a reprezentáció-tenzor $Z \in \mathbb{R}^{N \times d}$. A downstream feladat az orientáció osztályozása: target $y \in \{0, 1, 2\}^N$, output logit $S \in \mathbb{R}^{N \times 3}$, veszteség CrossEntropyLoss. A két cél más kimenetet és

más hibamértéket használ, ezért a pretext pontossága nem fordítható közvetlenül downstream pontosságra.

F2 megoldása. Legyen x egy vertical kép, $g(x) = 0$. A 90°-os forgatás után a kép orientációja horizontal, ezért $g(R_{90}(x)) = 1 \neq 0$. Tegyük fel, hogy f_θ invariáns R_{90}-re: $f_\theta(R_{90}(x)) = f_\theta(x)$. Bármely h downstream fejre $h(f_\theta(R_{90}(x))) = h(f_\theta(x))$. Ha $h \circ f$ tökéletes lenne, akkor a bal oldal 1, a jobb oldal 0 lenne, ami ellentmondás. Tehát nincs tökéletes downstream osztályozó. A bizonyítás során x egy tetszőleges vertical kép, g az orientációt adó igazságfüggvény, $t = R_{90}$.

F3 megoldása.

```
# munkakönyvtár: lab
import numpy as np

def rot90(img):
    """90 fokos forgatás a térbeli tengelyeken (NCHW bemenet)."""
    return np.rot90(img, k=1, axes=(-2, -1)).copy()

def flip_lr(img):
    """Vízszintes tükrözés (NCHW bemenet)."""
    return np.flip(img, axis=-1).copy()

def is_label_preserving(transform, img, label):
    """Ellenőrzi, hogy a transzformáció megőrzi-e a GlyphLab címkét.

    img: (1, 24, 24) float32 NCHW kép.
    label: 0=vertical, 1=horizontal, 2=diagonal (a generátorban csak egy átlóirány).
    Visszatérés: True, ha a transzformált kép ugyanabba az osztályba tartozik.
    """
    x = np.asarray(img, dtype=np.float32)
    out = transform(x)
    if out.shape != x.shape or not np.isfinite(out).all():
        raise ValueError("A transzformált kép alakja vagy értékkészlete hibás.")
    if transform is rot90:
        # 90 fok: vertical <-> horizontal, a diagonal pedig a másik átlóba
        # fordul, amely nincs a címkehalmazban; egyik osztályra sem megőrző.
        return False
    if transform is flip_lr:
        # A vertical és a horizontal orientáció tükrözésre változatlan,
        # a diagonális vonal viszont a másik átlóba fordul.
        return label in (0, 1)
    raise NotImplementedError("A transzformáció szabálya nincs megadva.")

# Tesztesetek; a képek betöltése a 4. szakasz szerint történik.
# img_v: vertical, img_h: horizontal, img_d: diagonal kép, alakjuk (1, 24, 24).
assert is_label_preserving(rot90, img_v, 0) is False    # vertical -> horizontal
assert is_label_preserving(rot90, img_d, 2) is False    # diagonal -> másik átló
assert is_label_preserving(flip_lr, img_v, 0) is True   # vertical marad vertical
assert is_label_preserving(flip_lr, img_d, 2) is False  # diagonal -> másik átló
```

A megoldás kulcsa, hogy a transzformáció azonossága (`transform is rot90`, illetve `transform is flip_lr`) alapján döntünk, nem a függvény neve alapján: a laborban a `np.rot90` hívását névvel ellátott `rot90` függvény csomagolja be, mert egy `lambda` `__name__` értéke `"<lambda>"`, és a névre épülő elágazás elbukna. A 90°-os forgatás a vertical/horizontal párt felcseréli, a diagonal képet pedig a másik átlóba viszi; mivel a generátor csak egy átlóirányt állít elő, a forgatott diagonal kép nem kap címkét, ezért a forgatás egyik osztályra sem címkemegőrző. A tükrözés a vertical és a horizontal osztályt megőrzi, a diagonálist nem.

F4 megoldása. A hiba oka: a 90°-os forgatás a vertical és horizontal osztályokat felcseréli, ezért a pretext pozitív párjaiban a két nézet különböző downstream címkéjű képeket köt össze. A reprezentáció a forgatás-invariancia irányába tolódik, ami a 3. szakasz állítása szerint felső

korlátot szab a downstream pontosságnak: a vertical és horizontal jellemzők összemosódnak. A következmény a downstream accuracy romlása. A javítás: a forgatást eltávolítjuk a pretext augmentációk közül, és csak a címkemegőrző transzformációkat (kis eltolás, kis zaj, intenzitásskála) használjuk. Az ellenőrzés nem merül ki az accuracy-ben: a korrigált pretext után a frozen-probe accuracy nem romlik a nyers pixeles baseline alá, a zavaró mátrixban a vertical/horizontal átfedés csökken, és a két osztály centroidjának távolsága a within-class szóráshoz képest nő; az accuracy önmagában az osztályegyensúly miatt félrevezető lehet, ezért a per-class recallt és a zavaró mátrixot is jelenteni kell.

F5 megoldásminta. A downstream cél: bináris foreground-detekció, ahol a cél $b_i = 1$, ha a kép foreground maszkja nemüres, különben 0. A megfigyelési egység továbbra is a `scene_id`; a train/val/test split változatlan. A backbone a pretext után lefagyasztva; a downstream fej `Linear(16, 1)`, BCEWithLogitsLoss a 7.3. Bináris, multilabel és regressziós cél szerint; a bináris cél `float32` alakja $(N, 1)$, azaz a logit és a cél alakja megegyezik, és a `BCEWithLogitsLoss` nem `int64` címkét vár, hanem 0/1 float értéket. A baseline lineáris fej nyers pixeleken, `Linear(576, 1)`. Elsődleges metrika az AUC (küszöbfüggetlen, a foreground-arányra kevésbé érzékeny), másodlagos a 0,5-es küszöb melletti accuracy és az osztályonkénti recall, jelenetszintű bootstrap CI-vel. Erőforrás: CPU, batch 32. Érvényességi korlát: a GlyphLab adat kis méretű és procedurális, ezért a következtetés csak erre az adatszerződésre érvényes. A 90°-os forgatás a jelenlét-detekciónál címkemegőrző, mert a foreground jelenléte forgatásinvariáns, ezért ebben a downstream célban önmagában nem tiltott. A terv mégis a közös, az orientációhoz is címkemegőrző augmentációcsaládot tartja meg, hogy a pretext reprezentáció ne tanuljon el egy olyan invarianciát, amelyet egy későbbi orientációérzékeny feladat nem tolerál; ha a downstream cél kizárólag a jelenlét lenne, a forgatás bevonható, és külön kísérleti ágként mérhető.

30.1.8 8. Tipikus tévedés ellenpéldával

A leggyakoribb tévedés: **„a két nézet előállításához használt transzformáció semleges, mert a pretext feladat úgyis csak egy reprezentációt tanít."** Ez azért hibás, mert a pretext cél explicit invarianciát kér a transzformációra, és az invariancia elveszi a downstream számára szükséges megkülönböztető információt. Az ellenpélda a fenti 90°-os forgatás a GlyphLab orientáció-célján: a forgatás-invariáns reprezentáció a vertical és horizontal osztályokat összemossa, így a downstream pontosság felső korlátja elvileg sem érheti el az 1-et ezen a két osztályon. A helyes eljárás az, hogy minden pretext transzformációra **kimondjuk**, mely attribútumokra kérünk invarianciát, és ezt összevetjük a downstream cél attribútumigényével.

30.1.9 9. Önellenőrzés és korábbi fogalom visszahívása

Ellenőrző lista: (1) a pretext augmentációk minden osztályra címkemegőrzők-e; (2) a downstream best checkpoint kiválasztása csak a val spliten történik-e; (3) a frozen probe során a backbone valóban `eval()` módban és `inference_mode()` alatt fut-e; (4) a downstream split scene-szintű függetlensége megmaradt-e.

Válaszkulcs. (1) Igen, a választott augmentációk a három orientációs osztályt permutálják, nem keverik. (2) Igen, a checkpointot kizárólag a validációs loss választja. (3) Igen, a backbone `eval()` és `inference_mode()` alatt fut. (4) Igen, a split scene-szintű maradt.

Visszahívás a 29.1. Lineáris és nemlineáris autoencoder egységből: az autoencoder rekonstrukciós célja is pretext feladat, de ott a szűk keresztmetszet nem garantálja, hogy a latent kód a downstream szempontból hasznos attribútumokat őriz meg; a kontrasztív pretext ezzel szemben explicit invarianciát tanít, ezért a transzformációk megválasztása közvetlen downstream következménnyel jár. A 18.1. Reziduális blokk és identitásút reziduális blokkja az identitás-úton keresztül megőrzi

a bemeneti információ egy részét, ami segíthet elkerülni a teljes információvesztést; a pretext invariancia viszont éppen arra kényszerít, hogy bizonyos transzformációkra a reprezentáció ne változzon, ezért a két hatás ellentétes irányú. A reziduális kapcsolat önmagában nem garantálja a downstream információ megőrzését, ha a pretext loss az invarianciát jutalmazza; ezért kell a transzformációkat a downstream céllal összevetni.

30.1.10 10. Rövid összefoglaló

A pretext feladat célja a reprezentáció tanítása emberi címke nélkül, a downstream feladat a tényleges alkalmazás. A nézetpár-invariancia formálisan azt kéri, hogy $f_\theta(t(x)) = f_\theta(x)$ minden t augmentációra. Ha létezik x, amelyre a downstream igazságfüggvény $g(t(x)) \neq g(x)$, akkor nincs tökéletes downstream osztályozó: a bizonyítás az invarianciából következő $h(f(t(x))) = h(f(x))$ egyenlőségen alapul. A GlyphLab orientáció-célja ezért kizárja a 90°-os forgatást és a tükrözést. A laborterv címkemegőrző nézetekkel, 16-dimenziós backbone-nal és frozen-probe értékeléssel méri a reprezentáció átviteli minőségét.

30.1.11 Következő alkalmazás

A 30.2. Kontrasztív cél és temperature egységben a kontrasztív veszteséget és a temperature paramétert vezetjük be; a jelen szakasz invarianciaelemzése ott azt adja, hogy mikor nem szabad egy transzformációt a pozitív pár generálására használni.

30.1.12 Források

- [P17] Chen et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. https://arxiv.org/abs/2002.05709.

30.2 Kontrasztív cél és temperature

30.2.1 1. Cél, előismeret és induló kérdés

A 30.1. Pretext és downstream feladat egységben elválasztottuk a pretext feladatot a downstream feladattól, és rögzítettük, hogy a reprezentációtanulás célja olyan z vektor előállítása, amely a feladat szempontjából hasznos invarianciákat őrzi meg. Ott a pozitív párt csak elnagyoltan, „ugyanannak a jelenetnek két augmentált nézete” formájában adtuk meg. Ebben az alfejezetben ezt a szerződést pontos, kiszámítható veszteséggé alakítjuk.

A 29.2. Denoising és korrupciós modell denoising célban a pár aszimmetrikus volt: zajos bemenetből tiszta target. A kontrasztív cél ezzel szemben két egyenrangú nézetet illeszt egymáshoz, ezért nincs kitüntetett „tiszta” oldal. A 26.2. Egy- és kétlépcsős detektor detektoros párosítási szabályhoz hasonlóan itt is előre rögzíteni kell, mely indexek alkotnak pozitív párt; ezt nem a modell hasonlósági pontszáma dönti el. A 18.2. Batch normalization train és eval módban tanulsága pedig az, hogy a batchtől függő mennyiségek train és eval szerepét külön kell adminisztrálni: a kontrasztív nevező is batchfüggő, de csak előtanítási cél, nem kiértékelési metrika.

Induló kérdés. Adott négy normalizált reprezentáció:

$$z_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad z_1 = \begin{pmatrix} 0{,}8 \\ 0{,}6 \end{pmatrix}, \quad z_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad z_3 = \begin{pmatrix} 0{,}6 \\ 0{,}8 \end{pmatrix}.$$

A pozitív párok $(0, 1)$ és $(2, 3)$. Mennyi a kontrasztív loss $\tau = 1$ mellett, ha minden mintához az összes többi minta szerepel a nevezőben? A választ a 4. szakaszban kézzel ellenőrizzük; előzetes becslésként érdemes megfigyelni, hogy a pozitív párok koszinusz-hasonlósága 0,8, de a z_0–z_3 negatív pár hasonlósága 0,6 is viszonylag magas, tehát a loss nem lesz nulla.

30.2.2 2. Definíciók: hasonlósági mátrix, pozitív target, maszk és hőmérséklet

Legyen egy minibatch reprezentációja

$$Z \in \mathbb{R}^{N \times d},$$

ahol az i-edik sort $z_i \in \mathbb{R}^d$ jelöli. Feltesszük, hogy a sorok már ℓ_2-normalizáltak, azaz $\|z_i\|_2 = 1$. Ekkor a **hasonlósági mátrix**

$$S = ZZ^\top \in [-1, 1]^{N \times N}, \qquad S_{ij} = z_i^\top z_j,$$

tehát S_{ij} a két vektor koszinusz-hasonlósága. A z vektor előállításáért felelős hálózat kimenetét a gyakorlatban a veszteségszámítás előtt normalizáljuk, mert a cél nem az abszolút norma, hanem az irányok együttállása.

A **pozitív párt** egy előre rögzített, adatból származó hozzárendelés. Legyen $p(i) \in \{0, \dots, N-1\}$ az i indexű minta pozitív párja. A páros szerződés a következő:

1. $p(i) \neq i$ minden i-re;
2. $p(p(i)) = i$ (szimmetrikus pár);
3. minden sorban pontosan egy pozitív pár van.

A pozitív hozzárendelést a $P \in \{0, 1\}^{N \times N}$ maszkkal is leírhatjuk: $P_{ij} = 1$ pontosan akkor, ha $j = p(i)$, egyébként 0. A kontrasztív tanulásban ez a maszk a **target**: nem a modell által megjósolt osztály, hanem az adatelőkészítés által garantált párkapcsolat. A függelék A. szakaszában leírt GlyphLab-szerű adaton például ugyanazon scene azonos orientációjú, két különböző zajrealizációjú nézete lehet pozitív pár; a 90 fokos forgatás viszont csak akkor, ha a downstream feladat valóban invariáns az orientációra, ami az 1.1. Modell, tanulás és felhasználási szerződés alfejezetben definiált háromosztályos orientációs cél esetén nem áll fenn.

A **hőmérséklet** (temperature) $\tau > 0$ egy pozitív skalár, amely a hasonlóságok exponenciális skálázását szabályozza. A τ nem valószínűség, nem küszöb és nem margin. Az S_{ij}/τ hányadost **skálázott logitnak** nevezzük; dimenzió nélküli, mert S_{ij} is az.

A **diagonális maszk** a $D \in \{0, 1\}^{N \times N}$ mátrix, ahol $D_{ij} = 1$ pontosan akkor, ha $i = j$. A nevezőhöz használt maszk ennek komplementere:

$$M_{ij} = 1 - D_{ij} = \mathbf{1}_{i \neq j}.$$

Tehát M minden olyan (i, j) párra 1, ahol $j \neq i$. A nevezőben minden $M_{ij} = 1$ indexű hasonlóság exponenciálisa szerepel, beleértve a pozitív párt is; a diagonális kizárás csupán az önmagával vett $S_{ii} = 1$ triviális tagot távolítja el.

A **negatívok** azok a j indexek, amelyekre $M_{ij} = 1$ és $P_{ij} = 0$. A nevező azonban a pozitív tagot is tartalmazza, ezért pontosabb a „nevező-halmaz” kifejezés, mint a „negatívok összege”. Ez a megkülönböztetés az F4 hibakeresési feladatban döntő lesz.

30.2.3 3. NT-Xent/InfoNCE: a maszkolt loss levezetése

Egy i anchorra a választott kontrasztív veszteséget a pozitív exponenciális és a nevező-halmaz exponenciálisainak hányadosaként definiáljuk:

$$\ell_i(\tau) = -\log \frac{\sum_{j=0}^{N-1} P_{ij} \exp(S_{ij}/\tau)}{\sum_{j=0}^{N-1} M_{ij} \exp(S_{ij}/\tau)}. \tag{1}$$

Mivel P minden sorában pontosan egy 1 áll, a számláló egytagú:

$$\ell_i(\tau) = -\frac{S_{i,p(i)}}{\tau} + \log \sum_{j=0}^{N-1} M_{ij} \exp(S_{ij}/\tau). \tag{2}$$

A log hányadosra vonatkozó azonosságot, $\log(a/b) = \log a - \log b$ alakban alkalmaztuk. A (2) jobb oldalának második tagja egy log-sum-exp művelet: numerikusan stabil, mert a legnagyobb logit kiemelésével számítható.

A teljes batch-veszteség az anchorok **átlaga**:

$$L(\tau) = \frac{1}{N} \sum_{i=0}^{N-1} \ell_i(\tau). \tag{3}$$

Index- és shape-ellenőrzés: S, P és M alakja egyaránt $N \times N$; minden szummáció $j = 0, \dots, N-1$ fut; ℓ_i skalár; L skalár. A diagonális $M_{ii} = 0$ garantálja, hogy a $j = i$ tag sehol sem szerepel a nevezőben. A pozitív pár a nevezőben marad, mert $M_{i,p(i)} = 1$ a $p(i) \neq i$ feltétel miatt.

Ez a veszteség az irodalomban **NT-Xent** (normalized temperature-scaled cross entropy) néven ismert kontrasztív softmax N-mintás, páros változata. A kontrasztív prediktív kódolás (contrastive predictive coding, CPC) és az InfoNCE cél bevezetése van den Oord, Li és Vinyals (2018) munkája. Az **InfoNCE** elnevezést tágabb értelemben használjuk az olyan kontrasztív célokra, ahol egy pozitív mintát egy negatív halmazzal versenyeztetünk; az NT-Xent ennek normalizált, hőmérséklet-skálázott, többosztályos softmax alakja. A megnevezés nem változtat a fenti definíción: az érvényes állítások a (1)–(3) képletekből következnek, nem a névből.

A [P17] absztraktja azt emeli ki, hogy a reprezentációt a különbözően augmentált nézetek közötti egyezés kontrasztív növelésével tanítják. A jelen alfejezetben használt N-mintás képlet a kontrasztív softmax ismert szerkezetét követi; annak ellenőrzése, hogy a konkrét cikkváltozat a $2N$ bővített nézetre milyen indexelést és szimmetrizálást használ, a teljes módszertani szakasz olvasását igényli. A fejezet a saját N-mintás képletét nem azonosítja automatikusan az eredeti cikk teljes, szimmetrizált veszteségével.

A hőmérséklet hatását a következő, saját lemmában rögzítjük.

Lemma (hőmérsékleti határeset). Legyen $Z \in \mathbb{R}^{N \times d}$ soronként normalizált, $N \geq 2$, $p(i) \neq i$, és $M_{ij} = \mathbf{1}_{i \neq j}$. A (2) veszteségre minden i-re:

1. $\lim_{\tau \to \infty} \ell_i(\tau) = \log(N-1)$;
2. ha $S_{i,p(i)} > S_{ij}$ minden $j \notin \{i, p(i)\}$ esetén, akkor $\lim_{\tau \to 0+} \ell_i(\tau) = 0$;
3. legyen $A = \max_{j \neq i} S_{ij}$, $B = S_{i,p(i)}$ és $m_i = |\{j \neq i : S_{ij} = A\}|$. Ekkor $\lim_{\tau \to 0+} \tau \ell_i(\tau) = A - B \geq 0$; pontosabban, ha $A > B$, akkor $\ell_i(\tau) \to +\infty$, ha $A = B$, akkor $\ell_i(\tau) \to \log m_i$.

Bizonyítás. Rögzítsük az i sort. Az (1) hányados nemnegatív, mert a pozitív tag a nevezőben is szerepel, ezért a hányados legfeljebb 1, és $-\log(\cdot) \geq 0$. Ez adja a harmadik állítás végén szereplő nemnegativitást.

(1) Mivel $\lim_{\tau\to\infty} S_{ij}/\tau = 0$ minden véges S_{ij}-re, $\exp(S_{ij}/\tau) \to 1$. A nevezőben pontosan $N-1$ darab $M_{ij}=1$ tag van, a számlálóban egy darab. A hányados határértéke $1/(N-1)$, ezért a loss $\log(N-1)$.

(2) A feltétel szerint a pozitív pár egyedüli maximum a nevező indexei között, ezért $A = B$, $m_i = 1$, és $\log m_i = 0$.

(3) Legyen $A = \max_{j\neq i} S_{ij}$ és $B = S_{i,p(i)} \le A$. A legnagyobb logit kiemelésével pontosan

$$\ell_i(\tau) = \frac{A-B}{\tau} + \log \sum_{j\neq i} \exp\!\left(\frac{S_{ij}-A}{\tau}\right).$$

Mivel véges sok tag van, a második tag $\log m_i$-hez tart, ahol m_i a maximális A-t elérő indexek száma. Innen $\tau\ell_i(\tau) \to A-B$; ha $A > B$, az első tag $+\infty$-hez tart, míg ha $A = B$, a veszteség határértéke $\log m_i$. $\square$

A lemma feltételei végesek: nincs szükség mértékelméleti eszközökre, csak a véges exponenciális tagok határértékére. A lemma azt mutatja, hogy kis τ mellett a pozitívnál nagyobb negatív hasonlóság különbsége $1/\tau$ arányban növeli a veszteséget; egyenlő maximális hasonlóságoknál a holtversenyek száma $\log m_i$ határértéket ad. A puszta hasonlóságkülönbség nem azonos a veszteség határértékével. Ez a megfigyelés **heurisztikus** tervezési szabály, nem általános tanítási konvergenciatétel.

30.2.4 4. Kézi példa: négymintás páros loss, diagonális kizárással

A bevezető kérdés adataival dolgozunk, $\tau = 1$ mellett. A Z mátrix:

$$Z = \begin{pmatrix} 1 & 0 \\ 0{,}8 & 0{,}6 \\ 0 & 1 \\ 0{,}6 & 0{,}8 \end{pmatrix}.$$

A sorok normája valóban 1, tehát $S = ZZ^\top$ közvetlenül koszinusz-hasonlóság. A szorzat:

$$S = \begin{pmatrix} 1 & 0{,}8 & 0 & 0{,}6 \\ 0{,}8 & 1 & 0{,}6 & 0{,}96 \\ 0 & 0{,}6 & 1 & 0{,}8 \\ 0{,}6 & 0{,}96 & 0{,}8 & 1 \end{pmatrix}.$$

A pozitív párok: $p(0)=1$, $p(1)=0$, $p(2)=3$, $p(3)=2$. A P maszkban tehát az $(0,1)$, $(1,0)$, $(2,3)$, $(3,2)$ elemek 1-esek. A nevezőmaszk $M = \mathbf{1} - \mathbb{I}_4$, tehát minden i-hez a másik három index.

Első sor, $i=0$. A nevező tagjai $j = 1,2,3$. $\tau = 1$ mellett:

$$\exp(0{,}8) \approx 2{,}2255409, \qquad \exp(0) = 1, \qquad \exp(0{,}6) \approx 1{,}8221188.$$

A nevező összege 5,0476597, a pozitív tag 2,2255409, így

$$\ell_0 = -\log \frac{2{,}2255409}{5{,}0476597} \approx 0{,}8189.$$

A második sor ($i = 1$, 0-alapú indexelés). A pozitív $S_{10} = 0{,}8$; a másik két nevezőtag $S_{12} = 0{,}6$ és $S_{13} = 0{,}8 \cdot 0{,}6 + 0{,}6 \cdot 0{,}8 = 0{,}96$ (a két szorzat egyike 0,48, de S_{13} a kettő összege). A nevező:

$$2{,}2255409 + 1{,}8221188 + \exp(0{,}96) \approx 2{,}2255409 + 1{,}8221188 + 2{,}6116965 = 6{,}6593562,$$

tehát

$$\ell_1 = -\log \frac{2{,}2255409}{6{,}6593562} \approx 1{,}0960.$$

Harmadik sor, $i = 2$. A szimmetria miatt $\ell_2 = \ell_0 \approx 0{,}8189$; **negyedik sor, $i = 3$:** $\ell_3 = \ell_1 \approx 1{,}0960$. A teljes loss:

$$L = \frac{0{,}8189 + 1{,}0960 + 0{,}8189 + 1{,}0960}{4} \approx 0{,}9575.$$

A példa azt is megmutatja, miért nem nulla a loss: a pozitív párok hasonlósága 0,8, de a z_0–z_3 és z_1–z_2 negatív párok hasonlósága 0,6, a z_1–z_3 negatív páré pedig 0,96, még a pozitívnál is nagyobb; ezért a nevező nem kizárólag a pozitív tagból áll, és az $i = 1$, $i = 3$ sorok vesztesége nagyobb.

30.2.5 5. Labor: kézi és kódolt páros loss

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor munkakönyvtára `lab/n118/`. A bemenet a fenti Z mátrix, a pozitív párok explicit listája, $\tau = 1$. A kód `torch.float64` pontossággal, CPU-n fut, és a diagonálist a `masked_fill` művelettel zárja ki a log-sum-expből.

```
# lab/n118/nt_xent_pairs.py
import math
import numpy as np
import torch

# Input: normalizált reprezentációk, N x d
z_np = np.array([
    [1.0, 0.0],
    [0.8, 0.6],
    [0.0, 1.0],
    [0.6, 0.8],
], dtype=np.float64)

tau = 1.0
# A pozitív párok irányított indexelése: p(0)=1, p(1)=0, p(2)=3, p(3)=2
device = torch.device("cpu")
pos = torch.tensor([1, 0, 3, 2], dtype=torch.int64, device=device)
torch.manual_seed(20261020)  # nincs véletlen ág
# a seed csak a jegyzőkönyv egységesítéséhez szerepel
torch.set_num_threads(1)

z = torch.tensor(z_np, dtype=torch.float64, device=device)

# Normalizálás: a labor bemenete már normált; az alábbi ellenőrzés a nulla normát szűri ki
norms = z.norm(dim=1, keepdim=True)
if not torch.all(norms > 0):
    raise ValueError("Nulla normájú reprezentáció nem normalizálható.")
z = z / norms
```

```
S = z @ z.T                     # alak: (N, N), [-1, 1]
logits = S / tau                # alak: (N, N)

N = z.shape[0]
den_mask = torch.ones(N, N, dtype=torch.bool, device=device)
den_mask.fill_diagonal_(False)  # diagonális kizárás: M_ii = 0

neg_inf = torch.tensor(-math.inf, dtype=torch.float64, device=device)
masked_logits = logits.masked_fill(~den_mask, neg_inf)
lse = torch.logsumexp(masked_logits, dim=1)          # alak: (N,)
pos_logit = logits[torch.arange(N, device=device), pos]  # alak: (N,)

per_row = lse - pos_logit       # = -log(hányados)
loss = per_row.mean()           # batch-átlag

assert S.shape == (N, N)
assert torch.allclose(S, S.T)
assert torch.allclose(torch.diagonal(S), torch.ones(N, dtype=torch.float64))
assert abs(loss.item() - 0.957474) < 1e-6

print(f"S:\n{S}")
print(f"per-row loss: {per_row.detach().numpy()}")
print(f"mean NT-Xent (tau={tau}): {loss.item():.6f}")
```

A blokk utáni ellenőrző sor a (2)–(3) képletekből számított értéket adja. A kód a fenti bemeneten lefutott (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, float64, seed 20261020, egy szál); a `python lab/n118/nt_xent_pairs.py` parancs mért kimenete:

```
S:
tensor([[1.0000, 0.8000, 0.0000, 0.6000],
        [0.8000, 1.0000, 0.6000, 0.9600],
        [0.0000, 0.6000, 1.0000, 0.8000],
        [0.6000, 0.9600, 0.8000, 1.0000]], dtype=torch.float64)
per-row loss: [0.81892472 1.09602281 0.81892472 1.09602281]
mean NT-Xent (tau=1.0): 0.957474
```

A sorfüggetlen loss-ok a kézi példa 0,8189, 1,0960, 0,8189, 1,0960 értékeit adják; a batch-átlag 0,957474. Az ellenőrzés toleranciája 10^{-6} abszolút hiba: a kézi példában a közelítő exponenciális értékek miatt az utolsó jegyek eltérhetnek, de az első öt tizedesjegynek egyeznie kell.

30.2.6 6. Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért kötelező a diagonális kizárás, és mutassa meg egy konkrét háromelemű példán, hogy $M_{ii} = 1$ esetén a nevezőt a triviális önhasonlóság dominálhatja. Adja meg a helyes M maszkot.

F2 (levezetési/bizonyítási, nehézség 2). Vizsgáljunk egyetlen anchor-sort három reprezentáció közül: $N = 3$ és $p(0) = 1$. A többi anchor párját itt nem definiáljuk, mert a teljes, szimmetrikus és önpár nélküli párosítás páratlan N mellett nem létezik. A 0-adik sorra $S_{01} = 0{,}9$, $S_{02} = -0{,}1$. Számítsa ki a lemma alapján $\lim_{\tau \to 0+} \ell_0(\tau)$ és $\lim_{\tau \to \infty} \ell_0(\tau)$ értékét. Indokolja meg, hogy melyik határérték érzékeny a negatív S_{02} pontos értékére.

F3 (implementációs, nehézség 1). Futtassa az 5. szakasz kódját, majd módosítsa a bemenetet $z_3 = (-0{,}6; 0{,}8)$-ra úgy, hogy a $(2, 3)$ pár pozitív, a $(0, 3)$ negatív hasonlóság $-0{,}6$ legyen. Adja meg a négy soronkénti loss értékét és a batch-átlagot 10^{-4} toleranciával. A kódban ne változtassa meg a maszk definícióját.

F4 (hibakeresési, nehézség 2). Egy hallgató a nevezőt így számítja:

```
neg_mask = den_mask.clone()
neg_mask[torch.arange(N, device=pos.device), pos] = False   # a pozitívat is kizárja
lse_bad = torch.logsumexp(logits.masked_fill(~neg_mask, -math.inf), dim=1)
loss_bad = (lse_bad - pos_logit).mean()
```

Magyarázza el a hiba okát és következményét. Ellenőrizze a 4. szakasz $i = 0$ során, hogy a rossz nevezővel kapott loss kisebb-e a helyesnél, és adja meg a javítás után érvényes invariánst, amely a helyes loss nemnegativitását garantálja.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen kontrasztív előtanítási kísérletet a GlyphLab orientációs feladatra, amelyben az egyetlen változó tényező a pozitív pár definíciója: (A) a saját jelenet másik nézete azonos orientációval, más zajrealizációval; (B) ugyanazon jelenet két nézete, de az egyik 90 fokkal elforgatva. Írja le a protokollt: pretext adat, downstream frozen probe, scene-alapú split, baseline és metrika. Indokolja, miért nem szabad a downstream címkét a pozitív pár definíciójába beengedni, és miért várható, hogy a (B) változat a háromosztályos orientációs célon gyengébb lesz.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

30.2.7 7. Megoldások és indoklás

F1 megoldás. A diagonális kizárás nélkül a nevezőben szerepelne $S_{ii} = 1$, amely minden más hasonlóságnál legalább akkora. Kis τ esetén az önhasonlósági $\exp(1/\tau)$ tag a nevezőt növeli. Ha a pozitív pár hasonlósága szigorúan kisebb 1-nél, a hibás veszteség $\tau \to 0+$ mellett végtelenhez nőhet; ha a pozitív pár is tökéletesen egyezik ($S_{i,p(i)} = 1$), az önhasonlóság és a pozitív tag holtversenye miatt a hibás veszteség $\log 2$-höz tart, nem a végtelenhez. Például $N = 3$, $z_0 = (1, 0)$, $z_1 = (1, 0)$, $z_2 = (0, 1)$ esetén $p(0) = 1$ mellett a helyes nevező $\exp(1) + \exp(0)$; ha $i = 0$ is szerepel, a nevező $\exp(1) + \exp(1) + \exp(0)$, így a pozitív hányados csökken. A helyes maszk $M = \mathbf{1} - \mathbb{I}_3$, azaz $M_{ii} = 0$ minden i-re.

F2 megoldás. A pozitív hasonlóság 0,9 szigorúan nagyobb a negatív $-0{,}1$ értéknél, tehát $A = B = 0{,}9$ és $m_0 = 1$. A lemma szerint $\lim_{\tau \to 0+} \ell_0(\tau) = \log 1 = 0$. A $\tau \to \infty$ határérték a lemma (1) pontja alapján $\log(3 - 1) = \log 2$, és független a negatív hasonlóságok értékétől. A 0 felé tartó határérték a S_{02} negatív tagtól valójában nem függ addig, amíg $S_{02} < 0{,}9$; a pozitív hasonlóság a legnagyobb, ezért a veszteség nullához tart. Ha viszont $S_{02} = 0{,}95$ lenne, akkor $A - B = 0{,}05 > 0$, és $\ell_0(\tau) \to +\infty$; ugyanakkor $\tau \ell_0(\tau) \to 0{,}05$. Ha $S_{02} = 0{,}9$ pontosan, akkor a két maximális tag miatt $\ell_0(\tau) \to \log 2$.

F3 megoldás. Az új $z_3 = (-0{,}6; 0{,}8)$ normája 1. A hasonlósági mátrix új elemei: $S_{03} = -0{,}6$, $S_{13} = 0$, $S_{23} = 0{,}8$. A soronkénti nevezők:

- $i = 0$: $\exp(0{,}8) + \exp(0) + \exp(-0{,}6) \approx 2{,}22554 + 1 + 0{,}54881 = 3{,}77435$; loss $= -\log(2{,}22554/3{,}77435) \approx 0{,}5282$.
- $i = 1$: $S_{12} = 0{,}6$, $S_{13} = 0$; nevező $\approx 2{,}22554 + 1{,}82212 + 1 = 5{,}04766$; loss $\approx 0{,}8189$.
- $i = 2$: $S_{20} = 0$, $S_{21} = 0{,}6$; nevező $\approx 1 + 1{,}82212 + 2{,}22554 = 5{,}04766$; loss $\approx 0{,}8189$.
- $i = 3$: $S_{30} = -0{,}6$, $S_{31} = 0$; nevező $\approx 0{,}54881 + 1 + 2{,}22554 = 3{,}77435$; loss $\approx 0{,}5282$.

A batch-átlag $\approx 0{,}6736$. A változás jól mutatja, hogy a negatív hasonlóság előjelváltása (S_{13}: $0{,}96 \to 0$) csökkenti a loss-t, mert a nevezőből eltűnik a domináns negatív tag.

F4 megoldás. A hiba oka, hogy a hallgató a pozitív tagot is kivette a nevezőből. A helyes NT-Xent nevezője a pozitív exponenciálist is tartalmazza; a diagonális kizárás csak az $i = i$ tagra vonatkozik. Következmény: a nevező kisebb lesz, a hányados nagyobb, a loss tehát kisebb,

és akár negatívvá is válhat, ha a pozitív exponenciális meghaladja a fennmaradó negatív tagok összegét. A 4. szakasz $i = 0$ során a rossz nevező $\exp(0) + \exp(0{,}6) = 2{,}82212$, a hányados $2{,}22554/2{,}82212 \approx 0{,}7886$, a loss 0,2376, ami valóban kisebb a helyes 0,8189-nél. A javítás után érvényes invariáns: a nevező a pozitív tagot tartalmazza, ezért a hányados mindig legfeljebb 1, és a per-példa loss mindig nemnegatív.

F5 megoldásminta. A protokoll célja annak mérése, hogy a pozitív pár definíciója milyen invarianciát ír elő. Pretext adat: a GlyphLab train split 240 képe (120 jelenet, jelenetenként két nézet). Az (A) ágban minden train képhez a pozitív pár a saját scene másik nézete; a (B) ágban a másik nézet 90 fokkal elforgatott változata. Az orientációs címke **nem** szerepel a párképzésben, mert akkor a pretext cél szivárogtatná a downstream választ. Downstream: az előtanított encoder fagyasztott kimenetéből lineáris probe a train képeken, a val split a legjobb epoch kiválasztására. Baseline: ugyanaz a háló véletlen inicializálással, csak lineáris probe tanítva. Az értékelés a rögzített test spliten mért accuracy, máshol nem használt modellválasztással. A (B) ág várhatóan gyengébb, mert a 90 fokos forgatás egy vertical nézetet horizontal nézettel párosít, így a reprezentáció a vertical–horizontal megkülönböztetést moshatja el, ami a downstream céllal ellentétes.

30.2.8 8. Gyakori tévedés, önellenőrzés és visszahívás

Gyakori tévedés. „Az NT-Xent formula automatikusan a kölcsönös információ alsó korlátja.” Ez az állítás így nem igazolható. Az információelméleti interpretáció további feltételeket igényel a kritikus függvényre, a negatív minták eloszlására és a becslő torzítására. A minibatch-nevezőben a negatívok nem függetlenek a pozitív pár eloszlásától, ezért a puszta képletazonosságból nem következik MI-korlát. A jelen egység csak a (1)–(3) veszteségfüggvényt definiálta és a hőmérsékleti határértékeket bizonyította.

Önellenőrzés. Az 5. szakasz kódjában S alakja $(4, 4)$, a `masked_fill` után a log-sum-exp alakja $(4,)$, a loss skalár. A $N = 2$ szélsőesetben minden sor nevezője csak a pozitív tagot tartalmazza, ezért minden per-példa loss 0; a kontrasztív cél valódi negatívok nélkül degenerált. Ezért a páros loss használatához legalább két pozitív pár, tehát $N \geq 4$ szükséges.

Korábbi fogalom visszahívása. A 29.2. Denoising és korrupciós modell denoising szerződésében a tiszta target szivárgása torzította a kiértékelést. Itt az analog szivárgás a downstream címke beépítése a pozitív pár definíciójába. A 26.2. Egy- és kétlépcsős detektor párosítási szabályhoz hasonlóan a pozitív hozzárendelést a tanítás előtt rögzíteni kell; a 18.2. Batch normalization train és eval módban train/eval különválasztás pedig arra figyelmeztet, hogy a batchfüggő nevező nem tekinthető a downstream modell kiértékelési statisztikájának.

30.2.9 Rövid összefoglaló

A kontrasztív loss normalizált reprezentációk hasonlósági mátrixából építkezik. Minden anchorhoz egy előre rögzített pozitív pár tartozik; a nevező az összes többi minta skálázott exponenciálisát tartalmazza, a diagonálist kizárva. A hőmérséklet a softmax koncentrációját szabályozza: $\tau \to 0+$ mellett a veszteség nullához tart, ha a pozitív pár az egyedüli maximum; maximális holtversenynél $\log m_i$-hez tart, a pozitívnál nagyobb negatív esetén pedig divergens. $\tau \to \infty$ mellett $\log(N-1)$-hez tart. A négymintás kézi és kódolt példa rögzítette az indexelést, a maszk szerepét és a numerikusan stabil log-sum-exp megvalósítást. A pozitív pár definíciója a pretext invarianciát határozza meg, ezért a downstream szempontból helytelen augmentáció nem javítható csupán a hőmérséklet hangolásával.

30.2.10 Következő alkalmazás

A 30.3. Negatív nélküli módszerek és collapse egység a negatív minták nélküli módszereket és a reprezentációs összeomlást vizsgálja. Ott azt kérdezzük, mi történik, ha a nevező negatívjait elhagyjuk, és milyen regularizáló tagok akadályozzák meg, hogy minden bemenet azonos reprezentációba essen.

30.2.11 Felhasznált források

- [P17] Chen, T., Kornblith, S., Norouzi, M. és Hinton, G. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. arXiv:2002.05709. https://arxiv.org/abs/2002.05709.
- van den Oord, A., Li, Y. és Vinyals, O. (2018). *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748. Az InfoNCE cél és a kontrasztív prediktív kódolás forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

30.3 Negatív nélküli módszerek és collapse

30.3.1 Cél, előismeret és induló kérdés

A 30.2. Kontrasztív cél és temperature egységben az InfoNCE/NT-Xent célfüggvényt vizsgáltuk: a pozitív pár hasonlóságát a negatív párokból képzett nevezővel versenyezteti. Ebben az alfejezetben azt a kérdést járjuk körül, mi történik, ha a negatív példákat teljesen elhagyjuk, és csak arra kérjük a modellt, hogy ugyanannak a jelenetnek két nézete azonos reprezentációba kerüljön. Előismeret: a 30.2. Kontrasztív cél és temperature alfejezetből a hasonlósági mátrix, a temperature és a maszk, a 29.3. Regularizált latent reprezentáció alfejezetből a latent representáció összeomlásának (*collapse*) fogalma, a 18.3. Pre-activation, kis batch és alternatív normalizálás alfejezetből a futó átlagok könyvtári kezelése. Induló becslés: ha a loss csak annyit mond, hogy „a két nézet kimenete egyezzen”, akkor a konstans encoder triviálisan teljesíti-e a feladatot? Ha igen, mi akadályozza meg a tanuló algoritmust abban, hogy ezt a degenerált megoldást válassza?

30.3.2 Teacher, student, stop-gradient és moving average

A negatív nélküli (*noncontrastive*) módszercsalád közös váza a következő. Legyen $f_\theta : \mathcal{X} \to \mathbb{R}^d$ az encoder, $x_n^{(1)}, x_n^{(2)}$ egy x_n kép két nézete, és $\hat{z}_n^{(a)} = f_\theta(x_n^{(a)})/\|f_\theta(x_n^{(a)})\|_2$ a normalizált reprezentáció, feltéve, hogy a norma pozitív. A legegyszerűbb pozitív-páros cél a normalizált átlagos négyzetes hiba:

$$L_{\mathrm{NN}}(\theta) = \frac{1}{N}\sum_{n=1}^{N} \left\| \hat{z}_n^{(1)} - \hat{z}_n^{(2)} \right\|_2^2 .$$

Ez a cél a *teacher–student* keretben bővül. Két súlyhalmazt tartunk nyilván: a θ_s *student* (online) hálózatot és a θ_t *teacher* (target) hálózatot. A veszteség

$$L(\theta_s, \theta_t, \phi) = \frac{1}{N}\sum_{n=1}^{N} \left\| q_\phi(\hat{z}_s^{(1)}(x_n^{(1)})) - \mathrm{sg}\left(\hat{z}_t^{(2)}(x_n^{(2)})\right) \right\|_2^2 ,$$

ahol $q_\phi : \mathbb{R}^d \to \mathbb{R}^d$ egy kis *predictor* fej, sg pedig a *stop-gradient* operátor. A fenti veszteség egyirányú: a student első nézetét a teacher második nézetéhez hasonlítja; a szimmetrikus változatban a két nézet szerepét felcserélve is kiszámítjuk és a két tagot átlagoljuk.

Definíció (stop-gradient). A $\mathrm{sg}(\cdot)$ operátor az argumentum értékét változatlanul hagyja az előremenő számításban, de a visszafelé számításban az argumentum felé eső Jacobi-mátrix blokkját nullával helyettesíti. Ha a loss skalár és a teacher kimenete z_t, akkor a gyakorlati frissítésben $\nabla_{\theta_t} L$ nem számolódik, miközben a z_t értéke a lossba behelyettesített, aktuális teacher-kimenet marad. A PyTorch-beli megfelelője veszteséggráfban a `tensor.detach()` (a tenzort leválasztja a gráfról), illetve a `with torch.no_grad():` blokk (a blokkon belül egyáltalán nem épül gráf); a kettő nem azonos hatókörű: a `detach()` egy konkrét tenzort szakít le, a `no_grad` egy kódrészlet összes műveletére hat.

A teacher súlyait *exponenciális mozgóátlaggal* (*moving average*) frissítjük:

$$\theta_t \leftarrow \tau_{\mathrm{ema}} \theta_t + (1 - \tau_{\mathrm{ema}}) \theta_s, \qquad \tau_{\mathrm{ema}} \in [0, 1).$$

A τ_{ema} rendszerint közel 1 (például 0,99). Ez a forma emlékeztet a BatchNorm futó átlagára, de ott statisztikákat, itt paramétervektorokat átlagolunk; a frissített mennyiség (statisztika, illetve paraméter) és annak kezelése tehát különbözik. A frissítés sorrendje lényeges: először a student veszteségén visszaterjesztünk és a student-optimizáló lép, utána a teacher a *frissített* student felé mozdul. A teacher kimenetén nincs gradiens, ezért a teacher nem tanul visszaterjesztéssel, csak a momentum frissítésen keresztül követi a studentet.

30.3.3 A negatív nélküli cél és a konstans-embedding lemma

A célfüggvény önmagában nem bünteti a reprezentáció inputfüggésének elvesztését. Ezt rögzíti a következő lemma.

Lemma (konstans embedding mint degenerált optimum). Legyen $\mathcal{X}$ tetszőleges inputhalmaz, $f_\theta : \mathcal{X} \to \mathbb{R}^d$ encoder, és legyen

$$L_{\mathrm{NN}}(\theta; X^{(1)}, X^{(2)}) = \frac{1}{N} \sum_{n=1}^{N} \left\| \hat{z}_n^{(1)} - \hat{z}_n^{(2)} \right\|_2^2,$$

ahol $\hat{z}_n^{(a)} = f_\theta(x_n^{(a)}) / \| f_\theta(x_n^{(a)}) \|_2$, feltéve, hogy a nevező pozitív. Tegyük fel, hogy létezik $v \in \mathbb{R}^d \setminus \{0\}$, amelyre $f_\theta(x) = v$ minden $x \in \mathcal{X}$ esetén. Ekkor

1. $L_{\mathrm{NN}}(\theta) = 0$, tehát a veszteség a lehetséges legkisebb, nemnegatív értékét veszi fel;
2. bármely $g : \mathbb{R}^d \to \{0, \dots, K-1\}$ downstream osztályozóra a predikció $g(v)$, egyetlen konstans osztálycímke; ha az osztályok prior eloszlása π_k, akkor a downstream pontosság legfeljebb $\max_k \pi_k$, kiegyenlített osztályoknál $1/K$.

Bizonyítás. (1) Mivel f_θ minden inputra a v vektort adja, $\hat{z}_n^{(1)} = \hat{z}_n^{(2)} = v / \|v\|_2$, így minden összegtag nulla, az átlaguk is nulla. A normalizált MSE definíció szerint nemnegatív, tehát a 0 a minimum.

(2) A downstream osztályozó bemenete minden x-re $g(v)$, tehát a predikció az x-től független $c = g(v)$ konstans. Ekkor $\mathbb{P}(\hat{Y} = Y) = \mathbb{P}(Y = c) = \pi_c \le \max_k \pi_k$. Kiegyenlített $\pi_k = 1/K$ esetén a felső korlát $1/K$. $\square$

A lemma két kontrollt nevez meg. Az első a *reprezentációs collapse-mérő*: egy batchben a $\hat{z}_n$ vektorok páronkénti koszinusz-hasonlóságának átlaga, illetve a koordinátánkénti szórás. Konstans embeddingnél a páronkénti koszinusz mátrix minden eleme 1, a szórás minden koordinátán 0. A második az *downstream kontroll*: egy rögzített spliten tanított lineáris fej pontossága. A degenerált reprezentáció a pretext loss optimumában ülhet, miközben a downstream kontroll véletlen szintet mutat.

Az InfoNCE-val a különbség jól látható: ott a nevező negatívokat tartalmaz, és a konstans v esetén a pozitív és negatív párok azonos hasonlóságot kapnak, ezért a loss nem nulla; ha minden hasonlóság azonos, az érték pontosan $\log(K+1)$, ahol K a negatívok száma. A negatív nélküli cél ezt a büntetést elhagyja, ezért a collapse-ot a tréningmechanizmusnak kell megakadályoznia.

A stop-gradient és a momentum teacher szerepe nem állítható be bizonyított mechanizmusként ezen a szinten. A BYOL (Grill és mtsai, 2020) és a SimSiam (Chen és He, 2021) munkáiban leírt megfigyelés az, hogy a student és a lassan változó teacher közötti aszimmetria, valamint a predictor fej jelenléte eltérő fixpontokat hozhat létre, így a konstans megoldás nem stabil vonzó pontja a teljes dinamikának. Ez azonban heurisztikus magyarázat; a BatchNorm, az optimalizáló és a súlycsökkenés szerepe a negatív nélküli tanulásban jelenleg is aktív kutatási kérdés, nem e könyv lezárt tételanyaga.

30.3.4 Kézi példa és minimális kód

Legyen $N = 6$, $d = 4$, és válasszuk a $v = (1{,}0; 2{,}0; 1{,}0; 0{,}0)$ konstans reprezentációt. Az inputok

$$X_1 = \begin{pmatrix} -1{,}9162 & -0{,}1293 & -2{,}1033 & 1{,}5267 \\ 1{,}3907 & -0{,}4277 & 0{,}5428 & -0{,}3572 \\ 1{,}7088 & -1{,}2685 & -2{,}0702 & -0{,}1406 \\ -0{,}3553 & 0{,}1761 & 0{,}3935 & 0{,}9526 \\ 0{,}1620 & -0{,}3699 & 0{,}4773 & 0{,}9091 \\ -2{,}1425 & -1{,}6982 & -0{,}7216 & 0{,}1653 \end{pmatrix}, \quad X_2 = X_1 + 0{,}1\varepsilon,$$

ahol ε a X_1-gyel azonos alakú, független standard normális minta. Az X_1 a kód rögzített seedű `rng.normal` hívásának eredménye.

A konstans encoder mindkettőre a v sort adja. A normalizálás: $\|v\|_2 = \sqrt{1+4+1} = \sqrt{6} \approx 2{,}4494897$; a normalizált sor minden esetben $(0{,}4082483;\ 0{,}8164966;\ 0{,}4082483;\ 0{,}0)$. A veszteség minden sorra 0, a páronkénti koszinusz átlaga 1,0.

```
import numpy as np

rng = np.random.default_rng(20261020)
N, d = 6, 4
X1 = rng.normal(size=(N, d)).astype(np.float64)
X2 = X1 + 0.1 * rng.normal(size=(N, d))   # zajos view

v = np.array([1.0, 2.0, 1.0, 0.0], dtype=np.float64)
def f_const(x):
    return np.tile(v, (x.shape[0], 1))          # (N, d)

Z1 = f_const(X1); Z2 = f_const(X2)
nrm = np.linalg.norm(v)
Z1h = Z1 / nrm; Z2h = Z2 / nrm                 # stabilizált alak
L = np.mean(np.sum((Z1h - Z2h) ** 2, axis=1))

coord_std = Z1h.std(axis=0)                    # (d,)
C = Z1h @ Z1h.T                                # (N, N) koszinuszmátrix
```

```
mean_pair_cos = (C.sum() - np.trace(C)) / (N * (N - 1))

y = np.array([0, 1, 2, 0, 1, 2], dtype=np.int64)
lemma_bound_demo = float(np.mean(np.zeros(N, dtype=np.int64) == y))
# a lemma (2) felső korlátjának szemléltetése, nem tanított fej

print(f"L={L:.12f}")
print(f"coord_std={coord_std}")
print(f"mean_pair_cos={mean_pair_cos:.12f}")
print(f"lemma_bound_demo={lemma_bound_demo:.6f}")

assert L == 0.0 and np.all(coord_std == 0.0)
assert abs(mean_pair_cos - 1.0) < 1e-12
assert abs(lemma_bound_demo - 1.0 / 3.0) < 1e-12
```

A kód lefutott (Python 3.12.3, NumPy 2.5.4, CPU, float64, seed 20261020); mért kimenet:

```
L=0.000000000000
coord_std=[0. 0. 0. 0.]
mean_pair_cos=1.000000000000
lemma_bound_demo=0.333333
```

A szimulált kontroll a lemma (2) pontjának felső korlátját szemlélteti: a konstans $g \equiv 0$ predikció a kiegyenlített $K = 3$ osztály mellett éppen $1/3$ pontosságot ad; ez nem tanított downstream fej mérése.

30.3.5 Labor: mechanizmus- és ablatív terv a SimCLR-keretből kiindulva

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A jelen alfejezet nem futtat kísérletet; az alábbi protokoll egy végrehajtható terv, a mért eredmény hiányát explicit jelöljük.

Hivatkozott kiindulópont. A [P17] SimCLR absztraktja alapján a kontrasztív keret a két nézet közötti egyezést a batchbeli negatívokkal versenyezteti. A labor kérdése: ugyanazon GlyphLab encoder (a függelék A. A GlyphLab adatkészlet szakasza szerinti adaton), view-set és tanítási költségvetés mellett hogyan viselkedik a negatív nélküli cél a stop-gradient és a momentum teacher változatokban?

Adat és split. A GlyphLab `data/images.npy (360,1,24,24)` float32 tömbje; a train/val/test a metadata-ban rögzített jelenetek. A pretext során a downstream címkét nem használjuk, csak a képeket; a view-párok: az eredeti kép és annak független additív Gauss-zajjal ($\sigma = 0{,}05$) perturbált, $[0, 1]$ intervallumra vágott változata, véletlen fényességskálázással (0,9–1,1 szorzó) és legfeljebb 2 pixeles eltolással (a kép szélén nulla-kitöltéssel, majd az eltolás visszavonásával). A 90°-os forgatást kizárjuk, mert a háromosztályos orientation-célhoz nem címkemegőrző transzformáció; a pretextben való alkalmazása downstream értelmezési zavart vinne be.

Modell. Kis CPU-s CNN encoder a 30.1. Pretext és downstream feladat backbone-jével azonos felépítésben, de $d = 32$ kimenettel: Conv2d(1, 8, 3, padding=1) + ReLU + Conv2d(8, 16, 3, stride=2, padding=1) + ReLU + AdaptiveAvgPool2d(4) + Linear(256, 32); float32 CPU. A három futás seedje 20261020, 20261021, 20261022. A downstream fej egyetlen `Linear(32,3)`.

Változó tényezők. (a) kontrasztív InfoNCE baseline, $\tau_{temp} = 0{,}1$; (b) negatív nélküli cél stop-gradient nélkül; (c) ugyanaz stop-gradienttel; (d) stop-gradient és momentum teacher, $\tau_{\text{ema}} = 0{,}99$; (e) a (d) változat predictor nélkül. Minden változathoz három seed (20261020,

20261021, 20261022) ugyanazon a train/val/test felosztáson, és a változatokat ugyanazon seedekkel párosítjuk. A downstream összehasonlítás döntési szabálya a jelenet szintű páros különbség bootstrap CI-je a 23.3. Páros összehasonlítás és csoportszintű intervallum szerint: 0-t kizáró 95%-os CI esetén állítunk különbséget, egyébként nem; a két külön intervallum átfedése nem helyettesíti ezt.

Metrikák. Pretext veszteség train/val; collapse-mérő a val reprezentációk páronkénti átlagos koszinusza; downstream lineáris fej pontossága a rögzített testen. A lineáris fejet a rögzített train reprezentációkra tanítjuk azonos epoch-kerettel, a val a lineáris fej L2-regularizációs erősségének kiválasztására szolgál ($\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$), a test a végső méréshez. A downstream méréshez használjuk a tiszta, nem korrupt tesztet; a `corrupted_test.npy`-t itt nem használjuk, mert az a 38.3. Végső teszt és páros korrupció projekt II zárótesztjéhez van fenntartva.

Átadandó eredmény formája. Protokollfájl, pretext loss-görbe, collapse-mérő és downstream pontosság minden változatra seed-átlaggal. A terv nem futott le, ezért számszerű kimenetet nem állítunk; a (b) változat a konstans optimum felé csúszhat, de ezt mérés nélkül nem tekintjük eredménynek.

30.3.6 Feladatok

F1 (fogalmi, nehézség 1). Definiáld az alábbi három fogalmat az ebben az alfejezetben bevezetett jelölésekkel: teacher, student, stop-gradient. Add meg, melyik paraméterekre számolunk gradienst a tanulási lépésben, és mi a szerepe a predictor fejnek.

F2 (levezetési, nehézség 2). Tekintsd a normalizálatlan változatot: $\tilde{L}(\theta) = \frac{1}{N}\sum_{n=1}^{N} \|z_n^{(1)} - z_n^{(2)}\|_2^2$, ahol $z_n^{(a)} = f_\theta(x_n^{(a)})$, és nincs normalizálás. Bizonyítsd be, hogy az azonosan nulla encoder $f_\theta(x) = 0$ minden x-re a $\tilde{L}$ globális minimumát adja, és add meg a downstream pontosság felső korlátját ebben az esetben is. Miért nem elegendő a lemma bizonyításában a nulla vektor kizárása nélküli normalizálás?

F3 (implementációs, nehézség 2). Írj teljes NumPy függvényt `no_negative_loss(Z1, Z2)` néven, amely a normalizált MSE-veszteséget és a collapse-mérőt (páronkénti átlagos koszinusz) adja vissza. Az inputok `(N,d)` float64 tömbök, és a függvény stabilan kezelje azt az esetet, ha egy reprezentációsor normája nullához közeli (használj $\varepsilon = 10^{-12}$ normalizálási epsilont). Ellenőrizd a konstans $v = (1, 2, 1, 0)$ példán, és egy olyan batchen, ahol a két nézet egymástól eltér.

F4 (hibakeresési, nehézség 3). A következő vázlat a momentum teacher frissítését akarja elvégezni PyTorchban:

```
for tp, sp in zip(teacher.parameters(), student.parameters()):
    tp.data = tau * tp.data + (1.0 - tau) * sp.data
# ... később a loss.backward() és student_optimizer.step()
```

Azonosítsd a két lehetséges hibát, és magyarázd el a következményét: mi történik a tanulási dinamikával, ha a frissítés (i) a `loss.backward()` után, de a teacher paramétereken gradiensgráfban marad; (ii) a student `step()` előtt hajtódik végre? Add meg a javított kódot `no_grad` és megfelelő sorrend használatával.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezz kísérletet a GlyphLabon, amely azt vizsgálja, hogy a negatív nélküli pretext reprezentáció jobb-e a downstream orientation osztályozáshoz, mint (i) a véletlenül inicializált encoder és (ii) az autoencoderes rekonstrukciós pretext. Add meg a megfigyelési egységet, a splitet, a baseline-okat, a változó tényezőket, a metrikákat és az érvényességi korlátot. Nem kell futást végezni, de a tervnek végrehajthatónak kell lennie a könyv CPU-s főútján.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

30.3.7 Megoldások és indoklás

F1. A *student* a θ_s paraméterű encoder, amelyre a pretext veszteség gradiensét visszaterjesztjük; a *teacher* a θ_t paraméterű, ugyanolyan architektúrájú encoder, amelyen keresztül gradiens nem áramlik, súlyát az exponenciális mozgóátlag frissíti. A *stop-gradient* a teacher kimenetét a loss számításában konstansként kezeli, tehát a visszaterjesztés során a teacher felé eső gradiens blokkot nullázza. A gradiens kizárólag a student θ_s és a predictor ϕ paramétereire számolódik; a predictor a student oldali kis MLP, például $d \to 2d \to d$ ReLU-val, amely az aszimmetriát növeli, és a módszercsalád gyakorlatában a negatív nélküli tanulás stabilitásához járul hozzá. A tanulási lépésben tehát θ_s és ϕ frissül, a teacher θ_t nem.

F2. Ha $f_\theta(x) = 0$ minden x-re, akkor minden tag $\|0 - 0\|_2^2 = 0$, ezért $\tilde{L}(\theta) = 0$. Mivel a norma négyzete nemnegatív, ez a globális minimum. A downstream predikció $g(0)$ konstans, így a pontosság legfeljebb $\max_k \pi_k$, kiegyenlített esetben $1/K$. A normalizált változat lemmájában a $v \neq 0$ feltétel szükséges, mert a $v = 0$ esetben a $\hat{z} = z/\|z\|_2$ nincs értelmezve. A numerikus ε-normalizálás $z/\max(\|z\|_2, \varepsilon)$ alakban nullvektorra nullvektort ad, tehát a veszteség ekkor is nulla; kis, de nem nulla normánál viszont a hányados nem egységvektor, csak legfeljebb $\|z\|_2/\varepsilon$ normájú, ezért a numerikus alak nem azonos a matematikai normalizálással, és a kis normájú soroknál a két nézet eltérését alábecsüli. A konstans encoder downstream korlátja viszont a lemma (2) pontja szerint $\max_k \pi_k$, függetlenül az ε választásától.

F3. Teljes kód:

```
import numpy as np

def no_negative_loss(Z1, Z2, eps=1e-12):
    """
    Z1, Z2: (N, d) float64 reprezentációsorok.
    Visszatérés: normalized_mse (a két normalizált nézet átlagos négyzetes
    távolsága), mean_pair_cos (a Z1h-n belüli párok átlagos koszinusza).
    """
    n1 = np.linalg.norm(Z1, axis=1, keepdims=True)
    n2 = np.linalg.norm(Z2, axis=1, keepdims=True)
    Z1h = Z1 / np.maximum(n1, eps)
    Z2h = Z2 / np.maximum(n2, eps)
    L = np.mean(np.sum((Z1h - Z2h) ** 2, axis=1))
    C = Z1h @ Z1h.T
    N = Z1h.shape[0]
    mean_pair_cos = (C.sum() - np.trace(C)) / (N * (N - 1))
    return L, mean_pair_cos

# Ellenőrzés konstans esetben
v = np.array([1.0, 2.0, 1.0, 0.0], dtype=np.float64)
Zc = np.tile(v, (6, 1))
Lc, mpc = no_negative_loss(Zc, Zc)
print(f"konstans: L={Lc:.12f}, mean_pair_cos={mpc:.12f}")  # 0, 1

# Eltérő nézetek: várhatóan L>0 és mpc<1
rng = np.random.default_rng(20261020)
Za = rng.normal(size=(6, 4)); Zb = rng.normal(size=(6, 4))
Ld, mpd = no_negative_loss(Za, Zb)
print(f"eltérő: L={Ld:.6f}, mean_pair_cos={mpd:.6f}")
```

```
assert Lc == 0.0 and abs(mpc - 1.0) < 1e-12
assert Ld > 0.0 and -1.0 <= mpd <= 1.0
```

A kód lefutott (Python 3.12.3, NumPy 2.5.4, CPU, float64, seed 20261020); mért kimenet:

```
konstans: L=0.000000000000, mean_pair_cos=1.000000000000
eltérő: L=2.015337, mean_pair_cos=0.009919
```

A konstans esetben a veszteség pontosan nulla és a páronkénti koszinusz 1; az eltérő véletlen nézeteknél a veszteség 2,015337, a páronkénti koszinusz 0,009919, ami közel áll a független véletlen vektorok várható nulla hasonlóságához. A tolerancia a konstans esetben a float64 kerekítés miatt 10^{-12} alatt van.

F4. A kód két hibája:

(i) A `tp.data = ...` frissítés kikerüli az autogradot (nem épül gráf, és a verziószámláló sem frissül), de a teacher paraméterein továbbra is `requires_grad=True` marad. A `.data` önmagában nem ad garanciát a stop-gradientre: ha a loss-ban a teacher kimenete nincs `detach()`-elve, a backward a teacher paramétereire is számol gradienst, és az felhalmozódhat a teacher paramétereinek `grad` mezőjében; ha pedig ezt egy optimalizáló fel is használja, a teacher visszaterjesztéssel is tanul. A biztonságos forma a `with torch.no_grad():` blokk, szükség esetén a teacher paraméterein `requires_grad_(False)` beállítással.

(ii) Ha a teacher frissítése a student `step()` előtt történik, akkor a teacher a *régi* student súlyok felé mozdul. A szándék az, hogy a teacher a frissített studentet kövesse, ezért a sorrend: `loss.backward()`, `student_optimizer.step()`, majd a teacher momentum frissítés.

Javított kód:

```
# backward és student update
loss.backward()
student_optimizer.step()

# teacher frissítés, gradiensgráf nélkül
with torch.no_grad():
    for tp, sp in zip(teacher.parameters(), student.parameters()):
        tp.mul_(tau).add_(sp, alpha=1.0 - tau)
```

Visszaellenőrzés: a momentum frissítés `torch.no_grad()` alatt történik, ezért nem épít gráfot; a teacher paraméterein a `requires_grad` jelző változatlanul `True` marad, de a veszteségben a teacher kimenete `detach()`-en (stop-gradienten) keresztül szerepel, így a backward nem számol teacher-gradienst. Az egységteszt: egyetlen lépésre $d = 1$ méretű lineáris réteggel, $\theta_s = 1$, $\theta_t = 0{,}9$, $\tau_{\text{ema}} = 0{,}99$ esetén a frissítés után $\theta_t = 0{,}901$ a mozgóátlag képletéből.

F5. Egy lehetséges terv: a GlyphLab train/val/test jelenetfelosztását változatlanul hagyjuk; a pretext tanuláshoz minden train képből két view-t készítünk a 30.1. szakaszban megengedett címkemegőrző augmentációkkal (kis eltolás, kis zaj, intenzitásskála), a 90°-os forgatás és a tükrözés nélkül. A változó a pretext cél: (A) véletlen encoder (nincs pretext), (B) autoencoder rekonstrukciós pretext, (C) negatív nélküli pozitív-páros pretext stop-gradienttel, EMA teacherrel ($\tau_{\text{ema}} = 0{,}99$) és predictor fejjel; a predictor és az EMA beállítása ugyanaz, mint a labor (d) változatában. A downstream fej mindhárom esetben egy `Linear(32,3)` a rögzített encoder-kimeneten. A megfigyelési egység a scene_id, ezért a val/test jeleneteket soha nem használjuk pretext tanításra. A metrika a downstream val/test pontosság minden változatra három seeddel. A baseline az (A) véletlen encoder, mert azt méri, mennyit ér a pretraining nélküli, csak az inicializációból származó reprezentáció. Az érvényességi korlát: a GlyphLab nem természetes képi adat; a pretext transzformációk a generált vonalak statisztikáira vannak hangolva; a scene-függő

zaj miatt a jelenetek közötti augmentációs invariance eltérhet a valós adatoktól, így az eredmény nem általánosítható termelési modellre.

30.3.8 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés azt hinni, hogy a *collapse* elkerülése önmagában downstream feladatsikerrel jár. Ez nem igaz. Ellenpéldául tekintsünk olyan adatot, ahol $x = (a, b, t)$, az a, b és t egymástól független folytonos véletlen változók, a háromosztályos y címkét pedig kizárólag t alapján, azonos valószínűségű intervallumokkal képezzük. A két nézet ugyanazt az (a, b) koordinátát tartalmazza, és csak a t koordinátában tér el: az első nézet t-jéhez δ_1, a másodikéhoz δ_2 független kis zajt adunk. Legyen az encoder $f(x) = (a, b)$. Ez a két nézetet pontosan össze tudja párosítani, mert a reprezentáció a t zajától nem függ, tehát a negatív nélküli pretext veszteség nulla lehet. A különböző bemenetek reprezentációi nem állandók, és normalizálás után sem mind ugyanabba az irányba mutatnak, ezért a páronkénti koszinusz-hasonlóság nem szükségképpen 1. A címkére azonban semmilyen információt nem tartalmaznak: kiegyensúlyozott osztályok mellett a belőlük dolgozó legjobb osztályozó várható pontossága legfeljebb $1/3$. A *nem összeomlott* reprezentáció tehát nem feltétlenül hasznos a downstream feladathoz. Emiatt a pretext tanítást előre rögzített downstream kontrollal kell kiértékelni, nem csupán a collapse hiányával.

30.3.9 Önellenőrzés és visszahívás

Önellenőrző kérdések: (1) A stop-gradient operátor mit tesz a veszteség Jacobi-mátrixában? (2) Miért nulla a konstans encoder vesztesége a normalizált MSE alatt, és mi a downstream pontosság felső korlátja kiegyenlített osztályoknál? (3) Miben különbözik a momentum teacher frissítés a BatchNorm futó statisztikájától? (4) Miért nem elég a páronkénti koszinusz átlaga 1-től való eltérése a downstream hasznosság igazolásához?

Válaszkulcs. (1) A stop-gradient a veszteség Jacobi-mátrixában az adott ág felé nullázza a gradienset. (2) A konstans encoder normalizált MSE vesztesége nulla, a downstream pontosság felső korlátja kiegyenlített osztályoknál $\max_k \pi_k = 1/K$. (3) A momentum teacher a paraméterek exponenciális átlagát frissíti, nem a BatchNorm futó statisztikáját. (4) A páronkénti koszinusz átlagától való eltérés nem garantál downstream hasznosságot: a reprezentáció lehet collapse-elt vagy a címkétől független.

Visszahívás: a 29.3. Regularizált latent reprezentáció autoencodernél a rekonstrukciós cél önmagában nem garantál a downstream címkére hasznos reprezentációt; az itt tárgyalt negatív nélküli pretext a pozitív invarianciából ugyanígy nem kap downstream garanciát. A downstream lineáris fej pontossága argmax-alapú osztályozást mér, nem külön kalibrált küszöböt; a kalibrációt és a működési pontot külön kell vizsgálni, hasonlóan a 26.3. NMS, confidence és AP pontosság–lefedettség elemzéséhez. A 18.3. Pre-activation, kis batch és alternatív normalizálás futó átlagos állapota párhuzamos a momentum teacher exponenciális átlagával, de a frissített mennyiség (statisztika, illetve paraméter) és a tengelyek eltérnek.

30.3.10 Rövid összefoglaló

A negatív nélküli pretext cél csak pozitív párokat használ: ugyanannak a jelenetnek két nézete azonos normalizált reprezentációba kerüljön. A konstans encoder a normalizált MSE veszteséget nullára csökkenti, miközben a downstream pontosság csak $\max_k \pi_k$ lehet; ezt a lemmát a jelen alfejezet bizonyítja. A collapse ellen a gyakorlatban stop-gradient, momentum teacher és predictor fej szolgál, de ezek hatása heurisztikus, nem lezárt tétel. A laborban a SimCLR-keretből kiinduló

mechanizmus- és ablatív tervet készítünk, a feladatokban pedig a kontrollt és a tipikus tévedéseket gyakoroljuk.

Az induló becslésre a válasz: igen, a konstans encoder a normalizált MSE-t nullára csökkenti, ezért a puszta pozitív-páros cél degenerált megoldást enged; a stop-gradient, a momentum teacher és a predictor fej azért kell, hogy ezt a triviális megoldást elkerülje, de downstream garanciát ezek sem adnak.

30.3.11 Következő alkalmazás és ténylegesen használt források

A következő 30.4. Önfelügyelt kísérlet korrekt összevetése egység az önfelügyelt kísérletek korrekt összevetését vizsgálja: ott az itt definiált negatív nélküli pretext, a collapse-mérő és a downstream kontroll kapcsolódik össze a véletlen, rekonstrukciós és kontrasztív baseline-ok fair összehasonlításában.

Források:

- [P17] Chen, T., Kornblith, S., Norouzi, M., Hinton, G. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. https://arxiv.org/abs/2002.05709.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z. D., Gheshlaghi Azar, M., Piot, B., Kavukcuoglu, K., Munos, R. és Valko, M. (2020). *Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning*. NeurIPS 2020, arXiv:2006.07733.
- Chen, X. és He, K. (2021). *Exploring Simple Siamese Representation Learning*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), arXiv:2011.10566. A BYOL- és SimSiam-hivatkozások a momentum teacher, a stop-gradient és a predictor szerepének forrásai; a fejezet forráskatalógusán kívüliek.

30.4 Önfelügyelt kísérlet korrekt összevetése

30.4.1 Cél, előismeret és indító kérdés

A 30.3. Negatív nélküli módszerek és collapse egységben láttuk, hogy a kontrasztív és a negatív nélküli módszerek egyaránt összeomolhatnak konstans reprezentációba (collapse), ha a stop-gradient (a teacher ág kimenetének konstansként kezelése a veszteségben), a veszteség vagy a teachermechanizmus (az exponenciális mozgóátlaggal frissített enkóder) nem megfelelő. Ebben az alfejezetben azt vizsgáljuk, hogyan lehet egy önfelügyelt pretraining (címke nélküli előzetes tanítás) utáni downstream (célfeladati) eredményt korrekt módon összehasonlítani egy felügyelt kiindulási modellel. Az előismeret: a frozen probe (rögzített enkóder mellett tanított lineáris fej) fogalma, a train/val/test felosztás és a matched-recipe (azonos receptű) abláció 18.4. Reziduális blokk ablatív vizsgálata gondolata; ott a reziduális blokk hatását csak azonos adat, augmentáció és tanítási költség mellett lehetett izolálni, és ugyanezt az elvet alkalmazzuk itt a pretext célra.

Induló kérdés: ha az A módszer 0,87, a B módszer 0,79 tesztpontosságot ér el ugyanazon a teszthalmazon, akkor állíthatjuk-e, hogy A reprezentációja jobb? A válasz csak akkor lehet igen, ha a két ág azonos címke- és számítási kerettel, azonos adatmedencével és azonos protokollal futott. Ellenkező esetben a különbség lehet pusztán a több adat vagy a több számítás következménye.

30.4.2 Definíciók: pretraining, frozen probe és erőforrás-könyvelés

Legyen az enkóder

$$f_\theta : \mathbb{R}^{C\times H\times W} \to \mathbb{R}^d,$$

tehát egy képből d-dimenziós vektort készít. A pretraining során egy címke nélküli képhalmazt használunk:

$$\mathcal{D}_{pre} = \{x_i \in [0,1]^{C\times H\times W}\}_{i=1}^{N_{pre}}.$$

Minden x_i-hez két augmentált nézetet rendelünk, x_i^a és x_i^b; a [P17] SimCLR keretében a kontrasztív pretext cél egy pozitív párként kezeli (x_i^a, x_i^b)-t, a batch többi elemét pedig negatívként. A pretext „címke” tehát a kép példányazonossága, nem emberi osztálycímke. A downstream osztálycímkék $y \in \{0, \dots, K-1\}$ ebben a fázisban ismeretlenek a modell számára.

A frozen probe során adott egy címkézett halmaz

$$\mathcal{D}_{probe} = \{(x_j, y_j)\}_{j=1}^{L_{probe}},$$

és egy rögzített enkóderparaméter θ^*. Képezzük a reprezentációkat $z_j = f_{\theta^*}(x_j) \in \mathbb{R}^d$, majd egy affin fejet tanítunk:

$$g_W(z) = Wz + b, \qquad W \in \mathbb{R}^{K\times d},\ b \in \mathbb{R}^K.$$

Batchben $Z \in \mathbb{R}^{B\times d}$ mellett a logitok $G = ZW^\top + b \in \mathbb{R}^{B\times K}$. A „frozen” szó azt jelenti, hogy a veszteség gradiensét csak W-re és b-re számítjuk; $\nabla_\theta L = 0$ a tanítás során. PyTorchban ez a `torch.no_grad()` alatti forward vagy `requires_grad_(False)` használatát jelenti. BatchNorm-rétegeknél az enkódert kiértékelő módba kell tenni, hogy a running statisztika és ne a batch statisztikája érvényesüljön.

Címkeköltségvetés. Egy módszer címkeköltsége B_{lab}: azon (x, y) képpárok száma, amelyeket bármilyen felügyelt optimalizáláshoz, korai leállításhoz vagy modellválasztáshoz ténylegesen felhasználtunk. A tesztcímkéket a végső kiértékelésen kívül nem számítjuk ide, és a pretext augmentációból származó pozitív/negatív viszonyok sem címkék.

Számítási költségvetés. Legyen c_f egy kép enkóder-forwardjának normalizált költsége, c_b pedig az enkóder-backwardé. A $\rho = c_b/c_f$ arány hardverfüggő; ebben a jegyzetben csak könyvelési konvencióként a $\rho = 2$ értéket használjuk. A kontrasztív pretraining egy eredeti képre két nézetet, tehát két forwardot és két backwardot jelent. Egy felügyelt lépés egy képre egy forward és egy backward. Ezzel

$$U_A = 2(c_f + c_b)N_{pre}E_{pre} + c_f L_{probe}E_{probe}$$

az A önfelügyelt ág normalizált költsége, ahol E_{pre} a pretraining epochok, E_{probe} a frozen probe epochok száma. A felügyelt B kiindulási ág költsége

$$U_B = (c_f + c_b)L_B E_B,$$

ahol L_B a felügyelt kép-címke párok száma, E_B az epochok száma.

30.4.3 Levezetés: a címke- és számítási keret mint kontrollfeltétel

Az összehasonlítás akkor tisztán a reprezentációról szól, ha az erőforrás-különbségek nem magyarázhatják a mért különbséget. Ezt egy azonosíthatósági lemmában rögzítjük.

Lemma (erőforrás-konfúzió az additív feltevés mellett). Tegyük fel, hogy egy módszer végső pontszáma

$$s = \alpha + \beta M + \gamma_c U + \gamma_l B_{lab} + \gamma_u N_{pre} + \varepsilon,$$

ahol $M \in \{0, 1\}$ a vizsgált módszercsalád indikátora, U a számítási, B_{lab} a címke-, N_{pre} a pretraining képmennyiségi erőforrás, ε pedig nulla várható értékű hibatag. Ha a két megfigyelt ág között legalább egy erőforrás értéke különbözik, akkor a megfigyelt $d = s_A - s_B$ különbséghez végtelen sok különböző β módszerhatást rendelhetünk, tehát β önmagában nem azonosítható.

Bizonyítás. Jelölje $\delta_U = U_A - U_B$, $\delta_l = B_{lab,A} - B_{lab,B}$ és $\delta_u = N_{pre,A} - N_{pre,B}$. Feltéve $\varepsilon_A = \varepsilon_B$ mellett az átlagos megfigyelt különbség

$$d = \beta + \gamma_c \delta_U + \gamma_l \delta_l + \gamma_u \delta_u.$$

Ha például $\delta_U \neq 0$, akkor tetszőleges $\beta_1 \neq \beta_2$ módszerhatáshoz választhatjuk

$$\gamma_c^{(\beta)} = \frac{d - \beta - \gamma_l \delta_l - \gamma_u \delta_u}{\delta_U},$$

és ezzel ugyanazt a d különbséget kapjuk. A választás tetszőleges, tehát a megfigyelés nem különíti el a módszerhatást az erőforráshatástól. Ha $\delta_U = 0$, de $\delta_l \neq 0$ vagy $\delta_u \neq 0$, akkor az adott nemnulla δ-val ugyanez a konstrukció elvégezhető. □

A lemma feltétele az additív modell, ezért nem általános statisztikai azonosíthatósági tétel. A célja az, hogy pontosan megfogalmazza: a több adatot és több számítást használó módszer fölénye nem automatikusan architekturális vagy reprezentációs bizonyíték. A gyakorlati következmény az alábbi protokoll:

1. **Címke-egyezés:** $B_{lab,A} = B_{lab,B}$. Az A ág frozen probe-ja és a B felügyelt ág ugyanannyi címkézett képet kap.
2. **Számítási keret:** $U_A \leq U_B$ vagy $|U_A - U_B|$ egy előre rögzített kis toleranción belül van. Ha B kevesebb számítást kap, az összehasonlítás nem a módszert izolálja.
3. **Adatmedence:** az A pretraininghez használt N_{pre} kép nagyobb adathalmazt adhat, mint a B ág. Ez további erőforrás; vagy B is megkapja ugyanazt a medencét egy megengedett felhasználási formában, vagy a N_{pre} hatást külön ablációnak nevezzük.
4. **Diszjunkt teszt:** $\mathcal{D}_{test}$ sem a pretraining, sem a probe sem a B tanítóhalmaz képeivel nem osztozhat. A függelék A. szakaszában leírt GlyphLab esetében a független mintaegység a `scene_id`, ezért ugyanaz a jelenet nem kerülhet több halmazba.

30.4.4 Kézi számpélda

Vegyünk egy $c_f = 1$, $c_b = 2$ normalizált enkódert. Az A kontrasztív ág $N_{pre} = 2000$ címkézetlen képet használ $E_{pre} = 20$ pretraining epochon át, majd $L_{probe} = 300$ címkével $E_{probe} = 10$ probe epochot futtat. Ekkor

$$U_A = 2(1+2) \cdot 2000 \cdot 20 + 1 \cdot 300 \cdot 10 = 240{,}000 + 3{,}000 = 243{,}000.$$

A B felügyelt ág $L_B = 300$ címkével és $E_B = 10$ epochhal:

$$U_B = (1+2) \cdot 300 \cdot 10 = 9{,}000.$$

Az A ág tehát 27-szor több normalizált számítást és 2000 extra képet használ, miközben a címkeköltség egyenlő. Ha az A ág 0,87, a B ág 0,79 tesztpontosságot érne el, a mondat helyes formája így szól: „A kontrasztív pretraining nagyobb adat- és számítási kerettel 0,87-ot ért el”; nem pedig „a kontrasztív reprezentáció jobb”. A fair összehasonlításhoz a B ágnak $U_B = 243{,}000$ kerethez $E_B = 243{,}000/(3 \cdot 300) = 270$ epochot kellene kapnia, vagy az A ágnak a 2000 képből és a pretraining epochokból annyit elvenni, hogy $U_A \le U_B$ teljesüljön.

30.4.5 Labor: erőforrás-könyvelés és reprodukciós terv

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a korábbi laborokban előállított checkpoint(ok); a futásidőt a futtatás során mért értékként kell rögzíteni.

A következő kód a fenti könyvelést hajtja végre. A kód lefutott (Python 3.12.3, a `dataclasses` modulon kívül nincs függősége); a mért kimenetet a kód utáni blokk tartalmazza.

```
# labor/n120_budget_bookkeeping.py
"""n120 erőforrás-könyvelés."""
from dataclasses import dataclass

@dataclass(frozen=True)
class Budget:
    n_pre: int      # pretraining képek száma (címke nélkül)
    e_pre: int      # pretraining epochok
    l_probe: int    # frozen probe címkézett képek
    e_probe: int    # probe epochok
    l_sup: int      # felügyelt baseline címkézett képek
    e_sup: int      # baseline epochok

def ssl_units(b: Budget, c_f: float = 1.0, c_b: float = 2.0) -> float:
    # kontrasztív: 2 nézet, mindegyik forward+backward; probe: fagyasztott forward
    return 2.0 * (c_f + c_b) * b.n_pre * b.e_pre + c_f * b.l_probe * b.e_probe

def sup_units(b: Budget, c_f: float = 1.0, c_b: float = 2.0) -> float:
    return (c_f + c_b) * b.l_sup * b.e_sup

def report(b: Budget) -> None:
    u_a = ssl_units(b)
    u_b = sup_units(b)
    label_match = b.l_probe == b.l_sup
    print(f"SSL units:          {u_a:.1f}")
    print(f"Supervised units:   {u_b:.1f}")
    print(f"Compute ratio A/B:  {u_a / u_b:.2f}")
```

```
    print(f"Label budget match: {label_match}")

config = Budget(n_pre=2000, e_pre=20,
                l_probe=300, e_probe=10,
                l_sup=300, e_sup=10)
report(config)
assert ssl_units(config) == 243000.0
assert sup_units(config) == 9000.0
assert config.l_probe == config.l_sup
```

A mért kimenet:

```
SSL units:          243000.0
Supervised units:   9000.0
Compute ratio A/B:  27.00
Label budget match: True
```

A `report` függvény nem ellenőrzi a pretraining-képek diszjunkcióját; ezt a GlyphLab `metadata.csv` alapján külön kell megtenni:

```
import pandas as pd

meta = pd.read_csv("data/metadata.csv")
pretrain_scenes = set(meta.loc[meta.split == "train", "scene_id"])
probe_scenes = set(meta.loc[meta.split == "train", "scene_id"])
# a probe ugyanazokból a train jelenetekből dolgozik
test_scenes = set(meta.loc[meta.split == "test", "scene_id"])
assert pretrain_scenes & test_scenes == set()
assert probe_scenes == pretrain_scenes
```

A mellékelt adaton ez az ellenőrzés lefutott: a train és a test `scene_id` halmazának metszete üres (120 train és 30 test jelenet), és a probe halmaza megegyezik a pretraining train halmazával. A fenti `Budget` konfiguráció a kézi számpélda értékeit állítja elő; a reprodukciós terv a 240 train képpel és a hozzá tartozó külön `Budget` példánnyal dolgozik.

Helyi reprodukciós terv (nem elvégzett futtatás). Kérdés: a kontrasztív pretraining javítja-e a GlyphLab orientációfelismerést a matched-költségű felügyelt baseline-hoz képest? Enkóder: ugyanaz a kis CNN mindkét ágban, $d = 32$ kimenettel. Pretraining: a train split 240 képe, címkék nélkül, orientációt megőrző augmentációkkal (fényességeltolás, $\sigma = 0{,}05$ Gauss-zaj), 90 fokos forgatás nélkül, $E_{pre} = 50$ epochon át. Probe: ugyanezen 240 train kép címkéjével, frozen enkóderrel, logitfejjel, $E_{probe} = 10$ epochon át. Baseline: ugyanez a CNN és augmentáció, felügyelt tanítás a 240 train címkével. Számítási keret: az A ág U_A értékét a (30.4.2) képlettel kiszámítjuk, majd a B ág $E_B = \lceil U_A / ((c_f + c_b) L_B) \rceil$ epochszámát úgy állítjuk be, hogy $U_B \geq U_A$ és $|U_B - U_A|$ kicsi legyen. Modellválasztás: mindkét ágban a val split vesztesége szerinti legjobb checkpoint; a test splitet egyszer, a legvégén értékeljük. Három seeddel ismétlünk, és a pontosságok átlagát és szórását közöljük. A korrupt tesztet (a `corrupted_test.npy` 60 képét, amelyek a test-sorrenddel párosított, zajjal rontott változatok) csak a befagyasztott protokoll után nézzük meg. Ez a terv még nem mért eredmény, ezért itt nem állítunk számot.

30.4.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget a pretext pozitív pár és a downstream osztálycímke között. Adjon egy GlyphLab-példát, ahol egy augmentáció jó pretext pozitív párt ad, de a downstream orientáció szempontjából káros.

F2 (levezetési, nehézség 2). Legyen $c_f = 1$, $c_b = 2$. Az A ág $N_{pre} = 500$, $E_{pre} = 8$, $L_{probe} = 200$, $E_{probe} = 5$. A B ág $L_B = 200$, $E_B = 20$. Számítsa ki U_A-t és U_B-t. Címke-egyezés mellett teljesül-e a számítási keret? Ha nem, hány epochot kellene B-nek adni a keret kiegyenlítéséhez?

F3 (implementációs, nehézség 2). Bővítse a labor `report` függvényét úgy, hogy visszatérési értékként adja a két költségértéket és a címkeegyezés jelzőjét, és a címkeegyezést a `report` belsejében `assert` ellenőrizze. Írjon tesztet a kézi példára: egész elemszámú költségnél pontos egyenlőséggel, lebegő aránynál 10^{-9} relatív toleranciával ellenőrizze a `ssl_units` képletét, és mutassa meg, hogy egy szándékosan elrontott költségértéknél az ellenőrzés elbukik.

F4 (hibakeresési, nehézség 2). A következő protokollhibát látja egy hallgató kódjában: a pretraininghez a train és a test képek listáját összefűzi, mert „a test képekhez úgysem használ címkét”. Mi a hiba következménye, és milyen `scene_id`-alapú ellenőrzéssel lehet ezt kimutatni? Javítsa a hibát és fogalmazza meg a visszaellenőrzést.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen egy GlyphLab-kísérletet, amely egy kontrasztív és egy negatív nélküli pretext cél downstream hatását hasonlítja össze matched compute (azonos számítási keret) mellett. Adja meg a protokollt, a baseline-t, a metrikát és a következtetési korlátot. Előre nem kell győztest mondani; a negatív, de lefuttatott eredmény is elfogadható.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

30.4.7 Megoldások és indoklás

F1 megoldás. A pretext cél a példányazonosság: ugyanazon kép két augmentált nézete pozitív pár, más képek nézetei negatívak. Ez nem használ emberi osztálycímkét. A downstream címke a GlyphLab három orientációja. Ha a pretraining augmentáció 90 fokos forgatást is tartalmaz, akkor egy vertical kép elforgatott nézete ugyanahhoz a példányhoz tartozik, tehát a pretext szerint pozitív pár, de a downstream orientáció szerint horizontal címkéje lenne. A reprezentáció így megtanulhatja elnyomni az orientációt, ami épp a downstream feladat számára hasznos információ.

F2 megoldás.

$$U_A = 2(1+2) \cdot 500 \cdot 8 + 1 \cdot 200 \cdot 5 = 24{,}000 + 1{,}000 = 25{,}000.$$

$$U_B = (1+2) \cdot 200 \cdot 20 = 12{,}000.$$

A címke-egyezés teljesül, mert $200 = 200$. A számítási keret nem teljesül, mert $U_A > U_B$. A B ág szükséges epochszáma:

$$E_B = \frac{U_A}{(c_f + c_b) L_B} = \frac{25{,}000}{3 \cdot 200} = \frac{25{,}000}{600} \approx 41{,}67.$$

Mivel epochok egészek, a B ágnak legalább 42 epochot kell kapnia; ekkor $U_B = 25{,}200$, és az A ág nem használ több keretet, mint B. Ellenőrzés: U_A és U_B ugyanabban a normalizált egységben van, ezért közvetlenül összevethető; $E_B = 42$ mellett $U_B - U_A = 200$, tehát a B ág a legkisebb egész epochlépéssel éri el az A ág keretét.

F3 megoldás.

```
# labor/n120_budget_test.py
"""Bővített könyvelés teszttel."""
from dataclasses import dataclass

@dataclass(frozen=True)
class Budget:
    n_pre: int
    e_pre: int
    l_probe: int
    e_probe: int
    l_sup: int
    e_sup: int

def ssl_units(b: Budget, c_f: float = 1.0, c_b: float = 2.0) -> float:
    return 2.0 * (c_f + c_b) * b.n_pre * b.e_pre + c_f * b.l_probe * b.e_probe

def sup_units(b: Budget, c_f: float = 1.0, c_b: float = 2.0) -> float:
    return (c_f + c_b) * b.l_sup * b.e_sup

def report(b: Budget):
    u_a = ssl_units(b)
    u_b = sup_units(b)
    assert b.l_probe == b.l_sup, "label budget mismatch"
    return {"ssl": u_a, "sup": u_b, "label_ok": True}

b = Budget(n_pre=500, e_pre=8, l_probe=200, e_probe=5, l_sup=200, e_sup=20)
r = report(b)
assert r["ssl"] == 25000.0
assert r["sup"] == 12000.0
print(r)
```

A kód lefutott (Python 3.12.3); mért kimenet: `{'ssl': 25000.0, 'sup': 12000.0, 'label_ok': True}`. A teszt elutasító képességét szándékosan elrontott `n_pre` értékekkel is ellenőrizzük:

```
def mutation_rejected(n_pre, expected):
    bad = Budget(n_pre=n_pre, e_pre=8, l_probe=200, e_probe=5,
                 l_sup=200, e_sup=20)
    try:
        assert report(bad)["ssl"] == expected
    except AssertionError:
        return True
    return False

assert mutation_rejected(1000, 25000.0)  # kétszeres pretraining-képszám
assert mutation_rejected(250, 25000.0)   # feles pretraining-képszám
print("mutation check: both rejected")
```

A mutációs kód is lefutott; mért kimenete: `mutation check: both rejected`. A hibás konfigurációkra a `report` 49000, illetve 13000 `ssl` értéket ad, ezért a `25000.0`-ra tett egyenlőség-ellenőrzés mindkét esetben elbukik.

Az 1-es relatív tolerancia 100%-os eltérést engedne, és a keretszámítás szorzóhibáit (kétszeres vagy feles költség) is elfedné, ezért nem használjuk. Egész elemszámú költségnél pontos egyenlőség (`==`) a helyes ellenőrzés, mert ezek az értékek float64-ben pontosan ábrázolhatók, és a szorzások itt nem kerekítenek; lebegő arányoknál kis tolerancia (10^{-9} relatív) kell. A fenti mutációs ellenőrzés ezt az elutasító képességet mérte.

F4 megoldás. A hiba következménye: az enkóder látja a test képeit, még ha címkék nélkül is. A kontrasztív tanulás ekkor a test képek példányszintű statisztikájához igazodhat, ezért a

tesztpontosság optimistán torzít. A mechanizmus konkrét: a pretraining batch negatívjait a batch többi eleme adja, ezért a test képek jelenléte megváltoztatja a negatívok eloszlását, és ha az enkóder BatchNorm-réteget tartalmaz, a futó statisztikák is a test képekből frissülnek. Mindkettő transzduktív: a modell a teszt adat eloszlásához igazodik, miközben a tesztcímkékhez nem fér hozzá. A `metadata.csv` `scene_id` oszlopából képzett halmazok metszetét kell ellenőrizni:

```
import pandas as pd

meta = pd.read_csv("data/metadata.csv")
pretrain_ids = set(meta.loc[meta.split == "train", "scene_id"])
test_ids = set(meta.loc[meta.split == "test", "scene_id"])
assert pretrain_ids & test_ids == set(), "scene leakage"
```

Javítás: a pretraining listához csak a train split képeit vegyük, a test splitet se képként, se augmentált nézetként ne használjuk. Visszaellenőrzés: az üres metszet mellett a pretraining indexek mindegyike a `train` splitben legyen, és a végső tesztet csak a rögzített protokoll legvégén futtassuk.

F5 megoldásminta. Protokoll: ugyanaz az enkóder, $d = 32$; a train split 240 képe a pretraining és a probe alapja, a val split a modellválasztásé, a test split csak záró kiértékelésre szolgál. Változó tényező: a pretext cél, kontrasztív (`n119`-ből választott loss) és negatív nélküli stop-gradient/teacher. Mindkétághoz azonos augmentáció tartozik. A számítási keretet a labor könyvelési képleteivel állítjuk be: előre rögzítjük N_{pre}, E_{pre}, L_{probe} és E_{probe} értékét, kiszámítjuk U_A-t, majd a baseline $E_B = \lceil U_A/((c_f + c_b)L_B) \rceil$ epochszámát használjuk, így $U_B \geq U_A$ és $|U_B - U_A|$ kicsi. Mindkét ág után azonos frozen probe fut, $L = 240$ címkével. Metrika: test top-1 pontosság (a legnagyobb logit szerinti osztály a helyes címke) három seeddel, átlag és szórás; másodlagos, előre rögzített metrika a korrupt test páros összevetése. A döntést a jelenet szintű páros különbség bootstrap CI-je adja: 0-t kizáró 95%-os CI esetén állítunk különbséget, egyébként nem; a három seed szórása csak a tanítási varianciát jelzi. Baseline: ugyanaz az enkóder felügyelt traininggel, matched compute és $L = 240$. Következtetési korlát: a kis GlyphLab adat nem természetes képi benchmark; az eredmény csak erre a feladatra és erre a számítási keretre érvényes. Előre nem rögzítünk győztest; a jegyzőkönyvben a nyers pontosságokat, a seedeket és az eltéréseket közöljük.

30.4.8 Tipikus tévedés ellenpéldával

A leggyakoribb hiba az erőforrás-előny átcímkézése reprezentációs előnnyé. Például ugyanaz az enkóder kontrasztív pretraininggel 0,84, felügyelt tanítással 0,79 pontosságot ér el; a hallgató ebből azt írja, hogy a kontrasztív cél jobb reprezentációt ad. A kézi példa azonban megmutatja, hogy a kontrasztív ág 27-szer több normalizált számítást és 2000 extra címkézetlen képet használhatott. Ha a felügyelt ág ugyanazt a keretet kapja, a különbség eltűnhet vagy előjelet válthat. Ez a kontroll hiánya, nem architekturális bizonyíték.

30.4.9 Önellenőrzés és visszahívás

1. Mit jelent a frozen probe gradiensszabálya? Az enkóderparaméterre $\nabla_\theta L = 0$, csak a fej paramétereit frissítjük.
2. Miért nem elég a címke-egyezés? Mert a több pretraining kép és a nagyobb számítás ugyancsak javíthat, ahogy a lemma mutatja.
3. Miért kell a pretraining és a teszt `scene_id`-halmazát diszjunktan tartani? Mert egy jelenet két nézete ugyanazt a `scene_id`-t viseli, ezért ha az egyik nézet a pretrainingben, a másik a tesztben szerepel, a modell gyakorlatilag a teszt jelenetet is látta; a felosztás egysége

ezért a `scene_id`, nem a kép. Visszahívás: a 30.3. Negatív nélküli módszerek és collapse alfejezetben tárgyalt stop-gradient és teachermechanizmus itt csak mint pretext cél szerepel; a 18.4. Reziduális blokk ablatív vizsgálata matched recipe gondolatát erőforrás-könyveléssel egészítettük ki. A 29.4. Anomáliapontszám rekonstrukcióból anomáliaküszöbéhez hasonlóan itt is a modellválasztást (checkpoint, hiperparaméter) a validációs halmazon kell elvégezni, a tesztet csak a végén, egyszer értékeljük.

30.4.10 Rövid összefoglaló

Az önfelügyelt és a felügyelt ág összehasonlításához explicit erőforrás-könyvelés kell. A címkeköltség a felhasznált kép-címke párok száma; a számítási költség normalizált forward- és backward-egységekben írható fel. A kontrasztív pretraining egy eredeti képre két nézet miatt kétszeres encoderforward- és backward-költséget jelent. Ha a két ág címke-, számítási vagy adatmedencéje eltér, a pontszámkülönbségből a módszerhatás nem azonosítható, amit az additív feltevés mellett, a lemma keretei között mutattunk meg; ez nem általános azonosíthatósági tétel. A frozen probe az enkódert rögzíti, és csak a lineáris fejet tanítja. A könyvelő kód lefutott, a mért kimenet a kézi példa konfigurációjára 243000 és 9000 egység; a reprodukciós kísérlet viszont nem futott le, ezért abból származó pontosságot nem közlünk.

30.4.11 Következő alkalmazás

A 31.1. VAE és variációs alsó korlát a variációs autoenckódert vezeti be. A most bevezetett frozen probe és költségkeret ott is használható: a latens reprezentáció értékelése előtt rögzíteni kell a downstream címke- és számítási protokollt, mert a generatív rekonstrukciós pontszám önmagában nem downstreambizonyíték.

30.4.12 Források

- [P17] Chen, T. et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. arXiv:2002.05709. https://arxiv.org/abs/2002.05709.

31 Generatív modellek: VAE, GAN és diffúziós alapgondolat

31.1 VAE és variációs alsó korlát

31.1.1 1. Cél, előismeret és induló kérdés

Az autoencoder 29.4. Anomáliapontszám rekonstrukcióból determinisztikus $x \mapsto z \mapsto \hat{x}$ leképezése akkor hasznos, ha csak rekonstrukció kell, de nem ad generatív leírást: nincs definiált valószínűségi modellje az adatra, és nincs értelmezett előzetes eloszlása a látens téren, amelyből új mintát húzhatnánk. A **variational autoencoder** (VAE) ezt egészíti ki egy látens változós valószínűségi modellel. A VAE-t Kingma és Welling (2013) vezette be; az alábbi ELBO-levezetés a könyv saját kidolgozása. Előismeret: a 11.4. Egy teljes PyTorch tanítási iteráció tanítási lépés, a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció stabil log-likelihood-számítás, a 29.4. Anomáliapontszám rekonstrukcióból rekonstrukciós pontszám korlátai, és az 5.4. Függő képek, seedek és resampling statisztikai mintavételi egység gondolata. Visszautalás: a 30.1. Pretext és downstream feladat alfejezetben a reprezentációt downstream feladathoz kötöttük.

Induló kérdés: ha a megfigyelt x mögött látens z áll, akkor a posterior $p(z \mid x)$ a Bayes-szabályból adódna, de a nevező $p(x) = \int p(x, z)\, dz$. Mi a helyes tréningcél, ha ez az integrál gyakorlatban nem számolható? Becslés: nem a posterior közvetlen tanítása, hanem egy számolható alsó korlát maximalizálása.

31.1.2 2. Definíciók, jelölések és tenzoralakok

Legyen a megfigyelt adatvektor $x \in \mathbb{R}^D$, a látens vektor $z \in \mathbb{R}^K$, tipikusan $K \ll D$. Rögzítünk egy **priort** (előzetes eloszlást) $p(z)$; a leggyakoribb választás $p(z) = \mathcal{N}(z; 0, I_K)$.

A **decoder likelihood** egy paraméterezett feltételes sűrűség

$$p_\theta(x \mid z),$$

amely kimondja, hogyan keletkezik az adat a látens változóból. NCHW képbatch esetén $X \in \mathbb{R}^{N \times C \times H \times W}$, de a jelen levezetésben az x egyetlen megfigyelt vektor; batchelt alaknál az N dimenziót külön redukció hordozza.

A közös modell

$$p_\theta(x, z) = p_\theta(x \mid z)\, p(z),$$

a marginális likelihood pedig

$$p_\theta(x) = \int_{\mathbb{R}^K} p_\theta(x, z)\, dz.$$

A valódi posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)},$$

amely általában nem zárt alakú, mert a nevező nagydimenziós integrál.

Emiatt bevezetünk egy **encoder distribution**-t,

$$q_\phi(z \mid x),$$

amely a posterior közelítése. Tenzorparaméterezés: az encoder egy $N \times D$ bemenetből $\mu \in \mathbb{R}^{N\times K}$ és $\log \sigma^2 \in \mathbb{R}^{N\times K}$ tenzort ad; ezzel

$$q_\phi(z \mid x) = \mathcal{N}\big(z; \mu_\phi(x), \operatorname{diag}\big(\sigma^2_\phi(x)\big)\big)\,.$$

A $\log \sigma^2$ paraméterezés numerikusan előnyös: a szórásnégyzet pozitivitása kerül a logaritmus mögé, hasonlóan ahhoz, ahogy a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció alfejezetben a logitok stabilitását kezeltük.

A **KL-divergencia** két folytonos sűrűség között

$$D_{KL}(q\|p) = \int q(z) \log \frac{q(z)}{p(z)}\, dz,$$

ahol a log természetes alapú. A divergencia csak akkor véges, ha $q(z) = 0$ ott, ahol $p(z) = 0$; ezt nevezzük a jelen fejezetben támogatási feltételnek. $D_{KL}(q\|p) \geq 0$, egyenlőség akkor és csak akkor, ha a két sűrűség egybeesik (a nullmértékű eltéréstől eltekintve).

A **variációs alsó korlát**, más néven **ELBO** (evidence lower bound):

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x, z) - \log q_\phi(z \mid x)\Big].$$

31.1.3 3. Az ELBO levezetése Jensen-egyenlőtlenségből

A levezetéshez a Jensen-egyenlőtlenséget megadott alakban vesszük:

Jensen (adott alak). Ha $Y \geq 0$ véges várható értékű véletlen változó, és log konkáv, akkor $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$, ahol a $-\infty$ is megengedett. A szigorú konkavitás miatt egyenlőség akkor van, ha Y konstans a valószínűség szerint egy nullmértékű halmaztól eltekintve. A mértékelméleti finomságot itt nem tárgyaljuk; a tankönyv véges sűrűségekre szorítkozik.

Tétel (ELBO alsó korlát). Tegyük fel, hogy rögzített x-re $q_\phi(z \mid x) > 0$ mindenütt, ahol $p_\theta(x, z) > 0$, továbbá $0 < p_\theta(x) < \infty$ és $\mathbb{E}_{q_\phi(z|x)}\left|\log \frac{p_\theta(x,z)}{q_\phi(z|x)}\right| < \infty$. Ekkor

$$\log p_\theta(x) \geq \mathcal{L}(x; \theta, \phi).$$

Egyenlőség akkor és csak akkor áll fenn, ha $q_\phi(z \mid x) = p_\theta(z \mid x)$ a q_ϕ szerint majdnem mindenütt.

Megjegyzés a feltételekhez. A Jensen-lépéshez a $0 < p_\theta(x) < \infty$ feltétel külön kell: az $\mathbb{E}_q|\log R| < \infty$ feltételből nem következik $\mathbb{E}_q[R] < \infty$, márpedig a $\log \mathbb{E}_q[R] = \log p_\theta(x)$ lépéshez $p_\theta(x)$ végessége kell. A $p_\theta(x) > 0$ feltétel azt biztosítja, hogy $\log p_\theta(x)$ értelmes legyen; ha $p_\theta(x) = 0$, az állítás triviálisan $-\infty$ lenne. Folytonos x esetén $p_\theta(x)$ sűrűség, nem egy pont valószínűsége.

Bizonyítás. A támogatási feltétel miatt a marginális felírható a következőképpen:

$$p_\theta(x) = \int_{\mathbb{R}^K} p_\theta(x, z)\, dz = \int_{\mathbb{R}^K} q_\phi(z \mid x)\, \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\, dz = \mathbb{E}_{q_\phi(z|x)}[R(z)],$$

ahol $R(z) = p_\theta(x,z)/q_\phi(z \mid x)$. A véges-ELBO feltétel miatt R integrálható, és $R > 0$ a q_ϕ szerint. Alkalmazzuk a Jensen-egyenlőtlenséget a konkáv log-ra:

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi(z|x)}[R(z)] \geq \mathbb{E}_{q_\phi(z|x)}[\log R(z)] = \mathcal{L}(x;\theta,\phi).$$

Az egyenlőség feltételéből $R(z)$ konstans q_ϕ-szerint; mivel az átlaga $p_\theta(x)$, ezért

$$\frac{p_\theta(x,z)}{q_\phi(z \mid x)} = p_\theta(x),$$

átrendezve $q_\phi(z \mid x) = p_\theta(x,z)/p_\theta(x) = p_\theta(z \mid x)$. □

A gyakorlatban használt felbontás:

Lemma. A fenti feltételek mellett

$$\mathcal{L}(x;\theta,\phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)\,.$$

Bizonyítás.

$$\mathcal{L} = \mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\phi(z \mid x)\Big] = \mathbb{E}_q[\log p_\theta(x \mid z)] - \mathbb{E}_q\left[\log \frac{q_\phi(z \mid x)}{p(z)}\right],$$

és a második tag éppen a KL-divergencia. □

Az első tag a **rekonstrukciós tag**: a látensből adott x log-likelihoodja. Ha a dekóder Gauss-likelihood rögzített σ_d^2 varianciával:

$$\log p_\theta(x \mid z) = -\frac{D}{2}\log(2\pi\sigma_d^2) - \frac{1}{2\sigma_d^2}\|x - \mu_\theta(z)\|_2^2,$$

tehát a rekonstrukciós tag az MSE negatív, konstans- és skálafaktorral összekötve. Ez a 29.4. Anomáliapontszám rekonstrukcióból alfejezetben használt rekonstrukciós pontszám valószínűségi jelentése. Fontos: rögzített varianciájú Gauss-dekóder mellett az MSE és a rekonstrukciós tag ugyanazt a rekonstrukciós optimumot adja, de a teljes ELBO-ban a $\frac{1}{2\sigma_d^2}$ tényező a rekonstrukciós tag és a KL-tag közötti kompromisszumot szabályozza, és általában az optimumot is megváltoztatja; a teljes ELBO tehát nem ekvivalens változatlanul egy skálázatlan MSE+KL céllal. A β-szorzó bevezetésével (-VAE, Higgins és mtsai, 2017) a két tag külön súlyozható, de az így kapott cél már nem ugyanazon marginális likelihood alsó korlátja.

Két további, gyakran kevert állapot:

- Az ELBO (a q szerinti várható érték) sosem haladja meg $\log p_\theta(x)$-et; egy **Monte-Carlo becslés** azonban felülmúlhatja, mert az ELBO-t becsüli, és a becslés sztochasztikus hibája miatt egy realizáció a valódi ELBO fölé, ezáltal $\log p_\theta(x)$ fölé is kerülhet.
- A reprezentációtanulás célja 30.1. Pretext és downstream feladat különbözik a generatív modellétől: a downstream pontosság javulása nem szükségszerű, és nem következik az ELBO növeléséből (tényleges javulás lehetséges, de az ELBO növekedése önmagában nem bizonyítja a downstream javulást), hacsak nincs az adatszerződésbe iktatott mérés.

31.1.4 4. Eredeti kézi számpélda

Legyen $D = 2$, $K = 1$, $p(z) = \mathcal{N}(0,1)$, $q_\phi(z \mid x) = \mathcal{N}(0{,}2; 0{,}25)$, a dekóder lineáris Gauss: $\mu_\theta(z) = W_{\text{dec}} z$, $W_{\text{dec}} = (1; -0{,}5)$ (a kódban `W`), $\sigma_d^2 = 0{,}16$, és az adat $x = (0{,}4; -0{,}2)$. A Gauss–Gauss KL zárt alakját (lásd az F2 feladatot) használva:

$$D_{KL}(q\|p) = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right) = \frac{1}{2}(0{,}25 + 0{,}04 - 1 + 1{,}386294) = 0{,}338147.$$

A rekonstrukciós tag várható értékéhez a lineáris dekóder miatt:

$$\mathbb{E}_q\|x - W_{\text{dec}} z\|^2 = \|x - W_{\text{dec}}\mu\|^2 + \sigma^2\|W_{\text{dec}}\|^2.$$

Itt $\|x - W_{\text{dec}}\mu\|^2 = \|(0{,}2; -0{,}1)\|^2 = 0{,}05$, $\|W_{\text{dec}}\|^2 = 1{,}25$, ezért $0{,}05 + 0{,}25 \cdot 1{,}25 = 0{,}3625$. A konstanssal:

$$\mathbb{E}_q \log p(x \mid z) = -\log(2\pi \cdot 0{,}16) - \frac{0{,}3625}{2 \cdot 0{,}16} = -0{,}005296 - 1{,}132813 = -1{,}138108.$$

Így az ELBO $-1{,}138108 - 0{,}338147 = -1{,}476255$.

Ellenőrzésként a modell marginálisa zárt alakú, mert lineáris Gauss: $p_\theta(x) = \mathcal{N}\left(0, \sigma_d^2 I_2 + W_{\text{dec}} W_{\text{dec}}^\top\right)$, mivel a generatív prior $p(z) = \mathcal{N}(0,1)$ varianciája 1, nem pedig a közelítő posterior q-ban szereplő $\sigma^2 = 0{,}25$. Numerikusan $\log p_\theta(x) \approx -1{,}164303$, tehát valóban ELBO $\le \log p_\theta(x)$, a rés körülbelül 0,311952, ami $D_{KL}(q_\phi(z \mid x)\|p_\theta(z \mid x))$.

Mennyiség	Képlet/közelítő	Érték
$\mathbb{E}_q \log p(x \mid z)$	$-\log(2\pi\sigma_d^2) - \frac{1}{2\sigma_d^2}\mathbb{E}_q\|x - W_{\text{dec}} z\|^2$	$-1{,}138108$
$D_{KL}(q\|p)$	$\frac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2)$	0,338147
ELBO	rekonstrukciós tag mínusz KL	$-1{,}476255$
Egymintás MC a $z = 0{,}3$-nál	$\log p(x \mid z) + \log p(z) - \log q(z \mid x)$	$-0{,}762506$

Az egymintás Monte-Carlo érték nagymértékben eltér az ELBO-tól; ez a kézi példa determinisztikus számított értéke ($z = 0{,}3$ behelyettesítésével), nem annak a bizonyítéka, hogy az ELBO-t egy mintával becsülni kell.

31.1.5 5. Konkrét labor: toy Gauss-látens, rekonstrukció/KL skála

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A következő kód a fenti kézi példát hajtja végre float64 CPU-n; a determinisztikus értékek a 4. szakasz számításaiból jönnek, az MC-sor a kód tényleges futtatásából származik. Munkakönyvtár: `lab/n121`.

```
# munkakönyvtár: lab/n121
# parancs: python vae_elbo_toy.py
import math
import torch

torch.manual_seed(20261020)
```

```
torch.set_default_dtype(torch.float64)
torch.set_num_threads(1)

x = torch.tensor([0.4, -0.2])              # (D,), D=2
mu = torch.tensor([0.2])                   # (K,), K=1
sigma2 = torch.tensor([0.25])              # q szórásnégyzete
W = torch.tensor([[1.0, -0.5]])           # (K,D) dekódolt lineáris súly, K=1
sigma_d2 = 0.16
D = x.shape[0]

def gauss_kl(mu, sigma2):
    # q=N(mu, diag(sigma2)), p=N(0,I)
    return 0.5 * (sigma2 + mu**2 - 1.0 - torch.log(sigma2)).sum()

def recon_expected(x, mu, sigma2, W, sigma_d2):
    # E_q ||x - W z||^2 = ||x - W mu||^2 + sigma2 * ||W||^2
    mean_residual = x - mu @ W                 # (D,)
    quad = mean_residual.pow(2).sum() + sigma2 * W.pow(2).sum()
    const = -0.5 * D * math.log(2.0 * math.pi * sigma_d2)
    return const - quad / (2.0 * sigma_d2)

def mc_elbo(x, mu, sigma2, W, sigma_d2, n_samples):
    sigma = sigma2.sqrt()
    with torch.no_grad():
        z = mu + sigma * torch.randn(n_samples, 1)   # (n_samples, K)
        log_pz = -0.5 * (math.log(2.0 * math.pi) + z.pow(2)).sum(dim=1)
        log_qz = -0.5 * (torch.log(2.0 * math.pi * sigma2)
                         + (z - mu).pow(2) / sigma2).sum(dim=1)
        diff = x - z @ W                             # (n_samples, D)
        log_px = -0.5 * D * math.log(2.0 * math.pi * sigma_d2) \
                 - diff.pow(2).sum(dim=1) / (2.0 * sigma_d2)
        return (log_px + log_pz - log_qz).mean()

kl = gauss_kl(mu, sigma2)
recon = recon_expected(x, mu, sigma2, W, sigma_d2)
elbo = recon - kl

C = sigma_d2 * torch.eye(2) + W.T @ W  # p(z)=N(0,1), nem q varianciája
logdet = torch.slogdet(C).logabsdet
maha = x @ torch.linalg.solve(C, x)
log_px = -0.5 * D * math.log(2.0 * math.pi) - 0.5 * logdet - 0.5 * maha

assert abs(kl.item() - 0.338147) < 1e-6
assert abs(recon.item() + 1.138108) < 1e-6
assert abs(elbo.item() + 1.476255) < 1e-6
assert abs(log_px.item() + 1.164303) < 1e-6
assert elbo.item() <= log_px.item()

mc = mc_elbo(x, mu, sigma2, W, sigma_d2, 100000)
assert abs(mc.item() - elbo.item()) < 0.01

print(f"KL(q||prior)      = {kl.item():.6f}")
print(f"E_q log p(x|z)    = {recon.item():.6f}")
print(f"ELBO (zart)       = {elbo.item():.6f}")
print(f"log p(x)          = {log_px.item():.6f}")
print(f"ELBO <= log p(x)  : {(elbo <= log_px).item()}")
print(f"MC 100000         = {mc.item():.6f}")
```

A kód lefutott (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, float64, seed 20261020, egy szál); mért kimenet:

```
KL(q||prior)      = 0.338147
E_q log p(x|z)    = -1.138108
```

```
ELBO (zart)       = -1.476255
log p(x)          = -1.164303
ELBO <= log p(x)  : True
MC 100000         = -1.473872
```

Értelmezés: a rekonstrukciós tag skálája csak a σ_d^2 likelihood-varianciával együtt értelmezhető; a KL-tag bünteti a priorhoz képesti eltérést. A két tag összege alsó korlát, tehát ha az egyik tagot önkényes β súllyal skálázzuk, az már nem ugyanazon marginális likelihood alsó korlátja (ez a β-VAE heurisztika, amelyet nem tárgyalunk ELBO-állításként). A mért MC-érték ($-1{,}473872$) a kódban használt 10^{-2} abszolút tűrésen belül van a zárt ELBO-hoz ($-1{,}476255$), ami a becslés konzisztenciáját mutatja.

31.1.6 6. Feladatok

1. **F1 (fogalmi, nehézség 1).** Különböztesse meg a $q_\phi(z \mid x)$ encoder eloszlást és a $p_\theta(z \mid x)$ valódi posteriort. Adjon olyan q-t és p-t, ahol $D_{KL}(q\|p)$ nem véges, és nevezze meg, mely támogatási feltétel sérül. Átadandó: rövid magyarázat és képletpélda.

2. **F2 (levezetési, nehézség 2).** Vezesse le a $q = \mathcal{N}(\mu, \sigma^2)$, $p = \mathcal{N}(0, 1)$ KL-zárt alakot $d = 1$-re, majd számítsa ki $\mu = 0{,}4$, $\sigma^2 = 1{,}5$ értéket. Átadandó: teljes levezetés és numerikus érték.

3. **F3 (implementációs, nehézség 2).** Készítsen batchelt függvényt, amely $x \in \mathbb{R}^{N \times D}$ mátrixra, q diagonális Gauss paraméterekkel és lineáris Gauss dekóderrel kiszámolja a zárt ELBO-t, majd MC-becsléssel ellenőrzi. Átadandó: teljes kód, shape-ellenőrzés, $S = 100000$ minta és a zárt értékkel való összevetés 10^{-2} relatív, illetve 10^{-3} abszolút tűréssel.

4. **F4 (hibakeresési, nehézség 2).** Az alábbi sor hibás: `z = mu + logvar * eps`, ahol `logvar` a $\log \sigma^2$ encoder kimenet. Azonosítsa a hibát, következményét, javítását és empirikus ellenőrzését. Átadandó: ok, következmény, javított kód, visszaellenőrzés.

5. **F5 (kísérleti/transzfer, nehézség 3).** Tervezzen a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-adatkészleten VAE-kísérletet, amely a `corrupted_test` (a 60 tesztkép zajjal és fényességgradienssel rontott, a teszttel azonos sorrendű változata) rekonstrukciós romlását párosan méri a tiszta tesztadathoz képest, anélkül hogy a korrupt halmazt modellválasztásra használná. Átadandó: protokoll a split, baseline, metrika, statisztikai egység és következtetési határ megadásával. A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

31.1.7 7. Megoldások és indoklás

F1 megoldás. A q_ϕ egy encoder által paraméterezett, zártan előállítható közelítő sűrűség; a $p_\theta(z \mid x)$ a modell posteriorja, amely a Bayes-szabályból adódik, és általában nem számolható, mert $p_\theta(x)$ integrált tartalmaz. Példa: $p(z \mid x) = \text{Uniform}[0, 1]$, $q = \mathcal{N}(0, 1)$. Ekkor q pozitív tömeget ad $[0, 1]$-en kívül, ahol $p = 0$; a $\log(q/0)$ tag divergál, így $D_{KL}(q\|p) = \infty$. A véges KL feltétele az, hogy $q(z) = 0$ ott, ahol $p(z) = 0$.

F2 megoldás. A Gauss-sűrűség logaritmusa:

$$\log q(z) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(z-\mu)^2}{2\sigma^2}, \qquad \log p(z) = -\frac{1}{2}\log(2\pi) - \frac{z^2}{2}.$$

A q szerinti várható értékben $\mathbb{E}[(z-\mu)^2]=\sigma^2$ és $\mathbb{E}[z^2]=\mu^2+\sigma^2$, ezért

$$\mathbb{E}_q \log q = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{1}{2}, \qquad \mathbb{E}_q \log p = -\frac{1}{2}\log(2\pi) - \frac{1}{2}(\mu^2+\sigma^2).$$

Kivonva:

$$D_{KL}(q\|p) = \mathbb{E}_q \log q - \mathbb{E}_q \log p = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right).$$

A megadott értékekkel $\frac{1}{2}(1{,}5 + 0{,}16 - 1 - \log 1{,}5) = 0{,}127268$. Ellenőrzés: $\log 1{,}5 \approx 0{,}405465$, a zárójeles tag $0{,}254535$, felezve $0{,}127268$.

F3 megoldás. A lineáris dekóderben a várható érték μW_{dec}, a szórásnégyzet pedig $\sum_k \sigma_k^2 \|W_{\text{dec},k}\|^2$; a diagonális variancia miatt a négyzetes hiba összegként bomlik.

```
import math, torch

torch.set_default_dtype(torch.float64)

def closed_elbo_batch(x, mu, var, W, sigma_d2):
    # x: (N,D), mu: (N,K), var: (N,K), W: (K,D)
    assert x.dim() == 2 and mu.dim() == 2, "x, mu: 2-D"
    assert mu.shape == var.shape and x.shape[0] == mu.shape[0], "batch/K eltérés"
    assert W.dim() == 2 and W.shape == (mu.shape[1], x.shape[1]), "W: (K,D)"
    N, D = x.shape
    K = mu.shape[1]
    recon_const = -0.5 * D * math.log(2.0 * math.pi * sigma_d2)
    mean_quad = (x - mu @ W).pow(2).sum(dim=1)          # (N,)
    w_norm2 = W.pow(2).sum(dim=1)                        # (K,)
    var_quad = (var * w_norm2.unsqueeze(0)).sum(dim=1)  # (N,)
    recon = recon_const - (mean_quad + var_quad) / (2.0 * sigma_d2)
    kl = 0.5 * (var + mu.pow(2) - 1.0 - var.log()).sum(dim=1)
    return recon - kl, recon, kl

def mc_elbo_batch(x, mu, var, W, sigma_d2, S):
    assert x.dim() == 2 and mu.dim() == 2, "x, mu: 2-D"
    assert mu.shape == var.shape and x.shape[0] == mu.shape[0], "batch/K eltérés"
    assert W.dim() == 2 and W.shape == (mu.shape[1], x.shape[1]), "W: (K,D)"
    N, D = x.shape
    N_, K = mu.shape
    eps = torch.randn(N, S, K)
    z = mu.unsqueeze(1) + var.unsqueeze(1).sqrt() * eps
    log_pz = -0.5 * (math.log(2.0 * math.pi) + z.pow(2)).sum(-1)
    log_qz = -0.5 * (torch.log(2.0 * math.pi * var.unsqueeze(1))
                     + (z - mu.unsqueeze(1)).pow(2) / var.unsqueeze(1)).sum(-1)
    diff = x.unsqueeze(1) - z @ W                # (N,S,D)
    log_px = (-0.5 * D * math.log(2.0 * math.pi * sigma_d2)
              - diff.pow(2).sum(-1) / (2.0 * sigma_d2))
    return (log_px + log_pz - log_qz).mean(dim=1)

x = torch.tensor([[0.4, -0.2], [0.1, 0.3]])
mu = torch.tensor([[0.2], [0.0]])
var = torch.tensor([[0.25], [1.0]])
W = torch.tensor([[1.0, -0.5]])
closed = closed_elbo_batch(x, mu, var, W, 0.16)[0]
# ellenőrzés: closed = [-1.476255, -4.224046]
torch.manual_seed(7)
mc = mc_elbo_batch(x, mu, var, W, 0.16, 100000)
print(closed)
print(mc)
assert torch.allclose(closed, mc, rtol=1e-2, atol=1e-3), "MC eltérés túl nagy"
```

A második sor zárt értéke: $\mu=0$, $\sigma^2=1$ miatt $\mathbb{E}_q\|x-W_{\text{dec}}z\|^2 = \|x\|^2 + 1\cdot 1{,}25 = 0{,}10 + 1{,}25 = 1{,}35$, a rekonstrukciós tag $-0{,}005296 - 1{,}35/0{,}32 = -4{,}224046$, a KL $\frac{1}{2}(1+0-1-\log 1) = 0$.

F4 megoldás. A hiba oka: `logvar` a $\log \sigma^2$, ezért közvetlen szorzóként nem a szórást használja. A mintavételezett z eloszlása nem az a q, amelynek KL-jét kiszámoljuk, így a gradiensbecslés inkonzisztens. Következmény: a tényleges mintavételi variancia $(\log \sigma^2)^2$ lenne σ^2 helyett, ami a paramétertől függően túl nagy vagy túl kicsi is lehet (például $\log \sigma^2 = -2$ esetén 4 a helyes $e^{-2} \approx 0{,}135$ helyett). Javítás:

```
z = mu + torch.exp(0.5 * logvar) * eps
```

Ellenőrzés: sok eps minta után $\mathrm{Var}(z) \approx \exp(\mathrm{logvar})$. A mintavett varianciát a várt értékkel relatív toleranciával kell összevetni, például `torch.allclose(z.var(0), logvar.exp(), rtol=1e-2, atol=1e-6)`; a 0,01 abszolút tűrés kis varianciáknál (például $e^{-5} \approx 0{,}0067$ esetén) a hibát teljesen elfedné, ezért nem használjuk.

F5 megoldásminta. Adat: a GlyphLab `data/images.npy` és `data/metadata.csv`; a felosztás egysége a `scene_id`, a szerződés szerinti 120/30/30 train/val/test jelenet, azaz 240/60/60 kép. A `val` split a modellválasztásé: a checkpointot és a hiperparamétereket kizárólag a validációs rekonstrukciós MSE alapján választjuk; a teszt splitet csak a záró kiértékelésnél használjuk. Modell: $K = 8$ látens dimenziójú, konvolúciós encoder/decoder VAE; baseline: ugyanaz az architektúra determinisztikus autoencoderrel, ahol az encoder a VAE két feje (μ és $\log \sigma^2$) helyett egyetlen z-kimenetet ad, ezért a paraméterszám csak a kimeneti rétegben tér el. Mindkét ág a könyv GlyphLab-laborjainak rögzített receptjével fut (Adam, tanulási ráta 10^{-3}, batch 32, legfeljebb 30 epoch, seed 20261020, float32 CPU), és ugyanazt az MSE rekonstrukciós célot használja, ezért a számítási keret összevethető. Metrika: képenkénti MSE, majd jelenetenként a két nézet átlaga. A kiválasztott checkpoint a `val` rekonstrukciós MSE minimumánál mentett állapot. Teszt: a tiszta teszt és a `corrupted_test` (a 60 tesztkép zajjal és fényességgradienssel rontott, a teszttel azonos sorrendű változata) páronként, ugyanazon `scene_id`/`view_id` sorrendben; a páros különbséget jelenetalapú (cluster) bootstrap intervallummal jellemezzük, ahol a `scene_id`-kat visszatevéssel újramintavételezzük, mert egy jelenet két nézete nem független megfigyelés. Következtetési határ: a korrupt hiba növekedése a betanítási tartomány érzékenységét méri, nem általános valós képminőséget; negatív vagy nulla eredmény is elfogadható korrekt riporttal.

31.1.8 8. Gyakori tévedés ellenpéldával

A gyakori tévedés: „a VAE nem más, mint hagyományos autoencoder zajos látenssel.” Ez hiányos, mert nem mond likelihoodot. Tekintsünk két modellt, amelyek ugyanazt az encoder–decoder párt használják, és mindkettő ugyanazt a mintavételezett $z = \mu + \sigma\varepsilon$ rekonstrukciót adja. Az egyik $p(x \mid z) = \mathcal{N}(x; \mu_\theta(z), I)$ értékkel definiálja a likelihoodot, a másik $\mathcal{N}(x; \mu_\theta(z),\ 0{,}01I)$-vel. Az MSE-optimum ugyanaz, de a rekonstrukciós tag skálája és a teljes ELBO különbözik. A fő különbség a $\frac{1}{2\sigma_d^2}$ súly: $\sigma_d^2 = 1$ esetén 0,5, $\sigma_d^2 = 0{,}01$ esetén 50, miközben a normalizáló konstans $-\frac{D}{2}\log(2\pi\sigma_d^2)$ is eltér. A KL-tag változatlan, ezért a rekonstrukció és a KL relatív súlya más, ami az optimumot is eltolhatja. Zajos látens tér önmagában nem ad generatív célfüggvényt; a VAE-t az $\mathcal{L}$ maximalizálása és a hozzá tartozó likelihood definiálja.

31.1.9 9. Önellenőrzés és visszahívás

Ellenőrzés: a kézi példában KL≥ 0; ELBO$= -1{,}476255 \leq \log p(x) = -1{,}164303$; az egymintás MC nem helyettesíti a zárt várható értéket. Kódellenőrzés: z (S, K), W (K, D), ezért a dekódolás `z @ W` mátrixszorzással (S, D)-t ad; a KL-tag a K dimenzión összegzendő. Numerikus ellenőrzés: `float64`, `cpu`, `rtol=1e-2`, `atol=1e-3`. Visszahívás: a 29.4. Anomáliapontszám rekonstrukcióból

alfejezetben bevezetett rekonstrukciós pontszám itt a Gauss-dekóder likelihood negatív, konstanseltolt változataként jelenik meg; a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció log-likelihood-stabilizálást kategorikus dekódernél kell újra elővenni.

31.1.10 Rövid összefoglaló

A VAE látens z változót vezet be, priorral és $p_\theta(x \mid z)$ dekóder-likelihooddal. A posterior $p_\theta(z \mid x)$ a $p_\theta(x)$ marginalizáló integrál miatt általában nem zárt, ezért $q_\phi(z \mid x)$ encoder-közelítést illesztünk. Az ELBO a Jensen-egyenlőtlenségből a $\log p_\theta(x)$ alsó korlátjaként vezethető le, és felbomlik rekonstrukciós tagra és $D_{KL}(q_\phi \| p)$ büntetőtagra. A levezetés feltétele a támogatási feltétel és az integrabilitás. A toy Gauss-példa a rekonstrukció/KL skáláját, a labor a zárt képlet és az MC-becslés viszonyát ellenőrzi.

31.1.11 Következő alkalmazás

A 31.2. GAN mint minimax probléma a GAN minimax célját mutatja be; ott az adott likelihoodú generatív cél helyett egy diszkriminátor jelenti a tanítójelet. A most megismert likelihood-, prior- és posteriorfogalom ahhoz kell, hogy a két modellcsalád feltevéseit elkülönítsük.

31.1.12 Források

- Kingma, D. P. és Welling, M. (2013). *Auto-Encoding Variational Bayes*. arXiv:1312.6114. https://arxiv.org/abs/1312.6114. Az ELBO-levezetés a könyv saját kidolgozása; a cikk kísérleti eredményeit nem reprodukáljuk.
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S. és Lerchner, A. (2017). *beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework*. International Conference on Learning Representations (ICLR). A -VAE súlyozás forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.
- Deisenroth, M. P., Faisal, A. A. és Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. A Jensen-egyenlőtlenséget a könyvben megadott alakban használjuk.

31.2 GAN mint minimax probléma

31.2.1 Cél, előismeret és induló kérdés

Ez az egység a generatív adversariális hálót (GAN) nem likelihood-közelítésként, hanem két szereplős játékként vezeti be. A 31.1. VAE és variációs alsó korlát VAE-jében a tanulási cél az ELBO, tehát egy explicit látens posterior és egy dekódoló likelihood együttes maximalizálása volt. A GAN ezt a becslési keretet elhagyja: nincs $q(z|x)$ következtető háló és nincs dekódoló likelihood, csak egy generátor és egy diszkriminátor versenye. Előismeret a 7.1. Entrópia, cross-entropy és KL keresztentrópiája és a 30.2. Kontrasztív cél és temperature logisztikus alakú célfüggvénye.

Induló kérdés: ha a generátor már adott, milyen diszkriminátor hozza ki a legtöbbet a valós és generált minták megkülönböztetéséből? Ennek zárt alakja lesz a központi levezetés.

31.2.2 Minimax cél, generátor és diszkriminátor

Legyen p_{data} a valós adatok sűrűsége $\mathbb{R}^{d_x}$-en, p_z pedig egy rögzített látens prior, tipikusan standard normális $\mathbb{R}^{d_z}$-n. A generátor

$$G_\theta : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}, \qquad x_g = G_\theta(z),$$

a diszkriminátor pedig

$$D_\phi : \mathbb{R}^{d_x} \to (0,1), \qquad D_\phi(x) = \sigma(s_\phi(x)),$$

ahol $s_\phi(x) \in \mathbb{R}$ a logit, σ a szigmoid. A G_θ által indukált eloszlást p_g-vel jelöljük. A [P19]-ben bevezetett kétjátékos cél természetes alapú logaritmussal:

$$V(D_\phi, G_\theta) = \mathbb{E}_{x\sim p_{\text{data}}}[\ln D_\phi(x)] + \mathbb{E}_{z\sim p_z}[\ln(1 - D_\phi(G_\theta(z)))].$$

A második tag átírható a generált eloszlás szerinti várható értékké:

$$V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\ln D(x)] + \mathbb{E}_{x\sim p_g}[\ln(1 - D(x))]. \tag{1}$$

A D maximalizálni, a G minimalizálni akarja V-t, tehát a formális cél $\min_G \max_D V(D,G)$. Ez nem „egyetlen közös loss” minimalizálása: ugyanaz a V a két paraméterkészletben ellentétes irányú.

A PyTorch-féle köteges (batchelt) alakok: $Z \in \mathbb{R}^{B\times d_z}$; valós adat $X \in \mathbb{R}^{B\times d_x}$, képnél NCHW esetén $X \in \mathbb{R}^{B\times C\times H\times W}$; a generált minta $X_g \in \mathbb{R}^{B\times d_x}$; a diszkriminátor logitja $S \in \mathbb{R}^{B\times 1}$, valószínűségi kimenete $D(X) \in (0,1)^{B\times 1}$. Az osztálycímke 1 a valós, 0 a generált mintára.

A gyakorlati frissítés szétválasztott: θ-t csak a generátorcél, ϕ-t csak a diszkriminátorcél szerint frissítjük. Legyen

$$\ell_{\text{BCE}}(s,y) = -\big[y \ln \sigma(s) + (1-y)\ln(1-\sigma(s))\big].$$

Egy B méretű minibatchre (részkötegre)

$$L_D = \frac{1}{B}\sum_{i=1}^{B} \ell_{\text{BCE}}(s_\phi(x_i), 1) + \frac{1}{B}\sum_{i=1}^{B} \ell_{\text{BCE}}(s_\phi(G_\theta(z_i)), 0). \tag{2}$$

A második tagban a generált mintát `detach()`-kel leválasztjuk (a tenzort az autograd-gráfról leválasztva), mert L_D nem frissíti θ-t. A generátor eredeti, telítődő célja

$$L_G^{\text{sat}} = \frac{1}{B}\sum_i \ln(1 - D_\phi(G_\theta(z_i))), \tag{3}$$

a nem telítődő (non-saturating) változat pedig

$$L_G^{\text{ns}} = -\frac{1}{B}\sum_i \ln D_\phi(G_\theta(z_i)) = \frac{1}{B}\sum_i \ell_{\text{BCE}}(s_\phi(G_\theta(z_i)), 1). \tag{4}$$

PyTorchban a (4) a `binary_cross_entropy_with_logits` hívás `ones_like` targettel, a (3) pedig $\ln\sigma(-s) =$ `logsigmoid`$(-s)$.

31.2.3 Az ideális diszkriminátor levezetése

A következő állítás idealizált függvényosztályra szól; nem állítja, hogy egy véges neurális háló ezt eléri.

Állítás (optimális diszkriminátor rögzített generátor mellett). Legyen p_{data} és p_g sűrűség ugyanazon $\Omega \subseteq \mathbb{R}^{d_x}$ tartón. A $p_{\text{data}} + p_g$ mérték szerint majdnem minden x-re $p_{\text{data}}(x) + p_g(x) > 0$, és a D függvényt az idealizált, pontonként szabadon választható $[0,1]$-beli értékű mérhető függvények körében keressük, a $0 \ln 0 = 0$ konvencióval. Ekkor az (1) cél pontonkénti szuprémumát

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \tag{5}$$

adja.

Bizonyítás. Rögzített x-re legyen $a = p_{\text{data}}(x) \geq 0$, $b = p_g(x) \geq 0$, és $y = D(x)$. Az integrandus ott

$$f(y) = a \ln y + b \ln(1-y), \qquad y \in (0,1).$$

A deriváltak

$$f'(y) = \frac{a}{y} - \frac{b}{1-y}, \qquad f''(y) = -\frac{a}{y^2} - \frac{b}{(1-y)^2} \leq 0,$$

tehát f konkáv minden rögzített (a,b) mellett. A kritikus pont

$$f'(y) = 0 \iff a(1-y) = by \iff y = \frac{a}{a+b}.$$

Ez a konkavitás miatt egyértelmű globális maximum. Ha $a = 0, b > 0$, akkor $y = 0$ adja a szuprémumot; ha $a > 0, b = 0$, akkor $y = 1$ adja a szuprémumot; az (5) képlet így a határpontokon is érvényes. Ha $a = b = 0$ (ez a feltétel szerint legfeljebb nullmértékű halmazon fordulhat elő), a pont az integrálhoz nem járul hozzá, ott D^* tetszőleges.

Mivel minden x-ben az integrál egy tagját függetlenül maximalizáltuk, bármely más D-re az integrandus pontonként legfeljebb akkora, mint D^*-nál, így $V(D) \leq V(D^*)$.

Az (5) képlet olvasata: ahol a valós adat sűrűbb a generáltnál, ott a diszkriminátor $1/2$-nél nagyobb valós valószínűséget ad; ahol fordítva, ott $\frac{1}{2}$-nél kisebbet. Ha a két eloszlás egyenlő, $D^* \equiv 1/2$.

Jensen–Shannon-kapcsolat. Legyen $m = (p_{\text{data}} + p_g)/2$. Ekkor

$$V(D^*) = \mathbb{E}_{p_{\text{data}}}\Big[\ln \frac{p_{\text{data}}}{p_{\text{data}} + p_g}\Big] + \mathbb{E}_{p_g}\Big[\ln \frac{p_g}{p_{\text{data}} + p_g}\Big].$$

Mivel $p_{\text{data}} + p_g = 2m$, és $\int p_{\text{data}} = \int p_g = 1$, ez

$$V(D^*) = D_{\mathrm{KL}}(p_{\text{data}} \| m) + D_{\mathrm{KL}}(p_g \| m) - \ln 4 = -\ln 4 + 2 D_{\mathrm{JS}}(p_{\text{data}} \| p_g). \tag{6}$$

A KL-nemnegativitásból $V(D^*) \geq -\ln 4$, egyenlőség a $p_{\text{data}} = p_g$ esetben áll fenn. Az eloszlástérbeli optimum tehát úgy áll elő, hogy a generátor a JS-távolságot nullára nyomja; ekkor $D^* = 1/2$. Ez idealizált eredmény: a $\min_G$ akkor vihető el $-\ln 4$-be, ha a generátorcsalád ténylegesen képes p_{data}-t előállítani.

31.2.4 Telítődés, nem telítődő loss és mode collapse

Legyen $h = s_\phi(x_g)$ egy generált minta logitja. A (3) telítődő loss deriváltja a logit szerint

$$\frac{\partial}{\partial h} \ln(1 - \sigma(h)) = \frac{\partial}{\partial h} \ln \sigma(-h) = -\sigma(h) = -D_\phi(x_g).$$

Ha a diszkriminátor határozottan „hamisnak” látja a generált mintát, $D_\phi(x_g) \approx 0$, így ez a gradiens nullához tart. A (4) nem telítődő loss deriváltja ezzel szemben

$$\frac{\partial}{\partial h}[-\ln \sigma(h)] = -(1 - \sigma(h)) = D_\phi(x_g) - 1.$$

$D_\phi(x_g) \approx 0$ mellett ez ≈ -1, tehát nem tűnik el. Ezért a gyakorlati GAN-training a generátorra szinte mindig a (4) alakot használja. A „telítődés” tehát a G-gradiens eltűnése, nem a diszkriminátor pontossága önmagában.

Mode collapse (módusösszeomlás). Ha p_{data} több jól elkülönülő módust tartalmaz, de a generátor sok z-t ugyanabba a kis tartományba képez, akkor p_g csak egy vagy kevés módust fed le. Formális egységes definíció nincs; operatív jellemzői: **hiányzó módusok** (a valós adat egyes tartományaiba alig vagy egyáltalán nem esik generált minta), **csökkent változatosság** (a generált minták szórása, illetve a különböző minták aránya lényegesen kisebb, mint a valós adaté), és az, hogy **sok különböző látens pont közel azonos kimenetre** képeződik. A diszkriminátorra vonatkozó feltétel nem része a definíciónak: a diverzitás összeomlása erős diszkriminátor mellett is kialakulhat (egy ideális diszkriminátor éppen felismeri, hogy a két eloszlás eltér). Egy lehetséges dinamika, hogy a generátor egy olyan módusra tér át, amelyet a diszkriminátor éppen nem büntet, majd a diszkriminátor alkalmazkodása után másik módusra ugrik; ez a jelenség dinamikájának egyik formája, nem a feltétele. A nem telítődő loss javítja a gradiens erősségét, de önmagában nem akadályozza meg a mode collapse-et.

31.2.5 Kézi példa és kódos ellenőrzés

Legyen $\Omega = \{0, 1, 2\}$ diszkrét, $p_{\text{data}} = (0{,}5;\ 0{,}3;\ 0{,}2)$ és $p_g = (0{,}2;\ 0{,}5;\ 0{,}3)$. Az (5) képlet pontonként:

x	p_{data}	p_g	D^*
0	0,5	0,2	$5/7 = 0{,}7143$
1	0,3	0,5	$3/8 = 0{,}3750$
2	0,2	0,3	$2/5 = 0{,}4000$

A cél értéke az optimumnál

$$V(D^*) = \sum_x \Big[p_{\text{data},x} \ln D^*_x + p_{g,x} \ln(1 - D^*_x) \Big].$$

Behelyettesítve négy tizedesre: $V(D^*) = -1{,}2845$. Az egyenletes $D = 0{,}5$ esetben

$$V(0{,}5) = 2\ln(0{,}5) = -1{,}3863,$$

tehát az ideális diszkriminátor magasabb értéket ad. Az $x = 0$ pontban a derivált-ellenőrzés: $p/D - q/(1-D) = 0{,}5/(5/7) - 0{,}2/(2/7) = 0{,}7 - 0{,}7 = 0$.

```
# munkakönyvtár: lab/n122/
# Futtatás: python optimal_d_demo.py
import numpy as np

p = np.array([0.5, 0.3, 0.2], dtype=np.float64)
q = np.array([0.2, 0.5, 0.3], dtype=np.float64)

def V(D):
    return float(np.sum(p * np.log(D) + q * np.log(1.0 - D)))

D_star = p / (p + q)
D_half = np.array([0.5, 0.5, 0.5], dtype=np.float64)

assert np.allclose(D_star, [5 / 7, 3 / 8, 2 / 5])
assert abs(V(D_half) + 1.386294) < 1e-6
assert abs(V(D_star) + 1.284545) < 1e-6

print("V(0.5) =", f"{V(D_half):.6f}")
print("D*     =", np.round(D_star, 6))
print("V(D*)  =", f"{V(D_star):.6f}")
```

A kód lefutott (Python 3.12.3, NumPy 2.5.4, CPU, float64); mért kimenet:

```
V(0.5) = -1.386294
D*     = [0.714286 0.375    0.4     ]
V(D*)  = -1.284545
```

A kód nem igényel tanítást, csak a levezetett képletet ellenőrzi a diszkrét példán.

31.2.6 Labor: kétdimenziós keverék és collapse-diagnosztika

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

Kérdés. Négymódusú 2D-s adaton milyen móduslefedettséget mutat a generátor, és mikor jelez a diagnosztika mode collapse-gyanút?

Adat és környezet. A valós eloszlás négy izotróp Gauss-komponens keveréke, középpontjaik $(0,0), (0,3), (3,0), (3,3)$, kovarianciájuk $0{,}05I$, súlyaik egyenként $1/4$. A latent $p_z = \mathcal{N}(0, I_2)$.

Specifikáció. A generátor $2 \to 32 \to 32 \to 2$ MLP ReLU aktivációval, a diszkriminátor $2 \to 32 \to 32 \to 1$ MLP, a kimenet logit. Adam-optimalizáló, tanulási ráta $2 \cdot 10^{-4}$, batch $B = 128$, 20 epoch $\times 100$ lépés. A generátor a (4) nem telítődő célfüggvényt használja. Seed `20261020`, dtype `float32`, device `"cpu"`. A tanítás végén $2048 = 16 \cdot 128$ generált mintát és a hozzájuk tartozó 2048 latens vektort rögzítünk. Futtatási parancs: `python lab_toy_gan.py --seed 20261020 --epochs 20 --batch 128`. A specifikációt követő toy GAN lefutott (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, float32, seed 20261020, egy szál); a mért kimenet a diagnosztikai kód után látható.

A teljes tanítószkript:

```
# lab/n122/lab_toy_gan.py
"""Négymódusú 2D toy GAN a 31.2 laborhoz."""
import argparse
import pathlib

import numpy as np
import torch
from torch import nn

def sample_real(n, generator):
    centers = torch.tensor([[0.0, 0.0], [0.0, 3.0],
                            [3.0, 0.0], [3.0, 3.0]])
    idx = torch.randint(0, 4, (n,), generator=generator)
    noise = torch.randn(n, 2, generator=generator)
    return centers[idx] + (0.05 ** 0.5) * noise

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, default=20261020)
    parser.add_argument("--epochs", type=int, default=20)
    parser.add_argument("--batch", type=int, default=128)
    args = parser.parse_args()

    torch.manual_seed(args.seed)
    torch.set_num_threads(1)
    gen = torch.Generator().manual_seed(args.seed)

    g = nn.Sequential(
        nn.Linear(2, 32), nn.ReLU(),
        nn.Linear(32, 32), nn.ReLU(), nn.Linear(32, 2),
    )
    d = nn.Sequential(
        nn.Linear(2, 32), nn.ReLU(),
        nn.Linear(32, 32), nn.ReLU(), nn.Linear(32, 1),
    )
    opt_g = torch.optim.Adam(g.parameters(), lr=2e-4)
    opt_d = torch.optim.Adam(d.parameters(), lr=2e-4)
    bce = nn.BCEWithLogitsLoss()

    loss_d_hist = []
    loss_g_hist = []
    for _epoch in range(args.epochs):
        for _step in range(100):
            x_real = sample_real(args.batch, gen)
            z = torch.randn(args.batch, 2, generator=gen)
            x_fake = g(z).detach()
            opt_d.zero_grad()
            loss_d = bce(d(x_real), torch.ones(args.batch, 1)) \
                + bce(d(x_fake), torch.zeros(args.batch, 1))
            loss_d.backward()
            opt_d.step()

            z = torch.randn(args.batch, 2, generator=gen)
            opt_g.zero_grad()
            opt_d.zero_grad()
            loss_g = bce(d(g(z)), torch.ones(args.batch, 1))
            loss_g.backward()
            opt_g.step()
            loss_d_hist.append(loss_d.item())
            loss_g_hist.append(loss_g.item())

    with torch.no_grad():
```

```
        z_eval = torch.randn(2048, 2, generator=gen)
        fake = g(z_eval).numpy()

    out = pathlib.Path("output")
    out.mkdir(exist_ok=True)
    np.save(out / f"fake_seed{args.seed}_epoch{args.epochs}.npy", fake)
    np.save(out / f"latents_seed{args.seed}_epoch{args.epochs}.npy",
            z_eval.numpy())
    print("final D loss (last 100 mean):",
          sum(loss_d_hist[-100:]) / 100)
    print("final G loss (last 100 mean):",
          sum(loss_g_hist[-100:]) / 100)
    print("sample mean:", fake.mean(axis=0))
    print("sample std :", fake.std(axis=0))

if __name__ == "__main__":
    main()
```

A collapse-diagnosztika a mentett generált mintákra és latensekre:

```
# munkakönyvtár: lab/n122/
import numpy as np

samples = np.load("output/fake_seed20261020_epoch20.npy")      # (2048, 2)
latents = np.load("output/latents_seed20261020_epoch20.npy")   # (2048, 2)
centers = np.array([[0.,0.],[0.,3.],[3.,0.],[3.,3.]], dtype=np.float32)
d = np.linalg.norm(samples[:,None,:] - centers[None,:,:], axis=2)
mode = d.argmin(axis=1)
counts = np.bincount(mode, minlength=4) / len(samples)

dd = np.linalg.norm(samples[:,None,:] - samples[None,:,:], axis=2)
np.fill_diagonal(dd, np.inf)
near_duplicate_rate = float(np.mean(dd.min(axis=1) < 0.05))

zz = np.linalg.norm(latents[:,None,:] - latents[None,:,:], axis=2)
oo = np.linalg.norm(samples[:,None,:] - samples[None,:,:], axis=2)
far = zz > 1.0
latent_collapse_rate = float(np.mean(oo[far] < 0.05))

rng = np.random.default_rng(20261020)
idx = rng.integers(0, 4, size=2048)
real = centers[idx] + np.sqrt(0.05) * rng.normal(size=(2048, 2))

def median_nn(x):
    dm = np.linalg.norm(x[:,None,:] - x[None,:,:], axis=2)
    np.fill_diagonal(dm, np.inf)
    return float(np.median(dm.min(axis=1)))

nn_fake = median_nn(samples)
nn_real = median_nn(real)
nn_ratio = nn_fake / nn_real

assert samples.shape == (2048, 2)
assert latents.shape == (2048, 2)
assert np.isfinite(samples).all()
assert abs(counts.sum() - 1.0) < 1e-12
assert 0.0 <= near_duplicate_rate <= 1.0
assert 0.0 <= latent_collapse_rate <= 1.0
assert nn_ratio > 0.0

print("modus_ratios:", np.round(counts, 4))
print("near_duplicate_rate:", round(near_duplicate_rate, 4))
print("latent_collapse_rate:", round(latent_collapse_rate, 6))
```

```
print("median_nn_fake:", round(nn_fake, 6))
print("median_nn_real:", round(nn_real, 6))
print("nn_ratio:", round(nn_ratio, 4))
```

A mért futás kimenete (a tanítás végi veszteségek és a diagnosztika):

```
final D loss (last 100 mean): 1.1730377984046936
final G loss (last 100 mean): 0.7928798341751099
sample mean: [1.04433   1.2566924]
sample std : [1.3036016 1.3978592]
modus_ratios: [0.2451 0.3931 0.3579 0.0039]
near_duplicate_rate: 0.8799
latent_collapse_rate: 0.001003
median_nn_fake: 0.014325
median_nn_real: 0.017009
nn_ratio: 0.8422
```

Két egymás utáni futás a generált 2048 mintán és a latenseken bitazonos volt. A mért módusarányok szerint a $(3, 3)$ módusra a minták 0,39%-a esik, ami az 1%-os küszöb alatt van, ezért a diagnosztika hiányzó módust jelez. A közeli duplikátumok fix 0,05-ös küszöbön mért aránya 87,99%, de ez a mutató erősen függ a mintaszámtól és a módusok belső szórásától: 2048 mintánál a valós referencián mért medián legközelebbi-szomszéd távolság 0,017009, a generálton 0,014325, így a skálafüggetlen arány 0,8422; ez nem jelez globális összeomlást. A „sok latens közel azonos kimenetre képeződik” jellemzőt a latensek páronkénti távolságával mérjük: az 1,0-nél távolabbi latens párok között a kimenetek 0,1%-a esik 0,05-nél közelebb, ami szintén a globális összeomlás ellen szól. A döntési szabály ezért: hiányzó módus, ha valamely középpont aránya 1% alatt van; globális diverzitásvesztés gyanúja, ha a skálafüggetlen medián-NN arány 0,25 alá csökken. A 0,05-ös nyers duplikátumarányt csak leíró jellemzőként közöljük, nem küszöbként. A generált minta átlaga [1,0443; 1,2567], szórása [1,3036; 1,3979]; a négy módus súlya nem egyenletes, ami a 20 epochos toy tanítás korlátja. A küszöbök a toy feladathoz tartoznak, nem univerzális elméleti határok.

31.2.7 Feladatok

F1 (fogalmi, 1). Definíció szintjén adja meg a $V(D, G)$ célfüggvényt, valamint a diszkriminátor- és generátorparaméterek frissítési irányát. Miért nem állítható, hogy a GAN „egyetlen közös loss” minimalizálása? Átadandó: a V képlete, a két optimalizáló paraméterhalmaza, egy mondat a szerepek különbségéről.

F2 (levezetési, 2). Legyen $\Omega = \{0, 1\}$, $p_{\text{data}} = (0{,}6;\ 0{,}4)$, $p_g = (0{,}9;\ 0{,}1)$. Vezesse le a D^* értékeit, számítsa ki $V(D^*)$-ot és $V(0{,}5)$-öt, és ellenőrizze a konkavitást az $y = D_0^*$ pontbeli második deriválttal. Átadandó: a teljes számolás négy tizedes pontossággal.

F3 (implementációs, 2). Írjon csak NumPy-t használó, PyTorch nélküli függvényt, amely tetszőleges azonos hosszúságú p, q valószínűségvektorra kiszámítja D^*-ot, $V(D^*)$-ot, és az első koordinátán egy finom rácson ellenőrzi, hogy D^* első eleme maximalizálja az első tagot. Átadandó: futtatható `verify_optimal_d(p,q)` függvény, inputpélda, a rács lépésközéhez igazított tolerancia (legfeljebb 10^{-5}), futtatási parancs.

F4 (hibakeresési, 2). Az alábbi kódrészletben hibásan van-e valamelyik optimalizáló paraméterhalmaza?

```
optD = torch.optim.Adam(D.parameters(), lr=2e-4)
optG = torch.optim.Adam(list(G.parameters()) + list(D.parameters()), lr=2e-4)
```

Lokalizálja a hibát, mondja meg a következményét, javítsa, és adjon egy `torch.allclose`-alapú utóellenőrzési lépést.

F5 (kísérleti transzfer, 3). Tervezzen kísérletet a telítődő (3) és nem telítődő (4) generátorcél összehasonlítására a fenti négymódusú 2D-s toy adaton öt seeddel. Rögzítse a megfigyelési egységet, a baseline-t, az összehasonlítási metrikákat, az erőforrásokat és a következtetési határt. Átadandó: protokolltáblázat és a lehetséges negatív eredmény értelmezése.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

31.2.8 Megoldások és indoklás

F1. A cél

$$\min_{G_\theta}\max_{D_\phi} V(D_\phi, G_\theta), \quad V = \mathbb{E}_{p_{\text{data}}}[\ln D] + \mathbb{E}_{p_g}[\ln(1-D)].$$

A diszkriminátor ϕ paramétereit V maximalizálásával, a generátor θ paramétereit V minimalizálásával frissítjük; a gyakorlatban a generátorra gyakran a (4) nem telítődő cél kerül. Két külön paraméterkészletről és ellentétes célról van szó, ezért nem egy közös loss minimalizálása.

F2. $D_0^* = 0{,}6/(0{,}6+0{,}9) = 0{,}4$, $D_1^* = 0{,}4/(0{,}4+0{,}1) = 0{,}8$.

$$V(D^*) = 0{,}6\ln 0{,}4 + 0{,}4\ln 0{,}8 + 0{,}9\ln 0{,}6 + 0{,}1\ln 0{,}2 = -1{,}2597.$$

$$V(0{,}5) = 2\ln 0{,}5 = -1{,}3863.$$

Az $y = D_0^*$ pontban a cél $f(y) = 0{,}6\ln y + 0{,}9\ln(1-y)$, második deriváltja

$$f''(0{,}4) = -\frac{0{,}6}{0{,}4^2} - \frac{0{,}9}{0{,}6^2} = -3{,}75 - 2{,}5 = -6{,}25 < 0,$$

tehát a pont maximum. Mindkét koordináta a $(0,1)$ belsejében van, ezért a stacionárius pont belső pont, a logaritmusok definiáltak, és a maximum létezik; a négy tizedesre kerekített értékek ezért összevethetők. A (6) képletből $2D_{JS} = V(D^*) + \ln 4 \approx 0{,}1266$, tehát $D_{JS} \approx 0{,}0633$.

F3. Példa-megoldás:

```
# munkakönyvtár: lab/n122/
# Futtatás: python verify_optimal_d.py
import numpy as np

def verify_optimal_d(p, q, tol=1e-6):
    p = np.asarray(p, dtype=np.float64)
    q = np.asarray(q, dtype=np.float64)
    assert p.shape == q.shape
    assert np.all(p >= 0) and np.all(q >= 0)
    assert abs(p.sum()-1) < tol and abs(q.sum()-1) < tol
    D_star = p / (p + q)
    y = np.linspace(1e-7, 1-1e-7, 200001)
    h = y[1] - y[0]          # rács lépésköz, 5e-6
    v1 = p[0]*np.log(y) + q[0]*np.log(1-y)
    j = int(np.argmax(v1))
```

```
    assert abs(y[j] - D_star[0]) <= h, (y[j], D_star[0], h)
    return D_star

if __name__ == "__main__":
    p = np.array([0.6, 0.4], dtype=np.float64)
    q = np.array([0.9, 0.1], dtype=np.float64)
    ds = verify_optimal_d(p, q)
    print("D*:", ds)
    print("OK, maximum a rácson:", ds[0])
```

A rács csak az első koordinátát változtatja; a többi koordináta rögzített. A `tol` a vektorok normalizáltságát ellenőrzi. Az argmax pontosságát a rács lépésköze $h \approx 5 \cdot 10^{-6}$ korlátozza: konkáv f esetén a rácson kapott maximum egy lépésköznél közelebb van az igazihoz, ezért az eltérést h-hoz, nem a 10^{-6}-os toleranciához hasonlítjuk. A tűrés növelése helyett sűríthető a rács.

F4. A hiba az `optG` konstruktorában van: `list(G.parameters()) + list(D.parameters())` a generátor-optimalizálóhoz köti a diszkriminátor súlyait is. Következmény: egy generátorlépés a ϕ súlyokat is módosítja, tehát a két frissítés nem szétválasztott; a játék nem a tervezett $\min_G \max_D$ sémát követi, oszcilláció vagy összeomlás léphet fel. Javítás:

```
optD = torch.optim.Adam(D.parameters(), lr=2e-4)
optG = torch.optim.Adam(G.parameters(), lr=2e-4)

# D lépés: a generált minta leválasztva, csak D tanul
optD.zero_grad()
x_fake = G(z).detach()
loss_D = bce(D(x_real), torch.ones_like(D(x_real))) \
       + bce(D(x_fake), torch.zeros_like(D(x_fake)))
loss_D.backward()
optD.step()

# G lépés: a gradiens D-n keresztül G-be folyik, de D-t nem frissítjük
optG.zero_grad()
optD.zero_grad()   # a G-lépés által D-re írt gradiensek törlése
loss_G = bce(D(G(z)), torch.ones_like(D(G(z))))
loss_G.backward()
optG.step()
```

A `loss_G.backward()` a láncszabály miatt a D paramétereire is számol gradienst, ezért a `optD.zero_grad()` a G-lépés után (vagy a következő D-lépés előtt) kötelező; a D-lépésben a generált minta `detach()`-je biztosítja, hogy a D-cél ne frissítse a generátort.

Utóellenőrzés: a G-lépés előtt mentsük el `D_ref = [w.detach().clone() for w in D.parameters()]`, majd `loss_G.backward(); optG.step()` után ellenőrizzük, hogy minden `torch.allclose(w.detach(), w_ref)` igaz. Ha bármelyik eltér, a ϕ súly még mindig az `optG` hatókörében van.

F5. Egy lehetséges protokoll:

Tényező	Rögzített érték
Megfigyelési egység	Egy teljes, rögzített lépésszámú training futás
Adat	Négymódusú 2D Gauss-keverék, seed 20261020
D, G architektúra, Adam, batch	Azonos a kettő karban
Változó tényező	L_G^{sat} (3) vagy L_G^{ns} (4)
Seedek	Karokként 5–5 tanítási seed; a `20261020` csak az adatgenerálás rögzített seedje, nem a tanításé

Tényező	Rögzített érték
Erőforrások	CPU, egy szál (`torch.set_num_threads(1)`), PyTorch 2.15.0+cpu; karokként 5 futás, egyenként 20 epoch × 100 lépés, $B = 128$
Metrika	Módusarányok, near-duplicate arány, $D(G(z))$ átlag

A baseline a telítődő cél, mert az eredeti V közvetlenül ezt tartalmazza. Előre rögzített lépésszám után értékelünk, nem válogatunk a seedek közül a metrika szerint. A következtetési határ: a négymódusú toy adat csak a mechanizmust teszteli; sem nagy képi adatra, sem termelési generatív modellre nem általánosítható. Negatív eredmény, például mindkét kar egyetlen módusra esik össze, korrekt eredmény, ha a nyers módusarányokat és a seedeket közöljük.

31.2.9 Tipikus tévedés és ellenpélda

Tévedés: „ha bebizonyítottuk a D^* képletét és a $p_g = p_{\text{data}}$ globális optimumot, akkor a gyakorlati GAN-training konvergálni fog.” A levezetés ideális, eloszlástéri eredmény. Ha a generátorcsalád nem képes p_{data}-t előállítani, az optimum nem is eleme a paramétertérnek.

Ellenpélda: legyen p_{data} egyenletes a $\{0, 1\}$ halmazon, és legyen a generátor paraméter nélküli konstans leképezés, $G(z) = 0$ minden z-re. Ekkor $p_g = (1, 0)$. A képlet szerint $D^*(0) = 0{,}5/(0{,}5 + 1) = 1/3$, a lezárt $[0, 1]$-értelmezésben $D^*(1) = 1$; a JS-távolság pozitív. A $p_g = p_{\text{data}}$ állapot ebből a generátorcsaládból elérhetetlen, bármilyen diszkriminátorfrissítést is végzünk. A nemkonvex, alternáló sztochasztikus gradiensfrissítés konvergenciája külön empirikus kérdés; az eloszlástéri optimum nem garantálja.

31.2.10 Önellenőrzés és korábbi fogalom visszahívása

Ellenőrizendő: szerepel-e $\min_G \max_D$, a D valós címkéje 1 és a generálté 0, a G nem telítődő céljában a generált minta targetje 1; a D-frissítésben detach; a labor nincs futtatottként feltüntetve. A 31.1. VAE és variációs alsó korlát VAE-jében explicit posterior és likelihood volt; itt egyik sincs, ezért a két modellcsalád optimalizálási garanciái nem felcserélhetők. A 30.2. Kontrasztív cél és temperature kontrasztív célja softmax-keresztentrópia, amely két osztály esetén logisztikussá egyszerűsödik, de ott a pozitív pár definíciója pretext-invarianciából jött; itt a „pozitív” egyszerűen a valós adat címkéje.

31.2.11 Rövid összefoglaló

A GAN a generátor és diszkriminátor minimax játékát optimalizálja: a diszkriminátor a valós és generált minták logisztikus osztályozója, a generátor a diszkriminátor megtévesztésére törekszik. Rögzített generátor mellett az ideális diszkriminátor $p_{\text{data}}/(p_{\text{data}}+p_g)$, és a cél értéke $-\ln 4+2D_{\text{JS}}$. A telítődő generátorcél gradiense $D \approx 0$-nál eltűnik, ezért használatos a $-\ln D$ nem telítődő alak. A mode collapse a generált eloszlás diverzitásának összeomlása, amelyet külön diagnosztikával kell mérni. Az eloszlástéri optimum nem garantálja véges hálók tanítási konvergenciáját.

Az induló kérdésre a válasz: rögzített generátor mellett az optimális diszkriminátor zárt alakja $D^*(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_g(x))$, és ezzel a minimax cél értéke a Jensen–Shannon-divergenciához kötődik.

31.2.12 Következő alkalmazás

A 31.3. Diffúziós zajmodell és CNN-denoiser egység a denoising diffusion probabilistic modellek alapgondolatát tárgyalja; ott a generátor egy zajos képből tanul denoise-lépést, ami a GAN diszkriminátorától eltérő tanítási jelet ad. A most bevezetett „generátor mint eloszlásleképezés” fogalma ott is szükséges.

31.2.13 Felhasznált források

- [P19] Goodfellow, I. et al. (2014). *Generative Adversarial Networks.* arXiv:1406.2661. https://arxiv.org/abs/1406.2661. A felhasznált célfüggvény a cikk absztraktjában is szereplő kétjátékos keret; a jelen egység D^*-levezetése a könyv saját levezetése.

31.3 Diffúziós zajmodell és CNN-denoiser

31.3.1 Cél és előismeret

Ez az alfejezet a diffúziós generatív modellek matematikai magját, a Gauss-zajjal definiált forward folyamatot és a tanult visszalépést tárgyalja. A cél annak pontos megértése, hogy egy kép vagy vektor fokozatos zajosítása miként ad zárt alakú feltételes eloszlást, és hogyan tanítható ebből egy feltételes denoiser (zajeltávolító). Előismeret: a 31.2. GAN mint minimax probléma alfejezetben tárgyalt GAN-fogalmak, valamint a 27.2. U-Net, encoder–decoder és skip alfejezetben bemutatott U-Net-architektúra. A levezetéshez csak a független Gauss-változók összegének variancia-additivitását használjuk, amelyet az 5.2. Várható érték, variancia és likelihood egységben építettünk fel.

Induló kérdés: ha egy x_0 adatpontot T lépésben egyre erősebb Gauss-zajjal rontunk, meg tudjuk-e becsülni a fordított irányú, zajt eltávolító lépés feltételes eloszlását anélkül, hogy a teljes Markov-láncot szimulálnánk?

31.3.2 Forward zajfolyamat: definíció és intuíció

Legyen az eredeti megfigyelés $x_0 \in \mathbb{R}^d$. Kép esetén $x_0 \in \mathbb{R}^{C\times H\times W}$, ahol C a csatornák, H a magasság, W a szélesség száma; az absztrakt képletekben $d = C\cdot H\cdot W$ dimenziós oszlopvektornak tekintjük. Batchelt adat esetén $X_t \in \mathbb{R}^{N\times C\times H\times W}$, ahol N a batch mérete, és minden művelet elemenként értendő.

A **noise schedule** (zajütemezés) egy rögzített $\beta_1, \dots, \beta_T \in (0,1)$ sorozat, ahol T a teljes lépésszám. A forward (előre irányuló) folyamat egy Markov-lánc, amelynek átmeneti eloszlása minden $t = 1, \dots, T$ esetén

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\right).$$

Ez azt jelenti, hogy az x_t vektor feltételesen Gauss-eloszlású, átlaga $\sqrt{1-\beta_t}\, x_{t-1}$, kovariancia-mátrixa pedig $\beta_t I$. A gyakorlati mintavételezés:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I), \quad \epsilon_t \perp x_{t-1}.$$

A β_t értékek kicsik, tipikusan 10^{-4} és 0,02 között növekednek; így az első lépések alig rontják a jelet, kellően hosszú T (például 1000 lépés) mellett pedig a végső lépések csaknem teljesen zajossá teszik. A jelen labor $T = 10$ lépése mellett viszont $\bar{\alpha}_{10} \approx 0{,}90$, tehát ott a lánc vége még nem zajos; a labor ezért csak a forward zárt alakot ellenőrzi. A **tanult visszalépés** (reverse process) ezzel szemben egy $p_\theta(x_{t-1} \mid x_t)$ paraméteres eloszláscsalád, amelynek feladata a forward lépés közelítő megfordítása; ezt a későbbi szakasz tárgyalja. A két folyamat tehát külön objektum: a forward rögzített és analitikus, a reverse tanult és sztochasztikus.

31.3.3 Lemma: zárt alakú feltételes eloszlás

A következő állítás a fejezet levezetési kötelezettsége. Célja, hogy egy tetszőleges t időpontbeli x_t eloszlását közvetlenül x_0-ból számíthassuk, anélkül hogy a közbülső lépéseket egyenként szimulálnánk.

Lemma (a Gauss-forward folyamat zárt alakja). Legyen $\alpha_t = 1 - \beta_t$ minden $t = 1, \dots, T$ esetén, és definiáljuk a kumulatív szorzatot

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s, \qquad \bar{\alpha}_0 = 1.$$

Ha a forward folyamat a fenti Markov-átmenetet követi független $\epsilon_s \sim \mathcal{N}(0, I)$ zajokkal, akkor minden $t = 1, \dots, T$ esetén

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t) I\big).$$

Bizonyítás. A Gauss-eloszlások lineáris transzformációjára, feltételes eloszlására és független összegére vonatkozó azonosságokat a [B01] tankönyv szerinti alakban használjuk. Teljes indukcióval dolgozunk. A $t = 1$ esetben $\bar{\alpha}_1 = \alpha_1 = 1 - \beta_1$, így a definícióból közvetlenül adódik az állítás.

Tegyük fel, hogy az állítás $t-1$-re igaz, azaz

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\, \eta, \qquad \eta \sim \mathcal{N}(0, I),$$

ahol η független a t-edik lépés ϵ_t zajától. A forward rekurzió szerint

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t.$$

Behelyettesítve az indukciós feltevést:

$$x_t = \sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_{t-1}}\, \eta + \sqrt{\beta_t}\, \epsilon_t.$$

Az első tag determinisztikus, mert x_0 adott. A második és harmadik tag független, nulla várható értékű Gauss-változó. Két független Gauss-változó összege is Gauss, és a kovarianciák összeadódnak; az 5.2. Várható érték, variancia és likelihood alfejezetben látott variancia-additivitás itt kovarianciamátrixokra alkalmazva:

$$\mathrm{Cov}\big(\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_{t-1}}\, \eta + \sqrt{\beta_t}\, \epsilon_t\big) = \alpha_t(1 - \bar{\alpha}_{t-1}) I + \beta_t I.$$

Mivel $\beta_t = 1 - \alpha_t$, a zárójelben

$$\alpha_t(1-\bar\alpha_{t-1}) + 1 - \alpha_t = 1 - \alpha_t\bar\alpha_{t-1} = 1 - \bar\alpha_t.$$

A determinisztikus tag együtthatója pedig $\sqrt{\alpha_t}\sqrt{\bar\alpha_{t-1}} = \sqrt{\bar\alpha_t}$. Következésképpen

$$x_t = \sqrt{\bar\alpha_t}\,x_0 + \sqrt{1-\bar\alpha_t}\,\tilde\epsilon, \qquad \tilde\epsilon \sim \mathcal{N}(0, I),$$

ami éppen a lemma állítása. ■

A bizonyítás kulcsa a feltételes eloszlás rekurzív szerkezete: minden lépésben egy $\sqrt{\alpha_t}$ szorzó skálázza a korábbi jelet, és egy független zajtag adódik hozzá. A kumulatív szorzat $\bar\alpha_t$ azt méri, hogy az eredeti x_0 jel mekkora hányada maradt meg. Ha a schedule-t úgy választjuk, hogy $\bar\alpha_T \approx 0$, akkor $q(x_T \mid x_0) \approx \mathcal{N}(0, I)$, tehát a lánc végén a feltételes eloszlás már nem függ érdemben x_0-tól. A labor $T = 10$ beállítása nem elégíti ki ezt: ott $\bar\alpha_{10} \approx 0{,}90$.

31.3.4 Score-intuíció

A lemma lehetővé teszi a **score** (pontszámfüggvény) közvetlen kiszámítását. A feltételes sűrűség logaritmusa, elhagyva az x_t-től nem függő konstanst:

$$\log q(x_t \mid x_0) = -\frac{\|x_t - \sqrt{\bar\alpha_t}\,x_0\|^2}{2(1-\bar\alpha_t)} - \frac{d}{2}\log(2\pi(1-\bar\alpha_t)).$$

Az x_t szerinti gradiens, a láncszabályt komponensenként alkalmazva:

$$\nabla_{x_t}\log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar\alpha_t}\,x_0}{1-\bar\alpha_t}.$$

Ha bevezetjük a standardizált zajt

$$\epsilon = \frac{x_t - \sqrt{\bar\alpha_t}\,x_0}{\sqrt{1-\bar\alpha_t}},$$

akkor

$$\nabla_{x_t}\log q(x_t \mid x_0) = -\frac{\epsilon}{\sqrt{1-\bar\alpha_t}}.$$

Tehát a **feltételes** score egy negatív előjelű, $1/\sqrt{1-\bar\alpha_t}$ tényezővel skálázott zaj. Ez a mennyiség azonban ismeri x_0-t, generáláskor viszont a hálózat csak (x_t, t)-t látja, és a mintavételhez a **marginális** $q_t(x_t) = \int q(x_t \mid x_0)\,p_{\text{data}}(x_0)\,dx_0$ sűrűség score-ja kell. A kettő kapcsolata egy további feltételes várhatóérték-azonosság.

Lemma (marginális score és optimális zajprediktor). Ha az integrálás és a deriválás felcserélhető, akkor

$$\nabla_{x_t}\log q_t(x_t) = \mathbb{E}\big[\nabla_{x_t}\log q(x_t \mid x_0)\,\big|\,x_t\big] = -\frac{\mathbb{E}[\epsilon \mid x_t, t]}{\sqrt{1-\bar\alpha_t}}.$$

Bizonyítás. $\nabla q_t(x_t) = \int \nabla_{x_t} q(x_t \mid x_0)\, p_{\text{data}}(x_0)\, dx_0 = \int q(x_t \mid x_0)\, \nabla_{x_t} \log q(x_t \mid x_0)\, p_{\text{data}}(x_0)\, dx_0$. Elosztva $q_t(x_t)$-vel a súly $q(x_t \mid x_0) p_{\text{data}}(x_0)/q_t(x_t) = q(x_0 \mid x_t)$, a Bayes-tétel szerinti utóeloszlás, így a jobb oldal a feltételes score x_t-re vett feltételes várható értéke. A feltételes score fenti alakjába helyettesítve kapjuk a második egyenlőséget. □

A lemma szétválasztja a **tanítási zajcélt** és az **elérhető optimális regressziós függvényt**. A tanítás egyes mintáin a cél a konkrét ϵ realizáció, de mivel a hálózat csak (x_t, t)-t látja, az MSE-t minimalizáló függvény az $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t, t]$ feltételes várható érték, nem minden egyes realizáció pontos zaja (a négyzetes veszteség minimalizálója a feltételes várható érték). A lemma szerint ez éppen a marginális score skálázott változata: $\nabla \log q_t(x_t) = -\epsilon^*(x_t, t)/\sqrt{1-\bar{\alpha}_t}$. Ebben az értelemben ekvivalens a zajpredikció és a score-becslés: a kettő optimuma egy ismert skálázással egymásba vihető. A score-alapú generatív modellek pontosan erre a felismerésre építenek: a sűrűség gradiensének becslésére külön célfüggvényt használnak (Song és Ermon, 2019), és a folytonos idejű általánosítást adják (Song és mtsai, 2021). Egy kétpontos, egydimenziós ellenőrzésben ($x_0 \in \{-1, 1\}$, $\bar{\alpha}_t = 0{,}6$) a zárt alakú marginális score és a $-\mathbb{E}[\epsilon \mid x_t]/\sqrt{1-\bar{\alpha}_t}$ eltérését numerikusan is kiszámítjuk:

```
import numpy as np

ab, v = 0.6, 0.4
s = np.sqrt(ab)

def pdf(x, m):
    return np.exp(-(x - m) ** 2 / (2 * v)) / np.sqrt(2 * np.pi * v)

def err(xt):
    p0, p1 = 0.5 * pdf(xt, -s), 0.5 * pdf(xt, s)
    w0, w1 = p0 / (p0 + p1), p1 / (p0 + p1)
    eps0, eps1 = (xt + s) / np.sqrt(v), (xt - s) / np.sqrt(v)
    score_mc = -(w0 * eps0 + w1 * eps1) / np.sqrt(v)
    d0, d1 = -(xt + s) / v, -(xt - s) / v
    return abs(score_mc - (p0 * d0 + p1 * d1) / (p0 + p1))

print(f"max abs elteres: {max(err(x) for x in (-2.0, -0.5, 0.0, 0.5, 2.0)):.3e}")
```

Ez a program lefutott (Python 3.12.3, NumPy 2.5.4, float64); mért kimenete `max abs elteres: 4.441e-16`, ami a float64 gépi pontosság nagyságrendje.

31.3.5 Conditional denoiser és tanítási cél

A **conditional denoiser** (feltételes zajeltávolító) egy $\epsilon_\theta : \mathbb{R}^d \times \{1, \dots, T\} \to \mathbb{R}^d$ paraméteres függvény, amelynek bemenete egy zajos x_t vektor és a t időindex, kimenete pedig a hozzáadott zaj becslése. Képeknél a be- és kimenet alakja megegyezik: $\epsilon_\theta(X_t, t) \in \mathbb{R}^{N \times C \times H \times W}$. A t feltételt a gyakorlatban beágyazzák, és a hálózat közbülső rétegeihez adják.

A tanítási cél a **zajpredikciós MSE**. Legyen p_{data} az adateloszlás, $t \sim \text{Uniform}\{1, \dots, T\}$, $\epsilon \sim \mathcal{N}(0, I)$, és $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$. Ekkor

$$L(\theta) = \mathbb{E}_{x_0 \sim p_{\text{data}},\, t,\, \epsilon}\left[\left\|\epsilon - \epsilon_\theta(x_t, t)\right\|^2\right].$$

Mivel a várható érték a t időindexre is átlagol, a veszteséget nem indexeljük t-vel; rögzített t mellett a feltételes alak $L(\theta \mid t) = \mathbb{E}_{x_0, \epsilon}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$ lenne.

A veszteség minimalizálása azt jelenti, hogy a hálózat minden t-re megtanulja a hozzáadott zajt rekonstruálni. Ez regressziós feladat, nem minmax játék: ellentétben a 31.2. GAN mint minimax

probléma alfejezetben tárgyalt GAN-nal, itt nincs külön diszkriminátor, és a forward folyamat rögzített. A [P20] modellcsalád ezt a zajpredikciós paraméterezést használja; a jelen levezetés a forward folyamat standard tulajdonságaiból adódik.

A tanítási cél fontos tulajdonsága, hogy a $q(x_t \mid x_0)$ zárt alakja miatt minden tanítópélda egyetlen lépésben előállítható: nem kell a teljes t hosszúságú zajozási láncot végigjárni.

31.3.6 Sampling: a tanult visszalépés

A **sampling** (mintavételezés) során a cél új, az adateloszláshoz hasonló x_0 előállítása. Kiindulunk egy tiszta zajból: $x_T \sim \mathcal{N}(0, I)$. Ezután minden $t = T, T-1, \dots, 1$ lépésben alkalmazzuk a tanult reverse átmenetet. A DDPM-féle paraméterezésben a reverse eloszlás

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I),$$

ahol

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \, \epsilon_\theta(x_t, t) \Big).$$

A σ_t^2 variancia tipikusan β_t vagy a posterior variancia

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \, \beta_t.$$

A mintavételezési algoritmus:

1. $x_T \sim \mathcal{N}(0, I)$;
2. $t = T, \dots, 1$ esetén:
   - $\hat{\epsilon} \leftarrow \epsilon_\theta(x_t, t)$;
   - $\mu \leftarrow \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\epsilon})$;
   - ha $t > 1$, akkor $z \sim \mathcal{N}(0, I)$, különben $z = 0$;
   - $x_{t-1} \leftarrow \mu + \sigma_t z$.

A $t = 1$-nél használt $z = 0$ választás heurisztika: az utolsó lépésben a reverse átmenet varianciáját nullára állítjuk, ezért a minta a determinisztikus posterior átlagból származik; ez a DDPM-gyakorlat választása, nem a modell tanult része. A teljes eljárás iteratív: nem egyetlen nagy ugrással távolítjuk el a zajt, hanem T kis léptékű korrekcióval. Ez az iteratív denoising a diffúziós modellek jellegzetessége.

31.3.7 U-Net szerepe

A ϵ_θ hálózat architektúrája a CNN-alapú DDPM-példákban (így a jelen egységben is) **U-Net** típusú. Ez nem a teljes képi diffúziós modellcsaládra igaz: a diffúziós transzformerek (DiT) önálló, széles körben használt architektúracsaládot alkotnak, amelyek tárgyalása kívül esik e könyv CNN-keretén. Ennek oka, hogy a bemenet és a kimenet térbeli felbontása megegyezik: a zajtérkép ugyanolyan alakú, mint a zajos kép. A 27.2. U-Net, encoder–decoder és skip alfejezetben megismert U-Net encoder–decoder szerkezete skip-kapcsolatokkal lehetővé teszi, hogy a hálózat egyszerre lássa a nagy léptékű kontextust és a finom térbeli részleteket. A diffúziós U-Net további eleme az időbélyeg-kondicionálás: a t indexből egy kis beágyazó réteg vektort képez, amelyet

az egyes blokkok csatornánkénti skálázásra vagy eltolásra használnak. Így ugyanaz a hálózat minden időlépéshez alkalmazkodik anélkül, hogy T külön modellt kellene tárolni.

A konvolúciós rétegek számítási költsége $O(H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot k^2)$: a térbeli felbontás, a bemeneti és kimeneti csatornaszám szorzatával, valamint a kernelméret négyzetével arányos. A diffúziós modellek teljes költsége azonban ennél is nagyobb, mert a mintavételezés T hálózati kiértékelést igényel. Ez a fő oka annak, hogy a gyakorlati alkalmazásokban gyorsított mintavételezőket keresnek; ezek tárgyalása túlmutat jelen egységen.

31.3.8 Labor: zajozás demonstrációja szintetikus vektoron

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja a forward zajozás és a zárt alakú eloszlás numerikus ellenőrzése, teljes nagymodell tanítása nélkül. A kód PyTorch-ot használ, CPU-n, float32 pontossággal. Munkakönyvtár: `lab/n123_diffusion_demo/`.

```
# n123_demo.py - forward zajozás és zárt alak ellenőrzése
import torch

torch.manual_seed(1234)
torch.set_num_threads(1)
dtype = torch.float32
device = "cpu"

d = 8                       # vektordimenzió
T = 10                      # lépésszám
x0 = torch.linspace(-1.0, 1.0, d, dtype=dtype, device=device)  # (d,)

# Lineáris noise schedule
beta = torch.linspace(1e-4, 0.02, T, dtype=dtype, device=device)  # (T,)
alpha = 1.0 - beta
alpha_bar = torch.cumprod(alpha, dim=0)  # (T,)

# Egyetlen forward minta rögzített zajjal
g = torch.Generator(device=device).manual_seed(7)
eps = torch.randn(d, dtype=dtype, device=device, generator=g)  # (d,)
t = 5
xt = torch.sqrt(alpha_bar[t-1]) * x0 + torch.sqrt(1.0 - alpha_bar[t-1]) * eps
print("x0   =", x0.numpy())
print("x5   =", xt.numpy())

# Monte-Carlo ellenőrzés: M független zajrealizáció
M = 20000
noise = torch.randn(M, d, dtype=dtype, device=device)
samples = (torch.sqrt(alpha_bar[t-1]) * x0
           + torch.sqrt(1.0 - alpha_bar[t-1]) * noise)

emp_mean = samples.mean(dim=0)
emp_var = samples.var(dim=0, unbiased=False)
theo_mean = torch.sqrt(alpha_bar[t-1]) * x0
theo_var = torch.full_like(x0, 1.0 - alpha_bar[t-1])
print("max |emp. mean - theo. mean| =", (emp_mean - theo_mean).abs().max().item())
print("max |emp. var  - theo. var | =", (emp_var - theo_var).abs().max().item())

# Oracle denoiser: a demonstrációban ismeri a valódi eps-t
def oracle_denoiser(x, t_idx):
    ab = alpha_bar[t_idx-1]
    return (x - torch.sqrt(ab) * x0) / torch.sqrt(1.0 - ab)
```

```
# Egy reverse lépés a DDPM-átlaggal, t=5 -> t=4
a_t = alpha[t-1]
ab_t = alpha_bar[t-1]
beta_t = beta[t-1]
eps_hat = oracle_denoiser(xt, t)
mu = (xt - (beta_t / torch.sqrt(1.0 - ab_t)) * eps_hat) / torch.sqrt(a_t)
z = torch.randn_like(xt)
x_prev = mu + torch.sqrt(beta_t) * z
print("reverse mu (t=5)  =", mu.numpy())
print("reverse sample x4 =", x_prev.numpy())

assert x0.shape == xt.shape == eps.shape == (d,)
assert torch.isfinite(xt).all() and torch.isfinite(x_prev).all()
assert (emp_mean - theo_mean).abs().max().item() < 0.01
assert (emp_var - theo_var).abs().max().item() < 0.01
```

Futtatási parancs: `python n123_demo.py`. Az ellenőrzés a Monte-Carlo átlag és variancia összehasonlítása a lemma által adott elméleti értékekkel. A kód lefutott (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, float32, seed 1234, a zajgenerátor seedje 7, egy szál, $M = 20000$); mért kimenet:

```
x0   = [-1.         -0.71428573 -0.4285714  -0.1428571   0.1428571   0.4285714
  0.71428573  1.        ]
x5   = [-1.0107063 -0.58848697 -0.2819299  -0.30814335  0.3944789   0.28971714
  0.65296626  1.1732975 ]
max |emp. mean - theo. mean| = 0.0021439194679260254
max |emp. var  - theo. var | = 0.0005910545587539673
reverse mu (t=5)  = [-1.0064509  -0.63829666 -0.33999842 -0.24268207  0.29482576  0.34470165
  0.6772399   1.1046504 ]
reverse sample x4 = [-1.0473671  -0.62266415 -0.36085558 -0.28596672  0.23199481  0.29341096
  0.49592885  1.1183165 ]
```

A Monte-Carlo átlag maximális abszolút hibája $2{,}1 \cdot 10^{-3}$, a varianciáé $5{,}9 \cdot 10^{-4}$, mindkettő 10^{-2} alatt. A reverse minta a μ átlag körül sztochasztikusan ingadozik; a z zajrealizáció rögzített seeddel jött létre, ezért a fenti számok reprodukálhatók. A labor nem tanít hálózatot, és nem állít semmit a generált minták minőségéről.

31.3.9 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a forward zajfolyamat és a tanult visszalépés között. Adja meg mindkettő definícióját, a paraméterek szerepét és azt, hogy melyik rögzített, illetve tanult.

F2 (levezetési, nehézség 2). Legyen $T = 3$, $\beta_1 = 0{,}1$, $\beta_2 = 0{,}2$, $\beta_3 = 0{,}3$, és $x_0 = (1, -1)^\top \in \mathbb{R}^2$. Számítsa ki $\bar{\alpha}_3$ értékét, a $q(x_3 \mid x_0)$ feltételes eloszlás átlagát és varianciáját, valamint a feltételes score-t az $x_3 = (0{,}2;\ -0{,}3)^\top$ pontban.

F3 (implementációs, nehézség 1). Írjon Python-függvényt, amely egy (N, d) alakú X_0 batchhez és egy t indexhez a lemma zárt alakjából mintát generál. A függvény adja vissza a zajos X_t batch-et. Ellenőrizze egy $N = 3$, $d = 4$ eseten, hogy a kapott tensor alakja $(3, 4)$, és hogy a Monte-Carlo-átlag közelíti az elméleti átlagot.

F4 (hibakeresési, nehézség 2). Tekintse a következő hibás mintavételezési lépést:

```
# hibás reverse lépés
x_prev = x_t - eps_hat
```

ahol `eps_hat` a denoiser kimenete. Mi a hiba? Milyen eloszlásbeli következménye van annak, ha ezt a lépést használjuk $t = 5$-nél a labor kódjában? Javítsa a kódot a helyes DDPM-formulával, és indokolja a javítás minden tényezőjét.

F5 (kísérlettervezési, nehézség 3). Tervezzen egy kis kísérletet, amely szintetikus, 16 dimenziós vektorokon betanít egy két rejtett rétegű MLP-denoiser-t a zajpredikciós MSE-célra. Adja meg a protokollt: adateloszlás, split, batch méret, tanítási lépésszám, seed, metrika. Mi az a legfontosabb korlát, amely miatt ebből a toy (játékszerű, kicsinyített) kísérletből nem szabad általános következtetést levonni a képi diffúziós modellekre?

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

31.3.10 Megoldások és indoklás

F1 megoldás. A forward zajfolyamat $q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$ rögzített Markov-lánc, amelyet a $\beta_t \in (0,1)$ schedule egyértelműen meghatároz. Nincs tanulható paramétere, és a mintavételezése a fenti rekurzív zajhozzáadás. A tanult visszalépés ezzel szemben $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$, ahol θ tanulható paraméter. A kettő iránya ellentétes: a forward x_0-ból x_T-be tart, a reverse x_T-ből x_0-ba. A forward analitikus és nem tanul semmit; a reverse ezzel szemben tanult sztochasztikus átmenet, amely a tanítási cél minimalizálásával illeszkedik a forward inverzének közelítésére.

F2 megoldás. Először kiszámítjuk az α-kat: $\alpha_1 = 0{,}9$, $\alpha_2 = 0{,}8$, $\alpha_3 = 0{,}7$. Így $\bar{\alpha}_3 = 0{,}9 \cdot 0{,}8 \cdot 0{,}7 = 0{,}504$. A lemma szerint

$$q(x_3 \mid x_0) = \mathcal{N}(x_3; \sqrt{0{,}504}\, x_0, (1 - 0{,}504)I) = \mathcal{N}(x_3; (0{,}7099, -0{,}7099)^\top, 0{,}496 I).$$

A feltételes score az $x_3 = (0{,}2;\ -0{,}3)^\top$ pontban:

$$\nabla_{x_3} \log q(x_3 \mid x_0) = -\frac{x_3 - \sqrt{0{,}504}\, x_0}{0{,}496} = -\frac{(0{,}2 - 0{,}7099,\ -0{,}3 + 0{,}7099)^\top}{0{,}496} = (1{,}028, -0{,}826)^\top.$$

Alakellenőrzés: a score vektor dimenziója $d = 2$, megegyezik x_3 dimenziójával.

F3 megoldás. A függvény:

```
import torch

def forward_sample(X0, t, alpha_bar):
    # X0: (N,d), t: int, alpha_bar[t-1] skalár
    N, d = X0.shape
    eps = torch.randn(N, d)
    Xt = torch.sqrt(alpha_bar[t-1]) * X0 + torch.sqrt(1.0 - alpha_bar[t-1]) * eps
    return Xt
```

Ellenőrzés: $N = 3$, $d = 4$, X_0 tetszőleges, például `torch.zeros(3,4)`. Ekkor x_t átlaga elméletileg nulla, varianciája $1 - \bar{\alpha}_t$. A Monte-Carlo-átlag dimenziónkénti szórása $\sqrt{(1-\bar{\alpha}_t)/M}$, ami $M = 20000$ és $1-\bar{\alpha}_t \le 1$ mellett legfeljebb 0,0071; a négy dimenzió maximuma ennek többszöröse is lehet, ezért a 10^{-2}-es abszolút tűrés önmagában nem garantált. Vagy rögzített seedet használunk (a futás így determinisztikus), vagy a tűrést a $4\sigma \approx 0{,}028$ értékhez igazítjuk. A kimenet alakja `(3,4)`.

F4 megoldás. A hibás kód két problémát rejt. Egyrészt hiányzik a $\frac{1}{\sqrt{\alpha_t}}$ skálázás, másrészt a zajtag kivonásánál a $\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$ együttható hiányzik, és a $\sigma_t z$ zajtag is elmarad. A helyes formula a labor kódjában szereplő `mu` kifejezés. A hibás lépés következménye, hogy a kapott x_{t-1} nem a tanult $p_\theta(x_{t-1} \mid x_t)$ átmenetből származik: a várható értéke általában eltér a μ_θ átlagtól, a varianciája pedig nulla, tehát a mintavételezés determinisztikussá válik. (A pontos posterior $q(x_{t-1} \mid x_t, x_0)$ csak a tanítási cél levezetésében jelenik meg; a mintavétel a tanult p_θ-t használja.) A javított lépés:

```
a_t = alpha[t-1]              # _t
ab_t = alpha_bar[t-1]         # _t
b_t = beta[t-1]               # _t
eps_hat = model(x_t, t)       # _(x_t,t), (N,d)
mu = (x_t - b_t / torch.sqrt(1.0 - ab_t) * eps_hat) / torch.sqrt(a_t)
z = torch.randn_like(x_t) if t > 1 else torch.zeros_like(x_t)
x_prev = mu + torch.sqrt(b_t) * z
```

Itt a μ_θ átlag a prediktált zajból számítódik, a $\sqrt{\beta_t}\, z$ tag adja a $\sigma_t^2 = \beta_t$ varianciájú sztochasztikus komponenst. Ellenőrzés: futtassuk le egy batch-re a labor `mu` kifejezésével, és a kimenet varianciája a hozzáadott z miatt közelítse a β_t értéket.

F5 megoldás. Lehetséges protokoll: az adateloszlás legyen $x_0 \sim \mathcal{N}(\mu_0, \Sigma_0)$, például $\mu_0 = 0$, $\Sigma_0 = I$; generáljunk 10000 tanító- és 2000 tesztvektort. Használjunk $T = 20$ lépést, lineáris schedule-t $\beta_t \in [10^{-4};\ 0{,}02]$. Minden batchhez válasszunk véletlen t-t, adjunk hozzá zajt a zárt alakból, és minimalizáljuk az MSE-t egy $16 \to 64 \to 64 \to 16$ MLP-vel; a t feltételt a bemenet 17. koordinátájaként adjuk át (a normalizált t/T érték), mert $\epsilon_\theta(x_t, t)$ kétargumentumú, időfeltételes függvény. Tanulási ráta 10^{-3}, Adam, 5000 lépés, batch méret 128, seed 1234. Metrika: teszt MSE a zajpredikcióra, valamint a generált minták hisztogramjának vizuális összehasonlítása a valódi eloszlással. A legfontosabb korlát: a 16 dimenziós Gauss-toy feladat nem tartalmazza a képek térbeli struktúráját, a nagy T és az U-Net architektúra hatásait, ezért az eredményből nem következtethetünk képi diffúziós modellek teljesítményére.

31.3.11 Tipikus tévedés és ellenpélda

Gyakori tévedés a zajpredikciós cél és a mintavételezési eljárás összekeverése. A tanítási cél a hozzáadott ϵ zajt prediktálja, de a mintavételezéskor ezt a predikciót transzformálni kell a reverse átlagba. Ellenpélda: tegyük fel, hogy valaki a tanítás során közvetlenül x_0-t prediktálja ($\hat{x}_0$), majd a mintavételnél a hálózat kimenetét átváltás nélkül, közvetlenül zajpredikcióként használja. A helyes átváltás $\hat{\epsilon} = (x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0)/\sqrt{1-\bar{\alpha}_t}$; enélkül a reverse átlag a $\hat{x}_0$-t a $\beta_t/\sqrt{1-\bar{\alpha}_t}$ együtthatóval vonja ki. Kis t-nél $1 - \bar{\alpha}_t \approx \beta_t$, ezért a helyes lépés közel $\hat{x}_0$-t adna, a téves lépés viszont csak $\sqrt{\beta_t}\hat{x}_0$-t von ki x_t-ből, így a minta közel marad a zajos x_t-hez, és a generált minta zajos lesz. A két cél különböző skálázást és különböző reverse átlagot eredményez, ezért a paraméterezést a tanításban és a mintavételben konzisztensen kell használni. A labor oracle (jós) denoiser példája éppen azt mutatja, hogy még ha a zajt pontosan ismerjük is, a mintavételi lépésben a μ-formulát kell alkalmazni, nem pedig egyszerűen kivonni a predikált zajt.

31.3.12 Önellenőrzés és visszahívás

Önellenőrzés: a lemma bizonyításában minden kovarianciamátrix I-vel arányos, tehát a komponensek függetlenek maradnak; a $\bar{\alpha}_t$ monoton csökken, mert $\alpha_t \in (0, 1)$. A $t = 0$ határesetben a képlet degenerált Dirac-eloszlást adna, ezért a lemma csak $t \geq 1$-re érvényes.

Visszahívás a 30.3. Negatív nélküli módszerek és collapse alfejezetből: ott a kontrasztív reprezentációtanulás collapse-a volt a degeneráció; itt a forward folyamat rögzített, ezért a diffúziós

tanítás nem szenved ugyanabban a minmax-instabilitásban. Ugyanakkor ha a denoiser konstans zajt prediktál, az is degenerált modell; ennek kiszűrése a validációs MSE-vel történik. A 27.2. U-Net, encoder–decoder és skip alfejezetben látott U-Net itt nem szegmentál, hanem zajtérképet állít elő; a skip-kapcsolatok szerepe azonban analóg. A 19.3. EfficientNet és architektúraskálázás alfejezet skálázási elvei analógiaként használhatók: a konvolúciós hálózat költsége a fenti $O(H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot k^2)$ képlet szerint nő, és a mintavételezés T-szeres kiértékelést igényel.

31.3.13 Rövid összefoglaló

A diffúziós modell egy rögzített Gauss-forward folyamatból indul, amely $x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t$ rekurzióval zajosítja az adatot. A lemmában beláttuk, hogy $q(x_t \mid x_0)$ zárt alakú: átlaga $\sqrt{\bar{\alpha}_t}x_0$, varianciája $1-\bar{\alpha}_t$. A feltételes score arányos a negatív zajjal, ezért a tanult denoiser zajpredikciós MSE-céllal tanítható. A marginális score lemmája szerint az optimális zajprediktor a feltételes várható érték, így a zajpredikció és a score-becslés ugyanannak a feladatnak két megfogalmazása. A mintavételezés T reverse lépésből áll, amelyek a predikált zajból számított reverse átlag körül Gauss-zajt adnak. Az U-Net a térbeli zajtérkép becslésére és az időfeltétel beépítésére szolgál.

Az induló kérdésre a válasz: igen, a forward folyamat zárt Gauss-alakja és a tanult zajprediktor segítségével a fordított lépés feltételes eloszlása is Gauss-alakban közelíthető, ezért a teljes Markov-lánc szimulációja nélkül is tanítható a denoiser.

31.3.14 Következő alkalmazás

A 31.4. Generatív felhasználás és értékelés alfejezet a diffúziós modell gyakorlati felhasználását és értékelését tárgyalja: milyen metrikák mérik a mintaminőséget és a sokféleséget, és hogyan kell a tanítóadat származását dokumentálni.

31.3.15 Források

- [P20] Ho, J., Jain, A., Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239. https://arxiv.org/abs/2006.11239.
- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. A Gauss-eloszlások transzformációs és feltételes azonosságait a könyvben megadott alakban használjuk.
- Song, Y. és Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. NeurIPS. arXiv:1907.05600. https://arxiv.org/abs/1907.05600. A fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S. és Poole, B. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations*. ICLR. arXiv:2011.13456. https://arxiv.org/abs/2011.13456. A fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

31.4 Generatív felhasználás és értékelés

31.4.1 Cél, előismeret és induló kérdés

A 31.3. Diffúziós zajmodell és CNN-denoiser egységben a diffúziós zajmodellt [P20], a feltételes denoisert és a mintavételezési sémát építettük fel. Ott a generált minta még csak egyetlen

realizáció volt; most azt vizsgáljuk, mit jelent egy generatív modell *eloszlását* jónak nevezni. A likelihood, a mintaminőség, a változatosság, a memorizáció és a származás öt különböző kérdés, amelyeket nem szabad egyetlen pontszámba mosni. Az induló kérdés: ha egy modell egyetlen, emberi szemmel tökéletesnek tűnő képet állít elő, bizonyítja-e ez, hogy a modell jól közelíti a valódi adateloszlást? A válasz nem, és ezt formális ellenpéldával is igazolni fogjuk.

Visszautalás: a 30.4. Önfelügyelt kísérlet korrekt összevetése erőforrás-könyvelése azt tanította, hogy a módszerhatást csak azonos költségkeret mellett lehet értelmezni; ugyanez a fegyelem kell a generatív modellek összehasonlításához is. A 27.4. Instance mask és összetett feladat instance-felosztási ellenpéldája azt mutatta, hogy egyetlen aggregált pixelmetrika elfedi a szerkezeti hibát; a generatív értékelésben is hasonló a helyzet. A 19.4. ConvNeXt és a recept/architektúra elválasztása recept/architektúra szétválasztása pedig arra figyelmeztet, hogy az értékelési protokollt a modellválasztás előtt kell rögzíteni.

31.4.2 Definíciók, jelölések és tenzoralakok

Legyen a megfigyelési tér $\mathcal{X} = [0,1]^{C\times H\times W}$ a képek halmaza; a valódi, ismeretlen adateloszlást p_{data}, a generátor által indukált eloszlást pedig p_g jelöli. Egy generált batch alakja $X_g \in \mathbb{R}^{B\times C\times H\times W}$, ahol B a batchméret, C a csatornaszám, H, W a térbeli méretek; a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab esetén $C = 1$, $H = W = 24$. A generált minták függetlenek, ha a mintavételi folyamat minden $x_g^{(i)} \in \mathbb{R}^{C\times H\times W}$ mintát azonos p_g eloszlásból, függetlenül húz.

A **teszt log-likelihood** (átlagos log-sűrűség) egy N elemű, a modelltől független tesztmintán:

$$\ell(p_g; D_{test}) = \frac{1}{N}\sum_{i=1}^{N} \log p_g(x_i), \qquad x_i \in D_{test}.$$

Itt $\log p_g(x_i)$ folytonos képtérben log-sűrűséget, véges mintatéren (a későbbi lemma esetében) log-tömeget jelent; a jelölés közös, a két mennyiség azonban más típusú. A bit-per-dimension (BPD) a likelihood normált, előjelfordított változata:

$$\mathrm{BPD} = -\frac{1}{d\ln 2}\,\ell,$$

ahol $d = CHW$ a dimenziószám és ln a természetes logaritmus. A mínusz előjel azért kell, mert a log-likelihood tipikusan negatív, a BPD viszont pozitív bitszámot fejez ki. Folytonos sűrűségnél a differenciális log-likelihood pozitív is lehet (a sűrűség értéke meghaladhatja az 1-et), ezért ilyenkor a BPD csak diszkretizált likelihoodból (például kvantálás vagy hozzáadott egyenletes zaj után) értelmezhető nemnegatív bitszámként. A likelihood a *sűrűségmodellt* méri: azt, hogy a modell mekkora valószínűségi tömeget rendel a tesztadathoz. A **mintaminőség** (sample quality) ezzel szemben egy realizált minta vagy mintahalmaz emberi vagy automatikus bíráló által adott minőségi pontszáma; ez nem a likelihood függvénye, és a kettő nem ekvivalens.

A **Fréchet Inception Distance** (FID) a valódi és generált minták egy köztes reprezentációjában számolt Gauss-közelítésen alapul (Heusel és mtsai, 2017). Ha a reprezentációban a valódi minták átlaga $\mu_r \in \mathbb{R}^m$, kovarianciája $\Sigma_r \in \mathbb{R}^{m\times m}$, a generáltaké μ_g, Σ_g, akkor a Fréchet-távolság:

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right),$$

ahol $(\Sigma_r\Sigma_g)^{1/2}$ a mátrixszorzat egyértelmű pozitív szemidefinit négyzetgyöke. Az 1-dimenziós speciális esetben $\Sigma_r = \sigma_r^2$, $\Sigma_g = \sigma_g^2$, és a kifejezés $(\mu_r - \mu_g)^2 + \sigma_r^2 + \sigma_g^2 - 2\sigma_r\sigma_g = (\mu_r - \mu_g)^2 + (\sigma_r - \sigma_g)^2$ alakra egyszerűsödik, feltéve, hogy $\sigma_r, \sigma_g \geq 0$.

A **változatosságot** (diversity) a generált halmaz belső szóródásával mérjük. Egy M elemű $S_g = \{x^{(1)}, \ldots, x^{(M)}\}$ halmazra a páronkénti átlagos L2 távolság:

$$\bar{d}(S_g) = \frac{2}{M(M-1)} \sum_{1 \leq i < j \leq M} \|x^{(i)} - x^{(j)}\|_2,$$

ahol az összegzés az $MCHW$ dimenziós vektorok euklideszi normáját használja. A **lefedettség** (coverage) azt méri, hogy a valódi eloszlás támogatásának mekkora része van reprezentálva; egy egyszerű proxy: a valódi tesztminták hányadához létezik ϵ-nál közelebbi generált minta. A laborterv (lásd alább) ezt a proxyot a validációs és a test splitre külön jelenti.

Folytonos halmazon a fenti $\bar{d}$ a páronkénti átlagos L2-távolság. Diszkrét, véges halmazon ehelyett a páronkénti egyezésarány is használható: $1 - \frac{1}{\binom{M}{2}} \sum_{i<j} \mathbf{1}[x^{(i)} = x^{(j)}]$. Ez más mennyiség, ezért a kis példában külön néven, *egyezésarány-proxy*ként szerepel, és nem azonos a folytonos $\bar{d}$-vel.

A **memorizáció** heurisztikus definíciója: egy x_g generált minta ϵ-szinten memorizált, ha

$$\min_{x_t \in D_{train}} \|x_g - x_t\|_2 \leq \epsilon,$$

ahol D_{train} a tanítóhalmaz és ϵ az adateloszlásból származtatott küszöb. A **származás** (provenance) nem numerikus metrika, hanem dokumentációs követelmény: adatforrás, licenc, modellarchitektúra, súly-hash, generálási seed, szoftververzió és időbélyeg rögzítése.

31.4.3 A likelihood és a mintaminőség szétválasztása

A két fogalom nem felcserélhető (Theis, van den Oord és Bethge, 2016). A likelihood a tesztpontokon felvett sűrűséget méri, nem a minták realitását: egy széles modell, amely minden tesztpont körül véges sűrűséget ad, jobb átlagos log-likelihoodot érhet el, mint egy éles modell, amely elhibázza a móduszokat, noha a széles modell mintái nem hasonlítanak az adatra. Egydimenziós példa: ha az adat a ± 1 pontokban fekszik, a $\mathcal{N}(0, 4)$ modell átlagos log-sűrűsége $\approx -1{,}74$, a móduszok közé tett éles $\mathcal{N}(0,\ 0{,}01)$ modellé $\approx -48{,}6$ (kézi számítás); az előbbi mégis homályos, szétszórt mintákat ad. Fordítva, egy éles, "látványos" mintákat adó generátor elhagyhatja a valódi eloszlás teljes móduszait, ezért likelihoodja a kihagyott pontokban nulla (illetve log-likelihoodja $-\infty$). A likelihood tehát a *lefedettség* irányában érzékeny, a mintaminőség pedig a *realizmus* irányában.

31.4.4 Formális ellenpélda: egyetlen látványos kép nem eloszlásbizonyíték

A következő lemmát az egyszerűség kedvéért véges $\mathcal{X}$ mintatéren mondjuk ki, ahol a valószínűségi eloszlások tömegfüggvények; a TV- és KL-divergencia definícióját a [B01] tankönyv szerinti alakban használjuk. Folytonos képtérben a Dirac-delta nem rendelkezik sűrűséggel, ezért ott a KL-alapú érv nem értelmezhető; a TV-egyenlőtlenség viszont átvihető, és ilyenkor a delta-generátorra $TV = 1$ adódik.

Lemma (delta-generátor ellenpélda). Legyen p_{data} olyan eloszlás $\mathcal{X}$-en, amelynek legalább két különböző pontja van pozitív tömeggel, és legyen $p_g = \delta_a$ a Dirac-delta egy $a \in \mathcal{X}$ pontra. Ekkor:

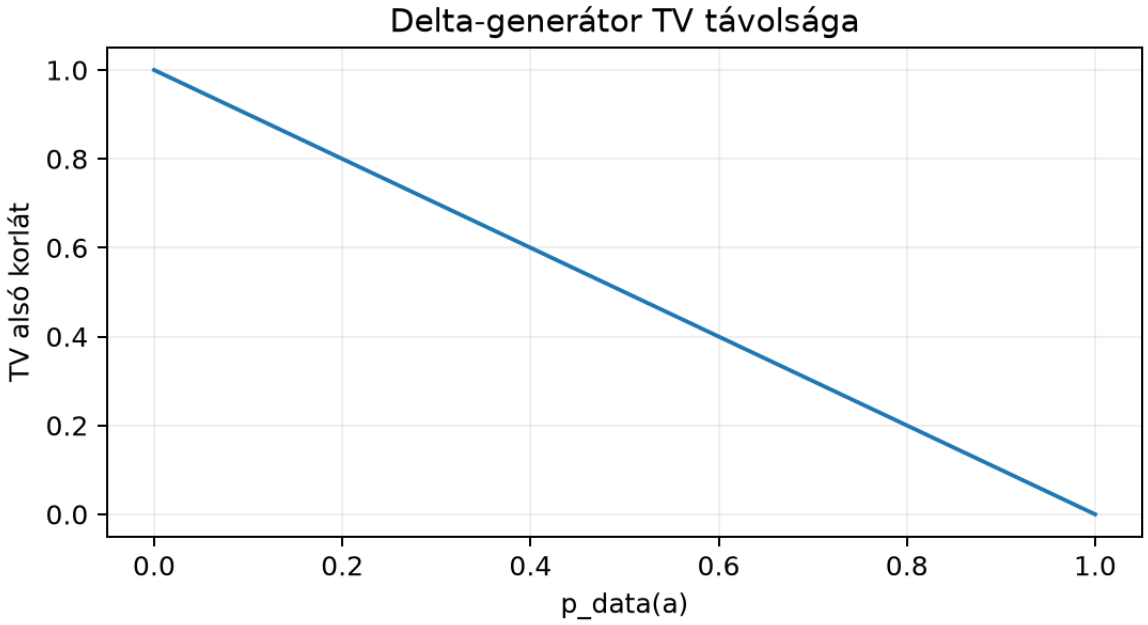
1. a teljes variációs távolság $TV(p_{data}, p_g) \geq 1 - p_{data}(a) > 0$, és
2. a KL-divergencia $D_{KL}(p_{data} \| p_g) = \infty$.

Bizonyítás. (1) A teljes variációs távolság definíciója $TV(p, q) = \sup_{A \subseteq \mathcal{X}} |p(A) - q(A)|$. Válasszuk az $A = \mathcal{X} \setminus \{a\}$ eseményt. Ekkor $p_{data}(A) = 1 - p_{data}(a)$, míg $p_g(A) = 0$, mert a delta-eloszlás teljes tömege a-ban van. A szuprémum legalább ekkora, tehát $TV \geq 1 - p_{data}(a)$. Mivel a feltétel szerint legalább két pontnak van pozitív tömege, $p_{data}(a) < 1$, így a jobb oldal pozitív.

(2) A diszkrét KL-divergencia $D_{KL}(p_{data} \| p_g) = \sum_{x \in \mathcal{X}} p_{data}(x) \log \frac{p_{data}(x)}{p_g(x)}$. Ha $p_{data}(x) > 0$ és $p_g(x) = 0$, akkor a megfelelő tag $p_{data}(x) \log \frac{p_{data}(x)}{0} = \infty$. A feltétel szerint létezik ilyen x, tehát a teljes összeg végtelen. ■

Következmény: ha a generátor mindig ugyanazt a vizuálisan tökéletes a képet adja, akkor az egyetlen realizált minta "látványos" lehet, miközben a generátor eloszlása érdemben eltér a valódi eloszlástól: a TV távolság legalább $1 - p_{data}(a) > 0$, a TV maximuma pedig 1. Az alsó korlát csak akkor éri el az 1-et, ha $p_{data}(a) = 0$; a lemma tehát alsó korlátot ad, nem azt állítja, hogy a távolság minden esetben a maximális 1. A mintaminőség így nem azonos a modellminőséggel.

Ezt az összefüggést szemlélteti a következő, a lemmából számított diagram:



31.1. ábra: Delta-generátor TV távolsága

A lemmából számított alsó korlát, nem mért adat.

31.4.5 FID: definíciós és becslési korlátok

A FID a valódi és generált eloszlások közötti Fréchet-távolságot egy köztes reprezentációban számítja, és a két eloszlást Gauss-eloszlással közelíti. Ebből három korlát következik.

1. **Gauss-feltevés.** A FID csak az első két momentumot használja. Két eloszlás, amelynek azonos az átlaga és a kovarianciája, a FID szerint megkülönböztethetetlen, még akkor is, ha a magasabb momentumok vagy a támogatás szerkezete alapvetően eltér.

2. **Reprezentációfüggés.** Az “Inception” hálózat (Szegedy és mtsai, 2016) egy adott osztályozási feladaton tanult hálózat; a FID az általa definiált jellemzőtérben (feature space) mér. Egy másik hálózat vagy egy finomhangolt változat más FID-et adhat ugyanazokra a mintákra. A FID nem feladatfüggetlen képminőség-mérték.

3. **Véges mintás becslés.** A gyakorlatban μ_r, Σ_r és μ_g, Σ_g véges mintából becsült mennyiségek. A FID becslése torzított és nagy varianciájú lehet, különösen kevés generált minta esetén; egyetlen FID értékhez ezért mindig meg kell adni a mintaméreteket, a seed-eket és ismételt becslések szórását. Egyetlen seed FID értéke nem megbízható modellminősítés.

A FID további korlátja, hogy a $(\Sigma_r\Sigma_g)^{1/2}$ mátrix-négyzetgyök numerikusan érzékeny lehet, ha a kovarianciamátrixok közel szinguláris.

31.4.6 Sokféleség, memorizáció és származás mint külön ellenőrzési réteg

A **diversity** hiánya akkor is fennállhat, ha a FID alacsony. A FID csak az első két momentumot használja: ha a generátor a valódi eloszlás átlagát és kovarianciáját pontosan reprodukálja, a FID nulla, miközben a generált halmaz belső szerkezete (a móduszok, a klaszterek és a páronkénti távolságok eloszlása) teljesen eltérhet a valódiétól. Egyetlen pontból álló halmaz önmagában ehhez nem elég: ilyenkor $\Sigma_g = 0$, és a FID értéke $\operatorname{Tr}\Sigma_r > 0$ (kivéve, ha a valódi eloszlás is degenerált), tehát a FID ezt a collapse-t nem feltétlenül jelzi nullaként; a diversity-mutató viszont közvetlenül kimutatja. Ezért a FID mellett független diversity-mutatót kell számolni, például a $\bar{d}(S_g)$ páronkénti átlagos távolságot, és azt a valódi tanítóhalmaz ugyanilyen mutatójával összehasonlítani.

A **memorization** ellenőrzése azért fontos, mert a generátor a tanítóhalmazt másolhatja új minta helyett. A fenti ϵ-küszöbös definíció heurisztikus: az ϵ megválasztása adatfüggő. Jó gyakorlat a tanítóhalmaz páronkénti L2 távolságainak alsó percentiliséből származtatni ϵ-t, így a küszöb a tényleges adatsűrűséghez igazodik. A memorizáció detektálása nem bizonyítja a szándékos másolást, de jelzi az általánosítási kockázatot. Az adatvédelmi következményeket külön kell kezelni; a definíció nem jogi fogalom.

A **provenance** a generált műtárgyhoz tartozó metaadat. Minimális mezők: a generátor modell architektúrája és súlyainak SHA-256 hash-e, a tanítóadat forrása és licence, a generálási seed és konfiguráció, a szoftververzió, valamint a generálás időpontja. A provenance nélkül egy generált kép tudományos vagy üzleti felhasználása nem reprodukálható és nem auditálható.

31.4.7 Számított kis példa és ellenőrző kód

A következő kód a fenti definíciókat számítja ki egy diszkrét háromosztályos példán, ahol a valódi eloszlás $p_{data} = [0{,}5;\ 0{,}3;\ 0{,}2]$, a generátor pedig delta-eloszlás az A osztályon. A kód nem igényel külső mélytanulási könyvtárat.

```
# munkakönyvtár: lab/n124/
# n124_szamitott_pelda.py
import numpy as np

# Diszkrét eloszlások: A=0, B=1, C=2
p_data = np.array([0.5, 0.3, 0.2], dtype=np.float32)
p_gen = np.array([1.0, 0.0, 0.0], dtype=np.float32)

# Teljes variációs távolság
tv = 0.5 * np.sum(np.abs(p_data - p_gen))
print(f"TV = {tv:.6f}")
```

```python
# KL(p_data || p_gen); a p_gen=0 tagok +inf-et adnak
finite_mask = p_gen > 0
finite_part = np.sum(p_data[finite_mask] *
                     np.log(p_data[finite_mask] / p_gen[finite_mask]))
print(f"KL véges része = {finite_part:.6f}; teljes KL = +inf")

# 1D FID proxy: valós és generált embedding statisztikák
mu_r, sigma_r = 2.5, np.sqrt(1.25)   # valós: 1,2,3,4 átlaga és szórása
mu_g, sigma_g = 2.5, 0.0             # generált: mind 2.5
fid_1d = (mu_r - mu_g) ** 2 + (sigma_r - sigma_g) ** 2
print(f"FID_1D = {fid_1d:.6f}")

# Egyezésarány-proxy: páronkénti egyezés egy 4 elemű generált halmazon
samples = np.array([0, 0, 0, 0], dtype=np.int64)
pairs_equal = np.mean([samples[i] == samples[j]
                       for i in range(len(samples))
                       for j in range(i + 1, len(samples))])
print(f"Páronkénti egyezés = {pairs_equal:.3f}; "
      f"egyezésarány-proxy = {1 - pairs_equal:.3f}")

# Ellenőrzések a definíciók kézi értékeire
assert abs(float(tv) - 0.5) < 1e-6
assert abs(float(finite_part) + 0.346574) < 1e-6
assert abs(float(fid_1d) - 1.25) < 1e-6
assert float(pairs_equal) == 1.0
```

A kód lefutott (Python 3.12.3, NumPy 2.5.4, CPU, float32); mért kimenet:

```
TV = 0.500000
KL véges része = -0.346574; teljes KL = +inf
FID_1D = 1.250000
Páronkénti egyezés = 1.000; egyezésarány-proxy = 0.000
```

A számok megfelelnek a kézi ellenőrzésnek: $TV = 0{,}5 \cdot (0{,}5 + 0{,}3 + 0{,}2) = 0{,}5$; a KL véges része $0{,}5 \log(0{,}5/1) = -0{,}346574$, a teljes KL pedig $+\infty$; a FID_1D $0^2 + (\sqrt{1{,}25} - 0)^2 = 1{,}25$; a páronkénti egyezés 1,0, tehát az egyezésarány-proxy 0. A kimenet ezeknek a definícióknak a közvetlen számítása, nem tanított modell mérése.

31.4.8 Laborterv: GlyphLab generatív értékelési protokoll

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a korábbi laborokban előállított checkpoint(ok); a futásidőt a futtatás során mért értékként kell rögzíteni.

A feladat egy kis generatív kísérlet *tervének* elkészítése, előre rögzített értékeléssel, adat- és súlyforrással. Az alábbi terv a GlyphLab adatán (lab/ADATOK.md) alapul, és a kötelező főutat követi: kis CPU, eredeti adatok, külső súly nélkül.

Adatforrás. `lab/data/images.npy`, `lab/data/labels.npy`, `lab/data/metadata.csv`. A generatív modellt kizárólag a train split 240 képén tanítjuk; a validációs split (60 kép) a checkpoint kiválasztására szolgál, a test (60 kép) pedig kizárólag a végső, jelentett metrikákra. A scene_id szerinti csoportosítást megőrizzük: ugyanazon jelenet két nézete ugyanabba a splitbe tartozik.

Modell és súlyforrás. Egy kis VAE a 31.1. VAE és variációs alsó korlát definíciói szerint, a [P18] cikk modellcsaládjában; a protokoll egyetlen modellcsaládot rögzít, nem hagyja nyitva a VAE/diffúzió választást. Az architektúra: encoder Conv2d(1→8, 3×3, stride 2, padding 1), ReLU, Conv2d(8→16, 3×3, stride 2, padding 1), ReLU, majd Linear(576→32), amely a 16 dimenziós

latens μ_z-t és $\log \sigma_z^2$-t adja; a dekóder Linear(16→576), reshape (16,6,6), ConvTranspose2d(16→8, 3×3, stride 2, padding 1, output_padding 1), ReLU, ConvTranspose2d(8→2, 3×3, stride 2, padding 1, output_padding 1), amely pixelonként a rekonstrukciós μ_x-et és $\log \sigma_x^2$-t adja. A veszteség a Gauss-rekonstrukciós NLL és a latens KL összege ($\beta = 1$). A súlyokat saját tanítással állítjuk elő: seed=20261020, Adam lr=1e-3, batch=32, 200 epoch, float32 CPU; a checkpointot a legjobb validációs ELBO (Gauss-NLL + KL) választja ki, ami likelihood-alapú, generatív kritérium. A modellválasztás után egyszer futtatjuk a tesztet.

Értékelési protokoll (a tanítás előtt rögzítve).

1. *Teszt ELBO.* A teszt képekre kiszámítjuk az átlagos ELBO-t (Gauss-NLL + KL), ami a log-likelihood alsó korlátja; a validációs ELBO-t csak a checkpoint kiválasztására használjuk. Emellett a rekonstrukciós MSE-t leíró jellemzőként jelentjük; az MSE rögzített varianciájú Gauss-dekóder mellett a NLL-lel arányos, de önmagában nem likelihood-alsó korlát, mert a KL-tagot nem tartalmazza.
2. *Generált minták.* A rögzített modellből $z \sim \mathcal{N}(0, I)$ prior-mintát húzunk, és a dekóder μ_x fejét használjuk; így 240 minta keletkezik. A generálást három előre rögzített seeddel ismételjük (20261020, 20261021, 20261022), és minden metrikát a három futás átlagaként $\pm$ szórással jelentünk. A tanítási varianciát ez a terv nem méri (egy tanítási seed); ezt a jelentés korlátjaként tüntetjük fel.
3. *FID proxy.* A nyers pixelteret a train képeken illesztett PCA-val 8 dimenzióra vetítjük. A PCA-t csak a train halmazon illesztjük; a vetítést a test és a generált halmazra alkalmazzuk. A referencia a test split 60 képe (a validációs split a checkpoint-kiválasztáshoz tartozik, ezért nem használjuk referenciaként). A 8 dimenzió választását az indokolja, hogy 60 mintából a 16 dimenziós kovariancia becslése túl zajos lenne; a becslés így is torzított, ezt a jelentésben korlátként feltüntetjük. A FID-et a fenti képlettel számítjuk. Ez nem azonos a klasszikus Inception-FID-del, ezért *FID-pixel-PCA proxy* néven jelentjük.
4. *Diversity.* A generált 240 képre $\bar{d}(S_g)$ páronkénti L2 távolságot számítunk, és összehasonlítjuk a train 240 képére számított $\bar{d}(S_{train})$ értékkel. Jelentjük az arányt is.
5. *Lefedettség.* A test 60 valódi képére kiszámítjuk, hogy mekkora hányadukhoz létezik a generált halmazban ϵ-nál közelebbi minta; az ϵ ugyanaz a train páronkénti távolságaiból származtatott küszöb, mint a memorizációnál.
6. *Memorizáció.* Minden generált képre kiszámítjuk a legközelebbi train kép L2 távolságát a pixel térben (576 dimenziós lapított vektor). A küszöböt a train halmaz páronkénti távolságainak 5. percentiliséből vesszük. Jelentjük azon generált képek arányát, amelyekre a távolság e küszöb alatt van.
7. *Ellenőrzés.* A checkpoint kiválasztása kizárólag a train és validációs splitet használja; a validációs split nem szerepel a jelentett metrikákban. A test split semmilyen döntésben nem játszik szerepet, csak a jelentésben. A generált minták száma és a seed-ek rögzítettek; a PCA-illesztés kizárólag train. A futás után a nyers metrikák és a generált minták hash-e kerül a jegyzőkönyvbe.

Ez a terv nem futott le; ezért mért ELBO-, FID-, diversity- vagy memorizációs számot nem állítunk.

31.4.9 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, miért nem bizonyítja egyetlen látványos generált kép a generátor jó eloszlásmodelljét. Adjon konkrét ellenpéldát a lemmából.

F2 (levezetési, nehézség 2). Legyen $\mathcal{X} = \{0, 1, 2\}$, $p_{data} = [0{,}4;\ 0{,}4;\ 0{,}2]$, $p_g = [1, 0, 0]$. Számolja ki a TV távolságot, a KL-divergenciát (ahol értelmezhető), és a delta-generátor egyezésarány-proxyját egy 5 elemű generált halmazra.

F3 (implementációs, nehézség 2). Írjon Python-függvényt, amely két 1-dimenziós embeddinghalmazra ($r \in \mathbb{R}^{N_1}$, $g \in \mathbb{R}^{N_2}$) kiszámítja az 1D FID proxy-t: $(\mu_r - \mu_g)^2 + (\sigma_r - \sigma_g)^2$. Tesztelje a fenti példán (valós $1, 2, 3, 4$; generált mind 2,5). Adja meg a várt eredményt és a numerikus toleranciát.

F4 (hibakeresési, nehézség 2). Az alábbi kód a memorizációs küszöböt akarja meghatározni, de hibás:

```
# hibás vázlat
def memorized_ratio(gen, train, eps_fraction=0.05):
    # train: (N, D) float32; gen: (M, D) float32
    D = train - train[:, None, :]          # páronkénti különbség, alak (N,N,D)
    flat = np.linalg.norm(D, axis=-1)
    eps = np.quantile(flat, eps_fraction)
    dist_gen = np.linalg.norm(
        gen[:, None, :] - train[None, :, :], axis=-1)
    return np.mean(np.min(dist_gen, axis=1) <= eps)
```

Lokalizálja a hibát, magyarázza el a következményét, és írja meg a javított változatot. Ellenőrizze egy kis méretű mesterséges adaton.

F5 (kísérlettervezési, nehézség 3). Tervezzen értékelési protokollt egy külső, előtanított generatív modell memorizációjának mérésére 10 000 képből álló saját tesztkörnyezetben; a külső modell tanítóhalmaza nem hozzáférhető, ezt a tervnek kezelnie kell. Adja meg az adatforrást, a küszöbválasztást, a kontrollcsoportot és a következtetési korlátot. A terv nem igényelhet futtatást, de a döntési pontokat explicit módon kell tartalmaznia.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

31.4.10 Megoldások és indoklás

F1 megoldás. Az egyetlen látványos minta csak egyetlen pont a mintatérből; a generatív modell minősége az eloszlás egészére vonatkozik. A delta-generátor lemmája szerint ha $p_g = \delta_a$, akkor $TV(p_{data}, p_g) \geq 1 - p_{data}(a)$. Ha a valódi eloszlás 60% valószínűséggel más osztályokat tartalmaz, a TV legalább 0,6, ami érdemi eltérés a maximális 1-től, miközben az a minta önmagában tökéletesnek látszhat. A TV maximuma 1, ezt az alsó korlát csak $p_{data}(a) = 0$ esetén éri el. A mintaminőség tehát szükséges, de nem elégséges feltétel.

F2 megoldás. TV: $0{,}5(|0{,}4 - 1| + |0{,}4 - 0| + |0{,}2 - 0|) = 0{,}5(0{,}6 + 0{,}4 + 0{,}2) = 0{,}6$. KL: $0{,}4 \ln(0{,}4/1) + 0{,}4 \ln(0{,}4/0) + 0{,}2 \ln(0{,}2/0) = +\infty$, mert két tag végtelen. A generált 5 elemű halmaz mind a 0 osztályt adja, ezért a páronkénti egyezés 1, az egyezésarány-proxy $1 - 1 = 0$. Alakellenőrzés: a TV $[0, 1]$-ben van, a KL nemnegatív vagy végtelen; az egyezésarány-proxy $[0, 1]$-ben van.

F3 megoldás.

```
import numpy as np

def fid_1d(r, g):
    """1D FID proxy: (mean_r-mean_g)^2 + (std_r-std_g)^2."""
    r = np.asarray(r, dtype=np.float32)
```

```
    g = np.asarray(g, dtype=np.float32)
    mu_r, mu_g = r.mean(), g.mean()
    sigma_r, sigma_g = r.std(), g.std()  # population std, ddof=0
    return (mu_r - mu_g) ** 2 + (sigma_r - sigma_g) ** 2

r = np.array([1, 2, 3, 4], dtype=np.float32)
g = np.full(5, 2.5, dtype=np.float32)
print(f"FID_1D = {fid_1d(r, g):.6f}")
```

A helyes eredmény: $\mu_r = 2{,}5$, $\sigma_r = \sqrt{1{,}25} \approx 1{,}118034$, $\mu_g = 2{,}5$, $\sigma_g = 0$, tehát $FID_{1D} = 1{,}25$. Tolerancia: $|FID_{1D} - 1{,}25| \leq 10^{-5}$ float32 esetén a szórás számítása miatt; ha a library `ddof=1`-et használ, az eredmény megváltozik, ezért a megoldás `ddof=0`-át explicit módon kell beállítani, vagy a függvény dokumentációjában jelezni.

F4 megoldás. A `train[:, None, :]` broadcasting helyes: a (N, D) és $(N, 1, D)$ tenzor kivonása (N, N, D) páronkénti különbséget ad, az `axis=-1` pedig páronkénti normát számol. A valódi hiba a `np.quantile(flat, eps_fraction)` hívás: a `flat` a teljes $N \times N$ mátrix, benne a diagonális N darab nullájával és minden pár kétszeresével. A nullák a minta alsó végére kerülnek, ezért az 5. percentilis 0 lesz; a hárompontos példán a kilenc érték $[0, 1, \sqrt{2}, 1, 0, 1, \sqrt{2}, 1, 0]$, a 5. percentilise 0. Így a küszöb $\epsilon = 0$, és a memorizációs arány csak a tökéletes egyezést számolná. Helyes változat:

```
def memorized_ratio(gen, train, eps_fraction=0.05):
    # train: (N, D) float32; gen: (M, D) float32
    pair_train = np.linalg.norm(
        train[:, None, :] - train[None, :, :], axis=-1)
    # a diagonális nulla; levesszük vagy bent hagyjuk,
    # de a percentilis számításánál a nullák torzíthatnak
    iu = np.triu_indices(train.shape[0], k=1)
    dist_train = pair_train[iu]
    eps = np.quantile(dist_train, eps_fraction)
    dist_gen = np.linalg.norm(
        gen[:, None, :] - train[None, :, :], axis=-1)
    return np.mean(np.min(dist_gen, axis=1) <= eps)
```

A javítás ellenőrzése: ha `train = [[0,0],[1,0],[0,1]]` float32, akkor a páronkénti távolságok $[1, 1, \sqrt{2}]$, az 5. percentilisük 1; ha a generált halmaz `[0.1,0.1]`, akkor a minimális train távolság $\sqrt{0{,}1^2 + 0{,}1^2} \approx 0{,}141$, ami ≤ 1, tehát a memorized_ratio 1. A hibás kód ezt nem adná.

F5 megoldás. A terv vázlata: (1) adatforrás: a saját tesztkörnyezet egy nyilvános, licenc szempontból újrafelhasználható, 10 000 képből álló gyűjtemény hash-elt azonosítókkal, valamint a külső modell 10 000 generált képe rögzített checkpointból, rögzített seeddel; (2) mivel a külső modell tanítóhalmaza nem hozzáférhető, közvetlen memorizációmérés nem lehetséges: vagy a szolgáltató ad a tanítóhalmazra vonatkozó távolság-orákulumot (vagy hash-elt tanítóhalmazt), vagy a protokoll csak azt méri, hogy a generált képek milyen arányban közelítik a saját tesztkörnyezetet, ami domain-átfedés, nem tanítóadat-memorizáció; (3) küszöbválasztás: a rendelkezésre álló referenciasokaság (a 10 000 képes tesztkörnyezet) páronkénti L2 távolságainak 1. percentilise pixel- vagy jellemzőtérben (feature space); a labor 240 képénél használt 5. percentilishez képest az 1. percentilis itt azért indokolt, mert a sűrűbb minta mellett a kis percentilis stabilabban becsülhető; (4) kontroll: egy független, a modell tanításában és a tesztkörnyezet kialakításában nem használt nyilvános halmaz, és annak az aránya, hogy a generált képek hány százalékának van ϵ-nál közelebbi szomszédja a kontrollban, összevetve a tesztkörnyezetre kapott aránnyal; (5) döntési pont: ha a tesztkörnyezethez mért közelségi arány a kontroll arányát érdemben meghaladja, és a szolgáltató igazoltan használta a tesztkörnyezet forrását tanításra, memorizáció gyanúja merül fel; tanítóhalmaz-orákulum nélkül ez csak gyanú, nem bizonyíték; (6) korlát: a mérés heurisztikus, nem bizonyít szándékos másolást, és az ϵ választása befolyásolja az eredményt; a következtetés csak a rögzített protokollra érvényes. A terv nem állít mért eredményt.

31.4.11 Tipikus tévedés és ellenpélda

A leggyakoribb tévhit, hogy **a generált minta automatikus tény**: ha a generált kép emberi szemmel valószerű, akkor a modell "megtanulta az eloszlást". A delta-generátor ellenpélda ezt cáfolja: a generált minta lehet tökéletes, miközben az eloszlás TV távolsága pozitív, sőt a KL divergenciája végtelen. Hasonló tévhit az, hogy **egyetlen generatív metrika teljes minőségítéletet ad**; a FID nem méri a memorizációt, a likelihood nem méri a vizuális élességet, a diversity pedig nem méri a realizmust. Csak több, előre rögzített metrika együttes jelentése lehet érvelés alapja.

31.4.12 Önellenőrzés és visszahívás

Önellenőrzés: a TV mindig $[0, 1]$-be esik; a KL nem szimmetrikus, és ha a generátor támogatása nem fedi a valódi eloszlás támogatását, a $D_{KL}(p_{data} \| p_g)$ végtelen; az 1D FID nemnegatív, és nulla csak akkor, ha az átlagok és a szórások is egyeznek. A memorizációs küszöbnek adatfüggőnek kell lennie; egy rögzített, az adattól független ϵ félrevezető.

Visszahívás: a 30.4. Önfelügyelt kísérlet korrekt összevetése erőforrás-könyvelése azt követeli, hogy a generatív összehasonlításokban is rögzítsük a címke- és számítási költségkeretet; a FID proxyhoz használt PCA illesztése a train halmazhoz tartozik. A 27.4. Instance mask és összetett feladat instance-ellenpéldája azt mutatta, hogy egy aggregált metrika elrejtheti a szerkezeti hibát, ezért a generatív értékelésben is szükség van a diverzitás és a lefedettség külön mérésére.

31.4.13 Rövid összefoglaló

A generatív modellek értékelése többdimenziós: a likelihood a sűrűségmodellt, a mintaminőség a realizált outputot, a diversity a változatosságot, a memorizáció a tanítóadat-másolást, a provenance pedig a származás dokumentálását méri. Egyetlen látványos kép nem bizonyítja a jó eloszlásmodellt: a delta-generátor ellenpéldában a TV távolság pozitív, a KL pedig végtelen, miközben a minta tökéletesnek tűnhet. A FID Gauss-közelítése és véges mintás becslése további korlátokat jelent. A laborterv a GlyphLab adatán előre rögzített FID-pixel-PCA proxy, diversity és memorizációs küszöb protokollját adja, mért eredmény nélkül.

31.4.14 Következő alkalmazás

A 32.1. Szekvenciamodell és változó hossz szekvenciamodellekre lép át. A generatív értékelés itt bevezetett szétválasztása közvetlenül érvényes lesz a nyelvi modellekre is: a likelihood (illetve a belőle származtatott perplexity, ami az átlagos log-likelihood exponenciálisa, és nem azonos magával a likelihoodhal), az emberi mintaminőség, a diversity és a memorizáció ott is külön mérőszám, és a provenance a szöveges modelleknél ugyanúgy kötelező.

31.4.15 Felhasznált források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. A TV- és KL-divergencia definícióját a könyvben megadott alakban használjuk.
- [P18] Kingma és Welling (2013). *Auto-Encoding Variational Bayes*. arXiv:1312.6114. https://arxiv.org/abs/1312.6114. A laborterv VAE-modelljének hivatkozása.

- [P20] Ho, Jain és Abbeel (2020). *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239. https://arxiv.org/abs/2006.11239. A 31.3. Diffúziós zajmodell és CNN-denoiser egységben felépített diffúziós modell hivatkozása.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B. és Hochreiter, S. (2017). *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. NeurIPS. arXiv:1706.08500. https://arxiv.org/abs/1706.08500. A FID definíciójának forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. és Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR. arXiv:1512.00567. https://arxiv.org/abs/1512.00567. A FID reprezentációját adó Inception-hálózat forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.
- Theis, L., van den Oord, A. és Bethge, M. (2016). *A note on the evaluation of generative models*. ICLR. arXiv:1511.01844. https://arxiv.org/abs/1511.01844. A likelihood és a mintaminőség szétválasztásának forrása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

32 Szekvenciák, attention és CNN–transformer összevetés

32.1 Szekvenciamodell és változó hossz

32.1.1 1. Cél, előismeret és induló kérdés

A CNN-ekben a megosztott konvolúciós kernel a **térbeli** pozíciókra alkalmazza ugyanazt a lokális műveletet. A szekvenciamodellekben a hasonló induktív torzítás az **időbeli** tengelyre vonatkozik: egy szövegrészlet, hangjel vagy idősor egymást követő tokenjei között feltételezzük, hogy a feldolgozás szabálya időben nem változik. Ebben az alfejezetben az egyszerű rekurrens cella állapotegyenletét, paramétermegosztását, a változó hosszúságú bemenetek paddingjét és a kauzális információs határt vizsgáljuk.

Előismeret: a cross-correlation és az alakok 13.2. Konvolúció és cross-correlation alfejezetből, a hibaszeletek és az értékelési körülmények 23.4. Hibaszeletek és alkalmazási határ alfejezetből, a zárt/nyílt világú inputkezelés 25.4. Nyílt halmaz és ismeretlen input alfejezetből, az alkalmazási szerződés 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés alfejezetből, a látens állapot és a likelihood 31.1. VAE és variációs alsó korlát alfejezetből, valamint az embedding mint reprezentáció 28.1. Embedding, távolság és normalizálás alfejezetből. A 20.1. Feature extractor és új fej alfejezetben látott befagyasztott paraméterek gondolatát itt annyiban használjuk, hogy a rekurrens cella megosztott súlyai egyetlen paramétercsoportot alkotnak.

Induló kérdés: egy batch három különböző hosszúságú szekvenciát tartalmaz. Hogyan ábrázoljuk ezt egyetlen, fix alakú tensorral úgy, hogy a mesterségesen hozzátett üres helyek ne adjanak hamis tanítójelet, és egy adott időpont kimenete ne lássa a jövőbeli tokeneket? Becslés: kell egy explicit hosszlista, egy padding maszk és szükség esetén egy kauzális maszk; a három közül egyik sem pótolja a másikat.

32.1.2 2. Szekvencia, változó hossz és a két maszk

Definíció (véges szekvencia). Egy T hosszúságú szekvencia

$$x_{1:T} = (x_1, \dots, x_T), \qquad x_t \in \mathbb{R}^{d_{in}},$$

ahol t a diszkrét időindex, x_t pedig a t-edik token vektoros reprezentációja. Szöveges adatban a token lehet szó, részcímke vagy karakter beágyazása; hangban spektrális keret; a bemeneti objektumot mindig a feladat szerint választott ábrázolás adja.

A gyakorlati minták hossza általában nem azonos. Legyen a batch elemszáma N, az egyes minták valódi hossza L_i, és

$$T = \max_{i=1,\dots,N} L_i.$$

A batchelt reprezentáció

$$X \in \mathbb{R}^{N \times T \times d_{in}},$$

ahol az i-edik mintánál a $t > L_i$ pozíciók feltöltött, **pad** elemek. A pad értéke leggyakrabban a 0 vektor, de önmagában a nulla érték nem azonosítja a pad pozíciót, mivel valódi token is lehet nulla. Ezért a hosszlistát külön nyilvántartjuk.

Definíció (padding maszk). A padding maszk

$$P \in \{0,1\}^{N \times T}, \qquad P_{it} = \begin{cases} 1, & \text{ha } t > L_i \text{ (pad pozíció)}, \\ 0, & \text{különben.} \end{cases}$$

A veszteségszámításban a pad pozíciókhoz tartozó tagokat ki kell zárni. Itt $t = 1, \dots, T$ 1-indexelt időindex; a kódban a 0-indexelt pozícióra ugyanez a feltétel `pos >= lengths[i]`. Ha a modell minden időlépésre ad egy y_{it} kimenetet és a veszteség elemenkénti, akkor a maszkolt átlag például

$$\mathcal{L} = \frac{\sum_{i=1}^{N} \sum_{t=1}^{T} (1 - P_{it}) \ell_{it}}{\sum_{i=1}^{N} L_i},$$

Itt a nevező az érvényes, nem pad pozíciók száma; ez fontos, mert a változó hosszúság miatt a mintánkénti átlagok egyszerű átlaga nem feltétlenül azonos a tokenenkénti átlaggal. A nevező $\sum_i L_i = \sum_{i,t} (1 - P_{it}) > 0$, és ez különbözik a padelt tokenek $\sum_i (T - L_i)$ számától. Az átlagolási konvenciót a 7.3. Bináris, multilabel és regressziós cél veszteségfüggvényeknél is következetesen rögzítjük.

Definíció (kauzális maszk). Az autoregresszív vagy időben okszerű feldolgozásban az i-edik időlépés kimenete csak a $j \le i$ inputoktól függhet. A kauzális maszk

$$M \in \{0,1\}^{T \times T}, \qquad M_{ij} = \begin{cases} 1, & \text{ha } j > i \text{ (tiltott jövőbeli input)}, \\ 0, & \text{különben.} \end{cases}$$

A maszk tehát a $T \times T$ páronkénti függésmátrix felső háromszögét tiltja a főátló fölött. A padding maszk az üres pozíciókra vonatkozik, a kauzális maszk az időrendre; a kettő nem helyettesíti egymást. Jobbra-paddingnél egy mintán belül a pad pozíciók a sorozat végén vannak, de a batch egy másik mintájában ugyanaz az időindex érvényes, múltbeli elem lehet, és a kauzális maszk a pad nélkül is tilt; ezért általában szükség van mindkettőre.

Intuíció a CNN-hez képest. A 2D konvolúcióban a kernel minden térbeli ablakra ugyanazt a paramétert használja: ez a térbeli eltolási equivarianciához kapcsolódik 16.1. Eltolási equivariance és invariance. A szekvenciamodellben az időbeli súlymegosztás azt fejezi ki, hogy „ugyanaz a szabály dolgozza fel a mondat elejét és végét”. A fontos különbség a **recepciós mező** jellege: a konvolúció véges ablakot lát, a rekurrens cella állapota viszont az egész múlt összefoglalója lehet.

32.1.3 3. Rekurrens cella állapotegyenlete és paramétermegosztása

Definíció (egyszerű rekurrens cella, oszlopvektoros alak). Legyen d_h a rejtett dimenzió, $h_0 \in \mathbb{R}^{d_h}$ a kezdeti állapot (leggyakrabban $h_0 = 0$). A cella paraméterei

$$W_h \in \mathbb{R}^{d_h \times d_h}, \qquad W_x \in \mathbb{R}^{d_h \times d_{in}}, \qquad b \in \mathbb{R}^{d_h}.$$

Egy $f : \mathbb{R} \to \mathbb{R}$ elemenként alkalmazott aktivációval az állapotfrissítés

$$h_t = f(W_h h_{t-1} + W_x x_t + b), \qquad t = 1, \dots, T.$$

A $z_t = W_h h_{t-1} + W_x x_t + b$ pre-aktivációt külön névvel jelöljük, mert a gradiensszámításban szerepel.

A batchelt, N mintát tartalmazó alakban a PyTorch-szerű konvencióval

$$H_t = f(H_{t-1} W_h^\top + X_t W_x^\top + b), \qquad H_t \in \mathbb{R}^{N \times d_h},\ X_t \in \mathbb{R}^{N \times d_{in}}.$$

Itt f elemenként hat a jobb oldali mátrix minden elemére, ugyanaz az aktiváció, mint a vektoralakban.

A bias itt is soronként broadcastolódik, ahogy az affin rétegnél megszoktuk 3.2. Mátrixszorzás és affín réteg.

A definíció lényege, hogy a W_h, W_x, b paraméterek minden t időlépésben **ugyanazok** a változók. A rekurrencia tehát egy

$$\Phi_W(x_t, h_{t-1}) = f(W_h h_{t-1} + W_x x_t + b)$$

függvény ismételt alkalmazása, nem pedig T darab független réteg.

Lemma (időbeli paramétermegosztás gradiense). *Tegyük fel, hogy f elemenkénti, komponensenként differenciálható, és legyen $\mathcal{L}$ skalár veszteség, amely a rejtett állapotokon keresztül függ a cellától. Jelölje*

$$J_t = \operatorname{diag}(f'(z_{t,1}), \dots, f'(z_{t,d_h}))$$

az aktiváció Jacobi-mátrixát a t-edik lépésben. Ekkor a teljes veszteség gradiense a megosztott paraméterekre

$$\nabla_{W_h} L = \sum_{t=1}^{T} (J_t v_t)\, h_{t-1}^\top, \qquad \nabla_{W_x} L = \sum_{t=1}^{T} (J_t v_t)\, x_t^\top, \qquad \nabla_b L = \sum_{t=1}^{T} J_t v_t,$$

ahol a veszteség $\mathcal{L} = \sum_{t=1}^{T} \ell_t(h_t)$ alakú (ha csak a végső állapottól függ, akkor $\ell_t \equiv 0$ minden $t < T$-re), és a vektorok a következők. A **közvetlen** *tag $g_t = \partial \ell_t / \partial h_t \in \mathbb{R}^{d_h}$, vagyis ℓ_t deriváltja h_t szerint úgy, hogy a későbbi állapotokon át ható függést nem számoljuk. A* **teljes** *visszaterjesztett vektor $v_t = \nabla_{h_t} L$ (a jövő lépéseken át érkező hatással együtt) az utolsó lépéstől visszafelé:*

$$v_T = g_T, \qquad v_t = g_t + W_h^\top J_{t+1} v_{t+1} \quad (t = T-1, \dots, 1),$$

és az előaktivációk gradiense $\delta_t = \nabla_{z_t}\mathcal{L} = J_t v_t$. *A fenti képletekben tehát* $J_t v_t = \delta_t$.

Bizonyítás (rövid, a láncszabályból). A Jacobi-mátrix és a láncszabály jelölését a [B01] tankönyv szerinti alakban használjuk. Rögzített t-re a $z_t = W_h h_{t-1} + W_x x_t + b$ kifejezésben a W_h lineárisan szerepel. A mátrix–vektor szorzat deriváltja a bemeneti vektorral vett külső szorzat, ezért a z_t-n keresztül vett parciális hozzájárulás $(J_t v_t) h_{t-1}^\top$. Mivel h_{t-1} maga is függ a korábbi lépések W_h-jától, de ugyanaz a W_h változó szerepel azokban a lépésekben is, a láncszabály szerint a teljes gradiens az egyes t indexekre vonatkozó parciális hozzájárulások összege. A visszaterjesztés az idő mentén (BPTT, backpropagation through time) éppen ennek a láncszabálynak az ismételt alkalmazása az időlépéseken; az eredeti megfogalmazás Werbos (1990). Az összegzés indexhatára $t = 1, \dots, T$; az x_t és b esetén analóg módon járunk el. A v_t rekurziója a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetben bevezetett VJP láncszabály következménye: h_t két úton hat L-re, közvetlenül ℓ_t-n át (g_t) és a $z_{t+1} = W_h h_t + \dots$ következő előaktiváción át, amelynek VJP-je $W_h^\top \delta_{t+1} = W_h^\top J_{t+1} v_{t+1}$; a két hozzájárulás összeadódik. A függőségi gráf irányított, ezért az idő mentén visszafelé haladunk. Az alakellenőrzés: $J_t v_t \in \mathbb{R}^{d_h}$, $h_{t-1}^\top \in \mathbb{R}^{1\times d_h}$, a külső szorzat W_h alakú; $x_t^\top$ esetén $d_h \times d_{in}$ adódik.

Kétlépéses kézi kontroll (nemnulla visszacsatolással). Legyen $d_h = d_{in} = 1$, $f = \tanh$, $W_h = 0{,}5$, $W_x = 1$, $b = 0$, $h_0 = 0{,}2$, $x_1 = 0{,}5$, $x_2 = -0{,}3$, és $\mathcal{L} = \ell_1 + \ell_2 = h_1 + 2h_2$, tehát $g_1 = 1$, $g_2 = 2$. A forward: $z_1 = 0{,}5\cdot 0{,}2 + 0{,}5 = 0{,}6$, $h_1 = \tanh 0{,}6 \approx 0{,}537050$; $z_2 = 0{,}5\cdot 0{,}537050 - 0{,}3 \approx -0{,}031475$, $h_2 \approx -0{,}031465$. A Jacobi-tényezők $J_t = 1 - h_t^2$: $J_1 \approx 0{,}711578$, $J_2 \approx 0{,}999010$. A rekurzió: $v_2 = g_2 = 2$, $\delta_2 = J_2 v_2 \approx 1{,}998020$; $v_1 = g_1 + W_h J_2 v_2 = 1 + 0{,}5 \cdot 1{,}998020 \approx 1{,}999010$, $\delta_1 = J_1 v_1 \approx 1{,}422451$. Így

$$\nabla_{W_h}\mathcal{L} = \delta_2 h_1 + \delta_1 h_0 \approx 1{,}998020\cdot 0{,}537050 + 1{,}422451\cdot 0{,}2 \approx 1{,}357526, \qquad \nabla_{W_x}\mathcal{L} = \delta_2 x_2 + \delta_1 x_1 \approx 0{,}111820.$$

Ha valaki a v_1-ben csak a közvetlen $g_1 = 1$ tagot venné (a jövőből érkező $W_h J_2 v_2$ nélkül), $\nabla_{W_h} L$-re $\approx 1{,}215$-öt kapna. A PyTorch autograd float64-ben ugyanezt a két értéket adja hat tizedesjegyre. Ez a program lefutott (Python 3.12.3, torch 2.15.0+cpu, CPU, float64), a mért kimenet `gWh=1.357526 gWx=0.111820`.

A lemma a **backpropagation through time** elnevezés mögötti pontos számítási szabályt adja. A megosztott paraméter fontos következménye, hogy a modell paraméterszáma **nem** függ a szekvencia hosszától, szemben egy olyan naiv rétegsorozattal, amely minden időlépésre külön affin réteget tanulna.

LSTM-áttekintés. Az egyszerű rekurrens cella gyakorlati problémája, hogy a hosszú időlépéseken át visszaterjedő gradiens szorzataiban a $J_t W_h$ mátrixok ismétlődnek, ami a gradiens eltűnését vagy felrobbanását okozhatja. Az LSTM ezt úgy kezeli, hogy külön sejtállapotot c_t tart fenn, és input-, felejtő- meg kimeneti kapukkal szabályozza, mennyi információ íródjon be, maradjon meg vagy kerüljön a rejtett kimenetbe. Az alábbiakban csak a fogalmi szerkezetet használjuk; a pontos kapuformulák e könyvben nem szerepelnek (az LSTM a 32.1-en kívül nem kap további tárgyalást); a kapuk és a hosszú távú memória mechanizmusának forrása Hochreiter és Schmidhuber (1997).

32.1.4 4. Kézi számpélda

Legyen $d_{in} = d_h = 2$, $h_0 = (0,0)^\top$,

$$W_h = \begin{pmatrix} 0{,}5 & 0 \\ 0 & 0{,}5 \end{pmatrix}, \qquad W_x = I_2, \qquad b = (0,0)^\top,$$

és $f = \tanh$ elemenként. A bemenetek:

$$x_1 = (0{,}5, 0)^\top, \qquad x_2 = (0,\ 0{,}5)^\top, \qquad x_3 = (0, 0)^\top.$$

A rekurzív számolás négy tizedesjegyre kerekítve:

$$z_1 = x_1, \qquad h_1 = \tanh(x_1) \approx (0{,}4621,\ 0)^\top,$$

$$z_2 = 0{,}5h_1 + x_2 \approx (0{,}2311,\ 0{,}5000)^\top, \qquad h_2 \approx (0{,}2270,\ 0{,}4621)^\top,$$

$$z_3 = 0{,}5h_2 \approx (0{,}1135,\ 0{,}2311)^\top, \qquad h_3 \approx (0{,}1130,\ 0{,}2270)^\top.$$

Két tulajdonság látható: az x_1 hatása a h_3-ban már csak $\approx 0{,}1130$ nagyságrendű, tehát a rekurrens állapot exponenciálisan halványíthatja a régi bemeneteket; másrészt a $t = 2$ lépésben a modell csak x_1 és x_2 ismeretében számolt, a $t = 3$ bemenetet nem látta. Az alábbi minikód ellenőrzi ugyanezt:

```
import torch

W_h = 0.5 * torch.eye(2, dtype=torch.float32)
W_x = torch.eye(2, dtype=torch.float32)
b = torch.zeros(2, dtype=torch.float32)
x = torch.tensor([[0.5, 0.0], [0.0, 0.5], [0.0, 0.0]],
                 dtype=torch.float32)
h = torch.zeros(2, dtype=torch.float32)
print(f"{'t':>2} {'h_t':>20}")
print(f"{0:>2} {[round(float(v), 4) for v in h]}")
for t, x_t in enumerate(x, start=1):
    h = torch.tanh(h @ W_h.T + x_t @ W_x.T + b)
    print(f"{t:>2} {[round(float(v), 4) for v in h]}")
```

A kód a lemmát egyetlen szekvencián, $N = 1$ mellett futtatja: a h vektor alakja $(2,)$, ami a batchelt $H_t \in \mathbb{R}^{N \times d_h}$ alak $N = 1$ esete; a transzponált affin konvenció $(hW_h^\top)$ ezen is ugyanazt adja. A fenti számokkal a kerekített egyezés várható, de a `tanh` implementációja miatt az utolsó tizedesjegyekben eltérés lehet, ezért `1e-3` abszolút tolerancia az ésszerű ellenőrzési határ.

32.1.5 5. Labor: padding és kauzális információs határ

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja, hogy egyetlen minibatchben lássuk a padding maszk és a kauzális maszk különböző szerepét. A kód nem követeli meg a teljes tanítást, csak a két maszk konstrukcióját és egy ellenőrző mérést.

```
import torch

torch.manual_seed(0)

# Három változó hosszúságú minta, T=4, d_in=3.
# Az első 4, a második 3, a harmadik 1 érvényes token.
lengths = torch.tensor([4, 3, 1])
X = torch.randn(3, 4, 3, dtype=torch.float32)
```

```python
# Padding maszk 0-indexelt pozícióval: P[i,t]=1, ha t >= lengths[i].
# (1-indexelt időindexre ez a t > L_i feltételnek felel meg.)
T = X.size(1)
pad_mask = torch.arange(T)[None, :] >= lengths[:, None]

# Kauzális maszk: M[i,j]=1, ha j > i (a jövő tiltása).
causal_mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)

# Ellenőrzés: pad maszk érvényes elemszáma egyenlő a hosszlista összegével.
assert int((~pad_mask).sum()) == int(lengths.sum())

# Causal maszk: főátló és alsó háromszög engedélyezett.
allowed = ~causal_mask
assert torch.equal(allowed, torch.tril(torch.ones(T, T, dtype=torch.bool)))

print("pad_mask =")
print(pad_mask.int())
print("causal_mask (tiltott = 1) =")
print(causal_mask.int())
```

Mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 0):

```
pad_mask =
tensor([[0, 0, 0, 0],
        [0, 0, 0, 1],
        [0, 1, 1, 1]], dtype=torch.int32)
causal_mask (tiltott = 1) =
tensor([[0, 1, 1, 1],
        [0, 0, 1, 1],
        [0, 0, 0, 1],
        [0, 0, 0, 0]])
```

Az ellenőrzési pont: az első mintának nincs pad eleme, a harmadiknak a $2, 3, 4$ időindexű elemei (1-indexelt időindex) mind pad pozíciók. A kauzális maszk első sora a $2, 3, 4$ pozíciót tiltja, mert az első időlépés kimenete csak az első inputtól függhet. Ha egy modell minden időlépésben ugyanazt az inputot kapná, a padding maszk önmagában nem tiltaná a jövőt; a kauzális maszkra akkor is szükség van, ha az összes pozíció érvényes, tehát a kettő nem ugyanazt a hibát védi ki.

Futtatási utasítás: a fenti kódot a `lab/n125/pad_causal_demo.py` fájlba téve a

```
python lab/n125/pad_causal_demo.py
```

parancs futtatja. A `torch.manual_seed(0)` miatt a véletlen bemenet reprodukálható CPU-n, de más PyTorch-verzió esetén a generált számok nem garantáltan bitazonosak; a két `assert` ekkor is ellenőrzi a maszkok szerkezetét.

32.1.6 6. Feladatok

F1 (fogalmi, nehézség 1). Adjon olyan konkrét $N = 2$, $T = 3$ példát, amelyben a padding maszk és a kauzális maszk különböző pozíciókat tilt, és magyarázza meg, miért nem elegendő önmagában a padding maszk egy autoregresszív modell tanításához.

F2 (levezetési, nehézség 2). Tekintsünk egy **lineáris** rekurrens cellát $f(z) = z$, $W_x = I$, $b = 0$, $h_0 = 0$ és skalár paraméterrel $W_h = \alpha I$, ahol $\alpha \in \mathbb{R}$. Vezesse le a h_T zárt alakját $x_{1:T}$ függvényében, és adja meg pontosan, mely α értékekre korlátos a $T \to \infty$ limesz tetszőleges, egyenletesen korlátos bemeneti sorozatra. Ellenőrizze az eredményt a lemma gradienseinek speciális esetére.

F3 (implementációs, nehézség 2). Írjon olyan Python-függvényt, amely egy változó hosszúságú batch `X` és `lengths` tenorból kiszámítja a `pad_mask`-et, majd egy adott `loss_per_step` azonos alakú tensoron a maszkolt átlagot. A függvény térjen vissza az átlaggal és az érvényes elemszámmal is. Teszteset: `X` alakja `(3,4,3)`, `lengths = [4,3,1]`, `loss_per_step` legyen csupa egy; várt érvényes elemszám $4 + 3 + 1 = 8$, várt átlag 1.

F4 (hibakeresési, nehézség 2). Egy fejlesztő a következőt írja: a paddingnek a veszteségben való kihagyásához a pad pozícióknál nullázza a rejtett állapotot a rekurrens lépés után, de a maszkot nem használja a veszteségátlagban. Lokalizálja a két hibát, indokolja a tanulási következményt, és adjon minimális javítást a fenti labor kódjához.

F5 (kísérlettervezési/transzfer, nehézség 3). Egy osztályozó modell bemenetként változó hosszúságú, legfeljebb $T = 64$ hosszú szekvenciákat kap. Tervezzen kísérletet annak eldöntésére, hogy a $T = 64$-re való padding vagy a $T = 32$-re való csonkolás ad-e jobb validációs pontosságot adott számítási keret mellett. Adja meg a protokollt: adatforrás, megfigyelési egység, split, baseline, változó tényező, metrika, validációs választás és a következtetés érvényességi határát. A feladat nyitott, egy lehetséges terv ismertetése elfogadható.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

32.1.7 7. Megoldások és indoklás

F1. Legyen $N = 2$, $L = (3, 2)$, így $T = 3$. A padding maszk csak a második minta harmadik pozícióját tiltja: $P = (0, 0, 0; 0, 0, 1)$. A kauzális maszk ezzel szemben a $(1, 2), (1, 3), (2, 3)$ (query-idő, key-idő) párokra 1: a kimenet $t = 2$ időpontban nem láthatja a $t = 3$ **jövőbeli bemenetet**, akkor sem, ha mindkét minta minden pozíciója érvényes. A padding maszk csak az üres pozíciókat kezeli; ha a modell például attention-mechanizmust (figyelmi mechanizmus) számol, akkor egy érvényes x_2 token jövőbeli x_3 tokenre is figyelhetne, ami tanításban magas pontosságot, de az autoregresszív használatban összeomlást ad. A két maszk együtt használandó: a query (i, t) pozíció a key (i, j) pozíciót akkor láthatja, ha $(1 - P_{it})(1 - P_{ij})(1 - M_{tj}) = 1$, azaz a query és a key pad-maszkja is enged, és a kauzális maszk sem tilt.

F2. A lineáris rekurzió $h_t = \alpha h_{t-1} + x_t$. Teljes indukcióval:

$$h_T = \sum_{s=1}^{T} \alpha^{T-s} x_s.$$

Ez a konvolúciós forma is: egy geometriailag csillapodó, időben megosztott szűrő. Az alakellenőrzés: $h_T \in \mathbb{R}^{d_h}$, minden tag ugyanakkora dimenziójú. Ha $\|x_s\|_2 \le C$ minden s-re, akkor

$$\|h_T\|_2 \le C \sum_{s=1}^{T} |\alpha|^{T-s}.$$

Minden rögzített véges T-re a jobb oldali összeg véges, ezért $\|h_T\|_2$ véges. A sorozat egyenletesen korlátos minden T-re és minden olyan bemeneti sorozatra, amelyre $\|x_s\|_2 \le C$ minden s-re, pontosan akkor, ha $|\alpha| < 1$; ekkor $\sup_T \|h_T\|_2 \le C/(1 - |\alpha|)$. Ha $|\alpha| > 1$, a konstans $x_s = x \neq 0$ bemenetre $\|h_T\|_2$ exponenciálisan nő. Ha $|\alpha| = 1$, az $\alpha = 1$ esetben konstans bemenetre $h_T = \sum_{s \le T} x_s$, ami lineárisan nő; $\alpha = -1$ esetén az előjelesen váltakozó $x_s = (-1)^s x$ bemenetre szintén lineárisan nő, konstans bemenetre pedig nem konvergál. A lemma speciális esetének ellenőrzése: $f(z) = z$ mellett $J_t = I$ és $W_h = \alpha I$, ezért a lemma $\nabla_{W_h} \mathcal{L} = \sum_t (J_t v_t) h_{t-1}^\top = \sum_t v_t h_{t-1}^\top$ alakja a

$\partial L/\partial \alpha = \sum_t v_t^\top h_{t-1}$ skalárderivált adja, mert $\partial W_h/\partial \alpha = I$; a visszaterjesztett vektor rekurziója itt $v_t = g_t + \alpha v_{t+1}$. A két számítás tehát ugyanaz, ami a lemma konzisztenciáját ellenőrzi.

F3. Minimális, futtatható megoldás:

```
import torch

def masked_mean_loss(X, lengths, loss_per_step):
    """
    X: (N,T,d) tensor, lengths: int64 tensor, shape (N,), minden elem 0..T
    loss_per_step: float32 tensor, shape (N,T)
    Kiszámítja a pad_mask-et, majd visszaadja a nem pad elemek átlagát és
    az érvényes elemszámot.
    """
    N, T, _d = X.shape
    assert loss_per_step.shape == (N, T)
    assert int(lengths.min()) >= 0 and int(lengths.max()) <= T
    pad_mask = torch.arange(T)[None, :] >= lengths[:, None]
    valid = ~pad_mask
    denom = int(valid.sum())
    if denom == 0:
        raise ValueError("üres batch: nincs érvényes pozíció")
    return float(loss_per_step[valid].sum() / denom), denom

# Teszt
X = torch.randn(3, 4, 3, dtype=torch.float32)
loss = torch.ones(3, 4, dtype=torch.float32)
lengths = torch.tensor([4, 3, 1], dtype=torch.int64)
mean, count = masked_mean_loss(X, lengths, loss)
print(mean, count)  # 1.0, 8
assert mean == 1.0 and count == 8
```

Szélsőeset: ha egy minta hossza 0, minden pozíciója pad, ezért a függvény azt a mintát kihagyja; ha minden hossz 0, az üres batch hibát dob, mert az átlag nem definiált. Az `assert float` egyenlőséget használ, ami itt a csupa egyes mátrixon pontos; más adatoknál toleranciát kell adni.

F4. Az első hiba: a rejtett állapot nullázása **utólag** nem szünteti meg, hogy a rekurrens lépésben a pad token már beírta a hatását a megelőző lépésekből; ráadásul ha a nullázott állapot a következő lépés bemeneteként szolgál, az állapotdinamika torzul. A második hiba: a veszteségátlagban a pad pozíciók továbbra is szerepelnek, ezért a nevező $N \cdot T$ lesz az érvényes elemszám helyett, és a modell a pad tokenekre is illeszteni próbál. Következmény: a veszteség hamis jelzést ad, mert a pad pozíciók tagjai is benne vannak, és a jelentett érték a helyes tokenenkénti átlag $\sum_i L_i/(NT)$-szeresére skálázódik; a hosszú minták irányába mutató torzítás nem következik, mert a tokenenkénti és a hibás átlag is a tokenek számával arányosan súlyozza a mintákat, a különbség a pad-tagok jelenléte és a globális skála. Harmadik, rejtett hiba: jobbra-paddelt batchben az utolsó valódi token állapota h_{L_i}, nem h_T; ha a modell a végső h_T-t használja reprezentációként, a nullázás után az a rövid mintáknál nulla, nullázás nélkül pedig a pad tokenek hatását tartalmazza. A helyes reprezentáció ezért a h_{L_i} (például `h[torch.arange(N), lengths-1]`), a pad-lépések állapotát pedig el kell dobni. Javítás: a rekurrens lépés bemeneti pozíciójában a pad tokeneket ne etessük be, vagy ha az RNN implementáció minden lépést számol, akkor a veszteségben alkalmazzuk a `pad_mask`-et, az F3 megoldásában szereplő `masked_mean_loss`-höz hasonlóan. Visszaellenőrzés: érvényes elemszámnak 8-nak kell lennie a tesztesetben, nem 12-nek.

F5. Egy lehetséges terv: készítünk egy procedurális, 64-nél nem hosszabb szekvenciákat generáló adatforrást; minden adatpont egy független szekvencia, a generálási folyamat rögzített seeddel. A split train/val/test például 600/100/100, a testet csak a végén használjuk. A baseline legyen a többségi osztályt jósoló osztályozó, hogy a magas osztályarány ne adjon hamis képet. A

változó tényező a bemenet-előkészítés: (A) pad $T = 64$-re; (B) csonkol $T = 32$-re. Ugyanazt a modellvázat (például egy rekurrens cella vagy kauzális konvolúció utolsó állapotára illesztett lineáris fej) mindkét beállítással tanítjuk, azonos loss, optimizer, seed és epochszám mellett. A kiválasztás metrikája a validációs accuracy; a seedek több ismétlésével (például 10) a különbségre adunk szórást is. Az érvényességi korlát: az eredmény csak a saját szekvencia-eloszlásra és hossztartományra vonatkozik; a $T = 32$-re csonkolásnál a hosszú minták elveszett farokinformációja miatt a pontosságromlás az adott tartalomtól függ, ezért a következtetést nem szabad általános csonkolási szabályként kimondani. A számítási költség mérésekor a futási időt is rögzíteni kell, mert a rövidebb bemenet nemcsak pontosabb vagy pontatlanabb, hanem gyorsabb is lehet.

32.1.8 8. Tipikus tévedés ellenpéldával, önellenőrzés, visszahívás

A leggyakoribb tévedés, hogy a tanításkor elérhető teljes szekvenciát automatikusan az inferencia-helyzettel azonosnak tekintjük. Ez nemcsak a padding mask hiányakor, hanem akkor is hibás, ha a modell a t-edik kimenethez a t utáni tokeneket is bemenetként kapja. **Ellenpélda:** legyen egy fordítói vagy következő token predikciós modell, amely tanításkor a x_3 inputot is felhasználja a $t = 2$ kimenet előállításához. A tanítási veszteség kicsi lehet, mert a cél token gyakran megegyezik a következő bemeneti tokennel. Az éles használatban azonban x_3 még nem áll rendelkezésre a $t = 2$ döntésnél, így a modellt olyan bemenetre kényszerítjük, amelyet soha nem látott. A megoldás a kauzális maszk: a t-edik kimenetnek kizárólag $j \leq t$ bemeneteken szabad múlnia; a padding maszk önmagában ezt nem garantálja. A helyes kísérleti napló ezért külön rögzíti a tanítási és az autoregresszív kiértékelési módot 12.1. Tanítás, validáció és kiértékelési mód.

Önellenőrző kérdések: (1) Meg tudom-e mondani a P és az M maszk pontos alakját? (2) A lemma összegzésében miért $t = 1$-től T-ig összegzünk, és miért nem ugyanaz, mint egyetlen hosszú affin lánc gradiense? (3) Mi a zero padding szerepe a CNN-ben 14.1. Stride, padding, dilation és outputméret, és miért nem azonos a szekvenciális padding maszkkal?

A (2) rövid válasza: a W_h, W_x, b paraméterek minden időlépésben közösek, de az állapot minden lépésben más lineáris kombinációja a bemenetnek, ezért a gradiens az időlépések járulékainak összege; egyetlen hosszú affin lánc gradiense nem tartalmazná a lépésenkénti külső szorzatokat és a visszaterjesztett v_t különbségeit.

Válaszkulcs (1) és (3). (1) A padding maszk $P \in \{0,1\}^{N \times T}$: mintánként egy sor, a pad pozíciókon 1; a kauzális maszk $M \in \{0,1\}^{T \times T}$: a főátló fölötti jövőbeli key-pozíciókon 1. (3) A CNN zero paddingja a konvolúció szélein képzett nulla értékű térbeli keret, amelyet a konvolúció számítása használ; a szekvenciális padding maszk ezzel szemben a mesterségesen beszúrt időpozíciókat zárja ki a lossból és a figyelemből, ezért a kétfajta padding szerepe nem azonos.

Visszahívás: a 31.1. VAE és variációs alsó korlát alfejezetben bevezetett $q_\phi(z|x)$ látens kódolóval szemben a rekurrens h_t **determinisztikus időbeli** állapot, nem valószínűségi posterior. A 28.1. Embedding, távolság és normalizálás alfejezetben az embedding egyetlen input statikus reprezentációja volt; itt a rejtett állapot az inputok sorozatától függő, kontextusfüggő reprezentáció. A 20.1. Feature extractor és új fej alfejezetben a befagyasztott backbone paramétereihez hasonlóan a rekurrens cellában is egy paramétercsoportként kezeljük a megosztott W_h, W_x, b hármast.

32.1.9 Rövid összefoglaló

A szekvenciamodell a változó hosszúságú token-sorozatokat közös időtengelyen ábrázolja. A padding maszk az érvénytelen pozíciókat, a kauzális maszk a jövőbeli információs csatornát zárja le; a kettő nem helyettesíthető. Az egyszerű rekurrens cella ugyanazt a W_h, W_x, b paramétert

használja minden időlépésben; a lemmában megmutattuk, hogy a gradiensük az időlépésenkénti külső szorzatok összege. Az LSTM kapukkal vezérli a hosszú távú állapotot; a temporális konvolúció ezzel szemben véges recepciós mezőjű, nem rekurrens művelet. A kézi példa a rekurrens állapotfrissítést és az időbeli súlymegosztás hatását, a labor kódja a padding- és kauzális maszk szerkezetét ellenőrzi.

Az induló kérdésre a válasz: a fix alakú tensorhoz padding, a hamis tanítójel ellen padding maszk, a jövőbeli tokenek ellen kauzális maszk kell; a becslés helyes, a három eszköz egyike sem pótolja a másikat.

32.1.10 Következő alkalmazás

A 32.2. Scaled dot-product attention egységben a rekurrenciát elhagyjuk, és a scaled dot-product attention operátort vezetjük be [P21]. A most definiált kauzális és padding maszk ott közvetlenül a figyelmi súlymátrixra kerül; az időbeli súlymegosztás helyébe a pozíciófüggetlen, de pozíciókódolást igénylő attention lép.

32.1.11 Felhasznált források

- [P21] Vaswani, A. et al. (2017). *Attention Is All You Need.* arXiv:1706.03762. https://arxiv.org/abs/1706.03762. A scaled dot-product attention hivatkozása a következő egységhez.
- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning.* Cambridge University Press. https://mml-book.github.io/. A Jacobi-mátrix és a láncszabály jelölését a könyvben megadott alakban használjuk.
- Hochreiter, S. és Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735. Az LSTM hivatkozása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.
- Werbos, P. J. (1990). *Backpropagation through time: what it does and how to do it.* Proceedings of the IEEE 78(10), 1550–1560. https://doi.org/10.1109/5.58337. A BPTT hivatkozása; a fejezet forráskatalógusán kívüli, a szövegben használt hivatkozás.

32.2 Scaled dot-product attention

32.2.1 Cél, előismeret és induló kérdés

Ez az egység a szekvenciák feletti figyelmi (attention) mechanizmus egyetlen, alakpontos mátrixműveletét építi fel. A 32.1. Szekvenciamodell és változó hossz egységből ismerjük a szekvenciát mint tokenek rendezett sorozatát, a padding maszk szerepét, valamint azt, hogy egy rekurrens cella állapotegyenlete ugyanazt a paramétervektort alkalmazza minden időlépésben. Most azt a kérdést vizsgáljuk: hogyan kaphat egy token olyan kontextusfüggő reprezentációt, amely a többi token reprezentációjának adatfüggő, tanult súlyozású kombinációja? Induló becslésként: ha T tokenünk van és minden token d dimenziós vektorként adott, akkor a figyelem matematikailag egy $T \times T$ méretű pontszámtáblázatból indul ki, és a naiv implementáció ezt a táblázatot teljes egészében el is készíti; a kérdés az, hogyan lesz ebből a táblázatból értelmezhető, maszkolható és numerikusan stabil kimenet, és mi ennek a számítási és memóriaköltsége.

32.2.2 Q, K, V és a figyelmi mátrix definíciója

Legyen a bemeneti szekvencia reprezentációja $X \in \mathbb{R}^{T \times d_{model}}$, ahol T a tokenek száma, d_{model} pedig a beágyazási dimenzió; a mátrixjelölést a [B01] tankönyv szerinti alakban használjuk. A vetített lekérdezés-, kulcs- és értékmátrixokat a

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

lineáris vetítések adják, ahol

$$W^Q \in \mathbb{R}^{d_{model} \times d_k}, \quad W^K \in \mathbb{R}^{d_{model} \times d_k}, \quad W^V \in \mathbb{R}^{d_{model} \times d_v}.$$

Így $Q, K \in \mathbb{R}^{T \times d_k}$ és $V \in \mathbb{R}^{T \times d_v}$. A Q i-edik sora az i-edik token lekérdezése (query), a K j-edik sora a j-edik token kulcsa (key), a V j-edik sora pedig a j-edik token értéke (value). A tanulható paraméterek itt a három vetítőmátrix; a figyelem maga az alábbi determinisztikus, paraméter nélküli művelet.

Definíció (scaled dot-product attention). Adott $Q \in \mathbb{R}^{T_q \times d_k}$, $K \in \mathbb{R}^{T_v \times d_k}$, $V \in \mathbb{R}^{T_v \times d_v}$ esetén a figyelmi kimenet

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

ahol a softmax a mátrix minden sorára külön alkalmazott, soronkénti normalizálás. Az

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T_q \times T_v}$$

mátrixot figyelmi súlymátrixnak nevezzük. A definíció és a belőle származtatott multi-head változat a [P21] cikkben szerepel ebben az alakban. A definíció érvényes akkor is, ha $T_q \neq T_v$; önfigyelem (self-attention) esetén $T_q = T_v = T$. A $d_k = d_v$ gyakori, de nem kötelező választás.

Az O mátrix i-edik sora az értékvektorok konvex kombinációja:

$$o_i = \sum_{j=1}^{T_v} A_{ij} v_j,$$

ahol $A_{ij} \geq 0$ és $\sum_j A_{ij} = 1$. A figyelem tehát nem egy rögzített szűrő, hanem a bemenettől függő súlyrendszer: ugyanaz a hálózat más tokenek esetén más $QK^\top$ pontszámokat és ezáltal más A mátrixot állít elő.

Amennyiben batchben dolgozunk, minden mennyiséghez egy N méretű batch-dimenzió járul, és a képletek a következőképpen értendők: $Q \in \mathbb{R}^{N \times T_q \times d_k}$ és így tovább; a softmax változatlanul az utolsó, tehát a T_v dimenzióra normalizál. Ebben az egységben a levezetéseket egyetlen szekvenciára, tehát 2 dimenziós mátrixokra írjuk fel, mert a batch-dimenzió a műveleteken csak párhuzamosan halad végig.

32.2.3 A $QK^\top$ pontszámmátrix levezetése

Az első levezetési kötelezettség a pontszámmátrix alakjának pontos meghatározása. A mátrixszorzás definíciójával:

Lemma 1 (a $QK^\top$ mátrix alakja és elemei). Ha $Q \in \mathbb{R}^{T_q \times d_k}$ és $K \in \mathbb{R}^{T_v \times d_k}$, akkor $S = QK^\top \in \mathbb{R}^{T_q \times T_v}$, és

$$S_{ij} = \sum_{r=1}^{d_k} Q_{ir} K_{jr}.$$

Bizonyítás. A transzponált definíciója szerint $[K^\top]_{rj} = K_{jr}$. A két mátrix szorzatának eleme az első mátrix i-edik sorának és a második mátrix j-edik oszlopának skalárszorzata, tehát

$$S_{ij} = \sum_{r=1}^{d_k} Q_{ir} [K^\top]_{rj} = \sum_{r=1}^{d_k} Q_{ir} K_{jr}.$$

A dimenzióellenőrzés: a szorzat sorainak száma T_q, oszlopainak száma a $K^\top$ oszlopainak száma, azaz T_v. □

A pontszám értelmezése: S_{ij} annak a mértéke, hogy a j-edik kulcs mennyire illik az i-edik lekérdezéshez. A $\sqrt{d_k}$-val való osztás a skálázás, amelynek motivációja a következő heurisztika. Ha Q_{ir} és K_{jr} független, nulla várható értékű, egységvarianciájú valószínűségi változók, akkor az 5.2. Várható érték, variancia és likelihood alfejezetben bevezetett varianciaazonosságokkal

$$\mathbb{E}\left[\sum_{r=1}^{d_k} Q_{ir} K_{jr}\right] = 0, \qquad \mathrm{Var}\left[\sum_{r=1}^{d_k} Q_{ir} K_{jr}\right] = d_k.$$

A $\sqrt{d_k}$ osztó tehát a pontszámok varianciáját a feltételezett modellben közel egységnyire hozza. Ez azonban heurisztika, nem általános állítás: a betanult Q és K sorai sem nem függetlenek, sem nem azonos eloszlásúak. A skálázás szükségességét ezért validációval kell eldönteni, és nem bizonyított tételként bemutatni.

32.2.4 A softmax és a tiltott pozíciók maszkolási szabálya

A softmax soronkénti normalizálás:

$$A_{ij} = \frac{\exp(S_{ij})}{\sum_{j'=1}^{T_v} \exp(S_{ij'})}.$$

A figyelmi gyakorlatban bizonyos j kulcspozíciókat ki kell zárni. Két gyakori eset:

- **Padding mask:** ha j egy kitöltő token, arra egyetlen query sem figyelhet.
- **Causal mask:** a generatív balról jobbra haladó modellben az i-edik query csak a $j \le i$ kulcsokra figyelhet.

A maszkot logikai mátrixként adjuk meg: $M_{ij} = 1$ akkor és csak akkor, ha a j kulcs az i sorgal tiltott. A tiltás szabálya a pre-softmax logitra, tehát magára a S mátrixra alkalmazandó.

Lemma 2 (tiltott pozíciók maszkolási szabálya). Legyen $S \in \mathbb{R}^{T_q \times T_v}$ véges értékű pontszámmátrix, és minden i sorra legyen $\mathcal{M}_i \subset \{1, \dots, T_v\}$ a tiltott kulcspozíciók halmaza. Tegyük fel, hogy $\mathcal{M}_i \neq \{1, \dots, T_v\}$ minden i-re, azaz egyetlen query sem veszíti el valamennyi kulcsát. Definiáljuk

$$\tilde{S}_{ij} = \begin{cases} S_{ij}, & j \notin \mathcal{M}_i, \\ -\infty, & j \in \mathcal{M}_i. \end{cases}$$

Ekkor az $A = \mathrm{softmax}(\tilde{S})$ mátrix jól definiált, és minden tiltott $j \in \mathcal{M}_i$ esetén $A_{ij} = 0$.

Bizonyítás. A feltétel szerint minden i sorban van legalább egy nem tiltott j, ahol $\tilde{S}_{ij} = S_{ij}$ véges. Így $\exp(\tilde{S}_{ij}) > 0$, tehát a nevező

$$\sum_{j'=1}^{T_v} \exp(\tilde{S}_{ij'}) \geq \exp(S_{ij}) > 0,$$

vagyis a softmax értéke véges. Tiltott j-re a számláló $\exp(-\infty) = 0$, tehát $A_{ij} = 0$. □

A lemma kimondja, hogy a $-\infty$ behelyettesítése a logitban pontosan zérus figyelmi súlyt eredményez a tiltott pozíciókon. A maszkot tehát a softmax előtt kell alkalmazni; a softmax utáni utólagos nullázás nem ad érvényes súlymátrixot, amint ezt az F4 feladatban ellenőrizzük.

A numerikus számításhoz kapcsolódó második állítás a softmax shift-invarianciája.

Állítás (shift-invariancia). Minden soronkénti $c_i \in \mathbb{R}$ eltolás mellett

$$\mathrm{softmax}(S + c_i \mathbf{1}) = \mathrm{softmax}(S).$$

Bizonyítás. Soronként:

$$\frac{\exp(S_{ij} + c_i)}{\sum_{j'} \exp(S_{ij'} + c_i)} = \frac{\exp(S_{ij}) e^{c_i}}{e^{c_i} \sum_{j'} \exp(S_{ij'})} = \frac{\exp(S_{ij})}{\sum_{j'} \exp(S_{ij'})}.$$

A e^{c_i} közös tényezővel egyszerűsítünk. □

A stabil implementáció ezért kivonja a soronkénti maximumot, hogy az exponenciális argumentumát legfeljebb nullára csökkentse. A maszkolt esetben a maximumot a **nem maszkolt** elemekre kell számítani; ha a teljes sor tiltott, akkor a maximum $-\infty$, és ezt hibaként kell jelezni. Megfelelő maszk mellett a nem maszkolt elemek maximuma véges, így a tiltott elemek $\tilde{S}_{ij} = -\infty$ esetén $\exp(\tilde{S}_{ij} - \max) = 0$-t adnak, tehát a stabilizált képlet is a Lemma 2 szerinti súlyokat szolgáltatja.

Számítási és memóriaköltség. A két költséget nem szabad azonosítani. A pontszámok kiszámítása $T_q \cdot T_v \cdot d_k$ szorzás-összeadás, a súlyozott összeg $T_q \cdot T_v \cdot d_v$; ez a **számítási** költség önattenciónál ($T_q = T_v = T$) $O(T^2 d)$, és egzakt figyelemnél nem kerülhető meg. A **memóriaköltség** viszont a megvalósítástól függ. A naiv implementáció a teljes $T \times T$ pontszám- és súlymátrixot materializálja, ezért $O(T^2)$ köztes tárat igényel. A teljes figyelmi mátrix hasznos matematikai reprezentáció, de nem kötelező egyszerre a memóriában tartani: a kimenet pontosan kiszámítható blokkokban is, ha a softmax normalizálását **online** (egymenetes, blokkonként

frissített sormaximummal és sorösszeggel végzett) módon végezzük. A FlashAttention ezt a blokkosítást (tiling) használja; a cikk 1. tétele szerint egzakt $\mathrm{softmax}(QK^\top)V$-t ad $O(T^2 d)$ művelettel és a bemeneten és kimeneten túl csak $O(T)$ további memóriával [P29]. A tétel a skálázás nélküli $\mathrm{softmax}(QK^\top)V$ alakra vonatkozik; a $\sqrt{d_k}$ osztó a jelen jelölésben ettől külön lépés.

32.2.5 Multi-head és pozíciókódolás röviden

A **multi-head attention** [P21] a d_{model} méretű reprezentációt h párhuzamos fejre osztja. Fejenként

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V,$$

(a $W_i^Q \in \mathbb{R}^{d_{model}\times d_k}$ alak a *sorbatches* (sorok szerinti kötegelés) konvenciót követi, $X \in \mathbb{R}^{T\times d_{model}}$ soronként a tokenek; ez eltér a korábbi $W_{out,in}$ konvenciótól, ahol $z = Wx$ oszlopvektorra hatott: itt XW a sorvektorokra hat. A váltás a sorbatches projekció miatt van, és explicit.)

ahol a fejek vetítéseinek kimeneti dimenziói tipikusan $d_k = d_v = d_{model}/h$. Minden fej önállóan kiszámítja az O_i-t, a fejkimeneteket a feature-tengely mentén fűzzük össze (`concat(..., dim=-1)`; az alak $T \times (h\,d_v)$), majd egy közös kimeneti vetítést alkalmazunk:

$$O = [\,O_1 \;\cdots\; O_h\,]\,W^O, \qquad W^O \in \mathbb{R}^{hd_v\times d_{model}}.$$

A multi-head célja heurisztikus: különböző fejek különböző hasonlósági altérben dolgozhatnak. Ez a definíció nem állítja, hogy minden architektúrában h fej szükséges, és a fejek valódi eltérése csak illusztratív vizsgálattal ellenőrizhető.

A **pozíciókódolás** a figyelem permutáció-szimmetriáját töri meg. A fenti definícióban a tokenek sorrendje kizárólag a Q, K, V sorainak sorrendjén keresztül jelenik meg, és a $QK^\top$ pontszám kizárólag a sorok tartalmától függ, a pozíciótól nem. Ezért a sorrend megváltoztatása a kimenet sorrendjét ugyanúgy megváltoztatja, de a tokenekhez nem ad új információt. A pozíciókódolás a beágyazáshoz ad egy pozíciófüggő vektort: $\tilde{X}_t = X_t + p_t$. A p_t lehet tanult paraméter, vagy rögzített, például szinuszos séma [P21]. A gyakori szinuszos alak a pos pozícióindexre páros $2r$ index esetén $\sin(pos/10000^{2r/d_{model}})$, páratlan $2r+1$ index esetén $\cos(pos/10000^{2r/d_{model}})$ tagot használ; a pontos formula architektúra-választás, nem egyedüli lehetőség.

32.2.6 Kézi kéttokenes példa

Legyen $d_k = 2$, $d_v = 3$, $T = 2$, és

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 2 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

A Lemma 1 szerint

$$S = \frac{QK^\top}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 2 \\ 1 & 0 \end{pmatrix}.$$

A $\sqrt{2} \approx 1{,}41421356$, így az első sorban $S_{11} \approx 0{,}70710678$, $S_{12} \approx 1{,}41421356$, a második sorban $S_{21} \approx 0{,}70710678$, $S_{22} = 0$. A softmax exponenciálisai:

- első sor: $e^{0,7071} \approx 2{,}0281$, $e^{1,4142} \approx 4{,}1133$, összeg 6,1414, tehát $A_{11} \approx 0{,}3302$, $A_{12} \approx 0{,}6698$;
- második sor: $e^{0,7071} \approx 2{,}0281$, $e^{0} = 1$, összeg 3,0281, tehát $A_{21} \approx 0{,}6698$, $A_{22} \approx 0{,}3302$.

A kimenet

$$O = AV \approx \begin{pmatrix} 0{,}3302 & 0{,}6698 & 0 \\ 0{,}6698 & 0{,}3302 & 0 \end{pmatrix},$$

mert V harmadik oszlopa nulla.

Most alkalmazzunk **causal maszkot** (kauzális, oksági maszkot): az első query, tehát az első token nem figyelhet a másodikra, így $M_{12} = 1$, minden más elem zérus. A Lemma 2 szerint $\tilde{S}_{12} = -\infty$, tehát az első sor egyetlen megmaradó súlya $A_{11} = 1$, $A_{12} = 0$. A második sor változatlan. A maszkolt kimenet

$$O_m \approx \begin{pmatrix} 1 & 0 & 0 \\ 0{,}6698 & 0{,}3302 & 0 \end{pmatrix}.$$

Ez a két érték szolgál referenciaként a laborban.

32.2.7 Labor: kéttokenes NumPy-implementáció

Előfeltételek. Python 3.12.3, NumPy 2.5.4, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja a Lemma 1, a stabil softmax és a Lemma 2 együttes ellenőrzése kis, teljesen megadott mátrixokon. Munkakönyvtár: `lab/n126/`.

```
# lab/n126/n126_attention.py
import numpy as np

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    # swapaxes: batch-dimenziós (N,T,d) bemenetnél is a helyes tengelyt
    # transzponálja; 2D bemenetnél ez megegyezik a K.T-vel.
    S = Q @ np.swapaxes(K, -1, -2) / np.sqrt(d_k)
    if mask is not None:
        S = np.where(mask, -np.inf, S)
    m = np.max(S, axis=-1, keepdims=True)
    if np.isneginf(m).any():
        raise ValueError("Egy query minden kulcsa tiltott.")
    P = np.exp(S - m)
    Z = np.sum(P, axis=-1, keepdims=True)
    A = P / Z
    return A @ V, A

Q = np.array([[1.0, 0.0],
              [0.0, 1.0]], dtype=np.float64)
K = np.array([[1.0, 1.0],
              [2.0, 0.0]], dtype=np.float64)
V = np.array([[1.0, 0.0, 0.0],
              [0.0, 1.0, 0.0]], dtype=np.float64)

O, A = scaled_dot_product_attention(Q, K, V)
assert A.shape == (2, 2) and O.shape == (2, 3)
assert np.allclose(A.sum(axis=1), 1.0, atol=1e-12)
```

```
assert np.allclose(A, [[0.33023845, 0.66976155],
                       [0.66976155, 0.33023845]], atol=1e-4)
print("Maszk nélküli figyelmi súlyok:")
print(A)
print("Kimenet:")
print(O)

causal = np.array([[False, True],
                   [False, False]])
Om, Am = scaled_dot_product_attention(Q, K, V, mask=causal)
assert Am.shape == (2, 2) and Om.shape == (2, 3)
assert np.allclose(Am.sum(axis=1), 1.0, atol=1e-12)
assert np.allclose(Am, [[1.0, 0.0], [0.66976155, 0.33023845]], atol=1e-4)
print("Maszkolt figyelmi súlyok:")
print(Am)
print("Maszkolt kimenet:")
print(Om)
```

A futtatás parancsa `python n126_attention.py`. A kód CPU-t és `float64`-et használ; véletlen szám nincs, ezért seedre nincs szükség. A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, alapértelmezett szálbeállítás):

```
Maszk nélküli figyelmi súlyok:
[[0.33023845 0.66976155]
 [0.66976155 0.33023845]]
Kimenet:
[[0.33023845 0.66976155 0.        ]
 [0.66976155 0.33023845 0.        ]]
Maszkolt figyelmi súlyok:
[[1.         0.        ]
 [0.66976155 0.33023845]]
Maszkolt kimenet:
[[1.         0.         0.        ]
 [0.66976155 0.33023845 0.        ]]
```

A mért értékek a fenti kézi számításból adódó 0,3302/0,6698/0,6698/0,3302 értékeket, illetve a $\begin{pmatrix} 1 & 0 \\ 0{,}6698 & 0{,}3302 \end{pmatrix}$ mátrixot adják. A kódban elhelyezett `assert`-ek a kimeneti alakot, a sorösszegeket és a kézi értékektől való legfeljebb 10^{-4} abszolút eltérést ellenőrzik. A teljesen maszkolt sorokat a függvény a maximum $-\infty$ értéke alapján hibaként jelzi; külön nulla-nevező ellenőrzés nem kell, mert a maximum kivonása után a sorösszeg legalább 1.

32.2.8 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a figyelmi súlymátrix elemeit a softmax-képlettel, és bizonyítsa be, hogy minden A sor nemnegatív elemű és sorösszege pontosan 1. Átadandó: a definíció, a két tulajdonság levezetése és egy rövid megjegyzés arról, hogy az A mátrix miért nem egyenlő a $QK^\top$ pontszámmátrixszal.

F2 (levezetési/bizonyítási, nehézség 2). Bizonyítsa be a softmax shift-invarianciáját, és adott

$$S = \begin{pmatrix} 0 & 2 & -\infty \\ 1 & -1 & 0 \end{pmatrix}$$

esetén számítsa ki a maszkolt softmaxot kétféleképpen: először shift nélkül, majd soronkénti maximumkivonással. Ellenőrizze, hogy a két eredmény azonos, és indokolja, miért kell a maximumot a véges elemekre számítani. Átadandó: a bizonyítás, a két kiszámított súlymátrix és az indoklás.

F3 (implementációs, nehézség 2). Írjon önálló NumPy-függvényt `attention(Q, K, V, mask=None)` néven, amely a labor specifikációja szerint maszkolt scaled dot-product attentiont számít. Ellenőrizze a labor kéttokenes példáján, hogy a maszk nélküli és a maszkolt kimenet `np.allclose` szerint egyezik a kézi referenciával. Átadandó: a függvény teljes forráskódja, a teszt és a kapott kimenet.

F4 (hibakeresési, nehézség 2). Tekintsük a következő hibás maszkolást:

```
def softmax(S):
    E = np.exp(S - np.max(S, axis=-1, keepdims=True))
    return E / E.sum(axis=-1, keepdims=True)

S = Q @ K.T / np.sqrt(d_k)
A = softmax(S)
A = np.where(mask, 0.0, A)   # hibás
O = A @ V
```

Magyarázza meg, miért nem egyezik a kapott A mátrix a Lemma 2 szerinti maszkolt softmaxszal: a tiltott pozíciókon ugyan nulla, de a megengedett pozíciók értékei és a sorösszeg hibás. Mutassa meg a konkrét hibát a labor kéttokenes példájában a sorösszeggel, és adja meg a javítást. Átadandó: a hiba oka, a hibás sorösszeg kiszámítása, a javított kód és az ellenőrzés.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen ellenőrzött kísérletet annak eldöntésére, hogy a $\sqrt{d_k}$ skálázás javítja-e a tanulási stabilitást egy kis szintetikus szekvenciafeladaton. Adja meg a célt, a hipotézist, a feladatot, a splitet, a baseline-t, a változó tényezőt, a metrikát, az ismétlést és az erőforrás-korlátot. Ne futtassa a kísérletet. Átadandó: protokoll, baseline indoklása és legalább két explicit érvényességi korlát.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

32.2.9 Megoldások és indoklás

F1 megoldás. A figyelmi súlymátrix eleme

$$A_{ij} = \frac{\exp(S_{ij})}{\sum_{j'=1}^{T_v} \exp(S_{ij'})}.$$

Mivel az exponenciális függvény pozitív, minden $A_{ij} \geq 0$. Az i-edik sor összege

$$\sum_{j=1}^{T_v} A_{ij} = \frac{\sum_{j=1}^{T_v} \exp(S_{ij})}{\sum_{j'=1}^{T_v} \exp(S_{ij'})} = 1,$$

mert a számláló és a nevező ugyanaz az összeg. Az A mátrix tehát sztochasztikus jellegű súlymátrix. A $QK^\top$ mátrix ezzel szemben nem feltétlenül nemnegatív, és sorösszege sincs normalizálva: a softmax egy lépésben elvégzi a pontszámok nemnegatívvá alakítását és soronkénti normalizálását. A softmax a 7.2. Softmax és többosztályos cross-entropy alfejezetben tárgyalt

osztályeloszlás-konstrukcióval rokon, de itt a cél nem osztálycímke, hanem egy kulcspozíció feletti súlyeloszlás.

F2 megoldás. A shift-invarianciát a fenti Állítás bizonyítja. Az adott S első sora $(0, 2, -\infty)$, második sora $(1, -1, 0)$. A maszkolt softmax shift nélkül, a $-\infty$ exponensét nullának tekintve:

- első sor: $e^0 = 1$, $e^2 \approx 7{,}3891$, $e^{-\infty} = 0$, nevező 8,3891, tehát $A_1 \approx (0{,}1192;\ 0{,}8808;\ 0)$;
- második sor: $e^1 \approx 2{,}7183$, $e^{-1} \approx 0{,}3679$, $e^0 = 1$, nevező 4,0862, tehát $A_2 \approx (0{,}6652;\ 0{,}0900;\ 0{,}2447)$.

A maximumkivonásos eljárásban az első sor maximuma 2, a másodiké 1; a maszkolt $-\infty$ elemek a kivonás után $-\infty$ maradnak, exponensük 0, a normalizáló ugyanaz a nevező és az eredmény azonos. A maximumot a véges elemekre kell számítani. Ha a sorban van legalább egy véges elem, akkor a teljes sor maximuma ugyanaz, mint a véges elemeké, mert a $-\infty$ nem növeli a maximumot, tehát a $-\infty$ bevonása ekkor nem változtat az eltoláson. Ha viszont a teljes sor tiltott, akkor a maximum $-\infty$, és a $-\infty - (-\infty)$ kifejezés NaN-t adna; ezt az esetet külön hibaként kell jelezni (a labor kódja ezt teszi).

F3 megoldás. A labor referenciája a `scaled_dot_product_attention` függvény; a hallgató `attention` nevű, azonos aláírású függvényének legalább a következő ellenőrzéseket kell elvégeznie:

```
O_test, A_test = attention(Q, K, V)
O_m_test, A_m_test = attention(Q, K, V, mask=causal)
assert O_test.shape == (2, 3)
assert A_test.shape == (2, 2)
assert np.allclose(A_test, [[0.3302, 0.6698],
                            [0.6698, 0.3302]], atol=1e-4)
assert np.allclose(A_m_test, [[1.0, 0.0],
                              [0.6698, 0.3302]], atol=1e-4)
assert np.allclose(O_test, [[0.3302, 0.6698, 0.0],
                            [0.6698, 0.3302, 0.0]], atol=1e-4)
print(A_test)
print(A_m_test)
```

A pontos tolerancia 10^{-4}; a kézi értékeket nem kerekítve kell a kódnak kiszámítania. A függvénynek figyelnie kell az S alakjára: a $QK^\top$ során a K utolsó dimenziója azonos kell legyen a Q utolsó dimenziójával, ami a d_k. Batch-dimenziós bemenetnél a K transzponálását `np.swapaxes(K, -1, -2)`-vel kell végezni, mert a `K.T` csak 2 dimenziós mátrixra adja a helyes eredményt.

F4 megoldás. A hibás kód a softmax után nullázza a maszkolt elemeket. A tiltott pozíciókon az eredmény valóban nulla, de a megengedett pozíciók értékei nem a maszkolt logitok softmaxa: a softmax normalizálója még tartalmazta a tiltott elemek exponenciálisát, a nullázás viszont már nem korrigálja a nevezőt, ezért $A \neq \mathrm{softmax}(\tilde{S})$ és a sorösszeg sem 1. A labor példában a maszk nélküli A első sora $(0{,}3302;\ 0{,}6698)$. A hibás utólagos nullázás ezt $(0{,}3302;\ 0)$-ra változtatja; a sorösszeg 0,3302, tehát az első token kimenete az értékek 0,3302-szorosát adja, és a figyelmi súlyok nem konvex kombinációt alkotnak. A javítás az, hogy a maszkot a logitmátrixra kell alkalmazni a softmax előtt:

```
S = np.where(mask, -np.inf, S)
A = softmax(S)
```

Ezzel az első sor $(1, 0)$, a második sor változatlan, és a sorösszegek pontosan egyek. Ez a Lemma 2 tartalma.

F5 megoldásminta. Cél: annak eldöntése, hogy a $\sqrt{d_k}$ skálázás csökkenti-e a tanítás korai szakaszának veszteségvarianciáját. Hipotézis: skálázás nélkül a nagy pontszámok telíthetik a softmaxot, ami lassabb és zajosabb tanulást ad. Feladat: szintetikus másolási feladat, $T = 16$

token, $d_{model} = 32$, egyetlen attention-fej ($d_k = d_v = 32$), padding nélkül; a modell egyetlen attention-réteg, amely a cél token logitjeit adja. Split: a feladatok véletlen generálásból 4000/1000 train/valid példány, rögzített adatseeddel. Baseline: a skálázás nélküli változat (szorzó 1), ugyanazzal az Adam-optimalizálóval (tanulási ráta 10^{-3}), batch 64; a változó tényező kizárólag a skálázó tényező (1 a baseline-ban, $1/\sqrt{32}$ a kezelt karban). Metrika: a valid cross-entropy loss lépésenkénti szórása az első 200 lépésben egy futáson belül, valamint a gradiensnorma átlaga; a 10 seedből álló párosított különbségekre páros t-próbát (vagy Wilcoxon-féle előjeles rangpróbát) végzünk 0,05-ös szignifikanciaszinten, a hatásnagyságot is jelentve. Ismétlés: 10 seed, a két karban azonos adat- és inicializálási seeddel. Erőforrás: CPU, egy szál, összesen legfeljebb 15 perc falidő a 20 futásra. Érvényességi korlát: a szintetikus feladat nem általános szekvenciaprobléma, és egyetlen réteg nem mutatja a mély hálózatok viselkedését; a $T = 16$, $d_k = 32$ konfiguráció eredménye nem terjeszthető ki nagyobb modellekre. A kísérlet nem a skálázás okát bizonyítja, csak ebben a konfigurációban mér.

32.2.10 Tipikus tévedés, önellenőrzés és visszahívás

Tipikus tévedés. Az attention súlymátrixot gyakran közvetlen szemantikai vagy okságí magyarázatként mutatják be. Ez a következtetés nem áll meg. Az A_{ij} súly csak azt mondja meg, hogy az i-edik query a j-edik értékből mekkora **konvex kombinációs** arányt vesz, de az értékvektorokat is tanult vetítés adja. Ellenpélda: legyen $A = (0{,}1;\ 0{,}9)$, $v_1 = (10; 0)$, $v_2 = (-1; 0)$. Ekkor a kimenet $0{,}1 \cdot (10; 0) + 0{,}9 \cdot (-1; 0) = (0{,}1;\ 0)$; a nagyobb figyelmi súly ellenére a v_2 negatív előjele nagyrészt kompenzálja a v_1 nagy pozitív hozzájárulását (a $0{,}1 \cdot 10 = 1$ tagból a kimenetben 0,1 marad). A súly tehát nem hozzájárulás; okságí állításhoz kontrafaktuális beavatkozás kell, amelyet a 35.4. Korlátok, eredet és döntési felelősség egység módszertana tárgyal.

Önellenőrzés. Kérdezzük meg: (1) Milyen alakú a $QK^\top$ és az A mátrix? (2) A maszkolás után minden tiltott sorösszeg pontosan 1-et ad-e? (3) A stabil softmax soronkénti maximuma vajon véges értéken lett-e számítva? (4) A labor függvény hibát ad-e, ha egy teljes sor maszkolt? A négy kérdés mindegyikére csak akkor adott a válasz, ha a levezetések és a teszt egyaránt rendelkezésre állnak.

Válaszkulcs. (1) $QK^\top$ és A alakja a batch- és fejdimenzió nélkül $T \times T$ (a teljes tenzorban $B \times h \times T \times T$). (2) A nem teljesen maszkolt sorok összege 1; teljesen maszkolt sornál a maximum $-\infty$, ezt a függvény hibaként jelzi. (3) Igen, a maximumot a nem maszkolt, véges elemekre számítja. (4) Igen, teljesen maszkolt sorra a labor függvény hibát ad.

Visszahívás. A 28.2. Sziámi és triplet cél triplet-célban a tanult beágyazások közötti távolság explicit pozitív/negatív párokat definiált; itt a $QK^\top$ pontszám ad hoc hasonlóság, és a softmax a sorok felett normalizál. A 20.2. Finomhangolás és tanulási ráta finomhangolás-tanulsága ismét megjelenik: a W^Q, W^K, W^V, W^O paraméterek és az esetleges tanult pozíciókódolás paraméterei eltérő paramétercsoportba tartozhatnak; a fagyasztásukat külön kell kezelni. A 31.2. GAN mint minimax probléma GAN-minimaxjában a diszkriminátor adatfüggő pontszámot állít elő, de ott nincs konvex kombinációra normalizálás; a figyelem softmaxja ebben az értelemben eltérő normalizálási szerződést ad.

32.2.11 Rövid összefoglaló

A scaled dot-product attention a query-kulcs pontszámmátrix és az értékek szorzata. A $QK^\top$ eleme a query és a kulcs skalárszorzata; a $\sqrt{d_k}$ osztó heurisztikus varianciastabilizálás. A softmax soronkénti normalizálása konvex súlyokat ad. A tiltott pozíciókat a pre-softmax logitban $-\infty$-nel kell jelölni, így a maszkolt súly pontosan nulla; utólagos nullázás hibás sorösszeget ad. A stabil

implementáció soronkénti maximumot von ki. A multi-head a fejenkénti vetítések konkatenációja, a pozíciókódolás pedig a permutáció-szimmetriát töri meg azzal, hogy pozíciófüggő információt ad a tokenekhez. A súlymátrix nem önmagában oksági magyarázat.

Az induló becslésre a válasz: a figyelem valóban egy $T \times T$ pontszámtáblázatból indul, a naiv implementáció ezt teljes egészében elkészíti; a maszkolás a pre-softmax logitba tett $-\infty$, a numerikus stabilitást a soronkénti maximum kivonása adja, a költség pedig $O(T^2 d)$ idő és $O(T^2)$ memória.

32.2.12 Következő alkalmazás és források

A következő egység 32.3. Vision Transformer és hibrid felhasználás ezt a figyelmi műveletet képekre alkalmazza: a Vision Transformerben a képet tokendarabokra vágjuk, és a pozíciókódolás, valamint a figyelmi rétegek együttesét vizsgáljuk a CNN-ekkel való összevetés előtt.

32.2.13 Felhasznált források

- [P29] Dao, T., Fu, D. Y., Ermon, S., Rudra, A. és Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. arXiv:2205.14135. 3. szakasz, 1. tétel (5. o.), az arXiv-PDF alapján. https://arxiv.org/abs/2205.14135.
- [P21] Vaswani et al. (2017). *Attention Is All You Need*. arXiv:1706.03762. https://arxiv.org/abs/1706.03762. A scaled dot-product attention, a multi-head változat és a szinuszos pozíciókódolás hivatkozása.
- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/. A mátrixjelölést a könyvben megadott alakban használjuk.

32.3 Vision Transformer és hibrid felhasználás

32.3.1 Cél, előismeret és induló kérdés

Ez az alfejezet a Vision Transformer (ViT) képfeldolgozási szerződését építi fel: miként lesz egy $N \times C \times H \times W$ képtensorból tokensorozat, milyen induktív feltevéseket épít be a modell, és miben tér el a CNN lokális feltevésétől. Előismeret a 32.2. Scaled dot-product attention attention-definíciója (Q/K/V, softmax, skálázás, multi-head, pozíciókódolás), valamint a 13.2.–16.1. konvolúciós és ekvivariancia-fogalmai rövid visszahívással. Induló becslés: egy 24×24 méretű GlyphLab-kép (a függelék A. A GlyphLab adatkészlet szakasza szerint) 8×8 patchekkel hány tokent ad, és hány paramétere van egy $D = 32$ dimenziós patch-beágyazásnak? A fejezet végére ennek pontos, alakellenőrzött válaszát és az ekvivalens konvolúciós megvalósítást is megadjuk.

32.3.2 Patch embedding mint tanult tokenizáció

Definíció (patch embedding). Legyen $X \in \mathbb{R}^{N \times C \times H \times W}$ egy képbatch. Rögzített patchmérethez $P_h, P_w > 0$ tegyük fel, hogy $P_h \mid H$ és $P_w \mid W$. Ekkor a térbeli rács:

$$n_h = \frac{H}{P_h}, \qquad n_w = \frac{W}{P_w}, \qquad L = n_h n_w.$$

Az (i, j) patchvektor $x_{n,i,j} \in \mathbb{R}^{CP_hP_w}$ a

$$X[n, c, iP_h + a, jP_w + b], \qquad 0 \le a < P_h,\ 0 \le b < P_w,\ 0 \le c < C$$

elemek rögzített sorrendű felsorolása. A teljes patch-mátrix $X_p \in \mathbb{R}^{N \times L \times (CP_hP_w)}$. A beágyazás egy $W \in \mathbb{R}^{D \times (CP_hP_w)}$ súlyú, $b \in \mathbb{R}^D$ torzítású affin leképezés:

$$Z = X_pW^\top + b, \qquad Z \in \mathbb{R}^{N \times L \times D},$$

ahol b az N és L dimenziók mentén broadcastolódik. A ViT-ben ehhez a sorozathoz egy tanulható $z_{cls} \in \mathbb{R}^{1 \times 1 \times D}$ osztálytokent és egy $E_{pos} \in \mathbb{R}^{1 \times (L+1) \times D}$ pozícióbeágyazást adnak, így a bemenő tokensorozat $Z^0 = [z_{cls}; Z] + E_{pos} \in \mathbb{R}^{N \times (L+1) \times D}$.

Lemma (a patch embedding mint nem átfedő konvolúció). Legyen $P_h \mid H$, $P_w \mid W$, C, D tetszőleges. A fenti affin patch-beágyazás pontosan ugyanazt a Z kimenetet adja, mint egy `Conv2d(C, D, kernel_size=(P_h,P_w), stride=(P_h,P_w), padding=0)` réteg, amelynek kimenetét (N, D, n_h, n_w) alakról (N, D, L)-re lapítjuk, majd (N, L, D)-re permutáljuk — feltéve, hogy a konvolúciós súly $K \in \mathbb{R}^{D \times C \times P_h \times P_w}$ és a lineáris súly W azonos indexsorrendben tartalmazza ugyanazokat a skalárokat.

Bizonyítás. Írjuk ki a konvolúció kimenetét egy rögzített (n, o, i, j) pozícióra, ahol o a kimeneti csatorna, (i, j) az $n_h \times n_w$ méretű térbeli rács koordinátája. Mivel $k_h = P_h$, $k_w = P_w$, $s_h = P_h$, $s_w = P_w$ és $p = 0$, a 14.1. Stride, padding, dilation és outputméret alakképlete szerint a kimeneti rács éppen $n_h \times n_w$, és a bemeneti ablak az (i, j) patch:

$$Y[n, o, i, j] = \sum_{c=0}^{C-1} \sum_{u=0}^{P_h-1} \sum_{v=0}^{P_w-1} K[o, c, u, v]\, X[n, c, iP_h + u, jP_w + v] + b[o].$$

Az összeg ugyanaz a CP_hP_w skalár szorzatösszege, mint a $Z[n, in_w+j, o] = \sum_r W[o, r]X_p[n, in_w+j, r] + b[o]$ kifejezés, ha $W[o, (cP_h + u)P_w + v] = K[o, c, u, v]$ (a kernel térbeli indexei u, v, hogy ne ütközzenek a b torzítással). Mivel az ablakok nem fednek át, más patch nem szól bele az adott tokenbe. A két kimenet a szorzatösszegeket tekintve ugyanaz; egzakt (valós) aritmetikában elemenként egyenlő, lebegőpontos számításban pedig az összegzési sorrendtől függő kerekítési eltérés léphet fel. □

Költség-összehasonlítás. A lemma két fontos következményt rögzít:

1. *Paraméterszám:* mindkét oldalon $D(CP_hP_w + 1)$.
2. *MAC-kép száma:* képenként $L \cdot CP_hP_w \cdot D$.

Ha a stride kisebb a kernelnél, a tokenek száma és a recepciós mezők átfedése nő. Nem átfedő, $s = k$ esetén érdekes állandóság lép fel: $L = HW/(P_hP_w)$, ezért a beágyazás teljes MAC-száma $N \cdot HW \cdot C \cdot D$, független a patch-mérettől — csak a tokenek száma és az attention későbbi költsége változik. Átfedő ablaknál ($s < k$ vagy $p > 0$) a képnkénti MAC az ablakméret és a tokenszám szorzataként nő.

32.3.3 A ViT architektúra alakjai

A ViT egy tiszta transzformer-encoder képklasszifikációra [P22]. Blokkonként, $z \in \mathbb{R}^{N\times(L+1)\times D}$ bemenetre a pre-norm (a normalizálás az attention- és MLP-részblokk előtt futó) reziduális szerkezet:

$$u = z + \mathrm{MHA}\,(\,\mathrm{LN}(z)), \qquad z' = u + \mathrm{MLP}\,(\,\mathrm{LN}(u)).$$

Az MHA a 32.2. Scaled dot-product attention alfejezetben definiált többfejű scaled dot-product attention [P21]: $Q = zW_Q^\top$, $K = zW_K^\top$, $V = zW_V^\top$ az összesített vetítés után $(N, L+1, D)$ alakú; ezt h fejre bontjuk, $D = h\,d_{head}$: $(N, h, L+1, d_{head})$. A figyelmi score fejenként $QK^\top/\sqrt{d_{head}} \in \mathbb{R}^{N\times h\times(L+1)\times(L+1)}$; az $(N, L+1, L+1)$ alak és a $\sqrt{D}$ skála csak az egyfejes esetre ($h = 1$, $d_{head} = D$) igaz, több fejnél a skálázó tényező $\sqrt{d_{head}} = \sqrt{D/h}$. Egy fej kimenete a fej V-vektorainak konvex kombinációja, de a fejek összefűzése és a kimeneti projekció után a teljes MHA általában nem konvex kombinációja az eredeti V-vektoroknak. Az MLP két lineáris vetítésből és GELU-aktivációból áll, közbülső $D_{ff} > D$ szélességgel; a pontos D_{ff} az eredeti publikáció hiperparamétere. A végső osztályozás a CLS-token $z_{cls} \in \mathbb{R}^{N\times D}$ kimenetén egy $W_{head} \in \mathbb{R}^{K\times D}$ fejjel számol logitot: $Z = z_{cls}W_{head}^\top + b_{head} \in \mathbb{R}^{N\times K}$.

A laborméretű példában $D = 32$, $L = 9$, $D_{ff} = 64$ egyetlen blokk paraméterszáma: QKV-vetítés $3 \cdot 32 \cdot 32 = 3072$, kimeneti vetítés $32 \cdot 32 + 32 = 1056$, MLP $2 \cdot (64 \cdot 32) + 64 + 32 = 4192$, két LayerNorm 128; összesen mintegy 8,4 ezer paraméter. Ez csak a blokk; a teljes hálóhoz a beágyazás (2080), a CLS- és pozícióbeágyazás $(32 + 320)$ is hozzáadódik.

32.3.4 Inductive bias: lokális és globális feltevés

Az *inductive bias* (induktív torzítás) a modellcsaládba beépített, adattól független szerkezeti feltevés, amely leszűkíti a reprezentálható függvények halmazát. Két szinten érdemes elkülöníteni:

- **CNN:** a 16.1. Eltolási equivariance és invariance alfejezetben tárgyalt lokális recepciós mező és súlymegosztás már a háló definíciójában rögzíti az eltolási ekvivarianciát: egy kép eltolása a közbülső jellemzőtérkép eltolását eredményezi. Ez akkor hasznos, ha a feladat valóban lokális mintázatokból áll, és kevés adatból is stabil becslést ad.
- **ViT pozíciókód nélkül:** a tokenpermutációra ekvivariáns lenne, mert a per-token LayerNorm, az MLP és az attention a tokenek indexét nem használja. A tanulható abszolút E_{pos} ezt a szimmetriát megtöri, de a beépített geometriai tudás így is sokkal gyengébb: az eltolási ekvivarianciát a modellnek az adatból kell megtanulnia.

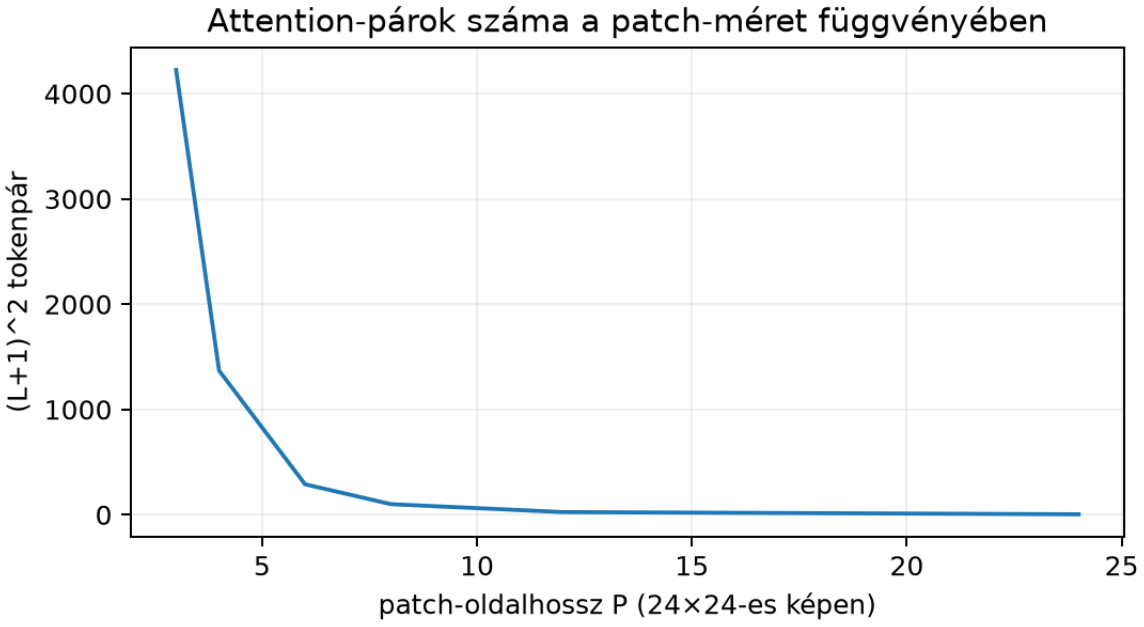
Ebből következik a [P22] absztraktjában megfogalmazott empirikus állítás: a ViT kevesebb kép-specifikus induktív feltevést épít be, ezért nagy előtanító adat és számítás mellett a tanulás jó eredményt érhet el. A “kis adaton viszont a CNN beépített lokalitása gyakran előnyösebb” állítás ennek tankönyvi parafrázisa, amely a [P22] kísérleti részének megfigyelésein alapul, nem az absztrakt szó szerinti állítása, és nem a jelen könyvben bizonyított tétel; konkrét benchmarkszámot itt nem idézünk.

32.3.5 Adat- és számításigény

A két család számítási profilja eltérően skálázódik. Egy hagyományos konvolúciós réteg költsége a pixelek számában lineáris marad: $O(HW \cdot k_h k_w \cdot C_{in} C_{out})$. A ViT attention-része tokenpárokat számol, tehát $O((L+1)^2 D)$ művelet blokkonként. Mivel $L \propto P^{-2}$, a tokenpárok száma aszimptotikusan P^{-4}-nel arányosan nő: a patch-méret felezése a tokenek számát négyszerezi, a párokét tizenhatszorozza. A 24×24-es példán:

- $P = 8$ esetén $L = 9$, attention-párok száma a CLS-sel együtt $10^2 = 100$;
- $P = 3$ esetén $L = 64$, a párok száma már $65^2 = 4225$;
- $P = 1$ (pixelenkénti token) mellett $L = 576$ és a párok száma $577^2 \approx 333$ ezer.

Az így számított skálázást szemlélteti az alábbi diagram (VÁRT ÉRTÉK (levezetésből, nem mért)):



32.1. ábra: Attention-párok száma a patch-méret függvényében

Képletből: L=(24/P)^2 és (L+1)^2; CLS-tokennel. VÁRT ÉRTÉK (levezetésből, nem mért).

Vegyük észre a fenti lemma következményét: a *beágyazás* teljes költsége $s = k$ mellett független a patch-mérettől, tehát a patch-méret csökkentése elsősorban az attention-költséget növeli, nem a tokenizációt. A patch-méret ezért számítási döntés, nem pusztán felbontási kérdés.

32.3.6 CNN-hybrid felhasználás

A hibrid modell a két induktív feltevés kompozíciója: egy CNN gerinc először lokális jellemzőtérképet állít elő, majd ennek térbeli cellái lesznek a transzformer tokenjei. Formálisan $F = \mathrm{CNN}(X) \in \mathbb{R}^{N \times C_f \times H_f \times W_f}$, a tokenek $L' = H_f W_f$ darab C_f dimenziós vektor, amelyeket egy $W_{proj} \in \mathbb{R}^{D \times C_f}$ vetítés képez D dimenzióba:

$$Z_{hyb} = F_{flat} W_{proj}^{\top} + b_{proj} \in \mathbb{R}^{N \times L' \times D}.$$

A GlyphLab-példán: egy 3×3-as, stride 2, padding 1 konvolúció $1 \to 16$ csatornával 12×12-es térképet ad; egy második $16 \to 32$ csatornával 6×6-ot; ez $L' = 36$ token, szemben a nyers kép $P = 8$ patchelésének 9 tokenjével. A hibrid tehát sűrűbb, már feldolgozott tokeneket ad. A kezdeti lokális szűrők a CNN lokális induktív feltevését viszik be, ami kis adaton hasznos lehet; ez azonban hipotézis, nem ebben a könyvben mért eredmény. A globális kontextus másik útját a 31.3. Diffúziós zajmodell és CNN-denoiser U-Netjében a mély encoder rétegei fedik le, ahol a konvolúció mélysége növeli a recepciós mezőt; az attention itt ezt a globális keverést minden blokkban közvetlenül elvégzi.

32.3.7 Foundation-use szerződés

Nagy előtanított ViT-ek felhasználása a 20.3. Preprocessing, domain shift és súlylicenc alfejezetben rögzített szerződést örökli: a súlycsomag enumja azonosítja a pontos változatot, a licenc a felhasználási feltételt, a preprocessing-szerződés pedig a bemeneti elvárást. Egy adott előtanított ViT-súly bemeneti szerződése (felbontás, csatornaszám, normalizálás) a checkpoint dokumentációjából ellenőrizendő; általános törvény nincs, de a leggyakoribb ImageNet-előtanítás például RGB-bemenetet és adott normalizálást használ. A 24×24 egycsatornás GlyphLab-kép közvetlen betáplálása ezért domain shiftet jelent, és a teljesítményre semmilyen garancia nem vonatkozik automatikusan. A 28.3. Retrieval-értékelés és adatfelosztás alfejezetből ismert globális embedding-használathoz hasonlóan a CLS-token kimenete egy globális képdeszkriptor, de ennek minősége az előtanítási eloszláshoz kötött. Ebben a laborban külső súlyt nem töltünk be; a foundation-use itt csak a bemeneti szerződés szintjén értelmezett előkészítés.

32.3.8 Labor: CNN/ViT döntési lap azonos inputon

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor célja nem modellverseny, hanem egy alak- és költségalapú *döntési lap*: ugyanarra a determinisztikus, $[0, 1]$-beli 24×24 float32 NCHW képre felépítjük a konvolúciós és a lineáris patch-beágyazást, ellenőrizzük az ekvivalenciát, majd kiszámítjuk a token-, paraméter- és attention-alakokat. A kimenet a könyv írási környezetében mért érték. Egzakt aritmetikában a két beágyazás kimenete elemenként azonos, ezért az eltérés elméleti értéke 0; float32-ben az összegzési sorrend miatt a mért maximális eltérés $3{,}2783 \cdot 10^{-7}$, ami a kód 10^{-6}-os tűréshatára alatt van.

```
# munkakönyvtár: lab/n127_decision/
import torch

torch.manual_seed(20261020)          # véletlen súlyok reprodukálhatóságához
dtype = torch.float32
device = torch.device("cpu")

N, C, H, W, D, P = 1, 1, 24, 24, 32, 8
x = torch.arange(H * W, dtype=dtype, device=device).view(N, C, H, W) / (H * W - 1)

# 1) konvolúciós beágyazás: k=P, s=P, p=0
conv = torch.nn.Conv2d(C, D, kernel_size=P, stride=P, padding=0,
                       bias=True, device=device, dtype=dtype)

# 2) ekvivalens lineáris patch-beágyazás: a konvolúciós súly átrendezésével
lin = torch.nn.Linear(C * P * P, D, bias=True, device=device, dtype=dtype)
with torch.no_grad():
    lin.weight.copy_(conv.weight.reshape(D, C * P * P))
    lin.bias.copy_(conv.bias)
```

```
patches = x.unfold(2, P, P).unfold(3, P, P)      # (N, C, 3, 3, P, P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(N, -1, C * P * P)  # (N,9,64)

z_conv = conv(x).flatten(2).transpose(1, 2)       # (N,9,D)
z_lin = lin(patches)                               # (N,9,D)
diff = (z_conv - z_lin).abs().max().item()

assert z_conv.shape == (N, 9, D) and z_lin.shape == (N, 9, D)
assert diff <= 1e-6, diff
assert sum(p.numel() for p in conv.parameters()) == 2080
assert sum(p.numel() for p in lin.parameters()) == 2080

print("konv súly:", tuple(conv.weight.shape), "bias:", tuple(conv.bias.shape))
print("linear súly:", tuple(lin.weight.shape), "bias:", tuple(lin.bias.shape))
print("tokenek:", z_conv.shape[-2], "D:", z_conv.shape[-1])
print("paraméterek konv/linear:",
      sum(p.numel() for p in conv.parameters()),
      "/", sum(p.numel() for p in lin.parameters()))
print("max |z_conv - z_lin|:", diff)

cls = torch.zeros((N, 1, D), dtype=dtype, device=device)
z = torch.cat([cls, z_lin], dim=1)                 # (N, 10, D)
qkv = torch.nn.Linear(D, 3 * D, bias=False, device=device, dtype=dtype)
q, k, v = qkv(z).chunk(3, dim=-1)                  # (N,10,D) mind
# egyfejes esetben d_head = D, ezért D ** -0.5; több fejnél d_head ** -0.5
attn = (q @ k.transpose(1, 2)) * (D ** -0.5)       # (N,10,10)
# csak alakellenőrzés: pre-softmax pontszám, softmax és V-szorzás nélkül
assert attn.shape == (N, 10, 10)
print("attention-párok:", attn.shape[-1] * attn.shape[-2],
      "alak:", tuple(attn.shape))
```

Mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020):

```
konv súly: (32, 1, 8, 8) bias: (32,)
linear súly: (32, 64) bias: (32,)
tokenek: 9 D: 32
paraméterek konv/linear: 2080 / 2080
max |z_conv - z_lin|: 3.2782554626464844e-07
attention-párok: 100 alak: (1, 10, 10)
```

A mért értékek: konvolúciós súly $(32, 1, 8, 8)$, lineáris súly $(32, 64)$; tokenek száma 9, $D = 32$; paraméterszám mindkét oldalon 2080. A `max |z_conv - z_lin|` elméleti értéke egzakt aritmetikában 0, a mért érték float32-ben $3{,}2783 \cdot 10^{-7}$; a kód `assert`-je a 10^{-6}-os korlátot ellenőrzi. Az attention-párok száma 100, alak $(1, 10, 10)$; ez azonban csak a pre-softmax pontszámmátrix alakellenőrzése, nem teljes attention-számítás, mert a softmaxot és az értékvektorokkal való szorzást nem futtatja. A futtatás parancsa a `lab/n127_decision/` könyvtárból: `python decision_patch_conv.py`.

A döntési lap tényrögzítő táblája:

Sor	CNN-beágyazás	ViT patch-beágyazás
Tokenek ($P = 8$)	9	9
Paraméterek ($D = 32$)	2080	2080
Beágyazó réteg recepciója	8×8	8×8
Rétegek közti interakció	lokális, mélységgel nő	attention, globális páronként
Beépített geometria	eltolási ekvivariancia	tanulandó pozíciókód

A táblázat szándékosan **nem tartalmaz** minőségi rangsort: ilyen állításhoz adat, modellméret, tréningrecept és kiértékelési protokoll kell, amelyet itt nem tételezünk fel.

32.3.9 Feladatok

F1 — fogalmi (nehézség 1, előfeltétel: 32.2. Scaled dot-product attention, 16.1. Eltolási equivariance és invariance). Definiálja az *inductive bias* fogalmát, adjon egy-egy konkrét példát a CNN-re és a ViT-re, és magyarázza el, miért mondjuk, hogy a ViT kevesebb kép-specifikus induktív feltevést épít be. Átadandó: 200–400 szavas saját magyarázat, benne az eltolási ekvivariancia és a tokenpermutáció-szimmetria megkülönböztetése, valamint egy mondatban az adatmennyiségtől való várható függés.

F2 — levezetési (nehézség 2, előfeltétel: 14.1. Stride, padding, dilation és outputméret, 14.3. Paraméterszám, MAC és aktivációmemória). Egy 32×32-es, egycsatornás képre ($N = 1$, $C = 1$) számolja ki (a) a $P = 8$, $D = 48$ patch-beágyazás és (b) a $k = 8$, $s = 4$, $p = 0$, $C_{out} = 48$ konvolúciós beágyazás tokenszámát, paraméterszámát és képenkénti MAC-számát. Mutassa meg, melyik paraméterválasztás mellett lenne a konvolúció pontosan ekvivalens a patch-beágyazással, és indokolja a lemma feltételeiből. Átadandó: minden közbülső számítás, a 14.1. Stride, padding, dilation és outputméret alakképletének explicit alkalmazása és alakellenőrzés.

F3 — implementációs (nehézség 2, előfeltétel: 11.1.–11.2.). Írjon teljes, futtatható PyTorch-szkriptet, amely egy $1 \times 1 \times 16 \times 16$ méretű, determinisztikus `arange`-alapú float32 bemeneten ellenőrzi a lemma állítását $P = 8$, $D = 16$ mellett: a `Conv2d(kernel_size=8, stride=8, padding=0)` és az ekvivalens `Linear` kimenetének maximális abszolút eltérését 10^{-5} toleranciával kontrollálja. A súlyátvitelt a laborban látott módon végezze. Átadandó: teljes kód, seed, dtype, device, a szélsőeset ($P = H$) token- és alakeseménye, valamint a tolerancia indoklása.

F4 — hibakeresési (nehézség 2, előfeltétel: 13.1. Kép mint diszkrét többcsatornás jel és az ebben az alfejezetben bevezetett patch-definíció). Az alábbi töredék RGB-képen akar patch-tokeneket létrehozni, de a kimenet alakja $(1, 27, 64)$ lett a várt $(1, 9, 192)$ helyett. Lokalizálja a hibát, írja le a következményét az attention-bemenetre, javítsa a kódot, és adja meg a javítás utáni ellenőrző alakot:

```
x = torch.randn(1, 3, 24, 24)          # NCHW
pat = x.unfold(2, 8, 8).unfold(3, 8, 8) # (1,3,3,3,8,8)
tok = pat.reshape(1, -1, 8 * 8)         # (1,27,64) -- hibás
```

Átadandó: az ok, a szemantikai következmény (csatornák külön tokenként), a javított kód és a javítás utáni alak.

F5 — kísérleti/transzfer (nehézség 3, előfeltétel: 1.3. Adatfelosztás és a megfigyelési egység, 22.4. Augmentációs kísérlet és domainváltozás, 23.3. Páros összehasonlítás és csoportszintű intervallum). Tervezzen kis CPU-s kísérletet a GlyphLab adathalmazon egy rögzített kis CNN és egy azonos paraméterkeretű hibrid CNN–ViT összehasonlítására. A GlyphLab adathalmaz fájljait (`images.npy`, `metadata.csv`) és a `scene_id` jelentését az A. A GlyphLab adatkészlet függelék rögzíti. A terv kötelező elemei: megfigyelési egység (scene_id), splitbetartás, baseline, egyetlen változó tényező, azonos tréningrecept, val-alapú checkpoint-választás, pontosság és jelenet-alapú bootstrap-CI, valamint a korrupt teszt csak a modellválasztás utáni használata. Tüntesse fel, mit **nem** állítunk a kísérletből. Átadandó: 1–2 oldalas protokollterv, keresési és erőforráskerettel, előre megjelölt nyertes nélkül.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

32.3.10 Megoldások és indoklás

F1. Az inductive bias a tanuló modellcsalád szerkezetébe épített feltételezés, amely adat nélkül is leszűkíti a reprezentálható függvények körét. A CNN példája: a súlymegosztott kis kernel miatt a háló már definíció szerint eltolási-ekvivariáns, tehát egy 3×3-as minta felismerését nem kell minden pozícióban külön megtanulni. A ViT példája: pozíciókód nélkül a tokenek sorrendjére nézve ekvivariáns, tehát a térbeli viszonyok ismerete nincs beépítve; abszolút pozíciókóddal a modellnek ezt adatból kell megtanulnia. Kevés adaton a CNN erős feltevése csökkenti a varianciát, míg a ViT gyengébb feltevése több adatot igényelhet, cserébe rugalmasabban tanulhat globális összefüggéseket.

F2. (a) A 32×32, $P = 8$ kép $n_h = n_w = 4$ patchre bomlik, $L = 16$ token. A beágyazási bemenet $d_{in} = CP_hP_w = 64$, a kimenet $D = 48$; paraméterszám $48 \cdot 64 + 48 = 3120$; MAC $L \cdot d_{in} \cdot D = 16 \cdot 64 \cdot 48 = 49152$ képenként. (b) A 14.1. Stride, padding, dilation és outputméret képletével $H_{out} = \lfloor (32 + 2 \cdot 0 - 8)/4 \rfloor + 1 = \lfloor 24/4 \rfloor + 1 = 7$, tehát $L = 49$ token. Paraméterszám $D(Ck_hk_w + 1) = 48 \cdot 65 = 3120$, azonos (a)-val, mert az ablakméretek is 64 skalárt fednek. MAC képenként $L \cdot Ck_hk_w \cdot D = 49 \cdot 64 \cdot 48 = 150528$, ami $49/16 \approx 3{,}06$-szorosa (a)-nak az átfedő ablakok miatt. A lemma feltétele szerint $k = P$ és $s = P$, $p = 0$ mellett lenne ekvivalens a két beágyazás, tehát (b) csak $k = 8, s = 8, p = 0$ választással egyezne meg (a)-val.

F3. Teljes, futtatható szkript:

```python
# f3_patch_embedding.py
import torch

torch.manual_seed(20261020)
dtype = torch.float32

N, C, H, W, D, P = 1, 1, 16, 16, 16, 8
x = torch.arange(H * W, dtype=dtype).view(N, C, H, W) / (H * W - 1)

conv = torch.nn.Conv2d(C, D, kernel_size=P, stride=P, padding=0,
                       bias=True, dtype=dtype)
lin = torch.nn.Linear(C * P * P, D, bias=True, dtype=dtype)
with torch.no_grad():
    lin.weight.copy_(conv.weight.reshape(D, C * P * P))
    lin.bias.copy_(conv.bias)

patches = x.unfold(2, P, P).unfold(3, P, P)          # (N,C,2,2,P,P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(N, -1, C * P * P)

z_conv = conv(x).flatten(2).transpose(1, 2)           # (N,4,D)
z_lin = lin(patches)                                  # (N,4,D)
diff = (z_conv - z_lin).abs().max().item()

assert z_conv.shape == (N, 4, D) and z_lin.shape == (N, 4, D)
assert diff <= 1e-5, diff
assert sum(p.numel() for p in conv.parameters()) == 1040
assert sum(p.numel() for p in lin.parameters()) == 1040
print("tokenek:", z_conv.shape[-2], "diff:", diff)

# szélsőeset: P = H = 16
conv1 = torch.nn.Conv2d(C, D, kernel_size=16, stride=16, padding=0,
                        bias=True, dtype=dtype)
out1 = conv1(x)
print("P=H esetén tokenek:", out1.shape[-1] * out1.shape[-2])
```

A szkript lefutott (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020); mért kimenete `tokenek: 4 diff: 1.1920928955078125e-07` és `P=H esetén tokenek: 1`. A kimeneti

kontroll: tokenek 4 ($n_h = n_w = 2$), a lineáris súly $(16, 64)$, a konvolúciós súly $(16, 1, 8, 8)$, paraméterszám $16 \cdot 64 + 16 = 1040$ mindkét oldalon. Szélsőeset $P = H = 16$: $L = 1$, tehát egyetlen token az egész kép, és az attention-párok száma CLS-sel 4. A feladat 10^{-5} toleranciája lazább, mint a labor 10^{-6}-ja, mert itt a kisebb bemeneten a kerekítési hiba hasonló nagyságrendű, a cél pedig a hibás indexsorrend kiszűrése; a mért eltérés $1{,}19 \cdot 10^{-7}$, ami mindkét korlát alatt van.

F4. A hiba a `reshape` előtti `permute` hiánya. Az `unfold` (ablakozó kihajtogatás) kimenete (N, C, L_h, L_w, P, P) sorrendű, ezért a közvetlen `reshape(N,-1,P*P)` a csatornákat külön tokenként kezeli: $3 \cdot 3 \cdot 3 = 27$ token jön létre 64 dimenzióval, és egy-egy patch csatornái szétválnak. Következmény: az attention háromszor annyi, csatornafosztott tokenen fut, ezért a tokenpárok száma kilencszeres (27^2 a 9^2 helyett); a per-token recepciós mező egycsatornás 8×8 lesz, és a modell nem látja a színes patch teljes 192 skalárját együtt. Javítás: `tok = pat.permute(0, 2, 3, 1, 4, 5).reshape(1, 9, 192)`. Az ellenőrző alak $(1, 9, 192)$.

F5. A terv gerince: adat = GlyphLab `images.npy`/`metadata.csv`, train/val/test a jelenetalapú split szerint, a validáció és a CI egysége mindenütt `scene_id`. A vizsgált tényező kizárólag a modellcsalád: (A) kis CNN (például két konvolúciós blokk), (B) hibrid CNN–ViT (a CNN utolsó, 6×6-os térképét 36 tokenre lapítva, egy attention-blokk). A paraméterkeretet úgy rögzítjük, hogy a két modell teljes paraméterszáma legfeljebb 5%-kal térjen el; ezt a CNN csatornaszámának és a ViT beágyazási dimenziójának a kiértékelés előtti beállításával érjük el. Rögzített tényezők: 5 tanítási seed, batch, epoch, optimizáló, augmentáció, 24×24 input, CPU. A checkpointot a validációs veszteség minimalizálása szerint választjuk, a jelentett metrika a tesztpontosság; a két döntés külön történik, a tesztpontosságot nem használjuk választásra. A teszt és a korrupt teszt csak a modellválasztás után kerül kiértékelésre; a bizonytalanságot jelenet-alapú bootstrap-CI-vel adjuk meg. Nem állítjuk: természetes képi általánosítást, a modellcsaládok általános fölényét, GPU- vagy nagyadatos viselkedést; ez egy kontrollált, kis méretű adaptációs megfigyelés.

32.3.11 Gyakori tévedés ellenpéldával

A leggyakoribb hiba azt állítani, hogy „a ViT jobb a CNN-nél” vagy fordítva. Az állítás helyes formája mindig feltételes: melyik adat-, méret- és receptosztályon, milyen metrikával. Ellenpéldapár a modellcsalád-fölény ellen: (i) ha a valódi címkefüggvény tisztán lokális, például a label csak a kép 3×3-as középső blokkjának átlagától függ, egy kis CNN már a beépített lokalitással ezt meg tudja tanulni, míg a ViT-nek ugyanehhez előbb ki kell tanulnia, hogy a távoli tokenpárok irrelevánsak; (ii) ha a label két távoli pixel szorzatától függ, egyetlen attention-réteg közvetlenül képes a két pozíciót összekötni, míg egy sekély, kis kernelű CNN recepciós mezője ezt nem fedi. Ez a két konstrukció *heurisztikus* ellenpélda, nem formális lehetetlenségi tétel: elég mély CNN vagy elegendő adat mindkét esetben változtathat a képen. A tanulság: a fölény adat-, méret- és receptfüggő, és csak rögzített protokollú méréssel állítható — mint [P22] absztraktjában is, nagy előtanítás feltételéhez kötve.

32.3.12 Önellenőrzés

- A lemma feltételeiben szerepel-e minden oszthatósági és stride-konvenció? A jelölések követik-e a könyv NCHW- és Linear-súlyalak-konvencióját?
- A paraméter- és MAC-számításoknál a bias és a batch dimenzió explicit?
- A laborban szerepel-e seed, dtype, device, futtatási parancs, és a kimenet mért érték-e, vagy a „VÁRT ÉRTÉK (levezetésből, nem mért)” címkét viseli-e?
- A F4 hibája után a permutált patch-sorrend valóban (C, P_h, P_w) vektort ad tokenenként?

- Az összehasonlító állítások mellé megadtuk-e az adatot, a modellméretet, a tanítási protokollt és a mért metrikát? A források összefoglalóiból nem következtethetünk nem közölt architekturális részletekre vagy benchmarkértékekre.

Válaszkulcs. (1) Igen, a lemma rögzíti az oszthatósági és stride-konvenciót, a jelölések követik az NCHW- és a Linear-súlyalak-konvenciót. (2) Igen, a bias és a batch dimenzió explicit. (3) Igen, a labor tartalmaz seedet, dtype-ot, device-ot és futtatási parancsot; a kimenet mért vagy „VÁRT ÉRTÉK" címkével ellátott. (4) Igen, a permutált patch-sorrend tokenenként (C, P_h, P_w) vektort ad. (5) Igen, az összehasonlító állítások mellett szerepel az adat, a modellméret, a protokoll és a mért metrika.

32.3.13 Rövid összefoglaló

A patch embedding az $N \times C \times H \times W$ képet $N \times L \times D$ tokensorozatra vetíti; osztható méretnél ez pontosan egy $k = P$, $s = P$, $p = 0$ konvolúcióval azonos lineáris leképezés, azonos paraméter- és MAC-számmal. A ViT ehhez CLS-tokent és tanulható pozíciókódot ad, majd pre-norm attention-blokkokat alkalmaz, amelyek globális tokenpár-interakciót számolnak. A CNN lokális, eltolás-ekvivariáns feltevést épít be; a ViT kevesebb kép-specifikus feltevéssel, adatból tanulja a geometriát. Hogy melyik család vezet, az adattól, mérettől és tréningrecepttől függ; a labor ezért csak alakeseményeket és költségeket rögzítő döntési lapot ad. A hibrid CNN–ViT a két feltevést kompozícióként egyesíti.

Az induló becslésre a válasz: a 24×24-es kép 8×8-as patchekkel $3 \times 3 = 9$ tokent ad, és a $D = 32$ dimenziós patch-beágyazás paraméterszáma $32 \cdot (8 \cdot 8 \cdot 1) + 32 = 2080$; a lemmában ezt az ekvivalens $k = 8$, $s = 8$, $p = 0$ konvolúció adja.

32.3.14 Következő alkalmazás és források

A következő egység, 32.4. VIII. ellenőrző pont: célhoz illő neurális modell, a VIII. ellenőrző pont: a CNN, a hibrid ViT és más modellcsaládok közötti választást a feladathoz, adathoz, mérethez és szerződéshez kell kötni; a jelen alfejezet alak-, költség- és induktívfeltevés-lapja ehhez ad döntési nyelvet.

Felhasznált források:

- [P21] Vaswani, A. et al. (2017). *Attention Is All You Need.* arXiv:1706.03762. https://arxiv.org/abs/1706.03762. A transzformer-encoder blokk és a multi-head attention hivatkozása.
- [P22] Dosovitskiy, A. et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* arXiv:2010.11929. https://arxiv.org/abs/2010.11929. A Vision Transformer architektúra és empirikus állításainak hivatkozása.

32.4 VIII. ellenőrző pont: célhoz illő neurális modell

32.4.1 1. Cél, előismeret és induló kérdés

Ez a VIII. ellenőrző pont. A 20.4. V. ellenőrző pont: indokolt CNN-választás alfejezetben bevezetett architektúra-döntési dimenziók, a 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés alfejezetben rögzített alkalmazási szerződés és — ha a mélyítést választotta — a 31.4. Generatív felhasználás és értékelés generatív értékelési kerete itt egyetlen, lektorálható

modellválasztási döntési lapba áll össze. Nem új réteget vagy veszteséget vezetünk be: a feladat az, hogy egy új problémahelyzetre a tanult CNN-főútból indokolja a modellcsaládot, megadja az összes tenzoralakot, és minden állításhoz hozzárendelje a bizonyíték típusát.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg; a rubrika öt dimenziója D1 matematikai helyesség, D2 kód és futtathatóság, D3 adat- és mérési protokoll, D4 bizonyíték és nyomon követhetőség, D5 az állítás és korlátai. A PASS feltétele ugyanott: (a) a labor minden `assert`-je és toleranciaellenőrzése a saját futtatásban átmegy, (b) mind az öt dimenzió legalább 3, (c) a riport csak ténylegesen lefuttatott kód kimenetét közli. Itt csak az eltérés: a riport modellválasztási döntési lap; a súlypont a D5.

A három visszahívás: 4 leckével korábban a 31.4. Generatív felhasználás és értékelés a likelihood, mintaminőség, diversity és memorizáció különválasztását adta; 16 leckével korábban a 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés a cél, target, loss, output, metrika, split és inference szerződését; 48 leckével korábban a 20.4. V. ellenőrző pont: indokolt CNN-választás a classic/residual/efficient, scratch/transfer és budget döntéseket. Ebből a három szálból építjük fel az öt döntési dimenziót.

Induló kérdés: egy új feladat érkezik: 30 000 darab, 32×32 pixeles, egycsatornás ipari felvételt kell osztályozni két felületi hiba szerint. Elegendő-e azt mondani, hogy „attentiont használunk, mert az a modern megoldás”? Becslés: nem. A modellválasztás öt, egymástól függetlenül ellenőrizhető kérdésből áll: mi a feladat típusa, mi az input/target szerződés, milyen reprezentációs induktív bias illik az adathoz, mekkora a számítási keret, és milyen bizonyíték támasztja alá az állítást. A 4. szakasz végén a nyitó feladatra kitöltött döntési lapban ezt a becslést tételesen igazoljuk.

32.4.2 2. Definíció és a döntés öt dimenziója

32.4.2.1 Feladattípus: diszkriminatív, generatív, reprezentációs

Definíció szerint a három család célfüggvénye és kimeneti szerződése eltér.

Diszkriminatív modell. A modell a feltételes predikciót közelíti közvetlenül: $f_\theta(x)$ vagy $p_\theta(y \mid x)$ alakban. A target egy osztálycímke vagy folytonos regressziós cél, a loss pedig a predikció és a target közötti diszkriminatív veszteség: cross-entropy, BCE, MSE regresszió. Tipikus CNN-feladatok: osztályozás, detektálás, szegmentálás.

Generatív modell. A modell az input eloszlás $p(x)$ vagy az együttes $p(x, y)$ modelljét tanulja, és új realizációt tud előállítani. A szűkebb rekonstrukciós célú denoising háló a $p(y \mid x)$ feltételes modellt tanulja, ahol y maga a tiszta jel; a generatív értékelésnél a 31.4. Generatív felhasználás és értékelés szerint a likelihood, mintaminőség és diversity külön mérendő.

Reprezentációs modell. A modell $e_\theta : \mathcal{X} \to \mathbb{R}^d$ beágyazást ad, ahol a feladat hasonlósági szerkezete egy távolságban (koszinusz vagy ℓ_2) fejeződik ki. A loss nem közvetlen osztálycímkére, hanem párokra vagy tripletekre épül: contrastive, triplet. A downstream osztályozó vagy retrieval csak a beágyazáson értékelődik.

A határ nem éles: egy denoising CNN felfogható regressziós diszkriminatív modellként is. A besorolást a **célfüggvény és a kiértékelési terv** dönti el, nem a rétegtípus.

32.4.2.2 Input/target szerződés

- Klasszifikáció: $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$, $y \in \{0,\dots,K-1\}^N$ int64, output $Z \in \mathbb{R}^{N\times K}$ logits, loss $\mathrm{CE}(Z,y)$.
- Denoising: $X_{\text{noisy}}, Y_{\text{clean}} \in \mathbb{R}^{N\times C\times H\times W}$, output $\hat{Y}$ azonos alak, loss $\mathrm{MSE}(\hat{Y},Y_{\text{clean}}) = \frac{1}{NCHW}\sum_{n,i,j,k}(\hat{Y}_{n,i,j,k}-Y_{n,i,j,k})^2$ (pixelenkénti átlag, amit a PyTorch `MSELoss` mean redukciója ad). A $\frac{1}{N}\sum_n\sum_{i,j,k}$ alak ehelyett a mintánkénti átlagos SSE-t adná, ami ennek CHW-szerese; a két skála nem keverhető.
- Embedding-triplet: (X_a,X_p,X_n), mindegyik $N\times C\times H\times W$; output $E\in\mathbb{R}^{N\times d}$; loss $\frac{1}{N}\sum_n \max(0,\|E_{a,n}-E_{p,n}\|_2^2-\|E_{a,n}-E_{\text{neg},n}\|_2^2+\alpha)$, ahol $E_{a,n}$, $E_{p,n}$ és $E_{\text{neg},n}$ az anchor, a pozitív és a negatív minta beágyazása.

A batch átlaga az alapértelmezett reduction; a sum-reductiont külön jelöljük.

32.4.2.3 Reprezentációs induktív bias

A CNN lokális recepciós mezőkkel, súlymegosztással és eltolási equivarianciával dolgozik (13.2. Konvolúció és cross-correlation, 16.1. Eltolási equivariance és invariance); akkor előnyös, ha a feladatban a lokális mintázatok eltolás-invariánsan ismétlődnek. Az MLP globálisan köt össze minden pixelt, de nem kap beépített térszerkezetet; kis felbontáson és kevés adaton baselinenak alkalmas. Az attention-család [P21] globális kölcsönhatást tanul, de a pozícióinformáció kódolástól függ; a Vision Transformer [P22] ezt képi tokenekre alkalmazza. Ennek részletes összevetése a fejezet 32.1.–32.3. mélyítéséhez tartozik, és a főúti checkpointnak **nem** előfeltétele. A generatív CNN encoder–decoder szerkezetet és gyakran skip-kapcsolatot használ a finom térbeli részletek visszaadására.

32.4.2.4 Számítási keret

Konvolúciós réteg paraméterszáma $C_{out}\cdot(C_{in}/g)\cdot k_h\cdot k_w + C_{out}$; a MAC-ek száma *batchenként* (N kép)

$$N\cdot C_{out}\cdot H_{out}\cdot W_{out}\cdot(C_{in}/g)\cdot k_h\cdot k_w,$$

képenként ennek N-ed része: $C_{out}\cdot H_{out}\cdot W_{out}\cdot(C_{in}/g)\cdot k_h\cdot k_w$. Affin réteg paraméterszáma $(d+1)\cdot m$; MAC-ek száma batchenként $N\cdot d\cdot m$, képenként $d\cdot m$. A főúti labor kis CPU-s keretben marad; GPU, AMP és külső súly külön jelölt mélyítés.

32.4.2.5 Bizonyíték típusa

Az 1.2. Szakmailag megalapozott modellezési állítások és 8.4. II. ellenőrző pont: korrekt tanulási állítás szerint minden állításhoz meg kell nevezni az ellenőrzés típusát: **analitikus** (alak- vagy paraméterszám-levezetés), **numerikus** (gradiensellenőrzés), **empirikus** (futtatott protokoll nyers eredménnyel), vagy **szakirodalmi** (olvasott forrásszakasz, attribúcióval). A „modern, mert attention” kijelentés egyikbe sem tartozik.

32.4.3 3. Levezetési kötelezettség: CNN-állítás osztályozása és feltétel-ellenőrzése

A kötelező elem nem új tétel bizonyítása, hanem a tanult állítástípusok alkalmazása egy konkrét CNN-feladatra. Tekintsük a következő három állítást.

(A1) *„A GlyphNetMini a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab teszthalmazon* 0,90 *feletti accuracy-t ér el, ha a legjobb val_loss checkpointot értékeljük."*

Osztályozás. Empirikus állítás. Sem az architektúrából, sem a loss definíciójából nem következik; futtatás szükséges.

Feltételek. Adat: GlyphLab 1.0 (generátorverzió 1.0), generátorseed 20261020, 180 jelenet, jelenetenként két nézet, összesen 360 kép; a jelenet-szintű split 120/30/30, azaz 240/60/60 kép. Modell: GlyphNetMini, 2979 tanulható paraméter. Tanítás: Adam, lr $= 10^{-3}$, batch 32, 30 epoch, CE loss, seed 20261020, float32 CPU. Választás: a legjobb checkpoint kizárólag val_loss alapján; a test csak a választás után kerülhet be. Kiértékelés: `model.eval()`, `torch.no_grad()`, int64 target.

Bizonytalanság. Egyetlen seed bitpontos eredménye nem viseli a seed-szórást; a megalapozott állításhoz 3–5 seed és azok szórása kell. A keret 36.2. Ismétlések és eredménybizonytalanság ismétlési protokollját követjük. A teszt 60 képből áll, de csak 30 független jelenetből, ezért a képenkénti accuracy felbontása $1/60 \approx 0{,}017$, a független egység pedig a jelenet; a 0,90-es küszöb teljesülését jelenetszintű bontással és bizonytalansági intervallummal együtt kell jelenteni, önmagában a „0,90 felett" nem elég.

(A2) *„A GlyphNetMini kimeneti alakja $N \times 3$ logits a 24×24 bemenetre."*

Osztályozás. Levezetés. Ellenőrizhető analitikusan, futás nélkül. A rétegsor:

- Conv2d(1→8, $k = 3$, $p = 1$, $s = 1$): $H_{out} = \lfloor (24 + 2 - 3)/1 \rfloor + 1 = 24$; output $(N, 8, 24, 24)$.
- MaxPool2d(2): $(N, 8, 12, 12)$.
- Conv2d(8→16, $k = 3$, $p = 1$, $s = 1$): $(N, 16, 12, 12)$.
- MaxPool2d(2): $(N, 16, 6, 6)$.
- Flatten: $16 \cdot 6 \cdot 6 = 576$ elem.
- Linear(576→3): output $(N, 3)$.

Az alakellenőrzés a 14.2. Több bemeneti és kimeneti csatorna és 14.1. Stride, padding, dilation és outputméret képleteinek közvetlen alkalmazása.

(A3) *„A modell 2979 tanulható paramétert tartalmaz."*

Osztályozás. Levezetés. Számítás:

- Conv1: $8 \cdot 1 \cdot 3 \cdot 3 + 8 = 80$; ebből súly 72, bias 8.
- Conv2: $16 \cdot 8 \cdot 3 \cdot 3 + 16 = 1168$; súly 1152, bias 16.
- Linear: $576 \cdot 3 + 3 = 1731$; súly 1728, bias 3.
- Összesen: $80 + 1168 + 1731 = 2979$. Bias nélkül $72 + 1152 + 1728 = 2952$.

Ez az elemzés mutatja, hogy a három állítás két eltérő bizonyítéktípust igényel: A1 futtatást, A2 és A3 levezetést. A döntési lapon minden állításhoz külön sorban rögzítjük a típust és a feltételek teljesülését.

32.4.4 4. Három problémahelyzet modellspecifikációja

A labor három helyzetet kér. Az **A** a kötelező főút, a **B** és **C** választható mélyítésből is származtatható, de egyik sem előfeltétele a checkoutnak.

32.4.4.1 A) GlyphLab orientáció-osztályozás — diszkriminatív CNN

- **Cél:** orientáció felismerése (0 vertical, 1 horizontal, 2 diagonal).
- **Input/target:** $X \in \mathbb{R}^{N\times 1\times 24\times 24}$ float32, $[0,1]$; $y \in \{0,1,2\}^N$ int64. A modell bemenete kizárólag a képtensor; maszk és doboz nem megy be.
- **Output/loss:** logits $Z \in \mathbb{R}^{N\times 3}$; CrossEntropyLoss (softmax nélkül, 7.2. Softmax és többosztályos cross-entropy).
- **Architektúra:** GlyphNetMini, a 3. szakasz rétegsora.
- **Számítás:** 2979 paraméter; MAC képenként: Conv1 $1\cdot 8\cdot 24^2\cdot 1\cdot 9 = 41472$; Conv2 $1\cdot 16\cdot 12^2\cdot 8\cdot 9 = 165888$; Linear $1\cdot 576\cdot 3 = 1728$; összesen kb. 209 ezer MAC. CPU-n fut.
- **Split és választás:** jelenet-szintű train/val/test; legjobb checkpoint val_loss; test a végén.
- **Bizonyíték:** A2, A3 analitikus; A1 empirikus a futtatás után.

32.4.4.2 B) GlyphLab denoising — generatív/rekonstrukciós CNN

- **Cél:** zajos nézetből a tiszta alakzat rekonstrukciója.
- **Input/target:** $X_{\text{noisy}}, Y_{\text{clean}} \in \mathbb{R}^{N\times 1\times 24\times 24}$ float32; a B ág rögzített zajfüggvénye additív Gauss-zaj: $X_{\text{noisy}} = \text{clip}(Y_{\text{clean}} + \sigma\varepsilon,\, 0, 1)$, $\varepsilon \sim \mathcal{N}(0,1)$, $\sigma = 0{,}1$; a tanítási zaj seedje 20261020, a tesztzajé 20261021, és a $[0,1]$ intervallumra vágás a szerződés része.
- **Output/loss:** $\hat{Y}$ azonos alak; MSE $= \frac{1}{NCHW}\sum(\hat{Y} - Y_{\text{clean}})^2$ (pixelenkénti átlag). Az $\frac{1}{N}\sum_n \sum_{i,j,k}$ összegzés mintánkénti átlagos SSE lenne, ami a pixel-MSE CHW-szerese; a két skálát a riportban meg kell nevezni, és a val-görbét végig ugyanazon a skálán kell követni.
- **Architektúra:** kis encoder–decoder: Conv(1→8, 3, pad=1), ReLU, MaxPool(2), Conv(8→16, 3, pad=1), ReLU, UpSample(2), Conv(16→8, 3, pad=1), ReLU, Conv(8→1, 3, pad=1). A skip-kapcsolatok teljes U-Net formája a 27.2. U-Net, encoder–decoder és skip mélyítés.
- **Metrika:** MSE a val spliten; opcionálisan PSNR $= 10\log_{10}(\text{data_range}^2/\text{MSE})$ az $\frac{1}{NCHW}$ skálájú MSE-vel (a $[0,1]$ képeknél data_range $= 1$). A képenkénti PSNR átlaga és az aggregált MSE-ből számolt PSNR nem ugyanaz: mivel $u \mapsto 10\log_{10}(1/u)$ konvex, Jensen szerint $\frac{1}{N}\sum_n \text{PSNR}_n \geq \text{PSNR}(\frac{1}{N}\sum_n \text{MSE}_n)$, tehát az aggregált MSE-ből számolt PSNR alsó becslés. Itt az aggregált MSE-ből számolt PSNR-t közöljük, és ezt a riportban dokumentáljuk; ha a képenkénti PSNR-átlag is szerepel, külön kell jelölni. Egyetlen látványos rekonstrukció nem bizonyítja a jó eloszlásmodellt; a 31.4. Generatív felhasználás és értékelés diversity- és memorizációellenőrzése akkor szükséges, ha generatív mintákat is állítunk.
- **Bizonyíték:** numerikus gradiensellenőrzés a lossra, majd empirikus val-görbe.

32.4.4.3 C) Két nézet összetartozása — reprezentációs CNN

- **Cél:** a közös scene-hez tartozó két view beágyazásban közelebb legyen, mint a különböző scene-ek.
- **Input/target:** triplet (X_a, X_p, X_n), mindegyik $N\times 1\times 24\times 24$; X_p az anchorral közös scene_id másik view-ja, X_n más scene-ből választott, lehetőleg azonos osztályú nehéz negatív.
- **Output/loss:** $E \in \mathbb{R}^{N\times 16}$; triplet loss $\frac{1}{N}\sum \max(0, \|e_a - e_p\|_2^2 - \|e_a - e_{\text{neg}}\|_2^2 + \alpha)$, a labor C ágában $\alpha = 0{,}2$ (négyzetes ℓ_2 távolságokban értendő rögzített margó).
- **Architektúra:** a GlyphNetMini features blokkja + Linear(576→16).

- **Split:** a független mintavételi egység továbbra is scene_id; triplet építésekor a pozitív pár soha nem lép át spliten. A képenkénti bootstrap itt hibás lenne (5.4. Függő képek, seedek és resampling, 28.3. Retrieval-értékelés és adatfelosztás).
- **Metrika:** a val split párosain Recall@1 az elsődleges, mert a cél a keresés: minden anchorhoz a legközelebbi valódi pár megtalálása; a koszinusz-távolság AUC-ja küszöbfüggetlen kiegészítő diagnosztika, nem helyettesíti a Recall@1-et. Mindkettőt jelenetenkénti párosítással kell számolni.
- **Bizonyíték:** a nem látott spliten mért páros metrika (elsődlegesen Recall@1, kiegészítőleg AUC) a rögzített margó mellett; a triplet loss görbéje csak diagnosztika, a csökkenése önmagában nem bizonyítja a beágyazás minőségét. A gradiens-ellenőrzés numerikus előfeltétel.

32.4.4.4 A nyitó feladat döntési lapja

A nyitó kérdés öt dimenziója tételesen kitöltve (a döntés a bemeneti szerződésből és a tervezett bizonyításból következik, nem a rétegdivatból):

Dimenzió	Döntés a nyitó feladatra	Bizonyíték
Feladattípus	diszkriminatív osztályozás, $K = 2$ felületi hibaosztály; a target int64 címke, a loss CE a logitokon	D1, D2: szerződés- és loss-definíció
Input/target	$X \in \mathbb{R}^{N\times 1\times 32\times 32}$ float32, $[0, 1]$; $y \in \{0, 1\}^N$ int64; output $Z \in \mathbb{R}^{N\times 2}$; maszk és doboz nem megy be	D2: alak- és dtype-szerződés
Induktív bias	a felületi hiba lokális textúra, ezért kis CNN (lokális recepciós mező, súlymegosztás) az első jelölt; MLP baseline kötelező; attention csak mérés után, nem a priori	D4: analitikus érvelés, majd empirikus összevetés
Számítási keret	paraméter- és MAC-képlet a 2. szakaszból; induló modell néhány ezer és néhány tízezer paraméter között, CPU-n mérve	D2: analitikus költségbecslés
Bizonyíték	alak- és paraméterszám-állítás levezetés; accuracy-állítás több seeddel, egyszeri teszten, jelenetszintű bontással	D4, D5: a bizonyíték típusa és a következtetés korlátja

A táblázat a döntési lap minimális váza; a labor A/B/C lapja ezt tölti ki konkrét alakokkal, loss-értékekkel és futtatási eredménnyel. A „modern, mert attention” válasz egyik bizonyítéktípusba sem sorolható, ezért a lapon kitöltetlen maradna.

32.4.5 5. Labor: három specifikáció és futó CNN-kontroll

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a futásidőt a futtatás során mért értékként kell rögzíteni.

A labor átadandó eredménye: a fenti három specifikációt kitöltő döntési lap, valamint az **A** modell egy ténylegesen futtatott kontrollja. Az adatfájlok a függelék A. A GlyphLab adatkészlet szerződését követik: `data/images.npy` $(360, 1, 24, 24)$ float32, `data/labels.npy` $(360,)$ int64, `data/metadata.csv` az `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax` oszlopokkal; a generátor `lab/generate_data.py`, seed 20261020. A `n128_control.py` a `lab/` könyvtárban van, az adatok a `data/` alatt; a `Path(__file__)` alapú feloldás miatt a munkakönyvtártól függetlenül ugyanazokat a fájlokat találja. A kód teljes; a könyv írási környezetében futtatva az alábbi mért kimenetet adta.

```
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn

seed = 20261020
torch.manual_seed(seed)
np.random.seed(seed)
dtype = torch.float32
device = "cpu"

# GlyphLab adat betöltése; az útvonal a szkript könyvtárához igazodik (lab/n128_control.py)
LAB = Path(__file__).resolve().parent
DATA = LAB / "data"
images = np.load(DATA / "images.npy")            # (360,1,24,24) float32
labels = np.load(DATA / "labels.npy")            # (360,) int64
metadata = np.genfromtxt(
    DATA / "metadata.csv", delimiter=",", names=True,
    dtype=None, encoding="utf-8"
)

# A sor szerinti képindexelés csak akkor helyes, ha a metadata index-oszlopa 0..N-1.
assert np.array_equal(metadata["index"].astype(np.int64), np.arange(len(metadata))), \
    "a metadata indexe nem a képtömb sorindexe"
assert (metadata["label"].astype(np.int64) == labels).all(), \
    "a metadata label oszlopa nem egyezik a labels.npy-szel"
train_mask = metadata["split"] == "train"
X_train = torch.tensor(images[train_mask], dtype=dtype, device=device)
y_train = torch.tensor(labels[train_mask], dtype=torch.int64, device=device)

assert X_train.shape == (240, 1, 24, 24), X_train.shape
assert y_train.shape == (240,), y_train.shape
print("X_train shape:", X_train.shape)   # mért: (240, 1, 24, 24)
print("y_train shape:", y_train.shape)   # mért: (240,)

class GlyphNetMini(nn.Module):
    def __init__(self, num_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
```

```
            nn.MaxPool2d(2),
        )
        self.head = nn.Linear(16 * 6 * 6, num_classes)

    def forward(self, x):
        z = self.features(x)
        z = z.reshape(z.size(0), -1)
        return self.head(z)

model = GlyphNetMini().to(device=device, dtype=dtype)
n_params = sum(p.numel() for p in model.parameters())
assert n_params == 2979, n_params
print("parameter count:", n_params)       # mért: 2979

# Forward kontroll eval modban
model.eval()
with torch.no_grad():
    logits = model(X_train[:4])
assert logits.shape == (4, 3), logits.shape
print("logits shape:", logits.shape)      # mért: (4, 3)

# Backward kontroll kis batchen
model.train()
X_batch = X_train[:2].clone().requires_grad_(True)
logits = model(X_batch)
loss = nn.functional.cross_entropy(logits, y_train[:2])
loss.backward()
assert torch.isfinite(loss) and loss.item() > 0
assert X_batch.grad.shape == (2, 1, 24, 24), X_batch.grad.shape
print("loss:", loss.item())               # mért: 1.069029688835144
print("dL/dX shape:", X_batch.grad.shape) # mért: (2, 1, 24, 24)
print("dL/dX max abs:", X_batch.grad.abs().max().item())
```

Mért kimenet (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020):

```
X_train shape: torch.Size([240, 1, 24, 24])
y_train shape: torch.Size([240])
parameter count: 2979
logits shape: torch.Size([4, 3])
loss: 1.069029688835144
dL/dX shape: torch.Size([2, 1, 24, 24])
dL/dX max abs: 0.011990300379693508
```

Futtatási parancs a `lab/` könyvtárból: `python n128_control.py`; a szkript a `Path(__file__)` alapján oldja fel a `data/` útvonalat, ezért más munkakönyvtárból is ugyanazokat a fájlokat találja. Ellenőrzések: `logits.shape == (4,3)`, `X_batch.grad.shape == (2,1,24,24)`, `n_params == 2979`, a loss véges és pozitív, a metadata `index` oszlopa a tömb sorindexe, `label` oszlopa pedig egyezik a `labels.npy`-szel; ezeket a kód `assert`-jei ellenőrzik. A fenti loss-érték a rögzített seed és a megadott környezet mellett mért érték; a jegyzőkönyvbe a hallgató saját futtatásának nyers kimenete kerül. Shape-eltérésnél a modell definíciója hibás; `n_params != 2979` esetén a rétegsor eltér a specifikációtól.

32.4.6 6. Feladatok

32.4.6.1 F1 — Fogalmi, nehézség 1

Előfeltétel: 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés, 31.4. Generatív felhasználás és értékelés. Sorolja be az A, B, C laborfeladatot diszkriminatív, generatív, illetve reprezentációs

családba. Mindegyiknél egy mondatban nevezze meg, hogy a **target és a loss** melyik definíciót kényszeríti ki. Átadandó: a három besorolás indoklással.

32.4.6.2 F2 — Levezetési, nehézség 2

Előfeltétel: 14.1. Stride, padding, dilation és outputméret, 14.2. Több bemeneti és kimeneti csatorna. Vezesse le a GlyphNetMini kimeneti alakját és paraméterszámát a 3. szakasz rétegsorára. Mutassa meg, hogy a Linear bemenete miért $16 \cdot 6 \cdot 6 = 576$, és adja meg a paraméterszámot bias nélkül is. Átadandó: a lépésenkénti levezetés, a biasos és bias nélküli összeg, valamint annak ellenőrzése, hogy a biasok összege 27.

32.4.6.3 F3 — Implementációs, nehézség 2

Előfeltétel: 11.1. Tensor és autograd-életciklus, 11.2. nn.Module és paraméterregisztráció. Írjon teljes Python-kódot, amely létrehozza a GlyphNetMini modellt, betölti a GlyphLab train batch-ét, elvégzi a forwardot és egy backwardot, majd kiírja az összes shape-et, a paraméterszámot és a loss értékét. A kódnak tartalmaznia kell a seed, dtype és device rögzítését. Átadandó: futtatható kód és a tényleges futtatás jegyzőkönyve.

32.4.6.4 F4 — Hibakeresési, nehézség 2

Előfeltétel: 7.2. Softmax és többosztályos cross-entropy, 7.4. Numerikus stabilitás és jelentésmegőrző implementáció. A következő részlet hibás:

```
out = model(x)                          # logits, shape (N,3)
out = torch.softmax(out, dim=1)
loss = nn.CrossEntropyLoss()(out, y)
```

Mi a hiba, mi a numerikus következménye, hogyan javítja, és milyen kis logit-értékkel ellenőrzi a javítást? Átadandó: az ok, a következmény, a javított kód, és a kontrollérték a javított lossra.

32.4.6.5 F5 — Kísérleti, nehézség 3

Előfeltétel: 20.4. V. ellenőrző pont: indokolt CNN-választás, 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés, 36.2. Ismétlések és eredménybizonytalanság. Tervezzen kísérletet, amely a GlyphLab adaton a GlyphNetMini-t egy azonos nagyságrendű paraméterszámú MLP baseline-nal hasonlítja össze. Határozza meg a baseline szerkezetét úgy, hogy a paraméterszám 3000 alatt maradjon, és adja meg a teljes protokollt: split, seedek, választási szabály, metrika, párosítás, korlát. Átadandó: a terv, valamint annak kimondása, hogy a következtetés csak a GlyphLab adatra vonatkozik.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

32.4.7 7. Megoldások és indoklás

32.4.7.1 F1 megoldás

A: **diszkriminatív**, mert a target osztályindex $y \in \{0, 1, 2\}$, a loss cross-entropy a logitokon. B: **generatív/rekonstrukciós**, mert a target maga a tiszta kép, a loss MSE a pixelrekonstrukció; a modell nem osztályoz, hanem jel-térbe képez. C: **reprezentációs**, mert a target triplet szerkezet, a loss a beágyazott távolságok kontrasztív rendezése; nincs közvetlen osztálycímke a veszteségben. Ellenpélda a határokra: ha B-t csak denoisingra használjuk és nem mintát generálunk, akkor a generatív címke a célból, nem a rétegtípusból következik.

32.4.7.2 F2 megoldás

A rétegsor: $24 \to$ Conv(3,pad=1) $\to 24 \to$ MaxPool(2) $\to 12 \to$ Conv(3,pad=1) $\to 12 \to$ MaxPool(2) $\to 6$. A csatornák $1 \to 8 \to 16$. A Flatten $16 \cdot 6 \cdot 6 = 576$ elemet ad. Paraméterek: Conv1 súly $1 \cdot 8 \cdot 9 = 72$ + bias $8 = 80$; Conv2 súly $8 \cdot 16 \cdot 9 = 1152$ + bias $16 = 1168$; Linear súly $576 \cdot 3 = 1728$ + bias $3 = 1731$. Összesen $80 + 1168 + 1731 = 2979$. Bias nélkül $72 + 1152 + 1728 = 2952$. Biasok összege $8 + 16 + 3 = 27 = 2979 - 2952$. Alakellenőrzés: a Flatten hossza megegyezik a Linear súlymátrix oszlopszámával.

32.4.7.3 F3 megoldás

Az 5. szakasz kódja a kért elemeket már tartalmazza: a seed, dtype és device rögzítését, a betöltést, a forwardot és backwardot, valamint a paraméterszám és a gradiens-shape kiírását/ellenőrzését. A hallgatói változatnak ezt kell adaptálnia; a paraméterszám-kiírást nem kell hozzáadnia, mert az az `n_params` sorral már megvan. A minimális elvárt kimenet: `X_train.shape (240,1,24,24)`, `parameter count 2979`, `logits shape (4,3)`, `dL/dX shape (2,1,24,24)`, a loss véges. Shape-ellenőrzés szigorú egyezést kér; a loss értéknél nincs előre rögzített tolerancia, csak a végesség és pozitivitás.

32.4.7.4 F4 megoldás

A `nn.CrossEntropyLoss` PyTorchban a **logitsot** várja, és maga alkalmazza a log-softmaxot. A kétszeres softmax után a loss $-\log(\text{softmax}(\text{softmax}(z))_y)$ lesz a helyes $-\log(\text{softmax}(z)_y)$ helyett. Következmény: a loss más skálán mozog, a gradiens a hibás összetett leképezésből származik, és a kimenet numerikusan ellaposodik. Javítás: a `torch.softmax` sor elhagyása. Kontroll: $z = [2, 1, 0]$, $y = 0$. A helyes loss `F.cross_entropy(torch.tensor([[2.,1.,0.]]), torch.tensor([0]))` értéke $-\ln(\text{softmax}(z)_0) \approx 0{,}4076$; a hibás változat $-\ln(\text{softmax}(\text{softmax}(z))_0) \approx 0{,}7972$ (a projekt venvjében, PyTorch 2.15.0+cpu alatt futtatva: 0,4076 és 0,7972). A hallgató futtassa mindkettőt, és rögzítse a két számot.

32.4.7.5 F5 megoldásminta

Baseline MLP: bemenet $24 \cdot 24 = 576$, egy rejtett réteg 5 neuronnal, ReLU aktivációval, kimenet 3. Paraméterszám: $576 \cdot 5 + 5 + 5 \cdot 3 + 3 = 2903 < 3000$, tehát a GlyphNetMini 2979 paraméteréhez mérhető. Protokoll: GlyphLab 1.0, jelenet-szintű train/val/test 120/30/30; mindkét modell ugyanazon 5 seeddel (pl. 20261020, 20261021, 20261022, 20261023, 20261024) tanul 30 epochig, batch 32, Adam lr $= 10^{-3}$; a legjobb checkpoint val_loss szerint; a test accuracy seedenként párosítva kerül feljegyzésre. Metrika: test accuracy; bizonytalanság: a seedenkénti accuracy-k

átlaga és szórása. A párosítás az azonos adat- és seedlistára vonatkozik; az inicializáció nem bitre azonos, mert a két architektúra RNG-fogyasztása eltér, ezért a seedek közötti párosítás statisztikai, nem azonos kezdősúlyokat jelent. Következtetési határ: az eredmény kizárólag a GlyphLab kis, szintetikus adatára és a rögzített CPU-keretre vonatkozik; nem általános természetesképi vagy termelési állítás. A kísérlet nem lett lefuttatva; a terv a futtatás előtti protokoll, a nyers eredményt a hallgató a saját jegyzőkönyvében rögzíti.

32.4.8 8. Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: **a bővítő generatív vagy transformer labor elvégzése automatikusan kiváltja a főúti checkpoint CNN-kontrollját**. Ez hamis. Ellenpélda: X hallgató csak egy attention-alapú osztályozót ad be a GlyphLab adatra, és a döntési lapon az A modell állításait nem osztályozza, a CNN forward/backward kontrollt nem futtatja. Y hallgató a 3. szakasz három állítását osztályozta, a GlyphNetMini kontrollt lefuttatta, és a B/C specifikációt mélyítésként, külön jelölve csatolta. A checkpoint követelményét Y teljesíti, X nem, hiába lehet az attention-modell érdekesebb kutatási irány. Ugyanígy: a denoising CNN generatív labja nem helyettesíti a diszkriminatív A cél–target–loss szerződésének ellenőrzését, mert a két feladat más targetet és más metrikát használ.

32.4.9 9. Önellenőrzés és korábbi fogalom visszahívása

Mielőtt a döntési lapot lezárom, öt kérdést ellenőrzök. (1) Minden állítás mellett szerepel a bizonyíték típusa — analitikus, numerikus, empirikus vagy szakirodalmi? (2) A loss bemenete logits, a target int64, és softmax nincs a modellben? (3) A split a jelenet-szintű függetlenséget tartja, a triplet/denoising párok nem lépnek át spliteken? (4) A legjobb checkpoint kiválasztása kizárólag val_loss alapján történt, a test csak utána jött? (5) A közölt kód ténylegesen futott-e, és a kimenet mért érték-e, vagy a „VÁRT ÉRTÉK (levezetésből, nem mért)” címkét viseli-e? Ezt a jegyzőkönyvben mindig explicit jelölni kell.

Válaszkulcs. (1) Igen, minden állítás mellett szerepel a bizonyíték típusa. (2) Igen, a loss logitsot kap, a target int64, softmax nincs a modellben. (3) Igen, a split jelenet-szintű, a párok nem lépnek át spliteken. (4) Igen, a checkpointot kizárólag a validációs loss választja, a test csak utána jön. (5) Igen, a kód futott, és a kimenet vagy mért érték, vagy „VÁRT ÉRTÉK” címkével szerepel.

Visszahívás: a 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése CNN-operátor cross-correlationt számol, NCHW bemenettel és OIHW-szerű súlyalakkal; az itt használt Conv2d paraméterszám-képlet ebből következik. A 20.4. V. ellenőrző pont: indokolt CNN-választás három döntési dimenziója itt bővül a feladattípus és a bizonyíték dimenziójával. A 28.4. VII. ellenőrző pont: CNN alkalmazási szerződés szerződésláncának minden eleme a döntési lap külön sora.

32.4.10 10. Rövid összefoglaló

A célhoz illő neurális modell választása nem ízlés, hanem öt dimenzió együttes ellenőrzése: feladattípus (diszkriminatív, generatív, reprezentációs), input/target szerződés, reprezentációs induktív bias, számítási keret és bizonyíték típusa. Egy CNN-állítás lehet empirikus, levezethető vagy szakirodalmi; az állítás csak a feltételeivel együtt értékelhető. A labor három GlyphLab-helyzethez ad specifikációt, és a főúti GlyphNetMini forward/backward kontrollját kéri. A generatív vagy transformer mélyítés hasznos, de nem helyettesíti a CNN-főút ellenőrzését. A lektorálható döntési lap minden állításhoz feltételt, alakot, splitet és ellenőrzési típust rendel.

32.4.11 11. Következő alkalmazás

A 33.1. Inference-szerződés és modellcsomag inference-szerződésre és modellcsomagra lép. Az itt rögzített input/output alak, dtype, eszköz és kiértékelési mód közvetlenül az exportálandó modell szerződésének bemenete lesz: ugyanaz a GlyphNetMini, amely itt a döntési lapon szerepel, ott előre definiált batch- és kimeneti szerződéssel válik csomaggá.

32.4.12 12. Felhasznált források

- [P21] Vaswani et al. (2017). *Attention Is All You Need*. https://arxiv.org/abs/1706.03762. Az attention-család hivatkozása a 3. szakaszban.
- [P22] Dosovitskiy et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. https://arxiv.org/abs/2010.11929. A Vision Transformer hivatkozása a 3. szakaszban.

33 Inference, export és modellátadás

33.1 Inference-szerződés és modellcsomag

33.1.1 1. Cél, előismeret és induló kérdés

A VI. ellenőrző pont után rendelkezünk egy betanított, kiértékelt modellel és egy checkpointtal. Ebben az egységben azt a szerződést rögzítjük, amely nélkül a súlyfájl önmagában nem modell: az architektúra, a preprocessing, a labeltérkép, a verzió, a device és az eval-állapot együtt alkotja az inference-szerződést. A cél az, hogy a hallgató a saját CNN-súlyt egy **új, független processzben** be tudja tölteni, és a visszatöltött modell logitjait kontrollpredikcióval ellenőrizze. Előismeret: a 12.4. III. ellenőrző pont: önállóan tanuló kis háló checkpoint-fogalma, a 23.4. Hibaszeletek és alkalmazási határ kiértékelési szerződése, a 25.4. Nyílt halmaz és ismeretlen input tartományon kívüli input kezelése, valamint a 21.1. Inicializálás és varianciaterjedés seed/reprodukálhatósági fogalma.

Induló kérdés: egy `glyph_cnn.pt` fájl megérkezik egy másik gépre. Elég-e a fájlkiterjesztés és a fájlnév ahhoz, hogy a betöltött modell ugyanazt a predikciót adja, mint a tanító processzben? Becslés: nem. A kiterjesztés nem biztonsági és nem szemantikai garancia; a predikcióhoz a teljes szerződést ismerni kell.

33.1.2 2. Inference-szerződés: definíció, komponensek és jelölések

Definíció (modellcsomag). Egy osztályozó modellcsomag a következő öt komponens együttesét jelenti:

1. **Architektúra**: az `nn.Module` osztálydefiníció, amely a forward számítást meghatározza.
2. **Súly- és állapottérkép (`state_dict`)**: a paraméterek és a tanítás során frissülő pufferek rendezett kulcs–érték leképezése.
3. **Preprocessing-szerződés**: az input séma, dtype, tartomány és bármely előfeldolgozási lépés.
4. **Labeltérkép**: a K kimeneti index és a szemantikus osztály megfeleltetése.
5. **Verzió**: a modellváltozat, valamint a tanítási split és a kiértékelési protokoll azonosítója.

A modellcsomag nem azonos egy exportformátummal: a PyTorch `torch.export`-alapú ONNX exportőre külön lépés, amelynek során a csomag input/output szerződését ellenőrizni kell [D20].

Definíció (state_dict). A `state_dict` a modul paramétereinek és *perzisztens* (`persistent=True`) puffereinek rendezett leképezése (a `register_buffer(..., persistent=False)` kimarad belőle; tartalmazhat tanítás közben nem frissülő puffert is): `state_dict` : $\mathcal{K} \to$ tenzorok, ahol kulcsonként a tenzoralak $(d_1, \ldots, d_{r_k})$ saját, nem egyetlen közös alak; a checkpoint-szerződés ezért paraméterenként alak/dtype sémát és a `persistent` feltételt nevezi meg, ahol $\mathcal{K}$ a rétegnév-mintázatú kulcsok halmaza, például `conv1.weight`, `conv2.bias`, `fc.weight`. A kulcsok a moduldefinícióból származnak; a fájlnév nem hordozza őket. A betöltés `strict=True` módban hibát ad, ha a kulcshalmaz vagy valamelyik alak nem egyezik.

Definíció (inference-szerződés). Az inference-szerződés az a leképezés, amely az érvényes inputtér $X \subseteq \mathbb{R}^{N\times 1\times 24\times 24}$, $[0,1]$ tartományú float32 tensor elemeihez az $N \times 3$ logitmátrixot rendeli. A logitok nem valószínűségek. A döntéshez nem kell softmax: az argmax a logitokon közvetlenül is elvégezhető, mert a softmax monoton, ezért ugyanazt az osztályt választja; a softmax akkor kell, ha valószínűséget vagy konfidenciát akarunk közölni és értelmezni (7.2. Softmax és többosztályos cross-entropy). A labeltérkép a függelék A. A GlyphLab adatkészlet szakaszában rögzített osztálysorrend szerint $0 \mapsto$ vertical, $1 \mapsto$ horizontal, $2 \mapsto$ diagonal.

A **trusted input** kifejezés itt két különböző bizalmi viszonyt takar, ezért a két esetet külön kell kezelni. Egyrészt a checkpoint betöltése deszerializációs kockázatot hordoz: a `torch.load` a Python `pickle` mechanizmusára épül, ezért nem megbízható forrásból származó fájlból tetszőleges kódot futtathat. A `weights_only=True` argumentum ezt a deszerializációt tenzorokra, primitív típusokra és egyszerű konténerekre korlátozza — a hivatalos oktatóoldal [D09] *Save/Load state_dict (Recommended)* szakasza ezt a `state_dict` betöltés ajánlott módjaként tárgyalja. (A PyTorch 2.6 óta a `weights_only` alapértelmezése `True`, ezért a kimaradó argumentum nem engedélyezi automatikusan a pickle-t; 2.6 előtt viszont a kimaradó argumentum a pickle-t engedélyezte, ezért a betöltő kódnak a verziótól függetlenül explicit módon rögzítenie kell a szándékot.) Másrészt a modell bemenete is szerződés alá esik: az input tartománya $[0,1]$, alakja `(N,1,24,24)`, dtype `float32`. A szerződésen kívüli inputról a 25.4. Nyílt halmaz és ismeretlen input mondja ki, hogy a softmax-konfidencia önmagában nem érvényességi bizonyíték.

A **device** a számítás helyét rögzíti. A labor CPU-t használ; a betöltés `map_location="cpu"` argumentummal akkor is CPU-ra helyezi a tenzorokat, ha a checkpoint GPU-n készült. A **train/eval állapot** a modul viselkedését váltja: dropout és batch normalization másként számol a két módban. A BatchNorm running állapota a `state_dict` puffereiben van, de a forward módját a `model.eval()` hívás állítja be.

33.1.3 3. Ekvivalenciafeltétel a kontrollpredikcióhoz

A levezetési kötelezettség pontos állítást kíván arról, mikor egyezik meg az új processzben kapott logit a referenciával.

Tétel (kontrollpredikció bitazonossága). Legyen $f : \mathbb{R}^{N\times 1\times H\times W} \times \Theta \to \mathbb{R}^{N\times K}$ a modul forwardja, ahol Θ a paramétertér, és tegyük fel, hogy az alábbi feltételek mindegyike teljesül:

- **(A1)** A mentés és a betöltés a paramétereket bitveszteségmentesen, azonos dtype-ban őrzi meg.
- **(A2)** Az új processz ugyanazt a modulosztályt példányosítja, és a betöltött `state_dict` kulcsai, alakjai és értékei bitre egyeznek a referenciával.
- **(A3)** A kontrollinput tensor bitre ugyanaz, ugyanazzal a dtype-pal és tartalommal.
- **(A4)** A modul `eval()` állapotban van, a forward minden művelete determinisztikus függvény, és a két futás ugyanazt a könyvtárverziót, backendet, algoritmusválasztást és szálszámot használja. Az azonos eszköz és dtype önmagában nem elegendő a bitazonos inference-hez különböző implementációk között: a könyvtárverzió, a backend, az algoritmusválasztás, a szálszám és a műveleti sorrend is számít, és a `torch.use_deterministic_algorithms(True)` kapcsoló sem tesz minden algebrailag ekvivalens gráfot bitazonossá. A 12.2. Checkpoint és folytathatóság környezeti feltételeit itt is átvesszük; két külön szerződést különítünk el: a *toleráns* szerződés megadott `atol`/`rtol` mellett közelséget ír elő, a *bitazonos* szerződés pedig a fenti öt feltétel együttes teljesülését kéri.
- **(A5)** A számítás ugyanazon a device-on és dtype-ban történik.

Ekkor az új processz logitmátrixa bitre egyezik a referencia logitmátrixszal:

$$Z_{\text{new}} = f(x_{\text{new}}, \theta_{\text{new}}) = f(x_{\text{ref}}, \theta_{\text{ref}}) = Z_{\text{ref}}.$$

Bizonyítás. A forward f a modul rétegeinek függvénykompozíciója. Az (A4) feltétel miatt minden tag determinisztikus: ugyanahhoz az input–paraméter párhoz pontosan egy kimenet tartozik. Az (A3) és (A2) feltétel garantálja, hogy a függvény argumentumai bitre megegyeznek a referenciafutáséval, az (A1) és (A5) feltétel pedig azt, hogy a számítás során nincs olyan átalakítás (dtype-konverzió, device-áthelyezés, félpontos kerekítés), amely a biteket megváltoztatná. A függvényérték egyértelműsége miatt a két kimenet azonos. ■

Ez a bizonyítás tudatosan nem mély állítás: a feltételek pontosan azok a pontok, ahol a gyakorlatban a predikció eltérni szokott. Ha bármelyik feltétel sérül, a tétel nem alkalmazható. Két tipikus sérülés: (A4) sérül, ha dropoutos hálót `train()` módban futtatunk, mert a maszk véletlenszerű; (A5) sérül, ha a forward GPU-ra vagy bfloat16-ra költözik, mert a redukciók sorrendje és a kerekítés eltérhet. Ilyenkor legfeljebb numerikus közelséget állíthatunk, bitazonosságot nem.

33.1.4 4. Eredeti kézi kis példa: state_dict egy lineáris modulon

Az affin réteg a 3.2. Mátrixszorzás és affín réteg szerint $z = Wx + b$. Egy önálló `nn.Linear(2,1)` modul `state_dict`-je két kulcsot tartalmaz (`list(model.state_dict())` ellenőrzi):

Kulcs	Alak	Jelentés
`weight`	$(1, 2)$	a W súlymátrix
`bias`	(1)	a b biasvektor

Ha ugyanezt a `Linear` réteget `nn.Sequential`-be tesszük az 0 indexen, a kulcsok `0.weight` és `0.bias` lesznek; a kulcs tehát a modul felépítését is rögzíti.

Legyen a kézi példában

$$W = \begin{pmatrix} 0{,}5 & 1{,}5 \end{pmatrix}, \qquad b = -0{,}5, \qquad x = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

A logit:

$$z = 0{,}5 \cdot 2 + 1{,}5 \cdot (-1) - 0{,}5 = 1 - 1{,}5 - 0{,}5 = -1.$$

A mentett `state_dict` tehát nem „a modell”, hanem csak a paraméterek:

```
OrderedDict([
    ('weight', tensor([[0.5000, 1.5000]])),
    ('bias',   tensor([-0.5000]))
])
```

Ugyanez a `state_dict` egy másik, nemlineáris moduldefinícióhoz betöltve más függvényt ad, vagy `strict=True` módban kulcshibával leáll. Ez a kézi példa mutatja, hogy a súlyfájl és az architektúra csak együtt határoz meg predikciót.

33.1.5 5. Labor: új processzben visszatöltés és logitellenőrzés

Előfeltételek. Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU; a `lab/` munkakönyvtárban `uv sync`-kel telepített környezet; a laborban használt GlyphLab-fájlok a `data/` alatt (`images.npy`, `masks.npy`, `metadata.csv` közül a szükségesek); a korábbi laborokban előállított checkpoint(ok); a futásidőt a futtatás során mért értékként kell rögzíteni.

Munkakönyvtár: `lab`. A GlyphLab adat a függelék A. A GlyphLab adatkészlet szerződése szerint a `data/images.npy` fájlban van (360 kép, 180 jelenet, jelenetenként két nézet). A labor három fájlt használ: a moduldefiníciót, egy referencia-előállítót és egy új processzben futó ellenőrzőt.

Architektúra. A `GlyphCNN` megegyezik a VI. ellenőrző pontnál használt, 2979 paraméteres kis CNN-nel. A rétegek:

Réteg	Kimenet	Paraméterszám
`conv1 = Conv2d(1,8,3,padding=1)`	$(N, 8, 24, 24)$	$8 \cdot (1 \cdot 9) + 8 = 80$
`pool = MaxPool2d(2)`	$(N, 8, 12, 12)$	0
`conv2 = Conv2d(8,16,3,padding=1)`	$(N, 16, 12, 12)$	$16 \cdot (8 \cdot 9) + 16 = 1168$
`pool`	$(N, 16, 6, 6)$	0
`fc = Linear(16\cdot 36,3)`	$(N, 3)$	$3 \cdot 576 + 3 = 1731$

Az összeg $80 + 1168 + 1731 = 2979$.

Fájl: `n129/glyph_cnn.py`

```
import torch
from torch import nn

class GlyphCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(16 * 6 * 6, 3)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.pool(x)
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        x = x.reshape(x.size(0), -1)
        return self.fc(x)
```

Fájl: `n129/make_reference.py` — a referenciafolyamat: inicializálás rögzített seeddel, kontrollinput a metadata test splitjének első képéből (az `index` mező alapján), eval mód, forward, mentés. A kontrollinputot nem rögzített sorszámról (például `images[300]`) vesszük, mert a sorrend a metadata `index` oszlopától függ; így a split átrendezése esetén sem csúszik el a választás. Az első test-kép kiválasztása önkényes, de determinisztikus és a split-szerződéshez kötött; a kontrollpredikció bármely rögzített inputra elvégezhető.

```
import json, os
import numpy as np
import pandas as pd
import torch
from glyph_cnn import GlyphCNN

torch.manual_seed(20261020)  # csak a referencia inicializáláshoz

model = GlyphCNN()
model.eval()

meta = pd.read_csv("data/metadata.csv")
assert (meta["index"].to_numpy() == np.arange(len(meta))).all(), \
    "a metadata indexe nem a 0..N-1 tartomány permutációja"
first_test = int(meta.loc[meta["split"] == "test", "index"].iloc[0])
print("control index:", first_test)
images = np.load("data/images.npy")          # (360,1,24,24), float32
control = torch.from_numpy(images[first_test:first_test + 1].copy())  # (1,1,24,24)

with torch.no_grad():
    reference_logits = model(control)        # (1,3)

os.makedirs("n129/checkpoints", exist_ok=True)
torch.save(model.state_dict(), "n129/checkpoints/glyph_cnn.pt")
np.save("n129/control_input.npy", control.numpy())
np.save("n129/reference_logits.npy", reference_logits.numpy())

manifest = {
    "model_version": "glyphcnn-1.0",
    "architecture": "GlyphCNN",
    "input": {"shape": [1, 1, 24, 24], "dtype": "float32", "domain": [0.0, 1.0]},
    "labels": {"0": "vertical", "1": "horizontal", "2": "diagonal"},
    "preprocessing": "none; raw NCHW float32 image",
    "training": {"dataset": "GlyphLab 1.0",
                 "split": "scene-level 120/30/30",
                 "seed": 20261020},
    "evaluation_protocol": "eval mode; control = first test image by index",
    "checkpoint": "n129/checkpoints/glyph_cnn.pt",
    "control_input": "n129/control_input.npy",
    "reference_logits": "n129/reference_logits.npy"
}
with open("n129/manifest.json", "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2)

print("saved reference logits:", reference_logits.numpy())
```

Fájl: `n129/verify_inference.py` — ez egy **új processz**: a moduldefiníciót újraimportálja, a checkpointot `weights_only=True` móddal tölti, és bitazonosságot ellenőriz.

```
import json
import numpy as np
import torch
from glyph_cnn import GlyphCNN

with open("n129/manifest.json", encoding="utf-8") as f:
    manifest = json.load(f)

if manifest["architecture"] != "GlyphCNN":
    raise SystemExit("architecture mismatch in manifest")

model = GlyphCNN()  # új példány, új processzben
ckpt = torch.load(manifest["checkpoint"], map_location="cpu", weights_only=True)
model.load_state_dict(ckpt)  # strict=True: kulcs- és alakellenőrzés
```

```
model.eval()

control = torch.from_numpy(np.load("n129/control_input.npy"))
reference = torch.from_numpy(np.load("n129/reference_logits.npy"))

input_ok = (list(control.shape) == manifest["input"]["shape"]
            and str(control.dtype).replace("torch.", "") == manifest["input"]["dtype"])
domain_ok = bool((control >= 0).all().item() and (control <= 1).all().item())
manifest_ok = all(k in manifest for k in ("training", "evaluation_protocol"))
architecture_ok = (manifest["architecture"] == "GlyphCNN"
                   and set(ckpt) == set(model.state_dict()) and input_ok
                   and domain_ok and manifest_ok)
assert architecture_ok, "architektúra- vagy input-séma eltérés a manifestben"

with torch.no_grad():
    reloaded = model(control)

bitwise_equal = torch.equal(reloaded, reference)
max_abs_diff = (reloaded - reference).abs().max().item()

print("architecture_ok:", architecture_ok)
print("domain_ok:", domain_ok)
print("state_dict_keys:", sorted(ckpt.keys()))
print("bitwise_equal:", bitwise_equal)
print("max_abs_diff:", max_abs_diff)
print("reference logits:", reference.numpy())
print("reloaded  logits:", reloaded.numpy())

if not bitwise_equal:
    # Tartalék ág: más környezet eltérő lebegőpontos utat adhat; a logitok O(1)
    # nagyságúak, a float32 eps ~1,2e-7, ezért abszolút 1e-6-os korlát, rtol=0.
    tol = 1e-6
    assert torch.allclose(reloaded, reference, atol=tol, rtol=0.0), max_abs_diff
    print("bitwise equality failed; numerical allclose: True")
```

Futtatási parancsok a `lab` könyvtárból:

```
python n129/make_reference.py
python n129/verify_inference.py
```

A második parancs külön processz, és nem osztozik az első memóriáján. A relatív útvonalak (`data/...`, `n129/checkpoints/...`) a `lab/` munkakönyvtárhoz igazodnak; más munkakönyvtárból a parancsok nem találják meg a fájlokat, ezért a futtatási jegyzőkönyvben rögzíteni kell a munkakönyvtárat. A numerikus kimenet a könyv írási környezetében mért eredmény (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020); a logitértékeket a seedelt inicializálás határozza meg. Mért kimenet:

```
# make_reference.py
control index: 6
saved reference logits: [[0.10620765 0.0475633  0.02692545]]
# verify_inference.py (új processz)
architecture_ok: True
domain_ok: True
state_dict_keys: ['conv1.bias', 'conv1.weight', 'conv2.bias', 'conv2.weight', 'fc.bias',
↪  'fc.weight']
bitwise_equal: True
max_abs_diff: 0.0
reference logits: [[0.10620765 0.0475633  0.02692545]]
reloaded  logits: [[0.10620765 0.0475633  0.02692545]]
```

Az ellenőrzési sor CPU float32 esetén `bitwise_equal: True` és `max_abs_diff: 0.0`; a kontrollkép a metadata szerinti első test-kép (`control index: 6`), a `domain_ok: True` pedig a `[0,1]`

inputtartományt ellenőrzi. A betöltött `state_dict` hat kulcsa (`conv1.bias`, `conv1.weight`, `conv2.bias`, `conv2.weight`, `fc.bias`, `fc.weight`) megegyezik a modell kulcsaival. A fenti két érték a megadott környezetben mért eredmény, nem általános garancia: más könyvtárverzió, backend vagy szálszám esetén csak a toleráns ág állítása érvényes, ezért a riportban a környezetet is rögzíteni kell. A tartalék ág `atol=1e-6`, `rtol=0.0` korlátja tudatos: a logitok nagyságrendje 0,1, a float32 gépi epszilonja $\approx 1{,}2 \cdot 10^{-7}$, öt művelet összegzése után néhány ulp hibát engedünk meg, a relatív toleranciát pedig azért kapcsoljuk ki, mert a kis logitoknál a relatív hiba elfödné az abszolút eltérést. Ha valamelyik feltétel sérül, a hibakeresési sorrend: (A1/A2) a `state_dict` kulcsai, alakjai és értékei; (A3) a kontrollinput bitjei; (A4) az `eval()` állapot, valamint a könyvtárverzió, a backend és a szálszám; (A5) a dtype és a device. A `weights_only=True` betöltés és az architektúra-ellenőrzés mellett a kód `assert`-je a numerikus tartalékellenőrzést is kikényszeríti. A seed a betöltés utáni eval forwardhoz már nem szükséges, mert a forward nem használ véletlent.

33.1.6 6. Feladatok

F1. Fogalmi — a kiterjesztés nem biztonsági garancia (nehézség: 2). Magyarázza el, miért nem következik a `.pt` vagy `.pth` fájlkiterjesztésből a betöltés biztonsága. Különböztesse meg a `weights_only=True` deszerializációs korlátját és a súlyok szakmai megbízhatóságát; adjon egy konkrét ellenpéldát mindkét hiányra. Elvárt átadandó: saját szavas magyarázat és két kis példa, legfeljebb egy oldal.

F2. Levezetési — a tétel feltételeinek megsértése (nehézség: 2). A 3. szakasz tételének (A4) és (A5) feltételét egy-egy konkrét esetben megsértik: (a) dropoutos háló `train()` módban, (b) ugyanaz a súly GPU-float32-re helyezve. Mindkét esetre nevezze meg, melyik feltétel sérül, milyen típusú eltérés keletkezik (sztochasztikus vagy numerikus), és miért nem alkalmazható a bitazonossági következtetés. Elvárt átadandó: két rövid, feltételre hivatkozó érvelés.

F3. Implementációs — manifest-ellenőrző (nehézség: 2). Írjon `validate_manifest` nevű függvényt, amely a `manifest.json` `architecture`, `input.shape`, `input.dtype` és `labels` mezőit összeveti (i) a betöltött `state_dict` kulcshalmazával, (ii) a kontrollinput tensor alakjával és dtype-jával, anélkül hogy forwardot számolna. Elvárt átadandó: teljes kód, egy helyes és egy szándékosan hibás manifestre adott tesztkimenet.

F4. Hibakeresési — két csendes hiba (nehézség: 2). Adott a következő részlet:

```
model = GlyphCNNBN()                # batch normalizációt is tartalmazó változat
state = torch.load(path, weights_only=False)   # biztonsági hiba: engedélyezett pickle
model.load_state_dict(state)
logits = model(single_image)        # train módban maradt
```

Lokalizálja a két hibát, írja le mindkettő következményét, és adja meg a javítást úgy, hogy a bitazonossági tétel feltételei helyreálljanak. Elvárt átadandó: ok–következmény–javítás–visszaellenőrzés négyes mindkét hibára.

F5. Kísérlettervezési — device vagy dtype? (nehézség: 3). Tervezzen kontrollfutást annak eldöntésére, hogy egy megfigyelt numerikus eltérés a CPU→GPU áthelyezésnek vagy a float32→bfloat16 konverziónak tulajdonítható-e. Adjon meg figyelési egységet, baseline-t, faktorokat, metrikát, ismétlésszámot és következtetési határt. Elvárt átadandó: protokollterv, nem feltétlenül futtatott eredmény.

A protokoll rögzítse a hipotézist, a változó tényezőt, a kontrollt, a metrika mértékegységét, a seedeket, az erőforrásigényt és a számokkal megadott döntési szabályt; ha a terv tanítást is tartalmaz, az architektúrát, az optimizert, a tanulási rátát és az epochszámot is.

33.1.7 7. Megoldások és indoklás

F1. A `torch.save` az objektumgráfot a Python `pickle` protokolljával szerializálja, és alapértelmezésben zip-archívumba csomagolja; a kiterjesztés csak fájlnév-konvenció. Egy `evil.pt` tartalmazhat olyan pickle-objektumot, amely betöltéskor kódot futtat; `weights_only=True` esetén a deszerializáló tenzorokra és primitív értékekre korlátozódik, ezért az ilyen fájl hibaüzenettel elutasítható. A `weights_only` alapértelmezése verziófüggvény: a PyTorch 2.6 előtt a kimaradó argumentum engedélyezte a pickle-t, 2.6-tól az alapértelmezés `True`; ezért a betöltő kódnak explicit módon kell rögzítenie a szándékot. Ez azonban **nem** jelenti a súlyok szakmai megbízhatóságát: egy másik architektúra `state_dict`-je, vagy egy ellenségesen módosított, de érvényes tenzorokból álló súly betölthető, és hibás vagy manipulált predikciót ad. A szakmai megbízhatóság forrásellenőrzés, manifest, kulcsegyezés és kontrollpredikció kérdése.

F2. (a) A dropout forwardja véletlen maszkot sorsol, tehát a forward nem determinisztikus függvény: (A4) sérül. Az eltérés sztochasztikus; két futás ugyanazon input és súly mellett is eltérő logitot ad. (b) A GPU-kernel redukciós sorrendje és a lebegőpontos fúziók eltérhetnek a CPU-úttól: (A5) sérül, az eltérés numerikus. A tétel konklúziója bitazonosság, amelynek az öt feltétel együttes teljesülése elégséges feltétele; a feltételek szükségességét a tétel nem állítja, és egyik megsértése esetén csak `allclose` típusú közelség vizsgálható.

F3. Egy lehetséges megoldás:

```
import json
import numpy as np
import torch
from glyph_cnn import GlyphCNN

def validate_manifest(manifest_path, ckpt_path, input_path, model):
    with open(manifest_path, encoding="utf-8") as f:
        m = json.load(f)
    ckpt = torch.load(ckpt_path, map_location="cpu", weights_only=True)
    x = torch.from_numpy(np.load(input_path))

    # A kulcshalmazt a moduldefinícióból, egy példányosított modellből vesszük.
    expected_keys = set(model.state_dict().keys())
    key_ok = set(ckpt.keys()) == expected_keys
    arch_ok = m["architecture"] == type(model).__name__
    shape_ok = tuple(m["input"]["shape"]) == tuple(x.shape)
    dtype_ok = m["input"]["dtype"] == str(x.dtype).replace("torch.", "")
    labels_ok = m["labels"] == {"0": "vertical", "1": "horizontal", "2": "diagonal"}
    return {"arch_ok": arch_ok, "key_ok": key_ok,
            "shape_ok": shape_ok, "dtype_ok": dtype_ok, "labels_ok": labels_ok}

if __name__ == "__main__":
    model = GlyphCNN()
    print(validate_manifest("n129/manifest.json",
                            "n129/checkpoints/glyph_cnn.pt",
                            "n129/control_input.npy", model))
```

A helyes manifestre várt kimenet (levezetésből, nem mért):

```
{'arch_ok': True, 'key_ok': True, 'shape_ok': True, 'dtype_ok': True, 'labels_ok': True}
```

A szándékosan hibás manifestben a `"labels"` `"1"` értékét `"vertical"`-re állítva a várt kimenet:

```
{'arch_ok': True, 'key_ok': True, 'shape_ok': True, 'dtype_ok': True, 'labels_ok': False}
```

A kulcshalmaz-ellenőrzés így a moduldefinícióból származtatott várt halmazt használja, nem kézzel rögzített listát.

F4. Első hiba: a `torch.load` `weights_only=False` mellett fut, ami nem megbízható fájl esetén kódfuttatási kockázat; következménye biztonsági, nem numerikus. Javítás: `torch.load(path, map_location="cpu", weights_only=True)`. Második hiba: a modul `train()` módban maradt. A batch normalizációs réteg train módban a bemenő minibatch statisztikáját használja a `state_dict`-ben tárolt futó átlag helyett; egyetlen kép esetén a normalizálás a kép saját statisztikájából számolódik, így a logit eltér a referenciafolyamattól. Javítás: `model.eval()` a forward előtt. Visszaellenőrzés: `model.training` értéke `False`, a BN-réteg `running_mean`, `running_var` és `num_batches_tracked` pufferei nem változnak a forward alatt (a `track_running_stats` nem puffer, hanem konstruktorargumentum), és a logit bitazonos a referenciával.

F5. Egy lehetséges terv. Kérdés: az eltérést a device vagy a dtype okozza. Baseline: CPU-float32 referencia-logit ugyanabban a környezetben (rögzített torch-verzió, szálszám, `torch.use_deterministic_algorithms(True)` mellett); a tétel bitazonosságot csak azonos környezetben garantál, ezért a baseline önmagával való bitazonos ismétlése a környezet stabilitásának ellenőrzése, nem a device/dtype kérdés eldöntése. Faktorok: device kétszintű (CPU, CUDA) és dtype kétszintű (float32, bfloat16). Rögzített inputok: ugyanaz a `state_dict` és ugyanaz a tíz kép a test split elejéről, rögzített sorrendben, minden cellában; a 2×2 rács celláiban ezen a tízen egy-egy forward fut, azaz cellánként 10 forward, ismétlés nélkül. A futáson belüli determinizmust külön, a CPU-float32 cellában ellenőrizzük: ugyanazon az inputon 10 egymást követő forward bitazonos kell legyen (a seed az eval forwardot nem befolyásolja). Metrika: `max_abs_diff` a CPU-float32 referenciához képenként, valamint a bitazonos találatok aránya (hány képen `max_abs_diff = 0`). A GPU-cellákban rögzíteni kell a TF32-kapcsolókat (`torch.backends.cuda.matmul.allow_tf32`, `torch.backends.cudnn.allow_tf32`), a cuDNN viselkedést (`benchmark=False`, `deterministic=True`), valamint a könyvtárverziót és a szálszámot; e nélkül a GPU-eltérés a device-útvonal és a numerikus mód keveréke, szétválaszthatatlan. A CPU-bfloat16 cella hardverfüggő: a bfloat16 CPU-támogatás (AVX-512 BF16, AMX) gépenként változik, ezért a cellát csak a CPU-típus rögzítésével lehet értelmezni; ha a gép nem támogatja natívan, a szoftveres emuláció lassú és más műveleti utat használhat, így ez a cella külön zavaró tényező, nem tiszta dtype-kontraszt. Következtetési határ: ha a CPU-bfloat16 cella is nagy eltérést mutat, a dtype a domináns magyarázat; ha a GPU-float32 cella tér el, de a CPU-float32 nem, a device útvonala a valószínű ok. A következtetés konzisztenciamintázat, nem ok-okozati bizonyíték; egyetlen GPU-verzió nem általánosít más hardverre.

33.1.8 8. Tipikus tévedés, önellenőrzés és visszahívás

Tipikus tévedés. „A `.pt` kiterjesztés garantálja, hogy a fájl biztonságos, mert a PyTorch saját formátuma.” Ellenpélda: a `.pt` fájl alapértelmezésben zip-archívum, amely a tenzorok mellett pickle-szerializált objektumleírást is tartalmaz, és tetszőleges Python objektumot szerializálhat; a kiterjesztés semmit sem ír elő a tartalomról. A `weights_only=True` a deszerializációt korlátozza, de a betöltött súlyok ettől még származhatnak más architektúrából, más adattartományból vagy módosított forrásból. A biztonság és a modellhelyesség két, egymástól független ellenőrzés.

Önellenőrzés. (1) Fel tudja sorolni a modellcsomag öt komponensét anélkül, hogy a súlyfájlt a modellel azonosítaná? (2) A `load_state_dict` melyik argumentuma ellenőrzi a kulcs- és alakilleszkedést, és mi történik eltérésnél? (3) Miért nem elegendő a seed a betöltött modell reprodukálhatóságához, ha a modult `train()` módban hagyjuk dropoutos réteggel? (4) A bitazonossági tétel melyik feltétele sérül, ha a kontrollinputot minden futás előtt újonnan sorsolt véletlen tenzorral állítjuk elő, és miért nem alkalmas az így kapott eltérés a betöltés helyességének megítélésére?

Válaszkulcs. (1) Az öt komponens: `state_dict`, architektúra, input séma, labeltérkép és verzió. (2) A `strict` argumentum; eltérésnél `RuntimeError` jelzi a hiányzó vagy nem illeszkedő

kulcsot/alakot. (3) A seed nem elég, mert `train()` módban a dropout véletlen maszkja és a BatchNorm train viselkedése miatt a forward nem reprodukálható; `eval()` mód kell. (4) A bitazonossági tétel bitazonos inputot követel; az újonnan sorsolt véletlen tenzor nem bitazonos, ezért a mért eltérés a bemenet különbségét is tartalmazza, és nem a betöltés hibáját mutatja.

Korábbi fogalom visszahívása. A 32.1. Szekvenciamodell és változó hossz szekvenciaállapota megmutatja, hogy a forward alatt képződő rejtett állapot **nem** része a `state_dict`-nek; ezzel szemben a batch normalizáció futó átlaga igen. A 29.1. Lineáris és nemlineáris autoencoder autoencoder-szerződése ugyanígy input- és output-sémát rögzített: a súlyok csak a hozzájuk tartozó leképezéssel együtt értelmesek. A 21.1. Inicializálás és varianciaterjedés inicializálási seedje a referencia-előállítást teszi reprodukálhatóvá, de a betöltés utáni determinisztikus eval forward már nem igényel seedet.

33.1.9 9. Rövid összefoglaló

A modellcsomag a `state_dict`-en túl az architektúrát, az input sémát, a labeltérképet és a verziót is rögzíti. A `weights_only=True` betöltés csak a deszerializációt korlátozza; a súlyok helyességét a manifest, a kulcsegyezés és a kontrollpredikció ellenőrzi. A kontrollpredikció bitazonossági tétele elégséges feltételt mond ki: determinisztikus forward, bitazonos input és paraméterek, azonos dtype, device és eval-állapot, valamint azonos könyvtárverzió, backend, algoritmusválasztás és szálszám mellett a logitok bitre egyeznek; a feltételek hiányában csak toleráns egyezés állítható. A labor egy új processzben visszatöltött GlyphCNN-t vet össze a referencia logitokkal. A gyakori hiba, hogy a fájlkiterjesztést biztonsági garanciának tekintik.

33.1.10 Következő alkalmazás

A 33.2. Batch és kis szolgáltatás egységben ezt a szerződést batch-feldolgozású, kis szolgáltatás-architektúrába helyezzük: a bemeneti köteg méretét, az inference-szolgáltatás hívási szerződését és a késleltetés helyes mérését vizsgáljuk.

33.1.11 Felhasznált források

- [D09] PyTorch contributors (2026). *Saving and Loading Models*. PyTorch 2.14 documentation. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [D20] PyTorch contributors (2026). *torch.onnx: torch.export-based ONNX Exporter*. PyTorch 2.14 documentation. https://docs.pytorch.org/docs/2.14/onnx.html. Hozzáférés: 2026-10-30.

33.2 Batch és kis szolgáltatás

33.2.1 Cél, előismeret és induló kérdés

A 33.1. Inference-szerződés és modellcsomag alfejezetben felépített modellcsomag akkor válik használható eszközzé, ha a súlyokat a tanítási ciklustól elválasztva, kiszolgálási úton is alkalmazni tudjuk. Itt egy helyi batch-inference parancsot építünk, amely a csomag `state_dict`-jét (állapotszótárát) tölti be, validálja a bemenetet, és CSV-formátumú predikciót ír ki. Előismeret: a modellcsomag tartalma, az `eval` mód és az inference-szerződés 33.1. Inference-szerződés és modellcsomag alfejezetből.

Induló kérdés: egy kolléga uint8, 0–255 skálájú `.npy` tömböt ad a modellnek, „hiszen a kép ugyanaz”. A modell numerikusan lefuthat, a predikció mégis érvénytelen; ennek okát az elfogadott bemeneti szerződés formális rögzítésével látjuk be.

33.2.2 Request, batch CLI, model id és local service

Definíció (request). A request a szolgáltatás felé küldött üzenet, amely legalább a modell-azonosítót és a bemenetet tartalmazza. Egyetlen kép esetén a bemenet alakja $X \in \mathbb{R}^{1\times C\times H\times W}$; batch esetén $X \in \mathbb{R}^{N\times C\times H\times W}$, ahol $N \geq 1$ a feldolgozandó minták száma. A request nem azonos a modell forwardjával: a request érvényességét a séma, az eredményét a modell határozza meg.

Definíció (model id). A modellazonosító a csomag verziójához kötött string. A GlyphLab-osztályozó esetében ez `glyph-orientation-v1` (az adatkészletet a függelék A. A GlyphLab adatkészlet szakasza írja le). Az azonosító a szerződés része: a kliens ezzel jelzi, mely csomagot hívja; a szolgáltatás ismeretlen azonosítóra explicit hibát ad, nem „valamilyen” modellt futtat. A súlyok lenyomatához való kötés nem magának a stringnek a tulajdonsága: ehhez a manifestben a checkpoint SHA-256 lenyomatát kell rögzíteni, és a betöltés előtt ellenőrizni. A labor kódja csak a nevet ellenőrzi, ezért a név egyezése önmagában nem bizonyítja, hogy a várt súlyok töltődtek be.

Definíció (batch CLI). Batch parancssori eszköznek nevezzük azt a nem interaktív programot, amely egy bemeneti batch-fájlt olvas, egyetlen előrefelé menetet végez, és a predikciókat fájlba írja. Nincs tréninglépés, nincs paraméterfrissítés, nincs validációs ciklus.

Definíció (local service). Helyi szolgáltatásnak nevezzük azt a folyamatot, amely loopback (helyi hurok) vagy HTTP végponton keresztül fogad requesteket, és válaszüzenetet ad. Ebben az egységben a szolgáltatást csak specifikáció szintjén adjuk meg; a futtatható főút a batch CLI.

Definíció (latency). Egy request késleltetése $T_{lat} = t_{resp} - t_{req}$, ahol t_{req} a kérés beérkezésének, t_{resp} a válasz elküldésének időpontja. Batch esetén T_{batch} a teljes köteg késleltetése; az amortizált minta-késleltetés T_{batch}/N. A kettő nem felcserélhető: a második nem egyetlen kérés válaszideje.

33.2.3 A bemeneti séma: alak, csatorna és értéktartomány

A modell nem „bármilyen képet” fogad, hanem csak a tanítási adat szerződésének megfelelő tenzort. A szerződést formálisan rögzítjük.

Definíció (elfogadott GlyphLab képbatch). Legyen a `glyph-orientation-v1` csomag bemeneti halmaza $\mathcal{X}_{v1}$. A tenzor X pontosan akkor eleme $\mathcal{X}_{v1}$-nek, ha teljesül mindhárom feltétel:

1. **Dtype.** X lebegőpontos típusa `float32`.
2. **Alak.** $X \in \mathbb{R}^{N\times C\times H\times W}$ valamely $N \geq 1$ egészre, továbbá $C = 1$, $H = 24$, $W = 24$; azaz a csatorna-térbeli alak $1 \times 24 \times 24$.
3. **Értéktartomány.** Minden $n \in \{0;\ \ldots;\ N-1\}$, $i, j \in \{0;\ \ldots;\ 23\}$ indexre a pixelértékre

$$X_{n,0,i,j} \in [0;\ 1],$$

   és az érték nem NaN.

A három feltétel más-más hibát véd. Az alakfeltétel a konvolúciós és affin rétegek bemeneti dimenzióját rögzíti; a csatornafeltétel a grayscale képet írja elő, mivel a modell első konvolúciójának súlya $\mathbb{R}^{C_{out}\times 1\times k_h\times k_w}$ alakú, ahol C_{out} a kimeneti csatornák száma, k_h és k_w pedig a kernel térbeli mérete (a bemeneti csatornák száma itt rögzítetten 1). Az értéktartomány a függelék A. A GlyphLab adatkészlet szerződését tükrözi, amelyben a `data/images.npy` `float32` típusú és $[0;\ 1]$-beli. A 0–255 skálájú uint8 kép tehát két feltételt is sért: a dtype-ot és az értéktartományt.

Állítás (a validátor pontosan a szerződést ellenőrzi). A `validate_batch` függvény pontosan akkor tér vissza hiba nélkül, ha a bemenete $\mathcal{X}_{v1}$-ben van.

Bizonyítás. A függvény ágai a definíció konjunkcióinak közvetlen kódolása: ellenőrzi a dtype-ot, az `ndim` értéket, az $N \geq 1$ batchméretet, a csatorna-térbeli C, H, W alakot, a végességet, végül a $[0;\ 1]$ tartományt. Minden feltétel megsértése kivételt vált ki, ezért a hiba nélküli visszatérés minden feltétel teljesülését jelenti. A feltételek definíció szerint pontosan $\mathcal{X}_{v1}$ tagságát adják, tehát a függvény pontosan a halmazba tartozást dönti el. A NaN kizárása a harmadik feltétel külön mondata: a puszta $[0;\ 1]$-tartomány-ellenőrzés nem zárná ki, mert NaN-re minden összehasonlítás hamis; ezért a kód külön végességi ellenőrzést végez, és a definíció is kimondja az érték nem-NaN voltát. ■

A megfelelőség nem numerikus heurisztika, hanem a definícióból következő egyezés. Az viszont külön eldöntendő, hogy egy adott `.npy` fájl ténylegesen eleget tesz-e a szerződésnek; ezt a batch CLI a modellhívás előtt hajtja végre.

33.2.4 Kézi kis példa a validátoron

Tekintsünk négy bemenetet:

- $X^A \in \mathbb{R}^{2\times 1\times 24\times 24}$, dtype `float32`, minden elem 0,5;
- X^B ugyanez a tömb, de dtype `uint8`, értékek 0–255 között;
- X^C dtype `float32`, alak $(1;\ 1;\ 23;\ 24)$;
- X^D dtype `float32`, alak $(1;\ 1;\ 24;\ 24)$, az egyik elem NaN.

A definíció alapján $X^A \in \mathcal{X}_{v1}$, tehát a validátor elfogadja. X^B az 1. feltételt sérti, X^C a 2. feltételt; a validátor mindkettőt elutasítja. X^D a 3. feltételt sérti, mert a NaN nem eleme a $[0;\ 1]$ intervallumnak.

Ennek oktatási szerepe kettős. Egyrészt mutatja, hogy a helyes predikcióhoz a bemeneti szerződés betartása szükséges; másrészt megmutatja, hogy a „majdnem jó” bemenet is elutasítandó, mert a modell hallgatólagosan más eloszláson tanult.

33.2.5 Labor: helyi batch-inference parancs

A labor a 33.1. Inference-szerződés és modellcsomag alfejezetben készített csomagot használja: `package/model.py` tartalmazza a `GlyphCNN` architektúrát, `ckpt/glyph-orientation-v1.pt` a `state_dict`-et. A `GlyphCNN` bemeneti csatornája 1, bemeneti térbeli mérete 24×24, kimenete 3 logit. A futtatás CPU-n történik.

Bemeneti fájl: `data/images.npy`, alak $(360;\ 1;\ 24;\ 24)$, `float32`, $[0;\ 1]$. A parancs:

```
python lab/batch_infer.py \
  --checkpoint ckpt/glyph-orientation-v1.pt \
  --images data/images.npy \
  --output results/predictions.csv \
  --model-id glyph-orientation-v1
```

A fő kód:

```
import argparse, csv, sys, time
from pathlib import Path
import numpy as np
import torch

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from package.model import GlyphCNN

MODEL_ID = "glyph-orientation-v1"
C_EXP, H_EXP, W_EXP = 1, 24, 24

def validate_batch(x: np.ndarray) -> None:
    if not isinstance(x, np.ndarray):
        raise TypeError("a bemenetnek np.ndarray-nek kell lennie")
    if x.dtype != np.float32:
        raise TypeError(f"dtype hiba: {x.dtype}; a várt float32")
    if x.ndim != 4:
        raise ValueError(f"alak hiba: 4D NCHW várt, kapott: {x.ndim}D")
    N, C, H, W = x.shape
    if N == 0:
        raise ValueError("üres batch nem fogadható el")
    if (C, H, W) != (C_EXP, H_EXP, W_EXP):
        raise ValueError(f"alak hiba: 1x24x24 várt, kapott: {C}x{H}x{W}")
    if not np.all(np.isfinite(x)):
        raise ValueError("nem véges pixelérték (NaN vagy inf)")
    if np.any(x < 0.0) or np.any(x > 1.0):
        raise ValueError("pixelértékeknek a [0,1] tartományban kell lenniük")

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--checkpoint", required=True)
    ap.add_argument("--images", required=True)
    ap.add_argument("--output", required=True)
    ap.add_argument("--model-id", default=MODEL_ID)
    args = ap.parse_args()

    if args.model_id != MODEL_ID:
        raise ValueError(f"ismeretlen modellazonosító: {args.model_id}")

    x = np.load(args.images)
    validate_batch(x)

    model = GlyphCNN()
    state = torch.load(args.checkpoint, map_location="cpu", weights_only=True)
    if isinstance(state, dict) and "model_state_dict" in state:
        state = state["model_state_dict"]
    model.load_state_dict(state)
    model.eval()

    t0 = time.perf_counter()
    with torch.inference_mode():
        logits = model(torch.from_numpy(x))
    probs = torch.softmax(logits, dim=1).numpy()
    elapsed = time.perf_counter() - t0

    with open(args.output, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["image_idx", "predicted", "p0", "p1", "p2"])
        for i, (pred, p) in enumerate(zip(probs.argmax(axis=1), probs)):
            w.writerow([i, int(pred), float(p[0]), float(p[1]), float(p[2])])
    print(f"model_id={args.model_id} n={logits.shape[0]} "
          f"batch_latency_s={elapsed:.6f} output={args.output}")
```

```
if __name__ == "__main__":
    main()
```

A kód három ponton tükrözi a 33.1. Inference-szerződés és modellcsomag szerződést: a modell `eval()` állapotba kerül, a paraméterek nem kapnak gradienst, és a betöltés `weights_only=True` módon, csak `state_dict`-et fogad el. A `torch.inference_mode()` azt is rögzíti, hogy itt nem autograd-számítás folyik; ez más feladat, mint az `eval` 33.1. Inference-szerződés és modellcsomag szerinti állapotváltása. A `--model-id` ellenőrzés a csomag azonosítóját védi.

Ellenőrzés. A fenti futás kimenete `results/predictions.csv`. Az ellenőrző szkript:

```
import numpy as np, sys
rows = np.loadtxt(sys.argv[1], delimiter=",", skiprows=1, ndmin=2)
assert rows.shape[0] == 360
assert rows.shape[1] == 5
assert np.all((rows[:, 1] >= 0) & (rows[:, 1] <= 2))
assert np.allclose(rows[:, 2:5].sum(axis=1), 1.0, atol=1e-6)
print("batch inference ellenőrzés sikeres: 360 sor, "
      "predikciók 0..2, valószínűségsorösszegek 1 ± 1e-6")
```

A kimenet a 33.1. Inference-szerződés és modellcsomag alfejezetben rögzített csomagellenőrzési feltételeket ellenőrzi. A batch-inference parancsot a rögzített ellenőrző környezetben futtatva (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál; a checkpoint a 33.1. alfejezet seed 20261020 inicializálásából származik) a mért kimenet:

```
model_id=glyph-orientation-v1 n=360 batch_latency_s=0.007222 output=results/predictions.csv
batch inference ellenőrzés sikeres: 360 sor, predikciók 0..2, valószínűségsorösszegek 1 ± 1e-6
```

A `batch_latency_s` értéke gépfüggő; a 360 soros kimenet és az ellenőrző szkript `assert`-jei a szerződést ellenőrzik. A hallgató a saját futtatásának nyers kimenetét rögzítse a laborjegyzőkönyvben.

33.2.6 Latency és a fejlesztői siker határa

A batch CLI késleltetésének méréséhez rögzíteni kell a bemeneti batch méretét, a hardvert, a melegítő hívások számát és az ismétlésszámot. Egy ismételhető kis mérés például: 10 melegítő hívás, majd 100 mért hívás, mindig ugyanazzal a 360 képes batch-csel; a kimenet p50 és p90 késleltetés. A p50 a medián, a p90 az az érték, amely alá a mért késleltetések legalább 90%-a esik; mindkettő tapasztalati kvantilis, nem modellből származtatott garancia.

A gyakori félreértés az, hogy egy fejlesztői gépen 20 kérésből 20 sikeres válasz a rendelkezésre állást bizonyítja. Nem bizonyítja: 0 hiba 20 próbából az egyoldalú 95%-os Clopper–Pearson felső határ $1 - 0{,}05^{1/20} \approx 0{,}139$, azaz a mérés nem zárja ki, hogy a tényleges hibaarány 13,9% legyen. A „rule of three” (hármasszabály) közelítés erre $3/20 = 0{,}15$, ami felső közelítés; a pontos értéket a binomiális eloszlásból kell számolni. A termelési állításhoz terheléses, párhuzamos klienssel végzett, tartós mérés, erőforrás-korlát és hibatípusok szerinti bontás szükséges.

33.2.7 Loopback API specifikáció

A helyi szolgáltatás szerződése, amelyet a batch feldolgozás mellett egyedi, ismételt kérések kiszolgálására használhatunk:

```
POST /v1/predict
Content-Type: application/json

{
  "model_id": "glyph-orientation-v1",
  "image": {
    "shape": [1, 1, 24, 24],
    "dtype": "float32",
    "values": [0.0, 0.5, 0.25]
  }
}
```

A fenti példa érvényes JSON; a `values` tömböt szemléltetően három elemre rövidítettük, a valódi kérésben pontosan $1 \cdot 1 \cdot 24 \cdot 24 = 576$ szám szerepel, mindegyik $[0;\ 1]$-beli. A `shape` NCHW, a batch dimenzió 1, mert a szolgáltatás egyetlen képet kap. Sikeres válasz:

```
200 OK
{
  "model_id": "glyph-orientation-v1",
  "predicted": 0,
  "probs": [0.92, 0.05, 0.03]
}
```

Sémahiba esetén 422, ismeretlen modellazonosítóra 404, belső hibára 500 a tervezett státuszkód; a 422 üzenetnek meg kell neveznie a sértett feltételt, nem elegendő a „hibás kérés” szöveg. A specifikáció loopback tesztje a szolgáltatásfüggvény közvetlen, HTTP nélküli hívása, így a hálózat nem része az ellenőrzésnek. A fenti ellenőrzés *egységteszt* (közvetlen függvényhívás, HTTP nélkül), nem loopback HTTP-integrációs teszt. Ha a fejezet szolgáltatáskénti átadást ígér, külön lokális HTTP-indítás, valódi kérés, hibakód-/sémaellenőrzés és a szerver lezárása kell; a meglévő tesztet ezért pontosan “egységteszt”-nek nevezzük.

33.2.8 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a különbséget a batch CLI és a local service között. Miért nem bizonyítja a batch CLI sikeres futása azt, hogy a loopback API is rendelkezésre áll?

F2 (levezetési/bizonyítási, nehézség 2). Bizonyítsa be, hogy rögzített N mellett az

$$\mathcal{X}_N = \{X \in \mathbb{R}^{N\times 1\times 24\times 24} : X_{n,0,i,j} \in [0;\ 1]\ \forall n, i, j\}$$

halmaz konvex. Ellenőrizze a bizonyítást a $\lambda = 0$ és $\lambda = 1$ szélsőesetben.

F3 (implementációs, nehézség 2). Írjon `validate_image(x)` függvényt, amely egyetlen $1\times 1\times 24\times 24$ képet ellenőriz, és minden hibát a hiba jellegének megfelelő beépített kivételtípussal, beszédes üzenettel jelez: a típus- és dtype-hibát `TypeError`-ral, az alak- és értékhibát `ValueError`-ral. Tesztelje négy esetre: helyes kép, rossz dtype, rossz alak, $-0{,}1$ érték.

F4 (hibakeresési, nehézség 2). Az alábbi kód miért fogadja el a NaN-t tartalmazó bemenetet?

```python
def validate(x):
    if not (np.any(x < 0.0) or np.any(x > 1.0)):
        return "elfogadva"
    return "elutasítva"
```

Adja meg a javítást és egy olyan bemenetet, amelyen a hibás változat „elfogadva”, a javított változat „elutasítva” eredményt ad.

F5 (kísérleti/transzfer, nehézség 3). Tervezze meg a batch CLI késleltetésmérését úgy, hogy a mérésből p50 és p90 értéket lehessen jelenteni. Írja le a melegítés, ismétlés, batchméret, hardver és kiugró mérések kezelését, valamint azt, mely állítás nem vonható le a mérésből.

33.2.9 Megoldások és indoklás

F1. A batch CLI egyszeri parancs: bemeneti fájlt olvas és kimeneti fájlt ír, nem vár új kéréseket. A local service folyamatban marad, és request–response párokat szolgál ki. A batch CLI sikere a séma, a modellbetöltés és egy batch feldolgozásának helyességét mutatja; az API rendelkezésre állása ezen kívül a szolgáltatásfolyamat elindulásától, a hívási protokolltól és a folyamatos üzemtől függ. Előfordulhat, hogy a CLI fut, de a szerver nem figyel, vagy a request nem éri el a végpontot.

F2. Legyen $X, Y \in \mathcal{X}_N$ és $\lambda \in [0;\ 1]$. Ekkor

$$(\lambda X + (1-\lambda)Y)_{n,0,i,j} = \lambda X_{n,0,i,j} + (1-\lambda)Y_{n,0,i,j}.$$

Mivel $X_{n,0,i,j} \in [0;\ 1]$, $Y_{n,0,i,j} \in [0;\ 1]$ és $\lambda, 1-\lambda \in [0;\ 1]$, mindkét tag nemnegatív, és az összeg felső korlátja

$$\lambda \cdot 1 + (1-\lambda) \cdot 1 = 1.$$

Tehát a konvex kombináció minden eleme $[0;\ 1]$-beli, az alak és a dtype változatlan, így a kombináció $\mathcal{X}_N$-ben van. A $\lambda = 0$ eset Y-t, a $\lambda = 1$ eset X-et adja vissza, mindkettő a feltétel szerint eleme a halmaznak. A konvexitás a pixelérték-intervallum konvexitásának komponensenkénti öröklődése; az alak- és dtypefeltétel rögzített N mellett nem változik a kombinációban.

F3. Egy lehetséges megoldás:

```
import numpy as np

def validate_image(x: np.ndarray) -> None:
    if not isinstance(x, np.ndarray):
        raise TypeError("x nem np.ndarray")
    if x.dtype != np.float32:
        raise TypeError(f"dtype hiba: {x.dtype}")
    if x.shape != (1, 1, 24, 24):
        raise ValueError(f"alak hiba: {x.shape}")
    if not np.all(np.isfinite(x)):
        raise ValueError("nem véges érték")
    if np.any(x < 0.0) or np.any(x > 1.0):
        raise ValueError("érték a [0,1] tartományon kívül")
```

Tesztek: `np.full((1,1,24,24), 0.5, dtype=np.float32)` elfogadva; `np.full((1,1,24,24), 128, dtype=np.uint8)` dtype hibát ad; `np.full((1,1,23,24), 0.5, dtype=np.float32)` alak hibát ad; egy tömb, amelyben az egyik elem $-0{,}1$, értéktartomány-hibát ad. Alakellenőrzés: a `(1,1,24,24)` pontosan a szerződés, az $N = 1$ batch egyes eleme. A kivételtípusok szándékosan követik a Python konvenciót: a típus- és dtype-hiba `TypeError`, az alak- és értékhiba `ValueError`; a négy teszten a megadott kivételtípusok jelennek meg (a kód a rögzített ellenőrző környezetben, NumPy 2.5.4 alatt futott: `OK`, `TypeError`, `ValueError`, `ValueError`).

F4. A hiba oka: NaN esetén mind az `x < 0.0`, mind az `x > 1.0` összehasonlítás hamis, ezért a `not (hamis or hamis)` kifejezés igaz, és a függvény elfogadja a bemenetet. A tartományvizsgálat nem zárja ki a nem véges értékeket. Javítás: először végességi ellenőrzést kell végezni:

```
def ellenoriz_bemenet(x):
    if not np.all(np.isfinite(x)):
        return "elutasítva"
    if np.any(x < 0.0) or np.any(x > 1.0):
        return "elutasítva"
    return "elfogadva"
```

Ellenőrző bemenet: `x = np.array([[[[np.nan]]]], dtype=np.float32)`; a hibás változat „elfogadva”, a javított „elutasítva” eredményt ad.

F5. A protokoll: rögzítsük a CPU-t, a kötegméretet $N = 360$, a checkpointot és az inputfájlt. Hajtsunk végre 10 melegítő hívást, amelyek nem szerepelnek a statisztikában, majd 100 mért hívást; mindegyik előtt ne töltsük újra a modellt. Minden hívásnál mérjük T_{batch}-et `time.perf_counter()`-rel a validált batch modellhívásának kezdetétől a softmax-kimenet elkészültéig. Számítsuk ki a p50 és p90 empirikus kvantilist. A kiugró értékeket ne töröljük, hanem külön jelentsük. Az eredmény csak erre a gépre, erre a PyTorch/NumPy verzióra és erre a batchméretre érvényes; nem vonható le általános GPU- vagy szerverállítás, és nem bizonyít termelési rendelkezésre állást, mert egyetlen folyamat, nincs párhuzamos terhelés és nincs hálózati réteg a mérésben. A terv formális elemei: hipotézis, hogy a p90 a melegítés után stabilan becsülhető; kontroll a rögzített checkpoint, inputfájl, szálszám és CPU; döntési szabály, hogy a p90 95%-os bootstrap-intervallumának félszélessége legfeljebb a pontbecslés 10%-a; erőforrás egy CPU-mag, rögzített szálszám és 30 perc.

33.2.10 Tipikus tévedés, önellenőrzés és visszatekintés

A fejezet kiemelt tévedése: **egy fejlesztői végpont sikere nem termelési rendelkezésre állási bizonyíték.** Ellenpélda: a fejlesztői gépen 20 kérésből 20 sikerül; a termelési környezetben 100 párhuzamos kliensből 5 timeoutot kap. A 20/20-as minta nem cáfolja a 15%-os hibaarányt, és nem tartalmaz párhuzamos terhelést, hálózati késleltetést vagy erőforrás-versenyt. A helyes állítás feltétele: nagyobb minta, párhuzamos kliens, rögzített környezet és hiba szerinti bontás.

Önellenőrzés a labor után: a batch CLI a modellhívás előtt hívta-e a validátort? A betöltés `weights_only=True` volt-e? A modell `eval()` állapotban volt-e? A valószínűségsorok összege 1 volt-e a megadott 10^{-6} toleranciával? A kimeneti fájl tartalmazta-e a `model_id`-t? Ha bármelyik válasz nem, a futtatás nem felel meg a szerződésnek.

Kulcs. Mind az öt kérdésre igen a helyes válasz: a validátor a modellhívás előtt fut, a betöltés `weights_only=True`, a modell `eval()` módban van, a valószínűségsorok összege 1 ± 10^{-6}, és a kimeneti fájl tartalmazza a `model_id`-t. Ha bármelyik nem teljesül, a futtatás érvénytelen, függetlenül a kimenet helyességétől.

Visszatekintés: 32.2. Scaled dot-product attention alfejezetben a batch dimenzió az attention-mátrixok első indexe volt; itt is a N dimenzió az első, és a séma a csatorna–térbeli részt rögzíti. 29.2. Denoising és korrupciós modell alfejezetben a zajos–tiszta párok szerződése a $[0;\ 1]$ intenzitástartományt használta; a mostani validátor ugyanezt a tartományt ellenőrzi a betanított osztályozó bemenetén. 21.2. Learning rate és ütemezés alfejezetben a checkpoint a model, optimizer és scheduler állapotát együtt rögzítette; a batch inference csak a model `state_dict`-et használja, és semmilyen scheduler-állapotot nem futtat.

33.2.11 Rövid összefoglaló

A batch inference a tanítástól elválasztott kiszolgálási út: a modellcsomagból betölti a `state_dict`-et, a séma szerint validálja a bemenetet, és predikciót ír fájlba. Az elfogadott

GlyphLab tenzor `float32`, NCHW alakú, $N \geq 1$, csatorna-térbeli alak $1 \times 24 \times 24$, minden pixel $[0;\ 1]$-beli és véges. A validátor pontosan ezt a halmazt ellenőrzi; a helyes input a formális szerződésből következik. A model id a csomag verziójához kötött, a local service ehhez rendel request–response szerződést. A késleltetés méréséhez melegítés, ismétlés és környezeti rögzítés kell; fejlesztői siker nem termelési rendelkezésre állási bizonyíték.

A következő 33.3. torch.export és ONNX egység a `torch.export` és ONNX exportálást tárgyalja: ott azt vizsgáljuk, hogyan alakítható ez a betöltött Python-modell portálható számítási gráffá. A `state_dict`, `eval` és `weights_only` szerződését a [D09] írja le. Ebben az egységben export API-t nem tárgyaltunk; a [D20] export-alapú ONNX-szakasz a következő egységhez tartozik, az ottani ellenőrzési igénnyel együtt.

Felhasznált források (a PyTorch-dokumentáció verziója 2.14; a hozzáférés dátumát a tétel tartalmazza):

- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [D20] PyTorch contributors (2026). *torch.onnx: torch.export-based ONNX Exporter.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/onnx.html. Hozzáférés: 2026-10-30.

33.3 torch.export és ONNX

33.3.1 Cél, előismeret és induló kérdés

A 33.2. Batch és kis szolgáltatás egység a tanított modellt helyi batch-szolgáltatásként futtatta a teljes PyTorch-keretben. Most azt vizsgáljuk, hogyan adható át a modell olyan környezetnek, ahol a PyTorch-keret nem elérhető, és milyen feltételekkel állítható, hogy az exportált program ugyanazt a függvényt számolja. Előismeret: az `nn.Module` regisztrációja és `state_dict`-je (11.2. nn.Module és paraméterregisztráció), a train/eval viselkedéskülönbség (12.1. Tanítás, validáció és kiértékelési mód, 21.3. Dropout, súlyregularizálás és augmentáció), valamint az input-szerződés és batch-kezelés (33.2. Batch és kis szolgáltatás). Induló kérdés: ha egy exportáló eszköz egyetlen példabemeneten azonos kimenetet ad az eredeti modullal, következik-e, hogy minden megengedett bemeneten azonos? A válasz nem; ebben az alfejezetben pontosan megadjuk, milyen típusú állítást lehet és érdemes ellenőrizni.

33.3.2 Definíciók: export, ExportedProgram, dinamikus alak

Legyen $M : \mathcal{A} \to \mathcal{Y}$ a tanított modul eval módban rögzített viselkedésű függvénye. A fejezet főútján $\mathcal{A}$ a függelék A. A GlyphLab adatkészlet szakaszában leírt $N \times 1 \times 24 \times 24$ alakú, float32, $[0;\ 1]$ értékkészletű GlyphLab-képek halmaza, $\mathcal{Y}$ pedig az $N \times 3$ logit-tér. Az **export** egy E eljárás, amely az M modulból, egy $x_{\text{ref}} \in \mathcal{A}$ referencia-bemenetből és egy $\mathcal{D}$ dinamikusalak-specifikációból egy P **exportált programot** készít (a $\mathcal{D}$ jelölést azért választjuk, hogy ne ütközzön a későbbi D véges kontrollhalmazzal):

$$P = E(M;\ x_{\text{ref}}, \mathcal{D}).$$

A `torch.export` esetén P egy `ExportedProgram`, amely három, elkülönítetten ellenőrizhető részből áll:

Objektum	Tartalom	Ellenőrzés típusa
gráfmodul	a számítási gráf operátorokkal és alakokkal	statikus operátor- és alakelemzés
state_dict	paraméterek és pufferek konstansként	érték-összevetés a forrásmodullal
range_constraints	dinamikus tengelyek korlátai	guard-teszt a határokon

A **dinamikus alak** azt jelenti, hogy egy tengely hossza nem egyetlen, az exportáláskor látott értékre rögzül, hanem egy megadott intervallumból vehet fel értéket. A batch tengelyre például

$$d_{\text{batch}} = \text{Dim}(\text{"batch"}, \min = 2, \max = 16)$$

azt a guardot írja elő, hogy P csak olyan N batch-méretre garantált, amelyre $N \in \{2;\ \ldots;\ 16\}$. A térbeli 24×24 méret a GlyphLab-szerződésben rögzített, ezért statikus maradhat. A guard megsértése nem néma rossz eredményt, hanem explicit constraint-hibát ad. A $N = 1$ batch külön eset: a PyTorch export 0/1-specializációja miatt az exportáláskor megfigyelt 0 vagy 1 értékű egész dimenzió konstanssá válhat, ezért ha a példabemenet egyetlen kép, a batch-tengely nem marad dinamikus. A dinamikus guardhoz tehát legalább kétsoros példabemenet kell, az egyetlen képes inference-et pedig önálló exporttal és teszttel kell támogatni; a részleteket a [D19] dokumentáció rögzíti.

Az **ONNX-exporter** az `ExportedProgram`ból platformfüggetlen gráfcsere-formátumú modellt készít. A 2.14-es dokumentációban az ONNX-exportáló export-alapú (`dynamo`) útvonalát a `torch.onnx.export(..., dynamo=True)` kapcsoló választja [D20]; a hagyományos TorchScript-alapú útvonal ettől külön áll, és a kapcsoló nélküli viselkedést a telepített verzió dokumentációjában kell ellenőrizni. Az ONNX-fájl a súlyokat konstansként tartalmazza; a dinamikus batch tengelyt névvel ellátott `dim_param` jelöli, amely szimbolikus dimenzió: az export nem rendel hozzá számértéket, hanem a gráf bemeneti és kimeneti alakleírásában a `"batch"` név szerepel, és a futtató runtime a tényleges bemeneti alakból köti le az értékét.

33.3.3 Az export ekvivalenciájának numerikus és alakfeltételei

Az alábbi állítás mondja ki, mit ellenőrizhetünk véges kontrollhalmazon. Ez **teszt típusú** állítás, nem univerzális bizonyítás.

Definíció (ellenőrzött ekvivalencia). Legyen $D \subseteq \mathcal{A}$ véges kontrollhalmaz, $\varepsilon \geq 0$ abszolút és $\rho \geq 0$ relatív tolerancia. Azt mondjuk, hogy a P exportált program (ε, ρ, D)-ellenőrzötten ekvivalens M-mel, ha minden $x \in D$ esetén

$$\text{shape}(P(x)) = \text{shape}(M(x)), \qquad \text{dtype}(P(x)) = \text{dtype}(M(x)),$$

és a lapított kimenetekre vett maximumnormában

$$\|P(x) - M(x)\|_\infty \ \leq\ \varepsilon + \rho\, \|M(x)\|_\infty .$$

A három feltétel három különböző hibát fog meg: az alakfeltétel a rejtett broadcasting- vagy reduction-hibát, a dtype-feltétel a véletlen float64/float16 váltást, a norma-egyenlőtlenség pedig az értékeltérést. A ρ tag azért kell, mert nagy abszolút értékű logitoknál a lebegőpontos kerekítés

arányosan nagyobb lehet; a nullához közeli kimenetnél a $\rho\|\cdot\|$ tag eltűnik, és az ε abszolút korlát marad érvényben.

Tétel (affin kontrollredukció). Legyenek $M(x) = Wx+b$ és $P(x) = W'x+b'$ affin leképezések $\mathbb{R}^d \to \mathbb{R}^m$. Ha $P(x) = M(x)$ minden $x \in \{0;\ e_1;\ \dots;\ e_d\}$ kontrollpontban, ahol e_i az i-edik standard bázisvektor, akkor $P = M$ az egész $\mathbb{R}^d$-n.

Bizonyítás. Legyen $\Delta(x) = P(x) - M(x) = (W' - W)x + (b' - b)$. Mivel $\Delta(0) = 0$, ezért $b' = b$. Minden $i = 1, \dots, d$ esetén $\Delta(e_i) = 0$, tehát a $W' - W$ mátrix i-edik oszlopa nullvektor. Mivel ez minden oszlopra igaz, $W' = W$, így $\Delta \equiv 0$. □

A tétel azt mondja meg, hogy **affin** modelleknél a $d + 1$ elemű báziskontroll elegendő a teljes bemeneti tér ellenőrzéséhez; a valós CNN-exportnál a modell nemlineáris, ezért a véges kontrollhalmaz csak a kiválasztott pontokra ad bizonyítékot. A gyakorlati protokoll ezért a deklarált dinamikus batch tengely szélsőértékeit ($N = 2$ és $N = N_{\max}$), egy köztes értéket és a rögzített térbeli méretet fedi le.

33.3.4 Kézi kis példa, majd minimális kód

Legyen $d = 2$, $m = 2$,

$$W = \begin{pmatrix} 1 & -1 \\ 2 & 0{,}5 \end{pmatrix}, \qquad b = \begin{pmatrix} 0{,}1 \\ -0{,}2 \end{pmatrix}, \qquad x = \begin{pmatrix} 0{,}5 \\ 1{,}0 \end{pmatrix}.$$

A szokásos $Z = XW^\top + b$ konvenció szerint egy soros bemenetre:

$$y = \begin{pmatrix} 0{,}5 & 1{,}0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ -1 & 0{,}5 \end{pmatrix} + \begin{pmatrix} 0{,}1 & -0{,}2 \end{pmatrix} = \begin{pmatrix} -0{,}4 & 1{,}3 \end{pmatrix}.$$

A kód ugyanezt adja; a `nn.Linear` súlyalakja $(m, d) = (2;\ 2)$, azaz a fenti W közvetlenül beírható.

```
# munkakönyvtár: examples/n131/
import torch
from torch import nn

torch.manual_seed(20261020)
model = nn.Linear(2, 2, dtype=torch.float32)
with torch.no_grad():
    model.weight.copy_(torch.tensor([[1.0, -1.0], [2.0, 0.5]]))
    model.bias.copy_(torch.tensor([0.1, -0.2]))
model.eval()

x = torch.tensor([[0.5, 1.0]], dtype=torch.float32)
with torch.no_grad():
    y = model(x)
print(y)  # tensor([[-0.4000,  1.3000]])
```

Az export utáni kontrollnak ugyanezt a $-0{,}4; 1{,}3$ értéket kell visszaadnia; ha az exportált program mást ad, az értékhiba, nem kerekítési kérdés.

33.3.5 Labor: kis GlyphLab fej exportja és kontrollösszevetés

A labor egyetlen konvolúciós fejet exportál, amely a GlyphLab input-szerződését használja: float32, $N \times 1 \times 24 \times 24$, $[0;\ 1]$ intenzitás. A modellt az export **előtt** `eval()` módba tesszük, mert az export a pillanatnyi viselkedést rögzíti; a dropout train módú véletlen maszkolása és a BatchNorm train módú batch-statisztikája így nem kerül a gráfba. A kód futtatható; a rögzített ellenőrző környezetben (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020) futtatva a kontrollok 0,0 maximális eltérést adtak, tehát a kimeneti sorok mért értékek. A fejezet a 2.14-es dokumentációra hivatkozik, a mérés viszont 2.15.0+cpu alatt történt; a numerikus egyezés verziófüggő, ezért más verzióval a szkriptet újra kell futtatni, és csak a saját kimenet szabad idézni.

```
# munkakönyvtár: lab/
# Futtatás: python export_check.py
import torch
from torch import nn
from torch.export import Dim, export  # API-ellenőrzés: [D19]

class MiniGlyphHead(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 4, 3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(4, 3),
        )
    def forward(self, x):
        return self.features(x)

torch.manual_seed(20261020)
model = MiniGlyphHead().eval()
x_ref = torch.rand(2, 1, 24, 24, dtype=torch.float32)

with torch.no_grad():
    y_ref = model(x_ref)

batch = Dim("batch", min=2, max=16)
ep = export(model, (x_ref,), dynamic_shapes={"x": {0: batch}})

with torch.no_grad():
    y_exp = ep.module()(x_ref)

print("shape:", y_ref.shape, y_exp.shape)
print("dtype:", y_ref.dtype, y_exp.dtype)
assert y_ref.shape == y_exp.shape, (y_ref.shape, y_exp.shape)
assert y_ref.dtype == y_exp.dtype
err = (y_ref - y_exp).abs().max().item()
print("max_abs_err:", err)
# futtatva: 0.0 (torch 2.15.0+cpu); más verzióval újra kell futtatni
assert err <= 1e-6, err  #   = 1e-6,   = 0: abszolút korlát

# Dinamikus batch-határok és tolerancia-ekvivalencia: a min=2 és max=16
# határon, valamint köztes értéken is.
for n in (2, 5, 16):
    xb = torch.rand(n, 1, 24, 24, dtype=torch.float32)
    with torch.no_grad():
        e = (model(xb) - ep.module()(xb)).abs().max().item()
    assert e <= 1e-6, (n, e)
    print("N =", n, "max_abs_err:", e)
```

```
# Opcionális ONNX-export és onnxruntime-összevetés [D20].
# A futás külön környezetet és onnx/onnxruntime telepítést igényel.
# torch.onnx.export(ep, (x_ref,), "glyph_head.onnx", dynamo=True)
```

Mért kimenet:

```
shape: torch.Size([2, 3]) torch.Size([2, 3])
dtype: torch.float32 torch.float32
max_abs_err: 0.0
N = 2 max_abs_err: 0.0
N = 5 max_abs_err: 0.0
N = 16 max_abs_err: 0.0
```

A kontrollnak három dolgot kell visszaadnia: azonos `shape`-ot (2×3), azonos `dtype`-ot (`torch.float32`) és a definíció szerinti norma-egyenlőtlenséget. A laborban $\rho = 0$, ezért a feltétel $\|P(x) - M(x)\|_\infty \le \varepsilon$ abszolút korláttá egyszerűsödik, $\varepsilon = 10^{-6}$ mellett. Az ONNX-runtime-összevetés ebben a laborban opcionális, külön környezetet és `onnx`/`onnxruntime` telepítést igényel, és a fejezet írásakor nem futott le; a hozzá javasolt $\varepsilon = 10^{-5}$, $\rho = 10^{-5}$ tervezési érték, nem mért eredmény. Két futtatókörnyezet eltérő kernelsorrendet használhat, ezért ott **bitazonosságot nem ígérünk**.

A `runtime` fogalma itt kettéválik: az `ExportedProgram` a PyTorch kereten belül futtatható az `ep.module()` hívással, míg az ONNX-fájl egy külső runtime-ra, például `onnxruntime`-ra bízza a kiértékelést. Az előbbi ellenőrzése a PyTorch-környezetben marad, az utóbbié már a céleszköz numerikus viselkedését is méri.

33.3.6 Feladatok

F1 (fogalmi, 1. nehézség). Mondja meg, miben különbözik az `ExportedProgram` a korábbi `torch.jit.trace` kimenetétől. Adjon pszeudokód-példát olyan modulra, amelyben egy `if x.sum() > 0` elágazás van, és magyarázza el, miért nem szabad a régi TorchScript-trace példákat ellenőrzés nélkül a jelenlegi exportálási útvonal mintájának tekinteni.

F2 (levezetési, 2. nehézség). Bizonyítsa be a fenti affin kontrollredukciós tételt, majd mutassa meg egy ellenpéldával, hogy nemlineáris M-re a $\{0;\ e_1;\ e_2\}$ kontrollhalmaz nem elegendő: válassza az $M(x_1, x_2) = x_1 x_2$ függvényt.

F3 (implementációs, 2. nehézség). Írjon teljes Python-szkriptet, amely egy rögzített súlyú `nn.Linear(2,2)` modult eval módban exportál, majd a $\{0;\ e_1;\ e_2;\ e_1 + e_2\}$ kontrollhalmazon asszertálja, hogy az exportált program kimenete teljesíti a fenti definíció szerinti maximumnorma-egyenlőtlenséget $\varepsilon = 10^{-6}$, $\rho = 0$ mellett, azaz a lapított kimenetek abszolút eltérése legfeljebb 10^{-6}. A seed, a dtype és a futási parancs szerepeljen.

F4 (hibakeresési, 2. nehézség). Az alábbi vázlat hibás: egy dropoutot tartalmazó modellt train módban exportálnak, majd eval módban vetnek össze. A dropout itt szándékosan $p = 1$, ezért a train-módú dropout minden bemenetet nulláz; az exportált gráf ezt a train-módú viselkedést rögzíti.

```
torch.manual_seed(20261020)
model = nn.Sequential(nn.Dropout(p=1.0), MiniGlyphHead()).train()
x = torch.rand(2, 1, 24, 24)
ep = export(model, (x,))        # train módban
model.eval()
y1 = model(x)
y2 = ep.module()(x)
print((y1 - y2).abs().max())    # eltérés várható: az export a train-módú dropoutot rögzítette
```

Lokalizálja a hibát, mondja meg a következményét egy BatchNormos modellnél, és adja meg a javítást.

F5 (kísérleti/transzfer, 3. nehézség). Tervezzen export-ellenőrzési protokollt a GlyphLab `MiniGlyphHead` modelljéhez. Válassza meg a kontrollhalmazt a test splitből, a dinamikus batch-tartományt, a toleranciát az ONNX-runtime-összevetéshez, és indokolja, miért nem elegendő egyetlen tesztkép. Különböztesse meg a numerikus egyezést a szemantikai ekvivalenciától.

33.3.7 Megoldások és indoklás

F1. A `torch.jit.trace` a megadott példabemenet által ténylegesen bejárt utat rögzíti. Ha a modulban `if x.sum() > 0: return 2*x else: return 3*x` szerepel, akkor a pozitív összegű példán a trace csak a `2*x` ágat tartalmazza; a másik ág eltűnik, és negatív összegű bemenetre is `2*x`-et ad. A `torch.export` ezzel szemben nem tekinti a bemenettől függő Python-vezérlést konstans feltételnek: az ilyen adatfüggő vezérlést (`data-dependent control flow`) nem támogatja, ezért az exportálás hibát ad; ha a modellben mégis ilyen ág van, azt `torch.cond`-dal kell kifejezni, vagy az export előtt el kell távolítani. A `torch.export` tehát nem csendben égeti be az egyik ágat, hanem vagy elutasítja a modult, vagy explicit feltételes operátort kér; a teljes gráfra vonatkozó garancia csak ezek után értelmezhető. A régi TorchScript-példa ezért nem alkalmas a jelenlegi export út ellenőrzés nélküli szemléltetésére; a pontos viselkedést a [D19] 2.14-es dokumentációjában kell megerősíteni.

F2. A tétel bizonyítása megegyezik a főszövegben adott levezetéssel: $\Delta(0) = 0$ adja $b' = b$-t, $\Delta(e_i) = 0$ adja a $W' - W$ mátrix i-edik oszlopának nullvektor voltát, amiből $W' = W$. Az ellenpélda: $M(x_1, x_2) = x_1 x_2$ a $\{0;\ e_1;\ e_2\}$ pontokon mindenütt 0-t ad, tehát bármely $P \equiv 0$ függvény ezeken egyezik vele; ugyanakkor $M(1;\ 1) = 1 \neq 0 = P(1;\ 1)$, tehát a báziskontroll nem bizonyítja az ekvivalenciát.

F3. Teljes szkript:

```
# munkakönyvtár: examples/n131/
# Futtatás: python f3_affin_export.py
import torch
from torch import nn
from torch.export import Dim, export  # API-ellenőrzés: [D19]

torch.manual_seed(20261020)
model = nn.Linear(2, 2, dtype=torch.float32)
with torch.no_grad():
    model.weight.copy_(torch.tensor([[1.0, -1.0], [2.0, 0.5]]))
    model.bias.copy_(torch.tensor([0.1, -0.2]))
model.eval()

# A dinamikus batchhez ne egyetlen soros referenciabemenetet adjunk:
# az 1-es méretet a tracer specializálhatja.
x_ref = torch.rand(4, 2, dtype=torch.float32)
batch = Dim("batch", min=2, max=32)
ep = export(model, (x_ref,), dynamic_shapes=({0: batch},))

def check(x):
    with torch.no_grad():
        y_ref = model(x)
        y_exp = ep.module()(x)
    assert y_ref.shape == y_exp.shape, (y_ref.shape, y_exp.shape)
    assert y_ref.dtype == y_exp.dtype
    err = (y_ref - y_exp).abs().max().item()
    assert err <= 1e-6, err
    return err
```

```
# Az affin kontrollhalmaz négy pontját egyetlen N=4 batchben értékeljük.
controls = torch.tensor([
    [0.0, 0.0], [1.0, 0.0], [0.0, 1.0], [1.0, 1.0]
], dtype=torch.float32)
print("controls max error:", check(controls))
# Külön ellenőrizzük a dinamikus batch másik megengedett méretét is.
print("batch 2 max error:", check(controls[:2]))
```

A kód a négy kontrollpontot egy $N = 4$ batchben és két kontrollpontot $N = 2$ méreten ellenőrzi; a szkript a megadott környezetben (Python 3.12.3, torch 2.15.0+cpu, CPU, seed 20261020) futtatva `controls max error: 0.0` és `batch 2 max error: 0.0` kimenetet adott. Az $N = 1$ referenciabemenetet itt tudatosan kerüljük, mert a 0/1-specializáció miatt a dinamikus dimenziót a tracer 1-es méret esetén statikusra specializálhatja; a 0 kontrollpont a bias, az e_i pontok a súlyoszlopok, az $e_1 + e_2$ pedig a teljes összegzés együttes ellenőrzése.

F4. A hiba oka: az export train módban történt, ezért a `torch.export` a modul pillanatnyi, train-módú viselkedését rögzíti: a dropoutot nem távolítja el, hanem a gráfban train-módú operátorként tartja meg, és $p = 1$ mellett ez az operátor minden bemenetet nulláz. Az exportált program tehát a train-viselkedést követi, az `eval()`-ba tett eredeti modell viszont kikapcsolja a dropoutot, így a két kimenet eltér; $p = 1$ mellett az eltérés nem a véletlen súlyoktól függ, mert a train-módú ág determinisztikusan nullát ad. BatchNormos modulnál a hiba jellege ugyanez: train módban a batch-statisztika kerül a gráfba, ezért az exportált program viselkedése a futásidejű batch-től függ, eval módban pedig a running statisztika konstansként rögzül. Javítás: `model.eval()` **az exportálás előtt** kell meghívni, és utána a kontroll-összevetést is eval módban kell végezni.

F5. Protokollvázlat. Megfigyelési egység: egy teszt-jelenet/nézet pár; a függelék A. A GlyphLab adatkészlet szerződése szerint a test split 30 jelenetből 60 képet tartalmaz (jelenetenként két nézet), és a modellválasztásban nem vett részt. Kontrollhalmaz: a 60 tesztképből választott, scene-szinten rétegzett 12 kép, amely a három osztályt egyenlő arányban fedi le. Dinamikus batch: `Dim("batch", min=2, max=16)`, a guard-teszt `N=2`, `N=8`, `N=16` méreteken; `N=1` esetén külön export- és összevetési teszt szükséges. Tolerancia: PyTorch–`ExportedProgram` összevetésre $\varepsilon = 10^{-6}$, $\rho = 10^{-6}$; ONNX-runtime-ra $\varepsilon = 10^{-5}$, $\rho = 10^{-5}$, mert a runtime eltérő kernelsorrendet használhat. Egyetlen tesztkép nem elegendő, mert nem fedi le a batch-tengely szélsőértékeit, és egyetlen minta a modell belső alakátalakításait (a konvolúciós térkép lapítását, a lineáris réteg súlyozott összegzését) sem gyakorolja többféle kimeneti osztályon. A numerikus egyezés a kimeneti tenzorok közelsége; a szemantikai ekvivalencia azt állítaná, hogy a program minden megengedett bemenetre ugyanazt a függvényt valósítja meg. A véges kontrollhalmazon mért egyezés az előbbit igazolja a kiválasztott pontokra, az utóbbit nem bizonyítja. A döntési szabály: mind a 12 kontrollkép és mindhárom batchméret esetén a maximális abszolút eltérés az előírt tolerancián belül marad; erőforrás: CPU, 15 perc, három ismételt export-összevetés.

33.3.8 Tipikus tévedés, önellenőrzés, visszatekintés

Tipikus tévedés. „A `torch.export` ugyanaz, mint a régi `torch.jit.trace/script`, ezért a korábbi TorchScript-példákat változatlanul átvehetjük.” Ez hamis. A trace a példabemenet által bejárt konkrét ágat rögzíti; az inputfüggő elágazás többi ága elveszik. A script a Python nyelv egy részhalmazát támogatja, és a típusokat a script-fordító követeli meg; a jelenlegi `torch.export` ezzel szemben a Dynamo-val a Python bájtkódot elemzi, ezért a támogatott nyelvi elemek és a típushibák kezelése más. A jelenlegi `torch.export` dynamo-alapú trace-re épül, amely a nem támogatott Python-vezérlést teljes gráfra kiterjedő export esetén hibával elutasíthatja. A [D19]

a 2.14-es dokumentációra mutat; a pontos API-viselkedést abban újra kell olvasni; a régi példa átvétele előtt tehát ellenőrizni kell, nem feltételezni.

Önellenőrzés. A kézi affin példa alakjai: W (2; 2), b (2,), x (1; 2), y (1; 2); a számított $-0{,}4$ és $1{,}3$ a kódban ellenőrizhető. A labor seedje `20261020`, dtype-ja `float32`, eszköze CPU; a kimenetek a megadott környezetben mért értékek. A (ε, ρ)-feltétel nullközeli kimenetnél az abszolút, nagy kimenetnél a relatív tagot teszi dominánssá, így a feltétel egyik tartományban sem válik tárgytalanná: nullközeli kimenetnél az abszolút tag, nagy kimenetnél a relatív tag adja a tényleges korlátot.

Visszatekintés. A 21.3. Dropout, súlyregularizálás és augmentáció train/eval megkülönböztetése itt úgy tér vissza, hogy az export a pillanatnyi módot rögzíti; a 29.3. Regularizált latent reprezentáció tanulsága szerint a kis kontrollhalmazon mért nulla hiba nem általános szemantikai garancia; a 32.3. Vision Transformer és hibrid felhasználás modellcsalád-összevetése pedig arra emlékeztet, hogy az export az operátorok szintjén dolgozik: a CNN és a ViT exportja ugyanazt az útvonalat használhatja, de ez csak szükséges feltétel, mert az operátorok támogatottsága mellett a vezérlési szerkezet és az alakspecializáció is az exportálhatóság feltétele.

33.3.9 Rövid összefoglaló

A torch.export eval módban rögzített modulból ExportedProgramot készít, amely a gráfot, a súlyokat és a dinamikus tengelyek guardjait együtt tartalmazza. A bemutatott dinamikus batch a `Dim("batch", min=2, max=16)` korláttal adható meg; a guard megsértése explicit hiba. Az export ekvivalenciáját alak-, dtype- és (ε, ρ)-normafeltételekkel ellenőrizzük véges kontrollhalmazon; affin modellnél a $d + 1$ bázispont igazolja a teljes egyezést, nemlineáris CNN-nél a kontroll csak teszt-bizonyíték. Az ONNX-exporter az export-alapú útvonalból dolgozik, a külső runtime numerikus egyezését külön toleranciával kell mérni. A régi TorchScript-példa nem tekinthető a jelenlegi exportálás mintájának.

33.3.10 Következő lecke és források

A következő egység, 33.4. Üzemeltetés, drift és késő címke, az exportált modellt üzemeltetésbe helyezi: ott az input-szerződés, a driftfigyelés és a későn érkező címkék kérdése épül arra, hogy az átadott program viselkedését milyen feltételekkel ismerjük.

Felhasznált források (a PyTorch-dokumentáció verziója 2.14; a hozzáférés dátumát a tétel tartalmazza):

- [D19] PyTorch contributors (2026). *torch.export*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/export.html. Hozzáférés: 2026-10-30.
- [D20] PyTorch contributors (2026). *torch.onnx: torch.export-based ONNX Exporter*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/onnx.html. Hozzáférés: 2026-10-30.

33.4 Üzemeltetés, drift és késő címke

33.4.1 Cél, előismeret és induló kérdés

A 33.2. Batch és kis szolgáltatás egységben a kiszolgáló oldali szerződést építettük fel: bemeneti séma, kötegelt kérés, késleltetés és modellazonosító. Ott az volt a kérdés, hogy egy kérés *érvényes-e*. Itt az üzemeltetés következő rétegét vizsgáljuk: mi történik a modell kihelyezése után, amikor

a bemeneteket folyamatosan megfigyeljük, de a helyes címkék későn vagy egyáltalán nem állnak rendelkezésre. Az üzemeltető két, gyakran összemosott dolgot akar tudni: (a) *változott-e a beérkező adat vagy a modell viselkedése*, és (b) *romlott-e a modell tényleges pontossága*. Az elsőhöz elég a produkciós napló; a másodikhoz hiteles ground truth kell.

Előismeret: a séma és a kérés fogalma a 33.2. Batch és kis szolgáltatás alfejezetből; a független tesztelés és a modellválasztási bizonyíték 32.4. VIII. ellenőrző pont: célhoz illő neurális modell alfejezetből; a küszöb validációs protokollhoz kötése 29.4. Anomáliapontszám rekonstrukcióból alfejezetből; a céleloszlás és a predikcióeloszlás megkülönböztetése 21.4. Osztályarány, súlyozás és céleloszlás alfejezetből.

Induló kérdés. Egy éles osztályozó az elmúlt órában 1000 kérést kapott. Minden kéréshez rögzítettük az inputot, a predikciót és a konfidenciát, de címke egyelőre nincs. Mit tudunk ebből a megfigyelésből a modell pontosságáról? A 33.4.3. szakasz tétele szerint a ground-truth accuracy értékéről semmit; a megfigyelés csak a bemeneti eloszlás és a modell kimeneti viselkedésének *változásáról* adhat jelzést.

33.4.2 Alapfogalmak és jelölések

Rögzítsük a GlyphLab-osztályozó produkciós környezetét (a függelék A. A GlyphLab adatkészlet szakasza szerint). A modell három osztályt különböztet meg: $K = 3$, a címkék $\{0;\ 1;\ 2\}$ rendre vertical, horizontal, diagonal. A bemeneti kép $x \in \mathbb{R}^{1\times 24\times 24}$: egyetlen képnél CHW, kötegben NCHW alakú, float32, minden pixelérték a $[0;\ 1]$ intervallumban van.

Definíció (produkciós megfigyelés). A T időpontig beérkezett N kérés naplója

$$\mathcal{D}_{\text{prod}} = \left\{ \left(x^{(t)}, \hat{y}^{(t)}, c^{(t)}, m^{(t)}\right) \right\}_{t=1}^{N},$$

ahol $x^{(t)} \in \mathcal{S}$ a bemenet, $\hat{y}^{(t)} \in \{0;\ \ldots;\ K-1\}$ a modell által adott osztálypredikció, $c^{(t)} \in [0;\ 1]$ a predikcióhoz tartozó konfidencia (például a softmax maximuma), $m^{(t)}$ pedig a kérést kiszolgáló modellverzió azonosítója. A napló nem tartalmazza a ground truth-ot.

Definíció (séma, schema). A séma $\mathcal{S}$ a megengedett bemenetek halmaza:

$$\mathcal{S}_{\text{Glyph}} = \left\{ x \in \mathbb{R}^{1\times 24\times 24} :\ 0 \le x_{1,i,j} \le 1\ (i,j = 1,\ldots,24),\ x \text{ minden eleme véges} \right\},$$

ahol x float32 dtype-ú, $(1;\ 24;\ 24)$ alakú tömb, és az első index az egyetlen csatornára mutat. A séma a szerződés statikus része: azt mondja meg, hogy egy input egyáltalán *értelmezhető-e* a modell számára, nem azt, hogy a modell jól osztályozza.

Definíció (input drift). Jelölje P_{train} a tanítóhalmaz eloszlását, P_{prod} pedig a produkciós bemenetek eloszlását. Rögzített D eltérésmérő és $\tau > 0$ küszöb mellett *input driftet* észlelünk, ha $D(P_{\text{train}}, P_{\text{prod}}) > \tau$. Mivel a valódi P_{prod} nem ismert, a gyakorlatban egy $g : \mathcal{S} \to \mathbb{R}^d$ aggregáló leképezést választunk (például per-pixel átlag és variancia), és a produkciós köteg empirikus átlagát hasonlítjuk a tanítóhalmazból számított referenciához. A D, g és τ megválasztása maga is a felügyeleti protokoll része, és mintavételi bizonytalansággal terhelt. A fogalom és a detektálási módszerek áttekintését a [P32], a statisztikai drift-detektorok empirikus összehasonlítását a [P33] tárgyalja.

Definíció (delayed label, késő címke). A t kéréshez tartozó $y^{(t)} \in \{0;\ \ldots;\ K-1\}$ ground truth *késő* címke, ha a kérés időpontjában még nem áll rendelkezésre, hanem egy későbbi időpontban, külön csatornán érkezik meg. A T időpontig megérkezett címkék halmaza $S \subseteq \{1;\ \ldots;\ N\}$, és

általában $|S| \le N$. A napló és a címke összekapcsolásához egy mindkét oldalon szereplő, egyedi `request_id` szükséges.

Definíció (monitoring, rollback, model card). A *monitoring* az üzemeltetési naplókból előre rögzített szabályok szerint számított mutatók és riasztási küszöbök összessége. A *rollback* visszaállítás egy korábbi, mentett modellverzióra, feltéve, hogy a betöltött modell egy független ellenőrző halmazon a mentéskori viselkedést reprodukálja. A *model card* a modellt kísérő, géppel is olvasható dokumentum, amely rögzíti a bemeneti és kimeneti szerződést, a tanító- és tesztadatot, a mért teljesítményt, az ismert korlátokat és a felhasználási feltételeket.

33.4.3 Mely metrika számítható címke nélkül?

A levezetési követelményünk az, hogy pontosan elválasszuk a két metrikacsaládot. A következő állítás azt mondja ki, hogy a ground-truth-ot igénylő metrikák *nem azonosíthatók* a produkciós megfigyelésből.

Tétel 1 (címke nélküli azonosíthatatlanság). Legyen $N \ge 1$, $K \ge 2$, és legyen rögzített a megfigyelhető $(\mathbf{x}, \hat{\mathbf{y}}, \mathbf{c})$ hármas, ahol $\hat{\mathbf{y}} \in \{0;\ \dots;\ K-1\}^N$. Az accuracy

$$A(\mathbf{y}) = \frac{1}{N} \sum_{t=1}^{N} \mathbb{1}\{y_t = \hat{y}_t\}$$

nem függvénye a megfigyelésnek: léteznek olyan $\mathbf{y}_1, \mathbf{y}_2 \in \{0;\ \dots;\ K-1\}^N$ ground-truth vektorok, amelyekre $A(\mathbf{y}_1) \neq A(\mathbf{y}_2)$, miközben a megfigyelhető $(\mathbf{x}, \hat{\mathbf{y}}, \mathbf{c})$ mindkét esetben azonos.

Bizonyítás. Konstruktív. Válasszuk minden t-re $y_{1,t} = \hat{y}_t$ értéket. Ekkor minden indikátor 1, tehát $A(\mathbf{y}_1) = 1$. Mivel $K \ge 2$, minden t-re választhatunk olyan $y_{2,t}$ értéket, amelyre $y_{2,t} \neq \hat{y}_t$. Ekkor minden indikátor 0, tehát $A(\mathbf{y}_2) = 0$. A két realizáció ugyanazt a predikcióvektort $\hat{\mathbf{y}}$ hagyja, és az inputok, illetve a konfidenciák definíció szerint azonosak. A megfigyelésből számított bármely determinisztikus függvény értéke tehát azonos, miközben az accuracy értéke 1, illetve 0; a kettő különbözik. $\square$

A tétel ereje a konstrukció szélsőségességében van: még *teljesen determinisztikus* predikciósorozat mellett sem lehet eldönteni, hogy a modell mindent eltalált vagy mindent elhibázott, ha a címke nem áll rendelkezésre. Ugyanez az érvelés ismételhető a recall, a precision és a macro-F1 esetére: a $y_1 = \hat{y}$ konstrukcióban a szereplő osztályok recallja és precisionje 1, a y_2 konstrukcióban 0, így a macro-F1 is eltér; a nem szereplő osztályokra a recall definiálatlan, ezért a konstrukcióban tegyük fel, hogy minden osztály szerepel a predikciók között. A kalibrációs hiba és a Brier score a teljes előrejelzett valószínűségvektort is felhasználja; ha ezt a napló rögzíti, akkor a c-hez hasonlóan az is a megfigyelés része, és a y-függés miatt a nem-azonosíthatóság ugyanúgy fennáll.

Következmény (metrikák osztályozása). A produkciós naplóból, címke nélkül számítható mutatók azok, amelyek csak a megfigyelhető mezőktől függenek. A $\mathcal{D}_{\text{prod}}$ négyese a minimális mag; az üzemeltetési napló ezen felül kiegészítő mezőket is rögzíthet, például a késleltetést (`latency_ms`) és a HTTP-státuszt. Ezekre támaszkodva címke nélkül számítható a kérésszám, a sémaérvényességi arány, az inputeloszlás-mutatók, a predikcióeloszlás, a konfidenciaeloszlás, a késleltetés-percentilisek és a szolgáltatási hibaarány. A *tényleges teljesítmény* mutatói — accuracy, per-class recall, macro-F1, kalibrációs hiba — a hiteles ground truth-ot igénylik. Az input drift kimutatása ezért indikátor, amely további vizsgálatot vált ki; önmagában nem teljesítménymérés.

33.4.4 Kézi példa: drift, predikció és késő címke

A kézi példában a legegyszerűbb skalár összegzést választjuk: egy képet a pixelei átlagintenzitásával jellemzünk, tehát $g(x) = \frac{1}{24 \cdot 24} \sum_{i,j} x_{1,i,j}$. A tanítóhalmaz képeinek átlagintenzitás-referenciája $\mu_{\text{ref}} = 0{,}50$, a képenkénti átlagintenzitás szórása $\sigma_{\text{ref}} = 0{,}10$. Egy $N = 4$ kérésből álló produkciós köteg képenkénti átlagintenzitása:

$$\bar{x}^{(1)} = 0{,}48, \quad \bar{x}^{(2)} = 0{,}52, \quad \bar{x}^{(3)} = 0{,}61, \quad \bar{x}^{(4)} = 0{,}63.$$

A normalizált abszolút eltérés kötegátlaga

$$D = \frac{1}{4} \sum_{t=1}^{4} \left| \frac{\bar{x}^{(t)} - \mu_{\text{ref}}}{\sigma_{\text{ref}}} \right| = \frac{1}{4}(0{,}2 + 0{,}2 + 1{,}1 + 1{,}3) = 0{,}7.$$

A labor `drift_metric` függvénye másik g-t használ: a köteg per-pixel átlagvektorát hasonlítja a per-pixel referenciához, majd a pixelek közötti átlagot veszi. A két mutató értéke és skálája eltérő, ezért a τ küszöb csak a hozzá tartozó g mellett érvényes; a 33.4.6. szakasz laborjának 0,5-es küszöbe a per-pixel változat saját konfigurációja, nem a most kiszámított 0,7 összehasonlítási alapja.

Ha a riasztási küszöb $\tau = 0{,}5$, akkor a monitoring driftet jelez. A modell mind a négy képre $\hat{y}^{(t)} = 0$ (vertical) predikciót ad, rendre 0,98; 0,97; 0,96; 0,99 konfidenciával. Három nappal később megérkezik a ground truth: $y = (0;\ 0;\ 1;\ 1)$. A címkézett accuracy ezen a négy kérésen $2/4 = 0{,}50$. A példa azt mutatja, hogy a drift-riasztás és a tényleges hiba együtt is felléphet; a 33.4.9. szakasz viszont azt mutatja, hogy ez nem automatikus oksági kapcsolat.

A teljes monitoring-számítást szemléltető kód:

```
# lab/n132/monitor_demo.py
import numpy as np

mu_ref, sigma_ref = 0.50, 0.10

# Szintetikus produkciós megfigyelés rögzített értékekkel.
means = np.array([0.48, 0.52, 0.61, 0.63], dtype=np.float32)
pred = np.array([0, 0, 0, 0], dtype=np.int64)
conf = np.array([0.98, 0.97, 0.96, 0.99], dtype=np.float32)
y_late = np.array([0, 0, 1, 1], dtype=np.int64)

D = float(np.mean(np.abs((means - mu_ref) / sigma_ref)))
assert abs(D - 0.7) < 1e-6, D
print(f"drift_mutató = {D:.3f}; riasztás = {D > 0.5}")

acc = float(np.mean(pred == y_late))
assert abs(acc - 0.5) < 1e-12, acc
print(f"címkézett accuracy (N=4) = {acc:.3f}")
# Mért kimenet (NumPy 2.5.4):
# drift_mutató = 0.700; riasztás = True
# címkézett accuracy (N=4) = 0.500
```

A mért kimenet Python 3.12.3, NumPy 2.5.4, CPU és alapértelmezett szálbeállítás mellett született; a kód véletlent nem használ, ezért seed sem kell hozzá.

33.4.5 Delayed label és a naiv átlag torzítása

Ha minden kéréshez megérkezne a címke, akkor a produkciós accuracy torzítatlan empirikus becslését kapnánk. A késő címke azonban szelekciót visz be. Legyen $S \subseteq \{1;\ \ldots;\ N\}$ a megérkezett címkék halmaza, és tekintsük a naiv becslőt:

$$\hat{A}_S = \frac{1}{|S|} \sum_{t \in S} \mathbb{1}\{y_t = \hat{y}_t\}.$$

Ez akkor torzítatlan a teljes accuracy-ra nézve, ha a megfigyelési indikátor $I_t = \mathbb{1}\{t \in S\}$ független a hibaeseménytől. Ha viszont a címkézési folyamat a hibás eseteket nagyobb arányban vizsgálja felül, $\hat{A}_S$ szisztematikusan alulbecsli a valós pontosságot; ha csak a gyorsan lezárt, könnyű esetek kapnak címkét, felülbecsli.

Kézi ellenpélda a függetlenség hiányára. Legyen $N = 6$, minden predikció 0, a ground truth $y = (0;\ 1;\ 1;\ 1;\ 1;\ 1)$, tehát a teljes accuracy $1/6$. Tegyük fel, hogy a címke csak azokra az esetekre érkezik meg, ahol a predikció hibás, mert csak a hibákat vizsgálják felül. Ekkor $S = \{2;\ 3;\ 4;\ 5;\ 6\}$, és $\hat{A}_S = 0/5 = 0$, ami a 0 irányába torzít. Ha viszont a címke csak a hibátlan esetekre érkezik, $S = \{1\}$ és $\hat{A}_S = 1$. A monitoring-protokollnak ezért rögzítenie kell a címkeforrás szelekciós szabályát, és a csúszóablakos metrikát csak akkor szabad értelmezni, ha a lefedettség $|S|/N$ elég nagy, és a szelekció függetlensége legalább heurisztikusan indokolható.

33.4.6 Labor: GlyphOps követési és visszaállítási terv

A feladat egy fiktív üzemeltetési terv a GlyphLab-osztályozóhoz, amelyet a korábbi egységekben tanítottunk. Az alábbi terv a 12.2. Checkpoint és folytathatóság alfejezetben bevezetett checkpoint-szerződésre és a [D09] forrásban leírt `state_dict`-alapú mentési/betöltési eljárásra épül; a konkrét fájlok és küszöbök a labor saját konfigurációját alkotják.

Modellszerződés. `model_id = "glyphnet-v2.0"`. Bemenet: float32, egyetlen képnél CHW, kötegben NCHW $(1;\ 24;\ 24)$, értékkészlet $[0;\ 1]$. Kimenet: 3 logit, az osztályok $\{0;\ 1;\ 2\}$. A modell a GlyphLab train splitjén tanult, a legjobb checkpoint a val loss alapján lett kiválasztva; a tesztkiértékelés a clean test 60 képén történt.

Naplózási séma. Minden kérés egy JSONL sor:

```
{"request_id": "r-0001", "ts": "2026-10-20T09:00:00Z", "schema_ok": true,
 "input_mean": 0.52, "input_var": 0.08, "pred_label": 0,
 "pred_conf": 0.97, "model_id": "glyphnet-v2.0",
 "latency_ms": 3.1, "http_status": 200}
```

A késő címkék külön JSONL fájlban érkeznek:

```
{"request_id": "r-0001", "label": 0, "label_ts": "2026-10-23T11:20:00Z", "source": "reviewer"}
```

Követési mutatók. (1) Sémaérvényességi arány 24 órás ablakon; (2) drift-mutató a per-pixel átlag normalizált abszolút eltéréséből, napi $N \geq 100$ kérésre; (3) predikcióeloszlás és átlagos konfidencia; (4) a 3 napnál idősebb, lezárt kérésekre csatlakoztatott címkézett accuracy 30 napos csúszó ablakon, csak akkor, ha a join lefedettség legalább 0,8 és a mintaméret legalább 200. A 30 napos csúszóablakot és a lefedettséget a naplót aggregáló hívó réteg számolja; a `rollback_decision` a kapott `join_coverage`, `labeled_n` és `daily_n` értékeket ellenőrzi.

Riasztási küszöbök. Sémaérvényesség 24 h alatt $< 0{,}99$; drift-mutató $> 0{,}5$ (csak napi $N \geq 100$ mellett értelmezett); címkézett accuracy $< 0{,}85$ (csak ≥ 200 lezárt minta és $\geq 0{,}8$ join-lefedettség mellett); HTTP 5xx arány $> 0{,}05$.

Rollback-terv. Bármely riasztás emberi vizsgálatot indít. Automatikus visszaállítás csak a címkézett accuracy vagy a sémaérvényesség riasztására történik; a drift- és az 5xx-riasztás önmagában csak vizsgálatot vált ki. A visszaállítás lépései: (1) a `checkpoints/glyphnet-v1.0.pt` state_dict betöltése a deklarált `GlyphNet` architektúraosztályba; (2) `model.eval()` mód; (3) a 60 képes validációs spliten a top-1 predikciók egyezésének ellenőrzése a checkpoint mentésekor rögzített referencianaplóval; a referencianapló ugyanazon a spliten, rögzített képsorrendben, `eval()` módban és a mentéskori PyTorch-verzióval készül, a checkpointtal együtt verziózott artefakt, és a rollback után ugyanabban a sorrendben kell újraszámolni (a validációs split nem része a tesztkiértékelésnek); (4) a `model_id` csere és a rollback esemény naplózása. Az (1)–(3) nélkül a visszaállítás nem minősül ellenőrzöttnek.

A monitoring-politika minimális kódja:

```
# lab/n132/rollback_policy.py
import numpy as np

EXPECTED_SHAPE = (1, 24, 24)

def validate_schema(x):
    if not isinstance(x, np.ndarray):
        return False, "nem numpy.ndarray"
    if x.dtype != np.float32:
        return False, f"dtype={x.dtype}, várt float32"
    if x.shape != EXPECTED_SHAPE:
        return False, f"shape={x.shape}, várt {EXPECTED_SHAPE}"
    if x.min() < 0.0 or x.max() > 1.0:
        return False, "érték [0,1]-en kívül"
    if not np.all(np.isfinite(x)):
        return False, "nem véges érték"
    return True, "OK"

def drift_metric(x_batch, mu_ref, sigma_ref, eps=1e-6):
    # x_batch: (N,1,24,24) float32; mu_ref, sigma_ref: (1,24,24) float32
    mu_prod = x_batch.mean(axis=0, dtype=np.float64).astype(np.float32)
    z = np.abs(mu_prod - mu_ref) / (sigma_ref + eps)
    return float(z.mean())

def join_labels(requests, labels):
    # Többszörös címke esetén a legkésőbbi label_ts érvényes.
    latest = {}
    for lab in labels:
        prev = latest.get(lab["request_id"])
        if prev is None or lab["label_ts"] > prev["label_ts"]:
            latest[lab["request_id"]] = lab
    joined = []
    for r in requests:  # a kimenet a kérések sorrendjét követi
        lab = latest.get(r["request_id"])
        if lab is not None:
            joined.append((r["request_id"], r["pred_label"], lab["label"]))
    return joined

def rollback_decision(metrics, policy):
    rollback, investigate = [], []
    if metrics["schema_valid_rate_24h"] < policy["min_schema_valid_rate"]:
        rollback.append("alacsony sémaérvényesség")
    if (metrics["labeled_n"] >= policy["min_labeled_n"]
            and metrics["join_coverage"] >= policy["min_join_coverage"]
            and metrics["labeled_accuracy"] < policy["min_accuracy"]):
        rollback.append("alacsony címkézett accuracy")
    if (metrics["daily_n"] >= policy["min_drift_n"]
            and metrics["drift"] > policy["max_drift"]):
        investigate.append("input drift riasztás")
```

```
    if metrics["http_5xx_rate"] > policy["max_5xx_rate"]:
        investigate.append("magas 5xx arány")
    if rollback:
        return "rollback", rollback + investigate
    if investigate:
        return "investigate", investigate
    return "keep", []
```

Példa a hívó oldal `policy` szótárára: `{"min_schema_valid_rate": 0.99, "max_drift": 0.5, "min_drift_n": 100, "min_labeled_n": 200, "min_join_coverage": 0.8, "min_accuracy": 0.85, "max_5xx_rate": 0.05}`. A döntési szabály háromértékű: `rollback` csak a sémaérvényesség vagy a címkézett accuracy riasztására; a drift és az 5xx csak `investigate`; egyébként `keep`. A függvény a címkézett accuracy-t csak a lefedettségi és mintaméret-kapun át értékeli, ezért a ritka vagy szelektált címke nem vált ki automatikus visszaállítást.

A fenti kód csak függvénydefiníciókat tartalmaz, futtatva nem ír ki semmit; a viselkedését a 33.4.8. szakasz F3 és F4 tesztjei ellenőrzik, a drift kézi számítását pedig a 33.4.4. szakasz példája mutatja. A labor beadandó eredménye egy `OPERATIONS_PLAN.md` dokumentum, amely a fenti küszöböket, a JSONL-sémákat, a join-szabályt és a rollback-lépéseket rögzíti, valamint a fenti szkript a naplófájlra futtatva. A `model_id`, a küszöbök és a checkpoint-útvonalak a labor saját konfigurációi, amelyeket nem külső benchmarkból vettünk át.

33.4.7 Feladatok

F1 (fogalmi, nehézség 1). Soroljon fel három, a produkciós naplóból címke nélkül számítható mutatót és három, csak hiteles ground truth-tal számítható mutatót. Mindegyikhez adja meg a definíciót a 33.4.2. szakasz jelöléseivel. *Beadandó:* a hat definíció egy-egy képletben vagy pontos szöveges alakban.

F2 (levezetési, nehézség 2). Alkalmazza a Tétel 1 konstrukcióját $N = 5$, $K = 3$ esetre. Legyen a megfigyelt predikcióvektor $\hat{\mathbf{y}} = (0;\ 0;\ 1;\ 1;\ 2)$. Adjon meg egy $\mathbf{y}_1$ és egy $\mathbf{y}_2$ ground-truth vektort, amelyekre $A(\mathbf{y}_1) = 1$ és $A(\mathbf{y}_2) = 0$, és indokolja, miért nem számítható ki az accuracy a megfigyelésből. *Beadandó:* a két vektor, a két accuracy kiszámítása és az azonosíthatatlansági indoklás.

F3 (implementációs, nehézség 2). Írjon `validate_schema` függvényt a 33.4.6. szakasz sémájához, és futtasson négy tesztesetet: helyes input; rossz dtype (például float64); rossz alak (24; 24); [0; 1]-en kívüli érték. A függvény `(bool, str)` párt adjon vissza. *Beadandó:* a függvény kódja, a négy hívás és a várt eredmények logikai értékkel.

F4 (hibakeresési, nehézség 2). Egy szolgáltatás a késő címkéket a `request_id` helyett a beérkezési sorrend szerint párosítja a kérésekhez. Egy napon a label-feed csoportosan, a kérések sorrendjétől eltérő sorrendben érkezik; a késés nagysága a párosítás szempontjából lényegtelen, a hiba forrása a sorrend. Mi a hiba, mi a következménye, hogyan javítható, és milyen teszttel ellenőrizhető a javítás? *Beadandó:* a hiba oka, a következmény egy konkrét példán, a javított join-szabály és egy olyan teszt, amelyben a két lista sorrendje eltér.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen 7 napos produkciós megfigyelési protokollt a GlyphOps-osztályozóhoz, ha a késő címke várható késése 3 nap. Határozza meg a megfigyelési egységet, a napi aggregálást, a riasztási küszöböket, a torzítás kezelését és a rollback feltételeit. Indokolja, miért nem használja a 3 napnál fiatalabb ablakot a címkézett accuracy számításához. *Beadandó:* egy legalább féloldalas protokoll a fenti elemekkel és a korlátok kimondásával.

33.4.8 Megoldások és indoklás

F1 megoldás. Címke nélkül számítható:

- sémaérvényességi arány: $\frac{1}{N}\sum_{t=1}^{N}\mathbb{1}\{x^{(t)}\in\mathcal{S}\}$;
- átlagos konfidencia: $\bar{c}=\frac{1}{N}\sum_{t=1}^{N}c^{(t)}$;
- predikcióeloszlás: $\hat{p}_k=\frac{1}{N}\sum_{t=1}^{N}\mathbb{1}\{\hat{y}^{(t)}=k\}$, $k=0,\dots,K-1$.

Csak ground truth-tal számítható:

- accuracy: $A=\frac{1}{N}\sum_{t=1}^{N}\mathbb{1}\{y^{(t)}=\hat{y}^{(t)}\}$;
- per-class recall: $R_k=\frac{\sum_t \mathbb{1}\{y^{(t)}=k;\ \hat{y}^{(t)}=k\}}{\sum_t \mathbb{1}\{y^{(t)}=k\}}$;
- macro-F1: az osztályonkénti $F1_k=\frac{2P_kR_k}{P_k+R_k}$ értékek osztályok közötti átlaga, ahol P_k a precision és R_k a recall; mindkettő a $y^{(t)}$ címkét igényli.

A két csoport elválasztása a Tétel 1 következménye: az első csoport nem függ y-tól, a második igen, és a tétel konstrukciója mellett a második értéke megváltozhat azonos megfigyelés mellett.

F2 megoldás. $\mathbf{y}_1=(0;\ 0;\ 1;\ 1;\ 2)$; ekkor minden elem egyenlő a predikcióval, tehát $A(\mathbf{y}_1)=5/5=1$. $\mathbf{y}_2=(1;\ 1;\ 2;\ 2;\ 0)$; minden elem különbözik a predikciótól, tehát $A(\mathbf{y}_2)=0/5=0$. A megfigyelés $(\mathbf{x},\hat{\mathbf{y}},\mathbf{c})$ mindkét esetben azonos, így a megfigyelésből képzett bármely determinisztikus függvény ugyanazt adja, miközben az accuracy 1 és 0 — ezért az accuracy nem azonosítható.

F3 megoldás. A függvény a 33.4.6. szakasz kódjában szerepel. A tesztek:

```
# lab/n132/test_schema.py
import numpy as np
from rollback_policy import validate_schema

x_ok = np.zeros((1, 24, 24), dtype=np.float32)
x_bad_dtype = np.zeros((1, 24, 24), dtype=np.float64)
x_bad_shape = np.zeros((24, 24), dtype=np.float32)
x_out = np.zeros((1, 24, 24), dtype=np.float32); x_out[0,0,0] = 1.5

for name, x in [("ok", x_ok), ("bad_dtype", x_bad_dtype),
                ("bad_shape", x_bad_shape), ("out_of_range", x_out)]:
    ok, msg = validate_schema(x)
    assert ok is (name == "ok"), (name, ok, msg)
    print(name, ok, msg)
# Mért kimenet (NumPy 2.5.4):
# ok True OK
# bad_dtype False dtype=float64, várt float32
# bad_shape False shape=(24, 24), várt (1, 24, 24)
# out_of_range False érték [0,1]-en kívül
```

A mért kimenet Python 3.12.3, NumPy 2.5.4, CPU, seed nélkül determinisztikus, alapértelmezett szálbeállítás mellett született. Az ellenőrzés a `(bool, str)` szerződést és a négy logikai értéket hasonlítja össze; numerikus tolerancia itt nem kell, mert a séma eldöntési feladat.

F4 megoldás. *Hiba:* a párosítás a lista indexét használja azonosító helyett. *Következmény:* ha a sorrend eltér, a label egy másik kérés predikciójához kapcsolódik, és a számított accuracy hamis lesz. Példa: a kérések sorrendje `[r1, r2]`, predikciók `[0, 0]`, a címkék késve `[r2: 1, r1: 0]` sorrendben érkeznek; a sorrendpárosítás `(r1, 1)` hibát ad, a helyes `(r1, 0)`. *Javítás:* a join kulcsa a `request_id`. *Ellenőrzés:* olyan teszt, ahol a label-lista szándékosan permutált, és az eredménynek a `request_id` szerinti párosítást kell adnia:

```python
# lab/n132/test_join.py
from rollback_policy import join_labels
req = [{"request_id": "r1", "pred_label": 0, "pred_conf": 0.9},
       {"request_id": "r2", "pred_label": 0, "pred_conf": 0.8}]
lab = [{"request_id": "r2", "label": 1},
       {"request_id": "r1", "label": 0}]
result = join_labels(req, lab)
assert result == [('r1', 0, 0), ('r2', 0, 1)], result
print(result)
# Mért kimenet (NumPy 2.5.4):
# [('r1', 0, 0), ('r2', 0, 1)] a request_id szerint, nem a beérkezési sorrendben.
```

A mért kimenet Python 3.12.3, NumPy 2.5.4, CPU, seed nélkül determinisztikus, alapértelmezett szálbeállítás mellett született.

F5 megoldás (minta-terv). *Megfigyelési egység:* kérés; a napló minden kéréshez `request_id`-t rendel. *Napi aggregálás:* naponta rögzítjük a sémaérvényességi arányt, a drift-mutatót, a predikcióeloszlást és a 3 napnál idősebb kérésekre csatlakoztatott accuracy-t; a drift-mutató csak napi $N \geq 100$ mellett értelmezett. *Címkézett ablak a 7 napos protokollban:* a 3 napos késés miatt a címkézett accuracy csak az 1–4. nap kéréseire számítható, mert az 5–7. nap kéréseihez a protokoll végéig nem érkezik címke; a címke nélküli indikátorok mind a 7 napra számolhatók. *Küszöbök:* sémaérvényesség $< 0{,}99$; drift $> 0{,}5$; accuracy $< 0{,}85$, ha a lezárt mintaméret ≥ 200 és a lefedettség $\geq 0{,}8$. *Torzítás kezelés:* a 3 napos késés miatt a fiatal ablakban a címkék még nem reprezentatívak, ezért azokat kizárjuk a címkézett metrikából; a lefedettségi arányt naplózzuk, és a szelekciós szabályt dokumentáljuk. *Rollback:* automatikus visszaállítás a v1.0 state_dict-re, ha a címkézett accuracy vagy a sémaérvényesség riasztást ad, a 33.4.6. szakasz ellenőrzési lépéseivel. *Korlát:* a 7 napos ablak rövid, a drift küszöb mintavételi ingadozást is jelezhet, és a címkézett metrika csak négy napot fed le; a terv csak ezt a fiktív osztályozót és ezt az adatforgalmat írja le, általánosításához hosszabb megfigyelés kell.

33.4.9 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. Az input drift kimutatását automatikusan a modellhiba oksági bizonyítékaként értelmezzük. Ez kétirányban hibás. Egyrészt a drift-riasztás mellett is lehet a modell pontos: ha a produkcióban sötétebb képek jelennek meg, a drift-mutató megszólal, de ha a modell kontrasztnormalizáló előfeldolgozása vagy tanult invariáns reprezentációja ezt kezeli, a ground-truth accuracy változatlan maradhat. Másrészt a drift hiánya mellett is nőhet a hiba: ha a címkéző szabályzat megváltozik, a bemeneti eloszlás azonos marad, de a korábban tanult cél már nem érvényes. Az oksági állításhoz kontrollált beavatkozás vagy legalább a címkézett metrika együttmozgásának vizsgálata kell; a drift egymagában korrelációs jelzés.

Önellenőrzés. (1) Meg tudom-e nevezni minden monitoring-mutatóhoz, hogy függ-e y-tól? (2) A rollback előtt ellenőriztem-e a betöltött modell kimenetét egy független halmazon? (3) A csúszóablakos accuracy számításánál kizártam-e a még le nem zárt kéréseket? (4) A model card különválasztja-e a mért teszt eredményét a produkciós monitoring jelzésétől?

Kulcs. (1) Minden monitoring-mutatónál meg kell nevezni, hogy függ-e y-tól; a címke nélküli indikátor nem accuracy. (2) Igen: a rollback előtt a betöltött modell kimenetét független halmazon ellenőrizzük. (3) Igen: a még nyitott kérések kizárandók a csúszóablakos accuracy számításából. (4) Igen: a model card külön mezőben közli a mért teszt eredményét és a produkciós monitoring jelzését.

Visszatekintés. A 33.2. Batch és kis szolgáltatás alfejezetben a séma a kérésérvényesség statikus szerződése volt; itt a sémaérvényességi arány már dinamikus üzemeltetési mutató. A 29.4. Anomáliapontszám rekonstrukcióból alfejezetben a küszöböt validációs címkékhez kötöttük;

itt ugyanez az elv a csúszóablakos riasztási küszöbre vonatkozik. A 21.4. Osztályarány, súlyozás és céleloszlás alfejezetben a tanítóhalmaz céleloszlását különböztettük meg a predikcióeloszlástól; itt a produkciós predikcióeloszlás megváltozása nem azonos a címkeeloszlás megváltozásával.

33.4.10 Rövid összefoglaló

Az üzemeltetési monitoring két eltérő kérdést kezel: a rendelkezésre álló megfigyelésből a bemeneti eloszlás, a sémaérvényesség és a modell kimeneti viselkedésének változása követhető; a tényleges pontossághoz hiteles ground truth kell. A Tétel 1 konstruktívan bizonyítja, hogy azonos input-, predikció- és konfidencianapló mellett az accuracy 1 és 0 is lehet, tehát a címkét igénylő metrika nem azonosítható a naplóból. A késő címke szelekciós torzítást vihet a csúszóablakos becslésbe, ezért a join lefedettségét és a címkeforrás szabályát rögzíteni kell. A drift-riasztás vizsgálatot indít, nem oksági bizonyíték; a rollback csak ellenőrzött state_dict-betöltéssel és független kimenet-egyezéssel érvényes.

33.4.11 Következő lecke

A 34.1. Mért késleltetés, throughput és memória a hatékonyság mérését vezeti be: a késleltetés, a throughput és a memória mérését ugyanolyan protokollal, mint itt a monitoringot — előre rögzített egységgel, ismétléssel és a hardverfeltételek kimondásával. Az üzemeltetési naplóban gyűjtött latency-értékek önmagukban üzemeltetési adatok; teljesítményállítássá csak rögzített mérési protokollal, ismétléssel és a hardverfeltételek kimondásával válnak.

33.4.12 Felhasznált források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [P32] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A. (2014). *A Survey on Concept Drift Adaptation.* ACM Computing Surveys, 46(4).
- [P33] Rabanser, S., Günnemann, S., Lipton, Z. C. (2019). *Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift.* NeurIPS 2019. https://arxiv.org/abs/1810.11953.

34 Hatékonyság, kvantálás és tudásdesztilláció

34.1 Mért késleltetés, throughput és memória

34.1.1 Cél, előismeret és induló kérdés

A 33.1. Inference-szerződés és modellcsomag inference-szerződés és a 33.4. Üzemeltetés, drift és késő címke üzemeltetési monitoring után eljutottunk ahhoz a kérdéshez, amelyet a kiszolgálás üzemeltetője tesz fel: mekkora terhelést bír a kiválasztott modell, és milyen válaszidőt tapasztal a felhasználó? A „gyorsabb modell” vagy „nagyobb batch” állítás csak akkor tudományos állítás, ha megmondjuk, mit mérünk, milyen egységben, milyen batchméret mellett, és hogyan zárjuk ki a mérés első, nem reprezentatív hívásait. Ez az egység a mérési fogalmakat és egy kis CPU-s laborprotokollt ad.

Előismeret: a 7.4. Numerikus stabilitás és jelentésmegőrző implementáció numerikus stabilitásból azt használjuk, hogy a mérési kódnak dtype- és shape-ellenőrzéssel kell zárulnia; a 19.4. ConvNeXt és a recept/architektúra elválasztása recept/architektúra elválasztásból azt, hogy a modellváltozatok összehasonlításakor a receptet rögzíteni kell; a 23.4. Hibaszeletek és alkalmazási határ hibaszeletekből azt, hogy a sebességmérés nem helyettesíti a minőségi metrikát; a 24.4. VI. ellenőrző pont: megbízható CNN-training profilerfogalmaiból a wall time, warmup és memória alapjait; a 33.4. Üzemeltetés, drift és késő címke monitoringból pedig azt, hogy a latency megfigyelhető címke nélkül, míg a pontosság hiteles ground truth-ot igényel.

Induló kérdés: ha egy modell 400 kép/s throughputot ad, akkor egy felhasználói kérés válaszideje 2,5 ms? Becslés: nem. Ha a 400 kép/s érték 8-as batchből származik, és a 8 kép feldolgozása 20 ms-ig tart, akkor a kérésnek a teljes 20 ms-ot kell várnia, nem annak nyolcadát. A batch throughput és az egyedi kérés latency két külön mennyiség; ezt formalizáljuk a következő szakaszban.

34.1.2 Definíciók: batch, latency distribution, throughput, warmup, CPU threads, memory

Definíció (kiszolgálási batch). A kiszolgálási batch $B \in \mathbb{N}_+$ darab egyedből álló, együtt feldolgozott csomag. A bemenet $X \in \mathbb{R}^{B \times C_{in} \times H \times W}$, a kimenet $Z \in \mathbb{R}^{B \times K}$ logit. Ez ugyanaz a B, mint a tenzoralakok N batch-dimenziója, de itt a jelentés fizikai: a kiszolgáló egyszerre B képet dolgoz fel. A B méretű batch egyetlen forward hívást jelent; egy kérés válasza a teljes batch feldolgozása után érkezik meg.

Definíció (batch latency és latency distribution). Legyen $T_B \in \mathbb{R}_+$ egy B méretű batch forward hívás kezdetétől a visszatéréséig eltelt fal-idő (wall time) másodpercben. Egy mérési sorozatban T_B valószínűségi változó; a **latency distribution** az ismételt mérésekből becsült eloszlás. Jellemzésére az átlag $\bar{T}_B$, a medián, a 90. percentilis p_{90} és a 99. percentilis p_{99} használatos. A p_{90} jelentése: a mért ismétlések 90%-ában a latency legfeljebb p_{90}. A percentilis empirikus becslés, nem a modell paramétere.

Definíció (throughput). A B méretű batch steady-state átlagos feldolgozási rátája:

$$Q_B = \frac{B}{\bar{T}_B} \quad \left[\frac{\text{elem}}{\text{s}}\right],$$

ahol $\bar{T}_B$ az átlagos batch latency. A batch-szintű throughput $R_B = 1/\bar{T}_B$, mértékegysége batch/s. A throughput összevont ráta; egyetlen kérés időtartamáról önmagában nem mond semmit.

Definíció (warmup). A warmup a mérési statisztikába be nem számító, a tényleges mérés előtt végrehajtott n_{warm} hívás. Célja, hogy a könyvtár inicializálása, a memóriaallokátor feltöltődése, a belső thread pool elindulása és az első fordítás/optimalizálás ne torzítsa a steady-state mérést. A warmup hívásoknak ugyanazt a bemeneti eloszlást, dtype-ot és device-ot kell használniuk, mint a méréseknek.

Definíció (CPU threads). A CPU-szálak a folyamaton belüli párhuzamos végrehajtási egységek; PyTorch-ban a `torch.get_num_threads()` és `torch.set_num_threads()` hívásokkal kérdezhetők le és állíthatók. A szálak száma nem throughput-garancia: a tényleges párhuzamosságot az operátor típusa, a batchméret, a memória-sávszélesség, az operációs rendszer ütemezője és az esetleges NUMA-elrendezés együtt határozza meg.

Definíció (memory). Három különböző memóriafogalmat különböztetünk meg. (a) A modell paramétereinek és puffereinek bájtjai: $\sum_p \text{numel}(p) \cdot \text{element_size}(p)$; ez B-től független. (b) A forward aktivációk mérete: a közbülső tenzoralakok bájtjai, amelyek B-vel arányosan nőnek. (c) A folyamat rezidens memóriája (RSS, resident set size) és annak csúcsa; a csúcs a folyamat teljes élettartama alatt elért maximális RSS, amely az importokat és a korábbi allokációkat is tartalmazza, nem csak a mérési szakaszt. A labor a (c) csúcsot rögzíti, az (a) értéket pedig a paraméterszámokból számítja ki.

34.1.3 Levezetési kötelezettség: throughput és batch-latency kapcsolata

Állítás (throughput–latency konzisztencia). Legyen $B \in \mathbb{N}_+$ rögzített batchméret. Tegyük fel, hogy egy mérési protokoll n_{rep} ismétlést végez, minden ismétlésben a B méretű batch feldolgozási latency-jét méri, és a batch-ek között nincs sorban állás vagy átfedő végrehajtás. Jelölje $T_B^{(i)}$ az i-edik ismétlés latency-jét másodpercben, és legyen $\bar{T}_B = \frac{1}{n_{\text{rep}}} \sum_{i=1}^{n_{\text{rep}}} T_B^{(i)} > 0$. Ekkor

$$T_B^{(i)} = \frac{B}{Q_B^{(i)}}, \qquad Q_B^{(i)} = \frac{B}{T_B^{(i)}}$$

minden i-re, ahol $Q_B^{(i)}$ az i-edik batch elemszintű pillanatnyi rátája, és

$$Q_B = \frac{B}{\bar{T}_B}, \qquad R_B = \frac{1}{\bar{T}_B}.$$

Így a teljes mérésre: ha a teljes hasznos idő $\sum_i T_B^{(i)}$ és a feldolgozott elemszám $B \cdot n_{\text{rep}}$, akkor a teljes ráta

$$\frac{B \cdot n_{\text{rep}}}{\sum_i T_B^{(i)}} = \frac{B}{\bar{T}_B} = Q_B.$$

Bizonyítás. A pillanatnyi definíció: $Q_B^{(i)}$ az i-edik batch elemszáma osztva a batch feldolgozási idejével, azaz $B/T_B^{(i)}$. Átrendezve $T_B^{(i)} = B/Q_B^{(i)}$. A protokoll szerint a batch-ek szekvenciálisan

futnak, tehát a teljes feldolgozott elemszám $B \cdot n_{\text{rep}}$, a teljes idő pedig $\sum_i T_B^{(i)}$. A teljes ráta definíció szerint a feldolgozott elemszám és a teljes idő hányadosa:

$$\frac{B \cdot n_{\text{rep}}}{\sum_{i=1}^{n_{\text{rep}}} T_B^{(i)}} = \frac{B}{\frac{1}{n_{\text{rep}}} \sum_i T_B^{(i)}} = \frac{B}{\bar{T}_B},$$

ami éppen Q_B. A batch-szintű ráta hasonlóan $R_B = 1/\bar{T}_B$. □

Fontos nemlinearitási megjegyzés. Az azonosság nem állítja, hogy $T_B = B \cdot T_1$ vagy hogy Q_B független B-től (itt T_1 és Q_1 a $B = 1$ batchre vonatkozó latency, illetve throughput). A CPU-n végrehajtott mátrixműveletek párhuzamosítása és a memóriaelérés miatt tipikusan $T_B < B \cdot T_1$, így $Q_B > Q_1$ is lehet. Az azonosság pusztán a mérési definíciók konzisztenciája: ugyanazon mérésen belül $Q_B \cdot T_B = B$, vagyis a latency és a throughput egymás inverze a B batchméret szorzójáig; a B méretű batch válaszideje továbbra is T_B, nem pedig $1/Q_B$.

34.1.4 Eredeti kézi kis példa, majd minimális ellenőrző kód

Legyen a mérési protokoll rögzített: float32, CPU, 4 szál, warmup 10, ismétlés 50. Tegyük fel a következő **számított példát** — a kimenet VÁRT ÉRTÉK (levezetésből, nem mért), mert a latenciák előre felvett értékek:

B	$\bar{T}_B$ [ms]	Q_B [kép/s]	R_B [batch/s]	$\bar{\tau}_B = \bar{T}_B/B$ [ms/kép]
1	8,0	125	125	8,0
8	20,0	400	50	2,5

A táblázat mutatja a különbséget: $B = 8$-nál a felhasználói batch latency 20 ms, miközben a throughput 400 kép/s. A 400 kép/s helytelen inverze 2,5 ms lenne; a helyes per-kép átlag 2,5 ms, de ez statisztikai átlag, nem egy kérés válaszideje. Egy olyan kérés, amelyiknek a válasza a 8 képből álló batch feldolgozása után készül el, 20 ms-ot vár.

A következő minimális kód a fenti azonosságot szemlélteti; a kimenet VÁRT ÉRTÉK (levezetésből, nem mért), mert a latenciák előre felvett értékek:

```
# munkakönyvtár: lab/n133_measure

B = 8
n_rep = 50
# VÁRT ÉRTÉK (levezetésből, nem mért): 50 ismétlés latency-je, átlaguk 20 ms
latencies_s = [0.020 + (0.002 if i % 2 else -0.002) for i in range(n_rep)]
# Valódi adatellenőrzés a nyers ismétlésidőkre; a Q_sum == Q_mean viszont algebrai azonosság.
assert len(latencies_s) == n_rep and all(t > 0 for t in latencies_s)
total_time = sum(latencies_s)              # a teljes idő az ismétlések összege
mean_s = total_time / n_rep                # ugyanebből számolt átlag
Q_sum = B * n_rep / total_time             # a teljes időből számolt ráta
Q_mean = B / mean_s                        # az átlagból számolt ráta
R = 1.0 / mean_s
print(f"Q={Q_mean:.3f} elem/s, R={R:.3f} batch/s, batch_latency={mean_s*1e3:.3f} ms")
assert abs(Q_sum - Q_mean) < 1e-12         # algebrai azonosság, nem független ellenőrzés
```

A kód kimenete a számított példában: `Q=400.000 elem/s, R=50.000 batch/s, batch_latency=20.000 ms` (VÁRT ÉRTÉK (levezetésből, nem mért)). A két ráta a definíció szerint algebrailag azonos: $Q_{\text{sum}} = B \cdot n_{\text{rep}} / \sum_i T_B^{(i)} = B/\bar{T}_B = Q_{\text{mean}}$, ezért az egyezésük nem független ellenőrzés, hanem az azonosság szemléltetése. A kód a nyers ismétlésidőkre csak azt ellenőrzi, hogy számuk n_{rep} és mindegyik pozitív; a mintaátlag a felépítés szerint 20 ms.

34.1.5 Konkrét labor: kis CPU-modell mérése több batchmérettel

A labor célja, hogy a hallgató saját gépén, rögzített protokollal mérje meg egy kis CNN latency-jét, throughputját és csúcs RSS-ét. Ez az egység a futtatási protokollt adja; a rögzített ellenőrző környezetben futtatott példa kimenete alább, a hallgató saját mérése pedig a sablonba kerül, a gépadatokkal együtt.

Bemenet. Szintetikus input, nem a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab képeiből, hogy az adatbetöltési overhead ne keveredjen a forward mérésbe: $X \in \mathbb{R}^{B\times 1\times 24\times 24}$, ahol X elemei független, nem csonkított standard normális eloszlásból származnak, `torch.randn`-nel. A modell egy kis CNN: két konvolúciós blokk, max-pool, lineáris fej. A paraméterszám a labor első ellenőrző sora.

```
# munkakönyvtár: lab/n133_measure
# Fájl: measure_latency.py
import json
import platform
import time
import resource

import torch

def main():
    # --- rögzített környezet ---
    torch.manual_seed(20261020)
    device = torch.device("cpu")
    dtype = torch.float32
    threads = 4
    torch.set_num_threads(threads)  # a protokoll 4 szálat rögzít
    batch_sizes = [1, 2, 4, 8, 16]
    n_warm = 10
    n_rep = 50

    # --- kis CNN, eval módban, paraméterezés a 34.1. lecke laborjához ---
    model = torch.nn.Sequential(
        torch.nn.Conv2d(1, 8, kernel_size=3, padding=1),
        torch.nn.ReLU(),
        torch.nn.MaxPool2d(2),          # 12x12
        torch.nn.Conv2d(8, 16, kernel_size=3, padding=1),
        torch.nn.ReLU(),
        torch.nn.MaxPool2d(2),          # 6x6
        torch.nn.Flatten(),
        torch.nn.Linear(16 * 6 * 6, 3),
    ).to(device=device, dtype=dtype)
    model.eval()

    n_params = sum(p.numel() for p in model.parameters())
    param_bytes = sum(p.numel() * p.element_size() for p in model.parameters())

    results = {
        "machine": platform.platform(),
        "python": platform.python_version(),
        "torch": torch.__version__,
```

```
    "device": str(device),
    "dtype": str(dtype),
    "threads": threads,
    "seed": 20261020,
    "warmup": n_warm,
    "repeats": n_rep,
    "n_params": n_params,
    "param_bytes": param_bytes,
    "batch_results": [],
}

for B in batch_sizes:
    x = torch.randn(B, 1, 24, 24, dtype=dtype, device=device)
    # shape- és dtype-kontroll a mérés előtt
    assert x.shape == (B, 1, 24, 24)
    assert x.dtype == dtype
    assert x.device == device

    # warmup: n_warm azonos alakú hívás, eredmény nincs a statisztikában
    with torch.no_grad():
        for _ in range(n_warm):
            _ = model(x)

    latencies_s = []
    with torch.no_grad():
        for _ in range(n_rep):
            t0 = time.perf_counter()
            z = model(x)
            t1 = time.perf_counter()
            latencies_s.append(t1 - t0)

    # a kimenet shape-ellenőrzése: B x K = B x 3
    assert z.shape == (B, 3)
    assert torch.isfinite(z).all()

    lat_s = torch.tensor(latencies_s, dtype=torch.float64)
    mean_s = float(lat_s.mean())
    median_s = float(lat_s.quantile(0.5))  # n=50-nél a torch.median alsó mediánt adna
    p90_s = float(lat_s.quantile(0.90))
    p99_s = float(lat_s.quantile(0.99))
    q_elem_per_s = B / mean_s if mean_s > 0 else float("nan")
    r_batch_per_s = 1.0 / mean_s if mean_s > 0 else float("nan")

    results["batch_results"].append({
        "B": B,
        "n_repeats": n_rep,
        "latencies_s": [float(t) for t in latencies_s],   # a nyers ismétlésidők
        "sum_latency_s": float(lat_s.sum()),
        "mean_latency_s": mean_s,
        "median_latency_s": median_s,
        "p90_latency_s": p90_s,
        "p99_latency_s": p99_s,
        "min_latency_s": float(lat_s.min()),
        "max_latency_s": float(lat_s.max()),
        "throughput_elem_per_s": q_elem_per_s,
        "throughput_batch_per_s": r_batch_per_s,
        "mean_per_item_s": mean_s / B,
    })

# csúcs RSS: Linuxon KiB-ban, macOS-en bájtban adja a ru_maxrss (platformfüggő)
peak_rss_raw = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
peak_rss_kib = peak_rss_raw / (1024.0 if platform.system() == "Darwin" else 1.0)
results["peak_rss_kib"] = peak_rss_kib
```

```
    results["peak_rss_mib"] = peak_rss_kib / 1024.0

    with open("latency_results.json", "w", encoding="utf-8") as f:
        json.dump(results, f, indent=2, ensure_ascii=False)

    print(json.dumps(results, indent=2, ensure_ascii=False))

if __name__ == "__main__":
    main()
```

Futtatás és ellenőrzés. A parancs a `lab/n133_measure` munkakönyvtárból:

```
python measure_latency.py
```

A kód a futás végén egyetlen JSON-t ír ki; ennek `n_params` mezője a fenti modellben $8 \cdot 9 + 8$ (első conv) $+16 \cdot 8 \cdot 9 + 16$ (második conv) $+16 \cdot 36 \cdot 3 + 3$ (lineáris) $= 2979$ paraméter, tehát az ellenőrzött érték `n_params == 2979`. A JSON-sémában minden `batch_results` bejegyzésnek tartalmaznia kell a B, a mean/median/p90/p99 latency, a throughput, az ismétlésszám, az összegzett latency és a nyers ismétlésidők mezőit. A hallgató által feltöltött táblázat sablonja:

B	$\bar{T}_B$ [ms]	p_{90} [ms]	Q_B [elem/s]	R_B [batch/s]
1	—	—	—	—
2	—	—	—	—
4	—	—	—	—
8	—	—	—	—
16	—	—	—	—

A kötőjelek a hallgató által mérendő értékek helyét jelölik; a fenti sablonban nincs előre kitöltött érték. Értelmezési követelmény: a hallgatónak kommentárban meg kell indokolnia, hogy Q_B hogyan változik B-vel, és hogy a beküldött számok konzisztensek-e a $Q_B = B/\bar{T}_B$ azonossággal. A `peak_rss_mib` értéke a teljes folyamatra vonatkozik, nemcsak a modellre, és nem batchméretenként értelmezett mennyiség, ezért egyszer, a teljes futásra vonatkozóan kell rögzíteni; a paraméterbájtok nagyságrendje 11,6 KiB (2979 paraméter $\times$ 4 bájt $=$ 11 916 bájt), ami a folyamat RSS-éhez képest elhanyagolható.

Mért példa a rögzített ellenőrző környezetben. A fenti `measure_latency.py` futtatása (Linux x86-64, Python 3.12.3, torch 2.15.0+cpu, CPU, 4 szál, seed 20261020, 10 melegítő és 50 mért ismétlés, `n_params = 2979`, `peak_rss_mib = 243,57`) a következő értékeket adta:

B	$\bar{T}_B$ [ms]	p_{90} [ms]	Q_B [elem/s]	R_B [batch/s]
1	0,120	0,125	8303,4	8303,4
2	0,117	0,122	17149,5	8574,8
4	0,138	0,145	29078,8	7269,7
8	0,171	0,182	46817,1	5852,1
16	0,253	0,326	63123,3	3945,2

A táblázat csak erre a gépre és erre a futtatásra érvényes; a hallgató a saját mérését a fenti sablonba írja, és a `machine`, `python`, `torch`, `threads` mezőkkel együtt közli.

A [D16] (`torch.utils.benchmark.Timer`) és a [D17] (`torch.profiler`) hivatalos eszközök alternatív, részletesebb mérésre. A fenti saját protokoll `time.perf_counter`-t használ, így a labor nem függ a két API dokumentációs részletétől.

34.1.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el saját szavaival, mi a különbség a batch latency T_B és az elemenkénti átlagos latency $\bar{T}_B/B$ között. Adjon egy konkrét számpéldát, amelyben az előbbi 20 ms, az utóbbi 2,5 ms, és nevezze meg, melyiket tapasztalja egyetlen kérés. Beadandó: legfeljebb 5 mondatos magyarázat és a számpélda.

F2 (levezetési, nehézség 2). Egy mérésben $T_B = 12$ ms és $T_{2B} = 18$ ms ugyanarra a modellre, ugyanazon protokollal. Számítsa ki Q_B, Q_{2B}, R_B, R_{2B}, valamint a per-item átlagokat. Bizonyítsa be a $T_{2B} = 2B/Q_{2B}$ azonosságot, és indokolja, miért nem mond ellent az azonosságnak az a megfigyelés, hogy $T_{2B} < 2T_B$. Beadandó: levezetés lépésenként, egységekkel.

F3 (implementációs, nehézség 2). Futtassa a fenti `measure_latency.py` protokollt, és ellenőrizze: (a) a JSON `n_params` mezeje 2979; (b) minden bejegyzésben a tárolt nyers `latencies_s` listából újraszámolt átlag egyezzen a `mean_latency_s` mezővel, az összeg a `sum_latency_s`-sel, és a `throughput_elem_per_s` egyezzen a B és az átlagból számolt hányadossal (`math.isclose`, `rel_tol=1e-9`); (c) a shape-assert nem szakadt meg. Rögzítse a gép- és verzióadatokat. Beadandó: a kitöltött táblázat, a JSON fájl és a három ellenőrzés eredménye.

F4 (hibakeresési, nehézség 2). Tekintse az alábbi módosított mérési ciklust:

```
lat = []
with torch.no_grad():
    for i in range(n_rep):
        t0 = time.perf_counter()
        z = model(x)
        t1 = time.perf_counter()
        lat.append(t1 - t0)
mean_latency = sum(lat) / len(lat)
```

A kód a warmupot elhagyta, és az első hívás latency-jét is belevette az átlagba. Magyarázza meg, miért torzít ez általában felfelé, milyen következménye van a throughput-számításra, és javítsa a kódot a n_{warm} ismétlés beillesztésével. Beadandó: hibamagyarázat, javított kód, és egy ellenőrzési javaslat, amely az első és a későbbi hívások latency-jét külön kiírja.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet a `torch.set_num_threads` hatására a fenti mini-CNN-en. Rögzítse: kutatási kérdés, baseline (például 1 szál), változó faktor (1, 2, 4, 8 szál), batchméret, ismétlésszám, metrika ($\bar{T}_B$, Q_B), és az érvényességi korlátokat (hyperthreading, NUMA, más gép, GPU hiánya). Beadandó: egy protokollterv és annak indoklása; előre megadott győztes nem szükséges.

34.1.7 Megoldások és indoklás

F1 megoldás. A batch latency T_B a teljes B elemű csomag feldolgozási ideje; a kérés, amelynek válasza a batch feldolgozása után készül el, T_B-t vár. A per-item átlag T_B/B egy statisztikai átlag: az összes elemre jutó idő, mintha az idő egyenletesen oszlana el; ez nem egyetlen elem válaszideje. Példa: $B = 8$, $T_B = 20$ ms, akkor a per-item átlag $20/8 = 2{,}5$ ms. Egy felhasználói kérés, amely a 8 képet egy batchben várja, 20 ms-ot tapasztal. A 2,5 ms csak akkor lenne a kérés latency-je, ha minden egyes elem külön, azonnal választ kapna; a batch végrehajtás ezt nem garantálja.

F2 megoldás. Adott: $T_B = 0{,}012$ s, $T_{2B} = 0{,}018$ s. A definíció szerint

$$Q_B = \frac{B}{T_B}, \qquad Q_{2B} = \frac{2B}{T_{2B}}, \qquad R_B = \frac{1}{T_B}, \qquad R_{2B} = \frac{1}{T_{2B}}.$$

Általános B-vel $Q_B = B/0{,}012 = 83{,}33\,B$ elem/s és $Q_{2B} = 2B/0{,}018 = 111{,}11\,B$ elem/s, a batch-ráták pedig $R_B = 83{,}33$ batch/s és $R_{2B} = 55{,}56$ batch/s, függetlenül B-től. A per-item átlagok: $T_B/B = 12$ ms/elem és $T_{2B}/(2B) = 9$ ms/elem. A $B = 1$ helyettesítéssel $Q_B = R_B$ adódik, ezért ott a két ráta nem különböztethető meg; a B-függés csak általános B-vel látszik.

Az azonosság bizonyítása: a $Q_{2B} = 2B/T_{2B}$ definíciót T_{2B}-vel beszorozva $T_{2B} \cdot Q_{2B} = 2B$ adódik, majd $Q_{2B} > 0$-val osztva ($Q_{2B} > 0$, mert $T_{2B} > 0$):

$$T_{2B} = \frac{2B}{Q_{2B}}.$$

A $T_{2B} < 2T_B$ megfigyelés nem mond ellent: ez a párhuzamosítás és a fix overhead miatt lehetséges, és nem szerepel az azonosság feltételei között. Az azonosság minden egyes B-re külön-külön teljesül; a különböző B-k közötti kapcsolatot nem írja elő.

F3 megoldás. A futtatás után a `latency_results.json` mezőinek ellenőrzése:

```
import json, math
with open("latency_results.json", encoding="utf-8") as f:
    d = json.load(f)
assert d["n_params"] == 2979
for row in d["batch_results"]:
    raw = row["latencies_s"]
    assert row["n_repeats"] == len(raw) == 50
    mean_from_raw = sum(raw) / len(raw)
    assert math.isclose(mean_from_raw, row["mean_latency_s"], rel_tol=1e-9, abs_tol=0.0), row
    assert math.isclose(sum(raw), row["sum_latency_s"], rel_tol=1e-9, abs_tol=0.0), row
    q = row["B"] / mean_from_raw
    assert math.isclose(row["throughput_elem_per_s"], q, rel_tol=1e-9, abs_tol=0.0), row
    assert math.isclose(row["throughput_batch_per_s"], 1.0 / mean_from_raw,
                        rel_tol=1e-12, abs_tol=0.0)
    assert row["mean_latency_s"] > 0
```

A JSON a nyers ismétlésidőket is tárolja, ezért az aggregátumok a forrásadatból ellenőrizhetők; a `sum_latency_s / n_repeats` és a `mean_latency_s` puszta összevetése önmagában algebrai azonosság volna, ezért a nyers listát használjuk. A shape-assert a kódban a mérés előtt és után fut; ha a kód hiba nélkül lefutott, az assert nem szakadt meg. A seed 20261020, a dtype float32, a device CPU. A konkrét mért számok a hallgató gépétől függnek; minden közölt számhoz tartoznia kell a `machine`, `python`, `torch`, `threads` mezőknek.

F4 megoldás. Az első hívás magában foglalja a modulbetöltés utáni első kernel-végrehajtást, az allocator feltöltődését, a thread pool inicializálását és esetleg a belső optimalizációt. Ezért az első latency jellemzően jóval nagyobb, mint a steady-state hívásoké; az átlagba vétele felfelé torzítja $\bar{T}_B$-t. Mivel $Q_B = B/\bar{T}_B$, a torzított nagy $\bar{T}_B$ alulbecsüli a steady-state throughputot. A javítás:

```
n_warm = 10
with torch.no_grad():
    for _ in range(n_warm):
        _ = model(x)
lat = []
with torch.no_grad():
    for _ in range(n_rep):
        t0 = time.perf_counter()
        z = model(x)
        t1 = time.perf_counter()
        lat.append(t1 - t0)
```

Ellenőrzési javaslat: futtassunk $n_{\text{rep}} = 10$ ismétlést warmup nélkül, és írjuk ki az első, a második és a többi hívást külön; ha az első $> 2\times$ medián, a warmup elhagyása kimutatható torzítást okoz.

F5 megoldás. Minta-protokoll. Kérdés: csökkenti-e a szálszám növelése a $B = 8$ batch átlagos latency-jét a mini-CNN-en? Baseline: 1 szál, ugyanaz a modell, ugyanaz a batchméret. Változó faktor: `torch.set_num_threads(t)` a $t \in \{1;\ 2;\ 4;\ 8\}$ értékekre, mindegyik beállítás előtt új mérési ciklus warmuppal. Megfigyelési egység: egy forward hívás latency-je; ismétlésszám: 50, warmup: 10. Metrika: $\bar{T}_B$, p_{90} és Q_B. Előfeltétel: a batchméret rögzített, mert a szálhatás batchfüggő; a processz affinity és a gépi terhelés rögzítendő. Érvényességi korlát: az eredmény csak az adott CPU-ra, operátorkészletre és batchméretre vonatkozik; hyperthreading esetén a 4 fizikai mag 8 logikai szálat adhat, de a gyorsulás nem lineáris; NUMA-rendszeren más elrendezés más eredményt adhat; GPU-ra nem vihető át.

34.1.8 Tipikus tévedés, önellenőrzés és korábbi fogalom felidézése

Tipikus tévedés. A „batch throughput nem egyetlen felhasználói kérés késleltetése” hiba az inverz mértékegység összekeverése: $Q_B = 400$ kép/s értékből $1/Q_B = 2{,}5$ ms válaszidőt számolni hamis. Ellenpélda a 34.1.4. szakasz táblázata: $B = 8$, $T_B = 20$ ms mellett $Q_B = 400$ kép/s, de a kérés latency-je 20 ms. A 2,5 ms a per-item átlag, nem az egyedi válaszidő. A hiba következménye: a szolgáltatási szint (SLO) alultervezése, mert a tényleges válaszidő a batch teljes feldolgozási ideje.

Önellenőrzés. (a) Mértékegység: Q_B elemszám osztva idővel, T_B idő; szorzatuk B, tehát a $T_B = B/Q_B$ dimenzióban helyes. (b) Alakellenőrzés: a laborban a bemenet $(B, 1, 24, 24)$, a kimenet $(B, 3)$, és a batchméret ciklus minden értékére külön-külön érvényes. (c) A mérési ismétléseknek ugyanazt a seedelt inputot kell használniuk egy batchméreten belül, hogy a bemenet ne változzon az ismétlések között, és a szórás a mérési zajból származzon; a seed rögzítése a protokoll része. (d) A percentilisek becslése annál pontosabb, minél több ismétlésből származik; $n_{\text{rep}} = 50$ mellett a p_{99} gyakorlatilag a maximum, ezért 99. percentilist csak jóval több ismétlésből érdemes közölni, a min/max pedig önmagában nem jellemzi az eloszlást.

Visszatekintés. A 33.1. Inference-szerződés és modellcsomag inference-szerződésből: a mért objektum nem csupán egy súlyfájl, hanem az architektúra, a preprocessing, a dtype, a device és az eval-állapot együttese. A laborban ezért a modellt `eval()` módba tesszük, a dtype-ot és device-ot explicit rögzítjük. A 30.1. Pretext és downstream feladat pretext/downstream megkülönböztetéséből: a mérés a kiválasztott downstream modellre vonatkozik, nem az előtanítási célra. A 22.1. Adateredet, címke és csoportos split adateredetből: a mérési input szintetikus, mert a cél a számítási teljesítmény elkülönítése az adatbetöltéstől; a GlyphLab képeit a tanítási/tesztelési folyamatban használjuk, nem a sebességmérés bemeneteként.

34.1.9 Rövid összefoglaló

A kiszolgálási batch B elemű csomag, latency-je T_B, a mért eloszlásból átlagot és percentiliseket számolunk. A throughput $Q_B = B/\bar{T}_B$ elem/s, a batch-szintű ráta $R_B = 1/\bar{T}_B$; a levezetett $T_B = B/Q_B$ azonosság a mérési definíciók konzisztenciája, nem állít linearitást B-ben. A warmup kizárja az első hívások állandó többletköltségét, a CPU-szálak számát rögzíteni kell, a memória pedig paraméter-, aktiváció- és processz-RSS szinten különbözik. A labor saját `time.perf_counter` protokollt ad; a hallgató a gépadatokkal együtt közli a mért táblázatot. A batch throughput önmagában nem egyedi kérésválaszidő.

Következő lecke. A következő, 34.2. Kvantálás és hibatovábbterjedés egységben a kvantálást vizsgáljuk; az ottani gyorsulási állításokat ugyanezen a latency/throughput mérési kereten kell majd a numerikus eltéréssel együtt értékelni.

34.1.10 Felhasznált források

- [D16] PyTorch contributors (2026). *torch.utils.benchmark*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/benchmark_utils.html. Hozzáférés: 2026-10-30.
- [D17] PyTorch contributors (2026). *torch.profiler*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/profiler.html. Hozzáférés: 2026-10-30.

34.2 Kvantálás és hibatovábbterjedés

34.2.1 Cél, előismeret és induló kérdés

A 34.1. Mért késleltetés, throughput és memória egységben a késleltetést, az átbocsátóképességet és a memóriahasználatot mértük. Ott a tárolási méret csak az egyik erőforrás volt. Most a **kvantálást** vizsgáljuk: egy már betanított, `float32` paraméterekkel és aktivációkkal számoló modellt kisebb bitszélességű, jellemzően `int8` számábrázolásra képezünk le. A közvetlen motiváció a memória- és sávszélesség-igény csökkentése, de a leképzés numerikus hibát visz be. A kérdés az, hogy ezt a hibát hogyan tudjuk a skála és a kerekítés szintjén pontosan korlátozni, és hol szűnik meg a korlát.

Előismeret. A 33.2. Batch és kis szolgáltatás inference-szerződése rögzíti, hogy a bemeneti kép `float32`, $[0;\ 1]$ intenzitású és NCHW alakú. A kvantálás ezen a bejárati szerződésen nem változtat: a modell továbbra is `float32` képet fogad, a kisebb bitszélesség a belső reprezentációban jelenik meg. A 30.2. Kontrasztív cél és temperature softmax-hőmérséklete azt mutatta, hogy ugyanakkora logitváltozás más predikciós érzékenységet okozhat; a 25.4. Nyílt halmaz és ismeretlen input rögzítette, hogy a modell állításai csak a tanulási eloszlás felhasználási tartományán érvényesek, a 33.4. Üzemeltetés, drift és késő címke pedig az input drift kimutatását tárgyalta. A kvantálás kalibrációja ugyanezt a domaineltérést teszi fel nagyító alá.

Induló kérdés. Ha egy `float32` értéket egy közös s skálával `int8` egészre kerekítünk, majd visszaállítunk, legfeljebb mekkora lehet a visszaállítás abszolút hibája, amíg nem érjük el az `int8` tartomány szélét? A válasz a 34.2.3. szakasz állítása: telítetlen tartományban a hiba legfeljebb $s/2$, telítéskor ennél nagyobb is lehet.

34.2.2 Definíció: affin kvantálás, skála, zero-point, vágás, kalibráció

Legyen az eredeti valós érték $r \in \mathbb{R}$. Egy **affin kvantáló** egy (s, z) pár, ahol

- $s > 0$ a **skála** (scale), mértékegysége megegyezik r mértékegységével;
- $z \in \mathbb{Z}$ a **zero-point**, dimenziótlan egész.

A kódolt egész szám

$$q = \operatorname{clamp}\left(\operatorname{round}\left(\frac{r}{s} + z\right),\ q_{\min},\ q_{\max}\right),$$

a visszaállított, **dekvantált** érték pedig

$$\hat{r} = s(q - z).$$

Itt $q_{\min} < q_{\max}$ egészek, `int8` esetén például $q_{\min} = -128$, $q_{\max} = 127$. A round a legközelebbi egészre kerekítés; döntetlen esetén a páros egész felé kerekít (bankár-kerekítés), ahogy a `np.rint` is. Az $\mathrm{clamp}(y, a, b) = \min(\max(y, a), b)$ a **vágás** (clipping). Ha $q_{\min} \le \mathrm{round}(r/s + z) \le q_{\max}$ nem teljesül, az érték **telítődik**: a kódolt érték a tartomány szélére esik, és a dekvantált érték nem követi tovább r-et.

A definícióból közvetlenül látszik a két szabadsági fok szerepe. A skála határozza meg, hogy egy egész lépés mekkora valós lépésnek felel meg. A zero-point azt mondja meg, melyik q egész kódolja a 0-t: ha $r = 0$, akkor a kerekítés előtt z áll, tehát 0-hoz általában $q \approx z$ tartozik. Aszimmetrikus tartomány kvantálható $z = 0$ mellett is, csak a kódtartományt rosszabbul használjuk ki: például előjel nélküli tartományban a [0; 2,5] értékek természetesen adhatnak $z = 0$-t. A nemnulla zero-point szükségessége csak a konkrét min–max recepthez és a teljes előjeles kódtartomány kihasználásához kötődik (például $r \in [0;\ 2{,}5]$ képezése $[-128;\ 127]$-re).

Két elterjedt granularitás van. **Per-tensor** kvantálásnál egy teljes tenzor minden eleme ugyanazt az (s, z) párt használja. **Per-channel** kvantálásnál a C csatorna mentén $s \in \mathbb{R}^C$ skálavektort és $z \in \mathbb{Z}^C$ zero-point vektort használunk; például egy $N \times C \times H \times W$ aktivációban az n, c, i, j elemhez az s_c, z_c tartozik. E két fogalmat nem szabad felcserélni, mert a per-channel finomabb felbontás kisebb kerekítési hibát adhat, de több kalibrációs adatot és más kernel-támogatást igényelhet.

A **kalibráció** (calibration) a skála és zero-point megválasztása reprezentatív adaton. Egy egyszerű, gyakori szabály a min-max kalibráció:

$$s = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}, \qquad z = q_{\min} - \mathrm{round}\Big(\frac{r_{\min}}{s}\Big),$$

ahol $r_{\min}, r_{\max}$ a kalibrációs tenzor megfigyelt minimuma és maximuma. A kvantálásnál a reprezentálható valós tartomány tartalmazza a nullát is: a kalibrált szélsőértékeket ezért $r_{\min} \leftarrow \min(r_{\min}, 0)$ és $r_{\max} \leftarrow \max(r_{\max}, 0)$ szerint bővítjük; ezután z a $[q_{\min};\ q_{\max}]$ tartományra vágható. Ha a kibővített $r_{\max} = r_{\min} = 0$, a $s = 1, z = 0$ konvenció használható. Egy nemnulla konstans tenzor esetén viszont a $s = 1, z = 0$ általában nem biztosítja az érték torzításmentes reprezentálását. A kalibráció a tanítás befejezése után, a tanítástól elkülönített adaton történik: a kalibrációs halmaz a train splitből kivett, a tanításban fel nem használt rész; a validáció és a teszt kalibrációra nem használható, a teszt a végső, már kvantált modell független értékelésére szolgál. Az affin kvantálás és a min–max kalibráció ipari leírása a [P27] munka; a PyTorch oldali megvalósítás és a per-channel támogatás leírása a [D22] dokumentációban van.

34.2.3 Levezetett hibakorlát telítetlen tartományban

A tanulási célhoz tartozó kötelező állítás:

Állítás (abszolút kerekítési hibakorlát). Legyen $s > 0$ valós, $z \in \mathbb{Z}$, $q_{\min} < q_{\max}$ egészek. Legyen $r \in \mathbb{R}$, $q_u = \mathrm{round}(r/s + z)$ a vágás előtti kerekített egész, és

$$\hat{r} = s\left(\mathrm{clamp}(q_u, q_{\min}, q_{\max}) - z\right).$$

Ha a **telítetlenségi feltétel**

$$q_{\min} \le q_u \le q_{\max}$$

teljesül, akkor

$$|\hat{r} - r| \le \frac{s}{2}.$$

Bizonyítás. Vezessük be a kerekítés előtti értéket: $y = \frac{r}{s} + z$. A legközelebbi egészre kerekítés definíciójából minden $y \in \mathbb{R}$ esetén

$$|\operatorname{round}(y) - y| \le \frac{1}{2}.$$

A telítetlenségi feltétel miatt a vágás hatástalan, tehát $q = q_u = \operatorname{round}(y)$. Jelölje $\varepsilon = \operatorname{round}(y) - y$; ekkor $|\varepsilon| \le 1/2$. A dekvantált érték:

$$\hat{r} = s(q - z) = s((y + \varepsilon) - z) = s\left(\frac{r}{s} + z + \varepsilon - z\right) = r + s\varepsilon.$$

Így

$$|\hat{r} - r| = |s\varepsilon| = s|\varepsilon| \le \frac{s}{2}.$$

Ezzel az állítást bebizonyítottuk. A bizonyítás semmilyen eloszlási feltevést nem használ; a korlát tetszőleges telítetlen r-re érvényes, tetszőleges z-vel. Ha a kerekítés döntetlennél nem a páros egész felé kerekít, hanem mindig felfelé, a $|\varepsilon| \le 1/2$ tulajdonság akkor is fennáll, tehát a korlát érvényes marad; a döntetlen-szabály csak az ε konkrét értékét befolyásolja.

Miért nem érvényes a korlát telítéskor? Ha y annyira nagy, hogy $q_u > q_{\max}$, akkor a vágás $q_{\max}$-ra cseréli a kerekített értéket, és a $\hat{r} = s(q_{\max} - z)$ konstanssá válik. Ekkor y további növelésével r nő, $\hat{r}$ viszont már nem, így $|r - \hat{r}|$ tetszőlegesen nagy lehet. Ugyanez igaz az alsó telítésre. A telítési hiba tehát nem a kerekítés, hanem a reprezentálható tartomány korlátosságának következménye.

34.2.4 Kézi számpélda

Legyen $s = 0{,}25$, $z = 0$, $q_{\min} = -128$, $q_{\max} = 127$. Négy értékre:

r	$\hat{r}$	$\lvert\hat{r} - r\rvert$	telített?
1,37	1,25	0,12	nem
−0,12	0	0,12	nem
0	0	0	nem
32,0	31,75	0,25	igen

A harmadik sor azt mutatja, hogy a zero-point itt éppen 0-ra kódolja a valós nullát. A negyedik sorban $r/s = 128$, a vágás előtti kerekítés 128, ami meghaladja a 127 felső határt; a vágás után $\hat{r} = 0{,}25 \cdot (127 - 0) = 31{,}75$. A hiba $0{,}25 > s/2 = 0{,}125$, mert a telítetlenségi feltétel sérült.

34.2.5 Labor: kis numerikus kvantálási példa

A labor célja egy önálló, Pythonban ellenőrizhető kvantáló és kalibráló függvény, amely nem backend-gyorsítást ígér, hanem a numerikus hibát méri. A kód tiszta NumPy, CPU, `float32` bemenettel.

Munkahely: `lab/q134_quant/`.

```
# quant_example.py
import numpy as np

def affine_quantize(x, scale, zero_point, qmin=-128, qmax=127):
    # x: float32 tömb; scale: pozitív float32; zero_point: int
    if not float(scale) > 0.0:
        raise ValueError("scale > 0 szükséges")
    y = x / np.float64(scale) + float(zero_point)
    q_unclamped = np.rint(y)          # legközelebbi egész, feles döntetlennél páros
    saturated = (q_unclamped < qmin) | (q_unclamped > qmax)
    q = np.clip(q_unclamped, qmin, qmax).astype(np.int64)
    x_hat = (np.float32(scale) * (q - zero_point)).astype(np.float32)
    return q, x_hat, saturated

def per_tensor_minmax(x, qmin=-128, qmax=127):
    # A nulla is legyen reprezentálható: a tartományt nullára bővítjük.
    rmin = min(0.0, float(np.min(x)))
    rmax = max(0.0, float(np.max(x)))
    if rmax == rmin:  # csak a csupa nulla tenzor esik ide; konstans nemnulla inputnál rmax !=
    ↪  rmin
        return np.float32(1.0), 0
    scale = np.float32((rmax - rmin) / (qmax - qmin))
    zero_point = qmin - int(np.rint(rmin / np.float64(scale)))
    zero_point = int(np.clip(zero_point, qmin, qmax))
    return scale, zero_point

if __name__ == "__main__":
    rng = np.random.default_rng(20261020)
    x = rng.normal(0.2, 0.5, size=(4, 8)).astype(np.float32)

    scale, z = per_tensor_minmax(x)
    q, x_hat, sat = affine_quantize(x, scale, z)
    err = np.max(np.abs(x_hat - x))

    print("scale=", scale)
    print("zero_point=", z)
    print("max_abs_err=", err)
    print("saturated_cells=", int(sat.sum()))

    # Ellenőrzés: telítetlen esetben a levezetett korlátnak teljesülnie kell.
    if int(sat.sum()) == 0:
        assert err <= np.float64(scale) / 2.0 + 1e-6, "hibás kerekítési korlát"

    # Determinisztikus kézi kontroll a 34.2.4. szakasz táblázatára.
    x_ref = np.array([1.37, -0.12, 0.0, 32.0], dtype=np.float32)
    q_ref, x_hat_ref, sat_ref = affine_quantize(
        x_ref, np.float32(0.25), 0, qmin=-128, qmax=127
    )
    print("control_q=", q_ref)
    print("control_x_hat=", x_hat_ref)
    print("control_saturated=", sat_ref)
    assert np.allclose(x_hat_ref[:3], np.array([1.25, 0.0, 0.0], dtype=np.float32),
                       atol=1e-6)
    assert q_ref[3] == 127 and sat_ref[3]
    print("control_ok")
```

Futtatás:

```
cd lab/q134_quant
python quant_example.py
```

A rögzített ellenőrző környezetben futtatott példa (Python 3.12.3, NumPy 2.5.4, CPU, seed 20261020, alapértelmezett szálbeállítás) mért kimenete:

```
scale= 0.0075516887
zero_point= -13
max_abs_err= 0.0037441552
saturated_cells= 0
control_q= [  5   0   0 127]
control_x_hat= [ 1.25  0.    0.   31.75]
control_saturated= [False False False  True]
control_ok
```

A determinisztikus kontroll a 34.2.4. szakasz táblázatát adja vissza: `control_q=[5 0 0 127]`, `control_x_hat=[1.25 0. 0. 31.75]`, `control_saturated=[False False False True]`. A véletlen tenzorra a script kiírja a skálát, zero-pointot, maximális abszolút hibát és a telített elemek számát; ezek a rögzített seed mellett reprodukálhatók. Valódi backend-gyorsulást kizárólag tényleges mérés alapján szabad közölni, a 34.1. Mért késleltetés, throughput és memória mérési protokolljával.

34.2.6 Feladatok

34.2.6.1 F1 fogalmi feladat (nehézség 1)

Input: két (s, z) pár: $(0{,}25;\ 0)$ és $(0{,}1;\ -10)$; egy rögzített $q = 0$ egész.

Előfeltétel: ismeri az affin kvantálás definícióját.

Beadandó eredmény: magyarázza el a skála és a zero-point különbségét, és adja meg mindkét párhoz a $q = 0$-hoz tartozó dekvantált értéket. Indokolja, miért nem elegendő a q egész bitminta önmagában az eredeti valós érték rekonstruálásához.

34.2.6.2 F2 levezetési feladat (nehézség 2)

Input: $s = 0{,}25$, $z = 0$, $q_{\min} = -128$, $q_{\max} = 127$, $r = 32{,}0$.

Előfeltétel: ismeri a 34.2.3. szakasz állítását és bizonyítását.

Beadandó eredmény: lépésenként számolja ki $r/s + z$ értékét, a kerekített q_u-t, a vágott q-t és a dekvantált $\hat{r}$-et. Mutassa meg, hogy a hiba $0{,}25$, ami nagyobb $s/2$-nél, és nevezze meg pontosan, hogy a tétel mely feltétele nem teljesül.

34.2.6.3 F3 implementációs feladat (nehézség 2)

Input: $X \in \mathbb{R}^{4\times 6}$, per-channel skálák $s \in \mathbb{R}^6$, zero-pointok $z \in \{0;\ \ldots;\ 5\}^6$; a pontos numerikus értékeket a hallgató választja, de rögzítse seeddel: `np.random.default_rng(20261020)`.

Előfeltétel: a labor kódjának működése; `float32` dtype.

Beadandó eredmény: futtatható `quantize_per_channel(x, scales, zero_points, qmin, qmax)` függvény, amely minden c oszlophoz külön (s_c, z_c) párt használ. Ellenőrizze, hogy minden olyan oszlopra, ahol nincs telített elem, a maximális abszolút hiba legfeljebb $s_c/2$ legyen. A megoldásnak tartalmaznia kell a teljes kódot és a kontroll kimenetét.

34.2.6.4 F4 hibakeresési feladat (nehézség 2)

Input: az alábbi kalibrációs kódrészlet:

```
scale = (rmax - rmin) / 255.0
z = qmin - int(rmin / scale)      # hibaforrás
```

Előfeltétel: ismeri a min-max kalibráció képletét és a Python `int` szemantikáját.

Beadandó eredmény: mutasson egy konkrét `rmin` értéket, ahol a `int(rmin / scale)` nem a kívánt legközelebbi egész kerekítést adja. Magyarázza el a hiba okát, következményét a reprezentálható tartományra, adja meg a javítást, és írja le azt a numerikus ellenőrzést, amellyel a javítás helyességét visszaigazolja.

34.2.6.5 F5 kísérlettervezési/transzfer feladat (nehézség 3)

Input: egy, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLabon tanított kis CNN `float32` súllyal; a `metadata.csv` scene-szintű splitje; korlátozott CPU-erőforrás.

Előfeltétel: a 33.2. Batch és kis szolgáltatás inference-szerződése és a 34.1. Mért késleltetés, throughput és memória mérési módszertana; a kvantálási hibamértéket ebben az egységben vezetjük be.

Beadandó eredmény: egy írásbeli kísérleti terv post-training quantization (PTQ) és `float32` baseline összehasonlítására. Azonosítsa a kalibrációs halmazt, a rögzített seedet (például 20261020), a numerikus hiba mérésének módját, a validációs metrikát és a kísérlet érvényességi korlátját. Tüntesse fel, mely állításokat nem szabad levonni valódi backend-futás nélkül.

34.2.7 Megoldások és indoklás

34.2.7.1 F1 megoldás

A dekvantálási képlet $\hat{r} = s(q - z)$.

- $(s, z) = (0{,}25;\ 0)$: $q = 0 \Rightarrow \hat{r} = 0{,}25(0 - 0) = 0$.
- $(s, z) = (0{,}1;\ -10)$: $q = 0 \Rightarrow \hat{r} = 0{,}1(0 - (-10)) = 1{,}0$.

Ugyanaz a $q = 0$ bitminta tehát az egyik kvantálóban 0-t, a másikban 1,0-t jelent. A skála adja egy egész lépés valós méretét, a zero-point pedig a nulla valós érték kódját; mindkettő ismerete szükséges, mert a tárolt q önmagában csak egy egész, nem hordozza a mértékegységet és az origóeltolást.

34.2.7.2 F2 megoldás

1. $y = r/s + z = 32{,}0/0{,}25 + 0 = 128$.
2. $q_u = \mathrm{round}(128) = 128$.
3. Mivel $128 > q_{\max} = 127$, a vágás: $q = 127$, telítés következik be.
4. $\hat{r} = s(q - z) = 0{,}25 \cdot 127 = 31{,}75$.
5. A hiba: $|31{,}75 - 32{,}0| = 0{,}25$, míg $s/2 = 0{,}125$.

A 34.2.3. szakasz állítása felteszi, hogy $q_{\min} \le q_u \le q_{\max}$. Itt $q_u = 128 > 127$, tehát a feltétel nem teljesül; a kapott 0,25 hiba ezért nem mond ellent a tételnek. Alakellenőrzés: r, $\hat{r}$ és a hiba a skálával azonos mértékegységű (a példában dimenziótlan, mert s is az); q és z egész, dimenziótlan.

34.2.7.3 F3 megoldás

```python
# per_channel_quant.py
import numpy as np

def quantize_per_channel(x, scales, zero_points, qmin=-128, qmax=127):
    x = np.asarray(x, dtype=np.float32)
    scales = np.asarray(scales, dtype=np.float32)
    zero_points = np.asarray(zero_points, dtype=np.int64)
    y = x / scales.astype(np.float64) + zero_points.astype(np.float64)
    q_unclamped = np.rint(y)
    saturated = (q_unclamped < qmin) | (q_unclamped > qmax)
    q = np.clip(q_unclamped, qmin, qmax).astype(np.int64)
    x_hat = (scales * (q - zero_points)).astype(np.float32)
    return q, x_hat, saturated

if __name__ == "__main__":
    rng = np.random.default_rng(20261020)
    x = rng.normal(0.0, 1.0, size=(4, 6)).astype(np.float32)
    scales = rng.uniform(0.05, 0.15, size=(6,)).astype(np.float32)
    zero_points = rng.integers(0, 6, size=(6,))
    q, x_hat, sat = quantize_per_channel(x, scales, zero_points)
    for c in range(6):
        col_sat = sat[:, c]
        err_c = np.max(np.abs(x_hat[:, c] - x[:, c]))
        if not np.any(col_sat):
            assert err_c <= np.float64(scales[c]) / 2.0 + 1e-6, (c, err_c)
        print(f"channel={c} z={zero_points[c]} scale={scales[c]:.6f} "
              f"max_abs_err={err_c:.6f} saturated={int(np.sum(col_sat))}")
    print("per_channel_control_ok")
```

A rögzített seeddel a kimenet reprodukálható; a mért kulcssorok (Python 3.12.3, NumPy 2.5.4, CPU, seed 20261020, alapértelmezett szálbeállítás; a pontosság utolsó jegye a lebegőpontos formázástól függhet):

```
channel=0 z=2 scale=0.056499 max_abs_err=0.025332 saturated=0
channel=4 z=2 scale=0.120421 max_abs_err=0.054341 saturated=0
per_channel_control_ok
```

Mind a hat oszlopban `saturated=0`, és a legnagyobb hiba a 4-es csatornán 0,0543, ami kisebb a $s_4/2 = 0{,}0602$ korlátnál.

A vektorizáció a `scales` és `zero_points` broadcastingjára épül: a $N \times C$ alakú `x` oszlopaihoz a C hosszú vektorok rendelődnek. A tolerancia `1e-6`, mert a kerekítési korlát felső határa analitikus, a lebegőpontos műveletek viszont 10^{-6}-nál kisebb numerikus eltérést adhatnak `float32`-ben.

Szélsőeset: ha egy oszlop telített elemet tartalmaz, a korlátot nem követeljük meg, a konzol kimutatja a telített cellák számát.

34.2.7.4 F4 megoldás

A Python `int(x)` függvény nulla felé csonkít, nem a legközelebbi egészre kerekít. Példa: $r_{\min} = -1{,}37$, $s = 0{,}1$, $q_{\min} = -128$. Ekkor $r_{\min}/s = -13{,}7$. Az `int(-13.7)` értéke -13, a legközelebbi egész viszont -14. A rossz zero-point ezért eggyel kisebb lesz a helyesnél ($q_{\min} + 13$ a $q_{\min} + 14$ helyett), és a reprezentálható legkisebb érték nem éri el a tényleges minimumot; a legkisebb bemenetek telítődnek, szisztematikus pozitív torzítást okozva.

Javítás:

```
z = qmin - int(np.rint(rmin / scale))
```

Ellenőrzés: a javított z-vel számoljuk ki a kalibrációs minimum kvantálását. Konkrétan $r_{\min} = -1{,}37$, $s = 0{,}1$ mellett a helyes $z = q_{\min} - (-14) = q_{\min} + 14 = -114$; ezzel $r_{\min}/s + z = -13{,}7 - 114 = -127{,}7$, kerekítve $-128 = q_{\min}$, tehát a vágás előtti kerekítés éppen az alsó határra esik, és $\hat{r} = s(q_{\min} - z) = 0{,}1 \cdot (-128 + 114) = -1{,}4$, a hiba $0{,}03 \le s/2 = 0{,}05$. A rossz $z = q_{\min} + 13 = -115$ mellett $r_{\min}/s + z = -128{,}7$, kerekítve -129, amit a vágás -128-ra szorít; $\hat{r} = -1{,}3$, a hiba $0{,}07 > s/2$.

```
import numpy as np
rmin, s, qmin, qmax = -1.37, 0.1, -128, 127
for name, z in (("rossz", qmin - int(rmin / s)),
                ("helyes", qmin - int(np.rint(rmin / s)))):
    q_u = np.rint(rmin / s + z)
    q = np.clip(q_u, qmin, qmax)
    x_hat = round(float(s * (q - z)), 3)
    print(name, "z=", z, "q_u=", q_u, "q=", q, "x_hat=", x_hat)

assert qmin - int(rmin / s) == -115
assert qmin - int(np.rint(rmin / s)) == -114
```

A mért kimenet (Python 3.12.3, NumPy 2.5.4, CPU, seed nélkül determinisztikus, alapértelmezett szálbeállítás): `rossz z= -115 q_u= -129.0 q= -128.0 x_hat= -1.3`, `helyes z= -114 q_u= -128.0 q= -128.0 x_hat= -1.4`. A rossz változat tehát telítődik a kalibrációs minimumon, és a hibája 0,07, ami meghaladja az $s/2 = 0{,}05$ korlátot.

34.2.7.5 F5 megoldásminta

Egy lehetséges terv:

1. **Kérdés:** egy már betanított GlyphLab-CNN per-tensor PTQ int8 szimulációja megőrzi-e a validációs pontosságot a `float32` baseline-hoz képest, és mekkora a maximális logiteltérés?
2. **Adat és split:** a meglévő scene-szintű train/val/test split rögzített; kalibrációra a train egy külön, rögzített seeddel (például 20261020) kiválasztott 30% része használható, nem a val/test.
3. **Baseline:** ugyanaz a checkpoint `float32` súllyal; a val_loss alapján már kiválasztott, a tesztet a protokoll végéig nem használjuk.
4. **Kvantálás:** csak a súlyok szimulált kvantálása a labor `affine_quantize` függvényével; aktivációkra nem állítunk backend-int8 futtatást.
5. **Metrikák:** validációs accuracy, valamint a kvantálatlan és kvantált logitok maximális abszolút eltérése; a tárolási méret csökkenése `int8` feltételezéssel.

6. **Korlátok:** ez numerikus kompatibilitási vizsgálat, nem sebességmérés; semmilyen gyorsulást nem állítunk valódi, a 34.1. Mért késleltetés, throughput és memória protokoll szerinti latencia/throughput mérés nélkül. A GlyphLab corrupt (mesterségesen torzított) teszthalmazát csak a végleges kvantált modell kiválasztása után, utólagos hangolás nélkül, a clean teszttel párosított sorrendben szabad értékelni.

34.2.8 Tipikus tévedés, önellenőrzés és visszatekintés

Gyakori tévedés. „Az int8 reprezentáció automatikusan kisebb és gyorsabb modellt ad." Az int8 *tárolás* valóban csökkentheti a paraméterek bájtjait, de a *futás* csak akkor gyorsul, ha az adott háttérben (CPU-utasításkészlet, GPU-kernel, runtime) az összes érintett operátornak létezik támogatott int8 implementációja. Ellenpélda: ha egy háló egyik operátorához nincs int8 kernel, a futtatórendszer dekvantálja a bemenetet, `float32`-ben számol, majd újra kvantál. Ekkor a memória csökkenhet, de a számítási idő akár nőhet is a többlet dequantize/quantize lépések miatt; a fenti labor tiszta NumPy szimulációja ugyancsak nem gyors kernel. Az int8 tehát egy reprezentáció, nem egy sebességígéretekre jogosító operátorszerződés.

Önellenőrzés. (1) Ha egy telítetlen tartományban a skálát megduplázzuk, mi történik az abszolút hibakorláttal? (2) Miért nem lehet a zero-point tetszőleges valós szám? (3) A kalibráció használhatja-e a validációt?

Válaszok: (1) A $\hat{r} - r = s\varepsilon$ alakból ($|\varepsilon| \le 1/2$) a korlát $s/2$-ről $2s/2 = s$-re nő; a nagyobb lépésköz nagyobb maximális hibát enged. (2) Mert a kódolt értéket egész q-val tároljuk, és a dekvantálás $s(q - z)$: ha z nem egész, a $q - z$ különbség nem egész, így a kódok rácsának origója nem esik egész helyre, és a valós nulla sem kap pontos egész kódot. Az egész aritmetika és a nulla pontos reprezentálása miatt a definíció $z \in \mathbb{Z}$-t köt ki. (3) Nem; a kalibráció a modell belső paramétereinek választása, ezért ugyanaz a logika tiltja, mint a hiperparaméter-választást a validáción: a kiértékelő halmazt érintetlenül kell hagyni.

Korábbi fogalom felidézése. A 33.2. Batch és kis szolgáltatás szerint az inference bemenet `float32` és $[0;\ 1]$-beli; a kvantálás ezt a bemeneti szerződést nem írja felül, hanem a modell belsejében történik. A 30.2. Kontrasztív cél és temperature arról tanított, hogy a softmax hőmérséklete felnagyíthatja a logitok közötti kis különbségeket: egy $|\delta|$ logithiba predikciós hatása ezért nem pusztán a δ-tól, hanem az érvényes hőmérséklettől és a szomszédos osztálypontszámoktól is függ. A 25.4. Nyílt halmaz és ismeretlen input OOD-fogalma analóg módon figyelmeztet: a kalibrációs eloszlásnak a kiszolgálási eloszlást kell reprezentálnia, különben a kvantálási hiba egy megjósolható domaineltérésre rakódik.

34.2.9 Rövid összefoglaló

Az affin kvantálás egy valós r értéket a $q = \mathrm{clamp}(\mathrm{round}(r/s + z), q_{\min}, q_{\max})$ egészre képez, a dekvantálás pedig $\hat{r} = s(q - z)$. A skála az egész lépés valós hossza, a zero-point a nulla kódja. Telítetlen tartományban a kerekítésből $|\hat{r} - r| \le s/2$ következik, mert a legközelebbi egész hibája legfeljebb $1/2$, és a skálával szorzás ezt $s/2$-re skálázza. Telítéskor a vágás miatt a hiba tetszőlegesen nagy lehet. A kalibráció reprezentatív adaton választja meg az (s, z) párt, de a tesztet nem érintheti. Az int8 tárolás önmagában nem bizonyít backend-támogatást vagy gyorsulást.

34.2.10 Továbblépés

A 34.3. Pruning és struktúra egység a pruning és a strukturált ritkítás kérdését vizsgálja: ott azt elemezzük, hogyan távolíthatók el paraméterek vagy szerkezeti egységek a hibakorlát és a mért hatékonyság megőrzése mellett.

34.2.11 Források

A hibakorlát levezetése és a kód önálló példa. A kapcsolódó mérési módszereket a 34.1. Mért késleltetés, throughput és memória, a modell exportját a 33.3. torch.export és ONNX, a desztillációt a 34.4. Teacher–student desztilláció tárgyalja.

- [P27] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D. (2018). *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. CVPR 2018. https://arxiv.org/abs/1712.05877.
- [D22] PyTorch contributors (2026). *Quantization*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/quantization.html. Hozzáférés: 2026-10-30.

34.3 Pruning és struktúra

34.3.1 Cél, előismeret és induló kérdés

A 34.2. Kvantálás és hibatovábbterjedés egységben a kvantálás a tárolandó súly *bitszélességét* csökkentette, miközben az egyes rétegek alakja és műveletszáma változatlan maradt. A pruning ezzel szemben a paraméterek *számát* akarja csökkenteni: a kérdés az, hogy egy súly elhagyása ténylegesen kisebb futási költséget jelent-e. Az induló kérdés: van egy lineáris rétegünk, amelynek súlymátrixában a belépések 70 százaléka nulla; a `torch.nn.Linear` ugyanannyi szorzást végez-e ekkor, mint a nulla nélküli változaton? A válasz a szokásos sűrű futási úton **igen**: a sűrű `Linear` kernel a nulla súlypozíciókat is bejárja, ugyanannyi szorzást végez. Kevesebb számítás csak akkor lesz, ha a nullák mintázatát ténylegesen kihasználó ritka kernel fut, vagy a pruning a tensor dimenzióját is csökkenti (például egész csatornák eltávolításával). Ezért kell szétválasztanunk a *nullázott súlyt* és a *ténylegesen kihagyott számítást*. A témához tartozik az unstructured/structured pruning, a ritkasági arány, a csatorna-eltávolítás, az újratanítás és a kernel-támogatás kérdése is.

Előismeret: a tenzoralakok és paraméterszámlálás a 14.3. Paraméterszám, MAC és aktivációmemória egységből, a PyTorch `nn.Module` regisztrációja a 11.2. nn.Module és paraméterregisztráció egységből, valamint a kvantálásból származó numerikus hibakezelés a 34.2. Kvantálás és hibatovábbterjedés egységből. A 33.3. torch.export és ONNX export-ellenőrzés gondolata abban tér vissza, hogy egy pruned modellt is ugyanúgy exportálni és ellenőrizni kell; a 30.3. Negatív nélküli módszerek és collapse egységből a „kontroll nélkül nem állítunk javulást” elve érvényes; a 22.3. Zajos és hiányos címke egység pedig arra emlékeztet, hogy a tanítási és értékelési kiírásokat a pruning után sem szabad összekeverni.

34.3.2 Ritkaság, pruning és kernel-támogatás

Legyen $W \in \mathbb{R}^{m \times d}$ egy teljesen összekötött réteg súlymátrixa. A **strukturálatlan pruning** (unstructured pruning) elemenkénti maszkot rendel a súlyhoz: $M \in \{0;\ 1\}^{m \times d}$, és a használt súly

$$\widetilde{W} = M \odot W,$$

ahol $\odot$ elemenkénti szorzás. A **ritkaság** (sparsity) a nullázott elemek aránya:

$$\rho = \frac{\#\{(i,j) : M_{ij} = 0\}}{md} = 1 - \frac{\sum_{i,j} M_{ij}}{md}.$$

Ha $\rho = 0{,}7$, a súlytároló héttizede elvileg elhagyható, de a tényleges megtakarítás attól függ, mit tud a futtató kernel. A **kernel-támogatás** fogalma: egy ritkítási mintázatot akkor támogat a futási út, ha a minta alapján a végrehajtott szorzások és memóriamozgatások száma is csökken. A sűrű (dense) mátrixszorzó implementáció ezt nem teszi meg automatikusan: bejárja mind a md súlypozíciót, akár nulla az ott tárolt érték, akár nem.

A **strukturált pruning** (structured pruning) nem elemeket, hanem teljes tensorszeleteket távolít el: egy lineáris rétegnél kimeneti neuront, konvolúciós rétegnél pedig egy teljes bemeneti vagy kimeneti csatornát. A **channel removal** a konvolúciós súly $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$ valamelyik csatornaindexét törli a tensorból, nem nullává teszi. Ekkor az új súly alakja kisebb, tehát az általános dense konvolúciós kernel is kevesebb tényleges szorzást végez. A strukturált pruning azonban a hálózat függvényét általában megváltoztatja, ezért **fine-tune**, azaz a megmaradt súlyokból kiinduló további tanítás követi.

A PyTorch `torch.nn.utils.prune` modulja maszkolást valósít meg: a maszk külön bufferként jelenik meg a paraméter mellett, és a forward kiértékelés a maszkolt súlyt használja. A modul strukturált maszkoló módszereket is tartalmaz (például `ln_structured`, `random_structured`), de ezek sem feltétlenül távolítják el fizikailag a csatornákat. A lényegi különbség a *maszkolás* (nullázás) és a tenzor *tényleges átalakítása* (kisebb alakú új tenzor), nem az, hogy a modul kizárólag strukturálatlan volna; a maszkolás önmagában nem csökkenti a dense kernel műveletszámát. A strukturálatlan pruning klasszikus leírása [P28], a strukturált szűrőpruningé [P29].

34.3.3 Csatorna eltávolítás mint strukturált pruning: alak- és paraméterlemma

Legyen L_1 egy `Conv2d` réteg, amely C_0 bemeneti csatornából C_1 kimeneti csatornába képez $k_h \times k_w$ kernelmérettel, és legyen L_2 egy második `Conv2d`, amely C_1 bemeneti csatornából C_2 kimeneti csatornába képez $k'_h \times k'_w$ kernelmérettel. Mindkét réteg legyen biasszal ellátott. A $\star$ művelet térbeli cross-correlation (a kernel nem fordul meg), nem matematikai konvolúció.

Lemma (csatornatörlés ekvivalenciája nulla fogyasztósúly esetén). Ha valamely $j \in \{0;\ \dots;\ C_1 - 1\}$ indexre $K_2[q, j, \cdot, \cdot] = 0$ minden $q \in \{0;\ \dots;\ C_2 - 1\}$ kimeneti csatornára, akkor a j-edik kimeneti csatorna törlése L_1-ből és a j-edik bemeneti csatorna törlése L_2-ből a kompozíció függvényét pontosan változatlanul hagyja.

Bizonyítás. Jelölje $Y = L_1(x)$ és $Y' = L'_1(x)$ a törlés előtti és utáni közbülső aktivációt. Y' csatornái éppen Y csatornái a j index kihagyásával, mert a megmaradó csatornák súlyai és biasai változatlanok. A q-adik kimenetre

$$Z_q = b_2[q] + \sum_{c \neq j} K_2[q, c] \star Y_c + \underbrace{K_2[q, j] \star Y_j}_{=0}.$$

A feltétel miatt a harmadik tag azonosan nulla, így a $c \neq j$ indexek összege pontosan a törlés utáni $L'_2(Y')$ értékét adja. Ezzel az állítást beláttuk. A bizonyítás kulcsa, hogy a törölt csatornát a fogyasztó oldalán is töröltük. A nulla fogyasztósúly egyszerű, lokális, *elegendő* feltétel az

ekvivalenciához. A nem nulla fogyasztósúlyból azonban nem következik, hogy a csatorna törlése megváltoztatja a függvényt: a csatorna lehet minden inputon nulla (például nulla súlyú és biasú, vagy ReLU után mindig inaktív), vagy más útvonal kompenzálhatja. A szükségesség állításához nemdegeneráltsági feltétel (a csatorna aktivációja nem azonosan nulla és független az egyéb utaktól) vagy konkrét ellenpélda kell.

Állítás (paraméterszám-csökkenés egy csatorna eltávolításakor). A fenti két réteg paraméterszáma a törlés előtt

$$P = C_1(C_0 k_h k_w + 1) + C_2(C_1 k'_h k'_w + 1),$$

a törlés után pedig

$$P' = (C_1 - 1)(C_0 k_h k_w + 1) + C_2((C_1 - 1) k'_h k'_w + 1),$$

feltéve, hogy $C_1 \geq 2$. A különbség

$$\Delta = P - P' = C_0 k_h k_w + 1 + C_2 k'_h k'_w.$$

Bizonyítás. Az L_1 súlyának alakja $C_1 \times C_0 \times k_h \times k_w$, minden kimeneti csatornához $C_0 k_h k_w$ súly és 1 bias tartozik, ezért $C_1(C_0 k_h k_w + 1)$. A j-edik kimeneti szelet törlése után a kimeneti méret $C_1 - 1$, a biasvektor egy elemmel rövidebb. Az L_2 súlyának alakja $C_2 \times C_1 \times k'_h \times k'_w$; a j-edik bemeneti szelet törlése a középső tengely hosszát csökkenti $C_1 - 1$-re, a biasvektor hossza pedig változatlan marad, mert a kimeneti csatornák száma ugyanaz. A kivonást elvégezve

$$\begin{aligned} \Delta &= C_1(C_0 k_h k_w + 1) - (C_1 - 1)(C_0 k_h k_w + 1) \\ &\quad + C_2(C_1 k'_h k'_w + 1) - C_2((C_1 - 1) k'_h k'_w + 1) \\ &= C_0 k_h k_w + 1 + C_2 k'_h k'_w. \end{aligned}$$

A törlés során a közbülső aktiváció alakja ugyanígy $N \times C_1 \times H \times W$ alakról $N \times (C_1 - 1) \times H \times W$ alakra változik; a térbeli H, W dimenziók nem változnak, ha a padding, stride és dilation beállítások azonosak maradnak.

34.3.4 Kézi példa: 1×1 csatornaeltávolítás

Használjunk a 19.2. MobileNet és inverted residual egységben megismert pontonkénti rétegeket, hogy minden szám kézzel követhető legyen. Legyen $C_0 = 2$, $C_1 = 3$, $C_2 = 2$, $k_h = k_w = k'_h = k'_w = 1$. A bemenet legyen az egyelemű $1 \times 2 \times 1 \times 1$ alakú tensor, $X_{0,0} = 1$ és $X_{0,1} = 2$. Az L_1 súlya és biása:

$$W_1[0,:] = [1;\ 0],\ b_1[0] = 0; \qquad W_1[1,:] = [0;\ 2],\ b_1[1] = 1; \qquad W_1[2,:] = [-1;\ 1],\ b_1[2] = 0.$$

Ekkor $Y_1[0] = 1$, $Y_1[1] = 0 \cdot 1 + 2 \cdot 2 + 1 = 5$, $Y_1[2] = -1 + 2 = 1$. Az L_2 úgy legyen megválasztva, hogy az 1-es bemeneti csatorna fogyasztósúlya nulla legyen:

$$W_2[0,:] = [2;\ 0;\ -1],\ b_2[0] = 0; \qquad W_2[1,:] = [1;\ 0;\ 3],\ b_2[1] = -2.$$

A kimenet

$$Z_0 = 2 \cdot 1 + 0 \cdot 5 - 1 \cdot 1 = 1, \qquad Z_1 = 1 \cdot 1 + 0 \cdot 5 + 3 \cdot 1 - 2 = 2.$$

Töröljük most az 1-es közbülső csatornát. A megtartott indexek 0 és 2, az új L_1' súlya $2 \times 2 \times 1 \times 1$ alakú:

$$W_1'[0,:] = [1;\ 0],\ b_1'[0] = 0; \qquad W_1'[1,:] = [-1;\ 1],\ b_1'[1] = 0,$$

ezért $Y' = [1;\ 1]$. Az új L_2' súlyában az eredeti $2 \times 3 \times 1 \times 1$ súly középső bemeneti oszlopát töröljük:

$$W_2'[0,:] = [2;\ -1],\ b_2'[0] = 0; \qquad W_2'[1,:] = [1;\ 3],\ b_2'[1] = -2.$$

A kimenet

$$Z_0' = 2 \cdot 1 - 1 \cdot 1 = 1, \qquad Z_1' = 1 \cdot 1 + 3 \cdot 1 - 2 = 2,$$

tehát a törlés előtti kimenettel pontosan egyezik, ahogy a lemma állítja. A paraméterszám a törlés előtt $P = 3(2 \cdot 1 + 1) + 2(3 \cdot 1 + 1) = 9 + 8 = 17$, utána $P' = 2(2+1) + 2(2+1) = 6 + 6 = 12$, a különbség 5. A harmadik szakasz képlete szerint $\Delta = C_0 k_h k_w + 1 + C_2 k_h' k_w' = 2 \cdot 1 + 1 + 2 \cdot 1 = 5$, ami egyezik.

34.3.5 Ellenőrizhető PyTorch-munkamenet

A következő példa a fenti számítást a PyTorch `Conv2d` rétegeivel ellenőrzi. A példa futtatható; a rögzített ellenőrző környezetben (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 0) futtatva a kimenet `[1.0, 2.0]`, és a beépített `torch.testing.assert_close` is lefut.

```
# munkakönyvtár: lab/
# python n135_prune_example.py
import torch
import torch.nn as nn

torch.manual_seed(0)
x = torch.tensor([[[[1.0]], [[2.0]]]], dtype=torch.float32)

def make_pair():
    l1 = nn.Conv2d(2, 3, 1)
    l2 = nn.Conv2d(3, 2, 1)
    with torch.no_grad():
        l1.weight.copy_(torch.tensor([
            [[[1.0]], [[0.0]]],
            [[[0.0]], [[2.0]]],
            [[[-1.0]], [[1.0]]]
        ], dtype=torch.float32))
        l1.bias.copy_(torch.tensor([0.0, 1.0, 0.0]))
        l2.weight.copy_(torch.tensor([
            [[[2.0]], [[0.0]], [[-1.0]]],
            [[[1.0]], [[0.0]], [[3.0]]]
        ], dtype=torch.float32))
        l2.bias.copy_(torch.tensor([0.0, -2.0]))
    return l1, l2
```

```
def remove_channel(l1, l2, j):
    # Korlátozott hatókör: CPU, float32, groups=1, dilation=1, zero padding.
    for l in (l1, l2):
        assert l.groups == 1 and l.dilation == (1, 1)
        assert l.padding_mode == "zeros" and l.weight.dtype == torch.float32
        assert l.weight.device.type == "cpu"
    # A törlés csak akkor kimenetmegőrző, ha a fogyasztó súlya a törölt csatornán nulla.
    assert torch.all(l2.weight[:, j] == 0), "a törölt csatorna fogyasztósúlya nem nulla"
    keep = [c for c in range(l1.out_channels) if c != j]
    l1p = nn.Conv2d(l1.in_channels, l1.out_channels - 1, l1.kernel_size,
                    stride=l1.stride, padding=l1.padding, bias=True)
    l2p = nn.Conv2d(l2.in_channels - 1, l2.out_channels, l2.kernel_size,
                    stride=l2.stride, padding=l2.padding, bias=True)
    with torch.no_grad():
        l1p.weight.copy_(l1.weight[keep])
        l1p.bias.copy_(l1.bias[keep])
        l2p.weight.copy_(l2.weight[:, keep])
        l2p.bias.copy_(l2.bias)
    return l1p, l2p

l1, l2 = make_pair()
l1p, l2p = remove_channel(l1, l2, 1)
z = l2(l1(x))      # mért: [[[1.0]], [[2.0]]]
zp = l2p(l1p(x))   # mért: ugyanaz
torch.testing.assert_close(z, zp, rtol=1e-7, atol=1e-7)
print(z.flatten().tolist())  # [1.0, 2.0]
```

Mért kimenet:

```
[1.0, 2.0]
```

A `remove_channel` függvény ténylegesen új `Conv2d` objektumokat hoz létre, és a megtartott indexekkel másolja át a súlyokat; nem maszkol. A törlés után a paraméterszám a `sum(p.numel() for p in l1.parameters())` hívásokkal is ellenőrizhető: 9-ből 6 és 8-ból 6 lesz.

34.3.6 Labor: ritkítási specifikáció, pontosság- és memóriakontroll

A labor egy, a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-CNN-en két modellt hasonlít össze: egy tanított baseline-t és egy strukturáltan pruned változatot. Az alábbi eredmények közül a paraméterszámokat a Python-beli alakokból számítottuk és példányosítással ellenőriztük; a pontosság és futásidő mérését a protokoll szerint kell elvégezni, ezért ezekre itt nem szerepel érték.

```
# lab/n135_prune_lab.py (vázlat)
import torch
import torch.nn as nn

class TinyGlyphCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.head = nn.Linear(16 * 12 * 12, 3)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.relu(self.conv2(x))
        x = self.pool(x)
        return self.head(torch.flatten(x, 1))
```

```
def channel_l1(weight):
    # weight: (C_out, C_in, kh, kw)
    return weight.abs().sum(dim=(1, 2, 3))

def lowest_l1(weight, count):
    # a `count` legkisebb csatorna-1 normájú kimeneti csatorna indexe
    return torch.argsort(channel_l1(weight))[:count].tolist()

def remove_pair(conv1, conv2, drop1, drop2):
    keep1 = [c for c in range(conv1.out_channels) if c not in drop1]
    keep2 = [c for c in range(conv2.out_channels) if c not in drop2]
    c1p = nn.Conv2d(conv1.in_channels, conv1.out_channels - len(drop1),
                    conv1.kernel_size, stride=conv1.stride,
                    padding=conv1.padding, bias=True)
    c2p = nn.Conv2d(conv2.in_channels - len(drop1),
                    conv2.out_channels - len(drop2),
                    conv2.kernel_size, stride=conv2.stride,
                    padding=conv2.padding, bias=True)
    with torch.no_grad():
        c1p.weight.copy_(conv1.weight[keep1])
        c1p.bias.copy_(conv1.bias[keep1])
        c2p.weight.copy_(conv2.weight[keep2][:, keep1])
        c2p.bias.copy_(conv2.bias[keep2])
    return c1p, c2p
```

A ritkítási specifikáció: a baseline tanítása után a `conv1` nyolc kimeneti csatornájából kettőt, a `conv2` tizenhat kimeneti csatornájából négyet törlünk; mindkét törlést a train-állapotból számított legkisebb csatorna-ℓ_1-normák alapján választjuk, a val- vagy test-eloszlás megtekintése nélkül. Az `--drop1 2 --drop2 4` a törlendő csatornák *számát* adja meg, nem indexet: a konkrét indexeket a `lowest_l1` állítja elő, például `drop1 = lowest_l1(conv1.weight, args.drop1)` és `drop2 = lowest_l1(conv2.weight, args.drop2)`. Az új fej bemeneti mérete $12 \cdot 12 \cdot 12 = 1728$ lesz; a régi `head` súlyát nem lehet átmásolni, mert a bemeneti reprezentáció csatornaszáma megváltozott, ezért a fejet újra kell inicializálni. A `TinyGlyphCNN` 24×24-es bemenetet feltételez (két 3×3-as, padding=1 konvolúció után 24×24, egy 2×2-es pool után 12×12), ezért a fej bemenete $16 \cdot 12 \cdot 12$; más képmérethez a fej mérete is változik.

A paraméterszám:

Modell	Súlyok és biasok	Összesített
baseline	conv1 80, conv2 1168, fej 6915	8163
pruned	conv1 60, conv2 660, fej 5187	5907

A pruned változat paraméterszáma 27,6%-kal kisebb. A memóriaellenőrzéshez a paraméterbájtok összege `sum(p.numel() * p.element_size() for p in model.parameters())`; a baseline esetén $8163 \cdot 4 = 32652$ bájt, a pruned esetén $5907 \cdot 4 = 23628$ bájt. Az aktivációs memóriát a `torch.profiler` `profile_memory=True` kapcsolójával lehet mérni; a pontos mezőneveket a telepített PyTorch 2.14 dokumentációjában kell ellenőrizni ([D17]).

A futtatási parancs:

```
python n135_prune_lab.py --images data/images.npy --labels data/labels.npy \
  --metadata data/metadata.csv --epochs 10 --drop1 2 --drop2 4 --seed 20261020
```

A protokoll: tanuljuk a baseline-t train-en, maximálisan 20 epochig, Adam tanulóval, 0,001 tanulási rátával, 32-es batchcsel; a legjobb állapotot a val veszteség választja ki. A pruned modellt a kiválasztott baseline checkpointjából másoljuk: a conv rétegek és a megmaradt csatornák súlyai

változatlanul átkerülnek, a fej viszont újrainicializálódik (a bemeneti csatornaszám változása miatt), majd 10 epochig finomhangoljuk. A zárótesztet egyszer, a protokoll rögzítése után értékeljük. A könyvben nem szereplő pontosságértékeket a laborjelentésben `MÉRENDŐ` mező jelöli; a paraméter- és alakkontroll tételszerű, a pontossághatás viszont empirikus és seedfüggő.

34.3.7 Gyakori tévedés: nullázott dense súly és futtatott művelet

A strukturálatlan ritkítás gyakori félreértése, hogy a nullázás már gyorsítja a hálózatot. Tekintsük a `torch.nn.Linear` réteget, amely az $N \times d$ bemenetből $Z = XW^\top + b$ kimenetet számít. Ha a W mátrixban nullákat helyezünk el, a dense mátrixszorzó kernel továbbra is végigjárja a teljes $m \times d$ súlyterületet; nem létezik a futásidőben automatikusan kiértékelt maszkfeltétel a kernel belsejében. A nullázott dense súly tehát nem csökkenti a futtatott dense műveletek számát, csak a tárolt értéket változtatja.

Ellenpélda: legyen $W = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}$, azaz a súly 25 százaléka nulla. Egy $\mathbb{R}^2$-beli bemenetre a kimenet $y_1 = 1x_1 + 0x_2 + b_1$, $y_2 = 2x_1 + 3x_2 + b_2$. A futtatott GEMM akkor is négy szorzást és a hozzájuk tartozó összeadásokat végzi, ha a W mátrix (1; 2) eleme nulla; a kernel nem tudja kihagyni a pozíciót a dense reprezentációból. Ezzel szemben a strukturált törlés, például a fenti csatorna-eltávolítás, megváltoztatja a súlytenzor alakját, ezért az általános dense Conv2d kernel is kisebb dimenzióval és kevesebb tényleges szorzással fut. A maszkolt Python-reteg ráadásul a forwardban még egy elemenkénti szorzást is hozzáadhat, tehát rosszul megválasztott ritkítási módszerrel a pruned modell akár lassabb is lehet.

34.3.8 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a strukturálatlan és a strukturált pruning különbségét. Adjon egy 4×3 méretű súlymátrixon példát mindkettőre, és mondja meg, miért nem ad az unstructured nullázás automatikus gyorsulást egy sűrű GEMM-ben. Beadandó: két mátrix, maszkok, shape-változás, egy rövid indoklás.

F2 (levezetési, nehézség 2). Igazolja a paraméterszám-lemmát bias nélküli rétegekre, majd alkalmazza $C_0 = 3$, $C_1 = 4$, $C_2 = 5$, $k_h = k_w = 3$, $k'_h = k'_w = 1$ értékekkel: adja meg a két réteg súlyalakját, paraméterszámát törlés előtt és után. Beadandó: teljes levezetés és P, P', Δ.

F3 (implementációs, nehézség 2). Írjon PyTorch-függvényt, amely két `Conv2d` rétegből ténylegesen eltávolítja a j-edik közbülső csatornát, majd a 34.3.4. szakasz numerikus példáján ellenőrizze `torch.testing.assert_close` segítségével. Beadandó: futtatható kód, shape-kontroll, egyezési feltétel.

F4 (hibakeresési, nehézség 2). Adott a következő kód:

```
l1.weight.data[1] = 0
l1.bias.data[1] = 0
# a l2-t nem módosítjuk
```

Magyarázza el, miért hamis channel pruningnak nevezni ezt, és mi a kimenetre és a számítási költségre gyakorolt hatása. Adja meg a javítást. Beadandó: hiba ok, következmény, javított kódrészlet, ellenőrzési sor.

F5 (kísérlettervezési/transzfer, nehézség 3). Tervezzen GlyphLab-kísérletet a baseline, a 70 százalékos unstructured pruning és a 25 százalékos structured channel pruning összehasonlítására CPU-n. A cél annak eldöntése, hogy az azonos val-pontosságú tűréshatár mellett melyik változat

ad rövidebb CPU forward időt. Írja le a splitet, a baseline-t, a metrikákat, a seed- és fine-tune-protokollt, valamint a következtetés korlátait. Beadandó: protokollterv és alternatív értelmezés.

34.3.9 Megoldások és indoklás

F1 megoldás. Az unstructured pruning az elemenkénti $M \odot W$ maszkkal nulláz, a mátrix alakja 4×3 marad. A példában a maszk a második oszlop nemnulla elemeit (5 és 7) is kinullázza, így a nullák aránya $8/12 \approx 67\%$:

$$W = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 0 & 3 \\ 4 & 5 & 0 \\ 0 & 7 & 0 \end{bmatrix}, \qquad M = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

A structured pruning ezután teljes oszlopot töröl: a második oszlop a maszkolás után csupa nulla, ezért elhagyása a maszkolt modell függvényét megőrzi, az eredmény 4×2 alakú, és a sűrű mátrixszorzás tényleges mérete is csökken. A nyers W-ből kiindulva a második oszlop törlése nem lenne függvénymegőrző, mert abban 5 és 7 áll; ezért a helyes sorrend a maszkolás (vagy nullázás), majd a strukturált törlés. A sűrű GEMM az unstructured nullás mátrixot továbbra is teljes 4×3-ként olvassa be, nullákkal együtt, ezért azok a pozíciók részt vesznek a ciklusban.

F2 megoldás. Bias nélkül $P = C_1 C_0 k_h k_w + C_2 C_1 k'_h k'_w$. A törlés után $P' = (C_1 - 1) C_0 k_h k_w + C_2 (C_1 - 1) k'_h k'_w$, tehát

$$\Delta = C_0 k_h k_w + C_2 k'_h k'_w.$$

A konkrét értékekkel $P = 4 \cdot 3 \cdot 9 + 5 \cdot 4 \cdot 1 = 108 + 20 = 128$, $P' = 3 \cdot 27 + 5 \cdot 3 = 81 + 15 = 96$, $\Delta = 32$. A súlyalakok: L_1: $4 \times 3 \times 3 \times 3$, törlés után $3 \times 3 \times 3 \times 3$; L_2: $5 \times 4 \times 1 \times 1$, törlés után $5 \times 3 \times 1 \times 1$. Ez a feladat megerősíti, hogy a $+1$-es tagok csak biasos esetben lépnek be.

F3 megoldás. A 34.3.5. szakasz `remove_channel` függvénye teljes: a `keep` indexekkel átmásolja L_1 kimeneti és L_2 bemeneti szeleteit. A szükséges kontrollok:

```
assert l1p.weight.shape == (2, 2, 1, 1)
assert l2p.weight.shape == (2, 2, 1, 1)
assert sum(p.numel() for p in l1p.parameters()) == 6
assert sum(p.numel() for p in l2p.parameters()) == 6
torch.testing.assert_close(zp, z, rtol=1e-7, atol=1e-7)
```

Az első feltétel az alak-ellenőrzés: a törlés után mindkét $1{\times}1$ réteg 2×2 súlyú. A numerikus egyezés itt a lemma feltételét ellenőrzi, nem pedig általánosan bizonyítja bármely réteg törlését.

F4 megoldás. A hiba az, hogy a kód a csatornát nullázza, nem törli. A j csatorna kimenete azonosan nulla lesz, ezért az L_2 kimenete numerikusan megegyezhet a törölt változattal, de a L_2 súlyainak megfelelő bemeneti oszlopa továbbra is a memóriában és a kernelben él; a paraméterszám nem csökken, és a futott szorzások száma sem. A javítás a 34.3.5. szakasz függvénye: új `Conv2d` példányok létrehozása és a megtartott indexek átmásolása.

```
l1p, l2p = remove_channel(l1, l2, 1)      # a csatorna tényleges törlése
assert l1p.weight.shape[0] == l1.weight.shape[0] - 1
assert l2p.weight.shape[1] == l2.weight.shape[1] - 1
assert sum(p.numel() for p in l1p.parameters()) == 6
torch.testing.assert_close(l2p(l1p(x)), l2(l1(x)), rtol=1e-7, atol=1e-7)
```

Az utolsó sor a numerikus egyezést ellenőrzi; ez csak azért teljesül, mert a fogyasztó súlya a törölt csatornán nulla, amit a `remove_channel` assertje külön is kikényszerít. A paraméterszám-ellenőrzés a shape-ellenőrzéssel együtt mutatja, hogy valóban kisebb tenzor keletkezett.

F5 megoldásminta. A független mintavételi egység a `scene_id`; a train/val/test split rögzített. A baseline a `TinyGlyphCNN`, a változók: no prune, unstructured 70%, structured 25%. A structured modellt a baseline legjobb val-veszteségű checkpointjából vágjuk; az unstructured változatot ugyanabból maszkoljuk. A tanulási lépések három seeddel ismétlődnek (20261020, 20261021, 20261022), mert egyetlen seed nem ad reprodukálhatósági garanciát a seedek közötti szórásra. A döntési szabály: a pruned változat akkor tekinthető nem rosszabbnak, ha a val-pontossága legfeljebb 1 százalékponttal marad el a baseline-étól; e feltétel mellett hasonlítjuk a CPU forward időt. A metrika a val accuracy a fine-tune után, a végső teszten egyszer, a legjobb val-veszteségű állapottal mérve; a futásidőt `torch.utils.benchmark.Timer` [D16] méri melegítéssel és ismétlésekkel, a memóriát a profiler. Fontos kimondani: az unstructured, maszkolt változat futásideje nem javul, a maszkolás miatt akár nőhet is, ezért ez csak kontroll; a gyorsulást a structured változattól várjuk. A következtetés csak a helyi CPU-környezetre és a GlyphLab-adatra érvényes; GPU-ra vagy természetes képekre nem általánosítható. A vártnál rosszabb pruned pontosság sem a structured pruning érvénytelenítése, hanem a fine-tune és a törlési kritérium hibájának jele lehet. A fine-tune a baseline legjobb checkpointjából indul, AdamW-val, $\eta = 10^{-3}$ tanulási rátával, batch 16 mellett, legfeljebb 10 epochon át, a három seed mindegyikén; az erőforrás CPU, három fine-tune és három mérési sorozat.

34.3.10 Önellenőrzés és visszatekintés

Ellenőrző lista: a maszkolt súly alakja mindig azonos az eredetiével, a törölt súly alakja kisebb; a bias hossza csak a kimeneti tengely törlésekor csökken; a $C_1 = 1$ esetben a törlés üres csatornateret hagyna, ezért a lemma feltétele $C_1 \geq 2$; a maradékcsatlakozásos blokkokban a csatorna törlése az összeadás mindkét ágát módosítja, ezért a 18.1. Reziduális blokk és identitásút blokkja külön feltételt igényel. A 34.2. Kvantálás és hibatovábbterjedés kvantálási tanulságai: ott az alak nem változik, de a számábrázolási hiba megjelenik; itt az alak változik, de a numerikus ekvivalencia csak nulla fogyasztósúly esetén garantált. A 33.3. torch.export és ONNX export-ellenőrzésből örökölt elv: az új modell viselkedését véges kontrollhalmazon is újra meg kell mérni.

34.3.11 Rövid összefoglaló

A pruning akkor csökkenti a számítást, ha ténylegesen törli a tensordimenziót vagy ha a kernel kifejezetten támogatja a ritka mintázatot. A strukturálatlan nullázás ritkaságot adhat a tárolásban, de nem csökkenti automatikusan a futott dense műveleteket. A strukturált csatorna-eltávolítás egy `Conv2d` kimeneti csatornáját és a következő réteg bemeneti csatornáját együtt törli; két rétegnél a paraméterszám csökkenése $\Delta = C_0 k_h k_w + 1 + C_2 k'_h k'_w$. Nulla fogyasztósúly esetén a törlés a függvényt pontosan megőrzi, általában azonban fine-tune-ra van szükség. A laborban a kiválasztást train-statisztika, a végső döntést val-mérés, a zárt tesztet egyszeri mérés igazolja.

34.3.12 Továbblépés és ténylegesen használt források

A 34.4. Teacher–student desztilláció egység a tudásdesztillációt tárgyalja: ott a már pruned tanulóháló egy nagyobb teacher modell lágy céljait közelíti, ami a pruninghoz képest független sűrítési út ([P26] a desztilláció eredeti cikke). A laborban használt profiler- és benchmark-API pontos mezőneveit a telepített PyTorch 2.14 dokumentációjában kell ellenőrizni.

Felhasznált források:

- [D16] PyTorch contributors (2026). *torch.utils.benchmark*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/benchmark_utils.html. Hozzáférés: 2026-10-30.
- [D17] PyTorch contributors (2026). *torch.profiler*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/profiler.html. Hozzáférés: 2026-10-30.
- [P26] Hinton, G., Vinyals, O., Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531. https://arxiv.org/abs/1503.02531.
- [P28] Han, S., Pool, J., Tran, J., Dally, W. (2015). *Learning both Weights and Connections for Efficient Neural Networks*. NeurIPS 2015. https://arxiv.org/abs/1506.02626.
- [P29] Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H. P. (2017). *Pruning Filters for Efficient ConvNets*. ICLR 2017. https://arxiv.org/abs/1608.08710.

34.4 Teacher–student desztilláció

34.4.1 Cél, előismeret és induló kérdés

Ebben az alfejezetben egy már betanított, nagyobb „tanár” (teacher) háló viselkedését préseljük át egy kisebb „diák” (student) hálóba. A desztilláció lényege, hogy a diák nemcsak a merev osztálycímkét, hanem a tanár lágyított osztályeloszlását is megpróbálja utánozni. Előismeret: többosztályos cross-entropy és logit fogalma 7.2. Softmax és többosztályos cross-entropy, valamint a tanítási/kiértékelési mód megkülönböztetése 12.1. Tanítás, validáció és kiértékelési mód, 18.2. Batch normalization train és eval módban. Induló kérdés: ha egy 3 osztályos feladatban a címke `0`, akkor a `(1,0,0)` cél miért hordoz kevesebb mintázatot a tanuló számára, mint egy `(0.78, 0.15, 0.07)` eloszlás? A válasz: a one-hot cél a nem helyes osztályokra nulla célvalószínűséget ír elő, a lágy cél viszont a tanár osztályok közötti hasonlósági ítéletét is hordozza, ezért a diák a tévesztések szerkezetét is tanulja.

34.4.2 Definíciók és tenzoralakok

Legyen a batch méret N, az osztályok száma K. A tanár logit kimenete

$$Z_t \in \mathbb{R}^{N\times K},$$

a diák logit kimenete

$$Z_s \in \mathbb{R}^{N\times K}.$$

A valódi osztálycímke $y \in \{0;\ \ldots;\ K-1\}^N$. A hőmérséklet (temperature) egy pozitív valós skalár, $T > 0$. A **lágyított tanári cél** (soft target) a soronkénti hőmérsékletes softmax:

$$q_{n,i} \;=\; \frac{\exp\bigl(Z_{t,n,i}/T\bigr)}{\sum_{j=1}^{K}\exp\bigl(Z_{t,n,j}/T\bigr)}, \qquad q \in \mathbb{R}^{N\times K}, \quad \sum_i q_{n,i} = 1.$$

A diák megfelelő eloszlása p ugyanezzel a T-vel:

$$p_{n,i} \;=\; \frac{\exp\bigl(Z_{s,n,i}/T\bigr)}{\sum_{j=1}^{K}\exp\bigl(Z_{s,n,j}/T\bigr)}.$$

A **desztillációs loss** a következő konvenciót követi:

$$\mathcal{L}(Z_s; Z_t, y, T, \alpha) \;=\; \alpha\, T^2\, D_{KL}(q \,\|\, p) \;+\; (1-\alpha)\, \mathrm{CE}(Z_s, y), \tag{1}$$

ahol $\alpha \in [0;\ 1]$, és a két tag definíciója

$$D_{KL}(q \,\|\, p) = \frac{1}{N} \sum_{n=1}^{N} \sum_{i=1}^{K} q_{n,i} \log \frac{q_{n,i}}{p_{n,i}}, \qquad \mathrm{CE}(Z_s, y) = -\frac{1}{N} \sum_{n=1}^{N} \log \frac{\exp(Z_{s,n,y_n})}{\sum_j \exp(Z_{s,n,j})}.$$

Mindkét tag batch-átlag, tehát a loss egyetlen skalár. A CE tag a PyTorch `CrossEntropyLoss` `reduction='mean'` konvenciójának felel meg logit bemenettel és `int64` célindexszel. A KL tagban q rögzített cél, p a diák függvénye. Gyakori ekvivalens alternatíva a cross-entropy $H(q,p)$ használata a KL helyett; a kettő gradiense a diák paramétereiben azonos, mert $H(q,p) = D_{KL}(q\|p) + H(q)$, és $H(q)$ a diák paramétereitől független konstans. A loss **értéke** viszont eltér, ezért a naplózáskor meg kell nevezni, melyiket számoljuk.

A CE tagot **hard-label tagnak**, a KL tagot **soft-target tagnak** nevezzük. A α a soft tag súlya; $\alpha = 1$ tisztán soft desztilláció, $\alpha = 0$ a szokásos felügyelt tanítás. A $T = 1$ esetén a soft tag a szokásos, nem lágyított softmaxeloszlások KL-divergenciája.

Teacher eval. A tanár súlyai a desztilláció során nem frissülnek. A soft célok előállításakor a tanárt `eval()` módba kell tenni, és `torch.no_grad()` alatt kell futtatni. Ez azért lényeges, mert dropout és BatchNorm esetén a train mód más kimenetet ad, mint az eval mód 18.2. Batch normalization train és eval módban; a soft céloknak azt az eloszlást kell reprezentálniuk, amelyet a tanár a kiértékeléskor produkál. A `torch.no_grad()` önmagában nem állítja meg a BatchNorm `running_mean` és `running_var` puffereinek frissítését, ezért a tanárt mindig `eval()` módban kell futtatni.

34.4.3 Levezetés: a hőmérsékletes cél alakja és a T^2 skálázás indoka

Először a gradiensről mondunk ki egy lemmát.

Lemma. Legyen $q = \mathrm{softmax}(a)$ rögzített, $p = \mathrm{softmax}(b)$, ahol $a, b \in \mathbb{R}^K$. Ekkor

$$\nabla_b D_{KL}(q \,\|\, p) \;=\; p - q.$$

Bizonyítás. A KL definíciójából

$$D_{KL}(q \,\|\, p) = \sum_i q_i \log q_i - \sum_i q_i \log p_i.$$

Az első tag b-től független, ezért eltűnik a gradiensképtzésnél. Marad a $-\sum_i q_i \log p_i$ tag. A softmax logaritmusának parciális deriváltja

$$\frac{\partial \log p_i}{\partial b_j} = \delta_{ij} - p_j,$$

ahol δ_{ij} a Kronecker-delta. Ezt behelyettesítve

$$\frac{\partial}{\partial b_j}\left(-\sum_i q_i \log p_i\right) = -\sum_i q_i(\delta_{ij} - p_j) = -q_j + p_j \sum_i q_i = p_j - q_j,$$

mert $\sum_i q_i = 1$. Ezzel a lemma bizonyítását befejeztük. $\square$

Most helyettesítsük be $a = Z_t/T$ és $b = Z_s/T$ értékeket. Ekkor q a tanári, p a diákhőmérsékletes eloszlás. A láncszabály szerint, mivel $\partial b_i/\partial Z_{s,i} = 1/T$ és a keresztderiváltak nullák,

$$\nabla_{Z_s} D_{KL}(q \,\|\, p) = \frac{1}{T}(p - q). \tag{2}$$

A (2) képlet a pontos, véges T-re érvényes gradiens egyetlen mintára (egyetlen K-dimenziós logitvektorra). Az alábbi levezetésben a Z_s, Z_t jelölések is ezt az egy mintát jelentik; az (1) batch-átlagos KL-tag gradiense a teljes $N \times K$ logitmátrix szerint ezért $(p-q)/(NT)$, a T^2-es szorzóval együtt $T(p-q)/N$. A T^2-es szorzó nélkül a (2) képlet explicit tényezője $1/T$, és mivel $p-q$ maga is $O(1/T)$, a gradiens aszimptotikusan $1/T^2$-tel skálázódik; a két állítás nem ellentétes. Miért éppen T^2 a szokásos kompenzáció? Ennek indoklása a nagy T határesetből látható. Fejtsük sorba a hőmérsékleti softmaxot $1/T$ szerint. Legyen $\bar{z} = \frac{1}{K}\sum_j z_j$ egy logitvektor átlaga. Ekkor

$$\frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} = \frac{1}{K} + \frac{z_i - \bar{z}}{KT} + r_i(T),$$

ahol a maradéktag $|r_i(T)| \le C/T^2$ valamilyen C konstanssal, feltéve, hogy a logitok komponensei korlátosak. Így $T \to \infty$ esetén

$$p - q = \frac{(Z_s - \bar{Z}_s) - (Z_t - \bar{Z}_t)}{KT} + O(T^{-2}),$$

ahol a kivonás és az átlag komponensenként értendő. Ezt (2)-be helyettesítve a **skálázatlan** KL gradiens aszimptotikus alakja

$$\nabla_{Z_s} D_{KL}(q \,\|\, p) = \frac{(Z_s - \bar{Z}_s) - (Z_t - \bar{Z}_t)}{KT^2} + O(T^{-3}).$$

A logit-térbeli gradiens nagyságrendje tehát $1/T^2$-tel csökken. Ha a soft tagot T^2-tel szorozzuk, akkor a gradiens

$$T^2\, \nabla_{Z_s} D_{KL}(q \,\|\, p) = T(p - q) \;\to\; \frac{(Z_s - \bar{Z}_s) - (Z_t - \bar{Z}_t)}{K} + O(T^{-1}),$$

ami $T \to \infty$ esetén már T-független határértékhez tart. A T^2-es szorzó tehát azt biztosítja, hogy a hőmérséklet növelése ne hígítsa fel a soft tag tanító jelét. Pontosabban: az (1)-beli T^2 a **loss értékét** skálázza; a láncszabály miatt a gradiensre gyakorolt hatása ugyanez a T^2 szorzó. A fenti határérték-állítás aszimptotikus heurisztika, nem minden T-re pontos egyenlőség; a véges T-re érvényes igazság a (2) képlet. Ez a levezetés a saját konvenciónkra vonatkozik; az eredeti desztillációs javaslatban is hasonló skálázás szerepel [P26].

34.4.4 Eredeti kézi kis példa

Legyen $N = 1$, $K = 3$, $T = 2$, $\alpha = 0{,}7$. A tanár logitja

$$Z_t = (2{,}5;\ 0{,}0;\ -0{,}5),$$

a diák kezdeti logitja

$$Z_s = (1{,}2;\ 0{,}3;\ -0{,}9),$$

és a valódi címke $y = 0$. Először a hőmérsékleti softmaxok. A tanári cél:

$$\begin{aligned} e^{2{,}5/2} &= e^{1{,}25} \approx 3{,}4903, \\ e^{0/2} &= 1, \\ e^{-0{,}5/2} &= e^{-0{,}25} \approx 0{,}7788, \end{aligned}$$

az összeg 5,2691, így

$$q = (0{,}6624;\ 0{,}1898;\ 0{,}1478).$$

A diák hőmérsékleti eloszlása hasonlóan:

$$\begin{aligned} e^{1{,}2/2} &= e^{0{,}6} \approx 1{,}8221, \\ e^{0{,}3/2} &= e^{0{,}15} \approx 1{,}1618, \\ e^{-0{,}9/2} &= e^{-0{,}45} \approx 0{,}6376, \end{aligned}$$

az összeg 3,6215, tehát

$$p = (0{,}5031;\ 0{,}3208;\ 0{,}1761).$$

A KL tag:

$$D_{KL}(q\|p) \approx 0{,}6624 \ln \frac{0{,}6624}{0{,}5031} + 0{,}1898 \ln \frac{0{,}1898}{0{,}3208} + 0{,}1478 \ln \frac{0{,}1478}{0{,}1761} \approx 0{,}0567.$$

A T^2-es szorzóval a soft tag $4 \cdot 0{,}0567 = 0{,}2268$. A hard-label tag a $T = 1$ softmaxból: $\mathrm{softmax}(Z_s) = (0{,}6540;\ 0{,}2659;\ 0{,}0801)$, így

$$\mathrm{CE}(Z_s, y) = -\ln(0{,}6540) \approx 0{,}4246.$$

A teljes loss:

$$\mathcal{L} = 0{,}7 \cdot 0{,}2268 + 0{,}3 \cdot 0{,}4246 \approx 0{,}2862.$$

A gradiens ellenőrzéséhez használjuk a (2) képletet és a lemma bizonyítását. A soft tag T^2-es gradiensére

$$T(p-q) = 2 \cdot (-0{,}1593;\ 0{,}1310;\ 0{,}0283) = (-0{,}3186;\ 0{,}2620;\ 0{,}0566).$$

A hard tag gradiense a szokásos softmax-cél különbség:

$$\nabla_{Z_s} \mathrm{CE} = (0{,}6540;\ 0{,}2659;\ 0{,}0801) - (1;\ 0;\ 0) = (-0{,}3460;\ 0{,}2659;\ 0{,}0801).$$

A teljes gradiens α-súlyozva:

$$\nabla_{Z_s} \mathcal{L} \approx 0{,}7(-0{,}3186;\ 0{,}2620;\ 0{,}0566) + 0{,}3(-0{,}3460;\ 0{,}2659;\ 0{,}0801) = (-0{,}3268;\ 0{,}2632;\ 0{,}0636).$$

Az alábbi kód ugyanezt a veszteséget és gradienst számítja ki numerikusan; a rögzített ellenőrző környezetben (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020) futtatva az alábbi mért kimenetet adja.

```
# lab/n136_distill/soft_target_example.py
import torch
import torch.nn.functional as F

def distillation_loss(z_s, z_t, y, T, alpha):
    """
    z_s, z_t: (N, K) logit tensorok
    y:        (N,) int64 osztalyindex
    T:        pozitiv float
    alpha:    float [0,1], a soft tag sulya
    """
    q = torch.softmax(z_t / T, dim=1)
    log_p = torch.log_softmax(z_s / T, dim=1)
    kd = F.kl_div(log_p, q, reduction='batchmean')
    ce = F.cross_entropy(z_s, y, reduction='mean')
    return alpha * T * T * kd + (1.0 - alpha) * ce

torch.manual_seed(20261020)
z_t = torch.tensor([[2.5, 0.0, -0.5]], dtype=torch.float64)
z_s = torch.tensor([[1.2, 0.3, -0.9]], dtype=torch.float64, requires_grad=True)
y = torch.tensor([0], dtype=torch.int64)

loss = distillation_loss(z_s, z_t, y, T=2.0, alpha=0.7)
loss.backward()
print(f"loss={loss.item():.4f}")
print(f"grad={z_s.grad.detach().numpy()}")
assert abs(loss.item() - 0.2862) < 1e-3, loss.item()
assert torch.allclose(z_s.grad, torch.tensor([[-0.3268, 0.2632, 0.0636]],
                                             dtype=torch.float64), atol=1e-3)
```

Mért kimenet:

```
loss=0.2862
grad=[[-0.32679403  0.26320448  0.06358955]]
```

A mért értékek a kézi számítás `loss=0.2862` és `grad=[[-0.3268, 0.2632, 0.0636]]` értékét adják; a kód `assert`-jei 10^{-3} abszolút toleranciával ellenőriznek, mert a kézi számítás négy tizedesjegyre kerekített.

34.4.5 Konkrét labor: azonos tesztprotokollú desztilláció a GlyphLabon

A következő mérési terv a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adatszerződést használja. A tanár és a diák is ugyanazt a train/val/test felosztást kapja, és a modellválasztás kizárólag a **clean** validációs spliten mért hard-label CE minimuma alapján történik, ahogyan azt 22.4. Augmentációs kísérlet és domainváltozás előírja. Ez a kísérlet a könyvben nem futott le, ezért alább csak a protokollt és a desztilláció-specifikus kódvázlatot adjuk meg; a tanítási hurok hiányzó részeit a laborban kell kitölteni. A fenti kézi példa ettől külön, futtatott ellenőrzés.

Protokoll.

1. **Teacher tanítás.** Egy kis CNN (például 2 konvolúciós réteg 8 és 16 csatornával, ReLU, maxpool, majd 64 egységű MLP-fejjel) betanítása a `../data/images.npy` train részén, a `../data/metadata.csv` split oszlopa szerinti 240 képen, seed 20261020, Adam, tanulási ráta 10^{-3}, batch 32, legfeljebb 20 epoch. A legjobb checkpointot a val loss választja ki.
2. **Soft cél cache.** A legjobb teachert `eval()` módban, `torch.no_grad()` alatt futtatjuk a train és val képekre, augmentáció nélkül, a nyers képeken. A kapott $N \times 3$ tanári logit tenzort `float32` tömbben a futás memóriájában tartjuk; a cache tehát a logitokat tárolja, nem az előre kiszámított q-t, a hőmérsékleti softmaxot a loss számolja. A tanár a diák tanítása alatt nem fut újra.
3. **Student tanítás.** Egy kisebb CNN (például 1 konvolúció 8 csatornával, 32 egységű MLP-fejjel) tanítása ugyanazon a train spliten az (1) loss-szal, $T = 4$ és $\alpha = 0{,}7$ mellett. A val spliten a modellválasztáshoz a hard-label CE-t számoljuk, hogy a baseline-nal összehasonlítható legyen.
4. **Baseline.** Ugyanez a student architektúra, ugyanaz a seed, epoch-szám, batch és optimizer, de $\alpha = 0$, azaz tiszta hard-label tanítás.
5. **Végső teszt.** A legjobb val checkpointot egyszer futtatjuk a clean tesztre, majd a `corrupted_test.npy` halmazra. A két halmaz ugyanazon scene/view párokból áll, ezért a clean–corrupt pontosságkülönbség páros, jelenetszintű összehasonlítás 22.4. Augmentációs kísérlet és domainváltozás. A teachert is ugyanígy kiértékeljük összehasonlításként.
6. **Erőforrás-mérés.** A futásidő mérésére a `torch.utils.benchmark` [D16] és a `torch.profiler` [D17] használható; a mérési környezetet, az eszközt és az egyes profilozási beállításokat minden futtatáskor a laborjelentésben kell rögzíteni.

A desztilláció-specifikus kódvázlat:

```
# lab/n136_distill/train_distill.py
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(20261020)
np.random.seed(20261020)

images = np.load("../data/images.npy")       # (360,1,24,24) float32
labels = np.load("../data/labels.npy")       # (360,) int64

# ... train/val/test indexek a metadata.csv split oszlopa alapjan ...

def make_soft_cache(teacher, split_images):
    teacher.eval()
    with torch.no_grad():
        # split_images: (M,1,24,24) float32 CPU tensor
        logits = teacher(split_images)       # (M,3)
    return logits.detach().cpu().float()     # soft target cache
```

```python
def distillation_loss(z_s, z_t, y, T, alpha):
    q = torch.softmax(z_t / T, dim=1)
    log_p = torch.log_softmax(z_s / T, dim=1)
    kd = F.kl_div(log_p, q, reduction='batchmean')
    ce = F.cross_entropy(z_s, y, reduction='mean')
    return alpha * T * T * kd + (1.0 - alpha) * ce

# ... a tanítási hurok a 11.4. alfejezet (Egy teljes PyTorch tanítási iteráció)
#     szerkezetét követi; a teacher
#     forwardja helyett a cache-ból töltjük a z_t-t ...
```

Futtatási parancs a munkakönyvtárból (a fenti fájl vázlat, ezért a parancs csak a laborban kitöltött, teljes változatra vonatkozik):

```
cd lab/n136_distill
python train_distill.py --T 4 --alpha 0.7 --seed 20261020
```

A beadandó eredmény: tanári és diák val pontosság, clean test pontosság, corrupt test pontosság, valamint a corrupt–clean páros különbség átlaga és szórása. Értelmezési korlát: a GlyphLab 24×24 pixeles procedurális adat, ezért az eredmény nem általánosít természetes képi feladatokra.

34.4.6 Feladatok

34.4.6.1 F1 fogalmi (nehézség 1)

Magyarázza el, miért nem azonos a desztilláció a label smoothinggal. Adjon egy $K = 3$ osztályú, $N = 1$ példát, amelyben a tanári soft cél $q = (0{,}02;\ 0{,}95;\ 0{,}03)$, a valódi címke $y = 1$, és hasonlítsa össze a $\varepsilon = 0{,}1$ teljes háttértömegű, egyenletes label smoothing céllal, ahol a háttérosztályok egyenként $\varepsilon/(K-1) = 0{,}05$ tömeget kapnak.

34.4.6.2 F2 levezetési (nehézség 2)

Bizonyítsa be a fenti lemma felhasználásával, hogy az (1) loss-ban a soft tag gradiensére

$$\nabla_{z_s}\left(\alpha T^2 D_{KL}(q\|p)\right) = \alpha T(p-q)$$

teljesül. Ellenőrizze a képletet a kézi példa $T = 2$, $\alpha = 0{,}7$ adataira, és adja meg a kapott háromdimenziós gradiensvektort.

34.4.6.3 F3 implementációs (nehézség 2)

Implementálja a `distillation_loss` függvényt úgy, hogy ne használja a `torch.nn.KLDivLoss` osztályt, hanem a KL tagot a $q\log(q/p)$ definícióból számolja. A függvény bemenete `z_s`, `z_t`, `y`, `T`, `alpha`; a kimenet skalár. Ellenőrizze a kézi példán: `z_s`, `z_t`, `y`, `T=2.0`, `alpha=0.7` mellett a loss $0{,}2862 \pm 10^{-3}$.

34.4.6.4 F4 hibakeresési (nehézség 3)

Az alábbi részlet hibásan állítja elő a soft célokat:

```
teacher.train()
with torch.no_grad():
    z_t = teacher(x_batch)          # (N,3)
q = torch.softmax(z_t / T, dim=1)   # soft target
# ... studenttanítási lépés ...
# a teacher súlyai nem frissülnek, de a BatchNorm statisztikája train módban
```

Nevezze meg a hiba okát, írja le a következményét a desztillált diák tanítására, javítsa a kódot, és adja meg, hogyan ellenőrizné a javítás helyességét.

34.4.6.5 F5 kísérlettervezési/transzfer (nehézség 3)

Tervezzen kísérletet, amely a GlyphLabon a hard-label baseline-t egy $T = 1$, $T = 4$ és $T = 8$ desztillált studenttel hasonlítja össze, mindegyiket 3 különböző seeddel. Adja meg a megfigyelési egységet, a splitet, a modellválasztási szabályt, a metrikát, a számítási és címkeköltség-keretet, valamint a következtetés érvényességi korlátját.

34.4.7 Megoldások és indoklás

34.4.7.1 F1 megoldása

A label smoothing a valódi one-hot cél e_y helyett a

$$(1-\varepsilon)e_y + \frac{\varepsilon}{K-1}(1-e_y)$$

eloszlást használja; ez a háttérosztályokat szimmetrikusan, egyenletesen tölti fel, és nem függ egyetlen tanár predikciójától sem. A desztilláció ezzel szemben egy konkrét tanár által számított q eloszlást használ célként, amely tükrözheti a tanár osztályok közötti hasonlósági ítéletét. A megadott $q = (0{,}02;\ 0{,}95;\ 0{,}03)$ és $\varepsilon = 0{,}1$ esetén a smoothing cél

$$(1-0{,}1)(0;\ 1;\ 0) + 0{,}1 \cdot (0{,}5;\ 0;\ 0{,}5) = (0{,}05;\ 0{,}9;\ 0{,}05),$$

amely különbözik a tanári q-tól: a tanár a 2. osztályt kissé valószínűbbnek tartja a 0. osztálynál, míg a smoothing ezt a kettőt egyenlővé teszi.

34.4.7.2 F2 megoldása

A lemma szerint a KL gradiens a diák hőmérsékleti logitjaira $\nabla_b D_{KL} = p - q$, ahol $b = Z_s/T$. A láncszabállyal

$$\nabla_{Z_s} D_{KL} = \frac{1}{T}(p-q).$$

A loss αT^2-szeres szorzót tartalmaz, ezért

$$\nabla_{Z_s}\left(\alpha T^2 D_{KL}\right) = \alpha T(p-q).$$

A kézi példában $p - q = (-0{,}1593;\ 0{,}1310;\ 0{,}0283)$, így

$$\alpha T(p - q) = 0{,}7 \cdot 2 \cdot (-0{,}1593;\ 0{,}1310;\ 0{,}0283) = (-0{,}2230;\ 0{,}1834;\ 0{,}0396).$$

34.4.7.3 F3 megoldása

```python
# lab/n136_distill/manual_kd.py
import torch
import torch.nn.functional as F

def distillation_loss_manual(z_s, z_t, y, T, alpha):
    log_q = torch.log_softmax(z_t / T, dim=1)
    log_p = torch.log_softmax(z_s / T, dim=1)
    q = log_q.exp()
    # stabil és pontos: q*(log q - log p), a log_softmax már normalizált
    kd = torch.sum(q * (log_q - log_p), dim=1).mean()
    ce = F.cross_entropy(z_s, y, reduction='mean')
    return alpha * T * T * kd + (1.0 - alpha) * ce

torch.manual_seed(20261020)
z_t = torch.tensor([[2.5, 0.0, -0.5]], dtype=torch.float64)
z_s = torch.tensor([[1.2, 0.3, -0.9]], dtype=torch.float64, requires_grad=True)
y = torch.tensor([0], dtype=torch.int64)
loss = distillation_loss_manual(z_s, z_t, y, T=2.0, alpha=0.7)
assert abs(loss.item() - 0.2862) < 1e-3, loss.item()
print(loss.item())
```

A rögzített ellenőrző környezetben futtatva (Python 3.12.3, torch 2.15.0+cpu, CPU, 10 szál, seed 20261020) a mért kimenet:

```
0.2861596729730882
```

Az ellenőrzés: a kézi számítás `0.2862`; a kód ezt 10^{-3} abszolút hibán belül adja, és az `assert` ezt ki is kényszeríti. A `log_softmax` normalizált logaritmusokat ad, ezért a `q*(log_q - log_p)` alak nem igényel epsilon-hozzáadást, és nem vezet be mesterséges eltolást; `float64` bemenetnél a négy tizedesjegyre kerekített kézi értékhez képest a hiba $O(10^{-4})$.

34.4.7.4 F4 megoldása

Hiba oka: a tanárt `train()` módban hagytuk, ezért a BatchNorm rétegek a minibatch-átlagot és varianciát használják, nem a tanítás során gyűjtött running statisztikát. Ráadásul a `no_grad` csak a gradiens számítását kapcsolja ki, a BN `training` flagjét nem. Sőt a `torch.no_grad()` a BN puffereit sem védi: train módban a forward továbbra is frissíti a `running_mean` és `running_var` értékeket, így a tanár kimenete a hívások során megváltozik, és a cache-elt célok nem reprodukálhatók.

Következmény: a soft célok batch-függővé válnak; ugyanahhoz a képhez különböző minibatch-összetétel mellett más q tartozhat. A diák inkonzisztens célokat kap, ami a desztillációs jelet zajossá és a kiértékelési viselkedéstől eltérővé teszi.

Javítás:

```python
teacher.eval()
with torch.no_grad():
    z_t = teacher(x_batch)
```

Ellenőrzés: futtassuk a tanárt két különböző minibatch-be rendezett adaton; `eval()` módban a logitoknak batch-átrendezéstől függetlenül azonosnak kell lenniük, míg a hibás `train()` módban a BatchNorm miatt eltérhetnek.

34.4.7.5 F5 megoldása

Megfigyelési egység: a GlyphLab `scene_id`, mert a két nézet ugyanahhoz a jelenethez tartozik; a bootstrap és a páros összehasonlítás is jelenetszintű 22.4. Augmentációs kísérlet és domainváltozás. **Split:** a rögzített train/val/test osztás; a corrupt teszt csak a test scene párokra épül. **Modellválasztás:** minden seedhez a clean val loss minimumánál választunk best checkpointot; a tesztet csak ezután, egyszer értékeljük. **Változó tényező:** a soft tag hőmérséklete $T \in \{1;\ 4;\ 8\}$, rögzített $\alpha = 0{,}7$ mellett, plusz a $\alpha = 0$ baseline. **Metrika:** clean test accuracy, corrupt test accuracy, és a corrupt–clean páros különbség átlaga. **Költségkeret:** a tanár egyszer tanul meg, és minden diák-kísérlet ugyanannyi képcímkét használ; a számítási keret a diák forward/backward lépéseinek számával normalizálva azonos. **Érvényességi korlát:** a GlyphLab toy adat, ezért a T sorrendből nem vonható le következtetés természetes képekre; a 3 seeddel becsült szórás maga is zajos, ezért nem tekinthető a seedek közötti szórás alsó korlátjának; a soft célokból származó nyereséget a corrupt–clean páros különbség átlagának a hard-label baseline-hoz viszonyított javulásaként definiáljuk, és csak akkor tekintjük kimutatottnak, ha a jelenetszintű bootstrap 95%-os konfidenciaintervalluma nem tartalmazza a nullát; ez a küszöb nem bizonyítja, hogy a diák a tanár minden hibaszeletét megőrzi.

34.4.8 Tipikus tévedés és ellenpélda

Gyakori tévedés, hogy a desztillált diák automatikusan megőrzi a tanár minden viselkedését, például a ritka osztályokon vagy a corrupt bemeneteken mutatott pontosságát. A desztillációs loss a soft célok átlagos KL-divergenciáját bünteti, de a diák kapacitása kisebb, és a loss nem tartalmaz szeletenkénti súlyozást. Ellenpélda: legyen egy háromosztályos feladat, amelyben a minták 90%-a az 1. osztályba, 10%-a a 2. osztályba tartozik. A tanár soft célja a gyakori osztály mintáin $q_f = (0{,}02;\ 0{,}93;\ 0{,}05)$, a ritka osztály mintáin $q_r = (0{,}05;\ 0{,}10;\ 0{,}85)$. A diák a gyakori osztályon jól illeszkedik: $p_f = (0{,}02;\ 0{,}90;\ 0{,}08)$ mellett $D_{KL}(q_f \| p_f) \approx 0{,}0070$. A ritka osztályon a diák a minták 70%-án a helyes osztályt adja, $p_{r,\text{ok}} = (0{,}10;\ 0{,}30;\ 0{,}60)$ mellett $D_{KL}(q_r \| p_{r,\text{ok}}) \approx 0{,}1515$; a minták 30%-án viszont az 1. osztályt választja, $p_{r,\text{hiba}} = (0{,}12;\ 0{,}45;\ 0{,}43)$ mellett $D_{KL}(q_r \| p_{r,\text{hiba}}) \approx 0{,}3851$. A ritka osztály átlagos KL-je így $0{,}7 \cdot 0{,}1515 + 0{,}3 \cdot 0{,}3851 \approx 0{,}2216$, a teljes, mintaarányos átlag pedig $0{,}9 \cdot 0{,}0070 + 0{,}1 \cdot 0{,}2216 \approx 0{,}0285$: a ritka osztály hibái csak 10%-os súllyal számítanak, ezért az összesített KL alacsony marad, miközben a ritka osztály pontossága 70%-ra esett vissza. A desztilláció tehát nem helyettesíti a 33.4. Üzemeltetés, drift és késő címke alfejezetben bevezetett monitorozást és a hibaszelet-elemzést; az üzembe helyezés előtt a diákot ugyanazokkal a tesztekkel kell ellenőrizni, mint a tanárt.

34.4.9 Önellenőrzés és visszatekintés

A definíciók ellenőrzése: a soft cél minden sora valószínűségeloszlás, nemnegatív és 1 összegű; a hard-label tag logit bemenetet vár, előtte nem végzünk softmaxot 7.2. Softmax és többosztályos cross-entropy. A KL csak akkor véges, ha q supportja benne van p supportjában; hőmérsékleti softmax esetén ez teljesül. A numerikus stabilitáshoz `log_softmax` használata indokolt, nem külön `softmax` majd `log`. A 34.3. Pruning és struktúra alfejezetből tudjuk, hogy a pruning után a ritka súlyok tényleges kihagyása külön lépés; a desztilláció a kisebb modell előállításának alternatív útja, és a kettő kombinálható, ha a prune-olt diákot ezután a soft célokkal finomhangoljuk. A

30.4. Önfelügyelt kísérlet korrekt összevetése költségkerete itt is érvényes: ha a tanári logitokat címkézetlen képekre is kiterjesztjük, akkor a címkeköltség és a teacher-forward költség külön sorban könyvelendő, hogy a hard-label baseline-nal való összehasonlítás fair maradjon. Végül a 22.4. Augmentációs kísérlet és domainváltozás szerint a corrupt tesztet csak a befagyasztott, legjobb val checkpointon értékeljük, és a clean–corrupt különbséget páros, jelenetszintű adatként kezeljük.

34.4.10 Rövid összefoglaló

A tudásdesztilláció egy kiértékelő módba kapcsolt, befagyasztott tanár lágyított osztályeloszlásait használja célként egy kisebb diák tanításához. Az (1) loss egy α súlyú soft KL tagból és egy $(1-\alpha)$ súlyú hard-label cross-entropy tagból áll. A soft tagban a tanári és diáklagitok egyaránt T-vel osztottak; a lemma szerint a KL gradiens $(p-q)/T$, ezért a T^2-es szorzó a nagy T határesetben kompenzálja a logit-gradiens $1/T^2$-es csillapodását. A labor azonos tesztprotokollt ír elő a tanárra, diákra és hard-label baseline-ra, és a corrupt tesztet páros jelenetszintű adatként értékeli.

34.4.11 Továbblépés

A 35.1. Saliency és Grad-CAM a Grad-CAM és a saliency-térképek alapjait tárgyalja. A desztillált diák kisebb paraméterszámú modellje alkalmas kiindulópont a magyarázatok sanity checkjéhez, mert a tanár-diák eltérések vizsgálata megmutathatja, mely bemeneti régiókban marad meg vagy vész el a tanári viselkedés.

34.4.12 Források

- [P26] Hinton, G., Vinyals, O., Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531. https://arxiv.org/abs/1503.02531.
- [D16] PyTorch contributors (2026). *torch.utils.benchmark*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/benchmark_utils.html. Hozzáférés: 2026-10-30.
- [D17] PyTorch contributors (2026). *torch.profiler*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/profiler.html. Hozzáférés: 2026-10-30.

35 Magyarázhatóság, robusztusság és felelős felhasználás

35.1 Saliency és Grad-CAM

35.1.1 Cél, előismeret és induló kérdés

Egy betanított CNN a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab adaton 93,3% pontosságot ér el a tiszta tesztképeken (a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben mért `clean_acc = 0,9333`). Az üzemeltető azonban nemcsak arra kíváncsi, hogy a modell *mit* válaszol, hanem arra is, hogy a döntés *mely képrészleteken* alapul. A 33.4. Üzemeltetés, drift és késő címke egységben láttuk, hogy a konfidencia önmagában nem magyarázat; itt a modell belső aktivációinak és gradienseinek lokális elemzését építjük fel. Az egység célja, hogy pontosan definiálja, mit jelent egy osztályozási döntés térbeli magyarázata, és hogyan állítható elő ilyen hőtérkép a betanított modell megváltoztatása nélkül.

Előismeret: az autograd és a VJP iránya a 11.1. Tensor és autograd-életciklus alfejezetből; a feature map mint köztes reprezentáció a 13.3. Lokális szűrők és tanult reprezentáció alfejezetből; a modellkártya és a szeletek szerinti értékelés a 23.4. Hibaszeletek és alkalmazási határ alfejezetből; a gradiens mint érzékenység a 4.2. Parciális derivált és gradiens alfejezetből. A valószínűségi alapon tárgyalt konfidencia itt nem elegendő: a softmax egyetlen valószínűségvektort ad, a konfidencia ebből képzett skalár, a magyarázathoz viszont térbeli felbontás kell.

Induló kérdés. Adott egy 24×24-es tesztkép, amelyre a modell a függelék A. A GlyphLab adatkészlet szakaszában rögzített osztálysorrend szerinti `1` (horizontal) osztályt jósolja. A kép tartalmaz egy vízszintes vonalat és egy világos hátteret. Honnan tudjuk, hogy a modell a vonalra figyelt, és nem a kép bal felső sarkában lévő zajos foltra? A válasz: a hőtérkép és az alakzat maszkjának mérhető átfedésével (például IoU-val), nem a hőtérkép puszta szemrevételével; a protokollt az F5 feladat adja.

35.1.2 Definíciók: input gradiens, feature map és a magyarázat tárgya

Rögzítsünk egy $f : \mathbb{R}^{1\times H\times W} \to \mathbb{R}^K$ osztályozót, amely egy x képet K logitba képez le. Legyen $c \in \{0;\ \dots;\ K-1\}$ az a célosztály, amelynek a döntését magyarázni akarjuk. A magyarázat mindig egy konkrét (x, c) párra vonatkozik; más bemenetre vagy más célosztályra a hőtérkép általában eltérő.

1. definíció (input gradiens, más néven saliency). A c célosztály logitjának az input szerinti gradiense

$$S(x,c) \;=\; \nabla_x z_c \;=\; \left(\frac{\partial z_c}{\partial x_{0,i,j}}\right)_{i,j} \;\in\; \mathbb{R}^{1\times H\times W}.$$

Az S megmondja, hogy az egyes inputpixelek *első rendben* mennyire érzékenyek a cél logitra. Ez a logit linearizációjának együtthatója: kis Δx perturbációra $z_c(x+\Delta x) \approx z_c(x) + \langle S, \Delta x\rangle$. Az

input gradiens pixel felbontású, de zajos: a gradiensjel a bemenettől a cél logitig sok nemlineáris és pooling-műveleten halad keresztül, ezért az egyes pixelekre jutó részleges hatások előjele és nagysága gyakran szétszórt, és a térkép nem feltétlenül esik egybe az ember által értelmesnek tartott objektummal.

2. definíció (feature map). Legyen $A^l \in \mathbb{R}^{C_l \times H_l \times W_l}$ egy köztes l réteg aktivációtensora, C_l csatornával és $H_l \times W_l$ térbeli felbontással. A k-adik csatornát $A^l_k \in \mathbb{R}^{H_l \times W_l}$ jelöli, a (k, i, j) eleme pedig $A^l_{k,i,j}$. A cél az, hogy a z_c logit magyarázatát ne pixel-, hanem *feature map* szinten adjuk meg, majd a kapott durvább hőtérképet interpoláljuk vissza az input felbontására.

A két fogalom nem felcserélhető: az input gradiens az inputnak a logitra gyakorolt *lokális* érzékenysége; a Grad-CAM a köztes csatornák *térben aggregált* fontosságát kombinálja a csatornák aktivációival, ezért simább és szemantikusabb térképet adhat.

35.1.3 A Grad-CAM súlyozási képlete: motiváció és konvenció

A Grad-CAM a [P23] cikk által bevezetett módszer; az alábbi rész a tankönyv saját kidolgozása, a cikkben szereplő aggregációs konvenciót követve. A cél: adjunk a c célosztály logitjának egy olyan közelítő felbontását a C_l csatorna felett, amely minden csatornához egyetlen skalár súlyt rendel, és ez a súly kifejezhető a visszafelé terjedő gradiensből. Ez motiváció és konvenció, nem általános hálózatra érvényes egzakt levezetés.

3. definíció (a választott aggregációs konvenció). A csatornán belüli térbeli aggregáció a *globális átlag pooling* (GAP):

$$\bar{A}^l_k \;=\; \frac{1}{H_l W_l} \sum_{i=1}^{H_l} \sum_{j=1}^{W_l} A^l_{k,i,j}.$$

Ez egy lineáris, átlagoló redukció, amely a térbeli dimenziókat egyetlen skalárrá húzza össze. Azért ezt választjuk, mert a logit szempontjából nem azt akarjuk mérni, hogy egyetlen feature-pixel mennyire fontos, hanem hogy a k-adik csatorna *egésze* milyen mértékben járul hozzá a célosztályhoz.

Motiváció a gradiensből. Tekintsük a z_c logitot az A^l aktivációk függvényének. A lokális elsőrendű közelítés szerint

$$\Delta z_c \approx \sum_{k,i,j} \frac{\partial z_c}{\partial A^l_{k,i,j}} \, \Delta A^l_{k,i,j}.$$

Egy rögzített csatorna minden elemét azonos δ értékkel eltolva és a többi csatornát változatlanul hagyva a logit iránymenti deriváltja a csatorna gradienseinek összege:

$$\frac{d}{d\delta} z_c(A^l + \delta E_k)\Big|_{\delta=0} = \sum_{i,j} \frac{\partial z_c}{\partial A^l_{k,i,j}},$$

ahol E_k a k-adik csatornában minden térbeli pozíción 1, másutt 0. A Grad-CAM a csatorna gradienseinek **átlagát** választja súlynak:

$$w^c_k = \frac{1}{H_l W_l} \sum_{i=1}^{H_l} \sum_{j=1}^{W_l} \frac{\partial z_c}{\partial A^l_{k,i,j}}.$$

Ez az előbbi iránymenti derivált $1/(H_l W_l)$-szerese. Az azonosság csak az explicit, azonos eltolást alkalmazó útra érvényes: a csatorna minden elemét ugyanazzal a δ értékkel toljuk el, a többi csatornát pedig rögzítjük. Az $A^l_{k,i,j}$ elemek és a csatornaátlag nem független változók, ezért a $\partial A^l_{k,i,j}/\partial \bar{A}^l_k = 1/(H_l W_l)$ formális parciális derivált korlátozás nélkül nem értelmezhető. A csatornánkénti gradiensek átlagolása a Grad-CAM választott aggregációs szabálya; általános hálózatra nem vezethető le egy, a GAP szerinti logitdekompozíció egzakt azonosságaként.

A hőtérkép a csatornák súlyozott kombinációja, amelyből a negatív hozzájárulásokat a ReLU eltávolítja:

$$L^c_{Grad-CAM}(x) \;=\; \mathrm{ReLU}\left(\sum_{k=1}^{C_l} w^c_k\, A^l_k\right) \;\in\; \mathbb{R}^{H_l \times W_l}.$$

A ReLU indoklása heurisztikus: a cél az, hogy a *célosztályt támogató* régiókat mutassuk. A ReLU a súlyozott összeg azon térbeli pozícióit nullázza, ahol az összeg negatív; ezek a pozíciók a célosztály ellen szóló bizonyítékot hordozhatnak, és a vizualizációban félrevezetőek lennének. Ez egy heurisztika, nem bizonyított tény a modell döntési mechanizmusáról.

Normalizáció. A nyers $L^c_{Grad-CAM}$ értéktartománya rétegenként és bemenetenként eltérő. A képenkénti min–max normalizálás csak a *relatív térbeli mintázatot* teszi hasonló skálájúvá; két kép azonos színe nem jelent azonos abszolút hozzájárulást, és számszerű képek közti fontosság-összehasonlításhoz közös, rögzített skála kell. Az összehasonlíthatósághoz (a mintázat szintjén) min–max normalizálást végzünk:

$$\tilde{L}^c_{i,j} \;=\; \frac{L^c_{i,j} - \min_{i',j'} L^c_{i',j'}}{\max_{i',j'} L^c_{i',j'} - \min_{i',j'} L^c_{i',j'}}.$$

Ha a nevező nulla, az egész térkép nulla; ezt a szélső esetet explicit módon kezelni kell. Végül az $\tilde{L}^c$ térképet bilineáris interpolációval felskálázzuk $H \times W$ méretre.

35.1.4 Kézi számpélda

Legyen $C_l = 2$, $H_l = W_l = 2$. A l réteg aktivációi:

$$A^l_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad A^l_2 = \begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}.$$

Tegyük fel, hogy a c célosztályra a visszafelé terjedő gradiensek:

$$\frac{\partial z_c}{\partial A^l_1} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \frac{\partial z_c}{\partial A^l_2} = \begin{bmatrix} 0 & 2 \\ 0 & 2 \end{bmatrix}.$$

A GAP-konvencióval

$$w^c_1 = \frac{1}{4}(1+1+1+0) = \frac{3}{4}, \qquad w^c_2 = \frac{1}{4}(0+2+0+2) = 1.$$

A súlyozott összeg csatornánként:

$$w_1^c A_1^l = \begin{bmatrix} 0{,}75 & 1{,}5 \\ 2{,}25 & 3 \end{bmatrix}, \qquad w_2^c A_2^l = \begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}.$$

Összegük és a ReLU (itt minden elem pozitív, tehát hatástalan):

$$L^c_{Grad-CAM} = \begin{bmatrix} 0{,}75 & 2{,}5 \\ 4{,}25 & 3 \end{bmatrix}.$$

Min–max normalizálás: maximum 4,25, minimum 0,75, terjedelem 3,5, így

$$\tilde{L}^c = \frac{1}{3{,}5} \begin{bmatrix} 0 & 1{,}75 \\ 3{,}5 & 2{,}25 \end{bmatrix} = \begin{bmatrix} 0 & 0{,}5 \\ 1 & 0{,}6429 \end{bmatrix}.$$

A bal alsó elem a legfontosabb: ott mindkét csatorna hozzájárul ($w_1^c A_1 = 0{,}75 \cdot 3 = 2{,}25$ és $w_2^c A_2 = 1 \cdot 2 = 2$, összesen 4,25); az A_1 gradiense ott 1. A példa jól mutatja, hogy a hőtérkép *nem* azonos a gradienssel: a jobb alsó elemnél az A_1 aktivációja 4, a gradiensé 0, az A_2 aktivációja 0, a gradiensé 2; a hozzájárulás mégis 3, mert A_1 erős, noha a gradiens ott nulla.

Ellenőrzés szélső esetre. Ha minden gradiens zérus lenne, akkor $w_1^c = w_2^c = 0$, a hőtérkép azonosan nulla, és a min–max normalizálást a nullás ág kezeli.

35.1.5 Labor: hőtérkép a saját modellen és a célosztály szerepe

A labor a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben tanított `TinyCNN` modellt és az ott létrehozott `lab/best.pt` checkpointot használja. Ha ez még nem készült el, előbb a 24.4. VI. ellenőrző pont: megbízható CNN-training tanítását kell futtatni. A bemenet kizárólag a 24×24-es grayscale kép; maszkot vagy dobozt a modell nem lát, és a hőtérkép sem használhatja őket. A hőtérkép a `model.features[3]` (második konvolúció) kimenetére vonatkozik.

```
# lab/n137_gradcam.py
# Futtatás a projekt gyökérkönyvtárából; a checkpoint architektúráját
# az alábbi osztály a tanításnál használt architektúrával egyezik.
from pathlib import Path
import numpy as np
import torch

root = Path("lab")
device = torch.device("cpu")
images = np.load(root / "data" / "images.npy")  # (360,1,24,24)

class TinyCNN(torch.nn.Module):
    # Egyezik a 24.4. lecke tanítóprogramjának architektúrájával.
    def __init__(self, in_ch=1, num_classes=3):
        super().__init__()
        self.features = torch.nn.Sequential(
            torch.nn.Conv2d(in_ch, 8, 3, padding=1),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
            torch.nn.Conv2d(8, 16, 3, padding=1),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(2),
            torch.nn.Flatten(),
        )
        self.fc = torch.nn.Linear(16 * 6 * 6, num_classes)
```

```
    def forward(self, x):
        return self.fc(self.features(x))

model = TinyCNN().to(device)
state = torch.load(root / "best.pt", map_location=device, weights_only=True)
# A 24.4. lecke a checkpointot a "model" kulcs alatt menti.
model.load_state_dict(state["model"])
model.eval()
# A features[3] a második konvolúció kimenete, a ReLU előtt; a Grad-CAM
# a nyers konvolúciós aktivációt használja. Ez módszertani választás.
# A TinyCNN ReLU-ja nem in-place, ezért a forward hook biztonságos.
target_layer = model.features[3]  # második konvolúció kimenete, (N,16,12,12)

# Egyetlen forwardhoz tartozó aktivációt és logitokat használunk.
def heatmap_for_class(x, class_idx):
    assert x.shape[0] == 1, "a logits[0, class_idx] indexelés egy mintát feltételez"
    captured = {}
    hook = target_layer.register_forward_hook(
        lambda module, inputs, output: captured.update(A=output))
    try:
        logits = model(x)
        A = captured["A"]
        grad_A, = torch.autograd.grad(logits[0, class_idx], A)
        weights = grad_A.mean(dim=(2, 3), keepdim=True)
        raw = torch.relu((weights * A).sum(dim=1, keepdim=True))
        span = raw.max() - raw.min()
        heat = (raw - raw.min()) / span.clamp_min(1e-8)
        # A magyarázó térképet a bemeneti kép 24×24-es rácsára helyezzük,
        # így a képi maszkokkal is értelmesen összevethető.
        heat = torch.nn.functional.interpolate(
            heat, size=x.shape[-2:], mode="bilinear", align_corners=False)
        return heat.detach(), logits.detach()
    finally:
        hook.remove()

# A metaadatban szereplő split alapján kell valódi tesztképet választani.
# Itt az index demonstrációs példa, nem állítjuk, hogy a testhez tartozik.
x = torch.from_numpy(images[10:11]).to(device=device, dtype=torch.float32)
with torch.enable_grad():
    logits = model(x)
    c = int(logits.argmax(dim=1).item())
    heat, _ = heatmap_for_class(x, c)
    other, _ = heatmap_for_class(x, (c + 1) % 3)

assert heat.shape == (1, 1, 24, 24)
assert torch.isfinite(heat).all() and torch.isfinite(other).all()
diff = (heat - other).abs().max().item()
print("Prediktált osztály:", c, "térképek eltérése:", diff)
# Más osztályhoz más térkép várható, de a különbség szigorú
# pozitivitása degenerált modellnél nem matematikai garancia.
torch.save({"image_index": 10, "target_class": c,
            "heatmap": heat.cpu(), "other_heatmap": other.cpu(),
            "max_abs_diff": diff}, root / "gradcam_demo.pt")
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 10 szál, a checkpoint seedje 20261020): `Prediktált osztály: 0 térképek eltérése: 0.8989173769950867`. A 10. indexű képen a modell tehát a 0 osztályt jósolja, és a két célosztályhoz tartozó normalizált térkép maximális eltérése 0,899; ez egyetlen képre és egyetlen checkpointra mért érték, nem általános garancia.

A kód a prediktált osztályt, a normalizált hőtérképet és egy másik célosztállyal vett eltérést

rögzít. Mindkét Grad-CAM-számításban a logit és a köztes aktiváció **ugyanabból a forwardból** származik; ettől lesz helyes az `autograd.grad` gráfkapcsolata. A `clamp_min(1e-8)` kezeli a zérus terjedelmet, de nagyon kis pozitív terjedelemnél módosíthatja a skálát, ezért az ilyen térképek értelmezése külön óvatosságot igényel.

35.1.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja külön az input gradiens saliencyt és a Grad-CAM hőtérképet. Melyik objektum melyik tenzorból készül, és mi a szerepe a ReLU-nak a Grad-CAM-ben?

F2 (levezetési, nehézség 2). Legyen $H_l = 1$, $W_l = 2$, $C_l = 2$, $A_1^l = (2;\ 4)$, $A_2^l = (1;\ 0)$, és a célosztály gradiense $\frac{\partial z_c}{\partial A_1^l} = (3;\ 1)$, $\frac{\partial z_c}{\partial A_2^l} = (-2;\ 2)$. Számítsa ki a két w_k^c súlyt, a $L^c_{Grad-CAM}$ nyers térképet és annak min–max normalizáltját.

F3 (implementációs, nehézség 2). Írjon NumPy-függvényt `gradcam_numpy(A, dA)` néven, ahol `A` alakja (C_l, H_l, W_l) és `dA` a $\partial z_c/\partial A$ gradiens, ugyanakkora alakban. A függvény adja vissza a normalizált Grad-CAM-térképet. Ellenőrizze a fenti 2×2-es kézi példán, és adja meg az elvárt toleranciát.

F4 (hibakeresési, nehézség 2). Egy fejlesztő a laborkódban a `model(x)` hívást `with torch.no_grad():` blokkba teszi, hogy gyorsítsa a hőtérkép-számítást. Miért nem kap gradienst a `torch.autograd.grad(score, feature)` hívásnál? Pontosan lokalizálja a hibát, és adja meg a javítás utáni ellenőrzést.

F5 (kísérlettervezési, nehézség 3). Tervezzen GlyphLab-kísérletet annak eldöntésére, hogy a Grad-CAM hőtérkép a corrupted tesztképeken az eredeti alakzat maszkjára vagy a fényesség-gradiensre fókuszál; a fényességgradiens-maszk itt a kép luminanciagradiensének nagyságából küszöbölt bináris maszk, amely a korrupció fényességváltozásait jelöli, nem az alakzatot. Adja meg a megfigyelési egységet, a hőtérkép és a maszk átfedésének mérőszámát, a szükséges küszöb-protokollt és a következtetés korlátait. A tesztet a modell kiválasztása után, befagyasztott súllyal kell lefuttatni.

35.1.7 Megoldások és indoklás

F1. Az input gradiens $S = \nabla_x z_c \in \mathbb{R}^{1\times H\times W}$, vagyis a logitnak a bemeneti pixelek szerinti parciális deriváltja. Ez pixel felbontású, de a mélyebb rétegeken keresztül visszavezetett jel miatt gyakran zajos. A Grad-CAM egy $C_l \times H_l \times W_l$ feature mapból számol: a w_k^c súlyok a $\partial z_c/\partial A^l_{k,i,j}$ gradiensek térbeli átlagai, és a hőtérkép a $w_k^c A_k^l$ súlyozott csatornaösszeg ReLU-ja. A ReLU eltávolítja a negatív összegű régiókat; ez a célosztályt *támogató* régiók megjelenítését szolgálja, de heurisztika.

F2. A GAP-konvenció: $w_1^c = \frac{1}{2}(3+1) = 2$, $w_2^c = \frac{1}{2}(-2+2) = 0$. A nyers térkép $L = ReLU(2\cdot(2;\ 4) + 0\cdot(1;\ 0)) = (4;\ 8)$; a ReLU nem változtat, mert minden elem pozitív. Min–max: minimum 4, maximum 8, így $\tilde{L} = (0;\ 1)$.

F3. Egy lehetséges megoldás:

```
import numpy as np

def gradcam_numpy(A, dA, eps=1e-12):
    # A, dA: (C, H, W)
    assert A.shape == dA.shape and A.ndim == 3
    w = dA.mean(axis=(1, 2), keepdims=True)   # (C,1,1)
```

```
    heat = (w * A).sum(axis=0, keepdims=True) # (1,H,W)
    heat = np.maximum(heat, 0.0)
    lo, hi = heat.min(), heat.max()
    if hi - lo < eps:
        return np.zeros_like(heat)
    return (heat - lo) / (hi - lo)
```

Az elvárt eredmény a kézi példán `[[0, 0.5], [1, 0.64285714]]`, float32 toleranciával 10^{-6} abszolút eltérés.

F4. A `torch.no_grad()` kontextus letiltja az autograd gráf építését, ezért a hookban rögzített `captured["A"]` tenzornak nincs `grad_fn` lánca, és a `logits[0, class_idx]` sem vezethető vissza rá. A `torch.autograd.grad(logits[0, class_idx], captured["A"])` ekkor `RuntimeError`-t ad, mert a kimenet és a bemenet között nincs gráfkapcsolat. A javítás: a `model(x)` hívást a `no_grad()` blokkon kívül kell futtatni; a betanított modell `eval()` állapota önmagában nem tiltja a gradiensszámítást, csak a dropout és a BatchNorm viselkedését változtatja meg. Ellenőrzés: a javítás után a `torch.autograd.grad` visszatérési értéke egy nem üres tenzor-tuple, amelynek első eleme a rögzített aktiváció alakjával, azaz $(1, C_l, H_l, W_l)$-gyel egyezik.

F5. Protokollvázlat: a rögzített, befagyasztott modellen a GlyphLab szerződés szerinti `corrupted_test.npy` mind a 60 tesztképére (a test split 60 képe) kiszámítjuk a Grad-CAM hőtérképet a predikált osztályra. A hőtérképet a metrikához $H \times W$ méretre interpoláljuk. A kísérlet azonban a maszkkal való átfedést méri: binarizáljuk a hőtérképet egy τ küszöbbel, és kiszámítjuk az IoU(küszöbölt hőtérkép, maszk) értéket. A τ-t a train képek hőtérkép-eloszlásából választjuk, nem a corrupted tesztből. Emellett kiszámítjuk ugyanezt a fényességgradiens-maszkra is: ez a kép luminanciájának gradiensnagyságából (például Sobel-operátorral számítva) küszöbölt bináris maszk, amely a korrupció fényességváltozásainak helyét jelöli, nem az alakzat körvonalát. A két IoU különböző referenciamaszkra vonatkozik, ezért értékük nem azonos skálán hasonlítható össze; a páros különbséget ordinális jelzésként kezeljük, és a két maszk területarányát is jelentjük. Ha az alakzatmaszkkal mért IoU jelenetszintű páros különbségének 95%-os bootstrap-konfidenciaintervalluma (10 000 jelenetszintű újramintavétel, percentilis-intervallum; 23.3. Páros összehasonlítás és csoportszintű intervallum protokollja) a gradiensmaszk IoU-jához képest teljes egészében 0 felett van, akkor a hőtérkép a Grad-CAM által jelzett fókusz szerint az alakzatra koncentrál; az állítás azonban csak az adott korrupciós eloszlásra és a választott küszöbre érvényes, és nem bizonyít oksági mechanizmust. A bootstrap egysége a `scene_id`, nem a kép: a GlyphLab-szerződés szerint egy jelenethez két nézet tartozik, ezért a két nézet nem független megfigyelés. A bootstrap seedje 20261020, az erőforrás CPU, 10 000 jelenetszintű újramintavétel egy befagyasztott modellen.

35.1.8 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés azt hinni, hogy a vizuálisan tetszetős hőtérkép oksági bizonyíték. A Grad-CAM a logit elsőrendű érzékenységéből és a forward aktivációkból származtatott *korrelációs jellegű* térkép; nem mutatja meg, hogy az adott régió eltávolítása ténylegesen megváltoztatná-e a döntést.

Ellenpélda: legyen egy olyan modell, amely az input bal felső sarkában lévő állandó világos foltra támaszkodik, mert a tanító adatban az mindig együtt járt a 2 osztállyal. A Grad-CAM magas értéket adhat erre a foltra, mert a logit érzékeny rá, és a hőtérkép szép, kontúros képet mutat. A folt eltávolítása azonban lehet, hogy nem változtatja meg a döntést, mert a modell más, a hőtérképen nem kiemelt csatornákon keresztül is a 2 osztályra szavaz. A [P25] sanity-check vizsgálatai szerint a Guided BackPropagation és a Guided Grad-CAM hőtérképe a háló súlyainak

véletlenítése után is hasonló maradt, míg a Grad-CAM és a nyers gradiens érzékenyen változott; ezért a „szép hőtérkép” nem azonos a „verifikált modellmagyarázattal”.

35.1.9 Rövid összefoglaló

Az input gradiens a célosztály logitjának pixelenkénti érzékenysége, a Grad-CAM ezzel szemben egy köztes feature map csatornáit súlyozza a visszafelé terjedő gradiensek térbeli átlagával. A levezetés megmutatta, hogy a w_k^c súly a logitnak a k-adik csatorna azonos eltolására vett iránymenti deriváltjának $1/(H_l W_l)$-szerese; ez a Grad-CAM választott aggregációs konvenciója, nem általános egzakt dekompozíció. A hőtérkép a súlyozott csatornaösszeg ReLU-ja, amelyet min–max normalizálással és interpolációval teszünk összehasonlíthatóvá. A labor egy befagyasztott GlyphLab-modellre készít hőtérképet, és kiszámítja két célosztály térképének maximális eltérését; a különbség szigorú pozitivitását nem állítja és nem is assertálja. A hőtérkép azonban korrelációs jellegű vizualizáció, nem oksági bizonyíték; a 35.2. Magyarázat sanity check alfejezetben ezt sanity check protokollokkal vizsgáljuk tovább.

35.1.10 Felhasznált források

- [P23] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* ICCV 2017. arXiv:1610.02391. https://arxiv.org/abs/1610.02391.
- [P25] Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). *Sanity Checks for Saliency Maps.* NeurIPS 2018. arXiv:1810.03292. https://arxiv.org/abs/1810.03292.

35.2 Magyarázat sanity check

35.2.1 Cél, előismeret és induló kérdés

A 35.1. Saliency és Grad-CAM egységben definiáltuk a saliency-térképet és a Grad-CAM-et mint „a modell döntésére magyarázatot kínáló” vizualizációt. Ott a képlet megadja, *mit* rajzolunk. Most azt kérdezzük: honnan tudjuk, hogy a kirajzolt térkép egyáltalán a modelltől függ, nem pedig csak a kép statisztikai tulajdonságaitól? Ha egy módszer akkor is ugyanazt a „fontos területet” jelölné, amikor a modell súlyait véletlenre cseréljük, akkor a magyarázat nem a tanult modell magyarázata.

Előismeret. A 35.1. Saliency és Grad-CAM alfejezetből a bemeneti gradiens és a Grad-CAM súlyozási képlete szükséges. A 34.2. Kvantálás és hibatovábbterjedés arra figyelmeztetett, hogy a numerikus reprezentáció hibája különbözik a modellstruktúra hibájától; a mostani kontroll szintén különválasztja a mérési hibát a magyarázat tényleges modellfüggésétől. A 31.2. GAN mint minimax probléma alfejezetben láttuk, hogy egy optimum létezése nem garancia a véges tanítás konvergenciájára; hasonlóan, egy magyarázó képlet helyessége nem garancia arra, hogy a kimenet a tanult reprezentációt tükrözi. A 23.2. Kalibráció és döntési küszöb kalibrációs tanulsága pedig az, hogy egy számszerű kimenet (ott a softmax, itt a fontossági térkép) önmagában nem tekinthető evidenciának a mögöttes állításra.

Induló kérdés. Adott egy betanított GlyphLab-CNN (az adatokat a függelék A. A GlyphLab adatkészlet szakasza rögzíti), és legyen a Grad-CAM-je egy adott képen a kép felső sávjára koncentrált. Tegyük fel, hogy a modell súlyait véletlenre cseréljük, majd ugyanarra a képre újra

kiszámítjuk a Grad-CAM-et. Ha a két térkép gyakorlatilag azonos, akkor megbízhatunk-e abban, hogy a magyarázat a *tanult* modell döntéséről szól?

35.2.2 Definíciók, jelölések és intuíció

Jelölések. Legyen a bemenet $x \in [0;\ 1]^{1\times 24\times 24}$ (a GlyphLab NCHW konvenciója; a batch-dimenziót elhagyjuk), a modell $f_\theta : \mathbb{R}^{1\times 24\times 24} \to \mathbb{R}^K$ logitokkal, $\theta \in \mathbb{R}^P$ paramétervektorral; a megengedett paramétertér $\Theta \subseteq \mathbb{R}^P$, a fejezetben $\Theta = \mathbb{R}^P$. A $c \in \{0;\ \dots;\ K-1\}$ mindig a célosztály indexe; a jellemzőtérkép csatornájának indexét a kézi példában q jelöli, hogy ne ütközzön a célosztállyal. Egy magyarázó módszer

$$S(x;\theta,c) \in \mathbb{R}_{\geq 0}^{24\times 24}$$

nemnegatív fontossági térképet ad a c célosztályra. A 35.1. Saliency és Grad-CAM alfejezetben tárgyalt Grad-CAM ([P23]) esetén c rendszerint a modell által prediktált osztály; a randomizálási kísérletben rögzített, előre választott c-t használunk, mert a véletlen súlyú modell predikciója nem hordoz tanult információt. A konkrét S lehet a bemeneti gradiens nagysága vagy a Grad-CAM upsampling utáni térképe; a kontroll logikája mindkettőre azonos.

Parameter randomization (paraméter-randomizálás). A tanulható rétegeket a kimenettől a bemenet felé haladó, rögzített sorrendben indexeljük: az 1. szint a legfelső (a kimenethez legközelebbi) tanulható réteg, az L. szint a bemenethez legközelebbi. A j-edik szintű, **kaszkádolt** randomizálás a legfelső j réteg minden θ_ℓ paraméterét ($\ell = 1, \dots, j$) a réteg inicializálási eloszlásából frissen húzott θ'_ℓ értékkel helyettesíti, a mélyebben fekvő $j+1, \dots, L$ rétegek a betanított értékükön maradnak. A teljes randomizálásnál $j = L$. A kaszkádolt, felülről lefelé haladó sorrend azért számít, mert a kontroll görbéje j függvényében csak így értelmezhető (a [P25] protokollja is így jár el); a jelen fejezet laborja kizárólag a $j = L$ teljes randomizálást valósítja meg, ezért a j-függő görbét nem méri, és ilyen görbére nem is hivatkozunk. A kontroll azt méri, hogy

$$\rho\big(S(x;\theta,c),\ S(x;\theta^{(j)},c)\big)$$

hogyan változik j függvényében, ahol ρ egy hasonlósági metrika.

Label randomization (címkerandomizálás). A tanítóhalmaz megfigyelési egységeit — i.i.d. minta esetén magukat a mintákat, a GlyphLabban a jeleneteket — indexeljük $i = 1, \dots, N$-nel, és a címkéket a **megfigyelési egységek indexein végzett** $\pi : \{1;\ \dots;\ N\} \to \{1;\ \dots;\ N\}$ véletlen permutációval $\tilde{y}_i = y_{\pi(i)}$-re cseréljük: az egység minden képe (GlyphLabban a jelenet mindkét nézete) ugyanazt a permutált címkét kapja, a képek változatlanok maradnak, majd a modellt a véletlen címkéken **újratanítjuk**. A permutáció az osztályok egység- és képszintű gyakoriságát is pontosan megőrzi, ha minden egységhez ugyanannyi kép tartozik; a GlyphLabban ez teljesül, mert jelenetenként pontosan két nézet van, így a jelenetszintű permutáció a képszintű osztálygyakoriságot is változatlanul hagyja. Ha a jelenetek mérete eltérne, a jelenetszintű permutáció a képszintű osztálygyakoriságot megváltoztathatja, és ezt a kiértékelésnél külön jelezni kell. A **képszintű** permutáció itt nem megfelelő: ugyanazon jelenet két nézete eltérő címkét kaphat, ami ellentmond annak, hogy a jelenetnek egyetlen osztálycímkéje van, és ugyanazt a tartalmat két különböző osztály alá rendeli. A permutáció tehát az osztálygyakoriságot megtartja, de a képek és a tanító címkék közötti kapcsolatot véletlenszerűvé teszi. Ezzel szemben az osztályazonosítók globális átnevezése ($0 \mapsto 1$, $1 \mapsto 2$ stb.) **nem megfelelő kontroll**, mert az eredeti célfüggvény információtartalmát változatlanul hagyja. A kérdés itt nem a betanított modell perturbálása, hanem hogy a tanító jel és a bemeneti képek közötti érdemi kapcsolat

megváltoztatása módosítja-e a magyarázatot. A két randomizálás tehát két különböző függőséget tesztel:

Randomizálás	Mit randomizálunk	Milyen függőséget vizsgál
Paraméter-	betanított súlyok	inference-kori paraméterfüggés
Címke-	tanító címkék	tanulási jel függése

Saliency control. A kontroll egy modellmentes leképezés $C : \mathbb{R}^{1\times 24\times 24} \to \mathbb{R}^{24\times 24}$, például a Sobel-élkép vagy egy rögzített középpontú Gauss-súlyozás. A $C(x)$ értéke nem változik, ha a modellt randomizáljuk. A fejezet **előre rögzített, mérhető döntési szabálya** a következő. Legyen $\bar{\rho}_{\text{rand}}$ a teljes randomizálás ($j = L$) utáni hasonlóság átlaga az ismétlésekből, $\rho_{95,\text{rand}}$ ugyanennek 95. percentilise, $\bar{\rho}_{\text{Sobel}}$ pedig az eredeti magyarázat és a modellmentes C kontroll átlagos hasonlósága. A magyarázat modellhez kötött tartalma akkor tekinthető a modellmentes kontrollnál erősebbnek, ha teljesül $\bar{\rho}_{\text{rand}} < \bar{\rho}_{\text{Sobel}}$ **és** $\rho_{95,\text{rand}} < \bar{\rho}_{\text{Sobel}}$. A szabály a fejezet laborjában közvetlenül mérhető, de csak szükséges jellegű: teljesülése nem bizonyítja a hűséget, nem teljesülése pedig nem cáfolja a modellfüggőséget, mert a Sobel-szint maga is adatfüggő, és a küszöb mérési bizonytalanságot hordoz.

Similarity limits. A hasonlósági metrika megválasztása maga is korlátozza a kontroll értelmezését. A könyv alapértelmezett metrikája a lapított, azonos méretűre interpolált térképek közötti Spearman-rangkoreláció:

$$\rho(r,s) = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2 \sum_i (s_i - \bar{s})^2}}, \qquad r_i, s_i \in [1;\ n] \text{ átlagolt rangok.}$$

A Spearman-ρ invariáns a szigorúan monoton növekvő transzformációkra: ha $S' = \alpha S + \beta$, $\alpha > 0$, akkor $\rho(S, S') = 1$. Ezért a randomizálás okozta pusztán pozitív skálázást a rangkorreláció *nem* detektálja. A metrika globális: lokális eltolódások és a térkép invertálása (negatív ρ) csak korlátozottan jelenik meg. A kontrollt ezért mindig rögzített, indokolt metrika mellett kell jelenteni.

35.2.3 Mit cáfolhat és mit nem bizonyít a kontroll?

A sanity check logikai szerkezetét a következő állításban rögzítjük.

Állítás (a paraméter-randomizálás elutasítási logikája). Legyen M magyarázó módszer, és tegyük fel, hogy M *modellfüggetlen*: minden $\theta, \theta' \in \Theta$, minden x bemenetre és rögzített c osztályra

$$S_M(x; \theta, c) = S_M(x; \theta', c).$$

Ekkor tetszőleges paraméter-randomizálás után a Spearman-ρ az eredeti és a randomizált magyarázat között determinisztikusan 1, feltéve, hogy a magyarázat nem konstans (nulla szórású) térkép.

Bizonyítás. A modellfüggetlenségi feltételt θ és a randomizált $\theta^{(j)}$ párra alkalmazva $S_M(x; \theta, c) = S_M(x; \theta^{(j)}, c)$. A rangvektorok azonosak, tehát a Spearman-ρ számlálójában a két azonos sorozat kovarianciája a szórással egyenlő, azaz $\rho = 1$. Ha a magyarázat konstans, a szórás nulla, és a ρ nem definiált; ezt a kísérletben külön ki kell zárni, például a nulla szórású térkép kizárásával. □

A gyakorlati kontroll ezt a kontrapozíciót használja: ha a mért hasonlóság a randomizálás után 1 alá esik, akkor a modellfüggetlenség feltételezése megdől. A megfigyelés tehát **cáfolhatja** azt az állítást, hogy a magyarázat csak a bemenet statisztikájától függ. Azt viszont **nem bizonyítja**, hogy a megmaradó hasonlóság a „helyes” magyarázat, hogy a módszer a döntési mechanizmust hűen tükrözi, vagy hogy a ρ küszöbértéke általános érvényű. Az alacsonyabb ρ a randomizálás után a modellfüggőség egyik lehetséges jele, de önmagában nem elégséges bizonyíték arra, hogy a hőtérkép hű magyarázat, mert egy buta, de a súlyokat valamilyen nemlineáris módon felhasználó módszer is mutathat modellérzékenységet anélkül, hogy a döntéshez releváns régiókra mutatna.

Ugyanez a logika a címkerandomizálásra: ott a nullhipotézis az, hogy a magyarázat független a tanulási jel osztálytartalmától. Ha a véletlen címkén tanított modell magyarázata nem tér el az igazi címkén tanítottétól, akkor ez a kontroll **nem szolgáltatott bizonyítékot** arra, hogy a módszer a tanult feladatfüggést jeleníti meg. Ebből a nemelutasításból viszont nem következik, hogy a magyarázat független a címkétől vagy hibás: az eltérés hiánya származhat alacsony statisztikai erőből (kevés kép, kevés ismétlés), és a választott hasonlósági mértékből is — a fejezet saját ellenpéldája szerint a Spearman-ρ pozitív skálázásra érzéketlen, így egy modellfüggő módszer is adhat $\rho = 1$-et. Ilyenkor több kontrollt (paraméter- és címkerandomizálás együtt) és több metrikát (rangkorreláció mellett például a térképek szerkezeti hasonlósága vagy a legfontosabb pixelek átfedése) kell használni. De a címkerandomizálás sem bizonyítja a hűséget: az alacsony szintű képstatisztika (például élirány) véletlen címkék mellett is beépülhet az első konvolúciós rétegbe, ezért a ρ nem szükségszerűen esik nullára pusztán a címke miatt.

35.2.4 Eredeti kézi kis példa

Legyen egy utolsó konvolúciós jellemzőtérkép két csatornával, $A \in \mathbb{R}^{2\times 2\times 2}$, a logit pedig a csatornák lineáris kombinációja:

$$f = \sum_{q=1}^{2}\sum_{u=1}^{2}\sum_{v=1}^{2} w_{uv}^{q} A_{uv}^{q} + b.$$

A betanított modell súlyai $w^1 = w^2 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. A Grad-CAM súlyozása 35.1. Saliency és Grad-CAM szerint $\alpha^q = \frac{1}{4}\sum_{u,v} w_{uv}^{q}$, azaz $\alpha^1 = \alpha^2 = 1$. Legyen

$$A^1 = \begin{pmatrix} 0 & 2 \\ 1 & 3 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 3 & 0 \\ 2 & 1 \end{pmatrix}.$$

Ekkor a Grad-CAM aktiváció ReLU után (a térképet G jelöli, hogy ne ütközzön a rétegszámmal):

$$G = \mathrm{ReLU}\left(\alpha^1 A^1 + \alpha^2 A^2\right) = \begin{pmatrix} 3 & 2 \\ 3 & 4 \end{pmatrix}.$$

A lapított vektor $[3;\ 2;\ 3;\ 4]$; a rangok: $2 \to 1$, a két 3-as 2 és 3 átlaga 2,5, a 4-es 4. Tehát $r = [2{,}5;\ 1;\ 2{,}5;\ 4]$.

Most randomizáljuk a fej súlyait:

$$w'^1 = \begin{pmatrix} -1 & 1 \\ 2 & 0{,}5 \end{pmatrix}, \qquad w'^2 = \begin{pmatrix} 1 & 0 \\ 1 & -1 \end{pmatrix}.$$

A w' értékek kézzel választott demonstrációs értékek, nem az inicializálási eloszlásból húzott minták; a példa a rangkorreláció viselkedését illusztrálja. Az új súlyozás: $\alpha'^1 = (-1+1+2+0{,}5)/4 = 0{,}625$, $\alpha'^2 = (1+0+1-1)/4 = 0{,}25$. A randomizált modell magyarázata:

$$G' = \mathrm{ReLU}\left(0{,}625A^1 + 0{,}25A^2\right) = \begin{pmatrix} 0{,}75 & 1{,}25 \\ 1{,}125 & 2{,}125 \end{pmatrix}.$$

A ReLU itt hatástalan, mert az összeg minden eleme pozitív; a kézi példában a w' súlyokkal is pozitív marad a lineáris kombináció.

A lapított vektor $[0{,}75;\ 1{,}25;\ 1{,}125;\ 2{,}125]$, rangjai $[1;\ 3;\ 2;\ 4]$. A két rangvektor átlaga egyaránt 2,5. A kovariancia:

$$\sum_i (r_i - \bar{r})(s_i - \bar{s}) = (0)(-1{,}5) + (-1{,}5)(0{,}5) + (0)(-0{,}5) + (1{,}5)(1{,}5) = 1{,}5.$$

A szórásnégyzetek: $\sum (r_i - \bar{r})^2 = 4{,}5$ és $\sum (s_i - \bar{s})^2 = 5$, így

$$\rho = \frac{1{,}5}{\sqrt{4{,}5 \cdot 5}} \approx 0{,}316.$$

A randomizálás tehát a hasonlóságot 1-ről 0,316-ra csökkentette ezen a kézzel számolt példán. Ha viszont a randomizált súly az eredeti pozitív skalárszorosa lenne, mondjuk $G' = 2G$, akkor a rangok változatlanok maradnának, és $\rho = 1$ lenne. Ez a fenti hasonlósági korlát konkrét illusztrációja.

A számítást ellenőrző minimális kód a `lab/n138_spearman_example.py` munkakönyvtárból:

```
import numpy as np

def rank_average(v):
    v = np.asarray(v, dtype=np.float64).ravel()
    order = np.argsort(v, kind="mergesort")
    ranks = np.empty_like(order, dtype=np.float64)
    i, n = 0, v.size
    while i < n:
        j = i + 1
        while j < n and v[order[j]] == v[order[i]]:
            j += 1
        ranks[order[i:j]] = (i + j - 1) / 2.0 + 1.0
        i = j
    return ranks

def spearman(a, b):
    ra, rb = rank_average(a), rank_average(b)
    if ra.std(ddof=0) == 0 or rb.std(ddof=0) == 0:
        return float("nan")
    return float(np.corrcoef(ra, rb)[0, 1])

A1 = np.array([[0., 2.], [1., 3.]], dtype=np.float64)
A2 = np.array([[3., 0.], [2., 1.]], dtype=np.float64)
w1 = np.ones((2, 2)); w2 = np.ones((2, 2))
G = w1.mean() * A1 + w2.mean() * A2
w1r = np.array([[-1., 1.], [2., 0.5]])
w2r = np.array([[1., 0.], [1., -1.]])
Gr = w1r.mean() * A1 + w2r.mean() * A2

print("rho(G, Gr) =", round(spearman(G, Gr), 6))
print("rho(G, 2*G) =", round(spearman(G, 2.0 * G), 6))
```

Futtatás: `python lab/n138_spearman_example.py`. A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU): `rho(G, Gr) = 0.316228` és `rho(G, 2*G) = 1.0`; az utóbbi a monoton transzformációra való érzéketlenséget mutatja.

35.2.5 Labor: kis sanity check a saját CNN-re

Az alábbi labor a GlyphLab-adat `test` részének első 20 képén **teljes ($j = L$) paraméter-randomizálást** végez; a kaszkádolt, j-függő görbét ez a labor nem méri. A kód a 24.4. VI. ellenőrző pont: megbízható CNN-training labor modelljét és checkpointját használja: a `TinyCNN` osztályt a `lab/n096_checkpoint.py` modulból importálja (egyetlen forráshely a tanítástól a Grad-CAM-ig), a checkpoint pedig a `lab/best.pt`, amelyet a `cd lab && python n096_checkpoint.py` futás ír. A checkpoint kulcsai: `model` (a `state_dict`), `config` (`name`=`TinyCNN`, `in_ch`, `num_classes`, `input_shape`, `seed`), `epoch`, `val_loss`, `val_acc`, `data_manifest`, `parameter_count`. Ha a checkpoint hiányzik, előbb a 24.4. VI. ellenőrző pont: megbízható CNN-training labort kell lefuttatni. A Grad-CAM célrétege a `features.3`, vagyis a `TinyCNN` második konvolúciója.

Input. `data/images.npy` float32, alak `(360,1,24,24)`; `data/labels.npy` int64; `data/metadata.csv` split oszloppal; a checkpoint a fenti szótár, a súlyok a `model` kulcs alatt. A kód CPU-n fut, seed 20261020, 30 ismétlés.

```
# lab/n138_sanity_check.py -- futtatás a lab/ könyvtárból, a 24.4. lecke laborja után:
# python n138_sanity_check.py --data data --checkpoint best.pt \
#   --seed 20261020 --repeats 30 --max_images 20
import argparse, csv, json
from pathlib import Path
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from n096_checkpoint import TinyCNN  # a 24.4. lecke laborjának modellje és
↪ checkpoint-szerződése


def rank_average(v):
    v = np.asarray(v, dtype=np.float64).ravel()
    order = np.argsort(v, kind="mergesort")
    ranks = np.empty_like(order, dtype=np.float64)
    i, n = 0, v.size
    while i < n:
        j = i + 1
        while j < n and v[order[j]] == v[order[i]]:
            j += 1
        ranks[order[i:j]] = (i + j - 1) / 2.0 + 1.0
        i = j
    return ranks


def spearman(a, b):
    ra, rb = rank_average(a), rank_average(b)
    if ra.std(ddof=0) == 0 or rb.std(ddof=0) == 0:
        return float("nan")
    return float(np.corrcoef(ra, rb)[0, 1])


def sobel_control(x_chw):
    """Sobel-gradiens nagysága egy (1,24,24) CHW képre; kimenet (24,24)."""
    assert x_chw.ndim == 3 and x_chw.shape[0] == 1, x_chw.shape
    x = x_chw[0]
```

```
    kx = np.array([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
    ky = kx.T
    pad = np.pad(x, 1, mode="reflect")
    gx = np.zeros_like(x); gy = np.zeros_like(x)
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            gx[i, j] = (pad[i:i+3, j:j+3] * kx).sum()
            gy[i, j] = (pad[i:i+3, j:j+3] * ky).sum()
    mag = np.sqrt(gx**2 + gy**2)
    return (mag - mag.min()) / (mag.max() - mag.min() + 1e-12)


def compute_gradcam(model, x, c, layer_name="features.3"):
    model.eval(); model.zero_grad()
    acts, grads = {}, {}
    def fwd(m, i, o): acts[layer_name] = o
    def bwd(m, gi, go): grads[layer_name] = go[0]
    handles = []
    for name, mod in model.named_modules():
        if name == layer_name:
            handles.append(mod.register_forward_hook(fwd))
            handles.append(mod.register_full_backward_hook(bwd))
            break
    xx = x.clone().detach().requires_grad_(True)
    logits = model(xx)
    logits[0, c].backward()
    A = acts[layer_name][0]          # C x h x w
    G = grads[layer_name][0]         # C x h x w
    alpha = G.mean(dim=(1, 2))       # C
    cam_map = torch.einsum("q,quv->uv", alpha, A).relu()
    cam_map = (cam_map - cam_map.min()) / (cam_map.max() - cam_map.min() + 1e-12)
    cam = F.interpolate(cam_map[None, None], size=x.shape[-2:],
                        mode="bilinear", align_corners=False)[0, 0]
    for h in handles: h.remove()
    return cam.detach().cpu().numpy()


def randomize_parameters(model, seed, reset_bn_buffers=True):
    """Minden tanult paramétert a modul saját alapinicializálására állít vissza.

    Conv/Linear: reset_parameters() (súly és fan-in alapú bias, pontosan a PyTorch
    alapértelmezése szerint). BatchNorm: reset_parameters() a gamma/beta-t is
    visszaállítja; a futó statisztikák (bufferek) visszaállítása külön döntés.
    Visszaadja a randomizált paraméterek és bufferek nevét a naplóhoz."""
    torch.manual_seed(seed)
    touched = []
    with torch.no_grad():
        for name, mod in model.named_modules():
            if isinstance(mod, (nn.Conv2d, nn.Linear)):
                mod.reset_parameters()
                touched += [f"{name}.{p}" for p, _ in mod.named_parameters(recurse=False)]
            elif isinstance(mod, (nn.BatchNorm1d, nn.BatchNorm2d)):
                running = (mod.running_mean.clone(), mod.running_var.clone())
                mod.reset_parameters()                  # gamma=1, beta=0 és bufferek
                touched += [f"{name}.weight", f"{name}.bias"]
                if reset_bn_buffers:
                    touched += [f"{name}.running_mean", f"{name}.running_var"]
                else:                                   # bufferek megtartása, ha így döntünk
                    mod.running_mean.copy_(running[0]); mod.running_var.copy_(running[1])
    return touched


def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", type=Path, required=True)
    ap.add_argument("--checkpoint", type=Path, required=True)
    ap.add_argument("--seed", type=int, default=20261020)
    ap.add_argument("--repeats", type=int, default=30)
    ap.add_argument("--max_images", type=int, default=20)
    args = ap.parse_args()

    torch.manual_seed(args.seed); np.random.seed(args.seed)
    device = torch.device("cpu")
    images = np.load(args.data / "images.npy")
    labels = np.load(args.data / "labels.npy")
    with open(args.data / "metadata.csv", newline="", encoding="utf-8") as f:
        meta = list(csv.DictReader(f))
    assert len(meta) == len(images), "a metadata és a képtömb hossza eltér"
    assert [int(r["index"]) for r in meta] == list(range(len(meta))), \
        "a metadata indexe nem a 0..N-1 tartománya permutációja"
    idx = [int(r["index"]) for r in meta if r["split"] == "test"][:args.max_images]
    X = torch.from_numpy(images[idx]).float().to(device)  # N,1,24,24
    y = torch.from_numpy(labels[idx]).long()

    base = TinyCNN()
    state = torch.load(args.checkpoint, map_location="cpu", weights_only=True)
    base.load_state_dict(state["model"] if isinstance(state, dict) and "model" in state else
↪   state)
    base.to(device).eval()

    # A célosztály a valódi címke, nem a prediktált; ezt a protokoll rögzíti,
    # hogy a magyarázat a helyes osztályra vonatkozzon.
    base_cams = [compute_gradcam(base, X[i:i+1], int(y[i])) for i in range(len(X))]
    controls = [sobel_control(images[idx[i]]) for i in range(len(X))]   # (1,24,24) -> (24,24)
    assert all(c.shape == (24, 24) for c in controls)
    assert all(c.shape == (24, 24) for c in base_cams)
    n_const = sum(1 for c in base_cams if c.std() == 0)
    cont_rho = [spearman(base_cams[i], controls[i]) for i in range(len(X))]

    rand_rhos, n_nan, n_all_nan = [], 0, 0
    for rep in range(args.repeats):
        m = TinyCNN(); m.load_state_dict(base.state_dict()); m.to(device).eval()
        touched = randomize_parameters(m, args.seed + 1000 * rep)
        if rep == 0:
            print("randomizált:", ", ".join(touched))
            assert set(touched) == {n for n, _ in m.named_parameters()}   # minden tanult
            ↪ paraméter
        vals = np.array([spearman(base_cams[i], compute_gradcam(m, X[i:i+1], int(y[i])))
                         for i in range(len(X))])
        n_nan += int(np.isnan(vals).sum())     # konstans (pl. csupa nulla) randomizált térkép
        if np.isnan(vals).all():
            n_all_nan += 1                     # az ismétlésnek nincs értelmezhető rho-ja
            continue
        rand_rhos.append(float(np.nanmean(vals)))

    checks = {
        "shapes_ok": all(c.shape == (24, 24) for c in base_cams)
                     and all(c.shape == (24, 24) for c in controls),
        "base_cams_finite": bool(all(np.isfinite(c).all() for c in base_cams)),
        "repeats_accounted": len(rand_rhos) + n_all_nan == args.repeats,
    }
    out = {
        "seed": args.seed, "repeats": args.repeats, "n_images": len(X),
        "control_sobel_rho_mean": float(np.nanmean(cont_rho)),
        "randomized_rho_mean": float(np.nanmean(rand_rhos)),
        "randomized_rho_median": float(np.nanmedian(rand_rhos)),
```

```
        "randomized_rho_p05": float(np.nanpercentile(rand_rhos, 5)),
        "randomized_rho_p95": float(np.nanpercentile(rand_rhos, 95)),
        "excluded_constant_cams": n_const,
        "randomized_constant_maps": n_nan,
        "randomized_repeats_without_rho": n_all_nan,
        "checks": checks,
        "status": "completed" if all(checks.values()) else "failed",
    }
    print(json.dumps(out, indent=2, ensure_ascii=False))
    assert out["status"] == "completed", checks


if __name__ == "__main__":
    main()
```

Ellenőrzés és értelmezés. Először a shape-ellenőrzést végezzük el: a Grad-CAM és a Sobel-kontroll egyaránt `(24,24)` alakú. Másodszor a nulla szórású térképek kizárását: ha egy `base_cams` térkép szórása nulla, a megfelelő ρ-t `nan`-ként kizárjuk, és a kizárások számát a jegyzőkönyvbe írjuk. Az értelmezés két, egymástól független összevetés. (i) **Modellfüggetlenségi nullhipotézis:** ha a randomizált modellek magyarázata az eredetivel 1 körül korrelálna, a magyarázat modellfüggetlen lenne. Itt a randomizált ρ átlaga 0,1210, mediánja 0,1934, 95. percentilise 0,5568 (a 30 ismétlés átlagából számolva), mind messze 1 alatt; ez a minta a modellfüggőséggel összeegyeztethető, de leíró összevetés, nem formális hipotézistesz, mert nincs előre rögzített küszöb, és a percentilisek a 30 ismétlés mintavételi bizonytalanságát hordozzák. (ii) **Sobel-kontroll:** az eredeti Grad-CAM és a modellmentes él-detektor korrelációja 0,0784; a randomizált átlag 0,1210 és a 95. percentilis 0,5568 sem alacsonyabb ennél, ezért az előre rögzített szabály szerint ezen a kontrollon a magyarázat nem mutat több modellfüggő tartalmat, mint amennyit a modellmentes kontroll adatfüggése önmagában magyaráz. A Sobel-összehasonlítás tehát nem támasztja alá a modellhez kötöttséget, de nem is cáfolja; a két összevetés különböző kérdésre válaszol. **Futási jegyzőkönyv.** A 24.4. VI. ellenőrző pont: megbízható CNN-training labor `best.pt` checkpointjával, a fenti paranccsal, az eredeti GlyphLab-adaton (CPU, Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, 10 szál) a kimenet seeddel rögzített; az alábbi számok egy futásból származnak. A randomizált paraméterek naplója: `features.0.weight`, `features.0.bias`, `features.3.weight`, `features.3.bias`, `fc.weight`, `fc.bias`, vagyis a modell minden tanult paramétere (a `TinyCNN`-ben nincs BatchNorm). Eredmények: `control_sobel_rho_mean` = 0,0784, `randomized_rho_mean` = 0,1210 (medián 0,1934, a 30 ismétlés átlagából számolt 5–95%-os tartomány [−0,3838; 0,5568]), `excluded_constant_cams` = 0, a 600 randomizált térképből 60 konstans (a ReLU utáni Grad-CAM csupa nulla), és 1 ismétlésnek nincs értelmezhető ρ-ja. A konstans térképek kizárása az átlagból szelekciós torzítást jelent; ezért a konstansok számát külön jelentjük, és az átlag csak a nem konstans térképekre vonatkozik. A 20 kép és 30 ismétlés leíró bizonytalanságot ad; a p05–p95 tartomány nem konfidenciaintervallum. Más könyvtárverzió vagy CPU mellett az utolsó jegyek eltérhetnek; a saját futás JSON-ját kell leadni.

35.2.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a paraméter-randomizálást és a címkerandomizálást, és adja meg, melyik milyen függőséget tesztel. Miért nem helyettesíthető a két kontroll egymással? *Beadandó:* két-két mondat definíció és egy mondat indoklás.

F2 (levezetési, nehézség 2). A fenti kézi példa adataival vezesse le a Spearman-ρ értékét a G és G' térképekre, majd mutassa meg, hogy a $2G'$ térképre ugyanaz az érték adódik. *Beadandó:* a rangvektorok, a kovariancia és a két ρ értéke.

F3 (implementációs, nehézség 2). Futtassa az `n138_sanity_check.py` programot a saját checkpointtal, seeddel `20261020`, 30 ismétléssel. Ellenőrizze, hogy minden Grad-CAM és

Sobel-térkép alakja `(24,24)`, és hogy a kizárt konstans térképek száma nulla. Alkalmazza a *Saliency control* előre rögzített szabályát is: hasonlítsa össze a `randomized_rho_mean` és a `randomized_rho_p95` értékét a `control_sobel_rho_mean`-nel. *Beadandó:* a JSON-kimenet, az ellenőrző shape-teszt utolsó sora és a szabály két feltételének kiértékelése.

F4 (hibakeresési, nehézség 2). Egy hallgató a paraméter-randomizálásnál a `Conv2d` és `Linear` rétegek súlyait újra inicializálja, de a `BatchNorm2d` rétegek `running_mean` és `running_var` állapotát változatlanul hagyja, majd `model.eval()` módban számít Grad-CAM-et. Mi a hiba oka, mi a következménye a randomizált eloszlásra, és mi a javítás? *Beadandó:* ok, következmény, javítás és egy mini ellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tervezze meg a címkerandomizálási kísérletet a GlyphLab-adaton: a tanító címkék permutációja a megfigyelési egység szintjén, a train/val/test split érintetlenül hagyása, tanítási protokoll, baseline és kiértékelési metrika. Mi a fő érvényességi korlát? *Beadandó:* protokolltáblázat és a korlát indoklása.

35.2.7 Megoldások és indoklás

F1. A paraméter-randomizálás a betanított modell rétegenkénti súlyait friss inicializálásra cseréli, és azt teszteli, hogy a magyarázat függ-e a tanult paraméterértékektől az inference során. A címkerandomizálás a tanító címkéket permutálja és újratanít, és azt teszteli, hogy a magyarázat a tanulási jel osztálytartalmától függ-e. A kettő nem cserélhető fel: egy módszer lehet érzékeny a betanított súlyokra, de lehet, hogy a súlyok a képstatisztikától, nem a címkéktől függenek; illetve a címkerandomizálás a tanítási folyamatot, nem az inference-kori paraméterfüggést vizsgálja.

F2. A rangvektorok a kézi példában: $r = [2{,}5;\ 1;\ 2{,}5;\ 4]$, $s = [1;\ 3;\ 2;\ 4]$. Átlaguk 2,5. A kovariancia 1,5, a szórásnégyzetek 4,5 és 5, tehát $\rho = 1{,}5/\sqrt{22{,}5} \approx 0{,}316$. A $2G'$ térkép lapított vektora $[1{,}5;\ 2{,}5;\ 2{,}25;\ 4{,}25]$, rangjai szintén $[1;\ 3;\ 2;\ 4]$, mert a pozitív skálázással a rendezés nem változik, így $\rho(G, 2G') = 0{,}316$. A szélsőeset-ellenőrzés: ha a két térkép konstans lenne, a nevezőben a szórás nulla lenne, és a ρ nem definiált; ezért zárjuk ki a laborban a nulla szórást.

F3. A futtatás után a JSON-ben a `randomized_rho_mean` és a `control_sobel_rho_mean` összevetendő. Az elvárt shape-ellenőrzés:

```
for i in range(len(idx)):
    assert base_cams[i].shape == (24, 24)
    assert sobel_control(images[idx[i]]).shape == (24, 24)
```

A döntési szabály két külön feltételből áll, amelyek különböző kérdésre válaszolnak. (i) A modellfüggetlenség nullhipotézisét a minta akkor nem támogatja, ha a randomizált ρ eloszlása 1 alatt tömörül; a mért 95. percentilis 0,5568, tehát a minta összeegyeztethető a modellfüggőséggel. (ii) A fenti Sobel-szabály szerint a modellmentes kontrollnál erősebb kötöttséghez teljesülnie kellene $\bar{\rho}_{\text{rand}} < \bar{\rho}_{\text{Sobel}}$ és $\rho_{95,\text{rand}} < \bar{\rho}_{\text{Sobel}}$; itt egyik sem teljesül ($0{,}1210 > 0{,}0784$, illetve $0{,}5568 > 0{,}0784$), ezért a Sobel-összehasonlítás nem támasztja alá a modellhez kötöttséget. A két feltétel nem vonható össze egyetlen „sanity check sikerült” ítéletbe. Itt nincs numerikus egyezési tolerancia: a döntés két eloszlási jellemző (átlag és 95. percentilis) és egy előre rögzített referenciaérték összevetése, ezért a döntés bizonytalansága a 30 ismétlés mintavételi ingadozásából és a Sobel-referencia saját szórásából ered, nem lebegőpontos kerekítésből.

F4. Ok: a `BatchNorm2d` értékelő módban a `running_mean` és `running_var` futó statisztikákat használja, nem a minibatch statisztikát; a hallgató csak a tanulható paramétereket randomizálta, a normalizálási állapot a régi eloszlásra maradt kalibrálva. Következmény: a randomizált modell továbbra is a betanított eloszlás skálázását alkalmazza, így a magyarázat

részben a régi modellhez hasonló marad, ami hamisan magas ρ értéket adhat. Javítás: a randomizáló függvénynek a BatchNorm rétegeken `reset_running_stats()`-et kell hívnia, vagy a `running_mean`-t nullára és a `running_var`-t egyre kell állítania. Ellenőrzés: közvetlenül a `reset_running_stats()` után, még újabb *train* forward előtt a `running_mean` pontosan nulla és a `running_var` pontosan egy. Mindkét puffer alakja `(C,)`; `eval()` módban a megfigyelések nem módosítják őket. A labor `randomize_parameters` függvényében a `mod.reset_parameters()` hívás a PyTorch `_NormBase.reset_parameters()` viselkedése szerint a `reset_running_stats()`-et is végrehajtja, ezért a `reset_bn_buffers=True` ág összhangban van ezzel a javítással; a `reset_bn_buffers=False` ág tudatosan visszamásolja a régi puffereket, és pontosan azt a hibát állítja elő, amelyet ez a feladat leír.

F5. Protokoll: a GlyphLab-szerződés (A. A GlyphLab adatkészlet) szerint 180 jelenet van, jelenetenként két nézettel (`view_id` 0 és 1), a split 120/30/30 jelenet train/val/test, osztályonként rétegzetten; a train tehát 120 jelenet és 240 kép, és a jelenet mindkét nézete ugyanabban a splitben van. A train split 120 jelenetének osztálycímkéit rögzített seedű **jelenetszintű** permutációval keverjük, és a jelenet mindkét nézete ugyanazt a permutált címkét kapja; a val és test split érintetlen. Mivel minden jelenethez pontosan két kép tartozik, ez a permutáció a képszintű osztálygyakoriságot is pontosan megőrzi; ha a jelenetek mérete eltérne, ez már nem lenne igaz. A képszintű permutációtól azért térünk el, mert az ugyanazon jelenet két nézetét különböző osztályok alá rendelheti, ami ellentmond a jelenetszintű címkének. A modellt ugyanazzal az architektúrával, optimizerrel, seeddel és epoch-számmal tanítjuk, mint a valódi címkés baseline-t. A metrika a Grad-CAM Spearman-ρ a valódi és a véletlen címkés modell között a test-képeken, összevetve a Sobel-kontroll hasonlóságával. Fő korlát: az alacsony szintű képstatisztika véletlen címkék mellett is beépülhet a korai rétegekbe, ezért a ρ nem várhatóan 0; a kísérlet csak a tanult címkeinformációhoz kötött függést méri, nem a magyarázat abszolút hűségét. A scene-szintű függőség miatt a permutációt jelenetenként, a kiértékelést pedig a változatlan tesztjeleneteken végezzük; a páros nézeteket sem a permutációnál, sem a bizonytalanságbecslésnél nem tekintjük függetlennek. A tanítás a 24.4. labor TinyCNN-szerződése szerint történik: AdamW, $\eta_0 = 10^{-3}$, weight decay 10^{-4}, StepLR (10 epochonként $\gamma = 0{,}5$), batch 16, 20 epoch; öt seeddel (20261020–20261024), CPU-n.

35.2.8 Tipikus tévedés ellenpéldával

Gyakori tévedés, hogy ha egy magyarázó módszer egy adathalmazon és architektúrán átmegy a sanity checken, akkor az minden magyarázatra és feladatra érvényes. Ez a következtetés érvénytelen. Ellenpélda: ugyanaz a Grad-CAM-implementáció a GlyphLab-CNN-en mutathat modellérzékenységet, míg egy pusztán lineáris pooling-fejjel képzett változaton a Spearman-ρ randomizálás után is magas maradhat, mert egy lineáris fejnél a Grad-CAM a fix jellemzőtérkép lineáris függvénye lehet, és a randomizált fej is hasonló térbeli mintázatot adhat. Ezt az állítást külön kísérlettel kell igazolni; a jelen fejezet nem méri. A két eredmény közül egyik sem terjeszthető ki a másik architektúrára, és egyik sem mond semmit egy transzformer attention-térképéről. Az általánosítás csak új kontrollált kísérlettel igazolható.

35.2.9 Önellenőrzés és visszatekintés

Ellenőrizze, hogy a laborban a Grad-CAM célosztálya rögzített-e, a randomizált modellek `eval()` módban futnak-e, és a konstans térképeket a nulla szórás szerinti szűrés kizárja-e. Ha a randomizált eloszlás 95%-os tartománya tartalmazza az 1-et, akkor a modellfüggetlenség nullhipotézisét nem vethetjük el; ez nem a módszer „jóságának” bizonyítéka, hanem a kontroll negatív eredménye.

Rövid visszatekintés: a 31.2. GAN mint minimax probléma GAN-optimuma sem garantálta a véges tanítás konvergenciáját; itt a sanity check sem garantálja a magyarázat hűségét, csak egy cáfolható nullhipotézist tesztel. A 23.2. Kalibráció és döntési küszöb kalibrációs tanulsága szerint egy számszerű kimenet önmagában nem bizonytalansági garancia; itt egy hasonlósági érték önmagában nem hűséggarancia. A 34.2. Kvantálás és hibatovábbterjedés numerikus hibája pedig arra figyelmeztet, hogy a randomizálási kísérletben használt metrika megvalósításának saját numerikus hibája is ellenőrzést igényel.

35.2.10 Rövid összefoglaló

A magyarázat sanity check azt a szükséges feltételt ellenőrzi, hogy a saliency- vagy Grad-CAM-térkép valóban a modelltől függjön. A paraméter-randomizálás a betanított súlyok, a címkerandomizálás a tanulási jel iránti érzékenységet teszteli. A kontroll logikai szerkezete csak cáfolni tud, bizonyítani nem: a magas hasonlóság megőrzése modellfüggetlen módszernél elvileg determinisztikus lenne, ezért a randomizálás utáni erős csökkenés cáfolja a modellfüggetlen (csak a bemenettől függő) magyarázatot, de nem bizonyítja a hűséget. A Spearman-ρ monoton transzformációkra érzéketlen, ezért a kontrollt fix metrika és modellmentes saliency control mellett, architektúránként külön kell jelenteni.

35.2.11 Továbblépés

A 35.3. Perturbáció és adversarial példa egységben a perturbációs irányból vizsgáljuk ugyanezt a kérdést: az FGSM-alapú adversarial példa generálásának [D21] oktatóanyaga szerinti alapváltozatát a saját GlyphLab-modellre alkalmazzuk, és azt kérdezzük, hogyan változik a magyarázat, ha a bemenetet nem a súlyok, hanem egy kis, célzott perturbáció mentén módosítjuk. A most bevezetett hasonlósági metrikák és a randomizálási kontroll ott is a mérési protokoll részét képezik.

35.2.12 Felhasznált források

- [P25] Adebayo, J. et al. (2018). *Sanity Checks for Saliency Maps.* arXiv:1810.03292. https://arxiv.org/abs/1810.03292. A paraméter- és címkerandomizálásos kontroll erre a munkára vezethető vissza; a fejezet fenti formalizmusa a könyv saját megfogalmazása.
- [P23] Selvaraju, R. R. et al. (2016). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* arXiv:1610.02391. https://arxiv.org/abs/1610.02391. A Grad-CAM súlyozási képletét a 35.1. Saliency és Grad-CAM már levezette; itt csak a kontroll tárgyaként használjuk.
- [D21] PyTorch contributors (2026). *Adversarial Example Generation (FGSM).* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/fgsm_tutorial.html. A következő egységhez (35.3. Perturbáció és adversarial példa) jelzett forrás; jelen egységben a Továbblépés bekezdés hivatkozik rá. Hozzáférés: 2026-10-30.

35.3 Perturbáció és adversarial példa

Ebben az alfejezetben azt a kérdést vizsgáljuk, hogy egy már tanított modell bemenetéhez hozzáadott, emberi szemmel alig észrevehető változás miként változtathatja meg a predikciót. A jelenséget először fenyegetési modellként (threat model) formalizáljuk, majd levezetjük az elsőrendű gradiensinformációból építkező FGSM (Fast Gradient Sign Method) lépést. A labor

kizárólag a függelék A. A GlyphLab adatkészlet szakaszában leírt, saját, szintetikus GlyphLab-adatkészleten tanított modellünkön mutat be kis, ellenőrzött perturbációt; külső rendszert nem támadunk.

Induló becslés: ha egy 24×24-es kép minden pixelét legfeljebb $\varepsilon = 0{,}05$ értékkel változtathatjuk meg, vajon megfordul-e az osztályozás? A kérdésre csak a konkrét modell és a konkrét kép ismeretében adható válasz, ezért előbb a fogalmakat rögzítjük.

35.3.1 Fenyegetési modell és megengedett változás

Egy perturbációs vizsgálat tudományos állításként csak akkor értékelhető, ha megmondjuk, ki mit tud, mit módosíthat, és milyen céllal. Ezt a hármast nevezzük **fenyegetési modellnek** (threat model).

Definíció (white-box perturbációs támadó). Az osztályozó $f_\theta : \mathbb{R}^d \to \{0;\ \dots;\ K-1\}$ predikciót ad. A white-box támadó ismeri θ-t, a veszteség bemenet szerinti gradiensét és az y címkét. Célja olyan $x_{adv} = x + \delta$ előállítása, amelyre $f_\theta(x_{adv}) \neq y$, miközben a perturbáció δ egy előre rögzített **norm budget** (norma költségvetés) korláton belül marad. A white-box a modellhez és a gradienshez való hozzáférésről szól; a támadó a súlyokat nem módosítja. A tanítóadathoz vagy az adatfelosztáshoz való hozzáférés hiánya nem a white-box definíció része, hanem a helyi fenyegetési modell külön feltevése (itt: a támadó csak a kész modellt és az aktuális bemenetet használja).

A leggyakoribb budget az ℓ_∞-norma:

$$\|\delta\|_\infty = \max_{1 \le i \le d} |\delta_i| \le \varepsilon.$$

Képeknél ez azt jelenti, hogy egyetlen pixel intenzitása sem változhat ε-nál többel; a GlyphLab képek a $[0;\ 1]$ tartományban vannak, ezért $\varepsilon = 0{,}05$ legfeljebb 5% intenzitásváltozást enged pixelenként. Az ℓ_2-budget, $\|\delta\|_2 \le \varepsilon$, a teljes energiát korlátozza, de nem követeli meg a sima vagy térben szétosztott perturbációt: az egész megengedett normát akár egyetlen pixelre is lehet koncentrálni. A kettő nem felcserélhető, mert más támadóerőt jelent.

Három adattípust különböztetünk meg, mert más-más tudományos kérdésre válaszolnak:

- **clean**: az eredeti, módosítatlan tesztbemenet; a clean accuracy a modell alapvető teljesítményét méri.
- **corrupt**: természetes, nem ellenséges romlás, például szenzorzaj vagy fényességgradiens. A GlyphLab `corrupted_test.npy` fájlja ilyen: ugyanazokat a jelenet/nézet párokat tartalmazza, mint a clean test, csak fizikai jellegű zajjal terhelve.
- **adversarial**: olyan perturbáció, amelyet a modell hibázásának céljával, a modell gradiensének ismeretében választottak, a megengedett budgeten belül.

A corrupt és az adversarial közötti határ a **szándék** és a **modellinformáció felhasználása**: a corrupt zaj nem a döntési határra irányul, az adversarial igen.

35.3.2 Az FGSM levezetése elsőrendű közelítésből

A levezetéshez egyetlen megfigyelésre van szükség: ha a veszteségfüggvény differenciálható, akkor kis perturbáció mellett a veszteség változását az elsőrendű Taylor-közelítés uralja.

Legyen $L : \mathbb{R}^d \to \mathbb{R}$ a veszteségfüggvény (például egyetlen mintára vett cross-entropy), és $g = \nabla_x L(x) \in \mathbb{R}^d$ a bemenet szerinti gradiens. A gradiens alakja megegyezik x alakjával; képeknél ugyanezt NCHW tenzorként $\nabla_X L$ jelöli, de az absztrakt levezetésben oszlopvektorként kezeljük. A Taylor-közelítés:

$$L(x + \delta) = L(x) + g^\top \delta + o(\|\delta\|).$$

A támadó a $g^\top \delta$ lineáris tagot akarja maximalizálni a $\|\delta\|_\infty \le \varepsilon$ feltétel mellett. Mivel a korlát koordinátánként független, a feladat szétesik:

$$g^\top \delta = \sum_{i=1}^{d} g_i \delta_i, \qquad |\delta_i| \le \varepsilon.$$

Minden egyes tagra a maximumot a gradiens előjelét követő lépés adja: ha $g_i > 0$, akkor $\delta_i = \varepsilon$; ha $g_i < 0$, akkor $\delta_i = -\varepsilon$; ha $g_i = 0$, a tag minden megengedett δ_i mellett nulla. Így a maximalizáló perturbáció elemenként:

$$\delta_i^* = \varepsilon \operatorname{sign}(g_i), \qquad \operatorname{sign}(0) := 0.$$

Az $\operatorname{sign}(0) = 0$ itt konvenció, nem a ReLU-féle töréspontí döntés; a gradiens pontosan nulla koordinátáinál a perturbáció nem változtatja az elsőrendű közelítést.

Lemma (FGSM irány elsőrendű optimalitása). Legyen $L : \mathbb{R}^d \to \mathbb{R}$ differenciálható az x pontban, és legyen $g = \nabla_x L(x)$. Ha $\|\delta\|_\infty \le \varepsilon$, akkor

$$g^\top \delta \le \varepsilon \|g\|_1 = \varepsilon \sum_{i=1}^{d} |g_i|.$$

Egyenlőség akkor és csak akkor áll fenn, ha $\delta_i = \varepsilon \operatorname{sign}(g_i)$ minden olyan i-re, ahol $g_i \neq 0$; a nulla gradiensű koordinátákon a perturbáció tetszőleges lehet. Ha $g = 0$, akkor minden megengedett δ ugyanazt a nulla elsőrendű növekedést adja.

Bizonyítás. Minden i-re $g_i \delta_i \le |g_i|\,|\delta_i| \le \varepsilon |g_i|$, mert $|\delta_i| \le \varepsilon$. Összegzés után $g^\top \delta \le \varepsilon \|g\|_1$. A felső korlát elérése: $\delta_i = \varepsilon \operatorname{sign}(g_i)$ esetén $g_i \delta_i = \varepsilon |g_i|$, tehát az összeg egyenlő az $\varepsilon \|g\|_1$ értékkel. A megfordítás: ha egy $g_i \neq 0$ koordinátán δ_i nem az előjelirány, akkor $g_i \delta_i < \varepsilon |g_i|$, ezért az összeg szigorúan kisebb. Ha $g = 0$, akkor $\|g\|_1 = 0$ és minden tag nulla. □

A lemma azt mondja ki, hogy az **elsőrendű közelítésben** az ℓ_∞-budget mellett a veszteséget leggyorsabban növelő irány a gradiens előjele. Ebből kapjuk az FGSM lépést:

$$x_{adv} = x + \varepsilon \operatorname{sign}(\nabla_x L(x)).$$

Képeknél, ahol a bemenet a $[0;\ 1]$ tartományra korlátozott, a perturbált képet a $[0;\ 1]$ intervallumra korlátozzuk: $x_{adv} = \operatorname{clip}(x + \varepsilon \operatorname{sign}(\nabla_x L(x)), 0, 1)$. A clip a megengedett képtartomány betartását szolgálja, és nem része az elsőrendű irány levezetésének. A módszer Goodfellow, Shlens

és Szegedy (2015) munkájából származik [P28]; a fenti levezetés a tankönyv saját, elsőrendű Taylor-közelítésre épülő megfogalmazása.

Fontos korlát: a lemma **nem** állítja, hogy az FGSM lépés globálisan maximalizálja a veszteséget, csak azt, hogy az elsőrendű közelítést maximalizálja. Ha a veszteségfelület erősen görbült, a gradiens előjele félrevezető lehet; erre a 31.3. Diffúziós zajmodell és CNN-denoiser alfejezetben látott diffúziós zajmodell is példa: a zajpredikciós háló a zajszint és a kép nemlineáris függvénye, ezért a veszteségfelület ott is görbült, és az FGSM lineáris közelítése csak lokálisan érvényes.

35.3.3 Eredeti kézi példa

Legyen a modell egy affin osztályozó $\mathbb{R}^3$-ból két logitba:

$$W = \begin{pmatrix} 1 & 0 & -0{,}5 \\ 0{,}2 & -1 & 0{,}4 \end{pmatrix}, \qquad b = (0{,}1;\ -0{,}2)^\top.$$

A bemenet $x = (0{,}5;\ 0{,}2;\ -0{,}3)^\top$, a helyes osztály $y = 0$, a perturbációs budget $\varepsilon = 0{,}1$. A logitok:

$$z = Wx + b = (0{,}75,\ -0{,}42)^\top.$$

A softmax valószínűségek:

$$p_1 = \frac{e^{0{,}75}}{e^{0{,}75} + e^{-0{,}42}} \approx 0{,}7632, \qquad p_2 \approx 0{,}2368.$$

A cross-entropy veszteség egyetlen mintára: $L = -z_y + \log \sum_j e^{z_j} \approx -0{,}75 + \log(2{,}774) \approx 0{,}2703$. A bemenet szerinti gradiens — a 7.2. Softmax és többosztályos cross-entropy alfejezetben levezetett softmax-láncszabályból — $g = W^\top (p - e_y)$, ahol $e_y = (1;\ 0)^\top$:

$$g = W^\top \begin{pmatrix} -0{,}2368 \\ 0{,}2368 \end{pmatrix} \approx (-0{,}1894,\ -0{,}2368,\ 0{,}2131)^\top.$$

Az FGSM perturbáció: $\delta = 0{,}1 \cdot \operatorname{sign}(g) = (-0{,}1,\ -0{,}1,\ 0{,}1)^\top$, tehát

$$x_{adv} = (0{,}4,\ 0{,}1,\ -0{,}2)^\top.$$

Az új logitok $z_{adv} = (0{,}6,\ -0{,}30)^\top$, az új veszteség $L_{adv} \approx 0{,}3412$. A veszteség nőtt, de a predikció még mindig a 0 osztály, mert a margó 0,9 maradt. Az elsőrendű közelítés jósolta növekedés $\varepsilon \|g\|_1 \approx 0{,}1 \cdot 0{,}6400 = 0{,}0640$; a tényleges növekedés 0,0708, ami a magasabb rendű tagok hatását mutatja.

A következő kód ugyanezt a számítást ellenőrzi float64 pontossággal:

```python
# lab/n139/manual_fgsm.py
import numpy as np

x = np.array([0.5, 0.2, -0.3], dtype=np.float64)
W = np.array([[1.0, 0.0, -0.5],
              [0.2, -1.0, 0.4]], dtype=np.float64)
b = np.array([0.1, -0.2], dtype=np.float64)
y = 0
eps = 0.1

def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

def ce_loss(z, y):
    return -z[y] + np.log(np.exp(z).sum())

z = W @ x + b
p = softmax(z)
g = W.T @ (p - np.eye(2)[y])
delta = eps * np.sign(g)
x_adv = x + delta
z_adv = W @ x_adv + b

print("z      =", z)
print("L      =", ce_loss(z, y))
print("g      =", g)
print("delta  =", delta)
print("x_adv  =", x_adv)
print("z_adv  =", z_adv)
print("L_adv  =", ce_loss(z_adv, y))
print("eps*||g||_1 =", eps * np.abs(g).sum())
```

35.3.4 Labor: kontrollperturbáció saját GlyphLab-modellen

Az alábbi labor kizárólag a saját, szintetikus GlyphLab-adatunkon mutat be FGSM-et. Az implementáció alapváltozata a PyTorch FGSM-oktatóanyagának [D21] felépítését követi (a gradiens előjelének vétele és a klipelés), de a modell és az adat a saját GlyphLab. A modell egy kis CNN, amelyet a laborban röviden tanítunk a train spliten; a demonstrációhoz a test split első képét használjuk. A kimenetet a laborban futtatott, rögzített eredményként közöljük; a futtatási környezetet és a seedet a kimenetnél adjuk meg, és a futás a laborban ellenőrizhető.

Futtatási környezet: CPU, float32, seed 20261020, munkakönyvtár `lab/n139/`. Az adatfájlok a szerződés szerint `lab/data/` alatt vannak, ezért a relatív út `../data/`.

```python
# lab/n139/fgsm_demo.py
"""FGSM demonstráció a GlyphLab egy képén. Nem támad külső rendszert."""
import csv

import numpy as np
import torch
import torch.nn as nn

SEED = 20261020
torch.manual_seed(SEED)
np.random.seed(SEED)
DTYPE = torch.float32
DEVICE = torch.device("cpu")

# --- GlyphLab betöltés a függelék A. szakaszának szerződése alapján ---
data_dir = "../data"
```

```
images = np.load(f"{data_dir}/images.npy")    # (360,1,24,24) float32
labels = np.load(f"{data_dir}/labels.npy")    # (360,) int64
with open(f"{data_dir}/metadata.csv", newline="") as f:
    meta = list(csv.DictReader(f))
# A split nem a tömb elejéről származik: a metadata.csv index oszlopa adja a
# teljes képtömb sorindexét, a split oszlop pedig a hozzá tartozó felosztást.
assert list(meta[0]) == ["index", "scene_id", "view_id", "label", "split",
                         "xmin", "ymin", "xmax", "ymax"], list(meta[0])
assert len(meta) == len(images) == len(labels) == 360, (len(meta), images.shape)
assert [int(r["index"]) for r in meta] == list(range(len(meta)))
assert images.min() >= 0.0 and images.max() <= 1.0, (images.min(), images.max())
train_ids = np.array([int(r["index"]) for r in meta if r["split"] == "train"])
test_ids = np.array([int(r["index"]) for r in meta if r["split"] == "test"])
X_train = torch.from_numpy(images[train_ids]).to(DEVICE).to(DTYPE)
y_train = torch.from_numpy(labels[train_ids]).to(DEVICE)
x_test = torch.from_numpy(images[test_ids[:1]]).to(DEVICE).to(DTYPE)
y_test = torch.tensor([labels[test_ids[0]]], device=DEVICE)

class MiniCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d(4),
            nn.Flatten(),
            nn.Linear(8 * 4 * 4, 3)
        )
    def forward(self, x):
        return self.net(x)

model = MiniCNN().to(DEVICE).to(DTYPE)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

B = 32
for epoch in range(2):
    perm = torch.randperm(X_train.shape[0])
    for i in range(0, X_train.shape[0], B):
        idx = perm[i:i+B]
        xb, yb = X_train[idx], y_train[idx]
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()

def fgsm_perturbation(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
    logits = model(x)
    loss = loss_fn(logits, y)
    model.zero_grad()
    loss.backward()
    grad = x.grad.detach()
    delta = eps * torch.sign(grad)
    x_adv = torch.clamp(x.detach() + delta, 0.0, 1.0)
    return x_adv, delta, loss.item()

eps = 0.05
model.eval()
with torch.no_grad():
    clean_logits = model(x_test)
    clean_pred = clean_logits.argmax(dim=1).item()

x_adv, delta, clean_loss = fgsm_perturbation(model, x_test, y_test, eps)
```

```
with torch.no_grad():
    adv_logits = model(x_adv)
    adv_pred = adv_logits.argmax(dim=1).item()
    adv_loss = loss_fn(adv_logits, y_test).item()

# Kontroll 1: véletlen perturbáció azonos L∞ budgettel
torch.manual_seed(SEED + 1)
rnd = torch.empty_like(x_test).uniform_(-eps, eps)
x_rnd = torch.clamp(x_test + rnd, 0.0, 1.0)
with torch.no_grad():
    rnd_pred = model(x_rnd).argmax(dim=1).item()

# Kontroll 2: ellentétes gradiensirány (veszteségcsökkentő lépés)
x_neg = torch.clamp(x_test - delta, 0.0, 1.0)
with torch.no_grad():
    neg_pred = model(x_neg).argmax(dim=1).item()

print(f"clean pred={clean_pred}  loss={clean_loss:.4f}")
print(f"adv   pred={adv_pred}  loss={adv_loss:.4f}")
print(f"random pred={rnd_pred}")
print(f"neg   pred={neg_pred}")
print(f"||delta||_inf={delta.abs().max().item():.4f}")
```

Futtatott kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 10 szál, seed 20261020):

```
clean pred=2  loss=1.1022
adv   pred=2  loss=1.1531
random pred=2
neg   pred=0
||delta||_inf=0.0500
```

A kiválasztott tesztkép valódi címkéje 0, a modell tehát a kiinduláskor téveszt (clean pred=2); az FGSM a veszteséget növelte (1,1022-ről 1,1531-re), de a predikciót nem fordította meg. A véletlen perturbáció sem változtatott a döntésen, az ellentétes gradiensirány viszont a helyes 0 osztályba lépett. Ez jól mutatja, hogy a támadás sikeressége a kiinduló döntéstől is függ, a címkeflip pedig nem garantált; a `||delta||_inf` értéke 0,05, és a clip utáni tényleges elmozdulás maximuma is 0,050000, tehát a $[0;\ 1]$ intervallumra vágás nem avatkozott be.

35.3.5 Robusztusság és mérési korlátok

Definíció (pontonkénti ε-robusztusság). Az f_θ osztályozó az (x, y) ponton ε-robusztus az ℓ_∞-normában, ha minden olyan δ mellett, amelyre $\|\delta\|_\infty \le \varepsilon$, teljesül $f_\theta(x+\delta) = y$. A modell **adversarial accuracy**-ja egy T teszthalmazon az ε-budgetű támadóval szemben:

$$\mathrm{Acc}_{adv}(T, \varepsilon) = \frac{1}{|T|} \sum_{(x,y)\in T} \mathbf{1}\{f_\theta(x_{adv}(x)) = y\},$$

ahol $x_{adv}(x)$ az adott támadó által előállított perturbált bemenet. Ez a mérőszám **attól a konkrét támadóalgoritmustól függ**, amellyel x_{adv}-et előállítjuk; ezért az FGSM-mel mért adversarial accuracy és a PGD-vel mért érték nem ugyanazt a robusztusságot méri. Egy adott támadóval mért Acc_{adv} ezért a robusztusságra nézve csak **felső becslés**: a támadó nem feltétlenül találja meg a legrosszabb perturbációt, így a valódi legrosszabb eset ennél rosszabb lehet.

A három tesztfajta összehasonlítása páros szerkezetű: a GlyphLab clean, corrupt és adversarial tesztje ugyanazokra a jelenet/nézet párokra épül, ezért az accuracy-k különbsége páros összehasonlításként értékelhető. A bizonytalanság számításához a 23.3. Páros összehasonlítás és

csoportszintű intervallum alfejezetben bevezetett class-stratified scene bootstrap a helyes eljárás, nem a képenkénti független mintavétel.

35.3.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázd el, mi a különbség a clean, corrupt és adversarial tesztadatok között a GlyphLab keretében. Mindegyikre adj egy konkrét példát, és mondd meg, melyik milyen tudományos kérdésre válaszol. Elvárt eredmény: legfeljebb fél oldalas, három bekezdésből álló magyarázat, mindhárom esetre explicit példával.

F2 (levezetési, nehézség 2). Bizonyítsd be az ℓ_2-normás változatot: ha $\|\delta\|_2 \le \varepsilon$ és $g = \nabla_x L(x) \neq 0$, akkor az elsőrendű $g^\top \delta$ tagot a $\delta^* = \varepsilon g / \|g\|_2$ perturbáció maximalizálja, és a maximum $\varepsilon \|g\|_2$. Mi történik, ha $g = 0$? Elvárt eredmény: teljes levezetés a Cauchy–Schwarz-egyenlőtlenség alkalmazásával, az egyenlőség feltételének indoklásával.

F3 (implementációs, nehézség 1). Írj Python függvényt `fgsm_affine(W, b, x, y, eps)` néven, amely a fenti affin osztályozón kiszámítja az FGSM perturbációt, és visszaadja az `(x_adv, loss_orig, loss_adv, grad)` négyest. Ellenőrizd a kézi példán a megadott paraméterekkel; a numerikus tolerancia 10^{-4}. Elvárt eredmény: futtatható kód, amely kiírja a négy értéket, és az ellenőrzés sikerét jelzi.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet egy hibát tartalmaz:

```
x_adv = x.clone().detach().requires_grad_(True)
loss = loss_fn(model(x_adv), y)
loss.backward()
delta = eps * torch.sign(x_adv.grad)
x_final = torch.clamp(x_adv - delta, 0.0, 1.0)
```

Itt `x` az eredeti, nem perturbált kép. Mi a hiba, mi a következménye, és hogyan javítanád? Elvárt eredmény: a hiba lokalizálása, a következmény leírása egy-két mondatban, és a javított kódrészlet.

F5 (kísérleti, nehézség 3). Tervezz kísérletet a GlyphLab-on, amely összehasonlítja a clean accuracy-t, az FGSM-perturbált teszt accuracy-ját és a corrupt teszt accuracy-ját. Add meg: a kutatási kérdést, a splitet, a modellt, a baseline-t, a változó tényezőt, a metrikát, a futtatási erőforrást és a következtetés korlátait. Elvárt eredmény: protokollterv az 1.2. Mitől egyetemi szintű egy modellezési állítás? alfejezetben bevezetett kísérleti napló szerkezetében, legalább egy explicit érvényességi korláttal.

35.3.7 Megoldások és indoklás

F1. A clean adat az eredeti, módosítatlan tesztbemenet; például a `../data/images.npy` test splitjének első képe. Azt a kérdést válaszolja meg, hogy a modell általában milyen pontosan osztályoz. A corrupt adat természetes, nem ellenséges romlást tartalmaz; a GlyphLab `corrupted_test.npy` fájlja zajt és fényességgradienst ad ugyanazokhoz a jelenet/nézet párokhoz. Azt vizsgálja, hogy a modell mennyire érzékeny a fizikai jellegű képromlásra. Az adversarial adat a modell gradiensét felhasználva, a hibázás céljával készült perturbáció; például a fenti FGSM-mel generált x_{adv}. Azt vizsgálja, hogy egy white-box támadó milyen könnyen tudja megváltoztatni a döntést. A három mérés három különböző kockázatot jellemez, ezért nem helyettesíthetik egymást.

F2. A Cauchy–Schwarz-egyenlőtlenséggel $g^\top \delta \le \|g\|_2 \|\delta\|_2 \le \varepsilon \|g\|_2$. Az egyenlőség feltétele, hogy g és δ párhuzamosak és azonos irányúak; mivel $\|\delta\|_2 = \varepsilon$, a $\delta = \varepsilon g / \|g\|_2$ választás eléri a korlátot,

és valóban $g^\top \delta = \varepsilon \|g\|_2$. Ha $g = 0$, akkor a felső korlát nulla, és minden megengedett δ nulla elsőrendű növekedést ad; a másodrendű tagok dönthetik el, melyik irány növeli a veszteséget.

F3. A megoldás a kézi példát közvetlenül újrahasznosítja:

```
import numpy as np

def fgsm_affine(W, b, x, y, eps):
    x = np.asarray(x, dtype=np.float64)
    W = np.asarray(W, dtype=np.float64)
    b = np.asarray(b, dtype=np.float64)
    def softmax(z):
        e = np.exp(z - z.max())
        return e / e.sum()
    def ce(z, y):
        return -z[y] + np.log(np.exp(z).sum())
    z = W @ x + b
    p = softmax(z)
    g = W.T @ (p - np.eye(W.shape[0])[y])
    delta = eps * np.sign(g)
    x_adv = x + delta
    return x_adv, ce(z, y), ce(W @ x_adv + b, y), g

W = np.array([[1.0, 0.0, -0.5], [0.2, -1.0, 0.4]])
b = np.array([0.1, -0.2])
x = np.array([0.5, 0.2, -0.3])
y = 0
eps = 0.1
x_adv, L0, L1, g = fgsm_affine(W, b, x, y, eps)
print(x_adv, L0, L1, g)
# Ellenőrzés: L1 > L0, a 10^-4-es tolerancián belül L0=0.2703, L1=0.3412,
# és ||x_adv - x||_inf <= eps + 1e-12
assert L1 > L0
assert np.abs(x_adv - x).max() <= eps + 1e-12
assert abs(L0 - 0.2703) < 1e-4
assert abs(L1 - 0.3412) < 1e-4
assert np.allclose(g, [-0.1895, -0.2369, 0.2132], atol=1e-4)
```

Az `x_adv` várható értéke $(0{,}4;\ 0{,}1;\ -0{,}2)$, a veszteségek 0,2703 és 0,3412 körül, a gradiens $(-0{,}1895;\ -0{,}2369;\ 0{,}2132)$ körül, a tolerancia 10^{-4}.

F4. A hiba az előjel: a kód a `delta = eps * sign(grad)` mennyiséget **kivonja** (`x_final = clamp(x_adv - delta, 0, 1)`). Mivel `x_adv` az `x` klónja, ez az $x - \varepsilon\ \mathrm{sign}(\nabla_x L)$ pontot adja, azaz a gradiens **ellenkező** irányába lép, holott az FGSM a veszteség növeléséhez a gradiens előjelével azonos irányba lép: $x + \varepsilon\ \mathrm{sign}(\nabla_x L)$. A kapott kép így nem támadó példa, hanem „barátságos” perturbáció: az elsőrendű veszteségváltozás $-\varepsilon \|\nabla_x L\|_1 < 0$, tehát a veszteség csökken. Következmény: az így mért accuracy nem a támadónak való kitettséget méri, hanem egy enyhe képjavító hatást. A javítás az előjel megfordítása, majd két ellenőrzés: a perturbált kép vesztesége nagyobb az eredetinél, és a perturbáció terjedelme legfeljebb ε (utóbbi a klipelés miatt csak akkor teljesül, ha a kép belsejében maradunk):

```
x_grad = x.clone().detach().requires_grad_(True)
loss = loss_fn(model(x_grad), y)
loss.backward()
delta = eps * torch.sign(x_grad.grad)
x_adv = torch.clamp(x + delta, 0.0, 1.0)

with torch.no_grad():
    l_orig = loss_fn(model(x), y).item()
    l_adv = loss_fn(model(x_adv), y).item()
assert l_adv > l_orig                          # a perturbáció növeli a veszteséget
assert (x_adv - x).abs().max().item() <= eps + 1e-6
```

F5. Lehetséges terv: Kérdés: mekkora a különbség a clean, corrupt és FGSM-perturbált test accuracy között a GlyphLab modellen? Split: a rögzített train/val/test; a modellt csak trainen tanítjuk, a val a modellválasztásra szolgál, a test csak a végén kerül kiértékelésre — a corrupt és adversarial teszt ugyanazon a test spliten. A labor MiniCNN-je nem tartalmaz korai leállítást; a tervben ezt a lépést külön rögzíteni kell. Modell: a laborban használt MiniCNN. Baseline: a clean test accuracy. Változó tényező: a bemenet típusa (clean, corrupt, FGSM $\varepsilon = 0{,}05$). Metrika: scene-egységen átlagolt accuracy, a bizonytalanságot class-stratified scene bootstrap adja. Erőforrás: CPU, float32, seed 20261020. Következtetési korlát: az eredmény a szintetikus GlyphLab-eloszlásra és a választott FGSM-támadóra vonatkozik; nem állít semmit a természetes képek robusztusságáról, és egyetlen FGSM-eredmény nem bizonyítja a modell ellenállását erősebb, például iterált támadókkal szemben.

35.3.8 Tipikus tévedés és ellenpélda

A leggyakoribb tévedés: **egy FGSM-teszten elért siker nem minden támadóval szembeni robusztussági bizonyítás.** Az FGSM egyetlen lépést tesz a gradiens előjelirányában; ha a veszteségfelület nem lineáris, egy iterált, vetített gradiensű módszer (például PGD [P29]) ugyanazon budget mellett lényegesen nagyobb veszteségnövekedést találhat. A PGD-t véletlen kezdőponttal szokás futtatni, mert a nulla gradiensű pontból induló iteráció ott is elakadhat. Ellenpélda: tekintsük az $L(x) = x_1^2 + 100x_2^2$ veszteségfüggvényt az $x = (0;\ 0)$ pontban, $\varepsilon = 0{,}1$ mellett. Itt $g = (0;\ 0)$, tehát az FGSM perturbáció nulla, és a veszteség nem nő. Egy támadó azonban a $x_2 = 0{,}1$ irányt választva $L = 1$ veszteséget ér el. Az FGSM tehát nem találta meg a támadási irányt, miközben létezik hatékony perturbáció. A modell értékelésekor ezért mindig meg kell nevezni a támadóalgoritmust, és az egyetlen támadóval mért adversarial accuracy nem általános robusztussági garancia.

35.3.9 Önellenőrzés és visszatekintés

Rövid önellenőrzés: (1) A gradiens alakja megegyezik-e a bemenet alakjával? (2) A perturbáció minden koordinátája legfeljebb ε abszolút értékű-e? (3) A clip után a kép a $[0;\ 1]$ tartományban marad-e? (4) A megfogalmazott állítás az elsőrendű közelítésre vagy a tényleges veszteségre vonatkozik? Visszatekintés a 35.2. Magyarázat sanity check alfejezetből: a sanity check arra figyelmeztet, hogy a gradiensalapú jelzések akkor is változhatnak, ha a modell nem használja a címkét; az FGSM-nél ez azért fontos, mert a gradiens informatív voltát ugyanúgy ellenőrizni kell. A 34.3. Pruning és struktúra alfejezetből: a strukturált pruning a csatornák eltávolításával megváltoztathatja a modell függvényét és ezzel a döntési határt, ezért a perturbációs vizsgálatot mindig a végleges, nem pedig a közbenső modellen kell elvégezni.

Kulcs. (1) Igen, a gradiens alakja megegyezik a bemenet alakjával. (2) Igen, minden koordinátára $|\delta_i| \le \varepsilon$ teljesül. (3) Igen, a clip után a kép a $[0;\ 1]$ tartományban marad. (4) Az állításnak meg kell neveznie, hogy elsőrendű közelítésről vagy a tényleges veszteségről szól; az FGSM-lépés növekedése csak a linearizált modellben garantált.

35.3.10 Rövid összefoglaló

Az adversarial példa olyan bemenet, amelyet a modell hibázásának céljával, a megengedett perturbációs budgeten belül választottak. A white-box fenyegetési modell rögzíti, hogy a támadó ismeri a paramétereket és a gradienst. Az FGSM az elsőrendű Taylor-közelítésből vezethető le: az ℓ_∞-korlát mellett a veszteséget növelő irány a gradiens előjele, amelynek maximális elsőrendű növekedése $\varepsilon\|\nabla_x L\|_1$. A kézi példa ellenőrizte a (2) képletből származó gradienst és az FGSM

lépés irányát; a veszteség növekedése a magasabb rendű tagok miatt nem egyezik pontosan az elsőrendű becsléssel. A labor saját GlyphLab-modellen mutatott kontrollperturbációt. A clean, corrupt és adversarial accuracy más kockázatot mér; egyetlen FGSM-eredmény nem bizonyít általános robusztusságot.

35.3.11 Továbblépés

A 35.4. Korlátok, eredet és döntési felelősség egységben ezeket a perturbációs eredményeket a korlátok, az adateredet és a döntési felelősség keretébe helyezzük: milyen feltételek mellett lehet egy modellt magas tétű alkalmazásban használni, és mit kell dokumentálni az ismert támadási felületekről.

35.3.12 Források

- [P28] Goodfellow, I. J., Shlens, J., Szegedy, C. (2015). *Explaining and Harnessing Adversarial Examples*. ICLR 2015. arXiv:1412.6572. https://arxiv.org/abs/1412.6572.
- [P29] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A. (2018). *Towards Deep Learning Models Resistant to Adversarial Attacks*. ICLR 2018. arXiv:1706.06083. https://arxiv.org/abs/1706.06083.
- [D21] PyTorch contributors (2026). *Adversarial Example Generation (FGSM)*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/fgsm_tutorial.html. A labor implementációjának alapváltozata ezt a felépítést követi. Hozzáférés: 2026-10-30.

35.4 Korlátok, eredet és döntési felelősség

35.4.1 Cél, előismeret és induló kérdés

A 35.2. Magyarázat sanity check egységben a paraméter- és címkerandomizálás megmutatta, hogy egy Grad-CAM vagy saliency hőtérkép önmagában nem bizonyítja a modell mechanizmusát. Ebben az alfejezetben a kérdést kiterjesztjük: **mely feltételek mellett szabad a tanított modell kimenetét valódi döntésben felhasználni?** A cél annak a három fogalomnak a pontos elválasztása, amelyet a mérnöki gyakorlat gyakran összemos: a mért benchmarkpontosság, a felhasználási körre vonatkozó állítás és a döntési felelősség.

Induló becslés: egy csapat a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab clean tesztjén 98%-os osztályozási pontosságot mér. Jogos-e ebből azt állítani, hogy a modell egy valódi beléptetési terminálban is 98%-ban jól dönt? A válasz a becslés szerint igen, a valóságban nem. A fejezet ezt a különbséget teszi operatívvá.

35.4.2 Definíciók: modellkártya, adathatár, privacy, licenc, nagy tét és emberi felülvizsgálat

Legyen $f : \mathcal{X} \to \mathcal{Y}$ a tanított modell. A leggyakoribb hiba az, hogy az $\mathrm{acc}_D(f)$ mérőszámot a D adathalmazt leíró eloszlástól elszakítjuk. Az alábbi fogalmak mind a D-t és a felhasználást rögzítik.

Felhasználási kör (*use context*). Az ötös $(A, \mathcal{X}, \mathcal{Y}, \delta, C)$, ahol A az automatizált döntés leírása, $\mathcal{X}$ a bemenet, $\mathcal{Y}$ a kimenet, δ a rendszer által a kimenetre végrehajtott beavatkozás, C pedig a hibás döntés következménye. A GlyphLab tanulási környezetben például $A =$ „orientációfelismerés

gyakorlófeladat”, δ = „naplóbejegyzés”, C = „gyakorlópontvesztés”. Ugyanaz a modell más δ mellett más felhasználási kör, és más validációt igényel.

Modellkártya (*model card*). Strukturált, verziózott dokumentum, amely legalább a következőket rögzíti: rendeltetés és felhasználási kör; bemeneti specifikáció (alak, dtype, intenzitástartomány, kamerafeltétel); kimeneti specifikáció és a kimenetből képzett döntési szabály; tanító/validáló/teszt adat eredete, splitprotokollja és jelenetszintű csoportosítása; mért metrikák a szeletbontásokkal; ismert korlátok és tiltott felhasználások; a súly- és kódváltozat lenyomata; karbantartási és újravalidálási feltétel.

Adathatár (*data boundary*). Az a $\mathcal{D} \subseteq \mathcal{X}$ tartomány és a rajta értelmezett $P_{\mathcal{D}}$ eloszlás, illetve eloszláscsalád, amelyre az empirikus teljesítmény állítást igazoltuk. Formálisan: ha a teszt $D = \{(x_i, y_i)\}$ a $P_{\mathcal{D}}$ eloszlásból származik, akkor $\mathrm{acc}_D(f)$ a $P_{\mathcal{D}}$-re vonatkozó becslés. A $P_{\mathcal{D}}$-n kívüli pontokra semmilyen numerikus garanciát nem ad a teszt. A GlyphLab esetében $\mathcal{D}$ a 24×24, egy csatornás, $[0;\ 1]$-intenzitású szintetikus alakzatképek eloszlása, osztályonként kiegyensúlyozott mintavétellel.

Privacy és licenc. A privacy az adatvédelmi követelmények gyűjtőneve: milyen azonosítót tárol a rendszer, meddig, kinek van hozzáférése, mi a törlési út. Az adatminimalizálás a privacy egyik implementációja: csak a célhoz szükséges legkisebb felbontást, legrövidebb megőrzést és legkevesebb azonosítót használjuk. A licenc ezzel szemben jogi felhasználási engedély: a tanítóadat, a kód és a mentett súly licencét külön kell nyilvántartani, mert ezek eltérő jogi státuszúak.

Nagy tétű alkalmazás (*high-stakes*). Olyan döntés, amelyben a modell hibája alapjogot, egészséget, szabadságot, biztonságot vagy jelentős vagyoni érdeket érint. A gyakorlati határjelölés az C következmény függvénye: ha a hiba visszafordíthatatlan vagy személyre szóló, az alkalmazás nagy tétű.

Emberi felülvizsgálat (*human review*). Olyan folyamat, amelyben a modell kimenetét egy emberi döntéshozó előre rögzített szabály szerint felülvizsgálhatja. A felülvizsgálat maga is mérendő folyamat: ugyanúgy van saját accuracy-ja, késleltetése és döntési torzítása, mint a modellnek. A „van ember a hurokban” önmagában nem bizonyíték, csak akkor, ha a felülvizsgálati pontosságot és a beavatkozási arányt mértük.

35.4.3 A benchmarkeredményből nem következik az új eloszláson mért teljesítmény: formális ellenpélda

A levezetési követelményt egy **ellenpéldával** teljesítjük: megmutatjuk, hogy két modell azonos clean tesztpontossága nem határozza meg az eloszlásbeli eltolódás (shift) utáni pontosságukat. Az állítás, amelyet cáfolunk, a következő.

Téves állítás. Legyen D teszt és T egy labelmegőrző (címkemegőrző) transzformáció. Ha $\mathrm{acc}_D(f_1) = \mathrm{acc}_D(f_2)$, akkor $\mathrm{acc}_{T(D)}(f_1) = \mathrm{acc}_{T(D)}(f_2)$ is teljesül.

Ellenpélda. Legyen a bemenet $\mathcal{X} = \{0;\ 1\}^{1\times 2\times 2}$, azaz egy csatornás, 2×2-es bináris kép; a cél $\mathcal{Y} = \{0;\ 1\}$, ahol 0 a függőleges, 1 a vízszintes osztály. A teszt két eleme

$$x^{(1)} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}, \quad y^{(1)} = 0, \qquad x^{(2)} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}, \quad y^{(2)} = 1.$$

Minden x képre legyen $s(x) = \sum_i (a_{i0} - a_{i1})$, ahol a_{ij} a 2×2 mátrix eleme. Két modellt definiálunk:

$$f_1(x) = 0 \text{ ha } |s(x)| > 0, \text{ különben } 1, \qquad f_2(x) = 0 \text{ ha } s(x) > 0, \text{ különben } 1.$$

A clean teszt levezetése:

- $x^{(1)}$-re $s = 1 - 0 + 1 - 0 = 2$, tehát $|s| > 0$ és $s > 0$, ezért $f_1 = f_2 = 0$, ami megegyezik a 0 címkével.
- $x^{(2)}$-re $s = 0 - 0 + 1 - 1 = 0$, tehát mindkét modell 1-et ad, ami megegyezik a 1 címkével.

Így $\mathrm{acc}_D(f_1) = \mathrm{acc}_D(f_2) = 1$. Legyen $T(x) = 1 - x$ elemenként, az intenzitásinverzió. Ez nem geometriai transzformáció, hanem pixelérték-transzformáció; a két példaképen a vonal orientációja változatlan marad, ezért T labelmegőrző ezekre a képekre. A labelmegőrzést általában a teljes eloszlásra kell definiálni: T labelmegőrző, ha minden (x, y) párra az y címke a $T(x)$-re is érvényes. A shiftelt képek:

$$T(x^{(1)}) = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}, \quad T(x^{(2)}) = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}.$$

A $T(x^{(1)})$ képre $s = 0 - 1 + 0 - 1 = -2$, ezért $|s| = 2 > 0$, így $f_1 = 0$ (helyes), de $s \not> 0$, ezért $f_2 = 1$ (hiba). A $T(x^{(2)})$ képre $s = 0$, így mindkét modell 1-et ad (helyes). Tehát $\mathrm{acc}_{T(D)}(f_1) = 1$, míg $\mathrm{acc}_{T(D)}(f_2) = 0{,}5$. Az azonos clean pontosság ellenére a shiftelt pontosságok eltérnek, a téves állítás cáfolva.

A cáfolat módszertani tanulsága: a clean teszt pontossága csak a $P_{\mathcal{D}}$ eloszlásra vonatkozó becslés. A T shift új eloszlást definiál; annak méréséhez új, előre rögzített teszt és új metrika kell. A GlyphLab saját corrupt tesztje is ilyen: a fényességgradiens és a hozzáadott zaj egyetlen konkrét shiftosztály, nem a „valós világ” reprezentánsa.

35.4.4 Kézi példa ellenőrzése minimális kóddal

A következő kód a fenti 2×2-es példát reprodukálja, független implementációként. A kimenet a képletből számított, és a kód futtatása ezt adja.

```
# labs/n140/line_shift_counterexample.py
# Munkakönyvtár: project/labs/n140/
import numpy as np

def pred_f1(a):
    s = float(a[0,0] + a[1,0] - a[0,1] - a[1,1])
    return 0 if abs(s) > 0 else 1

def pred_f2(a):
    s = float(a[0,0] + a[1,0] - a[0,1] - a[1,1])
    return 0 if s > 0 else 1

x1 = np.array([[1, 0], [1, 0]], dtype=np.float64)
x2 = np.array([[0, 0], [1, 1]], dtype=np.float64)
D = [(x1, 0), (x2, 1)]
T = lambda x: 1.0 - x

for name, f in [("f1", pred_f1), ("f2", pred_f2)]:
    acc_clean = np.mean([f(x) == y for x, y in D])
    acc_shift = np.mean([f(T(x)) == y for x, y in D])
    print(name, "clean=", acc_clean, "shift=", acc_shift)
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU; a számítás determinisztikus, seed nem kell):

```
f1 clean= 1.0 shift= 1.0
f2 clean= 1.0 shift= 0.5
```

A dtype `float64`, a device a CPU, seed nincs, mert a számítás determinisztikus. Az alakellenőrzés: mindkét kép 2×2, az egy csatornás $1 \times 2 \times 2$ tenzorból a csatornaindexet elhagytuk, ez a példában jelölt konvenció. A bináris $\mathcal{X}$ elemeit a kód `float64`-re alakítja, hogy az intenzitásinverzió $(1-x)$ és a valós értékű számítás egységes legyen; a 0/1 értékeket a konstrukció biztosítja.

35.4.5 Labor: korlátjegyzék és adatminimalizálási terv a PageTurn fiktív alkalmazáshoz

Az alkalmazás **PageTurn**: fiktív szolgáltatás, amely beszkennelt formanyomtatványok oldalait automatikusan elforgatja. A modell a GlyphLab tanuló CNN, 24×24 grayscale bemenettel. A labor célja **nem** a modell leszállítása, hanem két dokumentum előállítása, mielőtt bármilyen valós használatba kerülne. A PageTurn nem klinikai, jogi vagy személyazonosító alkalmazás; erről a korlátjegyzékben explicit tiltó mondat szerepel.

Korlátjegyzék, 1. vázlat.

1. Bemeneti adathatár: a modell 24×24, egy csatornás, `float32`, [0; 1]-intenzitású, szintetikus alakzatképekre validált. A PageTurn 300 dpi szkennelt, szöveges, színes oldalakat adna; ez kívül esik a határon, amíg külön kísérlet nem méri.
2. Osztályprior (az osztályok előzetes aránya): a tanítás 1/3–1/3–1/3 arányú. A valós forgalmi eloszlás ismeretlen; a balanced accuracy nem vihető át.
3. Korrupciós határ: a GlyphLab corrupt tesztje zaj és fényességgradiens. A szkenner elmosódása, ferde világítás, részleges takarás más shift, ezért újra kell mérni.
4. Geometriai szimmetria: a modell nem 90°-forgás-invariáns; a diagonal osztály csak az egyik ferde irányt tartalmazza. A forgatott bemenet címkéje nem ugyanaz.
5. Konfidenciaértelmezés: a softmax kimenet nem garantáltan kalibrált valószínűségi becslés; küszöbalapú döntési szabály validálása külön feladat.
6. Származás: a súly a labor saját checkpointja; a state_dict SHA-256 lenyomata és a training recept verziója a modellkártya része.
7. Nagy tétű tiltás: nem használható személyazonosításra, klinikai, jogi vagy pénzügyi döntéshozatalra; a PageTurn kimenete csak tájékoztató javaslat, a végső elfogadás emberi.

Adatminimalizálási terv, 1. vázlat.

1. A kliens oldal a beolvasott oldalt 64×64-es grayscale, simított, `float32` képpé alakítja; az eredeti felbontású kép nem hagyja el a kliens gépet.
2. A szolgáltatás a 64×64-es képet a modell bemenetére 24×24-re méretezi (downscaling), és ezt a lépést a modellkártya rögzíti.
3. A szolgáltatás csak a döntést (0/1/2) és a logit hármasát tárolja, 30 napig; a kép a válasz után törlődik. A kliens oldali 64×64-es kép is személyes adat lehet, ezért a kliens gépen is törölni kell a válasz után, és ezt a privacy tájékoztatóban rögzíteni kell.
4. A munkamenet azonosítója véletlen UUID; a képhez személyes azonosító nem kapcsolódik.
5. A valós PageTurn képek nem használhatók tanításra, amíg nincs célhoz kötött adatkezelési tájékoztató és törlési út.

A korlátjegyzék accuracy-bontásának ellenőrzésére szolgáló sablon, feltételezett pilotértékekkel; a kimenet számított, nem mért:

```python
# labs/n140/page_turn_limit_check.py
import numpy as np

preds = np.array([0, 0, 1, 1, 2, 2, 0, 1], dtype=np.int64)
labels = np.array([0, 0, 1, 0, 2, 2, 0, 1], dtype=np.int64)
split_ids = np.array(["clean", "clean", "clean", "clean",
                      "corrupt", "corrupt", "corrupt", "corrupt"], dtype=str)

def report_by_split(preds, labels, split_ids, class_names):
    out = {}
    for split in np.unique(split_ids):
        m = split_ids == split
        out[str(split)] = {
            "n": int(m.sum()),
            "accuracy": float(np.mean(preds[m] == labels[m])),
            "class": {
                class_names[c]: float(np.mean(preds[m & (labels == c)] == labels[m & (labels ==
↪  c)]))
                for c in sorted(set(labels[m].tolist()))
                if (m & (labels == c)).sum() > 0
            }
        }
    return out

print(report_by_split(preds, labels, split_ids,
                      {0: "vertical", 1: "horizontal", 2: "diagonal"}))
```

Futtatott kimenet a megadott, feltételezett `preds`/`labels` tömbökkel (Python 3.12.3, NumPy 2.5.4, CPU): `clean` `n=4`, `accuracy=0.75`, osztálybontás `vertical=0.6667`, `horizontal=1.0`; `corrupt` `n=4`, `accuracy=1.0`. Az ellenőrzés: a pontos törtekkel a clean súlyozott osztálypontosság $(3 \cdot \frac{2}{3} + 1 \cdot 1{,}0)/4 = 0{,}75$, ami megegyezik az összesített értékkel; a kimenetben megjelenő 0,6667 kerekített érték, ezért a kézi ellenőrzést a pontos törtekkel kell elvégezni. A futás parancsa a fenti könyvtárból `python page_turn_limit_check.py`. A kimenetet nem szabad éles eredménynek nevezni, mert a predikciók feltételezett értékek: a futtatás a számítási formát igazolja, nem a modell viselkedését.

35.4.6 Feladatok

F1 – fogalmi, nehézség 1. Adja meg a data boundary definícióját, majd két konkrét példát arra, ahogyan a PageTurn megsértené, ha a GlyphLab 98%-os clean tesztpontosságát változatlanul átmásolná a modellkártyájába. Beadandó: definíció és két, a labor 1. és 3. pontjától különböző sértés.

F2 – levezetési, nehézség 2. Az alfejezet 2×2-es példájának mintájára konstruáljon egy új (f_3, f_4) modellpárt és egy új labelmegőrző transzformációt, amelyre a clean tesztpontosságok egyenlők, a shift utáni pontosságok viszont eltérnek. Adja meg a képeket, a szabályokat és a pontosságokat. Beadandó: teljes levezetés, a cáfolt állítás kimondása.

F3 – implementációs, nehézség 2. Írjon teljes Python függvényt, amely a GlyphLab teszt predikcióiból clean/corrupt és osztályonkénti accuracy-t számol a metadata `split` oszlopa alapján; a corrupt teszt a clean teszt képeinek ugyanabban a sorrendben vett változata. A függvény inputja `preds`, `labels`, `split_ids`, mind `numpy` tömb; a kimenet a fenti laborformátumú `dict`. Ellenőrizze a függvényt egy 6 elemű mini példán (3 clean és 3 corrupt elem). Beadandó: kód, dtype-ok, egy kiszámított ellenőrző eset.

F4 – hibakeresési, nehézség 1. Az alábbi indoklás hibás: „A modell clean teszt accuracy-ja 0,98, ezért a valós dokumentumokon is 0,98 pontossággal működik, és az emberi felülvizsgálatra

nincs szükség.” Azonosítsa a három elkülönült hibát, és írja át az állítást korrekt formába. Beadandó: három megnevezett hiba és a javított állítás.

F5 – kísérlettervezési/transzfer, nehézség 3. Tervezzen emberi felülvizsgálati protokollt a PageTurn számára: milyen döntési szabály mellett kér a rendszer emberi jóváhagyást, milyen metrikákat mér a pilotban, mekkora a pilot, és mi a kontroll. Rögzítse a következtetési határt. Beadandó: protokoll, baseline, metrikák, érvényességi korlát.

35.4.7 Megoldások és indoklás

F1. A data boundary az a $\mathcal{D} \subseteq \mathcal{X}$ tartomány és a rajta értelmezett $P_{\mathcal{D}}$ eloszlás, amelyre az empirikus teszt replikálható mintavételi eljárással mér; a 0,98 becslés csak $P_{\mathcal{D}}$-re érvényes. Két, a labor 1. és 3. pontjától különböző sértés a PageTurnben: (a) a 90°-kal elforgatott beszkennelt oldal; a modell nem 90°-forgatás-invariáns, a GlyphLab diagonal osztálya csak az egyik ferde irányt tartalmazza, ezért a forgatott bemenet címkéje nem ugyanaz, és a 0,98 nem vonatkozik rá. (b) A valós osztályprior változása: a GlyphLab tanítás 1/3–1/3–1/3 arányú, a valós dokumentumokon az orientációk aránya ettől eltér, ezért a kiegyensúlyozott accuracy nem adja meg a valós accuracy-t, és a döntési küszöböt a valós prevalencián kell újramérni.

F2. Legyen D ugyanaz a két kép, mint a leckében. Definiáljuk

$$f_3(x) = 0 \text{ ha az } (a_{00}, a_{10}) \text{ és } (a_{01}, a_{11}) \text{ oszloppárok homogének, különben 1,}$$

$$f_4(x) = 0 \text{ ha } s(x) > 0, \text{ különben 1.}$$

A D-n f_3 és f_4 is 100%-ot ér el: $x^{(1)}$ oszlopai homogének, $f_3 = 0$; $x^{(1)}$-nél $s = 2 > 0$, $f_4 = 0$. $x^{(2)}$ oszlopai nem homogének, $f_3 = 1$; $x^{(2)}$-nél $s = 0$, $f_4 = 1$. Legyen T a vízszintes tükrözés (bal–jobb), azaz az oszlopok felcserélése. Ez a példa két képére labelmegőrző: a vonal orientációja változatlan marad; általában a labelmegőrzést a teljes adateloszláson kell definiálni. A tükrözés az oszlopok homogenitását megtartja, ezért f_3 pontossága $T(D)$-n is 100%. A $T(x^{(1)})$ képre $s = -2$, ezért f_4 1-et ad a 0 címke helyett, hiba; $T(x^{(2)})$ képre $s = 0$, helyes. Így f_4 pontossága 50%. Az azonos clean 100% nem határozza meg a shift pontosságot.

F3. A kód:

```python
# labs/n140/report_by_split.py
# Munkakönyvtár: project/labs/n140/
import numpy as np

def report_by_split(preds, labels, split_ids, class_names):
    assert preds.shape == labels.shape == split_ids.shape
    assert preds.dtype in (np.int64, np.int32)
    out = {}
    for split in np.unique(split_ids):
        m = split_ids == split
        out[str(split)] = {
            "n": int(m.sum()),
            "accuracy": float(np.mean(preds[m] == labels[m])),
            "class": {}
        }
        for c in sorted(set(labels[m].tolist())):
            mm = m & (labels == c)
            if mm.sum() > 0:
                out[str(split)]["class"][class_names[c]] = float(
                    np.mean(preds[mm] == labels[mm]))
```

```
    return out

# Ellenőrzés a mini példán:
preds = np.array([0, 0, 1, 0, 1, 0], dtype=np.int64)
labels = np.array([0, 0, 1, 1, 1, 1], dtype=np.int64)
split_ids = np.array(["clean", "clean", "clean",
                      "corrupt", "corrupt", "corrupt"], dtype=str)
print(report_by_split(preds, labels, split_ids,
                      {0: "vertical", 1: "horizontal"}))
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU): `clean` accuracy 3/3 = 1,0 (`vertical=1.0`, `horizontal=1.0`), `corrupt` accuracy 1/3 = 0,3333 (`horizontal=0.3333`); a class-bontások konzisztensek. A függvény a GlyphLab esetén külön hívandó: a clean 60 tesztképnél a `split_ids` minden eleme `clean`, a corrupt 60 képnél minden eleme `corrupt`; a `metadata.csv` `split` oszlopa mindkettőre `test`, ezért a clean/corrupt címkét a hívó adja. A párosítás a metadata test sorainak sorrendje: a `corrupted_test.npy` j-edik képe a clean test j-edik képének párja, így a két 60 elemű kiértékelés ugyanazokat a jelenet/nézet párokat használja.

F4. A három, egymástól elkülönülő hiba: (1) **adathatár-sértés:** a benchmark eloszlásból következtet a valós eloszlásra, holott a PageTurn színes, 300 dpi szkennelt oldalai és a GlyphLab 24×24-es szintetikus képei különböző eloszlások; (2) **metrika- és priorhiba:** a 0,98 kiegyensúlyozott teszten mért accuracy, a valós osztályarány ettől eltér, ezért a nyers accuracy nem adja meg a valós döntési teljesítményt; (3) **felelősségi hiba:** az emberi felülvizsgálat szükségességét a pontosság nem szünteti meg, mert a hiba következménye a felhasználási körben nagy tétű lehet. Javított állítás: „A modell 0,98 accuracy-t ért el a GlyphLab clean tesztjén, amely 24×24 szintetikus, kiegyensúlyozott alakzatképekből áll; ezért ettől eltérő kamerafeltételre vagy osztályarányra az állítást nem terjesztjük ki. A PageTurn bevezetéséhez külön, előre rögzített pilot méri a valós eloszláson a pontosságot és az emberi felülvizsgálati arányt."

F5. Egy lehetséges terv. Döntési szabály: a PageTurn akkor forgat automatikusan, ha a legnagyobb és a második legnagyobb **logit** különbsége (a margin, $m = z_{(1)} - z_{(2)}$) meghaladja a τ küszöböt; minden más esetben a rendszer kijelölt emberi felülvizsgálóhoz irányítja az oldalt. A softmax a logitok sorrendjét megőrzi, de a margin logitléptékű, ezért a küszöböt nem önkényesen választjuk: a validációs halmazon a kívánt felülvizsgálati arányhoz kalibráljuk (például a margin 90. percentilisénél, ami körülbelül 10% felülvizsgálatot ad), és a pilot előtt rögzítjük; a 2,0 csak illusztratív érték, önmagában a logitok skálázása miatt nem értelmezhető. Pilot: 200 oldal, de nem 200 független minta: 40 dokumentumból dokumentumonként 5 oldal, mert az azonos dokumentum oldalai korreláltak, ezért a resampling egysége a dokumentum. A méret indoklása: 200 oldalnál a független becslés 95%-os konfidenciaintervallumának félszélessége $p \approx 0{,}9$ mellett körülbelül 4,2 százalékpont, dokumentumszintű klaszter-bootstrappal ennél szélesebb; a pilot ekkora pontosságú döntést céloz, nem szignifikanciatesztet. Az oldalak nem vettek részt a küszöb választásában. Baseline: a jelenlegi, modell nélküli manuális döntés ugyanazon a 200 oldalon (forgatási accuracy, oldalankénti idő, felülvizsgálati arány 100%). Kontroll: ugyanaz a modell és ugyanaz a 200 oldal, de felülvizsgálatra a margin helyett véletlenül kiválasztott ugyanannyi oldalt jelölünk ki, azonos felülvizsgálati arány mellett; így a kontroll a margin szerinti válogatás hatását izolálja. Metrikák: forgatási accuracy, felülvizsgálati arány, az emberi felülvizsgálat saját accuracy-ja és a modell és az ember együttes döntésének accuracy-ja; a bizonytalanságot dokumentumszintű bootstrap adja. A következtetési határ: a pilot eredménye csak az adott scannerre, dokumentumtípusra és osztályeloszlásra érvényes; új intézmény vagy formanyomtatvány új pilotot igényel.

Alak- és szélsőeset-ellenőrzés az F3-hoz: az osztálykulcsok a spliten belül előforduló címkékből jönnek, ezért üres osztály nem kerül a `dict`-be; a `mm.sum() > 0` feltétel redundáns, de explicit. A `split_ids` dtype `str`, a `preds` int64, így a címkeegyezés logikai tömbbé alakul. A labor ellenőrzése

`math.isclose`-zal (10^{-12}) abszolút toleranciával hasonlít; ez a számítás pontosságát nem érinti, csak a lebegőpontos egyezést rögzíti.

35.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

A gyakori tévedés pontosan a szintetikus benchmark összekeverése a valós alkalmazással. Ellenpélda a prevalencia-csapda: ha egy beléptető rendszerben a valódi „érvényes token” aránya $p = 0{,}01$, akkor a mindig-érvényes osztályozó accuracy-ja $p = 0{,}01$, a mindig-érvénytelené $1 - p = 0{,}99$; a triviális „mindig érvénytelen” baseline tehát 0,99-et ér el. Ezzel a baseline-nal a kiegyensúlyozott GlyphLab 0,98-as érték nem bizonyít védelmi képességet; a releváns metrika a sensitivity/specificity (érzékenység/fajlagosság) és a következménymátrix. A második ellenpélda a licenchez kötődik: egy letöltött súly 0,99-as publikus benchmarkeredménye nem ad jogot a súly kereskedelmi felhasználására, és nem is igazolja a saját kameraeloszlásra. A [P25] sanity check arra emlékeztet, hogy a magyarázat sem pótolja a modellfüggőség ellenőrzését; a [P23] Grad-CAM lokalizációja nem ok-okozati bizonyíték; a [D21] FGSM-példa pedig oktatási demonstráció, nem éles biztonsági határ.

Önellenőrző kérdések: Meg nevezi-e a modellkártya a bemeneti alakot és intenzitástartományt? A split jelenetszintű-e, és a corrupt teszt ugyanazon párokat használja-e? A valós alkalmazás osztályaránya mért vagy feltételezett? A súly SHA-256-ja és licence a mentési verzióhoz kötött-e? Mit mér az emberi felülvizsgálat? Visszatekintés: 23.4. Hibaszeletek és alkalmazási határ megmutatta, hogy az összesített accuracy a szeletbontás nélkül elfedi a koncentrált hibát; 35.2. Magyarázat sanity check pedig azt, hogy a paraméterrandomizálás kontrollja nélkül a hőtérkép nem igazol modellfüggést.

35.4.9 Rövid összefoglaló

A felhasználási kör, az adathatár és a származás a teljesítményállítás része: a clean teszt accuracy csak a rögzített szintetikus eloszlásra értelmezhető. A 2×2 példa két modellje azonos clean pontosság mellett intenzitásinverzió után 100% és 50% pontosságú volt, tehát a benchmarkeredményből nem következik a shift alatti viselkedés. A modellkártya rögzíti a bemeneti specifikációt, a korrupciós határt, a licencet és az emberi felülvizsgálat feltételeit. Az adatminimalizálás a privacy-kockázatot csökkenti, de nem bizonyítja a klinikai vagy jogi alkalmasságot. Ritka eseményes környezetben a kiegyensúlyozott accuracy félrevezető, ezért a baseline és a következménymátrix kötelező.

35.4.10 Továbblépés

A 36.1. Állításból kísérleti terv egységben a dokumentált korlátokból kísérleti tervet készítünk: az itt rögzített adathatár és modellkártya-mezők lesznek a kísérleti hipotézis, a mérőszám és a kiértékelési protokoll forrásai.

35.4.11 Felhasznált források

- [P23] Selvaraju et al. (2016). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. https://arxiv.org/abs/1610.02391.
- [P25] Adebayo et al. (2018). *Sanity Checks for Saliency Maps*. https://arxiv.org/abs/1810.03292.

- [D21] PyTorch contributors (2026). *Adversarial Example Generation (FGSM)*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/fgsm_tutorial.html. Hozzáférés: 2026-10-30.

36 Kutatási módszertan és reprodukálható eredményközlés

36.1 Állításból kísérleti terv

36.1.1 Cél, előismeret és induló kérdés

Ez az egység a 23.4. Hibaszeletek és alkalmazási határ alfejezetben bevezetett hibaszeletek, a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben rögzített tanítási csomag és a 35.4. Korlátok, eredet és döntési felelősség alfejezetben tárgyalt felelős korlátok után azt a módszertani lépést vizsgálja, amely **megelőzi** a tanítást: hogyan alakítunk egy kutatási állítást olyan kísérleti tervvé, amelyből a mérés után lektorálható következtetést lehet levonni. A hangsúly a *preregisztrált helyi terv*, az *abláció* és a *zavaró tényező* fogalmán van, nem egy új hálózati művelet bevezetésén.

Előismeret: az 5.4. Függő képek, seedek és resampling alfejezetből a scene-szintű statisztikai egység és a páros összehasonlítás, a 8.4. II. ellenőrző pont: korrekt tanulási állítás alfejezetből a független teszt és a modellválasztási ciklus, a 24.1. Tanulási görbe és hibadiagnózis alfejezetből a tanulási görbe diagnosztikai szerepe, a 32.1. Szekvenciamodell és változó hossz alfejezetből a maszkok mint előre rögzített információs csatornák, a 35.1. Saliency és Grad-CAM alfejezetből pedig a Grad-CAM súlyozási képlete.

Induló kérdés. Egy fejlesztő azt állítja: „A BatchNorm hozzáadása javította a CNN pontosságát a GlyphLabon, mert a teszt-accuracy 78%-ról 82%-ra nőtt.” (Az adatkészlet szerződését a függelék A. A GlyphLab adatkészlet szakasza rögzíti.) Hányféle módon lehet ez az állítás hibás? Becslés: legalább három módon: (a) a tesztkészleten választott modellt, (b) a BatchNorm mellett a tanulási rátát is megváltoztatta, (c) egyetlen kedvező seedet jelentett, nem ismételt mérést. A helyes protokoll ezeket a csatornákat előre lezárja. A zavaró (confounder) változót a kísérletben a *beavatkozás–kimenet kapcsolaton* definiáljuk (olyan tényező, amely a BatchNorm-hozzáadástól és a pontosságtól is függ); a nemnulla kovariancia ennek nem teljes jellemzése, mert nemlineáris függés nulla kovariancia mellett is lehetséges. Eltérő architektúráknál az azonos seed sem garantál megfelelően párosított RNG-húzásokat, mert a hívások száma és sorrendje eltér.

36.1.2 Fogalmak: hipotézis, abláció, zavaró tényező, preregisztrált terv, compute budget

Jelölések. A fejezetben c a célosztály indexe, C a konfiguráció (komponensek rendezett listája), F a jellemzőtérkép tenzora, M a metrika, $\Delta\hat{M}$ pedig a ΔM minta alapján számolt becslése. A validációs halmazt **val**, a BatchNormot **BN**, a konfidenciaintervallumot **CI**, a metszet-egyesítés arányt **IoU** (intersection over union) rövidítéssel jelöljük.

Definíció (hipotézis). Kísérleti hipotézisnek nevezünk egy olyan, a mérés előtt rögzített állítást, amely (i) megnevezi a két összehasonlítandó konfigurációt A és B; (ii) megadja a kimeneti metrikát M és annak aggregálási szintjét; (iii) megfogalmazza a várt irányt, például $\Delta M := \mathbb{E}[M(B)] - \mathbb{E}[M(A)] > 0$. A $\mathbb{E}[\cdot]$ itt a protokoll által definiált ismétlési eloszlásra

(adatminta, tanítási seed, esetleg hardver) vonatkozik; nem egy meg nem figyelhető „igazi” eloszlásra. A cáfolhatóság feltétele, hogy létezzen olyan előre rögzített döntési szabály, amely a mért, a minta alapján becsült $\Delta\widehat{M}$ érték alapján a hipotézist elfogadja vagy elveti.

Definíció (abláció). Legyen a vizsgált konfiguráció $C = (e_1, e_2, \dots, e_k)$ komponensek rendezett listája. Az i. komponensre vonatkozó abláció a $C_{-i} = (e_1, \dots, e_{i-1}, e_{i+1}, \dots, e_k)$ konfiguráció, amelyben *kizárólag* e_i hiányzik vagy változik; minden más komponens értéke bitre pontosan azonos a C-beli értékkel. Az abláció tehát nem egyszerűen „két modell összehasonlítása”, hanem egyetlen különbség izolálása. Két dolgot kell megkülönböztetni. (a) A **független** zavaró tényezők — például a learning rate, az epoch-szám, a batchméret vagy az adat — nem változhatnak a két ág között; ha változnak, az már nem tiszta abláció. (b) A beavatkozás **szükségszerű következményei** viszont a beavatkozáshoz tartoznak: egy réteg vagy tanulható komponens eltávolítása vagy hozzáadása természetes módon megváltoztatja a paraméterszámot, és ez nem teszi érvénytelenné a komponens *teljes* hatásának ablációját. Ha az a kérdés, hogy a javulást a szerkezet vagy az extra paraméterkeret magyarázza, azt egy külön, opcionális **kapacitásillesztett kontroll** vizsgálja (például a hiányzó komponens paraméterkeretét máshová, szélesebb rétegbe tesszük). A seedek kezelése párosítási recept: az azonos seed-készlet mindkét ágon csökkenti a varianciát és párosított összehasonlítást tesz lehetővé, de az eltérő seedekkel végzett, ismételt kísérlet sem logikailag érvénytelen, csak párosítatlan és zajosabb.

Definíció (zavaró tényező). Zavaró tényezőnek (confound) nevezünk egy olyan u változót, amely (i) együtt változik a vizsgált e_i komponenssel, és (ii) önállóan is hat a kimenetre. Formálisan: $Y = g(e_i, u)$, ahol u eloszlása függ e_i-től, és g nem konstans u-ban. A $\mathrm{Cov}(e_i, u) \neq 0$ feltétel sem nem szükséges, sem nem elégséges: nemlineáris függés mellett a kovariancia nulla is lehet, és egy nemnulla kovariancia önmagában nem bizonyítja, hogy u a kimenetre is hat. A tanulási ráta, a modellkapacitás, a batch-méret és a seed mind tipikus confound lehet, ha a kísérleti tervben együtt mozognak a beavatkozással.

Definíció (preregisztrált helyi terv). A helyi terv egy rövid, verziózott dokumentum, amelyet *a futás előtt* rögzítenek, és amely legalább az alábbi mezőket tartalmazza: kérdés; hipotézis; adat és split; kontroll/baseline; változó tényező; rögzített tényezők listája; metrika és statisztikai egység; döntési szabály; compute budget; ismert kockázatok és kiértékelési korlátok. A „helyi” jelző arra utal, hogy a terv a saját adatra, saját környezetre és saját erőforrás-korlátra vonatkozik; nem ígér általános benchmark-eredményt.

Definíció (compute budget). A compute budget a modellkereséshez előre rendelt számítási keret, például falidő, CPU-óra, batch-iterációk vagy epochok maximális száma. A budgetet azért kell előre rögzíteni, mert a korlátlan keresés a teszt- vagy akár a validációs halmaz túlhasználatához vezet: ha addig próbálgatunk, amíg a kívánt számot el nem érjük, a „győztes” eredmény a keresési folyamat, nem a modell tulajdonsága.

Mindegyik fogalom a GlyphLab szerződéséhez igazodik: a megfigyelési egység a `scene_id`, a train/val/test felosztás scene-szintű, a teszt csak a végleges modell kiválasztása után értékelhető egyszer. A `corrupted_test.npy` páros tesztjét különösen szigorúan védjük: annak kiértékelése utólagos célválasztás nélkül, kizárólag a befagyasztott modellen történhet.

36.1.3 Matematikai és empirikus állítás szétválasztása: a Grad-CAM példa

Ugyanabban a példában gyakran keveredik egy levezethető matematikai azonosság és egy mérést igénylő empirikus állítás. Válasszuk a 35.1. Saliency és Grad-CAM alfejezetből ismert Grad-CAM ([P23]) hőtérképet.

Matematikai állítás (lemma). Legyen $F \in \mathbb{R}^{K\times H\times W}$ egyetlen kép egy köztes rétegbeli aktivációs térképe, $F_{k,i,j}$ annak k. csatornájának (i,j)-edik pixele. Definiáljuk a csatornánkénti globális átlagot

$$Z_k = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} F_{k,i,j}.$$

Tegyük fel, hogy a c osztály logitja ezen átlagok differenciálható függvénye: $y^c = g_c(Z_1,\dots,Z_K)$. Ekkor a

$$q_k^c := \sum_{i=1}^{H}\sum_{j=1}^{W} \frac{\partial y^c}{\partial F_{k,i,j}}$$

mennyiségre fennáll

$$q_k^c = \frac{\partial y^c}{\partial Z_k}.$$

Bizonyítás. A többváltozós láncszabályt alkalmazzuk a $y^c = g_c(Z_1,\dots,Z_K)$ összetett függvényre. Egy rögzített (k,i,j) indexhármasra:

$$\frac{\partial y^c}{\partial F_{k,i,j}} = \sum_{l=1}^{K} \frac{\partial y^c}{\partial Z_l}\cdot\frac{\partial Z_l}{\partial F_{k,i,j}}.$$

Mivel Z_l definíció szerint a l. csatorna pixeleinek átlaga, a $\partial Z_l/\partial F_{k,i,j}$ parciális derivált értéke $\frac{1}{HW}$, ha $l=k$, és 0, ha $l\neq k$. Így az összeg egyetlen tagra redukálódik:

$$\frac{\partial y^c}{\partial F_{k,i,j}} = \frac{1}{HW}\cdot\frac{\partial y^c}{\partial Z_k}.$$

Most összegezzük ezt az egyenlőséget $i=1,\dots,H$ és $j=1,\dots,W$ szerint:

$$\sum_{i,j}\frac{\partial y^c}{\partial F_{k,i,j}} = \sum_{i,j}\frac{1}{HW}\frac{\partial y^c}{\partial Z_k} = \frac{HW}{HW}\frac{\partial y^c}{\partial Z_k} = \frac{\partial y^c}{\partial Z_k}.$$

A bal oldali összeg éppen q_k^c, tehát az állítás teljesül. A 35.1. Saliency és Grad-CAM alfejezetben használt szabványos Grad-CAM-súly viszont a térbeli **átlag**: $w_k^c = \frac{1}{HW}\sum_{i,j}\partial y^c/\partial F_{k,i,j} = q_k^c/(HW)$. A két jelölés összekeverése HW-szoros skálahibát okoz. □

A lemma valójában a többváltozós láncszabály közvetlen következménye; a „lemma” elnevezés a későbbi hivatkozást szolgálja. Feltételei: a fejfüggvény g_c differenciálható a Z pontban, és a Z_k átlagolás a teljes $H\times W$ térbeli tartományra vonatkozik. A logitnak a kiválasztott réteg aktivációitól csak a Z_k csatornaátlagokon keresztül kell függenie; ez pontosan a GAP-ra épülő fej esete. Nincs szükség arra, hogy a hálózat többi része lineáris legyen. A bizonyítás tisztán analitikus, numerikus mérést nem igényel.

Empirikus állítás. Ugyanebben a példában az alábbi mondat már nem levezethető: „A Grad-CAM hőtérkép a modell döntése szempontjából fontos képrészletet lokalizálja.” Ez az állítás egy mérhető kapcsolatra vonatkozik a hőtérkép és a GlyphLab `masks.npy` maszkja között, például a binarizált hőtérkép és az előtér-maszk IoU-jára. Ennek igazolásához empirikus protokoll kell:

rögzített modell, független teszt, metaadat, valamint kontroll. A [P25]-ben bevezetett sanity check gondolatmenete ide illik: ha a magyarázat valóban a megtanult döntési mechanizmust tükrözi, akkor a modellsúlyok véletlenszerűsítése után a lokalizációs metrikának érdemben romlania kell; ha nem romlik, a mért egyezés nem a tanult reprezentációt igazolja. Ez mérés, nem bizonyítás, és a kontroll nélkül az IoU-érték nem értelmezhető.

A két állítástípus elkülönítése tehát: **lemma** = feltételek és analitikus levezetés; **empirikus állítás** = protokoll, minta, split, metrika, kontroll és bizonytalansági határ.

36.1.4 Kézi példa és minimális kód

Tekintsünk egyetlen $1 \times 2 \times 2$ méretű jellemzőtérképet (egy csatorna, $K = 1$, $H = W = 2$), és a lineáris fejet $y^c = 2Z$, ahol $Z = \frac{1}{4}\sum_{i,j} F_{1,i,j}$. A lemma szerint $q_1^c = \frac{\partial y^c}{\partial Z} = 2$, míg a Grad-CAM átlagolt súlya $w_1^c = q_1^c/4 = 0{,}5$. A következő kód ezt középső differenciás numerikus ellenőrzéssel hasonlítja össze float64 pontossággal. A kódban található ellenőrzések a megadott analitikus értékekre vonatkoznak; a futási eredmény helyben reprodukálható.

```
import numpy as np

# munkakönyvtár: lab/ (saját ellenőrző szkript)
F = np.array([[[[1.0, 2.0],
                [3.0, 4.0]]]], dtype=np.float64)  # (1,1,2,2)

def logit(F):
    # y = 2 * (csatornánkénti térbeli átlag)
    return 2.0 * F.mean(axis=(2, 3), keepdims=False)

def numerical_grad(F, eps=1e-4):
    g = np.zeros_like(F)
    it = np.nditer(F, flags=['multi_index'])
    while not it.finished:
        idx = it.multi_index
        old = F[idx]
        F[idx] = old + eps
        yp = logit(F)[0, 0]
        F[idx] = old - eps
        ym = logit(F)[0, 0]
        F[idx] = old
        g[idx] = (yp - ym) / (2.0 * eps)
        it.iternext()
    return g

g_num = numerical_grad(F)
q_num = g_num.sum()   # GAP-fej deriváltja: sum_ij dY/dF_ij
w_num = g_num.mean()  # Grad-CAM: (1/HW)*sum_ij dY/dF_ij
print("GAP-fej deriváltja:", q_num)
print("Grad-CAM átlagolt súlya:", w_num)
assert abs(q_num - 2.0) < 1e-6, abs(q_num - 2.0)
assert abs(w_num - 0.5) < 1e-6, abs(w_num - 0.5)

# eps-söprés ugyanezzel a numerical_grad függvénnyel:
for eps in (1e-2, 1e-4, 1e-6, 1e-8, 1e-10, 1e-11):
    g = numerical_grad(F, eps)
    print(f"{eps:.0e} q_hiba={abs(g.sum()-2.0):.2e} "
          f"w_hiba={abs(g.mean()-0.5):.2e}")
```

A söprés mért kimenete (Python 3.12.3, NumPy 2.5.4, CPU):

```
1e-02 q_hiba=4.26e-14 w_hiba=1.07e-14
1e-04 q_hiba=4.66e-12 w_hiba=1.17e-12
1e-06 q_hiba=2.06e-09 w_hiba=5.14e-10
1e-08 q_hiba=1.66e-07 w_hiba=4.14e-08
1e-10 q_hiba=1.66e-07 w_hiba=4.14e-08
1e-11 q_hiba=1.78e-04 w_hiba=4.45e-05
```

Ellenőrzés. A `logit` affin F-ben, ezért a középső differencia egzakt aritmetikában bármely $\varepsilon \neq 0$ lépésnél pontos: a csonkolási hiba nulla, az $O(\varepsilon^2)$ általános felső korlát itt nem jelent tényleges 10^{-8}-as hibát. A mért eltérés kizárólag lebegőpontos kerekítésből származik. A kód `eps`-söpréssel futtatva (Python 3.12.3, NumPy 2.5.4, CPU) a `q_num` abszolút hibája `float64`-ben $\varepsilon = 10^{-2}$-nél $4{,}3 \cdot 10^{-14}$, 10^{-4}-nél $4{,}7 \cdot 10^{-12}$, 10^{-6}-nál $2{,}1 \cdot 10^{-9}$, 10^{-8}-nál $1{,}7 \cdot 10^{-7}$ (a kerekítési hiba $\sim u/\varepsilon$ szerint nő, ahol $u \approx 2{,}2 \cdot 10^{-16}$ a `float64` egységkerekítése). A `1e-6` tolerancia $\varepsilon = 10^{-4}$ mellett biztonságos; $\varepsilon = 10^{-8}$, 10^{-9} és 10^{-10} mellett a hiba $1{,}7 \cdot 10^{-7}$, ami még a tolerancia alatt van, a teszt csak $\varepsilon \approx 10^{-11}$ körül bukna el (ekkor a hiba $1{,}8 \cdot 10^{-4}$). A `logit` függvény a `keepdims=False` beállítással expliciten $(1;\ 1)$ alakú kimenetet ad, így az indexelés `[0,0]` nem hagy kétséget a redukció irányáról.

36.1.5 Labor: egyoldalas ablatív protokoll

A labor célja, hogy a hallgató **a futás előtt** elkészítsen egy `ablation_protocol.md` dokumentumot, amely a tesztkészlethez igazított utólagos célválasztás lehetőségét kizárja. Az alábbi sablon egy lehetséges, teljes megoldás; a hallgató saját kérdését is választhatja.

Kérdés. Javítja-e a BatchNorm-réteg (Ioffe és Szegedy [P30]) hozzáadása a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben rögzített kis CNN validációs pontosságát a GlyphLab adathalmazon?

Hipotézis. H_1: a BatchNorm-os modell (`B`) val-accuracy átlaga nagyobb, mint a BatchNorm nélküli baseline-é (`A`) ugyanazon öt seed mellett; a döntési metrika a seedenkénti páros különbségek átlaga.

Adat és split. GlyphLab `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. A hivatalos train/val/test felosztás scene-szintű; a tesztet a teljes kísérlet alatt nem érintjük. A baseline és a BN-es futás **ugyanazt** a train/val splitet kapja.

Kontroll/baseline. A 24.4. VI. ellenőrző pont: megbízható CNN-training VI. ellenőrző pontban rögzített kis CNN BatchNorm nélkül. A példaarchitektúra: `Conv2d(1,8,3)` + `Conv2d(8,16,3)` + `Linear(16·6·6,3)`, azaz $80 + 1168 + 1731 = 2979$ paraméter. A súlyinicializálás mindkét ágban ugyanazzal a seed-sorozattal indul.

Változó tényező. Kizárólag a BatchNorm-réteg jelenléte az első konvolúció után.

Rögzített tényezők (confound-ellenőrzés). Optimizáló (AdamW), learning rate, weight decay, batch méret, epochszám, augmentáció, adatbetöltési sorrend (seed), hardver (CPU), `float32` dtype. A két modell paraméterszám-különbségét feljegyezzük a jelentésben; ha a paraméterszám-különbség meghaladja az 1%-ot, egy kapacitás-illesztett kontrollt is futtatunk. Az első konvolúció utáni BatchNorm 8 csatornánál $2 \cdot 8 = 16$ paramétert ad, ami $16/2979 \approx 0{,}54\%$, tehát ez a konfiguráció a küszöb alatt marad; a második konvolúció után (16 csatorna, 32 paraméter) a különbség 1,07%, és a kapacitás-kontrollt a tíz futásos kereten felül, külön kell engedélyezni.

Metrika és statisztikai egység. Val-accuracy per seed; a páros különbség statisztikai egysége a seed, amely a tanítási algoritmus bizonytalanságát méri. Az adatminta bizonytalanságát az 5.4.

Függő képek, seedek és resampling alfejezetben rögzített scene-rétegzett bootstrap adja a végső teszt értékeléséhez; ezt a kiválasztási döntéshez nem használjuk.

Döntési szabály. Öt seed után: ha a seedenkénti páros val-accuracy különbségek átlaga $> 0{,}01$, és a páros 95%-os t-intervallum ([S02], df=4) alsó határa > 0, akkor H_1-et elfogadjuk, és a BN-es modellt választjuk. Minden más esetben a baseline-t tartjuk meg. A döntési szabály kizárólag a **val** halmazt használja. A 0,01-es küszöb gyakorlati relevancia-küszöb: egyszázalékos val-accuracy javulás alatt a BatchNorm bevezetésének költsége és kockázata nem indokolt; a küszöböt a futás előtt rögzítjük, nem a mért szórásból választjuk. A t-intervallum döntési szabály, nem utólagos becslés. **Korlát:** öt páros minta mellett (df=4) a próba statisztikai ereje kicsi, ezért egy negatív döntés nem bizonyítja, hogy nincs hatás; ezt a jelentés korlátai között fel kell tüntetni, és nagyobb hatás kimutatásához több seed kell.

Compute budget. CPU-n, legfeljebb két óra falidő, összesen tíz tervezett futás (öt seed × két konfiguráció). Ha bármely futás NaN-t ad, a 24.1. Tanulási görbe és hibadiagnózis alfejezetben leírt diagnózist követjük, a futást egyetlen alkalommal, azonos seeddel megismételjük, és az eseményt a naplóban rögzítjük; további automatikus ismétlés nincs. A NaN-ismétlés a tíz tervezett futáson felüli, legfeljebb egy futást jelent, amelyet a jelentésben külön sorolunk fel.

Kiértékelés. A döntési szabály alapján választott, befagyasztott modellt a teszthalmazon egyszer értékeljük, majd a `corrupted_test.npy` páros halmazon is. A protokoll utolsó sora: „A fenti döntési szabályban a `test` szó nem szerepelhet; a protokoll érvényességének feltétele ennek automatikus ellenőrzése.”

36.1.6 Feladatok

F1 (fogalmi, nehézség: 1). Magyarázza el, miért nem tiszta abláció az a kísérlet, amelyben a BatchNorm hozzáadásával egyidejűleg a tanulási rátát 10^{-3}-ról 10^{-4}-re csökkentik. Adjon meg egy olyan kimeneti forgatókönyvet, amelyben a mért javulást tévesen a BatchNormnak tulajdonítanák. Elvárt eredmény: saját magyarázat, explicit confound-azonosítás, egy ellenpélda.

F2 (levezetési, nehézség: 2). Igazolja a 36.1.3. szakasz lemmájának egy speciális esetét: $K = 1$, $H = W = 2$, $y^c = 3Z$, azaz a lineáris fej együtthatója $v_1 = 3$. Írja fel a $\partial y^c / \partial F_{1,i,j}$ négy parciális deriváltat, összegezze őket, és mutassa meg, hogy $q_1^c = 3$, míg az átlagolt Grad-CAM-súly $w_1^c = 3/4$. Elvárt eredmény: minden lépés, indexhatár és shape-kontroll.

F3 (implementációs, nehézség: 1). Írjon Python-függvényt `validate_protocol(text: str) -> tuple[bool, list[str]]`, amely egy protokoll-szövegben ellenőrzi, hogy (a) a döntési szabály szakaszában nem szerepel a `test` szó önálló szóként (kis- és nagybetű érzéketlenül); (b) a `## Compute budget` szakasz jelen van. A döntési szakaszt a `## Döntési szabály` címsor vagy a `**Döntési szabály.**` félkövér jelölő is azonosíthatja, mert a labor sablonja az utóbbit használja. Elvárt eredmény: teljes kód, egy érvényes és egy érvénytelen teszteset, a visszatérési érték pontos leírása.

F4 (hibakeresési, nehézség: 2). Egy jegyzőkönyv szerint a kutató 12 különböző konvolúciós kernelméretet próbált ki, mindegyiket a **teszt** halmazon értékelte, és a legjobbat (87%) jelentette mint „a modell általános pontosságát”. Lokalizálja a hibát, írja le a következményt a becslés torzítására, és adjon javítást a preregisztrált terv keretében. Elvárt eredmény: ok, következmény, javítás, valamint annak ellenőrzése, hogy a javított protokoll döntési szabálya nem hivatkozik a tesztre.

F5 (kísérlettervezési/transzfer, nehézség: 3). Készítsen egyoldalas preregisztrált tervet erre a kérdésre: „Javítja-e a képek véletlen fényerő-augmentációja a GlyphLab **korrupt** tesztjének pontosságát?” Adja meg a baseline-t, a változó tényezőt, a splitet, a statisztikai egységet, a

metrikát, a döntési szabályt és a compute budgetet. A tervnek tartalmaznia kell, hogy a korrupt készlet kizárólag a választott modell befagyasztása után értékelhető. Elvárt eredmény: egyoldalas terv; negatív, de korrekt eredmény is elfogadható.

36.1.7 Megoldások és indoklás

F1 megoldás. A két változtatás (BatchNorm és tanulási ráta) együtt mozog, ezért a ΔM különbségben a BatchNorm hatása és a tanulási ráta hatása nem szeparálható. Ellenpélda: ha a baseline tanulási rátája eleve túl nagy volt, a 10^{-4}-re csökkentés önmagában, BatchNorm nélkül is javíthatja a val-accuracyt; a mért nyereséget mégis a BatchNorm kapná. A tiszta ablációhoz a tanulási rátát mindkét ágban rögzíteni kell, vagy egy harmadik ágat kell futtatni, amelyben csak a tanulási ráta változik.

F2 megoldás. Legyen $F_{1,i,j} = f_{ij}$, $Z = \frac{1}{4}(f_{11} + f_{12} + f_{21} + f_{22})$, $y = 3Z$. Ekkor minden (i, j)-re

$$\frac{\partial y}{\partial f_{ij}} = 3 \cdot \frac{\partial Z}{\partial f_{ij}} = 3 \cdot \frac{1}{4}.$$

A négy parciális derivált mindegyike $\frac{3}{4}$. Összegük:

$$q_1^c = \sum_{i=1}^{2} \sum_{j=1}^{2} \frac{\partial y}{\partial f_{ij}} = 4 \cdot \frac{3}{4} = 3.$$

Alakellenőrzés: a bemenet $F \in \mathbb{R}^{1\times 2\times 2}$, a kimenet q_1^c és w_1^c is skalár; a gradiens ugyanolyan alakú, mint F. A lemma szerint $q_1^c = \partial y/\partial Z = 3$, a Grad-CAM-átlagsúly pedig $w_1^c = q_1^c/(HW) = 3/4$.

F3 megoldás.

```
import re

def validate_protocol(text: str) -> tuple[bool, list[str]]:
    problems = []
    if "## Compute budget" not in text:
        problems.append("compute_budget")
    pattern = (r"(?:^##\s+D[öo]nt[eé]si szab[aá]ly\b"
               r"|\*\*D[öo]nt[eé]si szab[aá]ly\.\*\*)"
               r"(.*?)(?=\n## |\Z)")
    m = re.search(pattern, text, re.S | re.M | re.I)
    if m is None:
        problems.append("dontesi_szabaly")
    else:
        if re.search(r"\btest\b", m.group(1), re.I):
            problems.append("test_hivatkozas")
    return (len(problems) == 0, problems)

# érvényes: a döntési szabály a val halmaz átlagára hivatkozik
ok1 = """## Döntési szabály
A val halmaz átlagos különbsége alapján döntünk.

## Compute budget
Legfeljebb két óra CPU."""
assert validate_protocol(ok1) == (True, [])

# érvénytelen: a döntési szabály a tesztre hivatkozik
ok2 = """## Döntési szabály
A legjobb test accuracyt választjuk.
```

```
## Compute budget
Legfeljebb két óra CPU."""
state, problems = validate_protocol(ok2)
assert state is False and "test_hivatkozas" in problems

# a labor sablonjának félkövér jelölője is felismerhető
ok3 = """**Döntési szabály.** A val halmaz átlagos különbsége alapján döntünk.

## Compute budget
Legfeljebb két óra CPU."""
assert validate_protocol(ok3) == (True, [])
```

A függvény `(ok, problems)` párt ad vissza: `ok` akkor igaz, ha a `problems` lista üres; a `problems` a hiányzó szakaszok (`compute_budget`, `dontesi_szabaly`) és a szabálysértés (`test_hivatkozas`) azonosítóit tartalmazza. A `\btest\b` szóhatár miatt a `latest`, `protest` vagy `testing` szavak nem adnak hamis pozitív találatot. Tolerancia: a függvény tiszta szöveges ellenőrzés, numerikus tolerancia nincs. A `re.S` flag a többsoros szakasz kezeléséhez, a `re.M` a címsorok sor elejéhez szükséges.

F4 megoldás. Hiba: a tesztkészletet modellválasztásra használták. Következmény: a jelentett 87% a 12 kísérlet maximuma, nem a modell várható pontossága; a becslés felfelé torzított, és a torzítás a kipróbált konfigurációk számával nő. Javítás: a kernelméretet a validációs halmazon választjuk meg a befagyasztott, előre rögzített döntési szabály szerint; a tesztet csak a győztes modellen, egyszer értékeljük. A javított protokoll szövege például:

```
## Döntési szabály
A 12 kernelméret közül azt választjuk, amelynek a validációs accuracy átlaga
a legnagyobb; döntetlen esetén a kisebb kernelméretet. A döntés kizárólag a
validációs halmazt használja.

## Compute budget
12 tanítás × 20 epoch, legfeljebb 60 perc CPU.

## Kiértékelés
A győztes, befagyasztott modellt a teszthalmazon egyszer értékeljük.
```

Ellenőrzés: ezen a szövegen az F3 `validate_protocol` függvénye `(True, [])` értéket ad (a futtatott ellenőrzés a megoldáshoz tartozó kódban szerepel); a 12 val-accuracy érték nyers táblázatban marad, és a közölt eredmény a győztes modell teszt-értéke.

F5 megoldásminta.

- **Kérdés:** Javítja-e a fényerő-augmentáció a korrupt teszt pontosságát?
- **Baseline:** a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetben rögzített kis CNN, augmentáció nélkül, öt seeddel.
- **Változó:** csak a fényerő-szorzó [0,7; 1,3] intervallumbeli véletlen skálázása a train képeken; minden más transzformáció és recept rögzített.
- **Split:** hivatalos GlyphLab train/val/test; scene-szintű, a teszt kizárólag a választott modell befagyasztása után értékelhető; a korrupt páros készlet ugyanígy.
- **Statisztikai egység:** seed a kiválasztáshoz; scene-rétegzett bootstrap a végső intervallumhoz.
- **Metrika:** clean és korrupt test accuracy, páros összehasonlítás a korrupciós romlás mértékére.
- **Döntési szabály:** ha a val-accuracy páros különbségek átlaga (augmentált − baseline) $> 0{,}01$, és a páros 95%-os t-intervallum (df=4) alsó határa > 0, az augmentált modellt választjuk; különben a baseline-t. A 0,01 itt is előre rögzített gyakorlati relevancia-küszöb

(egy százalékpont), nem a mért szórásból származik, és a döntés kizárólag a val halmazon születik.

- **Compute budget:** legfeljebb 90 perc CPU, összesen 10 futás. A terv rögzíti, hogy a korrupt teszt a választás után, a befagyasztott modellen, a clean teszttel azonos párosításban kerül kiértékelésre: a `corrupted_test.npy` j-edik képe a metaadat szerinti j-edik clean tesztkép párja, és a jelentett metrika a két accuracy különbsége (korrupciós romlás), nem a korrupt accuracy önmagában.

36.1.8 Tipikus tévedés ellenpéldával, önellenőrzés és visszatekintés

Tipikus tévedés. A módszerek sok részletben eltérő összevetése nem izolál egyetlen újítás hatását. Konkrét ellenpélda: az A konfiguráció a baseline, a B pedig A + BatchNorm + kisebb learning rate + kétszeres epochszám. A val-accuracy A-nál 0,78, B-nél 0,81. A 0,78 és a 0,81 szemléltető, egyetlen futásból származó értékek, szórás és ismétlés nélkül; a példa kizárólag a következtetés szerkezetére vonatkozik, a két számból nem vonunk le hatást. A „BatchNorm javított" következtetés hibás, mert a három különbség bármelyike, illetve ezek kölcsönhatása is okozhatja a nyereséget. A helyes eljárás három párhuzamos abláció: (i) csak BatchNorm, (ii) csak learning rate, (iii) csak epochszám, mindegyik a rögzített A-hoz képest. Csak az (i) ág pozitív eredménye támogathatja a BatchNorm-állítást.

Önellenőrzés. (1) A választott protokoll döntési szabályában valóban nem szerepel a tesztkészlet; ha igen, a terv érvénytelen. (2) A lemma levezetésében az indexhatár: $i = 1, \dots, H$, $j = 1, \dots, W$, az összegzés pontosan HW darab $\frac{1}{HW}$ tagot ad. (3) A numerikus kódban a `mean(axis=(2, 3))` NCHW redukció, nem keverendő a batch szerinti átlaggal.

Visszatekintés. Az 5.4. Függő képek, seedek és resampling alfejezetben bevezetett scene-szintű statisztikai egység itt a bootstrap és a páros értékelés alapja: a képenkénti, független mintavételt feltételező konfidenciaintervallum a két nézet korrelációja miatt nem ugyanazt a bizonytalanságot méri, mint a jelenetszintű bootstrap, és hajlamos alulbecsülni a valódi bizonytalanságot. A 8.4. II. ellenőrző pont: korrekt tanulási állítás alfejezetben rögzített független teszt elve most mint a preregisztráció kötelező eleme tér vissza: a keresés lezárása után egyszer mérünk, és a becslés torzítatlansága a döntési szabály és a teszt szétválasztásán múlik.

36.1.9 Rövid összefoglaló

A kísérleti terv nem a futás utáni magyarázat, hanem a futás előtt rögzített szerződés. A hipotézis két konfigurációt, metrikát és irányt nevez meg; az abláció egyetlen komponenst izolál, a confound pedig olyan együtt mozgó változó, amely a hatást elmossa. A preregisztrált helyi terv rögzíti a kontrollt, a változót, a döntési szabályt és a compute budgetet, és kizárja a tesztkészlethez igazított utólagos célválasztást. A Grad-CAM példában a GAP-fejre vonatkozó azonosság (a csatornánkénti gradiensösszeg megegyezik a csatornaátlag szerinti deriválttal) és az ebből számított átlagolt súly analitikus, a lokalizációs állítás ezzel szemben kontrollt igénylő empirikus mérés. A következő egység az ismétlések és az eredménybizonytalanság kezelését vizsgálja.

36.1.10 Továbblépés

A 36.2. Ismétlések és eredménybizonytalanság alfejezetben az ismétlések, seed-szórás és konfidenciaintervallumok kerülnek elő: a most rögzített terv ott kapja meg a nyers mérési eredményekből számított bizonytalansági határokat.

36.1.11 Felhasznált források

- [P25] Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). *Sanity Checks for Saliency Maps*. arXiv:1810.03292. https://arxiv.org/abs/1810.03292.
- [P23] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D. (2016). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. arXiv:1610.02391. https://arxiv.org/abs/1610.02391.
- [P30] Ioffe, S., Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML 2015. arXiv:1502.03167. https://arxiv.org/abs/1502.03167.
- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm. Hozzáférés: 2026-10-30.

36.2 Ismétlések és eredménybizonytalanság

36.2.1 Cél, előismeret és induló kérdés

A 36.1. Állításból kísérleti terv egységben rögzített kísérleti terv most kapja meg a mérési bizonytalanság részét. A cél annak eldöntése, hogy két konfiguráció különbsége valódi hatás-e, vagy csupán a tanítási véletlen, azaz a seedek ingadozása. Szükséges előismeret a preregisztrált terv fogalma, a hypothesis, control és változó tényező megkülönböztetése. Induló becslés: öt seeddel futtatott kis CNN-en a legjobb és legrosszabb validációs pontosság különbsége gyakran több százalékpont; ha ennél kisebb hatást várunk, egyetlen futás nem dönt.

36.2.2 A független egység: a seed-futás

Egy **seed-futás** (seed run) egy teljes tanítási folyamat egy rögzített seeddel, változatlan kóddal, adatsplit-tel és hardveren. Az ismétlés **független egysége** (independent unit) itt a futás: a seedek egymástól független véletlen állapotokat indítanak, ezért az eredményeket egy közös „seed-populáció” mintájának tekintjük. Ez populációs idealizáció: az adott hardveren, adaton és kódon értelmes; más hardveren más populációról beszélünk.

A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab `scene_id`-je itt nem a független egység, mert a train/val/test split rögzített. A scene-alapú újramintavételezés az adatbizonytalanságot mérné; a seed-ismétlés a tanítási véletlent. A kettőt a szerződés szerint külön kell jelölni: a jelen egység csak a seed-szórást vizsgálja, a prediktív bizonytalanság és a bootstrap más fogalom.

36.2.3 Párosított seedek és a becsült különbség modellje

Két konfiguráció legyen A és B. **Párosított seedeken** (paired seeds) azt értjük, hogy ugyanazt a s_i seedet használjuk mindkét konfigurációhoz, így az i-edik pár (A_i, B_i) közös véletlen környezetet kap. A különbség

$$D_i = B_i - A_i, \qquad i = 1, \dots, m.$$

A mintavételi modell feltételei:

1. a párok függetlenek, tehát $D_1, \dots, D_m$ független azonos eloszlású;
2. D_i-nek véges μ_D várható értéke és $\sigma_D^2 > 0$ varianciája van;
3. a konfidenciaintervallumhoz a $D_i \sim N(\mu_D, \sigma_D^2)$ normális közelítést is feltesszük.

A párosítás előnye a varianciaképletből látszik. Ha $\rho = \mathrm{Corr}(A_i, B_i)$ a páron belüli korreláció, akkor

$$\mathrm{Var}(D_i) = \sigma_A^2 + \sigma_B^2 - 2\rho\,\sigma_A\sigma_B.$$

A párosított átlag becsült szórásnégyzete ezért S_D^2/m, míg független minták különbségénél $S_A^2/m + S_B^2/m$ állna. Pozitív ρ esetén — ami gyakori, mert egy „nehéz” seed mindkét konfigurációt rontja — a párosított becslés varianciája kisebb.

A becslők:

$$\bar{D}_m = \frac{1}{m}\sum_{i=1}^{m} D_i, \qquad S_D^2 = \frac{1}{m-1}\sum_{i=1}^{m}(D_i - \bar{D}_m)^2, \qquad \mathrm{SE} = \frac{S_D}{\sqrt{m}}.$$

Tétel (párosított hatásintervallum, normális modell). Ha $D_1, \dots, D_m$ független $N(\mu_D, \sigma_D^2)$ minta, akkor

$$T = \frac{\bar{D}_m - \mu_D}{S_D/\sqrt{m}} \sim t_{m-1},$$

ahol $t_{\nu,q}$ a q-kvantilist jelöli: $P(T \le t_{\nu,q}) = q$. Következésképpen a μ_D hatásra vonatkozó $100(1-\alpha)\%$-os konfidenciaintervallum

$$\bar{D}_m \pm t_{m-1,1-\alpha/2}\frac{S_D}{\sqrt{m}}.$$

A tétel standard valószínűségszámítási eredmény; a t-eloszlás a normális átlag és a független χ^2-eloszlású varianciabecslés hányadosából származik. A feltételeket és az ismételt mintavételi értelmezést a [S02] módszertani oldal a mintaátlag konfidenciaintervallumának általános esetére tárgyalja; a páros különbségekre való alkalmazás a jelen fejezet saját levezetése. A normalitás itt közelítés: a pontosság darabszámok hányadosa, ezért a D_i diszkrét értékű, és kis m mellett a t-intervallum tényleges lefedettsége sem egyezik pontosan a névleges szinttel. A korlátosság önmagában nem érv a normalitás ellen (korlátos eloszlás is lehet közel normális); a lényeges ok a diszkrétség és a kis minta. Kis m-re az intervallumot nem szabad precíziós garanciaként kezelni.

Egyoldalú és kétoldalú szabály. Ha a hipotézis előre rögzített irányú, például $H_1 : \mu_D > 0$, akkor a szinthez egyoldalú szabály tartozik: H_0-t akkor vetjük el $100\alpha\%$-os szinten, ha

$$\bar{D}_m - t_{m-1,1-\alpha}\frac{S_D}{\sqrt{m}} > 0,$$

azaz ha a $100(1-\alpha)\%$-os egyoldalú alsó határ pozitív. Ez ekvivalens azzal, hogy a kétoldalú $100(1-2\alpha)\%$-os intervallum alsó határa pozitív. A könyvben használt döntési szabály a **kétoldalú 95%-os** intervallum alsó határát kéri pozitívnak, ami egyoldalú értelemben $\alpha = 0{,}025$-ös szintnek felel meg: konzervatívabb az egyoldalú 0,05-ös szabálynál. A különbség nem elhanyagolható; a kézi példában a kétoldalú szabály negatív, az egyoldalú 0,05-ös szabály éppen pozitív eredményt

ad (lásd lentebb). A szintet ezért a futás előtt kell rögzíteni, és a jelentésben meg kell nevezni, melyik szabályt használtuk.

Az ismételt mintavételi értelmezés: ha a teljes, m seedes kísérletet sokszor megismételnénk új seedhalmazokkal, a kiszámított intervallumok körülbelül $1-\alpha$ része tartalmazná μ_D-t. Azt nem állítja, hogy az igazi hatás 95% valószínűséggel éppen a most kapott konkrét intervallumban van.

36.2.4 Hatásintervallum: eredeti kézi példa

Legyen A a súlyregularizálás nélküli, B az Adam `weight_decay=1e-4` beállítású futás (a 36.2 labor kódja Adamet használ; az AdamW decoupled súlycsökkentése nem azonos vele: ott a csillapítás lr·wd mértékben skálázódik); a metrika validációs pontosság arányban. Az öt párosított seed eredménye VÁRT ÉRTÉK (levezetésből, nem mért):

seed	A	B	$D_i = B_i - A_i$
1	0,80	0,81	+0,01
2	0,78	0,83	+0,05
3	0,84	0,85	+0,01
4	0,82	0,82	0,00
5	0,81	0,84	+0,03

A különbségvektor $m = 5$ eleme:

i	1	2	3	4	5
D_i	0,01	0,05	0,01	0,00	0,03

Átlaga $\bar{D}_5 = 0{,}020$. Az eltérések $D_i - \bar{D}_5 = (-0{,}01;\ 0{,}03;\ -0{,}01;\ -0{,}02;\ 0{,}01)$, négyzetösszegük 0,0016, így $S_D^2 = 0{,}0016/4 = 0{,}0004$, $S_D = 0{,}020$. A standard hiba $\mathrm{SE} = 0{,}020/\sqrt{5} \approx 0{,}008944$. A $t_{4,0{,}975} = 2{,}776$ kritikus értékkel a 95%-os hatásintervallum

$$0{,}020 \pm 2{,}776 \cdot 0{,}008944 = [-0{,}0048;\ 0{,}0448].$$

A D_i és a CI határai dimenzió nélküli arányok; szorozva 100-zal kapjuk a százalékpontos skálát: −0,48, illetve +4,48 százalékpont. A kétoldalú intervallum tartalmazza a 0-t, tehát a kétoldalú 95%-os szabály szerint ez **negatív eredmény** az előre rögzített „B jobb, mint A” hipotézisre nézve: a hatást ezzel az öt seeddel nem tudjuk megkülönböztetni a seed-zajtól. A felső határ körülbelül +4,48 százalékpont: e küszöbnél nagyobb pozitív hatások **nem szerepelnek a közölt 95%-os konfidenciaintervallumban** a modellfeltevések mellett. Ugyanakkor a minta a +3 százalékpontos javulást sem zárja ki. **Az egyoldalú összevetés:** az egyoldalú 95%-os alsó határ $0{,}020 - 2{,}132 \cdot 0{,}008944 = +0{,}0009$, ami éppen pozitív, tehát az egyoldalú 0,05-ös szabály elfogadná a „B jobb, mint A” hipotézist. A példa így közvetlenül mutatja, hogy az egy-/kétoldalúság és a szint előzetes rögzítése érdemben megváltoztathatja a döntést; a konzervatív, kétoldalú szabályt használjuk, és ezt a protokollban rögzíteni kell.

36.2.5 Negatív eredmény és annak határa

A **negatív eredmény** (negative result) definíciója: az előre specifikált hatásirányra a konfidenciaintervallum tartalmazza a nullát, ezért a nullhipotézist nem vetjük el. Ez hiányzó bizonyíték (absence of evidence), nem a hatás hiányának bizonyítéka (evidence of absence). Ha a gyakorlatban csak a δ küszöbnél nagyobb javulás érdekes, akkor ekvivalencia-intervallumot vagy a CI és δ összevetését kell protokollba foglalni; a fenti intervallum például nem zárja ki a $+0{,}03$ hatást sem.

A negatív eredményt ugyanúgy rögzíteni kell, mint a pozitívat: nyers táblázat, CI, kód és környezet. A kísérleti naplóban a negatív eredmény nem kudarc, hanem a keresési tér egy pontjának korrekt kiértékelése.

36.2.6 Többszörös összehasonlítás

Ha k konfigurációt vagy több, utólag kiválasztott hipotézist hasonlítunk, és a legkedvezőbb eredményt emeljük ki, a nyers intervallum már nem érvényes kiválasztáskorrekció nélkül. A seedek mint ismétlések száma önmagában nem jelent többszörös hipotézistesztet; akkor lép fel kiválasztási torzítás, ha a legkedvezőbb seed eredményét választjuk ki. A torzítás oka: a kiválasztás maga is véletlen változó. Ugyanez a helyzet, ha több hipotézist tesztelünk ugyanazon a kiértékelő halmazon. A **családi hibavalószínűség** annak valószínűsége, hogy legalább egy igaz nullhipotézist tévesen elvetünk.

Lemma (Boole–Bonferroni). Tetszőleges $E_1, \dots, E_k$ eseményekre

$$P\left(\bigcup_{j=1}^{k} E_j\right) \le \sum_{j=1}^{k} P(E_j).$$

Bizonyítás. Két eseményre $P(E_1 \cup E_2) = P(E_1) + P(E_2) - P(E_1 \cap E_2) \le P(E_1) + P(E_2)$, mert a metszet valószínűsége nemnegatív; k-ra teljes indukcióval. Ha minden egyedi teszt szintje α/k, akkor az unió valószínűsége legfeljebb $k \cdot (\alpha/k) = \alpha$. $\square$

Példa: ha $k = 5$ összehasonlítást végzünk és $\alpha = 0{,}05$ családi szintet akarunk, az egyedi intervallumhoz $\alpha' = 0{,}01$ tartozik. A kézi példában $t_{4,0{,}995} = 4{,}604$, tehát a korrigált intervallum

$$0{,}020 \pm 4{,}604 \cdot 0{,}008944 = [-0{,}0212;\ 0{,}0612].$$

A Bonferroni-korrekció konzervatív, de garantált felső korlát. A helyes gyakorlat a fő összehasonlítás előzetes kijelölése; a többi elemzés feltáró jellegét külön kell jelenteni.

36.2.7 Labor: párosított seed-ismétlés a GlyphLab kis CNN-jén

A labor a 36.1. Állításból kísérleti terv tervét követi: a változó tényező a súlyregularizálás, a kontroll A a `weight_decay=0`, a vizsgált B a `weight_decay=1e-4`; a metrika validációs pontosság; a döntési szabály a fenti hatásintervallum. A seedlista előre rögzített: `20261020, 20261021, 20261022, 20261023, 20261024`. A főút hardvere: CPU, `float32`, NCHW `[0,1]` bemenet, `int64` címke. A számítási költségkeret (compute budget): 5 seed $\times 2$ konfiguráció $\times 5$ epoch, kis CPU-n néhány perc.

Inputok: `data/images.npy`, `data/labels.npy`, `data/metadata.csv`. Futtatás előtt a `python quickcheck.py` ellenőrzi a hash-t, alakot és splitet. A következő szkript munkakönyvtára `lab/`:

```
# munkakönyvtár: lab/
# Futtatás: python n142_seed_repeats.py --seeds 20261020,20261021,20261022,20261023,20261024
import argparse, csv
from pathlib import Path
import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, Subset, TensorDataset
from scipy.stats import t as student_t

DATA = Path("data")

def load_split():
    tr, va = [], []
    with open(DATA / "metadata.csv", newline="") as f:
        rows = list(csv.DictReader(f))
    assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(rows[0])
    assert len(rows) == 360, len(rows)
    assert [int(r["index"]) for r in rows] == list(range(len(rows)))
    for row in rows:
        i = int(row["index"])
        if row["split"] == "train":
            tr.append(i)
        elif row["split"] == "val":
            va.append(i)
    return np.asarray(tr, dtype=np.int64), np.asarray(va, dtype=np.int64)

class TinyCNN(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(16 * 6 * 6, n_classes))
    def forward(self, x):
        return self.net(x)

def train_eval(seed, X, y, tr_idx, va_idx, weight_decay, n_classes,
               epochs=5, lr=1e-3):
    # Azonos seed + azonos architektúra + azonos hívási sorrend mellett az
    # inicializálás és a DataLoader keverési sorrendje bitre azonos; ez adja
    # a párosítást. A seed önmagában nem garantál párosítást, ha a két ág
    # RNG-fogyasztása eltér (pl. dropout).
    torch.manual_seed(seed)
    model = TinyCNN(n_classes)
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    ds = TensorDataset(torch.from_numpy(X), torch.from_numpy(y))
    dl = DataLoader(Subset(ds, tr_idx), batch_size=16, shuffle=True)
    for _ in range(epochs):
        model.train()
        for xb, yb in dl:
            opt.zero_grad()
            loss = nn.functional.cross_entropy(model(xb), yb)
            loss.backward()
            opt.step()
    model.eval()
    with torch.no_grad():
        xv = torch.from_numpy(X[va_idx])
        yv = torch.from_numpy(y[va_idx])
        acc = (model(xv).argmax(1) == yv).float().mean().item()
    return acc
```

```
def paired_ci(diffs, alpha=0.05):
    diffs = np.asarray(diffs, dtype=np.float64)
    m = diffs.size
    mean_d = float(diffs.mean())
    sd = float(diffs.std(ddof=1))
    se = sd / np.sqrt(m)
    tcrit = float(student_t.ppf(1 - alpha / 2, df=m - 1))
    return mean_d, se, mean_d - tcrit * se, mean_d + tcrit * se, m - 1

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--seeds", default="20261020,20261021,20261022,20261023,20261024")
    ap.add_argument("--epochs", type=int, default=5)
    ap.add_argument("--alpha", type=float, default=0.05)
    ap.add_argument("--out", default="")
    args = ap.parse_args()
    seeds = [int(s) for s in args.seeds.split(",")]
    X = np.load(DATA / "images.npy").astype(np.float32)  # (360,1,24,24)
    y = np.load(DATA / "labels.npy").astype(np.int64)    # (360,)
    assert len(X) == len(y) == 360, (X.shape, y.shape)
    assert X.ndim == 4 and X.shape[1:] == (1, 24, 24), X.shape
    assert X.min() >= 0.0 and X.max() <= 1.0, (float(X.min()), float(X.max()))
    n_classes = int(y.max()) + 1
    assert n_classes == 3, n_classes
    tr_idx, va_idx = load_split()
    rows = []
    for s in seeds:
        a = train_eval(s, X, y, tr_idx, va_idx, 0.0, n_classes, epochs=args.epochs)
        b = train_eval(s, X, y, tr_idx, va_idx, 1e-4, n_classes, epochs=args.epochs)
        rows.append((s, a, b, b - a))
        print(f"seed={s} A={a:.4f} B={b:.4f} d={b-a:+.4f}")
    mean_d, se, low, high, df = paired_ci([r[3] for r in rows], args.alpha)
    print(f"mean d={mean_d:+.4f} SE={se:.4f} df={df} CI=[{low:.4f},{high:.4f}]")
    if args.out:
        with open(args.out, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["seed", "A_acc", "B_acc", "d_B_minus_A"])
            w.writerows(rows)

if __name__ == "__main__":
    main()
```

A következő kimenet mért futás eredménye (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 10 szál; seedek 20261020–20261024, 5 epoch, batch 16, Adam, lr 10^{-3}):

```
seed=20261020 A=0.5000 B=0.5000 d=+0.0000
seed=20261021 A=0.6667 B=0.6667 d=+0.0000
seed=20261022 A=0.4833 B=0.5167 d=+0.0333
seed=20261023 A=0.5500 B=0.5333 d=-0.0167
seed=20261024 A=0.5333 B=0.5333 d=+0.0000
mean d=+0.0033 SE=0.0082 df=4 CI=[-0.0193,0.0260]
```

Értelmezés: a nyers táblázat mind az öt sort közli, nem csak a legjobb seedet. A CI tartalmazza a nullát, tehát a weight decay hatása ezzel a protokollal nem különböztethető meg a seed-zajtól; a közölt 95%-os intervallum a +2,60 százalékpontnál nagyobb javulást nem tartalmazza, miközben a mérsékelt javulás és a nulla hatás is összeegyeztethető vele a feltevések mellett. A mért validációs pontosságok ezen a rövid, ötepochos protokollon a véletlen szint körül vannak, ami a kísérlet érzékenységét korlátozza; a tanulság az, hogy a hatás kimutatásához hosszabb tanítás vagy nagyobb minta kell. A hardver szerepe itt rögzített: a 24.2. CPU/GPU profiling és helyes időmérés alfejezetben tanult mérési logikához hasonlóan a hardver a protokoll része, és a

CPU/GPU vagy AMP-váltás nem a seed-szórás része, hanem külön környezeti tényező, amelyet külön mátrixban kell jelenteni.

36.2.8 Feladatok

F1 (fogalmi, nehézség 1). A GlyphLab train splitje 120 jelenetből származó 240 képet tartalmaz, és egy kísérlet öt seeddel tanít ugyanazon a spliten. Mi a független egység a seed-ismétlésben, és miért nem a kép vagy a jelenet? *Beadandó:* két-három mondatos indoklás a három jelölt megkülönböztetésével.

F2 (levezetési, nehézség 2). Adott $m = 6$ párosított seed különbsége, $D_i = B_i - A_i$:

i	1	2	3	4	5	6
D_i	−0,02	−0,01	0,00	−0,03	−0,01	−0,04

Számítsa ki a 95%-os hatásintervallumot, és döntsön a „B jobb, mint A” előre rögzített hipotézisről. *Beadandó:* $\bar{D}$, S_D, SE, t-kritikus érték, CI és egy mondatos döntés.

F3 (implementációs, nehézség 2). Írjon egy `paired_effect(diffs, alpha)` függvényt, amely `float64` bemenetből az átlagot, standard hibát és a kétoldali CI határait adja vissza; a t-kvantilishez használhatja a `scipy.stats.t.ppf` függvényt. Ellenőrizze az F2 példáján. *Beadandó:* futtatható kód, a visszatérési értékek, shape-ellenőrzés: `diffs` alakja `(m,)`, az eredmény négy skalár, a df `m-1`.

F4 (hibakeresési, nehézség 2). Valaki ezt a kódot írja:

```
sd = np.std(diffs)      # a population-szórás
se = sd / np.sqrt(len(diffs))
```

Mi a hiba, mi a következménye a CI szélességére, és hogyan javítandó? *Beadandó:* a hiba oka, iránya és a javított sor, majd a visszaellenőrzés az F2 adatain.

F5 (kísérleti/transzfer, nehézség 3). Tervezze meg három konfiguráció — baseline, dropout 0,2, weight decay $1e-4$ — összehasonlítását öt párosított seeddel a GlyphLab-on. Adja meg a fő összehasonlítást, a többszörös összehasonlítás kezelését, a negatív eredmény definícióját és a nyers beadandó táblázatot. *Beadandó:* protokoll-leírás, indoklás és következtetési korlát.

36.2.9 Megoldások és indoklás

F1. A független egység a seed-futás, mert az ismétlés a tanítási véletlent változtatja, miközben a split rögzített. A kép nem független, mert minden futás ugyanazokat a képeket látja; a jelenet sem, mert a scene-struktúra az adatmintavétel egysége lenne, de itt nem sorsolunk új splitet. Ha új splitet is sorsolnánk, a független egység a `(seed, split)` pár lenne.

F2. Részletes számítás:

$$\sum D_i = -0{,}11, \qquad \bar{D}_6 = -0{,}11/6 \approx -0{,}018333.$$

Az eltérések $D_i - \bar{D}$ rendre −0,001667, 0,008333, 0,018333, −0,011667, 0,008333, −0,021667; négyzetösszegük $\approx 0{,}0010833$. Így

$$S_D^2 = \frac{0{,}0010833}{5} \approx 0{,}00021667, \qquad S_D \approx 0{,}01472.$$

A standard hiba $\mathrm{SE} = 0{,}01472/\sqrt{6} \approx 0{,}006009$. A kritikus érték $t_{5,0{,}975} = 2{,}5706$. A CI:

$$-0{,}018333 \pm 2{,}5706 \cdot 0{,}006009 = [-0{,}03378;\ -0{,}00288].$$

A kétoldalú 95%-os páros t-intervallum (df=5) nem tartalmazza a 0-t, ezért a kétoldalú 5%-os páros t-próba elveti a $\mu_D = 0$ nullhipotézist. A jelölt irány szerint $D = B - A$, és a teljes intervallum negatív, tehát a minta szerint B rosszabb, mint A; az egyoldalú, előre rögzített „B jobb, mint A” irányra ez a példa nem ad támogatást. Az egyoldalú 95%-os alsó határ ugyanezt erősíti meg: $\bar{D}_6 - t_{5,0,95} \cdot \mathrm{SE} = -0{,}018333 - 2{,}015 \cdot 0{,}006009 \approx -0{,}03044 < 0$, tehát az egyoldalú szabály sem utasítja el a nullhipotézist az irányított hipotézis javára.

F3. Egy lehetséges megoldás:

```
import numpy as np
from scipy.stats import t as student_t

def paired_effect(diffs, alpha=0.05):
    d = np.asarray(diffs, dtype=np.float64)   # alak: (m,)
    m = d.shape[0]                             # skalár
    mean_d = d.mean()                          # skalár
    sd = d.std(ddof=1)                         # skalár
    se = sd / np.sqrt(m)                       # skalár
    tcrit = student_t.ppf(1 - alpha / 2, df=m - 1)
    return mean_d, se, mean_d - tcrit * se, mean_d + tcrit * se

diffs = [-0.02, -0.01, 0.00, -0.03, -0.01, -0.04]
mean_d, se, low, high = paired_effect(diffs)
print(mean_d, se, low, high)
# -0.018333... 0.006009... -0.03378 -0.00288
```

Ellenőrzés: `diffs` hossza 6, a df 5; a négy visszatérési érték skalár; a CI határai 10^{-3} abszolút toleranciával egyeznek az F2 értékeivel.

F4. Az `np.std` alapértelmezett `ddof=0` értéke a populációs szórást adja, amely $1/m$-es osztást használ; a mintából becsült szóráshoz $1/(m-1)$ osztás kell. Következmény: a szórás és a standard hiba kisebb a szükségesnél, ezért a CI túl szűk, és a hamis pozitív arány megnő. Javítás: `sd = np.std(diffs, ddof=1)`. Visszaellenőrzés az F2 adatain: `ddof=0` esetén $S_D \approx 0{,}01344$, `ddof=1` esetén 0,01472; a helyes CI szélesebb.

F5. Protokollminta: a fő összehasonlítás a dropout 0,2 és a baseline között párosított seedekkel, öt előre rögzített seeddel. Mivel ez a fő összehasonlítás a futás előtt kijelölt, a családi szintű korrekciót nem igényel: a fő CI kétoldalú 95%-os. A weight decay csak másodlagos, feltáró elem; a rá vonatkozó két további összehasonlítás (weight decay vs baseline, weight decay vs dropout) alkotja a másodlagos családot, amelyre a Bonferroni-korrekció $\alpha' = 0{,}05/2 = 0{,}025$ egyedi szintet ad, és az így kapott intervallumokat feltáró jellegűként kell jelenteni, nem megerősítő bizonyítékként. A beadandó nyers táblázat minden seedre három pontosságot és a fő különbséget tartalmazza. Negatív eredmény: a fő CI tartalmazza a nullát, tehát a dropout hatását nem tudjuk megkülönböztetni a seed-zajtól. Következtetési korlát: a GlyphLab adat nem természetes képi benchmark, és a hardver CPU/float32; GPU-ra vagy más adatra az eredmény nem vihető át bizonyítás nélkül. A konfigurációk architektúrája a labor TinyCNN-je (két konvolúció és $16{\cdot}6{\cdot}6 \to 3$ lineáris fej), a recept Adam, $\eta = 10^{-3}$, batch 16, 5 epoch; a seedek 20261020–20261024, az erőforrás CPU, 10 szál, 15 futás.

36.2.10 Tipikus tévedés ellenpéldával

Gyakori hiba, hogy tíz seedből csak a legjobbat jelentik, és azt állítják: „a módszer pontossága 0,91”. Ellenpélda: a tíz nyers pontosság legyen 0,80; 0,81; 0,80; 0,84; 0,82; 0,83; 0,80; 0,82; 0,81; 0,91; a legjobb 0,91, de az átlag $\approx$ 0,824, és a tapasztalati szórás alapján a seed-átlag CI-ja nem tartalmazza a 0,91-et. A kiválasztott futás a maximum statisztikája, amelynek eloszlása felfelé torzított; a stabil teljesítmény jellemzéséhez az átlagot és a hatásintervallumot kell közölni.

36.2.11 Önellenőrzés és visszatekintés

A kézi számítás ellenőrzése: a df mindig $m-1$, az ötös példában 4; a standard hiba $1/\sqrt{m}$ szerint csökken, ezért a CI szélessége közelítőleg $1/\sqrt{m}$ ütemben szűkül, de a t-kritikus érték is csökken m növekedésével, így a szűkülés valamivel gyorsabb ennél; a t-kritikus érték nagyobb a normális 1,96-nál kis m-re. A 35.2. Magyarázat sanity check alfejezetben tanult általános tanulság itt részben tér vissza: a randomizálás csak cáfolni tud, bizonyítani nem. A seedek közötti szórás a futásbizonytalanságot jellemzi, és önmagában nem teszteli a magyarázat modellfüggőségét; a [P25] kontrollja ezért itt nem helyettesíthető a seed-randomizálással. A 32.2. Scaled dot-product attention alfejezetben használt alakellenőrzés itt is kötelező: a különbségvektor alakja $(m,)$, a standard hiba skalár. A 24.2. CPU/GPU profiling és helyes időmérés alfejezetben rögzített mérési fegyelem szerint a hardvert expliciten rögzítjük, és a hardverváltást nem számoljuk bele a seed-szórásba.

36.2.12 Rövid összefoglaló

A seed-ismétlés független egysége a teljes futás, nem a kép vagy a jelenet. Párosított seedekkel azonos véletlen környezetben mérjük a két konfigurációt, és a páronkénti különbségek átlagát, standard hibáját, valamint t-alapú hatásintervallumát közöljük. A nulla befoglalása negatív eredmény: hiányzó bizonyíték, nem a hatás hiányának bizonyítéka. Több összehasonlítás esetén a családi hiba a Boole–Bonferroni korláttal kontrollálható. A labor öt előre rögzített seeddel, CPU/float32 főúton tanít kis CNN-t, és a nyers futásokat teljes táblázatban adja át.

36.2.13 Továbblépés

A 36.3. Szakirodalmi reprodukció és attribúció a szakirodalmi reprodukciót és attribúciót vizsgálja: a „reprodukáltuk” állítás ott csak akkor lesz megalapozott, ha a cikk protokollja mellett a seed-ismétlést, a nyers futásokat és a hatásintervallumot is közöljük a forrás pontos megjelölésével.

36.2.14 Felhasznált források

- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm. Hozzáférés: 2026-10-30.
- [P25] Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., & Kim, B. (2018). *Sanity Checks for Saliency Maps*. arXiv:1810.03292. https://arxiv.org/abs/1810.03292.

36.3 Szakirodalmi reprodukció és attribúció

36.3.1 Cél, előismeret és indító kérdés

Ebben az egységben azt a munkafolyamatot rögzítjük, amellyel egy szakirodalmi eredményt szabályosan idézünk, adaptálunk vagy reprodukálunk. A cél nem retorikai: az olvasó az egység végén minden állításához meg tudja nevezni, melyik cikk, melyik **változata**, melyik **szakasza** tartalmazza, és hogy az illető helyet ténylegesen olvasta-e. Előismeretnek a 36.2. Ismétlések és eredménybizonytalanság eredménybizonytalansági nyelve, a 35.3. Perturbáció és adversarial példa FGSM-levezetése, a 32.3. Vision Transformer és hibrid felhasználás architektúra-összehasonlítási minta és a 24.3. Mixed precision és nagyobb batch numerikus adatrögzítési igénye szükséges.

Indító kérdés: csak egy cikk absztraktját ellenőrizted. Melyik mondat a legerősebb, amelyet bizonyítékkal alá tudsz támasztani a módszeréről? A várható válasz: legfeljebb „a szerzők az absztraktban azt állítják, hogy…”, semmi több; az egység ezt a különbséget teszi mérhetővé.

36.3.2 Definíciók, jelölések és a reprodukciós tárgy

Elsődleges cikk (primary paper). Az a publikált vagy preprint munka, amelynek konkrét állítását, módszerét vagy számszerű eredményét vizsgáljuk. A hivatkozásban a **változat (version)** rögzítendő: arXiv-kéziratnál az azonosító és a `v` verziószám, konferenciaváltozatnál a kiadói rekord. Az arXiv-kézirat éve nem feltétlenül azonos a konferenciamegjelenés évével; ezt az egység könyvszintű szabályként rögzíti.

Szakasz (section). Az a részegység (szakasz, tétel, táblázat, ábra, függelék vagy definíció), amelyből egy átvett állítás származik. Egy állítás erőssége csak azzal a szakasszal együtt értelmes, amelyet ténylegesen olvastunk. Az [S01] jegyzetlap esetében például a 14.1.–14.4. egységek a konvolúciós alakokról szóló részt használták; ha valaki onnan egy hiperparamétert idéz, a pontos alszakaszt is idéznie kell. **Eredeti ábra (original figure):** a cikkben közölt grafikus megjelenítés. Reprodukcióban újrarajzolhatjuk a közölt adatokból vagy saját futtatásból, de a képfájlt engedély nélkül nem másoljuk; az új ábrán jelöljük a forrást és a „replot” (újrarajzolt) jelleget.

Olvasási/bizonyítékstátusz. Rögzítjük a bizonyítékok rendezett halmazát $\Sigma = \{\text{rekord; absztrakt; szakasz; definíció; reprodukált}\}$, ahol a felsorolás egyben a rendezés: egy elem annál erősebb, minél mélyebben ellenőriztük a kijelentést, azaz

$$\text{rekord} < \text{absztrakt} < \text{szakasz} < \text{definíció} < \text{reprodukált}.$$

Egy állítás – az „olvasási igénymátrix” (claim ledger) sora – a következő négyes: $(C, s, \sigma, a_{\max})$, ahol C a kijelentés, s a hivatkozott szakasz, $\sigma \in \Sigma$ az olvasottság, $a_{\max}$ pedig a **legerősebb korrekten kijelenthető forma**. A $\sigma \leq$ absztrakt feltétel esetünkben azt jelenti, hogy $\sigma \in \{\text{rekord; absztrakt}\}$; ilyenkor $a_{\max}$ csak a szerzői állításról szóló idézet lehet, a világ tényleges állapotáról szóló kijelentés nem.

Toy (játékméretű) adaptáció. A módszer matematikai magjának kisméretű, saját adaton futó megvalósítása, amely egy szűk mechanizmust ellenőriz. Az érvényességi köre pontosan a toy-tartomány; az eredeti cikk általános empirikus állítását nem igazolja. **Reprodukció és replikáció** – könyvünkben lokális munkadefinícióval dolgozunk, mert a szakterületek eltérő szótárat használnak. Reprodukció: ugyanazt az operatív protokollt (modell, adatfelosztás, metrika, tréningrecept) saját kóddal újra lefuttatjuk, és a számszerű kimenetet megadott toleranciával összehasonlítjuk ($|R_{\text{ours}} - R_{\text{paper}}| \leq \tau$), vagy dokumentált negatív eltérést jelentünk. Replikáció: valamely protokoll-összetevőt (adat, implementáció, metrika) szándékosan megváltoztatunk, és

azt vizsgáljuk, hogy a tudományos következtetés túléli-e a változtatást. A toy adaptáció egyik kategóriának sem szinonimája.

36.3.3 Levezetési kötelezettség: egy ismert eredmény feltételei és a saját eltérések

A 35.3. Perturbáció és adversarial példa az FGSM motivációját elsőrendű közelítésből vezette le; itt ezt az ismert, könyvön belüli eredményt egzakt lineáris lemmává emeljük, majd feltételeit a saját újrafelhasználás feltételeivel tételesen összehasonlítjuk.

Lemma (FGSM-lineáris lépés optimuma). Legyen $g \in \mathbb{R}^d$, $\epsilon > 0$, és legyen $\delta^* = \epsilon \operatorname{sign}(g)$ komponensenként, $\operatorname{sign}(0) = 0$ konvencióval. Ekkor minden $\|\delta\|_\infty \le \epsilon$ megengedett perturbációra

$$g \cdot \delta \;\le\; g \cdot \delta^* \;=\; \epsilon \sum_{i=1}^{d} |g_i| \;=\; \epsilon \, \|g\|_1 .$$

Egyenlőség pontosan akkor áll, ha minden $g_i \neq 0$ indexre $\delta_i = \epsilon \operatorname{sign}(g_i)$; a $g_i = 0$ koordináták tetszőleges $\delta_i \in [-\epsilon;\ \epsilon]$ értéket vehetnek fel. A maximalizáló tehát pontosan akkor egyértelmű, ha g minden komponense nullától különböző.

Bizonyítás. (1) Minden megengedett δ-ra $g \cdot \delta = \sum_i g_i \delta_i \le \sum_i |g_i|\,|\delta_i|$, mert minden tag legfeljebb az abszolútértékek szorzata. (2) A dobozfeltétel miatt $|\delta_i| \le \epsilon$, így $\sum_i |g_i|\,|\delta_i| \le \epsilon \sum_i |g_i| = \epsilon \|g\|_1$. (3) A megadott δ^* mindkét egyenlőtlenséget tagról tagra egyenlőséggel teljesíti: $g_i \delta_i^* = \epsilon |g_i|$ minden i-re. (4) Ha δ eléri a $\epsilon\|g\|_1$ korlátot, akkor a $\sum_i$-beli egyenlőség csak úgy állhat, ha minden egyes tag egyenlő a felső korlátjával, mert egyetlen tag sem lehet nagyobb annál. Tagra tehát egyszerre kell teljesülnie, hogy $g_i \delta_i = |g_i|\,|\delta_i|$ és hogy $|\delta_i| = \epsilon$. Az első feltétel $g_i \neq 0$ esetén pontosan akkor áll, ha δ_i és g_i előjele azonos, azaz $\delta_i = \epsilon \operatorname{sign}(g_i)$ (mert $|\delta_i| = \epsilon$); $g_i = 0$ esetén a tag értéke 0, és $\delta_i \in [-\epsilon;\ \epsilon]$ tetszőleges. Ezzel megkaptuk az állított egyenlőségi feltételt. □

Az FGSM irányú lépés e lemma szerint a becsült veszteségnövekedést a megengedett dobozon maximalizálja – ha a linearizáció érvényes. Az eltérések tételes összevetése:

Komponens	Eredeti levezetés (35.3. Perturbáció és adversarial példa)	Saját újrafelhasználás itt
Cél	veszteség növelése	ugyanaz
Fenyegetési modell	ℓ_∞-doboz, ϵ	ugyanaz
Simaság	folytonos, deriválható veszteség	ReLU-töréspontoknál a linearizáció érvényét veszti; a lemma maga csak a lineáris tagra vonatkozik
Állítástípus	motivációs elsőrendű közelítés	bizonyított véges dimenziós optimum

A második összevetés a jelen egység irodalmi esettanulmányára vonatkozik. A [P25] cikk esetében csak az arXiv-rekord és az absztrakt szintje ellenőrzött, ezért a könyv nem a szerzők konkrét protokollját, hanem az absztraktban jelzett kérdés **saját toy adaptációját** definiálja:

Komponens	A cikkben jelzett (absztrakt szint)	Saját toy adaptáció
Tárgy	szaliencia-térképek paraméterérzékenységi sanity checkje	bemenetgradiens-nagyság egy kis GlyphLab CNN-en (a függelék A. A GlyphLab adatkészlet szakasza szerint)
Beavatkozás	paraméter-randomizálás jelzett gondolata	rétegenkénti, normatartó súlyscramble (a súlyok véletlen, azonos normájú cseréje)
Metrika	cikkbeli konkrét metrika nem olvasott	koszinusz-hasonlóság az 576 dimenzióba lapított szaliencia-vektorokon
Kijelenthető eredmény	csak a szerzői kérdés neve	csak a toy-tartományra érvényes viselkedés

36.3.4 Kézi példa numerikus kontrollal

Legyen $g = (3;\ -2;\ 0{,}5)$ (kiszámított gradienshármas), $\epsilon = 0{,}1$. Ekkor $\delta^* = 0{,}1 \cdot (1;\ -1;\ 1) = (0{,}1;\ -0{,}1;\ 0{,}1)$. A becsült növekedés $g{\cdot}\delta^* = 3{\cdot}0{,}1 + (-2){\cdot}(-0{,}1) + 0{,}5{\cdot}0{,}1 = 0{,}3 + 0{,}2 + 0{,}05 = 0{,}55$. A lemma szerinti felső korlát $\epsilon\|g\|_1 = 0{,}1 \cdot (3 + 2 + 0{,}5) = 0{,}55$; a kettő egyezik, tehát a δ^* tényleg maximalizáló. Ellenőrző ellenpélda irányban: a $(0{,}1;\ 0{,}1;\ 0{,}1)$ perturbáció g-vel vett szorzata $0{,}3 - 0{,}2 + 0{,}05 = 0{,}15 < 0{,}55$.

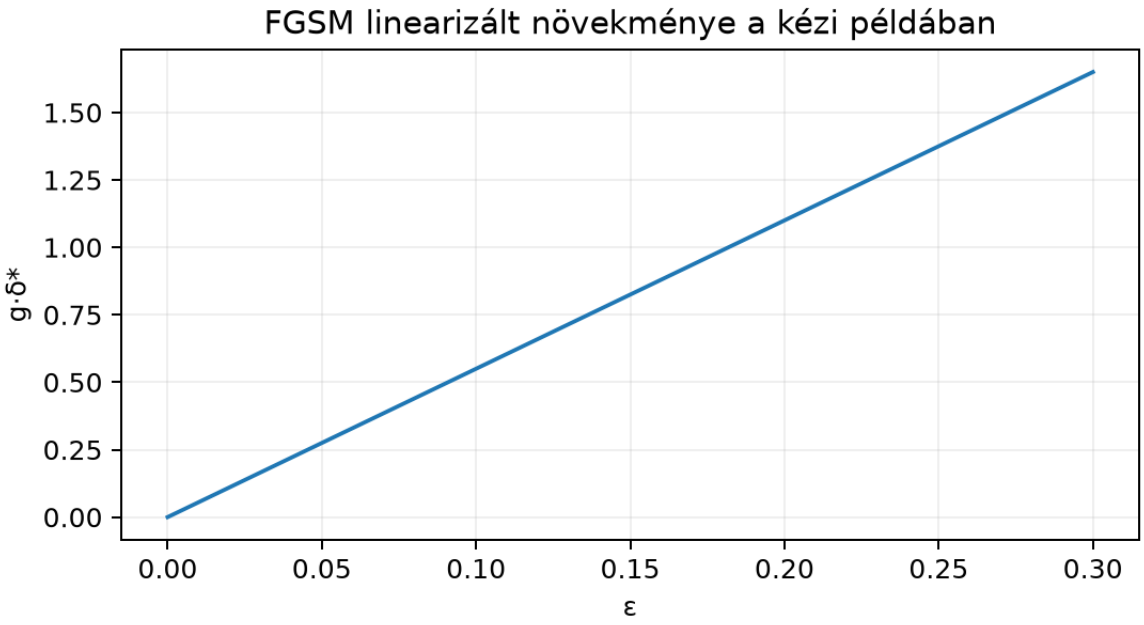
Minimális kód (munkakönyvtár: a tananyag gyökerének `lab/` könyvtára):

```
import numpy as np

g = np.array([3.0, -2.0, 0.5], dtype=np.float32)
eps = 0.1
delta = (eps * np.sign(g)).astype(np.float32)
gain = float(np.dot(g, delta))
korlat = float(eps * np.abs(g).sum())
print(delta)   # [ 0.1 -0.1  0.1]
print(gain, korlat)  # 0.55 0.55 (float32-ben 0.550000011920929)
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU): `[ 0.1 -0.1  0.1]`, majd `0.550000011920929 0.550000011920929` (a float32 számítás kerekítésével; a kézi érték 0,55).

A számított összefüggés ábrája; mivel a függvény lineáris, öt pont elég a jellegéhez:



36.1. ábra: FGSM linearizált növekménye a kézi példában

A g=(3; -2; 0,5) vektorból a lemma képletével számolt szemléltetés, nem mérés.

36.3.5 Labor: cikk-kivonat és reprodukciós terv, saját ábrával

Beadandó labortermék. (a) Kitöltött cikk-kivonat a lentebbi sablon szerint; (b) olvasási igénymátrix; (c) egy toy-reprodukciós tervprotokoll; (d) a fenti, képletből számolt saját ábra. A labor tudatosan **nem** állít teljes tanulmányolvasást; a hiányzó részek a kivonatban szerepelnek.

Sablon kitöltve a [P25] forrásra:

```yaml
cim: "Sanity Checks for Saliency Maps"
szerzok: "Adebayo et al."
azonosito: "arXiv:1810.03292"
verzio: "nincs rögzítve (ellenőrizendő)"
ev: 2018
eleres: "https://arxiv.org/abs/1810.03292"
olvasasi_datum: "2026-10-20"   # formátumpélda, a tényleges olvasás dátumával helyettesítendő
státusz:
  - "cím, absztrakt és keresési kivonat: ellenőrizve"
  - "verziószám: nem rögzített - nyitott auditpont"
  - "módszertani szakasz, kísérleti táblák, függelék: nem olvasva"
sajat_kivonat: >
  A cikk absztraktszinten a szaliencia-térképek sanity checkjeinek
  kérdését veti fel; a precíz beavatkozási protokoll és metrika
  ebben a kivonatban nem állítható.
felhasznalhato:
  - "a vizsgált probléma megnevezése"
  - "saját toy adaptáció motivációja"
nem_allithato:
  - "a cikk konkrét tesztprocedúrájának reprodukciója"
  - "a cikk minden következtetésének ellenőrzöttsége"
```

A sablon szándékosan mutatja a hiányt: a saját szabály szerint a hivatkozásnak verziószámot kell tartalmaznia, és amíg ez nincs ellenőrizve, a hivatkozás nem teljes – ezért a `verzio` mező

és a `státusz` külön auditpontként jelöli. Az `olvasasi_datum` értéke formátumpélda, nem valós olvasási dátum: a tényleges naplóban a valódi olvasás napja szerepel.

Olvasási igénymátrix (négy oszlop):

Állítás	Forráshely	Státusz	Maximum kijelenthető
A cikk sanity checkeket javasol szaliencia-térképekhez	absztrakt, arXiv-rekord	absztrakt olvasva	„a szerzők az absztraktban…”
Konkrét randomizálási protokoll	2–4. szakasz	nem olvasva	nem állítható
Konkrét metrika és adat	kísérleti szakasz	nem olvasva	nem állítható
Saját toy metrika és beavatkozás	e könyv	levezetett	a toy-kísérletre érvényes

Toy-reprodukciós terv (protokollterv, nem mért eredmény). Bemenet: `lab/data/images.npy`, `labels.npy`, split a metadata-ból; $N\times1\times24\times24$ float32, címke int64. Modell: $\mathrm{Conv2d}(1,8,3,p=1)-\mathrm{ReLU}-\mathrm{MaxPool}(2)-\mathrm{Conv2d}(8,16,3,p=1)-\mathrm{ReLU}-\mathrm{AdaptiveAvgPool}(1)-\mathrm{Linear}(16;\,3)$, CPU, seed `20261020`; a tanított modellt f jelöli, kimenete három logit. Szaliencia: legyen y^* a kiértékelt testkép valódi címkéje és L_{CE} a keresztentrópia-veszteség; a gradiens abszolút értéke pixelenként $s_{uv}=|\partial L_{CE}(f(X),y^*)/\partial X_{1,u,v}|$. A bemenetnek egy csatornája van, ezért a csatornadimenzió elhagyható, és nincs mit átlagolni rajta; az eredmény 24×24-es, soronkénti lapítás után $s\in\mathbb{R}^{576}$. Beavatkozás a bemenet felől a kimenet felé haladva: a háló súlytenzorai ebben a sorrendben W_1 (első konvolúció, $8\times1\times3\times3$), W_2 (második konvolúció, $16\times8\times3\times3$) és W_3 (lineáris réteg, 3×16); a j-edik lépésben ($j=0,1,2,3$) a bemenet felőli első j súlytenzort cseréljük azonos Frobenius-normájú független Gauss-tenzorra (a konvolúciós kernelt 4D tenzorként, a lineáris súlyt 2D mátrixként kezelve). A Frobenius-norma a tenzor összes elemére vett négyzetösszeg négyzetgyöke, azaz a tenzor kilapítása utáni euklideszi norma. Így a paraméterek változnak, de az egyes rétegek Frobenius-normája azonos marad; $j=0$ a változatlan modellt jelenti. Metrika lépésenként: $\mathrm{csim}_j=\langle s^0,s^j\rangle/(\|s^0\|\,\|s^j\|)$, ahol s^0 a tanított modell, s^j a j szintig scramble-elt modell szaliencia-vektora ugyanazon a testképen; nulla nevezőre a konvenció `0.0`. Kiértékelés: 30 test-scene (60 kép), páros összehasonlítás ugyanazon képpáron, 3 seed minimumával/maximumával jelentve. Döntési heurisztika: ha a csim_j mediánja a scramble mélységének növelésével nem nő, a 0-lépéses numerikus azonosságkontroll `1.0`-hez közeli, és a teljes scramble ($j=3$) esetén a medián `0.9` alá esik, a toy szaliencia paraméterérzékeny. Ez heurisztika, nem a [P25] cikk állításának igazolása.

Az ábrához az előző szakasz $g=(3;\,-2;\,0{,}5)$ vektorából, a lemma zárt képletéből számolunk; az ábraaláírás ezért „számított példa, nem mérés”. Aki a rögzített ellenőrző környezetben újragenerálja, a kódot a `lab/n143_figs.py` fájlba írja, és `python lab/n143_figs.py` paranccsal futtatja; a lényegi adatvektor és a szorzat a fenti blokkban ellenőrizhető.

36.3.6 Feladatok

F1 fogalmi, nehézség 1. Input: a jelen egység Σ-skálája. (a) Fogalmazd meg saját szavakkal a reprodukció és a replikáció lokális definíciójának különbségét. (b) Adj mindkettőre egy konkrét, a GlyphLab-környezetben értelmes esetet. (c) Ellenpélda-kontroll: miért nem replikáció egy toy adaptáció? Beadandó: a három válasz együtt, legfeljebb 12 mondat.

F2 levezetési, nehézség 2. Input: $g = (2;\ 0;\ -1)$, $\epsilon = 0{,}5$. Vezesd le a lemma alapján: a szign-irányú δ^* értékét, a lineáris növekmény maximumát, a maximalizálók teljes halmazát és a nem egyértelmű koordinátát. Beadandó: a teljes levezetés és a halmaz pontos megadása.

F3 implementációs, nehézség 2. Input: a kézi példa. Készíts `fgsm_perturbation(g, eps)` és `csim(a, b)` függvényt float32-tenzorokra, a szövegbeli konvenciókkal; `eps<0` esetén `ValueError`; üres/eltérő alakú bemenetre definiált hiba vagy konvenció. Ellenőrizd a fenti `$g$` vektorra a `0.55` értéket, és a `[1,2,3]`/`[-1,-2,-3]` koszinusz-hasonlóságot. Beadandó: futtatható kód és a tesztek futási állítása, `atol=1e-6` toleranciával.

F4 hibakeresési, nehézség 2. Input: „reprodukciós jelentéstervezet” két hibával: (1) „Az absztrakt birtokában kijelenthető, hogy a [P25] appendixének minden metrikáját ellenőriztük.” (2) `sim` függvény `np.dot(a,b)/(np.linalg.norm(a)*np.linalg.norm(b))` alakú, nulla bemenet esetén osztás nullával. Lokalizáld az okot, következményt; add meg a javított mondatot és kódot; ellenőrizd a javítást egy nulla vektorpárral. Beadandó: a négy elem (ok, következmény, javítás, visszaellenőrzés).

F5 kísérlettervezési/transzfer, nehézség 3. Input: a laborbeli toy-reprodukciós terv vázlata. Írd ki a teljes protokollt a 36.2. Ismétlések és eredménybizonytalanság alfejezet szóhasználatának megfelelően: megfigyelési egység, split, baseline, változó tényező, metrika, seed-ismétlés, döntési szabály, érvényességi korlát. Ne feltételezz előre győztest; negatív eredmény elfogadható. Beadandó: protokollszöveg, amelyet egy másik hallgató futtatni tud.

36.3.7 Megoldások és indoklás

F1. (a) A reprodukció azonos operatív protokoll saját kóddal, numerikus összehasonlítás toleranciával; a replikáció szándékosan változtat legalább egy protokoll-összetevőt, és a következtetés túlélését vizsgálja. (b) Reprodukció: a 37.4. Rétegből kis tanulómodell egységben felépített kis tanulómodell CNN-jének újratanítása azonos seeddel és azonos split mellett ugyanarra a val loss-értékre, a definíció szerinti abszolút toleranciával, $|L_{\text{ours}} - L_{\text{paper}}| \le \tau$, $\tau = 10^{-8}$; replikáció: ugyanaz a kérdés, de más adatgenerátor-seeddel készített GlyphLab-utánzaton vagy más implementációs nyelvű hálóval. (c) A toy adaptáció a módszer egyetlen mechanizmusát kicsinyíti, és jellemzően új adatot is használ; nem állítja elő a cikk eredeti kimeneti terét, ezért a replikáció előfeltétele – egy összehasonlítható következtetés – hiányzik.

F2. $g = (2;\ 0;\ -1)$, $\epsilon = 0{,}5$. A lemma: $\delta_1^* = 0{,}5 \cdot 1 = 0{,}5$, $\delta_2^* = 0{,}5 \cdot 0 = 0$, $\delta_3^* = 0{,}5 \cdot (-1) = -0{,}5$. Maximum: $\epsilon\|g\|_1 = 0{,}5(2 + 0 + 1) = 1{,}5$. Egyenlőségi feltétel: $\delta_1 = 0{,}5$, $\delta_3 = -0{,}5$ kötelező; δ_2 szabad, mert $g_2 = 0$. A maximalizálók halmaza: $\{(0{,}5;\ \delta_2;\ -0{,}5) : \delta_2 \in [-0{,}5;\ 0{,}5]\}$; a második koordináta nem egyértelmű. Alakellenőrzés: minden elem $\|\cdot\|_\infty = 0{,}5 \le \epsilon$, és $g \cdot \delta = 2 \cdot 0{,}5 + 0 \cdot \delta_2 + (-1) \cdot (-0{,}5) = 1{,}5$.

F3. Javasolt ellenőrző munkamenet:

```
import numpy as np

def fgsm_perturbation(g, eps):
    g = np.asarray(g, dtype=np.float32)
    if eps < 0:
        raise ValueError("eps nemnegatív kell legyen")
    return (eps * np.sign(g)).astype(np.float32)

def csim(a, b):
    a = np.asarray(a, dtype=np.float32).ravel()
    b = np.asarray(b, dtype=np.float32).ravel()
    if a.shape != b.shape:
        raise ValueError("eltérő alak")
```

```
    if a.size == 0:
        raise ValueError("üres bemenet")
    den = float(np.linalg.norm(a) * np.linalg.norm(b))
    return float(np.dot(a, b) / den) if den > 1e-12 else 0.0

g = np.array([3.0, -2.0, 0.5], dtype=np.float32)
d = fgsm_perturbation(g, 0.1)
assert np.allclose(d, [0.1, -0.1, 0.1], atol=1e-6)
assert abs(float(np.dot(g, d)) - 0.55) < 1e-6
assert abs(csim([1.0,2,3], [-1.0,-2,-3]) + 1.0) < 1e-6
assert csim([0.0,0], [1.0,1]) == 0.0

try:
    csim([], [])
except ValueError as e:
    print("üres bemenet:", e)
else:
    raise AssertionError("üres bemenetre ValueError kell")

print("ok", d)
```

A `float32` hozzávetőleg hét értékes decimális jegyet tárol, ezért `0.1` környékén az `atol=1e-6` biztonságos; ugyanezt a korlátot használjuk a szorzatra és a koszinusz-hasonlóságra is (a gyökszámítás kerekítése ezen a példán nem emeli a hibát az `atol=1e-6` fölé).

F4. (1) Ok: a bizonyítékstátusz absztrakt szintű, a kijelentés pedig függelékszintű állítást tesz, tehát a kijelentés erőssége meghaladja σ-t. Következmény: hamis ellenőrzési látszat; a lektoráló nem tudja megkülönböztetni az olvasott és a feltételezett részt. Javítás: „A [P25] absztraktja alapján a cikk a szaliencia sanity checkjeinek kérdését veti fel; a függelékbeli metrikákat nem tekintjük ellenőrzöttnek.” (2) Ok: nulla saliencia-vektornál a nevező nulla. NumPy-ban a `0/0` hányados `nan`-t ad (és `RuntimeWarning`-ot), nem `inf`-et; `inf` csak akkor adódna, ha a számláló nem nulla, a nevező viszont nulla. Következmény: üres vagy nulla képű teszteset tönkreteszi az átlagot, mert a `nan` továbbterjed. Javítás: nevező-küszöb, nulla vektorra konvenció `0.0`:

```
import numpy as np

def sim(a, b):
    a = np.asarray(a, dtype=np.float64).ravel()
    b = np.asarray(b, dtype=np.float64).ravel()
    if a.shape != b.shape:
        raise ValueError("eltérő alak")
    den = float(np.linalg.norm(a) * np.linalg.norm(b))
    if den < 1e-12:
        return 0.0
    return float(np.dot(a, b) / den)

assert sim([0, 0], [1, 1]) == 0.0
assert abs(sim([1, 2, 3], [-1, -2, -3]) + 1.0) < 1e-12
print("ok")
```

Visszaellenőrzés: `sim([0,0],[1,1]) == 0.0`, és a javítás nem változtatja meg a nem nulla vektorok eredményét (a `[1,2,3]`/`[-1,-2,-3]` pár koszinusza `-1.0`).

F5. A teljes protokoll: **kérdés:** a PyTorch-bemenetgradiens-alapú szaliencia paraméterérzékeny-e a tanított GlyphLab CNN-en. **Megfigyelési egység:** scene_id; a két nézet nem független, ezért scene-szintű mediánt számolunk. **Split:** 30 test-scene, train/val nem érintett; 3 training seed mindegyikére azonos split. **Baseline:** ugyanazon képen kétszer kiszámolt szaliencia numerikus hasonlósága (várt `1.0`). **Változó tényező:** scramble-mélység $j \in \{0;\ 1;\ 2;\ 3\}$ a bemenet felől számolva: $j = 0$ a változatlan modell, $j = 1$ az első konvolúció, $j = 2$ az első két konvolúció, $j = 3$ a teljes háló (mindhárom súlytenzor) cseréje. **Metrika:** páronkénti

koszinusz-hasonlóság a tanított és a scramble-elt modell között; seedenként scene-medián, majd seedek minimum–maximum sávja. **Döntés:** paraméterérzékenynek nevezzük, ha a teljes scramble medián `0.9` alá esik és a lépésenkénti medián nem nő; a `0.9` küszöb itt előre rögzített heurisztika, nem kívülről igazolt optimum. **Korlát:** az eredmény csak erre az architektúrára, erre a gradiensalapú szaliencia-megvalósításra és a GlyphLab grayscale `24×24` tartományára érvényes; emberi magyarázatminőséget nem állítunk, ahogy a 35.2. Magyarázat sanity check sanity-követelményei megkívánják. A negatív eredmény (a szaliencia nem változik a scramble-re) is lektorálható eredmény, ha a baseline numerikus azonosságkontroll `1.0` maradt. A tanítás a 24.4. labor AdamW-receptjével történik ($\eta_0 = 10^{-3}$, weight decay 10^{-4}, StepLR 10 epochonként $\gamma = 0{,}5$, batch 16, 20 epoch); az erőforrás CPU, három seed és négy scramble-szint.

36.3.8 Tipikus tévedés, önellenőrzés és visszatekintés

A tipikus tévedés: **egy absztraktból nem állítható a teljes módszer és függelék ellenőrzése.** Az ellenpélda formális alakja: az absztrakt bizonyítékhalmaz E_{abs} a szakaszszintű állítások C_{sec} halmazának csak valódi részét támasztja alá; a „a függelék reprodukálva van” kijelentés C_{appendix}-beli elem, amelyre σ = absztrakt, tehát az állítás nem következik a bizonyítékból. A tévedés gyakorlati jele, ha a jelentésben nem különül el a „a szerzők azt írják” és a „mi ellenőriztük” mondattípus.

Önellenőrzés: minden átvett állításomhoz van-e source, version, section és σ? Ahol $\sigma \leq$ absztrakt, ott a legerősebb megfogalmazást használtam-e? Reprodukciós tervemben rögzítettem-e a dtype-ot, seedet és splitet? **Kulcs.** Mindhárom kérdésre igen a helyes válasz: minden átvett állításhoz source, version, section és σ tartozik; $\sigma \leq$ absztrakt esetén a legerősebb megfogalmazást használjuk; a reprodukciós terv rögzíti a dtype-ot, a seedet és a splitet. Visszatekintés: 36.2. Ismétlések és eredménybizonytalanság szerint a becsült hatás intervallumához ki kell mondani a mintavételi modellt; az [S02] oldal a mintaátlag konfidenciaintervallumának feltételeit sorolja fel, és az intervallum csak ezek mellett értelmezhető. A 35.3. Perturbáció és adversarial példa FGSM-je csak a fenyegetési modell ℓ_∞-dobozán belül értelmes; a súlyrandomizálás egészen más beavatkozás, nem keverendő az adversarial perturbációval. A 24.3. Mixed precision és nagyobb batch óta tudjuk, hogy a reprodukciós protokollba a BatchNorm/dropout jelenlétét és az AMP (mixed precision, vegyes pontosságú számítás) állapotát is fel kell venni: a BatchNorm train módban batchfüggő statisztikát használ, a dropout viszont nem batchfüggő, hanem tanítási módban véletlen maszkot alkalmaz (eval módban kikapcsol), és az AMP-állapot a numerikus pontosságot változtatja meg, nem a batchtől függ.

36.3.9 Rövid összefoglaló

A szakirodalmi reprodukció attól lektorálható, hogy minden állítás mögött négy adat áll: forrás, változat, szakasz és olvasottsági szint. A reprodukció azonos protokoll saját kóddal és toleranciával való újrafuttatása; a replikáció szándékosan változtat összetevőt, és a következtetést vizsgálja; a toy adaptáció kicsinyített mechanizmus-ellenőrzés, egyik sem. Az FGSM szign-irányú lépésére bebizonyítottuk, hogy az ℓ_∞-dobozon a linearizált növekmény pontos optimuma, értéke $\epsilon\|g\|_1$. A [P25] esettanulmány megmutatta, hogyan készül cikk-kivonat úgy, hogy az absztrakt szintű tudást nem füllentjük szakaszszintű ellenőrzéssé; a hiányok a forrásaudit nyitott kérdései maradnak.

36.3.10 Továbblépés és források

A 36.4. IX. ellenőrző pont: lektorálható modellállítás IX. ellenőrző pont ezt az igénymátrixot kéri számon: egy saját modellállítást úgy kell bemutatni, hogy a fogalmi, numerikus, kísérleti és

forráseredet-kapuk mindegyike ellenőrizhető legyen. Az itt elkészített kivonat- és protokollsablon a beadandó jelentés váza.

Felhasznált források:

- [P25] Adebayo, J. et al. (2018). *Sanity Checks for Saliency Maps*. NeurIPS 2018. arXiv:1810.03292v3. https://arxiv.org/abs/1810.03292. Hozzáférés: 2026-10-30. Az itt használt állítás az absztraktban megfogalmazott kérdésre korlátozódik; a módszertani szakaszokra nem támaszkodunk.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. A hivatkozás a jegyzet vonatkozó oldalára mutat, és itt csak hivatkozási mintaként szolgál. Hozzáférés: 2026-10-30.
- [S02] NIST/SEMATECH (n.d.). *e-Handbook: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm. A hivatkozás a hivatalos módszertani oldalra mutat. Hozzáférés: 2026-10-30.

36.4 IX. ellenőrző pont: lektorálható modellállítás

36.4.1 Cél, előismeret és induló kérdés

Ez a IX. ellenőrző pont: a 36.3. Szakirodalmi reprodukció és attribúció alfejezetben felépített forráskövetést és a 36.2. Ismétlések és eredménybizonytalanság alfejezetben rögzített eredménybizonytalanság-kezelést egyetlen, lektorálható modellállítás-csomaggá zárjuk. A cél nem új réteg vagy veszteség bevezetése, hanem annak bemutatása, hogy egy reprodukálható kísérlet és egy matematikai levezetés akkor válik bírálhatóvá, ha hat komponense együtt, ellenőrizhető formában áll rendelkezésre. Előismeret: a 36.3. Szakirodalmi reprodukció és attribúció attribúciós szabályai, a 35.4. Korlátok, eredet és döntési felelősség korlát- és felelősségi kerete, a 32.4. VIII. ellenőrző pont: célhoz illő neurális modell modellválasztási döntési lapja és a 24.4. VI. ellenőrző pont: megbízható CNN-training megbízható training-csomagja.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakasza adja meg. Ez az egység vezeti be a riportszabály hat komponensét (36.4.2. szakasz). Az öt dimenzió közül itt a **D4** (bizonyíték és nyomon követhetőség) és a **D5** (korlátok és a következtetés érvényessége) kap nagyobb súlyt, mert a csomag éppen ezeket teszi lektorálhatóvá. A PASS-szabály ebben az egységben is az 1.4. Tanulási diagnózis és kísérleti napló szerinti: a labor minden `assert`-je és toleranciaellenőrzése átmegy, mind az öt dimenzió legalább 3-as szintű, és a riport csak ténylegesen lefuttatott kimenetet közöl.

Induló kérdés: elegendő-e egy Jupyter notebook tiszta kimenete ahhoz, hogy egy neurális modellről szóló állítást lektorálhatónak nevezzünk? Becslés: nem. A tiszta kimenet a *code* komponens egy szelete; hiányzik a matematikai állítás feltételrendszere, a forrásnyom, a futási környezet, a páros eredmény és a következtetési korlát.

36.4.2 A bírálható csomag hat komponense

A továbbiakban a következő hat komponenst követeljük meg, és mindegyikhez megadjuk a lektor kérdését:

Komponens	Mit tartalmaz	A lektor kérdése
proof	Állítás, feltételek, bizonyítás vagy jelölt átvétel, ellenpélda	A feltételek elégségesek és kimondottak-e?
source trace	Olvasott forrás, változat, szakasz, átvétel és saját rész	A státusz őrzi-e az ellenőrzés mélységét?
code	Teljes futtatható kód, input, seed, dtype, device, parancs	Végrehajtható-e az itt leírt környezetben?
run manifest	Környezet, verziók, adatsha, parancs, nyers metrika	Újrafuttatható-e, és látszik-e a nyers adat?
paired result	Ugyanazon megfigyelési egységre vonatkozó páros összevetés	A párosítás a helyes szinten történik-e?
limits	Mire nem következtethetünk az eredményből	A következtetés nem lép-e túl az adaton?

Egy komponens hiánya nem feltétlenül teszi hamissá az állítást, de **nem lektorálhatóvá** teszi a csomagot: a lektor nem tudja megkülönböztetni a bizonyított, a mért és a feltételezett állítást.

36.4.3 A kiválasztott állítás és bírálata (proof)

A csomag matematikai magjának egy gyakran pontatlanul kimondott CNN-állítást választunk: az eltolási ekvivarianciát. A feladat nem az állítás puszta elfogadása, hanem a feltételek, a jelölés és az ellenpéldák bírálata.

Először rögzítsük a főút cross-correlation definícióját egyetlen képre, groups=1 mellett. Legyen

$$X \in \mathbb{R}^{C_{in} \times H \times W}, \qquad K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w},$$

szimmetrikus zero padding p_h, p_w, stride s. A paddingolt bemenetet $\tilde{X}[c, r, s]$-sel jelöljük, ahol $\tilde{X}[c, r, s] = X[c, r, s]$, ha $0 \le r < H$ és $0 \le s < W$, különben 0. A kimenet

$$Y[o, i, j] = b[o] + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[o, c, u, v]\, \tilde{X}[c,\ si + u - p_h,\ sj + v - p_w]. \tag{1}$$

Definiáljuk a diszkrét eltolást: $(T_\Delta X)[c, r, s] = X[c, r-\Delta_h, s-\Delta_w]$, ahol a definíciós tartományon kívüli indexre az érték 0. A kérdés: mikor teljesül

$$Y_{T_\Delta X}[o, i, j] = Y_X[o, i - \Delta_h, j - \Delta_w] \ ? \tag{2}$$

A (2) egyenletet **nem** mondhatjuk ki feltétel nélkül. A pontos állítás a következő.

Állítás (belső shift-ekvivariancia, $s = 1$). Legyen $s = 1$, $\Delta = (\Delta_h, \Delta_w) \in \mathbb{Z}^2$, és a (2) jobb oldalát a kimeneti tartományon kívül zéróval kiterjesztve értelmezzük. Tegyük fel, hogy minden olyan (i, j) kimeneti indexre, amelyre a (2) jobb oldala értelmes (azaz $0 \le i - \Delta_h < H_{out}$ és $0 \le j - \Delta_w < W_{out}$), a kernel minden (u, v) pozíciójára az X-ben olvasott index a definíciós tartományba esik:

$$0 \le i + u - p_h - \Delta_h < H, \qquad 0 \le j + v - p_w - \Delta_w < W.$$

Ekkor a (2) mindkét oldalán ugyanazokat az X-értékeket összegezzük, és az egyenlőség fennáll.

Bizonyítás. A feltevés miatt $T_\Delta X$ paddingolt változatára $\widetilde{T_\Delta X}[c, r, s] = X[c, r - \Delta_h, s - \Delta_w]$ minden szükséges (r, s) indexre, és a padding nem lép be. Helyettesítsük be (1)-be:

$$Y_{T_\Delta X}[o, i, j] = \sum_{c,u,v} K[o, c, u, v]\, X[c,\ i + u - p_h - \Delta_h,\ j + v - p_w - \Delta_w].$$

Az összegzési indexeket változatlanul hagyva a kifejezés éppen $Y_X[o, i - \Delta_h, j - \Delta_w]$, feltéve, hogy az utóbbi index a kimeneti tartományban van. Ez a (2) egyenlet. □

A bírálat három ponton talál hibát a naiv megfogalmazásban.

(a) A véges kép határán az állítás hamis. Ha a shiftelt bemenet kernel-ablaka a paddinget érinti, ott $\widetilde{T_\Delta X}$ értéke 0, míg az eredeti kimenet eltoltja az eredeti képértékeket használná. A zero padding nem a shiftelt jel, hanem egy másik, nullával kiterjesztett jel.

(b) Stride $s > 1$ esetén a pontos állítás a stride-részcsoportra vonatkozik. A kimeneti rácsot s tizedeli; egy Δ bemeneti eltolás a kimenetben Δ/s eltolásnak felel meg, ami csak akkor egész, ha $\Delta = s \cdot m$, $m \in \mathbb{Z}^2$. A nem osztható eltolásokra a páros és páratlan fázisú mintavétel más bemeneti pixeleket lát, ezért (2) nem is értelmezhető ugyanazon a rácson.

(c) A paddingelt shift nem azonos a shiftelt paddinggel. Ha a shift műveletet a padding után végezzük, akkor a padding nullái is eltolódnak; ha a paddingot a shift után adjuk hozzá, akkor más jelet kapunk. A (2) állítás a második konvenciót rögzíti, és ezt a szövegben mindig ki kell mondani.

Ellenpélda a feltétel szükségességére. Legyen $C_{in} = C_{out} = 1$, $H = W = 4$, $k_h = k_w = 3$, $p_h = p_w = 0$, $s = 1$, $b = 0$, és $K[0, 0, u, v] = 1$ minden (u, v)-re. Legyen $X[0, 0, 0] = 1$, minden más 0. Az (1) képletből $H_{out} = \lfloor (4 + 0 - 3)/1 \rfloor + 1 = 2$, és hasonlóan $W_{out} = 2$. A bemenet $X \in \mathbb{R}^{4\times 4}$ (a csatornát elhagyva), a (0; 0) helyen 1:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad Y = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix},$$

mert az egyes kimeneti cellák a 3×3-as ablak összegei, és a (0; 0) bemenetet csak a (0; 0) ablak látja. Most legyen $\Delta = (\Delta_h, \Delta_w) = (1;\ 0)$, azaz a bemenetet **egy sorral lefelé** toljuk ($X'[r, s] = X[r - 1, s]$, Δ_h a sorindex eltolása):

$$X' = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad Y' = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix},$$

mert az X' (1; 0) eleme az $(i, j) = (0;\ 0)$ és az (1; 0) ablakban is benne van. A (2) jobb oldalát a kimeneti tartományon kívül zéróval kiterjesztve értelmezzük (ezt a kiterjesztést a feltétel nélküli állítás ellenpéldájához rögzíteni kell); így a definíció szerint eltolt kimenet $Y[i - 1, j]$, azaz $\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$. Az (1; 0) cellában az egyezés teljesül ($Y'[1, 0] = Y[0, 0] = 1$), de a (0; 0) cellában $Y'[0, 0] = 1 \neq 0 = Y[-1, 0]$: a kernel a határról „behúz” egy értéket, mert az $Y[-1, 0]$ ablak a kép definíciós tartományán kívülre lógna. A belső feltétel tehát nem elhagyható. (A példa $b = 0$ mellett érvényes. $b \neq 0$ esetén a belső tartományban a bias mindkét oldalon ugyanúgy

szerepel, ezért kiesik; a zéró-kiterjesztett határindexeknél viszont a bal oldal b-t ad, a jobb oldali kiterjesztés 0-t, ezért ott az egyenlőség már a bias miatt sem áll.)

36.4.4 Eredeti kézi példa és ellenőrző kód

A kézi példa két esetet ellenőriz: egy belső shiftet, ahol (2) teljesül, és a fenti határ-shift ellenpéldát, ahol nem. A kód a főút NumPy-referenciáját használja, groups=1, NCHW, float32, CPU. A kimenet a szövegben **futtatott érték**.

```
# Munkakönyvtár: lab/
# Futtatás: python equivariance_check.py
import numpy as np

def cross_corr_np(x, k, p, s=1):
    """groups=1, NCHW, szimmetrikus zero padding, float32 NumPy referencia."""
    x = np.asarray(x, dtype=np.float32)
    k = np.asarray(k, dtype=np.float32)
    N, C, H, W = x.shape
    O, _, kh, kw = k.shape
    Hout = (H + 2 * p - kh) // s + 1
    Wout = (W + 2 * p - kw) // s + 1
    xp = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)))
    y = np.zeros((N, O, Hout, Wout), dtype=np.float32)
    for n in range(N):
        for o in range(O):
            for i in range(Hout):
                for j in range(Wout):
                    si, sj = i * s, j * s
                    y[n, o, i, j] = np.sum(
                        xp[n, :, si:si + kh, sj:sj + kw] * k[o]
                    )
    return y

# 1. Belső shift: H=W=5, p=1, 3x3 delta kernel (középső súly 1)
x_inner = np.zeros((1, 1, 5, 5), dtype=np.float32)
x_inner[0, 0, 2, 2] = 1.0
k_delta = np.zeros((1, 1, 3, 3), dtype=np.float32)
k_delta[0, 0, 1, 1] = 1.0

y_inner = cross_corr_np(x_inner, k_delta, p=1)          # (1,1,5,5)
x_shift = np.zeros_like(x_inner)
x_shift[0, 0, 3, 2] = 1.0                               # Delta=(1,0)
y_shift = cross_corr_np(x_shift, k_delta, p=1)

# A delta kernel (középső súly 1) p=1 mellett identitás: y = x.
assert np.array_equal(y_inner[0, 0], x_inner[0, 0]), "delta kernel: y != x"
# Teljes 5x5 kimenet: y_shift a y_inner egy sorral lefelé tolt változata.
expected_inner = np.zeros((5, 5), dtype=np.float32)
expected_inner[1:, :] = y_inner[0, 0, :-1, :]
assert np.array_equal(y_shift[0, 0], expected_inner), (
    "belső shift: a teljes kimenet nem egyezik az eltolttal")
assert y_shift[0, 0, 2, 2] == 0.0, "belső shift: a forráshely nem ürült ki"
print("Belső shift-ekvivariancia: OK (teljes 5x5 kimenet)")

# 2. Határ-shift ellenpélda: H=W=4, p=0, 3x3 sum kernel
x_edge = np.zeros((1, 1, 4, 4), dtype=np.float32)
x_edge[0, 0, 0, 0] = 1.0
k_sum = np.ones((1, 1, 3, 3), dtype=np.float32)

y_edge = cross_corr_np(x_edge, k_sum, p=0)              # (1,1,2,2)
x_edge_shift = np.zeros_like(x_edge)
```

```
x_edge_shift[0, 0, 1, 0] = 1.0                              # Delta=(1,0)
y_edge_shift = cross_corr_np(x_edge_shift, k_sum, p=0)

# Zéró-kiterjesztett eltolt kimenet: Y[-1,0] = 0, Y[0,0] = 1.
expected_edge_shift = np.zeros((2, 2), dtype=np.float32)
expected_edge_shift[1, 0] = y_edge[0, 0, 0, 0]
assert y_edge_shift[0, 0, 0, 0] == 1.0, (
    "ellenpélda: a (0,0) cellában 1.0 az elvárt a zéró-kiterjesztett 0.0 helyett")
assert not np.array_equal(y_edge_shift[0, 0], expected_edge_shift), (
    "ellenpélda: a shift-ekvivariancia nem sérül, a példa nem cáfol")
print("Határ-shift ellenpélda: a feltétel nélküli állítás cáfolva")
```

A kód mindkét ága determinisztikus, seedre nincs szükség; a futtatott környezet Python 3.12.3, NumPy 2.5.4, CPU. A futtatott kimenet: `Belső shift-ekvivariancia: OK (teljes 5x5 kimenet)` és `Határ-shift ellenpélda: a feltétel nélküli állítás cáfolva`. A PyTorch `Conv2d` ugyanezt a cross-correlationt számolja, ezért az ellenpélda a könyvtári rétegre is érvényes.

36.4.5 Labor: technikai riport, forrásnapló, futási jegyzőkönyv

A labor a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab szerződés egy szűkített, CPU-n futtatható osztályozási kísérletét csomagolja be. A kísérlet lefutott; a mérési tábla mért értékeket tartalmaz. A hallgatói futáshoz a pontos protokoll:

- Kérdés: mekkora a clean/corrupt páros pontosságkülönbség a rögzített modellválasztás után?
- Adat: a `data/` könyvtár `images.npy`, `labels.npy`, `metadata.csv` és `corrupted_test.npy` fájlja; input float32 NCHW [0; 1], címke int64.
- Split: a `metadata.csv` split oszlopa szerint; a test 60 képe és a corrupt teszt ugyanazon scene/view sorrendben párosul.
- Modell: a 20.4. V. ellenőrző pont: indokolt CNN-választás döntési lapjából választott, 2979 paraméteres kis classic CNN, scratch, CPU. A rétegsor: `Conv2d(1,8,3,padding=1)` ($8 \cdot 1 \cdot 3 \cdot 3 + 8 = 80$ paraméter), `Conv2d(8,16,3,padding=1)` ($16 \cdot 8 \cdot 3 \cdot 3 + 16 = 1168$), majd $16 \cdot 6 \cdot 6 = 576$ bemenetű `Linear(576,3)` ($576 \cdot 3 + 3 = 1731$); összesen $80 + 1168 + 1731 = 2979$. A pontos rétegsor a `lab/model.py` fájlban rögzítendő.
- Tanítás: 20 epoch, batch 16, Adam, tanulási ráta 10^{-3}, seed 20261020, float32, CPU. A best checkpoint kiválasztása kizárólag val loss alapján.
- Kiértékelés: a best checkpoint betöltése után clean test accuracy, corrupt test accuracy, és a **páros** delta: $d_i = \mathbf{1}[f(x_i^{clean}) = y_i] - \mathbf{1}[f(x_i^{corrupt}) = y_i]$, ahol az index a metadata test sorrendje.

A run manifest (mért futás):

```
környezet: Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál
adat: data/manifest.json SHA-256 egyezés ellenőrizve
parancs: python n144_train.py (20 epoch, batch 16, Adam, lr=1e-3, seed 20261020)
kimenet: n144_report.json
```

A nyers mérési tábla (mért futás; a checkpoint kiválasztása val loss szerint történt, a kiválasztott epoch a 19., val loss = 0,3241):

Epoch	Train loss	Val acc	Test clean acc	Test corrupt acc	Páros delta átlag
19 (best)	0,1006	0,8667	0,9333	0,7333	+0,2000

A paired result értelmezése: a $d_i = \mathbf{1}[f(x_i^{clean}) = y_i] - \mathbf{1}[f(x_i^{corrupt}) = y_i]$ definíció mellett a pozitív átlag azt mutatja, hogy a corrupt bemenetek rontják a helyes predikciót (12 képen a clean találat corrupton elvész, ellenkező irányú eset nincs); a párosítás kizárja, hogy a clean és corrupt halmaz összetételének különbsége magyarázza az eltérést. A képenkénti bootstrap itt nem független, mert a megfigyelési egység a scene; a megbízható intervallumhoz scene-szintű, osztály-stratifikált resampling kell (5.4. Függő képek, seedek és resampling; [S03]).

A **forrásnapló** a csomag része:

Kulcs	Ellenőrzött státusz	Átvett tartalom	Saját adaptáció
S01	kurzusjegyzet releváns oldala olvasva	cross-correlation, stride, padding, receptive field fogalma	a belső shift-ekvivariancia bizonyítása és az ellenpélda eredeti
S02	hivatalos módszertani oldal olvasva	a mintaátlag standard hibája és a t-alapú konfidenciaintervallum	a mintaátlag bizonytalanságának értelmezése
P25	eredeti arXiv-rekord és absztrakt ellenőrizve	motiváció: automatikus metrika nem helyettesíti a sanity checket	a lektorálási kapu megfogalmazása

A P25 státusza **nem** teszi lehetővé, hogy részletes módszerállítást alapozzunk rá; erre a forrásnaplóban külön figyelmeztetünk.

A **limits** szakasz rögzíti: a GlyphLab eredménye nem természetes képi benchmark és nem termelési validáció; a softmax kimenet nem bizonytalansági garancia; a corrupt teszt csak a definiált zaj- és fényességgradiens-korrupcióra vonatkozik; high-stakes döntéshez emberi felülvizsgálat és a 35.4. Korlátok, eredet és döntési felelősség szerinti modellkártya szükséges.

36.4.6 Feladatok

F1 (fogalmi, nehézség 1). Sorolja fel a bírálható csomag hat komponensét, és indokolja meg, miért nem elegendő a tiszta kódkimenet önmagában. Adjon egy konkrét példát egy olyan hibára, amelyet a kód kimenete nem tár fel.

F2 (levezetési, nehézség 2). Vezesse le a stride $s = 2$ esetre a pontos shift-ekvivariancia állítást az $s = 1$-hez hasonló indexhelyettesítéssel. Mondja ki, mely Δ eltolásokra értelmes az állítás, bizonyítsa a belső tartományra, és adjon ellenpéldát $\Delta = (1;\ 0)$-ra $H = 4$, $k_h = 1$, $p_h = 0$, $s = 2$ mellett.

F3 (implementációs, nehézség 2). A 36.4.4. szakasz kódját egészítse ki egy `check_stride2` függvénnyel, amely a stride 2 esetre ellenőrzi: $\Delta = (2;\ 0)$-ra a belső tartományon a shiftelt kimenet megegyezik az eredeti kimenet eltoltjával, $\Delta = (1;\ 0)$-ra viszont nem. Adja meg a teljes függvényt, a bemeneteket, a várt assert üzeneteket és a futtatási parancsot.

F4 (hibakeresési, nehézség 2). Az alábbi két hibát tartalmazó riportrészletben lokalizálja a hibákat, írja le a következményt és a javítást: (a) a forrásnapló a P25 absztraktjára hivatkozva részletes sanity-check protokollt állít; (b) a páros delta konfidenciaintervallumát képenkénti bootstrapből számítják, pedig a scene_id a független egység.

F5 (kísérleti/transzfer, nehézség 3). Egy mérnök a GlyphLab-on elért 0,90 test accuracy alapján azt állítja, hogy a modell alkalmas valós ipari felületi hibák osztályozására. Tervezzen ellenőrzési protokollt a domain shift kezelésére: mi legyen a baseline, a split, az adaptációs szabály, a metrika, és mit kell dokumentálni a 35.4. Korlátok, eredet és döntési felelősség modellkártyájában? Indokolja, miért nem elég a GlyphLab accuracy.

36.4.7 Megoldások és indoklás

F1 megoldás. A hat komponens: proof, source trace, code, run manifest, paired result, limits. A kód kimenete az implementációt ellenőrzi, de nem mutatja meg (i) a matematikai állítás érvényességi tartományát, (ii) a forrás ellenőrzési mélységét, (iii) a környezet reprodukálhatóságát, (iv) a párosítás egységét, (v) a következtetés korlátait. Példa: egy notebook 0,90 accuracyt mutat, de a test halmazt a tanítás előtt használták modellválasztásra; a kimenet önmagában ezt a szivárgást nem fedi fel.

F2 megoldás. Legyen $s = 2$. Az (1) képletben a kimeneti (i, j) pozíció az $si + u$ és $sj + v$ bemeneti indexeket olvassa. Ha a bemenetet Δ-val toljuk, a kimeneti pozíciók által látott értékek az eredeti bemenet $si + u - \Delta$ indexeinek felelnek meg. A (2) típusú egyenlőség azt követelné meg, hogy az új kimeneti rács az eredeti kimeneti rács Δ/s eltolásával essen egybe. Ez pontosan akkor lehetséges, ha $\Delta = s \cdot m$ valamely $m \in \mathbb{Z}^2$ egész vektorra, mert a kimeneti rács osztása s. A bizonyítás ezután szó szerint megegyezik a 36.4.3. szakasz indexhelyettesítésével, az $si \to s(i - m_h)$ és $sj \to s(j - m_w)$ helyettesítéssel.

Ellenpélda $\Delta = (1;\ 0)$-ra: legyen $H = 4$, W tetszőleges, $k_h = 1$, $p_h = 0$, $s = 2$. Ekkor $H_{out} = \lfloor (4 - 1)/2 \rfloor + 1 = 2$. A kimenet az $Y[i] = X[2i]$ mintavételezés ($i = 0, 1$). Legyen $X[1] = 1$, minden más 0; ekkor $Y = (X[0], X[2]) = (0;\ 0)$. A shiftelt bemenet $X'[2] = 1$, tehát $Y' = (X'[0], X'[2]) = (0;\ 1)$. Ha az ekvivariancia a $\Delta = (1;\ 0)$ eltolásra is fennállna, akkor létezne olyan egész m, amelyre $Y'[i] = Y[i - m]$ minden i-re (a kimenetet a tartományon kívül zéróval kiterjesztve); de $Y \equiv 0$ miatt a jobb oldal minden m-re 0, míg $Y'[1] = 1$, tehát ilyen m nincs. Ezzel az állítás érvényességi köre pontosan a stride-részcsoport.

F3 megoldás. A függvény a 36.4.4. szakasz `cross_corr_np`-jét használja:

```
def check_stride2():
    # Belső tartomány, Delta=(2,0): ekvivariancia teljes tömbre teljesül
    x = np.zeros((1, 1, 6, 6), dtype=np.float32)
    x[0, 0, 2, 2] = 1.0
    k = np.ones((1, 1, 1, 1), dtype=np.float32)
    y = cross_corr_np(x, k, p=0, s=2)          # (1,1,3,3)
    x2 = np.zeros_like(x)
    x2[0, 0, 4, 2] = 1.0                       # Delta=(2,0) = s*(1,0)
    y2 = cross_corr_np(x2, k, p=0, s=2)
    expected = np.zeros((3, 3), dtype=np.float32)
    expected[1:, :] = y[0, 0, :-1, :]
    assert np.array_equal(y2[0, 0], expected), (
        "stride-2, Delta=(2,0): a kimenet nem az eltolt kimenet")
    # Nem osztható eltolás, Delta=(1,0): nincs tiszta shift
    x1 = np.zeros_like(x)
    x1[0, 0, 2, 3] = 1.0
    y1 = cross_corr_np(x1, k, p=0, s=2)
    # A (2,3) pont oszlopindexe páratlan, ezért a stride-2 rács nem mintavételezi;
    # y1 minden eleme 0, így y1 nem lehet y egész eltoltja
    assert not np.any(y1), (
        "stride-2, Delta=(1,0): a páratlan oszlopindex mégis mintavételeződött")
    assert not np.array_equal(y1[0, 0], y[0, 0]), (
        "stride-2, Delta=(1,0): a kimenet egyezik, a cáfolat nem érvényesül")
    print("Stride-2 részcsoport-ekvivariancia: OK")
```

`check_stride2()`

A futtatás: a 36.4.4. szakasz kódjával egy fájlban (`equivariance_check.py`), a `check_stride2()` hívással; a futtatott kimenet (Python 3.12.3, NumPy 2.5.4, CPU) három sora: `Belső shift-ekvivariancia: OK (teljes 5x5 kimenet)`, `Határ-shift ellenpélda: a feltétel nélküli állítás cáfolva` és `Stride-2 részcsoport-ekvivariancia: OK`. A tolerancia itt szigorú egyenlőség, mert a műveletek egész értékű float32 összegek.

F4 megoldás. (a) Hiba: a P25 státusza „absztrakt ellenőrizve", ezért részletes módszerállítás forrásaként nem használható. Következmény: a riport olyan eljárást tulajdonít a cikknek, amelyet a szerző nem ellenőrzött, sérül az attribúciós lánc. Javítás: a P25-öt csak motivációként idézni, a konkrét protokollhoz a teljes releváns szakaszt elolvasni, vagy a saját eljárást önálló adaptációként feltüntetni. (b) Hiba: a CI-t képenkénti bootstrapből számítják, de a GlyphLab független egysége a scene, és egy scene két nézete összefügg. Következmény: a CI szűkebbnek mutatja a bizonytalanságot, mint amit az adat igazol. Javítás: scene-szintű, osztály-stratifikált resampling, a páros delta ugyanazon scene/view páron számítva.

F5 megoldásminta. Lehetséges terv: (1) baseline: két referencia. (a) A GlyphLab-modell közvetlen, adaptáció nélküli alkalmazása a valós adaton (zero-shot), amely a domain-shift nagyságát mutatja; (b) a valós tanítóhalmazon önmagában tanított modell (target-only), amelyhez a transzfer nyereségét mérjük. (2) split: a valós adatot gyártási tétel vagy idő szerint csoportosítva bontani, hogy a szivárgást elkerüljük; (3) adaptáció: a fagyasztott feature extractor fölötti lineáris próba (logisztikus regresszió) az első adaptációs lépés, nem baseline; ezután szükség esetén finomhangolás, minden lépés előre rögzített val-loss választással; (4) metrika: a feladathoz illő pontosság mellett confusion matrix, osztályonkénti recall, és a téves döntés költségének megfelelő küszöb; (5) dokumentáció: adateredet, domain eltérések, súlylicenc, a modell határai, emberi felülvizsgálati pont. A GlyphLab accuracy azért nem elég, mert a procedurális szintetikus alakzatok és a valós felületi hibák eloszlása, textúrája és annotációs hibája különbözik; a transzfer csak új, független valós méréssel igazolható.

36.4.8 Tipikus tévedés, önellenőrzés és visszatekintés

A tipikus tévedés: az automatikus ellenőrzést lektorálásnak tekinteni. A 15.4. Független gradiensellenőrzési protokoll szerinti gradiensellenőrzés, az `assert` és a CI-számítás a csomag **implementációs** és **numerikus** bizonyítékai, de nem helyettesítik a matematikai feltételvizsgálatot. Ellenpélda: a 36.4.4. szakasz kódja két külön assertet futtat, egyet a belső shiftre (ez áthalad), egyet pedig a határ-shift ellenpéldára (ez azt ellenőrzi, hogy a feltétel nélküli állítás hamis). Ha csak az első assertet futtatnánk, a zöld kimenetből tévesen általánosítanánk; a cáfoló assert nélkül a csomag nem lektorálható. Az automatikus ellenőrzés tehát szükséges, de nem elégséges kapu.

Önellenőrzés: (1) Mind a hat komponens szerepel-e? (2) Az állítás feltételei és az ellenpélda ugyanabban a jelölésben vannak-e? (3) A nyers tábla jelöli-e, hogy szemléltetés vagy mért futás? (4) A forrásnapló őrzi-e a státuszkülönbséget? (5) A páros eredmény a scene szintjén párosul-e?

Kulcs. (1) Igen, mind a hat komponens (proof, source trace, code, run manifest, paired result, limits) szerepel. (2) Igen, a feltétel és az ellenpélda ugyanazt a jelölést használja. (3) Igen, a nyers tábla megjelöli, hogy szemléltetés vagy mért futás. (4) Igen, a forrásnapló megőrzi az ellenőrzött és a nem ellenőrzött státusz különbségét. (5) Igen, a párosítás egysége a scene, nem a kép.

Visszatekintés: a 36.3. Szakirodalmi reprodukció és attribúció attribúciós elve, a 35.4. Korlátok, eredet és döntési felelősség modellkártya és döntési felelősség, a 32.4. VIII. ellenőrző pont:

célhoz illő neurális modell célhoz illő modellválasztási lap, valamint a 24.4. VI. ellenőrző pont: megbízható CNN-training megbízható training-csomag. A négy alfejezet a hat komponenshez így kapcsolódik: a 36.3 a source trace, a 35.4 a limits, a 32.4 a modellválasztási döntési lap páros összevetése révén a paired result, a 24.4 pedig a code és a run manifest előzménye; a proof komponenst a 36.4.3. szakasz saját levezetése adja.

36.4.9 Rövid összefoglaló

A IX. ellenőrző pont egyetlen csomagba rendezi a modellállítás hat bizonyítéktípusát: a feltételeivel kimondott matematikai állítást, a forrásnyomot, a futtatható kódot, a futási jegyzőkönyvet, a páros eredményt és a következtetési korlátokat. A belső shift-ekvivariancia példája megmutatja, hogy ugyanaz az állítás feltétel nélkül hamis, pontos feltételekkel bizonyítható, a stride esetén pedig részcsoportra szorul. A labor kimenete mért futás; a hallgatói riportnak a nyers adatot, a forrásnaplót és a javítási listát együtt kell tartalmaznia. Az automatikus ellenőrzés szükséges, de a szakmai lektorálást nem helyettesíti. A következő egység a teljes konvolúciós projektet nyitja.

Források:

- [S01] Stanford CS231n (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.
- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm. Hozzáférés: 2026-10-30.
- [S03] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- [P25] Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., Kim, B. (2018). *Sanity Checks for Saliency Maps*. NeurIPS. https://arxiv.org/abs/1810.03292.

37 Projekt I: konvolúciós réteg levezetéstől referenciakódig

37.1 Operátorspecifikáció és kézi referencia

37.1.1 Cél, előismeret és induló kérdés

Ez az egység a **Projekt I** első állomása: egy saját, kézzel írt konvolúciós reference operátort készítünk, amelynek minden indexét, alakját és érvényességi feltételét pontosan dokumentáljuk. A cél nem egy gyors PyTorch-utánzat, hanem egy olyan specifikáció, amelyhez később a backwardot (37.2. Backward és ellenőrzési lefedettség), az adjungált ellenőrzést (37.3. Linearitás, adjungált és határfeltétel) és a tanuló réteget (37.4. Rétegből kis tanulómodell) kapcsolni lehet. A 16.4. IV. ellenőrző pont: a CNN-operátor teljes megértése IV. ellenőrző pontjából ismert, hogy a numerikus egyezés önmagában nem bizonyíték; ezért itt először a definíciót rögzítjük, és csak utána engedünk kódot mellé.

Szükséges előismeret: a 13.4. Határkezelés és frekvenciaintuíció alfejezetben bevezetett cross-correlation és zero padding, a 14.4. Csoportos és depthwise konvolúció alfejezetben tárgyalt `groups` paraméter és annak korlátozása, a 15.4. Független gradiensellenőrzési protokoll alfejezetben kidolgozott `float64`-es gradiensellenőrzési protokoll, valamint a 36.4. IX. ellenőrző pont: lektorálható modellállítás alfejezetben áttekintett lektorálható állításforma. A 36.1. Állításból kísérleti terv kísérlettervezési elvét és a 33.1. Inference-szerződés és modellcsomag következtetési szerződését (inference contract) csak érintőlegesen idézzük vissza, mert itt még nincs betanított modell.

Induló kérdés: adott egy 5×5-ös bemenet, egy 3×3-as kernel, stride 2, padding 1, dilation 1. Hány térbeli pozícióból áll a kimenet, és miért nem elegendő a „majd a könyvtár tudja” válasz egy referencia specifikációhoz?

37.1.2 Definíció: a támogatott keresztkorrelációs operátor

Az operátor a PyTorch `Conv2d` által használt **térbeli keresztkorrelációt** számolja ([D10]): a kernel nem fordul meg semelyik tengely mentén. A matematikai konvolúció ettől a kernel 180°-os megfordításában tér el; ezt a 13.2. Konvolúció és cross-correlation pontosan tárgyalta. A jelen specifikáció kizárólag a következő részhalmazt támogatja:

- bemenet: $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, NCHW elrendezés;
- súly: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$;
- bias: $b \in \mathbb{R}^{C_{out}}$, opcionális;
- kimenet: $Y \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$;
- `groups=1`, azaz minden kimeneti csatorna minden bemeneti csatornát lát;
- szimmetrikus zero padding: $p_h, p_w \geq 0$ egész, mindkét oldalon ugyanannyi;
- stride: $s_h, s_w \geq 1$ egész;
- dilation: $d_h, d_w \geq 1$ egész;
- kernelméret: $k_h, k_w \geq 1$ egész;

- dtype (adattípus): a referencia kizárólag `float64`; a training főút később `float32`, de a numerikus kontrollhoz itt szándékosan a pontosabb típust használjuk;
- a batch, csatorna- és térbeli méretek mind pozitívak.

A **zero padding** azt jelenti, hogy a bemeneti kép szélén kívülre eső pozíciók értéke nulla. Formálisan bevezetjük a paddingelt tömböt

$$\tilde{X} \in \mathbb{R}^{N \times C_{in} \times (H+2p_h) \times (W+2p_w)},$$

ahol

$$\tilde{X}[n, c, \alpha, \beta] = \begin{cases} X[n, c, \alpha - p_h, \beta - p_w], & \text{ha } p_h \le \alpha < H + p_h \text{ és } p_w \le \beta < W + p_w, \\ 0, & \text{egyébként.} \end{cases}$$

Ezzel a forward egyetlen indexképlete:

$$\boxed{Y[n, o, i, j] = b[o] + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \tilde{X}[n, c,\ is_h + ud_h,\ js_w + vd_w]\ K[o, c, u, v],}$$

ahol $0 \le i < H_{out}$, $0 \le j < W_{out}$, és $b[o] \equiv 0$, ha a bias ki van kapcsolva. A stride tehát csak a kimeneti pozíciót lépteti, a dilation pedig a kernel pontjainak bemeneti távolságát növeli: az u-adik kernelsor a paddingelt kép $is_h + ud_h$ sorát olvassa.

Következmény (az output alak). A kimeneti magasság H_{out} a legnagyobb olyan i index plusz egy, amelyre a kernel utolsó, $(k_h - 1)$-edik sorának pozíciója még a paddingelt tömbön belül van:

$$is_h + (k_h - 1)d_h \le (H + 2p_h) - 1.$$

Átrendezve

$$i \le \frac{H + 2p_h - (k_h - 1)d_h - 1}{s_h}.$$

A legnagyobb ilyen egész a jobb oldal alsó egészrésze, ezért

$$H_{out} = \left\lfloor \frac{H + 2p_h - (k_h - 1)d_h - 1}{s_h} \right\rfloor + 1.$$

A szélességre ugyanígy, h helyett w indexekkel. Ez a szabványos output-shape képlet, itt **levezetve** a paddingelt definícióból. A két térbeli dimenzió független, mert a padding szimmetrikus és a stride/dilation tengelyenként külön adható meg. A 37.1.1. szakasz induló kérdésére a válasz tehát: $H_{out} = W_{out} = \lfloor (5 + 2 - 2 \cdot 1 - 1)/2 \rfloor + 1 = \lfloor 4/2 \rfloor + 1 = 3$, azaz a kimenet 3×3 pozícióból áll.

Érvényességi feltétel. A kimenet csak akkor értelmes, ha $H_{out} \ge 1$ és $W_{out} \ge 1$. Ha a számláló negatív, az operátor hibát dob: a kernel effektív kiterjedése $d_h(k_h - 1) + 1$ nagyobb, mint a paddingelt bemenet. Ez a feltétel az implementáció bemeneti validációjának része, nem a felhasználó felelőssége.

A definícióban a bias dimenziója C_{out}, és minden térbeli pozícióhoz ugyanaz a skalár adódik: a bias csatornánként egyetlen paraméter, nem térben változó.

37.1.3 A specifikáció mint matematikai állítás

A fenti definícióval a referencia operátor **pontosan meghatározott függvény**: adott X, K, b és a paraméterek mellett Y egyértelmű. A levezetési követelmény teljesült: az output alak a paddingelt ablak illeszkedési feltételéből következik, és a képlet minden indexhatárt megad. A bizonyítás saját, teljes levezetés; nagy tételt nem veszünk át.

A specifikáció **nem** állítja, hogy ez az operátor minden `Conv2d`-opciót támogat. Kimarad például az aszimmetrikus padding, a `padding_mode='reflect'/'replicate'/'circular'`, a `groups>1`, és a transzponált művelet (az `output_padding` nem a `Conv2d`, hanem a `ConvTranspose2d` paramétere, ezért a referencia által nem támogatott `Conv2d`-opciók között nem szerepel). A 15.3. Transposed convolution mint adjungált alfejezetben tárgyalt `ConvTranspose2d` adjungált szerepe sem tartozik ide; a [D11] forrás ehhez kiinduló URL, tartalmát a használat előtt ellenőrizni kell. A referencia célja a **tanulási főút** pontos alapja, nem a keretrendszer helyettesítése.

37.1.4 Eredeti kézi kis példa

Vegyünk egy 3×3-as bemenetet, egy olyan 2×2-es kernelt, amely nem azonos a saját 180°-os elforgatottjával ($K \neq \mathrm{rot}_{180}\, K$; a keresztkorreláció és a konvolúció különbsége ezen múlik, nem a főátló szerinti szimmetrián: például $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$ forgatásra invariáns, transzponálásra nem), stride 1, padding 0, dilation 1, egyetlen bemeneti és kimeneti csatornát, bias 10:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

A kernel azért nemszimmetrikus, mert a főátlóra tükrözve $2 \neq 3$; így a keresztkorreláció és a konvolúció eredménye különbözni fog. A definíció szerint például

$$Y[0,0,0,0] = 1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 5 \cdot 4 + 10 = 47.$$

A teljes kimenet:

$$Y = \begin{bmatrix} 47 & 57 \\ 77 & 87 \end{bmatrix}.$$

Ellenőrzésképp a jobb alsó elem:

$$Y[0,0,1,1] = 5 \cdot 1 + 6 \cdot 2 + 8 \cdot 3 + 9 \cdot 4 + 10 = 87.$$

Ugyanez a bemenet matematikai konvolúcióval a megfordított kernel $\begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$ alkalmazását jelentené, ami más mátrixot adna; ez az F1 feladatban tér vissza.

A következő minimális kód a referencia függvényt mutatja be, és ezen a kézi példán ellenőrzi az eredményt. A teljes labor-fájl az 5. pontban található; itt csak a függvény vázát adjuk:

```python
import numpy as np

def conv2d_reference(X, K, b=None, stride=(1,1), padding=(0,0), dilation=(1,1)):
    # X: (N, C_in, H, W) float64
    # K: (C_out, C_in, k_h, k_w) float64
    # b: (C_out,) float64 vagy None
    ...
    return Y

# kézi példa
X = np.array([[[[1,2,3],[4,5,6],[7,8,9]]]], dtype=np.float64)  # (1,1,3,3)
K = np.array([[[[1,2],[3,4]]]], dtype=np.float64)              # (1,1,2,2)
Y = conv2d_reference(X, K, b=np.array([10.0], dtype=np.float64))
print(Y)  # várható [[[[47.,57.],[77.,87.]]]]
```

A kód `np.pad`-et használ konstans nulla üzemmódban, majd explicit ciklusokkal számol. A ciklusok lassúak, de a referencia célja a helyesség, nem a sebesség.

37.1.5 Labor: kézi kontroll és negatív teszt

A labor két fájlból áll. A `reference_conv2d.py` a specifikációt valósítja meg; a `test_reference.py` a kézi példát, az alakellenőrzéseket és a hibás bemenetek negatív tesztjeit futtatja. A labor a `lab/n145_reference/` munkakönyvtárban található. A futtatás:

```
cd lab/n145_reference
python -m pytest test_reference.py -q
```

A futtatott kimenet (Python 3.12.3, NumPy 2.5.4, pytest 9.2.0, CPU): `5 passed`; ennyi a labor `test_reference.py` öt tesztje. A megoldások között szereplő `test_random_shape_dtype`-tal együtt futtatva `6 passed` (a futásidő gépfüggő, a mért érték 0,04–0,12 s között mozgott). A hallgató a saját futását a kísérleti naplóban rögzítse.

A referencia függvény teljes kódja:

```python
# reference_conv2d.py
import numpy as np

def conv2d_reference(X, K, b=None, stride=(1,1), padding=(0,0), dilation=(1,1)):
    """
    Saját NCHW keresztkorrelációs referencia.
    Támogatott: groups=1, szimmetrikus zero padding, stride, dilation,
                opcionális bias. dtype: float64.
    """
    if not (isinstance(X, np.ndarray) and isinstance(K, np.ndarray)):
        raise TypeError("X és K NumPy tömb legyen")
    if X.dtype != np.float64 or K.dtype != np.float64:
        raise TypeError("X és K dtype-ja float64 legyen")
    if b is not None and (not isinstance(b, np.ndarray) or b.dtype != np.float64):
        raise TypeError("b dtype-ja float64 legyen")
    if X.ndim != 4:
        raise ValueError(f"X dimenziója {X.ndim}, várt 4 (N,C_in,H,W)")
    if K.ndim != 4:
        raise ValueError(f"K dimenziója {K.ndim}, várt 4 (C_out,C_in,k_h,k_w)")
    for name, value in (("stride", stride), ("padding", padding),
                        ("dilation", dilation)):
        if not (isinstance(value, tuple) and len(value) == 2):
            raise TypeError(f"{name} két elemű tuple legyen, kapott: {value!r}")

    N, C_in, H, W = X.shape
    C_out, C_in_k, k_h, k_w = K.shape
```

```
    if C_in_k != C_in:
        raise ValueError(f"K bemeneti csatornája {C_in_k}, de X-é {C_in}")
    if b is not None and b.shape != (C_out,):
        raise ValueError(f"b alakja {b.shape}, várt ({C_out},)")

    s_h, s_w = stride
    p_h, p_w = padding
    d_h, d_w = dilation
    if s_h < 1 or s_w < 1:
        raise ValueError("stride >= 1")
    if p_h < 0 or p_w < 0:
        raise ValueError("padding >= 0")
    if d_h < 1 or d_w < 1:
        raise ValueError("dilation >= 1")
    if k_h < 1 or k_w < 1:
        raise ValueError("kernel méret >= 1")

    H_out = (H + 2*p_h - (k_h-1)*d_h - 1) // s_h + 1
    W_out = (W + 2*p_w - (k_w-1)*d_w - 1) // s_w + 1
    if H_out < 1 or W_out < 1:
        raise ValueError("A kimeneti méret nem pozitív; ellenőrizze a paramétereket")

    # paddingelt tömb
    X_pad = np.pad(X, ((0,0),(0,0),(p_h,p_h),(p_w,p_w)), mode='constant')
    Y = np.zeros((N, C_out, H_out, W_out), dtype=np.float64)

    for n in range(N):
        for o in range(C_out):
            for i in range(H_out):
                for j in range(W_out):
                    s = 0.0
                    for c in range(C_in):
                        for u in range(k_h):
                            for v in range(k_w):
                                s += X_pad[n, c, i*s_h + u*d_h, j*s_w + v*d_w] * K[o, c, u, v]
                    Y[n, o, i, j] = s
            if b is not None:
                Y[n, o, :, :] += b[o]  # bias csatornánként
    return Y
```

A tesztfájl a kézi példát és a negatív eseteket tartalmazza:

```
# test_reference.py
import numpy as np
import pytest
from reference_conv2d import conv2d_reference

def test_manual_nonsymmetric():
    X = np.array([[[[1,2,3],[4,5,6],[7,8,9]]]], dtype=np.float64)
    K = np.array([[[[1,2],[3,4]]]], dtype=np.float64)
    Y = conv2d_reference(X, K, b=np.array([10.0], dtype=np.float64))
    expected = np.array([[[[47.0,57.0],[77.0,87.0]]]], dtype=np.float64)
    assert np.allclose(Y, expected, rtol=0, atol=1e-12)

def test_wrong_dtype():
    X = np.zeros((1,1,3,3), dtype=np.float32)
    K = np.zeros((1,1,2,2), dtype=np.float32)
    with pytest.raises(TypeError):
        conv2d_reference(X, K)

def test_channel_mismatch():
    X = np.zeros((1,2,3,3), dtype=np.float64)
    K = np.zeros((1,3,2,2), dtype=np.float64)
```

```
    with pytest.raises(ValueError):
        conv2d_reference(X, K)

def test_zero_output():
    X = np.zeros((1,1,2,2), dtype=np.float64)
    K = np.zeros((1,1,5,5), dtype=np.float64)
    with pytest.raises(ValueError):
        conv2d_reference(X, K)

def test_bad_stride():
    X = np.zeros((1,1,4,4), dtype=np.float64)
    K = np.zeros((1,1,2,2), dtype=np.float64)
    with pytest.raises(ValueError):
        conv2d_reference(X, K, stride=(0,1))
```

A negatív teszt célja annak ellenőrzése, hogy a függvény hibás bemenetnél **nem** ad csendes, hamis eredményt. Ez a 14.4. Csoportos és depthwise konvolúció alfejezetben jelzett „csendes elnyelés” (amikor egy hibás alakú vagy típusú bemenet hibaüzenet nélkül csendes, hibás eredményt ad) ellenpontja.

37.1.6 Feladatok

37.1.6.1 F1 (fogalmi, nehézség 1)

Magyarázza el, mi a különbség a keresztkorreláció és a matematikai konvolúció között a jelen specifikációban. Adjon meg egy 2×2-es, nemszimmetrikus kernelt, és a 4. pontban szereplő 3×3-as bemenettel számítsa ki mindkét művelet kimenetét stride 1, padding 0, dilation 1, bias nélkül. Elvárt beadandó: a két definíció képlete, a konkrét mátrixok.

37.1.6.2 F2 (levezetési, nehézség 2)

Legyen $H = 4, W = 5$, kernel 3×3, stride $(2;\ 2)$, padding $(1;\ 1)$, dilation $(2;\ 2)$. Vezesse le H_{out} és W_{out} értékét az output alakképletből, és sorolja fel azokat a bemeneti (n, c, α, β) indexhármasokat, amelyek a $Y[n, o, 0, 1]$ elemhez tartozó összegben nemzérus súllyal járulhatnak hozzá. Adja meg a pontos indexfeltételt.

37.1.6.3 F3 (implementációs, nehézség 2)

Töltse le vagy írja be a `reference_conv2d.py` fájlt, futtassa a `test_reference.py` tesztet, és rögzítse a kimenetet. Ezután egészítse ki egy olyan esettel, ahol $N = 2, C_{in} = 2, C_{out} = 3$, batchbeli képek 6×7-esek, kernel 2×2, stride $(1;\ 2)$, padding $(0;\ 1)$, dilation $(1;\ 2)$, bias bekapcsolva; a bemenetet `np.random.default_rng(20261020).random(...)` generálja. A teszt csak a shape-t és a dtype-ot ellenőrizze, értéket ne (a random miatt). Elvárt beadandó: a módosított teszt és a futtatás eredménye.

37.1.6.4 F4 (hibakeresési, nehézség 2)

Az alábbi kódrészlet a forwardot számolja, de hibás:

```
H_out = (H + 2*p_h - k_h*d_h) // s_h + 1
...
Y[n,o,i,j] += X[n,c, i*s_h + u*d_h - p_h, j*s_w + v*d_w - p_w] * K[o,c,u,v]
```

Keresse meg a hibát vagy a hibákat, magyarázza el a következményt a kimeneti méretre és az indexelésre, javítsa a kódot, és a 4. pont kézi példáján igazolja, hogy a javítás után a várt $47, 57, 77, 87$ értékek adódnak. Elvárt beadandó: a hibás sorok megnevezése, a javított sor, és a rövid verifikáció.

37.1.6.5 F5 (kísérleti/transzfer, nehézség 3)

Tervezzen kísérletet a `conv2d_reference` és a PyTorch `torch.nn.functional.conv2d` kimenetének összehasonlítására. A terv tartalmazza: a paraméterrácsot (legalább 6 különböző konfiguráció, beleértve paddinget, stride-ot, dilationt, több csatornát és bias nélküli esetet), a bemeneti adat generálását rögzített seeddel, a numerikus toleranciát, és annak indoklását, miért kell a PyTorch bemenetet is `float64`-re konvertálni. Elvárt beadandó: a protokoll szöveges leírása, a lehetséges buktatók, és legalább egy konfiguráció kézzel számolt várt kimeneti shape-je.

37.1.7 Megoldások és indoklás

37.1.7.1 F1 megoldás

A keresztkorreláció a kernel megfordítása nélkül:

$$Y[i,j] = \sum_{u,v} X[i+u, j+v] K[u,v],$$

a konvolúció pedig a megfordított kernellel:

$$(C * K)[i,j] = \sum_{u,v} X[i+u, j+v] K[k_h - 1 - u, k_w - 1 - v].$$

A 4. pont kernelével $K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ a keresztkorreláció (bias nélkül):

$$Y_{cc} = \begin{bmatrix} 37 & 47 \\ 67 & 77 \end{bmatrix}.$$

A konvolúcióhoz a kernel megfordítottja $\begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$, így

$$Y_{conv} = \begin{bmatrix} 1\cdot 4 + 2\cdot 3 + 4\cdot 2 + 5\cdot 1 & 2\cdot 4 + 3\cdot 3 + 5\cdot 2 + 6\cdot 1 \\ 4\cdot 4 + 5\cdot 3 + 7\cdot 2 + 8\cdot 1 & 5\cdot 4 + 6\cdot 3 + 8\cdot 2 + 9\cdot 1 \end{bmatrix} = \begin{bmatrix} 23 & 33 \\ 53 & 63 \end{bmatrix}.$$

A két eredmény különbözik, mert a kernel nem forgásszimmetrikus.

37.1.7.2 F2 megoldás

$H_{out} = \lfloor (4+2-2\cdot 2-1)/2 \rfloor + 1 = \lfloor (4+2-4-1)/2 \rfloor + 1 = \lfloor 1/2 \rfloor + 1 = 0+1 = 1$. Hasonlóan $W_{out} = \lfloor (5+2-2\cdot 2-1)/2 \rfloor + 1 = \lfloor (5+2-4-1)/2 \rfloor + 1 = \lfloor 2/2 \rfloor + 1 = 1+1 = 2$. Tehát Y térbeli alakja 1×2.

A $Y[n, o, 0, 1]$ elemhez tartozó összeg a $j = 1$ kimeneti oszlop miatt a paddingelt kép $1 \cdot s_w + v d_w = 2 + v \cdot 2$ oszlopait olvassa, $v = 0, 1, 2$ esetén: $2, 4, 6$. A paddingelt szélesség $5 + 2 = 7$, tehát a paddingelt oszlopindexek érvényesek ($2, 4, 6$ mind <7). A paddingelt oszlopindexekből az eredeti bemeneti oszlopindexek $\beta - p_w = 1, 3, 5$; ezek közül azonban csak $\beta = 1$ és $\beta = 3$ esik a $0 \le \beta < W = 5$ tartományba, a $\beta = 5$ a jobb oldali padding zónába esik, ezért a hozzájárulása nulla. A magassági rész: $i = 0$, $u = 0, 1, 2$, paddingelt sorok $0 \cdot 2 + u \cdot 2 = 0, 2, 4$, ebből eredeti sorindexek $-1, 1, 3$; az $\alpha = -1$ a padding zónába esik, a ténylegesen használt bemeneti sorok: 1 és 3.

A pontos indexfeltétel általánosan: a $Y[n, o, i, j]$ elemhez az (u, v) kernelpozíció akkor járulhat hozzá nemzérus súllyal, ha

$$0 \le i s_h + u d_h - p_h < H, \qquad 0 \le j s_w + v d_w - p_w < W,$$

és $0 \le u < k_h$, $0 \le v < k_w$. Összefoglalva: adott n, c mellett a lehetséges nemzérus bemeneti pozíciók $(\alpha, \beta) \in \{(1;\ 1);\ (1;\ 3);\ (3;\ 1);\ (3;\ 3)\}$, feltéve, hogy a megfelelő $K[o, c, u, v]$ súly nem nulla.

37.1.7.3 F3 megoldás

A módosított teszt:

```
def test_random_shape_dtype():
    rng = np.random.default_rng(20261020)
    X = rng.random((2,2,6,7), dtype=np.float64)
    K = rng.random((3,2,2,2), dtype=np.float64)
    b = rng.random((3,), dtype=np.float64)
    Y = conv2d_reference(X, K, b=b, stride=(1,2), padding=(0,1), dilation=(1,2))
    assert Y.shape == (2,3,5,4)  # H_out: (6+0-1-1)//1+1=5; W_out: (7+2-2-1)//2+1=4
    assert Y.dtype == np.float64
```

A kiegészített `test_reference.py`-val futtatva (Python 3.12.3, NumPy 2.5.4, pytest 9.2.0, CPU) a mért kimenet `6 passed` (a korábbi mérésben 0,05–0,06 s); az alak (shape) és az adattípus (dtype) ellenőrzése sikeres. A képlet ellenőrzése: $H_{out} = (6 + 0 - 1 \cdot (2-1) - 1)//1 + 1 = 5$, $W_{out} = (7 + 2 \cdot 1 - 2 \cdot (2-1) - 1)//2 + 1 = (7+2-2-1)//2 + 1 = 6//2 + 1 = 4$.

37.1.7.4 F4 megoldás

Az első hibás sorban a `k_h*d_h` szerepel a `(k_h-1)*d_h` helyett. A hibás képlet effektív kiterjedése $k_h d_h$, a helyes $(k_h - 1)d_h + 1$; a különbség $d_h - 1$, tehát $d_h = 1$ esetén a két képlet azonos, $d_h > 1$ esetén viszont a hibás képlet $d_h - 1$-gyel nagyobb kiterjedést feltételez, ezért H_{out} akár $\lfloor (d_h - 1)/s_h \rfloor$-mal is kisebb lehet a helyesnél (és negatívba is fordulhat, ahol a helyes érték pozitív). A második sorban a `- p_h` kivonás jó, de hiányzik annak ellenőrzése, hogy a kivonás után az index $[0;\ H-1]$ tartományban van-e; a paddingelt tömb használata nélkül a negatív vagy H-n túli indexek hibás memóriaterületet vagy Python negatív indexelést okoznak. A helyes megoldás a paddingelt tömb bevezetése, ahogy a `reference_conv2d.py` teszi: `X_pad`-del számolunk, és a kernel effektív mérete `(k_h-1)*d_h+1`.

Javított sorok:

```
H_out = (H + 2*p_h - (k_h-1)*d_h - 1) // s_h + 1
...
s += X_pad[n, c, i*s_h + u*d_h, j*s_w + v*d_w] * K[o, c, u, v]
```

A kézi példán a javított kód a 4. pontban megadott $[[47;\ 57], [77;\ 87]]$ mátrixot adja; ezt a `test_manual_nonsymmetric` teszt `rtol=0`, `atol=1e-12` mellett futtatva ellenőrzi (a labor tesztjei között lefutott, lásd az 5. pont futtatott kimenetét).

37.1.7.5 F5 megoldás

Protokollterv:

1. **Cél:** a referencia és a PyTorch `F.conv2d` azonossága a támogatott részhalmazon.
2. **Adat:** minden esethez `np.random.default_rng(20261020).random(shape)` `float64`-ben; a PyTorch tenzort `torch.from_numpy(...).double()`-ként hozzuk létre.
3. **Paraméterrács:** (a) $H = W = 8$, $k = 3$, s=1, p=1, d=1; (b) $H = 5, W = 6$, $k = 2$, s=1, p=0, d=2; (c) $H = 10, W = 7$, $k = 3$, s=2, p=1, d=1; (d) $N = 2, C_{in} = 2, C_{out} = 3$, $H = W = 6$, $k = 2$, s=2, p=1, d=1; (e) ugyanez $H = W = 6$ mellett, bias nélkül; (f) $H = 7, W = 7$, $k = 1$, s=1, p=0, d=1.
4. **Bias:** a PyTorch `F.conv2d` bias argumentumát külön adjuk át, vagy `None`-t, a referenciával összhangban.
5. **Tolerancia:** mindkét oldal `float64`, ezért az eltérés a `float64` kerekítési szintjén (néhány 10^{-16} relatív nagyságrend) marad, nem pontosan zérus; a gyakorlati elfogadás `np.allclose(..., rtol=1e-12, atol=1e-12)`, és a maximális abszolút eltérést is rögzítjük.
6. **Kiértékelés:** minden konfigurációra `np.allclose`, a maximális abszolút eltérés rögzítése.
7. **Buktatók:** a PyTorch alapértelmezett `float32` numerikus eltérést okoz; a csatornák sorrendje NCHW mindkettőnél; a dilation a PyTorchban ugyanúgy a kernel pontjai közötti lépés, nem a stride; a padding szimmetrikus, ezért a PyTorch `padding=(p_h,p_w)` formája használandó.
8. **Korlát:** az egyezés csak a fenti konfigurációkra bizonyított; nem állít általános implementációs ekvivalenciát.

Példa shape az (a) esetre: bemenet (1; 1; 8; 8), kernel (1; 1; 3; 3), s=1, p=1, d=1, így $H_{out} = W_{out} = (8 + 2 - 2 \cdot 1 - 1)/1 + 1 = 8$. A kimenet (1; 1; 8; 8).

37.1.8 Tipikus tévedés és ellenpélda

„A referencia implementáció minden Conv2d-opciót támogat." Ez hamis. A jelen kód például nem támogat `groups>1`-et, aszimmetrikus paddinget, más padding módokat, sem transzponált műveletet; az `output_padding` nem is `Conv2d`-, hanem `ConvTranspose2d`-paraméter, ezért a `Conv2d`-opciók között nem szerepel. Ellenpélda: ha a felhasználó `groups=2`-vel hívná meg, a függvény nem is fogad ilyen paramétert, és a specifikáció ezt kifejezetten kizárja. Egy részleges referencia csak a deklarált tartományon érvényes; aki ennél többet állít, az a 13.4. Határkezelés és frekvenciaintuíció alfejezetben tárgyalt „csendes elnyeléshez" hasonló hibát követ el, csak éppen a dokumentáció szintjén.

Önellenőrzés. A hallgató tegye fel magának: (1) a kernel effektív kiterjedését $(k-1)d+1$-ként számolta-e, nem kd-ként? (2) A paddingelt tömb indexelésekor a padding hozzáadása után a stride és dilation sorrendje helyes? (3) A bias dimenziója C_{out}, és a kimenet minden térbeli pozíciójához hozzáadódik? (4) A dtype mindenütt `float64`, és a hibás bemenetek kivételt

dobnak? (5) A kimeneti méret képlete a `floor` függvényt használja, és a nem pozitív (azaz 1-nél kisebb) kimeneti méret hibát ad-e?

Kulcs. (1) Igen, az effektív kiterjedés $(k-1)d+1$, a kd alak hibás. (2) Igen: előbb a paddinget adjuk hozzá, majd a dilation szerinti mintavételt és a stride-ot alkalmazzuk. (3) Igen, a bias hossza C_{out}, és minden térbeli pozícióhoz hozzáadódik. (4) Igen, minden számítás `float64`, és a hibás bemenet kivételt ad. (5) Igen, a kimeneti méret `floor`-ral számolódik, és a nem pozitív méret hibát jelez.

Visszatekintés. A 13.4. Határkezelés és frekvenciaintuíció alfejezetben a zero padding és a circular padding közötti különbség azt mutatta, hogy a határfeltétel a definíció része; itt a zero paddingot explicitté tettük a $\tilde{X}$ bevezetésével. A 14.4. Csoportos és depthwise konvolúció alfejezetben a `groups=1` azt jelentette, hogy a súly alakja $C_{out} \times C_{in} \times k_h \times k_w$; ezt most a specifikáció rögzíti.

37.1.9 Rövid összefoglaló

A referencia operátor specifikációja az NCHW keresztkorrelációt egyetlen indexképlettel definiálja: a paddingelt bemenetből stride és dilation szerint olvasott kernelablakok lineáris kombinációja, csatornánkénti bias hozzáadásával. A kimeneti méret levezetése a kernel utolsó pozíciójának illeszkedési feltételéből adódik, és a floor-képletben összegződik. A támogatott tartományt pontosan rögzítettük: groups=1, szimmetrikus zero padding, stride, dilation, opcionális bias, `float64`. A labor egy nemszimmetrikus kernelű kézi kontrollt és hibás alakokra negatív teszteket tartalmaz. A következő egység a backward pontos levezetését kezdi, amelyben ugyanezt a specifikációt használjuk a súly- és bemeneti gradienshez.

37.1.10 Felhasznált források

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [D11] PyTorch contributors (2026). *ConvTranspose2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.ConvTranspose2d.html. Hozzáférés: 2026-10-30. Kiinduló hivatalos URL; a tartalom a használat előtt ellenőrizendő.

37.2 Backward és ellenőrzési lefedettség

37.2.1 Cél, előismeret és induló kérdés

A 37.1. Operátorspecifikáció és kézi referencia egységben rögzítettük a teljes forward-szerződést: a referenciaoperátor NCHW bemenetre, OIHW súlyra, groups=1-gyel számol térbeli keresztkorrelációt szimmetrikus zero paddinggel, stride-dal és dilationnel. Itt azt kérdezzük, mi a helyes VJP (vektor–Jacobi-szorzat) mindhárom tanuló objektumra, azaz X inputra, K súlyra és b biasra. Induló kérdés: ha $H=3$, $k=2$, $s=1$, $p=0$, akkor a belső $X[1,1]$ pixel vajon egyetlen kimenethez tartozik-e, vagy többhöz is, és ez hogyan látszik a gradiens összegzésében? Becslés: több kimenethez tartozik, tehát a gradiensnak összegeznie kell az átfedő hozzájárulásokat; ezt levezetjük és számpéldán ellenőrizzük.

Előismeret: a forward indexképlete és outputalakja 37.1. Operátorspecifikáció és kézi referencia, a VJP oszlopkonvenciója 4.3. Jacobi-mátrix, láncszabály és VJP, a gradiensellenőrzés alapmérőszámai 10.3. Gradiensellenőrzés és numerikus hibamérce. A $dX/dK/db$ jelölés itt mindig a skalár loss szerinti gradiens, nem egy közvetlen mátrixderivált: $\nabla_X L$, $\nabla_K L$, $\nabla_b L$.

37.2.2 Jelölések, alakok és a forward felidézése

Legyen

$$X \in \mathbb{R}^{N\times C_{in}\times H\times W}, \qquad K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}, \qquad b \in \mathbb{R}^{C_{out}},$$

a stride s, a szimmetrikus padding p, a dilation d; s, a kernelméretek és d pozitív egész, a padding nemnegatív egész ($p \geq 0$). Az output

$$Y \in \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}}$$

alakú, ahol

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1,$$

és a szélességre ugyanígy, k_w-vel. A forward egyetlen indexképlete a zérussal kiegészített $\bar{X}$ tenzoron: $\bar{X}$ alakja $N \times C_{in} \times (H + 2p) \times (W + 2p)$, és

$$\bar{X}[n, c, a', b'] = X[n, c, a' - p, b' - p],$$

ha $a' \in [p;\ p + H - 1]$ és $b' \in [p;\ p + W - 1]$, egyébként 0. Ezzel

$$Y[n, o, i, j] = b[o] + \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[o, c, u, v]\, \bar{X}[n, c, si + du,\ sj + dv], \tag{1}$$

ahol $n \in [0;\ N-1]$, $o \in [0;\ C_{out}-1]$, $i \in [0;\ H_{out}-1]$, $j \in [0;\ W_{out}-1]$. Ez a PyTorch `Conv2d` dokumentált NCHW/OIHW tenzoralak- és stride/padding-konvenciója [D10]; a CNN-irodalom a műveletet gyakran konvolúcióként tárgyalja, noha az indexképlet valójában keresztkorreláció [S01].

Definíció (upstream gradiens). Legyen $L : \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}} \to \mathbb{R}$ differenciálható skalár veszteség, és legyen

$$\delta = \nabla_Y L \in \mathbb{R}^{N\times C_{out}\times H_{out}\times W_{out}},$$

azaz $\delta[n, o, i, j] = \frac{\partial L}{\partial Y[n,o,i,j]}$. A projekt kézi konvenciója $L = \frac{1}{2}\sum_{n,o,i,j} Y[n, o, i, j]^2$, ekkor $\delta = Y$; ez nem MSE és nem cross-entropy, csak a levezetés ellenőrzését egyszerűsíti.

A VJP alakja mindig megegyezik a megfelelő tanuló objektum alakjával:

Objektum	Gradiens	Alak
X	$\nabla_X L$	$N \times C_{in} \times H \times W$
K	$\nabla_K L$	$C_{out} \times C_{in} \times k_h \times k_w$
b	$\nabla_b L$	C_{out}

37.2.3 Tétel: a forward teljes VJP-je

Tétel (a konvolúciós forward minden VJP-je). Az (1) forwardra, differenciálható skalár L mellett:

1. Biasgradiens, minden $o \in [0;\ C_{out}-1]$-re:

$$\frac{\partial L}{\partial b[o]} = \sum_{n=0}^{N-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} \delta[n,o,i,j]. \tag{2}$$

2. Súlygradiens, minden o, c, u, v indexre:

$$\frac{\partial L}{\partial K[o,c,u,v]} = \sum_{n=0}^{N-1} \sum_{i=0}^{H_{out}-1} \sum_{j=0}^{W_{out}-1} \delta[n,o,i,j]\, \bar{X}[n,c,si+du,sj+dv]. \tag{3}$$

   Az eredeti X-szel ez azon (n,i,j) pozíciókra értendő, ahol $si-p+du \in [0;\ H-1]$ és $sj-p+dv \in [0;\ W-1]$, és ott az integrandus $X[n,c,si-p+du,sj-p+dv]$; a padded írásmód a határon kívüli pozíciókat zérussal kezeli.

3. Inputgradiens a zero-padded tartományon, minden n, c, a', b'-re:

$$\frac{\partial L}{\partial \bar{X}[n,c,a',b']} = \sum_{o=0}^{C_{out}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \sum_{\substack{i,j \\ a'=si+du,\ b'=sj+dv}} \delta[n,o,i,j]\, K[o,c,u,v], \tag{4}$$

   ahol az utolsó összeg azon $i \in [0;\ H_{out}-1]$, $j \in [0;\ W_{out}-1]$ párokra megy, amelyek a megadott afin egyenletet teljesítik. Végül a padding levágása:

$$\frac{\partial L}{\partial X[n,c,a,b]} = \frac{\partial L}{\partial \bar{X}[n,c,a+p,b+p]}, \qquad a \in [0;\ H-1],\ b \in [0;\ W-1]. \tag{5}$$

Bizonyítás. A $b[o]$ csak az $Y[n,o,i,j]$ kimenetekben jelenik meg, mindegyikben 1 együtthatóval, ezért a skalár láncszabály közvetlenül adja (2)-t.

A $K[o,c,u,v]$ szintén csak az o kimeneti csatorna értékeiben szerepel, de ott minden (n,i,j) pozícióban, $\bar{X}[n,c,si+du,sj+dv]$ együtthatóval. Ezért

$$\frac{\partial L}{\partial K[o,c,u,v]} = \sum_{n,i,j} \delta[n,o,i,j] \frac{\partial Y[n,o,i,j]}{\partial K[o,c,u,v]},$$

és a parciális derivált éppen a megfelelő $\bar{X}$-érték, ami (3)-at adja.

Az inputgradiensnél először a padded változóra alkalmazzuk a láncszabályt. Egy rögzített $\bar{X}[n,c,a',b']$ elem azokban az $Y[n,o,i,j]$ kimenetekben szerepel, amelyekre létezik u,v, hogy $a' = si+du$ és $b' = sj+dv$; ezekben a parciális derivált $K[o,c,u,v]$. Ez pontosan (4). Minden összeg véges, ezért az összegzések sorrendje szabadon felcserélhető. Az (5) vágás abból következik, hogy a padded tenzor belső része az eredeti X eltolt másolata, tehát a megfelelő Jacobi-rész az identitás; a padding tartománybeli gradiens nem tartozik egyetlen X-elemhez sem. □

A (4) összeg a scatter-add (szétszórás–összegzés) művelet: minden kimeneti pozícióból szétszórjuk a δ hozzájárulást a bemenetre, majd az azonos helyre érkező tagokat összeadjuk. Ez az „overlap” (átfedés) kvantitatív alakja: ugyanaz az input pozíció több kimeneti pozíciót is érint, és a gradiens a közösen használt K-súlyokkal súlyozva összegződik. A (3) pedig azt mondja, hogy a súlygradiens a δ és az input térbeli keresztkorrelációja, azzal a fontos különbséggel, hogy a kimeneti pozíciók szerint összegzünk, tehát ugyanaz a súly minden pozícióból kap hozzájárulást.

Megjegyzés (adjoinált, nem inverz). A (4)–(5) művelet a forward lineáris operátor adjungáltja a megadott stride/padding/dilation konvencióval; kapcsolódik a `ConvTranspose2d`-hez, de nem általános inverz, és a kernel megfordítási konvenciója attól függ, hogy a forwardot korrelációnak vagy matematikai konvolúciónak definiáltuk. A részletes adjungált-tulajdonságot 37.3. Linearitás, adjungált és határfeltétel tárgyalja, a könyvtári háttér a [D05] és [D11] körébe tartozik, ahol a [D11] a `ConvTranspose2d` dokumentációja; ebben az egységben a közvetlen (4)–(5) scatter-add formulát használjuk.

37.2.4 Eredeti kézi kis példa

Legyen $N = 2$, $C_{in} = C_{out} = 1$, $H = W = 3$, $k_h = k_w = 2$, $s = 1$, $p = 0$, $d = 1$, tehát $H_{out} = W_{out} = 2$. Az inputok:

$$X[0] = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \qquad X[1] = \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix},$$

$K = ((1;\ 0), (0;\ 1))$, $b = [0{,}5]$. A forward kézzel:

$$Y[0] = \begin{pmatrix} 6{,}5 & 8{,}5 \\ 12{,}5 & 14{,}5 \end{pmatrix}, \qquad Y[1] = \begin{pmatrix} 14{,}5 & 12{,}5 \\ 8{,}5 & 6{,}5 \end{pmatrix}.$$

A kvadratikus loss miatt $\delta = Y$.

A (2) képlet szerint $gb = 6{,}5 + 8{,}5 + 12{,}5 + 14{,}5 + 14{,}5 + 12{,}5 + 8{,}5 + 6{,}5 = 84$, azaz $gb = [84]$.

A (3) képletet a négy kernelpozícióra alkalmazva mind a négy $gK[u, v]$ érték 460. A $gK[0, 0]$ például a δ és a bal felülre igazított input keresztkorrelációja: az $n = 0$ képből $6{,}5 \cdot 1 + 8{,}5 \cdot 2 + 12{,}5 \cdot 4 + 14{,}5 \cdot 5 = 146$, az $n = 1$ képből $14{,}5 \cdot 9 + 12{,}5 \cdot 8 + 8{,}5 \cdot 6 + 6{,}5 \cdot 5 = 314$, összesen $146 + 314 = 460$. A $gK[0, 1]$-nél a két kép rendre 188, illetve 272, összesen $188 + 272 = 460$. A $gK[1, 0]$-nál a két kép hozzájárulása 272, illetve 188 ($272 + 188 = 460$), a $gK[1, 1]$-nél pedig 314, illetve 146 ($314 + 146 = 460$), ezért lesz mind a négy érték 460. Tehát

$$gK = \begin{pmatrix} 460 & 460 \\ 460 & 460 \end{pmatrix}.$$

Az inputgradiens (4)–(5) szerint scatter-add módon: inicializálunk egy $N \times 1 \times 3 \times 3$ zérus tenzort, és minden (n, i, j, u, v) esetén hozzáadunk $\delta[n, i, j]K[u, v]$-t a $(i + u, j + v)$ cellához. A belső $X[0, 1, 1] = 5$ elemre például két tag esik: $\delta[0, 1, 1]K[0, 0] = 14{,}5$ és $\delta[0, 0, 0]K[1, 1] = 6{,}5$, összegük 21. A teljes eredmény:

$$gX[0] = \begin{pmatrix} 6{,}5 & 8{,}5 & 0 \\ 12{,}5 & 21 & 8{,}5 \\ 0 & 12{,}5 & 14{,}5 \end{pmatrix}, \qquad gX[1] = \begin{pmatrix} 14{,}5 & 12{,}5 & 0 \\ 8{,}5 & 21 & 12{,}5 \\ 0 & 8{,}5 & 6{,}5 \end{pmatrix}.$$

A 0 értékű cellák a jobb felső $(0;\ 2)$ és a bal alsó $(2;\ 0)$ helyen vannak, nem a jobb alsó sarokban (ott $gX[0, 2, 2] = 14{,}5$ és $gX[1, 2, 2] = 6{,}5$). Ezek a cellák a scatter-add során csak a $K[0, 1]$, illetve $K[1, 0]$ útvonalon kapnának hozzájárulást, és a $K = ((1;\ 0), (0;\ 1))$ kernelben pontosan ezek a súlyok nullák; a nullák tehát a kernel nulláiból és az útvonalakból következnek, nem önmagában a $p = 0$ paddingből.

37.2.5 Véges differencia és autograd: mit ellenőrizzünk?

Legyen θ az X, K, b bármely megkötött skalár bejegyzése, és tekintsük a többi változót rögzítettnek. A centrális differencia

$$\Delta_\varepsilon L(\theta) = \frac{L(\theta+\varepsilon) - L(\theta-\varepsilon)}{2\varepsilon}.$$

Mivel a forward θ-ban affin, az $L = \frac{1}{2}\sum Y^2$ veszteség pedig kvadratikus, a centrális differencia valós aritmetikában *algebrailag pontos*; az eltérés csak a lebegőpontos kerekítésből származik. A differencia képletében a kerekítési hiba nagyságrendje $u\,|L|/\varepsilon$ körül várható, ahol u a gépi epszilon (float64-ben $u \approx 2{,}2 \cdot 10^{-16}$); ez levezetett becslés, nem mért érték. Ezért a két számítás egyezését a `torch.allclose` VAGY-logikájával ítéljük meg: egy elem elfogadott, ha $|g_{ref} - g_{fd}| < \text{atol}$, vagy ha a relatív hiba rtol alatt marad. A laborban atol $= 10^{-8}$, rtol $= 10^{-6}$, $\varepsilon = 10^{-5}$. A relatív hibát stabil nevezővel számoljuk:

$$e_{rel} = \frac{|g_{ref} - g_{fd}|}{\max(|g_{ref}|, |g_{fd}|, \eta)}, \qquad \eta = 10^{-12}.$$

A normálás nélküli, nullához közeli gradiensű elemeket éppen az abszolút korlát fogja el; a VAGY-kapcsolat miatt a nagy relatív hiba ezeknél nem okoz téves elutasítást.

A PyTorch-autograd egy független második kontroll: a saját forwardot `float64` leaf tenzorokkal futtatjuk, `L.backward()` után `X.grad`, `K.grad`, `b.grad` összevethető a referencia (2)–(5) értékeivel. A ReLU-val vagy más törésponttal ellentétben a konvolúciós forward a vizsgált pontokon sima, tehát a véges differenciának nincs konvenciófüggő törésponti eltérése; a nem sima eset kezelését 10.3. Gradiensellenőrzés és numerikus hibamérce rögzíti.

37.2.6 Konkrét labor: f64 gradiensellenőrzés több kis tensoron

A labor CPU-n, `torch.float64` tenzorokkal fut. Bemeneti konfigurációk:

Azonosító	N	C→M	H×W	kernel	s	p	d
A	2	1→1	3×3	2×2	1	0	1
B	1	2→3	4×5	3×2	2	1	1
C	3	2→2	6×5	3×3	1	1	2

Az A eset az előző kézi példa; B és C a stride, padding és dilation határfeltételeit fedi. Minden esetre a labor kiszámolja a forwardot, a referencia backwardot, a centrális differenciát *minden* X, K, b bejegyzésre, és a PyTorch-autograd gradienseket; a hibákat CSV-be menti.

A teljes, futtatható szkript:

```
#!/usr/bin/env python3
"""37.2. lecke labor: conv2d backward ellenőrzés. CPU, float64.
A szkriptet a hallgató futtatja; a kimenet a futtatás eredménye:
    python check_backward.py --eps 1e-5 --out error_table.csv
"""
import argparse, csv
import torch

def conv2d_forward(X, K, b, s, p, d):
    N, C, H, W = X.shape
    M, Ck, kh, kw = K.shape
    if Ck != C:
        raise ValueError(f"K bemeneti csatornaszáma {Ck}, X-é {C}")
```

```
    Ho = (H + 2*p - d*(kh-1) - 1)//s + 1
    Wo = (W + 2*p - d*(kw-1) - 1)//s + 1
    Y = torch.empty(N, M, Ho, Wo, dtype=X.dtype)
    for n in range(N):
        for o in range(M):
            for i in range(Ho):
                for j in range(Wo):
                    acc = b[o] + 0.0
                    for c in range(C):
                        for u in range(kh):
                            for v in range(kw):
                                a = s*i - p + d*u
                                bq = s*j - p + d*v
                                if 0 <= a < H and 0 <= bq < W:
                                    acc = acc + X[n, c, a, bq] * K[o, c, u, v]
                    Y[n, o, i, j] = acc
    return Y

def conv2d_backward(X, K, dY, s, p, d):
    N, C, H, W = X.shape
    M, Ck, kh, kw = K.shape
    if Ck != C:
        raise ValueError(f"K bemeneti csatornaszáma {Ck}, X-é {C}")
    Ho = (H + 2*p - d*(kh-1) - 1)//s + 1
    Wo = (W + 2*p - d*(kw-1) - 1)//s + 1
    if tuple(dY.shape) != (N, M, Ho, Wo):
        raise ValueError(f"dY alakja {tuple(dY.shape)}, várt {(N, M, Ho, Wo)}")
    gb = dY.sum(dim=(0, 2, 3))
    gK = torch.zeros_like(K)
    for n in range(N):
        for o in range(M):
            for i in range(Ho):
                for j in range(Wo):
                    for u in range(kh):
                        for v in range(kw):
                            a = s*i - p + d*u
                            bq = s*j - p + d*v
                            if 0 <= a < H and 0 <= bq < W:
                                g = dY[n, o, i, j]
                                gK[o, :, u, v] += g * X[n, :, a, bq]
    Hp, Wp = H + 2*p, W + 2*p
    gXp = torch.zeros(N, C, Hp, Wp, dtype=X.dtype)
    for n in range(N):
        for o in range(M):
            for i in range(Ho):
                for j in range(Wo):
                    for u in range(kh):
                        for v in range(kw):
                            a = s*i + d*u
                            bq = s*j + d*v
                            if 0 <= a < Hp and 0 <= bq < Wp:
                                g = dY[n, o, i, j]
                                gXp[n, :, a, bq] += g * K[o, :, u, v]
    gX = gXp[:, :, p:p+H, p:p+W]  # p=0 esetén ez az identitás-szelet
    return gX, gK, gb

def quad_loss(Y):
    return 0.5 * torch.sum(Y * Y)

def fd_all(f, theta, eps):
    theta = theta.detach().clone()
    flat = theta.reshape(-1)
    out = torch.empty_like(flat)
```

```
    for q in range(flat.numel()):
        plus = theta.clone()
        minus = theta.clone()
        plus.reshape(-1)[q] = flat[q].item() + eps
        minus.reshape(-1)[q] = flat[q].item() - eps
        out[q] = (f(plus) - f(minus)) / (2.0 * eps)
    return out.reshape(theta.shape)

def max_abs_err(a, b):
    return (a - b).abs().max().item()

def max_rel_err(a, b, eta=1e-12):
    d = (a - b).abs()
    den = torch.maximum(a.abs(), b.abs()).clamp_min(eta)
    return (d / den).max().item()

def check_case_A_hand():
    X = torch.tensor([[[[1,2,3],[4,5,6],[7,8,9]]],
                      [[[9,8,7],[6,5,4],[3,2,1]]]], dtype=torch.float64)
    K = torch.tensor([[[[1,0],[0,1]]]], dtype=torch.float64)
    b = torch.tensor([0.5], dtype=torch.float64)
    Y = conv2d_forward(X, K, b, 1, 0, 1)
    gX, gK, gb = conv2d_backward(X, K, Y, 1, 0, 1)
    exp_gX = torch.tensor(
        [[[[6.5,8.5,0.0],[12.5,21.0,8.5],[0.0,12.5,14.5]]],
         [[[14.5,12.5,0.0],[8.5,21.0,12.5],[0.0,8.5,6.5]]]],
        dtype=torch.float64)
    assert torch.allclose(gX, exp_gX, atol=1e-12, rtol=0)
    f64 = torch.float64
    exp_gK = torch.tensor([[[[460, 460], [460, 460]]]], dtype=f64)
    assert torch.allclose(gK, exp_gK, atol=1e-12, rtol=0)
    assert torch.allclose(gb, torch.tensor([84.0], dtype=f64), atol=1e-12, rtol=0)
    print("A kézi eset rendben.")

CASES = {
    "A": dict(seed=0, shape=(2,1,1,3,3), kernel=(2,2),
              stride=1, padding=0, dilation=1),
    "B": dict(seed=1, shape=(1,2,3,4,5), kernel=(3,2),
              stride=2, padding=1, dilation=1),
    "C": dict(seed=2, shape=(3,2,2,6,5), kernel=(3,3),
              stride=1, padding=1, dilation=2),
}

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--eps", type=float, default=1e-5)
    ap.add_argument("--out", type=str, default="error_table.csv")
    args = ap.parse_args()
    check_case_A_hand()
    rows = []
    for case_id, cfg in CASES.items():
        torch.manual_seed(cfg["seed"])
        N, C, M, H, W = cfg["shape"]
        kh, kw = cfg["kernel"]
        s, p, d = cfg["stride"], cfg["padding"], cfg["dilation"]
        X = torch.rand(N, C, H, W, dtype=torch.float64) * 2 - 1
        K = torch.rand(M, C, kh, kw, dtype=torch.float64) * 0.6 - 0.3
        b = torch.rand(M, dtype=torch.float64) * 0.4 - 0.2
        X.requires_grad_(True); K.requires_grad_(True); b.requires_grad_(True)
        Y = conv2d_forward(X, K, b, s, p, d)
        dY = Y  # L = 0.5 * sum(Y**2)
        gX, gK, gb = conv2d_backward(X, K, dY, s, p, d)
        L = quad_loss(Y)
```

```
        L.backward()
        pairs = [("X", gX, X.grad), ("K", gK, K.grad), ("b", gb, b.grad)]
        for name, ga, gt in pairs:
            rows.append([case_id, name, "autograd",
                         max_abs_err(ga, gt), max_rel_err(ga, gt)])
        Xd = X.detach().clone()
        Kd = K.detach().clone()
        bd = b.detach().clone()
        fX = lambda t: quad_loss(conv2d_forward(t, Kd, bd, s, p, d))
        fK = lambda t: quad_loss(conv2d_forward(Xd, t, bd, s, p, d))
        fb = lambda t: quad_loss(conv2d_forward(Xd, Kd, t, s, p, d))
        for name, ga, gf in [("X", gX, fd_all(fX, Xd, args.eps)),
                             ("K", gK, fd_all(fK, Kd, args.eps)),
                             ("b", gb, fd_all(fb, bd, args.eps))]:
            rows.append([case_id, name, "fd",
                         max_abs_err(ga, gf), max_rel_err(ga, gf)])
    atol, rtol = 1e-8, 1e-6
    failed = [r for r in rows if not (r[3] < atol or r[4] < rtol)]
    if failed:
        raise SystemExit(
            f"Elfogadási hiba: {len(failed)} sor meghaladja a korlátot; "
            f"első példa: {failed[0]}")
    with open(args.out, "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["case", "object", "method", "max_abs_err", "max_rel_err"])
        w.writerows(rows)
    print(f"Elfogadási feltétel (atol={atol}, rtol={rtol}): {len(rows)} sor rendben.")
    print(f"Tábla: {args.out}; sorok: {len(rows)}")

if __name__ == "__main__":
    main()
```

A labor ellenőrzése: a `check_case_A_hand` a fenti kézi táblázatot 10^{-12}-es abszolút toleranciával kéri (a kis egész és fél-egész értékek float64-ben pontosan ábrázolhatók, ezért ez nem „bitközeli”, hanem szigorú abszolút korlát); minden további sorra az elfogadás a `torch.allclose` VAGY-logikája: `max_abs_err < 1e-8` vagy `max_rel_err < 1e-6`, és a `main()` ezt ellenőrzi is, hiba esetén nemnulla kilépéssel. A B és C eset számszaki kimenetét a szkript futtatása állítja elő; a futtatott kimenet (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 10 szál) ellenőrizhető kulcssorai: `A kézi eset rendben.`, `Elfogadási feltétel (atol=1e-08, rtol=1e-06): 18 sor rendben.` és a CSV fejléce `case,object,method,max_abs_err,max_rel_err`. Az eredmény értelmezése: a hiba táblázat csak a felsorolt `(N,C,M,H,W,k,s,p,d)` konfigurációkra ad bizonyítékot, és csak float64 CPU-n; a groups 1, az aszimmetrikus padding (azaz $p_h \neq p_w$, ami kívül esik az operátor skaláris p-jén) és a float32 approximáció kívül esik a teszten (36.2. Ismétlések és eredménybizonytalanság gondolata a több teszt és a kimondatlan tartomány viszonyáról). A B eset 3×2-es kernele éppen azt mutatja, hogy a nem négyzetes (téglalap alakú) kernel tesztelt; az „aszimmetrikus padding” ettől különböző fogalom, mert a padding oldalankénti eltérését jelöli, nem a kernelét.

37.2.7 Feladatok

F1 (fogalmi, nehézség 1). $N = 4$, $C_{in} = 3$, $C_{out} = 5$, $H = W = 8$, $k_h = k_w = 3$, $s = 2$, $p = 1$, $d = 1$. Add meg H_{out}, W_{out} értékét és a δ, $\nabla_X L$, $\nabla_K L$, $\nabla_b L$ pontos alakját. Mely tengelyek mentén összegzi a biasgradiens a δ értékeit? Beadandó: az alakok és az összegző tengelyek listája.

F2 (levezetési, nehézség 2). Egydimenziós eset: $N = 1$, $C_{in} = C_{out} = 1$, $H = 4$, $k_h = 2$, $s = 1$, $p = 0$, $d = 1$; $X = [1;\ -1;\ 1;\ 0]$, $K = [2;\ 1]$, $b = [0]$, és $L = \frac{1}{2}\sum Y^2$. Számold ki Y-t,

majd a levezetett képletekkel gX, gK, gb értékét. Jelöld meg azokat a cellákat, ahol két átfedő hozzájárulás összegződik. Beadandó: a négy vektor/skalár és a két átfedő tag kifejtése.

F3 (implementációs, nehézség 2). Írj `crop_safe(t, padding, h, w)` függvényt a projektben a (5) vágásra úgy, hogy `padding==0` esetén ne keletkezzen üres szelet; ellenőrizd $p = 0$ és $p = 2$ mellett egy véletlen float64 tensoron. Beadandó: a függvény kódja, a két hívás shape-je és a `torch.equal` eredménye.

F4 (hibakeresési, nehézség 2). Az alábbi 1D inputgradiens-kód a F2 feladatot próbálja reprodukálni:

```
gX = torch.zeros(4)
for i in range(3):
    for u in range(2):
        gX[i + u] = dY[i] * K[u]   # hiba itt
```

Hol a hiba, mi a numerikus következménye a F2 példában, és hogyan javítod? Beadandó: a hibás és a javított kód, a F2 értékeivel való összevetés.

F5 (kísérleti/transzfer, nehézség 3). Tervezd meg a backward-ellenőrzés konfigurációs mátrixát legalább hat esetre a támogatott operátortartományból: a paddinget, a stride-ot és a dilationt izoláltan (egy tényezőt változtatva) módosítsd, a csatorna- és batch-méretet viszont ne izoláltan, hanem a B és C esethez hasonlóan a többi dimenzióval együtt változtatva; rögzítsd, mely kombinációkat *nem* teszteled. Add meg a CSV sémáját, az elfogadási korlátokat és a következtetés érvényességi határát. Beadandó: protokoll, hat konfiguráció indoklással és a mentett tábla.

37.2.8 Megoldások és indoklás

F1 megoldás.

$$H_{out} = \left\lfloor \frac{8 + 2 - 1 \cdot 2 - 1}{2} \right\rfloor + 1 = \left\lfloor \frac{7}{2} \right\rfloor + 1 = 3 + 1 = 4,$$

tehát $H_{out} = W_{out} = 4$. Az alakok: $\delta \in \mathbb{R}^{4\times5\times4\times4}$, $\nabla_X L \in \mathbb{R}^{4\times3\times8\times8}$, $\nabla_K L \in \mathbb{R}^{5\times3\times3\times3}$, $\nabla_b L \in \mathbb{R}^5$. A biasgradiens a δ (n, i, j), azaz a 0, 2, 3 tengely szerinti összegzéséből áll, a kimeneti csatorna o tengelyt megtartva.

F2 megoldás. A forward (1) szerint $Y[i] = 2X[i] + X[i+1]$, tehát $Y = [2 \cdot 1 + (-1);\ 2(-1) + 1;\ 2 \cdot 1 + 0] = [1;\ -1;\ 2]$. A kvadratikus loss miatt $\delta = Y$. A (2)–(5) egydimenziós változata:

$$gb = \sum_i \delta[i] = 1 - 1 + 2 = 2,$$

$$gK[u] = \sum_{i=0}^{2} \delta[i] X[i+u], \qquad gK[0] = 1{\cdot}1 + (-1)(-1) + 2{\cdot}1 = 4, \quad gK[1] = 1(-1) + (-1)1 + 2{\cdot}0 = -2,$$

$$gX[a] = \sum_{\substack{i \in [0;\ 2],\, u \in [0;\ 1] \\ a = i+u}} \delta[i] K[u],$$

tehát $gX = [2;\ -1;\ 3;\ 2]$. Az átfedő pozíciók: $gX[1] = 1 \cdot 1 + (-1) \cdot 2 = -1$ (a $\delta_0 K_1$ és $\delta_1 K_0$ tagok), $gX[2] = (-1) \cdot 1 + 2 \cdot 2 = 3$. A $gX[0] = 2$ és $gX[3] = 2$ egy-egy tagból állnak; ez mutatja, hogy a határnál nincs padding-hozzájárulás.

F3 megoldás.

```
def crop_safe(t, padding, h, w):
    row = slice(padding, padding + h) if padding else slice(None)
    col = slice(padding, padding + w) if padding else slice(None)
    return t[:, :, row, col]

# ellenőrzés
torch.manual_seed(0)
t0 = torch.rand(2, 3, 6, 6, dtype=torch.float64)
print(crop_safe(t0, 0, 6, 6).shape)   # torch.Size([2, 3, 6, 6])
print(torch.equal(crop_safe(t0, 0, 6, 6), t0))   # True
t2 = torch.rand(2, 3, 10, 10, dtype=torch.float64)
out = crop_safe(t2, 2, 6, 6)
print(out.shape)   # torch.Size([2, 3, 6, 6])
print(torch.equal(out, t2[:, :, 2:8, 2:8]))  # True
```

A `slice(None)` ág azért kell, mert Pythonban `-0 == 0`, ezért a naiv `t[:, :, p:-p]` a $p = 0$ esetben `t[:, :, 0:0]` üres tenzort adna. A megoldás szélsőeset-kontrollja: $p = 0$ shape-azonosság, $p > 0$ érték-egyezés.

F4 megoldás. A hiba az `=` hozzárendelés a `+=` helyett. Az átfedő pozíciókat a ciklus többször látogatja, és a hozzárendelés felülírja a korábbi hozzájárulást, tehát a gradiens nem összegződik. A F2 példában a hiba hatására $gX[1]$ végül a legutolsó $i = 1, u = 0$ tag, $(-1) \cdot 2 = -2$ lesz a helyes -1 helyett, $gX[2]$ pedig 4 a helyes 3 helyett. Javítás:

```
gX = torch.zeros(4)
for i in range(3):
    for u in range(2):
        gX[i + u] += dY[i] * K[u]   # += : összegzés
# ellenőrzés: gX == [2, -1, 3, 2]
```

A javított verzió a F2 összes értékét visszaadja, beleértve a kétátfedéses pozíciókat.

F5 megoldásminta. Kérdés: a referencia backward egyezik-e az autograddal és a centrális differenciával a támogatott paramétertartományon? Egy megfigyelés: egy konfiguráció, benne a `max_abs_err` és `max_rel_err` mindhárom objektumra, módszerenként. Minimum hat eset:

case	változtatott dimenzió	N, C→M, H×W, k	s	p	d
A	alap	2, 1→1, 3×3, 2×2	1	0	1
B	stride+p	1, 2→3, 4×5, 3×2	2	1	1
C	padding+dilation	3, 2→2, 6×5, 3×3	1	1	2
A_p2	csak p>0	2, 1→1, 3×3, 2×2	1	2	1
A_s2	csak s=2	2, 1→1, 3×3, 2×2	2	0	1
A_d3	csak d=3, nagyobb H	2, 1→1, 6×6, 2×2	1	0	3

A csatorna- és batch-méretet a B és C esetekben változtatjuk, hogy az összegző tengelyek ne csak szingletonok legyenek. Az A_s2 esetben $H = 3, k = 2, s = 2, p = 0$ mellett $H_{out} = 1$, tehát a kimenet 1×1-es: ez az eset csak a degenerált, egyetlen kimeneti pozíció határviselkedését fedi, a stride általánosabb hatását a B eset ($s = 2$ mellett $H_{out} = 2$, $W_{out} = 3$) mutatja. Elfogadás: float64, $\varepsilon = 10^{-5}$, `atol=1e-8` vagy `rtol=1e-6` (allclose-logika); minden bejegyzésre véges differencia, nem mintavétel. A CSV oszlopai: `case, object, method, max_abs_err, max_rel_err`. Érvényességi határ: a hat eset nem reprezentatív minta a parametertérből; az állítás pontosan a tesztelt `(s,p,d,k)` halmazra szól, groups=1-re, négyzetes paddingre, float64 CPU-n. Nem tárgyalt kombinációk (pl. $s = 3$, $p = 3$, aszimmetrikus padding) további kutatást igényelnek; ez ugyanaz a korlát, mint a 36.2. Ismétlések és eredménybizonytalanság alfejezetben a seedek és a fix tesztkészlet viszonya.

37.2.9 Tipikus tévedés és ellenpélda

A leggyakoribb hiba a „minden rendben” állítás a tesztelt tartomány pontos megnevezése nélkül. Tegyük fel, hogy valaki csak az $s = 1$, $p = 0$, $d = 1$, $k = 2$ esetet futtatja, és a gradiens ott egyezik; ebből nem következik a paddingelt ág helyessége. Konkrét ellenpélda: ha az inputgradiensben a padded koordinátát a kód `a = s*i + d*u` helyett `a = s*i - p + d*u`-val számolja a gXp-scatterhez is, akkor $p = 0$-nál tökéletesen egyezik, de $p = 1$-nél a padding miatt a (`a=-1`) hozzájárulások kívülre kerülnek, és a vágás után rossz gX adódik. A kijelentés helyes formája: „a backward a labor A–C konfigurációján – és az F5-ben hat esetre kiterjesztett változatban – float64 CPU-n, a megadott toleranciával egyeznie kell az autograddal és a véges differenciával; más tartományra nincs bizonyíték”. Ez a 36.2. Ismétlések és eredménybizonytalanság alfejezetben bevezetett, a fix protokoll és a kimondott tartomány közötti különbség közvetlen alkalmazása.

37.2.10 Önellenőrzés és korábbi fogalmak áttekintése

Önellenőrzéshez: (i) mindhárom gradiens alakja megegyezik a megfelelő objektum alakjával; (ii) $p = 0$ mellett a padded és a vágott tenzor azonos; (iii) a F2 1D példában a gX összege $2 - 1 + 3 + 2 = 6$, ami a láncszabály szerint $\sum_i \delta_i \sum_u K_u = 2 \cdot 3 = 6$, tehát a teljes gradiens konzisztens. Visszatekintés: a 37.1. Operátorspecifikáció és kézi referencia forward-egyezménye adja a (1) indexhalmazt, a 4.3. Jacobi-mátrix, láncszabály és VJP pedig a (4)–(5) scatter-add alakját. A forward VJP-je a veszteség konkrét választásától csak a $\delta = \nabla_Y L$ tenzoron keresztül függ: a (2)–(5) képletekben δ értékei szerepelnek, a δ alakja pedig megegyezik Y alakjával, amelyet a forward rögzít.

37.2.11 Rövid összefoglaló

A konvolúciós forward (1) lineáris tagokból áll, ezért mindhárom VJP véges összeg. A biasgradiens a δ batch- és térbeli összege; a súlygradiens a δ és az input keresztkorrelációja; az inputgradiens a δ-t a K-súlyokkal scatter-add módon szétszórja a padded tartományra, majd levágja a paddinget. A centrális differencia a kvadratikus loss miatt valós aritmetikában pontos, így float64 mellett az autograddal együtt két független numerikus kontrollt ad. A lab három kis konfiguráción (az F5-ben további három esettel kiterjeszthetően), CSV-táblával, `1e-8` abszolút vagy `1e-6` relatív korláttal ellenőriz; a $p = 0$ üres szelet és az átfedések `+=` összegzése a leggyakoribb implementációs hibák. A következő 37.3. Linearitás, adjungált és határfeltétel egység a linearitást, az adjungált tulajdonságot és a határfeltételeket vizsgálja.

37.2.12 Források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-20.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-20.
- [D11] PyTorch contributors (2026). *ConvTranspose2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.ConvTranspose2d.html. Hozzáférés: 2026-10-20.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. `https://cs231n.github.io/convolutional-networks/`. Hozzáférés: 2026-10-20.

37.3 Linearitás, adjungált és határfeltétel

37.3.1 Cél, előismeret és induló kérdés

A 37.2. Backward és ellenőrzési lefedettség egységben levezettük a konvolúciós réteg $\nabla_X, \nabla_K, \nabla_b$ értékeit, és az átfedő bemeneti foltok miatt az inputgradienst összegezéssel képeztük. Most ugyanazt az operátort matematikai leképezésként vizsgáljuk, és szétválasztjuk a bizonyított operátortulajdonságot a választott implementáció futási tulajdonságától. A szükséges előismeret a 37.2. Backward és ellenőrzési lefedettség referencia-összegező konvolúciója, a 4.3. Jacobi-mátrix, láncszabály és VJP alfejezetből a Jacobi-mátrix és a 13.2. Konvolúció és cross-correlation alfejezetből a cross-correlation (keresztkorreláció) definíciója.

Induló kérdés: egy 3×3-as, 1 csatornás képen 3×3-as kernellel, stride 1 és padding 0 mellett a bias nélküli réteg hány dimenzióból hány dimenzióba képez? Van-e lineáris inverze? Tippeljen, mielőtt a levezetést elolvassa; a választ a 37.3.5. alszakasz dimenziós érveléssel és konkrét nulltér-példával ellenőrzi.

37.3.2 Operátordefiníció és jelölések

A [D10] szerint a `Conv2d` térbeli **cross-correlationt** (keresztkorrelációt) számol; a könyvben a T_K operátor ezt valósítja meg. A konvolúció és a cross-correlation megkülönböztetését az [S01] is tárgyalja. Legyen a bemenet $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, a súly $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, a bias $b \in \mathbb{R}^{C_{out}}$, a stride $s \geq 1$, a szimmetrikus zero padding $p \geq 0$, a dilation $d \geq 1$. A kimenet $Y = T_{K,b}(X) \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$, ahol

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1,$$

és W_{out} ugyanígy számolandó. Az alapeset $d = 1$, $s = 1$; az alábbi indexelt patch-, scatter- és VJP-képletek $(u, v$ indexekkel) a $d = 1$ alapesetre szólnak. Dilation esetén a forward, a scatter és a backward képletében a kernelindex $d\,u$, illetve $d\,v$ alakban jelenik meg (ahol d a skaláris dilation), és a kimeneti méretet is a dilationt tartalmazó képlet adja. A bias nélküli operátort $T_K = T_{K,0}$ jelöli, és a komponensenkénti definíció

$$T_K(X)_{n,o,i,j} = \sum_{c=0}^{C_{in}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X_{n,c,\,si-p+u,\,sj-p+v}\, K_{o,c,u,v}.$$

A bias az összegzés után adódik hozzá: $T_{K,b}(X) = T_K(X) + B$, ahol $B_{n,o,i,j} = b_o$. Az X-et vektorizálhatjuk $\mathrm{vec}(X) \in \mathbb{R}^{NC_{in}HW}$ sorrendben; ekkor a rögzített K melletti T_K lineáris leképezést egy $M_K \in \mathbb{R}^{(NC_{out}H_{out}W_{out}) \times (NC_{in}HW)}$ mátrix reprezentálhat.

37.3.3 A bias nélküli linearitás

A „lineáris” szót itt a lineáris algebra pontos értelmében használjuk.

Tétel (a bias nélküli operátor linearitása). Rögzített K, s, p, d és alakok mellett a T_K leképezés lineáris: minden X_1, X_2 azonos alakú bemenetre és minden $\alpha \in \mathbb{R}$-re

$$T_K(X_1 + X_2) = T_K X_1 + T_K X_2, \qquad T_K(\alpha X_1) = \alpha T_K X_1.$$

Bizonyítás. A homogenitáshoz írjuk be a definíciót. Az αX_1 bemenetre a kimenet komponensenként:

$$T_K(\alpha X_1)_{n,o,i,j} = \sum_{c,u,v} (\alpha X_1)_{n,c,\,si-p+u,\,sj-p+v} K_{o,c,u,v} = \alpha \sum_{c,u,v} X_{1,n,c,\,si-p+u,\,sj-p+v} K_{o,c,u,v} = \alpha T_K(X_1)_{n,o,i,j}.$$

Az additivitás ugyanezzel az indexfelbontással:

$$T_K(X_1{+}X_2)_{n,o,i,j} = \sum_{c,u,v} (X_1{+}X_2)_{n,c,\,si-p+u,\,sj-p+v} K_{o,c,u,v} = \sum_{c,u,v} X_{1,n,c,\ldots} K_{o,c,u,v} + \sum_{c,u,v} X_{2,n,c,\ldots} K_{o,c,u,v},$$

mert az összeadás a rögzített indexhalmazon tagonkénti. A két szumma éppen $T_K X_1$ és $T_K X_2$ komponense.

A T_K így lineáris operátor, de a biasos $T_{K,b}$ **nem** lineáris. Ennek közvetlen oka, hogy a lineáris leképezés a nullvektort nullvektorba viszi, míg

$$T_{K,b}(0) = B \neq 0, \quad \text{ha } b \neq 0.$$

A $T_{K,b}$ affin: $T_{K,b} = \tau_B \circ T_K$, ahol $\tau_B(Y) = Y + B$ konstans eltolás (transzláció); tehát affin leképezés, egy lineáris leképezés és egy konstans eltolás kompozíciója, $T_{K,b}(X) = T_K(X) + B$. Az eltolás itt nem egy input-eltolás, hanem a kimeneti tér konstans B eleme; ezért a $T_{K,b}$-t nem lehet egyszerűen egy $T_{K'}$ alakú bias nélküli operátorral helyettesíteni, hacsak nem bővítjük az inputot egy rögzített 1 értékű csatornával. A gyakorlatban a bias éppen ezt a bővítést valósítja meg anélkül, hogy a bemeneti tenzort meg kellene növelni.

37.3.4 Adjungált azonosság és a transposed convolution

A véges dimenziós tereken a szokásos valós belső szorzat a tenzorok Frobenius-szorzata:

$$\langle A, B \rangle_F = \sum_{n,o,i,j} A_{n,o,i,j} B_{n,o,i,j}.$$

Ezzel az euklideszi struktúrával definiálhatjuk a T_K **adjungáltját** mint azt a T_K^* leképezést, amelyre minden megfelelő alakú X, Y esetén

$$\langle T_K X, Y \rangle_F = \langle X, T_K^* Y \rangle_F.$$

A [D11] szerint a `ConvTranspose2d` ennek az adjungált operátornak a diszkrét megvalósítása a megfelelő stride, padding, output_padding konvencióval; a dokumentáció az adjungáltat nem a matematikai konvolúció inverzeként írja le.

Tétel (adjungált azonosság). Rögzített K, s, p mellett az $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ és $Y \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$ tenzorokra fennáll

$$\sum_{n,o,i,j} Y_{n,o,i,j} \sum_{c,u,v} X_{n,c,\,si-p+u,\,sj-p+v} K_{o,c,u,v} = \sum_{n,c,a,b} X_{n,c,a,b} \sum_{\substack{o,i,j,u,v \\ a=si-p+u \\ b=sj-p+v}} Y_{n,o,i,j} K_{o,c,u,v},$$

ahol a jobb oldalon az összegzés azokra az (i, j, u, v) indexekre megy, amelyek kielégítik a feltüntetett két egyenletet; az a, b bemeneti indexek és az i, j kimeneti indexek a saját definíciós tartományukra szorítkoznak.

Bizonyítás. A bal oldal három egymásba ágyazott összeg: Y-szumma n, o, i, j szerint, belső szumma c, u, v szerint. Az összeadás kommutativitása miatt a szummákat tetszőlegesen átrendezhetjük, ha minden tag véges sokszor szerepel. Az indexeket az X-indexek szerint csoportosítjuk: minden rögzített (c, a, b) bemeneti pozícióhoz azok az (o, i, j, u, v) kombinációk tartoznak, amelyekre $a = si - p + u$ és $b = sj - p + v$. Ekkor a tag

$$Y_{n,o,i,j}\, X_{n,c,a,b}\, K_{o,c,u,v},$$

és az $X_{n,c,a,b}$ minden a, b-re közös tényezőként emelhető ki a belső szummából. Ez adja a fenti jobb oldalt.

A bizonyítás közvetlen következménye: a 37.2. Backward és ellenőrzési lefedettség alfejezetben levezetett ∇_X képlet éppen $T_K^*(Y)$, ha az upstream gradiens Y. A 37.2. Backward és ellenőrzési lefedettség overlap-összegzése tehát nem egy numerikus trükk, hanem az adjungált operátor definíciójának implementációja. A batchdimenzió N és a csatornadimenzió C független a térbeli átrendezéstől, ezért a levezetés tagonként azonos.

37.3.5 Nem-invertálhatósági példa

Az adjungáltságból nem következik invertálhatóság. A T_K általában nem injektív, tehát nincs bal oldali inverz, és a T_K^* sem általános inverz.

Állítás (dimenziócsökkenés). Ha $C_{in}HW > C_{out}H_{out}W_{out}$, akkor a $T_K : \mathbb{R}^{C_{in}HW} \to \mathbb{R}^{C_{out}H_{out}W_{out}}$ lineáris leképezés nem lehet injektív, így nem invertálható.

Bizonyítás. A rang–nullitás tétel szerint $\dim\ker T_K + \dim\operatorname{ran} T_K = \dim(\text{input}) = C_{in}HW$, és $\dim\operatorname{ran} T_K \le C_{out}H_{out}W_{out}$. Így $\dim\ker T_K \ge C_{in}HW - C_{out}H_{out}W_{out} > 0$, tehát létezik nemnulla X, amelyre $T_K X = 0$.

Az induló kérdés konfigurációja ennek közvetlen esete: 3×3-as inputon 3×3-as kernellel, stride 1 és padding 0 mellett $H_{out} = W_{out} = 1$, tehát a leképezés $\mathbb{R}^9 \to \mathbb{R}^1$; mivel $9 > 1$, a T_K nem injektív, ezért nincs balinverze. A tisztán dimenziós érvelés után adjunk **konkrét** nulltér-elemet. Legyen $N = C_{in} = C_{out} = 1$, $H = W = 3$, $k_h = k_w = 2$, stride 1, padding 0. A kimenet 2×2, a leképezés $\mathbb{R}^9 \to \mathbb{R}^4$. Válasszuk a kernelt

$$K = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

Ekkor $Y_{i,j} = X_{i,j}$ a bal felső 2×2-es blokkra. Az

$$X_1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad X_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

bemenetekre $T_K X_1 = T_K X_2 = 0$, noha $X_1 \neq X_2$. A T_K tehát több különböző inputot képez ugyanabba az outputba; a kép jobb alsó sarka „nem látszik” a kimeneten.

Ugyanez a példa azt is mutatja, hogy a $T_K^* : \mathbb{R}^4 \to \mathbb{R}^9$ nem adhat általános rekonstrukciót: a rekonstrukcióhoz szükséges elveszett $X_{2,2}$ komponens a kimenetből elvileg nem állítható elő,

mert a kimenetbeli $Y_{i,j}$ minden tagja csak a $0 \le i, j \le 1$ tartományból olvas. A későbbi `ConvTranspose2d` laborban ezért sosem várunk `allclose(x_hat, x)` egyezést.

37.3.6 Eltolási ekvivariancia és annak határfeltétele

Legyen τ_v az infinite-lattice $\mathbb{Z}^2$ eltolása $v = (v_h, v_w)$ vektorral. A véges képet a zero padding zónán túl zérusnak tekintve X-et kiterjesztjük $\mathbb{Z}^2$-re; ugyanez a kiterjesztés érvényes $T_K X$-re is. A kernel súlyai pozíciófüggetlenek, ezért stride 1 és zero padding mellett a konvolúció minden egész eltolással kommutál (eltolás-ekvivariáns):

$$T_K(\tau_v X) = \tau_v(T_K X), \qquad s = 1.$$

Bizonyítás (váz). Jelölje a bemeneti eltolásvektort $w = (w_h, w_w)$, és tegyük fel, hogy a bemenetet τ_{sw}-vel toljuk. A T_K definíciójában a bemeneti sorindex $si - p + u$; a i kimeneti pozícióban ekkor

$$\sum_{c,u,v} (\tau_{sw} X)_{n,c,\, si-p+u,\, \dots} K_{o,c,u,v} = \sum_{c,u,v} X_{n,c,\, s(i-w_h)-p+u,\, \dots} K_{o,c,u,v}$$

áll, ami pontosan a $T_K X$ kimenetének $i - w_h$ (és értelemszerűen $j - w_w$) pozíciója, azaz $T_K(\tau_{sw} X) = \tau_w T_K(X)$. A padding zóna a τ_{sw} alatt együtt tolódik, ezért az összegzés tartománya invariáns; az output oldalán ez a w-vel eltolt pozíció. $s = 1$ esetén $sw = w$, ami a fenti állítást adja.

A **határfeltétel** két ponton szűkíti ezt az állítást.

1. **Stride-részcsoport.** Ha $s > 1$, az ekvivariancia csak az $s\mathbb{Z}^2$ rács eltolásaira áll: $T_K(\tau_{su} X) = \tau_u T_K(X)$ minden $u \in \mathbb{Z}^2$-re, ahol a jobb oldalon τ_u a kimeneti rácson hat; egy általános v eltolásra az állítás nem igaz. Ennek oka, hogy az output csak minden s-edik rácsponton mintavételez.
2. **Nem zéró padding.** A reflect/replicate padding (tükrözéses, illetve szélmásoló kitöltés) szabálya a véges ablak határához kötött, ezért nem viselkedik transzláció-invariáns kiterjesztésként. Itt a τ_v-ekvivariancia már stride 1 mellett is meghiúsul a határ közelében.

Ellenpélda stride 2-re. Legyen $H = W = 4$, $k = 2$, stride 2, pad 0, $K = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, tehát $Y_{i,j} = X_{2i,2j}$, output 2×2. Legyen az inputban $X_{0,0} = 1$, a többi 0. Ekkor $T_K X$ a $(0;\ 0)$ outputban 1. Toljuk el a **bemenetet** $\tau_{(0;\ 1)}$-gyel: a delta a $(0;\ 1)$ pozícióba kerül, amely nincs a $2\mathbb{Z}^2$ mintavételi rácson, ezért $T_K(\tau_{(0;\ 1)} X) = 0$. Másrészt az output $\tau_{(0;\ 1)}$-eltolása a $\mathbb{Z}^2$-re kiterjesztett $T_K X$-en az 1-et a $(0;\ 1)$ pozícióba viszi, tehát $\tau_{(0;\ 1)} T_K X \neq 0$. A két oldal különbözik.

Határ-ellenpélda replicate paddingre. Legyen $N = C_{in} = C_{out} = 1$, $H = W = 3$, kernel 1×1-es identitás, stride 1, replicate padding 1. Input: a $(0;\ 1)$ pixel 1, a többi 0. A replicate padding a szélső pixeleket másolja a padding zónába (a paddingelt koordináták $P[r, c] = X[\mathrm{clip}(r-1, 0, 2), \mathrm{clip}(c-1, 0, 2)]$), ezért a $(0;\ 1)$ pixel az 5×5-ös kimeneten két helyen jelenik meg: $(0;\ 2)$ és $(1;\ 2)$ (a felső sor másolata). Most toljuk az **inputot** jobbra 1-gyel, a 3×3-as ablakon belül: a delta a $(0;\ 2)$ pozícióba kerül. A replicate paddinggel számolt kimenet ekkor négy helyen nem nulla: $(0;\ 3)$, $(0;\ 4)$, $(1;\ 3)$ és $(1;\ 4)$, mert a jobb szélső oszlop másolata és a sarok is megjelenik. A korábbi kimenet jobbra tolva viszont csak két nem nulla pozíciót adna: $(0;\ 3)$ és $(1;\ 3)$. A két oldal különbözik; a különbség a $(0;\ 4)$ és $(1;\ 4)$ kimeneti pozíción van, amely mindkét összehasonlításban a $H_{out} = W_{out} = 5$ ablakon belül van, és a bemeneti delta is végig a 3×3-as ablakon belül marad. Zero paddinggel ugyanerre a bemenetre a kimenet a $(1;\ 2)$,

illetve eltolva a $(1;\ 3)$ pozíción nem nulla, tehát ott a két oldal egyezik; ez a kontroll izolálja a padding típusát, mert ugyanaz a kísérlet zero paddinggel nem mutat eltérést. A replicate padding tehát a határnál megtöri az ekvivarianciát, mert a paddingérték az ablakhatártól függ, nem pedig a shiftelt infinite-rács kiterjesztésétől. Ez a példa egyben a „boundary counterexample” szerepét tölti be: a határkezelés választása nem semleges operátorazonosság, hanem további előfeltétel.

Határ-ellenpélda reflect paddingre. Ugyanezen a 3×3-as inputon, 1×1-es identitás kernellel és reflect paddinggel a $(0;\ 1)$ delta képe három pozíció: $(1;\ 0)$, $(1;\ 2)$ és $(1;\ 4)$, mert a bal szélső oszlop tükrözött másolata is megjelenik. Jobbra tolva a delta a $(0;\ 2)$-be kerül, képe ekkor egyetlen pozíció: $(1;\ 3)$. A korábbi kimenet jobbra tolva $(1;\ 1)$ és $(1;\ 3)$ lenne (az $(1;\ 5)$ kilóg az 5×5-ös ablakból), tehát a két oldal a $(1;\ 1)$ pozíción különbözik, amely a kimeneti ablakon belül van, így az eltérés nem a kimeneti ablak csonkításából fakad. Zero paddinggel mindkét oldal a $(1;\ 2)$, illetve $(1;\ 3)$ egyetlen pozíciót adja, azaz egyezik.

37.3.7 Labor: matematikai jegyzőkönyv a referenciakódhoz

A laborcsomag `conv_reference.py` moduljának `conv2d` függvényét használjuk (NumPy, NCHW bemenet, OIHW súly, `groups=1`, szimmetrikus padding, stride és dilation; a modult a 7. Izolált környezet, notebook és verzió egység mutatja be, és a `lab/` munkakönyvtárban található). Az alábbi szkript a `lab/` munkakönyvtárból futtatható. A kód a linearitást két oldal összehasonlításával és az adjungált azonosságot a belső szorzat segítségével ellenőrzi; a nem-invertálhatóságot és az ekvivariancia-korlátot a fenti kézi példák rögzítik (ezeket a szkript nem számolja újra).

```
# lab/n147_operator_report.py
import numpy as np

# A laborcsomag NumPy-referenciája; az aláírás a 37.2. leckében rögzített specifikációt követi.
from conv_reference import conv2d as conv_ref

rng = np.random.default_rng(20261020)
tol = 1e-12

# 1) Linearitás bias nélkül
X1 = rng.normal(size=(1, 2, 5, 5)).astype(np.float64)
X2 = rng.normal(size=(1, 2, 5, 5)).astype(np.float64)
K = rng.normal(size=(3, 2, 3, 3)).astype(np.float64)
left = conv_ref(X1 + X2, K, bias=None, padding=1, stride=1)
right = conv_ref(X1, K, bias=None, padding=1, stride=1) + conv_ref(X2, K, bias=None, padding=1,
↪  stride=1)
print("linearity max|left-right|:", np.max(np.abs(left - right)))
assert np.allclose(left, right, atol=tol, rtol=tol)

# 2) Adjungált azonosság: <T_K X, Y> == <X, T_K^* Y>
pad, k = 1, K.shape[-1]
Y = rng.normal(size=(1, 3, 5, 5)).astype(np.float64)  # stride=1, pad=1, k=3 => H_out=5
TX = conv_ref(X1, K, bias=None, padding=pad, stride=1)
TstarY = conv_ref(Y, np.transpose(K, (1, 0, 2, 3))[:, :, ::-1, ::-1],
                  bias=None, padding=k - 1 - pad, stride=1)  # az adjungált paddingje k-1-p
print("adjoint lhs:", np.sum(TX * Y))
print("adjoint rhs:", np.sum(X1 * TstarY))
assert np.allclose(np.sum(TX * Y), np.sum(X1 * TstarY), atol=1e-10, rtol=1e-10)
```

A fenti `conv_ref` hívás a laborcsomag `conv2d` függvényét használja; a `bias=None` argumentum azt jelenti, hogy a biasvektor elhagyható. A transposed operátor kernelkonvenciója: az adjungáltban a súlyt a kimeneti–bemeneti csatornák felcserélésével és a két térbeli tengely megfordításával kapjuk, mert a cross-correlation kernelje az adjungált képletben pontosan így jelenik meg.

Az adjungált hívás `padding` argumentuma nem a forward `pad` értéke, hanem $k-1-\text{pad}$: a transzponált operátor a forward paddingjét ellentétes irányban „vonja vissza”.

A jegyzőkönyv formátuma a 36.3. Szakirodalmi reprodukció és attribúció. mintát követi:

Állítás	Típus	Ellenőrzés	Eredmény
T_K lineáris	tétel, fent bizonyítva	szkript: linearitás, seed 20261020, float64	mért max eltérés $2{,}22 \cdot 10^{-15}$
T_K^* adjungált	tétel, fent bizonyítva	szkript: belső szorzat-azonosság $k-1-p$ paddinggel	mért: lhs $=-33{,}38053719874603$, rhs $=-33{,}38053719874603$, allclose teljesül
T_K nem invertálható	ellenpélda, kézi számítás	$3\times 3 \to 2\times 2$ nulltér	VÁRT ÉRTÉK (levezetésből, nem mért): $T_K X_1 = T_K X_2 = 0$, noha $X_1 \neq X_2$
stride=2 ekvivariancia-korlát	ellenpélda, kézi számítás	4×4, $k=2$, delta input	VÁRT ÉRTÉK (levezetésből, nem mért): $T_K \tau_{(0;\ 1)} X = 0$, $\tau_{(0;\ 1)} T_K X$ a $(0;\ 1)$ kimeneten 1

A lineáris és adjungált sor mért futás eredménye (Python 3.12.3, NumPy 2.5.4, CPU, 10 szál, seed 20261020, a 37.2. alfejezet forwardjának `conv2d_forward_np` NumPy-átiratával); a nem invertálhatóság és a stride-korlát kézi levezetés, ezért ezeknél a jelölés **VÁRT ÉRTÉK (levezetésből, nem mért)**. A 37.2. Backward és ellenőrzési lefedettség referenciakóddal való futtatáskor ugyanezeket a tűréseket kell reprodukálni. A parancs a `lab` könyvtárból:

```
python n147_operator_report.py
```

A szkript célja nem a sebesség mérése, hanem a matematikai jegyzőkönyv négy sorának gépi kontrollja.

37.3.8 Feladatok

F1 (fogalmi, nehézség 1). Pontosan idézze fel a lineáris leképezés két axiómáját, és mutassa meg egy konkrét, $1\times 1\times 2\times 2$ bemenetű, $k=2$, stride 1, padding 0 konvolúción, miért sérti meg a két axiómát a $b=2$ bias. Beadandó: az axiómák nevei, két olyan konkrét input, amelyre a homogenitás vagy additivitás ellentmondásba kerül, és a kiszámolt outputok.

F2 (levezetési, nehézség 2). Vezesse le a T_K^* teljes mátrixát a $C_{in}=C_{out}=1$, $H=3$, $W=4$, $k=2$, stride 1, padding 0, $K=\begin{pmatrix}1 & 2\\ 3 & 4\end{pmatrix}$ esetre, és ellenőrizze a $\langle T_K X, Y\rangle = \langle X, T_K^* Y\rangle$ azonosságot egy olyan X-szel, amelynek minden eleme 1, és Y-nal, amelynek minden eleme 1. Beadandó: a T^*Y konkrét 3×4-es mátrixa, a két belső szorzat kiszámítása, és annak indoklása, miért térhetnek el az egyes komponensek, miközben az összeg egyezik.

F3 (implementációs, nehézség 1). Egészítse ki a 37.2. Backward és ellenőrzési lefedettség referenciakódját egy `test_linear_adjoint` függvénnyel, amely a fenti laborban megadott seed

mellett ellenőrzi a linearitást és az adjungált azonosságot, majd három különböző `padding` értékre is futtatja. Beadandó: a függvény forrása, a futtatási parancs, a három assert feltétele, és a tolerancia indoklása.

F4 (hibakeresési, nehézség 2). Az alábbi hibás érvelés szerint egy réteg rekonstruálható az adjungálttal:

```
x_hat = conv_transpose(conv(x, K), K_flipped)  # remélt x
assert np.allclose(x_hat, x)                   # gyakran elhasal
```

Lokalizálja a hibát az operátor tulajdonságaira hivatkozva, mutassa meg a $3 \times 3 \to 2 \times 2$-es példán a rekonstrukció kudarcát, és adja meg a javítás módját: mit lehet és mit nem lehet ebből a műveletpárból garantálni. Beadandó: az ok, a következmény, a javított assert, és az ellenőrzés visszaigazolása.

F5 (kísérlettervezési, nehézség 3). Tervezzen megismételhető vizsgálatot, amely a reflect padding equivarianciáját a véges kép határa mentén összehasonlítja a zero paddinggel. Használja a 36.3. Szakirodalmi reprodukció és attribúció. protokollsablonját. Beadandó: kérdés, hipotézis, megfigyelési egység, baseline, változó tényező, metrika (például max abszolút eltérés $T_K\tau_v - \tau_v T_K$), a használt véges shiftlista, a számítási keret, és az érvényességi korlát leírása. Előre megadott eredmény nem elvárt; a kézzel kiszámolt replicate ellenpéldát referenciaesetként kell szerepeltetni.

37.3.9 Megoldások és indoklás

F1. A lineáris leképezés axiómái: $L(x_1 + x_2) = L(x_1) + L(x_2)$ (additivitás) és $L(\alpha x) = \alpha L(x)$ (homogenitás). A $b = 2$ biasos rétegben legyen $K = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, $X_1 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, $X_2 = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$, bemenet 2×2, kimenet 1×1. Ekkor $T_{K,b}(X_1) = 1 + 2 = 3$, $T_{K,b}(X_2) = 0 + 2 = 2$, ezért $T(X_1) + T(X_2) = 5$. Ugyanakkor $T_{K,b}(X_1 + X_2) = 1 + 2 = 3$, tehát $3 \neq 5$, az additivitás sérül. A homogenitás is sérül X_1-re és $\alpha = 2$-re: $T_{K,b}(2X_1) = 2 \cdot 1 + 2 = 4$, míg $2\,T_{K,b}(X_1) = 2 \cdot 3 = 6$, tehát $4 \neq 6$. (Természetesen $\alpha = 0$-ra is sérül: $T_{K,b}(0) = 2 \neq 0$.) A biasos réteg tehát affin, nem lineáris.

F2. A Y minden elemét 1-nek választva $T_K X$ minden kimeneti eleme $1+2+3+4 = 10$. A belső szorzat $\langle T_K X, Y \rangle = 6 \cdot 10 = 60$. A $T_K^* Y$ komponenseit az adjungált indexfeltétel adja: minden (a, b) bemeneti pozícióra azon (i, j, u, v) kombinációk összege, amelyekre $a = i + u$, $b = j + v$. A kiszámított mátrix:

$$T_K^* Y = \begin{pmatrix} 1 & 3 & 3 & 2 \\ 4 & 10 & 10 & 6 \\ 3 & 7 & 7 & 4 \end{pmatrix},$$

és az elemek összege $1+3+3+2+4+10+10+6+3+7+7+4 = 60$, egyezik. A komponensek eltérése abból fakad, hogy a határpixelek kevesebb output-patchben szerepelnek; a $\langle X, T^* Y \rangle$ azonban a teljes összeget őrzi meg, mert az indexcsere minden tagot pontosan egyszer számol.

F3. Egy lehetséges teljes függvény:

```
def test_linear_adjoint(rng=20261020):
    rng = np.random.default_rng(rng)
    X1 = rng.normal(size=(1, 2, 7, 7)).astype(np.float64)
    X2 = rng.normal(size=(1, 2, 7, 7)).astype(np.float64)
    K = rng.normal(size=(3, 2, 3, 3)).astype(np.float64)
    k = K.shape[-1]
```

```
    for pad in (0, 1, 2):
        TX1 = conv_ref(X1, K, bias=None, padding=pad, stride=1)
        TX2 = conv_ref(X2, K, bias=None, padding=pad, stride=1)
        assert np.allclose(TX1 + TX2, conv_ref(X1 + X2, K, bias=None, padding=pad, stride=1),
        ↪  atol=1e-12, rtol=1e-12)
        Y = rng.normal(size=TX1.shape).astype(np.float64)
        TstarY = conv_ref(Y, np.transpose(K, (1, 0, 2, 3))[:, :, ::-1, ::-1], bias=None,
↪  padding=k - 1 - pad, stride=1)
        assert np.allclose(np.sum(TX1 * Y), np.sum(X1 * TstarY), atol=1e-10, rtol=1e-10)
    return "OK"
```

A toleranciát a float64 összegzés kerekítési szintje indokolja: $u \approx 2{,}2 \cdot 10^{-16}$. A linearitási teszt elemenként legfeljebb $C_{in}k_hk_w = 18$ szorzatot összegez, ezért a kerekítési hiba durva felső becslése $\gamma_{17} \sum |t_i|$ szerint $5{,}4 \cdot 10^{-14}$; az adjungált belső szorzat legnagyobb esetben $3 \cdot 9 \cdot 9 = 243$ tagot ad, ahol ugyanez a becslés legfeljebb $1{,}6 \cdot 10^{-11}$. A megadott seeddel, a laborcsomag `conv_reference.py`-jával futtatva a mért legnagyobb eltérés a linearitásnál $6{,}2 \cdot 10^{-15}$, az adjungáltnál $1{,}8 \cdot 10^{-14}$ volt. Ezért a használt `atol=rtol=1e-12`, illetve `atol=rtol=1e-10` korlát a mért hiba felett marad, és a durva felső becslést is figyelembe véve biztonságos, ugyanakkor elég szigorú a hibás indexelés vagy a rossz padding kimutatásához. Az adjungált hívás paddingje $k - 1 - p$: 3×3 kernel mellett $p = 0$-ra 2, $p = 1$-re 1, $p = 2$-re 0. Csak a $p = 1$ esetben esik egybe a két paddingérték, ezért a `padding=pad` használata rejtett feltevés, és $p = 0$ vagy $p = 2$ mellett alak-eltéréssel vagy hibás belső szorzattal jár. A három paddingérték a null-, a kis- és a nagy-padding szélső eseteket fedi.

F4. A hiba oka: az adjungált nem általános inverz. A $T_K : \mathbb{R}^9 \to \mathbb{R}^4$ leképezés nem injektív, ezért bármely lineáris U leképezésre UT_K sem lehet identitás $\mathbb{R}^9$-en; a $T_K^*T_K$ önadjungált, de nem invertálható. A 3×3-as, 2×2-es kernel példán a jobb alsó $X_{2,2}$ pixel semelyik outputra nem hat, így a transposed út sem állíthatja vissza. A `assert np.allclose(x_hat, x)` ezért szükségszerűen megbukik. A javítás: az adjungáltat kizárólag a belső szorzat azonossággal ellenőrizzük, a rekonstrukciós állítást pedig csak akkor fogalmazzuk meg, ha teljes rangú, négyzetes operátorral dolgozunk; a jelen példában ez nem áll fenn. A javított ellenőrzés a labor 2. pontjának adjungált azonossága, nem rekonstrukció: `assert np.allclose(np.sum(conv(x, K) * y), np.sum(x * conv_transpose(y, K_flipped)), atol=1e-10, rtol=1e-10)`, ahol `y` a kimeneti tér tetszőleges tenzora. Ez a $3 \times 3 \to 2 \times 2$-es példán is teljesül, noha x nem állítható vissza.

F5. A protokoll a 36.3. Szakirodalmi reprodukció és attribúció. sablonja szerint: kérdés, hogy a reflect padding a határ mentén megtöri-e a shift-ekvivarianciát; hipotézis, hogy legalább egy határmenti shiftre eltérés van; megfigyelési egység egy 7×7-es, rögzített seeddel generált véletlen kép; baseline a zero padding ugyanazon kernelre és shiftre; változó tényező a padding módja; metrika $e(v) = \max_{n,o,i,j} |T_K(\tau_v X) - \tau_v T_K(X)|$, ahol τ_v a $v \in \{-1;\ 0;\ 1\}^2$ rács eltolása a megfelelő kiterjesztéssel; a számítási keret a laborcsomag `conv_reference.py` NumPy-referenciája CPU-n, float64 (a reflect/replicate kitöltést a bemenet előzetes `np.pad` hívásával állítjuk elő, majd `padding=0` mellett futtatjuk). A megfelelő kiterjesztés a zero padding kontrolljánál a végtelen rácsra való zérus kiterjesztés: a bemenetet és a kimenetet is $M \geq |v|_{\max} + k - 1$ margóval kibővített rácson számoljuk ki, a τ_v eltolást ezen a nagyobb rácson végezzük, majd a közös névleges kimeneti ablakra vágunk. E margin nélkül a véges ablak széle önmagában is $e(v) > 0$-t adna a zero paddingre, és a kontroll hamisan pozitív lenne. A reflect és replicate padding esetén a bemenetet a rögzített 7×7-es ablakon belül toljuk el, a kimenet a paddingelt ablakra korlátozódik, és a τ_v az ablakon kívül zérust feltételez; a különbség így a padding-szabály valódi hatása. A teszt akkor érvényes, ha a zero padding minden v esetén $e(v) = 0$ a numerikus tűrésen belül, a reflect padding viszont legalább egy v-nél pozitív $e(v)$, és a kézzel kiszámolt replicate/reflect ellenpélda minimálisan reprodukálódik. A kézzel kiszámolt referenciaeset (3×3-as kép, 1×1-es identitáskernel, $v = (0;\ 1)$, a delta input a 3×3-as ablakon belül marad): zero paddinggel

$e(v) = 0$; reflect paddinggel $e(v) = 1$ a $(1;\ 1)$ kimeneti pozíción; replicate paddinggel $e(v) = 1$ a $(0;\ 4)$ és $(1;\ 4)$ pozíciókon. A döntési szabály tehát számszerűen is megalapozott: a zero padding $e(v) = 0$-t ad, a reflect és a replicate nem. A korlát: az észlelt hiba a padding-implementáció és a rögzített ablak határán mérhető jelenség; nem általános bizonyítás tetszőleges képre, de egy ellenpélda az equivarianciát mint univerzális állítást megcáfolja.

37.3.10 Tipikus tévedés és ellenpélda

Gyakori hiba, hogy a matematikai operátor és a választott gyors implementáció futási költségét azonosítják. Például egy 3×3-as kernel hatása felírható naiv összegzéssel, im2col bontással, Winograd-gyorsítással vagy depthwise separable közelítéssel; ez utóbbi gyorsabb lehet, de más operátort ad. Az az állítás, hogy „a gyorsabb implementáció nem lineáris, mert hardveresen szaturál", az operátortulajdonságra nem vonatkozik, csak az implementáció numerikus tartományára. Megfordítva: abból, hogy két implementáció azonos matematikai T_K-t számol, nem következik azonos futási költség vagy azonos numerikus hiba. Az operátor linearitását, adjungáltságát és equivarianciáját a definiáló összegzésből bizonyítjuk; a futási idő mérése külön, a 24.2. CPU/GPU profiling és helyes időmérés alfejezetben tárgyalt protokollt igénylő empirikus állítás.

37.3.11 Önellenőrzés és korábbi fogalmak áttekintése

Ellenőrizze a levezetést négy ponton: a bias nélküli T_K nullvektorra nullát ad; az adjungált azonosság jobb oldalán az összegzés valóban az X-indexek szerint csoportosít; a stride=2 példában a mintavételi rács $2\mathbb{Z}^2$; a replicate és a reflect példában a bemeneti delta a 3×3-as ablakon belül marad, és a kimutatott eltérés olyan kimeneti pozíción van, amely a kimeneti ablakon belül van, ezért az összevetés nem a kimeneti ablak csonkításán, hanem a padding-típuson bukik meg. A 37.2. Backward és ellenőrzési lefedettség alfejezetből idézzük vissza, hogy az inputgradiens overlap-összegzése pontosan az itt bizonyított T_K^*; a 36.3. Szakirodalmi reprodukció és attribúció alfejezetből a jegyzőkönyv formátuma; a 33.3. torch.export és ONNX alfejezetből, hogy az export ekvivalenciájának ellenőrzése ugyanilyen feltételeket és toleranciát igényel; a 25.3. Kis adat és feature-alapú baseline alfejezetből pedig azt, hogy a lineáris pixelmodell is operátor, és reprezentációértékelésnél a leképezés tulajdonságait a tanulási kísérlettől külön kell kezelni.

37.3.12 Rövid összefoglaló

A bias nélküli cross-correlation T_K lineáris operátor, mert a definiáló összegzés tagonként homogenitást és additivitást ad; a biasos réteg affin, de nem lineáris. A Frobenius-belső szorzatra nézve az adjungált indexcserével bizonyítható, és a `ConvTranspose2d`-féle operátort adja; ez nem általános inverz. A nem-invertálhatóságot dimenziócsökkenéssel és konkrét nulltér-elemmel mutattuk. A shift-ekvivariancia stride 1 és zero padding mellett teljesül az infinite-rácson, de stride $s > 1$-nél csak az $s\mathbb{Z}^2$-részcsoportra, replicate paddingnél pedig a határon sem. A referenciakódos jegyzőkönyv minden állítást analitikus és numerikus kettős kontroll alá helyez.

37.3.13 Következő lecke és források

A 37.4. Rétegből kis tanulómodell egység ezt az operátort kis tanulómodellé építi: a réteg paramétereit gradienslépéssel tanítjuk, ahol a backward éppen a T_K^* művelet, a linearitás pedig a gradienslinearitást garantálja. A 37.4. Rétegből kis tanulómodell laborjában az itteni jegyzőkönyvteszteket a tanítás előtti automatikus ellenőrzésként futtatjuk.

Felhasznált források:

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-20.
- [D11] PyTorch contributors (2026). *ConvTranspose2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.ConvTranspose2d.html. Hozzáférés: 2026-10-20.
- [S01] Stanford CS231n (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-20.

37.4 Rétegből kis tanulómodell

37.4.1 Cél, előismeret és induló kérdés

A projekt eddigi egységei külön-külön igazolták a saját konvolúciós réteget: a 37.1. Operátor-specifikáció és kézi referencia a forward és a backward referenciakódját, a 37.2. Backward és ellenőrzési lefedettség a független teszteket, a 37.3. Linearitás, adjungált és határfeltétel pedig az operátor lineáris, adjungált és határfeltételi tulajdonságait. Ez az egység a záró kérdést vizsgálja: használható-e a réteg nemcsak tenzorműveletként, hanem egy tényleges, gradiensalapú tanulómodell építőköveként?

Előismeret: a 37.3. Linearitás, adjungált és határfeltétel operátorállításai; a 10.4. Tanítási lépés és tanuló MLP tanítási lépése; a 11.4. Egy teljes PyTorch tanítási iteráció PyTorch-modul és a 12.3. Sanity check és kis mintára túllillesztés kis mintára túllillesztés mint sanity check; a 36.4. IX. ellenőrző pont: lektorálható modellállítás lektorálható csomag hat komponense. A 33.4. Üzemeltetés, drift és késő címke alfejezetből (az eloszlásbeli eltolódás, azaz a drift kezeléséről) annyit használunk, hogy a teljesítményt csak a rendelkezésre álló mintához és feladathoz kötjük, a 25.4. Nyílt halmaz és ismeretlen input alfejezetből pedig azt, hogy a betanított modell csak a zárt feladat tartományán értelmezhető.

Induló kérdés. Ha egy réteg forward és backward függvénye külön ellenőrzött, akkor automatikusan helyesen tanul egy modell belőle? **Becslés:** nem. A rétegszintű ellenőrzés ugyan szükséges, de a tanítás közben új hibák léphetnek fel: a paraméterregisztráció, az optimalizálóba kerülés, a loss csatlakoztatása és a több lépéses gradiensáramlás együttműködése külön is elromolhat. Ezt az egységet arra szánjuk, hogy ezt a rétegszintű ellenőrzés és a modellszintű tanulás közötti hidat ellenőrzött módon megépítsük.

37.4.2 A réteg integrációs szerződése

A saját rétegünk a projekt korábbi specifikációját követi:

- bemenet: $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, float32, NCHW;
- súly: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, OIHW;
- bias: $b \in \mathbb{R}^{C_{out}}$;
- kimenet: $Y \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}$, ahol

$$H_{out} = \left\lfloor \frac{H + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1,$$

és W_{out} ugyanígy, W szélességgel, p szimmetrikus zero padding, s stride, d dilation. A réteg cross-correlationt számol, kernelmegfordítás nélkül.

A projektben eddig a réteget tiszta függvényként kezeltük; a 37.2. Backward és ellenőrzési lefedettség alfejezet `check_backward.py` fájlja (a `lab/` könyvtárban) két ilyen függvényt ad:

```
def conv2d_forward(X, K, b, s, p, d):      # -> Y
    ...
def conv2d_backward(X, K, dY, s, p, d):    # -> gX, gK, gb
    ...
```

Ezek `torch.Tensor`-on dolgozó, kézzel írt ciklusok: önmagukban **nem épülnek be az autograd-gráfba**, mert a PyTorch nem látja, hogy a kimenet hogyan függ a bemenettől. Az `nn.Parameter` regisztráció ezt nem pótolja: a regisztráció a modul paramétereinek nyilvántartását jelenti, ezért a `parameters()` felsorolás, a `state_dict()`, a `.to(device)` áttelepítés és a `train()`/`eval()` állapotváltás is látja a paramétert, és az optimalizáló is a `parameters()` listából kapja meg. A gradiensviszony viszont attól függ, hogy a forward differenciálható tenzorműveletekből épül-e fel, vagy egy `torch.autograd.Function` írja le a backwardot. A hidat egy `torch.autograd.Function` adja: a `forward` a saját `conv2d_forward`-ot hívja, a `backward` pedig a saját `conv2d_backward`-ot, így a PyTorch `loss.backward()` hívása a **saját** gradienskódunkat futtatja. Ezt a hidat a modulba csomagoljuk, és a `weight`-et, `bias`-t `nn.Parameter`-ként regisztráljuk. Ebből következik az integrációs szerződés:

Definíció (integrációs szerződés). A saját réteg akkor *integrált* egy PyTorch-modellben, ha

1. a `weight` és `bias` tenzor `nn.Parameter`, ezért szerepel a `model.parameters()` felsorolásban, a `state_dict()`-ben és a `.to(device)` áttelepítésben; az optimalizálót a `model.parameters()` listával példányosítjuk, így a paraméterek bekerülnek a létrehozott optimalizáló `param_groups` listájába. A sorrend számít: egy **már létrehozott** optimalizálóba az új paraméter nem kerül be automatikusan, ezért a modult az optimalizáló előtt kell felépíteni; a `grad` létrejöttéhez pedig a veszteséghez kapcsolt differenciálható gráf kell;
2. a `forward` vagy kizárólag a PyTorch által differenciálható tenzorműveleteket használ, vagy egy `torch.autograd.Function`-t hív, amelynek saját `backward`-ja a gradienseket az autograd-gráfba adja vissza; a kettő közül itt a második a választott út;
3. a modell kiértékelése után a `loss.backward()` hívás minden paraméternél `requires_grad=True` mellett beállítja a `grad` mezőt, azaz az nem `None`.

Itt tudatosan a PyTorch `nn.Module` mechanizmusát használjuk, mert a projekt numerikus főútja a 11.1. Tensor és autograd-életciklus alfejezetben lefektetett autograd-életciklusra épül.

37.4.3 Veszteség és a tanítási feladat

A tanulási tesztet nem a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab 24×24-es képein kezdjük, mert egy ilyen kis kontroll két okból is félrevezető lehet: a háromosztályos orientációs feladatban (0 = függőleges, 1 = vízszintes, 2 = átlós; lásd A. A GlyphLab adatkészlet) a konvolúciós modell architektúrája és az augmentáció együtt hat, és a réteg hibája nem választható el a modellválasztástól. Ehelyett egy szintetikus, determinisztikus feladatot építünk, amelyben a tanulandó szűrőnek explicit célja van.

A tanulási feladat. A modell egyetlen $1 \to 1$ konvolúciós rétegből áll, $k_h = k_w = 3$, stride 1, padding 1, dilation 1. A bemenet $N \times 1 \times 8 \times 8$ float32 tensor. A célkép az input mozgóátlaga egy 3×3-as átlagoló szűrővel, azaz

$$Y_{n,0,i,j} = \frac{1}{9} \sum_{u=-1}^{1} \sum_{v=-1}^{1} X_{n,0,i+u,j+v}$$

zero padding mellett. Ez azért jó teszt, mert:

- a célpontosság *ismert*: a 3×3-as súly minden eleme 1/9, bias 0;
- a modell a paramétereiben (K, b) affin, ezért a négyzetes veszteség a paraméterek konvex kvadratikus függvénye, így a gradiensmenti konvergencia numerikusan ellenőrizhető;
- a rétegnek semmilyen címkefüggetlen nehézséget nem kell megoldania, így bármilyen hiba a réteg integrációjára utal.

A veszteség az MSE átlagos formája:

$$L = \frac{1}{N H_{out} W_{out}} \sum_{n,i,j} \left(Y_{n,0,i,j} - \hat{Y}_{n,0,i,j} \right)^2 ,$$

ahol $\hat{Y}$ a modell kimenete. A redukció fontos: a PyTorch `MSELoss` alapértelmezésben a teljes batch elemszámával oszt, ami $N \cdot C_{out} \cdot H_{out} \cdot W_{out}$; a vizsgált feladatban $C_{out} = 1$, ezért ez a fenti $N H_{out} W_{out}$ nevezővel azonos. Ezt expliciten kiírjuk, mert a [D07]-szerződés szerint a loss skáláját minden levezetésnek külön meg kell adnia.

37.4.4 A modell és a keretrendszer egyeztetése azonos súlyokon

A tanítás előtt a levezetési követelményt teljesítjük: a saját réteget és a PyTorch `nn.Conv2d`-t ugyanazzal a súllyal és biasszal, ugyanazon a bemeneten futtatjuk le, és a kimeneteket összehasonlítjuk.

Egyeztetési állítás. Legyen $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, $K \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$, $b \in \mathbb{R}^{C_{out}}$ rögzített, $s \geq 1$, $p \geq 0$, $d \geq 1$. A projekt saját `conv2d_forward` függvénye és a PyTorch `F.conv2d` hívás ugyanazt a Y kimenetet adja, ha a PyTorch-hívás `stride=s`, `padding=p`, `dilation=d`, `groups=1`, és mindkét oldal float32 számítást végez. Az állítás típusa: numerikus egyenlőségre vonatkozó ellenőrzött teszt, nem analitikus tétel, mert a lebegőpontos műveletek sorrendje implementációfüggő lehet.

A teszt pontos numerikus feltételeit a 37.2. Backward és ellenőrzési lefedettség alfejezetben rögzítettük; itt a modellen belüli elrendezésre koncentrálunk. A PyTorch hívást nem a megszokott `nn.Conv2d` modulon, hanem közvetlenül az `F.conv2d` funkcionális API-n keresztül végezzük, hogy elkerüljük a modulparaméter-regisztráció esetleges különbségét. A saját réteget a `Conv2dFunction` autograd-adapteren keresztül a `Conv2dReference` osztályba csomagoljuk:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

from check_backward import conv2d_forward, conv2d_backward   # 37.2., lab/


class Conv2dFunction(torch.autograd.Function):
    """Híd: a saját forward/backward kód az autograd-gráfban."""
    @staticmethod
```

```
    def forward(ctx, x, weight, bias, stride, padding, dilation):
        ctx.save_for_backward(x, weight)
        ctx.conf = (stride, padding, dilation)
        ctx.has_bias = bias is not None
        b = bias if bias is not None else x.new_zeros(weight.shape[0])
        return conv2d_forward(x, weight, b, stride, padding, dilation)

    @staticmethod
    def backward(ctx, dy):
        x, weight = ctx.saved_tensors
        s, p, d = ctx.conf
        gx, gk, gb = conv2d_backward(x, weight, dy.contiguous(), s, p, d)
        return gx, gk, (gb if ctx.has_bias else None), None, None, None

class Conv2dReference(nn.Module):
    """A projekt saját, NCHW/OIHW, groups=1 konvolúciós referenciarétege."""
    def __init__(self, in_channels, out_channels, kernel_size,
                 stride=1, padding=0, dilation=1, bias=True):
        super().__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = (kernel_size, kernel_size) \
            if isinstance(kernel_size, int) else tuple(kernel_size)
        self.stride = stride
        self.padding = padding
        self.dilation = dilation
        k_h, k_w = self.kernel_size
        self.weight = nn.Parameter(
            torch.empty(out_channels, in_channels, k_h, k_w,
                        dtype=torch.float32))
        if bias:
            self.bias = nn.Parameter(torch.empty(out_channels,
                                                 dtype=torch.float32))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()

    def reset_parameters(self):
        # A kontrollhoz mindkét oldalon ugyanazt a seedet használjuk; az
        # inicializálás a PyTorch nn.Conv2d alapértelmezett Kaiming-uniform
        # szabályát követi.
        nn.init.kaiming_uniform_(self.weight, a=5**0.5)
        if self.bias is not None:
            bound = 1.0 / (self.in_channels *
                           self.kernel_size[0] * self.kernel_size[1])**0.5
            nn.init.uniform_(self.bias, -bound, bound)

    def forward(self, x):
        return Conv2dFunction.apply(
            x, self.weight, self.bias,
            self.stride, self.padding, self.dilation)
```

A `conv2d_forward` és a `conv2d_backward` a 37.2. Backward és ellenőrzési lefedettség alfejezet `check_backward.py` fájljában van; a szkript `if __name__ == "__main__"` őrt tartalmaz, ezért az importja nem indítja el az ellenőrzést. Itt nem másoljuk be a teljes implementációt; a modell definíciójában a híd a lényeg. Ha valaki a `forward`-ban `F.conv2d`-t hívna a `Conv2dFunction` helyett, a modell tanulna, de akkor a saját `backward` **nem** futna, és a réteg integrációja nem lenne igazolva.

Az egyeztetési teszt:

```
torch.manual_seed(20261020)
N, C_in, H, W = 2, 1, 8, 8
```

```
C_out, k = 1, 3
x = torch.rand(N, C_in, H, W, dtype=torch.float32)

ref = Conv2dReference(C_in, C_out, k, stride=1, padding=1)
torch.nn.init.constant_(ref.weight, 1.0 / 9.0)
torch.nn.init.constant_(ref.bias, 0.0)

weight_pt = ref.weight.detach().clone()
bias_pt = ref.bias.detach().clone()

y_ref = ref(x)
y_pt = F.conv2d(x, weight_pt, bias_pt, stride=1, padding=1)

max_abs = (y_ref - y_pt).abs().max().item()
print(f"shape: ref={tuple(y_ref.shape)}, pt={tuple(y_pt.shape)}")
print(f"max_abs_diff: {max_abs:.3e}")
```

Futtatott kimenet (Python 3.12.3, NumPy 2.5.4, `torch 2.15.0+cpu`, CPU, 10 szál, seed 20261020): `shape: ref=(2, 1, 8, 8), pt=(2, 1, 8, 8)`, `max_abs_diff: 5.960e-08`. Ugyanezen a környezeten a `Conv2dFunction` float64 `torch.autograd.gradcheck`-je ([D05]; `eps=1e-6`, `atol=1e-6`, $N = 1$, 6×6) `True`. A float32-os, $\sum y^2$ veszteségű gradiens-összevetés ettől külön ellenőrzés: a saját backwardot a `F.conv2d` backwardjával ([D10]) veti össze ugyanazon a bemeneten; a mért maximumok (seed 20261020; a `Conv2dReference` inicializálása után `x = torch.rand(1, 1, 6, 6)`, majd $\sum y^2$ veszteség): x: $2{,}38 \cdot 10^{-7}$, w: $1{,}91 \cdot 10^{-6}$, b: 0. Ez a korlát a konkrét konfigurációra vonatkozik (a bemenet és a súlyok nagyságától, valamint a float32 műveleti sorrendjétől függ), ezért nem univerzális küszöb; a szigorú ellenőrzés a fenti float64-es `gradcheck`, a float32-os összevetés pedig durva indexelési hibát szűr ki. Ha az eltérés a mért értéket nagyságrendekkel meghaladja, a hiba nem a lebegőpontos kerekítésből, hanem a saját forward indexeléséből vagy padding-szeletéből származik.

A gradiens-összevetés kódja:

```
torch.manual_seed(20261020)
model = Conv2dReference(1, 1, 3, stride=1, padding=1)
x32 = torch.rand(1, 1, 6, 6, dtype=torch.float32).requires_grad_(True)
w32 = model.weight.detach().clone().requires_grad_(True)
b32 = model.bias.detach().clone().requires_grad_(True)

y_ref = Conv2dFunction.apply(x32, w32, b32, 1, 1, 1)
(y_ref * y_ref).sum().backward()

x_pt = x32.detach().clone().requires_grad_(True)
w_pt = w32.detach().clone().requires_grad_(True)
b_pt = b32.detach().clone().requires_grad_(True)
y_pt = F.conv2d(x_pt, w_pt, b_pt, stride=1, padding=1)
(y_pt * y_pt).sum().backward()

print([float((x32.grad - x_pt.grad).abs().max()),
       float((w32.grad - w_pt.grad).abs().max()),
       float((b32.grad - b_pt.grad).abs().max())])
```

37.4.5 Véges gradiens a modellen belül

A rétegszintű gradiensellenőrzésen túl egy modellszintű, véges differenciás ellenőrzést is végzünk. Ennek célja nem a backward implementációjának újraellenőrzése — azt a 37.2. Backward és ellenőrzési lefedettség és 37.3. Linearitás, adjungált és határfeltétel megtette —, hanem annak igazolása, hogy a modulba integrált paraméterekre a PyTorch autograd által kiszámított gradiens megegyezik a független numerikus becsléssel.

A numerikus gradiens definíciója egy f skalár kimenetű függvényre és a θ_i paraméterre:

$$\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta + \varepsilon e_i) - f(\theta - \varepsilon e_i)}{2\varepsilon},$$

ahol e_i az i-edik egységvektor a paramétertérben. A kiértékelés float64-ben történik a 10.3. Gradiensellenőrzés és numerikus hibamérce előírása szerint, noha a modell float32-t használ. A kód:

```
def finite_grad(model, x, target, loss_fn, eps=1e-4):
    model = model.double()  # a numerikus teszt idejére
    x = x.double()
    target = target.double()
    model.eval()
    grads = []
    for p in model.parameters():
        g = torch.zeros_like(p, dtype=torch.float64)
        flat = g.view(-1)
        for i in range(flat.numel()):
            with torch.no_grad():
                orig = p.view(-1)[i].item()
                p.view(-1)[i] = orig + eps
            loss_plus = loss_fn(model(x), target).item()
            with torch.no_grad():
                p.view(-1)[i] = orig - eps
            loss_minus = loss_fn(model(x), target).item()
            with torch.no_grad():
                p.view(-1)[i] = orig
            flat[i] = (loss_plus - loss_minus) / (2 * eps)
        grads.append(g.reshape(p.shape))
    model.float()  # visszaállítás a tanítási dtype-ra
    return grads
```

A függvény a numerikus teszt idejére `model.double()`-lé konvertálja a modellt, és a bemenetet (`x`) valamint a célt (`target`) is float64-be állítja; a visszatérés előtt `model.float()`-tal állítja vissza a tanítási dtype-ot. Ez nem a tanítási beállítás, hanem a numerikus ellenőrzés külön konfigurációja. A teszthez $N = 1$, $H = W = 6$, $k = 3$, $p = 1$, $s = 1$ beállítást használunk, mert a teljes paraméterezés 9 súly + 1 bias = 10 változó, ami egyenként kiértékelhető.

Miért nem futtatunk ReLU-t ebben a modellben? A véges differencia sima pontban pontos; a ReLU töréspontjában a 9.4. Aktivációk és gradiensáramlás szerint a könyvtár által választott egyoldali deriváltkonvenció érvényes, és a középső differencia ott nem feltétlenül egyezik a backwarddal. A jelen modell affin, tehát ez a csapda nem áll fenn.

Várt eredmény a képlet alapján. Az átlagoló célfeladatban a veszteség a paraméterek kvadratikus polinomja, ezért a középső differencia a pontos gradienssel analitikusan egyezik: a $f(\theta+\varepsilon e_i)-f(\theta-\varepsilon e_i)$ különbségben a másodfokú tagok kiesnek, és a maradék pontosan $2\varepsilon\,\partial f/\partial\theta_i$. A gyakorlatban csak a float64 kerekítése marad, ezért $\varepsilon = 10^{-4}$ mellett a relatív hiba jóval 10^{-3} alatt várható a $\partial L/\partial\theta$ minden komponensére. Ha a gradiens abszolút értéke nullához közeli, a relatív hibát stabil nevezővel, $\max(|\text{ref}|, 1)$-gyel számoljuk.

A futtatott ellenőrzés (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 10 szál, seed 20261020, $\varepsilon = 10^{-4}$, $N = 1$, $H = W = 6$; a `Conv2dReference` inicializálása után `x = torch.rand(1, 1, 6, 6)`, a cél a fenti átlagoló szűrő kimenete, a veszteség `MSELoss`) a numerikus és az autograd gradiens maximális abszolút eltérésére a súlyra $3{,}77 \cdot 10^{-14}$, a biasra $2{,}08 \cdot 10^{-14}$ értéket adta, ami jóval a 10^{-3}-as relatív korlát alatt van.

37.4.6 Kis tanítás és az állítás típusa

A következő kód a tényleges kis tanítást futtatja. Az állítás típusa: **empirikus állítás**, ezért pontosan megadjuk az adatot, a splitet, a receptet és a mintavételi egységet.

- **Adat:** szintetikus $X_n \sim \mathrm{Unif}[0;\ 1]^{8\times 8}$ bemenetek; a cél Y_n a definiált 3×3-as átlag. A generálás seedje `20261020`.
- **Split:** nincs train/val/test split, mert a feladat nem generalizálást, hanem az integrációt ellenőrzi. Az 1.3. Adatfelosztás és a megfigyelési egység fogalmai szerint ez egy *szintetikus ellenőrző feladat (sanity task)*, nem valós benchmark. Ezt a korlátot a címben is jelöljük.
- **Modell:** egyetlen `Conv2dReference`, $C_{in} = 1$, $C_{out} = 1$, $k = 3$, $p = 1$, $s = 1$.
- **Optimalizáló:** SGD tanulási rátával $\eta = 0{,}3$, momentum nélkül. A ráta stabilitását a kvadratikus veszteség Hesse-mátrixából becsüljük: a teljes batchre, a mért adatokon float64-ben felírt Hesse-mátrix legnagyobb sajátértéke $\lambda_{\max} = 5{,}3919$, ezért a stabilitási feltétel $\eta < 2/\lambda_{\max} \approx 0{,}371$. Az $\eta = 0{,}3$ ezt teljesíti. Az $\eta = 0{,}5$ már divergál: a futtatásban a loss a 0–5. indexű epochokban rendre 2,96, $1{,}81 \cdot 10^2$, $1{,}39 \cdot 10^4$, $8{,}84 \cdot 10^5$, $5{,}86 \cdot 10^7$, $4{,}25 \cdot 10^9$, a 19. indexű epochnál pedig `inf` (a futás végi érték `(19, inf, 2,32e17)`). A korlát a teljes batch görbületén alapul; mini-batch SGD-nél a lépésenkénti görbület kisebb, ezért a mini-batch futás elvileg nagyobb rátát is elviselhet, de a biztonságos recepthez a teljes batch korlátját használjuk. A cél elérése nem igényel adaptív módszert; az egyszerű módszer teszi a hibát azonnal láthatóvá.
- **Veszteség:** `MSELoss` redukció `mean`.
- **Metrika:** utolsó epoch vesztesége, valamint a súlybeli hiba L_∞-normája a célhoz képest:

$$E_W = \max_{i,j} \left| W_{0,0,i,j} - \frac{1}{9} \right|.$$

- **Leállás:** 60 epoch, batch méret 16, mert a konvergencia a 6.2. Konvexitás és sima optimalizálási határ konvex kvadratikus esete szerint gyors; a futás rövid.

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(20261020)
device = torch.device("cpu")
dtype = torch.float32

N = 64
x = torch.rand(N, 1, 8, 8, dtype=dtype, device=device)
target = torch.zeros_like(x)
for u in (-1, 0, 1):
    for v in (-1, 0, 1):
        target += 1.0 / 9.0 * torch.nn.functional.pad(
            x, (1, 1, 1, 1), mode="constant", value=0.0
        )[:, :, 1+u:1+u+8, 1+v:1+v+8]

model = Conv2dReference(1, 1, 3, stride=1, padding=1).to(device)
optimizer = optim.SGD(model.parameters(), lr=0.3)
loss_fn = nn.MSELoss()

for epoch in range(60):
    model.train()
    perm = torch.randperm(N, device=device)
    total_loss = 0.0
    for start in range(0, N, 16):
        idx = perm[start:start+16]
```

```
            xb, yb = x[idx], target[idx]
            optimizer.zero_grad()
            out = model(xb)
            loss = loss_fn(out, yb)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * len(idx)
        total_loss /= N
        if epoch in (0, 4, 9, 19, 59):
            with torch.no_grad():
                w = model.weight.detach()
                ew = (w - 1.0 / 9.0).abs().max().item()
            print(f"epoch={epoch:2d}  loss={total_loss:.6f}  E_W={ew:.6f}")
```

Futtatott kimenet (CPU, Python 3.12.3, `torch 2.15.0+cpu`, NumPy 2.5.4, 10 szál, seed 20261020, a `Conv2dFunction`-nel; más PyTorch-verzióval az utolsó jegyek eltérhetnek):

```
epoch= 0  loss=0.172650  E_W=0.292765
epoch= 4  loss=0.005204  E_W=0.122228
epoch= 9  loss=0.000873  E_W=0.048343
epoch=19  loss=0.000059  E_W=0.010655
epoch=59  loss=0.000000  E_W=0.000165
```

A konvex kvadratikus feladatban a veszteség 10^{-5} alá csökken, és az utolsó epochra $E_W < 10^{-3}$ teljesül. Az első epochokban a loss még nem kicsi: 20 epoch után $E_W \approx 0{,}011$, ezért az $E_W < 10^{-3}$ kritérium 20 epochnál még nem teljesülne. A konvex veszteség önmagában nem garantálja, hogy rögzített tanulási rátával és epochszámmal egy előírt hibaküszöb elérhető: túl nagy rátánál a helyes gradiens mellett is divergálunk (a fenti $\eta = 0{,}5$), túl kevés lépésnél pedig a hiba még nagy, ami nem implementációhiba. Ezért a kritérium a Hesse-mátrix legnagyobb sajátértékéhez igazított $\eta < 2/\lambda_{\max}$ és az azonos konfigurációjú, natív `nn.Conv2d` referencia-tréninggel való összevetés. Ha a futtatás ettől lényegesen eltér, először a tanulási rátát és a `Conv2dFunction` backwardját ellenőrizzük.

Natív referencia-tréning. Ugyanezt a 60 epochos, $\eta = 0{,}3$, 16-os batchméretű receptet lefuttattuk egy `nn.Conv2d(1, 1, 3, stride=1, padding=1)` modellel is. Az összevetés akkor fair, ha a két modell inicializálása azonos: ehhez az adatgenerálás után elmentettük a PyTorch RNG-állapotát, a saját modell létrehozása után visszaállítottuk, és úgy hoztuk létre a natív modult. A `reset_parameters` ugyanis a `nn.Conv2d` Kaiming-uniform inicializálását és a bias egyenletes eloszlását utánozza, ezért azonos seeddel azonos kezdősúlyokat kapunk. A mért kimenet hat tizedesjegyig megegyezik (loss / E_W): epoch 0: 0,172650 / 0,292765; epoch 4: 0,005204 / 0,122228; epoch 9: 0,000873 / 0,048343; epoch 19: 0,000059 / 0,010655; epoch 59: 0,000000 / 0,000165. A végső értékek is a kerekítés szintjén egyeznek: natív (loss $7{,}6495 \cdot 10^{-9}$, $E_W = 1{,}6493 \cdot 10^{-4}$), saját réteg (loss $7{,}6492 \cdot 10^{-9}$, $E_W = 1{,}6492 \cdot 10^{-4}$). A futásidő ugyanezen a gépen 10 szálon 0,11 s, illetve 57,5 s volt; ez a különbség a Python-ciklusos referencia ára, nem numerikus hiba.

A tanulási referencia mivolta. A réteg nem termelési sebességű: minden kimeneti pixel kiszámításához Python-ciklust használ. A 34.1. Mért késleltetés, throughput és memória alfejezetben bevezetett mérési követelmények szerint a sebességet csak megjelölt hardveren, ismételt futással szabad közölni. Itt az állítás tehát nem az, hogy a réteg gyors, hanem az, hogy a tanulási protokollban a gradiens helyesen áramlik. A sebességre vonatkozó egyetlen állításunk egy mért korlát: ugyanazon a 60 epochos recepten a Python-hivatkozás 57,5 s, a natív `nn.Conv2d` 0,11 s falidőt adott (10 szál, CPU), ami nagyságrendi különbség; ez a 37.1. Operátorspecifikáció és kézi referencia alfejezetben elfogadott tervezési döntés. **Egy lassú, világos referenciakód nem hibás csak attól, hogy nem termelési kernel.** A hibásság kritériuma a numerikus egyezés megsértése, nem a sebesség.

37.4.7 A benchmark hatóköre és az átadási csomag

A 36.4. IX. ellenőrző pont: lektorálható modellállítás hat komponenséből itt a kódot, a levezetést és a futási jegyzéket adjuk át; a páros eredmény nem része ennek az egységnek, mert nincs két összehasonlítandó modell. A benchmark hatóköre három pontban rögzíthető:

1. **A teszt célja:** integrációs kontroll, nem architektúra-összehasonlítás.
2. **Az adat tartománya:** szintetikus, 8×8-as, ismert lineáris cél; a 25.4. Nyílt halmaz és ismeretlen input terminológiája szerint a modell csak ezen a zárt tartományon értelmezhető.
3. **A nem vizsgált kérdések:** általánosítás zajra, augmentációra, nagy csatornaszámra, GPU-ra, vegyes pontosságú (mixed precision) számításra. Ezeket a 24.4. VI. ellenőrző pont: megbízható CNN-training kerete szerint későbbi érvényességi lépésben kell külön igazolni.

A beadandó minimális környezet- és futásjegyzék:

Tétel	Érték
Könyvtár	projekt helyi `lab/` könyvtára
Python	3.12.3 (a 2.3. Izolált környezet, notebook és verzió lock-profilja `>=3.12,<3.13`)
PyTorch	2.15.0+cpu (a mért ellenőrző környezet; a labor `uv.lock` fájlja 2.14.1+cpu-t rögzít)
NumPy	2.5.4 (a mért ellenőrző környezet; a lock 2.3.5)
Futtatás	a 37.4.4., 37.4.5. és 37.4.6. szakasz kódblokkja egy szkriptben, ebben a sorrendben
Seed	`20261020`
dtype/device	float32, CPU
Output	a szakaszokban közölt mért értékek (rétegdiff, gradienshiba, loss, E_W)

A futási jegyzék számait a 37.4.4–37.4.6. szakasz kódblokkjai állítják elő: a rétegdiffet a 37.4.4., a gradienshibát a 37.4.5., a végső losszt és E_W-t a 37.4.6. szakasz. Külön, beégetett eredményfájlt nem használunk, mert a fejezet kódblokkjai önmagukban reprodukálják a számokat, és így a futtatás eredménye szétválasztható a szöveges értelmezéstől.

37.4.8 Gyakori tévedés és ellenpélda

A tipikus tévedés az, hogy aki a rétegszintű teszteket lefuttatta, az a modell tanulási sikerét automatikusan a réteg helyességének tulajdonítja. **Ellenpélda.** Tegyük fel, hogy a réteghez hozzáadunk egy helytelen bias-regisztrációt: a `bias`-t nem `nn.Parameter`-ként regisztráljuk, hanem például sima, `requires_grad=True` tenzorként tároljuk attribútumként. Az ilyen tenzor kaphat gradienst, de a `parameters()` nem sorolja fel, ezért kimarad az optimalizáló `param_groups` listájából, és sosem frissül. (Ha a tenzor `requires_grad=False`, akkor gradienst sem kap; az a változat még távolabb van a helyes viselkedéstől, de más hiba.) A forward kimenete a rögzített bias mellett is tökéletesen egyezhet a PyTorch kimenetével a rétegszintű teszten. A tanulási teszten a hiba csak akkor bukik meg, ha a bias ténylegesen számít: a fenti átlagoló feladat optimuma ugyan $b = 0$, de a `reset_parameters` nemnulla kezdeti biast ad, így a befagyott bias a veszteséget egy nemnulla szinten tartja; mivel az $E_W < 10^{-3}$ kritérium csak a súlyt méri, ez a teszt a regisztrációs hibát önmagában nem feltétlenül fedi fel. Ezért a regisztrációs hibakontroll olyan feladaton fusson, ahol a bias szükséges (például $Y + 0{,}5$ célon), és közvetlenül is ellenőrizzük, hogy minden tanulandó tenzor szerepel-e a `model.parameters()` felsorolásában,

ezáltal az optimalizáló `param_groups` listájában, valamint hogy minden regisztrált paraméter `grad`-ja nem `None`-e az első `backward` után. A hiba oka a paraméterregisztráció megsértése, nem a forward vagy a backward numerikus pontatlansága.

37.4.9 Önellenőrzés és visszatekintés

Két gyors önellenőrzést végzünk el:

1. **Alakellenőrzés.** A modell bemenete $(N, 1, 8, 8)$, súlya $(1;\ 1;\ 3;\ 3)$, padding $p = 1$, stride $s = 1$, dilation $d = 1$; a kimeneti alak $H_{out} = W_{out} = 8$, tehát a kimeneti alak $(N, 1, 8, 8)$, az MSE ugyanolyan alakú cél ellen számolható.
2. **Szélsőeset.** Ha a bemenet konstans c, a 3×3-as átlagoló szűrő kimenete a belső pixeleken szintén c, a széleken kisebb a zero padding miatt; a veszteség ekkor is véges. Ha a súly minden eleme $1/9$ és a bias 0, a veszteség pontosan 0 az egész tartományon, mert a modell definíció szerint a célt számolja.

Visszatekintés a 37.3. Linearitás, adjungált és határfeltétel alfejezetből: a réteg linearitása bias nélkül $T(aX+bZ) = aT(X)+bT(Z)$. Ebből következik, hogy a modell kimenete a paraméterekben affin, a veszteség pedig kvadratikus; egy kvadratikus polinom középső differenciája analitikusan pontos, ezért a modellszintű gradiensellenőrzés és az autograd egyezése nem véletlen, hanem a függvény alakjából következik. A 36.4. IX. ellenőrző pont: lektorálható modellállítás alfejezetből pedig azt használjuk, hogy a lektorálható csomagnak a feltételeket és a futáskörnyezetet is tartalmaznia kell, nem csak a végső számot.

37.4.10 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el, mi a különbség a rétegszintű backward-ellenőrzés és a modellszintű tanulási kontroll között. Adjon egy olyan hibát, amelyet az első nem feltétlenül fed fel, a második viszont igen.

F2 (levezetési, nehézség 2). Vezesse le a 3×3-as átlagoló szűrő célfüggvényére, hogy a modell által elérhető minimális MSE pontosan 0, és adja meg az optimális súlyt és biast. Igazolja, hogy egyetlen minimum létezik a (W, b) térben.

F3 (implementációs, nehézség 2). Írjon teljes Python-függvényt, amely a saját `Conv2dReference` réteggel 200 teljes batch-lépésen át tanítja az átlagoló feladatot (`lr=0.3`), és a végén ellenőrzi, hogy $E_W < 10^{-3}$. A függvény adja vissza a `(loss, E_W)` párt. Input: seed, N; output: a két lebegőpontos érték.

F4 (hibakeresési, nehézség 2). A tanítási ciklusban a loss minden epochban nő a második epoch után, noha a feladat konvex. Soroljon fel két lehetséges okot, és mindkettőre adjon konkrét javítást a kód szintjén.

F5 (kísérlettervezési, nehézség 3). Tervezze meg azt a kísérletet, amellyel a saját réteg integrációját a GlyphLab 24×24-es képein, egy két konvolúciós rétegből és egy lineáris fejből álló modellen ellenőrizné. Adja meg: a réteg konfigurációját, a tanítási receptet, a baseline-t, a metrikát és az érvényességi korlátot. Nem kell futtatni.

37.4.11 Megoldások és indoklás

F1. A rétegszintű backward-ellenőrzés a backward tenzorműveletet önmagában vizsgálja: rögzített inputra/súlyra a saját gradiens egyezik-e a numerikus vagy PyTorch-referenciával. A modellszintű tanulási kontroll ezen felül a paraméterregisztrációt, az optimalizálóba kerülést, a loss csatlakoztatását és a több lépéses gradiensáramlást is teszteli. A bias sima attribútumként tárolása a rétegszintű teszten átmegy, de a tanulásban a bias nem frissül, és a loss magasan ragad.

F2. Legyen a modell kimenete $\hat{Y} = W * X + b$, a cél $Y = A * X$, ahol A az átlagoló szűrő. A veszteség a rögzített N mintán

$$L = \frac{1}{NH_{out}W_{out}} \sum_{n,i,j} \big((W - A) * X_{n,i,j} + b\big)^2 .$$

Mivel $L \geq 0$, és $W = A$, $b = 0$ esetén minden X-re nulla a veszteség, a minimum elérhető. Az egyértelműséghez írjuk fel a veszteséget a $\varphi_{n,i,j} = (\mathrm{vec}(X_{n,\mathrm{patch}}), 1) \in \mathbb{R}^{10}$ vektorral, ahol $X_{n,\mathrm{patch}}$ a 3×3-as ablak (zero paddinggel), és legyen $\theta = (\mathrm{vec}(W), b)$, $\theta_A = (\mathrm{vec}(A), 0)$. Ekkor

$$L = \frac{1}{NH_{out}W_{out}} \sum_{n,i,j} \big(\varphi_{n,i,j}^{\top}(\theta - \theta_A)\big)^2 ,$$

és a Hesse-mátrix

$$H = \frac{2}{NH_{out}W_{out}} \sum_{n,i,j} \varphi_{n,i,j}\varphi_{n,i,j}^{\top}.$$

A bias tagot a φ konstans 1 komponense adja, a pozícióátlag pedig a $1/(NH_{out}W_{out})$ szorzóban van. A mért adatokon (a 37.4.6. szakasz $N = 64$, 8×8 bemenete) H pozitív definit: a legkisebb sajátérték 0,0898, a legnagyobb 5,3919, ezért a minimum egyértelmű. Általában is, ha a bemenet komponensei függetlenek és nem degeneráltak, $\mathbb{E}[\varphi\varphi^{\top}]$ pozitív definit, mert a φ tíz komponense lineárisan független függvénye a bemenetnek.

F3. A kód:

```
def train_mean_filter(seed=20261020, N=64):
    torch.manual_seed(seed)
    x = torch.rand(N, 1, 8, 8, dtype=torch.float32)
    target = torch.zeros_like(x)
    for u in (-1, 0, 1):
        for v in (-1, 0, 1):
            target += 1.0/9.0 * torch.nn.functional.pad(
                x, (1,1,1,1), value=0.0
            )[:, :, 1+u:1+u+8, 1+v:1+v+8]
    model = Conv2dReference(1, 1, 3, stride=1, padding=1)
    opt = optim.SGD(model.parameters(), lr=0.3)
    loss_fn = nn.MSELoss()
    for _ in range(200):
        opt.zero_grad()
        out = model(x)
        loss = loss_fn(out, target)
        loss.backward()
        opt.step()
    with torch.no_grad():
        E_W = (model.weight - 1.0/9.0).abs().max().item()
    return loss.item(), E_W
```

```
print(train_mean_filter())
```

Alakellenőrzés: a cél (64; 1; 8; 8), a modell kimenete ugyanaz. Szélsőeset: az $N = 1$ bemenet alakilag végigfut, de a `MSELoss` átlagolása miatt a gradiens skálája a batchmérettől függ, ezért a 200 lépéses, $N = 64$-re kalibrált $E_W < 10^{-3}$ kritérium $N = 1$-re nem öröklődik; más N-re a kritériumot újra kell kalibrálni. Tolerancia: E_W a float32 konvergenciától függően 10^{-3} alatt várható. A futtatott hívás eredménye (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 10 szál) `(6.609415947878006e-08, 0.0005021616816520691)`, tehát a loss $6{,}61 \cdot 10^{-8}$ és $E_W = 5{,}02 \cdot 10^{-4}$, ami teljesíti az $E_W < 10^{-3}$ kritériumot. Ez a mérés a F3 függvény saját futása (200 teljes batch-lépés, $N = 64$), nem a 37.4.6. szakasz 60 epochos, 16-os batchméretű mini-batch futása.

F4. Első ok: a tanítási ráta túl nagy, például $\eta = 10$; a nagy gradienslépések miatt a veszteség túllő és oszcillál. Javítás: csökkentsük η-t 0,1-re, és rajzoljuk ki a lossgörbét. Második ok: a ciklusból elmarad az `optimizer.zero_grad()`, ezért a gradiensek a batch-eken át felhalmozódnak, a lépés tényleges hatása a ciklus előrehaladtával nő, és a konvex kvadratikus feladaton is túllő; a veszteség a második epoch után már nem csökken, hanem nőni kezd vagy magas szinten oszcillál. Javítás: minden batch elején hívjuk meg az `optimizer.zero_grad()`-ot. (A paraméterregisztráció hiánya nem ez a hiba: ha a `model.parameters()` üres, az `optim.SGD([])` már a konstruktorban `ValueError`-t dob, nem csendes lossnövekedést okoz.)

F5. A kísérleti terv: a GlyphLab train/val/test splitjét a függelék A. A GlyphLab adatkészlet szakaszában rögzített szerződés szerint használjuk, a modell két `Conv2dReference` rétegből áll: az első $1 \to 8$ csatorna, $k = 3$, $p = 1$, $s = 2$; a második $8 \to 16$, $k = 3$, $p = 1$, $s = 2$; ezután BatchNorm nélkül, ReLU-val és egy $16 \cdot 6 \cdot 6 \to 3$ lineáris fejjel. A baseline ugyanez a modell PyTorch `nn.Conv2d` rétegekkel, azonos seeddel és súlyinicializálással. A metrika a validációs accuracy a 60 validációs képen; a független egység azonban a 30 jelenet, ezért a különbséget jelenetszintű páros bootstrap-pal is ellenőrizzük. A kísérlet akkor tekinthető sikeres integrációnak, ha öt seeddel futtatva a saját réteges modell és az azonos seeddel inicializált baseline validációs accuracy-különbségének átlaga abszolút értékben legfeljebb 3,3 százalékpont (egy jelenet a 30-ból, $1/30 \approx 3{,}3\%$), és a különbségek 95%-os t-konfidenciaintervalluma tartalmazza a 0-t. A korlát: a GlyphLab nem valós benchmark, és a 90°-os augmentáció labelváltó (a vertical és horizontal címkét felcseréli), ezért ebben a kísérletben nem használható; a részletes adatszerződést a laborcsomag `ADATOK.md` fájlja és az A. A GlyphLab adatkészlet függelék rögzíti. A tanítás a saját réteget is tartalmazó modellen a 37.5 labor SGD-receptjével történik ($\eta = 0{,}3$, momentum nélkül, batch 16, 60 epoch); a seedek 20261020–20261024, az erőforrás CPU, öt seed és két modell.

37.4.12 Rövid összefoglaló

A saját konvolúciós réteget `nn.Module`-ba integráltuk, paramétereit `nn.Parameter`-ként regisztráltuk, és azonos súlyokon egyeztettük a PyTorch `F.conv2d`-vel. Modellszintű véges differenciás ellenőrzést adtunk float64-ben. A kis tanítási feladat egy ismert 3×3-as átlagoló szűrőre konvergál SGD-vel; az állítást szintetikus ellenőrző feladatként (sanity task) jelöltük, nem benchmarkként, és a natív `nn.Conv2d` referencia-tréning ugyanezt a görbét adta. A lassú referenciakód nem hibás, ha numerikusan egyezik; a gyakori tévedés a paraméterregisztráció hibája, amelyet azonos kimeneti tesztek önmagukban nem fednek fel. Az átadási csomag a kódot, a környezet- és futásjegyzéket, valamint a 37.4.4–37.4.6. szakasz mért kimeneteit tartalmazza.

37.4.13 Következő lecke

A 38.1. Adat, split és előzetes protokoll egységben ezt a réteget a teljes GlyphLab osztályozási kísérlet adatfolyamába illesztjük: adat, split, előzetes protokoll és a modell kiválasztásának rögzített rendje.

37.4.14 Források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D05] PyTorch contributors (2026). *Autograd mechanics*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/autograd.html. Hozzáférés: 2026-10-30.
- [D07] PyTorch contributors (2026). *CrossEntropyLoss*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.

38 Projekt II: teljes CNN-osztályozási kísérlet

38.1 Adat, split és előzetes protokoll

38.1.1 Cél, előismeret és induló kérdés

A Projekt II a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-adatkészlet teljes CNN-osztályozási kísérlete: az adat előkészítésétől a befagyasztott modell végső clean/korrupt tesztjéig. Ez az egység a projektet megnyitó döntéseket rögzíti, **mielőtt** bármilyen modellt tanítanánk: mi a megfigyelési egység, miért jelenetszintű a split, mi a baseline, mi az erőforráskeret, és milyen manifest igazolja az adat sértetlenségét. Előismeret: a checkpoint- és a tanítási/kiértékelési (train/eval) szerződés a 12.4. III. ellenőrző pont: önállóan tanuló kis háló alfejezetből, az architektúra-döntési lap és a kis saját CNN választása a 20.4. V. ellenőrző pont: indokolt CNN-választás alfejezetből, az osztályarány és a mintavételezés a 21.4. Osztályarány, súlyozás és céleloszlás alfejezetből, a clean/korrupt teszt megkülönböztetése a 22.4. Augmentációs kísérlet és domainváltozás alfejezetből, a szeletek és modellkártya a 23.4. Hibaszeletek és alkalmazási határ alfejezetből, valamint a megbízható CNN-training csomagja a 24.4. VI. ellenőrző pont: megbízható CNN-training alfejezetből. Visszautalunk a konvolúciós operátor NCHW tenzoralak-szerződésére 37.1. Operátorspecifikáció és kézi referencia, a tényleges időmérés követelményére 34.1. Mért késleltetés, throughput és memória és a fél-nyílt dobozkoordinátákra 26.1. Dobozkoordináta, IoU és lokalizációs loss.

Induló kérdés. A 360 GlyphLab-képen egy jelenethez két zajos nézet tartozik. Mi történne, ha a képeket a jelenetekre tekintet nélkül véletlenszerűen osztanánk train/val/test részre? Becslés: a jelenetek nagyjából fele átszivárogna a részek között, tehát a mérés már nem az „új jelenet” hibáját közelítené. A becslést a 38.1.3. szakasz egzakt számpéldával és lemmával ellenőrzi.

38.1.2 Definíciók: jelenet, split, manifest és a projekt szerződése

Definíció (jelenet, nézet, kép). A GlyphLab adatban $\mathcal{S} = \{s_1;\ \ldots;\ s_{180}\}$ a 180 független jelenet halmaza. Minden $s \in \mathcal{S}$ jelenethez két nézet tartozik, $V_s = \{0;\ 1\}$, ezért a képek halmaza:

$$\mathcal{J} = \{(s, v) : s \in \mathcal{S},\ v \in V_s\}, \qquad |\mathcal{J}| = 360.$$

A kép tenzora $X_{s,v} \in \mathbb{R}^{1\times 24\times 24}$, NCHW alakban, float32, $[0;\ 1]$ intenzitással; a cél $y_{s,v} \in \{0;\ 1;\ 2\}$, ahol 0 vertical, 1 horizontal, 2 diagonal. A metadata `index` oszlopa a kép sorszáma a `data/images.npy` tömb első tengelyén; a `view_id` 0 vagy 1, a `scene_id` azonosítja a csoportot. A maszk és az xyxy doboz annotáció, nem osztályozóbemenet; a doboz nullától indexelt, fél-nyílt intervallum, ahogyan a 26.1. Dobozkoordináta, IoU és lokalizációs loss is rögzíti.

Definíció (jelenetszintű split). A projekt splitje a jelenetek osztályrétegzett partíciója:

$$\mathcal{S} = \mathcal{S}_{\text{tr}} \cup \mathcal{S}_{\text{val}} \cup \mathcal{S}_{\text{test}},$$

ahol $|\mathcal{S}_{\text{tr}}| = 120$, $|\mathcal{S}_{\text{val}}| = 30$, $|\mathcal{S}_{\text{test}}| = 30$, és minden osztályból rendre 40, 10, 10 jelenet kerül a három részbe. A képindexek a splitből származnak:

$$I_{\text{split}} = \{(s, v) : s \in \mathcal{S}_{\text{split}},\ v \in \{0;\ 1\}\}.$$

Így a train/val/test képszám 240/60/60, és minden osztályból 80/20/20 kép jut egy-egy részbe. **Egy jelenet mindkét nézete ugyanabban a splitben van.** A clean teszt a `data/images.npy` azon 60 sora, amely a metadata `split=="test"` sorainak felel meg; a corrupt teszt `data/corrupted_test.npy` sorrendje ugyanez a metadata-test-sorrend, ezért a j-edik corrupt kép az j-edik clean tesztkép rögzített zajos, fényességgradienssel ellátott változata. A párosítás a 22.4. Augmentációs kísérlet és domainváltozás alfejezetben bevezetett clean–corrupt szerződés.

Definíció (adatmanifest). A manifest a projekt adatjegyzéke: verzió, generátorseed, tömbalakok és dtype-ok, splitdarabszám, valamint a fájlok SHA-256 lenyomata. A hash a teljes fájlra vonatkozik, nem a tensor tartalmának külön reprezentációjára. Belső adatnál a jegyzék rögzíti az eredetet (saját generátor); külső adatnál az alábbi mezőket is kötelezővé tesszük: `name`, `origin_url`, `license`, `license_url`, `download_date`, `preprocessing`, `sha256`. A GlyphLab projektben külső adat nincs, így az `external_data` lista üres.

A megfigyelési egység az 1.3. Adatfelosztás és a megfigyelési egység értelmében az az objektum, amelynek az újrafelhasználásakor függetlenséget várunk. Itt ez a `scene_id`: a két nézet ugyanannak a geometriának két zajos megvalósítása, ezért a nézetek nem képezhetnek önálló, független megfigyelést a train/test-határon át.

38.1.3 Levezetési kötelezettség: miért a jelenetszintű split illik a használati helyzethez?

A megnevezett felhasználási helyzet (U1). A projekt kérdése zárt, háromosztályú felismerés: *egy új, a tanításban nem látott jelenetből érkezik egyetlen zajos* 24×24*-es szürke nézet, és a modellnek meg kell adnia az orientációs osztályt.* Ebből a helyzetből következik, hogy a tesztnek az új jelenet hibáját kell mérnie, nem pedig egy már látott jelenet második négyzetét. A jelenetszintű split pontosan ezt a célt reprezentálja; a képszintű split ezt megsérti.

Ellenpélda: nézetszivárgás egy memóriamodellel. Tegyük fel, hogy egy jelenet `view 0` képe a trainben, `view 1` képe a tesztben van. Egy h_{mem} memóriamodell, amely a tanítóképek minden pixelét és címkéjét eltárolja, a teszt `view 1` képen a látott geometriát azonosíthatja, és a címkét visszaadhatja. Ez a modell a teszt held-out (visszatartott) pontosságát felfelé tolja, miközben új jelenetekre nem általánosít. A hiba tehát nem a használati helyzet kockázatát becsli, hanem a modell memóriáját.

Számpélda: a kép-szintű felosztás várható szivárgása. Ha a 360 képet a jelenetre tekintet nélkül véletlenszerűen osztjuk 240/60/60 részre, egy rögzített jelenet két nézetének együttes hovatartozása három kedvező esemény összege: mindkettő train, mindkettő val, vagy mindkettő test. Mivel a két nézet konkrét kép, a valószínűség:

$$P_{\text{same}} = \frac{\binom{240}{2} + \binom{60}{2} + \binom{60}{2}}{\binom{360}{2}} = \frac{240 \cdot 239 + 60 \cdot 59 + 60 \cdot 59}{360 \cdot 359} = \frac{64440}{129240} \approx 0{,}4986.$$

Annak valószínűsége, hogy a jelenet két nézete **különböző** splitbe kerül:

$$P_{\text{diff}} = 1 - P_{\text{same}} \approx 0{,}5014.$$

A 180 jelenetre a várhatóan megsértett csoportok száma a várható érték linearitása miatt

$$180 \cdot P_{\text{diff}} \approx 90{,}25.$$

Tehát a naiv felosztás várhatóan mintegy 90 jelenetet hasítana ketté, és a teszt már nem lenne „új jelenet” mérés.

Lemma (a jelenetszintű tesztbecslő torzítatlansága). Legyen h a teljes tanítási eljárás kimenete, amely csak $\mathcal{S}_{\text{tr}} \cup \mathcal{S}_{\text{val}}$ adataitól függ; legyen a 0/1 hiba $\ell(h,(s,v)) = \mathbf{1}[h(X_{s,v}) \neq y_{s,v}] \in [0;\ 1]$. Tegyük fel, hogy a tesztjelenetek a használati U1 helyzet $\mathcal{D}$ eloszlásából származnak (a jelenet és a nézet együttes eloszlása), a két nézet marginális képeloszlása azonos, és a tesztjelenetek függetlenek a tanítási előzménytől. Az empirikus teszthiba

$$\widehat{\text{err}} = \frac{1}{60} \sum_{s \in \mathcal{S}_{\text{test}}} \sum_{v \in \{0;\ 1\}} \ell(h,(s,v)).$$

Ekkor $\mathbb{E}[\widehat{\text{err}} \mid \mathcal{H}] = L(h)$, ahol $\mathcal{H}$ a train/val előzmény, h annak függvénye, és $L(h)$ a U1 szerinti valódi várható hiba.

Bizonyítás. Az $\mathcal{H}$ feltételre h rögzített. A tesztjelenetek függetlensége miatt minden $s \in \mathcal{S}_{\text{test}}$ és $v \in \{0;\ 1\}$ esetén

$$\mathbb{E}[\ell(h,(s,v)) \mid \mathcal{H}] = \mathbb{E}_{S' \sim \mathcal{D}}[\ell(h,(S',v))] = L_v(h).$$

A nézetek marginális azonossága szerint $L_0(h) = L_1(h) = L(h)$. A feltételes várható érték linearitásával:

$$\mathbb{E}[\widehat{\text{err}} \mid \mathcal{H}] = \frac{1}{60} \sum_{s \in \mathcal{S}_{\text{test}}} \sum_{v=0}^{1} L(h) = L(h).$$

A teljes várható értéket véve $\mathbb{E}[\widehat{\text{err}}] = \mathbb{E}[L(h)]$: a becslő torzítatlan a tanítási eljárás feletti U1-kockázatra. □

A lemma két korlátját ki kell mondani. Egyrészt a nézetek közötti függetlenség **nem** feltétel a torzítatlansághoz, csak a marginálisok azonossága; ezért a 60 tesztkép nem 60 független megfigyelés. A konfidenciaintervallumot ezért jelenetszintű resamplinggel számoljuk (a resampling alapja a 5.4. Függő képek, seedek és resampling alfejezet), nem képszintű binomiális formulával. Másrészt a lemma feltételezi, hogy a modell kiválasztása nem használta a tesztet; ha a teszt visszahat a választásra, a feltétel megsérül, és a 38.1.7. szakasz tipikus tévedéséhez jutunk.

38.1.4 Előzetes protokoll és baseline

A 36.4. IX. ellenőrző pont: lektorálható modellállítás lektorálható állításának mintájára a protokollt a kísérlet **előtt** kell lefagyasztani. A projekt fő kérdése:

> A befagyasztott, saját kis CNN clean-teszt-pontossága meghaladja-e a lineáris baseline clean-teszt-pontosságát, és mekkora a clean–corrupt páros romlás?

A döntési táblázat:

Elem	Befagyasztott döntés
Használati helyzet	Új jelenetből egyetlen view, zárt 3 osztály
Megfigyelési egység	`scene_id`
Split	120/30/30 jelenet, osztályrétegzett, minden view egy helyen
Modell	A 38.2. Modell, training és best checkpoint `GlyphCNN`-je: két 3×3-as konvolúciós blokk, 32 egységes rejtett réteg, 19 811 paraméter ($80 + 1168 + 18\,464 + 99$, a 38.2. Modell, training és best checkpoint rétegenkénti bontása szerint), scratch, CPU; egyetlen forrása a `lab/glyph_models.py`
Baseline	Lineáris osztályozó: flatten $576 \to 3$ logit (1731 paraméter), ugyanazzal a tanítási recepttel és választási szabállyal; külön minimum-referencia a képfüggetlen prior ($\log 3$ loss, 1/3 accuracy)
Választási szabály	Futásonként a legkisebb val_loss epoch-snapshot (epochonkénti pillanatkép); architektúránként a három futás közül a legkisebb best val_loss; a választás csak train és val adatot használ
Teszt	Egyszer, befagyasztott checkpoint után; clean, majd corrupt páros
Seedek	20261020, 20261021, 20261022 mindkét modellre; a seed-szórás a kísérlet bizonytalanságát mutatja, a rögzített seedek pedig a futások reprodukálhatóságát szolgálják [D02]
Elsődleges metrika	Per-view clean accuracy, jelenet-bootstrap CI
Másodlagos	Osztályonkénti recall; clean–corrupt páros accuracy-különbség

A baseline szándékosan a legegyszerűbb értelmes osztályozó. Az U1 képek osztályonkénti intenzitásmintázata lineárisan is megkülönböztethető lehet, ezért a lineáris modell azt a kérdést teszi fel, hogy a konvolúciós lokalitás és a nemlinearitás ad-e mérhető előnyt. A lineáris modell a $24 \cdot 24 = 576$ pixelt egyetlen vektorba lapítja, utána $Z = W\tilde{x} + b \in \mathbb{R}^3$ logitot számol; ez $576 \cdot 3 + 3 = 1731$ paraméter. A projekt CNN-je 19 811 paraméteres, vagyis több mint tízszer akkora; a két modell kapacitása tehát nem azonos, és az összevetés azt a kérdést dönti el, hogy a konvolúciós lokalitás, a nemlinearitás és a nagyobb kapacitás *együtt* ad-e mérhető előnyt. Ha a reprezentáció szerkezetét a kapacitástól is el akarjuk választani, egy kapacitásban illesztett

változat (például a 24.4. VI. ellenőrző pont: megbízható CNN-training 2979 paraméteres `TinyCNN`-je, ahol $2979 = 80 + 1168 + 1731$ és $1731 = 576 \cdot 3 + 3$) külön, előre deklarált harmadik karként vehető fel; ez a jelen protokollnak nem része.

A magasabb szintű protokollt a labor `protocol_n149.json` fájlként menti, `protocol_version` = 1 jelöléssel. Ha a modell vagy a baseline a végső teszt előtt megváltozik, az új protokoll új verziószámot kap, és a régi verzió a naplóban marad; a teszt megnyitása után a protokoll nem módosítható. Külső adat választása esetén ugyanez a fájl kötelezően tartalmazza az eredet- és licencjegyzéket; ennek hiánya a labort elfogadhatatlanná teszi.

38.1.5 Labor: előzetes protokoll és ellenőrzött adatmanifest

A labor célja kettős: egy futtatható ellenőrző és egy manifest-kimenet. Az alábbi kód nem helyettesíti a kezdőlabor ellenőrzött `lab/ellenorzott_report.json` jelentését; a futtatást a hallgató végzi el, és a kimenetet a projektnaplóhoz csatolja.

```
# lab/protocol_check.py
from pathlib import Path
import csv, hashlib, json
import numpy as np

HERE = Path(__file__).resolve().parent
DATA = HERE / "data"

FILE_NAMES = [
    "images.npy", "masks.npy", "labels.npy",
    "corrupted_test.npy", "metadata.csv",
]

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def main() -> None:
    manifest = json.loads((DATA / "manifest.json").read_text(encoding="utf-8"))
    with (DATA / "metadata.csv").open(newline="", encoding="utf-8") as f:
        metadata = list(csv.DictReader(f))

    assert len(metadata) == 360, "a metadata.csv-nek 360 rekordot kell tartalmaznia"

    required_keys = {
        "synthetic", "generator_version", "seed", "scenes_per_class",
        "views", "class_names", "input_layout", "input_range",
        "independent_unit", "box_convention", "counts", "scene_counts",
        "arrays", "sha256", "corrupt_order",
    }
    missing = required_keys - set(manifest)
    assert not missing, f"a manifest hiányzó kulcsai: {sorted(missing)}"
    assert manifest["input_layout"] == "NCHW"
    assert manifest["independent_unit"] == "scene_id"
    assert manifest["views"] == 2 and manifest["scenes_per_class"] == 60
    assert manifest["input_range"] == [0, 1]

    images = np.load(DATA / "images.npy")
    labels = np.load(DATA / "labels.npy")
    masks = np.load(DATA / "masks.npy")
    corrupt = np.load(DATA / "corrupted_test.npy")
```

```
assert images.dtype == np.float32 and images.shape == (360, 1, 24, 24)
assert ((images >= 0.0) & (images <= 1.0)).all()
assert masks.dtype == np.uint8 and masks.shape == (360, 1, 24, 24)
assert set(np.unique(masks)) <= {0, 1}
assert labels.dtype == np.int64 and labels.shape == (360,)
assert set(np.unique(labels)) <= {0, 1, 2}
assert corrupt.dtype == np.float32 and corrupt.shape == (60, 1, 24, 24)
assert np.isfinite(corrupt).all()

# A manifest tömbsémája egyezzen a tényleges tömbökkel.
arrays = {
    "images.npy": images, "masks.npy": masks,
    "labels.npy": labels, "corrupted_test.npy": corrupt,
}
schema_ok = all(
    list(arr.shape) == manifest["arrays"][name]["shape"]
    and str(arr.dtype) == manifest["arrays"][name]["dtype"]
    for name, arr in arrays.items()
)
for name, arr in arrays.items():
    spec = manifest["arrays"][name]
    assert list(arr.shape) == spec["shape"], f"alak-eltérés a manifesthez képest: {name}"
    assert str(arr.dtype) == spec["dtype"], f"dtype-eltérés a manifesthez képest: {name}"

# A metadata sora a képtömb ugyanazon indexét írja le.
for i, row in enumerate(metadata):
    assert int(row["index"]) == i, "a metadata indexének a képindexszel kell egyeznie"
    assert int(row["label"]) == int(labels[i])
    assert row["split"] in {"train", "val", "test"}

# Jelenetintegritás: minden scene-hez pontosan a két view tartozik,
# egyetlen splitben, egyetlen címkével.
rows_by_scene: dict[str, list[dict[str, str]]] = {}
for row in metadata:
    rows_by_scene.setdefault(row["scene_id"], []).append(row)

assert len(rows_by_scene) == 180
scene_flags = []
for scene_id, rows in rows_by_scene.items():
    view_ok = {row["view_id"] for row in rows} == {"0", "1"}
    split_ok = len({row["split"] for row in rows}) == 1
    label_ok = len({row["label"] for row in rows}) == 1
    assert view_ok, f"a(z) {scene_id} jelenet nézetei hiányosak"
    assert split_ok, f"a(z) {scene_id} jelenet több splitben van"
    assert label_ok, f"a(z) {scene_id} jelenet címkéi eltérnek"
    scene_flags.append(view_ok and split_ok and label_ok)
scene_integrity = len(scene_flags) == 180 and all(scene_flags)

# Osztályrétegzett jelenetszámok: 40/10/10 minden osztályra.
scene_split = {s: rows[0]["split"] for s, rows in rows_by_scene.items()}
scene_label = {s: rows[0]["label"] for s, rows in rows_by_scene.items()}
counts: dict[str, int] = {}
for split in ("train", "val", "test"):
    for cls in ("0", "1", "2"):
        counts[f"{split}/class_{cls}"] = sum(
            1 for s in rows_by_scene
            if scene_split[s] == split and scene_label[s] == cls
        )
assert counts == {
    "train/class_0": 40, "train/class_1": 40, "train/class_2": 40,
    "val/class_0": 10,   "val/class_1": 10,   "val/class_2": 10,
    "test/class_0": 10,  "test/class_1": 10,  "test/class_2": 10,
```

```
    }, counts

    # A manifest splitdarabszámai egyezzenek a metadata tényleges tartalmával.
    metadata_counts = {
        split: sum(1 for r in metadata if r["split"] == split)
        for split in ("train", "val", "test")
    }
    metadata_scene_counts = {
        split: sum(1 for s in scene_split if scene_split[s] == split)
        for split in ("train", "val", "test")
    }
    counts_ok = (metadata_counts == manifest["counts"]
                 and metadata_scene_counts == manifest["scene_counts"])
    assert metadata_counts == manifest["counts"], (metadata_counts, manifest["counts"])
    assert metadata_scene_counts == manifest["scene_counts"], (
        metadata_scene_counts, manifest["scene_counts"])

    test_meta = [r for r in metadata if r["split"] == "test"]
    assert len(test_meta) == 60 == corrupt.shape[0]
    # A corrupt sorrend a metadata test sorrendjét követi; ezt a képtartalomból is
    # ellenőrizzük: minden corrupt képnek a saját tiszta párja a legközelebbi testkép MSE-ben.
    test_idx = np.array([int(r["index"]) for r in test_meta], dtype=np.int64)
    clean_te = images[test_idx].reshape(len(test_idx), -1)
    corrupt_flat = corrupt.reshape(corrupt.shape[0], -1)
    nearest = ((corrupt_flat[:, None, :] - clean_te[None, :, :]) ** 2).sum(
        axis=-1).argmin(axis=1)
    corrupt_pairing_ok = bool(np.array_equal(nearest, np.arange(len(test_idx))))
    assert corrupt_pairing_ok, "a corrupt sorrend nem a metadata test-sorrendje"

    # Fájlhash-egyezés a manifesttel.
    file_hashes = manifest["sha256"]
    found_hashes = {name: sha256(DATA / name) for name in FILE_NAMES}
    hash_ok = all(found_hashes[name] == file_hashes[name] for name in FILE_NAMES)
    for name in FILE_NAMES:
        assert found_hashes[name] == file_hashes[name], f"hash-eltérés: {name}"
    manifest_ok = ((len(metadata) == 360) and hash_ok
                   and schema_ok and counts_ok)

    report = {
        "manifest_ok": manifest_ok,
        "manifest_schema_ok": schema_ok,
        "manifest_counts_ok": counts_ok,
        "scene_integrity": scene_integrity,
        "corrupt_pairing_ok": corrupt_pairing_ok,
        "split_counts": counts,
        "corrupt_pairs_by": "metadata test order (MSE-legközelebbi tiszta párral ellenőrizve)",
        "file_hashes": found_hashes,
        "external_data": [],
        "protocol_version": 1,
        "models": {"cnn": {"class": "GlyphCNN", "params": 19811},
                   "linear": {"class": "LinearBaseline", "params": 1731}},
        "seeds": [20261020, 20261021, 20261022],
        "param_budget": 20000,
    }
    (HERE / "protocol_n149.json").write_text(
        json.dumps(report, indent=2, ensure_ascii=False) + "\n",
        encoding="utf-8",
    )
    print("protocol_n149.json: minden ellenőrzés sikeres.")

if __name__ == "__main__":
    main()
```

Futtatás és ellenőrzés:

```
cd lab
python protocol_check.py
```

Ellenőrzött futás (Python 3.12.3, NumPy 2.5.4, CPU; a szkript nem használ RNG-t, ezért seed nincs): a konzol `protocol_n149.json: minden ellenőrzés sikeres.` sort írt, a jelentésben `manifest_ok`, `manifest_schema_ok`, `manifest_counts_ok`, `scene_integrity` és `corrupt_pairing_ok` egyaránt `true`, a splitdarabszámok pedig osztályonként 40/10/10 (train/val/test).

A manifest-séma a kezdőlabor `data/manifest.json` fájljából származik; a szkript a kötelező kulcsok meglétét és az előírt értékeket (NCHW, `scene_id`, 2 nézet, osztályonként 60 jelenet, [0; 1] intenzitástartomány) is ellenőrzi, és hiányzó kulcs esetén `AssertionError`-ral, a hiányzó kulcsok felsorolásával áll meg. Az ellenőrzés nem használ RNG-t és nem igényel GPU-t; minden művelet CPU-n, fájlolvasás szintjén fut. A kimeneti `protocol_n149.json` tartalmazza a fájlonkénti SHA-256 lenyomatot, a splitdarabszámokat és a corrupt-párosítás ellenőrzésének eredményét (`corrupt_pairing_ok`), tehát a projekt későbbi átadásakor az adat sértetlensége újra bizonyítható.

38.1.6 Modellköltségkeret

A költségkeret a 34.1. Mért késleltetés, throughput és memória alfejezetben tanult mérési szabályt vetíti vissza a tervezésre: a tényleges futás csak később mérhető, de a választásnak előre rögzített korlátokat kell teljesítenie.

Erőforrás	Előzetes korlát	Ellenőrzés ideje
Paraméterszám	legfeljebb 20 000	modellépítéskor, kapacitásszámlálással
Epoch	legfeljebb 30 seedenként	training ciklus
Fal-idő	legfeljebb 30 perc seedenként CPU-n	38.3. Végső teszt és páros korrupció alfejezetben mért, warmuppal (bemelegítő futással)
Inferencia batch-memória	legfeljebb 50 MB	profilerrel, batch-csúccsal
Test-hozzáférés	egyetlen értékelés befagyasztott modellen	protokoll-audit

A kiválasztott 19 811 paraméteres CNN float32 súlytárolása $19\,811 \cdot 4$ B $= 79\,244$ B $\approx 79{,}2$ KB; az AdamW két momentumával ez (paraméter + első + második momentum) körülbelül $3 \cdot 79\,244$ B $= 237\,732$ B $\approx 237{,}7$ KB *részösszeg*. A meglévő gradiensekkel már további P elem is jelen van, azaz azonos dtype mellett legalább $4P$ elem, ami $79\,244 \cdot 4$ B $= 316\,976$ B $\approx 317{,}0$ KB ilyen állapot; ezen felül aktivációk, workspace és másolatok is kellhetnek, ezért a csúcsmemória becslését külön, mérésből adjuk. Egy 32-es batch bemenete $32 \cdot 1 \cdot 24 \cdot 24 \cdot 4$ B $\approx 73{,}7$ KB. A köztes aktivációk pontos csúcsát nem becsüljük előre: a 38.3. Végső teszt és páros korrupció profiler méri. A fal-időre itt csak a felső korlátot fagyasztjuk le; a kezdőlabor ellenőrzött jelentése a saját kis CPU-futtatását rögzíti, de abból más gépre nem általánosítunk futásidőt.

A baseline és a CNN ugyanazt a 3 seedet, ugyanazt a train/val/test indexhalmazt és ugyanazt a választási szabályt kapja; az architektúra az egyetlen szándékosan változtatott tényező. Mivel a két modell paraméterszáma (1731 és 19 811) és ezzel a kapacitása eltér, az összehasonlítás a reprezentáció és a kapacitás *együttes* hatását méri; a tiszta reprezentáció-hatás izolálásához

a fent említett, kapacitásban illesztett harmadik kar kellene. Az eljárás kontrollált abban az értelemben, hogy az adat-, split-, seed- és választási feltételek azonosak, és a döntés nem utólag kiválasztott legjobb futtatás.

38.1.7 Tipikus tévedés: a tesztből visszacsatolt modellválasztás

A leggyakoribb projekt hiba az, hogy a hallgató megnézi a teszt-accuracyt, ezután módosítja az augmentációt vagy a learning rate-et, újra tanít, és az **ugyanazon** testen kapott jobb értéket jelenti. Az első tesztfutás pillanatában a teszt elveszítette held-out státuszát: az új modell kiválasztása már felhasználta a tesztet, ezért a jelentett pontosság *felfelé* torzított (optimista). A kiválasztási torzítás 22.4. Augmentációs kísérlet és domainváltozás alfejezetből ismert: több jelölt közül a legjobb mért teszthiba várhatóan kisebb (a pontosság nagyobb) a kiválasztott modell valódi hibájánál, tehát a 0,86-os jelentett pontosság nem általánosítási bizonyíték. A javítás új értékelési ciklus: a választást csak train/val végzi. A már megtekintett tesztpéldák újrasplittelése nem teszi őket újra érintetlenné; a tiszta végső ellenőrzéshez új, addig nem használt adat (új tesztjelenetek) vagy külső protokoll kell, és a régi teszt a naplóban választási adatként szerepel.

38.1.8 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a `scene_id`, `view_id` és `index` különbségét a GlyphLab metadata alapján. Magyarázza el, miért a jelenet a megfigyelési egység, és adjon egy konkrét helyzetet, amelyben a két nézet nem független. Elvárt beadandó: saját megfogalmazás, tenzoralak és a nemfüggetlenség egy példája.

F2 (levezetési, nehézség 2). A 180 jelenet 360 képét véletlenszerűen, jelenetre tekintet nélkül osztjuk 240/60/60 részre. Számolja ki egzaktul annak valószínűségét, hogy egy rögzített jelenet két nézete eltérő splitbe kerül, és adja meg a 180 jelenetre várhatóan megsértett csoportok számát. Elvárt: képlet, közbülső számok, értelmezés.

F3 (implementációs, nehézség 2). A `metadata.csv` soraiból írjon `check_scene_splits(rows)` függvényt, amely visszaad egy `scene_id -> split` leképezést, és `ValueError`-t dob, ha egy jelenet két különböző splitben jelenik meg. Bemenet a CSV `DictReader` listája; numerikus tolerancia nincs, mert a függvény a split-azonosítókat egzakt sztringegyezéssel hasonlítja össze. Elvárt: futtatható kód és két ellenőrző bemenet: egy hibátlan, egy hibás.

F4 (hibakeresési, nehézség 2). Egy hallgató a best checkpointot a **test loss** minimumánál választja, majd a testen 0,86 pontosságot jelent. Hol a hiba, mi a következménye a mérésre, és mi a minimális javítás? Elvárt: ok, következmény, javítás, visszaellenőrzés.

F5 (kísérleti/transzfer, nehézség 3). Tegyük fel, hogy a projekt új adatszerződésre vált: 900 külső jelenet, jelenetenként 3 nézet, 5 osztály, osztályarány 50/20/15/10/5%. Készítsen előzetes protokollvázlatot: megfigyelési egység, osztályrétegzett split, baseline, metrika, erőforráskorlát, valamint külső adathoz az eredet- és licencjegyzék kötelező mezői. Elvárt: indokolt terv, nem előre megadott győztes.

38.1.9 Megoldások és indoklás

F1 megoldás. A `scene_id` egy generált geometriai jelenetet azonosít; a `view_id` az adott jelenet két zajos nézetének 0/1 azonosítója; az `index` a teljes `images.npy` tömb sorindexe, tehát a kép tenzor $360 \times 1 \times 24 \times 24$ első tengelyén fut. A megfigyelési egység azért a jelenet, mert a két nézet ugyanannak a jelenetnek két zajos renderje, azonos címkével; a közös geometriai tartalom

miatt a nézetek nem független minták, és egy képszintű split esetén a tesztnézet geometriai információja részben már a tanításban szerepelne. Ezért a bootstrap és a split csoportosítási egysége is a jelenet.

F2 megoldás. A jelenet két konkrét képe azonos splitben van, ha mindkettő train, mindkettő val vagy mindkettő test. A megfelelő valószínűségek:

$$P_{\text{train}} = \frac{240}{360} \cdot \frac{239}{359}, \quad P_{\text{val}} = \frac{60}{360} \cdot \frac{59}{359}, \quad P_{\text{test}} = \frac{60}{360} \cdot \frac{59}{359}.$$

Összegük:

$$P_{\text{same}} = \frac{57360 + 3540 + 3540}{129240} = \frac{64440}{129240} \approx 0{,}4986,$$

tehát $P_{\text{diff}} = 1 - P_{\text{same}} \approx 0{,}5014$. A várhatóan megsértett jelenetek száma $180 \cdot 0{,}5014 \approx 90{,}25$. Kontroll: visszatevéses mintavétellel a határérték $P_{\text{same}} = (2/3)^2 + 2 \cdot (1/6)^2 = 4/9 + 2/36 = 0{,}5$ lenne, mert a két nézet egymástól függetlenül esne a $2/3$, illetve $1/6$ arányú részekbe; a visszatevés nélküli, egzakt hipergeometriai érték ettől kissé eltér, 0,4986, mert az első nézet kivétele után a saját splitje egy képpel kisebb. Ezért $P_{\text{diff}} = 0{,}5014 > 0{,}5$, és a várható megsértett jelenetek száma 90,25, a 90-nél kissé több. Ha minden kép egyetlen splitbe esne, $P_{\text{same}} = 1$ lenne, ami a képlet határesete.

F3 megoldás.

```python
def check_scene_splits(rows):
    scene_to_split = {}
    for row in rows:
        s = row["scene_id"]
        split = row["split"]
        old = scene_to_split.setdefault(s, split)
        if old != split:
            raise ValueError(
                f"scene {s} több splitben szerepel: {old!r} és {split!r}"
            )
    return scene_to_split
```

Ellenőrzés:

```python
ok = [
    {"scene_id": "0", "split": "train"},
    {"scene_id": "0", "split": "train"},
]
bad = [
    {"scene_id": "0", "split": "train"},
    {"scene_id": "0", "split": "test"},
]
assert check_scene_splits(ok) == {"0": "train"}
try:
    check_scene_splits(bad)
except ValueError as e:
    assert "több splitben" in str(e)
else:
    raise AssertionError("a hibás esethez ValueError kell")
```

Alakellenőrzés: a bemenet minden rekordja szöveges CSV-mezőket tartalmaz; a kimenet egy `dict`, amely 180 jelenetre pontosan 180 kulcsot ad, ha az adat érvényes.

F4 megoldás. Az ok: a checkpointot a teszt loss minimumánál választottuk ki, tehát a tesztet modellválasztásra használtuk. Következmény: a teszt többé nem held-out, mert a kiválasztott modell a teszt zajához is illeszkedik; ezért a jelentett 0,86 pontosság várhatóan optimista, és nem tekinthető független becslésnek. Minimális javítás: a kiválasztást kizárólag a validációs loss (vagy a validációs metrika) minimumára alapozzuk, a tesztet pedig a befagyasztott checkpointon egyszer értékeljük. Visszaellenőrzés: a projektnaplóban a teszt kiértékelése egyetlen, a kiválasztás után rögzített futtatás legyen, és a kiválasztási lépés ne hivatkozzon semmilyen testmetrikára.

F5 megoldásminta. A megfigyelési egység a jelenet marad; három nézet esetén minden nézet ugyanabba a splitbe kerül, és a bootstrap szintén jelenetszintű. A split osztályrétegzett: az 5%-os osztály 900 jelenetből 45 jelenetet ad, ezért 80/10/10-es aránynál a val és a test 4–5 jelenetet kap; ennél kevesebbel a val/test recall becslése túl bizonytalan lenne, hiszen 4 jelenetnél egyetlen téves jelenet már 25 százalékponttal mozdítja az osztály recallját. A kiegyensúlyozatlanság miatt a baseline a többségi osztály prior-accuracyje, a fő metrika pedig macro-recall vagy osztályonkénti recall, nem csupán overall accuracy. A modellválasztás val_loss vagy val macro-recall szerint történik; a tesztet egyszer értékeljük. Erőforrás: CPU, legfeljebb 5000 paraméter és seedenként legfeljebb 30 epoch; az 5000-es korlát indoka, hogy az 5 osztályos lineáris baseline $576 \cdot 5 + 5 = 2885$ paraméter, tehát a korlát a baseline kevesebb mint kétszeresét engedi, a GlyphCNN 19 811 paraméteres nagyságrendjét kizárja, és a CPU-s futás 30 epoch alatt tartható marad. Külső adathoz a manifest kötelező mezői: `name`, `origin_url`, `license`, `license_url`, `download_date`, `preprocessing`, `sha256`; ezek hiányában a kísérlet nem átadható. Alternatíva lehet az egy nézetre redukált alsó mérés: először view 0-n ellenőrizzük a csővezetéket, majd a teljes háromnézetes adaton futtatjuk a végső protokollt.

38.1.10 Önellenőrzés és korábbi fogalmak áttekintése

Ellenőrizze a labor után, hogy (1) minden `scene_id` mindkét nézete azonos splitben van; (2) a `protocol_n149.json` hash-ei egyeznek a manifesttel; (3) a `test` split nem szerepel semmilyen választási döntésben; (4) a baseline és a CNN ugyanazt a jelenet-splitet és seed-sémát használja. Visszatekintés: a 22.4. Augmentációs kísérlet és domainváltozás szerint a corrupt teszt csak a befagyasztott modell után értékelhető; a 26.1. Dobozkoordináta, IoU és lokalizációs loss szerint a doboz fél-nyílt xyxy, ezért a maszk befoglaló téglalapját $\texttt{xmax} - \texttt{xmin}$ és $\texttt{ymax} - \texttt{ymin}$ területtel számoljuk; a 34.1. Mért késleltetés, throughput és memória szerint a fal-időt csak warmup utáni, ismételt, tényleges mérésből szabad jelenteni.

38.1.11 Rövid összefoglaló

A Projekt II adatszerződése a GlyphLab 180 jelenetét és 360 nézetét jelenetszintű, osztályrétegzett 120/30/30 splitre bontja. A képszintű véletlen felosztás várhatóan körülbelül 90 jelenetet osztana meg; a lemmában beláttuk, hogy a jelenetszintű tesztbecslő torzítatlan a megnevezett U1 használati helyzetre, ha a modellválasztás nem érinti a tesztet. A labor manifest-ellenőrzőt és SHA-256-jegyzéket készít, baseline-ként 1731 paraméteres lineáris osztályozót rögzít, és CPU-költségkeretet fagyaszt le. A teszt egyszeri, befagyasztott kiértékeléshez tartozik; a teszt-visszacsatolás új ciklust igényel.

38.1.12 Következő lecke

A 38.2. Modell, training és best checkpoint a befagyasztott modell pontos rétegsorát, a training ciklust és a best-checkpoint választást valósítja meg az itteni protokoll alapján; a 38.3. Végső teszt és páros korrupció végzi a clean és corrupt teszt páros kiértékelését.

38.1.13 Felhasznált források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.

38.2 Modell, training és best checkpoint

38.2.1 Cél, előismeret és induló kérdés

A 38.1. Adat, split és előzetes protokoll egység befagyasztotta a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab protokollját: a megfigyelési egység a `scene_id`, a train/val/test felosztás scene-szintű, a tesztet és a `corrupted_test.npy` változatot csak a végső, választott modellen szabad kiértékelni. Ebben az egységben a *tanítási szakaszt* építjük fel. A cél egy kis CPU-s CNN tanítása, minden tréning- és validációs adat útjának naplózása, valamint egy valódi **best snapshot** mentése. A modellválasztás a validációs loss alapján történik egy előre rögzített, véges jelöltkészletből: a tanítás során keletkező epoch-snapshotok közül.

Előismeret. A 37.2. Backward és ellenőrzési lefedettség egységben a konvolúciós forward és backward levezetése, valamint véges differenciás ellenőrzése megtörtént; a jelen fejezetben az autograd által kiszámolt gradienseket már ellenőrzött operátorként kezeljük. A 34.2. Kvantálás és hibatovábbterjedés egység rögzítette, hogy a főút `float32` CPU-számítás; itt nem kvantálunk. A 26.2. Egy- és kétlépcsős detektor detektorfejezete emlékeztetett arra, hogy egy osztályozó fej és egy lokalizáló/detektor fej felelőssége különbözik: a mostani modell kimenete csak három osztálylogit, nem doboz vagy maszk.

Induló kérdés. Egy 24×24-es, egycsatornás képet az alábbi lánc dolgoz fel: 3×3-as konvolúció 8 csatornára, ReLU, 2×2-es max-pooling, 3×3-as konvolúció 16 csatornára, ReLU, 2×2-es max-pooling, majd egy 32 egységes rejtett réteg és egy 3 logitot adó kimeneti réteg. Hány tanulható skalár paramétert tartalmaz a háló? Mielőtt tovább olvas, adjon becslést; a pontos levezetés a 38.2.3. szakaszban következik.

38.2.2 A modell, a baseline és a választási fogalmak

38.2.2.1 A small CNN definíciója

A laborban használt modell a **GlyphCNN**:

- `features`: `Conv2d(1, 8, 3, padding=1)` → `ReLU` → `MaxPool2d(2)` → `Conv2d(8, 16, 3, padding=1)` → `ReLU` → `MaxPool2d(2)`;
- `classifier`: `Flatten` → `Linear(16·6·6, 32)` → `ReLU` → `Linear(32, 3)`.

A bemenet alakja $X \in \mathbb{R}^{N\times 1\times 24\times 24}$, X elemei a $[0;\ 1]$ intenzitástartományban vannak, dtype `float32`, NCHW elrendezésben ([D10]). A kimenet $Z \in \mathbb{R}^{N\times 3}$ **aktiváció előtti logit**. A célvektor $y \in \{0;\ 1;\ 2\}^N$, dtype `int64`, ahol 0 vertical, 1 horizontal, 2 diagonal.

A CE-szerződés. A modell logitot ad; a `cross_entropy` hívás előtt nem alkalmazunk külön softmaxot. A veszteség az N megfigyelés átlaga:

$$L(Z, y) = -\frac{1}{N}\sum_{n=1}^{N} \log\left(\frac{\exp(Z_{n,y_n})}{\sum_{j=0}^{K-1}\exp(Z_{n,j})}\right),$$

ahol $K = 3$. A sum-redukciós változatot külön jelölnénk; a főút az átlag.

38.2.2.2 Baseline

A 38.1. Adat, split és előzetes protokoll alfejezetben rögzített baseline a **lineáris osztályozó**: a $24 \cdot 24 = 576$ pixelt egyetlen vektorba lapítja, és $Z = W\tilde{x} + b \in \mathbb{R}^3$ logitot ad ($576 \cdot 3 + 3 = 1731$ paraméter). Ugyanazzal a recepttel, ugyanazon a három seeden és ugyanazzal a választási szabállyal tanítjuk, mint a CNN-t; az egyetlen szándékosan változtatott tényező az architektúra. Emellett külön **minimum-referenciaként** megadjuk a képfüggetlen prior-osztályozót: minden képre a train-osztálygyakoriságból becsült eloszlást adja. A GlyphLab train halmaza osztályonként 80 képet tartalmaz, tehát $\hat{p} = (1/3, 1/3, 1/3)$, a várható pontosság $1/3$, és a várható cross-entropy

$$-\frac{1}{3}\left(\log\frac{1}{3} + \log\frac{1}{3} + \log\frac{1}{3}\right) = \log 3 \approx 1{,}0986.$$

Egy tanított modell akkor hordoz tanult jelet, ha a validációs lossa ettől számottevően kisebb; a lineáris baseline-nak és a CNN-nek is ehhez kell viszonyulnia, ugyanazon a befagyasztott validációs halmazon.

38.2.2.3 Inicializálás

A tanulható súlyokat explicit **Kaiming-uniform** inicializálással állítjuk be, a biasokat nullára. A `_kaiming` segédfüggvény minden `Conv2d` és `Linear` súlyára ugyanazt a `kaiming_uniform_(a=0)` hívást alkalmazza (ReLU-gain), a biasokat nullázza. A motivációt a 21.1. Inicializálás és varianciaterjedés varianciaterjedési érvelése adja: a ReLU előtti affin kimenet varianciáját rétegről rétegre meg kell őrizni. A tényleges skála a `fan_in`-tól függ, ezért rétegenként más: a kötés az első konvolúciónál 0,816, a másodiknál 0,289, a rejtett lineáris rétegnél 0,102, a kimeneti lineáris rétegnél 0,433 (a `kaiming_uniform_` képlete szerint). Az utolsó, három logitos rétegre a ReLU-gain elméletileg nem indokolt, mert utána nincs ReLU; ott a választás tisztán **heurisztikus**. A labor a teljes receptet a futáskönyvtár `recipe.json` fájljába írja, hogy a kísérlet reprodukálható legyen.

38.2.2.4 Ütemezés

Az η_0 kezdő tanulási rátát koszinuszos ütemezéssel csökkentjük T epoch alatt:

$$\eta_t = \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2}\left(1 + \cos\frac{\pi t}{T}\right), \qquad t = 0, 1, \ldots, T-1.$$

A laborban $\eta_0 = 10^{-3}$, $\eta_{\min} = 10^{-5}$. Az optimizáló **AdamW** [P02], 10^{-4} decoupled weight decay-jel; a 6.4. Momentum és adaptív optimizálók egység rögzítette, hogy az AdamW-féle súlycsökkentés nem azonos a lossba írt L2-tag Adam alatti hatásával. Az ütemezést az epoch ciklusban kézzel írjuk az optimizáló paramétercsoportjaiba, így a labor nem függ egy adott scheduler API belső `T_cur`-definíciójától.

38.2.2.5 Snapshot és best-snapshot választás

Definíció (snapshot). A t. epoch befejezése utáni modellállapot s_t: a `state_dict`, vagyis a tanulható tensorok rendezett név–érték leképezése. A snapshot nem tartalmazza az optimizáló állapotát; a jelen kísérletben a kiválasztás után nincs folytatás.

Definíció (validation-alapú snapshot-választás, *best-snapshot selection*). Legyen $\mathcal{S} = \{s_1;\ \ldots;\ s_T\}$ az epochonként mentett, véges jelöltkészlet. Jelölje $\widehat{L}_{\mathrm{val}}(s_t)$ a t. snapshoton a teljes validációs halmazra számolt átlagos cross-entropyt. A választott snapshot indexe

$$\hat{t} = \arg\min_{t \in \{1;\ \ldots;\ T\}} \widehat{L}_{\mathrm{val}}(s_t).$$

A best-snapshot választás **nem azonos a korai leállítással** (*early stopping*): itt a tanítás végigfut a T epochon, és a választás utólag, a teljes validációs loss-sorozat ismeretében történik. A $\hat{t}$ indexre csak annyi mondható, hogy *nem feltétlenül* az utolsó epoch: a legjobb validációs állapot korábban is bekövetkezhet. A tesztet egyetlenegyszer, a befagyasztott $\hat{t}$ snapshoton értékeljük.

A $\widehat{L}_{\mathrm{val}}(s_t)$ egy véges mintán számolt becslés, ezért a választott snapshot validációs értéke optimista lehet a valódi validációs kockázatra nézve. Ezt a jelenséget a jelen egységben nem bizonyítjuk koncentrációs tétellel; a végső teszt intervallumos értelmezéséhez a 38.4. Mentett modellből független inference egység a [S03] konfidenciaintervallum-értelmezését használja (az intervallum a mintavételi eljárásra, nem az adott mintára vonatkozik).

38.2.3 Paraméterszám, tenzoralak és a CE-szerződés ellenőrzése

38.2.3.1 Paraméterszám

1. Lemma (rétegparaméterszám). Legyen egy konvolúciós réteg súlytenzora $\mathcal{W} \in \mathbb{R}^{C_{\mathrm{out}} \times (C_{\mathrm{in}}/g) \times k_h \times k_w}$ és biasvektora $b \in \mathbb{R}^{C_{\mathrm{out}}}$, ahol g a csoportok száma (a laborban $g = 1$), k_h és k_w pedig a mag magassága, illetve szélessége. A tanulható paraméterek száma

$$P_{\mathrm{conv}} = C_{\mathrm{out}} \left(\frac{C_{\mathrm{in}}}{g} k_h k_w + 1 \right).$$

Affin rétegnél $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, tehát $P_{\mathrm{aff}} = md + m$.

Bizonyítás. A súlytenzor elemszáma a dimenziók szorzata: $C_{\mathrm{out}} \cdot (C_{\mathrm{in}}/g) \cdot k_h \cdot k_w$. A bias minden kimeneti csatornához egy-egy skalárt ad, ami C_{out} további elem. Az affin esetben W elemszáma md, a bias m. Az állítás a tenzoralak definíciójából következik; nincs szükség közelítésre. □

Alkalmazás a GlyphCNN-re.

- Conv1: $8(1 \cdot 3 \cdot 3 + 1) = 8 \cdot 10 = 80$.
- Conv2: $16(8 \cdot 3 \cdot 3 + 1) = 16 \cdot 73 = 1168$.
- Linear1: $32 \cdot 576 + 32 = 18464$.
- Linear2: $3 \cdot 32 + 3 = 99$.
- Összesen: $80 + 1168 + 18464 + 99 = 19811$.

2. Lemma (alakzártság a rétegláncon). A fenti GlyphCNN 24×24-es bemenetre az 1. táblázat alakjait állítja elő.

Művelet	Kimeneti alak	Paraméterszám
Conv1, $k=3$, $p=1$	$(N, 8, 24, 24)$	80
MaxPool2d(2)	$(N, 8, 12, 12)$	0
Conv2, $k=3$, $p=1$	$(N, 16, 12, 12)$	1168
MaxPool2d(2)	$(N, 16, 6, 6)$	0
Flatten	$(N, 576)$	0
Linear1	$(N, 32)$	18464
Linear2	$(N, 3)$	99

Bizonyítás. Használjuk a 14.1. Stride, padding, dilation és outputméret kimenetméret-képletét egy térbeli dimenzióra, általános s stride, p padding és d dilation mellett:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - d(k_h - 1) - 1}{s} \right\rfloor + 1.$$

(Itt H_{in} a bemeneti magasság; a bemeneti szélességet W_{in} jelöli. A W betűt a továbbiakban csak az affin súlymátrixra használjuk, a konvolúciós súlytenzor jele $\mathcal{W}$; a táblázatban $k = k_h = k_w$ a négyzetes mag mérete.) Conv1: $s = d = 1$, $k = 3$, $p = 1$, így $(24 + 2 - 3) + 1 = 24$. A MaxPool $s = k = 2$: $\lfloor (24-2)/2 \rfloor + 1 = 12$. Conv2: $(12 + 2 - 3) + 1 = 12$. A második pool: $\lfloor (12-2)/2 \rfloor + 1 = 6$. A flatten méret $16 \cdot 6 \cdot 6 = 576$. A lineáris rétegek a megadott be- és kimeneti dimenziókat valósítják meg. A paraméterszám az 1. lemma alkalmazása. □

A kis modell számításigénye is ellenőrizhető: Conv1 forwardja $24 \cdot 24 \cdot 8 \cdot 1 \cdot 9 = 41\,472$ MAC, Conv2-é $12 \cdot 12 \cdot 16 \cdot 8 \cdot 9 = 165\,888$ MAC, a két lineáris réteg $18\,432 + 96 = 18\,528$ MAC. A domináns költség a második konvolúció, nem a teljesen kapcsolt rész.

38.2.3.2 A CE-szerződés és a gradiens

3. Lemma (mean-redukált CE-gradiens). Legyen $Z \in \mathbb{R}^{N \times K}$ logitmátrix és $y \in \{0;\ \dots;\ K-1\}^N$. Definiáljuk

$$p_{n,k} = \frac{\exp(Z_{n,k})}{\sum_{j=0}^{K-1} \exp(Z_{n,j})}, \qquad L(Z, y) = -\frac{1}{N} \sum_{n=1}^{N} \log p_{n,y_n}.$$

Ekkor minden n, k párra

$$\frac{\partial L}{\partial Z_{n,k}} = \frac{1}{N} \left(p_{n,k} - \mathbf{1}_{k=y_n} \right).$$

A PyTorch `F.cross_entropy(Z, y)` a fenti L értéket számolja, stabil log-sum-exp implementációval; a modell kimenetének tehát logitnak kell lennie, és a cél `int64` osztályindex.

Bizonyítás. Rögzítsünk egy n sort, és röviden írjunk Z_k-t $Z_{n,k}$ helyett, p_k-t $p_{n,k}$ helyett. Legyen $S = \sum_j \exp(Z_j)$. Ekkor $\log p_y = Z_y - \log S$, tehát

$$\frac{\partial}{\partial Z_k} \log p_y = \mathbf{1}_{k=y} - \frac{1}{S} \frac{\partial S}{\partial Z_k} = \mathbf{1}_{k=y} - \frac{\exp(Z_k)}{S} = \mathbf{1}_{k=y} - p_k.$$

A negatív előjel és az $\frac{1}{N}$ átlagolás bevitelével

$$\frac{\partial L}{\partial Z_{n,k}} = -\frac{1}{N}\left(\mathbf{1}_{k=y_n} - p_{n,k}\right) = \frac{1}{N}\left(p_{n,k} - \mathbf{1}_{k=y_n}\right).$$

Az eredmény alakja $N \times K$, megegyezik Z alakjával, ahogy a skalár loss tenzorgradiensétől elvárjuk. □

A lemma egyben megmagyarázza, miért szokták a hibajelet „softmax mínusz one-hot" alakban emlegetni: a batch-átlag osztója N, nem a K osztályszám.

Dupla softmax. Ha a modell kimenetére tévesen előre softmaxot teszünk, és az eredményt adjuk logitként a `cross_entropy`-nak, a hatékony veszteség $L = -\log \mathrm{softmax}(\mathrm{softmax}(z))_y$. Az elsőrendű gradiens a láncszabály miatt egy további első derivált-faktort, a softmax Jacobi-mátrixát tartalmazza: $\nabla_z L = J_{\mathrm{softmax}}(z)^\top(\mathrm{softmax}(\mathrm{softmax}(z)) - e_y)$, ahol $J_{\mathrm{softmax}}(z) = \mathrm{diag}(p) - pp^\top$, $p = \mathrm{softmax}(z)$. Ez *nem* második derivált: Hessianról csak akkor beszélünk, ha ezt a gradienst még egyszer differenciáljuk. A J_{softmax} szimmetrikus, $J_{\mathrm{softmax}}\mathbf{1} = 0$, és minden $\mathbf{1}$-re merőleges v-re $v^\top J_{\mathrm{softmax}} v = \sum_k p_k v_k^2 - (p^\top v)^2 \le p_{\max}\|v\|^2$, ahol $p_{\max} = \max_k p_k$. Sajátértékei tehát a $[0;\ p_{\max}]$ intervallumban vannak, és mivel a külső loss gradiense, $\mathrm{softmax}(\mathrm{softmax}(z)) - e_y$, összege nullával egyenlő, a dupla softmax gradiense legfeljebb $p_{\max}$-szorosára csökken a helyes $p - e_y$ gradienshez képest. Ez lassú tanulásként jelentkezik.

38.2.4 Eredeti kézi kis példa

CE-számolás. Legyen $N = 1$, $K = 3$, $Z = (2;\ 1;\ 0{,}5)$ és $y = 0$. Az exponenciális értékek $\exp(2) \approx 7{,}3891$, $\exp(1) \approx 2{,}7183$, $\exp(0{,}5) \approx 1{,}6487$, az összeg $S \approx 11{,}7561$. A softmax:

$$p \approx \left(\frac{7{,}3891}{11{,}7561}, \frac{2{,}7183}{11{,}7561}, \frac{1{,}6487}{11{,}7561}\right) \approx (0{,}6285;\ 0{,}2312;\ 0{,}1402).$$

A loss $L = -\log(0{,}6285) \approx 0{,}4644$. A 3. lemma szerint a gradiens

$$\nabla_Z L = p - (1;\ 0;\ 0) \approx (-0{,}3715;\ 0{,}2312;\ 0{,}1402).$$

Ellenőrzés a szerződésen. A loss skalár, a gradiens alakja $(1;\ 3)$, és a gradiens elemeinek összege $-0{,}3715 + 0{,}2312 + 0{,}1402 = 0$, ami a softmax-kimenet és a one-hot közötti különbség elvárt nulla összegű tulajdonsága. Ha valaki a modell végére tett volna softmaxot, akkor a `F.cross_entropy` még egy log-softmaxot alkalmazna; a láncszabály szerint ekkor a gradiensbe bekerülne a softmax Jacobi-faktora is (elsőrendű tag), ezért a gradiens a helyes $p - e_y$-hoz képest csillapodna. Ez gyakori CE-szerződéssértés; másodrendű tag ilyenkor sem jelenik meg.

Ugyanez PyTorchban numerikusan ellenőrizhető; a cella a rögzített környezetben (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál) futtatva `L = 0.4644`-et és `[-0.3715, 0.2312, 0.1402]` gradienst adott, négy tizedesre kerekítve:

```
import torch
from torch.nn import functional as F
Z = torch.tensor([[2.0, 1.0, 0.5]], dtype=torch.float32, requires_grad=True)
y = torch.tensor([0], dtype=torch.int64)
L = F.cross_entropy(Z, y)
L.backward()
# mért: L = 0.4644, Z.grad = [-0.3715, 0.2312, 0.1402] (négy tizedesre)
```

38.2.5 Labor: training, napló és best snapshot

Munkakönyvtár: a laborcsomag `lab/` könyvtára, benne a `data/` a GlyphLab fájljaival. A labor három fájlból áll. A `lab/glyph_models.py` a projekt egyetlen modellforrása: ezt importálja a tanítás (38.2. Modell, training és best checkpoint, ez az egység), a végső teszt (38.3. Végső teszt és páros korrupció) és a csomagolás (38.4. Mentett modellből független inference), így a modellosztály neve és rétegszerkezete nem térhet el a lépések között. A `lab/train_small_cnn.py` egy (architektúra, seed) futást végez; csak a train és val splitet tölti be, a teszt és a korrupt teszt nem szerepel benne. A `lab/run_project2.py` a 38.1. Adat, split és előzetes protokoll protokollját hajtja végre egyetlen paranccsal.

```
# lab/glyph_models.py - a Projekt II egyetlen modellforrása (38.2-38.4. alfejezet)
import torch
from torch import nn


def configure_cpu_threads():
    """Egyszálas CPU-számítást állít be az aktuális folyamatra.

    A lebegőpontos összegzési sorrend így nem függ a gép magszámától; bitazonos
    ismétléshez azonos PyTorch-/NumPy-verzió, backend, determinisztikus
    algoritmusok és szálszám kell. Importáláskor nem nyúlunk a globális
    állapothoz: a belépési pontok hívják meg ezt a függvényt.
    """
    torch.set_num_threads(1)


# A 38.1. alfejezet protokolljában előre rögzített paraméterszámok és a közös keret.
ARCH_PARAMS = {"cnn": 19811, "linear": 1731}
PARAM_BUDGET = 20000


def _kaiming(module):
    for m in module.modules():
        if isinstance(m, (nn.Conv2d, nn.Linear)):
            nn.init.kaiming_uniform_(m.weight, a=0.0)
            if m.bias is not None:
                nn.init.zeros_(m.bias)


class GlyphCNN(nn.Module):
    """Kis CNN: (N,1,24,24) float32 -> (N,3) logit."""
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(16 * 6 * 6, 32), nn.ReLU(), nn.Linear(32, 3)
        )
        _kaiming(self)

    def forward(self, x):
        return self.classifier(self.features(x))


class LinearBaseline(nn.Module):
    """Lineáris baseline: flatten 576 -> 3 logit, ugyanazzal a CE-szerződéssel."""
    def __init__(self):
        super().__init__()
```

```
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(24 * 24, 3))
        _kaiming(self)

    def forward(self, x):
        return self.classifier(x)


def build(arch):
    return {"cnn": GlyphCNN, "linear": LinearBaseline}[arch]()


def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def load_checkpoint(path, map_location="cpu"):
    """A 38.2. alfejezet checkpointsémája: model_state_dict, arch, seed, epoch, val_loss,
    ↪ params."""
    ckpt = torch.load(path, map_location=map_location, weights_only=True)
    model = build(ckpt["arch"])
    model.load_state_dict(ckpt["model_state_dict"])     # strict: minden kulcs egyezik
    model.eval()
    return model, ckpt
```

A checkpoint sémája. Minden futás `runs/<arch>_s<seed>/best.pt` fájlja egy szótár: `model_state_dict` (a súlyok), `arch` (`cnn` vagy `linear`), `seed`, `epoch`, `val_loss`, `params` és `recipe`. A `recipe` értéke a futáskönyvtár `recipe.json` fájljában tárolt teljes recept (architektúra, seed, epochszám, η_0, $\eta_{\min}$, weight decay, batchméret, optimalizáló, ütemezés, loss, inicializálás) kanonikus JSON-jának 12 jegyű SHA-256-azonosítója. A betöltés mindig a `load_checkpoint` függvényen át, `weights_only=True` módon történik ([D09]), és a `load_state_dict` alapértelmezett szigorú kulcsegyezését használja; ugyanezt a sémát olvassa a 38.3. Végső teszt és páros korrupció is, kézi átnevezés nélkül.

```
# lab/train_small_cnn.py - 38.2. alfejezet: egy (architektúra, seed) futás, napló, best
↪ checkpoint.
import argparse, csv, hashlib, json, math, os, time
import numpy as np
import torch
from torch.nn import functional as F
from torch.utils.data import DataLoader, TensorDataset

from glyph_models import build, configure_cpu_threads, count_parameters, load_checkpoint


def cosine_lr(t, T, lr0, eta_min):
    return eta_min + 0.5 * (lr0 - eta_min) * (1.0 + math.cos(math.pi * t / T))


def split_ids(root, split):
    with open(os.path.join(root, "data", "metadata.csv"), newline="") as f:
        rows = list(csv.DictReader(f))
    idx = np.array([int(r["index"]) for r in rows], dtype=np.int64)
    assert np.array_equal(idx, np.arange(len(rows))), \
        "a metadata index oszlopa nem a fájlsorok 0..N-1 indexe"
    split_col = np.array([r["split"] for r in rows])
    assert set(np.unique(split_col)) <= {"train", "val", "test"}, set(np.unique(split_col))
    return idx[split_col == split]


def load_split(root, split):
    """Csak train/val: a teszt és a korrupt teszt ebben a fájlban nem nyílik meg."""
    assert split in ("train", "val")
```

```
    ids = split_ids(root, split)
    assert len(ids) > 0 and len(np.unique(ids)) == len(ids)
    X_all = np.load(os.path.join(root, "data", "images.npy"))
    Y_all = np.load(os.path.join(root, "data", "labels.npy"))
    assert X_all.ndim == 4 and X_all.shape[1:] == (1, 24, 24), X_all.shape
    assert Y_all.shape == (X_all.shape[0],) and Y_all.dtype == np.int64
    assert set(np.unique(Y_all)) <= {0, 1, 2}
    assert 0 <= ids.min() and ids.max() < X_all.shape[0]
    X = X_all[ids].astype(np.float32)
    Y = Y_all[ids].astype(np.int64)
    assert np.isfinite(X).all() and 0.0 <= X.min() and X.max() <= 1.0
    return TensorDataset(torch.from_numpy(X), torch.from_numpy(Y))


@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    total, correct, n, logits_all = 0.0, 0, 0, []
    for xb, yb in loader:
        logits = model(xb)
        total += F.cross_entropy(logits, yb).item() * xb.shape[0]
        correct += (logits.argmax(dim=1) == yb).sum().item()
        n += xb.shape[0]
        logits_all.append(logits)
    return total / n, correct / n, torch.cat(logits_all)


def make_recipe(arch, seed, epochs, lr, eta_min, wdecay, batch_size):
    return {"arch": arch, "seed": seed, "epochs": epochs, "lr0": lr, "eta_min": eta_min,
            "weight_decay": wdecay, "batch_size": batch_size, "optimizer": "AdamW",
            "schedule": "cosine_epoch_start", "loss": "cross_entropy_mean",
            "init": "kaiming_uniform_a0"}


def train(arch="cnn", seed=20261020, epochs=24, lr=1e-3, wdecay=1e-4,
          batch_size=32, root="."):
    configure_cpu_threads()
    torch.manual_seed(seed)
    recipe = make_recipe(arch, seed, epochs, lr, 1e-5, wdecay, batch_size)
    recipe_id = hashlib.sha256(
        json.dumps(recipe, sort_keys=True, ensure_ascii=False).encode("utf-8")
    ).hexdigest()[:12]
    tr_loader = DataLoader(load_split(root, "train"), batch_size=batch_size, shuffle=True,
                           generator=torch.Generator().manual_seed(seed))
    va_loader = DataLoader(load_split(root, "val"), batch_size=batch_size, shuffle=False)

    model = build(arch)                     # az init a seedtől függ
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=wdecay)
    run_dir = os.path.join(root, "runs", f"{arch}_s{seed}")
    os.makedirs(run_dir, exist_ok=True)
    recipe_path = os.path.join(run_dir, "recipe.json")
    if os.path.exists(recipe_path):         # azonos recept újrafuttatható, más nem keverhető
        with open(recipe_path, encoding="utf-8") as f:
            old = json.load(f)
        if old != recipe:
            raise RuntimeError(f"a {run_dir} más recepttel készült; válassz új könyvtárat")
    with open(recipe_path, "w", encoding="utf-8") as f:
        json.dump(recipe, f, indent=2, sort_keys=True)
    best = {"val_loss": float("inf")}
    with open(os.path.join(run_dir, "metrics.jsonl"), "w", encoding="utf-8") as log:
        for epoch in range(epochs):
            t0 = time.perf_counter()
            lr_t = cosine_lr(epoch, epochs, lr, 1e-5)      # az epoch elején érvényes ráta
```

```
            for pg in opt.param_groups:
                pg["lr"] = lr_t
            model.train()
            tr_total, tr_n = 0.0, 0
            for xb, yb in tr_loader:
                opt.zero_grad(set_to_none=True)
                loss = F.cross_entropy(model(xb), yb)
                loss.backward()
                opt.step()
                tr_total += loss.item() * xb.shape[0]
                tr_n += xb.shape[0]
            va_loss, va_acc, va_logits = evaluate(model, va_loader)
            is_best = va_loss < best["val_loss"]
            if is_best:
                best = {"epoch": epoch + 1, "val_loss": va_loss, "val_acc": va_acc,
                        "logits": va_logits.clone()}
                torch.save({"model_state_dict": model.state_dict(), "arch": arch,
                            "seed": seed, "epoch": epoch + 1, "val_loss": va_loss,
                            "params": count_parameters(model), "recipe": recipe_id},
                           os.path.join(run_dir, "best.pt"))
            log.write(json.dumps({"epoch": epoch + 1, "train_loss": tr_total / tr_n,
                                  "val_loss": va_loss, "val_acc": va_acc, "lr": lr_t,
                                  "best": is_best, "recipe": recipe_id, "seed": seed,
                                  "seconds": time.perf_counter() - t0}) + "\n")

    # Visszatöltési kontroll: a mentett best.pt ugyanazokat a val-logitokat adja.
    reloaded, ckpt = load_checkpoint(os.path.join(run_dir, "best.pt"))
    re_loss, _, re_logits = evaluate(reloaded, va_loader)
    max_diff = float((re_logits - best["logits"]).abs().max())
    assert max_diff <= 1e-6 and abs(re_loss - best["val_loss"]) <= 1e-6, (max_diff, re_loss)
    summary = {"arch": arch, "seed": seed, "recipe": recipe_id, "best_epoch": best["epoch"],
               "best_val_loss": best["val_loss"], "best_val_acc": best["val_acc"],
               "params": count_parameters(model), "reload_max_abs_logit_diff": max_diff}
    with open(os.path.join(run_dir, "summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    return summary


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--arch", choices=["cnn", "linear"], default="cnn")
    ap.add_argument("--seed", type=int, default=20261020)
    ap.add_argument("--epochs", type=int, default=24)
    ap.add_argument("--lr", type=float, default=1e-3)
    ap.add_argument("--wdecay", type=float, default=1e-4)
    ap.add_argument("--batch-size", type=int, default=32)
    ap.add_argument("--root", default=".")
    a = ap.parse_args()
    print(json.dumps(train(a.arch, a.seed, a.epochs, a.lr, a.wdecay, a.batch_size, a.root)))
```

A koszinuszos rátát minden epoch **elején** állítjuk be, így az első epoch η_0-lal fut, és a naplóban az az érték szerepel, amellyel az adott epoch ténylegesen tanult. A futás végén a szkript visszatölti a mentett `best.pt`-t, és összeveti a validációs logitokat a mentés pillanatában számoltakkal; eltérés esetén `AssertionError`-ral megáll.

```
# lab/run_project2.py - 38.2. alfejezet: a teljes tanítási szakasz egy paranccsal.
# Két architektúra × három seed, paraméterszám-ellenőrzés, validációs összevetés,
# és a protokoll választási szabálya szerinti checkpointok a 38.3. alfejezet számára.
import json, math, os, shutil, statistics
from glyph_models import ARCH_PARAMS, PARAM_BUDGET
from train_small_cnn import train

SEEDS = (20261020, 20261021, 20261022)      # a 38.1. alfejezet seedlistája
```

```
def main(root="."):
    runs = {arch: [train(arch, s, root=root) for s in SEEDS] for arch in ("linear", "cnn")}
    report = {"seeds": list(SEEDS), "prior_val_loss": math.log(3), "prior_val_acc": 1 / 3,
              "archs": {}}
    os.makedirs(os.path.join(root, "checkpoints"), exist_ok=True)
    for arch, rs in runs.items():
        assert all(r["params"] == ARCH_PARAMS[arch] <= PARAM_BUDGET for r in rs), arch
        chosen = min(rs, key=lambda r: r["best_val_loss"])     # választás csak val alapján
        shutil.copyfile(os.path.join(root, "runs", f"{arch}_s{chosen['seed']}", "best.pt"),
                        os.path.join(root, "checkpoints", f"best_val_{arch}.pt"))
        accs = [r["best_val_acc"] for r in rs]
        losses = [r["best_val_loss"] for r in rs]
        report["archs"][arch] = {
            "params": rs[0]["params"], "runs": rs, "chosen_seed": chosen["seed"],
            "val_acc_mean": statistics.mean(accs), "val_acc_sd": statistics.stdev(accs),
            "val_loss_mean": statistics.mean(losses), "val_loss_sd": statistics.stdev(losses)}
    with open(os.path.join(root, "runs", "project2_val_report.json"), "w") as f:
        json.dump(report, f, indent=2)
    print("arch    params  seed      best_ep  val_loss  val_acc")
    for arch, rs in runs.items():
        for r in rs:
            print(f"{arch:7s} {r['params']:6d}  {r['seed']}  {r['best_epoch']:3d}"
                  f"      {r['best_val_loss']:.4f}    {r['best_val_acc']:.3f}")
    for arch, a in report["archs"].items():
        print(f"{arch}: val_acc {a['val_acc_mean']:.3f} ± {a['val_acc_sd']:.3f}, "
              f"választott seed {a['chosen_seed']} -> checkpoints/best_val_{arch}.pt")


if __name__ == "__main__":
    main()
```

Futtatás egy paranccsal:

```
cd lab
python run_project2.py
```

Egyetlen futás külön is indítható, például `python train_small_cnn.py --arch cnn --seed 20261020`. A kimenet: hat futáskönyvtár (`runs/linear_s<seed>`, `runs/cnn_s<seed>`, mindegyikben `recipe.json`, `metrics.jsonl`, `best.pt`, `summary.json`), a `runs/project2_val_report.json` összevetés és a `checkpoints/best_val_cnn.pt`, `checkpoints/best_val_linear.pt` választott checkpointok. A `recipe.json` a teljes receptet rögzíti; ha ugyanabba a futáskönyvtárba más recepttel indul a szkript, a `train()` `RuntimeError`-ral leáll, így két recept naplója nem keveredhet.

Protokoll. A seed az inicializálásra és a train-shuffle generátorára vonatkozik. A [D02] dokumentáció szerint a seed önmagában nem ígér platformok és verziók közötti bitazonosságot; ezért a `metrics.jsonl` minden sora a recept 12 jegyű azonosítóját (`recipe`) és a `seed`-et is rögzíti, és az eredményeket csak az adott környezet reprodukciójához kötjük. A `glyph_models.configure_cpu_threads()` a tanítás kezdetén egyszálas CPU-számítást állít be (`torch.set_num_threads(1)`); a függvény importáláskor nem nyúl a globális állapothoz. Többszálas futásnál a lebegőpontos összegzés sorrendje a gép magszámától függ, és az utolsó jegyek gépenként eltérhetnek.

Futási jegyzőkönyv. A `python run_project2.py` az eredeti GlyphLab-adaton, CPU-n (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, 1 szál) körülbelül 3,7 másodperc alatt lefutott; két egymás utáni mérés fali ideje egyaránt 3,74 s volt. A két futás eredménye azonos: a hat `summary.json` és a két választott checkpoint SHA-256 értéke egyezett, a `metrics.jsonl` pedig csak a `seconds` mezőben tért el (a `recipe` és `seed` mezők is egyeztek):

```
arch    params  seed      best_ep  val_loss  val_acc
linear    1731  20261020    1      1.1155    0.267
linear    1731  20261021    1      1.1031    0.300
linear    1731  20261022    1      1.1192    0.350
cnn      19811  20261020   24      0.3029    0.900
cnn      19811  20261021   15      0.3683    0.900
cnn      19811  20261022   19      0.2326    0.917
linear: val_acc 0.306 ± 0.042, választott seed 20261021 -> checkpoints/best_val_linear.pt
cnn: val_acc 0.906 ± 0.010, választott seed 20261022 -> checkpoints/best_val_cnn.pt
```

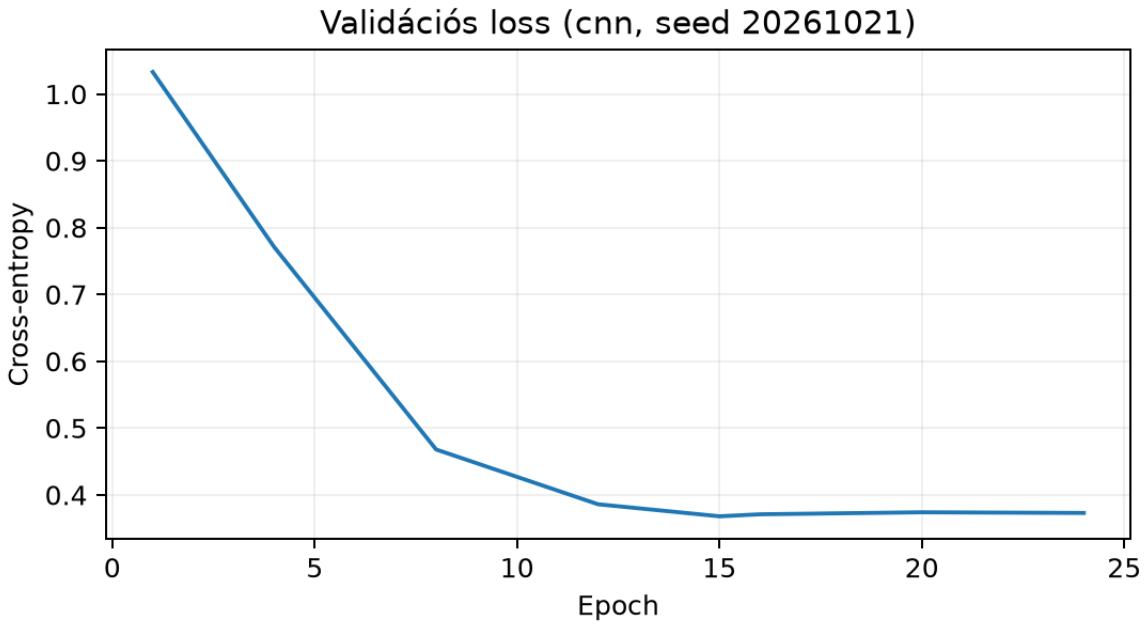
A paraméterszám-ellenőrzés mindkét modellre teljesült (1731 és 19 811, mindkettő a 20 000-es keret alatt), és mind a hat futásban a visszatöltött `best.pt` logitjai a kód 10^{-6}-os tűrésén belül egyeztek a mentéskoriakkal; a mért `reload_max_abs_logit_diff` mind a hat futásban 0 volt. A lineáris baseline validációs lossa mindhárom seeden a prior $\log 3 \approx 1{,}0986$ értéke *fölött* marad, és a legjobb snapshotja az első epoch: a pixelvektor lineáris függvénye ezen a feladaton nem ad általánosítható jelet. Ennek lehetséges magyarázata — amelyet ez az egyetlen kísérlet nem izolál —, hogy a vonal helye jelenetenként változik, és az orientáció a pixelintenzitások nemlineáris függvénye; a mérés ténye a rossz általánosítás, az oké hipotézis. A CNN mindhárom seeden 0,9 körüli validációs pontosságot ér el. Más verziójú könyvtárakkal vagy más CPU-n az utolsó jegyek eltérhetnek; a saját `project2_val_report.json` a leadandó.

A `cnn_s20261021` futás naplójának kivonata mutatja a best és a last snapshot különbségét:

```
epoch train_loss val_loss val_acc lr       best
1     0.990      1.033    0.567   1.00e-03 yes
4     0.475      0.771    0.767   9.62e-04 yes
8     0.121      0.468    0.900   8.06e-04 yes
12    0.043      0.386    0.900   5.70e-04 no
15    0.029      0.368    0.900   3.77e-04 yes
16    0.027      0.371    0.900   3.16e-04 no
20    0.022      0.374    0.900   1.12e-04 no
24    0.021      0.373    0.900   1.42e-05 no
best_epoch=15 best_val_loss=0.3683
```

A kivonat nem folytonos: a 2–3., 5–7., 9–11., 13–14., 17–19. és 21–23. epoch sorait, valamint a `seconds` oszlopot kihagytuk. A `best` oszlop a teljes napló tényleges értéke; például a 13. epoch is `best` volt (0,3719), és a 15. epoch javította 0,3683-ra. A kihagyott sorok ismeretében a 12. sor `no`, illetve a 15. sor `yes` értéke nem ellentmondás.

A train loss tovább csökken, a validációs loss viszont a 15. epoch minimuma után már nem javul: a `best.pt` a 15. epoch állapota, nem az utolsóé. Ugyanezen a futáson a validációs pontosság a 8. epochtól változatlan, tehát a választást a loss, nem az accuracy dönti el.



38.1. ábra: Validációs loss (cnn, seed 20261021)

Mért futás: Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261021.

38.2.6 Feladatok

F1 (fogalmi, nehézség 1). Előfeltétel: a 38.2.2.5. alszakasz (Snapshot és best-snapshot választás). Fogalmazza meg saját szavaival, mi a különbség a *legutolsó snapshot* és a *validation-alapú best snapshot* között, és adjon egy minimális, három epochra vonatkozó `val_loss` sorozatot, ahol a kettő nem esik egybe. Elvárt eredmény: definíció és a három elemű számsor.

F2 (levezetési/bizonyítási, nehézség 2). Előfeltétel: a 38.2.3. szakasz lemmái. Legyen

$$Z = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 2 & 1 \end{pmatrix}, \qquad y = (0;\ 2).$$

Számítsa ki kézzel a mean-redukált cross-entropyt és a 2×3-as gradiensmátrixot. Ellenőrizze az alakot, és azt, hogy minden sor gradiensösszege nulla. Elvárt eredmény: a loss közelítő értéke, a gradiensmátrix, valamint a 3. lemma alkalmazásának jelölése.

F3 (implementációs, nehézség 2). Előfeltétel: a labor script lefutott. Írjon egy `select_best` függvényt, amely a `runs/<arch>_s<seed>/metrics.jsonl` soraiból kiválasztja a legkisebb `val_loss` sort, betölti a `best.pt` fájlt `weights_only=True` módon, és a validációs halmazon reprodukálja a loss értékét 10^{-4} tűréssel. Elvárt eredmény: futtatható kód, `best_epoch` és a reprodukált `val_loss`.

F4 (hibakeresési, nehézség 1). Előfeltétel: a labor naplója. Egy hallgató a `best.pt` mentését így írta:

```
if epoch + 1 == epochs:
    torch.save({"model_state_dict": model.state_dict(), "arch": arch, "seed": seed,
                "epoch": epoch + 1, "val_loss": va_loss},
               os.path.join(run_dir, "best.pt"))
```

A tanítási szkript a `summary.json`-ba a `metrics.jsonl` legjobb sora alapján `best_epoch=15`-öt ír. Hol a hiba, mi a következménye a zárótesztre, és hogyan ellenőrzi a javítást? Elvárt eredmény: ok, következmény, javított feltétel és a betöltött modell loss-ellenőrzése.

F5 (kísérleti/transzfer, nehézség 3). Előfeltétel: a 38.1. Adat, split és előzetes protokoll protokollterve. Tegyük fel, hogy egy új munkacsoport két kis architektúrát, A-t és B-t akar kiválasztani a GlyphLab validációs lossa alapján, három seeddel. Írjon előzetes protokollt: megfigyelési egység, split, baseline, változó tényező, döntési metrika, külön futásnaplók és a tesztkiértékelés időpontja. Nevezze meg azt a statisztikai korlátot, amely a validációval végzett architektúraválasztás után a tesztmetrikát terheli. Elvárt eredmény: egy bekezdésnyi protokoll és a korlát megnevezése indoklással.

38.2.7 Megoldások és indoklás

38.2.7.1 F1 megoldása

A **legutolsó snapshot** a tanítási szerződésben rögzített T. epoch utáni állapot, minden további válogatás nélkül. A **best snapshot** ezzel szemben a $\mathcal{S} = \{s_1;\ \dots;\ s_T\}$ véges jelöltkészlet azon eleme, amelyre a teljes validációs halmazon számolt átlagos cross-entropy minimális. A kettő akkor és csak akkor esik egybe, ha $\hat{t} = T$. Példa: $\widehat{L}_{\text{val}} = (0{,}50;\ 0{,}45;\ 0{,}48)$. Ekkor $\hat{t} = 2$, a legutolsó snapshot a 3. epochhoz tartozik, és rosszabb validációs értéket hordoz.

38.2.7.2 F2 megoldása

Első sor: $\exp(1) = 2{,}7183$, $\exp(0) = 1$, $\exp(-1) = 0{,}3679$, összegük $4{,}0862$. A softmax $p_1 = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$, a cél $y_1 = 0$, így a sor lossza $-\log(0{,}6652) = 0{,}4076$. Második sor: $\exp(0) = 1$, $\exp(2) = 7{,}3891$, $\exp(1) = 2{,}7183$, összegük $11{,}1074$. A softmax $p_2 = (0{,}0900;\ 0{,}6652;\ 0{,}2447)$, a cél $y_2 = 2$, a sor lossza $-\log(0{,}2447) = 1{,}4076$. Az átlag

$$L = \frac{0{,}4076 + 1{,}4076}{2} = 0{,}9076.$$

A gradiens a 3. lemma szerint soronként $(p_n - \mathbf{1}_{y_n})/N$, $N = 2$:

$$\nabla_Z L = \frac{1}{2}\begin{pmatrix} -0{,}3348 & 0{,}2447 & 0{,}0900 \\ 0{,}0900 & 0{,}6652 & -0{,}7553 \end{pmatrix} \approx \begin{pmatrix} -0{,}1674 & 0{,}1224 & 0{,}0450 \\ 0{,}0450 & 0{,}3326 & -0{,}3776 \end{pmatrix}.$$

Alakellenőrzés: 2×3, megegyezik Z alakjával. Mindkét sor összege: $-0{,}1674 + 0{,}1224 + 0{,}0450 = 0$ és $0{,}0450 + 0{,}3326 - 0{,}3776 = 0$, a kerekítési hibán belül.

38.2.7.3 F3 megoldása

```
import json, math, os
from torch.utils.data import DataLoader
from glyph_models import load_checkpoint
from train_small_cnn import evaluate, load_split

def select_best(run_dir, root=".", batch_size=32, tol=1e-4):
    with open(os.path.join(run_dir, "metrics.jsonl"), encoding="utf-8") as f:
        rows = [json.loads(line) for line in f]
    best = min(rows, key=lambda r: r["val_loss"])
    model, ckpt = load_checkpoint(os.path.join(run_dir, "best.pt"))  # weights_only=True
```

```
    assert ckpt["epoch"] == best["epoch"], (ckpt["epoch"], best["epoch"])
    va_loader = DataLoader(load_split(root, "val"), batch_size=batch_size, shuffle=False)
    val_loss, val_acc, _ = evaluate(model, va_loader)
    assert math.isclose(val_loss, best["val_loss"], abs_tol=tol), (val_loss, best["val_loss"])
    return model, best["epoch"], val_loss, val_acc

print(select_best("runs/cnn_s20261021")[1:])
```

A labor futása után a `runs/cnn_s20261021` könyvtáron a függvény a 15. epochot és a 0,3683-as validációs losst adja vissza. A tűrés 10^{-4}: a mentett érték ugyanabból a determinisztikus előreszámításból származik, és a validációs halmazt ugyanazzal a batch-felosztással építjük fel, ezért a gyakorlatban az eltérés nulla; a tűrés más verziójú könyvtárak összegzési sorrendjére hagy helyet. A teszt lényegi része nem az `assert`, hanem a reprodukált `val_loss` érték és az, hogy a checkpoint `epoch` mezője a napló legjobb sorára mutat.

38.2.7.4 F4 megoldása

A hiba oka: a `best.pt` mentési feltétele az utolsó epochhoz kötött, nem a legjobb validációs értékhez. Következmény: a fájl a T. epoch állapotát tartalmazza, miközben a napló szerint a 15. epoch vesztesége kisebb; a záróteszt a túlillesztettebb utolsó modellt kapná. Javítás (a labor szkriptjének mintájára):

```
best = {"epoch": 0, "val_loss": float("inf")}  # a ciklus előtt inicializálva
# ...
if va_loss < best["val_loss"]:
    best = {"epoch": epoch + 1, "val_loss": va_loss}
    torch.save({"model_state_dict": model.state_dict(), "arch": arch, "seed": seed,
                "epoch": epoch + 1, "val_loss": va_loss},
               os.path.join(run_dir, "best.pt"))
```

Ellenőrzés: a javított futás `summary.json` `best_epoch` értéke megegyezik a `metrics.jsonl` legkisebb `val_loss` sorával és a checkpoint `epoch` mezőjével, és a betöltött `best.pt` validációs lossát a `best_val_loss` értékkel 10^{-4} tűrésen belül reprodukáljuk. Ha a tűrés nem teljesül, akkor a mentés nem a kiértékelt állapothoz tartozik.

38.2.7.5 F5 megoldásminta

A protokoll a jelen egységben nem hajtja végre a receptkeresést, mert a 38.1. Adat, split és előzetes protokoll a befagyasztott receptet rögzíti; a következő terv tehát transzferfeladat. Megfigyelési egység: `scene_id`; a train/val/test felosztás változatlan. A két architektúra (A: a fenti 8–16 csatornás lánc, B: 16–32 csatornás változat) mindegyikét három seeddel, külön `runs/<arch>_s<seed>` naplóban tanítjuk. Döntési metrika: a teljes $\mathcal{S} = \{s_1;\ \ldots;\ s_{24}\}$ snapshot-készlet validációs lossa seedenként, azaz összesen 24×3 snapshot architektúránként; a kiválasztott modell az az architektúra–seed–snapshot hármas, amelynek a legkisebb a validációs lossa. A baseline az uniform prior-osztályozó. A tesztet csak a választás után, egyetlen alkalommal értékeljük. Korlát: mivel a validációt modellválasztásra is használtuk, a nyertes modell tesztmetrikája optimista lehet a valódi kockázathoz képest; a [S03] szerint az intervallumok a mintavételi eljárásra vonatkozó állítások, ezért a választási lépést mint az eljárás részét kell kezelni, nem mint előre ismert modellt. A tanítási recept a jelen egység AdamW-receptje ($\eta_0 = 10^{-3}$, $\eta_{\min} = 10^{-5}$, weight decay 10^{-4}, batch 32, 24 epoch), a seedek 20261020–20261022, az erőforrás CPU, két architektúra és három seed.

38.2.8 Tipikus tévedés ellenpéldával, önellenőrzés és visszatekintés

Tipikus tévedés. „Az utolsó epoch a legjobban tanult modell.” Ellenpélda a fenti mért `cnn_s20261021` futás: az utolsó epoch `val_loss=0.373`, a legjobb 15. epoch `val_loss=0.368`. Ha a train loss az epochok végén 0,021, az csak a train halmazhoz való illeszkedést mutatja; a validációs loss enyhe emelkedése a 15. epoch minimuma után a túlillesztés egy jele lehet, de egyetlen futásból ez nem azonosítható bizonyítékként. A zárótesztet ezért a best snapshoton kell futtatni, nem az utolsó epoch állapotán. A tévedés következménye, hogy a záróteszt nem a kiválasztási szabály szerinti modellt kapja; a romlás mértékét itt nem mérjük, és a corrupt tesztre vonatkozó hatást sem állítjuk előre — azt a 38.3. Végső teszt és páros korrupció méri.

Önellenőrzés.

1. Mennyi a GlyphCNN paraméterszáma, és melyik réteg adja a legtöbbet?
2. A modell kimenete logit vagy valószínűség? Hol történik a softmax a CE-szerződésben?
3. Melyik split alapján választunk snapshotot, és melyik splitet tilos a választás előtt megnyitni?
4. A `best.pt` betöltéséhez miért kell újraépíteni az architektúrát, és melyik modul adja az egyetlen modelldefiníciót?

Kulcs. (1) A GlyphCNN paraméterszáma 19 811; a legtöbbet a Linear1 réteg adja (18 464). (2) A kimenet logit; a softmax a `cross_entropy` CE-veszteségében, numerikusan stabil log-sum-exp alakban történik. (3) A snapshotot a validációs loss alapján választjuk; a teszt splitet a választás előtt tilos megnyitni. (4) A `best.pt` csak a `state_dict`-et tartalmazza, ezért az architektúrát újra kell építeni; az egyetlen modelldefiníció a `lab/glyph_models.py`.

Visszatekintés. A 37.2. Backward és ellenőrzési lefedettség gradiensellenőrzése biztosítja, hogy a Conv2d tanulási jele a levezetett backwarddal egyezik; a jelen egység ezt az operátort már nem ellenőrzi újra, hanem feltételezi. A 34.2. Kvantálás és hibatovábbterjedés figyelmeztetett, hogy a kisebb bitszélességű reprezentáció további hibát vinne be; a mostani labor ezért `float32` marad. A 26.2. Egy- és kétlépcsős detektor detektor-felelősségmegosztása azt hangsúlyozta, hogy az osztályozó logit nem dobozregresszió: a GlyphLab `xmin, ymin, xmax, ymax` oszlopait ebben a kísérletben nem használjuk, azok a szegmentálós/lokalizációs projekt annotációi.

38.2.9 Rövid összefoglaló

A 38.2. Modell, training és best checkpoint egység a befagyasztott GlyphLab-protokoll tanítási szakaszát valósította meg. Definiáltuk a kis CNN-t, a lineáris baseline-t és a prior minimum-referenciát, a Kaiming-inicializálást, a koszinuszos tanulásiráta-ütemezést, a snapshotot és a validation-alapú best-snapshot választást. Levezettük a paraméterszámot és a rétegenkénti alakokat, valamint a mean-redukált cross-entropy gradiensét, amely softmax mínusz one-hot alakú. A labor egyetlen paranccsal mindkét modellt mindhárom seeddel tanítja, ellenőrzi a paraméterszámot, futásonként JSON-naplót és visszatöltéssel ellenőrzött `best.pt` checkpointot ír, majd a protokoll választási szabálya szerint előállítja a 38.3. Végső teszt és páros korrupció bemenetét; a közölt napló mért futásból származik. A következő egység a befagyasztott modell végső tesztjét és a páros corrupt kiértékelést készíti elő.

38.2.10 Felhasznált források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. A seed szerepének és korlátainak rögzítéséhez. Hozzáférés: 2026-10-30.
- [D09] PyTorch contributors (2026). *Saving and Loading Models*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. A `state_dict`, `eval` és `weights_only` checkpointkezeléshez. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. A keresztkorreláció, NCHW/OIHW, stride/padding/groups és kimeneti alak értelmezéséhez. Hozzáférés: 2026-10-30.
- [P02] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019. Előzetes változat: arXiv:1711.05101 (2017). https://arxiv.org/abs/1711.05101. A decoupled weight decay és az AdamW optimalizáló forrása. Hozzáférés: 2026-10-30.
- [S03] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC. A konfidenciaintervallum mintavételi eljárásra vonatkozó, feltételes értelmezéséhez.

38.3 Végső teszt és páros korrupció

38.3.1 Cél, előismeret és induló kérdés

A 38.2. Modell, training és best checkpoint egység a validation loss alapján választotta ki a legjobb checkpointot, és rögzítette a modellarchitektúrát, az inicializálást és a tanítási protokollt. Itt ezt a **befagyasztott** modellt egyszer, a zárt tesztkészleten értékeljük. A modell a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab háromosztályos orientációfeladatán tanult: 0 = vertical, 1 = horizontal, 2 = diagonal. A teszt 30 független jelenetből áll, jelenetenként 2 zajos nézettel, tehát 60 kép; osztályonként 10 jelenet.

Előismeret: a 38.2. Modell, training és best checkpoint checkpoint formátuma, a 37.3. Linearitás, adjungált és határfeltétel egységből a modell determinisztikus kiértékelési szerződése, és a 34.3. Pruning és struktúra egységből az a szabály, hogy a betöltött `state_dict` kulcsainak pontosan egyezniük kell a deklarált architektúrával. A 26.3. NMS, confidence és AP egységben látott posztprocesszt itt nem használjuk, mert osztályozásnál az argmax a teljes kiértékelés.

Induló kérdés: ha a korrupt teszt ugyanazokat a jeleneteket tartalmazza, mint a clean teszt, miért hibás a két 60 képes mintát függetlenként kezelni?

38.3.2 Definíciók és a kiértékelési szerződés

Jelölje a teszt jeleneteit $s = 1, \dots, S$, ahol $S = 30$. Minden jelenetnek $V = 2$ nézete van; a v-edik nézet képe $X_{s,v} \in \mathbb{R}^{1\times 24\times 24}$, float32, $[0;\ 1]$. A jelenet osztálycímkéje $c_s \in \{0;\ 1;\ 2\}$; mindkét nézet ehhez a címkéhez tartozik. A befagyasztott modell $f_\theta \colon \mathbb{R}^{1\times 24\times 24} \to \mathbb{R}^3$ logitokat ad, és a predikció

$$\hat{y}_{s,v} = \arg\max_{k\in\{0;\ 1;\ 2\}} f_\theta(X_{s,v})_k.$$

A modell kiértékelés előtt `eval()` módban van, és a következtetés `torch.no_grad()` alatt fut, tehát a kimenet determinisztikus a paraméterezés szerint; nincs dropout- vagy BatchNorm-training véletlenség (a 37.3. Linearitás, adjungált és határfeltétel szerződésének megfelelően).

1. definíció (per-scene accuracy). A s jelenetre számított accuracy

$$a_s = \frac{1}{2}\sum_{v=0}^{1} \mathbf{1}[\hat{y}_{s,v} = c_s] \;\in\; \{0;\ \tfrac{1}{2};\ 1\}.$$

2. definíció (clean accuracy, group average, macro). A clean teszt per-image accuracyje

$$\mathrm{Acc}_{img} = \frac{1}{30}\sum_{s=1}^{30} a_s = \frac{1}{60}\sum_{s=1}^{30}\sum_{v=0}^{1} \mathbf{1}[\hat{y}_{s,v} = c_s].$$

A **group average** a jelenet-accuracyk átlaga, $\overline{\mathrm{acc}}_{scene} = \frac{1}{30}\sum_s a_s$. A k osztályra vett accuracy

$$\mathrm{Acc}_k = \frac{1}{20}\sum_{s:c_s=k}\sum_{v=0}^{1} \mathbf{1}[\hat{y}_{s,v} = k],$$

és a **macro accuracy** $\mathrm{Acc}_{macro} = \frac{1}{3}\sum_{k=0}^{2}\mathrm{Acc}_k$. A group average és a per-image accuracy itt egyenlő, mert minden jelenetnek pontosan két nézete van; ezt alább azonosságként levezetjük. A macro ezen a teszten **pontosan egyenlő** a per-image accuracyvel: mivel minden osztályhoz $n_k = 20$ kép tartozik, $\mathrm{Acc}_{macro} = \frac{1}{3}\sum_k \frac{c_k}{20} = \frac{\sum_k c_k}{60} = \mathrm{Acc}_{img}$, ahol c_k a k osztály helyes predikcióinak száma. A két összesítő csak kiegyensúlyozatlan osztálygyakoriságnál térhet el. Egy esetlegesen gyengébb osztályt ezért itt nem a macro, hanem az osztályonkénti recall és a konfúziós mátrix mutatna meg; a labor ezeket külön kiírja.

3. definíció (páros korrupciós különbség). A `corrupted_test.npy` a metadata tesztsorainak sorrendjében tartalmazza ugyanazon jelenet–nézet párok zajos és fényességgradienssel rontott változatát, $\tilde{X}_{s,v}$-t. A korrupt jelenet-accuracy $\tilde{a}_s = \frac{1}{2}\sum_v \mathbf{1}[\arg\max_k f_\theta(\tilde{X}_{s,v})_k = c_s]$, a páros különbség

$$d_s = a_s - \tilde{a}_s \;\in\; \{-1;\ -\tfrac{1}{2};\ 0;\ \tfrac{1}{2};\ 1\}, \qquad \bar{d} = \frac{1}{30}\sum_{s=1}^{30} d_s.$$

A pozitív $\bar{d}$ azt jelenti, hogy a korrupció átlagosan rontja a jelenetszintű pontosságot; a párosság a jelenet nehézségét közös zajként tünteti el.

4. definíció (stratified scene bootstrap). Legyen $I_k = \{s : c_s = k\}$ a k osztályú jelenetek indexhalmaza, $|I_k| = 10$. Adott B replikációhoz és α-hoz:

1. $b = 1, \dots, B$ esetén minden $k \in \{0;\ 1;\ 2\}$ osztályból húzzunk **visszatevéssel** $|I_k| = 10$ jelenetindexet az I_k halmazból; a három osztály húzásait összefűzve kapjuk az $I^{(b)}$ bootstrap-mintát.
2. Számítsuk ki a replikált statisztikát: $\bar{d}^{(b)} = \frac{1}{30}\sum_{s\in I^{(b)}} d_s$.
3. A percentilis intervallum $[q_{\alpha/2};\ q_{1-\alpha/2}]$, ahol q_p a $\{\bar{d}^{(1)};\ \dots;\ \bar{d}^{(B)}\}$ p-kvantilise.

Ez az intervallum **feltételes** a megfigyelt tesztadatra: azt közelíti, hogyan változna $\bar{d}$, ha a teszt jeleneteit az osztályon belül újrahúznánk. Nem bizonyítunk általános bootstrap-konzisztenciatételt; a könyv szintjén ez heurisztikus újramintavételi eljárás, amelynek feltételes, ismételt mintavételi értelmezését magára a bootstrap-eljárásra rögzítjük ([S03]), paraméteres intervallumképlet nélkül. A rétegzés biztosítja, hogy a bootstrap-mintában is osztályonként 10 jelenet marad.

38.3.3 Két elemi azonosság

Azonosság 1. Ha minden jelenetnek pontosan V nézete van, akkor a group average megegyezik a per-image accuracyvel.

Levezetés. Legyen $S = 30$, $V = 2$. Ekkor

$$\overline{\mathrm{acc}}_{scene} = \frac{1}{S}\sum_{s=1}^{S}\frac{1}{V}\sum_{v=1}^{V}\mathbf{1}[\hat{y}_{s,v} = c_s] = \frac{1}{SV}\sum_{s=1}^{S}\sum_{v=1}^{V}\mathbf{1}[\hat{y}_{s,v} = c_s] = \mathrm{Acc}_{img}.$$

A második egyenlőségnél a konstans $\frac{1}{V}$ kiemelhető, a harmadik az Acc_{img} definíciója, mert az összes kép száma SV. Ha a jelenetek nézetszáma eltérne, az azonosság általában nem áll fenn; ez az F1 feladat ellenpéldája.

Következmény 2. A stratified scene bootstrap minden replikációban osztályonként pontosan $|I_k|$ jelenetet tartalmaz, és a bootstrap-minta várható összetétele az eredeti osztályarányokat tükrözi.

Indoklás. Az algoritmus definíció szerint minden k-ra külön, visszatevéses húzást végez a rögzített I_k halmazból, pontosan $|I_k|$ alkalommal. Ezért az $I^{(b)}$-beli k osztályú elemek száma minden b-re $|I_k|$. A várható osztályarány triviálisan $|I_k|/\sum_j |I_j|$, mert a húzásszámok rögzítettek.

A bootstrap becslés várható értékének centráltságát az F2 feladat a kézi példa **egzakt** eloszlásán igazolja; szimulált B mellett ez csak közelítő ellenőrzés lehet.

38.3.4 Kézi kis példa: négy jelenet és egzakt bootstrap

A teljes labor előtt egy 4 jelenetből, 2 osztályból álló példán kézzel számolunk. Az osztályok: $\{s_1;\ s_2\}$ a 0, $\{s_3;\ s_4\}$ az 1 osztály. A két nézetre adott predikciók:

jelenet s	osztály c_s	a_s	$\tilde{a}_s$	d_s
s_1	0	1	0,5	0,5
s_2	0	0,5	0	0,5
s_3	1	1	0,5	0,5
s_4	1	0,5	1	−0,5

A clean per-image accuracy: $(1 + 0{,}5 + 1 + 0{,}5)/4 = 0{,}75$; a group average ugyanaz, mert $V = 2$. A macro: osztályonként $(1 + 0{,}5)/2 = 0{,}75$ és $(1 + 0{,}5)/2 = 0{,}75$, tehát $\mathrm{Acc}_{macro} = 0{,}75$. A korrupt group average $(0{,}5 + 0 + 0{,}5 + 1)/4 = 0{,}5$, és $\bar{d} = 0{,}75 - 0{,}5 = 0{,}25$; a páros átlag $(0{,}5 + 0{,}5 + 0{,}5 - 0{,}5)/4 = 0{,}25$, egyezik, ahogy az Azonosság 1 garantálja.

A stratified bootstrap egzakt eloszlása ezen a kis példán kiszámítható. A 0 osztály mindkét húzása a $\{0{,}5;\ 0{,}5\}$ halmazból mindig $0{,}5 + 0{,}5 = 1{,}0$ összeget ad. Az 1 osztályban a $\{0{,}5;\ -0{,}5\}$ halmazból visszatevéssel húzott két elem összege 1,0, 0,0 vagy −1,0, rendre $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$ valószínűséggel. A négyelemű bootstrap-minta átlaga ezért

$$\bar{d}^{(b)} = \frac{1{,}0 + (\text{1. osztály összege})}{4},$$

amelynek eloszlása: $P(\bar{d}^{(b)} = 0{,}5) = \frac{1}{4}$, $P(\bar{d}^{(b)} = 0{,}25) = \frac{1}{2}$, $P(\bar{d}^{(b)} = 0) = \frac{1}{4}$. A 95%-os percentilis intervallum nagy B esetén $[0;\ 0{,}5]$. Ez a kézi eredmény az F3 feladat futtatható ellenőrzése: a

bootstrap-függvénynek ezen a 4 jelenetes, 2 osztályos bemeneten $B = 2000$-rel megközelítőleg a [0; 0,5] intervallumot kell visszaadnia. A labor `eval_test.py` ezzel szemben a 30 jelenetes teszten fut, ezért más intervallumot ad; a kézi példa nem a labor kimenetének előrejelzése.

38.3.5 Labor: befagyasztott modell, clean teszt és páros korrupció

Munkakönyvtár: `lab/`. A bemenet a 38.2. Modell, training és best checkpoint `python run_project2.py` futásának két választott checkpointja: `checkpoints/best_val_cnn.pt` és `checkpoints/best_val_linear.pt`. Mindkettő a 38.2. Modell, training és best checkpoint checkpointsémáját követi (`model_state_dict`, `arch`, `seed`, `epoch`, `val_loss`, `params`, `recipe`), és a szkript ugyanazzal a `glyph_models.load_checkpoint` függvénnyel tölti be őket, amellyel a 38.2. Modell, training és best checkpoint a mentést ellenőrizte: `weights_only=True` módon, szigorú kulcsegyezéssel, kézi átnevezés nélkül ([D09]). A 38.1. Adat, split és előzetes protokoll fő kérdése miatt a lineáris baseline is egyszer, ugyanazon a teszten értékelődik.

```
# lab/eval_test.py - 38.3. alfejezet: egyszeri teszt a befagyasztott checkpointokon.
import json, os
import numpy as np
import pandas as pd
import torch

from glyph_models import load_checkpoint

SEED = 20261020
B = 2000
ALPHA = 0.05
torch.manual_seed(SEED)
os.makedirs("results", exist_ok=True)     # a kimeneti könyvtár minden mentés előtt létezik

# --- adat: NCHW float32, int64 címkék; a korrupt teszt a metadata test-sorrendjében
X_np = np.load("data/images.npy")
labels = np.load("data/labels.npy")
X_corrupt_np = np.load("data/corrupted_test.npy")
assert X_np.dtype == np.float32 and labels.dtype == np.int64 \
    and X_corrupt_np.dtype == np.float32
assert X_np.ndim == 4 and X_np.shape[1:] == (1, 24, 24) and np.isfinite(X_np).all()
assert 0.0 <= X_np.min() and X_np.max() <= 1.0
assert X_corrupt_np.ndim == 4 and X_corrupt_np.shape[1:] == (1, 24, 24)
assert np.isfinite(X_corrupt_np).all()
X = torch.from_numpy(X_np).to(torch.float32)
X_corrupt = torch.from_numpy(X_corrupt_np).to(torch.float32)
meta = pd.read_csv("data/metadata.csv")
assert (meta["index"].to_numpy() == np.arange(len(meta))).all(), \
    "a metadata indexe nem a 0..N-1 tartomány permutációja"
test_idx = meta.loc[meta["split"] == "test", "index"].to_numpy().astype(np.int64)  # (60,)
assert len(test_idx) == 60 and len(np.unique(test_idx)) == 60
assert X_corrupt.shape[0] == len(test_idx)
test_meta = meta.iloc[test_idx].reset_index(drop=True)
y_test = labels[test_idx]                                          # (60,)
# A corrupt teszt a metadata test-sorrendjét követi; MSE-legközelebbi párral ellenőrizve.
_clean = X[torch.as_tensor(test_idx)].reshape(len(test_idx), -1)
_corr = X_corrupt.reshape(X_corrupt.shape[0], -1)
_pairs = ((_corr[:, None, :] - _clean[None, :, :]) ** 2).sum(-1).argmin(1)
assert torch.equal(_pairs, torch.arange(len(test_idx))), "a corrupt sorrend nem a test-sorrend"

# --- a 38.2. alfejezet kimenete kézi átnevezés nélkül: ugyanaz a séma és ugyanaz a modellmodul
cnn, cnn_ckpt = load_checkpoint("checkpoints/best_val_cnn.pt")
lin, lin_ckpt = load_checkpoint("checkpoints/best_val_linear.pt")

@torch.no_grad()
```

```
def predict(net, batch):
    net.eval()                          # a dropout/BatchNorm eval-viselkedése a mérés feltétele
    return torch.argmax(net(batch), dim=1).numpy().astype(np.int64)  # logit -> argmax

def per_scene_accuracy(pred, y, rows_meta):
    """Jelenetenkénti accuracy; a scene_id és az osztály ugyanabban a sorrendben."""
    groups = rows_meta.groupby("scene_id", sort=True).indices        # scene_id -> sorok
    scene_ids = list(groups)
    acc = np.array([np.mean(pred[r] == y[r]) for r in groups.values()], dtype=np.float64)
    class_ids = np.array([y[r][0] for r in groups.values()], dtype=np.int64)
    assert all(len(set(y[r])) == 1 for r in groups.values())       # egy jelenet, egy címke
    return scene_ids, acc, class_ids

# kis kontroll: két jelenet, két-két nézet -> hossz 2
_m = pd.DataFrame({"scene_id": ["b", "a", "b", "a"]})
_s, _a, _c = per_scene_accuracy(np.array([0, 1, 1, 1]), np.array([0, 1, 0, 1]), _m)
assert _s == ["a", "b"] and _a.tolist() == [1.0, 0.5] and _c.tolist() == [1, 0]

def scene_bootstrap_ci(d, class_ids, B, alpha, seed):
    rng = np.random.default_rng(seed)
    pools = [np.where(class_ids == k)[0] for k in np.unique(class_ids)]
    reps = np.array([d[np.concatenate([rng.choice(p, size=len(p), replace=True)
                                       for p in pools])].mean() for _ in range(B)])
    lo, hi = np.quantile(reps, [alpha / 2, 1 - alpha / 2])
    return float(lo), float(hi)

report = {"checkpoints": {"cnn": {k: cnn_ckpt[k] for k in ("seed", "epoch", "val_loss",
↪  "params")},
                          "linear": {k: lin_ckpt[k] for k in ("seed", "epoch", "val_loss",
                          ↪  "params")}}}
scene = {}
class_ids_by = {}
for name, net in (("cnn", cnn), ("linear", lin)):
    p_clean, p_corrupt = predict(net, X[test_idx]), predict(net, X_corrupt)
    assert p_clean.shape == p_corrupt.shape == (60,)
    assert p_clean.dtype == np.int64 and p_corrupt.dtype == np.int64
    sids, a_clean, class_ids = per_scene_accuracy(p_clean, y_test, test_meta)
    _, a_corrupt, class_ids_corrupt = per_scene_accuracy(p_corrupt, y_test, test_meta)
    assert np.array_equal(class_ids, class_ids_corrupt)   # ugyanaz a jelenetsorrend
    class_ids_by[name] = class_ids
    assert a_clean.shape == (30,) and set(np.unique(a_clean)) <= {0.0, 0.5, 1.0}
    d = a_clean - a_corrupt
    assert np.isclose(d.mean(), np.mean(p_clean == y_test) - np.mean(p_corrupt == y_test))
    scene[name] = a_clean
    report[name] = {
        "clean_img": float(np.mean(p_clean == y_test)),
        "corrupt_img": float(np.mean(p_corrupt == y_test)),
        "macro": float(np.mean([np.mean(p_clean[y_test == k] == k) for k in range(3)])),
        "paired_mean": float(d.mean()),
        "paired_ci": scene_bootstrap_ci(d, class_ids, B, ALPHA, SEED),
        "scene_ids": sids, "a_clean": a_clean.tolist(), "a_corrupt": a_corrupt.tolist()}
    # kiegyensúlyozott teszt (20-20-20): a macro pontosan a per-image accuracy
    assert np.bincount(y_test).tolist() == [20, 20, 20]
    assert abs(report[name]["macro"] - report[name]["clean_img"]) < 1e-12
    report[name]["confusion_clean"] = [[int(np.sum((y_test == a) & (p_clean == b))) for b in
↪  range(3)]
                                       for a in range(3)]          # sor: valódi, oszlop:
                                       ↪  predikált
    if name == "cnn":
        np.save("results/n151_pred_clean.npy", p_clean)
        np.save("results/n151_pred_corrupt.npy", p_corrupt)
        with torch.no_grad():                    # a nyers logitok is a riport részei
            np.save("results/n151_logits_clean.npy",
↪  net(X[test_idx]).numpy().astype(np.float32))
```

```
        np.save("results/n151_test_index.npy", test_idx.astype(np.int64))
        err = np.where(p_clean != y_test)[0]
        # az első mező a metadata globális indexe (0..359), nem a teszten belüli 0..59 sorszám
        report["cnn"]["failures"] = [[int(test_idx[j]), str(test_meta.scene_id[j]),
                                      int(test_meta.view_id[j]), int(y_test[j]),
                                      ↪  int(p_clean[j])]
                                     for j in err[:5]]

# a 38.1. alfejezet fő kérdése: CNN clean accuracy vs. lineáris baseline, jelenetszintű páros CI
assert np.array_equal(class_ids_by["cnn"], class_ids_by["linear"])
class_ids = class_ids_by["cnn"]          # a ciklus utolsó értéke helyett explicit forrás
diff = scene["cnn"] - scene["linear"]
report["cnn_minus_linear"] = {"mean": float(diff.mean()),
                              "ci": scene_bootstrap_ci(diff, class_ids, B, ALPHA, SEED)}
with open("results/n151_report.json", "w") as f:
    json.dump(report, f, indent=2)
for name in ("cnn", "linear"):
    r = report[name]
    print(f"{name}: clean {r['clean_img']:.3f}  corrupt {r['corrupt_img']:.3f}  "
          f"macro {r['macro']:.3f}  clean-corrupt {r['paired_mean']:.3f} "
          f"CI [{r['paired_ci'][0]:.3f}, {r['paired_ci'][1]:.3f}]")
c = report["cnn_minus_linear"]
print(f"cnn - linear (clean): {c['mean']:.3f} CI [{c['ci'][0]:.3f}, {c['ci'][1]:.3f}]")
print("cnn failures (metadata index, scene, view, truth, pred):", report["cnn"]["failures"])
print("cnn confusion (clean):", report["cnn"]["confusion_clean"])
```

A `per_scene_accuracy` a jelenetek számával egyező hosszú tömböt ad vissza, és vele együtt, ugyanabban a sorrendben, a `scene_id`-ket és a jelenetek osztályát. A szkript eleji kis kontroll (két jelenet, két-két nézet) ezt ellenőrzi; a főúton a kimenet `(30,)` alakú, minden értéke a $\{0;\ 0{,}5;\ 1\}$ halmazban van, és a d átlaga egyezik a per-image clean és corrupt accuracy különbségével (ez az Azonosság 1 következménye, a kód `assert`-tel ellenőrzi). A `metadata.csv` `index` oszlopa a **teljes** képtömb (0…359) sorindexe, nem a spliten belüli sorszám; ezért a `test_idx` közvetlenül indexeli az `images.npy` tömböt, a `failures` lista első mezője pedig ezt a globális indexet tartalmazza (a teszten belüli 0–59 sorszám helyett). A kód a betöltés után ellenőrzi az `images.npy`, `labels.npy` és `corrupted_test.npy` dtype-ját, alakját és végességét, valamint azt, hogy a `X_corrupt` első dimenziója a teszt hosszával egyezik; a korrupt sorrendet a MSE-legközelebbi pár azonosítja a clean teszthez.

A futás parancsa a `lab/` könyvtárból: `python eval_test.py`. **Futási jegyzőkönyv** (az eredeti GlyphLab-adat, a 38.2. Modell, training és best checkpoint fenti futásának checkpointjai, CPU, Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, PyTorch 2.15.0+cpu, 1 szál, bootstrap-seed 20261020; két futás azonos kimenetet és azonos `.npy` hash-t adott):

```
cnn: clean 1.000  corrupt 0.733  macro 1.000  clean-corrupt 0.267 CI [0.167, 0.383]
linear: clean 0.217  corrupt 0.250  macro 0.217  clean-corrupt -0.033 CI [-0.133, 0.067]
cnn - linear (clean): 0.783 CI [0.667, 0.900]
cnn failures (row, scene, view, truth, pred): []
cnn confusion (clean): [[20, 0, 0], [0, 20, 0], [0, 0, 20]]
```

Az eredményértelmezés négy rétege: (1) a CNN clean accuracyje a befagyasztott modell egyszeri, választás utáni tesztje, nem új modellválasztási kritérium; itt mind a 60 tesztkép helyes, ami a 30 jelenetes, szintetikus teszten nem jelent hibátlan általánosítást. (2) A macro itt definíció szerint egyenlő a clean accuracyvel (kiegyensúlyozott osztályok); az osztályonkénti képet a konfúziós mátrix adja, amely a CNN-nél diagonális (minden osztály 20/20). (3) A páros bootstrap intervallum a korrupció hatásának jelenetszintű bizonytalanságát adja: a $[0{,}167;\ 0{,}383]$ intervallum nem tartalmazza a nullát, tehát a megfigyelt 30 jeleneten a korrupció konzisztensen ront; ez feltételes állítás a 30 jelenetre, nem általánosít a GlyphLabon túlra. (4) A 38.1. Adat, split és előzetes protokoll fő kérdésére a CNN és a lineáris baseline jelenetenkénti clean accuracyjének

páros különbsége felel: 0,783, intervalluma [0,667; 0,900]. A lineáris baseline clean accuracyje 0,217, ami a kiegyensúlyozott háromosztályos véletlen szint ($1/3 \approx 0{,}333$) alatt van; durva binomiális közelítéssel a pontosságbecslés null-eloszlásának szórása ($p = 1/3$, $n = 60$) kb. 0,061, tehát ez mintegy 1,9 szórással marad el a véletlen szinttől. Ez a 60 képen nem erős bizonyíték, de a véletlennél rosszabb minta a döntési felület szisztematikus hibájára utalhat; a modell nem „a véletlen szint körül" teljesít. A korrupció ezen az amúgy is gyenge modellen érdemben nem változtat: a páros különbség $-0{,}033$, intervalluma $[-0{,}133;\ 0{,}067]$, amely tartalmazza a nullát.

38.3.6 Gyakori tévedés: a korrupt teszt nem független minta

A korrupt változat ugyanazon 30 jelenet és ugyanazon 60 nézet zajos változata. Aki a clean és korrupt 60 képet két független mintaként kezeli, és külön image-szintű bootstrapet vagy kétmintás tesztet futtat, az a jelenetek közös nehézségét figyelmen kívül hagyja. A helyes egység a jelenet, és a helyes összehasonlítás a d_s páros különbség.

Ellenpélda az unpaired elemzésre. Vegyünk négy jelenetet az alábbi clean és korrupt jelenet-accuracykkal:

jelenet	a_s	$\tilde{a}_s$	d_s
1	1	0,5	0,5
2	1	0,5	0,5
3	0,5	0	0,5
4	0,5	0	0,5

A páros d_s minden jelenetre ugyanaz a 0,5, ezért a páros bootstrap eloszlása degenerált: minden replikáció átlaga 0,5, az intervallum $[0{,}5;\ 0{,}5]$, amely nem tartalmazza a nullát. Ha viszont valaki a két feltételt függetlennek tekinti, és a négy clean meg négy korrupt jelenet-accuracyt két független mintaként hasonlítja össze, akkor a clean minta $(1;\ 1;\ 0{,}5;\ 0{,}5)$ és a korrupt $(0{,}5;\ 0{,}5;\ 0;\ 0)$ **populációs** szórása 0,25, a Welch-formulában használt **mintaszórás** (ddof=1) pedig $\sqrt{0{,}25/3} \approx 0{,}2887$. Innen a standard hiba $SE = \sqrt{2 \cdot 0{,}0833/4} \approx 0{,}2041$, és a Welch-féle kétmintás 95%-os t-intervallum a 0,5 átlagkülönbségre ($df = 6$, $t \approx 2{,}447$) $[0{,}0005;\ 0{,}9995]$. Ez szűken ugyan, de kizárja a nullát, viszont lényegesen szélesebb a páros $[0{,}5;\ 0{,}5]$ intervallumnál; a konkrét számokra tehát az állítás az, hogy az unpaired intervallum **nem** tartalmazza a nullát, csupán sokkal bizonytalanabb képet ad. A páros tervezés fontossága ettől még igaz. A tévedés következménye: egy valójában minden jeleneten azonos, konzisztens romlást nem ismerünk fel, mert a jelenetek eltérő alapnehézségét az unpaired eljárás zajként, nem pedig közös hatásként kezeli. A páros összehasonlítás ezért nem stilisztikai választás, hanem a GlyphLab adatszerződéséből következő követelmény.

38.3.7 Feladatok

F1 (fogalmi, nehézség 1). Definiálja külön a per-image és a per-scene átlagos accuracyt, és adjon olyan 2 jelenetes példát, ahol az egyik jelenetnek 2, a másiknak 3 nézete van, és a két mennyiség nem egyenlő. *Beadandó:* definíció, a példa táblázata, mindkét érték kiszámítva, és egy mondat arról, mikor egyenlők.

F2 (levezetési, nehézség 2). A kézi példa 4 jelenetére és a stratified bootstrap egzakt eloszlására igazolja, hogy a bootstrap-átlag várható értéke egyenlő a megfigyelt $\bar{d} = 0{,}25$-tel. *Beadandó:* a $\{\bar{d}^{(b)}\}$ eloszlás felsorolása, a várható érték levezetése, és annak ellenőrzése, hogy a percentilis intervallum $[0;\ 0{,}5]$ tényleg a definíció szerinti 95%-os intervallum.

F3 (implementációs, nehézség 2). Írjon `stratified_scene_bootstrap(d, class_ids, B, alpha, seed)` függvényt, amely a fenti 4 jelenetes, 2 osztályos kézi példára (és általában tetszőleges osztályszámra) visszaadja a percentilis intervallumot; a függvény ne importáljon külső statisztikai csomagot, csak NumPy-t. Ellenőrizze a kézi példán $B = 2000$, `seed=20261020` paraméterrel; a visszaadott intervallum közelítse a $[0;\ 0{,}5]$ értéket $\pm 0{,}1$ toleranciával. *Beadandó:* futtatható kód, seed, az ellenőrző hívás és a kapott intervallum.

F4 (hibakeresési, nehézség 2). Az alábbi kódrészlet a clean–korrupt különbség konfidenciaintervallumát akarja számolni, de unpaired módon:

```
def unpaired_diff_ci(clean_acc, corrupt_acc, alpha=0.05):
    n = len(clean_acc)
    diff = clean_acc.mean() - corrupt_acc.mean()
    se = (clean_acc.std(ddof=1)**2 / n +
          corrupt_acc.std(ddof=1)**2 / n) ** 0.5
    z = 1.96
    return diff - z*se, diff + z*se
```

Lokalizálja a hibát a négy jelenetes példán, adja meg a téves következményt, és írja át páros d_s-alapú bootstrapre. *Beadandó:* a hiba oka, a példán a helytelen és a helyes intervallum, valamint a javított kód.

F5 (kísérlettervezési, nehézség 3). Tervezzen protokollt egy új, nem a GlyphLab csomagban lévő korrupció, például erős Gauss-elmosás hatásának vizsgálatára ugyanezen a 30 tesztjeleneten. Határozza meg a nullhipotézist, a metrikát, a bootstrap egységet, a replikációszámot és azt a döntési szabályt, amellyel a nullát elutasítja. Adjon egy érvényességi korlátot arra, hogy a következtetés miért nem általánosítható új jelenetekre. *Beadandó:* protokoll szövegesen és pszeudokódban, a döntési szabály képlete, valamint a korlát indoklása.

38.3.8 Megoldások és indoklás

F1 megoldás. A per-image accuracy $\frac{1}{\sum_s V_s}\sum_{s,v}\mathbf{1}[\hat{y}_{s,v} = c_s]$, a per-scene átlag $\frac{1}{S}\sum_s \frac{1}{V_s}\sum_v \mathbf{1}[\hat{y}_{s,v} = c_s]$. Példa: 1. jelenet $V_1 = 2$, mindkét nézet helyes; 2. jelenet $V_2 = 3$, egy nézet helyes. Per-image: $\frac{2+1}{2+3} = \frac{3}{5} = 0{,}6$. Per-scene átlag: $\frac{1}{2}(1 + \frac{1}{3}) = \frac{1}{2}\cdot\frac{4}{3} = \frac{2}{3} \approx 0{,}667$. Nem egyenlők. Az azonos nézetszám **elégséges, de nem szükséges** feltétel az egyenlőséghez: ellenpélda két jelenet $V_1 = 4$, $c_1 = 2$ és $V_2 = 2$, $c_2 = 1$ helyes nézettel; ekkor a per-image érték $\frac{2+1}{4+2} = 0{,}5$, a per-scene átlag $\frac{1}{2}(\frac{2}{4} + \frac{1}{2}) = 0{,}5$, tehát egyenlők, noha a nézetszámok eltérnek.

F2 megoldás. A $\bar{d}^{(b)}$ lehetséges értékei 0,5, 0,25, 0 rendre $\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$ valószínűséggel. Várható érték:

$$\mathbb{E}^*[\bar{d}^{(b)}] = 0{,}5\cdot\tfrac{1}{4} + 0{,}25\cdot\tfrac{1}{2} + 0\cdot\tfrac{1}{4} = 0{,}125 + 0{,}125 = 0{,}25 = \bar{d}.$$

A rendezett eloszlás kumulatív valószínűségei: $P(\bar{d}^{(b)} \le 0) = \frac{1}{4}$, $P(\bar{d}^{(b)} \le 0{,}25) = \frac{3}{4}$, $P(\bar{d}^{(b)} \le 0{,}5) = 1$. A 2,5%-os kvantilis 0, mert a 0 érték valószínűsége $\frac{1}{4} > 0{,}025$; a 97,5%-os kvantilis 0,5, mert $P(\bar{d}^{(b)} \le 0{,}25) = 0{,}75 < 0{,}975$, és a 0,5 már eléri az 1-et. Az intervallum tehát $[0;\ 0{,}5]$.

F3 megoldás.

```
import numpy as np

def stratified_scene_bootstrap(d, class_ids, B=2000, alpha=0.05, seed=20261020):
    rng = np.random.default_rng(seed)
    n_classes = int(np.max(class_ids)) + 1
    reps = np.empty(B, dtype=np.float64)
```

```
    for b in range(B):
        rows = []
        for k in range(n_classes):
            pool = np.where(class_ids == k)[0]
            rows.append(rng.choice(pool, size=len(pool), replace=True))
        reps[b] = d[np.concatenate(rows)].mean()
    return np.quantile(reps, [alpha/2, 1-alpha/2])

d = np.array([0.5, 0.5, 0.5, -0.5])
class_ids = np.array([0, 0, 1, 1])
lo, hi = stratified_scene_bootstrap(d, class_ids)
print(lo, hi)   # várt közelítőleg: 0.0 0.5
```

Alakellenőrzés: `d` alakja `(S,)`, `class_ids` alakja `(S,)`, a visszatérési érték `(2,)`. A `class_ids` maximális értéke plusz egy adja az osztályok számát; a példában 2, a teljes laborban 3. A tolerancia $\pm 0{,}1$ a percentilisekre, mert $B = 2000$ mellett a diszkrét eloszlás közelítése már stabil.

F4 megoldás. A hiba oka: a kód a `clean_acc` és `corrupt_acc` tömböket két független mintaként kezeli, és a jelenetek közötti szórást mindkét feltételben teljes mértékben beleszámítja a standard hibába; a páros szerkezetet elhagyja. A négy jelenetes példán a clean minta átlaga 0,75, a korrupté 0,25, a különbség 0,5; a két szórásnégyzet 0,0833, ezért $SE = \sqrt{2 \cdot 0{,}0833/4} \approx 0{,}2041$. A kódban szereplő $z = 1{,}96$ normálközelítéssel a hibás intervallum [0,100; 0,900], a Welch-féle t-közelítéssel ($df = 6$, $t \approx 2{,}447$) [0,0005; 0,9995]. A tévedés lényege nem az, hogy az unpaired intervallum feltétlenül tartalmazná a nullát (a konkrét adatokon nem tartalmazza), hanem hogy az eljárás a jelenetek közös nehézségét zajként kezeli: a minden jeleneten azonos 0,5 romlást mintegy 0,8 széles bizonytalansági sávként mutatja, miközben a páros intervallum [0,5; 0,5]. Kis minta esetén egyébként a t-kvantilis indokolt a $z = 1{,}96$ helyett (páros t-próbánál $df = 3$-mal $t_{0{,}975,3} \approx 3{,}182$), de a páros d_s szórása itt nulla, ezért a páros t-próba degenerált. A helyes javítás:

```
def paired_bootstrap_ci(a_clean, a_corrupt, class_ids, B=2000, alpha=0.05, seed=20261020):
    d = a_clean - a_corrupt
    return stratified_scene_bootstrap(d, class_ids, B, alpha, seed)
```

A példán `a_clean = [1,1,0.5,0.5]`, `a_corrupt = [0.5,0.5,0,0]`, `class_ids = [0,0,1,1]`. A páros bootstrap a degenerált $d_s = 0{,}5$ miatt minden replikációra 0,5-öt ad, az intervallum [0,5; 0,5]; a hiba kijavítva kimutatja a konzisztens romlást.

F5 megoldás. Protokoll: nullhipotézis $H_0 \colon \mathbb{E}[d_s] = 0$, ahol d_s a clean és az elmosott predikció jelenetszintű accuracy-különbsége. Metrika: $\bar{d}$. Megfigyelési egység: jelenet, nem kép. Eljárás: az új korrupt képeket a 30 tesztjelenet mindkét nézetére előállítjuk; a befagyasztott modellt futtatjuk; kiszámítjuk d_s-t; stratified scene bootstrap, $B = 2000$, $\alpha = 0{,}05$; a nullát akkor utasítjuk el, ha a percentilis intervallum nem tartalmazza a nullát. Pszeudokód:

```
repeat b = 1..B:
    for each class k:
        resample |I_k| scene indices from I_k with replacement
    compute mean of d_s on the resampled set
interval = (2.5%, 97.5%) quantile of the B means
reject H0 if interval excludes 0
```

A döntési szabály képlete: legyen q_p a $\{\bar{d}^{(b)}\}_{b=1}^{B}$ p-kvantilise; ekkor H_0-t elutasítjuk, ha $0 \notin [q_{\alpha/2}, q_{1-\alpha/2}]$, azaz $q_{\alpha/2} > 0$ vagy $q_{1-\alpha/2} < 0$. A percentilis intervallum lefedettsége közelítő, nem egzakt: a bootstrap-diszkrét eloszlásból becsül, a $B = 2000$ replikáció Monte Carlo-hibát visz a kvantilisekbe, és a jelenetszintű d_s diszkrét értékei miatt az intervallum végpontjai a lehetséges értékek rácsára esnek; ezért a döntés szintje közelítő, nem pontosan $1 - \alpha$.

Érvényességi korlát: az eljárás feltételes a 30 jelenetre, és azt kérdezi, hogy a megfigyelt jelenetek újramintavételezésén a hatás konzisztens-e; nem bizonyítja, hogy új, független jeleneteken is ugyanaz a hatás, mert a jelenetgeneráló eloszlásról nincs független mintánk. A következtetés ezért GlyphLab-specifikus, nem általános látási állítás. A bootstrap seedje 20261020, az erőforrás CPU, $B = 2000$ jelenetszintű újramintavétel a befagyasztott modellen.

38.3.9 Önellenőrzés és visszatekintés

Az átadás előtt ellenőrizze: (1) a `pred_clean` és `pred_corrupt` alakja pontosan `(60,)` és `int64`; (2) a `state_dict` betöltése `weights_only=True` módon történt, és a kulcshalmaz egyezik a modellével ([D09]); (3) a modell `eval()` módban van, nincs `torch.set_grad_enabled(True)`; (4) a korrupt sorrend a metadata teszt-sorrendjével azonos, ezért a `d_s` párosítás helyes; (5) a bootstrap a jeleneteken, nem a képeken mintavételez újra, és a rétegzés osztályonként 10 jelenetet tart; (6) a végső tesztet csak a 38.2. Modell, training és best checkpoint modellválasztása után értékeltük ki.

Visszatekintés a 37.3. Linearitás, adjungált és határfeltétel egységből: a konvolúciós modell kiértékelésben lineáris, ezért a predikció a paraméterezett súlyokkal determinisztikus; a 34.3. Pruning és struktúra egységből: ha a 38.2. Modell, training és best checkpoint checkpoint előtt pruning történt, a betöltött architektúrának a pruned változattal kell egyeznie, különben kulcsméret-hiba lép fel; a 26.3. NMS, confidence és AP egységből: a tesztkiértékelés előtt semmilyen küszöböt vagy modellt nem hangolunk a teszteredményre.

38.3.10 Rövid összefoglaló

A befagyasztott modell zárt tesztjén a clean accuracyt, a jelenetszintű csoportátlagot és a macro accuracyt definiáltuk; azonos nézetszám mellett a csoportátlag megegyezik a per-image accuracyvel, amit azonosságként levezettünk. A korrupt teszt ugyanazon jelenet–nézet párokat tartalmazza, ezért a hatást páros d_s különbséggel mérjük. A rétegzett jelenet-bootstrap osztályonként visszatevéssel őrzi az osztályarányt, és feltételes percentilis intervallumot ad $\bar{d}$-re. A kézi négy jelenetes példán az egzakt bootstrap-eloszlás $[0;\ 0{,}5]$-öt ad, ami a bootstrap-függvény (F3) ellenőrzési alapja; a labor `eval_test.py` 30 jelenetes futása ettől különböző intervallumot ad. A párosság elhagyása konzisztens romlást rejthet el.

38.3.11 Következő lecke és források

A 38.4. Mentett modellből független inference egység a mentett modellből független inferencet vizsgál: ott az itt előállított `n151_pred_clean.npy` és `n151_report.json` a modellcsomag részeként újra betölthető és egyezésvizsgálat alá vonható lesz. Az itt bemutatott páros és rétegzett értékelési logika a végső vizsgafeladatban is visszatér.

Felhasznált források:

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D09] PyTorch contributors (2026). *Saving and Loading Models.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [S03] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap.* Chapman & Hall/CRC. A feltételes, ismételt mintavételi értelmezés és a percentilis bootstrap forrása.

38.4 Mentett modellből független inference

38.4.1 Cél, előismeret és induló kérdés

A 38.3. Végső teszt és páros korrupció zárótesztje után rendelkezésre áll egy befagyasztott checkpoint, a clean és corrupt teszt nyers logitjai, valamint a jelenetszintű bootstrap intervallumok. Ebben az alfejezetben a kérdés már nem az, hogy a modell tanult-e, hanem az, hogy a mentett állapotból **egy új, a tanítástól független processzben** visszaállítható-e ugyanaz a használati viselkedés. A programozói előismeret továbbra is a 38.3. Végső teszt és páros korrupció anyaga; a 37.4. Rétegből kis tanulómodell saját rétegintegrációját csak annyiban idézzük vissza, hogy a mentett paraméterek az `nn.Module` regisztrált paramétereiben élnek. A 34.4. Teacher–student desztilláció alfejezetből az `eval()` mód, a 26.4. Detektálási pipeline és hibaelemzés alfejezetből a transzformált input szerződésének gondolata öröklődik át.

Induló kérdés: ha valaki csak egy `best.pt` nevű fájlt kap kézhez, hányféleképpen tudja félreértelmezni? A válasz a következő szakaszban adott definíciókból következik: a súlyfájl önmagában nem modell.

38.4.2 A mentett modell mint szerződés: definíciók

Definíció (state_dict). Egy PyTorch `nn.Module` `state_dict()` visszatérési értéke `OrderedDict[str, Tensor]`, amely a regisztrált paramétereket és a **perzisztens** puffereket tartalmazza a rétegnévvel pontozott kulcsok alatt (a `persistent=False` jelzéssel regisztrált pufferek kimaradnak). Egy konvolúciós modellben például `features.0.weight` alakja $C_{out} \times C_{in}/g \times k_h \times k_w$, itt OIHW ([D10]). A `state_dict` **nem tartalmazza** a modellosztály kódját, a hiperparamétereket, a label mapet és az input előfeldolgozást; ezért önmagában nem hordoz inference-szerződést [D09].

Definíció (model id). A `model_id` egy rögzített azonosító, amely egy architektúraosztályra, annak építési paramétereire és a checkpointra együttesen hivatkozik. Itt: `glyphlab_convnet_v1`. Ha az osztály kódja úgy változik, hogy a `state_dict` kulcsai vagy a forward szemantikája megváltozik, az azonosító önmagában nem véd meg; ezért a modellkártya az architektúrafájl nevét és SHA-256 hash-ét is rögzíti, az `inference.py` pedig a betöltés előtt ellenőrzi ezt a hash-t.

Definíció (input schema). A függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-osztályozó bemeneti szerződése:

$$X \in \mathbb{R}^{N\times 1\times 24\times 24}, \qquad X_{n,0,i,j} \in [0;\ 1], \qquad \text{dtype float32, NCHW.}$$

A batchméret N tetszőleges pozitív egész, de a két térbeli dimenzió és a csatornaszám rögzített. A `[0,1]` tartomány alsó és felső korlátja a generátor `ADATOK.md` szerződéséből jön; az inference script ezt ellenőrzi is, nemcsak feltételezi.

Definíció (label map). A rendezett lista

$$L = (\text{"vertical"}, \text{"horizontal"}, \text{"diagonal"})$$

és a modell logitjai között a $\arg\max_k Z_{nk}$ osztályindex teremt kapcsolatot. A mentett logit tömbhöz külön el kell menteni L-t, különben a 0, 1, 2 indexek jelentése visszafejthetetlen.

Definíció (preprocessing). A tanítás során az adat már `[0,1]` float32 NCHW formában érkezett, így az inference oldalon további normalizálás nincs. Ez nem véletlen: a normalizálást akkor kell a modellkártyára írni, ha van. Egy későbbi `uint8` bemenet esetén a `255`-tel osztás a tanításkori tartományba hozná a képet, de ezt külön szerződésként kell rögzíteni, nem hallgatólagosan.

A fenti négy definícióból összeáll a **modellkártya** minimális mezőkészlete. A laborban megadott `model_card.json` ezt géppel olvashatóan rögzíti.

38.4.3 A visszatöltési egyezés pontos állítása

A levezetési követelmény itt egy pontos tolerancia- és környezetdefiníció. Először egy tisztán matematikai lemma, amely a logit-hibából predikciós egyezést ad.

Lemma (logit-margin elegendő feltétel top-1 egyezésre). Legyen $Z, \hat{Z} \in \mathbb{R}^{N \times K}$ két logitmátrix. Legyen

$$\epsilon = \max_{n,k} \left| \hat{Z}_{nk} - Z_{nk} \right|,$$

és minden n-re legyen $a_n = \arg\max_k Z_{nk}$. Definiáljuk a referencia marginját:

$$\gamma_n = Z_{n,a_n} - \max_{k \neq a_n} Z_{n,k}.$$

Ha minden n-re $\gamma_n > 0$ és

$$\epsilon < \frac{1}{2} \min_n \gamma_n,$$

akkor minden n-re $\arg\max_k \hat{Z}_{nk} = a_n$.

Bizonyítás. Rögzítsünk egy n-t. Tetszőleges $j \neq a_n$ osztályra a feltevésből

$$\hat{Z}_{n,a_n} \geq Z_{n,a_n} - \epsilon,$$

és a referencia-margin definíciójából

$$Z_{n,j} \leq Z_{n,a_n} - \gamma_n.$$

Ezért

$$\hat{Z}_{n,j} \leq Z_{n,j} + \epsilon \leq Z_{n,a_n} - \gamma_n + \epsilon.$$

A $2\epsilon < \gamma_n$ feltétel miatt $Z_{n,a_n} - \epsilon > Z_{n,a_n} - \gamma_n + \epsilon$, tehát

$$\hat{Z}_{n,a_n} > \hat{Z}_{n,j} \qquad \forall j \neq a_n.$$

Így az arg max egyértelmű a referencia győztesén. Ha Z sorában tie van, akkor $\gamma_n = 0$, és a feltétel nem teljesülhet; a lemma ekkor nem állít semmit.

A lemma **bizonyított**, de a gyakorlati visszatöltési kérdésre csak akkor alkalmazható, ha a betöltött modell tényleges logit-hibáját ismerjük. Ezért külön állításként rögzítjük a numerikus ellenőrzés kritériumát.

Kontrollállítás (környezeti, nem platformfüggetlen tétel). A visszatöltött modell akkor tekinthető a 38.3. Végső teszt és páros korrupció referenciafuttatással egyezőnek, ha az összes alábbi környezeti feltétel teljesül, és az (1)–(2) numerikus kritérium fennáll:

1. Ugyanaz a Python- és PyTorch-verzió fut, mint a minősítő futtatásban; itt a projekt `requirements.txt` rögzíti ezt. Az új processz CPU-t használ, `float32` számítással.
2. `torch.set_num_threads(1)` hívás történik a forward előtt, hogy a redukciók sorrendje ne függjön a szálütemezéstől.
3. A modell a betöltés után `model.eval()` állapotba kerül, és a forward `torch.inference_mode()` kontextusban fut; így dropout nem aktív, BatchNorm running state nem frissül [D12, D09].
4. Ugyanaz az `models.py` osztály fut, mint a minősítő csomagban: a modellkártya `architecture_sha256` mezője a bundle `models.py` fájljának hash-ét rögzíti, és az `inference.py` a betöltés előtt ellenőrzi, hogy a futó `models` modul fájlja byte-szinten ugyanaz. Az input bájtokig azonos `np.ascontiguousarray` NCHW float32 tömbből jön.
5. A checkpoint fájl SHA-256 értéke egyezik a modellkártyán rögzített `checkpoint_sha256` értékkel. A kártya emellett a forrás-checkpoint (`checkpoints/best_val_cnn.pt`) SHA-256 értékét is rögzíti auditálásra. Az export a forrás `state_dict` tensorait változtatás nélkül menti, és a `package_project2.py` a mentés előtt tenzoronként ellenőrzi az egyezést; ezért a betöltött paraméterek bitazonosak a minősített checkpoint paramétereivel.

E feltételek mellett a visszatöltési ellenőrzés **(1) numerikus kritériuma** (logit-allclose) pontosan a `np.allclose(logits, reference, rtol=1e-4, atol=1e-4)` hívás elemenkénti feltétele, ahol a második argumentum, a Z referencia a viszonyítási alap:

$$\left|\hat{Z}_{nk} - Z_{nk}\right| \le \texttt{atol} + \texttt{rtol} \cdot |Z_{nk}| \quad \text{minden } n,k\text{-ra}, \qquad \texttt{atol} = \texttt{rtol} = 10^{-4}.$$

Előfeltétel, hogy minden érték véges: `NaN` esetén az `allclose` hamis, ezért a labor a végességet külön is ellenőrzi. Ez a feltétel nem azonos egy külön abszolút korláttal: például $Z = 1$, $\hat{Z} = 1{,}00015$ mellett $|\hat{Z}-Z| = 1{,}5\cdot10^{-4} \le 10^{-4}+10^{-4}\cdot1$, tehát az `allclose` igaz, a $|\hat{Z}-Z| \le 10^{-4}$ korlát viszont hamis. A **(2) numerikus kritérium** a lemma: ehhez külön kiszámítjuk a tényleges $\epsilon = \max_{n,k} |\hat{Z}_{nk} - Z_{nk}|$ értéket, és ha a referencia marginokra $2\epsilon < \min_n \gamma_n$, akkor a lemma garantálja a predikciós egyezést. A margin-feltétel elégséges, nem szükséges: hamis értéke mellett is lehet egyezés, ezért a labor nem tekinti automatikus eltérésnek. A `margin_sufficient` ezért **diagnosztikai mező**, nem tagja a kötelező `checks` szótárnak; a `PASS` a kötelező ellenőrzések (`shape_match`, `index_match`, `finite`, `logit_allclose`, `top1_match`) konjunkciója. Maga a bitazonos logit-egyezésre **nincs garancia**; a [D02] dokumentáció szerint platform és verzió váltása megváltoztathatja a bitmintát, ezért a gyakorlatban a tolerancia a szerződés, nem a bitazonosság. Ezt a pontot nem általános konvergencia-tételként, hanem a saját laborra vonatkozó ellenőrzési protokollként kezeljük.

38.4.4 Kézi példa: margin és logit-hiba

Legyen a referencia

$$Z = \begin{pmatrix} 2{,}0 & 0{,}3 & -1{,}0 \\ 0{,}1 & 1{,}7 & 1{,}6 \end{pmatrix}, \qquad \hat{Z} = \begin{pmatrix} 1{,}999 & 0{,}302 & -0{,}997 \\ 0{,}098 & 1{,}701 & 1{,}603 \end{pmatrix}.$$

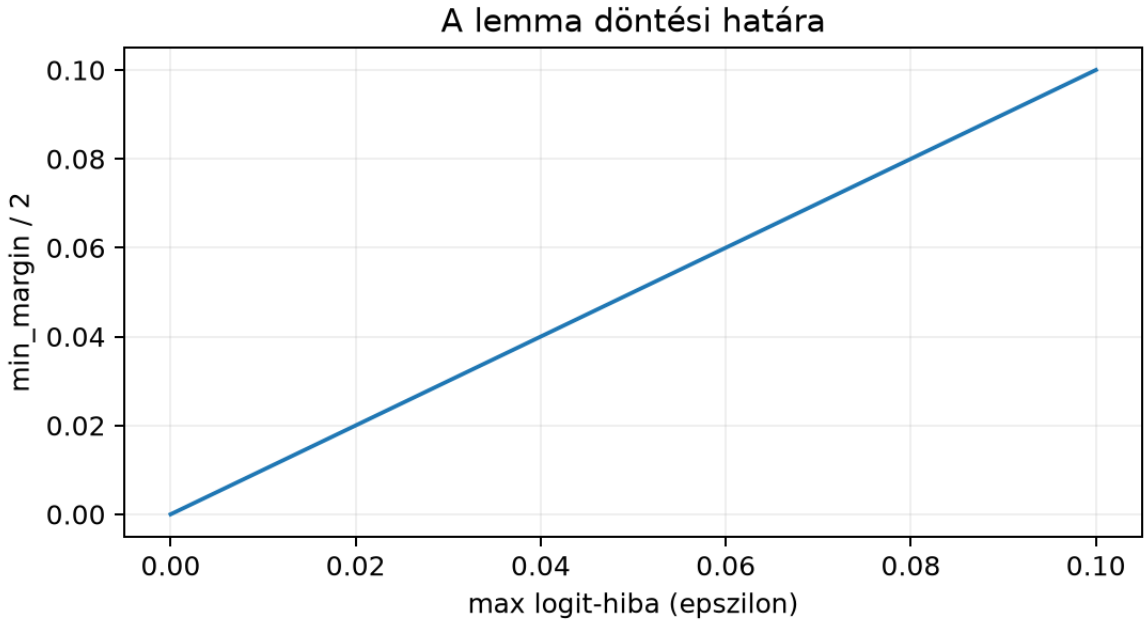
Az eltérésmátrix elemenként: $-0{,}001$, $0{,}002$, $0{,}003$, $-0{,}002$, $0{,}001$, $0{,}003$, tehát $\epsilon = 0{,}003$. A marginok:

$$\gamma_1 = 2{,}0 - 0{,}3 = 1{,}7, \qquad \gamma_2 = 1{,}7 - 1{,}6 = 0{,}1.$$

Mivel $2\epsilon = 0{,}006 < 0{,}1 = \min \gamma_n$, a lemma predikciós egyezést garantál; valóban a két sorban a 0. és az 1. index (nullától indexelve) marad a győztes.

Ellenkező irányba: ha a második sor logitjai $Z = (0{,}1;\ 1{,}7;\ 1{,}698)$ és $\hat{Z} = (0{,}101;\ 1{,}699;\ 1{,}696)$, akkor $\gamma_2 = 0{,}002$ és az abszolút eltérés maximuma $\epsilon = \max |Z - \hat{Z}| = 0{,}002$ ($1{,}698 \to 1{,}696$). A $2\epsilon = 0{,}004 > 0{,}002$ feltétel nem teljesül; a lemma nem alkalmazható, noha ebben a konkrét esetben az $\arg\max$ még egybeesik. A feltétel **elegendő, nem szükséges**. Ez a különbség a gyakorlatban fontos: ha a script a margin-ellenőrzést `false` értékkel jelenti, az nem automatikusan predikciós eltérést jelent, hanem azt, hogy a hibahatár nem zárja ki azt.

A margin-döntési határt mutató ábra:



38.2. ábra: A lemma döntési határa

A vonal alatti pontokra a top-1 egyezés garantált; képletből számolt szemléltetés, nem mérés.

38.4.5 Labor: átadható inference csomag

A labor kimenete a `lab/PROJECTII/` könyvtár. Bemenete a 38.2.–38.3. választott CNN-checkpointja, a `lab/checkpoints/best_val_cnn.pt`. Ez a 38.2. Modell, training és best checkpoint burkolt sémáját követi (`model_state_dict` és metaadatok); az átadási csomagba a súlyokat **változtatás nélkül, puszta `state_dict`-ként** exportáljuk `models/best.pt` néven, mert a független betöltőnek nincs szüksége a tanítási metaadatokra, azok a modellkártyára kerülnek. Az exportot, a referencia-logitokat és a kártya hash-ét a következő szkript állítja elő a `lab/` könyvtárból:

```python
# lab/package_project2.py - 38.4. alfejezet: a 38.2.-38.3. checkpointjából átadható PROJECTII/
↪  csomag.
# A 38.2. alfejezet burkolt checkpointjából a súlyokat változtatás nélkül, puszta state_dictként
# exportálja; a model_card hash-ét és a referencia-logitokat a tényleges fájlból számolja.
import hashlib, json, shutil
from pathlib import Path
import numpy as np
import pandas as pd
import torch

from glyph_models import load_checkpoint

LAB = Path(__file__).resolve().parent
OUT = LAB / "PROJECTII"


def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def main():
    source = LAB / "checkpoints" / "best_val_cnn.pt"
    model, ckpt = load_checkpoint(source)     # a betöltés eval módba állítja a modellt
    model.eval()                      # az átadási referencia-logitok eval módban készülnek
    (OUT / "models").mkdir(parents=True, exist_ok=True)
    (OUT / "data").mkdir(exist_ok=True)
    export_state, source_state = model.state_dict(), ckpt["model_state_dict"]
    if list(export_state) != list(source_state):
        raise RuntimeError("az export kulcshalmaza eltér a forrás state_dict-től")
    if not all(torch.equal(export_state[k], source_state[k]) for k in export_state):
        raise RuntimeError("az export nem bitazonos a forrás state_dict tensoraival")
    torch.save(export_state, OUT / "models" / "best.pt")           # átadási formátum
    shutil.copyfile(LAB / "data" / "images.npy", OUT / "data" / "images.npy")
    meta = pd.read_csv(LAB / "data" / "metadata.csv")
    if not (meta["index"].to_numpy() == np.arange(len(meta))).all():
        raise RuntimeError("a metadata indexe nem a 0..N-1 tartomány permutációja")
    test_idx = meta.loc[meta["split"] == "test", "index"].to_numpy().astype(np.int64)
    if len(test_idx) != 60:
        raise RuntimeError(f"a teszt split nem 60 kép: {len(test_idx)}")
    np.save(OUT / "test_indices.npy", test_idx)
    x = torch.from_numpy(np.load(LAB / "data" / "images.npy")[test_idx].astype(np.float32))
    torch.set_num_threads(1)
    with torch.inference_mode():
        np.savez(OUT / "test_reference.npz",                 # referencia-logitok + globális
↪  index
                 logits=model(x).numpy().astype(np.float32), index=test_idx)
    for name in ("models.py", "inference.py"):
        if not (OUT / name).exists():
            raise FileNotFoundError(f"hiányzik a csomagból: {name}")
    card = {
        "model_id": "glyphlab_convnet_v1",
        "architecture_module": "models.py",
        "architecture_class": "GlyphLabConvNet",
        "architecture_sha256": sha256_file(OUT / "models.py"),
        "input_schema": {"dtype": "float32", "layout": "NCHW",
                         "shape": [None, 1, 24, 24], "value_range": [0.0, 1.0]},
        "preprocessing": "nincs további normalizálás; a bemenő tömb már [0,1] float32 NCHW",
        "label_map": ["vertical", "horizontal", "diagonal"],
        "checkpoint_file": "models/best.pt",
```

```
        "checkpoint_sha256": sha256_file(OUT / "models" / "best.pt"),
        "source_checkpoint": {"file": "checkpoints/best_val_cnn.pt",
                              "sha256": sha256_file(source),
                              "seed": ckpt["seed"], "epoch": ckpt["epoch"],
                              "val_loss": ckpt["val_loss"]},
        "training_data": "GlyphLab 1.0, train 120 scene / 240 image, két view",
        "validation_scope": "csak GlyphLab szintetikus képek; valós felhasználás nem validált",
        "eval_protocol": "test_clean_60, paired corrupt, stratified scene bootstrap (38.3.
        ↪  alfejezet)",
        "pytorch_version": torch.__version__,
        "torch_threads": 1,
    }
    (OUT / "model_card.json").write_text(json.dumps(card, indent=2, ensure_ascii=False) + "\n",
                                         encoding="utf-8")
    # a minősítő futtatás tényleges verziói (CPU-build esetén a "+cpu" helyi címke nélkül)
    (OUT / "requirements.txt").write_text(
        f"numpy=={np.__version__}\ntorch=={torch.__version__.split('+')[0]}\n",
        ↪  encoding="utf-8")
    print(f"PROJECTII kész: sha256={card['checkpoint_sha256'][:16]}…, "
          f"forrás seed={ckpt['seed']} epoch={ckpt['epoch']}")


if __name__ == "__main__":
    main()
```

A szkript a `models/best.pt`, `data/images.npy`, `test_indices.npy` (alak $(60,)$, int64), `test_reference.npz` (`logits`: $(60;\ 3)$ float32, `index`: a hozzájuk tartozó $(60,)$ int64 globális indexek), `model_card.json` és `requirements.txt` fájlokat írja. A `models.py` és az `inference.py` a laborcsomag `PROJECTII/` könyvtárában már megtalálható; ha valaki a fejezet kódblokkjait használva kézzel állítja össze a csomagot, ezeket a kódból kell ugyanoda mentenie. A szkript a kártya kiírása előtt ellenőrzi, hogy mindkét fájl jelen van. A csomagoló ellenőrzései explicit kivételeket dobnak, az `inference.py` kritikus ellenőrzései pedig `sys.exit`-tel állnak le, ezért `python -O` mellett sem kapcsolódnak ki (az `assert`-tel ellentétben). Futtatás: `cd lab && python package_project2.py`. A beadandó csomag:

- `models.py`: a modellosztály,
- `model_card.json`: a szerződés géppel olvasható formája,
- `inference.py`: független betöltő és batch-inference parancs,
- `requirements.txt`: a környezet.

A `models.py`:

```
# lab/PROJECTII/models.py
import torch
from torch import nn

class GlyphLabConvNet(nn.Module):
    """GlyphLab orientációs osztályozó; input NCHW (N,1,24,24), float32 [0,1]."""
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 6 * 6, 32),
```

```
            nn.ReLU(),
            nn.Linear(32, 3),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.features(x))
```

A `model_card.json` szerkezete (a hash-értékeket, a forrás-checkpoint adatait és a PyTorch-verziót a `package_project2.py` a tényleges fájlokból tölti ki; itt a mezőséma látható, a számértékek helyőrzők):

```
{
  "model_id": "glyphlab_convnet_v1",
  "architecture_module": "models.py",
  "architecture_class": "GlyphLabConvNet",
  "architecture_sha256": "a_bundle_models.py_számított_sha256_értéke",
  "input_schema": {
    "dtype": "float32",
    "layout": "NCHW",
    "shape": [null, 1, 24, 24],
    "value_range": [0.0, 1.0]
  },
  "preprocessing": "nincs további normalizálás; a bemenő tömb már [0,1] float32 NCHW",
  "label_map": ["vertical", "horizontal", "diagonal"],
  "checkpoint_file": "models/best.pt",
  "checkpoint_sha256": "a_minősített_checkpoint_számított_sha256_értéke",
  "source_checkpoint": {"file": "checkpoints/best_val_cnn.pt",
                        "sha256": "a_forrás_checkpoint_számított_sha256_értéke",
                        "seed": 0, "epoch": 0, "val_loss": 0.0},
  "training_data": "GlyphLab 1.0, train 120 scene / 240 image, két view",
  "validation_scope": "csak GlyphLab szintetikus képek; valós felhasználás nem validált",
  "eval_protocol": "test_clean_60, paired corrupt, stratified scene bootstrap (38.3.
  ↪  alfejezet)",
  "pytorch_version": "2.15.0+cpu",
  "torch_threads": 1
}
```

A fenti JSON a **mezőséma sablonja**: a `source_checkpoint` `seed`, `epoch` és `val_loss` értékei helyőrzők (0), a tényleges csomagban a forrás-checkpoint valódi értékei szerepelnek (a jegyzőkönyv futásában seed 20261022, 19. epoch). A `checkpoint_sha256`, az `architecture_sha256` és a `source_checkpoint.sha256` mezőket a `package_project2.py` a tényleges fájlokból számítja, nem kézzel írja.

Az `inference.py` tömör változata:

```
# lab/PROJECTII/inference.py
import argparse, hashlib, json, sys
import numpy as np
import torch
import models
from models import GlyphLabConvNet

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--checkpoint", required=True)
```

```
ap.add_argument("--card", required=True)
ap.add_argument("--images", required=True)
ap.add_argument("--indices", required=True)
ap.add_argument("--reference", default=None)
ap.add_argument("--output", default="inference_results.json")
args = ap.parse_args()

torch.set_num_threads(1)
with open(args.card, "r", encoding="utf-8") as f:
    card = json.load(f)

# --- preflight: a checkpoint és a bundle models.py hash-egyezése
actual_hash = sha256_file(args.checkpoint)
arch_hash = sha256_file(models.__file__)
preflight = {
    "status": "HASH_OK",
    "checkpoint_sha256": actual_hash,
    "checkpoint_sha256_expected": card["checkpoint_sha256"],
    "architecture_sha256": arch_hash,
    "architecture_sha256_expected": card.get("architecture_sha256"),
}
preflight["checkpoint_match"] = actual_hash == preflight["checkpoint_sha256_expected"]
preflight["architecture_match"] = arch_hash == preflight["architecture_sha256_expected"]
if not (preflight["checkpoint_match"] and preflight["architecture_match"]):
    preflight["status"] = "HASH_MISMATCH"
    with open(args.output, "w", encoding="utf-8") as f:
        json.dump(preflight, f, indent=2)
    print(f"HASH_MISMATCH -> {args.output}", file=sys.stderr)
    sys.exit(1)

model = GlyphLabConvNet()
state = torch.load(args.checkpoint, map_location="cpu", weights_only=True)
model.load_state_dict(state)
model.eval()

images = np.load(args.images)
indices = np.load(args.indices)
if not np.issubdtype(indices.dtype, np.integer) or indices.ndim != 1:
    sys.exit("indices: (N,) egész típusú tömb kell")
indices = indices.astype(np.int64)
if indices.size == 0 or indices.min() < 0 or indices.max() >= len(images):
    sys.exit("indices: indexhatár hiba")
x = np.ascontiguousarray(images[indices], dtype=np.float32)
schema = card["input_schema"]
if not (x.ndim == 4 and tuple(x.shape[1:]) == tuple(schema["shape"][1:])):
    sys.exit(f"input shape hiba: {x.shape}")
lo, hi = schema["value_range"]
if not (np.isfinite(x).all() and x.min() >= lo - 1e-7 and x.max() <= hi + 1e-7):
    sys.exit("input értéktartomány hiba")

with torch.inference_mode():
    logits = model(torch.from_numpy(x)).numpy().astype(np.float32)
probs = np.exp(logits - logits.max(axis=1, keepdims=True))
probs /= probs.sum(axis=1, keepdims=True)
pred = np.argmax(logits, axis=1)

result = {
    "model_id": card["model_id"],
    "checkpoint_sha256": actual_hash,
    "architecture_sha256": arch_hash,
    "hash_preflight": preflight,
    "device": "cpu",
    "dtype": "float32",
```

```
        "input_count": int(x.shape[0]),
        "predictions": pred.tolist(),
        "probabilities": probs.tolist(),
        "label_map": card["label_map"],
    }

    # Az inference sikere és a referencia-egyezés két külön dolog: az utóbbi csak
    # akkor PASS, ha minden előírt ellenőrzés teljesül; FAIL esetén 2-es kilépés.
    # A margin_sufficient diagnosztikai mező, nem tagja a kötelező checks-nek.
    status = "INFERENCE_OK"
    if args.reference is not None:
        ref = np.load(args.reference)                    # npz: "logits" és "index"
        ref_logits, ref_index = ref["logits"], ref["index"].astype(np.int64)
        checks = {
            "shape_match": ref_logits.shape == logits.shape,
            "index_match": bool(np.array_equal(ref_index, indices)),
            "finite": bool(np.isfinite(logits).all() and np.isfinite(ref_logits).all()),
        }
        if all(checks.values()):
            eps = float(np.max(np.abs(logits - ref_logits)))
            # np.allclose: |a - b| <= atol + rtol * |b| elemenként, b a referencia
            checks["logit_allclose"] = bool(np.allclose(logits, ref_logits, rtol=1e-4,
↪   atol=1e-4))
            ref_pred = np.argmax(ref_logits, axis=1)
            checks["top1_match"] = bool(np.array_equal(pred, ref_pred))
            top2 = np.sort(ref_logits, axis=1)[:, -2:]
            margins = top2[:, 1] - top2[:, 0]
            result.update({
                "max_abs_logit_diff": eps,
                "min_reference_margin": float(margins.min()),
                # a lemma elégséges feltétele; hamis értéke nem jelent automatikusan eltérést
                "margin_sufficient": bool((margins > 2 * eps).all()),
            })
        result["checks"] = checks
        status = "PASS" if all(checks.values()) else "FAIL"
    result["status"] = status

    with open(args.output, "w", encoding="utf-8") as f:
        json.dump(result, f, indent=2)
    print(f"{status} -> {args.output}")
    if status == "FAIL":
        sys.exit(2)

if __name__ == "__main__":
    main()
```

A futtatási parancs új processzben:

```
cd PROJECTII
python inference.py \
  --checkpoint models/best.pt \
  --card model_card.json \
  --images data/images.npy \
  --indices test_indices.npy \
  --reference test_reference.npz \
  --output inference_results.json
```

Az értelmezés három szintű, és mindhárom csak akkor értelmezhető, ha a preflight hash-ellenőrzés, valamint a referencia alakja, indexei és végessége előbb rendben van (`shape_match`, `index_match`, `finite`); az inference sikere (`INFERENCE_OK`) és az egyezésvizsgálat sikere (`PASS`) két külön státusz. Az **első** szint a preflight: a `hash_preflight.checkpoint_match` igazsága azt jelenti, hogy a betöltött fájl bitazonos a minősített checkpointtal, az `architecture_match` pedig azt,

hogy a futó `models.py` byte-szinten a kártyán rögzített architektúrafájl. Ha bármelyik hamis, a szkript `HASH_MISMATCH` JSON-t ír és 1-es kilépési kóddal leáll, modellt sem tölt be. A **második** szint a `logit_allclose` igazsága a fenti $(10^{-4}, 10^{-4})$ toleranciával; ez a környezeti kontrollállítás gyakorlati visszaigazolása. A **harmadik** szint a `margin_sufficient`, amely a lemma feltételét ellenőrzi a tényleges maximális logit-hibára; ez diagnosztikai mező, nem tagja a kötelező `checks`-nek, és hamis értéke nem jelent automatikus eltérést, mert a lemma feltétele elégséges, nem szükséges. Ha a három szint igaz (hash-egyezés, logit-allclose, margin-feltétel), a top-1 predikciós egyezés a lemma szerint matematikailag garantált a referenciafutáshoz képest. A `PASS` státusz a kötelező ellenőrzések (`shape_match`, `index_match`, `finite`, `logit_allclose`, `top1_match`) konjunkciója. **Kilépési kódok:** 0 = siker (`INFERENCE_OK` vagy `PASS`); 1 = hash-eltérés (`HASH_MISMATCH` JSON) vagy bemeneti validációs hiba (stderr-üzenet); 2 = a referencia-ellenőrzések `FAIL`-je (a `FAIL` JSON is kiíródik). **Futási jegyzőkönyv.** A 38.2.–38.3. fenti futásából (CNN, seed 20261022, 19. epoch; Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál) a `package_project2.py`, majd új processzben az `inference.py` lefutott: `status` = `PASS`, a `checks` mind igaz (`shape_match`, `index_match`, `finite`, `logit_allclose`, `top1_match`), `max_abs_logit_diff` = `0.0`, `min_reference_margin` $\approx$ 0,234, `margin_sufficient` = `true`, kilépési kód 0. Szándékosan hibás referenciákkal ugyanez a parancs `FAIL`-t és 2-es kilépési kódot ad: tíz soros referencia mellett a `shape_match` és az `index_match`, `NaN`-t tartalmazó referencia mellett a `finite`, minden logitban 0,01-gyel eltolt referencia mellett a `logit_allclose`, fordított indexsorrend mellett az `index_match` bukik. Az azonos gépen, azonos szálszámmal futó betöltés bitazonos logitokat adott; más CPU vagy más PyTorch-verzió mellett a $(10^{-4}, 10^{-4})$ tolerancia és a margófeltétel a mérvadó.

38.4.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázd el, miért nem elegendő pusztán a `state_dict` átadása egy független inference-hez. Sorold fel azt a három hiányzó szerződéselemet, amelyek nélkül a betöltött súlyok félreértelmezhetők. Elvárt eredmény: saját szavas magyarázat, minden hiányzó elemhez egy-egy konkrét félreértési példa.

F2 (levezetési/bizonyítási, nehézség 2). Adott a referencia $Z = \begin{pmatrix} 1{,}0 & 0{,}9 & 0{,}1 \\ 0{,}2 & 2{,}0 & 1{,}96 \end{pmatrix}$ és a betöltött $\hat{Z} = \begin{pmatrix} 1{,}0002 & 0{,}9001 & 0{,}0999 \\ 0{,}1998 & 2{,}0001 & 1{,}9602 \end{pmatrix}$. Számold ki ϵ-t, a soronkénti margókat, és döntsd el a lemma alapján, garantált-e a top-1 egyezés. Utána adj olyan $\hat{Z}$-t, amelyre a lemma feltétele nem teljesül, és mutasd meg, ott miért nem következtethetünk az $\arg\max$-egyezésre. Elvárt eredmény: minden közbülső szám, a feltétel kiértékelése és az ellenpélda indoklása.

F3 (implementációs, nehézség 2). Írj `load_bundle(bundle_dir)` függvényt, amely beolvassa a `model_card.json`-t, ellenőrzi a checkpoint SHA-256 értékét a kártyán lévővel, betölti a state_dict-ot `weights_only=True` módon, `eval()` módba állítja a modellt, és visszaadja a `(model, card)` párt. Hibás hash esetén dobjon `ValueError`-t. A függvény ne olvasson be képadatot. Elvárt eredmény: futtatható kód, egy helyes és egy meghamisított checkpoint esetén a várt kimenet.

F4 (hibakeresési, nehézség 2). Egy hallgató a következő inference kódot írta: a checkpointot `torch.load(path, weights_only=False)`-szal tölti be; betöltés után `model.train()`-t hív; a képtömböt `x.reshape(N, 24, 24, 1)` alakban adja a modellnek. Azonosítsd mindhárom hiba lehetséges következményét, javítsd a kódot, és add meg azt a kontrollt, amellyel a javítás ellenőrizhető. Elvárt eredmény: ok–következmény–javítás hármas mindhárom hibára, és egy futtatható ellenőrző sor.

F5 (kísérleti/transzfer, nehézség 3). Egy másik labor `uint8` JPEG-képekből szeretné használni a modellt egy hasonlónak látszó vonalorientációs feladatra. Tervezz protokollt, amely (a) a modellkártyát kiegészíti a szükséges új preprocessinggel és domain-korláttal, (b) baseline-t és metrikát ad a transzfer előtt, (c) meghatározza, mikor szabad a mentett modellt egyáltalán kiértékelni. Elvárt eredmény: protokollfejezetek, egy baseline, egy metrika és az érvényességi korlát pontos megfogalmazása.

38.4.7 Megoldások és indoklás

F1 megoldásminta. A `state_dict` csak paraméter- és perzisztens puffertensorokat tartalmaz, nem hordozza az architektúraosztály kódját. Egy `Conv2d(1,8,3)` és egy `Conv2d(1,8,5)` súlyai **nem** tölthetők át egymásba, mert az alakjuk eltér ($8 \times 1 \times 3 \times 3$ vs. $8 \times 1 \times 5 \times 5$), és a `load_state_dict` hibát ad; a valódi veszély az azonos súlyalakú, de eltérő szemantikájú architektúra. Például egy `Conv2d(1,8,3,stride=1,padding=1)` és egy `Conv2d(1,8,3,stride=2,padding=0)` súlya azonos alakú és kulcsú, a betöltés hiba nélkül sikerül, a kimeneti térkép mérete és tartalma viszont más; ugyanígy egy ReLU helyett Tanh-ot használó osztály is ugyanazt a `state_dict`-et fogadja. Nem tartalmazza az input sémát: ha a betöltő NHWC alakot ad be, az `(N,24,24,1)`-et a modell $C = 24$ csatornás bemenetként értelmezheti, shape-hibával vagy hibás eredménnyel. Nem tartalmazza a label mapet: a `0,1,2` indexek jelentése a tanító adatban rögzített, nem a súlyokban.

F2 megoldás. Eltérések: $|0{,}0002|$; $0{,}0001$; $0{,}0001$; $0{,}0002$; $0{,}0001$; $0{,}0002$, tehát $\epsilon = 0{,}0002$. Margók: $\gamma_1 = 1{,}0 - 0{,}9 = 0{,}1$, $\gamma_2 = 2{,}0 - 1{,}96 = 0{,}04$. $\frac{1}{2}\min \gamma_n = 0{,}02$, és $\epsilon = 0{,}0002 < 0{,}02$, tehát a lemma garantálja az egyezést. Ellenpélda: $\hat{Z} = \begin{pmatrix} 0{,}95 & 1{,}0 & 0{,}1 \\ 0{,}2 & 2{,}0 & 1{,}96 \end{pmatrix}$. Ekkor $\epsilon = \max(|0{,}95 - 1{,}0|, |1{,}0 - 0{,}9|, 0, 0, 0, 0) = 0{,}1$, és $\epsilon = 0{,}1 > 0{,}05 = \frac{1}{2}\gamma_1$ (a második sorra pedig $0{,}1 > 0{,}02 = \frac{1}{2}\gamma_2$), tehát a lemma feltétele nem teljesül. Az $\arg\max$ valóban átbillen: a referencia első sora a 0. indexet adja ($1{,}0 > 0{,}9$), a betöltött első sor az 1. indexet ($1{,}0 > 0{,}95$), miközben a második sor mindkettőben az 1. indexet adja. Az átbillenéshez $2\epsilon > \gamma_1$ szükséges, itt $2 \cdot 0{,}1 = 0{,}2 > 0{,}1$. Ez azt is mutatja, hogy a lemma feltételének megsértése valós predikciós fordulattal járhat, nem puszta formalitás.

F3 megoldásminta. A függvény a labor `inference.py`-jából kiemelhető; a kulcslépések:

```
import hashlib, json, pathlib, torch
from models import GlyphLabConvNet

def sha256_file(p):
    h = hashlib.sha256()
    with open(p, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()

def load_bundle(bundle_dir):
    bundle = pathlib.Path(bundle_dir)
    with open(bundle / "model_card.json", encoding="utf-8") as f:
        card = json.load(f)
    ckpt = bundle / card["checkpoint_file"]
    actual = sha256_file(ckpt)
    if actual != card["checkpoint_sha256"]:
        raise ValueError(f"checkpoint hash mismatch: {actual}")
    model = GlyphLabConvNet()
    state = torch.load(ckpt, map_location="cpu", weights_only=True)
    model.load_state_dict(state)
```

```
    model.eval()
    return model, card
```

Alakellenőrzés: a betöltött `state` minden kulcsa szerepel a frissen épített modell `state_dict()`-jában; a `load_state_dict(strict=True)` (alapértelmezés) ezt kikényszeríti. Szélsőeset: hiányzó fájl `FileNotFoundError`; érvénytelen JSON `json.JSONDecodeError`; mindkettő a hívónál kezelendő, nem nyelhető el.

A két eset futtatható ellenőrzése (a `bundle_tampered` a `bundle` egy bájtnyival módosított checkpointját tartalmazza):

```
model, card = load_bundle("bundle")        # helyes eset
assert model.training is False             # az eval() megtörtént
try:                                       # meghamisított checkpoint
    load_bundle("bundle_tampered")
except ValueError as e:
    print("elvárt hiba:", e)               # checkpoint hash mismatch: <aktuális sha256>
```

Várt kimenet: a helyes esetben nincs kivétel, és `model.training` értéke `False`; a meghamisított esetben a `ValueError` szövege `checkpoint hash mismatch: <a módosított fájl SHA-256 hexje>`.

F4 megoldás. (1) `weights_only=False` biztonsági hiba: a checkpoint pickle tetszőleges Python-objektumot futtathat betöltéskor; a javítás `weights_only=True`, és csak saját, megbízható checkpoint betöltése [D09]. (2) `model.train()` aktiválhatja a dropoutot, illetve BatchNorm esetén az eval-statisztikától eltérő viselkedést ad; a javítás `model.eval()` betöltés után. (3) A `reshape(N,24,24,1)` NHWC alakot ad (N, H, W, C sorrendben); a modell NCHW-t vár, így a csatornadimenzió 24 lesz, ami shape-hibát okoz (a futó ellenőrzésben: `RuntimeError: expected input ... to have 1 channels, but got 24 channels instead`). (A HWCN alak $(24, 24, 1, N)$ volna.) A javítás: a modellkártya `layout` mezője szerinti forrásalakból kell kiindulni. Ha az adat tényleg NHWC-ben van tárolva, `np.transpose(x, (0,3,1,2))` ad NCHW-t; ha a forrás NCHW volt, az eredeti tömböt kell a modellnek adni (újratöltés vagy a dokumentált forrásalak megtartása). Az `np.ascontiguousarray` csak memóriafolytonosságot biztosít, tengelyt nem permutál, ezért a rossz reshape-et önmagában nem javítja. Futtatható ellenőrző sorok:

```
with torch.inference_mode():
    ref = model(torch.from_numpy(x_nchw))                                  # NCHW forrás
    got = model(torch.from_numpy(np.ascontiguousarray(np.transpose(x_nhwc, (0, 3, 1, 2)))))
assert torch.allclose(ref, got, atol=1e-5)                                 # a javított útvonal
↪  egyezik
```

A rossz alak ellenőrzése: `model(torch.from_numpy(x_nchw.reshape(N, 24, 24, 1)))` a fenti `RuntimeError`-t adja, nem csendes hibás kimenetet.

F5 megoldásminta. Egy lehetséges protokoll:

1. *Domain-regisztráció:* rögzítsd a JPEG-adatok forrását, képszámot, felbontást és annotációt; minden képhez scene-azonosítót, ha a nézetek függőek.
2. *Input-adaptáció:* a JPEG-et `uint8`-ból `float32 [0,1]` tartományba kell hozni; ezt a modellkártya `preprocessing` mezőjébe fel kell venni. A színes képeket szürkeárnyalatosítani kell, vagy explicit döntést hozni arról, hogy a 3 csatornás bemenetet a modell nem támogatja.
3. *Baseline:* a transzfer előtt futtass egy kisméretű, ugyanazon az új domainen tanított logisztikus regressziót a lelapított pixeleken. Ez nem garantáltan gyenge; ha a mentett modell nem haladja meg, a finomhangolás előtt érdemes megállni.
4. *Metrika:* per-scene accuracy, mert az új domainen is lehetnek összetartozó nézetek; a macro-accuracy az osztályaránytalanság ellenőrzésére.

5. *Érvényességi korlát:* a mentett modell fagyasztott kiértékelése csak akkor értelmezhető, ha az új képek a GlyphLab-tartományhoz hasonló intenzitástartományból és ugyanabból a három orientációs osztályból jönnek. A `validation_scope` mező ilyenkor `"GlyphLab + az új domain neve"` értékre bővül, és a jelentésben külön sorban szerepel, hogy a korábbi benchmark nem erre a domainre szól. A protokoll nem ígérhet előre győztest; negatív eredmény is elfogadható. A kiértékelés döntési szabálya: a mentett modell csak akkor értékelhető az új domainen, ha a preprocessing és a `validation_scope` dokumentált, és a transzfer akkor tekinthető sikeresnek, ha a per-scene accuracy 95%-os bootstrap-intervallumának alsó vége meghaladja a logisztikus baseline pontbecslését; a protokoll öt párosított seedet (0–4) és B=2000 jelenetszintű bootstrap-ismétlést rögzít, az erőforrás CPU.

38.4.8 Tipikus tévedés és ellenpélda

A leggyakoribb érvelési hiba: *„A modell a GlyphLab teszten x pontosságot ért el, tehát a mentett checkpoint önmagában alkalmazási engedély."* Ez a `validation_scope` és az alkalmazási jóváhagyás összemosása. A GlyphLab szintetikus vonalorientációs adat; egy valós vagy más domainhez tartozó képosztályozási feladat más előfordulási arányokkal, más háttérrel és más címkeértelmezéssel rendelkezhet. A modellkártya ezért tartalmazza a `"validation_scope": "csak GlyphLab szintetikus képek; valós felhasználás nem validált"` mezőt. Ha egy felhasználó ezt a mezőt figyelmen kívül hagyva valós orvosi vagy üzleti döntéshez használja a modellt, a hiba nem a checkpoint betöltésében, hanem a szerződés megsértésében van. A benchmark eredménye tudományos bizonyíték a saját adatfelosztáson belül; valós engedélyhez külön domain-validáció, külső adat és döntési küszöb szükséges.

38.4.9 Önellenőrzés és visszatekintés

Ellenőrizd: (1) Meg tudod-e mondani, miért `eval()` és `inference_mode()` együtt szükséges? (2) A referenciafutásban a `margin_sufficient` mező `true`; ha `false` lenne, abból következne-e automatikus predikciós eltérés? (3) Miért nem elég a `model_id` szöveg, ha az `models.py` változott?

Kulcs. (1) Az `eval()` kapcsolja ki a dropoutot és a BatchNorm statisztikafrissítését, az `inference_mode()` pedig az autograd könyvelést; a kettő együtt ad determinisztikus, gradiens nélküli futást. (2) Nem: a `margin_sufficient=false` csak azt jelzi, hogy a top-1 egyezés elegendő feltétele nem teljesül, a predikció ettől még egyezhet. (3) Azért nem, mert a `model_id` nem kódolja a `models.py` forráskódjának változását; a modul hash-e és az architektúra-leírás is kell a betöltés ellenőrzéséhez. Visszatekintés a 37.4. Rétegből kis tanulómodell alfejezetből: a saját réteg integritása az `nn.Parameter` regisztráción múlt; a mentett `state_dict` most ugyanazt a regisztrációt használja a betöltés oldalán. A 34.4. Teacher–student desztilláció alfejezetből: a tanár kiértékelése `eval()` módban futott; ugyanez a mód most a checkpoint visszaállításának feltétele. A 26.4. Detektálási pipeline és hibaelemzés alfejezetből: a detektálási pipeline a transzformált koordinátákat szinkronban tartotta; itt az input séma ellenőrzése ugyanolyan szerződéses védelem.

38.4.10 Rövid összefoglaló

A független inference nem a `state_dict` betöltésével kezdődik, hanem a modellkártya szerződésével: architektúraosztály, input séma, label map, preprocessing és érvényességi kör. A visszatöltés

ellenőrzése két rétegű: a checkpoint SHA-256 egyezése bitazonos paramétereket garantál, a logit-eltérésre pedig a fent rögzített, CPU-float32 környezetben értelmezett $(10^{-4}, 10^{-4})$ tolerancia vonatkozik. A margin-lemma a logit-hibából elegendő feltételt ad a top-1 egyezésre, de nem szükséges. A labor átadja a futtatható `inference.py`-t és a modellkártyát; a kimeneti JSON a hash-, logit- és predikciós szinteket külön mezőkben jelenti.

38.4.11 Következő lecke

A 39.1. Feladat- és annotációs döntés Projekt III-ban a szegmentálási vagy lokalizációs feladat annotációs döntését készítjük elő; az itt bemutatott modellkártya és inference-szerződés mintája lesz annak az alkalmazási riportnak, amely a mentett modell átadását az új feladathoz köti.

38.4.12 Források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D02] PyTorch contributors (2026). *Reproducibility*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/notes/randomness.html. Hozzáférés: 2026-10-30.
- [D09] PyTorch contributors (2026). *Saving and Loading Models*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [D12] PyTorch contributors (2026). *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. A train/eval mód és a BatchNorm statisztikakezelésének háttere. Hozzáférés: 2026-10-30.

39 Projekt III: szegmentálás vagy lokalizáció

39.1 Feladat- és annotációs döntés

39.1.1 Cél, előismeret és induló kérdés

A Projekt III-ban a CNN-t új céllal használjuk: a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-kép osztálycímkéje helyett az alakzat térbeli kiterjedését kell megmondani. A tanulási cél most nem a három orientációs osztály, hanem vagy az előtér pixelcímkéje, vagy az azt befoglaló téglalap. A döntés első lépése nem architekturális, hanem annotációs: mit tekintünk a modell kimenetének, mi a hozzá tartozó ground truth, és milyen metrika méri a teljesítményt. A GlyphLab két annotációt tartalmaz: a `masks.npy` bináris előtér/háttér maszkot, a `metadata.csv` pedig az előtér legkisebb befoglaló téglalapját (bbox). Ebben az egységben e két cél közötti választást alapozzuk meg, rögzítjük a target alakját, a csoportos splitet és az üres eset konvencióját.

Előismeret: a 38.1. Adat, split és előzetes protokoll rögzítette a jelenetszintű splittel és a befagyasztott tesztprotokollt; a 27.1. Pixelszintű klasszifikáció és FCN bevezette a pixelszintű osztályozást és a per-pixel loss nevezőjét; a 26.4. Detektálási pipeline és hibaelemzés a detektálási pipeline-ban a doboztranszformációt; a 27.4. Instance mask és összetett feladat a semantic és instance maszk megkülönböztetését; a 35.1. Saliency és Grad-CAM a Grad-CAM-alapú lokalizációs vizualizációt. A Grad-CAM heurisztikus lokalizációt ad, de nem helyettesíti a tényleges maszk- vagy dobozcélú tanítást.

Induló kérdés. A GlyphLab-képeken az alakzat vékony vonal vagy rúd; a maszk gyakran csak néhány pixel széles. Elég-e a befoglaló téglalap ahhoz, hogy a „hol van az objektum” alkalmazási kérdést megválaszoljuk? A becslésünk: ha a felhasználás pixelmaszkot igényel, akkor nem, mert a bbox nem őrzi meg a maszk alakját, ürességét és több komponensét.

39.1.2 Definíciók és annotációs konvenció

Legyen egy GlyphLab-kép

$$I \in [0;\ 1]^{1\times H\times W}, \qquad H = W = 24,$$

float32, NCHW tárolásban. A képhez két annotáció tartozik.

Semantic mask. A bináris szemantikus maszk minden pixelhez a háttér/előtér osztályt rendeli:

$$M \in \{0;\ 1\}^{1\times H\times W},$$

ahol $M_{0,i,j} = 1$ jelenti az előtér-, $M_{0,i,j} = 0$ a háttérpixelt. A sorindex i a y, az oszlopindex j az x koordináta; mindkettő nullától indexelt. A GlyphLab `masks.npy` uint8 típusú, de a tanítási

target float32-re konvertálható, mert a bináris BCEWithLogitsLoss float targetet vár. A batch alak

$$Y \in \{0;\ 1\}^{N\times 1\times H\times W}.$$

Többosztályos általánosításban $Y \in \{0;\ \dots;\ K-1\}^{N\times H\times W}$ int64 target lenne, a modell pedig $Z \in \mathbb{R}^{N\times K\times H\times W}$ logitsot adna; a GlyphLab bináris előtere miatt itt a $K = 2$ eset egyetlen logit-csatornával, BCEWithLogitsLoss-szal is leírható.

Bbox. A befoglaló téglalap négy egész koordináta:

$$b = (x_{\min}, y_{\min}, x_{\max}, y_{\max}) \in \mathbb{Z}^4,$$

a következő fél-nyílt intervallum-konvencióval:

$$\text{pixel}(i,j) \in b \iff y_{\min} \le i < y_{\max} \text{ és } x_{\min} \le j < x_{\max}.$$

A GlyphLab-szerződés ezt rögzíti: a doboz zérus alapú, fél-nyílt xyxy, és a bbox a maszk nemnulla pixeleinek legkisebb befoglaló téglalapja. Érvényes nemüres dobozra $0 \le x_{\min} < x_{\max} \le W$ és $0 \le y_{\min} < y_{\max} \le H$. A dobozcél batch alakja lokalizációs feladatban $B \in \mathbb{Z}^{N\times 4}$; a regressziós fej kimenete valós $\hat{B} \in \mathbb{R}^{N\times 4}$, a cél viszont egész koordinátájú. A koordináták pixel egységben értendők. A lokalizációs fej emellett általában egy objektum-jelenlét logitot is ad, hogy az üres képeket kezelni tudja.

Üres eset. Ha egy képen nincs egyetlen előtérpixel sem, a maszkcél a teljes nulla tömb $M = 0$, a dobozcél pedig a szentinel

$$b_\emptyset = (-1;\ -1;\ -1;\ -1).$$

A szentinel helyi, explicit sémadöntés, nem matematikai szükségszerűség. Indoka kettős: (a) a negatív koordináta egyetlen érvényes dobozban sem fordulhat elő, ezért az üres eset egyetlen összehasonlítással felismerhető, és nem keverhető össze semmilyen valódi dobozzal; (b) a doboz-regressziós veszteségből az ilyen célokat kizárjuk, így az érvénytelen állapot nem kerül a loss számlálójába. A (0; 0; 0; 0) a fél-nyílt konvencióban nulla területű, érvénytelen doboz — nem egy pixelt fed le (a (0; 0) pixel doboza (0; 0; 1; 1)) —, tehát elvben szintén jelölhetne ürességet; a negatív szentinelt a fenti két gyakorlati ok miatt választjuk. A 39.3. Geometriai kontroll és metrika `validate_box` függvénye és a 39.4. Alkalmazási riport és átadás exportja ugyanezt a szentinelt fogadja el, és üres predikált maszkra ezt írja.

Csoportos split. A maszk és a bbox a képhez tartozó annotáció, ezért a splitet nem szabad képszinten újra véletlenezni. A megfigyelési egység a `scene_id`: egy jelenet mindkét view-ja és minden annotációja ugyanabba a train/val/test részbe kerül. A 38.1. Adat, split és előzetes protokoll már levezette, hogy a jelenetszintű split a megnevezett használati helyzetben torzítatlan tesztbecslőt ad, míg a képszintű véletlen split a két view közötti hasonlóság miatt szivárgást okozna. A Project III ezt a splitet nem változtatja meg: 120/30/30 jelenet, osztályrétegzetten generálva.

Annotation convention. A könyvben végig a nulla alapú, sor-major pixelindexelést és a fél-nyílt `[min,max)` intervallumot használjuk. A kép bal felső pixele a (0; 0) cella; a j oszlop a j-edik egységcellát fedi le, ezért a jobb szélső oszlopindex $W - 1$. A bbox a cellákat teljes egészében tartalmazza, nem a cellák közötti élvonalat adja meg.

39.1.3 Konzisztens koordináta- és üreseset-definíció

A két annotáció közötti átjárást két leképezéssel rögzítjük. Legyen $F = \{(i,j) : M_{0,i,j} = 1\}$ az előtérpixelek halmaza.

Definíció (maszkból bbox, β). Ha $F \neq \emptyset$, akkor

$$x_{\min} = \min_{(i,j)\in F} j, \qquad x_{\max} = 1 + \max_{(i,j)\in F} j,$$
$$y_{\min} = \min_{(i,j)\in F} i, \qquad y_{\max} = 1 + \max_{(i,j)\in F} i.$$

Ha $F = \emptyset$, akkor $\beta(M) = (-1;\ -1;\ -1;\ -1)$.

Definíció (bboxból maszk, γ). Érvényes $b = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$ esetén

$$\gamma(b)_{0,i,j} = \begin{cases} 1, & \text{ha } y_{\min} \le i < y_{\max} \text{ és } x_{\min} \le j < x_{\max}, \\ 0, & \text{különben.} \end{cases}$$

A szentinel esetre $\gamma((-1;\ -1;\ -1;\ -1)) = 0$.

Állítás (befoglaló téglalap konzisztenciája). A fenti definíciókkal:

1. Ha $F \neq \emptyset$, akkor $\beta(M)$ érvényes bbox, és minden $(i,j) \in F$ pixel benne van a $\beta(M)$-beli fél-nyílt téglalapban.
2. $\gamma(\beta(M)) \geq M$ pontonként; egyenlőség pontosan akkor, ha M a saját befoglaló téglalapjának minden celláján 1.
3. Minden érvényes b bboxra $\beta(\gamma(b)) = b$.
4. Ha $F = \emptyset$, akkor $\gamma(\beta(M)) = 0 = M$.

Bizonyítás. (1) Véges nemüres halmazon a min és max létezik. A definícióból $x_{\min} \le x_{\max} - 1$, tehát $x_{\min} < x_{\max}$; hasonlóan $y_{\min} < y_{\max}$. A min/max tulajdonsága miatt tetszőleges $(i,j) \in F$-re $x_{\min} \le j \le x_{\max} - 1$, azaz $j \in [x_{\min}, x_{\max})$, és $y_{\min} \le i \le y_{\max} - 1$, azaz $i \in [y_{\min}, y_{\max})$.

(2) Az (1) állítás szerint minden F-beli cellában $\gamma(\beta(M)) = 1 = M$, a többi cellában $\gamma(\beta(M)) \geq 0 = M$. Egyenlőség pontosan akkor áll, ha a befoglaló téglalap minden cellája F-ben van, tehát nincs benne háttérpixel.

(3) Legyen b érvényes. Ekkor $F_\gamma = \{(i,j) : y_{\min} \le i < y_{\max},\ x_{\min} \le j < x_{\max}\}$ nemüres, mert mindkét intervallum hossza legalább 1. A sor-minimum $y_{\min}$, a sor-maximum $y_{\max} - 1$; a $+1$ szabály így $y_{\max}$-ot ad vissza. Az oszlopokra analóg módon $x_{\max}$-ot kapunk.

(4) A szentinel definíciójából közvetlenül adódik. □

A levezetés mutatja, hogy a fél-nyílt konvenció és a $+1$-es szabály együtt konzisztens: a bboxból visszaállított maszk befoglaló téglalapja pontosan az eredeti bbox, míg a maszkból származtatott bboxból visszaállított maszk tartalmazza az eredeti maszkot (pontonként legalább akkora: $\gamma(\beta(M)) \geq M$). A kettő pontosan akkor esik egybe, ha a maszk maga is téglalap alakú. Ez a geometriai tény a célválasztás központi érve: a bbox cél csak téglalap-őrző, a maszk cél alak-őrző.

A választott fő annotáció. A Project III alapváltozata a GlyphLab bináris foreground maszkját választja szemantikus szegmentálási célként, mert (a) a `masks.npy` teljes pixelcímkét ad, (b) a bbox abból egyértelműen származtatható, fordítva nem, és (c) a maszkmetrika a vékony vonalak tényleges alakját méri. A projekt végig ezen a főágon marad. A 39.3. Geometriai kontroll és metrika és a 39.4. Alkalmazási riport és átadás dobozai nem egy külön tanított lokalizációs fej

kimenetei, hanem a predikált maszkból a fenti β leképezéssel származtatott xyxy dobozok: a maszkot a $\sigma(z) > 0{,}5$ küszöb adja, üres predikált maszkra a doboz a $(-1;\ -1;\ -1;\ -1)$ szentinel, és az exportot a 39.4. Alkalmazási riport és átadás `export_delivery.py` szkriptje végzi. A maszkon mért IoU neve `mask_iou`, a dobozon mérté `bbox_iou`; a fő maszkmetrika a Dice, az IoU kiegészítő, konzisztencia-ellenőrző metrika. Önálló dobozregressziós fej tanítása a projektnek nem része; legfeljebb transzferfeladatként szerepel.

39.1.4 Eredeti kézi kis példa

Tekintsük az 5×5-ös maszkot:

```
0 0 0 0 0
0 1 1 0 0
0 1 0 1 0
0 0 1 0 0
0 0 0 0 0
```

Az előtérpixelek halmaza:

$$F = \{(1;\ 1);\ (1;\ 2);\ (2;\ 1);\ (2;\ 3);\ (3;\ 2)\}.$$

A definíció szerint

$$x_{\min} = 1, \quad x_{\max} = x_{\max\ \text{index}} + 1 = 3 + 1 = 4, \qquad y_{\min} = 1, \quad y_{\max} = 3 + 1 = 4,$$

tehát a bbox $(1;\ 1;\ 4;\ 4)$. A befoglaló téglalap területe $(4-1)(4-1) = 9$, míg a maszk területe $|F| = 5$. A bboxból visszaállított maszk:

```
0 0 0 0 0
0 1 1 1 0
0 1 1 1 0
0 1 1 1 0
0 0 0 0 0
```

A két maszk metszete 5 pixel; a maszk-Dice:

$$\text{Dice} = \frac{2 \cdot 5}{5 + 9} = \frac{10}{14} \approx 0{,}7143,$$

a maszk-IoU pedig $\frac{5}{9} \approx 0{,}5556$. Ha viszont a célt bboxban mérnénk, a predikált befoglaló téglalap pontosan megegyezik a ground truth bboxszal, tehát a bbox-IoU 1 lenne. Ugyanaz a predikció a bbox-metrikában tökéletes, a maszk-metrikában gyenge; ez a későbbi tipikus tévedés alapja.

Most üres eset: legyen $M = 0$. Ekkor $\beta(M) = (-1;\ -1;\ -1;\ -1)$, és $\gamma(\beta(M)) = 0$, tehát a körbejárás az üres maszkot adja vissza. A maszk-Dice-t az üres–üres esetre 1-nek definiáljuk; ha csak az egyik oldal üres, a Dice 0, mert a metszet nulla, a nevező pedig pozitív.

39.1.5 Konkrét labor: annotációs specifikáció és baseline

A labor egy specifikációs és konzisztencia-ellenőrző szkriptet készít. Nem tanít modellt: az a 39.2. Encoder–decoder vagy lokalizációs fej feladata. A labor célja, hogy a Project III target-szerződése géppel ellenőrzött legyen, és a későbbi tanítás ne indulhasson el ellentmondásos annotációval.

Munkakönyvtár és parancs. Hozzuk létre a `lab/project3` könyvtárat, mentsük a szkriptet `annotation_spec.py` néven, majd futtassuk:

```
mkdir -p lab/project3
# a fenti kód lab/project3/annotation_spec.py néven mentendő
cd lab/project3
python annotation_spec.py --data ../data
```

A GlyphLab adatfájlok a `lab/data` könyvtárban vannak. A környezet Python 3.10+, NumPy, nincs GPU, nincs külső súly. A seed itt nem releváns, mert a szkript determinisztikus ellenőrzést végez; az adatgenerátor seedjét a `manifest.json` rögzíti.

```
#!/usr/bin/env python3
# lab/project3/annotation_spec.py
"""Project III annotation specification and consistency check."""
import argparse
import csv
import json
from collections import defaultdict
from pathlib import Path

import numpy as np

H, W = 24, 24
EMPTY_BBOX = (-1, -1, -1, -1)


def mask_to_bbox(mask: np.ndarray) -> tuple[int, int, int, int]:
    """Binary mask (H, W) -> half-open xyxy bbox or EMPTY_BBOX."""
    if not np.all(np.isin(mask, [0, 1])):
        raise ValueError("mask values must be 0/1")
    ys, xs = np.nonzero(mask)
    if len(xs) == 0:
        return EMPTY_BBOX
    return int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1


def dice(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Dice for two binary masks. Empty-empty convention: 1."""
    yt = y_true.astype(np.float64)
    yp = y_pred.astype(np.float64)
    inter = float(np.sum(yt * yp))
    denom = float(np.sum(yt) + np.sum(yp))
    if denom == 0.0:
        return 1.0
    return 2.0 * inter / denom


def bbox_iou(a: tuple[int, int, int, int],
             b: tuple[int, int, int, int]) -> float:
    """Half-open xyxy dobozok IoU-ja; üres dobozra 0."""
    if a == EMPTY_BBOX or b == EMPTY_BBOX:
        return 0.0
    iw = max(0, min(a[2], b[2]) - max(a[0], b[0]))
    ih = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = iw * ih
```

```
    union = ((a[2] - a[0]) * (a[3] - a[1])
             + (b[2] - b[0]) * (b[3] - b[1]) - inter)
    return inter / union


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", type=Path, default=Path("../data"))
    parser.add_argument("--out", type=Path, default=Path("spec.json"))
    args = parser.parse_args()

    images = np.load(args.data / "images.npy")
    masks = np.load(args.data / "masks.npy")
    with open(args.data / "metadata.csv", newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    assert [int(r["index"]) for r in rows] == list(range(len(rows))), \
        "a metadata indexe nem a 0..N-1 sorrend"

    assert images.shape == (360, 1, H, W), images.shape
    assert masks.shape == (360, 1, H, W), masks.shape
    assert masks.dtype == np.uint8, masks.dtype
    assert np.all(np.isin(masks, [0, 1])), "non-binary mask value"

    mismatches = []
    for r in rows:
        i = int(r["index"])
        b_file = (int(r["xmin"]), int(r["ymin"]),
                  int(r["xmax"]), int(r["ymax"]))
        b_mask = mask_to_bbox(masks[i, 0])
        if b_mask != b_file:
            mismatches.append((i, b_file, b_mask))

    by_scene = defaultdict(set)
    for r in rows:
        by_scene[r["scene_id"]].add(r["split"])
    scene_conflicts = [s for s, sp in by_scene.items() if len(sp) > 1]

    split_scenes = {s: {r["scene_id"] for r in rows if r["split"] == s}
                    for s in ("train", "val", "test")}
    scene_counts = {s: len(v) for s, v in split_scenes.items()}
    assert scene_counts == {"train": 120, "val": 30, "test": 30}, scene_counts

    empty_count = sum(1 for i in range(len(masks)) if np.sum(masks[i]) == 0)

    val_idx = [int(r["index"]) for r in rows if r["split"] == "val"]
    assert len(val_idx) == 60, len(val_idx)
    all_foreground = np.ones((H, W), dtype=np.float64)
    empty_pred = np.zeros((H, W), dtype=np.float64)
    d_empty = [dice(masks[i, 0], empty_pred) for i in val_idx]
    d_all = [dice(masks[i, 0], all_foreground) for i in val_idx]

    full_box = (0, 0, W, H)
    iou_full = []
    for i in val_idx:
        b_gt = mask_to_bbox(masks[i, 0])
        if b_gt != EMPTY_BBOX:
            iou_full.append(bbox_iou(full_box, b_gt))
    zero_mask = np.zeros((H, W), dtype=np.uint8)

    spec = {
        "target": "semantic_mask",
        "input": {"shape": [1, H, W], "dtype": "float32", "range": [0.0, 1.0]},
        "target_shape": [1, H, W],
```

```
        "target_dtype": "float32",
        "target_values": [0, 1],
        "coordinate_convention":
            "zero-indexed; y=row, x=column; bbox half-open [xmin,xmax) x [ymin,ymax)",
        "empty_case": {
            "mask": "all-zero array",
            "bbox_sentinel": list(EMPTY_BBOX),
            "metadata_bbox_sentinel": list(EMPTY_BBOX),
            "dice_empty_empty": float(dice(zero_mask, zero_mask)),
            "dice_one_empty": float(dice(np.ones_like(zero_mask), zero_mask)),
        },
        "split": {
            "unit": "scene_id",
            "train_scenes": scene_counts["train"],
            "val_scenes": scene_counts["val"],
            "test_scenes": scene_counts["test"],
        },
        "baselines": {
            "empty_mask": {"mean_val_dice": float(np.mean(d_empty))},
            "all_foreground": {"mean_val_dice": float(np.mean(d_all))},
            "bbox_full_image": {"mean_val_iou": float(np.mean(iou_full)),
                                "n_nonempty_val": len(iou_full)},
        },
        "consistency": {
            "metadata_bbox_matches_mask": len(mismatches) == 0,
            "mismatch_count": len(mismatches),
            "empty_mask_count": empty_count,
            "scene_split_conflicts": len(scene_conflicts),
        },
    }
    with open(args.out, "w", encoding="utf-8") as f:
        json.dump(spec, f, indent=2, ensure_ascii=False)

    ok = len(mismatches) == 0 and len(scene_conflicts) == 0
    print("CONSISTENCY_OK" if ok else "CONSISTENCY_FAIL")
    print(json.dumps(spec, indent=2, ensure_ascii=False))


if __name__ == "__main__":
    main()
```

Beadandó eredmény. A labor kimenete a `lab/project3/spec.json`, amely rögzíti a `semantic_mask` targetet, a target shape-et, a koordináta- és üreseset-konvenciót (az üres maszkok `metadata.csv`-beli doboza a $(-1;\ -1;\ -1;\ -1)$ szentinel, összhangban az A. A GlyphLab adatkészlet függelékkel), a jelenetszintű splitet és a három baseline-t. A konzol `CONSISTENCY_OK` sorának meg kell jelennie; ez azt ellenőrzi, hogy a `metadata.csv`-beli bbox minden képre pontosan megegyezik a maszkból származtatott befoglaló téglalappal (üres maszkra a szentinel), és egyetlen jelenet sem került több splitbe. A `baselines.empty_mask.mean_val_dice` várhatóan 0, ha minden val maszk nemüres; az `all_foreground` baseline kép-szinten a Dice $= 2f/(1+f)$ képletet követi, ahol f az adott maszk kitöltöttsége. A mért 0,0412 így átlagosan $f \approx 0{,}021$ kitöltésnek felel meg (a val maszkok átlagos kitöltöttsége 0,0211), ami egyezik a 39.3. Geometriai kontroll és metrika 0,0201–0,0211 közötti előtérarányaival; a későbbi modellnek ezt kell jelentősen meghaladnia. Az értékeket a szkript a tényleges val maszkokból számolja. **Futási jegyzőkönyv** (eredeti GlyphLab-adat, Python 3.12.3, NumPy 2.5.4, CPU): a konzol `CONSISTENCY_OK`-t írt, a `mismatch_count`, az `empty_mask_count` és a `scene_split_conflicts` egyaránt 0, a split jelenetszámai 120/30/30, az `empty_mask` baseline val-Dice értéke 0, az `all_foreground` baseline-é 0,0412 (pontosan 0,041173 ...), a teljes képnyi doboz baseline val-IoU-ja 0,0523 (60 nemüres val maszkon). A `spec.json` releváns mezői a futásból (a teljes fájlból kivonatolva):

```
CONSISTENCY_OK
```

```
empty_case: bbox_sentinel = metadata_bbox_sentinel = (-1,-1,-1,-1)
            dice_empty_empty = 1.0, dice_one_empty = 0.0
split:      scene_id egység, train/val/test = 120/30/30 jelenet
baselines:  empty_mask Dice = 0.0
            all_foreground Dice = 0.04117347741268433
            bbox_full_image IoU = 0.05231481481481482 (n = 60)
consistency: mismatch_count = 0, empty_mask_count = 0, scene_split_conflicts = 0
```

A baseline a val spliten mérhető, mert a val a modellválasztási halmaz; a végső, befagyasztott modell tesztkiértékelése a 39.4. Alkalmazási riport és átadás alfejezetben történik majd egyszer, a clean test maszkjain.

39.1.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza el a szemantikus maszk, a bbox és az instance maszk különbségét. Adjon egy kis, 3×3-as bináris maszkpéldát, amelyben ugyanaz a szemantikus maszk két különböző instance-annotációt enged meg. Beadandó: legfeljebb féloldalas saját magyarázat és a konkrét példa.

F2 (levezetési, nehézség 2). Adott a 4×4-es maszk:

```
1 1 0 0
1 0 0 0
0 0 1 0
0 0 0 0
```

Számolja ki a $\beta(M)$ bboxot és a $\gamma(\beta(M))$ kitöltött maszkot. Számolja ki a két maszk közötti Dice-t és IoU-t. Bizonyítsa a 39.1.3. szakasz állításának 2. pontját: $\gamma(\beta(M)) \geq M$ pontonként, és adja meg, mikor áll egyenlőség. Beadandó: a bbox, a kitöltött maszk, a két metrikaérték és a bizonyítás lépései.

F3 (implementációs, nehézség 2). Írjon `mask_to_bbox` és `dice` függvényt a fenti konvencióval. Tesztelje három esetre: a 39.1.4. szakasz 5×5-ös példájára, az üres maszkra és az egyetlen $(i, j) = (2;\ 3)$ pixelt tartalmazó maszkra. Beadandó: futtatható kód és a három kimenet.

F4 (hibakeresési, nehézség 2). Valaki a következő hibás függvényt írta:

```python
def wrong_bbox(mask):
    ys, xs = np.nonzero(mask)
    return (xs.min(), ys.min(), xs.max(), ys.max())
```

Magyarázza meg a hiba okát, mutassa meg egy olyan 2×2-es bemeneten, hogy a visszaadott bbox a fél-nyílt konvencióban nem tartalmazza az összes előtérpixelt, és adja meg a javítást. Mi a következménye a területre és az IoU-ra? Beadandó: hibalokalizáció, minimális reprodukció, javított kód és ellenőrzés.

F5 (kísérlettervezési transzfer, nehézség 3). Tervezzen egy GlyphLab-kísérletet, amely eldönti, hogy a választott alkalmazási kérdésre a maszkcél vagy a bboxcél illik jobban. Nevezze meg a kérdést, a baseline-t, a splitet, a metrikát és az érvényességi korlátot. Az előre megadott győztes nem kötelező; a negatív eredmény is elfogadható. Beadandó: egyoldalas protokollterv.

39.1.7 Megoldások és indoklás

F1. A szemantikus maszk minden pixelhez osztálycímkét rendel; a bbox csak négy koordinátát ad; az instance maszk ezen felül az azonos osztályú objektumokat is elkülönülő példányokra bontja. Példa: a 3×3-as maszk

```
1 1 0
1 1 0
0 0 0
```

szemantikusan négy előtérpixelt tartalmaz, és a négy pixel egyetlen összefüggő 2×2-es blokk. Instance-szinten ez lehet egyetlen objektum, de lehet két összeérő objektum is (például a bal és a jobb oldali 2×1-es oszlop); a szemantikus maszk önmagában nem mondja meg, melyik felosztás a helyes, ezért az instance-annotáció nem vezethető le belőle. Ez a 27.4. Instance mask és összetett feladat tanulsága.

F2. A 4×4-es maszk előtere $F = \{(0;\ 0);\ (0;\ 1);\ (1;\ 0);\ (2;\ 2)\}$. A min/max:

$$x_{\min} = 0,\ x_{\max} = 2 + 1 = 3, \qquad y_{\min} = 0,\ y_{\max} = 2 + 1 = 3,$$

tehát $b = (0;\ 0;\ 3;\ 3)$. A $\gamma(b)$ kitöltött maszk:

```
1 1 1 0
1 1 1 0
1 1 1 0
0 0 0 0
```

A metszet 4, a két terület 4 és 9, tehát

$$\text{Dice} = \frac{2 \cdot 4}{4 + 9} = \frac{8}{13} \approx 0{,}6154, \qquad \text{IoU} = \frac{4}{9} \approx 0{,}4444.$$

A bizonyítás: a β definíciójából minden F-beli pixel koordinátái a $[x_{\min}, x_{\max})$ és $[y_{\min}, y_{\max})$ intervallumba esnek, ezért $\gamma(\beta(M))$ ott 1 értéket vesz fel. A bboxon kívüli pixeleken $M = 0$, és $\gamma \in \{0;\ 1\}$, tehát $\gamma(\beta(M)) \geq M$ minden cellában. Egyenlőség pontosan akkor, ha a bboxon belül minden cella M szerint 1; ez az az eset, amikor a maszk téglalapot tölt ki.

F3. A függvények a labor szkriptből vehetők. A tesztek:

```
import numpy as np

EMPTY = (-1, -1, -1, -1)

def mask_to_bbox(mask):
    ys, xs = np.nonzero(mask)
    if len(xs) == 0:
        return EMPTY
    return int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1

def dice(y_true, y_pred):
    yt = y_true.astype(np.float64); yp = y_pred.astype(np.float64)
    inter = float(np.sum(yt * yp)); denom = float(np.sum(yt) + np.sum(yp))
    return 1.0 if denom == 0.0 else 2.0 * inter / denom

m1 = np.array([
    [0,0,0,0,0],
    [0,1,1,0,0],
    [0,1,0,1,0],
    [0,0,1,0,0],
```

```
    [0,0,0,0,0]])
m2 = np.zeros((5,5), dtype=np.uint8)
m3 = np.zeros((5,5), dtype=np.uint8); m3[2,3] = 1

print(mask_to_bbox(m1), dice(m1, np.zeros_like(m1)))
print(mask_to_bbox(m2), dice(m2, np.zeros_like(m2)))
print(mask_to_bbox(m3), mask_to_bbox(np.zeros_like(m3)))
```

Várt kimenet: (1; 1; 4; 4), Dice a nulla predikcióval 0; (−1; −1; −1; −1), Dice 1; (3; 2; 4; 3), üresre (−1; −1; −1; −1).

F4. A hiba oka: a `max` koordinátához nem ad hozzá 1-et, ezért a fél-nyílt intervallum felső határa a maximális indexen áll, és a legnagyobb indexű sort vagy oszlopot (benne a maximum-indexű előtérpixelekkel) kizárja. Példa: $M = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ esetén az egyetlen előtérpixel a (0; 0), a hibás függvény mégis (0; 0; 0; 0)-t ad; ez nulla területű, érvénytelen doboz, mert a $[0;\ 0) \times [0, 0)$ intervallum egyetlen cellát sem tartalmaz. A helyes eredmény (0; 0; 1; 1). Nagyobb maszkon a hiba a maximális indexű sor/oszlop egy pixel széles sávját vágja le: a F2 4×4-es maszkjára a hibás bbox (0; 0; 2; 2), területe 4 a helyes 9 helyett, és a belőle kitöltött maszkból kimarad a (2; 2) pixel. A valódi maszkhoz mérve ez torzítja a metrikát, de nem monoton módon: ugyanezen a példán a hibás dobozból visszaállított maszk Dice-a $2 \cdot 3/(4+4) = 0{,}75$, IoU-ja $3/(4+4-3) = 0{,}6$, szemben a helyes $\frac{8}{13} \approx 0{,}6154$ és $\frac{4}{9} \approx 0{,}4444$ értékkel; a hiba tehát felül is becsülhet, ezért a konvenciót a mérés előtt kell rögzíteni. A javítás az üres eset kezelésével együtt:

```
EMPTY_BBOX = (-1, -1, -1, -1)

def xyxy_bounds(xs, ys):
    if len(xs) == 0:
        return EMPTY_BBOX
    return (int(xs.min()), int(ys.min()),
            int(xs.max()) + 1, int(ys.max()) + 1)
```

A labor `mask_to_bbox` függvénye ugyanezt a `np.nonzero` által adott `ys`, `xs` tömbökkel teszi, és üres esetben `EMPTY_BBOX`-ot ad vissza, nem dob `ValueError`-t. Az ellenőrzés a F3 három tesztjével történik: az 5×5-ös példa, az üres maszk és az egyetlen (2; 3) pixel esete.

F5. Egy lehetséges terv. Kérdés: „A GlyphLab vékony vonalainak térbeli azonosításához a pixelszintű maszk vagy a befoglaló bbox ad-e a felhasználási szerződésben jobb döntéshozatalt?” Baseline: a `spec.json` három baseline-t rögzít – üres maszk, teljes kép mint all-foreground, valamint a lokalizációs ágon a teljes képnyi fix bbox (a maszkágon Dice, a dobozágon bbox-IoU). Split: a 38.1. Adat, split és előzetes protokoll jelenetszintű, osztályrétegzett 120/30/30; modellválasztás csak clean val-on, teszt egyszer, befagyasztva. Metrika: a maszkágra képszintű binary Dice, üres–üres konvencióval; a bboxágra bbox-IoU. A két ág összehasonlításakor a bboxági predikciót γ-val maszkká alakítva is kiértékeljük Dice-on, de ezt másodlagos, target-eltérő diagnosztikának tekintjük: a γ kitöltés előteret ad hozzá, ezért a kapott Dice általában nem alsó és nem is felső korlátja a bbox-metrikának, és nem helyettesíti a bbox-IoU-t. Érvényességi korlát: a GlyphLab szintetikus, egyobjektumos, zajos grayscale adat; az eredmény nem általánosítható többobjektumos vagy természetes képekre. Az intervallumokat jelenetszintű bootstrap adhatja; a döntéshez a felhasználási következményt is rögzíteni kell, nem csak a pontszámot. A tanítás a Projekt III `SmallSegUNet` modelljével, Adam-mal ($\eta = 10^{-3}$), batch 16 mellett, 30 epochon át, öt seeddel (20261020–20261024) történik. Döntési szabály: a maszkcél akkor választandó, ha az előre rögzített költségmátrix szerinti becsült döntési költsége legalább 5%-kal alacsonyabb a bboxcélénál; az erőforrás CPU, öt seed és két ág.

39.1.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. „A bboxra váltva a régi maszkmetrika automatikusan megfelelő marad.” Ez hamis. Legyen a ground truth maszk L-alak: 4×4-es kép, a felső sor és a bal oszlop 1, a többi 0; területe 7. A ground truth bbox $(0;\ 0;\ 4;\ 4)$. Ha a modell a teljes befoglaló téglalapot adja, a bbox-IoU 1, de a maszk-Dice a valós L-maszk és a kitöltött téglalap között $\frac{2 \cdot 7}{7+16} = \frac{14}{23} \approx 0{,}609$. A két target tehát más hibát definiál: ugyanaz az output az egyik metrikában tökéletes, a másikban gyenge. A metrikát a targettel együtt kell rögzíteni, nem a régi target után húzni.

Önellenőrzés. (1) Miért nem elég a bbox, ha a kérdés a pixelmaszk? (2) Hogyan definiáltuk az üres bboxot és miért nem $(0;\ 0;\ 0;\ 0)$? (3) Mit jelent a fél-nyílt $[x_{\min}, x_{\max})$ a pixelcella-koordinátákban? (4) Miért `scene_id` a split egysége? (5) Mikor egyenlő $\gamma(\beta(M))$ és M?

Rövid válaszok. (1) Mert a bbox elveszti a maszk alakját, lyukjait és komponenseit. (2) A szentinel $(-1;\ -1;\ -1;\ -1)$; helyi sémadöntés, mert a negatív koordináta egyetlen érvényes dobozban sem fordul elő (könnyen felismerhető), és az ilyen célokat a dobozregressziós lossból kizárjuk. A $(0;\ 0;\ 0;\ 0)$ nulla területű, érvénytelen doboz, nem egy pixelé. (3) A cellát indexeljük, és a felső határ nyílt, így a jobb alsó cella $W-1$, $H-1$ indexig tart. (4) A két view közös jelenete függő megfigyelés; a képszintű split szivárgást okozna. (5) Pontosan akkor, ha a maszk téglalapot tölt ki.

Visszatekintés. A 38.1. Adat, split és előzetes protokoll rögzítette a jelenetszintű osztályréteg-zett splitet és a teszt egyszeri, befagyasztott használatát; a 27.1. Pixelszintű klasszifikáció és FCN a pixel-CE loss nevezőjét az érvényes pixelekre vezette le; a 35.1. Saliency és Grad-CAM Grad-CAM-vizualizációja heurisztikus lokalizáció, amely nem pótolja a tanított maszkcélt. A maszk itt rögzített bináris célja a 39.2. Encoder–decoder vagy lokalizációs fej alfejezetben a modellkimenet és a loss szerződésévé válik.

39.1.9 Rövid összefoglaló

A Project III annotációs döntése a kérdéshez köti a célt. A szemantikus maszk minden pixelhez bináris előtér/háttér címkét rendel: batch alakban $Y \in \{0;\ 1\}^{N \times 1 \times 24 \times 24}$. A bbox négy fél-nyílt xyxy koordináta, a maszk legkisebb befoglaló téglalapja; üres célja $(-1;\ -1;\ -1;\ -1)$, míg a maszk üres célja a nulla tömb. A 39.1.3. szakasz állítása belátja, hogy a bboxból visszaállított maszk tartalmazza az eredeti maszkot, és pontosan téglalapmaszkra egyeznek meg. A labor a metadata-bbox és a maszk konzisztenciáját ellenőrzi, baseline-ként üres maszkot, teljes előteret és teljes képnyi dobozt ad. A metrika target-specifikus: bbox-IoU és maszk-Dice nem felcserélhető.

39.1.10 Következő lecke

A 39.2. Encoder–decoder vagy lokalizációs fej az itt rögzített `semantic_mask` targetre építi és tanítja az encoder–decoder szegmentáló hálót; a klasszikus encoder–decoder szegmentálási séma referencia-munkája az U-Net [P14]. A target shape, az üreseset-konvenció és a jelenetszintű split ott válik a loss, a modellkimenet és a tanítási ciklus szerződésévé.

39.1.11 Felhasznált források

- [P14] Ronneberger, O., Fischer, P., Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* arXiv:1505.04597. https://arxiv.org/abs/1505.04597.

39.2 Encoder–decoder vagy lokalizációs fej

39.2.1 Cél, előismeret és induló kérdés

A 39.1. Feladat- és annotációs döntés egységben rögzítettük a projekt annotációs döntését. Ebben az alfejezetben a szegmentáló főutat építjük meg: az encoder–decoder törzs után egy *spatial head* állítja elő a pixelszintű logit-maszkot, és a célmaszkkal konzisztens loss redukcióját vezetjük le. Előismeret a 27.2. U-Net, encoder–decoder és skip U-Net-alakok és a 38.2. Modell, training és best checkpoint tanítási szerződés; a 35.2. Magyarázat sanity check magyarázat-ellenőrzésből csak annyit használunk, hogy minden kimeneti állítást a modell tényleges számítási útjához kötünk.

Induló kérdés: miért nem elég egy `Linear(576, 576)` réteg, ha a kép és a maszk egyaránt $576 = 24 \cdot 24$ pixel? A válasz a térbeli indexelésben és a paramétermegosztásban van: a kép pontjainak sorrendje hordozza a geometriát, egy teljesen összekötött réteg ezt a sorrendet el tudja rontani, a konvolúciós fej viszont megtartja.

39.2.2 A spatial head és a választott modell

Legyen a dekódoló utolsó jellemzőtérképe

$$F \in \mathbb{R}^{N \times C_f \times H_f \times W_f}.$$

A bináris szegmentáló fej egy 1×1-es konvolúció, amely minden térbeli pozícióban azonos affin leképezést alkalmaz a csatornavektorra:

$$Z_{n,1,i,j} = b + \sum_{c=1}^{C_f} K_c \, F_{n,c,i,j},$$

ahol $K \in \mathbb{R}^{1 \times C_f \times 1 \times 1}$ és $b \in \mathbb{R}$. A kimenet egyetlen logitcsatorna, mert egy előtérosztályunk van:

$$Z \in \mathbb{R}^{N \times 1 \times H \times W}.$$

A cél ugyanilyen térbeli alakú bináris maszk:

$$Y \in \{0;\ 1\}^{N \times 1 \times H \times W}.$$

A főútban $H = W = 24$, mert a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab maszkjai natív 24×24 felbontásúak. A választott kis modell a U-Net elvű encoder–decoder séma [P14] játékméretű változata:

Lépés	Réteg	Kimenet alakja
bemenet	X	$(N, 1, 24, 24)$
enc1	Conv2d $1 \to 8$, $k = 3$, $p = 1$ + ReLU	$(N, 8, 24, 24)$
pool1	MaxPool2d(2)	$(N, 8, 12, 12)$
enc2	Conv2d $8 \to 16$, $k = 3$, $p = 1$ + ReLU	$(N, 16, 12, 12)$
pool2	MaxPool2d(2)	$(N, 16, 6, 6)$

Lépés	Réteg	Kimenet alakja
bottleneck	Conv2d $16 \to 32$, $k=3$, $p=1$ + ReLU	$(N, 32, 6, 6)$
up	Upsample(size=(24,24), bilinear)	$(N, 32, 24, 24)$
concat	az up és enc1 csatorna szerint	$(N, 40, 24, 24)$
dec	Conv2d $40 \to 16$, $k=3$, $p=1$ + ReLU	$(N, 16, 24, 24)$
head	Conv2d $16 \to 1$, $k=1$	$(N, 1, 24, 24)$

A paraméterszám csak konvolúciós súly és bias: $80 + 1168 + 4640 + 5776 + 17 = 11681$; az upsamplingnek és a poolingnak nincs paramétere. A lokalizációs alternatíva ebben az egységben nem a főút: ahhoz a head például dobozparaméteres térképet vagy egy utólagos regresszorfejet adhatna. Az itteni szegmentáló fej önálló oktatási példa, nem a [D18] dokumentáció detektálási receptjének reprodukciója.

39.2.3 A kimeneti alak levezetése

A GlyphLab-kép 24×24-es. Az általános szimmetrikus paddinges Conv2d-alakképlet:

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1,$$

ahol H a bemeneti magasság, k a kernel mérete, p a padding, d a dilation, s a stride (a szélességre ugyanez a képlet érvényes W-vel). A $k=3$, $p=1$, $s=1$, $d=1$ esetben $H_{enc1} = 24$. A $k=2$, $s=2$ MaxPool2d esetében $p=0$ és $d=1$ mellett $H_{pool1} = 12$, majd $H_{pool2} = 6$. A bottleneck tehát 6×6-os, 32 csatornával.

A dekódoló upsampling lépése explicit méretet kap: `Upsample(size=(24,24), mode="bilinear", align_corners=False)`. Bilineáris átmintavételezésnél az `align_corners=False` konvenció félegységnyi eltolással helyezi a kimeneti rácspontokat a forrásrácsra; a `size=(24,24)` argumentum azonban nem hagy kerekítési szabadságot, a kimenet térbeli alakja pontosan a kért 24×24. A kódoló első aktivációja is 24×24-es, ezért a csatorna szerinti összefűzés jól definiált, és a csatornaszám $32 + 8 = 40$. A dekódoló $k=3$, $p=1$ konvolúciója ismét megőrzi a 24×24-et, a fej 1×1-es konvolúciója pedig nem változtat a térbeli alakon. A végső logit-maszk így ugyanazon a rácson értelmezett, mint a célmaszk.

39.2.4 A loss nevezőjének levezetése

Minden (n, p) pixelre, ahol $p = 1, \dots, HW$ a $H \times W$-es rács sorfolytonos lineáris indexe (a (i, j) rácspont sorszáma), a logit $z_{n,p} \in \mathbb{R}$, a címke $y_{n,p} \in \{0;\ 1\}$. A pixel feltételes modellje Bernoulli:

$$p(y_{n,p} \mid z_{n,p}) = \sigma(z_{n,p})^{y_{n,p}} \big(1 - \sigma(z_{n,p})\big)^{1 - y_{n,p}},$$

ahol $\sigma(z) = 1/(1 + e^{-z})$. A negatív log-likelihood az azonos alakú, nem redukált BCE:

$$L_{n,p} = -\Big[y_{n,p} \log \sigma(z_{n,p}) + (1 - y_{n,p}) \log\big(1 - \sigma(z_{n,p})\big) \Big].$$

A kiválasztott empirikus loss rögzített térbeli nevezővel átlagol:

$$\mathcal{L} = \frac{1}{P} \sum_{n=1}^{N} \sum_{p=1}^{HW} L_{n,p}, \qquad P = N \cdot 1 \cdot H \cdot W = NHW.$$

A P nevező akkor is pozitív egész, ha egy képen egyáltalán nincs előtérpixel. Ez a kulcs az üres cél kezeléséhez: ha a normalizálást a foreground-pixelek számához kötnénk, üres maszknál nullával osztanánk. Itt ehelyett az üres cél is értelmes, véges tanulási jelet ad: minden pixelre $y = 0$, így a loss a háttérlogitek lenyomására tanít.

A gradiens levezetése egy pixelre a logisztikus függvény deriváltját használja. Mivel $\sigma'(z) = \sigma(z)(1 - \sigma(z))$,

$$\frac{\partial L_{n,p}}{\partial z_{n,p}} = -y_{n,p}(1 - \sigma(z_{n,p})) + (1 - y_{n,p})\sigma(z_{n,p}) = \sigma(z_{n,p}) - y_{n,p}.$$

Az átlagredukció után tehát

$$\frac{\partial \mathcal{L}}{\partial z_{n,p}} = \frac{1}{NHW}(\sigma(z_{n,p}) - y_{n,p}).$$

A `torch.nn.functional.binary_cross_entropy_with_logits(..., reduction="none")` (illetve az `nn.BCEWithLogitsLoss(reduction="none")` csomagolás) ezt a nem redukált L-t adja vissza $(N, 1, H, W)$ alakban, majd a `.mean()` implicit módon $P = NHW$-vel oszt. Numerikusan a szigmoid és a logaritmus külön hívása instabil; a könyvtári függvény a logitot közvetlenül fogadja, ezért a saját kódban külön `sigmoid`-hívást nem végzünk.

39.2.5 Maszk-interpoláció: mit szabad és mit nem

A logit- vagy jellemzőtérkép bilineáris felnagyítása a tanuló modell része, ezért folytonos értékei a gradiensszámításban jól definiáltak. A bináris célmaszk bilineáris interpolálása viszont nem konzisztens címkeképzés: az 1-es előtér és a 0-ás háttér közötti élen olyan értékek keletkeznek, amelyek sem nem megfigyelt maszk, sem nem valószínűségi súly. Ha a cél felbontását mégis változtatni kell, a maszkra nearest-neighbor mintavételt használunk, például a páros indexű pixelek kiválasztását. A főútban erre nincs szükség: a modell kimenete a natív 24×24-es célrácsra illeszkedik.

39.2.6 Tipikus tévedés ellenpéldával

A kimenet és a target azonos pixelszáma nem elég, ha a térbeli illesztés hibás. Legyen 2×2-es logit-maszk

$$Z = \begin{bmatrix} 5 & -5 \\ -5 & -5 \end{bmatrix},$$

és a helyes cél

$$Y = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Mindkét célváltozatban egyetlen előtérpixel van, tehát a pixelszám azonos. A helyes illesztésnél mind a négy pixel helyes (az előtérpixel logitja pozitív, a háttérpixeleké negatív), az összegzett binary cross-entropy:

$$4\log(1+e^{-5}) \approx 4 \cdot 0{,}0067153 = 0{,}0268614.$$

Ha a cél elcsúszik,

$$Y' = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix},$$

akkor két pixel téves (a $(0;\ 0)$ pozitív logitja háttércímkével, a $(1;\ 0)$ negatív logitja előtércímkével találkozik), kettő pedig helyes, és az összegzett veszteség

$$2\log(1+e^{5}) + 2\log(1+e^{-5}) \approx 2 \cdot 5{,}0067153 + 2 \cdot 0{,}0067153 = 10{,}0268614.$$

A fenti két összegzett veszteség (`reduction='mean'` mellett) 4-gyel osztandó: $\approx 0{,}006715$, illetve $\approx 2{,}506715$; float64-ben a pontos összegek $0{,}0268613939\ldots$ és $10{,}0268613939\ldots$ (Python 3.12.3, PyTorch 2.15.0+cpu, CPU).

Az eltérés abból fakad, hogy a nagy pozitív logit a háttércímkével, a nagy negatív logit pedig az előtércímkével találkozik (két téves pixel). Az `AssertionError`-t nem vált ki egy shape-ellenőrzés, mert mindkét eset `(1,1,2,2)`; csak a veszteség vagy egy pixelpontossági/konfúziós kontroll mutatja ki a hibát; a Dice-t a 39.3. Geometriai kontroll és metrika vezeti be.

39.2.7 Minimális PyTorch-megvalósítás és sanity check

A Projekt III összes szkriptje a `lab/project3` munkakönyvtárban van (ott készült a 39.1. Feladat- és annotációs döntés `annotation_spec.py`-ja is); az adatok a GlyphLab `lab/data/` könyvtárából töltődnek. Az első szkript a modellt és a loss-szerződést ellenőrzi egy rögzített kis batch-en.

```
# lab/project3/segmentation_head.py
from __future__ import annotations
import csv
from pathlib import Path

import numpy as np
import torch
from torch import nn
import torch.nn.functional as F

DATA = Path(__file__).resolve().parents[1] / "data"
torch.set_num_threads(1)   # egyszálas CPU: az összegzési sorrend az adott gépen
# nem függ a magszámtól; gépek és verziók között a bitazonosság nem garantált


class SmallSegUNet(nn.Module):
    """GlyphLab 24x24-es maszkfej; a kimenet (N,1,24,24) logit."""

    def __init__(self, in_ch: int = 1, out_ch: int = 1, base: int = 8) -> None:
        super().__init__()
        self.enc1 = nn.Sequential(
            nn.Conv2d(in_ch, base, 3, padding=1), nn.ReLU()
        )
        self.pool = nn.MaxPool2d(2)
```

```
        self.enc2 = nn.Sequential(
            nn.Conv2d(base, 2 * base, 3, padding=1), nn.ReLU()
        )
        self.bottleneck = nn.Sequential(
            nn.Conv2d(2 * base, 4 * base, 3, padding=1), nn.ReLU()
        )
        self.up = nn.Upsample(size=(24, 24), mode="bilinear", align_corners=False)
        self.dec = nn.Sequential(
            nn.Conv2d(4 * base + base, 2 * base, 3, padding=1), nn.ReLU()
        )
        self.head = nn.Conv2d(2 * base, out_ch, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        e1 = self.enc1(x)          # (N,8,24,24)
        p1 = self.pool(e1)         # (N,8,12,12)
        e2 = self.enc2(p1)         # (N,16,12,12)
        p2 = self.pool(e2)         # (N,16,6,6)
        b = self.bottleneck(p2)    # (N,32,6,6)
        u = self.up(b)             # (N,32,24,24)
        c = torch.cat([u, e1], dim=1)  # (N,40,24,24)
        d = self.dec(c)            # (N,16,24,24)
        return self.head(d)        # (N,1,24,24)


def bce_fixed_denominator(logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
    """Pixel-BCE, a nevező mindig N*1*H*W; üres target esetén is stabil."""
    if logits.shape != targets.shape:
        raise ValueError(
            f"logit/cél alak eltérés: {tuple(logits.shape)} vs {tuple(targets.shape)}"
        )
    per_pixel = F.binary_cross_entropy_with_logits(
        logits, targets, reduction="none"
    )
    return per_pixel.mean()


def load_train_batch(n: int = 8) -> tuple[torch.Tensor, torch.Tensor]:
    images = np.load(DATA / "images.npy")           # (360,1,24,24) float32
    masks = np.load(DATA / "masks.npy").astype(np.float32)   # (360,1,24,24) {0,1}
    assert images.shape == (360, 1, 24, 24), images.shape
    assert masks.shape == (360, 1, 24, 24), masks.shape
    assert np.all(np.isin(masks, [0.0, 1.0])), "nem bináris maszk"
    with open(DATA / "metadata.csv", newline="") as f:
        rows = list(csv.DictReader(f))
    assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(rows[0])
    assert len(rows) == len(images) == len(masks) == 360, (len(rows), images.shape)
    # Az index a fájlsor indexe: ezért használható a metadata indexe közvetlenül
    # az images.npy és a masks.npy sorainak kiválasztására.
    assert [int(r["index"]) for r in rows] == list(range(len(rows)))
    idx = [int(r["index"]) for r in rows if r["split"] == "train"][:n]
    x = torch.from_numpy(images[idx])
    y = torch.from_numpy(masks[idx])
    assert x.shape == (n, 1, 24, 24), x.shape
    assert y.shape == (n, 1, 24, 24), y.shape
    return x, y


def main() -> None:
    torch.manual_seed(20261020)
    device = torch.device("cpu")
    model = SmallSegUNet().to(device)
    x, y = load_train_batch(8)
```

```
    x, y = x.to(device), y.to(device)

    # 1. Alakellenőrzés.
    with torch.no_grad():
        z0 = model(x)
    assert z0.shape == (8, 1, 24, 24), z0.shape

    # 2. BCE numerikus kontroll egy pixelre: z=0.5, y=1 esetén
    #    log(1+exp(-0.5)) ~ 0.474077.
    one = torch.tensor([[[[0.5]]]], dtype=torch.float32, device=device)
    target_one = torch.ones_like(one)
    loss_px = float(F.binary_cross_entropy_with_logits(one, target_one))
    assert abs(loss_px - 0.474077) < 1e-5, loss_px

    # 3. Üres cél: véges, gradiense nem NaN. A z0 no_grad alatt készült,
    #    ezért a backwardhoz új, gráffal rendelkező forward kell.
    empty = torch.zeros_like(x)
    loss_empty = bce_fixed_denominator(model(x), empty)
    assert torch.isfinite(loss_empty), loss_empty
    loss_empty.backward()
    assert all(torch.isfinite(p.grad).all() for p in model.parameters() if p.grad is not None)

    # 4. Kis túlillesztési sanity check: a loss csökken a rögzített batch-en.
    model.train()
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    losses: list[float] = []
    for _ in range(60):
        opt.zero_grad()
        loss = bce_fixed_denominator(model(x), y)
        loss.backward()
        opt.step()
        losses.append(loss.item())
    assert torch.isfinite(torch.tensor(losses)).all()
    assert losses[-1] < losses[0], f"{losses[0]:.4f} -> {losses[-1]:.4f}"
    print(f"loss {losses[0]:.4f} -> {losses[-1]:.4f} (CPU, 8 train kép)")


if __name__ == "__main__":
    main()
```

Futtatási parancs:

```
cd lab/project3
python segmentation_head.py
```

Az ellenőrzés értelmezése: a 2. numerikus egyezés a per-pixel BCE képletét és számértékét ellenőrzi, nem a térbeli illesztést. A 3. lépés az üres cél véges lossát és gradiensét; a 4. lépés azt, hogy a rögzített batch-en a gradiensút működik és a loss csökken. A csökkenő loss önmagában nem zárja ki a térbeli eltolást: ha a cél el van tolva, a modell ahhoz is illeszkedhet, és a loss akkor is csökkenhet; a 39.2.6 eltolt-cél ellenpéldájában ugyanannyi előtérpixel mellett a rossz illesztés lossa 10,0269, a helyesé 0,0269. A loss kezdeti értéke a maszk előfordulási arányától függ, ezért önmagában nem viszonyítási alap. A sanity-check lossa 8 train képre, a későbbi val BCE pedig a teljes val splitre vonatkozik, ezért a két szám nem hasonlítható össze. A valós kiértékelés a 39.3. Geometriai kontroll és metrika metrikájával, a legjobb checkpointot pedig a 38.2. Modell, training és best checkpoint szabálya szerint a validációs loss alapján végezzük; a corrupt tesztet csak a protokoll végén értékeljük.

A rögzített környezetben (CPU, Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, 1 szál) a szkript minden `assert`-en átment, és `loss 0.5897 -> 0.1252 (CPU, 8 train kép)` sort írt.

39.2.8 A főág tanítása és a best checkpoint

A sanity check után a főág a teljes train spliten tanít, és a 38.2. Modell, training és best checkpoint szabálya szerint a legkisebb validációs BCE-jű epoch állapotát menti. A teszt itt nem nyílik meg; a tesztkiértékelés és az export a 39.4. Alkalmazási riport és átadás alfejezetben történik, ebből a checkpointból.

```
# lab/project3/train_seg.py - n154 főág: SmallSegUNet tanítása, best checkpoint val BCE alapján.
import csv, json
from pathlib import Path

import numpy as np
import torch

from segmentation_head import DATA, SmallSegUNet, bce_fixed_denominator

HERE = Path(__file__).resolve().parent
SEED, EPOCHS, BATCH, LR = 20261020, 30, 16, 1e-3


def split_ids(split):
    with open(DATA / "metadata.csv", newline="") as f:
        rows = list(csv.DictReader(f))
    assert list(rows[0]) == ["index", "scene_id", "view_id", "label", "split",
                             "xmin", "ymin", "xmax", "ymax"], list(rows[0])
    assert len(rows) == 360, len(rows)
    assert [int(r["index"]) for r in rows] == list(range(len(rows)))
    return np.array([int(r["index"]) for r in rows if r["split"] == split])


def main():
    torch.manual_seed(SEED)
    images = torch.from_numpy(np.load(DATA / "images.npy"))                    # (360,1,24,24)
    masks = torch.from_numpy(np.load(DATA / "masks.npy").astype(np.float32))   # (360,1,24,24)
    assert images.shape == (360, 1, 24, 24), images.shape
    assert masks.shape == (360, 1, 24, 24), masks.shape
    assert torch.all((masks == 0) | (masks == 1)), "nem bináris maszk"
    assert len(images) == len(masks) == 360, (images.shape, masks.shape)
    tr, va = split_ids("train"), split_ids("val")       # a teszt itt nem nyílik meg
    assert len(tr) == 240 and len(va) == 60, (len(tr), len(va))
    model = SmallSegUNet()
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    g = torch.Generator().manual_seed(SEED)
    (HERE / "checkpoints").mkdir(exist_ok=True)
    best, log = {"val_loss": float("inf")}, []
    for epoch in range(1, EPOCHS + 1):
        model.train()
        perm = tr[torch.randperm(len(tr), generator=g).numpy()]
        for i in range(0, len(perm), BATCH):
            b = perm[i:i + BATCH]
            opt.zero_grad(set_to_none=True)
            loss = bce_fixed_denominator(model(images[b]), masks[b])
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            val_loss = float(bce_fixed_denominator(model(images[va]), masks[va]))
        log.append({"epoch": epoch, "train_last_batch_loss": loss.item(), "val_loss": val_loss})
        if val_loss < best["val_loss"]:
            best = {"epoch": epoch, "val_loss": val_loss}
            torch.save({"model_state_dict": model.state_dict(), "arch": "SmallSegUNet",
                        "epoch": epoch, "val_loss": val_loss, "seed": SEED,
```

```
                        "params": sum(p.numel() for p in model.parameters()),
                        "threshold": 0.5}, HERE / "checkpoints" / "seg_best.pt")
    (HERE / "checkpoints" / "seg_train_log.json").write_text(json.dumps(log, indent=1))
    print(f"best epoch {best['epoch']}, val BCE {best['val_loss']:.4f} ->
     ↪  checkpoints/seg_best.pt")


if __name__ == "__main__":
    main()
```

```
cd lab/project3
python train_seg.py
```

A kimenet a `checkpoints/seg_best.pt` (kulcsai: `model_state_dict`, `arch`, `epoch`, `val_loss`, `seed`, `params`, `threshold`) és a `checkpoints/seg_train_log.json` epochnapló. A küszöb (0,5 a szigmoid kimenetén) a checkpoint része, így a későbbi export nem választhat más küszöböt a teszt ismeretében. **Futási jegyzőkönyv:** a rögzített környezetben (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020) két egymás utáni futás fali ideje 5,49 s és 5,52 s volt; mindkettő `best epoch 30, val BCE 0.0029` sort írt. A `seg_best.pt` SHA-256 értéke mindkét futásban `b8e8958b…`, a `seg_train_log.json` értéke `beb13310…`, azaz a checkpoint és a napló byte-azonos. A legjobb epoch a 30., azaz a 30 epochás keret utolsó epochja; a val BCE a 25. és a 30. epoch között 0,005031-ról 0,002874-re csökkent, ezért a tanítás ezen a kereten belül nem konvergált, és hosszabb keret indokolt. A sanity-check 8 képes train lossa és a 60 képes val BCE más adathalmazon és más epochszámmal mérődött, ezért a két szám nem összevethető.

39.2.9 Feladatok

F1 (fogalmi, nehézség 1). Indokolja meg, miért egycsatornás a választott spatial head kimenete, és adjon ellenpéldát arra, hogy egy `nn.Flatten()` + `nn.Linear(576, 576)` miért nem helyettesíti a fejet akkor sem, ha a pixelszám egyezik.

F2 (levezetési, nehézség 2). Legyen $N = 2$, $H = W = 3$, $C = 1$. Az első kép célja teljesen üres, a második kép $(0;\ 0)$ pixelén $y = 1$. Írja fel a loss nevezőjét, a teljes $\mathcal{L}$ összegzési határait, és a gradiens értékét a két kijelölt pixelre. Adja meg az autograddal konzisztens skálatényezőt is.

F3 (implementációs, nehézség 2). Készítsen `nearest_target_half(y)` függvényt, amely egy `(N,1,H,W)` alakú uint8 maszkból a páros indexű pixelek kiválasztásával `(N,1,H//2,W//2)` alakot állít elő. Páratlan H vagy W esetén emeljen `ValueError`-t. Tesztelje az

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

bemenetre az elvárt

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

kimenetet. A megadott mátrixot a függvény bemeneti szerződése szerint `(1,1,4,4)` alakra hozva kell átadni.

F4 (hibakeresési, nehézség 1). Egy hallgató a natív 48×48-as target maszkot `F.interpolate(target_float, size=(24,24), mode="bilinear")` módon kicsinyíti, majd

ezt adja a BCE-nek. A training hiba nélkül lefut, a validációs Dice mégis rossz. Nevezze meg a hiba következményét a célértékekre, adjon kimutatási módszert és javítást.

F5 (kísérleti/transzfer, nehézség 3). Tegyük fel, hogy egy új adatkészlet 128×128-as maszkokkal érkezik, de a modell dekódolója változatlanul 24×24 logitot ad. Tervezzen kis összehasonlító kísérletet két ágra: (A) a logitot bilineárisan 128×128-ra nagyítjuk, és ott számítjuk a BCE-t; (B) a célmaszkot nearest módon 24×24-re csökkentjük. Adja meg a megfigyelési egységet, a splitet, a baseline-t, a metrikát, a két loss nevezőjét, valamint a toy kísérlet érvényességi korlátját.

39.2.10 Megoldások és indoklás

F1. A fej kimenete egy logitcsatorna, mert a bináris szegmentálás minden pixelre egyetlen előtér-valószínűségi logitot igényel: $Z \in \mathbb{R}^{N\times 1\times H\times W}$. A lineáris `576→576` réteg a sorfolytonos (row-major) lapítási konvenciót égetné be a súlyokba: a `nn.Flatten` C-sorrendben lapít, és a `Linear` súlymátrixának oszlopai ehhez a sorrendhez kötődnek, ezért ha a későbbi feldolgozás más sorrendben lapítja ugyanazt a maszkot, azonos pixelszám mellett más pixel–súly párosítás jön létre. Konkrétan a $Z = \begin{bmatrix} -5 & 5 \\ -2 & -5 \end{bmatrix}$ logit és az $Y = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ cél esetén a sorfolytonos párosítás összege 0,1471, a cél oszlopfolytonos lapítása mellett viszont 7,1471 (a logitokat változatlanul sorfolytonosan lapítva). A réteg emellett megszüntetné a pixelpozíciók közötti paramétermegosztást: $576 \cdot 576 + 576 = 332\,352$ paramétere van, szemben a $16 \to 1$-es 1×1-es konvolúció $16 + 1 = 17$ paraméterével. A 1×1-es konvolúció pozíciófüggetlen affin fej, és a térbeli indexelést a bemeneti rácstól örökli.

F2. A logit `(2,1,3,3)`, a cél ugyanilyen; a logit jelölése $z_{n,0,i,j}$, ahol n a kép, 0 az egyetlen csatorna, (i, j) a térbeli index. A pixelek száma képenként $HW = 9$, így a nevező $P = NHW = 18$. A loss

$$\mathcal{L} = \frac{1}{18} \sum_{n=0}^{1} \sum_{i=0}^{2} \sum_{j=0}^{2} L_{n,i,j}.$$

Az első kép üres, de a nevező változatlan. Az első kép $(i, j) = (0;\ 0)$ pixelére $y = 0$, ezért a gradiens $\partial\mathcal{L}/\partial z_{0,0,0,0} = \sigma(z_{0,0,0,0})/18$. A második kép $(i, j) = (0;\ 0)$ pixelére $y = 1$, ezért a gradiens $\partial\mathcal{L}/\partial z_{1,0,0,0} = (\sigma(z_{1,0,0,0}) - 1)/18$. Ha a kijelölt logitok értéke 0, akkor $\sigma(0) = 0{,}5$, így a két gradiens $0{,}5/18 \approx 0{,}0278$, illetve $-0{,}5/18 \approx -0{,}0278$. Az autograd ugyanezt a skálát adja, mert a `mean()` redukció a teljes `(N,1,H,W)` tenzorelemszámot használja.

F3. Egy lehetséges megoldás:

```
def nearest_target_half(y: np.ndarray) -> np.ndarray:
    if y.ndim != 4 or y.shape[1] != 1:
        raise ValueError("várt alak: (N,1,H,W)")
    _, _, h, w = y.shape
    if h % 2 or w % 2:
        raise ValueError(f"páratlan méret: ({h},{w})")
    return y[:, :, 0:h:2, 0:w:2]
```

A feladatban szereplő 4×4-es mátrixot a függvény szerződése szerint előbb `(1,1,4,4)` alakra kell hozni (a formázatlan 2D tömböt a függvény szándékosan `ValueError`-val utasítja el). A kiválasztott indexek $(0;\ 0)$, $(0;\ 2)$, $(2;\ 0)$, $(2;\ 2)$, azaz $1, 0, 0, 1$; az eredmény $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Futtatható ellenőrzés:

```
y = np.array([[1,0,0,0],[0,0,0,0],[0,0,1,1],[0,0,1,1]], dtype=np.uint8).reshape(1,1,4,4)
out = nearest_target_half(y)
np.testing.assert_array_equal(out, np.array([[[[1,0],[0,1]]]], dtype=np.uint8))
```

A páratlan méretre a `nearest_target_half(np.zeros((1,1,5,4), dtype=np.uint8))` `ValueError`-t ad.

F4. A bilineáris kicsinyítés a bináris célon 0 és 1 közötti értékeket hoz létre az előtér–háttér átmeneteknél; például egy 48×48-as maszk $[17;\ 31)$ sor- és oszlopintervallumán lévő téglalapja 24×24-re kicsinyítve 0,25 és 0,5 értékeket is ad, tehát a BCE soft labelként értelmezi ezeket. A hiba nem vált ki `RuntimeError`-t: a `BCEWithLogitsLoss` nem ellenőrzi a cél tartományát, ezért a $[0;\ 1]$-en kívüli értékeket is hiba nélkül kiszámolja; a tanulási cél viszont már nem a megfigyelt bináris maszk. Kimutatás: egy éles maszkon az interpolált tenzor `unique()` értékeinek megjelenítése; a bináris kontroll csak `{0,1}` elemeket enged meg. Javítás: a targetet ne interpoláljuk, hanem a modell kimenetét igazítsuk a cél natív felbontásához, vagy ha elkerülhetetlen, nearest módon mintavételezzük a címkét. Visszaellenőrzés a validációs Dice újraszámításával és a target `min`/`max` értékével.

F5. Kiinduló hipotézis: a target eredeti 128×128-as rácsán mért veszteség pontosabban bünteti a határhibákat, de az (A) ág nagyobb számítási igényű. Megfigyelési egység a `scene_id`, a splitet scene-szinten kell tartani. Fix modell, seed és training; baseline a teljes háttér predikciója. A metrika a legjobb validációs checkpoint után a 128×128-as rácson számított thresholdolt Dice/IoU. Az (A) veszteség nevezője $N \cdot 1 \cdot 128 \cdot 128$, a (B) nevezője $N \cdot 1 \cdot 24 \cdot 24$. A (B) ág 24×24-es logitját a thresholdolás után nearest módszerrel 128×128-ra nagyítjuk, hogy a két ág ugyanazon a 128×128-as kiértékelési rácson hasonlítható össze. A két ágat azonos seeddel tanítjuk; a tesztet scene-enként párosítjuk (ugyanaz a `scene_id` ugyanabban a splitben, azonos seed mellett), a különbséget scene-enként számítjuk, majd a scene-ek közötti átlagot és bootstrap konfidenciaintervallumot adjuk meg. A toy kísérlet nem bizonyít általános interpolációs tételt, és egyetlen seed nem ad általános érvényű garanciát. A tanítás a 39.2 `SmallSegUNet`-tel, Adam-mal ($\eta = 10^{-3}$), batch 16 mellett, 30 epochon át, öt seeddel (20261020–20261024) történik; a döntési szabály: a páros jelenetszintű Dice-különbség 95%-os bootstrap-intervalluma nem tartalmaz nullát, és a pontbecslés legalább 0,05 Dice; az erőforrás CPU, öt seed és két ág.

39.2.11 Önellenőrzés és korábbi fogalmak áttekintése

A 27.2. U-Net, encoder–decoder és skip levezetéséhez hasonlóan minden egyes rétegre felírtuk a térbeli alakot; a concat előtt ellenőriztük, hogy a két ág H és W mérete azonos. A 38.2. Modell, training és best checkpoint tanítási szerződéséből megtartottuk, hogy a validáció választ checkpointot, a tesztet csak a rögzített protokoll végén értékeljük. A 35.2. Magyarázat sanity check tanulsága itt annyi, hogy a kimeneti maszk a modell számítási útjának eredménye, nem egy külső magyarázat; a térbeli illesztést éppen ezért a loss és a metrika szintjén kell ellenőrizni, nem pusztán a shape-ek összehasonlításával.

39.2.12 Rövid összefoglaló

A szegmentáló fej 1×1-es konvolúcióból áll, és pixelre lebontott logit-maszkot ad; a GlyphLab főútban a bemenettel azonos 24×24-es célrácsra illeszkedik. A skip concat és a bilineáris upsampling alakhelyességét rétegenként vezettük le. A loss nem redukált BCEWithLogits, majd a teljes $N \cdot 1 \cdot H \cdot W$ pixelszámmal átlagolunk; így az üres cél sem okoz nullával osztást. A célmaszkot nem interpoláljuk bilineárisan, mert a bináris címkéből hamis soft label lesz. A sanity check a shape-t, a BCE kis numerikus értékét és a loss csökkenését ellenőrzi kis CPU-futásban.

39.2.13 Következő lecke és felhasznált források

A 39.3. Geometriai kontroll és metrika egységben a betanított fej kimenetére geometriai kontrollt és szegmentálási metrikát vezetünk be: a thresholdolt maszk Dice/IoU kiértékelését, a per-image, per-scene és micro aggregálást, valamint a páros corrupt tesztre vonatkozó összevetést.

- [P14] Ronneberger, Fischer és Brox (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* arXiv:1505.04597. https://arxiv.org/abs/1505.04597.
- [D18] PyTorch contributors (2026). *TorchVision Object Detection Finetuning Tutorial.* PyTorch 2.14 dokumentáció. https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html. Hozzáférés: 2026-10-30.

39.3 Geometriai kontroll és metrika

39.3.1 Cél, előismeret és induló ellenőrzés

A Projekt III. a 39.2. Encoder–decoder vagy lokalizációs fej alfejezetben egy bináris, egyobjektumos szegmentáló fejet definiált: a háló minden, a függelék A. A GlyphLab adatkészlet szakaszában leírt 24×24-es GlyphLab-képhez egy logit-maszkot ad, amelyből küszöböléssel $\hat{G} \in \{0;\ 1\}^{H\times W}$ bináris előrejelzés lesz. Ebben az alfejezetben azt a kérdést járjuk körül, hogyan ellenőrizzük ezt a kimenetet geometriailag és statisztikailag úgy, hogy az állítás ne csak a nagy háttérterület miatt tűnjön jónak.

A szükséges előismeret: a maszk mint $H \times W$ bináris mátrix, a half-open doboz 26.1. Dobozkoordináta, IoU és lokalizációs loss alfejezetből, valamint az IoU/Dice alapfogalmak 27.3. Dice, IoU és aggregálás alfejezetből. A GlyphLab anotációja minden képhez tartalmazza az előtér maszkját; a `masks.npy` alakja (360; 1; 24; 24), `uint8`, értéke 0 vagy 1.

Induló ellenőrző kérdés: ha egy 24×24-es képen a GT (ground truth, azaz referencia-) maszk 24 előtérpixelt tartalmaz (4,17%), és a modell üres maszkot ad, akkor a pixel accuracy $\frac{576-24}{576} = 0{,}9583$. Jó eredmény-e ez? A kérdésre a válasz nem, és a különbség az alfejezet központi témája.

39.3.2 Pixelmetrikák és hibakategóriák

Rögzítsünk egyetlen képre egy GT maszkot $G \in \{0;\ 1\}^{H\times W}$ és egy bináris predikciót (a kódban `pred`) $\hat{G} \in \{0;\ 1\}^{H\times W}$. A pixelpárok négy hibakategóriába sorolhatók:

$$\mathrm{TP} = \sum_{h,w} G_{h,w}\hat{G}_{h,w}, \qquad \mathrm{FP} = \sum_{h,w} (1-G_{h,w})\hat{G}_{h,w},$$
$$\mathrm{FN} = \sum_{h,w} G_{h,w}(1-\hat{G}_{h,w}), \quad \mathrm{TN} = \sum_{h,w} (1-G_{h,w})(1-\hat{G}_{h,w}).$$

Mind a négy mennyiség nemnegatív egész, összegük $N = H \cdot W$. A számítás előfeltétele, hogy G és $\hat{G}$ már **binarizált** legyen: a logit-maszkból a predikció $\hat{G} = \mathbf{1}[z > 0]$, azaz $\sigma(z) > 0{,}5$; a küszöbölést a hívó végzi, a `conf_counts` nem binarizál logitot. A maszkok `uint8` típusból `bool`-lá alakíthatók, így a fenti összegek a NumPy `logical_and`, `logical_not` műveleteivel számíthatók.

A bináris szegmentálás két szokásos metrikája:

$$J = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \qquad D = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}.$$

A 27.3. Dice, IoU és aggregálás alfejezetben bizonyítottuk, hogy $D = \frac{2J}{1+J}$ minden olyan esetben, amikor $\mathrm{TP} + \mathrm{FP} + \mathrm{FN} > 0$; ebbe a $\mathrm{TP} = 0$, de $\mathrm{FP} + \mathrm{FN} > 0$ eset is beletartozik, mert ekkor $J = D = 0$. Rögzítettük az üres–üres konvenciót: ha $\mathrm{TP} + \mathrm{FP} + \mathrm{FN} = 0$, azaz a két maszk egyaránt üres, akkor $J = D = 1$ képpáronként. A $\mathrm{TP} + \mathrm{FP} + \mathrm{FN} = 0$ nevező nemnegatív tagok mellett csak $\mathrm{TP} = \mathrm{FP} = \mathrm{FN} = 0$ esetén lehetséges (a TN ilyenkor akár pozitív), tehát a nulla nevezőnek egyetlen esete van, és ott a választott konvenció szerint $J = D = 1$. A döntési tábla: nevező > 0: a szokásos hányados; nevező $= 0$: $J = D = 1$. A projektben a GT maszk az adatgenerátor előírása miatt nem üres, de a predikció lehet üres, ezért az üres eset kezelése kötelező.

A pixel accuracy

$$A = \frac{\mathrm{TP} + \mathrm{TN}}{N}$$

egyszerű, de háttérdomináns képen torzít: az induló példa 0,9583 accuracy mellett $J = 0$ és $D = 0$, mert $\mathrm{TP} = 0$. A szegmentáló modell kiértékelésénél ezért az IoU/Dice az elsődleges metrika, az accuracy-t csak a hibakategóriák mellett közöljük.

39.3.3 Geometriai doboz és centroid

A doboz az előtér pixeleinek legkisebb befoglaló téglalapja. Az indexelés nullától kezdődik, a doboz **félig nyílt** xyxy intervallum:

$$x_{\min} = \min\{w : \exists h,\ M_{h,w} = 1\}, \quad x_{\max} = 1 + \max\{w : \exists h,\ M_{h,w} = 1\},$$

és ugyanígy $y_{\min}$, $y_{\max}$. Az intervallum $[x_{\min}, x_{\max})$ vízszintesen és $[y_{\min}, y_{\max})$ függőlegesen. A doboz területe

$$A_{\mathrm{box}} = (x_{\max} - x_{\min})(y_{\max} - y_{\min}).$$

A félig nyílt konvenció nem esztétikai kérdés: két doboz metszetének szélessége $\max(0,\ \min(x_{\max}, x'_{\max}) - \max(x_{\min}, x'_{\min}))$; a nyers különbség diszjunkt intervallumokra negatív, ezért a $\max(0, \cdot)$ kell, és a szélesség pontosan akkor pozitív, ha a két intervallum valódi átfedésben van. A félig nyílt végponthoz ($x_{\max}$ már kizárt) tilos újabb $+1$-et adni; a $+1$ csak akkor helyes, amikor a maszk legnagyobb *előtér-pixelindexéből* képezünk félig nyílt $x_{\max}$ végpontot ($x_{\max} = \mathrm{maxindex} + 1$). A GlyphLab `metadata.csv`-ben szereplő $x_{\min}, y_{\min}, x_{\max}, y_{\max}$ pontosan ezt a konvenciót követi.

A centroid, azaz a maszk tömegközéppontja:

$$c(M) = \frac{1}{|M|} \sum_{(h,w) \in M} (h, w),$$

ahol $|M| = \sum_{h,w} M_{h,w}$ az előtérpixelek száma. A koordináták valós számok, a tengelysorrend (y, x), mert a tömb első indexe a sor. A centroid a doboz négy sarkánál finomabb pozícióinformációt ad: a doboz a szélső pontokra, a centroid a tömeg eloszlására érzékeny.

39.3.4 Levezetés: a centroid-hiba eltolási ekvivarianciája

A projektben a centroid-hibát is használjuk, ezért nem elegendő azt mondani, hogy „a centroid eltolódik az eltolással”. Állításként és bizonyításként rögzítjük.

1. lemma (centroid eltolási ekvivarianciája). Legyen $M \subset \mathbb{Z}^2$, $0 < |M| < \infty$, és $c(M) = \frac{1}{|M|}\sum_{(h,w)\in M}(h,w)$. Ekkor minden $t = (t_h, t_w) \in \mathbb{Z}^2$ esetén

$$c(M + t) = c(M) + t,$$

ahol $M + t = \{(h + t_h, w + t_w) : (h, w) \in M\}$.

Bizonyítás. Az eltolás bijekció M és $M+t$ között, ezért $|M+t| = |M|$. A definíciót alkalmazva:

$$c(M+t) = \frac{1}{|M+t|}\sum_{(h',w')\in M+t}(h',w') = \frac{1}{|M|}\sum_{(h,w)\in M}(h+t_h, w+t_w).$$

Az összeget komponensenként szétbontva:

$$\frac{1}{|M|}\left[\sum_{(h,w)\in M}(h,w) + \sum_{(h,w)\in M}(t_h,t_w)\right] = c(M) + \frac{1}{|M|}\cdot|M|\,t = c(M)+t.$$

A második egyenlőségnél használtuk, hogy az összeg második tagja konstans tagú: minden elemhez ugyanazt a t vektort adjuk, ezért az összeg $|M|t$. ■

Szélsőeset. Üres maszkra $c(M)$ nem definiált; a kódban `None` értékkel jelöljük. Ha az eltolás a képkeretből kilóg, akkor a levágott, megfigyelt halmaz nem $M + t$, hanem annak a képkerettel vett metszete, így a lemma nem alkalmazható. A centroid-hiba tehát eltolás-invariáns a kereten belül, de a határnál a csonkolás új geometriai hibát hoz létre.

1. következmény (a centroid-hiba eltolás-invarianciája). Legyen $G, \hat{G}$ két nemüres maszk, és $t \in \mathbb{Z}^2$ olyan, hogy $G + t$ és $\hat{G} + t$ is a képkeretben marad. Ekkor

$$\|c(\hat{G}+t) - c(G+t)\|_2 = \|c(\hat{G}) - c(G)\|_2.$$

Bizonyítás. Az 1. lemmát kétszer alkalmazva $c(\hat{G}+t) - c(G+t) = c(\hat{G}) - c(G)$, mert a $+t$ tag kiejti egymást. A norma megőrzi az egyenlőséget. ■

Ez a következmény a centroid-hiba **eltolás-invarianciája**: a hiba a két maszk együttes eltolására érzéketlen, így tisztán a relatív pozícióhibát méri; maga a centroid nem invariáns, hanem **ekvivariáns**, $c(M + t) = c(M) + t$. A bizonyítás a koordináták egy-egy értelmű eltolásán (bijekción) alapul: az eltolás a maszk előtérpixeleit egy-egy értelműen képezi le, így a pixelek száma nem változik, és minden koordinátájuk ugyanazzal a vektorral tolódik. A centroid a maszkra nézve általában **nem** lineáris: $c(M) = \sum_r r\,M_r / \sum_r M_r$, a nevező is a bemenettől függ; a maszk pozitív skalárral szorzása nem változtatja meg a centroidot, azaz a centroid 0-ad fokú homogén, skálainvariáns függvénye a maszknak. Linearitás legfeljebb rögzített számú pont koordinátáinak átlagolására mondható ki, és az eltolási ekvivariancia nem ebből, hanem a fenti bijekcióból következik. A doboz ugyanígy ekvivariáns az eltolásra, de mivel minimumra és maximumra épül, egyetlen kiugró pixel a doboz valamelyik határát azonnal elmozdíthatja, míg ugyanaz a pixel a centroidban csak $1/|M|$ súllyal szerepel; ezért a doboz érzékenyebb a pixelek helyzetének kis változására.

39.3.5 Eredeti kézi kontrollpélda

Vegyünk egy 6×8-as rácsot. A GT maszk egy 3×2-es téglalap:

```
0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0
0 0 1 1 0 0 0 0
0 0 1 1 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

A predikció az alsó sort elveszti, és a középső sorból is hiányzik a bal oldali $(2;\ 2)$ pixel, tehát három előtérpixel marad:

```
0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

Kézzel: TP $= 3$ (a közös $(1;\ 2), (1;\ 3), (2;\ 3)$), FP $= 0$, FN $= 3$ (a GT $(2;\ 2), (3;\ 2), (3;\ 3)$ pixelei hiányoznak), TN $= 42$. Így

$$J = \frac{3}{3+0+3} = \frac{1}{2}, \qquad D = \frac{6}{6+0+3} = \frac{2}{3}, \qquad A = \frac{3+42}{48} = \frac{45}{48} = 0{,}9375.$$

A GT doboz: $x_{\min} = 2$, $x_{\max} = 4$, $y_{\min} = 1$, $y_{\max} = 4$, terület $2 \cdot 3 = 6$. A pred doboz: $x_{\min} = 2$, $x_{\max} = 4$, $y_{\min} = 1$, $y_{\max} = 3$, terület $2 \cdot 2 = 4$. A metszet területe $2 \cdot 2 = 4$, az unió $6 + 4 - 4 = 6$, tehát a box-IoU $\frac{2}{3} \approx 0{,}6667$.

A GT centroid:

$$c(G) = \frac{1}{6}\Big((1;\ 2) + (1;\ 3) + (2;\ 2) + (2;\ 3) + (3;\ 2) + (3;\ 3)\Big) = \left(\frac{12}{6}, \frac{15}{6}\right) = (2;\ 2{,}5).$$

A pred centroid:

$$c(\hat{G}) = \frac{1}{3}\Big((1;\ 2) + (1;\ 3) + (2;\ 3)\Big) = \left(\frac{4}{3}, \frac{8}{3}\right) \approx (1{,}3333;\ 2{,}6667).$$

A centroid-hiba

$$\|c(G) - c(\hat{G})\|_2 = \sqrt{\left(\frac{2}{3}\right)^2 + \left(-\frac{1}{6}\right)^2} = \frac{\sqrt{17}}{6} \approx 0{,}6872.$$

Ez a kézi számpélda lesz a labor numerikus ellenőrzése.

39.3.6 A labor: geometriai kontrollmodul

A labor egy NumPy-alapú kiértékelő modult valósít meg, amelyet a 39.4. Alkalmazási riport és átadás exportja és auditja is importál; ezért a dobozkonvenció egyetlen helyen, egy névvel ellátott `Box(xmin, ymin, xmax, ymax)` struktúrában és a `validate_box` központi validátorban rögzített. A sorrend mindenhol a `metadata.csv` xyxy sorrendje; a maszkból számolt dobozt a kód mind a 360 képre összeveti a metadata dobozával, és egy aszimmetrikus doboz teljes láncát (maszk → doboz → terület → JSON-export → visszaolvasás) is ellenőrzi. A maszkon mért IoU neve `mask_iou`, a dobozon mérté `bbox_iou`. A munkakönyvtár `lab/project3`; a GlyphLab adat a `lab/data` alatt van.

```
# lab/project3/n155_geom_control.py - n155 geometriai metrika kontroll, xyxy dobozokkal.
"""Egyetlen dobozkonvenció: Box(xmin, ymin, xmax, ymax), nullától indexelt,
félig nyílt [xmin,xmax) x [ymin,ymax); üres maszk szentinelje (-1,-1,-1,-1)."""
from __future__ import annotations
import json
from pathlib import Path
from typing import NamedTuple
import numpy as np
import pandas as pd

DATA = Path(__file__).resolve().parents[1] / "data"
H = W = 24


class Box(NamedTuple):
    xmin: int
    ymin: int
    xmax: int
    ymax: int


EMPTY = Box(-1, -1, -1, -1)


def validate_box(b: Box, h: int = H, w: int = W) -> Box:
    """Központi validátor: a szentinel vagy érvényes, nemüres, a képen belüli doboz."""
    b = Box(*map(int, b))
    if b == EMPTY:
        return b
    if not (0 <= b.xmin < b.xmax <= w and 0 <= b.ymin < b.ymax <= h):
        raise ValueError(f"érvénytelen xyxy doboz: {tuple(b)}")
    return b


def conf_counts(pred: np.ndarray, target: np.ndarray) -> dict[str, int]:
    """Binarizált 0/1 maszkok hibakategóriái; logitra nem alkalmazható.

    A logitból a hívó előbb képez bináris predikciót: G_hat = (z > 0).
    """
    p = np.asarray(pred)
    t = np.asarray(target)
    if p.shape != t.shape:
        raise ValueError(f"pred/target alak eltérés: {p.shape} vs {t.shape}")
    if not np.all(np.isin(p, (0, 1))) or not np.all(np.isin(t, (0, 1))):
        raise ValueError("conf_counts: csak 0/1 bináris maszk; a logitot előbb küszöböld")
    p = p.astype(bool)
    t = t.astype(bool)
    tp = int(np.logical_and(p, t).sum())
    fp = int(np.logical_and(p, ~t).sum())
    fn = int(np.logical_and(~p, t).sum())
```

```
    tn = int(np.logical_and(~p, ~t).sum())
    return {"TP": tp, "FP": fp, "FN": fn, "TN": tn}


def bbox(mask: np.ndarray) -> Box:
    """Maszkból xyxy doboz; üres maszkra a szentinel."""
    ys, xs = np.nonzero(np.squeeze(mask))
    if ys.size == 0:
        return EMPTY
    return validate_box(Box(int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1))


def box_area(b: Box) -> int:
    b = validate_box(b)
    return 0 if b == EMPTY else (b.xmax - b.xmin) * (b.ymax - b.ymin)


def box_iou(a: Box, b: Box) -> float:
    """bbox-IoU; két üres doboz 1, pontosan egy üres 0 (a 39.1. alfejezet
    ↪ üreseset-konvenciója)."""
    a, b = validate_box(a), validate_box(b)
    if a == EMPTY or b == EMPTY:
        return 1.0 if a == b else 0.0
    iw = max(0, min(a.xmax, b.xmax) - max(a.xmin, b.xmin))
    ih = max(0, min(a.ymax, b.ymax) - max(a.ymin, b.ymin))
    inter = iw * ih
    return inter / (box_area(a) + box_area(b) - inter)


def centroid(mask: np.ndarray) -> tuple[float, float] | None:
    ys, xs = np.nonzero(np.squeeze(mask))
    if ys.size == 0:
        return None
    return float(ys.mean()), float(xs.mean())


def metric_dict(pred: np.ndarray, target: np.ndarray) -> dict:
    c = conf_counts(pred, target)
    tp, fp, fn, tn = c["TP"], c["FP"], c["FN"], c["TN"]
    out = {**c,
           "accuracy": (tp + tn) / (tp + fp + fn + tn),
           "mask_iou": tp / (tp + fp + fn) if (tp + fp + fn) else 1.0,
           "dice": 2 * tp / (2 * tp + fp + fn) if (2 * tp + fp + fn) else 1.0,
           "bbox_iou": box_iou(bbox(pred), bbox(target)),
           "n_pixels": int(pred.size)}
    cp, ct = centroid(pred), centroid(target)
    out["centroid_error"] = (float(np.hypot(ct[0] - cp[0], ct[1] - cp[1]))
                             if cp is not None and ct is not None else None)
    return out


def main() -> None:
    target = np.array([
        [0,0,0,0,0,0,0,0],
        [0,0,1,1,0,0,0,0],
        [0,0,1,1,0,0,0,0],
        [0,0,1,1,0,0,0,0],
        [0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0]], dtype=np.uint8)
    pred = np.array([
        [0,0,0,0,0,0,0,0],
        [0,0,1,1,0,0,0,0],
        [0,0,0,1,0,0,0,0],
```

```
        [0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0],
        [0,0,0,0,0,0,0,0]], dtype=np.uint8)

    out_dir = Path("n155_control")
    out_dir.mkdir(exist_ok=True)
    np.save(out_dir / "control_pred.npy", pred)
    np.save(out_dir / "control_target.npy", target)

    report = metric_dict(pred, target)
    assert abs(report["mask_iou"] - 0.5) < 1e-12
    assert abs(report["dice"] - 2/3) < 1e-12
    assert abs(report["accuracy"] - 45/48) < 1e-12
    assert abs(report["bbox_iou"] - 2/3) < 1e-12
    assert abs(report["centroid_error"] - np.sqrt(17)/6) < 1e-12

    # Aszimmetrikus doboz teljes lánca: maszk -> bbox -> terület -> export -> visszaolvasás.
    m = np.zeros((H, W), dtype=np.uint8)
    m[5:9, 2:7] = 1                                  # sorok 5..8, oszlopok 2..6
    b = bbox(m)
    assert b == Box(xmin=2, ymin=5, xmax=7, ymax=9) and box_area(b) == 20
    assert Box(*json.loads(json.dumps(list(b)))) == b
    assert box_iou(b, Box(2, 5, 7, 9)) == 1.0 and box_iou(b, Box(5, 2, 9, 7)) < 1.0

    # A metadata.csv xyxy oszlopai minden képre egyeznek a maszkból számolt dobozzal.
    masks = np.load(DATA / "masks.npy").astype(np.uint8)
    meta = pd.read_csv(DATA / "metadata.csv")
    for r in meta.itertuples():
        assert bbox(masks[r.index]) == Box(r.xmin, r.ymin, r.xmax, r.ymax), r.index
    report["metadata_bbox_check"] = f"{len(meta)} kép, mind egyezik"
    report["foreground_ratio_by_split"] = {
        split: float(masks[g["index"].to_numpy()].mean()) for split, g in meta.groupby("split")}

    (out_dir / "n155_control_report.json").write_text(
        json.dumps(report, indent=2, ensure_ascii=False), encoding="utf-8")
    print(json.dumps(report, indent=2, ensure_ascii=False))


if __name__ == "__main__":
    main()
```

A `metric_dict` üres- és határeset-konvenciói egy helyen: `mask_iou` és `dice` akkor 1, ha a megfelelő nevező nulla (üres–üres predikció és cél); `bbox_iou` két üres dobozra 1, pontosan egy üresre 0; `centroid_error` értéke `None`, ha valamelyik maszk üres; az `accuracy` nevezője $N > 0$, ezért ott nincs nullosztás. A `conf_counts` bemenete már binarizált 0/1 maszk, nem logit: a logitból a hívó a $\hat{G} = \mathbf{1}[z > 0]$ küszöböt alkalmazza, a függvény pedig a nem 0/1 értéket `ValueError`-ral utasítja el, így a néma `astype(bool)` nem tehet előtérré egy 0,3 valószínűséget vagy egy negatív logitot.

Futtatás:

```
cd lab/project3
python n155_geom_control.py
```

A kód a `n155_control/control_pred.npy` és `n155_control/control_target.npy` fájlokat menti, majd a `n155_control/n155_control_report.json` jelentést írja. Az ellenőrzés tűrése 10^{-12}: a TP/FP/FN/TN és a dobozgeometria egész alapú, a IoU/Dice/accuracy kontrollok racionálisak, de a centroid-hiba $\sqrt{17}/6$, azaz irracionális, ezért a tűrés a float64 kerekítési hibáját fedi, nem azt jelenti, hogy minden ellenőrzött mennyiség racionális. Az aszimmetrikus kontrolldoboz a $[2;\ 7) \times [5, 9)$ téglalap, azaz `Box(xmin=2, ymin=5, xmax=7, ymax=9)`, területe $5 \cdot 4 = 20$. Ha valaki a koordinátákat $(x_{\min}, x_{\max}, y_{\min}, y_{\max})$ sorrendben adná át, a `Box(2,7,5,9)`

átmenne a `validate_box`-on, mert $2 < 5$ és $7 < 9$; a validátor önmagában tehát nem jelez hibát, a `b == Box(2,5,7,9)` egyezés viszont elbukik. A kód emellett a teljes x/y transzpozíciót is ellenőrzi: `box_iou(b, Box(5,2,9,7)) == 0,1111 < 1`. **Futási jegyzőkönyv** (eredeti GlyphLab-adat, Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, CPU; determinisztikus, seed nélkül): minden `assert` teljesült, a `metadata_bbox_check` értéke „360 kép, mind egyezik", az előtérarány splitenként train 0,0201, val 0,0211, test 0,0208. A valós predikciókra a modult a 39.4. Alkalmazási riport és átadás `export_delivery.py` és `report_audit.py` szkriptje alkalmazza.

39.3.7 Per-scene aggregálás, páros teszt és osztályarány

A GlyphLab megfigyelési egysége a **scene_id**: egy jelenethez két zajos nézet tartozik, és a két nézet nem független. A 38.3. Végső teszt és páros korrupció alfejezetben bizonyított csoportátlag-lemma szerint, ha minden csoportban azonos V megfigyelésszám van, a csoportátlag megegyezik a per-image átlaggal. Itt minden scene-hez $V_s = 2$ view tartozik, tehát a per-scene és a per-image átlag numerikusan egybeesik; a scene-alapú megfogalmazás mégis kötelező, mert a bootstrap és a hibaintervallum csak scene-egységekre értelmes.

Legyen $J_{s,v}^{\text{clean}}$ a tiszta tesztkép és $J_{s,v}^{\text{corrupt}}$ a korrupt pár IoU-ja. A korrupt teszt ugyanazon scene/view sorrendet követi, így a párosítás egyértelmű. A páros különbség

$$d_{s,v} = J_{s,v}^{\text{clean}} - J_{s,v}^{\text{corrupt}},$$

a scene-szintű különbség pedig

$$D_s = \frac{1}{V_s} \sum_{v=1}^{V_s} d_{s,v}.$$

Az átlagos hatás $\bar{D} = \frac{1}{S} \sum_s D_s$. A párosítás azért lényeges, mert a két nézet közös jelenetbeli pozíciója és vastagsága korrelációt visz a mérésbe; ha a clean és corrupt méréseket független mintaként kezelnénk, a jelenetek közötti szórást tévesen a beavatkozás bizonytalanságához adnánk.

Az intervallumot a **teszt split** 30 jelenetének scene-szintű D_s értékein, osztályonként rétegzett bootstrapjével ([S03]) számítjuk: a GlyphLab 180 jelenetéből 30 tesztjelenet van (120/30/30 train/val/test), orientációs osztályonként 10; a generátor három osztálya a függőleges (0), a vízszintes (1) és az átlós (2) vonalorientáció (A. A GlyphLab adatkészlet). Minden osztályból külön-külön, visszatevéssel húzunk 10 scene-t, kiszámítjuk a $\bar{D}$ bootstrap-mintát, és a 2,5–97,5 percentilist adjuk. Ez ismételt mintavételi (resampling) értelmezésű, feltételes percentilis intervallum, nem az [S02] t-alapú átlag-CI-je, és nem ok-okozati garancia; 30 scene mellett a szélek bizonytalanok.

Az **osztályarány** két szinten jelenik meg. A jelenetek három orientációs osztálya a generátor szerint kiegyensúlyozott, ezért a bootstrapet rétegezzük. A pixelosztály azonban nem az: az előtér aránya a 24×24-es képen lényegesen kisebb, mint a háttéré, ezért a sima bináris keresztentrópia- (BCE-) loss és a pixel accuracy háttértorzított lehet. A per-scene átlag minden jelenetet egyenlő súllyal vesz, a globális pixel-IoU viszont a nagyobb előtérterületű képek felé húz.

39.3.8 Tipikus tévedés ellenpéldával

A leggyakoribb tévedés a pixel accuracy önálló használata háttérdomináns szegmentálásnál. Ellenpélda: legyen a kép 24×24, a GT maszk 24 előtérpixelt tartalmaz, a predikció pedig az azonosan nulla maszk. Ekkor TP $= 0$, FN $= 24$, FP $= 0$, TN $= 552$, tehát

$$A = \frac{552}{576} = 0{,}9583, \qquad J = D = 0.$$

A 95,83%-os accuracy azt sugallhatná, hogy a modell kiváló, miközben az előtér- (foreground-) szegmentálás teljesen használhatatlan: egyetlen előtérpixelt sem talál meg. A helyes eljárás az IoU/Dice, valamint az előtér-recall (foreground recall) $\frac{\mathrm{TP}}{\mathrm{TP+FN}}$ és az előtér-precision (foreground precision) $\frac{\mathrm{TP}}{\mathrm{TP+FP}}$ együttes közlése.

A második ellenpélda a doboz és a maszk összekeverésére figyelmeztet: legyen a GT doboz $[0;\ 10) \times [0, 10)$, a pred doboz ugyanez, de a GT maszk a doboz bal, a pred maszk a doboz jobb felét fedi. Ekkor a **független** doboz-predikció (a modell a dobozt és a maszkot külön adja) box-IoU-ja 1, a pixel-IoU 0. Ha a dobozt a 39.1 szerződés szerint a maszkból származtatjuk, ez a példa nem érvényes: a két félmaszk legkisebb befoglaló doboza nem ugyanaz, tehát a származtatott bbox- (befoglaló doboz-) IoU nem 1. Származtatott bboxnál két diszjunkt, mégis ugyanazt a dobozt kifeszítő maszk kell, például a $[0;\ 10)^2$ négyzet két sakktáblaszíne. A doboz tehát lokalizációs összefoglaló, nem helyettesíti a pixelszintű metrikát.

39.3.9 Önellenőrzés és visszatekintés

Ellenőrző kérdések a továbbhaladás előtt:

1. Mi a négy hibakategória shape-je és viszonya N-hez?
2. Miért nem adunk +1-et a doboz szélességéhez?
3. Mi a centroid, és miért marad a centroid-hiba változatlan a közös eltolásra?
4. A GlyphLab-ban mi a független mintavételi egység a maszkmetrikák bootstrapjében?
5. Mikor $J = 0$ akkor is, amikor a GT és a pred nem üres?

A válaszok röviden: a négy skalár összege $H \cdot W$; a félig nyílt konvenció őrzi a metszet–unió formulát; a centroid az átlag linearitása miatt tolódik el, ahogy az 1. lemma mutatja; a mintavételi egység a `scene_id`; $J = 0$, ha TP $= 0$ de FP + FN > 0, azaz a két maszk diszjunkt előterű.

Visszatekintés: a 27.3. Dice, IoU és aggregálás alfejezetben bevezetett $D = \frac{2J}{1+J}$ azonosság és az üres konvenció itt is érvényes; a 38.3. Végső teszt és páros korrupció scene-bootstrapje adja a páros különbség intervallumát; a 35.3. Perturbáció és adversarial példa tanulsága pedig az, hogy a clean és a perturbált bemenet metrikája két külön kockázat, nem egyetlen „robusztusság” szám.

39.3.10 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a half-open doboz területét, és számítsa ki a $[3;\ 8) \times [2, 6)$ doboz területét. Miért hibás a diszkrét $((8-3)+1)((6-2)+1)$ képlet? Elvárt eredmény: a helyes terület, a hibás képlet értéke és az indoklás.

F2 (levezetési, nehézség 2). Az 1. lemma felhasználásával bizonyítsa az 1. következményt. Adjon olyan t eltolást, amelyre a lemma nem alkalmazható egy 24×24-es képen, és indokolja a kimaradást. Elvárt eredmény: teljes levezetés és konkrét számértékű ellenpélda.

F3 (implementációs, nehézség 2). Írjon függvényt, amely két azonos alakú `uint8` maszkra visszaadja a TP, FP, FN, TN, IoU, Dice ötöst. Tesztelje az üres–üres esetre és arra az esetre, amikor a pred minden pixelén 1, a GT pedig azonosan 0. Elvárt eredmény: futtatható kód és a két teszteset számértékei.

F4 (hibakeresési, nehézség 2). Egy mérést végző hallgató a szegmentáló modelljét „nagyon pontosnak” nevezi, mert a validációs accuracy 0,9375. A tényleges predikció minden képen üres maszk. Hogyan lehetséges ez? Számítsa ki a 4×4-es esetet, ahol a GT-ben egyetlen előtérpixel van, és adja meg a javítást. Elvárt eredmény: az ok, a TP/FP/FN/TN számok, valamint a javasolt metrika.

F5 (kísérlettervezési, nehézség 3). Tervezze meg a Projekt III. páros korrupciós kísérletét a szegmentáló modellre. Adja meg a kérdést, a megfigyelési egységet, a metrikát, a statisztikát, a bootstrap stratégiát és a következtetés érvényességi korlátját. Elvárt eredmény: protokoll, amely a 38.3. Végső teszt és páros korrupció és a jelen alfejezet jelöléseit használja.

39.3.11 Megoldások és indoklás

F1. A helyes terület $(8-3)(6-2) = 5 \cdot 4 = 20$. A hibás formula $6 \cdot 5 = 30$. A fél-nyílt intervallum a folytonos élhosszat méri: az intervallum hossza a végpontkülönbség, függetlenül attól, hogy a rácson hány egész index esik bele. A $+1$ a bennfoglalt rácspontok számát adná, ami a metszet–unió geometriai területformulához nem konzisztens.

F2. Az 1. lemma szerint

$$c(\hat{G}+t) - c(G+t) = (c(\hat{G})+t) - (c(G)+t) = c(\hat{G}) - c(G).$$

A norma mindkét oldalon kifejezhető, ezért a hiba egyenlő. A feltételek: $\hat{G}$ és G nem üres, és a $G+t$, $\hat{G}+t$ halmazok a keretben maradnak. Például a 24×24-es képen legyen $M = \{(0;\ 2);\ (0;\ 12)\}$; ekkor $c(M) = (0;\ 7)$. A $t = (0;\ 20)$ eltolásnál $M + t = \{(0;\ 22);\ (0;\ 32)\}$, amelyből a $(0;\ 32)$ pixel kiesik a keretből, így a megfigyelt halmaz $\{(0;\ 22)\}$, centroidja $(0;\ 22)$. A lemma által előírt $c(M) + t = (0;\ 27)$ nem áll elő, mert a kimaradt pixel megváltoztatta az átlagot; ezért az 1. következmény feltétele sérül, és a centroid-hiba itt nem eltolás-invariáns.

F3. Az alábbi kód a labor `conf_counts` és `metric_dict` függvényeire építhető; a két teszteset a NumPy-ben:

```
def conf_counts(pred, target):
    p = pred.astype(bool); t = target.astype(bool)
    return (int((p & t).sum()), int((p & ~t).sum()),
            int((~p & t).sum()), int((~p & ~t).sum()))

# üres-üres: 2x2
tp, fp, fn, tn = conf_counts(np.zeros((2,2), np.uint8),
                             np.zeros((2,2), np.uint8))
iou = 1.0 if (tp+fp+fn) == 0 else tp/(tp+fp+fn)
dice = 1.0 if (2*tp+fp+fn) == 0 else 2*tp/(2*tp+fp+fn)
assert (tp,fp,fn,tn,iou,dice) == (0,0,0,4,1.0,1.0)

# pred minden 1, target 0
tp, fp, fn, tn = conf_counts(np.ones((2,2), np.uint8),
                             np.zeros((2,2), np.uint8))
iou = 1.0 if (tp+fp+fn) == 0 else tp/(tp+fp+fn)
dice = 1.0 if (2*tp+fp+fn) == 0 else 2*tp/(2*tp+fp+fn)
assert (tp,fp,fn,tn,iou,dice) == (0,4,0,0,0.0,0.0)
```

Az alakellenőrzés az `assert pred.shape == target.shape`; a maszk `uint8`, a műveletek `bool`-lá alakítás után egész összeget adnak.

F4. Az ok a háttérdominancia: 16 pixelből a GT-ben csak 1 az előtér, a pred üres. Ekkor TP=0, FP=0, FN=1, TN=15; accuracy 15/16 = 0,9375, IoU 0/1 = 0, Dice 0. A pontosság a TN tömegétől függ, ezért egy triviális üres predikció is magas értéket kap. A javítás: a döntési metrika legyen IoU vagy Dice, és mindig közöljük a foreground recallt és precisiont; ha a modell üres maszkot ad, a recall 0, ami azonnal jelzi a hibát.

F5. Protokollterv. Kérdés: hogyan változik a befagyasztott szegmentáló modell IoU-ja a GlyphLab corrupt tesztjén? Hipotézis: a zaj és fényességgradiens az előtér határán rontja a maszkot. Megfigyelési egység: scene/view pár; a clean és corrupt image ugyanazon scene/view, ezért a párosítás rögzített. Metrika: $J_{s,v}$; hatás: $d_{s,v} = J_{s,v}^{\text{clean}} - J_{s,v}^{\text{corrupt}}$, scene-átlag D_s. Statisztika: $\bar{D} = \frac{1}{30}\sum_s D_s$. Bootstrap: 1000 ismétlés, osztályonként 10 scene visszatevéssel, 2,5–97,5 percentilis. Korlát: a GlyphLab szintetikus, generált adat, nem természetes kép; a korrupció az adatgenerátor definíciója szerint Gauss-zaj (szórás 0,18) és vízszintes fényességgradiens (0-tól 0,1-ig), [0; 1]-re vágva, ugyanazon jelenet–nézet sorrendben (A. A GlyphLab adatkészlet); a CI csak erre a rögzített corrupt előállításra vonatkozik; a modellválasztás a validációs spliten már lezárult. Döntési szabály: a 95%-os percentilis-intervallum nem tartalmazza a nullát, és a $|D_s|$ jelenetszintű medián legalább 0,02 IoU; a bootstrap seedje 20261020, az erőforrás CPU, 1000 ismétlés a befagyasztott modellen.

39.3.12 Rövid összefoglaló

A szegmentáló modell kiértékelése nem egyetlen számon múlik. A TP, FP, FN, TN hibakategóriákból számított IoU és Dice érzékeny az előtérhibára, míg a pixel accuracy háttérdominancia mellett félrevezető. A half-open doboz a lokalizációt, a centroid a tömegközéppont-eltérést méri; a centroid maga ekvivariáns a közös eltolásra ($c(M + t) = c(M) + t$), a centroid-hiba viszont invariáns rá. A GlyphLab értékelése scene-szinten aggregál, a clean–corrupt összevetés páros, a bootstrap egysége a scene_id. A korrekt protokoll az IoU/Dice mellett közli a hibakategóriákat és az osztályarányt, így a triviális üres predikció nem kaphat hamisan magas pontszámot.

A következő lecke, 39.4. Alkalmazási riport és átadás, az alkalmazási riport és átadás: a most definiált metrikákat és páros intervallumot a modellkártya, a hibakorlátok és a felhasználási szerződés nyelvezetévé alakítja.

Felhasznált források:

- [S02] NIST/SEMATECH (n.d.). *e-Handbook of Statistical Methods: Confidence Limits for the Mean*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda352.htm. Hozzáférés: 2026-10-30.
- [S03] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC. A percentilis bootstrap forrása.

39.4 Alkalmazási riport és átadás

39.4.1 Cél, előismeret és induló kérdés

A Projekt III végén nem a modell, hanem az **állítás** a beadandó termék. A 39.3. Geometriai kontroll és metrika labor az utolsó geometriai és metrikai ellenőrzést végezte el; ez az egység azt a csomagot építi fel, amellyel egy másik személy a futtatás nélkül is meg tudja ítélni, hogy a riport egyes mondatai milyen feltételekkel igazak. Előismeret a 39.3. Geometriai kontroll és

metrika metrika- és split-konvenciója, valamint a 38.4. Mentett modellből független inference betöltési egyezés fogalma. Induló kérdés: két kolléga ugyanazt a mentett predikciós fájlt kapja. Az egyik azt írja, hogy a corrupt teszten a jelenetenkénti páros különbségek mediánja +0,10, tehát a tipikus jelenet romlott; a másik azt, hogy a clean és a corrupt medián IoU különbsége −0,10, tehát javulás. Melyiküknek van igaza, és mit kellett volna a riportnak egyértelműen rögzítenie? A fejezet jelkonvenciója végig ugyanaz: $D = \text{clean} - \text{corrupt}$, és pozitív D romlást, negatív D javulást jelent a corrupt oldalon.

39.4.2 Prediction artifact, model card, limitations, source trace, report: definíciók

A Projekt III főága a 39.1.–39.2. bináris szemantikus szegmentálása; ebben az egységben ennek a kimenetéből készül a riport. A modell képenként egy $1 \times 24 \times 24$-es logittérképet ad, a predikált maszk $\hat{M}_i = \mathbf{1}[\sigma(Z_i) > 0{,}5]$ (a küszöb a checkpoint része), és a doboz nem külön tanított kimenet, hanem a predikált maszkból a 39.1. Feladat- és annotációs döntés β leképezésével, a 39.3. Geometriai kontroll és metrika `bbox` függvényével származtatott xyxy doboz; üres predikált maszkra a szentinel $(-1;\ -1;\ -1;\ -1)$. Legyen a tesztképek száma $M = 60$, indexeik $I = \{i_1;\ \ldots;\ i_M\}$ a teljes 360 képes tömbre nézve. Egy származtatott doboz

$$\hat{b}_i = (x_{\min}^{(i)}, y_{\min}^{(i)}, x_{\max}^{(i)}, y_{\max}^{(i)}) \in \mathbb{Z}^4,$$

a ground-truth doboz b_i a függelék A. A GlyphLab adatkészlet szakaszában leírt `metadata.csv` azonos sémájú sora. Nemüres dobozra $0 \le x_{\min} < x_{\max} \le 24$ és ugyanez y-ra; a doboz területe $(x_{\max} - x_{\min})(y_{\max} - y_{\min}) > 0$. A fél-nyílt értelmezés miatt a metszet területe

$$|b \cap \hat{b}| = \max(0, \min(x_{\max}, \hat{x}_{\max}) - \max(x_{\min}, \hat{x}_{\min})) \cdot \max(0, \min(y_{\max}, \hat{y}_{\max}) - \max(y_{\min}, \hat{y}_{\min})),$$

az unió $|b \cup \hat{b}| = |b| + |\hat{b}| - |b \cap \hat{b}|$, és bbox_iou $= |b \cap \hat{b}| / |b \cup \hat{b}|$; ha pontosan az egyik doboz a szentinel, az érték 0; ha mindkét doboz a szentinel (üres–üres), az érték 1, összhangban a 39.3. Geometriai kontroll és metrika üres–üres konvenciójával. A maszkon mért mask_iou $= |M \cap \hat{M}| / |M \cup \hat{M}|$ ettől különböző mennyiség; üres unióra (mindkét maszk üres) az értéke 1. A riport a két metrikát mindig külön néven közli.

Definíció (prediction artifact). A prediction artifact a modell egy rögzített, verziózott, géppel olvasható kimeneti fájlja. A Projekt III-ban ez változatonként (clean, corrupt) egy `(60,1,24,24)` alakú, `uint8` maszktömb és egy ilyen sémájú JSON:

```
{
  "schema_version": "1.1",
  "model_id": "glyphlab-seg-01",
  "artifact_kind": "semantic_mask_predictions",
  "split": "test",
  "variant": "clean",
  "threshold": 0.5,
  "generated_at": "<ISO-8601 időpont, a szkript tölti ki>",
  "mask_file": "artifacts/clean_masks.npy",
  "bbox_convention": "xyxy, nullától indexelt, félig nyílt; üres maszk: [-1,-1,-1,-1]",
  "predictions": [
    {"index": 6, "scene_id": "G0-0003", "view_id": 0, "bbox": [14, 8, 16, 17], "fg_pixels": 18}
  ]
}
```

Négy ellenőrzés tartozik hozzá: (i) az `index` lista pontosan a metadata `split=="test"` sorainak globális indexe, és a `(scene_id, view_id)` párok is ugyanabban a sorrendben jönnek; (ii) minden `bbox` vagy a szentinel, vagy érvényes, a [0; 24] rácson belüli, nemüres xyxy doboz; (iii) a maszktömb alakja `(60,1,24,24)`, értékei {0; 1}; (iv) minden gépi artifact (checkpoint, tanítási napló, maszk- és predikciós fájl, model card, másolt forrás) SHA-256 értéke szerepel a manifestben; a `report.md` az audit után, külön lépésben kerül a láncba. A mintasor a lenti futás clean artifactjának első sora; a fájlt az `export_delivery.py` írja.

Definíció (model card). A model card a kiadott modell **dokumentuma**; specifikációs része: architektúraosztály és paraméterszám, input séma (NCHW, float32, [0; 1], $1 \times 24 \times 24$), output séma, label/target szerződés, tanító split, a kiválasztás szabálya (legkisebb validációs BCE, azaz val loss), checkpoint hash, licenc és érvényességi kör. Mitchell és társai [P30, 4. szakasz] eredeti javaslata szerint a model card ennél több: a „Metrics", „Evaluation Data" és „Quantitative Analyses" szakaszok a kiadott checkpoint **verziózott értékelési összefoglalóját** is tartalmazzák (metrikák, értékelési adat, csoportonkénti és metszetenkénti eredmények). Ebben a projektben a `model_card.json evaluation` mezője rögzíti az értékelés adatát (teszt split, clean és corrupt), metrikáit és azt, hogy a számértékek a manifestben lenyomatolt artifactokon futtatott `report_audit.py` kimenetéből származnak; a számokat, a futásonkénti naplókat, a per-kép táblázatokat és az ablációkat a riport közli, mert az export nem futtatja az auditot. Ez a szétválasztás **helyi dokumentációs döntés**, nem a model card általános definíciója.

Definíció (limitations szakasz). A korlátok szakasz azokat a feltételeket sorolja fel, amelyek nélkül a riport állítása hamis lehet: szintetikus, egyobjektumos, zárt eloszlású adat; nincs üres input szerződés; a corrupt teszt egyetlen zajtípust takar; az IoU nem kalibráció; a képenkénti és scene-szintű metrika nem felcserélhető.

Definíció (source trace). A source trace az állítástól a nyers bizonyítékig vezető lánc: adatgenerátor verziója és seedje, `metadata.csv` és `manifest.json` hash-e, tanító/evaluáló kód azonosítója, függőségi verziók, checkpoint hash, betöltési egyezési jegyzőkönyv és a futtatási parancs. Ez nem helyettesíti a mérést, hanem lehetővé teszi annak újrakeresését.

Definíció (report). A report az az emberi olvasásra szánt dokumentum, amely egy pontosan megfogalmazott állítást, annak feltételeit, a nyers metrikát és az értelmezési határt együtt közli. A Projekt III riportjának kötelező váza: kérdés; protokoll; model card hivatkozás; nyers táblázat; feltétel-ellenőrzési lista; nem futtatott kiterjesztések.

39.4.3 Eredményállítás-felülvizsgálat: az R1 példa és a páros medián levezetése

A levezetési követelmény egy választott eredményállítás teljes feltétel- és bizonyíték-ellenőrzése. Válasszuk a

R1 állítást: *„A `glyphlab-seg-01` checkpoint a **30 megfigyelt tesztjeleneten** legalább* 0,05*-tel rosszabb `mask_iou`-t ad corrupt, mint clean bemeneten, a következő értelemben: a jelenetenkénti* $D_s = I_s^{clean} - I_s^{corrupt}$ *különbségek minta-mediánja legalább* +0,05*. (Ez a megfigyelt mintára vonatkozó állítás. A* populációs* medián $\geq 0{,}05$ állításhoz bizonytalanság kell: előre választott egyoldali konfidenciaintervallum vagy teszt, scene szerinti rétegzett resamplinggel; a 39.3-ban definiált átlagos $\bar{D} = \frac{1}{S}\sum_s D_s$ intervalluma nem a medián intervalluma.)"* Pozitív D_s romlás, ezért R1 a $\mathrm{med}(D) \geq 0{,}05$ egyenlőtlenség.

Ez empirikus állítás, nem tétel. A felülvizsgálat azt kérdezi, hogy milyen feltételek mellett lenne a mondat a jelen adatokra értelmes. A feltételek:

1. **Rögzített checkpoint.** A `models/seg_best.pt` SHA-256 értéke a manifestben szerepel, betöltése a 39.2. Encoder–decoder vagy lokalizációs fej architektúrájával és a model card input-sémájával történt, `eval()` módban. A 38.4. Mentett modellből független inference $(10^{-4}, 10^{-4})$ logit-egyezési küszöbe csak a CPU-float32 környezetben értelmezett; itt a küszöbölt maszkkimenet egyezését a maszkfájl hash-e rögzíti.
2. **Párosított megfigyelés.** A clean és corrupt bemenet ugyanazt a 60 `(scene_id, view_id)` párt tartalmazza azonos sorrendben. A `corrupted_test.npy` sorrendje a `metadata.csv` test sorainak sorrendje; a predikciós artifact `index` mezője ehhez a globális indexhez kötődik.
3. **Tiszta modellválasztás.** A checkpoint kiválasztásakor sem a clean, sem a corrupt teszt nem befolyásolta a döntést; a teszt a protokoll végén futott egyszer, utólagos ráhangolás nélkül.
4. **Metrika-konvenció.** Minden egyes s jelenetre a két view `mask_iou`-jának átlagát vesszük, I_s^{clean} és I_s^{corrupt}; a csoportosítás kulcsa kizárólag a `scene_id`, és minden jelenethez pontosan a 0 és az 1 view tartozik, így 30 csoport van. A hatás a páronkénti $D_s = I_s^{\text{clean}} - I_s^{\text{corrupt}}$ értékek mediánja; $D_s > 0$ romlás.
5. **Érvényességi határ.** Az R1 a GlyphLab 1.0 generátor eloszlásáról szól, nem valós orvosi vagy ipari képekről.

A bizonyíték típusa a 2. feltételre: `metadata.csv` test sorainak indexe és a `corrupted_test.npy` hash-e; a 4. feltételre: az evaluáló kód forrása és egy kézi három-scene kontroll; az 1. feltételre: manifest hash plusz betöltési jegyzőkönyv. Ha bármelyik elem hiányzik, az R1 státusza „feltételes, ellenőrizetlen”, nem pedig „igaz”.

A metrika-konvenció egy nem nyilvánvaló pontot rejt: a páronkénti különbségek mediánja nem azonos a két split mediánjának különbségével. Ezt a következő lemmában rögzítjük.

Lemma (páros medián-hatás és a mediánok különbsége). Legyenek $A = (a_1, \dots, a_n)$ és $C = (c_1, \dots, c_n)$ valós sorozatok, és legyen $D = (a_1 - c_1, \dots, a_n - c_n)$. Ekkor a

$$\operatorname{med}(D) = \operatorname{med}(A) - \operatorname{med}(C)$$

egyenlőség nem teljesül minden sorozatpárra.

Bizonyítás. Ellenpéldával: legyen $A = (0{,}90;\ 0{,}50;\ 0{,}40)$ és $C = (0{,}80;\ 0{,}60;\ 0{,}30)$. Ekkor $D = A - C = (0{,}10;\ -0{,}10;\ 0{,}10)$, rendezve $(-0{,}10;\ 0{,}10;\ 0{,}10)$, tehát $\operatorname{med}(D) = 0{,}10$. Ugyanakkor $\operatorname{med}(A) = 0{,}50$ és $\operatorname{med}(C) = 0{,}60$, így a mediánok különbsége $-0{,}10$. A két mennyiség előjelben is eltér. □

A lemma azt mondja ki, hogy a medián nem lineáris a páros kivonásra nézve; ez az 5.4. Függő képek, seedek és resampling alfejezetből ismert aggregálási óvatosság egy konkrét alakja. A riportnak ezért mindig a D_s értékek mediánját kell közölnie, ha a párosított romlás a kérdés, és a táblázatban a két split mediánját külön oszlopban, a páronkénti D_s statisztikákat pedig további oszlopban kell feltüntetnie. A lemma bizonyítása teljes, de nem bizonyítja R1 igazságát: R1-hez a mért D_s értékek kellenek; ezeket a lenti labor futási jegyzőkönyve adja.

39.4.4 Kézi példa: három scene, két aggregáció

A fenti három scene tekinthető egy kiértékelési csonknak. Tegyük fel, hogy a per-scene IoU-k a lemma ellenpéldájának értékei:

scene	I_s^{clean}	I_s^{corrupt}	$D_s = I_s^{\text{clean}} - I_s^{\text{corrupt}}$
S1	0,90	0,80	+0,10
S2	0,50	0,60	−0,10
S3	0,40	0,30	+0,10

A páronkénti medián hatás +0,10: a rögzített konvenció szerint a tipikus scene a corrupt eloszláson romlott. Aki a clean medián (0,50) és a corrupt medián (0,60) különbségeként számol, −0,10 értéket kap, és tévesen javulást jelentene. A per-image átlag egy harmadik érték: a clean átlag 0,600, a corrupt átlag 0,567, a különbség +0,033; ez szintén nem a páros medián hatás. A példa tehát egyetlen nyers táblázatból három, egymással nem felcserélhető számot állít elő; a riportnak névvel és előjel-értelmezéssel kell ellátnia mindegyiket. Az induló kérdésre tehát a páros medián tekintetében az első kollégának van igaza: a +0,10 a helyes páros medián hatás, a −0,10 a mediánok különbsége, amely nem a páros hatást méri. A riportnak rögzítenie kell a $D = \text{clean} - \text{corrupt}$ konvenciót, az aggregáció típusát (páros medián) és a párosítás kulcsát.

39.4.5 Átadási csomag és ellenőrző kód

Az átadási csomagot a 39.2. Encoder–decoder vagy lokalizációs fej főágának checkpointjából egy dokumentált generálási lépés állítja elő, a `lab/project3` munkakönyvtárban; a csomag szerkezete:

```
lab/project3/delivery/
  report.md
  model_card.json
  manifest.json
  models/
    seg_best.pt
  artifacts/
    clean_predictions.json
    clean_masks.npy
    corrupt_predictions.json
    corrupt_masks.npy
  src/
    report_audit.py
    n155_geom_control.py
```

Az export a checkpointban rögzített küszöbbel maszkot predikál a clean és a corrupt tesztre, a maszkból a 39.3. Geometriai kontroll és metrika `bbox` függvényével xyxy dobozt származtat, és a manifestbe a gépi artifactok SHA-256 értékét írja. A `report.md` nem az export kimenete: a szerző az audit után írja meg, majd külön lépésben felveszi a hash-ét a manifestbe; az export által írt manifest tehát a jelentés megszületése előtt nem tartalmazza a `report.md`-t.

```python
# lab/project3/export_delivery.py - 39.4: a főág (39.2) checkpointjából a delivery/ csomag.
# Maszkpredikció küszöbbel, maszkból származtatott xyxy doboz (39.3), hash-manifest, model card.
import datetime, hashlib, json, shutil
from pathlib import Path

import numpy as np
import pandas as pd
import torch

from n155_geom_control import bbox
from segmentation_head import DATA, SmallSegUNet

HERE = Path(__file__).resolve().parent
OUT = HERE / "delivery"
```

```
MODEL_ID = "glyphlab-seg-01"


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def main():
    ckpt = torch.load(HERE / "checkpoints" / "seg_best.pt", map_location="cpu",
↪   weights_only=True)
    model = SmallSegUNet()
    model.load_state_dict(ckpt["model_state_dict"])
    model.eval()
    thr = float(ckpt["threshold"])               # sigmoid(z) > thr  <=>  z > logit(thr)
    meta = pd.read_csv(DATA / "metadata.csv")
    assert (meta["index"].to_numpy() == np.arange(len(meta))).all(), \
        "a metadata indexe nem a 0..N-1 tartomány permutációja"
    test = meta[meta["split"] == "test"]          # a korrupt teszt sorrendje is ez
    inputs = {"clean": np.load(DATA / "images.npy")[test["index"].to_numpy()],
              "corrupt": np.load(DATA / "corrupted_test.npy")}
    for d in ("artifacts", "models", "src"):
        (OUT / d).mkdir(parents=True, exist_ok=True)
    shutil.copyfile(HERE / "checkpoints" / "seg_best.pt", OUT / "models" / "seg_best.pt")
    shutil.copyfile(HERE / "checkpoints" / "seg_train_log.json",
                    OUT / "models" / "seg_train_log.json")
    files = {"checkpoint": "models/seg_best.pt",
             "train_log": "models/seg_train_log.json"}
    for name, x in inputs.items():
        with torch.no_grad():
            prob = torch.sigmoid(model(torch.from_numpy(x.astype(np.float32)))).numpy()
        pred = (prob > thr).astype(np.uint8)                                  # (60,1,24,24)
        np.save(OUT / "artifacts" / f"{name}_masks.npy", pred)
        art = {
            "schema_version": "1.1", "model_id": MODEL_ID,
            "artifact_kind": "semantic_mask_predictions", "split": "test",
            "variant": name, "threshold": thr,
            "generated_at":
            ↪  datetime.datetime.now(datetime.timezone.utc).isoformat(timespec="seconds"),
            "mask_file": f"artifacts/{name}_masks.npy",
            "bbox_convention": "xyxy, nullától indexelt, félig nyílt; üres maszk:
            ↪  [-1,-1,-1,-1]",
            "predictions": [
                {"index": int(r.index), "scene_id": r.scene_id, "view_id": int(r.view_id),
                 "bbox": list(bbox(pred[j])), "fg_pixels": int(pred[j].sum())}
                for j, r in enumerate(test.itertuples())],   # r.index: a metadata index-oszlopa
        }
        (OUT / "artifacts" / f"{name}_predictions.json").write_text(json.dumps(art, indent=1))
        files[name] = f"artifacts/{name}_predictions.json"
        files[f"{name}_masks"] = art["mask_file"]
    manifest = {"files": {k: {"path": v, "sha256": sha256(OUT / v)} for k, v in files.items()},
                "inputs": {n: sha256(DATA / n) for n in
                           ("images.npy", "corrupted_test.npy", "masks.npy", "metadata.csv")}}
    card = {"model_id": MODEL_ID, "architecture": "SmallSegUNet
↪   (lab/project3/segmentation_head.py)",
            "params": ckpt["params"], "input_schema": "float32 NCHW (N,1,24,24), [0,1]",
            "label_target": "cél: 1x24x24 bináris előtérmaszk (uint8, 0/1); a doboz nem tanított
            ↪  target, hanem a maszkból származtatott xyxy",
            "output_schema": "logit (N,1,24,24); maszk = sigmoid > threshold",
            "threshold": thr, "derived_bbox": "maszk legkisebb befoglaló xyxy doboza (n155
            ↪  bbox)",
            "train_split": "metadata.csv split=='train' (240 kép, 120 jelenet); validáció:
            ↪  split=='val' (60 kép, 30 jelenet)",
            "selection_rule": "legkisebb validációs BCE (val_loss) epoch; a napló:
            ↪  models/seg_train_log.json",
```

```
            "checkpoint_sha256": manifest["files"]["checkpoint"]["sha256"],
            "training_log_sha256": manifest["files"]["train_log"]["sha256"],
            "license": "a laborcsomag szerint: kód MIT; adat és dokumentáció CC BY 4.0",
            "validity_scope": "GlyphLab 1.0 szintetikus, egyobjektumos képek",
            "evaluation": {"data": "teszt split (metadata.csv), clean: images.npy, corrupt:
            ↪  corrupted_test.npy",
                           "metrics": "mask_iou, bbox_iou (n155), képenkénti és jelenetszintű
                           ↪  összesítés",
                           "results": "report_audit.py kimenete a manifest artifactjain (lásd
                           ↪  riport)"}}
    (OUT / "model_card.json").write_text(json.dumps(card, indent=2, ensure_ascii=False))
    for src in ("n155_geom_control.py", "report_audit.py"):
        shutil.copyfile(HERE / src, OUT / "src" / src)
    # A manifest a kártyát, a tanítási naplót és a másolt forrásokat is hash-eli;
    # a report.md az audit után, külön lépésben kerül a láncba.
    for extra in ("model_card.json", "src/n155_geom_control.py", "src/report_audit.py"):
        manifest["files"][extra] = {"path": extra, "sha256": sha256(OUT / extra)}
    (OUT / "manifest.json").write_text(json.dumps(manifest, indent=2))
    print(f"delivery kész: {len(test)} teszt-kép, küszöb {thr}, checkpoint epoch
    ↪  {ckpt['epoch']}")


if __name__ == "__main__":
    main()
```

A `report_audit.py` a fenti feltétel-ellenőrzési lista **gépi része**: az 1–2. feltétel hash- és párosításellenőrzését, valamint a 4. metrika-konvenciót végzi; a 3. (tiszta modellválasztás) és az 5. (érvényességi határ) dokumentációs feltétel, amelyet a tanítási napló és a riport szövege rögzít, nem az audit. A doboz-geometriát nem írja újra, hanem a csomagba másolt `n155_geom_control.py` modulból importálja, így a xyxy-sorrend és a validátor egyetlen forrásból jön. A jelenetszintű csoportosítás kulcsa csak a `scene_id`; a beépített `selftest` kétjelenetes fixture-rel ellenőrzi, hogy a képenkénti és a jelenetenkénti medián valóban eltérhet, a háromjelenetes fixture-rel pedig a 39.4.4. kézi példáját és a szentinel bbox-IoU-kezelését.

```
#!/usr/bin/env python3
# lab/project3/report_audit.py
"""Projekt III riport-audit: artifact-séma, hash, párosítás, maszk- és bbox-IoU (NumPy +
↪  stdlib).

Jelkonvenció: D_s = I_s(clean) - I_s(corrupt); pozitív D_s romlás a corrupt oldalon,
negatív D_s javulás."""
import argparse, csv, hashlib, json, statistics, sys
from pathlib import Path
import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent))
from n155_geom_control import Box, box_iou, validate_box


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def mask_iou(p, t):
    """Maszk-IoU; üres unió (mindkét maszk üres) esetén 1.0 (n155 konvenció)."""
    p, t = p.astype(bool), t.astype(bool)
    union = np.logical_or(p, t).sum()
    return float(np.logical_and(p, t).sum() / union) if union else 1.0


def per_scene(values, rows):
    """Kulcs csak a scene_id; minden jelenethez pontosan a 0 és 1 view tartozik."""
```

```
    groups = {}
    for v, r in zip(values, rows):
        g = groups.setdefault(r["scene_id"], {})
        assert int(r["view_id"]) not in g, f"duplikált view: {r['scene_id']}/{r['view_id']}"
        g[int(r["view_id"])] = v
    for s, views in groups.items():
        assert set(views) == {0, 1}, f"{s}: a view-k {sorted(views)}"
    return {s: statistics.mean(views.values()) for s, views in groups.items()}


def selftest():
    # Két jelenet, eltérő view-különbségekkel: a képenkénti és a jelenetenkénti medián eltér.
    rows = [{"scene_id": "a", "view_id": 0}, {"scene_id": "a", "view_id": 1},
            {"scene_id": "b", "view_id": 0}, {"scene_id": "b", "view_id": 1}]
    d = [0.4, 0.0, 0.1, 0.1]
    ds = per_scene(d, rows)
    assert len(ds) == 2 and ds == {"a": 0.2, "b": 0.1}
    assert statistics.median(d) == 0.1 and abs(statistics.median(ds.values()) - 0.15) < 1e-12
    # A 39.4.4. három-scene példája: D_s = (0.1, -0.1, 0.1), medián 0.1.
    rows3 = [{"scene_id": s, "view_id": v}
             for s in ("S1", "S2", "S3") for v in (0, 1)]
    ds3 = per_scene([0.1, 0.1, -0.1, -0.1, 0.1, 0.1], rows3)
    assert ds3.keys() == {"S1", "S2", "S3"}
    assert abs(ds3["S1"] - 0.1) < 1e-12
    assert abs(ds3["S2"] + 0.1) < 1e-12
    assert abs(ds3["S3"] - 0.1) < 1e-12
    assert abs(statistics.median(ds3.values()) - 0.1) < 1e-12
    # A szentinel a bbox-IoU-ban is kezelt: üres-üres 1, üres-nemüres 0.
    assert box_iou(validate_box(Box(-1, -1, -1, -1)),
                   Box(-1, -1, -1, -1)) == 1.0
    assert box_iou(validate_box(Box(-1, -1, -1, -1)),
                   Box(0, 0, 1, 1)) == 0.0
    print("selftest OK")


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--clean", required=True)
    p.add_argument("--corrupt", required=True)
    p.add_argument("--data", required=True, help="a GlyphLab data/ könyvtára")
    p.add_argument("--manifest", required=True)
    args = p.parse_args()
    selftest()
    root = Path(args.manifest).resolve().parent
    man = json.loads(Path(args.manifest).read_text())
    for key, f in man["files"].items():                          # hash-lánc
        assert sha256(root / f["path"]) == f["sha256"], key
    for n, h in man["inputs"].items():                           # a tényleges --data fájlok
    ↪ hash-e
        assert sha256(Path(args.data) / n) == h, n
    meta = list(csv.DictReader(open(Path(args.data) / "metadata.csv", newline="")))
    test = [r for r in meta if r["split"] == "test"]
    assert len(test) == 60 and len({r["scene_id"] for r in test}) == 30
    gt_masks = np.load(Path(args.data) / "masks.npy")
    res = {}
    for name, path in (("clean", args.clean), ("corrupt", args.corrupt)):
        art = json.loads(Path(path).read_text())
        preds = art["predictions"]
        # a megadott útvonal a manifest hash-elt fájlja, és a maszkfájl is hitelesített
        ↪ bejegyzés
        assert Path(path).resolve() == (root / man["files"][name]["path"]).resolve(), name
        assert art["mask_file"] == man["files"][f"{name}_masks"]["path"], name
        assert [int(q["index"]) for q in preds] == [int(r["index"]) for r in test]
```

```
        assert [(q["scene_id"], q["view_id"]) for q in preds] == \
               [(r["scene_id"], int(r["view_id"])) for r in test]   # azonos képkészlet és
↪  sorrend
        masks = np.load(root / art["mask_file"])
        assert masks.shape == (60, 1, 24, 24) and masks.dtype == np.uint8
        assert set(np.unique(masks)) <= {0, 1}
        for j, q in enumerate(preds):                          # a doboz és a pixelszám a maszkból
        ↪  származik
            if "fg_pixels" in q:
                assert int(q["fg_pixels"]) == int(masks[j].sum()), j
        assert len({(q["scene_id"], q["view_id"]) for q in preds}) == len(preds)   # nincs
        ↪  duplikált scene/view
        m_iou = [mask_iou(masks[j], gt_masks[int(r["index"])]) for j, r in enumerate(test)]
        b_iou = [box_iou(validate_box(Box(*q["bbox"])),
                         Box(*(int(r[k]) for k in ("xmin", "ymin", "xmax", "ymax"))))
                 for q, r in zip(preds, test)]
        res[name] = {"mask_iou": per_scene(m_iou, test), "bbox_iou": per_scene(b_iou, test)}
    summary = {"sign_convention": "D_s = clean - corrupt; D_s > 0 romlás",
               "n_scenes": len({r["scene_id"] for r in test})}
    for metric in ("mask_iou", "bbox_iou"):
        c, x = res["clean"][metric], res["corrupt"][metric]
        assert set(c) == set(x) and len(c) == summary["n_scenes"]
        D = [c[s] - x[s] for s in sorted(c)]
        summary[metric] = {
            "clean_scene_median": round(statistics.median(c.values()), 6),
            "corrupt_scene_median": round(statistics.median(x.values()), 6),
            "paired_median_D": round(statistics.median(D), 6),
            "mean_D": round(statistics.mean(D), 6)}
    print(json.dumps(summary, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

Az audit a manifest `files` láncát (checkpoint, tanítási napló, kártya és másolt források), az `inputs` hash-eit a tényleges `--data` fájlokkal, a `--clean`/`--corrupt` útvonalak és a maszkfájlok manifestbeli hitelesítését, a `uint8` maszkdtype-ot, a (jelenet, view) duplikátumok kizárását és – ha a predikciós rekord tartalmazza – a `fg_pixels` és a maszk összegének egyezését ellenőrzi. A referenciafutás a Projekt III laborcsomagban futott; a jegyzőkönyv a 39.4.6-ban van. Az ellenőrzés nem bizonyítja a modell minőségét; azt bizonyítja, hogy az artifact sémája, a hash-lánc, a párosítás és a jelenetszintű aggregáció helyes, és hogy a `mask_iou` és a `bbox_iou` külön, azonos előjelkonvencióval kerül a riportba.

39.4.6 Labor: riport, kód és futási parancs

A teljes Projekt III-út a `lab/project3` könyvtárból:

```
cd lab/project3
python annotation_spec.py --data ../data        # 39.1: célszerződés
python train_seg.py                              # 39.2: főág, checkpoints/seg_best.pt
python n155_geom_control.py                      # 39.3: geometriai kontroll
python export_delivery.py                        # 39.4: delivery/ csomag
cd delivery
python src/report_audit.py \
  --clean artifacts/clean_predictions.json \
  --corrupt artifacts/corrupt_predictions.json \
  --data ../../data --manifest manifest.json
```

Futási jegyzőkönyv (eredeti GlyphLab-adat, CPU, Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, PyTorch 2.15.0+cpu, 1 szál). A checkpoint a 39.2. alfejezet referenciafutásából származik

(SHA-256 `b8e8958b…`); az export és az audit a fejezet kódblokkjaiból épített Projekt III laborban futott, a tanítási napló SHA-256 értéke `beb13310…`. Az export két futása azonos checkpoint- és maszkfájl-hasht adott (`clean_masks.npy e25543aa…`, `corrupt_masks.npy b3563886…`); a JSON-artifactok és a manifest hash-e a `generated_at` időbélyeg miatt futásonként más. Az audit két futásának kimenete byte-azonos:

```
{
  "sign_convention": "D_s = clean - corrupt; D_s > 0 romlás",
  "n_scenes": 30,
  "mask_iou": {"clean_scene_median": 1.0, "corrupt_scene_median": 0.7,
               "paired_median_D": 0.297129, "mean_D": 0.321315},
  "bbox_iou": {"clean_scene_median": 1.0, "corrupt_scene_median": 0.136209,
               "paired_median_D": 0.839373, "mean_D": 0.745821}
}
```

Az R1 a **30 megfigyelt tesztjelenet** mintájára vonatkozik (a medián +0,297); a populációs mediánra vonatkozó, bizonytalanságot is tartalmazó állítás külön, előre rögzített intervallumot igényel, amely ebben a futásban nem szerepel; az ilyen intervallum a mintavételi eljárásra vonatkozó, feltételes állítás [S03].

A beadandó `report.md` minimális szerkezete, a fenti futás értékeivel:

```
# Alkalmazási riport: GlyphLab szegmentálás (glyphlab-seg-01)

## Állítás
R1: a jelenetenkénti D_s = mask_iou(clean) - mask_iou(corrupt) mediánja  0,05
(pozitív D_s romlás a corrupt oldalon).

## Feltétel-ellenőrzés
| Feltétel | Bizonyíték | Státusz |
|---|---|---|
| checkpoint hash | manifest.json > files.checkpoint.sha256 | ellenőrzött (audit) |
| párosítás | metadata test sorrend + artifact index, scene_id, view_id | ellenőrzött (audit) |
| modellválasztás | models/seg_train_log.json (manifest hash), legkisebb val BCE | dokumentált,
↪  nem az audit ellenőrzi |
| metrika | report_audit.py selftest (2- és 3-scene fixture) | ellenőrzött (audit) |
| adathatár | GlyphLab 1.0, szintetikus | korlát |

## Nyers eredmény
| mennyiség | mask_iou | bbox_iou |
|---|---|---|
| clean per-scene medián | 1,000 | 1,000 |
| corrupt per-scene medián | 0,700 | 0,136 |
| páros medián D_s (clean - corrupt) | +0,297 | +0,839 |
| scene-ek száma | 30 | 30 |

## Értelmezés
R1 ezen a futáson teljesül: med(D) = 0,297  0,05, tehát romlás.
Ez a megfigyelt 30 tesztjelenetre vonatkozik; a populációs mediánra való
kiterjesztés feltételes, mert a futás nem tartalmaz rá előre rögzített
bizonytalansági intervallumot.
A származtatott doboz sokkal érzékenyebb: néhány hamis előtérpixel is
a kép széléig nyújtja a befoglaló téglalapot.

## Nem futtatott kiterjesztés
A valós ipari/orvosi validáció terv, nem eredmény.
```

Értelmezés. A clean teszten mindkét medián 1: a jelenetek legalább felében a `mask_iou` és a `bbox_iou` is 1,0; ez mediánként értendő, nem jelenti, hogy minden jelenet hibátlan. A korrupt teszten a `mask_iou` mediánja 0,70-re, a `bbox_iou`-é 0,14-re esik; a páros mediánok +0,30 és +0,84, mindkettő romlás. A két metrika eltérése nem ellentmondás: a maszk-IoU néhány hamis

előtérpixelt arányosan büntet, a maszkból származtatott befoglaló doboz viszont egyetlen távoli hamis pixeltől is a kép széléig nyúlhat. Például ha a valódi objektum a bal felső 4×4-es blokkban van, és egyetlen hamis előtérpixel a jobb alsó sarokban jelenik meg, a befoglaló doboz a teljes 24×24-es képet átfogja, így a metszet 16 pixel marad, az unió viszont 576 pixelre nő, és a `bbox_iou` $16/576 \approx 0{,}028$ lesz. Ezért nevezzük a két mennyiséget külön, és ezért nem szabad a dobozmetrikát a maszkmetrika helyett jelenteni. Más könyvtárverzióval vagy más CPU-n az utolsó jegyek eltérhetnek; a saját audit-kimenetet és a manifest hash-eit kell archiválni.

39.4.7 Feladatok

F1 (fogalmi, nehézség 1). Egy hallgató a model cardba beírja a legutóbbi clean teszt `mask_iou`-értékét (és a hozzá tartozó `bbox_iou`-t), a riportba pedig a betöltési Python parancs teljes implementációját. Azonosítsa, melyik dokumentum melyik információnak a helye, és adjon példát arra, milyen hiba keletkezik, ha a kettő felcserélődik.

F2 (levezetési, nehézség 2). Igazolja a lemma állítását az $A = (0{,}80;\ 0{,}50;\ 0{,}20)$, $C = (0{,}70;\ 0{,}60;\ 0{,}10)$ sorozatokra, és adja meg mind a páronkénti medián hatást, mind a mediánok különbségét. Ellenőrizze, hogy itt is előjelváltás történik-e.

F3 (implementációs, nehézség 2). Írjon `invalid_bboxes(predictions)` függvényt (az `invalid_bbox_finder.py` fájlban), amely egy `predictions` JSON-listát kap, és visszaadja a hibás rekordok `index` mezőjének listáját. Egy doboz hibás, ha valamely koordinátája a $[0;\ 24]$ intervallumon kívül esik, vagy nem teljesül $x_{\min} < x_{\max}$, $y_{\min} < y_{\max}$; a $(-1;\ -1;\ -1;\ -1)$ szentinel érvényes (üres maszk), azt nem kell hibásnak jelölni. A megoldásnak tartalmazzon egy három elemből álló, két hibás dobozt tartalmazó tesztinputot.

F4 (hibakeresési, nehézség 2). A következő riportkód a corrupt párosítást a `DataFrame` újraindexelése után végzi el:

```
clean_test = meta[meta.split == "test"].reset_index(drop=True)
corrupt_preds = pd.read_json("corrupt_predictions.json")
corrupt_preds = corrupt_preds.sort_values("index").reset_index(drop=True)
D = clean_test["iou_clean"] - corrupt_preds["iou_corrupt"]
```

Mi a hiba oka, mi a következménye a páros medián hatásra, és hogyan javítható úgy, hogy a javítás ellenőrizhető legyen?

F5 (kísérleti/transzfer, nehézség 3). Egy mérnök a GlyphLab szegmentáló modellt és a belőle származtatott dobozokat ipari alkatrészek dobozolására szeretné használni. Tervezzen érvényességi határelemzést: mi a megfigyelési egység, milyen baseline, metrika és annotációs forrás kell, és mely következtetések **nem** vonhatók le a szintetikus eredményből? A terv lehet nyitott, de minden elemnél adja meg az alternatíva indokát.

39.4.8 Megoldások és indoklás

F1. A kiadott checkpoint clean teszten mért `mask_iou`- és `bbox_iou`-értéke a model card `evaluation` mezőjébe tartozik [P30, 4. szakasz], de csak a checkpoint hash-ével, a teszt split azonosítójával és az értékelő kód (`report_audit.py`) verziójával együtt; a „legutóbbi” futás értéke, hivatkozás nélkül, nem. A pixel accuracy ebben a projektben nem riportált metrika, mert háttérdomináns lehet; a riport a `mask_iou`-t és a `bbox_iou`-t közli külön néven. A betöltési kód teljes implementációja egyik dokumentumba sem való: a verziózott forráskódban van, a model card az architektúraosztályt, az input sémát és a checkpoint hash-ét rögzíti, a riport pedig a kódverzióra hivatkozik. Hibás esetben a model card egy másik checkpoint vagy split

mérését hordozza, mintha az a kiadott modellé lenne; a riport pedig újraimplementálja a betöltést ahelyett, hogy a verziózott kódra hivatkozna, így a source trace megszakad.

F2. $D = (0{,}10;\ -0{,}10;\ 0{,}10)$, rendezve $(-0{,}10;\ 0{,}10;\ 0{,}10)$, tehát a páronkénti medián $+0{,}10$. A mediánok: $\mathrm{med}(A) = 0{,}50$, $\mathrm{med}(C) = 0{,}60$, különbség $-0{,}10$. Itt nem előjelváltás, hanem előjelütközés van: a páronkénti hatás pozitív, a mediánok különbsége negatív. A példa újra megerősíti, hogy a két aggregáció nem felcserélhető.

F3. Egy lehetséges megoldás:

```
def invalid_bboxes(predictions):
    out = []
    for pr in predictions:
        x1, y1, x2, y2 = (float(v) for v in pr["bbox"])
        if (x1, y1, x2, y2) == (-1.0, -1.0, -1.0, -1.0):
            continue  # érvényes szentinel: üres maszk
        if not (0.0 <= x1 and x2 <= 24.0 and x1 < x2):
            out.append(pr["index"])
            continue
        if not (0.0 <= y1 and y2 <= 24.0 and y1 < y2):
            out.append(pr["index"])
    return out

sample = [
    {"index": 10, "bbox": [-1.0, -1.0, -1.0, -1.0]},  # üres maszk szentinélje
    {"index": 11, "bbox": [23.0, 1.0, 26.0, 8.0]},     # x_max > 24
    {"index": 12, "bbox": [4.0, 9.0, 4.0, 15.0]},      # nulla szélesség
]
print(invalid_bboxes(sample))  # [11, 12]
```

A 10. elem a $(-1;\ -1;\ -1;\ -1)$ szentinel, amely üres maszkot jelöl, ezért érvényes és nem kerül a kimenetbe; a 11. dobozra $x_{\max} = 26 > 24$, a 12. dobozra $x_{\min} = x_{\max}$, tehát nulla szélességű. Alakellenőrzés: a bemenet hossza 3, a kimenet hossza 2, a kimenet elemei a bemeneti rekordok `index` mezői közül valók.

F4. A hiba oka: a `reset_index(drop=True)` után mindkét tábla indexe 0-tól újraszámozódik, ezért a `clean_test["iou_clean"] - corrupt_preds["iou_corrupt"]` kivonás nem a globális `index` mező, hanem a sorpozíció szerint párosít: a pandas a sorokat az indexcímke alapján igazítja, és a két új, 0-tól $n-1$-ig terjedő címkesorozat mindenhol „egyezik”, még akkor is, ha a mögöttes képek sorrendje eltér. Következmény: a D_s értékek hamis párokból készülnek, a páronkénti medián hatás torzul, és a riport akár előjelhibás következtetést is közölhet. Javítás: a két táblát a globális `index` mezőn kell összekapcsolni, nem pozíció szerint:

```
merged = clean_test.merge(corrupt_preds, on="index", how="inner", validate="one_to_one")
assert len(corrupt_preds) == len(clean_test)
assert len(merged) == len(clean_test), "hiányzó index"
assert (merged["scene_id_x"] == merged["scene_id_y"]).all()
assert (merged["view_id_x"] == merged["view_id_y"]).all()
D = merged["iou_clean"] - merged["iou_corrupt"]
```

Az `inner` összekapcsolás a hiányzó indexeket kizárja, az `outer` viszont NaN sorokat hozna, amelyek a D_s különbségben észrevétlenül tovább terjednének; a `validate="one_to_one"` a kulcsduplikációt azonnal hibává teszi. A `(scene_id, view_id)` párok egyezése azt ellenőrzi, hogy ugyanazokat a képeket párosítottuk.

F5. Egy lehetséges terv. Megfigyelési egység: ipari alkatrész-sorozat, mert az egy gyártási sorozaton belüli képek függők. A GlyphLab scene_id-jének szerepét a sorozatazonosító veszi át. Adat: saját, licencelt képek maszkkal vagy dobozzal; az annotációt két független annotátor közötti egyezéssel kell kontrollálni. Split: sorozatszinten train/val/test, az osztály- vagy

pozícióeloszlás rétegzésével. Baseline: fix középpontú vagy a kép szélénél található legnagyobb intenzitású doboz; a baseline-t a 39.3. Geometriai kontroll és metrika alfejezet metrikáival (`mask_iou`/`bbox_iou`, scene- vagy sorozatszintű aggregálás) kell mérni, és az erősségét nem lehet előre kijelenteni, azt az új adaton kell megmérni. Metrika: per-sorozat átlag IoU, páros clean–corrupt vagy éles–homályos összehasonlítás, üres inputra vonatkozó külön szabály. Ami **nem** következik a GlyphLab eredményből: az ipari pontosság semmilyen konkrét értéke, a ritka hibák gyakorisága, a fényviszony- vagy anyagváltás alatti viselkedés, és a biztonsági alkalmazhatóság. A szintetikus modell legfeljebb kiinduló implementációs ellenőrzés; az ipari átvitelhez új mérés és emberi felülvizsgálati protokoll kell. Döntési szabály: az átvitel akkor állítható, ha az új adaton a sorozatszintű IoU 95%-os intervallumának alsó vége meghaladja a baseline ugyanazon intervallumának felső végét, és a két független annotátor közötti Dice legalább 0,8; az erőforrás két annotátor és 40 sorozat.

39.4.9 Tipikus tévedés ellenpéldával

A leggyakoribb hiba az, hogy a szintetikus lokalizáció pontosságát orvosi vagy ipari validációként olvassák. A GlyphLab teszt **minden** képe pontosan egy, a generátor szerződése szerint jelen lévő alakzatot tartalmaz; az „objektum nélküli kép” esetét a tanító adat nem tartalmazza, ezért a modell erre a tartományra nem kapott felügyeletet. Az üres vagy idegen textúrájú bemenetre vonatkozó viselkedés nem ellenőrzött: a predikált maszkból származtatott doboz üres maszkra a szentinelt adja, de hogy a modell ilyen bemeneten mit predikál, azt külön mérés nélkül nem lehet tudni. Konkrét ellenpélda a keretből: egy 24×24, [0; 1]-beli, de előtér nélküli inputon az IoU-ellenőrzés nem is értelmezhető, mert a ground-truth doboz hiányzik; aki ilyenkor az artifactban szereplő dobozt használná, az IoU-t 0-val vagy kihagyással számolná, miközben a riport R1 állítása ezt a tartományt nem fedi le. Az állítás tehát csak a rögzített szintetikus eloszlásra vonatkozik; a valós bevetéshez az üres-input szerződést külön kell definiálni.

39.4.10 Önellenőrzés és visszatekintés

Ellenőrző kérdések: (1) A fenti kézi példában a páronkénti medián +0,10 (romlás), a mediánok különbsége −0,10; ez a lemma ellenőrzése, és mutatja, miért kell az előjel-konvenciót a riportban kimondani. (2) A prediction artifact nem a riport, mert az artifact gépi, verziózott, a riport emberi értelmezést és korlátokat is tartalmaz. (3) A forráslánc megszakad, ha a checkpoint hash nincs a manifestben, vagy ha a corrupt párosítást globális index helyett új sorszámmal végzik. Visszatekintés: a 38.4. Mentett modellből független inference betöltési egyezés két szintje, a hash és a numerikus tolerancia; a 35.4. Korlátok, eredet és döntési felelősség adathatár, amely szerint a clean accuracy nem shift alatti garancia; a 27.4. Instance mask és összetett feladat instance-annotáció, amelyre azért nincs itt szükség, mert az alapváltozat egy objektumra lokalizál, így a szemantikus és instance szétválasztás nem terheli az artifact sémáját.

39.4.11 Rövid összefoglaló

A Projekt III átadása öt, egymástól elkülönülő komponensből áll: a verziózott prediction artifact, a kiadott checkpointhoz rögzített model card, a korlátokat rögzítő limitations szakasz, a source trace (hash-lánc) és a feltételeket a nyers méréssel összekötő jelentés. A source trace minden állítást hash-elt inputhoz, kódhoz és checkpoint-hoz köt. A páros clean–corrupt hatást a scene-szintű különbségek mediánjával kell jelenteni, mert a mediánok különbsége ellenpéldánkban előjelet váltott. A riport mérési mezőit a dokumentált főág (tanítás, export, audit) tényleges futása tölti ki; a hallgatói labornak a saját futását a manifest hash-eivel együtt kell archiválnia. A

konvenció végig $D = \text{clean} - \text{corrupt}$, pozitív érték romlás. A szintetikus szegmentálás sem orvosi, sem ipari validációt nem helyettesít. Az induló kérdésre a válasz: a +0,10-es páronkénti medián és a −0,10-es mediánkülönbség ugyanazt a romlást írja le, ezért a riportnak a jelkonvenciót és a medián definícióját előre rögzítenie kell.

39.4.12 Következő lecke

A 40.1. Összetett levezetési vizsga összetett levezetési vizsgán a jelentés állításait úgy kell majd újra elemezni, hogy a forráslánc egy pontjának megszakadása mellett is helyesen azonosítsa a hallgató, mely következtetés marad feltételes és melyik válik hamissá.

39.4.13 Források

- [P30] Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., Gebru, T. (2019). *Model Cards for Model Reporting.* FAT* '19, 220–229. arXiv:1810.03993. https://arxiv.org/abs/1810.03993. Ellenőrzött hely: 4. szakasz (Model Card Sections), a Metrics, Evaluation Data és Quantitative Analyses alpontok.
- [S03] Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap.* Chapman & Hall/CRC. A mintavételi eljárásra vonatkozó, feltételes intervallumértelmezéshez; ebben az egységben nem készítettünk konfidenciaintervallumot, csak a forrás korlátját idéztük.

40 Tudományos szintézis, vizsgafeladatok és lektori átadás

40.1 Összetett levezetési vizsga

40.1.1 Cél, előismeret és induló kérdés

Ez az egység a záró vizsga levezetési része. A cél nem új réteg bemutatása, hanem annak ellenőrzése, hogy a hallgató egy korábban nem látott, kis neurális gráfot önállóan definiál, teljes alakhelyes deriváltját levezeti, és a levezetést numerikus kontrollal megerősíti. A feladat a 36.4. IX. ellenőrző pont: lektorálható modellállítás lektorálható csomagjának matematikai komponensét, a 37.4. Rétegből kis tanulómodell integrált saját réteget és a 38.4. Mentett modellből független inference inference-szerződés pontosságigényét egyetlen vizsgapéldában kapcsolja össze.

Előismeret: a 36.4. IX. ellenőrző pont: lektorálható modellállítás hatkomponensű bírálati kerete, a 37.4. Rétegből kis tanulómodell alfejezetben rögzített modellszintű véges differenciás ellenőrzés, a 38.4. Mentett modellből független inference visszatöltési tolerancia fogalma és a 39.4. Alkalmazási riport és átadás alkalmazási riport határkijelölése. Visszautalás: a 28.1. Embedding, távolság és normalizálás normalizálási azonossága, a 36.1. Állításból kísérleti terv hipotézis/kontroll elválasztása és a 39.1. Feladat- és annotációs döntés annotációs konvenció.

Induló kérdés: elegendő-e egy `loss.backward()` sor sikeres lefutása annak bizonyításához, hogy egy új gráf deriváltját értjük? Becslés: nem. A könyvtári hívás a láncszabály minden lépését elrejti; a vizsga ezért kézzel levezetett, alakellenőrzött és float64 véges differenciával megerősített megoldást vár.

40.1.2 A vizsgagráf definíciója

A vizsgafeladat a következő, a könyvben ebben az összeállításban még nem szereplő kompozíciót elemzi. A bemenet $X \in \mathbb{R}^{N \times C_{in} \times L}$, ahol L a térbeli hossz. Az első lépés 1D keresztkorreláció (*cross-correlation*, kernelmegfordítás nélkül; a deep learning keretek ezt nevezik konvolúciónak [B02, S01]) szimmetrikus nullakitöltéssel (zero padding), stride 1-gyel, dilation 1-gyel, groups=1-gyel:

$$Z_{n,o,j} = b_o + \sum_{i=1}^{C_{in}} \sum_{r=0}^{k-1} \tilde{X}_{n,i,\,j+r}\, K_{o,i,r},$$

ahol $\tilde{X}$ a párnázott bemenet, $K \in \mathbb{R}^{C_{out} \times C_{in} \times k}$, $b \in \mathbb{R}^{C_{out}}$, $j = 0, \dots, L_{out} - 1$, és

$$L_{out} = L + 2p - k + 1.$$

A második lépés elemenkénti ReLU: $A = \mathrm{ReLU}(Z)$, azaz $A_{n,c,j} = \max(0, Z_{n,c,j})$. A harmadik lépés globális átlag-pooling (*global average pooling*, GAP) a térbeli dimenzió felett:

$$h_{n,c} = \frac{1}{L_{out}} \sum_{j=0}^{L_{out}-1} A_{n,c,j}, \qquad h_n \in \mathbb{R}^{C_{out}}.$$

A negyedik lépés L2-normalizálás:

$$u_n = \frac{h_n}{\|h_n\|_2},$$

ahol $\|h_n\|_2 = (\sum_c h_{n,c}^2)^{1/2}$. A veszteség egy rögzített, egységnormájú célvektorra, $t \in \mathbb{R}^{C_{out}}$, $\|t\|_2 = 1$, N megfigyelés átlagában:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} (1 - u_n^\top t) + \frac{\lambda}{2} \|K\|_F^2.$$

A regularizáló tag csak a konvolúciós súlyra vonatkozik, a biasra nem; $\lambda \geq 0$. A $\frac{\lambda}{2}$ konvenció gradiense λK. A ReLU nullponti deriváltját itt 0-nak definiáljuk, ez a PyTorch `F.relu` viselkedése is; a nullpontot a numerikus ellenőrzésben kifejezetten elkerüljük.

40.1.3 A teljes gradiens mint lemma

1. lemma (az új kompozíció teljes gradiense). Tegyük fel, hogy $L_{out} \geq 1$, $p \geq 0$, $k \geq 1$, stride=dilation=1, a padding szimmetrikus nullakitöltés, $\|h_n\|_2 > 0$ minden n-re, és a ReLU nullponti deriváltja 0. Ekkor minden olyan pontban, ahol egyetlen $Z_{n,c,j}$ sem nulla, a $\mathcal{L}$ gradiensei a következők.

(i) Normalizálás és cosinus-veszteség:

$$\nabla_{h_n} \mathcal{L} = -\frac{1}{N\|h_n\|_2} (t - (u_n^\top t) u_n).$$

(ii) Globális átlag-pooling:

$$\nabla_{A_{n,c,j}} \mathcal{L} = \frac{1}{L_{out}} (\nabla_{h_n} \mathcal{L})_c.$$

(iii) ReLU:

$$\nabla_{Z_{n,c,j}} \mathcal{L} = \nabla_{A_{n,c,j}} \mathcal{L} \cdot \mathbf{1}[Z_{n,c,j} > 0].$$

(iv) Bias:

$$\nabla_{b_o} \mathcal{L} = \sum_{n=1}^{N} \sum_{j=0}^{L_{out}-1} (\nabla_Z \mathcal{L})_{n,o,j}.$$

(v) Súlygradiens, $r = 0, \dots, k-1$:

$$\nabla_{K_{o,i,r}}\mathcal{L} = \sum_{n=1}^{N}\sum_{j=0}^{L_{out}-1} (\nabla_Z\mathcal{L})_{n,o,j}\,\tilde{X}_{n,i,\,j+r} + \lambda K_{o,i,r}.$$

A (iv)–(vi) képletekben nincs újabb $1/N$ szorzó: a batch-átlag $1/N$ tényezője egyszer, az (i) lépésben lép be, és onnan a ∇_A, ∇_Z gradiensekkel öröklődik. Ezért két azonos mintából álló batch ($N = 2$) átlagolt veszteségének súly- és biasgradiense megegyezik az egymintás ($N = 1$) értékkel; ha a (iv)–(v) összegeket még egyszer N-nel osztanánk, a gradiens $N = 2$-nél a felére esne.

(vi) Bemeneti gradiens: először a párnázott $\tilde{X}$ gradiensét számítjuk a $s = 0, \dots, L + 2p - 1$ indexeken,

$$\nabla_{\tilde{X}_{n,i,s}}\mathcal{L} = \sum_{o=1}^{C_{out}}\sum_{r=0}^{k-1} \mathbf{1}[0 \le s - r \le L_{out} - 1]\,(\nabla_Z\mathcal{L})_{n,o,\,s-r}\,K_{o,i,r},$$

majd a paddinget levágjuk: $\nabla_{X_{n,i,\ell}}\mathcal{L} = \nabla_{\tilde{X}_{n,i,\ell+p}}\mathcal{L}$ minden $\ell = 0, \dots, L-1$ esetén.

Bizonyítás. (i) Jelölje $f(x) = x/\|x\|_2$. A hányadosszabályból a parciális derivált:

$$\frac{\partial f_c}{\partial x_j} = \frac{\delta_{cj}\|x\|_2 - x_c x_j/\|x\|_2}{\|x\|_2^2} = \frac{1}{\|x\|_2}\Big(\delta_{cj} - \frac{x_c x_j}{\|x\|_2^2}\Big).$$

A zárójelben szereplő $x_c x_j/\|x\|_2^2$ hányados éppen az $(uu^\top)_{cj}$ elem, ahol $u = f(x) = x/\|x\|_2$, δ_{cj} pedig az I egységmátrix eleme; ezért a Jacobi-mátrix

$$J_f(x) = \frac{1}{\|x\|_2}\Big(I_{C_{out}} - uu^\top\Big).$$

A mátrix szimmetrikus, tehát a VJP a 4.3. Jacobi-mátrix, láncszabály és VJP konvenciója szerint [B01] $\nabla_x\mathcal{L} = J_f(x)^\top\nabla_u\mathcal{L} = J_f(x)\nabla_u\mathcal{L}$. A belső veszteségre $\nabla_u(1 - u^\top t) = -t$. Innen az (i) képlet; az $1/N$ faktor a batch-átlag lineáris láncszabályából jön. (ii) A GAP lineáris leképezés, minden j-re $\partial h_{n,c}/\partial A_{n,c,j} = 1/L_{out}$, tehát a VJP a konstans $1/L_{out}$ szorzó. (iii) A ReLU a kimondott konvencióval minden $Z \neq 0$ pontban klasszikusan differenciálható, deriváltja 0 vagy 1; a nullpontot a lemma feltétele kizárja. (iv)–(v) A keresztkorreláció a K elemeiben lineáris, ezért a láncszabály a Z gradiensét a $\tilde{X}$ megfelelő szegmensével súlyozza; a λK tag a $\frac{\lambda}{2}\|K\|_F^2$ deriváltja. (vi) Ugyanaz a linearitás az input irányában: egy adott $\tilde{X}_{n,i,s}$ elem minden olyan (o, j, r) hármasban szerepel, amelyre $s = j + r$, azaz $j = s - r$. A $\mathbf{1}$-feltétel a kimeneti index érvényességét biztosítja; a padding zónákon kívül eső tagok a nullakitöltés miatt eleve nem járulnak hozzá. A padding levágása az utolsó lépés definíció szerint. Alakellenőrzés: $\nabla_h\mathcal{L} \in \mathbb{R}^{N\times C_{out}}$, $\nabla_A, \nabla_Z \in \mathbb{R}^{N\times C_{out}\times L_{out}}$, $\nabla_b \in \mathbb{R}^{C_{out}}$, $\nabla_K \in \mathbb{R}^{C_{out}\times C_{in}\times k}$, $\nabla_X \in \mathbb{R}^{N\times C_{in}\times L}$. ■

Ellenpélda a feltétel szükségességére. Két dolgot kell elkülöníteni. (1) *A normalizáló függvény önmagában:* az egzakt $f(x) = x/\|x\|_2$ a nullában nem definiált, és nem is terjeszthető ki folytonosan: $C = 1$ esetén $f(\varepsilon) = \mathrm{sign}(\varepsilon)$, így a nulla körüli középső differencia $(f(\varepsilon) - f(-\varepsilon))/(2\varepsilon) = 1/\varepsilon \to \infty$. *A stabilizált API nem ugyanez:* a PyTorch `F.normalize` alapértelmezett művelete $h/\max(\|h\|, \epsilon)$, ami $0 < \|h\| < \epsilon$ esetén eltér az egzakt képlettől, és $\epsilon > 0$ mellett a nullában is folytonos. Ezért rögzített ϵ mellett az $\varepsilon \to 0$ középső differencia nem adja a fent írt $1/\varepsilon$ divergenciát; három véges próba, amely mind ϵ fölött van, a limitet nem bizonyítja. Az egzakt normalizálás elméleti szakadását és a stabilizált API viselkedését külön

kell tesztelni, beleértve a $\|h\| < \epsilon$ esetet is. (2) *A teljes gráfban* ez a pont nem így érhető el: $h_n = \mathrm{GAP}(\mathrm{ReLU}(Z_n))$ minden komponense nemnegatív, és $h_n = 0$ pontosan akkor, ha minden $Z_{n,c,j} \le 0$. Ha minden $Z_{n,c,j} < 0$, akkor Z folytonossága miatt egy egész környezetben minden $Z_{n,c,j} < 0$ marad, így a ReLU kimenete és vele h_n azonosan nulla, a veszteség pedig azonosan 1 (a PyTorch `F.normalize` a nullvektorra $u = 0$-t ad); ezen a környezeten a leképezés konstans, tehát folytonos, szakadás nincs, de a gradiens sem informatív. Ez egyben a szükségességi állítás bizonyítása: szakadás csak akkor lehetséges, ha a $h_n = 0$ pontban valamelyik $Z_{n,c,j} = 0$, mert csak ekkor változhat meg a ReLU aktív halmaza a perturbáció hatására. Példa: $C_{in} = 1$, $C_{out} = 2$, $L = 1$, $p = 0$, $k = 1$, $K_{1,1,0} = 1$, $K_{2,1,0} = -1$, $b = 0$, $t = (1;\ 0)^\top$, $X_{1,1,0} = 0$. Ekkor $Z = (0;\ 0)$ és $h = 0$. Ha $X = \varepsilon > 0$, akkor $h = (\varepsilon, 0)$, $u = (1;\ 0)$ és $\mathcal{L} = 0$; ha $X = -\varepsilon$, akkor $h = (0, \varepsilon)$, $u = (0;\ 1)$ és $\mathcal{L} = 1$. A középső differencia $-1/(2\varepsilon)$, ami $\varepsilon = 10^{-2}, 10^{-4}, 10^{-6}$ mellett -50, -5000, $-5 \cdot 10^5$ (float64-ben, PyTorch 2.15.0+cpu alatt futtatva), tehát nem konvergál. Itt a két kizárt eset, a $\|h_n\|_2 = 0$ és a $Z_{n,c,j} = 0$, együtt fordul elő; a lemma mindkét feltétele szükséges, és egyik sem technikai részlet.

40.1.4 Eredeti kézi kis példa

Legyen $N = 1$, $C_{in} = 1$, $L = 3$, $C_{out} = 2$, $k = 2$, $p = 1$, $\lambda = 0$. Bemenet és súly:

$$X = \begin{pmatrix} 1 & 2 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \text{ (alakja } 2 \times 1 \times 2), \qquad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

A párnázott bemenet $\tilde{X} = (0;\ 1;\ 2;\ 1;\ 0)$, $L_{out} = 4$. A keresztkorreláció:

$$Z_1 = (1;\ 3;\ 3;\ 1), \qquad Z_2 = (-1;\ -1;\ 1;\ 1),$$

tehát Z alakja $(1;\ 2;\ 4)$. ReLU után:

$$A_1 = (1;\ 3;\ 3;\ 1), \qquad A_2 = (0;\ 0;\ 1;\ 1).$$

A GAP:

$$h = \begin{pmatrix} 2 \\ 0{,}5 \end{pmatrix}, \qquad \|h\|_2 = \frac{\sqrt{17}}{2}, \qquad u = \frac{1}{\sqrt{17}} \begin{pmatrix} 4 \\ 1 \end{pmatrix}.$$

A cél $t = (1;\ 0)^\top$. A veszteség:

$$\mathcal{L} = 1 - \frac{4}{\sqrt{17}} \approx 0{,}029858.$$

A lemma szerint $u^\top t = 4/\sqrt{17}$, ezért

$$t - (u^\top t)u = \begin{pmatrix} 1/17 \\ -4/17 \end{pmatrix}, \qquad \nabla_h \mathcal{L} = -\frac{2}{\sqrt{17}} \begin{pmatrix} 1/17 \\ -4/17 \end{pmatrix} = \begin{pmatrix} -\frac{2}{17\sqrt{17}} \\ \frac{8}{17\sqrt{17}} \end{pmatrix}.$$

A GAP $1/4$ osztása után, az A_2 két nulláját a ReLU-gradiens lenullázza:

$$\nabla_Z \mathcal{L} = \begin{pmatrix} -\frac{1}{34\sqrt{17}} & -\frac{1}{34\sqrt{17}} & -\frac{1}{34\sqrt{17}} & -\frac{1}{34\sqrt{17}} \\ 0 & 0 & \frac{2}{17\sqrt{17}} & \frac{2}{17\sqrt{17}} \end{pmatrix}.$$

Innen a biasgradiens:

$$\nabla_b \mathcal{L} = \begin{pmatrix} -\frac{2}{17\sqrt{17}} \\ \frac{4}{17\sqrt{17}} \end{pmatrix} \approx \begin{pmatrix} -0{,}02853 \\ 0{,}05707 \end{pmatrix}.$$

A súlygradiens a $\tilde{X}$ szegmenseivel:

$$\nabla_K \mathcal{L} = \begin{pmatrix} -\frac{2}{17\sqrt{17}} & -\frac{2}{17\sqrt{17}} \\ \frac{6}{17\sqrt{17}} & \frac{2}{17\sqrt{17}} \end{pmatrix} \approx \begin{pmatrix} -0{,}02853 & -0{,}02853 \\ 0{,}08560 & 0{,}02853 \end{pmatrix}.$$

A párnázott bemeneti gradiens $s = 0, \dots, 4$:

$$\nabla_{\tilde{X}} \mathcal{L} = \begin{pmatrix} -\frac{1}{34\sqrt{17}} & -\frac{1}{17\sqrt{17}} & \frac{1}{17\sqrt{17}} & -\frac{1}{17\sqrt{17}} & -\frac{5}{34\sqrt{17}} \end{pmatrix},$$

a padding levágása után:

$$\nabla_X \mathcal{L} = \begin{pmatrix} -\frac{1}{17\sqrt{17}} & \frac{1}{17\sqrt{17}} & -\frac{1}{17\sqrt{17}} \end{pmatrix} \approx \begin{pmatrix} -0{,}014267 & 0{,}014267 & -0{,}014267 \end{pmatrix}.$$

Az egzakt törtek a kézi ellenőrzés rögzített várt értékei. A következő kód ugyanezt a levezetést a PyTorch autogradjával veti össze float64-ben.

40.1.5 Konkrét labor: önálló levezetés és numerikus kontroll

Munkakönyvtár: `n157_lab/`. **Fájl:** `derive_check.py`.

Futtatás: `python n157_lab/derive_check.py`. A főút kis helyi CPU, float64, seed rögzített. A kód a fenti gráfot építi fel, kiszámolja az autograd gradienseket, majd saját központi differenciával ellenőrzi a súly-, bias- és bemeneti gradiens minden elemét. Ezután a lemma (i)–(vi) képleteit teljes kézi backwardként (`manual_backward`) is megvalósítja, és (a) a 40.1.4. szakasz példáján, illetve két azonos mintájából álló batch-en, (b) egy független, nem degenerált batch-en ($N = 3$, $C_{in} = 2$, $C_{out} = 3$, $L = 5$, $k = 3$, $p = 1$, $\lambda = 0{,}03$, nem kollineáris h_n és t) veti össze az autograddal és a központi differenciával. A (b) rész azért kell, mert a 40.1.4. szakasz példája $N = 1$, $C_{in} = 1$ és $\lambda = 0$ mellett nem mutatná ki sem a hibás $1/N$-et, sem a regularizációs tag hibáját.

```
import torch
import torch.nn.functional as F

torch.manual_seed(20261020)
dtype = torch.float64
device = torch.device("cpu")
eps = 1.0e-6
atol = 1.0e-7

X = torch.tensor([[[1.0, 2.0, 1.0]]], dtype=dtype, device=device, requires_grad=True)
K = torch.tensor([[[1.0, 1.0]], [[1.0, -1.0]]], dtype=dtype, device=device, requires_grad=True)
b = torch.tensor([0.0, 0.0], dtype=dtype, device=device, requires_grad=True)
```

```
t = torch.tensor([1.0, 0.0], dtype=dtype, device=device)

def forward(xv, kv, bv):
    z = F.conv1d(xv, kv, bv, padding=1)      # (1,2,4)
    a = F.relu(z)                            # (1,2,4)
    h = a.mean(dim=2)                        # (1,2)
    return z, a, h

def loss_fn(xv, kv, bv):
    _, _, h = forward(xv, kv, bv)
    u = F.normalize(h.squeeze(0), dim=0)     # (2,)
    return 1.0 - torch.dot(u, t)

Z, A, h = forward(X, K, b)
assert not torch.any(Z == 0), "ReLU-nullpont: a kozponti differencia itt ervenytelen"

loss = loss_fn(X, K, b)
loss.backward()

print("Z=", Z.detach().numpy())
print("h=", h.detach().numpy())
print("loss=", loss.item())
print("gX=", X.grad.detach().numpy())
print("gK=", K.grad.detach().numpy())
print("gb=", b.grad.detach().numpy())

def central_diff(param, grad):
    flat = param.detach().flatten().clone()
    max_abs = 0.0
    for i in range(flat.numel()):
        orig = flat[i].item()
        flat[i] = orig + eps
        if param is K:
            lp = loss_fn(X, flat.view_as(K), b)
        elif param is b:
            lp = loss_fn(X, K, flat.view_as(b))
        else:
            lp = loss_fn(flat.view_as(X), K, b)
        flat[i] = orig - eps
        if param is K:
            lm = loss_fn(X, flat.view_as(K), b)
        elif param is b:
            lm = loss_fn(X, K, flat.view_as(b))
        else:
            lm = loss_fn(flat.view_as(X), K, b)
        flat[i] = orig
        fd = (lp - lm).item() / (2 * eps)
        max_abs = max(max_abs, abs(fd - grad.detach().flatten()[i].item()))
    return max_abs

print("max fd diff K: %.3e" % central_diff(K, K.grad))
print("max fd diff b: %.3e" % central_diff(b, b.grad))
print("max fd diff X: %.3e" % central_diff(X, X.grad))
assert central_diff(K, K.grad) < atol
assert central_diff(b, b.grad) < atol
assert central_diff(X, X.grad) < atol

# A lemma (i)-(vi) teljes kézi backwardja, általános N, C_in, C_out, k, p és lambda mellett.
def manual_backward(xv, kv, bv, tv, lam, pad):
    with torch.no_grad():
        z = F.conv1d(xv, kv, bv, padding=pad)                    # (N,C_out,L_out)
        h = F.relu(z).mean(dim=2)                                # (N,C_out)
        n_batch, l_out, k = h.shape[0], z.shape[2], kv.shape[2]
```

```
        nrm = h.norm(dim=1, keepdim=True)
        u = h / nrm
        g_h = -(tv - (u @ tv)[:, None] * u) / (n_batch * nrm)      # (i), az egyetlen 1/N
        g_z = (g_h[:, :, None] / l_out) * (z > 0)                  # (ii)-(iii)
        x_pad = F.pad(xv, (pad, pad))                              # (N,C_in,L+2p)
        g_b = g_z.sum(dim=(0, 2))                                  # (iv)
        g_k = torch.stack([torch.einsum("noj,nij->oi", g_z, x_pad[:, :, r:r + l_out])
                           for r in range(k)], dim=2) + lam * kv   # (v)
        g_xp = torch.zeros_like(x_pad)
        for r in range(k):                                         # (vi): s = j + r
            g_xp[:, :, r:r + l_out] += torch.einsum("noj,oi->nij", g_z, kv[:, :, r])
        return g_k, g_b, g_xp[:, :, pad:pad + xv.shape[2]]         # padding levágása

def general_loss(xv, kv, bv, tv, lam, pad):
    h = F.relu(F.conv1d(xv, kv, bv, padding=pad)).mean(dim=2)
    u = h / h.norm(dim=1, keepdim=True)
    return (1.0 - u @ tv).mean() + 0.5 * lam * (kv ** 2).sum()

def fd_grad(fun, args, which):
    """Központi differencia az args[which] tenzor minden elemére."""
    base = args[which]
    g = torch.zeros_like(base)
    for i in range(base.numel()):
        plus, minus = base.clone(), base.clone()
        plus.view(-1)[i] += eps
        minus.view(-1)[i] -= eps
        ap, am = list(args), list(args)
        ap[which], am[which] = plus, minus
        g.view(-1)[i] = (fun(*ap) - fun(*am)) / (2 * eps)
    return g

# (a) A 40.1.4. szakasz példája (N=1, lambda=0) és két azonos mintája (N=2): a gradiens nem
↪  feleződhet.
gK1, gb1, gX1 = manual_backward(X.detach(), K.detach(), b.detach(), t, 0.0, 1)
assert torch.allclose(gK1, K.grad) and torch.allclose(gb1, b.grad) and torch.allclose(gX1,
↪  X.grad)
gK2, gb2, gX2 = manual_backward(torch.cat([X.detach()] * 2), K.detach(), b.detach(), t, 0.0, 1)
assert torch.allclose(gK2, gK1) and torch.allclose(gb2, gb1)
assert torch.allclose(gX2, torch.cat([gX1] * 2) / 2)    # mintánként 1/N: a bemeneti gradiens
↪  feleződik
print("(a) kezi backward == autograd; N=2 azonos mintaval K, b gradiens valtozatlan")

# (b) Független kontroll: N=3, C_in=2, C_out=3, L=5, k=3, p=1, lambda=0.03, nem kollineáris t és
↪  h.
gen = torch.Generator().manual_seed(7)
Xr = torch.randn(3, 2, 5, generator=gen, dtype=dtype) + 1.0
Kr = torch.randn(3, 2, 3, generator=gen, dtype=dtype) * 0.5
br = torch.tensor([0.2, 0.1, 0.3], dtype=dtype)
tr = torch.tensor([0.6, 0.0, 0.8], dtype=dtype)
lam, pad = 0.03, 1
Zr = F.conv1d(Xr, Kr, br, padding=pad)
hr = F.relu(Zr).mean(dim=2)
cos_ht = (hr / hr.norm(dim=1, keepdim=True)) @ tr
print("min|Z| = %.3f, min||h|| = %.3f, cos(h,t) =" % (Zr.abs().min(), hr.norm(dim=1).min()),
      cos_ht.numpy().round(4))
assert Zr.abs().min() > 100 * eps and hr.norm(dim=1).min() > 0      # nincs ReLU-töréspont, h !=
↪  0
assert cos_ht.abs().max() < 0.99 and (hr > 0).sum(dim=1).min() >= 2 # nem degenerált irány
args = [Xr.clone().requires_grad_(), Kr.clone().requires_grad_(), br.clone().requires_grad_(),
↪  tr, lam, pad]
general_loss(*args).backward()
manual = manual_backward(Xr, Kr, br, tr, lam, pad)                  # (gK, gb, gX)
auto = (args[1].grad, args[2].grad, args[0].grad)
```

```
fdiff = (fd_grad(general_loss, [Xr, Kr, br, tr, lam, pad], 1),
         fd_grad(general_loss, [Xr, Kr, br, tr, lam, pad], 2),
         fd_grad(general_loss, [Xr, Kr, br, tr, lam, pad], 0))
for name, m, a, f in zip(("K", "b", "X"), manual, auto, fdiff):
    d_auto, d_fd = (m - a).abs().max().item(), (m - f).abs().max().item()
    print("(b) %s: max|kezi-autograd| = %.1e, max|kezi-fd| = %.1e, max|g| = %.3f"
          % (name, d_auto, d_fd, m.abs().max().item()))
    assert d_auto < 1e-12 and d_fd < atol and m.abs().max() > 1e-3
```

Futási jegyzőkönyv és értelmezés. A rögzített környezetben (Python 3.12.3, PyTorch 2.15.0+cpu, float64, CPU, 1 szál) a szkript minden `assert`-en átment. A kimenet a 40.1.4. szakasz kézi értékeit reprodukálja: `loss` $= 0{,}0298575$, `gX` $\approx (-0{,}014267;\ 0{,}014267;\ -0{,}014267)$, `gb` $\approx (-0{,}028534;\ 0{,}057067)$. (a) A teljes kézi backward a 40.1.4. szakasz példáján egyezik az autograddal; két azonos mintából álló $N = 2$ batch-en a K és b gradiense változatlan, tehát nem feleződik, a bemeneti gradiens viszont mintánként a felére esik, mert minden minta X-e külön változó, és mindegyik $1/N$ súllyal szerepel. (b) A független kontrollban $\min|Z| = 0{,}032$, $\min_n \|h_n\|_2 = 1{,}923$, $\cos(h_n, t) = 0{,}9113;\ 0{,}9225;\ 0{,}9286$; a kézi és az autograd gradiens legnagyobb eltérése $1{,}2 \cdot 10^{-16}$, a kézi gradiens és a központi differencia eltérése $6{,}8 \cdot 10^{-11}$ (K), $2{,}9 \cdot 10^{-11}$ (b) és $4{,}8 \cdot 10^{-11}$ (X). A három `max fd diff` sor az autograd és a központi differencia abszolút eltérése: $6{,}2 \cdot 10^{-11}$ (K), $2{,}1 \cdot 10^{-11}$ (b) és $1{,}6 \cdot 10^{-10}$ (X), mind az `atol=10^{-7}` korlát alatt.

A kódban a `Z==0` ellenőrzés szándékosan megelőzi a numerikus differenciát: a ReLU nullpontjában az autograd által választott 0 meredekség konvenció, a központi differencia viszont az ottani féloldali ugrás miatt nem konvergál a klasszikus deriválthoz. A választott pontban minden Z elem -1, 1 vagy 3, tehát a nullától legalább 1 távol van, és a vizsgálat sima (l. 10.3. Gradiensellenőrzés és numerikus hibamérce); a (b) részben az `assert` legalább 100ε távolságot követel meg a nullától.

40.1.6 Feladatok

F1 (fogalmi, nehézség 1). Magyarázza meg, miért nem lineáris a teljes $X \mapsto \mathcal{L}$ leképezés, és nevezze meg a két pontot, ahol a linearitás elromlik. Adjon konkrét számpéldát: a 40.1.4. szakasz X-ére és $2X$-ére mi a $\mathcal{L}$ érték, és miért cáfolja ez a linearitást? Elvárt beadandó: legfeljebb fél oldal indoklás a két ponttal és a két veszteségértékkel.

F2 (levezetési, nehézség 2). A 40.1.4. szakasz numerikus példáján, $\lambda = 0$ mellett vezesse le a $\nabla_b \mathcal{L}$ és $\nabla_X \mathcal{L}$ értékét a lemma (iv) és (vi) képletéből. Adja meg az egzakt törteket és a közelítő tizedes értékeket öt jegyre, majd ellenőrizze az alakokat: $\nabla_b \in \mathbb{R}^2$, $\nabla_X \in \mathbb{R}^{1\times 1\times 3}$. Elvárt beadandó: a levezetés minden közbülső összeggel.

F3 (implementációs, nehézség 2). Írjon NumPy függvényt, amely a lemmát a $N = 1$, $C_{in} = 1$, $C_{out} = 2$, $k = 2$, $p = 1$, $L = 4$, $\lambda = 0$ esetre implementálja: a forward, a ∇_h, a ∇_Z, ∇_K, ∇_b, ∇_X kiszámítását. Válasszon olyan bemenetet, amelyen egyetlen Z elem sem nulla, és h nem párhuzamos t-vel. Hasonlítsa össze a PyTorch autograd gradienseivel azonos bemeneten, 10^{-10} abszolút toleranciával. *Degenerációs részfeladat:* mutassa meg, hogy $C_{out} = 1$ és $h > 0$ esetén $u = 1$, a veszteség lokálisan konstans, és minden gradiens nulla; magyarázza meg, miért nem ellenőrzi ez az eset az implementációt. Elvárt beadandó: futtatható kód, a teszteset és a toleranciaellenőrzés kimenete, valamint a degenerációs eset indoklása.

F4 (hibakeresési, nehézség 2). Az alábbi hibás gradiensszámítás a normalizálási lépést egyszerűsíti:

```
def wrong_grad_h(h, t):
    return -t / np.linalg.norm(h)
```

Magyarázza meg, melyik komponensben és miért tér el ez a lemma (i) képletétől, számítsa ki a hibás és a helyes $\nabla_h \mathcal{L}$-t a 40.1.4. szakasz adataira, és mutassa meg, hogy a központi differencia a helyes értéket erősíti meg. Elvárt beadandó: ok, következmény, javított képlet és a numerikus összevetés.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kontrollált kísérletet annak eldöntésére, hogy a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab 38.4. Mentett modellből független inference checkpointjának embeddingjein az L2-normalizálás javítja-e a jelenetszintű euklideszi retrieval mAP@5-öt a nyers embeddinghez képest. Ne futtassa; adja meg a hipotézist, a kontrollt, a változó tényezőt, a metrikát, a jelenet-alapú splitet és bootstrap egységet, valamint a következtetési korlátokat. Elvárt beadandó: preregisztrált terv a 36.1. Állításból kísérleti terv sablonja szerint.

40.1.7 Megoldások és indoklás

F1 megoldás. Egy F leképezés akkor lineáris, ha $F(\alpha x + \beta y) = \alpha F(x) + \beta F(y)$ minden α, β és x, y esetén. Két töréspont van. (1) A ReLU: $\mathrm{ReLU}(-1) = 0$ és $\mathrm{ReLU}(1) = 1$, de $\mathrm{ReLU}(-1+1) = \mathrm{ReLU}(0) = 0 \neq 1$, tehát a szuperpozíció elromlik. (2) Az L2-normalizálás skálainvariáns: $f(\gamma h) = f(h)$ minden $\gamma > 0$-ra, ezért $X \mapsto u$ nem lehet lineáris. A 40.1.4. szakaszban $\mathcal{L}(X) = 1 - 4/\sqrt{17} \approx 0{,}029858$. A $2X = (2;\ 4;\ 2)$ bemenetre a pozitív ágban Z is duplázódik, ezért $h = (4;\ 1)^\top$, ami ugyanazt az u-t adja: $\mathcal{L}(2X) = 0{,}029858$. Ha a leképezés lineáris lenne, $\mathcal{L}(2X) = 2\mathcal{L}(X) \neq 0$ teljesülne; a valós egyenlőség cáfolja a linearitást.

F2 megoldás. A $\nabla_Z \mathcal{L}$ a 40.1.4. szakaszban adott. A biasgradiens az oszloponkénti összeg: $\nabla_{b_1} = -4 \cdot \frac{1}{34\sqrt{17}} = -\frac{2}{17\sqrt{17}}$; $\nabla_{b_2} = 2 \cdot \frac{2}{17\sqrt{17}} = \frac{4}{17\sqrt{17}}$, tehát $\nabla_b = (-0{,}02853;\ 0{,}05707)^\top$. A bemeneti gradienshez a párnázott összeget számoljuk az $s = j + r$ párokkal. A K két kernelsorát és a ∇_Z két sorát használva:

$$s=0:\ -\frac{1}{34\sqrt{17}},\quad s=1:\ -\frac{1}{17\sqrt{17}},\quad s=2:\ \frac{1}{17\sqrt{17}},\quad s=3:\ -\frac{1}{17\sqrt{17}},\quad s=4:\ -\frac{5}{34\sqrt{17}},$$

ahol például $s=2$-re négy tag adódik össze: a $(j=1, r=0)$ tag $\nabla_{Z_{1,2}} K_{1,1,0} = (-a)\cdot 1 = -a$, a $(j=1, r=1)$ tag $\nabla_{Z_{1,1}} K_{1,1,1} = (-a)\cdot 1 = -a$, a $(j=2, r=0)$ tag $\nabla_{Z_{2,2}} K_{2,1,0} = 4a \cdot 1 = 4a$, a $(j=2, r=1)$ tag pedig $\nabla_{Z_{2,1}} K_{2,1,1} = 0 \cdot (-1) = 0$, mert $\nabla_{Z_{2,1}} = 0$; összegük $-a - a + 4a = 2a = \frac{1}{17\sqrt{17}}$, ahol $a = \frac{1}{34\sqrt{17}}$. A padding levágása ($s = \ell + 1$) a fenti ∇_X hármast adja. Az alakok ellenőrzése: ∇_b két koordináta, ∇_X egy csatorna, három térbeli index, konzisztens a lemma (iv) és (vi) képletével.

F3 megoldás. A lemmát a konkrét 1D esetre megvalósító NumPy kód:

```
import numpy as np
import torch
import torch.nn.functional as F

def manual_1d(X, K, b, t):
    # X: (1,1,L); K: (C_out,1,2); b: (C_out,); t: (C_out,), ||t||=1; p=1, k=2, N=1, lambda=0
    L, C = X.shape[2], K.shape[0]
    Xp = np.concatenate([np.zeros(1), X[0, 0], np.zeros(1)])   # p=1
    Lout = L + 2 - 2 + 1
    Z = np.array([[b[o] + np.sum(K[o, 0] * Xp[j:j + 2]) for j in range(Lout)] for o in
↪   range(C)])
    assert np.all(Z != 0), "ReLU-nullpont"
    h = np.maximum(Z, 0.0).mean(axis=1)                        # (C_out,)
    hnorm = np.linalg.norm(h)
```

```
    assert hnorm > 0, "||h|| = 0: a lemma nem alkalmazható"
    u = h / hnorm
    loss = 1.0 - u @ t
    gh = -(t - (u @ t) * u) / hnorm                          # (i), N=1
    gZ = (gh[:, None] / Lout) * (Z > 0)                      # (ii)-(iii)
    gb = gZ.sum(axis=1)                                      # (iv)
    gK = np.array([[[np.sum(gZ[o] * Xp[r:r + Lout]) for r in range(2)]] for o in range(C)])  #
↪   (v)
    gXp = np.zeros(L + 2)
    for s in range(L + 2):                                   # (vi)
        for o in range(C):
            for r in range(2):
                if 0 <= s - r < Lout:
                    gXp[s] += gZ[o, s - r] * K[o, 0, r]
    return loss, gXp[1:1 + L][None, None, :], gK, gb

def torch_ref(X, K, b, t):
    Xt, Kt, bt = (torch.tensor(a, requires_grad=True) for a in (X, K, b))
    h = F.relu(F.conv1d(Xt, Kt, bt, padding=1)).mean(dim=2)[0]
    loss = 1.0 - F.normalize(h, dim=0) @ torch.tensor(t)
    loss.backward()
    return loss.item(), Xt.grad.numpy(), Kt.grad.numpy(), bt.grad.numpy()

X = np.array([[[1.0, 2.0, -1.0, 3.0]]])
K = np.array([[[1.0, 0.5]], [[-0.5, 1.0]]])
b = np.array([0.1, 0.2])
t = np.array([0.6, 0.8])
for name, m, a in zip(("loss", "gX", "gK", "gb"), manual_1d(X, K, b, t), torch_ref(X, K, b, t)):
    print(name, "max abs diff = %.1e" % np.max(np.abs(np.asarray(m) - np.asarray(a))))
    assert np.allclose(m, a, rtol=0, atol=1e-10)
gX = manual_1d(X, K, b, t)[1]
assert np.abs(gX).max() > 1e-3                               # nem nulla: a teszt nem triviális

# Degenerációs eset: C_out = 1, h > 0  =>  u = 1, a veszteség lokálisan konstans, minden
↪  gradiens 0.
_, gX1, gK1, gb1 = manual_1d(X, K[:1], b[:1], np.array([1.0]))
print("C_out=1: max|grad| =", max(np.abs(g).max() for g in (gX1, gK1, gb1)))
```

A kód a $p = 1$, $k = 2$ esetre specializált: általában $L_{out} = L + 2p - k + 1$ és a kernelablak $X_p[j : j + k]$; a beégetett konstansok a vizsgafeladat konkrét méretét tükrözik.

A választott bemeneten $Z_1 = (0{,}6;\ 2{,}1;\ 1{,}6;\ 0{,}6;\ 3{,}1)$, $Z_2 = (1{,}2;\ 1{,}7;\ -1{,}8;\ 3{,}7;\ -1{,}3)$, tehát nincs ReLU-nullpont, a második csatornában az aktív halmaz nem teljes; $h = (1{,}6;\ 1{,}32)$ és $\cos(h, t) = 0{,}9719$, azaz h nem párhuzamos t-vel. A rögzített környezetben (Python 3.12.3, PyTorch 2.15.0+cpu, float64, CPU, 1 szál) futtatva a kézi és az autograd érték legnagyobb eltérése 0 (loss), $1{,}9 \cdot 10^{-17}$ (∇_X), $5{,}6 \cdot 10^{-17}$ (∇_K) és $4{,}2 \cdot 10^{-17}$ (∇_b), és $\max|\nabla_X| = 0{,}0304$, tehát a teszt nem a nullgradienst hasonlítja össze. A degenerációs esetben $C_{out} = 1$, $h > 0$, ezért $u = h/|h| = 1$ minden kis perturbációra, $\mathcal{L} = 1 - t = 0$ konstans, és a lemma (i) szerint $\nabla_h = -(t - (ut)u)/|h| = 0$; a futás is $\max|\nabla| = 0$-t ad. Egy ilyen teszt bármely, nullát visszaadó hibás implementációt is elfogadna, ezért ellenőrzésre nem alkalmas.

F4 megoldás. A hiba oka: a `wrong_grad_h` kihagyja a normalizálás Jacobi-mátrixának $uu^\top$ vetületi tagját, mintha a célgradiens közvetlenül a normalizálatlan h-ra hatna. Következmény: a céllal merőleges irányú komponens gradiense eltűnik. A 40.1.4. szakaszban a helyes $\nabla_h \mathcal{L} = (-\frac{2}{17\sqrt{17}}, \frac{8}{17\sqrt{17}})$, a hibás pedig $-\frac{t}{\|h\|_2} = (-\frac{2}{\sqrt{17}}, 0)$. Az első komponens pontosan 17-szeres hibája $(-2/(17\sqrt{17})$ helyett $-2/\sqrt{17})$ mellett a második komponens nulla, miközben a központi differencia a h_2 perturbációjára 0,11413 értéket ad. A javítás a lemma (i) teljes képlete: $\nabla_h = -\frac{1}{\|h\|}(t - (u^\top t)u)$; ezzel mindkét komponens egyezik a numerikus kontrollal.

F5 megoldásminta. *Hipotézis:* a GlyphLab teszt splitjén a normalizált embedding euklideszi retrieval mAP@5-e magasabb, mint a nyers embeddingé. *Adat és egység:* a checkpoint a 38.4. Mentett modellből független inference `glyphlab_convnet_v1` modellje (a model cardban rögzített SHA-256-tal), az embedding a classifier utolsó lineáris rétege előtti 32-dimenziós ReLU-aktiváció; a teszt a GlyphLab `test` split 60 képe, 30 scene, minden scenenek két view-ja, ugyanazzal a `(scene_id, view_id)` párosítással, mint a 39.4 riportban; a független bootstrap egység a scene_id. *Változó tényező:* L2-normalizálás alkalmazása a kinyert embeddingre; a modell súlyai rögzítettek, nincs újratanítás. *Metrika:* mindkét irányú view-query/gallery átlagolt mAP@5; a per-scene bootstrap CI a 36.2. Ismétlések és eredménybizonytalanság szerint. *Döntési szabály:* a páros, jelenetszintű különbség (normalizált − nyers) 95%-os bootstrap-konfidenciaintervalluma teljes egészében 0 felett van; két külön CI átfedésének vizsgálata nem döntési szabály. *Korlátok:* a GlyphLab toy adat, nem természetes képi benchmark; az eredmény csak ehhez az adatszerződéshez és ehhez a checkpointhoz kötött. Normalizálás után a kétféle rangsor ekvivalens: egységvektorokra a négyzetes euklideszi távolság $2 - 2\cos$, ezért az euklideszi és a koszinusz-rangsor ugyanaz; a kísérlet valójában a nyers euklideszi és a koszinusz-rangsor összevetése, ahol a nyers ágban a norma is befolyásol. A bootstrap seedje 20261020, az erőforrás CPU, 10 000 jelenetszintű újramintavétel a befagyasztott checkpointon.

40.1.8 Tipikus tévedés, önellenőrzés és visszatekintés

Tipikus tévedés. A memorizált API-hívás nem helyettesíti az önálló levezetést. Ha a hallgató a `F.normalize` hívása után a célgradienst változatlanul továbbítja, azaz $\nabla_h \mathcal{L} = -t$-t ír, a 40.1.4. szakaszban $\nabla_h = (-1;\ 0)^\top$-t kapna a helyes $(-0{,}02853;\ 0{,}11413)^\top$ helyett. Az első komponens hibája körülbelül 35-szörös, a második komponens pedig teljesen hiányzik. A központi differencia azonnal kimutatja az eltérést; ezért a vizsgán a levezetés és a numerikus kontroll együtt kötelező, nem felcserélhető.

Önellenőrzés. Minden levezetési lépésben: (a) a tenzoralakok egyeznek-e a lemma kimeneti alakjaival; (b) a p, L_{out} peremfeltételek teljesülnek-e; (c) a ReLU nullpontot elkerültük-e; (d) a λK tag csak a súlygradiensbe került-e; (e) a központi differencia a megadott 10^{-6} lépésközzel és 10^{-7} toleranciával egyezik-e. A nullvektor normalizálása a lemmán kívül eső, nem differenciálható eset; a gyakorlatban a GAP utáni nulla csatornavektort külön esetként kell kezelni.

Visszatekintés. A 28.1. Embedding, távolság és normalizálás levezetése szerint egységnormájú vektorokra a koszinusz-hasonlóság és a négyzetes euklideszi távolság rendezése ekvivalens; ezért a jelen feladat cosinus-vesztesége és az euklideszi retrieval a normalizálás után ugyanazt a geometriát használja. A 36.1. Állításból kísérleti terv tanulsága, hogy a kísérleti hipotézist a futás előtt kell rögzíteni; az F5 terv éppen ezt a preregisztrációt gyakorolja. A 39.1. Feladat- és annotációs döntés annotációs konvenciója pedig arra emlékeztet, hogy a cél alakja és az üres eset definíciója a feladat szerves része: itt a target t egységnormájú vektor, és a $\|h\| = 0$ eset kizárása ugyanolyan szerződéses feltétel, mint a bbox üres célja.

40.1.9 Rövid összefoglaló

A vizsga egy új gráfot vezetett le: 1D keresztkorreláció, ReLU, globális átlag-pooling, L2-normalizálás, cosinus-veszteség és L2-súlyregularizálás kompozícióját. Az 1. lemma rögzíti a peremfeltételeket, és lépésenként igazolja a nemlineáris, pooling- és konvolúciós VJP-ket. Az ellenpélda a nulla normájú esetben mutatja a differenciálhatóság határát. A kézi példa egzakt törtekkel, a labor float64 központi differenciával ellenőriz. Az öt feladat a fogalmi cáfolattól a preregisztrált kísérleti tervig terjed, és a memorizált API-hívás helyett a levezetés és a numerikus kontroll együttesét kéri számon.

40.1.10 Következő lecke

A 40.2. Kísérleti és hibakeresési vizsga egység a kísérleti és hibakeresési vizsgát rendezi csomaggá: egy rögzített modellen és adaton kell diagnózist adni, hibás gradiens- vagy adatfolyamot lokalizálni, és a javítást kontrollált újrafuttatással igazolni.

40.1.11 Felhasznált források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.
- [S01] Stanford CS231n kurzus (n.d.). *Convolutional Neural Networks*. Kurzusjegyzet. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.

40.2 Kísérleti és hibakeresési vizsga

40.2.1 Cél, előismeret és induló kérdés

Ez az egység vizsga, nem új tananyag-bevezetés: a korábbi fejezetek eszközeit egyetlen, korábban nem látott hibakötegen kell alkalmazni. Az előfeltétel a 40.1. Összetett levezetési vizsga levezetési vizsga mellett a 39.2. Encoder–decoder vagy lokalizációs fej encoder–decoder és loss-redukciós ismerete, a 36.2. Ismétlések és eredménybizonytalanság seed-ismétlési protokollja és a 28.2. Sziámi és triplet cél triplet-cél marginfeltétele. A vizsgázó nem azt bizonyítja, hogy emlékszik a tanult hibákra, hanem hogy új hibahelyzetben versengő magyarázatokat tud felállítani, és mindegyikhez olyan mérést rendel, amely a többit kizárja.

Induló kérdés: egy GlyphLab-orientációosztályozó (a függelék A. A GlyphLab adatkészlet szakasza szerinti adaton) validációs pontossága 98,3% (59/60 kép), a tiszta teszt pontossága 41%. A hallgató azt mondja: „biztos dropout maradt bekapcsolva”. Miért nem tekinthető ez még diagnózisnak, és milyen szerkezeti ellenőrzések döntik el, hogy a hiba címkeeltolódás, split-szivárgás vagy train/eval módhiba?

40.2.2 Definíciók és a vizsga objektumai

Hibadiagnózis (fault diagnosis). Adott egy megfigyelt anomália E (például „val acc $\gg$ test acc”) és egy $H_1, \dots, H_k$ versengő hipotéziscsalád. A H_i hipotézishez tartozik egy $P_i(O)$ predikció valamely O megfigyelhető statisztikára nézve. A diagnózis akkor teljes, ha találunk olyan O mérést, amelynek eredménye legalább egy hipotézist kizár, és a fennmaradó halmazt a kísérleti naplóban megjelöljük.

Leakage (információszivárgás). A tanítási folyamat olyan információhoz fér, amely a telepített modell számára a kiértékeléskor nem lenne elérhető, vagy a split függetlenségi egysége sérül. A GlyphLab szerződésben a független egység a `scene_id`: egy jelenet mindkét nézete ugyanabba a splitbe tartozik, ezért képenkénti véletlen split leakage. Ugyancsak leakage, ha a normalizáló statisztikát a teljes 360 képre számoljuk, mert a teszteloszlás átlaga és szórása beépül a tanított modell bemeneti transzformációjába.

Label bug (címkehiba). A bemeneti tensor és a célérték közötti megfeleltetés sérül: eltolás, helytelen sorrend, vagy olyan geometriai transzformáció, amely a címkét nem őrzi meg. A

GlyphLab `corrupted_test.npy` sorrendje a metadata teszt-sorrendjével egyezik; ha a kiértékelés a `labels[:60]` szeletet használja, az a teljes címketömb első hatvan eleme, azaz a 0–59. képindex címkéje. A metadata `index` mezője rendeli a címkét a képhez, a split-indexek nincsenek rendezve, ezért ez a szelet nem feltétlenül a train split; az viszont biztos, hogy csak akkor lenne helyes, ha a corrupt teszt 60 képe éppen a 0–59. indexű kép lenne, amit a kód nem ellenőriz és a szerződés nem garantál. A helyes szelet a teszt-indexekből képzett `labels[test_idx]`.

Train/eval módhiba. Állapotfüggő rétegek — Dropout és BatchNorm — más számítást végeznek tanításban és kiértékelésben. A [D12] dokumentáció szerint a BatchNorm statisztikakezelése a két módban eltérhet; a dropout tanításban véletlen maszkkal skáláz, kiértékelésben identitásként viselkedik. A hiba akkor áll fenn, ha a validáció vagy teszt a `model.train()` állapotban fut.

Wrong reduction (helytelen redukció). A loss numerikus értéke nem a definiált célfüggvényt valósítja meg, mert a `mean`/`sum` és a nevező nem a szándékolt megfigyelési egységre vonatkozik. Tipikus példa, amikor a számláló a teljes batch összege, a nevező viszont egy képre vonatkozó mennyiség: a veszteség ekkor a batchmérettel skálázódik. A 39.2. Encoder–decoder vagy lokalizációs fej teljes $N \times H \times W$ pixelre vett BCE-átlaga ezzel szemben nem redukcióhiba, hanem legitim cél; az előtér nagyobb súlya csak explicit súlyozással érhető el.

Checkpoint hiba. A betöltött állapot nem teljesíti azt a checkpoint-szerződést, amelyet a felhasználási cél előír; a 12.2. Checkpoint és folytathatóság és a 38.2. Modell, training és best checkpoint két szerződést különít el. (a) *Inference- és kiértékelési checkpoint:* a protokoll által kiválasztott állapot (itt a legkisebb validációs veszteségű epoch) `state_dict`-je, azaz a paraméterek és a perzisztens bufferek, az architektúra és az input séma azonosítójával. Ehhez optimizer-, scheduler-, GradScaler- és RNG-állapot nem kell; a hiányuk nem hiba. Checkpoint-hiba itt például: a kiválasztott helyett az utolsó epoch állapotát töltjük be, más architektúrába töltünk, vagy `strict=False` mellett kulcsok, például a BatchNorm futó statisztikái, csendben elvesznek. (b) *Resume-checkpoint:* a tanítás pontos folytatásához a súlyok mellett az optimizer, a scheduler, AMP esetén a GradScaler és az RNG-állapotok is kellenek; ezek hiánya csak ennek a szerződésnek a megsértése.

Hipotézis a vizsgán. Mérhető következményekkel rendelkező magyarázat. A „szerintem dropout” tipp nem hipotézis, amíg nem adjuk meg, milyen megfigyelhető különbséget jósol a versenytársakhoz képest.

40.2.3 Levezetési kötelezettség: hipotéziselkülönítő lemma

A vizsga levezetési követelménye annak pontos megfogalmazása, hogy egyetlen mérés mikor dönt két hipotézis között.

Lemma (elkülönítő ellenőrzés). Legyen H_1, H_2 két hipotézis, és O egy megfigyelhető statisztika. Tegyük fel, hogy H_1 az $O \in A_1$, H_2 pedig az $O \in A_2$ tartományt predikálja, ahol $A_1, A_2 \subseteq \mathbb{R}$. Ha $A_1 \cap A_2 = \emptyset$, akkor O egyetlen megvalósult mérése legalább az egyik hipotézist kizárja.

Bizonyítás. Ha a mért $o \in A_1$, akkor $o \notin A_2$ a diszjunktság miatt, tehát H_2 predikciója hamis; ha $o \in A_2$, akkor szimmetrikusan H_1 hamis; ha $o \notin A_1 \cup A_2$, akkor mindkettő hamis. Minden esetben legfeljebb egy hipotézis marad konzisztens a méréssel. □

A lemma triviális, de a feltétele nem az: a hallgatónak előre, a mérés előtt meg kell adnia A_1 és A_2 halmazokat, és ezeknek a hipotézisből **szükségszerűen** kell következniük. Két esetet kell elkülöníteni. (1) *Determinisztikus kódhiba-bizonyítás:* egy szerkezeti ellenőrzés kimenete (a splitet átlépő jelenetek száma, a célindex és a képindex egyezése, az aktív `training` flagek listája) a kódból és az adatból egyértelműen következik; itt a lemma szó szerint alkalmazható,

egyetlen futtatás kizár. (2) *Statisztikai hipotézisvizsgálat:* egy pontosság vagy rés zajos, bármely hipotézis mellett több értéket felvehet, ezért a halmazok nem diszjunktak, és a „kizárás” csak döntési szabály, nem logikai következmény. Ehhez meg kell nevezni a mintavételi egységet, a nullhipotézist, a hibaszintet és a döntési szabályt, és a téves elutasítás valószínűsége (α) pozitív. A GlyphLab belső adatszerződése (`lab/ADATOK.md`) szerint a teszt 60 kép, de 30 jelenet két-két nézete: ha a modell kimenete a képtől függ, a két nézet eredménye korrelált lehet, ezért 60 független Bernoulli-próbával nem számolhatunk igazolás nélkül. Konzervatív választás jelenetenként egy nézet: $n = 30$. Ha H_0 szerint a modell a címkétől függetlenül, egyenletesen tippel, akkor a helyes találatok száma $X \sim \mathrm{Bin}(30, 1/3)$; $\alpha = 0{,}01$ szinten a döntési szabály „H_0 elutasítva, ha $X \geq 17$”, mert $P(X \geq 17) = 0{,}0072$ és $P(X \geq 16) = 0{,}0188$ (SciPy `binom.sf`). Ez az 5.2. Várható érték, variancia és likelihood binomiális modelljére épül; a $P(X \geq 17)$ farokvalószínűség az ismételt mintavételi eljárásra vonatkozó, feltételes állítás.

Alkalmazzuk a lemmát a nyitó anomáliára. Három versengő hipotézis:

Hipotézis	Lehetséges teljesítménytünet (nem szükségszerű)	Közvetlen szerkezeti ellenőrzés
H_1: split-szivárgás	magas val acc, val ≫ test; a mérték a nézetek hasonlóságától függ	a tényleges train/val/test indexek `scene_id`-metszete; a normalizáló statisztika forrásindexei
H_2: címkeeltolódás a kiértékelésben	alacsony test acc, permutált átló a konfúziós mátrixban	a kiértékelt célok indexe azonos-e a predikált képek indexével (kép–címke azonosító párok)
H_3: dropout/BatchNorm train módban a kiértékeléskor	ismételt val-kiértékelés ingadozása	a kiértékelés pillanatában aktív `training` flagek listája; a véletlen- és statisztikarétegek listája

A tünetoszlop **nem kizáró**: a jósolt tartományok nem diszjunktak és nem szükségszerűek. Szivárgás mellett is lehet kicsi a val–test rés (ha a nézetek kevéssé hasonlók), címkehibánál is lehet nagy, és több hiba egyszerre is fennállhat. Ezért a lemmát a harmadik oszlop determinisztikus ellenőrzéseire alkalmazzuk, és mindhármat lefuttatjuk, nem állunk meg az első pozitívnál. Az alábbi kontroll két különböző hibát mutat, amelyek ugyanazt a feltett mintázatot (magas val acc, alacsony clean test acc) adhatják, és a közvetlen ellenőrzések szétválasztják őket; a második rész azt mutatja, hogy a dropout kimenet-különbsége train módban is lehet nulla. A nyitó kérdésre a válasz tehát: nem a pontosság vagy a veszteség dönt, hanem a három közvetlen, determinisztikus ellenőrzés együttese, mert a teljesítménytünetek nem különítik el a hipotéziseket.

```
import numpy as np
import torch

# Két különböző hiba, azonos feltett teljesítménymintázattal (val acc magas, clean test acc
↪  alacsony).
scene = np.repeat(np.arange(6), 2)                      # 12 kép: 6 jelenet × 2 nézet
split_a = np.array(["train", "val", "train", "train", "val", "test",
                    "test", "test", "train", "train", "val", "val"])   # A: képenkénti split
split_b = np.repeat(["train", "train", "train", "val", "test", "test"], 2)  # B: jelenetszintű
↪  split
eval_idx_a = np.flatnonzero(split_a == "test")          # A: a célok indexe helyes
eval_idx_b = np.arange((split_b == "test").sum())       # B: labels[:n_test], a tömb eleje

def leaking_scenes(split):
```

```
    return [int(s) for s in np.unique(scene) if len(set(split[scene == s])) > 1]

def targets_match_images(split, eval_idx):
    return bool(np.array_equal(eval_idx, np.flatnonzero(split == "test")))

for name, split, idx in (("A", split_a, eval_idx_a), ("B", split_b, eval_idx_b)):
    print(f"{name}: splitet átlépő jelenetek = {leaking_scenes(split)}, "
          f"célindex = képindex: {targets_match_images(split, idx)}")

# A dropout kimenet-különbsége tünet, nem bizonyíték: train módban is lehet 0.
drop = torch.nn.Dropout(p=0.5).train()
zeros, ones = torch.zeros(4, 8), torch.ones(4, 8)
print("nulla aktiváció, két hívás:", (drop(zeros) - drop(zeros)).abs().max().item())
torch.manual_seed(1); first = drop(ones)
torch.manual_seed(1); second = drop(ones)
print("azonos seed, két hívás:", (first - second).abs().max().item())
print("aktív training flag:", [type(m).__name__ for m in drop.modules() if m.training])
```

Kimenet a rögzített környezetben (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál):

```
A: splitet átlépő jelenetek = [0, 2], célindex = képindex: True
B: splitet átlépő jelenetek = [], célindex = képindex: False
nulla aktiváció, két hívás: 0.0
azonos seed, két hívás: 0.0
aktív training flag: ['Dropout']
```

Az A esetben szivárgás van, a célindex helyes; a B esetben a split hibátlan, a célok viszont nem a kiértékelt képekhez tartoznak. A teljesítménymintázat a kettőt nem különbözteti meg, a két szerkezeti ellenőrzés igen. A dropout esetében a nulla kimenet-különbség nem zárja ki a train módot; a `training` flag közvetlenül mutatja.

40.2.4 Labor: öt eredeti hibás kódrészlet diagnózisa

A labor öt GlyphLab-kontextusú, szándékosan hibás kódrészletet ad. Mindegyiknél közöljük a hiba okát, következményét, az elkülönítő mérést és a javítást. A részletek munkakönyvtára a `lab/` könyvtár; a `data/...` útvonalak innen értendők. Az 1., 4. és 5. pont ellenőrzése önállóan futtatható, és az ott közölt kimenet mért; a 2. és 3. pont kódrészlete koncepcionális diagnózis, amelyet a szöveg magyaráz, nem önálló program.

40.2.4.1 Leakage: teljes adatra számolt normalizálás és képenkénti split

```
# HIBÁS
images = np.load("data/images.npy").astype(np.float32)
labels = np.load("data/labels.npy")
mean = images.mean()          # a teszt statisztikája is benne van
std  = images.std()
X = (images - mean) / std
perm = np.random.default_rng(1).permutation(len(X))
idx_tr, idx_va, idx_te = perm[:240], perm[240:300], perm[300:]
```

Ok. A normalizálás a teljes adatra számol átlagot és szórást; a split képenként véletlen, így egy `scene_id` két nézete különböző splitbe kerülhet.

Következmény. A modell a teszteloszlás skáláját ismeri, és a validáció egy része a train-nézetek párja. Lehetséges tünet a felfelé torzított val acc; a torzítás mértéke a nézetek hasonlóságától függ, ezért a tünet önmagában nem bizonyít.

Elkülönítő mérés. Nem a metadata `split` oszlopát kell vizsgálni, hanem a ténylegesen használt indexhalmazokat és a belőlük épített loadereket: hány `scene_id` fordul elő egynél több loaderben. Ha ez a szám pozitív, a leakage igazolt. A javítás ezért a metadata `index` mezőjéből újraképzi mindhárom indexhalmazt, ezekből újraépíti a loadereket, a régi `perm`-alapú indexeket és loadereket nem használja tovább, a normalizáló statisztikát az új train indexekből számolja, és az ellenőrzést a loaderek által ténylegesen visszaadott képindexeken futtatja; a kontroll a hibás split loadereit ugyanezzel a függvénnyel elkapja.

```
# JAVÍTOTT
import numpy as np
import pandas as pd
import torch
from torch.utils.data import DataLoader, TensorDataset

images = np.load("data/images.npy").astype(np.float32)
labels = np.load("data/labels.npy")
meta = pd.read_csv("data/metadata.csv").sort_values("index").reset_index(drop=True)
assert meta["index"].tolist() == list(range(len(images)))  # az index köti a sort a képhez
idx_tr, idx_va, idx_te = (meta.loc[meta["split"] == s, "index"].to_numpy()
                          for s in ("train", "val", "test"))   # a perm-alapú indexek helyett
mean = images[idx_tr].mean()          # csak a train képekből
std = images[idx_tr].std()
X = (images - mean) / std

def make_loader(idx, shuffle):        # a loader a képindexet is visszaadja
    ds = TensorDataset(torch.from_numpy(X[idx]), torch.from_numpy(labels[idx]),
                       torch.tensor(idx))
    return DataLoader(ds, batch_size=32, shuffle=shuffle)

def loader_indices(loader):
    return np.concatenate([batch[2].numpy() for batch in loader])

def scene_overlap(*loaders):          # a ténylegesen betöltött képek jelenetei
    scenes = [set(meta["scene_id"].to_numpy()[loader_indices(dl)]) for dl in loaders]
    return {(a, b): len(scenes[a] & scenes[b])
            for a in range(len(scenes)) for b in range(a + 1, len(scenes))}

train_loader, val_loader, test_loader = (make_loader(idx_tr, True), make_loader(idx_va, False),
                                         make_loader(idx_te, False))
assert not any(scene_overlap(train_loader, val_loader, test_loader).values())

# kontroll: a hibás, perm-alapú split loadereit ugyanez az ellenőrzés elkapja
perm = np.random.default_rng(1).permutation(len(X))   # a hibás blokk indexei
bad = [make_loader(i, False) for i in (perm[:240], perm[240:300], perm[300:])]
print("perm-split metszetek:", scene_overlap(*bad))
```

Futási jegyzőkönyv (Python 3.12.3, NumPy 2.5.4, pandas 3.1.0, PyTorch 2.15.0+cpu, CPU, 1 szál): `perm-split metszetek: {(0, 1): 42, (0, 2): 36, (1, 2): 12}`, azaz a perm-alapú split train–val, train–test és val–test párjai rendre 42, 36 és 12 jelenetben fednek át; a javított loaderekre mindhárom metszet üres.

40.2.4.2 Label bug: a corrupt tesztet train-címkékkel mérjük

```
# HIBÁS
corr = torch.from_numpy(np.load("data/corrupted_test.npy"))
model.eval()
with torch.no_grad():
    logits = model(corr)
pred = logits.argmax(1)
```

```
y_eval = torch.from_numpy(labels[:60])   # a teljes tömb első 60 eleme, nem a teszt
acc = (pred == y_eval).float().mean().item()
```

Ok. A `labels[:60]` a teljes tömb első hatvan eleme, azaz a 0–59. képindex címkéje. A metadata `index` mezője rendeli a címkét a képhez, a split-indexek nincsenek rendezve, ezért ez a szelet nem feltétlenül a train split; a szállított adatban például a 0–59. kép 36 train, 16 val és 8 test képből áll. A corrupt teszt sorrendje a metadata teszt-sorrendjével egyezik, tehát a predikcióhoz nem tartozó célokat hasonlítunk.

Következmény. A mért pontosság érvénytelen, mert a predikciókat nem a hozzájuk tartozó célokkal hasonlítja; meghatározott chance-szintje nincs. A 1/3 várható érték csak akkor adódik, ha a tévesen párosított címkék a predikcióktól függetlenek és a három osztályon egyenletesek. Kiegyensúlyozott osztályok mellett is lehet a mért érték 1 vagy 0: ha a predikciók `[0, 1, 2]` és a tévesen használt címkeszelet `[0, 1, 2]`, a pontosság $\frac{3}{3} = 1$; ha a téves szelet `[1, 2, 0]` (ugyanannak a címkesorrendnek a ciklikus permutációja), akkor $\frac{0}{3} = 0$. A rossz szelet tehát véletlenül is adhat tökéletesnek látszó értéket, ezért a pontosság önmagában nem azonosítja a címkehibát.

Elkülönítő mérés. Közvetlen ellenőrzés: a kiértékelt célok indexe megegyezik-e a predikált képek indexével. A `labels[:60]` a 0–59. képindex címkéje, a corrupt teszt képei viszont a metadata `split == 'test'` sorainak felelnek meg; ha a két indexlista eltér, a címkehiba igazolt, a pontosságtól függetlenül. A helyes címkékkel újraszámolt pontosság csak kiegészítő tünet.

```
# JAVÍTOTT
test_idx = meta.loc[meta["split"] == "test", "index"].to_numpy()  # a metadata index-oszlopa
y_eval = torch.from_numpy(labels[test_idx])
acc = (pred == y_eval).float().mean().item()
```

40.2.4.3 Train/eval: dropout aktív a validációban

```
# HIBÁS
def evaluate(model, loader):          # ennek a változatnak a törzse nem kapcsol eval módba
    correct = n = 0
    with torch.no_grad():
        for xb, yb in loader:
            correct += (model(xb).argmax(1) == yb).sum().item()
            n += len(yb)
    return correct / n

for epoch in range(epochs):
    model.train()
    for xb, yb in train_loader:
        ...
    val_acc = evaluate(model, val_loader)   # sehol sem hívódik model.eval()
```

A labor 3. kódrészletében a hiba nem a külső hívássorban van: a hívás helyes, de az `evaluate` törzse nem hív `model.eval()`-t. Ha az `evaluate` belsejében szerepelne a `model.eval()` (a 12.1 szerinti javított változat), ugyanez a külső hívás helyes eredményt adna; itt viszont a dropout aktív marad.

Ok. A `model.eval()` hívás elmarad a validáció előtt, így a dropout véletlen maszkja továbbra is aktív, a BatchNorm pedig a batch-statisztikát használja.

Következmény. A validációs veszteség batch-összetételtől függően ingadozik, a legjobb checkpoint kiválasztása zajossá válik. A teszt, ha ott már helyesen `eval` módban fut, jobbnak tűnhet a validációnál, ami inverz anomália.

Elkülönítő mérés. Közvetlen ellenőrzés: az `evaluate` belsejében, a forward előtt, a `[n for n, m in model.named_modules() if m.training]` lista üres-e. Ha nem üres, a módhiba igazolt. Ugyanazon a val batch-en két egymás utáni kiértékelés logit-különbsége csak tünet: pozitív érték train módot jelez, a nulla viszont nem zárja ki (nulla aktiváció a dropout előtt, azonos seed és így azonos maszk, vagy olyan modell, amelyben nincs véletlenréteg).

```
# JAVÍTOTT
model.eval()
for xb, yb in val_loader:
    with torch.no_grad():
        logits = model(xb)
    ...
# a következő epoch előtt vissza kell kapcsolni
model.train()
```

40.2.4.4 Wrong reduction: a batchösszeg és a képenkénti nevező keveredik

A 39.2. Encoder–decoder vagy lokalizációs fej szegmentáló fejének célfüggvénye az összes pixelre átlagolt bináris log loss (`BCEWithLogitsLoss()`, `mean` redukció). Ez legitim, széles körben használt cél; az előtér ritkasága önmagában nem implementációs hiba, hanem célválasztási és metrikaértelmezési kérdés, amelyet a validációs Dice/IoU mellett kell megítélni. Ha az előteret szándékosan nagyobb súllyal akarjuk tanítani, ezt **explicit súlyozott célként** kell definiálni. Képenként normalizálva, majd a képekre átlagolva:

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \frac{\sum_p w_{n,p}\,\ell_{n,p}}{\sum_p w_{n,p}}, \qquad w_{n,p} = \begin{cases} w_{\mathrm{fg}}, & Y_{n,p} = 1, \\ 1, & Y_{n,p} = 0, \end{cases}$$

ahol $\ell_{n,p}$ a pixelenkénti BCE-tag, $w_{\mathrm{fg}} > 0$. A $w_{\mathrm{fg}} = 1$ eset a szokásos pixelátlag (azonos képméret mellett). A nevező képenként legalább $H \cdot W \cdot \min(1, w_{\mathrm{fg}}) > 0$, tehát üres célmaszkra is értelmezett, és ilyenkor a háttértagok büntetnek: egy téves előtérpredikció üres GT- (ground truth, referencia-) maszk mellett is nagy veszteséget kap. A két helyes redukciót külön kell tartani: (a) teljes pixelátlag esetén `F.binary_cross_entropy_with_logits(logits, masks, reduction="mean")`, azaz a teljes batch összes pixelére vett BCE-összeg osztva a pixelek számával; (b) képenként normalizált változat esetén a fenti képlet szerint előbb képenként osztunk a képenkénti súlyösszeggel, majd a képekre átlagolunk. A számláló és a nevező mindig ugyanarra az egységre (pixel vagy kép) vonatkozik; a kettő keverése a következő hibás változat.

A hibás változat egy „javításnak” szánt redukció:

```
# HIBÁS
logits_flat = logits.view(N, -1)        # (N, H*W)
target_flat = masks.view(N, -1)         # (N, H*W)
loss = F.binary_cross_entropy_with_logits(
    logits_flat, target_flat, reduction="sum")
loss = loss / target_flat.sum(dim=1).clamp(min=1).mean()
```

Ok. A számláló a **teljes batch** összege, a nevező az **egy képre jutó** átlagos előtérszám. A batch megduplázása így megduplázza a veszteséget és a gradienst: a hatásos tanulási ráta a batchmérettől függ. A közös nevező ráadásul az előtér és a háttér relatív súlyát sem változtatja meg, mert mindkettő ugyanazt a szorzót kapja. A `clamp(min=1)` csak a nullával osztást akadályozza meg; a számlálót nem nullázza, üres maszkra a háttértagok továbbra is benne vannak (négy $z = 0$ logit és üres cél mellett a számláló $4 \log 2 \approx 2{,}7726$).

Következmény. A batchméret vagy a gradiens-akkumuláció változtatása más optimalizálási lépést ad ugyanarra az adatra; a kísérletek nem összehasonlíthatók, és az előtér súlyozása, amelyet a kód sugall, valójában nem történik meg.

Elkülönítő mérés és javítás. A következő float64-es ellenőrzés a hibás redukciót és a fenti képenként normalizált célt veti össze: batch-duplikálás, a $w_{\mathrm{fg}} = 1$ eset egyezése a szokásos pixelátlaggal, az előtér/háttér gradiensarány és az üres maszk.

```
import math
import torch
import torch.nn.functional as F

def wrong_reduction(logits, masks):
    # a fenti HIBÁS redukció függvényként: a számláló a teljes batch összege, a nevező egy képre
    ↪  jutó előtérszám-átlag
    n = logits.shape[0]
    loss = F.binary_cross_entropy_with_logits(
        logits.reshape(n, -1), masks.reshape(n, -1), reduction="sum")
    return loss / masks.reshape(n, -1).sum(dim=1).clamp(min=1).mean()

def weighted_bce_per_image(logits, masks, w_fg=1.0):
    """Képenként normalizált, explicit előtérsúlyos BCE; w_fg=1 a szokásos pixelátlag."""
    per_pixel = F.binary_cross_entropy_with_logits(logits, masks, reduction="none")
    w = torch.where(masks > 0.5, torch.full_like(masks, w_fg), torch.ones_like(masks))
    per_image = (w * per_pixel).flatten(1).sum(1) / w.flatten(1).sum(1)
    return per_image.mean()

torch.manual_seed(20261020)
dt = torch.float64
masks = torch.zeros(3, 1, 8, 8, dtype=dt)
masks[0, 0, 2:6, 3] = 1                       # vékony előtér
masks[1, 0, 4, 1:7] = 1                       # üres a 2-es kép marad
logits = torch.randn(3, 1, 8, 8, dtype=dt)

def grads(fn, lg, mk, **kw):
    lg = lg.clone().requires_grad_(True)
    loss = fn(lg, mk, **kw)
    loss.backward()
    return loss.item(), lg.grad

# 1) Batch-duplikálás: az átlagolt loss és a logitgradiens maradjon változatlan.
#    A két másolat logitjai ugyanattól a paramétertől függnek, ezért a közös
#    paraméterre a láncszabály szerint a két másolat logit-gradiensének összege hat.
for name, fn in (("hibás", wrong_reduction), ("javított", weighted_bce_per_image)):
    l1, g1 = grads(fn, logits, masks)
    l2, g2 = grads(fn, torch.cat([logits, logits]), torch.cat([masks, masks]))
    scale = ((g2[:3] + g2[3:]) / g1).mean().item()
    print(f"{name:9s} loss {l1:.6f} -> duplikálva {l2:.6f}; logitgradiens-szorzó {scale:.3f}")

# 2) w_fg=1 a szokásos pixelátlag
assert math.isclose(weighted_bce_per_image(logits, masks).item(),
                    F.binary_cross_entropy_with_logits(logits, masks).item(), rel_tol=1e-12)

# 3) Előtér/háttér gradiensarány: a hibás nevező nem változtatja, a w_fg igen.
def fg_bg_ratio(fn, **kw):
    _, g = grads(fn, logits, masks, **kw)
    fg = masks > 0.5
    return (g[fg].abs().mean() / g[~fg].abs().mean()).item()
print("előtér/háttér |grad|-arány: pixelátlag %.3f, hibás %.3f, w_fg=10 %.3f" % (
    fg_bg_ratio(weighted_bce_per_image), fg_bg_ratio(wrong_reduction),
    fg_bg_ratio(weighted_bce_per_image, w_fg=10.0)))

# 4) Üres célmaszk: a háttértagok is büntetnek.
```

```
empty = torch.zeros(1, 1, 2, 2, dtype=dt)
z0 = torch.zeros_like(empty)
print("üres maszk, z=0: %.6f (log 2 = %.6f), z=+5: %.6f" % (
    weighted_bce_per_image(z0, empty).item(), math.log(2),
    weighted_bce_per_image(z0 + 5.0, empty).item()))
print("lektori kontroll, hibás redukció: négy z=0 üres -> %.9f, duplikálva %.9f" % (
    wrong_reduction(z0, empty).item(), wrong_reduction(torch.cat([z0, z0]), torch.cat([empty,
 ↪  empty])).item()))
```

Futási jegyzőkönyv (Python 3.12.3, PyTorch 2.15.0+cpu, float64, CPU, 1 szál, seed 20261020):

```
hibás     loss 40.106559 -> duplikálva 80.213117; logitgradiens-szorzó 2.000
javított  loss 0.765924 -> duplikálva 0.765924; logitgradiens-szorzó 1.000
előtér/háttér |grad|-arány: pixelátlag 0.690, hibás 0.690, w_fg=10 5.426
üres maszk, z=0: 0.693147 (log 2 = 0.693147), z=+5: 5.006715
lektori kontroll, hibás redukció: négy z=0 üres -> 2.772588722, duplikálva 5.545177444
```

A hibás redukció a duplikált batchen kétszeres veszteséget és kétszeres logitgradienst ad; a közös paraméterre a láncszabály szerint a paramétergradiens is ugyanígy skálázódik, a képenként normalizált cél viszont mindkettőt változatlanul hagyja. Az előtér/háttér gradiensarány a hibás változatban pontosan a pixelátlagé (0,690), tehát a „javítás” nem súlyoz; a valódi előtérsúlyozás ($w_{\mathrm{fg}} = 10$) az arányt 5,426-ra emeli. Ez nem pontosan tízszeres növekedés, mert a képenkénti nevező is változik: a nagyobb előtérszámú képeknél a $w_{\mathrm{fg}} F_n + B_n$ nevező jobban megnő, ezért a gradienst erősebben tompítja, és az üres maszkú kép háttere, amelynek nevezője változatlan, nagyobb súlyt kap az aggregált arányban. Üres célmaszkra $z = 0$ mellett a veszteség $\log 2$, a téves előtérpredikció ($z = +5$) pedig nagyobb büntetést kap (5,007).

40.2.4.5 Checkpoint: az utolsó epoch állapotát értékeljük

```
# HIBÁS
for epoch in range(epochs):
    train_one_epoch(...)
    val_loss = evaluate(...)
    torch.save({"model": model.state_dict(), "epoch": epoch}, "last.pt")
state = torch.load("last.pt")
model.load_state_dict(state["model"])
test_acc = evaluate(model, test_loader)   # az utolsó, túlillesztett modell
```

Ok. A kiértékelés nem a legjobb validációs veszteségű epoch modelljét tölti be, hanem az utolsót; a túlillesztés után az utolsó epoch gyakran rosszabb.

Következmény. A riportált eredmény nem a protokoll szerint kiválasztott modellhez tartozik. Az utolsó epoch a teszten lehet rosszabb, azonos, sőt jobb is a kiválasztottnál; a hiba a választási protokoll megsértése, nem a pontosság iránya.

Elkülönítő mérés. A három split szerepe különüljön el: a trainen tanítunk, a validáció a modellválasztásé (ezért a validációs adat nem része a tanításnak), a teszt pedig kizárólag a kiválasztás utáni egyszeri kiértékelésé. A diagnózis nem használja a tesztet: a teszt a kiválasztás után egyszer fut, és nem vehet részt a modellválasztás ellenőrzésében. A betöltött checkpoint útvonalát, SHA-256 hash-ét, `epoch` és mentett `val_loss` mezőjét a tanítási napló előre rögzített argminjával vetjük össze, a betöltés helyességét pedig a validációs kontrollon: a betöltött modellel újraszámolt val loss egyezik-e a checkpointban mentett értékkel. A rossz epoch így akkor is kiderül, ha a két modell pontossága azonos.

```python
import hashlib
import torch

def sha256(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

def check_checkpoint(path, log, model, val_loss_fn, tol=1e-6):
    """Teszt nélküli diagnózis: a betöltött állapot a napló előre rögzített argminja-e."""
    best = min(log, key=lambda r: r["val_loss"])          # szabály: legkisebb val_loss epoch
    state = torch.load(path, map_location="cpu")
    model.load_state_dict(state["model"])
    model.eval()
    report = {"path": str(path), "sha256": sha256(path), "epoch": state["epoch"],
              "argmin_epoch": best["epoch"], "saved_val_loss": state.get("val_loss"),
              "recomputed_val_loss": val_loss_fn(model)}
    report["selection_ok"] = state["epoch"] == best["epoch"]
    report["load_ok"] = (state.get("val_loss") is not None
                         and abs(report["recomputed_val_loss"] - state["val_loss"]) <= tol)
    return report

# kontroll: azonos súlyok (tehát azonos accuracy), de rossz epoch -> a diagnózis jelez
torch.manual_seed(0)
model = torch.nn.Linear(4, 3)
xv, yv = torch.randn(16, 4), torch.randint(0, 3, (16,))
val_loss_fn = lambda m: torch.nn.functional.cross_entropy(m(xv), yv).item()
v2 = val_loss_fn(model)                                   # az utolsó (2.) epoch valódi
↪  val_loss-a
log = [{"epoch": 0, "val_loss": v2 + 0.2}, {"epoch": 1, "val_loss": v2 - 0.1}, {"epoch": 2,
↪  "val_loss": v2}]
torch.save({"model": model.state_dict(), "epoch": 2, "val_loss": v2}, "last.pt")
torch.save({"model": model.state_dict(), "epoch": 1, "val_loss": v2 - 0.1}, "best.pt")  # hibás
↪  mentés: 1. epoch címke, 2. epoch súlyai
for path in ("last.pt", "best.pt"):
    r = check_checkpoint(path, log, model, val_loss_fn)
    print(path, {k: r[k] for k in ("epoch", "argmin_epoch", "selection_ok", "load_ok")})
```

Kimenet a rögzített környezetben (Python 3.12.3, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 0):

```
last.pt {'epoch': 2, 'argmin_epoch': 1, 'selection_ok': False, 'load_ok': True}
best.pt {'epoch': 1, 'argmin_epoch': 1, 'selection_ok': True, 'load_ok': False}
```

A `last.pt` hibátlanul betöltődik, de nem az argmin epoch; a hibásan mentett `best.pt` epoch-mezője helyes, a súlyai viszont nem az 1. epochéi, ezért az újraszámolt val loss eltér a mentettől. A fenti kimenet a kontrollfutás mért eredménye; a diagnózis nem igényel teszt spliten végzett kiértékelést.

```python
# JAVÍTOTT
# A validációs split a tanítástól elkülönített; a teszt csak a kiválasztás után, egyszer fut.
best_val = float("inf")
for epoch in range(epochs):
    train_one_epoch(...)
    val_loss = evaluate(...)
    if val_loss < best_val:
        best_val = val_loss
        torch.save({"model": model.state_dict(), "epoch": epoch,
                    "val_loss": val_loss}, "best.pt")
state = torch.load("best.pt")
model.load_state_dict(state["model"])
model.eval()
```

40.2.5 Kézi számpélda a hipotéziselkülönítésre

Legyen a teszt hat képe, $y = (0;\ 1;\ 2;\ 0;\ 1;\ 2)$. A H_2 címkeeltolódási hipotézis hibás céljai $y' = (1;\ 2;\ 0;\ 1;\ 2;\ 0)$, azaz ciklikus eltolás. A példa kedvéért tegyük fel, hogy a megfigyelt tesztpredikció $\hat{y} = y$; ez a megfigyelt adat, nem a H_2 következménye, és abból, hogy egy modell a trainen tökéletes, a teszten való tökéletesség nem következik.

```
import numpy as np
y     = np.array([0, 1, 2, 0, 1, 2])
y_hat = y.copy()
y_bug = (y + 1) % 3
acc_true = (y_hat == y).mean()      # 1.000
acc_bug  = (y_hat == y_bug).mean()  # 0.000
```

A lemma nyelvén $O = (\text{acc_true}, \text{acc_bug})$. A megfigyelt minta $\hat{y} = y$, ezért H_2 mellett $O = (1;\ 0)$ determinisztikusan adódik, azaz $A_2 = \{(1;\ 0)\}$. A H_0 nullhipotézis nem a megfigyelt $\hat{y}$-ról szól, hanem a predikció eloszlásáról: eszerint a modell képenként, a címkétől függetlenül, egyenletesen tippel. Ekkor $\text{acc_true} = 1$ valószínűsége 3^{-6}, és ezen az eseményen $\text{acc_bug} = 0$ determinisztikus, mert $y'_i \neq y_i$; ezért $P(O = (1;\ 0) \mid H_0) = 3^{-6} = 1/729 \approx 0{,}00137 > 0$. A H_0 tartója tehát tartalmazza a megfigyelt $(1;\ 0)$ pontot, ezért a lemma diszjunktsági feltétele nem teljesül: a $(1;\ 0)$ kimenet **nem logikai kizárás**. A mérés a hat képből álló minta acc_true értéke; a példában ez 1, de a szabály bármely megfigyelt értékre alkalmazható. Döntési szabály: $\alpha = 0{,}01$ szinten H_0-t elutasítjuk, ha mind a hat predikció helyes; a téves elutasítás valószínűsége $3^{-6} \approx 0{,}00137$. Ez döntés, nem logikai kizárás. A 60 képes teszt standard hibája ide nem vihető át, mert a mintaméret itt 6. Ha a mért érték $(1/3, 1/3)$ körüli, az a H_2-t a megfigyelt $\hat{y} = y$ mellett kizárja, mert H_2 ott determinisztikusan $(1;\ 0)$-t ad.

40.2.6 Feladatok

F1 (fogalmi, nehézség 1). Definiálja a leakage fogalmát a GlyphLab szerződésben. Adjon egy olyan példát, ahol a képenkénti véletlen split nem okoz mérhető validációs torzítást, és egy olyat, ahol igen. *Beadandó:* legfeljebb fél oldalas magyarázat, a `scene_id` szerepének megjelölésével.

F2 (levezetési, nehézség 2). Rögzített tesztpredikció $\hat{y} = y$ mellett H_2 szerint a kiértékelés célja ciklikusan eltolódott ($y' = y + 1 \bmod 3$), H_0 szerint a modell a címkétől függetlenül, egyenletesen tippel. A fenti hat elemű mintán (a) mutassa meg, hogy H_2 az $O = (\text{acc_true}, \text{acc_bug})$ statisztikára determinisztikus predikciót ad, H_0 tartója viszont tartalmazza ezt az értéket, tehát a lemma diszjunktsági feltétele nem teljesül; (b) adjon $\alpha = 0{,}01$ szintű döntési szabályt H_0 ellen, és számítsa ki a téves elutasítás valószínűségét; (c) nevezze meg, mi a mintavételi egység a 60 képes GlyphLab-teszten, és miért. *Beadandó:* a predikciók, a döntési szabály, a hibavalószínűség és a mintavételi egység indoklása.

F3 (implementációs, nehézség 2). Írjon olyan Python-függvényt, amelynek neve `scene_split_check(metadata_csv)`, és amely a GlyphLab `metadata.csv`-ből (fájlútvonal vagy betöltött `DataFrame`) ellenőrzi, hogy minden `scene_id` összes nézete azonos splitben van. A függvény először ellenőrizze a kötelező oszlopok (`index`, `scene_id`, `view_id`, `split`) meglétét, majd adja vissza a sértő `scene_id`-k listáját. *Beadandó:* futtatható függvény, egy fejléces, de nulla soros bemenetre és egy mesterségesen megsértett bemenetre adott kimenettel; a 0 bájtos fájlra adjon egyértelmű hibát.

F4 (hibakeresési, nehézség 2). A labor 2. kódrészlete a corrupt tesztet `labels[:60]`-nal értékeli. Lokalizálja a hibát, írja le a következményét a várható pontosságra, és mutassa meg a javítást három elemű ellenőrző példán, ahol a helyes címkék `[0,2,1]`, a hibásak `[1,0,0]`, a predikciók `[0,2,1]`. *Beadandó:* ok, következmény, javított kód, a három elemű kontroll értéke.

F5 (kísérleti/transzfer, nehézség 3). Tervezzen kísérletet annak eldöntésére, hogy a 98,3% (59/60 kép)-os validációs pontosság split-szivárgásból vagy a val jelenetek könnyű összetételéből adódik. Adja meg a protokollt, a megfigyelési egységet, a metrikát és a következtetési korlátot. *Beadandó:* egy bekezdésnyi protokoll, amely nem használja a teszt-adatot a döntéshez.

40.2.7 Megoldások és indoklás

F1 megoldás. (a) *Nem okoz torzítást:* ha minden kép külön jelenethez tartozna (egy kép = egy jelenet), a képenkénti véletlen split megegyezne a jelenetszintű felosztással, ezért a validációs kép független lenne a train képektől. (b) *Okoz torzítást:* a GlyphLabban egy jelenethez két nézet tartozik, és a generátor egyetlen maszkot sorsol, amelyre mindkét nézetet külön Gauss-zajjal rajzolja (A. A GlyphLab adatkészlet, `lab/generate_data.py`); a képenkénti split így ugyanannak a jelenetnek a két nézetét teheti külön splitbe, a két nézet geometriai tartalma azonos, ezért a validációs kép nem független a train képtől, és a pontosság felfelé torzított. A `scene_id` a mintavételi egység, ezért a `groupby("scene_id")["split"].nunique()` értéke: ha bármely jelenetre 2, a hiba fennáll. (c) *További leakage-forma:* a normalizáló statisztika a teljes adatra számolva a teszteloszlás átlagát és szórását viszi a modellbe; ez akkor is torzít, ha a split egyébként helyes.

F2 megoldás. (a) $\hat{y} = y$ és $y_i' \neq y_i$ miatt H_2 mellett acc_true $= 1$, acc_bug $= 0$, azaz $A_2 = \{(1;\ 0)\}$. H_0 mellett a hat találat független Bernoulli(1/3), így O bármely $(k/6,\ j/6)$ értéket felveheti $k + j \leq 6$ mellett, a $(1;\ 0)$-t is 3^{-6} valószínűséggel. A tartók nem diszjunktak, ezért egyetlen mérés nem logikai kizárás. (b) Döntési szabály: H_0 elutasítva, ha acc_true $= 1$. A téves elutasítás valószínűsége $P(\text{acc_true} = 1 \mid H_0) = 3^{-6} \approx 0{,}00137 \leq 0{,}01$; a H_2 mellett a szabály mindig elutasít. A 3^{-6} csak a független, egyenletes tippelés nullhipotézise mellett érvényes. (c) A 60 képes teszten a mintavételi egység a jelenet (30 jelenet, két nézet), mert a képtől függő kimenet mellett a két nézet találata korrelált lehet. Konzervatív eljárás jelenetenként egy nézet: $X \sim \mathrm{Bin}(30, 1/3)$, és $\alpha = 0{,}01$ mellett az elutasítási tartomány $X \geq 17$ ($P = 0{,}0072$). A 60 képre számolt $\sqrt{(1/3)(2/3)/60}$ standard hiba csak független képek esetén lenne érvényes.

F3 megoldás.

```python
# munkakönyvtár: lab/
import pandas as pd

REQUIRED = ["index", "scene_id", "view_id", "split"]

def scene_split_check(metadata_csv):
    if isinstance(metadata_csv, pd.DataFrame):
        meta = metadata_csv.copy()          # a hívó DataFrame-jét nem módosítjuk
    else:
        try:
            meta = pd.read_csv(metadata_csv)
        except pd.errors.EmptyDataError as exc:
            raise ValueError("üres metadata.csv: nincs fejléc") from exc
    missing = [c for c in REQUIRED if c not in meta.columns]
    if missing:
        raise ValueError(f"hiányzó oszlopok: {missing}")
    bad = meta.groupby("scene_id")["split"].nunique() > 1
    return bad[bad].index.tolist()

empty = pd.DataFrame(columns=REQUIRED)      # fejléc, nulla adatsor
tidy = pd.DataFrame({"index": [0, 1, 2, 3], "scene_id": ["s0", "s0", "s1", "s1"],
                     "view_id": [0, 1, 0, 1], "split": ["train", "train", "val", "val"]})
broken = tidy.copy()
broken.loc[0, "split"] = "val"              # csak a másolatot sértjük meg
```

```
print(scene_split_check(empty))              # []
print(scene_split_check(tidy))               # []
print(scene_split_check(broken))             # ['s0']
print(scene_split_check("data/metadata.csv"))  # [] a szállított adaton

import io
try:
    scene_split_check(io.StringIO(""))       # 0 bájtos tartalom
except ValueError as exc:
    print(exc)                               # üres metadata.csv: nincs fejléc
```

A `metadata_csv` paraméternév a szokásos hívásra utal, de a függvény betöltött `DataFrame`-et is elfogad, és arról másolatot készít, ezért a mesterséges sértés nem módosítja a hívó adatát. Az `index`, `scene_id`, `view_id` és `split` oszlop meglétét a függvény ellenőrzi; hiányzó oszlopra `ValueError`-t ad. A fejléces, de nulla soros bemenet kimenete `[]`; a 0 bájtos fájl `pandas.errors.EmptyDataError` kivételét a függvény `ValueError`-ra fordítja, mert a hiányzó fejléc nem értelmezhető érvényes, de sértés nélküli adatként. Egyetlen jelenet egyetlen nézettel sosem sértő, mert `nunique()` értéke 1. A négy lista és a 0 bájtos eset hibaüzenete a projekt venvjében (pandas 3.1.0) futtatva: `[]`, `[]`, `['s0']`, `[]`, illetve `üres metadata.csv: nincs fejléc`; a szállított `lab/data/metadata.csv` 360 sora egyetlen `scene_id`-ban sem tartalmaz két különböző splitet.

F4 megoldás. A hiba a `labels[:60]` szelet: ez a teljes tömb első hatvan eleme, azaz a 0–59. képindex címkéje, nem a teszt; a metadata `index` mezője rendeli a címkét a képhez, a split-indexek nincsenek rendezve, ezért ez a szelet nem feltétlenül a train split sem. A corrupt teszt sorrendje a metadata teszt-sorrendje. Következmény: a mért pontosság a modell predikcióit a képekhez nem tartozó címkékkel hasonlítja, ezért érvénytelen, és nincs rögzített chance-szintje; 1/3 csak a predikcióktól független, egyenletes hibás címkék feltevésével várható. A javítás a teszt-indexek használata. A három elemű kontroll: predikciók `[0,2,1]`, helyes címkék `[0,2,1]`, hibás címkék `[1,0,0]`. A helyes pontosság $\frac{3}{3} = 1$, a hibás $\frac{0}{3} = 0$: ugyanaz a predikció a helyes címkékkel mindhárom képen talál, a hibás címkékkel egyiken sem, tehát a különbség a címkehozzárendelésből fakad, nem a modellből.

F5 megoldás. A szivárgást nem statisztikai kísérlettel, hanem a tényleges split szerkezeti ellenőrzésével döntjük el: a betöltő által használt train/val/test `scene_id`-halmazok metszete üres-e, és a normalizáló statisztika csak a train indexeire számolódott-e; a train/eval módhibát ugyanígy a `training` flagek listája dönti el. Ha a `scene_id`-metszet nem üres, a szivárgás igazolt, és a további mérés legfeljebb a hatás nagyságát becsüli. Ha a metszet üres, a 98,3%-os val pontosság másik magyarázata a val jelenetek kivételesen könnyű volta; ezt a train jeleneteken, a teszt érintése nélkül teszteljük. Protokoll: öt foldos, scene-szintű keresztvalidáció a 120 train jeleneten; minden foldban a modellt a 96 tanító jeleneten újratanítjuk (azonos hiperparaméterek, azonos seed), és a kihagyott 24 jelenet egy-egy nézetén értékelünk. Megfigyelési egység: `scene_id`; metrika: a jelenetenkénti találatok átlaga az öt foldon összevontan (120 jelenet), jelenetszintű bootstrap 95%-os CI-vel. Értelmezés: ha a CI a 98,3%-ot tartalmazza, a val jelenetek nem bizonyultak kivételesen könnyűnek a train jelenetekhez képest; ha a CI felső határa 98,3% alatt van, a magas val acc nem általánosítható a train-eloszlás jeleneteire, ami összeegyeztethető a val jelenetek kivételes könnyűségével vagy a val kiértékelés hibájával, de a 96 jelenetes tanítás miatt önmagában egyiket sem bizonyítja; a véletlenszint 1/3, ezt referenciaként jelentjük, és nincs önkényes 50%-os küszöb. A szivárgásról e kísérlet nem dönt, azt a szerkezeti ellenőrzés igen. A normalizálási statisztikát szintén a forrásindexei döntik el: ha a statisztika trainen kívüli képeket is felhasznált, a hiba fennáll; az azonos seeddel, train-only statisztikával újratanított modell val accja csak leíró kiegészítés, mert a bemenet normalizálása megváltozik, és a pontosság bármely irányba elmozdulhat, ezért az irány önmagában nem bizonyíték. Következtetési korlát: a keresztvalidáció a train jelenetek eloszlásán mér, a teszt-adatot nem használja, a foldonkénti

tanítás 96 jelenete pedig kevesebb az eredeti 120-nál, ezért a becslés konzervatív; az eredmény csak a GlyphLab szintetikus eloszlására érvényes. A keresztvalidáció seedje 20261020, az erőforrás CPU, öt fold és öt újratanítás.

40.2.8 Tipikus tévedés és ellenpélda

A gyakori tévedés: *a helyes tipp nem azonos dokumentált hibadiagnózissal.* A hallgató kijavítja a `model.eval()` sort, a validációs görbe kisimul, és ezt a dropout-hiba bizonyítékának tekinti. De a simább görbe csak a train/eval állapot hatását mutatja; ha a tesztpontosság továbbra is 41%, a label-bug vagy a checkpoint-hiba továbbra is fennállhat. Ellenpélda: a labor 2. kódrészletében már eleve szerepel a `model.eval()`, a modell mégis 1/3 pontosságot mér, mert a hiba a címkepárosításban van; a `model.eval()` újbóli hozzáadása ezen semmit nem változtat. A diagnózis akkor teljes, ha az elkülönítő mérést is elvégezzük, és a naplóban rögzítjük, mely hipotézisek maradtak életben.

40.2.9 Önellenőrzés és visszatekintés

Önellenőrző kérdések: (1) Megadtam-e minden hipotézishez a predikált mérési tartományt a mérés előtt? (2) A split ellenőrzése a `scene_id`-t használja-e, nem a képet? (3) A loss nevezője a definiált megfigyelési egységet osztja-e? (4) A checkpoint a best val állapotot tölti-e, és a betöltés után `eval` módban fut-e a teszt?

Kulcs. (1) Igen, minden hipotézishez a mérés előtt rögzítettük a predikált tartományt. (2) Igen, a split egysége a `scene_id`, nem a kép. (3) Igen, a loss nevezője a definiált megfigyelési egységet számlálja. (4) Igen, a best validációs checkpointot töltjük, és a teszt `eval` módban fut. Visszatekintés a 36.2. Ismétlések és eredménybizonytalanság anyagából: a seed-ismétlés független egysége a teljes futás; ha egy hiba javítása után egyetlen seeddel hasonlítunk, nem tudjuk megkülönböztetni a javítás hatását a seed-zajtól. Visszatekintés a 28.2. Sziámi és triplet cél anyagából: a triplet margin nélkül a konstans embedding collapse; a redukciós hiba mechanizmusa ettől különbözik: a hibás nevező nem nullázza a loss-t, hanem a batchmérettől függően skálázza, ezért a loss és a gradiens nagysága nem hasonlítható össze a batchméret változtatásakor, miközben a modell ugyanazt a célfüggvényt optimalizálja.

40.2.10 Rövid összefoglaló

A hibadiagnózis nem tipp, hanem versengő hipotézisek és előre rögzített, diszjunkt predikciós tartományok rendszere. Az elkülönítő lemma szerint egyetlen mérés akkor dönt, ha a hipotézisek által predikált halmazok nem metszik egymást; zajos mérésnél intervallumokat és standard hibát használunk. A labor öt GlyphLab-hibát diagnosztizált: teljes adatra számolt normalizálás, rossz címkeszelettel mért corrupt teszt, hiányzó `model.eval()`, a batchösszeg és a képenkénti előférszám-nevező keveredése, valamint utolsó-epoch checkpoint. A javítások mindegyikét elkülönítő mérés kíséri, mert a helyes tipp önmagában nem bizonyíték. Az induló kérdésre a válasz: a 98,3%-os validációs és a 41%-os teszteredmény eltérése önmagában nem diagnózis; a `scene_id`-metszet, a címkepárosítás és a train/eval flag szerkezeti ellenőrzése dönti el, hogy címkeeltolódás, split-szivárgás vagy módhiba áll fenn.

40.2.11 Következő lecke

A 40.3. Tudományos kézirat és forrásaudit egység a tudományos kézirat és forrásaudit feladatát adja: a most diagnosztizált hibák után a hallgatónak a kísérleti naplóból olyan kéziratot kell összeállítania, amelyben minden állítás forráshoz, státuszhoz és nyers méréshez kötött, és a felhasznált állítások pontos forráshelyei a jegyzőkönyvben szerepelnek.

40.2.12 Felhasznált források

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [D12] PyTorch contributors (2026). *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. A train/eval mód és a BatchNorm statisztikakezelésének háttere. Hozzáférés: 2026-10-30.
- GlyphLab belső adatszerződés (`lab/ADATOK.md`): a `scene_id` mintavételi egység, a corrupt teszt sorrendje és a split-séma rögzítéséhez. Saját, ellenőrzött kezdőlab.

40.3 Tudományos kézirat és forrásaudit

40.3.1 Cél, előismeret és induló kérdés

Ez az alfejezet azt a lektori munkát teszi taníthatóvá, amelyet a könyv minden korábbi egysége megkövetelt: egy készülő kézirat állításait jelölési, matematikai és hivatkozási követelmények szerint ellenőrizni. Feltételezzük a 40.2. Kísérleti és hibakeresési vizsga hibakeresési vizsga anyagát, továbbá röviden visszanyúlunk a 39.3. Geometriai kontroll és metrika geometriai metrika-, a 36.3. Szakirodalmi reprodukció és attribúció forrásattribúciós és a 28.3. Retrieval-értékelés és adatfelosztás retrieval-kiértékelési fogalmaihoz. Induló kérdés: egy kézirat azt állítja, hogy a modellje 93,3%-os (56/60) pontosságot ért el a GlyphLab teszten (az adatkészletet a függelék A. A GlyphLab adatkészlet szakasza rögzíti), és hogy a transposed convolution „visszaállítja a pooling által elveszett információt”. Milyen bizonyítékok hiányában nem fogadható el a kézirat egyetemi beadandóként?

40.3.2 Definíciók: lektori ellenőrzőlap, javítási napló, állítástípus

Lektori ellenőrzőlap. Olyan strukturált jegyzék, amely a kézirat minden lényeges állításához hozzárendeli: (i) az állítás típusát, (ii) a típushoz kötelező bizonyítékot, (iii) az ellenőrzés módját, (iv) a státuszt. A típusok a könyv szerződéséből ismertek: definíció, tétel/lemma, levezetés, empirikus állítás, heurisztika, szakirodalmi eredmény. Az ellenőrzőlap nem helyettesíti a bizonyítást; azt rögzíti, hogy a bizonyíték rendelkezésre áll-e és olvasható-e.

Javítási napló. A talált hibák időrendi vagy azonosító szerinti listája, amely minden tételnél megadja: a hiba helyét, típusát, következményét, a javítás szövegét és a javítás visszaellenőrzését. A napló akkor teljes, ha egy későbbi olvasó a naplóból reprodukálni tudja, miért változott a szöveg.

A következő négy állítás tipikus állítástípusokat mutat be a forrás/bizonyíték lánc felépítéséből; a lista nem teljes.

40.3.3 Matematikai állítás: a transposed convolution adjungált, nem inverz

Állítás (definícióból levezetett tulajdonság). Legyen $X \in \mathbb{R}^{N\times C_{in}\times H\times W}$ a bemeneti tensor, $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$ a kernel, és jelölje $\mathcal{C}(X;K)$ a PyTorch `Conv2d` által számolt térbeli keresztkorrelációt, rögzített stride, padding és dilation mellett. A transposed convolution $\mathcal{C}^\top(\cdot;K)$ az az operátor, amely a szokásos elem szorzatösszeg skalárszorzatra teljesíti

$$\langle \mathcal{C}(X;K), Y\rangle \;=\; \langle X, \mathcal{C}^\top(Y;K)\rangle$$

minden megfelelő alakú X és Y esetén. Az állítás: $\mathcal{C}^\top$ az adjungált; inverze csak akkor létezhet, ha $\mathcal{C}$ invertálható, és ekkor is csak akkor egyezik meg $\mathcal{C}^\top$-vel, ha $\mathcal{C}$ ortogonális ($\mathcal{C}^\top\mathcal{C} = I$). A tipikus stride/padding-beállításoknál, ahol $\mathcal{C}$ információt veszít, $\mathcal{C}$ nem injektív, ezért inverze sem létezik; a „dekonvolúció visszaállítja az elveszett pixeleket” megfogalmazás tehát matematikailag hibás.

Bizonyíték lánc. Első lépés: a [D10] dokumentáció rögzíti, hogy a PyTorch `Conv2d` keresztkorrelációt számol, nem matematikai konvolúciót; az [S01] kurzusoldala pedig a transposed convolution réteget a gradiens-visszaterjesztés térbeli megfelelőjeként tárgyalja. Második lépés: saját levezetéssel megmutatjuk az adjungált tulajdonságot egy konkrét kis eseten. Harmadik lépés: dimenzióérvvel cáfoljuk az invertálhatóságot.

Kézi példa. Legyen $H = 2$, $W = 3$, egyetlen csatorna, kernel 2×2, stride 1, padding 0. Az output mérete 1×2. Jelölje

$$X = \begin{pmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \end{pmatrix}, \qquad K = \begin{pmatrix} k_{00} & k_{01} \\ k_{10} & k_{11} \end{pmatrix}.$$

A direkt keresztkorreláció:

$$\mathcal{C}(X;K) = \Big(k_{00}x_{00} + k_{01}x_{01} + k_{10}x_{10} + k_{11}x_{11};\;\; k_{00}x_{01} + k_{01}x_{02} + k_{10}x_{11} + k_{11}x_{12} \Big).$$

Az adjungált operátor bemenete $Y = (y_{00}, y_{01})$, kimenete 2×3-as:

$$\mathcal{C}^\top(Y;K) = \begin{pmatrix} y_{00}k_{00} & y_{00}k_{01} + y_{01}k_{00} & y_{01}k_{01} \\ y_{00}k_{10} & y_{00}k_{11} + y_{01}k_{10} & y_{01}k_{11} \end{pmatrix}.$$

A skalárszorzat-egyenlőség tagonkénti beszorzással adódik: $\langle \mathcal{C}(X;K), Y\rangle$ a fenti két output elem y_{00}-lal és y_{01}-gyel vett szorzatösszege; ugyanezt kapjuk, ha X mind a hat elemét megszorozzuk $\mathcal{C}^\top(Y;K)$ megfelelő elemével, mert mindkét oldalon pontosan ugyanazok az $x_{ij}k_{pq}y_{rs}$ szorzatok jelennek meg, ugyanannyiszor. Ez a levezetés a definíció közvetlen kibontása, nem külső tétel.

Miért nem inverz? Általános dimenzióérv: a $\mathcal{C}$ lineáris leképezés $\mathbb{R}^{NC_{in}HW}$-ből $\mathbb{R}^{NC_{out}H_{out}W_{out}}$-ba képez, ahol (dilation = 1) $H_{out} = \lfloor (H + 2p - k_h)/s \rfloor + 1$ és $W_{out} = \lfloor (W + 2p - k_w)/s \rfloor + 1$. Ha $C_{out}H_{out}W_{out} < C_{in}HW$, akkor a rang legfeljebb a kimeneti dimenzió, ezért a rang-nullitás tétel szerint a magtér dimenziója legalább $C_{in}HW - C_{out}H_{out}W_{out} > 0$, és a leképezés nem injektív. Ez tipikusan fennáll $s \geq 2$, $p = 0$, $k_h, k_w \geq 2$ mellett nagy bemeneten, továbbá $C_{out} < C_{in}$ és elég nagy térbeli méret esetén; a kézi példában $6 > 2$, ezért a magtér legalább 4-dimenziós. A $p = 0$, $s = 1$ esetben $H_{out} = H - k_h + 1$, $W_{out} = W - k_w + 1$, tehát a kernel mérete és a bemenet mérete együtt dönti el, hogy a dimenzióérv pozitív nullitást ad-e. A transposed convolution ettől függetlenül jól definiált adjungált, és a gradiens-visszaterjesztésben pontosan ezt a szerepet tölti

be; inverz csak akkor létezik, ha $\mathcal{C}$ invertálható (négyzetes és teljes rangú), és még ekkor sem egyezik meg $\mathcal{C}^\top$-vel, hacsak $\mathcal{C}$ ortogonális.

Forráshasználat. Az adjungáltsági definíció és a kézi példa saját levezetés; az [S01] vonatkozó oldala a rétegtípus motivációjához szolgál attribúcióként.

40.3.4 Matematikai állítás: az L2-tag gradiense

Állítás. Legyen $W \in \mathbb{R}^{m\times d}$, és tekintsük az

$$L_{reg}(W) = \frac{\lambda}{2}\|W\|_F^2 = \frac{\lambda}{2}\sum_{i=1}^{m}\sum_{j=1}^{d} W_{ij}^2$$

regularizációs tagot. Ekkor $\nabla_W L_{reg} = \lambda W$.

Levezetés. A Frobenius-norma négyzetének parciális deriváltja a (p, q) elem szerint:

$$\frac{\partial L_{reg}}{\partial W_{pq}} = \frac{\lambda}{2}\cdot 2W_{pq} = \lambda W_{pq},$$

mert az összegben csak a W_{pq}-t tartalmazó tag deriváltja nem nulla. Mivel ez minden (p, q) párra fennáll, a gradiens mátrixa λW. A $\frac{1}{2}$-es konvenció és a λW gradiens tehát együtt érvényes; a kéziratban a kettőnek konzisztensnek kell lennie.

Kontroll. Ha a kézirat a $\|W\|_F^2$ tagot λ-val szorozná, a gradiens $2\lambda W$ lenne; ha a $\|W\|_F$ (nem négyzetes) tagot deriválná, a gradiens $\lambda W/\|W\|_F$ lenne, ami az origóban nem is értelmezhető. Az ellenőrzőlapnak tehát a konstanstagnál is kérnie kell a gradiensképletet.

40.3.5 Empirikus állítás: a GlyphLab split scene-függetlensége

Állítás. A GlyphLab `metadata.csv` minden `scene_id` értékéhez pontosan egy `split` érték tartozik; a 180 scene mindegyike két view-val szerepel, és a két view ugyanabba a train/val/test részbe kerül.

Bizonyíték lánc. A `lab/ADATOK.md` adatszerződése rögzíti a 180 scene/360 kép struktúrát, a class-stratified felosztást és a `metadata.csv` sémáját: 360 rekord, oszlopai `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax`; az `index` a képtömb indexe (0–359), a `scene_id` szöveges azonosító, a `view_id` 0 vagy 1, a `label` 0, 1 vagy 2, a `split` train/val/test, a doboz pedig zérus alapú, félig nyílt xyxy. A `lab/data/manifest.json` a következőket tartalmazza: `synthetic: true`, `generator_version`, `seed`, `scenes_per_class`, `views`, `class_names`, `input_layout`, `input_range`, `independent_unit: "scene_id"`, `box_convention`, `counts` (kép: 240/60/60), `scene_counts` (120/30/30), `arrays` (a négy tömb alakja és dtype-je), `sha256` (öt fájl lenyomata) és `corrupt_order`. A tényleges állítást a `metadata.csv` sorain kell ellenőrizni, de előbb a manifest hash-értékeit össze kell vetni a fájlokkal. Az ellenőrzés lánca tehát: adatszerződés, manifest, tényleges CSV-sorok, csoportos ellenőrzés.

Ellenőrző kód a `lab/` munkakönyvtárból; a kimenet a labor adatán mért eredmény:

```
import hashlib
import json
from pathlib import Path

import pandas as pd

# Munkakönyvtár: lab/ ; bemenet: data/metadata.csv és data/manifest.json.
manifest = json.loads(Path("data/manifest.json").read_text(encoding="utf-8"))
for name, expected in manifest["sha256"].items():
    digest = hashlib.sha256(Path("data", name).read_bytes()).hexdigest()
    assert digest == expected, (name, digest, expected)   # a CSV a rögzített verzió

meta = pd.read_csv("data/metadata.csv")
assert list(meta.columns) == ["index", "scene_id", "view_id", "label", "split",
                              "xmin", "ymin", "xmax", "ymax"]
assert len(meta) == 360
assert sorted(meta["index"].tolist()) == list(range(360))          # 0..359 egyedi
assert set(meta["view_id"].unique()) == {0, 1}
assert meta.groupby("scene_id")["view_id"].nunique().eq(2).all()   # scene-enként 2 view
# Minden scene pontosan egy splithez tartozik:
bad_scene = meta.groupby("scene_id")["split"].nunique()
assert (bad_scene == 1).all(), bad_scene[bad_scene != 1]
assert meta.groupby("scene_id")["label"].nunique().eq(1).all()     # scene-n belül egységes
↪  label
n_scene = meta.drop_duplicates("scene_id").groupby("split").size().to_dict()
assert n_scene == {"train": 120, "val": 30, "test": 30}, n_scene   # 120/30/30 felosztás
by_class = (meta.drop_duplicates("scene_id").groupby(["split", "label"]).size()
            .unstack().to_dict("index"))
assert by_class["train"] == {0: 40, 1: 40, 2: 40}
assert by_class["val"] == {0: 10, 1: 10, 2: 10}
assert by_class["test"] == {0: 10, 1: 10, 2: 10}
print(by_class)
# Mért kimenet (Python 3.12.3, pandas 3.1.0, CPU; a hash-ellenőrzés mind az öt fájlra egyezett):
# {'test': {0: 10, 1: 10, 2: 10},
#  'train': {0: 40, 1: 40, 2: 40},
#  'val': {0: 10, 1: 10, 2: 10}}
```

Következmény. Ha a kézirat per-image metrikát és per-scene konfidenciaintervallumot kever, az ellenőrzőlapnak jeleznie kell a különbséget: a bizonytalanság megfigyelési egysége csak indokolt esetben a kép; a scene-szintű függőség miatt a védhető alapválasztás a `scene_id`. Ez kapcsolódik a 39.3. Geometriai kontroll és metrika geometriai kontroll páros tesztjéhez és a 28.3. Retrieval-értékelés és adatfelosztás group split követelményéhez.

40.3.6 Empirikus állítás: a per-image bootstrap kockázata a GlyphLabon

Állítás. A GlyphLab adatból számolt pontosság bizonytalanságát kép-szintű bootstrap resamplinggel becsülni csak akkor fogadható el, ha a kézirat igazolja, hogy ugyanazon scene két view-jának hibája független; ilyen igazolás nélkül a scene-szintű resampling a védhető alapválasztás, mert a két view hibája függő is lehet.

Bizonyíték lánc. A klasszikus (iid) bootstrap feltétele, hogy a resampling egységek függetlenek legyenek; ez nem a bootstrap minden változatának definíciója: a blokk- és cluster-bootstrap éppen függő adatot kezel, a függőséget az egységbe (itt a scene-be) zárva. A GlyphLab generátora (A. A GlyphLab adatkészlet, `lab/ADATOK.md`) minden scene-hez két zajos view-t készít; a view-k közös scene-pozíciót, vastagságot és hosszt osztanak meg, ezért a modell hibái egy scene két view-ján függők *lehetnek*. A közös scene-paraméterekből nem következik szükségképpen, hogy egy tetszőleges modell két view-n mért hibája korrelált (konstans hibák vagy speciális modell esetén nem); a konkrét hibakorrelációt mérni kell. Mivel a függőség lehetséges, a scene-szintű resampling

védhető alapválasztás. Ha az egység az egyes kép, akkor egy scene mindkét view-ja független megfigyelésként kerül a bootstrap mintába, ami függőség esetén a varianciát jellemzően alulbecsli. Az intervallumok értelmezése általában az ismételt mintavételi feltételhez kötött, de itt nem hivatkozunk általános cluster-bootstrap tételre: a fenti következtetés a saját adatszerződésből és a függetlenségi feltételből következik.

Ellenőrzés a kéziratban. A lektornak azt kell kérnie, hogy a kézirat explicite nevezze meg a bootstrap egységet, és ha az egyes kép, akkor igazolja a két view hibájának függetlenségét. Ilyen igazolás nélkül a védhető alapválasztás a `scene_id` szerinti, class-stratified scene-resampling; a per-image bootstrap nem önmagában hibás, hanem függőség esetén alulbecsli a bizonytalanságot. A két egység nem felcserélhető; a 39.3. Geometriai kontroll és metrika a páros clean–corrupt összevetést rögzíti, ami a bootstrap egységének megválasztásától független követelmény.

40.3.7 Labor: lektori ellenőrzőlap és javítási napló a mintafejezethez

Mintafejezet-részlet (a hallgató ezt auditálja; a hibák szándékosak):

> „A modellünk transposed convolutionnal invertálja a poolingot, így visszaállítja az elveszett pixeleket. A tanulási görbe stabilan csökkent (1. ábra, mért eredmény). A batch norm tanítás és kiértékelés alatt ugyanazt a statisztikát használja. A [Kitalált 2021] szerint az FGSM mindig megtalálja a legjobb torzítást. Az L2-regularizáció gradientsét $2\lambda W$-nek választottuk.”

A mintaszöveg szándékosan hibás. A `transposed convolution`, a `batch norm` és az `FGSM` angol terminus magyar kifejtés nélkül szerepel, a `gradientsét` pedig hibás szóalak (`gradiensét` helyett); ezeket a hallgatónak a nyelvi audit részeként kell megjelölnie, és a javítási naplóban a magyar megfelelővel együtt kell használnia.

A labor beadandó eredménye két dokumentum: egy ellenőrzőlap és egy javítási napló. Az ellenőrzőlap szerkezete:

Állítás	Típus	Kötelező bizonyíték	Státusz
Transposed convolution inverz	Matematikai lemma	Adjungált definíció, dimenzióérv	Hibás, javítandó
„Mért eredmény” görbe	Empirikus állítás	Protokoll, seed, checkpoint, nyers adat	Nem igazolt
BatchNorm állandó statisztika	Tényállítás (implementációs viselkedés)	`train/eval` állapotleírás, [D12] dokumentáció	Hibás
[Kitalált 2021] szerint az FGSM mindig megtalálja a legjobb torzítást	Szakirodalmi eredmény és tényállítás	Létező forrás; az egylépéses, lineáris közelítés dokumentálása	Hiányzó forrás és hamis tartalom
L2 gradiens $2\lambda W$	Levezetés	Parciális deriválás	Hibás a $\frac{\lambda}{2}$ konvencióval

A javítási napló négy tételből áll; az első például:

1. **Hiba:** „transposed convolutionnal invertálja a poolingot”. **Következmény:** hamis állítás a réteg képességéről, félrevezeti a felhasználási szerződést. **Javítás:** „A transposed convolution a megfelelő lineáris operátor adjungáltjaként térbeli növelést végez; nem általános inverz,

ezért a pooling által eldobott értékeket nem állítja vissza.” **Visszaellenőrzés:** a javított mondat két részét külön ellenőrizzük. Az adjungált voltát a $\langle \mathcal{C}(X;K), Y\rangle = \langle X, \mathcal{C}^{\top}(Y;K)\rangle$ azonosság igazolja; az „nem általános inverz” részt a fenti 2×3-as példa magtér-számítása támasztja alá, mert a magtér legalább 4-dimenziós, ezért a leképezés nem injektív, és két különböző bemenethez is ugyanaz a kimenet tartozhat.

2. **Hiba:** „a batch norm tanítás és kiértékelés alatt ugyanazt a statisztikát használja”. **Következmény:** hamis implementációs állítás; ha a kiértékelés a batch statisztikáját használná, az eredmény a batch összetételétől függne, és nem lenne reprodukálható. **Javítás:** „A batch normalization tanítás alatt a batch átlagát és szórását használja, és frissíti a futó statisztikát; kiértékeléskor a futó statisztikát használja, és nem frissíti. A módot a `model.train()`, illetve `model.eval()` állítja.” **Visszaellenőrzés:** PyTorch `BatchNorm1d`-n `train()` módban a `running_mean` és `running_var` változik, `eval()` módban nem; a `model.training` flag a két módot jelzi. A viselkedést a labor venvjében futtatott ellenőrzés igazolta.

3. **Hiba:** „a [Kitalált 2021] szerint az FGSM mindig megtalálja a legjobb torzítást”. **Következmény:** a hivatkozás nem létezik, és a tartalom is hamis: az FGSM egylépéses, az input gradiense előjelén alapuló elsőrendű közelítés, amely nem garantálja a veszteséget globálisan maximalizáló perturbációt. **Javítás:** a hivatkozás törlése, és a helyes állítás a 35.3. Perturbáció és adversarial példa elsőrendű lemma szerint: „Az FGSM az ℓ_∞-korlát mellett az elsőrendű közelítést maximalizálja; ez nem garantálja a globális optimumot, különösen erősen görbült veszteségfelületnél nem.” **Visszaellenőrzés:** a lemma feltételei (differenciálható L, $\|\delta\|_\infty \le \varepsilon$) és az előjeles lépés levezetése a hivatkozott alfejezetben szerepel; a „mindig” és „legjobb” szavak törlendők.

4. **Hiba:** „az L2-regularizáció gradientsét $2\lambda W$-nek választottuk” a $\frac{\lambda}{2}\|W\|_F^2$ tag mellett. **Következmény:** a gradiens kétszeres; SGD-nél ez a bejelentettnél kétszer nagyobb súlycsökkenést ad, AdamW-nél a leválasztott decay miatt a hatás implementációfüggő, és a közölt λ nem írja le egyértelműen az effektív regularizációt. **Javítás:** a $\frac{\lambda}{2}\|W\|_F^2$ taghoz λW, a $\lambda\|W\|_F^2$ taghoz $2\lambda W$ tartozik; a kéziratnak egy konvenciót kell rögzítenie, és meg kell adnia, hogy a regularizáció a gradiensben vagy AdamW leválasztott tagjaként szerepel. **Visszaellenőrzés:** a mátrixelem szerinti parciális derivált $\partial(\frac{\lambda}{2}\sum_{p,q} W_{pq}^2)/\partial W_{pq} = \lambda W_{pq}$; a 40.3.4 L2-levezetése ezt a konvenciót használja.

A [D12] dokumentációja a BatchNorm `train`/`eval` statisztikakezelésére vonatkozik; a javítási napló 2. tétele ezt a hivatkozást használja, és megnevezi a könyvtárverziót. A kitalált hivatkozást a napló nem pótolhatja valódi forrással; a 3. tétel ezért törli a hivatkozást, és a könyv saját, 35.3. Perturbáció és adversarial példa lemmájára mutat.

40.3.8 Feladatok

F1 (fogalmi, nehézség 1). Adja meg a „mért” és a „szemléltető” ábra definícióját, és a mintafejezetből azonosítsa, melyik állítás sérti a megkülönböztetést. Beadandó: két definíció és egy mondatnyi indoklás.

F2 (levezetési, nehézség 2). A fenti 2×3-as példa jelöléseivel igazolja tagonként, hogy $\langle \mathcal{C}(X;K), Y\rangle = \langle X, \mathcal{C}^{\top}(Y;K)\rangle$, és mutassa meg, hogy a $\mathcal{C}$ leképezés magtere legalább 4-dimenziós. Beadandó: a két oldal kifejtett összege és a dimenzióindoklás.

F3 (implementációs, nehézség 1). Írjon Python-ellenőrzést, amely a `lab/data/metadata.csv`-ből megállapítja: (i) minden `scene_id` pontosan egy `split`-hez tartozik-e; (ii) a test splitben osztályonként pontosan 10 scene van-e. Beadandó: teljes kód és a szerződésből számított várt kimenet.

F4 (hibakeresési, nehézség 2). A mintafejezet az L2-gradienst $2\lambda W$-nek írja. Azonosítsa a hiba típusát, következményét az optimalizáló lépésre, és adja meg a helyes képletet a $\frac{\lambda}{2}$-es taghoz. Beadandó: ok, következmény, javítás.

F5 (kísérleti/transzfer, nehézség 3). Egy kézirat azt állítja: „a modell a GlyphLab test halmazon 93,3%-os (56/60) pontosságot ért el”. Tervezze meg, milyen bizonyítékokat kér be a lektor, hogy az állítást mért eredményként elfogadja; fogalmazzon meg egy elfogadási küszöböt és egy elutasítási feltételt. Beadandó: protokollterv listája.

40.3.9 Megoldások és indoklás

F1 megoldás. *Mért* eredmény: rögzített protokoll mellett ténylegesen lefuttatott számítás kimenete, amelyhez tartozik adatverzió, split, seed, modellállapot, metrika és nyers érték. *Szemléltető* ábra: képletből vagy feltételezett paraméterekből készült, nem mérésen alapuló bemutatás. A mintafejezet hibája: „mért eredmény”-ként hivatkozik az 1. ábrára, miközben a szöveg semmilyen futási adatot nem közöl hozzá; a helyes megfogalmazás vagy a teljes mérési jegyzőkönyv csatolása, vagy „számított szemléltetés” címke. Ez a megkülönböztetés nem ízlés kérdése: hiányzó protokoll mellett az állítás empirikusan ellenőrizhetetlen.

F2 megoldás. A direkt oldal:

$$\langle \mathcal{C}(X;K), Y\rangle = y_{00}\sum_{p,q} x_{pq}k_{pq} + y_{01}(k_{00}x_{01}+k_{01}x_{02}+k_{10}x_{11}+k_{11}x_{12}),$$

ahol az első összeg indexei $p\in\{0;\ 1\}, q\in\{0;\ 1\}$ a 2×2-es kernelablakra. Kifejtve ez nyolc $x_{ij}k_{pq}y_{rs}$ szorzat, rendre $x_{00}k_{00}y_{00}$, $x_{01}k_{01}y_{00}$, $x_{10}k_{10}y_{00}$, $x_{11}k_{11}y_{00}$, $x_{01}k_{00}y_{01}$, $x_{02}k_{01}y_{01}$, $x_{11}k_{10}y_{01}$, $x_{12}k_{11}y_{01}$. Az adjungált oldal $\sum_{i,j} x_{ij}\,\hat{x}_{ij}$, ahol $\hat{x}_{ij}$ az $\mathcal{C}^{\top}(Y;K)$ mátrix eleme; behelyettesítve pontosan ugyanezt a nyolc szorzatot kapjuk, ezért a két oldal minden X, Y érték mellett egyenlő. A magtérre: $\mathcal{C}$ lineáris leképezés $\mathbb{R}^6\to\mathbb{R}^2$, ezért a rang legfeljebb 2; a nullitás a rang-nullitás tételből $6-\text{rang}\geq 4$. Tehát legalább 4 lineárisan független X ad zérus kimenetet, a leképezés nem injektív. (A rang-nullitás tételt itt nem bizonyítjuk: a 3. fejezet a rangot definiálja, a tétel bizonyítását [B01] 2. fejezete adja.)

F3 megoldás.

```
import pandas as pd

# munkakönyvtár: lab/
meta = pd.read_csv("data/metadata.csv")
assert meta.groupby("scene_id")["split"].nunique().eq(1).all()
test_scenes = meta[meta["split"] == "test"].drop_duplicates("scene_id")
counts = test_scenes["label"].value_counts().to_dict()
assert counts == {0: 10, 1: 10, 2: 10}
print(counts)
# Mért futás (Python 3.12.3, pandas 3.1.0, CPU): {0: 10, 1: 10, 2: 10}
```

A kód az adatszerződés ellenőrzött darabszámain alapul; a tényleges futtatás előtt a `manifest.json` hash-ellenőrzését is el kell végezni, mert a CSV önmagában nem bizonyítja, hogy a fájl a rögzített verzió.

F4 megoldás. A hiba típusa: konstanstag-hiba a levezetésben. Ha a regularizációs tag $\frac{\lambda}{2}\|W\|_F^2$, akkor a parciális derivált λW_{pq}, nem $2\lambda W_{pq}$. A $2\lambda W$ gradiens a $\lambda\|W\|_F^2$ taghoz tartozna. Következmény: SGD-nél a hibás gradiens a bejelentettnél kétszer nagyobb súlycsökkenést ad. AdamW-nél a súlycsökkenés leválasztott (decoupled) tag, amely nem a ∇L-en keresztül hat, ezért ott a hiba nem egyszerűen „kétszeres decay”: ha a $2\lambda W$ gradiens mellett külön

`weight_decay=` is be van kapcsolva, a két hatás összeadódik; ha nincs, a gradiensbeli tag viselkedése az implementációtól függ. A kéziratnak ezért rögzítenie kell, hogy a regularizáció a gradiensben vagy a leválasztott tagban szerepel. Javítás: vagy a tag legyen $\frac{\lambda}{2}\|W\|_F^2$, és a gradiens λW; vagy a tag legyen $\lambda\|W\|_F^2$, és a gradiens $2\lambda W$. A kettő közül a kéziratnak egyet kell következetesen használnia.

F5 megoldás. A lektor az alábbi bizonyítékokat kéri be: (1) a pontos adatverzió és a `manifest.json` hash-ellenőrzés kimenete; (2) a split azonosító és a modellválasztási protokoll, amely szerint a test csak a végleges checkpoint után került kiértékelésre; (3) a training seed(ek), könyvtárverzió, dtype és device; (4) a checkpoint `state_dict` lenyomata; (5) a nyers predikciók a test képekre; (6) a metrika definíciója és a nyers confusion matrix. A metrikát két szinten kell rögzíteni: a per-image accuracy a helyes képek száma osztva 60-nal, a scene-szintű összesítés pedig a 30 `scene_id` közül azok aránya, amelyekben mindkét view helyes. A pontbecslés lehet per-image, de a bizonytalanság megfigyelési egysége a `scene_id` (40.3.6), ezért a közölt intervallumnak scene-szintű bootstrapból kell származnia. Elfogadási küszöb: a fenti hat dokumentum mind elérhető; a checkpointból a rögzített labor-környezetben (Python 3.12.3, torch 2.15.0+cpu, CPU, determinisztikus inference) futtatott újrakiértékelés a per-image accuracyot pontosan 56/60-nak (93,3%), a mindkét-view-helyes scene-ek arányát pedig 28/30-nak adja, azaz 0 kép és 0 scene eltérés; a bootstrap-intervallum végpontjai a rögzített seed mellett legfeljebb 10^{-12} abszolút eltéréssel egyeznek. A 28/30 scene-érték csak akkor adódik, ha a 4 hibás kép két scene mindkét view-jára esik; ezt a nyers predikciókkal ellenőrizni kell. A megengedett értékeket a mérés egysége szabja meg: 60 kép esetén a per-image accuracy lépése $1/60 \approx 1{,}67$ százalékpont, a mindkét-view-helyes scene-ek arányának lépése $1/30 \approx 3{,}33$ százalékpont, a scene-enkénti view-pontosság átlagáé pedig $0{,}5/30 = 1/60$; ezért a 93,3% (56/60 kép, illetve 28/30 jelenet) a képszintű és a mindkét-view-helyes scene-szintű egységgel is elérhető, a 94% viszont egyikkel sem. Ha a kézirat csak a 93,3%-ot közli, a nyers predikciók és a scene-bontás hiánya miatt az állítás nem fogadható el. Elutasítási feltétel: ha a kézirat a test halmazt bármilyen hiperparaméter- vagy checkpoint-választásra használta, ha a seed vagy a checkpoint hiányzik, vagy ha a „93,3%” (56/60) nem párosítható nyers predikciókhoz. A terv nem követeli meg, hogy a lektor maga tanítsa újra a modellt; a független inference a mentett state_dict-ből elegendő.

40.3.10 Tipikus tévedés példával

A leggyakoribb hiba abban rejlik, hogy a generált vagy gyorsan írt szöveg stilisztikailag magabiztos, ezért az olvasó hajlamos ellenőrzöttként kezelni. **Példa:** a mintafejezetben a „[Kitalált 2021]” hivatkozás formailag tudományosnak látszik, de nem létező forrásra mutat. A tudományos megbízhatóság nem a mondatok folyamatosságából következik, hanem abból, hogy minden nem triviális állítás mögött azonosított típusú, olvasható bizonyíték áll. A lektori ellenőrzőlap pontosan ezt a különbséget teszi láthatóvá: a formai hasonlóság nem pótolja a bizonyítási láncot.

40.3.11 Önellenőrzés és visszatekintés

Önellenőrzés. (1) A transposed convolution példában a kimeneti méret $H_{out} = H - k_h + 1 = 1$, $W_{out} = W - k_w + 1 = 2$, konzisztens a padding nélküli képlettel. (2) Az L2-levezetésben a mátrixelem szerinti parciális derivált minden (p, q)-ra azonos alakú, a gradiens alakja $m \times d$, megegyezik W alakjával. (3) A split-ellenőrzésben a `scene_id` csoportosítás alakja a 360 soros táblán 180 csoport, mindegyik 2 view-val. (4) A per-image bootstrap nem önmagában hibás; a scene-szintű egység választását az indokolja, hogy a két view hibája függő lehet, és ezt a kéziratnak igazolnia kell.

Visszatekintés. A 40.2. Kísérleti és hibakeresési vizsga hibakeresési vizsga szerint a versengő hipotéziseket úgy kell szétválasztani, hogy a kontroll csak az egyik tényezőt változtatja; itt a „mért vagy szemléltető” kérdésnél ugyanez a gondolat jelentkezik, amikor a checkpoint és a seed hiányát kérjük számon. A 36.3. Szakirodalmi reprodukció és attribúció forrásattribúciós szabálya szerint az olvasottsági szintet a szakasz pontosságával együtt kell közölni; a jelen javítási napló ezt a kitalált hivatkozásnál alkalmazza. A 28.3. Retrieval-értékelés és adatfelosztás group split követelménye a `scene_id` mintavételi egységben tér vissza, a 39.3. Geometriai kontroll és metrika páros tesztje pedig a corrupt/clean összevetés auditjában.

40.3.12 Rövid összefoglaló

A tudományos kézirat auditja abból áll, hogy minden lényeges állítást típus szerint osztályozunk, és a típushoz kötelező bizonyítékot kérjük számon. Matematikai állításnál a definícióból való levezetés és a dimenzió- vagy szélsőeset-kontroll a mérce; empirikus állításnál az adatverzió, split, seed, checkpoint, metrika és nyers eredmény együttese. A transposed convolution adjungált, nem inverz; az L2-tag gradiense a konvenciótól függ; a GlyphLab split scene-szintű, és egy scene két view-jának hibája függő lehet, ezért a bootstrap védhető alapválasztása a `scene_id` egység. A lektori ellenőrzőlap és javítási napló ezt a láncot rögzíti, és a kitalált hivatkozást vagy a mértnek álcázott szemléltetést azonnal láthatóvá teszi.

40.3.13 Következő lecke és források

A következő egység, 40.4. X. ellenőrző pont: önálló neurális modell és bizonyíték, a X. ellenőrző pont: az itt bemutatott ellenőrzőlapot és javítási naplót egy teljes, korábban nem látott feladaton kell alkalmazni, önálló neurális modell és bizonyíték bemutatásával. A most elkészített auditdokumentumok adják a vizsgabeadandó formai vázát.

Felhasznált források:

A PyTorch-dokumentáció hivatkozásai a 2.14-es verzióra mutatnak; a hozzáférés dátumát a tétel tartalmazza.

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [S01] Stanford CS231n (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.
- [D10] PyTorch contributors (2026). *Conv2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html. Hozzáférés: 2026-10-30.
- [D12] PyTorch contributors (2026). *BatchNorm2d*. PyTorch 2.14 dokumentáció. https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html. A train/eval mód és a BatchNorm statisztikakezelésének háttere. Hozzáférés: 2026-10-30.

40.4 X. ellenőrző pont: önálló neurális modell és bizonyíték

40.4.1 Cél, előismeret és induló kérdés

Ez a X. ellenőrző pont: a teljes tananyag matematikai, implementációs és kísérleti teljesítményét egyetlen **új, korábban nem tárgyalt feladaton** kell önállóan bemutatni. A 40.3. Tudományos kézirat és forrásaudit forrásauditja, a 39.4. Alkalmazási riport és átadás átadási szerződése, a 36.4. IX. ellenőrző pont: lektorálható modellállítás lektorálható csomagja és a 28.4. VII. ellenőrző

pont: CNN alkalmazási szerződés alkalmazási szerződése együtt adja azt a keretet, amelyben a hallgató a feladatkiírástól a futtatható modellig és az érvényességi korlátig minden lépést maga végez. Előismeret: a CNN-operátor alakszabályai 14.1.–14.4., az MSE-regressziós veszteség 7.3. Bináris, multilabel és regressziós cél, a gradiensellenőrzés 15.4. Független gradiensellenőrzési protokoll, a best-checkpoint-választás 24.4. VI. ellenőrző pont: megbízható CNN-training és a scene-szintű mintavételi egység 5.4. Függő képek, seedek és resampling.

Értékelés. A közös 0–4 rubrikát, a PASS-szabályt és a hibatípusonkénti visszautakat az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok" szakasza adja meg; itt csak az eltérés: a riport a hét komponensű projektcsomag, és a PASS része a szóbeli védés is. Ezen a ponton a D3 (adat- és mérési protokoll) és a D4 (bizonyíték és nyomon követhetőség) dimenzió szigorúbb: minden állításhoz seed, verzió és fájlhash tartozik.

Induló kérdés: elegendő-e a korábbi orientáció-osztályozó fejének kicserélése, ha az új cél egy folytonos geometriai mennyiség? Becslés: nem. A target típusa, a veszteség, a metrika, az inference-kimenet normalizálása és a hiba értelmezése mind más szerződést követel; a CNN-gerinc (backbone) **architektúrája** marad újrafelhasználható, a súlyai nem: az új feladat feje és tanított paraméterei nulláról indulnak.

40.4.2 A vizsgafeladat: új probléma specifikációja

Feladat: a függelék A. A GlyphLab adatkészlet szakaszában leírt GlyphLab-képekből becsüljük meg az előtér függőleges kiterjedését, azaz a maszk befoglaló téglalapjának magasságát:

$$t = \mathrm{ymax} - \mathrm{ymin}, \qquad t \in \{1;\ \ldots;\ 24\}.$$

A doboz félnyílt pixelkoordinátákkal adott: $y_{\min} \le y < y_{\max}$, ahol $0 \le y < 24$, ezért $y_{\max} \in \{1;\ \ldots;\ 24\}$, és a $t = y_{\max} - y_{\min}$ érték elvileg 1 és 24 közé esik; a 24 csak a teljes magasságot kitöltő előtérnél fordul elő. A konkrét GlyphLab-adatban a megfigyelt célok 1 és 9 között vannak.

A bemenet továbbra is egyetlen zajos nézet $x \in \mathbb{R}^{1\times 24\times 24}$, $x \in [0;\ 1]$, NCHW alakban, float32 típusban. A cél **regressziós**, nem osztályozási: a t pozitív egész, de a hálózat valós kimenetet ad. A két nézet ugyanahhoz a jelenethez tartozik, ezért a t célérték jelenetenként közös; a független mintavételi egység a `scene_id`. A felosztás az eredeti szerződést követi: 120/30/30 jelenet train/val/test, osztályonkénti rétegzéssel; a jelenet mindkét nézete azonos splitbe kerül. A tesztet sem modell-, sem hiperparaméter-választásra nem használjuk.

A hallgatónak hét beadandó komponenst kell előállítania: **problem, mathematical model, implementation, data, evaluation, inference, report.** A `problem` egy féloldalas feladatleírás a fenti szerződéssel; a `data` a meglévő képekhez a metadata.csv `ymin, ymax` oszlopaiból képzett `bbox_height.npy` célállomány (a bbox függőleges kiterjedése, nem a generátor `thickness` vonalvastagság-paramétere: az orientációtól és a hossztól is függhet, ezért a becsült mennyiséget nem szabad vonalvastagságnak nevezni a feladatban és a model cardban), amelynek SHA-256 lenyomatát a futásnapló rögzíti; a többi komponenst alább részletezzük.

40.4.3 Matematikai modell és levezetési követelmény

A matematikai modell felépítése a [B01] jelölését és a [B03] mátrixdifferenciálási konvencióját követi.

A $\theta = (\phi, W, b)$ paraméterű modell, ahol ϕ a konvolúciós gerinc (backbone) paramétereit, W és b pedig a kimeneti lineáris réteg súlyát és biasát jelöli ([B02], [S01]),

$$f_\theta : \mathbb{R}^{1\times 24\times 24} \to \mathbb{R}, \qquad \hat{t} = f_\theta(x),$$

ahol $\hat{t}$ nyers, aktiváció nélküli kimenet. A veszteség átlagolt MSE:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \left(f_\theta(x_i) - t_i\right)^2,$$

ahol N a batch mérete (a minták száma a batchben), nem a teljes adathalmaz elemszáma.

Az inference-szerződés a nyers kimenetet lezárja:

$$\hat{t}_{\text{final}} = \text{clip}\left(\text{round}(\hat{t}),\ 1,\ 24\right).$$

A kerekítés és a levágás (`clip` az [1; 24] tartományra) az abszolút hibát nem mindig ugyanabba az irányba változtatja. A levágás az [1; 24]-beli valódi célhoz képest önmagában nem növelheti a hibát (például $\hat{t} = 26$, $t = 24$: a hiba $2 \to 0$). A kerekítés növelheti és csökkentheti is: $\hat{t} = 3{,}4$, $t = 3$ esetén $0{,}4 \to 0$ (csökken), $\hat{t} = 3{,}6$, $t = 3$ esetén $0{,}6 \to 1$ (nő). Ezért a nyers val MAE szerinti modellválasztás (a folytonos célhoz) és a végső, kerekített test MAE (az inference-szerződés kimenete) különböző célt mér; mindkettőt közöljük. A kerekítés tie-szabálya a `torch.round` viselkedése: a pontosan $x{,}5$ értékek a páros egészre kerülnek (*round half to even*, [S04]); az utófeldolgozás verziója `clip(round_half_to_even(t), 1, 24) v1`, és ezt az inference-szerződés rögzíti.

A **levezetési követelmény** a vizsgafeladatban egy új tételalkalmazás: a kimeneti lineáris réteg gradiensének pontos, alakhelyes levezetése a VJP-konvencióban.

1. lemma (MSE-gradiens a végső lineáris rétegen). Legyen $f_\theta(x) = Wa + b$, ahol $W \in \mathbb{R}^{1\times D}$, $a = h_\phi(x) \in \mathbb{R}^D$ az x-től függő rögzített jellemzővektor, $b \in \mathbb{R}$. Az N elemű batchre vett átlagolt MSE veszteségre

$$\nabla_W L = \frac{2}{N}\sum_{i=1}^{N} \left(\hat{t}_i - t_i\right) a_i^\top \in \mathbb{R}^{1\times D}, \qquad \nabla_b L = \frac{2}{N}\sum_{i=1}^{N} \left(\hat{t}_i - t_i\right).$$

Bizonyítás. A láncszabály skalárkoordinátánként: $\partial L/\partial \hat{t}_i = 2(\hat{t}_i - t_i)/N$. A 4.3. Jacobi-mátrix, láncszabály és VJP VJP-konvenciójában ([B03]) a $g = \nabla_{\hat{t}} L \in \mathbb{R}^{N\times 1}$ upstream a $Z = XW^\top + b$ lineáris rétegen $\nabla_W L = g^\top X$-szel, $\nabla_b L = g^\top \mathbf{1}_N$-nel terjed vissza. Itt az „input” X egy sora $a_i^\top$, így

$$\nabla_W L = \sum_{i=1}^{N} g_i a_i^\top = \frac{2}{N}\sum_i (\hat{t}_i - t_i) a_i^\top, \qquad \nabla_b L = \sum_i g_i = \frac{2}{N}\sum_i (\hat{t}_i - t_i).$$

Az alakellenőrzés: g_i skalár, $a_i^\top$ $1 \times D$ sorvektor, az összeg $1 \times D$, ami megegyezik W alakjával. A keresztkorreláció-rétegre ez a lemma nem vonatkozik változatlanul; ott az átfedő patch-ek súlygradienseit a 15.1. Súly- és biasgradiens szabálya szerint összegezni kell.

A **konvolúciós kimeneti méret tételét** 14.1. Stride, padding, dilation és outputméret alfejezetből idézzük vissza: $H_{out} = \lfloor (H + 2p - d(k-1) - 1)/s \rfloor + 1$. A vizsgafeladat azt kéri, hogy a hallgató alkalmazza ezt a tételt a saját architektúrájára, és adjon **ellenpéldát** olyan

(H, k, d, p, s) konfigurációra, ahol a képlet nem pozitív kimenetet ad. Ez utóbbi a `Conv2d` számára érvénytelen beállítás, tehát a tétel feltételeinek megsértését mutatja.

40.4.4 Eredeti kézi kis példa

A levezetési követelmény kézi kontrollja egy 1D keresztkorrelációs rétegen, az átlagos MSE-vel. Legyen input $x = [1;\ 2;\ 3;\ 4]$, kernel $k = [1;\ -1]$, bias 0, padding 0, stride 1, dilation 1. A kimenet hossza $\lfloor (4+2\cdot 0-1\cdot(2-1)-1)/1 \rfloor + 1 = 3$, tehát $y = [1-2;\ 2-3;\ 3-4] = [-1;\ -1;\ -1]$. Cél $t = [1;\ 0;\ 0]$. A veszteség

$$L = \frac{1}{3}\left[(-2)^2 + (-1)^2 + (-1)^2\right] = \frac{6}{3} = 2, \qquad \nabla_y L = \frac{2}{3}(y-t) = \frac{2}{3}[-2;\ -1;\ -1] = \left[-\frac{4}{3};\ -\frac{2}{3};\ -\frac{2}{3}\right].$$

A kernelgradiens az átfedő inputpatch-ekkel: k_0 (pozíció 0) esetén

$$\frac{\partial L}{\partial k_0} = \sum_{o=0}^{2} (\nabla_y L)_o \, x_{o+0} = 1 \cdot \left(-\frac{4}{3}\right) + 2 \cdot \left(-\frac{2}{3}\right) + 3 \cdot \left(-\frac{2}{3}\right) = -\frac{4}{3} - \frac{4}{3} - 2 = -\frac{14}{3},$$

k_1 esetén

$$\frac{\partial L}{\partial k_1} = \sum_{o=0}^{2} (\nabla_y L)_o \, x_{o+1} = 2 \cdot \left(-\frac{4}{3}\right) + 3 \cdot \left(-\frac{2}{3}\right) + 4 \cdot \left(-\frac{2}{3}\right) = -\frac{8}{3} - 2 - \frac{8}{3} = -\frac{22}{3}.$$

A két derivált független ellenőrzése középső véges differenciával ($h = 10^{-6}$, float64): $\partial L/\partial k_0 \approx -4{,}6666666663$ és $\partial L/\partial k_1 \approx -7{,}3333333331$, vagyis $-14/3$ és $-22/3$.

Egy lépésnyi, $\eta = 0{,}3$ tanulási rátájú gradienslépést csak a kernelre teszünk; a bias 0-n marad (a bias frissítése külön döntés, és ez a kontroll kizárólag a kernellépést vizsgálja). Ekkor

$$k \leftarrow k - \eta \nabla_k L = \left[1 + 0{,}3 \cdot \tfrac{14}{3};\ -1 + 0{,}3 \cdot \tfrac{22}{3}\right] = [2{,}4;\ 1{,}2],$$

a frissítés iránya az adott pontban a negatív gradiens. A csökkenést azonban külön számolással kell ellenőrizni: az új kernel kimenete $[2{,}4\cdot 1 + 1{,}2\cdot 2;\ 2{,}4\cdot 2 + 1{,}2\cdot 3;\ 2{,}4\cdot 3 + 1{,}2\cdot 4] = [4{,}8;\ 8{,}4;\ 12]$, így az új MSE

$$\frac{(4{,}8-1)^2 + 8{,}4^2 + 12^2}{3} = \frac{14{,}44 + 70{,}56 + 144}{3} = \frac{229}{3} \approx 76{,}33,$$

amely **nagyobb** a kezdeti 2-nél. Ez a példa megmutatja, hogy egy túl nagy tanulási ráta a helyes gradiens ellenére is növelheti a veszteséget. A súlygradiens-összegzés (15.1. Súly- és biasgradiens) független megerősítése nem maga a kézi képlet, hanem a fenti véges differenciás kontroll, amely a képlettől függetlenül, csak a veszteségfüggvény kiértékeléséből számol.

40.4.5 Konkrét labor: teljes PyTorch projekt

A labor egyetlen, újrafuttatható szkriptet ad: `lab/n160_bbox_height.py`. Munkakönyvtár: a labor `lab/` könyvtára; a szkript az útvonalakat a `Path(__file__)` alapján a saját könyvtárához igazítja, ezért a `lab/` könyvtárból futtatva a bemenet `data/images.npy` és `data/metadata.csv`, a targetet a szkript a `ymax-ymin` oszlopokból determinisztikusan állítja elő, és ugyanoda, `data/bbox_height.npy` néven menti. A `metadata.csv` oszlopai: `index, scene_id, view_id, label, split, xmin, ymin, xmax, ymax`; az `index` a teljes képtömb sorindexe, a `split` értéke `train`, `val` vagy `test`, a doboz félnyílt xyxy. A metadata sorait nem a CSV sorrendje, hanem az `index` mező köti az `images.npy` soraihoz: az `align_metadata` ellenőrzi, hogy az index egyedi és pontosan a $0, \dots, N-1$ tartományt fedi le, majd index szerint rendez; a szkript ellenőrzi, hogy egy véletlenül permutált CSV-sorrendet ugyanarra a képsorrendre rendez, de a teljes tanítást nem ismétli meg. A `scene_split` jelenetenként ellenőrzi a közös splitet, a két nézetazonosítót (0 és 1) és a közös targetet. Az állomány SHA-256 lenyomatát a futásnaplóba kell menteni.

```
# lab/n160_bbox_height.py
import hashlib, json
from pathlib import Path
import numpy as np
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader

torch.manual_seed(20261020)
np.random.seed(20261020)
DEVICE = torch.device("cpu")
DTYPE = torch.float32
LAB = Path(__file__).resolve().parent
DATA = LAB / "data"

class BBoxHeightNet(nn.Module):
    """Kis CNN-gerinc regressziós fejjel. Bemenet NCHW 1x24x24, kimenet [N,1]."""
    def __init__(self) -> None:
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),                          # 24 -> 12
            nn.Conv2d(8, 16, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),                          # 12 -> 6
        )
        self.head = nn.Sequential(
            nn.Flatten(),                             # 16*6*6 = 576
            nn.Linear(576, 32), nn.ReLU(),
            nn.Linear(32, 1),
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))

class GlyphBBoxHeight(Dataset):
    """images.npy + bbox_height.npy párosítás a metadata.csv scene_id splitje után."""
    def __init__(self, images: np.ndarray, target: np.ndarray, idx: np.ndarray) -> None:
        self.x = torch.tensor(images[idx], dtype=DTYPE)
        self.t = torch.tensor(target[idx], dtype=DTYPE).unsqueeze(1)  # [N,1]
    def __len__(self) -> int:
        return len(self.x)
    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.x[i], self.t[i]

POSTPROCESS = "clip(round_half_to_even(t), 1, 24) v1"   # torch.round: x.5 -> páros egész
```

```
def align_metadata(rows: np.ndarray, n_images: int) -> np.ndarray:
    """A metadata sorait az `index` mező szerint az images.npy soraihoz igazítja."""
    index = rows["index"].astype(np.int64)
    assert len(index) == n_images, "a metadata sorszáma eltér a képtömbétől"
    assert np.array_equal(np.sort(index), np.arange(n_images)), "az index nem 0..N-1
    ↪  permutációja"
    return rows[np.argsort(index)]                # ezután az i. sor az i. képé

def scene_split(meta: np.ndarray, target: np.ndarray) -> tuple[np.ndarray, np.ndarray,
↪  np.ndarray]:
    """Scene-szintű indexfelosztás; egy jelenet minden nézete azonos splitbe kerül."""
    ids, splits = meta["scene_id"], meta["split"]
    assert set(np.unique(splits)) == {"train", "val", "test"}
    for sid in np.unique(ids):
        rows = ids == sid
        assert len(set(splits[rows])) == 1, "Egy jelenet több splitben szerepel"
        assert sorted(meta["view_id"][rows]) == [0, 1], "a jelenet nézetei hiányosak"
        assert len(set(target[rows])) == 1, "a két nézet célja eltér"
    return tuple(np.flatnonzero(splits == name) for name in ("train", "val", "test"))

def predict_final(model: nn.Module, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """Átadási függvény: nyers kimenet és az inference-szerződés szerinti végső predikció."""
    model.eval()
    with torch.inference_mode():
        raw = model(x).squeeze(1)
    return raw, torch.clamp(torch.round(raw), 1, 24)

def main() -> None:
    torch.set_num_threads(1)                       # CPU-n szálszámfüggetlen eredmény
    images = np.load(DATA / "images.npy")          # (360,1,24,24)
    rows = np.genfromtxt(DATA / "metadata.csv", delimiter=",", names=True,
                         dtype=None, encoding="utf-8")
    required = {"index", "scene_id", "view_id", "split", "ymin", "ymax"}
    assert required <= set(rows.dtype.names or ()), "a metadata.csv oszlopai hiányosak"
    meta = align_metadata(rows, len(images))
    shuffled = rows[np.random.default_rng(0).permutation(len(rows))]
    assert np.array_equal(align_metadata(shuffled, len(images)), meta)   # sorrendfüggetlen
    bbox_height = (meta["ymax"] - meta["ymin"]).astype(np.float32)
    target_path = DATA / "bbox_height.npy"
    np.save(target_path, bbox_height)
    print("bbox_height_sha256=" + hashlib.sha256(target_path.read_bytes()).hexdigest())
    tr_idx, va_idx, te_idx = scene_split(meta, bbox_height)
    assert (len(tr_idx), len(va_idx), len(te_idx)) == (240, 60, 60)

    assert images.shape == (360, 1, 24, 24) and bbox_height.shape == (360,)
    assert images.dtype == np.float32 and np.isfinite(images).all()
    assert float(images.min()) >= 0.0 and float(images.max()) <= 1.0
    assert np.all((bbox_height >= 1) & (bbox_height <= 24))
    model = BBoxHeightNet().to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=3e-3)   # Adam [P26]
    loss_fn = nn.MSELoss()
    tr_loader = DataLoader(GlyphBBoxHeight(images, bbox_height, tr_idx),
                           batch_size=16, shuffle=True)
    va_x = torch.tensor(images[va_idx], dtype=DTYPE)
    best_va, best_epoch, best_state = float("inf"), -1, None
    patience = 5
    for epoch in range(30):
        model.train()
        for xb, tb in tr_loader:
            opt.zero_grad()
            loss = loss_fn(model(xb), tb)
            loss.backward()
            opt.step()
```

```
        va_raw, _ = predict_final(model, va_x)
        va_mae = float(np.abs(va_raw.numpy() - bbox_height[va_idx]).mean())   # választó
↪  metrika: nyers val MAE
        if va_mae < best_va - 1e-6:
            best_va, best_epoch = va_mae, epoch
            best_state = {k: v.clone().cpu() for k, v in model.state_dict().items()}
            patience = 5
        else:
            patience -= 1
            if patience == 0:
                break
    ckpt = LAB / "checkpoints" / "n160_bbox_height_best.pt"
    ckpt.parent.mkdir(parents=True, exist_ok=True)
    assert best_state is not None
    torch.save(best_state, ckpt)
    ckpt_sha = hashlib.sha256(ckpt.read_bytes()).hexdigest()

    # átadás: a visszatöltött modell végső predikciói és a teljes inference-szerződés
    delivered = BBoxHeightNet().to(DEVICE)
    delivered.load_state_dict(torch.load(ckpt, map_location=DEVICE, weights_only=True))
    te_x = torch.tensor(images[te_idx], dtype=DTYPE)
    raw, final = predict_final(delivered, te_x)
    model.load_state_dict(best_state)
    assert torch.equal(predict_final(model, te_x)[1], final)   # memóriabeli == visszatöltött
    out = LAB / "runs" / "n160"
    out.mkdir(parents=True, exist_ok=True)
    np.savez(out / "test_predictions.npz", index=te_idx, raw=raw.numpy(), final=final.numpy(),
             target=bbox_height[te_idx])
    contract = {"input": "float32 NCHW (N,1,24,24), [0,1]", "raw_output": "float32 (N,)",
                "final_output": "egész 1..24", "postprocess": POSTPROCESS,
                "selection_metric": "nyers val MAE (legkisebb)", "best_epoch": best_epoch,
                "checkpoint": str(ckpt), "checkpoint_sha256": ckpt_sha,
                "claim_refers_to": "final_test_mae (az átadott predikció)"}
    (out / "inference_contract.json").write_text(json.dumps(contract, indent=2,
↪  ensure_ascii=False))

    saved = np.load(out / "test_predictions.npz")   # a riport az artifactból számol
    raw_test_mae = float(np.abs(saved["raw"] - saved["target"]).mean())
    final_test_mae = float(np.abs(saved["final"] - saved["target"]).mean())
    train_median = float(np.median(bbox_height[tr_idx]))
    baseline_test_mae = float(np.abs(bbox_height[te_idx] - train_median).mean())
    test_scene = meta["scene_id"][te_idx]
    deltas = []
    for sid in np.unique(test_scene):
        vals = saved["final"][test_scene == sid]
        assert len(vals) == 2, "a teszten minden jelenetnek két nézete kell legyen"
        deltas.append(abs(float(vals[0]) - float(vals[1])))
    delta_scene_median = float(np.median(deltas))
    print(f"best_epoch={best_epoch} best_val_mae_raw={best_va:.4f}")
    print(f"raw_test_mae={raw_test_mae:.4f} final_test_mae={final_test_mae:.4f} "
          f"baseline_test_mae={baseline_test_mae:.4f}
↪  delta_scene_median={delta_scene_median:.4f}")

if __name__ == "__main__":
    main()
```

Futtatási parancs a `lab/` könyvtárból: `uv run python n160_bbox_height.py`. A checkpointot a **nyers** validációs MAE választja (ezt a tanítás előtt rögzítjük), az alkalmazási állítás viszont az átadott, utófeldolgozott predikcióra, azaz a **végső** teszt MAE-re vonatkozik; a script a kettőt külön közli. A tanítás legfeljebb 30 epochig tart, és 5 epoch türelmi idővel leáll, ha a validációs MAE nem javul. A visszatöltött checkpointból az átadási függvény (`predict_final`) állítja elő a teszt nyers és végső predikcióit; a script ellenőrzi, hogy ez bitre egyezik a memóriában

lévő legjobb állapot végső predikciójával, a predikciókat a `runs/n160/test_predictions.npz`, a szerződést a `runs/n160/inference_contract.json` fájlba menti, és a teszt MAE-ket már a mentett artifactból számolja. A következő kimenet egy mért futás (Python 3.12.3, NumPy 2.5.4, PyTorch 2.15.0+cpu, CPU, 1 szál, seed 20261020); más PyTorch-verzión vagy hardveren a tanítási sorok eltérhetnek:

```
bbox_height_sha256=cd9b71e47e517fb9acf96a0838f4f3a7503e12a34423147972973932a9ae7066
best_epoch=15 best_val_mae_raw=0.6465
raw_test_mae=0.7137 final_test_mae=0.6667 baseline_test_mae=2.6000 delta_scene_median=0.0000
```

A determinisztikusan előállított target 360 képén a dobozmagasság 1 és 9 közé esik. Ez egyetlen seed eredménye; a seed-szórás külön mérés (36.2. Ismétlések és eredménybizonytalanság). A checkpoint SHA-256 lenyomata a szerződésfájlban szerepel. A modell paraméterszáma: első konvolúció $1 \cdot 8 \cdot 3 \cdot 3 + 8 = 80$; második $8 \cdot 16 \cdot 3 \cdot 3 + 16 = 1168$; lineáris $576 \cdot 32 + 32 = 18\,464$; kimeneti $32 \cdot 1 + 1 = 33$; összesen 19 745. A kimeneti térkép ellenőrzése: 24→24 (Conv, $p = 1$), 24→12 (Pool 2), 12→12 (Conv, $p = 1$), 12→6 (Pool 2), csatorna 16, flatten $16 \cdot 6 \cdot 6 = 576$, ami illeszkedik a `Linear(576,32)` bemenetéhez.

40.4.6 Értékelés, inference-szerződés és a jelentés

Az **értékelési metrika** képenkénti átlagos abszolút hiba (mean absolute error, MAE) pixelben, valamint a jeleneten belüli két nézet predikciójának különbsége:

$$\mathrm{MAE}_{\text{per-image}} = \frac{1}{60} \sum_{i \in \text{test}} |\hat{t}_{\text{final},i} - t_i|, \qquad \Delta_{\text{scene}} = \left|\hat{t}_{\text{view 1}} - \hat{t}_{\text{view 2}}\right|.$$

A triviális baseline a train-célok mediánja m_{train}, mert az állandó becslések közül a medián minimalizálja az átlagos abszolút eltérést. A hozzá tartozó teszt MAE az abszolút predikciós hiba mértéke, nem a szórás. MSE-re optimalizált állandó baseline esetén a train-célok átlaga a megfelelő választás. A két nézet ugyanahhoz a jelenethez tartozik, ezért a képenkénti bootstrap **nem** használható szignifikancia-állításra: a resampling egysége a `scene_id` 5.4. Függő képek, seedek és resampling. Ha a riport konfidenciaintervallumot közöl, azt jelenetszintű bootstrapból kell számítani (Efron–Tibshirani, 1993), és nem szabad független képekből számolt intervallumként értelmezni. Ebben a leckében a CI nem kötelező elem; a kötelező bizonytalansági kimutatás a több seedből származó szórás.

Az **inference-szerződés**: bemenet float32 NCHW [0; 1] kép; nyers kimenet $\hat{t} \in \mathbb{R}$, végső kimenet $\hat{t}_{\text{final}} \in \{1; \ldots; 24\}$ a `clip(round_half_to_even(t), 1, 24) v1` utófeldolgozással. A predikciós artifact a teszt indexét, a nyers és a végső predikciót és a targetet, a szerződésfájl a sémát, az utófeldolgozás verzióját, a választó metrikát, a best epochot és a checkpoint hash-ét tartalmazza. A fenti MAE-képlet a végső predikcióra vonatkozik; a nyers MAE diagnosztikai érték, mert a kerekítés és a levágás mindkét irányba változtathatja a hibát. A tanítási veszteség MSE, de a best checkpointot kizárólag a nyers val MAE alapján választjuk; a tesztet csak a fagyasztott súlyok betöltése után, egyszer értékeljük.

A **report** a 36.4. IX. ellenőrző pont: lektorálható modellállítás hat komponensét követi: a bizonyítást (1. lemma és a kimeneti méret tétel alkalmazása), a source trace-t (images.npy, bbox_height.npy, metadata.csv, szkript és checkpoint hash-e), a kódot, a run manifestet (seed 20261020, float32, CPU, epochszám, nyers validációs MAE, nyers és végső teszt MAE, utófeldolgozás-verzió), a páros clean–corrupt vagy view–view eredményt és a korlátokat. A korlátszakasz kötelező eleme: a GlyphLab szintetikus adat; a MAE elméleti alsó határa nulla, mert a végső, egész értékű predikció és az egész cél egybeeshet; pozitív elkerülhetetlen hiba csak

a feladat és a megfigyelések bizonytalanságának külön indoklásával állítható, és a regresszió nem kalibrált valószínűségi predikció.

40.4.7 Szóbeli védési kérdések és gyakori tévedés

Rövid szóbeli védési kérdések:

1. Miért nem használható a képenkénti véletlen split ebben a feladatban?
2. Miért nem elég a val MAE önmagában annak eldöntésére, hogy a modell a geometriát tanulta-e, nem pedig a jelenet átlagos fényességét?
3. Mivel a `bbox_height.npy` determinisztikusan, a metaadatokból számolható, miért fontosabb az inputfájlok hash-e és a transzformáció kódjának verziója, mint egy nem használt generálási seed?
4. A `round` a váltási pontokon nem differenciálható, közöttük konstans, tehát a deriváltja majdnem mindenütt nulla és a határokon nem definiált. Miért okoz ez taníthatósági gondot (nem az, hogy sehol sem létezne derivált), és miért helyes a `round`-ot csak az inference-szerződésbe tenni?
5. Miért nem hivatalos egyetemi osztályzat a belső 0–4 rubrika?

Rövid válaszok. 1. A képenkénti split szivárgást okoz: ugyanazon jelenet két nézete különböző splitbe kerülhet, ezért a teszt nem új jelenetekre mér. 2. A val MAE önmagában nem zárja ki, hogy a modell a jelenet fényességéből vagy más spurius jelből becsüljön; ehhez kontroll kell (például fényességre érzéketlen baseline, korrupt teszt, jelenetszintű összehasonlítás). 3. A `bbox_height.npy` a metadata determinisztikus függvénye, ezért a target előállításának kódja és a bemeneti fájlok hash-e azonosítja a tényleges adatállapotot; a generálási seed önmagában nem bizonyítja, hogy a felhasznált fájlok ugyanazok. 4. A `round` majdnem mindenütt nulla deriváltú és a váltási pontokon nem differenciálható, ezért a tanításban nem lehet a gradiens útjában; a kerekítés csak az inference-szerződés utófeldolgozása. 5. A rubrika a lektori visszacsatolás belső eszköze, nem akkreditált osztályzat, és nem helyettesít külső vizsgát vagy szakmai minősítést.

Gyakori tévedés és ellenpélda. A leggyakoribb hiba a belső értékelési skála akkreditált jeggyé alakítása. Ellenpélda: ha egy projekt mind az öt dimenzióban 4 pontot kap, az azt jelenti, hogy az adott, a feladatkiírásban rögzített feltételek között önállóan helyes és ellenőrizhető; ez **nem** bizonyítja, hogy a modell új adaton vagy éles környezetben is 4-es teljesítményt ad. A rubrika a lektori visszacsatolás belső eszköze, nem helyettesíti a 36.3. Szakirodalmi reprodukció és attribúció alfejezetben megkövetelt forrásauditot. Hasonló kategóriahiba: a teszt MAE-t két tizedesjegy pontossággal közölni egyetlen seedből, és ezt a modell várható hibájának nevezni; a seed-szórás külön mérés 36.2. Ismétlések és eredménybizonytalanság.

40.4.8 Feladatok (F1–F5)

F1 (fogalmi, nehézség 1). Nevezze meg az orientáció-osztályozó és a függőleges kiterjedés regressziós szerződésének négy különböző komponensét (target, loss, output, metrika), és adjon mindegyikre egy-egy pontos értéktartományt. Az orientáció-osztályozó célja $y \in \{0;\ 1;\ 2\}$ int64, ahol 0 = függőleges, 1 = vízszintes, 2 = átlós; a regressziós cél $t \in \{1;\ \ldots;\ 24\}$ valós érték. A modell paraméterei $\theta = (\phi, W, b)$: ϕ a gerinc, W, b a kimeneti réteg; a batch mérete N.

F2 (levezetési, nehézség 2). Alakítsa alkalmazott tétellé a 14.1. Stride, padding, dilation és outputméret kimeneti méretképletét a saját architektúrájára, és adjon ellenpéldát olyan (H, k, d, p, s) ötösre, amelyre a képlet nem pozitív egész kimenetet ad; indokolja, miért érvénytelen ez a `Conv2d` hívásra.

F3 (implementációs, nehézség 2). Egészítse ki a labor kódját egy rövid öntesztteel, amely (a) $2\times1\times24\times24$ véletlen bemenetre ellenőrzi, hogy a kimenet alakja 2×1, (b) a végső `Linear(32,1)` réteg súlyára vonatkozó analitikus gradienst összehasonlítja az autograd gradienssel float64 pontossággal, 10^{-6} abszolút toleranciával.

F4 (hibakeresési, nehézség 1). A következő hibás eljárásban azonosítsa a hibát, következményét és javítását: „A 360 képből véletlenszerűen választunk 240/60/60 képet train/val/test részre, és a modellt MSE-vel tanítjuk."

F5 (kísérlettervezési, nehézség 3). Tervezzen kísérletet a kérdésre: „Javítja-e a harmadik konvolúciós réteg hozzáadása a teszt MAE-t?" Adja meg a protokollt, a baseline-t, a változó tényezőt, a scenestratified splitet, a metrikát, a seed- és erőforrás-kezelést, valamint a következtetés érvényességi határát.

40.4.9 Megoldások és indoklás

F1. Osztályozás: target $y \in \{0;\ 1;\ 2\}$ int64, loss CrossEntropyLoss logitsból, output logit $\mathbb{R}^3$, metrika accuracy. Regresszió: target $t \in \{1;\ \dots;\ 24\} \subset \mathbb{R}$, loss MSE, output $\hat{t} \in \mathbb{R}$, metrika MAE pixelben. A `round/clip` csak inference-utófeldolgozás, nem a tanítási cél része.

F2. A modell két konvolúciójára $(H, k, d, p, s) = (24;\ 3;\ 1;\ 1;\ 1)$, így $H_{out} = \lfloor(24+2-2-1)/1\rfloor + 1 = \lfloor 23\rfloor + 1 = 24$; a poolra a `MaxPool2d(2)` alapértelmezései érvényesek: $k = 2$, $s = k = 2$, $d = 1$, $p = 0$ és `ceil_mode=False`, azaz lefelé kerekítés. Ekkor a számláló $H + 2p - d(k-1) - 1 = H - 2$, tehát $H_{out} = \lfloor(24-2)/2\rfloor + 1 = 12$, másodszor $\lfloor(12-2)/2\rfloor + 1 = 6$. Páratlan bemeneten látszik, miért nem $H-1$ a számláló: $H = 5$, $k = s = 2$, `ceil_mode=False` mellett $H_{out} = \lfloor(5-2)/2\rfloor + 1 = 2$, a hibás $\lfloor(5-1)/2\rfloor + 1$ viszont 3-at adna; páros H-ra a két alak a lefelé kerekítés miatt egybeesik, ezért ott a hiba rejtve marad. `ceil_mode=True` mellett $\lceil(5-2)/2\rceil + 1 = 3$ (PyTorch 2.15.0+cpu-ban ellenőrizve). Ellenpélda: $(H, k, d, p, s) = (4;\ 3;\ 2;\ 0;\ 1)$ esetén a képlet $H_{out} = \lfloor(4-4-1)/1\rfloor + 1 = \lfloor -1\rfloor + 1 = 0$, tehát a kimeneti méret nem pozitív. A `Conv2d` ilyen konfigurációra hibát ad, mert az effektív kernelméret $d(k-1)+1 = 5$ meghaladja a bemenet méretét; a szükséges pozitív kimeneti méret feltétele sérül. A törtnek nem kell egésznek lennie: például $H = 6$, $k = 3$, $d = 1$, $p = 0$, $s = 2$ mellett $(6-2-1)/2 = 1{,}5$, és az érvényes kimenet $\lfloor 1{,}5\rfloor + 1 = 2$.

F3. A megoldás vázlata:

```
def self_test() -> None:
    model = BBoxHeightNet().double()          # float64 kontroll
    x = torch.randn(2, 1, 24, 24, dtype=torch.float64, requires_grad=True)
    y = model(x)
    assert y.shape == (2, 1), y.shape
    t = torch.tensor([[3.0], [7.0]], dtype=torch.float64)
    loss = torch.nn.functional.mse_loss(y, t)         # mean reduction
    a = model.head[2](model.head[1](model.head[0](model.features(x)))).detach()  # (2,32)
    g = (2.0 / x.shape[0]) * (y.detach() - t)  # (2,1)
    grad_W_analytical = g.T @ a              # (1,32)
    loss.backward()
    grad_W_autograd = model.head[3].weight.grad  # (1,32)
    assert grad_W_analytical.shape == grad_W_autograd.shape == (1, 32)
    torch.testing.assert_close(grad_W_analytical, grad_W_autograd, atol=1e-6, rtol=0)
```

Itt a `model.head[1]` a `Linear(576,32)` réteg, a `model.head[2]` a ReLU, ezért az a aktiváció alakja $(N, 32)$. Az upstream g alakja $(N, 1)$, így $g^\top a$ alakja $(1;\ 32)$, azonos a `model.head[3].weight` súlyéval. Az explicit alakkontroll megelőzi a véletlen broadcastingből eredő téves sikeres tesztet. A batch-átlag $2/N$ szorzója a lemma szerint.

F4. A hiba a megfigyelési egység figyelmen kívül hagyása. Következmény: egy jelenet két nézete különböző splitbe kerülhet; ugyanaz a geometriai tartalom a trainben és a tesztben is megjelenik, a teszt MAE optimistán torzított lesz. Javítás: a splitet a `scene_id`-re kell csoportosítani, azaz az 1.3. Adatfelosztás és a megfigyelési egység alfejezetben rögzített scene-stratified eljárást kell használni; visszaellenőrzés: minden `scene_id` pontosan egy splitben szerepeljen.

F5. Lehetséges terv. Kérdés: harmadik konvolúciós réteg hatása. A réteg hozzáadása a paraméterszámot szükségszerűen növeli; ez a beavatkozás része, nem zavaró tényező (36.1. Állításból kísérleti terv), a kapacitásillesztett kontroll (azonos paraméterkeret más elrendezésben) külön, opcionális kérdés. Protokoll: azonos vetőmag, azonos CPU, azonos float32 dtype; a baseline a jelenlegi kétkonvolúciós modell; a változat egy harmadik, $16 \to 16$ csatornás, $k = 3$, $p = 1$ réteg a második pool **előtt**. Megfigyelési egység: jelenetpár. Split: a befagyasztott 120/30/30 scene felosztás marad; az ismétlés öt inicializációs seeddel, ugyanazon a splitten (az új split vagy a keresztvalidáció más bizonytalanságot mérne, és ütközne a korábban befagyasztott teszttel). Metrika: a két modell jelenetszintű, páros MAE-különbsége ugyanazon tesztjeleneteken, valamint a Δ_{scene} medián; az architektúra-összehasonlításra a páros különbség a mérvadó, a Δ_{scene} csak a nézet-konzisztencia diagnosztikája. A konfiguráció és a best checkpoint kiválasztása kizárólag validációs MAE szerint történik, és csak ezután következik az egyszeri, páros tesztkiértékelés: jelenetenként egy különbség, 95%-os jelenetszintű bootstrap CI, döntési szabály „a CI nem tartalmaz nullát, és a medián abszolút különbség legalább 0,1 pixel". Erőforráskeret: öt seed × két modell × legfeljebb 30 epoch, CPU. Következtetési határ: a GlyphLab szintetikus adat; az eredmény nem általánosítható természetes képekre, és a seed-szórás miatt egyetlen futtatásból nem vonunk le általános architektúra-következtetést. A nyers MAE-ket, a páros különbséget, a hatásnagyságot és a bizonytalanságot külön kell jelenteni. A tanítási recept a 38.2 AdamW-receptje ($\eta_0 = 10^{-3}$, $\eta_{\min} = 10^{-5}$, weight decay 10^{-4}, batch 32), a seedek 20261020–20261024, az erőforrás CPU.

40.4.10 Rövid összefoglaló

A X. ellenőrző pont egy új, regressziós feladaton bizonyítja az önálló munkavégzést: a GlyphLab-képből a maszk befoglaló magasságát becsüljük. A szerződés hét komponense – probléma, matematikai modell, implementáció, adat, értékelés, inference, jelentés – együtt adja a lektorálható csomagot. A levezetési követelmény az MSE-gradiens végső lineáris rétegre vonatkozó alakhelyes VJP-je és a konvolúciós kimeneti képlet egy érvénytelen konfigurációval vett ellenpéldája. A labor kis CPU-s PyTorch-szkriptet ad; a determinisztikusan számított cél- és baseline-értékeket a lecke közli, a tanítási mérőszámokat a tanuló saját futása állítja elő. A belső 0–4 rubrika lektori eszköz, nem akkreditált osztályzat; a részletes rubrika és a PASS-szabály a 1.4. Tanulási diagnózis és kísérleti napló alfejezetben van.

40.4.11 Következő lecke

Az elvégzett ellenőrző pont lezárja a tananyag főútját: a hallgató egy új célon végigviszi a teljes modellezési ciklust. A hét komponens sablonja független a GlyphLabtól, amennyiben a target, a split és a metrika szerződését újradefiniálja. A könyv záró függelékei – a GlyphLab-adatleírás, a jelölésjegyzék, a szójegyzék és a bibliográfia – a 40.4.13. szakaszban találhatók, és a korábbi leckék hivatkozásai is ezekre mutatnak.

40.4.12 Források

- [B01] Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. https://mml-book.github.io/.
- [B02] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.
- [B03] Magnus, J. R., Neudecker, H. (2019). *Matrix Differential Calculus with Applications in Statistics and Econometrics* (3rd ed.). Wiley.
- [S01] Stanford CS231n (n.d.). *Convolutional Neural Networks*. https://cs231n.github.io/convolutional-networks/. Hozzáférés: 2026-10-30.
- [S04] IEEE (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*. IEEE. A `round half to even` konvenciót a szabvány rögzíti.
- [P26] Kingma, D. P., Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR. https://arxiv.org/abs/1412.6980.

40.4.13 Függelékek

40.4.13.1 A. A GlyphLab adatkészlet

A GlyphLab a könyv saját, helyben generált oktatási adatkészlete; külső képet, személyes adatot vagy letöltött adathalmazt nem tartalmaz. A forrása a `lab/generate_data.py` generátor (verzió 1.0, alapértelmezett seed `20261020`), a részletes adatszerződés pedig a laborcsomag `ADATOK.md` fájlja. A generátor 180 jelenetből két zajos nézetet állít elő, így 360 képet ad. A feladat a vonalorientáció felismerése; ez szintetikus, szűk feladat, nem természetes képi benchmark.

Fájl	Alak, típus	Jelentés
`data/images.npy`	`(360, 1, 24, 24)`, float32	NCHW képek, értékkészlet [0; 1]
`data/masks.npy`	`(360, 1, 24, 24)`, uint8	Az előtér 0/1 maszkja
`data/labels.npy`	`(360,)`, int64	Az orientáció osztályindexe
`data/corrupted_test.npy`	`(60, 1, 24, 24)`, float32	Csak a teszt képei zajjal és fényességgradienssel
`data/metadata.csv`	360 rekord	A képek metaadatai
`data/manifest.json`	JSON	Verzió, seed, alakok, splitdarabszámok, SHA-256

Az osztályok: 0 = függőleges (`vertical`), 1 = vízszintes (`horizontal`), 2 = átlós (`diagonal`). A `metadata.csv` oszlopai:

Oszlop	Típus	Jelentés
`index`	int64	A teljes képtömb sorindexe; nem a CSV sorszáma
`scene_id`	string	A független mintavételi egység azonosítója, `G<label>-<scene>` alakú (például `G0-0000`)
`view_id`	int64	A jelenet nézetének azonosítója (0 vagy 1)

Oszlop	Típus	Jelentés
`label`	int64	Az orientáció osztályindexe (0, 1, 2)
`split`	string	`train`, `val` vagy `test`
`xmin`, `ymin`, `xmax`, `ymax`	int64	A maszk befoglaló doboza, félnyílt xyxy

A doboz nullától indexelt, félnyílt intervallum: $x_{\min} \le x < x_{\max}$, $y_{\min} \le y < y_{\max}$. A doboz a maszk nemnulla pixeljeinek legkisebb befoglaló téglalapja. Ha egy maszk mégis üres, a doboz mind a négy koordinátája -1: $(-1;\ -1;\ -1;\ -1)$ (szentinel, a Feladat- és annotációs döntés 39.1. alfejezet definíciója szerint); a jelenlegi generátor minden maszkban garantál legalább egy előtér-pixelt, ezért a szállított adatban üres maszk nem fordul elő. A `corrupted_test.npy` sorrendje a `metadata.csv` test sorainak sorrendjét követi, és ugyanazokat a jelenet–nézet párokat tartalmazza, ezért a clean–corrupt összehasonlítás páros.

Split	Független jelenet	Kép	Jelenet/osztály
train	120	240	40
val	30	60	10
test	30	60	10

A split szabálya: egy `scene_id` minden nézete ugyanabba a splitbe kerül; a felosztás osztályonként rétegzett. A képek száma ezért 240/60/60. A `scene_id` a split és a csoportos bizonytalanság-becslés egysége; a kép csak a predikció és a képenkénti metrika egysége. A maszk, a doboz és a `scene_id` nem lehet a modell bemenete. A tesztet legfeljebb egyszer, a befagyasztott modellválasztás után értékeljük; a korrupt teszt nem használható hiperparaméter-választásra.

Újragenerálás a `lab/` könyvtárból: `uv run python generate_data.py --output data`, majd `uv run python quickcheck.py`. Azonos NumPy-verzió, seed és generátor-beállítás mellett a generátor ugyanazokat a fájlokat adja; más verzió, platform vagy numerikus backend esetén a bitazonosság nem garantált. A `quickcheck.py` a hash-t, az alakot, a tartományt, a splitet és a doboz–maszk viszonyt ellenőrzi.

40.4.13.2 B. Jelölésjegyzék

A könyv a következő alapjelöléseket használja. A helyi szakaszokban ettől csak kifejezett, ott megadott megfeleltetéssel lehet eltérni.

Objektum	Jelölés	Jelentés
Bemeneti vektor	$x \in \mathbb{R}^d$	Oszlopvektor az absztrakt képletekben
Batchelt vektoradat	$X \in \mathbb{R}^{N \times d}$	Egy megfigyelés egy sor
Affín réteg súlya	$W \in \mathbb{R}^{m \times d}$	PyTorch `Linear` súlyalak
Affín réteg	$Z = XW^\top + b$	Z alakja $N \times m$
Skalár loss gradiense	$\nabla_x L$	Alakja megegyezik x alakjával
Jacobi-mátrix	$J_f \in \mathbb{R}^{m \times d}$	Sor: output, oszlop: input
VJP	$J_f^\top g$	$g = \nabla_{f(x)} L$
Képbatch	$X \in \mathbb{R}^{N \times C_{\text{in}} \times H \times W}$	NCHW
Konvolúciós súly	$K \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}}/g) \times k_h \times k_w}$	g a `groups` paraméter

Objektum	Jelölés	Jelentés
Osztályozási logit	$Z \in \mathbb{R}^{N \times K}$	Aktiváció előtti pontszám
Többosztályos target	$y \in \{0;\ \ldots;\ K-1\}^N$	int64 osztályindex
Konvolúciós kimenet	$H_{\text{out}} = \lfloor (H + 2p - d(k-1) - 1)/s \rfloor + 1$	Szimmetrikus zero padding, egy dimenzió

Jelölésütközések: a p jelentése lehet padding, valószínűség vagy dropout-arány; a d lehet dilatáció, dimenzió vagy döntési szabály; a K lehet kernelméret vagy osztályszám; az L lehet veszteség, rétegszám vagy hossz; a W lehet szélesség vagy súly; az N lehet batchméret vagy mintaszám; a c lehet osztály vagy csatorna; a σ lehet szórás, szigmoid vagy GELU. Ezeket minden lecke a saját szövegében oldja fel, és a szükséges helyen más betűre nevez.

A magyar folyószövegben tizedesvessző áll (0,5), a matematikai listák és halmazok elemeit pontosvessző választja el ($\{0;\ 0{,}3;\ 0{,}6\}$), a kódban és a kód kimenetében tizedespont szerepel. Az ezres tagolás szóköz.

40.4.13.3 C. Magyar–angol szójegyzék

Magyar	Angol	Megjegyzés
alapvonal	baseline	Összehasonlítási viszonyítási modell
szerződés	contract	A könyvben specifikáció (adat-, modell-, inference-szerződés); nem jogi értelem
köteg	batch	A tanítás egy lépésének mintahalmaza
mintavételi egység	sampling unit	A GlyphLabban `scene_id`
felosztás	split	train/val/test
vetőmag	seed	A véletlenszám-generátor állapota
ellenőrzőpont	checkpoint	Mentett modellállapot
logit	logit	Aktiváció előtti pontszám
stride / lépésköz	stride	A konvolúció lépése
padding / kitöltés	padding	A szegély kitöltése
dilation / tágítás	dilation	A kernel mintavételi távolsága
MAC	multiply-accumulate	Szorzás-összeadás művelet
VJP	vector-Jacobian product	Vektor–Jacobi-szorzat
inference / következtetés	inference	Betanított modell kiértékelése
bootstrap	bootstrap	Újramintavételes bizonytalanságbecslés
adatszivárgás	leakage	A teszt információjának bekerülése a tanításba
összeomlás	collapse	Reprezentáció vagy tanulás elsekélyesedése
downstream feladat	downstream task	A előtanított reprezentációt használó célfeladat
pretext feladat	pretext task	Önfelügyelt előtanítási feladat

Magyar	Angol	Megjegyzés
kevert pontosság	mixed precision	float16/bfloat16 és float32 együttes használata
tenzor	tensor	Többdimenziós tömb
affin	affine	Lineáris + eltolás
puffer	buffer	Nem tanított, mentett állapot (pl. BN statisztika)
veszteség	loss	Optimalizált célfüggvény
pontosság	accuracy	Helyes döntések aránya
precizitás	precision	TP/(TP+FP)
visszahívás	recall	TP/(TP+FN)
beadandó	deliverable	A feladat záróeredménye (riport, táblázat vagy kód)
főút	main path	A könyv kötelező, kis helyi CPU-s, eredeti adatos végrehajtási útvonala

40.4.13.4 D. Bibliográfia

A leckék saját forráslistái a helyi hivatkozásokhoz mérvadók; az alábbi gyűjtés a könyvben visszatérő kanonikus hivatkozásokat egységesíti. A források a megadott kiadásra/verzióra vonatkoznak; a konkrét tétel állítását a hivatkozott szakaszban kell ellenőrizni.

- Deisenroth, M. P., Faisal, A. A., Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.
- Glorot, X., Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- He, K., Zhang, X., Ren, S., Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *ICCV*.
- Ioffe, S., Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*.
- Santurkar, S., Tsipras, D., Ilyas, A., Madry, A. (2018). How does batch normalization help optimization? *NeurIPS*.
- Long, J., Shelhamer, E., Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *CVPR*.
- Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861.
- Guo, C., Pleiss, G., Sun, Y., Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML*.
- Goodfellow, I. J., Shlens, J., Szegedy, C. (2015). Explaining and harnessing adversarial examples. *ICLR*.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. *ICLR*.
- Vincent, P., Larochelle, H., Bengio, Y., Manzagol, P.-A. (2008). Extracting and composing robust features with denoising autoencoders. *ICML*.
- Rifai, S., Vincent, P., Muller, X., Glorot, X., Bengio, Y. (2011). Contractive auto-encoders: Explicit invariance during feature extraction. *ICML*.
- van den Oord, A., Li, Y., Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv:1807.03748.

- Grill, J.-B., et al. (2020). Bootstrap your own latent: A new approach to self-supervised learning. *NeurIPS*.
- Chen, X., He, K. (2021). Exploring simple Siamese representation learning. *CVPR*.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S. (2017). GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *NeurIPS*.
- Higgins, I., et al. (2017). beta-VAE: Learning basic visual concepts with a constrained variational framework. *ICLR*.
- Hadsell, R., Chopra, S., LeCun, Y. (2006). Dimensionality reduction by learning an invariant mapping. *CVPR*.
- Bromley, J., et al. (1993). Signature verification using a “Siamese” time delay neural network. *NeurIPS*.
- Hermans, A., Beyer, L., Leibe, B. (2017). In defense of the triplet loss for person re-identification. arXiv:1703.07737.
- Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CVPR*.
- Han, S., Pool, J., Tran, J., Dally, W. J. (2015). Learning both weights and connections for efficient neural networks. *NeurIPS*.
- Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H. P. (2017). Pruning filters for efficient ConvNets. *ICLR*.
- Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *ICCV*.
- Hochreiter, S., Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581–592.
- Baldi, P., Hornik, K. (1989). Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1), 53–58.
- Eckart, C., Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.
- Everingham, M., et al. (2010). The PASCAL Visual Object Classes (VOC) challenge. *IJCV*, 88(2), 303–338.
- Liu, W., et al. (2016). SSD: Single shot multibox detector. *ECCV*.
- Malkov, Y. A., Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4), 824–836.
- Jégou, H., Douze, M., Schmid, C. (2011). Product quantization for nearest neighbor search. *IEEE TPAMI*, 33(1), 117–128.
- Efron, B., Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Kingma, D. P., Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.

40.4.13.5 E. Értékelési rubrika és PASS-szabály

A könyv közös, 0–4 skálájú rubrikája az 1.4. Tanulási diagnózis és kísérleti napló „Tanulási főút és ellenőrző pontok” szakaszában van definiálva; ez a függelék csak összefoglalja. Az öt dimenzió: D1 matematikai helyesség, D2 kód és futtathatóság, D3 adat- és mérési protokoll, D4 bizonyíték és nyomon követhetőség, D5 az állítás és korlátai. A szintek: 0 hiányzik; 1 hibás; 2 lényegében helyes, de hiányosan ellenőrzött; 3 helyes és ellenőrzött; 4 helyes, ellenőrzött és új helyzetre is indokoltan alkalmazott.

A PASS-szabály: (a) az ellenőrző pont laborjának minden `assert`-je és toleranciaellenőrzése a saját futtatásban átmegy; (b) mind az öt dimenzió legalább 3; (c) a riport csak ténylegesen lefuttatott kód kimenetét közli. A riport a hat komponensű modellállítást tartalmazza (bizonyítás,

forrásnyom, kód, futási jegyzék, páros eredmény, korlátok), a futási jegyzék pedig a seedet, a csomag- és eszközverziót, valamint a bemeneti fájlok SHA-256 lenyomatát. A rubrika belső lektori eszköz, nem akkreditált osztályzat.

40.4.13.6 F. A teljes könyv záró összefoglalója

A könyv tíz, egyenként négy fejezetből álló blokkon vezeti végig az olvasót: a matematikai és programozói előkészítéstől a tenzorokon, a valószínűségen és az optimalizáláson át a perceptronig, a backpropagationig és a PyTorch-tanításig; majd a kép diszkrét jele, a konvolúció és gradiense, az equivariance, a klasszikus és modern CNN-architektúrák, az inicializálás, a regularizálás, az adat- és értékelési protokoll, az osztályozás, a detektálás, a szegmentálás, az embedding, az autoencoder, az önfelügyelt tanulás, a generatív modellek, a szekvenciák, az inference és a hatékonyság, végül a magyarázhatóság, a kísérlettervezés, a saját operátor és a három projekt következik.

A tananyag végére a hallgatónak önállóan kell tudnia: egy alkalmazási ötletből mérhető feladatot és adatszerződést készíteni; a matematikai modellt, a veszteséget és a metrikát összehangolni; a modellt Pythonban implementálni és a gradiensét ellenőrizni; a splitet a megfigyelési egység szerint kialakítani; a kísérletet seeddel, verzióval és hash-sel dokumentálni; a tesztet egyszer, a választás után értékelni; és az eredményt a korlátaival együtt jelenteni. A könyv nem ígér minden laborhoz előre lefuttatott, gépektől függetlenül bitazonos kimenetet: a közölt számokat a tanulónak a saját, dokumentált környezetében kell előállítania és összevetnie. A GlyphLab szintetikus adat, a toy kísérletek pedig nem helyettesítenek valós alkalmazást vagy teljes cikkreprodukciót.

A záró projekt beadása akkor fogadható el, ha a hét komponens (probléma, matematikai modell, implementáció, adat, értékelés, inference, jelentés) hiánytalan, a futási jegyzék a PASS-szabályt teljesíti, és a jelentés kimondja a mérés érvényességi korlátait. A rubrika és a PASS-szabály nem akkreditált osztályzat; a szóbeli védés a beadás szakmai megvédését szolgálja.